

My Project

Generated by Doxygen 1.9.8

Chapter 1

Cellular Automata Viewer

This folder contains the first standalone cellular-automata tool for the Genesys Qt GUI.

Current scope:

- 2D demo built from the existing cellular-automata plugin classes
- UI selectors for rule, neighborhood, and state preset
- Forest Fire preset with four enumerated states
- mouse editing with a selectable paint state
- discrete stepping plus timer-driven auto-run
- JSON save/load for lattice size, time, and cell states

Known limitations in this first pass:

- the viewer is intentionally demo-focused, not a universal CA builder yet
- more advanced neighborhood/radius combinations remain plugin-side follow-up work

Chapter 2

Genesys MCP Connector

Python MCP server that exposes GenESyS simulation capabilities to AI assistants (Claude Desktop, Claude Code, and any MCP-compatible client).

2.1 Prerequisites

- Python 3.10+
- `genesys-worker` running (see `source/applications/worker/README.md`)

2.2 Install

```
cd source/applications/mcp
pip install -e .
```

2.3 Run

```
# via entry point
genesys-mcp

# or via module
python -m genesys_mcp
```

The server communicates over stdio (default FastMCP transport). Set `GENESYS_URL` to point to a non-default worker app address:

```
GENESYS_URL=http://192.168.1.10:8080 genesys-mcp
```

2.4 Claude Desktop integration

Add to `~/ .config/claude/claude_desktop_config.json` (Linux) or `~/Library/Application Support/Claude/claude_desktop_config.json` (macOS):

```
{
  "mcpServers": {
    "genesys": {
      "command": "genesys-mcp",
      "env": {
        "GENESYS_URL": "http://localhost:8080"
      }
    }
  }
}
```

```
}
}
```

If `genesys-mcp` is not on `PATH` (e.g. installed in a `venv`), use the full path:

```
{
  "mcpServers": {
    "genesys": {
      "command": "/path/to/venv/bin/genesys-mcp",
      "env": {
        "GENESYS_URL": "http://localhost:8080"
      }
    }
  }
}
```

2.5 Claude Code integration

```
claude mcp add genesys -- genesys-mcp
```

Or with a custom URL:

```
claude mcp add genesys -e GENESYS_URL=http://localhost:8080 -- genesys-mcp
```

2.6 Workflow

All tools that interact with the simulator require a session token. Call `genesys_auth` once per conversation and pass the returned token to every subsequent tool.

```
1. genesys_auth          → { token: "..." }
2. genesys_new_model     (token)
3. genesys_import_model_language (token, model_text)
4. genesys_configure_simulation (token, replications=10, length=480.0)
5. genesys_run_simulation (token)
6. genesys_create_job    (token) → { jobId: "..." }
7. genesys_run_job       (token, jobId)
8. genesys_job_result    (token, jobId)
```

2.7 Available tools (18)

Tool	Auth required	Description
<code>genesys_health</code>	No	Check worker app reachability
<code>genesys_auth</code>	No	Create session, returns token
<code>genesys_info</code>	Yes	Simulator version and licence
<code>genesys_new_model</code>	Yes	Create empty model
<code>genesys_get_model</code>	Yes	Describe current model
<code>genesys_save_model</code>	Yes	Save model to workspace file
<code>genesys_load_model</code>	Yes	Load model from workspace file
<code>genesys_import_model_language</code>	Yes	Import model from text
<code>genesys_simulation_status</code>	Yes	Current simulation state
<code>genesys_configure_simulation</code>	Yes	Set replication/length/warmup params
<code>genesys_run_simulation</code>	Yes	Run simulation synchronously
<code>genesys_step_simulation</code>	Yes	Advance one event step
<code>genesys_worker_info</code>	No	Worker identity metadata
<code>genesys_worker_capabilities</code>	No	Worker capability flags
<code>genesys_create_job</code>	Yes	Create job from current model
<code>genesys_job_status</code>	Yes	Query job state
<code>genesys_run_job</code>	Yes	Execute job synchronously
<code>genesys_job_result</code>	Yes	Retrieve job result summary

Chapter 3

Genesys Worker Application

First functional implementation of the **Worker** application type for GenESyS.

3.1 Current scope

- CMake target: `genesys_worker_app` (enabled with `-DGENESYS_BUILD_WORKER_APPLICATION=ON`).
- Executable name: `genesys-worker`.
- Layered architecture for HTTP transport, API routing, session/token, and simulator service.
- Stateful session management with one [Simulator](#) per session and per-session workspace directories.
- Endpoints:
 - GET `/health`
 - POST `/api/v1/auth/session`
 - GET `/api/v1/simulator/info` (requires Authorization: Bearer <token>)
 - POST `/api/v1/models` (requires Authorization: Bearer <token>)
 - GET `/api/v1/models/current` (requires Authorization: Bearer <token>)
 - POST `/api/v1/models/save` (requires Authorization: Bearer <token>)
 - POST `/api/v1/models/load` (requires Authorization: Bearer <token>)
 - GET `/api/v1/simulation/status` (requires Authorization: Bearer <token>)
 - POST `/api/v1/simulation/config` (requires Authorization: Bearer <token>)
 - POST `/api/v1/simulation/run` (requires Authorization: Bearer <token>)
 - POST `/api/v1/simulation/step` (requires Authorization: Bearer <token>)

3.2 Worker cycle Stage 1 (new cycle)

The Worker application is now also being evolved as a **worker API** that can be discovered by a future distributed orchestrator application.

This stage adds two public worker discovery endpoints (no Bearer token required):

- GET `/api/v1/worker/info`
- GET `/api/v1/worker/capabilities`

These routes expose worker identity metadata and current implementation capability flags. They are intentionally limited to discovery only and do not include job submission, polling, or background execution in this stage.

3.3 Worker cycle Stage 2

This stage starts worker model-ingress support with one authenticated and session-scoped endpoint:

- `POST /api/v1/worker/models/import-language` (requires Authorization: Bearer <token>)

The request body must be plain text GenESyS model language/specification. The worker imports this text into the session simulator as the current model.

This stage still does **not** include multipart/binary upload, distributed job submission, polling, or background orchestration.

3.4 Worker cycle Stage 3

This stage introduces a minimal worker job abstraction with authenticated, session-scoped endpoints:

- `POST /api/v1/worker/jobs` (requires Authorization: Bearer <token>)
- `GET /api/v1/worker/jobs/{jobId}` (requires Authorization: Bearer <token>)

Behavior in this stage:

- job creation requires a valid current model in the session;
- each job is created in in-memory storage with state `queued`;
- job creation persists a session-workspace snapshot file (for example `job_<id>.gen`) to decouple future execution from live model mutations;
- job metadata can be queried by id from the same authenticated session.

This stage still does **not** execute jobs, provide result retrieval, add polling loops, or introduce background worker threads.

3.5 Worker cycle Stage 4

This stage adds explicit, synchronous execution of existing worker jobs:

- `POST /api/v1/worker/jobs/{jobId}/run` (requires Authorization: Bearer <token>)

Behavior in this stage:

- execution is triggered only for an existing job id;
- job state is updated in memory through `queued -> running -> finished|failed`;
- execution is synchronous and performed by the session simulator using the job snapshot file;
- `GET /api/v1/worker/jobs/{jobId}` remains the basic state inspection/polling mechanism after run requests.

This stage still does **not** include background execution, queue worker loops, cancellation, streaming updates, or a dedicated job result endpoint.

3.6 Worker cycle Stage 5

This stage adds terminal result retrieval for previously executed worker jobs:

- `GET /api/v1/worker/jobs/{jobId}/result` (requires Authorization: Bearer <token>)

Behavior in this stage:

- result retrieval is available only after the job reaches a terminal state (`finished` or `failed`);
- queued or running jobs return a conflict response because no terminal result exists yet;
- the endpoint returns a minimal structured simulation summary persisted at the end of synchronous execution;
- execution remains synchronous and non-background (`POST /run` still performs the run inline);
- cancellation is still not available;
- distributed orchestration remains out of scope.

3.7 Worker cycle Stage 6

This stage hardens HTTP transport behavior for existing worker and session APIs without changing route contracts:

- request reading is now incremental (`recv` loop) instead of single-shot reads;
- request assembly is bounded by a transport limit of **1 MiB** (larger payloads return `413 Payload Too Large`);
- socket receive/send timeouts are configured to **5 seconds** per client connection;
- when timeout happens before a complete request is assembled, the server returns `408 Request Timeout`;
- transport-level error payloads remain JSON-shaped and consistent with current server error responses.

This stage is explicitly operational hardening only. Application-layer endpoints and worker cycle stages 1-5 contracts remain unchanged.

3.8 How to run

```
cmake -S . -B build/worker-debug -G Ninja -DGENESYS_BUILD_WORKER_APPLICATION=ON
cmake --build build/worker-debug --target genesys_worker_app
./build/worker-debug/source/applications/worker/genesys-worker --port 8080
```

Optional parameters:

- `--port <N>`: HTTP port (default 8080).
- `--max-requests <N>`: exit after N requests (useful for scripted smoke tests).

3.9 Security notice

Session creation in Stage 1 is provisional and stateful in memory. Generated tokens are opaque random strings and **not** production-grade authentication.

For Stage 3 model persistence:

- Save/load operations are restricted to each session workspace directory.
- API requests accept only a `filename` `basename` (no paths).
- Filenames must match `[A-Za-z0-9._-]` and cannot include `..`, `/`, or `\`.

For Stage 4 simulation configuration:

- `POST /api/v1/simulation/config` currently uses a minimal contract parser without external JSON libraries.
- All configuration fields are required in the request body: `numberOfReplications`, `replicationLength`, `warmUpPeriod`, `pauseOnEvent`, `pauseOnReplication`, `initializeStatistics`, `initializeSystem`.

For Stage 5 simulation execution:

- `POST /api/v1/simulation/run` and `POST /api/v1/simulation/step` are synchronous wrappers over `ModelSimulation::start()` and `ModelSimulation::step()`.
- This stage intentionally does not introduce asynchronous/background simulation control endpoints.

Chapter 4

Todo List

Class `GraphicalConnection`

Evaluate decoupling model metadata from this item to keep this class purely visual.

Member `GraphicalConnection::GraphicalConnection` (`GraphicalComponentPort` *sourceGraphicalPort, `GraphicalComponentPort` *destinationGraphicalPort, unsigned int portSourceConnection=0, unsigned int portDestinationConnection=0, `QColor` color=Qt::black, `QGraphicsItem` *parent=nullptr)

Extrair para método dedicado `_bindPorts()` para legibilidade.

Member `ModelGraphicsScene::notifyGraphicalModelChange` (`GraphicalModelEvent::EventType` eventType, `GraphicalModelEvent::EventObjectType` eventObjectType, `QGraphicsItem` *item)

Consider broadcasting to all attached views when multi-view editing is supported.

Member `ModelGraphicsView::notifySceneGraphicalModelEventHandler` (const `GraphicalModelEvent` &modelGraphicsEvent)

Replace callback with signal/slot to improve composability.

actualize property editor?

Member `SimulationController::ensureReady` (bool checkModel, bool modelChecked, const std::function< bool()> &runModelCheck, const std::function< bool()> &syncModelFromText) const

Replace callback-based contract by dedicated interfaces to improve testability.

Chapter 5

Namespace Index

5.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

genesys_mcp	61
genesys_mcp.__main__	61
genesys_mcp.client	61
genesys_mcp.server	62
GuiCrashDiagnostics	
Crash diagnostics helpers for the Qt GUI process	67
QCP	68

Chapter 6

Hierarchical Index

6.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AnimationQueue	120
ApiRouter	130
Assignment	140
BacteriaColony::BacteriumState	164
BacteriaColony::BarrierSegment	167
BaseMVController	171
BioSBMLBridge	227
BioSBMLExportResult	228
BioSBMLImportResult	228
BoundaryCondition	271
Boundary_Closed	268
Boundary_Fixed	270
QCPAxisPainterPrivate::CachedLabel	282
QCPLabelPainterPrivate::CachedLabel	282
ModelLifecycleController::Callbacks	282
SimulationEventController::Callbacks	283
Cell	290
CellularAutomata	322
CellularAutomataBase	328
CellularAutomata_1DTimed	324
CellularAutomata_Classic	326
CellularAutomataDemoBuilder	337
CellularAutomataDemoModel	337
CellularAutomataDemoSettings	338
CellularAutomataEvent	338
CellularAutomataStateColorMap	342
Collector_if	353
CollectorDatafile_if	356
CollectorDatafileDefaultImpl1	359
CollectorDefaultImpl1	363
GroProgramRuntime::ColonyMutation	365
GroProgramIrr::Command	367
Util::CommandResult	367
CppCompiler::CompilationResult	375

CompilerThread	376
ComponentItem	376
Connection	381
ConnectionChannel	381
ConnectionManager	382
CppModelExporter	417
SimulatorSessionService::CreateSessionResult	429
QtBoolPropertyManagerPrivate::Data	430
QtDatePropertyManagerPrivate::Data	430
QtDateTimePropertyManagerPrivate::Data	431
QtDoublePropertyManagerPrivate::Data	431
QtEnumPropertyManagerPrivate::Data	431
QtFlagPropertyManagerPrivate::Data	432
QtIntPropertyManagerPrivate::Data	432
QtPointFPropertyManagerPrivate::Data	432
QtRectFPropertyManagerPrivate::Data	433
QtRectPropertyManagerPrivate::Data	433
QtSizeFPropertyManagerPrivate::Data	433
QtSizePropertyManagerPrivate::Data	434
QStringPropertyManagerPrivate::Data	434
QTimePropertyManagerPrivate::Data	434
DataDefinitionItem	438
DataDefinitionRelationItem	438
ModelInspectorController::DataDefinitionSelection	439
AnimationPlot::DataSeries	440
Delay::DEFAULT_VALUES	447
ModalModelDefault::DEFAULT_VALUES	447
Seize::DEFAULT_VALUES	448
SourceModelComponent::DEFAULT_VALUES	448
DefaultNodeTransition	457
EFSMTransition	535
PetriTransition	1060
DialogUtilityController	477
DrawingItem	514
EditCommandController	533
EditorFactoryPrivate< Editor >	534
EditorFactoryPrivate< QComboBox >	534
QtEnumEditorFactoryPrivate	1840
EditorFactoryPrivate< QKeySequenceEdit >	534
QtKeySequenceEditorFactoryPrivate	1880
EditorFactoryPrivate< QScrollBar >	534
QtScrollBarFactoryPrivate	1931
EditorFactoryPrivate< QSlider >	534
QtSliderFactoryPrivate	1953
EditorFactoryPrivate< QtBoolEdit >	534
QtCheckBoxFactoryPrivate	1788
EditorFactoryPrivate< QtCharEdit >	534
QtCharEditorFactoryPrivate	1780
EditorFactoryPrivate< QColorEditWidget >	534
QtColorEditorFactoryPrivate	1791
EditorFactoryPrivate< QtDateEdit >	534
QtDateEditFactoryPrivate	1810
EditorFactoryPrivate< QtDateTimeEdit >	534
QtDateTimeEditFactoryPrivate	1820
EditorFactoryPrivate< QtDoubleEdit >	534

QtDoubleSpinBoxFactoryPrivate	1836
EditorFactoryPrivate< QFontEditWidget >	534
QtFontEditorFactoryPrivate	1855
EditorFactoryPrivate< QtIntEdit >	534
QtSpinBoxFactoryPrivate	1957
EditorFactoryPrivate< QtStringEdit >	534
QtLineEditFactoryPrivate	1888
EditorFactoryPrivate< QtTimeEdit >	534
QtTimeEditFactoryPrivate	1967
PersistenceRecord::Entry	564
Event	573
InternalEvent	737
Exact	575
Exception	
genesys_mcp.client.GenesysClientError	635
CompartmentExchangeComponent::ExchangeRule	581
GroProgramRuntime::ExecutionResult	582
ExperimentManager	588
ExperimentManager_if	589
ExperimentManagerDefaultImpl1	591
ExpressionCategory	594
ExpressionParameter	594
ExpressionTemplate	594
ParserManager::GenerateNewParserResult	632
GenesysApplication_if	633
BaseGenesysTerminalApplication	167
AirportSecurityExample	91
AirportSecurityExampleExtended	93
Airport_Extended1	89
AnElectronicAssemblyAndTestSystem	108
Assembly_Line	131
Banking_Transactions	165
Book_Cap02_Example01	266
BufferFIFO	280
ContinuousModel	388
Example_BasicOrderShipping	576
Example_PortModel	578
Example_PublicTransport	580
FullSimulationOfComplexModel	630
GenesysShell	638
Half_Adder	725
Loja01	780
OperatingSystem02	1011
OperatingSystem03	1013
Rectifier	2017
Smart_AddingResource	2253
Smart_AlternatingEntityCreation	2255
Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime	2257
Smart_ArrivalsEntityTypeVsAttribute	2259
Smart_AssignExample	2261
Smart_AssignWriteSeizes	2263
Smart_Attribute_Variable	2265
Smart_AutomaticStatisticsCollection	2267
Smart_BacteriaColony_GRO	2269
Smart_BasicModeling	2271
Smart_BatchAndSeparate	2273

Smart_BatchSeparate	2275
Smart_BioKineticLawRegulation	2277
Smart_BioNetworkAnalysis	2279
Smart_BioReversibleKineticLaw	2281
Smart_BioReversibleMassAction	2283
Smart_Buffer	2285
Smart_CellularAutomata	2287
Smart_CellularAutomata1	2289
Smart_Clone	2291
Smart_ContinuousFlowEntities	2293
Smart_CppForG	2295
Smart_Create	2297
Smart_DecideNWayByChance	2299
Smart_DefaultModalModel	2301
Smart_DefiningAttributesAsStrings	2303
Smart_DefiningControlLogic	2305
Smart_DefiningResourceCapacity	2307
Smart_Delay	2309
Smart_DelayBasedOnReplication	2311
Smart_Dummy	2313
Smart_EntitiesProcessedByPriority	2315
Smart_EvaluatingConditionsBeforeEnteringQueue	2317
Smart_Expression	2319
Smart_FSMState	2323
Smart_Failures	2321
Smart_GroColonyGrowth	2325
Smart_GroColonyLifecycle	2327
Smart_HoldSearchRemove	2329
Smart_InventoryAndHoldingCosts	2331
Smart_LNODE	2333
Smart_MarkovChain	2335
Smart_MaxArrivalsField	2337
Smart_ModalModel	2339
Smart_ModalModelDefaultNetwork	2341
Smart_ModalModelFSM	2343
Smart_ModalModelPetriNet	2345
Smart_ModelInfoModelSimulation	2347
Smart_ModelRunUntil1000Parts	2349
Smart_ModuleDisplayVariables	2351
Smart_ODE	2353
Smart_OnEvent	2355
Smart_OverlappingResources	2358
Smart_ParallelProcessingOfEntities	2360
Smart_Parser	2362
Smart_ParserModelFunctions	2364
Smart_PetriPlace	2366
Smart_PlacingEntitiesInQueueSets	2368
Smart_Plugin	2370
Smart_PriorityExample	2372
Smart_Process	2374
Smart_ProcessArena	2376
Smart_ProcessSet	2378
Smart_PythonForG	2380
Smart_R_Simulator	2381
Smart_Record	2384
Smart_Record_Arena	2386
Smart_RemovingAndReorderingEntitiesInAQueue	2388
Smart_ResourceCosting	2390

Smart_ResourceScheduleCosting	2392
Smart_ResourceSets	2394
Smart_RouteStation	2396
Smart_SeizeDelayRelease	2398
Smart_SeizeDelayReleaseMany	2400
Smart_SeizeDelayReleaseNoDataDefs	2402
Smart_SeizingMultipleSimultaneosly	2404
Smart_SelectingRouteBasedOnProbability	2406
Smart_SelectingShorterQueue	2408
Smart_Sequence	2410
Smart_SimulationControlResponse	2412
Smart_SynchronizingParallelEntities	2414
Smart_ValueAdded	2416
Smart_WaitForSignal	2418
Smart_WaitScanCondition	2420
Smart_WaitSignal	2422
StartAIAssistant	2463
WcmBioNetworkBridge	2641
WcmCompartmentTransportWholeCellMvp	2643
WcmFluxBalanceWholeCellMvp	2645
WcmGeneExpression	2647
WcmLifecycleStress	2649
WcmMgenitaliumKarr2012	2652
WcmPathwayStressWholeCellMvp	2654
WcmSimpleSsa	2656
WcmWholeCellMvp	??
WcmYeastDidacticWholeCellMvp	??
WcmYeastEukaryoticWholeCellMvp	??
BaseGenesysWebApplication	169
GenesysShell_if	640
genesys_mcp.client.GenesysClient	634
GenesysPropertyDescriptor	636
GenesysPropertyIntrospection	637
GeneticCircuitSimulationSummary	660
GraphicalAssociation	677
GraphicalAutomaticPositioningStrategy	678
GraphicalAutomaticPositioningStrategyFactory	679
GraphicalConnectionStyle	684
GraphicalContextMenuController	685
GraphicalDataDefinitionLayout	685
GraphicalModelBuilder	688
GraphicalModelEvent	698
GraphicalModelItemRenderContext	699
GraphicalModelItemRenderer	700
GraphicalModelItemRenderStrategy	700
GraphicalModelSerializer	701
GraphvizModelExporter	703
ModelGraphicsScene::GRID	705
GroProgramAst	711
GroProgramCompiler	711
GroProgramIr	712
GroProgramParser	713
GroProgramRuntime	713
GroProgramRuntimeState	714
GroupItem	715
GuiActionContribution	716
GuiDockContribution	716
GuiExtensionManager	719

GuiExtensionPlugin	720
BacteriaColonyViewerGuiExtensionPlugin	157
BioNetworkAwareGuiExtensionPlugin	192
BioNetworkEditorGuiExtensionPlugin	193
BioNetworkExecutionGuiExtensionPlugin	195
BioParameterManagerGuiExtensionPlugin	202
BioReactionKineticsGuiExtensionPlugin	216
BioReactionStoichiometryGuiExtensionPlugin	218
BioSBMLInteroperabilityGuiExtensionPlugin	228
BioSimulationResultsGuiExtensionPlugin	237
BioSpeciesManagerGuiExtensionPlugin	250
CellularAutomataGuiExtensionPlugin	339
CoreGuiExtensionPlugin	396
GeneticCircuitGuiExtensionPlugin	647
GuiExtensionConfigurationGuiExtensionPlugin	718
ModelPluginInspectorGuiExtensionPlugin	941
GuiExtensionPluginAutoRegistrar	721
GuiExtensionPluginCatalog	721
GuiExtensionPresentationMetadata	721
GuiExtensionRegistry	722
GuiExtensionRuntimeContext	722
GuiScopeTrace	723
GuiThemeManager	723
GuiWindowContribution	724
HttpRequest	732
HttpResponse	733
IssueReportDraft	738
IssueReportRelayService	739
IssueReportRuntimeContext	740
IssueReportSubmissionResult	740
Json	741
QCPLabelPainterPrivate::LabelData	750
Lattice	751
LicenceManager	759
List< T >	761
ListObservable< T >	763
List< Assignment * >	761
List< Counter * >	761
List< DefaultNode * >	761
List< DefaultNodeTransition * >	761
List< double >	761
List< Entity * >	761
List< Event * >	761
List< Failure * >	761
List< Model * >	761
List< ModelComponent * >	761
List< ModelDataDefinition * >	761
List< modelEventHandler >	761
List< modelEventHandlerMethod >	761
List< ODEfunction * >	761
List< PairSignalDataEventHandler * >	761
List< PickableStationItem * >	761
List< Plugin * >	761
List< PluginLoadIssue >	761
List< Queue * >	761
List< Resource * >	761
List< SchedulableItem * >	761
List< SeizableItem * >	761

List< SequenceStep * >	761
List< ShellCommand * >	761
List< SimulationControl * >	761
List< simulationEventHandler >	761
List< simulationEventHandlerMethod >	761
List< SimulationExperiment * >	761
List< SimulationResponse * >	761
List< SortedResourceEventHandler * >	761
List< SparseValueStore * >	761
List< StatisticsCollector * >	761
List< std::string >	761
List< traceErrorListener >	761
List< traceErrorListenerMethod >	761
List< traceListener >	761
List< traceListenerMethod >	761
List< traceSimulationListener >	761
List< traceSimulationListenerMethod >	761
List< void * >	761
List< Waiting * >	761
LocalRule	766
LocalRule_CopyCurrentState	768
LocalRule_Elementary	769
LocalRule_FlorestalFire	771
LocalRule_GameOfLife	772
LocalRule_Growty	774
LocalRule_HppLatticeGas	775
LocalRule_SandRock_Student	777
LocalRule_SandRock_Student	777
LocalRule_ShowNeighbordhood	779
Model	858
ModelChecker_if	869
ModelCheckerDefaultImpl1	872
ModelComponentManager	884
ModelDataDefinitionAssociations	905
ModelDataManager	907
ModelEvent	912
SimulatorSessionService::ModelImportResult	924
ModelInfo	925
SimulatorSessionService::ModelInfoResult	929
ModelInspectorController	929
ModelLanguageSynchronizer	930
ModelLifecycleController	932
ModelManager	934
SimulatorSessionService::ModelPersistenceResult	940
ModelSimulation	943
GroProgramRuntime::MotionMutation	950
dialogBreakpoint::MVCResult	951
GroProgramAst::NamedProgram	952
GroProgramIrr::NamedProgramDefinition	952
Neighborhood	961
Neighborhood_Center	962
Neighborhood_Moore	964
Neighborhood_VonNeumann	966
ParserManager::NewParser	968
ODEfunction	998
OnEventManager	1010
Parser_if	1023

ParserDefaultImpl1	1026
ParserDefaultImpl2	1029
ParserChangesInformation	1026
ParserManager	1032
Persistence_if	1042
PersistenceDefaultImpl2	1045
PersistencePartialLoadImpl1	1047
PersistenceRecord	1049
PersistentObject_base	1051
ModelDataDefinition	889
AISupport	95
Attribute	141
Variable	2603
BacteriaSignalGrid	158
BioCompartment	179
BioNetwork	185
BioParameter	197
BioReaction	204
BioSimulatorRunner	239
BioSpecies	244
Counter	398
CppCompiler	404
DiffusionField	486
DummyElement	528
Entity	544
EntityGroup	553
EntityType	558
Failure	595
File	601
Formula	608
GeneticCircuit	641
GeneticCircuitPart	648
GeneticRegulation	668
GroProgram	705
Label	745
MetabolicNetwork	812
MetabolicReaction	818
ModelComponent	874
AIAssistant	81
Access	73
Assign	133
BacteriaColony	148
Batch	172
BioRunnerCommand	220
BioSimulate	230
BioStateProjectionComponent	251
BioSteadyState	259
Buffer	273
CellCycleCheckpointComponent	291
CellDivisionEvent	300
CellFateDecisionComponent	307
CellGrowthComponent	315
CellularAutomataComp	329
Clone	344
CompartmentExchangeComponent	368
ContinuousSystemComponent	390
CppForG	410
Decide	440

DefaultNode	450
FSMState	616
PetriPlace	1054
Delay	458
DiffEquations	479
DiffusionSimulate	492
DropOff	514
DummyComponent	522
Enter	537
EukaryoticCellCycleComponent	565
Exit	582
FtsZPolymerizationComponent	622
GeneticCircuitSimulate	654
GeneticExpressionStep	661
LSODE	782
Leave	752
MarkovChain	791
Match	798
MetabolicFluxBalance	805
MetabolicStateProjectionComponent	823
MetabolicSubmodelComponent	831
ModalModelDefault	838
ModalModelFSM	846
ModalModelPetriNet	852
PathwayStressResponseComponent	1034
PickStation	1062
Process	1101
PythonForG	1117
RSimulator	2063
Record	2009
Release	2019
Remove	2026
PickUp	1070
ResourceAllocationComponent	2047
Route	2056
SPICECircuit	2438
AND	101
Capacitor	283
Diode	499
NAND	952
NMOS	968
NOR	975
NOT	982
OR	1016
PMOS	1094
Resistor	2034
Vpulse	2611
Vsine	2618
Vsource	2625
XNOR	??
XOR	??
SPICENode	2445
Search	2098
Seize	2107
Separate	2114
Signal	2142
SinkModelComponent	2247
Dispose	506

SourceModelComponent	2425
Create	421
Start	2456
StochasticReactionComponent	2501
StochasticTranscription	2514
StochasticTranslation	2521
Stop	2529
Store	2541
Submodel	2548
Unstore	2584
Wait	2632
Write	??
MolecularSpecies	945
ODESolver	999
OLD_ODEElement	1005
OctaveRunner	992
PythonRuntime	1123
Queue	1998
RSimulatorRunner	2070
Resource	2040
SPICERunner	2451
Schedule	2092
Sequence	2122
Set	2135
SignalData	2149
Station	2469
StatisticsCollector	2480
StochasticReactionRule	2508
Storage	2536
WholeCellState	??
SequenceStep	2127
PickableStationItem	1062
Plugin	1076
PluginCatalogController	1079
PluginConnector_if	1080
PluginConnectorDummyImpl1	1083
PluginConnectorStaticImpl1	1086
PluginInformation	1089
PluginInsertionOptions	1090
PluginLoadIssue	1091
PluginManager	1092
GroProgramRuntime::PopulationMutation	1101
BioStateProjectionComponent::Projection	1110
MetabolicStateProjectionComponent::Projection	1110
PropertyEditorController	1112
PropertyEditorGenesys	1113
PropertyGenesysBase	1115
PropertyGenesys	1113
SimulationResponse	2222
SimulationControl	2159
SimulationControlBool	2162
SimulationControlDouble	2165
SimulationControlGeneric< T >	2168
SimulationControlGenericClass< T, M, C >	2172
SimulationControlGenericClassNotDC< T, M, C >	2177
SimulationControlGenericEnum< T, E >	2182
SimulationControlGenericList< T, M, C >	2185

SimulationControlGenericListPointer< T, M, C >	2189
SimulationControlInt	2194
SimulationControlSourceCodeString	2199
SimulationControlString	2203
SimulationControlTimeUnit	2206
SimulationControlUInt	2209
SimulationControlUShort	2212
SimulationResponseDouble	2224
QCPAbstractPaintBuffer	1146
QCPPaintBufferPixmap	1556
QCPAxisPainterPrivate	1201
QCPAxisTicker	1223
QCPAxisTickerDateTime	1228
QCPAxisTickerFixed	1235
QCPAxisTickerLog	1238
QCPAxisTickerPi	1241
QCPAxisTickerText	1245
QCPAxisTickerTime	1251
QCPBarsData	1268
QCPColorGradient	1276
QCPColorMapData	1295
QCPCurveData	1331
QCPDataContainer< DataType >	1334
QCPDataRange	1344
QCPDataSelection	1348
QCPErrorBarsData	1369
QCPFinancialData	1384
QCPGraphData	1398
QCPItemAnchor	1405
QCPItemPosition	1440
QCPLabelPainterPrivate	1476
QCPLineEnding	1550
QCPPlottableInterface1D	1564
QCPAbstractPlottable1D< QCPBarsData >	1165
QCPBars	1256
QCPAbstractPlottable1D< QCPCurveData >	1165
QCPCurve	1320
QCPAbstractPlottable1D< QCPFinancialData >	1165
QCPFinancial	1370
QCPAbstractPlottable1D< QCPGraphData >	1165
QCPGraph	1386
QCPAbstractPlottable1D< QCPStatisticalBoxData >	1165
QCPStatisticalBox	1674
QCPAbstractPlottable1D< DataType >	1165
QCPErrorBars	1356
QCPRange	1644
QCPScatterStyle	1652
QCPSelectionDecorator	1659
QCPSelectionDecoratorBracket	1663
QCPStatisticalBoxData	1685
QCPVector2D	1698
QDialog	
DialogFind	467
DialogModelInformation	468
DialogPluginManager	469
DialogReportIssue	470

DialogSelectCounter	471
DialogSelectVariable	472
DialogSimulationConfigure	473
DialogSystemPreferences	475
DialogTimerConfigure	476
ExpressionBuilder	593
GuiExtensionConfigurationDialog	717
WebWorkerDialog	??
dialogBreakpoint	466
QGraphicsLinellItem	
GraphicalDiagramConnection	686
QGraphicsObject	
GraphicalAnimateExpression	677
GraphicalComponentPort	679
GraphicalConnection	681
GraphicalModelDataDefinition	696
GraphicalModelComponent	691
QGraphicsPixmapItem	
GraphicalImageAnimation	687
QGraphicsRectItem	
AnimationCounter	111
AnimationPlaceholder	116
AnimationAttribute	110
AnimationEntity	112
AnimationEvent	114
AnimationExpression	115
AnimationPlot	118
AnimationQueueDisplay	120
AnimationResource	122
AnimationStation	123
AnimationStatistics	124
AnimationTimer	126
AnimationVariable	128
QGraphicsScene	
ModelGraphicsScene	912
QGraphicsView	
ModelGraphicsView	921
QItemDelegate	
QtPropertyEditorDelegate	1913
QLineEdit	
FindLineEdit	607
QMainWindow	
AIAssistantWindow	88
CellularAutomataViewerWindow	344
DataAnalyzerWindow	435
MainWindow	789
OptimizerWindow	1015
QObject	
CellularAutomataViewerController	342
ComboBoxEnum	366
DataComponentEditor	436
DataComponentProperty	437
QCPBarsGroup	1271
QCPLayer	1479
QCPLayerable	1484
QCPAbstractItem	1129
QCPIItemBracket	1408
QCPIItemCurve	1415

QCPItemEllipse	1421
QCPItemLine	1428
QCPItemPixmap	1434
QCPItemRect	1448
QCPItemStraightLine	1454
QCPItemText	1459
QCPItemTracer	1468
QCPAbstractPlottable	1150
QCPAbstractPlottable1D< QCPBarsData >	1165
QCPAbstractPlottable1D< QCPCurveData >	1165
QCPAbstractPlottable1D< QCPFinancialData >	1165
QCPAbstractPlottable1D< QCPGraphData >	1165
QCPAbstractPlottable1D< QCPStatisticalBoxData >	1165
QCPAbstractPlottable1D< DataType >	1165
QCPColorMap	1283
QCPErrors	1356
QCPAxis	1174
QCPGrid	1401
QCPLayoutElement	1500
QCPAbstractLegendItem	1139
QCPPlottableLegendItem	1568
QCPPolarLegendItem	1639
QCPAxisRect	1203
QCPColorScaleAxisRectPrivate	1313
QCPColorScale	1303
QCPLayout	1492
QCPLayoutGrid	1510
QCPLegend	1534
QCPLayoutInset	1525
QCPPolarAxisAngular	1574
QCPTextElement	1688
QCPPolarAxisRadial	1601
QCPPolarGraph	1624
QCPPolarGrid	1635
QCPSelectionRect	1668
QCPMarginGroup	1553
QtAbstractEditorFactoryBase	1748
QtAbstractEditorFactory< QtCharPropertyManager >	1744
QtCharEditorFactory	1778
QtAbstractEditorFactory< QtBoolPropertyManager >	1744
QtCheckBoxFactory	1786
QtAbstractEditorFactory< QtColorPropertyManager >	1744
QtColorEditorFactory	1789
QtAbstractEditorFactory< QtCursorPropertyManager >	1744
QtCursorEditorFactory	1799
QtAbstractEditorFactory< QtDatePropertyManager >	1744
QtDateEditFactory	1808
QtAbstractEditorFactory< QDateTimePropertyManager >	1744
QtDateTimeEditFactory	1818
QtAbstractEditorFactory< QtDoublePropertyManager >	1744
QtDoubleSpinBoxFactory	1834
QtAbstractEditorFactory< QtEnumPropertyManager >	1744
QtEnumEditorFactory	1838
QtAbstractEditorFactory< QtFontPropertyManager >	1744
QtFontEditorFactory	1853
QtAbstractEditorFactory< QtKeySequencePropertyManager >	1744

QtKeySequenceEditorFactory	1878
QtAbstractEditorFactory< QtStringPropertyManager >	1744
QtLineEditFactory	1886
QtAbstractEditorFactory< QtIntPropertyManager >	1744
QtScrollBarFactory	1929
QtSliderFactory	1951
QtSpinBoxFactory	1955
QtAbstractEditorFactory< QtTimePropertyManager >	1744
QtTimeEditFactory	1965
QtAbstractEditorFactory< QtVariantPropertyManager >	1744
QtVariantEditorFactory	1980
QtAbstractEditorFactory< PropertyManager >	1744
QtAbstractPropertyManager	1759
QtBoolPropertyManager	1767
QtCharPropertyManager	1782
QtColorPropertyManager	1794
QtCursorPropertyManager	1802
QtDatePropertyManager	1811
QtDateTimePropertyManager	1822
QtDoublePropertyManager	1827
QtEnumPropertyManager	1841
QtFlagPropertyManager	1847
QtFontPropertyManager	1858
QtGroupPropertyManager	1867
QtIntPropertyManager	1871
QtKeySequencePropertyManager	1882
QtLocalePropertyManager	1889
QtPointFPropertyManager	1895
QtPointPropertyManager	1901
QtRectFPropertyManager	1916
QtRectPropertyManager	1923
QtSizeFPropertyManager	1932
QtSizePolicyPropertyManager	1940
QtSizePropertyManager	1945
QtStringPropertyManager	1959
QtTimePropertyManager	1969
QtVariantPropertyManager	1987
QtMetaEnumWrapper	1894
QPainter	
QCPPainter	1559
QPlainTextEdit	
CodeEditor	352
QtAbstractPropertyBrowserPrivate	1757
QtAbstractPropertyManagerPrivate	1765
QtBoolPropertyManagerPrivate	1770
QtBrowserItem	1771
QtBrowserItemPrivate	1772
QtButtonPropertyBrowserPrivate	1776
QtCharPropertyManagerPrivate	1785
QtColorPropertyManagerPrivate	1798
QtCursorDatabase	1799
QtCursorEditorFactoryPrivate	1801
QtCursorPropertyManagerPrivate	1806
QtDatePropertyManagerPrivate	1817
QtDateTimePropertyManagerPrivate	1825
QtDoublePropertyManagerPrivate	1834
QtEnumPropertyManagerPrivate	1847
QtEnumPropertyType	1847

QtFlagPropertyManagerPrivate	1852
QtFlagPropertyType	1853
QtFontPropertyManagerPrivate	1862
QtGroupBoxPropertyBrowserPrivate	1867
QtGroupPropertyType	1870
QtIntPropertyManagerPrivate	1877
QtKeySequencePropertyManagerPrivate	1885
QtLocalePropertyManagerPrivate	1893
QtMetaEnumProvider	1894
QtPointFPropertyManagerPrivate	1900
QtPointPropertyManagerPrivate	1905
QtProperty	1906
QtVariantProperty	1983
QtPropertyBrowserUtils	1913
QtPropertyPrivate	1915
QtRectFPropertyManagerPrivate	1922
QtRectPropertyManagerPrivate	1927
QTreeWidgetItem	
PropertyEditor	1111
QtPropertyEditorView	1914
QtSizeFPropertyManagerPrivate	1939
QtSizePolicyPropertyManagerPrivate	1944
QtSizePropertyManagerPrivate	1950
QtStringPropertyManagerPrivate	1963
QtTimePropertyManagerPrivate	1972
QtTreePropertyBrowserPrivate	1979
QtVariantEditorFactoryPrivate	1982
QtVariantPropertyManagerPrivate	1996
QtVariantPropertyPrivate	1997
QueueableItem	2004
QUndoCommand	
AddUndoCommand	80
DeleteUndoCommand	465
GroupUndoCommand	715
MoveUndoCommand	950
PasteUndoCommand	1033
UngroupUndoCommand	2583
QVariantAnimation	
AnimationTransition	127
QWidget	
CellularAutomataLatticeWidget	341
DataAnalyzerWindow::DoePreviewPlot	513
DataAnalyzerWindow::HistogramPreview	727
LineNumberArea	760
OptimizerWindow::ProgressPlot	1109
QCustomPlot	1705
QtAbstractPropertyBrowser	1750
QtButtonPropertyBrowser	1773
QtGroupBoxPropertyBrowser	1864
QtTreePropertyBrowser	1973
ObjectPropertyBrowser	989
QtBoolEdit	1766
QtCharEdit	1777
QtColorEditWidget	1793
QtDateEdit	1806
QtDateTimeEdit	1817
QtDoubleEdit	1826
QtFontEditWidget	1857

QtIntEdit	1870
QtStringEdit	1958
QtTimeEdit	1964
GroProgramParser::Result	2054
Sampler_if::RNG_Parameters	2055
SamplerDefaultImpl1::DefaultImpl1RNG_Parameters	449
Sampler_if	2076
SamplerDefaultImpl1	2082
ScenarioExperiment_if	2088
SceneToolController	2088
SchedulableItem	2092
SeizableItem	2105
Serializer	2129
CppSerializer	418
GenSerializer	673
JsonSerializer	742
XmlSerializer	??
SessionContext	2132
SessionManager	2133
GroProgramRuntime::SignalMutation	2154
SimpleHttpServer	2154
SimulatorSessionService::SimulationActionResult	2155
SimulationCommandController	2156
SimulatorSessionService::SimulationConfigInput	2157
SimulatorSessionService::SimulationConfigResult	2158
SimulationController	2196
SimulationEvent	2215
SimulationEventController	2216
SimulationExperiment	2217
SimulationReporter_if	2218
SimulationReporterDefaultImpl1	2219
SimulationScenario	2225
SimulatorSessionService::SimulationStatusResult	2227
Simulator	2227
SimulatorFacade	2230
SimulatorSessionService::SimulatorInfoResult	2236
SimulatorSessionService	2237
WebWorkerRuntime::Snapshot	2423
SortFile	2424
SourceCodeString	2425
SparseValueStore	2437
State	2464
CompositeState	377
HppLatticeGasState	728
RealState	2007
GroProgramAst::Statement	2465
StateSet	2465
CompositeStateSet	379
HppLatticeGasStateSet	730
IntegerStateSet	734
NaturalStateSet	959
RealIntervalStateSet	2005
StateSet_Enumerable	2467
Statistics_if	2475
StatisticsDatafile_if	2485
StatisticsDatafileDefaultImpl1	2490

StatisticsDefaultImpl1	2497
BioReaction::StoichiometricTerm	2528
MetabolicReaction::StoichiometricTerm	2529
StochasticReactionRule::StoichiometricTerm	2529
SystemCommandExecutor_if	2555
SystemShellCommandExecutor	2563
SystemCommandResult	2556
SystemDependency	2556
SystemDependencyCheckEntry	2557
SystemDependencyCheckResult	2558
SystemDependencyInstallEntry	2558
SystemDependencyInstallResult	2559
SystemDependencyResolver	2560
SystemPreferences	2560
QCPCAxisPainterPrivate::TickLabelData	2564
Token	2565
TokenService	2565
TraceConsoleController	2565
TraceEvent	2567
TraceErrorEvent	2566
TraceSimulationEvent	2573
TraceSimulationProcess	2574
TraceManager	2568
TraitsGUI< T >	2575
TraitsGUI< GAssociation >	2576
TraitsGUI< GComponentPort >	2576
TraitsGUI< GConnection >	2576
TraitsGUI< GMainWindow >	2577
TraitsGUI< GModelComponent >	2577
TraitsGUI< GModelDataDefinition >	2578
TraitsGUI< GModelGraphicPic >	2578
TraitsGUI< GScene >	2579
TraitsGUI< GView >	2579
TraitsKernel< T >	2579
TraitsKernel< Collector_if >	2579
TraitsKernel< Model >	2580
TraitsKernel< ModelChecker_if >	2580
TraitsKernel< ModelComponent >	2580
TraitsKernel< ModelDataDefinition >	2581
TraitsKernel< Parser_if >	2581
TraitsKernel< Persistence_if >	2581
TraitsKernel< PluginConnector_if >	2581
TraitsKernel< Sampler_if >	2582
TraitsKernel< SimulationReporter_if >	2582
TraitsKernel< Simulator >	2582
TraitsKernel< Statistics_if >	2583
TraitsKernel< StatisticsDatafile_if >	2583
Util	2591
Waiting	2639
WaitingResource	2640
WebWorkerRuntime	??
QtButtonPropertyBrowserPrivate::WidgetItem	??
QtGroupBoxPropertyBrowserPrivate::WidgetItem	??
SimulatorSessionService::WorkerCapabilitiesResult	??
SimulatorSessionService::WorkerInfoResult	??
WorkerJob	??
SimulatorSessionService::WorkerJobCreationResult	??

SimulatorSessionService::WorkerJobInfoResult	??
WorkerJobManager	??
SimulatorSessionService::WorkerJobQueryResult	??
SimulatorSessionService::WorkerJobResultInfo	??
SimulatorSessionService::WorkerJobResultQueryResult	??
SimulatorSessionService::WorkerJobRunResult	??
WorkerJobTerminalResult	??

Chapter 7

Class Index

7.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Access	73
AddUndoCommand	80
AIAssistant	81
AIAssistantWindow	88
Airport_Extended1	89
AirportSecurityExample	91
AirportSecurityExampleExtended	93
AISupport	95
AND	101
AnElectronicAssemblyAndTestSystem	108
AnimationAttribute	110
AnimationCounter	111
AnimationEntity	112
AnimationEvent	114
AnimationExpression	115
AnimationPlaceholder	116
AnimationPlot	118
AnimationQueue	120
AnimationQueueDisplay	120
AnimationResource	122
AnimationStation	123
AnimationStatistics	124
AnimationTimer	126
AnimationTransition	127
AnimationVariable	128
ApiRouter	
Routes HTTP requests to Web API operations	130
Assembly_Line	131
Assign	133
Assignment	140
Attribute	141
BacteriaColony	
First GenESyS component for Gro-inspired bacteria colony simulation	148
BacteriaColonyViewerGuiExtensionPlugin	157
BacteriaSignalGrid	
Reusable discrete signal-field definition for bacteria colonies	158

BacteriaColony::BacteriumState	164
Banking_Transactions	165
BacteriaColony::BarrierSegment	167
BaseGenesysTerminalApplication	167
BaseGenesysWebApplication	
Entry point for the standalone Worker application flavor of Genesys	169
BaseMVController	
Compatibility base class that binds the Qt UI and the simulator model	171
Batch	172
BioCompartment	
Defines a biological compartment for biochemical and whole-cell models	179
BioNetwork	185
BioNetworkAwareGuiExtensionPlugin	192
BioNetworkEditorGuiExtensionPlugin	193
BioNetworkExecutionGuiExtensionPlugin	195
BioParameter	197
BioParameterManagerGuiExtensionPlugin	202
BioReaction	204
BioReactionKineticsGuiExtensionPlugin	216
BioReactionStoichiometryGuiExtensionPlugin	218
BioRunnerCommand	
Component wrapper over BioSimulatorRunner command execution	220
BioSBMLBridge	227
BioSBMLExportResult	228
BioSBMLImportResult	228
BioSBMLInteroperabilityGuiExtensionPlugin	228
BioSimulate	
Component that triggers deterministic BioNetwork simulation	230
BioSimulationResultsGuiExtensionPlugin	237
BioSimulatorRunner	239
BioSpecies	244
BioSpeciesManagerGuiExtensionPlugin	250
BioStateProjectionComponent	
Projects selected BioSpecies values into a WholeCellState	251
BioSteadyState	
Component that checks steady-state diagnostics over BioNetwork results	259
Book_Cap02_Example01	266
Boundary_Closed	268
Boundary_Fixed	270
BoundaryCondition	271
Buffer	273
BufferFIFO	280
QCPAxisPainterPrivate::CachedLabel	282
QCPLabelPainterPrivate::CachedLabel	282
ModelLifecycleController::Callbacks	282
SimulationEventController::Callbacks	283
Capacitor	283
Cell	290
CellCycleCheckpointComponent	
Evaluates lifecycle checkpoints and advances whole-cell step time	291
CellDivisionEvent	300
CellFateDecisionComponent	
Routes entities according to the current whole-cell lifecycle state	307
CellGrowthComponent	315
CellularAutomata	322
CellularAutomata_1DTimed	
Complete uniform CA specific for unidimensional lattices where 2nd dimension is time (usefull for elementary CA)	324

CellularAutomata_Classic	
Complete uniform CA	326
CellularAutomataBase	328
CellularAutomataComp	329
CellularAutomataDemoBuilder	337
CellularAutomataDemoModel	337
CellularAutomataDemoSettings	338
CellularAutomataEvent	338
CellularAutomataGuiExtensionPlugin	339
CellularAutomataLatticeWidget	341
CellularAutomataStateColorMap	342
CellularAutomataViewerController	342
CellularAutomataViewerWindow	344
Clone	344
CodeEditor	352
Collector_if	
Interface responsible for collecting observations of a stochastic variable	353
CollectorDatafile_if	
Collector interface specialized in file-backed value storage	356
CollectorDatafileDefaultImpl1	
Default file-backed implementation of CollectorDatafile_if	359
CollectorDefaultImpl1	
Default in-memory implementation of Collector_if	363
GroProgramRuntime::ColonyMutation	365
ComboBoxEnum	366
GroProgramIrr::Command	367
Util::CommandResult	367
CompartmentExchangeComponent	
Moves metabolite proxies between whole-cell regions	368
CppCompiler::CompilationResult	375
CompilerThread	376
ComponentItem	376
CompositeState	377
CompositeStateSet	379
Connection	381
ConnectionChannel	
Defines an input or output port that connects a ModelComponent to another	381
ConnectionManager	
Maintains outgoing connections from a component to downstream components	382
ContinuousModel	388
ContinuousSystemComponent	
Component that advances a continuous ODE system when entities arrive	390
CoreGuiExtensionPlugin	396
Counter	
Model datum used to accumulate event counts (or weighted counts)	398
CppCompiler	404
CppForG	410
CppModelExporter	
Service responsible for C++ code representation of the current simulation model	417
CppSerializer	418
Create	421
SimulatorSessionService::CreateSessionResult	
Contains session creation identifiers returned to API clients	429
QtBoolPropertyManagerPrivate::Data	430
QtDatePropertyManagerPrivate::Data	430
QtDateTimePropertyManagerPrivate::Data	431
QtDoublePropertyManagerPrivate::Data	431
QtEnumPropertyManagerPrivate::Data	431

QtFlagPropertyManagerPrivate::Data	432
QtIntPropertyManagerPrivate::Data	432
QtPointFPropertyManagerPrivate::Data	432
QtRectFPropertyManagerPrivate::Data	433
QtRectPropertyManagerPrivate::Data	433
QtSizeFPropertyManagerPrivate::Data	433
QtSizePropertyManagerPrivate::Data	434
QtStringPropertyManagerPrivate::Data	434
QtTimePropertyManagerPrivate::Data	434
DataAnalyzerWindow	435
DataComponentEditor	436
DataComponentProperty	437
DataDefinitionItem	438
DataDefinitionRelationItem	438
ModelInspectorController::DataDefinitionSelection	439
AnimationPlot::DataSeries	440
Decide	440
Delay::DEFAULT_VALUES	447
ModalModelDefault::DEFAULT_VALUES	
Attributes that should be loaded or saved with this component (Persistent Fields)	447
Seize::DEFAULT_VALUES	448
SourceModelComponent::DEFAULT_VALUES	448
SamplerDefaultImpl1::DefaultImpl1RNG_Parameters	
Parameter bundle for the default RNG algorithm	449
DefaultNode	450
DefaultNodeTransition	457
Delay	458
DeleteUndoCommand	465
dialogBreakpoint	466
DialogFind	467
DialogModelInformation	468
DialogPluginManager	469
DialogReportIssue	
Modal form used to collect a user-facing issue report before relay submission	470
DialogSelectCounter	471
DialogSelectVariable	472
DialogSimulationConfigure	473
DialogSystemPreferences	475
DialogTimerConfigure	476
DialogUtilityController	
Controller that orchestrates auxiliary dialogs and utility actions	477
DiffEquations	479
DiffusionField	486
DiffusionSimulate	
Component that triggers a DiffusionField solve when entities arrive	492
Diode	499
Dispose	506
DataAnalyzerWindow::DoePreviewPlot	513
DrawingItem	514
DropOff	514
DummyComponent	522
DummyElement	528
EditCommandController	
Controller for edit/clipboard/grouping command orchestration in the scene	533
EditorFactoryPrivate< Editor >	534
EFSMTransition	535
Enter	537
Entity	544

EntityGroup	553
EntityType	558
PersistenceRecord::Entry	
"Raw" entry, users should treat this as an opaque type since payloads could be in any format	564
EukaryoticCellCycleComponent	
Coordinates a didactic eukaryotic cell cycle over a shared WholeCellState	565
Event	573
Exact	575
Example_BasicOrderShipping	576
Example_PortModel	578
Example_PublicTransport	580
CompartmentExchangeComponent::ExchangeRule	581
GroProgramRuntime::ExecutionResult	582
Exit	582
ExperimentManager	
Manages higher-level simulation experiments	588
ExperimentManager_if	
Interface that coordinates execution of scenario-based simulation experiments	589
ExperimentManagerDefaultImpl1	591
ExpressionBuilder	593
ExpressionCategory	594
ExpressionParameter	594
ExpressionTemplate	594
Failure	595
File	601
FindLineEdit	607
Formula	608
FSMState	616
FtsZPolymerizationComponent	622
FullSimulationOfComplexModel	630
ParserManager::GenerateNewParserResult	
Result bundle for parser generation requests	632
GenesysApplication_if	633
genesys_mcp.client.GenesysClient	634
genesys_mcp.client.GenesysClientError	635
GenesysPropertyDescriptor	636
GenesysPropertyIntrospection	637
GenesysShell	638
GenesysShell_if	640
GeneticCircuit	641
GeneticCircuitGuiExtensionPlugin	647
GeneticCircuitPart	648
GeneticCircuitSimulate	654
GeneticCircuitSimulationSummary	660
GeneticExpressionStep	661
GeneticRegulation	668
GenSerializer	673
GraphicalAnimateExpression	677
GraphicalAssociation	677
GraphicalAutomaticPositioningStrategy	678
GraphicalAutomaticPositioningStrategyFactory	
Factory for the automatic positioning strategy selected from GUI preferences	679
GraphicalComponentPort	679
GraphicalConnection	
Graphical edge that represents model-level component connections	681
GraphicalConnectionStyle	684
GraphicalContextMenuController	
Builds context menus for the graphical model view using existing main-window actions	685

GraphicalDataDefinitionLayout	
Pure helper that computes arc placement for graphical data definitions	685
GraphicalDiagramConnection	686
GraphicalImageAnimation	687
GraphicalModelBuilder	
Service that reconstructs graphical components/connections from kernel model data	688
GraphicalModelComponent	
Visual component item that extends a data definition with ports and internal data	691
GraphicalModelDataDefinition	
Visual representation of a model data definition in the scene	696
GraphicalModelEvent	
Lightweight event envelope used to notify GUI changes in the graphics scene	698
GraphicalModelItemRenderContext	699
GraphicalModelItemRenderer	700
GraphicalModelItemRenderStrategy	700
GraphicalModelSerializer	
Persistence service for saving/loading graphical and textual model representations	701
GraphvizModelExporter	
Service that builds DOT content and renders model images for the GUI	703
ModelGraphicsScene::GRID	705
GroProgram	
Reusable Gro source program attached to a GenESyS model	705
GroProgramAst	
Minimal parsed representation of a Gro source program	711
GroProgramCompiler	
Converts parsed Gro AST into the first non-executable semantic IR	711
GroProgramIr	
Initial semantic representation compiled from GroProgramAst	712
GroProgramParser	
Plugin-side parser boundary for reusable Gro source programs	713
GroProgramRuntime	
Executes the supported Gro IR commands without simulator binding	713
GroProgramRuntimeState	
Mutable state used by the plugin-side Gro runtime helper	714
GroupItem	715
GroupUndoCommand	715
GuiActionContribution	716
GuiDockContribution	716
GuiExtensionConfigurationDialog	717
GuiExtensionConfigurationGuiExtensionPlugin	718
GuiExtensionManager	719
GuiExtensionPlugin	720
GuiExtensionPluginAutoRegistrar	721
GuiExtensionPluginCatalog	721
GuiExtensionPresentationMetadata	721
GuiExtensionRegistry	722
GuiExtensionRuntimeContext	722
GuiScopeTrace	
Emits a scoped trace entry for a GUI method and measures its runtime	723
GuiThemeManager	
Applies the visual theme used by the Qt application and model views	723
GuiWindowContribution	724
Half_Adder	725
DataAnalyzerWindow::HistogramPreview	727
HppLatticeGasState	728
HppLatticeGasStateSet	730
HttpRequest	
Lightweight representation of one parsed HTTP request	732

HttpResponse	
Minimal HTTP response payload produced by the web API router	733
IntegerStateSet	734
InternalEvent	737
IssueReportDraft	
User-authored issue report collected by the GUI form before transport	738
IssueReportRelayService	
HTTPS client service that sends issue reports to a secure relay endpoint	739
IssueReportRuntimeContext	
Runtime diagnostics captured by the GUI after the user approves submission	740
IssueReportSubmissionResult	
Relay submission outcome returned to the controller layer	740
Json	741
JsonSerializer	742
Label	745
QCPLabelPainterPrivate::LabelData	750
Lattice	
N-dimensional lattice	751
Leave	752
LicenceManager	759
LineNumberArea	760
List< T >	
Lightweight wrapper around <code>std::list</code> with convenience helpers used by the kernel	761
ListObservable< T >	
Observable-oriented list variant used by kernel structures	763
LocalRule	766
LocalRule_CopyCurrentState	768
LocalRule_Elementary	769
LocalRule_FlorestalFire	771
LocalRule_GameOfLife	772
LocalRule_Growty	774
LocalRule_HppLatticeGas	775
LocalRule_SandRock_Student	777
LocalRule_ShowNeighborhood	779
Loja01	780
LSODE	782
MainWindow	
Main Qt window that composes controllers/services and preserves legacy GUI entry points	789
MarkovChain	791
Match	798
MetabolicFluxBalance	805
MetabolicNetwork	812
MetabolicReaction	818
MetabolicStateProjectionComponent	
Projects flux-balance outputs into a WholeCellState	823
MetabolicSubmodelComponent	831
ModalModelDefault	838
ModalModelFSM	846
ModalModelPetriNet	852
Model	858
ModelChecker_if	869
ModelCheckerDefaultImpl1	872
ModelComponent	874
ModelComponentManager	
Owns and organizes the set of ModelComponent objects in a model	884
ModelDataDefinition	889
ModelDataDefinitionAssociations	905

ModelDataManager	
Central registry for ModelDataDefinition instances owned by a model	907
ModelEvent	912
ModelGraphicsScene	
Scene that owns and manipulates the graphical representation of a model	912
ModelGraphicsView	921
SimulatorSessionService::ModelImportResult	
Contains model import output and error state	924
ModelInfo	925
SimulatorSessionService::ModelInfoResult	
Describes the currently active model information for a session	929
ModelInspectorController	
Controller for model inspection views and scene-selection synchronization	929
ModelLanguageSynchronizer	
Service layer that synchronizes textual model language with kernel model state	930
ModelLifecycleController	
Controller for model/application lifecycle flows delegated from MainWindow	932
ModelManager	
The ModelManager class	934
SimulatorSessionService::ModelPersistenceResult	
Contains save/load model operation output and error state	940
ModelPluginInspectorGuiExtensionPlugin	941
ModelSimulation	
Controls the mature, currently functional simulation-execution layer of a model	943
MolecularSpecies	945
GroProgramRuntime::MotionMutation	950
MoveUndoCommand	950
dialogBreakpoint::MVCResult	951
GroProgramAst::NamedProgram	952
GroProgramIr::NamedProgramDefinition	952
NAND	952
NaturalStateSet	959
Neighborhood	
Defines the complete neighborhood of a cell	961
Neighborhood_Center	962
Neighborhood_Moore	964
Neighborhood_VonNeumann	966
ParserManager::NewParser	
Paths associated with a generated parser artifact set	968
NMOS	968
NOR	975
NOT	982
ObjectPropertyBrowser	989
OctaveRunner	
Model data definition that executes Octave code through Rscript	992
ODEfunction	998
ODESolver	999
OLD_ODEelement	1005
OnEventManager	1010
OperatingSystem02	1011
OperatingSystem03	1013
OptimizerWindow	1015
OR	1016
Parser_if	
Interface for evaluating model expressions and stochastic functions	1023
ParserChangesInformation	1026
ParserDefaultImpl1	1026
ParserDefaultImpl2	1029

ParserManager	1032
PasteUndoCommand	1033
PathwayStressResponseComponent	
Applies sustained pathway-collapse responses to a whole-cell state	1034
Persistence_if	
Interface for model serialization/deserialization services	1042
PersistenceDefaultImpl2	1045
PersistencePartialLoadImpl1	1047
PersistenceRecord	
Serialization/Deserialization structure which behaves like an associative array	1049
PersistentObject_base	
Dynamic serializable interface	1051
PetriPlace	1054
PetriTransition	1060
PickableStationItem	1062
PickStation	1062
PickUp	1070
Plugin	1076
PluginCatalogController	
Controller responsible for plugin catalog presentation and plugin palette behavior	1079
PluginConnector_if	
Interface for dynamic discovery/loading/unloading of plugin libraries	1080
PluginConnectorDummyImpl1	
In-memory plugin connector that resolves built-in plugin metadata	1083
PluginConnectorStaticImpl1	
Static plugin connector stub used by the kernel interface	1086
PluginInformation	1089
PluginInsertionOptions	
Optional policy hooks used while inserting a plugin	1090
PluginLoadIssue	
Diagnostic record for a plugin candidate that could not be loaded	1091
PluginManager	
Manages discovery, loading, lookup and instantiation of simulation plugins	1092
PMOS	1094
GroProgramRuntime::PopulationMutation	1101
Process	1101
OptimizerWindow::ProgressPlot	1109
BioStateProjectionComponent::Projection	1110
MetabolicStateProjectionComponent::Projection	1110
PropertyEditor	1111
PropertyEditorController	
Controller that orchestrates property editor state and scene-selection coupling	1112
PropertyEditorGenesys	1113
PropertyGenesys	1113
PropertyGenesysBase	
Base metadata for kernel-side control/response abstractions	1115
PythonForG	1117
PythonRuntime	1123
QCPAbstractItem	
The abstract base class for all items in a plot	1129
QCPAbstractLegendItem	
The abstract base class for all entries in a QCPLegend	1139
QCPAbstractPaintBuffer	
The abstract base class for paint buffers, which define the rendering backend	1146
QCPAbstractPlottable	
The abstract base class for all data representing objects in a plot	1150
QCPAbstractPlottable1D< DataType >	
A template base class for plottables with one-dimensional data	1165

QCPAxis	
Manages a single axis inside a QCustomPlot	1174
QCPAxisPainterPrivate	1201
QCPAxisRect	
Holds multiple axes and arranges them in a rectangular shape	1203
QCPAxisTicker	
The base class tick generator used by QCPAxis to create tick positions and tick labels	1223
QCPAxisTickerDateTime	
Specialized axis ticker for calendar dates and times as axis ticks	1228
QCPAxisTickerFixed	
Specialized axis ticker with a fixed tick step	1235
QCPAxisTickerLog	
Specialized axis ticker suited for logarithmic axes	1238
QCPAxisTickerPi	
Specialized axis ticker to display ticks in units of an arbitrary constant, for example pi	1241
QCPAxisTickerText	
Specialized axis ticker which allows arbitrary labels at specified coordinates	1245
QCPAxisTickerTime	
Specialized axis ticker for time spans in units of milliseconds to days	1251
QCPBars	
A plottable representing a bar chart in a plot	1256
QCPBarsData	
Holds the data of one single data point (one bar) for QCPBars	1268
QCPBarsGroup	
Groups multiple QCPBars together so they appear side by side	1271
QCPColorGradient	
Defines a color gradient for use with e.g. QCPColorMap	1276
QCPColorMap	
A plottable representing a two-dimensional color map in a plot	1283
QCPColorMapData	
Holds the two-dimensional data of a QCPColorMap plottable	1295
QCPColorScale	
A color scale for use with color coding data such as QCPColorMap	1303
QCPColorScaleAxisRectPrivate	1313
QCPCurve	
A plottable representing a parametric curve in a plot	1320
QCPCurveData	
Holds the data of one single data point for QCPCurve	1331
QCPDataContainer< DataType >	
The generic data container for one-dimensional plottables	1334
QCPDataRange	
Describes a data range given by begin and end index	1344
QCPDataSelection	
Describes a data set by holding multiple QCPDataRange instances	1348
QCPErrorBars	
A plottable that adds a set of error bars to other plottables	1356
QCPErrorBarsData	
Holds the data of one single error bar for QCPErrorBars	1369
QCPFinancial	
A plottable representing a financial stock chart	1370
QCPFinancialData	
Holds the data of one single data point for QCPFinancial	1384
QCPGraph	
A plottable representing a graph in a plot	1386
QCPGraphData	
Holds the data of one single data point for QCPGraph	1398
QCPGrid	
Responsible for drawing the grid of a QCPAxis	1401

QCItemAnchor	An anchor of an item to which positions can be attached to	1405
QCItemBracket	A bracket for referencing/highlighting certain parts in the plot	1408
QCItemCurve	A curved line from one point to another	1415
QCItemEllipse	An ellipse	1421
QCItemLine	A line from one point to another	1428
QCItemPixmap	An arbitrary pixmap	1434
QCItemPosition	Manages the position of an item	1440
QCItemRect	A rectangle	1448
QCItemStraightLine	A straight line that spans infinitely in both directions	1454
QCItemText	A text label	1459
QCItemTracer	Item that sticks to QCPGraph data points	1468
QCLabelPainterPrivate		1476
QCLayer	A layer that may contain objects, to control the rendering order	1479
QCLayerable	Base class for all drawable objects	1484
QCLayout	The abstract base class for layouts	1492
QCLayoutElement	The abstract base class for all objects that form the layout system	1500
QCLayoutGrid	A layout that arranges child elements in a grid	1510
QCLayoutInset	A layout that places child elements aligned to the border or arbitrarily positioned	1525
QCLegend	Manages a legend inside a QCustomPlot	1534
QCLineEnding	Handles the different ending decorations for line-like items	1550
QCPMarginGroup	A margin group allows synchronization of margin sides if working with multiple layout elements	1553
QCPPaintBufferPixmap	A paint buffer based on QPixmap, using software raster rendering	1556
QCPPainter	QPainter subclass used internally	1559
QCPPlottableInterface1D	Defines an abstract interface for one-dimensional plottables	1564
QCPPlottableLegendItem	A legend item representing a plottable with an icon and the plottable name	1568
QCPPolarAxisAngular	The main container for polar plots, representing the angular axis as a circle	1574
QCPPolarAxisRadial	The radial axis inside a radial plot	1601
QCPPolarGraph	A radial graph used to display data in polar plots	1624
QCPPolarGrid	The grid in both angular and radial dimensions for polar plots	1635

QCPPolarLegendItem	
A legend item for polar plots	1639
QCPRange	
Represents the range an axis is encompassing	1644
QCPScatterStyle	
Represents the visual appearance of scatter points	1652
QCPSelectionDecorator	
Controls how a plottable's data selection is drawn	1659
QCPSelectionDecoratorBracket	
A selection decorator which draws brackets around each selected data segment	1663
QCPSelectionRect	
Provides rect/rubber-band data selection and range zoom interaction	1668
QCPStatisticalBox	
A plottable representing a single statistical box in a plot	1674
QCPStatisticalBoxData	
Holds the data of one single data point for QCPStatisticalBox	1685
QCPTextElement	
A layout element displaying a text	1688
QCPVector2D	
Represents two doubles as a mathematical 2D vector	1698
QCustomPlot	
The central class of the library. This is the QWidget which displays the plot and interacts with the user	1705
QtAbstractEditorFactory< PropertyManager >	1744
QtAbstractEditorFactoryBase	1748
QtAbstractPropertyBrowser	1750
QtAbstractPropertyBrowserPrivate	1757
QtAbstractPropertyManager	1759
QtAbstractPropertyManagerPrivate	1765
QtBoolEdit	1766
QtBoolPropertyManager	1767
QtBoolPropertyManagerPrivate	1770
QtBrowserItem	1771
QtBrowserItemPrivate	1772
QtButtonPropertyBrowser	1773
QtButtonPropertyBrowserPrivate	1776
QtCharEdit	1777
QtCharEditorFactory	1778
QtCharEditorFactoryPrivate	1780
QtCharPropertyManager	1782
QtCharPropertyManagerPrivate	1785
QtCheckBoxFactory	1786
QtCheckBoxFactoryPrivate	1788
QtColorEditorFactory	1789
QtColorEditorFactoryPrivate	1791
QtColorEditWidget	1793
QtColorPropertyManager	1794
QtColorPropertyManagerPrivate	1798
QtCursorDatabase	1799
QtCursorEditorFactory	1799
QtCursorEditorFactoryPrivate	1801
QtCursorPropertyManager	1802
QtCursorPropertyManagerPrivate	1806
QtDateEdit	1806
QtDateEditFactory	1808
QtDateEditFactoryPrivate	1810
QtDatePropertyManager	1811
QtDatePropertyManagerPrivate	1817

QtDateTimeEdit	1817
QtDateTimeEditFactory	1818
QtDateTimeEditFactoryPrivate	1820
QtDateTimePropertyManager	1822
QtDateTimePropertyManagerPrivate	1825
QtDoubleEdit	1826
QtDoublePropertyManager	1827
QtDoublePropertyManagerPrivate	1834
QtDoubleSpinBoxFactory	1834
QtDoubleSpinBoxFactoryPrivate	1836
QtEnumEditorFactory	1838
QtEnumEditorFactoryPrivate	1840
QtEnumPropertyManager	1841
QtEnumPropertyManagerPrivate	1847
QtEnumPropertyType	1847
QtFlagPropertyManager	1847
QtFlagPropertyManagerPrivate	1852
QtFlagPropertyType	1853
QtFontEditorFactory	1853
QtFontEditorFactoryPrivate	1855
QtFontEditWidget	1857
QtFontPropertyManager	1858
QtFontPropertyManagerPrivate	1862
QtGroupBoxPropertyBrowser	1864
QtGroupBoxPropertyBrowserPrivate	1867
QtGroupPropertyManager	1867
QtGroupPropertyType	1870
QtIntEdit	1870
QtIntPropertyManager	1871
QtIntPropertyManagerPrivate	1877
QtKeySequenceEditorFactory	1878
QtKeySequenceEditorFactoryPrivate	1880
QtKeySequencePropertyManager	1882
QtKeySequencePropertyManagerPrivate	1885
QtLineEditFactory	1886
QtLineEditFactoryPrivate	1888
QtLocalePropertyManager	1889
QtLocalePropertyManagerPrivate	1893
QtMetaEnumProvider	1894
QtMetaEnumWrapper	1894
QtPointFPropertyManager	1895
QtPointFPropertyManagerPrivate	1900
QtPointPropertyManager	1901
QtPointPropertyManagerPrivate	1905
QtProperty	1906
QtPropertyBrowserUtils	1913
QtPropertyEditorDelegate	1913
QtPropertyEditorView	1914
QtPropertyPrivate	1915
QtRectFPropertyManager	1916
QtRectFPropertyManagerPrivate	1922
QtRectPropertyManager	1923
QtRectPropertyManagerPrivate	1927
QtScrollBarFactory	1929
QtScrollBarFactoryPrivate	1931
QtSizeFPropertyManager	1932
QtSizeFPropertyManagerPrivate	1939
QtSizePolicyPropertyManager	1940

QtSizePolicyPropertyManagerPrivate	1944
QtSizePropertyManager	1945
QtSizePropertyManagerPrivate	1950
QtSliderFactory	1951
QtSliderFactoryPrivate	1953
QtSpinBoxFactory	1955
QtSpinBoxFactoryPrivate	1957
QtStringEdit	1958
QtStringPropertyManager	1959
QtStringPropertyManagerPrivate	1963
QtTimeEdit	1964
QtTimeEditFactory	1965
QtTimeEditFactoryPrivate	1967
QtTimePropertyManager	1969
QtTimePropertyManagerPrivate	1972
QtTreePropertyBrowser	1973
QtTreePropertyBrowserPrivate	1979
QtVariantEditorFactory	1980
QtVariantEditorFactoryPrivate	1982
QtVariantProperty	1983
QtVariantPropertyManager	1987
QtVariantPropertyManagerPrivate	1996
QtVariantPropertyPrivate	1997
Queue	1998
QueueableItem	2004
RealIntervalStateSet	2005
RealState	2007
Record	2009
Rectifier	2017
Release	2019
Remove	2026
Resistor	2034
Resource	2040
ResourceAllocationComponent	2047
GroProgramParser::Result	2054
Sampler_if::RNG_Parameters	
Encapsulates generator-specific configuration/state parameters	2055
Route	2056
RSimulator	
Component that sends configured R commands to an internal RSimulatorRunner	2063
RSimulatorRunner	
Model data definition that executes R code through Rscript	2070
Sampler_if	
Abstraction for random-number generation and probability-distribution sampling	2076
SamplerDefaultImpl1	
Default pseudo-random sampler implementation used by the kernel	2082
ScenarioExperiment_if	2088
SceneToolController	
Controller for scene visualization, drawing tools, alignment, and zoom orchestration	2088
SchedulableItem	2092
Schedule	2092
Search	2098
SeizableItem	2105
Seize	2107
Separate	2114
Sequence	2122
SequenceStep	2127

Serializer	
Serialization/Deserialization interface for simulation models	2129
SessionContext	
Owns the state that belongs to one authenticated web session	2132
SessionManager	
Creates, stores and resolves authenticated web sessions	2133
Set	2135
Signal	2142
SignalData	2149
GroProgramRuntime::SignalMutation	2154
SimpleHttpServer	
Minimal blocking HTTP/1.1 server for single-request-per-connection handling	2154
SimulatorSessionService::SimulationActionResult	
Contains the result of a simulation action such as run or step	2155
SimulationCommandController	
Controller that orchestrates UI simulation commands over SimulationController guards	2156
SimulatorSessionService::SimulationConfigInput	
Defines user-provided simulation parameters to apply to the current model	2157
SimulatorSessionService::SimulationConfigResult	
Contains the result of applying simulation configuration	2158
SimulationControl	
Read/write kernel-side simulation control abstraction	2159
SimulationControlBool	2162
SimulationControlDouble	2165
SimulationControlGeneric< T >	2168
SimulationControlGenericClass< T, M, C >	2172
SimulationControlGenericClassNotDC< T, M, C >	2177
SimulationControlGenericEnum< T, E >	2182
SimulationControlGenericList< T, M, C >	2185
SimulationControlGenericListPointer< T, M, C >	2189
SimulationControlInt	2194
SimulationController	
Controller that validates simulation command readiness for MainWindow wrappers	2196
SimulationControlSourceCodeString	2199
SimulationControlString	2203
SimulationControlTimeUnit	2206
SimulationControlUInt	2209
SimulationControlUShort	2212
SimulationEvent	2215
SimulationEventController	
Controller for simulation event callbacks, UI updates, and handler wiring	2216
SimulationExperiment	
Represents a higher-level simulation experiment definition	2217
SimulationReporter_if	
Interface for reporting simulation results and experiment-facing outputs	2218
SimulationReporterDefaultImpl1	2219
SimulationResponse	
Read-only kernel-side simulation response abstraction	2222
SimulationResponseDouble	2224
SimulationScenario	
Represents a candidate simulation scenario within the unfinished experiment layer	2225
SimulatorSessionService::SimulationStatusResult	
Captures the current simulation execution state	2227
Simulator	2227
SimulatorFacade	
High-level non-owning facade over Simulator and its main composed services	2230
SimulatorSessionService::SimulatorInfoResult	
Describes simulator identity metadata for a token-scoped session	2236

SimulatorSessionService	
Encapsulates session-scoped simulator operations exposed by the Web API	2237
SinkModelComponent	2247
Smart_AddingResource	2253
Smart_AlternatingEntityCreation	2255
Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime	2257
Smart_ArrivalsEntityTypeVsAttribute	2259
Smart_AssignExample	2261
Smart_AssignWriteSeizes	2263
Smart_Attribute_Variable	2265
Smart_AutomaticStatisticsCollection	2267
Smart_BacteriaColony_GRO	2269
Smart_BasicModeling	2271
Smart_BatchAndSeparate	2273
Smart_BatchSeparate	2275
Smart_BioKineticLawRegulation	2277
Smart_BioNetworkAnalysis	2279
Smart_BioReversibleKineticLaw	2281
Smart_BioReversibleMassAction	2283
Smart_Buffer	2285
Smart_CellularAutomata	2287
Smart_CellularAutomata1	2289
Smart_Clone	2291
Smart_ContinuousFlowEntities	2293
Smart_CppForG	2295
Smart_Create	2297
Smart_DecideNWayByChance	2299
Smart_DefaultModalModel	2301
Smart_DefiningAttributesAsStrings	2303
Smart_DefiningControlLogic	2305
Smart_DefiningResourceCapacity	2307
Smart_Delay	2309
Smart_DelayBasedOnReplication	2311
Smart_Dummy	2313
Smart_EntitiesProcessedByPriority	2315
Smart_EvaluatingConditionsBeforeEnteringQueue	2317
Smart_Expression	2319
Smart_Failures	2321
Smart_FSMState	2323
Smart_GroColonyGrowth	2325
Smart_GroColonyLifecycle	2327
Smart_HoldSearchRemove	2329
Smart_InventoryAndHoldingCosts	2331
Smart_LSODE	2333
Smart_MarkovChain	2335
Smart_MaxArrivalsField	2337
Smart_ModalModel	2339
Smart_ModalModelDefaultNetwork	2341
Smart_ModalModelFSM	2343
Smart_ModalModelPetriNet	2345
Smart_ModelInfoModelSimulation	2347
Smart_ModelRunUntil1000Parts	2349
Smart_ModuleDisplayVariables	2351
Smart_ODE	2353
Smart_OnEvent	2355
Smart_OverlappingResources	2358
Smart_ParallelProcessingOfEntities	2360
Smart_Parser	2362

Smart_ParserModelFunctions	2364
Smart_PetriPlace	2366
Smart_PlacingEntitiesInQueueSets	2368
Smart_Plugin	2370
Smart_PriorityExample	2372
Smart_Process	2374
Smart_ProcessArena	2376
Smart_ProcessSet	2378
Smart_PythonForG	2380
Smart_R_Simulator	
Builds a minimal Create -> RSimulator -> Dispose model	2381
Smart_Record	2384
Smart_Record_Arena	2386
Smart_RemovingAndReorderingEntitiesInAQueue	2388
Smart_ResourceCosting	2390
Smart_ResourceScheduleCosting	2392
Smart_ResourceSets	2394
Smart_RouteStation	2396
Smart_SeizeDelayRelease	2398
Smart_SeizeDelayReleaseMany	2400
Smart_SeizeDelayReleaseNoDataDefs	2402
Smart_SeizingMultipleSimultaneously	2404
Smart_SelectingRouteBasedOnProbability	2406
Smart_SelectingShorterQueue	2408
Smart_Sequence	2410
Smart_SimulationControlResponse	2412
Smart_SynchronizingParallelEntities	2414
Smart_ValueAdded	2416
Smart_WaitForSignal	2418
Smart_WaitScanCondition	2420
Smart_WaitSignal	2422
WebWorkerRuntime::Snapshot	
Captures a thread-safe snapshot of the worker state	2423
SortFile	
Utility class that sorts numeric values persisted in a data file	2424
SourceCodeString	2425
SourceModelComponent	2425
SparseValueStore	
Sparse N-dimensional numeric store using GenESyS textual index keys	2437
SPIECircuit	2438
SPICENode	2445
SPICERunner	2451
Start	2456
StartAIAssistant	2463
State	2464
GroProgramAst::Statement	2465
StateSet	2465
StateSet_Enumerable	2467
Station	2469
Statistics_if	
Interface for statistical synthesis over values provided by a Collector_if	2475
StatisticsCollector	2480
StatisticsDatafile_if	
Extension of Statistics_if for metrics that require full data access	2485
StatisticsDatafileDefaultImpl1	
Default implementation of StatisticsDatafile_if for file-backed samples	2490
StatisticsDefaultImpl1	
Default online statistics implementation over a Collector_if source	2497

StochasticReactionComponent	2501
StochasticReactionRule	2508
StochasticTranscription	2514
StochasticTranslation	2521
BioReaction::StoichiometricTerm	2528
MetabolicReaction::StoichiometricTerm	2529
StochasticReactionRule::StoichiometricTerm	2529
Stop	2529
Storage	2536
Store	2541
Submodel	2548
SystemCommandExecutor_if	
Minimal command execution interface used to make system dependency checks testable	2555
SystemCommandResult	
Captures the result of executing a system command used by dependency checks	2556
SystemDependency	2556
SystemDependencyCheckEntry	
One dependency entry produced by a system dependency preflight	2557
SystemDependencyCheckResult	
Full preflight result for the system dependencies declared by one plugin	2558
SystemDependencyInstallEntry	
One command result from an attempted system dependency installation	2558
SystemDependencyInstallResult	
Aggregates commands executed while trying to satisfy missing system dependencies	2559
SystemDependencyResolver	
Evaluates and optionally installs plugin system dependencies	2560
SystemPreferences	
Persistent application preferences used by the Qt GUI	2560
SystemShellCommandExecutor	
Shell-based executor used by production plugin dependency preflight	2563
QCPAxisPainterPrivate::TickLabelData	2564
Token	2565
TokenService	2565
TraceConsoleController	
Controller that routes simulator trace events to GUI text widgets	2565
TraceErrorEvent	2566
TraceEvent	2567
TraceManager	2568
TraceSimulationEvent	2573
TraceSimulationProcess	2574
TraitsGUI< T >	
GUI-specific trait constants used to tune default rendering behavior	2575
TraitsGUI< GAssociation >	2576
TraitsGUI< GComponentPort >	2576
TraitsGUI< GConnection >	2576
TraitsGUI< GMainWindow >	2577
TraitsGUI< GModelComponent >	2577
TraitsGUI< GModelDataDefinition >	2578
TraitsGUI< GModelGraphicPic >	2578
TraitsGUI< GScene >	2579
TraitsGUI< GView >	2579
TraitsKernel< T >	2579
TraitsKernel< Collector_if >	2579
TraitsKernel< Model >	2580
TraitsKernel< ModelChecker_if >	2580
TraitsKernel< ModelComponent >	2580
TraitsKernel< ModelDataDefinition >	2581
TraitsKernel< Parser_if >	2581

TraitsKernel< Persistence_if >	2581
TraitsKernel< PluginConnector_if >	2581
TraitsKernel< Sampler_if >	2582
TraitsKernel< SimulationReporter_if >	2582
TraitsKernel< Simulator >	2582
TraitsKernel< Statistics_if >	2583
TraitsKernel< StatisticsDatafile_if >	2583
UngroupUndoCommand	2583
Unstore	2584
Util	2591
Variable	2603
Vpulse	2611
Vsine	2618
Vsource	2625
Wait	2632
Waiting	2639
WaitingResource	2640
WcmBioNetworkBridge	
Demonstrates a minimal deterministic/stochastic whole-cell bridge	2641
WcmCompartmentTransportWholeCellMvp	
Whole-cell MVP with compartment-aware metabolic transport	2643
WcmFluxBalanceWholeCellMvp	
Whole-cell MVP driven by flux-balance metabolism	2645
WcmGeneExpression	2647
WcmLifecycleStress	
Demonstrates starvation and death routing in a whole-cell stress scenario	2649
WcmMgenitaliumKarr2012	2652
WcmPathwayStressWholeCellMvp	
Whole-cell stress example driven by sustained low pathway activity	2654
WcmSimpleSsa	2656
WcmWholeCellMvp	
Minimal whole-cell MVP combining metabolism, expression, growth, and division	??
WcmYeastDidacticWholeCellMvp	
Didactic yeast-oriented whole-cell MVP built on GEM-inspired metabolism	??
WcmYeastEukaryoticWholeCellMvp	
Didactic eukaryotic whole-cell MVP inspired by budding yeast	??
WebWorkerDialog	
Lightweight control window for the embedded GenESyS web worker	??
WebWorkerRuntime	
Owns the embedded worker runtime used by the GUI and the standalone worker app	??
WholeCellState	??
QtButtonPropertyBrowserPrivate::WidgetItem	??
QtGroupBoxPropertyBrowserPrivate::WidgetItem	??
SimulatorSessionService::WorkerCapabilitiesResult	
Describes currently supported worker features	??
SimulatorSessionService::WorkerInfoResult	
Describes worker identity metadata for distributed discovery	??
WorkerJob	
Stores metadata for a worker job snapshot owned by the web worker	??
SimulatorSessionService::WorkerJobCreationResult	
Contains worker job creation output and error state	??
SimulatorSessionService::WorkerJobInfoResult	
Describes worker job metadata exposed by worker stage 3 endpoints	??
WorkerJobManager	
Owns worker job records in memory and assigns unique identifiers	??
SimulatorSessionService::WorkerJobQueryResult	
Contains worker job lookup output and error state	??

SimulatorSessionService::WorkerJobResultInfo	
Describes terminal worker job result data exposed by worker stage 5 endpoint	??
SimulatorSessionService::WorkerJobResultQueryResult	
Contains worker job terminal result lookup output and error state	??
SimulatorSessionService::WorkerJobRunResult	
Contains worker job run output and error state	??
WorkerJobTerminalResult	
Stores the terminal simulation summary persisted for a finished or failed worker job	??
Write	??
XmlSerializer	??
XNOR	??
XOR	??

Chapter 8

File Index

8.1 File List

Here is a list of all documented files with brief descriptions:

applications/ BaseGenesysTerminalApplication.h	??
applications/ GenesysApplication_if.h	??
applications/gui/ai_assistant/ AIAssistantWindow.h	??
applications/gui/genesys/ BaseMVController.h	??
applications/gui/genesys/ GuiCrashDiagnostics.h	??
applications/gui/genesys/ GuiScopeTrace.h	??
applications/gui/genesys/ guithememanager.h	??
applications/gui/genesys/ mainwindow.cpp	
Composition-root partition of MainWindow implementation	??
applications/gui/genesys/ mainwindow.h	??
applications/gui/genesys/ mainwindow_controller.cpp	
Controller-delegation partition of MainWindow implementation	??
applications/gui/genesys/ mainwindow_modelrepresentations.cpp	
Model representation/persistence wrapper partition of MainWindow implementation	??
applications/gui/genesys/ mainwindow_scene.cpp	
Scene callback and scene synchronization partition of MainWindow implementation	??
applications/gui/genesys/ mainwindow_simulator.cpp	
Simulator trace/event wrapper partition of MainWindow implementation	??
applications/gui/genesys/ MovedGuiCompatibilityIncludes.h	??
applications/gui/genesys/ qcustomplot.h	??
applications/gui/genesys/ systempreferences.h	??
applications/gui/genesys/ TraitsGUI.h	??
applications/gui/genesys/ UtilGUI.h	??
applications/gui/genesys/actions/ AddUndoCommand.h	??
applications/gui/genesys/actions/ DeleteUndoCommand.h	??
applications/gui/genesys/actions/ GroupUndoCommand.h	??
applications/gui/genesys/actions/ MoveUndoCommand.h	??
applications/gui/genesys/actions/ PasteUndoCommand.h	??
applications/gui/genesys/actions/ UngroupUndoCommand.h	??
applications/gui/genesys/animations/ AnimationCounter.h	??
applications/gui/genesys/animations/ AnimationPlaceholder.h	??
applications/gui/genesys/animations/ AnimationQueue.h	??
applications/gui/genesys/animations/ AnimationTimer.h	??
applications/gui/genesys/animations/ AnimationTransition.h	??
applications/gui/genesys/animations/ AnimationVariable.h	??

applications/gui/genesys/cellular_automata/CellularAutomataDemoBuilder.h	??
applications/gui/genesys/cellular_automata/CellularAutomataLatticeWidget.h	??
applications/gui/genesys/cellular_automata/CellularAutomataStateColorMap.h	??
applications/gui/genesys/cellular_automata/CellularAutomataViewerController.h	??
applications/gui/genesys/cellular_automata/CellularAutomataViewerWindow.h	??
applications/gui/genesys/codeeditor/CodeEditor.h	??
applications/gui/genesys/codeeditor/LineNumberArea.h	??
applications/gui/genesys/controllers/DialogUtilityController.h	??
applications/gui/genesys/controllers/EditCommandController.h	??
applications/gui/genesys/controllers/GraphicalContextMenuController.h	??
applications/gui/genesys/controllers/ModelInspectorController.h	??
applications/gui/genesys/controllers/ModelLifecycleController.h	??
applications/gui/genesys/controllers/PluginCatalogController.h	??
applications/gui/genesys/controllers/PropertyEditorController.h	??
applications/gui/genesys/controllers/SceneToolController.h	??
applications/gui/genesys/controllers/SimulationCommandController.h	??
applications/gui/genesys/controllers/SimulationController.h	??
applications/gui/genesys/controllers/SimulationEventController.h	??
applications/gui/genesys/controllers/TraceConsoleController.h	??
applications/gui/genesys/dialogs/dialogBreakpoint.h	??
applications/gui/genesys/dialogs/DialogFind.h	??
applications/gui/genesys/dialogs/Dialogmodelinformation.h	??
applications/gui/genesys/dialogs/dialogpluginmanager.h	??
applications/gui/genesys/dialogs/DialogReportIssue.h	??
applications/gui/genesys/dialogs/DialogSelectCounter.h	??
applications/gui/genesys/dialogs/DialogSelectVariable.h	??
applications/gui/genesys/dialogs/dialogsimulationconfigure.h	??
applications/gui/genesys/dialogs/dialogsystempreferences.h	??
applications/gui/genesys/dialogs/DialogTimerConfigure.h	??
applications/gui/genesys/extensions/GuiExtensionConfigurationDialog.h	??
applications/gui/genesys/extensions/GuiExtensionContracts.h	??
applications/gui/genesys/extensions/GuiExtensionManager.h	??
applications/gui/genesys/extensions/GuiExtensionPluginCatalog.h	??
applications/gui/genesys/graphics/GraphicalAnimateExpression.h	??
applications/gui/genesys/graphics/GraphicalAssociation.h	??
applications/gui/genesys/graphics/GraphicalComponentPort.h	??
applications/gui/genesys/graphics/GraphicalConnection.h	??
applications/gui/genesys/graphics/GraphicalConnectionStyle.h	??
applications/gui/genesys/graphics/GraphicalDiagramConnection.h	??
applications/gui/genesys/graphics/GraphicalImageAnimation.h	??
applications/gui/genesys/graphics/GraphicalModelComponent.h	??
applications/gui/genesys/graphics/GraphicalModelDataDefinition.h	??
applications/gui/genesys/graphics/GraphicalModelItemRenderStrategy.h	??
applications/gui/genesys/graphics/ModelGraphicsScene.h	??
applications/gui/genesys/graphics/ModelGraphicsView.h	??
applications/gui/genesys/propertyeditor/ComboBoxEnum.h	??
applications/gui/genesys/propertyeditor/DataComponentEditor.h	??
applications/gui/genesys/propertyeditor/DataComponentProperty.h	??
applications/gui/genesys/propertyeditor/ObjectPropertyBrowser.h	??
applications/gui/genesys/propertyeditor/PropertyEditor.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qtbuttonpropertybrowser.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qtgroupboxpropertybrowser.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qtreepropertybrowser.h	??
applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.h	??

applications/gui/genesys/services/CppModelExporter.h	??
applications/gui/genesys/services/GraphicalAutomaticPositioningStrategy.h	??
applications/gui/genesys/services/GraphicalDataDefinitionLayout.h	??
applications/gui/genesys/services/GraphicalModelBuilder.h	??
applications/gui/genesys/services/GraphicalModelSerializer.h	??
applications/gui/genesys/services/GraphvizModelExporter.h	??
applications/gui/genesys/services/IssueReportRelayService.h	??
applications/gui/genesys/services/ModelLanguageSynchronizer.h	??
applications/gui/genesys/services/ToolLauncher.h	??
applications/gui/genesys/tools/dataanalyzer/DataAnalyzerWindow.h	??
applications/gui/genesys/tools/expressionbuilder/ExpressionBuilder.h	??
applications/gui/genesys/tools/optimizer/OptimizerWindow.h	??
applications/gui/httpworker/WebWorkerDialog.h	??
applications/modelSpecific/arenaExamples/Airport_Extended1.h	??
applications/modelSpecific/arenaExamples/AirportSecurityExample.h	??
applications/modelSpecific/arenaExamples/AirportSecurityExampleExtended.h	??
applications/modelSpecific/arenaExamples/Assembly_Line.h	??
applications/modelSpecific/arenaExamples/Banking_Transactions.h	??
applications/modelSpecific/arenaExamples/Example_BasicOrderShipping.h	??
applications/modelSpecific/arenaExamples/Example_PortModel.h	??
applications/modelSpecific/arenaExamples/Example_PublicTransport.h	??
applications/modelSpecific/arenaSmarts/Smart_AddingResource.h	??
applications/modelSpecific/arenaSmarts/Smart_AlternatingEntityCreation.h	??
applications/modelSpecific/arenaSmarts/Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime.h	??
applications/modelSpecific/arenaSmarts/Smart_ArrivalsEntityTypeVsAttribute.h	??
applications/modelSpecific/arenaSmarts/Smart_AssignExample.h	??
applications/modelSpecific/arenaSmarts/Smart_AutomaticStatisticsCollection.h	??
applications/modelSpecific/arenaSmarts/Smart_BasicModeling.h	??
applications/modelSpecific/arenaSmarts/Smart_BatchAndSeparate.h	??
applications/modelSpecific/arenaSmarts/Smart_ContinuousFlowEntities.h	??
applications/modelSpecific/arenaSmarts/Smart_Create.h	??
applications/modelSpecific/arenaSmarts/Smart_DecideNWayByChance.h	??
applications/modelSpecific/arenaSmarts/Smart_DefiningAttributesAsStrings.h	??
applications/modelSpecific/arenaSmarts/Smart_DefiningControlLogic.h	??
applications/modelSpecific/arenaSmarts/Smart_DefiningResourceCapacity.h	??
applications/modelSpecific/arenaSmarts/Smart_DelayBasedOnReplication.h	??
applications/modelSpecific/arenaSmarts/Smart_EntitiesProcessedByPriority.h	??
applications/modelSpecific/arenaSmarts/Smart_EvaluatingConditionsBeforeEnteringQueue.h	??
applications/modelSpecific/arenaSmarts/Smart_Expression.h	??
applications/modelSpecific/arenaSmarts/Smart_InventoryAndHoldingCosts.h	??
applications/modelSpecific/arenaSmarts/Smart_MaxArrivalsField.h	??
applications/modelSpecific/arenaSmarts/Smart_ModelRunUntil1000Parts.h	??
applications/modelSpecific/arenaSmarts/Smart_ModuleDisplayVariables.h	??
applications/modelSpecific/arenaSmarts/Smart_OverlappingResources.h	??
applications/modelSpecific/arenaSmarts/Smart_ParallelProcessingOfEntities.h	??
applications/modelSpecific/arenaSmarts/Smart_PlacingEntitiesInQueueSets.h	??
applications/modelSpecific/arenaSmarts/Smart_PriorityExample.h	??
applications/modelSpecific/arenaSmarts/Smart_ProcessArena.h	??
applications/modelSpecific/arenaSmarts/Smart_Record_Arena.h	??
applications/modelSpecific/arenaSmarts/Smart_RemovingAndReorderingEntitiesInAQueue.h	??
applications/modelSpecific/arenaSmarts/Smart_ResourceCosting.h	??
applications/modelSpecific/arenaSmarts/Smart_ResourceScheduleCosting.h	??
applications/modelSpecific/arenaSmarts/Smart_ResourceSets.h	??
applications/modelSpecific/arenaSmarts/Smart_SeizingMultipleSimultaneously.h	??
applications/modelSpecific/arenaSmarts/Smart_SelectingRouteBasedOnProbability.h	??
applications/modelSpecific/arenaSmarts/Smart_SelectingShorterQueue.h	??
applications/modelSpecific/arenaSmarts/Smart_SynchronizingParallelEntities.h	??
applications/modelSpecific/arenaSmarts/Smart_ValueAdded.h	??

applications/modelSpecific/arenaSmarts/Smart_WaitForSignal.h	??
applications/modelSpecific/book/Book_Cap02_Example01.h	??
applications/modelSpecific/smarts/AI/StartAIAssistant.h	??
applications/modelSpecific/smarts/AnalyticalModeling/Smart_MarkovChain.h	??
applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BacteriaColony_GRO.h	??
applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioKineticLawRegulation.h	??
applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioNetworkAnalysis.h	??
applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioReversibleKineticLaw.h	??
applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioReversibleMassAction.h	??
applications/modelSpecific/smarts/BiochemicalSimulation/Smart_GroColonyGrowth.h	??
applications/modelSpecific/smarts/BiochemicalSimulation/Smart_GroColonyLifecycle.h	??
applications/modelSpecific/smarts/Continuous/Smart_LSODE.h	??
applications/modelSpecific/smarts/Continuous/Smart_ODE.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_Buffer.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_Clone.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_Delay.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_Failures.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_HoldSearchRemove.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_Process.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_ProcessSet.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_SeizeDelayRelease.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_SeizeDelayReleaseMany.h	??
applications/modelSpecific/smarts/DiscreteProcessing/Smart_SeizeDelayReleaseNoDataDefs.h	??
applications/modelSpecific/smarts/ExternalIntegration/Smart_CppForG.h	??
applications/modelSpecific/smarts/ExternalIntegration/Smart_PythonForG.h	??
applications/modelSpecific/smarts/ExternalIntegration/Smart_R_Simulator.h	??
applications/modelSpecific/smarts/Grouping/Smart_BatchSeparate.h	??
applications/modelSpecific/smarts/InputOutput/Smart_Record.h	??
applications/modelSpecific/smarts/Logic/Smart_AssignWriteSeizes.h	??
applications/modelSpecific/smarts/Logic/Smart_Attribute_Variable.h	??
applications/modelSpecific/smarts/Logic/Smart_ModelInfoModelSimulation.h	??
applications/modelSpecific/smarts/Logic/Smart_Parser.h	??
applications/modelSpecific/smarts/Logic/Smart_ParserModelFunctions.h	??
applications/modelSpecific/smarts/Logic/Smart_Plugin.h	??
applications/modelSpecific/smarts/MaterialHandling/Smart_RouteStation.h	??
applications/modelSpecific/smarts/MaterialHandling/Smart_Sequence.h	??
applications/modelSpecific/smarts/ModalModel/Smart_CellularAutomata.h	??
applications/modelSpecific/smarts/ModalModel/Smart_CellularAutomata1.h	??
applications/modelSpecific/smarts/ModalModel/Smart_DefaultModalModel.h	??
applications/modelSpecific/smarts/ModalModel/Smart_Dummy.h	??
applications/modelSpecific/smarts/ModalModel/Smart_FSMState.h	??
applications/modelSpecific/smarts/ModalModel/Smart_Modalmodel.h	??
applications/modelSpecific/smarts/ModalModel/Smart_ModalModelDefaultNetwork.h	??
applications/modelSpecific/smarts/ModalModel/Smart_ModalModelFSM.h	??
applications/modelSpecific/smarts/ModalModel/Smart_ModalModelPetriNet.h	??
applications/modelSpecific/smarts/ModalModel/Smart_PetriPlace.h	??
applications/modelSpecific/smarts/Synchronization/Smart_OnEvent.h	??
applications/modelSpecific/smarts/Synchronization/Smart_SimulationControlResponse.h	??
applications/modelSpecific/smarts/Synchronization/Smart_WaitScanCondition.h	??
applications/modelSpecific/smarts/Synchronization/Smart_WaitSignal.h	??
applications/modelSpecific/teaching/AnElectronicAssemblyAndTestSystem.h	??
applications/modelSpecific/teaching/BufferFIFO.h	??
applications/modelSpecific/teaching/ContinuousModel.h	??
applications/modelSpecific/teaching/FullSimulationOfComplexModel.h	??
applications/modelSpecific/teaching/Half_Adder.h	??
applications/modelSpecific/teaching/OperatingSystem02.h	??
applications/modelSpecific/teaching/OperatingSystem03.h	??
applications/modelSpecific/teaching/Rectifier.h	??

applications/modelSpecific/teaching/Loja/Loja01.h	??
applications/modelSpecific/wcm/WcmBioNetworkBridge.h	??
applications/modelSpecific/wcm/WcmCompartmentTransportWholeCellMvp.h	??
applications/modelSpecific/wcm/WcmFluxBalanceWholeCellMvp.h	??
applications/modelSpecific/wcm/WcmGeneExpression.h	??
applications/modelSpecific/wcm/WcmLifecycleStress.h	??
applications/modelSpecific/wcm/WcmMgenitaliumKarr2012.h	??
applications/modelSpecific/wcm/WcmPathwayStressWholeCellMvp.h	??
applications/modelSpecific/wcm/WcmSimpleSsa.h	??
applications/modelSpecific/wcm/WcmWholeCellMvp.h	??
applications/modelSpecific/wcm/WcmYeastDidacticWholeCellMvp.h	??
applications/modelSpecific/wcm/WcmYeastEukaryoticWholeCellMvp.h	??
applications/shell/GenesysShell.h	??
applications/shell/GenesysShell_if.h	??
applications/worker/BaseGenesysWebApplication.h	??
applications/worker/api/ApiRouter.h	??
applications/worker/auth/TokenService.h	??
applications/worker/http/HttpRequest.h	??
applications/worker/http/HttpResponse.h	??
applications/worker/http/SimpleHttpServer.h	??
applications/worker/service/SimulatorSessionService.h	??
applications/worker/service/WebWorkerRuntime.h	??
applications/worker/session/SessionContext.h	??
applications/worker/session/SessionManager.h	??
applications/worker/worker/WorkerJob.h	??
applications/worker/worker/WorkerJobManager.h	??
kernel/TraitsKernel.h	??
kernel/simulator/ConnectionManager.h	??
kernel/simulator/DefineGetterSetter.h	??
kernel/simulator/Event.h	??
kernel/simulator/ExperimentManager.h	??
kernel/simulator/ExperimentManagerDefaultImpl1.h	??
kernel/simulator/ExperimentManager_if.h	??
kernel/simulator/GenesysPropertyIntrospection.h	??
kernel/simulator/LicenceManager.h	??
kernel/simulator/OnEventManager.h	??
kernel/simulator/Parser_if.h	??
kernel/simulator/ParserChangesInformation.h	??
kernel/simulator/ParserDefaultImpl1.h	??
kernel/simulator/ParserDefaultImpl2.h	??
kernel/simulator/ParserManager.h	??
kernel/simulator/Persistence.h	??
kernel/simulator/Plugin.h	??
kernel/simulator/PluginConnector_if.h	??
kernel/simulator/PluginInformation.h	??
kernel/simulator/PluginManager.h	??
kernel/simulator/PropertyGenesys.h	??
kernel/simulator/ScenarioExperiment_if.h	??
kernel/simulator/SimulationControlAndResponse.h	??
kernel/simulator/SimulationExperiment.h	??
kernel/simulator/SimulationReporter_if.h	??
kernel/simulator/SimulationReporterDefaultImpl1.h	??
kernel/simulator/SimulationScenario.h	??
kernel/simulator/Simulator.h	??
kernel/simulator/SimulatorFacade.h	??
kernel/simulator/SinkModelComponent.h	??
kernel/simulator/SourceModelComponent.h	??
kernel/simulator/SystemDependencyResolver.h	??

kernel/simulator/TraceManager.h	??
kernel/simulator/essentialPlugins/Attribute.h	??
kernel/simulator/essentialPlugins/Counter.h	??
kernel/simulator/essentialPlugins/Entity.h	??
kernel/simulator/essentialPlugins/EntityType.h	??
kernel/simulator/essentialPlugins/SparseValueStore.h	??
kernel/simulator/essentialPlugins/StatisticsCollector.h	??
kernel/simulator/model/Model.h	??
kernel/simulator/model/ModelChecker_if.h	??
kernel/simulator/model/ModelCheckerDefaultImpl1.h	??
kernel/simulator/model/ModelComponent.h	??
kernel/simulator/model/ModelComponentManager.h	??
kernel/simulator/model/ModelDataDefinition.h	??
kernel/simulator/model/ModelDataDefinitionAssociations.h	??
kernel/simulator/model/ModelDataManager.h	??
kernel/simulator/model/ModelInfo.h	??
kernel/simulator/model/ModelManager.h	??
kernel/simulator/model/ModelSimulation.h	??
kernel/simulator/persistence/CppSerializer.h	??
kernel/simulator/persistence/GenSerializer.h	??
kernel/simulator/persistence/JsonSerializer.h	??
kernel/simulator/persistence/Persistence_if.h	??
kernel/simulator/persistence/PersistenceDefaultImpl2.h	??
kernel/simulator/persistence/PersistencePartialLoadImpl1.h	??
kernel/simulator/persistence/Serializer.h	??
kernel/simulator/persistence/XmlSerializer.h	??
kernel/statistics/Collector_if.h	??
kernel/statistics/CollectorDatafile_if.h	??
kernel/statistics/CollectorDatafileDefaultImpl1.h	??
kernel/statistics/CollectorDefaultImpl1.h	??
kernel/statistics/Sampler_if.h	??
kernel/statistics/SamplerDefaultImpl1.h	??
kernel/statistics/SorttFile.h	??
kernel/statistics/Statistics_if.h	??
kernel/statistics/StatisticsDataFile_if.h	??
kernel/statistics/StatisticsDataFileDefaultImpl1.h	??
kernel/statistics/StatisticsDefaultImpl1.h	??
kernel/util/Exact.h	??
kernel/util/List.h	??
kernel/util/ListObservable.h	??
kernel/util/Util.h	??
plugins/PluginConnectorDummyImpl1.h	??
plugins/PluginConnectorStaticImpl1.h	??
plugins/components/AI/AIAssistant.h	??
plugins/components/AnalyticalModeling/MarkovChain.h	??
plugins/components/BiochemicalSimulation/BacteriaColony.h	??
plugins/components/BiochemicalSimulation/BioRunnerCommand.h	??
plugins/components/BiochemicalSimulation/BioSimulate.h	??
plugins/components/BiochemicalSimulation/BioSteadyState.h	??
plugins/components/BiochemicalSimulation/GeneticCircuitSimulate.h	??
plugins/components/BiochemicalSimulation/GeneticExpressionStep.h	??
plugins/components/BiochemicalSimulation/MetabolicFluxBalance.h	??
plugins/components/Continuous/ContinuousSystemComponent.h	??
plugins/components/Continuous/DiffEquations.h	??
plugins/components/Continuous/DiffusionSimulate.h	??
plugins/components/Continuous/LSODE.h	??
plugins/components/Continuous/OLD_ODEElement.h	??
plugins/components/Decisions/Decide.h	??

plugins/components/Decisions/DropOff.h	??
plugins/components/Decisions/PickUp.h	??
plugins/components/Decisions/Remove.h	??
plugins/components/Decisions/Search.h	??
plugins/components/DiscreteProcessing/Buffer.h	??
plugins/components/DiscreteProcessing/Clone.h	??
plugins/components/DiscreteProcessing/Delay.h	??
plugins/components/DiscreteProcessing/Process.h	??
plugins/components/DiscreteProcessing/Release.h	??
plugins/components/DiscreteProcessing/Seize.h	??
plugins/components/DiscreteProcessing/auxiliar/QueueableItem.h	??
plugins/components/DiscreteProcessing/auxiliar/SeizableItem.h	??
plugins/components/ElectronicsSimulation/SPICECircuit.h	??
plugins/components/ElectronicsSimulation/SPICENode.h	??
plugins/components/ExternalIntegration/CppForG.h	??
plugins/components/ExternalIntegration/PythonForG.h	??
plugins/components/ExternalIntegration/RSimulator.h	??
plugins/components/Grouping/Batch.h	??
plugins/components/Grouping/Separate.h	??
plugins/components/InputOutput/Record.h	??
plugins/components/InputOutput/Write.h	??
plugins/components/Logic/Assign.h	??
plugins/components/Logic/Create.h	??
plugins/components/Logic/Dispose.h	??
plugins/components/MaterialHandling/Access.h	??
plugins/components/MaterialHandling/Enter.h	??
plugins/components/MaterialHandling/Exit.h	??
plugins/components/MaterialHandling/Leave.h	??
plugins/components/MaterialHandling/PickableStationItem.h	??
plugins/components/MaterialHandling/PickStation.h	??
plugins/components/MaterialHandling/Route.h	??
plugins/components/MaterialHandling/Start.h	??
plugins/components/MaterialHandling/Stop.h	??
plugins/components/MaterialHandling/Store.h	??
plugins/components/MaterialHandling/Unstore.h	??
plugins/components/ModalModel/CellularAutomataComp.h	??
plugins/components/ModalModel/DefaultNode.h	??
plugins/components/ModalModel/DefaultTransitionExtensions.h	??
plugins/components/ModalModel/FSMState.h	??
plugins/components/ModalModel/ModalModelDefault.h	??
plugins/components/ModalModel/ModalModelFSM.h	??
plugins/components/ModalModel/ModalModelPetriNet.h	??
plugins/components/ModalModel/PetriPlace.h	??
plugins/components/ModalModel/Submodel.h	??
plugins/components/ModalModel/CellularAutomata/Boundary_Closed.h	??
plugins/components/ModalModel/CellularAutomata/Boundary_Fixed.h	??
plugins/components/ModalModel/CellularAutomata/BoundaryCondition.h	??
plugins/components/ModalModel/CellularAutomata/Cell.h	??
plugins/components/ModalModel/CellularAutomata/CellularAutomata.h	??
plugins/components/ModalModel/CellularAutomata/CellularAutomata_1DTimed.h	??
plugins/components/ModalModel/CellularAutomata/CellularAutomata_Classic.h	??
plugins/components/ModalModel/CellularAutomata/CellularAutomataAllHeaders.h	??
plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h	??
plugins/components/ModalModel/CellularAutomata/Lattice.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_Elementary.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_FlorestalFire.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_GameOfLife.h	??

plugins/components/ModalModel/CellularAutomata/LocalRule_Growty.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_HppLatticeGas.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_SandRock.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_SandRock_Student.h	??
plugins/components/ModalModel/CellularAutomata/LocalRule_ShowNeighborhood.h	??
plugins/components/ModalModel/CellularAutomata/Neighborhood.h	??
plugins/components/ModalModel/CellularAutomata/Neighborhood_Center.h	??
plugins/components/ModalModel/CellularAutomata/Neighborhood_Moore.h	??
plugins/components/ModalModel/CellularAutomata/Neighborhood_VonNeumann.h	??
plugins/components/ModalModel/CellularAutomata/State.h	??
plugins/components/ModalModel/CellularAutomata/State_Numerical.h	??
plugins/components/ModalModel/CellularAutomata/StateSet.h	??
plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h	??
plugins/components/Synchronization/Match.h	??
plugins/components/Synchronization/Signal.h	??
plugins/components/Synchronization/Wait.h	??
plugins/components/Template/DummyComponent.h	??
plugins/components/WholeCellModeling/BioStateProjectionComponent.h	??
plugins/components/WholeCellModeling/CellCycleCheckpointComponent.h	??
plugins/components/WholeCellModeling/CellDivisionEvent.h	??
plugins/components/WholeCellModeling/CellFateDecisionComponent.h	??
plugins/components/WholeCellModeling/CellGrowthComponent.h	??
plugins/components/WholeCellModeling/CompartmentExchangeComponent.h	??
plugins/components/WholeCellModeling/EukaryoticCellCycleComponent.h	??
plugins/components/WholeCellModeling/FtsZPolymerizationComponent.h	??
plugins/components/WholeCellModeling/MetabolicStateProjectionComponent.h	??
plugins/components/WholeCellModeling/MetabolicSubmodelComponent.h	??
plugins/components/WholeCellModeling/PathwayStressResponseComponent.h	??
plugins/components/WholeCellModeling/ResourceAllocationComponent.h	??
plugins/components/WholeCellModeling/StochasticReactionComponent.h	??
plugins/components/WholeCellModeling/StochasticTranscription.h	??
plugins/components/WholeCellModeling/StochasticTranslation.h	??
plugins/data/AI/AISupport.h	??
plugins/data/BiochemicalSimulation/BacteriaSignalGrid.h	??
plugins/data/BiochemicalSimulation/BioNetwork.h	??
plugins/data/BiochemicalSimulation/BioParameter.h	??
plugins/data/BiochemicalSimulation/BioReaction.h	??
plugins/data/BiochemicalSimulation/BioSBMLBridge.h	??
plugins/data/BiochemicalSimulation/BioSimulatorRunner.h	??
plugins/data/BiochemicalSimulation/BioSpecies.h	??
plugins/data/BiochemicalSimulation/GeneticCircuit.h	??
plugins/data/BiochemicalSimulation/GeneticCircuitPart.h	??
plugins/data/BiochemicalSimulation/GeneticRegulation.h	??
plugins/data/BiochemicalSimulation/GroProgram.h	??
plugins/data/BiochemicalSimulation/GroProgramAst.h	??
plugins/data/BiochemicalSimulation/GroProgramCompiler.h	??
plugins/data/BiochemicalSimulation/GroProgramIrr.h	??
plugins/data/BiochemicalSimulation/GroProgramParser.h	??
plugins/data/BiochemicalSimulation/GroProgramRuntime.h	??
plugins/data/BiochemicalSimulation/MetabolicNetwork.h	??
plugins/data/BiochemicalSimulation/MetabolicReaction.h	??
plugins/data/Continuous/DiffusionField.h	??
plugins/data/Continuous/ODESolver.h	??
plugins/data/DiscreteProcessing/Failure.h	??
plugins/data/DiscreteProcessing/Queue.h	??
plugins/data/DiscreteProcessing/Resource.h	??
plugins/data/DiscreteProcessing/Schedule.h	??
plugins/data/ExternalIntegration/CppCompiler.h	??

plugins/data/ExternalIntegration/ OctaveRunner.h	??
plugins/data/ExternalIntegration/ PythonRuntime.h	??
plugins/data/ExternalIntegration/ RSimulatorRunner.h	??
plugins/data/ExternalIntegration/ SPICERunner.h	??
plugins/data/Grouping/ EntityGroup.h	??
plugins/data/InputOutput/ File.h	??
plugins/data/Logic/ AssignmentItem.h	??
plugins/data/Logic/ Formula.h	??
plugins/data/Logic/ Label.h	??
plugins/data/Logic/ Set.h	??
plugins/data/Logic/ Variable.h	??
plugins/data/MaterialHandling/ Sequence.h	??
plugins/data/MaterialHandling/ Station.h	??
plugins/data/MaterialHandling/ Storage.h	??
plugins/data/Synchronization/ SignalData.h	??
plugins/data/Template/ DummyElement.h	??
plugins/data/WholeCellModeling/ BioCompartment.h	??
plugins/data/WholeCellModeling/ MolecularSpecies.h	??
plugins/data/WholeCellModeling/ StochasticReactionRule.h	??
plugins/data/WholeCellModeling/ WholeCellState.h	??

Chapter 9

Namespace Documentation

9.1 genesys_mcp Namespace Reference

Namespaces

- namespace [__main__](#)
- namespace [client](#)
- namespace [server](#)

9.1.1 Detailed Description

Genesys Simulator MCP Connector.

9.2 genesys_mcp.__main__ Namespace Reference

9.2.1 Detailed Description

Allow running the MCP server with: `python -m genesys_mcp`

9.3 genesys_mcp.client Namespace Reference

Classes

- class [GenesysClient](#)
- class [GenesysClientError](#)

9.3.1 Detailed Description

HTTP client wrapping the Genesys Worker Application REST API.

9.4 genesys_mcp.server Namespace Reference

Functions

- [GenesysClient](#) `_client` (str base_url, str|None token=None)
- dict[str, str] `_err` ([GenesysClientError](#) e)
- dict[str, str] `_conn_err` (httpx.RequestError e)
- dict[str, Any] [genesys_health](#) (str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_auth](#) (str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_info](#) (str token, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_new_model](#) (str token, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_get_model](#) (str token, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_save_model](#) (str token, str filename, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_load_model](#) (str token, str filename, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_import_model_language](#) (str token, str model_text, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_simulation_status](#) (str token, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_configure_simulation](#) (str token, int number_of_replications, float replication_length, float warm_up_period=0.0, bool pause_on_event=False, bool pause_on_replication=False, bool initialize_statistics=True, bool initialize_system=True, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_run_simulation](#) (str token, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_step_simulation](#) (str token, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_worker_info](#) (str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_worker_capabilities](#) (str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_create_job](#) (str token, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_job_status](#) (str token, str job_id, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_run_job](#) (str token, str job_id, str base_url=_DEFAULT_URL)
- dict[str, Any] [genesys_job_result](#) (str token, str job_id, str base_url=_DEFAULT_URL)
- None `main` ()

Variables

- [mcp](#)
- `_DEFAULT_URL` = os.environ.get("GENESYS_URL", "http://localhost:8080")

9.4.1 Detailed Description

Genesys MCP Server -- exposes GenESyS simulation capabilities via MCP.

9.4.2 Function Documentation

9.4.2.1 genesys_auth()

```
dict[str, Any] genesys_mcp.server.genesys_auth (
    str base_url = _DEFAULT_URL )
```

Create a new Genesys session and return the bearer token.

Returns {"token": "<bearer-token>", ...}.

Pass the token to all subsequent tools as the 'token' parameter.

9.4.2.2 genesys_configure_simulation()

```
dict[str, Any] genesys_mcp.server.genesys_configure_simulation (
    str token,
    int number_of_replications,
    float replication_length,
    float warm_up_period = 0.0,
    bool pause_on_event = False,
    bool pause_on_replication = False,
    bool initialize_statistics = True,
    bool initialize_system = True,
    str base_url = _DEFAULT_URL )
```

Configure simulation parameters before running.

Args:

token: Bearer token from genesys_auth.
number_of_replications: How many independent replications to run.
replication_length: Simulated time length of each replication.
warm_up_period: Simulated time before statistics start being collected.
pause_on_event: Pause simulation after each event (step-by-step mode).
pause_on_replication: Pause between replications.
initialize_statistics: Reset statistics counters at start of each replication.
initialize_system: Reset system state at start of each replication.
base_url: Genesys worker app base URL.

9.4.2.3 genesys_create_job()

```
dict[str, Any] genesys_mcp.server.genesys_create_job (
    str token,
    str base_url = _DEFAULT_URL )
```

Create a worker job from the session's current model.

The job is created in 'queued' state. A snapshot of the model is persisted so later execution is decoupled from live model mutations.
Returns {"jobId": "<id>", "state": "queued", ...}.
Requires a valid session token from genesys_auth.

9.4.2.4 genesys_get_model()

```
dict[str, Any] genesys_mcp.server.genesys_get_model (
    str token,
    str base_url = _DEFAULT_URL )
```

Return a description of the session's current simulation model.

Requires a valid session token from genesys_auth.

9.4.2.5 genesys_health()

```
dict[str, Any] genesys_mcp.server.genesys_health (
    str base_url = _DEFAULT_URL )
```

Check that the Genesys worker application is reachable.

Returns the health status object from GET /health.
No authentication required.

9.4.2.6 genesys_import_model_language()

```
dict[str, Any] genesys_mcp.server.genesys_import_model_language (
    str token,
    str model_text,
    str base_url = _DEFAULT_URL )
```

Import a simulation model from GenESyS model language text.

'model_text' is the full plain-text GenESyS model specification.
The imported model becomes the session's current model.
Requires a valid session token from genesys_auth.

9.4.2.7 genesys_info()

```
dict[str, Any] genesys_mcp.server.genesys_info (
    str token,
    str base_url = _DEFAULT_URL )
```

Return simulator version, name, and licence information.

Requires a valid session token from genesys_auth.

9.4.2.8 genesys_job_result()

```
dict[str, Any] genesys_mcp.server.genesys_job_result (
    str token,
    str job_id,
    str base_url = _DEFAULT_URL )
```

Retrieve the simulation result summary for a finished or failed job.

Only available after the job reaches a terminal state.
Requires a valid session token from genesys_auth.

9.4.2.9 genesys_job_status()

```
dict[str, Any] genesys_mcp.server.genesys_job_status (
    str token,
    str job_id,
    str base_url = _DEFAULT_URL )
```

Return the current state of a worker job (queued/running/finished/failed).

Requires a valid session token from genesys_auth.

9.4.2.10 genesys_load_model()

```
dict[str, Any] genesys_mcp.server.genesys_load_model (
    str token,
    str filename,
    str base_url = _DEFAULT_URL )
```

Load a model from a file inside the session workspace.

'filename' must be a basename (no path separators).

Requires a valid session token from genesys_auth.

9.4.2.11 genesys_new_model()

```
dict[str, Any] genesys_mcp.server.genesys_new_model (
    str token,
    str base_url = _DEFAULT_URL )
```

Create a new empty simulation model in the current session.

The new model becomes the session's current model.

Requires a valid session token from genesys_auth.

9.4.2.12 genesys_run_job()

```
dict[str, Any] genesys_mcp.server.genesys_run_job (
    str token,
    str job_id,
    str base_url = _DEFAULT_URL )
```

Execute a worker job synchronously.

The job must be in 'queued' state. This call blocks until the simulation finishes. Poll genesys_job_status or call genesys_job_result afterwards.

Requires a valid session token from genesys_auth.

9.4.2.13 genesys_run_simulation()

```
dict[str, Any] genesys_mcp.server.genesys_run_simulation (
    str token,
    str base_url = _DEFAULT_URL )
```

Run the current simulation synchronously and return a summary.

Configure simulation parameters first with `genesys_configure_simulation`.
Requires a valid session token from `genesys_auth`.

9.4.2.14 genesys_save_model()

```
dict[str, Any] genesys_mcp.server.genesys_save_model (
    str token,
    str filename,
    str base_url = _DEFAULT_URL )
```

Save the current model to a file inside the session workspace.

'filename' must be a basename (no path separators). Only alphanumeric characters, dots, hyphens, and underscores are allowed.
Requires a valid session token from `genesys_auth`.

9.4.2.15 genesys_simulation_status()

```
dict[str, Any] genesys_mcp.server.genesys_simulation_status (
    str token,
    str base_url = _DEFAULT_URL )
```

Return the current simulation status (state, replication, clock, etc.).

Requires a valid session token from `genesys_auth`.

9.4.2.16 genesys_step_simulation()

```
dict[str, Any] genesys_mcp.server.genesys_step_simulation (
    str token,
    str base_url = _DEFAULT_URL )
```

Advance the simulation by one event step.

Useful for step-by-step inspection. Configure with `pause_on_event=True` first.
Requires a valid session token from `genesys_auth`.

9.4.2.17 genesys_worker_capabilities()

```
dict[str, Any] genesys_mcp.server.genesys_worker_capabilities (
    str base_url = _DEFAULT_URL )
```

Return the current worker's capability flags.

Describes what simulation features this worker instance supports.
No authentication required.

9.4.2.18 genesys_worker_info()

```
dict[str, Any] genesys_mcp.server.genesys_worker_info (
    str base_url = _DEFAULT_URL )
```

Return worker identity metadata (id, version, role).

No authentication required.

9.4.3 Variable Documentation

9.4.3.1 mcp

```
genesys_mcp.server.mcp
```

Initial value:

```
00001 = FastMCP(
00002     "Genesys Simulator",
00003     instructions=(
00004         "Use these tools to create, configure, and run GenESyS discrete-event "
00005         "simulation models. Always call genesys_auth first to obtain a session "
00006         "token before using any other tool."
00007     ),
00008 )
```

9.5 GuiCrashDiagnostics Namespace Reference

Crash diagnostics helpers for the Qt GUI process.

Functions

- void **installFatalSignalHandlers** ()
Installs temporary fatal-signal handlers that print a crash backtrace.

9.5.1 Detailed Description

Crash diagnostics helpers for the Qt GUI process.

This namespace keeps temporary signal-handling utilities isolated from the rest of the GUI runtime so they can be installed only when needed during crash investigation.

9.6 QCP Namespace Reference

Enumerations

- enum [ResolutionUnit](#) { [ruDotsPerMeter](#) , [ruDotsPerCentimeter](#) , [ruDotsPerInch](#) }
- enum [ExportPen](#) { [epNoCosmetic](#) , [epAllowCosmetic](#) }
- enum [SignDomain](#) { [sdNegative](#) , [sdBoth](#) , [sdPositive](#) }
- enum [MarginSide](#) {
[msLeft](#) = 0x01 , [msRight](#) = 0x02 , [msTop](#) = 0x04 , [msBottom](#) = 0x08 ,
[msAll](#) = 0xFF , [msNone](#) = 0x00 }
- enum [AntialiasedElement](#) {
[aeAxes](#) = 0x0001 , [aeGrid](#) = 0x0002 , [aeSubGrid](#) = 0x0004 , [aeLegend](#) = 0x0008 ,
[aeLegendItems](#) = 0x0010 , [aePlottables](#) = 0x0020 , [aeItems](#) = 0x0040 , [aeScatters](#) = 0x0080 ,
[aeFills](#) = 0x0100 , [aeZeroLine](#) = 0x0200 , [aeOther](#) = 0x8000 , [aeAll](#) = 0xFFFF ,
[aeNone](#) = 0x0000 }
- enum [PlottingHint](#) { [phNone](#) = 0x000 , [phFastPolylines](#) = 0x001 , [phImmediateRefresh](#) = 0x002 ,
[phCacheLabels](#) = 0x004 }
- enum [Interaction](#) {
[iNone](#) = 0x000 , [iRangeDrag](#) = 0x001 , [iRangeZoom](#) = 0x002 , [iMultiSelect](#) = 0x004 ,
[iSelectPlottables](#) = 0x008 , [iSelectAxes](#) = 0x010 , [iSelectLegend](#) = 0x020 , [iSelectItems](#) = 0x040 ,
[iSelectOther](#) = 0x080 , [iSelectPlottablesBeyondAxisRect](#) = 0x100 }
- enum [SelectionRectMode](#) { [srmNone](#) , [srmZoom](#) , [srmSelect](#) , [srmCustom](#) }
- enum [SelectionType](#) {
[stNone](#) , [stWhole](#) , [stSingleData](#) , [stDataRange](#) ,
[stMultipleDataRanges](#) }

Functions

- bool [isInvalidData](#) (double value)
- bool [isInvalidData](#) (double value1, double value2)
- void [setMarginValue](#) (QMargins &margins, [QCP::MarginSide](#) side, int value)
- int [getMarginValue](#) (const QMargins &margins, [QCP::MarginSide](#) side)

Variables

- const QMetaObject [staticMetaObject](#)

9.6.1 Detailed Description

The [QCP](#) Namespace contains general enums, QFlags and functions used throughout the [QCustomPlot](#) library. It provides QMetaObject-based reflection of its enums and flags via [QCP::staticMetaObject](#).

9.6.2 Enumeration Type Documentation

9.6.2.1 AntialiasedElement

```
enum QCP::AntialiasedElement
```

Defines what objects of a plot can be forcibly drawn antialiased/not antialiased. If an object is neither forcibly drawn antialiased nor forcibly drawn not antialiased, it is up to the respective element how it is drawn. Typically it provides a [setAntialiased](#) function for this.

[AntialiasedElements](#) is a flag of or-combined elements of this enum type.

See also

[QCustomPlot::setAntialiasedElements](#), [QCustomPlot::setNotAntialiasedElements](#)

Enumerator

aeAxes	0x0001 Axis base line and tick marks
aeGrid	0x0002 Grid lines
aeSubGrid	0x0004 Sub grid lines
aeLegend	0x0008 Legend box
aeLegendItems	0x0010 Legend items
aePlottables	0x0020 Main lines of plottables
aeItems	0x0040 Main lines of items
aeScatters	0x0080 Scatter symbols of plottables (excluding scatter symbols of type ssPixmap)
aeFills	0x0100 Borders of fills (e.g. under or between graphs)
aeZeroLine	0x0200 Zero-lines, see QCPGrid::setZeroLinePen
aeOther	0x8000 Other elements that don't fit into any of the existing categories
aeAll	0xFFFF All elements
aeNone	0x0000 No elements

9.6.2.2 ExportPen

enum [QCP::ExportPen](#)

Defines how cosmetic pens (pens with numerical width 0) are handled during export.

See also

[QCustomPlot::savePdf](#)

Enumerator

epNoCosmetic	Cosmetic pens are converted to pens with pixel width 1 when exporting.
epAllowCosmetic	Cosmetic pens are exported normally (e.g. in PDF exports, cosmetic pens always appear as 1 pixel on screen, independent of viewer zoom level)

9.6.2.3 Interaction

enum [QCP::Interaction](#)

Defines the mouse interactions possible with [QCustomPlot](#).

`Interactions` is a flag of or-combined elements of this enum type.

See also

[QCustomPlot::setInteractions](#)

Enumerator

iNone	0x000 None of the interactions are possible
-------	---

Enumerator

iRangeDrag	0x001 Axis ranges are draggable (see QCPAxisRect::setRangeDrag , QCPAxisRect::setRangeDragAxes)
iRangeZoom	0x002 Axis ranges are zoomable with the mouse wheel (see QCPAxisRect::setRangeZoom , QCPAxisRect::setRangeZoomAxes)
iMultiSelect	0x004 The user can select multiple objects by holding the modifier set by QCustomPlot::setMultiSelectModifier while clicking
iSelectPlottables	0x008 Plottables are selectable (e.g. graphs, curves, bars,... see QCPAbstractPlottable)
iSelectAxes	0x010 Axes are selectable (or parts of them, see QCPAxis::setSelectableParts)
iSelectLegend	0x020 Legends are selectable (or their child items, see QCPLegend::setSelectableParts)
iSelectItems	0x040 Items are selectable (Rectangles, Arrows, Textitems, etc. see QCPAbstractItem)
iSelectOther	0x080 All other objects are selectable (e.g. your own derived layerables, other layout elements,...)
iSelectPlottablesBeyondAxisRect	0x100 When performing plottable selection/hit tests, this flag extends the sensitive area beyond the axis rect

9.6.2.4 MarginSide

enum [QCP::MarginSide](#)

Defines the sides of a rectangular entity to which margins can be applied.

See also

[QCPLayoutElement::setAutoMargins](#), [QCPAxisRect::setAutoMargins](#)

Enumerator

msLeft	0x01 left margin
msRight	0x02 right margin
msTop	0x04 top margin
msBottom	0x08 bottom margin
msAll	0xFF all margins
msNone	0x00 no margin

9.6.2.5 PlottingHint

enum [QCP::PlottingHint](#)

Defines plotting hints that control various aspects of the quality and speed of plotting.

See also

[QCustomPlot::setPlottingHints](#)

Enumerator

phNone	0x000 No hints are set
phFastPolylines	joins, thus is most effective for pen sizes larger than 1. It is only used for solid line pens. 0x001 Graph/Curve lines are drawn with a faster method. This reduces the quality especially of the line segment
phImmediateRefresh	This is set by default to prevent the plot from freezing on fast consecutive replots (e.g. user drags ranges with mouse). 0x002 causes an immediate repaint() instead of a soft update() when QCustomPlot::replot() is called with parameter QCustomPlot::rpRefreshHint .
phCacheLabels	0x004 axis (tick) labels will be cached as pixmaps, increasing replot performance.

9.6.2.6 ResolutionUnit

enum [QCP::ResolutionUnit](#)

Defines the different units in which the image resolution can be specified in the export functions.

See also

[QCustomPlot::savePng](#), [QCustomPlot::saveJpg](#), [QCustomPlot::saveBmp](#), [QCustomPlot::saveRastered](#)

Enumerator

ruDotsPerMeter	Resolution is given in dots per meter (dpm)
ruDotsPerCentimeter	Resolution is given in dots per centimeter (dpcm)
ruDotsPerInch	Resolution is given in dots per inch (DPI/PPI)

9.6.2.7 SelectionRectMode

enum [QCP::SelectionRectMode](#)

Defines the behaviour of the selection rect.

See also

[QCustomPlot::setSelectionRectMode](#), [QCustomPlot::selectionRect](#), [QCPSelectionRect](#)

Enumerator

srmNone	The selection rect is disabled, and all mouse events are forwarded to the underlying objects, e.g. for axis range dragging.
srmZoom	When dragging the mouse, a selection rect becomes active. Upon releasing, the axes that are currently set as range zoom axes (QCPAxisRect::setRangeZoomAxes) will have their ranges zoomed accordingly.
srmSelect	When dragging the mouse, a selection rect becomes active. Upon releasing, plottable data points that were within the selection rect are selected, if the plottable's selectability setting permits. (See data selection mechanism for details.)
srmCustom	When dragging the mouse, a selection rect becomes active. It is the programmer's responsibility to connect according slots to the selection rect's signals (e.g. QCPSelectionRect::accepted) in order to process the user interaction.

9.6.2.8 SelectionType

enum [QCP::SelectionType](#)

Defines the different ways a plottable can be selected. These images show the effect of the different selection types, when the indicated selection rect was dragged:

td	td	td	td	td
----	----	----	----	----

See also

[QCPAbstractPlottable::setSelectable](#), [QCPDataSelection::enforceType](#)

Enumerator

stNone	The plottable is not selectable.
stWhole	Selection behaves like stMultipleDataRanges , but if there are any data points selected, the entire plottable is drawn as selected.
stSingleData	One individual data point can be selected at a time.
stDataRange	Multiple contiguous data points (a data range) can be selected.
stMultipleDataRanges	Any combination of data points/ranges can be selected.

9.6.2.9 SignDomain

enum [QCP::SignDomain](#)

Represents negative and positive sign domain, e.g. for passing to [QCPAbstractPlottable::getKeyRange](#) and [QCPAbstractPlottable::getValueRange](#).

This is primarily needed when working with logarithmic axis scales, since only one of the sign domains can be visible at a time.

Enumerator

sdNegative	The negative sign domain, i.e. numbers smaller than zero.
sdBoth	Both sign domains, including zero, i.e. all numbers.
sdPositive	The positive sign domain, i.e. numbers greater than zero.

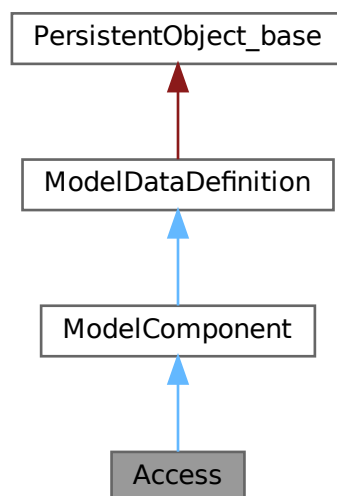
Chapter 10

Class Documentation

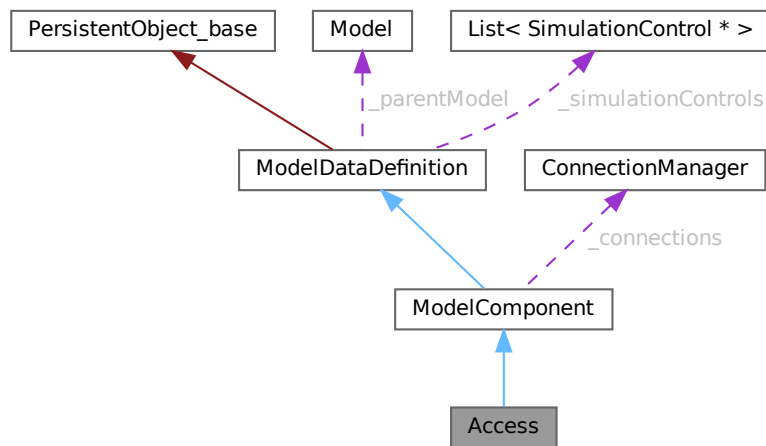
10.1 Access Class Reference

```
#include <plugins/components/MaterialHandling/Access.h>
```

Inheritance diagram for Access:



Collaboration diagram for Access:



Public Member Functions

- **Access** ([Model](#) *model, std::string name="")
 - virtual std::string [show](#) () override
- virtual public methods*

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.1.1 Detailed Description

[Access](#) module DESCRIPTION The [Access](#) module allocates one or more cells of a conveyor to an entity for movement from one station to another. Once the entity has control of the cells on the conveyor, it may then be conveyed to the next station. When an entity arrives at an [Access](#) module, it will wait until the appropriate number of contiguous cells on the conveyor are empty and aligned with the entity's station location. TYPICAL USES Parts accessing a conveyor to be sent to a paint booth Glass accessing a conveyor to be transferred to a cutting station PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart. Conveyor Name Name of the conveyor that the entity desires.

10.1.2 of Cells Number of contiguous conveyor cells the entity requires for

movement on the conveyor. [Queue](#) Type Determines the type of queue used to hold the entities, either an individual [Queue](#), a queue [Set](#), and Internal queue or an [Attribute](#) or Expression that evaluate to the queue name. [Queue](#) Name Name of the queue that will hold the entity until it accesses the conveyor. [Set](#) Name Name of the set of queues. [Set](#) Index Defines the index into the queue set. Note that this is the index into the set and not the name of the queue in the set. For example, the only valid entries for a queue set containing three members is an expression that evaluates to 1, 2, or 3. [Attribute](#) Name Defines the name of the attribute that stores the queue name to which entities will reside. Expression Defines the name of the expression that stores the queue name to which entities will reside.

10.1.3 Member Function Documentation

10.1.3.1 `_check()`

```
bool Access::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.1.3.2 `_loadInstance()`

```
bool Access::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.1.3.3 `_onDispatchEvent()`

```
void Access::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.1.3.4 `_saveInstance()`

```
void Access::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.1.3.5 `show()`

```
std::string Access::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

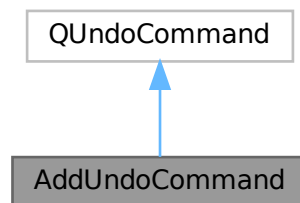
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

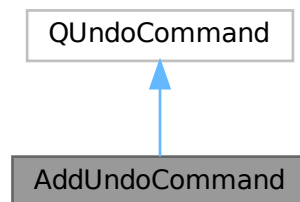
- plugins/components/MaterialHandling/Access.h
- plugins/components/MaterialHandling/Access.cpp

10.2 AddUndoCommand Class Reference

Inheritance diagram for AddUndoCommand:



Collaboration diagram for AddUndoCommand:



Public Member Functions

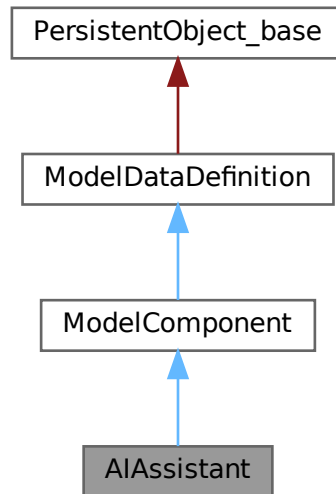
- **AddUndoCommand** (QGraphicsItem *item, [ModelGraphicsScene](#) *scene, QUndoCommand *parent=nullptr)
- void **undo** () override
- void **redo** () override

The documentation for this class was generated from the following files:

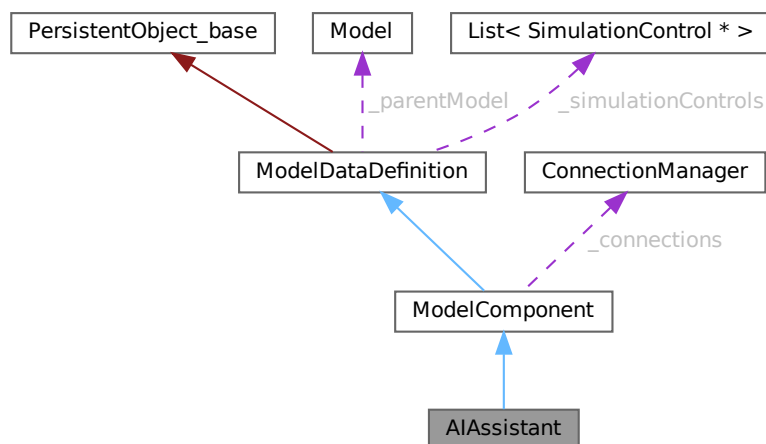
- applications/gui/genesys/actions/AddUndoCommand.h
- applications/gui/genesys/actions/AddUndoCommand.cpp

10.3 AIAssistant Class Reference

Inheritance diagram for AIAssistant:



Collaboration diagram for AIAssistant:



Public Member Functions

- **AIAssistant** (`Model` *model, std::string name="")
- void **setAISupport** (`AISupport` *support)

- [AISupport](#) * **getAISupport** () const
- void **setAISupportName** (std::string value)
- std::string **getAISupportName** () const
- void **setPromptTemplate** (std::string value)
- std::string **getPromptTemplate** () const
- void **setConversationScope** (std::string value)
- std::string **getConversationScope** () const
- void **setConversationKeyExpression** (std::string value)
- std::string **getConversationKeyExpression** () const
- void **setActionMappings** (std::string value)
- std::string **getActionMappings** () const
- void **setDelayExpression** (std::string value)
- std::string **getDelayExpression** () const
- void **setDefaultOutputPort** (unsigned int value)
- unsigned int **getDefaultOutputPort** () const
- void **setErrorOutputPort** (unsigned int value)
- unsigned int **getErrorOutputPort** () const
- void **setAllowDirectOutputPort** (bool value)
- bool **isAllowDirectOutputPort** () const
- void **setMinimumConfidence** (double value)
- double **getMinimumConfidence** () const
- void **setSaveConfidenceAttribute** (std::string value)
- std::string **getSaveConfidenceAttribute** () const
- void **setSaveSuccessAttribute** (std::string value)
- std::string **getSaveSuccessAttribute** () const
- std::string **getLastPrompt** () const
- std::string **getLastResponse** () const
- std::string **getLastError** () const
- bool **renderPrompt** (std::string &result, std::string &error)
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.3.1 Member Function Documentation

10.3.1.1 `_check()`

```
bool AIAssistant::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.3.1.2 `_loadInstance()`

```
bool AIAssistant::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.3.1.3 `_onDispatchEvent()`

```
void AIAssistant::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.3.1.4 _saveInstance()

```
void AIAssistant::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.3.1.5 show()

```
std::string AIAssistant::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

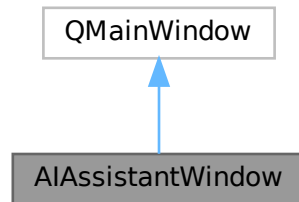
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

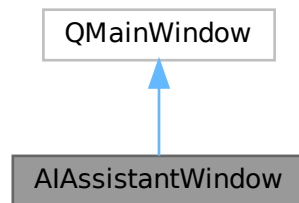
- plugins/components/AI/AIAssistant.h
- plugins/components/AI/AIAssistant.cpp

10.4 AIAssistantWindow Class Reference

Inheritance diagram for AIAssistantWindow:



Collaboration diagram for AIAssistantWindow:



Public Member Functions

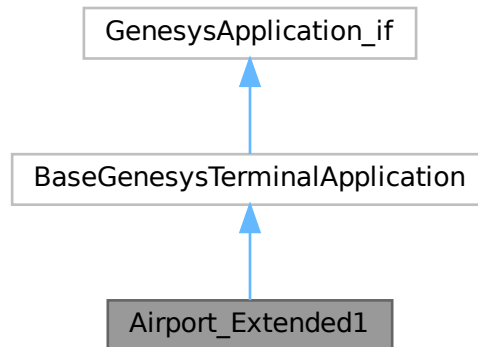
- **AIAssistantWindow** ([Simulator](#) *simulator, QWidget *parent=nullptr)

The documentation for this class was generated from the following files:

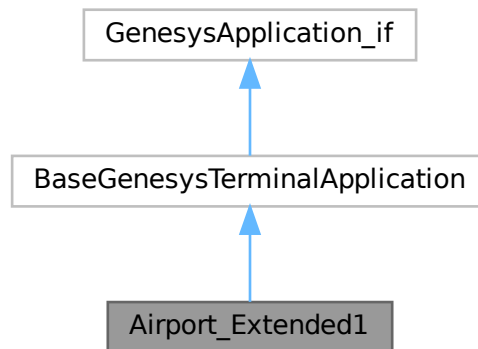
- applications/gui/ai_assistant/AIAssistantWindow.h
- applications/gui/ai_assistant/AIAssistantWindow.cpp

10.5 Airport_Extended1 Class Reference

Inheritance diagram for Airport_Extended1:



Collaboration diagram for Airport_Extended1:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.5.1 Member Function Documentation

10.5.1.1 main()

```
int Airport_Extended1::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

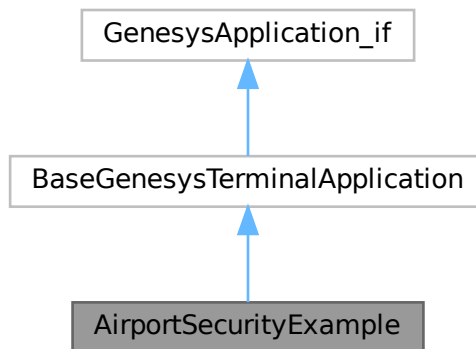
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

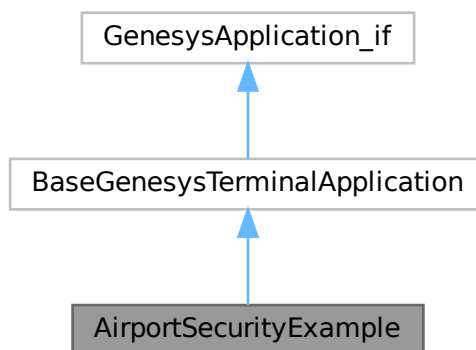
- applications/modelSpecific/arenaExamples/Airport_Extended1.h
- applications/modelSpecific/arenaExamples/Airport_Extended1.cpp

10.6 AirportSecurityExample Class Reference

Inheritance diagram for AirportSecurityExample:



Collaboration diagram for AirportSecurityExample:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.6.1 Member Function Documentation

10.6.1.1 main()

```
int AirportSecurityExample::main (
    int argc,
    char ** argv ) [override], [virtual]
```

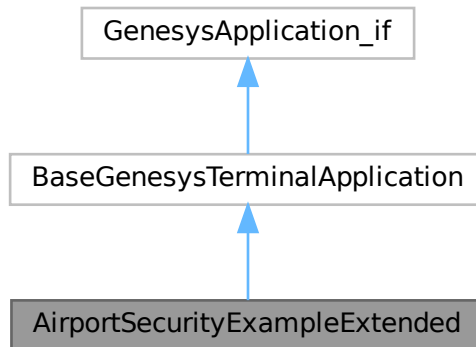
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

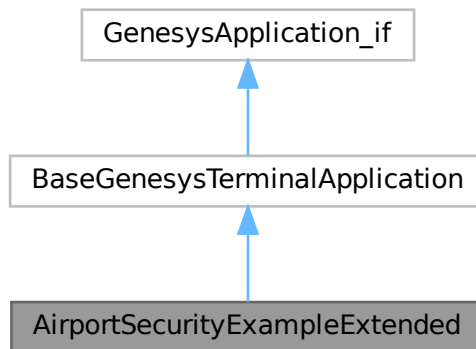
- applications/modelSpecific/arenaExamples/AirportSecurityExample.h
- applications/modelSpecific/arenaExamples/AirportSecurityExample.cpp

10.7 AirportSecurityExampleExtended Class Reference

Inheritance diagram for AirportSecurityExampleExtended:



Collaboration diagram for AirportSecurityExampleExtended:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.7.1 Member Function Documentation

10.7.1.1 main()

```
int AirportSecurityExampleExtended::main (
    int argc,
    char ** argv ) [override], [virtual]
```

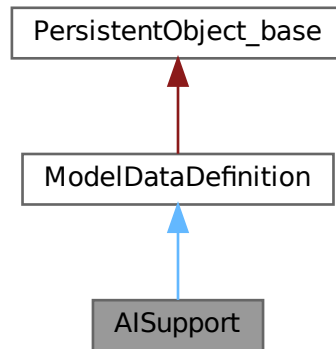
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

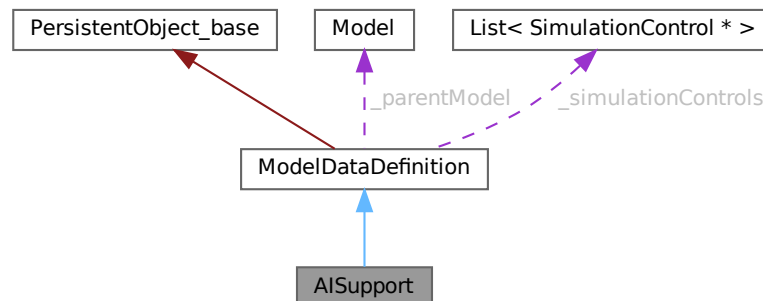
- applications/modelSpecific/arenaExamples/AirportSecurityExampleExtended.h
- applications/modelSpecific/arenaExamples/AirportSecurityExampleExtended.cpp

10.8 AISupport Class Reference

Inheritance diagram for AISupport:



Collaboration diagram for AISupport:



Public Member Functions

- **AISupport** ([Model](#) *model, std::string name="")
- AIConversationResponse **converse** (const std::string &conversationKey, const std::string &prompt)
- void **clearConversation** (const std::string &conversationKey)
- void **clearAllConversations** ()
- void **setProvider** (std::string value)
- std::string **getProvider** () const
- void **setModel** (std::string value)
- std::string **getModel** () const
- void **setBaseUrl** (std::string value)
- std::string **getBaseUrl** () const

- void **setApiKeyEnvironmentVariable** (std::string value)
- std::string **getApiKeyEnvironmentVariable** () const
- void **setSystemInstructions** (std::string value)
- std::string **getSystemInstructions** () const
- void **setTemperature** (double value)
- double **getTemperature** () const
- void **setMaxOutputTokens** (unsigned int value)
- unsigned int **getMaxOutputTokens** () const
- void **setTimeoutSeconds** (unsigned int value)
- unsigned int **getTimeoutSeconds** () const
- void **setHighReasoningMode** (bool value)
- bool **isHighReasoningMode** () const
- void **setConversationEnabled** (bool value)
- bool **isConversationEnabled** () const
- void **setMaxHistoryMessages** (unsigned int value)
- unsigned int **getMaxHistoryMessages** () const
- void **setClearHistoryBetweenReplications** (bool value)
- bool **isClearHistoryBetweenReplications** () const
- unsigned long **getCallCount** () const
- unsigned long **getFailureCount** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

- Returns the full map of internal child data definitions.*
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelDataDefinition](#)

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_initBetweenReplications` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification _id
- std::string _typename
- bool _reportStatistics
- bool _hasChanged
- unsigned int _modelLevel = 0
- [Model](#) * _parentModel
- [List](#)< [SimulationControl](#) * > * _simulationControls = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level _traceLevelSpecific
- bool _traceLevelSpecificEnabled = false

10.8.1 Member Function Documentation

10.8.1.1 _check()

```
bool AISupport::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.8.1.2 _initBetweenReplications()

```
void AISupport::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.8.1.3 _loadInstance()

```
bool AISupport::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.8.1.4 `_saveInstance()`

```
void AISupport::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.8.1.5 `show()`

```
std::string AISupport::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

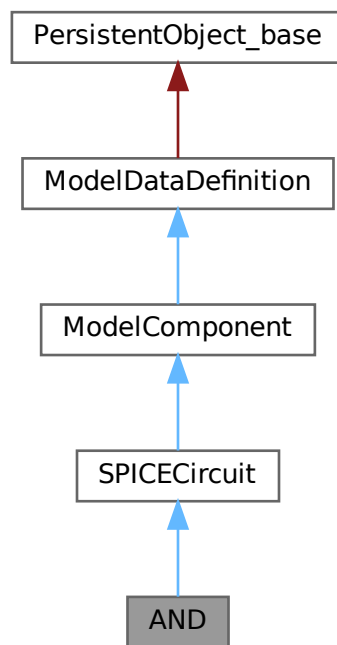
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

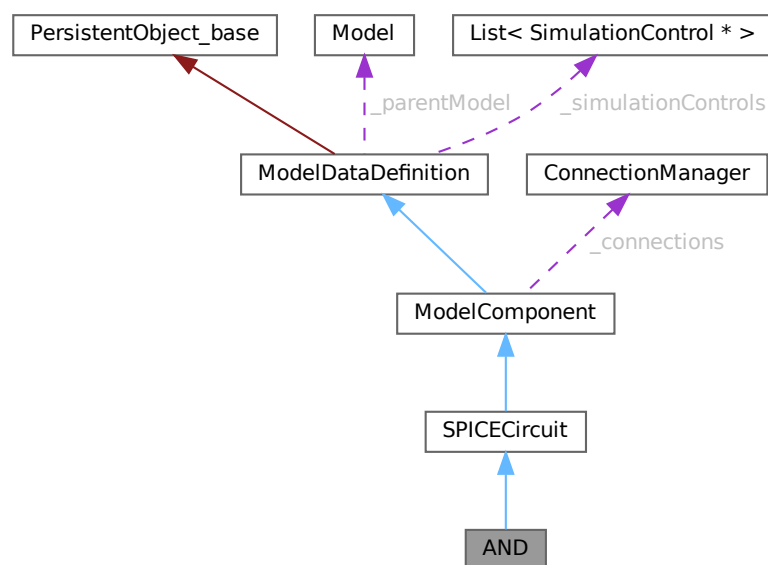
- `plugins/data/AI/AISupport.h`
- `plugins/data/AI/AISupport.cpp`

10.9 AND Class Reference

Inheritance diagram for AND:



Collaboration diagram for AND:



Public Member Functions

- **AND** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setPMOSWidth** (float PMOSWidth)
- void **setPMOSLength** (float length)
- void **setNMOSWidth** (float NMOSWidth)
- void **setNMOSLength** (float NMOSLength)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

- Updates the logical name of this instance.*
- `const std::string & getName () const`
- Returns the logical name of this instance.*
- `void setLabel (const std::string &label)`
- Updates the user-visible label of this instance.*
- `const std::string & getLabel () const`
- Returns the user-visible label of this instance.*
- `const std::string & getClassname () const`
- Returns the runtime class name used by the plugin system.*
- `Model * getParentModel () const`
- Returns the parent model that owns this definition.*
- `bool isReportStatistics () const`
- Indicates whether this data definition contributes report statistics.*
- `void setReportStatistics (bool reportStatistics)`
- Enables or disables report-statistics generation for this definition.*
- `ModelDataDefinition * getInternalData (const std::string &name) const`
- Looks up a named internal child data definition.*
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
- Returns the full map of internal child data definitions.*
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
- Returns the full map of attached data definitions.*
- `bool hasChanged () const`
- Indicates whether this instance has been modified since the last reset.*
- `void setHasChanged (bool hasChanged)`
- Updates the changed flag for this data definition.*
- `unsigned int getLevel () const`
- Returns the level in the model hierarchy where this definition lives.*
- `void setModelLevel (unsigned int _modelLevel)`
- Sets the model nesting level for this definition.*
- `List< SimulationControl * > * getSimulationControls () const`
- Returns the simulation controls owned by this data definition.*
- `TraceManager::Level getTraceLevelSpecific () const`
- Returns the trace level that applies specifically to this definition.*
- `void setTraceLevelSpecific (TraceManager::Level level)`
- Updates the trace level used by this definition.*
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
- Defines the specific trace level and optionally enables it immediately.*
- `bool isTraceLevelSpecificEnabled () const`
- Indicates whether the specific trace level is enabled.*
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
- Enables or disables the specific trace level.*

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) * **GetPluginInformation** ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
- static std::vector< std::string > **split** (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void **_createNonEditableDataDefinitions** () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual bool **_check** (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_initBetweenReplications** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)

- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.9.1 Member Function Documentation

10.9.1.1 [_loadInstance\(\)](#)

```
bool AND::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.9.1.2 `_saveInstance()`

```
void AND::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.9.1.3 Build()

```
void AND::Build ( ) [override], [virtual]
```

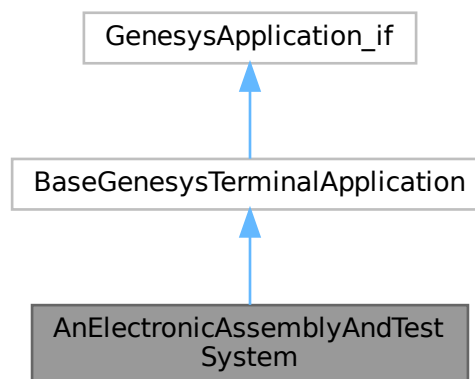
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

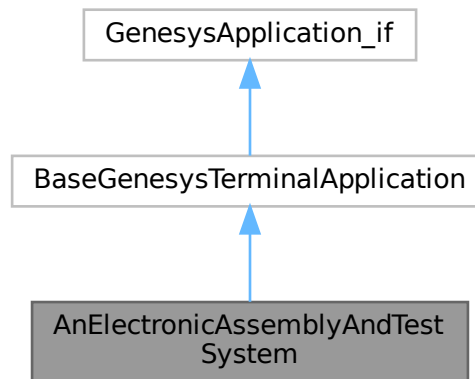
- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

10.10 AnElectronicAssemblyAndTestSystem Class Reference

Inheritance diagram for AnElectronicAssemblyAndTestSystem:



Collaboration diagram for AnElectronicAssemblyAndTestSystem:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.10.1 Member Function Documentation

10.10.1.1 main()

```
int AnElectronicAssemblyAndTestSystem::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

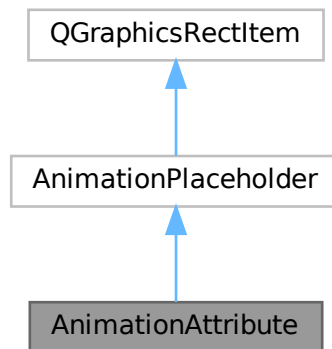
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

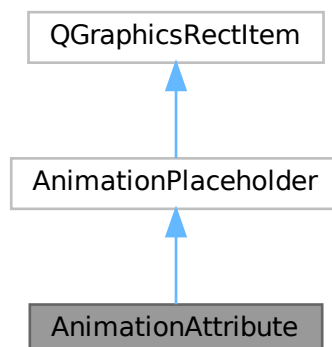
- applications/modelSpecific/teaching/AnElectronicAssemblyAndTestSystem.h
- applications/modelSpecific/teaching/AnElectronicAssemblyAndTestSystem.cpp

10.11 AnimationAttribute Class Reference

Inheritance diagram for AnimationAttribute:



Collaboration diagram for AnimationAttribute:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

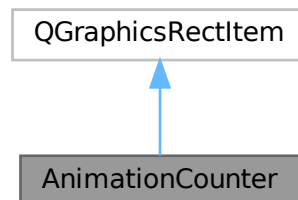
- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

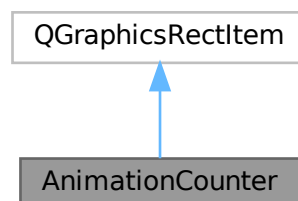
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

10.12 AnimationCounter Class Reference

Inheritance diagram for AnimationCounter:



Collaboration diagram for AnimationCounter:



Public Member Functions

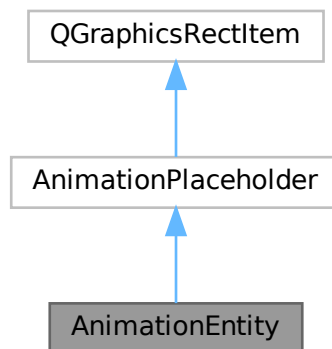
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- double **getValue** ()
- Counter * **getCounter** ()
- void **setValue** (double value)
- void **setCounter** (Counter *myCounter)
- void **setIdCounter** (int id)
- void **setNameCounter** (std::string name)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** ()
- bool **isDrawingFinalized** ()
- void **setWhenLoaded** (QList< Counter * > *counters)

The documentation for this class was generated from the following files:

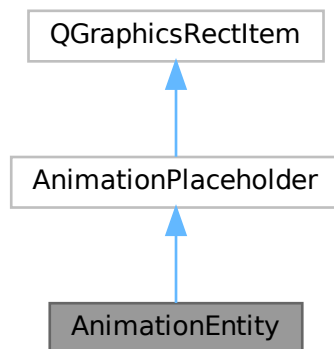
- applications/gui/genesys/animations/AnimationCounter.h
- applications/gui/genesys/animations/AnimationCounter.cpp

10.13 AnimationEntity Class Reference

Inheritance diagram for AnimationEntity:



Collaboration diagram for AnimationEntity:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

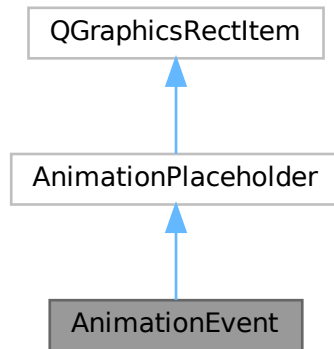
- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

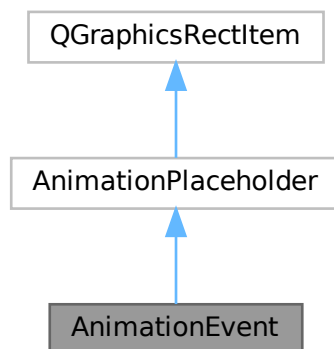
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

10.14 AnimationEvent Class Reference

Inheritance diagram for AnimationEvent:



Collaboration diagram for AnimationEvent:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)

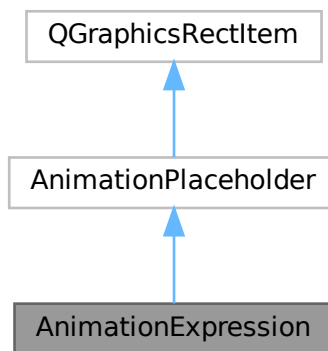
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

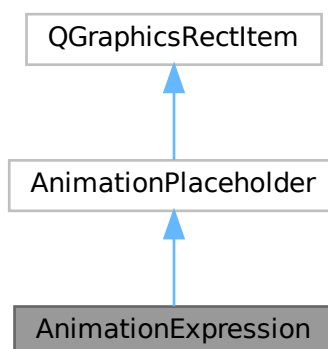
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

10.15 AnimationExpression Class Reference

Inheritance diagram for AnimationExpression:



Collaboration diagram for AnimationExpression:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

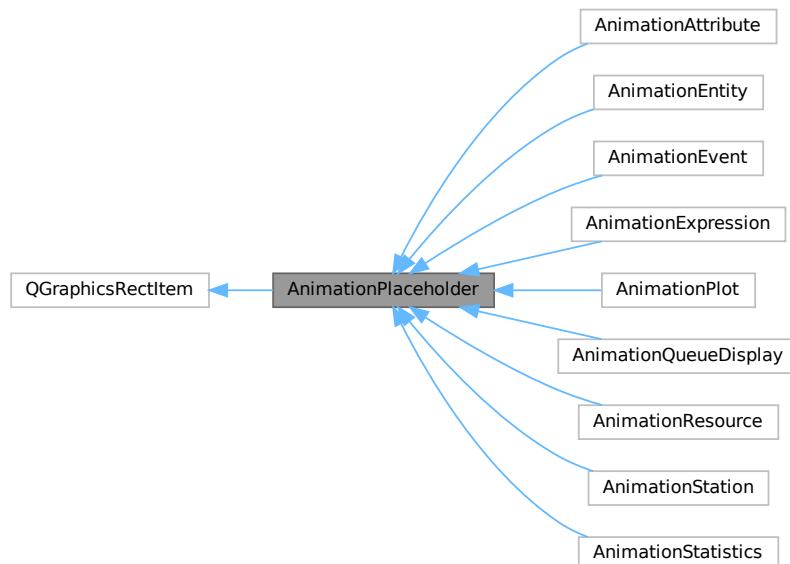
- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

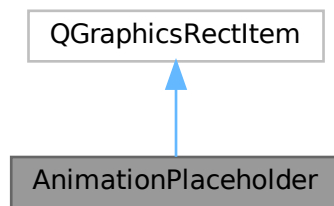
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

10.16 AnimationPlaceholder Class Reference

Inheritance diagram for AnimationPlaceholder:



Collaboration diagram for AnimationPlaceholder:



Public Member Functions

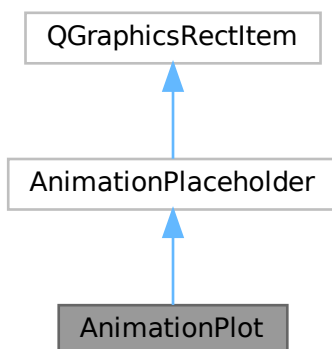
- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

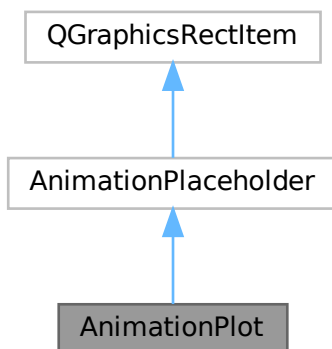
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

10.17 AnimationPlot Class Reference

Inheritance diagram for AnimationPlot:



Collaboration diagram for AnimationPlot:



Classes

- struct [DataSeries](#)

Public Types

- enum class **PlotType** { **Line** = 0 , **Bar** = 1 }

Public Member Functions

- QString **getTitle** () const
- void **setTitle** (const QString &title)
- QString **getYAxisTitle** () const
- void **setYAxisTitle** (const QString &title)
- QString **getXAxisTitle** () const
- void **setXAxisTitle** (const QString &title)
- PlotType **getPlotType** () const
- void **setPlotType** (PlotType type)
- QString **getDatasetsText** () const
- void **setDatasetsText** (const QString &datasetsText)
- bool **getShowGridLines** () const
- void **setShowGridLines** (bool show)
- bool **getShowTicks** () const
- void **setShowTicks** (bool show)
- double **getXAxisMin** () const
- void **setXAxisMin** (double value)
- double **getXAxisMax** () const
- void **setXAxisMax** (double value)
- double **getYAxisMin** () const
- void **setYAxisMin** (double value)
- double **getYAxisMax** () const
- void **setYAxisMax** (double value)
- void **clearValues** ()
- void **appendSample** ([Model](#) *model)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override

Public Member Functions inherited from [AnimationPlaceholder](#)

- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

10.18 AnimationQueue Class Reference

Public Member Functions

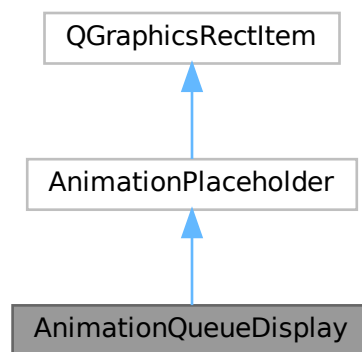
- **AnimationQueue** ([ModelGraphicsScene](#) *myScene, [ModelComponent](#) *component)
- [GraphicalModelComponent](#) * **getGraphicalComponent** () const
- void **addAnimationQueue** (bool visible)
- void **removeAnimationQueue** ()
- void **verifyAddAnimationQueue** (bool visible)
- void **verifyRemoveAnimationQueue** ()
- QPointF **calculatePositionImageQueue** (unsigned int indexQueue, unsigned int sizeQueue, unsigned int width, unsigned int height)

The documentation for this class was generated from the following files:

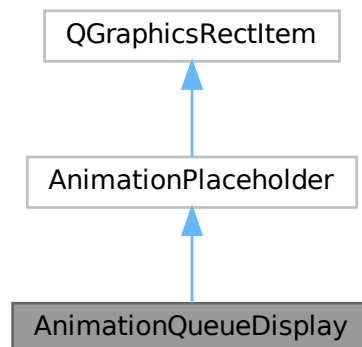
- applications/gui/genesys/animations/AnimationQueue.h
- applications/gui/genesys/animations/AnimationQueue.cpp

10.19 AnimationQueueDisplay Class Reference

Inheritance diagram for AnimationQueueDisplay:



Collaboration diagram for AnimationQueueDisplay:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

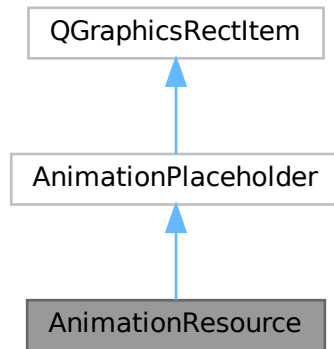
- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

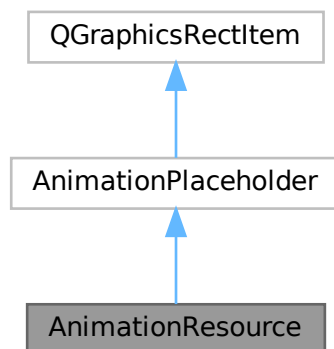
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

10.20 AnimationResource Class Reference

Inheritance diagram for AnimationResource:



Collaboration diagram for AnimationResource:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)

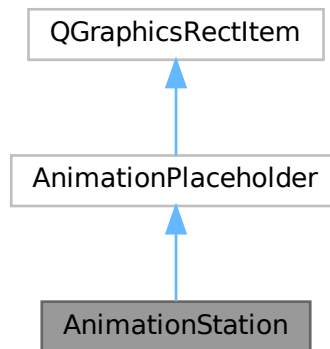
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

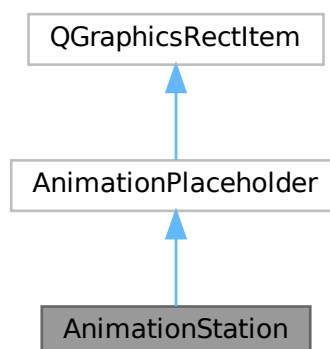
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

10.21 AnimationStation Class Reference

Inheritance diagram for AnimationStation:



Collaboration diagram for AnimationStation:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

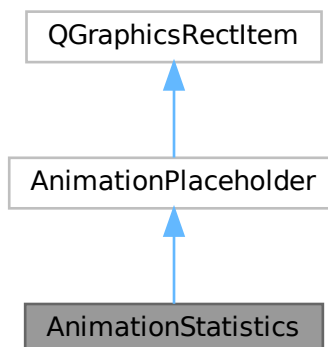
- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

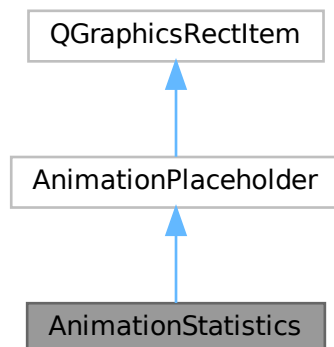
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

10.22 AnimationStatistics Class Reference

Inheritance diagram for AnimationStatistics:



Collaboration diagram for AnimationStatistics:



Additional Inherited Members

Public Member Functions inherited from [AnimationPlaceholder](#)

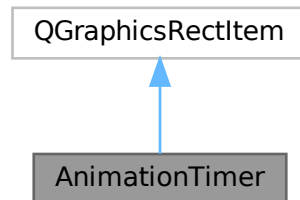
- **AnimationPlaceholder** (const QString &animationType)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QString **getAnimationType** () const
- QString **getTargetName** () const
- void **setTargetName** (const QString &targetName)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** () const
- bool **isDrawingFinalized** () const

The documentation for this class was generated from the following files:

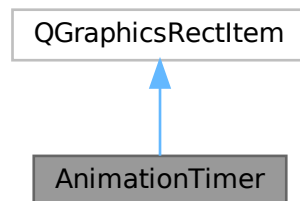
- applications/gui/genesys/animations/AnimationPlaceholder.h
- applications/gui/genesys/animations/AnimationPlaceholder.cpp

10.23 AnimationTimer Class Reference

Inheritance diagram for AnimationTimer:



Collaboration diagram for AnimationTimer:



Public Member Functions

- **AnimationTimer** ([ModelGraphicsScene](#) *myScene)
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- double **getTime** ()
- unsigned int **getInitialHours** ()
- unsigned int **getInitialMinutes** ()
- unsigned int **getInitialSeconds** ()
- unsigned int **getHours** ()
- unsigned int **getMinutes** ()
- unsigned int **getSeconds** ()
- Util::TimeFormat **getTimeFormat** ()
- void **setTime** (double value)
- void **setInitialHours** (unsigned int initialHours)
- void **setInitialMinutes** (unsigned int initialMinutes)
- void **setInitialSeconds** (unsigned int initialSeconds)
- void **setHours** (unsigned int hours)
- void **setMinutes** (unsigned int minutes)

- void **setSeconds** (unsigned int seconds)
- void **setTimeFormat** (Util::TimeFormat timeFormat)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- void **convertSeconds** ()
- bool **isDrawingInialized** ()
- bool **isDrawingFinalized** ()

Static Public Member Functions

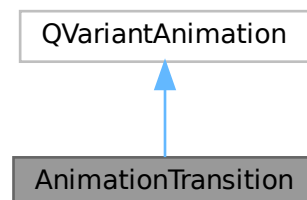
- static void **setConversionFactorToSeconds** (double factor)

The documentation for this class was generated from the following files:

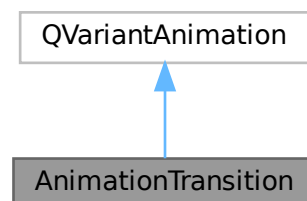
- applications/gui/genesys/animations/AnimationTimer.h
- applications/gui/genesys/animations/AnimationTimer.cpp

10.24 AnimationTransition Class Reference

Inheritance diagram for AnimationTransition:



Collaboration diagram for AnimationTransition:



Public Slots

- void **onAnimationValueChanged** (const QVariant &value)
- void **onAnimationFinished** ()

Public Member Functions

- **AnimationTransition** ([ModelGraphicsScene](#) *myScene, [ModelComponent](#) *graphicalStartComponent, [ModelComponent](#) *graphicalEndComponent, bool viewSimulation)
- [GraphicalModelComponent](#) * **getGraphicalStartComponent** () const
- [GraphicalModelComponent](#) * **getGraphicalEndComponent** () const
- [GraphicalConnection](#) * **getGraphicalConnection** () const
- QList< QPointF > **getPointsForAnimation** () const
- [GraphicalImageAnimation](#) * **getImageAnimation** () const
- unsigned int **getPortNumber** () const
- bool **isReadyToRun** () const
- void **setImageAnimation** ([GraphicalImageAnimation](#) *imageAnimation)
- void **startAnimation** ()
- void **stopAnimation** ()
- void **restartAnimation** ()
- void **configureAnimation** ()
- void **updateDurationIfNeeded** ()
- void **connectValueChangedSignal** ()
- void **connectFinishedSignal** ()

Static Public Member Functions

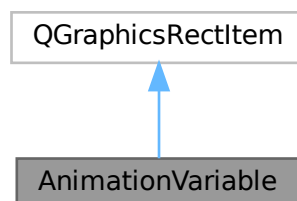
- static double **getTimeExecution** ()
- static void **setTimeExecution** (double timeExecution)
- static void **setPause** (bool pause)
- static void **setRunning** (bool running)

The documentation for this class was generated from the following files:

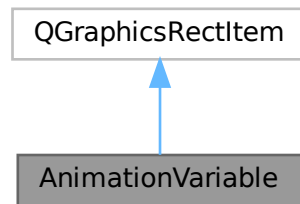
- applications/gui/genesys/animations/AnimationTransition.h
- applications/gui/genesys/animations/AnimationTransition.cpp

10.25 AnimationVariable Class Reference

Inheritance diagram for AnimationVariable:



Collaboration diagram for AnimationVariable:



Public Member Functions

- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- double **getValue** ()
- [ModelDataDefinition](#) * **getDataDefinition** () const
- [Variable](#) * **getVariable** ()
- [Attribute](#) * **getAttribute** ()
- QString **getTargetName** () const
- int **getTargetId** () const
- unsigned int **getDimensionCount** () const
- unsigned int **getDimensionSize** (unsigned int dimensionIndex) const
- unsigned int **getSliceIndex** (unsigned int dimensionIndex) const
- void **setValue** (double value)
- void **setDataDefinition** ([ModelDataDefinition](#) *definition)
- void **setVariable** ([Variable](#) *variable)
- void **setAttribute** ([Attribute](#) *attribute)
- void **setModel** ([Model](#) *model)
- void **setTargetName** (std::string name)
- void **setIdVariable** (int id)
- void **setSliceIndex** (unsigned int dimensionIndex, unsigned int value)
- void **setWhenLoaded** (QList< [ModelDataDefinition](#) * > *definitions)
- void **refreshValue** ([Entity](#) *entity=nullptr)
- void **startDrawing** (QGraphicsSceneMouseEvent *event)
- void **continueDrawing** (QGraphicsSceneMouseEvent *event)
- void **stopDrawing** (QGraphicsSceneMouseEvent *event)
- void **adjustSizeAndPosition** (QGraphicsSceneMouseEvent *event)
- bool **isDrawingInialized** ()
- bool **isDrawingFinalized** ()

The documentation for this class was generated from the following files:

- applications/gui/genesys/animations/AnimationVariable.h
- applications/gui/genesys/animations/AnimationVariable.cpp

10.26 ApiRouter Class Reference

Routes HTTP requests to Web API operations.

```
#include <applications/worker/api/ApiRouter.h>
```

Public Member Functions

- [ApiRouter](#) ([SimulatorSessionService](#) &simulatorService)
Creates an API router bound to the simulator session service.
- [HttpResponse](#) [handle](#) (const [HttpRequest](#) &request) const
Handles one HTTP request and returns the corresponding response.

10.26.1 Detailed Description

Routes HTTP requests to Web API operations.

10.26.2 Constructor & Destructor Documentation

10.26.2.1 ApiRouter()

```
ApiRouter::ApiRouter (
    SimulatorSessionService & simulatorService ) [explicit]
```

Creates an API router bound to the simulator session service.

Dispatches HTTP routes for the Genesys web API.

Parameters

<i>simulatorService</i>	Service used to execute API operations.
-------------------------	---

The router keeps request parsing, token extraction, payload decoding and JSON formatting in one place so the HTTP server can stay transport-only.

10.26.3 Member Function Documentation

10.26.3.1 handle()

```
HttpResponse ApiRouter::handle (
    const HttpRequest & request ) const
```

Handles one HTTP request and returns the corresponding response.

Resolves a request into one of the public API operations.

Parameters

<i>request</i>	HTTP request containing method, path, headers, and body.
----------------	--

Returns

HTTP response with status, content type, and JSON payload.

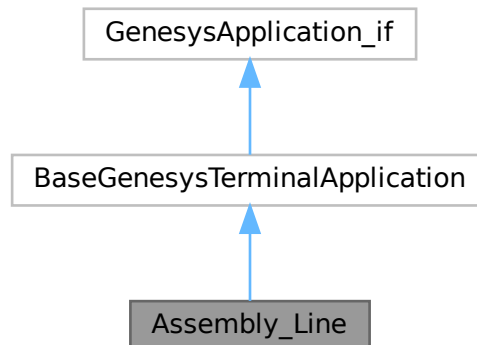
The route chain stays intentionally explicit so the generated PDF makes the supported endpoints and their validation rules easy to audit.

The documentation for this class was generated from the following files:

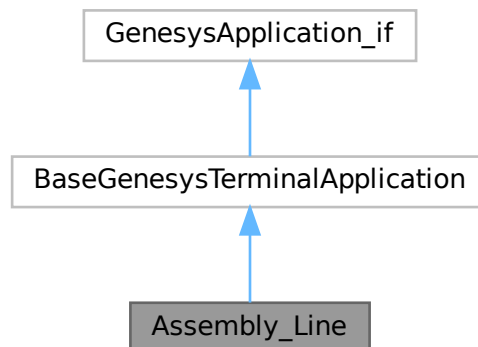
- applications/worker/api/ApiRouter.h
- applications/worker/api/ApiRouter.cpp

10.27 Assembly_Line Class Reference

Inheritance diagram for Assembly_Line:



Collaboration diagram for `Assembly_Line`:



Public Member Functions

- virtual int `main` (int argc, char **argv) override

Public Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void `Simulate` (`Simulator` *genesys, `Model` *model, `PluginManager` *plugins)
- void `setDefaultTraceHandlers` (`TraceManager` *tm)
- void `setDefaultEventHandlers` (`OnEventManager` *oem)

Additional Inherited Members

Protected Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void `traceHandler` (`TraceEvent` e)
- virtual void `traceErrorHandler` (`TraceErrorEvent` e)
- virtual void `traceReportHandler` (`TraceEvent` e)
- virtual void `traceSimulationHandler` (`TraceSimulationEvent` e)
- virtual void `onAfterProcessEventHandler` (`SimulationEvent` *re)
- virtual void `onBreakpointHandler` (`SimulationEvent` *re)
- virtual void `onEntityCreateHandler` (`SimulationEvent` *re)
- virtual void `onEntityMoveHandler` (`SimulationEvent` *re)
- virtual void `onSimulationStartHandler` (`SimulationEvent` *re)
- virtual void `onReplicationStepHandler` (`SimulationEvent` *re)
- virtual void `onReplicationStartHandler` (`SimulationEvent` *re)
- virtual void `onProcessEventHandler` (`SimulationEvent` *re)
- virtual void `onReplicationEndHandler` (`SimulationEvent` *re)
- virtual void `onSimulationEndHandler` (`SimulationEvent` *re)
- virtual void `onSimulationPausedHandler` (`SimulationEvent` *re)
- virtual void `onSimulationResumeHandler` (`SimulationEvent` *re)
- virtual void `onEntityRemoveHandler` (`SimulationEvent` *re)

10.27.1 Member Function Documentation

10.27.1.1 main()

```
int Assembly_Line::main (
    int argc,
    char ** argv )  [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

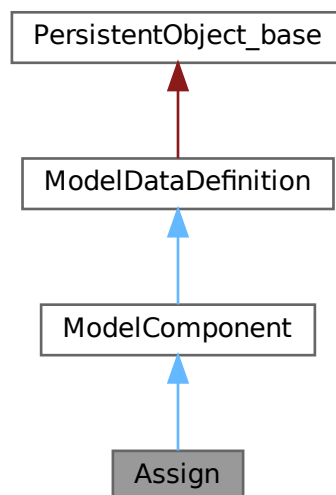
The documentation for this class was generated from the following files:

- applications/modelSpecific/arenaExamples/Assembly_Line.h
- applications/modelSpecific/arenaExamples/Assembly_Line.cpp

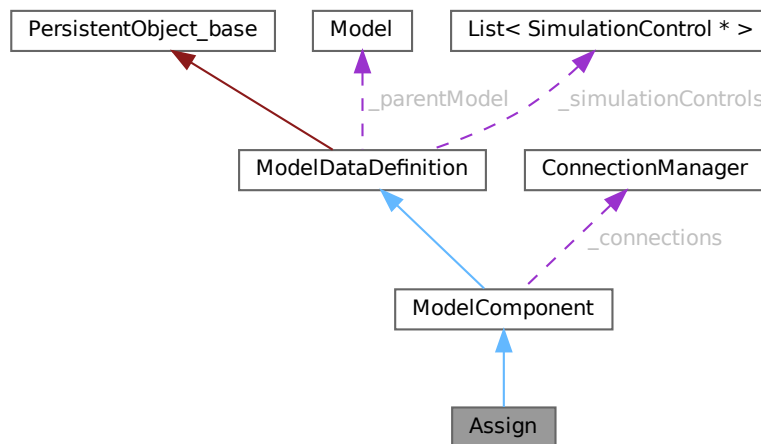
10.28 Assign Class Reference

```
#include <plugins/components/Logic/Assign.h>
```

Inheritance diagram for Assign:



Collaboration diagram for Assign:



Public Member Functions

- **Assign** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- **List< Assignment * >** * **getAssignments** () const
- void **addAssignment** ([Assignment](#) *newAssignment)
- void **removeAssignment** ([Assignment](#) *assignment)

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- *new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- *Connects this component to another component.*
- bool **hasBreakpointAt** ()
- *Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string _description)
- *Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const
- *Returns the free-form description associated with this component.*
- void **remove** ([ModelComponent](#) *comp)
- *Removes a connected component from this component.*

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_createEditableDataDefinitions** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.28.1 Detailed Description

[Assign](#) module DESCRIPTION This module is used for assigning new values to variables, entity attributes, entity types, entity pictures, or other system variables. Multiple assignments can be made with a single [Assign](#) module. TYPICAL USES Accumulate the number of subassemblies added to a part Change an entity's type to represent the customer copy of a multi-page form Establish a customer's priority PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Assignments Specifies the one or more assignments that will be made when an entity executes the module. Type Type of assignment to be made. Other can include system variables, such as resource capacity or simulation end time. [Variable](#) Name Name of the variable that will be assigned a new value when an entity enters the module. Applies only when Type is [Variable](#), [Variable](#) Array (1D), or [Variable](#) Array (2D). Row Specifies the row index for a variable array. Column Specifies the column index for a variable array. [Attribute](#) Name Name of the entity attribute that will be assigned a new value when the entity enters the module. Applies only when Type is [Attribute](#). [Entity](#) Type New entity type that will be assigned to the entity when the entity enters the module. Applies only when Type is [Entity](#) Type. [Entity](#) Picture New entity picture that will be assigned to the entity when the entity enters the module. Applies only when Type is [Entity](#) Picture. Other Identifies the special system variable that will be assigned a new value when an entity enters the module. Applies only when Type is Other. New Value [Assignment](#) value of the attribute, variable, or other system variable. Does not apply when Type is [Entity](#) Type or [Entity](#) Picture.

10.28.2 Member Function Documentation

10.28.2.1 `_check()`

```
bool Assign::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.28.2.2 _createEditableDataDefinitions()

```
void Assign::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

10.28.2.3 _loadInstance()

```
bool Assign::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.28.2.4 _onDispatchEvent()

```
void Assign::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.28.2.5 _saveInstance()

```
void Assign::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.28.2.6 show()

```
std::string Assign::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/Logic/Assign.h
- plugins/components/Logic/Assign.cpp

10.29 Assignment Class Reference

```
#include <plugins/data/Logic/AssignmentItem.h>
```

Public Member Functions

- **Assignment** ([Model](#) *model, std::string destination, std::string expression="1", bool isAttributeNotVariable=true)
- **Assignment** (std::string destination, std::string expression, bool isAttributeNotVariable=true)
- void **setDestination** (std::string _destination)
- std::string **getDestination** () const
- void **setExpression** (std::string _expression)
- std::string **getExpression** () const
- void **setAttributeNotVariable** (bool isAttributeNotVariable)
- bool **isAttributeNotVariable** () const
- void **setChangeCallback** (std::function< void([Assignment](#) &)> callback)
- std::string **getName** () const
- [List](#)< [SimulationControl](#) * > * **getSimulationControls** () const
- void **ensureSimulationControls** ([Model](#) *model)
- void **_addSimulationControl** ([SimulationControl](#) *control)
- std::string **getTypeDC** ()
- bool **loadInstance** ([PersistenceRecord](#) *fields, unsigned int parentIndex)
- void **saveInstance** ([PersistenceRecord](#) *fields, unsigned int parentIndex, bool saveDefault)

10.29.1 Detailed Description

While the assign ans sequence classes allows you to perform multiple assignments, the assignmentItem class defines an assignment itself.

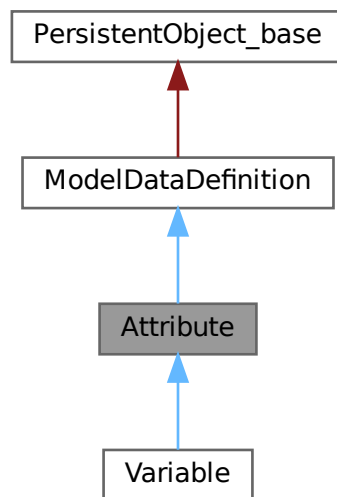
The documentation for this class was generated from the following files:

- plugins/data/Logic/AssignmentItem.h
- plugins/data/Logic/AssignmentItem.cpp

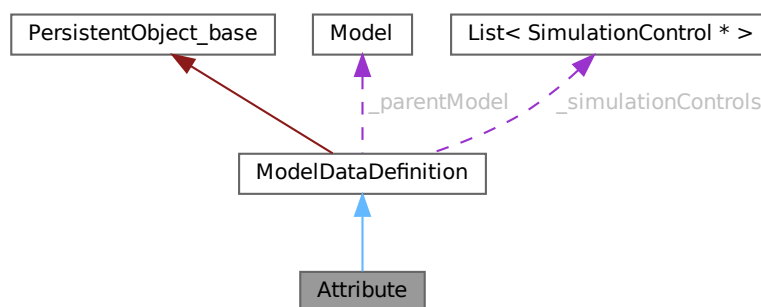
10.30 Attribute Class Reference

```
#include <kernel/simulator/essentialPlugins/Attribute.h>
```

Inheritance diagram for Attribute:



Collaboration diagram for Attribute:



Public Member Functions

- **Attribute** ([Model](#) *model, std::string name="", std::string dataDefinitionType=Util::TypeOf< [Attribute](#) >())
Creates an entity attribute definition in the model.
- virtual ~**Attribute** () override
Releases sparse initial values owned by this definition.
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- virtual std::string **getInitialValuesText** () const
Returns the initial values serialized in bracket notation.
- virtual void **setInitialValuesText** (std::string valuesText)
Parses bracket notation and replaces the initial value store.
- double **getInitialValue** (std::string index="")
Reads the initial sparse value, returning 0.0 when the index is absent.
- void **setInitialValue** (double value, std::string index="")
Writes the initial sparse value at the scalar or indexed position.
- void **setInitialValues** (const std::vector< std::pair< std::string, double > > values)
Replaces/extends initial sparse values from textual index/value pairs.
- void **copyInitialValuesFrom** (const [Attribute](#) &source)
Copies the full initial sparse store from another attribute.
- void **insertDimentionSize** (unsigned int size)
Appends one dimension size to the attribute definition.
- std::list< unsigned int > * **getDimensionSizes** () const
Returns dimension sizes for compatibility with existing callers.
- std::map< std::string, double > * **getInitialValues** () const
Returns mutable initial sparse values for compatibility with existing callers.
- [SparseValueStore](#) * **getInitialValueStore** ()
Returns the sparse initial value store used by this attribute.
- bool **isInitialValuesTextValid** () const
Returns whether the last textual initial value is syntactically valid.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string dataDefinitionType, std::string name="", bool insert↔IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

- Returns the runtime class name used by the plugin system.*
- **Model * getParentModel () const**
Returns the parent model that owns this definition.
- **bool isReportStatistics () const**
Indicates whether this data definition contributes report statistics.
- **void setReportStatistics (bool reportStatistics)**
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition * getInternalData (const std::string &name) const**
Looks up a named internal child data definition.
- **std::map< std::string, ModelDataDefinition * > * getInternalData () const**
Returns the full map of internal child data definitions.
- **std::map< std::string, ModelDataDefinition * > * getAttachedData () const**
Returns the full map of attached data definitions.
- **bool hasChanged () const**
Indicates whether this instance has been modified since the last reset.
- **void setHasChanged (bool hasChanged)**
Updates the changed flag for this data definition.
- **unsigned int getLevel () const**
Returns the level in the model hierarchy where this definition lives.
- **void setModelLevel (unsigned int _modelLevel)**
Sets the model nesting level for this definition.
- **List< SimulationControl * > * getSimulationControls () const**
Returns the simulation controls owned by this data definition.
- **TraceManager::Level getTraceLevelSpecific () const**
Returns the trace level that applies specifically to this definition.
- **void setTraceLevelSpecific (TraceManager::Level level)**
Updates the trace level used by this definition.
- **void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)**
Defines the specific trace level and optionally enables it immediately.
- **bool isTraceLevelSpecificEnabled () const**
Indicates whether the specific trace level is enabled.
- **void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)**
Enables or disables the specific trace level.

Static Public Member Functions

- **static PluginInformation * GetPluginInformation ()**
- **static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)**
- **static ModelDataDefinition * NewInstance (Model *model, std::string name="")**

Static Public Member Functions inherited from ModelDataDefinition

- **static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)**
Loads a model data definition instance from serialized fields.
- **static ModelDataDefinition * NewInstance (Model *model, std::string name="")**
Creates a new instance of a data definition using the plugin registry.
- **static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)**
Serializes a data definition instance into a persistence record.
- **static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)**

Validates a data definition instance.

- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)

- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.30.1 Detailed Description

[Attribute](#) module DESCRIPTION This data module is used to define an attribute's dimension, data type and initial value(s). An attribute is a characteristic of all entities created, but with a specific value that can differ from one entity to another. Attributes can be referenced in other modules (for example, the [Decide](#) module), can be reassigned a new value with the [Assign](#) module, and can be used in any expression. [Attribute](#) values are unique for each entity, as compared to Variables which are global to the simulation module. There are three methods for manually editing the Initial Values of an [Attribute](#) module:

- Using the standard spreadsheet interface. In the module spreadsheet, right-click on the Initial Values cell and select the Edit via spreadsheet menu item. The values for two-dimensional arrays should be entered one column at a time. Array elements not explicitly assigned are assumed to have the last entered value.
- Using the module dialog box. In the module spreadsheet, right-click on any cell and select the Edit via dialog menu item. The values for two-dimensional arrays should be entered one column at a time. Array elements not explicitly assigned are assumed to have the last entered value.

- Using the two-dimensional (2-D) spreadsheet interface. In the module spreadsheet, click on the Initial Values cell. TYPICAL USES Due date of an order (entity) Priority of an order (entity) Color of a part (entity) PROMPTS Prompt Description Name The unique name of the attribute being defined. Rows Number of rows in a one- or two-dimensional attribute. Columns Number of columns in a two-dimensional attribute. Data Type The data type of the values stored in the attribute. Valid types are Real and String. The default type is Real. Initial Values Lists the initial value or values of the attribute. You can assign new values to the attribute by using the [Assign](#) module. Initial Value [Entity](#) attribute value when entity is created and enters the system.

10.30.2 Member Function Documentation

10.30.2.1 `_check()`

```
bool Attribute::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [Variable](#).

10.30.2.2 `_loadInstance()`

```
bool Attribute::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [Variable](#).

10.30.2.3 `_saveInstance()`

```
void Attribute::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [Variable](#).

10.30.2.4 `getInitialValuesText()`

```
std::string Attribute::getInitialValuesText ( ) const [virtual]
```

Returns the initial values serialized in bracket notation.

Reimplemented in [Variable](#).

10.30.2.5 `setInitialValuesText()`

```
void Attribute::setInitialValuesText (
    std::string valuesText ) [virtual]
```

Parses bracket notation and replaces the initial value store.

Reimplemented in [Variable](#).

10.30.2.6 `show()`

```
std::string Attribute::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [Variable](#).

The documentation for this class was generated from the following files:

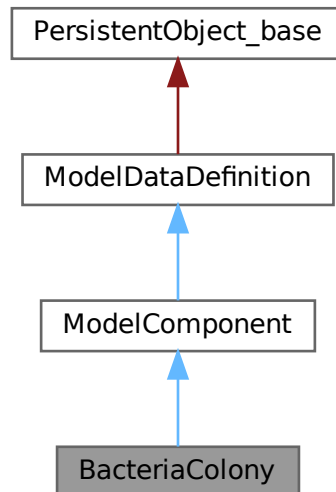
- kernel/simulator/essentialPlugins/Attribute.h
- kernel/simulator/essentialPlugins/Attribute.cpp

10.31 BacteriaColony Class Reference

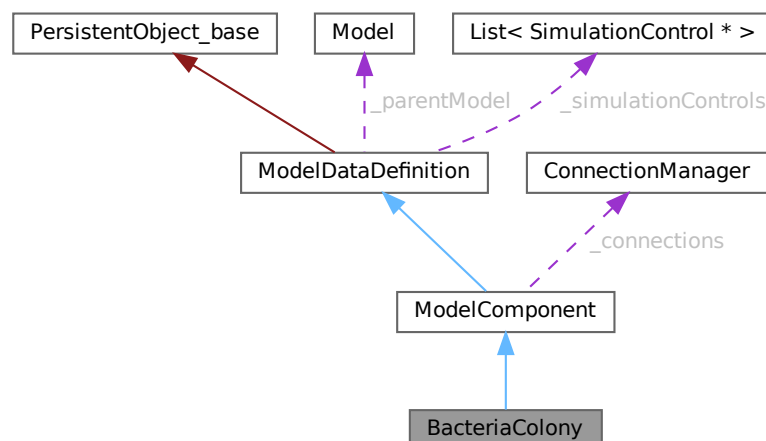
First GenESyS component for Gro-inspired bacteria colony simulation.

```
#include <plugins/components/BiochemicalSimulation/BacteriaColony.h>
```

Inheritance diagram for BacteriaColony:



Collaboration diagram for BacteriaColony:



Classes

- struct [BacteriumState](#)
- struct [BarrierSegment](#)

Public Member Functions

- **BacteriaColony** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
virtual public methods
- void **setGroProgram** ([GroProgram](#) *groProgram)
Sets the reusable Gro program used by this colony.
- [GroProgram](#) * **getGroProgram** () const
Returns the reusable Gro program used by this colony.
- void **setSimulationStep** (double simulationStep)
Sets the internal colony time increment used by one colony step.
- double **getSimulationStep** () const
Returns the internal colony time increment.
- void **setNumSteps** (unsigned int numSteps)
Sets the number of colony steps processed before forwarding one entity.
- unsigned int **getNumSteps** () const
Returns the number of colony steps processed before forwarding one entity.
- void **setColonyTimeUnit** ([Util::TimeUnit](#) colonyTimeUnit)
Sets the time unit used by the colony time-related attributes.
- [Util::TimeUnit](#) **getColonyTimeUnit** () const
Returns the time unit used by the colony time-related attributes.
- double **getColonyTime** () const
Returns the current colony time from the enclosing [ModelSimulation](#).
- void **setInitialPopulation** (unsigned int initialPopulation)
Sets the initial bacteria population summary for each replication.
- unsigned int **getInitialPopulation** () const
Returns the initial bacteria population summary.
- unsigned int **getPopulationSize** () const
Returns the current bacteria population summary.
- void **setBioNetwork** ([BioNetwork](#) *bioNetwork)
Sets the optional [BioNetwork](#) reused as the colony biochemical state.
- [BioNetwork](#) * **getBioNetwork** () const
Returns the optional [BioNetwork](#) reused as the colony biochemical state.
- void **setSignalGrid** ([BacteriaSignalGrid](#) *signalGrid)
Sets the reusable signal-grid definition used by this colony.
- [BacteriaSignalGrid](#) * **getSignalGrid** () const
Returns the reusable signal-grid definition used by this colony.
- std::size_t **getInternalBacteriaCount** () const
Returns the number of internal bacteria currently owned by the colony.
- const [BacteriumState](#) & **getBacteriumState** (std::size_t index) const
Returns one internal bacterium state by zero-based index.
- double **getBacteriumAge** (std::size_t index) const
Returns the age of one internal bacterium at the current colony time.
- double **getBacteriumPositionX** (std::size_t index) const
Returns the continuous X coordinate used by the viewer.
- double **getBacteriumPositionY** (std::size_t index) const

- Returns the continuous Y coordinate used by the viewer.*

 - double **getBacteriumDirectionRadians** (std::size_t index) const

Returns the bacterium direction in radians.
- double **getBacteriumVolume** (std::size_t index) const

Returns the current bacterium volume.
- double **getBacteriumSize** (std::size_t index) const

Returns the current bacterium size used for rendering.
- double **getBacteriumGfp** (std::size_t index) const

Returns the current GFP amount used for rendering.
- double **getBacteriumRfp** (std::size_t index) const

Returns the current RFP amount used for rendering.
- double **getBacteriumRuntimeVariableValue** (std::size_t index, const std::string &variableName) const

Returns the last stored value for one bacterium-scoped runtime variable.
- bool **hasBacteriumRuntimeVariable** (std::size_t index, const std::string &variableName) const

Tells whether one bacterium currently stores one runtime scalar variable.
- double **getRuntimeVariableValue** (const std::string &variableName) const

Returns the last stored value for one runtime scalar variable.
- bool **hasRuntimeVariable** (const std::string &variableName) const

Tells whether the colony currently stores one runtime scalar variable.
- double **getSignalValueAt** (unsigned int x, unsigned int y) const

Returns the runtime signal value stored at one grid coordinate.
- std::vector< std::vector< double > > **getSignalMatrix** () const

Returns the current runtime signal field as a row-major matrix.
- std::string **getSignalMatrixDump** (std::size_t maxRows=0, std::size_t maxColumns=0) const

Returns a textual dump of the current runtime signal field.
- double **getBacteriumLocalSignal** (std::size_t index) const

Returns the runtime signal value seen by one bacterium at its current cell.
- void **setChemostatMode** (bool enabled)

Enables or disables the colony chemostat mode used by Gro microfluidics.
- bool **getChemostatMode** () const

Returns whether the colony chemostat mode is enabled.
- void **addBarrier** (double x1, double y1, double x2, double y2)

Adds one barrier segment to the colony environment.
- const std::vector< [BarrierSegment](#) > & **getBarriers** () const

Returns the current barrier segments used by the viewer.
- const std::vector< double > & **getMappedCellValues** () const

Returns the latest per-cell values produced by map_to_cells, if any.
- const std::string & **getMappedCellExpression** () const

Returns the expression used by the latest map_to_cells call, if any.
- void **setGridWidth** (unsigned int gridWidth)

Sets the discrete spatial grid width reserved for the first colony model.
- unsigned int **getGridWidth** () const

Returns the discrete spatial grid width.
- void **setGridHeight** (unsigned int gridHeight)

Sets the discrete spatial grid height reserved for the first colony model.
- unsigned int **getGridHeight** () const

Returns the discrete spatial grid height.
- [GroProgramRuntime::ExecutionResult](#) **executeGroProgram** ()

Executes the configured Gro program once against the current colony state.

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionType, std::string name="", bool insert↵
 IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.

- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelSpecific`, bool `traceLevelSpecificEnabled=true`)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation` ()
- static `ModelComponent * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
- static `ModelDataDefinition * NewInstance` (`Model *model`, `std::string name=""`)

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent *component`)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent *component`)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord *fields`, `ModelComponent *component`)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event *event`)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.31.1 Detailed Description

First GenESyS component for Gro-inspired bacteria colony simulation.

[BacteriaColony](#) owns bacteria and signal-field runtime state while using the enclosing [ModelSimulation](#) time as the single colony clock. The component references a reusable [GroProgram](#) and executes the first supported runtime commands through plugin-side parser/runtime helpers.

10.31.2 Member Function Documentation

10.31.2.1 `_check()`

```
bool BacteriaColony::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.31.2.2 `_createAttachedAttributes()`

```
void BacteriaColony::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.31.2.3 `_initBetweenReplications()`

```
void BacteriaColony::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.31.2.4 `_loadInstance()`

```
bool BacteriaColony::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.31.2.5 `_onDispatchEvent()`

```
void BacteriaColony::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.31.2.6 `_saveInstance()`

```
void BacteriaColony::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.31.2.7 `show()`

```
std::string BacteriaColony::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

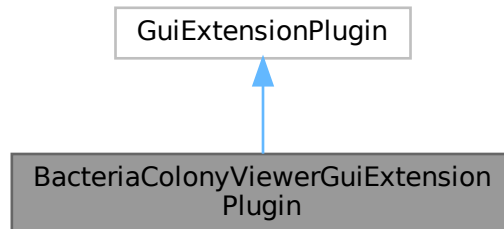
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

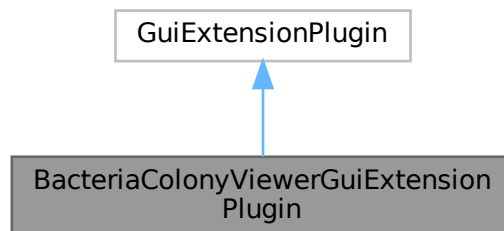
- plugins/components/BiochemicalSimulation/BacteriaColony.h
- plugins/components/BiochemicalSimulation/BacteriaColony.cpp

10.32 BacteriaColonyViewerGuiExtensionPlugin Class Reference

Inheritance diagram for BacteriaColonyViewerGuiExtensionPlugin:



Collaboration diagram for BacteriaColonyViewerGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId () const override`
- `std::string displayName () const override`
- `std::string version () const override`
- `std::vector< std::string > requiredModelPlugins () const override`
- `void registerContributions (GuiExtensionRegistry *registry) const override`

10.32.1 Member Function Documentation

10.32.1.1 displayName()

```
std::string BacteriaColonyViewerGuiExtensionPlugin::displayName ( ) const [inline], [override],
[virtual]
```

Implements [GuiExtensionPlugin](#).

10.32.1.2 extensionId()

```
std::string BacteriaColonyViewerGuiExtensionPlugin::extensionId ( ) const [inline], [override],  
[virtual]
```

Implements [GuiExtensionPlugin](#).

10.32.1.3 registerContributions()

```
void BacteriaColonyViewerGuiExtensionPlugin::registerContributions (   
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.32.1.4 requiredModelPlugins()

```
std::vector< std::string > BacteriaColonyViewerGuiExtensionPlugin::requiredModelPlugins ( )  
const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.32.1.5 version()

```
std::string BacteriaColonyViewerGuiExtensionPlugin::version ( ) const [inline], [override],  
[virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

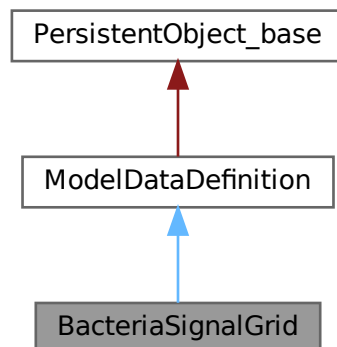
- applications/gui/genesys/extensions/BacteriaColonyViewerGuiExtensionPlugin.cpp

10.33 BacteriaSignalGrid Class Reference

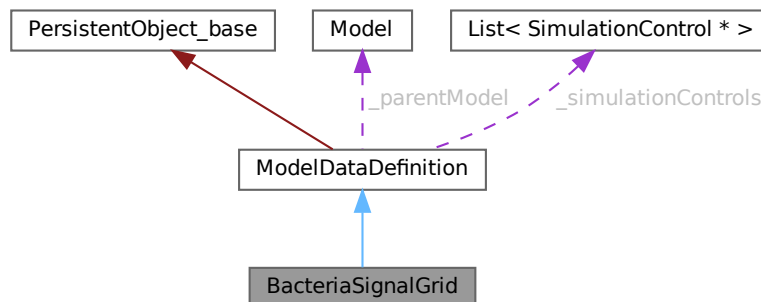
Reusable discrete signal-field definition for bacteria colonies.

```
#include <plugins/data/BiochemicalSimulation/BacteriaSignalGrid.h>
```

Inheritance diagram for BacteriaSignalGrid:



Collaboration diagram for BacteriaSignalGrid:



Public Member Functions

- **BacteriaSignalGrid** (`Model` *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **setWidth** (unsigned int width)
- unsigned int **getWidth** () const
- void **setHeight** (unsigned int height)
- unsigned int **getHeight** () const
- void **setInitialSignal** (double initialSignal)
- double **getInitialSignal** () const
- void **setDiffusionRate** (double diffusionRate)
- double **getDiffusionRate** () const
- void **setDecayRate** (double decayRate)
- double **getDecayRate** () const

- void **setInitialValues** (std::string initialValues)
- std::string **getInitialValues** () const
- unsigned int **getCellCount** () const
- bool **buildInitialField** (std::vector< double > &values, std::string &errorMessage) const

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert←IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)
Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.33.1 Detailed Description

Reusable discrete signal-field definition for bacteria colonies.

The data definition persists the spatial configuration and initial scalar values of one signal field. Colonies copy this configuration into their runtime state so model configuration and simulation mutation stay separated.

10.33.2 Member Function Documentation

10.33.2.1 `_check()`

```
bool BacteriaSignalGrid::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.33.2.2 `_loadInstance()`

```
bool BacteriaSignalGrid::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.33.2.3 `_saveInstance()`

```
void BacteriaSignalGrid::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.33.2.4 show()

```
std::string BacteriaSignalGrid::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/BiochemicalSimulation/BacteriaSignalGrid.h
- plugins/data/BiochemicalSimulation/BacteriaSignalGrid.cpp

10.34 BacteriaColony::BacteriumState Struct Reference**Public Attributes**

- unsigned int **id** = 0
- unsigned int **parentId** = 0
- unsigned int **generation** = 0
- std::string **programName** = ""
- std::vector< double > **programArguments**
- unsigned int **divisionCount** = 0
- double **birthTime** = 0.0
- double **lastUpdateTime** = 0.0
- double **lastDivisionTime** = 0.0
- double **positionX** = 0.0
- double **positionY** = 0.0
- double **directionRadians** = 0.0
- double **volume** = 1.0
- double **size** = 1.0
- double **gfp** = 0.0
- double **rfp** = 0.0
- double **yfp** = 0.0
- double **cfp** = 0.0
- double **speed** = 0.1

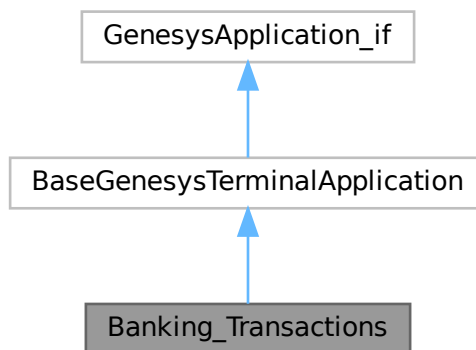
- unsigned int **tickCount** = 0
- unsigned int **gridX** = 0
- unsigned int **gridY** = 0
- bool **hasExplicitGridPosition** = false
- bool **justDivided** = false
- bool **daughter** = false
- bool **alive** = true
- std::map< std::string, double > **runtimeVariables**

The documentation for this struct was generated from the following file:

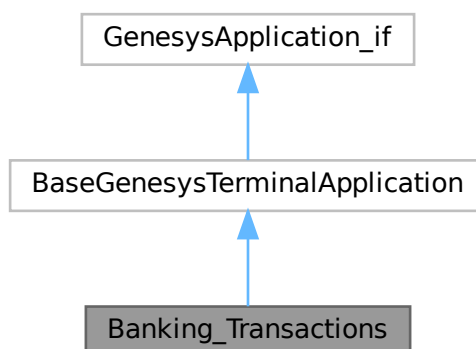
- plugins/components/BiochemicalSimulation/BacteriaColony.h

10.35 Banking_Transactions Class Reference

Inheritance diagram for Banking_Transactions:



Collaboration diagram for Banking_Transactions:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.35.1 Member Function Documentation

10.35.1.1 [main\(\)](#)

```
int Banking_Transactions::main (
    int argc,
    char ** argv )  [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/arenaExamples/Banking_Transactions.h
- applications/modelSpecific/arenaExamples/Banking_Transactions.cpp

10.36 BacteriaColony::BarrierSegment Struct Reference

Public Attributes

- double **x1** = 0.0
- double **y1** = 0.0
- double **x2** = 0.0
- double **y2** = 0.0

The documentation for this struct was generated from the following file:

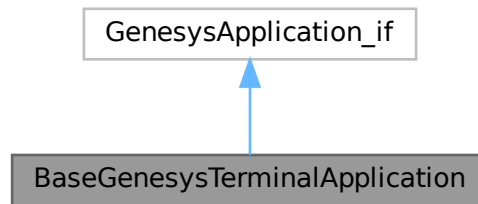
- `plugins/components/BiochemicalSimulation/BacteriaColony.h`

10.37 BaseGenesysTerminalApplication Class Reference

Inherits [GenesysApplication_if](#).

Inherited by [AirportSecurityExample](#), [AirportSecurityExampleExtended](#), [Airport_Extended1](#), [AnElectronicAssemblyAndTestSystem](#), [Assembly_Line](#), [Banking_Transactions](#), [Book_Cap02_Example01](#), [BufferFIFO](#), [ContinuousModel](#), [Example_BasicOrderShipping](#), [Example_PortModel](#), [Example_PublicTransport](#), [FullSimulationOfComplexModel](#), [GenesysShell](#), [Half_Adder](#), [Loja01](#), [OperatingSystem02](#), [OperatingSystem03](#), [Rectifier](#), [Smart_AddingResource](#), [Smart_AlternatingEntityCreation](#), [Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime](#), [Smart_ArrivalsEntityTypeVsAttribute](#), [Smart_AssignExample](#), [Smart_AssignWriteSeizes](#), [Smart_Attribute_Variable](#), [Smart_AutomaticStatisticsCollection](#), [Smart_BacteriaColony_GRO](#), [Smart_BasicModeling](#), [Smart_BatchAndSeparate](#), [Smart_BatchSeparate](#), [Smart_BioKineticLawRegulation](#), [Smart_BioNetworkAnalysis](#), [Smart_BioReversibleKineticLaw](#), [Smart_BioReversibleMassAction](#), [Smart_Buffer](#), [Smart_CellularAutomata](#), [Smart_CellularAutomata1](#), [Smart_Clone](#), [Smart_ContinuousFlowEntities](#), [Smart_CppForG](#), [Smart_Create](#), [Smart_DecideNWayByChance](#), [Smart_DefaultModalModel](#), [Smart_DefiningAttributesAsStrings](#), [Smart_DefiningControlLogic](#), [Smart_DefiningResourceCapacity](#), [Smart_Delay](#), [Smart_DelayBasedOnReplication](#), [Smart_Dummy](#), [Smart_EntitiesProcessedByPriority](#), [Smart_EvaluatingConditionsBeforeEnteringQueue](#), [Smart_Expression](#), [Smart_FSMState](#), [Smart_Failures](#), [Smart_GroColonyGrowth](#), [Smart_GroColonyLifecycle](#), [Smart_HoldSearchRemove](#), [Smart_InventoryAndHoldingCosts](#), [Smart_LNODE](#), [Smart_MarkovChain](#), [Smart_MaxArrivalsField](#), [Smart_ModalModel](#), [Smart_ModalModelDefaultNetwork](#), [Smart_ModalModelFSM](#), [Smart_ModalModelPetriNet](#), [Smart_ModelInfoModelSimulation](#), [Smart_ModelRunUntil1000Parts](#), [Smart_ModuleDisplayVariables](#), [Smart_ODE](#), [Smart_OnEvent](#), [Smart_OverlappingResources](#), [Smart_ParallelProcessingOfEntities](#), [Smart_Parser](#), [Smart_ParserModelFunctions](#), [Smart_PetriPlace](#), [Smart_PlacingEntitiesInQueue](#), [Smart_Plugin](#), [Smart_PriorityExample](#), [Smart_Process](#), [Smart_ProcessArena](#), [Smart_ProcessSet](#), [Smart_PythonForG](#), [Smart_R_Simulator](#), [Smart_Record](#), [Smart_Record_Arena](#), [Smart_RemovingAndReorderingEntitiesInAQueue](#), [Smart_ResourceCosting](#), [Smart_ResourceScheduleCosting](#), [Smart_ResourceSets](#), [Smart_RouteStation](#), [Smart_SeizeDelayRelease](#), [Smart_SeizeDelayReleaseMany](#), [Smart_SeizeDelayReleaseNoDataDefs](#), [Smart_SeizingMultipleSimulta](#), [Smart_SelectingRouteBasedOnProbability](#), [Smart_SelectingShorterQueue](#), [Smart_Sequence](#), [Smart_SimulationControlResponse](#), [Smart_SynchronizingParallelEntities](#), [Smart_ValueAdded](#), [Smart_WaitForSignal](#), [Smart_WaitScanCondition](#), [Smart_WaitSignal](#), [StartAIAssistant](#), [WcmBioNetworkBridge](#), [WcmCompartmentTransportWholeCellMvp](#), [WcmFluxBalanceWholeCellMvp](#), [WcmGeneExpression](#), [WcmLifecycleStress](#), [WcmMgenitaliumKarr2012](#), [WcmPathwayStressWholeCellMvp](#), [WcmSimpleSsa](#), [WcmWholeCellMvp](#), [WcmYeastDidacticWholeCellMvp](#), and [WcmYeastEukaryoticWholeCellMvp](#).

Collaboration diagram for BaseGenesysTerminalApplication:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override
- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Protected Member Functions

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.37.1 Member Function Documentation

10.37.1.1 main()

```

int BaseGenesysTerminalApplication::main (
    int argc,
    char ** argv ) [override], [virtual]
  
```

Implements [GenesysApplication_if](#).

Reimplemented in [Airport_Extended1](#), [Assembly_Line](#), [Banking_Transactions](#), [Example_BasicOrderShipping](#), [Example_PortModel](#), [Example_PublicTransport](#), [Smart_AddingResource](#), [Smart_AlternatingEntityCreation](#), [Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime](#), [Smart_AutomaticStatisticsCollection](#), [Smart_ContinuousFlowEntities](#), [Smart_Create](#), [Smart_DecideNWayByChance](#), [Smart_DefiningAttributesAsStrings](#), [Smart_DefiningControlLogic](#), [Smart_DefiningResourceCapacity](#), [Smart_EntitiesProcessedByPriority](#), [Smart_EvaluatingConditionsBeforeEnteringQueue](#), [Smart_Expression](#), [Smart_MaxArrivalsField](#), [Smart_ModelRunUntil1000Parts](#), [Smart_ModuleDisplayVariables](#), [Smart_OverlappingResources](#), [Smart_ParallelProcessingOfEntities](#), [Smart_PlacingEntitiesInQueueSets](#), [Smart_PriorityExample](#), [Smart_ProcessArena](#), [Smart_Record_Arena](#), [Smart_RemovingAndReorderingEntitiesInAQueue](#), [Smart_ResourceSets](#), [Smart_SeizingMultipleSimultaneously](#), [Smart_SelectingRouteBasedOnProbability](#), [Smart_SelectingShorterQueue](#), [Smart_SynchronizingParallelEntities](#), [Smart_WaitForSignal](#), [Smart_MarkovChain](#), [Smart_Buffer](#), [Smart_Clone](#), [Smart_Delay](#), [Smart_Failures](#), [Smart_HoldSearchRemove](#), [Smart_Process](#), [Smart_ProcessSet](#), [Smart_SeizeDelayRelease](#), [Smart_SeizeDelayReleaseMany](#), [Smart_SeizeDelayReleaseNoDataDefs](#), [Smart_R_Simulator](#), [Smart_Record](#), [Smart_Attribute_Variable](#), [Smart_ModelInfoModelSimulation](#), [Smart_Parser](#), [Smart_ParserModelFunctions](#), [Smart_RouteStation](#), [Smart_CellularAutomata](#), [Smart_CellularAutomata1](#), [Smart_DefaultModalModel](#), [Smart_Dummy](#), [Smart_ModalModel](#), [Smart_OnEvent](#), [Smart_WaitScanCondition](#), [Smart_WaitSignal](#), [FullSimulationOfComplexModel](#), [Half_Adder](#), and [Rectifier](#).

10.37.1.2 setDefaultEventHandlers()

```
void BaseGenesysTerminalApplication::setDefaultEventHandlers (
    OnEventManager * oem )
```

Add default method to handle simulator events so information is plotted to output screen

The documentation for this class was generated from the following files:

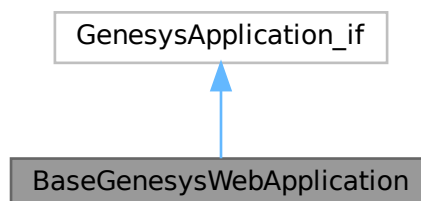
- applications/BaseGenesysTerminalApplication.h
- applications/BaseGenesysTerminalApplication.cpp

10.38 BaseGenesysWebApplication Class Reference

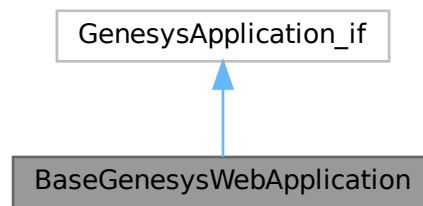
Entry point for the standalone Worker application flavor of Genesys.

```
#include <applications/worker/BaseGenesysWebApplication.h>
```

Inheritance diagram for BaseGenesysWebApplication:



Collaboration diagram for BaseGenesysWebApplication:



Public Member Functions

- `int main (int argc, char **argv)` override
Parses command-line arguments, starts the HTTP server and runs the worker app.

10.38.1 Detailed Description

Entry point for the standalone Worker application flavor of Genesys.

The worker variant bootstraps a session manager, HTTP server and router, then keeps processing requests until the configured request limit is reached or a transport error occurs.

10.38.2 Member Function Documentation

10.38.2.1 main()

```

int BaseGenesysWebApplication::main (
    int argc,
    char ** argv ) [override], [virtual]
  
```

Parses command-line arguments, starts the HTTP server and runs the worker app.

Boots the web stack and keeps the request loop alive until shutdown.

Parameters

<i>argc</i>	Number of command-line arguments.
<i>argv</i>	Command-line argument vector.

Returns

Zero on normal termination; non-zero on startup or server failure.

This entry point wires token management, session creation, worker-job storage, API routing and the embedded HTTP server into one runtime pipeline.

Implements [GenesysApplication_if](#).

The documentation for this class was generated from the following files:

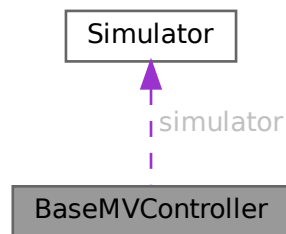
- applications/worker/BaseGenesysWebApplication.h
- applications/worker/BaseGenesysWebApplication.cpp

10.39 BaseMVController Class Reference

Compatibility base class that binds the Qt UI and the simulator model.

```
#include <applications/gui/genesys/BaseMVController.h>
```

Collaboration diagram for BaseMVController:



Protected Attributes

- `Ui::MainWindow * ui`
MainWindow UI built from the Qt Designer form.
- `Simulator * simulator`
Active simulator instance owned elsewhere by the application.

10.39.1 Detailed Description

Compatibility base class that binds the Qt UI and the simulator model.

Legacy GUI controllers in Genesys share the same two core pointers: the main window UI object and the active simulator instance. This base class keeps that pairing in one place so higher-level controllers can reuse it.

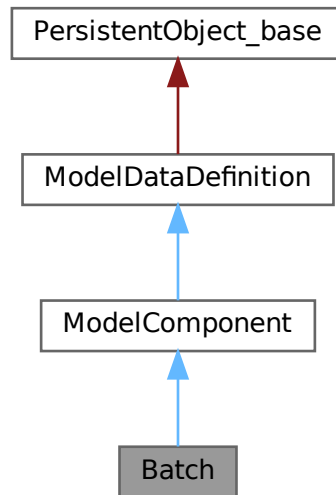
The documentation for this class was generated from the following files:

- applications/gui/genesys/BaseMVController.h
- applications/gui/genesys/BaseMVController.cpp

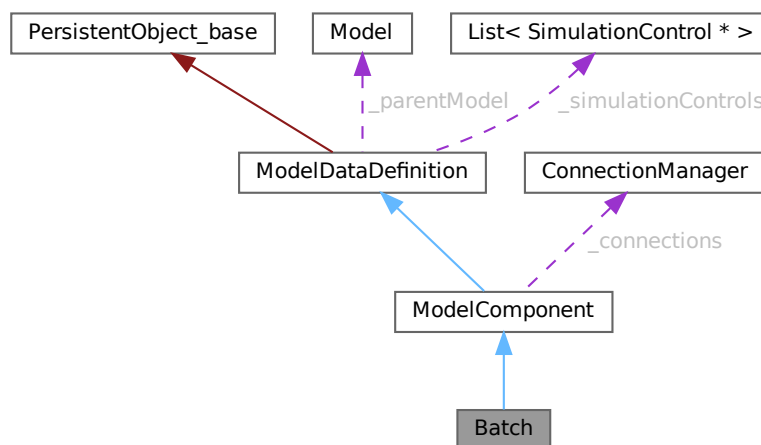
10.40 Batch Class Reference

```
#include <plugins/components/Grouping/Batch.h>
```

Inheritance diagram for Batch:



Collaboration diagram for Batch:



Public Types

- enum class **BatchType** : int { **Temporary** = 0 , **Permanent** = 1 , num_elements = 2 }
- enum class **Rule** : int { **Any** = 0 , **ByAttribute** = 1 , num_elements = 2 }
- enum class **GroupedAttribs** : int { **FirstEntity** = 0 , **LastEntity** = 1 , **SumAttributes** = 2 , num_elements = 3 }

Public Member Functions

- **Batch** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- virtual public methods*
- void **setGroupedEntityType** ([EntityType](#) *groupedEntityType)
- void **setGroupedEntityTypeName** (std::string groupedEntityTypeName)
- [EntityType](#) * **getGroupedEntityType** () const
- void **setBatchType** (Batch::BatchType batchType)
- Batch::BatchType **getBatchType** () const
- void **setAttributeName** (std::string attributeName)
- std::string **getAttributeName** () const
- void **setBatchSize** (std::string batchSize)
- std::string **getBatchSize** () const
- void **setRule** (Batch::Rule _rule)
- Batch::Rule **getRule** () const
- void **setGroupedAttributes** (Batch::GroupedAttribs _groupedAttributes)
- Batch::GroupedAttribs **getGroupedAttributes** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- constructors*
- virtual ~**ModelComponent** ()
- Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- Connects this component to another component.*
- bool **hasBreakpointAt** ()
- Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string _description)
- Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const
- Returns the free-form description associated with this component.*
- void **remove** ([ModelComponent](#) *comp)
- Removes a connected component from this component.*

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- Releases resources owned by this data definition.*
- Util::identification **getId** () const
- Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & **getName** () const
- Returns the logical name of this instance.*

- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static std::string [convertEnumToStr](#) ([BatchType](#) type)
- static std::string [convertEnumToStr](#) ([Rule](#) rule)
- static std::string [convertEnumToStr](#) ([GroupedAttribs](#) attribs)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_createNonEditableDataDefinitions](#) () override
- virtual void [_createEditableDataDefinitions](#) () override
- virtual void [_createAttachedAttributes](#) () override
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [ModelDataDefinition](#)

- [bool](#) [_getSaveDefaultsOption](#) ()
- [virtual](#) [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- [virtual](#) [void](#) [_initBetweenReplications](#) ()
- [virtual](#) [void](#) [_addSimulationControl](#) ([SimulationControl](#) *control)
- [virtual](#) [void](#) [_createInternalAndAttachedData](#) ()
- [virtual](#) [void](#) [_createInternalStatisticReporters](#) ()
- [void](#) [_templateCreateInternalStatisticReporters](#) ()
- [void](#) [_templateCreateAttachedAttributes](#) ()
- [void](#) [_templateCreateNonEditableDataDefinitions](#) ()
- [void](#) [_templateCreateEditableDataDefinitions](#) ()
- [void](#) [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- [void](#) [_internalDataClear](#) ()
- [void](#) [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- [void](#) [_internalDataRemove](#) (const std::string &key)
- [void](#) [_attachedDataClear](#) ()
- [void](#) [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_attachedDataRemove](#) (const std::string &key)
- [void](#) [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- [void](#) [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_statisticReporterRemove](#) (const std::string &key)
- [void](#) [_statisticReportersClear](#) ()
- [void](#) [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- [void](#) [_mandatoryAttachedAttributesClear](#) ()
- [void](#) [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- [void](#) [_mandatoryEditableDataDefinitionsClear](#) ()
- [void](#) [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- [void](#) [_optionalEditableDataDefinitionsClear](#) ()
- [void](#) [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- [void](#) [_mandatoryNonEditableDataDefinitionsClear](#) ()
- [void](#) [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- [void](#) [_optionalNonEditableDataDefinitionsClear](#) ()
- [void](#) [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- [void](#) [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- [void](#) [traceError](#) (const std::string &text, const std::exception &e)
- [void](#) [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- [void](#) [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- [void](#) [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- [void](#) [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- [void](#) [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.40.1 Detailed Description

[Batch](#) module DESCRIPTION This module is intended as the grouping mechanism within the simulation model. Batches can be permanently or temporarily grouped. Temporary batches must later be split using the [Separate](#) module. Batches may be made with any specified number of entering entities or may be matched together based on an attribute. Entities arriving at the [Batch](#) module are placed in a queue until the required number of entities has accumulated. Once accumulated, a new representative entity is created. TYPICAL USES Collect a number of parts before starting processing Reassemble previously separated copies of a form Bring together a patient and his record before commencing an appointment PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Type Method of batching entities together. [Batch](#) Size Number of entities to be batched. Save Criterion Method for assigning representative entity's user-defined attribute values. Rule Determines how incoming entities will be batched. Any [Entity](#) will take the first "Batch Size" number of entities and put them together. By [Attribute](#) signifies that the values of the specified attribute must match for entities to be grouped. For example, if [Attribute](#) Name is Color, all entities must have the same Color value to be grouped; otherwise, they will wait at the module for additional incoming entities. [Attribute](#) Name Name of the attribute whose value must match the value of the other incoming entities in order for a group to be made. Applies only when Rule is By [Attribute](#). Representative [Entity](#) The entity type for the representative entity.

10.40.2 Member Function Documentation

10.40.2.1 `_check()`

```
bool Batch::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.40.2.2 _createAttachedAttributes()

```
void Batch::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.40.2.3 _createEditableDataDefinitions()

```
void Batch::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.40.2.4 _createNonEditableDataDefinitions()

```
void Batch::_createNonEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.40.2.5 _loadInstance()

```
bool Batch::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.40.2.6 _onDispatchEvent()

```
void Batch::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.40.2.7 _saveInstance()

```
void Batch::_saveInstance (
```



```
PersistenceRecord * fields,
bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.40.2.8 show()

```
std::string Batch::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

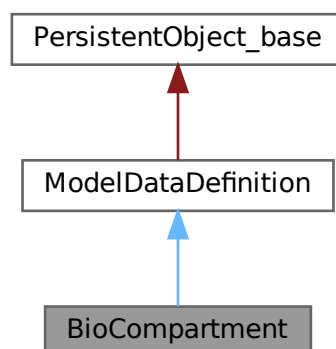
- plugins/components/Grouping/Batch.h
- plugins/components/Grouping/Batch.cpp

10.41 BioCompartment Class Reference

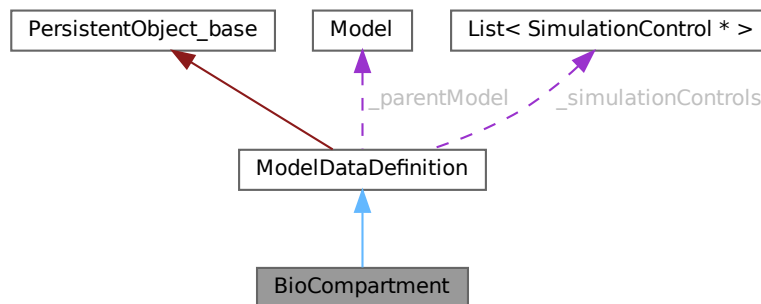
Defines a biological compartment for biochemical and whole-cell models.

```
#include <plugins/data/WholeCellModeling/BioCompartment.h>
```

Inheritance diagram for BioCompartment:



Collaboration diagram for BioCompartment:



Public Member Functions

- **BioCompartment** ([Model](#) *model, std::string name="")
- void **setCompartmentType** (std::string type)
- std::string **getCompartmentType** () const
- void **setParentCompartmentName** (std::string parentName)
- std::string **getParentCompartmentName** () const
- void **setMembraneBounded** (bool membraneBounded)
- bool **isMembraneBounded** () const
- void **setVolumeFraction** (double volumeFraction)
- double **getVolumeFraction** () const
- void **setCopyNumber** (unsigned int copyNumber)
- unsigned int **getCopyNumber** () const
- void **setRole** (std::string role)
- std::string **getRole** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insertIntoModel=true)
- *Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- *Releases resources owned by this data definition.*
- Util::identification **getId** () const
- *Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- *Updates the logical name of this instance.*
- const std::string & **getName** () const
- *Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
- *Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const

- Returns the user-visible label of this instance.*
- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.41.1 Detailed Description

Defines a biological compartment for biochemical and whole-cell models.

[BioCompartment](#) represents a named physical or logical cellular region such as cytosol, nucleus, mitochondrion, bud, membrane, or extracellular medium. It is a [ModelDataDefinition](#) and therefore stores structural metadata used by other plugins, but it does not execute simulation behavior itself.

The class is intentionally generic so it can support both prokaryotic and eukaryotic cells. Parent-child relationships allow hierarchical structures such as:

- extracellular -> cell surface
- cytosol -> nucleus
- cytosol -> mitochondrion
- cytosol -> bud

10.41.2 Member Function Documentation

10.41.2.1 `_check()`

```
bool BioCompartment::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.41.2.2 `_createEditableDataDefinitions()`

```
void BioCompartment::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.41.2.3 `_loadInstance()`

```
bool BioCompartment::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.41.2.4 `_saveInstance()`

```
void BioCompartment::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.41.2.5 show()

```
std::string BioCompartment::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

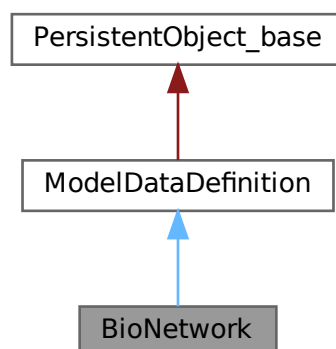
The documentation for this class was generated from the following files:

- plugins/data/WholeCellModeling/BioCompartment.h
- plugins/data/WholeCellModeling/BioCompartment.cpp

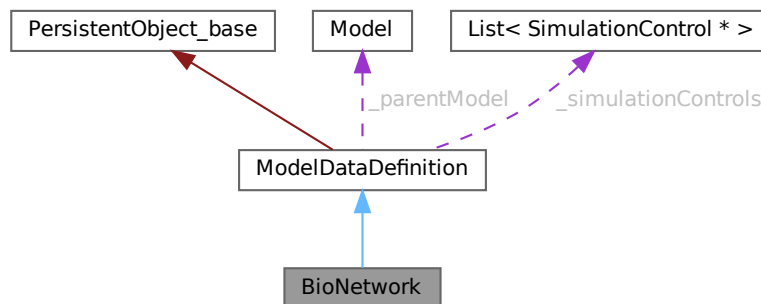
10.42 BioNetwork Class Reference

```
#include <plugins/data/BiochemicalSimulation/BioNetwork.h>
```

Inheritance diagram for BioNetwork:



Collaboration diagram for BioNetwork:



Public Member Functions

- **BioNetwork** ([Model](#) *model, std::string name="")
- void **setStartTime** (double startTime)
- double **getStartTime** () const
- void **setStopTime** (double stopTime)
- double **getStopTime** () const
- void **setStepSize** (double stepSize)
- double **getStepSize** () const
- void **setCurrentTime** (double currentTime)
- double **getCurrentTime** () const
- void **setAutoSchedule** (bool autoSchedule)
- bool **getAutoSchedule** () const
- void **setLastStatus** (std::string lastStatus)
- std::string **getLastStatus** () const
- void **setLastErrorMessage** (std::string lastErrorMessage)
- std::string **getLastErrorMessage** () const
- void **setLastResponsePayload** (std::string lastResponsePayload)
- std::string **getLastResponsePayload** () const
- void **addSpecies** (std::string speciesName)
- void **addReaction** (std::string reactionName)
- void **clearSpecies** ()
- void **clearReactions** ()
- const std::vector< std::string > & **getSpeciesNames** () const
- const std::vector< std::string > & **getReactionNames** () const
- const BioSimulationResult & **getLastSimulationResult** () const
- bool **getSpeciesTimeCourseDataset** (const std::string &speciesName, SimulationResultsDataset *dataset, std::string *errorMessage=nullptr) const
- bool **buildOdeSystemForAnalysis** (MassActionOdeSystem *system, std::string &errorMessage) const
- bool **getStoichiometryMatrix** (BioStoichiometryMatrix *matrix, std::string *errorMessage=nullptr) const
- bool **getReactionRateTimeCourse** (BioReactionRateTimeCourse *timeCourse, std::string *errorMessage=nullptr) const
- bool **checkLastSampleSteadyState** (double tolerance, BioSteadyStateCheck *check, std::string *errorMessage=nullptr) const
- bool **scanLocalParameterSensitivity** (double relativeStep, double absoluteStep, BioSensitivityScan *scan, std::string *errorMessage=nullptr) const

- bool **simulate** (std::string &errorMessage)
- bool **simulate** (double startTime, double stopTime, double stepSize, std::string &errorMessage)
- bool **advanceOneStep** (std::string &errorMessage)
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)

Defines the specific trace level and optionally enables it immediately.

- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createEditableDataDefinitions](#) () override
Refreshes editable associations to referenced biochemical definitions.

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool` [_getSaveDefaultsOption](#) ()
- `virtual` [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- `virtual void` [_addSimulationControl](#) ([SimulationControl](#) *control)
- `virtual void` [_createInternalAndAttachedData](#) ()
- `virtual void` [_createInternalStatisticReporters](#) ()
- `virtual void` [_createAttachedAttributes](#) ()
- `virtual void` [_createNonEditableDataDefinitions](#) ()
- `void` [_templateCreateInternalStatisticReporters](#) ()
- `void` [_templateCreateAttachedAttributes](#) ()
- `void` [_templateCreateNonEditableDataDefinitions](#) ()
- `void` [_templateCreateEditableDataDefinitions](#) ()
- `void` [_internalStatisticReportersClear](#) ()
- *legacy helper methods used by the granular create* hooks*
- `void` [_internalDataClear](#) ()
- `void` [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- `void` [_internalDataRemove](#) (const std::string &key)
- `void` [_attachedDataClear](#) ()
- `void` [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_attachedDataRemove](#) (const std::string &key)
- `void` [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- `void` [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_statisticReporterRemove](#) (const std::string &key)
- `void` [_statisticReportersClear](#) ()
- `void` [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- `void` [_mandatoryAttachedAttributesClear](#) ()
- `void` [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- `void` [_mandatoryEditableDataDefinitionsClear](#) ()
- `void` [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- `void` [_optionalEditableDataDefinitionsClear](#) ()
- `void` [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- `void` [_mandatoryNonEditableDataDefinitionsClear](#) ()
- `void` [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- `void` [_optionalNonEditableDataDefinitionsClear](#) ()
- `void` [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- *method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void` [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- *not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void` [traceError](#) (const std::string &text, const std::exception &e)
- `void` [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- `void` [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- `void` [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- `void` [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- *Traces text to the simulation output.*
- `void` [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.42.1 Detailed Description

Native deterministic biochemical network runner.

[BioNetwork](#) advances [BioSpecies](#) with fixed-step RK4 over a mass-action ODE system. Networks may define explicit [BioSpecies](#) and [BioReaction](#) membership by name; when a membership list is empty, the runner falls back to discovering every registered definition of that type in the owning model for compatibility with existing models.

10.42.2 Member Function Documentation

10.42.2.1 [_check\(\)](#)

```
bool BioNetwork::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.42.2.2 [_createEditableDataDefinitions\(\)](#)

```
void BioNetwork::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Refreshes editable associations to referenced biochemical definitions.

[BioNetwork](#) stores model membership by name for persistence. During model checking these associations keep referenced species, reactions, and parameters from being pruned as orphan [ModelDataDefinitions](#).

Reimplemented from [ModelDataDefinition](#).

10.42.2.3 [_initBetweenReplications\(\)](#)

```
void BioNetwork::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.42.2.4 `_loadInstance()`

```
bool BioNetwork::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.42.2.5 `_saveInstance()`

```
void BioNetwork::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.42.2.6 `show()`

```
std::string BioNetwork::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

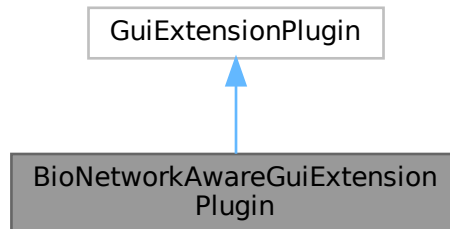
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

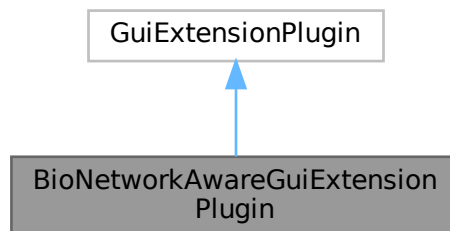
- plugins/data/BiochemicalSimulation/BioNetwork.h
- plugins/data/BiochemicalSimulation/BioNetwork.cpp

10.43 BioNetworkAwareGuiExtensionPlugin Class Reference

Inheritance diagram for BioNetworkAwareGuiExtensionPlugin:



Collaboration diagram for BioNetworkAwareGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId () const override`
- `std::string displayName () const override`
- `std::string version () const override`
- `std::vector< std::string > requiredModelPlugins () const override`
- `void registerContributions (GuiExtensionRegistry *registry) const override`

10.43.1 Member Function Documentation

10.43.1.1 displayName()

```
std::string BioNetworkAwareGuiExtensionPlugin::displayName ( ) const [inline], [override],
[virtual]
```

Implements [GuiExtensionPlugin](#).

10.43.1.2 extensionId()

```
std::string BioNetworkAwareGuiExtensionPlugin::extensionId ( ) const [inline], [override],  
[virtual]
```

Implements [GuiExtensionPlugin](#).

10.43.1.3 registerContributions()

```
void BioNetworkAwareGuiExtensionPlugin::registerContributions (   
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.43.1.4 requiredModelPlugins()

```
std::vector< std::string > BioNetworkAwareGuiExtensionPlugin::requiredModelPlugins ( ) const  
[inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.43.1.5 version()

```
std::string BioNetworkAwareGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

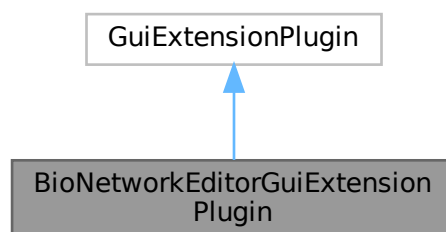
Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

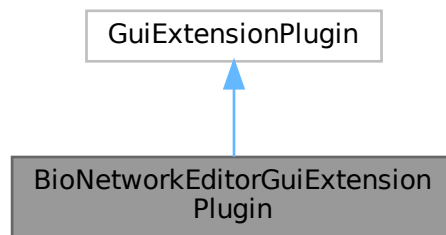
- applications/gui/genesys/extensions/BioNetworkAwareGuiExtensionPlugin.cpp

10.44 BioNetworkEditorGuiExtensionPlugin Class Reference

Inheritance diagram for BioNetworkEditorGuiExtensionPlugin:



Collaboration diagram for BioNetworkEditorGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId () const` override
- `std::string displayName () const` override
- `std::string version () const` override
- `std::vector< std::string > requiredModelPlugins () const` override
- `void registerContributions (GuiExtensionRegistry *registry) const` override

10.44.1 Member Function Documentation

10.44.1.1 displayName()

```
std::string BioNetworkEditorGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.44.1.2 extensionId()

```
std::string BioNetworkEditorGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.44.1.3 registerContributions()

```
void BioNetworkEditorGuiExtensionPlugin::registerContributions (   
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.44.1.4 requiredModelPlugins()

```
std::vector< std::string > BioNetworkEditorGuiExtensionPlugin::requiredModelPlugins ( ) const  
[inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.44.1.5 version()

```
std::string BioNetworkEditorGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

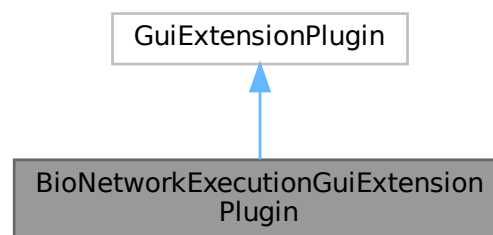
Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

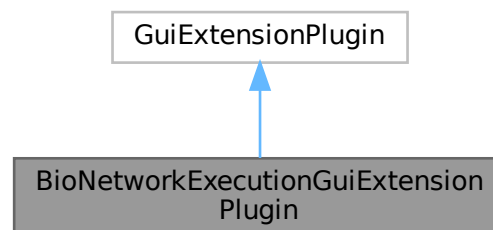
- applications/gui/genesys/extensions/BioNetworkEditorGuiExtensionPlugin.cpp

10.45 BioNetworkExecutionGuiExtensionPlugin Class Reference

Inheritance diagram for BioNetworkExecutionGuiExtensionPlugin:



Collaboration diagram for BioNetworkExecutionGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

10.45.1 Member Function Documentation

10.45.1.1 displayName()

```
std::string BioNetworkExecutionGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.45.1.2 extensionId()

```
std::string BioNetworkExecutionGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.45.1.3 registerContributions()

```
void BioNetworkExecutionGuiExtensionPlugin::registerContributions (   
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.45.1.4 requiredModelPlugins()

```
std::vector< std::string > BioNetworkExecutionGuiExtensionPlugin::requiredModelPlugins ( )   
const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.45.1.5 version()

```
std::string BioNetworkExecutionGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

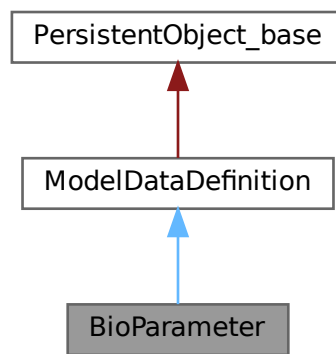
The documentation for this class was generated from the following file:

- `applications/gui/genesys/extensions/BioNetworkExecutionGuiExtensionPlugin.cpp`

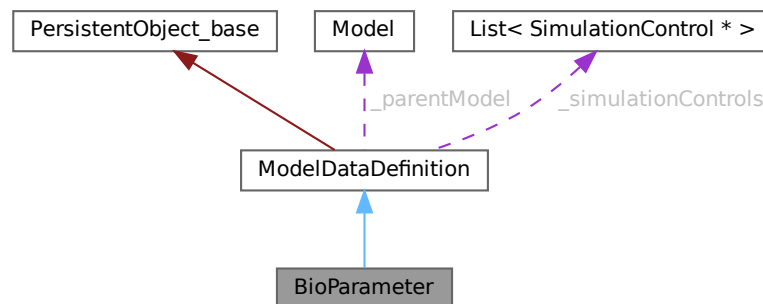
10.46 BioParameter Class Reference

```
#include <plugins/data/BiochemicalSimulation/BioParameter.h>
```

Inheritance diagram for BioParameter:



Collaboration diagram for BioParameter:



Public Member Functions

- **BioParameter** ([Model](#) *model, std::string name="")
- void **setValue** (double value)
- double **getValue** () const
- void **setUnit** (std::string unit)
- std::string **getUnit** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_initBetweenReplications** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createNonEditableDataDefinitions** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()

- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.46.1 Detailed Description

Scalar biochemical parameter, typically used as a kinetic constant.

[BioReaction](#) may reference a [BioParameter](#) by name. Parameter lookup remains a model-data responsibility, so missing references are reported by [BioReaction](#) validation rather than by this value object.

10.46.2 Member Function Documentation

10.46.2.1 `_check()`

```
bool BioParameter::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.46.2.2 `_loadInstance()`

```
bool BioParameter::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.46.2.3 `_saveInstance()`

```
void BioParameter::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.46.2.4 `show()`

```
std::string BioParameter::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal [ModelDataDefinitions](#) such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

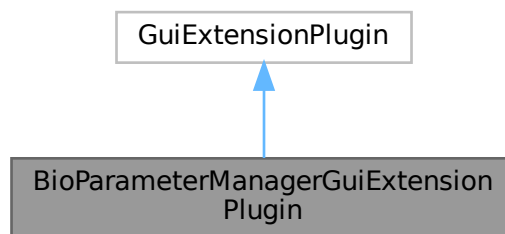
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

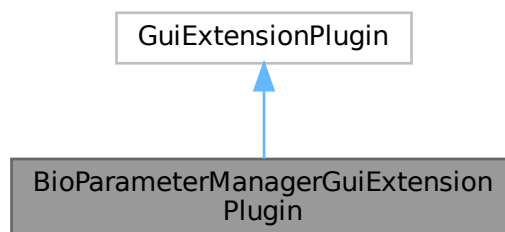
- `plugins/data/BiochemicalSimulation/BioParameter.h`
- `plugins/data/BiochemicalSimulation/BioParameter.cpp`

10.47 BioParameterManagerGuiExtensionPlugin Class Reference

Inheritance diagram for BioParameterManagerGuiExtensionPlugin:



Collaboration diagram for BioParameterManagerGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

10.47.1 Member Function Documentation

10.47.1.1 displayName()

```
std::string BioParameterManagerGuiExtensionPlugin::displayName ( ) const [inline], [override],  
[virtual]
```

Implements [GuiExtensionPlugin](#).

10.47.1.2 extensionId()

```
std::string BioParameterManagerGuiExtensionPlugin::extensionId ( ) const [inline], [override],  
[virtual]
```

Implements [GuiExtensionPlugin](#).

10.47.1.3 registerContributions()

```
void BioParameterManagerGuiExtensionPlugin::registerContributions (   
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.47.1.4 requiredModelPlugins()

```
std::vector< std::string > BioParameterManagerGuiExtensionPlugin::requiredModelPlugins ( )  
const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.47.1.5 version()

```
std::string BioParameterManagerGuiExtensionPlugin::version ( ) const [inline], [override],  
[virtual]
```

Implements [GuiExtensionPlugin](#).

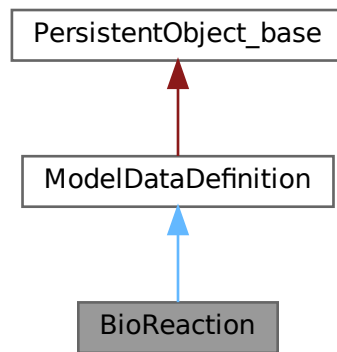
The documentation for this class was generated from the following file:

- applications/gui/genesys/extensions/BioParameterManagerGuiExtensionPlugin.cpp

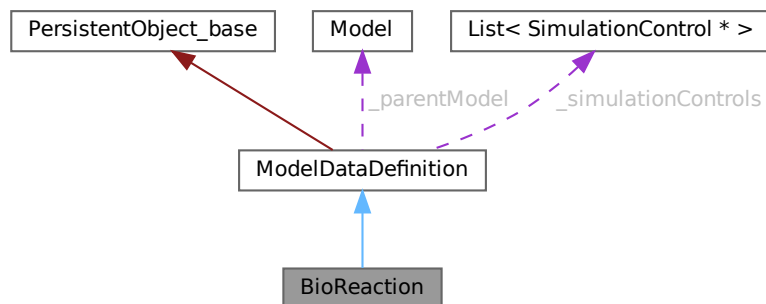
10.48 BioReaction Class Reference

```
#include <plugins/data/BiochemicalSimulation/BioReaction.h>
```

Inheritance diagram for BioReaction:



Collaboration diagram for BioReaction:



Classes

- struct [StoichiometricTerm](#)

Public Member Functions

- [BioReaction](#) ([Model](#) *model, std::string name="")
Creates an empty biochemical reaction definition.
- virtual ~**BioReaction** () override=default

- Releases reaction-owned resources.*

 - void [addReactant](#) (std::string speciesName, double stoichiometry=1.0)

Adds a reactant species to the reaction.
- void [addProduct](#) (std::string speciesName, double stoichiometry=1.0)

Adds a product species to the reaction.
- void [addModifier](#) (std::string speciesName)

Adds a modifier species that influences the reaction rate.
- void **clearReactants** ()

Removes all reactants from the reaction.
- void **clearProducts** ()

Removes all products from the reaction.
- void **clearModifiers** ()

Removes all modifiers from the reaction.
- const std::vector< [StoichiometricTerm](#) > & [getReactants](#) () const

Returns the registered reactants.
- const std::vector< [StoichiometricTerm](#) > & [getProducts](#) () const

Returns the registered products.
- const std::vector< std::string > & [getModifiers](#) () const

Returns the registered modifiers.
- void [setRateConstant](#) (double rateConstant)

Sets the forward rate constant.
- double [getRateConstant](#) () const

Returns the forward rate constant.
- void [setRateConstantParameterName](#) (std::string rateConstantParameterName)

Stores the name of the parameter used for the forward rate constant.
- std::string [getRateConstantParameterName](#) () const

Returns the forward-rate parameter name.
- double [resolveRateConstant](#) () const

Resolves the effective forward rate constant.
- void [setReverseRateConstant](#) (double reverseRateConstant)

Sets the reverse rate constant.
- double [getReverseRateConstant](#) () const

Returns the reverse rate constant.
- void [setReverseRateConstantParameterName](#) (std::string reverseRateConstantParameterName)

Stores the name of the parameter used for the reverse rate constant.
- std::string [getReverseRateConstantParameterName](#) () const

Returns the reverse-rate parameter name.
- double [resolveReverseRateConstant](#) () const

Resolves the effective reverse rate constant.
- void [setKineticLawExpression](#) (std::string kineticLawExpression)

Sets the forward kinetic-law expression.
- std::string [getKineticLawExpression](#) () const

Returns the forward kinetic-law expression.
- void [setReverseKineticLawExpression](#) (std::string reverseKineticLawExpression)

Sets the reverse kinetic-law expression.
- std::string [getReverseKineticLawExpression](#) () const

Returns the reverse kinetic-law expression.
- void [setReversible](#) (bool reversible)

Marks the reaction as reversible or irreversible.
- bool [isReversible](#) () const

Indicates whether the reaction is reversible.
- virtual std::string [show](#) () override

Returns a textual representation of the reaction.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
Loads a biochemical reaction from serialized fields.
- static [PluginInformation](#) * [GetPluginInformation](#) ()
Returns plugin metadata for this reaction type.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new empty reaction instance.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()

- void **_internaStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.48.1 Detailed Description

Irreversible biochemical reaction definition.

Reactants, products, and modifiers are stored by species name so the reaction can be persisted independently from runtime pointers. Modifiers participate in kinetic laws without being consumed or produced. Reactions may omit reactants for synthesis or omit products for degradation, but must define at least one reactant or product. The current execution contract supports direct rate constants, a [BioParameter](#) reference, or an optional kinetic-law expression that resolves BioSpecies/BioParameter names at [BioNetwork](#) execution time. Reversible reactions use either a separate reverse kinetic-law expression or a reverse mass-action rate constant over the product side of the reaction.

10.48.2 Constructor & Destructor Documentation

10.48.2.1 BioReaction()

```
BioReaction::BioReaction (
    Model * model,
    std::string name = "" )
```

Creates an empty biochemical reaction definition.

Parameters

<i>model</i>	Parent model.
<i>name</i>	Initial reaction name.

10.48.3 Member Function Documentation

10.48.3.1 _check()

```
bool BioReaction::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.48.3.2 _createAttachedAttributes()

```
void BioReaction::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.48.3.3 _loadInstance()

```
bool BioReaction::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.48.3.4 `_saveInstance()`

```
void BioReaction::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.48.3.5 `addModifier()`

```
void BioReaction::addModifier (
    std::string speciesName )
```

Adds a modifier species that influences the reaction rate.

Parameters

<i>speciesName</i>	Modifier species name.
--------------------	------------------------

10.48.3.6 `addProduct()`

```
void BioReaction::addProduct (
    std::string speciesName,
    double stoichiometry = 1.0 )
```

Adds a product species to the reaction.

Parameters

<i>speciesName</i>	Product species name.
<i>stoichiometry</i>	Product stoichiometry.

10.48.3.7 `addReactant()`

```
void BioReaction::addReactant (
    std::string speciesName,
    double stoichiometry = 1.0 )
```

Adds a reactant species to the reaction.

Parameters

<i>speciesName</i>	Reactant species name.
<i>stoichiometry</i>	Reactant stoichiometry.

10.48.3.8 getKineticLawExpression()

```
std::string BioReaction::getKineticLawExpression ( ) const
```

Returns the forward kinetic-law expression.

Returns

Forward kinetic-law expression.

10.48.3.9 getModifiers()

```
const std::vector< std::string > & BioReaction::getModifiers ( ) const
```

Returns the registered modifiers.

Returns

Constant reference to the modifier names.

10.48.3.10 GetPluginInformation()

```
PluginInformation * BioReaction::GetPluginInformation ( ) [static]
```

Returns plugin metadata for this reaction type.

Returns

[Plugin](#) information block.

10.48.3.11 getProducts()

```
const std::vector< BioReaction::StoichiometricTerm > & BioReaction::getProducts ( ) const
```

Returns the registered products.

Returns

Constant reference to the product terms.

10.48.3.12 getRateConstant()

```
double BioReaction::getRateConstant ( ) const
```

Returns the forward rate constant.

Returns

Forward rate constant.

10.48.3.13 getRateConstantParameterName()

```
std::string BioReaction::getRateConstantParameterName ( ) const
```

Returns the forward-rate parameter name.

Returns

Parameter name used to resolve the forward rate constant.

10.48.3.14 getReactants()

```
const std::vector< BioReaction::StoichiometricTerm > & BioReaction::getReactants ( ) const
```

Returns the registered reactants.

Returns

Constant reference to the reactant terms.

10.48.3.15 getReverseKineticLawExpression()

```
std::string BioReaction::getReverseKineticLawExpression ( ) const
```

Returns the reverse kinetic-law expression.

Returns

Reverse kinetic-law expression.

10.48.3.16 getReverseRateConstant()

```
double BioReaction::getReverseRateConstant ( ) const
```

Returns the reverse rate constant.

Returns

Reverse rate constant.

10.48.3.17 getReverseRateConstantParameterName()

```
std::string BioReaction::getReverseRateConstantParameterName ( ) const
```

Returns the reverse-rate parameter name.

Returns

Parameter name used to resolve the reverse rate constant.

10.48.3.18 isReversible()

```
bool BioReaction::isReversible ( ) const
```

Indicates whether the reaction is reversible.

Returns

`true` when the reaction has a reverse branch.

10.48.3.19 LoadInstance()

```
ModelDataDefinition * BioReaction::LoadInstance (
    Model * model,
    PersistenceRecord * fields ) [static]
```

Loads a biochemical reaction from serialized fields.

Parameters

<i>model</i>	Parent model.
<i>fields</i>	Serialized fields.

Returns

Newly created reaction instance.

10.48.3.20 NewInstance()

```
ModelDataDefinition * BioReaction::NewInstance (
    Model * model,
    std::string name = "" ) [static]
```

Creates a new empty reaction instance.

Parameters

<i>model</i>	Parent model.
<i>name</i>	Initial reaction name.

Returns

Newly created reaction instance.

10.48.3.21 resolveRateConstant()

```
double BioReaction::resolveRateConstant ( ) const
```

Resolves the effective forward rate constant.

Returns

Forward rate constant after parameter resolution.

10.48.3.22 resolveReverseRateConstant()

```
double BioReaction::resolveReverseRateConstant ( ) const
```

Resolves the effective reverse rate constant.

Returns

Reverse rate constant after parameter resolution.

10.48.3.23 setKineticLawExpression()

```
void BioReaction::setKineticLawExpression (
    std::string kineticLawExpression )
```

Sets the forward kinetic-law expression.

Parameters

<i>kineticLawExpression</i>	Expression text.
-----------------------------	------------------

10.48.3.24 setRateConstant()

```
void BioReaction::setRateConstant (
    double rateConstant )
```

Sets the forward rate constant.

Parameters

<i>rateConstant</i>	Forward rate constant value.
---------------------	------------------------------

10.48.3.25 setRateConstantParameterName()

```
void BioReaction::setRateConstantParameterName (
    std::string rateConstantParameterName )
```

Stores the name of the parameter used for the forward rate constant.

Parameters

<i>rateConstantParameterName</i>	Parameter name.
----------------------------------	-----------------

10.48.3.26 setReverseKineticLawExpression()

```
void BioReaction::setReverseKineticLawExpression (
    std::string reverseKineticLawExpression )
```

Sets the reverse kinetic-law expression.

Parameters

<i>reverseKineticLawExpression</i>	Expression text.
------------------------------------	------------------

10.48.3.27 setReverseRateConstant()

```
void BioReaction::setReverseRateConstant (
    double reverseRateConstant )
```

Sets the reverse rate constant.

Parameters

<i>reverseRateConstant</i>	Reverse rate constant value.
----------------------------	------------------------------

10.48.3.28 setReverseRateConstantParameterName()

```
void BioReaction::setReverseRateConstantParameterName (
    std::string reverseRateConstantParameterName )
```

Stores the name of the parameter used for the reverse rate constant.

Parameters

<i>reverseRateConstantParameterName</i>	Parameter name.
---	-----------------

10.48.3.29 setReversible()

```
void BioReaction::setReversible (
    bool reversible )
```

Marks the reaction as reversible or irreversible.

Parameters

<i>reversible</i>	New reversible flag.
-------------------	----------------------

10.48.3.30 show()

```
std::string BioReaction::show ( ) [override], [virtual]
```

Returns a textual representation of the reaction.

Returns

Human-readable reaction summary.

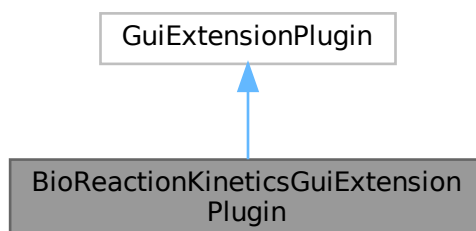
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

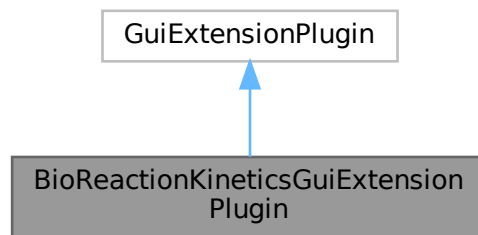
- plugins/data/BiochemicalSimulation/BioReaction.h
- plugins/data/BiochemicalSimulation/BioReaction.cpp

10.49 BioReactionKineticsGuiExtensionPlugin Class Reference

Inheritance diagram for BioReactionKineticsGuiExtensionPlugin:



Collaboration diagram for BioReactionKineticsGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId () const` override
- `std::string displayName () const` override
- `std::string version () const` override
- `std::vector< std::string > requiredModelPlugins () const` override
- `void registerContributions (GuiExtensionRegistry *registry) const` override

10.49.1 Member Function Documentation

10.49.1.1 displayName()

```
std::string BioReactionKineticsGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.49.1.2 extensionId()

```
std::string BioReactionKineticsGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.49.1.3 registerContributions()

```
void BioReactionKineticsGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.49.1.4 requiredModelPlugins()

```
std::vector< std::string > BioReactionKineticsGuiExtensionPlugin::requiredModelPlugins ( )  
const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.49.1.5 version()

```
std::string BioReactionKineticsGuiExtensionPlugin::version ( ) const [inline], [override],  
[virtual]
```

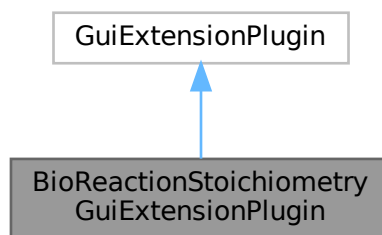
Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

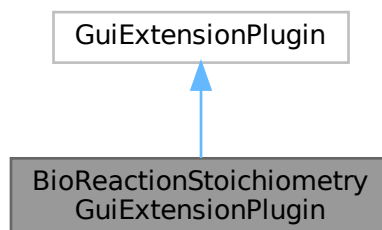
- applications/gui/genesys/extensions/BioReactionKineticsGuiExtensionPlugin.cpp

10.50 BioReactionStoichiometryGuiExtensionPlugin Class Reference

Inheritance diagram for BioReactionStoichiometryGuiExtensionPlugin:



Collaboration diagram for BioReactionStoichiometryGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

10.50.1 Member Function Documentation

10.50.1.1 displayName()

```
std::string BioReactionStoichiometryGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.50.1.2 extensionId()

```
std::string BioReactionStoichiometryGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.50.1.3 registerContributions()

```
void BioReactionStoichiometryGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.50.1.4 requiredModelPlugins()

```
std::vector< std::string > BioReactionStoichiometryGuiExtensionPlugin::requiredModelPlugins (
) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.50.1.5 version()

```
std::string BioReactionStoichiometryGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

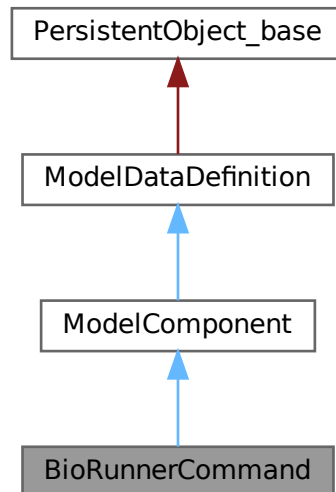
- `applications/gui/genesys/extensions/BioReactionStoichiometryGuiExtensionPlugin.cpp`

10.51 BioRunnerCommand Class Reference

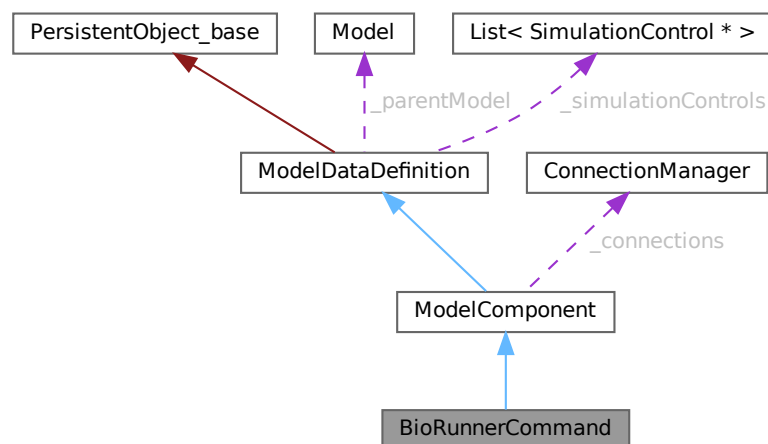
Component wrapper over [BioSimulatorRunner](#) command execution.

```
#include <plugins/components/BiochemicalSimulation/BioRunnerCommand.h>
```

Inheritance diagram for BioRunnerCommand:



Collaboration diagram for BioRunnerCommand:



Public Member Functions

- **BioRunnerCommand** ([Model](#) *model, std::string name="")
- void **setRunner** ([BioSimulatorRunner](#) *runner)
- [BioSimulatorRunner](#) * **getRunner** () const
- void **setCommand** (std::string command)
- std::string **getCommand** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastStatus** (std::string lastStatus)
- std::string **getLastStatus** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.

- `const std::string & getLabel () const`
Returns the user-visible label of this instance.
- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelspecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.51.1 Detailed Description

Component wrapper over [BioSimulatorRunner](#) command execution.

[BioRunnerCommand](#) dispatches one configured command into a referenced [BioSimulatorRunner](#) whenever an entity arrives and stores the last runner status/payload metadata for downstream model logic.

10.51.2 Member Function Documentation

10.51.2.1 `_check()`

```
bool BioRunnerCommand::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.51.2.2 `_createEditableDataDefinitions()`

```
void BioRunnerCommand::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.51.2.3 `_loadInstance()`

```
bool BioRunnerCommand::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.51.2.4 `_onDispatchEvent()`

```
void BioRunnerCommand::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.51.2.5 `_saveInstance()`

```
void BioRunnerCommand::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.51.2.6 `show()`

```
std::string BioRunnerCommand::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/BiochemicalSimulation/BioRunnerCommand.h
- plugins/components/BiochemicalSimulation/BioRunnerCommand.cpp

10.52 BioSBMLBridge Class Reference

Static Public Member Functions

- static bool **importFromString** ([Model](#) *model, const std::string &sbmlText, const std::string &preferredNetworkName, [BioSBMLImportResult](#) *result, std::string &errorMessage)
- static bool **exportToString** ([Model](#) *model, const std::string &requestedNetworkName, std::string *sbmlText, [BioSBMLExportResult](#) *result, std::string &errorMessage)

The documentation for this class was generated from the following files:

- plugins/data/BiochemicalSimulation/BioSBMLBridge.h
- plugins/data/BiochemicalSimulation/BioSBMLBridge.cpp

10.53 BioSBMLExportResult Struct Reference

Public Attributes

- std::string **networkName**
- unsigned int **speciesExported** = 0
- unsigned int **parametersExported** = 0
- unsigned int **reactionsExported** = 0
- std::vector< std::string > **warnings**

The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/BioSBMLBridge.h

10.54 BioSBMLImportResult Struct Reference

Public Attributes

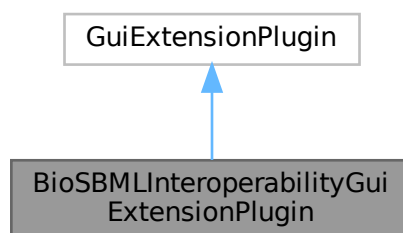
- std::string **networkName**
- unsigned int **speciesImported** = 0
- unsigned int **parametersImported** = 0
- unsigned int **reactionsImported** = 0
- std::vector< std::string > **warnings**

The documentation for this struct was generated from the following file:

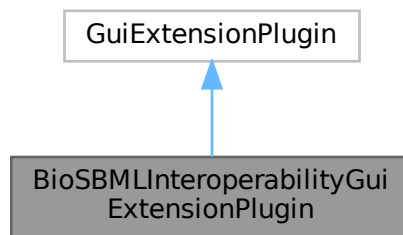
- plugins/data/BiochemicalSimulation/BioSBMLBridge.h

10.55 BioSBMLInteroperabilityGuiExtensionPlugin Class Reference

Inheritance diagram for BioSBMLInteroperabilityGuiExtensionPlugin:



Collaboration diagram for BioSBMLInteroperabilityGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId () const` override
- `std::string displayName () const` override
- `std::string version () const` override
- `std::vector< std::string > requiredModelPlugins () const` override
- `void registerContributions (GuiExtensionRegistry *registry) const` override

10.55.1 Member Function Documentation

10.55.1.1 displayName()

```
std::string BioSBMLInteroperabilityGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.55.1.2 extensionId()

```
std::string BioSBMLInteroperabilityGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.55.1.3 registerContributions()

```
void BioSBMLInteroperabilityGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.55.1.4 requiredModelPlugins()

```
std::vector< std::string > BioSBMLInteroperabilityGuiExtensionPlugin::requiredModelPlugins ( )  
const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.55.1.5 version()

```
std::string BioSBMLInteroperabilityGuiExtensionPlugin::version ( ) const [inline], [override],  
[virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

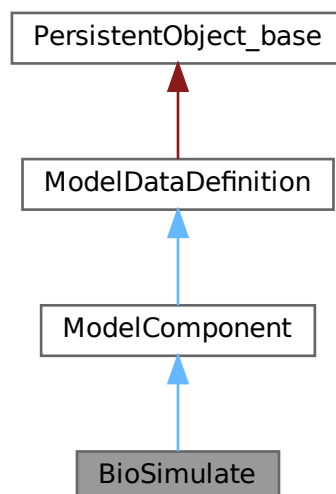
- applications/gui/genesys/extensions/BioSBMLInteroperabilityGuiExtensionPlugin.cpp

10.56 BioSimulate Class Reference

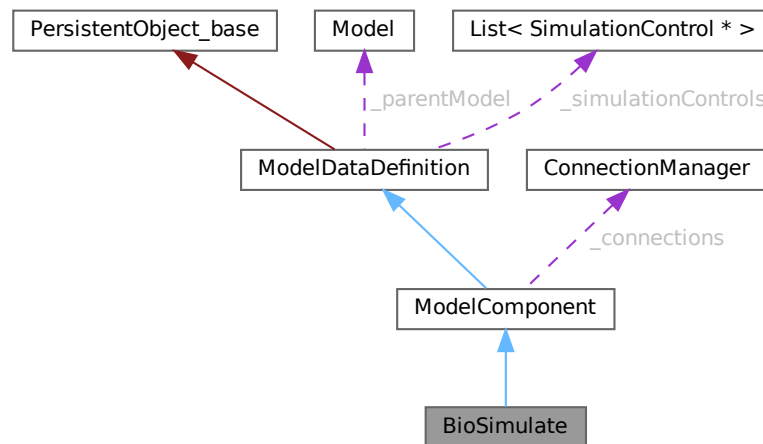
Component that triggers deterministic [BioNetwork](#) simulation.

```
#include <plugins/components/BiochemicalSimulation/BioSimulate.h>
```

Inheritance diagram for BioSimulate:



Collaboration diagram for BioSimulate:



Public Member Functions

- **BioSimulate** (**Model** *model, std::string name="")
- void **setBioNetwork** (**BioNetwork** *bioNetwork)
- **BioNetwork** * **getBioNetwork** () const
- void **setUseNetworkTimeWindow** (bool useNetworkTimeWindow)
- bool **getUseNetworkTimeWindow** () const
- void **setStartTime** (double startTime)
- double **getStartTime** () const
- void **setStopTime** (double stopTime)
- double **getStopTime** () const
- void **setStepSize** (double stepSize)
- double **getStepSize** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.

- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) (ModelComponent *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) (Model *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- Model * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.

- `TraceManager::Level` `getTraceLevelSpecific ()` const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` level)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled ()` const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation` * `GetPluginInformation ()`
- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent` *component)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent` *component)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelComponent` *component)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event` *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelDataDefinition` *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition` *modeldatum)
Resets simulation-time state between replications.
- static void `CreateInternalData` (`ModelDataDefinition` *modeldatum)
Creates internal and attached data required by this instance.
- static bool `CreateRelatedDataElements` (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()

- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.56.1 Detailed Description

Component that triggers deterministic [BioNetwork](#) simulation.

[BioSimulate](#) delegates simulation execution to a referenced [BioNetwork](#). Each arriving entity can either run the network with its own configured time window or with component-provided start/stop/step values.

10.56.2 Member Function Documentation

10.56.2.1 _check()

```
bool BioSimulate::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.56.2.2 _createEditableDataDefinitions()

```
void BioSimulate::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.56.2.3 _loadInstance()

```
bool BioSimulate::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.56.2.4 _onDispatchEvent()

```
void BioSimulate::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.56.2.5 _saveInstance()

```
void BioSimulate::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.56.2.6 show()

```
std::string BioSimulate::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

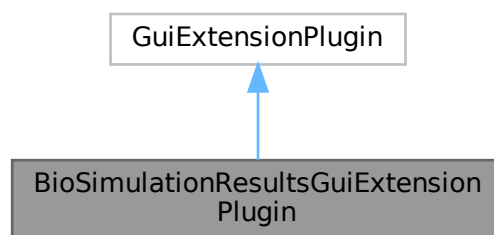
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

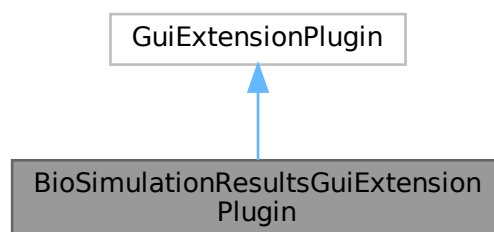
- plugins/components/BiochemicalSimulation/BioSimulate.h
- plugins/components/BiochemicalSimulation/BioSimulate.cpp

10.57 BioSimulationResultsGuiExtensionPlugin Class Reference

Inheritance diagram for BioSimulationResultsGuiExtensionPlugin:



Collaboration diagram for BioSimulationResultsGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

10.57.1 Member Function Documentation

10.57.1.1 displayName()

```
std::string BioSimulationResultsGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.57.1.2 extensionId()

```
std::string BioSimulationResultsGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.57.1.3 registerContributions()

```
void BioSimulationResultsGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.57.1.4 requiredModelPlugins()

```
std::vector< std::string > BioSimulationResultsGuiExtensionPlugin::requiredModelPlugins ( )
const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.57.1.5 version()

```
std::string BioSimulationResultsGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

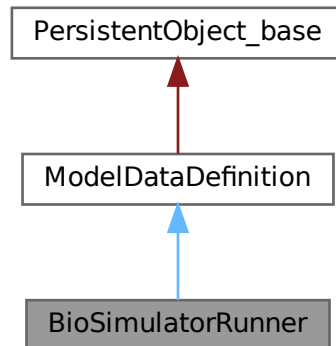
Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

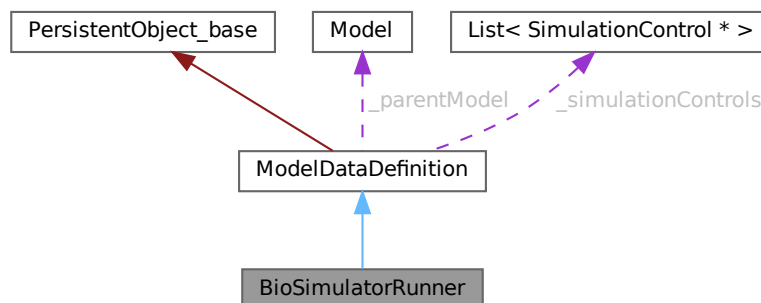
- `applications/gui/genesys/extensions/BioSimulationResultsGuiExtensionPlugin.cpp`

10.58 BioSimulatorRunner Class Reference

Inheritance diagram for BioSimulatorRunner:



Collaboration diagram for BioSimulatorRunner:



Public Member Functions

- **BioSimulatorRunner** ([Model](#) *model, std::string name="")
- void **setBackend** (std::string backend)
- std::string **getBackend** () const
- void **setModelSourceType** (std::string modelSourceType)
- std::string **getModelSourceType** () const
- void **setModelSource** (std::string modelSource)
- std::string **getModelSource** () const
- void **setCommand** (std::string command)
- std::string **getCommand** () const
- void **setLastStatus** (std::string lastStatus)

- `std::string getLastStatus ()` const
- `void setLastErrorMessage (std::string lastErrorMessage)`
- `std::string getLastErrorMessage ()` const
- `void setLastResponsePayload (std::string lastResponsePayload)`
- `std::string getLastResponsePayload ()` const
- `void setLastResponseFilename (std::string lastResponseFilename)`
- `std::string getLastResponseFilename ()` const
- `void setTargetBioNetworkName (std::string targetBioNetworkName)`
- `std::string getTargetBioNetworkName ()` const
- `void setWorkingDirectory (std::string workingDirectory)`
- `std::string getWorkingDirectory ()` const
- `void setWorkingInputFilename (std::string workingInputFilename)`
- `std::string getWorkingInputFilename ()` const
- `void setWorkingOutputFilename (std::string workingOutputFilename)`
- `std::string getWorkingOutputFilename ()` const
- `void setEndpointOrLibrary (std::string endpointOrLibrary)`
- `std::string getEndpointOrLibrary ()` const
- `void setTimeoutSeconds (unsigned int timeoutSeconds)`
- `unsigned int getTimeoutSeconds ()` const
- `void setAutoValidateModel (bool autoValidateModel)`
- `bool getAutoValidateModel ()` const
- `bool executeCommand (std::string &errorMessage)`
- `virtual std::string show ()` override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- `virtual ~ModelDataDefinition ()`
Releases resources owned by this data definition.
- `Util::identification getId ()` const
Returns the unique identifier of this instance.
- `void setName (const std::string &name)`
Updates the logical name of this instance.
- `const std::string & getName ()` const
Returns the logical name of this instance.
- `void setLabel (const std::string &label)`
Updates the user-visible label of this instance.
- `const std::string & getLabel ()` const
Returns the user-visible label of this instance.
- `const std::string & getClassname ()` const
Returns the runtime class name used by the plugin system.
- `Model * getParentModel ()` const
Returns the parent model that owns this definition.
- `bool isReportStatistics ()` const
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name)` const
Looks up a named internal child data definition.

- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelDataDefinition](#)

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.58.1 Member Function Documentation

10.58.1.1 [_check\(\)](#)

```
bool BioSimulatorRunner::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.58.1.2 [_loadInstance\(\)](#)

```
bool BioSimulatorRunner::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.58.1.3 [_saveInstance\(\)](#)

```
void BioSimulatorRunner::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.58.1.4 show()

```
std::string BioSimulatorRunner::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

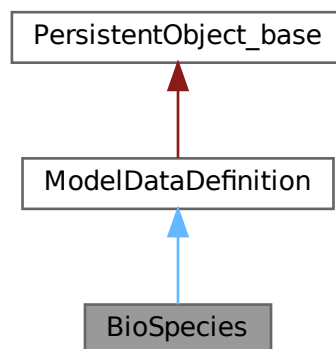
The documentation for this class was generated from the following files:

- plugins/data/BiochemicalSimulation/BioSimulatorRunner.h
- plugins/data/BiochemicalSimulation/BioSimulatorRunner.cpp

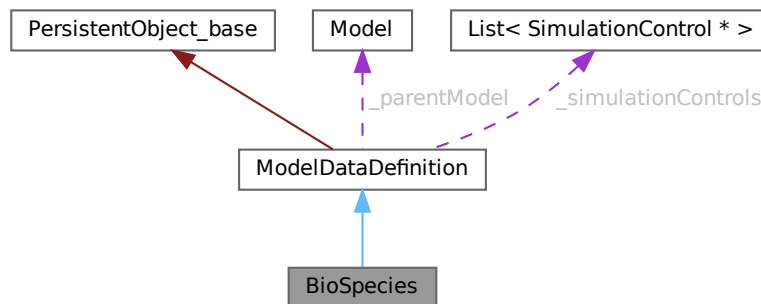
10.59 BioSpecies Class Reference

```
#include <plugins/data/BiochemicalSimulation/BioSpecies.h>
```

Inheritance diagram for BioSpecies:



Collaboration diagram for BioSpecies:



Public Member Functions

- **BioSpecies** ([Model](#) *model, std::string name="")
- void **setInitialAmount** (double initialAmount)
- double **getInitialAmount** () const
- void **setAmount** (double amount)
- double **getAmount** () const
- void **setConstant** (bool constant)
- bool **isConstant** () const
- void **setBoundaryCondition** (bool boundaryCondition)
- bool **isBoundaryCondition** () const
- void **setUnit** (std::string unit)
- std::string **getUnit** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- *Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- *Releases resources owned by this data definition.*
- Util::identification [getId](#) () const
- *Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)
- *Updates the logical name of this instance.*
- const std::string & [getName](#) () const
- *Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
- *Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
- *Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const

- Returns the runtime class name used by the plugin system.*
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- **List**< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- **TraceManager::Level** **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (**TraceManager::Level** level)
Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (**TraceManager::Level** traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const
Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static **ModelDataDefinition** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields)
- static **PluginInformation** * **GetPluginInformation** ()
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")

Static Public Member Functions inherited from **ModelDataDefinition**

- static **ModelDataDefinition** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelDataDefinition** *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)

Validates a data definition instance.

- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)

- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.59.1 Detailed Description

Biochemical state variable used by [BioReaction](#) and [BioNetwork](#).

A [BioSpecies](#) stores an initial amount for replication reset and a current amount for simulation. Constant and boundary-condition flags follow SBML-like semantics: [BioNetwork](#) still uses the current amount as kinetic input, but it must not overwrite the species amount while advancing the ODE state.

10.59.2 Member Function Documentation

10.59.2.1 **_check()**

```
bool BioSpecies::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.59.2.2 `_initBetweenReplications()`

```
void BioSpecies::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.59.2.3 `_loadInstance()`

```
bool BioSpecies::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.59.2.4 `_saveInstance()`

```
void BioSpecies::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.59.2.5 `show()`

```
std::string BioSpecies::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

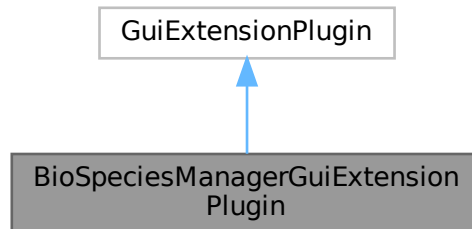
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

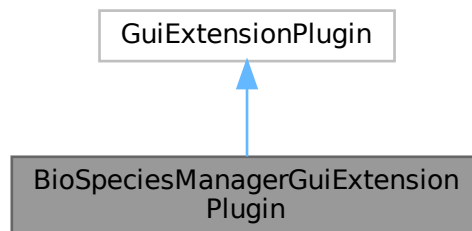
- plugins/data/BiochemicalSimulation/BioSpecies.h
- plugins/data/BiochemicalSimulation/BioSpecies.cpp

10.60 BioSpeciesManagerGuiExtensionPlugin Class Reference

Inheritance diagram for BioSpeciesManagerGuiExtensionPlugin:



Collaboration diagram for BioSpeciesManagerGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId () const override`
- `std::string displayName () const override`
- `std::string version () const override`
- `std::vector< std::string > requiredModelPlugins () const override`
- `void registerContributions (GuiExtensionRegistry *registry) const override`

10.60.1 Member Function Documentation

10.60.1.1 displayName()

```
std::string BioSpeciesManagerGuiExtensionPlugin::displayName ( ) const [inline], [override],
[virtual]
```

Implements [GuiExtensionPlugin](#).

10.60.1.2 extensionId()

```
std::string BioSpeciesManagerGuiExtensionPlugin::extensionId ( ) const [inline], [override],  
[virtual]
```

Implements [GuiExtensionPlugin](#).

10.60.1.3 registerContributions()

```
void BioSpeciesManagerGuiExtensionPlugin::registerContributions (   
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.60.1.4 requiredModelPlugins()

```
std::vector< std::string > BioSpeciesManagerGuiExtensionPlugin::requiredModelPlugins ( ) const  
[inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.60.1.5 version()

```
std::string BioSpeciesManagerGuiExtensionPlugin::version ( ) const [inline], [override],  
[virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

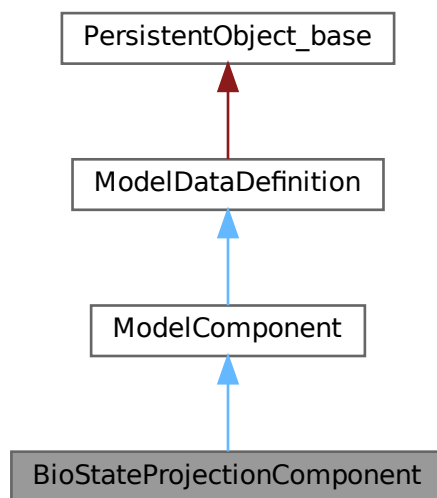
- applications/gui/genesys/extensions/BioSpeciesManagerGuiExtensionPlugin.cpp

10.61 BioStateProjectionComponent Class Reference

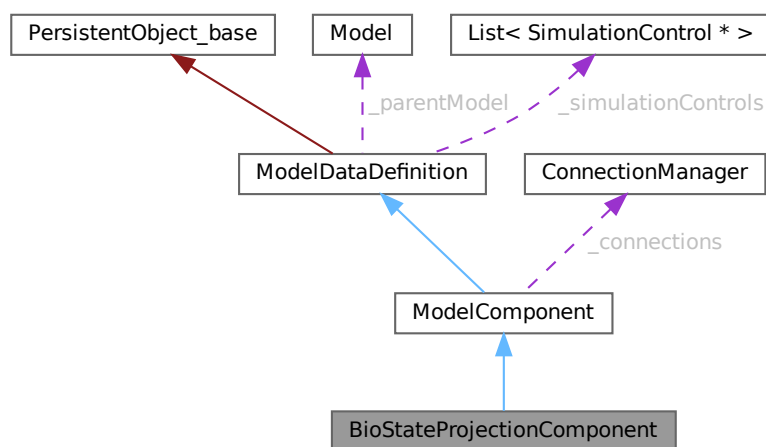
Projects selected [BioSpecies](#) values into a [WholeCellState](#).

```
#include <plugins/components/WholeCellModeling/BioStateProjectionComponent.h>
```

Inheritance diagram for BioStateProjectionComponent:



Collaboration diagram for BioStateProjectionComponent:



Classes

- struct [Projection](#)

Public Types

- enum class **ProjectionKind** { **MoleculeCount** , **MetaboliteAmount** }

Public Member Functions

- **BioStateProjectionComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **clearProjections** ()
- void **addMoleculeProjection** (const std::string &sourceSpeciesName, const std::string &targetKey="", double scale=1.0, double offset=0.0)
- void **addMetaboliteProjection** (const std::string &sourceSpeciesName, const std::string &targetKey="", double scale=1.0, double offset=0.0)
- unsigned int **getProjectionCount** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

- Updates the user-visible label of this instance.*

 - `const std::string & getLabel () const`

Returns the user-visible label of this instance.
- `const std::string & getClassname () const`

Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`

Returns the parent model that owns this definition.
- `bool isReportStatistics () const`

Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`

Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`

Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.
- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.
- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`

Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`

Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`

Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _initBetweenReplications ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createInternalStatisticReporters ()`
- `virtual void _createAttachedAttributes ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
- legacy helper methods used by the granular create* hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.61.1 Detailed Description

Projects selected [BioSpecies](#) values into a [WholeCellState](#).

[BioStateProjectionComponent](#) bridges the biochemical and whole-cell layers by copying chosen [BioSpecies](#) amounts into [WholeCellState](#) molecule counts or metabolite pools at simulation event boundaries. It is intentionally small and explicit: the caller decides which source species feed which whole-cell fields and which scale/offset transform applies to each projection.

10.61.2 Member Function Documentation

10.61.2.1 `_check()`

```
bool BioStateProjectionComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.61.2.2 `_createEditableDataDefinitions()`

```
void BioStateProjectionComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.61.2.3 _loadInstance()

```
bool BioStateProjectionComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.61.2.4 _onDispatchEvent()

```
void BioStateProjectionComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.61.2.5 _saveInstance()

```
void BioStateProjectionComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.61.2.6 show()

```
std::string BioStateProjectionComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

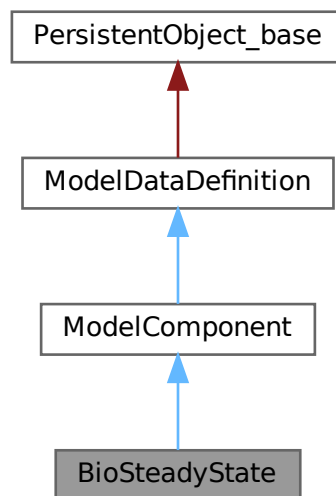
- `plugins/components/WholeCellModeling/BioStateProjectionComponent.h`
- `plugins/components/WholeCellModeling/BioStateProjectionComponent.cpp`

10.62 BioSteadyState Class Reference

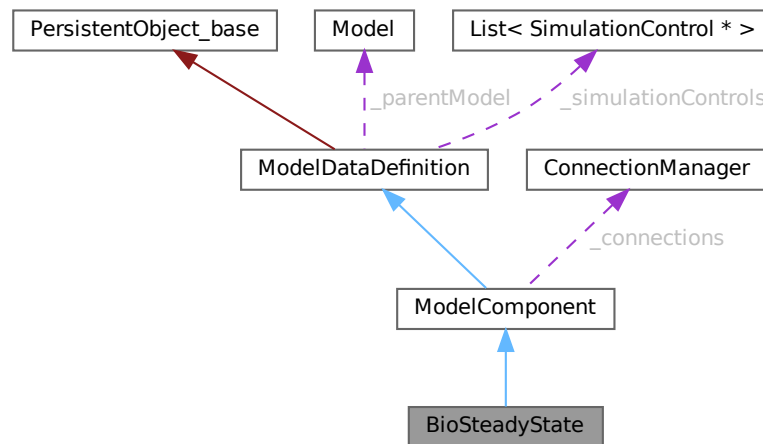
Component that checks steady-state diagnostics over [BioNetwork](#) results.

```
#include <plugins/components/BiochemicalSimulation/BioSteadyState.h>
```

Inheritance diagram for BioSteadyState:



Collaboration diagram for BioSteadyState:



Public Member Functions

- **BioSteadyState** ([Model](#) *model, std::string name="")
- void **setBioNetwork** ([BioNetwork](#) *bioNetwork)
- [BioNetwork](#) * **getBioNetwork** () const
- void **setTolerance** (double tolerance)
- double **getTolerance** () const
- void **setRunSimulationBeforeCheck** (bool runSimulationBeforeCheck)
- bool **getRunSimulationBeforeCheck** () const
- void **setLastSteady** (bool lastSteady)
- bool **getLastSteady** () const
- void **setLastMaxAbsoluteDerivative** (double lastMaxAbsoluteDerivative)
- double **getLastMaxAbsoluteDerivative** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.

- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.

- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void `_createAttachedAttributes` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()

- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.62.1 Detailed Description

Component that checks steady-state diagnostics over [BioNetwork](#) results.

[BioSteadyState](#) can optionally trigger one simulation run before evaluating steady-state through [BioNetwork](#) analysis helpers.

10.62.2 Member Function Documentation

10.62.2.1 **_check()**

```
bool BioSteadyState::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.62.2.2 `_createAttachedAttributes()`

```
void BioSteadyState::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.62.2.3 `_loadInstance()`

```
bool BioSteadyState::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.62.2.4 `_onDispatchEvent()`

```
void BioSteadyState::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.62.2.5 `_saveInstance()`

```
void BioSteadyState::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.62.2.6 show()

```
std::string BioSteadyState::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

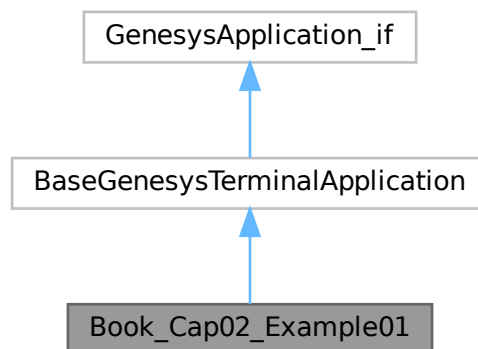
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

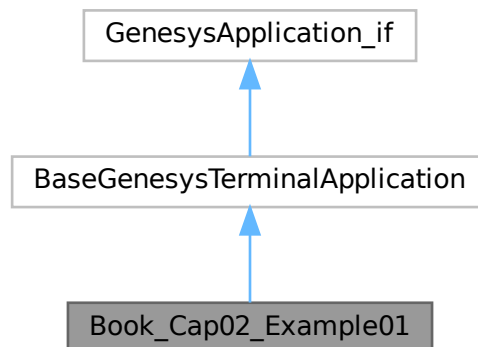
- plugins/components/BiochemicalSimulation/BioSteadyState.h
- plugins/components/BiochemicalSimulation/BioSteadyState.cpp

10.63 Book_Cap02_Example01 Class Reference

Inheritance diagram for Book_Cap02_Example01:



Collaboration diagram for Book_Cap02_Example01:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.63.1 Member Function Documentation

10.63.1.1 main()

```
int Book_Cap02_Example01::main (
    int argc,
    char ** argv ) [override], [virtual]
```

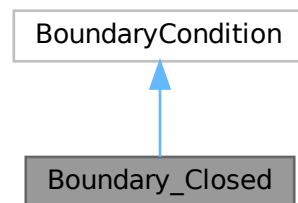
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

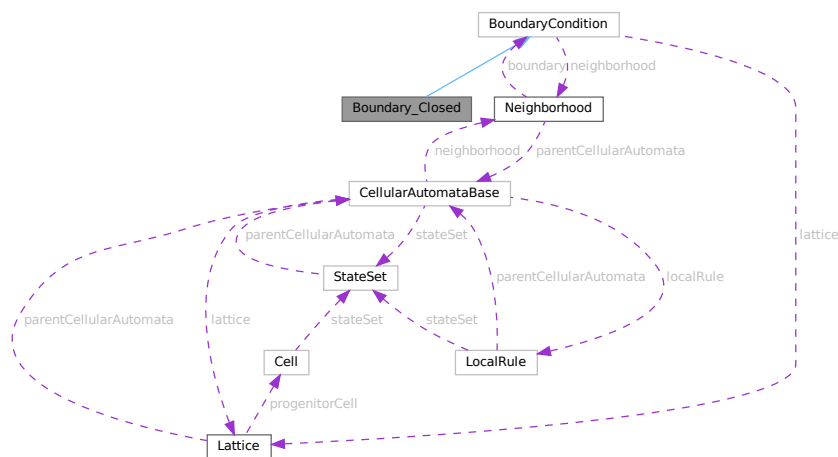
- applications/modelSpecific/book/Book_Cap02_Example01.h
- applications/modelSpecific/book/Book_Cap02_Example01.cpp

10.64 Boundary_Closed Class Reference

Inheritance diagram for Boundary_Closed:



Collaboration diagram for Boundary_Closed:



Public Member Functions

- **Boundary_Closed** ([Lattice](#) *lattice=nullptr, [Neighborhood](#) *neighborhood=nullptr)
- **Boundary_Closed** (const [Boundary_Closed](#) &orig)
- virtual [Cell](#) * **getNeighborCell** (std::vector< int > cellNDimPosition, std::vector< int > neighborNDimPosition) override
Pure virtual method that has to be overiden by derived class, responsible for defining the missing neighbor of a cell.
- void **setFixedState** (const [State](#) &fixedState)
- const [State](#) & **getFixedState** () const

Public Member Functions inherited from [BoundaryCondition](#)

- **BoundaryCondition** ([Lattice](#) *lattice=nullptr, [Neighborhood](#) *neighborhood=nullptr)
- **BoundaryCondition** (const [BoundaryCondition](#) &orig)
- virtual std::string **show** ()
- void **setNeighborhood** ([Neighborhood](#) *neighborhood)
- [Neighborhood](#) * **getNeighborhood** () const
- void **setLattice** ([Lattice](#) *lattice)
- [Lattice](#) * **getLattice** () const

Additional Inherited Members

Protected Attributes inherited from [BoundaryCondition](#)

- [Lattice](#) * **lattice**
- [Neighborhood](#) * **neighborhood**

10.64.1 Member Function Documentation

10.64.1.1 getNeighborCell()

```
virtual Cell * Boundary_Closed::getNeighborCell (
    std::vector< int > cellPosition,
    std::vector< int > neighborCellPosition ) [inline], [override], [virtual]
```

Pure virtual method that has to be overiden by derived class, responsible for defining the missing neighbor of a cell.

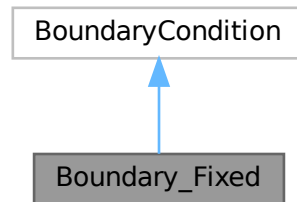
Implements [BoundaryCondition](#).

The documentation for this class was generated from the following file:

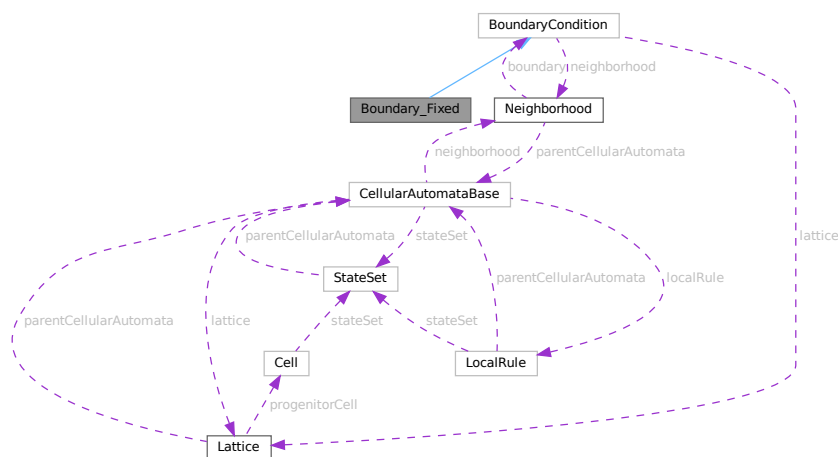
- plugins/components/ModalModel/CellularAutomata/Boundary_Closed.h

10.65 Boundary_Fixed Class Reference

Inheritance diagram for Boundary_Fixed:



Collaboration diagram for Boundary_Fixed:



Public Member Functions

- **Boundary_Fixed** ([Lattice](#) *lattice=nullptr, [Neighborhood](#) *neighborhood=nullptr)
- **Boundary_Fixed** (const [Boundary_Fixed](#) &orig)
- virtual [Cell](#) * [getNeighborCell](#) (std::vector< int > cellNDimPosition, std::vector< int > neighborNDimPosition) override
Pure virtual method that has to be overriden by derived class, responsible for defining the missing neighbor of a cell.
- void **setFixedState** (const [State](#) &fixedState)
- const [State](#) & **getFixedState** () const

Public Member Functions inherited from [BoundaryCondition](#)

- **BoundaryCondition** ([Lattice](#) *lattice=nullptr, [Neighborhood](#) *neighborhood=nullptr)
- **BoundaryCondition** (const [BoundaryCondition](#) &orig)
- virtual std::string **show** ()
- void **setNeighborhood** ([Neighborhood](#) *neighborhood)
- [Neighborhood](#) * **getNeighborhood** () const
- void **setLattice** ([Lattice](#) *lattice)
- [Lattice](#) * **getLattice** () const

Additional Inherited Members

Protected Attributes inherited from [BoundaryCondition](#)

- [Lattice](#) * **lattice**
- [Neighborhood](#) * **neighborhood**

10.65.1 Member Function Documentation

10.65.1.1 getNeighborCell()

```
virtual Cell * Boundary_Fixed::getNeighborCell (
    std::vector< int > cellPosition,
    std::vector< int > neighborCellPosition ) [inline], [override], [virtual]
```

Pure virtual method that has to be overriden by derived class, responsible for defining the missing neighbor of a cell.

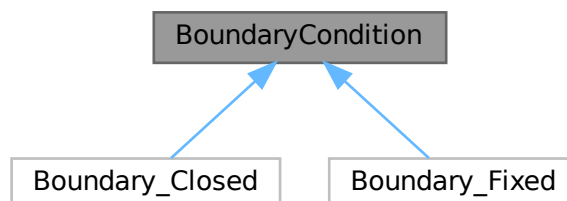
Implements [BoundaryCondition](#).

The documentation for this class was generated from the following file:

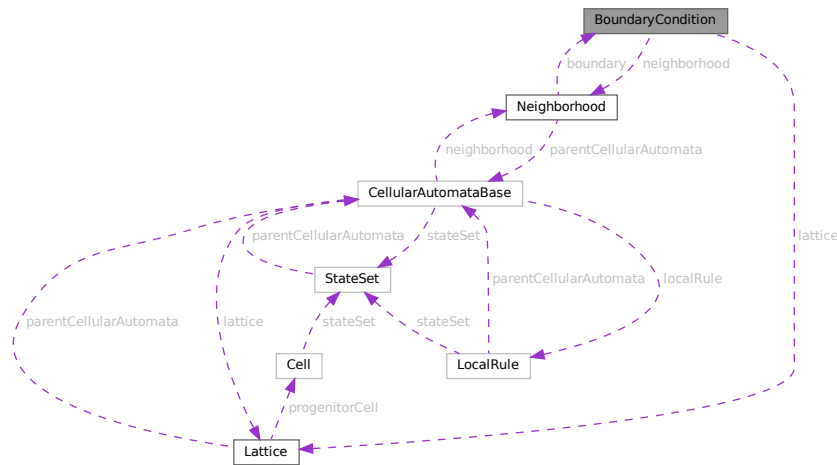
- plugins/components/ModalModel/CellularAutomata/Boundary_Fixed.h

10.66 BoundaryCondition Class Reference

Inheritance diagram for BoundaryCondition:



Collaboration diagram for BoundaryCondition:



Public Member Functions

- **BoundaryCondition** ([Lattice](#) *lattice=nullptr, [Neighborhood](#) *neighborhood=nullptr)
- **BoundaryCondition** (const [BoundaryCondition](#) &orig)
- virtual std::string **show** ()
- virtual [Cell](#) * **getNeighborCell** (std::vector< int > cellPosition, std::vector< int > neighborCellPosition)=0
Pure virtual method that has to be overiden by derived class, responsible for defining the missing neighbor of a cell.
- void **setNeighborhood** ([Neighborhood](#) *neighborhood)
- [Neighborhood](#) * **getNeighborhood** () const
- void **setLattice** ([Lattice](#) *lattice)
- [Lattice](#) * **getLattice** () const

Protected Attributes

- [Lattice](#) * **lattice**
- [Neighborhood](#) * **neighborhood**

10.66.1 Member Function Documentation

10.66.1.1 getNeighborCell()

```
virtual Cell * BoundaryCondition::getNeighborCell (
    std::vector< int > cellPosition,
    std::vector< int > neighborCellPosition ) [pure virtual]
```

Pure virtual method that has to be overiden by derived class, responsible for defining the missing neighbor of a cell.

Implemented in [Boundary_Closed](#), and [Boundary_Fixed](#).

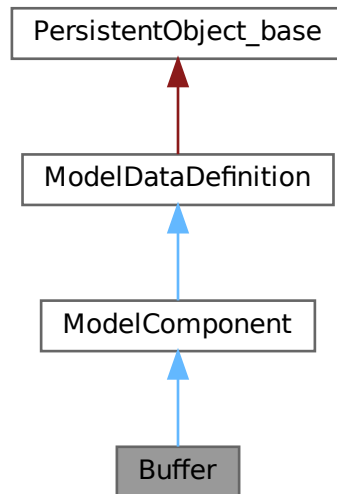
The documentation for this class was generated from the following file:

- plugins/components/ModalModel/CellularAutomata/BoundaryCondition.h

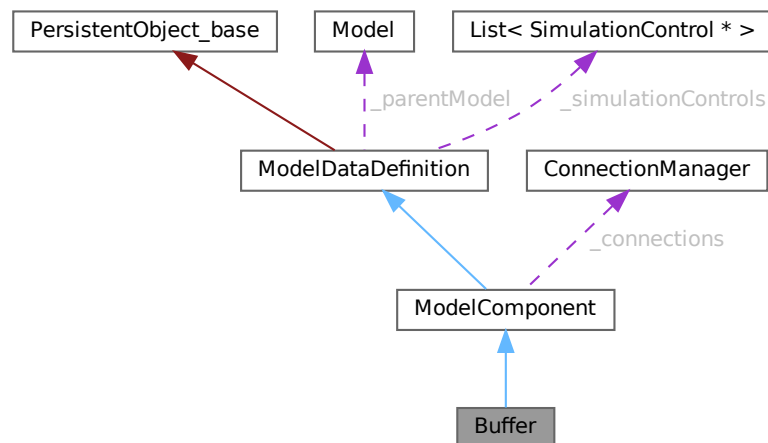
10.67 Buffer Class Reference

```
#include <plugins/components/DiscreteProcessing/Buffer.h>
```

Inheritance diagram for Buffer:



Collaboration diagram for Buffer:



Public Types

- enum class **AdvanceOn** : int { **NewArrivals** = 0 , **Signal** = 1 , **num_elements** = 2 }
- enum class **ArrivalOnFullBufferRule** : int { **Dispose** = 0 , **SendToBulkPort** = 1 , **ReplaceLastPosition** = 2 , **num_elements** = 3 }

Public Member Functions

- **Buffer** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
- *virtual public methods*
- Buffer::ArrivalOnFullBufferRule **getarrivalOnFullBufferRule** () const
- void **setArrivalOnFullBufferRule** (Buffer::ArrivalOnFullBufferRule newArrivalOnFullBufferRule)
- Buffer::AdvanceOn **getadvanceOn** () const
- void **setAdvanceOn** (Buffer::AdvanceOn newAdvanceOn)
- unsigned int **getcapacity** () const
- void **setCapacity** (unsigned int newCapacity)
- [SignalData](#) * **getsignal** () const
- void **setSignal** ([SignalData](#) *newSignal)

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- *new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- *Connects this component to another component.*
- bool **hasBreakpointAt** ()
- *Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string _description)
- *Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const
- *Returns the free-form description associated with this component.*
- void **remove** ([ModelComponent](#) *comp)
- *Removes a connected component from this component.*

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
- *Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- *Releases resources owned by this data definition.*
- Util::identification **getId** () const
- *Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- *Updates the logical name of this instance.*
- const std::string & **getName** () const
- *Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
- *Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const
- *Returns the user-visible label of this instance.*

- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static std::string convertEnumToStr (AdvanceOn advance)`
- `static std::string convertEnumToStr (ArrivalOnFullBufferRule arrival)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) () override
- virtual void [_initBetweenReplications](#) () override
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *property) override
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void [traceReport](#) (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.67.1 Detailed Description

This component ...

10.67.2 Member Function Documentation

10.67.2.1 `_addSimulationControl()`

```
void Buffer::_addSimulationControl (
    SimulationControl * property ) [override], [protected], [virtual]
```

This method is necessary only for those components that instantiate internal elements that must exist before simulation starts and even before model checking. That's the case of components that have internal [StatisticsCollectors](#), since others components may refer to them as expressions (as in "TVAG(ThisCSTAT)") and therefore the model datum must exist before checking such expression

Reimplemented from [ModelDataDefinition](#).

10.67.2.2 `_check()`

```
bool Buffer::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.67.2.3 _createEditableDataDefinitions()

```
void Buffer::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.67.2.4 _getParserChangesInformation()

```
ParserChangesInformation * Buffer::_getParserChangesInformation ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

10.67.2.5 _initBetweenReplications()

```
void Buffer::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.67.2.6 _loadInstance()

```
bool Buffer::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.67.2.7 _onDispatchEvent()

```
void Buffer::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.67.2.8 `_saveInstance()`

```
void Buffer::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.67.2.9 `show()`

```
std::string Buffer::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

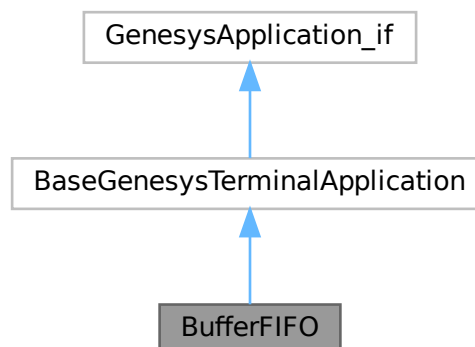
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

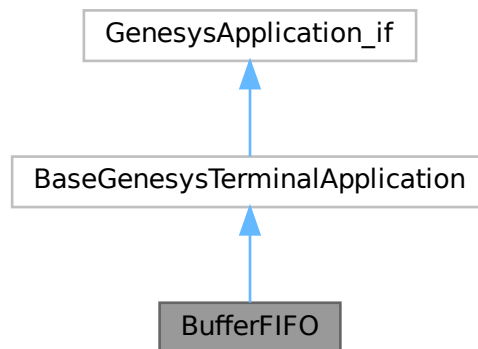
- plugins/components/DiscreteProcessing/Buffer.h
- plugins/components/DiscreteProcessing/Buffer.cpp

10.68 BufferFIFO Class Reference

Inheritance diagram for BufferFIFO:



Collaboration diagram for BufferFIFO:



Public Member Functions

- virtual int [main](#) (int argc, char **argv)

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.68.1 Member Function Documentation

10.68.1.1 main()

```
virtual int BufferFIFO::main (
    int argc,
    char ** argv ) [inline], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following file:

- applications/modelSpecific/teaching/BufferFIFO.h

10.69 QCPAxisPainterPrivate::CachedLabel Struct Reference

Public Attributes

- QPoint **offset**
- QPixmap **pixmap**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/qcustomplot.h

10.70 QCPLabelPainterPrivate::CachedLabel Struct Reference

Public Attributes

- QPoint **offset**
- QPixmap **pixmap**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/qcustomplot.h

10.71 ModelLifecycleController::Callbacks Struct Reference

Public Attributes

- std::function< void(const std::string &)> **insertCommandInConsole**
- std::function< void([Model](#) *)> **initUiForNewModel**
- std::function< void()> **actualizeActions**
- std::function< void()> **actualizeTabPanels**
- std::function< void(bool)> **actualizeModelTextHasChanged**
- std::function< bool()> **checkModel**
- std::function< bool()> **setSimulationModelBasedOnText**
- std::function< void()> **clearModelEditors**
- std::function< bool(QString)> **saveGraphicalModel**
- std::function< bool(QFile *, QString)> **saveTextModel**
- std::function< [Model](#) *(std::string)> **loadGraphicalModel**
- std::function< void()> **connectSceneSignals**
- std::function< void(const char *)> **disconnectSceneSignals**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/controllers/ModelLifecycleController.h

10.72 SimulationEventController::Callbacks Struct Reference

Public Attributes

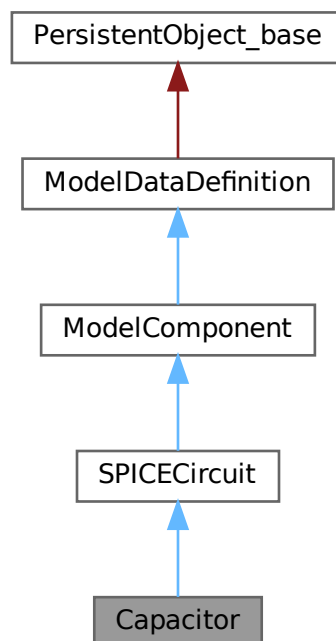
- `std::function< void()>` **actualizeActions**
- `std::function< void(SimulationEvent *)>` **actualizeSimulationEvents**
- `std::function< void(bool)>` **actualizeDebugEntities**
- `std::function< void(bool)>` **actualizeDebugVariables**
- `std::function< void(SimulationEvent *)>` **actualizeGraphicalModel**

The documentation for this struct was generated from the following file:

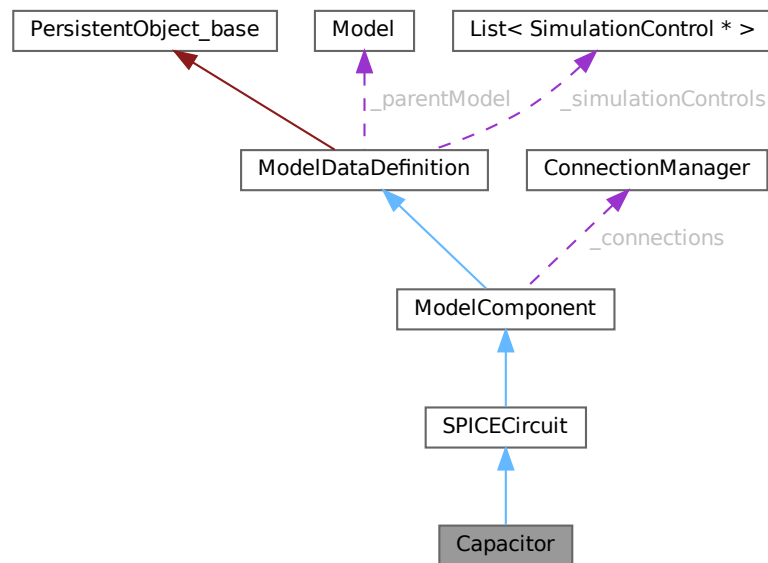
- `applications/gui/genesys/controllers/SimulationEventController.h`

10.73 Capacitor Class Reference

Inheritance diagram for Capacitor:



Collaboration diagram for Capacitor:



Public Member Functions

- **Capacitor** ([Model](#) *model, std::string name="")
- void **setCapacitance** (float capacitance)
- void **Build** () override

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)

- *Connects this component to another component.*
- bool `hasBreakpointAt ()`
Indicates whether a simulation breakpoint is set on this component.
- void `setDescription (std::string _description)`
Updates the free-form description shown by the GUI.
- std::string `getDescription () const`
Returns the free-form description associated with this component.
- void `remove (ModelComponent *comp)`
Removes a connected component from this component.

Public Member Functions inherited from `ModelDataDefinition`

- `ModelDataDefinition (Model *model, std::string datadefinitionType, std::string name="", bool insertIntoModel=true)`
Creates a model data definition and optionally inserts it into the model.
- virtual `~ModelDataDefinition ()`
Releases resources owned by this data definition.
- Util::identification `getId () const`
Returns the unique identifier of this instance.
- void `setName (const std::string &name)`
Updates the logical name of this instance.
- const std::string & `getName () const`
Returns the logical name of this instance.
- void `setLabel (const std::string &label)`
Updates the user-visible label of this instance.
- const std::string & `getLabel () const`
Returns the user-visible label of this instance.
- const std::string & `getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- bool `isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- void `setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- std::map< std::string, `ModelDataDefinition * > * getInternalData () const
Returns the full map of internal child data definitions.`
- std::map< std::string, `ModelDataDefinition * > * getAttachedData () const
Returns the full map of attached data definitions.`
- bool `hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- unsigned int `getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`

- Returns the simulation controls owned by this data definition.*
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- `void` `setTraceLevelSpecific` (`TraceManager::Level` level)
Updates the trace level used by this definition.
- `void` `defineTraceLevelSpecific` (`TraceManager::Level` traceLevelspecific, `bool` traceLevelSpecific↵ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- `bool` `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- `void` `setTraceLevelSpecificEnabled` (`bool` traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- `static` `PluginInformation` * `GetPluginInformation` ()
- `static` `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- `static` `ModelDataDefinition` * `NewInstance` (`Model` *model, `std::string` name="")

Static Public Member Functions inherited from `SPICECircuit`

- `static` `PluginInformation` * `GetPluginInformation` ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static` `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- `static` `ModelDataDefinition` * `NewInstance` (`Model` *model, `std::string` name="")
- `static` `std::vector`< `std::string` > `split` (`std::string` word, `char` split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from `ModelComponent`

- `static` `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static` `bool` `Check` (`ModelComponent` *component)
Validates a component instance.
- `static` `void` `CreateInternalData` (`ModelComponent` *component)
Creates internal data required by a component instance.
- `static` `void` `SaveInstance` (`PersistenceRecord` *fields, `ModelComponent` *component)
Serializes a component instance into a persistence record.
- `static` `void` `DispatchEvent` (`Event` *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()

- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
 - legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * [_connections](#) = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification [_id](#)
- std::string [_typename](#)
- bool [_reportStatistics](#)
- bool [_hasChanged](#)
- unsigned int [_modelLevel](#) = 0
- [Model](#) * [_parentModel](#)
- [List](#)< [SimulationControl](#) * > * [_simulationControls](#) = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level [_traceLevelSpecific](#)
- bool [_traceLevelSpecificEnabled](#) = false

10.73.1 Member Function Documentation

10.73.1.1 [_loadInstance\(\)](#)

```
bool Capacitor::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.73.1.2 [_saveInstance\(\)](#)

```
void Capacitor::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.73.1.3 [Build\(\)](#)

```
void Capacitor::Build ( ) [override], [virtual]
```

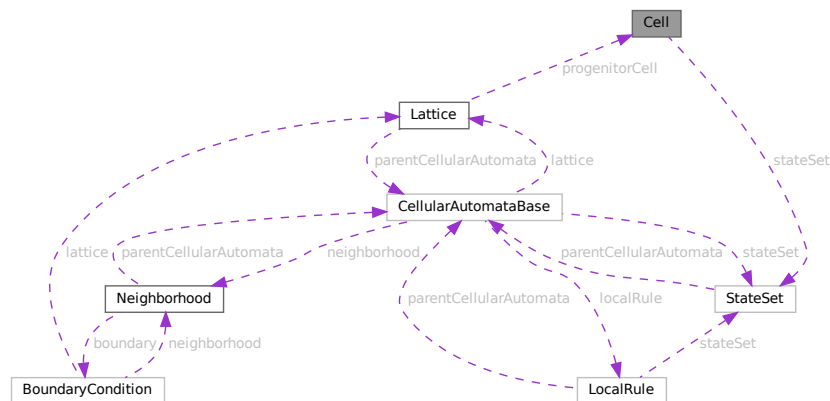
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

- plugins/components/ElectronicsSimulation/SPIECircuit.h
- plugins/components/ElectronicsSimulation/SPIECircuit.cpp

10.74 Cell Class Reference

Collaboration diagram for Cell:



Public Member Functions

- **Cell** ([StateSet](#) *stateSet=nullptr, long cellNum=0, std::vector< int > position={})
- **Cell** (long cellNum, std::vector< int > position)
- **Cell** (const [Cell](#) &orig)
- [Cell](#) & **operator=** (const [Cell](#) &orig)
- virtual std::string **show** ()
- bool **updateState** ()
update cell state (currentState = nextState)
- bool **isUpdatePending** ()
true if nextState differs from currentState
- virtual void **draw** ()
virtual method that can be override by derived classes
- void **setCellNumber** (long cellNum)
cell rank position in the lattice vector
- long **getCellNumber** () const
- void **setPosition** (std::vector< int > position)
- std::vector< int > **getPosition** () const
n-dimensional position
- void **setStateSet** ([StateSet](#) *stateSet)
- [StateSet](#) * **getStateSet** () const
- const [State](#) & **getPreviousState** () const
- bool **setCurrentState** (const [State](#) ¤tState)
- const [State](#) & **getCurrentState** () const
- bool **setNextState** (const [State](#) &nextState)
- const [State](#) & **getNextState** () const
- void **setNeighbors** (std::vector< [Cell](#) * > neighbors)
- std::vector< [Cell](#) * > **getNeighbors** () const

Protected Member Functions

- bool **acceptsState** (const [State](#) &state) const

Protected Attributes

- std::unique_ptr< [State](#) > **previousState**
- std::unique_ptr< [State](#) > **currentState**
- std::unique_ptr< [State](#) > **nextState**
- [StateSet](#) * **stateSet** = nullptr
- bool **updatePending**
- std::vector< int > **position**
- long **cellNum**
- std::vector< [Cell](#) * > **neighbors**

The documentation for this class was generated from the following files:

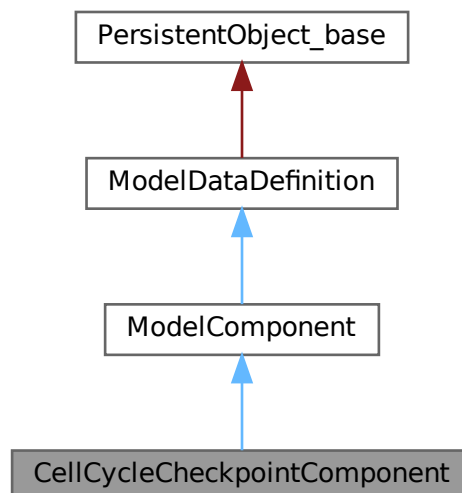
- plugins/components/ModalModel/CellularAutomata/Cell.h
- plugins/components/ModalModel/CellularAutomata/Cell.cpp

10.75 CellCycleCheckpointComponent Class Reference

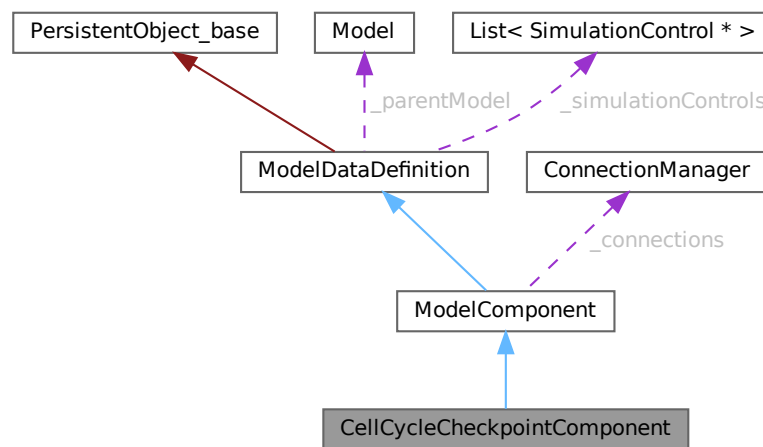
Evaluates lifecycle checkpoints and advances whole-cell step time.

```
#include <plugins/components/WholeCellModeling/CellCycleCheckpointComponent.h>
```

Inheritance diagram for CellCycleCheckpointComponent:



Collaboration diagram for CellCycleCheckpointComponent:



Public Member Functions

- **CellCycleCheckpointComponent** (**Model** *model, std::string name="")
- void **setWholeCellState** (**WholeCellState** *state)
- **WholeCellState** * **getWholeCellState** () const
- void **setDeltaT** (double deltaT)
- double **getDeltaT** () const
- void **setAdvanceWholeCellClock** (bool advance)
- bool **getAdvanceWholeCellClock** () const
- void **setEnergyMetaboliteKey** (std::string key)
- std::string **getEnergyMetaboliteKey** () const
- void **setCompartmentEnergyRegion** (std::string region)
- std::string **getCompartmentEnergyRegion** () const
- void **setCompartmentEnergyMetaboliteKey** (std::string key)
- std::string **getCompartmentEnergyMetaboliteKey** () const
- void **setStarvationAtpThreshold** (double threshold)
- double **getStarvationAtpThreshold** () const
- void **setCompartmentStarvationThreshold** (double threshold)
- double **getCompartmentStarvationThreshold** () const
- void **setLethalStarvationSteps** (unsigned int steps)
- unsigned int **getLethalStarvationSteps** () const
- void **setDivisionMassThreshold** (double threshold)
- double **getDivisionMassThreshold** () const
- void **setFtsZRingKey** (std::string key)
- std::string **getFtsZRingKey** () const
- void **setFtsZThreshold** (double threshold)
- double **getFtsZThreshold** () const
- void **setCriticalPathwayActivityKey** (std::string key)
- std::string **getCriticalPathwayActivityKey** () const
- void **setCriticalPathwayActivityThreshold** (double threshold)
- double **getCriticalPathwayActivityThreshold** () const
- unsigned int **getStarvationStreak** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.

- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelSpecific`, bool `traceLevelSpecificEnabled=true`)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation` ()
- static `ModelComponent * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
- static `ModelDataDefinition * NewInstance` (`Model *model`, `std::string name=""`)

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent *component`)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent *component`)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord *fields`, `ModelComponent *component`)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event *event`)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.75.1 Detailed Description

Evaluates lifecycle checkpoints and advances whole-cell step time.

[CellCycleCheckpointComponent](#) centralizes two concerns that otherwise become scattered across individual submodels:

- explicit advancement of [WholeCellState](#) current time and step count; and
- annotation of a coarse lifecycle phase based on energy availability, division readiness, and viability.

The component does not execute metabolism or division itself. It only reads [WholeCellState](#) and writes back lifecycle metadata such as:

- LifecyclePhase = newborn, growth, division_ready, starved, dead
- Viable

This keeps [WholeCellState](#) as a passive data container and concentrates the event-boundary checkpoint logic in a dedicated [ModelComponent](#).

10.75.2 Member Function Documentation

10.75.2.1 `_check()`

```
bool CellCycleCheckpointComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.75.2.2 `_createEditableDataDefinitions()`

```
void CellCycleCheckpointComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.75.2.3 `_initBetweenReplications()`

```
void CellCycleCheckpointComponent::_initBetweenReplications ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.75.2.4 `_loadInstance()`

```
bool CellCycleCheckpointComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.75.2.5 `_onDispatchEvent()`

```
void CellCycleCheckpointComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.75.2.6 _saveInstance()

```
void CellCycleCheckpointComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.75.2.7 show()

```
std::string CellCycleCheckpointComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

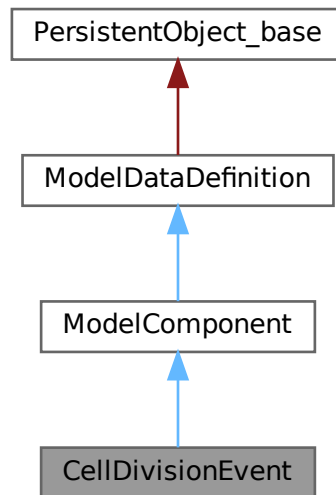
The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/CellCycleCheckpointComponent.h
- plugins/components/WholeCellModeling/CellCycleCheckpointComponent.cpp

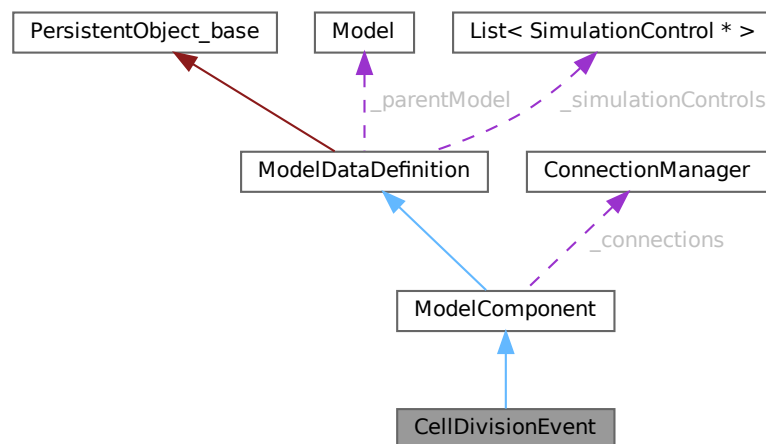
10.76 CellDivisionEvent Class Reference

```
#include <plugins/components/WholeCellModeling/CellDivisionEvent.h>
```

Inheritance diagram for CellDivisionEvent:



Collaboration diagram for CellDivisionEvent:



Public Member Functions

- **CellDivisionEvent** ([Model](#) *model, std::string name="")

- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setDivisionMassThreshold** (double threshold)
- double **getDivisionMassThreshold** () const
- void **setFtsZThreshold** (double threshold)
- double **getFtsZThreshold** () const
- void **setFtsZRingKey** (std::string key)
- std::string **getFtsZRingKey** () const
- void **setRandomSeed** (unsigned int seed)
- unsigned int **getRandomSeed** () const
- bool **getLastDivided** () const
- int **getDivisionCount** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

- Returns the user-visible label of this instance.*
- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool \_getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * \_getParserChangesInformation ()`
- `virtual void \_addSimulationControl (SimulationControl *control)`
- `virtual void \_createInternalAndAttachedData ()`
- `virtual void \_createInternalStatisticReporters ()`
- `virtual void \_createAttachedAttributes ()`
- `virtual void \_createNonEditableDataDefinitions ()`
- `void \_templateCreateInternalStatisticReporters ()`
- `void \_templateCreateAttachedAttributes ()`
- `void \_templateCreateNonEditableDataDefinitions ()`
- `void \_templateCreateEditableDataDefinitions ()`
- `void \_internalStatisticReportersClear ()`
- *legacy helper methods used by the granular create* hooks*
- `void \_internalDataClear ()`
- `void \_internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void \_internalDataRemove (const std::string &key)`
- `void \_attachedDataClear ()`
- `void \_attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_attachedDataRemove (const std::string &key)`
- `void \_attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void \_statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_statisticReporterRemove (const std::string &key)`
- `void \_statisticReportersClear ()`
- `void \_mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryAttachedAttributeRemove (const std::string &key)`
- `void \_mandatoryAttachedAttributesClear ()`
- `void \_mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void \_mandatoryEditableDataDefinitionsClear ()`
- `void \_optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_optionalEditableDataDefinitionRemove (const std::string &key)`
- `void \_optionalEditableDataDefinitionsClear ()`
- `void \_mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void \_mandatoryNonEditableDataDefinitionsClear ()`
- `void \_optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void \_optionalNonEditableDataDefinitionsClear ()`
- `void \_checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- *method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.76.1 Detailed Description

Discrete cell division event for whole-cell model simulations.

Implements the *M. genitalium* CellDivision process (Karr et al. 2012): Trigger: cell mass $\geq$ divisionMass↔ Threshold (default $2 \times$ cellInitialDryWeight) [AND](#) (ftsZRingCompletion key in [WholeCellState](#) $\geq$ ftsZThreshold [OR](#) ftsZThreshold == 0, i.e., mass-only trigger)

On division:

- Each integer molecule count is binomially partitioned between daughters (each copy independently assigned to daughter 1 with $p=0.5$)
- This simulated cell becomes daughter 1
- [Cell](#) mass and volume are set to half the pre-division values
- Simulation step count is reset to 0 (new generation)
- [WholeCellState](#) lifecycle metadata is updated to reflect the new generation

Output ports: Port 0: entity forwarded when no division occurred this step (normal flow) Port 1: entity forwarded immediately after a division event (optional handler)

The division mass threshold is read from [WholeCellState](#) metabolite pool entry "param.cellDivisionMassThreshold" when set by loadParameters(). Override with the divisionMassThreshold property for manual configuration.

FtsZ ring completion is read from [WholeCellState](#) molecule count key ftsZRingKey (default "FtsZ_ring_completion", expected range 0–1000 as integer per-mille value; divide by 1000 for fraction).

10.76.2 Member Function Documentation

10.76.2.1 `_check()`

```
bool CellDivisionEvent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.76.2.2 `_createEditableDataDefinitions()`

```
void CellDivisionEvent::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.76.2.3 `_initBetweenReplications()`

```
void CellDivisionEvent::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.76.2.4 `_loadInstance()`

```
bool CellDivisionEvent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.76.2.5 `_onDispatchEvent()`

```
void CellDivisionEvent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for [ModelComponents](#), not for [ModelDataElements](#).

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.76.2.6 `_saveInstance()`

```
void CellDivisionEvent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.76.2.7 `show()`

```
std::string CellDivisionEvent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

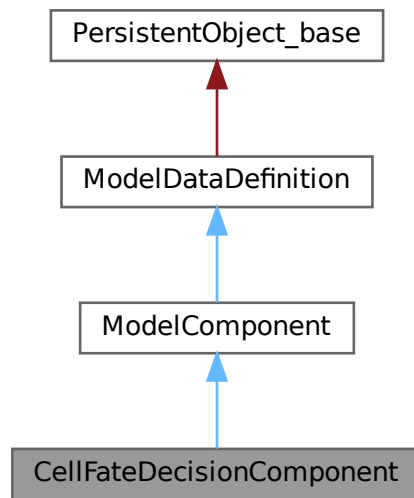
- `plugins/components/WholeCellModeling/CellDivisionEvent.h`
- `plugins/components/WholeCellModeling/CellDivisionEvent.cpp`

10.77 CellFateDecisionComponent Class Reference

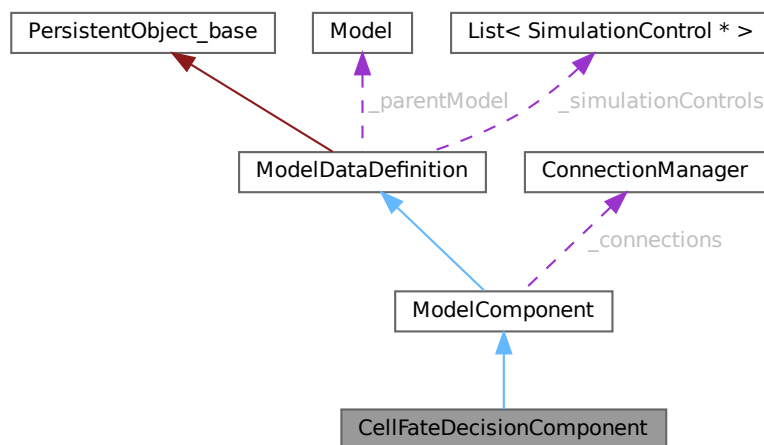
Routes entities according to the current whole-cell lifecycle state.

```
#include <plugins/components/WholeCellModeling/CellFateDecisionComponent.h>
```

Inheritance diagram for CellFateDecisionComponent:



Collaboration diagram for CellFateDecisionComponent:



Public Member Functions

- **CellFateDecisionComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setDivisionReadyPhase** (std::string phase)
- std::string **getDivisionReadyPhase** () const

- void **setStarvedPhase** (std::string phase)
- std::string **getStarvedPhase** () const
- void **setArrestedPhase** (std::string phase)
- std::string **getArrestedPhase** () const
- void **setDeadPhase** (std::string phase)
- std::string **getDeadPhase** () const
- unsigned int **getLastRoutedPort** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.

- bool `isReportStatistics ()` const
Indicates whether this data definition contributes report statistics.
- void `setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name)` const
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData ()` const
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData ()` const
Returns the full map of attached data definitions.
- bool `hasChanged ()` const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- unsigned int `getLevel ()` const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls ()` const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific ()` const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled ()` const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation ()`
- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check (ModelComponent *component)`
Validates a component instance.
- static void `CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- static void `SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- static void `DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.77.1 Detailed Description

Routes entities according to the current whole-cell lifecycle state.

[CellFateDecisionComponent](#) converts the coarse lifecycle annotation stored in [WholeCellState](#) into explicit simulation branching:

- port 0: viable default flow
- port 1: division-ready flow
- port 2: starved-but-still-viable flow
- port 3: dead or non-viable flow

The component does not mutate biological state. It only reads viability and lifecycle phase from [WholeCellState](#) and routes the entity to the appropriate downstream component.

10.77.2 Member Function Documentation

10.77.2.1 `_check()`

```
bool CellFateDecisionComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.77.2.2 `_createEditableDataDefinitions()`

```
void CellFateDecisionComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.77.2.3 `_initBetweenReplications()`

```
void CellFateDecisionComponent::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.77.2.4 `_loadInstance()`

```
bool CellFateDecisionComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.77.2.5 `_onDispatchEvent()`

```
void CellFateDecisionComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.77.2.6 _saveInstance()

```
void CellFateDecisionComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.77.2.7 show()

```
std::string CellFateDecisionComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

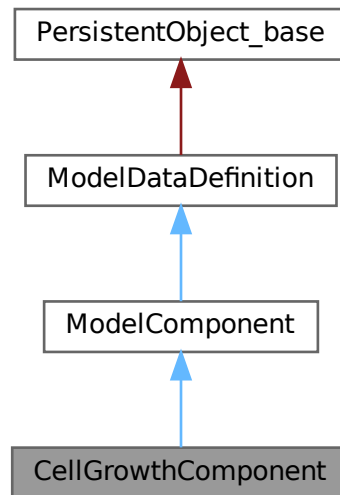
The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/CellFateDecisionComponent.h
- plugins/components/WholeCellModeling/CellFateDecisionComponent.cpp

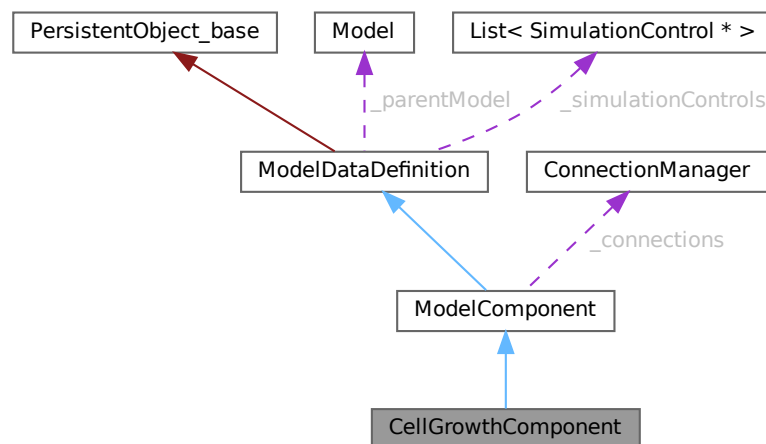
10.78 CellGrowthComponent Class Reference

```
#include <plugins/components/WholeCellModeling/CellGrowthComponent.h>
```

Inheritance diagram for CellGrowthComponent:



Collaboration diagram for CellGrowthComponent:



Public Member Functions

- **CellGrowthComponent** ([Model](#) *model, std::string name="")

- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setGrowthRate** (double rate)
- double **getGrowthRate** () const
- void **setEnergyMetaboliteKey** (std::string key)
- std::string **getEnergyMetaboliteKey** () const
- void **setEnergyHalfSaturation** (double value)
- double **getEnergyHalfSaturation** () const
- void **setDensity** (double density)
- double **getDensity** () const
- void **setDeltaT** (double dt)
- double **getDeltaT** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool` [_getSaveDefaultsOption](#) ()
- `virtual` [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- `virtual void` [_addSimulationControl](#) ([SimulationControl](#) *control)
- `virtual void` [_createInternalAndAttachedData](#) ()
- `virtual void` [_createInternalStatisticReporters](#) ()
- `virtual void` [_createAttachedAttributes](#) ()
- `virtual void` [_createNonEditableDataDefinitions](#) ()
- `void` [_templateCreateInternalStatisticReporters](#) ()
- `void` [_templateCreateAttachedAttributes](#) ()
- `void` [_templateCreateNonEditableDataDefinitions](#) ()
- `void` [_templateCreateEditableDataDefinitions](#) ()
- `void` [_internalStatisticReportersClear](#) ()
- *legacy helper methods used by the granular create* hooks*
- `void` [_internalDataClear](#) ()
- `void` [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- `void` [_internalDataRemove](#) (const std::string &key)
- `void` [_attachedDataClear](#) ()
- `void` [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_attachedDataRemove](#) (const std::string &key)
- `void` [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- `void` [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_statisticReporterRemove](#) (const std::string &key)
- `void` [_statisticReportersClear](#) ()
- `void` [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- `void` [_mandatoryAttachedAttributesClear](#) ()
- `void` [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- `void` [_mandatoryEditableDataDefinitionsClear](#) ()
- `void` [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- `void` [_optionalEditableDataDefinitionsClear](#) ()
- `void` [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- `void` [_mandatoryNonEditableDataDefinitionsClear](#) ()
- `void` [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- `void` [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- `void` [_optionalNonEditableDataDefinitionsClear](#) ()
- `void` [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- *method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void` [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- *not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void` [traceError](#) (const std::string &text, const std::exception &e)
- `void` [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- `void` [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- `void` [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- `void` [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- *Traces text to the simulation output.*
- `void` [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.78.1 Detailed Description

Continuous cell growth applied at each simulation step.

Implements exponential mass growth: $\text{mass}(t+\Delta t) = \text{mass}(t) \times (1 + \text{growthRate} \times \Delta t)$ $\text{volume}(\text{fL}) = (\text{mass_g} \times 1\text{e-}3 / \text{density_kg_m3}) \times 1\text{e}15$

Parameter defaults from CovertLab/WholeCell parameters.json (MIT): `growthRate` = 2.1393e-05 /s (states. $\leftrightarrow$ MetabolicReaction.meanInitialGrowthRate) `density` = 1100.0 kg/m³ (states.Geometry.density) `deltaT` = 60.0 s (must match simulation clock step)

10.78.2 Member Function Documentation

10.78.2.1 `_check()`

```
bool CellGrowthComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.78.2.2 `_createEditableDataDefinitions()`

```
void CellGrowthComponent::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.78.2.3 `_initBetweenReplications()`

```
void CellGrowthComponent::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.78.2.4 `_loadInstance()`

```
bool CellGrowthComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.78.2.5 `_onDispatchEvent()`

```
void CellGrowthComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.78.2.6 `_saveInstance()`

```
void CellGrowthComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.78.2.7 show()

```
std::string CellGrowthComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

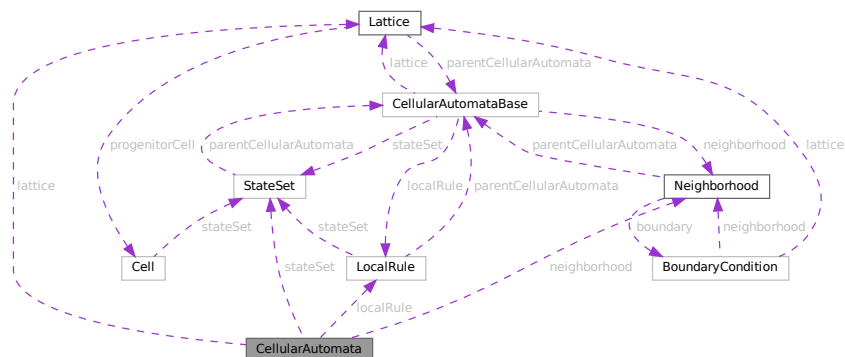
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/CellGrowthComponent.h
- plugins/components/WholeCellModeling/CellGrowthComponent.cpp

10.79 CellularAutomata Class Reference

Collaboration diagram for CellularAutomata:



Public Member Functions

- **CellularAutomata** (std::string name="", [Lattice](#) *lattice=nullptr, [StateSet](#) *stateSet=nullptr, [Neighborhood](#) *neighborhood=nullptr, [LocalRule](#) *localRule=nullptr)
- **CellularAutomata** (const [CellularAutomata](#) &orig)
- virtual bool **init** ()
Init lattice and cells.
- virtual void **step** ()
Create next configuration $c(t+1) = G(c(t))$
- virtual void **start** ()
Run (step) until end of simulation.
- virtual void **pause** ()

Pass the "run".

- `template<typename Class >`
`void addObserver (Class *object, void(Class::*function)(CellularAutomataEvent))`
- `std::string getName () const`
- `void setName (const std::string name)`
- `void setLocalRule (LocalRule *localRule)`
- `LocalRule * getLocalRule () const`
- `void setNeighborhood (Neighborhood *neighborhood)`
- `Neighborhood * getNeighborhood () const`
- `void setStateSet (StateSet *stateSet)`
- `StateSet * getStateSet () const`
- `void setLattice (Lattice *lattice)`
- `Lattice * getLattice () const`
- `void setSimulationLength (unsigned int simulationLength)`
- `unsigned int getSimulationLength () const`
- `unsigned int getSimulatedTime () const`
- `bool isRunning () const`

Protected Member Functions

- `void notifyObservers (CellularAutomataEvent event)`
Invoke observers then something (an event) happens.
- `virtual void applyLocalRule ()=0`
Pure virtual method that has to be overridden by derived classes.

Protected Attributes

- `Lattice * lattice`
- `StateSet * stateSet`
- `Neighborhood * neighborhood`
- `LocalRule * localRule`
- `std::string name`
- `unsigned int simulatedTime`
- `unsigned int simulationLength`
- `bool running`

The documentation for this class was generated from the following files:

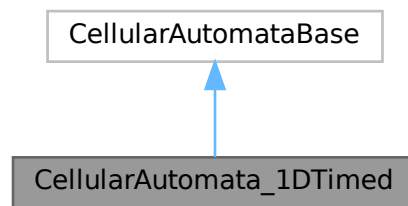
- `plugins/components/ModalModel/CellularAutomata/CellularAutomata.h`
- `plugins/components/ModalModel/CellularAutomata/CellularAutomata.cpp`

10.80 CellularAutomata_1DTimed Class Reference

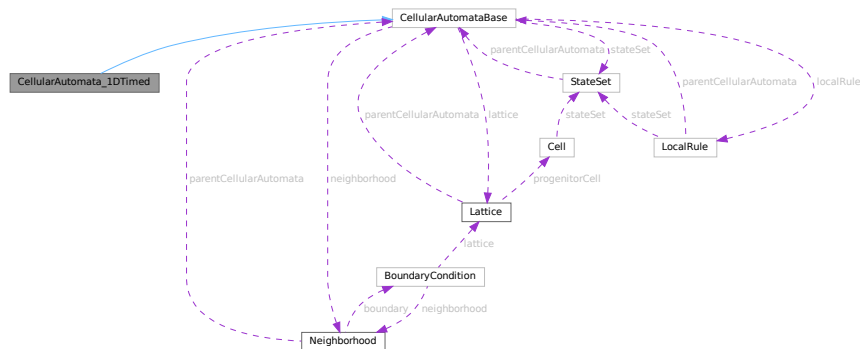
Complete uniform CA specific for unidimensional lattices where 2nd dimension is time (usefull for elementary CA)

```
#include <plugins/components/ModalModel/CellularAutomata/CellularAutomata_1DTimed.h>
```

Inheritance diagram for CellularAutomata_1DTimed:



Collaboration diagram for CellularAutomata_1DTimed:



Public Member Functions

- **CellularAutomata_1DTimed** ([Lattice](#) *lattice=nullptr, [StateSet](#) *stateSet=nullptr, [Neighborhood](#) *neighborhood=nullptr, [LocalRule](#) *localRule=nullptr)
- **CellularAutomata_1DTimed** (const [CellularAutomata_1DTimed](#) &orig)

Public Member Functions inherited from CellularAutomataBase

- **CellularAutomataBase** ([Lattice](#) *lattice=nullptr, [StateSet](#) *stateSet=nullptr, [Neighborhood](#) *neighborhood=nullptr, [LocalRule](#) *localRule=nullptr)
- **CellularAutomataBase** (const [CellularAutomataBase](#) &orig)
- virtual bool **init** ()

Init lattice and cells.

- virtual void **step** ()
Create next configuration $c(t+1) = G(c(t))$
- void **setLocalRule** ([LocalRule](#) *localRule)
- [LocalRule](#) * **getLocalRule** () const
- void **setNeighborhood** ([Neighborhood](#) *neighborhood)
- [Neighborhood](#) * **getNeighborhood** () const
- void **setStateSet** ([StateSet](#) *stateSet)
- [StateSet](#) * **getStateSet** () const
- void **setLattice** ([Lattice](#) *lattice)
- [Lattice](#) * **getLattice** () const

Protected Member Functions

- virtual void [applyLocalRule](#) () override
Pure virtual method that has to be overridden by derived classes.

Additional Inherited Members

Protected Attributes inherited from [CellularAutomataBase](#)

- [Lattice](#) * **lattice**
- [StateSet](#) * **stateSet**
- [Neighborhood](#) * **neighborhood**
- [LocalRule](#) * **localRule**

10.80.1 Detailed Description

Complete uniform CA specific for unidimensional lattices where 2nd dimension is time (usefull for elementary CA)

10.80.2 Member Function Documentation

10.80.2.1 [applyLocalRule\(\)](#)

```
virtual void CellularAutomata_1DTimed::applyLocalRule ( ) [inline], [override], [protected], [virtual]
```

Pure virtual method that has to be overridden by derived classes.

Implements [CellularAutomataBase](#).

The documentation for this class was generated from the following file:

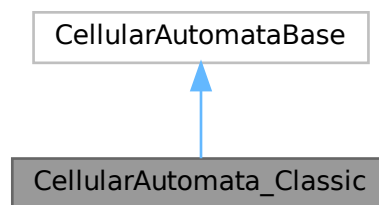
- plugins/components/ModalModel/CellularAutomata/CellularAutomata_1DTimed.h

10.81 CellularAutomata_Classic Class Reference

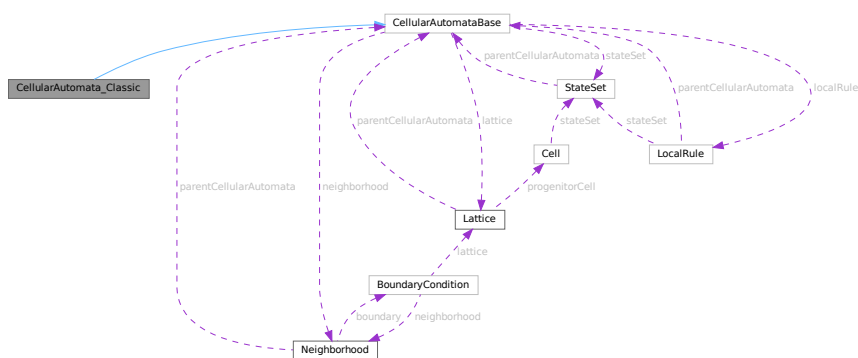
Complete uniform CA.

```
#include <plugins/components/ModalModel/CellularAutomata/CellularAutomata_↵
Classic.h>
```

Inheritance diagram for CellularAutomata_Classic:



Collaboration diagram for CellularAutomata_Classic:



Public Member Functions

- **CellularAutomata_Classic** ([Lattice](#) *lattice=nullptr, [StateSet](#) *stateSet=nullptr, [Neighborhood](#) *neighborhood=nullptr, [LocalRule](#) *localRule=nullptr)
- **CellularAutomata_Classic** (const [CellularAutomata_Classic](#) &orig)

Public Member Functions inherited from CellularAutomataBase

- **CellularAutomataBase** ([Lattice](#) *lattice=nullptr, [StateSet](#) *stateSet=nullptr, [Neighborhood](#) *neighborhood=nullptr, [LocalRule](#) *localRule=nullptr)
- **CellularAutomataBase** (const [CellularAutomataBase](#) &orig)
- virtual bool **init** ()

Init lattice and cells.

- virtual void **step** ()
- *Create next configuration $c(t+1) = G(c(t))$*
- void **setLocalRule** ([LocalRule](#) *localRule)
- [LocalRule](#) * **getLocalRule** () const
- void **setNeighborhood** ([Neighborhood](#) *neighborhood)
- [Neighborhood](#) * **getNeighborhood** () const
- void **setStateSet** ([StateSet](#) *stateSet)
- [StateSet](#) * **getStateSet** () const
- void **setLattice** ([Lattice](#) *lattice)
- [Lattice](#) * **getLattice** () const

Protected Member Functions

- virtual void [applyLocalRule](#) () override
- Pure virtual method that has to be overridden by derived classes.*

Additional Inherited Members

Protected Attributes inherited from [CellularAutomataBase](#)

- [Lattice](#) * **lattice**
- [StateSet](#) * **stateSet**
- [Neighborhood](#) * **neighborhood**
- [LocalRule](#) * **localRule**

10.81.1 Detailed Description

Complete uniform CA.

10.81.2 Member Function Documentation

10.81.2.1 [applyLocalRule\(\)](#)

```
virtual void CellularAutomata_Classic::applyLocalRule ( ) [inline], [override], [protected], [virtual]
```

Pure virtual method that has to be overridden by derived classes.

< overrides pure virtual method

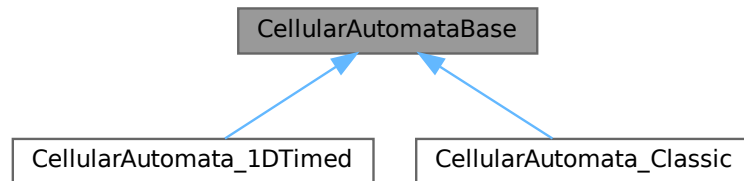
Implements [CellularAutomataBase](#).

The documentation for this class was generated from the following file:

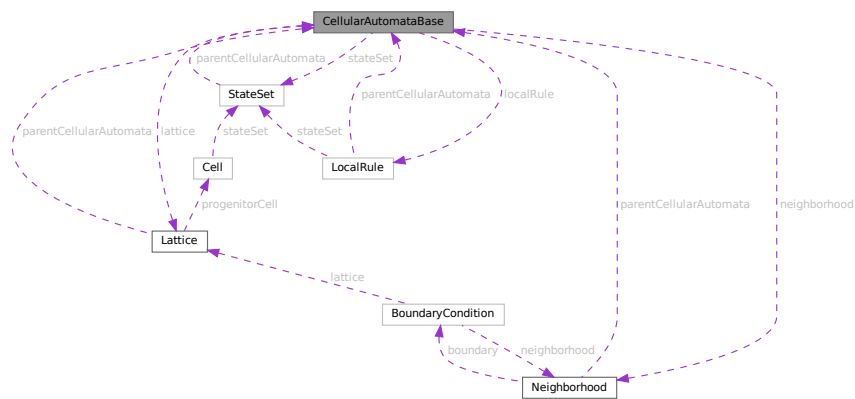
- plugins/components/ModalModel/CellularAutomata/CellularAutomata_Classic.h

10.82 CellularAutomataBase Class Reference

Inheritance diagram for CellularAutomataBase:



Collaboration diagram for CellularAutomataBase:



Public Member Functions

- **CellularAutomataBase** ([Lattice](#) *lattice=nullptr, [StateSet](#) *stateSet=nullptr, [Neighborhood](#) *neighborhood=nullptr, [LocalRule](#) *localRule=nullptr)
- **CellularAutomataBase** (const [CellularAutomataBase](#) &orig)
- virtual bool **init** ()
Init lattice and cells.
- virtual void **step** ()
Create next configuration $c(t+1) = G(c(t))$
- void **setLocalRule** ([LocalRule](#) *localRule)
- [LocalRule](#) * **getLocalRule** () const
- void **setNeighborhood** ([Neighborhood](#) *neighborhood)
- [Neighborhood](#) * **getNeighborhood** () const
- void **setStateSet** ([StateSet](#) *stateSet)
- [StateSet](#) * **getStateSet** () const
- void **setLattice** ([Lattice](#) *lattice)
- [Lattice](#) * **getLattice** () const

Protected Member Functions

- virtual void [applyLocalRule](#) ()=0
Pure virtual method that has to be overridden by derived classes.

Protected Attributes

- [Lattice](#) * **lattice**
- [StateSet](#) * **stateSet**
- [Neighborhood](#) * **neighborhood**
- [LocalRule](#) * **localRule**

10.82.1 Member Function Documentation**10.82.1.1 [applyLocalRule\(\)](#)**

```
void CellularAutomataBase::applyLocalRule ( ) [protected], [pure virtual]
```

Pure virtual method that has to be overridden by derived classes.

Implemented in [CellularAutomata_1DTimed](#), and [CellularAutomata_Classic](#).

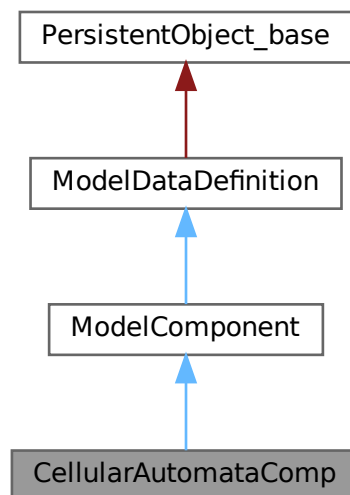
The documentation for this class was generated from the following files:

- plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h
- plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.cpp

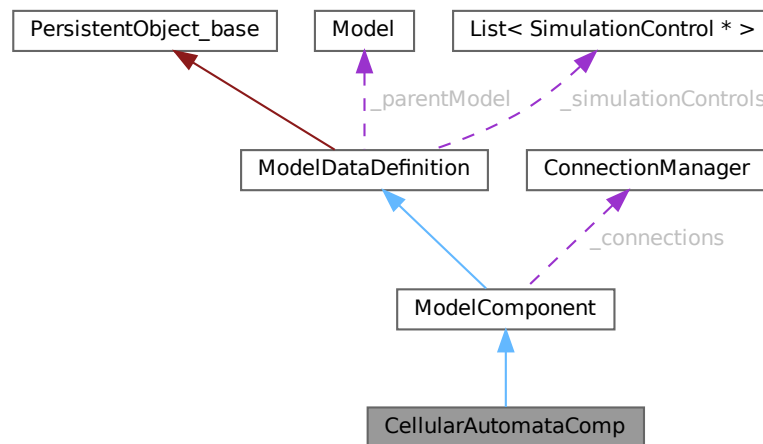
10.83 CellularAutomataComp Class Reference

```
#include <plugins/components/ModalModel/CellularAutomataComp.h>
```

Inheritance diagram for CellularAutomataComp:



Collaboration diagram for CellularAutomataComp:



Public Types

- enum class [CellularAutomataType](#) : int {
CLASSIC = 1 , **TIMED_1D** = 2 , **ASYNCHRONOUS** = 3 , **NONUNIFORMRULE** = 4 ,
NONUNIFORMNEIGHBOOR = 5 , **USERDEFINED** = 6 }
enums
- enum class **LatticeType** : int {
RETICULAR = 1 , **TRIANGULAR** = 2 , **HEXAGONAL** = 3 , **NETWORK** = 4 ,
USERDEFINED = 5 }
- enum class **NeighborhoodType** : int {
CENTERED = 1 , **BACKWARD** = 2 , **FORWARD** = 3 , **VONNEUMANN** = 4 ,
MOORE = 5 , **USERDEFINED** = 6 }
- enum class **BoundaryType** : int {
FIXED = 1 , **CLOSED** = 2 , **REFLEXIVE** = 3 , **ADIABATIC** = 4 ,
USERDEFINED = 5 }
- enum class **StateSetType** : int {
ENUMERATED = 1 , **INTEGERBASED** = 2 , **BITBASED** = 3 , **DOUBLEBASED** = 4 ,
USERDEFINED = 5 }
- enum class **LocalRuleType** : int {
ELEMENTAR_CA = 1 , **GAME_OF_LIFE** = 2 , **BIASED_COMPETITION** = 3 , **HPP** = 4 ,
SAND_ROCK = 5 , **USERDEFINED** = 99 }
- enum class **AutomataType** : int { **Temporary** = 0 , **Permanent** = 1 }

Public Member Functions

- **CellularAutomataComp** ([Model](#) *model, std::string name="")
constructors
- [CellularAutomataComp::CellularAutomataType](#) **getCellularAutomataType** () const
new public user methods for this component
- void **setCellularAutomataType** ([CellularAutomataComp::CellularAutomataType](#) newCellularAutomataType)
- [CellularAutomataComp::LatticeType](#) **getLatticeType** () const

- void **setLatticeType** (CellularAutomataComp::LatticeType newLatticeStructure)
- CellularAutomataComp::NeighborhoodType **getNeighborhoodType** () const
- void **setNeighborhoodType** (CellularAutomataComp::NeighborhoodType newNeighborhood)
- CellularAutomataComp::BoundaryType **geBoundaryType** () const
- void **setBoundaryType** (CellularAutomataComp::BoundaryType newBoundary)
- CellularAutomataComp::StateSetType **getStateSetType** () const
- void **setStateSetType** (CellularAutomataComp::StateSetType newStateSetType)
- Lattice * **getlattice** () const
- Neighborhood * **getNeighborhood** () const
- StateSet * **getStateSet** () const
- virtual std::string **show** () override
- *virtual public methods*
- CellularAutomataComp::LocalRuleType **getlocalRuleType** () const
- void **setLocalRuleType** (CellularAutomataComp::LocalRuleType newLocalRuleType)
- LocalRule * **getlocalRule** () const

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (Model *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- ConnectionManager * **getConnectionManager** () const
- *new public user methods for this component*
- void **connectTo** (ModelComponent *component, unsigned int inputPortNumber=0)
- *Connects this component to another component.*
- bool **hasBreakpointAt** ()
- *Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string _description)
- *Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const
- *Returns the free-form description associated with this component.*
- void **remove** (ModelComponent *comp)
- *Removes a connected component from this component.*

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (Model *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- *Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- *Releases resources owned by this data definition.*
- Util::identification **getId** () const
- *Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- *Updates the logical name of this instance.*
- const std::string & **getName** () const
- *Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
- *Updates the user-visible label of this instance.*

- `const std::string & getLabel () const`
Returns the user-visible label of this instance.
- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelspecific, bool traceLevelSpecific←
Enabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual bool [_check](#) (std::string &errorMessage)
virtual protected methods that could be overridden by derived classes, if needed
- virtual void [_initBetweenReplications](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.83.1 Detailed Description

This component ...

10.83.2 Member Function Documentation

10.83.2.1 `_check()`

```
bool CellularAutomataComp::_check (
    std::string * errorMessage ) [protected], [virtual]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by [ModelChecker](#) during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model to run without errors in this component

10.83.2.2 `_initBetweenReplications()`

```
void CellularAutomataComp::_initBetweenReplications ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this [ModelData](#)s. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to simulator kernel to reflect the changes

This method is called by [ModelSimulation](#) when initiating the replication. The model should set all value for a new replication (Ex: setting back to 0 any internal counter, clearing lists, etc.

Reimplemented from [ModelDataDefinition](#).

10.83.2.3 _loadInstance()

```
bool CellularAutomataComp::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.83.2.4 _onDispatchEvent()

```
void CellularAutomataComp::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

This method is only for ModelComponents, not ModelDataElements.

Implements [ModelComponent](#).

10.83.2.5 _saveInstance()

```
void CellularAutomataComp::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.83.2.6 show()

```
std::string CellularAutomataComp::show ( ) [override], [virtual]
```

virtual public methods

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/ModalModel/CellularAutomataComp.h
- plugins/components/ModalModel/CellularAutomataComp.cpp

10.84 CellularAutomataDemoBuilder Class Reference

Static Public Member Functions

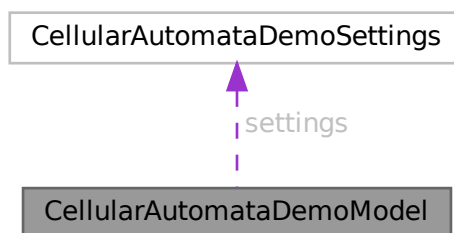
- static std::unique_ptr< [CellularAutomataDemoModel](#) > **buildDemo** ([CellularAutomataDemoSettings](#) settings={})

The documentation for this class was generated from the following files:

- applications/gui/genesys/cellular_automata/CellularAutomataDemoBuilder.h
- applications/gui/genesys/cellular_automata/CellularAutomataDemoBuilder.cpp

10.85 CellularAutomataDemoModel Struct Reference

Collaboration diagram for CellularAutomataDemoModel:



Public Member Functions

- **CellularAutomataDemoModel** ([CellularAutomataDemoSettings](#) settings={})
- int **columns** () const
- int **rows** () const
- bool **isValidPosition** (int x, int y) const
- [Cell](#) * **cellAt** (int x, int y) const
- long **stateAt** (int x, int y) const
- QString **stateTextAt** (int x, int y) const
- bool **setStateAt** (int x, int y, long value)
- void **fill** (long value)
- void **seedDefaultPattern** ()
- std::vector< long > **availablePaintStates** () const

Public Member Functions

- **CellularAutomataEvent** ([CellularAutomata](#) *ca)

Public Attributes

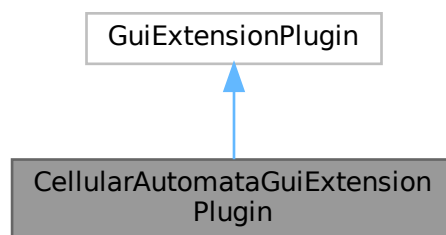
- [CellularAutomata](#) * **ca**

The documentation for this class was generated from the following file:

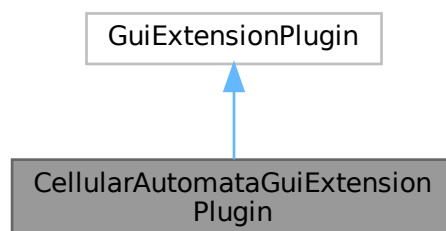
- plugins/components/ModalModel/CellularAutomata/CellularAutomata.h

10.88 CellularAutomataGuiExtensionPlugin Class Reference

Inheritance diagram for CellularAutomataGuiExtensionPlugin:



Collaboration diagram for CellularAutomataGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

10.88.1 Member Function Documentation

10.88.1.1 displayName()

```
std::string CellularAutomataGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.88.1.2 extensionId()

```
std::string CellularAutomataGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.88.1.3 registerContributions()

```
void CellularAutomataGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.88.1.4 requiredModelPlugins()

```
std::vector< std::string > CellularAutomataGuiExtensionPlugin::requiredModelPlugins ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.88.1.5 version()

```
std::string CellularAutomataGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

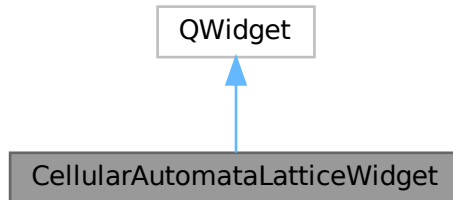
Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

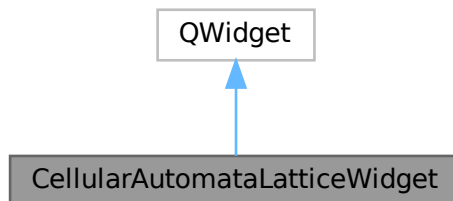
- `applications/gui/genesys/extensions/CellularAutomataGuiExtensionPlugin.cpp`

10.89 CellularAutomataLatticeWidget Class Reference

Inheritance diagram for CellularAutomataLatticeWidget:



Collaboration diagram for CellularAutomataLatticeWidget:



Public Member Functions

- **CellularAutomataLatticeWidget** (QWidget *parent=nullptr)
- void **setController** ([CellularAutomataViewerController](#) *controller)

Protected Member Functions

- void **paintEvent** (QPaintEvent *event) override
- void **mousePressEvent** (QMouseEvent *event) override
- void **mouseMoveEvent** (QMouseEvent *event) override
- void **mouseReleaseEvent** (QMouseEvent *event) override
- void **contextMenuEvent** (QContextMenuEvent *event) override
- QSize **minimumSizeHint** () const override

The documentation for this class was generated from the following files:

- applications/gui/genesys/cellular_automata/CellularAutomataLatticeWidget.h
- applications/gui/genesys/cellular_automata/CellularAutomataLatticeWidget.cpp

10.90 CellularAutomataStateColorMap Class Reference

Static Public Member Functions

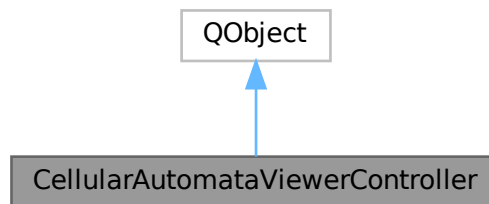
- static QColor **colorForState** (long state)
- static QColor **colorForStateText** (const QString &stateText, long fallbackState)
- static QColor **backgroundColor** ()
- static QColor **gridLineColor** ()
- static QColor **emptyTextColor** ()

The documentation for this class was generated from the following file:

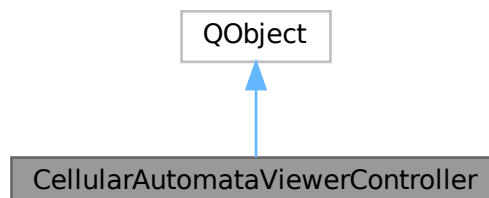
- applications/gui/genesys/cellular_automata/CellularAutomataStateColorMap.h

10.91 CellularAutomataViewerController Class Reference

Inheritance diagram for CellularAutomataViewerController:



Collaboration diagram for CellularAutomataViewerController:



Signals

- void **modelChanged** ()
- void **settingsChanged** ()
- void **timeChanged** (unsigned int time)
- void **runningChanged** (bool running)
- void **paintStateChanged** ()
- void **statusMessage** (const QString &message)

Public Member Functions

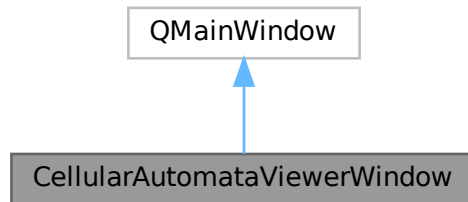
- **CellularAutomataViewerController** (QObject *parent=nullptr)
- [CellularAutomataDemoModel](#) * **model** () const
- [CellularAutomataDemoSettings](#) **settings** () const
- QSize **latticeSize** () const
- bool **hasDemo** () const
- unsigned int **simulatedTime** () const
- bool **isRunning** () const
- QString **configurationSummary** () const
- std::vector< long > **availablePaintStates** () const
- long **cellState** (int x, int y) const
- bool **setCellState** (int x, int y, long value)
- void **fill** (long value)
- bool **hasSelectedPaintState** () const
- std::optional< long > **selectedPaintState** () const
- QString **selectedPaintStateText** () const
- void **setSelectedPaintState** (long state)
- void **clearSelectedPaintState** ()
- int **autoStepIntervalMs** () const
- void **setAutoStepIntervalMs** (int intervalMs)
- void **rebuildDemo** ([CellularAutomataDemoSettings](#) settings={})
- void **setSettings** ([CellularAutomataDemoSettings](#) settings)
- void **setRulePreset** (CellularAutomataRulePreset preset)
- void **setNeighborhoodPreset** (CellularAutomataNeighborhoodPreset preset)
- void **setBoundaryPreset** (CellularAutomataBoundaryPreset preset)
- void **setStatePreset** (CellularAutomataStatePreset preset)
- void **resetToInitialState** ()
- void **step** ()
- void **start** ()
- void **pause** ()
- void **stop** ()
- bool **saveConfiguration** (const QString &filePath, QString *errorMessage=nullptr) const
- bool **loadConfiguration** (const QString &filePath, QString *errorMessage=nullptr)

The documentation for this class was generated from the following files:

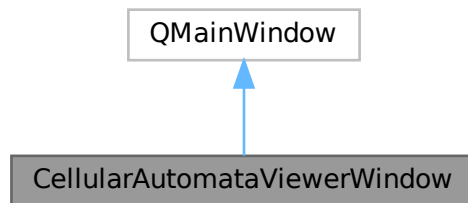
- applications/gui/genesys/cellular_automata/CellularAutomataViewerController.h
- applications/gui/genesys/cellular_automata/CellularAutomataViewerController.cpp

10.92 CellularAutomataViewerWindow Class Reference

Inheritance diagram for CellularAutomataViewerWindow:



Collaboration diagram for CellularAutomataViewerWindow:



Public Member Functions

- **CellularAutomataViewerWindow** (QWidget *parent=nullptr)

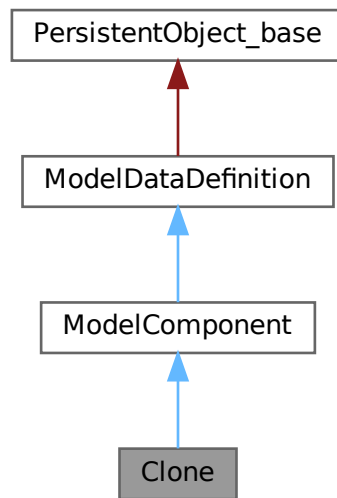
The documentation for this class was generated from the following files:

- applications/gui/genesys/cellular_automata/CellularAutomataViewerWindow.h
- applications/gui/genesys/cellular_automata/CellularAutomataViewerWindow.cpp

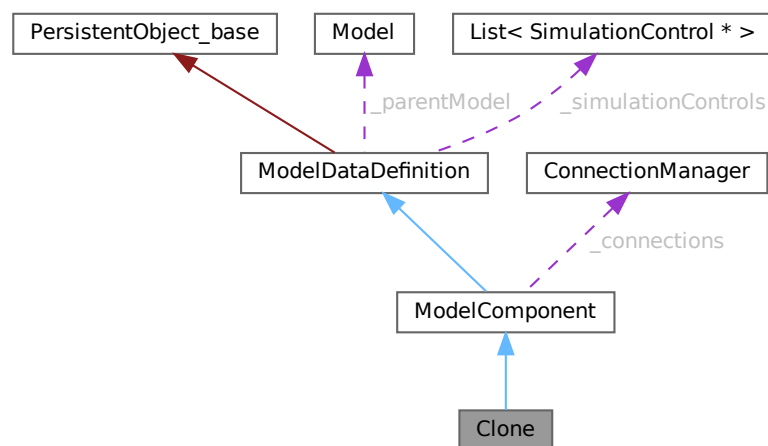
10.93 Clone Class Reference

```
#include <plugins/components/DiscreteProcessing/Clone.h>
```

Inheritance diagram for Clone:



Collaboration diagram for Clone:



Public Member Functions

- **Clone** (**Model** *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- void **setNumClonesExpression** (std::string numClones)
- std::string **getNumClonesExpression** () const

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.

- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelSpecific`, bool `traceLevelSpecificEnabled`=true)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation` ()
- static `ModelComponent * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
- static `ModelDataDefinition * NewInstance` (`Model *model`, `std::string name=""`)

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent *component`)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent *component`)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord *fields`, `ModelComponent *component`)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event *event`)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createInternalStatisticReporters](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.93.1 Detailed Description

This component ...

10.93.2 Member Function Documentation

10.93.2.1 `_check()`

```
bool Clone::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.93.2.2 `_createInternalStatisticReporters()`

```
void Clone::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.93.2.3 `_initBetweenReplications()`

```
void Clone::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.93.2.4 `_loadInstance()`

```
bool Clone::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.93.2.5 `_onDispatchEvent()`

```
void Clone::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.93.2.6 `_saveInstance()`

```
void Clone::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.93.2.7 `show()`

```
std::string Clone::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

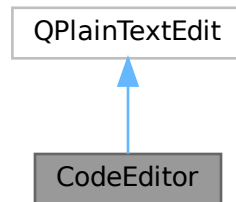
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

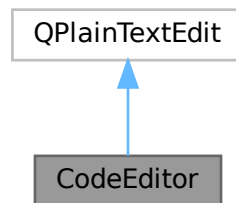
- plugins/components/DiscreteProcessing/Clone.h
- plugins/components/DiscreteProcessing/Clone.cpp

10.94 CodeEditor Class Reference

Inheritance diagram for CodeEditor:



Collaboration diagram for CodeEditor:



Public Member Functions

- **CodeEditor** (QWidget *parent=nullptr)
- void **lineNumberAreaPaintEvent** (QPaintEvent *event)
- int **lineNumberAreaWidth** ()

Protected Member Functions

- void **resizeEvent** (QResizeEvent *event) override

The documentation for this class was generated from the following files:

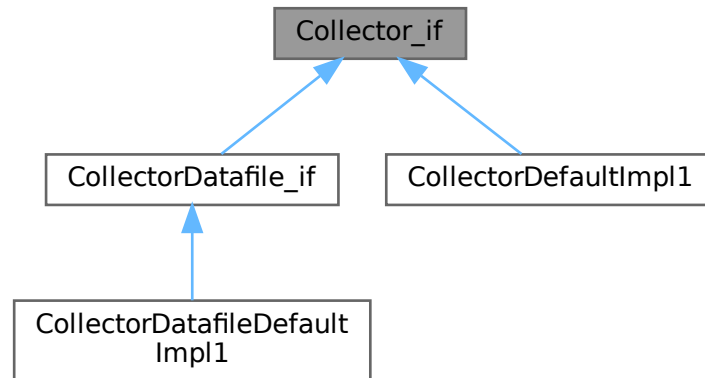
- applications/gui/genesys/codeeditor/CodeEditor.h
- applications/gui/genesys/codeeditor/CodeEditor.cpp

10.95 Collector_if Class Reference

Interface responsible for collecting observations of a stochastic variable.

```
#include <kernel/statistics/Collector_if.h>
```

Inheritance diagram for Collector_if:



Public Member Functions

- virtual void `clear` ()=0
clear
- virtual void `addValue` (double value, double weight=1)=0
addValue
- virtual double `getLastValue` ()=0
getLastValue
- virtual unsigned long `numElements` ()=0
numElements
- virtual void `setAddValueHandler` (CollectorAddValueHandler addValueHandler)=0
setAddValueHandler
- virtual void `setClearHandler` (CollectorClearHandler clearHandler)=0
setClearHandler

10.95.1 Detailed Description

Interface responsible for collecting observations of a stochastic variable.

A collector stores values generated during simulation and optionally supports callbacks for integration with online statistics, tracing or UI updates.

10.95.2 Member Function Documentation

10.95.2.1 addValue()

```
virtual void Collector_if::addValue (
    double value,
    double weight = 1 ) [pure virtual]
```

addValue

Parameters

<i>value</i>	
<i>weight</i>	

Adds a new observed value (with optional weight).

Implemented in [CollectorDatafileDefaultImpl1](#), and [CollectorDefaultImpl1](#).

10.95.2.2 clear()

```
virtual void Collector_if::clear ( ) [pure virtual]
```

clear

Clears all collected values and resets internal state.

Implemented in [CollectorDatafileDefaultImpl1](#), and [CollectorDefaultImpl1](#).

10.95.2.3 getLastValue()

```
virtual double Collector_if::getLastValue ( ) [pure virtual]
```

getLastValue

Returns

Returns the last collected value.

Implemented in [CollectorDatafileDefaultImpl1](#), and [CollectorDefaultImpl1](#).

10.95.2.4 numElements()

```
virtual unsigned long Collector_if::numElements ( ) [pure virtual]
```

numElements

Returns

Returns the number of stored observations.

Implemented in [CollectorDatafileDefaultImpl1](#), and [CollectorDefaultImpl1](#).

10.95.2.5 setAddValueHandler()

```
virtual void Collector_if::setAddValueHandler (
    CollectorAddValueHandler addValueHandler ) [pure virtual]
```

setAddValueHandler

Parameters

<i>addValueHandler</i>	
------------------------	--

Registers a callback triggered after each added value.

Implemented in [CollectorDatafileDefaultImpl1](#), and [CollectorDefaultImpl1](#).

10.95.2.6 setClearHandler()

```
virtual void Collector_if::setClearHandler (
    CollectorClearHandler clearHandler ) [pure virtual]
```

setClearHandler

Parameters

<i>clearHandler</i>	
---------------------	--

Registers a callback triggered when the collector is cleared.

Implemented in [CollectorDatafileDefaultImpl1](#), and [CollectorDefaultImpl1](#).

The documentation for this class was generated from the following file:

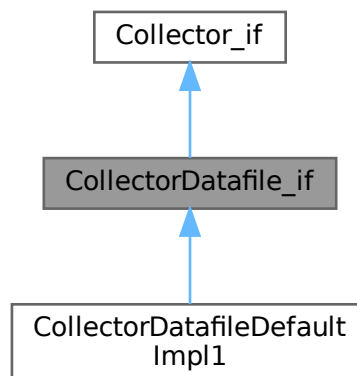
- kernel/statistics/Collector_if.h

10.96 CollectorDatafile_if Class Reference

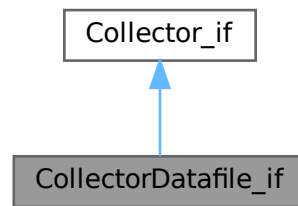
Collector interface specialized in file-backed value storage.

```
#include <kernel/statistics/CollectorDatafile_if.h>
```

Inheritance diagram for CollectorDatafile_if:



Collaboration diagram for CollectorDatafile_if:



Public Member Functions

- virtual double `getValue` (unsigned int rank)=0
getValue
- virtual void `seekFirstValue` ()=0
seekFirstValue
- virtual double `getNextValue` ()=0
getNextValue
- virtual std::string `getDataFilename` ()=0
getDataFilename
- virtual void `setDataFilename` (std::string filename)=0
setDataFilename

Public Member Functions inherited from `Collector_if`

- virtual void `clear` ()=0
clear
- virtual void `addValue` (double value, double weight=1)=0
addValue
- virtual double `getLastValue` ()=0
getLastValue
- virtual unsigned long `numElements` ()=0
numElements
- virtual void `setAddValueHandler` (CollectorAddValueHandler addValueHandler)=0
setAddValueHandler
- virtual void `setClearHandler` (CollectorClearHandler clearHandler)=0
setClearHandler

10.96.1 Detailed Description

Collector interface specialized in file-backed value storage.

Besides standard collection operations, implementations provide sequential/random access to persisted samples and filename management for the underlying dataset.

10.96.2 Member Function Documentation

10.96.2.1 getDataFilename()

```
virtual std::string CollectorDatafile_if::getDataFilename ( ) [pure virtual]
```

getDataFilename

Returns

Returns the filename used to persist collected values.

Implemented in [CollectorDatafileDefaultImpl1](#).

10.96.2.2 getNextValue()

```
virtual double CollectorDatafile_if::getNextValue ( ) [pure virtual]
```

getNextValue

Returns

Get the next value in the file and advances the pointer

Implemented in [CollectorDatafileDefaultImpl1](#).

10.96.2.3 getValue()

```
virtual double CollectorDatafile_if::getValue (
    unsigned int rank ) [pure virtual]
```

getValue

Parameters

<i>rank</i>	
-------------	--

Returns

Get a value from a specific position

Implemented in [CollectorDatafileDefaultImpl1](#).

10.96.2.4 seekFirstValue()

```
virtual void CollectorDatafile_if::seekFirstValue ( ) [pure virtual]
```

seekFirstValue

[Set](#) the pointer to the first value in the file

Implemented in [CollectorDatafileDefaultImpl1](#).

10.96.2.5 setDataFilename()

```
virtual void CollectorDatafile_if::setDataFilename (
    std::string filename ) [pure virtual]
```

setDataFilename

Parameters

<i>filename</i>	
-----------------	--

Sets/updates the filename used to store collected data.

Implemented in [CollectorDatafileDefaultImpl1](#).

The documentation for this class was generated from the following file:

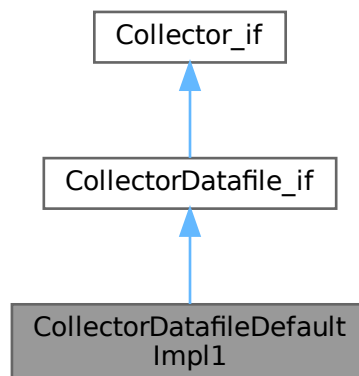
- kernel/statistics/CollectorDatafile_if.h

10.97 CollectorDatafileDefaultImpl1 Class Reference

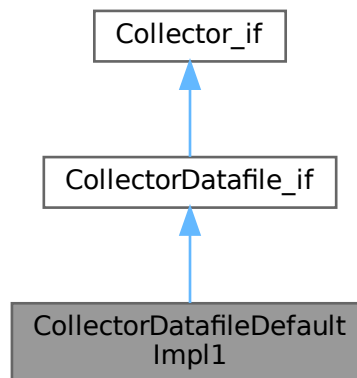
Default file-backed implementation of [CollectorDatafile_if](#).

```
#include <kernel/statistics/CollectorDatafileDefaultImpl1.h>
```

Inheritance diagram for CollectorDatafileDefaultImpl1:



Collaboration diagram for CollectorDatafileDefaultImpl1:



Public Member Functions

- **CollectorDatafileDefaultImpl1 ()**
Creates an empty file-backed collector instance.
- virtual void **clear ()** override
Clears collector state and resets file cursor/content as implemented.
- virtual void **addValue** (double value, double weight=1) override
Appends a value (and optional weight) to the underlying storage.
- virtual double **getLastValue** () override
Returns the most recently collected value.
- virtual unsigned long **numElements** () override
Returns the number of collected elements.
- virtual double **getValue** (unsigned int num) override
Returns the value stored at ordinal position num.
- virtual double **getNextValue** () override
Returns the next value from current file cursor and advances cursor.
- virtual void **seekFirstValue** () override
Repositions internal cursor to the first stored value.
- virtual std::string **getDataFilename** () override
Returns the filename currently used as data backing store.
- virtual void **setDataFilename** (std::string filename) override
Sets the filename used as data backing store.
- virtual void **setAddValueHandler** (CollectorAddValueHandler addValueHandler) override
Registers callback invoked whenever a value is added.
- virtual void **setClearHandler** (CollectorClearHandler clearHandler) override
Registers callback invoked whenever the collector is cleared.

10.97.1 Detailed Description

Default file-backed implementation of [CollectorDatafile_if](#).

Stores collected values in an associated data file and keeps lightweight counters/cache for last value and number of elements.

10.97.2 Member Function Documentation

10.97.2.1 addValue()

```
void CollectorDatafileDefaultImpl1::addValue (
    double value,
    double weight = 1 ) [override], [virtual]
```

Appends a value (and optional weight) to the underlying storage.

Implements [Collector_if](#).

10.97.2.2 clear()

```
void CollectorDatafileDefaultImpl1::clear ( ) [override], [virtual]
```

Clears collector state and resets file cursor/content as implemented.

Implements [Collector_if](#).

10.97.2.3 getDataFilename()

```
std::string CollectorDatafileDefaultImpl1::getDataFilename ( ) [override], [virtual]
```

Returns the filename currently used as data backing store.

Implements [CollectorDatafile_if](#).

10.97.2.4 getLastValue()

```
double CollectorDatafileDefaultImpl1::getLastValue ( ) [override], [virtual]
```

Returns the most recently collected value.

Implements [Collector_if](#).

10.97.2.5 getNextValue()

```
double CollectorDatafileDefaultImpl1::getNextValue ( ) [override], [virtual]
```

Returns the next value from current file cursor and advances cursor.

Implements [CollectorDatafile_if](#).

10.97.2.6 getValue()

```
double CollectorDatafileDefaultImpl1::getValue (
    unsigned int num ) [override], [virtual]
```

Returns the value stored at ordinal position num.

Implements [CollectorDatafile_if](#).

10.97.2.7 numElements()

```
unsigned long CollectorDatafileDefaultImpl1::numElements ( ) [override], [virtual]
```

Returns the number of collected elements.

Implements [Collector_if](#).

10.97.2.8 seekFirstValue()

```
void CollectorDatafileDefaultImpl1::seekFirstValue ( ) [override], [virtual]
```

Repositions internal cursor to the first stored value.

Implements [CollectorDatafile_if](#).

10.97.2.9 setAddValueHandler()

```
void CollectorDatafileDefaultImpl1::setAddValueHandler (
    CollectorAddValueHandler addValueHandler ) [override], [virtual]
```

Registers callback invoked whenever a value is added.

Implements [Collector_if](#).

10.97.2.10 setClearHandler()

```
void CollectorDatafileDefaultImpl1::setClearHandler (
    CollectorClearHandler clearHandler ) [override], [virtual]
```

Registers callback invoked whenever the collector is cleared.

Implements [Collector_if](#).

10.97.2.11 setDataFilename()

```
void CollectorDatafileDefaultImpl1::setDataFilename (
    std::string filename ) [override], [virtual]
```

Sets the filename used as data backing store.

Implements [CollectorDatafile_if](#).

The documentation for this class was generated from the following files:

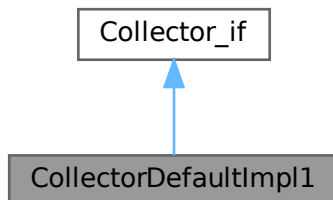
- kernel/statistics/CollectorDatafileDefaultImpl1.h
- kernel/statistics/CollectorDatafileDefaultImpl1.cpp

10.98 CollectorDefaultImpl1 Class Reference

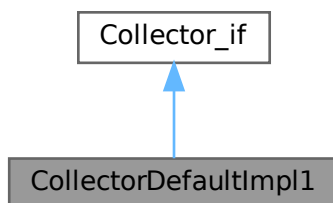
Default in-memory implementation of [Collector_if](#).

```
#include <kernel/statistics/CollectorDefaultImpl1.h>
```

Inheritance diagram for CollectorDefaultImpl1:



Collaboration diagram for CollectorDefaultImpl1:



Public Member Functions

- **CollectorDefaultImpl1 ()**
Creates an empty collector.
- virtual void **clear ()** override
Clears all collected data and resets counters.
- virtual void **addValue** (double value, double weight=1) override
Adds a new observation to the collector.
- virtual double **getLastValue ()** override
Returns the latest observed value.
- virtual unsigned long **numElements ()** override
Returns total number of observed values.
- virtual void **setAddValueHandler** (CollectorAddValueHandler addValueHandler) override
Registers callback notified after each `addValue` call.
- virtual void **setClearHandler** (CollectorClearHandler clearHandler) override
Registers callback notified when `clear` is invoked.

10.98.1 Detailed Description

Default in-memory implementation of [Collector_if](#).

Maintains minimal aggregate state (last value and count) and supports optional callbacks for add/clear events.

10.98.2 Member Function Documentation

10.98.2.1 addValue()

```
void CollectorDefaultImpl1::addValue (
    double value,
    double weight = 1 ) [override], [virtual]
```

Adds a new observation to the collector.

Implements [Collector_if](#).

10.98.2.2 clear()

```
void CollectorDefaultImpl1::clear ( ) [override], [virtual]
```

Clears all collected data and resets counters.

Implements [Collector_if](#).

10.98.2.3 getLastValue()

```
double CollectorDefaultImpl1::getLastValue ( ) [override], [virtual]
```

Returns the latest observed value.

Implements [Collector_if](#).

10.98.2.4 numElements()

```
unsigned long CollectorDefaultImpl1::numElements ( ) [override], [virtual]
```

Returns total number of observed values.

Implements [Collector_if](#).

10.98.2.5 setAddValueHandler()

```
void CollectorDefaultImpl1::setAddValueHandler (
    CollectorAddValueHandler addValueHandler ) [override], [virtual]
```

Registers callback notified after each addValue call.

Implements [Collector_if](#).

10.98.2.6 setClearHandler()

```
void CollectorDefaultImpl1::setClearHandler (
    CollectorClearHandler clearHandler ) [override], [virtual]
```

Registers callback notified when `clear` is invoked.

Implements [Collector_if](#).

The documentation for this class was generated from the following files:

- kernel/statistics/CollectorDefaultImpl1.h
- kernel/statistics/CollectorDefaultImpl1.cpp

10.99 GroProgramRuntime::ColonyMutation Struct Reference

Public Types

- enum class **Type** {
Reset , **SpawnSeed** , **SetChemostatMode** , **AddBarrier** ,
MapToCells , **SetSignalAt** , **SetSignalRect** , **SetSignalGridWidth** ,
SetSignalGridHeight , **GetSignalMatrix** , **DumpSignalField** }

Public Attributes

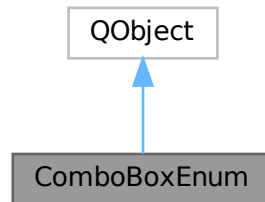
- Type **type** = Type::Reset
- std::vector< std::string > **arguments**
- std::vector< double > **numericArguments**
- std::string **expressionText**
- std::size_t **previewRows** = 0
- std::size_t **previewColumns** = 0

The documentation for this struct was generated from the following file:

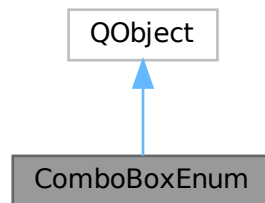
- plugins/data/BiochemicalSimulation/GroProgramRuntime.h

10.100 ComboBoxEnum Class Reference

Inheritance diagram for ComboBoxEnum:



Collaboration diagram for ComboBoxEnum:



Public Types

- using **AfterChange** = `std::function< void()>`

Public Member Functions

- **ComboBoxEnum** ([PropertyEditorGenesys](#) *editor, [SimulationControl](#) *property, AfterChange afterChange={})
- void **open_box** ()
- void **configure_options** ([SimulationControl](#) *property)

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/ComboBoxEnum.h
- applications/gui/genesys/propertyeditor/ComboBoxEnum.cpp

10.101 GroProgramlr::Command Struct Reference

Public Types

- enum class **Kind** { **RawStatement** , **FunctionCall** , **Assignment** , **IfStatement** }

Public Member Functions

- bool **isFunctionCall** () const
- bool **isAssignment** () const
- bool **isIfStatement** () const

Public Attributes

- Kind **kind** = Kind::RawStatement
- std::string **sourceText** = ""
- std::string **functionName** = ""
- std::vector< std::string > **arguments**
- std::string **assignmentTarget** = ""
- std::string **expressionText** = ""
- bool **assignmentOnlyIfUnset** = false
- std::vector< [Command](#) > **thenCommands**
- std::vector< [Command](#) > **elseCommands**

The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/GroProgramlr.h

10.102 Util::CommandResult Class Reference

Public Attributes

- bool **success** = false
- std::string **commandStdOutput** = ""
- std::string **commandErrOutput** = ""
- std::string **workingDirectory** = "./"
- std::string **destinationPath** = "./"

The documentation for this class was generated from the following file:

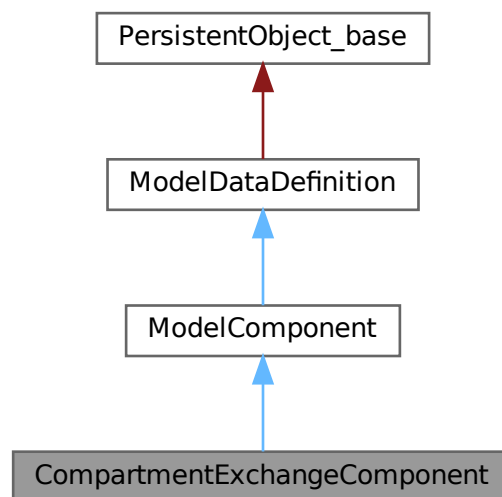
- kernel/util/Util.h

10.103 CompartmentExchangeComponent Class Reference

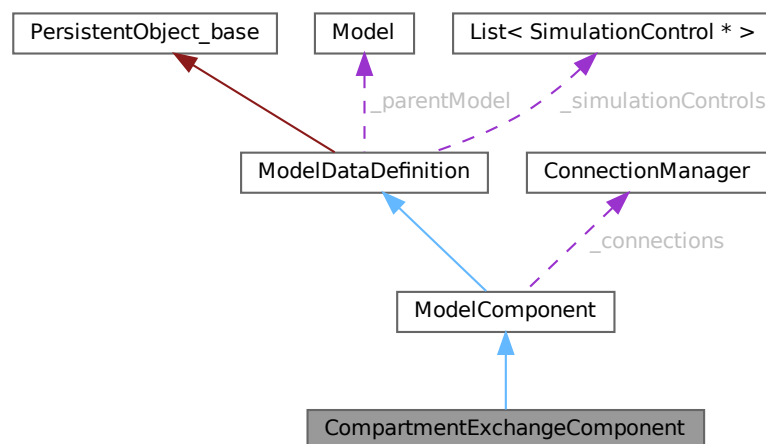
Moves metabolite proxies between whole-cell regions.

```
#include <plugins/components/WholeCellModeling/CompartmentExchangeComponent.h>
```

Inheritance diagram for CompartmentExchangeComponent:



Collaboration diagram for CompartmentExchangeComponent:



Classes

- struct [ExchangeRule](#)

Public Member Functions

- **CompartmentExchangeComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setSourceRegion** (std::string region)
- std::string **getSourceRegion** () const
- void **setSourceMetaboliteKey** (std::string key)
- std::string **getSourceMetaboliteKey** () const
- void **setTargetRegion** (std::string region)
- std::string **getTargetRegion** () const
- void **setTargetMetaboliteKey** (std::string key)
- std::string **getTargetMetaboliteKey** () const
- void **setExchangeFraction** (double fraction)
- double **getExchangeFraction** () const
- void **setBaselineAmount** (double amount)
- double **getBaselineAmount** () const
- void **setDriverPathwayKey** (std::string key)
- std::string **getDriverPathwayKey** () const
- void **setDriverScale** (double scale)
- double **getDriverScale** () const
- void **setMaxTransferAmount** (double amount)
- double **getMaxTransferAmount** () const
- void **setConserveMass** (bool conserve)
- bool **getConserveMass** () const
- void **clearExchangeRules** ()
- void **addExchangeRule** (const std::string &label, const std::string &sourceRegion, const std::string &sourceMetaboliteKey, const std::string &targetRegion, const std::string &targetMetaboliteKey, double exchangeFraction, double baselineAmount=0.0, const std::string &driverPathwayKey="", double driverScale=1.0, double maxTransferAmount=0.0, bool conserveMass=true)
- unsigned int **getExchangeRuleCount** () const
- const std::vector< [ExchangeRule](#) > & **getExchangeRules** () const
- double **getLastTransferAmount** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.

- void `setDescription` (std::string _description)
Updates the free-form description shown by the GUI.
- std::string `getDescription` () const
Returns the free-form description associated with this component.
- void `remove` (`ModelComponent` *comp)
Removes a connected component from this component.

Public Member Functions inherited from `ModelDataDefinition`

- `ModelDataDefinition` (`Model` *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~`ModelDataDefinition` ()
Releases resources owned by this data definition.
- Util::identification `getId` () const
Returns the unique identifier of this instance.
- void `setName` (const std::string &name)
Updates the logical name of this instance.
- const std::string & `getName` () const
Returns the logical name of this instance.
- void `setLabel` (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & `getLabel` () const
Returns the user-visible label of this instance.
- const std::string & `getClassname` () const
Returns the runtime class name used by the plugin system.
- `Model` * `getParentModel` () const
Returns the parent model that owns this definition.
- bool `isReportStatistics` () const
Indicates whether this data definition contributes report statistics.
- void `setReportStatistics` (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition` * `getInternalData` (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, `ModelDataDefinition` * > * `getInternalData` () const
Returns the full map of internal child data definitions.
- std::map< std::string, `ModelDataDefinition` * > * `getAttachedData` () const
Returns the full map of attached data definitions.
- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< `SimulationControl` * > * `getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.

- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_initBetweenReplications` () override
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()

- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.103.1 Detailed Description

Moves metabolite proxies between whole-cell regions.

[CompartmentExchangeComponent](#) transfers a fraction of a source metabolite pool in [WholeCellState](#) into a target pool on each dispatched event. The component can represent didactic transport, exchange, or redistribution policies between global and compartment-scoped pools without embedding that logic directly into examples.

Source and target regions follow this convention:

- empty region name: global [WholeCellState](#) metabolite pool;
- non-empty region name: compartment-scoped pool.

10.103.2 Member Function Documentation

10.103.2.1 `_check()`

```
bool CompartmentExchangeComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.103.2.2 `_createEditableDataDefinitions()`

```
void CompartmentExchangeComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.103.2.3 `_initBetweenReplications()`

```
void CompartmentExchangeComponent::_initBetweenReplications ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.103.2.4 `_loadInstance()`

```
bool CompartmentExchangeComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.103.2.5 `_onDispatchEvent()`

```
void CompartmentExchangeComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.103.2.6 _saveInstance()

```
void CompartmentExchangeComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.103.2.7 show()

```
std::string CompartmentExchangeComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/CompartmentExchangeComponent.h
- plugins/components/WholeCellModeling/CompartmentExchangeComponent.cpp

10.104 CppCompiler::CompilationResult Class Reference

Public Attributes

- bool **success** = false
- std::string **compilationStdOutput** = ""
- std::string **compilationErrOutput** = ""
- std::string **destinationPath** = ""
- unsigned int **lineNumber** = 0

The documentation for this class was generated from the following file:

- plugins/data/ExternalIntegration/CppCompiler.h

10.105 CompilerThread Class Reference

Public Member Functions

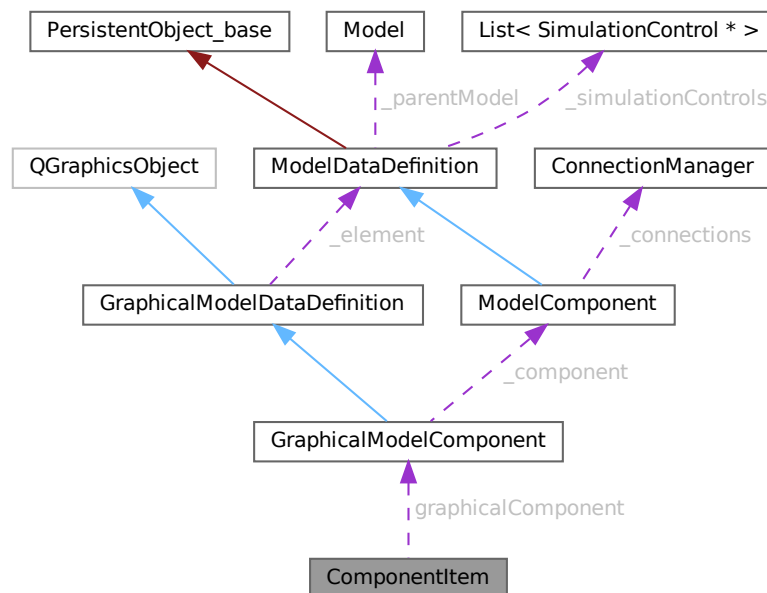
- void **operator()** ()

The documentation for this class was generated from the following file:

- plugins/data/ExternalIntegration/CppCompiler.h

10.106 ComponentItem Struct Reference

Collaboration diagram for ComponentItem:



Public Attributes

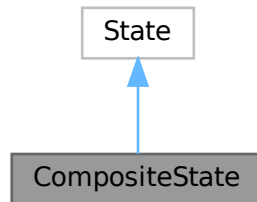
- [GraphicalModelComponent](#) * **graphicalComponent**
- QPointF **initialPosition**
- QList< [GraphicalConnection](#) * > **inputConnections**
- QList< [GraphicalConnection](#) * > **outputConnections**

The documentation for this struct was generated from the following file:

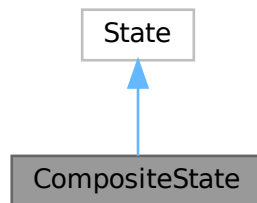
- applications/gui/genesys/actions/AddUndoCommand.h

10.107 CompositeState Class Reference

Inheritance diagram for CompositeState:



Collaboration diagram for CompositeState:



Public Types

- using **Field** = std::pair< std::string, std::unique_ptr< [State](#) > >

Public Member Functions

- **CompositeState** (const [CompositeState](#) &orig)
- [CompositeState](#) & **operator=** (const [CompositeState](#) &orig)
- std::unique_ptr< [State](#) > **clone** () const override
- bool **equals** (const [State](#) &other) const override
- std::string **toString** () const override
- std::string **typeName** () const override
- bool **addField** (std::string name, const [State](#) &state)
- const std::vector< Field > & **fields** () const

Public Member Functions inherited from [State](#)

- **State** (long value)
- **State** (const [State](#) &orig)
- virtual std::string **show** () const
- virtual long **getValue** () const
- virtual void **setValue** (long value)
- template<typename T >
const T * **as** () const noexcept
- template<typename T >
T * **as** () noexcept

Additional Inherited Members

Protected Attributes inherited from [State](#)

- long **_value**

10.107.1 Member Function Documentation

10.107.1.1 clone()

```
std::unique_ptr< State > CompositeState::clone ( ) const [inline], [override], [virtual]
```

Reimplemented from [State](#).

10.107.1.2 equals()

```
bool CompositeState::equals (
    const State & other ) const [inline], [override], [virtual]
```

Reimplemented from [State](#).

10.107.1.3 toString()

```
std::string CompositeState::toString ( ) const [inline], [override], [virtual]
```

Reimplemented from [State](#).

10.107.1.4 typeName()

```
std::string CompositeState::typeName ( ) const [inline], [override], [virtual]
```

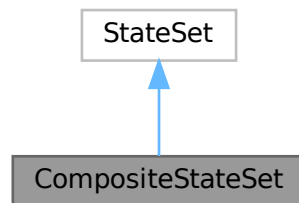
Reimplemented from [State](#).

The documentation for this class was generated from the following file:

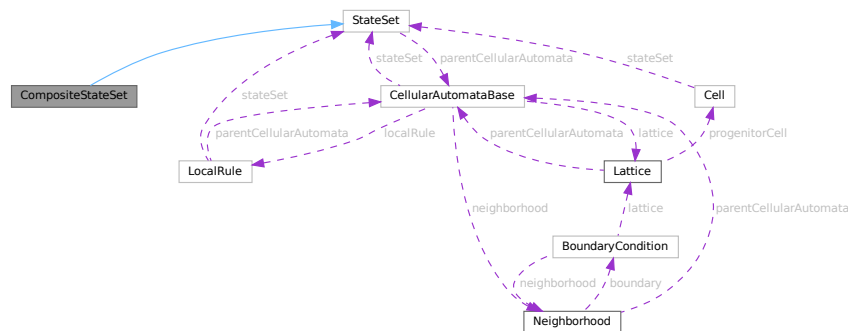
- plugins/components/ModalModel/CellularAutomata/State.h

10.108 CompositeStateSet Class Reference

Inheritance diagram for CompositeStateSet:



Collaboration diagram for CompositeStateSet:



Public Member Functions

- **CompositeStateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- bool [contains](#) (const [State](#) &state) const override
- std::unique_ptr< [State](#) > [createDefaultState](#) () const override
- bool [isComposite](#) () const override
- std::string [name](#) () const override

Public Member Functions inherited from [StateSet](#)

- **StateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- **StateSet** (const [StateSet](#) &orig)
- virtual std::unique_ptr< [State](#) > [parseState](#) (const std::string &text) const
- virtual bool [isFinite](#) () const
- virtual bool [isDiscrete](#) () const
- virtual bool [isContinuous](#) () const
- virtual std::vector< std::unique_ptr< [State](#) > > [enumerateStates](#) () const

Additional Inherited Members

Protected Attributes inherited from [StateSet](#)

- [CellularAutomataBase](#) * parentCellularAutomata

10.108.1 Member Function Documentation

10.108.1.1 contains()

```
bool CompositeStateSet::contains (
    const State & state ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.108.1.2 createDefaultState()

```
std::unique_ptr< State > CompositeStateSet::createDefaultState ( ) const [inline], [override],
[virtual]
```

Reimplemented from [StateSet](#).

10.108.1.3 isComposite()

```
bool CompositeStateSet::isComposite ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.108.1.4 name()

```
std::string CompositeStateSet::name ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

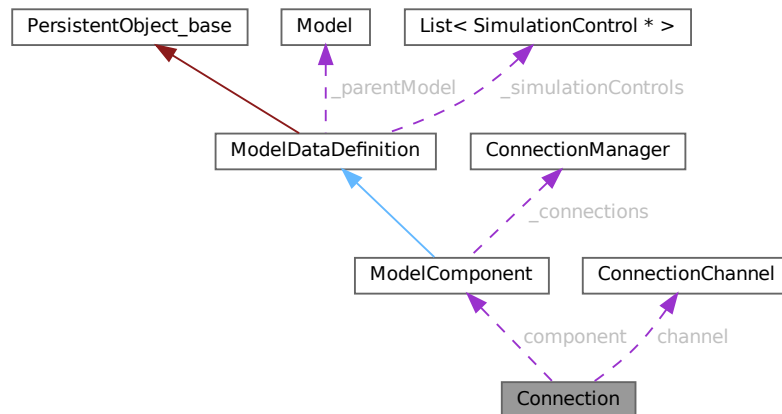
The documentation for this class was generated from the following file:

- plugins/components/ModalModel/CellularAutomata/StateSet.h

10.109 Connection Struct Reference

```
#include <kernel/simulator/ConnectionManager.h>
```

Collaboration diagram for Connection:



Public Attributes

- [ModelComponent](#) * **component**
- [ConnectionChannel](#) **channel**

10.109.1 Detailed Description

Defines a connection between two or more [ModelComponents](#). A (receiver) component's inputPort may receive connection from one or more (sender) components' outputPorts. A (sender) component's outputPort is connected to one and only one (receiver) component's inputPort.

The documentation for this struct was generated from the following file:

- kernel/simulator/ConnectionManager.h

10.110 ConnectionChannel Struct Reference

The [ConnectionChannel](#) class defines an input or output port that connects a [ModelComponent](#) to another.

```
#include <kernel/simulator/ConnectionManager.h>
```

Public Attributes

- unsigned int **portNumber** = 0
- std::string **portDescription** = ""

10.110.1 Detailed Description

The [ConnectionChannel](#) class defines an input or output port that connects a [ModelComponent](#) to another.

The documentation for this struct was generated from the following file:

- kernel/simulator/ConnectionManager.h

10.111 ConnectionManager Class Reference

Maintains outgoing connections from a component to downstream components.

```
#include <kernel/simulator/ConnectionManager.h>
```

Public Member Functions

- **ConnectionManager** ()
Creates an empty connection manager with zero constraints.
- unsigned int **size** ()
size
- [Connection](#) * **getFrontConnection** ()
getFrontConnection
- [Connection](#) * **getConnectionAtPort** (unsigned int rank)
getConnectionAtPort
- void **insert** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Inserts a new connection to a target component.
- void **insert** ([Connection](#) *connection)
insert
- void **insertAtPort** (unsigned int port, [Connection](#) *connection)
insertAtPort
- void **remove** ([Connection](#) *connection)
remove
- void **removeAtPort** (unsigned int port)
removeAtPort
- std::map< unsigned int, [Connection](#) * > * **connections** () const
Returns the internal output-port to connection map.
- unsigned int **getCurrentOutputConnectionsSize** () const
Returns the number of currently configured outgoing connections.
- void **setMaxOutputConnections** (unsigned int _maxOutputConnections)
Sets the maximum number of outgoing connections allowed.
- unsigned int **getMaxOutputConnections** () const
Returns the maximum number of outgoing connections allowed.
- void **setMinOutputConnections** (unsigned int _minOutputConnections)
Sets the minimum number of outgoing connections allowed.
- unsigned int **getMinOutputConnections** () const
Returns the minimum number of outgoing connections allowed.
- unsigned int **getCurrentInputConnectionsSize** () const
Returns the number of currently configured incoming connections.
- void **setMaxInputConnections** (unsigned int _maxInputConnections)

- Sets the maximum number of incoming connections allowed.*
- unsigned int `getMaxInputConnections` () const
Returns the maximum number of incoming connections allowed.
- void `setMinInputConnections` (unsigned int _minInputConnections)
Sets the minimum number of incoming connections allowed.
- unsigned int `getMinInputConnections` () const
Returns the minimum number of incoming connections allowed.

10.111.1 Detailed Description

Maintains outgoing connections from a component to downstream components.

A connection maps an output rank (port) to a destination component/input port. The manager also stores min/max input/output constraints used during validation.

10.111.2 Member Function Documentation

10.111.2.1 connections()

```
std::map< unsigned int, Connection * > * ConnectionManager::connections ( ) const
```

Returns the internal output-port to connection map.

Returns

Map of output ports to outgoing connections.

10.111.2.2 getConnectionAtPort()

```
Connection * ConnectionManager::getConnectionAtPort (
    unsigned int rank )
```

getConnectionAtPort

Parameters

<i>rank</i>	
-------------	--

Returns

`Connection` bound to output rank `rank`.

10.111.2.3 getCurrentInputConnectionsSize()

```
unsigned int ConnectionManager::getCurrentInputConnectionsSize ( ) const
```

Returns the number of currently configured incoming connections.

Returns

Current input connection count.

10.111.2.4 getCurrentOutputConnectionsSize()

```
unsigned int ConnectionManager::getCurrentOutputConnectionsSize ( ) const
```

Returns the number of currently configured outgoing connections.

Returns

Current output connection count.

10.111.2.5 getFrontConnection()

```
Connection * ConnectionManager::getFrontConnection ( )
```

getFrontConnection

Returns

[Connection](#) at the first available output rank.

10.111.2.6 getMaxInputConnections()

```
unsigned int ConnectionManager::getMaxInputConnections ( ) const
```

Returns the maximum number of incoming connections allowed.

Returns

Maximum incoming connection count.

10.111.2.7 getMaxOutputConnections()

```
unsigned int ConnectionManager::getMaxOutputConnections ( ) const
```

Returns the maximum number of outgoing connections allowed.

Returns

Maximum outgoing connection count.

10.111.2.8 getMinInputConnections()

```
unsigned int ConnectionManager::getMinInputConnections ( ) const
```

Returns the minimum number of incoming connections allowed.

Returns

Minimum incoming connection count.

10.111.2.9 getMinOutputConnections()

```
unsigned int ConnectionManager::getMinOutputConnections ( ) const
```

Returns the minimum number of outgoing connections allowed.

Returns

Minimum outgoing connection count.

10.111.2.10 insert() [1/2]

```
void ConnectionManager::insert (
    Connection * connection )
```

insert

Parameters

<i>connection</i>	
-------------------	--

Inserts a prebuilt connection object.

10.111.2.11 insert() [2/2]

```
void ConnectionManager::insert (
    ModelComponent * component,
    unsigned int inputPortNumber = 0 )
```

Inserts a new connection to a target component.

Creates and inserts a connection to *component* at input port *inputPortNumber*. When the port is not specified, the connection uses the default input port 0.

Parameters

<i>component</i>	Target component.
<i>inputPortNumber</i>	Target input port number.

10.111.2.12 insertAtPort()

```
void ConnectionManager::insertAtPort (
    unsigned int port,
    Connection * connection )
```

insertAtPort

Parameters

<i>port</i>	
<i>connection</i>	

Forces insertion/replacement at a specific output port.

10.111.2.13 remove()

```
void ConnectionManager::remove (
    Connection * connection )
```

remove

Parameters

<i>connection</i>	
-------------------	--

Removes a connection by pointer match.

10.111.2.14 removeAtPort()

```
void ConnectionManager::removeAtPort (
    unsigned int port )
```

removeAtPort

Parameters

<i>port</i>	
-------------	--

Removes the connection associated with output port *port*.

10.111.2.15 setMaxInputConnections()

```
void ConnectionManager::setMaxInputConnections (
    unsigned int _maxInputConnections )
```

Sets the maximum number of incoming connections allowed.

Parameters

<code>_maxInputConnections</code>	Maximum incoming connection count.
-----------------------------------	------------------------------------

10.111.2.16 setMaxOutputConnections()

```
void ConnectionManager::setMaxOutputConnections (
    unsigned int _maxOutputConnections )
```

Sets the maximum number of outgoing connections allowed.

Parameters

<code>_maxOutputConnections</code>	Maximum outgoing connection count.
------------------------------------	------------------------------------

10.111.2.17 setMinInputConnections()

```
void ConnectionManager::setMinInputConnections (
    unsigned int _minInputConnections )
```

Sets the minimum number of incoming connections allowed.

Parameters

<code>_minInputConnections</code>	Minimum incoming connection count.
-----------------------------------	------------------------------------

10.111.2.18 setMinOutputConnections()

```
void ConnectionManager::setMinOutputConnections (
    unsigned int _minOutputConnections )
```

Sets the minimum number of outgoing connections allowed.

Parameters

<code>_minOutputConnections</code>	Minimum outgoing connection count.
------------------------------------	------------------------------------

10.111.2.19 size()

```
unsigned int ConnectionManager::size ( )
```

size

Returns

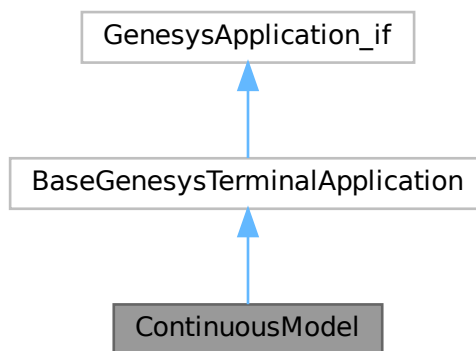
Number of configured output connections.

The documentation for this class was generated from the following files:

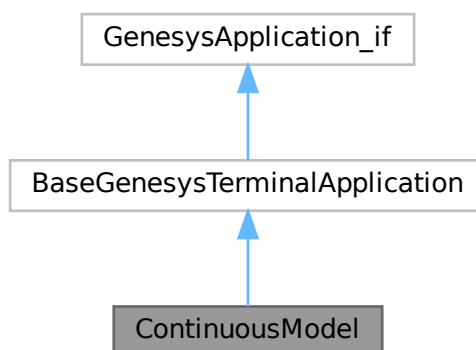
- kernel/simulator/ConnectionManager.h
- kernel/simulator/ConnectionManager.cpp

10.112 ContinuousModel Class Reference

Inheritance diagram for ContinuousModel:



Collaboration diagram for ContinuousModel:



Public Member Functions

- virtual void [Simulate](#) ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual int [main](#) (int argc, char **argv) override
- void [setDefaultTraceHandlers](#) ([TraceManager](#) *tm)
- void [setDefaultEventHandlers](#) ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void [traceHandler](#) ([TraceEvent](#) e)
- virtual void [traceErrorHandler](#) ([TraceErrorEvent](#) e)
- virtual void [traceReportHandler](#) ([TraceEvent](#) e)
- virtual void [traceSimulationHandler](#) ([TraceSimulationEvent](#) e)
- virtual void [onAfterProcessEventHandler](#) ([SimulationEvent](#) *re)
- virtual void [onBreakpointHandler](#) ([SimulationEvent](#) *re)
- virtual void [onEntityCreateHandler](#) ([SimulationEvent](#) *re)
- virtual void [onEntityMoveHandler](#) ([SimulationEvent](#) *re)
- virtual void [onSimulationStartHandler](#) ([SimulationEvent](#) *re)
- virtual void [onReplicationStepHandler](#) ([SimulationEvent](#) *re)
- virtual void [onReplicationStartHandler](#) ([SimulationEvent](#) *re)
- virtual void [onProcessEventHandler](#) ([SimulationEvent](#) *re)
- virtual void [onReplicationEndHandler](#) ([SimulationEvent](#) *re)
- virtual void [onSimulationEndHandler](#) ([SimulationEvent](#) *re)
- virtual void [onSimulationPausedHandler](#) ([SimulationEvent](#) *re)
- virtual void [onSimulationResumeHandler](#) ([SimulationEvent](#) *re)
- virtual void [onEntityRemoveHandler](#) ([SimulationEvent](#) *re)

10.112.1 Member Function Documentation

10.112.1.1 Simulate()

```
void ContinuousModel::Simulate (
    Simulator * genesys,
    Model * model,
    PluginManager * plugins ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

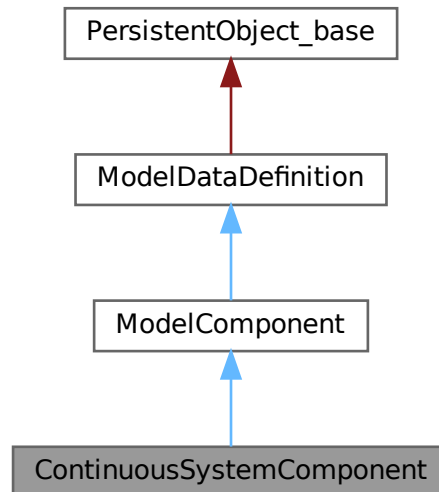
- applications/modelSpecific/teaching/ContinuousModel.h
- applications/modelSpecific/teaching/ContinuousModel.cpp

10.113 ContinuousSystemComponent Class Reference

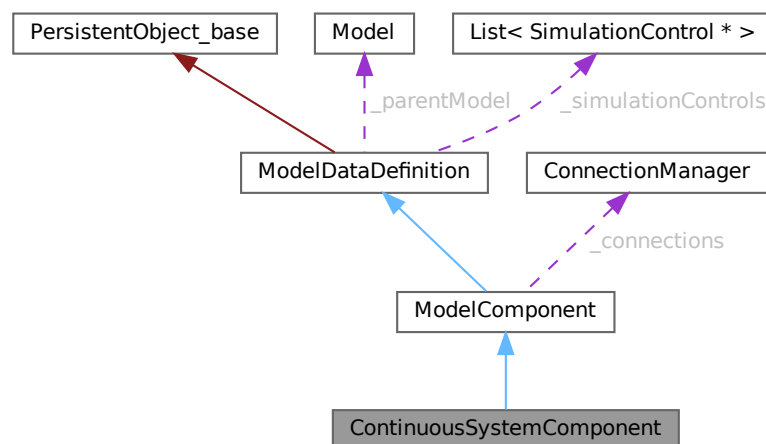
Component that advances a continuous ODE system when entities arrive.

```
#include <plugins/components/Continuous/ContinuousSystemComponent.h>
```

Inheritance diagram for ContinuousSystemComponent:



Collaboration diagram for ContinuousSystemComponent:



Public Member Functions

- **ContinuousSystemComponent** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
- *virtual public methods*
- void [setOdeSolver](#) ([ODESolver](#) *solver)
- [ODESolver](#) * [getOdeSolver](#) () const
- void [setOdeSolverName](#) (std::string name)
- std::string [getOdeSolverName](#) () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * [getConnectionManager](#) () const
- *new public user methods for this component*
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- *Connects this component to another component.*
- bool [hasBreakpointAt](#) ()
- *Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string _description)
- *Updates the free-form description shown by the GUI.*
- std::string [getDescription](#) () const
- *Returns the free-form description associated with this component.*
- void [remove](#) ([ModelComponent](#) *comp)
- *Removes a connected component from this component.*

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵IntoModel=true)
- *Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- *Releases resources owned by this data definition.*
- Util::identification [getId](#) () const
- *Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)
- *Updates the logical name of this instance.*
- const std::string & [getName](#) () const
- *Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
- *Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
- *Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const
- *Returns the runtime class name used by the plugin system.*
- [Model](#) * [getParentModel](#) () const
- *Returns the parent model that owns this definition.*

- bool `isReportStatistics ()` const
Indicates whether this data definition contributes report statistics.
- void `setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name)` const
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData ()` const
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData ()` const
Returns the full map of attached data definitions.
- bool `hasChanged ()` const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- unsigned int `getLevel ()` const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls ()` const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific ()` const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled ()` const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation ()`
- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`
- static void `SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
- static bool `Check (ModelComponent *component)`
- static void `CreateInternalData (ModelComponent *component)`
- static void `DispatchEvent (Event *event)`

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check (ModelComponent *component)`
Validates a component instance.
- static void `CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- static void `SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- static void `DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.113.1 Detailed Description

Component that advances a continuous ODE system when entities arrive.

[ContinuousSystemComponent](#) holds a reference to an [ODESolver](#) data definition and triggers integration each time an entity enters through port 0. The solved state is written back to the [ODESolver](#) so downstream components can read the updated values. The entity is forwarded through output port 0 after integration completes.

10.113.2 Member Function Documentation

10.113.2.1 `_check()`

```
bool ContinuousSystemComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.113.2.2 `_createEditableDataDefinitions()`

```
void ContinuousSystemComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.113.2.3 `_loadInstance()`

```
bool ContinuousSystemComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.113.2.4 `_onDispatchEvent()`

```
void ContinuousSystemComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.113.2.5 `_saveInstance()`

```
void ContinuousSystemComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.113.2.6 `show()`

```
std::string ContinuousSystemComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

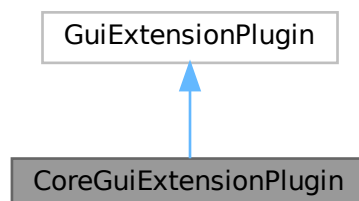
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

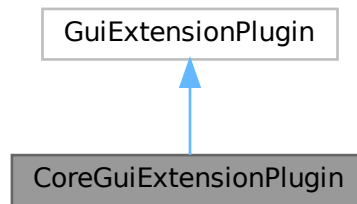
- plugins/components/Continuous/ContinuousSystemComponent.h
- plugins/components/Continuous/ContinuousSystemComponent.cpp

10.114 CoreGuiExtensionPlugin Class Reference

Inheritance diagram for CoreGuiExtensionPlugin:



Collaboration diagram for CoreGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId () const` override
- `std::string displayName () const` override
- `std::string version () const` override
- `std::vector< std::string > requiredModelPlugins () const` override
- `void registerContributions (GuiExtensionRegistry *registry) const` override

10.114.1 Member Function Documentation

10.114.1.1 displayName()

```
std::string CoreGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.114.1.2 extensionId()

```
std::string CoreGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.114.1.3 registerContributions()

```
void CoreGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.114.1.4 requiredModelPlugins()

```
std::vector< std::string > CoreGuiExtensionPlugin::requiredModelPlugins ( ) const [inline],
[override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.114.1.5 version()

```
std::string CoreGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

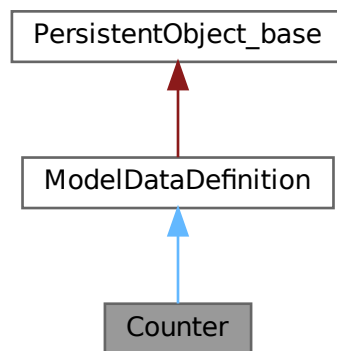
- applications/gui/genesys/extensions/CoreGuiExtensionPlugin.cpp

10.115 Counter Class Reference

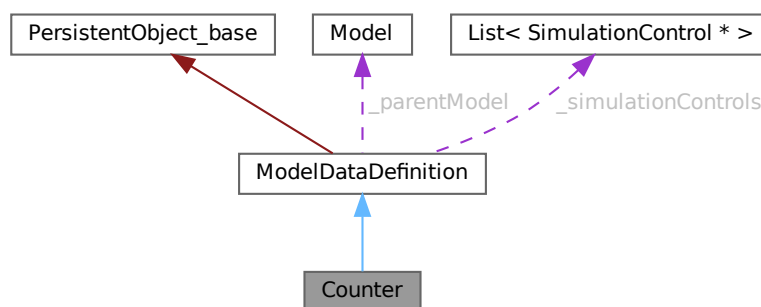
[Model](#) datum used to accumulate event counts (or weighted counts).

```
#include <kernel/simulator/essentialPlugins/Counter.h>
```

Inheritance diagram for Counter:



Collaboration diagram for Counter:



Public Member Functions

- **Counter** ([Model](#) *model, std::string name="", [ModelDataDefinition](#) *parent=nullptr)
Creates a counter model datum with optional parent relationship.
- virtual std::string [show](#) () override
Returns a textual representation of this data definition.
- void [clear](#) ()
clear
- void [incCountValue](#) (double value=1.0)
incCountValue
- double [getCountValue](#) () const
getCountValue
- [ModelDataDefinition](#) * [getParent](#) () const
getParent

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.

- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createInternalStatisticReporters ()`
- `virtual void _createAttachedAttributes ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `virtual void _createEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
- legacy helper methods used by the granular create* hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.115.1 Detailed Description

[Model](#) datum used to accumulate event counts (or weighted counts).

The counter value starts cleared for each replication (depending on model lifecycle hooks) and can be incremented by arbitrary amounts.

10.115.2 Member Function Documentation

10.115.2.1 `_check()`

```
bool Counter::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.115.2.2 `_initBetweenReplications()`

```
void Counter::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.115.2.3 `_loadInstance()`

```
bool Counter::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.115.2.4 `_saveInstance()`

```
void Counter::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.115.2.5 clear()

```
void Counter::clear ( )
```

clear

Resets the internal count value to zero.

10.115.2.6 getCountValue()

```
double Counter::getCountValue ( ) const
```

getCountValue

Returns

Current accumulated count value.

10.115.2.7 getParent()

```
ModelDataDefinition * Counter::getParent ( ) const
```

getParent

Returns

Parent model datum associated with this counter (if any).

10.115.2.8 incCountValue()

```
void Counter::incCountValue (
    double value = 1.0 )
```

incCountValue

Parameters

<i>value</i>	
--------------	--

Adds `value` to the internal count (default increment is 1).

10.115.2.9 `show()`

```
std::string Counter::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal `ModelDataDefinitions` such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

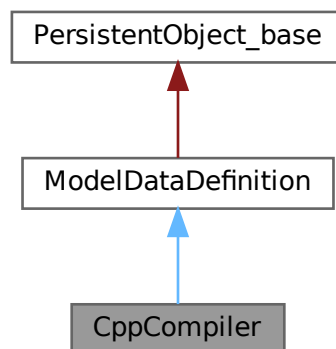
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

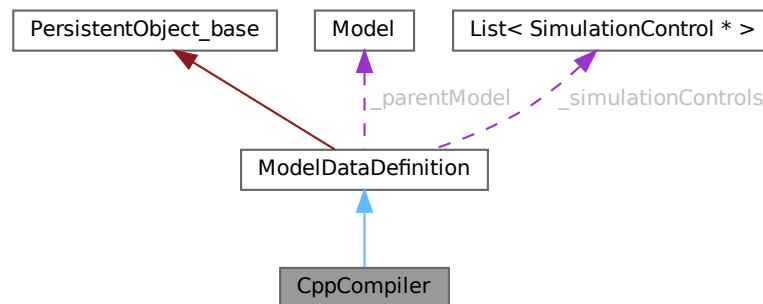
- `kernel/simulator/essentialPlugins/Counter.h`
- `kernel/simulator/essentialPlugins/Counter.cpp`

10.116 CppCompiler Class Reference

Inheritance diagram for `CppCompiler`:



Collaboration diagram for CppCompiler:



Classes

- class [CompilationResult](#)

Public Member Functions

- **CppCompiler** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **setSourceFilename** (std::string sourceFilename)
- std::string **getSourceFilename** () const
- bool **IsLibraryLoaded** () const
- void **setFlagsExecutable** (std::string _flagsExecutable)
- std::string **getFlagsExecutable** () const
- void **setLibraryLoaded** (bool libraryLoaded)
- void **setFlagsStaticLibrary** (std::string _flagsStaticLibrary)
- std::string **getFlagsStaticLibrary** () const
- void **setFlagsDynamicLibrary** (std::string _flagsDynamicLibrary)
- std::string **getFlagsDynamicLibrary** () const
- void **setFlagsGeneral** (std::string _flagsGeneral)
- std::string **getFlagsGeneral** () const
- void **setCompilerCommand** (std::string _compilerCommand)
- std::string **getCompilerCommand** () const
- void **setOutputDir** (std::string _outputDir)
- std::string **getOutputDir** () const
- void **setTempDir** (std::string _tempDir)
- std::string **getTempDir** () const
- void **setOutputFilename** (std::string _outputFilename)
- std::string **getOutputFilename** () const
- void **setObjectFiles** (std::string _objectFiles)
- std::string **getObjectFiles** () const
- [CompilationResult](#) **compileToExecutable** ()
- [CompilationResult](#) **compileToDynamicLibrary** ()
- [CompilationResult](#) **compileToStaticLibrary** ()
- bool **loadLibrary** (std::string &errorMessage)
- bool **unloadLibrary** ()
- void * **getDynamicLibraryHandler** () const

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_initBetweenReplications** () override
- [CompilationResult](#) **_invokeCompiler** (std::string command)
- std::string **_read** (std::string filename)

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createNonEditableDataDefinitions** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()

legacy helper methods used by the granular create hooks*

- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.116.1 Member Function Documentation

10.116.1.1 `_check()`

```
bool CppCompiler::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Validate minimal compiler configuration before use.

Reimplemented from [ModelDataDefinition](#).

10.116.1.2 `_initBetweenReplications()`

```
void CppCompiler::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.116.1.3 `_loadInstance()`

```
bool CppCompiler::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Load persisted compiler configuration.

The structure intentionally follows the same pattern used by other [ModelDataDefinition](#) implementations to simplify future maintenance.

Reimplemented from [ModelDataDefinition](#).

10.116.1.4 `_saveInstance()`

```
void CppCompiler::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Persist compiler configuration.

Keep field names symmetric to `_loadInstance` for predictable serialization behavior and easier backward-compatible migrations.

Reimplemented from [ModelDataDefinition](#).

10.116.1.5 show()

```
std::string CppCompiler::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

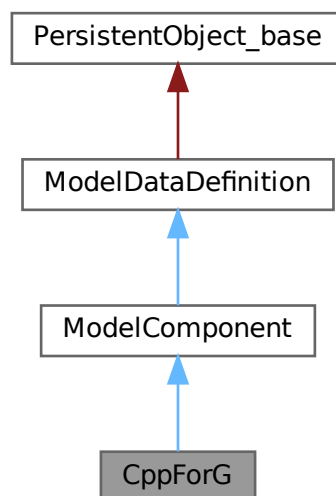
The documentation for this class was generated from the following files:

- plugins/data/ExternalIntegration/CppCompiler.h
- plugins/data/ExternalIntegration/CppCompiler.cpp

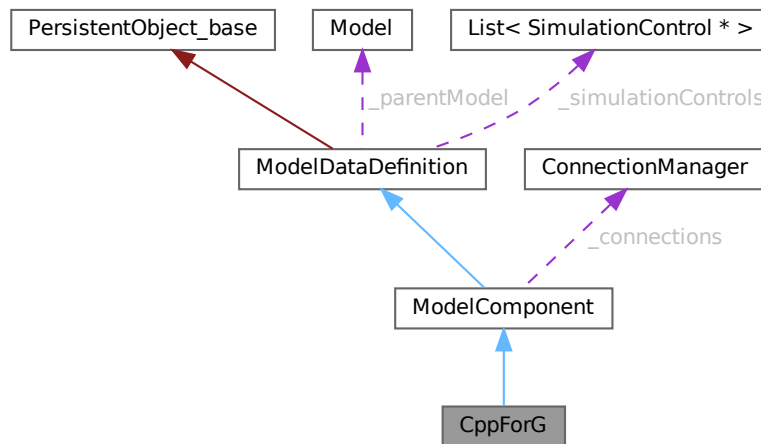
10.117 CppForG Class Reference

```
#include <plugins/components/ExternalIntegration/CppForG.h>
```

Inheritance diagram for CppForG:



Collaboration diagram for CppForG:



Public Member Functions

- **CppForG** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- void **setCppCompiler** ([CppCompiler](#) * _cppCompiler)
- [CppCompiler](#) * **getCppCompiler** () const
- void **setInitBetweenReplicationCode** (std::string _initBetweenReplicationCode)
- std::string **getInitBetweenReplicationCode** () const
- void **setOnDispatchEventCode** (std::string _onDispatchEventCode)
- std::string **getOnDispatchEventCode** () const
- void **setIncludesCode** (std::string _includesCode)
- std::string **getIncludesCode** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- *new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- *Connects this component to another component.*
- bool **hasBreakpointAt** ()
- *Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string _description)
- *Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const
- *Returns the free-form description associated with this component.*
- void **remove** ([ModelComponent](#) *comp)
- *Removes a connected component from this component.*

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_initBetweenReplications** () override
- virtual void **_createNonEditableDataDefinitions** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createInternalStatisticReporters ()`
- `virtual void _createAttachedAttributes ()`
- `virtual void _createEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
 - legacy helper methods used by the granular create* hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
 - Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.117.1 Detailed Description

This component ...

10.117.2 Member Function Documentation

10.117.2.1 `_check()`

```
bool CppForG::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.117.2.2 `_createNonEditableDataDefinitions()`

```
void CppForG::_createNonEditableDataDefinitions ( ) [override], [protected], [virtual]
```

This method is necessary only for those components that instantiate internal elements that must exist before simulation starts and even before model checking. That's the case of components that have internal [StatisticsCollectors](#), since others components may refer to them as expressions (as in "TVAG(ThisCSTAT)") and therefore the model datum must exist before checking such expression

Reimplemented from [ModelDataDefinition](#).

10.117.2.3 `_initBetweenReplications()`

```
void CppForG::_initBetweenReplications ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

10.117.2.4 `_loadInstance()`

```
bool CppForG::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.117.2.5 `_onDispatchEvent()`

```
void CppForG::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.117.2.6 `_saveInstance()`

```
void CppForG::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.117.2.7 show()

```
std::string CppForG::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/ExternalIntegration/CppForG.h
- plugins/components/ExternalIntegration/CppForG.cpp

10.118 CppModelExporter Class Reference

Service responsible for C++ code representation of the current simulation model.

```
#include <applications/gui/genesys/services/CppModelExporter.h>
```

Public Member Functions

- **CppModelExporter** ([Simulator](#) *simulator, QPlainTextEdit *cppCodeEditor)
Creates the exporter service for the C++ model-representation pane.
- std::string [addCppCodeLine](#) (const std::string &line, unsigned int indent=0) const
Formats one C++ output line with the requested indentation level.
- void **actualizeModelCppCode** () const
Rebuilds the full C++ model representation shown in the GUI editor pane.

10.118.1 Detailed Description

Service responsible for C++ code representation of the current simulation model.

The refactoring keeps C++ representation generation outside [MainWindow](#) by delegating this concern to a dedicated service. [MainWindow](#) wrappers call into this class when model or UI state changes require regeneration.

Responsibilities:

- format indented C++ output lines used by exporter routines;
- rebuild the full C++ representation shown in the GUI code pane.

Boundaries:

- it does not compile or execute generated code;
- it does not persist files directly;
- it does not manage lifecycle/simulation/scene controller flows.

10.118.2 Member Function Documentation

10.118.2.1 addCppCodeLine()

```
std::string CppModelExporter::addCppCodeLine (
    const std::string & line,
    unsigned int indent = 0 ) const
```

Formats one C++ output line with the requested indentation level.

Returns

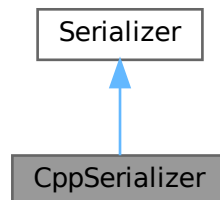
Formatted line that can be appended to the representation buffer.

The documentation for this class was generated from the following files:

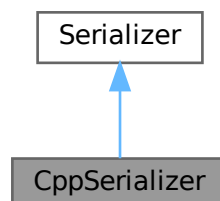
- applications/gui/genesys/services/CppModelExporter.h
- applications/gui/genesys/services/CppModelExporter.cpp

10.119 CppSerializer Class Reference

Inheritance diagram for CppSerializer:



Collaboration diagram for CppSerializer:



Public Member Functions

- **CppSerializer** ([Model](#) *model)
- [PersistenceRecord](#) * [newPersistenceRecord](#) () override
Creates an empty persistence record.
- bool [dump](#) (std::ostream &output) override
Writes formatted contents to a given output stream.
- bool [load](#) (std::istream &input) override
Attempts to load contents from a given input stream.
- bool [get](#) (const std::string &name, [PersistenceRecord](#) *entry) override
Tries to fetch an entry with the given label as a key.
- bool [put](#) (const std::string name, const std::string type, const Util::identification id, [PersistenceRecord](#) *fields) override
Upserts an entry into this structure.
- int [for_each](#) (std::function< int(const std::string &) > delegate) override
Iterates over this structure's entries, ordered by ID.

10.119.1 Member Function Documentation

10.119.1.1 dump()

```
bool CppSerializer::dump (
    std::ostream & output ) [override], [virtual]
```

Writes formatted contents to a given output stream.

Parameters

<i>output</i>	output stream to write to.
---------------	----------------------------

Returns

whether the current state of this structure's contents was deemed valid for serialization.

Implements [Serializer](#).

10.119.1.2 for_each()

```
int CppSerializer::for_each (
    std::function< int(const std::string &) > delegate ) [override], [virtual]
```

Iterates over this structure's entries, ordered by ID.

Parameters

<i>delegate</i>	delegate which is applied to every entry-identifying key, until it returns a non-zero value.
-----------------	--

Returns

either zero, or the first non-zero value returned by the delegate.

Implements [Serializer](#).

10.119.1.3 get()

```
bool CppSerializer::get (
    const std::string & name,
    PersistenceRecord * entry ) [override], [virtual]
```

Tries to fetch an entry with the given label as a key.

Parameters

<i>name</i>	a label which uniquely identifies an entry.
<i>entry</i>	(out-param) a pointer to a caller-owned entry, which is filled on success.

Returns

whether the entry was found and the out-param was actually written to.

Implements [Serializer](#).

10.119.1.4 load()

```
bool CppSerializer::load (
    std::istream & input ) [override], [virtual]
```

Attempts to load contents from a given input stream.

Returns

whether the input was well-formatted and loading was successful.

Implements [Serializer](#).

10.119.1.5 newPersistenceRecord()

```
PersistenceRecord * CppSerializer::newPersistenceRecord ( ) [override], [virtual]
```

Creates an empty persistence record.

Although the returned record can be freed normally with `delete`, care must be taken to only use records which come from a implementation, when configuring that implementation's contents.

Implements [Serializer](#).

10.119.1.6 put()

```
bool CppSerializer::put (
    const std::string name,
    const std::string type,
    const Util::identification id,
    PersistenceRecord * fields ) [override], [virtual]
```

Upserts an entry into this structure.

Parameters

<i>name</i>	a label which uniquely identifies the entry.
<i>type</i>	a type name.
<i>id</i>	a component/datadef ID number.
<i>fields</i>	a persistence record for the entry, whose contents will be copied if valid.

Returns

whether the entry was valid and was actually copied into this structure.

Implements [Serializer](#).

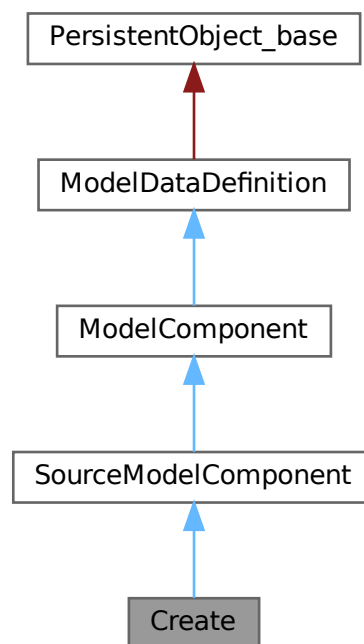
The documentation for this class was generated from the following files:

- kernel/simulator/persistence/CppSerializer.h
- kernel/simulator/persistence/CppSerializer.cpp

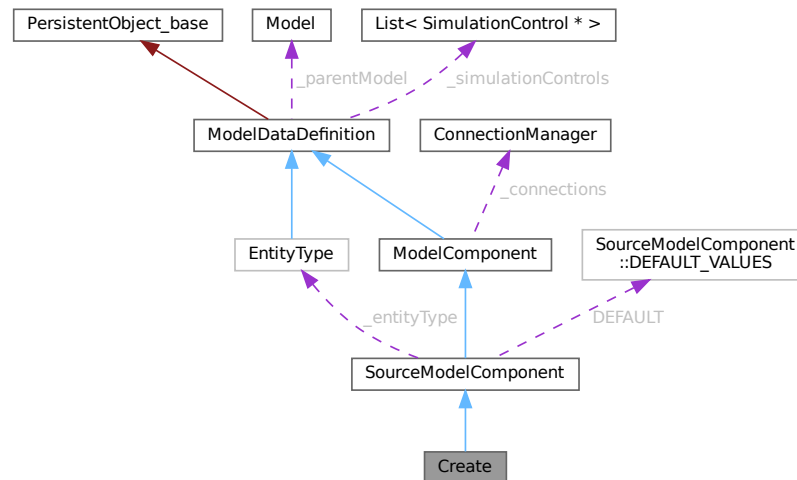
10.120 Create Class Reference

```
#include <plugins/components/Logic/Create.h>
```

Inheritance diagram for Create:



Collaboration diagram for Create:



Public Member Functions

- **Create** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
- *virtual public methods*
- void **setTimeBetweenCreationsFormula** ([Formula](#) *_timeBetweenCreationsFormula)
- [Formula](#) * **getTimeBetweenCreationsFormula** () const
- void **setTimeBetweenCreationsSchedule** ([Schedule](#) *_timeBetweenCreationsSchedule)
- [Schedule](#) * **getTimeBetweenCreationsSchedule** () const
- double **testePropertyCreateDouble** () const
- void **setTestePropertyCreateDouble** (double newTestePropertyCreateDouble)

Public Member Functions inherited from [SourceModelComponent](#)

- **SourceModelComponent** ([Model](#) *model, std::string componentTypename, std::string name="")
- void [setFirstCreation](#) (double _firstCreation)
- *setFirstCreation*
- double [getFirstCreation](#) () const
- *getFirstCreation*
- void [setEntityType](#) ([EntityType](#) *_entityType)
- *setEntityType*
- void [setEntityTypename](#) (std::string entityTypeName)
- *setEntityTypename*
- [EntityType](#) * [getEntityType](#) () const
- *getEntityType*
- void [setTimeUnit](#) ([Util::TimeUnit](#) _timeUnit)
- *setTimeUnit*
- [Util::TimeUnit](#) [getTimeUnit](#) () const
- *getTimeUnit*

- void [setTimeBetweenCreationsExpression](#) (std::string _timeBetweenCreations, [Util::TimeUnit](#) _timeUnit=Util::TimeUnit::unknown)
setTimeBetweenCreationsExpression
- std::string [getTimeBetweenCreationsExpression](#) () const
getTimeBetweenCreationsExpression
- void [setMaxCreations](#) (unsigned long _maxCreations)
setMaxCreations
- void [setMaxCreations](#) (std::string _maxCreationsExpression)
setMaxCreations
- std::string [getMaxCreations](#) () const
getMaxCreations
- unsigned int [getEntitiesCreated](#) () const
getEntitiesCreated
- void [setEntitiesCreated](#) (unsigned int _entitiesCreated)
setEntitiesCreated
- void [setEntitiesPerCreation](#) (unsigned int _entitiesPerCreation)
setEntitiesPerCreation
- unsigned int [getEntitiesPerCreation](#) () const
getEntitiesPerCreation

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual [~ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insertIntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual [~ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)

- Updates the logical name of this instance.*
- `const std::string & getName () const`
Returns the logical name of this instance.
- `void setLabel (const std::string &label)`
Updates the user-visible label of this instance.
- `const std::string & getLabel () const`
Returns the user-visible label of this instance.
- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_initBetweenReplications](#) () override
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createInternalStatisticReporters](#) () override
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [SourceModelComponent](#)

- virtual void [_createAttachedAttributes](#) ()

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SourceModelComponent](#)

- [EntityType](#) * **_entityType** = nullptr
- const struct [SourceModelComponent::DEFAULT_VALUES](#) **DEFAULT**
- double **_firstCreation** = DEFAULT.firstCreation
- unsigned int **_entitiesPerCreation** = DEFAULT.entitiesPerCreation
- std::string **_maxCreationsExpression** = DEFAULT.maxCreationsExpression
- std::string **_timeBetweenCreationsExpression** = DEFAULT.timeBetweenCreationsExpression
- [Util::TimeUnit](#) **_timeBetweenCreationsTimeUnit** = DEFAULT.timeBetweenCreationsTimeUnit
- unsigned int **_entitiesCreatedSoFar** = 0

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
- virtual protected methods that could be overridden by derived classes, if needed*

Protected Attributes inherited from [ModelDataDefinition](#)

- [Util::identification](#) **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager::Level](#) **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.120.1 Detailed Description

[Create](#) is the most basic component to include the first entities into the model, and therefore is a source component (derived from [SourceModelComponent](#)) [Create](#) module DESCRIPTION This module is intended as the starting point for entities in a simulation model. Entities are created using a schedule or based on a time between arrivals. Entities then leave the module to begin processing through the system. The entity type is specified in this module. TYPICAL USES The start of a part's production in a manufacturing line A document's arrival (for example, order, check, application) into a business process A customer's arrival at a service process (for example, retail store, restaurant, information desk) PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. [Entity](#) Type Name of the entity type to be generated. Type Type of arrival stream to be generated. Types include Random (uses an exponential distribution, user specifies mean), [Schedule](#) (uses an exponential distribution, mean determined from the specified [Schedule](#) module), Constant (user specifies constant value; for example, 100), or Expression (drop-down list of various distributions). Value Determines the mean of the exponential distribution (if Random is used) or the constant value (if Constant is used) for the time between arrivals. Applies only when Type is Random or Constant. [Schedule](#) Name Identifies the name of the schedule to be used. The schedule defines the arrival pattern for entities arriving to the system. Applies only when Type is [Schedule](#). Expression Any distribution or value specifying the time between arrivals. Applies only when Type is Expression. Units Time units used for interarrival and first creation times. Does not apply when Type is [Schedule](#). Entities per Arrival Number of entities that will enter the system at a given time with each arrival. Max Arrivals Maximum number of entities that this module will generate. When this value is reached, the creation of new entities by this module ceases. First Creation Starting time for the first entity to arrive into the system. Does not apply when Type is [Schedule](#).

10.120.2 Member Function Documentation

10.120.2.1 `_check()`

```
bool Create::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [SourceModelComponent](#).

10.120.2.2 `_createEditableDataDefinitions()`

```
void Create::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [SourceModelComponent](#).

10.120.2.3 `_createInternalStatisticReporters()`

```
void Create::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.120.2.4 `_initBetweenReplications()`

```
void Create::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [SourceModelComponent](#).

10.120.2.5 `_loadInstance()`

```
bool Create::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SourceModelComponent](#).

10.120.2.6 `_onDispatchEvent()`

```
void Create::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.120.2.7 _saveInstance()

```
void Create::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SourceModelComponent](#).

10.120.2.8 show()

```
std::string Create::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [SourceModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/Logic/Create.h
- plugins/components/Logic/Create.cpp

10.121 SimulatorSessionService::CreateSessionResult Struct Reference

Contains session creation identifiers returned to API clients.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Public Attributes

- std::string **sessionId**
- std::string **accessToken**

10.121.1 Detailed Description

Contains session creation identifiers returned to API clients.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

10.122 QtBoolPropertyManagerPrivate::Data Struct Reference

Public Attributes

- bool **val**
- bool **initVal**
- bool **isInitialed**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.123 QtDatePropertyManagerPrivate::Data Struct Reference

Public Member Functions

- QDate **minimumValue** () const
- QDate **maximumValue** () const
- void **setMinimumValue** (QDate newMinVal)
- void **setMaximumValue** (QDate newMaxVal)

Public Attributes

- QDate **val** { QDate::currentDate() }
- QDate **minVal** { QDate(1752, 9, 14) }
- QDate **maxVal** { QDate(9999, 12, 31) }
- QDate **initVal** { QDate::currentDate() }
- bool **isInitialed**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.124 QDateTimePropertyManagerPrivate::Data Struct Reference

Public Attributes

- QDateTime **val** { QDateTime() }
- QDateTime **initVal** { QDateTime() }
- bool **isInitialed** { false }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.125 QtDoublePropertyManagerPrivate::Data Struct Reference

Public Member Functions

- double **minimumValue** () const
- double **maximumValue** () const
- void **setMinimumValue** (double newMinVal)
- void **setMaximumValue** (double newMaxVal)

Public Attributes

- double **val** { 0 }
- double **initVal** { 0 }
- double **minVal** { -DBL_MAX }
- double **maxVal** { DBL_MAX }
- double **singleStep** { 1 }
- int **decimals** { 2 }
- bool **isInitialed** { false }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.126 QtEnumPropertyManagerPrivate::Data Struct Reference

Public Attributes

- int **val** { -1 }
- QStringList **enumNames**
- QMap< int, QIcon > **enumIcons**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.127 QtFlagPropertyManagerPrivate::Data Struct Reference

Public Attributes

- int **val** { -1 }
- QStringList **flagNames**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.128 QtIntPropertyManagerPrivate::Data Struct Reference

Public Member Functions

- int **minimumValue** () const
- int **maximumValue** () const
- void **setMinimumValue** (int newMinVal)
- void **setMaximumValue** (int newMaxVal)

Public Attributes

- int **val** { 0 }
- int **initVal** { 0 }
- int **minVal** { -INT_MAX }
- int **maxVal** { INT_MAX }
- int **singleStep** { 1 }
- bool **isInitialed** { false }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.129 QtPointFPropertyManagerPrivate::Data Struct Reference

Public Attributes

- QPointF **val**
- int **decimals** { 2 }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.130 QtRectFPropertyManagerPrivate::Data Struct Reference

Public Attributes

- QRectF **val** { 0, 0, 0, 0 }
- QRectF **constraint**
- int **decimals** { 2 }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.131 QtRectPropertyManagerPrivate::Data Struct Reference

Public Attributes

- QRect **val** { 0, 0, 0, 0 }
- QRect **constraint**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.132 QSizeFPropertyManagerPrivate::Data Struct Reference

Public Member Functions

- QSizeF **minimumValue** () const
- QSizeF **maximumValue** () const
- void **setMinimumValue** (const QSizeF &newMinVal)
- void **setMaximumValue** (const QSizeF &newMaxVal)

Public Attributes

- QSizeF **val** { 0, 0 }
- QSizeF **minVal** { 0, 0 }
- QSizeF **maxVal** { std::numeric_limits<qreal>::max(), std::numeric_limits<qreal>::max() }
- int **decimals** { 2 }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.133 QSizePropertyManagerPrivate::Data Struct Reference

Public Member Functions

- QSize **minimumValue** () const
- QSize **maximumValue** () const
- void **setMinimumValue** (const QSize &newMinVal)
- void **setMaximumValue** (const QSize &newMaxVal)

Public Attributes

- QSize **val** { 0, 0 }
- QSize **minVal** { 0, 0 }
- QSize **maxVal** { INT_MAX, INT_MAX }

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.134 QStringPropertyManagerPrivate::Data Struct Reference

Public Attributes

- QString **val**
- QString **initVal**
- bool **isInitialed**
- QRegularExpression **regExp**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.135 QtTimePropertyManagerPrivate::Data Struct Reference

Public Attributes

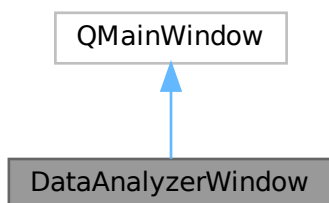
- QTime **val** { QTime() }
- QTime **initVal** { QTime() }
- bool **isInitialed** { false }

The documentation for this struct was generated from the following file:

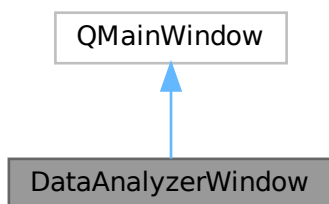
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.136 DataAnalyzerWindow Class Reference

Inheritance diagram for DataAnalyzerWindow:



Collaboration diagram for DataAnalyzerWindow:



Classes

- class [DoePreviewPlot](#)
- class [HistogramPreview](#)

Public Member Functions

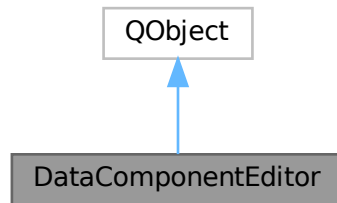
- **DataAnalyzerWindow** ([Simulator](#) *simulator, QString &lastDataAnalyzerPath, QWidget *parent=nullptr)

The documentation for this class was generated from the following files:

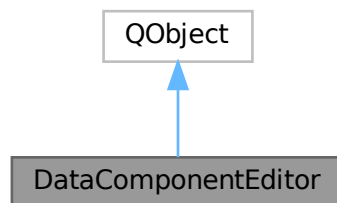
- applications/gui/genesys/tools/dataanalyzer/DataAnalyzerWindow.h
- applications/gui/genesys/tools/dataanalyzer/DataAnalyzerWindow.cpp

10.137 DataComponentEditor Class Reference

Inheritance diagram for DataComponentEditor:



Collaboration diagram for DataComponentEditor:



Public Types

- using **AfterChange** = `std::function< void()>`

Public Member Functions

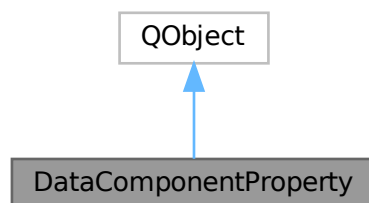
- **DataComponentEditor** ([PropertyEditorGenesys](#) *editor, [SimulationControl](#) *property, AfterChange after↔ Change={})
- **DataComponentEditor** ([PropertyEditorGenesys](#) *editor, [List](#)< [SimulationControl](#) * > *properties, After↔ Change afterChange={})
- void **open_window** ([SimulationControl](#) *property)
- void **open_window** ([List](#)< [SimulationControl](#) * > *properties)
- void **configure_properties** ([SimulationControl](#) *property)
- void **configure_properties** ([List](#)< [SimulationControl](#) * > *properties)

The documentation for this class was generated from the following files:

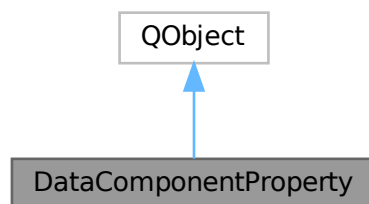
- `applications/gui/genesys/propertyeditor/DataComponentEditor.h`
- `applications/gui/genesys/propertyeditor/DataComponentEditor.cpp`

10.138 DataComponentProperty Class Reference

Inheritance diagram for DataComponentProperty:



Collaboration diagram for DataComponentProperty:



Public Types

- using **AfterChange** = `std::function< void()>`

Public Member Functions

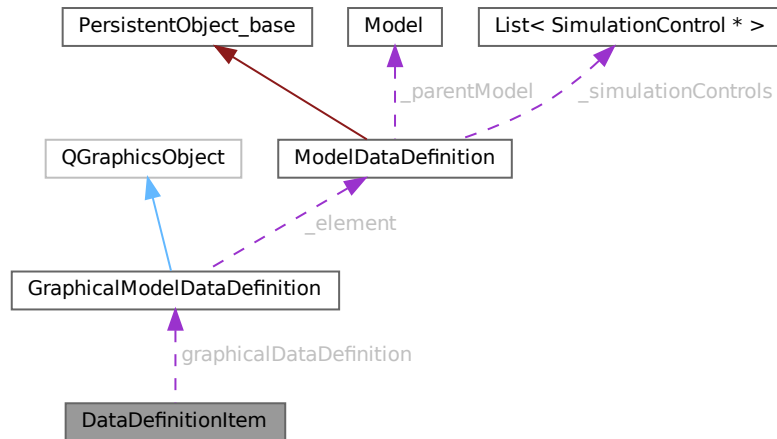
- **DataComponentProperty** ([PropertyEditorGenesys](#) *editor, [SimulationControl](#) *property, bool necessary↔
Config, AfterChange afterChange={})
- void **open_window** ()
- void **config_values** ()
- bool **isInList** (const std::string &value) const

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/DataComponentProperty.h
- applications/gui/genesys/propertyeditor/DataComponentProperty.cpp

10.139 DataDefinitionItem Struct Reference

Collaboration diagram for DataDefinitionItem:



Public Attributes

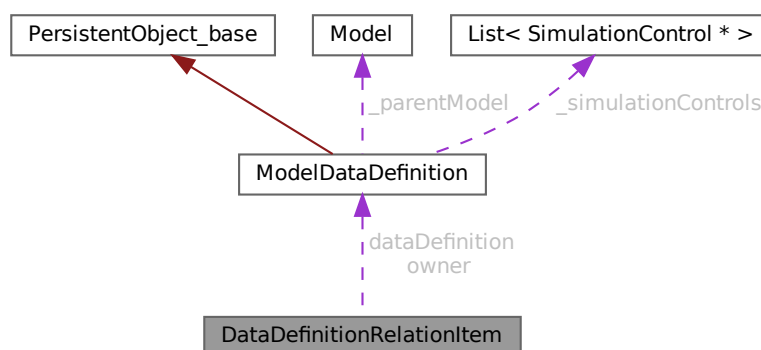
- [GraphicalModelDataDefinition](#) * **graphicalDataDefinition**
- `QPointF` **initialPosition**
- `bool` **initiallySelected**

The documentation for this struct was generated from the following file:

- `applications/gui/genesys/actions/DeleteUndoCommand.h`

10.140 DataDefinitionRelationItem Struct Reference

Collaboration diagram for DataDefinitionRelationItem:



Public Attributes

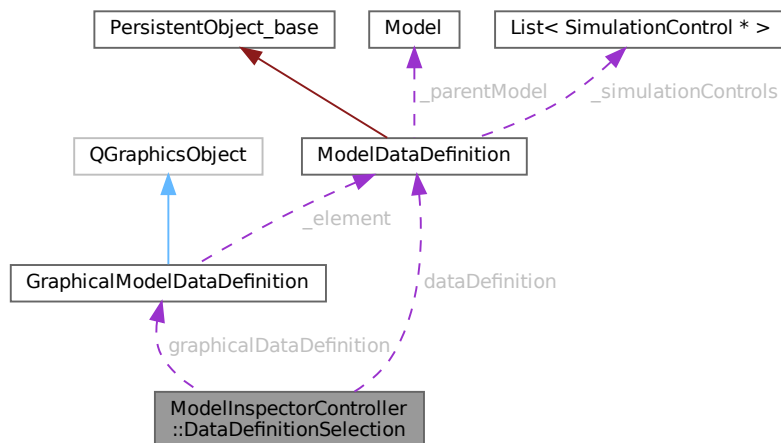
- [ModelDataDefinition](#) * **owner**
- [ModelDataDefinition](#) * **dataDefinition**
- std::string **key**
- bool **internal**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/actions/DeleteUndoCommand.h

10.141 ModelInspectorController::DataDefinitionSelection Struct Reference

Collaboration diagram for ModelInspectorController::DataDefinitionSelection:

**Public Attributes**

- bool **handled** = false
- [ModelDataDefinition](#) * **dataDefinition** = nullptr
- [GraphicalModelDataDefinition](#) * **graphicalDataDefinition** = nullptr

The documentation for this struct was generated from the following file:

- applications/gui/genesys/controllers/ModelInspectorController.h

10.142 AnimationPlot::DataSeries Struct Reference

Public Attributes

- QString **label**
- QString **expression**
- QColor **color** = QColor(31, 119, 180)
- qreal **lineWidth** = 2.0
- QVector< double > **values**

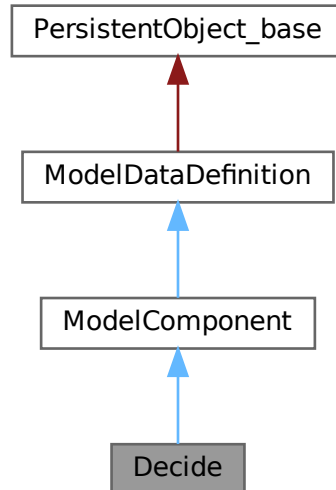
The documentation for this struct was generated from the following file:

- applications/gui/genesys/animations/AnimationPlaceholder.h

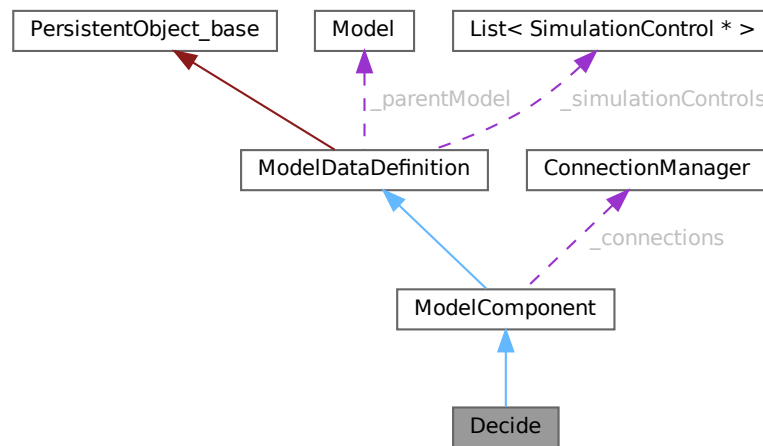
10.143 Decide Class Reference

```
#include <plugins/components/Decisions/Decide.h>
```

Inheritance diagram for Decide:



Collaboration diagram for Decide:



Public Member Functions

- **Decide** ([Model](#) *model, std::string name="")
- [List](#)< std::string > * **getConditions** () const
- void **addConditions** (std::string newCondition)
- void **removeConditions** (std::string condition)
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_createInternalStatisticReporters** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _initBetweenReplications ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createAttachedAttributes ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `virtual void _createEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
- *legacy helper methods used by the granular create* hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- *method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- `Util::identification _id`
- `std::string _typename`
- `bool _reportStatistics`
- `bool _hasChanged`
- `unsigned int _modelLevel = 0`
- [Model](#) * `_parentModel`
- `List< SimulationControl * > *_simulationControls` = new `List<SimulationControl*>()`
- `TraceManager::Level _traceLevelSpecific`
- `bool _traceLevelSpecificEnabled = false`

10.143.1 Detailed Description

[Decide](#) module DESCRIPTION This module allows for decision-making processes in the system. It includes options to make decisions based on one or more conditions (for example, if entity type is Gold Card) or based on one or more probabilities (for example, 75%, true; 25%, false). Conditions can be based on attribute values (for example, Priority), variable values (for example, Number Denied), the entity type, or an expression (for example, NQ(ProcessA.Queue)). There are two exit points out of the [Decide](#) module when its specified type is either 2-way by Chance or 2-way by Condition. There is one exit point for “true” entities and one for “false” entities. When the N-way by Chance or by Condition type is specified, multiple exit points are shown for each condition or probability and a single “else” exit. The number of entities that exit from each type (true/false) is displayed for 2-way by Chance or by Condition modules only. TYPICAL USES Dispatching a faulty part for rework Branching accepted vs. rejected checks Sending priority customers to a dedicated process Prompt Description Name Unique module identifier displayed on the module shape. Type Indicates whether the decision is based on a condition (if X>Y) or by chance/percentage (for example, 60%, yes; 40%, no). The type can be specified as either 2-way or N-way. 2-way allows for one condition or probability (plus the “false” exit). N-way allows for any number of conditions or probabilities to be specified as well as an “else” exit. Conditions Defines one or more conditions used to direct entities to different modules. Applies only when Type is N-way by Condition. Percentages Defines one or more percentages used to direct entities to different modules. Applies only when Type is N-way by Chance. Percent True Value that will be checked to determine the percentage of entities sent out a given True exit. If Types of conditions that are available for evaluation: [Variable](#), [Variable](#) Array (1D), [Variable](#) Array (2D), [Attribute](#), [Entity](#) Type, Expression. Named Specifies the name of the variable, attribute, or entity type that will be evaluated when an entity enters the module. Does not apply when Type is Expression. Is Evaluator for the condition. Applies only to [Attribute](#) and [Variable](#) conditions. Row Specifies the row index for a variable array. Applies only when Type is N-way by Condition or 2-way by Condition and [Variable](#) is Array 1-D or Array 2-D. Column Specifies the column index for a variable array. Applies only when Type is N-way by Condition or 2-way by Condition and [Variable](#) is Array 1-D or Array 2-D. Value Expression that will be either compared to an attribute or variable or that will be evaluated as a single expression to determine if it is true or false. Does not apply to [Entity](#) Type condition. If Type is Expression, this value must also include the evaluator (for example, Color<>Red).

10.143.2 Member Function Documentation

10.143.2.1 `_check()`

```
bool Decide::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.143.2.2 _createInternalStatisticReporters()

```
void Decide::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.143.2.3 _loadInstance()

```
bool Decide::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.143.2.4 _onDispatchEvent()

```
void Decide::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.143.2.5 _saveInstance()

```
void Decide::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.143.2.6 show()

```
std::string Decide::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/Decisions/Decide.h
- plugins/components/Decisions/Decide.cpp

10.144 Delay::DEFAULT_VALUES Struct Reference

Public Attributes

- const std::string **delayExpression** = "1.0"
- const [Util::TimeUnit](#) **delayTimeUnit** = Util::TimeUnit::second
- const Util::AllocationType **allocation** = Util::AllocationType::Wait

The documentation for this struct was generated from the following file:

- plugins/components/DiscreteProcessing/Delay.h

10.145 ModalModelDefault::DEFAULT_VALUES Struct Reference

Attributes that should be loaded or saved with this component (Persistent Fields)

```
#include <plugins/components/ModalModel/ModalModelDefault.h>
```

Public Attributes

- const std::string **timeDelayExpressionPerDispatch** = "0"
- const [Util::TimeUnit](#) **timeDelayPerDispatchTimeUnit** = Util::TimeUnit::second
- const unsigned int **maxTransitionsPerDispatch** = 1

10.145.1 Detailed Description

Attributes that should be loaded or saved with this component (Persistent Fields)

This method is called by ModelChecker and is necessary only for those components that instantiate internal elements that must exist before simulation starts and even before model checking. That's the case of components that have internal StatisticsCollectors, since others components may refer to them as expressions (as in "TVAG(This←CSTAT)") and therefore the modeldatum must exist before checking such expression

This method is not used yet. It should be usefull for new UIs Default values for the attributes. Used on initing, loading and saving

The documentation for this struct was generated from the following file:

- plugins/components/ModalModel/ModalModelDefault.h

10.146 Seize::DEFAULT_VALUES Struct Reference

Public Attributes

- const Util::AllocationType **allocationType** = Util::AllocationType::Others
- const unsigned short **priority** = 0
- const std::string **priorityExpression** = ""
- const unsigned int **seizeRequestSize** = 1

The documentation for this struct was generated from the following file:

- plugins/components/DiscreteProcessing/Seize.h

10.147 SourceModelComponent::DEFAULT_VALUES Struct Reference

Public Attributes

- const double **firstCreation** = 0.0
- const unsigned int **entitiesPerCreation** = 1
- const std::string **maxCreationsExpression** = std::to_string(std::numeric_limits<unsigned int>::max())
- const std::string **timeBetweenCreationsExpression** = "EXPO(1.0)"
- const Util::TimeUnit **timeBetweenCreationsTimeUnit** = Util::TimeUnit::second
- const std::string **entityTypename** = "entitytype"

The documentation for this struct was generated from the following file:

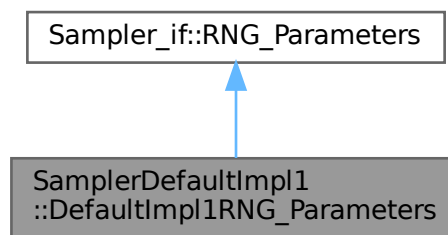
- kernel/simulator/SourceModelComponent.h

10.148 SamplerDefaultImpl1::DefaultImpl1RNG_Parameters Struct Reference

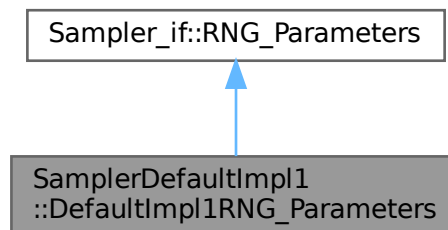
Parameter bundle for the default RNG algorithm.

```
#include <kernel/statistics/SamplerDefaultImpl1.h>
```

Inheritance diagram for SamplerDefaultImpl1::DefaultImpl1RNG_Parameters:



Collaboration diagram for SamplerDefaultImpl1::DefaultImpl1RNG_Parameters:



Public Attributes

- uint32_t **seed** = 16021974
Initial seed/state.
- uint32_t **a** = 279470273u
Multiplicative factor in the linear congruential generator.
- uint32_t **m** = 0xffffffffb
Modulus used by the generator.
- uint32_t **c** = 0
Increment term used by the generator.

10.148.1 Detailed Description

Parameter bundle for the default RNG algorithm.

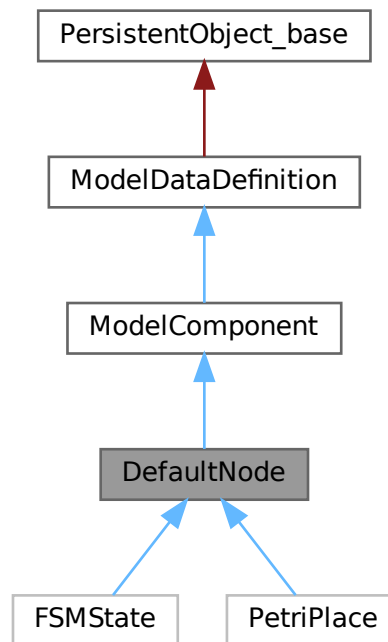
The documentation for this struct was generated from the following file:

- kernel/statistics/SamplerDefaultImpl1.h

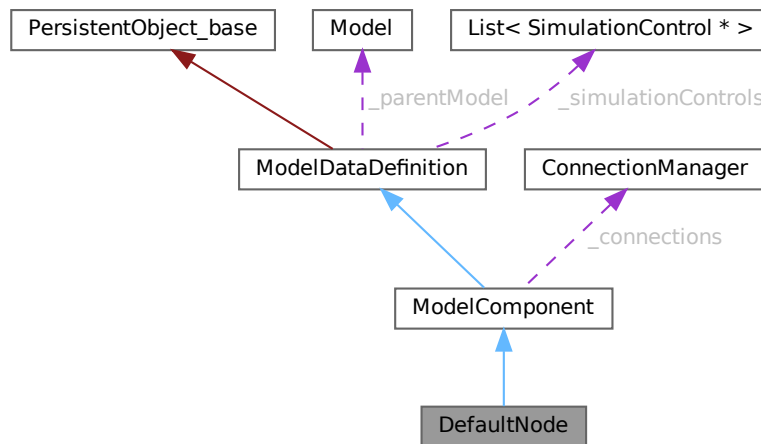
10.149 DefaultNode Class Reference

```
#include <plugins/components/ModalModel/DefaultNode.h>
```

Inheritance diagram for DefaultNode:



Collaboration diagram for DefaultNode:



Public Member Functions

- [DefaultNode](#) ([Model](#) *model, std::string componentTypename, std::string name)

constructors

- **DefaultNode** ([Model](#) *model, std::string name="")
- void **addTransition** ([DefaultNodeTransition](#) *transition)

new public user methods for this component

- void **removeTransition** ([DefaultNodeTransition](#) *transition)
- [List](#)< [DefaultNodeTransition](#) * > * **getTransitions** () const
- void **setInitialNode** (bool initialNode)
- bool **isInitialNode** () const
- void **setFinalNode** (bool finalNode)
- bool **isFinalNode** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")

constructors

- virtual ~**ModelComponent** ()

Releases component-owned resources and unregisters it from the model.

- [ConnectionManager](#) * **getConnectionManager** () const

new public user methods for this component

- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)

Connects this component to another component.

- bool **hasBreakpointAt** ()

Indicates whether a simulation breakpoint is set on this component.

- void **setDescription** (std::string _description)

Updates the free-form description shown by the GUI.

- `std::string getDescription () const`
Returns the free-form description associated with this component.
- `void remove (ModelComponent *comp)`
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const
Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)
Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual bool `_check` (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()

- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.149.1 Detailed Description

This component ...

10.149.2 Constructor & Destructor Documentation

10.149.2.1 DefaultNode()

```
DefaultNode::DefaultNode (
    Model * model,
    std::string componentTypename,
    std::string name )
```

constructors

Externalize function GetPluginInformation to be accessible through dynamic linked library.

10.149.3 Member Function Documentation

10.149.3.1 `_loadInstance()`

```
bool DefaultNode::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

Reimplemented in [FSMState](#), and [PetriPlace](#).

10.149.3.2 `_onDispatchEvent()`

```
void DefaultNode::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

This method is only for ModelComponents, not ModelDataElements.

Implements [ModelComponent](#).

10.149.3.3 `_saveInstance()`

```
void DefaultNode::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

Reimplemented in [FSMState](#), and [PetriPlace](#).

10.149.3.4 `show()`

```
std::string DefaultNode::show ( ) [override], [virtual]
```

virtual public methods

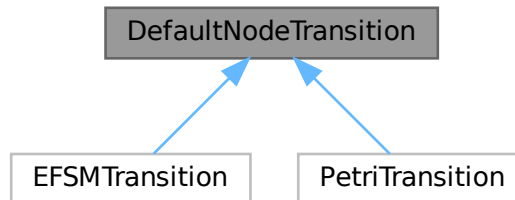
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- `plugins/components/ModalModel/DefaultNode.h`
- `plugins/components/ModalModel/DefaultNode.cpp`

10.150 DefaultNodeTransition Class Reference

Inheritance diagram for DefaultNodeTransition:



Public Types

- enum class **TransitionKind** : int { **DETERMINISTIC** = 0 , **PROBABILISTIC** = 1 }

Public Member Functions

- **DefaultNodeTransition** ([DefaultNode](#) *source, [DefaultNode](#) *destination, std::string name="")
- void **setSource** ([DefaultNode](#) *source)
- [DefaultNode](#) * **getSource** () const
- void **setDestination** ([DefaultNode](#) *destination)
- [DefaultNode](#) * **getDestination** () const
- void **setName** (std::string name)
- std::string **getName** () const
- void **setGuardExpression** (std::string guardExpression)
- std::string **getGuardExpression** () const
- void **setOutputExpression** (std::string outputExpression)
- std::string **getOutputExpression** () const
- void **setInputEvent** (std::string inputEvent)
- std::string **getInputEvent** () const
- void **setPriority** (unsigned int priority)
- unsigned int **getPriority** () const
- void **setProbability** (double probability)
- double **getProbability** () const
- void **setTransitionKind** (TransitionKind transitionKind)
- TransitionKind **getTransitionKind** () const
- virtual bool **canFire** ([Model](#) *model, [Entity](#) *entity) const
- virtual void **execute** ([Model](#) *model, [Entity](#) *entity) const

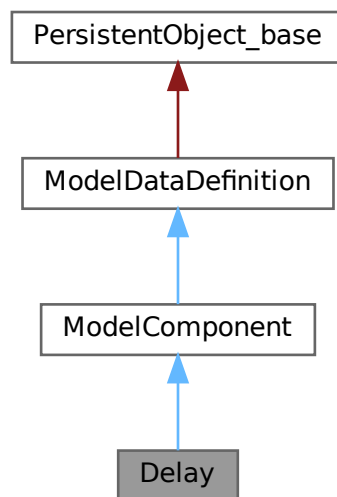
The documentation for this class was generated from the following files:

- plugins/components/ModalModel/DefaultNode.h
- plugins/components/ModalModel/DefaultNode.cpp

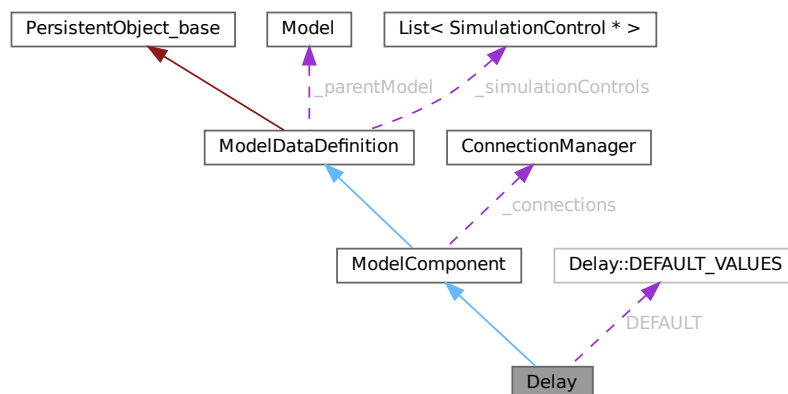
10.151 Delay Class Reference

```
#include <plugins/components/DiscreteProcessing/Delay.h>
```

Inheritance diagram for Delay:



Collaboration diagram for Delay:



Classes

- struct `DEFAULT_VALUES`

Public Member Functions

- **Delay** ([Model](#) *model, std::string name="")
- void **setDelayExpression** (std::string _delayExpression, [Util::TimeUnit](#) _delayTimeUnit=[Util::TimeUnit](#)↔::unknown)
- std::string **delayExpression** () const
- void **setDelay** (double delay)
- double **delay** () const
- void **setDelayTimeUnit** ([Util::TimeUnit](#) _delayTimeUnit)
- [Util::TimeUnit](#) **delayTimeUnit** () const
- void **setAllocation** ([Util::AllocationType](#) allocation)
- [Util::AllocationType](#) **getAllocation** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↔IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- [Util::identification](#) **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

- Returns the user-visible label of this instance.*

 - `const std::string & getClassname () const`

Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`

Returns the parent model that owns this definition.
- `bool isReportStatistics () const`

Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`

Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`

Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.
- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.
- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`

Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`

Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`

Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Public Attributes

- const struct [Delay::DEFAULT_VALUES](#) **DEFAULT**

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createInternalStatisticReporters](#) () override
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _initBetweenReplications ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `virtual void _createEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
- *legacy helper methods used by the granular create* hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- *method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
- *not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
- *Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Friends

- class **DelayProbe**

Additional Inherited Members**Protected Attributes inherited from [ModelComponent](#)**

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.151.1 Detailed Description

[Delay](#) module DESCRIPTION The [Delay](#) module delays an entity by a specified amount of time. When an entity arrives at a [Delay](#) module, the time delay expression is evaluated and the entity remains in the module for the resulting time period. The time is then allocated to the entity's value-added, non-value added, transfer, wait, or other time. Associated costs are calculated and allocated as well. TYPICAL USES Processing a check at a bank Performing a setup on a machine Transferring a document to another department PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Allocation Type of category to which the entity's incurred delay time and cost will be added. [Delay](#) Time Determines the value of the delay for the entity. Units Time units used for the delay time.

10.151.2 Member Function Documentation**10.151.2.1 [_check\(\)](#)**

```
bool Delay::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.151.2.2 [_createAttachedAttributes\(\)](#)

```
void Delay::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.151.2.3 _createInternalStatisticReporters()

```
void Delay::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.151.2.4 _loadInstance()

```
bool Delay::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.151.2.5 _onDispatchEvent()

```
void Delay::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.151.2.6 _saveInstance()

```
void Delay::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.151.2.7 show()

```
std::string Delay::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

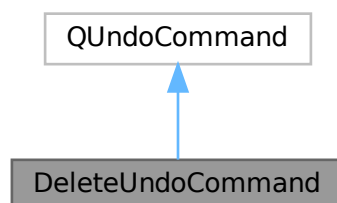
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

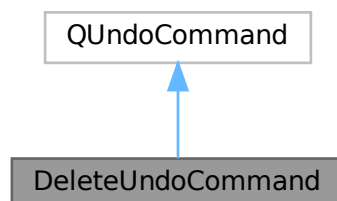
- plugins/components/DiscreteProcessing/Delay.h
- plugins/components/DiscreteProcessing/Delay.cpp

10.152 DeleteUndoCommand Class Reference

Inheritance diagram for DeleteUndoCommand:



Collaboration diagram for DeleteUndoCommand:



Public Member Functions

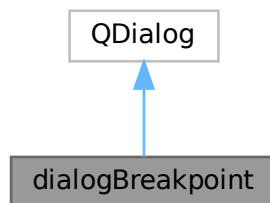
- **DeleteUndoCommand** (QList< QGraphicsItem * > items, [ModelGraphicsScene](#) *scene, QUndoCommand *parent=nullptr)
- void **undo** () override
- void **redo** () override

The documentation for this class was generated from the following files:

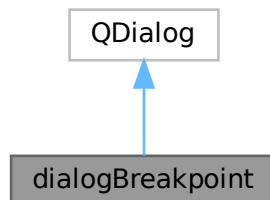
- applications/gui/genesys/actions/DeleteUndoCommand.h
- applications/gui/genesys/actions/DeleteUndoCommand.cpp

10.153 dialogBreakpoint Class Reference

Inheritance diagram for dialogBreakpoint:



Collaboration diagram for dialogBreakpoint:



Classes

- class [MVCTestResult](#)

Public Member Functions

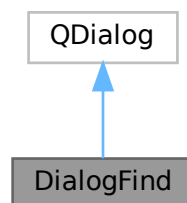
- void **setMVCModel** ([Simulator](#) *simulator)
- [dialogBreakpoint::MVCResult](#) * **getMVCResult** ()

The documentation for this class was generated from the following files:

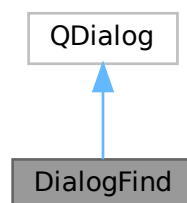
- applications/gui/genesys/dialogs/dialogBreakpoint.h
- applications/gui/genesys/dialogs/dialogBreakpoint.cpp

10.154 DialogFind Class Reference

Inheritance diagram for DialogFind:



Collaboration diagram for DialogFind:



Public Slots

- void **findClicked** ()
- void **nextClicked** ()
- void **previousClicked** ()

Public Member Functions

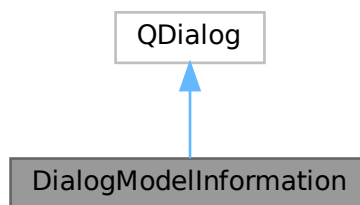
- **DialogFind** (QWidget *parent=nullptr, [ModelGraphicsScene](#) *scene=nullptr)
- QString **getFindText** ()

The documentation for this class was generated from the following files:

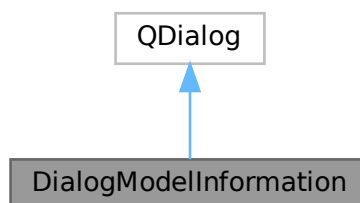
- applications/gui/genesys/dialogs/DialogFind.h
- applications/gui/genesys/dialogs/DialogFind.cpp

10.155 DialogModelInformation Class Reference

Inheritance diagram for DialogModelInformation:



Collaboration diagram for DialogModelInformation:



Public Slots

- void **accept** () override
- void **reject** () override

Public Member Functions

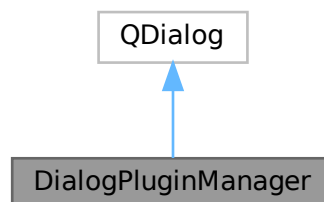
- **DialogModelInformation** (QWidget *parent=nullptr)
- void **setModelInfo** ([ModelInfo](#) *modelInfo)

The documentation for this class was generated from the following files:

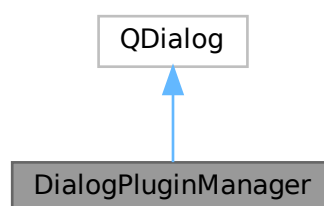
- applications/gui/genesys/dialogs/Dialogmodelinformation.h
- applications/gui/genesys/dialogs/Dialogmodelinformation.cpp

10.156 DialogPluginManager Class Reference

Inheritance diagram for DialogPluginManager:



Collaboration diagram for DialogPluginManager:

**Public Member Functions**

- [DialogPluginManager](#) (QWidget *parent=nullptr)
Builds the plugin manager dialog and prepares the loaded/problem tables.
- void **setSimulator** ([Simulator](#) *simulator)
- void **setPluginCatalogRefreshCallback** (std::function< void()> refreshCallback)
- void **showProblemPluginsTab** ()
Selects the problems tab when the dialog is opened for startup diagnostics.

10.156.1 Constructor & Destructor Documentation

10.156.1.1 DialogPluginManager()

```
DialogPluginManager::DialogPluginManager (
    QWidget * parent = nullptr ) [explicit]
```

Builds the plugin manager dialog and prepares the loaded/problem tables.

The dialog is responsible for plugin insertion/removal, dependency diagnostics, and recovery of blocked plugins that require system-package installation before they can load.

The documentation for this class was generated from the following files:

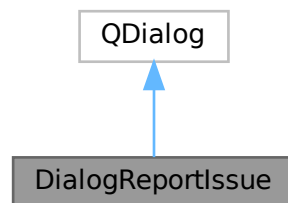
- applications/gui/genesys/dialogs/dialogpluginmanager.h
- applications/gui/genesys/dialogs/dialogpluginmanager.cpp

10.157 DialogReportIssue Class Reference

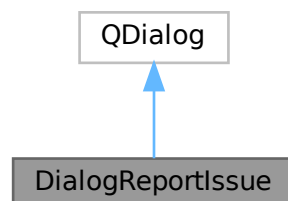
Modal form used to collect a user-facing issue report before relay submission.

```
#include <applications/gui/genesys/dialogs/DialogReportIssue.h>
```

Inheritance diagram for DialogReportIssue:



Collaboration diagram for DialogReportIssue:



Public Member Functions

- **DialogReportIssue** (QWidget *parent=nullptr)
- **IssueReportDraft draft** () const
Returns a normalized draft from the current form fields.
- void **setRelayConfigurationStatus** (const QString &endpoint, bool configured, const QString &detail)
Updates relay availability text and enables/disables the send action accordingly.

10.157.1 Detailed Description

Modal form used to collect a user-facing issue report before relay submission.

Responsibilities:

- collect category, title, description, and optional diagnostic consent flags;
- expose a validated **IssueReportDraft** for the controller layer;
- reflect whether the report relay is configured and available.

Boundaries:

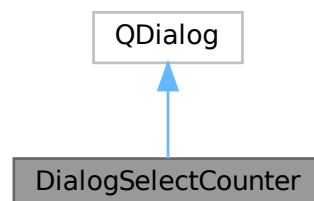
- it does not perform HTTP transport;
- it does not talk to GitHub directly;
- it does not inspect application runtime state on its own.

The documentation for this class was generated from the following files:

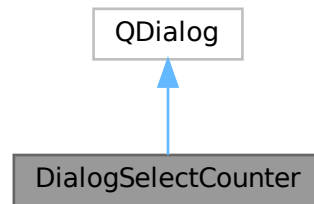
- applications/gui/genesys/dialogs/DialogReportIssue.h
- applications/gui/genesys/dialogs/DialogReportIssue.cpp

10.158 DialogSelectCounter Class Reference

Inheritance diagram for DialogSelectCounter:



Collaboration diagram for DialogSelectCounter:



Public Member Functions

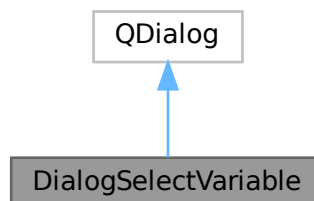
- **DialogSelectCounter** (QWidget *parent=nullptr)
- `Counter * selectedIndex ()` const
- void **setCounterItems** (QList< Counter * > *counters, Counter *counter)

The documentation for this class was generated from the following files:

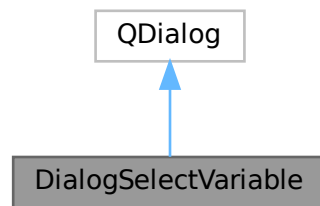
- applications/gui/genesys/dialogs/DialogSelectCounter.h
- applications/gui/genesys/dialogs/DialogSelectCounter.cpp

10.159 DialogSelectVariable Class Reference

Inheritance diagram for DialogSelectVariable:



Collaboration diagram for DialogSelectVariable:



Public Member Functions

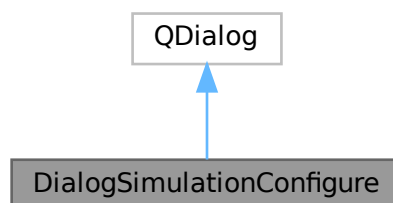
- **DialogSelectVariable** (QWidget *parent=nullptr)
- [ModelDataDefinition](#) * **selectedIndex** () const
- void **setVariableItems** (QList< [ModelDataDefinition](#) * > *definitions, [ModelDataDefinition](#) *definition)

The documentation for this class was generated from the following files:

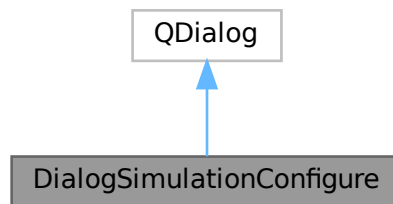
- applications/gui/genesys/dialogs/DialogSelectVariable.h
- applications/gui/genesys/dialogs/DialogSelectVariable.cpp

10.160 DialogSimulationConfigure Class Reference

Inheritance diagram for DialogSimulationConfigure:



Collaboration diagram for DialogSimulationConfigure:



Public Slots

- void [accept](#) () override
Applies the edited configuration back to the model only when the dialog is accepted.
- void [reject](#) () override
Asks for confirmation before discarding unsaved changes.

Public Member Functions

- [DialogSimulationConfigure](#) (QWidget *parent=nullptr)
Builds the simulation-configuration dialog and seeds the static combobox content.
- void **setModelSimulation** ([ModelSimulation](#) *modelSimulation)
- void **setExperimentManager** ([ExperimentManager](#) *experimentManager)
- void **setParallelizationSettings** (bool *enabled, int *localThreads, int *batchSize)

10.160.1 Constructor & Destructor Documentation

10.160.1.1 DialogSimulationConfigure()

```
DialogSimulationConfigure::DialogSimulationConfigure (
    QWidget * parent = nullptr ) [explicit]
```

Builds the simulation-configuration dialog and seeds the static combobox content.

The dialog currently owns a placeholder entry for distributed parallelization while the local simulation and parallelization settings are still exposed as editable GUI state.

10.160.2 Member Function Documentation

10.160.2.1 accept

```
void DialogSimulationConfigure::accept ( ) [override], [slot]
```

Applies the edited configuration back to the model only when the dialog is accepted.

This keeps the original values intact until the user confirms the dialog, while still supporting partial ownership of the parallelization fields when no model simulation is currently attached.

10.160.2.2 reject

```
void DialogSimulationConfigure::reject ( ) [override], [slot]
```

Asks for confirmation before discarding unsaved changes.

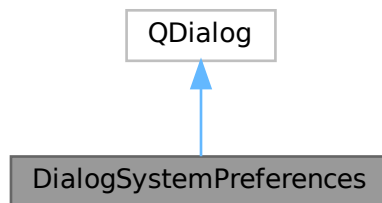
The dialog checks the current widget state against the snapshot captured at load time so the user does not lose pending configuration edits accidentally.

The documentation for this class was generated from the following files:

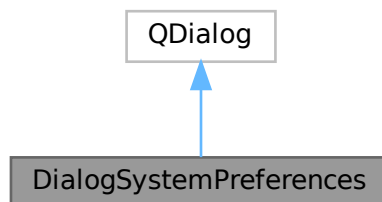
- applications/gui/genesys/dialogs/dialogsimulationconfigure.h
- applications/gui/genesys/dialogs/dialogsimulationconfigure.cpp

10.161 DialogSystemPreferences Class Reference

Inheritance diagram for DialogSystemPreferences:



Collaboration diagram for DialogSystemPreferences:



Public Member Functions

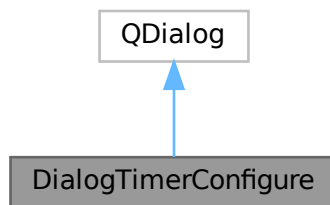
- **DialogSystemPreferences** ([Simulator](#) *simulator=nullptr, QWidget *parent=nullptr)

The documentation for this class was generated from the following files:

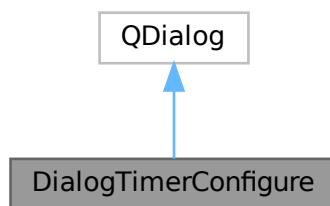
- applications/gui/genesys/dialogs/dialogsystempreferences.h
- applications/gui/genesys/dialogs/dialogsystempreferences.cpp

10.162 DialogTimerConfigure Class Reference

Inheritance diagram for DialogTimerConfigure:



Collaboration diagram for DialogTimerConfigure:



Public Member Functions

- **DialogTimerConfigure** (QWidget *parent=nullptr)
- void **setAnimationTimer** ([AnimationTimer](#) *animationTimer)
- void **updateAnimationTimer** ()

The documentation for this class was generated from the following files:

- applications/gui/genesys/dialogs/DialogTimerConfigure.h
- applications/gui/genesys/dialogs/DialogTimerConfigure.cpp

10.163 DialogUtilityController Class Reference

Controller that orchestrates auxiliary dialogs and utility actions.

```
#include <applications/gui/genesys/controllers/DialogUtilityController.h>
```

Public Member Functions

- **DialogUtilityController** ([MainWindow](#) *ownerWidget, [Simulator](#) *simulator, [Ui::MainWindow](#) *ui, [ModelGraphicsView](#) *graphicsView, std::function< void()> showMessageNotImplemented, std::function< void(bool)> actualizeDebugBreakpoints, std::function< bool()> createModelImage, std::function< void()> actualizeActions, std::function< void()> actualizeTabPanes, std::function< void()> reloadPluginCatalog, std::function< [ModelGraphicsScene](#) *()> currentScene, double &optimizerPrecision, unsigned int &optimizerMaxSteps, bool ¶llelizationEnabled, int ¶llelizationThreads, int ¶llelizationBatchSize, QString &lastDataAnalyzerPath)
Builds the utility controller that backs legacy [MainWindow](#) dialog wrappers.
- void **onActionAboutAboutTriggered** ()
Opens the About dialog while preserving legacy action routing.
- void **onActionAboutLicenceTriggered** ()
Opens the license dialog from the compatibility façade action slot.
- void **onActionAboutGetInvolvedTriggered** ()
Opens the get-involved/help contribution dialog flow.
- void **onActionAboutReportIssueTriggered** ()
Opens the report-issue workflow and submits it through the configured relay.
- void **onActionEditFindTriggered** ()
Runs find workflow for the currently focused text editor pane.
- void **onActionEditReplaceTriggered** ()
Runs replace workflow for the currently focused text editor pane.
- void **onActionToolsParserGrammarCheckerTriggered** ()
Opens parser grammar checker utility without altering model state.
- void **onActionToolsOptimizerTriggered** ()
Opens optimization settings dialog and stores lightweight preferences.
- void **onActionToolsAIAssistantTriggered** ()
Opens the AI Assistant panel for LLM-driven simulation model generation.
- void **onActionToolsDataAnalyzerTriggered** ()
Launches the Data Analyzer workstation using persisted last-path compatibility state.
- void **onActionViewConfigureTriggered** ()
Opens view configuration dialog and applies delegated scene/UI refreshes.
- void **onActionSimulatorPreferencesTriggered** ()
Opens simulator preferences utility dialog.
- void **onActionSimulatorsPluginManagerTriggered** (bool showProblemPlugins=false)
Opens plugin manager dialog from the compatibility action surface.
- void **onPushButtonBreakpointInsertClicked** ()
Inserts a breakpoint entry based on current scene selection.
- void **onPushButtonBreakpointRemoveClicked** ()
Removes selected breakpoint entries and refreshes debug breakpoint pane.
- void **onPushButtonExportClicked** ()
Exports current debug/trace context using legacy push-button route.
- void **onActionParallelizationTriggered** ()
Opens parallelization configuration and persists run-preference flags.

10.163.1 Detailed Description

Controller that orchestrates auxiliary dialogs and utility actions.

This Phase-11 controller gathers non-core menu/toolbar actions that were still in [MainWindow](#), such as About dialogs, breakpoint utilities, optimization/preferences dialogs, data analyzer launch flow, and parallelization toggles.

Responsibilities:

- execute dialog-driven utility actions delegated from [MainWindow](#) wrappers;
- access shared UI/simulator state through narrow injected callbacks/references;
- keep compatibility behavior for debug breakpoints and auxiliary tooling actions.

Boundaries:

- it does not own simulator/model lifecycle;
- it does not manage scene command pipeline or property editor internals;
- it acts as a controller-level compatibility wrapper, not as a domain service.

10.163.2 Member Function Documentation

10.163.2.1 onActionAboutReportIssueTriggered()

```
void DialogUtilityController::onActionAboutReportIssueTriggered ( )
```

Opens the report-issue workflow and submits it through the configured relay.

Opens the GUI issue-report flow, captures runtime context, and submits the payload to the configured relay.

The report draft can include the current model text, console tail, simulation and report logs, the latest process log, and an optional screenshot. The submission result is then surfaced back to the user, including a browser link when the relay returns a created GitHub issue URL.

10.163.2.2 onActionEditReplaceTriggered()

```
void DialogUtilityController::onActionEditReplaceTriggered ( )
```

Runs replace workflow for the currently focused text editor pane.

Opens the editor replacement dialog and preserves the current find/replace workflow.

The dialog keeps the last search terms, supports case-sensitive matching, wraps the search to the start of the document, and performs single or bulk replacement directly on the code editor without detaching the current selection state.

10.163.2.3 onActionToolsParserGrammarCheckerTriggered()

```
void DialogUtilityController::onActionToolsParserGrammarCheckerTriggered ( )
```

Opens parser grammar checker utility without altering model state.

Builds the parser diagnostics window with grammar overview, DOT export, catalog, and console tabs.

The dialog combines a collapsible grammar map, a DOT text editor, a searchable function catalog, a parser console, and a lightweight model symbol browser so the user can inspect and exercise the expression language from one place.

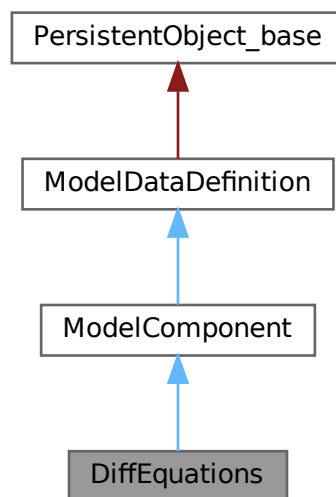
The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/DialogUtilityController.h
- applications/gui/genesys/controllers/DialogUtilityController.cpp

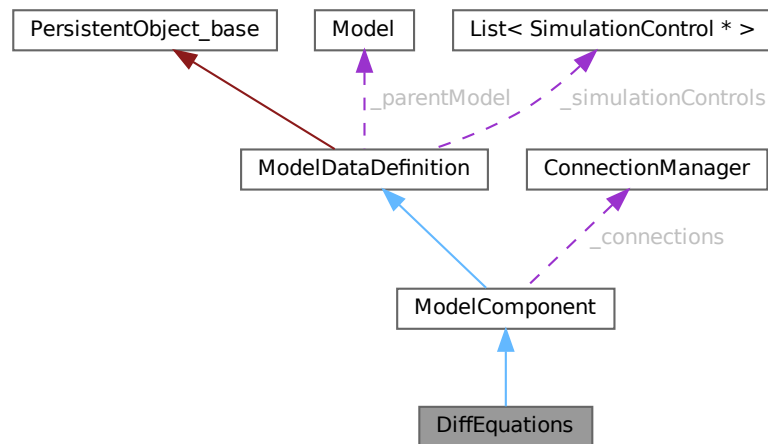
10.164 DiffEquations Class Reference

```
#include <plugins/components/Continuous/DiffEquations.h>
```

Inheritance diagram for DiffEquations:



Collaboration diagram for DiffEquations:



Public Member Functions

- **DiffEquations** ([Model](#) *model, std::string name="")
constructors
- [List](#)< std::string > * **getEquations** () const
new public user methods for this component
- [List](#)< std::string > * **getStateVariableNames** () const
- double **getprecision** () const
- void **setPrecision** (double newPrecision)
- double **getmaxSteps** () const
- void **setMaxSteps** (double newMaxSteps)
- double **getfinalTime** () const
- void **setFinalTime** (double newFinalTime)
- [List](#)< double > * **getvaluesOnFinalTime** () const
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)

- *Updates the free-form description shown by the GUI.*
- `std::string getDescription () const`
Returns the free-form description associated with this component.
- `void remove (ModelComponent *comp)`
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification getId () const
Returns the unique identifier of this instance.
- void setName (const std::string &name)
Updates the logical name of this instance.
- const std::string & getName () const
Returns the logical name of this instance.
- void setLabel (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & getLabel () const
Returns the user-visible label of this instance.
- const std::string & getClassname () const
Returns the runtime class name used by the plugin system.
- **Model** * getParentModel () const
Returns the parent model that owns this definition.
- bool isReportStatistics () const
Indicates whether this data definition contributes report statistics.
- void setReportStatistics (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * getInternalData (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * getInternalData () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * getAttachedData () const
Returns the full map of attached data definitions.
- bool hasChanged () const
Indicates whether this instance has been modified since the last reset.
- void setHasChanged (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int getLevel () const
Returns the level in the model hierarchy where this definition lives.
- void setModelLevel (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< **SimulationControl** * > * getSimulationControls () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level getTraceLevelSpecific () const
Returns the trace level that applies specifically to this definition.
- void setTraceLevelSpecific (TraceManager::Level level)

Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)

Defines the specific trace level and optionally enables it immediately.

- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.

- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()

static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)

static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)

- static bool [Check](#) ([ModelComponent](#) *component)

Validates a component instance.

- static void [CreateInternalData](#) ([ModelComponent](#) *component)

Creates internal data required by a component instance.

- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)

Serializes a component instance into a persistence record.

- static void [DispatchEvent](#) ([Event](#) *event)

Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)

Loads a model data definition instance from serialized fields.

- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Creates a new instance of a data definition using the plugin registry.

- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)

Serializes a data definition instance into a persistence record.

- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)

Validates a data definition instance.

- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)

Resets simulation-time state between replications.

- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)

Creates internal and attached data required by this instance.

- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)

Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual bool `_check` (std::string &errorMessage) override
virtual protected methods that could be overridden by derived classes, if needed
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()

- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.164.1 Detailed Description

This component ...

10.164.2 Member Function Documentation

10.164.2.1 _check()

```
bool DiffEquations::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by ModelChecker during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model too run without errors in this component

Reimplemented from [ModelDataDefinition](#).

10.164.2.2 `_createEditableDataDefinitions()`

```
void DiffEquations::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelData. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

This method is called by [ModelSimulation](#) when initiating the replication. The model should set all value for a new replication (Ex: setting back to 0 any internal counter, clearing lists, etc).

This method is not used yet. It should be usefull for new UIs

Reimplemented from [ModelDataDefinition](#).

10.164.2.3 `_loadInstance()`

```
bool DiffEquations::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.164.2.4 `_onDispatchEvent()`

```
void DiffEquations::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

This method is only for ModelComponents, not ModelDataElements.

Implements [ModelComponent](#).

10.164.2.5 `_saveInstance()`

```
void DiffEquations::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.164.2.6 `getEquations()`

```
List< std::string > * DiffEquations::getEquations ( ) const
```

new public user methods for this component

equations should be in terms of state variable names and t

10.164.2.7 `show()`

```
std::string DiffEquations::show ( ) [override], [virtual]
```

virtual public methods

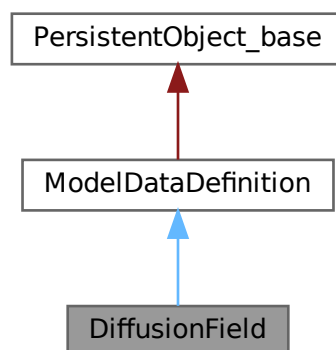
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

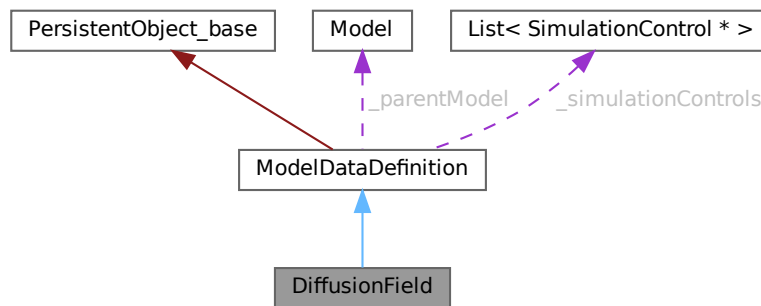
- `plugins/components/Continuous/DiffEquations.h`
- `plugins/components/Continuous/DiffEquations.cpp`

10.165 DiffusionField Class Reference

Inheritance diagram for DiffusionField:



Collaboration diagram for DiffusionField:



Public Member Functions

- **DiffusionField** ([Model](#) *model, std::string name="")
- void **setDimensions** (unsigned int dimensions)
- unsigned int **getDimensions** () const
- void **setPointsPerDimension** (unsigned int pointsPerDimension)
- unsigned int **getPointsPerDimension** () const
- void **setDomainLength** (double domainLength)
- double **getDomainLength** () const
- void **setDiffusionCoefficient** (double diffusionCoefficient)
- double **getDiffusionCoefficient** () const
- void **setBoundaryCondition** (std::string boundaryCondition)
- std::string **getBoundaryCondition** () const
- void **setInitialCondition** (std::string initialCondition)
- std::string **getInitialCondition** () const
- void **setInitialParameter** (double initialParameter)
- double **getInitialParameter** () const
- void **setStartTime** (double startTime)
- double **getStartTime** () const
- void **setStopTime** (double stopTime)
- double **getStopTime** () const
- void **setStepSize** (double stepSize)
- double **getStepSize** () const
- void **setCurrentTime** (double currentTime)
- double **getCurrentTime** () const
- void **setAutoSchedule** (bool autoSchedule)
- bool **getAutoSchedule** () const
- void **setOdeSolver** (std::string odeSolver)
- std::string **getOdeSolver** () const
- void **setLastStatus** (std::string lastStatus)
- std::string **getLastStatus** () const
- void **setLastErrorMessage** (std::string lastErrorMessage)
- std::string **getLastErrorMessage** () const
- double **getTotalMass** () const
- double **getMaxValue** () const
- double **getL2Norm** () const

- `const std::vector< double > & getField () const`
- `double getFieldValue (const std::vector< unsigned int > &multiIndex) const`
Field value at a multi-index (size must equal Dimensions); 0 if invalid.
- `bool simulate (std::string &errorMessage)`
- `bool simulate (double startTime, double stopTime, double stepSize, std::string &errorMessage)`
- `bool advanceOneStep (std::string &errorMessage)`
- `std::string showFieldAsAscii () const`
- `virtual std::string show () override`
Returns a textual representation of this data definition.

Public Member Functions inherited from `ModelDataDefinition`

- `ModelDataDefinition (Model *model, std::string datadefinitionType, std::string name="", bool insert↵ IntoModel=true)`
Creates a model data definition and optionally inserts it into the model.
- `virtual ~ModelDataDefinition ()`
Releases resources owned by this data definition.
- `Util::identification getId () const`
Returns the unique identifier of this instance.
- `void setName (const std::string &name)`
Updates the logical name of this instance.
- `const std::string & getName () const`
Returns the logical name of this instance.
- `void setLabel (const std::string &label)`
Updates the user-visible label of this instance.
- `const std::string & getLabel () const`
Returns the user-visible label of this instance.
- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`

- Returns the simulation controls owned by this data definition.*
- `TraceManager::Level` [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static `ModelDataDefinition` * [LoadInstance](#) (`Model` *model, `PersistenceRecord` *fields)
- static `PluginInformation` * [GetPluginInformation](#) ()
- static `ModelDataDefinition` * [NewInstance](#) (`Model` *model, std::string name="")

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * [LoadInstance](#) (`Model` *model, `PersistenceRecord` *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * [NewInstance](#) (`Model` *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) (`PersistenceRecord` *fields, `ModelDataDefinition` *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) (`ModelDataDefinition` *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) (`ModelDataDefinition` *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) (`ModelDataDefinition` *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) (`PersistenceRecord` *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) (`PersistenceRecord` *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createInternalStatisticReporters ()`
- `virtual void _createAttachedAttributes ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `virtual void _createEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
- *legacy helper methods used by the granular create* hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- *method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.165.1 Member Function Documentation

10.165.1.1 `_check()`

```
bool DiffusionField::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.165.1.2 `_initBetweenReplications()`

```
void DiffusionField::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.165.1.3 `_loadInstance()`

```
bool DiffusionField::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.165.1.4 `_saveInstance()`

```
void DiffusionField::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.165.1.5 show()

```
std::string DiffusionField::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

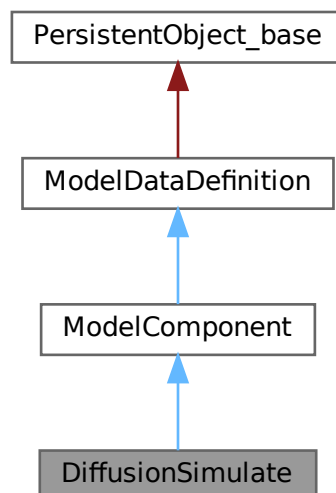
- plugins/data/Continuous/DiffusionField.h
- plugins/data/Continuous/DiffusionField.cpp

10.166 DiffusionSimulate Class Reference

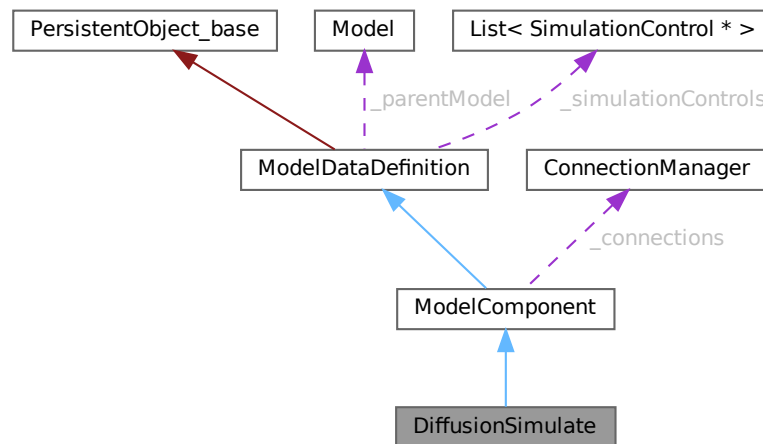
Component that triggers a [DiffusionField](#) solve when entities arrive.

```
#include <plugins/components/Continuous/DiffusionSimulate.h>
```

Inheritance diagram for DiffusionSimulate:



Collaboration diagram for DiffusionSimulate:



Public Member Functions

- **DiffusionSimulate** (**Model** *model, std::string name="")
- void **setDiffusionField** (**DiffusionField** *diffusionField)
- **DiffusionField** * **getDiffusionField** () const
- void **setUseFieldTimeWindow** (bool useFieldTimeWindow)
- bool **getUseFieldTimeWindow** () const
- void **setStartTime** (double startTime)
- double **getStartTime** () const
- void **setStopTime** (double stopTime)
- double **getStopTime** () const
- void **setStepSize** (double stepSize)
- double **getStepSize** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.

- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) (ModelComponent *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.

- `TraceManager::Level` `getTraceLevelSpecific ()` const
Returns the trace level that applies specifically to this definition.
- `void` `setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void` `defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool` `isTraceLevelSpecificEnabled ()` const
Indicates whether the specific trace level is enabled.
- `void` `setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation *` `GetPluginInformation ()`
- `static ModelComponent *` `LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition *` `NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `ModelComponent`

- `static ModelComponent *` `LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static bool` `Check (ModelComponent *component)`
Validates a component instance.
- `static void` `CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- `static void` `SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- `static void` `DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- `static ModelDataDefinition *` `LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition *` `NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void` `SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool` `Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- `static void` `InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- `static void` `CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- `static bool` `CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()

- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.166.1 Detailed Description

Component that triggers a [DiffusionField](#) solve when entities arrive.

[DiffusionSimulate](#) delegates the continuous diffusion solve to a referenced [DiffusionField](#) (a [ModelDataDefinition](#) in plugins/data/Continuous). Each arriving entity runs the field either with its own configured time window or with component-provided start/stop/step values, then forwards the entity. This is the discrete->continuous bridge: a discrete-event component drives the continuous PDE subsystem. For event-clock-synchronized advance, enable AutoSchedule on the [DiffusionField](#) instead, so it steps once per kernel [InternalEvent](#).

10.166.2 Member Function Documentation

10.166.2.1 _check()

```
bool DiffusionSimulate::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.166.2.2 _createEditableDataDefinitions()

```
void DiffusionSimulate::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.166.2.3 _loadInstance()

```
bool DiffusionSimulate::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.166.2.4 _onDispatchEvent()

```
void DiffusionSimulate::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.166.2.5 _saveInstance()

```
void DiffusionSimulate::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.166.2.6 show()

```
std::string DiffusionSimulate::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

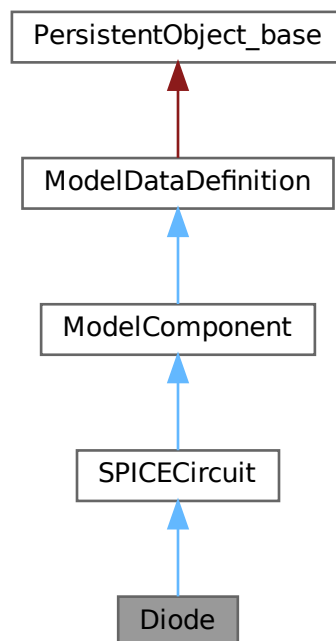
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

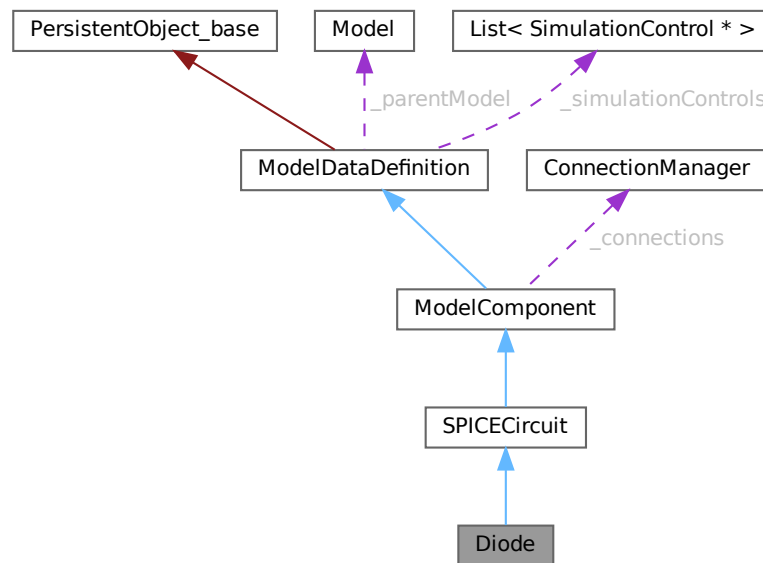
- plugins/components/Continuous/DiffusionSimulate.h
- plugins/components/Continuous/DiffusionSimulate.cpp

10.167 Diode Class Reference

Inheritance diagram for Diode:



Collaboration diagram for Diode:



Public Member Functions

- **Diode** (**Model** *model, std::string name="")
- void **Build** () override
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from **SPICECircuit**

- **SPICECircuit** (**Model** *model, std::string name="")
constructors
- void **setRunner** (**SPICERunner** *runner)
new public user methods for this components
- **SPICERunner** * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)

- Connects this component to another component.*

 - bool [hasBreakpointAt](#) ()
 - Indicates whether a simulation breakpoint is set on this component.*
 - void [setDescription](#) (std::string _description)
 - Updates the free-form description shown by the GUI.*
 - std::string [getDescription](#) () const
 - Returns the free-form description associated with this component.*
 - void [remove](#) ([ModelComponent](#) *comp)
 - Removes a connected component from this component.*

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~[ModelDataDefinition](#) ()
- Releases resources owned by this data definition.*
- Util::identification [getId](#) () const
- Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & [getName](#) () const
- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
- Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const
- Returns the runtime class name used by the plugin system.*
- [Model](#) * [getParentModel](#) () const
- Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const
- Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)
- Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
- Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
- Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
- Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const
- Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))
- Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const
- Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int _modelLevel)
- Sets the model nesting level for this definition.*
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const

- Returns the simulation controls owned by this data definition.*
- `TraceManager::Level` [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- `void` [setTraceLevelSpecific](#) (`TraceManager::Level` level)
Updates the trace level used by this definition.
- `void` [defineTraceLevelSpecific](#) (`TraceManager::Level` traceLevelspecific, `bool` traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- `bool` [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- `void` [setTraceLevelSpecificEnabled](#) (`bool` traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- `static` [PluginInformation](#) * [GetPluginInformation](#) ()
- `static` [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- `static` [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, `std::string` name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- `static` [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static` [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- `static` [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, `std::string` name="")
- `static` `std::vector< std::string >` [split](#) (`std::string` word, `char` split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from [ModelComponent](#)

- `static` [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static` `bool` [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- `static` `void` [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- `static` `void` [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- `static` `void` [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()

- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
 - legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * [_connections](#) = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification [_id](#)
- std::string [_typename](#)
- bool [_reportStatistics](#)
- bool [_hasChanged](#)
- unsigned int [_modelLevel](#) = 0
- [Model](#) * [_parentModel](#)
- [List](#)< [SimulationControl](#) * > * [_simulationControls](#) = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level [_traceLevelSpecific](#)
- bool [_traceLevelSpecificEnabled](#) = false

10.167.1 Member Function Documentation

10.167.1.1 [_loadInstance\(\)](#)

```
bool Diode::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.167.1.2 [_saveInstance\(\)](#)

```
void Diode::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.167.1.3 [Build\(\)](#)

```
void Diode::Build ( ) [override], [virtual]
```

Reimplemented from [SPICECircuit](#).

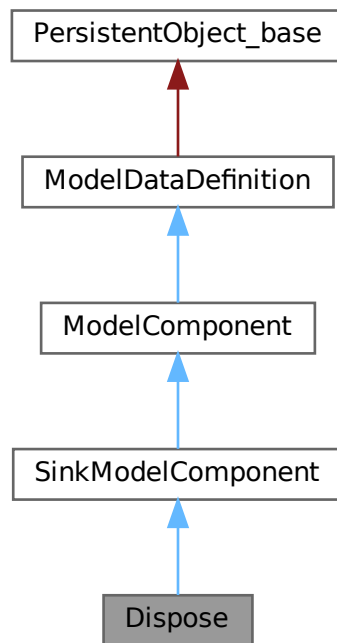
The documentation for this class was generated from the following files:

- plugins/components/ElectronicsSimulation/SPIECircuit.h
- plugins/components/ElectronicsSimulation/SPIECircuit.cpp

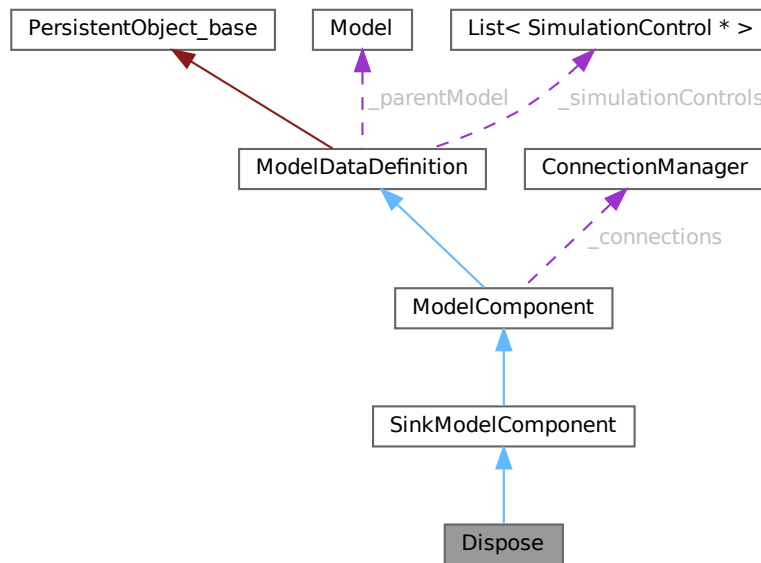
10.168 Dispose Class Reference

```
#include <plugins/components/Logic/Dispose.h>
```

Inheritance diagram for Dispose:



Collaboration diagram for Dispose:



Public Member Functions

- **Dispose** ([Model](#) *model, std::string name="")
 - virtual std::string **show** () override
- virtual public methods*

Public Member Functions inherited from [SinkModelComponent](#)

- **SinkModelComponent** ([Model](#) *model, std::string componentTypename, std::string name="")

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void **_initBetweenReplications** () override
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_createInternalStatisticReporters** () override
- virtual void **_createAttachedAttributes** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `virtual void _createEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
- legacy helper methods used by the granular create* hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
 - method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
 - Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.168.1 Detailed Description

[Dispose](#) module DESCRIPTION This module is intended as the ending point for entities in a simulation model. [Entity](#) statistics may be recorded before the entity is disposed of. TYPICAL USES Parts leaving the modeled facility The termination of a business process Customers departing from the store Prompt Description Name Unique module identifier displayed on the module shape. [Record Entity](#) Statistics Determines whether or not the incoming entity's statistics will be recorded. Statistics include value-added time, non-value-added time, wait time, transfer time, other time, total time, value-added cost, non-value-added cost, wait cost, transfer cost, other cost, and total cost.

10.168.2 Member Function Documentation

10.168.2.1 `_check()`

```
bool Dispose::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [SinkModelComponent](#).

10.168.2.2 `_createAttachedAttributes()`

```
void Dispose::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.168.2.3 `_createInternalStatisticReporters()`

```
void Dispose::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.168.2.4 _initBetweenReplications()

```
void Dispose::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.168.2.5 _loadInstance()

```
bool Dispose::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SinkModelComponent](#).

10.168.2.6 _onDispatchEvent()

```
void Dispose::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.168.2.7 _saveInstance()

```
void Dispose::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SinkModelComponent](#).

10.168.2.8 show()

```
std::string Dispose::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

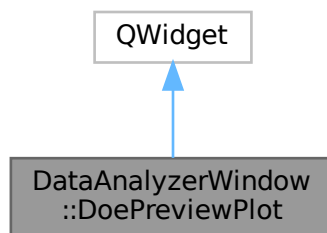
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

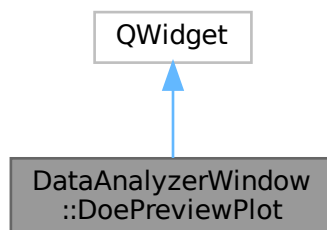
- plugins/components/Logic/Dispose.h
- plugins/components/Logic/Dispose.cpp

10.169 DataAnalyzerWindow::DoePreviewPlot Class Reference

Inheritance diagram for DataAnalyzerWindow::DoePreviewPlot:



Collaboration diagram for DataAnalyzerWindow::DoePreviewPlot:



Public Member Functions

- **DoePreviewPlot** (QWidget *parent=nullptr)
- void **setMode** (const QString &mode)

Protected Member Functions

- void **paintEvent** (QPaintEvent *) override

The documentation for this class was generated from the following file:

- applications/gui/genesys/tools/dataanalyzer/DataAnalyzerWindow.cpp

10.170 DrawingItem Struct Reference

Public Attributes

- QGraphicsItem * **myDrawingItem**
- QPointF **initialPosition**

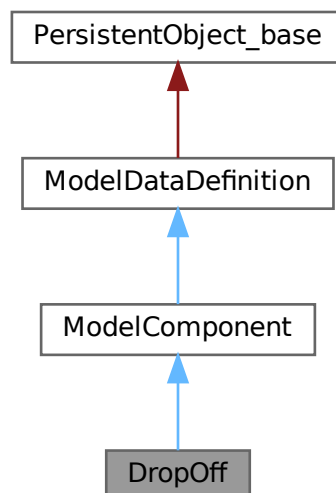
The documentation for this struct was generated from the following file:

- applications/gui/genesys/actions/AddUndoCommand.h

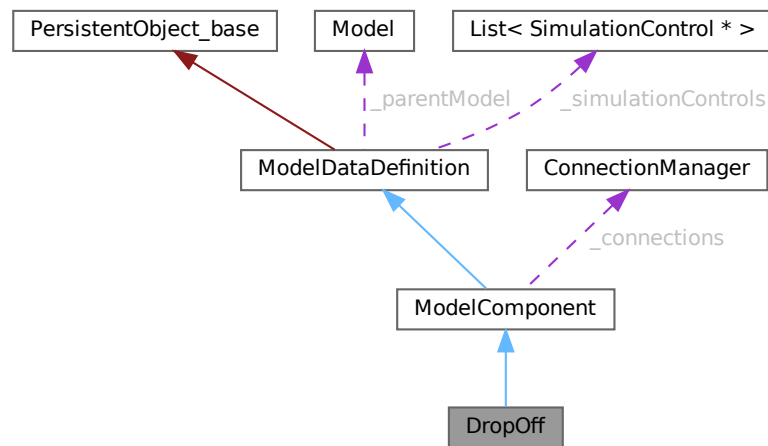
10.171 DropOff Class Reference

```
#include <plugins/components/Decisions/DropOff.h>
```

Inheritance diagram for DropOff:



Collaboration diagram for DropOff:



Public Member Functions

- **DropOff** ([Model](#) *model, std::string name="")
 - virtual std::string [show](#) () override
- virtual public methods*

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.171.1 Detailed Description

Dropoff module DESCRIPTION The Dropoff module removes a specified number of entities from the entity's group and sends them to another module, as specified by a graphical connection. Group user-defined attribute value and internal attributes may be given to the dropped-off entities based on a specified rule. TYPICAL USES Loading shelves with product Separating a form for use in various departments PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Quantity Number of entities that will be dropped off from an incoming representative grouped entity. Starting Rank Starting rank of the entities to be dropped off, based on the entities in the group. Member Attributes Method of determining how to assign the representative entity attribute values (other than costs/times) to the dropped-off original entities. [Attribute](#) Name Name of representative entity attribute(s) assigned to droppedoff original entities of the group

10.171.2 Member Function Documentation

10.171.2.1 `_check()`

```
bool DropOff::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.171.2.2 `_loadInstance()`

```
bool DropOff::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.171.2.3 `_onDispatchEvent()`

```
void DropOff::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.171.2.4 _saveInstance()

```
void DropOff::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.171.2.5 show()

```
std::string DropOff::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

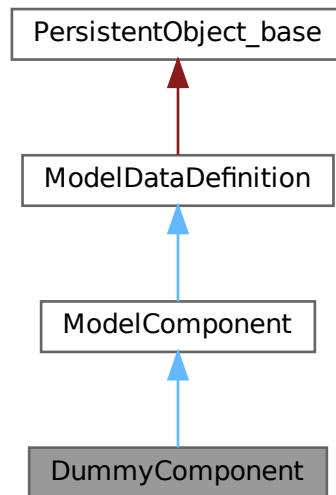
The documentation for this class was generated from the following files:

- plugins/components/Decisions/DropOff.h
- plugins/components/Decisions/DropOff.cpp

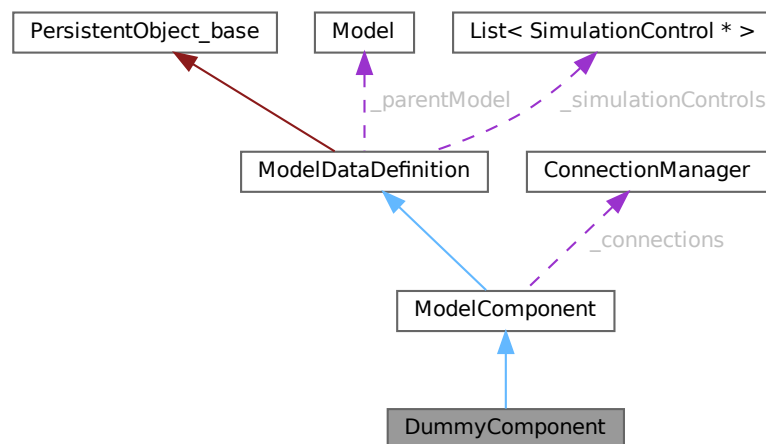
10.172 DummyComponent Class Reference

```
#include <plugins/components/Template/DummyComponent.h>
```

Inheritance diagram for DummyComponent:



Collaboration diagram for DummyComponent:



Public Member Functions

- `DummyComponent` (`Model` *model, std::string name="")

constructors

- virtual std::string **show** () override

new public user methods for this component

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")

constructors

- virtual ~**ModelComponent** ()

Releases component-owned resources and unregisters it from the model.

- **ConnectionManager** * **getConnectionManager** () const

new public user methods for this component

- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)

Connects this component to another component.

- bool **hasBreakpointAt** ()

Indicates whether a simulation breakpoint is set on this component.

- void **setDescription** (std::string _description)

Updates the free-form description shown by the GUI.

- std::string **getDescription** () const

Returns the free-form description associated with this component.

- void **remove** (**ModelComponent** *comp)

Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.

- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.

- Util::identification **getId** () const

Returns the unique identifier of this instance.

- void **setName** (const std::string &name)

Updates the logical name of this instance.

- const std::string & **getName** () const

Returns the logical name of this instance.

- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.

- const std::string & **getLabel** () const

Returns the user-visible label of this instance.

- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.

- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.

- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.

- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.

- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

- Looks up a named internal child data definition.*

 - `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.
- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.
- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↔ Enabled=true)`

Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`

Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`

Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`

static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`

static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static bool Check (ModelComponent *component)`

Validates a component instance.
- `static void CreateInternalData (ModelComponent *component)`

Creates internal data required by a component instance.
- `static void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`

Serializes a component instance into a persistence record.
- `static void DispatchEvent (Event *event)`

Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.172.1 Detailed Description

This component ...

10.172.2 Constructor & Destructor Documentation

10.172.2.1 DummyComponent()

```
DummyComponent::DummyComponent (
    Model * model,
    std::string name = "" )
```

constructors

Externalize function GetPluginInformation to be accessible through dynamic linked library.

10.172.3 Member Function Documentation

10.172.3.1 _loadInstance()

```
bool DummyComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.172.3.2 _onDispatchEvent()

```
void DummyComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

This method is only for ModelComponents, not ModelDataElements.

Implements [ModelComponent](#).

10.172.3.3 _saveInstance()

```
void DummyComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.172.3.4 show()

```
std::string DummyComponent::show ( ) [override], [virtual]
```

new public user methods for this component

virtual public methods

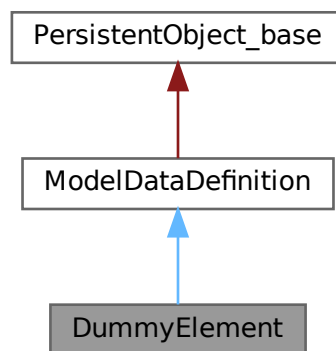
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

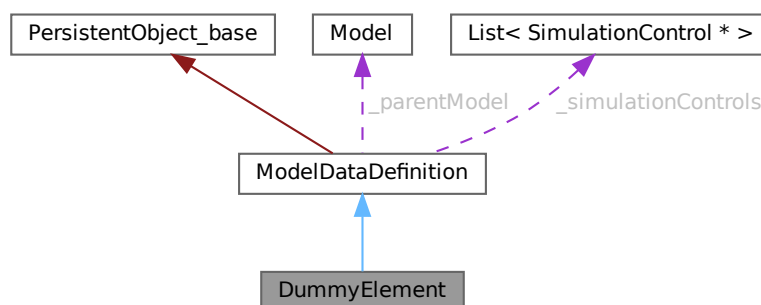
- plugins/components/Template/DummyComponent.h
- plugins/components/Template/DummyComponent.cpp

10.173 DummyElement Class Reference

Inheritance diagram for DummyElement:



Collaboration diagram for DummyElement:



Public Member Functions

- **DummyElement** ([Model](#) *model, std::string name="")

constructors

- void **setSomeString** (const std::string &someString)

new public user methods for this component

- std::string **getSomeString** () const
- void **setSomeUInt** (unsigned int someUInt)
- unsigned int **getSomeUInt** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insertIntoModel=true)

Creates a model data definition and optionally inserts it into the model.

- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.

- Util::identification **getId** () const

Returns the unique identifier of this instance.

- void **setName** (const std::string &name)

Updates the logical name of this instance.

- const std::string & **getName** () const

Returns the logical name of this instance.

- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.

- const std::string & **getLabel** () const

Returns the user-visible label of this instance.

- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.

- [Model](#) * **getParentModel** () const

Returns the parent model that owns this definition.

- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.

- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.

- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.

- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const

Returns the full map of internal child data definitions.

- std::map< std::string, [ModelDataDefinition](#) * > * **getAttachedData** () const

Returns the full map of attached data definitions.

- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.

- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.

- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.

- void **setModelLevel** (unsigned int _modelLevel)

- Sets the model nesting level for this definition.*
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from ModelDataDefinition

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- `virtual bool _loadInstance (PersistenceRecord *fields) override`
virtual protected method that must be overridden
- `virtual void _saveInstance (PersistenceRecord *fields, bool saveDefaultValues) override`
Saves object fields into a serialization structure.
- `virtual bool _check (std::string &errorMessage) override`
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.173.1 Member Function Documentation

10.173.1.1 [_check\(\)](#)

```
bool DummyElement::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.173.1.2 [_loadInstance\(\)](#)

```
bool DummyElement::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelDataDefinition](#).

10.173.1.3 [_saveInstance\(\)](#)

```
void DummyElement::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.173.1.4 show()

```
std::string DummyElement::show ( ) [override], [virtual]
```

virtual public methods

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/Template/DummyElement.h
- plugins/data/Template/DummyElement.cpp

10.174 EditCommandController Class Reference

Controller for edit/clipboard/grouping command orchestration in the scene.

```
#include <applications/gui/genesys/controllers/EditCommandController.h>
```

Public Member Functions

- **EditCommandController** ([Simulator](#) *simulator, [ModelGraphicsView](#) *graphicsView, std::function< void()> actualizeActions, bool *cut, QList< [GraphicalModelComponent](#) * > **gmcCopies, QList< [GraphicalConnection](#) * > **portsCopies, QList< QGraphicsItem * > **drawCopy, QList< QGraphicsItemGroup * > **groupCopy)
Creates the edit-command controller used by [MainWindow](#) compatibility slots.
- void **onActionEditUndoTriggered** () const
Delegates undo command handling without changing scene undo semantics.
- void **onActionEditRedoTriggered** () const
Delegates redo command handling without changing scene undo semantics.
- void **onActionEditCutTriggered** () const
Delegates cut command handling while preserving compatibility copy buffers.
- void **onActionEditCopyTriggered** () const
Delegates copy command handling while preserving compatibility copy buffers.
- void **onActionEditPasteTriggered** () const
Delegates paste command handling while preserving compatibility copy buffers.
- void **onActionEditDeleteTriggered** () const
Delegates delete command handling while preserving undo integration.
- void **onActionEditGroupTriggered** () const
Delegates group command handling through scene grouping APIs.
- void **onActionEditUngroupTriggered** () const
Delegates ungroup command handling through scene grouping APIs.
- void **onActionViewGroupTriggered** () const
Delegates view-group command handling through scene grouping APIs.
- void **onActionViewUngroupTriggered** () const
Delegates view-ungroup command handling through scene grouping APIs.
- void **saveItemForCopy** (QList< [GraphicalModelComponent](#) * > *gmcList, QList< [GraphicalConnection](#) * > *connList) const
Stores eligible selected items in compatibility clipboard structures.
- void **helpCopy** () const
Performs deep-copy preparation for subsequent paste command execution.

10.174.1 Detailed Description

Controller for edit/clipboard/grouping command orchestration in the scene.

This Phase-9 controller keeps [MainWindow](#) slots as compatibility wrappers while centralizing clipboard buffers, grouping operations, and undo-aware edit command dispatch against the current graphical scene.

Responsibilities:

- execute undo/redo/cut/copy/paste/delete/group/ungroup command flows;
- preserve legacy clipboard structures used by graphical components and drawings;
- trigger action refresh callbacks after command execution.

Boundaries:

- it does not own scene lifetime (resolved from injected graphics view);
- it does not persist clipboard state across sessions;
- it does not manage simulation events, model lifecycle, or property-editor synchronization.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/EditCommandController.h
- applications/gui/genesys/controllers/EditCommandController.cpp

10.175 EditorFactoryPrivate< Editor > Class Template Reference

Public Types

- typedef QList< Editor * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< Editor *, [QtProperty](#) * > **EditorToPropertyMap**

Public Member Functions

- Editor * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, Editor *e)
- void **slotEditorDestroyed** (QObject *object)

Public Attributes

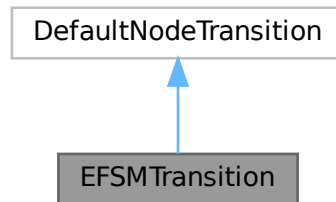
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

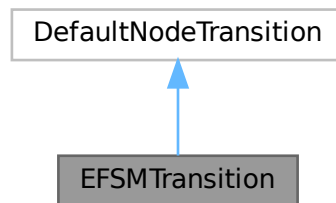
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.176 EFSMTransition Class Reference

Inheritance diagram for EFSMTransition:



Collaboration diagram for EFSMTransition:



Public Member Functions

- **EFSMTransition** ([DefaultNode](#) *source, [DefaultNode](#) *destination, std::string name="")
- void **setTriggerEvent** (std::string triggerEvent)
- std::string **getTriggerEvent** () const
- void **setProbabilityExpression** (std::string probabilityExpression)
- std::string **getProbabilityExpression** () const
- virtual bool **canFire** ([Model](#) *model, [Entity](#) *entity) const override
- virtual void **execute** ([Model](#) *model, [Entity](#) *entity) const override

Public Member Functions inherited from [DefaultNodeTransition](#)

- **DefaultNodeTransition** ([DefaultNode](#) *source, [DefaultNode](#) *destination, std::string name="")
- void **setSource** ([DefaultNode](#) *source)
- [DefaultNode](#) * **getSource** () const
- void **setDestination** ([DefaultNode](#) *destination)
- [DefaultNode](#) * **getDestination** () const

- void **setName** (std::string name)
- std::string **getName** () const
- void **setGuardExpression** (std::string guardExpression)
- std::string **getGuardExpression** () const
- void **setOutputExpression** (std::string outputExpression)
- std::string **getOutputExpression** () const
- void **setInputEvent** (std::string inputEvent)
- std::string **getInputEvent** () const
- void **setPriority** (unsigned int priority)
- unsigned int **getPriority** () const
- void **setProbability** (double probability)
- double **getProbability** () const
- void **setTransitionKind** (TransitionKind transitionKind)
- TransitionKind **getTransitionKind** () const

Additional Inherited Members

Public Types inherited from [DefaultNodeTransition](#)

- enum class **TransitionKind** : int { **DETERMINISTIC** = 0 , **PROBABILISTIC** = 1 }

10.176.1 Member Function Documentation

10.176.1.1 canFire()

```
bool EFSMTransition::canFire (
    Model * model,
    Entity * entity ) const [override], [virtual]
```

Reimplemented from [DefaultNodeTransition](#).

10.176.1.2 execute()

```
void EFSMTransition::execute (
    Model * model,
    Entity * entity ) const [override], [virtual]
```

Reimplemented from [DefaultNodeTransition](#).

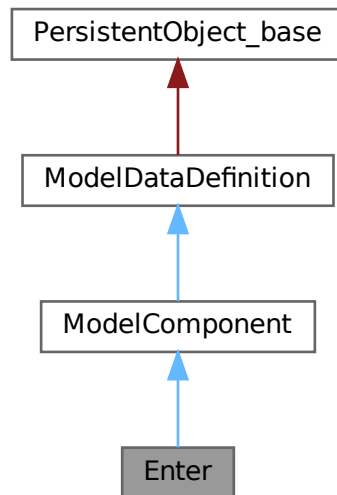
The documentation for this class was generated from the following files:

- plugins/components/ModalModel/DefaultTransitionExtensions.h
- plugins/components/ModalModel/DefaultTransitionExtensions.cpp

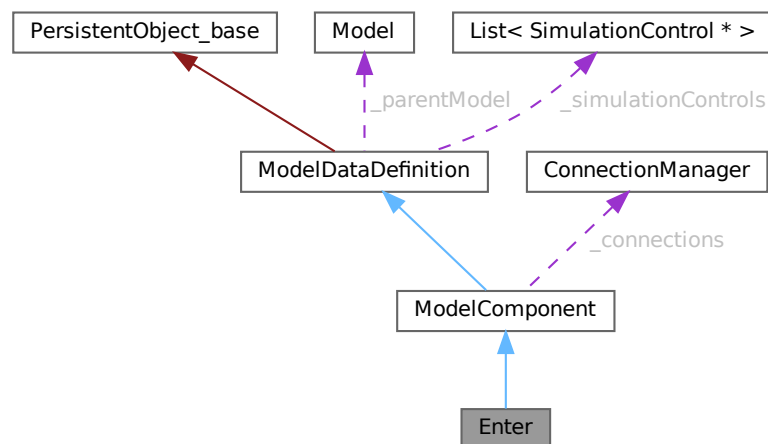
10.177 Enter Class Reference

```
#include <plugins/components/MaterialHandling/Enter.h>
```

Inheritance diagram for Enter:



Collaboration diagram for Enter:



Public Member Functions

- **Enter** ([Model](#) *model, std::string name="")

- virtual std::string **show** () override
virtual public methods
- void **setStation** (Station *_station)
- void **setStationName** (std::string stationName)
- Station * **getStation** () const

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (Model *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (ModelComponent *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (ModelComponent *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (Model *model, std::string datadefinitionTypename, std::string name="", bool insert↔
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- Model * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

- Enables or disables report-statistics generation for this definition.*
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation ()`
- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `bool Check (ModelComponent *component)`
Validates a component instance.
- static `void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- static `void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- static `void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createInternalStatisticReporters](#) () override
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.177.1 Detailed Description

Enter module DESCRIPTION The **Enter** module defines a station (or a set of stations) corresponding to a physical or logical location where processing occurs. When an entity arrives at an **Enter** module, an unloading delay may occur and any transfer device used to transfer the entity to the **Enter** module's station may be released. The station (or each station within the defined set) has a matching Activity Area that is used to report all times and costs accrued by the entities in this station. This Activity Area's name is the same as the station. If a parent Activity Area is defined, then it also accrues any times and costs by the entities in this station. TYPICAL USES The start of a part's production in a series of parallel processes where the part's forklift needs to be released The start of a document's processing after the document has been created where the mail clerk resource needs to be released PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart **Station** Type Type of station, either a single **Station** or station **Set**. **Station** Name Name of the individual station. A given station can only exist once within a model. Parent Activity Area Name of the Activity Area's parent. Associated Intersection Name of the intersection associated with this station in a guided transporter network. Report Statistics Specifies whether or not statistics will automatically be collected and stored in the report database for this station and its corresponding activity area. **Set** Name Name of the station set. A given station set can only exist once within a model. Save **Attribute** Specifies the attribute to be used to store the index into the station set for an entity entering this module. **Set** Members This repeat group permits you to define the individual stations that are to be members of the specified station set. A station set must have at least one member station. Active when **Station** Type is **Set**. **Station** Name This field indicates the name of a station that is to be a member of this station set. A given station can only exist within a model once. Therefore, an individual station can only be the member of one station set, and that individual station may not be the name of a station in another module. Parent Activity Area Name of the Activity Area's parent for the station set member. Associated Intersection Name of the intersection associated with this station set in a guided transporter network. Report Statistics Specifies whether or not statistics will automatically be collected and stored in the report database for this station set member and its corresponding activity area. Allocation Type of category to which the entity's incurred delay time and cost will be added. **Delay** This field defines the delay that will be experienced by entities immediately upon arrival at the station. Units Time units used for the delay time. Transfer In If a resource, transporter, or conveyor was used to transfer the entity to this station, this can be used to release, free, or exit the device. If **Release Resource** is selected, the specified resource is released. If Free Transporter is selected, the specified transporter is freed. If **Exit Conveyor** is selected, the specified conveyor is exited. Transporter Name Name of the transporter to be freed upon arrival to the station. Active when Transfer Name is Free Transporter. Unit Number Unit number of the transporter if the transporter is multicapacity. Conveyor Name Name of the conveyor to exit upon arrival to the station. **Resource** Type Type of allocation, either single **Resource** or resource **Set**. **Resource** Name Name of the resource to release. Active when Transfer Name is **Release Resource**. **Set** Name Name of the resource set from which the resource is to be released. **Release** Rule Determines which member of the set is to be released, either the Last Member Seized, First Member Seized, or Specific Member. **Set** Index Index into the set that determines which member of the set is to be released. **Attribute** Name Name of the attribute that determines the instance number of the resource to release. Expression Expression value that determines the instance number of the resource to release.

10.177.2 Member Function Documentation

10.177.2.1 `_check()`

```
bool Enter::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.177.2.2 `_createEditableDataDefinitions()`

```
void Enter::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.177.2.3 _createInternalStatisticReporters()

```
void Enter::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.177.2.4 _loadInstance()

```
bool Enter::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.177.2.5 _onDispatchEvent()

```
void Enter::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.177.2.6 _saveInstance()

```
void Enter::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.177.2.7 show()

```
std::string Enter::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

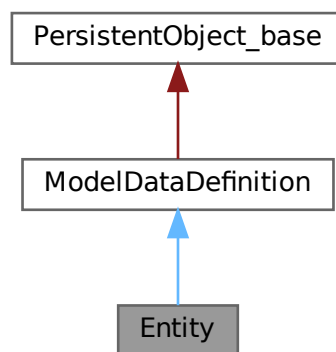
The documentation for this class was generated from the following files:

- plugins/components/MaterialHandling/Enter.h
- plugins/components/MaterialHandling/Enter.cpp

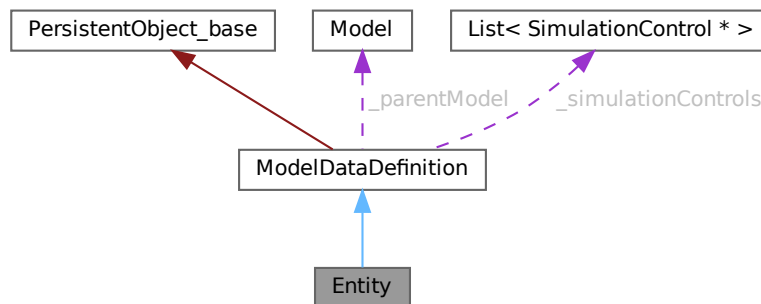
10.178 Entity Class Reference

```
#include <kernel/simulator/essentialPlugins/Entity.h>
```

Inheritance diagram for Entity:



Collaboration diagram for Entity:



Public Member Functions

- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **setEntityType** (std::string entityType)
setEntityType
- std::string **getEntityType** () const
getEntityType
- void **setEntityType** (EntityType *entityType)
setEntityType
- EntityType * **getEntityType** () const
getEntityType
- double **getAttributeValue** (std::string attributeName, std::string index="")
getAttributeValue
- double **getAttributeValue** (Util::identification attributelD, std::string index="")
getAttributeValue
- void **setAttributeValue** (std::string attributeName, double value, std::string index="", bool createlfNot↵
Found=false)
setAttributeValue
- void **setAttributeValue** (Util::identification attributelD, double value, std::string index="")
setAttributeValue
- void **copyAttributeValues** (std::string destinationAttributeName, std::string sourceAttributeName)
Copies all sparse values from one attribute to another for this entity.
- Util::identification **entityNumber** () const
entityNumber

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (Model *model, std::string datadefinitionType, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.

- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Friends

- class **Model**

Additional Inherited Members

Static Public Member Functions inherited from **ModelDataDefinition**

- static **ModelDataDefinition** * **LoadInstance** (**Model** *model, **PersistenceRecord** *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static **ModelDataDefinition** * **NewInstance** (**Model** *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** (**PersistenceRecord** *fields, **ModelDataDefinition** *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** (**ModelDataDefinition** *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** (**ModelDataDefinition** *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** (**ModelDataDefinition** *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Attributes inherited from **ModelDataDefinition**

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- **Model** * **_parentModel**
- List< **SimulationControl** * > * **_simulationControls** = new List<**SimulationControl***>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.178.1 Detailed Description

Entity module DESCRIPTION This data module defines the various entity types and their initial picture values in a simulation. Initial costing information and holding costs are also defined for the entity. TYPICAL USES Items being produced or assembled (parts, pallets) Documents (forms, e-mails, faxes, reports) People moving through a process (customers, callers) PROMPTS Prompt Description Name The unique name of the attribute being defined. Rows Number of rows in a one- or two-dimensional attribute. Columns Number of columns in a two-dimensional attribute. Data Type The data type of the values stored in the attribute. Valid types are Real and String. The default type is Real. Initial Values Lists the initial value or values of the attribute. You can assign new values to the attribute by using the **Assign** module. Initial Value **Entity** attribute value when entity is created and enters the system. Prompt Description **Entity** Type The name of the entity type being defined. This name must be unique. Initial Picture Graphical representation of the entity at the start of the simulation. This value can be changed during the simulation using the **Assign** module. Holding Cost/Hour Hourly cost of processing the entity through the system. This cost is incurred when the entity is anywhere in the system. Initial VA Cost Initial cost value that will be assigned to the value-added cost attribute of the entity. This attribute accrues the costs incurred when an entity is spending time in a value-added activity. Initial NVA Cost Initial cost value that will be assigned to the non-value-added cost attribute

of the entity. This attribute accrues the costs incurred when an entity is spending time in a non-value-added activity. Initial [Waiting](#) Cost Initial cost value that will be assigned to the waiting-cost attribute of the entity. This attribute accrues the costs incurred when an entity is spending time in a wait activity; for example, waiting to be batched or waiting for resource(s) at a [Process](#) module. Initial Transfer Cost Initial cost value that will be assigned to the transfer cost attribute of the entity. This attribute accrues the costs incurred when an entity is spending time in a transfer activity. Initial Other Cost Initial cost value that will be assigned to the other cost attribute of the entity. This attribute accrues the costs incurred when an entity is spending time in another activity. Report Statistics Specifies whether or not statistics will be collected automatically and stored in the report database for this entity type.

10.178.2 Member Function Documentation

10.178.2.1 `_check()`

```
bool Entity::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.178.2.2 `_loadInstance()`

```
bool Entity::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.178.2.3 `_saveInstance()`

```
void Entity::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.178.2.4 `entityNumber()`

```
Util::identification Entity::entityNumber ( ) const
```

`entityNumber`

Returns

Unique runtime identifier for this entity instance.

10.178.2.5 getAttributeValue() [1/2]

```
double Entity::getAttributeValue (
    std::string attributeName,
    std::string index = "" )
```

getAttributeValue**Parameters**

<i>index</i>	
<i>attributeName</i>	

Returns

Current attribute value for the provided name/index.

10.178.2.6 getAttributeValue() [2/2]

```
double Entity::getAttributeValue (
    Util::identification attributeID,
    std::string index = "" )
```

getAttributeValue**Parameters**

<i>index</i>	
<i>attributeID</i>	

Returns

Current attribute value for the provided id/index.

10.178.2.7 getEntityType()

```
EntityType * Entity::getEntityType ( ) const
```

getEntityType**Returns**

Pointer to the associated entity type.

10.178.2.8 getEntityTypeName()

```
std::string Entity::getEntityTypeName ( ) const
```

getEntityTypeName

Returns

Name of the associated entity type.

10.178.2.9 setAttributeValue() [1/2]

```
void Entity::setAttributeValue (
    std::string attributeName,
    double value,
    std::string index = "",
    bool createIfNotFound = false )
```

setAttributeValue

Parameters

<i>index</i>	
<i>attributeName</i>	
<i>value</i>	
<i>createIfNotFound</i>	

Assigns an attribute value by name and optionally creates missing attributes.

10.178.2.10 setAttributeValue() [2/2]

```
void Entity::setAttributeValue (
    Util::identification attributeID,
    double value,
    std::string index = "" )
```

setAttributeValue

Parameters

<i>index</i>	
<i>attributeID</i>	
<i>value</i>	

Assigns an attribute value by attribute identifier.

10.178.2.11 setEntityType()

```
void Entity::setEntityType (
    EntityType * entityType )
```

setEntityType

Parameters

<i>entityType</i>	
-------------------	--

Directly assigns the entity type pointer.

10.178.2.12 setEntityTypeName()

```
void Entity::setEntityTypeName (
    std::string entityTypeName )
```

setEntityTypeName

Parameters

<i>entityTypeName</i>	
-----------------------	--

Sets entity type by name (indirect lookup through model data manager).

10.178.2.13 show()

```
std::string Entity::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

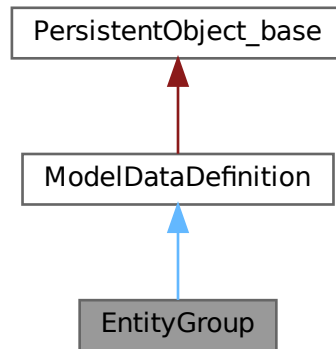
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

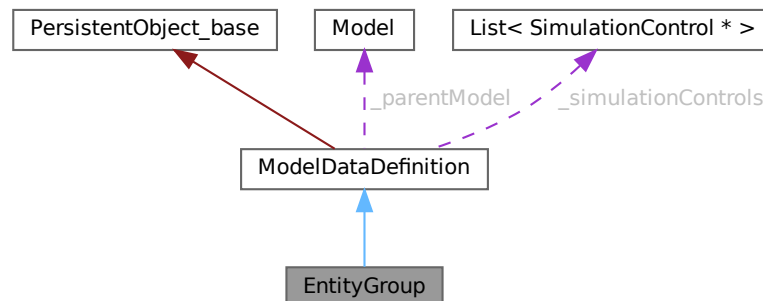
- kernel/simulator/essentialPlugins/Entity.h
- kernel/simulator/essentialPlugins/Entity.cpp

10.179 EntityGroup Class Reference

Inheritance diagram for EntityGroup:



Collaboration diagram for EntityGroup:



Public Member Functions

- **EntityGroup** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **insertElement** (unsigned int idKey, [Entity](#) *modeldatum)
- void **removeElement** (unsigned int idKey, [Entity](#) *modeldatum)
- [List< Entity * >](#) * **getGroup** (unsigned int idKey)

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_initBetweenReplications** () override
- virtual void **_createInternalStatisticReporters** () override
- virtual void **_createAttachedAttributes** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createNonEditableDataDefinitions** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()

- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.179.1 Member Function Documentation

10.179.1.1 `_check()`

```
bool EntityGroup::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.179.1.2 `_createAttachedAttributes()`

```
void EntityGroup::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.179.1.3 `_createInternalStatisticReporters()`

```
void EntityGroup::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.179.1.4 `_initBetweenReplications()`

```
void EntityGroup::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.179.1.5 `_loadInstance()`

```
bool EntityGroup::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.179.1.6 `_saveInstance()`

```
void EntityGroup::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.179.1.7 show()

```
std::string EntityGroup::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

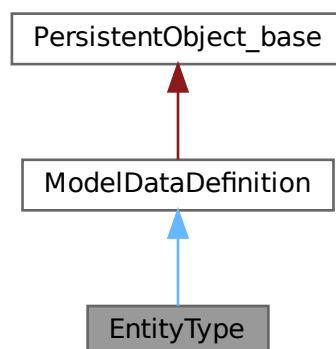
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

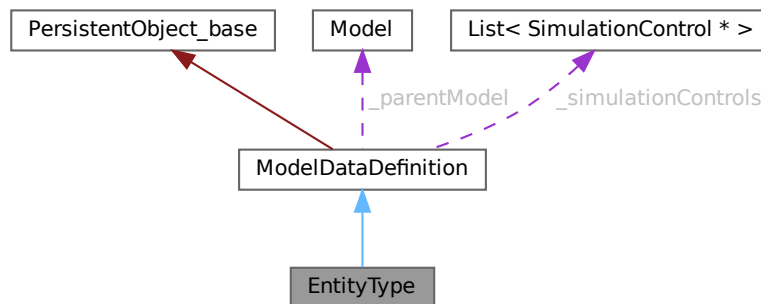
- plugins/data/Grouping/EntityGroup.h
- plugins/data/Grouping/EntityGroup.cpp

10.180 EntityType Class Reference

Inheritance diagram for EntityType:



Collaboration diagram for EntityType:



Public Member Functions

- **EntityType** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
Returns a textual representation of this data definition.
- void **setInitialWaitingCost** (double _initialWaitingCost)
- double **initialWaitingCost** () const
- void **setInitialOtherCost** (double _initialOtherCost)
- double **initialOtherCost** () const
- void **setInitialNVACost** (double _initialNVACost)
- double **initialNVACost** () const
- void **setInitialIVACost** (double _initialIVACost)
- double **initialIVACost** () const
- void **setInitialPicture** (std::string _initialPicture)
- std::string **initialPicture** () const
- [StatisticsCollector](#) * [addGetStatisticsCollector](#) (std::string name)
addGetStatisticsCollector

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↔IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const

- Returns the user-visible label of this instance.*

 - const std::string & [getClassname](#) () const

Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const

Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const

Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const

Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const

Returns the full map of attached data definitions.
- bool [hasChanged](#) () const

Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))

Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const

Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const

Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const

Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)

Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createInternalStatisticReporters](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.180.1 Member Function Documentation

10.180.1.1 _createInternalStatisticReporters()

```
void EntityType::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.180.1.2 `_initBetweenReplications()`

```
void EntityType::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.180.1.3 `_loadInstance()`

```
bool EntityType::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.180.1.4 `_saveInstance()`

```
void EntityType::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.180.1.5 `addGetStatisticsCollector()`

```
StatisticsCollector * EntityType::addGetStatisticsCollector (
    std::string name )
```

`addGetStatisticsCollector`

Parameters

<i>name</i>	
-------------	--

Returns

10.180.1.6 `show()`

```
std::string EntityType::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- kernel/simulator/essentialPlugins/EntityType.h
- kernel/simulator/essentialPlugins/EntityType.cpp

10.181 PersistenceRecord::Entry Struct Reference

"Raw" entry, users should treat this as an opaque type since payloads could be in any format.

```
#include <kernel/simulator/Persistence.h>
```

Public Types

- enum **Kind** { **text** , **numeric** }

Public Attributes

- std::string **first**
- Payload **second**
- Kind **kind** {Kind::text}

10.181.1 Detailed Description

"Raw" entry, users should treat this as an opaque type since payloads could be in any format.

The documentation for this struct was generated from the following file:

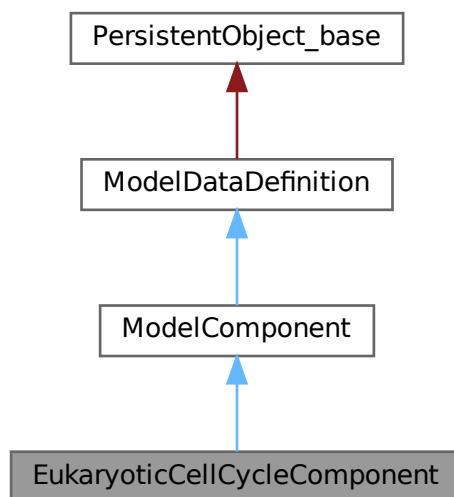
- kernel/simulator/Persistence.h

10.182 EukaryoticCellCycleComponent Class Reference

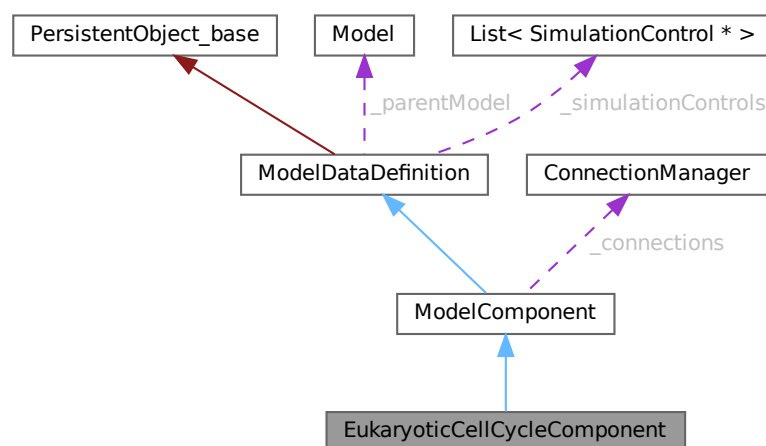
Coordinates a didactic eukaryotic cell cycle over a shared [WholeCellState](#).

```
#include <plugins/components/WholeCellModeling/EukaryoticCellCycleComponent.h>
```

Inheritance diagram for EukaryoticCellCycleComponent:



Collaboration diagram for EukaryoticCellCycleComponent:



Public Member Functions

- **EukaryoticCellCycleComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setDeltaT** (double deltaT)
- double **getDeltaT** () const
- void **setAdvanceWholeCellClock** (bool advance)
- bool **getAdvanceWholeCellClock** () const
- void **setCytosolCompartmentName** (std::string name)
- std::string **getCytosolCompartmentName** () const
- void **setNucleusCompartmentName** (std::string name)
- std::string **getNucleusCompartmentName** () const
- void **setMitochondrionCompartmentName** (std::string name)
- std::string **getMitochondrionCompartmentName** () const
- void **setBudCompartmentName** (std::string name)
- std::string **getBudCompartmentName** () const
- void **setEnergyMetaboliteKey** (std::string key)
- std::string **getEnergyMetaboliteKey** () const
- void **setCytosolicEnergyMetaboliteKey** (std::string key)
- std::string **getCytosolicEnergyMetaboliteKey** () const
- void **setMitochondrialEnergyMetaboliteKey** (std::string key)
- std::string **getMitochondrialEnergyMetaboliteKey** () const
- void **setBudEnergyMetaboliteKey** (std::string key)
- std::string **getBudEnergyMetaboliteKey** () const
- void **setGrowthFluxKey** (std::string key)
- std::string **getGrowthFluxKey** () const
- void **setRespirationFluxKey** (std::string key)
- std::string **getRespirationFluxKey** () const
- void **setBudProgressKey** (std::string key)
- std::string **getBudProgressKey** () const
- void **setDnaReplicationProgressKey** (std::string key)
- std::string **getDnaReplicationProgressKey** () const
- void **setSpindleAssemblyProgressKey** (std::string key)
- std::string **getSpindleAssemblyProgressKey** () const
- void **setMitoticExitProgressKey** (std::string key)
- std::string **getMitoticExitProgressKey** () const
- void **setGlobalAtpThreshold** (double threshold)
- double **getGlobalAtpThreshold** () const
- void **setCytosolicAtpThreshold** (double threshold)
- double **getCytosolicAtpThreshold** () const
- void **setMitochondrialAtpThreshold** (double threshold)
- double **getMitochondrialAtpThreshold** () const
- void **setBudAtpThreshold** (double threshold)
- double **getBudAtpThreshold** () const
- void **setGrowthFluxThreshold** (double threshold)
- double **getGrowthFluxThreshold** () const
- void **setRespirationFluxThreshold** (double threshold)
- double **getRespirationFluxThreshold** () const
- void **setBudProgressRate** (double rate)
- double **getBudProgressRate** () const
- void **setDnaReplicationRate** (double rate)
- double **getDnaReplicationRate** () const
- void **setSpindleAssemblyRate** (double rate)
- double **getSpindleAssemblyRate** () const

- void **setMitoticExitRate** (double rate)
- double **getMitoticExitRate** () const
- void **setBudProgressThreshold** (double threshold)
- double **getBudProgressThreshold** () const
- void **setDnaReplicationThreshold** (double threshold)
- double **getDnaReplicationThreshold** () const
- void **setSpindleAssemblyThreshold** (double threshold)
- double **getSpindleAssemblyThreshold** () const
- void **setMitoticExitThreshold** (double threshold)
- double **getMitoticExitThreshold** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

- Returns the runtime class name used by the plugin system.*
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `ModelComponent`

- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static bool Check (ModelComponent *component)`
Validates a component instance.
- `static void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- `static void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- `static void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.182.1 Detailed Description

Coordinates a didactic eukaryotic cell cycle over a shared [WholeCellState](#).

[EukaryoticCellCycleComponent](#) provides an explicit, event-driven lifecycle layer for budding-yeast-like whole-cell models. It interprets shared state variables and compartment-specific energy proxies, then annotates [WholeCellState](#) with a coarse-grained eukaryotic cycle:

- newborn
- g1_budding
- s_phase
- g2_phase
- m_phase
- division_ready
- arrested
- dead

The component does not execute metabolism, transport, expression, or division itself. Instead, it consumes their outputs from [WholeCellState](#) and advances four progress variables stored as pathway activities:

- bud growth progress
- DNA replication progress
- spindle assembly progress
- mitotic exit progress

This design keeps structural biology in [BioCompartment](#), mutable system state in [WholeCellState](#), and dynamic checkpoint logic in a dedicated [ModelComponent](#).

10.182.2 Member Function Documentation

10.182.2.1 `_check()`

```
bool EukaryoticCellCycleComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.182.2.2 `_createEditableDataDefinitions()`

```
void EukaryoticCellCycleComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.182.2.3 _initBetweenReplications()

```
void EukaryoticCellCycleComponent::_initBetweenReplications ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.182.2.4 _loadInstance()

```
bool EukaryoticCellCycleComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.182.2.5 _onDispatchEvent()

```
void EukaryoticCellCycleComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.182.2.6 _saveInstance()

```
void EukaryoticCellCycleComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.182.2.7 show()

```
std::string EukaryoticCellCycleComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

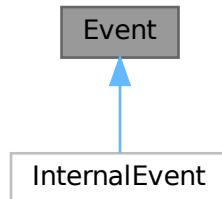
The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/EukaryoticCellCycleComponent.h
- plugins/components/WholeCellModeling/EukaryoticCellCycleComponent.cpp

10.183 Event Class Reference

```
#include <kernel/simulator/Event.h>
```

Inheritance diagram for Event:

**Public Member Functions**

- **Event** (double time, [Entity](#) *entity, [ModelComponent](#) *component, unsigned int componentinputPortNumber=0)
- **Event** (double time, [Entity](#) *entity, [Connection](#) *connection)
- double [getTime](#) () const
getTime
- [ModelComponent](#) * [getComponent](#) () const
getComponent
- [Entity](#) * [getEntity](#) () const
getEntity
- unsigned int [getComponentinputPortNumber](#) () const
getComponentinputPortNumber
- void [setEntity](#) ([Entity](#) *_entity)
setEntity
- virtual std::string **show** ()

Protected Attributes

- double `_time`

10.183.1 Detailed Description

An instantaneous event, triggered at a certain moment by an entity upon reaching a component. The simulated time advances in discrete points in time and that are the instants that an event is triggered.

10.183.2 Member Function Documentation

10.183.2.1 `getComponent()`

```
ModelComponent * Event::getComponent ( ) const
```

`getComponent`

Returns

10.183.2.2 `getComponentinputPortNumber()`

```
unsigned int Event::getComponentinputPortNumber ( ) const
```

`getComponentinputPortNumber`

Returns

10.183.2.3 `getEntity()`

```
Entity * Event::getEntity ( ) const
```

`getEntity`

Returns

10.183.2.4 `getTime()`

```
double Event::getTime ( ) const
```

`getTime`

Returns

10.183.2.5 `setEntity()`

```
void Event::setEntity (
    Entity * _entity )
```

`setEntity`

Parameters

<code>_entity</code>	
----------------------	--

The documentation for this class was generated from the following files:

- kernel/simulator/Event.h
- kernel/simulator/Event.cpp

10.184 Exact Class Reference

Public Member Functions

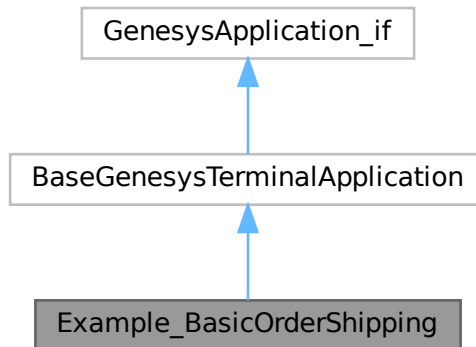
- **Exact** (int num, int den=1)
- const [Exact](#) & **operator+** (const [Exact](#) &right)
- const [Exact](#) & **operator+=** (const [Exact](#) &right)
- const [Exact](#) & **operator-=** (const [Exact](#) &right)
- const [Exact](#) & **operator*=** (const [Exact](#) &right)
- const [Exact](#) & **operator/=** (const [Exact](#) &right)
- const [Exact](#) & **operator=** (const [Exact](#) &right)
- const [Exact](#) & **operator=** (const int &right)
- bool **operator==** (const [Exact](#) &right)
- bool **operator==** (const double &right)
- bool **operator<** (const [Exact](#) &right)
- bool **operator<** (const double &right)
- bool **operator<=** (const [Exact](#) &right)
- bool **operator<=** (const double &right)
- bool **operator>** (const [Exact](#) &right)
- bool **operator>** (const double &right)
- bool **operator>=** (const [Exact](#) &right)
- bool **operator>=** (const double &right)
- void **Simplify** ()
- double **toFloat** ()

The documentation for this class was generated from the following file:

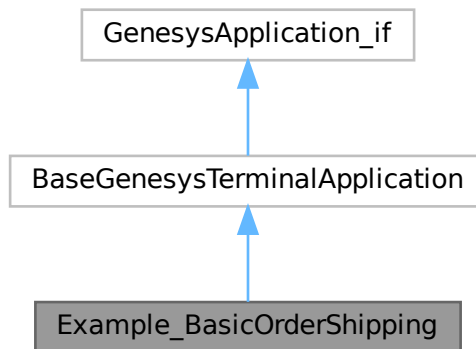
- kernel/util/Exact.h

10.185 Example_BasicOrderShipping Class Reference

Inheritance diagram for Example_BasicOrderShipping:



Collaboration diagram for Example_BasicOrderShipping:



Public Member Functions

- virtual int `main` (int argc, char **argv) override

Public Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void `Simulate` (`Simulator` *genesys, `Model` *model, `PluginManager` *plugins)
- void `setDefaultTraceHandlers` (`TraceManager` *tm)
- void `setDefaultEventHandlers` (`OnEventManager` *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.185.1 Member Function Documentation

10.185.1.1 main()

```
int Example_BasicOrderShipping::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

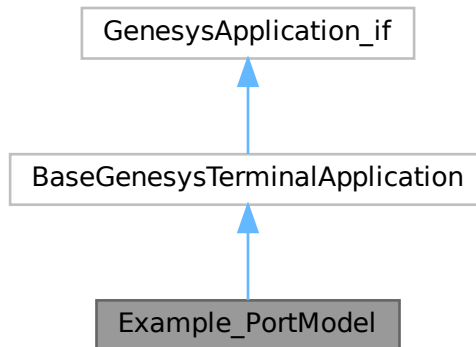
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

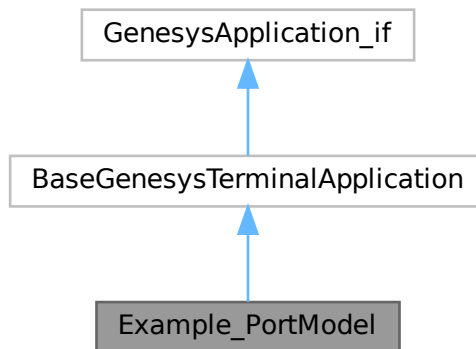
- applications/modelSpecific/arenaExamples/Example_BasicOrderShipping.h
- applications/modelSpecific/arenaExamples/Example_BasicOrderShipping.cpp

10.186 Example_PortModel Class Reference

Inheritance diagram for Example_PortModel:



Collaboration diagram for Example_PortModel:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.186.1 Member Function Documentation

10.186.1.1 main()

```
int Example_PortModel::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

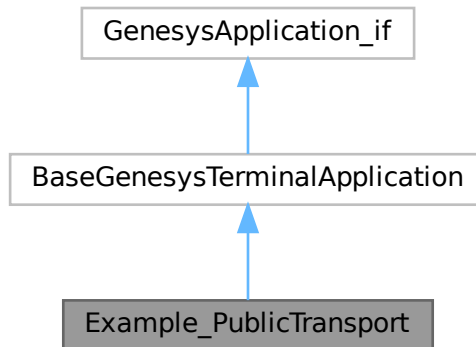
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

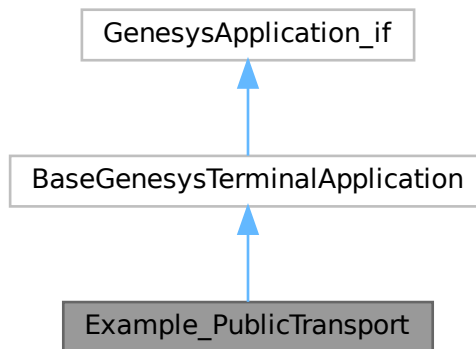
- applications/modelSpecific/arenaExamples/Example_PortModel.h
- applications/modelSpecific/arenaExamples/Example_PortModel.cpp

10.187 Example_PublicTransport Class Reference

Inheritance diagram for Example_PublicTransport:



Collaboration diagram for Example_PublicTransport:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.187.1 Member Function Documentation

10.187.1.1 main()

```
int Example_PublicTransport::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/arenaExamples/Example_PublicTransport.h
- applications/modelSpecific/arenaExamples/Example_PublicTransport.cpp

10.188 CompartmentExchangeComponent::ExchangeRule Struct Reference

Public Attributes

- std::string **label**
- std::string **sourceRegion**
- std::string **sourceMetaboliteKey**
- std::string **targetRegion**
- std::string **targetMetaboliteKey**
- double **exchangeFraction** = 0.0
- double **baselineAmount** = 0.0
- std::string **driverPathwayKey**
- double **driverScale** = 1.0
- double **maxTransferAmount** = 0.0
- bool **conserveMass** = true

The documentation for this struct was generated from the following file:

- plugins/components/WholeCellModeling/CompartmentExchangeComponent.h

10.189 GroProgramRuntime::ExecutionResult Struct Reference

Public Attributes

- bool **succeeded** = true
- std::string **errorMessage** = ""
- unsigned int **executedCommands** = 0
- std::vector< [PopulationMutation](#) > **populationMutations**
- std::vector< [SignalMutation](#) > **signalMutations**
- std::vector< [MotionMutation](#) > **motionMutations**
- std::vector< [ColonyMutation](#) > **colonyMutations**
- std::vector< double > **mappedCellValues**
- std::string **mappedCellExpression**
- std::vector< double > **signalMatrixValues**
- unsigned int **signalMatrixWidth** = 0
- unsigned int **signalMatrixHeight** = 0
- std::string **signalFieldDump**
- std::vector< std::string > **messages**
- std::map< std::string, double > **assignedVariables**
- std::vector< std::string > **unsupportedCommands**
- std::vector< std::string > **skippedRawStatements**

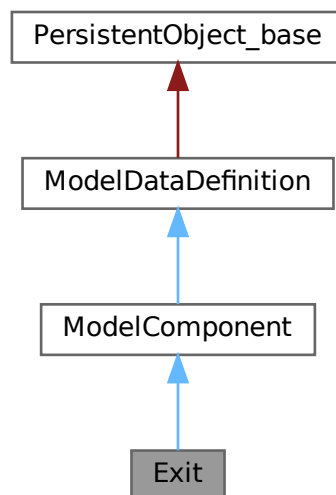
The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/GroProgramRuntime.h

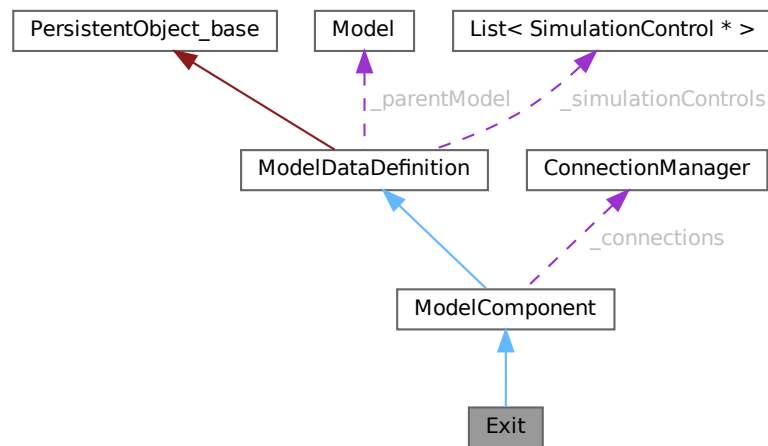
10.190 Exit Class Reference

```
#include <plugins/components/MaterialHandling/Exit.h>
```

Inheritance diagram for Exit:



Collaboration diagram for Exit:



Public Member Functions

- **Exit** ([Model](#) *model, std::string name="")
 - virtual std::string [show](#) () override
- virtual public methods*

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.190.1 Detailed Description

[Exit](#) module DESCRIPTION The [Exit](#) module releases the entity's cells on the specified conveyor. If another entity is waiting in queue for the conveyor at the same station when the cells are released, it will then access the conveyor. TYPICAL USES Cases exit a conveyor for packing Bad parts are removed from the conveyor and disposed Passengers remove luggage from the baggage claim conveyor PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart. Conveyor Name Name of the conveyor on which the entity will exit. If left blank, the previously accessed conveyor is assumed.

10.190.2 of Cells Number of contiguous conveyor cells the entity will relinquish.

10.190.3 Member Function Documentation

10.190.3.1 `_check()`

```
bool Exit::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.190.3.2 `_loadInstance()`

```
bool Exit::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.190.3.3 `_onDispatchEvent()`

```
void Exit::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.190.3.4 _saveInstance()

```
void Exit::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.190.3.5 show()

```
std::string Exit::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/MaterialHandling/Exit.h
- plugins/components/MaterialHandling/Exit.cpp

10.191 ExperimentManager Class Reference

Manages higher-level simulation experiments.

```
#include <kernel/simulator/ExperimentManager.h>
```

Public Member Functions

- **ExperimentManager** ([Simulator](#) *simulator)
- [SimulationExperiment](#) * **newSimulationExperiment** ()
- void **insert** ([SimulationExperiment](#) *experiment)
- void **remove** ([SimulationExperiment](#) *experiment)
- void **setCurrent** ([SimulationExperiment](#) *experiment)
- bool **saveSimulationExperiment** (std::string filename)
- bool **loadSimulationExperiment** (std::string filename)
- unsigned int **size** ()
- [SimulationExperiment](#) * **front** ()
- [SimulationExperiment](#) * **current** ()
- [SimulationExperiment](#) * **next** ()
- [List](#)< [SimulationExperiment](#) * > * **getExperiments** () const

10.191.1 Detailed Description

Manages higher-level simulation experiments.

Historically, GenESyS initially supported only the basic experiment controls available through [ModelSimulation](#), especially the number of replications and the replication length.

This class represents a later attempt to move more advanced experiment design concerns into the simulation kernel, including higher-level experiment sets, scenario management and future [Process](#) Analyzer-like workflows.

This layer is still under development and should be treated as an incomplete experimental abstraction on top of the already functional [ModelSimulation](#) mechanism.

The documentation for this class was generated from the following files:

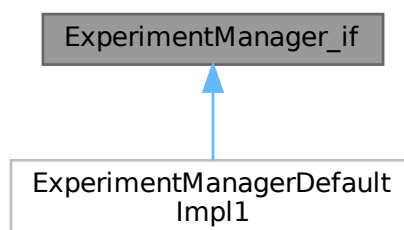
- kernel/simulator/ExperimentManager.h
- kernel/simulator/ExperimentManager.cpp

10.192 ExperimentManager_if Class Reference

Interface that coordinates execution of scenario-based simulation experiments.

```
#include <kernel/simulator/ExperimentManager_if.h>
```

Inheritance diagram for ExperimentManager_if:



Public Member Functions

- virtual [List](#)< [SimulationScenario](#) * > * [getScenarios](#) () const =0
Returns the list of scenarios that compose the experiment.
- virtual void [startSimulationOfScenario](#) ([SimulationScenario](#) *scenario)=0
Starts simulation for a specific scenario.
- virtual void [startExperiment](#) ()=0
Starts full experiment execution (all enabled scenarios).
- virtual void [stopExperiment](#) ()=0
Requests interruption of the currently running experiment.
- virtual void [addTraceSimulationHandler](#) (traceSimulationProcessListener traceSimulationProcessListener)=0
Registers a callback to receive simulation progress traces for the experiment.

10.192.1 Detailed Description

Interface that coordinates execution of scenario-based simulation experiments.

An experiment manager owns a set of scenarios, starts/stops scenario runs and exposes progress tracing hooks so higher-level tools can monitor experiment status.

10.192.2 Member Function Documentation

10.192.2.1 [addTraceSimulationHandler\(\)](#)

```
virtual void ExperimentManager_if::addTraceSimulationHandler (
    traceSimulationProcessListener traceSimulationProcessListener ) [pure virtual]
```

Registers a callback to receive simulation progress traces for the experiment.

Implemented in [ExperimentManagerDefaultImpl1](#).

10.192.2.2 [getScenarios\(\)](#)

```
virtual List< SimulationScenario * > * ExperimentManager_if::getScenarios ( ) const [pure virtual]
```

Returns the list of scenarios that compose the experiment.

Implemented in [ExperimentManagerDefaultImpl1](#).

10.192.2.3 [startExperiment\(\)](#)

```
virtual void ExperimentManager_if::startExperiment ( ) [pure virtual]
```

Starts full experiment execution (all enabled scenarios).

Implemented in [ExperimentManagerDefaultImpl1](#).

10.192.2.4 startSimulationOfScenario()

```
virtual void ExperimentManager_if::startSimulationOfScenario (
    SimulationScenario * scenario ) [pure virtual]
```

Starts simulation for a specific scenario.

Implemented in [ExperimentManagerDefaultImpl1](#).

10.192.2.5 stopExperiment()

```
virtual void ExperimentManager_if::stopExperiment ( ) [pure virtual]
```

Requests interruption of the currently running experiment.

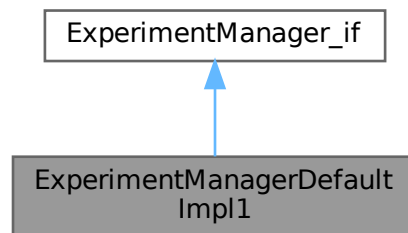
Implemented in [ExperimentManagerDefaultImpl1](#).

The documentation for this class was generated from the following file:

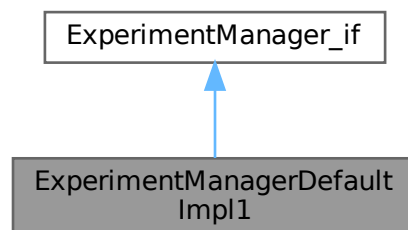
- kernel/simulator/ExperimetManager_if.h

10.193 ExperimentManagerDefaultImpl1 Class Reference

Inheritance diagram for ExperimentManagerDefaultImpl1:



Collaboration diagram for ExperimentManagerDefaultImpl1:



Public Member Functions

- virtual [List< SimulationScenario * > * getScenarios \(\)](#) const override
Returns the list of scenarios that compose the experiment.
- virtual void [startSimulationOfScenario \(SimulationScenario *scenario\)](#) override
Starts simulation for a specific scenario.
- virtual void [startExperiment \(\)](#) override
Starts full experiment execution (all enabled scenarios).
- virtual void [stopExperiment \(\)](#) override
Requests interruption of the currently running experiment.
- virtual void [addTraceSimulationHandler](#) (traceSimulationProcessListener traceSimulationProcessListener) override
Registers a callback to receive simulation progress traces for the experiment.

10.193.1 Member Function Documentation

10.193.1.1 addTraceSimulationHandler()

```
void ExperimentManagerDefaultImpl1::addTraceSimulationHandler (
    traceSimulationProcessListener traceSimulationProcessListener ) [override], [virtual]
```

Registers a callback to receive simulation progress traces for the experiment.

Implements [ExperimentManager_if](#).

10.193.1.2 getScenarios()

```
List< SimulationScenario * > * ExperimentManagerDefaultImpl1::getScenarios ( ) const [override],
[virtual]
```

Returns the list of scenarios that compose the experiment.

Implements [ExperimentManager_if](#).

10.193.1.3 startExperiment()

```
void ExperimentManagerDefaultImpl1::startExperiment ( ) [override], [virtual]
```

Starts full experiment execution (all enabled scenarios).

Implements [ExperimentManager_if](#).

10.193.1.4 startSimulationOfScenario()

```
void ExperimentManagerDefaultImpl1::startSimulationOfScenario (
    SimulationScenario * scenario ) [override], [virtual]
```

Starts simulation for a specific scenario.

Implements [ExperimentManager_if](#).

10.193.1.5 stopExperiment()

```
void ExperimentManagerDefaultImpl::stopExperiment ( ) [override], [virtual]
```

Requests interruption of the currently running experiment.

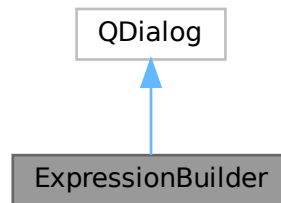
Implements [ExperimentManager_if](#).

The documentation for this class was generated from the following files:

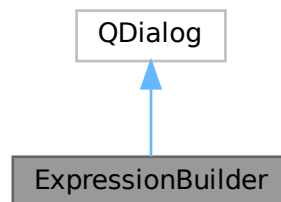
- kernel/simulator/ExperimentManagerDefaultImpl.h
- kernel/simulator/ExperimentManagerDefaultImpl.cpp

10.194 ExpressionBuilder Class Reference

Inheritance diagram for ExpressionBuilder:



Collaboration diagram for ExpressionBuilder:



Public Member Functions

- **ExpressionBuilder** (QWidget *parent=nullptr)

The documentation for this class was generated from the following files:

- applications/gui/genesys/tools/expressionbuilder/ExpressionBuilder.h
- applications/gui/genesys/tools/expressionbuilder/ExpressionBuilder.cpp

10.195 ExpressionCategory Struct Reference

Public Attributes

- QString **name**
- QList< [ExpressionTemplate](#) > **expressions**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/tools/expressionbuilder/ExpressionBuilder.h

10.196 ExpressionParameter Struct Reference

Public Attributes

- QString **name**
- QString **defaultValue**
- QString **description**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/tools/expressionbuilder/ExpressionBuilder.h

10.197 ExpressionTemplate Struct Reference

Public Attributes

- QString **name**
- QString **pattern**
- QString **description**
- QList< [ExpressionParameter](#) > **parameters**

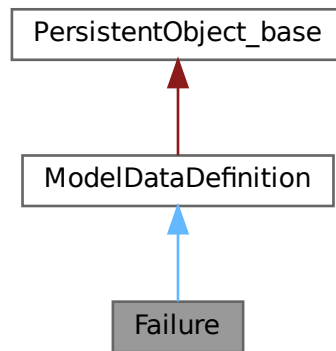
The documentation for this struct was generated from the following file:

- applications/gui/genesys/tools/expressionbuilder/ExpressionBuilder.h

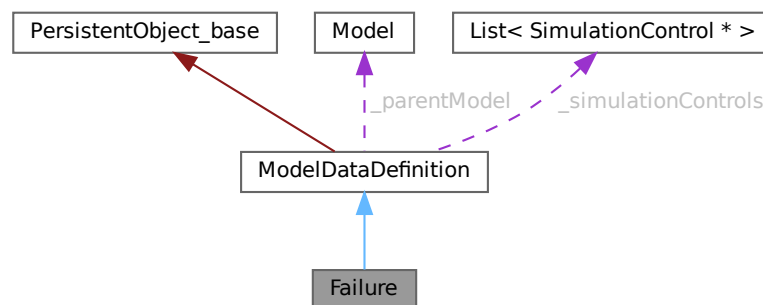
10.198 Failure Class Reference

```
#include <plugins/data/DiscreteProcessing/Failure.h>
```

Inheritance diagram for Failure:



Collaboration diagram for Failure:



Public Types

- enum class **FailureType** : int { **COUNT** = 0 , **TIME** = 1 , **num_elements** = 2 }
- enum class **FailureRule** : int { **IGNORE** = 0 , **PREEMPT** = 1 , **WAIT** = 2 , **num_elements** = 3 }

Public Member Functions

- **Failure** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.

- void **checkIsGoingToFailByCount** ([Resource](#) *resource)
- void **setFailureType** (FailureType _failureType)
- Failure::FailureType **getFailureType** () const
- void **setCountExpression** (std::string countExpression)
- std::string **getCountExpression** () const
- void **setDownTimeTimeUnit** ([Util::TimeUnit](#) downTimeTimeUnit)
- [Util::TimeUnit](#) **getDownTimeTimeUnit** () const
- void **setDownTimeExpression** (std::string downTimeExpression)
- std::string **getDownTimeExpression** () const
- void **setUpTimeTimeUnit** ([Util::TimeUnit](#) upTimeTimeUnit)
- [Util::TimeUnit](#) **getUpTimeTimeUnit** () const
- void **setUpTimeExpression** (std::string upTimeExpression)
- std::string **getUpTimeExpression** () const
- void **setFailureRule** (FailureRule _failureRule)
- FailureRule **getFailureRule** () const
- [List](#)< [Resource](#) * > * **failingResources** () const
- void **addResource** ([Resource](#) *newResource)
- void **removeResource** ([Resource](#) *resource)

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const

- Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int [_modelLevel](#))
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) [traceLevelspecific](#), bool [traceLevelSpecific](#)↔
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool [traceLevelSpecificEnabled](#))
Enables or disables the specific trace level.

Static Public Member Functions

- static std::string [convertEnumToStr](#) ([FailureType](#) type)
- static std::string [convertEnumToStr](#) ([FailureRule](#) rule)
- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_initBetweenReplications` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification _id
- std::string _typename
- bool _reportStatistics
- bool _hasChanged
- unsigned int _modelLevel = 0
- [Model](#) * _parentModel
- [List](#)< [SimulationControl](#) * > * _simulationControls = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level _traceLevelSpecific
- bool _traceLevelSpecificEnabled = false

10.198.1 Detailed Description

[Failure](#) module DESCRIPTION The [Failure](#) module is designed for use with resources. When a failure occurs, the entire resource (regardless of its capacity) is failed. Failures are designed to be used with single-capacity resources or with multiple-capacity resources whose individual resource units all fail at the same time. TYPICAL USES Break-down information for a machine Cash register tape refill every “x” customers Random computer shutdowns or restarts PROMPTS Recordset Name of the recordset in the specified file from which to read values. This field is available only if you specify a [File](#) Name with a file access type, path, and recordset. Arena uses the Rows and Columns properties to determine the amount of data to read from the recordset. A recordset is required for all file types except .xml. The recordset size must be equal to or greater than the number of rows and columns specified for the expression. Expression Values Lists the value or values of the expression. This property is not available if you specify a [File](#) Name from which to read expression values. Expression Value Expression value associated with the expression name. Prompt Description Name The name of the failure associated with one or more resources. Type Determines if the failure is time-based or count-based. Count Defines the number of resource releases for count-based failures. Valid when the Type is Count. Up Time Defines the time between failures for time-based failures. Valid when the Type is Time. Up Time Units Time units for the time between failures (Up Time) for timebased failures. Down Time Defines the duration of the failure. Down Time Units Time units for the duration of the failure (Down Time). Uptime in this [State](#) only Defines the state that should be considered for the time between failures (only for time-based failures). If state is not specified, then all states are considered (that is, the time between failures does not depend on the time spent in a specific state, but rather on the total simulation time). For example, you might want to define a failure to be based only on the state Busy, and therefore, the time between downtimes would be based on the amount of time that a resource is busy, not simulated clock time.

10.198.2 Member Function Documentation

10.198.2.1 `_check()`

```
bool Failure::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Validate failure expressions according to selected failure mode.

Reimplemented from [ModelDataDefinition](#).

10.198.2.2 `_initBetweenReplications()`

```
void Failure::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.198.2.3 `_loadInstance()`

```
bool Failure::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Load [Failure](#) persistence fields.

This method mirrors [_saveInstance](#) field names to preserve a deterministic serialization contract.

Reimplemented from [ModelDataDefinition](#).

10.198.2.4 `_saveInstance()`

```
void Failure::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Persist [Failure](#) configuration fields.

Reimplemented from [ModelDataDefinition](#).

10.198.2.5 show()

```
std::string Failure::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

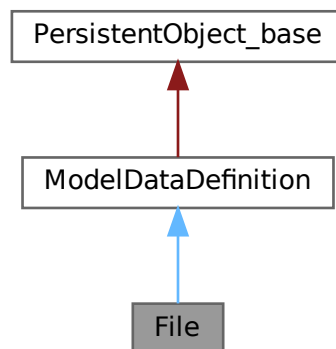
The documentation for this class was generated from the following files:

- plugins/data/DiscreteProcessing/Failure.h
- plugins/data/DiscreteProcessing/Failure.cpp

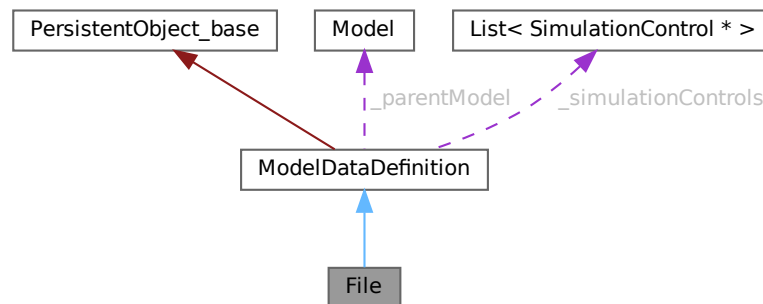
10.199 File Class Reference

```
#include <plugins/data/InOutOutput/File.h>
```

Inheritance diagram for File:



Collaboration diagram for File:



Public Types

- enum class **AccessMode** : unsigned int { **Read** = 0 , **Write** = 1 , **Append** = 2 , **ReadWrite** = 3 }

Public Member Functions

- **File** ([Model](#) *model, std::string name="")
- void **setSystemFilename** (std::string systemFilename)
- std::string **getSystemFilename** () const
- std::string **getFilenameOnly** () const
- std::string **getPathOnly** () const
- void **setAccessMode** (AccessMode accessMode)
- AccessMode **getAccessMode** () const
- void **setAccessModeAsString** (std::string accessMode)
- std::string **getAccessModeAsString** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↔ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- Releases resources owned by this data definition.*
- Util::identification **getId** () const
- Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & **getName** () const
- Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
- Updates the user-visible label of this instance.*

- `const std::string & getLabel () const`
Returns the user-visible label of this instance.
- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.199.1 Detailed Description

File module DESCRIPTION Use the **File** module to access external files for the ReadWrite module, **Variable** module, and Expression module. The **File** module identifies the system file name and defines the access type and operational characteristics of the file. TYPICAL USES **File** containing predefined airline flight data **File** specifying customer order times and relevant information **File** to write user model configuration data from menu input PROMPTS Prompt Description Name The name of the file whose characteristics are being defined. This name must be unique. **Access** Type The file type. Operating System **File** Name Name of the actual file that is being read from or to which it is being written. Connecting String **Connection** string used to open ADO connection to the data source. Structure **File** structure, which can be unformatted, free format, or a specific C or FORTRAN format. End of **File** Action Type of action to occur if an end of file condition is reached. Initialize Option Action to be taken on file at beginning of each simulation replication. Comment Character Character indicating comment record. Recordset Name Name used to identify the recordset in the Expression, ReadWrite, and **Variable** modules. This name must be unique within the file. This field is available for Microsoft Excel, Microsoft Excel 2007, Microsoft **Access**, Microsoft **Access** 2007, and ActiveX Data Objects files. CommandText Text of the command that will be used to open the recordset (for example, SQL statement, procedure name, table name.) This field is available for ActiveX Data Object files only. CommandType Type of command entered in the CommandText. Named Range The named range in the Excel workbook to which the recordset refers. Table Name The name of the table in the **Access** database to which the recordset refers.

10.199.2 Member Function Documentation

10.199.2.1 `_check()`

```
bool File::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.199.2.2 `_getParserChangesInformation()`

```
ParserChangesInformation * File::_getParserChangesInformation ( ) [override], [protected],
[virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to simulator kernel to reflect the changes Return parser deltas required by [File](#).

[File](#) currently does not require parser extensions.

Reimplemented from [ModelDataDefinition](#).

10.199.2.3 `_loadInstance()`

```
bool File::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.199.2.4 `_saveInstance()`

```
void File::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.199.2.5 show()

```
std::string File::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

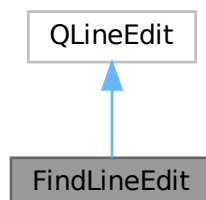
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

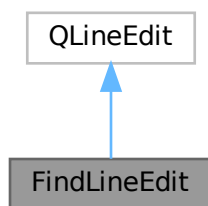
- plugins/data/InputOutput/File.h
- plugins/data/InputOutput/File.cpp

10.200 FindLineEdit Class Reference

Inheritance diagram for FindLineEdit:



Collaboration diagram for FindLineEdit:



Public Member Functions

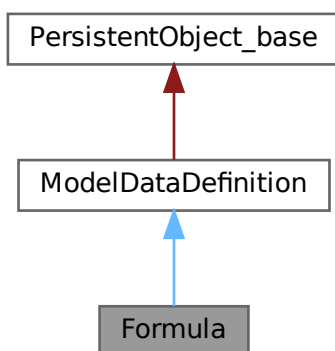
- **FindLineEdit** (QWidget *parent=nullptr)

The documentation for this class was generated from the following file:

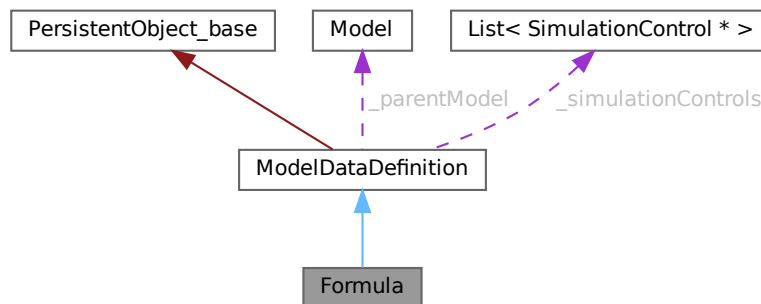
- applications/gui/genesys/dialogs/DialogFind.h

10.201 Formula Class Reference

Inheritance diagram for Formula:



Collaboration diagram for Formula:



Public Member Functions

- [Formula](#) ([Model](#) *model, std::string name="")
Creates an empty formula definition.
- virtual ~**Formula** () override
Releases formula-owned resources.
- virtual std::string [show](#) () override
Returns a textual representation of the formula.
- unsigned int [size](#) ()
Returns the number of indexed expressions stored in this formula.
- void [setExpression](#) (std::string formulaExpression, std::string index="")
Stores or updates the expression associated with an index.
- std::string [getExpression](#) (std::string index="")
Returns the expression associated with an index.
- double [getValue](#) (std::string index)
Evaluates the expression associated with an index.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const

- Returns the user-visible label of this instance.*

 - `const std::string & getClassname () const`

Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`

Returns the parent model that owns this definition.
- `bool isReportStatistics () const`

Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`

Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`

Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.
- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.
- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`

Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`

Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`

Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`

Returns plugin metadata for this formula type.
- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`

Loads a formula from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Creates a new empty formula instance.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.201.1 Constructor & Destructor Documentation

10.201.1.1 Formula()

```
Formula::Formula (
    Model * model,
    std::string name = "" )
```

Creates an empty formula definition.

Parameters

<i>model</i>	Parent model.
<i>name</i>	Initial formula name.

10.201.2 Member Function Documentation

10.201.2.1 `_check()`

```
bool Formula::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.201.2.2 `_createAttachedAttributes()`

```
void Formula::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes @ToDo: **NOT AN ATTRIBUTE**

Reimplemented from [ModelDataDefinition](#).

10.201.2.3 `_loadInstance()`

```
bool Formula::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Load formula expressions map.

Reimplemented from [ModelDataDefinition](#).

10.201.2.4 `_saveInstance()`

```
void Formula::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Persist formula expressions map.

Reimplemented from [ModelDataDefinition](#).

10.201.2.5 `getExpression()`

```
std::string Formula::getExpression (
    std::string index = "" )
```

Returns the expression associated with an index.

Parameters

<i>index</i>	Optional index key used for vectorized formulas.
--------------	--

Returns

Stored formula text, or an empty string when absent.

10.201.2.6 `GetPluginInformation()`

```
PluginInformation * Formula::GetPluginInformation ( ) [static]
```

Returns plugin metadata for this formula type.

Returns

[Plugin](#) information block.

10.201.2.7 `getValue()`

```
double Formula::getValue (
    std::string index )
```

Evaluates the expression associated with an index.

Parameters

<i>index</i>	Optional index key used for vectorized formulas.
--------------	--

Returns

Numeric value of the indexed expression.

10.201.2.8 `LoadInstance()`

```
ModelDataDefinition * Formula::LoadInstance (
```

```

    Model * model,
    PersistenceRecord * fields ) [static]

```

Loads a formula from serialized fields.

Parameters

<i>model</i>	Parent model.
<i>fields</i>	Serialized fields.

Returns

Newly created formula instance.

10.201.2.9 NewInstance()

```

ModelDataDefinition * Formula::NewInstance (
    Model * model,
    std::string name = "" ) [static]

```

Creates a new empty formula instance.

Parameters

<i>model</i>	Parent model.
<i>name</i>	Initial formula name.

Returns

Newly created formula instance.

10.201.2.10 setExpression()

```

void Formula::setExpression (
    std::string formulaExpression,
    std::string index = "" )

```

Stores or updates the expression associated with an index.

Parameters

<i>formulaExpression</i>	Formula text.
<i>index</i>	Optional index key used for vectorized formulas.

10.201.2.11 show()

```

std::string Formula::show ( ) [override], [virtual]

```

Returns a textual representation of the formula.

Returns

Human-readable formula summary.

Reimplemented from [ModelDataDefinition](#).

10.201.2.12 size()

```
unsigned int Formula::size ( )
```

Returns the number of indexed expressions stored in this formula.

Returns

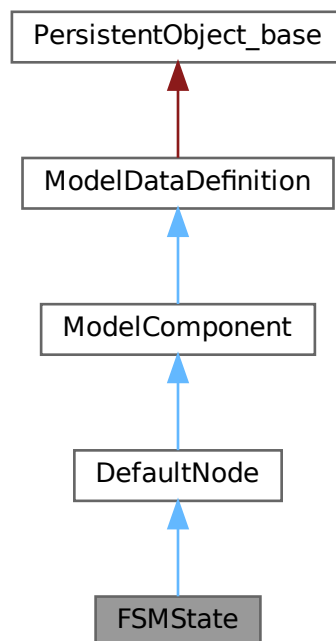
Number of expression slots.

The documentation for this class was generated from the following files:

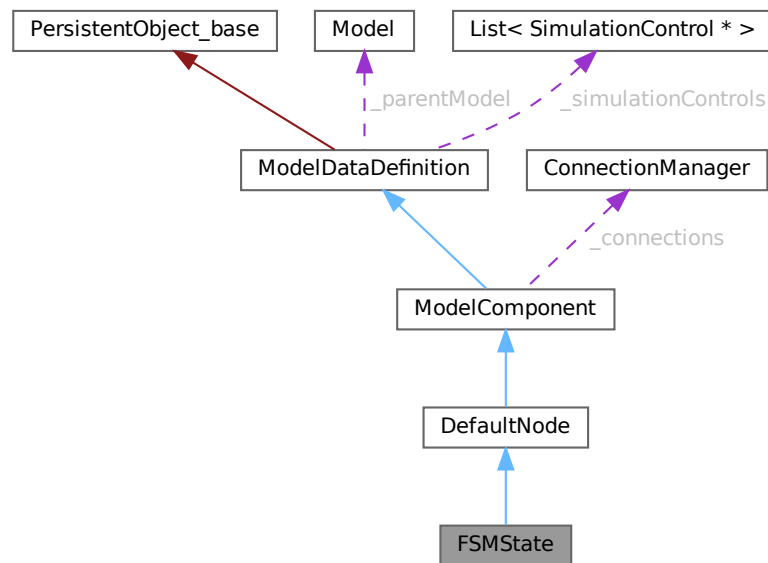
- plugins/data/Logic/Formula.h
- plugins/data/Logic/Formula.cpp

10.202 FSMState Class Reference

Inheritance diagram for FSMState:



Collaboration diagram for FSMState:



Public Member Functions

- **FSMState** ([Model](#) *model, std::string name="")
- void **setEntryActionExpression** (std::string expression)
- std::string **getEntryActionExpression** () const
- void **setExitActionExpression** (std::string expression)
- std::string **getExitActionExpression** () const

Public Member Functions inherited from [DefaultNode](#)

- [DefaultNode](#) ([Model](#) *model, std::string componentTypename, std::string name)
constructors
- **DefaultNode** ([Model](#) *model, std::string name="")
- void **addTransition** ([DefaultNodeTransition](#) *transition)
new public user methods for this component
- void **removeTransition** ([DefaultNodeTransition](#) *transition)
- [List](#)< [DefaultNodeTransition](#) * > * **getTransitions** () const
- void **setInitialNode** (bool initialNode)
- bool **isInitialNode** () const
- void **setFinalNode** (bool finalNode)
- bool **isFinalNode** () const
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.

- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- List< `SimulationControl` * > * `getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelSpecific`, bool `traceLevelSpecificEnabled`)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation` * `GetPluginInformation` ()
- static `ModelComponent` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`)
- static `ModelDataDefinition` * `NewInstance` (`Model` *`model`, std::string `name`="")

Static Public Member Functions inherited from `DefaultNode`

- static `PluginInformation` * `GetPluginInformation` ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `ModelComponent` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`)
- static `ModelDataDefinition` * `NewInstance` (`Model` *`model`, std::string `name`="")

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent` *`component`)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent` *`component`)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord` *`fields`, `ModelComponent` *`component`)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event` *`event`)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [DefaultNode](#)

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)

- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.202.1 Member Function Documentation

10.202.1.1 `_loadInstance()`

```
bool FSMState::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [DefaultNode](#).

10.202.1.2 `_saveInstance()`

```
void FSMState::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [DefaultNode](#).

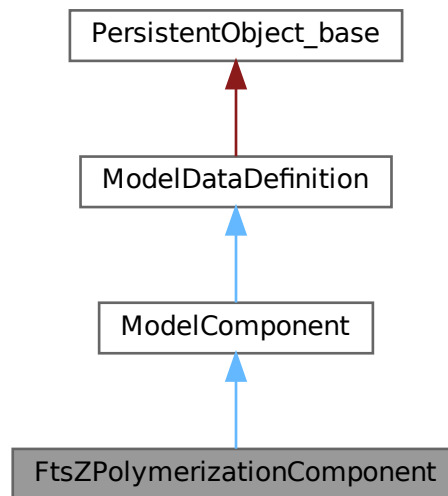
The documentation for this class was generated from the following files:

- `plugins/components/ModalModel/FSMState.h`
- `plugins/components/ModalModel/FSMState.cpp`

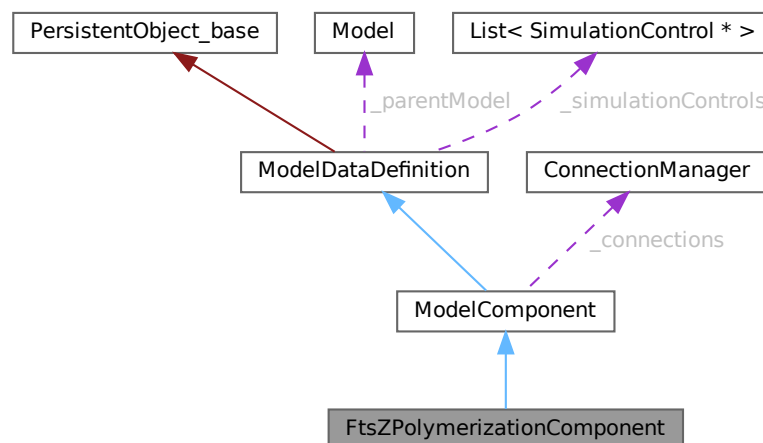
10.203 FtsZPolymerizationComponent Class Reference

```
#include <plugins/components/WholeCellModeling/FtsZPolymerizationComponent.h>
```

Inheritance diagram for FtsZPolymerizationComponent:



Collaboration diagram for FtsZPolymerizationComponent:



Public Member Functions

- **FtsZPolymerizationComponent** (`Model` *model, std::string name="")
- void **setWholeCellState** (`WholeCellState` *state)
- `WholeCellState` * **getWholeCellState** () const
- void **setActivationFwd** (double rate)
- double **getActivationFwd** () const

- void **setActivationRev** (double rate)
- double **getActivationRev** () const
- void **setVolumeNorm** (double norm)
- double **getVolumeNorm** () const
- void **setDeltaT** (double dt)
- double **getDeltaT** () const
- void **setFtsZRingKey** (std::string key)
- std::string **getFtsZRingKey** () const
- void **setFtsZMonomerKey** (std::string key)
- std::string **getFtsZMonomerKey** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

- Returns the runtime class name used by the plugin system.*
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from ModelComponent

- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static bool Check (ModelComponent *component)`
Validates a component instance.
- `static void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- `static void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- `static void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.203.1 Detailed Description

FtsZ ring polymerization dynamics for whole-cell division timing.

Implements a simplified two-state Euler ODE for the FtsZ ring completion fraction $f \in [0,1]$, stored as a per-mille integer (0–1000) in [WholeCellState](#) under the key ftsZRingKey (default "FtsZ_ring_completion").

Monomer availability is read from [WholeCellState](#) under ftsZMonomerKey (default "FtsZ_monomer"). The reference count N_{ref} is captured on the first `_onDispatchEvent` call (N_{m} at $t=0$) and held constant; the fraction $N_{\text{m}}/N_{\text{ref}}$ scales the polymerization rate (lower monomer count = slower growth).

ODE (per step): $f_{\text{new}} = f + (k_{\text{poly}} * \phi * (1-f) - k_{\text{depoly}} * f) * \text{deltaT}$ where $k_{\text{poly}} = \text{activationFwd} * \text{volumeNorm}$ [1/s] $k_{\text{depoly}} = \text{activationRev} * \text{volumeNorm}$ [1/s] $\phi = N_{\text{m}} / N_{\text{ref}}$ [dimensionless monomer availability]

Default $\text{volumeNorm} = 3.94\text{e-}5$ is calibrated so that $k_{\text{poly}} \approx 4.3\text{e-}5/\text{s}$, which gives $\sim 35\%$ ring completion at $t \approx 12\,000\text{ s}$ — consistent with the *M. genitalium* cell cycle. `activationFwd` and `activationRev` are read from [WholeCellState](#) "param.ftsZActivationFwd"/"param.ftsZActivationRev" when `loadParameters()` has been called; otherwise the component uses its own stored defaults.

10.203.2 Member Function Documentation

10.203.2.1 `_check()`

```
bool FtsZPolymerizationComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.203.2.2 `_createEditableDataDefinitions()`

```
void FtsZPolymerizationComponent::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.203.2.3 `_initBetweenReplications()`

```
void FtsZPolymerizationComponent::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.203.2.4 `_loadInstance()`

```
bool FtsZPolymerizationComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.203.2.5 `_onDispatchEvent()`

```
void FtsZPolymerizationComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for `ModelComponents`, not for `ModelDataElements`.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.203.2.6 `_saveInstance()`

```
void FtsZPolymerizationComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.203.2.7 `show()`

```
std::string FtsZPolymerizationComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

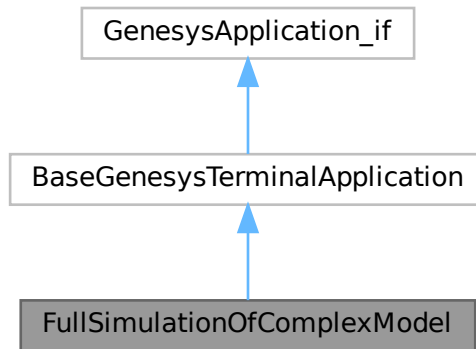
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

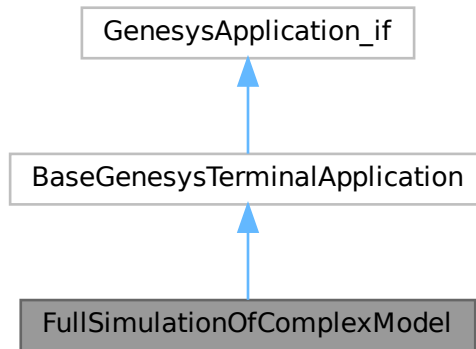
- plugins/components/WholeCellModeling/FtsZPolymerizationComponent.h
- plugins/components/WholeCellModeling/FtsZPolymerizationComponent.cpp

10.204 FullSimulationOfComplexModel Class Reference

Inheritance diagram for FullSimulationOfComplexModel:



Collaboration diagram for FullSimulationOfComplexModel:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.204.1 Member Function Documentation

10.204.1.1 main()

```
int FullSimulationOfComplexModel::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

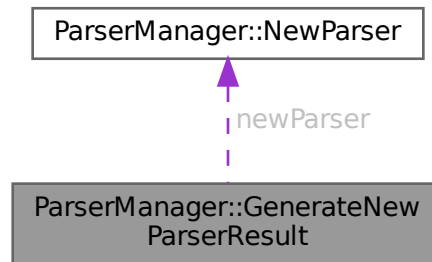
- applications/modelSpecific/teaching/FullSimulationOfComplexModel.h
- applications/modelSpecific/teaching/FullSimulationOfComplexModel.cpp

10.205 ParserManager::GenerateNewParserResult Struct Reference

Result bundle for parser generation requests.

```
#include <kernel/simulator/ParserManager.h>
```

Collaboration diagram for ParserManager::GenerateNewParserResult:



Public Attributes

- bool **result** = false
- std::string **bisonMessages**
- std::string **lexMessages**
- std::string **compilationMessages**
- [NewParser](#) **newParser**

10.205.1 Detailed Description

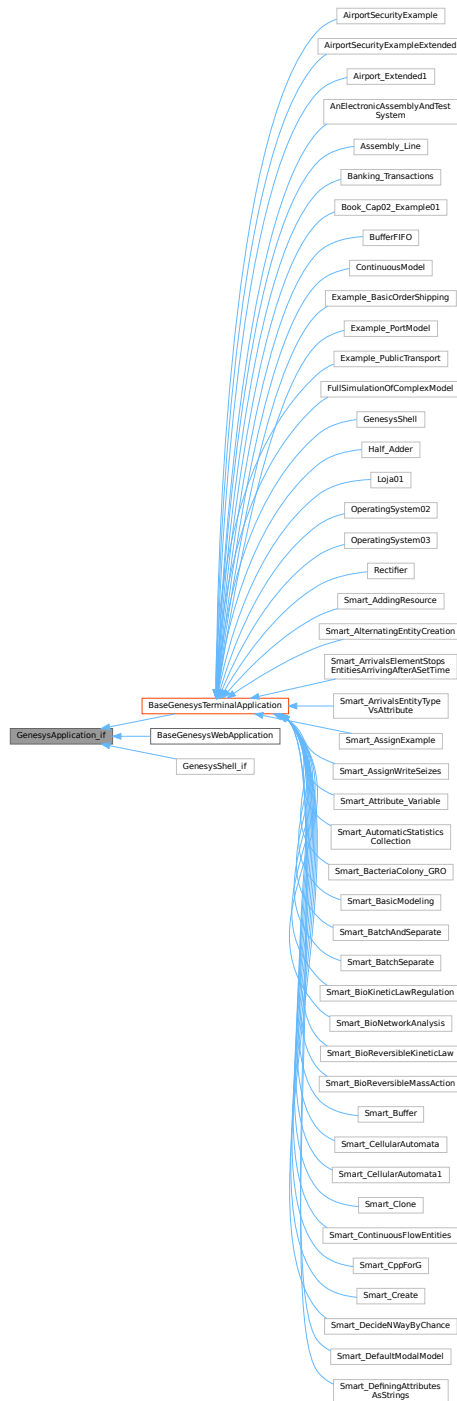
Result bundle for parser generation requests.

The documentation for this struct was generated from the following file:

- kernel/simulator/ParserManager.h

10.206 GenesysApplication_if Class Reference

Inheritance diagram for GenesysApplication_if:



Public Member Functions

- virtual int [main](#) (int argc, char **argv)=0

10.206.1 Member Function Documentation

10.206.1.1 main()

```
virtual int GenesysApplication_if::main (
    int argc,
    char ** argv ) [pure virtual]
```

Implemented in [Airport_Extended1](#), [Assembly_Line](#), [Banking_Transactions](#), [Example_BasicOrderShipping](#), [Example_PortModel](#), [Example_PublicTransport](#), [Smart_AddingResource](#), [Smart_AlternatingEntityCreation](#), [Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime](#), [Smart_AutomaticStatisticsCollection](#), [Smart_ContinuousFlowEntities](#), [Smart_Create](#), [Smart_DecideNWayByChance](#), [Smart_DefiningAttributesAsStrings](#), [Smart_DefiningControlLogic](#), [Smart_DefiningResourceCapacity](#), [Smart_EntitiesProcessedByPriority](#), [Smart_EvaluatingConditionsBeforeEnteringQueue](#), [Smart_Expression](#), [Smart_MaxArrivalsField](#), [Smart_ModelRunUntil1000Parts](#), [Smart_ModuleDisplayVariables](#), [Smart_OverlappingResources](#), [Smart_ParallelProcessingOfEntities](#), [Smart_PlacingEntitiesInQueueSets](#), [Smart_PriorityExample](#), [Smart_ProcessArena](#), [Smart_Record_Arena](#), [Smart_RemovingAndReorderingEntitiesInAQueue](#), [Smart_ResourceSets](#), [Smart_SeizingMultipleSimultaneously](#), [Smart_SelectingRouteBasedOnProbability](#), [Smart_SelectingShorterQueue](#), [Smart_SynchronizingParallelEntities](#), [Smart_WaitForSignal](#), [Smart_MarkovChain](#), [Smart_Buffer](#), [Smart_Clone](#), [Smart_Delay](#), [Smart_Failures](#), [Smart_HoldSearchRemove](#), [Smart_Process](#), [Smart_ProcessSet](#), [Smart_SeizeDelayRelease](#), [Smart_SeizeDelayReleaseMany](#), [Smart_SeizeDelayReleaseNoDataDefs](#), [Smart_R_Simulator](#), [Smart_Record](#), [Smart_Attribute_Variable](#), [Smart_ModelInfoModelSimulation](#), [Smart.Parser](#), [Smart.ParserModelFunctions](#), [Smart_RouteStation](#), [Smart_CellularAutomata](#), [Smart_CellularAutomata1](#), [Smart_DefaultModalModel](#), [Smart_Dummy](#), [Smart_ModalModel](#), [Smart_OnEvent](#), [Smart_WaitScanCondition](#), [Smart_WaitSignal](#), [FullSimulationOfComplexModel](#), [Half_Adder](#), [Rectifier](#), and [BaseGenesysWebApplication](#).

The documentation for this class was generated from the following file:

- applications/GenesysApplication_if.h

10.207 genesys_mcp.client.GenesysClient Class Reference

Public Member Functions

- None **__init__** (self, str base_url="http://localhost:8080", float timeout=30.0)
- dict[str, Any] **health** (self)
- dict[str, Any] **create_session** (self)
- dict[str, Any] **simulator_info** (self)
- dict[str, Any] **new_model** (self)
- dict[str, Any] **get_model** (self)
- dict[str, Any] **save_model** (self, str filename)
- dict[str, Any] **load_model** (self, str filename)
- dict[str, Any] **simulation_status** (self)
- dict[str, Any] **configure_simulation** (self, int number_of_replications, float replication_length, float warm←_up_period, bool pause_on_event=False, bool pause_on_replication=False, bool initialize_statistics=True, bool initialize_system=True)
- dict[str, Any] **run_simulation** (self)
- dict[str, Any] **step_simulation** (self)
- dict[str, Any] **worker_info** (self)
- dict[str, Any] **worker_capabilities** (self)
- dict[str, Any] **import_model_language** (self, str model_text)
- dict[str, Any] **create_job** (self)
- dict[str, Any] **get_job** (self, str job_id)
- dict[str, Any] **run_job** (self, str job_id)
- dict[str, Any] **get_job_result** (self, str job_id)
- None **close** (self)
- [GenesysClient](#) **__enter__** (self)
- None **__exit__** (self, *object _)

Protected Member Functions

- dict[str, str] **_headers** (self)
- Any **_get** (self, str path)
- Any **_post** (self, str path, Any body=None)
- Any **_post_text** (self, str path, str text)

Protected Attributes

- **_base_url**
- **_http**
- **_token**

10.207.1 Detailed Description

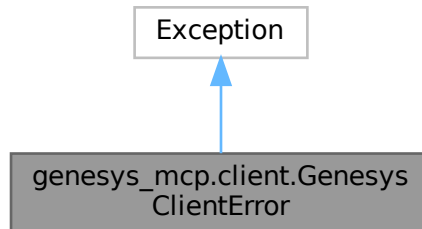
Stateful HTTP client for one Genesys worker-app session.

The documentation for this class was generated from the following file:

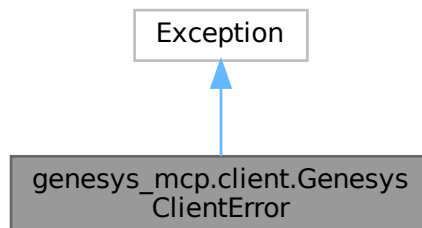
- applications/mcp/genesys_mcp/client.py

10.208 genesys_mcp.client.GenesysClientError Class Reference

Inheritance diagram for genesys_mcp.client.GenesysClientError:



Collaboration diagram for genesys_mcp.client.GenesysClientError:



Public Member Functions

- None `__init__` (self, int status_code, str message)

Public Attributes

- `status_code`

10.208.1 Detailed Description

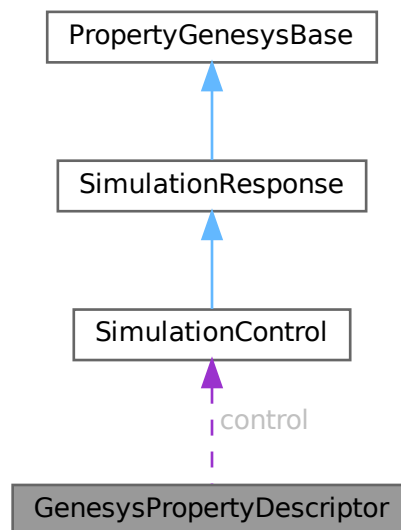
Raised when the Genesys REST API returns an error.

The documentation for this class was generated from the following file:

- `applications/mcp/genesys_mcp/client.py`

10.209 GenesysPropertyDescriptor Struct Reference

Collaboration diagram for GenesysPropertyDescriptor:



Public Attributes

- [SimulationControl](#) * **control** = nullptr
- std::string **ownerClassName**
- std::string **ownerElementName**
- std::string **displayName**
- std::string **technicalTypeName**
- GenesysPropertyKind **kind** = GenesysPropertyKind::Unknown
- bool **readOnly** = true
- bool **isList** = false
- bool **isClass** = false
- bool **isEnum** = false
- bool **isInlineObject** = false
- bool **isModelDataDefinitionReference** = false
- bool **supportsInlineExpansion** = false
- bool **supportsListEditor** = false
- bool **supportsExistingObjectSelection** = false
- bool **supportsObjectCreation** = false
- bool **supportsNewListElementCreation** = false
- SimulationControlEditorHint **editorHint** = SimulationControlEditorHint::Default
- std::string **currentValue**
- std::vector< std::string > **choices**
- std::string **currentReferenceType**
- std::vector< std::string > **creatableReferenceTypes**
- std::string **currentListElementType**
- std::vector< std::string > **creatableListElementTypes**

The documentation for this struct was generated from the following file:

- kernel/simulator/GenesysPropertyIntrospection.h

10.210 GenesysPropertyIntrospection Class Reference

Static Public Member Functions

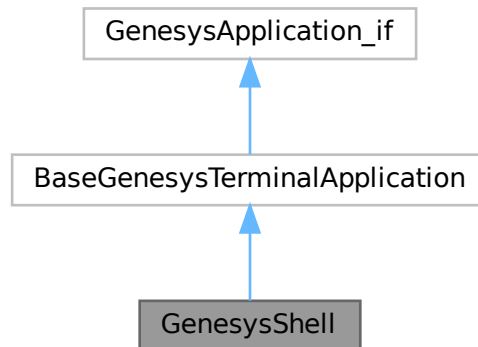
- static [GenesysPropertyDescriptor](#) **describe** ([SimulationControl](#) *control)
- static std::vector< [GenesysPropertyDescriptor](#) > **describe** (List< [SimulationControl](#) * > *controls)
- static GenesysPropertyKind **deduceKind** (const [SimulationControl](#) *control)
- static bool **setValue** ([SimulationControl](#) *control, const std::string &value, bool remove=false, std::string *errorMessage=nullptr)
- static std::vector< [SimulationControl](#) * > **children** ([SimulationControl](#) *control, int index=0)

The documentation for this class was generated from the following files:

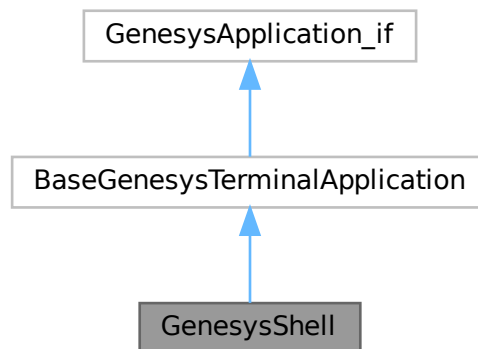
- kernel/simulator/GenesysPropertyIntrospection.h
- kernel/simulator/GenesysPropertyIntrospection.cpp

10.211 GenesysShell Class Reference

Inheritance diagram for GenesysShell:



Collaboration diagram for GenesysShell:



Public Member Functions

- **GenesysShell** (const [GenesysShell](#) &orig)
- virtual int [main](#) (int argc, char **argv) override
- void **cmdScript** ()
- void **cmdExecuteScript** ()
- void **cmdHelp** ()
- void **cmdQuit** ()
- void **cmdBash** ()
- void **cmdFacade** ()

- void **cmdLicence** ()
- void **cmdInfo** ()
- void **cmdSim** ()
- void **cmdData** ()
- void **cmdComponent** ()
- void **cmdExperiment** ()
- void **cmdParser** ()
- void **cmdTraceLevel** ()
- void **cmdPlugin** ()
- void **cmdSimulation** ()
- void **cmdReplication** ()
- void **cmdModel** ()
- void **cmdModelLoad** ()
- void **cmdModelSave** ()

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.211.1 Member Function Documentation

10.211.1.1 main()

```
int GenesysShell::main (
    int argc,
    char ** argv ) [override], [virtual]
```

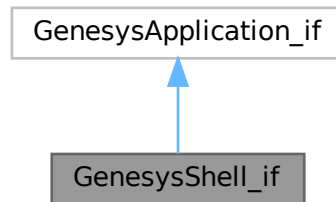
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

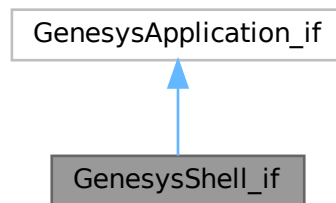
- applications/shell/GenesysShell.h
- applications/shell/GenesysShell.cpp

10.212 GenesysShell_if Class Reference

Inheritance diagram for GenesysShell_if:



Collaboration diagram for GenesysShell_if:



Public Member Functions

- virtual void **openModel** (std::string filename)=0
- virtual void **saveModelAs** (std::string filename)=0
- virtual void **saveModel** ()=0
- virtual void **listElements** ()=0
- virtual void **listComponents** ()=0
- virtual void **listHosts** ()=0
- virtual void **listPlugins** ()=0
- virtual void **deleteTraceFiles** ()=0
- virtual void **traceLevel** (TraceManager::TraceLevel tracelevel)=0
- virtual void **addPlugin** (std::string filename)=0
- virtual void **addFromFile** (std::string filename)=0
- virtual void **readCommandsFromFile** (std::string filename)=0
- virtual void **redirectTrace** (std::string trace, std::string dest, std::string filename)=0
- virtual void **closeModel** ()=0
- virtual void **createModel** ()=0
- virtual void **execLinuxCommand** (std::string command)=0

- virtual void **verboseMode** (bool on)=0
- virtual void **check** ()=0
- virtual void **getGenesysInfo** ()=0
- virtual void **getCommandLine** ()=0
- virtual void **sendFile** (std::string filename, std::string hostname, std::string portname)=0
- virtual void **setActivationCode** (std::string code)=0
- virtual void **receiveFile** (std::string filename)=0
- virtual void **startSimulation** ()=0
- virtual void **stepSimulation** ()=0
- virtual void **stopSimulation** ()=0
- virtual void **showInit** ()=0
- virtual void **showHelp** ()=0
- virtual void **showHostName** ()=0

Public Member Functions inherited from [GenesysApplication_if](#)

- virtual int [main](#) (int argc, char **argv)=0

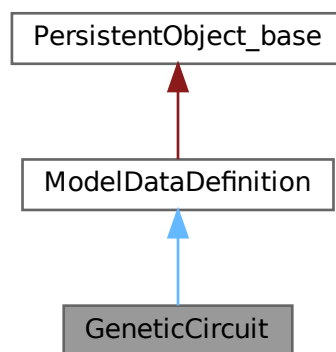
The documentation for this class was generated from the following file:

- applications/shell/GenesysShell_if.h

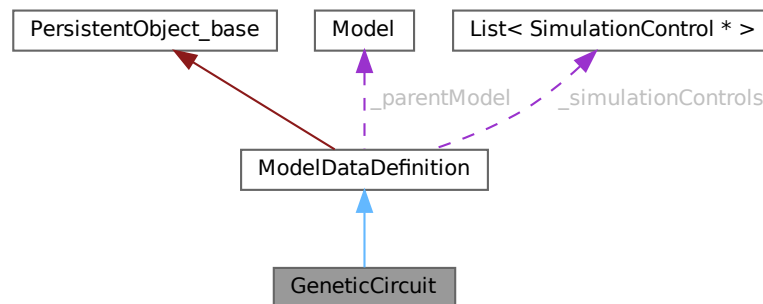
10.213 GeneticCircuit Class Reference

```
#include <plugins/data/BiochemicalSimulation/GeneticCircuit.h>
```

Inheritance diagram for GeneticCircuit:



Collaboration diagram for GeneticCircuit:



Public Member Functions

- **GeneticCircuit** ([Model](#) *model, std::string name="")
- void **addPart** (std::string partName)
- void **addRegulation** (std::string regulationName)
- void **clearParts** ()
- void **clearRegulations** ()
- const std::vector< std::string > & **getPartNames** () const
- const std::vector< std::string > & **getRegulationNames** () const
- void **setHostOrganism** (std::string hostOrganism)
- std::string **getHostOrganism** () const
- void **setCompartment** (std::string compartment)
- std::string **getCompartment** () const
- void **setEnabled** (bool enabled)
- bool **isEnabled** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↵ IntoModel=true)
- *Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- *Releases resources owned by this data definition.*
- Util::identification **getId** () const
- *Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- *Updates the logical name of this instance.*
- const std::string & **getName** () const
- *Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
- *Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const

- Returns the user-visible label of this instance.*
- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager::Level](#) **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.213.1 Detailed Description

Aggregate genetic circuit definition.

A [GeneticCircuit](#) stores an ordered sequence of genetic parts and a set of regulatory relationships by name so it can be persisted independently from runtime object pointers.

10.213.2 Member Function Documentation

10.213.2.1 `_check()`

```
bool GeneticCircuit::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.213.2.2 `_loadInstance()`

```
bool GeneticCircuit::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.213.2.3 `_saveInstance()`

```
void GeneticCircuit::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.213.2.4 `show()`

```
std::string GeneticCircuit::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

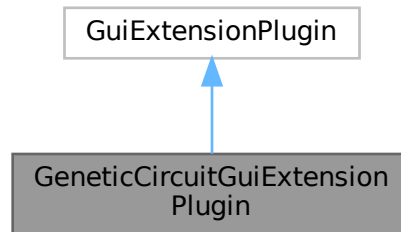
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

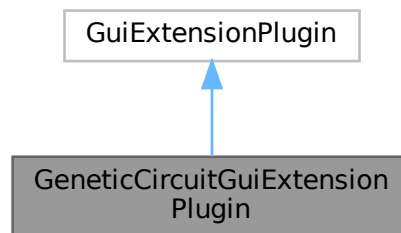
- plugins/data/BiochemicalSimulation/GeneticCircuit.h
- plugins/data/BiochemicalSimulation/GeneticCircuit.cpp

10.214 GeneticCircuitGuiExtensionPlugin Class Reference

Inheritance diagram for GeneticCircuitGuiExtensionPlugin:



Collaboration diagram for GeneticCircuitGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

10.214.1 Member Function Documentation

10.214.1.1 displayName()

```
std::string GeneticCircuitGuiExtensionPlugin::displayName ( ) const [inline], [override],
[virtual]
```

Implements [GuiExtensionPlugin](#).

10.214.1.2 extensionId()

```
std::string GeneticCircuitGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.214.1.3 registerContributions()

```
void GeneticCircuitGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.214.1.4 requiredModelPlugins()

```
std::vector< std::string > GeneticCircuitGuiExtensionPlugin::requiredModelPlugins ( ) const
[inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.214.1.5 version()

```
std::string GeneticCircuitGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

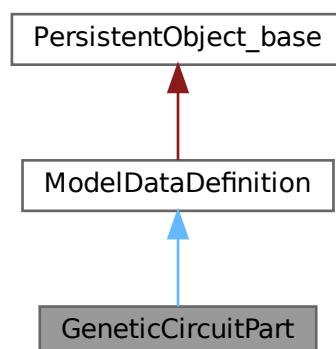
The documentation for this class was generated from the following file:

- applications/gui/genesys/extensions/GeneticCircuitGuiExtensionPlugin.cpp

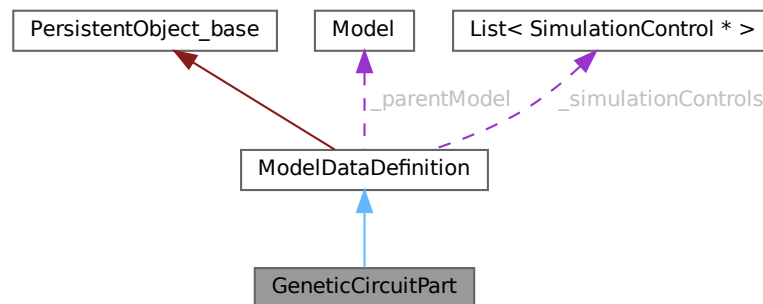
10.215 GeneticCircuitPart Class Reference

```
#include <plugins/data/BiochemicalSimulation/GeneticCircuitPart.h>
```

Inheritance diagram for GeneticCircuitPart:



Collaboration diagram for GeneticCircuitPart:



Public Member Functions

- **GeneticCircuitPart** ([Model](#) *model, std::string name="")
- void **setPartType** (std::string partType)
- std::string **getPartType** () const
- void **setSequence** (std::string sequence)
- std::string **getSequence** () const
- void **setProductSpeciesName** (std::string productSpeciesName)
- std::string **getProductSpeciesName** () const
- void **setCopyNumber** (double copyNumber)
- double **getCopyNumber** () const
- void **setBasalExpressionRate** (double basalExpressionRate)
- double **getBasalExpressionRate** () const
- void **setDegradationRate** (double degradationRate)
- double **getDegradationRate** () const
- void **setEnabled** (bool enabled)
- bool **isEnabled** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- Releases resources owned by this data definition.*
- Util::identification [getId](#) () const
- Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & [getName](#) () const
- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)

- Updates the user-visible label of this instance.*

 - `const std::string & getLabel () const`

Returns the user-visible label of this instance.
- `const std::string & getClassname () const`

Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`

Returns the parent model that owns this definition.
- `bool isReportStatistics () const`

Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`

Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`

Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.
- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.
- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`

Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`

Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`

Enables or disables the specific trace level.

Static Public Member Functions

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.215.1 Detailed Description

Genetic circuit structural part definition.

This data definition represents a reusable genetic part (promoter, RBS, CDS, terminator, etc.) and optionally links it to the [BioSpecies](#) produced by the part expression flow.

10.215.2 Member Function Documentation

10.215.2.1 `_check()`

```
bool GeneticCircuitPart::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.215.2.2 `_loadInstance()`

```
bool GeneticCircuitPart::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.215.2.3 `_saveInstance()`

```
void GeneticCircuitPart::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.215.2.4 `show()`

```
std::string GeneticCircuitPart::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

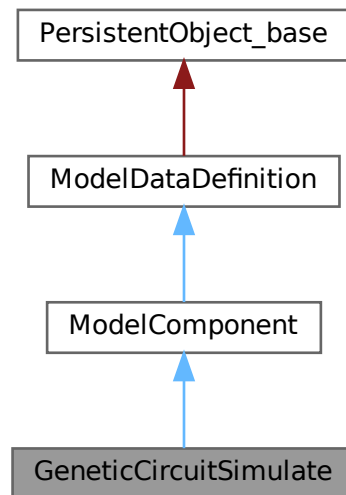
The documentation for this class was generated from the following files:

- plugins/data/BiochemicalSimulation/GeneticCircuitPart.h
- plugins/data/BiochemicalSimulation/GeneticCircuitPart.cpp

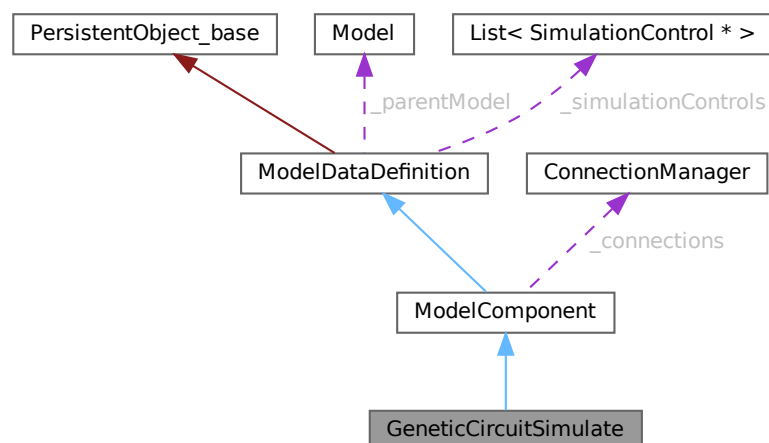
10.216 GeneticCircuitSimulate Class Reference

```
#include <plugins/components/BiochemicalSimulation/GeneticCircuitSimulate.h>
```

Inheritance diagram for GeneticCircuitSimulate:



Collaboration diagram for GeneticCircuitSimulate:



Public Member Functions

- **GeneticCircuitSimulate** ([Model](#) *model, std::string name="")
- void **setGeneticCircuit** ([GeneticCircuit](#) *geneticCircuit)
- [GeneticCircuit](#) * **getGeneticCircuit** () const
- void **setStartTime** (double startTime)
- double **getStartTime** () const
- void **setStopTime** (double stopTime)
- double **getStopTime** () const
- void **setStepSize** (double stepSize)
- double **getStepSize** () const
- void **setApplyRegulation** (bool applyRegulation)
- bool **getApplyRegulation** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastSampleCount** (unsigned int lastSampleCount)
- unsigned int **getLastSampleCount** () const
- void **setLastTotalExpression** (double lastTotalExpression)
- double **getLastTotalExpression** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- bool **simulate** (std::string &errorMessage)
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
 - Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
 - Releases resources owned by this data definition.*
- Util::identification **getId** () const
 - Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
 - Updates the logical name of this instance.*
- const std::string & **getName** () const
 - Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
 - Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const
 - Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const
 - Returns the runtime class name used by the plugin system.*
- **Model** * **getParentModel** () const
 - Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const
 - Indicates whether this data definition contributes report statistics.*
- void **setReportStatistics** (bool reportStatistics)
 - Enables or disables report-statistics generation for this definition.*
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
 - Looks up a named internal child data definition.*
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
 - Returns the full map of internal child data definitions.*
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
 - Returns the full map of attached data definitions.*
- bool **hasChanged** () const
 - Indicates whether this instance has been modified since the last reset.*
- void **setHasChanged** (bool hasChanged)
 - Updates the changed flag for this data definition.*
- unsigned int **getLevel** () const
 - Returns the level in the model hierarchy where this definition lives.*
- void **setModelLevel** (unsigned int _modelLevel)
 - Sets the model nesting level for this definition.*
- List< **SimulationControl** * > * **getSimulationControls** () const
 - Returns the simulation controls owned by this data definition.*
- TraceManager::Level **getTraceLevelSpecific** () const
 - Returns the trace level that applies specifically to this definition.*
- void **setTraceLevelSpecific** (TraceManager::Level level)
 - Updates the trace level used by this definition.*
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵
Enabled=true)
 - Defines the specific trace level and optionally enables it immediately.*
- bool **isTraceLevelSpecificEnabled** () const
 - Indicates whether the specific trace level is enabled.*
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)
 - Enables or disables the specific trace level.*

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
- static bool **simulateCircuit** ([Model](#) *model, [GeneticCircuit](#) *geneticCircuit, double startTime, double stopTime, double stepSize, bool applyRegulation, [GeneticCircuitSimulationSummary](#) *summary, std::string &errorMessage)

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void **_createEditableDataDefinitions** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _initBetweenReplications ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createInternalStatisticReporters ()`
- `virtual void _createAttachedAttributes ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
- legacy helper methods used by the granular create* hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.216.1 Detailed Description

Component that simulates a [GeneticCircuit](#) over a time window.

10.216.2 Member Function Documentation

10.216.2.1 `_check()`

```
bool GeneticCircuitSimulate::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.216.2.2 `_createEditableDataDefinitions()`

```
void GeneticCircuitSimulate::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.216.2.3 `_loadInstance()`

```
bool GeneticCircuitSimulate::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.216.2.4 `_onDispatchEvent()`

```
void GeneticCircuitSimulate::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.216.2.5 _saveInstance()

```
void GeneticCircuitSimulate::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.216.2.6 show()

```
std::string GeneticCircuitSimulate::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/BiochemicalSimulation/GeneticCircuitSimulate.h
- plugins/components/BiochemicalSimulation/GeneticCircuitSimulate.cpp

10.217 GeneticCircuitSimulationSummary Struct Reference**Public Attributes**

- bool **succeeded** = false
- unsigned int **sampleCount** = 0
- double **totalExpression** = 0.0
- std::string **message** = ""

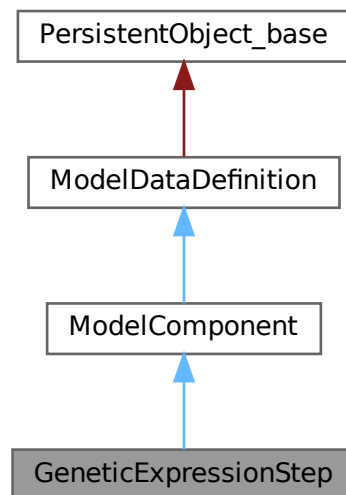
The documentation for this struct was generated from the following file:

- plugins/components/BiochemicalSimulation/GeneticCircuitSimulate.h

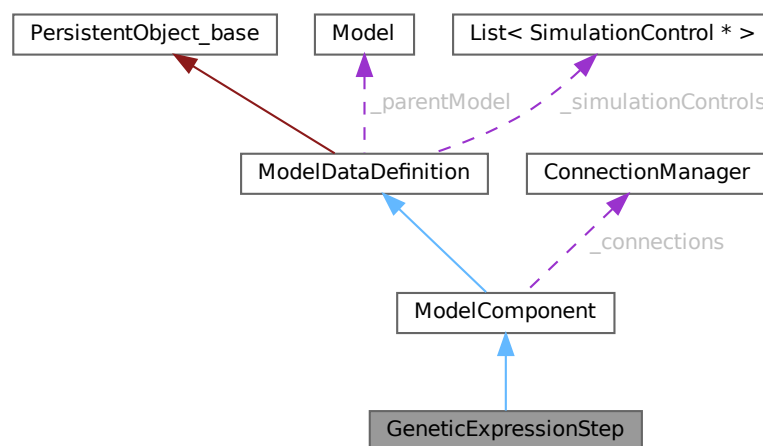
10.218 GeneticExpressionStep Class Reference

```
#include <plugins/components/BiochemicalSimulation/GeneticExpressionStep.h>
```

Inheritance diagram for GeneticExpressionStep:



Collaboration diagram for GeneticExpressionStep:



Public Member Functions

- **GeneticExpressionStep** ([Model](#) *model, std::string name="")

- void **setGeneticCircuit** ([GeneticCircuit](#) *geneticCircuit)
- [GeneticCircuit](#) * **getGeneticCircuit** () const
- void **setTimeStep** (double timeStep)
- double **getTimeStep** () const
- void **setApplyRegulation** (bool applyRegulation)
- bool **getApplyRegulation** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastTotalExpression** (double lastTotalExpression)
- double **getLastTotalExpression** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.218.1 Detailed Description

Component that executes one expression update step over a [GeneticCircuit](#).

10.218.2 Member Function Documentation

10.218.2.1 `_check()`

```
bool GeneticExpressionStep::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.218.2.2 `_createEditableDataDefinitions()`

```
void GeneticExpressionStep::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.218.2.3 `_loadInstance()`

```
bool GeneticExpressionStep::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.218.2.4 `_onDispatchEvent()`

```
void GeneticExpressionStep::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.218.2.5 _saveInstance()

```
void GeneticExpressionStep::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.218.2.6 show()

```
std::string GeneticExpressionStep::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

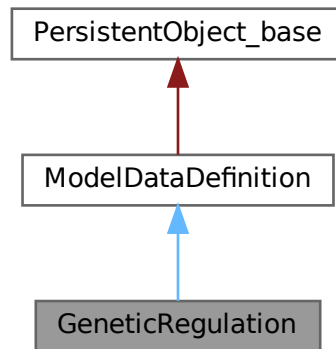
The documentation for this class was generated from the following files:

- plugins/components/BiochemicalSimulation/GeneticExpressionStep.h
- plugins/components/BiochemicalSimulation/GeneticExpressionStep.cpp

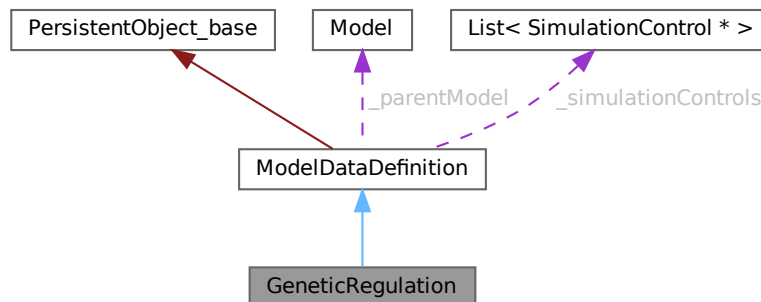
10.219 GeneticRegulation Class Reference

```
#include <plugins/data/BiochemicalSimulation/GeneticRegulation.h>
```

Inheritance diagram for GeneticRegulation:



Collaboration diagram for GeneticRegulation:



Public Member Functions

- **GeneticRegulation** ([Model](#) *model, std::string name="")
- void **setRegulatorSpeciesName** (std::string regulatorSpeciesName)
- std::string **getRegulatorSpeciesName** () const
- void **setTargetPartName** (std::string targetPartName)
- std::string **getTargetPartName** () const
- void **setRegulationType** (std::string regulationType)
- std::string **getRegulationType** () const
- void **setHillCoefficient** (double hillCoefficient)
- double **getHillCoefficient** () const

- void **setDissociationConstant** (double dissociationConstant)
- double **getDissociationConstant** () const
- void **setMaxFoldChange** (double maxFoldChange)
- double **getMaxFoldChange** () const
- void **setLeakiness** (double leakiness)
- double **getLeakiness** () const
- void **setEnabled** (bool enabled)
- bool **isEnabled** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↔IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const

- Returns the simulation controls owned by this data definition.*
- `TraceManager::Level` [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.219.1 Detailed Description

Transcriptional regulation relationship for genetic circuits.

10.219.2 Member Function Documentation

10.219.2.1 `_check()`

```
bool GeneticRegulation::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.219.2.2 `_loadInstance()`

```
bool GeneticRegulation::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.219.2.3 `_saveInstance()`

```
void GeneticRegulation::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.219.2.4 show()

```
std::string GeneticRegulation::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

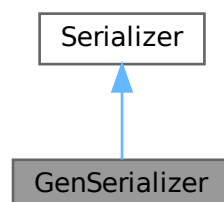
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

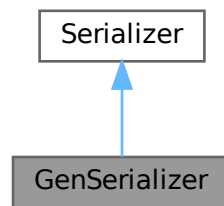
- plugins/data/BiochemicalSimulation/GeneticRegulation.h
- plugins/data/BiochemicalSimulation/GeneticRegulation.cpp

10.220 GenSerializer Class Reference

Inheritance diagram for GenSerializer:



Collaboration diagram for GenSerializer:



Public Member Functions

- **GenSerializer** ([Model](#) *model)
- [PersistenceRecord](#) * [newPersistenceRecord](#) () override
Creates an empty persistence record.
- bool [dump](#) (std::ostream &output) override
Writes formatted contents to a given output stream.
- bool [load](#) (std::istream &input) override
Attempts to load contents from a given input stream.
- bool [get](#) (const std::string &name, [PersistenceRecord](#) *entry) override
Tries to fetch an entry with the given label as a key.
- bool [put](#) (const std::string name, const std::string type, const Util::identification id, [PersistenceRecord](#) *fields) override
Upserts an entry into this structure.
- int [for_each](#) (std::function< int(const std::string &) > delegate) override
Iterates over this structure's entries, ordered by ID.

Friends

- class **PersistenceDefaultImpl2**

10.220.1 Member Function Documentation

10.220.1.1 dump()

```
bool GenSerializer::dump (
    std::ostream & output ) [override], [virtual]
```

Writes formatted contents to a given output stream.

Parameters

<i>output</i>	output stream to write to.
---------------	----------------------------

Returns

whether the current state of this structure's contents was deemed valid for serialization.

Implements [Serializer](#).

10.220.1.2 for_each()

```
int GenSerializer::for_each (
    std::function< int(const std::string &) > delegate ) [override], [virtual]
```

Iterates over this structure's entries, ordered by ID.

Parameters

<i>delegate</i>	delegate which is applied to every entry-identifying key, until it returns a non-zero value.
-----------------	--

Returns

either zero, or the first non-zero value returned by the delegate.

Implements [Serializer](#).

10.220.1.3 get()

```
bool GenSerializer::get (
    const std::string & name,
    PersistenceRecord * entry ) [override], [virtual]
```

Tries to fetch an entry with the given label as a key.

Parameters

<i>name</i>	a label which uniquely identifies an entry.
<i>entry</i>	(out-param) a pointer to a caller-owned entry, which is filled on success.

Returns

whether the entry was found and the out-param was actually written to.

Implements [Serializer](#).

10.220.1.4 load()

```
bool GenSerializer::load (
    std::istream & input ) [override], [virtual]
```

Attempts to load contents from a given input stream.

Returns

whether the input was well-formatted and loading was successful.

Implements [Serializer](#).

10.220.1.5 newPersistenceRecord()

```
PersistenceRecord * GenSerializer::newPersistenceRecord ( ) [override], [virtual]
```

Creates an empty persistence record.

Although the returned record can be freed normally with `delete`, care must be taken to only use records which come from a implementation, when configuring that implementation's contents.

Implements [Serializer](#).

10.220.1.6 put()

```
bool GenSerializer::put (
    const std::string name,
    const std::string type,
    const Util::identification id,
    PersistenceRecord * fields ) [override], [virtual]
```

Upserts an entry into this structure.

Parameters

<i>name</i>	a label which uniquely identifies the entry.
<i>type</i>	a type name.
<i>id</i>	a component/datadef ID number.
<i>fields</i>	a persistence record for the entry, whose contents will be copied if valid.

Returns

whether the entry was valid and was actually copied into this structure.

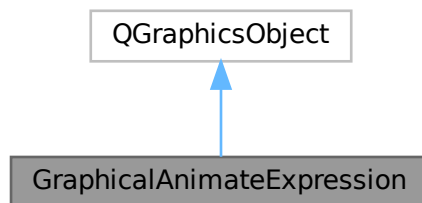
Implements [Serializer](#).

The documentation for this class was generated from the following files:

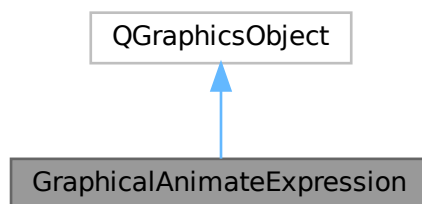
- kernel/simulator/persistence/GenSerializer.h
- kernel/simulator/persistence/GenSerializer.cpp

10.221 GraphicalAnimateExpression Class Reference

Inheritance diagram for GraphicalAnimateExpression:



Collaboration diagram for GraphicalAnimateExpression:



The documentation for this class was generated from the following files:

- applications/gui/genesys/graphics/GraphicalAnimateExpression.h
- applications/gui/genesys/graphics/GraphicalAnimateExpression.cpp

10.222 GraphicalAssociation Class Reference

The documentation for this class was generated from the following files:

- applications/gui/genesys/graphics/GraphicalAssociation.h
- applications/gui/genesys/graphics/GraphicalAssociation.cpp

10.223 GraphicalAutomaticPositioningStrategy Class Reference

Public Member Functions

- virtual [~GraphicalAutomaticPositioningStrategy](#) ()=default
Strategy interface for automatic placement of graphical data definitions.
- virtual QPointF **initialComponentLayoutOrigin** () const =0
Returns the origin used when laying out the first graphical component.
- virtual QPointF [dataDefinitionPosition](#) (const QRectF &anchorBounds, const QSizeF &childSize, int index, int count, bool upperArc, int radialLayer) const =0
Computes the position of one child item around an anchor rectangle.

10.223.1 Constructor & Destructor Documentation

10.223.1.1 ~GraphicalAutomaticPositioningStrategy()

```
virtual GraphicalAutomaticPositioningStrategy::~~GraphicalAutomaticPositioningStrategy ( )
[virtual], [default]
```

Strategy interface for automatic placement of graphical data definitions.

Implementations decide how child graphical items are laid out around an anchor item. The GUI uses this abstraction so layout policy can be selected from preferences without changing the reconstruction code that consumes the strategy.

10.223.2 Member Function Documentation

10.223.2.1 dataDefinitionPosition()

```
virtual QPointF GraphicalAutomaticPositioningStrategy::dataDefinitionPosition (
    const QRectF & anchorBounds,
    const QSizeF & childSize,
    int index,
    int count,
    bool upperArc,
    int radialLayer ) const [pure virtual]
```

Computes the position of one child item around an anchor rectangle.

Parameters

<i>anchorBounds</i>	Bounds of the item being decorated.
<i>childSize</i>	Size of the child item to place.
<i>index</i>	Zero-based child index in the current arc.
<i>count</i>	Number of children to place in the same arc.
<i>upperArc</i>	True when the child should be positioned above the anchor.
<i>radialLayer</i>	Additional radial layer for nested attachments.

Returns

Scene position where the child should be placed.

The documentation for this class was generated from the following file:

- applications/gui/genesys/services/GraphicalAutomaticPositioningStrategy.h

10.224 GraphicalAutomaticPositioningStrategyFactory Class Reference

Factory for the automatic positioning strategy selected from GUI preferences.

```
#include <applications/gui/genesys/services/GraphicalAutomaticPositioningStrategy.h>
```

Static Public Member Functions

- static std::unique_ptr< [GraphicalAutomaticPositioningStrategy](#) > **createFromPreferences** ()
Creates the preferred automatic positioning strategy instance.

10.224.1 Detailed Description

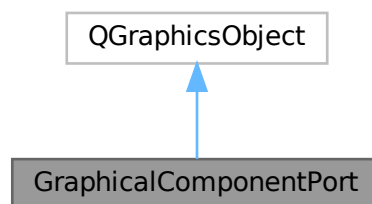
Factory for the automatic positioning strategy selected from GUI preferences.

The documentation for this class was generated from the following files:

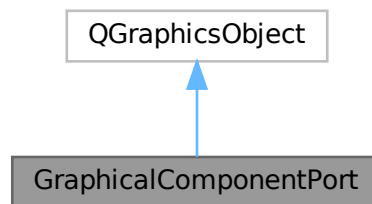
- applications/gui/genesys/services/GraphicalAutomaticPositioningStrategy.h
- applications/gui/genesys/services/GraphicalAutomaticPositioningStrategy.cpp

10.225 GraphicalComponentPort Class Reference

Inheritance diagram for GraphicalComponentPort:



Collaboration diagram for GraphicalComponentPort:



Public Member Functions

- [GraphicalComponentPort](#) ([GraphicalModelComponent](#) *componentGraph, bool **isInputPort**, unsigned int **portNum**=0, [QGraphicsItem](#) *parent=nullptr)
Visual port attached to a graphical component for connection routing.
- [QRectF](#) **boundingRect** () const override
Returns the local bounding rectangle used for hit testing and painting.
- void **paint** ([QPainter](#) *painter, const [QStyleOptionGraphicsItem](#) *option, [QWidget](#) *widget) override
Paints the port handle using the component and selection styling.
- qreal **width** () const
Returns the logical width used by the port geometry.
- qreal **height** () const
Returns the logical height used by the port geometry.
- void **addGraphicalConnection** ([GraphicalConnection](#) *connection)
Registers one connection attached to this port.
- void **removeGraphicalConnection** ([GraphicalConnection](#) *connection)
Unregisters one connection attached to this port.
- unsigned int **portNum** () const
Returns the port number within the owning component.
- bool **isInputPort** () const
Returns true when this is an input port.
- [GraphicalModelComponent](#) * **graphicalComponent** () const
Returns the owning graphical component.
- [QList](#)< [GraphicalConnection](#) * > * **getConnections** () const
Returns the list of connections currently attached to this port.

Protected Member Functions

- virtual [QVariant](#) **itemChange** ([QGraphicsItem::GraphicsItemChange](#) change, const [QVariant](#) &value) override
Keeps the port aligned when its parent component moves or is reparented.

10.225.1 Constructor & Destructor Documentation

10.225.1.1 GraphicalComponentPort()

```
GraphicalComponentPort::GraphicalComponentPort (
    GraphicalModelComponent * componentGraph,
    bool isInputPort,
    unsigned int portNum = 0,
    QGraphicsItem * parent = nullptr )
```

Visual port attached to a graphical component for connection routing.

Ports are the connection endpoints used by the scene to attach [GraphicalConnection](#) items to a component. The port item keeps track of connected edges and exposes enough geometry for the scene to render and route links consistently.

The documentation for this class was generated from the following files:

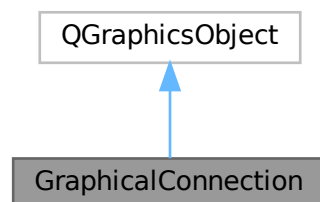
- applications/gui/genesys/graphics/GraphicalComponentPort.h
- applications/gui/genesys/graphics/GraphicalComponentPort.cpp

10.226 GraphicalConnection Class Reference

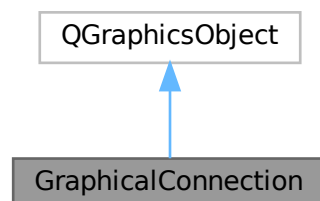
Graphical edge that represents model-level component connections.

```
#include <applications/gui/genesys/graphics/GraphicalConnection.h>
```

Inheritance diagram for GraphicalConnection:



Collaboration diagram for GraphicalConnection:



Public Types

- enum class **ConnectionType** : int { **HORIZONTAL** = 1 , **VERTICAL** = 2 , **DIRECT** = 3 , **USERDEFINED** = 4 }

Public Member Functions

- [GraphicalConnection](#) ([GraphicalComponentPort](#) *sourceGraphicalPort, [GraphicalComponentPort](#) *destination↔
GraphicalPort, unsigned int portSourceConnection=0, unsigned int portDestinationConnection=0, QColor
color=Qt::black, QGraphicsItem *parent=nullptr)
Creates a graphical connection between source and destination ports.
- GraphicalConnection** (const [GraphicalConnection](#) &orig)
Copy constructor used by restoration and clipboard flows.
- virtual ~**GraphicalConnection** ()
Releases the connection item and its auxiliary routing state.
- QRectF **boundingRect** () const override
Returns item bounding rectangle for Qt painting system.
- QPainterPath **shape** () const override
Returns the precise clickable shape around the visible connection path.
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override
Paints connection path and selection handles.
- [Connection](#) * **getSource** () const
Returns auxiliary source [Connection](#) metadata.
- [Connection](#) * **getDestination** () const
Returns auxiliary destination [Connection](#) metadata.
- [GraphicalComponentPort](#) * **getSourceGraphicalPort** ()
Returns source graphical port.
- [GraphicalComponentPort](#) * **getDestinationGraphicalPort** ()
Returns destination graphical port.
- [GraphicalModelComponent](#) * **getSourceGraphicalComponent** () const
Returns source graphical component resolved from source port when available.
- [GraphicalModelComponent](#) * **getDestinationGraphicalComponent** () const
Returns destination graphical component resolved from destination port when available.
- void **updateDimensionsAndPosition** ()
Recomputes local geometry based on current port positions.
- [GraphicalConnection::ConnectionType](#) **connectionType** () const
Returns configured connection routing type.
- unsigned int **getPortSourceConnection** () const
Returns source output port index associated with model connection.
- unsigned int **getPortDestinationConnection** () const
Returns destination input port index associated with model connection.
- void **setConnectionType** ([GraphicalConnection::ConnectionType](#) newConnectionType)
Sets routing type used during painting.
- QList< QPointF > **getPoints** () const
Returns sampled scene points used for serialization/inspection.
- bool **usesCurvedStyle** () const
Returns true when the current visual style uses curved model connection paths.
- QPainterPath **animationPathForImage** (qreal imageWidth, qreal imageHeight) const
Returns the scene path followed by entity animation in modern connection style.

Protected Member Functions

- virtual bool **sceneEvent** (QEvent *event) override

10.226.1 Detailed Description

Graphical edge that represents model-level component connections.

[GraphicalConnection](#) links two [GraphicalComponentPort](#) objects and keeps auxiliary [Connection](#) metadata synchronized with the simulator model.

The item is also used for the diagram-level routing that the GUI derives from the model structure when rendering connections inside the scene.

Todo Evaluate decoupling model metadata from this item to keep this class purely visual.

10.226.2 Constructor & Destructor Documentation

10.226.2.1 GraphicalConnection()

```
GraphicalConnection::GraphicalConnection (
    GraphicalComponentPort * sourceGraphicalPort,
    GraphicalComponentPort * destinationGraphicalPort,
    unsigned int portSourceConnection = 0,
    unsigned int portDestinationConnection = 0,
    QColor color = Qt::black,
    QGraphicsItem * parent = nullptr )
```

Creates a graphical connection between source and destination ports.

Bloco 1: inicialização de metadados de conexão. Mantemos wrappers de [Connection](#) para facilitar sincronização com kernel.

Bloco 2: configuração visual/interativa do item.

Bloco 3: cálculo inicial de geometria da conexão.

Bloco 4: registro bidirecional nas portas para atualização quando houver movimento.

Todo Extrair para método dedicado `_bindPorts()` para legibilidade.

10.226.3 Member Function Documentation

10.226.3.1 boundingRect()

```
QRectF GraphicalConnection::boundingRect ( ) const [override]
```

Returns item bounding rectangle for Qt painting system.

Bloco 1: calcula margem extra baseada na porta para reduzir clipping visual.

Bloco 2: retorna retângulo local com margem.

10.226.3.2 paint()

```
void GraphicalConnection::paint (
    QPainter * painter,
    const QStyleOptionGraphicsItem * option,
    QWidget * widget ) [override]
```

Paints connection path and selection handles.

Bloco 2: normaliza coordenadas locais considerando orientação relativa.

Bloco 3: gera path de desenho com base no tipo de roteamento.

Bloco 4: renderiza o path final da conexão.

Bloco 5: desenha “handles” quando selecionado.

10.226.3.3 updateDimensionsAndPosition()

```
void GraphicalConnection::updateDimensionsAndPosition ( )
```

Recomputes local geometry based on current port positions.

Bloco 1: coleta de geometria absoluta de portas.

Bloco 2: projeta esta conexão para um sistema de coordenadas local mínimo.

Bloco 3: mantém apenas atualização geométrica local.

The documentation for this class was generated from the following files:

- applications/gui/genesys/graphics/GraphicalConnection.h
- applications/gui/genesys/graphics/GraphicalConnection.cpp

10.227 GraphicalConnectionStyle Class Reference

Public Types

- enum class **RouteType** { **Horizontal** , **Vertical** , **Direct** , **UserDefined** }

Static Public Member Functions

- static bool **usesModernCurves** ()
- static QPainterPath **modelConnectionPath** (const QPointF &source, const QPointF &destination, RouteType routeType, bool modernCurves)
- static QPainterPath **diagramConnectionPath** (const QPointF &source, const QPointF &destination, bool modernCurves)
- static QPointF **pointAtProgress** (const QPainterPath &path, qreal progress)

The documentation for this class was generated from the following files:

- applications/gui/genesys/graphics/GraphicalConnectionStyle.h
- applications/gui/genesys/graphics/GraphicalConnectionStyle.cpp

10.228 GraphicalContextMenuController Class Reference

Builds context menus for the graphical model view using existing main-window actions.

```
#include <applications/gui/genesys/controllers/GraphicalContextMenuController.h>
```

Public Member Functions

- **GraphicalContextMenuController** ([ModelGraphicsView](#) *graphicsView, Ui::MainWindow *ui, QAction *openSubmodelAction, std::function< [ModelGraphicsScene](#) *()> currentScene, std::function< void()> actualizeActions)
Creates the controller without taking ownership of UI or scene objects.
- void **handleGraphicsViewContextMenu** (QContextMenuEvent *event)
Shows the context menu matching the item or background under the mouse.

10.228.1 Detailed Description

Builds context menus for the graphical model view using existing main-window actions.

The controller intentionally reuses QAction instances from the main window so shortcuts, enabled state, icons, checked state, and command handlers remain centralized.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/GraphicalContextMenuController.h
- applications/gui/genesys/controllers/GraphicalContextMenuController.cpp

10.229 GraphicalDataDefinitionLayout Class Reference

Pure helper that computes arc placement for graphical data definitions.

```
#include <applications/gui/genesys/services/GraphicalDataDefinitionLayout.h>
```

Static Public Member Functions

- static QPointF [arcPosition](#) (const QRectF &anchorBounds, const QSizeF &childSize, int index, int count, bool upperArc, int radialLayer=0)
Computes a child position along the upper or lower arc around an anchor item.

10.229.1 Detailed Description

Pure helper that computes arc placement for graphical data definitions.

This helper keeps the geometry math independent from the GUI controllers and scene code so layout policies can be reused in both live rebuilds and persisted-state restoration.

10.229.2 Member Function Documentation

10.229.2.1 arcPosition()

```
QPointF GraphicalDataDefinitionLayout::arcPosition (
    const QRectF & anchorBounds,
    const QSizeF & childSize,
    int index,
    int count,
    bool upperArc,
    int radialLayer = 0 ) [static]
```

Computes a child position along the upper or lower arc around an anchor item.

Parameters

<i>anchorBounds</i>	Bounds of the anchor graphical item.
<i>childSize</i>	Size of the child item to place.
<i>index</i>	Zero-based child index.
<i>count</i>	Total number of children in the arc.
<i>upperArc</i>	True for the upper arc, false for the lower arc.
<i>radialLayer</i>	Additional radial offset used for nested attachment rings.

Returns

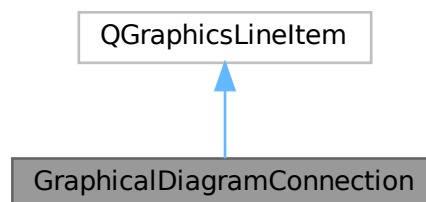
Scene position for the requested child.

The documentation for this class was generated from the following files:

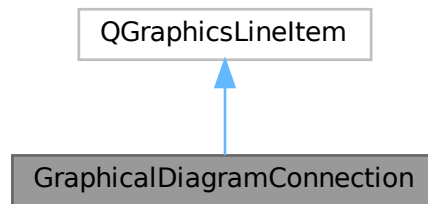
- applications/gui/genesys/services/GraphicalDataDefinitionLayout.h
- applications/gui/genesys/services/GraphicalDataDefinitionLayout.cpp

10.230 GraphicalDiagramConnection Class Reference

Inheritance diagram for GraphicalDiagramConnection:



Collaboration diagram for GraphicalDiagramConnection:



Public Types

- enum class **ConnectionType** { **ATTACHED** , **INTERNAL** }

Public Member Functions

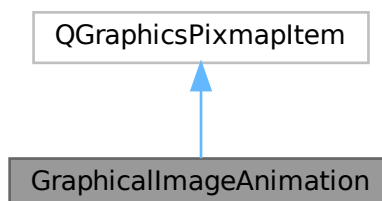
- **GraphicalDiagramConnection** (QGraphicsItem *dataDefinition, QGraphicsItem *linkedTo, ConnectionType type)
- void **refreshGeometry** ()
- QRectF **boundingRect** () const override
- QPainterPath **shape** () const override
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget=nullptr) override
- QGraphicsItem * **getDataDefinition** ()
- QGraphicsItem * **getLinkedDataDefinition** ()
- ConnectionType **getConnectionType** ()

The documentation for this class was generated from the following files:

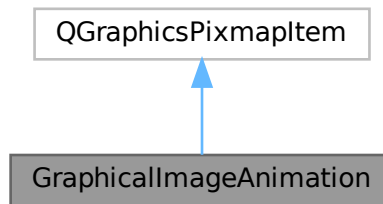
- applications/gui/genesys/graphics/GraphicalDiagramConnection.h
- applications/gui/genesys/graphics/GraphicalDiagramConnection.cpp

10.231 GraphicalImageAnimation Class Reference

Inheritance diagram for GraphicalImageAnimation:



Collaboration diagram for GraphicalImageAnimation:



Public Member Functions

- **GraphicalImageAnimation** (const QPointF startPoint, unsigned int width, unsigned int height, const QString imageName)
- QString **getImagePath** () const
- QString **getImageName** () const
- QPointF **getStartPoint** () const
- unsigned int **getWidth** () const
- unsigned int **getHeight** () const
- void **setImageName** (const QString &imageName)
- void **setStartPoint** (const QPointF &startPoint)
- void **setWidth** (unsigned int imageWidth)
- void **setHeight** (unsigned int imageHeight)
- void **updateImage** ()

The documentation for this class was generated from the following files:

- applications/gui/genesys/graphics/GraphicalImageAnimation.h
- applications/gui/genesys/graphics/GraphicalImageAnimation.cpp

10.232 GraphicalModelBuilder Class Reference

Service that reconstructs graphical components/connections from kernel model data.

```
#include <applications/gui/genesys/services/GraphicalModelBuilder.h>
```

Public Member Functions

- [GraphicalModelBuilder](#) ([Simulator](#) *simulator, [ModelGraphicsView](#) *graphicsView, [ModelGraphicsScene](#) *scene, std::map< std::string, QColor > *pluginCategoryColor, QTextEdit *console)
Creates the reconstruction service used by load/rebuild compatibility flows.
- void [setModelLevelFilter](#) (unsigned int modelLevel)
Restricts this builder and the target scene to one [ModelDataDefinition](#) level.
- void [clearModelLevelFilter](#) ()
Clears the level restriction and restores full-model rebuild semantics.
- void [recursivelyGenerateGraphicalModelFromModel](#) ([ModelComponent](#) *component, List< [ModelComponent](#) * > *visited, std::map< [ModelComponent](#) *, [GraphicalModelComponent](#) * > *map, int *x, int *y, int *ymax, int sequenceInLine)
Recursively rebuilds graphical items and links from one model-component branch.
- void [generateGraphicalModelFromModel](#) ()
Rebuilds the full scene model representation from kernel source components.

Static Public Member Functions

- static void [synchronizeGraphicalDataDefinitionsLayer](#) ([Simulator](#) *simulator, [ModelGraphicsScene](#) *scene)
Synchronizes only data-definition graphical layer with current model preserving existing layout.

10.232.1 Detailed Description

Service that reconstructs graphical components/connections from kernel model data.

This builder is used after model loading to recreate the scene representation while keeping [MainWindow](#) as a compatibility façade. It acts as a model-representation bridge between the kernel component graph and GUI graphical items.

Responsibilities:

- recursively create graphical nodes/connections for component branches;
- generate a full scene representation from model source components;
- apply plugin-category visual metadata needed during reconstruction.

Boundaries:

- it does not persist files or parse textual model language;
- it does not manage selection/property editor/simulation command flows;
- it operates as a reconstruction service, not a controller.

10.232.2 Constructor & Destructor Documentation

10.232.2.1 GraphicalModelBuilder()

```
GraphicalModelBuilder::GraphicalModelBuilder (
    Simulator * simulator,
    ModelGraphicsView * graphicsView,
    ModelGraphicsScene * scene,
    std::map< std::string, QColor > * pluginCategoryColor,
    QTextEdit * console )
```

Creates the reconstruction service used by load/rebuild compatibility flows.

Builds the graphical reconstruction service with explicit dependencies.

10.232.3 Member Function Documentation

10.232.3.1 generateGraphicalModelFromModel()

```
void GraphicalModelBuilder::generateGraphicalModelFromModel ( )
```

Rebuilds the full scene model representation from kernel source components.

Rebuilds the full scene representation from the current kernel model.

10.232.3.2 recursivelyGenerateGraphicalModelFromModel()

```
void GraphicalModelBuilder::recursivelyGenerateGraphicalModelFromModel (
    ModelComponent * component,
    List< ModelComponent * > * visited,
    std::map< ModelComponent *, GraphicalModelComponent * > * map,
    int * x,
    int * y,
    int * ymax,
    int sequenceInLine )
```

Recursively rebuilds graphical items and links from one model-component branch.

Recursively rebuilds graphical items and connections from one branch.

10.232.3.3 setModelLevelFilter()

```
void GraphicalModelBuilder::setModelLevelFilter (
    unsigned int modelLevel )
```

Restricts this builder and the target scene to one [ModelDataDefinition](#) level.

Root model tabs use level 0 today. [Submodel](#) tabs will use the owner component/data definition level, so rebuilds can be reused without mixing visual scopes.

10.232.3.4 synchronizeGraphicalDataDefinitionsLayer()

```
void GraphicalModelBuilder::synchronizeGraphicalDataDefinitionsLayer (
    Simulator * simulator,
    ModelGraphicsScene * scene ) [static]
```

Synchronizes only data-definition graphical layer with current model preserving existing layout.

Synchronizes the data-definition layer against the current kernel model.

The documentation for this class was generated from the following files:

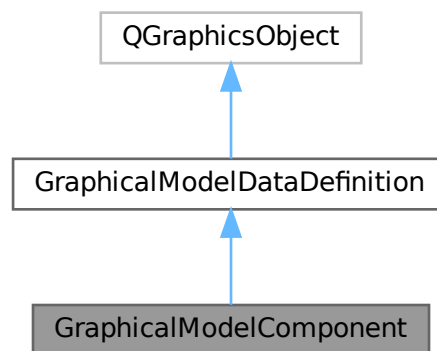
- applications/gui/genesys/services/GraphicalModelBuilder.h
- applications/gui/genesys/services/GraphicalModelBuilder.cpp

10.233 GraphicalModelComponent Class Reference

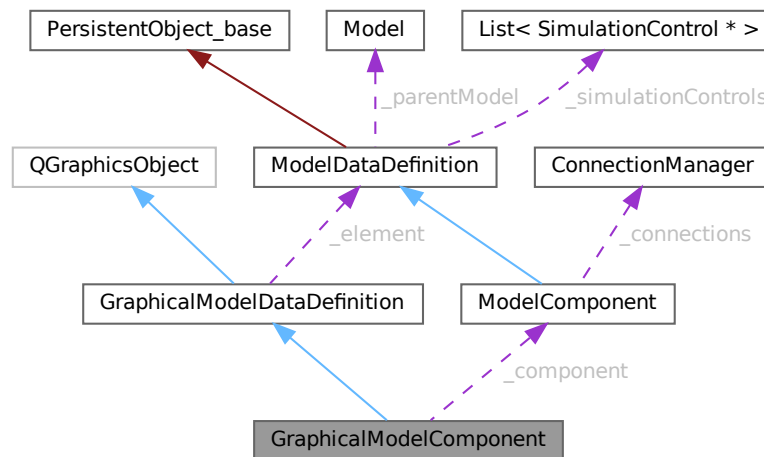
Visual component item that extends a data definition with ports and internal data.

```
#include <applications/gui/genesys/graphics/GraphicalModelComponent.h>
```

Inheritance diagram for GraphicalModelComponent:



Collaboration diagram for GraphicalModelComponent:



Public Member Functions

- **GraphicalModelComponent** ([Plugin](#) *plugin, [ModelComponent](#) *component, QPointF position, QColor color=Qt::blue, QGraphicsItem *parent=nullptr)
Creates a graphical component bound to a kernel component and plugin metadata.
- **GraphicalModelComponent** (const [GraphicalModelComponent](#) &orig)
Copy constructor used by clipboard and restoration flows.
- virtual ~**GraphicalModelComponent** ()
Releases the component item, including its generated port items and queues.
- QRectF **boundingRect** () const override
Returns the rectangle used for painting and hit testing the component body.
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override
Paints the component body, ports, and selection indicators.
- [ModelComponent](#) * **getComponent** () const
Returns the kernel component represented by this item.
- qreal **getHeight** () const
Returns the current item height derived from the render strategy.
- QPointF **getOldPosition** () const
Returns the previous scene position used by undo and restoration flows.
- void **setOldPosition** (QPointF oldPosition)
Stores the previous scene position used by undo and restoration flows.
- QList< [GraphicalComponentPort](#) * > **getGraphicalInputPorts** () const
Returns the input ports generated for this component.
- QList< [GraphicalComponentPort](#) * > **getGraphicalOutputPorts** () const
Returns the output ports generated for this component.
- QColor **getColor** () const
Returns the component color used by the renderer.
- void **setColor** (const QColor &color)
Updates the component color used by the renderer.
- QList< [ModelDataDefinition](#) * > * **getInternalData** () const

- Returns the list of internal data definitions attached to the component.*

 - `QList< ModelDataDefinition * > * getAttachedData () const`

Returns the list of externally attached data definitions.
- `EntityType * getEntityType () const`

Returns the entity type synthesized or associated with the component.
- `void setEntityType (EntityType *entityType)`

Sets the entity type synthesized or associated with the component.
- `unsigned int getOccupiedInputPorts () const`

Returns how many input ports are currently occupied.
- `unsigned int getOccupiedOutputPorts () const`

Returns how many output ports are currently occupied.
- `void setOccupiedInputPorts (unsigned int value)`

Stores the number of occupied input ports tracked by the layout code.
- `void setOccupiedOutputPorts (unsigned int value)`

Stores the number of occupied output ports tracked by the layout code.
- `QString getAnimationImageName ()`

Returns the animation image identifier used for queue visualization.
- `void setAnimationImageName (QString name)`

Updates the animation image identifier used by the renderer.
- `QMap< Queue *, QPair< unsigned int, unsigned int > > * getMapQueue ()`

Returns the queue-to-metadata map used by the queue overlay renderer.
- `QList< QList< GraphicalImageAnimation * > * > * getImagesQueue ()`

Returns the grouped queue image lists used by the renderer.
- `void verifyQueue ()`

Verifies whether the component currently references a queue animation context.
- `bool hasQueue ()`

Returns true when a queue animation context is available.
- `unsigned int getIndexQueue (Queue *queue)`

Returns the queue index assigned during queue-overlay population.
- `unsigned int getSizeQueue (Queue *queue)`

Returns the queue size recorded during queue-overlay population.
- `void populateMapQueue (QList< Queue * > queues)`

Populates the queue overlay metadata from the supplied queue list.
- `bool insertImageQueue (Queue *queue, GraphicalImageAnimation *image)`

Inserts one animation image entry for the given queue.
- `QList< GraphicalImageAnimation * > * removeImageQueue (Queue *queue, unsigned int quantity↵ Removed)`

Removes a batch of animation images for the given queue.
- `bool insertImageQueue (GraphicalImageAnimation *image)`

Inserts one animation image using the current queue context.
- `GraphicalImageAnimation * removeImageQueue ()`

Removes the most recently inserted queue animation image.
- `void actualizeMapQueue (Queue *queue)`

Refreshes the queue metadata after the queue model changes.
- `void visivableImageQueue (bool visible)`

Shows or hides the queue image overlay.
- `void clearQueues ()`

Clears the cached queue overlay state.

Public Member Functions inherited from [GraphicalModelDataDefinition](#)

- **GraphicalModelDataDefinition** ([Plugin](#) *plugin, [ModelDataDefinition](#) *element, QPointF position, QColor color=Qt::blue, QGraphicsItem *parent=nullptr)
Creates one visual item bound to a kernel data definition.
- **GraphicalModelDataDefinition** (const [GraphicalModelDataDefinition](#) &orig)
Copy constructor used by scene-copy and restore flows.
- virtual ~**GraphicalModelDataDefinition** ()
Releases the graphical item and any owned helper state.
- QRectF **boundingRect** () const override
Returns the item bounds used by Qt for painting and hit testing.
- QPainterPath **shape** () const override
Returns the precise shape used for selection and collision checks.
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override
Paints the data-definition body, label, and selection frame.
- [ModelDataDefinition](#) * **getDataDefinition** () const
Returns the kernel data definition represented by this item.
- bool **isEditableInPropertyEditor** () const
Returns whether the property editor may edit this item in place.
- void **setEditableInPropertyEditor** (bool editable)
Marks the item as editable or read-only in the property editor.
- QPointF **getOldPosition** () const
Returns the last persisted scene position before a move operation.
- void **setOldPosition** (qreal x, qreal y)
Stores the old scene position used by undo and restoration flows.
- QColor **getColor** () const
Returns the color used to paint the item.
- void **setColor** (QColor newColor)
Sets the color used to paint the item.
- qreal **getHeight** () const
Returns the height used by the current render strategy.

Static Public Member Functions

- static bool **compareQueuesById** (const [Queue](#) *a, const [Queue](#) *b)
Compares queues by identifier so queue overlays can be ordered deterministically.

Protected Member Functions

- virtual bool [sceneEvent](#) (QEvent *event) override
- QVariant **itemChange** (GraphicsItemChange change, const QVariant &value) override
- [GraphicalModelItemRenderContext](#) **renderContext** () const override

Protected Member Functions inherited from [GraphicalModelDataDefinition](#)

- QColor **myrgba** (uint64_t color)

Protected Attributes

- qreal **_width** = TraitsGUI<GModelComponent>::width
- qreal **_height** = _width * TraitsGUI<GModelComponent>::heightProportion
- unsigned int **_margin** = TraitsGUI<GModelComponent>::margin
- unsigned int **_selWidth** = TraitsGUI<GModelComponent>::selectionWidth
- **ModelComponent** * **_component**
- QColor **_color**
- QPointF **_oldPosition**
- qreal **_stretchPosTop** = TraitsGUI<GModelComponent>::stretchPos
- qreal **_stretchPosBottom** = TraitsGUI<GModelComponent>::stretchPos
- qreal **_stretchPosLeft** = TraitsGUI<GModelComponent>::stretchPos
- qreal **_stretchPosRigth** = TraitsGUI<GModelComponent>::stretchPos
- qreal **_stretchRigth** = TraitsGUI<GModelComponent>::stretch
- qreal **_stretchLeft** = TraitsGUI<GModelComponent>::stretch
- qreal **_stretchRigthMidle** = TraitsGUI<GModelComponent>::stretch
- qreal **_stretchLeftMidle** = TraitsGUI<GModelComponent>::stretch
- qreal **_stretchTop** = TraitsGUI<GModelComponent>::stretch
- qreal **_stretchBottom** = TraitsGUI<GModelComponent>::stretch
- qreal **_stretchTopMidle** = TraitsGUI<GModelComponent>::stretch
- qreal **_stretchBottomMidle** = TraitsGUI<GModelComponent>::stretch

Protected Attributes inherited from [GraphicalModelDataDefinition](#)

- qreal **_width** = TraitsGUI<GModelDataDefinition>::width
- qreal **_height** = _width * TraitsGUI<GModelDataDefinition>::heightProportion
- unsigned int **_margin** = TraitsGUI<GModelDataDefinition>::margin
- unsigned int **_selWidth** = TraitsGUI<GModelDataDefinition>::selectionWidth
- **ModelDataDefinition** * **_element**
- bool **_editableInPropertyEditor** = false
- QColor **_color**
- QPointF **_oldPosition**
- qreal **_stretchPosTop** = TraitsGUI<GModelDataDefinition>::stretchPos
- qreal **_stretchPosBottom** = TraitsGUI<GModelDataDefinition>::stretchPos
- qreal **_stretchPosLeft** = TraitsGUI<GModelDataDefinition>::stretchPos
- qreal **_stretchPosRigth** = TraitsGUI<GModelDataDefinition>::stretchPos
- qreal **_stretchRigth** = TraitsGUI<GModelDataDefinition>::stretch
- qreal **_stretchLeft** = TraitsGUI<GModelDataDefinition>::stretch
- qreal **_stretchRigthMidle** = TraitsGUI<GModelDataDefinition>::stretch
- qreal **_stretchLeftMidle** = TraitsGUI<GModelDataDefinition>::stretch
- qreal **_stretchTop** = TraitsGUI<GModelDataDefinition>::stretch
- qreal **_stretchBottom** = TraitsGUI<GModelDataDefinition>::stretch
- qreal **_stretchTopMidle** = TraitsGUI<GModelDataDefinition>::stretch
- qreal **_stretchBottomMidle** = TraitsGUI<GModelDataDefinition>::stretch

10.233.1 Detailed Description

Visual component item that extends a data definition with ports and internal data.

The scene uses this class for kernel model components that need both the visual body and the connection ports used to route edges between components.

10.233.2 Member Function Documentation

10.233.2.1 renderContext()

[GraphicalModelItemRenderContext](#) `GraphicalModelComponent::renderContext ( ) const` `[override]`, `[protected]`, `[virtual]`

Reimplemented from [GraphicalModelDataDefinition](#).

10.233.2.2 sceneEvent()

`bool GraphicalModelComponent::sceneEvent (`
 `QEvent * event )` `[override]`, `[protected]`, `[virtual]`

Reimplemented from [GraphicalModelDataDefinition](#).

The documentation for this class was generated from the following files:

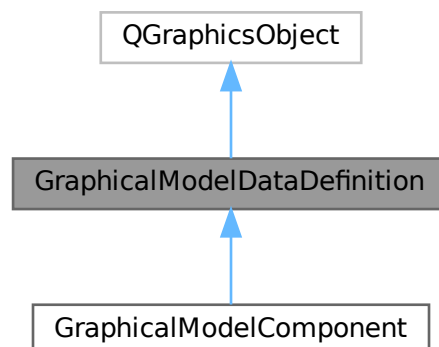
- applications/gui/genesys/graphics/GraphicalModelComponent.h
- applications/gui/genesys/graphics/GraphicalModelComponent.cpp

10.234 GraphicalModelDataDefinition Class Reference

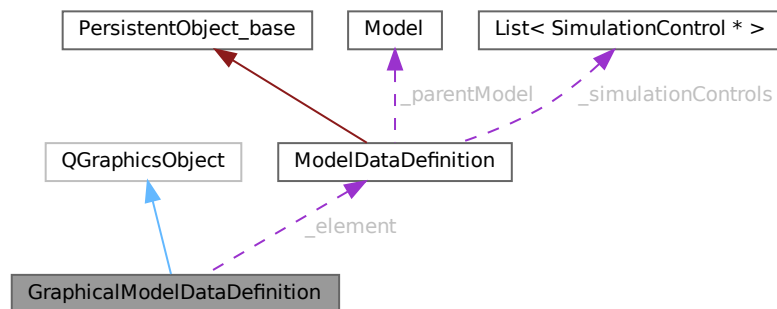
Visual representation of a model data definition in the scene.

```
#include <applications/gui/genesys/graphics/GraphicalModelDataDefinition.h>
```

Inheritance diagram for `GraphicalModelDataDefinition`:



Collaboration diagram for GraphicalModelDataDefinition:



Public Member Functions

- **GraphicalModelDataDefinition** ([Plugin](#) *plugin, [ModelDataDefinition](#) *element, QPointF position, QColor color=Qt::blue, QGraphicsItem *parent=nullptr)
Creates one visual item bound to a kernel data definition.
- **GraphicalModelDataDefinition** (const [GraphicalModelDataDefinition](#) &orig)
Copy constructor used by scene-copy and restore flows.
- virtual ~**GraphicalModelDataDefinition** ()
Releases the graphical item and any owned helper state.
- QRectF **boundingRect** () const override
Returns the item bounds used by Qt for painting and hit testing.
- QPainterPath **shape** () const override
Returns the precise shape used for selection and collision checks.
- void **paint** (QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override
Paints the data-definition body, label, and selection frame.
- [ModelDataDefinition](#) * **getDataDefinition** () const
Returns the kernel data definition represented by this item.
- bool **isEditableInPropertyEditor** () const
Returns whether the property editor may edit this item in place.
- void **setEditableInPropertyEditor** (bool editable)
Marks the item as editable or read-only in the property editor.
- QPointF **getOldPosition** () const
Returns the last persisted scene position before a move operation.
- void **setOldPosition** (qreal x, qreal y)
Stores the old scene position used by undo and restoration flows.
- QColor **getColor** () const
Returns the color used to paint the item.
- void **setColor** (QColor newColor)
Sets the color used to paint the item.
- qreal **getHeight** () const
Returns the height used by the current render strategy.

Protected Member Functions

- QColor **myrgba** (uint64_t color)
- virtual [GraphicalModelItemRenderContext](#) **renderContext** () const
- virtual bool **sceneEvent** (QEvent *event) override

Protected Attributes

- qreal **_width** = [TraitsGUI](#)<GModelDataDefinition>::width
- qreal **_height** = _width * [TraitsGUI](#)<GModelDataDefinition>::heightProportion
- unsigned int **_margin** = [TraitsGUI](#)<GModelDataDefinition>::margin
- unsigned int **_selWidth** = [TraitsGUI](#)<GModelDataDefinition>::selectionWidth
- [ModelDataDefinition](#) * **_element**
- bool **_editableInPropertyEditor** = false
- QColor **_color**
- QPointF **_oldPosition**
- qreal **_stretchPosTop** = [TraitsGUI](#)<GModelDataDefinition>::stretchPos
- qreal **_stretchPosBottom** = [TraitsGUI](#)<GModelDataDefinition>::stretchPos
- qreal **_stretchPosLeft** = [TraitsGUI](#)<GModelDataDefinition>::stretchPos
- qreal **_stretchPosRigth** = [TraitsGUI](#)<GModelDataDefinition>::stretchPos
- qreal **_stretchRigth** = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal **_stretchLeft** = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal **_stretchRigthMiddle** = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal **_stretchLeftMiddle** = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal **_stretchTop** = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal **_stretchBottom** = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal **_stretchTopMiddle** = [TraitsGUI](#)<GModelDataDefinition>::stretch
- qreal **_stretchBottomMiddle** = [TraitsGUI](#)<GModelDataDefinition>::stretch

10.234.1 Detailed Description

Visual representation of a model data definition in the scene.

This is the base graphical item used for non-component data definitions. Derived classes such as [GraphicalModelComponent](#) extend it with ports, internal data, and richer layout.

The documentation for this class was generated from the following files:

- applications/gui/genesys/graphics/GraphicalModelDataDefinition.h
- applications/gui/genesys/graphics/GraphicalModelDataDefinition.cpp

10.235 GraphicalModelEvent Class Reference

Lightweight event envelope used to notify GUI changes in the graphics scene.

```
#include <applications/gui/genesys/graphics/ModelGraphicsScene.h>
```


Public Types

- enum class **EventType** : int {
CREATE = 1 , **REMOVE** = 2 , **EDIT** = 3 , **CLONE** = 4 ,
OTHER = 9 }
- enum class **EventObjectType** : int {
COMPONENT = 1 , **DATADEFINITION** = 2 , **CONNECTION** = 3 , **DRAWING** = 4 ,
ANIMATION = 5 , **PLOT** = 6 , **OTHER** = 9 }

Public Member Functions

- **GraphicalModelEvent** (GraphicalModelEvent::EventType eventType, GraphicalModelEvent::EventObjectType eventObjectType, QGraphicsItem *item)
Creates one event payload for the given item and event categories.

Public Attributes

- GraphicalModelEvent::EventType **eventType**
- GraphicalModelEvent::EventObjectType **eventObjectType**
- QGraphicsItem * **item**

10.235.1 Detailed Description

Lightweight event envelope used to notify GUI changes in the graphics scene.

The documentation for this class was generated from the following file:

- applications/gui/genesys/graphics/ModelGraphicsScene.h

10.236 GraphicalModelItemRenderContext Struct Reference

Public Types

- enum class **ItemKind** { **DataDefinition** , **Component** }

Public Attributes

- ItemKind **kind** = ItemKind::DataDefinition
- QRectF **bounds**
- QColor **fillColor**
- QString **primaryText**
- QString **secondaryText**
- QString **tertiaryText**
- QString **semanticClassName**
- QString **semanticSubtype**
- bool **selected** = false
- bool **breakpoint** = false
- qreal **width** = 0.0
- qreal **height** = 0.0

- int **penWidth** = 1
- int **raise** = 0
- int **margin** = 0
- int **selectionWidth** = 0
- qreal **stretchPosTop** = 0.5
- qreal **stretchPosBottom** = 0.5
- qreal **stretchPosLeft** = 0.5
- qreal **stretchPosRight** = 0.5
- qreal **stretchRight** = 0.0
- qreal **stretchLeft** = 0.0
- qreal **stretchRightMiddle** = 0.0
- qreal **stretchLeftMiddle** = 0.0
- qreal **stretchTop** = 0.0
- qreal **stretchBottom** = 0.0
- qreal **stretchTopMiddle** = 0.0
- qreal **stretchBottomMiddle** = 0.0
- uint64_t **borderColor** = 0
- uint64_t **raisedColor** = 0
- uint64_t **sunkenColor** = 0
- uint64_t **textColor** = 0
- uint64_t **textShadowColor** = 0
- uint64_t **selectionColor** = 0
- uint64_t **breakpointColor** = 0

The documentation for this struct was generated from the following file:

- applications/gui/genesys/graphics/GraphicalModelItemRenderStrategy.h

10.237 GraphicalModelItemRenderer Class Reference

Static Public Member Functions

- static void **paint** (QPainter *painter, const [GraphicalModelItemRenderContext](#) &context)
- static QPainterPath **shape** (const [GraphicalModelItemRenderContext](#) &context)
- static const [GraphicalModelItemRenderStrategy](#) & **currentStrategy** ()

The documentation for this class was generated from the following files:

- applications/gui/genesys/graphics/GraphicalModelItemRenderStrategy.h
- applications/gui/genesys/graphics/GraphicalModelItemRenderStrategy.cpp

10.238 GraphicalModelItemRenderStrategy Class Reference

Public Member Functions

- virtual const char * **name** () const =0
- virtual void **paint** (QPainter *painter, const [GraphicalModelItemRenderContext](#) &context) const =0
- virtual QPainterPath **shape** (const [GraphicalModelItemRenderContext](#) &context) const =0

The documentation for this class was generated from the following file:

- applications/gui/genesys/graphics/GraphicalModelItemRenderStrategy.h

10.239 GraphicalModelSerializer Class Reference

Persistence service for saving/loading graphical and textual model representations.

```
#include <applications/gui/genesys/services/GraphicalModelSerializer.h>
```

Public Member Functions

- [GraphicalModelSerializer](#) ([Simulator](#) *simulator, QWidget *ownerWidget, QPlainTextEdit *modelTextEditor, [ModelGraphicsView](#) *graphicsView, QSlider *zoomSlider, QAction *actionShowGrid, QAction *actionShowRule, QAction *actionShowSnap, QAction *actionShowGuides, QAction *actionShowInternalElements, QAction *actionShowEditableElements, QAction *actionShowAttachedElements, QAction *actionShowRecursiveElements, QTextEdit *console, QString *modelFilename, std::function< void()> clearModelEditors, std::function< void()> rebuildGraphicalModelFromModel, std::function< void()> applyShowInternalElements, std::function< void()> applyShowEditableElements, std::function< void()> applyShowAttachedElements)
- Creates the persistence service that backs [MainWindow](#) lifecycle wrappers.*
- bool [saveTextModel](#) (QFile *saveFile, const QString &data) const
- Saves textual model language using the established compatibility format.*
- bool [saveGraphicalModel](#) (const QString &filename) const
- Saves full graphical .gui persistence sections without format changes.*
- [Model](#) * [loadGraphicalModel](#) (const std::string &filename) const
- Loads .gui/.gen model files and restores persisted graphical/UI state.*

10.239.1 Detailed Description

Persistence service for saving/loading graphical and textual model representations.

This Phase-2 extraction removes serialization concerns from [MainWindow](#) while preserving file compatibility through wrapper delegation. It bridges GUI state (scene/view/actions/editors) and kernel model loading/saving workflows.

Responsibilities:

- save textual model content with the existing line-based format;
- save full graphical .gui state sections used by legacy compatibility;
- load persisted model files and restore graphical/editor UI state via callbacks.

Boundaries:

- it does not own widgets or simulator lifetime;
- it does not run simulation commands or trace rendering;
- it coordinates persistence/serialization only.

10.239.2 Constructor & Destructor Documentation

10.239.2.1 GraphicalModelSerializer()

```
GraphicalModelSerializer::GraphicalModelSerializer (
    Simulator * simulator,
    QWidget * ownerWidget,
    QPlainTextEdit * modelTextEditor,
    ModelGraphicsView * graphicsView,
    QSlider * zoomSlider,
    QAction * actionShowGrid,
    QAction * actionShowRule,
    QAction * actionShowSnap,
    QAction * actionShowGuides,
    QAction * actionShowInternalElements,
    QAction * actionShowEditableElements,
    QAction * actionShowAttachedElements,
    QAction * actionShowRecursiveElements,
    QTextEdit * console,
    QString * modelFilename,
    std::function< void()> clearModelEditors,
    std::function< void()> rebuildGraphicalModelFromModel,
    std::function< void()> applyShowInternalElements,
    std::function< void()> applyShowEditableElements,
    std::function< void()> applyShowAttachedElements )
```

Creates the persistence service that backs [MainWindow](#) lifecycle wrappers.

Builds the serializer with explicit, narrow dependencies from [MainWindow](#).

10.239.3 Member Function Documentation

10.239.3.1 loadGraphicalModel()

```
Model * GraphicalModelSerializer::loadGraphicalModel (
    const std::string & filename ) const
```

Loads `.gui/.gen` model files and restores persisted graphical/UI state.

Loads `.gui` or `.gen` model files and restores persisted GUI state.

Returns

Loaded kernel model pointer or nullptr when loading fails.

10.239.3.2 saveGraphicalModel()

```
bool GraphicalModelSerializer::saveGraphicalModel (
    const QString & filename ) const
```

Saves full graphical `.gui` persistence sections without format changes.

Persists the complete graphical model, including view options and overlays.

Returns

true when serialization succeeds.

10.239.3.3 saveTextModel()

```
bool GraphicalModelSerializer::saveTextModel (
    QFile * saveFile,
    const QString & data ) const
```

Saves textual model language using the established compatibility format.

Writes model text line by line using the legacy persistence format.

Returns

true when write succeeds.

The documentation for this class was generated from the following files:

- applications/gui/genesys/services/GraphicalModelSerializer.h
- applications/gui/genesys/services/GraphicalModelSerializer.cpp

10.240 GraphvizModelExporter Class Reference

Service that builds DOT content and renders model images for the GUI.

```
#include <applications/gui/genesys/services/GraphvizModelExporter.h>
```

Public Member Functions

- **GraphvizModelExporter** ([Simulator](#) *simulator, QLabel *modelGraphicLabel, QCheckBox *showStatistics, QCheckBox *showEditable, QCheckBox *showShared, QCheckBox *showRecursive, QCheckBox *show↔ Levels, std::function< bool()> ensureModelSynchronized)
Creates the Graphviz exporter service used by [MainWindow](#) compatibility wrappers.
- std::string [adjustDotName](#) (std::string name) const
Normalizes identifiers to Graphviz-safe names.
- void **insertTextInDot** (std::string text, unsigned int compLevel, unsigned int compRank, std::map< unsigned int, std::map< unsigned int, std::list< std::string > * > * > *dotmap, bool isNode=false) const
Inserts a DOT line into the hierarchical rank/level map used for image generation.
- void **recursiveCreateModelGraphicPicture** ([ModelDataDefinition](#) *componentOrData, std::list< [ModelDataDefinition](#) * > *visited, std::map< unsigned int, std::map< unsigned int, std::list< std::string > * > * > *dotmap) const
Recursively traverses model/data nodes to build DOT content fragments.
- bool [createModelImage](#) () const
Generates and displays the model image after synchronization preconditions.

10.240.1 Detailed Description

Service that builds DOT content and renders model images for the GUI.

This service encapsulates Graphviz-oriented model representation logic extracted from [MainWindow](#). It keeps image-generation behavior stable while [MainWindow](#) exposes compatibility wrappers used by actions and tabs.

Responsibilities:

- normalize identifiers for DOT compatibility;
- build hierarchical DOT fragments from model/data-definition traversal;
- generate and display model images after ensuring text/model synchronization.

Boundaries:

- it does not own model synchronization policy (uses injected callback);
- it does not manage simulation commands, traces, or lifecycle actions;
- it provides export/representation service behavior, not controller orchestration.

10.240.2 Member Function Documentation

10.240.2.1 `adjustDotName()`

```
std::string GraphvizModelExporter::adjustDotName (
    std::string name ) const
```

Normalizes identifiers to Graphviz-safe names.

Parameters

<i>name</i>	Raw label/name from model representation.
-------------	---

Returns

DOT-safe identifier preserving compatibility naming intent.

10.240.2.2 `createModelImage()`

```
bool GraphvizModelExporter::createModelImage ( ) const
```

Generates and displays the model image after synchronization preconditions.

Returns

true when DOT generation and image rendering complete successfully.

The documentation for this class was generated from the following files:

- applications/gui/genesys/services/GraphvizModelExporter.h
- applications/gui/genesys/services/GraphvizModelExporter.cpp

10.241 ModelGraphicsScene::GRID Struct Reference

Public Member Functions

- void **clear** ()

Public Attributes

- unsigned int **interval**
- QPen **pen**
- std::list< QGraphicsLineItem * > * **lines**
- bool **visible**

The documentation for this struct was generated from the following files:

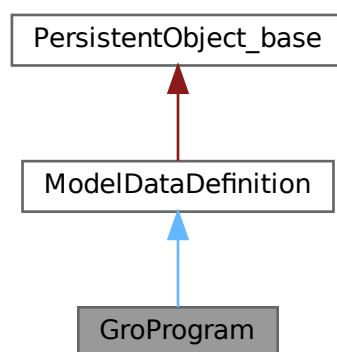
- applications/gui/genesys/graphics/ModelGraphicsScene.h
- applications/gui/genesys/graphics/ModelGraphicsScene.cpp

10.242 GroProgram Class Reference

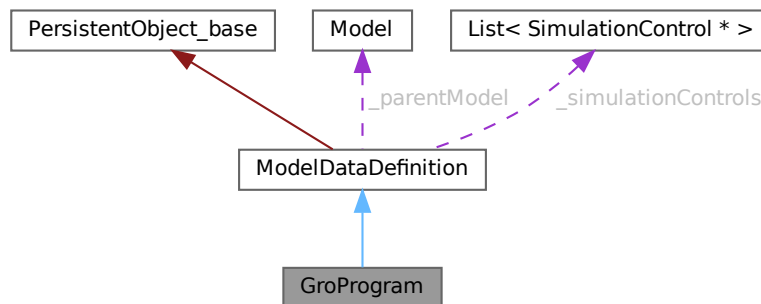
Reusable Gro source program attached to a GenESyS model.

```
#include <plugins/data/BiochemicalSimulation/GroProgram.h>
```

Inheritance diagram for GroProgram:



Collaboration diagram for GroProgram:



Public Member Functions

- **GroProgram** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **setSourceCode** (std::string sourceCode)
Replaces the stored Gro source text.
- void **setSourceCodeProperty** ([SourceCodeString](#) sourceCode)
Replaces the stored Gro source text through the property-editor wrapper type.
- std::string **getSourceCode** () const
Returns the stored Gro source text.
- [SourceCodeString](#) **getSourceCodeProperty** () const
Returns the stored Gro source text wrapped for the property editor.
- bool **createDefaultGroProgram** (const std::string &filename="")
Creates a small starter Gro program and optionally writes it to a file.
- bool **validateSyntax** (std::string &errorMessage) const
Performs a permissive lexical sanity check for the stored source.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↔ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.

- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.242.1 Detailed Description

Reusable Gro source program attached to a GenESyS model.

[GroProgram](#) is intentionally a [ModelDataDefinition](#) so several biological components can share the same Gro source text. This first implementation only stores the source and performs a small lexical sanity check; complete Gro parsing and executable semantics belong to later plugin-side helper classes.

10.242.2 Member Function Documentation

10.242.2.1 `_check()`

```
bool GroProgram::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.242.2.2 `_loadInstance()`

```
bool GroProgram::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.242.2.3 `_saveInstance()`

```
void GroProgram::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.242.2.4 `show()`

```
std::string GroProgram::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/BiochemicalSimulation/GroProgram.h
- plugins/data/BiochemicalSimulation/GroProgram.cpp

10.243 GroProgramAst Struct Reference

Minimal parsed representation of a Gro source program.

```
#include <plugins/data/BiochemicalSimulation/GroProgramAst.h>
```

Classes

- struct [NamedProgram](#)
- struct [Statement](#)

Public Types

- enum class **SourceForm** { **RawStatements** , **ProgramBlock** }

Public Member Functions

- bool **isProgramBlock** () const
- bool **hasNamedPrograms** () const

Public Attributes

- SourceForm **sourceForm** = SourceForm::RawStatements
- std::string **programName** = ""
- std::vector< std::string > **programParameters**
- std::string **bodySource** = ""
- std::vector< [Statement](#) > **statements**
- std::vector< [NamedProgram](#) > **namedPrograms**

10.243.1 Detailed Description

Minimal parsed representation of a Gro source program.

This AST intentionally models only the first integration target: either a recognized `program name()` { ... } block or a permissive raw statement list. Execution semantics and typed Gro constructs belong to later phases.

The documentation for this struct was generated from the following file:

- `plugins/data/BiochemicalSimulation/GroProgramAst.h`

10.244 GroProgramCompiler Class Reference

Converts parsed Gro AST into the first non-executable semantic IR.

```
#include <plugins/data/BiochemicalSimulation/GroProgramCompiler.h>
```

Public Member Functions

- [GroProgramIr](#) **compile** (const [GroProgramAst](#) &ast) const
Builds a lightweight semantic IR from a parsed Gro AST.

10.244.1 Detailed Description

Converts parsed Gro AST into the first non-executable semantic IR.

The documentation for this class was generated from the following files:

- plugins/data/BiochemicalSimulation/GroProgramCompiler.h
- plugins/data/BiochemicalSimulation/GroProgramCompiler.cpp

10.245 GroProgramIr Struct Reference

Initial semantic representation compiled from [GroProgramAst](#).

```
#include <plugins/data/BiochemicalSimulation/GroProgramIr.h>
```

Classes

- struct [Command](#)
- struct [NamedProgramDefinition](#)

Public Member Functions

- bool **isProgramBlock** () const
- bool **hasNamedProgram** (const std::string &name) const

Public Attributes

- GroProgramAst::SourceForm **sourceForm** = GroProgramAst::SourceForm::RawStatements
- std::string **programName** = ""
- std::vector< [Command](#) > **commands**
- std::map< std::string, [NamedProgramDefinition](#) > **namedPrograms**

10.245.1 Detailed Description

Initial semantic representation compiled from [GroProgramAst](#).

The IR keeps the parsed program in a runtime-oriented shape. It can represent function calls, assignments, and basic conditional blocks without pushing any knowledge into the simulator kernel itself.

The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/GroProgramIr.h

10.246 GroProgramParser Class Reference

Plugin-side parser boundary for reusable Gro source programs.

```
#include <plugins/data/BiochemicalSimulation/GroProgramParser.h>
```

Classes

- struct [Result](#)

Public Member Functions

- [Result](#) **parse** (const std::string &sourceCode) const
Parses a Gro source string using the current lexical boundary.

10.246.1 Detailed Description

Plugin-side parser boundary for reusable Gro source programs.

This helper currently preserves the permissive lexical validation that used to live inside [GroProgram](#). It is intentionally small so later phases can grow a real grammar, AST, and semantic checks without changing [GroProgram](#) storage.

The documentation for this class was generated from the following files:

- plugins/data/BiochemicalSimulation/GroProgramParser.h
- plugins/data/BiochemicalSimulation/GroProgramParser.cpp

10.247 GroProgramRuntime Class Reference

Executes the supported Gro IR commands without simulator binding.

```
#include <plugins/data/BiochemicalSimulation/GroProgramRuntime.h>
```

Classes

- struct [ColonyMutation](#)
- struct [ExecutionResult](#)
- struct [MotionMutation](#)
- struct [PopulationMutation](#)
- struct [SignalMutation](#)

Public Types

- enum class **PopulationMutationType** { **Grow** , **Divide** , **Die** , **SetPopulation** }
- enum class **SignalMutationType** { **Emit** , **Consume** , **Set** }
- enum class **MotionMutationType** { **Run** , **Tumble** }

Public Member Functions

- **ExecutionResult execute** (const [GroProgramIr](#) &ir, [GroProgramRuntimeState](#) &state) const
Executes supported commands and reports unsupported commands.

Static Public Member Functions

- static bool **evaluateExpression** (const std::string &expressionText, const [GroProgramRuntimeState](#) &state, double &value, std::string &errorMessage)
Evaluates one scalar Gro expression against one runtime state.

10.247.1 Detailed Description

Executes the supported Gro IR commands without simulator binding.

This helper is intentionally independent from [BacteriaColony](#) and the GenESyS event scheduler. It now supports persistent scalar variables, arithmetic expressions, and basic `if/else` control flow while still keeping the runtime local to the biological plugins.

The documentation for this class was generated from the following files:

- `plugins/data/BiochemicalSimulation/GroProgramRuntime.h`
- `plugins/data/BiochemicalSimulation/GroProgramRuntime.cpp`

10.248 GroProgramRuntimeState Struct Reference

Mutable state used by the plugin-side Gro runtime helper.

```
#include <plugins/data/BiochemicalSimulation/GroProgramRuntime.h>
```

Public Attributes

- double **colonyTime** = 0.0
- double **simulationStep** = 1.0
- unsigned int **populationSize** = 1
- unsigned int **tickCount** = 0
- std::map< std::string, double > **contextVariables**
- std::map< std::string, double > **variables**

10.248.1 Detailed Description

Mutable state used by the plugin-side Gro runtime helper.

The documentation for this struct was generated from the following file:

- `plugins/data/BiochemicalSimulation/GroProgramRuntime.h`

10.249 GroupItem Struct Reference

Public Attributes

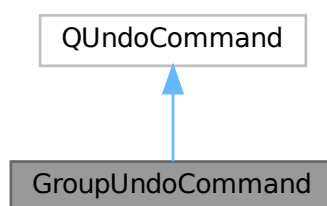
- QGraphicsItemGroup * **group**
- QList< [ComponentItem](#) > **myComponentItems**
- QPointF **initialPosition**

The documentation for this struct was generated from the following file:

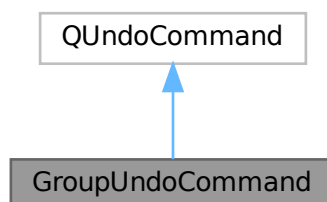
- applications/gui/genesys/actions/AddUndoCommand.h

10.250 GroupUndoCommand Class Reference

Inheritance diagram for GroupUndoCommand:



Collaboration diagram for GroupUndoCommand:



Public Member Functions

- **GroupUndoCommand** (QList< [GraphicalModelComponent](#) * > graphicalComponents, [ModelGraphicsScene](#) *scene, QUndoCommand *parent=nullptr)
- void **undo** () override
- void **redo** () override

The documentation for this class was generated from the following files:

- applications/gui/genesys/actions/GroupUndoCommand.h
- applications/gui/genesys/actions/GroupUndoCommand.cpp

10.251 GuiActionContribution Struct Reference

Public Attributes

- std::string **id**
- std::string **text**
- std::string **menuPath**
- std::string **toolBarId**
- std::string **statusTip**
- std::string **shortcut**
- std::function< bool(const [GuiExtensionRuntimeContext](#) &)> **isVisible**
- std::function< bool(const [GuiExtensionRuntimeContext](#) &)> **isEnabled**
- std::function< void(const [GuiExtensionRuntimeContext](#) &)> **trigger**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/extensions/GuiExtensionContracts.h

10.252 GuiDockContribution Struct Reference

Public Attributes

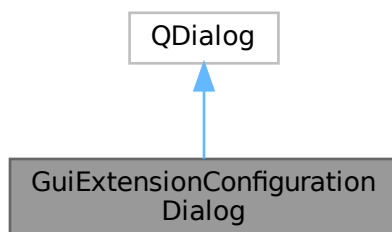
- std::string **id**
- std::string **title**
- std::string **areaHint**
- std::function< bool(const [GuiExtensionRuntimeContext](#) &)> **isVisible**
- std::function< QWidget *(const [GuiExtensionRuntimeContext](#) &)> **createWidget**

The documentation for this struct was generated from the following file:

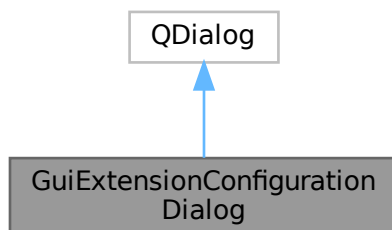
- applications/gui/genesys/extensions/GuiExtensionContracts.h

10.253 GuiExtensionConfigurationDialog Class Reference

Inheritance diagram for GuiExtensionConfigurationDialog:



Collaboration diagram for GuiExtensionConfigurationDialog:



Public Member Functions

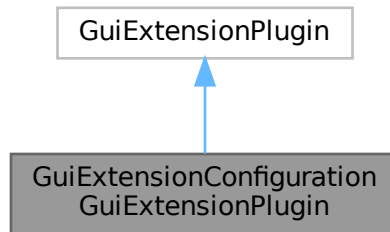
- **GuiExtensionConfigurationDialog** (QWidget *parent, std::vector< const [GuiExtensionPlugin](#) * > plugins, std::function< void()> onSaved={})

The documentation for this class was generated from the following files:

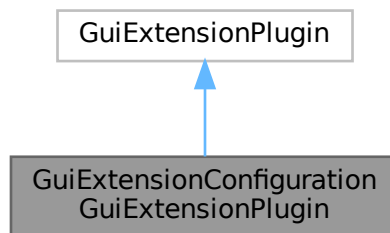
- applications/gui/genesys/extensions/GuiExtensionConfigurationDialog.h
- applications/gui/genesys/extensions/GuiExtensionConfigurationDialog.cpp

10.254 GuiExtensionConfigurationGuiExtensionPlugin Class Reference

Inheritance diagram for GuiExtensionConfigurationGuiExtensionPlugin:



Collaboration diagram for GuiExtensionConfigurationGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId () const override`
- `std::string displayName () const override`
- `std::string version () const override`
- `std::vector< std::string > requiredModelPlugins () const override`
- `void registerContributions (GuiExtensionRegistry *registry) const override`

10.254.1 Member Function Documentation

10.254.1.1 displayName()

```
std::string GuiExtensionConfigurationGuiExtensionPlugin::displayName ( ) const [inline],
[override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.254.1.2 extensionId()

```
std::string GuiExtensionConfigurationGuiExtensionPlugin::extensionId ( ) const [inline],
[override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.254.1.3 registerContributions()

```
void GuiExtensionConfigurationGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.254.1.4 requiredModelPlugins()

```
std::vector< std::string > GuiExtensionConfigurationGuiExtensionPlugin::requiredModelPlugins (
) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.254.1.5 version()

```
std::string GuiExtensionConfigurationGuiExtensionPlugin::version ( ) const [inline], [override],
[virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

- applications/gui/genesys/extensions/GuiExtensionConfigurationGuiExtensionPlugin.cpp

10.255 GuiExtensionManager Class Reference**Public Member Functions**

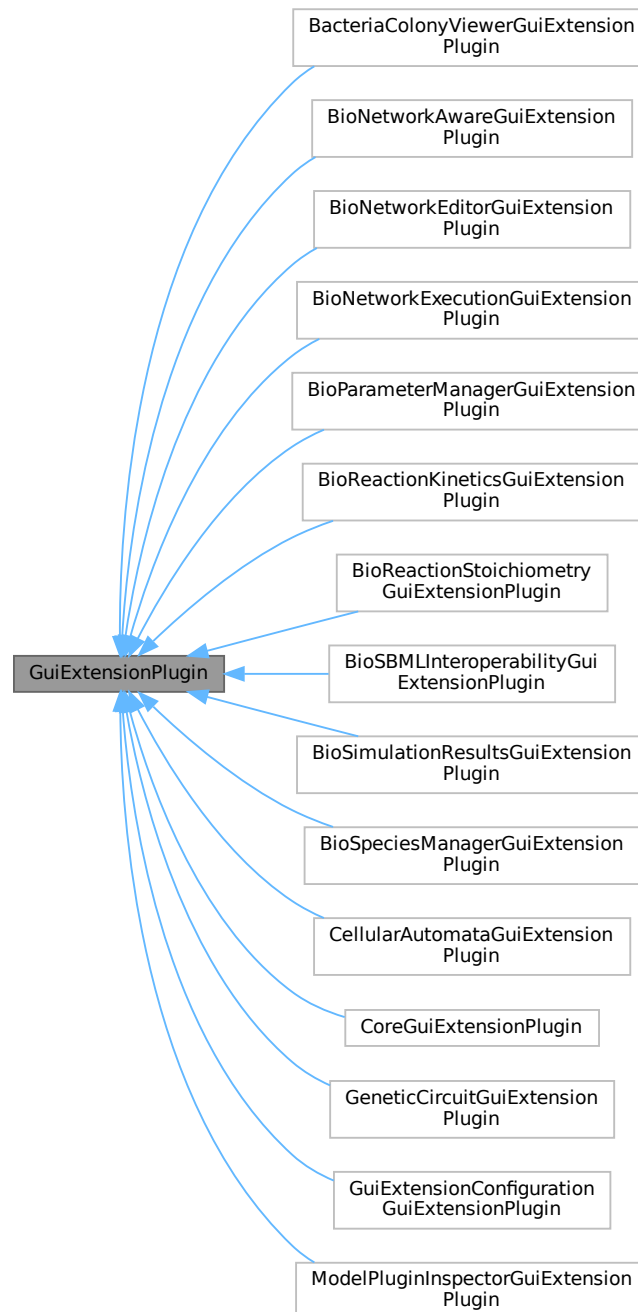
- **GuiExtensionManager** (QMainWindow *mainWindow)
- void **setPlugins** (std::vector< const [GuiExtensionPlugin](#) * > plugins)
- void **setLoadedModelPluginIds** (std::vector< std::string > modelPluginIds)
- void **rebuild** (const [GuiExtensionRuntimeContext](#) &context)
- void **clear** ()

The documentation for this class was generated from the following files:

- applications/gui/genesys/extensions/GuiExtensionManager.h
- applications/gui/genesys/extensions/GuiExtensionManager.cpp

10.256 GuiExtensionPlugin Class Reference

Inheritance diagram for GuiExtensionPlugin:



Public Member Functions

- virtual std::string **extensionId** () const =0
- virtual std::string **displayName** () const =0

- virtual std::string **version** () const =0
- virtual std::vector< std::string > **requiredModelPlugins** () const =0
- virtual void **registerContributions** ([GuiExtensionRegistry](#) *registry) const =0

The documentation for this class was generated from the following file:

- applications/gui/genesys/extensions/GuiExtensionContracts.h

10.257 GuiExtensionPluginAutoRegistrar Class Reference

Public Member Functions

- **GuiExtensionPluginAutoRegistrar** (const [GuiExtensionPlugin](#) *plugin)

The documentation for this class was generated from the following files:

- applications/gui/genesys/extensions/GuiExtensionPluginCatalog.h
- applications/gui/genesys/extensions/GuiExtensionPluginCatalog.cpp

10.258 GuiExtensionPluginCatalog Class Reference

Static Public Member Functions

- static const std::vector< const [GuiExtensionPlugin](#) * > & **plugins** ()
- static std::vector< const [GuiExtensionPlugin](#) * > **resolvedPlugins** ()
- static [GuiExtensionPresentationMetadata](#) **presentationMetadata** (const std::string &extensionId)
- static void **registerPlugin** (const [GuiExtensionPlugin](#) *plugin)

The documentation for this class was generated from the following files:

- applications/gui/genesys/extensions/GuiExtensionPluginCatalog.h
- applications/gui/genesys/extensions/GuiExtensionPluginCatalog.cpp

10.259 GuiExtensionPresentationMetadata Struct Reference

Public Attributes

- std::string **category**
- std::string **group**
- int **priority** = 0

The documentation for this struct was generated from the following file:

- applications/gui/genesys/extensions/GuiExtensionPluginCatalog.h

10.260 GuiExtensionRegistry Class Reference

Public Member Functions

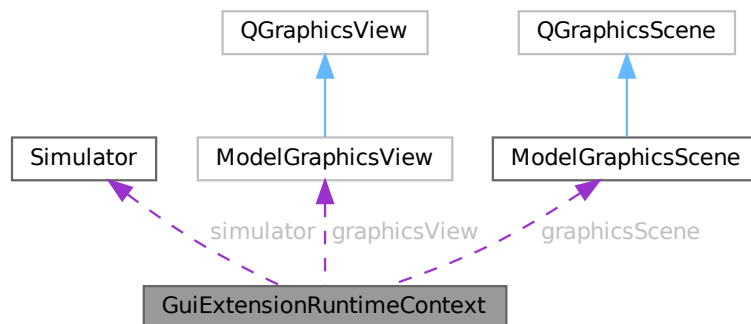
- void **addAction** (GuiActionContribution contribution)
- void **addDock** (GuiDockContribution contribution)
- void **addWindow** (GuiWindowContribution contribution)
- const std::vector< GuiActionContribution > & **actions** () const
- const std::vector< GuiDockContribution > & **docks** () const
- const std::vector< GuiWindowContribution > & **windows** () const
- void **clear** ()

The documentation for this class was generated from the following file:

- applications/gui/genesys/extensions/GuiExtensionContracts.h

10.261 GuiExtensionRuntimeContext Struct Reference

Collaboration diagram for GuiExtensionRuntimeContext:



Public Attributes

- Simulator * **simulator** = nullptr
- QMainWindow * **mainWindow** = nullptr
- Ui::MainWindow * **ui** = nullptr
- ModelGraphicsView * **graphicsView** = nullptr
- ModelGraphicsScene * **graphicsScene** = nullptr

The documentation for this struct was generated from the following file:

- applications/gui/genesys/extensions/GuiExtensionContracts.h

10.262 GuiScopeTrace Class Reference

Emits a scoped trace entry for a GUI method and measures its runtime.

```
#include <applications/gui/genesys/GuiScopeTrace.h>
```

Public Member Functions

- [GuiScopeTrace](#) (const char *methodName, const void *thisPtr)
Starts a scoped GUI trace entry for a critical method invocation.
- [~GuiScopeTrace](#) ()
Ends the scoped trace and logs the elapsed execution time.

10.262.1 Detailed Description

Emits a scoped trace entry for a GUI method and measures its runtime.

Instances are usually created on the stack at the beginning of a method so the constructor logs entry and the destructor logs the elapsed time when the scope exits.

10.262.2 Constructor & Destructor Documentation

10.262.2.1 GuiScopeTrace()

```
GuiScopeTrace::GuiScopeTrace (
    const char * methodName,
    const void * thisPtr )
```

Starts a scoped GUI trace entry for a critical method invocation.

Parameters

<i>methodName</i>	Name of the traced method.
<i>thisPtr</i>	Pointer to the current object, when available.

The documentation for this class was generated from the following files:

- applications/gui/genesys/GuiScopeTrace.h
- applications/gui/genesys/GuiScopeTrace.cpp

10.263 GuiThemeManager Class Reference

Applies the visual theme used by the Qt application and model views.

```
#include <applications/gui/genesys/guithememanager.h>
```

Static Public Member Functions

- static void [applyApplicationTheme](#) (QApplication *application)
Applies application-wide theme settings to the Qt application object.
- static void [applyModelGraphicsTheme](#) (ModelGraphicsView *graphicsView)
Applies the model-graphics theme to a specific graphics view.

10.263.1 Detailed Description

Applies the visual theme used by the Qt application and model views.

The theme manager centralizes palette-level adjustments so the application and graphics scene remain visually consistent when preferences change.

10.263.2 Member Function Documentation

10.263.2.1 [applyApplicationTheme\(\)](#)

```
void GuiThemeManager::applyApplicationTheme (
    QApplication * application ) [static]
```

Applies application-wide theme settings to the Qt application object.

Parameters

<i>application</i>	Target QApplication instance.
--------------------	-------------------------------

10.263.2.2 [applyModelGraphicsTheme\(\)](#)

```
void GuiThemeManager::applyModelGraphicsTheme (
    ModelGraphicsView * graphicsView ) [static]
```

Applies the model-graphics theme to a specific graphics view.

Parameters

<i>graphicsView</i>	Target graphics view.
---------------------	-----------------------

The documentation for this class was generated from the following files:

- applications/gui/genesys/guithememanager.h
- applications/gui/genesys/guithememanager.cpp

10.264 GuiWindowContribution Struct Reference

Public Attributes

- std::string **id**

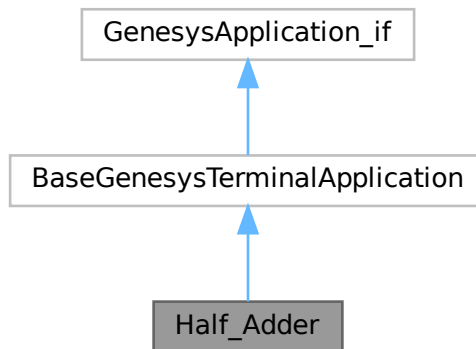
- `std::string menuPath`
- `std::string text`
- `std::function< bool(const GuiExtensionRuntimeContext &)> isVisible`
- `std::function< void(const GuiExtensionRuntimeContext &)> open`

The documentation for this struct was generated from the following file:

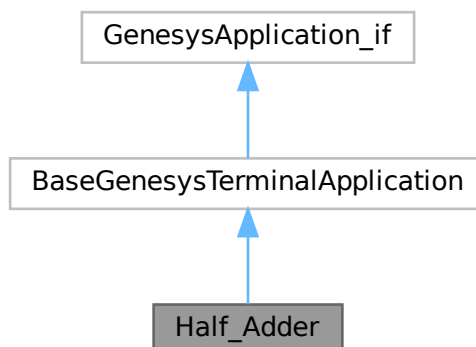
- `applications/gui/genesys/extensions/GuiExtensionContracts.h`

10.265 Half_Adder Class Reference

Inheritance diagram for Half_Adder:



Collaboration diagram for Half_Adder:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.265.1 Member Function Documentation

10.265.1.1 [main\(\)](#)

```
int Half_Adder::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

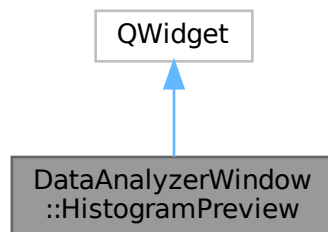
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

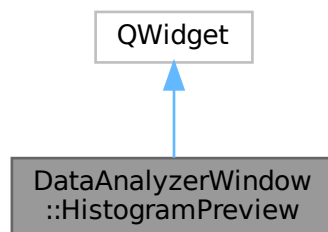
- applications/modelSpecific/teaching/Half_Adder.h
- applications/modelSpecific/teaching/Half_Adder.cpp

10.266 DataAnalyzerWindow::HistogramPreview Class Reference

Inheritance diagram for DataAnalyzerWindow::HistogramPreview:



Collaboration diagram for DataAnalyzerWindow::HistogramPreview:



Public Member Functions

- **HistogramPreview** (QWidget *parent=nullptr)
- void **setValues** (const QList< double > &values)

Protected Member Functions

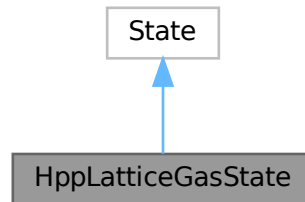
- void **paintEvent** (QPaintEvent *) override

The documentation for this class was generated from the following file:

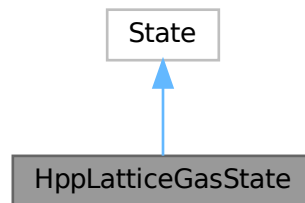
- applications/gui/genesys/tools/dataanalyzer/DataAnalyzerWindow.cpp

10.267 HppLatticeGasState Class Reference

Inheritance diagram for HppLatticeGasState:



Collaboration diagram for HppLatticeGasState:



Public Types

- enum **Direction** : size_t { **North** = 0 , **East** = 1 , **South** = 2 , **West** = 3 }

Public Member Functions

- **HpplatticeGasState** (std::array< bool, 4 > occupations)
- std::unique_ptr< [State](#) > [clone](#) () const override
- bool [equals](#) (const [State](#) &other) const override
- std::string [toString](#) () const override
- std::string [typeName](#) () const override
- long [getValue](#) () const override
- void [setValue](#) (long value) override
- bool **has** (Direction direction) const
- void **set** (Direction direction, bool occupied)
- const std::array< bool, 4 > & **occupations** () const

Public Member Functions inherited from [State](#)

- **State** (long value)
- **State** (const [State](#) &orig)
- virtual std::string **show** () const
- template<typename T >
const T * **as** () const noexcept
- template<typename T >
T * **as** () noexcept

Static Public Member Functions

- static long **maskFromOccupations** (const std::array< bool, 4 > &occupations)

Additional Inherited Members

Protected Attributes inherited from [State](#)

- long **_value**

10.267.1 Member Function Documentation

10.267.1.1 clone()

```
std::unique_ptr< State > HppLatticeGasState::clone ( ) const [inline], [override], [virtual]
```

Reimplemented from [State](#).

10.267.1.2 equals()

```
bool HppLatticeGasState::equals (
    const State & other ) const [inline], [override], [virtual]
```

Reimplemented from [State](#).

10.267.1.3 getValue()

```
long HppLatticeGasState::getValue ( ) const [inline], [override], [virtual]
```

Reimplemented from [State](#).

10.267.1.4 setValue()

```
void HppLatticeGasState::setValue (
    long value ) [inline], [override], [virtual]
```

Reimplemented from [State](#).

10.267.1.5 toString()

```
std::string HppLatticeGasState::toString ( ) const [inline], [override], [virtual]
```

Reimplemented from [State](#).

10.267.1.6 typeName()

```
std::string HppLatticeGasState::typeName ( ) const [inline], [override], [virtual]
```

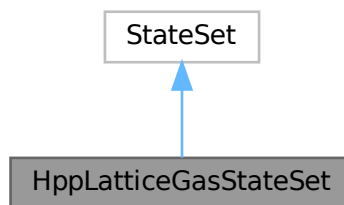
Reimplemented from [State](#).

The documentation for this class was generated from the following file:

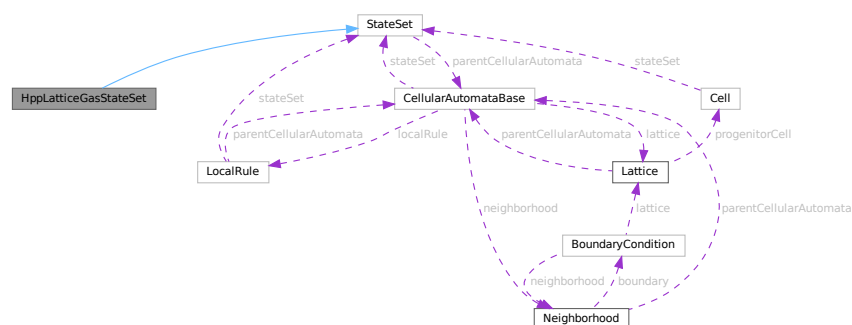
- plugins/components/ModalModel/CellularAutomata/State.h

10.268 HppLatticeGasStateSet Class Reference

Inheritance diagram for HppLatticeGasStateSet:



Collaboration diagram for HppLatticeGasStateSet:



Public Member Functions

- **HppLatticeGasStateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- bool **contains** (const [State](#) &state) const override
- std::unique_ptr< [State](#) > **createDefaultState** () const override
- std::unique_ptr< [State](#) > **parseState** (const std::string &text) const override
- bool **isFinite** () const override
- bool **isDiscrete** () const override
- bool **isComposite** () const override
- std::vector< std::unique_ptr< [State](#) > > **enumerateStates** () const override
- std::string **name** () const override

Public Member Functions inherited from [StateSet](#)

- **StateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- **StateSet** (const [StateSet](#) &orig)
- virtual bool **isContinuous** () const

Additional Inherited Members

Protected Attributes inherited from [StateSet](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**

10.268.1 Member Function Documentation

10.268.1.1 contains()

```
bool HppLatticeGasStateSet::contains (
    const State & state ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.268.1.2 createDefaultState()

```
std::unique_ptr< State > HppLatticeGasStateSet::createDefaultState ( ) const [inline], [override],
[virtual]
```

Reimplemented from [StateSet](#).

10.268.1.3 enumerateStates()

```
std::vector< std::unique_ptr< State > > HppLatticeGasStateSet::enumerateStates ( ) const
[inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.268.1.4 isComposite()

```
bool HppLatticeGasStateSet::isComposite ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.268.1.5 isDiscrete()

```
bool HppLatticeGasStateSet::isDiscrete ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.268.1.6 isFinite()

```
bool HppLatticeGasStateSet::isFinite ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.268.1.7 name()

```
std::string HppLatticeGasStateSet::name ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.268.1.8 parseState()

```
std::unique_ptr< State > HppLatticeGasStateSet::parseState (
    const std::string & text ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

The documentation for this class was generated from the following file:

- `plugins/components/ModalModel/CellularAutomata/StateSet.h`

10.269 HttpRequest Struct Reference

Lightweight representation of one parsed HTTP request.

```
#include <applications/worker/http/HttpRequest.h>
```

Public Member Functions

- `std::string header (const std::string &name) const`
Returns a header value using a case-insensitive lookup.

Public Attributes

- `std::string` **method**
HTTP method token as received from the request line.
- `std::string` **path**
Request target path used by the API router.
- `std::string` **body**
Raw body payload passed to JSON parsers and handlers.
- `std::unordered_map< std::string, std::string >` **headers**
Lowercase header map used for case-insensitive lookups.

10.269.1 Detailed Description

Lightweight representation of one parsed HTTP request.

The worker application uses a small transport surface, so the request keeps only the pieces needed by the router: method, path, raw body and normalized header lookup.

10.269.2 Member Function Documentation

10.269.2.1 header()

```
std::string HttpRequest::header (
    const std::string & name ) const [inline]
```

Returns a header value using a case-insensitive lookup.

Parameters

<i>name</i>	Header name to search.
-------------	------------------------

Returns

Header value when present, or an empty string otherwise.

The documentation for this struct was generated from the following file:

- `applications/worker/http/HttpRequest.h`

10.270 HttpResponse Struct Reference

Minimal HTTP response payload produced by the web API router.

```
#include <applications/worker/http/HttpResponse.h>
```

Public Attributes

- int **status** = 200
HTTP status code returned to the client.
- std::string **contentType** = "application/json"
MIME type advertised in the response.
- std::string **body**
Raw response body, usually JSON.
- std::unordered_map< std::string, std::string > **headers**
Arbitrary response headers written by the router or handlers.

10.270.1 Detailed Description

Minimal HTTP response payload produced by the web API router.

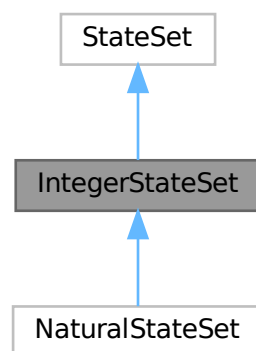
The response mirrors the small transport model used by the embedded server: status, content type, body, and a simple header map.

The documentation for this struct was generated from the following file:

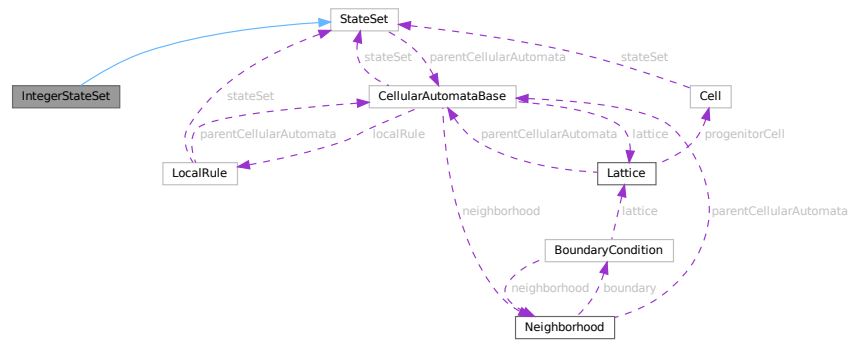
- applications/worker/http/HttpResponse.h

10.271 IntegerStateSet Class Reference

Inheritance diagram for IntegerStateSet:



Collaboration diagram for IntegerStateSet:



Public Member Functions

- **IntegerStateSet** ([CellularAutomataBase](#) *parentCellularAutomata, std::optional< long > minimum=std::nullopt, std::optional< long > maximum=std::nullopt)
- bool **contains** (const [State](#) &state) const override
- std::unique_ptr< [State](#) > **createDefaultState** () const override
- std::unique_ptr< [State](#) > **parseState** (const std::string &text) const override
- bool **isFinite** () const override
- bool **isDiscrete** () const override
- std::vector< std::unique_ptr< [State](#) > > **enumerateStates** () const override
- std::string **name** () const override

Public Member Functions inherited from [StateSet](#)

- **StateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- **StateSet** (const [StateSet](#) &orig)
- virtual bool **isContinuous** () const
- virtual bool **isComposite** () const

Additional Inherited Members

Protected Attributes inherited from [StateSet](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**

10.271.1 Member Function Documentation

10.271.1.1 contains()

```
bool IntegerStateSet::contains (
    const State & state ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.271.1.2 createDefaultState()

```
std::unique_ptr< State > IntegerStateSet::createDefaultState ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.271.1.3 enumerateStates()

```
std::vector< std::unique_ptr< State > > IntegerStateSet::enumerateStates ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.271.1.4 isDiscrete()

```
bool IntegerStateSet::isDiscrete ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.271.1.5 isFinite()

```
bool IntegerStateSet::isFinite ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.271.1.6 name()

```
std::string IntegerStateSet::name ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.271.1.7 parseState()

```
std::unique_ptr< State > IntegerStateSet::parseState (
    const std::string & text ) const [inline], [override], [virtual]
```

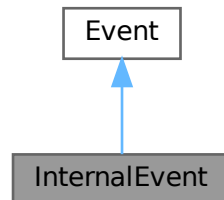
Reimplemented from [StateSet](#).

The documentation for this class was generated from the following file:

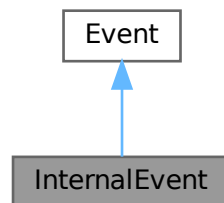
- `plugins/components/ModalModel/CellularAutomata/StateSet.h`

10.272 InternalEvent Class Reference

Inheritance diagram for InternalEvent:



Collaboration diagram for InternalEvent:



Public Member Functions

- **InternalEvent** (double time, std::string description)
- template<typename Class >
void **setEventHandler** (Class *object, void(Class::*function)(void *), void *parameter)
- virtual std::string **show** () override
- const InternalEventHandler & **eventHandler** () const
- const std::string & **description** () const
- void **dispatchEvent** () const
- void * **object** () const
- void * **parameter** () const

Public Member Functions inherited from **Event**

- **Event** (double time, **Entity** *entity, **ModelComponent** *component, unsigned int componentinputPort↔Number=0)
- **Event** (double time, **Entity** *entity, **Connection** *connection)
- double **getTime** () const

- getTime*
- [ModelComponent](#) * [getComponent](#) () const
- getComponent*
- [Entity](#) * [getEntity](#) () const
- getEntity*
- unsigned int [getComponentinputPortNumber](#) () const
- getComponentinputPortNumber*
- void [setEntity](#) ([Entity](#) *_entity)
- setEntity*

Additional Inherited Members

Protected Attributes inherited from [Event](#)

- double [_time](#)

10.272.1 Member Function Documentation

10.272.1.1 [show\(\)](#)

```
virtual std::string InternalEvent::show ( ) [inline], [override], [virtual]
```

Reimplemented from [Event](#).

The documentation for this class was generated from the following file:

- [kernel/simulator/Event.h](#)

10.273 IssueReportDraft Struct Reference

User-authored issue report collected by the GUI form before transport.

```
#include <applications/gui/genesys/services/IssueReportRelayService.h>
```

Public Attributes

- QString **categoryKey**
- QString **categoryLabel**
- QString **title**
- QString **summary**
- QString **reproductionSteps**
- QString **expectedBehavior**
- QString **observedBehavior**
- QString **additionalContext**
- QString **contactEmail**
- bool **includeScreenshot** = false
- bool **includeLogTail** = true
- bool **includeModelText** = false

10.273.1 Detailed Description

User-authored issue report collected by the GUI form before transport.

The documentation for this struct was generated from the following file:

- applications/gui/genesys/services/IssueReportRelayService.h

10.274 IssueReportRelayService Class Reference

HTTPS client service that sends issue reports to a secure relay endpoint.

```
#include <applications/gui/genesys/services/IssueReportRelayService.h>
```

Static Public Member Functions

- static QString **configuredEndpoint** ()
- static QString **optionalClientKey** ()
- static int **requestTimeoutMs** ()
- static bool **isEndpointConfigured** (QString *reason=NULLPTR)
- static [IssueReportSubmissionResult](#) **submit** (const [IssueReportDraft](#) &draft, const [IssueReportRuntimeContext](#) &runtimeContext)

10.274.1 Detailed Description

HTTPS client service that sends issue reports to a secure relay endpoint.

Responsibilities:

- validate relay configuration from environment;
- serialize the GUI report and runtime diagnostics into JSON;
- submit the payload via HTTPS and normalize the relay response.

Boundaries:

- it does not talk directly to GitHub with a secret embedded in the client;
- it does not own UI widgets or present dialogs;
- it assumes an external relay exists and enforces server-side policy.

The documentation for this class was generated from the following files:

- applications/gui/genesys/services/IssueReportRelayService.h
- applications/gui/genesys/services/IssueReportRelayService.cpp

10.275 IssueReportRuntimeContext Struct Reference

Runtime diagnostics captured by the GUI after the user approves submission.

```
#include <applications/gui/genesys/services/IssueReportRelayService.h>
```

Public Attributes

- QString **applicationName**
- QString **applicationVersion**
- QString **qtVersion**
- QString **operatingSystem**
- QString **currentModelName**
- QString **currentModelSimulanguage**
- QString **consoleTail**
- QString **simulationTail**
- QString **reportsTail**
- QString **processLogTail**
- QByteArray **screenshotPngBase64**

10.275.1 Detailed Description

Runtime diagnostics captured by the GUI after the user approves submission.

The documentation for this struct was generated from the following file:

- applications/gui/genesys/services/IssueReportRelayService.h

10.276 IssueReportSubmissionResult Struct Reference

Relay submission outcome returned to the controller layer.

```
#include <applications/gui/genesys/services/IssueReportRelayService.h>
```

Public Attributes

- bool **success** = false
- int **httpStatus** = 0
- int **issueNumber** = 0
- QString **issueUrl**
- QString **errorMessage**
- QString **responseBody**

10.276.1 Detailed Description

Relay submission outcome returned to the controller layer.

The documentation for this struct was generated from the following file:

- applications/gui/genesys/services/IssueReportRelayService.h

10.277 Json Struct Reference

Public Types

- enum **Kind** {
 null , **boolean** , **number** , **string** ,
 array , **object** }

Public Attributes

- enum Json::Kind **kind**
- double **value**
- std::string **text**
- std::vector< std::unique_ptr< [Json](#) > > [sequence](#)
- std::unordered_map< std::string, std::unique_ptr< [Json](#) > > [fields](#)

10.277.1 Member Data Documentation

10.277.1.1 fields

```
std::unordered_map<std::string, std::unique_ptr<Json> > Json::fields
```

Initial value:

```
{  
    }
```

10.277.1.2 sequence

```
std::vector<std::unique_ptr<Json> > Json::sequence
```

Initial value:

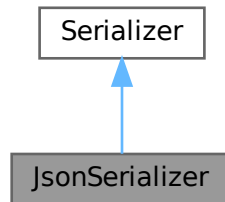
```
{  
    }
```

The documentation for this struct was generated from the following file:

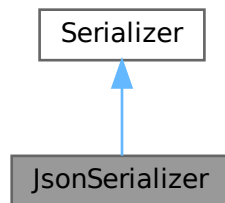
- kernel/simulator/persistence/JsonSerializer.cpp

10.278 JsonSerializer Class Reference

Inheritance diagram for JsonSerializer:



Collaboration diagram for JsonSerializer:



Public Member Functions

- **JsonSerializer** ([Model](#) *model)
- [PersistenceRecord](#) * [newPersistenceRecord](#) () override
Creates an empty persistence record.
- bool [dump](#) (std::ostream &output) override
Writes formatted contents to a given output stream.
- bool [load](#) (std::istream &input) override
Attempts to load contents from a given input stream.
- bool [get](#) (const std::string &name, [PersistenceRecord](#) *entry) override
Tries to fetch an entry with the given label as a key.
- bool [put](#) (const std::string name, const std::string type, const Util::identification id, [PersistenceRecord](#) *fields) override
Upserts an entry into this structure.
- int [for_each](#) (std::function< int(const std::string &) > delegate) override
Iterates over this structure's entries, ordered by ID.

10.278.1 Member Function Documentation

10.278.1.1 dump()

```
bool JsonSerializer::dump (
    std::ostream & output ) [override], [virtual]
```

Writes formatted contents to a given output stream.

Parameters

<i>output</i>	output stream to write to.
---------------	----------------------------

Returns

whether the current state of this structure's contents was deemed valid for serialization.

Implements [Serializer](#).

10.278.1.2 for_each()

```
int JsonSerializer::for_each (
    std::function< int(const std::string &) > delegate ) [override], [virtual]
```

Iterates over this structure's entries, ordered by ID.

Parameters

<i>delegate</i>	delegate which is applied to every entry-identifying key, until it returns a non-zero value.
-----------------	--

Returns

either zero, or the first non-zero value returned by the delegate.

Implements [Serializer](#).

10.278.1.3 get()

```
bool JsonSerializer::get (
    const std::string & name,
    PersistenceRecord * entry ) [override], [virtual]
```

Tries to fetch an entry with the given label as a key.

Parameters

<i>name</i>	a label which uniquely identifies an entry.
<i>entry</i>	(out-param) a pointer to a caller-owned entry, which is filled on success.

Returns

whether the entry was found and the out-param was actually written to.

Implements [Serializer](#).

10.278.1.4 load()

```
bool JsonSerializer::load (
    std::istream & input ) [override], [virtual]
```

Attempts to load contents from a given input stream.

Returns

whether the input was well-formatted and loading was successful.

Implements [Serializer](#).

10.278.1.5 newPersistenceRecord()

```
PersistenceRecord * JsonSerializer::newPersistenceRecord ( ) [override], [virtual]
```

Creates an empty persistence record.

Although the returned record can be freed normally with `delete`, care must be taken to only use records which come from a implementation, when configuring that implementation's contents.

Implements [Serializer](#).

10.278.1.6 put()

```
bool JsonSerializer::put (
    const std::string name,
    const std::string type,
    const Util::identification id,
    PersistenceRecord * fields ) [override], [virtual]
```

Upserts an entry into this structure.

Parameters

<i>name</i>	a label which uniquely identifies the entry.
<i>type</i>	a type name.
<i>id</i>	a component/datadef ID number.
<i>fields</i>	a persistence record for the entry, whose contents will be copied if valid.

Returns

whether the entry was valid and was actually copied into this structure.

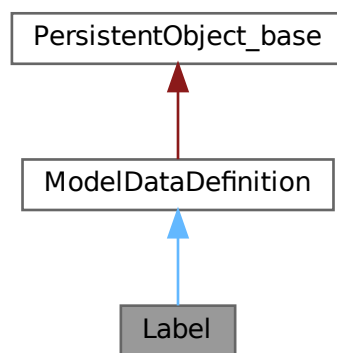
Implements [Serializer](#).

The documentation for this class was generated from the following files:

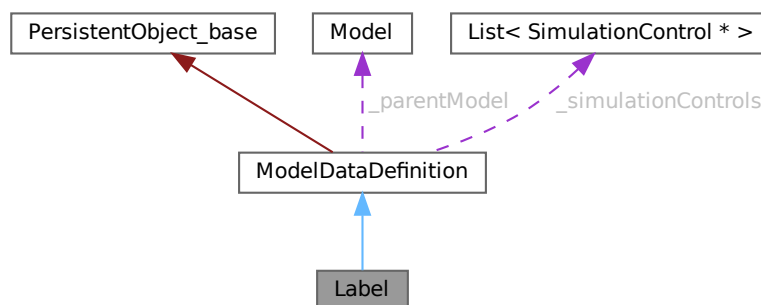
- kernel/simulator/persistence/JsonSerializer.h
- kernel/simulator/persistence/JsonSerializer.cpp

10.279 Label Class Reference

Inheritance diagram for Label:



Collaboration diagram for Label:



Public Member Functions

- **Label** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **setLabel** (std::string _label)
- std::string **getLabel** () const
- void **setEnterIntoLabelComponent** ([ModelComponent](#) *enteringLabelComponent)
- [ModelComponent](#) * **getEnterIntoLabelComponent** () const
- void **sendEntityToLabelComponent** ([Entity](#) *entity, double timeDelay)

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↔ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * **getSimulationControls** () const

- Returns the simulation controls owned by this data definition.*
- `TraceManager::Level` [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.279.1 Member Function Documentation

10.279.1.1 [_check\(\)](#)

```
bool Label::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.279.1.2 [_loadInstance\(\)](#)

```
bool Label::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.279.1.3 [_saveInstance\(\)](#)

```
void Label::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.279.1.4 show()

```
std::string Label::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/Logic/Label.h
- plugins/data/Logic/Label.cpp

10.280 QCPLabelPainterPrivate::LabelData Struct Reference

Public Attributes

- [AnchorSide](#) **side**
- double **rotation**
- QTransform **transform**
- QString **basePart**
- QString **expPart**
- QString **suffixPart**
- QRect **baseBounds**
- QRect **expBounds**
- QRect **suffixBounds**
- QRect **totalBounds**
- QRect **rotatedTotalBounds**
- QFont **baseFont**
- QFont **expFont**
- QColor **color**

The documentation for this struct was generated from the following file:

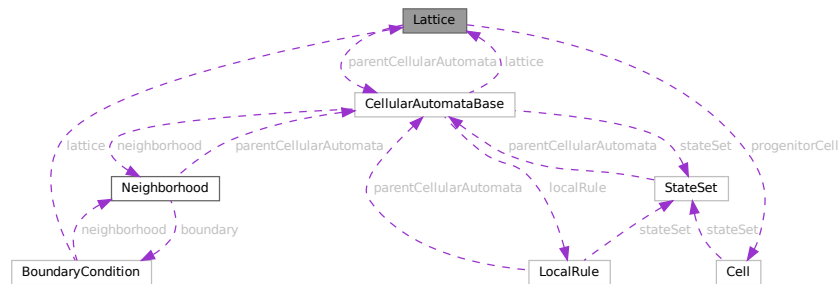
- applications/gui/genesys/qcustomplot.h

10.281 Lattice Class Reference

n-dimensional lattice

```
#include <plugins/components/ModalModel/CellularAutomata/Lattice.h>
```

Collaboration diagram for Lattice:



Public Member Functions

- **Lattice** ([CellularAutomataBase](#) *parentCellularAutomata, [Cell](#) *progenitorCell=nullptr, std::vector< unsigned short > dimensions={}, LatticeType latticeType=LatticeType::RETICULAR)
- **Lattice** (const [Lattice](#) &orig)
- virtual std::string **show** ()
- bool **init** ()
 - init all cells*
- virtual long **cellNDimPosition2Number** (const std::vector< int > position)
 - Maps n-dimensional to cell number.*
- virtual std::vector< int > **cellNumber2NDimPosition** (const long cellNumber)
 - Maps cell number to n-dimensional.*
- [Cell](#) * **getCell** (const long cellNumber)
- [Cell](#) * **getCell** (const std::vector< int > position)
- void **setCell** (const std::vector< int > position, [Cell](#) *cell)
- bool **setCellState** (std::vector< int > position, [State](#) *state=nullptr, [Cell](#) *cell=nullptr)
- bool **setCellState** (long cellNumber, [State](#) *state=nullptr, [Cell](#) *cell=nullptr)
- void **setProgenitorCell** ([Cell](#) *progenitorCell)
 - set cell used to fill the lattice*
- [Cell](#) * **getProgenitorCell** () const
- bool **setNumCells** (unsigned short dimension, unsigned int numCells, bool createDimensions=true)
- unsigned short **getNumDimensions** ()
- unsigned int **getNumCell** (unsigned short dimension)
- LatticeType **getLatticeType** () const
- unsigned long **getCellsSize** () const
- unsigned long **getTotalCells** () const
- unsigned long **calculateTotalCells** ()
- bool **isInit** () const
- void **setDimensions** (std::vector< unsigned short > dimensions)
- std::vector< unsigned short > **getDimensions** () const
- unsigned short **getDimension** (unsigned short dimensionNumber) const
- std::vector< [Cell](#) * > **getCells** () const

Protected Attributes

- [CellularAutomataBase](#) * **parentCellularAutomata**
- std::vector< [Cell](#) * > **cells**
- std::vector< unsigned short > **dimensions**
- [Cell](#) * **progenitorCell**
- LatticeType **latticeType**
- bool **hasBeenInit**
- unsigned long **totalCells**

10.281.1 Detailed Description

n-dimensional lattice

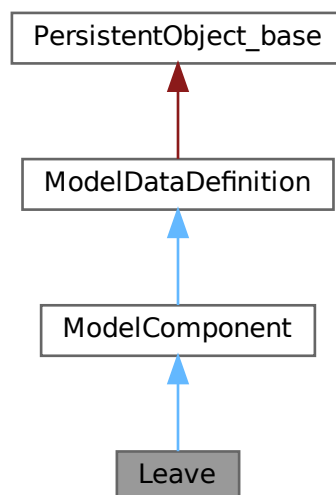
The documentation for this class was generated from the following files:

- plugins/components/ModalModel/CellularAutomata/Lattice.h
- plugins/components/ModalModel/CellularAutomata/Lattice.cpp

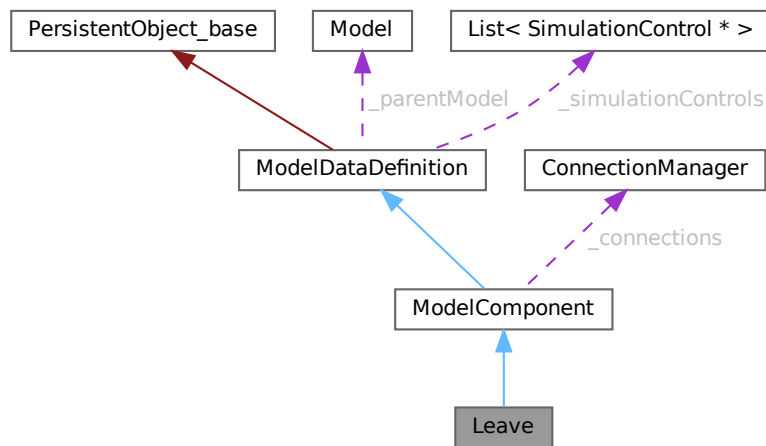
10.282 Leave Class Reference

```
#include <plugins/components/MaterialHandling/Leave.h>
```

Inheritance diagram for Leave:



Collaboration diagram for Leave:



Public Member Functions

- **Leave** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
virtual public methods
- void **setStation** ([Station](#) *_station)
- void **setStationName** (std::string stationName)
- [Station](#) * **getStation** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void **_createInternalStatisticReporters** () override
- virtual void **_createEditableDataDefinitions** () override
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _initBetweenReplications ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createAttachedAttributes ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
- *legacy helper methods used by the granular create* hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- *method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
- *not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
- *Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * [_connections](#) = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification [_id](#)
- std::string [_typename](#)
- bool [_reportStatistics](#)
- bool [_hasChanged](#)
- unsigned int [_modelLevel](#) = 0
- [Model](#) * [_parentModel](#)
- [List](#)< [SimulationControl](#) * > * [_simulationControls](#) = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level [_traceLevelSpecific](#)
- bool [_traceLevelSpecificEnabled](#) = false

10.282.1 Detailed Description

[Leave](#) module DESCRIPTION The [Leave](#) module is used to transfer an entity to a station or module. An entity may be transferred in two ways. It can be transferred to a module that defines a station by referencing the station and routing, conveying, or transporting to that station, or a graphical connection can be used to transfer an entity to another module. When an entity arrives at a [Leave](#) module, it may wait to obtain a transfer device (resource, transporter, or conveyor). When the transfer device has been obtained, the entity may experience a loading delay. Finally, the entity is transferred from this module to a destination module or station. TYPICAL USES The end of a part's production in a series of parallel processes where the part needs a forklift to be transferred to shipping PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart. Allocation Type of category to which the entity's incurred delay time and cost will be added. [Delay](#) Specifies a load time incurred after getting a transfer device. Units Time units used for the delay time. Transfer Out Determines whether a resource ([Seize Resource](#)), transporter (Request Transporter), or conveyor ([Access Conveyor](#)) is required prior to transferring the entity out of this module. Priority Indicates the priority of the module when either seizing a resource or requesting a transporter when there are entities waiting for that resource/transporter from other modules. This field is not visible when the Transfer Type is None or [Access Conveyor](#). Transporter Name Name of the transporter to request. [Queue](#) Type Type of queue, either a single [Queue](#), queue [Set](#), Internal queue, [Attribute](#), or Expression. [Queue](#) Name Name of the individual queue. [Queue Set](#) Name Name of the queue set. [Set](#) Index Defines the index into the queue set. Note that this is the index into the set and not the name of the queue in the set. [Queue Attribute](#) Name The attribute name that will be evaluated to indicate which queue is to be used. [Queue](#) Expression The expression that will be evaluated to indicate which queue is to be used. Selection Rule Method of selecting among available transporters in a set. Cyclical will cycle through available members. Random will randomly select a member. Preferred Order will always select the first available member. Specific Member requires an input attribute value to specify which member of the set (previously saved in the Save [Attribute](#) field). Largest Distance selects the transporter farthest away, and Smallest Distance selects the closest transporter. Save [Attribute Attribute](#) name used to store the index number into the set of the member that is chosen. This attribute can later be referenced with the Specific Member selection rule. Active when Transfer Out is Request Transporter. Index [Set Attribute](#) name whose value identifies the index number into the set of the member requested. The entity must have a value for the attribute before utilizing this option. [Resource](#) Type Type of resource for seizing, either specifying a particular [Resource](#), selecting from a pool of resources (that is, a resource [Set](#)), [Attribute](#), or Expression. [Resource](#) Name Name of the resource to seize. Conveyor Name Name of the conveyor to access.

10.282.2 of Cells Number of contiguous cells the entity requires.

Connect Type Determines if the entity is to [Route](#), Convey, or Transport to another station or Connect to another module. Move Time Time to route from this module to the destination station. Units Time units used for the move time. [Station](#) Type The entity's destination station type either an individual [Station](#), a station based on an [Attribute](#) or Expression value, or By [Sequence](#). [Station](#) Name Name of the individual destination station. [Attribute](#) Name The attribute name that will be evaluated to indicate the station. Expression The expression that will be evaluated to indicate the station.

10.282.3 Member Function Documentation

10.282.3.1 `_check()`

```
bool Leave::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.282.3.2 `_createEditableDataDefinitions()`

```
void Leave::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.282.3.3 `_createInternalStatisticReporters()`

```
void Leave::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.282.3.4 `_loadInstance()`

```
bool Leave::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.282.3.5 `_onDispatchEvent()`

```
void Leave::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.282.3.6 `_saveInstance()`

```
void Leave::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.282.3.7 `show()`

```
std::string Leave::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/MaterialHandling/Leave.h
- plugins/components/MaterialHandling/Leave.cpp

10.283 LicenceManager Class Reference

```
#include <kernel/simulator/LicenceManager.h>
```

Public Member Functions

- **LicenceManager** ([Simulator](#) *simulator)
- const std::string **showLicence** () const
- const std::string **showLimits** () const
- const std::string **showActivationCode** () const
- bool **lookforActivationCode** ()
- bool **insertActivationCode** ()
- void **removeActivationCode** ()
- unsigned int **getModelComponentsLimit** ()
- unsigned int **getModelDatasLimit** ()
- unsigned int **getEntityLimit** ()
- unsigned int **getHostsLimit** ()
- unsigned int **getThreadsLimit** ()

10.283.1 Detailed Description

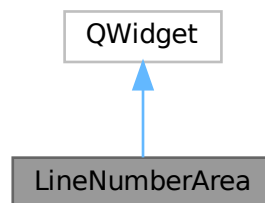
[LicenceManager](#) just prints a licence agreement and checks for some resource limits.

The documentation for this class was generated from the following files:

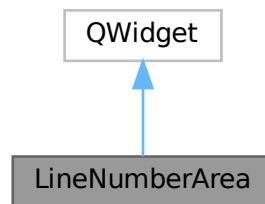
- kernel/simulator/LicenceManager.h
- kernel/simulator/LicenceManager.cpp

10.284 LineNumberArea Class Reference

Inheritance diagram for LineNumberArea:



Collaboration diagram for LineNumberArea:



Public Member Functions

- `LineNumberArea` (`CodeEditor *editor`)
- `QSize sizeHint ()` const override

Protected Member Functions

- void `paintEvent` (`QPaintEvent *event`) override

The documentation for this class was generated from the following file:

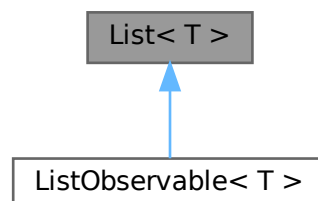
- `applications/gui/genesys/codeeditor/LineNumberArea.h`

10.285 List< T > Class Template Reference

Lightweight wrapper around `std::list` with convenience helpers used by the kernel.

```
#include <kernel/util/List.h>
```

Inheritance diagram for List< T >:



Public Types

- `using CompFunct = std::function< bool(const T, const T) >`

Public Member Functions

- `List ()`
Creates an empty list and positions the internal iterator at the beginning.
- `List (const List< T > &origin)`
Creates a copy of the source list.
- `List< T > & operator= (const List< T > &origin)`
- `unsigned int size ()`
Returns the number of stored elements.
- `bool empty ()`
Indicates whether the list is empty.
- `void clear ()`
Removes all elements from the list.
- `void pop_front ()`
Removes the first element from the list.
- `template<class Compare >`
`void sort (Compare comp)`
Sorts elements using the provided comparator.
- `std::list< T > * list () const`
Returns direct access to the encapsulated `std::list` structure.
- `T create ()`
Creates a new default element of type `T`.
- `template<typename U >`
`T create (U arg)`
Creates a new `T` element using a construction argument.
- `std::string show ()`
Generates a textual representation of elements for debugging.
- `std::list< T >::iterator find (T element)`
Searches for an element and returns an iterator to it (or `end ()`).
- `void insert (T element)`
Inserts an element preserving the current ordering policy.
- `void remove (T element)`
Removes all occurrences of the provided element.
- `void setAtRank (unsigned int rank, T element)`
Replaces (or appends) an element at a specific rank.
- `T getAtRank (unsigned int rank)`
Returns the element at the provided rank.
- `T next ()`
Advances the internal iterator and returns the next element.
- `T front ()`
Moves to the beginning and returns the first element.
- `T last ()`
Moves to the end and returns the last element.
- `T previous ()`
Moves the internal iterator backward and returns the previous element.
- `T current ()`
Returns the element at the current internal iterator position.
- `void setSortFunc (CompFunct _sortFunc)`
Sets the comparison function used for ordered insertions.

Protected Attributes

- `std::list< T > * _list`
- `CompFuncT _sortFunc`
- `std::list< T >::iterator _it`

Default function: insert at the end of the list.

10.285.1 Detailed Description

```
template<typename T>
class List< T >
```

Lightweight wrapper around `std::list` with convenience helpers used by the kernel.

The class centralizes insertion/removal, rank-based access and iterator-like navigation patterns heavily used in simulator internals and plugin infrastructure.

10.285.2 Member Data Documentation**10.285.2.1 _sortFunc**

```
template<typename T >
CompFuncT List< T >::_sortFunc [protected]
```

Initial value:

```
{[]}(const T, const T) {
    return false;
}}
```

The documentation for this class was generated from the following files:

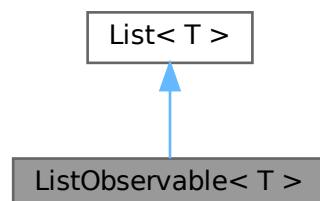
- `applications/gui/genesys/services/GraphicalModelBuilder.h`
- `kernel/util/List.h`

10.286 ListObservable< T > Class Template Reference

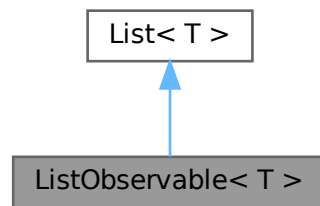
Observable-oriented list variant used by kernel structures.

```
#include <kernel/util/ListObservable.h>
```

Inheritance diagram for ListObservable< T >:



Collaboration diagram for `ListObservable< T >`:



Public Types

- `using CompFunc` = `std::function< bool(const T, const T) >`

Public Types inherited from `List< T >`

- `using CompFunc` = `std::function< bool(const T, const T) >`

Public Member Functions

- **`ListObservable ()`**
Initializes an empty observable list.
- **`void clear ()`**
Removes all elements from the observable list.
- **`void pop_front ()`**
Removes the first element from the observable list.
- **`void insert (T element)`**
Inserts an element respecting the configured ordering.
- **`void remove (T element)`**
Removes an element from the observable list.
- **`void setAtRank (unsigned int rank, T element)`**
Sets the element at a specific list index.

Public Member Functions inherited from `List< T >`

- **`List ()`**
Creates an empty list and positions the internal iterator at the beginning.
- **`List (const List< T > &origin)`**
Creates a copy of the source list.
- **`List< T > & operator= (const List< T > &origin)`**
- **`unsigned int size ()`**
Returns the number of stored elements.
- **`bool empty ()`**

- Indicates whether the list is empty.*

 - **void clear ()**

Removes all elements from the list.
- **void pop_front ()**

Removes the first element from the list.
- **template<class Compare >**
void sort (Compare comp)

Sorts elements using the provided comparator.
- **std::list< T > * list () const**

Returns direct access to the encapsulated `std::list` structure.
- **T create ()**

Creates a new default element of type `T`.
- **template<typename U >**
T create (U arg)

Creates a new `T` element using a construction argument.
- **std::string show ()**

Generates a textual representation of elements for debugging.
- **std::list< T >::iterator find (T element)**

Searches for an element and returns an iterator to it (or `end ()`).
- **void insert (T element)**

Inserts an element preserving the current ordering policy.
- **void remove (T element)**

Removes all occurrences of the provided element.
- **void setAtRank (unsigned int rank, T element)**

Replaces (or appends) an element at a specific rank.
- **T getAtRank (unsigned int rank)**

Returns the element at the provided rank.
- **T next ()**

Advances the internal iterator and returns the next element.
- **T front ()**

Moves to the beginning and returns the first element.
- **T last ()**

Moves to the end and returns the last element.
- **T previous ()**

Moves the internal iterator backward and returns the previous element.
- **T current ()**

Returns the element at the current internal iterator position.
- **void setSortFunc (CompFunct _sortFunc)**

Sets the comparison function used for ordered insertions.

Additional Inherited Members

Protected Attributes inherited from List< T >

- **std::list< T > * _list**
 - **CompFunct _sortFunc**
 - **std::list< T >::iterator _it**
- Default function: insert at the end of the list.*

10.286.1 Detailed Description

```
template<typename T>  
class ListObservable< T >
```

Observable-oriented list variant used by kernel structures.

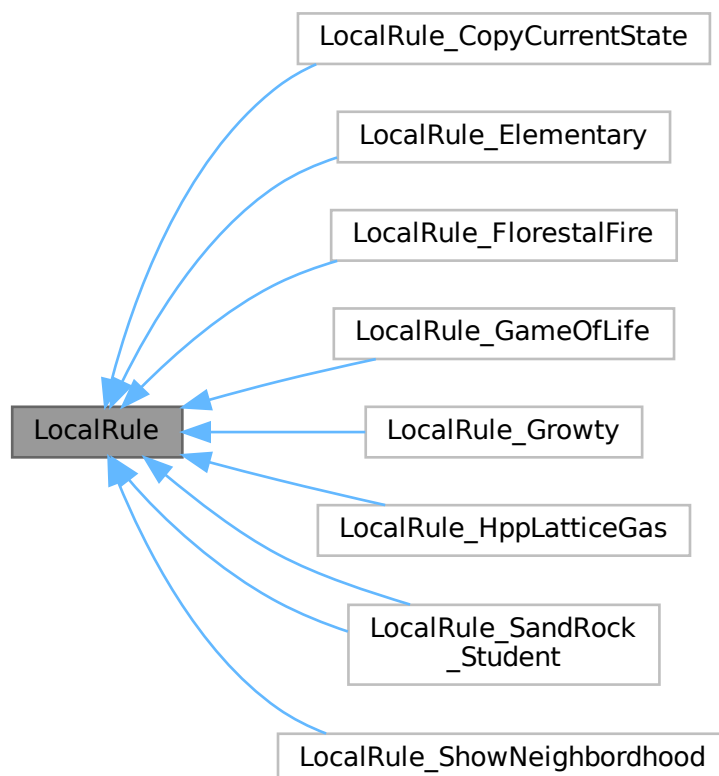
This template specializes `List` behavior for contexts where list mutations are relevant to external observers/managers, while preserving familiar list helpers.

The documentation for this class was generated from the following file:

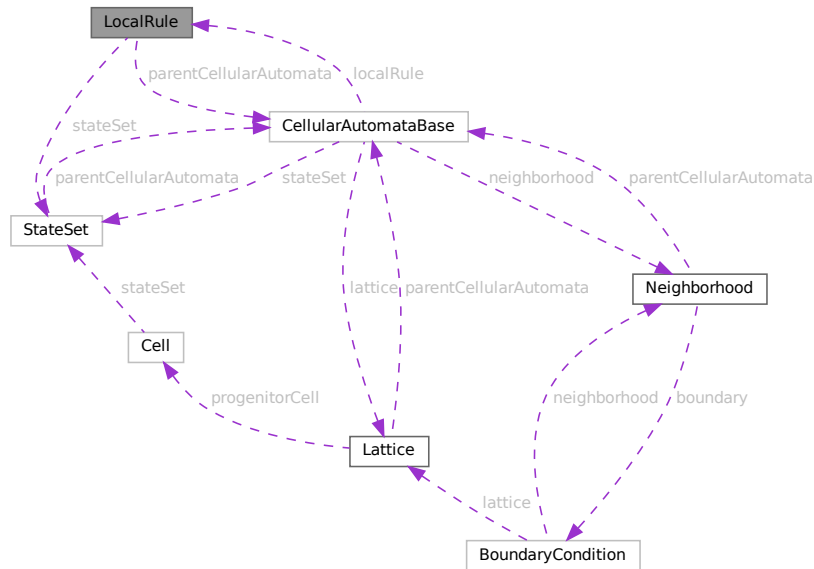
- kernel/util/ListObservable.h

10.287 LocalRule Class Reference

Inheritance diagram for LocalRule:



Collaboration diagram for LocalRule:



Public Member Functions

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- virtual void **applyRule** ([Cell](#) *cell)=0
Pure virtual method that has to be overriden by derived class, responsible for applying the local rule to a cell.
- void **setStateSet** ([StateSet](#) *stateSet)

Protected Attributes

- [CellularAutomataBase](#) * **parentCellularAutomata**
- [StateSet](#) * **stateSet**

10.287.1 Member Function Documentation

10.287.1.1 applyRule()

```
virtual void LocalRule::applyRule (
    Cell * cell ) [pure virtual]
```

Pure virtual method that has to be overriden by derived class, responsible for applying the local rule to a cell.

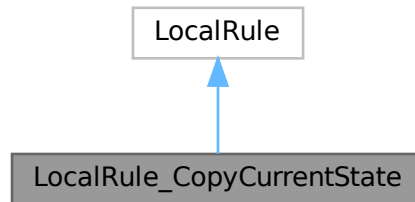
Implemented in [LocalRule_ShowNeighborhood](#), [LocalRule_CopyCurrentState](#), [LocalRule_Elementary](#), [LocalRule_FlorestalFire](#), [LocalRule_GameOfLife](#), [LocalRule_Growty](#), [LocalRule_HppLatticeGas](#), [LocalRule_SandRock_Student](#), and [LocalRule_SandRock_Student](#).

The documentation for this class was generated from the following file:

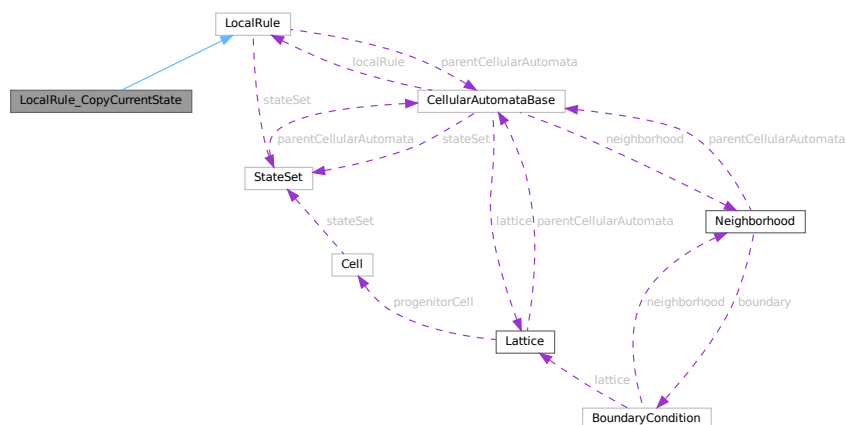
- plugins/components/ModalModel/CellularAutomata/LocalRule.h

10.288 LocalRule_CopyCurrentState Class Reference

Inheritance diagram for LocalRule_CopyCurrentState:



Collaboration diagram for LocalRule_CopyCurrentState:



Public Member Functions

- **LocalRule_CopyCurrentState** ([CellularAutomataBase](#) *parentCellularAutomata)
- void **applyRule** ([Cell](#) *cell) override
Pure virtual method that has to be overriden by derived class, responsible for applying the local rule to a cell.

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * `parentCellularAutomata`
- [StateSet](#) * `stateSet`

10.288.1 Member Function Documentation

10.288.1.1 `applyRule()`

```
void LocalRule_CopyCurrentState::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

Pure virtual method that has to be overiden by derived class, responsable for applying the local rule to a cell.

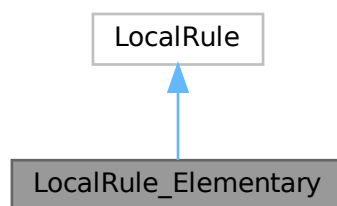
Implements [LocalRule](#).

The documentation for this class was generated from the following file:

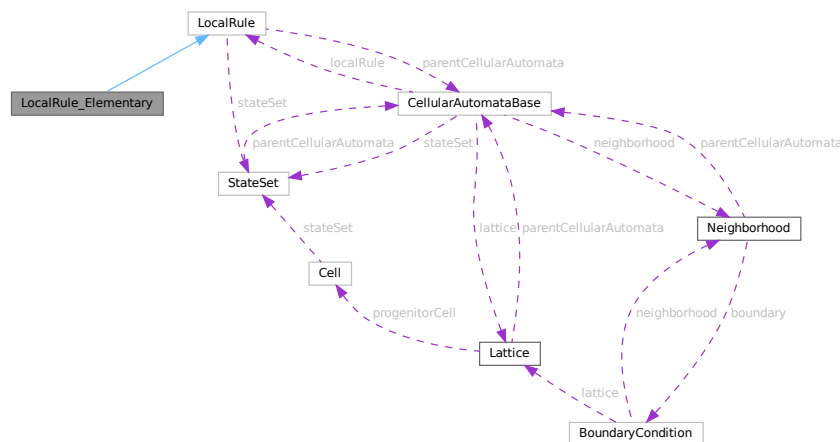
- applications/gui/genesys/cellular_automata/CellularAutomataDemoBuilder.h

10.289 LocalRule_Elementary Class Reference

Inheritance diagram for LocalRule_Elementary:



Collaboration diagram for LocalRule_Elementary:



Public Member Functions

- **LocalRule_Elementary** ([CellularAutomataBase](#) *parentCellularAutomata, uint8_t ruleNumber=30, [StateSet](#) *stateSet=nullptr)
- **LocalRule_Elementary** (const [LocalRule_Elementary](#) &orig)
- virtual void **applyRule** ([Cell](#) *cell) override
Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.
- uint8_t **getRuleNumber** () const
- void **setRuleNumber** (const uint8_t ruleNumber)

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**
- [StateSet](#) * **stateSet**

10.289.1 Member Function Documentation

10.289.1.1 applyRule()

```
virtual void LocalRule_Elementary::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

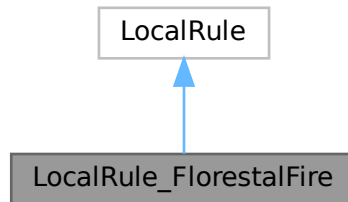
Implements [LocalRule](#).

The documentation for this class was generated from the following file:

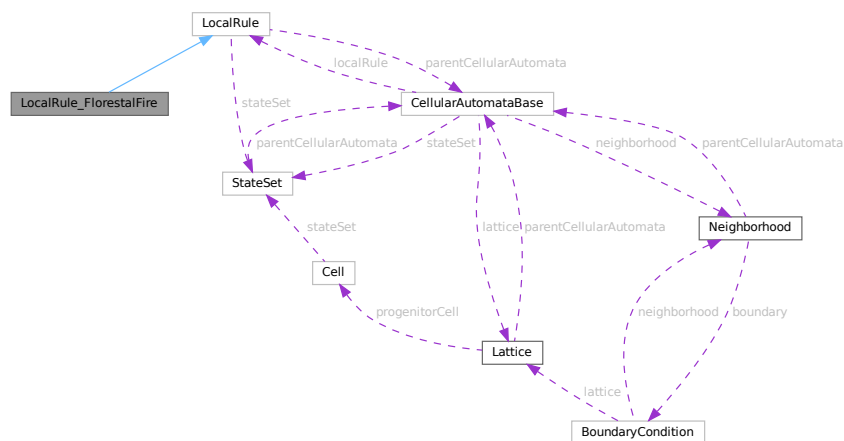
- plugins/components/ModalModel/CellularAutomata/LocalRule_Elementary.h

10.290 LocalRule_FlorestalFire Class Reference

Inheritance diagram for LocalRule_FlorestalFire:



Collaboration diagram for LocalRule_FlorestalFire:



Public Member Functions

- **LocalRule_FlorestalFire** ([CellularAutomataBase](#) *parentCellularAutomata)
- **LocalRule_FlorestalFire** (const [LocalRule_FlorestalFire](#) &orig)
- virtual void [applyRule](#) ([Cell](#) *cell) override

Pure virtual method that has to be overriden by derived class, responsible for applying the local rule to a cell.

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * `parentCellularAutomata`
- [StateSet](#) * `stateSet`

10.290.1 Member Function Documentation

10.290.1.1 `applyRule()`

```
virtual void LocalRule_FlorestalFire::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

Pure virtual method that has to be overiden by derived class, responsable for applying the local rule to a cell.

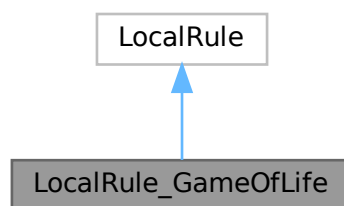
Implements [LocalRule](#).

The documentation for this class was generated from the following file:

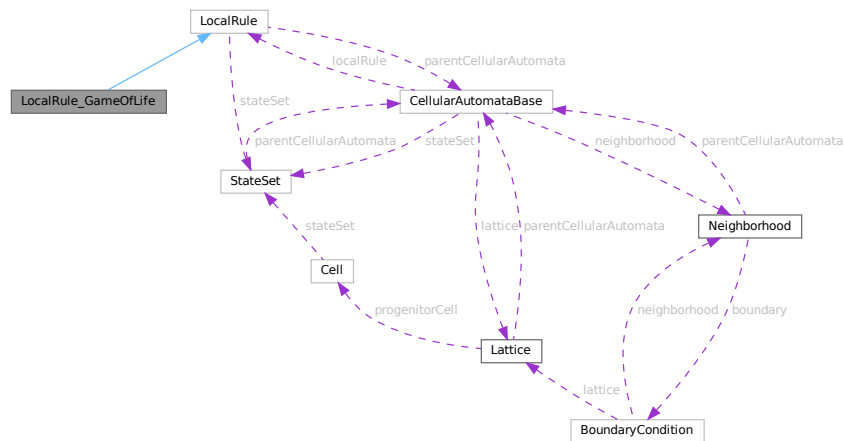
- `plugins/components/ModalModel/CellularAutomata/LocalRule_FlorestalFire.h`

10.291 [LocalRule_GameOfLife](#) Class Reference

Inheritance diagram for [LocalRule_GameOfLife](#):



Collaboration diagram for LocalRule_GameOfLife:



Public Member Functions

- **LocalRule_GameOfLife** ([CellularAutomataBase](#) *parentCellularAutomata)
- **LocalRule_GameOfLife** (const [LocalRule_GameOfLife](#) &orig)
- virtual void **applyRule** ([Cell](#) *cell) override

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**
- [StateSet](#) * **stateSet**

10.291.1 Member Function Documentation

10.291.1.1 **applyRule()**

```
virtual void LocalRule_GameOfLife::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

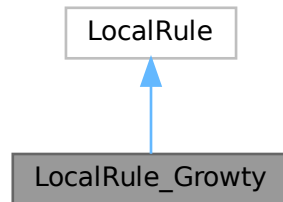
Implements [LocalRule](#).

The documentation for this class was generated from the following file:

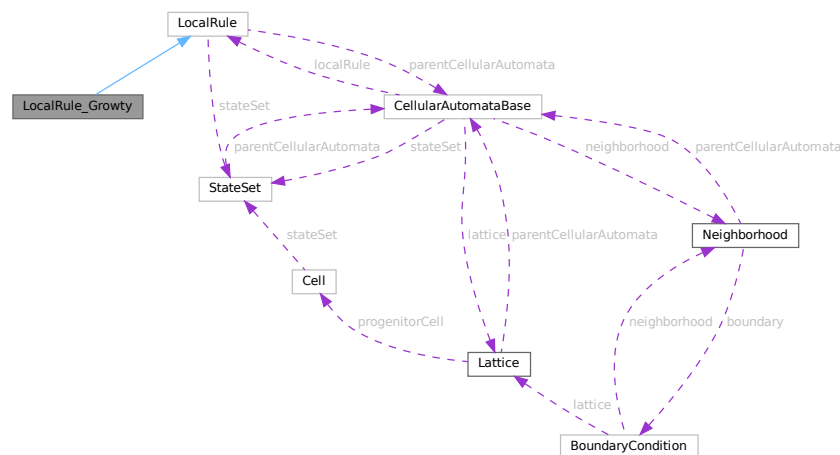
- plugins/components/ModalModel/CellularAutomata/LocalRule_GameOfLife.h

10.292 LocalRule_Growty Class Reference

Inheritance diagram for LocalRule_Growty:



Collaboration diagram for LocalRule_Growty:



Public Member Functions

- **LocalRule_Growty** ([CellularAutomataBase](#) *parentCellularAutomata)
- **LocalRule_Growty** (const [LocalRule_Growty](#) &orig)
- virtual void [applyRule](#) ([Cell](#) *cell) override

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * `parentCellularAutomata`
- [StateSet](#) * `stateSet`

10.292.1 Member Function Documentation

10.292.1.1 `applyRule()`

```
virtual void LocalRule_Growty::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

Pure virtual method that has to be overriden by derived class, responsable for applying the local rule to a cell.

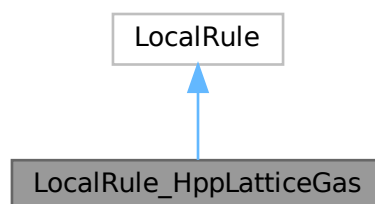
Implements [LocalRule](#).

The documentation for this class was generated from the following file:

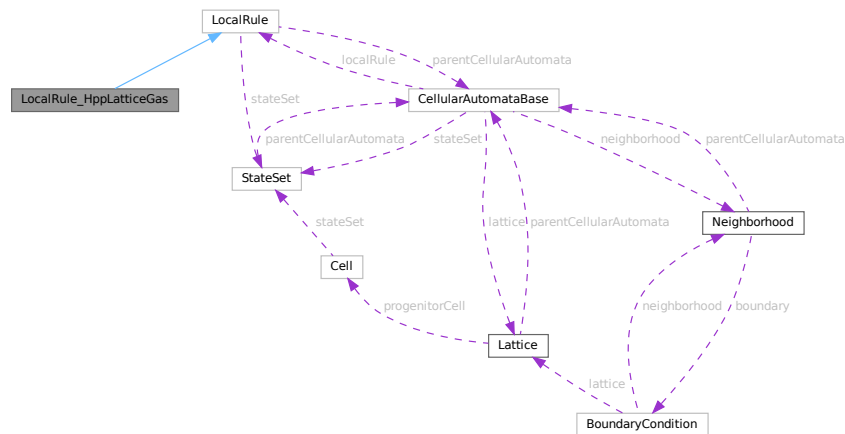
- `plugins/components/ModalModel/CellularAutomata/LocalRule_Growty.h`

10.293 LocalRule_HppLatticeGas Class Reference

Inheritance diagram for LocalRule_HppLatticeGas:



Collaboration diagram for LocalRule_HppLatticeGas:



Public Member Functions

- **LocalRule_HppLatticeGas** ([CellularAutomataBase](#) *parentCellularAutomata)
- void **applyRule** ([Cell](#) *cell) override

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**
- [StateSet](#) * **stateSet**

10.293.1 Member Function Documentation

10.293.1.1 applyRule()

```
void LocalRule_HppLatticeGas::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

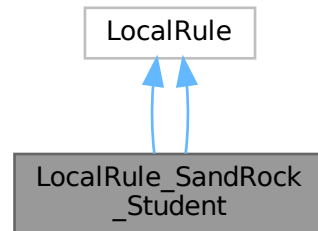
Implements [LocalRule](#).

The documentation for this class was generated from the following file:

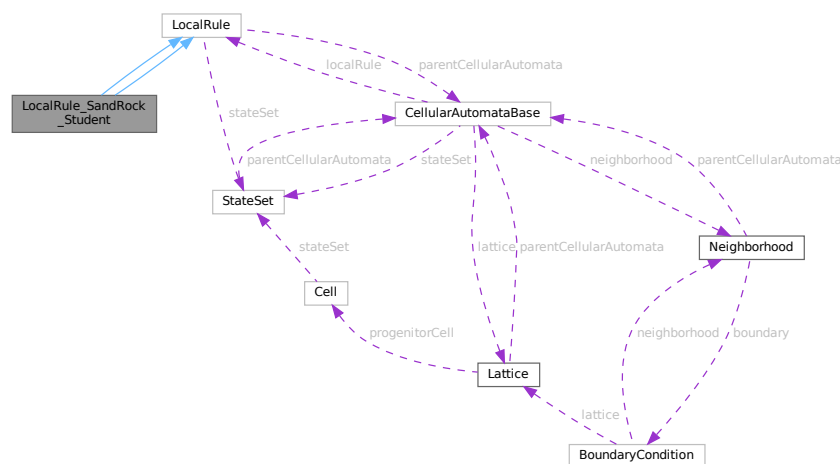
- plugins/components/ModalModel/CellularAutomata/LocalRule_HppLatticeGas.h

10.294 LocalRule_SandRock_Student Class Reference

Inheritance diagram for LocalRule_SandRock_Student:



Collaboration diagram for LocalRule_SandRock_Student:



Public Member Functions

- **LocalRule_SandRock_Student** (`CellularAutomataBase` *parentCellularAutomata)
- **LocalRule_SandRock_Student** (const `LocalRule_SandRock_Student` &orig)
- virtual void **applyRule** (`Cell` *cell) override

Pure virtual method that has to be overriden by derived class, responsible for applying the local rule to a cell.

- **LocalRule_SandRock_Student** (`CellularAutomataBase` *parentCellularAutomata)
- **LocalRule_SandRock_Student** (const `LocalRule_SandRock_Student` &orig)
- virtual void **applyRule** (`Cell` *cell) override

Pure virtual method that has to be overriden by derived class, responsible for applying the local rule to a cell.

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Static Public Attributes

- static constexpr long **EMPTY** = 0
- static constexpr long **SAND** = 1
- static constexpr long **ROCK** = 2

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**
- [StateSet](#) * **stateSet**

10.294.1 Member Function Documentation

10.294.1.1 `applyRule()` [1/2]

```
virtual void LocalRule_SandRock_Student::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

Implements [LocalRule](#).

10.294.1.2 `applyRule()` [2/2]

```
virtual void LocalRule_SandRock_Student::applyRule (
    Cell * cell ) [inline], [override], [virtual]
```

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

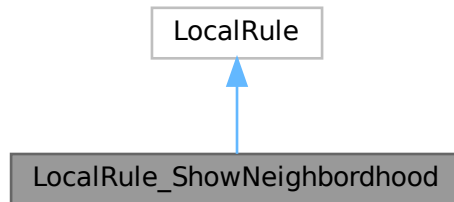
Implements [LocalRule](#).

The documentation for this class was generated from the following files:

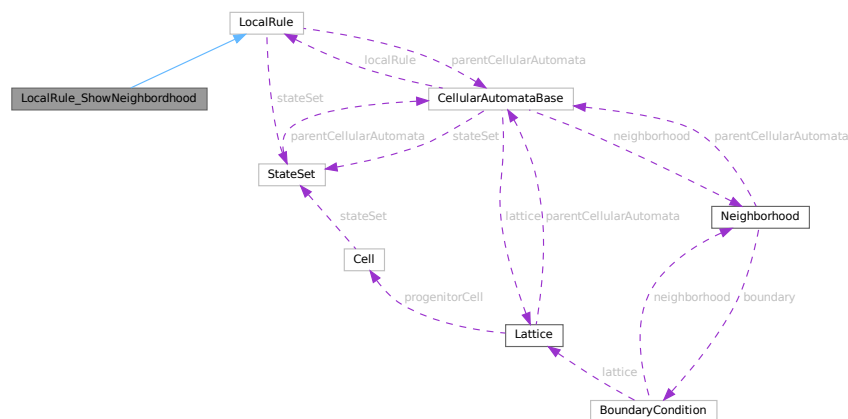
- plugins/components/ModalModel/CellularAutomata/LocalRule_SandRock.h
- plugins/components/ModalModel/CellularAutomata/LocalRule_SandRock_Student.h

10.295 LocalRule_ShowNeighborhood Class Reference

Inheritance diagram for LocalRule_ShowNeighborhood:



Collaboration diagram for LocalRule_ShowNeighborhood:



Public Member Functions

- **LocalRule_ShowNeighborhood** ([CellularAutomataBase](#) *parentCellularAutomata)
- **LocalRule_ShowNeighborhood** (const [LocalRule_ShowNeighborhood](#) &orig)
- virtual void **applyRule** ([Cell](#) *cell)

Pure virtual method that has to be overiden by derived class, responsible for applying the local rule to a cell.

Public Member Functions inherited from [LocalRule](#)

- **LocalRule** ([CellularAutomataBase](#) *parentCellularAutomata, [StateSet](#) *stateSet=nullptr)
- **LocalRule** (const [LocalRule](#) &orig)
- void **setStateSet** ([StateSet](#) *stateSet)

Additional Inherited Members

Protected Attributes inherited from [LocalRule](#)

- [CellularAutomataBase](#) * `parentCellularAutomata`
- [StateSet](#) * `stateSet`

10.295.1 Member Function Documentation

10.295.1.1 `applyRule()`

```
virtual void LocalRule_ShowNeighborhood::applyRule (  
    Cell * cell ) [inline], [virtual]
```

Pure virtual method that has to be overiden by derived class, responsable for applying the local rule to a cell.

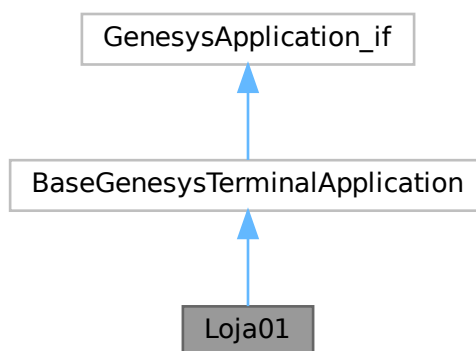
Implements [LocalRule](#).

The documentation for this class was generated from the following file:

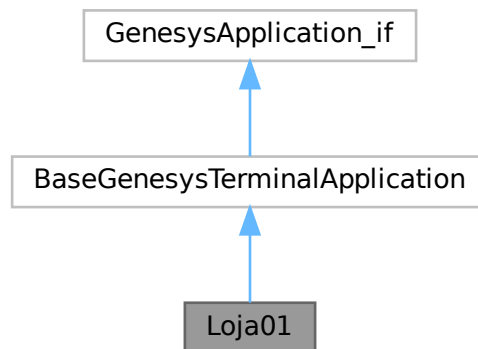
- `plugins/components/ModalModel/CellularAutomata/LocalRule_ShowNeighborhood.h`

10.296 Loja01 Class Reference

Inheritance diagram for Loja01:



Collaboration diagram for Loja01:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.296.1 Member Function Documentation

10.296.1.1 main()

```
int Loja01::main (
    int argc,
    char ** argv )  [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

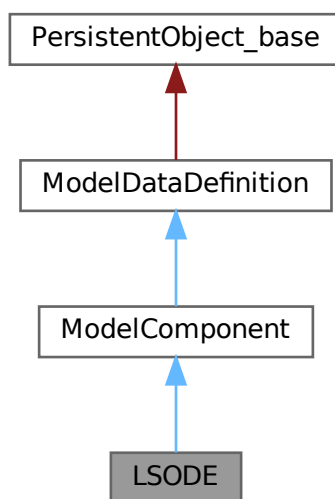
The documentation for this class was generated from the following files:

- applications/modelSpecific/teaching/Loja/Loja01.h
- applications/modelSpecific/teaching/Loja/Loja01.cpp

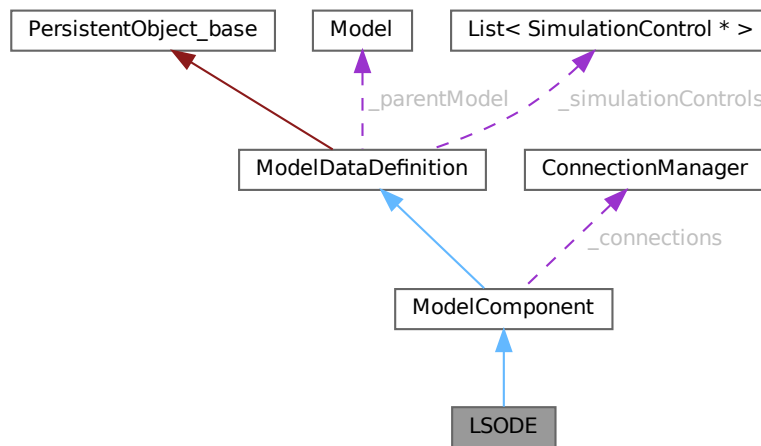
10.297 LODE Class Reference

```
#include <plugins/components/Continuous/LSODE.h>
```

Inheritance diagram for LODE:



Collaboration diagram for LODE:



Public Member Functions

- **LSODE** (**Model** *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- void **setTimeVariable** (**Variable** *_timeVariable)
- **Variable** * **getTimeVariable** () const
- void **setStep** (double _step)
- double **getStep** () const
- void **setVariable** (**Variable** *_variables)
- **Variable** * **getVariable** () const
- **List**< std::string > * **getDiffEquations** () const
- void **addDiffEquation** (std::string newDiffEquation)
- void **removeDiffEquation** (std::string diffEquation)
- void **setFilename** (std::string filename)
- std::string **getFileName** () const

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- **ConnectionManager** * **getConnectionManager** () const
- *new public user methods for this component*
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
- *Connects this component to another component.*
- bool **hasBreakpointAt** ()
- *Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string _description)

- Updates the free-form description shown by the GUI.*
- `std::string getDescription () const`
Returns the free-form description associated with this component.
- `void remove (ModelComponent *comp)`
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
 IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification getId () const
Returns the unique identifier of this instance.
- void setName (const std::string &name)
Updates the logical name of this instance.
- const std::string & getName () const
Returns the logical name of this instance.
- void setLabel (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & getLabel () const
Returns the user-visible label of this instance.
- const std::string & getClassname () const
Returns the runtime class name used by the plugin system.
- **Model** * getParentModel () const
Returns the parent model that owns this definition.
- bool isReportStatistics () const
Indicates whether this data definition contributes report statistics.
- void setReportStatistics (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * getInternalData (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * getInternalData () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * getAttachedData () const
Returns the full map of attached data definitions.
- bool hasChanged () const
Indicates whether this instance has been modified since the last reset.
- void setHasChanged (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int getLevel () const
Returns the level in the model hierarchy where this definition lives.
- void setModelLevel (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< **SimulationControl** * > * getSimulationControls () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level getTraceLevelSpecific () const
Returns the trace level that applies specifically to this definition.
- void setTraceLevelSpecific (TraceManager::Level level)

Updates the trace level used by this definition.

- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)

Defines the specific trace level and optionally enables it immediately.

- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void `_onDispatchEvent` (`Entity` *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool `_loadInstance` (`PersistenceRecord` *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` (`PersistenceRecord` *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from `ModelDataDefinition`

- bool `_getSaveDefaultsOption` ()
- virtual `ParserChangesInformation` * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` (`SimulationControl` *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, `ModelDataDefinition` *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()

- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.297.1 Detailed Description

This component ...

10.297.2 Member Function Documentation

10.297.2.1 _check()

```
bool LODE::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.297.2.2 `_loadInstance()`

```
bool LSODe::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.297.2.3 `_onDispatchEvent()`

```
void LSODe::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.297.2.4 `_saveInstance()`

```
void LSODe::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.297.2.5 `show()`

```
std::string LSODe::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

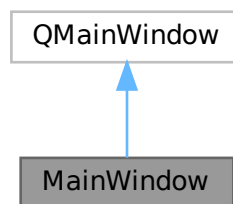
- `plugins/components/Continuous/LSODE.h`
- `plugins/components/Continuous/LSODE.cpp`

10.298 MainWindow Class Reference

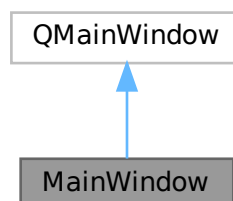
Main Qt window that composes controllers/services and preserves legacy GUI entry points.

```
#include <applications/gui/genesys/mainwindow.h>
```

Inheritance diagram for MainWindow:



Collaboration diagram for MainWindow:



Public Member Functions

- **MainWindow** (QWidget *parent=nullptr)
Builds the composition root, wiring controllers/services and legacy Qt surface.
- **~MainWindow** ()
Releases UI resources while preserving existing Qt ownership semantics.
- [ModelGraphicsScene](#) * **myScene** () const
Returns the current model graphics scene used by delegated controllers/services.
- bool **graphicalModelHasChanged** () const
Indicates whether graphical representation has unsaved modifications.
- void **setGraphicalModelHasChanged** (bool [graphicalModelHasChanged](#))
Updates graphical-change flag used by persistence/action synchronization.
- void **unselectDrawIcons** ()
Compatibility wrapper that clears currently selected draw tool actions.
- bool **checkSelectedDrawIcons** ()
Compatibility wrapper that reports whether any draw tool action is selected.
- void **refreshGuiExtensions** ()
Public refresh hook for graphical extension contributions.
- void **refreshPluginCatalog** ()
Public refresh hook for plugin catalog tree.

Protected Member Functions

- void **closeEvent** (QCloseEvent *event) override

Friends

- class **SimulationEventController**

10.298.1 Detailed Description

Main Qt window that composes controllers/services and preserves legacy GUI entry points.

[MainWindow](#) remains the composition root of GenesysQtGUI: it owns the generated UI, wires extracted controllers/services, and exposes the original Qt slot surface expected by actions, widgets, and kernel callback registration. In the refactored architecture it primarily acts as a compatibility façade that delegates operational responsibilities to focused collaborators.

Responsibilities:

- bootstrap UI construction and dependency wiring for controllers/services;
- host compatibility wrappers used by menus/toolbars/scene callbacks;
- coordinate cross-cutting state that still spans multiple controllers.

Boundaries:

- detailed domain workflows (simulation commands, lifecycle, persistence, scene tools, property editing, trace/event rendering) are delegated to dedicated controller/service classes whenever extraction already exists;

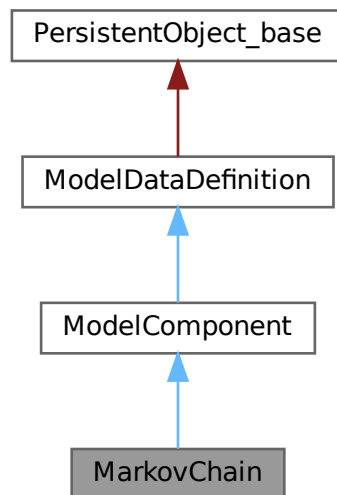
- [MainWindow](#) does not replace kernel ownership semantics.

The documentation for this class was generated from the following files:

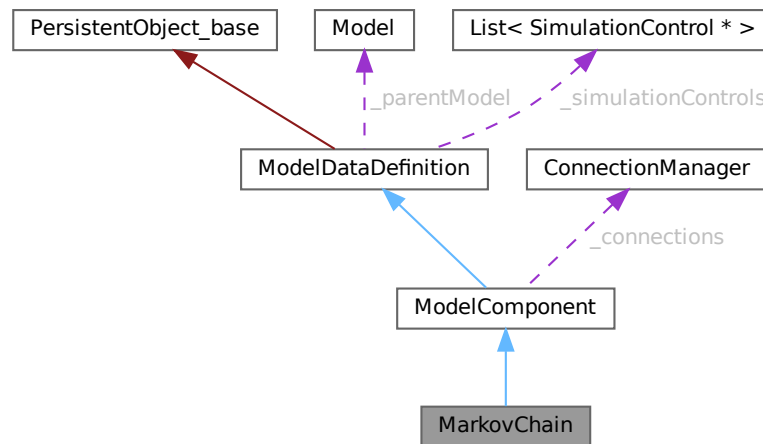
- applications/gui/genesys/mainwindow.h
- applications/gui/genesys/[mainwindow.cpp](#)
- applications/gui/genesys/[mainwindow_controller.cpp](#)
- applications/gui/genesys/[mainwindow_modelrepresentations.cpp](#)
- applications/gui/genesys/[mainwindow_scene.cpp](#)
- applications/gui/genesys/[mainwindow_simulator.cpp](#)

10.299 MarkovChain Class Reference

Inheritance diagram for MarkovChain:



Collaboration diagram for MarkovChain:



Public Member Functions

- **MarkovChain** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
- *virtual public methods*
- void **setTransitionProbabilityMatrix** ([ModelDataDefinition](#) *transitionMatrix)
- [ModelDataDefinition](#) * **getTransitionMatrix** () const
- void **setInitialDistribution** ([ModelDataDefinition](#) *initialDistribution)
- [ModelDataDefinition](#) * **getInitialDistribution** () const
- void **setCurrentState** ([ModelDataDefinition](#) *currentState)
- [ModelDataDefinition](#) * **getCurrentState** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * [getConnectionManager](#) () const
- *new public user methods for this component*
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- *Connects this component to another component.*
- bool [hasBreakpointAt](#) ()
- *Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string _description)
- *Updates the free-form description shown by the GUI.*
- std::string [getDescription](#) () const
- *Returns the free-form description associated with this component.*
- void [remove](#) ([ModelComponent](#) *comp)
- *Removes a connected component from this component.*

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()

Releases resources owned by this data definition.
- Util::identification [getId](#) () const

Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)

Updates the logical name of this instance.
- const std::string & [getName](#) () const

Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const

Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const

Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const

Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const

Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const

Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const

Returns the full map of attached data definitions.
- bool [hasChanged](#) () const

Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))

Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const

Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const

Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)

Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void **_initBetweenReplications** () override
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_createEditableDataDefinitions** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- `Util::identification _id`
- `std::string _typename`
- `bool _reportStatistics`
- `bool _hasChanged`
- `unsigned int _modelLevel = 0`
- [Model](#) * `_parentModel`
- `List< SimulationControl * > * _simulationControls` = new `List<SimulationControl*>()`
- `TraceManager::Level _traceLevelSpecific`
- `bool _traceLevelSpecificEnabled = false`

10.299.1 Member Function Documentation

10.299.1.1 `_check()`

```
bool MarkovChain::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.299.1.2 `_createEditableDataDefinitions()`

```
void MarkovChain::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.299.1.3 `_initBetweenReplications()`

```
void MarkovChain::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.299.1.4 `_loadInstance()`

```
bool MarkovChain::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.299.1.5 `_onDispatchEvent()`

```
void MarkovChain::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for [ModelComponents](#), not for [ModelDataElements](#).

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.299.1.6 `_saveInstance()`

```
void MarkovChain::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.299.1.7 `show()`

```
std::string MarkovChain::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

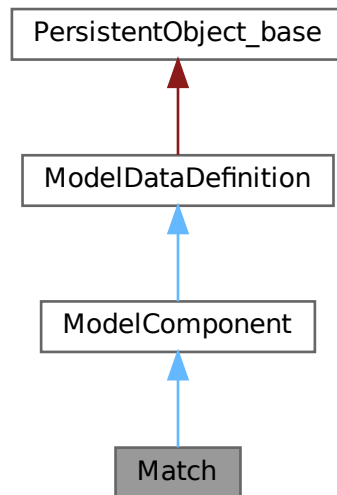
The documentation for this class was generated from the following files:

- plugins/components/AnalyticalModeling/MarkovChain.h
- plugins/components/AnalyticalModeling/MarkovChain.cpp

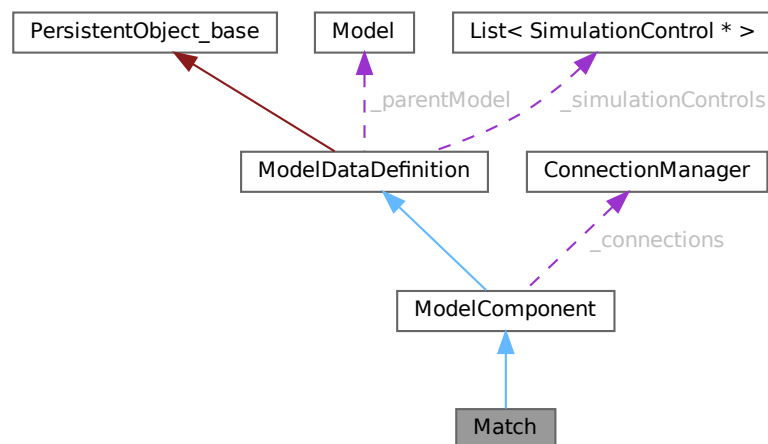
10.300 Match Class Reference

```
#include <plugins/components/Synchronization/Match.h>
```

Inheritance diagram for Match:



Collaboration diagram for Match:



Public Types

- enum class **Rule** : int { **Any** = 0 , **ByAttribute** = 1 , **num_elements** = 2 }

Public Member Functions

- **Match** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
virtual public methods
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- void **setRule** (Match::Rule _rule)
- Match::Rule **getRule** () const
- void **setAttributeName** (std::string _attributeName)
- std::string **getAttributeName** () const
- void **setMatchSize** (std::string _matchSize)
- std::string **getMatchSize** () const
- void **setNumberOfQueues** (unsigned int _numberOfQueues)
- unsigned int **getNumberOfQueues** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.

- `const std::string & getName () const`
Returns the logical name of this instance.
- `void setLabel (const std::string &label)`
Updates the user-visible label of this instance.
- `const std::string & getLabel () const`
Returns the user-visible label of this instance.
- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static std::string convertEnumToStr (Rule rule)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_createEditableDataDefinitions](#) () override
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()

- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
 - legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
 - virtual protected methods that could be overridden by derived classes, if needed*

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification_id
- std::string _typename
- bool _reportStatistics
- bool _hasChanged
- unsigned int _modelLevel = 0
- [Model](#) * _parentModel
- [List](#)< [SimulationControl](#) * > * _simulationControls = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level _traceLevelSpecific
- bool _traceLevelSpecificEnabled = false

10.300.1 Detailed Description

[Match](#) module DESCRIPTION The [Match](#) module brings together a specified number of entities waiting in different queues. The match may be accomplished when there is at least one entity in each of the desired queues. Additionally, an attribute may be specified such that the entities waiting in the queues must have the same attribute values before the match is initiated. When an entity arrives at the [Match](#) module, it is placed in one of up to five associated queues, based on the entry point to which it is connected. Entities will remain in their respective queues until a match exists. Once a match exists, one entity from each queue is released to be matched. The matched entities are then synchronized to depart from the module. TYPICAL USES Assembling a part Gathering various products for a customer order Synchronizing a customer exit with a filled order Prompt Description Name Unique module identifier displayed on the module shape. Number to [Match](#) Number of matching entities that must reside in different queues before a match may be completed. Type Method for matching the incoming entities. If Type is Any Entities, one entity must reside in each queue for a match to be made. If Type is Based on [Attribute](#), one entity must reside in each queue with the same attribute value. [Attribute](#) Name [Attribute](#) name that is used for identifying an arriving entity's match value. Applies only when Type is Based on [Attribute](#).

10.300.2 Member Function Documentation

10.300.2.1 _check()

```
bool Match::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.300.2.2 _createEditableDataDefinitions()

```
void Match::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.300.2.3 _loadInstance()

```
bool Match::_loadInstance (
    PersistenceRecord * fields ) [override], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.300.2.4 `_onDispatchEvent()`

```
void Match::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.300.2.5 `_saveInstance()`

```
void Match::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.300.2.6 `show()`

```
std::string Match::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

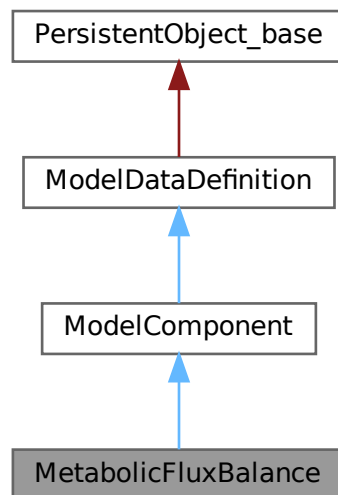
The documentation for this class was generated from the following files:

- plugins/components/Synchronization/Match.h
- plugins/components/Synchronization/Match.cpp

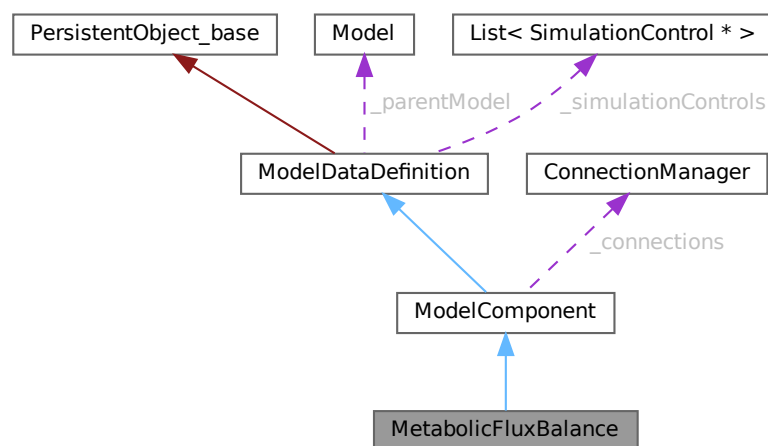
10.301 MetabolicFluxBalance Class Reference

```
#include <plugins/components/BiochemicalSimulation/MetabolicFluxBalance.h>
```

Inheritance diagram for MetabolicFluxBalance:



Collaboration diagram for MetabolicFluxBalance:



Public Member Functions

- **MetabolicFluxBalance** ([Model](#) *model, std::string name="")

- void **setMetabolicNetwork** ([MetabolicNetwork](#) *metabolicNetwork)
- [MetabolicNetwork](#) * **getMetabolicNetwork** () const
- void **setObjectiveReactionName** (std::string objectiveReactionName)
- std::string **getObjectiveReactionName** () const
- void **setLastSucceeded** (bool lastSucceeded)
- bool **getLastSucceeded** () const
- void **setLastObjectiveValue** (double lastObjectiveValue)
- double **getLastObjectiveValue** () const
- void **setLastMessage** (std::string lastMessage)
- std::string **getLastMessage** () const
- const std::map< std::string, double > & **getLastFluxes** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.

- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from ModelComponent

- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static bool Check (ModelComponent *component)`
Validates a component instance.
- `static void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- `static void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- `static void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.301.1 Detailed Description

Component that evaluates flux-balance analysis (FBA) over a [MetabolicNetwork](#).

10.301.1.1 LP solver dependency (GLPK)

[MetabolicFluxBalance](#) requires GLPK for metabolic networks with more than a few dozen reactions (e.g., M. genitalium: 641 reactions; yeast-GEM: 4131 reactions). Without GLPK, a built-in basis-enumeration fallback is used, which is only practical for small toy networks.

10.301.1.1.1 Installation

Ubuntu/Debian: `sudo apt-get install libglpk-dev` Fedora/RHEL: `sudo dnf install glpk-devel` macOS (Brew): `brew install glpk`

10.301.1.1.2 Re-configuration after installation

`cmake -preset terminal-app # or any other preset` `cmake -build build/terminal-app -target genesys_plugins_↔ components`

CMake will print at configure time: GenESyS: GLPK found — [MetabolicFluxBalance](#) ... will use GLPK LP solver or: GenESyS: GLPK not found — ... (suitable for small models only; install libglpk-dev ...)

The preprocessor guard is GENESYS_HAVE_GLPK. The GLPK solver wrapper is in `source/tools/Biochemical/↔ GlpkFluxBalanceSolver.h`.

10.301.2 Member Function Documentation

10.301.2.1 `_check()`

```
bool MetabolicFluxBalance::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.301.2.2 `_createEditableDataDefinitions()`

```
void MetabolicFluxBalance::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.301.2.3 `_loadInstance()`

```
bool MetabolicFluxBalance::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.301.2.4 `_onDispatchEvent()`

```
void MetabolicFluxBalance::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.301.2.5 `_saveInstance()`

```
void MetabolicFluxBalance::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.301.2.6 `show()`

```
std::string MetabolicFluxBalance::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

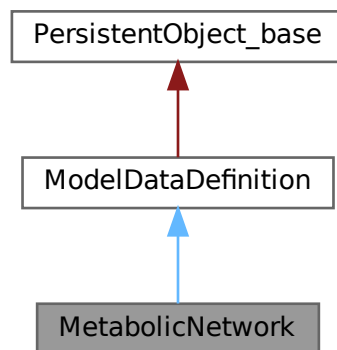
The documentation for this class was generated from the following files:

- `plugins/components/BiochemicalSimulation/MetabolicFluxBalance.h`
- `plugins/components/BiochemicalSimulation/MetabolicFluxBalance.cpp`

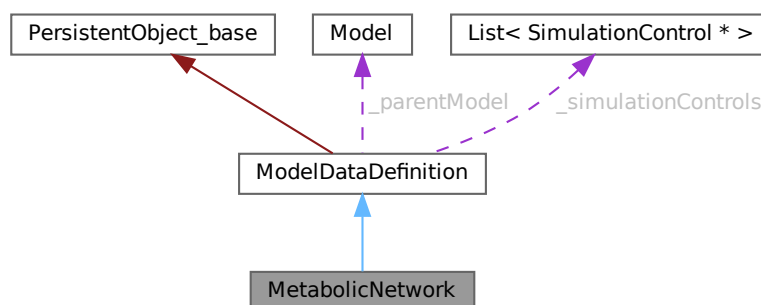
10.302 MetabolicNetwork Class Reference

```
#include <plugins/data/BiochemicalSimulation/MetabolicNetwork.h>
```

Inheritance diagram for MetabolicNetwork:



Collaboration diagram for MetabolicNetwork:



Public Member Functions

- **MetabolicNetwork** ([Model](#) *model, std::string name="")
- void **addReaction** (std::string reactionName)
- void **addExchangeSpecies** (std::string speciesName)
- void **clearReactions** ()
- void **clearExchangeSpecies** ()
- const std::vector< std::string > & **getReactionNames** () const
- const std::vector< std::string > & **getExchangeSpeciesNames** () const
- void **setObjectiveReactionName** (std::string objectiveReactionName)
- std::string **getObjectiveReactionName** () const
- void **setObjectiveSense** (std::string objectiveSense)
- std::string **getObjectiveSense** () const
- void **setCompartment** (std::string compartment)
- std::string **getCompartment** () const
- void **setEnabled** (bool enabled)
- bool **isEnabled** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- Releases resources owned by this data definition.*
- Util::identification **getId** () const
- Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & **getName** () const
- Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
- Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const
- Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const
- Returns the runtime class name used by the plugin system.*
- [Model](#) * **getParentModel** () const
- Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const
- Indicates whether this data definition contributes report statistics.*
- void **setReportStatistics** (bool reportStatistics)
- Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const
- Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const
- Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) * > * **getAttachedData** () const
- Returns the full map of attached data definitions.*

- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelSpecific`, bool `traceLevelSpecificEnabled`=true)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `ModelDataDefinition * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
- static `PluginInformation * GetPluginInformation` ()
- static `ModelDataDefinition * NewInstance` (`Model *model`, `std::string name=""`)

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition * LoadInstance` (`Model *model`, `PersistenceRecord *fields`, bool `insertIntoModel`)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition * NewInstance` (`Model *model`, `std::string name=""`)
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord *fields`, `ModelDataDefinition *modeldatum`)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition *modeldatum`, `std::string &errorMessage`)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition *modeldatum`)
Resets simulation-time state between replications.
- static void `CreateInternalData` (`ModelDataDefinition *modeldatum`)
Creates internal and attached data required by this instance.
- static bool `CreateRelatedDataElements` (`ModelDataDefinition *modeldatum`, `std::string &errorMessage`)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.302.1 Detailed Description

Aggregate metabolic network definition.

A [MetabolicNetwork](#) stores the reaction membership and objective metadata for future flux-balance style components.

10.302.2 Member Function Documentation

10.302.2.1 [_check\(\)](#)

```
bool MetabolicNetwork::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.302.2.2 [_createEditableDataDefinitions\(\)](#)

```
void MetabolicNetwork::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.302.2.3 `_loadInstance()`

```
bool MetabolicNetwork::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.302.2.4 `_saveInstance()`

```
void MetabolicNetwork::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.302.2.5 `show()`

```
std::string MetabolicNetwork::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

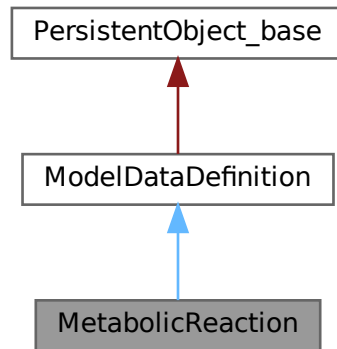
The documentation for this class was generated from the following files:

- `plugins/data/BiochemicalSimulation/MetabolicNetwork.h`
- `plugins/data/BiochemicalSimulation/MetabolicNetwork.cpp`

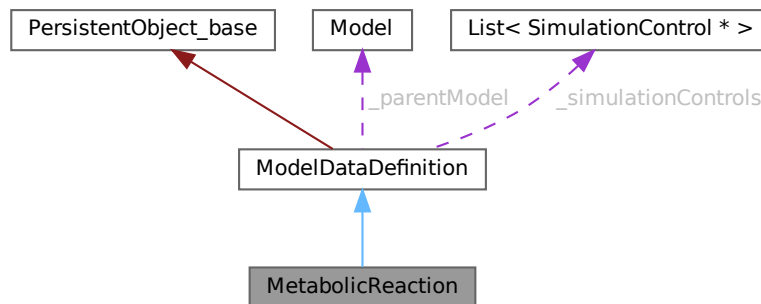
10.303 MetabolicReaction Class Reference

```
#include <plugins/data/BiochemicalSimulation/MetabolicReaction.h>
```

Inheritance diagram for MetabolicReaction:



Collaboration diagram for MetabolicReaction:



Classes

- struct [StoichiometricTerm](#)

Public Member Functions

- **MetabolicReaction** ([Model](#) *model, std::string name="")
- void **addReactant** (std::string speciesName, double stoichiometry=1.0)
- void **addProduct** (std::string speciesName, double stoichiometry=1.0)
- void **clearReactants** ()

- void **clearProducts** ()
- const std::vector< [StoichiometricTerm](#) > & **getReactants** () const
- const std::vector< [StoichiometricTerm](#) > & **getProducts** () const
- void **setLowerBound** (double lowerBound)
- double **getLowerBound** () const
- void **setUpperBound** (double upperBound)
- double **getUpperBound** () const
- void **setObjectiveCoefficient** (double objectiveCoefficient)
- double **getObjectiveCoefficient** () const
- void **setReversible** (bool reversible)
- bool **isReversible** () const
- void **setGeneRule** (std::string geneRule)
- std::string **getGeneRule** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↔ IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- Releases resources owned by this data definition.*
- Util::identification **getId** () const
- Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & **getName** () const
- Returns the logical name of this instance.*
- void **setLabel** (const std::string &label)
- Updates the user-visible label of this instance.*
- const std::string & **getLabel** () const
- Returns the user-visible label of this instance.*
- const std::string & **getClassname** () const
- Returns the runtime class name used by the plugin system.*
- [Model](#) * **getParentModel** () const
- Returns the parent model that owns this definition.*
- bool **isReportStatistics** () const
- Indicates whether this data definition contributes report statistics.*
- void **setReportStatistics** (bool reportStatistics)
- Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const
- Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const
- Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) * > * **getAttachedData** () const
- Returns the full map of attached data definitions.*
- bool **hasChanged** () const
- Indicates whether this instance has been modified since the last reset.*
- void **setHasChanged** (bool hasChanged)
- Updates the changed flag for this data definition.*

- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List< SimulationControl * > * getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition * LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation * GetPluginInformation](#) ()
- static [ModelDataDefinition * NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition * LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition * NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.303.1 Detailed Description

Constraint-based metabolic reaction definition.

10.303.2 Member Function Documentation

10.303.2.1 `_check()`

```
bool MetabolicReaction::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.303.2.2 `_createEditableDataDefinitions()`

```
void MetabolicReaction::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.303.2.3 `_loadInstance()`

```
bool MetabolicReaction::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.303.2.4 `_saveInstance()`

```
void MetabolicReaction::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.303.2.5 show()

```
std::string MetabolicReaction::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

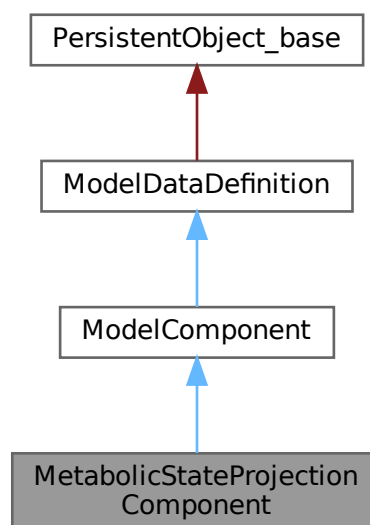
- plugins/data/BiochemicalSimulation/MetabolicReaction.h
- plugins/data/BiochemicalSimulation/MetabolicReaction.cpp

10.304 MetabolicStateProjectionComponent Class Reference

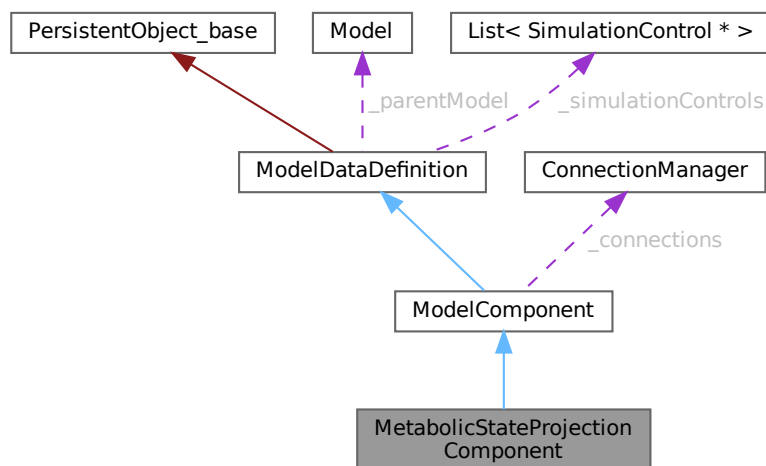
Projects flux-balance outputs into a [WholeCellState](#).

```
#include <plugins/components/WholeCellModeling/MetabolicStateProjectionComponent.h>
```

Inheritance diagram for MetabolicStateProjectionComponent:



Collaboration diagram for MetabolicStateProjectionComponent:



Classes

- struct [Projection](#)

Public Types

- enum class **ProjectionKind** { **MetaboliteAmount** , **CompartmentMetaboliteAmount** , **PathwayActivity** }
- enum class **ProjectionUpdateMode** { **Overwrite** , **Accumulate** , **Turnover** }

Public Member Functions

- **MetabolicStateProjectionComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setFluxBalanceComponent** ([MetabolicFluxBalance](#) *component)
- [MetabolicFluxBalance](#) * **getFluxBalanceComponent** () const
- void **setFluxBalanceComponentName** (std::string componentName)
- std::string **getFluxBalanceComponentName** () const
- void **clearObjectiveProjection** ()
- void **setObjectiveAsMetabolite** (const std::string &targetKey, double scale=1.0, double offset=0.0, ProjectionUpdateMode updateMode=ProjectionUpdateMode::Overwrite, double turnoverFraction=0.0)
- void **setObjectiveAsCompartmentMetabolite** (const std::string &compartmentName, const std::string &targetKey, double scale=1.0, double offset=0.0, ProjectionUpdateMode updateMode=ProjectionUpdateMode::Overwrite, double turnoverFraction=0.0)
- void **setObjectiveAsPathwayActivity** (const std::string &targetKey, double scale=1.0, double offset=0.0, ProjectionUpdateMode updateMode=ProjectionUpdateMode::Overwrite, double turnoverFraction=0.0)
- bool **hasObjectiveProjection** () const
- void **clearFluxProjections** ()

- void **addMetaboliteProjection** (const std::string &reactionName, const std::string &targetKey, double scale=1.0, double offset=0.0, ProjectionUpdateMode updateMode=ProjectionUpdateMode::Overwrite, double turnoverFraction=0.0)
 - void **addCompartmentMetaboliteProjection** (const std::string &reactionName, const std::string &compartmentName, const std::string &targetKey, double scale=1.0, double offset=0.0, ProjectionUpdateMode updateMode=ProjectionUpdateMode::Overwrite, double turnoverFraction=0.0)
 - void **addPathwayProjection** (const std::string &reactionName, const std::string &targetKey, double scale=1.0, double offset=0.0, ProjectionUpdateMode updateMode=ProjectionUpdateMode::Overwrite, double turnoverFraction=0.0)
 - unsigned int **getProjectionCount** () const
 - void **setLastSucceeded** (bool lastSucceeded)
 - bool **getLastSucceeded** () const
 - void **setLastMessage** (std::string lastMessage)
 - std::string **getLastMessage** () const
 - virtual std::string **show** () override
- virtual public methods*

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionType, std::string name="", bool insertIntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.

- `const std::string & getLabel () const`
Returns the user-visible label of this instance.
- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelspecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _initBetweenReplications ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createInternalStatisticReporters ()`
- `virtual void _createAttachedAttributes ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
- legacy helper methods used by the granular create* hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.304.1 Detailed Description

Projects flux-balance outputs into a [WholeCellState](#).

[MetabolicStateProjectionComponent](#) bridges FBA-style submodels with the whole-cell shared state. It reads the latest objective value and reaction fluxes computed by a [MetabolicFluxBalance](#) component and writes selected projections into [WholeCellState](#) as:

- global metabolite proxies;
- compartment-specific metabolite proxies;
- pathway activity summaries.

This component intentionally does not solve the metabolic optimization itself. It only transfers selected results to the whole-cell layer so that growth, checkpoints, and gene-expression submodels can consume them.

10.304.2 Member Function Documentation

10.304.2.1 `_check()`

```
bool MetabolicStateProjectionComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.304.2.2 _createEditableDataDefinitions()

```
void MetabolicStateProjectionComponent::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.304.2.3 _loadInstance()

```
bool MetabolicStateProjectionComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.304.2.4 _onDispatchEvent()

```
void MetabolicStateProjectionComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.304.2.5 _saveInstance()

```
void MetabolicStateProjectionComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.304.2.6 show()

```
std::string MetabolicStateProjectionComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

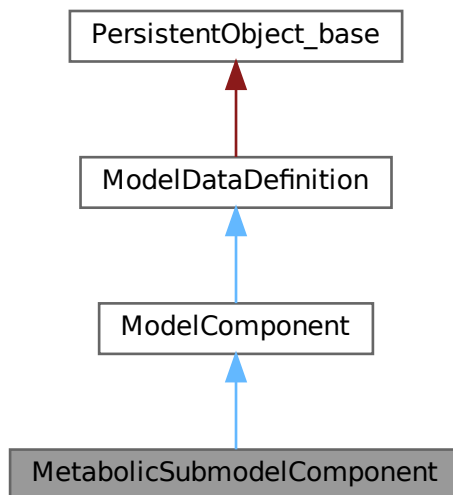
The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/MetabolicStateProjectionComponent.h
- plugins/components/WholeCellModeling/MetabolicStateProjectionComponent.cpp

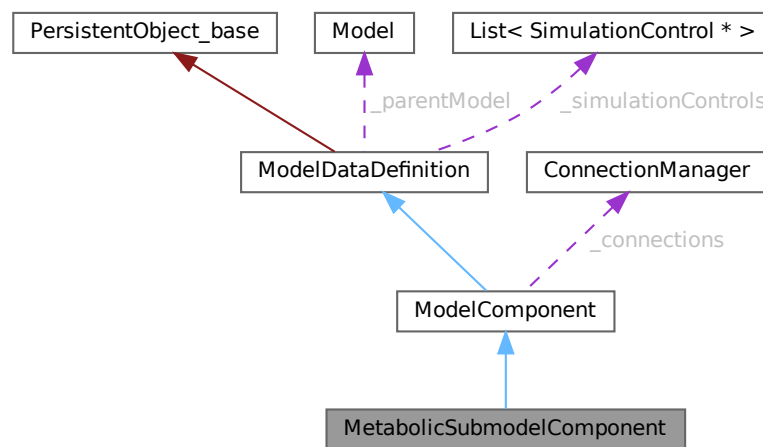
10.305 MetabolicSubmodelComponent Class Reference

```
#include <plugins/components/WholeCellModeling/MetabolicSubmodelComponent.h>
```

Inheritance diagram for MetabolicSubmodelComponent:



Collaboration diagram for MetabolicSubmodelComponent:



Public Member Functions

- **MetabolicSubmodelComponent** ([Model](#) *model, std::string name="")
 - void **setWholeCellState** ([WholeCellState](#) *state)
 - [WholeCellState](#) * **getWholeCellState** () const
 - void **setAtpYieldRate** (double rate)
 - double **getAtpYieldRate** () const
 - void **setConsumptionFraction** (double frac)
 - double **getConsumptionFraction** () const
 - void **setAtpPoolMax** (double maxPool)
 - double **getAtpPoolMax** () const
 - void **setDeltaT** (double dt)
 - double **getDeltaT** () const
 - virtual std::string **show** () override
- virtual public methods*

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()

Releases resources owned by this data definition.
- Util::identification [getId](#) () const

Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)

Updates the logical name of this instance.
- const std::string & [getName](#) () const

Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const

Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const

Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const

Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const

Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const

Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const

Returns the full map of attached data definitions.
- bool [hasChanged](#) () const

Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))

Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const

Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const

Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)

Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_initBetweenReplications** () override
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void **_createEditableDataDefinitions** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool \_getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * \_getParserChangesInformation ()`
- `virtual void \_addSimulationControl (SimulationControl *control)`
- `virtual void \_createInternalAndAttachedData ()`
- `virtual void \_createInternalStatisticReporters ()`
- `virtual void \_createAttachedAttributes ()`
- `virtual void \_createNonEditableDataDefinitions ()`
- `void \_templateCreateInternalStatisticReporters ()`
- `void \_templateCreateAttachedAttributes ()`
- `void \_templateCreateNonEditableDataDefinitions ()`
- `void \_templateCreateEditableDataDefinitions ()`
- `void \_internalStatisticReportersClear ()`
- *legacy helper methods used by the granular create* hooks*
- `void \_internalDataClear ()`
- `void \_internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void \_internalDataRemove (const std::string &key)`
- `void \_attachedDataClear ()`
- `void \_attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_attachedDataRemove (const std::string &key)`
- `void \_attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void \_statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_statisticReporterRemove (const std::string &key)`
- `void \_statisticReportersClear ()`
- `void \_mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryAttachedAttributeRemove (const std::string &key)`
- `void \_mandatoryAttachedAttributesClear ()`
- `void \_mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void \_mandatoryEditableDataDefinitionsClear ()`
- `void \_optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_optionalEditableDataDefinitionRemove (const std::string &key)`
- `void \_optionalEditableDataDefinitionsClear ()`
- `void \_mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void \_mandatoryNonEditableDataDefinitionsClear ()`
- `void \_optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void \_optionalNonEditableDataDefinitionsClear ()`
- `void \_checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- *method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.305.1 Detailed Description

Minimal metabolic submodel that produces ATP and NADPH each step.

Replaces a full FBA step with a volume-proportional energy production model calibrated for *M. genitalium* (Karr et al. 2012):

$$\text{ATP_produced} = \text{atpYieldRate} \times \text{cellVolume(fL)} \times \text{deltaT [mmol]} \quad \text{ATP_pool(new)} = \min(\text{ATP_pool} + \text{ATP_produced} - \text{ATP_consumed}, \text{atpPoolMax})$$

ATP consumption is estimated as a fixed fraction of the current pool (consumption rate = consumptionFraction × ATP_pool).

Results are written to the [WholeCellState](#) metabolite pool under keys "ATP" and "ADP". Downstream components can read "ATP" to gate reactions.

For a full FBA integration, replace this component with a proper [MetabolicFluxBalance](#) + network wiring. This component is intentionally minimal so that Phases 2–5 can be validated without GLPK-specific setup.

10.305.2 Member Function Documentation

10.305.2.1 `_check()`

```
bool MetabolicSubmodelComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.305.2.2 `_createEditableDataDefinitions()`

```
void MetabolicSubmodelComponent::_createEditableDataDefinitions ( ) [override], [protected],  
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.305.2.3 `_initBetweenReplications()`

```
void MetabolicSubmodelComponent::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.305.2.4 `_loadInstance()`

```
bool MetabolicSubmodelComponent::_loadInstance (   
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.305.2.5 `_onDispatchEvent()`

```
void MetabolicSubmodelComponent::_onDispatchEvent (   
    Entity * entity,   
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.305.2.6 `_saveInstance()`

```
void MetabolicSubmodelComponent::_saveInstance (   
    PersistenceRecord * fields,   
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.305.2.7 show()

```
std::string MetabolicSubmodelComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

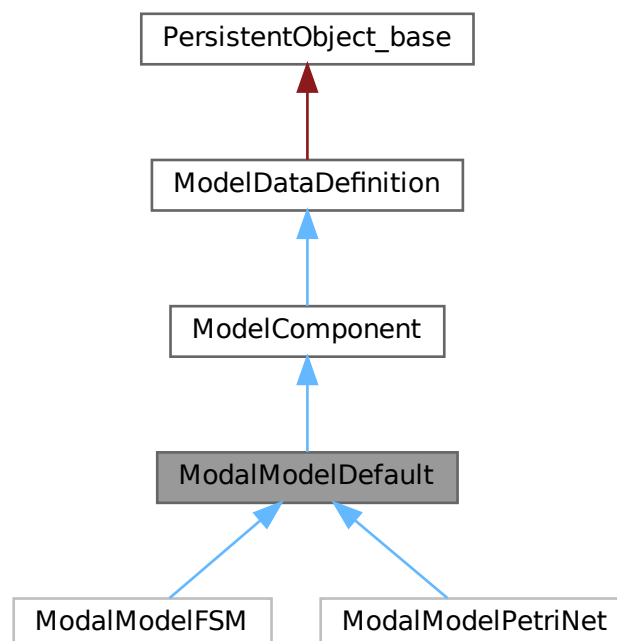
The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/MetabolicSubmodelComponent.h
- plugins/components/WholeCellModeling/MetabolicSubmodelComponent.cpp

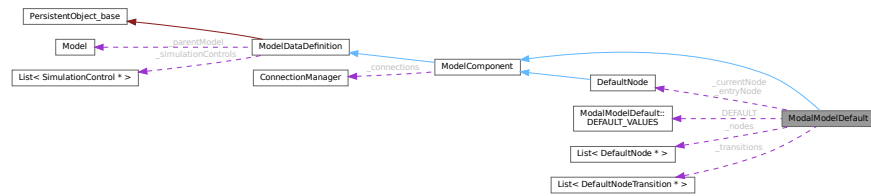
10.306 ModalModelDefault Class Reference

```
#include <plugins/components/ModalModel/ModalModelDefault.h>
```

Inheritance diagram for ModalModelDefault:



Collaboration diagram for ModalModelDefault:



Classes

- struct [DEFAULT_VALUES](#)

Attributes that should be loaded or saved with this component (Persistent Fields)

Public Member Functions

- [ModalModelDefault](#) ([Model](#) *model, std::string name="")
constructors
- virtual void **setMaxTransitionsPerDispatch** (unsigned int maxTransitionsPerDispatch)
- virtual unsigned int **getMaxTransitionsPerDispatch** () const
- virtual [List< DefaultNode * >](#) * **getNodes** () const
- virtual [List< DefaultNodeTransition * >](#) * **getTransitions** () const
- [DefaultNode](#) * **getEntryNode** ()
- void **setEntryNode** ([DefaultNode](#) *const entry_node)
- [DefaultNode](#) * **getCurrentNode** ()
- std::string **getTimeDelayExpressionPerDispatch** ()
- void **setTimeDelayExpressionPerDispatch** (const std::string time_delay_expression_per_dispatch)
- [Util::TimeUnit](#) **getTimeDelayPerDispatchTimeUnit** ()
- void **setTimeDelayPerDispatchTimeUnit** (const [Util::TimeUnit](#) time_delay_per_dispatch_time_unit)
- virtual void **addNode** ([DefaultNode](#) *node)
new public user methods for this component
- virtual void **removeNode** ([DefaultNode](#) *node)
- virtual void **addTransition** ([DefaultNodeTransition](#) *transition)
- virtual void **removeTransition** ([DefaultNodeTransition](#) *transition)
- virtual void **addOutputExpressionReference** ([ModelDataDefinition](#) *expressionReference)
- virtual void **removeOutputExpressionReference** ([DefaultNodeTransition](#) *expressionReference)
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)

- Connects this component to another component.*

 - bool [hasBreakpointAt](#) ()
 - Indicates whether a simulation breakpoint is set on this component.*
 - void [setDescription](#) (std::string _description)
 - Updates the free-form description shown by the GUI.*
 - std::string [getDescription](#) () const
 - Returns the free-form description associated with this component.*
 - void [remove](#) ([ModelComponent](#) *comp)
 - Removes a connected component from this component.*

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~[ModelDataDefinition](#) ()
- Releases resources owned by this data definition.*
- Util::identification [getId](#) () const
- Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & [getName](#) () const
- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
- Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const
- Returns the runtime class name used by the plugin system.*
- [Model](#) * [getParentModel](#) () const
- Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const
- Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)
- Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
- Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
- Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
- Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const
- Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))
- Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const
- Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int _modelLevel)
- Sets the model nesting level for this definition.*
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const

- Returns the simulation controls owned by this data definition.*
- `TraceManager::Level` [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- `void` [setTraceLevelSpecific](#) (`TraceManager::Level` level)
Updates the trace level used by this definition.
- `void` [defineTraceLevelSpecific](#) (`TraceManager::Level` traceLevelspecific, `bool` traceLevelSpecific↔ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- `bool` [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- `void` [setTraceLevelSpecificEnabled](#) (`bool` traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- `static` [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static` [ModelComponent](#) * [LoadInstance](#) (`Model` *model, [PersistenceRecord](#) *fields)
- `static` [ModelDataDefinition](#) * [NewInstance](#) (`Model` *model, `std::string` name="")

Static Public Member Functions inherited from [ModelComponent](#)

- `static` [ModelComponent](#) * [LoadInstance](#) (`Model` *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static` `bool` [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- `static` `void` [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- `static` `void` [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- `static` `void` [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- `static` [ModelDataDefinition](#) * [LoadInstance](#) (`Model` *model, [PersistenceRecord](#) *fields, `bool` insertIntoModel)
Loads a model data definition instance from serialized fields.
- `static` [ModelDataDefinition](#) * [NewInstance](#) (`Model` *model, `std::string` name="")
Creates a new instance of a data definition using the plugin registry.
- `static` `void` [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- `static` `bool` [Check](#) ([ModelDataDefinition](#) *modeldatum, `std::string` &errorMessage)
Validates a data definition instance.
- `static` `void` [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- `static` `void` [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- `static` `bool` [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, `std::string` &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual bool `_check` (std::string &errorMessage) override
virtual protected methods that could be overridden by derived classes, if needed
- virtual void `_initBetweenReplications` () override
- virtual void `_createAttachedAttributes` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()

- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Protected Attributes

- const struct [ModalModelDefault::DEFAULT_VALUES](#) **DEFAULT**
- unsigned int **_maxTransitionsPerDispatch** = DEFAULT.maxTransitionsPerDispatch
- std::string **_timeDelayExpressionPerDispatch** = DEFAULT.timeDelayExpressionPerDispatch
- [Util::TimeUnit](#) **_timeDelayPerDispatchTimeUnit** = DEFAULT.timeDelayPerDispatchTimeUnit
- [DefaultNode](#) * **_currentNode** = nullptr
- [DefaultNode](#) * **_entryNode** = nullptr
- [List](#)< [DefaultNode](#) * > * **_nodes** = new [List](#)<[DefaultNode](#)*>()
internal COMPONENTS (since it's a modal model / network) (Composition)
- [List](#)< [DefaultNodeTransition](#) * > * **_transitions** = new [List](#)<[DefaultNodeTransition](#)*>()

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- [Util::identification](#) **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.306.1 Detailed Description

This component ...

10.306.2 Constructor & Destructor Documentation

10.306.2.1 ModalModelDefault()

```
ModalModelDefault::ModalModelDefault (
    Model * model,
    std::string name = "" )
```

constructors

Externalize function GetPluginInformation to be accessible through dynamic linked library.

10.306.3 Member Function Documentation

10.306.3.1 _check()

```
bool ModalModelDefault::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by ModelChecker during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model to run without errors in this component

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [ModalModelFSM](#), and [ModalModelPetriNet](#).

10.306.3.2 _createAttachedAttributes()

```
void ModalModelDefault::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.306.3.3 _initBetweenReplications()

```
void ModalModelDefault::_initBetweenReplications ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelData. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to simulator kernel to reflect the changes

This method is called by [ModelSimulation](#) when initiating the replication. The model should set all value for a new replication (Ex: setting back to 0 any internal counter, clearing lists, etc.

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [ModalModelFSM](#).

10.306.3.4 `_loadInstance()`

```
bool ModalModelDefault::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.306.3.5 `_onDispatchEvent()`

```
void ModalModelDefault::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

This method is only for ModelComponents, not ModelDataElements.

Implements [ModelComponent](#).

10.306.3.6 `_saveInstance()`

```
void ModalModelDefault::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.306.3.7 `show()`

```
std::string ModalModelDefault::show ( ) [override], [virtual]
```

virtual public methods

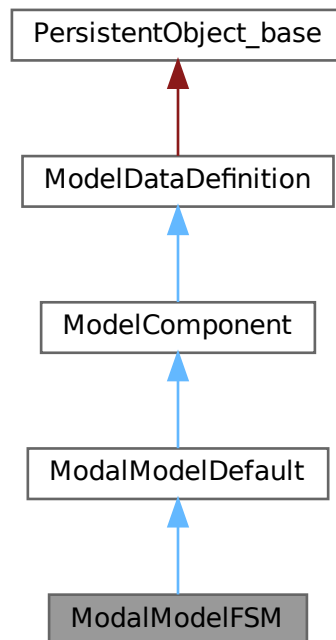
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

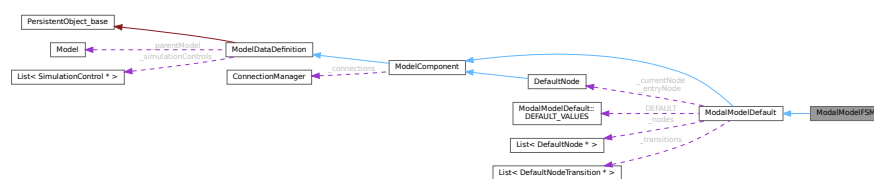
- plugins/components/ModalModel/ModalModelDefault.h
- plugins/components/ModalModel/ModalModelDefault.cpp

10.307 ModalModelFSM Class Reference

Inheritance diagram for ModalModelFSM:



Collaboration diagram for ModalModelFSM:



Public Member Functions

- **ModalModelFSM** (`Model` *model, std::string name="")

Public Member Functions inherited from `ModalModelDefault`

- **ModalModelDefault** (`Model` *model, std::string name="")
constructors
- virtual void **setMaxTransitionsPerDispatch** (unsigned int maxTransitionsPerDispatch)
- virtual unsigned int **getMaxTransitionsPerDispatch** () const

- virtual [List](#)< [DefaultNode](#) * > * **getNodes** () const
- virtual [List](#)< [DefaultNodeTransition](#) * > * **getTransitions** () const
- [DefaultNode](#) * **getEntryNode** ()
- void **setEntryNode** ([DefaultNode](#) *const entry_node)
- [DefaultNode](#) * **getCurrentNode** ()
- std::string **getTimeDelayExpressionPerDispatch** ()
- void **setTimeDelayExpressionPerDispatch** (const std::string time_delay_expression_per_dispatch)
- [Util::TimeUnit](#) **getTimeDelayPerDispatchTimeUnit** ()
- void **setTimeDelayPerDispatchTimeUnit** (const [Util::TimeUnit](#) time_delay_per_dispatch_time_unit)
- virtual void **addNode** ([DefaultNode](#) *node)
new public user methods for this component
- virtual void **removeNode** ([DefaultNode](#) *node)
- virtual void **addTransition** ([DefaultNodeTransition](#) *transition)
- virtual void **removeTransition** ([DefaultNodeTransition](#) *transition)
- virtual void **addOutputExpressionReference** ([ModelDataDefinition](#) *expressionReference)
- virtual void **removeOutputExpressionReference** ([DefaultNodeTransition](#) *expressionReference)
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- [Util::identification](#) **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const

- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModalModelDefault](#)

- static [PluginInformation](#) * **GetPluginInformation** ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_check** (std::string &errorMessage) override
virtual protected methods that could be overridden by derived classes, if needed
- virtual void **_initBetweenReplications** ()

Protected Member Functions inherited from [ModalModelDefault](#)

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModalModelDefault](#)

- const struct [ModalModelDefault::DEFAULT_VALUES](#) **DEFAULT**
- unsigned int **_maxTransitionsPerDispatch** = DEFAULT.maxTransitionsPerDispatch
- std::string **_timeDelayExpressionPerDispatch** = DEFAULT.timeDelayExpressionPerDispatch
- [Util::TimeUnit](#) **_timeDelayPerDispatchTimeUnit** = DEFAULT.timeDelayPerDispatchTimeUnit
- [DefaultNode](#) * **_currentNode** = nullptr
- [DefaultNode](#) * **_entryNode** = nullptr
- [List](#)< [DefaultNode](#) * > * **_nodes** = new [List](#)<[DefaultNode](#)*>()
internal COMPONENTS (since it's a modal model / network) (Composition)
- [List](#)< [DefaultNodeTransition](#) * > * **_transitions** = new [List](#)<[DefaultNodeTransition](#)*>()

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- [Util::identification](#) **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.307.1 Member Function Documentation

10.307.1.1 [_check\(\)](#)

```
bool ModalModelFSM::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by ModelChecker during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model too run without errors in this component

Reimplemented from [ModalModelDefault](#).

10.307.1.2 `_initBetweenReplications()`

```
void ModalModelFSM::_initBetweenReplications ( ) [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

This method is called by [ModelSimulation](#) when initiating the replication. The model should set all value for a new replication (Ex: setting back to 0 any internal counter, clearing lists, etc.

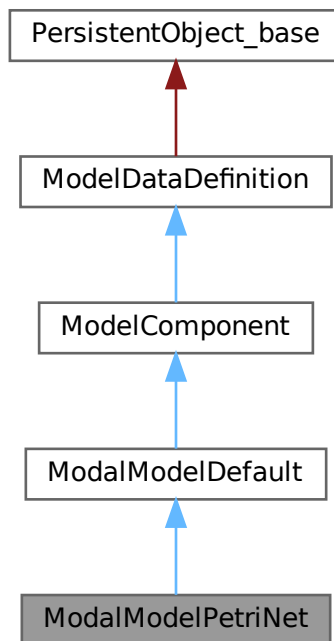
Reimplemented from [ModalModelDefault](#).

The documentation for this class was generated from the following files:

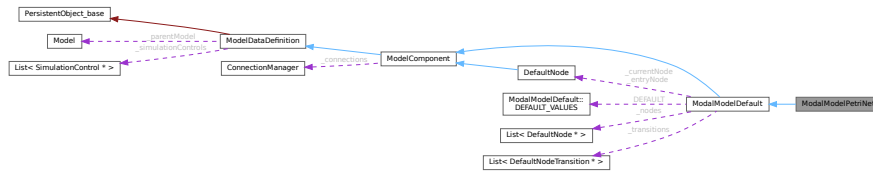
- `plugins/components/ModalModel/ModalModelFSM.h`
- `plugins/components/ModalModel/ModalModelFSM.cpp`

10.308 ModalModelPetriNet Class Reference

Inheritance diagram for ModalModelPetriNet:



Collaboration diagram for ModalModelPetriNet:



Public Member Functions

- **ModalModelPetriNet** ([Model](#) *model, std::string name="")

Public Member Functions inherited from [ModalModelDefault](#)

- [ModalModelDefault](#) ([Model](#) *model, std::string name="")
constructors
- virtual void **setMaxTransitionsPerDispatch** (unsigned int maxTransitionsPerDispatch)
- virtual unsigned int **getMaxTransitionsPerDispatch** () const
- virtual [List](#)< [DefaultNode](#) * > * **getNodes** () const
- virtual [List](#)< [DefaultNodeTransition](#) * > * **getTransitions** () const
- [DefaultNode](#) * **getEntryNode** ()
- void **setEntryNode** ([DefaultNode](#) *const entry_node)
- [DefaultNode](#) * **getCurrentNode** ()
- std::string **getTimeDelayExpressionPerDispatch** ()
- void **setTimeDelayExpressionPerDispatch** (const std::string time_delay_expression_per_dispatch)
- [Util::TimeUnit](#) **getTimeDelayPerDispatchTimeUnit** ()
- void **setTimeDelayPerDispatchTimeUnit** (const [Util::TimeUnit](#) time_delay_per_dispatch_time_unit)
- virtual void **addNode** ([DefaultNode](#) *node)
new public user methods for this component
- virtual void **removeNode** ([DefaultNode](#) *node)
- virtual void **addTransition** ([DefaultNodeTransition](#) *transition)
- virtual void **removeTransition** ([DefaultNodeTransition](#) *transition)
- virtual void **addOutputExpressionReference** ([ModelDataDefinition](#) *expressionReference)
- virtual void **removeOutputExpressionReference** ([DefaultNodeTransition](#) *expressionReference)
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()

- Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const

- Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModalModelDefault](#)

- static [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_check` (std::string &errorMessage) override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModalModelDefault](#)

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void `_initBetweenReplications` () override
- virtual void `_createAttachedAttributes` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)

- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModalModelDefault](#)

- const struct [ModalModelDefault::DEFAULT_VALUES](#) **DEFAULT**
- unsigned int **_maxTransitionsPerDispatch** = DEFAULT.maxTransitionsPerDispatch
- std::string **_timeDelayExpressionPerDispatch** = DEFAULT.timeDelayExpressionPerDispatch
- [Util::TimeUnit](#) **_timeDelayPerDispatchTimeUnit** = DEFAULT.timeDelayPerDispatchTimeUnit
- [DefaultNode](#) * **_currentNode** = nullptr
- [DefaultNode](#) * **_entryNode** = nullptr
- [List](#)< [DefaultNode](#) * > * **_nodes** = new [List](#)<[DefaultNode](#)*>()
internal COMPONENTS (since it's a modal model / network) (Composition)
- [List](#)< [DefaultNodeTransition](#) * > * **_transitions** = new [List](#)<[DefaultNodeTransition](#)*>()

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- [Util::identification](#) **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.308.1 Member Function Documentation

10.308.1.1 `_check()`

```
bool ModalModelPetriNet::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by ModelChecker during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model to run without errors in this component

Reimplemented from [ModalModelDefault](#).

The documentation for this class was generated from the following files:

- plugins/components/ModalModel/ModalModelPetriNet.h
- plugins/components/ModalModel/ModalModelPetriNet.cpp

10.309 Model Class Reference

```
#include <kernel/simulator/model/Model.h>
```

Public Member Functions

- [Model](#) ([Simulator](#) *simulator, unsigned int level=0)
Creates a model bound to a simulator.
- virtual `~Model` ()
Releases model-owned runtime state.
- bool [save](#) (std::string filename)
Saves the model to a file.
- bool [load](#) (std::string filename)
Loads the model from a file.
- bool [check](#) ()
Validates the model structure and configuration.
- void [clear](#) ()
Clears the current model content.
- std::string [showLanguage](#) ()
Returns the model specification in the source language.
- void [show](#) ()
Displays the model contents to the configured trace output.
- bool [insert](#) ([ModelDataDefinition](#) *elemOrComp)
Inserts a model data definition or component into the model.
- void [remove](#) ([ModelDataDefinition](#) *elemOrComp)
Removes a model data definition or component from the model.
- std::list< [ModelDataDefinition](#) * > [collectDataDefinitionsRemovedWith](#) (const std::list< [ModelDataDefinition](#) * > &roots) const
Computes the data definitions that should be removed together with a set of roots.
- [Entity](#) * [createEntity](#) (std::string name, bool insertIntoModel=true)

- Creates an entity owned by this model.*
- void `removeEntity` (`Entity` *entity)
- Removes an entity owned by this model.*
- void `sendEntityToComponent` (`Entity` *entity, `Connection` *connection, double timeDelay=0.0)
- Schedules an entity transfer through a specific connection.*
- void `sendEntityToComponent` (`Entity` *entity, `ModelComponent` *component, double timeDelay=0.0, unsigned int componentInputPortNumber=0)
- Schedules an entity transfer to a specific component and input port.*
- double `parseExpression` (const std::string expression)
- Parses and evaluates an expression.*
- double `parseExpression` (const std::string expression, bool &success, std::string &errorMessage)
- Parses and evaluates an expression while reporting success or failure.*
- bool `checkExpression` (const std::string expression, const std::string expressionName, std::string &error← Message)
- Checks whether an expression is syntactically valid.*
- void `checkReferencesToDataDefinitions` (std::string expression, std::map< std::string, std::list< std::string > * > *referencedDataDefinitions)
- Collects data-definition references found in an expression.*
- Util::identification `getId` () const
- Returns the unique model identifier.*
- bool `hasChanged` () const
- Indicates whether the model has pending changes.*
- void `setHasChanged` (bool hasChanged)
- Updates the changed flag for the model and its owned persistent objects.*
- `OnEventManager` * `getOnEventManager` () const
- Returns the event manager that coordinates model-level events.*
- `ModelDataManager` * `getDataManager` () const
- Returns the data manager for model data definitions.*
- `ModelComponentManager` * `getComponentManager` () const
- Returns the component manager for model components.*
- `ModelInfo` * `getInfos` () const
- Returns the model metadata object.*
- `Simulator` * `getParentSimulator` () const
- Returns the parent simulator that owns this model.*
- `ModelSimulation` * `getSimulation` () const
- Returns the simulation controller associated with this model.*
- List< `Event` * > * `getFutureEvents` () const
- Returns the future-event queue.*
- List< `SimulationControl` * > * `getControls` () const
- Returns the simulation controls exposed by the model.*
- List< `SimulationResponse` * > * `getResponses` () const
- Returns the simulation responses exposed by the model.*
- void `setTracer` (`TraceManager` *_traceManager)
- Sets the trace manager used by this model.*
- `TraceManager` * `getTracer` () const
- Returns the trace manager used by this model.*
- `Persistence_if` * `getPersistence` () const
- Returns the persistence interface used by this model.*
- unsigned int `getLevel` () const
- Returns the hierarchy level of the model.*

10.309.1 Detailed Description

[Model](#) is probably the most important class of Genesys kernel. It represents a discrete event-driven simulation model. Each model is responsible for controlling its own simulation, ie, for sequentially processing events and collecting statistical results. A model is mainly represented by a collection of components ([ModelComponent](#)), adequately configured and connected, and a collection of under layered modeldatum ([ModelDataDefinition](#)).

10.309.2 Constructor & Destructor Documentation

10.309.2.1 Model()

```
Model::Model (
    Simulator * simulator,
    unsigned int level = 0 )
```

Creates a model bound to a simulator.

Parameters

<i>simulator</i>	Parent simulator that owns the model.
<i>level</i>	Hierarchy level of the model within the simulator.

10.309.3 Member Function Documentation

10.309.3.1 check()

```
bool Model::check ( )
```

Validates the model structure and configuration.

Checks the integrity and consistency of the model, may correct some inconsistencies, and returns whether the model is ready to be simulated.

Returns

`true` when the model is ready to simulate.

10.309.3.2 checkExpression()

```
bool Model::checkExpression (
    const std::string expression,
    const std::string expressionName,
    std::string & errorMessage )
```

Checks whether an expression is syntactically valid.

This method is invoked by ModelComponents and ModelDatas in their private `_check()` methods to validate user-defined expressions.

Parameters

<i>expression</i>	Expression text.
<i>expressionName</i>	Logical name used in diagnostics.
<i>errorMessage</i>	Output error message when validation fails.

Returns

`true` when the expression is valid.

10.309.3.3 checkReferencesToDataDefinitions()

```
void Model::checkReferencesToDataDefinitions (
    std::string expression,
    std::map< std::string, std::list< std::string > * > * referencedDataDefinitions
)
```

Collects data-definition references found in an expression.

Parameters

<i>expression</i>	Expression text.
<i>referencedDataDefinitions</i>	Output map of referenced data definitions by name.

10.309.3.4 collectDataDefinitionsRemovedWith()

```
std::list< ModelDataDefinition * > Model::collectDataDefinitionsRemovedWith (
    const std::list< ModelDataDefinition * > & roots ) const
```

Computes the data definitions that should be removed together with a set of roots.

Parameters

<i>roots</i>	Root objects selected for removal.
--------------	------------------------------------

Returns

Flattened list of dependent data definitions to remove as well.

10.309.3.5 createEntity()

```
Entity * Model::createEntity (
    std::string name,
    bool insertIntoModel = true )
```

Creates an entity owned by this model.

Parameters

<i>name</i>	Entity name.
<i>insertIntoModel</i>	If <code>true</code> , inserts the entity into the model immediately.

Returns

Newly created entity.

10.309.3.6 getComponentManager()

```
ModelComponentManager * Model::getComponentManager ( ) const
```

Returns the component manager for model components.

Returns

Component manager instance.

10.309.3.7 getControls()

```
List< SimulationControl * > * Model::getControls ( ) const
```

Returns the simulation controls exposed by the model.

These are the values that can be externally controlled and usually correspond to input parameters that must be changed for an experimental design.

Returns

[List](#) of writable simulation controls.

10.309.3.8 getDataManager()

```
ModelDataManager * Model::getDataManager ( ) const
```

Returns the data manager for model data definitions.

Provides access to the class that manages the basic elements of the simulation model, such as queues, resources, variables and similar model data definitions.

Returns

Data manager instance.

10.309.3.9 getFutureEvents()

```
List< Event * > * Model::getFutureEvents ( ) const
```

Returns the future-event queue.

The future events list is chronologically sorted. Events are scheduled by components while processing other events, and a replication evolves by sequentially processing the first event in this list. The list is initialized with the events first described by source components.

Returns

Chronologically ordered list of scheduled events.

10.309.3.10 getId()

```
Util::identification Model::getId ( ) const
```

Returns the unique model identifier.

Returns

[Model](#) identifier.

10.309.3.11 getInfos()

```
ModelInfo * Model::getInfos ( ) const
```

Returns the model metadata object.

Returns

[Model](#) info instance.

10.309.3.12 getLevel()

```
unsigned int Model::getLevel ( ) const
```

Returns the hierarchy level of the model.

Provides access to the class that performs the trace of simulation and replications.

Returns

[Model](#) level.

10.309.3.13 `getOnEventManager()`

```
OnEventManager * Model::getOnEventManager ( ) const
```

Returns the event manager that coordinates model-level events.

Provides access to the class that manages events generated by the model, such as the beginning of a new simulation or replication and the processing of an event.

Returns

On-event manager instance.

10.309.3.14 `getParentSimulator()`

```
Simulator * Model::getParentSimulator ( ) const
```

Returns the parent simulator that owns this model.

Returns

Parent simulator.

10.309.3.15 `getPersistence()`

```
Persistence_if * Model::getPersistence ( ) const
```

Returns the persistence interface used by this model.

Returns

Persistence interface instance.

10.309.3.16 `getResponses()`

```
List< SimulationResponse * > * Model::getResponses ( ) const
```

Returns the simulation responses exposed by the model.

These are the exits or simulation results that can be read externally and usually correspond to statistics gathered for an experimental design.

Returns

List of readable simulation responses.

10.309.3.17 getSimulation()

```
ModelSimulation * Model::getSimulation ( ) const
```

Returns the simulation controller associated with this model.

Provides access to the class that manages the model simulation.

Returns

Simulation controller instance.

10.309.3.18 getTracer()

```
TraceManager * Model::getTracer ( ) const
```

Returns the trace manager used by this model.

Returns

Trace manager instance.

10.309.3.19 hasChanged()

```
bool Model::hasChanged ( ) const
```

Indicates whether the model has pending changes.

Returns

`true` when the model or one of its owned objects changed.

10.309.3.20 insert()

```
bool Model::insert (
    ModelDataDefinition * elemOrComp )
```

Inserts a model data definition or component into the model.

This is a generic entry point for either `ComponentManager->insert()` or `ModelDataManager->insert()`.

Parameters

<i>elemOrComp</i>	Object to insert.
-------------------	-------------------

Returns

`true` when the object was accepted.

10.309.3.21 load()

```
bool Model::load (
    std::string filename )
```

Loads the model from a file.

Parameters

<i>filename</i>	Input filename.
-----------------	-----------------

Returns

`true` when loading succeeds.

10.309.3.22 parseExpression() [1/2]

```
double Model::parseExpression (
    const std::string expression )
```

Parses and evaluates an expression.

The parser always returns a double, even when the expression has syntax errors, in which case the result is 0.

Parameters

<i>expression</i>	Expression text.
-------------------	------------------

Returns

Numeric result of the expression, or 0 on syntax failure.

10.309.3.23 parseExpression() [2/2]

```
double Model::parseExpression (
    const std::string expression,
    bool & success,
    std::string & errorMessage )
```

Parses and evaluates an expression while reporting success or failure.

The parser always returns a double, even when the expression has syntax errors, in which case the result is 0. This overload also reports whether an error occurred.

Parameters

<i>expression</i>	Expression text.
<i>success</i>	Output flag that receives the parsing result.
<i>errorMessage</i>	Output error message when parsing fails.

Returns

Numeric result of the expression, or 0 on syntax failure.

10.309.3.24 remove()

```
void Model::remove (
    ModelDataDefinition * elemOrComp )
```

Removes a model data definition or component from the model.

This is a generic entry point for either `ComponentManager->remove\(\)` or `ModelDataManager->remove\(\)`.

Parameters

<i>elemOrComp</i>	Object to remove.
-------------------	-------------------

10.309.3.25 removeEntity()

```
void Model::removeEntity (
    Entity * entity )
```

Removes an entity owned by this model.

Parameters

<i>entity</i>	Entity to remove.
---------------	-----------------------------------

10.309.3.26 save()

```
bool Model::save (
    std::string filename )
```

Saves the model to a file.

Parameters

<i>filename</i>	Output filename.
-----------------	------------------

Returns

`true` when saving succeeds.

10.309.3.27 sendEntityToComponent() [1/2]

```
void Model::sendEntityToComponent (
    Entity * entity,
    Connection * connection,
    double timeDelay = 0.0 )
```

Schedules an entity transfer through a specific connection.

Used by components to send entities to the next connected component, or by the model itself while processing an event.

Parameters

<i>entity</i>	Entity to transfer.
<i>connection</i>	Destination connection.
<i>timeDelay</i>	Delay applied before the entity reaches the destination.

10.309.3.28 sendEntityToComponent() [2/2]

```
void Model::sendEntityToComponent (
    Entity * entity,
    ModelComponent * component,
    double timeDelay = 0.0,
    unsigned int componentinputPortNumber = 0 )
```

Schedules an entity transfer to a specific component and input port.

Used by components to send entities to the next connected component, or by the model itself while processing an event.

Parameters

<i>entity</i>	Entity to transfer.
<i>component</i>	Destination component.
<i>timeDelay</i>	Delay applied before the entity reaches the destination.
<i>componentinputPortNumber</i>	Destination input port number.

10.309.3.29 setHasChanged()

```
void Model::setHasChanged (
    bool hasChanged )
```

Updates the changed flag for the model and its owned persistent objects.

Parameters

<i>hasChanged</i>	New changed-state value.
-------------------	--------------------------

10.309.3.30 setTracer()

```
void Model::setTracer (
    TraceManager * _traceManager )
```

Sets the trace manager used by this model.

Parameters

<i>_traceManager</i>	Trace manager instance.
----------------------	-------------------------

10.309.3.31 showLanguage()

```
std::string Model::showLanguage ( )
```

Returns the model specification in the source language.

Returns

Textual model specification.

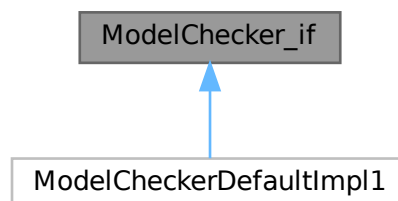
The documentation for this class was generated from the following files:

- kernel/simulator/model/Model.h
- kernel/simulator/model/Model.cpp

10.310 ModelChecker_if Class Reference

```
#include <kernel/simulator/model/ModelChecker_if.h>
```

Inheritance diagram for ModelChecker_if:



Public Member Functions

- virtual bool [checkAll](#) ()=0
checkAll
- virtual bool [checkConnected](#) ()=0
checkConnected
- virtual bool [checkSymbols](#) ()=0
checkSymbols
- virtual bool [checkActivationCode](#) ()=0
checkActivationCode
- virtual bool [checkLimits](#) ()=0
checkLimits

10.310.1 Detailed Description

The ModelChecker is responsible for verifying the model consistency, fixing inconsistencies whenever possible

10.310.2 Member Function Documentation

10.310.2.1 [checkActivationCode](#)()

```
virtual bool ModelChecker_if::checkActivationCode ( ) [pure virtual]
```

[checkActivationCode](#)

Returns

Checks if the installed version has acquired a valid activation code for commercial use

Implemented in [ModelCheckerDefaultImpl1](#).

10.310.2.2 [checkAll](#)()

```
virtual bool ModelChecker_if::checkAll ( ) [pure virtual]
```

[checkAll](#)

Returns

Invokes all other checks and returns true only if all of them returned true

Implemented in [ModelCheckerDefaultImpl1](#).

10.310.2.3 checkConnected()

```
virtual bool ModelChecker_if::checkConnected ( ) [pure virtual]
```

checkConnected

Returns

Checks if components are consistently connected to other to form a valid process-oriented model, describing how entities proceed to the flow

Implemented in [ModelCheckerDefaultImpl1](#).

10.310.2.4 checkLimits()

```
virtual bool ModelChecker_if::checkLimits ( ) [pure virtual]
```

checkLimits

Returns

Checks if the model (its components and elements) fits the limits based on the activation code

Implemented in [ModelCheckerDefaultImpl1](#).

10.310.2.5 checkSymbols()

```
virtual bool ModelChecker_if::checkSymbols ( ) [pure virtual]
```

checkSymbols

Returns

Checks if user-defined strings for symbols required by components, usually expressions or functions, are valid or references existing and valid elements.

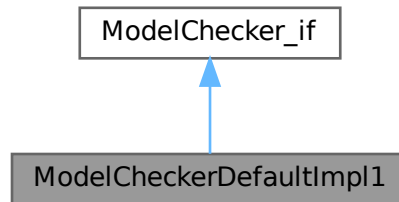
Implemented in [ModelCheckerDefaultImpl1](#).

The documentation for this class was generated from the following file:

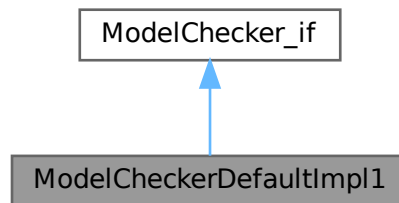
- kernel/simulator/model/ModelChecker_if.h

10.311 ModelCheckerDefaultImpl1 Class Reference

Inheritance diagram for ModelCheckerDefaultImpl1:



Collaboration diagram for ModelCheckerDefaultImpl1:



Public Member Functions

- **ModelCheckerDefaultImpl1** ([Model](#) *model)
- virtual bool [checkAll](#) () override
checkAll
- virtual bool [checkConnected](#) () override
checkConnected
- virtual bool [checkSymbols](#) () override
checkSymbols
- virtual bool [checkActivationCode](#) () override
checkActivationCode
- virtual bool [checkLimits](#) () override
checkLimits

10.311.1 Member Function Documentation

10.311.1.1 checkActivationCode()

```
bool ModelCheckerDefaultImpl1::checkActivationCode ( ) [override], [virtual]
```

checkActivationCode

Returns

Checks if the installed version has acquired a valid activation code for commercial use

Implements [ModelChecker_if](#).

10.311.1.2 checkAll()

```
bool ModelCheckerDefaultImpl1::checkAll ( ) [override], [virtual]
```

checkAll

Returns

Invokes all other checks and returns true only if all of them returned true

Implements [ModelChecker_if](#).

10.311.1.3 checkConnected()

```
bool ModelCheckerDefaultImpl1::checkConnected ( ) [override], [virtual]
```

checkConnected

Returns

Checks if components are consistently connected to other to form a valid process-oriented model, describing how entities proceed to the flow

Implements [ModelChecker_if](#).

10.311.1.4 checkLimits()

```
bool ModelCheckerDefaultImpl1::checkLimits ( ) [override], [virtual]
```

checkLimits

Returns

Checks if the model (its components and elements) fits the limits based on the activation code

Implements [ModelChecker_if](#).

10.311.1.5 checkSymbols()

```
bool ModelCheckerDefaultImpl1::checkSymbols ( ) [override], [virtual]
```

checkSymbols

Returns

Checks if user-defined strings for symbols required by components, usually expressions or functions, are valid or references existing and valid elements.

Implements [ModelChecker_if](#).

The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelCheckerDefaultImpl1.h
- kernel/simulator/model/ModelCheckerDefaultImpl1.cpp

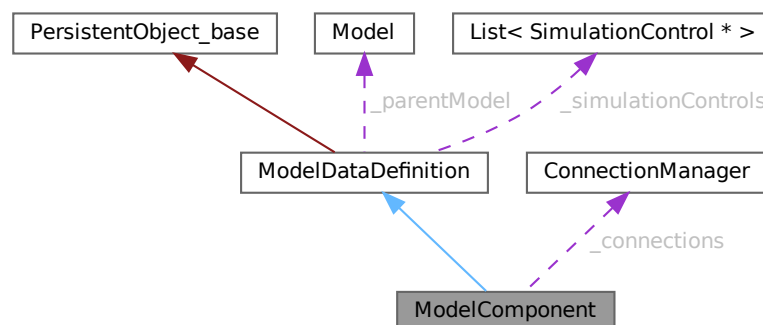
10.312 ModelComponent Class Reference

```
#include <kernel/simulator/model/ModelComponent.h>
```

Inherits [ModelDataDefinition](#).

Inherited by [AIAssistant](#), [Access](#), [Assign](#), [BacteriaColony](#), [Batch](#), [BioRunnerCommand](#), [BioSimulate](#), [BioStateProjectionComponent](#), [BioSteadyState](#), [Buffer](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CellularAutomataComp](#), [Clone](#), [CompartmentExchangeComponent](#), [ContinuousSystemComponent](#), [CppForG](#), [Decide](#), [DefaultNode](#), [Delay](#), [DiffEquations](#), [DiffusionSimulate](#), [DropOff](#), [DummyComponent](#), [Enter](#), [EukaryoticCellCycleComponent](#), [Exit](#), [FtsZPolymerizationComponent](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [LSODE](#), [Leave](#), [MarkovChain](#), [Match](#), [MetabolicFluxBalance](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [ModalModelDefault](#), [PathwayStressResponseComponent](#), [PickStation](#), [Process](#), [PythonForG](#), [R Simulator](#), [Record](#), [Release](#), [Remove](#), [ResourceAllocationComponent](#), [Route](#), [SPICECircuit](#), [SPICENode](#), [Search](#), [Seize](#), [Separate](#), [Signal](#), [SinkModelComponent](#), [SourceModelComponent](#), [Start](#), [StochasticReactionComponent](#), [StochasticTranscription](#), [StochasticTranslation](#), [Stop](#), [Store](#), [Submodel](#), [Unstore](#), [Wait](#), and [Write](#).

Collaboration diagram for ModelComponent:



Public Member Functions

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.
- virtual std::string [show](#) () override
virtual public methods

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.

- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static bool Check (ModelComponent *component)`
Validates a component instance.
- `static void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- `static void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- `static void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber)=0
Handles a dispatched event for this component type.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual bool `_check` (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()

- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Protected Attributes

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.312.1 Detailed Description

A component of the model is a block that represents a specific behavior to be simulated. The behavior is triggered when an entity arrives at the component, which corresponds to the occurrence of an event. A simulation model corresponds to a set of interconnected components to form the process by which the entity is submitted.

Parameters

<i>model</i>	The model this component belongs to
--------------	-------------------------------------

10.312.2 Constructor & Destructor Documentation

10.312.2.1 [ModelComponent](#)()

```
ModelComponent::ModelComponent (
    Model * model,
```

```
std::string componentTypename,
std::string name = "" )
```

constructors

Creates a model component and registers it in the component manager.

Parameters

<i>model</i>	Parent model.
<i>componentTypename</i>	Runtime type name for the concrete component.
<i>name</i>	Initial component name.

10.312.3 Member Function Documentation

10.312.3.1 `_loadInstance()`

```
bool ModelComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [SinkModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), [XNOR](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [FSMState](#), [ModalModelDefault](#), [PetriPlace](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), and [StochasticTranslation](#).

10.312.3.2 `_onDispatchEvent()`

```
virtual void ModelComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [protected], [pure virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implemented in [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [ModalModelDefault](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), and [StochasticTranslation](#).

10.312.3.3 `_saveInstance()`

```
void ModelComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [SinkModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), [XNOR](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [FSMState](#), [ModalModelDefault](#), [PetriPlace](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), and [StochasticTranslation](#).

10.312.3.4 `Check()`

```
bool ModelComponent::Check (
    ModelComponent * component ) [static]
```

Validates a component instance.

Parameters

<i>component</i>	Component to validate.
------------------	------------------------

Returns

`true` when the component is valid.

10.312.3.5 connectTo()

```
void ModelComponent::connectTo (
    ModelComponent * component,
    unsigned int inputPortNumber = 0 )
```

Connects this component to another component.

Parameters

<i>component</i>	Target component.
<i>inputPortNumber</i>	Input port number of the target component.

10.312.3.6 CreateInternalData()

```
void ModelComponent::CreateInternalData (
    ModelComponent * component ) [static]
```

Creates internal data required by a component instance.

Parameters

<i>component</i>	Component whose related data should be created.
------------------	---

10.312.3.7 DispatchEvent()

```
void ModelComponent::DispatchEvent (
    Event * event ) [static]
```

Dispatches a simulation event to the target component.

This method triggers the simulation behavior of the component. It is invoked when an event corresponding to this component is taken from the future events list or when an entity arrives through a connection.

Parameters

<i>event</i>	Event to process.
--------------	-----------------------------------

10.312.3.8 getConnectionManager()

```
ConnectionManager * ModelComponent::getConnectionManager ( ) const
```

new public user methods for this component

Returns the connection manager owned by this component.

This manager stores the components directly connected to the output. Usually a component has a single output, but it may have none (such as [Dispose](#)) or more than one (such as [Decide](#)). Along with each connected component, the manager stores the input port number where the entity will be sent. Usually components have a single input, but they may have none (such as [Create](#)) or more than one (such as [Match](#)).

Returns

[Connection](#) manager that stores outgoing connections.

10.312.3.9 getDescription()

```
std::string ModelComponent::getDescription ( ) const
```

Returns the free-form description associated with this component.

Returns

Description text.

10.312.3.10 hasBreakpointAt()

```
bool ModelComponent::hasBreakpointAt ( )
```

Indicates whether a simulation breakpoint is set on this component.

Returns

`true` when the simulation breakpoint list contains this component.

10.312.3.11 LoadInstance()

```
static ModelComponent * ModelComponent::LoadInstance (
    Model * model,
    PersistenceRecord * fields ) [static]
```

static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component

Loads a component instance from serialized fields.

Parameters

<i>model</i>	Parent model that will own the component.
<i>fields</i>	Serialized fields for the component.

Returns

Newly created component instance.

10.312.3.12 remove()

```
void ModelComponent::remove (
    ModelComponent * comp )
```

Removes a connected component from this component.

Parameters

<i>comp</i>	Component to remove from the outgoing connection set.
-------------	---

10.312.3.13 SaveInstance()

```
void ModelComponent::SaveInstance (
    PersistenceRecord * fields,
    ModelComponent * component ) [static]
```

Serializes a component instance into a persistence record.

Parameters

<i>fields</i>	Output persistence record.
<i>component</i>	Component to serialize.

10.312.3.14 setDescription()

```
void ModelComponent::setDescription (
    std::string _description )
```

Updates the free-form description shown by the GUI.

Parameters

<i>_description</i>	New description text.
---------------------	-----------------------

10.312.3.15 show()

```
std::string ModelComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [ModalModelDefault](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), and [StochasticTranslation](#).

10.312.4 Member Data Documentation**10.312.4.1 _connections**

```
ConnectionManager* ModelComponent::_connections = new ConnectionManager() [protected]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by [ModelChecker](#) during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model to run without errors in this component

This method returns all changes in the parser that are needed by plugins of this [ModelData](#)s. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to the simulator kernel to reflect the changes

This method is called by [ModelSimulation](#) when initiating the replication. The model should set all value for a new replication (Ex: setting back to 0 any internal counter, clearing lists, etc).

Legacy compatibility hook. Prefer the granular create* methods and the composed association manager.

This method is not used yet. It should be useful for new UIs

The documentation for this class was generated from the following files:

- [kernel/simulator/model/ModelComponent.h](#)
- [kernel/simulator/model/ModelComponent.cpp](#)

10.313 ModelComponentManager Class Reference

Owns and organizes the set of [ModelComponent](#) objects in a model.

```
#include <kernel/simulator/model/ModelComponentManager.h>
```

Public Member Functions

- **ModelComponentManager** ([Model](#) *model)
Creates a component manager attached to a model.
- bool [insert](#) ([ModelComponent](#) *comp)
insert
- void [remove](#) ([ModelComponent](#) *comp)
remove
- [ModelComponent](#) * [find](#) (std::string name)
find
- [ModelComponent](#) * [find](#) (Util::identification id)
find
- void [clear](#) ()
clear
- unsigned int [getNumberOfComponents](#) ()
getNumberOfComponents
- std::list< [ModelComponent](#) * >::iterator [begin](#) ()
begin
- std::list< [ModelComponent](#) * >::iterator [end](#) ()
end
- [ModelComponent](#) * [front](#) ()
front
- [ModelComponent](#) * [next](#) ()
next
- bool [hasChanged](#) () const
hasChanged
- void [setHasChanged](#) (bool _hasChanged)
setHasChanged
- std::list< [SourceModelComponent](#) * > * [getSourceComponents](#) ()
getSourceComponents
- std::list< [ModelComponent](#) * > * [getTransferInComponents](#) ()
getTransferInComponents
- std::list< [ModelComponent](#) * > * [getAllComponents](#) () const
getAllComponents

10.313.1 Detailed Description

Owns and organizes the set of [ModelComponent](#) objects in a model.

Provides insertion/removal, lookup and iteration helpers while tracking structural changes relevant to model validation/persistence workflows.

10.313.2 Member Function Documentation**10.313.2.1 begin()**

```
std::list< ModelComponent * >::iterator ModelComponentManager::begin ( )
```

[begin](#)

Returns

Iterator to the first component.

10.313.2.2 clear()

```
void ModelComponentManager::clear ( )
```

clear

Removes all managed components from the model.

10.313.2.3 end()

```
std::list< ModelComponent * >::iterator ModelComponentManager::end ( )
```

end

Returns

Iterator past the last component.

10.313.2.4 find() [1/2]

```
ModelComponent * ModelComponentManager::find (
    std::string name )
```

find

Parameters

<i>name</i>	
-------------	--

Returns

Pointer to the component, or `nullptr` when not found.

10.313.2.5 find() [2/2]

```
ModelComponent * ModelComponentManager::find (
    Util::identification id )
```

find

Parameters

<i>id</i>	
-----------	--

Returns

Pointer to the component, or `nullptr` when not found.

10.313.2.6 front()

```
ModelComponent * ModelComponentManager::front ( )
```

front

Returns

First component in iteration order.

10.313.2.7 getAllComponents()

```
std::list< ModelComponent * > * ModelComponentManager::getAllComponents ( ) const
```

getAllComponents

Returns

10.313.2.8 getNumberOfComponents()

```
unsigned int ModelComponentManager::getNumberOfComponents ( )
```

getNumberOfComponents

Returns

Number of managed components.

10.313.2.9 getSourceComponents()

```
std::list< SourceModelComponent * > * ModelComponentManager::getSourceComponents ( )
```

getSourceComponents

Returns

10.313.2.10 getTransferInComponents()

```
std::list< ModelComponent * > * ModelComponentManager::getTransferInComponents ( )
```

getTransferInComponents

Returns

10.313.2.11 hasChanged()

```
bool ModelComponentManager::hasChanged ( ) const
```

hasChanged

Returns

10.313.2.12 insert()

```
bool ModelComponentManager::insert (
    ModelComponent * comp )
```

insert

Parameters

<i>comp</i>	
-------------	--

Returns

True when insertion succeeds.

10.313.2.13 next()

```
ModelComponent * ModelComponentManager::next ( )
```

next

Returns

Next component in iteration order.

10.313.2.14 remove()

```
void ModelComponentManager::remove (
    ModelComponent * comp )
```

remove

Parameters

<i>comp</i>	
-------------	--

10.313.2.15 setHasChanged()

```
void ModelComponentManager::setHasChanged (
    bool _hasChanged )
```

setHasChanged

Parameters

<i>_hasChanged</i>	
--------------------	--

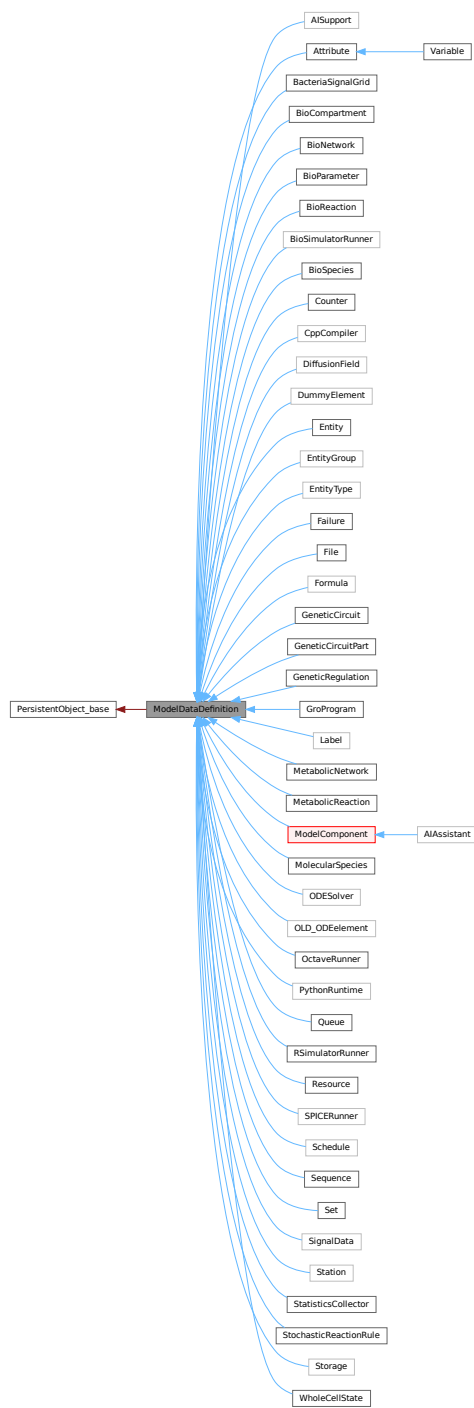
The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelComponentManager.h
- kernel/simulator/model/ModelComponentManager.cpp

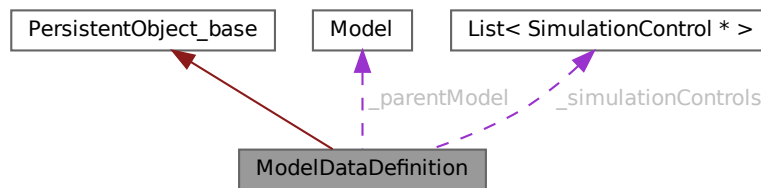
10.314 ModelDataDefinition Class Reference

```
#include <kernel/simulator/model/ModelDataDefinition.h>
```

Inheritance diagram for ModelDataDefinition:



Collaboration diagram for ModelDataDefinition:



Public Member Functions

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.

- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List< SimulationControl * > * getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.
- virtual std::string [show](#) ()
Returns a textual representation of this data definition.

Static Public Member Functions

- static [ModelDataDefinition * LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition * NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()

- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Protected Attributes

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

Friends

- class **ModelDataDefinitionAssociations**

10.314.1 Detailed Description

This class is the basis for any model datum of the model (such as [Queue](#), [Resource](#), [Variable](#), etc.) and also for any component of the model. It has the infrastructure to read and write on file and to verify symbols.

10.314.2 Constructor & Destructor Documentation

10.314.2.1 ModelDataDefinition()

```
ModelDataDefinition::ModelDataDefinition (
    Model * model,
    std::string datadefinitionTypename,
    std::string name = "",
    bool insertIntoModel = true )
```

Creates a model data definition and optionally inserts it into the model.

Parameters

<i>model</i>	Parent model that owns this data definition.
<i>datadefinitionTypename</i>	Runtime type name for the concrete data definition.
<i>name</i>	Initial object name.
<i>insertIntoModel</i>	If <code>true</code> , the new instance is registered in the model immediately.

10.314.3 Member Function Documentation

10.314.3.1 _addSimulationControl()

```
void ModelDataDefinition::_addSimulationControl (
    SimulationControl * control ) [protected], [virtual]
```

Reimplemented in [Buffer](#), [PickStation](#), and [Schedule](#).

10.314.3.2 _check()

```
bool ModelDataDefinition::_check (
    std::string & errorMessage ) [protected], [virtual]
```

could be overridden by derived classes

Reimplemented in [Attribute](#), [Counter](#), [Entity](#), [StatisticsCollector](#), [SinkModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [OLD_ODEElement](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [CppForG](#), [PythonForG](#), [R Simulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [ModalModelDefault](#), [ModalModelFSM](#), [ModalModelPetriNet](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseCom](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), [StochasticTranslation](#), [AISupport](#), [BacteriaSignalGrid](#), [BioNetwork](#), [BioParameter](#), [BioReaction](#), [BioSimulatorRunner](#), [BioSpecies](#), [GeneticCircuit](#), [GeneticCircuitPart](#), [GeneticRegulation](#), [GroProgram](#), [MetabolicNetwork](#), [MetabolicReaction](#), [DiffusionField](#), [ODESolver](#), [Failure](#), [Queue](#), [Resource](#), [Schedule](#), [CppCompiler](#), [OctaveRunner](#), [PythonRuntime](#), [R SimulatorRunner](#), [SPICERunner](#), [EntityGroup](#), [File](#), [Formula](#), [Label](#), [Set](#), [Variable](#), [Sequence](#), [Station](#), [Storage](#), [SignalData](#), [DummyElement](#), [BioCompartment](#), [MolecularSpecies](#), [StochasticReactionRule](#), and [WholeCellState](#).

10.314.3.3 _createAttachedAttributes()

```
void ModelDataDefinition::_createAttachedAttributes ( ) [protected], [virtual]
```

Reimplemented in [Formula](#).

10.314.3.4 _createEditableDataDefinitions()

```
void ModelDataDefinition::_createEditableDataDefinitions ( ) [protected], [virtual]
```

Reimplemented in [DiffEquations](#), [Remove](#), [Search](#), [R Simulator](#), [Assign](#), and [BioNetwork](#).

10.314.3.5 _createInternalAndAttachedData()

```
void ModelDataDefinition::_createInternalAndAttachedData ( ) [protected], [virtual]
```

Legacy compatibility hook. Prefer the granular create* methods and the composed association manager.

10.314.3.6 _createNonEditableDataDefinitions()

```
void ModelDataDefinition::_createNonEditableDataDefinitions ( ) [protected], [virtual]
```

Reimplemented in [SPICECircuit](#), and [CppForG](#).

10.314.3.7 `_getParserChangesInformation()`

```
ParserChangesInformation * ModelDataDefinition::_getParserChangesInformation ( ) [protected],
[virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

Reimplemented in [Buffer](#), [PickStation](#), [Queue](#), [Schedule](#), [File](#), [Set](#), and [Storage](#).

10.314.3.8 `_getSaveDefaultsOption()`

```
bool ModelDataDefinition::_getSaveDefaultsOption ( ) [protected]
```

Returns a list of keys (names) of internal ModelDatas, such as Counters, StatisticsCollectors and others. Children↔ Elements are ModelDatas used by this [ModelDataDefinition](#) thar are needed before model checking

10.314.3.9 `_initBetweenReplications()`

```
void ModelDataDefinition::_initBetweenReplications ( ) [protected], [virtual]
```

Reimplemented in [ModalModelFSM](#), [CppForG](#), [CellularAutomataComp](#), and [ModalModelDefault](#).

10.314.3.10 `_loadInstance()`

```
bool ModelDataDefinition::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Implements [PersistentObject_base](#).

Reimplemented in [Attribute](#), [Counter](#), [Entity](#), [EntityType](#), [StatisticsCollector](#), [ModelComponent](#), [SinkModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [OLD_ODEelement](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), [XNOR](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [FSMState](#), [ModalModelDefault](#), [PetriPlace](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelCompone](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), [StochasticTranslation](#), [AISupport](#), [BacteriaSignalGrid](#), [BioNetwork](#), [BioParameter](#), [BioReaction](#), [BioSimulatorRunner](#), [BioSpecies](#), [GeneticCircuit](#), [GeneticCircuitPart](#), [GeneticRegulation](#), [GroProgram](#), [MetabolicNetwork](#), [MetabolicReaction](#), [DiffusionField](#), [ODESolver](#), [Failure](#), [Queue](#), [Resource](#), [Schedule](#), [CppCompiler](#), [OctaveRunner](#), [PythonRuntime](#), [RSimulatorRunner](#), [SPICERunner](#), [EntityGroup](#), [File](#), [Formula](#), [Label](#), [Set](#), [Variable](#), [Sequence](#), [Station](#), [Storage](#), [SignalData](#), [DummyElement](#), [BioCompartment](#), [MolecularSpecies](#), [StochasticReactionRule](#), and [WholeCellState](#).

10.314.3.11 `_saveInstance()`

```
void ModelDataDefinition::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Implements [PersistentObject_base](#).

Reimplemented in [Attribute](#), [Counter](#), [Entity](#), [EntityType](#), [StatisticsCollector](#), [ModelComponent](#), [SinkModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [OLD_ODEelement](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), [XNOR](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [FSMState](#), [ModalModelDefault](#), [PetriPlace](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), [StochasticTranslation](#), [AISupport](#), [BacteriaSignalGrid](#), [BioNetwork](#), [BioParameter](#), [BioReaction](#), [BioSimulatorRunner](#), [BioSpecies](#), [GeneticCircuit](#), [GeneticCircuitPart](#), [GeneticRegulation](#), [GroProgram](#), [MetabolicNetwork](#), [MetabolicReaction](#), [DiffusionField](#), [ODESolver](#), [Failure](#), [Queue](#), [Resource](#), [Schedule](#), [CppCompiler](#), [OctaveRunner](#), [PythonRuntime](#), [RSimulatorRunner](#), [SPICERunner](#), [EntityGroup](#), [File](#), [Formula](#), [Label](#), [Set](#), [Variable](#), [Sequence](#), [Station](#), [Storage](#), [SignalData](#), [DummyElement](#), [BioCompartment](#), [MolecularSpecies](#), [StochasticReactionRule](#), and [WholeCellState](#).

10.314.3.12 Check()

```
bool ModelDataDefinition::Check (
    ModelDataDefinition * modeldatum,
    std::string & errorMessage ) [static]
```

Validates a data definition instance.

This helper invokes the instance [_check\(\)](#) method to validate the object and any data it creates or references.

Parameters

<i>modeldatum</i>	Instance to validate.
<i>errorMessage</i>	Output error message when validation fails.

Returns

`true` when the instance is valid.

10.314.3.13 CreateInternalData()

```
void ModelDataDefinition::CreateInternalData (
    ModelDataDefinition * modeldatum ) [static]
```

Creates internal and attached data required by this instance.

This helper invokes the protected [_check\(\)](#) path of the instance, which creates internal ModelDataDefinitions and other related data such as attributes or variables.

Parameters

<i>modeldatum</i>	Instance whose associated data should be created.
-------------------	---

10.314.3.14 CreateRelatedDataElements()

```
bool ModelDataDefinition::CreateRelatedDataElements (
    ModelDataDefinition * modeldatum,
    std::string & errorMessage ) [static]
```

Creates related data and validates the instance in a single call.

Materializes related data and runs local checks for the instance in a single step.

Parameters

<i>modeldatum</i>	Instance whose related data should be materialized.
<i>errorMessage</i>	Output error message when validation fails.

Returns

`true` when creation and validation succeed.

10.314.3.15 defineTraceLevelSpecific()

```
void ModelDataDefinition::defineTraceLevelSpecific (
    TraceManager::Level traceLevelspecific,
    bool traceLevelSpecificEnabled = true )
```

Defines the specific trace level and optionally enables it immediately.

Parameters

<i>traceLevelspecific</i>	New trace level.
<i>traceLevelSpecificEnabled</i>	Whether the specific trace level starts enabled.

10.314.3.16 getAttachedData()

```
std::map< std::string, ModelDataDefinition * > * ModelDataDefinition::getAttachedData ( )
const
```

Returns the full map of attached data definitions.

Returns

Map of attached data indexed by name.

10.314.3.17 getClassname()

```
const std::string & ModelDataDefinition::getClassname ( ) const
```

Returns the runtime class name used by the plugin system.

Returns

Concrete class name.

10.314.3.18 getId()

```
Util::identification ModelDataDefinition::getId ( ) const
```

Returns the unique identifier of this instance.

Returns

Instance identifier.

10.314.3.19 getInternalData() [1/2]

```
std::map< std::string, ModelDataDefinition * > * ModelDataDefinition::getInternalData ( )  
const
```

Returns the full map of internal child data definitions.

Returns

Map of internal children indexed by name.

10.314.3.20 getInternalData() [2/2]

```
ModelDataDefinition * ModelDataDefinition::getInternalData (   
    const std::string & name ) const
```

Looks up a named internal child data definition.

Parameters

<i>name</i>	Child name to search for.
-------------	---------------------------

Returns

Matching internal child, or `nullptr` when not found.

10.314.3.21 `getLabel()`

```
const std::string & ModelDataDefinition::getLabel ( ) const
```

Returns the user-visible label of this instance.

Returns

Instance label.

10.314.3.22 `getLevel()`

```
unsigned int ModelDataDefinition::getLevel ( ) const
```

Returns the level in the model hierarchy where this definition lives.

Returns

[Model](#) nesting level.

10.314.3.23 `getName()`

```
const std::string & ModelDataDefinition::getName ( ) const
```

Returns the logical name of this instance.

Returns

Instance name.

10.314.3.24 `getParentModel()`

```
Model * ModelDataDefinition::getParentModel ( ) const
```

Returns the parent model that owns this definition.

Returns

Parent model pointer, or `nullptr` when detached.

10.314.3.25 `getSimulationControls()`

```
List< SimulationControl * > * ModelDataDefinition::getSimulationControls ( ) const
```

Returns the simulation controls owned by this data definition.

Preferred API for writable controls owned by this model data definition.

Returns

Writable control list managed by the definition.

10.314.3.26 getTraceLevelSpecific()

```
TraceManager::Level ModelDataDefinition::getTraceLevelSpecific ( ) const
```

Returns the trace level that applies specifically to this definition.

Returns

Explicit trace level for this instance.

10.314.3.27 hasChanged()

```
bool ModelDataDefinition::hasChanged ( ) const
```

Indicates whether this instance has been modified since the last reset.

Returns

`true` when the object has pending changes.

10.314.3.28 InitBetweenReplications()

```
void ModelDataDefinition::InitBetweenReplications (
    ModelDataDefinition * modeldatum ) [static]
```

Resets simulation-time state between replications.

Parameters

<i>modeldatum</i>	Instance to reset.
-------------------	--------------------

10.314.3.29 isReportStatistics()

```
bool ModelDataDefinition::isReportStatistics ( ) const
```

Indicates whether this data definition contributes report statistics.

Returns

`true` when this definition emits simulation statistics.

10.314.3.30 isTraceLevelSpecificEnabled()

```
bool ModelDataDefinition::isTraceLevelSpecificEnabled ( ) const
```

Indicates whether the specific trace level is enabled.

Returns

`true` when the instance-specific trace level is active.

10.314.3.31 LoadInstance()

```
ModelDataDefinition * ModelDataDefinition::LoadInstance (
    Model * model,
    PersistenceRecord * fields,
    bool insertIntoModel ) [static]
```

Loads a model data definition instance from serialized fields.

This helper receives a map of fields read from a file or other source, creates the instance, and invokes its protected `_loadInstance()` method. The instance can be automatically inserted into the simulation model when requested.

Parameters

<i>model</i>	Parent model that will own the loaded instance.
<i>fields</i>	Serialized fields to deserialize.
<i>insertIntoModel</i>	If <code>true</code> , registers the loaded instance in the model.

Returns

Newly created instance when loading succeeds.

10.314.3.32 NewInstance()

```
ModelDataDefinition * ModelDataDefinition::NewInstance (
    Model * model,
    std::string name = "" ) [static]
```

Creates a new instance of a data definition using the plugin registry.

This helper invokes the constructor and returns a new instance that usually requires a cast to the correct subclass. It is used when plugins are connected through dynamically loaded libraries.

Parameters

<i>model</i>	Parent model that will own the instance.
<i>name</i>	Initial object name.

Returns

Newly created data definition instance.

10.314.3.33 SaveInstance()

```
void ModelDataDefinition::SaveInstance (
    PersistenceRecord * fields,
    ModelDataDefinition * modeldatum ) [static]
```

Serializes a data definition instance into a persistence record.

This helper calls the instance `_saveInstance()` method and returns a field map that can be saved to a file or other destination.

Parameters

<i>fields</i>	Output record that receives the serialized fields.
<i>modeldatum</i>	Instance to serialize.

10.314.3.34 setHasChanged()

```
void ModelDataDefinition::setHasChanged (
    bool hasChanged )
```

Updates the changed flag for this data definition.

Parameters

<i>hasChanged</i>	New changed-state value.
-------------------	--------------------------

10.314.3.35 setLabel()

```
void ModelDataDefinition::setLabel (
    const std::string & label )
```

Updates the user-visible label of this instance.

Parameters

<i>label</i>	New label.
--------------	------------

10.314.3.36 setModelLevel()

```
void ModelDataDefinition::setModelLevel (
    unsigned int _modelLevel )
```

Sets the model nesting level for this definition.

Parameters

<i>_modelLevel</i>	New hierarchy level.
--------------------	----------------------

10.314.3.37 setName()

```
void ModelDataDefinition::setName (
    const std::string & name )
```

Updates the logical name of this instance.

Parameters

<i>name</i>	New name.
-------------	-----------

10.314.3.38 setReportStatistics()

```
void ModelDataDefinition::setReportStatistics (
    bool reportStatistics )
```

Enables or disables report-statistics generation for this definition.

Parameters

<i>reportStatistics</i>	New report-statistics flag.
-------------------------	-----------------------------

10.314.3.39 setTraceLevelSpecific()

```
void ModelDataDefinition::setTraceLevelSpecific (
    TraceManager::Level level )
```

Updates the trace level used by this definition.

Parameters

<i>level</i>	New trace level.
--------------	------------------

10.314.3.40 setTraceLevelSpecificEnabled()

```
void ModelDataDefinition::setTraceLevelSpecificEnabled (
    bool traceLevelSpecificEnabled )
```

Enables or disables the specific trace level.

Parameters

<i>traceLevelSpecificEnabled</i>	New enabled-state value.
----------------------------------	--------------------------

10.314.3.41 show()

```
std::string ModelDataDefinition::show ( ) [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented in [Attribute](#), [Counter](#), [Entity](#), [EntityType](#), [StatisticsCollector](#), [ModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [OLD_ODEelement](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [R Simulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [ModalModelDefault](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), [StochasticTranslation](#), [AISupport](#), [BacteriaSignalGrid](#), [BioNetwork](#), [BioParameter](#), [BioReaction](#), [BioSimulatorRunner](#), [BioSpecies](#), [GeneticCircuit](#), [GeneticCircuitPart](#), [GeneticRegulation](#), [GroProgram](#), [MetabolicNetwork](#), [MetabolicReaction](#), [DiffusionField](#), [ODESolver](#), [Failure](#), [Queue](#), [Resource](#), [Schedule](#), [CppCompiler](#), [OctaveRunner](#), [PythonRuntime](#), [R SimulatorRunner](#), [SPICERunner](#), [EntityGroup](#), [File](#), [Formula](#), [Label](#), [Set](#), [Variable](#), [Sequence](#), [Station](#), [Storage](#), [SignalData](#), [DummyElement](#), [BioCompartment](#), [MolecularSpecies](#), [StochasticReactionRule](#), and [WholeCellState](#).

10.314.3.42 traceSimulation()

```
void ModelDataDefinition::traceSimulation (
    void * thisobject,
    const std::string & text,
    TraceManager::Level level = TraceManager::Level::L8_detailed ) [protected]
```

Traces text to the simulation output.

Used only while a simulation is running, for example when components or data elements need to inform something.

Parameters

<i>thisobject</i>	Caller object used for trace attribution.
<i>text</i>	Trace text to emit.
<i>level</i>	Trace severity level.

The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelDataDefinition.h
- kernel/simulator/model/ModelDataDefinition.cpp

10.315 ModelDataDefinitionAssociations Class Reference

```
#include <kernel/simulator/model/ModelDataDefinitionAssociations.h>
```

Public Member Functions

- **ModelDataDefinitionAssociations** ([ModelDataDefinition](#) *owner)
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
- [ModelDataDefinition](#) * **getInternalData** (const std::string &key) const
- void **clearAll** ()
- void **clearInternalData** ()
- void **clearAttachedData** ()
- void **internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **internalDataRemove** (const std::string &key)
- void **attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **attachedDataRemove** (const std::string &key)
- void **attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
- void **createInternalAndAttachedData** ()
- void **createInternalStatisticReporters** ()
- void **createAttachedAttributes** ()
- void **createNonEditableDataDefinitions** ()
- void **createEditableDataDefinitions** ()
- void **insertStatisticReporter** (const std::string &key, [ModelDataDefinition](#) *data)
- void **removeStatisticReporter** (const std::string &key)
- void **clearStatisticReporters** ()
- void **insertMandatoryAttachedAttribute** (const std::string &key, [ModelDataDefinition](#) *data)
- void **removeMandatoryAttachedAttribute** (const std::string &key)
- void **clearMandatoryAttachedAttributes** ()
- void **insertMandatoryEditableDataDefinition** (const std::string &key, [ModelDataDefinition](#) *data)
- void **removeMandatoryEditableDataDefinition** (const std::string &key)
- void **clearMandatoryEditableDataDefinitions** ()
- void **insertOptionalEditableDataDefinition** (const std::string &key, [ModelDataDefinition](#) *data)
- void **removeOptionalEditableDataDefinition** (const std::string &key)
- void **clearOptionalEditableDataDefinitions** ()
- void **insertMandatoryNonEditableDataDefinition** (const std::string &key, [ModelDataDefinition](#) *data)
- void **removeMandatoryNonEditableDataDefinition** (const std::string &key)
- void **clearMandatoryNonEditableDataDefinitions** ()
- void **insertOptionalNonEditableDataDefinition** (const std::string &key, [ModelDataDefinition](#) *data)
- void **removeOptionalNonEditableDataDefinition** (const std::string &key)
- void **clearOptionalNonEditableDataDefinitions** ()

10.315.1 Detailed Description

Owns the internal/attached registries and the new lifecycle buckets for a [ModelDataDefinition](#).

The class starts as a compatibility layer around the current internal/attached storage so the kernel can move bucket by bucket without breaking direct callers that still inspect the legacy aggregate maps.

The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelDataDefinitionAssociations.h
- kernel/simulator/model/ModelDataDefinitionAssociations.cpp

10.316 ModelDataManager Class Reference

Central registry for [ModelDataDefinition](#) instances owned by a model.

```
#include <kernel/simulator/model/ModelDataManager.h>
```

Public Member Functions

- **ModelDataManager** ([Model](#) *model)
Creates a data manager attached to model.
- bool [insert](#) ([ModelDataDefinition](#) *anElement)
insert
- void [remove](#) ([ModelDataDefinition](#) *anElement)
remove
- bool [insert](#) (std::string datadefinitionTypename, [ModelDataDefinition](#) *anElement)
insert
- void [remove](#) (std::string datadefinitionTypename, [ModelDataDefinition](#) *anElement)
remove
- bool [check](#) (std::string datadefinitionTypename, [ModelDataDefinition](#) *anElement, std::string expression↵
Name, std::string &errorMessage)
check
- bool [check](#) (std::string datadefinitionTypename, std::string elementName, std::string expressionName, bool
mandatory, std::string &errorMessage)
check
- void **clear** ()
clear
- [ModelDataDefinition](#) * [getDataDefinition](#) (std::string datadefinitionTypename, Util::identification id)
getDataDefinition
- [ModelDataDefinition](#) * [getDataDefinition](#) (std::string datadefinitionTypename, std::string name)
getDataDefinition
- unsigned int [getNumberOfDataDefinitions](#) (std::string datadefinitionTypename)
getNumberOfDataDefinitions
- unsigned int [getNumberOfDataDefinitions](#) ()
getNumberOfDataDefinitions
- int [getRankOf](#) (std::string datadefinitionTypename, std::string name)
getRankOf
- std::list< std::string > [getDataDefinitionClassnames](#) () const
getDataDefinitionClassnames
- [List](#)< [ModelDataDefinition](#) * > * **getDataDefinitionList** (std::string datadefinitionTypename) const
- void **show** ()
Prints a textual summary of registered model data definitions.
- [Model](#) * **getParentModel** () const
Returns the model that owns this data manager.
- bool **hasChanged** () const
Indicates whether managed data collections were modified.
- void **setHasChanged** (bool _hasChanged)
Updates the internal changed flag.

10.316.1 Detailed Description

Central registry for [ModelDataDefinition](#) instances owned by a model.

This manager groups model data by typename, supports insertion/removal/lookup operations and offers consistency checks used by components and model validation.

10.316.2 Member Function Documentation

10.316.2.1 `check()` [1/2]

```
bool ModelDataManager::check (
    std::string datadefinitionTypename,
    ModelDataDefinition * anElement,
    std::string expressionName,
    std::string & errorMessage )
```

check

Parameters

<i>datadefinitionTypename</i>	
<i>anElement</i>	
<i>expressionName</i>	
<i>errorMessage</i>	

Returns

10.316.2.2 `check()` [2/2]

```
bool ModelDataManager::check (
    std::string datadefinitionTypename,
    std::string elementName,
    std::string expressionName,
    bool mandatory,
    std::string & errorMessage )
```

check

Parameters

<i>datadefinitionTypename</i>	
<i>elementName</i>	
<i>expressionName</i>	
<i>mandatory</i>	
<i>errorMessage</i>	

Returns

10.316.2.3 getDataDefinition() [1/2]

```
ModelDataDefinition * ModelDataManager::getDataDefinition (
    std::string datadefinitionTypename,
    std::string name )
```

getDataDefinition

Parameters

<i>datadefinitionTypename</i>	
<i>name</i>	

Returns

10.316.2.4 getDataDefinition() [2/2]

```
ModelDataDefinition * ModelDataManager::getDataDefinition (
    std::string datadefinitionTypename,
    Util::identification id )
```

getDataDefinition

Parameters

<i>datadefinitionTypename</i>	
<i>id</i>	

Returns

10.316.2.5 getDataDefinitionClassnames()

```
std::list< std::string > ModelDataManager::getDataDefinitionClassnames ( ) const
```

getDataDefinitionClassnames

Returns

Snapshot list of current data-definition class names returned by value.

10.316.2.6 getNumberOfDataDefinitions() [1/2]

```
unsigned int ModelDataManager::getNumberOfDataDefinitions ( )
```

getNumberOfDataDefinitions

Returns

10.316.2.7 getNumberOfDataDefinitions() [2/2]

```
unsigned int ModelDataManager::getNumberOfDataDefinitions (
    std::string datadefinitionTypename )
```

getNumberOfDataDefinitions

Parameters

<i>datadefinitionTypename</i>	
-------------------------------	--

Returns

10.316.2.8 getRankOf()

```
int ModelDataManager::getRankOf (
    std::string datadefinitionTypename,
    std::string name )
```

getRankOf

Parameters

<i>datadefinitionTypename</i>	
<i>name</i>	

Returns

returns the position (1st position=0) of the modeldatum if found, or negative value if not found

10.316.2.9 insert() [1/2]

```
bool ModelDataManager::insert (
    ModelDataDefinition * anElement )
```

insert

Parameters

<i>anElement</i>	
------------------	--

Returns

10.316.2.10 insert() [2/2]

```
bool ModelDataManager::insert (
    std::string datadefinitionTypename,
    ModelDataDefinition * anElement )
```

insert

Parameters

<i>datadefinitionTypename</i>	
<i>anElement</i>	

Returns

Deprected

10.316.2.11 remove() [1/2]

```
void ModelDataManager::remove (
    ModelDataDefinition * anElement )
```

remove

Parameters

<i>anElement</i>	Deprected
------------------	-----------

10.316.2.12 remove() [2/2]

```
void ModelDataManager::remove (
    std::string datadefinitionTypename,
    ModelDataDefinition * anElement )
```

remove

Parameters

<i>datadefinitionTypename</i>	
<i>anElement</i>	

The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelDataManager.h
- kernel/simulator/model/ModelDataManager.cpp

10.317 ModelEvent Class Reference

```
#include <kernel/simulator/OnEventManager.h>
```

Public Member Functions

- **ModelEvent** ([Model](#) *model)
- const [Model](#) * **getModel** ()

10.317.1 Detailed Description

Stores an event that happened on a specific model

The documentation for this class was generated from the following file:

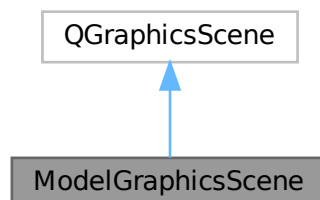
- kernel/simulator/OnEventManager.h

10.318 ModelGraphicsScene Class Reference

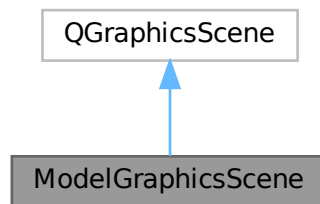
Scene that owns and manipulates the graphical representation of a model.

```
#include <applications/gui/genesys/graphics/ModelGraphicsScene.h>
```

Inheritance diagram for ModelGraphicsScene:



Collaboration diagram for ModelGraphicsScene:



Classes

- struct [GRID](#)

Public Types

- enum **DrawingMode** {
NONE , **LINE** , **TEXT** , **RECTANGLE** ,
ELLIPSE , **POLYGON** , **POLYGON_POINTS** , **POLYGON_FINISHED** ,
COUNTER , **VARIABLE** , **TIMER** , **ATTRIBUTE** ,
ENTITY , **EVENT** , **EXPRESSION** , **PLOT** ,
QUEUE_PLACEHOLDER , **RESOURCE** , **STATION** , **STATISTICS** }

Public Member Functions

- **ModelGraphicsScene** (qreal x, qreal y, qreal width, qreal height, QObject *parent=nullptr)
- **ModelGraphicsScene** (const [ModelGraphicsScene](#) &orig)=delete
- [ModelGraphicsScene](#) & **operator=** (const [ModelGraphicsScene](#) &other)=delete
- [GraphicalModelComponent](#) * **addGraphicalModelComponent** ([Plugin](#) *plugin, [ModelComponent](#) *component, QPointF position, QColor color=Qt::blue, bool notify=false, [GraphicalModelComponent](#) *auto↔ ConnectSource=nullptr)
Creates and inserts one graphical component into the scene.
- [GraphicalConnection](#) * **addGraphicalConnection** ([GraphicalComponentPort](#) *sourcePort, [GraphicalComponentPort](#) *destinationPort, unsigned int portSourceConnection, unsigned int portDestinationConnection, bool notify=false)
Creates and inserts one graphical connection between two ports.
- [GraphicalModelDataDefinition](#) * **addGraphicalModelDataDefinition** ([Plugin](#) *plugin, [ModelDataDefinition](#) *element, QPointF position, QColor color=Qt::blue)
Creates and inserts one graphical data definition into the scene.
- [GraphicalDiagramConnection](#) * **addGraphicalDiagramConnection** (QGraphicsItem *dataDefinition, QGraphicsItem *linkedTo, [GraphicalDiagramConnection::ConnectionType](#) type)
Creates or updates a diagram-level connection between data definitions.
- void **initializeAnimationDrawing** (QGraphicsSceneMouseEvent *mouseEvent)
Starts the interactive drawing workflow for animation-related items.
- void **continueAnimationDrawing** (QGraphicsSceneMouseEvent *mouseEvent)
Continues the interactive drawing workflow.

- void **finishAnimationDrawing** (QGraphicsSceneMouseEvent *mouseEvent)
Finalizes the interactive drawing workflow.
- void **startTextEditing** ()
Opens inline text editing for the current text item.
- void **removeComponent** (GraphicalModelComponent *gmc, bool notify=false)
Removes one component and optionally emits the graphical change notification.
- void **clearConnectionsComponent** (GraphicalModelComponent *gmc)
Clears input and output connections attached to one component.
- void **clearInputConnectionsComponent** (GraphicalModelComponent *graphicalComponent)
Removes all incoming connections from one component.
- void **clearOutputConnectionsComponent** (GraphicalModelComponent *graphicalComponent)
Removes all outgoing connections from one component.
- void **clearPorts** (GraphicalConnection *connection, GraphicalModelComponent *source, GraphicalModelComponent *destination)
Clears the connection ports associated with one connection before removal.
- void **connectComponents** (GraphicalConnection *connection, GraphicalModelComponent *source=nullptr, GraphicalModelComponent *destination=nullptr, bool notify=false)
Connects one prepared connection to its source and destination items.
- bool **connectSource** (GraphicalConnection *connection, GraphicalModelComponent *source=nullptr)
Binds a source component to a connection while checking port constraints.
- bool **connectDestination** (GraphicalConnection *connection, GraphicalModelComponent *destination=nullptr)
Binds a destination component to a connection while checking port constraints.
- void **redoConnections** (GraphicalModelComponent *graphicalComponent, QList< GraphicalConnection * > *inputConnections, QList< GraphicalConnection * > *outputConnections)
Recreates all connection links for one graphical component.
- void **removeComponentInModel** (GraphicalModelComponent *gmc)
Removes one component from the kernel model as part of scene deletion.
- void **insertComponent** (GraphicalModelComponent *gmc, QList< GraphicalConnection * > *inputConnections, QList< GraphicalConnection * > *outputConnections, bool addGMC=true, bool addAllGMC=true, bool notify=false)
Inserts an existing component back into the scene and restores its connections.
- void **removeGraphicalConnection** (GraphicalConnection *graphicalConnection, GraphicalModelComponent *source, GraphicalModelComponent *destination, bool notify=false)
Removes a graphical connection and updates the source/destination items.
- void **removeConnectionInModel** (GraphicalConnection *graphicalConnection, GraphicalModelComponent *source)
Removes the connection from the kernel model while keeping scene state consistent.
- void **addGeometry** (QPointF endPoint, bool moving)
Adds one freeform drawing geometry item to the scene.
- void **addDrawing** (QGraphicsItem *item, bool notify=false)
Adds one drawing item to the scene and optionally notifies listeners.
- bool **addDrawingGeometry** (QGraphicsItem *item)
Attempts to register one geometry drawing item.
- bool **addDrawingAnimation** (QGraphicsItem *item)
Attempts to register one animation drawing item.
- void **removeGraphicalModelDataDefinition** (GraphicalModelDataDefinition *gmdd)
Removes one graphical data definition from the scene.
- void **detachGraphicalModelDataDefinition** (GraphicalModelDataDefinition *gmdd)
Detaches one graphical data definition without deleting the underlying model data.
- void **restoreGraphicalModelDataDefinition** (GraphicalModelDataDefinition *gmdd)
Restores one previously detached graphical data definition.
- void **removeGraphicalDiagramConnection** (GraphicalDiagramConnection *connection)

- Removes one diagram connection from the scene.*
- void **clearGraphicalModelDataDefinitions** ()
Clears all graphical data definitions from the scene.
- void **clearGraphicalDiagramConnections** ()
Clears all diagram connections from the scene.
- void **sanitizeGraphicalDataDefinitionsBookkeeping** ()
Repairs bookkeeping used to track graphical data definitions.
- void **setDiagramLayerState** (bool diagramCreated, bool visible)
Shows or hides the diagram layer and its creation state.
- QList< QGraphicsItem * > **userOperableItems** (const QList< QGraphicsItem * > &items) const
Filters out scene items that the user cannot directly manipulate.
- QList< QGraphicsItem * > **userDeletableItems** (const QList< QGraphicsItem * > &items) const
Filters the selected items down to items that can be deleted.
- void **ensureInitialInternalDataDefinitionGrouping** (GraphicalModelDataDefinition *dataDefinition, GraphicalModelComponent *component)
Optionally groups newly created internal data definitions under their component.
- void **setPersistedGuiRestoreInProgress** (bool restoring)
Marks that persisted GUI state is being restored.
- bool **isPersistedGuiRestoreInProgress** () const
Returns whether persisted GUI state restoration is in progress.
- void **removeDrawing** (QGraphicsItem *item, bool notify=false)
Removes one freeform drawing item from the scene.
- bool **removeDrawingGeometry** (QGraphicsItem *item)
- bool **removeDrawingAnimation** (QGraphicsItem *item)
- void **removeGroup** (QGraphicsItemGroup *group, bool notify=false)
Removes one graphical group item from the scene.
- void **clearGraphicalModelComponents** ()
Clears all graphical component items from the scene.
- void **clearGraphicalModelConnections** ()
Clears all graphical connection items from the scene.
- void **groupComponents** (bool notify=false)
Groups the current selection into a scene group when possible.
- void **groupModelComponents** (QList< GraphicalModelComponent * > *graphicalComponents, QGraphicsItemGroup *group)
Groups the supplied graphical components into the provided group item.
- void **ungroupComponents** (bool notify=false)
Ungroups the current selection when it contains a valid group item.
- void **ungroupModelComponents** (QGraphicsItemGroup *group)
Ungroups one group item and restores its child components.
- void **notifyGraphicalModelChange** (GraphicalModelEvent::EventType eventType, GraphicalModelEvent::EventObjectType eventObjectType, QGraphicsItem *item)
Emits a scene-level change notification used by higher GUI layers.
- QList< GraphicalModelComponent * > **graphicalModelComponentItems** ()
Returns the list of graphical component items currently in the scene.
- GraphicalModelComponent * **findGraphicalModelComponent** (Util::identification id)
Finds one graphical component by kernel identification.
- GraphicalModelDataDefinition * **findGraphicalModelDataDefinition** (ModelDataDefinition *dataDefinition)
Finds one graphical data definition by kernel pointer.
- GraphicalModelComponent * **resolveSourceComponent** (GraphicalConnection *connection) const
Resolves the source component associated with one graphical connection.
- GraphicalModelComponent * **resolveDestinationComponent** (GraphicalConnection *connection) const

- Resolves the destination component associated with one graphical connection.*

 - **GRID * grid ()**
Returns the current grid state owned by the scene.
 - void **showGrid ()**
Shows the grid using the current grid configuration.
 - void **setGridVisible** (bool visible)
Sets grid visibility without toggling the state implicitly.
 - bool **isGridVisible ()** const
Returns whether the grid is currently visible.
 - void **snapItemsToGrid ()**
Snaps movable items to the active grid spacing.
 - void **actualizeDiagramArrows ()**
Recomputes the direction arrows used by diagram connections.
 - void **showDiagrams ()**
Shows all diagram-layer items.
 - void **hideDiagrams ()**
Hides all diagram-layer items.
 - bool **existDiagram ()**
Returns whether diagram-layer items exist.
 - bool **visibleDiagram ()**
Returns whether diagram-layer items are currently visible.
 - void **destroyDiagram ()**
Destroys the diagram layer and its cached items.
 - void **createDiagrams ()**
Creates the diagram layer and its cached items.
 - QUndoStack * **getUndoStack ()**
Returns the undo stack used by the scene.
 - **Simulator * getSimulator ()**
Returns the simulator bound to this scene.
 - void **setUndoStack** (QUndoStack *undo)
Replaces the undo stack used by the scene.
 - void **beginConnection ()**
Starts a new component connection gesture.
 - void **setSimulator** (**Simulator** *simulator)
Sets the simulator dependency used by scene logic.
 - void **setPropertyEditor** (**PropertyEditorGenesys** *propEditor)
Sets the property editor dependency used by scene logic.
 - void **setPropertyList** (std::map< **SimulationControl** *, **DataComponentProperty** * > *propList)
Sets the property list dependency used by scene logic.
 - void **setPropertyEditorUI** (std::map< **SimulationControl** *, **DataComponentEditor** * > *propEditorUI)
Sets the property-editor widget map used by scene logic.
 - void **setComboBox** (std::map< **SimulationControl** *, **ComboBoxEnum** * > *propCombo)
Sets the enum combo-box map used by scene logic.
 - void **setObjectBeingDragged** (QTreeWidgetItem *objectBeingDragged)
Records which tree item is currently being dragged into the scene.
 - void **setRestoringPersistedGuiLayout** (bool restoring)
Marks whether the scene is restoring persisted GUI layout.
 - bool **isRestoringPersistedGuiLayout ()** const
Returns whether the scene is restoring persisted GUI layout.
 - void **setParentWidget** (QWidget *parentWidget)
Sets the parent widget used by transient UI interactions.

- unsigned short **connectingStep** () const
Returns the current step in the connection gesture.
- void **setConnectingStep** (unsigned short [connectingStep](#))
Updates the current step in the connection gesture.
- void **setSnapToGrid** (bool activated)
Enables or disables grid snapping.
- bool **getSnapToGrid** ()
Returns whether grid snapping is enabled.
- void **arrangeModels** (int direction)
Rearranges model items using the specified layout direction.
- void **setDrawingMode** (DrawingMode drawingMode)
Sets the current drawing mode used by mouse gestures.
- void **clearDrawingMode** ()
Clears the current drawing mode.
- void **setGraphicalComponentPort** ([GraphicalComponentPort](#) *in)
Sets the current port used by connection gestures.
- DrawingMode **getDrawingMode** ()
Returns the current drawing mode.
- void **setAction** (QAction *action)
Sets the QAction that tracks the current scene tool.
- QList< [GraphicalModelComponent](#) * > * **getAllComponents** ()
Returns all component items owned by the scene.
- QList< [GraphicalConnection](#) * > * **getAllConnections** ()
Returns all connection items owned by the scene.
- QList< [GraphicalModelDataDefinition](#) * > * **getAllDataDefinitions** ()
Returns all data-definition items owned by the scene.
- QList< [GraphicalDiagramConnection](#) * > * **getAllGraphicalDiagramsConnections** ()
Returns all diagram-connection items owned by the scene.
- QMap< QGraphicsItemGroup *, QList< [GraphicalModelComponent](#) * > > **getListComponentsGroup** ()
Returns the mapping used to restore grouped components.
- void **insertComponentGroup** (QGraphicsItemGroup *group, QList< [GraphicalModelComponent](#) * > componentsGroup)
Stores the component list associated with one graphical group.
- void **insertOldPositionItem** (QGraphicsItem *item, QPointF position)
Stores the previous scene position for one item.
- QPointF **getOldPositionItem** (QGraphicsItem *item) const
Returns the previous scene position stored for one item.
- QList< [AnimationTransition](#) * > * **getAnimationsTransition** ()
Returns the active transition animations tracked by the scene.
- QList< [AnimationCounter](#) * > * **getAnimationsCounter** ()
Returns the active counter animations tracked by the scene.
- QList< [AnimationVariable](#) * > * **getAnimationsVariable** ()
Returns the active variable animations tracked by the scene.
- QList< [AnimationTimer](#) * > * **getAnimationsTimer** ()
Returns the active timer animations tracked by the scene.
- QList< [AnimationPlaceholder](#) * > * **getAnimationsPlaceholder** ()
Returns the placeholder animation items tracked by the scene.
- QMap< Event *, QList< [AnimationTransition](#) * > > * **getAnimationPaused** ()
Returns paused animations indexed by their simulation event.
- bool **checkIgnoreEvent** ()
Returns true when the scene should suppress animation-driven event handling.

- void **clearAnimations** ()
Clears all animation categories tracked by the scene.
- void **clearAnimationsTransition** ()
Clears transition animations tracked by the scene.
- void **clearAnimationsCounter** ()
Clears counter animations tracked by the scene.
- void **clearAnimationsVariable** ()
Clears variable animations tracked by the scene.
- void **clearAnimationsTimer** ()
Clears timer animations tracked by the scene.
- void **clearAnimationsPlaceholder** ()
Clears placeholder animations tracked by the scene.
- void **clearAnimationsQueue** ()
Clears queue animation overlays tracked by the scene.
- void **animatePlot** ()
Refreshes plot animation overlays for the current scene state.
- QList< QString > * **getImagesAnimation** ()
Returns the list of animation image identifiers used by the scene.
- void **drawingCounter** ()
Creates counter overlay geometry.
- void **drawingVariable** ()
Creates variable overlay geometry.
- void **drawingTimer** ()
Creates timer overlay geometry.
- void **drawingAttribute** ()
Creates attribute overlay geometry.
- void **drawingEntity** ()
Creates entity overlay geometry.
- void **drawingEvent** ()
Creates event overlay geometry.
- void **drawingExpression** ()
Creates expression overlay geometry.
- void **drawingPlot** ()
Creates plot overlay geometry.
- void **drawingQueue** ()
Creates queue overlay geometry.
- void **drawingResource** ()
Creates resource overlay geometry.
- void **drawingStation** ()
Creates station overlay geometry.
- void **drawingStatistics** ()
Creates statistics overlay geometry.
- void **clearAnimationsValues** ()
Clears animation-specific cached values.
- void **setCounters** ()
Synchronizes the component counters used by animation overlays.
- void **setVariables** ()
Synchronizes the variable counters used by animation overlays.
- void **insertRestoredDataDefinitions** (bool loaded)
Restores data definitions removed by previous delete/check cycles.
- void **saveDataDefinitions** ()

- Caches data definitions before destructive scene operations.*
- void **requestGraphicalDataDefinitionsSync** ()
Requests an immediate data-definition synchronization pass.
- void **scheduleGraphicalDataDefinitionsSync** ()
Schedules a deferred data-definition synchronization pass.
- bool **isGraphicalDataDefinitionsSyncInProgress** () const
Returns whether a data-definition sync is currently in progress.
- void **setShowStatisticsDataDefinitions** (bool show)
Shows or hides statistics-related data definitions.
- bool **showStatisticsDataDefinitions** () const
Returns whether statistics-related data definitions are visible.
- void **setShowEditableDataDefinitions** (bool show)
Shows or hides editable data definitions.
- bool **showEditableDataDefinitions** () const
Returns whether editable data definitions are visible.
- void **setShowSharedDataDefinitions** (bool show)
Shows or hides shared data definitions.
- bool **showSharedDataDefinitions** () const
Returns whether shared data definitions are visible.
- void **setShowRecursiveDataDefinitions** (bool show)
Shows or hides recursive data-definition expansion.
- bool **showRecursiveDataDefinitions** () const
Returns whether recursive data-definition expansion is enabled.
- void **setModelLevelFilter** (unsigned int modelLevel)
Restricts graphical rebuild/synchronization services to one kernel model level.
- void **clearModelLevelFilter** ()
Clears any level restriction and lets rebuild services render all model levels.
- bool **hasModelLevelFilter** () const
Returns true when this scene is scoped to a specific [ModelDataDefinition](#) level.
- unsigned int **modelLevelFilter** () const
Returns the active level used by rebuild services when the filter is enabled.
- void **setShowInternalDataDefinitions** (bool show)
Shows or hides internal data definitions.
- bool **showInternalDataDefinitions** () const
Returns whether internal data definitions are visible.
- void **setShowAttachedDataDefinitions** (bool show)
Shows or hides attached data definitions.
- bool **showAttachedDataDefinitions** () const
Returns whether attached data definitions are visible.
- void **setConnectionGeometryUpdatesBlocked** (bool blocked)
Temporarily blocks connection-geometry updates.
- bool **areConnectionGeometryUpdatesBlocked** () const
Returns whether connection-geometry updates are blocked.
- QList< QGraphicsItem * > * **getGraphicalModelDataDefinitions** () const
Returns the scene items classified as model data definitions.
- QList< QGraphicsItem * > * **getGraphicalModelComponents** () const
Returns the scene items classified as model components.
- QList< QGraphicsItem * > * **getGraphicalConnections** () const
Returns the scene items classified as connections.
- QList< QGraphicsItem * > * **getGraphicalGeometries** () const
Returns the scene items classified as geometry drawings.

- `QList< QGraphicsItem * > * getGraphicalAnimations () const`
Returns the scene items classified as animations.
- `QList< QGraphicsItem * > * getGraphicalEntities () const`
Returns the scene items classified as entities.
- `QList< QGraphicsItem * > * getGraphicalDiagramsConnections () const`
Returns the scene items classified as diagram connections.
- `QList< QGraphicsItemGroup * > * getGraphicalGroups () const`
Returns the groups currently tracked by the scene.
- `void animateTransition (ModelComponent *source, ModelComponent *destination, bool viewSimulation, Event *event)`
Starts a transition animation between two model components.
- `void runAnimateTransition (AnimationTransition *animationTransition, Event *event, bool restart=false)`
Runs one transition animation instance until completion.
- `void handleAnimationStateChanged (QAbstractAnimation::State newState, QEventLoop *loop, Event *event, AnimationTransition *animationTransition)`
Handles animation state changes for transition-driven event loops.
- `void animateQueueInsert (ModelComponent *component, bool visible)`
Creates the queue-insert animation overlay for one component.
- `void animateQueueRemove (ModelComponent *component)`
Creates the queue-remove animation overlay for one component.
- `void animateCounter ()`
Updates the counter animation overlay.
- `void animateVariable (Entity *entity=nullptr)`
Updates the variable animation overlay.
- `void animateTimer (double time)`
Updates the timer animation overlay.

Protected Member Functions

- `virtual void contextMenuEvent (QGraphicsSceneContextMenuEvent *contextMenuEvent)`
- `virtual void dragEnterEvent (QGraphicsSceneDragDropEvent *event)`
- `virtual void dragLeaveEvent (QGraphicsSceneDragDropEvent *event)`
- `virtual void dragMoveEvent (QGraphicsSceneDragDropEvent *event)`
- `virtual void dropEvent (QGraphicsSceneDragDropEvent *event)`
- `virtual void focusInEvent (QFocusEvent *focusEvent)`
- `virtual void focusOutEvent (QFocusEvent *focusEvent)`
- `virtual void keyPressEvent (QKeyEvent *keyEvent)`
- `virtual void keyReleaseEvent (QKeyEvent *keyEvent)`
- `virtual void mouseDoubleClickEvent (QGraphicsSceneMouseEvent *mouseEvent)`
- `virtual void mouseMoveEvent (QGraphicsSceneMouseEvent *mouseEvent)`
- `virtual void mousePressEvent (QGraphicsSceneMouseEvent *mouseEvent)`
- `virtual void mouseReleaseEvent (QGraphicsSceneMouseEvent *mouseEvent)`
- `virtual void wheelEvent (QGraphicsSceneWheelEvent *wheelEvent)`

10.318.1 Detailed Description

Scene that owns and manipulates the graphical representation of a model.

The scene manages components, ports, connections, drawings, animations, and the auxiliary data-definition structures used by the GUI. It is the central editing surface for the graphical model and the main consumer of the view/controller services documented elsewhere.

10.318.2 Member Function Documentation

10.318.2.1 notifyGraphicalModelChange()

```
void ModelGraphicsScene::notifyGraphicalModelChange (
    GraphicalModelEvent::EventType eventType,
    GraphicalModelEvent::EventObjectType eventObjectType,
    QGraphicsItem * item )
```

Emits a scene-level change notification used by higher GUI layers.

Notifies view-level handlers about changes in graphical model.

[Event](#) object is stack-allocated and forwarded by const-reference to avoid heap ownership ambiguity.

Todo Consider broadcasting to all attached views when multi-view editing is supported.

10.318.2.2 setModelLevelFilter()

```
void ModelGraphicsScene::setModelLevelFilter (
    unsigned int modelLevel )
```

Restricts graphical rebuild/synchronization services to one kernel model level.

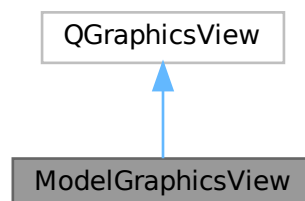
[ModelGraphicsScene](#) keeps this rendering context because some synchronization entry points are static services called from the property editor. Storing the active level in the scene lets those calls preserve the same root/submodel scope as the visible tab.

The documentation for this class was generated from the following files:

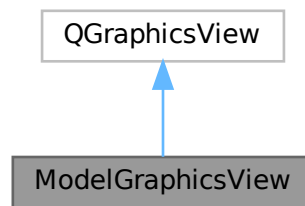
- applications/gui/genesys/graphics/ModelGraphicsScene.h
- applications/gui/genesys/graphics/ModelGraphicsScene.cpp

10.319 ModelGraphicsView Class Reference

Inheritance diagram for ModelGraphicsView:



Collaboration diagram for ModelGraphicsView:



Public Member Functions

- [ModelGraphicsView](#) (QWidget *parent=nullptr)
View wrapper that hosts the model graphics scene and delegates GUI events.
- **ModelGraphicsView** (const [ModelGraphicsView](#) &orig)
Copy constructor used by legacy compatibility paths.
- virtual ~**ModelGraphicsView** ()
Releases the Qt view and its compatibility state.
- [ModelGraphicsScene](#) * **getScene** ()
Returns the scene currently owned by the view.
- void **showGrid** ()
Requests grid visibility from the underlying scene.
- void **clear** ()
Clears the current scene contents and compatibility state.
- void **beginConnection** ()
Starts the connection-creation workflow in the scene.
- void **selectModelComponent** ([ModelComponent](#) *component)
Highlights the requested component in the scene and centers it when possible.
- void **setSimulator** ([Simulator](#) *simulator)
Injects the simulator dependency used by scene callbacks.
- void **setPropertyEditor** ([PropertyEditorGenesys](#) *propEditor)
Injects the property editor dependency used by scene callbacks.
- void **setPropertyList** (std::map< [SimulationControl](#) *, [DataComponentProperty](#) * > *propList)
Injects the property list map used by scene/editor synchronization.
- void **setPropertyEditorUI** (std::map< [SimulationControl](#) *, [DataComponentEditor](#) * > *propEditorUI)
Injects the property-editor widget map used by scene/editor synchronization.
- void **setComboBox** (std::map< [SimulationControl](#) *, [ComboBoxEnum](#) * > *propBox)
Injects the enum combo-box map used by scene/editor synchronization.
- void **setEnabled** (bool enabled)
Enables or disables interaction while keeping the scene wiring intact.
- QList< QGraphicsItem * > **selectedItems** ()
Returns the current selected items from the underlying scene.
- template<typename Class >
void **setSceneMouseEventHandler** (Class *object, void(Class::*function)(QGraphicsSceneMouseEvent *))

- `template<typename Class >`
`void setGraphicalModelEventHandler (Class *object, void(Class::*function)(const GraphicalModelEvent &))`
- `template<typename Class >`
`void setSceneWheelInEventHandler (Class *object, void(Class::*function)())`
- `template<typename Class >`
`void setSceneWheelOutEventHandler (Class *object, void(Class::*function)())`
- `template<typename Class >`
`void setContextMenuEventHandler (Class *object, void(Class::*function)(QContextMenuEvent *))`
- `void notifySceneMouseEventHandler (QGraphicsSceneMouseEvent *mouseEvent)`
Notifies the registered mouse-event handler, if any.
- `void notifySceneWheelInEventHandler ()`
Notifies the registered wheel-in handler, if any.
- `void notifySceneWheelOutEventHandler ()`
Notifies the registered wheel-out handler, if any.
- `void notifySceneGraphicalModelEventHandler (const GraphicalModelEvent &modelGraphicsEvent)`
Notifies the registered graphical-model-event handler, if any.
- `void setCanNotifyGraphicalModelEventHandlers (bool can)`
Enables or disables graphical-model event emission from the view.
- `void clearEventHandlers ()`
Clears all custom event handlers installed by the composition root.
- `void setParentWidget (QWidget *parentWidget)`
Sets the widget used as parent for dialogs and transient controls.
- `void setRuleVisible (bool visible)`
Enables or disables ruler rendering over the graphics viewport.
- `bool isRuleVisible () const`
Returns whether ruler rendering is active.
- `void setGuidesVisible (bool visible)`
Enables or disables guide line rendering over the graphics viewport.
- `bool isGuidesVisible () const`
Returns whether guide line rendering is active.

Protected Member Functions

- `void changed (const QList< QRectF > ®ion)`
- `void focusItemChanged (QGraphicsItem *newFocusItem, QGraphicsItem *oldFocusItem, Qt::FocusReason reason)`
- `void sceneRectChanged (const QRectF &rect)`
- `void selectionChanged ()`
- `virtual void contextMenuEvent (QContextMenuEvent *event) override`
- `virtual void dragEnterEvent (QDragEnterEvent *event) override`
- `virtual void keyPressEvent (QKeyEvent *event) override`
- `virtual void keyReleaseEvent (QKeyEvent *event) override`
- `virtual void wheelEvent (QWheelEvent *event) override`
- `virtual void drawForeground (QPainter *painter, const QRectF &rect) override`

10.319.1 Constructor & Destructor Documentation

10.319.1.1 ModelGraphicsView()

```
ModelGraphicsView::ModelGraphicsView (
    QWidget * parent = nullptr )
```

View wrapper that hosts the model graphics scene and delegates GUI events.

The view keeps the scene-specific input handlers outside the raw Qt widget layer so the [MainWindow](#) composition root can wire context menus, wheel behavior, and selection changes through injected callbacks.

10.319.2 Member Function Documentation

10.319.2.1 notifySceneGraphicalModelEventHandler()

```
void ModelGraphicsView::notifySceneGraphicalModelEventHandler (
    const GraphicalModelEvent & modelGraphicsEvent )
```

Notifies the registered graphical-model-event handler, if any.

Dispatches graphical model change event to registered callback.

Parameters

<i>modelGraphicsEvent</i>	Event payload from scene.
---------------------------	---------------------------

Todo Replace callback with signal/slot to improve composability.

Todo actualize property editor?

The documentation for this class was generated from the following files:

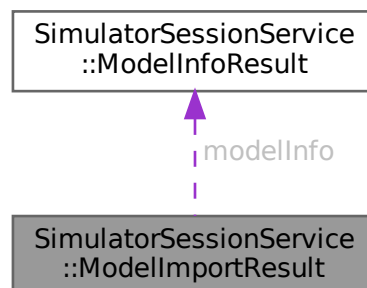
- applications/gui/genesys/graphics/ModelGraphicsView.h
- applications/gui/genesys/graphics/ModelGraphicsView.cpp

10.320 SimulatorSessionService::ModelImportResult Struct Reference

Contains model import output and error state.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::ModelImportResult:



Public Attributes

- bool **success** = false
- [ModellImportError](#) **error** = ModellImportError::None
- [ModellInfoResult](#) **modellInfo**

10.320.1 Detailed Description

Contains model import output and error state.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

10.321 ModellInfo Class Reference

```
#include <kernel/simulator/model/ModelInfo.h>
```

Public Member Functions

- std::string **show** ()
- void [setName](#) (std::string _name)
setName
- std::string [getName](#) () const
getName
- void [setAnalystName](#) (std::string _analystName)
setAnalystName
- std::string [getAnalystName](#) () const
getAnalystName
- void [setDescription](#) (std::string _description)
setDescription
- std::string [getDescription](#) () const
getDescription
- void [setProjectTitle](#) (std::string _projectTitle)
setProjectTitle
- std::string [getProjectTitle](#) () const
getProjectTitle
- void [setVersion](#) (std::string _version)
setVersion
- std::string [getVersion](#) () const
getVersion
- void [loadInstance](#) ([PersistenceRecord](#) *fields)
loadInstance
- void [saveInstance](#) ([PersistenceRecord](#) *fields)
saveInstance
- bool [hasChanged](#) () const
hasChanged
- void **setHasChanged** (bool [hasChanged](#))
Updates the changed flag for model metadata.

10.321.1 Detailed Description

[ModelInfo](#) stores basic model project information, such as the project and analyst names.

10.321.2 Member Function Documentation

10.321.2.1 `getAnalystName()`

```
std::string ModelInfo::getAnalystName ( ) const
```

`getAnalystName`

Returns

10.321.2.2 `getDescription()`

```
std::string ModelInfo::getDescription ( ) const
```

`getDescription`

Returns

10.321.2.3 `getName()`

```
std::string ModelInfo::getName ( ) const
```

`getName`

Returns

10.321.2.4 `getProjectTitle()`

```
std::string ModelInfo::getProjectTitle ( ) const
```

`getProjectTitle`

Returns

10.321.2.5 getVersion()

```
std::string ModelInfo::getVersion ( ) const
```

getVersion

Returns

10.321.2.6 hasChanged()

```
bool ModelInfo::hasChanged ( ) const
```

hasChanged

Returns

10.321.2.7 loadInstance()

```
void ModelInfo::loadInstance (
    PersistenceRecord * fields )
```

loadInstance

Parameters

<i>fields</i>	
---------------	--

10.321.2.8 saveInstance()

```
void ModelInfo::saveInstance (
    PersistenceRecord * fields )
```

saveInstance

Parameters

<i>fields</i>	
---------------	--

10.321.2.9 setAnalystName()

```
void ModelInfo::setAnalystName (
    std::string _analystName )
```

setAnalystName

Parameters

<i>_analystName</i>	
---------------------	--

10.321.2.10 setDescription()

```
void ModelInfo::setDescription (
    std::string _description )
```

setDescription

Parameters

<i>_description</i>	
---------------------	--

10.321.2.11 setName()

```
void ModelInfo::setName (
    std::string _name )
```

setName

Parameters

<i>_name</i>	
--------------	--

10.321.2.12 setProjectTitle()

```
void ModelInfo::setProjectTitle (
    std::string _projectTitle )
```

setProjectTitle

Parameters

<i>_projectTitle</i>	
----------------------	--

10.321.2.13 setVersion()

```
void ModelInfo::setVersion (
    std::string _version )
```

setVersion

Parameters

<code>_version</code>	
-----------------------	--

The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelInfo.h
- kernel/simulator/model/ModelInfo.cpp

10.322 SimulatorSessionService::ModelInfoResult Struct Reference

Describes the currently active model information for a session.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Public Attributes

- bool **exists** = false
- unsigned int **modelId** = 0
- bool **hasChanged** = false
- unsigned int **level** = 0
- std::string **name**
- std::string **analystName**
- std::string **projectTitle**
- std::string **version**
- std::string **description**
- unsigned int **componentCount** = 0

10.322.1 Detailed Description

Describes the currently active model information for a session.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

10.323 ModelInspectorController Class Reference

Controller for model inspection views and scene-selection synchronization.

```
#include <applications/gui/genesys/controllers/ModelInspectorController.h>
```

Classes

- struct [DataDefinitionSelection](#)

Public Member Functions

- **ModelInspectorController** ([Simulator](#) *simulator, QTreeWidget *componentsTree, QTreeWidget *dataDefinitionsTree, [ModelGraphicsView](#) *graphicsView)
Creates the model-inspector delegated controller.
- void **actualizeModelComponents** (bool force) const
Synchronizes the Components tree from the current model representation.
- void **actualizeModelDataDefinitions** (bool force) const
Synchronizes the Data Definitions tree from current model data definitions.
- void **beginDataDefinitionNameEdit** (QTreeWidgetItem *item, int column) const
Starts in-place rename editing for editable data-definition entries.
- void **applyDataDefinitionNameChange** (QTreeWidgetItem *item, int column) const
Commits edited data-definition names back to the kernel model.
- void **syncSelectedComponentTreeItemToScene** () const
Synchronizes tree selection with scene selection and viewport focus.
- [DataDefinitionSelection](#) **syncSelectedDataDefinitionTreeItemToScene** () const
Synchronizes Data Definitions tree selection with visible GMDDs, when any.

10.323.1 Detailed Description

Controller for model inspection views and scene-selection synchronization.

In the refactored GUI architecture, this controller receives delegated callbacks from [MainWindow](#) to keep model-inspector responsibilities outside the composition root while preserving the existing UI contract. It synchronizes Components/Data Definitions trees with the current kernel model and keeps tree-to-scene selection behavior consistent.

Responsibilities:

- refresh model component and data-definition trees from [Simulator](#) state;
- manage in-place rename flow for editable data-definition names;
- propagate current tree selection to the graphical scene and viewport.

Boundaries:

- it does not create/destroy models or files (model lifecycle remains elsewhere);
- it does not own scene command logic, undo/redo, or simulation commands;
- it does not act as a persistence or export service.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/ModelInspectorController.h
- applications/gui/genesys/controllers/ModelInspectorController.cpp

10.324 ModelLanguageSynchronizer Class Reference

Service layer that synchronizes textual model language with kernel model state.

```
#include <applications/gui/genesys/services/ModelLanguageSynchronizer.h>
```


Public Member Functions

- [ModelLanguageSynchronizer](#) ([Simulator](#) *simulator, [QPlainTextEdit](#) *modelTextEditor, bool *textModelHasChangedFlag, [QWidget](#) *ownerWidget, std::function< void()> onModelCreatedOrLoaded)
Creates the synchronization service used by [MainWindow](#) compatibility wrappers.
- void [actualizeModelSimLanguage](#) () const
Rebuilds textual model representation from the current kernel model.
- bool [setSimulationModelBasedOnText](#) () const
Parses text editor content and applies it to the simulator model.

10.324.1 Detailed Description

Service layer that synchronizes textual model language with kernel model state.

This service was extracted from [MainWindow](#) representation code to isolate text/model synchronization concerns. [MainWindow](#) remains the composition root and delegates wrapper methods to this class.

Responsibilities:

- generate/update the textual model representation from current [Simulator](#) model;
- parse/apply textual edits back into simulator model state;
- notify [MainWindow](#) via callback when model creation/loading side effects are required.

Boundaries:

- it does not own simulator or editor widgets;
- it does not handle graphical persistence, Graphviz export, or C++ code export;
- it is a synchronization service, not a controller.

10.324.2 Constructor & Destructor Documentation

10.324.2.1 ModelLanguageSynchronizer()

```
ModelLanguageSynchronizer::ModelLanguageSynchronizer (
    Simulator * simulator,
    QPlainTextEdit * modelTextEditor,
    bool * textModelHasChangedFlag,
    QWidget * ownerWidget,
    std::function< void()> onModelCreatedOrLoaded )
```

Creates the synchronization service used by [MainWindow](#) compatibility wrappers.

Stores explicit dependencies once to keep wrapper calls thin.

10.324.3 Member Function Documentation

10.324.3.1 `actualizeModelSimLanguage()`

```
void ModelLanguageSynchronizer::actualizeModelSimLanguage ( ) const
```

Rebuilds textual model representation from the current kernel model.

Regenerates the text editor content from the current kernel model.

This is part of the synchronization bridge used after lifecycle, property, and scene updates.

10.324.3.2 `setSimulationModelBasedOnText()`

```
bool ModelLanguageSynchronizer::setSimulationModelBasedOnText ( ) const
```

Parses text editor content and applies it to the simulator model.

Applies the text editor content to the current kernel model.

Returns

true when synchronization succeeds and model state is valid for downstream workflows.

The documentation for this class was generated from the following files:

- applications/gui/genesys/services/ModelLanguageSynchronizer.h
- applications/gui/genesys/services/ModelLanguageSynchronizer.cpp

10.325 ModelLifecycleController Class Reference

Controller for model/application lifecycle flows delegated from [MainWindow](#).

```
#include <applications/gui/genesys/controllers/ModelLifecycleController.h>
```

Classes

- struct [Callbacks](#)

Public Member Functions

- **ModelLifecycleController** (QWidget *ownerWidget, [Simulator](#) *simulator, Ui::MainWindow *ui, QString *modelName, bool *textModelHasChanged, bool *graphicalModelHasChanged, bool *closingApproved, bool *loaded, bool ¶llelizationEnabled, int ¶llelizationThreads, int ¶llelizationBatchSize, [Callbacks](#) callbacks)
Creates the lifecycle orchestration controller used by [MainWindow](#) wrappers.
- void **onActionModelNewTriggered** () const
Starts the new-model lifecycle flow and reinitializes GUI/model state.
- void **onActionModelOpenTriggered** () const
Starts model-open flow with persistence loading and UI synchronization.
- bool **openModelFile** (const QString &fileName) const
Opens a model from an explicit file path using the same lifecycle flow as the open action.
- void **onActionModelSaveTriggered** () const
Starts model-save flow for graphical/text representations.
- void **onActionModelCloseTriggered** () const
Closes current model after pending-change checks and cleanup orchestration.
- void **onActionModelInformationTriggered** () const
Opens model information dialog for the current model context.
- void **onActionModelCheckTriggered** () const
Runs model-check workflow as a delegated lifecycle operation.
- void **onActionSimulationConfigureTriggered** () const
Opens simulation configuration workflow anchored to lifecycle façade.
- void **onActionSimulatorExitTriggered** () const
Executes application-exit flow with pending-change confirmation semantics.
- bool **hasPendingModelChanges** () const
Indicates whether current model/text state has unsaved changes.
- bool **confirmApplicationExit** () const
Requests exit confirmation preserving compatibility prompt behavior.

10.325.1 Detailed Description

Controller for model/application lifecycle flows delegated from [MainWindow](#).

This controller centralizes lifecycle actions (new/open/save/close/check/configure/exit) while [MainWindow](#) remains the composition root and compatibility façade. It uses explicit callback injection to invoke existing UI update and persistence routines without changing signatures or ownership.

Responsibilities:

- orchestrate user-triggered lifecycle actions and guard pending-change confirmations;
- coordinate model load/save paths through injected serializer callbacks;
- trigger UI/tab/action refreshes required after lifecycle transitions.

Boundaries:

- it does not directly implement file formats (delegated to services);
- it does not own simulator/model objects;
- it does not manage scene tools, property editor internals, or simulation traces.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/ModelLifecycleController.h
- applications/gui/genesys/controllers/ModelLifecycleController.cpp

10.326 ModelManager Class Reference

The [ModelManager](#) class.

```
#include <kernel/simulator/model/ModelManager.h>
```

Public Member Functions

- [ModelManager](#) ([Simulator](#) *simulator)
Creates a model manager bound to a simulator instance.
- virtual ~[ModelManager](#) ()
Releases the model manager and any owned bookkeeping structures.
- [Model](#) * [newModel](#) ()
Creates a new model, inserts it into the open-model list and makes it current.
- void [insert](#) ([Model](#) *model)
Inserts an existing model into the open-model list.
- void [remove](#) ([Model](#) *model)
Removes a model from the open-model list.
- bool [setCurrent](#) ([Model](#) *model)
Makes an open model the current one.
- bool [saveModel](#) (std::string filename)
Saves the current model to a file.
- [Model](#) * [loadModel](#) (std::string filename)
Loads a model from a file and opens it in the manager.
- [Model](#) * [createFromLanguage](#) (std::string modelSpecification)
Creates a model from a textual model specification.
- unsigned int [size](#) ()
Returns the number of open models.
- std::vector< [Model](#) * > [models](#) () const
Returns a stable snapshot of the currently open models in tab/navigation order.
- bool [hasModel](#) ([Model](#) *model) const
Checks whether a model is owned by this manager.
- int [indexOf](#) ([Model](#) *model) const
Returns the open-model index of a model.
- [Model](#) * [modelAt](#) (unsigned int index) const
Returns the model at a given open-model index.
- int [currentIndex](#) () const
Returns the index of the current model.
- [Model](#) * [front](#) ()
Returns the first open model.
- [Model](#) * [last](#) ()
Returns the last open model.
- [Model](#) * [current](#) ()
Returns the current model.
- [Model](#) * [next](#) ()
Advances to the next open model.
- [Model](#) * [previous](#) ()
Moves to the previous open model.
- bool [canGoNext](#) () const
Indicates whether a next model exists.
- bool [canGoPrevious](#) () const
Indicates whether a previous model exists.

10.326.1 Detailed Description

The [ModelManager](#) class.

10.326.2 Constructor & Destructor Documentation

10.326.2.1 ModelManager()

```
ModelManager::ModelManager (
    Simulator * simulator )
```

Creates a model manager bound to a simulator instance.

Parameters

<i>simulator</i>	Parent simulator that owns this manager.
------------------	--

10.326.3 Member Function Documentation

10.326.3.1 canGoNext()

```
bool ModelManager::canGoNext ( ) const
```

Indicates whether a next model exists.

Returns

`true` when a call to `next()` can advance.

10.326.3.2 canGoPrevious()

```
bool ModelManager::canGoPrevious ( ) const
```

Indicates whether a previous model exists.

Returns

`true` when a call to `previous()` can retreat.

10.326.3.3 createFromLanguage()

```
Model * ModelManager::createFromLanguage (
    std::string modelSpecification )
```

Creates a model from a textual model specification.

Parameters

<i>modelSpecification</i>	Source text in the model language.
---------------------------	------------------------------------

Returns

Newly created model instance, or `nullptr` on failure.

10.326.3.4 current()

```
Model * ModelManager::current ( )
```

Returns the current model.

Returns

Currently selected model, or `nullptr` when none is selected.

10.326.3.5 currentIndex()

```
int ModelManager::currentIndex ( ) const
```

Returns the index of the current model.

Returns

Zero-based current model index, or -1 when there is no current model.

10.326.3.6 front()

```
Model * ModelManager::front ( )
```

Returns the first open model.

Returns

First model in open-model order, or `nullptr` when empty.

10.326.3.7 hasModel()

```
bool ModelManager::hasModel (
    Model * model ) const
```

Checks whether a model is owned by this manager.

Parameters

<i>model</i>	Model pointer to test.
--------------	--

Returns

True when model is in the open-model list.

10.326.3.8 indexOf()

```
int ModelManager::indexOf (
    Model * model ) const
```

Returns the open-model index of a model.

Parameters

<i>model</i>	Model pointer to locate.
--------------	--

Returns

Zero-based index, or -1 when not open.

10.326.3.9 insert()

```
void ModelManager::insert (
    Model * model )
```

Inserts an existing model into the open-model list.

Parameters

<i>model</i>	Model to register.
--------------	------------------------------------

10.326.3.10 last()

```
Model * ModelManager::last ( )
```

Returns the last open model.

Returns

Last model in open-model order, or `nullptr` when empty.

10.326.3.11 loadModel()

```
Model * ModelManager::loadModel (
    std::string filename )
```

Loads a model from a file and opens it in the manager.

Parameters

<i>filename</i>	Input filename.
-----------------	-----------------

Returns

Loaded model instance, or `nullptr` on failure.

10.326.3.12 modelAt()

```
Model * ModelManager::modelAt (
    unsigned int index ) const
```

Returns the model at a given open-model index.

Parameters

<i>index</i>	Zero-based model index.
--------------	-------------------------

Returns

[Model](#) pointer, or `nullptr` when index is out of range.

10.326.3.13 models()

```
std::vector< Model * > ModelManager::models ( ) const
```

Returns a stable snapshot of the currently open models in tab/navigation order.

Returns

Vector with the open model pointers owned by this manager.

10.326.3.14 newModel()

```
Model * ModelManager::newModel ( )
```

Creates a new model, inserts it into the open-model list and makes it current.

Returns

The newly opened current model.

10.326.3.15 next()

```
Model * ModelManager::next ( )
```

Advances to the next open model.

Returns

Next model in order, or `nullptr` when already at the end.

10.326.3.16 previous()

```
Model * ModelManager::previous ( )
```

Moves to the previous open model.

Returns

Previous model in order, or `nullptr` when already at the beginning.

10.326.3.17 remove()

```
void ModelManager::remove (
    Model * model )
```

Removes a model from the open-model list.

Parameters

<i>model</i>	Model to remove.
--------------	------------------

10.326.3.18 saveModel()

```
bool ModelManager::saveModel (
    std::string filename )
```

Saves the current model to a file.

Parameters

<i>filename</i>	Output filename.
-----------------	------------------

Returns

`true` when saving succeeds.

10.326.3.19 setCurrent()

```
bool ModelManager::setCurrent (
    Model * model )
```

Makes an open model the current one.

Parameters

<i>model</i>	Model to activate.
--------------	--------------------

Returns

`true` when the model was found and selected.

10.326.3.20 size()

```
unsigned int ModelManager::size ( )
```

Returns the number of open models.

Returns

Open-model count.

The documentation for this class was generated from the following files:

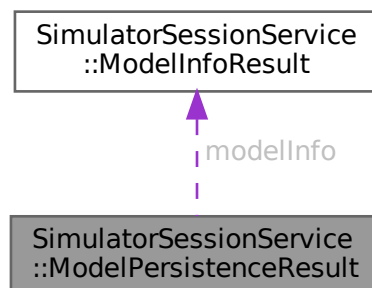
- kernel/simulator/model/ModelManager.h
- kernel/simulator/model/ModelManager.cpp

10.327 SimulatorSessionService::ModelPersistenceResult Struct Reference

Contains save/load model operation output and error state.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::ModelPersistenceResult:



Public Attributes

- bool **success** = false
- [PersistenceError](#) **error** = PersistenceError::None
- std::string **filename**
- [ModelInfoResult](#) **modelInfo**

10.327.1 Detailed Description

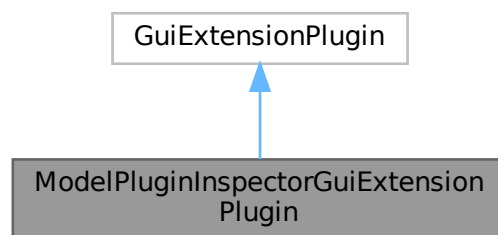
Contains save/load model operation output and error state.

The documentation for this struct was generated from the following file:

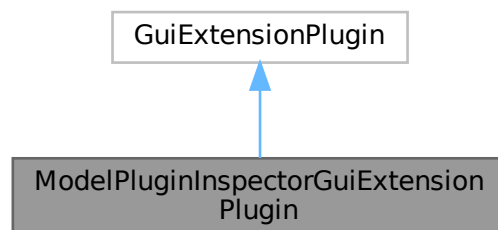
- applications/worker/service/SimulatorSessionService.h

10.328 ModelPluginInspectorGuiExtensionPlugin Class Reference

Inheritance diagram for ModelPluginInspectorGuiExtensionPlugin:



Collaboration diagram for ModelPluginInspectorGuiExtensionPlugin:



Public Member Functions

- `std::string extensionId ()` const override
- `std::string displayName ()` const override
- `std::string version ()` const override
- `std::vector< std::string > requiredModelPlugins ()` const override
- `void registerContributions (GuiExtensionRegistry *registry)` const override

10.328.1 Member Function Documentation

10.328.1.1 displayName()

```
std::string ModelPluginInspectorGuiExtensionPlugin::displayName ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.328.1.2 extensionId()

```
std::string ModelPluginInspectorGuiExtensionPlugin::extensionId ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.328.1.3 registerContributions()

```
void ModelPluginInspectorGuiExtensionPlugin::registerContributions (
    GuiExtensionRegistry * registry ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.328.1.4 requiredModelPlugins()

```
std::vector< std::string > ModelPluginInspectorGuiExtensionPlugin::requiredModelPlugins ( )
const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

10.328.1.5 version()

```
std::string ModelPluginInspectorGuiExtensionPlugin::version ( ) const [inline], [override], [virtual]
```

Implements [GuiExtensionPlugin](#).

The documentation for this class was generated from the following file:

- `applications/gui/genesys/extensions/ModelPluginInspectorGuiExtensionPlugin.cpp`

10.329 ModelSimulation Class Reference

Controls the mature, currently functional simulation-execution layer of a model.

```
#include <kernel/simulator/model/ModelSimulation.h>
```

Public Member Functions

- **ModelSimulation** ([Model](#) *model)
- `std::string` **show** ()
- `void` **start** ()

Starts a sequential execution of a simulation, ie, a set of replications of this model.
- `void` **pause** ()
- `void` **step** ()

Executes the processing of a single event, the next one in the future events list.
- `void` **stop** ()
- `void` **setNumberOfReplications** (unsigned int _numberOfReplications)
- unsigned int **getNumberOfReplications** () const
- `void` **setReplicationLength** (double _replicationLength, [Util::TimeUnit](#) _replicationLengthTimeUnit=[Util::TimeUnit::unknown](#))
- double **getReplicationLength** () const
- `void` **setReplicationLengthTimeUnit** ([Util::TimeUnit](#) _replicationLengthTimeUnit)
- [Util::TimeUnit](#) **getReplicationLengthTimeUnit** () const
- `void` **setReplicationReportBaseTimeUnit** ([Util::TimeUnit](#) _replicationReportBaseTimeUnit)
- [Util::TimeUnit](#) **getReplicationBaseTimeUnit** () const
- `void` **setWarmUpPeriod** (double warmUpPeriod, [Util::TimeUnit](#) warmUpPeriodTimeUnit=[Util::TimeUnit::unknown](#))
- double **getWarmUpPeriod** () const
- `void` **setWarmUpPeriodTimeUnit** ([Util::TimeUnit](#) _warmUpPeriodTimeUnit)
- [Util::TimeUnit](#) **getWarmUpPeriodTimeUnit** () const
- `void` **setTerminatingCondition** (std::string _terminatingCondition)
- std::string **getTerminatingCondition** () const
- `void` **setPauseOnEvent** (bool _pauseOnEvent)
- bool **isPauseOnEvent** () const
- `void` **setStepByStep** (bool _stepByStep)
- bool **isStepByStep** () const
- `void` **setInitializeStatistics** (bool _initializeStatistics)
- bool **isInitializeStatistics** () const
- `void` **setInitializeSystem** (bool _initializeSystem)
- bool **isInitializeSystem** () const
- `void` **setPauseOnReplication** (bool _pauseBetweenReplications)
- bool **isPauseOnReplication** () const
- `void` **setReporter** ([SimulationReporter_if](#) *_simulationReporter)
- [SimulationReporter_if](#) * **getReporter** () const
- double **getSimulatedTime** () const
- bool **isRunning** () const
- bool **isPaused** () const
- unsigned int **getCurrentReplicationNumber** () const
- `void` **setShowReportsAfterReplication** (bool showReportsAfterReplication)
- bool **isShowReportsAfterReplication** () const
- `void` **setShowReportsAfterSimulation** (bool showReportsAfterSimulation)
- bool **isShowReportsAfterSimulation** () const
- [List](#)< double > * **getBreakpointsOnTime** () const

- [List](#)< [Entity](#) * > * **getBreakpointsOnEntity** () const
- [List](#)< [ModelComponent](#) * > * **getBreakpointsOnComponent** () const
- const [List](#)< [ModelDataDefinition](#) * > * **getSimulationStatisticsAggregates** () const
- void **loadInstance** ([PersistenceRecord](#) *fields)
- void **saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaults)
- [Event](#) * **getCurrentEvent** () const
- void **setShowSimulationResponsesInReport** (bool _showSimulationResponsesInReport)
- bool **isShowSimulationResponsesInReport** () const
- void **setShowSimulationControlsInReport** (bool _showSimulationControlsInReport)
- bool **isShowSimulationControlsInReport** () const

Friends

- class **Model**

10.329.1 Detailed Description

Controls the mature, currently functional simulation-execution layer of a model.

The [ModelSimulation](#) controls the simulation of a model, allowing to start, pause, resume e stop a simulation, composed by a set of replications.

Historically, this class has been the original and operational experiment mechanism in GenESyS. It already supports the core execution workflow based on replications, replication length, warm-up configuration and event processing.

Later experiment-oriented classes such as [SimulationExperiment](#) and [SimulationScenario](#) were introduced as an attempt to move richer design-of-experiments concerns into the kernel. Those newer classes are still under development, while [ModelSimulation](#) remains the stable execution core.

10.329.2 Member Function Documentation

10.329.2.1 isRunning()

```
bool ModelSimulation::isRunning ( ) const
```

The current time in the model being simulated, i.e., the instant when the current event was triggered

10.329.2.2 start()

```
void ModelSimulation::start ( )
```

Starts a sequential execution of a simulation, ie, a set of replications of this model.

Checks the model and if ok then initialize the simulation, execute repeatedly each replication and then show simulation statistics

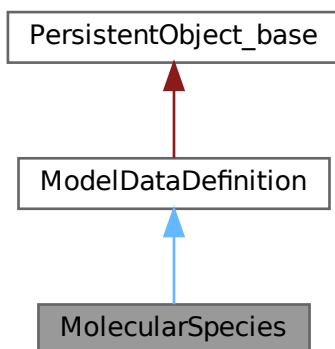
The documentation for this class was generated from the following files:

- kernel/simulator/model/ModelSimulation.h
- kernel/simulator/model/ModelSimulation.cpp

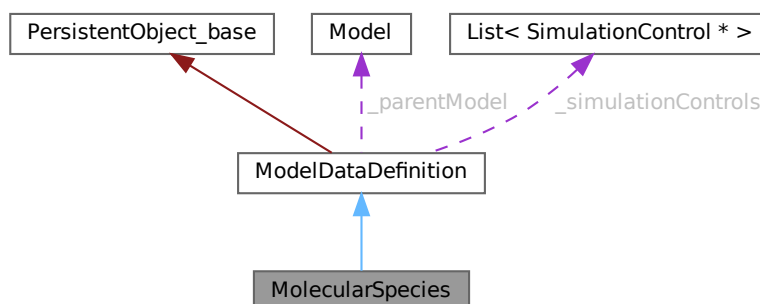
10.330 MolecularSpecies Class Reference

```
#include <plugins/data/WholeCellModeling/MolecularSpecies.h>
```

Inheritance diagram for MolecularSpecies:



Collaboration diagram for MolecularSpecies:



Public Member Functions

- **MolecularSpecies** ([Model](#) *model, std::string name="")
- void **setInitialCount** (int initialCount)
- int **getInitialCount** () const
- void **setCount** (int count)
- int **getCount** () const
- void **setCompartment** (std::string compartment)
- std::string **getCompartment** () const
- void **setUnit** (std::string unit)
- std::string **getUnit** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_initBetweenReplications** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createNonEditableDataDefinitions** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()

- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.330.1 Detailed Description

Discrete molecular species with integer copy count for Gillespie SSA.

Unlike [BioSpecies](#) (continuous double amounts for ODE), [MolecularSpecies](#) tracks integer molecule counts. Compartment and unit annotate the biological context without affecting simulation logic.

10.330.2 Member Function Documentation

10.330.2.1 `_check()`

```
bool MolecularSpecies::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.330.2.2 `_initBetweenReplications()`

```
void MolecularSpecies::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.330.2.3 `_loadInstance()`

```
bool MolecularSpecies::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.330.2.4 `_saveInstance()`

```
void MolecularSpecies::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.330.2.5 show()

```
std::string MolecularSpecies::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/WholeCellModeling/MolecularSpecies.h
- plugins/data/WholeCellModeling/MolecularSpecies.cpp

10.331 GroProgramRuntime::MotionMutation Struct Reference

Public Attributes

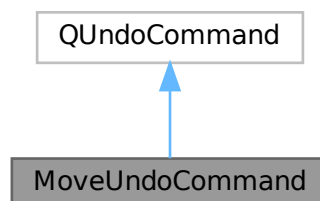
- MotionMutationType **type** = MotionMutationType::Run
- double **value** = 0.0
- double **previousDirection** = 0.0
- double **resultingDirection** = 0.0
- double **previousSpeed** = 0.0
- double **resultingSpeed** = 0.0

The documentation for this struct was generated from the following file:

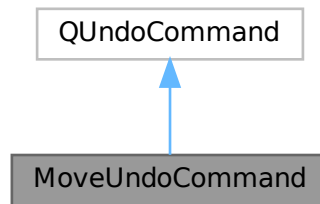
- plugins/data/BiochemicalSimulation/GroProgramRuntime.h

10.332 MoveUndoCommand Class Reference

Inheritance diagram for MoveUndoCommand:



Collaboration diagram for MoveUndoCommand:



Public Member Functions

- **MoveUndoCommand** (QList< QGraphicsItem * > item, [ModelGraphicsScene](#) *scene, QList< QPointF > &oldPos, QList< QPointF > &newPos, QUndoCommand *parent=nullptr)
- void **undo** () override
- void **redo** () override

The documentation for this class was generated from the following files:

- applications/gui/genesys/actions/MoveUndoCommand.h
- applications/gui/genesys/actions/MoveUndoCommand.cpp

10.333 dialogBreakpoint::MVCResult Class Reference

Public Member Functions

- **MVCResult** (std::string type, std::string on)

Public Attributes

- std::string **type**
- std::string **on**

The documentation for this class was generated from the following file:

- applications/gui/genesys/dialogs/dialogBreakpoint.h

10.334 GroProgramAst::NamedProgram Struct Reference

Public Attributes

- std::string **name** = ""
- std::vector< std::string > **parameters**
- std::string **bodySource** = ""
- std::vector< [Statement](#) > **statements**

The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/GroProgramAst.h

10.335 GroProgramIr::NamedProgramDefinition Struct Reference

Public Attributes

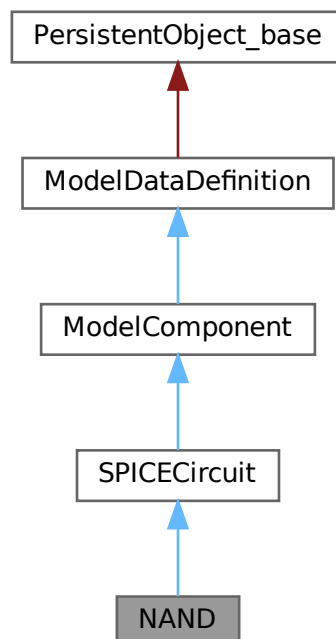
- std::vector< std::string > **parameters**
- std::string **sourceText** = ""
- std::vector< [Command](#) > **commands**

The documentation for this struct was generated from the following file:

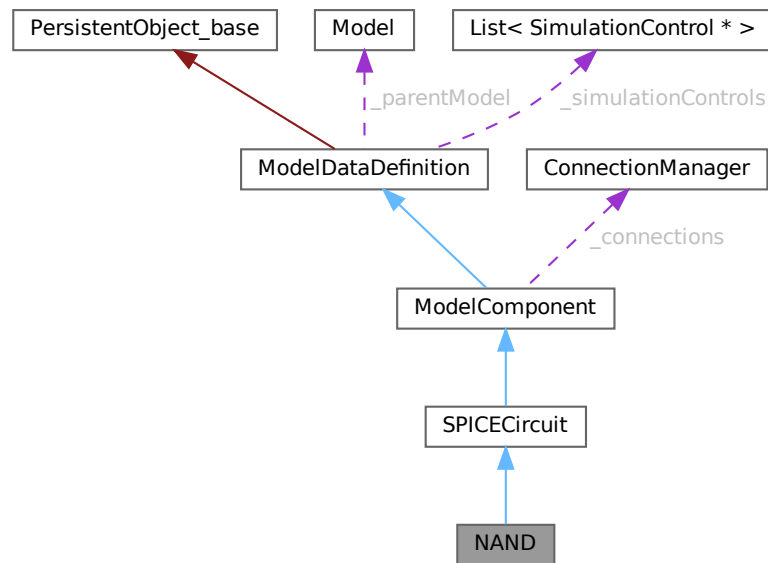
- plugins/data/BiochemicalSimulation/GroProgramIr.h

10.336 NAND Class Reference

Inheritance diagram for NAND:



Collaboration diagram for NAND:



Public Member Functions

- **NAND** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setPMOSWidth** (float PMOSWidth)
- void **setPMOSLength** (float length)
- void **setNMOSWidth** (float NMOSWidth)
- void **setNMOSLength** (float NMOSLength)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.

- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.

- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List< SimulationControl * > * getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation * GetPluginInformation](#) ()
- static [ModelComponent * LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition * NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation * GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent * LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition * NewInstance](#) ([Model](#) *model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent * LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()

- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
 - legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.336.1 Member Function Documentation

10.336.1.1 `_loadInstance()`

```
bool NAND::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.336.1.2 `_saveInstance()`

```
void NAND::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.336.1.3 `Build()`

```
void NAND::Build ( ) [override], [virtual]
```

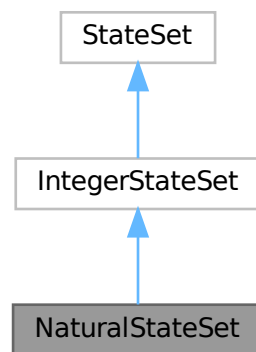
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

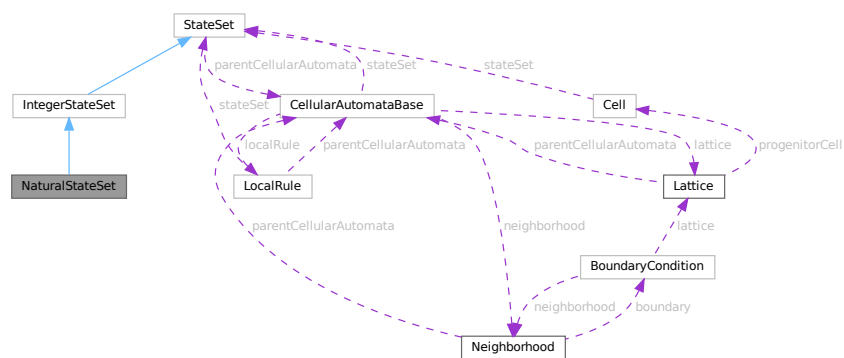
- plugins/components/ElectronicsSimulation/SPIECircuit.h
- plugins/components/ElectronicsSimulation/SPIECircuit.cpp

10.337 NaturalStateSet Class Reference

Inheritance diagram for NaturalStateSet:



Collaboration diagram for NaturalStateSet:



Public Member Functions

- **NaturalStateSet** ([CellularAutomataBase](#) *parentCellularAutomata, std::optional< long > maximum=std::nullopt)
- std::string [name](#) () const override

Public Member Functions inherited from [IntegerStateSet](#)

- **IntegerStateSet** ([CellularAutomataBase](#) *parentCellularAutomata, std::optional< long > minimum=std::nullopt, std::optional< long > maximum=std::nullopt)
- bool [contains](#) (const [State](#) &state) const override
- std::unique_ptr< [State](#) > [createDefaultState](#) () const override
- std::unique_ptr< [State](#) > [parseState](#) (const std::string &text) const override
- bool [isFinite](#) () const override
- bool [isDiscrete](#) () const override
- std::vector< std::unique_ptr< [State](#) > > [enumerateStates](#) () const override

Public Member Functions inherited from [StateSet](#)

- **StateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- **StateSet** (const [StateSet](#) &orig)
- virtual bool [isContinuous](#) () const
- virtual bool [isComposite](#) () const

Additional Inherited Members

Protected Attributes inherited from [StateSet](#)

- [CellularAutomataBase](#) * [parentCellularAutomata](#)

10.337.1 Member Function Documentation

10.337.1.1 [name\(\)](#)

```
std::string NaturalStateSet::name ( ) const [inline], [override], [virtual]
```

Reimplemented from [IntegerStateSet](#).

The documentation for this class was generated from the following file:

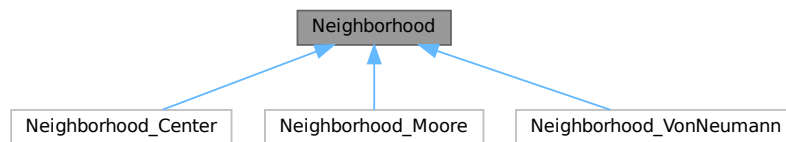
- plugins/components/ModalModel/CellularAutomata/StateSet.h

10.338 Neighborhood Class Reference

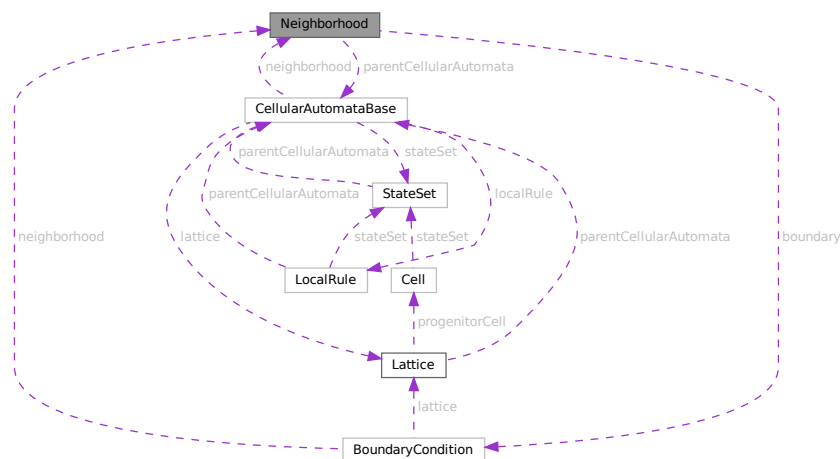
Defines the complete neighborhood of a cell.

```
#include <plugins/components/ModalModel/CellularAutomata/Neighborhood.h>
```

Inheritance diagram for Neighborhood:



Collaboration diagram for Neighborhood:



Public Member Functions

- **Neighborhood** ([CellularAutomataBase](#) *parentCellularAutomata, unsigned short radius=1, [BoundaryCondition](#) *boundary=nullptr, bool includeCellItself=false)
- **Neighborhood** (const [Neighborhood](#) &orig)
- virtual std::string **show** ()
- virtual std::vector< [Cell](#) * > **getNeighbors** ([Cell](#) *cell)=0
pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself.
- void **setRadius** (unsigned short radius)
- unsigned short **getRadius** () const
- void **setBoundary** ([BoundaryCondition](#) *boundary)
- [BoundaryCondition](#) * **getBoundary** () const
- void **setIncludeCellItself** (bool includeCellItself)
- bool **getIncludeCellItself** () const

Protected Member Functions

- [Cell](#) * **getNeighborCell** (std::vector< int > cellPosition, std::vector< std::pair< unsigned short, int > > dimensionChanges)

Protected Attributes

- std::string **name**
- [CellularAutomataBase](#) * **parentCellularAutomata**
- [BoundaryCondition](#) * **boundary**
- unsigned short **radius**
- bool **includeCellItself**

10.338.1 Detailed Description

Defines the complete neighborhood of a cell.

10.338.2 Member Function Documentation

10.338.2.1 getNeighbors()

```
virtual std::vector< Cell * > Neighborhood::getNeighbors (
    Cell * cell ) [pure virtual]
```

pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself.

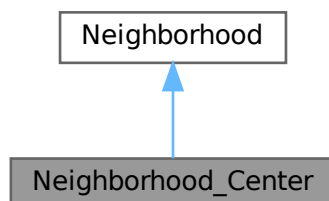
Implemented in [Neighborhood_Center](#), [Neighborhood_Moore](#), and [Neighborhood_VonNeumann](#).

The documentation for this class was generated from the following file:

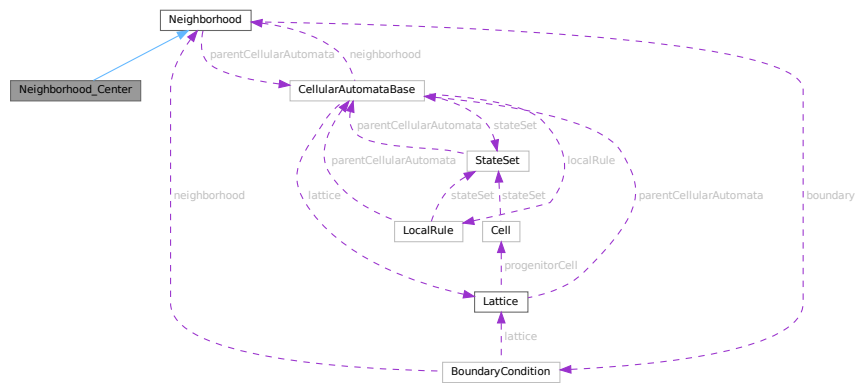
- plugins/components/ModalModel/CellularAutomata/Neighborhood.h

10.339 Neighborhood_Center Class Reference

Inheritance diagram for Neighborhood_Center:



Collaboration diagram for Neighborhood_Center:



Public Member Functions

- **Neighborhood_Center** ([CellularAutomataBase](#) *parentCellularAutomata, unsigned short radius=1, [BoundaryCondition](#) *boundary=nullptr, bool includeCellItself=false)
- **Neighborhood_Center** (const [Neighborhood_Center](#) &orig)
- virtual std::vector< [Cell](#) * > **getNeighbors** ([Cell](#) *cell) override
pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself.

Public Member Functions inherited from [Neighborhood](#)

- **Neighborhood** ([CellularAutomataBase](#) *parentCellularAutomata, unsigned short radius=1, [BoundaryCondition](#) *boundary=nullptr, bool includeCellItself=false)
- **Neighborhood** (const [Neighborhood](#) &orig)
- virtual std::string **show** ()
- void **setRadius** (unsigned short radius)
- unsigned short **getRadius** () const
- void **setBoundary** ([BoundaryCondition](#) *boundary)
- [BoundaryCondition](#) * **getBoundary** () const
- void **setIncludeCellItself** (bool includeCellItself)
- bool **getIncludeCellItself** () const

Additional Inherited Members

Protected Member Functions inherited from [Neighborhood](#)

- [Cell](#) * **getNeighborCell** (std::vector< int > cellPosition, std::vector< std::pair< unsigned short, int > > dimensionChanges)

Protected Attributes inherited from [Neighborhood](#)

- std::string **name**
- [CellularAutomataBase](#) * **parentCellularAutomata**
- [BoundaryCondition](#) * **boundary**
- unsigned short **radius**
- bool **includeCellItself**

10.339.1 Member Function Documentation

10.339.1.1 getNeighbors()

```
virtual std::vector< Cell * > Neighborhood_Center::getNeighbors (
    Cell * cell ) [inline], [override], [virtual]
```

pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself.

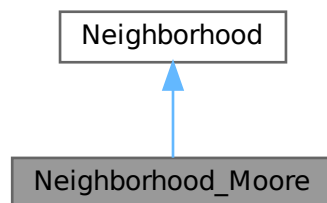
Implements [Neighborhood](#).

The documentation for this class was generated from the following file:

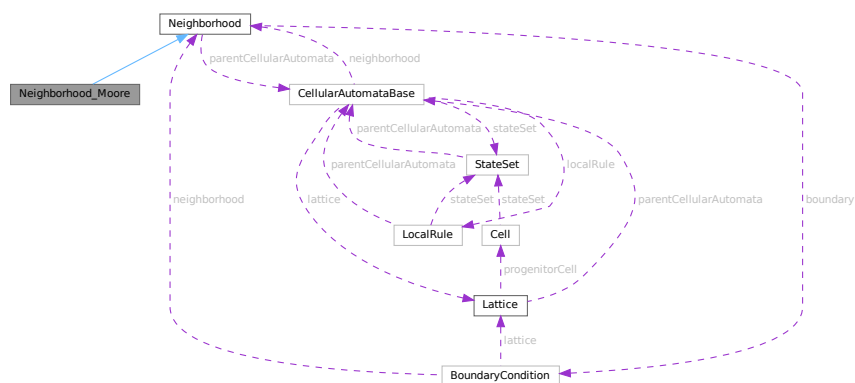
- plugins/components/ModalModel/CellularAutomata/Neighborhood_Center.h

10.340 Neighborhood_Moore Class Reference

Inheritance diagram for Neighborhood_Moore:



Collaboration diagram for Neighborhood_Moore:



Public Member Functions

- **Neighborhood_Moore** ([CellularAutomataBase](#) *parentCellularAutomata, unsigned short radius=1, [BoundaryCondition](#) *boundary=nullptr)
- **Neighborhood_Moore** (const [Neighborhood_Moore](#) &orig)
- virtual std::string [show](#) () override
- virtual std::vector< [Cell](#) * > [getNeighbors](#) ([Cell](#) *cell) override

pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself.

Public Member Functions inherited from [Neighborhood](#)

- **Neighborhood** ([CellularAutomataBase](#) *parentCellularAutomata, unsigned short radius=1, [BoundaryCondition](#) *boundary=nullptr, bool includeCellItself=false)
- **Neighborhood** (const [Neighborhood](#) &orig)
- void **setRadius** (unsigned short radius)
- unsigned short **getRadius** () const
- void **setBoundary** ([BoundaryCondition](#) *boundary)
- [BoundaryCondition](#) * **getBoundary** () const
- void **setIncludeCellItself** (bool includeCellItself)
- bool **getIncludeCellItself** () const

Additional Inherited Members

Protected Member Functions inherited from [Neighborhood](#)

- [Cell](#) * **getNeighborCell** (std::vector< int > cellPosition, std::vector< std::pair< unsigned short, int > > dimensionChanges)

Protected Attributes inherited from [Neighborhood](#)

- std::string **name**
- [CellularAutomataBase](#) * **parentCellularAutomata**
- [BoundaryCondition](#) * **boundary**
- unsigned short **radius**
- bool **includeCellItself**

10.340.1 Member Function Documentation

10.340.1.1 [getNeighbors\(\)](#)

```
virtual std::vector< Cell * > Neighborhood_Moore::getNeighbors (
    Cell * cell ) [inline], [override], [virtual]
```

pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself.

Implements [Neighborhood](#).

Public Member Functions inherited from [Neighborhood](#)

- **Neighborhood** ([CellularAutomataBase](#) *parentCellularAutomata, unsigned short radius=1, [BoundaryCondition](#) *boundary=nullptr, bool includeCellItself=false)
- **Neighborhood** (const [Neighborhood](#) &orig)
- void **setRadius** (unsigned short radius)
- unsigned short **getRadius** () const
- void **setBoundary** ([BoundaryCondition](#) *boundary)
- [BoundaryCondition](#) * **getBoundary** () const
- void **setIncludeCellItself** (bool includeCellItself)
- bool **getIncludeCellItself** () const

Additional Inherited Members

Protected Member Functions inherited from [Neighborhood](#)

- [Cell](#) * **getNeighborCell** (std::vector< int > cellPosition, std::vector< std::pair< unsigned short, int > > dimensionChanges)

Protected Attributes inherited from [Neighborhood](#)

- std::string **name**
- [CellularAutomataBase](#) * **parentCellularAutomata**
- [BoundaryCondition](#) * **boundary**
- unsigned short **radius**
- bool **includeCellItself**

10.341.1 Member Function Documentation

10.341.1.1 getNeighbors()

```
virtual std::vector< Cell * > Neighborhood_VonNeumann::getNeighbors (
    Cell * cell ) [inline], [override], [virtual]
```

pure virtual method that has to be overridden by derived class, responsible for defining the neighborhood itself.

Implements [Neighborhood](#).

10.341.1.2 show()

```
virtual std::string Neighborhood_VonNeumann::show ( ) [inline], [override], [virtual]
```

Reimplemented from [Neighborhood](#).

The documentation for this class was generated from the following file:

- plugins/components/ModalModel/CellularAutomata/Neighborhood_VonNeumann.h

10.342 ParserManager::NewParser Struct Reference

Paths associated with a generated parser artifact set.

```
#include <kernel/simulator/ParserManager.h>
```

Public Attributes

- std::string **bisonFilename**
- std::string **flexFilename**
- std::string **compiledParserFilename**

10.342.1 Detailed Description

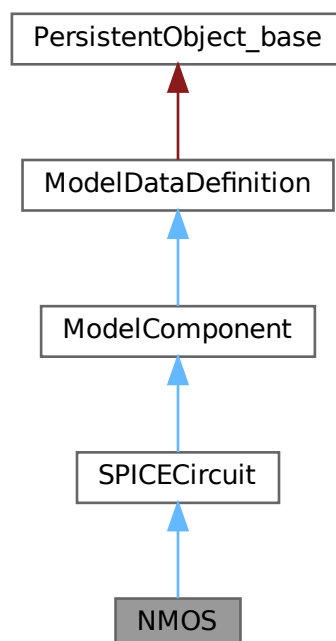
Paths associated with a generated parser artifact set.

The documentation for this struct was generated from the following file:

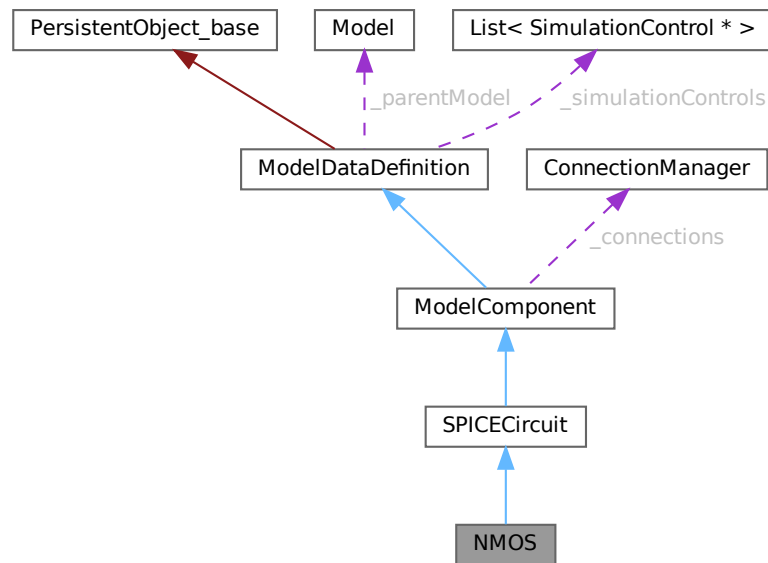
- kernel/simulator/ParserManager.h

10.343 NMOS Class Reference

Inheritance diagram for NMOS:



Collaboration diagram for NMOS:



Public Member Functions

- **NMOS** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setWidth** (float width)
- void **setLength** (float length)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component

- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.
- bool **hasChanged** () const
Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int **getLevel** () const
Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)
Sets the model nesting level for this definition.

- `List< SimulationControl * > * getSimulationControls ()` const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific ()` const
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled ()` const
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation ()`
- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `SPICECircuit`

- static `PluginInformation * GetPluginInformation ()`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`
- static `std::vector< std::string > split (std::string word, char split_char)`
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `bool Check (ModelComponent *component)`
Validates a component instance.
- static `void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- static `void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- static `void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()

- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
 - legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.343.1 Member Function Documentation

10.343.1.1 `_loadInstance()`

```
bool NMOS::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.343.1.2 `_saveInstance()`

```
void NMOS::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.343.1.3 `Build()`

```
void NMOS::Build ( ) [override], [virtual]
```

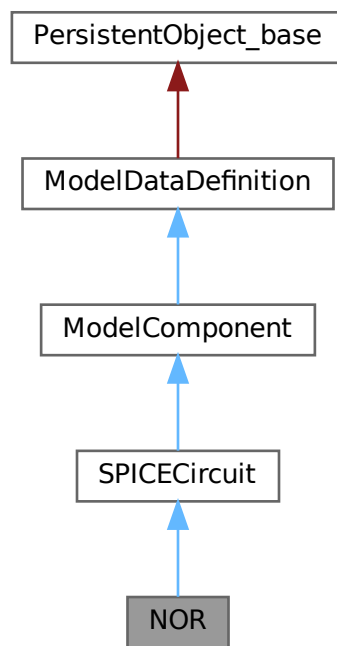
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

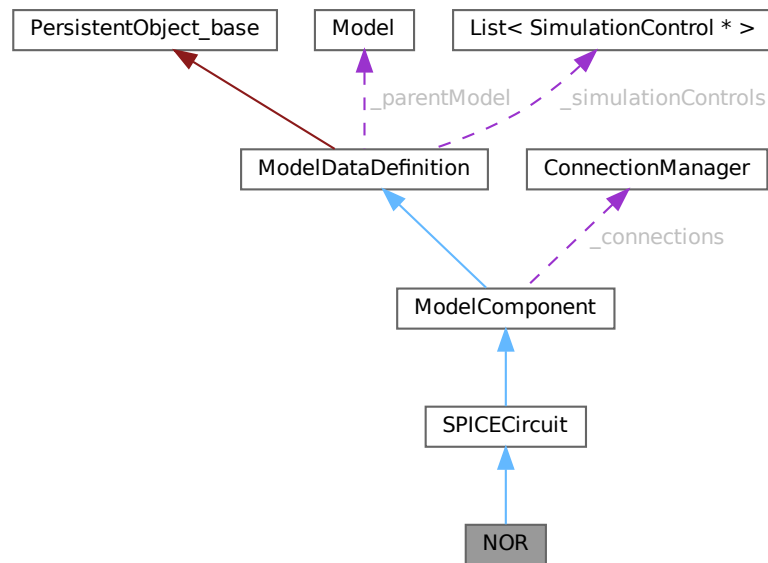
- `plugins/components/ElectronicsSimulation/SPIECEircuit.h`
- `plugins/components/ElectronicsSimulation/SPIECEircuit.cpp`

10.344 NOR Class Reference

Inheritance diagram for NOR:



Collaboration diagram for NOR:



Public Member Functions

- **NOR** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setPMOSWidth** (float PMOSWidth)
- void **setPMOSLength** (float length)
- void **setNMOSWidth** (float NMOSWidth)
- void **setNMOSLength** (float NMOSLength)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.

- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.

- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List< SimulationControl * > * getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()

- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
 - legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * [_connections](#) = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification [_id](#)
- std::string [_typename](#)
- bool [_reportStatistics](#)
- bool [_hasChanged](#)
- unsigned int [_modelLevel](#) = 0
- [Model](#) * [_parentModel](#)
- [List](#)< [SimulationControl](#) * > * [_simulationControls](#) = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level [_traceLevelSpecific](#)
- bool [_traceLevelSpecificEnabled](#) = false

10.344.1 Member Function Documentation

10.344.1.1 [_loadInstance\(\)](#)

```
bool NOR::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.344.1.2 [_saveInstance\(\)](#)

```
void NOR::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.344.1.3 [Build\(\)](#)

```
void NOR::Build ( ) [override], [virtual]
```

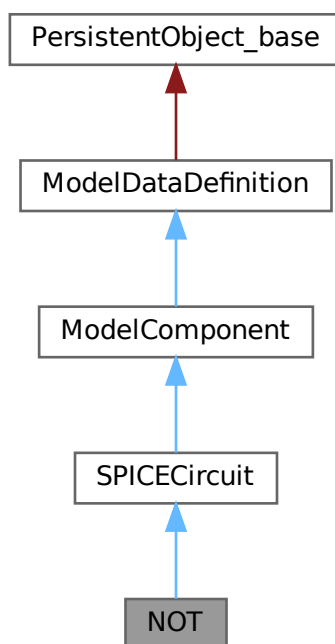
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

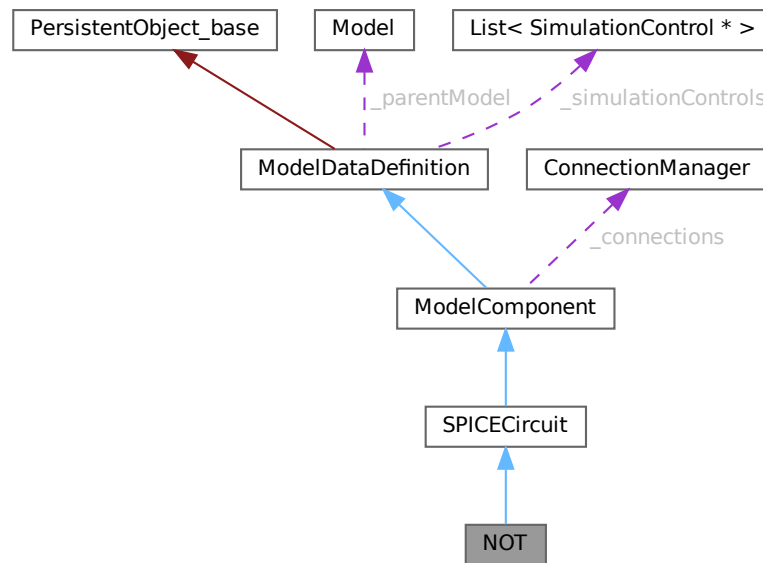
- plugins/components/ElectronicsSimulation/SPIECEircuit.h
- plugins/components/ElectronicsSimulation/SPIECEircuit.cpp

10.345 NOT Class Reference

Inheritance diagram for NOT:



Collaboration diagram for NOT:



Public Member Functions

- **NOT** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setPMOSWidth** (float PMOSWidth)
- void **setPMOSLength** (float length)
- void **setNMOSWidth** (float NMOSWidth)
- void **setNMOSLength** (float NMOSLength)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.

- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.

- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List< SimulationControl * > * getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()

- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
 - legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.345.1 Member Function Documentation

10.345.1.1 `_loadInstance()`

```
bool NOT::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.345.1.2 `_saveInstance()`

```
void NOT::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.345.1.3 `Build()`

```
void NOT::Build ( ) [override], [virtual]
```

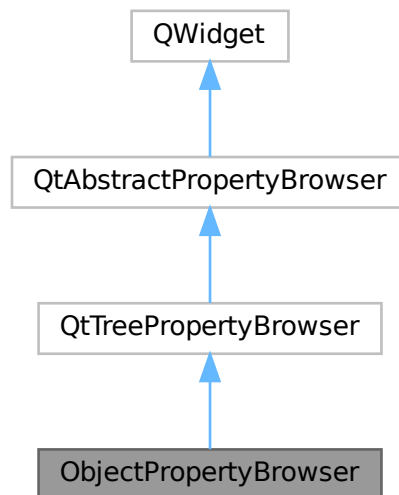
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

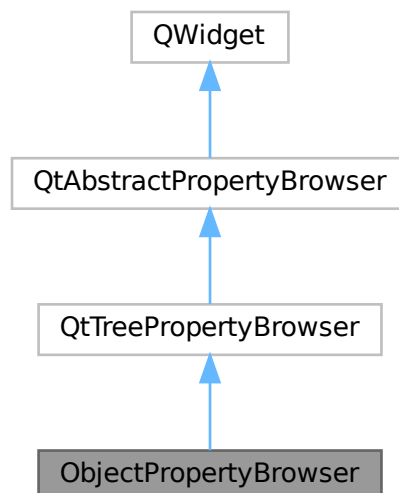
- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

10.346 ObjectPropertyBrowser Class Reference

Inheritance diagram for ObjectPropertyBrowser:



Collaboration diagram for ObjectPropertyBrowser:



Public Types

- using **ModelChangedCallback** = std::function< void()>

Public Types inherited from [QtTreePropertyBrowser](#)

- enum [ResizeMode](#) { **Interactive** , **Stretch** , **Fixed** , **ResizeToContents** }

Public Slots

- void **objectUpdated** ()

Public Slots inherited from [QtAbstractPropertyBrowser](#)

- [QtBrowserItem](#) * **addProperty** ([QtProperty](#) *property)
- [QtBrowserItem](#) * **insertProperty** ([QtProperty](#) *property, [QtProperty](#) *afterProperty)
- void **removeProperty** ([QtProperty](#) *property)

Public Member Functions

- **ObjectPropertyBrowser** (QWidget *parent=nullptr)
- void **setActiveObject** (QObject *obj, [ModelDataDefinition](#) *mdd=nullptr, const QSet< QString > &graphicallyRepresentedModelObjects={}, const QSet< QString > &editableModelObjects={}, [PropertyEditorGenesys](#) *peg=nullptr, std::map< [SimulationControl](#) *, [DataComponentProperty](#) * > *pl=nullptr, std::map< [SimulationControl](#) *, [DataComponentEditor](#) * > *peUI=nullptr, std::map< [SimulationControl](#) *, [ComboBoxEnum](#) * > *pb=nullptr)
- void **setActiveGraphicsItem** (QGraphicsItem *item)
- void **clearCurrentlyConnectedObject** ()
- void **setModelChangedCallback** (ModelChangedCallback callback)
- bool **isCommitPipelineBusy** () const

Public Member Functions inherited from [QtTreePropertyBrowser](#)

- [QtTreePropertyBrowser](#) (QWidget *parent=0)
- ~[QtTreePropertyBrowser](#) ()
- int **indentation** () const
- void **setIndentation** (int i)
- bool **rootIsDecorated** () const
- void **setRootIsDecorated** (bool show)
- bool **alternatingRowColors** () const
- void **setAlternatingRowColors** (bool enable)
- bool **isHeaderVisible** () const
- void **setHeaderVisible** (bool visible)
- [ResizeMode](#) **resizeMode** () const
- void **setResizeMode** ([ResizeMode](#) mode)
- int **splitterPosition** () const
- void **setSplitterPosition** (int position)
- void **setExpanded** ([QtBrowserItem](#) *item, bool **expanded**)
- bool **isExpanded** ([QtBrowserItem](#) *item) const
- bool **isItemVisible** ([QtBrowserItem](#) *item) const
- void **setItemVisible** ([QtBrowserItem](#) *item, bool visible)
- void **setBackgroundColor** ([QtBrowserItem](#) *item, const QColor &color)
- QColor **backgroundColor** ([QtBrowserItem](#) *item) const
- QColor **calculatedBackgroundColor** ([QtBrowserItem](#) *item) const
- void **setPropertyWithoutValueMarked** (bool mark)
- bool **propertiesWithoutValueMarked** () const
- void **editItem** ([QtBrowserItem](#) *item)

Public Member Functions inherited from [QtAbstractPropertyBrowser](#)

- [QtAbstractPropertyBrowser](#) (QWidget *parent=0)
- [~QtAbstractPropertyBrowser](#) ()
- QList< [QtProperty](#) * > [properties](#) () const
- QList< [QtBrowserItem](#) * > [items](#) ([QtProperty](#) *property) const
- [QtBrowserItem](#) * [topLevelItem](#) ([QtProperty](#) *property) const
- QList< [QtBrowserItem](#) * > [topLevelItems](#) () const
- void [clear](#) ()
- template<class PropertyManager >
void [setFactoryForManager](#) (PropertyManager *manager, [QtAbstractEditorFactory](#)< PropertyManager > *factory)
- void [unsetFactoryForManager](#) ([QtAbstractPropertyManager](#) *manager)
- [QtBrowserItem](#) * [currentItem](#) () const
- void [setCurrentItem](#) ([QtBrowserItem](#) *)

Protected Member Functions

- void [keyPressEvent](#) (QKeyEvent *event) override
- void [contextMenuEvent](#) (QContextMenuEvent *event) override
- void [mouseDoubleClickEvent](#) (QMouseEvent *event) override

Protected Member Functions inherited from [QtTreePropertyBrowser](#)

- void [itemInserted](#) ([QtBrowserItem](#) *item, [QtBrowserItem](#) *afterItem) override
- void [itemRemoved](#) ([QtBrowserItem](#) *item) override
- void [itemChanged](#) ([QtBrowserItem](#) *item) override

Protected Member Functions inherited from [QtAbstractPropertyBrowser](#)

- virtual QWidget * [createEditor](#) ([QtProperty](#) *property, QWidget *parent)

Additional Inherited Members

Signals inherited from [QtTreePropertyBrowser](#)

- void [collapsed](#) ([QtBrowserItem](#) *item)
- void [expanded](#) ([QtBrowserItem](#) *item)

Signals inherited from [QtAbstractPropertyBrowser](#)

- void [currentItemChanged](#) ([QtBrowserItem](#) *)

Properties inherited from [QtTreePropertyBrowser](#)

- int **indentation**
indentation of the items in the tree view.
- bool **rootIsDecorated**
whether to show controls for expanding and collapsing root items.
- bool **alternatingRowColors**
whether to draw the background using alternating colors. By default this property is set to true.
- bool **headerVisible**
whether to show the header.
- [ResizeMode](#) **resizeMode**
the resize mode of sections in the header.
- int **splitterPosition**
the position of the splitter between the columns.
- bool [propertiesWithoutValueMarked](#)
whether to enable or disable marking properties without value.

The documentation for this class was generated from the following files:

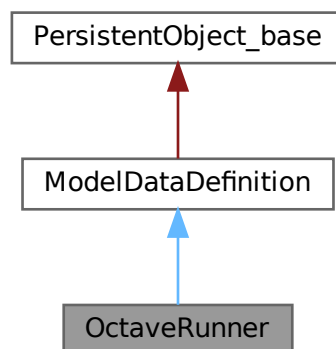
- applications/gui/genesys/propertyeditor/ObjectPropertyBrowser.h
- applications/gui/genesys/propertyeditor/ObjectPropertyBrowser.cpp

10.347 OctaveRunner Class Reference

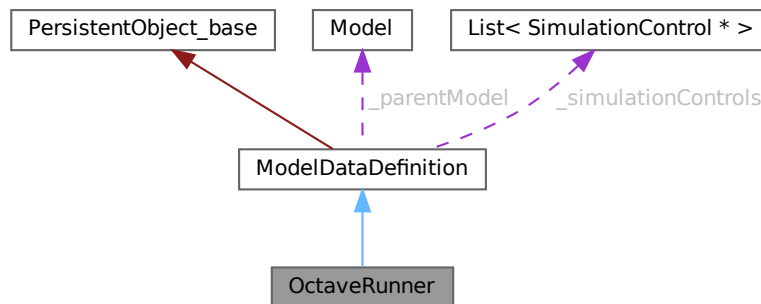
[Model](#) data definition that executes Octave code through Rscript.

```
#include <plugins/data/ExternalIntegration/OctaveRunner.h>
```

Inheritance diagram for OctaveRunner:



Collaboration diagram for OctaveRunner:



Public Member Functions

- **OctaveRunner** ([Model](#) *model, std::string name="")
Creates an Octave Runner bound to a model.
- virtual **~OctaveRunner** ()=default
Destroys the Octave Runner.
- void **setOctaveExecutable** (std::string octaveExecutable)
Sets the Rscript executable name or path.
- std::string **getOctaveExecutable** () const
Returns the Rscript executable name or path.
- void **setWorkingDirectory** (std::string workingDirectory)
Sets the working directory used for generated scripts and command execution.
- std::string **getWorkingDirectory** () const
Returns the working directory used for generated scripts and command execution.
- void **setPreludeScript** (std::string preludeScript)
Sets optional Octave code prepended before the command block.
- std::string **getPreludeScript** () const
Returns optional Octave code prepended before the command block.
- void **setCommand** (std::string command)
Sets the Octave command/script body to execute.
- std::string **getCommand** () const
Returns the Octave command/script body to execute.
- void **setLastStatus** (std::string lastStatus)
Sets the last execution status.
- std::string **getLastStatus** () const
Returns the last execution status.
- void **setLastExitCode** (int lastExitCode)
Sets the last process exit code.
- int **getLastExitCode** () const
Returns the last process exit code.
- void **setLastStdout** (std::string lastStdout)
Sets stdout captured from the last Octave execution.
- std::string **getLastStdout** () const
Returns stdout captured from the last Octave execution.

- void **setLastStderr** (std::string lastStderr)
Sets stderr captured from the last Octave execution.
- std::string **getLastStderr** () const
Returns stderr captured from the last Octave execution.
- void **setLastResponsePayload** (std::string lastResponsePayload)
Sets the response payload exposed to other tools.
- std::string **getLastResponsePayload** () const
Returns the response payload exposed to other tools.
- void **setLastScriptFilename** (std::string lastScriptFilename)
Sets the generated Octave script filename from the last execution.
- std::string **getLastScriptFilename** () const
Returns the generated Octave script filename from the last execution.
- void **setLastResponseFilename** (std::string lastResponseFilename)
Sets the stdout file path from the last execution.
- std::string **getLastResponseFilename** () const
Returns the stdout file path from the last execution.
- bool **executeCommand** (std::string &errorMessage)
Executes the configured Octave code and updates last-execution fields atomically.
- virtual std::string **show** () override
Returns a compact human-readable description of the runner.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const
Returns the full map of internal child data definitions.

- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
Loads an Octave Runner from a persistence record.
- `static PluginInformation * GetPluginInformation ()`
Returns plugin metadata and system dependency declarations.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new Octave Runner instance for plugin construction.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
Loads persisted Octave Runner configuration and last-execution state.
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves Octave Runner configuration and last-execution state.
- virtual bool `_check` (std::string &errorMessage) override
Validates the runner configuration without executing Octave.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.347.1 Detailed Description

[Model](#) data definition that executes Octave code through Rscript.

The runner stores a compact Octave execution configuration, writes a temporary script, executes it with Rscript, and keeps status, exit code, stdout, stderr and response payload inside the model data definition for later inspection.

10.347.2 Member Function Documentation

10.347.2.1 [_check\(\)](#)

```
bool OctaveRunner::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

Validates the runner configuration without executing Octave.

Reimplemented from [ModelDataDefinition](#).

10.347.2.2 `_loadInstance()`

```
bool OctaveRunner::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

Loads persisted Octave Runner configuration and last-execution state.

Reimplemented from [ModelDataDefinition](#).

10.347.2.3 `_saveInstance()`

```
void OctaveRunner::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves Octave Runner configuration and last-execution state.

Reimplemented from [ModelDataDefinition](#).

10.347.2.4 `show()`

```
std::string OctaveRunner::show ( ) [override], [virtual]
```

Returns a compact human-readable description of the runner.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- `plugins/data/ExternalIntegration/OctaveRunner.h`
- `plugins/data/ExternalIntegration/OctaveRunner.cpp`

10.348 ODEfunction Class Reference**Public Member Functions**

- **ODEfunction** (std::string expression, double initialPoint, double initialValue)

Public Attributes

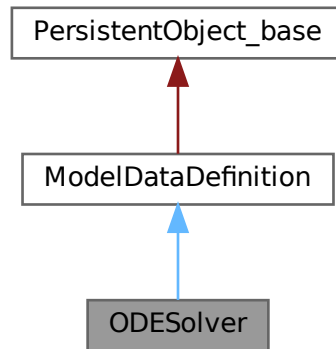
- std::string **expression**
- double **initialPoint**
- double **initialValue**

The documentation for this class was generated from the following file:

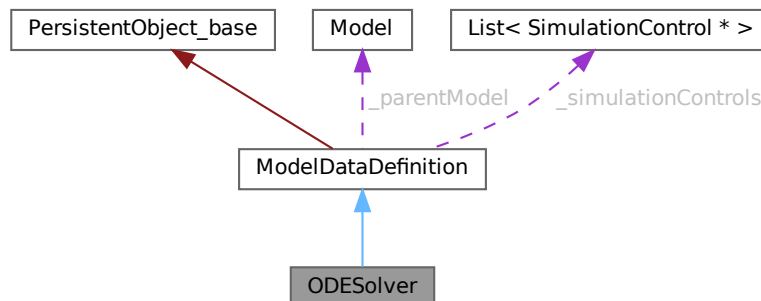
- `plugins/components/Continuous/OLD_ODEelement.h`

10.349 ODESolver Class Reference

Inheritance diagram for ODESolver:



Collaboration diagram for ODESolver:



Public Member Functions

- **ODESolver** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **setTimeVariableName** (std::string timeVariableName)
- std::string **getTimeVariableName** () const
- void **setStateVariableNames** (std::vector< std::string > stateVariableNames)
- std::vector< std::string > **getStateVariableNames** () const
- void **setEquationExpressions** (std::vector< std::string > equationExpressions)
- std::vector< std::string > **getEquationExpressions** () const
- void **setStep** (double step)

- double **getStep** () const
- void **setPrecision** (double precision)
- double **getPrecision** () const
- void **setMaxSteps** (int maxSteps)
- int **getMaxSteps** () const
- void **setCurrentTime** (double currentTime)
- double **getCurrentTime** () const
- void **setStateValues** (std::vector< double > stateValues)
- std::vector< double > **getStateValues** () const
- void **setInitialStateValues** (std::vector< double > initialStateValues)
- std::vector< double > **getInitialStateValues** () const
- void **setAutoAdvance** (bool autoAdvance)
- bool **isAutoAdvance** () const
- void **setOutputFile** (std::string outputFile)
- std::string **getOutputFile** () const
- void **integrate** (double targetTime)
- void **resetState** ()
- double **getStateValue** (unsigned int index) const
- void **setStateValue** (unsigned int index, double value)
- void **synchronizeStateFromVariables** ()
- void **writeStateToVariables** () const

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const

- Returns the full map of attached data definitions.*
- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- List< `SimulationControl` * > * `getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- TraceManager::Level `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (TraceManager::Level `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (TraceManager::Level `traceLevelSpecific`, bool `traceLevelSpecificEnabled`=true)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `ModelDataDefinition` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`)
- static `PluginInformation` * `GetPluginInformation` ()
- static `ModelDataDefinition` * `NewInstance` (`Model` *`model`, std::string `name`="")
- static void `SaveInstance` (`PersistenceRecord` *`fields`, `ModelDataDefinition` *`dataDef`)
- static bool `Check` (`ModelDataDefinition` *`dataDef`, std::string &`errorMessage`)
- static void `InitBetweenReplications` (`ModelDataDefinition` *`dataDef`)
- static void `CreateInternalData` (`ModelDataDefinition` *`dataDef`)

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`, bool `insertIntoModel`)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * `NewInstance` (`Model` *`model`, std::string `name`="")
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord` *`fields`, `ModelDataDefinition` *`modeldatum`)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition` *`modeldatum`, std::string &`errorMessage`)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition` *`modeldatum`)
Resets simulation-time state between replications.
- static void `CreateInternalData` (`ModelDataDefinition` *`modeldatum`)
Creates internal and attached data required by this instance.
- static bool `CreateRelatedDataElements` (`ModelDataDefinition` *`modeldatum`, std::string &`errorMessage`)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_initBetweenReplications` () override
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.349.1 Member Function Documentation

10.349.1.1 **_check()**

```
bool ODESolver::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.349.1.2 **_createEditableDataDefinitions()**

```
void ODESolver::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.349.1.3 **_initBetweenReplications()**

```
void ODESolver::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.349.1.4 `_loadInstance()`

```
bool ODESolver::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.349.1.5 `_saveInstance()`

```
void ODESolver::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.349.1.6 `show()`

```
std::string ODESolver::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

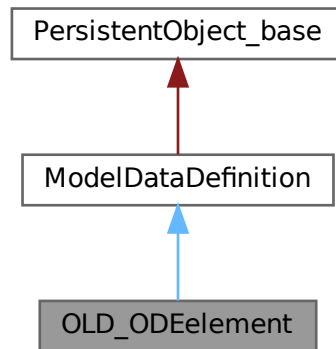
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

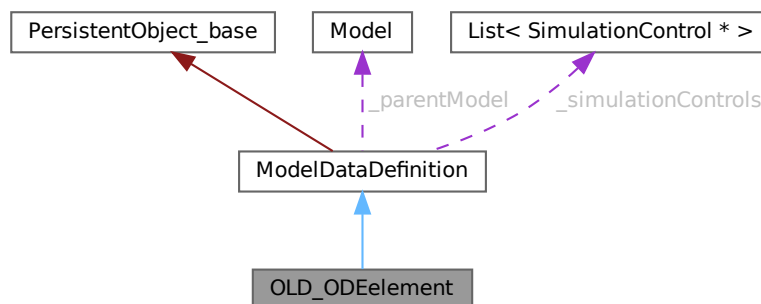
- plugins/data/Continuous/ODESolver.h
- plugins/data/Continuous/ODESolver.cpp

10.350 OLD_ODEelement Class Reference

Inheritance diagram for OLD_ODEelement:



Collaboration diagram for OLD_ODEelement:



Public Member Functions

- **OLD_ODEelement** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- double **solve** ()
- void **setStepH** (double _h)
- double **getStepH** () const
- void **setEndTime** (double _endTime)
- double **getEndTime** () const
- [List< ODEfunction * >](#) * **getODEfunctions** () const
- void **addODEfunction** ([ODEfunction](#) *newFunction)
- void **removeODEfunction** ([ODEfunction](#) *function)

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_initBetweenReplications** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createNonEditableDataDefinitions** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()

- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.350.1 Member Function Documentation

10.350.1.1 `_check()`

```
bool OLD_ODElement::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.350.1.2 `_loadInstance()`

```
bool OLD_ODElement::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.350.1.3 `_saveInstance()`

```
void OLD_ODElement::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.350.1.4 `show()`

```
std::string OLD_ODElement::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/components/Continuous/OLD_ODElement.h
- plugins/components/Continuous/OLD_ODElement.cpp

10.351 OnEventManager Class Reference

```
#include <kernel/simulator/OnEventManager.h>
```

Public Member Functions

- void **addOnModelCheckSucessHandler** (modelEventHandler EventHandler)
- void **addOnModelLoadHandler** (modelEventHandler EventHandler)
- void **addOnModelSaveHandler** (modelEventHandler EventHandler)
- void **addOnReplicationStartHandler** (simulationEventHandler EventHandler)
- void **addOnReplicationStepHandler** (simulationEventHandler EventHandler)
- void **addOnReplicationEndHandler** (simulationEventHandler EventHandler)
- void **addOnProcessEventHandler** (simulationEventHandler EventHandler)
- void **addOnAfterProcessEventHandler** (simulationEventHandler EventHandler)
- void **addOnEntityCreateHandler** (simulationEventHandler EventHandler)
- void **addOnEntityMoveHandler** (simulationEventHandler EventHandler)
- void **addOnEntityRemoveHandler** (simulationEventHandler EventHandler)
- void **addOnSimulationStartHandler** (simulationEventHandler EventHandler)
- void **addOnSimulationPausedHandler** (simulationEventHandler EventHandler)
- void **addOnSimulationResumeHandler** (simulationEventHandler EventHandler)
- void **addOnSimulationEndHandler** (simulationEventHandler EventHandler)
- void **addOnBreakpointHandler** (simulationEventHandler EventHandler)
- template<typename Class >
void **addOnModelCheckSuccessHandler** (Class *object, void(Class::*function)(ModelEvent *))
- template<typename Class >
void **addOnModelLoadHandler** (Class *object, void(Class::*function)(ModelEvent *))
- template<typename Class >
void **addOnModelSaveHandler** (Class *object, void(Class::*function)(ModelEvent *))
- template<typename Class >
void **addOnReplicationStartHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnReplicationStepHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnReplicationEndHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnProcessEventHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnAfterProcessEventHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnEntityCreateHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnEntityMoveHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnEntityRemoveHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnSimulationStartHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnSimulationPausedHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnSimulationResumeHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnSimulationEndHandler** (Class *object, void(Class::*function)(SimulationEvent *))
- template<typename Class >
void **addOnBreakpointHandler** (Class *object, void(Class::*function)(SimulationEvent *))

- void **NotifyModelCheckSuccessHandlers** ([ModelEvent](#) *se)
- void **NotifyModelLoadHandlers** ([ModelEvent](#) *se)
- void **NotifyModelSaveHandlers** ([ModelEvent](#) *se)
- void **NotifyReplicationStartHandlers** ([SimulationEvent](#) *se)
- void **NotifyReplicationStepHandlers** ([SimulationEvent](#) *se)
- void **NotifyReplicationEndHandlers** ([SimulationEvent](#) *se)
- void **NotifyProcessEventHandlers** ([SimulationEvent](#) *se)
- void **NotifyAfterProcessEventHandlers** ([SimulationEvent](#) *se)
- void **NotifyEntityCreateHandlers** ([SimulationEvent](#) *se)
- void **NotifyEntityMoveHandlers** ([SimulationEvent](#) *se)
- void **NotifyEntityRemoveHandlers** ([SimulationEvent](#) *se)
- void **NotifySimulationStartHandlers** ([SimulationEvent](#) *se)
- void **NotifySimulationPausedHandlers** ([SimulationEvent](#) *se)
- void **NotifySimulationResumeHandlers** ([SimulationEvent](#) *se)
- void **NotifySimulationEndHandlers** ([SimulationEvent](#) *se)
- void **NotifyBreakpointHandlers** ([SimulationEvent](#) *se)

10.351.1 Detailed Description

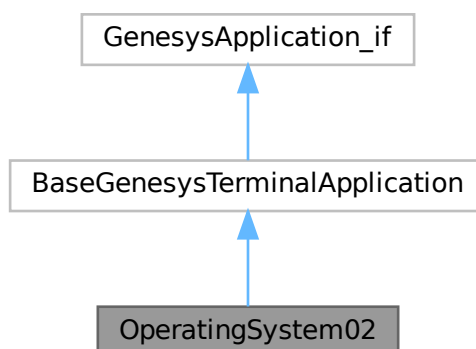
[OnEventManager](#) allows external methods to hook interval simulation events as listeners (or observers) of specific events. All methods added as listeners of an event will be invoked when that event is triggered.

The documentation for this class was generated from the following files:

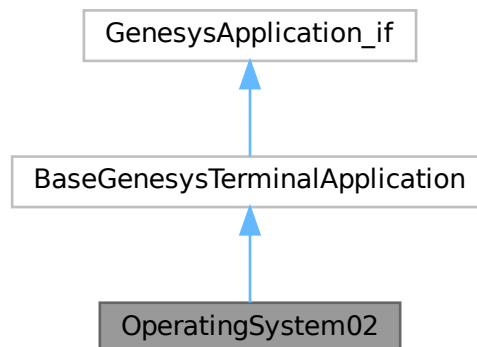
- kernel/simulator/OnEventManager.h
- kernel/simulator/OnEventManager.cpp

10.352 OperatingSystem02 Class Reference

Inheritance diagram for OperatingSystem02:



Collaboration diagram for OperatingSystem02:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.352.1 Member Function Documentation

10.352.1.1 main()

```
int OperatingSystem02::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

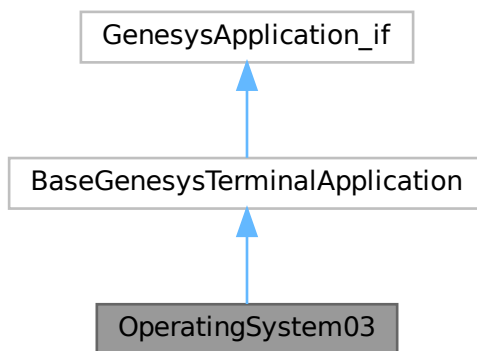
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

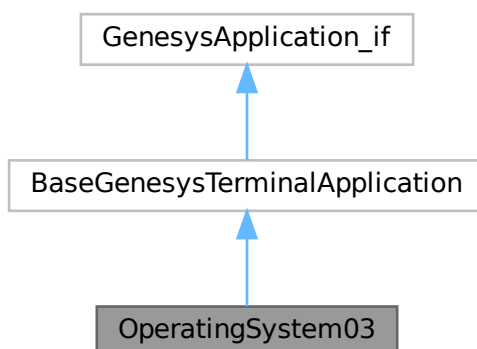
- applications/modelSpecific/teaching/OperatingSystem02.h
- applications/modelSpecific/teaching/OperatingSystem02.cpp

10.353 OperatingSystem03 Class Reference

Inheritance diagram for OperatingSystem03:



Collaboration diagram for OperatingSystem03:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.353.1 Member Function Documentation

10.353.1.1 [main\(\)](#)

```
int OperatingSystem03::main (
    int argc,
    char ** argv )  [override], [virtual]
```

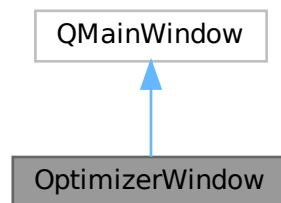
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

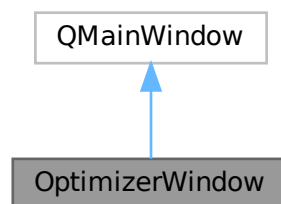
- applications/modelSpecific/teaching/OperatingSystem03.h
- applications/modelSpecific/teaching/OperatingSystem03.cpp

10.354 OptimizerWindow Class Reference

Inheritance diagram for OptimizerWindow:



Collaboration diagram for OptimizerWindow:



Classes

- class [ProgressPlot](#)

Public Member Functions

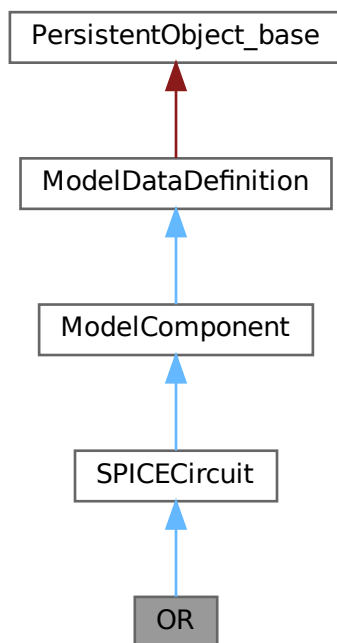
- **OptimizerWindow** ([Simulator](#) *simulator, QWidget *parent=nullptr)

The documentation for this class was generated from the following files:

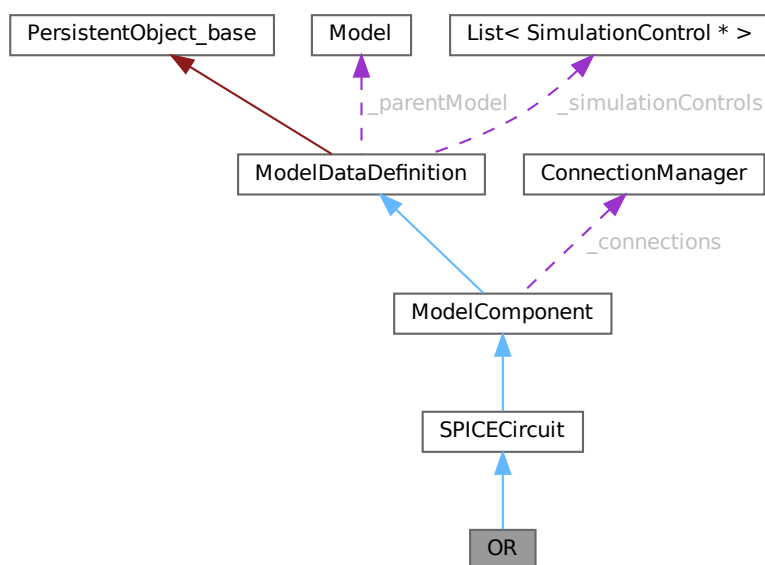
- applications/gui/genesys/tools/optimizer/OptimizerWindow.h
- applications/gui/genesys/tools/optimizer/OptimizerWindow.cpp

10.355 OR Class Reference

Inheritance diagram for OR:



Collaboration diagram for OR:



Public Member Functions

- **OR** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setPMOSWidth** (float PMOSWidth)
- void **setPMOSLength** (float length)
- void **setNMOSWidth** (float NMOSWidth)
- void **setNMOSLength** (float NMOSLength)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

- Updates the logical name of this instance.*
- `const std::string & getName () const`
Returns the logical name of this instance.
- `void setLabel (const std::string &label)`
Updates the user-visible label of this instance.
- `const std::string & getLabel () const`
Returns the user-visible label of this instance.
- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) * **GetPluginInformation** ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
- static std::vector< std::string > **split** (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void **_createNonEditableDataDefinitions** () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual bool **_check** (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_initBetweenReplications** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)

- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.355.1 Member Function Documentation

10.355.1.1 [_loadInstance\(\)](#)

```
bool OR::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.355.1.2 `_saveInstance()`

```
void OR::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.355.1.3 Build()

```
void OR::Build ( ) [override], [virtual]
```

Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

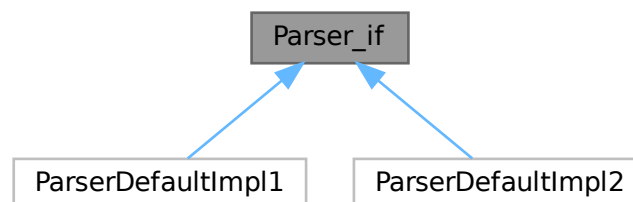
- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

10.356 Parser_if Class Reference

Interface for evaluating model expressions and stochastic functions.

```
#include <kernel/simulator/Parser_if.h>
```

Inheritance diagram for Parser_if:

**Public Member Functions**

- virtual double [parse](#) (const std::string expression)=0
parse
- virtual double [parse](#) (const std::string expression, bool &success, std::string &errorMessage)=0
parse
- virtual std::string [getErrorMessage](#) ()=0
getErrorMessage
- virtual void [setSampler](#) ([Sampler_if](#) *sampler)=0
Sets the sampler used by stochastic language functions.
- virtual [Sampler_if](#) * [getSampler](#) () const =0
Returns the sampler currently associated with the parser.
- virtual genesyspp_driver [getParser](#) () const =0
getParser

10.356.1 Detailed Description

Interface for evaluating model expressions and stochastic functions.

Parser implementations are used by model components/data definitions to evaluate arithmetic/logical expressions, integrating deterministic symbols and stochastic calls backed by a [Sampler_if](#) instance.

10.356.2 Member Function Documentation

10.356.2.1 `getErrorMessage()`

```
virtual std::string Parser_if::getErrorMessage ( ) [pure virtual]
```

`getErrorMessage`

Returns

Returns the last error message produced by the parser.

Implemented in [ParserDefaultImpl1](#), and [ParserDefaultImpl2](#).

10.356.2.2 `getParser()`

```
virtual genesyspp_driver Parser_if::getParser ( ) const [pure virtual]
```

`getParser`

Returns

Exposes the internal parser driver for advanced integration scenarios.

Implemented in [ParserDefaultImpl2](#).

10.356.2.3 `getSampler()`

```
virtual Sampler\_if * Parser_if::getSampler ( ) const [pure virtual]
```

Returns the sampler currently associated with the parser.

The returned pointer is non-owning. Callers must consult concrete parser documentation for lifetime/ownership details of the underlying instance.

Implemented in [ParserDefaultImpl2](#).

10.356.2.4 `parse()` [1/2]

```
virtual double Parser_if::parse (
    const std::string expression ) [pure virtual]
```

`parse`

Parameters

<i>expression</i>	
-------------------	--

Returns

Evaluates an expression and returns its numeric value; may throw on syntax/semantic errors.

Implemented in [ParserDefaultImpl1](#), and [ParserDefaultImpl2](#).

10.356.2.5 parse() [2/2]

```
virtual double Parser_if::parse (
    const std::string expression,
    bool & success,
    std::string & errorMessage ) [pure virtual]
```

parse

Parameters

<i>expression</i>	
<i>success</i>	
<i>errorMessage</i>	

Returns

Evaluates an expression without throwing and reports status via output parameters.

Implemented in [ParserDefaultImpl1](#), and [ParserDefaultImpl2](#).

10.356.2.6 setSampler()

```
virtual void Parser_if::setSampler (
    Sampler\_if * sampler ) [pure virtual]
```

Sets the sampler used by stochastic language functions.

Parameters

<i>sampler</i>	Pointer to sampler instance that must outlive parser usage unless an implementation-specific ownership transfer is explicitly documented.
----------------	---

This interface does not enforce ownership transfer. Concrete parser implementations may own a sampler created/provided at construction time, but samplers passed through `setSampler` are expected to be externally

owned by default unless documented otherwise by the implementation.

Implemented in [ParserDefaultImpl2](#).

The documentation for this class was generated from the following file:

- kernel/simulator/Parser_if.h

10.357 ParserChangesInformation Class Reference

Public Member Functions

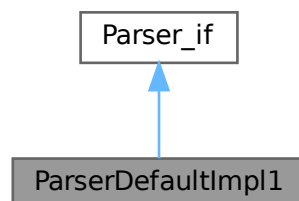
- `std::string` **getincludes** () const
- `void` **setincludes** (const `std::string` &newIncludes)
- `std::string` **gettokens** () const
- `void` **setTokens** (const `std::string` &newTokens)
- `std::string` **gettypeObjs** () const
- `void` **setTypeObjs** (const `std::string` &newTypeObjs)
- `std::string` **getexpressions** () const
- `void` **setExpressions** (const `std::string` &newExpressions)
- `std::string` **getexpressionProductions** () const
- `void` **setExpressionProductions** (const `std::string` &newExpressionProductions)
- `std::string` **getassignments** () const
- `void` **setAssignments** (const `std::string` &newAssignments)
- `std::string` **getfunctionProductions** () const
- `void` **setFunctionProductions** (const `std::string` &newFunctionProductions)

The documentation for this class was generated from the following files:

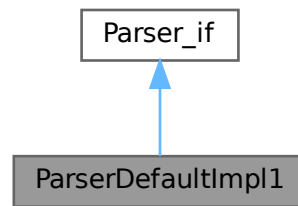
- kernel/simulator/ParserChangesInformation.h
- kernel/simulator/ParserChangesInformation.cpp

10.358 ParserDefaultImpl1 Class Reference

Inheritance diagram for ParserDefaultImpl1:



Collaboration diagram for ParserDefaultImpl1:



Public Member Functions

- **ParserDefaultImpl1** ([Model](#) *model)
- virtual double [parse](#) (const std::string expression) override
parse
- virtual double [parse](#) (const std::string expression, bool &success, std::string &errorMessage) override
parse
- virtual std::string [getErrorMessage](#) () override
getErrorMessage

Public Member Functions inherited from [Parser_if](#)

- virtual void [setSampler](#) ([Sampler_if](#) *sampler)=0
Sets the sampler used by stochastic language functions.
- virtual [Sampler_if](#) * [getSampler](#) () const =0
Returns the sampler currently associated with the parser.
- virtual genesyspp_driver [getParser](#) () const =0
getParser

10.358.1 Member Function Documentation

10.358.1.1 [getErrorMessage\(\)](#)

```
std::string ParserDefaultImpl1::getErrorMessage ( ) [override], [virtual]
```

[getErrorMessage](#)

Returns

Returns the last error message produced by the parser.

Implements [Parser_if](#).

10.358.1.2 [parse\(\)](#) [1/2]

```
double ParserDefaultImpl1::parse (
    const std::string expression ) [override], [virtual]
```

[parse](#)

Parameters

<i>expression</i>	
-------------------	--

Returns

Evaluates an expression and returns its numeric value; may throw on syntax/semantic errors.

Implements [Parser_if](#).

10.358.1.3 parse() [2/2]

```
double ParserDefaultImpl1::parse (
    const std::string expression,
    bool & success,
    std::string & errorMessage ) [override], [virtual]
```

parse**Parameters**

<i>expression</i>	
<i>success</i>	
<i>errorMessage</i>	

Returns

Evaluates an expression without throwing and reports status via output parameters.

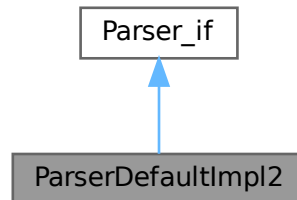
Implements [Parser_if](#).

The documentation for this class was generated from the following files:

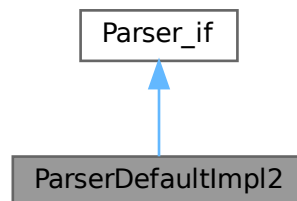
- kernel/simulator/ParserDefaultImpl1.h
- kernel/simulator/ParserDefaultImpl1.cpp

10.359 ParserDefaultImpl2 Class Reference

Inheritance diagram for ParserDefaultImpl2:



Collaboration diagram for ParserDefaultImpl2:



Public Member Functions

- `ParserDefaultImpl2` (`Model` *model, `Sampler_if` *sampler, bool throws=false)
Creates parser wrapper bound to a model and an initial sampler.
- virtual `~ParserDefaultImpl2` ()
Destroys parser wrapper and releases only parser-owned sampler.
- virtual double `parse` (const std::string expression) override
parse
- virtual double `parse` (const std::string expression, bool &success, std::string &errorMessage) override
parse
- virtual std::string `getErrorMessage` () override
getErrorMessage
- virtual void `setSampler` (`Sampler_if` * _sampler) override
Replaces parser sampler with an externally-owned sampler.
- virtual `Sampler_if` * `getSampler` () const override
Returns the sampler currently associated with the parser.
- virtual genesyspp_driver `getParser` () const override
getParser

10.359.1 Constructor & Destructor Documentation

10.359.1.1 ParserDefaultImpl2()

```
ParserDefaultImpl2::ParserDefaultImpl2 (
    Model * model,
    Sampler_if * sampler,
    bool throws = false )
```

Creates parser wrapper bound to a model and an initial sampler.

The constructor stores `sampler` in the internal driver and marks it as parser-owned when non-null. This preserves current model behavior where the parser starts with an internally managed sampler instance.

10.359.1.2 ~ParserDefaultImpl2()

```
ParserDefaultImpl2::~~ParserDefaultImpl2 ( ) [virtual]
```

Destroys parser wrapper and releases only parser-owned sampler.

The destructor deletes the current sampler only when ownership is flagged as internal. Samplers installed through `setSampler(...)` are non-owned by default and therefore not deleted at destruction time.

10.359.2 Member Function Documentation

10.359.2.1 getErrorMessage()

```
std::string ParserDefaultImpl2::getErrorMessage ( ) [override], [virtual]
```

`getErrorMessage`

Returns

Returns the last error message produced by the parser.

Implements [Parser_if](#).

10.359.2.2 getParser()

```
genesyspp_driver ParserDefaultImpl2::getParser ( ) const [override], [virtual]
```

`getParser`

Returns

Exposes the internal parser driver for advanced integration scenarios.

Implements [Parser_if](#).

10.359.2.3 getSampler()

```
Sampler_if * ParserDefaultImpl2::getSampler ( ) const [override], [virtual]
```

Returns the sampler currently associated with the parser.

The returned pointer is non-owning. Callers must consult concrete parser documentation for lifetime/ownership details of the underlying instance.

Implements [Parser_if](#).

10.359.2.4 parse() [1/2]

```
double ParserDefaultImpl2::parse (
    const std::string expression ) [override], [virtual]
```

parse

Parameters

<i>expression</i>	
-------------------	--

Returns

Evaluates an expression and returns its numeric value; may throw on syntax/semantic errors.

Implements [Parser_if](#).

10.359.2.5 parse() [2/2]

```
double ParserDefaultImpl2::parse (
    const std::string expression,
    bool & success,
    std::string & errorMessage ) [override], [virtual]
```

parse

Parameters

<i>expression</i>	
<i>success</i>	
<i>errorMessage</i>	

Returns

Evaluates an expression without throwing and reports status via output parameters.

Implements [Parser_if](#).

10.359.2.6 setSampler()

```
void ParserDefaultImpl2::setSampler (
    Sampler\_if * _sampler ) [override], [virtual]
```

Replaces parser sampler with an externally-owned sampler.

If the parser currently owns a previous sampler, that previous sampler is released before binding the new pointer. The provided sampler pointer is then tracked as non-owned.

Implements [Parser_if](#).

The documentation for this class was generated from the following files:

- kernel/simulator/ParserDefaultImpl2.h
- kernel/simulator/ParserDefaultImpl2.cpp

10.360 ParserManager Class Reference

Classes

- struct [GenerateNewParserResult](#)
Result bundle for parser generation requests.
- struct [NewParser](#)
Paths associated with a generated parser artifact set.

Public Member Functions

- [ParserManager::GenerateNewParserResult](#) generateNewParser ([ParserChangesInformation](#) *changes)
generateNewParser
- bool [connectNewParser](#) ([ParserManager::NewParser](#) newParser)
connectNewParser

10.360.1 Member Function Documentation

10.360.1.1 connectNewParser()

```
bool ParserManager::connectNewParser (
    ParserManager::NewParser newParser )
```

connectNewParser

Parameters

<i>newParser</i>	
------------------	--

Returns

10.360.1.2 generateNewParser()

```
ParserManager::GenerateNewParserResult ParserManager::generateNewParser (
    ParserChangesInformation * changes )
```

generateNewParser

Parameters

<i>changes</i>	
----------------	--

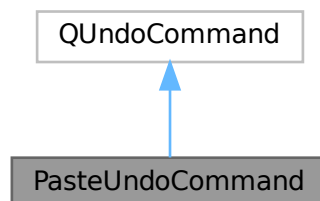
Returns

The documentation for this class was generated from the following files:

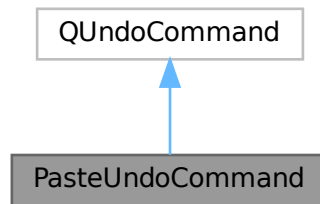
- kernel/simulator/ParserManager.h
- kernel/simulator/ParserManager.cpp

10.361 PasteUndoCommand Class Reference

Inheritance diagram for PasteUndoCommand:



Collaboration diagram for PasteUndoCommand:



Public Member Functions

- **PasteUndoCommand** (QList< [GraphicalModelComponent](#) * > *graphicalComponents, QList< [GraphicalConnection](#) * > *connections, QList< QGraphicsItemGroup * > *groups, QList< QGraphicsItem * > *drawing, [ModelGraphicsScene](#) *scene, QUndoCommand *parent=nullptr)
- void **undo** () override
- void **redo** () override

The documentation for this class was generated from the following files:

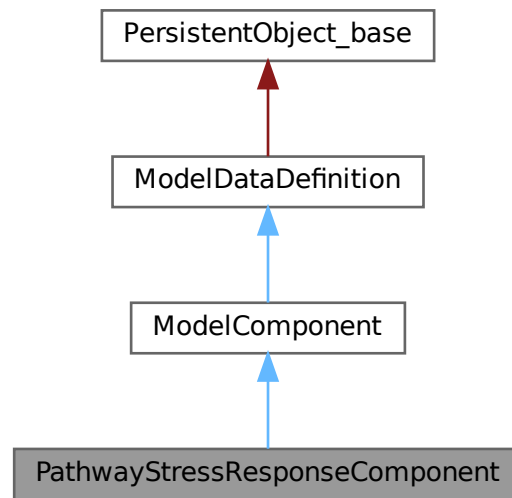
- applications/gui/genesys/actions/PasteUndoCommand.h
- applications/gui/genesys/actions/PasteUndoCommand.cpp

10.362 PathwayStressResponseComponent Class Reference

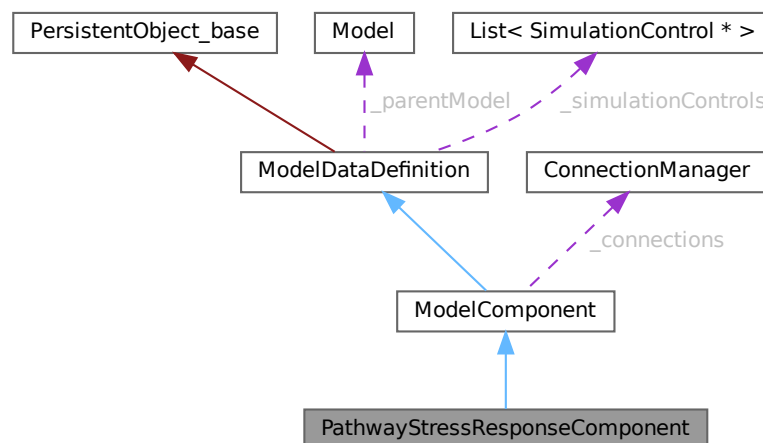
Applies sustained pathway-collapse responses to a whole-cell state.

```
#include <plugins/components/WholeCellModeling/PathwayStressResponseComponent.h>
```


Inheritance diagram for PathwayStressResponseComponent:



Collaboration diagram for PathwayStressResponseComponent:



Public Member Functions

- **PathwayStressResponseComponent** (`Model` *model, std::string name="")
- void **setWholeCellState** (`WholeCellState` *state)
- `WholeCellState` * **getWholeCellState** () const
- void **setMonitoredPathwayKey** (std::string key)
- std::string **getMonitoredPathwayKey** () const

- void **setStressThreshold** (double threshold)
- double **getStressThreshold** () const
- void **setArrestAfterSteps** (unsigned int steps)
- unsigned int **getArrestAfterSteps** () const
- void **setDeathAfterSteps** (unsigned int steps)
- unsigned int **getDeathAfterSteps** () const
- void **setArrestPhase** (std::string phase)
- std::string **getArrestPhase** () const
- void **setDeadPhase** (std::string phase)
- std::string **getDeadPhase** () const
- void **setRecoveryPhase** (std::string phase)
- std::string **getRecoveryPhase** () const
- void **setResetStreakOnRecovery** (bool reset)
- bool **getResetStreakOnRecovery** () const
- unsigned int **getStressStreak** () const
- double **getLastObservedActivity** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const

- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool \_getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * \_getParserChangesInformation ()`
- `virtual void \_addSimulationControl (SimulationControl *control)`
- `virtual void \_createInternalAndAttachedData ()`
- `virtual void \_createInternalStatisticReporters ()`
- `virtual void \_createAttachedAttributes ()`
- `virtual void \_createNonEditableDataDefinitions ()`
- `void \_templateCreateInternalStatisticReporters ()`
- `void \_templateCreateAttachedAttributes ()`
- `void \_templateCreateNonEditableDataDefinitions ()`
- `void \_templateCreateEditableDataDefinitions ()`
- `void \_internalStatisticReportersClear ()`
- *legacy helper methods used by the granular create* hooks*
- `void \_internalDataClear ()`
- `void \_internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void \_internalDataRemove (const std::string &key)`
- `void \_attachedDataClear ()`
- `void \_attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_attachedDataRemove (const std::string &key)`
- `void \_attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void \_statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_statisticReporterRemove (const std::string &key)`
- `void \_statisticReportersClear ()`
- `void \_mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryAttachedAttributeRemove (const std::string &key)`
- `void \_mandatoryAttachedAttributesClear ()`
- `void \_mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void \_mandatoryEditableDataDefinitionsClear ()`
- `void \_optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_optionalEditableDataDefinitionRemove (const std::string &key)`
- `void \_optionalEditableDataDefinitionsClear ()`
- `void \_mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void \_mandatoryNonEditableDataDefinitionsClear ()`
- `void \_optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void \_optionalNonEditableDataDefinitionsClear ()`
- `void \_checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- *method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.362.1 Detailed Description

Applies sustained pathway-collapse responses to a whole-cell state.

[PathwayStressResponseComponent](#) watches one pathway activity proxy already stored in [WholeCellState](#) and converts repeated low-activity windows into higher-level lifecycle responses such as arrest and death.

The component is intentionally simple:

- it does not solve metabolism;
- it does not schedule time;
- it does not route entities.

Instead, it acts as a dedicated biological stress-response layer that can be placed between checkpointing and fate-routing components in hybrid whole-cell examples.

10.362.2 Member Function Documentation

10.362.2.1 `_check()`

```
bool PathwayStressResponseComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.362.2.2 `_createEditableDataDefinitions()`

```
void PathwayStressResponseComponent::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.362.2.3 `_initBetweenReplications()`

```
void PathwayStressResponseComponent::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.362.2.4 `_loadInstance()`

```
bool PathwayStressResponseComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.362.2.5 `_onDispatchEvent()`

```
void PathwayStressResponseComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.362.2.6 `_saveInstance()`

```
void PathwayStressResponseComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.362.2.7 show()

```
std::string PathwayStressResponseComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

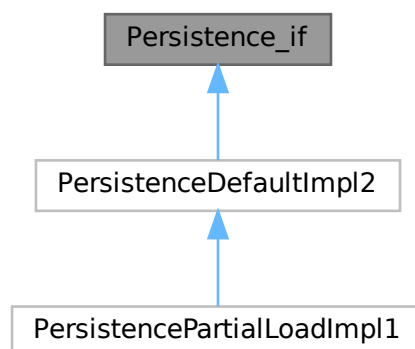
- plugins/components/WholeCellModeling/PathwayStressResponseComponent.h
- plugins/components/WholeCellModeling/PathwayStressResponseComponent.cpp

10.363 Persistence_if Class Reference

Interface for model serialization/deserialization services.

```
#include <kernel/simulator/persistence/Persistence_if.h>
```

Inheritance diagram for Persistence_if:



Public Types

- enum class **Options** : int {
SAVEDEFAULTS = 1 , **HIDEIDKEY** = 2 , **HIDETYPEKEY** = 4 , **HIDENAMEKEY** = 8 ,
SORTALPHLY = 16 }

Public Member Functions

- virtual bool [save](#) (std::string filename)=0
Serializes and saves the model to the provided filename.
- virtual bool [load](#) (std::string filename)=0
Loads and deserializes a model from file.
- virtual bool [hasChanged](#) ()=0
Indicates whether persistent state changed since the last save/load.
- virtual void [setHasChanged](#) (bool [hasChanged](#))=0
Updates the persistent-state changed flag.
- virtual bool [getOption](#) (Persistence_if::Options option)=0
Returns whether a persistence option is currently enabled.
- virtual void [setOption](#) (Persistence_if::Options option, bool value)=0
Enables or disables a persistence option.
- virtual std::string [getFormattedField](#) (PersistenceRecord *fields)=0
Formats a set of fields as text according to the persistence format.

10.363.1 Detailed Description

Interface for model serialization/deserialization services.

Persistence implementations convert in-memory model structures to/from external representations (e.g., GEN/↔ XML/JSON-like formats) and expose formatting/options hooks used by the kernel persistence pipeline.

10.363.2 Member Function Documentation

10.363.2.1 getFormattedField()

```
virtual std::string Persistence_if::getFormattedField (
    PersistenceRecord * fields ) [pure virtual]
```

Formats a set of fields as text according to the persistence format.

Implemented in [PersistenceDefaultImpl2](#).

10.363.2.2 getOption()

```
virtual bool Persistence_if::getOption (
    Persistence_if::Options option ) [pure virtual]
```

Returns whether a persistence option is currently enabled.

Implemented in [PersistenceDefaultImpl2](#).

10.363.2.3 hasChanged()

```
virtual bool Persistence_if::hasChanged ( ) [pure virtual]
```

Indicates whether persistent state changed since the last save/load.

Implemented in [PersistenceDefaultImpl2](#).

10.363.2.4 load()

```
virtual bool Persistence_if::load (
    std::string filename ) [pure virtual]
```

Loads and deserializes a model from file.

Implemented in [PersistenceDefaultImpl2](#), and [PersistencePartialLoadImpl1](#).

10.363.2.5 save()

```
virtual bool Persistence_if::save (
    std::string filename ) [pure virtual]
```

Serializes and saves the model to the provided filename.

Implemented in [PersistenceDefaultImpl2](#).

10.363.2.6 setHasChanged()

```
virtual void Persistence_if::setHasChanged (
    bool hasChanged ) [pure virtual]
```

Updates the persistent-state changed flag.

Implemented in [PersistenceDefaultImpl2](#).

10.363.2.7 setOption()

```
virtual void Persistence_if::setOption (
    Persistence_if::Options option,
    bool value ) [pure virtual]
```

Enables or disables a persistence option.

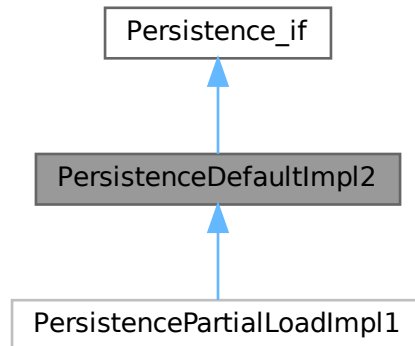
Implemented in [PersistenceDefaultImpl2](#).

The documentation for this class was generated from the following file:

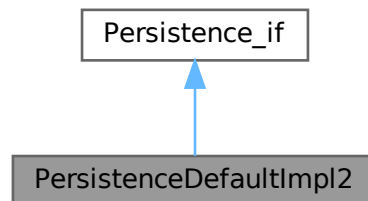
- kernel/simulator/persistence/Persistence_if.h

10.364 PersistenceDefaultImpl2 Class Reference

Inheritance diagram for PersistenceDefaultImpl2:



Collaboration diagram for PersistenceDefaultImpl2:



Public Member Functions

- **PersistenceDefaultImpl2** ([Model](#) *model)
- bool [save](#) (std::string filename) override
Serializes and saves the model to the provided filename.
- bool [load](#) (std::string filename) override
Loads and deserializes a model from file.
- bool [hasChanged](#) () override
Indicates whether persistent state changed since the last save/load.
- void [setHasChanged](#) (bool [hasChanged](#)) override
Updates the persistent-state changed flag.
- bool [getOption](#) (Persistence_if::Options option) override
Returns whether a persistence option is currently enabled.

- void [setOption](#) (Persistence_if::Options option, bool value) override
Enables or disables a persistence option.
- std::string [getFormattedField](#) (PersistenceRecord *fields) override
Formats a set of fields as text according to the persistence format.

Additional Inherited Members

Public Types inherited from [Persistence_if](#)

- enum class **Options** : int {
 SAVEDEFAULTS = 1 , HIDEIDKEY = 2 , HIDETYPEKEY = 4 , HIDENAMEKEY = 8 ,
 SORTALPHLY = 16 }

10.364.1 Member Function Documentation

10.364.1.1 [getFormattedField\(\)](#)

```
std::string PersistenceDefaultImpl2::getFormattedField (
    PersistenceRecord * fields ) [override], [virtual]
```

Formats a set of fields as text according to the persistence format.

Implements [Persistence_if](#).

10.364.1.2 [getOption\(\)](#)

```
bool PersistenceDefaultImpl2::getOption (
    Persistence_if::Options option ) [override], [virtual]
```

Returns whether a persistence option is currently enabled.

Implements [Persistence_if](#).

10.364.1.3 [hasChanged\(\)](#)

```
bool PersistenceDefaultImpl2::hasChanged ( ) [override], [virtual]
```

Indicates whether persistent state changed since the last save/load.

Implements [Persistence_if](#).

10.364.1.4 [load\(\)](#)

```
bool PersistenceDefaultImpl2::load (
    std::string filename ) [override], [virtual]
```

Loads and deserializes a model from file.

Implements [Persistence_if](#).

Reimplemented in [PersistencePartialLoadImpl1](#).

10.364.1.5 save()

```
bool PersistenceDefaultImpl2::save (
    std::string filename ) [override], [virtual]
```

Serializes and saves the model to the provided filename.

Implements [Persistence_if](#).

10.364.1.6 setHasChanged()

```
void PersistenceDefaultImpl2::setHasChanged (
    bool hasChanged ) [override], [virtual]
```

Updates the persistent-state changed flag.

Implements [Persistence_if](#).

10.364.1.7 setOption()

```
void PersistenceDefaultImpl2::setOption (
    Persistence_if::Options option,
    bool value ) [override], [virtual]
```

Enables or disables a persistence option.

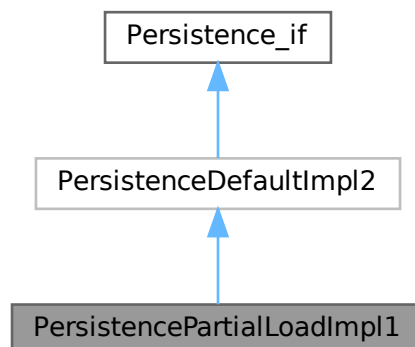
Implements [Persistence_if](#).

The documentation for this class was generated from the following files:

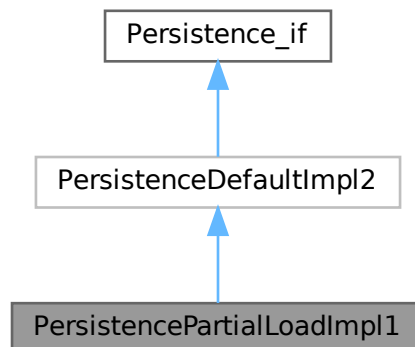
- kernel/simulator/persistence/PersistenceDefaultImpl2.h
- kernel/simulator/persistence/PersistenceDefaultImpl2.cpp

10.365 PersistencePartialLoadImpl1 Class Reference

Inheritance diagram for PersistencePartialLoadImpl1:



Collaboration diagram for PersistencePartialLoadImpl1:



Public Member Functions

- **PersistencePartialLoadImpl1** ([Model](#) *model)
- bool [load](#) (std::string filename) override
Loads and deserializes a model from file.

Public Member Functions inherited from [PersistenceDefaultImpl2](#)

- **PersistenceDefaultImpl2** ([Model](#) *model)
- bool [save](#) (std::string filename) override
Serializes and saves the model to the provided filename.
- bool [hasChanged](#) () override
Indicates whether persistent state changed since the last save/load.
- void [setHasChanged](#) (bool [hasChanged](#)) override
Updates the persistent-state changed flag.
- bool [getOption](#) (Persistence_if::Options option) override
Returns whether a persistence option is currently enabled.
- void [setOption](#) (Persistence_if::Options option, bool value) override
Enables or disables a persistence option.
- std::string [getFormattedField](#) ([PersistenceRecord](#) *fields) override
Formats a set of fields as text according to the persistence format.

Additional Inherited Members

Public Types inherited from [Persistence_if](#)

- enum class **Options** : int {
 SAVEDEFAULTS = 1 , **HIDEIDKEY** = 2 , **HIDETYPEKEY** = 4 , **HIDENAMEKEY** = 8 ,
 SORTALPHLY = 16 }

10.365.1 Member Function Documentation

10.365.1.1 load()

```
bool PersistencePartialLoadImpl1::load (
    std::string filename ) [override], [virtual]
```

Loads and deserializes a model from file.

Reimplemented from [PersistenceDefaultImpl2](#).

The documentation for this class was generated from the following files:

- kernel/simulator/persistence/PersistencePartialLoadImpl1.h
- kernel/simulator/persistence/PersistencePartialLoadImpl1.cpp

10.366 PersistenceRecord Class Reference

Serialization/Deserialization structure which behaves like an associative array.

```
#include <kernel/simulator/Persistence.h>
```

Classes

- struct [Entry](#)
"Raw" entry, users should treat this as an opaque type since payloads could be in any format.

Public Types

- using **Payload** = std::string
- using **Iterator** = std::unordered_map< std::string, [Entry](#) >::iterator

Public Member Functions

- virtual [PersistenceRecord](#) * **newInstance** ()
- virtual std::string **loadField** (const std::string &key, const std::string &defaultValue="")
- virtual double **loadField** (const std::string &key, double defaultValue)
- virtual unsigned int **loadField** (const std::string &key, unsigned int defaultValue)
- virtual int **loadField** (const std::string &key, int defaultValue)
- virtual [Util::TimeUnit](#) **loadField** (const std::string &key, [Util::TimeUnit](#) defaultValue)
- virtual void **saveField** (const std::string &key, const std::string &value, const std::string &defaultValue, const bool saveIfDefault=false)
- virtual void **saveField** (const std::string &key, const std::string &value)
- virtual void **saveField** (const std::string &key, double value, const double defaultValue, const bool saveIfDefault=false)
- virtual void **saveField** (const std::string &key, unsigned int value, const unsigned int defaultValue, const bool saveIfDefault=false)
- virtual void **saveField** (const std::string &key, unsigned int value)
- virtual void **saveField** (const std::string &key, int value, const int defaultValue, const bool saveIfDefault=false)

- virtual void **saveField** (const std::string &key, [Util::TimeUnit](#) value, const [Util::TimeUnit](#) defaultValue, const bool saveIfDefault=false)
- **PersistenceRecord** ([Persistence_if](#) &config)
- std::size_t **size** () const
Returns the number of distinct entries in the record.
- void **insert** ([Entry](#) entry)
Inserts a raw entry in the record.
- void **insert** (Iterator it, Iterator [end](#))
Inserts multiple entries from the given iterators.
- void **erase** (const std::string &key)
Removes a specific entry from the record.
- void **clear** ()
Removes all entries from this record.
- Iterator **find** (const std::string &key)
Returns an iterator over a specific entry, if found.
- Iterator **begin** ()
Returns an iterator over raw entries.
- Iterator **end** ()
Past-the-end sentinel iterator.

10.366.1 Detailed Description

Serialization/Deserialization structure which behaves like an associative array.

Simulation models/components/definitions use this as a "visitor", which is guided across their innards during ser/deserialization and filled with fields. See [PersistentObject_base](#).

When this structure reaches the main model persistence interface, it finishes the serialization process in a format-specific manner. See [Persistence_if](#) and [Serializer](#).

10.366.2 Member Function Documentation

10.366.2.1 insert()

```
void PersistenceRecord::insert (
    Iterator it,
    Iterator end )
```

Inserts multiple entries from the given iterators.

Insert the raw entry payload from the source iterator range. Skip entries that already exist to avoid overwriting without warning.

The documentation for this class was generated from the following files:

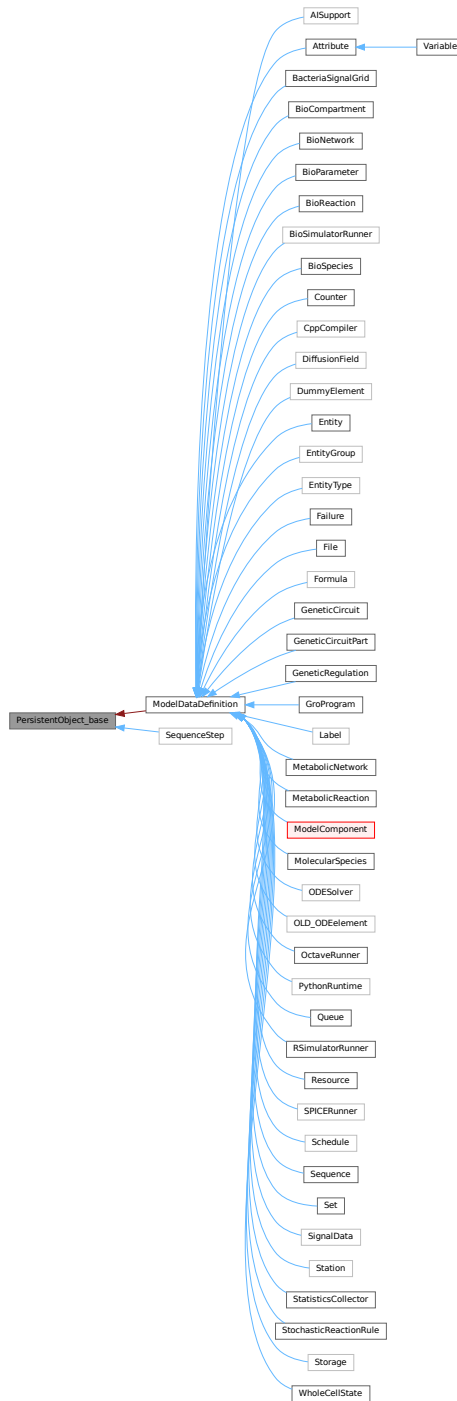
- kernel/simulator/Persistence.h
- kernel/simulator/Persistence.cpp

10.367 PersistentObject_base Class Reference

Dynamic serializable interface.

```
#include <kernel/simulator/Persistence.h>
```

Inheritance diagram for PersistentObject_base:



Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields)=0
Loads object fields from a deserialization structure.
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues=false)=0
Saves object fields into a serialization structure.

10.367.1 Detailed Description

Dynamic serializable interface.

Any object that's supposed to be saved to and/or loaded from a file must implement this interface.

10.367.2 Member Function Documentation

10.367.2.1 `_loadInstance()`

```
virtual bool PersistentObject_base::_loadInstance (
    PersistenceRecord * fields ) [protected], [pure virtual]
```

Loads object fields from a deserialization structure.

Parameters

<i>fields</i>	a table-like structure which can be queried for parsed fields.
---------------	--

Returns

a boolean which is true iff the load operation was successful.

Implemented in [Attribute](#), [Counter](#), [Entity](#), [EntityType](#), [StatisticsCollector](#), [ModelComponent](#), [ModelDataDefinition](#), [SinkModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [OLD_ODEelement](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), [XNOR](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [FSMState](#), [ModalModelDefault](#), [PetriPlace](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), [StochasticTranslation](#), [AISupport](#), [BacteriaSignalGrid](#), [BioNetwork](#), [BioParameter](#), [BioReaction](#), [BioSimulatorRunner](#), [BioSpecies](#), [GeneticCircuit](#), [GeneticCircuitPart](#), [GeneticRegulation](#), [GroProgram](#), [MetabolicNetwork](#), [MetabolicReaction](#), [DiffusionField](#), [ODESolver](#), [Failure](#), [Queue](#), [Resource](#), [Schedule](#), [CppCompiler](#), [OctaveRunner](#), [PythonRuntime](#), [RSimulatorRunner](#), [SPICERunner](#), [EntityGroup](#), [File](#), [Formula](#), [Label](#), [Set](#), [Variable](#), [SequenceStep](#), [Sequence](#), [Station](#), [Storage](#), [SignalData](#), [DummyElement](#), [BioCompartment](#), [MolecularSpecies](#), [StochasticReactionRule](#), and [WholeCellState](#).

10.367.2.2 _saveInstance()

```
virtual void PersistentObject_base::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues = false ) [protected], [pure virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

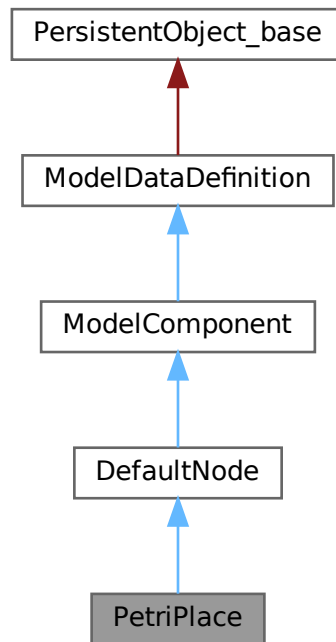
Implemented in [Attribute](#), [Counter](#), [Entity](#), [EntityType](#), [StatisticsCollector](#), [ModelComponent](#), [ModelDataDefinition](#), [SinkModelComponent](#), [SourceModelComponent](#), [AIAssistant](#), [MarkovChain](#), [BacteriaColony](#), [BioRunnerCommand](#), [BioSimulate](#), [BioSteadyState](#), [GeneticCircuitSimulate](#), [GeneticExpressionStep](#), [MetabolicFluxBalance](#), [ContinuousSystemComponent](#), [DiffEquations](#), [DiffusionSimulate](#), [LSODE](#), [OLD_ODEelement](#), [Decide](#), [DropOff](#), [Remove](#), [Search](#), [Buffer](#), [Clone](#), [Delay](#), [Process](#), [Release](#), [Seize](#), [SPICECircuit](#), [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), [XNOR](#), [SPICENode](#), [CppForG](#), [PythonForG](#), [RSimulator](#), [Batch](#), [Separate](#), [Record](#), [Write](#), [Assign](#), [Create](#), [Dispose](#), [Access](#), [Enter](#), [Exit](#), [Leave](#), [PickStation](#), [Route](#), [Start](#), [Stop](#), [Store](#), [Unstore](#), [CellularAutomataComp](#), [DefaultNode](#), [FSMState](#), [ModalModelDefault](#), [PetriPlace](#), [Submodel](#), [Match](#), [Signal](#), [Wait](#), [DummyComponent](#), [BioStateProjectionComponent](#), [CellCycleCheckpointComponent](#), [CellDivisionEvent](#), [CellFateDecisionComponent](#), [CellGrowthComponent](#), [CompartmentExchangeComponent](#), [EukaryoticCellCycleComponent](#), [FtsZPolymerizationComponent](#), [MetabolicStateProjectionComponent](#), [MetabolicSubmodelComponent](#), [PathwayStressResponseComponent](#), [ResourceAllocationComponent](#), [StochasticReactionComponent](#), [StochasticTranscription](#), [StochasticTranslation](#), [AISupport](#), [BacteriaSignalGrid](#), [BioNetwork](#), [BioParameter](#), [BioReaction](#), [BioSimulatorRunner](#), [BioSpecies](#), [GeneticCircuit](#), [GeneticCircuitPart](#), [GeneticRegulation](#), [GroProgram](#), [MetabolicNetwork](#), [MetabolicReaction](#), [DiffusionField](#), [ODESolver](#), [Failure](#), [Queue](#), [Resource](#), [Schedule](#), [CppCompiler](#), [OctaveRunner](#), [PythonRuntime](#), [RSimulatorRunner](#), [SPICERunner](#), [EntityGroup](#), [File](#), [Formula](#), [Label](#), [Set](#), [Variable](#), [SequenceStep](#), [Sequence](#), [Station](#), [Storage](#), [SignalData](#), [DummyElement](#), [BioCompartment](#), [MolecularSpecies](#), [StochasticReactionRule](#), and [WholeCellState](#).

The documentation for this class was generated from the following file:

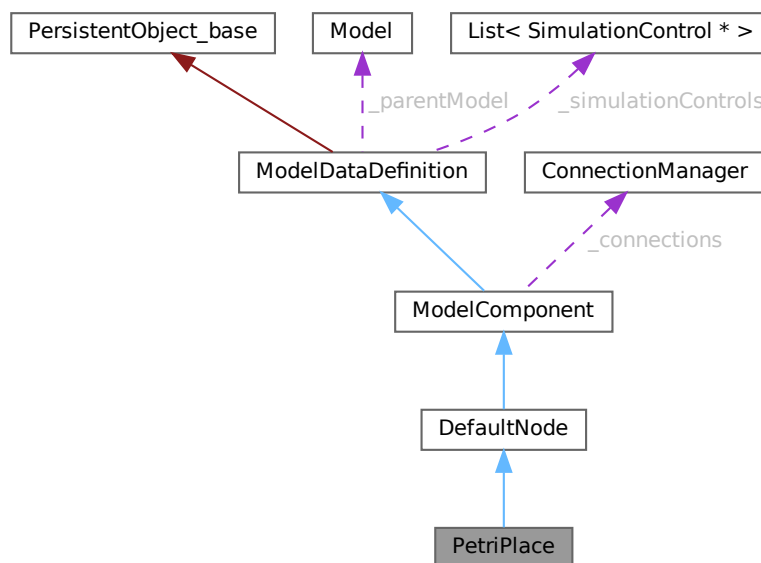
- `kernel/simulator/Persistence.h`

10.368 PetriPlace Class Reference

Inheritance diagram for PetriPlace:



Collaboration diagram for PetriPlace:



Public Member Functions

- **PetriPlace** ([Model](#) *model, std::string name="")
- unsigned int **getTokens** (std::string color="default") const
- void **addTokens** (unsigned int quantity, std::string color="default")
- bool **removeTokens** (unsigned int quantity, std::string color="default")
- std::map< std::string, unsigned int > * **getAllTokens** ()

Public Member Functions inherited from [DefaultNode](#)

- [DefaultNode](#) ([Model](#) *model, std::string componentTypename, std::string name)

constructors

- **DefaultNode** ([Model](#) *model, std::string name="")
- void **addTransition** ([DefaultNodeTransition](#) *transition)
- new public user methods for this component*
- void **removeTransition** ([DefaultNodeTransition](#) *transition)
- [List](#)< [DefaultNodeTransition](#) * > * **getTransitions** () const
- void **setInitialNode** (bool initialNode)
- bool **isInitialNode** () const
- void **setFinalNode** (bool finalNode)
- bool **isFinalNode** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")

constructors

- virtual ~**ModelComponent** ()

Releases component-owned resources and unregisters it from the model.

- [ConnectionManager](#) * **getConnectionManager** () const

new public user methods for this component

- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)

Connects this component to another component.

- bool **hasBreakpointAt** ()

Indicates whether a simulation breakpoint is set on this component.

- void **setDescription** (std::string _description)

Updates the free-form description shown by the GUI.

- std::string **getDescription** () const

Returns the free-form description associated with this component.

- void **remove** ([ModelComponent](#) *comp)

Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [DefaultNode](#)

- static [PluginInformation](#) * **GetPluginInformation** ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [DefaultNode](#)

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))

not just an easy access to trace manager, but wrappers to check if specific trace level applies

- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.

- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
- virtual protected methods that could be overridden by derived classes, if needed*

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.368.1 Member Function Documentation

10.368.1.1 [_loadInstance\(\)](#)

```
bool PetriPlace::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [DefaultNode](#).

10.368.1.2 [_saveInstance\(\)](#)

```
void PetriPlace::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

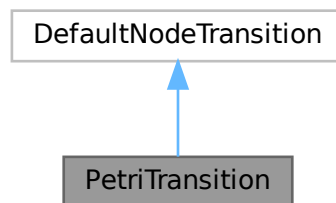
Reimplemented from [DefaultNode](#).

The documentation for this class was generated from the following files:

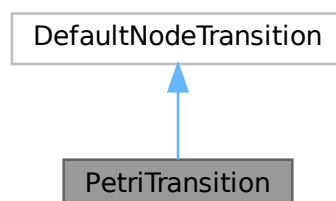
- plugins/components/ModalModel/PetriPlace.h
- plugins/components/ModalModel/PetriPlace.cpp

10.369 PetriTransition Class Reference

Inheritance diagram for PetriTransition:



Collaboration diagram for PetriTransition:



Public Member Functions

- **PetriTransition** ([DefaultNode](#) *source, [DefaultNode](#) *destination, std::string name="")
- void **setInputArcWeight** (std::string color, unsigned int weight)
- void **setOutputArcWeight** (std::string color, unsigned int weight)
- virtual bool **canFire** ([Model](#) *model, [Entity](#) *entity) const override
- virtual void **execute** ([Model](#) *model, [Entity](#) *entity) const override

Public Member Functions inherited from [DefaultNodeTransition](#)

- **DefaultNodeTransition** ([DefaultNode](#) *source, [DefaultNode](#) *destination, std::string name="")
- void **setSource** ([DefaultNode](#) *source)
- [DefaultNode](#) * **getSource** () const
- void **setDestination** ([DefaultNode](#) *destination)
- [DefaultNode](#) * **getDestination** () const
- void **setName** (std::string name)
- std::string **getName** () const
- void **setGuardExpression** (std::string guardExpression)
- std::string **getGuardExpression** () const
- void **setOutputExpression** (std::string outputExpression)
- std::string **getOutputExpression** () const
- void **setInputEvent** (std::string inputEvent)
- std::string **getInputEvent** () const
- void **setPriority** (unsigned int priority)
- unsigned int **getPriority** () const
- void **setProbability** (double probability)
- double **getProbability** () const
- void **setTransitionKind** (TransitionKind transitionKind)
- TransitionKind **getTransitionKind** () const

Additional Inherited Members

Public Types inherited from [DefaultNodeTransition](#)

- enum class **TransitionKind** : int { **DETERMINISTIC** = 0 , **PROBABILISTIC** = 1 }

10.369.1 Member Function Documentation

10.369.1.1 canFire()

```
bool PetriTransition::canFire (
    Model * model,
    Entity * entity ) const [override], [virtual]
```

Reimplemented from [DefaultNodeTransition](#).

10.369.1.2 execute()

```
void PetriTransition::execute (
    Model * model,
    Entity * entity ) const [override], [virtual]
```

Reimplemented from [DefaultNodeTransition](#).

The documentation for this class was generated from the following files:

- plugins/components/ModalModel/DefaultTransitionExtensions.h
- plugins/components/ModalModel/DefaultTransitionExtensions.cpp

10.370 PickableStationItem Class Reference

Public Member Functions

- **PickableStationItem** ([Model](#) *model, std::string stationName, std::string queueName="", std::string resourceName="")
- **PickableStationItem** ([Model](#) *model, std::string stationName, std::string expression)
- **PickableStationItem** ([Station](#) *station, [Queue](#) *queue)
- **PickableStationItem** ([Station](#) *station, [Resource](#) *resource)
- **PickableStationItem** ([Station](#) *station, std::string expression)
- **PickableStationItem** ([Station](#) *station)
- std::string **getExpression** () const
- [Queue](#) * **getQueue** () const
- [Resource](#) * **getResource** () const
- [Station](#) * **getStation** () const
- void **setExpression** (std::string _expression)
- void **setQueue** ([Queue](#) *_queue)
- void **setResource** ([Resource](#) *_resource)
- void **setStation** ([Station](#) *_station)
- std::string **getName** ()
- void **_addSimulationControl** ([SimulationControl](#) *control)
- [List](#)< [SimulationControl](#) * > * **getSimulationControls** () const
- std::string **getTypeDC** ()

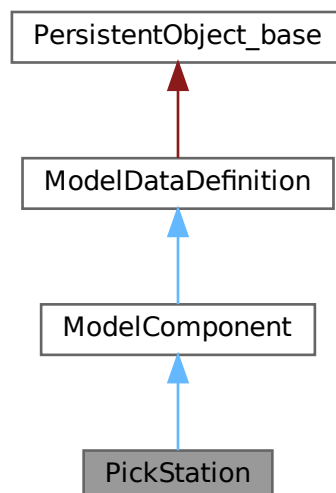
The documentation for this class was generated from the following files:

- plugins/components/MaterialHandling/PickableStationItem.h
- plugins/components/MaterialHandling/PickableStationItem.cpp

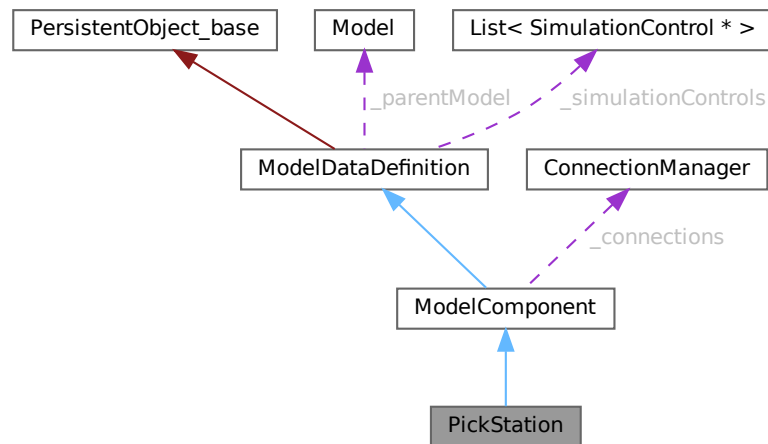
10.371 PickStation Class Reference

```
#include <plugins/components/MaterialHandling/PickStation.h>
```

Inheritance diagram for PickStation:



Collaboration diagram for PickStation:



Public Types

- enum class **TestCondition** : int { **MINIMUM** = 0 , **MAXIMUM** = 1 , **num_elements** = 2 }

Public Member Functions

- **PickStation** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- void **setTestCondition** (PickStation::TestCondition _testCondition)
- PickStation::TestCondition **getTestCondition** () const
- void **setSaveAttribute** (std::string _saveAttribute)
- std::string **getSaveAttribute** () const
- List< [PickableStationItem](#) * > * **getPickableStationItems** () const
- void **addPickableStationItem** ([PickableStationItem](#) *newItem)
- void **removePickableStationItem** ([PickableStationItem](#) *item)
- void **setPickConditionExpression** (bool _pickConditionExpression)
- bool **isPickConditionExpression** () const
- void **setPickConditionNumberInQueue** (bool _pickConditionNumberInQueue)
- bool **isPickConditionNumberInQueue** () const
- void **setPickConditionNumberBusyResource** (bool _pickConditionNumberBusyResource)
- bool **isPickConditionNumberBusyResource** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- *constructors*
- virtual ~**ModelComponent** ()
- *Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const

- new public user methods for this component*
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↔IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

- Sets the model nesting level for this definition.*

• `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific←Enabled=true)`

Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`

Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`

Enables or disables the specific trace level.

Static Public Member Functions

- `static std::string convertEnumToStr (TestCondition condition)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from ModelComponent

- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static bool Check (ModelComponent *component)`
Validates a component instance.
- `static void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- `static void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- `static void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from ModelDataDefinition

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` () override
- virtual void `_initBetweenReplications` () override
- virtual void `_addSimulationControl` ([SimulationControl](#) *property) override
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()

- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.371.1 Detailed Description

[PickStation](#) module DESCRIPTION The [PickStation](#) module allows an entity to select a particular station from the multiple stations specified. This module picks among the group of stations based on the selection logic defined with the module. The entity may then route, transport, convey, or connect to the station specified. If the method chosen is connect, the selected station is assigned to an entity attribute. The station selection process is based on the minimum or maximum value of a variety of system variables and expressions. TYPICAL USES A part sent to a processing station based on machine's availability at each station A loan application sent to a set of loan officers based on the number sent to each officer A customer selecting among cashier lines based on the least number waiting in each line PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart. Test Condition Test condition to use for the station selection process, either Minimum or Maximum. Number En [Route](#) to [Station](#) The number of entities transferring to the station is considered in the station selection process. Number in [Queue](#) The number of entities in the queue at the station is considered in the station selection process. Number of Resources Busy The number of busy resources at the station is considered in the station selection process. Expression Determines if an additional user-defined expression is considered in the station selection process. Transfer Type Determines how an entity will be transferred out of this module to its next destination station—either [Route](#), Convey, Transport, or Connect. Save [Attribute](#) Defines the name of the attribute that will store the station name that is selected, visible when the transfer method is Connect. [Route](#) Time Move time of the entity from its current station to the station determined through this module. Units Time units for route-time parameters.

10.371.2 Member Function Documentation

10.371.2.1 `_addSimulationControl()`

```
void PickStation::_addSimulationControl (
    SimulationControl * property ) [override], [protected], [virtual]
```

This method is necessary only for those components that instantiate internal elements that must exist before simulation starts and even before model checking. That's the case of components that have internal `StatisticsCollectors`, since others components may refer to them as expressions (as in "TVAG(ThisCSTAT)") and therefore the model datum must exist before checking such expression

Reimplemented from [ModelDataDefinition](#).

10.371.2.2 `_check()`

```
bool PickStation::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.371.2.3 `_createEditableDataDefinitions()`

```
void PickStation::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.371.2.4 `_getParserChangesInformation()`

```
ParserChangesInformation * PickStation::_getParserChangesInformation ( ) [override], [protected],
[virtual]
```

This method returns all changes in the parser that are needed by plugins of this `ModelData`s. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

10.371.2.5 `_initBetweenReplications()`

```
void PickStation::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.371.2.6 `_loadInstance()`

```
bool PickStation::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.371.2.7 `_onDispatchEvent()`

```
void PickStation::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.371.2.8 `_saveInstance()`

```
void PickStation::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.371.2.9 `show()`

```
std::string PickStation::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

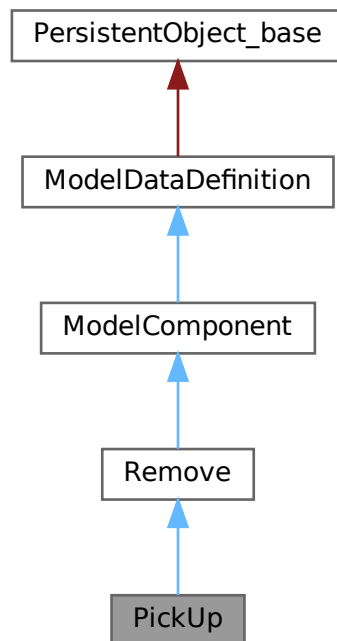
The documentation for this class was generated from the following files:

- plugins/components/MaterialHandling/PickStation.h
- plugins/components/MaterialHandling/PickStation.cpp

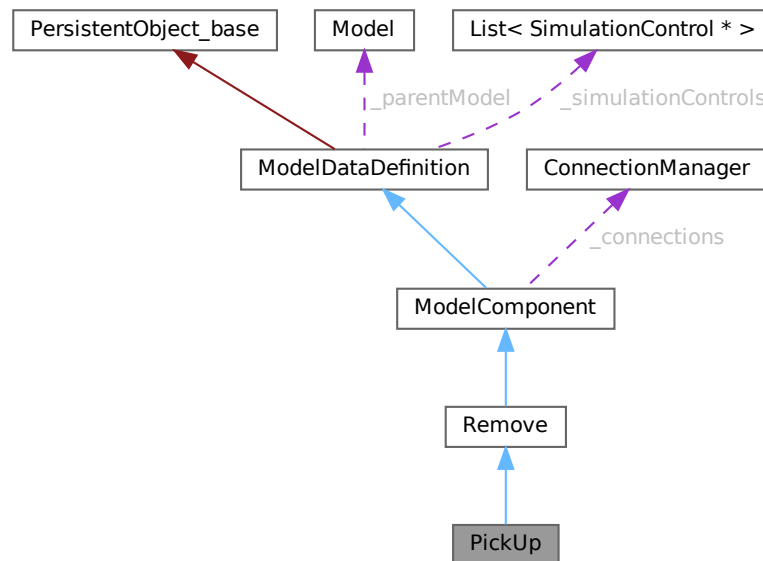
10.372 PickUp Class Reference

```
#include <plugins/components/Decisions/PickUp.h>
```

Inheritance diagram for PickUp:



Collaboration diagram for Pickup:



Public Member Functions

- **PickUp** ([Model](#) *model, std::string name="")

Public Member Functions inherited from [Remove](#)

- **Remove** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- virtual public methods*
- void **setRemoveStartRank** (std::string _removeFromRank)
- std::string **getRemoveStartRank** () const
- void **setRemoveEndRank** (std::string _removeEndRank)
- std::string **getRemoveEndRank** () const
- void **setRemoveFromType** (Remove::RemoveFromType _removeFromType)
- Remove::RemoveFromType **getRemoveFromType** () const
- void **setRemoveFrom** ([ModelDataDefinition](#) *_removeFrom)
- [ModelDataDefinition](#) * **getRemoveFrom** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- constructors*
- virtual ~**ModelComponent** ()
- Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- new public user methods for this component*

- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.

- `List< SimulationControl * > * getSimulationControls ()` const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific ()` const
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled ()` const
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation ()`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `Remove`

- static `std::string convertEnumToStr (RemoveFromType type)`
- static `PluginInformation * GetPluginInformation ()`
- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `bool Check (ModelComponent *component)`
Validates a component instance.
- static `void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- static `void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- static `void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Additional Inherited Members

Public Types inherited from [Remove](#)

- enum class [RemoveFromType](#) : int { [QUEUE](#) = 0 , [ENTITYGROUP](#) = 1 , [num_elements](#) = 2 }

Protected Member Functions inherited from [Remove](#)

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()

legacy helper methods used by the granular create hooks*

- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)

Traces text to the simulation output.

- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.372.1 Detailed Description

Pickup module DESCRIPTION The Pickup module removes a number of consecutive entities from a given queue starting at a specified rank in the queue. The entities that are picked up are added to the end of the incoming entity's group. TYPICAL USES Gathering an order from various queue locations Gathering completed forms for an office order Picking up students at a bus stop for school PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Quantity Number of entities to pick up. [Queue](#) Name Name of the queue from which the entities will be picked up, starting at the specified rank. Starting Rank Starting rank of the entities to pick up from the queue, [Queue](#) Name.

The documentation for this class was generated from the following files:

- `plugins/components/Decisions/PickUp.h`
- `plugins/components/Decisions/PickUp.cpp`

10.373 Plugin Class Reference

```
#include <kernel/simulator/Plugin.h>
```

Public Member Functions

- [Plugin](#) (std::string filename_so_dll)
Creates a plugin wrapper for a dynamic library file.
- [Plugin](#) (StaticGetPluginInformation getInformation)
Creates a plugin wrapper from a compiled-in metadata callback.
- virtual ~[Plugin](#) ()
Releases metadata allocated for the wrapped plugin.
- std::string [show](#) ()
Returns a human-readable description of the plugin state.
- bool [isIsValidPlugin](#) () const
Indicates whether the plugin metadata was successfully initialized.
- [PluginInformation](#) * [getPluginInfo](#) () const
Returns the plugin metadata object.
- [ModelDataDefinition](#) * [loadNew](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
Creates a new data definition instance from serialized fields.
- bool [loadAndInsertNew](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
Loads a new instance and inserts it into the model in one step.
- [ModelDataDefinition](#) * [newInstance](#) ([Model](#) *model, std::string name="")
Creates a new in-memory instance of the plugin's primary type.

10.373.1 Detailed Description

A [Plugin](#) represents a dynamically linked component class ([ModelComponent](#)) or modeldatum class ([ModelDataDefinition](#)); It gives access to a [ModelComponent](#) so it can be used by the model. Classes like [Create](#), [Delay](#), and [Dispose](#) are examples of Plugins. It corresponds directly to the "Expansible" part (the capitalized 'E') of the GenESyS acronymous Plugins are **NOT** implemented yet

10.373.2 Constructor & Destructor Documentation

10.373.2.1 Plugin() [1/2]

```
Plugin::Plugin (
    std::string filename_so_dll )
```

Creates a plugin wrapper for a dynamic library file.

Parameters

<code>filename_so_dll</code>	Plugin library filename or path.
------------------------------	----------------------------------

10.373.2.2 Plugin() [2/2]

```
Plugin::Plugin (
    StaticGetPluginInformation getInformation )
```

Creates a plugin wrapper from a compiled-in metadata callback.

Parameters

<code>getInformation</code>	Callback that returns the plugin metadata.
-----------------------------	--

10.373.3 Member Function Documentation**10.373.3.1 getPluginInfo()**

```
PluginInformation * Plugin::getPluginInfo ( ) const
```

Returns the plugin metadata object.

Returns

Plugin information block, or `nullptr` when invalid.

10.373.3.2 isValidPlugin()

```
bool Plugin::isValidPlugin ( ) const
```

Indicates whether the plugin metadata was successfully initialized.

Returns

`true` when this wrapper contains a valid plugin.

10.373.3.3 loadAndInsertNew()

```
bool Plugin::loadAndInsertNew (
    Model * model,
    PersistenceRecord * fields )
```

Loads a new instance and inserts it into the model in one step.

Parameters

<i>model</i>	Parent model that will own the new instance.
<i>fields</i>	Serialized fields to deserialize.

Returns

`true` when loading and insertion succeed.

10.373.3.4 loadNew()

```
ModelDataDefinition * Plugin::loadNew (
    Model * model,
    PersistenceRecord * fields )
```

Creates a new data definition instance from serialized fields.

Creates a new [ModelDataDefinition](#) from fields loaded from a file or another persistence source.

Parameters

<i>model</i>	Parent model that will own the new instance.
<i>fields</i>	Serialized fields to deserialize.

Returns

New data definition instance, or `nullptr` on failure.

10.373.3.5 newInstance()

```
ModelDataDefinition * Plugin::newInstance (
    Model * model,
    std::string name = "" )
```

Creates a new in-memory instance of the plugin's primary type.

Parameters

<i>model</i>	Parent model that will own the instance.
<i>name</i>	Initial object name.

Returns

Newly created instance, or `nullptr` on failure.

10.373.3.6 show()

```
std::string Plugin::show ( )
```

Returns a human-readable description of the plugin state.

Returns

Textual summary used in traces and diagnostics.

responsibilities should remain in PluginConnector/PluginManager layers.

The documentation for this class was generated from the following files:

- kernel/simulator/Plugin.h
- kernel/simulator/Plugin.cpp

10.374 PluginCatalogController Class Reference

Controller responsible for plugin catalog presentation and plugin palette behavior.

```
#include <applications/gui/genesys/controllers/PluginCatalogController.h>
```

Public Member Functions

- **PluginCatalogController** ([Simulator](#) *simulator, QTreeWidget *pluginsTree, QPlainTextEdit *modelText↵ Editor, std::map< std::string, QColor > *pluginCategoryColor)
Creates the plugin-catalog controller used by [MainWindow](#) compatibility slots.
- void **insertPluginUI** ([Plugin](#) *plugin) const
Inserts one plugin entry into the categorized plugin tree representation.
- void **reloadFromPluginManager** () const
Rebuilds the categorized plugin tree from the current [PluginManager](#) state.
- void **applyCategoryExpansionPolicy** () const
Applies the default expansion policy for plugin category roots.
- void **showContextMenu** (const QPoint &pos) const
Shows the plugin-tree context menu used to reorder top-level categories.
- void **insertFakePlugins** () const
Inserts compatibility fake plugins used by legacy GUI flows.
- void **handlePluginItemDoubleClicked** (QTreeWidgetItem *item, int column) const
Handles plugin double-click by delegating insertion/selection behavior.
- void **handlePluginItemClicked** (QTreeWidgetItem *item, int column) const
Handles plugin single-click behavior without mutating model lifecycle state.
- void **moveTopLevelCategory** (QTreeWidgetItem *item, int delta) const
Moves a top-level category before or after its siblings.

10.374.1 Detailed Description

Controller responsible for plugin catalog presentation and plugin palette behavior.

This controller encapsulates the plugin-tree behavior used by [MainWindow](#) wrappers, keeping plugin browsing and insertion affordances consistent while reducing direct widget logic in the [MainWindow](#) compatibility façade.

Responsibilities:

- populate plugin entries grouped by category with legacy color conventions;
- provide compatibility helpers for fake-plugin insertion paths;
- react to plugin-tree click/double-click events delegated by [MainWindow](#).

Boundaries:

- it does not load plugin binaries or manage kernel plugin lifecycle;
- it does not own textual model synchronization beyond delegated editor insertions;
- it is a UI controller layer, not a persistence/export service.

The documentation for this class was generated from the following files:

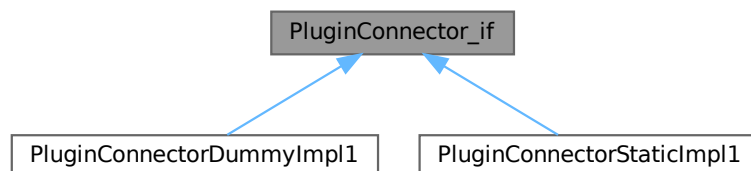
- applications/gui/genesys/controllers/PluginCatalogController.h
- applications/gui/genesys/controllers/PluginCatalogController.cpp

10.375 PluginConnector_if Class Reference

Interface for dynamic discovery/loading/unloading of plugin libraries.

```
#include <kernel/simulator/PluginConnector_if.h>
```

Inheritance diagram for PluginConnector_if:



Public Member Functions

- virtual [Plugin](#) * [check](#) (const std::string dynamicLibraryFilename)=0
check
- virtual [Plugin](#) * [connect](#) (const std::string dynamicLibraryFilename)=0
connect
- virtual [List](#)< std::string > * [find](#) ()=0
find dynamicLibraryFilenames
- virtual bool [disconnect](#) (const std::string dynamicLibraryFilename)=0
disconnect
- virtual bool [disconnect](#) ([Plugin](#) *plugin)=0
disconnect

10.375.1 Detailed Description

Interface for dynamic discovery/loading/unloading of plugin libraries.

Connector implementations bridge OS-level dynamic library handling and the kernel plugin abstraction, validating candidates and managing plugin lifecycles.

10.375.2 Member Function Documentation

10.375.2.1 [check\(\)](#)

```
virtual Plugin * PluginConnector_if::check (
    const std::string dynamicLibraryFilename ) [pure virtual]
```

[check](#)

Parameters

<i>dynamicLibraryFilename</i>	
-------------------------------	--

Returns

Validates whether a dynamic library contains a compatible plugin without permanently connecting it.

Implemented in [PluginConnectorDummyImpl1](#), and [PluginConnectorStaticImpl1](#).

10.375.2.2 [connect\(\)](#)

```
virtual Plugin * PluginConnector_if::connect (
    const std::string dynamicLibraryFilename ) [pure virtual]
```

[connect](#)

Parameters

<i>dynamicLibraryFilename</i>	
-------------------------------	--

Returns

Loads and connects a plugin from a dynamic library.

Implemented in [PluginConnectorDummyImpl1](#), and [PluginConnectorStaticImpl1](#).

10.375.2.3 disconnect() [1/2]

```
virtual bool PluginConnector_if::disconnect (
    const std::string dynamicLibraryFilename ) [pure virtual]
```

disconnect

Parameters

<i>dynamicLibraryFilename</i>	
-------------------------------	--

Returns

[List](#) of all Plugins found and connected

Disconnects the plugin associated with the provided library file.

Implemented in [PluginConnectorDummyImpl1](#), and [PluginConnectorStaticImpl1](#).

10.375.2.4 disconnect() [2/2]

```
virtual bool PluginConnector_if::disconnect (
    Plugin * plugin ) [pure virtual]
```

disconnect

Parameters

<i>plugin</i>	
---------------	--

Returns

Disconnects a previously connected plugin instance.

Implemented in [PluginConnectorDummyImpl1](#), and [PluginConnectorStaticImpl1](#).

10.375.2.5 find()

```
virtual List< std::string > * PluginConnector_if::find ( ) [pure virtual]
```

find dynamicLibraryFileNames

Parameters

--	--

return List of dynamicLibraryFileNames

Searches for candidate plugin libraries in the configured environment.

Implemented in [PluginConnectorDummyImpl1](#), and [PluginConnectorStaticImpl1](#).

The documentation for this class was generated from the following file:

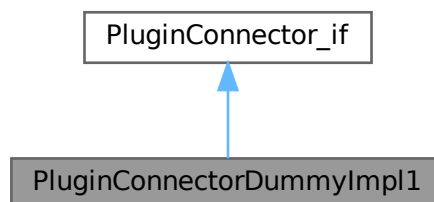
- kernel/simulator/PluginConnector_if.h

10.376 PluginConnectorDummyImpl1 Class Reference

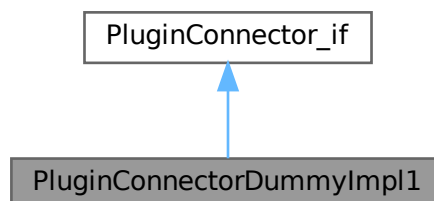
In-memory plugin connector that resolves built-in plugin metadata.

```
#include <plugins/PluginConnectorDummyImpl1.h>
```

Inheritance diagram for PluginConnectorDummyImpl1:



Collaboration diagram for PluginConnectorDummyImpl1:



Public Member Functions

- [Plugin](#) * [check](#) (const std::string dynamicLibraryFilename) override
Verifies whether a built-in plugin name can be resolved.
- [Plugin](#) * [connect](#) (const std::string dynamicLibraryFilename) override
Resolves a built-in plugin by filename alias.
- [List](#)< std::string > * [find](#) () override
Returns the list of built-in plugin filenames recognized by this connector.
- bool [disconnect](#) (const std::string dynamicLibraryFilename) override
Disconnects a plugin identified by filename.
- bool [disconnect](#) ([Plugin](#) *plugin) override
Disconnects a plugin instance.

10.376.1 Detailed Description

In-memory plugin connector that resolves built-in plugin metadata.

This connector does not load shared libraries. Instead, it maps known file names to compiled-in plugin classes and returns the corresponding metadata.

10.376.2 Member Function Documentation

10.376.2.1 [check\(\)](#)

```
Plugin * PluginConnectorDummyImpl1::check (
    const std::string dynamicLibraryFilename ) [override], [virtual]
```

Verifies whether a built-in plugin name can be resolved.

Parameters

<i>dynamicLibraryFilename</i>	Plugin library filename or alias.
-------------------------------	---

Returns

Metadata for the resolved plugin, or `nullptr` if unknown.

Implements [PluginConnector_if](#).

10.376.2.2 [connect\(\)](#)

```
Plugin * PluginConnectorDummyImpl1::connect (
    const std::string dynamicLibraryFilename ) [override], [virtual]
```

Resolves a built-in plugin by filename alias.

Parameters

<i>dynamicLibraryFilename</i>	Plugin library filename or alias.
-------------------------------	---

Returns

Metadata for the resolved plugin, or `nullptr` if unknown.

Implements [PluginConnector_if](#).

10.376.2.3 disconnect() [1/2]

```
bool PluginConnectorDummyImpl1::disconnect (
    const std::string dynamicLibraryFilename ) [override], [virtual]
```

Disconnects a plugin identified by filename.

Parameters

<i>dynamicLibraryFilename</i>	Plugin library filename or alias.
-------------------------------	---

Returns

Always `true` in the current dummy implementation.

Implements [PluginConnector_if](#).

10.376.2.4 disconnect() [2/2]

```
bool PluginConnectorDummyImpl1::disconnect (
    Plugin * plugin ) [override], [virtual]
```

Disconnects a plugin instance.

Parameters

<i>plugin</i>	Plugin instance to disconnect.
---------------	--

Returns

Always `true` in the current dummy implementation.

Implements [PluginConnector_if](#).

10.376.2.5 find()

```
List< std::string > * PluginConnectorDummyImpl1::find ( ) [override], [virtual]
```

Returns the list of built-in plugin filenames recognized by this connector.

Returns

Newly allocated list with known plugin aliases.

Implements [PluginConnector_if](#).

The documentation for this class was generated from the following files:

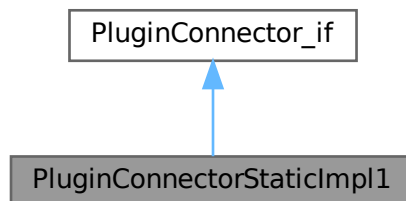
- plugins/PluginConnectorDummyImpl1.h
- kernel/simulator/PluginConnectorDummyBootstrap.cpp
- plugins/PluginConnectorDummyImpl1.cpp

10.377 PluginConnectorStaticImpl1 Class Reference

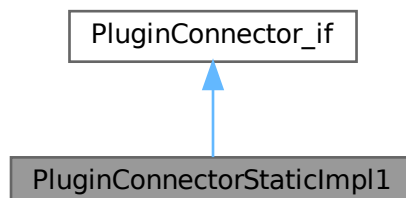
Static plugin connector stub used by the kernel interface.

```
#include <plugins/PluginConnectorStaticImpl1.h>
```

Inheritance diagram for PluginConnectorStaticImpl1:



Collaboration diagram for PluginConnectorStaticImpl1:



Public Member Functions

- [Plugin](#) * [check](#) (const std::string dynamicLibraryFilename) override
Checks whether a plugin library can be connected.
- [Plugin](#) * [connect](#) (const std::string dynamicLibraryFilename) override
Connects a plugin library.
- [List](#)< std::string > * [find](#) () override
Lists the available plugin libraries.
- bool [disconnect](#) (const std::string dynamicLibraryFilename) override
Disconnects a plugin by library filename.
- bool [disconnect](#) ([Plugin](#) *plugin) override
Disconnects a plugin instance.

10.377.1 Detailed Description

Static plugin connector stub used by the kernel interface.

This implementation currently provides a no-op connector that satisfies the plugin connector contract without loading dynamic libraries.

10.377.2 Member Function Documentation

10.377.2.1 [check\(\)](#)

```
Plugin * PluginConnectorStaticImpl1::check (
    const std::string dynamicLibraryFilename ) [override], [virtual]
```

Checks whether a plugin library can be connected.

Parameters

<i>dynamicLibraryFilename</i>	Path to the plugin library.
-------------------------------	-----------------------------

Returns

Always `nullptr` in the current static stub implementation.

Implements [PluginConnector_if](#).

10.377.2.2 [connect\(\)](#)

```
Plugin * PluginConnectorStaticImpl1::connect (
    const std::string dynamicLibraryFilename ) [override], [virtual]
```

Connects a plugin library.

Parameters

<i>dynamicLibraryFilename</i>	Path to the plugin library.
-------------------------------	-----------------------------

Returns

Always `nullptr` in the current static stub implementation.

Implements [PluginConnector_if](#).

10.377.2.3 disconnect() [1/2]

```
bool PluginConnectorStaticImpl1::disconnect (
    const std::string dynamicLibraryFilename ) [override], [virtual]
```

Disconnects a plugin by library filename.

Parameters

<i>dynamicLibraryFilename</i>	Path to the plugin library.
-------------------------------	-----------------------------

Returns

Always `true` in the current static stub implementation.

Implements [PluginConnector_if](#).

10.377.2.4 disconnect() [2/2]

```
bool PluginConnectorStaticImpl1::disconnect (
    Plugin * plugin ) [override], [virtual]
```

Disconnects a plugin instance.

Parameters

<i>plugin</i>	Plugin instance to disconnect.
---------------	--

Returns

Always `true` in the current static stub implementation.

Implements [PluginConnector_if](#).

10.377.2.5 find()

```
List< std::string > * PluginConnectorStaticImpl1::find ( ) [override], [virtual]
```

Lists the available plugin libraries.

Returns

Always `nullptr` in the current static stub implementation.

Implements [PluginConnector_if](#).

The documentation for this class was generated from the following files:

- `plugins/PluginConnectorStaticImpl1.h`
- `plugins/PluginConnectorStaticImpl1.cpp`

10.378 PluginInformation Class Reference

Public Member Functions

- **PluginInformation** (`std::string pluginTypename`, `StaticLoaderComponentInstance componentloader`, `StaticConstructorDataDefinitionInstance elementConstructor`)
- **PluginInformation** (`std::string pluginTypename`, `StaticLoaderDataDefinitionInstance elementloader`, `StaticConstructorDataDefinitionInstance elementConstructor`)
- `StaticLoaderDataDefinitionInstance` **getDataDefinitionLoader** () const
- `StaticLoaderComponentInstance` **GetComponentLoader** () const
- `StaticConstructorDataDefinitionInstance` **getDataDefinitionConstructor** () const
- `bool` **isGenerateReport** () const
- `bool` **isComponent** () const
- `bool` **isSendTransfer** () const
- `bool` **isReceiveTransfer** () const
- `bool` **isSink** () const
- `bool` **isSource** () const
- `std::string` **getObservation** () const
- `std::string` **getVersion** () const
- `std::string` **getDate** () const
- `std::string` **getAuthor** () const
- `std::string` **getPluginTypename** () const
- `void` **insertDynamicLibFileDependence** (`std::string filename`)
- `void` **setDynamicLibFilenameDependencies** (`std::list< std::string > *dynamicLibFilenameDependencies`)
- `std::list< std::string > *` **getDynamicLibFilenameDependencies** () const
- `void` **insertSystemDependency** (`const SystemDependency &dependency`)
- `const std::list< SystemDependency > *` **getSystemDependencies** () const
- `bool` **hasSystemDependencies** () const
- `void` **setGenerateReport** (`bool generateReport`)
- `void` **setSendTransfer** (`bool sendTransfer`)
- `void` **setReceiveTransfer** (`bool receiveTransfer`)
- `void` **setSink** (`bool Sink`)
- `void` **setSource** (`bool Source`)
- `void` **setObservation** (`std::string observation`)
- `void` **setVersion** (`std::string version`)
- `void` **setDate** (`std::string date`)
- `void` **setAuthor** (`std::string author`)
- `void` **setMaximumOutputs** (`unsigned short _maximumOutputs`)
- `unsigned short` **getMaximumOutputs** () const
- `void` **setMinimumOutputs** (`unsigned short _minimumOutputs`)
- `unsigned short` **getMinimumOutputs** () const
- `void` **setMaximumInputs** (`unsigned short _maximumInputs`)

- unsigned short **getMaximumInputs** () const
- void **setMinimumInputs** (unsigned short _minimumInputs)
- unsigned short **getMinimumInputs** () const
- void **setDescriptionHelp** (std::string _descriptionHelp)
- std::string **getDescriptionHelp** () const
- void **setFields** (std::map< std::string, std::string > *_fiewlds)
- std::map< std::string, std::string > * **getFields** () const
- void **setLanguageTemplate** (std::string _languageTemplate)
- std::string **getLanguageTemplate** () const
- void **setCategory** (std::string _category)
- std::string **getCategory** () const

Static Public Member Functions

- static std::string **categoryFolderName** (const std::string &category)

The documentation for this class was generated from the following files:

- kernel/simulator/PluginInformation.h
- kernel/simulator/PluginInformation.cpp

10.379 PluginInsertionOptions Struct Reference

Optional policy hooks used while inserting a plugin.

```
#include <kernel/simulator/PluginManager.h>
```

Public Attributes

- std::function< bool(const [SystemDependencyCheckResult](#) &)> **confirmSystemDependencyInstallation**
Called before running any install command for missing system dependencies.

10.379.1 Detailed Description

Optional policy hooks used while inserting a plugin.

The kernel reports dependency diagnostics through this structure and leaves user confirmation to the caller. An empty callback means non-interactive insertion: missing dependencies are reported through traces and no install command is executed.

The documentation for this struct was generated from the following file:

- kernel/simulator/PluginManager.h

10.380 PluginLoadIssue Class Reference

Diagnostic record for a plugin candidate that could not be loaded.

```
#include <kernel/simulator/PluginManager.h>
```

Public Types

- enum class [Reason](#) {
InvalidPlugin , **MissingSystemDependency** , **DynamicDependencyFailure** , **ConnectionFailure** ,
InsertionFailure , **Exception** }

High-level reason why a plugin candidate was not loaded.

Public Member Functions

- **PluginLoadIssue** ()
Creates an empty diagnostic record.
- **PluginLoadIssue** (std::string filename, std::string pluginTypename, [Reason](#) reason, std::string message, [SystemDependencyCheckResult](#) systemDependencyResult={}) noexcept
Creates a diagnostic record for a failed plugin load attempt.
- const std::string & **getFilename** () const
Returns the filename passed to the plugin connector.
- const std::string & **getPluginTypename** () const
Returns the plugin typename when it could be read from [PluginInformation](#).
- [Reason](#) **getReason** () const
Returns the high-level failure reason.
- const std::string & **getMessage** () const
Returns the human-readable diagnostic message captured by the manager.
- const [SystemDependencyCheckResult](#) & **getSystemDependencyResult** () const
Returns dependency diagnostics captured during system dependency preflight.
- bool **hasSystemDependencyResult** () const
Returns true when this issue includes system dependency diagnostics.
- std::string **diagnosticText** () const
Formats the issue for trace logs or GUI details.

Static Public Member Functions

- static std::string **reasonToString** ([Reason](#) reason)
Returns a stable textual label for a failure reason.

10.380.1 Detailed Description

Diagnostic record for a plugin candidate that could not be loaded.

The manager keeps these records so interactive front-ends can explain startup autoload failures after the application UI is ready. The kernel stores only technical diagnostics; GUI code decides how to present and resolve them.

The documentation for this class was generated from the following files:

- kernel/simulator/PluginManager.h
- kernel/simulator/PluginManager.cpp

10.381 PluginManager Class Reference

Manages discovery, loading, lookup and instantiation of simulation plugins.

```
#include <kernel/simulator/PluginManager.h>
```

Public Member Functions

- **PluginManager** ([Simulator](#) *simulator)
Creates a plugin manager bound to a simulator instance.
- **PluginManager** ([Simulator](#) *simulator, [PluginConnector_if](#) *pluginConnector, [SystemCommandExecutor_if](#) *systemCommandExecutor)
Creates a plugin manager with injectable connector and command executor.
- virtual ~**PluginManager** ()
Destroys owned plugin wrappers and connector resources.
- std::string **show** ()
Returns a human-readable summary of currently connected plugins.
- [List](#)< [Plugin](#) * > * **completePluginsFieldsAndTemplates** ()
Completes plugin metadata (fields/templates) after loading.
- bool **check** (const std::string &dynamicLibraryFilename)
Validates whether a dynamic library provides a compatible plugin.
- [SystemDependencyCheckResult](#) **checkSystemDependencies** (const std::string &dynamicLibraryFilename)
Evaluates system dependencies declared by a dynamic-library plugin without inserting it.
- [List](#)< std::string > * **discoverPluginFilenames** () const
Discovers plugin filenames through the configured connector without inserting them.
- [List](#)< [PluginLoadIssue](#) > * **getPluginLoadIssues** () const
Returns plugin candidates that failed to load during check/connect/preflight.
- void **clearPluginLoadIssues** ()
Clears all stored plugin load diagnostics.
- void **clearPluginLoadIssue** (const std::string &dynamicLibraryFilename)
Removes stored diagnostics for one plugin candidate filename.
- [Plugin](#) * **insert** (const std::string &dynamicLibraryFilename)
Loads and inserts a plugin from a dynamic library file.
- [Plugin](#) * **insert** (const std::string &dynamicLibraryFilename, const [PluginInsertionOptions](#) &options)
Loads and inserts a plugin, optionally asking the caller to authorize dependency installation.
- bool **remove** (const std::string &dynamicLibraryFilename)
Removes/disconnects a plugin by its dynamic library filename.
- bool **remove** ([Plugin](#) *plugin)
Removes/disconnects a plugin by pointer.
- [Plugin](#) * **find** (const std::string &pluginTypeName)
Finds a connected plugin by plugin type name.
- std::vector< std::string > **getDataDefinitionPluginTypeNames** () const
Returns connected plugin type names that can create [ModelDataDefinition](#) instances.
- std::string **sourceIncludePathFor** (const std::string &pluginTypeName)
Returns the source include path for a connected plugin type.
- [List](#)< [Plugin](#) * > * **autoInsertPlugins** (const std::string &pluginsListFilename, const bool lookForPluginsIf←
FilenameNotFound=true)
Auto-loads plugins listed in file (or discovered automatically as fallback).
- [List](#)< [Plugin](#) * > * **autoInsertPlugins** (const std::string &pluginsListFilename, const bool lookForPluginsIf←
FilenameNotFound, const [PluginInsertionOptions](#) &options)

- Auto-loads plugins listed in file using insertion policy hooks.*
- [List](#)< [Plugin](#) * > * **autoInsertPlugins** ()
Auto-loads plugins discovered automatically).
- [List](#)< [Plugin](#) * > * **autoInsertPlugins** (const [PluginInsertionOptions](#) &options)
Auto-loads plugins discovered automatically using insertion policy hooks.
- [Plugin](#) * **front** ()
Returns the first plugin in the internal plugin list.
- [Plugin](#) * **next** ()
Returns the next plugin using the internal iterator cursor.
- [Plugin](#) * **last** ()
Returns the last plugin in the internal plugin list.
- unsigned int **size** ()
Returns the number of connected plugins.
- [Plugin](#) * **getAtRank** (unsigned int rank)
Returns the plugin at a specific list rank.
- [ModelDataDefinition](#) * **newInstance** (const std::string &pluginTypename, [Model](#) *model, std::string name="")
Creates a new model data/component instance using a plugin typename.
- template<typename T >
T * **newInstance** ([Model](#) *model, std::string name="")

10.381.1 Detailed Description

Manages discovery, loading, lookup and instantiation of simulation plugins.

The plugin manager centralizes dynamic-library checks/connections through a [PluginConnector_if](#) and keeps the in-memory list of connected plugins used by the simulator to create model data/component instances.

10.381.2 Constructor & Destructor Documentation

10.381.2.1 PluginManager()

```
PluginManager::PluginManager (
    Simulator * simulator,
    PluginConnector\_if * pluginConnector,
    SystemCommandExecutor\_if * systemCommandExecutor )
```

Creates a plugin manager with injectable connector and command executor.

Ownership of both injected pointers is transferred to the manager. This constructor exists primarily for unit tests and narrow integration seams.

10.381.3 Member Function Documentation

10.381.3.1 getDataDefinitionPluginTypenames()

```
std::vector< std::string > PluginManager::getDataDefinitionPluginTypenames ( ) const
```

Returns connected plugin type names that can create [ModelDataDefinition](#) instances.

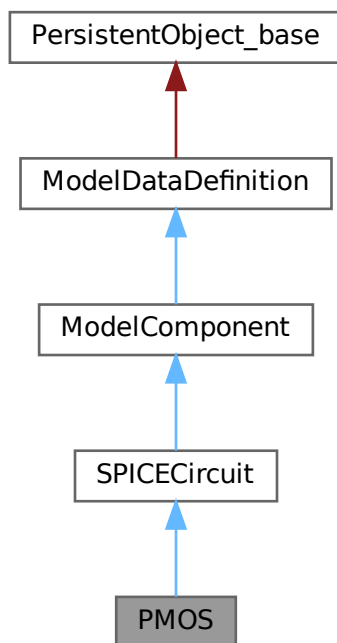
Polymorphic data structures such as [Set](#) use this snapshot to expose concrete, creatable element types to editors without hard-coding plugin names in the GUI.

The documentation for this class was generated from the following files:

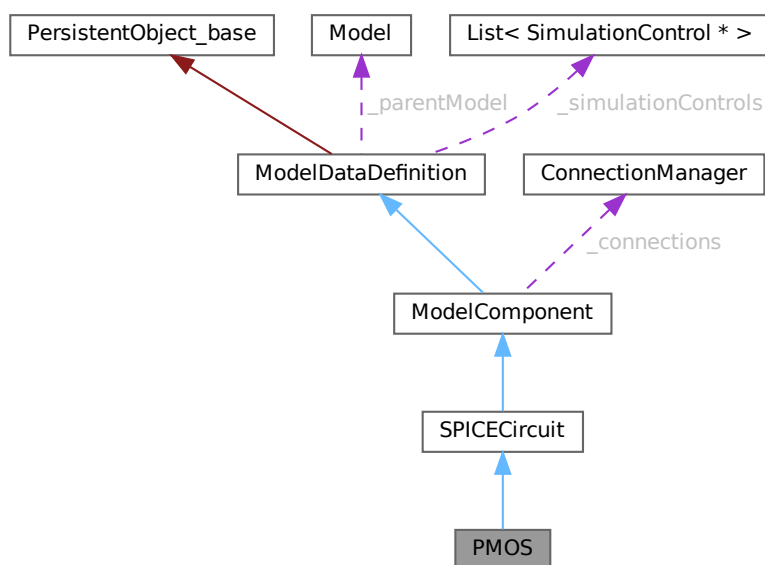
- kernel/simulator/PluginManager.h
- kernel/simulator/PluginManager.cpp

10.382 PMOS Class Reference

Inheritance diagram for PMOS:



Collaboration diagram for PMOS:



Public Member Functions

- **PMOS** ([Model](#) *model, std::string name="")
- void [Build](#) () override
- void **setWidth** (float width)
- void **setLength** (float length)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string [show](#) () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const

- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) * **GetPluginInformation** ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
- static std::vector< std::string > **split** (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void **_createNonEditableDataDefinitions** () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual bool **_check** (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_initBetweenReplications** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)

- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.382.1 Member Function Documentation

10.382.1.1 [_loadInstance\(\)](#)

```
bool PMOS::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.382.1.2 `_saveInstance()`

```
void PMOS::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.382.1.3 Build()

```
void PMOS::Build ( ) [override], [virtual]
```

Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

10.383 GroProgramRuntime::PopulationMutation Struct Reference

Public Attributes

- PopulationMutationType **type** = PopulationMutationType::Grow
- unsigned int **value** = 0
- unsigned int **previousPopulationSize** = 0
- unsigned int **resultingPopulationSize** = 0

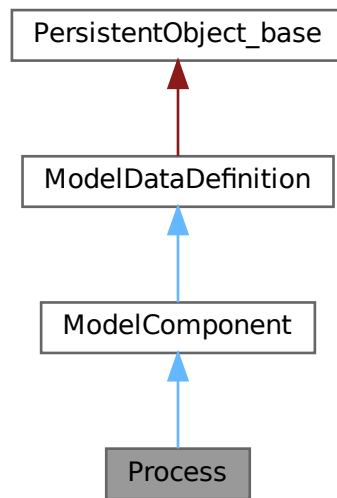
The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/GroProgramRuntime.h

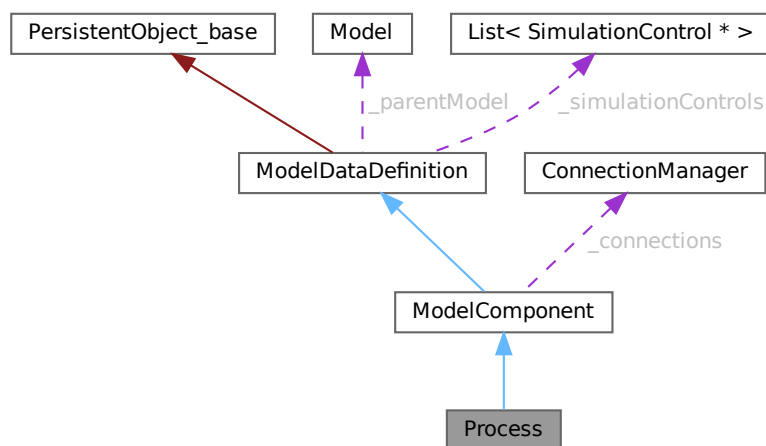
10.384 Process Class Reference

```
#include <plugins/components/DiscreteProcessing/Process.h>
```

Inheritance diagram for Process:



Collaboration diagram for Process:



Public Member Functions

- **Process** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- void **setPriority** (unsigned short _priority)
- unsigned short **getPriority** () const

- void **setPriorityExpression** (std::string priority)
- std::string **getPriorityExpression** () const
- void **setAllocationType** (Util::AllocationType _allocationType)
- Util::AllocationType **getAllocationType** () const
- List< SeizableItem * > * **getSeizeRequests** () const
- void **addSeizeRequest** (SeizableItem *newRequest)
- void **removeSeizeRequest** (SeizableItem *request)
- void **setQueueableItem** (QueueableItem *_queueableItem)
- QueueableItem * **getQueueableItem** () const
- void **setDelayExpression** (std::string _delayExpression)
- void **setDelayExpression** (std::string _delayExpression, Util::TimeUnit _delayTimeUnit)
- std::string **delayExpression** () const
- void **setDelayTimeUnit** (Util::TimeUnit _delayTimeUnit)
- Util::TimeUnit **delayTimeUnit** () const

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

- Returns the user-visible label of this instance.*

 - `const std::string & getClassname () const`

Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`

Returns the parent model that owns this definition.
- `bool isReportStatistics () const`

Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`

Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`

Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.
- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.
- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`

Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`

Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`

Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createNonEditableDataDefinitions](#) () override
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.384.1 Detailed Description

This component ...

10.384.2 Member Function Documentation

10.384.2.1 `_check()`

```
bool Process::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.384.2.2 `_createEditableDataDefinitions()`

```
void Process::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.384.2.3 `_createNonEditableDataDefinitions()`

```
void Process::_createNonEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.384.2.4 `_loadInstance()`

```
bool Process::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.384.2.5 `_onDispatchEvent()`

```
void Process::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.384.2.6 `_saveInstance()`

```
void Process::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.384.2.7 `show()`

```
std::string Process::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

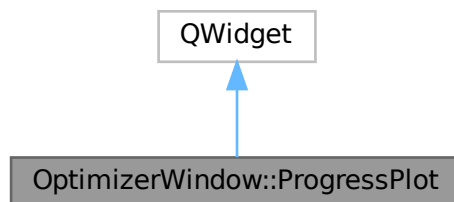
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

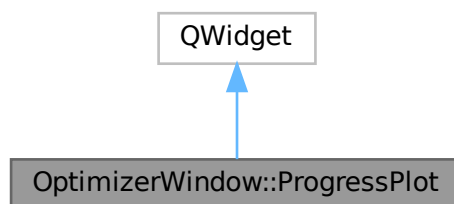
- plugins/components/DiscreteProcessing/Process.h
- plugins/components/DiscreteProcessing/Process.cpp

10.385 OptimizerWindow::ProgressPlot Class Reference

Inheritance diagram for OptimizerWindow::ProgressPlot:



Collaboration diagram for OptimizerWindow::ProgressPlot:



Public Member Functions

- **ProgressPlot** (`QWidget *parent=nullptr`)
- void **setValues** (`const QVector< double > &values`)

Protected Member Functions

- void **paintEvent** (QPaintEvent *) override

The documentation for this class was generated from the following file:

- applications/gui/genesys/tools/optimizer/OptimizerWindow.cpp

10.386 BioStateProjectionComponent::Projection Struct Reference

Public Attributes

- std::string **sourceSpeciesName**
- std::string **targetKey**
- ProjectionKind **kind** = ProjectionKind::MoleculeCount
- double **scale** = 1.0
- double **offset** = 0.0

The documentation for this struct was generated from the following file:

- plugins/components/WholeCellModeling/BioStateProjectionComponent.h

10.387 MetabolicStateProjectionComponent::Projection Struct Reference

Public Attributes

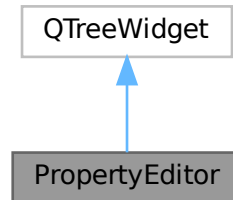
- std::string **reactionName**
- std::string **targetKey**
- ProjectionKind **kind** = ProjectionKind::MetaboliteAmount
- ProjectionUpdateMode **updateMode** = ProjectionUpdateMode::Overwrite
- double **scale** = 1.0
- double **offset** = 0.0
- double **turnoverFraction** = 0.0
- std::string **compartmentName**

The documentation for this struct was generated from the following file:

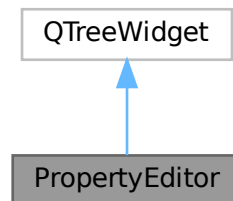
- plugins/components/WholeCellModeling/MetabolicStateProjectionComponent.h

10.388 PropertyEditor Class Reference

Inheritance diagram for PropertyEditor:



Collaboration diagram for PropertyEditor:



Public Member Functions

- **PropertyEditor** (QWidget *parent=nullptr)
- void **setModelBlock** ([ModelDataDefinition](#) *modelblock)

10.388.1 Member Function Documentation

10.388.1.1 setModelBlock()

```
void PropertyEditor::setModelBlock (  
    ModelDataDefinition * modelblock )
```

!@TODO Era para ser getClassname());

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/PropertyEditor.h
- applications/gui/genesys/propertyeditor/PropertyEditor.cpp

10.389 PropertyEditorController Class Reference

Controller that orchestrates property editor state and scene-selection coupling.

```
#include <applications/gui/genesys/controllers/PropertyEditorController.h>
```

Public Member Functions

- **PropertyEditorController** ([ObjectPropertyBrowser](#) *propertyBrowser, [ModelGraphicsView](#) *graphicsView, [PropertyEditorGenesys](#) *propertyGenesys, std::map< [SimulationControl](#) *, [DataComponentProperty](#) * > *propertyList, std::map< [SimulationControl](#) *, [DataComponentEditor](#) * > *propertyEditorUI, std::map< [SimulationControl](#) *, [ComboBoxEnum](#) * > *propertyBox, std::function< void()> actualizeModelSimLanguage, std::function< void(bool)> actualizeModelComponents, std::function< void(bool)> actualizeModelDataDefinitions, std::function< void()> actualizeModelCppCode, std::function< bool()> createModelImage, std::function< void()> actualizeTabPanels, std::function< void()> actualizeActions)
Creates the property-editor controller and its synchronization callbacks.
- void **sceneSelectionChanged** () const
Synchronizes current scene selection into property editor bindings.
- void **onPropertyEditorModelChanged** () const
Executes post-commit synchronization across model representations and panes.
- void **clearPropertyEditorSelection** () const
Clears property editor selection/bindings during lifecycle resets.
- void **bindDataDefinitionFromInspector** ([ModelDataDefinition](#) *dataDefinition, QObject *graphicalObject=nullptr) const
Binds a DataDefinition selected from the inspector tree to the property editor.
- bool **isPostCommitPipelineActive** () const
Reports whether the deferred global refresh pipeline is active.

10.389.1 Detailed Description

Controller that orchestrates property editor state and scene-selection coupling.

This Phase-6 extraction isolates property editor wiring from [MainWindow](#) while preserving the existing compatibility façade methods. It coordinates selection-driven property binding and the deferred refresh cascade required after property commits.

Responsibilities:

- synchronize current scene selection into PropertyEditorGenesys/ObjectPropertyBrowser;
- execute post-edit refresh callbacks for model language, trees, C++ export, and tabs;
- coalesce heavy refresh requests to avoid re-entrant UI update storms.

Boundaries:

- it does not own property objects/editors lifetime (maps are injected);
- it does not alter undo/redo infrastructure or scene command orchestration;
- it does not persist models or execute simulation commands.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/PropertyEditorController.h
- applications/gui/genesys/controllers/PropertyEditorController.cpp

10.390 PropertyEditorGenesys Class Reference

Public Member Functions

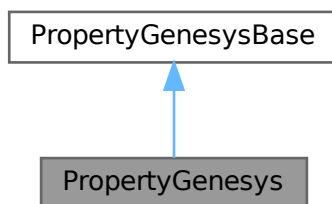
- `std::list< ModelComponent * > getElements ()`
- `void changeSimulationControl (SimulationControl *control, const std::string &value, bool remove=false)`
- `SimulationControl * findSimulationControl (const std::string &id, const std::string &attribute)`
- `void addElement (ModelComponent *component)`

The documentation for this class was generated from the following file:

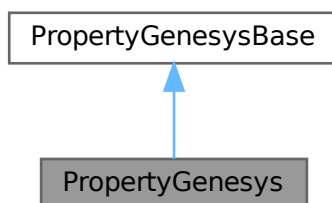
- `kernel/simulator/PropertyGenesys.h`

10.391 PropertyGenesys Class Reference

Inheritance diagram for PropertyGenesys:



Collaboration diagram for PropertyGenesys:



Public Member Functions

- **PropertyGenesys** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", std::string propertyId="", std::string toolTip="", std::string statusTip="")
- std::list< [PropertyGenesys](#) * > **subProperties** () const
- std::string **toolTip** () const
- std::string **statusTip** () const
- std::string **propertyName** () const
- std::string **propertyId** () const
- bool **isEnabled** () const
- bool **isModified** () const
- bool **isSubProperty** () const
- bool **hasValue** () const
- std::string **valueText** () const
- void **setToolTip** (const std::string text)
- void **setStatusTip** (const std::string text)
- void **setWhatsThis** (const std::string text)
- void **setPropertyName** (const std::string text)
- void **setPropertyId** (const std::string text)
- void **setEnabled** (bool enable)
- void **setModified** (bool modified)
- virtual bool **compare** ([PropertyGenesys](#) *otherProperty) const
- void **addSubProperty** ([PropertyGenesys](#) *property)
- void **insertSubProperty** ([PropertyGenesys](#) *property, [PropertyGenesys](#) *afterProperty)
- void **removeSubProperty** ([PropertyGenesys](#) *property)

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Protected Member Functions

- void **propertyChanged** ()

Protected Attributes

- std::string **_className**
- std::string **_propertyName**
- std::string **_whatsThis**
- std::string **_propertyId**
- std::string **_toolTip**
- std::string **_statusTip**
- std::list< [PropertyGenesys](#) * > **_subProperties**
- bool **_isEnabled**
- bool **_isModified**
- bool **_hasValue**
- bool **_isSubProperty**

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

The documentation for this class was generated from the following file:

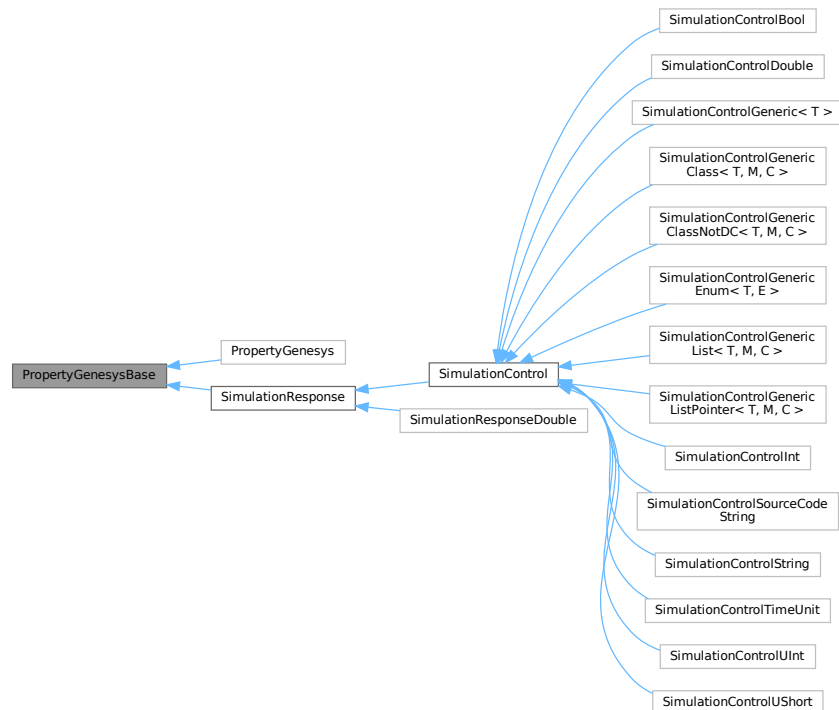
- kernel/simulator/SimulationControlAndResponse.h

10.392 PropertyGenesysBase Class Reference

Base metadata for kernel-side control/response abstractions.

```
#include <kernel/simulator/SimulationControlAndResponse.h>
```

Inheritance diagram for PropertyGenesysBase:



Public Member Functions

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Protected Attributes

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.392.1 Detailed Description

Base metadata for kernel-side control/response abstractions.

Historically, the experiment layer in GenESyS evolved from the basic [ModelSimulation](#) controls toward a more generic mechanism capable of exposing arbitrary getters and setters from model-related classes.

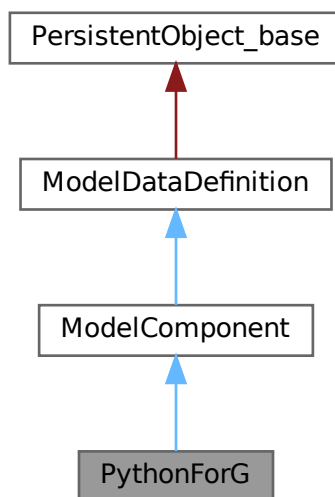
Despite the legacy name, this base currently lives in the simulation kernel and supports kernel-side experiment/control abstractions rather than a user-interface property editor.

The documentation for this class was generated from the following file:

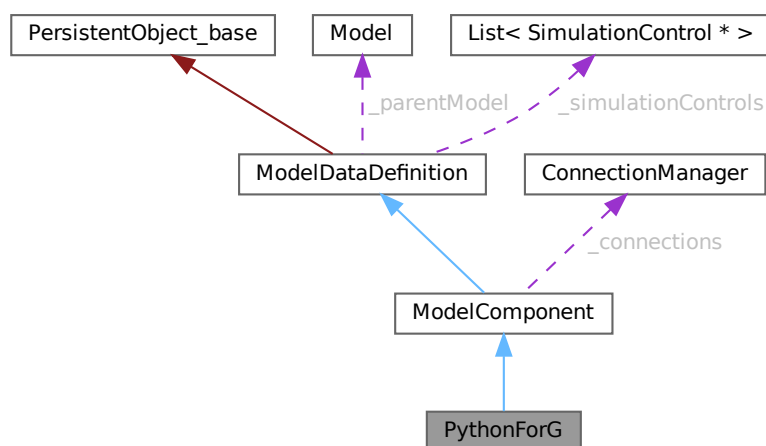
- kernel/simulator/SimulationControlAndResponse.h

10.393 PythonForG Class Reference

Inheritance diagram for PythonForG:



Collaboration diagram for PythonForG:



Public Member Functions

- **PythonForG** ([Model](#) *model, std::string name="")
- void **setInitBetweenReplicationCode** (std::string initBetweenReplicationCode)

- `std::string getInitBetweenReplicationCode ()` const
- void `setInitBetweenReplicationCodeProperty (SourceCodeString initBetweenReplicationCode)`
- `SourceCodeString getInitBetweenReplicationCodeProperty ()` const
- void `setOnDispatchEventCode (std::string onDispatchEventCode)`
- `std::string getOnDispatchEventCode ()` const
- void `setOnDispatchEventCodeProperty (SourceCodeString onDispatchEventCode)`
- `SourceCodeString getOnDispatchEventCodeProperty ()` const
- void `setForwardEntityOnError (bool forwardEntityOnError)`
- bool `isForwardEntityOnError ()` const
- `PythonRuntime * getPythonRuntime ()` const
- virtual `std::string show ()` override

virtual public methods

Public Member Functions inherited from `ModelComponent`

- `ModelComponent (Model *model, std::string componentTypename, std::string name="")`
constructors
- virtual `~ModelComponent ()`
Releases component-owned resources and unregisters it from the model.
- `ConnectionManager * getConnectionManager ()` const
new public user methods for this component
- void `connectTo (ModelComponent *component, unsigned int inputPortNumber=0)`
Connects this component to another component.
- bool `hasBreakpointAt ()`
Indicates whether a simulation breakpoint is set on this component.
- void `setDescription (std::string _description)`
Updates the free-form description shown by the GUI.
- `std::string getDescription ()` const
Returns the free-form description associated with this component.
- void `remove (ModelComponent *comp)`
Removes a connected component from this component.

Public Member Functions inherited from `ModelDataDefinition`

- `ModelDataDefinition (Model *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)`
Creates a model data definition and optionally inserts it into the model.
- virtual `~ModelDataDefinition ()`
Releases resources owned by this data definition.
- `Util::identification getId ()` const
Returns the unique identifier of this instance.
- void `setName (const std::string &name)`
Updates the logical name of this instance.
- const `std::string & getName ()` const
Returns the logical name of this instance.
- void `setLabel (const std::string &label)`
Updates the user-visible label of this instance.
- const `std::string & getLabel ()` const
Returns the user-visible label of this instance.
- const `std::string & getClassname ()` const

- Returns the runtime class name used by the plugin system.*
- **Model * getParentModel () const**
Returns the parent model that owns this definition.
- **bool isReportStatistics () const**
Indicates whether this data definition contributes report statistics.
- **void setReportStatistics (bool reportStatistics)**
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition * getInternalData (const std::string &name) const**
Looks up a named internal child data definition.
- **std::map< std::string, ModelDataDefinition * > * getInternalData () const**
Returns the full map of internal child data definitions.
- **std::map< std::string, ModelDataDefinition * > * getAttachedData () const**
Returns the full map of attached data definitions.
- **bool hasChanged () const**
Indicates whether this instance has been modified since the last reset.
- **void setHasChanged (bool hasChanged)**
Updates the changed flag for this data definition.
- **unsigned int getLevel () const**
Returns the level in the model hierarchy where this definition lives.
- **void setModelLevel (unsigned int _modelLevel)**
Sets the model nesting level for this definition.
- **List< SimulationControl * > * getSimulationControls () const**
Returns the simulation controls owned by this data definition.
- **TraceManager::Level getTraceLevelSpecific () const**
Returns the trace level that applies specifically to this definition.
- **void setTraceLevelSpecific (TraceManager::Level level)**
Updates the trace level used by this definition.
- **void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)**
Defines the specific trace level and optionally enables it immediately.
- **bool isTraceLevelSpecificEnabled () const**
Indicates whether the specific trace level is enabled.
- **void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)**
Enables or disables the specific trace level.

Static Public Member Functions

- **static PluginInformation * GetPluginInformation ()**
- **static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)**
- **static ModelDataDefinition * NewInstance (Model *model, std::string name="")**

Static Public Member Functions inherited from ModelComponent

- **static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)**
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- **static bool Check (ModelComponent *component)**
Validates a component instance.
- **static void CreateInternalData (ModelComponent *component)**
Creates internal data required by a component instance.
- **static void SaveInstance (PersistenceRecord *fields, ModelComponent *component)**
Serializes a component instance into a persistence record.
- **static void DispatchEvent (Event *event)**
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createNonEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.393.1 Member Function Documentation

10.393.1.1 `_check()`

```
bool PythonForG::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.393.1.2 `_createNonEditableDataDefinitions()`

```
void PythonForG::_createNonEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.393.1.3 `_initBetweenReplications()`

```
void PythonForG::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.393.1.4 `_loadInstance()`

```
bool PythonForG::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.393.1.5 `_onDispatchEvent()`

```
void PythonForG::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.393.1.6 `_saveInstance()`

```
void PythonForG::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.393.1.7 `show()`

```
std::string PythonForG::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

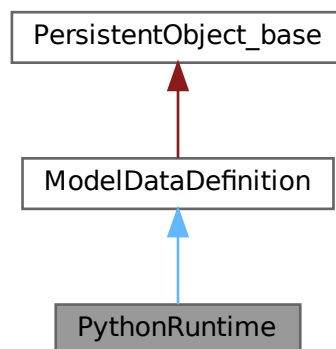
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/ExternalIntegration/PythonForG.h
- plugins/components/ExternalIntegration/PythonForG.cpp

10.394 PythonRuntime Class Reference

Inheritance diagram for PythonRuntime:



- Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & [getName](#) () const
- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
- Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const
- Returns the runtime class name used by the plugin system.*
- [Model](#) * [getParentModel](#) () const
- Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const
- Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)
- Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
- Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
- Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
- Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const
- Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))
- Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const
- Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int _modelLevel)
- Sets the model nesting level for this definition.*
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
- Returns the simulation controls owned by this data definition.*
- TraceManager::Level [getTraceLevelSpecific](#) () const
- Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) (TraceManager::Level level)
- Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
- Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const
- Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
- Enables or disables the specific trace level.*

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.394.1 Member Function Documentation

10.394.1.1 **_check()**

```
bool PythonRuntime::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.394.1.2 `_loadInstance()`

```
bool PythonRuntime::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.394.1.3 `_saveInstance()`

```
void PythonRuntime::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.394.1.4 `show()`

```
std::string PythonRuntime::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

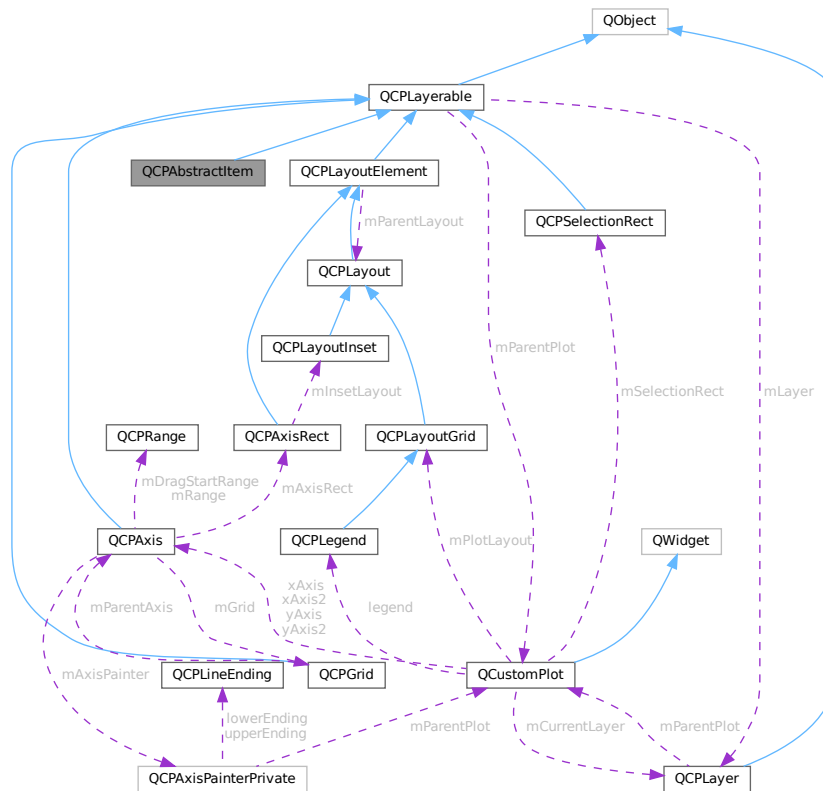
Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/ExternalIntegration/PythonRuntime.h
- plugins/data/ExternalIntegration/PythonRuntime.cpp

Collaboration diagram for QCPAbstractItem:



Signals

- void [selectionChanged](#) (bool selected)
- void **selectableChanged** (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

Public Member Functions

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool **clipToAxisRect** () const
- [QCPAxisRect](#) * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void [setClipToAxisRect](#) (bool clip)
- void [setClipAxisRect](#) ([QCPAxisRect](#) *rect)
- Q_SLOT void [setSelectable](#) (bool selectable)
- Q_SLOT void [setSelected](#) (bool selected)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_[↔](#)
DECL_OVERRIDE=0

- `QList< QCPLayerable * > positions () const`
- `QList< QCPLayerable * > anchors () const`
- `QCPLayerable * position (const QString &name) const`
- `QCPLayerable * anchor (const QString &name) const`
- `bool hasAnchor (const QString &name) const`

Public Member Functions inherited from QCPLayerable

- `QCPLayerable (QCustomPlot *plot, QString targetLayer=QString(), QCPLayerable *parentLayerable=nullptr)`
- `bool visible () const`
- `QCustomPlot * parentPlot () const`
- `QCPLayerable * parentLayerable () const`
- `QCPLayer * layer () const`
- `bool antialiased () const`
- `void setVisible (bool on)`
- `Q_SLOT bool setLayer (QCPLayer *layer)`
- `bool setLayer (const QString &layerName)`
- `void setAntialiased (bool enabled)`
- `bool realVisibility () const`

Protected Member Functions

- `virtual QCP::Interaction selectionCategory () const Q_DECL_OVERRIDE`
- `virtual QRect clipRect () const Q_DECL_OVERRIDE`
- `virtual void applyDefaultAntialiasingHint (QCPPainter *painter) const Q_DECL_OVERRIDE`
- `virtual void draw (QCPPainter *painter) Q_DECL_OVERRIDE=0`
- `virtual void selectEvent (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE`
- `virtual void deselectEvent (bool *selectionStateChanged) Q_DECL_OVERRIDE`
- `virtual QPointF anchorPixelPosition (int anchorId) const`
- `double rectDistance (const QRectF &rect, const QPointF &pos, bool filledRect) const`
- `QCPLayerable * createPosition (const QString &name)`
- `QCPLayerable * createAnchor (const QString &name, int anchorId)`

Protected Member Functions inherited from QCPLayerable

- `virtual void parentPlotInitialized (QCustomPlot *parentPlot)`
- `virtual void mousePressEvent (QMouseEvent *event, const QVariant &details)`
- `virtual void mouseMoveEvent (QMouseEvent *event, const QPointF &startPos)`
- `virtual void mouseReleaseEvent (QMouseEvent *event, const QPointF &startPos)`
- `virtual void mouseDoubleClickEvent (QMouseEvent *event, const QVariant &details)`
- `virtual void wheelEvent (QWheelEvent *event)`
- `void initializeParentPlot (QCustomPlot *parentPlot)`
- `void setParentLayerable (QCPLayerable *parentLayerable)`
- `bool moveToLayer (QCPLayer *layer, bool prepend)`
- `void applyAntialiasingHint (QCPPainter *painter, bool localAntialiased, QCP::AntialiasedElement overrideElement) const`

Protected Attributes

- bool **mClipToAxisRect**
- QPointer< [QCPAxisRect](#) > **mClipAxisRect**
- QList< [QCPItemPosition](#) * > **mPositions**
- QList< [QCPItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPItemAnchor](#)

10.395.1 Detailed Description

The abstract base class for all items in a plot.

In [QCustomPlot](#), items are supplemental graphical elements that are neither plottables ([QCPAbstractPlottable](#)) nor axes ([QCPAxis](#)). While plottables are always tied to two axes and thus plot coordinates, items can also be placed in absolute coordinates independent of any axes. Each specific item has at least one [QCPItemPosition](#) member which controls the positioning. Some items are defined by more than one coordinate and thus have two or more [QCPItemPosition](#) members (For example, [QCPItemRect](#) has *topLeft* and *bottomRight*).

This abstract base class defines a very basic interface like visibility and clipping. Since this class is abstract, it can't be instantiated. Use one of the subclasses or create a subclass yourself to create new items.

The built-in items are:

QCPItemLine	A line defined by a start and an end point. May have different ending styles on each side (e.g. arrows).
QCPItemStraightLine	A straight line defined by a start and a direction point. Unlike QCPItemLine , the straight line is infinitely long and has no endings.
QCPItemCurve	A curve defined by start, end and two intermediate control points. May have different ending styles on each side (e.g. arrows).
QCPItemRect	A rectangle
QCPItemEllipse	An ellipse
QCPItemPixmap	An arbitrary pixmap
QCPItemText	A text label
QCPItemBracket	A bracket which may be used to reference/highlight certain parts in the plot.
QCPItemTracer	An item that can be attached to a QCPGraph and sticks to its data points, given a key coordinate.

10.395.2 Clipping

Items are by default clipped to the main axis rect (they are only visible inside the axis rect). To make an item visible outside that axis rect, disable clipping via [setClipToAxisRect\(false\)](#).

On the other hand if you want the item to be clipped to a different axis rect, specify it via [setClipAxisRect](#). This `clipAxisRect` property of an item is only used for clipping behaviour, and in principle is independent of the coordinate axes the item might be tied to via its position members ([QCPItemPosition::setAxes](#)). However, it is common that the axis rect for clipping also contains the axes used for the item positions.

10.395.3 Using items

First you instantiate the item you want to use and add it to the plot:

by default, the positions of the item are bound to the x- and y-Axis of the plot. So we can just set the plot coordinates where the line should start/end:

If we don't want the line to be positioned in plot coordinates but a different coordinate system, e.g. absolute pixel positions on the [QCustomPlot](#) surface, we need to change the position type like this:

Then we can set the coordinates, this time in pixels:

and make the line visible on the entire [QCustomPlot](#), by disabling clipping to the axis rect:

For more advanced plots, it is even possible to set different types and parent anchors per X/Y coordinate of an item position, using for example [QCPItemPosition::setTypeX](#) or [QCPItemPosition::setParentAnchorX](#). For details, see the documentation of [QCPItemPosition](#).

10.395.4 Creating own items

To create an own item, you implement a subclass of [QCPAbstractItem](#). These are the pure virtual functions, you must implement:

- [selectTest](#)
- `draw`

See the documentation of those functions for what they need to do.

10.395.4.1 Allowing the item to be positioned

As mentioned, item positions are represented by [QCPItemPosition](#) members. Let's assume the new item shall have only one point as its position (as opposed to two like a rect or multiple like a polygon). You then add a public member of type [QCPItemPosition](#) like so:

```
QCPItemPosition * const myPosition;
```

the `const` makes sure the pointer itself can't be modified from the user of your new item (the [QCPItemPosition](#) instance it points to, can be modified, of course). The initialization of this pointer is made easy with the `createPosition` function. Just assign the return value of this function to each [QCPItemPosition](#) in the constructor of your item. `createPosition` takes a string which is the name of the position, typically this is identical to the variable name. For example, the constructor of `QCPItemExample` could look like this:

```
QCPItemExample::QCPItemExample(QCustomPlot *parentPlot) :
    QCPAbstractItem(parentPlot),
    myPosition(createPosition("myPosition"))
{
    // other constructor code
}
```

10.395.4.2 The draw function

To give your item a visual representation, reimplement the draw function and use the passed [QCPPainter](#) to draw the item. You can retrieve the item position in pixel coordinates from the position member(s) via [QCPItemPosition::pixelPosition](#).

To optimize performance you should calculate a bounding rect first (don't forget to take the pen width into account), check whether it intersects the clipRect, and only draw the item at all if this is the case.

10.395.4.3 The selectTest function

Your implementation of the [selectTest](#) function may use the helpers [QCPVector2D::distanceSquaredToLine](#) and [rectDistance](#). With these, the implementation of the selection test becomes significantly simpler for most items. See the documentation of [selectTest](#) for what the function parameters mean and what the function should return.

10.395.4.4 Providing anchors

Providing anchors ([QCPItemAnchor](#)) starts off like adding a position. First you create a public member, e.g.

```
QCPItemAnchor * const bottom;
```

and create it in the constructor with the [createAnchor](#) function, assigning it a name and an anchor id (an integer enumerating all anchors on the item, you may create an own enum for this). Since anchors can be placed anywhere, relative to the item's position(s), your item needs to provide the position of every anchor with the reimplementation of the [anchorPixelPosition\(int anchorId\)](#) function.

In essence the [QCPItemAnchor](#) is merely an intermediary that itself asks your item for the pixel position when anything attached to the anchor needs to know the coordinates.

10.395.5 Constructor & Destructor Documentation

10.395.5.1 QCPAbstractItem()

```
QCPAbstractItem::QCPAbstractItem (
    QCustomPlot * parentPlot ) [explicit]
```

Base class constructor which initializes base class members.

10.395.6 Member Function Documentation

10.395.6.1 anchor()

```
QCPItemAnchor * QCPAbstractItem::anchor (
    const QString & name ) const
```

Returns the [QCPItemAnchor](#) with the specified *name*. If this item doesn't have an anchor by that name, returns `nullptr`.

This function provides an alternative way to access item anchors. Normally, you access anchors directly by their member pointers (which typically have the same variable name as *name*).

See also

[anchors](#), [position](#)

10.395.6.2 anchors()

```
QList< QCPItemAnchor * > QCPAbstractItem::anchors ( ) const [inline]
```

Returns all anchors of the item in a list. Note that since a position ([QCPItemPosition](#)) is always also an anchor, the list will also contain the positions of this item.

See also

[positions](#), [anchor](#)

10.395.6.3 applyDefaultAntialiasingHint()

```
void QCPAbstractItem::applyDefaultAntialiasingHint (   
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

10.395.6.4 clipRect()

```
QRect QCPAbstractItem::clipRect ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.395.6.5 deselectEvent()

```
void QCPAbstractItem::deselectEvent (   
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.395.6.6 draw()

```
void QCPAbstractItem::draw (   
    QCPPainter * painter ) [protected], [pure virtual]
```

Implements [QCPLayerable](#).

10.395.6.7 hasAnchor()

```
bool QCPAbstractItem::hasAnchor (   
    const QString & name ) const
```

Returns whether this item has an anchor with the specified *name*.

Note that you can check for positions with this function, too. This is because every position is also an anchor ([QCPItemPosition](#) inherits from [QCPItemAnchor](#)).

See also

[anchor](#), [position](#)

10.395.6.8 position()

```
QCItemPosition * QCPAbstractItem::position (
    const QString & name ) const
```

Returns the [QCItemPosition](#) with the specified *name*. If this item doesn't have a position by that name, returns nullptr.

This function provides an alternative way to access item positions. Normally, you access positions directly by their member pointers (which typically have the same variable name as *name*).

See also

[positions](#), [anchor](#)

10.395.6.9 positions()

```
QList< QCItemPosition * > QCPAbstractItem::positions ( ) const [inline]
```

Returns all positions of the item in a list.

See also

[anchors](#), [position](#)

10.395.6.10 selectEvent()

```
void QCPAbstractItem::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.395.6.11 selectionCategory()

```
QCP::Interaction QCPAbstractItem::selectionCategory ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.395.6.12 selectionChanged

```
void QCPAbstractItem::selectionChanged (
    bool selected ) [signal]
```

This signal is emitted when the selection state of this item has changed, either by user interaction or by a direct call to [setSelected](#).

10.395.6.13 selectTest()

```
virtual double QCPAbstractItem::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [pure virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the mouseReleaseEvent occurs, and the finally selected object is notified via the selectEvent/ deselectEvent methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to selectEvent when the parent [QCustomPlot](#) decides on the basis of this selectTest call, that the object was successfully selected. The subsequent call to selectEvent will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent selectEvent, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

selectEvent, deselectEvent, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Reimplemented from [QCPLayerable](#).

Implemented in [QCPItemStraightLine](#), [QCPItemLine](#), [QCPItemCurve](#), [QCPItemRect](#), [QCPItemText](#), [QCPItemEllipse](#), [QCPItemPixmap](#), [QCPItemTracer](#), and [QCPItemBracket](#).

10.395.6.14 setClipAxisRect()

```
void QCPAbstractItem::setClipAxisRect (
    QCPAxisRect * rect )
```

Sets the clip axis rect. It defines the rect that will be used to clip the item when [setClipToAxisRect](#) is set to true.

See also

[setClipToAxisRect](#)

10.395.6.15 `setClipToAxisRect()`

```
void QCPAbstractItem::setClipToAxisRect (
    bool clip )
```

Sets whether the item shall be clipped to an axis rect or whether it shall be visible on the entire [QCustomPlot](#). The axis rect can be set with [setClipAxisRect](#).

See also

[setClipAxisRect](#)

10.395.6.16 `setSelectable()`

```
void QCPAbstractItem::setSelectable (
    bool selectable )
```

Sets whether the user can (de-)select this item by clicking on the [QCustomPlot](#) surface. (When [QCustomPlot::setInteractions](#) contains [QCustomPlot::iSelectItems](#).)

However, even when *selectable* was set to false, it is possible to set the selection manually, by calling [setSelected](#).

See also

[QCustomPlot::setInteractions](#), [setSelected](#)

10.395.6.17 `setSelected()`

```
void QCPAbstractItem::setSelected (
    bool selected )
```

Sets whether this item is selected or not. When selected, it might use a different visual appearance (e.g. pen and brush), this depends on the specific item though.

The entire selection mechanism for items is handled automatically when [QCustomPlot::setInteractions](#) contains [QCustomPlot::iSelectItems](#). You only need to call this function when you wish to change the selection state manually.

This function can change the selection state even when [setSelectable](#) was set to false.

emits the [selectionChanged](#) signal when *selected* is different from the previous selection state.

See also

[setSelectable](#), [selectTest](#)

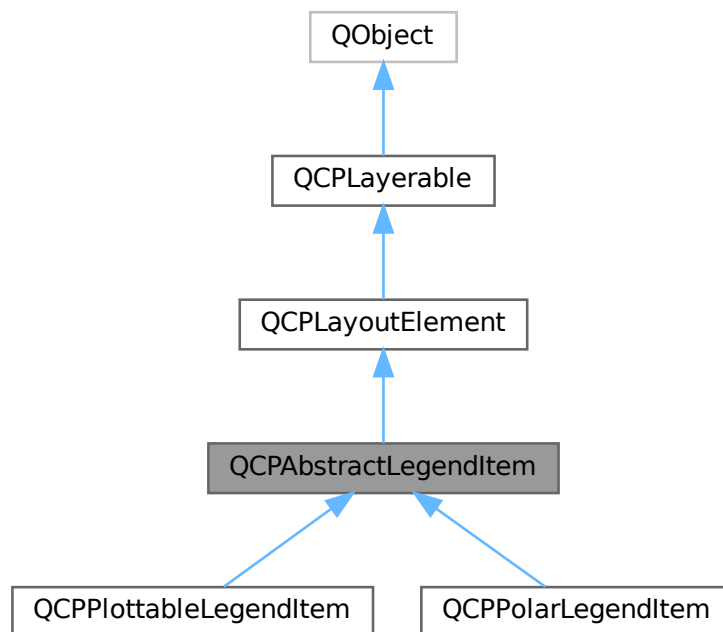
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.396 QCPAbstractLegendItem Class Reference

The abstract base class for all entries in a [QCPLegend](#).

Inheritance diagram for QCPAbstractLegendItem:



Public Member Functions inherited from QCPLLayoutElement

- [QCPLLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLLayout](#) * [layout](#) () const
- [QRect](#) [rect](#) () const
- [QRect](#) [outerRect](#) () const
- [QMargins](#) [margins](#) () const
- [QMargins](#) [minimumMargins](#) () const
- [QCP::MarginSides](#) [autoMargins](#) () const
- [QSize](#) [minimumSize](#) () const
- [QSize](#) [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- [QHash](#)< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const [QRect](#) &[rect](#))
- void [setMargins](#) (const [QMargins](#) &margins)
- void [setMinimumMargins](#) (const [QMargins](#) &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)
- void [setMinimumSize](#) (const [QSize](#) &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const [QSize](#) &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)
- virtual void [update](#) ([UpdatePhase](#) phase)
- virtual [QSize](#) [minimumOuterSizeHint](#) () const
- virtual [QSize](#) [maximumOuterSizeHint](#) () const
- virtual [QList](#)< [QCPLLayoutElement](#) * > [elements](#) (bool recursive) const

Public Member Functions inherited from QCPLayerable

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool [visible](#) () const
- [QCustomPlot](#) * [parentPlot](#) () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * [layer](#) () const
- bool [antialiased](#) () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const [QString](#) &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual [QRect](#) [clipRect](#) () const Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE=0
- virtual void [selectEvent](#) ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState← Changed) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()
- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QCPLegend](#) * **mParentLegend**
- QFont **mFont**
- QColor **mTextColor**
- QFont **mSelectedFont**
- QColor **mSelectedTextColor**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCPLegend](#)

Additional Inherited Members

Public Types inherited from [QCPLayoutElement](#)

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

10.396.1 Detailed Description

The abstract base class for all entries in a [QCPLegend](#).

It defines a very basic interface for entries in a [QCPLegend](#). For representing plottables in the legend, the subclass [QCPLottableLegendItem](#) is more suitable.

Only derive directly from this class when you need absolute freedom (e.g. a custom legend entry that's not even associated with a plottable).

You must implement the following pure virtual functions:

- draw (from [QCPLayerable](#))

You inherit the following members you may use:

QCPLegend * mParentLegend	A pointer to the parent QCPLegend .
QFont mFont	The generic font of the item. You should use this font for all or at least the most prominent text of the item.

10.396.2 Constructor & Destructor Documentation

10.396.2.1 QCPLegendItem()

```
QCPLegendItem::QCPLegendItem (
    QCPLegend * parent )    [explicit]
```

Constructs a [QCPLegendItem](#) and associates it with the [QCPLegend](#) *parent*. This does not cause the item to be added to *parent*, so [QCPLegend::addItem](#) must be called separately.

10.396.3 Member Function Documentation

10.396.3.1 applyDefaultAntialiasingHint()

```
void QCPLegendItem::applyDefaultAntialiasingHint (
    QCPainter * painter ) const    [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

10.396.3.2 clipRect()

```
QRect QCPAbstractLegendItem::clipRect ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.396.3.3 deselectEvent()

```
void QCPAbstractLegendItem::deselectEvent (
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.396.3.4 draw()

```
virtual void QCPAbstractLegendItem::draw (
    QCPainter * painter ) [protected], [pure virtual]
```

Reimplemented from [QCPLayoutElement](#).

10.396.3.5 selectEvent()

```
void QCPAbstractLegendItem::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.396.3.6 selectionCategory()

```
QCP::Interaction QCPAbstractLegendItem::selectionCategory ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.396.3.7 selectionChanged

```
void QCPAbstractLegendItem::selectionChanged (
    bool selected ) [signal]
```

This signal is emitted when the selection state of this legend item has changed, either by user interaction or by a direct call to [setSelected](#).

10.396.3.8 selectTest()

```
double QCPAbstractLegendItem::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Layout elements are sensitive to events inside their outer rect. If *pos* is within the outer rect, this method returns a value corresponding to 0.99 times the parent plot's selection tolerance. However, layout elements are not selectable by default. So if *onlySelectable* is true, -1.0 is returned.

See [QCPLayerable::selectTest](#) for a general explanation of this virtual method.

[QCPLayoutElement](#) subclasses may reimplement this method to provide more specific selection test behaviour.

Reimplemented from [QCPLayoutElement](#).

10.396.3.9 setFont()

```
void QCPAbstractLegendItem::setFont (
    const QFont & font )
```

Sets the default font of this specific legend item to *font*.

See also

[setTextColor](#), [QCPLegend::setFont](#)

10.396.3.10 setSelectable()

```
void QCPAbstractLegendItem::setSelectable (
    bool selectable )
```

Sets whether this specific legend item is selectable.

See also

[setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.396.3.11 setSelected()

```
void QCPAbstractLegendItem::setSelected (
    bool selected )
```

Sets whether this specific legend item is selected.

It is possible to set the selection state of this item by calling this function directly, even if `setSelectable` is set to false.

See also

[setSelectableParts](#), [QCustomPlot::setInteractions](#)

10.396.3.12 setSelectedFont()

```
void QCPAbstractLegendItem::setSelectedFont (
    const QFont & font )
```

When this legend item is selected, *font* is used to draw generic text, instead of the normal font set with [setFont](#).

See also

[setFont](#), [QCPLegend::setSelectedFont](#)

10.396.3.13 setSelectedTextColor()

```
void QCPAbstractLegendItem::setSelectedTextColor (
    const QColor & color )
```

When this legend item is selected, *color* is used to draw generic text, instead of the normal color set with [setTextColor](#).

See also

[setTextColor](#), [QCPLegend::setSelectedTextColor](#)

10.396.3.14 setTextColor()

```
void QCPAbstractLegendItem::setTextColor (
    const QColor & color )
```

Sets the default text color of this specific legend item to *color*.

See also

[setFont](#), [QCPLegend::setTextColor](#)

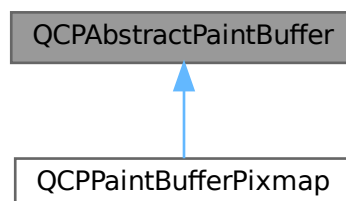
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.397 QCPAbstractPaintBuffer Class Reference

The abstract base class for paint buffers, which define the rendering backend.

Inheritance diagram for QCPAbstractPaintBuffer:



Public Member Functions

- [QCPAbstractPaintBuffer](#) (const QSize &size, double devicePixelRatio)
- QSize **size** () const
- bool **invalidated** () const
- double **devicePixelRatio** () const
- void [setSize](#) (const QSize &size)
- void [setInvalidated](#) (bool invalidated=true)
- void [setDevicePixelRatio](#) (double ratio)
- virtual [QCPPainter](#) * [startPainting](#) ()=0
- virtual void [donePainting](#) ()
- virtual void [draw](#) ([QCPPainter](#) *painter) const =0
- virtual void [clear](#) (const QColor &color)=0

Protected Member Functions

- virtual void [reallocateBuffer](#) ()=0

Protected Attributes

- QSize **mSize**
- double **mDevicePixelRatio**
- bool **mInvalidated**

10.397.1 Detailed Description

The abstract base class for paint buffers, which define the rendering backend.

This abstract base class defines the basic interface that a paint buffer needs to provide in order to be usable by [QCustomPlot](#).

A paint buffer manages both a surface to draw onto, and the matching paint device. The size of the surface can be changed via [setSize](#). External classes ([QCustomPlot](#) and [QCPLayer](#)) request a painter via [startPainting](#) and then perform the draw calls. Once the painting is complete, [donePainting](#) is called, so the paint buffer implementation can do clean up if necessary. Before rendering a frame, each paint buffer is usually filled with a color using [clear](#) (usually the color is `Qt::transparent`), to remove the contents of the previous frame.

The simplest paint buffer implementation is [QCPPaintBufferPixmap](#) which allows regular software rendering via the raster engine. Hardware accelerated rendering via pixel buffers and frame buffer objects is provided by [QCPPaintBufferGLPbuffer](#) and [QCPPaintBufferGLFbo](#). They are used automatically if [QCustomPlot::setOpenGl](#) is enabled.

10.397.2 Constructor & Destructor Documentation

10.397.2.1 QCPAbstractPaintBuffer()

```
QCPAbstractPaintBuffer::QCPAbstractPaintBuffer (
    const QSize & size,
    double devicePixelRatio ) [explicit]
```

Creates a paint buffer and initializes it with the provided *size* and *devicePixelRatio*.

Subclasses must call their [reallocateBuffer](#) implementation in their respective constructors.

10.397.3 Member Function Documentation

10.397.3.1 clear()

```
void QCPAbstractPaintBuffer::clear (
    const QColor & color ) [pure virtual]
```

Fills the entire buffer with the provided *color*. To have an empty transparent buffer, use the named color `Qt::transparent`.

This method must not be called if there is currently a painter (acquired with [startPainting](#)) active.

Implemented in [QCPPaintBufferPixmap](#).

10.397.3.2 donePainting()

```
void QCPAbstractPaintBuffer::donePainting ( ) [inline], [virtual]
```

If you have acquired a [QCPPainter](#) to paint onto this paint buffer via [startPainting](#), call this method as soon as you are done with the painting operations and have deleted the painter.

paint buffer subclasses may use this method to perform any type of cleanup that is necessary. The default implementation does nothing.

10.397.3.3 draw()

```
void QCPAbstractPaintBuffer::draw (
    QCPPainter * painter ) const [pure virtual]
```

Draws the contents of this buffer with the provided *painter*. This is the method that is used to finally join all paint buffers and draw them onto the screen.

Implemented in [QCPPaintBufferPixmap](#).

10.397.3.4 reallocateBuffer()

```
void QCPAbstractPaintBuffer::reallocateBuffer ( ) [protected], [pure virtual]
```

Reallocates the internal buffer with the currently configured size ([setSize](#)) and device pixel ratio, if applicable ([setDevicePixelRatio](#)). It is called as soon as any of those properties are changed on this paint buffer.

Note

Subclasses of [QCPAbstractPaintBuffer](#) must call their reimplementation of this method in their constructor, to perform the first allocation (this can not be done by the base class because calling pure virtual methods in base class constructors is not possible).

Implemented in [QCPPaintBufferPixmap](#).

10.397.3.5 setDevicePixelRatio()

```
void QCPAbstractPaintBuffer::setDevicePixelRatio (
    double ratio )
```

Sets the device pixel ratio to *ratio*. This is useful to render on high-DPI output devices. The ratio is automatically set to the device pixel ratio used by the parent [QCustomPlot](#) instance.

The buffer is reallocated (by calling [reallocateBuffer](#)), so any painters that were obtained by [startPainting](#) are invalidated and must not be used after calling this method.

Note

This method is only available for Qt versions 5.4 and higher.

10.397.3.6 setInvalidated()

```
void QCPAbstractPaintBuffer::setInvalidated (
    bool invalidated = true )
```

Sets the invalidated flag to *invalidated*.

This mechanism is used internally in conjunction with isolated replotting of [QCPLayer](#) instances (in [QCPLayer::ImBuffered](#) mode). If [QCPLayer::replot](#) is called on a buffered layer, i.e. an isolated repaint of only that layer (and its dedicated paint buffer) is requested, [QCustomPlot](#) will decide depending on the invalidated flags of other paint buffers whether it also replots them, instead of only the layer on which the replot was called.

The invalidated flag is set to true when [QCPLayer](#) association has changed, i.e. if layers were added or removed from this buffer, or if they were reordered. It is set to false as soon as all associated [QCPLayer](#) instances are drawn onto the buffer.

Under normal circumstances, it is not necessary to manually call this method.

10.397.3.7 setSize()

```
void QCPAbstractPaintBuffer::setSize (
    const QSize & size )
```

Sets the paint buffer size.

The buffer is reallocated (by calling [reallocateBuffer](#)), so any painters that were obtained by [startPainting](#) are invalidated and must not be used after calling this method.

If *size* is already the current buffer size, this method does nothing.

10.397.3.8 startPainting()

```
QCPPainter * QCPAbstractPaintBuffer::startPainting ( ) [pure virtual]
```

Returns a [QCPPainter](#) which is ready to draw to this buffer. The ownership and thus the responsibility to delete the painter after the painting operations are complete is given to the caller of this method.

Once you are done using the painter, delete the painter and call [donePainting](#).

While a painter generated with this method is active, you must not call [setSize](#), [setDevicePixelRatio](#) or [clear](#).

This method may return 0, if a painter couldn't be activated on the buffer. This usually indicates a problem with the respective painting backend.

Implemented in [QCPPaintBufferPixmap](#).

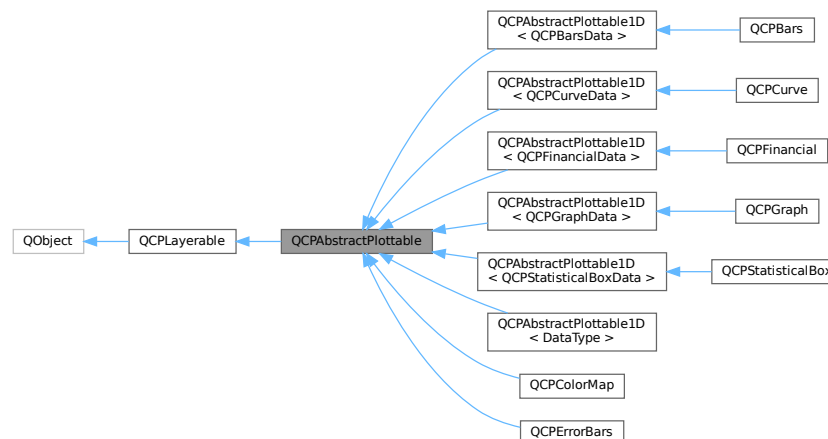
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

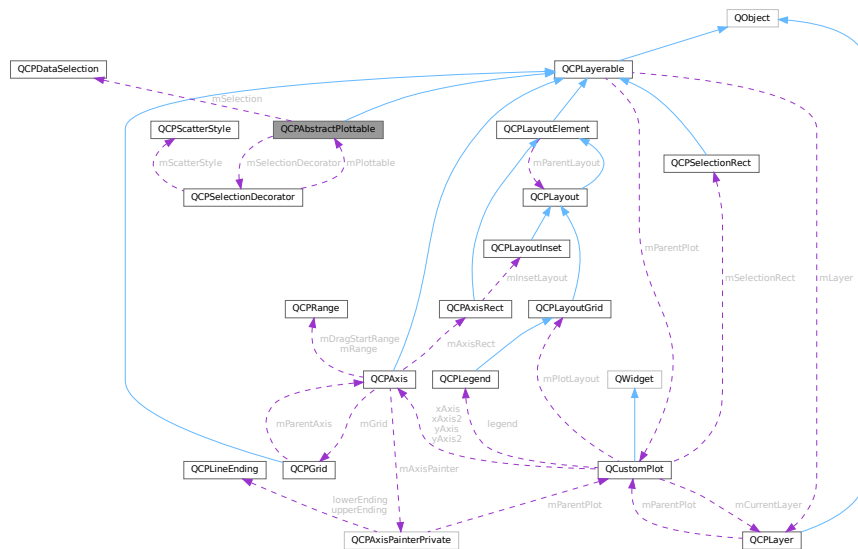
10.398 QCPAbstractPlottable Class Reference

The abstract base class for all data representing objects in a plot.

Inheritance diagram for QCPAbstractPlottable:



Collaboration diagram for QCPAbstractPlottable:



Signals

- void [selectionChanged](#) (bool [selected](#))
- void [selectionChanged](#) (const [QCPDataSelection](#) &[selection](#))
- void [selectableChanged](#) ([QCP::SelectionType](#) [selectable](#))

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *[newLayer](#))

Public Member Functions

- [QCPAbstractPlottable](#) ([QCPAxis](#) *[keyAxis](#), [QCPAxis](#) *[valueAxis](#))
- [QString](#) [name](#) () const
- bool [antialiasedFill](#) () const
- bool [antialiasedScatters](#) () const
- [QPen](#) [pen](#) () const
- [QBrush](#) [brush](#) () const
- [QCPAxis](#) * [keyAxis](#) () const
- [QCPAxis](#) * [valueAxis](#) () const
- [QCP::SelectionType](#) [selectable](#) () const
- bool [selected](#) () const
- [QCPDataSelection](#) [selection](#) () const
- [QCPSelectionDecorator](#) * [selectionDecorator](#) () const
- void [setName](#) (const [QString](#) &[name](#))
- void [setAntialiasedFill](#) (bool [enabled](#))
- void [setAntialiasedScatters](#) (bool [enabled](#))
- void [setPen](#) (const [QPen](#) &[pen](#))
- void [setBrush](#) (const [QBrush](#) &[brush](#))
- void [setKeyAxis](#) ([QCPAxis](#) *[axis](#))

- void [setValueAxis](#) ([QCPAxis](#) *axis)
- Q_SLOT void [setSelectable](#) ([QCP::SelectionType](#) selectable)
- Q_SLOT void [setSelection](#) ([QCPDataSelection](#) selection)
- void [setSelectionDecorator](#) ([QCPSelectionDecorator](#) *decorator)
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_DECL_OVERRIDE](#)=0
- virtual [QCPPlottableInterface1D](#) * [interface1D](#) ()
- virtual [QCPRange](#) [getKeyRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const =0
- virtual [QCPRange](#) [getValueRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const =0
- void [coordsToPixels](#) (double key, double value, double &x, double &y) const
- const [QPointF](#) [coordsToPixels](#) (double key, double value) const
- void [pixelsToCoords](#) (double x, double y, double &key, double &value) const
- void [pixelsToCoords](#) (const [QPointF](#) &pixelPos, double &key, double &value) const
- void [rescaleAxes](#) (bool onlyEnlarge=false) const
- void [rescaleKeyAxis](#) (bool onlyEnlarge=false) const
- void [rescaleValueAxis](#) (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool [addToLegend](#) ([QCPLegend](#) *legend)
- bool [addToLegend](#) ()
- bool [removeFromLegend](#) ([QCPLegend](#) *legend) const
- bool [removeFromLegend](#) () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const [QString](#) &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual [QRect](#) [clipRect](#) () const [Q_DECL_OVERRIDE](#)
- virtual void [draw](#) ([QCPPainter](#) *painter) [Q_DECL_OVERRIDE](#)=0
- virtual [QCP::Interaction](#) [selectionCategory](#) () const [Q_DECL_OVERRIDE](#)
- void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const [Q_DECL_OVERRIDE](#)
- virtual void [selectEvent](#) ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionStateChanged) [Q_DECL_OVERRIDE](#)
- virtual void [deselectEvent](#) (bool *selectionStateChanged) [Q_DECL_OVERRIDE](#)
- virtual void [drawLegendIcon](#) ([QCPPainter](#) *painter, const [QRectF](#) &rect) const =0
- void [applyFillAntialiasingHint](#) ([QCPPainter](#) *painter) const
- void [applyScattersAntialiasingHint](#) ([QCPPainter](#) *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QString **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- QPen **mPen**
- QBrush **mBrush**
- QPointer< [QCPAxis](#) > **mKeyAxis**
- QPointer< [QCPAxis](#) > **mValueAxis**
- [QCP::SelectionType](#) **mSelectable**
- [QCPDataSelection](#) **mSelection**
- [QCPSelectionDecorator](#) * **mSelectionDecorator**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class **QCustomPlot**
- class **QCPAxis**
- class **QCPPlottableLegendItem**

10.398.1 Detailed Description

The abstract base class for all data representing objects in a plot.

It defines a very basic interface like name, pen, brush, visibility etc. Since this class is abstract, it can't be instantiated. Use one of the subclasses or create a subclass yourself to create new ways of displaying data (see "Creating own plottables" below). Plottables that display one-dimensional data (i.e. data points have a single key dimension and one or multiple values at each key) are based off of the template subclass [QCPAbstractPlottable1D](#), see details there.

All further specifics are in the subclasses, for example:

- A normal graph with possibly a line and/or scatter points [QCPGraph](#) (typically created with [QCustomPlot::addGraph](#))
- A parametric curve: [QCPCurve](#)
- A bar chart: [QCPBars](#)
- A statistical box plot: [QCPStatisticalBox](#)
- A color encoded two-dimensional map: [QCPColorMap](#)
- An OHLC/Candlestick chart: [QCPFinancial](#)

10.398.2 Creating own plottables

Subclassing directly from [QCPAbstractPlottable](#) is only recommended if you wish to display two-dimensional data like [QCPColorMap](#), i.e. two logical key dimensions and one (or more) data dimensions. If you want to display data with only one logical key dimension, you should rather derive from [QCPAbstractPlottable1D](#).

If subclassing [QCPAbstractPlottable](#) directly, these are the pure virtual functions you must implement:

- [selectTest](#)
- [draw](#)
- [drawLegendIcon](#)
- [getKeyRange](#)
- [getValueRange](#)

See the documentation of those functions for what they need to do.

For drawing your plot, you can use the [coordsToPixels](#) functions to translate a point in plot coordinates to pixel coordinates. This function is quite convenient, because it takes the orientation of the key and value axes into account for you (x and y are swapped when the key axis is vertical and the value axis horizontal). If you are worried about performance (i.e. you need to translate many points in a loop like [QCPGraph](#)), you can directly use [QCPAxis::coordToPixel](#). However, you must then take care about the orientation of the axis yourself.

Here are some important members you inherit from [QCPAbstractPlottable](#):

QCustomPlot * mParentPlot	A pointer to the parent QCustomPlot instance. The parent plot is inferred from the axes that are passed in the constructor.
QString mName	The name of the plottable.
QPen mPen	The generic pen of the plottable. You should use this pen for the most prominent data representing lines in the plottable (e.g. QCPGraph uses this pen for its graph lines and scatters)
QBrush mBrush	The generic brush of the plottable. You should use this brush for the most prominent fillable structures in the plottable (e.g. QCPGraph uses this brush to control filling under the graph)
QPointer< QCPAxis > mKeyAxis , mValueAxis	The key and value axes this plottable is attached to. Call their QCPAxis::coordToPixel functions to translate coordinates to pixels in either the key or value dimension. Make sure to check whether the pointer is <code>nullptr</code> before using it. If one of the axes is null, don't draw the plottable.
QCPSelectionDecorator mSelectionDecorator	The currently set selection decorator which specifies how selected data of the plottable shall be drawn and decorated. When drawing your data, you must consult this decorator for the appropriate pen/brush before drawing unselected/selected data segments. Finally, you should call its QCPSelectionDecorator::drawDecoration method at the end of your draw implementation.
QCP::SelectionType mSelectable	In which composition, if at all, this plottable's data may be selected. Enforcing this setting on the data selection is done by QCPAbstractPlottable automatically.
QCPDataSelection mSelection	Holds the current selection state of the plottable's data, i.e. the selected data ranges (QCPDataRange).

10.398.3 Constructor & Destructor Documentation

10.398.3.1 QCPAbstractPlottable()

```
QCPAbstractPlottable::QCPAbstractPlottable (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis )
```

Constructs an abstract plottable which uses *keyAxis* as its key axis ("x") and *valueAxis* as its value axis ("y"). *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#) instance and have perpendicular orientations. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

Since [QCPAbstractPlottable](#) is an abstract class that defines the basic interface to plottables, it can't be directly instantiated.

You probably want one of the subclasses like [QCPGraph](#) or [QCPCurve](#) instead.

10.398.4 Member Function Documentation

10.398.4.1 addToLegend() [1/2]

```
bool QCPAbstractPlottable::addToLegend ( )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds this plottable to the legend of the parent [QCustomPlot](#) ([QCustomPlot::legend](#)).

See also

[removeFromLegend](#)

10.398.4.2 addToLegend() [2/2]

```
bool QCPAbstractPlottable::addToLegend (
    QCPLegend * legend )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds this plottable to the specified *legend*.

Creates a [QCPPlottableLegendItem](#) which is inserted into the legend. Returns true on success, i.e. when the legend exists and a legend item associated with this plottable isn't already in the legend.

If the plottable needs a more specialized representation in the legend, you can create a corresponding subclass of [QCPPlottableLegendItem](#) and add it to the legend manually instead of calling this method.

See also

[removeFromLegend](#), [QCPLegend::addItem](#)

10.398.4.3 applyDefaultAntialiasingHint()

```
void QCPAbstractPlottable::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

10.398.4.4 clipRect()

```
QRect QCPAbstractPlottable::clipRect ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.398.4.5 coordsToPixels() [1/2]

```
const QPointF QCPAbstractPlottable::coordsToPixels (
    double key,
    double value ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Transforms the given *key* and *value* to pixel coordinates and returns them in a [QPointF](#).

10.398.4.6 coordsToPixels() [2/2]

```
void QCPAbstractPlottable::coordsToPixels (
    double key,
    double value,
    double & x,
    double & y ) const
```

Convenience function for transforming a key/value pair to pixels on the [QCustomPlot](#) surface, taking the orientations of the axes associated with this plottable into account (e.g. whether key represents x or y).

key and *value* are transformed to the coordinates in pixels and are written to *x* and *y*.

See also

[pixelsToCoords](#), [QCPAxis::coordToPixel](#)

10.398.4.7 deselectEvent()

```
void QCPAbstractPlottable::deselectEvent (
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.398.4.8 draw()

```
virtual void QCPAbstractPlottable::draw (
    QCPPainter * painter ) [protected], [pure virtual]
```

Implements [QCPLayerable](#).

10.398.4.9 getKeyRange()

```
QCPRange QCPAbstractPlottable::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [pure virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implemented in [QCPGraph](#), [QCPCurve](#), [QCPBars](#), [QCPStatisticalBox](#), [QCPColorMap](#), [QCPFinancial](#), and [QCPErrorBars](#).

10.398.4.10 `getValueRange()`

```
QCPRange QCPAbstractPlottable::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [pure virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either `QCP::sdNegative` or `QCP::sdPositive` in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to `QCP::sdNegative` and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to `QCP::sdBoth` (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to `QCPRange()`), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as `QCPRange::validRange`, since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of `QCPRange::validRange`.

See also

[rescaleAxes](#), [getKeyRange](#)

Implemented in [QCPGraph](#), [QCPCurve](#), [QCPBars](#), [QCPStatisticalBox](#), [QCPColorMap](#), [QCPFinancial](#), and [QCPErrorBars](#).

10.398.4.11 `interface1D()`

```
QCPPlottableInterface1D * QCPAbstractPlottable::interface1D ( ) [inline], [virtual]
```

If this plottable is a one-dimensional plottable, i.e. it implements the [QCPPlottableInterface1D](#), returns the *this* pointer with that type. Otherwise (e.g. in the case of a [QCPColorMap](#)) returns zero.

You can use this method to gain read access to data coordinates while holding a pointer to the abstract base class only.

Reimplemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalBoxData >](#), and [QCPErrorBars](#).

10.398.4.12 `pixelsToCoords()` [1/2]

```
void QCPAbstractPlottable::pixelsToCoords (
    const QPointF & pixelPos,
    double & key,
    double & value ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the pixel input *pixelPos* as plot coordinates *key* and *value*.

10.398.4.13 pixelsToCoords() [2/2]

```
void QCPAbstractPlottable::pixelsToCoords (
    double x,
    double y,
    double & key,
    double & value ) const
```

Convenience function for transforming a x/y pixel pair on the [QCustomPlot](#) surface to plot coordinates, taking the orientations of the axes associated with this plottable into account (e.g. whether key represents x or y).

x and y are transformed to the plot coordinates and are written to *key* and *value*.

See also

[coordsToPixels](#), [QCPAxis::coordToPixel](#)

10.398.4.14 removeFromLegend() [1/2]

```
bool QCPAbstractPlottable::removeFromLegend ( ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes the plottable from the legend of the parent [QCustomPlot](#).

See also

[addToLegend](#)

10.398.4.15 removeFromLegend() [2/2]

```
bool QCPAbstractPlottable::removeFromLegend (
    QCPLegend * legend ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes the plottable from the specified *legend*. This means the [QCPPlottableLegendItem](#) that is associated with this plottable is removed.

Returns true on success, i.e. if the legend exists and a legend item associated with this plottable was found and removed.

See also

[addToLegend](#), [QCPLegend::removeItem](#)

10.398.4.16 `rescaleAxes()`

```
void QCPAbstractPlottable::rescaleAxes (
    bool onlyEnlarge = false ) const
```

Rescales the key and value axes associated with this plottable to contain all displayed data, so the whole plottable is visible. If the scaling of an axis is logarithmic, `rescaleAxes` will make sure not to rescale to an illegal range i.e. a range containing different signs and/or zero. Instead it will stay in the current sign domain and ignore all parts of the plottable that lie outside of that domain.

onlyEnlarge makes sure the ranges are only expanded, never reduced. So it's possible to show multiple plottables in their entirety by multiple calls to `rescaleAxes` where the first call has *onlyEnlarge* set to false (the default), and all subsequent set to true.

See also

[rescaleKeyAxis](#), [rescaleValueAxis](#), [QCustomPlot::rescaleAxes](#), [QCPAxis::rescale](#)

10.398.4.17 `rescaleKeyAxis()`

```
void QCPAbstractPlottable::rescaleKeyAxis (
    bool onlyEnlarge = false ) const
```

Rescales the key axis of the plottable so the whole plottable is visible.

See [rescaleAxes](#) for detailed behaviour.

10.398.4.18 `rescaleValueAxis()`

```
void QCPAbstractPlottable::rescaleValueAxis (
    bool onlyEnlarge = false,
    bool inKeyRange = false ) const
```

Rescales the value axis of the plottable so the whole plottable is visible. If *inKeyRange* is set to true, only the data points which are in the currently visible key axis range are considered.

Returns true if the axis was actually scaled. This might not be the case if this plottable has an invalid range, e.g. because it has no data points.

See [rescaleAxes](#) for detailed behaviour.

10.398.4.19 `selectableChanged`

```
void QCPAbstractPlottable::selectableChanged (
    QCP::SelectionType selectable ) [signal]
```

This signal is emitted when the selectability of this plottable has changed.

See also

[setSelectable](#)

10.398.4.20 selected()

```
bool QCPAbstractPlottable::selected ( ) const [inline]
```

Returns true if there are any data points of the plottable currently selected. Use [selection](#) to retrieve the current [QCPDataSelection](#).

10.398.4.21 selectEvent()

```
void QCPAbstractPlottable::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.398.4.22 selection()

```
QCPDataSelection QCPAbstractPlottable::selection ( ) const [inline]
```

Returns a [QCPDataSelection](#) encompassing all the data points that are currently selected on this plottable.

See also

[selected](#), [setSelection](#), [setSelectable](#)

10.398.4.23 selectionCategory()

```
QCP::Interaction QCPAbstractPlottable::selectionCategory ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.398.4.24 selectionChanged [1/2]

```
void QCPAbstractPlottable::selectionChanged (
    bool selected ) [signal]
```

This signal is emitted when the selection state of this plottable has changed, either by user interaction or by a direct call to [setSelection](#). The parameter *selected* indicates whether there are any points selected or not.

See also

[selectionChanged\(const QCPDataSelection &selection\)](#)

10.398.4.25 selectionChanged [2/2]

```
void QCPAbstractPlottable::selectionChanged (
    const QCPDataSelection & selection ) [signal]
```

This signal is emitted when the selection state of this plottable has changed, either by user interaction or by a direct call to [setSelection](#). The parameter *selection* holds the currently selected data ranges.

See also

[selectionChanged\(bool selected\)](#)

10.398.4.26 selectionDecorator()

```
QCPSelectionDecorator * QCPAbstractPlottable::selectionDecorator ( ) const [inline]
```

Provides access to the selection decorator of this plottable. The selection decorator controls how selected data ranges are drawn (e.g. their pen color and fill), see [QCPSelectionDecorator](#) for details.

If you wish to use an own [QCPSelectionDecorator](#) subclass, pass an instance of it to [setSelectionDecorator](#).

10.398.4.27 selectTest()

```
virtual double QCPAbstractPlottable::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [pure virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPLayerable](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the mouseReleaseEvent occurs, and the finally selected object is notified via the selectEvent/ deselectEvent methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to selectEvent when the parent [QCustomPlot](#) decides on the basis of this selectTest call, that the object was successfully selected. The subsequent call to selectEvent will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent selectEvent, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

[selectEvent](#), [deselectEvent](#), [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Reimplemented from [QCPLayerable](#).

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalBox >](#), [QCPColorMap](#), [QCPFinancial](#), and [QCPErrorBars](#).

10.398.4.28 setAntialiasedFill()

```
void QCPAbstractPlottable::setAntialiasedFill (
    bool enabled )
```

Sets whether fills of this plottable are drawn antialiased or not.

Note that this setting may be overridden by [QCustomPlot::setAntialiasedElements](#) and [QCustomPlot::setNotAntialiasedElements](#).

10.398.4.29 setAntialiasedScatters()

```
void QCPAbstractPlottable::setAntialiasedScatters (
    bool enabled )
```

Sets whether the scatter symbols of this plottable are drawn antialiased or not.

Note that this setting may be overridden by [QCustomPlot::setAntialiasedElements](#) and [QCustomPlot::setNotAntialiasedElements](#).

10.398.4.30 setBrush()

```
void QCPAbstractPlottable::setBrush (
    const QBrush & brush )
```

The brush is used to draw basic fills of the plottable representation in the plot. The Fill can be a color, gradient or texture, see the usage of QBrush.

For example, the [QCPGraph](#) subclass draws the fill under the graph with this brush, when it's not set to Qt::NoBrush.

See also

[setPen](#)

10.398.4.31 setKeyAxis()

```
void QCPAbstractPlottable::setKeyAxis (
    QCPAxis * axis )
```

The key axis of a plottable can be set to any axis of a [QCustomPlot](#), as long as it is orthogonal to the plottable's value axis. This function performs no checks to make sure this is the case. The typical mathematical choice is to use the x-axis ([QCustomPlot::xAxis](#)) as key axis and the y-axis ([QCustomPlot::yAxis](#)) as value axis.

Normally, the key and value axes are set in the constructor of the plottable (or [QCustomPlot::addGraph](#) when working with QCPGraphs through the dedicated graph interface).

See also

[setValueAxis](#)

10.398.4.32 setName()

```
void QCPAbstractPlottable::setName (
    const QString & name )
```

The name is the textual representation of this plottable as it is displayed in the legend ([QCPLegend](#)). It may contain any UTF-8 characters, including newlines.

10.398.4.33 setPen()

```
void QCPAbstractPlottable::setPen (
    const QPen & pen )
```

The pen is used to draw basic lines that make up the plottable representation in the plot.

For example, the [QCPGraph](#) subclass draws its graph lines with this pen.

See also

[setBrush](#)

10.398.4.34 setSelectable()

```
void QCPAbstractPlottable::setSelectable (
    QCP::SelectionType selectable )
```

Sets whether and to which granularity this plottable can be selected.

A selection can happen by clicking on the [QCustomPlot](#) surface (When [QCustomPlot::setInteractions](#) contains [QCP::iSelectPlottables](#)), by dragging a selection rect (When [QCustomPlot::setSelectionRectMode](#) is [QCP::srmSelect](#)), or programmatically by calling [setSelection](#).

See also

[setSelection](#), [QCP::SelectionType](#)

10.398.4.35 setSelection()

```
void QCPAbstractPlottable::setSelection (
    QCPDataSelection selection )
```

Sets which data ranges of this plottable are selected. Selected data ranges are drawn differently (e.g. color) in the plot. This can be controlled via the selection decorator (see [selectionDecorator](#)).

The entire selection mechanism for plottables is handled automatically when [QCustomPlot::setInteractions](#) contains [iSelectPlottables](#). You only need to call this function when you wish to change the selection state programmatically.

Using [setSelectable](#) you can further specify for each plottable whether and to which granularity it is selectable. If *selection* is not compatible with the current [QCP::SelectionType](#) set via [setSelectable](#), the resulting selection will be adjusted accordingly (see [QCPDataSelection::enforceType](#)).

emits the [selectionChanged](#) signal when *selected* is different from the previous selection state.

See also

[setSelectable](#), [selectTest](#)

10.398.4.36 setSelectionDecorator()

```
void QCPAbstractPlottable::setSelectionDecorator (
    QCPSelectionDecorator * decorator )
```

Use this method to set an own [QCPSelectionDecorator](#) (subclass) instance. This allows you to customize the visual representation of selected data ranges further than by using the default [QCPSelectionDecorator](#).

The plottable takes ownership of the *decorator*.

The currently set decorator can be accessed via [selectionDecorator](#).

10.398.4.37 setValueAxis()

```
void QCPAbstractPlottable::setValueAxis (
    QCPAxis * axis )
```

The value axis of a plottable can be set to any axis of a [QCustomPlot](#), as long as it is orthogonal to the plottable's key axis. This function performs no checks to make sure this is the case. The typical mathematical choice is to use the x-axis ([QCustomPlot::xAxis](#)) as key axis and the y-axis ([QCustomPlot::yAxis](#)) as value axis.

Normally, the key and value axes are set in the constructor of the plottable (or [QCustomPlot::addGraph](#) when working with QCPGraphs through the dedicated graph interface).

See also

[setKeyAxis](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.399 QCPAbstractPlottable1D< DataType > Class Template Reference

A template base class for plottables with one-dimensional data.

```
#include <applications/gui/genesys/qcustomplot.h>
```


- virtual [QCPRange dataValueRange](#) (int index) const Q_DECL_OVERRIDE
- virtual QPointF [dataPixelPosition](#) (int index) const Q_DECL_OVERRIDE
- virtual bool [sortKeysMainKey](#) () const Q_DECL_OVERRIDE
- virtual [QCPDataSelection selectTestRect](#) (const QRectF &rect, bool onlySelectable) const Q_DECL_OVERRIDE
- virtual int [findBegin](#) (double sortKey, bool expandedRange=true) const Q_DECL_OVERRIDE
- virtual int [findEnd](#) (double sortKey, bool expandedRange=true) const Q_DECL_OVERRIDE
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE
- virtual [QCPPlottableInterface1D * interface1D](#) () Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- QString [name](#) () const
- bool [antialiasedFill](#) () const
- bool [antialiasedScatters](#) () const
- QPen [pen](#) () const
- QBrush [brush](#) () const
- [QCPAxis](#) * [keyAxis](#) () const
- [QCPAxis](#) * [valueAxis](#) () const
- [QCP::SelectionType selectable](#) () const
- bool [selected](#) () const
- [QCPDataSelection selection](#) () const
- [QCPSelectionDecorator](#) * [selectionDecorator](#) () const
- void [setName](#) (const QString &name)
- void [setAntialiasedFill](#) (bool enabled)
- void [setAntialiasedScatters](#) (bool enabled)
- void [setPen](#) (const QPen &pen)
- void [setBrush](#) (const QBrush &brush)
- void [setKeyAxis](#) ([QCPAxis](#) *axis)
- void [setValueAxis](#) ([QCPAxis](#) *axis)
- Q_SLOT void [setSelectable](#) ([QCP::SelectionType](#) selectable)
- Q_SLOT void [setSelection](#) ([QCPDataSelection](#) selection)
- void [setSelectionDecorator](#) ([QCPSelectionDecorator](#) *decorator)
- virtual [QCPRange getKeyRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const =0
- virtual [QCPRange getValueRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const =0
- void [coordsToPixels](#) (double key, double value, double &x, double &y) const
- const QPointF [coordsToPixels](#) (double key, double value) const
- void [pixelsToCoords](#) (double x, double y, double &key, double &value) const
- void [pixelsToCoords](#) (const QPointF &pixelPos, double &key, double &value) const
- void [rescaleAxes](#) (bool onlyEnlarge=false) const
- void [rescaleKeyAxis](#) (bool onlyEnlarge=false) const
- void [rescaleValueAxis](#) (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool [addToLegend](#) ([QCPLegend](#) *legend)
- bool [addToLegend](#) ()
- bool [removeFromLegend](#) ([QCPLegend](#) *legend) const
- bool [removeFromLegend](#) () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const [QString](#) &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- void **getDataSegments** ([QList](#)< [QCPDataRange](#) > &selectedSegments, [QList](#)< [QCPDataRange](#) > &unselectedSegments) const
- void **drawPolyline** ([QCPPainter](#) *painter, const [QVector](#)< [QPointF](#) > &lineData) const

Protected Member Functions inherited from [QCPAbstractPlottable](#)

- virtual [QRect](#) **clipRect** () const Q_DECL_OVERRIDE
- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE=0
- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void **selectEvent** ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState↔ Changed) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void **drawLegendIcon** ([QCPPainter](#) *painter, const [QRectF](#) &rect) const =0
- void **applyFillAntialiasingHint** ([QCPPainter](#) *painter) const
- void **applyScattersAntialiasingHint** ([QCPPainter](#) *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **mouseMoveEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseReleaseEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseDoubleClickEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **wheelEvent** ([QWheelEvent](#) *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QSharedPointer](#)< [QCPDataContainer](#)< [DataType](#) > > **mDataContainer**

Protected Attributes inherited from [QCPAbstractPlottable](#)

- QString **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- QPen **mPen**
- QBrush **mBrush**
- QPointer< [QCPAxis](#) > **mKeyAxis**
- QPointer< [QCPAxis](#) > **mValueAxis**
- [QCP::SelectionType](#) **mSelectable**
- [QCPDataSelection](#) **mSelection**
- [QCPSelectionDecorator](#) * **mSelectionDecorator**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from [QCPAbstractPlottable](#)

- void [selectionChanged](#) (bool selected)
- void [selectionChanged](#) (const [QCPDataSelection](#) &selection)
- void [selectableChanged](#) ([QCP::SelectionType](#) selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.399.1 Detailed Description

```
template<class DataType>
class QCPAbstractPlottable1D< DataType >
```

A template base class for plottables with one-dimensional data.

This template class derives from [QCPAbstractPlottable](#) and from the abstract interface [QCPPlottableInterface1D](#). It serves as a base class for all one-dimensional data (i.e. data with one key dimension), such as [QCPGraph](#) and [QCPCurve](#).

The template parameter *DataType* is the type of the data points of this plottable (e.g. [QCPGraphData](#) or [QCPCurveData](#)). The main purpose of this base class is to provide the member *mDataContainer* (a shared pointer to a [QCPDataContainer<DataType>](#)) and implement the according virtual methods of the [QCPPlottableInterface1D](#), such that most subclassed plottables don't need to worry about this anymore.

Further, it provides a convenience method for retrieving selected/unselected data segments via [getDataSegments](#). This is useful when subclasses implement their draw method and need to draw selected segments with a different pen/brush than unselected segments (also see [QCPSelectionDecorator](#)).

This class implements basic functionality of [QCPAbstractPlottable::selectTest](#) and [QCPPlottableInterface1D::selectTestRect](#), assuming point-like data points, based on the 1D data interface. In spite of that, most plottable subclasses will want to reimplement those methods again, to provide a more accurate hit test based on their specific data visualization geometry.

10.399.2 Constructor & Destructor Documentation

10.399.2.1 QCPAbstractPlottable1D()

```
template<class DataType >
QCPAbstractPlottable1D< DataType >::QCPAbstractPlottable1D (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis )
```

Forwards *keyAxis* and *valueAxis* to the [QCPAbstractPlottable](#) constructor and allocates the *mDataContainer*.

10.399.3 Member Function Documentation

10.399.3.1 dataCount()

```
template<class DataType >
int QCPAbstractPlottable1D< DataType >::dataCount ( ) const [virtual]
```

Returns the number of data points of the plottable.

Implements [QCPPlottableInterface1D](#).

10.399.3.2 dataMainKey()

```
template<class DataType >
double QCPAbstractPlottable1D< DataType >::dataMainKey (
    int index ) const [virtual]
```

Returns the main key of the data point at the given *index*.

What the main key is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

10.399.3.3 dataMainValue()

```
template<class DataType >
double QCPAbstractPlottable1D< DataType >::dataMainValue (
    int index ) const [virtual]
```

Returns the main value of the data point at the given *index*.

What the main value is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

10.399.3.4 dataPixelPosition()

```
template<class DataType >
QPointF QCPAbstractPlottable1D< DataType >::dataPixelPosition (
    int index ) const [virtual]
```

Returns the pixel position on the widget surface at which the data point at the given *index* appears.

Usually this corresponds to the point of dataMainKey/ dataMainValue, in pixel coordinates. However, depending on the plottable, this might be a different apparent position than just a coord-to-pixel transform of those values. For example, [QCPBars](#) apparent data values can be shifted depending on their stacking, bar grouping or configured base value.

Implements [QCPPlottableInterface1D](#).

Reimplemented in [QCPBars](#).

10.399.3.5 dataSortKey()

```
template<class DataType >
double QCPAbstractPlottable1D< DataType >::dataSortKey (
    int index ) const [virtual]
```

Returns the sort key of the data point at the given *index*.

What the sort key is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

10.399.3.6 dataValueRange()

```
template<class DataType >
QCPRange QCPAbstractPlottable1D< DataType >::dataValueRange (
    int index ) const [virtual]
```

Returns the value range of the data point at the given *index*.

What the value range is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

10.399.3.7 drawPolyline()

```
template<class DataType >
void QCPAbstractPlottable1D< DataType >::drawPolyline (
    QCPainter * painter,
    const QVector< QPointF > & lineData ) const [protected]
```

A helper method which draws a line with the passed *painter*, according to the pixel data in *lineData*. NaN points create gaps in the line, as expected from [QCustomPlot](#)'s plottables (this is the main difference to QPainter's regular drawPolyline, which handles NaNs by lagging or crashing).

Further it uses a faster line drawing technique based on [QCPPainter::drawLine](#) rather than `QPainter::drawPolyline` if the configured [QCustomPlot::setPlottingHints\(\)](#) and *painter* style allows.

10.399.3.8 findBegin()

```
template<class DataType >
int QCPAbstractPlottable1D< DataType >::findBegin (
    double sortKey,
    bool expandedRange = true ) const [virtual]
```

Returns the index of the data point with a (sort-)key that is equal to, just below, or just above *sortKey*. If *expandedRange* is true, the data point just below *sortKey* will be considered, otherwise the one just above.

This can be used in conjunction with [findEnd](#) to iterate over data points within a given key range, including or excluding the bounding data points that are just beyond the specified range.

If *expandedRange* is true but there are no data points below *sortKey*, 0 is returned.

If the container is empty, returns 0 (in that case, [findEnd](#) will also return 0, so a loop using these methods will not iterate over the index 0).

See also

[findEnd](#), [QCPDataContainer::findBegin](#)

Implements [QCPPlottableInterface1D](#).

10.399.3.9 findEnd()

```
template<class DataType >
int QCPAbstractPlottable1D< DataType >::findEnd (
    double sortKey,
    bool expandedRange = true ) const [virtual]
```

Returns the index one after the data point with a (sort-)key that is equal to, just above, or just below *sortKey*. If *expandedRange* is true, the data point just above *sortKey* will be considered, otherwise the one just below.

This can be used in conjunction with [findBegin](#) to iterate over data points within a given key range, including the bounding data points that are just below and above the specified range.

If *expandedRange* is true but there are no data points above *sortKey*, the index just above the highest data point is returned.

If the container is empty, returns 0.

See also

[findBegin](#), [QCPDataContainer::findEnd](#)

Implements [QCPPlottableInterface1D](#).

10.399.3.10 getDataSegments()

```
template<class DataType >
void QCPAbstractPlottable1D< DataType >::getDataSegments (
    QList< QCPDataRange > & selectedSegments,
    QList< QCPDataRange > & unselectedSegments ) const [protected]
```

Splits all data into selected and unselected segments and outputs them via *selectedSegments* and *unselectedSegments*, respectively.

This is useful when subclasses implement their draw method and need to draw selected segments with a different pen/brush than unselected segments (also see [QCPSelectionDecorator](#)).

See also

[setSelection](#)

10.399.3.11 interface1D()

```
template<class DataType >
QCPPlottableInterface1D * QCPAbstractPlottable1D< DataType >::interface1D ( ) [inline], [virtual]
```

Returns a [QCPPlottableInterface1D](#) pointer to this plottable, providing access to its 1D interface.

\seebaseclassmethod

Reimplemented from [QCPAbstractPlottable](#).

10.399.3.12 selectTest()

```
template<class DataType >
double QCPAbstractPlottable1D< DataType >::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Implements a point-selection algorithm assuming the data (accessed via the 1D data interface) is point-like. Most subclasses will want to reimplement this method again, to provide a more accurate hit test based on the true data visualization geometry.

If *details* is not 0, it will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

\seebaseclassmethod

Implements [QCPAbstractPlottable](#).

Reimplemented in [QCPGraph](#), [QCPCurve](#), [QCPBars](#), [QCPStatisticalBox](#), and [QCPFinancial](#).

10.399.3.13 selectTestRect()

```
template<class DataType >
QCPDataSelection QCPAbstractPlottable1D< DataType >::selectTestRect (
    const QRectF & rect,
    bool onlySelectable ) const [virtual]
```

Implements a rect-selection algorithm assuming the data (accessed via the 1D data interface) is point-like. Most subclasses will want to reimplement this method again, to provide a more accurate hit test based on the true data visualization geometry.

\see `baseclassmethod`

Implements [QCPPlottableInterface1D](#).

Reimplemented in [QCPBars](#), [QCPStatisticalBox](#), and [QCPFinancial](#).

10.399.3.14 sortKeyIsMainKey()

```
template<class DataType >
bool QCPAbstractPlottable1D< DataType >::sortKeyIsMainKey ( ) const [virtual]
```

Returns whether the sort key ([dataSortKey](#)) is identical to the main key ([dataMainKey](#)).

What the sort and main keys are, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

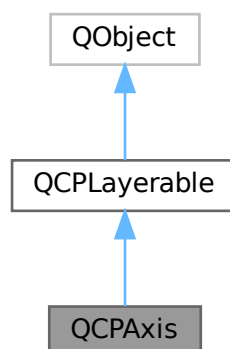
The documentation for this class was generated from the following file:

- applications/gui/genesys/qcustomplot.h

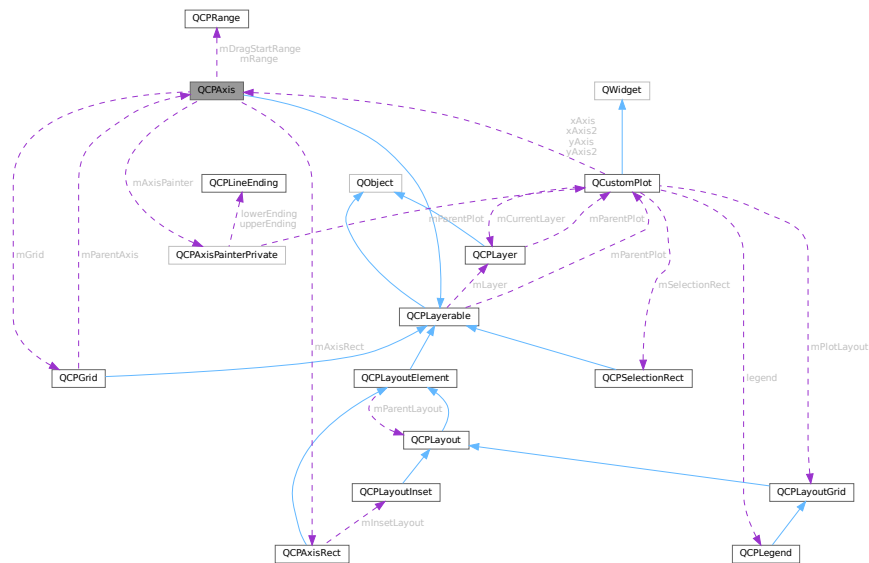
10.400 QCPAxis Class Reference

Manages a single axis inside a [QCustomPlot](#).

Inheritance diagram for QCPAxis:



Collaboration diagram for QCPAxis:



Public Types

- enum [AxisType](#) { [atLeft](#) = 0x01 , [atRight](#) = 0x02 , [atTop](#) = 0x04 , [atBottom](#) = 0x08 }
- enum [LabelSide](#) { [lsInside](#) , [lsOutside](#) }
- enum [ScaleType](#) { [stLinear](#) , [stLogarithmic](#) }
- enum [SelectablePart](#) { [spNone](#) = 0 , [spAxis](#) = 0x001 , [spTickLabels](#) = 0x002 , [spAxisLabel](#) = 0x004 }

Signals

- void [rangeChanged](#) (const [QCPRange](#) &newRange)
- void [rangeChanged](#) (const [QCPRange](#) &newRange, const [QCPRange](#) &oldRange)
- void [scaleTypeChanged](#) ([QCPAxis::ScaleType](#) scaleType)
- void [selectionChanged](#) (const [QCPAxis::SelectableParts](#) &parts)
- void [selectableChanged](#) (const [QCPAxis::SelectableParts](#) &parts)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

Public Member Functions

- [QCPAxis](#) ([QCPAxisRect](#) *parent, [AxisType](#) type)
- [AxisType](#) [axisType](#) () const
- [QCPAxisRect](#) * [axisRect](#) () const
- [ScaleType](#) [scaleType](#) () const
- const [QCPRange](#) [range](#) () const
- bool [rangeReversed](#) () const
- QSharedPointer< [QCPAxisTicker](#) > [ticker](#) () const
- bool [ticks](#) () const

- bool **tickLabels** () const
- int **tickLabelPadding** () const
- QFont **tickLabelFont** () const
- QColor **tickLabelColor** () const
- double **tickLabelRotation** () const
- [LabelSide](#) **tickLabelSide** () const
- QString **numberFormat** () const
- int **numberPrecision** () const
- QVector< double > **tickVector** () const
- QVector< QString > **tickVectorLabels** () const
- int **tickLengthIn** () const
- int **tickLengthOut** () const
- bool **subTicks** () const
- int **subTickLengthIn** () const
- int **subTickLengthOut** () const
- QPen **basePen** () const
- QPen **tickPen** () const
- QPen **subTickPen** () const
- QFont **labelFont** () const
- QColor **labelColor** () const
- QString **label** () const
- int **labelPadding** () const
- int **padding** () const
- int **offset** () const
- SelectableParts **selectedParts** () const
- SelectableParts **selectableParts** () const
- QFont **selectedTickLabelFont** () const
- QFont **selectedLabelFont** () const
- QColor **selectedTickLabelColor** () const
- QColor **selectedLabelColor** () const
- QPen **selectedBasePen** () const
- QPen **selectedTickPen** () const
- QPen **selectedSubTickPen** () const
- [QCPLineEnding](#) **lowerEnding** () const
- [QCPLineEnding](#) **upperEnding** () const
- [QCPGrid](#) * **grid** () const
- Q_SLOT void **setScaleType** ([QCPAxis::ScaleType](#) type)
- Q_SLOT void **setRange** (const [QCPRange](#) &range)
- void **setRange** (double lower, double upper)
- void **setRange** (double position, double size, Qt::AlignmentFlag alignment)
- void **setRangeLower** (double lower)
- void **setRangeUpper** (double upper)
- void **setRangeReversed** (bool reversed)
- void **setTicker** (QSharedPointer< [QCPAxisTicker](#) > ticker)
- void **setTicks** (bool show)
- void **setTickLabels** (bool show)
- void **setTickLabelPadding** (int padding)
- void **setTickLabelFont** (const QFont &font)
- void **setTickLabelColor** (const QColor &color)
- void **setTickLabelRotation** (double degrees)
- void **setTickLabelSide** ([LabelSide](#) side)
- void **setNumberFormat** (const QString &formatCode)
- void **setNumberPrecision** (int precision)
- void **setTickLength** (int inside, int outside=0)
- void **setTickLengthIn** (int inside)

- void [setTickLengthOut](#) (int outside)
- void [setSubTicks](#) (bool show)
- void [setSubTickLength](#) (int inside, int outside=0)
- void [setSubTickLengthIn](#) (int inside)
- void [setSubTickLengthOut](#) (int outside)
- void [setBasePen](#) (const QPen &pen)
- void [setTickPen](#) (const QPen &pen)
- void [setSubTickPen](#) (const QPen &pen)
- void [setLabelFont](#) (const QFont &font)
- void [setLabelColor](#) (const QColor &color)
- void [setLabel](#) (const QString &str)
- void [setLabelPadding](#) (int padding)
- void [setPadding](#) (int padding)
- void [setOffset](#) (int offset)
- void [setSelectedTickLabelFont](#) (const QFont &font)
- void [setSelectedLabelFont](#) (const QFont &font)
- void [setSelectedTickLabelColor](#) (const QColor &color)
- void [setSelectedLabelColor](#) (const QColor &color)
- void [setSelectedBasePen](#) (const QPen &pen)
- void [setSelectedTickPen](#) (const QPen &pen)
- void [setSelectedSubTickPen](#) (const QPen &pen)
- Q_SLOT void [setSelectableParts](#) (const QCPAxis::SelectableParts &selectableParts)
- Q_SLOT void [setSelectedParts](#) (const QCPAxis::SelectableParts &selectedParts)
- void [setLowerEnding](#) (const QCPLLineEnding &ending)
- void [setUpperEnding](#) (const QCPLLineEnding &ending)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_
DECL_OVERRIDE
- Qt::Orientation [orientation](#) () const
- int [pixelOrientation](#) () const
- void [moveRange](#) (double diff)
- void [scaleRange](#) (double factor)
- void [scaleRange](#) (double factor, double center)
- void [setScaleRatio](#) (const QCPAxis *otherAxis, double ratio=1.0)
- void [rescale](#) (bool onlyVisiblePlottables=false)
- double [pixelToCoord](#) (double value) const
- double [coordToPixel](#) (double value) const
- SelectablePart [getPartAt](#) (const QPointF &pos) const
- QList< QCPAbstractPlottable * > [plottables](#) () const
- QList< QCPGraph * > [graphs](#) () const
- QList< QCPAbstractItem * > [items](#) () const

Public Member Functions inherited from QCPLayerable

- QCPLayerable (QCustomPlot *plot, QString targetLayer=QString(), QCPLayerable *parentLayerable=nullptr)
- bool **visible** () const
- QCustomPlot * **parentPlot** () const
- QCPLayerable * **parentLayerable** () const
- QCPLayer * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) (QCPLayer *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Static Public Member Functions

- static [AxisType](#) [marginSideToAxisType](#) ([QCP::MarginSide](#) side)
- static [Qt::Orientation](#) [orientation](#) ([AxisType](#) type)
- static [AxisType](#) [opposite](#) ([AxisType](#) type)

Protected Member Functions

- virtual int **calculateMargin** ()
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const [Q_DECL_OVERRIDE](#)
- virtual void [draw](#) ([QCPPainter](#) *painter) [Q_DECL_OVERRIDE](#)
- virtual [QCP::Interaction](#) [selectionCategory](#) () const [Q_DECL_OVERRIDE](#)
- virtual void [selectEvent](#) ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState, bool *selectionStateChanged) [Q_DECL_OVERRIDE](#)
- virtual void [deselectEvent](#) (bool *selectionStateChanged) [Q_DECL_OVERRIDE](#)
- virtual void [mousePressEvent](#) ([QMouseEvent](#) *event, const [QVariant](#) &details) [Q_DECL_OVERRIDE](#)
- virtual void [mouseMoveEvent](#) ([QMouseEvent](#) *event, const [QPointF](#) &startPos) [Q_DECL_OVERRIDE](#)
- virtual void [mouseReleaseEvent](#) ([QMouseEvent](#) *event, const [QPointF](#) &startPos) [Q_DECL_OVERRIDE](#)
- virtual void [wheelEvent](#) ([QWheelEvent](#) *event) [Q_DECL_OVERRIDE](#)
- void **setupTickVectors** ()
- [QPen](#) **getBasePen** () const
- [QPen](#) **getTickPen** () const
- [QPen](#) **getSubTickPen** () const
- [QFont](#) **getTickLabelFont** () const
- [QFont](#) **getLabelFont** () const
- [QColor](#) **getTickLabelColor** () const
- [QColor](#) **getLabelColor** () const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual [QRect](#) **clipRect** () const
- virtual void [mouseDoubleClickEvent](#) ([QMouseEvent](#) *event, const [QVariant](#) &details)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [AxisType](#) **mAxisType**
- [QCPAxisRect](#) * **mAxisRect**
- int **mPadding**
- [Qt::Orientation](#) **mOrientation**
- [SelectableParts](#) **mSelectableParts**
- [SelectableParts](#) **mSelectedParts**
- [QPen](#) **mBasePen**
- [QPen](#) **mSelectedBasePen**
- [QString](#) **mLabel**
- [QFont](#) **mLabelFont**
- [QFont](#) **mSelectedLabelFont**

- QColor **mLabelColor**
- QColor **mSelectedLabelColor**
- bool **mTickLabels**
- QFont **mTickLabelFont**
- QFont **mSelectedTickLabelFont**
- QColor **mTickLabelColor**
- QColor **mSelectedTickLabelColor**
- int **mNumberPrecision**
- QLatin1Char **mNumberFormatChar**
- bool **mNumberBeautifulPowers**
- bool **mTicks**
- bool **mSubTicks**
- QPen **mTickPen**
- QPen **mSelectedTickPen**
- QPen **mSubTickPen**
- QPen **mSelectedSubTickPen**
- [QCPRange](#) **mRange**
- bool **mRangeReversed**
- [ScaleType](#) **mScaleType**
- [QCPGrid](#) * **mGrid**
- [QCPAxisPainterPrivate](#) * **mAxisPainter**
- QSharedPointer< [QCPAxisTicker](#) > **mTicker**
- QVector< double > **mTickVector**
- QVector< QString > **mTickVectorLabels**
- QVector< double > **mSubTickVector**
- bool **mCachedMarginValid**
- int **mCachedMargin**
- bool **mDragging**
- [QCPRange](#) **mDragStartRange**
- QCP::AntialiasedElements **mAADragBackup**
- QCP::AntialiasedElements **mNotAADragBackup**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class **QCustomPlot**
- class **QCPGrid**
- class **QCPAxisRect**

10.400.1 Detailed Description

Manages a single axis inside a [QCustomPlot](#).

Usually doesn't need to be instantiated externally. Access [QCustomPlot](#)'s default four axes via [QCustomPlot::xAxis](#) (bottom), [QCustomPlot::yAxis](#) (left), [QCustomPlot::xAxis2](#) (top) and [QCustomPlot::yAxis2](#) (right).

Axes are always part of an axis rect, see [QCPAxisRect](#).

Naming convention of axis parts

Overview of the spacings and paddings that define the geometry of an axis. The dashed gray line on the left represents the [QCustomPlot](#) widget border.

Each axis holds an instance of [QCPAxisTicker](#) which is used to generate the tick coordinates and tick labels. You can access the currently installed [ticker](#) or set a new one (possibly one of the specialized subclasses, or your own subclass) via [setTicker](#). For details, see the documentation of [QCPAxisTicker](#).

10.400.2 Member Enumeration Documentation

10.400.2.1 AxisType

enum [QCPAxis::AxisType](#)

Defines at which side of the axis rect the axis will appear. This also affects how the tick marks are drawn, on which side the labels are placed etc.

Enumerator

atLeft	0x01 Axis is vertical and on the left side of the axis rect
atRight	0x02 Axis is vertical and on the right side of the axis rect
atTop	0x04 Axis is horizontal and on the top side of the axis rect
atBottom	0x08 Axis is horizontal and on the bottom side of the axis rect

10.400.2.2 LabelSide

enum [QCPAxis::LabelSide](#)

Defines on which side of the axis the tick labels (numbers) shall appear.

See also

[setTickLabelSide](#)

Enumerator

IsInside	Tick labels will be displayed inside the axis rect and clipped to the inner axis rect.
IsOutside	Tick labels will be displayed outside the axis rect.

10.400.2.3 ScaleType

```
enum QCPAxis::ScaleType
```

Defines the scale of an axis.

See also

[setScaleType](#)

Enumerator

stLinear	Linear scaling.
stLogarithmic	Logarithmic scaling with correspondingly transformed axis coordinates (possibly also setTicker to a QCPAxisTickerLog instance).

10.400.2.4 SelectablePart

```
enum QCPAxis::SelectablePart
```

Defines the selectable parts of an axis.

See also

[setSelectableParts](#), [setSelectedParts](#)

Enumerator

spNone	None of the selectable parts.
spAxis	The axis backbone and tick marks.
spTickLabels	Tick labels (numbers) of this axis (as a whole, not individually)
spAxisLabel	The axis label.

10.400.3 Constructor & Destructor Documentation**10.400.3.1 QCPAxis()**

```
QCPAxis::QCPAxis (
    QCPAxisRect * parent,
    AxisType type ) [explicit]
```

Constructs an Axis instance of Type *type* for the axis rect *parent*.

Usually it isn't necessary to instantiate axes directly, because you can let [QCustomPlot](#) create them for you with [QCPAxisRect::addAxis](#). If you want to use own QCPAxis-subclasses however, create them manually and then inject them also via [QCPAxisRect::addAxis](#).

10.400.4 Member Function Documentation

10.400.4.1 applyDefaultAntialiasingHint()

```
void QCPAxis::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

10.400.4.2 coordToPixel()

```
double QCPAxis::coordToPixel (
    double value ) const
```

Transforms *value*, in coordinates of the axis, to pixel coordinates of the [QCustomPlot](#) widget.

10.400.4.3 deselectEvent()

```
void QCPAxis::deselectEvent (
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.400.4.4 draw()

```
void QCPAxis::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPLayerable](#).

10.400.4.5 getPartAt()

```
QCPAxis::SelectablePart QCPAxis::getPartAt (
    const QPointF & pos ) const
```

Returns the part of the axis that is hit by *pos* (in pixels). The return value of this function is independent of the user-selectable parts defined with [setSelectableParts](#). Further, this function does not change the current selection state of the axis.

If the axis is not visible ([setVisible](#)), this function always returns [spNone](#).

See also

[setSelectedParts](#), [setSelectableParts](#), [QCustomPlot::setInteractions](#)

10.400.4.6 graphs()

```
QList< QCPGraph * > QCPAxis::graphs ( ) const
```

Returns a list of all the graphs that have this axis as key or value axis.

See also

[plottables](#), [items](#)

10.400.4.7 grid()

```
QCPGrid * QCPAxis::grid ( ) const [inline]
```

Returns the [QCPGrid](#) instance belonging to this axis. [Access](#) it to set details about the way the grid is displayed.

10.400.4.8 items()

```
QList< QCPAbstractItem * > QCPAxis::items ( ) const
```

Returns a list of all the items that are associated with this axis. An item is considered associated with an axis if at least one of its positions uses the axis as key or value axis.

See also

[plottables](#), [graphs](#)

10.400.4.9 marginSideToAxisType()

```
QCPAxis::AxisType QCPAxis::marginSideToAxisType (
    QCP::MarginSide side ) [static]
```

Transforms a margin side to the logically corresponding axis type. ([QCP::msLeft](#) to [QCPAxis::atLeft](#), [QCP::msRight](#) to [QCPAxis::atRight](#), etc.)

10.400.4.10 mouseMoveEvent()

```
void QCPAxis::mouseMoveEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user moves the mouse while holding a mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter `startPos` indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

10.400.4.11 mousePressEvent()

```
void QCPAxis::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses a mouse button while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *details* contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the *details* parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

[QCustomPlot](#) uses an event propagation system that works the same as Qt's system. If your layerable doesn't reimplement the [mousePressEvent](#) or explicitly calls `event->ignore()` in its reimplementation, the event will be propagated to the next layerable in the stacking order.

Once a layerable has accepted the [mousePressEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) or [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

10.400.4.12 mouseReleaseEvent()

```
void QCPAxis::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user releases the mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

10.400.4.13 moveRange()

```
void QCPAxis::moveRange (
    double diff )
```

If the scale type ([setScaleType](#)) is [stLinear](#), *diff* is added to the lower and upper bounds of the range. The range is simply moved by *diff*.

If the scale type is [stLogarithmic](#), the range bounds are multiplied by *diff*. This corresponds to an apparent "linear" move in logarithmic scaling by a distance of $\log(\text{diff})$.

10.400.4.14 opposite()

```
QCPAxis::AxisType QCPAxis::opposite (
    QCPAxis::AxisType type ) [static]
```

Returns the axis type that describes the opposite axis of an axis with the specified *type*.

10.400.4.15 orientation() [1/2]

```
Qt::Orientation QCPAxis::orientation ( ) const [inline]
```

Returns the orientation of this axis. The axis orientation (horizontal or vertical) is deduced from the axis type (left, top, right or bottom).

See also

[orientation\(AxisType type\)](#), [pixelOrientation](#)

10.400.4.16 orientation() [2/2]

```
static Qt::Orientation QCPAxis::orientation (
    AxisType type ) [inline], [static]
```

Returns the orientation of the specified axis type

See also

[orientation\(\)](#), [pixelOrientation](#)

10.400.4.17 pixelOrientation()

```
int QCPAxis::pixelOrientation ( ) const [inline]
```

Returns which direction points towards higher coordinate values/keys, in pixel space.

This method returns either 1 or -1. If it returns 1, then going in the positive direction along the orientation of the axis in pixels corresponds to going from lower to higher axis coordinates. On the other hand, if this method returns -1, going to smaller pixel values corresponds to going from lower to higher axis coordinates.

For example, this is useful to easily shift axis coordinates by a certain amount given in pixels, without having to care about reversed or vertically aligned axes:

```
double newKey = keyAxis->pixelToCoord(keyAxis->coordToPixel(oldKey)+10*keyAxis->pixelOrientation());
```

newKey will then contain a key that is ten pixels towards higher keys, starting from *oldKey*.

10.400.4.18 pixelToCoord()

```
double QCPAxis::pixelToCoord (
    double value ) const
```

Transforms *value*, in pixel coordinates of the [QCustomPlot](#) widget, to axis coordinates.

10.400.4.19 plottables()

```
QList< QCPAbstractPlottable * > QCPAxis::plottables ( ) const
```

Returns a list of all the plottables that have this axis as key or value axis.

If you are only interested in plottables of type [QCPGraph](#), see [graphs](#).

See also

[graphs](#), [items](#)

10.400.4.20 rangeChanged [1/2]

```
void QCPAxis::rangeChanged (
    const QCPRange & newRange ) [signal]
```

This signal is emitted when the range of this axis has changed. You can connect it to the [setRange](#) slot of another axis to communicate the new range to the other axis, in order for it to be synchronized.

You may also manipulate/correct the range with [setRange](#) in a slot connected to this signal. This is useful if for example a maximum range span shall not be exceeded, or if the lower/upper range shouldn't go beyond certain values (see [QCPRange::bounded](#)). For example, the following slot would limit the x axis to ranges between 0 and 10:

```
customPlot->xAxis->setRange(newRange.bounded(0, 10))
```

10.400.4.21 rangeChanged [2/2]

```
void QCPAxis::rangeChanged (
    const QCPRange & newRange,
    const QCPRange & oldRange ) [signal]
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Additionally to the new range, this signal also provides the previous range held by the axis as *oldRange*.

10.400.4.22 rescale()

```
void QCPAxis::rescale (
    bool onlyVisiblePlottables = false )
```

Changes the axis range such that all plottables associated with this axis are fully visible in that dimension.

See also

[QCPAbstractPlottable::rescaleAxes](#), [QCustomPlot::rescaleAxes](#)

10.400.4.23 scaleRange() [1/2]

```
void QCPAxis::scaleRange (
    double factor )
```

Scales the range of this axis by *factor* around the center of the current axis range. For example, if *factor* is 2.0, then the axis range will double its size, and the point at the axis range center won't have changed its position in the [QCustomPlot](#) widget (i.e. coordinates around the center will have moved symmetrically closer).

If you wish to scale around a different coordinate than the current axis range center, use the overload [scaleRange\(double factor, double center\)](#).

10.400.4.24 scaleRange() [2/2]

```
void QCPAxis::scaleRange (
    double factor,
    double center )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Scales the range of this axis by *factor* around the coordinate *center*. For example, if *factor* is 2.0, *center* is 1.0, then the axis range will double its size, and the point at coordinate 1.0 won't have changed its position in the [QCustomPlot](#) widget (i.e. coordinates around 1.0 will have moved symmetrically closer to 1.0).

See also

[scaleRange\(double factor\)](#)

10.400.4.25 scaleTypeChanged

```
void QCPAxis::scaleTypeChanged (
    QCPAxis::ScaleType scaleType ) [signal]
```

This signal is emitted when the scale type changes, by calls to [setScaleType](#)

10.400.4.26 selectableChanged

```
void QCPAxis::selectableChanged (
    const QCPAxis::SelectableParts & parts ) [signal]
```

This signal is emitted when the selectability changes, by calls to [setSelectableParts](#)

10.400.4.27 selectEvent()

```
void QCPAxis::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.400.4.28 selectionCategory()

```
QCP::Interaction QCPAxis::selectionCategory ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.400.4.29 selectionChanged

```
void QCPAxis::selectionChanged (
    const QCPAxis::SelectableParts & parts ) [signal]
```

This signal is emitted when the selection state of this axis has changed, either by user interaction or by a direct call to [setSelectedParts](#).

10.400.4.30 selectTest()

```
double QCPAxis::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the `mouseReleaseEvent` occurs, and the finally selected object is notified via the `selectEvent/` `deselectEvent` methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to `selectEvent` when the parent [QCustomPlot](#) decides on the basis of this `selectTest` call, that the object was successfully selected. The subsequent call to `selectEvent` will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent `selectEvent`, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

`selectEvent`, `deselectEvent`, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Reimplemented from [QCPLayerable](#).

10.400.4.31 setBasePen()

```
void QCPAxis::setBasePen (
    const QPen & pen )
```

Sets the pen, the axis base line is drawn with.

See also

[setTickPen](#), [setSubTickPen](#)

10.400.4.32 setLabel()

```
void QCPAxis::setLabel (
    const QString & str )
```

Sets the text of the axis label that will be shown below/above or next to the axis, depending on its orientation. To disable axis labels, pass an empty string as *str*.

10.400.4.33 setLabelColor()

```
void QCPAxis::setLabelColor (
    const QColor & color )
```

Sets the color of the axis label.

See also

[setLabelFont](#)

10.400.4.34 setLabelFont()

```
void QCPAxis::setLabelFont (
    const QFont & font )
```

Sets the font of the axis label.

See also

[setLabelColor](#)

10.400.4.35 setLabelPadding()

```
void QCPAxis::setLabelPadding (
    int padding )
```

Sets the distance between the tick labels and the axis label.

See also

[setTickLabelPadding](#), [setPadding](#)

10.400.4.36 setLowerEnding()

```
void QCPAxis::setLowerEnding (
    const QCPLLineEnding & ending )
```

Sets the style for the lower axis ending. See the documentation of [QCPLLineEnding](#) for available styles.

For horizontal axes, this method refers to the left ending, for vertical axes the bottom ending. Note that this meaning does not change when the axis range is reversed with [setRangeReversed](#).

See also

[setUpperEnding](#)

10.400.4.37 setNumberFormat()

```
void QCPAxis::setNumberFormat (
    const QString & formatCode )
```

Sets the number format for the numbers in tick labels. This *formatCode* is an extended version of the format code used e.g. by `QString::number()` and `QLocale::toString()`. For reference about that, see the "Argument Formats" section in the detailed description of the `QString` class.

formatCode is a string of one, two or three characters.

The first character is identical to the normal format code used by Qt. In short, this means: 'e'/'E' scientific format, 'f' fixed format, 'g'/'G' scientific or fixed, whichever is shorter. For the 'e', 'E', and 'f' formats, the precision set by [setNumberPrecision](#) represents the number of digits after the decimal point. For the 'g' and 'G' formats, the precision represents the maximum number of significant digits, trailing zeroes are omitted.

The second and third characters are optional and specific to [QCustomPlot](#):

If the first char was 'e' or 'g', numbers are/might be displayed in the scientific format, e.g. "5.5e9", which is ugly in a plot. So when the second char of *formatCode* is set to 'b' (for "beautiful"), those exponential numbers are formatted in a more natural way, i.e. "5.5 [multiplication sign] 10 [superscript] 9". By default, the multiplication sign is a centered dot. If instead a cross should be shown (as is usual in the USA), the third char of *formatCode* can be set to 'c'. The inserted multiplication signs are the UTF-8 characters 215 (0xD7) for the cross and 183 (0xB7) for the dot.

Examples for *formatCode*:

- `g` normal format code behaviour. If number is small, fixed format is used, if number is large, normal scientific format is used
- `gb` If number is small, fixed format is used, if number is large, scientific format is used with beautifully typeset decimal powers and a dot as multiplication sign
- `ebc` All numbers are in scientific format with beautifully typeset decimal power and a cross as multiplication sign
- `fb` illegal format code, since fixed format doesn't support (or need) beautifully typeset decimal powers. Format code will be reduced to 'f'.
- `hello` illegal format code, since first char is not 'e', 'E', 'f', 'g' or 'G'. Current format code will not be changed.

10.400.4.38 `setNumberPrecision()`

```
void QCPAxis::setNumberPrecision (
    int precision )
```

Sets the precision of the tick label numbers. See `QLocale::toString(double i, char f, int prec)` for details. The effect of precisions are most notably for number Formats starting with 'e', see [setNumberFormat](#)

10.400.4.39 `setOffset()`

```
void QCPAxis::setOffset (
    int offset )
```

Sets the offset the axis has to its axis rect side.

If an axis rect side has multiple axes and automatic margin calculation is enabled for that side, only the offset of the inner most axis has meaning (even if it is set to be invisible). The offset of the other, outer axes is controlled automatically, to place them at appropriate positions.

10.400.4.40 `setPadding()`

```
void QCPAxis::setPadding (
    int padding )
```

Sets the padding of the axis.

When [QCPAxisRect::setAutoMargins](#) is enabled, the padding is the additional outer most space, that is left blank.

The axis padding has no meaning if [QCPAxisRect::setAutoMargins](#) is disabled.

See also

[setLabelPadding](#), [setTickLabelPadding](#)

10.400.4.41 `setRange()` [1/3]

```
void QCPAxis::setRange (
    const QCPRange & range )
```

Sets the range of the axis.

This slot may be connected with the [rangeChanged](#) signal of another axis so this axis is always synchronized with the other axis range, when it changes.

To invert the direction of an axis, use [setRangeReversed](#).

10.400.4.42 `setRange()` [2/3]

```
void QCPAxis::setRange (
    double lower,
    double upper )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the lower and upper bound of the axis range.

To invert the direction of an axis, use [setRangeReversed](#).

There is also a slot to set a range, see [setRange\(const QCPRange &range\)](#).

10.400.4.43 `setRange()` [3/3]

```
void QCPAxis::setRange (
    double position,
    double size,
    Qt::AlignmentFlag alignment )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the range of the axis.

The *position* coordinate indicates together with the *alignment* parameter, where the new range will be positioned. *size* defines the size of the new axis range. *alignment* may be `Qt::AlignLeft`, `Qt::AlignRight` or `Qt::AlignCenter`. This will cause the left border, right border, or center of the range to be aligned with *position*. Any other values of *alignment* will default to `Qt::AlignCenter`.

10.400.4.44 `setRangeLower()`

```
void QCPAxis::setRangeLower (
    double lower )
```

Sets the lower bound of the axis range. The upper bound is not changed.

See also

[setRange](#)

10.400.4.45 `setRangeReversed()`

```
void QCPAxis::setRangeReversed (
    bool reversed )
```

Sets whether the axis range (direction) is displayed reversed. Normally, the values on horizontal axes increase left to right, on vertical axes bottom to top. When *reversed* is set to true, the direction of increasing values is inverted.

Note that the range and data interface stays the same for reversed axes, e.g. the *lower* part of the [setRange](#) interface will still reference the mathematically smaller number than the *upper* part.

10.400.4.46 `setRangeUpper()`

```
void QCPAxis::setRangeUpper (
    double upper )
```

Sets the upper bound of the axis range. The lower bound is not changed.

See also

[setRange](#)

10.400.4.47 `setScaleRatio()`

```
void QCPAxis::setScaleRatio (
    const QCPAxis * otherAxis,
    double ratio = 1.0 )
```

Scales the range of this axis to have a certain scale *ratio* to *otherAxis*. The scaling will be done around the center of the current axis range.

For example, if *ratio* is 1, this axis is the *yAxis* and *otherAxis* is *xAxis*, graphs plotted with those axes will appear in a 1:1 aspect ratio, independent of the aspect ratio the axis rect has.

This is an operation that changes the range of this axis once, it doesn't fix the scale ratio indefinitely. Note that calling this function in the constructor of the [QCustomPlot](#)'s parent won't have the desired effect, since the widget dimensions aren't defined yet, and a `resizeEvent` will follow.

10.400.4.48 `setScaleType()`

```
void QCPAxis::setScaleType (
    QCPAxis::ScaleType type )
```

Sets whether the axis uses a linear scale or a logarithmic scale.

Note that this method controls the coordinate transformation. For logarithmic scales, you will likely also want to use a logarithmic tick spacing and labeling, which can be achieved by setting the axis ticker to an instance of [QCPAxisTickerLog](#) :

See the documentation of [QCPAxisTickerLog](#) about the details of logarithmic axis tick creation.

[setNumberPrecision](#)

10.400.4.49 `setSelectableParts()`

```
void QCPAxis::setSelectableParts (
    const QCPAxis::SelectableParts & selectableParts )
```

Sets whether the user can (de-)select the parts in *selectable* by clicking on the [QCustomPlot](#) surface. (When [QCustomPlot::setInteractions](#) contains `iSelectAxes`.)

However, even when *selectable* is set to a value not allowing the selection of a specific part, it is still possible to set the selection of this part manually, by calling [setSelectedParts](#) directly.

See also

[SelectablePart](#), [setSelectedParts](#)

10.400.4.50 setSelectedBasePen()

```
void QCPAxis::setSelectedBasePen (
    const QPen & pen )
```

Sets the pen that is used to draw the axis base line when selected.

See also

[setBasePen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.400.4.51 setSelectedLabelColor()

```
void QCPAxis::setSelectedLabelColor (
    const QColor & color )
```

Sets the color that is used for the axis label when it is selected.

See also

[setLabelColor](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.400.4.52 setSelectedLabelFont()

```
void QCPAxis::setSelectedLabelFont (
    const QFont & font )
```

Sets the font that is used for the axis label when it is selected.

See also

[setLabelFont](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.400.4.53 setSelectedParts()

```
void QCPAxis::setSelectedParts (
    const QCPAxis::SelectableParts & selectedParts )
```

Sets the selected state of the respective axis parts described by [SelectablePart](#). When a part is selected, it uses a different pen/font.

The entire selection mechanism for axes is handled automatically when [QCustomPlot::setInteractions](#) contains `SelectableParts`. You only need to call this function when you wish to change the selection state manually.

This function can change the selection state of a part, independent of the [setSelectableParts](#) setting.

emits the [selectionChanged](#) signal when *selected* is different from the previous selection state.

See also

[SelectablePart](#), [setSelectableParts](#), [selectTest](#), [setSelectedBasePen](#), [setSelectedTickPen](#), [setSelectedSubTickPen](#), [setSelectedTickLabelFont](#), [setSelectedLabelFont](#), [setSelectedTickLabelColor](#), [setSelectedLabelColor](#)

10.400.4.54 setSubTickPen()

```
void QCPAxis::setSubTickPen (
    const QPen & pen )
```

Sets the pen that is used to draw the subticks when selected.

See also

[setSubTickPen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.400.4.55 setSelectedTickLabelColor()

```
void QCPAxis::setSelectedTickLabelColor (
    const QColor & color )
```

Sets the color that is used for tick labels when they are selected.

See also

[setTickLabelColor](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.400.4.56 setSelectedTickLabelFont()

```
void QCPAxis::setSelectedTickLabelFont (
    const QFont & font )
```

Sets the font that is used for tick labels when they are selected.

See also

[setTickLabelFont](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.400.4.57 setSelectedTickPen()

```
void QCPAxis::setSelectedTickPen (
    const QPen & pen )
```

Sets the pen that is used to draw the (major) ticks when selected.

See also

[setTickPen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.400.4.58 setSubTickLength()

```
void QCPAxis::setSubTickLength (
    int inside,
    int outside = 0 )
```

Sets the length of the subticks in pixels. *inside* is the length the subticks will reach inside the plot and *outside* is the length they will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setTickLength](#), [setSubTickLengthIn](#), [setSubTickLengthOut](#)

10.400.4.59 setSubTickLengthIn()

```
void QCPAxis::setSubTickLengthIn (
    int inside )
```

Sets the length of the inward subticks in pixels. *inside* is the length the subticks will reach inside the plot.

See also

[setSubTickLengthOut](#), [setSubTickLength](#), [setTickLength](#)

10.400.4.60 setSubTickLengthOut()

```
void QCPAxis::setSubTickLengthOut (
    int outside )
```

Sets the length of the outward subticks in pixels. *outside* is the length the subticks will reach outside the plot. If *outside* is greater than zero, the tick labels will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setSubTickLengthIn](#), [setSubTickLength](#), [setTickLength](#)

10.400.4.61 setSubTickPen()

```
void QCPAxis::setSubTickPen (
    const QPen & pen )
```

Sets the pen, subtick marks will be drawn with.

See also

[setSubTickCount](#), [setSubTickLength](#), [setBasePen](#)

10.400.4.62 setSubTicks()

```
void QCPAxis::setSubTicks (
    bool show )
```

Sets whether sub tick marks are displayed.

Sub ticks are only potentially visible if (major) ticks are also visible (see [setTicks](#))

See also

[setTicks](#)

10.400.4.63 setTicker()

```
void QCPAxis::setTicker (
    QSharedPointer< QCPAxisTicker > ticker )
```

The axis ticker is responsible for generating the tick positions and tick labels. See the documentation of [QCPAxisTicker](#) for details on how to work with axis tickers.

You can change the tick positioning/labeling behaviour of this axis by setting a different [QCPAxisTicker](#) subclass using this method. If you only wish to modify the currently installed axis ticker, access it via [ticker](#).

Since the ticker is stored in the axis as a shared pointer, multiple axes may share the same axis ticker simply by passing the same shared pointer to multiple axes.

See also

[ticker](#)

10.400.4.64 setTickLabelColor()

```
void QCPAxis::setTickLabelColor (
    const QColor & color )
```

Sets the color of the tick labels.

See also

[setTickLabels](#), [setTickLabelFont](#)

10.400.4.65 setTickLabelFont()

```
void QCPAxis::setTickLabelFont (
    const QFont & font )
```

Sets the font of the tick labels.

See also

[setTickLabels](#), [setTickLabelColor](#)

10.400.4.66 `setTickLabelPadding()`

```
void QCPAxis::setTickLabelPadding (
    int padding )
```

Sets the distance between the axis base line (including any outward ticks) and the tick labels.

See also

[setLabelPadding](#), [setPadding](#)

10.400.4.67 `setTickLabelRotation()`

```
void QCPAxis::setTickLabelRotation (
    double degrees )
```

Sets the rotation of the tick labels. If *degrees* is zero, the labels are drawn normally. Else, the tick labels are drawn rotated by *degrees* clockwise. The specified angle is bound to values from -90 to 90 degrees.

If *degrees* is exactly -90, 0 or 90, the tick labels are centered on the tick coordinate. For other angles, the label is drawn with an offset such that it seems to point toward or away from the tick mark.

10.400.4.68 `setTickLabels()`

```
void QCPAxis::setTickLabels (
    bool show )
```

Sets whether tick labels are displayed. Tick labels are the numbers drawn next to tick marks.

10.400.4.69 `setTickLabelSide()`

```
void QCPAxis::setTickLabelSide (
    LabelSide side )
```

Sets whether the tick labels (numbers) shall appear inside or outside the axis rect.

The usual and default setting is [IsOutside](#). Very compact plots sometimes require tick labels to be inside the axis rect, to save space. If *side* is set to [IsInside](#), the tick labels appear on the inside are additionally clipped to the axis rect.

10.400.4.70 `setTickLength()`

```
void QCPAxis::setTickLength (
    int inside,
    int outside = 0 )
```

Sets the length of the ticks in pixels. *inside* is the length the ticks will reach inside the plot and *outside* is the length they will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setSubTickLength](#), [setTickLengthIn](#), [setTickLengthOut](#)

10.400.4.71 setTickLengthIn()

```
void QCPAxis::setTickLengthIn (
    int inside )
```

Sets the length of the inward ticks in pixels. *inside* is the length the ticks will reach inside the plot.

See also

[setTickLengthOut](#), [setTickLength](#), [setSubTickLength](#)

10.400.4.72 setTickLengthOut()

```
void QCPAxis::setTickLengthOut (
    int outside )
```

Sets the length of the outward ticks in pixels. *outside* is the length the ticks will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setTickLengthIn](#), [setTickLength](#), [setSubTickLength](#)

10.400.4.73 setTickPen()

```
void QCPAxis::setTickPen (
    const QPen & pen )
```

Sets the pen, tick marks will be drawn with.

See also

[setTickLength](#), [setBasePen](#)

10.400.4.74 setTicks()

```
void QCPAxis::setTicks (
    bool show )
```

Sets whether tick marks are displayed.

Note that setting *show* to false does not imply that tick labels are invisible, too. To achieve that, see [setTickLabels](#).

See also

[setSubTicks](#)

10.400.4.75 `setUpperEnding()`

```
void QCPAxis::setUpperEnding (
    const QCPLLineEnding & ending )
```

Sets the style for the upper axis ending. See the documentation of [QCPLLineEnding](#) for available styles.

For horizontal axes, this method refers to the right ending, for vertical axes the top ending. Note that this meaning does not change when the axis range is reversed with [setRangeReversed](#).

See also

[setLowerEnding](#)

10.400.4.76 `ticker()`

```
QSharedPointer< QCPAxisTicker > QCPAxis::ticker ( ) const [inline]
```

Returns a modifiable shared pointer to the currently installed axis ticker. The axis ticker is responsible for generating the tick positions and tick labels of this axis. You can access the [QCPAxisTicker](#) with this method and modify basic properties such as the approximate tick count ([QCPAxisTicker::setTickCount](#)).

You can gain more control over the axis ticks by setting a different [QCPAxisTicker](#) subclass, see the documentation there. A new axis ticker can be set with [setTicker](#).

Since the ticker is stored in the axis as a shared pointer, multiple axes may share the same axis ticker simply by passing the same shared pointer to multiple axes.

See also

[setTicker](#)

10.400.4.77 `wheelEvent()`

```
void QCPAxis::wheelEvent (
    QWheelEvent * event ) [protected], [virtual]
```

This event gets called when the user turns the mouse scroll wheel while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`.

The `event->angleDelta()` indicates how far the mouse wheel was turned, which is usually +/- 120 for single rotation steps. However, if the mouse wheel is turned rapidly, multiple steps may accumulate to one event, making the delta larger. On the other hand, if the wheel has very smooth steps or none at all, the delta may be smaller.

The default implementation does nothing.

See also

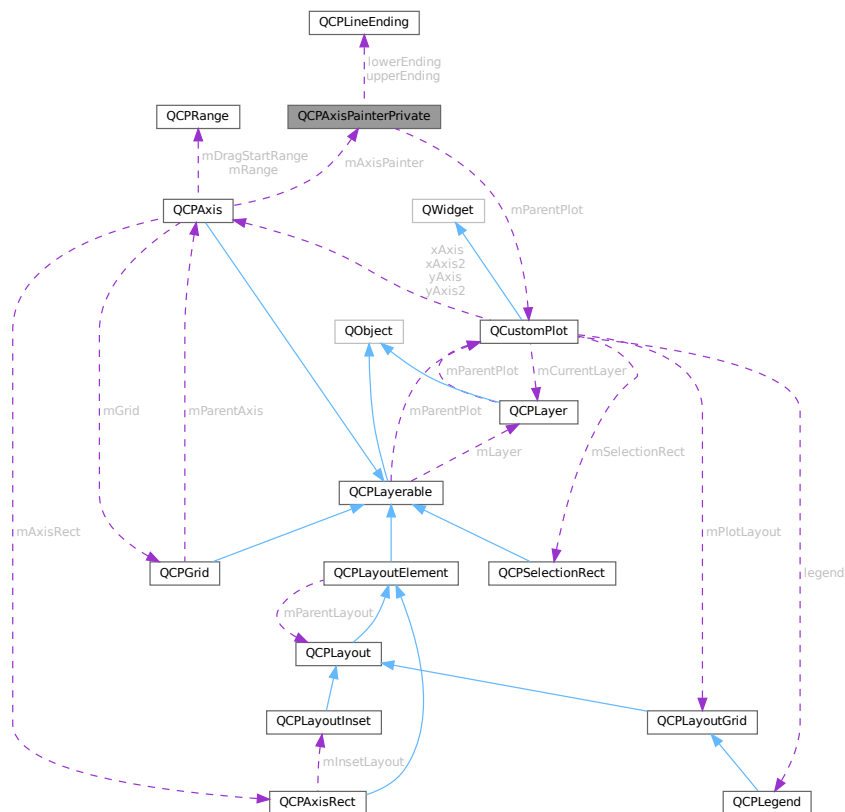
[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#)

Reimplemented from [QCPLayerable](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

Collaboration diagram for QCPAxisPainterPrivate:



- struct `CachedLabel`
- struct `TickLabelData`

- **QCPAxisPainterPrivate** (QCustomPlot *parentPlot)
- virtual void **draw** (QCPPainter *painter)
- virtual int **size** ()
- void **clearCache** ()
- QRect **axisSelectionBox** () const
- QRect **tickLabelsSelectionBox** () const
- QRect **labelSelectionBox** () const

Public Attributes

- [QCPAxis::AxisType](#) type
- QPen **basePen**
- [QCPLineEnding](#) lowerEnding
- [QCPLineEnding](#) upperEnding
- int **labelPadding**
- QFont **labelFont**
- QColor **labelColor**
- QString **label**
- int **tickLabelPadding**
- double **tickLabelRotation**
- [QCPAxis::LabelSide](#) tickLabelSide
- bool **substituteExponent**
- bool **numberMultiplyCross**
- int **tickLengthIn**
- int **tickLengthOut**
- int **subTickLengthIn**
- int **subTickLengthOut**
- QPen **tickPen**
- QPen **subTickPen**
- QFont **tickLabelFont**
- QColor **tickLabelColor**
- QRect **axisRect**
- QRect **viewportRect**
- int **offset**
- bool **abbreviateDecimalPowers**
- bool **reversedEndings**
- QVector< double > **subTickPositions**
- QVector< double > **tickPositions**
- QVector< QString > **tickLabels**

Protected Member Functions

- virtual QByteArray **generateLabelParameterHash** () const
- virtual void **placeTickLabel** ([QCPPainter](#) *painter, double position, int distanceToAxis, const QString &text, QSize *tickLabelsSize)
- virtual void **drawTickLabel** ([QCPPainter](#) *painter, double x, double y, const [TickLabelData](#) &labelData) const
- virtual [TickLabelData](#) **getTickLabelData** (const QFont &font, const QString &text) const
- virtual QPointF **getTickLabelDrawOffset** (const [TickLabelData](#) &labelData) const
- virtual void **getMaxTickLabelSize** (const QFont &font, const QString &text, QSize *tickLabelsSize) const

Protected Attributes

- [QCustomPlot](#) * **mParentPlot**
- QByteArray **mLabelParameterHash**
- QCache< QString, [CachedLabel](#) > **mLabelCache**
- QRect **mAxisSelectionBox**
- QRect **mTickLabelsSelectionBox**
- QRect **mLabelSelectionBox**

10.401.1 Constructor & Destructor Documentation

10.401.1.1 QCPAxisPainterPrivate()

```
QCPAxisPainterPrivate::QCPAxisPainterPrivate (
    QCustomPlot * parentPlot ) [explicit]
```

Constructs a [QCPAxisPainterPrivate](#) instance. Make sure to not create a new instance on every redraw, to utilize the caching mechanisms.

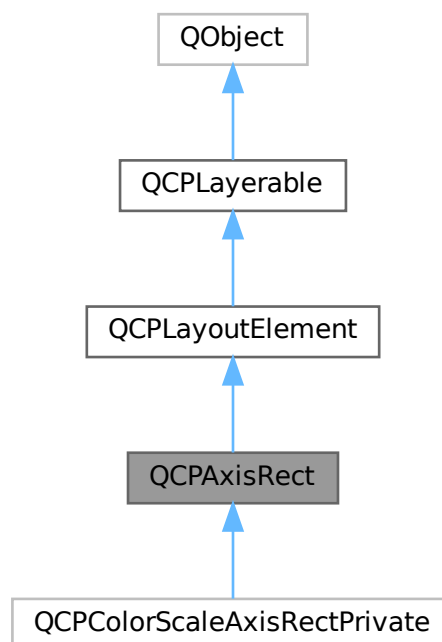
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.402 QCPAxisRect Class Reference

Holds multiple axes and arranges them in a rectangular shape.

Inheritance diagram for QCPAxisRect:



- `QList< QCPAxis * > axes () const`
- `QCPAxis * addAxis (QCPAxis::AxisType type, QCPAxis *axis=nullptr)`
- `QList< QCPAxis * > addAxes (QCPAxis::AxisTypes types)`
- `bool removeAxis (QCPAxis *axis)`
- `QCPLayoutInset * insetLayout () const`
- `void zoom (const QRectF &pixelRect)`
- `void zoom (const QRectF &pixelRect, const QList< QCPAxis * > &affectedAxes)`
- `void setupFullAxesBox (bool connectRanges=false)`
- `QList< QCPAbstractPlottable * > plottables () const`
- `QList< QCPGraph * > graphs () const`
- `QList< QCPAbstractItem * > items () const`
- `int left () const`
- `int right () const`
- `int top () const`
- `int bottom () const`
- `int width () const`
- `int height () const`
- `QSize size () const`
- `QPoint topLeft () const`
- `QPoint topRight () const`
- `QPoint bottomLeft () const`
- `QPoint bottomRight () const`
- `QPoint center () const`
- `virtual void update (UpdatePhase phase) Q_DECL_OVERRIDE`
- `virtual QList< QCPLayoutElement * > elements (bool recursive) const Q_DECL_OVERRIDE`

Public Member Functions inherited from QCPLayoutElement

- `QCPLayoutElement (QCustomPlot *parentPlot=nullptr)`
- `QCPLayout * layout () const`
- `QRect rect () const`
- `QRect outerRect () const`
- `QMargins margins () const`
- `QMargins minimumMargins () const`
- `QCP::MarginSides autoMargins () const`
- `QSize minimumSize () const`
- `QSize maximumSize () const`
- `SizeConstraintRect sizeConstraintRect () const`
- `QCPMarginGroup * marginGroup (QCP::MarginSide side) const`
- `QHash< QCP::MarginSide, QCPMarginGroup * > marginGroups () const`
- `void setOuterRect (const QRect &rect)`
- `void setMargins (const QMargins &margins)`
- `void setMinimumMargins (const QMargins &margins)`
- `void setAutoMargins (QCP::MarginSides sides)`
- `void setMinimumSize (const QSize &size)`
- `void setMinimumSize (int width, int height)`
- `void setMaximumSize (const QSize &size)`
- `void setMaximumSize (int width, int height)`
- `void setSizeConstraintRect (SizeConstraintRect constraintRect)`
- `void setMarginGroup (QCP::MarginSides sides, QCPMarginGroup *group)`
- `virtual QSize minimumOuterSizeHint () const`
- `virtual QSize maximumOuterSizeHint () const`
- `virtual double selectTest (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE`

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const [QString](#) &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side) Q_DECL_OVERRIDE
- virtual void **layoutChanged** () Q_DECL_OVERRIDE
- virtual void **mousePressEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details) Q_DECL_OVERRIDE
- virtual void **mouseMoveEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos) Q_DECL_OVERRIDE
- virtual void **mouseReleaseEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos) Q_DECL_OVERRIDE
- virtual void **wheelEvent** ([QWheelEvent](#) *event) Q_DECL_OVERRIDE
- void **drawBackground** ([QCPPainter](#) *painter)
- void **updateAxesOffset** ([QCPAxis::AxisType](#) type)

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual [QRect](#) **clipRect** () const
- virtual void **selectEvent** ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState↔ Changed)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mouseDoubleClickEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QBrush **mBackgroundBrush**
- QPixmap **mBackgroundPixmap**
- QPixmap **mScaledBackgroundPixmap**
- bool **mBackgroundScaled**
- Qt::AspectRatioMode **mBackgroundScaledMode**
- [QCPLayoutInset](#) * **mInsetLayout**
- Qt::Orientations **mRangeDrag**
- Qt::Orientations **mRangeZoom**
- QList< QPointer< [QCPAxis](#) > > **mRangeDragHorzAxis**
- QList< QPointer< [QCPAxis](#) > > **mRangeDragVertAxis**
- QList< QPointer< [QCPAxis](#) > > **mRangeZoomHorzAxis**
- QList< QPointer< [QCPAxis](#) > > **mRangeZoomVertAxis**
- double **mRangeZoomFactorHorz**
- double **mRangeZoomFactorVert**
- QList< [QCPRange](#) > **mDragStartHorzRange**
- QList< [QCPRange](#) > **mDragStartVertRange**
- QCP::AntialiasedElements **mAADragBackup**
- QCP::AntialiasedElements **mNotAADragBackup**
- bool **mDragging**
- QHash< [QCPAxis::AxisType](#), QList< [QCPAxis](#) * > > **mAxes**

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)

Additional Inherited Members

Public Types inherited from [QCPLayoutElement](#)

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.402.1 Detailed Description

Holds multiple axes and arranges them in a rectangular shape.

This class represents an axis rect, a rectangular area that is bounded on all sides with an arbitrary number of axes.

Initially [QCustomPlot](#) has one axis rect, accessible via [QCustomPlot::axisRect\(\)](#). However, the layout system allows to have multiple axis rects, e.g. arranged in a grid layout ([QCustomPlot::plotLayout](#)).

By default, [QCPAxisRect](#) comes with four axes, at bottom, top, left and right. They can be accessed via [axis](#) by providing the respective axis type ([QCPAxis::AxisType](#)) and index. If you need all axes in the axis rect, use [axes](#). The top and right axes are set to be invisible initially ([QCPAxis::setVisible](#)). To add more axes to a side, use [addAxis](#) or [addAxes](#). To remove an axis, use [removeAxis](#).

The axis rect layerable itself only draws a background pixmap or color, if specified ([setBackground](#)). It is placed on the "background" layer initially (see [QCPLayer](#) for an explanation of the [QCustomPlot](#) layer system). The axes that are held by the axis rect can be placed on other layers, independently of the axis rect.

Every axis rect has a child layout of type [QCPLayoutInset](#). It is accessible via [insetLayout](#) and can be used to have other layout elements (or even other layouts with multiple elements) hovering inside the axis rect.

If an axis rect is clicked and dragged, it processes this by moving certain axis ranges. The behaviour can be controlled with [setRangeDrag](#) and [setRangeDragAxes](#). If the mouse wheel is scrolled while the cursor is on the axis rect, certain axes are scaled. This is controllable via [setRangeZoom](#), [setRangeZoomAxes](#) and [setRangeZoomFactor](#). These interactions are only enabled if [QCustomPlot::setInteractions](#) contains [QCP::iRangeDrag](#) and [QCP::iRangeZoom](#).

Overview of the spacings and paddings that define the geometry of an axis. The dashed line on the far left indicates the viewport/widget border.

10.402.2 Constructor & Destructor Documentation

10.402.2.1 [QCPAxisRect\(\)](#)

```
QCPAxisRect::QCPAxisRect (
    QCustomPlot * parentPlot,
    bool setupDefaultAxes = true ) [explicit]
```

Creates a [QCPAxisRect](#) instance and sets default values. An axis is added for each of the four sides, the top and right axes are set invisible initially.

10.402.3 Member Function Documentation

10.402.3.1 addAxes()

```
QList< QCPAxis * > QCPAxisRect::addAxes (
    QCPAxis::AxisTypes types )
```

Adds a new axis with [addAxis](#) to each axis rect side specified in *types*. This may be an `or`-combination of [QCPAxis::AxisType](#), so axes can be added to multiple sides at once.

Returns a list of the added axes.

See also

[addAxis](#), [setupFullAxesBox](#)

10.402.3.2 addAxis()

```
QCPAxis * QCPAxisRect::addAxis (
    QCPAxis::AxisType type,
    QCPAxis * axis = nullptr )
```

Adds a new axis to the axis rect side specified with *type*, and returns it. If *axis* is 0, a new [QCPAxis](#) instance is created internally. [QCustomPlot](#) owns the returned axis, so if you want to remove an axis, use [removeAxis](#) instead of deleting it manually.

You may inject [QCPAxis](#) instances (or subclasses of [QCPAxis](#)) by setting *axis* to an axis that was previously created outside [QCustomPlot](#). It is important to note that [QCustomPlot](#) takes ownership of the axis, so you may not delete it afterwards. Further, the *axis* must have been created with this axis rect as parent and with the same axis type as specified in *type*. If this is not the case, a debug output is generated, the axis is not added, and the method returns `nullptr`.

This method can not be used to move *axis* between axis rects. The same *axis* instance must not be added multiple times to the same or different axis rects.

If an axis rect side already contains one or more axes, the lower and upper endings of the new axis ([QCPAxis::setLowerEnding](#), [QCPAxis::setUpperEnding](#)) are set to [QCPLineEnding::esHalfBar](#).

See also

[addAxes](#), [setupFullAxesBox](#)

10.402.3.3 applyDefaultAntialiasingHint()

```
void QCPAxisRect::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

10.402.3.4 axes() [1/2]

```
QList< QCPAxis * > QCPAxisRect::axes ( ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns all axes of this axis rect.

10.402.3.5 axes() [2/2]

```
QList< QCPAxis * > QCPAxisRect::axes (
    QCPAxis::AxisTypes types ) const
```

Returns all axes on the axis rect sides specified with *types*.

types may be a single [QCPAxis::AxisType](#) or an `or`-combination, to get the axes of multiple sides.

See also

[axis](#)

10.402.3.6 axis()

```
QCPAxis * QCPAxisRect::axis (
    QCPAxis::AxisType type,
    int index = 0 ) const
```

Returns the axis with the given *index* on the axis rect side specified with *type*.

See also

[axisCount](#), [axes](#)

10.402.3.7 axisCount()

```
int QCPAxisRect::axisCount (
    QCPAxis::AxisType type ) const
```

Returns the number of axes on the axis rect side specified with *type*.

See also

[axis](#)

10.402.3.8 bottom()

```
int QCPAxisRect::bottom ( ) const [inline]
```

Returns the pixel position of the bottom border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.402.3.9 bottomLeft()

```
QPoint QCPAxisRect::bottomLeft ( ) const [inline]
```

Returns the bottom left corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.402.3.10 bottomRight()

```
QPoint QCPAxisRect::bottomRight ( ) const [inline]
```

Returns the bottom right corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.402.3.11 calculateAutoMargin()

```
int QCPAxisRect::calculateAutoMargin (
    QCP::MarginSide side ) [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

10.402.3.12 center()

```
QPoint QCPAxisRect::center ( ) const [inline]
```

Returns the center of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.402.3.13 draw()

```
void QCPAxisRect::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

10.402.3.14 elements()

```
QList< QCPLayoutElement * > QCPAxisRect::elements (
    bool recursive ) const [virtual]
```

Returns a list of all child elements in this layout element. If *recursive* is true, all sub-child elements are included in the list, too.

Warning

There may be `nullptr` entries in the returned list. For example, [QCPLayoutGrid](#) may have empty cells which yield `nullptr` at the respective index.

Reimplemented from [QCPLayoutElement](#).

10.402.3.15 graphs()

```
QList< QCPGraph * > QCPAxisRect::graphs ( ) const
```

Returns a list of all the graphs that are associated with this axis rect.

A graph is considered associated with an axis rect if its key or value axis (or both) is in this axis rect.

See also

[plottables](#), [items](#)

10.402.3.16 height()

```
int QCPAxisRect::height ( ) const [inline]
```

Returns the pixel height of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.402.3.17 insetLayout()

```
QCPLayoutInset * QCPAxisRect::insetLayout ( ) const [inline]
```

Returns the inset layout of this axis rect. It can be used to place other layout elements (or even layouts with multiple other elements) inside/on top of an axis rect.

See also

[QCPLayoutInset](#)

10.402.3.18 items()

```
QList< QCPAbstractItem * > QCPAxisRect::items ( ) const
```

Returns a list of all the items that are associated with this axis rect.

An item is considered associated with an axis rect if any of its positions has key or value axis set to an axis that is in this axis rect, or if any of its positions has [QCPLItemPosition::setAxisRect](#) set to the axis rect, or if the clip axis rect ([QCPAbstractItem::setClipAxisRect](#)) is set to this axis rect.

See also

[plottables](#), [graphs](#)

10.402.3.19 layoutChanged()

```
void QCPAxisRect::layoutChanged ( ) [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

10.402.3.20 left()

```
int QCPAxisRect::left ( ) const [inline]
```

Returns the pixel position of the left border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.402.3.21 mouseMoveEvent()

```
void QCPAxisRect::mouseMoveEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user moves the mouse while holding a mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

Reimplemented in [QCPColorScaleAxisRectPrivate](#).

10.402.3.22 mousePressEvent()

```
void QCPAxisRect::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses a mouse button while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *details* contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the *details* parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

[QCustomPlot](#) uses an event propagation system that works the same as Qt's system. If your layerable doesn't reimplement the [mousePressEvent](#) or explicitly calls `event->ignore()` in its reimplementation, the event will be propagated to the next layerable in the stacking order.

Once a layerable has accepted the [mousePressEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) or [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

Reimplemented in [QCPColorScaleAxisRectPrivate](#).

10.402.3.23 mouseReleaseEvent()

```
void QCPAxisRect::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user releases the mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

Reimplemented in [QCPColorScaleAxisRectPrivate](#).

10.402.3.24 plottables()

```
QList< QCPAbstractPlottable * > QCPAxisRect::plottables ( ) const
```

Returns a list of all the plottables that are associated with this axis rect.

A plottable is considered associated with an axis rect if its key or value axis (or both) is in this axis rect.

See also

[graphs](#), [items](#)

10.402.3.25 rangeDragAxes()

```
QList< QCPAxis * > QCPAxisRect::rangeDragAxes (
    Qt::Orientation orientation )
```

Returns all range drag axes of the *orientation* provided.

See also

[rangeZoomAxis](#), [setRangeZoomAxes](#)

10.402.3.26 rangeDragAxis()

```
QCPAxis * QCPAxisRect::rangeDragAxis (
    Qt::Orientation orientation )
```

Returns the range drag axis of the *orientation* provided. If multiple axes were set, returns the first one (use [rangeDragAxes](#) to retrieve a list with all set axes).

See also

[setRangeDragAxes](#)

10.402.3.27 rangeZoomAxes()

```
QList< QCPAxis * > QCPAxisRect::rangeZoomAxes (
    Qt::Orientation orientation )
```

Returns all range zoom axes of the *orientation* provided.

See also

[rangeDragAxis](#), [setRangeDragAxes](#)

10.402.3.28 rangeZoomAxis()

```
QCPAxis * QCPAxisRect::rangeZoomAxis (
    Qt::Orientation orientation )
```

Returns the range zoom axis of the *orientation* provided. If multiple axes were set, returns the first one (use [rangeZoomAxes](#) to retrieve a list with all set axes).

See also

[setRangeZoomAxes](#)

10.402.3.29 rangeZoomFactor()

```
double QCPAxisRect::rangeZoomFactor (
    Qt::Orientation orientation )
```

Returns the range zoom factor of the *orientation* provided.

See also

[setRangeZoomFactor](#)

10.402.3.30 `removeAxis()`

```
bool QCPAxisRect::removeAxis (
    QCPAxis * axis )
```

Removes the specified *axis* from the axis rect and deletes it.

Returns true on success, i.e. if *axis* was a valid axis in this axis rect.

See also

[addAxis](#)

10.402.3.31 `right()`

```
int QCPAxisRect::right ( ) const [inline]
```

Returns the pixel position of the right border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.402.3.32 `setBackground()` [1/3]

```
void QCPAxisRect::setBackground (
    const QBrush & brush )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets *brush* as the background brush. The axis rect background will be filled with this brush. Since axis rects place themselves on the "background" layer by default, the axis rect backgrounds are usually drawn below everything else.

The brush will be drawn before (under) any background pixmap, which may be specified with [setBackground\(const QPixmap &pm\)](#).

To disable drawing of a background brush, set *brush* to `Qt::NoBrush`.

See also

[setBackground\(const QPixmap &pm\)](#)

10.402.3.33 `setBackground()` [2/3]

```
void QCPAxisRect::setBackground (
    const QPixmap & pm )
```

Sets *pm* as the axis background pixmap. The axis background pixmap will be drawn inside the axis rect. Since axis rects place themselves on the "background" layer by default, the axis rect backgrounds are usually drawn below everything else.

For cases where the provided pixmap doesn't have the same size as the axis rect, scaling can be enabled with [setBackgroundScaled](#) and the scaling mode (i.e. whether and how the aspect ratio is preserved) can be set with [setBackgroundScaledMode](#). To set all these options in one call, consider using the overloaded version of this function.

Below the pixmap, the axis rect may be optionally filled with a brush, if specified with [setBackground\(const QBrush &brush\)](#).

See also

[setBackgroundScaled](#), [setBackgroundScaledMode](#), [setBackground\(const QBrush &brush\)](#)

10.402.3.34 setBackground() [3/3]

```
void QCPAxisRect::setBackground (
    const QPixmap & pm,
    bool scaled,
    Qt::AspectRatioMode mode = Qt::KeepAspectRatioByExpanding )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Allows setting the background pixmap of the axis rect, whether it shall be scaled and how it shall be scaled in one call.

See also

[setBackground\(const QPixmap &pm\)](#), [setBackgroundScaled](#), [setBackgroundScaledMode](#)

10.402.3.35 setBackgroundScaled()

```
void QCPAxisRect::setBackgroundScaled (
    bool scaled )
```

Sets whether the axis background pixmap shall be scaled to fit the axis rect or not. If *scaled* is set to true, you may control whether and how the aspect ratio of the original pixmap is preserved with [setBackgroundScaledMode](#).

Note that the scaled version of the original pixmap is buffered, so there is no performance penalty on replots. (Except when the axis rect dimensions are changed continuously.)

See also

[setBackground](#), [setBackgroundScaledMode](#)

10.402.3.36 setBackgroundScaledMode()

```
void QCPAxisRect::setBackgroundScaledMode (
    Qt::AspectRatioMode mode )
```

If scaling of the axis background pixmap is enabled ([setBackgroundScaled](#)), use this function to define whether and how the aspect ratio of the original pixmap passed to [setBackground](#) is preserved.

See also

[setBackground](#), [setBackgroundScaled](#)

10.402.3.37 setRangeDrag()

```
void QCPAxisRect::setRangeDrag (
    Qt::Orientations orientations )
```

Sets which axis orientation may be range dragged by the user with mouse interaction. What orientation corresponds to which specific axis can be set with [setRangeDragAxes\(QCPAxis *horizontal, QCPAxis *vertical\)](#). By default, the horizontal axis is the bottom axis (xAxis) and the vertical axis is the left axis (yAxis).

To disable range dragging entirely, pass `nullptr` as *orientations* or remove [QCP::iRangeDrag](#) from [QCustomPlot::setInteractions](#). To enable range dragging for both directions, pass `Qt::Horizontal | Qt::Vertical` as *orientations*.

In addition to setting *orientations* to a non-zero value, make sure [QCustomPlot::setInteractions](#) contains [QCP::iRangeDrag](#) to enable the range dragging interaction.

See also

[setRangeZoom](#), [setRangeDragAxes](#), [QCustomPlot::setNoAntialiasingOnDrag](#)

10.402.3.38 setRangeDragAxes() [1/3]

```
void QCPAxisRect::setRangeDragAxes (
    QCPAxis * horizontal,
    QCPAxis * vertical )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the axes whose range will be dragged when [setRangeDrag](#) enables mouse range dragging on the [QCustomPlot](#) widget. Pass `nullptr` if no axis shall be dragged in the respective orientation.

Use the overload taking a list of axes, if multiple axes (more than one per orientation) shall react to dragging interactions.

See also

[setRangeZoomAxes](#)

10.402.3.39 setRangeDragAxes() [2/3]

```
void QCPAxisRect::setRangeDragAxes (
    QList< QCPAxis * > axes )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

This method allows to set up multiple axes to react to horizontal and vertical dragging. The drag orientation that the respective axis will react to is deduced from its orientation ([QCPAxis::orientation](#)).

In the unusual case that you wish to e.g. drag a vertically oriented axis with a horizontal drag motion, use the overload taking two separate lists for horizontal and vertical dragging.

10.402.3.40 setRangeDragAxes() [3/3]

```
void QCPAxisRect::setRangeDragAxes (
    QList< QCPAxis * > horizontal,
    QList< QCPAxis * > vertical )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

This method allows to set multiple axes up to react to horizontal and vertical dragging, and define specifically which axis reacts to which drag orientation (irrespective of the axis orientation).

10.402.3.41 setRangeZoom()

```
void QCPAxisRect::setRangeZoom (
    Qt::Orientations orientations )
```

Sets which axis orientation may be zoomed by the user with the mouse wheel. What orientation corresponds to which specific axis can be set with [setRangeZoomAxes\(QCPAxis *horizontal, QCPAxis *vertical\)](#). By default, the horizontal axis is the bottom axis (xAxis) and the vertical axis is the left axis (yAxis).

To disable range zooming entirely, pass `nullptr` as *orientations* or remove [QCP::iRangeZoom](#) from [QCustomPlot::setInteractions](#). To enable range zooming for both directions, pass `Qt::Horizontal | Qt::Vertical` as *orientations*.

In addition to setting *orientations* to a non-zero value, make sure [QCustomPlot::setInteractions](#) contains [QCP::iRangeZoom](#) to enable the range zooming interaction.

See also

[setRangeZoomFactor](#), [setRangeZoomAxes](#), [setRangeDrag](#)

10.402.3.42 setRangeZoomAxes() [1/3]

```
void QCPAxisRect::setRangeZoomAxes (
    QCPAxis * horizontal,
    QCPAxis * vertical )
```

Sets the axes whose range will be zoomed when [setRangeZoom](#) enables mouse wheel zooming on the [QCustomPlot](#) widget. Pass `nullptr` if no axis shall be zoomed in the respective orientation.

The two axes can be zoomed with different strengths, when different factors are passed to [setRangeZoomFactor\(double horizontalFactor, double verticalFactor\)](#).

Use the overload taking a list of axes, if multiple axes (more than one per orientation) shall react to zooming interactions.

See also

[setRangeDragAxes](#)

10.402.3.43 `setRangeZoomAxes()` [2/3]

```
void QCPAxisRect::setRangeZoomAxes (
    QList< QCPAxis * > axes )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

This method allows to set up multiple axes to react to horizontal and vertical range zooming. The zoom orientation that the respective axis will react to is deduced from its orientation ([QCPAxis::orientation](#)).

In the unusual case that you wish to e.g. zoom a vertically oriented axis with a horizontal zoom interaction, use the overload taking two separate lists for horizontal and vertical zooming.

10.402.3.44 `setRangeZoomAxes()` [3/3]

```
void QCPAxisRect::setRangeZoomAxes (
    QList< QCPAxis * > horizontal,
    QList< QCPAxis * > vertical )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

This method allows to set multiple axes up to react to horizontal and vertical zooming, and define specifically which axis reacts to which zoom orientation (irrespective of the axis orientation).

10.402.3.45 `setRangeZoomFactor()` [1/2]

```
void QCPAxisRect::setRangeZoomFactor (
    double factor )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets both the horizontal and vertical zoom *factor*.

10.402.3.46 `setRangeZoomFactor()` [2/2]

```
void QCPAxisRect::setRangeZoomFactor (
    double horizontalFactor,
    double verticalFactor )
```

Sets how strong one rotation step of the mouse wheel zooms, when range zoom was activated with [setRangeZoom](#). The two parameters *horizontalFactor* and *verticalFactor* provide a way to let the horizontal axis zoom at different rates than the vertical axis. Which axis is horizontal and which is vertical, can be set with [setRangeZoomAxes](#).

When the zoom factor is greater than one, scrolling the mouse wheel backwards (towards the user) will zoom in (make the currently visible range smaller). For zoom factors smaller than one, the same scrolling direction will zoom out.

10.402.3.47 setupFullAxesBox()

```
void QCPAxisRect::setupFullAxesBox (
    bool connectRanges = false )
```

Convenience function to create an axis on each side that doesn't have any axes yet and set their visibility to true. Further, the top/right axes are assigned the following properties of the bottom/left axes:

- range ([QCPAxis::setRange](#))
- range reversed ([QCPAxis::setRangeReversed](#))
- scale type ([QCPAxis::setScaleType](#))
- tick visibility ([QCPAxis::setTicks](#))
- number format ([QCPAxis::setNumberFormat](#))
- number precision ([QCPAxis::setNumberPrecision](#))
- tick count of ticker ([QCPAxisTicker::setTickCount](#))
- tick origin of ticker ([QCPAxisTicker::setTickOrigin](#))

Tick label visibility ([QCPAxis::setTickLabels](#)) of the right and top axes are set to false.

If *connectRanges* is true, the [rangeChanged](#) signals of the bottom and left axes are connected to the [QCPAxis::setRange](#) slots of the top and right axes.

10.402.3.48 size()

```
QSize QCPAxisRect::size ( ) const [inline]
```

Returns the pixel size of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.402.3.49 top()

```
int QCPAxisRect::top ( ) const [inline]
```

Returns the pixel position of the top border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.402.3.50 topLeft()

```
QPoint QCPAxisRect::topLeft ( ) const [inline]
```

Returns the top left corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.402.3.51 topRight()

```
QPoint QCPAxisRect::topRight ( ) const [inline]
```

Returns the top right corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.402.3.52 update()

```
void QCPAxisRect::update (
    UpdatePhase phase ) [virtual]
```

This method is called automatically upon replot and doesn't need to be called by users of [QCPAxisRect](#).

Calls the base class implementation to update the margins (see [QCPLayoutElement::update](#)), and finally passes the [rect](#) to the inset layout ([insetLayout](#)) and calls its [QCPInsetLayout::update](#) function.

\see [baseclassmethod](#)

Reimplemented from [QCPLayoutElement](#).

Reimplemented in [QCPColorScaleAxisRectPrivate](#).

10.402.3.53 wheelEvent()

```
void QCPAxisRect::wheelEvent (
    QWheelEvent * event ) [protected], [virtual]
```

This event gets called when the user turns the mouse scroll wheel while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`.

The `event->angleDelta()` indicates how far the mouse wheel was turned, which is usually +/- 120 for single rotation steps. However, if the mouse wheel is turned rapidly, multiple steps may accumulate to one event, making the delta larger. On the other hand, if the wheel has very smooth steps or none at all, the delta may be smaller.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#)

Reimplemented from [QCPLayerable](#).

Reimplemented in [QCPColorScaleAxisRectPrivate](#).

10.402.3.54 width()

```
int QCPAxisRect::width ( ) const [inline]
```

Returns the pixel width of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.402.3.55 zoom() [1/2]

```
void QCPAxisRect::zoom (
    const QRectF & pixelRect )
```

Zooms in (or out) to the passed rectangular region *pixelRect*, given in pixel coordinates.

All axes of this axis rect will have their range zoomed accordingly. If you only wish to zoom specific axes, use the overloaded version of this method.

See also

[QCustomPlot::setSelectionRectMode](#)

10.402.3.56 zoom() [2/2]

```
void QCPAxisRect::zoom (
    const QRectF & pixelRect,
    const QList< QCPAxis * > & affectedAxes )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Zooms in (or out) to the passed rectangular region *pixelRect*, given in pixel coordinates.

Only the axes passed in *affectedAxes* will have their ranges zoomed accordingly.

See also

[QCustomPlot::setSelectionRectMode](#)

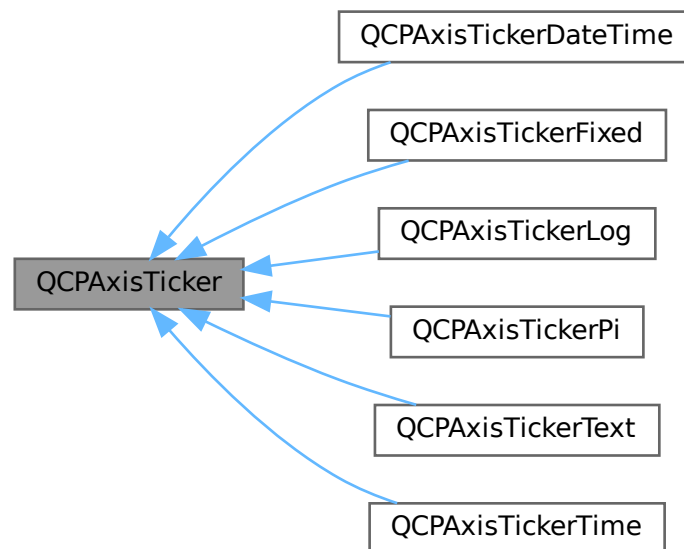
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.403 QCPAxisTicker Class Reference

The base class tick generator used by [QCPAxis](#) to create tick positions and tick labels.

Inheritance diagram for QCPAxisTicker:



Public Types

- enum [TickStepStrategy](#) { [tssReadability](#) , [tssMeetTickCount](#) }

Public Member Functions

- [QCPAxisTicker](#) ()
- [TickStepStrategy](#) **tickStepStrategy** () const
- int **tickCount** () const
- double **tickOrigin** () const
- void [setTickStepStrategy](#) ([TickStepStrategy](#) strategy)
- void [setTickCount](#) (int count)
- void [setTickOrigin](#) (double origin)
- virtual void [generate](#) (const [QCPRange](#) &range, const [QLocale](#) &locale, [QChar](#) formatChar, int precision, [QVector](#)< double > &ticks, [QVector](#)< double > *subTicks, [QVector](#)< [QString](#) > *tickLabels)

Protected Member Functions

- virtual double [getTickStep](#) (const [QCPRange](#) &range)
- virtual int [getSubTickCount](#) (double tickStep)
- virtual [QString](#) [getTickLabel](#) (double tick, const [QLocale](#) &locale, [QChar](#) formatChar, int precision)
- virtual [QVector](#)< double > [createTickVector](#) (double tickStep, const [QCPRange](#) &range)
- virtual [QVector](#)< double > **createSubTickVector** (int subTickCount, const [QVector](#)< double > &ticks)
- virtual [QVector](#)< [QString](#) > **createLabelVector** (const [QVector](#)< double > &ticks, const [QLocale](#) &locale, [QChar](#) formatChar, int precision)
- void **trimTicks** (const [QCPRange](#) &range, [QVector](#)< double > &ticks, bool keepOneOutlier) const
- double **pickClosest** (double target, const [QVector](#)< double > &candidates) const
- double **getMantissa** (double input, double *magnitude=nullptr) const
- double **cleanMantissa** (double input) const

Protected Attributes

- [TickStepStrategy](#) **mTickStepStrategy**
- int **mTickCount**
- double **mTickOrigin**

10.403.1 Detailed Description

The base class tick generator used by [QCPAxis](#) to create tick positions and tick labels.

Each [QCPAxis](#) has an internal [QCPAxisTicker](#) (or a subclass) in order to generate tick positions and tick labels for the current axis range. The ticker of an axis can be set via [QCPAxis::setTicker](#). Since that method takes a `QSharedPointer<QCPAxisTicker>`, multiple axes can share the same ticker instance.

This base class generates normal tick coordinates and numeric labels for linear axes. It picks a reasonable tick step (the separation between ticks) which results in readable tick labels. The number of ticks that should be approximately generated can be set via [setTickCount](#). Depending on the current tick step strategy ([setTickStepStrategy](#)), the algorithm either sacrifices readability to better match the specified tick count ([QCPAxisTicker::tssMeetTickCount](#)) or relaxes the tick count in favor of better tick steps ([QCPAxisTicker::tssReadability](#)), which is the default.

The following more specialized axis ticker subclasses are available, see details in the respective class documentation:

QCPAxisTickerFixed	td
QCPAxisTickerLog	td
QCPAxisTickerPi	td
QCPAxisTickerText	td
QCPAxisTickerDateTime	td
QCPAxisTickerTime	td

10.403.2 Creating own axis tickers

Creating own axis tickers can be achieved very easily by subclassing [QCPAxisTicker](#) and reimplementing some or all of the available virtual methods.

In the simplest case you might wish to just generate different tick steps than the other tickers, so you only reimplement the method `getTickStep`. If you additionally want control over the string that will be shown as tick label, reimplement `getTickLabel`.

If you wish to have complete control, you can generate the tick vectors and tick label vectors yourself by reimplementing `createTickVector` and `createLabelVector`. The default implementations use the previously mentioned virtual methods `getTickStep` and `getTickLabel`, but your reimplementations don't necessarily need to do so. For example in the case of unequal tick steps, the method `getTickStep` loses its usefulness and can be ignored.

The sub tick count between major ticks can be controlled with `getSubTickCount`. Full sub tick placement control is obtained by reimplementing `createSubTickVector`.

See the documentation of all these virtual methods in [QCPAxisTicker](#) for detailed information about the parameters and expected return values.

10.403.3 Member Enumeration Documentation

10.403.3.1 TickStepStrategy

enum [QCPAxisTicker::TickStepStrategy](#)

Defines the strategies that the axis ticker may follow when choosing the size of the tick step.

See also

[setTickStepStrategy](#)

Enumerator

tssReadability	A nicely readable tick step is prioritized over matching the requested number of ticks (see setTickCount)
tssMeetTickCount	Less readable tick steps are allowed which in turn facilitates getting closer to the requested tick count.

10.403.4 Constructor & Destructor Documentation

10.403.4.1 QCPAxisTicker()

```
QCPAxisTicker::QCPAxisTicker ( )
```

Constructs the ticker and sets reasonable default values. Axis tickers are commonly created managed by a QSharedPointer, which then can be passed to [QCPAxis::setTicker](#).

10.403.5 Member Function Documentation

10.403.5.1 createTickVector()

```
QVector< double > QCPAxisTicker::createTickVector (
    double tickStep,
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented in [QCPAxisTickerText](#).

10.403.5.2 generate()

```
void QCPAxisTicker::generate (
    const QCPRange & range,
    const QLocale & locale,
    QChar formatChar,
    int precision,
    QVector< double > & ticks,
    QVector< double > * subTicks,
    QVector< QString > * tickLabels ) [virtual]
```


This is the method called by [QCPAxis](#) in order to actually generate tick coordinates (*ticks*), tick label strings (*tickLabels*) and sub tick coordinates (*subTicks*).

The ticks are generated for the specified *range*. The generated labels typically follow the specified *locale*, *formatChar* and number *precision*, however this might be different (or even irrelevant) for certain [QCPAxisTicker](#) subclasses.

The output parameter *ticks* is filled with the generated tick positions in axis coordinates. The output parameters *subTicks* and *tickLabels* are optional (set them to `nullptr` if not needed) and are respectively filled with sub tick coordinates, and tick label strings belonging to *ticks* by index.

10.403.5.3 getSubTickCount()

```
int QCPAxisTicker::getSubTickCount (
    double tickStep ) [protected], [virtual]
```

Reimplemented in [QCPAxisTickerText](#).

10.403.5.4 getTickLabel()

```
QString QCPAxisTicker::getTickLabel (
    double tick,
    const QLocale & locale,
    QChar formatChar,
    int precision ) [protected], [virtual]
```

Reimplemented in [QCPAxisTickerText](#).

10.403.5.5 getTickStep()

```
double QCPAxisTicker::getTickStep (
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented in [QCPAxisTickerText](#).

10.403.5.6 setTickCount()

```
void QCPAxisTicker::setTickCount (
    int count )
```

Sets how many ticks this ticker shall aim to generate across the axis range. Note that *count* is not guaranteed to be matched exactly, as generating readable tick intervals may conflict with the requested number of ticks.

Whether the readability has priority over meeting the requested *count* can be specified with [setTickStepStrategy](#).

10.403.5.7 setTickOrigin()

```
void QCPAxisTicker::setTickOrigin (
    double origin )
```

Sets the mathematical coordinate (or "offset") of the zeroth tick. This tick coordinate is just a concept and doesn't need to be inside the currently visible axis range.

By default *origin* is zero, which for example yields ticks {-5, 0, 5, 10, 15,...} when the tick step is five. If *origin* is now set to 1 instead, the correspondingly generated ticks would be {-4, 1, 6, 11, 16,...}.

10.403.5.8 setTickStepStrategy()

```
void QCPAxisTicker::setTickStepStrategy (
    QCPAxisTicker::TickStepStrategy strategy )
```

Sets which strategy the axis ticker follows when choosing the size of the tick step. For the available strategies, see [TickStepStrategy](#).

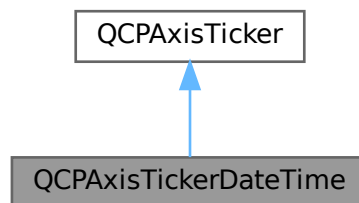
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

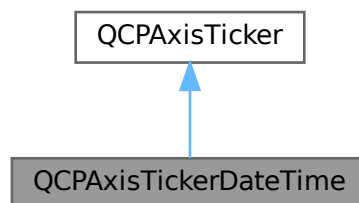
10.404 QCPAxisTickerDateTime Class Reference

Specialized axis ticker for calendar dates and times as axis ticks.

Inheritance diagram for QCPAxisTickerDateTime:



Collaboration diagram for QCPAxisTickerDateTime:



Public Member Functions

- [QCPAxisTickerDateTime](#) ()
- QString **dateTimeFormat** () const
- Qt::TimeSpec **dateTimeSpec** () const
- QTimeZone **timeZone** () const
- void [setDateTimeFormat](#) (const QString &format)
- void [setDateTimeSpec](#) (Qt::TimeSpec spec)
- void [setTimeZone](#) (const QTimeZone &zone)
- void [setTickOrigin](#) (double origin)
- void [setTickOrigin](#) (const QDateTime &origin)

Public Member Functions inherited from [QCPAxisTicker](#)

- [QCPAxisTicker](#) ()
- [TickStepStrategy](#) **tickStepStrategy** () const
- int **tickCount** () const
- double **tickOrigin** () const
- void [setTickStepStrategy](#) ([TickStepStrategy](#) strategy)
- void [setTickCount](#) (int count)
- void [setTickOrigin](#) (double origin)
- virtual void [generate](#) (const [QCPRange](#) &range, const QLocale &locale, QChar formatChar, int precision, QVector< double > &ticks, QVector< double > *subTicks, QVector< QString > *tickLabels)

Static Public Member Functions

- static QDateTime [keyToDateTime](#) (double key)
- static double [dateTimeToKey](#) (const QDateTime &dateTime)
- static double [dateTimeToKey](#) (const QDate &date, Qt::TimeSpec timeSpec=Qt::LocalTime)

Protected Types

- enum **DateStrategy** { **dsNone** , **dsUniformTimeInDay** , **dsUniformDayInMonth** }

Protected Member Functions

- virtual double [getTickStep](#) (const [QCPRange](#) &range) Q_DECL_OVERRIDE
- virtual int [getSubTickCount](#) (double tickStep) Q_DECL_OVERRIDE
- virtual QString [getTickLabel](#) (double tick, const QLocale &locale, QChar formatChar, int precision) Q_DECL_OVERRIDE
- virtual QVector< double > [createTickVector](#) (double tickStep, const [QCPRange](#) &range) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPAxisTicker](#)

- virtual QVector< double > **createSubTickVector** (int subTickCount, const QVector< double > &ticks)
- virtual QVector< QString > **createLabelVector** (const QVector< double > &ticks, const QLocale &locale, QChar formatChar, int precision)
- void **trimTicks** (const [QCPRange](#) &range, QVector< double > &ticks, bool keepOneOutlier) const
- double **pickClosest** (double target, const QVector< double > &candidates) const
- double **getMantissa** (double input, double *magnitude=nullptr) const
- double **cleanMantissa** (double input) const

Protected Attributes

- QString **mDateTimeFormat**
- Qt::TimeSpec **mDateTimeSpec**
- QTimeZone **mTimeZone**
- enum QCPAxisTickerDateTime::DateStrategy **mDateStrategy**

Protected Attributes inherited from [QCPAxisTicker](#)

- [TickStepStrategy](#) **mTickStepStrategy**
- int **mTickCount**
- double **mTickOrigin**

Additional Inherited Members

Public Types inherited from [QCPAxisTicker](#)

- enum [TickStepStrategy](#) { [tssReadability](#) , [tssMeetTickCount](#) }

10.404.1 Detailed Description

Specialized axis ticker for calendar dates and times as axis ticks.

This [QCPAxisTicker](#) subclass generates ticks that correspond to real calendar dates and times. The plot axis coordinate is interpreted as Unix Time, so seconds since Epoch (January 1, 1970, 00:00 UTC). This is also used for example by QDateTime in the `toTime_t()` / `setTime_t()` methods with a precision of one second. Since Qt 4.7, millisecond accuracy can be obtained from QDateTime by using `QDateTime::fromMsecsSinceEpoch()` / `1000.0`. The static methods [dateTimeToKey](#) and [keyToDateTime](#) conveniently perform this conversion achieving a precision of one millisecond on all Qt versions.

The format of the date/time display in the tick labels is controlled with [setDateTimeFormat](#). If a different time spec or time zone shall be used for the tick label appearance, see [setDateTimeSpec](#) or [setTimeZone](#), respectively.

This ticker produces unequal tick spacing in order to provide intuitive date and time-of-day ticks. For example, if the axis range spans a few years such that there is one tick per year, ticks will be positioned on 1. January of every year. This is intuitive but, due to leap years, will result in slightly unequal tick intervals (visually unnoticeable). The same can be seen in the image above: even though the number of days varies month by month, this ticker generates ticks on the same day of each month.

If you would like to change the date/time that is used as a (mathematical) starting date for the ticks, use the [setTickOrigin\(const QDateTime &origin\)](#) method overload, which takes a QDateTime. If you pass 15. July, 9:45 to this method, the yearly ticks will end up on 15. July at 9:45 of every year.

The ticker can be created and assigned to an axis like this:

Note

If you rather wish to display relative times in terms of days, hours, minutes, seconds and milliseconds, and are not interested in the intricacies of real calendar dates with months and (leap) years, have a look at [QCPAxisTickerTime](#) instead.

10.404.2 Constructor & Destructor Documentation

10.404.2.1 QCPAxisTickerDateTime()

```
QCPAxisTickerDateTime::QCPAxisTickerDateTime ( )
```

Constructs the ticker and sets reasonable default values. Axis tickers are commonly created managed by a [QSharedPointer](#), which then can be passed to [QCPAxis::setTicker](#).

10.404.3 Member Function Documentation

10.404.3.1 createTickVector()

```
 QVector< double > QCPAxisTickerDateTime::createTickVector (
    double tickStep,
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

10.404.3.2 dateTimeToKey() [1/2]

```
double QCPAxisTickerDateTime::dateTimeToKey (
    const QDate & date,
    Qt::TimeSpec timeSpec = Qt::LocalTime ) [static]
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

A convenience method which turns a [QDate](#) object into a double value that corresponds to seconds since Epoch (1. Jan 1970, 00:00 UTC). This is the format used as axis coordinates by [QCPAxisTickerDateTime](#).

The returned value will be the start of the passed day of *date*, interpreted in the given *timeSpec*.

See also

[keyToDateTime](#)

10.404.3.3 dateTimeToKey() [2/2]

```
double QCPAxisTickerDateTime::dateTimeToKey (
    const QDateTime & dateTime ) [static]
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

A convenience method which turns a [QDateTime](#) object into a double value that corresponds to seconds since Epoch (1. Jan 1970, 00:00 UTC). This is the format used as axis coordinates by [QCPAxisTickerDateTime](#).

The accuracy achieved by this method is one millisecond, irrespective of the used Qt version (it works around the lack of a [QDateTime::toMsecsSinceEpoch](#) in Qt 4.6)

See also

[keyToDateTime](#)

10.404.3.4 getSubTickCount()

```
int QCPAxisTickerDateTime::getSubTickCount (
    double tickStep ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

10.404.3.5 getTickLabel()

```
QString QCPAxisTickerDateTime::getTickLabel (
    double tick,
    const QLocale & locale,
    QChar formatChar,
    int precision ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

10.404.3.6 getTickStep()

```
double QCPAxisTickerDateTime::getTickStep (
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

10.404.3.7 keyToDateTime()

```
QDateTime QCPAxisTickerDateTime::keyToDateTime (
    double key ) [static]
```

A convenience method which turns *key* (in seconds since Epoch 1. Jan 1970, 00:00 UTC) into a QDateTime object. This can be used to turn axis coordinates to actual QDateTimes.

The accuracy achieved by this method is one millisecond, irrespective of the used Qt version (it works around the lack of a QDateTime::fromMSecsSinceEpoch in Qt 4.6)

See also

[dateTimeToKey](#)

10.404.3.8 setDateTimeFormat()

```
void QCPAxisTickerDateTime::setDateTimeFormat (
    const QString & format )
```

Sets the format in which dates and times are displayed as tick labels. For details about the *format* string, see the documentation of QDateTime::toString().

Typical expressions are

d	The day as a number without a leading zero (1 to 31)
dd	The day as a number with a leading zero (01 to 31)
ddd	The abbreviated localized day name (e.g. 'Mon' to 'Sun'). Uses the system locale to localize the name, i.e. <code>QLocale::system()</code> .
dddd	The long localized day name (e.g. 'Monday' to 'Sunday'). Uses the system locale to localize the name, i.e. <code>QLocale::system()</code> .
M	The month as a number without a leading zero (1 to 12)
MM	The month as a number with a leading zero (01 to 12)
MMM	The abbreviated localized month name (e.g. 'Jan' to 'Dec'). Uses the system locale to localize the name, i.e. <code>QLocale::system()</code> .
MMMM	The long localized month name (e.g. 'January' to 'December'). Uses the system locale to localize the name, i.e. <code>QLocale::system()</code> .
YY	The year as a two digit number (00 to 99)
YYYY	The year as a four digit number. If the year is negative, a minus sign is prepended, making five characters.
h	The hour without a leading zero (0 to 23 or 1 to 12 if AM/PM display)
hh	The hour with a leading zero (00 to 23 or 01 to 12 if AM/PM display)
H	The hour without a leading zero (0 to 23, even with AM/PM display)
HH	The hour with a leading zero (00 to 23, even with AM/PM display)
m	The minute without a leading zero (0 to 59)
mm	The minute with a leading zero (00 to 59)
s	The whole second, without any leading zero (0 to 59)
ss	The whole second, with a leading zero where applicable (00 to 59)
z	The fractional part of the second, to go after a decimal point, without trailing zeroes (0 to 999). Thus "s.z" reports the seconds to full available (millisecond) precision without trailing zeroes.
zzz	The fractional part of the second, to millisecond precision, including trailing zeroes where applicable (000 to 999).
AP or A	Use AM/PM display. A/AP will be replaced by an upper-case version of either <code>QLocale::amText()</code> or <code>QLocale::pmText()</code> .
ap or a	Use am/pm display. a/ap will be replaced by a lower-case version of either <code>QLocale::amText()</code> or <code>QLocale::pmText()</code> .
t	The timezone (for example "CEST")

Newlines can be inserted with `"\n"`, literal strings (even when containing above expressions) by encapsulating them using single-quotes. A literal single quote can be generated by using two consecutive single quotes in the format.

See also

[setDateTimeSpec](#), [setTimeZone](#)

10.404.3.9 setDateTimeSpec()

```
void QCPAxisTickerDateTime::setDateTimeSpec (
    Qt::TimeSpec spec )
```

Sets the time spec that is used for creating the tick labels from corresponding dates/times.

The default value of QDateTime objects (and also [QCPAxisTickerDateTime](#)) is `Qt::LocalTime`. However, if the displayed tick labels shall be given in UTC, set *spec* to `Qt::UTC`.

Tick labels corresponding to other time zones can be achieved with [setTimeZone](#) (which sets *spec* to `Qt::TimeZone` internally). Note that if *spec* is afterwards set to not be `Qt::TimeZone` again, the [setTimeZone](#) setting will be ignored accordingly.

See also

[setDateTimeFormat](#), [setTimeZone](#)

10.404.3.10 `setTickOrigin()` [1/2]

```
void QCPAxisTickerDateTime::setTickOrigin (
    const QDateTime & origin )
```

Sets the tick origin (see [QCPAxisTicker::setTickOrigin](#)) as a QDateTime *origin*.

This is useful to define the month/day/time recurring at greater tick interval steps. For example, If you pass 15. July, 9:45 to this method and the tick interval happens to be one tick per year, the ticks will end up on 15. July at 9:45 of every year.

10.404.3.11 `setTickOrigin()` [2/2]

```
void QCPAxisTickerDateTime::setTickOrigin (
    double origin )
```

Sets the tick origin (see [QCPAxisTicker::setTickOrigin](#)) in seconds since Epoch (1. Jan 1970, 00:00 UTC). For the date time ticker it might be more intuitive to use the overload which directly takes a QDateTime, see [setTickOrigin\(const QDateTime &origin\)](#).

This is useful to define the month/day/time recurring at greater tick interval steps. For example, If you pass 15. July, 9:45 to this method and the tick interval happens to be one tick per year, the ticks will end up on 15. July at 9:45 of every year.

10.404.3.12 `setTimeZone()`

```
void QCPAxisTickerDateTime::setTimeZone (
    const QTimeZone & zone )
```

Sets the time zone that is used for creating the tick labels from corresponding dates/times. The time spec ([setDateTimeSpec](#)) is set to `Qt::TimeZone`.

See also

[setDateTimeFormat](#), [setTimeZone](#)

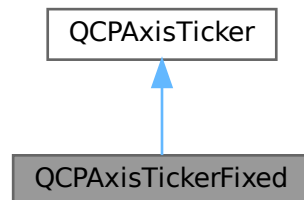
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

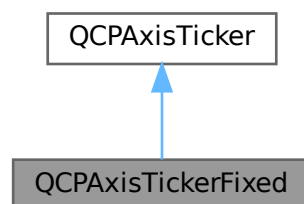
10.405 QCPAxisTickerFixed Class Reference

Specialized axis ticker with a fixed tick step.

Inheritance diagram for QCPAxisTickerFixed:



Collaboration diagram for QCPAxisTickerFixed:



Public Types

- enum [ScaleStrategy](#) { [ssNone](#) , [ssMultiples](#) , [ssPowers](#) }

Public Types inherited from [QCPAxisTicker](#)

- enum [TickStepStrategy](#) { [tssReadability](#) , [tssMeetTickCount](#) }

Public Member Functions

- [QCPAxisTickerFixed](#) ()
- double **tickStep** () const
- [ScaleStrategy](#) **scaleStrategy** () const
- void [setTickStep](#) (double step)
- void [setScaleStrategy](#) ([ScaleStrategy](#) strategy)

Public Member Functions inherited from [QCPAxisTicker](#)

- [QCPAxisTicker](#) ()
- [TickStepStrategy](#) **tickStepStrategy** () const
- int **tickCount** () const
- double **tickOrigin** () const
- void **setTickStepStrategy** ([TickStepStrategy](#) strategy)
- void **setTickCount** (int count)
- void **setTickOrigin** (double origin)
- virtual void **generate** (const [QCPRange](#) &range, const QLocale &locale, QChar formatChar, int precision, QVector< double > &ticks, QVector< double > *subTicks, QVector< QString > *tickLabels)

Protected Member Functions

- virtual double **getTickStep** (const [QCPRange](#) &range) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPAxisTicker](#)

- virtual int **getSubTickCount** (double tickStep)
- virtual QString **getTickLabel** (double tick, const QLocale &locale, QChar formatChar, int precision)
- virtual QVector< double > **createTickVector** (double tickStep, const [QCPRange](#) &range)
- virtual QVector< double > **createSubTickVector** (int subTickCount, const QVector< double > &ticks)
- virtual QVector< QString > **createLabelVector** (const QVector< double > &ticks, const QLocale &locale, QChar formatChar, int precision)
- void **trimTicks** (const [QCPRange](#) &range, QVector< double > &ticks, bool keepOneOutlier) const
- double **pickClosest** (double target, const QVector< double > &candidates) const
- double **getMantissa** (double input, double *magnitude=NULLPTR) const
- double **cleanMantissa** (double input) const

Protected Attributes

- double **mTickStep**
- [ScaleStrategy](#) **mScaleStrategy**

Protected Attributes inherited from [QCPAxisTicker](#)

- [TickStepStrategy](#) **mTickStepStrategy**
- int **mTickCount**
- double **mTickOrigin**

10.405.1 Detailed Description

Specialized axis ticker with a fixed tick step.

This [QCPAxisTicker](#) subclass generates ticks with a fixed tick step set with [setTickStep](#). It is also possible to allow integer multiples and integer powers of the specified tick step with [setScaleStrategy](#).

A typical application of this ticker is to make an axis only display integers, by setting the tick step of the ticker to 1.0 and the scale strategy to [ssMultiples](#).

Another case is when a certain number has a special meaning and axis ticks should only appear at multiples of that value. In this case you might also want to consider [QCPAxisTickerPi](#) because despite the name it is not limited to only pi symbols/values.

The ticker can be created and assigned to an axis like this:

10.405.2 Member Enumeration Documentation

10.405.2.1 ScaleStrategy

enum [QCPAxisTickerFixed::ScaleStrategy](#)

Defines how the axis ticker may modify the specified tick step ([setTickStep](#)) in order to control the number of ticks in the axis range.

See also

[setScaleStrategy](#)

Enumerator

ssNone	Modifications are not allowed, the specified tick step is absolutely fixed. This might cause a high tick density and overlapping labels if the axis range is zoomed out.
ssMultiples	An integer multiple of the specified tick step is allowed. The used factor follows the base class properties of setTickStepStrategy and setTickCount .
ssPowers	An integer power of the specified tick step is allowed.

10.405.3 Constructor & Destructor Documentation

10.405.3.1 QCPAxisTickerFixed()

[QCPAxisTickerFixed::QCPAxisTickerFixed](#) ()

Constructs the ticker and sets reasonable default values. Axis tickers are commonly created managed by a [QSharedPointer](#), which then can be passed to [QCPAxis::setTicker](#).

10.405.4 Member Function Documentation

10.405.4.1 getTickStep()

```
double QCPAxisTickerFixed::getTickStep (
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

10.405.4.2 setScaleStrategy()

```
void QCPAxisTickerFixed::setScaleStrategy (
    QCPAxisTickerFixed::ScaleStrategy strategy )
```

Sets whether the specified tick step ([setTickStep](#)) is absolutely fixed or whether modifications may be applied to it before calculating the finally used tick step, such as permitting multiples or powers. See [ScaleStrategy](#) for details.

The default strategy is [ssNone](#), which means the tick step is absolutely fixed.

10.405.4.3 setTickStep()

```
void QCPAxisTickerFixed::setTickStep (
    double step )
```

Sets the fixed tick interval to *step*.

The axis ticker will only use this tick step when generating axis ticks. This might cause a very high tick density and overlapping labels if the axis range is zoomed out. Using [setScaleStrategy](#) it is possible to relax the fixed step and also allow multiples or powers of *step*. This will enable the ticker to reduce the number of ticks to a reasonable amount (see [setTickCount](#)).

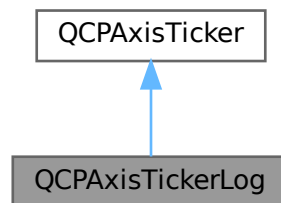
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

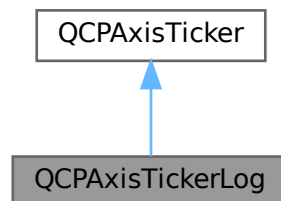
10.406 QCPAxisTickerLog Class Reference

Specialized axis ticker suited for logarithmic axes.

Inheritance diagram for QCPAxisTickerLog:



Collaboration diagram for QCPAxisTickerLog:



Public Member Functions

- [QCPAxisTickerLog](#) ()
- double **logBase** () const
- int **subTickCount** () const
- void [setLogBase](#) (double base)
- void [setSubTickCount](#) (int subTicks)

Public Member Functions inherited from [QCPAxisTicker](#)

- [QCPAxisTicker](#) ()
- [TickStepStrategy](#) **tickStepStrategy** () const
- int **tickCount** () const
- double **tickOrigin** () const
- void [setTickStepStrategy](#) ([TickStepStrategy](#) strategy)
- void [setTickCount](#) (int count)
- void [setTickOrigin](#) (double origin)
- virtual void [generate](#) (const [QCPRange](#) &range, const QLocale &locale, QChar formatChar, int precision, QVector< double > &ticks, QVector< double > *subTicks, QVector< QString > *tickLabels)

Protected Member Functions

- virtual int [getSubTickCount](#) (double tickStep) Q_DECL_OVERRIDE
- virtual QVector< double > [createTickVector](#) (double tickStep, const [QCPRange](#) &range) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPAxisTicker](#)

- virtual double [getTickStep](#) (const [QCPRange](#) &range)
- virtual QString [getTickLabel](#) (double tick, const QLocale &locale, QChar formatChar, int precision)
- virtual QVector< double > **createSubTickVector** (int subTickCount, const QVector< double > &ticks)
- virtual QVector< QString > **createLabelVector** (const QVector< double > &ticks, const QLocale &locale, QChar formatChar, int precision)
- void **trimTicks** (const [QCPRange](#) &range, QVector< double > &ticks, bool keepOneOutlier) const
- double **pickClosest** (double target, const QVector< double > &candidates) const
- double **getMantissa** (double input, double *magnitude=nullptr) const
- double **cleanMantissa** (double input) const

Protected Attributes

- double **mLogBase**
- int **mSubTickCount**
- double **mLogBaseLnInv**

Protected Attributes inherited from [QCPAxisTicker](#)

- [TickStepStrategy](#) **mTickStepStrategy**
- int **mTickCount**
- double **mTickOrigin**

Additional Inherited Members

Public Types inherited from [QCPAxisTicker](#)

- enum [TickStepStrategy](#) { [tssReadability](#) , [tssMeetTickCount](#) }

10.406.1 Detailed Description

Specialized axis ticker suited for logarithmic axes.

This [QCPAxisTicker](#) subclass generates ticks with unequal tick intervals suited for logarithmic axis scales. The ticks are placed at powers of the specified log base ([setLogBase](#)).

Especially in the case of a log base equal to 10 (the default), it might be desirable to have tick labels in the form of powers of ten without mantissa display. To achieve this, set the number precision ([QCPAxis::setNumberPrecision](#)) to zero and the number format ([QCPAxis::setNumberFormat](#)) to scientific (exponential) display with beautifully typeset decimal powers, so a format string of "eb". This will result in the following axis tick labels:

The ticker can be created and assigned to an axis like this:

Note that the nature of logarithmic ticks imply that there exists a smallest possible tick step, corresponding to one multiplication by the log base. If the user zooms in further than that, no new ticks would appear, leading to very sparse or even no axis ticks on the axis. To prevent this situation, this ticker falls back to regular tick generation if the axis range would be covered by too few logarithmically placed ticks.

10.406.2 Constructor & Destructor Documentation

10.406.2.1 QCPAxisTickerLog()

```
QCPAxisTickerLog::QCPAxisTickerLog ( )
```

Constructs the ticker and sets reasonable default values. Axis tickers are commonly created managed by a [QSharedPointer](#), which then can be passed to [QCPAxis::setTicker](#).

10.406.3 Member Function Documentation

10.406.3.1 createTickVector()

```
QVector< double > QCPAxisTickerLog::createTickVector (
    double tickStep,
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

10.406.3.2 getSubTickCount()

```
int QCPAxisTickerLog::getSubTickCount (
    double tickStep ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

10.406.3.3 setLogBase()

```
void QCPAxisTickerLog::setLogBase (
    double base )
```

Sets the logarithm base used for tick coordinate generation. The ticks will be placed at integer powers of *base*.

10.406.3.4 setSubTickCount()

```
void QCPAxisTickerLog::setSubTickCount (
    int subTicks )
```

Sets the number of sub ticks in a tick interval. Within each interval, the sub ticks are spaced linearly to provide a better visual guide, so the sub tick density increases toward the higher tick.

Note that *subTicks* is the number of sub ticks (not sub intervals) in one tick interval. So in the case of logarithm base 10 an intuitive sub tick spacing would be achieved with eight sub ticks (the default). This means e.g. between the ticks 10 and 100 there will be eight ticks, namely at 20, 30, 40, 50, 60, 70, 80 and 90.

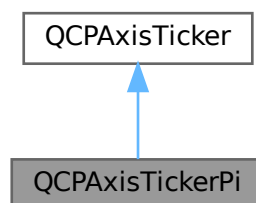
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

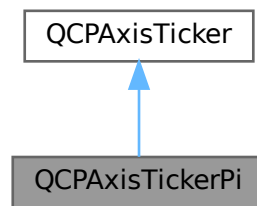
10.407 QCPAxisTickerPi Class Reference

Specialized axis ticker to display ticks in units of an arbitrary constant, for example pi.

Inheritance diagram for QCPAxisTickerPi:



Collaboration diagram for QCPAxisTickerPi:



Public Types

- enum [FractionStyle](#) { [fsFloatingPoint](#) , [fsAsciiFractions](#) , [fsUnicodeFractions](#) }

Public Types inherited from [QCPAxisTicker](#)

- enum [TickStepStrategy](#) { [tssReadability](#) , [tssMeetTickCount](#) }

Public Member Functions

- [QCPAxisTickerPi](#) ()
- QString **piSymbol** () const
- double **piValue** () const
- bool **periodicity** () const
- [FractionStyle](#) **fractionStyle** () const
- void [setPiSymbol](#) (QString symbol)
- void [setPiValue](#) (double pi)
- void [setPeriodicity](#) (int multiplesOfPi)
- void [setFractionStyle](#) ([FractionStyle](#) style)

Public Member Functions inherited from [QCPAxisTicker](#)

- [QCPAxisTicker](#) ()
- [TickStepStrategy](#) **tickStepStrategy** () const
- int **tickCount** () const
- double **tickOrigin** () const
- void [setTickStepStrategy](#) ([TickStepStrategy](#) strategy)
- void [setTickCount](#) (int count)
- void [setTickOrigin](#) (double origin)
- virtual void [generate](#) (const [QCPRange](#) &range, const QLocale &locale, QChar formatChar, int precision, QVector< double > &ticks, QVector< double > *subTicks, QVector< QString > *tickLabels)

Protected Member Functions

- virtual double [getTickStep](#) (const [QCPRange](#) &range) Q_DECL_OVERRIDE
- virtual int [getSubTickCount](#) (double tickStep) Q_DECL_OVERRIDE
- virtual QString [getTickLabel](#) (double tick, const QLocale &locale, QChar formatChar, int precision) Q_DECL_OVERRIDE
- void **simplifyFraction** (int &numerator, int &denominator) const
- QString **fractionToString** (int numerator, int denominator) const
- QString **unicodeFraction** (int numerator, int denominator) const
- QString **unicodeSuperscript** (int number) const
- QString **unicodeSubscript** (int number) const

Protected Member Functions inherited from [QCPAxisTicker](#)

- virtual QVector< double > [createTickVector](#) (double tickStep, const [QCPRange](#) &range)
- virtual QVector< double > **createSubTickVector** (int subTickCount, const QVector< double > &ticks)
- virtual QVector< QString > **createLabelVector** (const QVector< double > &ticks, const QLocale &locale, QChar formatChar, int precision)
- void **trimTicks** (const [QCPRange](#) &range, QVector< double > &ticks, bool keepOneOutlier) const
- double **pickClosest** (double target, const QVector< double > &candidates) const
- double **getMantissa** (double input, double *magnitude=nullptr) const
- double **cleanMantissa** (double input) const

Protected Attributes

- QString **mPiSymbol**
- double **mPiValue**
- int **mPeriodicity**
- [FractionStyle](#) **mFractionStyle**
- double **mPiTickStep**

Protected Attributes inherited from [QCPAxisTicker](#)

- [TickStepStrategy](#) **mTickStepStrategy**
- int **mTickCount**
- double **mTickOrigin**

10.407.1 Detailed Description

Specialized axis ticker to display ticks in units of an arbitrary constant, for example pi.

This [QCPAxisTicker](#) subclass generates ticks that are expressed with respect to a given symbolic constant with a numerical value specified with [setPiValue](#) and an appearance in the tick labels specified with [setPiSymbol](#).

Ticks may be generated at fractions of the symbolic constant. How these fractions appear in the tick label can be configured with [setFractionStyle](#).

The ticker can be created and assigned to an axis like this:

10.407.2 Member Enumeration Documentation**10.407.2.1 FractionStyle**

```
enum QCPAxisTickerPi::FractionStyle
```

Defines how fractions should be displayed in tick labels.

See also

[setFractionStyle](#)

Enumerator

fsFloatingPoint	Fractions are displayed as regular decimal floating point numbers, e.g. "0.25" or "0.125".
fsAsciiFractions	Fractions are written as rationals using ASCII characters only, e.g. "1/4" or "1/8".
fsUnicodeFractions	Fractions are written using sub- and superscript UTF-8 digits and the fraction symbol.

10.407.3 Constructor & Destructor Documentation

10.407.3.1 QCPAxisTickerPi()

```
QCPAxisTickerPi::QCPAxisTickerPi ( )
```

Constructs the ticker and sets reasonable default values. Axis tickers are commonly created managed by a [QSharedPointer](#), which then can be passed to [QCPAxis::setTicker](#).

10.407.4 Member Function Documentation

10.407.4.1 getSubTickCount()

```
int QCPAxisTickerPi::getSubTickCount (
    double tickStep ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

10.407.4.2 getTickLabel()

```
QString QCPAxisTickerPi::getTickLabel (
    double tick,
    const QLocale & locale,
    QChar formatChar,
    int precision ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

10.407.4.3 getTickStep()

```
double QCPAxisTickerPi::getTickStep (
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

10.407.4.4 setFractionStyle()

```
void QCPAxisTickerPi::setFractionStyle (
    QCPAxisTickerPi::FractionStyle style )
```

Sets how the numerical/fractional part preceding the symbolic constant is displayed in tick labels. See [FractionStyle](#) for the various options.

10.407.4.5 setPeriodicity()

```
void QCPAxisTickerPi::setPeriodicity (
    int multiplesOfPi )
```

Sets whether the axis labels shall appear periodically and if so, at which multiplicity of the symbolic constant.

To disable periodicity, set *multiplesOfPi* to zero.

For example, an axis that identifies 0 with 2pi would set *multiplesOfPi* to two.

10.407.4.6 setPiSymbol()

```
void QCPAxisTickerPi::setPiSymbol (
    QString symbol )
```

Sets how the symbol part (which is always a suffix to the number) shall appear in the axis tick label.

If a space shall appear between the number and the symbol, make sure the space is contained in *symbol*.

10.407.4.7 setPiValue()

```
void QCPAxisTickerPi::setPiValue (
    double pi )
```

Sets the numerical value that the symbolic constant has.

This will be used to place the appropriate fractions of the symbol at the respective axis coordinates.

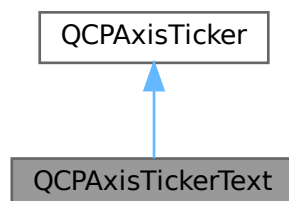
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

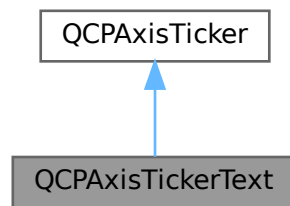
10.408 QCPAxisTickerText Class Reference

Specialized axis ticker which allows arbitrary labels at specified coordinates.

Inheritance diagram for QCPAxisTickerText:



Collaboration diagram for QCPAxisTickerText:



Public Member Functions

- [QCPAxisTickerText](#) ()
- QMap< double, QString > & [ticks](#) ()
- int [subTickCount](#) () const
- void [setTicks](#) (const QMap< double, QString > &ticks)
- void [setTicks](#) (const QVector< double > &positions, const QVector< QString > &labels)
- void [setSubTickCount](#) (int subTicks)
- void [clear](#) ()
- void [addTick](#) (double position, const QString &label)
- void [addTicks](#) (const QMap< double, QString > &ticks)
- void [addTicks](#) (const QVector< double > &positions, const QVector< QString > &labels)

Public Member Functions inherited from [QCPAxisTicker](#)

- [QCPAxisTicker](#) ()
- [TickStepStrategy](#) [tickStepStrategy](#) () const
- int [tickCount](#) () const
- double [tickOrigin](#) () const
- void [setTickStepStrategy](#) ([TickStepStrategy](#) strategy)
- void [setTickCount](#) (int count)
- void [setTickOrigin](#) (double origin)
- virtual void [generate](#) (const [QCPRange](#) &range, const QLocale &locale, QChar formatChar, int precision, QVector< double > &ticks, QVector< double > *subTicks, QVector< QString > *tickLabels)

Protected Member Functions

- virtual double [getTickStep](#) (const [QCPRange](#) &range) Q_DECL_OVERRIDE
- virtual int [getSubTickCount](#) (double tickStep) Q_DECL_OVERRIDE
- virtual QString [getTickLabel](#) (double tick, const QLocale &locale, QChar formatChar, int precision) Q_DECL_OVERRIDE
- virtual QVector< double > [createTickVector](#) (double tickStep, const [QCPRange](#) &range) Q_DECL_OVERRIDE

Protected Member Functions inherited from QCPAxisTicker

- virtual QVector< double > **createSubTickVector** (int subTickCount, const QVector< double > &ticks)
- virtual QVector< QString > **createLabelVector** (const QVector< double > &ticks, const QLocale &locale, QChar formatChar, int precision)
- void **trimTicks** (const QCPRange &range, QVector< double > &ticks, bool keepOneOutlier) const
- double **pickClosest** (double target, const QVector< double > &candidates) const
- double **getMantissa** (double input, double *magnitude=nullptr) const
- double **cleanMantissa** (double input) const

Protected Attributes

- QMap< double, QString > **mTicks**
- int **mSubTickCount**

Protected Attributes inherited from QCPAxisTicker

- TickStepStrategy **mTickStepStrategy**
- int **mTickCount**
- double **mTickOrigin**

Additional Inherited Members

Public Types inherited from QCPAxisTicker

- enum TickStepStrategy { **tssReadability** , **tssMeetTickCount** }

10.408.1 Detailed Description

Specialized axis ticker which allows arbitrary labels at specified coordinates.

This [QCPAxisTicker](#) subclass generates ticks which can be directly specified by the user as coordinates and associated strings. They can be passed as a whole with [setTicks](#) or one at a time with [addTick](#). Alternatively you can directly access the internal storage via [ticks](#) and modify the tick/label data there.

This is useful for cases where the axis represents categories rather than numerical values.

If you are updating the ticks of this ticker regularly and in a dynamic fashion (e.g. dependent on the axis range), it is a sign that you should probably create an own ticker by subclassing [QCPAxisTicker](#), instead of using this one.

The ticker can be created and assigned to an axis like this:

10.408.2 Constructor & Destructor Documentation

10.408.2.1 QCPAxisTickerText()

```
QCPAxisTickerText::QCPAxisTickerText ( )
```

Constructs the ticker and sets reasonable default values. Axis tickers are commonly created managed by a QSharedPointer, which then can be passed to [QCPAxis::setTicker](#).

10.408.3 Member Function Documentation

10.408.3.1 addTick()

```
void QCPAxisTickerText::addTick (
    double position,
    const QString & label )
```

Adds a single tick to the axis at the given axis coordinate *position*, with the provided tick *label*.

See also

[addTicks](#), [setTicks](#), [clear](#)

10.408.3.2 addTicks() [1/2]

```
void QCPAxisTickerText::addTicks (
    const QMap< double, QString > & ticks )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided *ticks* to the ones already existing. The map key of *ticks* corresponds to the axis coordinate, and the map value is the string that will appear as tick label.

An alternative to manipulate ticks is to directly access the internal storage with the [ticks](#) getter.

See also

[addTick](#), [setTicks](#), [clear](#)

10.408.3.3 addTicks() [2/2]

```
void QCPAxisTickerText::addTicks (
    const QVector< double > & positions,
    const QVector< QString > & labels )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided ticks to the ones already existing. The entries of *positions* correspond to the axis coordinates, and the entries of *labels* are the respective strings that will appear as tick labels.

An alternative to manipulate ticks is to directly access the internal storage with the [ticks](#) getter.

See also

[addTick](#), [setTicks](#), [clear](#)

10.408.3.4 clear()

```
void QCPAxisTickerText::clear ( )
```

Clears all ticks.

An alternative to manipulate ticks is to directly access the internal storage with the [ticks](#) getter.

See also

[setTicks](#), [addTicks](#), [addTick](#)

10.408.3.5 createTickVector()

```
QVector< double > QCPAxisTickerText::createTickVector (
    double tickStep,
    const QCPRange & range ) [protected], [virtual]
```

Returns the externally provided tick coordinates which are in the specified *range*. If available, one tick above and below the range is provided in addition, to allow possible sub tick calculation. The parameter *tickStep* is ignored.

\seebaseclassmethod

Reimplemented from [QCPAxisTicker](#).

10.408.3.6 getSubTickCount()

```
int QCPAxisTickerText::getSubTickCount (
    double tickStep ) [protected], [virtual]
```

Returns the sub tick count that was configured with [setSubTickCount](#).

\seebaseclassmethod

Reimplemented from [QCPAxisTicker](#).

10.408.3.7 getTickLabel()

```
QString QCPAxisTickerText::getTickLabel (
    double tick,
    const QLocale & locale,
    QChar formatChar,
    int precision ) [protected], [virtual]
```

Returns the tick label which corresponds to the key *tick* in the internal tick storage. Since the labels are provided externally, *locale*, *formatChar*, and *precision* are ignored.

\seebaseclassmethod

Reimplemented from [QCPAxisTicker](#).

10.408.3.8 `getTickStep()`

```
double QCPAxisTickerText::getTickStep (
    const QCPRange & range ) [protected], [virtual]
```

Since the tick coordinates are provided externally, this method implementation does nothing.

\see `baseclassmethod`

Reimplemented from [QCPAxisTicker](#).

10.408.3.9 `setSubTickCount()`

```
void QCPAxisTickerText::setSubTickCount (
    int subTicks )
```

Sets the number of sub ticks that shall appear between ticks. For [QCPAxisTickerText](#), there is no automatic sub tick count calculation. So if sub ticks are needed, they must be configured with this method.

10.408.3.10 `setTicks()` [1/2]

```
void QCPAxisTickerText::setTicks (
    const QMap< double, QString > & ticks )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the ticks that shall appear on the axis. The map key of *ticks* corresponds to the axis coordinate, and the map value is the string that will appear as tick label.

An alternative to manipulate ticks is to directly access the internal storage with the [ticks](#) getter.

See also

[addTicks](#), [addTick](#), [clear](#)

10.408.3.11 `setTicks()` [2/2]

```
void QCPAxisTickerText::setTicks (
    const QVector< double > & positions,
    const QVector< QString > & labels )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the ticks that shall appear on the axis. The entries of *positions* correspond to the axis coordinates, and the entries of *labels* are the respective strings that will appear as tick labels.

See also

[addTicks](#), [addTick](#), [clear](#)

10.408.3.12 ticks()

```
QMap< double, QString > & QCPAxisTickerText::ticks ( ) [inline]
```

Returns a non-const reference to the internal map which stores the tick coordinates and their labels.

You can access the map directly in order to add, remove or manipulate ticks, as an alternative to using the methods provided by [QCPAxisTickerText](#), such as [setTicks](#) and [addTick](#).

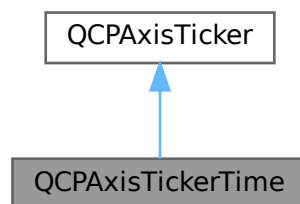
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

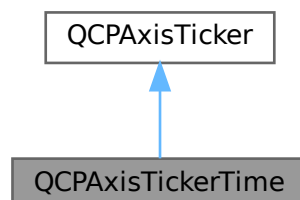
10.409 QCPAxisTickerTime Class Reference

Specialized axis ticker for time spans in units of milliseconds to days.

Inheritance diagram for QCPAxisTickerTime:



Collaboration diagram for QCPAxisTickerTime:



Public Types

- enum [TimeUnit](#) {
 [tuMilliseconds](#) , [tuSeconds](#) , [tuMinutes](#) , [tuHours](#) ,
 [tuDays](#) }

Public Types inherited from [QCPAxisTicker](#)

- enum [TickStepStrategy](#) { [tssReadability](#) , [tssMeetTickCount](#) }

Public Member Functions

- [QCPAxisTickerTime](#) ()
- QString [timeFormat](#) () const
- int [fieldWidth](#) ([TimeUnit](#) unit) const
- void [setTimeFormat](#) (const QString &format)
- void [setFieldWidth](#) ([TimeUnit](#) unit, int width)

Public Member Functions inherited from [QCPAxisTicker](#)

- [QCPAxisTicker](#) ()
- [TickStepStrategy](#) [tickStepStrategy](#) () const
- int [tickCount](#) () const
- double [tickOrigin](#) () const
- void [setTickStepStrategy](#) ([TickStepStrategy](#) strategy)
- void [setTickCount](#) (int count)
- void [setTickOrigin](#) (double origin)
- virtual void [generate](#) (const [QCPRange](#) &range, const QLocale &locale, QChar formatChar, int precision, QVector< double > &ticks, QVector< double > *subTicks, QVector< QString > *tickLabels)

Protected Member Functions

- virtual double [getTickStep](#) (const [QCPRange](#) &range) Q_DECL_OVERRIDE
- virtual int [getSubTickCount](#) (double tickStep) Q_DECL_OVERRIDE
- virtual QString [getTickLabel](#) (double tick, const QLocale &locale, QChar formatChar, int precision) Q_DECL_OVERRIDE
- void [replaceUnit](#) (QString &text, [TimeUnit](#) unit, int value) const

Protected Member Functions inherited from [QCPAxisTicker](#)

- virtual QVector< double > [createTickVector](#) (double tickStep, const [QCPRange](#) &range)
- virtual QVector< double > [createSubTickVector](#) (int subTickCount, const QVector< double > &ticks)
- virtual QVector< QString > [createLabelVector](#) (const QVector< double > &ticks, const QLocale &locale, QChar formatChar, int precision)
- void [trimTicks](#) (const [QCPRange](#) &range, QVector< double > &ticks, bool keepOneOutlier) const
- double [pickClosest](#) (double target, const QVector< double > &candidates) const
- double [getMantissa](#) (double input, double *magnitude=nullptr) const
- double [cleanMantissa](#) (double input) const

Protected Attributes

- QString **mTimeFormat**
- QHash< [TimeUnit](#), int > **mFieldWidth**
- [TimeUnit](#) **mSmallestUnit**
- [TimeUnit](#) **mBiggestUnit**
- QHash< [TimeUnit](#), QString > **mFormatPattern**

Protected Attributes inherited from [QCPAxisTicker](#)

- [TickStepStrategy](#) **mTickStepStrategy**
- int **mTickCount**
- double **mTickOrigin**

10.409.1 Detailed Description

Specialized axis ticker for time spans in units of milliseconds to days.

This [QCPAxisTicker](#) subclass generates ticks that corresponds to time intervals.

The format of the time display in the tick labels is controlled with [setTimeFormat](#) and [setFieldWidth](#). The time coordinate is in the unit of seconds with respect to the time coordinate zero. Unlike with [QCPAxisTickerDateTime](#), the ticks don't correspond to a specific calendar date and time.

The time can be displayed in milliseconds, seconds, minutes, hours and days. Depending on the largest available unit in the format specified with [setTimeFormat](#), any time spans above will be carried in that largest unit. So for example if the format string is "%m:%s" and a tick at coordinate value 7815 (being 2 hours, 10 minutes and 15 seconds) is created, the resulting tick label will show "130:15" (130 minutes, 15 seconds). If the format string is "%h:%m:%s", the hour unit will be used and the label will thus be "02:10:15". Negative times with respect to the axis zero will carry a leading minus sign.

The ticker can be created and assigned to an axis like this:

Here is an example of a time axis providing time information in days, hours and minutes. Due to the axis range spanning a few days and the wanted tick count ([setTickCount](#)), the ticker decided to use tick steps of 12 hours:

The format string for this example is

Note

If you rather wish to display calendar dates and times, have a look at [QCPAxisTickerDateTime](#) instead.

10.409.2 Member Enumeration Documentation

10.409.2.1 TimeUnit

```
enum QCPAxisTickerTime::TimeUnit
```

Defines the logical units in which fractions of time spans can be expressed.

See also

[setFieldWidth](#), [setTimeFormat](#)

Enumerator

tuMilliseconds	Milliseconds, one thousandth of a second (%z in setTimeFormat)
tuSeconds	Seconds (%s in setTimeFormat)
tuMinutes	Minutes (%m in setTimeFormat)
tuHours	Hours (%h in setTimeFormat)
tuDays	Days (%d in setTimeFormat)

10.409.3 Constructor & Destructor Documentation

10.409.3.1 QCPAxisTickerTime()

```
QCPAxisTickerTime::QCPAxisTickerTime ( )
```

Constructs the ticker and sets reasonable default values. Axis tickers are commonly created managed by a QSharedPointer, which then can be passed to [QCPAxis::setTicker](#).

10.409.4 Member Function Documentation

10.409.4.1 getSubTickCount()

```
int QCPAxisTickerTime::getSubTickCount (
    double tickStep ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

10.409.4.2 getTickLabel()

```
QString QCPAxisTickerTime::getTickLabel (
    double tick,
    const QLocale & locale,
    QChar formatChar,
    int precision ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

10.409.4.3 getTickStep()

```
double QCPAxisTickerTime::getTickStep (
    const QCPRange & range ) [protected], [virtual]
```

Reimplemented from [QCPAxisTicker](#).

10.409.4.4 setFieldWidth()

```
void QCPAxisTickerTime::setFieldWidth (
    QCPAxisTickerTime::TimeUnit unit,
    int width )
```

Sets the field width of the specified *unit* to be *width* digits, when displayed in the tick label. If the number for the specific unit is shorter than *width*, it will be padded with an according number of zeros to the left in order to reach the field width.

See also

[setTimeFormat](#)

10.409.4.5 setTimeFormat()

```
void QCPAxisTickerTime::setTimeFormat (
    const QString & format )
```

Sets the format that will be used to display time in the tick labels.

The available patterns are:

- %z for milliseconds
- %s for seconds
- %m for minutes
- %h for hours
- %d for days

The field width (zero padding) can be controlled for each unit with [setFieldWidth](#).

The largest unit that appears in *format* will carry all the remaining time of a certain tick coordinate, even if it overflows the natural limit of the unit. For example, if %m is the largest unit it might become larger than 59 in order to consume larger time values. If on the other hand %h is available, the minutes will wrap around to zero after 59 and the time will carry to the hour digit.

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

Public Types

- enum [WidthType](#) { [wtAbsolute](#) , [wtAxisRectRatio](#) , [wtPlotCoords](#) }

Public Member Functions

- [QCPBars](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- double **width** () const
- [WidthType](#) **widthType** () const
- [QCPBarsGroup](#) * **barsGroup** () const
- double **baseValue** () const
- double **stackingGap** () const
- [QCPBars](#) * **barBelow** () const
- [QCPBars](#) * **barAbove** () const
- QSharedPointer< [QCPBarsDataContainer](#) > **data** () const
- void **setData** (QSharedPointer< [QCPBarsDataContainer](#) > data)
- void **setData** (const QVector< double > &keys, const QVector< double > &values, bool alreadySorted=false)
- void **setWidth** (double width)
- void **setWidthType** ([WidthType](#) widthType)
- void **setBarsGroup** ([QCPBarsGroup](#) *barsGroup)
- void **setBaseValue** (double baseValue)
- void **setStackingGap** (double pixels)
- void **addData** (const QVector< double > &keys, const QVector< double > &values, bool alreadySorted=false)
- void **addData** (double key, double value)
- void **moveBelow** ([QCPBars](#) *bars)
- void **moveAbove** ([QCPBars](#) *bars)
- virtual [QCPDataSelection](#) **selectTestRect** (const QRectF &rect, bool onlySelectable) const Q_DECL_OVERRIDE
- virtual double **selectTest** (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE
- virtual [QCPRange](#) **getKeyRange** (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const Q_DECL_OVERRIDE
- virtual [QCPRange](#) **getValueRange** (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const Q_DECL_OVERRIDE
- virtual QPointF **dataPixelPosition** (int index) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractPlottable1D](#)< [QCPBarsData](#) >

- [QCPAbstractPlottable1D](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- virtual int **dataCount** () const Q_DECL_OVERRIDE
- virtual double **dataMainKey** (int index) const Q_DECL_OVERRIDE
- virtual double **dataSortKey** (int index) const Q_DECL_OVERRIDE
- virtual double **dataMainValue** (int index) const Q_DECL_OVERRIDE
- virtual [QCPRange](#) **dataValueRange** (int index) const Q_DECL_OVERRIDE
- virtual bool **sortKeysIsMainKey** () const Q_DECL_OVERRIDE
- virtual int **findBegin** (double sortKey, bool expandedRange=true) const Q_DECL_OVERRIDE
- virtual int **findEnd** (double sortKey, bool expandedRange=true) const Q_DECL_OVERRIDE
- virtual [QCPPlottableInterface1D](#) * **interface1D** () Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QString](#) **name** () const
- bool **antialiasedFill** () const
- bool **antialiasedScatters** () const
- [QPen](#) **pen** () const
- [QBrush](#) **brush** () const
- [QCPAxis](#) * **keyAxis** () const
- [QCPAxis](#) * **valueAxis** () const
- [QCP::SelectionType](#) **selectable** () const
- bool **selected** () const
- [QCPDataSelection](#) **selection** () const
- [QCPSelectionDecorator](#) * **selectionDecorator** () const
- void **setName** (const [QString](#) &name)
- void **setAntialiasedFill** (bool enabled)
- void **setAntialiasedScatters** (bool enabled)
- void **setPen** (const [QPen](#) &pen)
- void **setBrush** (const [QBrush](#) &brush)
- void **setKeyAxis** ([QCPAxis](#) *axis)
- void **setValueAxis** ([QCPAxis](#) *axis)
- Q_SLOT void **setSelectable** ([QCP::SelectionType](#) selectable)
- Q_SLOT void **setSelection** ([QCPDataSelection](#) selection)
- void **setSelectionDecorator** ([QCPSelectionDecorator](#) *decorator)
- void **coordsToPixels** (double key, double value, double &x, double &y) const
- const [QPointF](#) **coordsToPixels** (double key, double value) const
- void **pixelsToCoords** (double x, double y, double &key, double &value) const
- void **pixelsToCoords** (const [QPointF](#) &pixelPos, double &key, double &value) const
- void **rescaleAxes** (bool onlyEnlarge=false) const
- void **rescaleKeyAxis** (bool onlyEnlarge=false) const
- void **rescaleValueAxis** (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool **addToLegend** ([QCPLegend](#) *legend)
- bool **addToLegend** ()
- bool **removeFromLegend** ([QCPLegend](#) *legend) const
- bool **removeFromLegend** () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const [QString](#) &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void [drawLegendIcon](#) ([QCPPainter](#) *painter, const QRectF &rect) const Q_DECL_OVERRIDE
- void [getVisibleDataBounds](#) ([QCPBarsDataContainer::const_iterator](#) &begin, [QCPBarsDataContainer::const_iterator](#) &end) const
- QRectF [getBarRect](#) (double key, double value) const
- void [getPixelWidth](#) (double key, double &lower, double &upper) const
- double [getStackedBaseValue](#) (double key, bool positive) const

Protected Member Functions inherited from [QCPAbstractPlottable1D](#)< [QCPBarsData](#) >

- void [getDataSegments](#) (QList< [QCPDataRange](#) > &selectedSegments, QList< [QCPDataRange](#) > &unselectedSegments) const
- void [drawPolyline](#) ([QCPPainter](#) *painter, const QVector< QPointF > &lineData) const

Protected Member Functions inherited from [QCPAbstractPlottable](#)

- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- void [applyFillAntialiasingHint](#) ([QCPPainter](#) *painter) const
- void [applyScattersAntialiasingHint](#) ([QCPPainter](#) *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Static Protected Member Functions

- static void [connectBars](#) ([QCPBars](#) *lower, [QCPBars](#) *upper)

Protected Attributes

- double [mWidth](#)
- [WidthType](#) [mWidthType](#)
- [QCPBarsGroup](#) * [mBarsGroup](#)
- double [mBaseValue](#)
- double [mStackingGap](#)
- QPointer< [QCPBars](#) > [mBarBelow](#)
- QPointer< [QCPBars](#) > [mBarAbove](#)

Protected Attributes inherited from [QCPAbstractPlottable1D](#)< [QCPBarsData](#) >

- [QSharedPointer](#)< [QCPDataContainer](#)< [QCPBarsData](#) > > **mDataContainer**

Protected Attributes inherited from [QCPAbstractPlottable](#)

- [QString](#) **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- [QPen](#) **mPen**
- [QBrush](#) **mBrush**
- [QPointer](#)< [QCPAxis](#) > **mKeyAxis**
- [QPointer](#)< [QCPAxis](#) > **mValueAxis**
- [QCP::SelectionType](#) **mSelectable**
- [QCPDataSelection](#) **mSelection**
- [QCPSelectionDecorator](#) * **mSelectionDecorator**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer](#)< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPLegend](#)
- class [QCPBarsGroup](#)

Additional Inherited Members

Signals inherited from [QCPAbstractPlottable](#)

- void [selectionChanged](#) (bool [selected](#))
- void [selectionChanged](#) (const [QCPDataSelection](#) &[selection](#))
- void [selectableChanged](#) ([QCP::SelectionType](#) [selectable](#))

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *[newLayer](#))

10.410.1 Detailed Description

A plottable representing a bar chart in a plot.

To plot data, assign it with the [setData](#) or [addData](#) functions.

10.410.2 Changing the appearance

The appearance of the bars is determined by the pen and the brush ([setPen](#), [setBrush](#)). The width of the individual bars can be controlled with [setWidthType](#) and [setWidth](#).

Bar charts are stackable. This means, two [QCPBars](#) plottables can be placed on top of each other (see [QCPBars::moveAbove](#)). So when two bars are at the same key position, they will appear stacked.

If you would like to group multiple [QCPBars](#) plottables together so they appear side by side as shown below, use [QCPBarsGroup](#).

10.410.3 Usage

Like all data representing objects in [QCustomPlot](#), the [QCPBars](#) is a plottable ([QCPAbstractPlottable](#)). So the plottable-interface of [QCustomPlot](#) applies ([QCustomPlot::plottable](#), [QCustomPlot::removePlottable](#), etc.)

Usually, you first create an instance:

which registers it with the [QCustomPlot](#) instance of the passed axes. Note that this [QCustomPlot](#) instance takes ownership of the plottable, so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead. The newly created plottable can be modified, e.g.:

10.410.4 Member Enumeration Documentation

10.410.4.1 WidthType

```
enum QCPBars::WidthType
```

Defines the ways the width of the bar can be specified. Thus it defines what the number passed to [setWidth](#) actually means.

See also

[setWidthType](#), [setWidth](#)

Enumerator

wtAbsolute	Bar width is in absolute pixels.
wtAxisRectRatio	Bar width is given by a fraction of the axis rect size.
wtPlotCoords	Bar width is in key coordinates and thus scales with the key axis range.

10.410.5 Constructor & Destructor Documentation

10.410.5.1 QCPBars()

```
QCPBars::QCPBars (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis ) [explicit]
```

Constructs a bar chart which uses *keyAxis* as its key axis ("x") and *valueAxis* as its value axis ("y"). *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#) instance and not have the same orientation. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

The created [QCPBars](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPBars](#), so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead.

10.410.6 Member Function Documentation

10.410.6.1 [addData\(\)](#) [1/2]

```
void QCPBars::addData (
    const QVector< double > & keys,
    const QVector< double > & values,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided points in *keys* and *values* to the current data. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

10.410.6.2 [addData\(\)](#) [2/2]

```
void QCPBars::addData (
    double key,
    double value )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts. Adds the provided data point as *key* and *value* to the current data.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

10.410.6.3 [barAbove\(\)](#)

```
QCPBars * QCPBars::barAbove ( ) const [inline]
```

Returns the bars plottable that is directly above this bars plottable. If there is no such plottable, returns `nullptr`.

See also

[barBelow](#), [moveBelow](#), [moveAbove](#)

10.410.6.4 barBelow()

```
QCPBars * QCPBars::barBelow ( ) const [inline]
```

Returns the bars plottable that is directly below this bars plottable. If there is no such plottable, returns `nullptr`.

See also

[barAbove](#), [moveBelow](#), [moveAbove](#)

10.410.6.5 data()

```
QSharedPointer< QCPBarsDataContainer > QCPBars::data ( ) const [inline]
```

Returns a shared pointer to the internal data storage of type `QCPBarsDataContainer`. You may use it to directly manipulate the data, which may be more convenient and faster than using the regular [setData](#) or [addData](#) methods.

10.410.6.6 dataPixelPosition()

```
QPointF QCPBars::dataPixelPosition (
    int index ) const [virtual]
```

Returns the pixel position on the widget surface at which the data point at the given *index* appears.

Usually this corresponds to the point of `dataMainKey/ dataMainValue`, in pixel coordinates. However, depending on the plottable, this might be a different apparent position than just a coord-to-pixel transform of those values. For example, [QCPBars](#) apparent data values can be shifted depending on their stacking, bar grouping or configured base value.

Reimplemented from [QCPAbstractPlottable1D< QCPBarsData >](#).

10.410.6.7 draw()

```
void QCPBars::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

10.410.6.8 drawLegendIcon()

```
void QCPBars::drawLegendIcon (
    QCPPainter * painter,
    const QRectF & rect ) const [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

10.410.6.9 getKeyRange()

```
QCPRange QCPBars::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implements [QCPAbstractPlottable](#).

10.410.6.10 getValueRange()

```
QCPRange QCPBars::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implements [QCPAbstractPlottable](#).

10.410.6.11 moveAbove()

```
void QCPBars::moveAbove (
    QCPBars * bars )
```

Moves this bars plottable above *bars*. In other words, the bars of this plottable will appear above the bars of *bars*. The move target *bars* must use the same key and value axis as this plottable.

Inserting into and removing from existing bar stacking is handled gracefully. If *bars* already has a bars object above itself, this bars object is inserted between the two. If this bars object is already between two other bars, the two other bars will be stacked on top of each other after the operation.

To remove this bars plottable from any stacking, set *bars* to `nullptr`.

See also

[moveBelow](#), [barBelow](#), [barAbove](#)

10.410.6.12 moveBelow()

```
void QCPBars::moveBelow (
    QCPBars * bars )
```

Moves this bars plottable below *bars*. In other words, the bars of this plottable will appear below the bars of *bars*. The move target *bars* must use the same key and value axis as this plottable.

Inserting into and removing from existing bar stacking is handled gracefully. If *bars* already has a bars object below itself, this bars object is inserted between the two. If this bars object is already between two other bars, the two other bars will be stacked on top of each other after the operation.

To remove this bars plottable from any stacking, set *bars* to `nullptr`.

See also

[moveBelow](#), [barAbove](#), [barBelow](#)

10.410.6.13 selectTest()

```
double QCPBars::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Implements a selectTest specific to this plottable's point geometry.

If *details* is not 0, it will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

\see baseclass method [QCPAbstractPlottable::selectTest](#)

Reimplemented from [QCPAbstractPlottable1D< QCPBarsData >](#).

10.410.6.14 selectTestRect()

```
QCPDataSelection QCPBars::selectTestRect (
    const QRectF & rect,
    bool onlySelectable ) const [virtual]
```

Returns a data selection containing all the data points of this plottable which are contained (or hit by) *rect*. This is used mainly in the selection rect interaction for data selection (data selection mechanism).

If *onlySelectable* is true, an empty [QCPDataSelection](#) is returned if this plottable is not selectable (i.e. if [QCPAbstractPlottable::setSelectable](#) is [QCP::stNone](#)).

Note

rect must be a normalized rect (positive or zero width and height). This is especially important when using the rect of [QCPSelectionRect::accepted](#), which is not necessarily normalized. Use `QRect::normalized()` when passing a rect which might not be normalized.

Reimplemented from [QCPAbstractPlottable1D< QCPBarsData >](#).

10.410.6.15 setBarsGroup()

```
void QCPBars::setBarsGroup (
    QCPBarsGroup * barsGroup )
```

Sets to which [QCPBarsGroup](#) this [QCPBars](#) instance belongs to. Alternatively, you can also use [QCPBarsGroup::append](#).

To remove this [QCPBars](#) from any group, set *barsGroup* to `nullptr`.

10.410.6.16 setBaseValue()

```
void QCPBars::setBaseValue (
    double baseValue )
```

Sets the base value of this bars plottable.

The base value defines where on the value coordinate the bars start. How far the bars extend from the base value is given by their individual value data. For example, if the base value is set to 1, a bar with data value 2 will have its lowest point at value coordinate 1 and highest point at 3.

For stacked bars, only the base value of the bottom-most [QCPBars](#) has meaning.

The default base value is 0.

10.410.6.17 setData() [1/2]

```
void QCPBars::setData (
    const QVector< double > & keys,
    const QVector< double > & values,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data with the provided points in *keys* and *values*. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

See also

[addData](#)

10.410.6.18 setData() [2/2]

```
void QCPBars::setData (
    QSharedPointer< QCPBarsDataContainer > data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data container with the provided *data* container.

Since a QSharedPointer is used, multiple [QCPBars](#) may share the same data container safely. Modifying the data in the container will then affect all bars that share the container. Sharing can be achieved by simply exchanging the data containers wrapped in shared pointers:

If you do not wish to share containers, but create a copy from an existing container, rather use the [QCPDataContainer<DataType>::set](#) method on the bar's data container directly:

See also

[addData](#)

10.410.6.19 setStackingGap()

```
void QCPBars::setStackingGap (
    double pixels )
```

If this bars plottable is stacked on top of another bars plottable ([moveAbove](#)), this method allows specifying a distance in *pixels*, by which the drawn bar rectangles will be separated by the bars below it.

10.410.6.20 `setWidth()`

```
void QCPBars::setWidth (
    double width )
```

Sets the width of the bars.

How the number passed as *width* is interpreted (e.g. screen pixels, plot coordinates,...), depends on the currently set width type, see [setWidthType](#) and [WidthType](#).

10.410.6.21 `setWidthType()`

```
void QCPBars::setWidthType (
    QCPBars::WidthType widthType )
```

Sets how the width of the bars is defined. See the documentation of [WidthType](#) for an explanation of the possible values for *widthType*.

The default value is [wtPlotCoords](#).

See also

[setWidth](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.411 QCPBarsData Class Reference

Holds the data of one single data point (one bar) for [QCPBars](#).

Public Member Functions

- [QCPBarsData](#) ()
- [QCPBarsData](#) (double key, double value)
- double [sortKey](#) () const
- double [mainKey](#) () const
- double [mainValue](#) () const
- [QCPRange](#) [valueRange](#) () const

Static Public Member Functions

- static [QCPBarsData](#) [fromSortKey](#) (double [sortKey](#))
- static bool [sortKeysIsMainKey](#) ()

Public Attributes

- double **key**
- double **value**

10.411.1 Detailed Description

Holds the data of one single data point (one bar) for [QCPBars](#).

The stored data is:

- *key*: coordinate on the key axis of this bar (this is the *mainKey* and the *sortKey*)
- *value*: height coordinate on the value axis of this bar (this is the *mainValue*)

The container for storing multiple data points is [QCPBarsDataContainer](#). It is a typedef for [QCPDataContainer](#) with [QCPBarsData](#) as the `DataType` template parameter. See the documentation there for an explanation regarding the data type's generic methods.

See also

[QCPBarsDataContainer](#)

10.411.2 Constructor & Destructor Documentation

10.411.2.1 QCPBarsData() [1/2]

```
QCPBarsData::QCPBarsData ( )
```

Constructs a bar data point with key and value set to zero.

10.411.2.2 QCPBarsData() [2/2]

```
QCPBarsData::QCPBarsData (
    double key,
    double value )
```

Constructs a bar data point with the specified *key* and *value*.

10.411.3 Member Function Documentation

10.411.3.1 fromSortKey()

```
static QCPBarsData QCPBarsData::fromSortKey (
    double sortKey ) [inline], [static]
```

Returns a data point with the specified *sortKey*. All other members are set to zero.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.411.3.2 mainKey()

```
double QCPBarsData::mainKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.411.3.3 mainValue()

```
double QCPBarsData::mainValue ( ) const [inline]
```

Returns the *value* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.411.3.4 sortKey()

```
double QCPBarsData::sortKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.411.3.5 sortKeyIsMainKey()

```
static static bool QCPBarsData::sortKeyIsMainKey ( ) [inline], [static]
```

Since the member *key* is both the data point key coordinate and the data ordering parameter, this method returns true.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.411.3.6 valueRange()

```
QCPRange QCPBarsData::valueRange ( ) const [inline]
```

Returns a [QCPRange](#) with both lower and upper boundary set to *value* of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

Public Member Functions

- [QCPBarsGroup](#) ([QCustomPlot](#) *parentPlot)
- [SpacingType](#) **spacingType** () const
- double **spacing** () const
- void **setSpacingType** ([SpacingType](#) spacingType)
- void **setSpacing** (double spacing)
- [QList](#)< [QCPBars](#) * > **bars** () const
- [QCPBars](#) * **bars** (int index) const
- int **size** () const
- bool **isEmpty** () const
- void **clear** ()
- bool **contains** ([QCPBars](#) *bars) const
- void **append** ([QCPBars](#) *bars)
- void **insert** (int i, [QCPBars](#) *bars)
- void **remove** ([QCPBars](#) *bars)

Protected Member Functions

- void **registerBars** ([QCPBars](#) *bars)
- void **unregisterBars** ([QCPBars](#) *bars)
- double **keyPixelOffset** (const [QCPBars](#) *bars, double keyCoord)
- double **getPixelSpacing** (const [QCPBars](#) *bars, double keyCoord)

Protected Attributes

- [QCustomPlot](#) * **mParentPlot**
- [SpacingType](#) **mSpacingType**
- double **mSpacing**
- [QList](#)< [QCPBars](#) * > **mBars**

Friends

- class **QCPBars**

10.412.1 Detailed Description

Groups multiple [QCPBars](#) together so they appear side by side.

When showing multiple [QCPBars](#) in one plot which have bars at identical keys, it may be desirable to have them appearing next to each other at each key. This is what adding the respective [QCPBars](#) plottables to a [QCPBarsGroup](#) achieves. (An alternative approach is to stack them on top of each other, see [QCPBars::moveAbove](#).)

10.412.2 Usage

To add a [QCPBars](#) plottable to the group, create a new group and then add the respective bars instances:

Alternatively to appending to the group like shown above, you can also set the group on the [QCPBars](#) plottable via [QCPBars::setBarsGroup](#).

The spacing between the bars can be configured via [setSpacingType](#) and [setSpacing](#). The bars in this group appear in the plot in the order they were appended. To insert a bars plottable at a certain index position, or to reposition a bars plottable which is already in the group, use [insert](#).

To remove specific bars from the group, use either [remove](#) or call [QCPBars::setBarsGroup\(0\)](#) on the respective bars plottable.

To clear the entire group, call [clear](#), or simply delete the group.

10.412.3 Example

The image above is generated with the following code:

10.412.4 Member Enumeration Documentation

10.412.4.1 SpacingType

```
enum QCPBarsGroup::SpacingType
```

Defines the ways the spacing between bars in the group can be specified. Thus it defines what the number passed to [setSpacing](#) actually means.

See also

[setSpacingType](#), [setSpacing](#)

Enumerator

stAbsolute	Bar spacing is in absolute pixels.
stAxisRectRatio	Bar spacing is given by a fraction of the axis rect size.
stPlotCoords	Bar spacing is in key coordinates and thus scales with the key axis range.

10.412.5 Constructor & Destructor Documentation

10.412.5.1 QCPBarsGroup()

```
QCPBarsGroup::QCPBarsGroup (
    QCustomPlot * parentPlot ) [explicit]
```

Constructs a new bars group for the specified [QCustomPlot](#) instance.

10.412.6 Member Function Documentation

10.412.6.1 `append()`

```
void QCPBarsGroup::append (
    QCPBars * bars )
```

Adds the specified *bars* plottable to this group. Alternatively, you can also use [QCPBars::setBarsGroup](#) on the *bars* instance.

See also

[insert](#), [remove](#)

10.412.6.2 `bars()` [1/2]

```
QList< QCPBars * > QCPBarsGroup::bars ( ) const [inline]
```

Returns all bars currently in this group.

See also

`bars(int index)`

10.412.6.3 `bars()` [2/2]

```
QCPBars * QCPBarsGroup::bars (
    int index ) const
```

Returns the [QCPBars](#) instance with the specified *index* in this group. If no such [QCPBars](#) exists, returns `nullptr`.

See also

[bars\(\)](#), [size](#)

10.412.6.4 `clear()`

```
void QCPBarsGroup::clear ( )
```

Removes all [QCPBars](#) plottables from this group.

See also

[isEmpty](#)

10.412.6.5 contains()

```
bool QCPBarsGroup::contains (
    QCPBars * bars ) const [inline]
```

Returns whether the specified *bars* plottable is part of this group.

10.412.6.6 insert()

```
void QCPBarsGroup::insert (
    int i,
    QCPBars * bars )
```

Inserts the specified *bars* plottable into this group at the specified index position *i*. This gives you full control over the ordering of the bars.

bars may already be part of this group. In that case, *bars* is just moved to the new index position.

See also

[append](#), [remove](#)

10.412.6.7 isEmpty()

```
bool QCPBarsGroup::isEmpty ( ) const [inline]
```

Returns whether this bars group is empty.

See also

[size](#)

10.412.6.8 remove()

```
void QCPBarsGroup::remove (
    QCPBars * bars )
```

Removes the specified *bars* plottable from this group.

See also

[contains](#), [clear](#)

10.412.6.9 setSpacing()

```
void QCPBarsGroup::setSpacing (
    double spacing )
```

Sets the spacing between adjacent bars. What the number passed as *spacing* actually means, is defined by the current [SpacingType](#), which can be set with [setSpacingType](#).

See also

[setSpacingType](#)

10.412.6.10 setSpacingType()

```
void QCPBarsGroup::setSpacingType (
    SpacingType spacingType )
```

Sets how the spacing between adjacent bars is interpreted. See [SpacingType](#).

The actual spacing can then be specified with [setSpacing](#).

See also

[setSpacing](#)

10.412.6.11 size()

```
int QCPBarsGroup::size ( ) const [inline]
```

Returns the number of [QCPBars](#) plottables that are part of this group.

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.413 QCPCOLORGradient Class Reference

Defines a color gradient for use with e.g. [QCPCOLORMap](#).

Public Types

- enum [ColorInterpolation](#) { [ciRGB](#) , [ciHSV](#) }
- enum [NanHandling](#) { [nhNone](#) , [nhLowestColor](#) , [nhHighestColor](#) , [nhTransparent](#) , [nhNanColor](#) }
- enum [GradientPreset](#) { [gpGrayscale](#) , [gpHot](#) , [gpCold](#) , [gpNight](#) , [gpCandy](#) , [gpGeography](#) , [gpIon](#) , [gpThermal](#) , [gpPolar](#) , [gpSpectrum](#) , [gpJet](#) , [gpHues](#) }

Public Member Functions

- [QCPColorGradient](#) ()
- [QCPColorGradient](#) ([GradientPreset](#) preset)
- bool **operator==** (const [QCPColorGradient](#) &other) const
- bool **operator!=** (const [QCPColorGradient](#) &other) const
- int **levelCount** () const
- QMap< double, QColor > **colorStops** () const
- [ColorInterpolation](#) **colorInterpolation** () const
- [NanHandling](#) **nanHandling** () const
- QColor **nanColor** () const
- bool **periodic** () const
- void **setLevelCount** (int n)
- void **setColorStops** (const QMap< double, QColor > &colorStops)
- void **setColorStopAt** (double position, const QColor &color)
- void **setColorInterpolation** ([ColorInterpolation](#) interpolation)
- void **setNanHandling** ([NanHandling](#) handling)
- void **setNanColor** (const QColor &color)
- void **setPeriodic** (bool enabled)
- void **colorize** (const double *data, const [QCPRange](#) &range, QRgb *scanLine, int n, int dataIndexFactor=1, bool logarithmic=false)
- void **colorize** (const double *data, const unsigned char *alpha, const [QCPRange](#) &range, QRgb *scanLine, int n, int dataIndexFactor=1, bool logarithmic=false)
- QRgb **color** (double position, const [QCPRange](#) &range, bool logarithmic=false)
- void **loadPreset** ([GradientPreset](#) preset)
- void **clearColorStops** ()
- [QCPColorGradient](#) **inverted** () const

Protected Member Functions

- bool **stopsUseAlpha** () const
- void **updateColorBuffer** ()

Protected Attributes

- int **mLevelCount**
- QMap< double, QColor > **mColorStops**
- [ColorInterpolation](#) **mColorInterpolation**
- [NanHandling](#) **mNanHandling**
- QColor **mNanColor**
- bool **mPeriodic**
- QVector< QRgb > **mColorBuffer**
- bool **mColorBufferInvalidated**

10.413.1 Detailed Description

Defines a color gradient for use with e.g. [QCPColorMap](#).

This class describes a color gradient which can be used to encode data with color. For example, [QCPColorMap](#) and [QCPColorScale](#) have [setGradient](#) methods which take an instance of this class. Colors are set with [setColorStopAt\(double position, const QColor &color\)](#) with a *position* from 0 to 1. In between these defined color positions, the color will be interpolated linearly either in RGB or HSV space, see [setColorInterpolation](#).

Alternatively, load one of the preset color gradients shown in the image below, with [loadPreset](#), or by directly specifying the preset in the constructor.

Apart from red, green and blue components, the gradient also interpolates the alpha values of the configured color stops. This allows to display some portions of the data range as transparent in the plot.

How NaN values are interpreted can be configured with [setNanHandling](#).

The constructor [QCPColorGradient\(GradientPreset preset\)](#) allows directly converting a [GradientPreset](#) to a [QCPColorGradient](#). This means that you can directly pass [GradientPreset](#) to all the *setGradient* methods, e.g.:

The total number of levels used in the gradient can be set with [setLevelCount](#). Whether the color gradient shall be applied periodically (wrapping around) to data values that lie outside the data range specified on the plottable instance can be controlled with [setPeriodic](#).

10.413.2 Member Enumeration Documentation

10.413.2.1 ColorInterpolation

```
enum QCPColorGradient::ColorInterpolation
```

Defines the color spaces in which color interpolation between gradient stops can be performed.

See also

[setColorInterpolation](#)

Enumerator

ciRGB	Color channels red, green and blue are linearly interpolated.
ciHSV	Color channels hue, saturation and value are linearly interpolated (The hue is interpolated over the shortest angle distance)

10.413.2.2 GradientPreset

```
enum QCPColorGradient::GradientPreset
```

Defines the available presets that can be loaded with [loadPreset](#). See the documentation there for an image of the presets.

Enumerator

gpGrayscale	Continuous lightness from black to white (suited for non-biased data representation)
gpHot	Continuous lightness from black over firey colors to white (suited for non-biased data representation)
gpCold	Continuous lightness from black over icy colors to white (suited for non-biased data representation)
gpNight	Continuous lightness from black over weak blueish colors to white (suited for non-biased data representation)
gpCandy	Blue over pink to white.
gpGeography	Colors suitable to represent different elevations on geographical maps.
gplon	Half hue spectrum from black over purple to blue and finally green (creates banding illusion but allows more precise magnitude estimates)
gpThermal	Colors suitable for thermal imaging, ranging from dark blue over purple to orange, yellow and white.
gpPolar	Colors suitable to emphasize polarity around the center, with blue for negative, black in the middle and red for positive values.
gpSpectrum	An approximation of the visible light spectrum (creates banding illusion but allows more precise magnitude estimates)
gpJet	Hue variation similar to a spectrum, often used in numerical visualization (creates banding illusion but allows more precise magnitude estimates)
gpHues	Full hue cycle, with highest and lowest color red (suitable for periodic data, such as angles and phases, see setPeriodic)

10.413.2.3 NanHandling

```
enum QCPColorGradient::NanHandling
```

Defines how NaN data points shall appear in the plot.

See also

[setNanHandling](#), [setNanColor](#)

Enumerator

nhNone	NaN data points are not explicitly handled and shouldn't occur in the data (this gives slight performance improvement)
nhLowestColor	NaN data points appear as the lowest color defined in this QCPColorGradient .
nhHighestColor	NaN data points appear as the highest color defined in this QCPColorGradient .
nhTransparent	NaN data points appear transparent.
nhNanColor	NaN data points appear as the color defined with setNanColor .

10.413.3 Constructor & Destructor Documentation

10.413.3.1 QCPColorGradient() [1/2]

```
QCPColorGradient::QCPColorGradient ( )
```

Constructs a new, empty [QCPColorGradient](#) with no predefined color stops. You can add own color stops with [setColorStopAt](#).

The color level count is initialized to 350.

10.413.3.2 QCPColorGradient() [2/2]

```
QCPColorGradient::QCPColorGradient (
    GradientPreset preset )
```

Constructs a new [QCPColorGradient](#) initialized with the colors and color interpolation according to *preset*.

The color level count is initialized to 350.

10.413.4 Member Function Documentation

10.413.4.1 clearColorStops()

```
void QCPColorGradient::clearColorStops ( )
```

Clears all color stops.

See also

[setColorStops](#), [setColorStopAt](#)

10.413.4.2 colorize() [1/2]

```
void QCPColorGradient::colorize (
    const double * data,
    const QCPRange & range,
    QRgb * scanLine,
    int n,
    int dataIndexFactor = 1,
    bool logarithmic = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

This method is used to quickly convert a *data* array to colors. The colors will be output in the array *scanLine*. Both *data* and *scanLine* must have the length *n* when passed to this function. The data range that shall be used for mapping the data value to the gradient is passed in *range*. *logarithmic* indicates whether the data values shall be mapped to colors logarithmically.

if *data* actually contains 2D-data linearized via `[row*columnCount + column]`, you can set *dataIndexFactor* to `columnCount` to convert a column instead of a row of the data array, in *scanLine*. *scanLine* will remain a regular (1D) array. This works because *data* is addressed `data[i*dataIndexFactor]`.

Use the overloaded method to additionally provide alpha map data.

The QRgb values that are placed in *scanLine* have their r, g, and b components premultiplied with alpha (see `QImage::Format_ARGB32_Premultiplied`).

10.413.4.3 `colorize()` [2/2]

```
void QCPColorGradient::colorize (
    const double * data,
    const unsigned char * alpha,
    const QCPRange & range,
    QRgb * scanLine,
    int n,
    int dataIndexFactor = 1,
    bool logarithmic = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Additionally to the other overload of `colorize`, this method takes the array *alpha*, which has the same size and structure as *data* and encodes the alpha information per data point.

The QRgb values that are placed in *scanLine* have their r, g and b components premultiplied with alpha (see `QImage::Format_ARGB32_Premultiplied`).

10.413.4.4 `inverted()`

```
QCPColorGradient QCPColorGradient::inverted ( ) const
```

Returns an inverted gradient. The inverted gradient has all properties as this `QCPColorGradient`, but the order of the color stops is inverted.

See also

[setColorStops](#), [setColorStopAt](#)

10.413.4.5 `loadPreset()`

```
void QCPColorGradient::loadPreset (
    GradientPreset preset )
```

Clears the current color stops and loads the specified *preset*. A preset consists of predefined color stops and the corresponding color interpolation method.

The available presets are:

10.413.4.6 `setColorInterpolation()`

```
void QCPColorGradient::setColorInterpolation (
    QCPColorGradient::ColorInterpolation interpolation )
```

Sets whether the colors in between the configured color stops (see [setColorStopAt](#)) shall be interpolated linearly in RGB or in HSV color space.

For example, a sweep in RGB space from red to green will have a muddy brown intermediate color, whereas in HSV space the intermediate color is yellow.

10.413.4.7 setColorStopAt()

```
void QCPColorGradient::setColorStopAt (
    double position,
    const QColor & color )
```

Sets the *color* the gradient will have at the specified *position* (from 0 to 1). In between these color stops, the color is interpolated according to [setColorInterpolation](#).

See also

[setColorStops](#), [clearColorStops](#)

10.413.4.8 setColorStops()

```
void QCPColorGradient::setColorStops (
    const QMap< double, QColor > & colorStops )
```

Sets at which positions from 0 to 1 which color shall occur. The positions are the keys, the colors are the values of the passed QMap *colorStops*. In between these color stops, the color is interpolated according to [setColorInterpolation](#).

A more convenient way to create a custom gradient may be to clear all color stops with [clearColorStops](#) (or creating a new, empty [QCPColorGradient](#)) and then adding them one by one with [setColorStopAt](#).

See also

[clearColorStops](#)

10.413.4.9 setLevelCount()

```
void QCPColorGradient::setLevelCount (
    int n )
```

Sets the number of discretization levels of the color gradient to *n*. The default is 350 which is typically enough to create a smooth appearance. The minimum number of levels is 2.

10.413.4.10 setNanColor()

```
void QCPColorGradient::setNanColor (
    const QColor & color )
```

Sets the color that NaN data is represented by, if [setNanHandling](#) is set to `ref nhNanColor`.

See also

[setNanHandling](#)

10.413.4.11 setNanHandling()

```
void QCPColorGradient::setNanHandling (
    QCPColorGradient::NanHandling handling )
```

Sets how NaNs in the data are displayed in the plot.

See also

[setNanColor](#)

10.413.4.12 setPeriodic()

```
void QCPColorGradient::setPeriodic (
    bool enabled )
```

Sets whether data points that are outside the configured data range (e.g. [QCPColorMap::setDataRange](#)) are colored by periodically repeating the color gradient or whether they all have the same color, corresponding to the respective gradient boundary color.

As shown in the image above, gradients that have the same start and end color are especially suitable for a periodic gradient mapping, since they produce smooth color transitions throughout the color map. A preset that has this property is [gpHues](#).

In practice, using periodic color gradients makes sense when the data corresponds to a periodic dimension, such as an angle or a phase. If this is not the case, the color encoding might become ambiguous, because multiple different data values are shown as the same color.

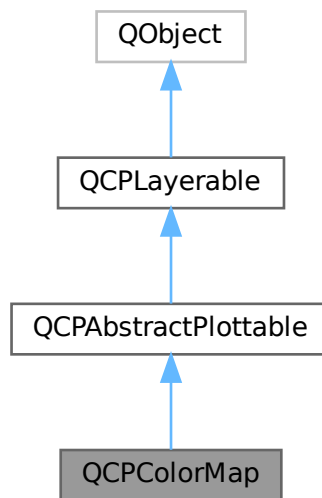
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

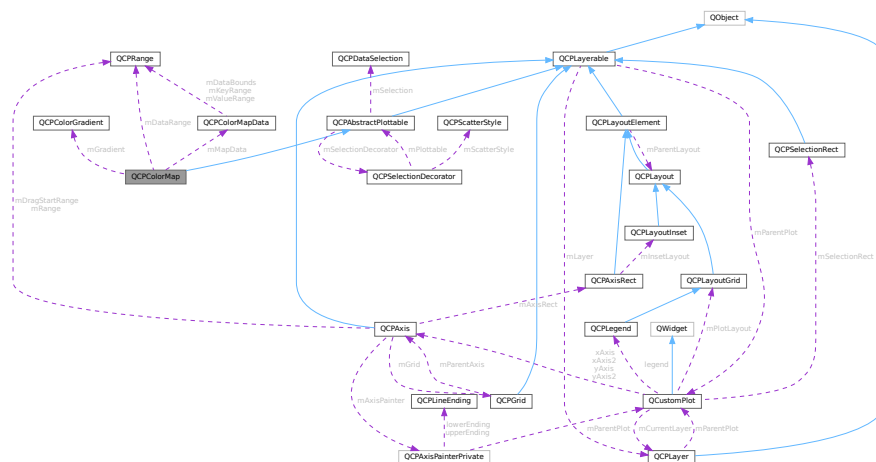
10.414 QCPColorMap Class Reference

A plottable representing a two-dimensional color map in a plot.

Inheritance diagram for QCPColorMap:



Collaboration diagram for QCPColorMap:



Signals

- void [dataRangeChanged](#) (const [QCPRange](#) &newRange)
- void [dataScaleTypeChanged](#) ([QCPAxis::ScaleType](#) scaleType)
- void [gradientChanged](#) (const [QCPColorGradient](#) &newGradient)

Signals inherited from [QCPAbstractPlottable](#)

- void [selectionChanged](#) (bool selected)
- void [selectionChanged](#) (const [QCPDataSelection](#) &selection)
- void [selectableChanged](#) ([QCP::SelectionType](#) selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

Public Member Functions

- [QCPColorMap](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QCPColorMapData](#) * [data](#) () const
- [QCPRange](#) [dataRange](#) () const
- [QCPAxis::ScaleType](#) [dataScaleType](#) () const
- bool [interpolate](#) () const
- bool [tightBoundary](#) () const
- [QCPColorGradient](#) [gradient](#) () const
- [QCPColorScale](#) * [colorScale](#) () const
- void [setData](#) ([QCPColorMapData](#) *data, bool copy=false)
- Q_SLOT void [setDataRange](#) (const [QCPRange](#) &dataRange)
- Q_SLOT void [setDataScaleType](#) ([QCPAxis::ScaleType](#) scaleType)
- Q_SLOT void [setGradient](#) (const [QCPColorGradient](#) &gradient)
- void [setInterpolate](#) (bool enabled)
- void [setTightBoundary](#) (bool enabled)
- void [setColorScale](#) ([QCPColorScale](#) *colorScale)
- void [rescaleDataRange](#) (bool recalculateDataBounds=false)
- Q_SLOT void [updateLegendIcon](#) (Qt::TransformationMode transformMode=Qt::SmoothTransformation, const QSize &thumbSize=QSize(32, 18))
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE
- virtual [QCPRange](#) [getKeyRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const Q_DECL_OVERRIDE
- virtual [QCPRange](#) [getValueRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- QString [name](#) () const
- bool [antialiasedFill](#) () const
- bool [antialiasedScatters](#) () const
- QPen [pen](#) () const
- QBrush [brush](#) () const
- [QCPAxis](#) * [keyAxis](#) () const
- [QCPAxis](#) * [valueAxis](#) () const
- [QCP::SelectionType](#) [selectable](#) () const
- bool [selected](#) () const
- [QCPDataSelection](#) [selection](#) () const
- [QCPSelectionDecorator](#) * [selectionDecorator](#) () const
- void [setName](#) (const QString &name)
- void [setAntialiasedFill](#) (bool enabled)
- void [setAntialiasedScatters](#) (bool enabled)
- void [setPen](#) (const QPen &pen)
- void [setBrush](#) (const QBrush &brush)
- void [setKeyAxis](#) ([QCPAxis](#) *axis)
- void [setValueAxis](#) ([QCPAxis](#) *axis)
- Q_SLOT void [setSelectable](#) ([QCP::SelectionType](#) selectable)

- Q_SLOT void [setSelection](#) ([QCPDataSelection](#) selection)
- void [setSelectionDecorator](#) ([QCPSelectionDecorator](#) *decorator)
- virtual [QCPPlootableInterface1D](#) * [interface1D](#) ()
- void [coordsToPixels](#) (double key, double value, double &x, double &y) const
- const QPointF [coordsToPixels](#) (double key, double value) const
- void [pixelsToCoords](#) (double x, double y, double &key, double &value) const
- void [pixelsToCoords](#) (const QPointF &pixelPos, double &key, double &value) const
- void [rescaleAxes](#) (bool onlyEnlarge=false) const
- void [rescaleKeyAxis](#) (bool onlyEnlarge=false) const
- void [rescaleValueAxis](#) (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool [addToLegend](#) ([QCPLegend](#) *legend)
- bool [addToLegend](#) ()
- bool [removeFromLegend](#) ([QCPLegend](#) *legend) const
- bool [removeFromLegend](#) () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void **updateMapImage** ()
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void [drawLegendIcon](#) ([QCPPainter](#) *painter, const QRectF &rect) const Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPAbstractPlottable](#)

- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- void [applyFillAntialiasingHint](#) ([QCPPainter](#) *painter) const
- void [applyScattersAntialiasingHint](#) ([QCPPainter](#) *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QCPRange](#) **mDataRange**
- [QCPAxis::ScaleType](#) **mDataScaleType**
- [QCPCoMapData](#) * **mMapData**
- [QCPCoMapGradient](#) **mGradient**
- bool **mInterpolate**
- bool **mTightBoundary**
- QPointer< [QCPCoMapScale](#) > **mColorScale**
- QImage **mMapImage**
- QImage **mUndersampledMapImage**
- QPixmap **mLegendIcon**
- bool **mMapImageInvalidated**

Protected Attributes inherited from [QCPAbstractPlottable](#)

- QString **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- QPen **mPen**
- QBrush **mBrush**
- QPointer< [QCPAxis](#) > **mKeyAxis**
- QPointer< [QCPAxis](#) > **mValueAxis**
- [QCP::SelectionType](#) **mSelectable**
- [QCPDataSelection](#) **mSelection**
- [QCPSelectionDecorator](#) * **mSelectionDecorator**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class **QCustomPlot**
- class **QCPLegend**

10.414.1 Detailed Description

A plottable representing a two-dimensional color map in a plot.

The data is stored in the class [QCPColorMapData](#), which can be accessed via the [data\(\)](#) method.

A color map has three dimensions to represent a data point: The *key* dimension, the *value* dimension and the *data* dimension. As with other plottables such as graphs, *key* and *value* correspond to two orthogonal axes on the [QCustomPlot](#) surface that you specify in the [QCPColorMap](#) constructor. The *data* dimension however is encoded as the color of the point at (*key*, *value*).

Set the number of points (or *cells*) in the key/value dimension via [QCPColorMapData::setSize](#). The plot coordinate range over which these points will be displayed is specified via [QCPColorMapData::setRange](#). The first cell will be centered on the lower range boundary and the last cell will be centered on the upper range boundary. The data can be set by either accessing the cells directly with [QCPColorMapData::setCell](#) or by addressing the cells via their plot coordinates with [QCPColorMapData::setData](#). If possible, you should prefer [setCell](#), since it doesn't need to do any coordinate transformation and thus performs a bit better.

The cell with index (0, 0) is at the bottom left, if the color map uses normal (i.e. not reversed) key and value axes.

To show the user which colors correspond to which *data* values, a [QCPColorScale](#) is typically placed to the right of the axis rect. See the documentation there for details on how to add and use a color scale.

10.414.2 Changing the appearance

Most important to the appearance is the color gradient, which can be specified via [setGradient](#). See the documentation of [QCPColorGradient](#) for details on configuring a color gradient.

The *data* range that is mapped to the colors of the gradient can be specified with [setDataRange](#). To make the data range encompass the whole data set minimum to maximum, call [rescaleDataRange](#). If your data may contain NaN values, use [QCPColorGradient::setNanHandling](#) to define how they are displayed.

10.414.3 Transparency

Transparency in color maps can be achieved by two mechanisms. On one hand, you can specify alpha values for color stops of the [QCPColorGradient](#), via the regular [QColor](#) interface. This will cause the color map data which gets mapped to colors around those color stops to appear with the accordingly interpolated transparency.

On the other hand you can also directly apply an alpha value to each cell independent of its data, by using the alpha map feature of [QCPColorMapData](#). The relevant methods are [QCPColorMapData::setAlpha](#), [QCPColorMapData::fillAlpha](#) and [QCPColorMapData::clearAlpha\(\)](#).

The two transparencies will be joined together in the plot and otherwise not interfere with each other. They are mixed in a multiplicative matter, so an alpha of e.g. 50% (128/255) in both modes simultaneously, will result in a total transparency of 25% (64/255).

10.414.4 Usage

Like all data representing objects in [QCustomPlot](#), the [QCPColorMap](#) is a plottable ([QCPAbstractPlottable](#)). So the plottable-interface of [QCustomPlot](#) applies ([QCustomPlot::plottable](#), [QCustomPlot::removePlottable](#), etc.)

Usually, you first create an instance:

which registers it with the [QCustomPlot](#) instance of the passed axes. Note that this [QCustomPlot](#) instance takes ownership of the plottable, so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead. The newly created plottable can be modified, e.g.:

Note

The [QCPColorMap](#) always displays the data at equal key/value intervals, even if the key or value axis is set to a logarithmic scaling. If you want to use [QCPColorMap](#) with logarithmic axes, you shouldn't use the [QCPColorMapData::setData](#) method as it uses a linear transformation to determine the cell index. Rather directly access the cell index with [QCPColorMapData::setCell](#).

10.414.5 Constructor & Destructor Documentation

10.414.5.1 QCPColorMap()

```
QCPColorMap::QCPColorMap (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis ) [explicit]
```

Constructs a color map with the specified *keyAxis* and *valueAxis*.

The created [QCPColorMap](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPColorMap](#), so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead.

10.414.6 Member Function Documentation

10.414.6.1 data()

```
QCPColorMapData * QCPColorMap::data ( ) const [inline]
```

Returns a pointer to the internal data storage of type [QCPColorMapData](#). Access this to modify data points (cells) and the color map key/value range.

See also

[setData](#)

10.414.6.2 dataRangeChanged

```
void QCPColorMap::dataRangeChanged (
    const QCPRange & newRange ) [signal]
```

This signal is emitted when the data range changes.

See also

[setDataRange](#)

10.414.6.3 dataScaleTypeChanged

```
void QCPColorMap::dataScaleTypeChanged (
    QCPAxis::ScaleType scaleType ) [signal]
```

This signal is emitted when the data scale type changes.

See also

[setDataScaleType](#)

10.414.6.4 draw()

```
void QCPColorMap::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

10.414.6.5 drawLegendIcon()

```
void QCPColorMap::drawLegendIcon (
    QCPPainter * painter,
    const QRectF & rect ) const [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

10.414.6.6 getKeyRange()

```
QCPRange QCPColorMap::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implements [QCPAbstractPlottable](#).

10.414.6.7 getValueRange()

```
QCPRange QCPCoLorMap::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implements [QCPAbstractPlottable](#).

10.414.6.8 gradientChanged

```
void QCPCoLorMap::gradientChanged (
    const QCPCoLorGradient & newGradient ) [signal]
```

This signal is emitted when the gradient changes.

See also

[setGradient](#)

10.414.6.9 rescaleDataRange()

```
void QCPCoLorMap::rescaleDataRange (
    bool recalculateDataBounds = false )
```

Sets the data range ([setDataRange](#)) to span the minimum and maximum values that occur in the current data set. This corresponds to the [rescaleKeyAxis](#) or [rescaleValueAxis](#) methods, only for the third data dimension of the color map.

The minimum and maximum values of the data set are buffered in the internal [QCPCoLorMapData](#) instance ([data](#)). As data is updated via its [QCPCoLorMapData::setCell](#) or [QCPCoLorMapData::setData](#), the buffered minimum and maximum values are updated, too. For performance reasons, however, they are only updated in an expanding fashion. So the buffered maximum can only increase and the buffered minimum can only decrease. In consequence, changes to the data that actually lower the maximum of the data set (by overwriting the cell holding the current maximum with a smaller value), aren't recognized and the buffered maximum overestimates the true maximum of the data set. The same happens for the buffered minimum. To recalculate the true minimum and maximum by explicitly looking at each cell, the method [QCPCoLorMapData::recalculateDataBounds](#) can be used. For convenience, setting the parameter *recalculateDataBounds* calls this method before setting the data range to the buffered minimum and maximum.

See also

[setDataRange](#)

10.414.6.10 selectTest()

```
double QCPColorMap::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the mousePressEvent occurs, and the finally selected object is notified via the selectEvent/ deselectEvent methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to selectEvent when the parent [QCustomPlot](#) decides on the basis of this selectTest call, that the object was successfully selected. The subsequent call to selectEvent will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent selectEvent, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

selectEvent, deselectEvent, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractPlottable](#).

10.414.6.11 setColorScale()

```
void QCPColorMap::setColorScale (
    QCPColorScale * colorScale )
```

Associates the color scale *colorScale* with this color map.

This means that both the color scale and the color map synchronize their gradient, data range and data scale type ([setGradient](#), [setDataRange](#), [setDataScaleType](#)). Multiple color maps can be associated with one single color scale. This causes the color maps to also synchronize those properties, via the mutual color scale.

This function causes the color map to adopt the current color gradient, data range and data scale type of *colorScale*. After this call, you may change these properties at either the color map or the color scale, and the setting will be applied to both.

Pass `nullptr` as *colorScale* to disconnect the color scale from this color map again.

10.414.6.12 setData()

```
void QCPColorMap::setData (
    QCPColorMapData * data,
    bool copy = false )
```

Replaces the current [data](#) with the provided *data*.

If *copy* is set to true, the *data* object will only be copied. if false, the color map takes ownership of the passed data and replaces the internal data pointer with it. This is significantly faster than copying for large datasets.

10.414.6.13 setDataRange()

```
void QCPColorMap::setDataRange (
    const QCPRange & dataRange )
```

Sets the data range of this color map to *dataRange*. The data range defines which data values are mapped to the color gradient.

To make the data range span the full range of the data set, use [rescaleDataRange](#).

See also

[QCPColorScale::setDataRange](#)

10.414.6.14 setDataScaleType()

```
void QCPColorMap::setDataScaleType (
    QCPAxis::ScaleType scaleType )
```

Sets whether the data is correlated with the color gradient linearly or logarithmically.

See also

[QCPColorScale::setDataScaleType](#)

10.414.6.15 setGradient()

```
void QCPColorMap::setGradient (
    const QCPColorGradient & gradient )
```

Sets the color gradient that is used to represent the data. For more details on how to create an own gradient or use one of the preset gradients, see [QCPColorGradient](#).

The colors defined by the gradient will be used to represent data values in the currently set data range, see [setDataRange](#). Data points that are outside this data range will either be colored uniformly with the respective gradient boundary color, or the gradient will repeat, depending on [QCPColorGradient::setPeriodic](#).

See also

[QCPColorScale::setGradient](#)

10.414.6.16 setInterpolate()

```
void QCPColorMap::setInterpolate (
    bool enabled )
```

Sets whether the color map image shall use bicubic interpolation when displaying the color map shrunk or expanded, and not at a 1:1 pixel-to-data scale.

10.414.6.17 setTightBoundary()

```
void QCPColorMap::setTightBoundary (
    bool enabled )
```

Sets whether the outer most data rows and columns are clipped to the specified key and value range (see [QCPColorMapData::setKeyRange](#), [QCPColorMapData::setValueRange](#)).

if *enabled* is set to false, the data points at the border of the color map are drawn with the same width and height as all other data points. Since the data points are represented by rectangles of one color centered on the data coordinate, this means that the shown color map extends by half a data point over the specified key/value range in each direction.

10.414.6.18 updateLegendIcon()

```
void QCPColorMap::updateLegendIcon (
    Qt::TransformationMode transformMode = Qt::SmoothTransformation,
    const QSize & thumbSize = QSize(32, 18) )
```

Takes the current appearance of the color map and updates the legend icon, which is used to represent this color map in the legend (see [QCPLegend](#)).

The *transformMode* specifies whether the rescaling is done by a faster, low quality image scaling algorithm (Qt::FastTransformation) or by a slower, higher quality algorithm (Qt::SmoothTransformation).

The current color map appearance is scaled down to *thumbSize*. Ideally, this should be equal to the size of the legend icon (see [QCPLegend::setIconSize](#)). If it isn't exactly the configured legend icon size, the thumb will be rescaled during drawing of the legend item.

See also

[setDataRange](#)

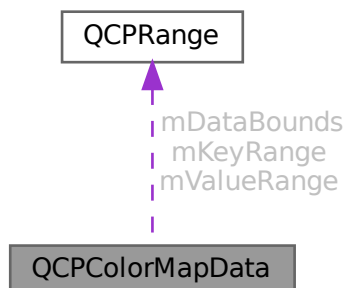
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.415 QCPColormapData Class Reference

Holds the two-dimensional data of a [QCPColormap](#) plottable.

Collaboration diagram for QCPColormapData:



Public Member Functions

- [QCPColormapData](#) (int keySize, int valueSize, const [QCPRange](#) &keyRange, const [QCPRange](#) &valueRange)
- [QCPColormapData](#) (const [QCPColormapData](#) &other)
- [QCPColormapData](#) & [operator=](#) (const [QCPColormapData](#) &other)
- int **keySize** () const
- int **valueSize** () const
- [QCPRange](#) **keyRange** () const
- [QCPRange](#) **valueRange** () const
- [QCPRange](#) **dataBounds** () const
- double **data** (double key, double value)
- double **cell** (int keyIndex, int valueIndex)
- unsigned char **alpha** (int keyIndex, int valueIndex)
- void **setSize** (int keySize, int valueSize)
- void **setKeySize** (int keySize)
- void **setValueSize** (int valueSize)
- void **setRange** (const [QCPRange](#) &keyRange, const [QCPRange](#) &valueRange)
- void **setKeyRange** (const [QCPRange](#) &keyRange)
- void **setValueRange** (const [QCPRange](#) &valueRange)
- void **setData** (double key, double value, double z)
- void **setCell** (int keyIndex, int valueIndex, double z)
- void **setAlpha** (int keyIndex, int valueIndex, unsigned char [alpha](#))
- void **recalculateDataBounds** ()
- void **clear** ()
- void **clearAlpha** ()
- void **fill** (double z)
- void **fillAlpha** (unsigned char [alpha](#))
- bool **isEmpty** () const
- void **coordToCell** (double key, double value, int *keyIndex, int *valueIndex) const
- void **cellToCoord** (int keyIndex, int valueIndex, double *key, double *value) const

Protected Member Functions

- bool **createAlpha** (bool initializeOpaque=true)

Protected Attributes

- int **mKeySize**
- int **mValueSize**
- [QCPRange](#) **mKeyRange**
- [QCPRange](#) **mValueRange**
- bool **mlsEmpty**
- double * **mData**
- unsigned char * **mAlpha**
- [QCPRange](#) **mDataBounds**
- bool **mDataModified**

Friends

- class **QCPCColorMap**

10.415.1 Detailed Description

Holds the two-dimensional data of a [QCPCColorMap](#) plottable.

This class is a data storage for [QCPCColorMap](#). It holds a two-dimensional array, which [QCPCColorMap](#) then displays as a 2D image in the plot, where the array values are represented by a color, depending on the value.

The size of the array can be controlled via [setSize](#) (or [setKeySize](#), [setValueSize](#)). Which plot coordinates these cells correspond to can be configured with [setRange](#) (or [setKeyRange](#), [setValueRange](#)).

The data cells can be accessed in two ways: They can be directly addressed by an integer index with [setCell](#). This is the fastest method. Alternatively, they can be addressed by their plot coordinate with [setData](#). plot coordinate to cell index transformations and vice versa are provided by the functions [coordToCell](#) and [cellToCoord](#).

A [QCPCColorMapData](#) also holds an on-demand two-dimensional array of alpha values which (if allocated) has the same size as the data map. It can be accessed via [setAlpha](#), [fillAlpha](#) and [clearAlpha](#). The memory for the alpha map is only allocated if needed, i.e. on the first call of [setAlpha](#). [clearAlpha](#) restores full opacity and frees the alpha map.

This class also buffers the minimum and maximum values that are in the data set, to provide [QCPCColorMap::rescaleDataRange](#) with the necessary information quickly. Setting a cell to a value that is greater than the current maximum increases this maximum to the new value. However, setting the cell that currently holds the maximum value to a smaller value doesn't decrease the maximum again, because finding the true new maximum would require going through the entire data array, which might be time consuming. The same holds for the data minimum. This functionality is given by [recalculateDataBounds](#), such that you can decide when it is sensible to find the true current minimum and maximum. The method [QCPCColorMap::rescaleDataRange](#) offers a convenience parameter *recalculateDataBounds* which may be set to true to automatically call [recalculateDataBounds](#) internally.

10.415.2 Constructor & Destructor Documentation

10.415.2.1 QCPCColorMapData() [1/2]

```
QCPCColorMapData::QCPCColorMapData (
    int keySize,
    int valueSize,
    const QCPCRange & keyRange,
    const QCPCRange & valueRange )
```

Constructs a new [QCPCColorMapData](#) instance. The instance has *keySize* cells in the key direction and *valueSize* cells in the value direction. These cells will be displayed by the [QCPCColorMap](#) at the coordinates *keyRange* and *valueRange*.

See also

[setSize](#), [setKeySize](#), [setValueSize](#), [setRange](#), [setKeyRange](#), [setValueRange](#)

10.415.2.2 QCPCColorMapData() [2/2]

```
QCPCColorMapData::QCPCColorMapData (
    const QCPCColorMapData & other )
```

Constructs a new [QCPCColorMapData](#) instance copying the data and range of *other*.

10.415.3 Member Function Documentation

10.415.3.1 alpha()

```
unsigned char QCPCColorMapData::alpha (
    int keyIndex,
    int valueIndex )
```

Returns the alpha map value of the cell with the indices *keyIndex* and *valueIndex*.

If this color map data doesn't have an alpha map (because [setAlpha](#) was never called after creation or after a call to [clearAlpha](#)), returns 255, which corresponds to full opacity.

See also

[setAlpha](#)

10.415.3.2 cellToCoord()

```
void QCPCoMapData::cellToCoord (
    int keyIndex,
    int valueIndex,
    double * key,
    double * value ) const
```

Transforms cell indices given by *keyIndex* and *valueIndex* to cell indices of this [QCPCoMapData](#) instance. The resulting coordinates are returned via the output parameters *key* and *value*.

If you are only interested in a key or value coordinate, you may pass `nullptr` as *key* or *value*.

Note

The [QCPCoMap](#) always displays the data at equal key/value intervals, even if the key or value axis is set to a logarithmic scaling. If you want to use [QCPCoMap](#) with logarithmic axes, you shouldn't use the [QCPCoMapData::cellToCoord](#) method as it uses a linear transformation to determine the cell index.

See also

[coordToCell](#), [QCPCoMap::pixelToCoord](#)

10.415.3.3 clear()

```
void QCPCoMapData::clear ( )
```

Frees the internal data memory.

This is equivalent to calling [setSize\(0, 0\)](#).

10.415.3.4 clearAlpha()

```
void QCPCoMapData::clearAlpha ( )
```

Frees the internal alpha map. The color map will have full opacity again.

10.415.3.5 coordToCell()

```
void QCPCoMapData::coordToCell (
    double key,
    double value,
    int * keyIndex,
    int * valueIndex ) const
```

Transforms plot coordinates given by *key* and *value* to cell indices of this [QCPCoMapData](#) instance. The resulting cell indices are returned via the output parameters *keyIndex* and *valueIndex*.

The retrieved key/value cell indices can then be used for example with [setCell](#).

If you are only interested in a key or value index, you may pass `nullptr` as *valueIndex* or *keyIndex*.

Note

The [QCPCoMap](#) always displays the data at equal key/value intervals, even if the key or value axis is set to a logarithmic scaling. If you want to use [QCPCoMap](#) with logarithmic axes, you shouldn't use the [QCPCoMapData::coordToCell](#) method as it uses a linear transformation to determine the cell index.

See also

[cellToCoord](#), [QCPCoMap::coordToPixel](#)

10.415.3.6 fill()

```
void QCPColormapData::fill (
    double z )
```

Sets all cells to the value *z*.

10.415.3.7 fillAlpha()

```
void QCPColormapData::fillAlpha (
    unsigned char alpha )
```

Sets the opacity of all color map cells to *alpha*. A value of 0 for *alpha* results in a fully transparent color map, and a value of 255 results in a fully opaque color map.

If you wish to restore opacity to 100% and free any used memory for the alpha map, rather use [clearAlpha](#).

See also

[setAlpha](#)

10.415.3.8 isEmpty()

```
bool QCPColormapData::isEmpty ( ) const [inline]
```

Returns whether this instance carries no data. This is equivalent to having a size where at least one of the dimensions is 0 (see [setSize](#)).

10.415.3.9 operator=()

```
QCPColormapData & QCPColormapData::operator= (
    const QCPColormapData & other )
```

Overwrites this color map data instance with the data stored in *other*. The alpha map state is transferred, too.

10.415.3.10 recalculateDataBounds()

```
void QCPColormapData::recalculateDataBounds ( )
```

Goes through the data and updates the buffered minimum and maximum data values.

Calling this method is only advised if you are about to call [QCPColormap::rescaleDataRange](#) and can not guarantee that the cells holding the maximum or minimum data haven't been overwritten with a smaller or larger value respectively, since the buffered maximum/minimum values have been updated the last time. Why this is the case is explained in the class description ([QCPColormapData](#)).

Note that the method [QCPColormap::rescaleDataRange](#) provides a parameter *recalculateDataBounds* for convenience. Setting this to true will call this method for you, before doing the rescale.

10.415.3.11 setAlpha()

```
void QCPColorMapData::setAlpha (
    int keyIndex,
    int valueIndex,
    unsigned char alpha )
```

Sets the alpha of the color map cell given by *keyIndex* and *valueIndex* to *alpha*. A value of 0 for *alpha* results in a fully transparent cell, and a value of 255 results in a fully opaque cell.

If an alpha map doesn't exist yet for this color map data, it will be created here. If you wish to restore full opacity and free any allocated memory of the alpha map, call [clearAlpha](#).

Note that the cell-wise alpha which can be configured here is independent of any alpha configured in the color map's gradient ([QCPColorGradient](#)). If a cell is affected both by the cell-wise and gradient alpha, the alpha values will be blended accordingly during rendering of the color map.

See also

[fillAlpha](#), [clearAlpha](#)

10.415.3.12 setCell()

```
void QCPColorMapData::setCell (
    int keyIndex,
    int valueIndex,
    double z )
```

Sets the data of the cell with indices *keyIndex* and *valueIndex* to *z*. The indices enumerate the cells starting from zero, up to the map's size-1 in the respective dimension (see [setSize](#)).

In the standard plot configuration (horizontal key axis and vertical value axis, both not range-reversed), the cell with indices (0, 0) is in the bottom left corner and the cell with indices (keySize-1, valueSize-1) is in the top right corner of the color map.

See also

[setData](#), [setSize](#)

10.415.3.13 setData()

```
void QCPColorMapData::setData (
    double key,
    double value,
    double z )
```

Sets the data of the cell, which lies at the plot coordinates given by *key* and *value*, to *z*.

Note

The [QCPColorMap](#) always displays the data at equal key/value intervals, even if the key or value axis is set to a logarithmic scaling. If you want to use [QCPColorMap](#) with logarithmic axes, you shouldn't use the [QCPColorMapData::setData](#) method as it uses a linear transformation to determine the cell index. Rather directly access the cell index with [QCPColorMapData::setCell](#).

See also

[setCell](#), [setRange](#)

10.415.3.14 setKeyRange()

```
void QCPColorMapData::setKeyRange (
    const QCPRange & keyRange )
```

Sets the coordinate range the data shall be distributed over in the key dimension. Together with the value range, This defines the rectangular area covered by the color map in plot coordinates.

The outer cells will be centered on the range boundaries given to this function. For example, if the key size ([setKeySize](#)) is 3 and *keyRange* is set to [QCPRange \(2, 3\)](#) there will be cells centered on the key coordinates 2, 2.5 and 3.

See also

[setRange](#), [setValueRange](#), [setSize](#)

10.415.3.15 setKeySize()

```
void QCPColorMapData::setKeySize (
    int keySize )
```

Resizes the data array to have *keySize* cells in the key dimension.

The current data is discarded and the map cells are set to 0, unless the map had already the requested size.

Setting *keySize* to zero frees the internal data array and [isEmpty](#) returns true.

See also

[setKeyRange](#), [setSize](#), [setValueSize](#)

10.415.3.16 setRange()

```
void QCPColorMapData::setRange (
    const QCPRange & keyRange,
    const QCPRange & valueRange )
```

Sets the coordinate ranges the data shall be distributed over. This defines the rectangular area covered by the color map in plot coordinates.

The outer cells will be centered on the range boundaries given to this function. For example, if the key size ([setKeySize](#)) is 3 and *keyRange* is set to [QCPRange \(2, 3\)](#) there will be cells centered on the key coordinates 2, 2.5 and 3.

See also

[setSize](#)

10.415.3.17 setSize()

```
void QCPColormapData::setSize (
    int keySize,
    int valueSize )
```

Resizes the data array to have *keySize* cells in the key dimension and *valueSize* cells in the value dimension.

The current data is discarded and the map cells are set to 0, unless the map had already the requested size.

Setting at least one of *keySize* or *valueSize* to zero frees the internal data array and [isEmpty](#) returns true.

See also

[setRange](#), [setKeySize](#), [setValueSize](#)

10.415.3.18 setValueRange()

```
void QCPColormapData::setValueRange (
    const QCPRange & valueRange )
```

Sets the coordinate range the data shall be distributed over in the value dimension. Together with the key range, This defines the rectangular area covered by the color map in plot coordinates.

The outer cells will be centered on the range boundaries given to this function. For example, if the value size ([setValueSize](#)) is 3 and *valueRange* is set to [QCPRange](#) (2, 3) there will be cells centered on the value coordinates 2, 2.5 and 3.

See also

[setRange](#), [setKeyRange](#), [setSize](#)

10.415.3.19 setValueSize()

```
void QCPColormapData::setValueSize (
    int valueSize )
```

Resizes the data array to have *valueSize* cells in the value dimension.

The current data is discarded and the map cells are set to 0, unless the map had already the requested size.

Setting *valueSize* to zero frees the internal data array and [isEmpty](#) returns true.

See also

[setValueRange](#), [setSize](#), [setKeySize](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

Signals

- void [dataRangeChanged](#) (const [QCPRange](#) &newRange)
- void [dataScaleTypeChanged](#) ([QCPAxis::ScaleType](#) scaleType)
- void [gradientChanged](#) (const [QCPColorGradient](#) &newGradient)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

Public Member Functions

- [QCPColorScale](#) ([QCustomPlot](#) *parentPlot)
- [QCPAxis](#) * [axis](#) () const
- [QCPAxis::AxisType](#) [type](#) () const
- [QCPRange](#) [dataRange](#) () const
- [QCPAxis::ScaleType](#) [dataScaleType](#) () const
- [QCPColorGradient](#) [gradient](#) () const
- [QString](#) [label](#) () const
- int [barWidth](#) () const
- bool [rangeDrag](#) () const
- bool [rangeZoom](#) () const
- void [setType](#) ([QCPAxis::AxisType](#) type)
- Q_SLOT void [setDataRange](#) (const [QCPRange](#) &dataRange)
- Q_SLOT void [setDataScaleType](#) ([QCPAxis::ScaleType](#) scaleType)
- Q_SLOT void [setGradient](#) (const [QCPColorGradient](#) &gradient)
- void [setLabel](#) (const [QString](#) &str)
- void [setBarWidth](#) (int width)
- void [setRangeDrag](#) (bool enabled)
- void [setRangeZoom](#) (bool enabled)
- [QList< QCPColorMap * >](#) [colorMaps](#) () const
- void [rescaleDataRange](#) (bool onlyVisibleMaps)
- virtual void [update](#) ([UpdatePhase](#) phase) Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPLayoutElement](#)

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- [QRect](#) [rect](#) () const
- [QRect](#) [outerRect](#) () const
- [QMargins](#) [margins](#) () const
- [QMargins](#) [minimumMargins](#) () const
- [QCP::MarginSides](#) [autoMargins](#) () const
- [QSize](#) [minimumSize](#) () const
- [QSize](#) [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- [QHash< QCP::MarginSide, QCPMarginGroup * >](#) [marginGroups](#) () const
- void [setOuterRect](#) (const [QRect](#) &rect)
- void [setMargins](#) (const [QMargins](#) &margins)
- void [setMinimumMargins](#) (const [QMargins](#) &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)

- void [setMinimumSize](#) (const QSize &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const QSize &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) (QCP::MarginSides sides, [QCPLayerable](#) *group)
- virtual QSize [minimumOuterSizeHint](#) () const
- virtual QSize [maximumOuterSizeHint](#) () const
- virtual QList< [QCPLayerable](#) * > [elements](#) (bool recursive) const
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE
- virtual void [wheelEvent](#) (QWheelEvent *event) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QCPAxis::AxisType](#) **mType**
- [QCPRange](#) **mDataRange**
- [QCPAxis::ScaleType](#) **mDataScaleType**
- [QCPColorGradient](#) **mGradient**
- **int mBarWidth**
- [QPointer< QCPColorScaleAxisRectPrivate >](#) **mAxisRect**
- [QPointer< QCPAxis >](#) **mColorAxis**

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- **QSize mMinimumSize**
- **QSize mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- **QRect mRect**
- **QRect mOuterRect**
- **QMargins mMargins**
- **QMargins mMinimumMargins**
- [QCP::MarginSides](#) **mAutoMargins**
- [QHash< QCP::MarginSide, QCPMarginGroup * >](#) **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- **bool mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer< QCPLayerable >](#) **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- **bool mAntialiased**

Friends

- class [QCPColorScaleAxisRectPrivate](#)

Additional Inherited Members

Public Types inherited from [QCPLayoutElement](#)

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

10.416.1 Detailed Description

A color scale for use with color coding data such as [QCPCColorMap](#).

This layout element can be placed on the plot to correlate a color gradient with data values. It is usually used in combination with one or multiple [QCPCColorMaps](#).

The color scale can be either horizontal or vertical, as shown in the image above. The orientation and the side where the numbers appear is controlled with [setType](#).

Use [QCPCColorMap::setColorScale](#) to connect a color map with a color scale. Once they are connected, they share their gradient, data range and data scale type ([setGradient](#), [setDataRange](#), [setDataScaleType](#)). Multiple color maps may be associated with a single color scale, to make them all synchronize these properties.

To have finer control over the number display and axis behaviour, you can directly access the [axis](#). See the documentation of [QCPAxis](#) for details about configuring axes. For example, if you want to change the number of automatically generated ticks, call

Placing a color scale next to the main axis rect works like with any other layout element:

In this case we have placed it to the right of the default axis rect, so it wasn't necessary to call [setType](#), since [QCPAxis::atRight](#) is already the default. The text next to the color scale can be set with [setLabel](#).

For optimum appearance (like in the image above), it may be desirable to line up the axis rect and the borders of the color scale. Use a [QCPMarginGroup](#) to achieve this:

Color scales are initialized with a non-zero minimum top and bottom margin ([setMinimumMargins](#)), because vertical color scales are most common and the minimum top/bottom margin makes sure it keeps some distance to the top/bottom widget border. So if you change to a horizontal color scale by setting [setType](#) to [QCPAxis::atBottom](#) or [QCPAxis::atTop](#), you might want to also change the minimum margins accordingly, e.g. `setMinimumMargins(QMargins(6, 0, 6, 0))`.

10.416.2 Constructor & Destructor Documentation

10.416.2.1 QCPCColorScale()

```
QCPCColorScale::QCPCColorScale (
    QCustomPlot * parentPlot ) [explicit]
```

Constructs a new [QCPCColorScale](#).

10.416.3 Member Function Documentation

10.416.3.1 applyDefaultAntialiasingHint()

```
void QCPCColorScale::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

10.416.3.2 axis()

```
QCPAxis * QCPColorScale::axis ( ) const [inline]
```

Returns the internal [QCPAxis](#) instance of this color scale. You can access it to alter the appearance and behaviour of the axis. [QCPColorScale](#) duplicates some properties in its interface for convenience. Those are [setDataRange](#) ([QCPAxis::setRange](#)), [setDataScaleType](#) ([QCPAxis::setScaleType](#)), and the method [setLabel](#) ([QCPAxis::setLabel](#)). As they each are connected, it does not matter whether you use the method on the [QCPColorScale](#) or on its [QCPAxis](#).

If the type of the color scale is changed with [setType](#), the axis returned by this method will change, too, to either the left, right, bottom or top axis, depending on which type was set.

10.416.3.3 colorMaps()

```
QList< QCPColorMap * > QCPColorScale::colorMaps ( ) const
```

Returns a list of all the color maps associated with this color scale.

10.416.3.4 dataRangeChanged

```
void QCPColorScale::dataRangeChanged (
    const QCPRange & newRange ) [signal]
```

This signal is emitted when the data range changes.

See also

[setDataRange](#)

10.416.3.5 dataScaleTypeChanged

```
void QCPColorScale::dataScaleTypeChanged (
    QCPAxis::ScaleType scaleType ) [signal]
```

This signal is emitted when the data scale type changes.

See also

[setDataScaleType](#)

10.416.3.6 gradientChanged

```
void QCPColorScale::gradientChanged (
    const QCPColorGradient & newGradient ) [signal]
```

This signal is emitted when the gradient changes.

See also

[setGradient](#)

10.416.3.7 mouseMoveEvent()

```
void QCPColorScale::mouseMoveEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user moves the mouse while holding a mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

10.416.3.8 mousePressEvent()

```
void QCPColorScale::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses a mouse button while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *details* contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the *details* parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

[QCustomPlot](#) uses an event propagation system that works the same as Qt's system. If your layerable doesn't reimplement the [mousePressEvent](#) or explicitly calls `event->ignore()` in its reimplementation, the event will be propagated to the next layerable in the stacking order.

Once a layerable has accepted the [mousePressEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) or [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

10.416.3.9 mouseReleaseEvent()

```
void QCPColorScale::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user releases the mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter `startPos` indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

10.416.3.10 rescaleDataRange()

```
void QCPColorScale::rescaleDataRange (
    bool onlyVisibleMaps )
```

Changes the data range such that all color maps associated with this color scale are fully mapped to the gradient in the data dimension.

See also

[setDataRange](#)

10.416.3.11 setBarWidth()

```
void QCPColorScale::setBarWidth (
    int width )
```

Sets the width (or height, for horizontal color scales) the bar where the gradient is displayed will have.

10.416.3.12 setDataRange()

```
void QCPColorScale::setDataRange (
    const QCPRange & dataRange )
```

Sets the range spanned by the color gradient and that is shown by the axis in the color scale.

It is equivalent to calling [QCPColorMap::setDataRange](#) on any of the connected color maps. It is also equivalent to directly accessing the [axis](#) and setting its range with [QCPAxis::setRange](#).

See also

[setDataScaleType](#), [setGradient](#), [rescaleDataRange](#)

10.416.3.13 setDataScaleType()

```
void QCPColorScale::setDataScaleType (
    QCPAxis::ScaleType scaleType )
```

Sets the scale type of the color scale, i.e. whether values are associated with colors linearly or logarithmically.

It is equivalent to calling [QCPColorMap::setDataScaleType](#) on any of the connected color maps. It is also equivalent to directly accessing the [axis](#) and setting its scale type with [QCPAxis::setScaleType](#).

Note that this method controls the coordinate transformation. For logarithmic scales, you will likely also want to use a logarithmic tick spacing and labeling, which can be achieved by setting the color scale's [axis](#) ticker to an instance of [QCPAxisTickerLog](#) :

See the documentation of [QCPAxisTickerLog](#) about the details of logarithmic axis tick creation.

See also

[setDataRange](#), [setGradient](#)

10.416.3.14 setGradient()

```
void QCPColorScale::setGradient (
    const QCPColorGradient & gradient )
```

Sets the color gradient that will be used to represent data values.

It is equivalent to calling [QCPColorMap::setGradient](#) on any of the connected color maps.

See also

[setDataRange](#), [setDataScaleType](#)

10.416.3.15 setLabel()

```
void QCPColorScale::setLabel (
    const QString & str )
```

Sets the axis label of the color scale. This is equivalent to calling [QCPAxis::setLabel](#) on the internal [axis](#).

10.416.3.16 setRangeDrag()

```
void QCPColorScale::setRangeDrag (
    bool enabled )
```

Sets whether the user can drag the data range ([setDataRange](#)).

Note that [QCP::iRangeDrag](#) must be in the [QCustomPlot](#)'s interactions ([QCustomPlot::setInteractions](#)) to allow range dragging.

10.416.3.17 `setRangeZoom()`

```
void QCPColorScale::setRangeZoom (
    bool enabled )
```

Sets whether the user can zoom the data range ([setDataRange](#)) by scrolling the mouse wheel.

Note that [QCP::iRangeZoom](#) must be in the [QCustomPlot](#)'s interactions ([QCustomPlot::setInteractions](#)) to allow range dragging.

10.416.3.18 `setType()`

```
void QCPColorScale::setType (
    QCPAxis::AxisType type )
```

Sets at which side of the color scale the axis is placed, and thus also its orientation.

Note that after setting *type* to a different value, the axis returned by [axis\(\)](#) will be a different one. The new axis will adopt the following properties from the previous axis: The range, scale type, label and ticker (the latter will be shared and not copied).

10.416.3.19 `update()`

```
void QCPColorScale::update (
    UpdatePhase phase ) [virtual]
```

Updates the layout element and sub-elements. This function is automatically called before every replot by the parent layout element. It is called multiple times, once for every [UpdatePhase](#). The phases are run through in the order of the enum values. For details about what happens at the different phases, see the documentation of [UpdatePhase](#).

Layout elements that have child elements should call the [update](#) method of their child elements, and pass the current *phase* unchanged.

The default implementation executes the automatic margin mechanism in the [upMargins](#) phase. Subclasses should make sure to call the base class implementation.

Reimplemented from [QCPLayoutElement](#).

10.416.3.20 `wheelEvent()`

```
void QCPColorScale::wheelEvent (
    QWheelEvent * event ) [protected], [virtual]
```

This event gets called when the user turns the mouse scroll wheel while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`.

The `event->angleDelta()` indicates how far the mouse wheel was turned, which is usually +/- 120 for single rotation steps. However, if the mouse wheel is turned rapidly, multiple steps may accumulate to one event, making the delta larger. On the other hand, if the wheel has very smooth steps or none at all, the delta may be smaller.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#)

Reimplemented from [QCPLayerable](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

Inheritance diagram for QCPColorScaleAxisRectPrivate:



- Generated by Doxygen

Public Member Functions inherited from [QCPAxisRect](#)

- [QCPAxisRect](#) ([QCustomPlot](#) *parentPlot, bool setupDefaultAxes=true)
- [QPixmap](#) **background** () const
- [QBrush](#) **backgroundBrush** () const
- bool **backgroundScaled** () const
- [Qt::AspectRatioMode](#) **backgroundScaledMode** () const
- [Qt::Orientations](#) **rangeDrag** () const
- [Qt::Orientations](#) **rangeZoom** () const
- [QCPAxis](#) * **rangeDragAxis** ([Qt::Orientation](#) orientation)
- [QCPAxis](#) * **rangeZoomAxis** ([Qt::Orientation](#) orientation)
- [QList](#)< [QCPAxis](#) * > **rangeDragAxes** ([Qt::Orientation](#) orientation)
- [QList](#)< [QCPAxis](#) * > **rangeZoomAxes** ([Qt::Orientation](#) orientation)
- double **rangeZoomFactor** ([Qt::Orientation](#) orientation)
- void **setBackground** (const [QPixmap](#) &pm)
- void **setBackground** (const [QPixmap](#) &pm, bool scaled, [Qt::AspectRatioMode](#) mode=[Qt::KeepAspectRatio](#)↔
ByExpanding)
- void **setBackground** (const [QBrush](#) &brush)
- void **setBackgroundScaled** (bool scaled)
- void **setBackgroundScaledMode** ([Qt::AspectRatioMode](#) mode)
- void **setRangeDrag** ([Qt::Orientations](#) orientations)
- void **setRangeZoom** ([Qt::Orientations](#) orientations)
- void **setRangeDragAxes** ([QCPAxis](#) *horizontal, [QCPAxis](#) *vertical)
- void **setRangeDragAxes** ([QList](#)< [QCPAxis](#) * > axes)
- void **setRangeDragAxes** ([QList](#)< [QCPAxis](#) * > horizontal, [QList](#)< [QCPAxis](#) * > vertical)
- void **setRangeZoomAxes** ([QCPAxis](#) *horizontal, [QCPAxis](#) *vertical)
- void **setRangeZoomAxes** ([QList](#)< [QCPAxis](#) * > axes)
- void **setRangeZoomAxes** ([QList](#)< [QCPAxis](#) * > horizontal, [QList](#)< [QCPAxis](#) * > vertical)
- void **setRangeZoomFactor** (double horizontalFactor, double verticalFactor)
- void **setRangeZoomFactor** (double factor)
- int **axisCount** ([QCPAxis::AxisType](#) type) const
- [QCPAxis](#) * **axis** ([QCPAxis::AxisType](#) type, int index=0) const
- [QList](#)< [QCPAxis](#) * > **axes** ([QCPAxis::AxisTypes](#) types) const
- [QList](#)< [QCPAxis](#) * > **axes** () const
- [QCPAxis](#) * **addAxis** ([QCPAxis::AxisType](#) type, [QCPAxis](#) *axis=nullptr)
- [QList](#)< [QCPAxis](#) * > **addAxes** ([QCPAxis::AxisTypes](#) types)
- bool **removeAxis** ([QCPAxis](#) *axis)
- [QCPLayoutInset](#) * **insetLayout** () const
- void **zoom** (const [QRectF](#) &pixelRect)
- void **zoom** (const [QRectF](#) &pixelRect, const [QList](#)< [QCPAxis](#) * > &affectedAxes)
- void **setupFullAxesBox** (bool connectRanges=false)
- [QList](#)< [QCPAbstractPlottable](#) * > **plottables** () const
- [QList](#)< [QCPGraph](#) * > **graphs** () const
- [QList](#)< [QCPAbstractItem](#) * > **items** () const
- int **left** () const
- int **right** () const
- int **top** () const
- int **bottom** () const
- int **width** () const
- int **height** () const
- [QSize](#) **size** () const
- [QPoint](#) **topLeft** () const
- [QPoint](#) **topRight** () const
- [QPoint](#) **bottomLeft** () const
- [QPoint](#) **bottomRight** () const
- [QPoint](#) **center** () const
- virtual [QList](#)< [QCPLayoutElement](#) * > **elements** (bool recursive) const [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from QCPLayouElement

- QCPLayouElement (QCustomPlot *parentPlot=nullptr)
- QCPLayou * layout () const
- QRect rect () const
- QRect outerRect () const
- QMargins margins () const
- QMargins minimumMargins () const
- QCP::MarginSides autoMargins () const
- QSize minimumSize () const
- QSize maximumSize () const
- SizeConstraintRect sizeConstraintRect () const
- QCPLMarginGroup * marginGroup (QCP::MarginSide side) const
- QHash< QCP::MarginSide, QCPLMarginGroup * > marginGroups () const
- void setOuterRect (const QRect &rect)
- void setMargins (const QMargins &margins)
- void setMinimumMargins (const QMargins &margins)
- void setAutoMargins (QCP::MarginSides sides)
- void setMinimumSize (const QSize &size)
- void setMinimumSize (int width, int height)
- void setMaximumSize (const QSize &size)
- void setMaximumSize (int width, int height)
- void setSizeConstraintRect (SizeConstraintRect constraintRect)
- void setMarginGroup (QCP::MarginSides sides, QCPLMarginGroup *group)
- virtual QSize minimumOuterSizeHint () const
- virtual QSize maximumOuterSizeHint () const
- virtual double selectTest (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE

Public Member Functions inherited from QCPLayerable

- QCPLayerable (QCustomPlot *plot, QString targetLayer=QString(), QCPLayerable *parentLayerable=nullptr)
- bool visible () const
- QCustomPlot * parentPlot () const
- QCPLayerable * parentLayerable () const
- QCPLayer * layer () const
- bool antialiased () const
- void setVisible (bool on)
- Q_SLOT bool setLayer (QCPLayer *layer)
- bool setLayer (const QString &layerName)
- void setAntialiased (bool enabled)
- bool realVisibility () const

Protected Member Functions

- virtual void draw (QCPPainter *painter) Q_DECL_OVERRIDE
- void updateGradientImage ()
- Q_SLOT void axisSelectionChanged (QCPAxis::SelectableParts selectedParts)
- Q_SLOT void axisSelectableChanged (QCPAxis::SelectableParts selectableParts)
- virtual int calculateAutoMargin (QCP::MarginSide side) Q_DECL_OVERRIDE
- virtual void mousePressEvent (QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE
- virtual void mouseMoveEvent (QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE
- virtual void mouseReleaseEvent (QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE
- virtual void wheelEvent (QWheelEvent *event) Q_DECL_OVERRIDE
- virtual void update (UpdatePhase phase) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPAxisRect](#)

- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [layoutChanged](#) () Q_DECL_OVERRIDE
- void [drawBackground](#) ([QCPPainter](#) *painter)
- void [updateAxesOffset](#) ([QCPAxis::AxisType](#) type)

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QCP::Interaction](#) [selectionCategory](#) () const
- virtual [QRect](#) [clipRect](#) () const
- virtual void [selectEvent](#) ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState↔ Changed)
- virtual void [deselectEvent](#) (bool *selectionStateChanged)
- virtual void [mouseDoubleClickEvent](#) ([QMouseEvent](#) *event, const [QVariant](#) &details)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QCPColorScale](#) * [mParentColorScale](#)
- [QImage](#) [mGradientImage](#)
- bool [mGradientImageInvalidated](#)

Protected Attributes inherited from [QCPAxisRect](#)

- [QBrush](#) [mBackgroundBrush](#)
- [QPixmap](#) [mBackgroundPixmap](#)
- [QPixmap](#) [mScaledBackgroundPixmap](#)
- bool [mBackgroundScaled](#)
- [Qt::AspectRatioMode](#) [mBackgroundScaledMode](#)
- [QCPLayoutInset](#) * [mInsetLayout](#)
- [Qt::Orientations](#) [mRangeDrag](#)
- [Qt::Orientations](#) [mRangeZoom](#)
- [QList](#)< [QPointer](#)< [QCPAxis](#) > > [mRangeDragHorzAxis](#)
- [QList](#)< [QPointer](#)< [QCPAxis](#) > > [mRangeDragVertAxis](#)
- [QList](#)< [QPointer](#)< [QCPAxis](#) > > [mRangeZoomHorzAxis](#)
- [QList](#)< [QPointer](#)< [QCPAxis](#) > > [mRangeZoomVertAxis](#)
- double [mRangeZoomFactorHorz](#)
- double [mRangeZoomFactorVert](#)
- [QList](#)< [QCPRange](#) > [mDragStartHorzRange](#)
- [QList](#)< [QCPRange](#) > [mDragStartVertRange](#)
- [QCP::AntialiasedElements](#) [mAADragBackup](#)
- [QCP::AntialiasedElements](#) [mNotAADragBackup](#)
- bool [mDragging](#)
- [QHash](#)< [QCPAxis::AxisType](#), [QList](#)< [QCPAxis](#) * > > [mAxes](#)

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class **QCPCColorScale**

Additional Inherited Members

Public Types inherited from [QCPLayoutElement](#)

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.417.1 Constructor & Destructor Documentation

10.417.1.1 QCPCColorScaleAxisRectPrivate()

```
QCPCColorScaleAxisRectPrivate::QCPCColorScaleAxisRectPrivate (
    QCPCColorScale * parentColorScale ) [explicit]
```

Creates a new instance, as a child of *parentColorScale*.

10.417.2 Member Function Documentation

10.417.2.1 calculateAutoMargin()

```
int QCPAxisRect::calculateAutoMargin (
    QCP::MarginSide side ) [protected], [virtual]
```

Reimplemented from [QCPAxisRect](#).

10.417.2.2 draw()

```
void QCPColorScaleAxisRectPrivate::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Reimplemented from [QCPAxisRect](#).

10.417.2.3 mouseMoveEvent()

```
void QCPAxisRect::mouseMoveEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user moves the mouse while holding a mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPAxisRect](#).

10.417.2.4 mousePressEvent()

```
void QCPAxisRect::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses a mouse button while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *details* contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the *details* parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

[QCustomPlot](#) uses an event propagation system that works the same as Qt's system. If your layerable doesn't reimplement the [mousePressEvent](#) or explicitly calls `event->ignore()` in its reimplementation, the event will be propagated to the next layerable in the stacking order.

Once a layerable has accepted the [mousePressEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) or [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPAxisRect](#).

10.417.2.5 mouseReleaseEvent()

```
void QCPAxisRect::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user releases the mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter `startPos` indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPAxisRect](#).

10.417.2.6 update()

```
void QCPAxisRect::update (
    UpdatePhase phase ) [protected], [virtual]
```

This method is called automatically upon replot and doesn't need to be called by users of [QCPAxisRect](#).

Calls the base class implementation to update the margins (see [QCPLayoutElement::update](#)), and finally passes the [rect](#) to the inset layout ([insetLayout](#)) and calls its `QCPInsetLayout::update` function.

\seebaseclassmethod

Reimplemented from [QCPAxisRect](#).

10.417.2.7 wheelEvent()

```
void QCPAxisRect::wheelEvent (
    QWheelEvent * event ) [protected], [virtual]
```

This event gets called when the user turns the mouse scroll wheel while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`.

The `event->angleDelta()` indicates how far the mouse wheel was turned, which is usually +/- 120 for single rotation steps. However, if the mouse wheel is turned rapidly, multiple steps may accumulate to one event, making the delta larger. On the other hand, if the wheel has very smooth steps or none at all, the delta may be smaller.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#)

Reimplemented from [QCPAxisRect](#).

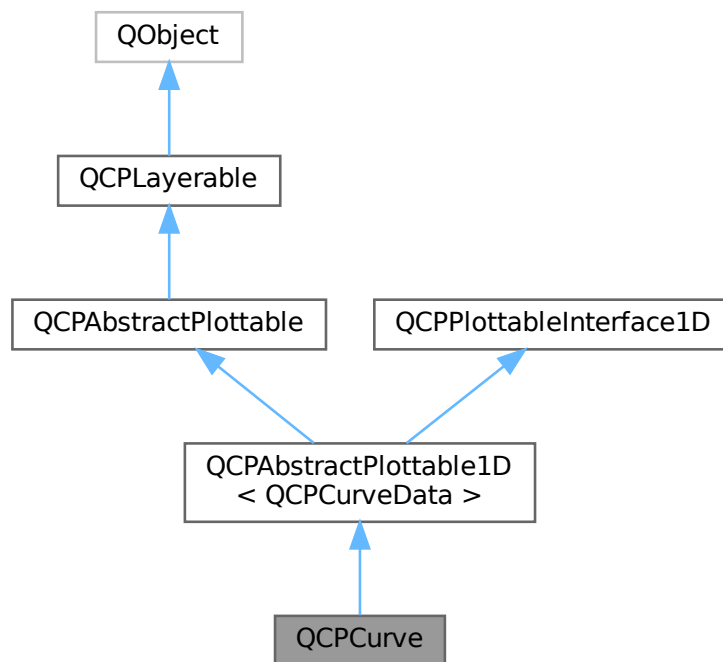
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

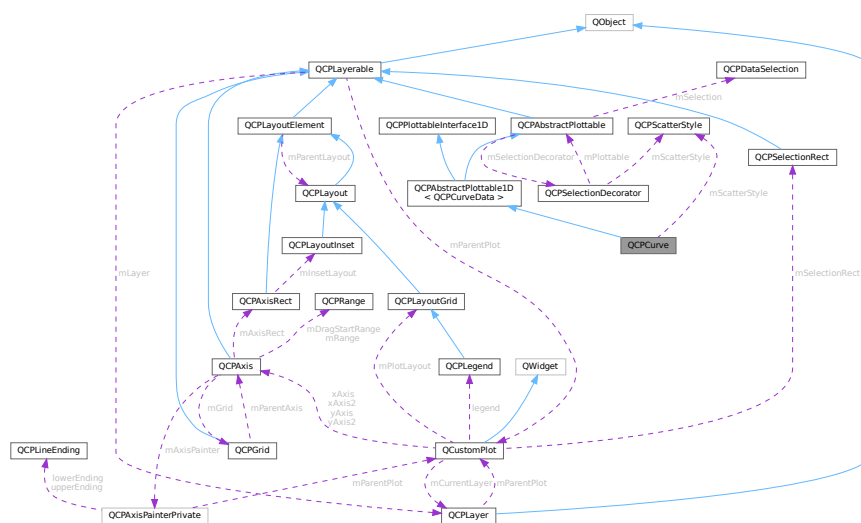
10.418 QCPCurve Class Reference

A plottable representing a parametric curve in a plot.

Inheritance diagram for QCPCurve:



Collaboration diagram for QCPCurve:



Public Types

- enum [LineStyle](#) { [IsNone](#) , [IsLine](#) }

Public Member Functions

- [QCPCurve](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QSharedPointer](#)< [QCPCurveDataContainer](#) > [data](#) () const
- [QCPScatterStyle](#) [scatterStyle](#) () const
- int [scatterSkip](#) () const
- [LineStyle](#) [lineStyle](#) () const
- void [setData](#) ([QSharedPointer](#)< [QCPCurveDataContainer](#) > [data](#))
- void [setData](#) (const [QVector](#)< double > &t, const [QVector](#)< double > &keys, const [QVector](#)< double > &values, bool alreadySorted=false)
- void [setData](#) (const [QVector](#)< double > &keys, const [QVector](#)< double > &values)
- void [setScatterStyle](#) (const [QCPScatterStyle](#) &style)
- void [setScatterSkip](#) (int skip)
- void [setLineStyle](#) ([LineStyle](#) style)
- void [addData](#) (const [QVector](#)< double > &t, const [QVector](#)< double > &keys, const [QVector](#)< double > &values, bool alreadySorted=false)
- void [addData](#) (const [QVector](#)< double > &keys, const [QVector](#)< double > &values)
- void [addData](#) (double t, double key, double value)
- void [addData](#) (double key, double value)
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) [getKeyRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) [getValueRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from [QCPAbstractPlottable1D](#)< [QCPCurveData](#) >

- [QCPAbstractPlottable1D](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- virtual int [dataCount](#) () const [Q_DECL_OVERRIDE](#)
- virtual double [dataMainKey](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual double [dataSortKey](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual double [dataMainValue](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) [dataValueRange](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual [QPointF](#) [dataPixelPosition](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual bool [sortKeysIsMainKey](#) () const [Q_DECL_OVERRIDE](#)
- virtual [QCPDataSelection](#) [selectTestRect](#) (const [QRectF](#) &rect, bool onlySelectable) const [Q_DECL_OVERRIDE](#)
- virtual int [findBegin](#) (double sortKey, bool expandedRange=true) const [Q_DECL_OVERRIDE](#)
- virtual int [findEnd](#) (double sortKey, bool expandedRange=true) const [Q_DECL_OVERRIDE](#)
- virtual [QCPPlottableInterface1D](#) * [interface1D](#) () [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QString](#) **name** () const
- bool **antialiasedFill** () const
- bool **antialiasedScatters** () const
- [QPen](#) **pen** () const
- [QBrush](#) **brush** () const
- [QCPAxis](#) * **keyAxis** () const
- [QCPAxis](#) * **valueAxis** () const
- [QCP::SelectionType](#) **selectable** () const
- bool **selected** () const
- [QCPDataSelection](#) **selection** () const
- [QCPSelectionDecorator](#) * **selectionDecorator** () const
- void **setName** (const [QString](#) &name)
- void **setAntialiasedFill** (bool enabled)
- void **setAntialiasedScatters** (bool enabled)
- void **setPen** (const [QPen](#) &pen)
- void **setBrush** (const [QBrush](#) &brush)
- void **setKeyAxis** ([QCPAxis](#) *axis)
- void **setValueAxis** ([QCPAxis](#) *axis)
- Q_SLOT void **setSelectable** ([QCP::SelectionType](#) selectable)
- Q_SLOT void **setSelection** ([QCPDataSelection](#) selection)
- void **setSelectionDecorator** ([QCPSelectionDecorator](#) *decorator)
- void **coordsToPixels** (double key, double value, double &x, double &y) const
- const [QPointF](#) **coordsToPixels** (double key, double value) const
- void **pixelsToCoords** (double x, double y, double &key, double &value) const
- void **pixelsToCoords** (const [QPointF](#) &pixelPos, double &key, double &value) const
- void **rescaleAxes** (bool onlyEnlarge=false) const
- void **rescaleKeyAxis** (bool onlyEnlarge=false) const
- void **rescaleValueAxis** (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool **addToLegend** ([QCPLegend](#) *legend)
- bool **addToLegend** ()
- bool **removeFromLegend** ([QCPLegend](#) *legend) const
- bool **removeFromLegend** () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const [QString](#) &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void [drawLegendIcon](#) ([QCPPainter](#) *painter, const [QRectF](#) &rect) const Q_DECL_OVERRIDE
- virtual void [drawCurveLine](#) ([QCPPainter](#) *painter, const [QVector](#)< [QPointF](#) > &lines) const
- virtual void [drawScatterPlot](#) ([QCPPainter](#) *painter, const [QVector](#)< [QPointF](#) > &points, const [QCPScatterStyle](#) &style) const
- void [getCurveLines](#) ([QVector](#)< [QPointF](#) > *lines, const [QCPDataRange](#) &dataRange, double penWidth) const
- void [getScatters](#) ([QVector](#)< [QPointF](#) > *scatters, const [QCPDataRange](#) &dataRange, double scatterWidth) const
- int [getRegion](#) (double key, double value, double keyMin, double valueMax, double keyMax, double valueMin) const
- [QPointF](#) [getOptimizedPoint](#) (int otherRegion, double otherKey, double otherValue, double key, double value, double keyMin, double valueMax, double keyMax, double valueMin) const
- [QVector](#)< [QPointF](#) > [getOptimizedCornerPoints](#) (int prevRegion, int currentRegion, double prevKey, double prevValue, double key, double value, double keyMin, double valueMax, double keyMax, double valueMin) const
- bool [mayTraverse](#) (int prevRegion, int currentRegion) const
- bool [getTraverse](#) (double prevKey, double prevValue, double key, double value, double keyMin, double valueMax, double keyMax, double valueMin, [QPointF](#) &crossA, [QPointF](#) &crossB) const
- void [getTraverseCornerPoints](#) (int prevRegion, int currentRegion, double keyMin, double valueMax, double keyMax, double valueMin, [QVector](#)< [QPointF](#) > &beforeTraverse, [QVector](#)< [QPointF](#) > &afterTraverse) const
- double [pointDistance](#) (const [QPointF](#) &pixelPoint, [QCPCurveDataContainer::const_iterator](#) &closestData) const

Protected Member Functions inherited from [QCPAbstractPlottable1D](#)< [QCPCurveData](#) >

- void [getDataSegments](#) ([QList](#)< [QCPDataRange](#) > &selectedSegments, [QList](#)< [QCPDataRange](#) > &unselectedSegments) const
- void [drawPolyline](#) ([QCPPainter](#) *painter, const [QVector](#)< [QPointF](#) > &lineData) const

Protected Member Functions inherited from [QCPAbstractPlottable](#)

- virtual [QRect](#) [clipRect](#) () const Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- void [applyFillAntialiasingHint](#) ([QCPPainter](#) *painter) const
- void [applyScattersAntialiasingHint](#) ([QCPPainter](#) *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void [mouseMoveEvent](#) ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void [mouseReleaseEvent](#) ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void [mouseDoubleClickEvent](#) ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void [wheelEvent](#) ([QWheelEvent](#) *event)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QCPScatterStyle](#) mScatterStyle
- int mScatterSkip
- [LineStyle](#) mLineStyle

Protected Attributes inherited from [QCPAbstractPlottable1D](#)< [QCPCurveData](#) >

- QSharedPointer< [QCPDataContainer](#)< [QCPCurveData](#) > > mDataContainer

Protected Attributes inherited from [QCPAbstractPlottable](#)

- QString mName
- bool mAntialiasedFill
- bool mAntialiasedScatters
- QPen mPen
- QBrush mBrush
- QPointer< [QCPAxis](#) > mKeyAxis
- QPointer< [QCPAxis](#) > mValueAxis
- [QCP::SelectionType](#) mSelectable
- [QCPDataSelection](#) mSelection
- [QCPSelectionDecorator](#) * mSelectionDecorator

Protected Attributes inherited from [QCPLayerable](#)

- bool mVisible
- [QCustomPlot](#) * mParentPlot
- QPointer< [QCPLayerable](#) > mParentLayerable
- [QCPLayer](#) * mLayer
- bool mAntialiased

Friends

- class [QCustomPlot](#)
- class [QCPLegend](#)

Additional Inherited Members

Signals inherited from [QCPAbstractPlottable](#)

- void [selectionChanged](#) (bool selected)
- void [selectionChanged](#) (const [QCPDataSelection](#) &selection)
- void [selectableChanged](#) ([QCP::SelectionType](#) selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.418.1 Detailed Description

A plottable representing a parametric curve in a plot.

Unlike [QCPGraph](#), plottables of this type may have multiple points with the same key coordinate, so their visual representation can have *loops*. This is realized by introducing a third coordinate t , which defines the order of the points described by the other two coordinates x and y .

To plot data, assign it with the [setData](#) or [addData](#) functions. Alternatively, you can also access and modify the curve's data via the [data](#) method, which returns a pointer to the internal QCPCurveDataContainer.

Gaps in the curve can be created by adding data points with NaN as key and value (`qQNaN()` or `std::numeric_limits<double>::quiet_NaN()`) in between the two data points that shall be separated.

10.418.2 Changing the appearance

The appearance of the curve is determined by the pen and the brush ([setPen](#), [setBrush](#)).

10.418.3 Usage

Like all data representing objects in [QCustomPlot](#), the [QCPCurve](#) is a plottable ([QCPAbstractPlottable](#)). So the plottable-interface of [QCustomPlot](#) applies ([QCustomPlot::plottable](#), [QCustomPlot::removePlottable](#), etc.)

Usually, you first create an instance:

which registers it with the [QCustomPlot](#) instance of the passed axes. Note that this [QCustomPlot](#) instance takes ownership of the plottable, so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead. The newly created plottable can be modified, e.g.:

10.418.4 Member Enumeration Documentation

10.418.4.1 LineStyle

```
enum QCPCurve::LineStyle
```

Defines how the curve's line is represented visually in the plot. The line is drawn with the current pen of the curve ([setPen](#)).

See also

[setLineStyle](#)

Enumerator

IsNone	No line is drawn between data points (e.g. only scatters)
IsLine	Data points are connected with a straight line.

10.418.5 Constructor & Destructor Documentation

10.418.5.1 QCPCurve()

```
QCPCurve::QCPCurve (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis ) [explicit]
```

Constructs a curve which uses *keyAxis* as its key axis ("x") and *valueAxis* as its value axis ("y"). *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#) instance and not have the same orientation. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

The created [QCPCurve](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPCurve](#), so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead.

10.418.6 Member Function Documentation

10.418.6.1 addData() [1/4]

```
void QCPCurve::addData (
    const QVector< double > & keys,
    const QVector< double > & values )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided points in *keys* and *values* to the current data. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

The *t* parameter of each data point will be set to the integer index of the respective key/value pair.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

10.418.6.2 addData() [2/4]

```
void QCPCurve::addData (
    const QVector< double > & t,
    const QVector< double > & keys,
    const QVector< double > & values,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided points in *t*, *keys* and *values* to the current data. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

10.418.6.3 addData() [3/4]

```
void QCPCurve::addData (
    double key,
    double value )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided data point as *key* and *value* to the current data.

The *t* parameter is generated automatically by increments of 1 for each point, starting at the highest *t* of previously existing data or 0, if the curve data is empty.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

10.418.6.4 addData() [4/4]

```
void QCPCurve::addData (
    double t,
    double key,
    double value )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts. Adds the provided data point as *t*, *key* and *value* to the current data.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

10.418.6.5 data()

```
QSharedPointer< QCPCurveDataContainer > QCPCurve::data ( ) const [inline]
```

Returns a shared pointer to the internal data storage of type `QCPCurveDataContainer`. You may use it to directly manipulate the data, which may be more convenient and faster than using the regular [setData](#) or [addData](#) methods.

10.418.6.6 draw()

```
void QCPCurve::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

10.418.6.7 drawLegendIcon()

```
void QCPCurve::drawLegendIcon (
    QCPPainter * painter,
    const QRectF & rect ) const [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

10.418.6.8 getKeyRange()

```
QCPRange QCPCurve::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implements [QCPAbstractPlottable](#).

10.418.6.9 getValueRange()

```
QCPRange QCPCurve::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implements [QCPAbstractPlottable](#).

10.418.6.10 selectTest()

```
double QCPCurve::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Implements a selectTest specific to this plottable's point geometry.

If *details* is not 0, it will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

\seebaseclassmethod [QCPAbstractPlottable::selectTest](#)

Reimplemented from [QCPAbstractPlottable1D< QCPCurveData >](#).

10.418.6.11 setData() [1/3]

```
void QCPCurve::setData (
    const QVector< double > & keys,
    const QVector< double > & values )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data with the provided points in *keys* and *values*. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

The *t* parameter of each data point will be set to the integer index of the respective key/value pair.

See also

[addData](#)

10.418.6.12 setData() [2/3]

```
void QCPCurve::setData (
    const QVector< double > & t,
    const QVector< double > & keys,
    const QVector< double > & values,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data with the provided points in *t*, *keys* and *values*. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *t* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

See also

[addData](#)

10.418.6.13 setData() [3/3]

```
void QCPCurve::setData (
    QSharedPointer< QCPCurveDataContainer > data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data container with the provided *data* container.

Since a QSharedPointer is used, multiple QCPCurves may share the same data container safely. Modifying the data in the container will then affect all curves that share the container. Sharing can be achieved by simply exchanging the data containers wrapped in shared pointers:

If you do not wish to share containers, but create a copy from an existing container, rather use the [QCPDataContainer<DataType>::set](#) method on the curve's data container directly:

See also

[addData](#)

10.418.6.14 setLineStyle()

```
void QCPCurve::setLineStyle (
    QCPCurve::LineStyle style )
```

Sets how the single data points are connected in the plot or how they are represented visually apart from the scatter symbol. For scatter-only plots, set *style* to [IsNone](#) and [setScatterStyle](#) to the desired scatter style.

See also

[setScatterStyle](#)

10.418.6.15 setScatterSkip()

```
void QCPCurve::setScatterSkip (
    int skip )
```

If scatters are displayed (scatter style not [QCPScatterStyle::ssNone](#)), *skip* number of scatter points are skipped/not drawn after every drawn scatter point.

This can be used to make the data appear sparser while for example still having a smooth line, and to improve performance for very high density plots.

If *skip* is set to 0 (default), all scatter points are drawn.

See also

[setScatterStyle](#)

10.418.6.16 setScatterStyle()

```
void QCPCurve::setScatterStyle (
    const QCPScatterStyle & style )
```

Sets the visual appearance of single data points in the plot. If set to [QCPScatterStyle::ssNone](#), no scatter points are drawn (e.g. for line-only plots with appropriate line style).

See also

[QCPScatterStyle](#), [setLineStyle](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.419 QCPCurveData Class Reference

Holds the data of one single data point for [QCPCurve](#).

Public Member Functions

- [QCPCurveData](#) ()
- [QCPCurveData](#) (double t, double key, double value)
- double [sortKey](#) () const
- double [mainKey](#) () const
- double [mainValue](#) () const
- [QCPRange](#) [valueRange](#) () const

Static Public Member Functions

- static [QCPCurveData](#) [fromSortKey](#) (double [sortKey](#))
- static bool [sortKeysIsMainKey](#) ()

Public Attributes

- double **t**
- double **key**
- double **value**

10.419.1 Detailed Description

Holds the data of one single data point for [QCPCurve](#).

The stored data is:

- *t*: the free ordering parameter of this curve point, like in the mathematical vector $(x(t), y(t))$. (This is the *sortKey*)
- *key*: coordinate on the key axis of this curve point (this is the *mainKey*)
- *value*: coordinate on the value axis of this curve point (this is the *mainValue*)

The container for storing multiple data points is [QCPCurveDataContainer](#). It is a typedef for [QCPDataContainer](#) with [QCPCurveData](#) as the `DataType` template parameter. See the documentation there for an explanation regarding the data type's generic methods.

See also

[QCPCurveDataContainer](#)

10.419.2 Constructor & Destructor Documentation

10.419.2.1 QCPCurveData() [1/2]

```
QCPCurveData::QCPCurveData ( )
```

Constructs a curve data point with *t*, *key* and *value* set to zero.

10.419.2.2 QCPCurveData() [2/2]

```
QCPCurveData::QCPCurveData (
    double t,
    double key,
    double value )
```

Constructs a curve data point with the specified *t*, *key* and *value*.

10.419.3 Member Function Documentation

10.419.3.1 fromSortKey()

```
static QCPCurveData QCPCurveData::fromSortKey (
    double sortKey ) [inline], [static]
```

Returns a data point with the specified *sortKey* (assigned to the data point's *t* member). All other members are set to zero.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.419.3.2 mainKey()

```
double QCPCurveData::mainKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPCDataContainer](#).

10.419.3.3 mainValue()

```
double QCPCurveData::mainValue ( ) const [inline]
```

Returns the *value* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPCDataContainer](#).

10.419.3.4 sortKey()

```
double QCPCurveData::sortKey ( ) const [inline]
```

Returns the *t* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPCDataContainer](#).

10.419.3.5 sortKeyIsMainKey()

```
static static bool QCPCurveData::sortKeyIsMainKey ( ) [inline], [static]
```

Since the member *key* is the data point key coordinate and the member *t* is the data ordering parameter, this method returns false.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPCDataContainer](#).

10.419.3.6 valueRange()

```
QCPCRange QCPCurveData::valueRange ( ) const [inline]
```

Returns a [QCPCRange](#) with both lower and upper boundary set to *value* of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPCDataContainer](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.420 QCPDataContainer< DataType > Class Template Reference

The generic data container for one-dimensional plottables.

```
#include <applications/gui/genesys/qcustomplot.h>
```

Public Types

- typedef QVector< DataType >::const_iterator **const_iterator**
- typedef QVector< DataType >::iterator **iterator**

Public Member Functions

- [QCPDataContainer](#) ()
- int [size](#) () const
- bool [isEmpty](#) () const
- bool [autoSqueeze](#) () const
- void [setAutoSqueeze](#) (bool enabled)
- void [set](#) (const [QCPDataContainer](#)< DataType > &data)
- void [set](#) (const QVector< DataType > &data, bool alreadySorted=false)
- void [add](#) (const [QCPDataContainer](#)< DataType > &data)
- void [add](#) (const QVector< DataType > &data, bool alreadySorted=false)
- void [add](#) (const DataType &data)
- void [removeBefore](#) (double sortKey)
- void [removeAfter](#) (double sortKey)
- void [remove](#) (double sortKeyFrom, double sortKeyTo)
- void [remove](#) (double sortKey)
- void [clear](#) ()
- void [sort](#) ()
- void [squeeze](#) (bool preAllocation=true, bool postAllocation=true)
- const_iterator [constBegin](#) () const
- const_iterator [constEnd](#) () const
- iterator [begin](#) ()
- iterator [end](#) ()
- const_iterator [findBegin](#) (double sortKey, bool expandedRange=true) const
- const_iterator [findEnd](#) (double sortKey, bool expandedRange=true) const
- const_iterator [at](#) (int index) const
- [QCPRange](#) [keyRange](#) (bool &foundRange, [QCP::SignDomain](#) signDomain=[QCP::sdBoth](#))
- [QCPRange](#) [valueRange](#) (bool &foundRange, [QCP::SignDomain](#) signDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)())
- [QCPDataRange](#) [dataRange](#) () const
- void [limitIteratorsToDataRange](#) (const_iterator &[begin](#), const_iterator &[end](#), const [QCPDataRange](#) &[dataRange](#)) const

Protected Member Functions

- void [preallocateGrow](#) (int minimumPreallocSize)
- void [performAutoSqueeze](#) ()

Protected Attributes

- bool **mAutoSqueeze**
- QVector< DataType > **mData**
- int **mPreallocSize**
- int **mPreallocIteration**

Related Symbols

(Note that these are not member symbols.)

- template<class DataType >
bool [qcpLessThanSortKey](#) (const DataType &a, const DataType &b)

10.420.1 Detailed Description

```
template<class DataType>
class QCPDataContainer< DataType >
```

The generic data container for one-dimensional plottables.

This class template provides a fast container for data storage of one-dimensional data. The data type is specified as template parameter (called *DataType* in the following) and must provide some methods as described in the [next section](#).

The data is stored in a sorted fashion, which allows very quick lookups by the sorted key as well as retrieval of ranges (see [findBegin](#), [findEnd](#), [keyRange](#)) using binary search. The container uses a preallocation and a postallocation scheme, such that appending and prepending data (with respect to the sort key) is very fast and minimizes reallocations. If data is added which needs to be inserted between existing keys, the merge usually can be done quickly too, using the fact that existing data is always sorted. The user can further improve performance by specifying that added data is already itself sorted by key, if he can guarantee that this is the case (see for example [add\(const QVector<DataType> &data, bool alreadySorted\)](#)).

The data can be accessed with the provided const iterators ([constBegin](#), [constEnd](#)). If it is necessary to alter existing data in-place, the non-const iterators can be used ([begin](#), [end](#)). Changing data members that are not the sort key (for most data types called *key*) is safe from the container's perspective.

Great care must be taken however if the sort key is modified through the non-const iterators. For performance reasons, the iterators don't automatically cause a re-sorting upon their manipulation. It is thus the responsibility of the user to leave the container in a sorted state when finished with the data manipulation, before calling any other methods on the container. A complete re-sort (e.g. after finishing all sort key manipulation) can be done by calling [sort](#). Failing to do so can not be detected by the container efficiently and will cause both rendering artifacts and potential data loss.

Implementing one-dimensional plottables that make use of a [QCPDataContainer<T>](#) is usually done by subclassing from [QCPAbstractPlottable1D<T>](#), which introduces an according *mDataContainer* member and some convenience methods.

10.420.2 Requirements for the `DataType` template parameter

The template parameter `DataType` is the type of the stored data points. It must be trivially copyable and have the following public methods, preferably inline:

- `double sortKey() const`
Returns the member variable of this data point that is the sort key, defining the ordering in the container. Often this variable is simply called *key*.
- `static DataType fromSortKey(double sortKey)`
Returns a new instance of the data type initialized with its sort key set to *sortKey*.
- `static bool sortKeyIsMainKey()`
Returns true if the sort key is equal to the main key (see method `mainKey` below). For most plottables this is the case. It is not the case for example for [QCPCurve](#), which uses *t* as sort key and *key* as main key. This is the reason why [QCPCurve](#) unlike [QCPGraph](#) can display parametric curves with loops.
- `double mainKey() const`
Returns the variable of this data point considered the main key. This is commonly the variable that is used as the coordinate of this data point on the key axis of the plottable. This method is used for example when determining the automatic axis rescaling of key axes ([QCPAxis::rescale](#)).
- `double mainValue() const`
Returns the variable of this data point considered the main value. This is commonly the variable that is used as the coordinate of this data point on the value axis of the plottable.
- `QCPRange valueRange() const`
Returns the range this data point spans in the value axis coordinate. If the data is single-valued (e.g. [QCPGraphData](#)), this is simply a range with both lower and upper set to the main data point value. However if the data points can represent multiple values at once (e.g. [QCPFinancialData](#) with its *high*, *low*, *open* and *close* values at each *key*) this method should return the range those values span. This method is used for example when determining the automatic axis rescaling of value axes ([QCPAxis::rescale](#)).

10.420.3 Constructor & Destructor Documentation

10.420.3.1 `QCPDataContainer()`

```
template<class DataType >
QCPDataContainer< DataType >::QCPDataContainer ( )
```

Constructs a [QCPDataContainer](#) used for plottable classes that represent a series of key-sorted data

10.420.4 Member Function Documentation

10.420.4.1 `add()` [1/3]

```
template<class DataType >
void QCPDataContainer< DataType >::add (
    const DataType & data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided single data point to the current data.

See also

[remove](#)

10.420.4.2 add() [2/3]

```
template<class DataType >
void QCPDataContainer< DataType >::add (
    const QCPDataContainer< DataType > & data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided *data* to the current data in this container.

See also

[set](#), [remove](#)

10.420.4.3 add() [3/3]

```
template<class DataType >
void QCPDataContainer< DataType >::add (
    const QVector< DataType > & data,
    bool alreadySorted = false )
```

Adds the provided data points in *data* to the current data.

If you can guarantee that the data points in *data* have ascending order with respect to the *DataType*'s sort key, set *alreadySorted* to true to avoid an unnecessary sorting run.

See also

[set](#), [remove](#)

10.420.4.4 at()

```
template<class DataType >
QCPDataContainer::const_iterator QCPDataContainer< DataType >::at (
    int index ) const [inline]
```

Returns a const iterator to the element with the specified *index*. If *index* points beyond the available elements in this container, returns [constEnd](#), i.e. an iterator past the last valid element.

You can use this method to easily obtain iterators from a [QCPDataRange](#), see the data selection page for an example.

10.420.4.5 begin()

```
template<class DataType >
QCPDataContainer::iterator QCPDataContainer< DataType >::begin ( ) [inline]
```

Returns a non-const iterator to the first data point in this container.

You can manipulate the data points in-place through the non-const iterators, but great care must be taken when manipulating the sort key of a data point, see [sort](#), or the detailed description of this class.

10.420.4.6 clear()

```
template<class DataType >
void QCPDataContainer< DataType >::clear ( )
```

Removes all data points.

See also

[remove](#), [removeAfter](#), [removeBefore](#)

10.420.4.7 constBegin()

```
template<class DataType >
QCPDataContainer::const_iterator QCPDataContainer< DataType >::constBegin ( ) const [inline]
```

Returns a const iterator to the first data point in this container.

10.420.4.8 constEnd()

```
template<class DataType >
QCPDataContainer::const_iterator QCPDataContainer< DataType >::constEnd ( ) const [inline]
```

Returns a const iterator to the element past the last data point in this container.

10.420.4.9 dataRange()

```
template<class DataType >
QCPDataRange QCPDataContainer< DataType >::dataRange ( ) const [inline]
```

Returns a [QCPDataRange](#) encompassing the entire data set of this container. This means the begin index of the returned range is 0, and the end index is [size](#).

10.420.4.10 end()

```
template<class DataType >
QCPDataContainer::iterator QCPDataContainer< DataType >::end ( ) [inline]
```

Returns a non-const iterator to the element past the last data point in this container.

You can manipulate the data points in-place through the non-const iterators, but great care must be taken when manipulating the sort key of a data point, see [sort](#), or the detailed description of this class.

10.420.4.11 findBegin()

```
template<class DataType >
QCPDataContainer< DataType >::const_iterator QCPDataContainer< DataType >::findBegin (
    double sortKey,
    bool expandedRange = true ) const
```

Returns an iterator to the data point with a (sort-)key that is equal to, just below, or just above *sortKey*. If *expandedRange* is true, the data point just below *sortKey* will be considered, otherwise the one just above.

This can be used in conjunction with [findEnd](#) to iterate over data points within a given key range, including or excluding the bounding data points that are just beyond the specified range.

If *expandedRange* is true but there are no data points below *sortKey*, [constBegin](#) is returned.

If the container is empty, returns [constEnd](#).

See also

[findEnd](#), [QCPPlottableInterface1D::findBegin](#)

10.420.4.12 findEnd()

```
template<class DataType >
QCPDataContainer< DataType >::const_iterator QCPDataContainer< DataType >::findEnd (
    double sortKey,
    bool expandedRange = true ) const
```

Returns an iterator to the element after the data point with a (sort-)key that is equal to, just above or just below *sortKey*. If *expandedRange* is true, the data point just above *sortKey* will be considered, otherwise the one just below.

This can be used in conjunction with [findBegin](#) to iterate over data points within a given key range, including the bounding data points that are just below and above the specified range.

If *expandedRange* is true but there are no data points above *sortKey*, [constEnd](#) is returned.

If the container is empty, [constEnd](#) is returned.

See also

[findBegin](#), [QCPPlottableInterface1D::findEnd](#)

10.420.4.13 isEmpty()

```
template<class DataType >
bool QCPDataContainer< DataType >::isEmpty ( ) const [inline]
```

Returns whether this container holds no data points.

10.420.4.14 keyRange()

```
template<class DataType >
QCPRange QCPDataContainer< DataType >::keyRange (
    bool & foundRange,
    QCP::SignDomain signDomain = QCP::sdBoth )
```

Returns the range encompassed by the (main-)key coordinate of all data points. The output parameter *foundRange* indicates whether a sensible range was found. If this is false, you should not use the returned [QCPRange](#) (e.g. the data container is empty or all points have the same key).

Use *signDomain* to control which sign of the key coordinates should be considered. This is relevant e.g. for logarithmic plots which can mathematically only display one sign domain at a time.

If the *DataType* reports that its main key is equal to the sort key (*sortKeyIsMainKey*), as is the case for most plottables, this method uses this fact and finds the range very quickly.

See also

[valueRange](#)

10.420.4.15 limitIteratorsToDataRange()

```
template<class DataType >
void QCPDataContainer< DataType >::limitIteratorsToDataRange (
    const_iterator & begin,
    const_iterator & end,
    const QCPDataRange & dataRange ) const
```

Makes sure *begin* and *end* mark a data range that is both within the bounds of this data container's data, as well as within the specified *dataRange*. The initial range described by the passed iterators *begin* and *end* is never expanded, only contracted if necessary.

This function doesn't require for *dataRange* to be within the bounds of this data container's valid range.

10.420.4.16 remove() [1/2]

```
template<class DataType >
void QCPDataContainer< DataType >::remove (
    double sortKey )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes a single data point at *sortKey*. If the position is not known with absolute (binary) precision, consider using [remove\(double sortKeyFrom, double sortKeyTo\)](#) with a small fuzziness interval around the suspected position, depending on the precision with which the (sort-)key is known.

See also

[removeBefore](#), [removeAfter](#), [clear](#)

10.420.4.17 remove() [2/2]

```
template<class DataType >
void QCPDataContainer< DataType >::remove (
    double sortKeyFrom,
    double sortKeyTo )
```

Removes all data points with (sort-)keys between *sortKeyFrom* and *sortKeyTo*. If *sortKeyFrom* is greater or equal to *sortKeyTo*, the function does nothing. To remove a single data point with known (sort-)key, use [remove\(double sortKey\)](#).

See also

[removeBefore](#), [removeAfter](#), [clear](#)

10.420.4.18 removeAfter()

```
template<class DataType >
void QCPDataContainer< DataType >::removeAfter (
    double sortKey )
```

Removes all data points with (sort-)keys greater than or equal to *sortKey*.

See also

[removeBefore](#), [remove](#), [clear](#)

10.420.4.19 removeBefore()

```
template<class DataType >
void QCPDataContainer< DataType >::removeBefore (
    double sortKey )
```

Removes all data points with (sort-)keys smaller than or equal to *sortKey*.

See also

[removeAfter](#), [remove](#), [clear](#)

10.420.4.20 set() [1/2]

```
template<class DataType >
void QCPDataContainer< DataType >::set (
    const QCPDataContainer< DataType > & data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data in this container with the provided *data*.

See also

[add](#), [remove](#)

10.420.4.21 `set()` [2/2]

```
template<class DataType >
void QCPDataContainer< DataType >::set (
    const QVector< DataType > & data,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data in this container with the provided *data*

If you can guarantee that the data points in *data* have ascending order with respect to the *DataType*'s sort key, set *alreadySorted* to true to avoid an unnecessary sorting run.

See also

[add](#), [remove](#)

10.420.4.22 `setAutoSqueeze()`

```
template<class DataType >
void QCPDataContainer< DataType >::setAutoSqueeze (
    bool enabled )
```

Sets whether the container automatically decides when to release memory from its post- and preallocation pools when data points are removed. By default this is enabled and for typical applications shouldn't be changed.

If auto squeeze is disabled, you can manually decide when to release pre-/postallocation with [squeeze](#).

10.420.4.23 `size()`

```
template<class DataType >
int QCPDataContainer< DataType >::size ( ) const [inline]
```

Returns the number of data points in the container.

10.420.4.24 `sort()`

```
template<class DataType >
void QCPDataContainer< DataType >::sort ( )
```

Re-sorts all data points in the container by their sort key.

When setting, adding or removing points using the [QCPDataContainer](#) interface ([set](#), [add](#), [remove](#), etc.), the container makes sure to always stay in a sorted state such that a full resort is never necessary. However, if you choose to directly manipulate the sort key on data points by accessing and modifying it through the non-const iterators ([begin](#), [end](#)), it is your responsibility to bring the container back into a sorted state before any other methods are called on it. This can be achieved by calling this method immediately after finishing the sort key manipulation.

10.420.4.25 squeeze()

```
template<class DataType >
void QCPDataContainer< DataType >::squeeze (
    bool preAllocation = true,
    bool postAllocation = true )
```

Frees all unused memory that is currently in the preallocation and postallocation pools.

Note that [QCPDataContainer](#) automatically decides whether squeezing is necessary, if [setAutoSqueeze](#) is left enabled. It should thus not be necessary to use this method for typical applications.

The parameters *preAllocation* and *postAllocation* control whether pre- and/or post allocation should be freed, respectively.

10.420.4.26 valueRange()

```
template<class DataType >
QCPRange QCPDataContainer< DataType >::valueRange (
    bool & foundRange,
    QCP::SignDomain signDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() )
```

Returns the range encompassed by the value coordinates of the data points in the specified key range (*inKeyRange*), using the full *DataType::valueRange* reported by the data points. The output parameter *foundRange* indicates whether a sensible range was found. If this is false, you should not use the returned [QCPRange](#) (e.g. the data container is empty or all points have the same value).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Use *signDomain* to control which sign of the value coordinates should be considered. This is relevant e.g. for logarithmic plots which can mathematically only display one sign domain at a time.

See also

[keyRange](#)

10.420.5 Friends And Related Symbol Documentation

10.420.5.1 qcpLessThanSortKey()

```
template<class DataType >
bool qcpLessThanSortKey (
    const DataType & a,
    const DataType & b ) [related]
```

Returns whether the sort key of *a* is less than the sort key of *b*.

See also

[QCPDataContainer::sort](#)

The documentation for this class was generated from the following file:

- applications/gui/genesys/qcustomplot.h

10.421 QCPDataRange Class Reference

Describes a data range given by begin and end index.

Public Member Functions

- [QCPDataRange](#) ()
- [QCPDataRange](#) (int begin, int end)
- bool **operator==** (const [QCPDataRange](#) &other) const
- bool **operator!=** (const [QCPDataRange](#) &other) const
- int **begin** () const
- int **end** () const
- int [size](#) () const
- int [length](#) () const
- void [setBegin](#) (int begin)
- void [setEnd](#) (int end)
- bool [isValid](#) () const
- bool [isEmpty](#) () const
- [QCPDataRange bounded](#) (const [QCPDataRange](#) &other) const
- [QCPDataRange expanded](#) (const [QCPDataRange](#) &other) const
- [QCPDataRange intersection](#) (const [QCPDataRange](#) &other) const
- [QCPDataRange adjusted](#) (int changeBegin, int changeEnd) const
- bool [intersects](#) (const [QCPDataRange](#) &other) const
- bool [contains](#) (const [QCPDataRange](#) &other) const

Related Symbols

(Note that these are not member symbols.)

- QDebug [operator<<](#) (QDebug d, const [QCPDataRange](#) &dataRange)

10.421.1 Detailed Description

Describes a data range given by begin and end index.

[QCPDataRange](#) holds two integers describing the begin ([setBegin](#)) and end ([setEnd](#)) index of a contiguous set of data points. The *end* index corresponds to the data point just after the last data point of the data range, like in standard iterators.

Data Ranges are not bound to a certain plottable, thus they can be freely exchanged, created and modified. If a non-contiguous data set shall be described, the class [QCPDataSelection](#) is used, which holds and manages multiple instances of [QCPDataRange](#). In most situations, [QCPDataSelection](#) is thus used.

Both [QCPDataRange](#) and [QCPDataSelection](#) offer convenience methods to work with them, e.g. [bounded](#), [expanded](#), [intersects](#), [intersection](#), [adjusted](#), [contains](#). Further, addition and subtraction operators (defined in [QCPDataSelection](#)) can be used to join/subtract data ranges and data selections (or mixtures), to retrieve a corresponding [QCPDataSelection](#).

QCustomPlot's data selection mechanism is based on [QCPDataSelection](#) and [QCPDataRange](#).

Note

Do not confuse [QCPDataRange](#) with [QCPRange](#). A [QCPRange](#) describes an interval in floating point plot coordinates, e.g. the current axis range.

10.421.2 Constructor & Destructor Documentation

10.421.2.1 QCPDataRange() [1/2]

```
QCPDataRange::QCPDataRange ( )
```

Creates an empty [QCPDataRange](#), with begin and end set to 0.

10.421.2.2 QCPDataRange() [2/2]

```
QCPDataRange::QCPDataRange (
    int begin,
    int end )
```

Creates a [QCPDataRange](#), initialized with the specified *begin* and *end*.

No checks or corrections are made to ensure the resulting range is valid ([isValid](#)).

10.421.3 Member Function Documentation

10.421.3.1 adjusted()

```
QCPDataRange QCPDataRange::adjusted (
    int changeBegin,
    int changeEnd ) const [inline]
```

Returns a data range where *changeBegin* and *changeEnd* were added to the begin and end indices, respectively.

10.421.3.2 bounded()

```
QCPDataRange QCPDataRange::bounded (
    const QCPDataRange & other ) const
```

Returns a data range that matches this data range, except that parts exceeding *other* are excluded.

This method is very similar to [intersection](#), with one distinction: If this range and the *other* range share no intersection, the returned data range will be empty with begin and end set to the respective boundary side of *other*, at which this range is residing. ([intersection](#) would just return a range with begin and end set to 0.)

10.421.3.3 contains()

```
bool QCPDataRange::contains (
    const QCPDataRange & other ) const
```

Returns whether all data points of *other* are also contained inside this data range.

See also

[intersects](#)

10.421.3.4 expanded()

```
QCPDataRange QCPDataRange::expanded (
    const QCPDataRange & other ) const
```

Returns a data range that contains both this data range as well as *other*.

10.421.3.5 intersection()

```
QCPDataRange QCPDataRange::intersection (
    const QCPDataRange & other ) const
```

Returns the data range which is contained in both this data range and *other*.

This method is very similar to [bounded](#), with one distinction: If this range and the *other* range share no intersection, the returned data range will be empty with begin and end set to 0. ([bounded](#) would return a range with begin and end set to one of the boundaries of *other*, depending on which side this range is on.)

See also

[QCPDataSelection::intersection](#)

10.421.3.6 intersects()

```
bool QCPDataRange::intersects (
    const QCPDataRange & other ) const
```

Returns whether this data range and *other* share common data points.

See also

[intersection](#), [contains](#)

10.421.3.7 isEmpty()

```
bool QCPDataRange::isEmpty ( ) const [inline]
```

Returns whether this range is empty, i.e. whether its begin index equals its end index.

See also

[size](#), [length](#)

10.421.3.8 isValid()

```
bool QCPDataRange::isValid ( ) const [inline]
```

Returns whether this range is valid. A valid range has a begin index greater or equal to 0, and an end index greater or equal to the begin index.

Note

Invalid ranges should be avoided and are never the result of any of [QCustomPlot](#)'s methods (unless they are themselves fed with invalid ranges). Do not pass invalid ranges to [QCustomPlot](#)'s methods. The invalid range is not inherently prevented in [QCPDataRange](#), to allow temporary invalid begin/end values while manipulating the range. An invalid range is not necessarily empty ([isEmpty](#)), since its [length](#) can be negative and thus non-zero.

10.421.3.9 length()

```
int QCPDataRange::length ( ) const [inline]
```

Returns the number of data points described by this data range. Equivalent to [size](#).

10.421.3.10 setBegin()

```
void QCPDataRange::setBegin (
    int begin ) [inline]
```

Sets the begin of this data range. The *begin* index points to the first data point that is part of the data range.

No checks or corrections are made to ensure the resulting range is valid ([isValid](#)).

See also

[setEnd](#)

10.421.3.11 setEnd()

```
void QCPDataRange::setEnd (
    int end ) [inline]
```

Sets the end of this data range. The *end* index points to the data point just after the last data point that is part of the data range.

No checks or corrections are made to ensure the resulting range is valid ([isValid](#)).

See also

[setBegin](#)

10.421.3.12 size()

```
int QCPDataRange::size ( ) const [inline]
```

Returns the number of data points described by this data range. This is equal to the end index minus the begin index.

See also

[length](#)

10.421.4 Friends And Related Symbol Documentation

10.421.4.1 operator<<()

```
QDebug operator<< (
    QDebug d,
    const QCPDataRange & dataRange ) [related]
```

Prints *dataRange* in a human readable format to the qDebug output.

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.422 QCPDataSelection Class Reference

Describes a data set by holding multiple [QCPDataRange](#) instances.

Public Member Functions

- [QCPDataSelection](#) ()
- [QCPDataSelection](#) (const [QCPDataRange](#) &range)
- bool [operator==](#) (const [QCPDataSelection](#) &other) const
- bool [operator!=](#) (const [QCPDataSelection](#) &other) const
- [QCPDataSelection](#) & [operator+=](#) (const [QCPDataSelection](#) &other)
- [QCPDataSelection](#) & [operator+=](#) (const [QCPDataRange](#) &other)
- [QCPDataSelection](#) & [operator-=](#) (const [QCPDataSelection](#) &other)
- [QCPDataSelection](#) & [operator-=](#) (const [QCPDataRange](#) &other)
- int [dataRangeCount](#) () const
- int [dataPointCount](#) () const
- [QCPDataRange](#) [dataRange](#) (int index=0) const
- QList< [QCPDataRange](#) > [dataRanges](#) () const
- [QCPDataRange](#) [span](#) () const
- void [addDataRange](#) (const [QCPDataRange](#) &dataRange, bool [simplify](#)=true)
- void [clear](#) ()
- bool [isEmpty](#) () const
- void [simplify](#) ()
- void [enforceType](#) ([QCP::SelectionType](#) type)
- bool [contains](#) (const [QCPDataSelection](#) &other) const
- [QCPDataSelection](#) [intersection](#) (const [QCPDataRange](#) &other) const
- [QCPDataSelection](#) [intersection](#) (const [QCPDataSelection](#) &other) const
- [QCPDataSelection](#) [inverse](#) (const [QCPDataRange](#) &outerRange) const

Friends

- const [QCPDataSelection operator+](#) (const [QCPDataSelection](#) &a, const [QCPDataSelection](#) &b)
- const [QCPDataSelection operator+](#) (const [QCPDataRange](#) &a, const [QCPDataSelection](#) &b)
- const [QCPDataSelection operator+](#) (const [QCPDataSelection](#) &a, const [QCPDataRange](#) &b)
- const [QCPDataSelection operator+](#) (const [QCPDataRange](#) &a, const [QCPDataRange](#) &b)
- const [QCPDataSelection operator-](#) (const [QCPDataSelection](#) &a, const [QCPDataSelection](#) &b)
- const [QCPDataSelection operator-](#) (const [QCPDataRange](#) &a, const [QCPDataSelection](#) &b)
- const [QCPDataSelection operator-](#) (const [QCPDataSelection](#) &a, const [QCPDataRange](#) &b)
- const [QCPDataSelection operator-](#) (const [QCPDataRange](#) &a, const [QCPDataRange](#) &b)

Related Symbols

(Note that these are not member symbols.)

- QDebug [operator<<](#) (QDebug d, const [QCPDataSelection](#) &selection)

10.422.1 Detailed Description

Describes a data set by holding multiple [QCPDataRange](#) instances.

[QCPDataSelection](#) manages multiple instances of [QCPDataRange](#) in order to represent any (possibly disjoint) set of data selection.

The data selection can be modified with addition and subtraction operators which take [QCPDataSelection](#) and [QCPDataRange](#) instances, as well as methods such as [addDataRange](#) and [clear](#). Read access is provided by [dataRange](#), [dataRanges](#), [dataRangeCount](#), etc.

The method [simplify](#) is used to join directly adjacent or even overlapping [QCPDataRange](#) instances. [QCPDataSelection](#) automatically simplifies when using the addition/subtraction operators. The only case when [simplify](#) is left to the user, is when calling [addDataRange](#), with the parameter *simplify* explicitly set to false. This is useful if many data ranges will be added to the selection successively and the overhead for simplifying after each iteration shall be avoided. In this case, you should make sure to call [simplify](#) after completing the operation.

Use [enforceType](#) to bring the data selection into a state complying with the constraints for selections defined in [QCP::SelectionType](#).

QCustomPlot's data selection mechanism is based on [QCPDataSelection](#) and [QCPDataRange](#).

10.422.2 Iterating over a data selection

As an example, the following code snippet calculates the average value of a graph's data [selection](#):

10.422.3 Constructor & Destructor Documentation

10.422.3.1 QCPDataSelection() [1/2]

```
QCPDataSelection::QCPDataSelection ( ) [explicit]
```

Creates an empty [QCPDataSelection](#).

10.422.3.2 QCPDataSelection() [2/2]

```
QCPDataSelection::QCPDataSelection (
    const QCPDataRange & range ) [explicit]
```

Creates a [QCPDataSelection](#) containing the provided *range*.

10.422.4 Member Function Documentation

10.422.4.1 addDataRange()

```
void QCPDataSelection::addDataRange (
    const QCPDataRange & dataRange,
    bool simplify = true )
```

Adds the given *dataRange* to this data selection. This is equivalent to the += operator but allows disabling immediate simplification by setting *simplify* to false. This can improve performance if adding a very large amount of data ranges successively. In this case, make sure to call [simplify](#) manually, after the operation.

10.422.4.2 clear()

```
void QCPDataSelection::clear ( )
```

Removes all data ranges. The data selection then contains no data points.

[isEmpty](#)

10.422.4.3 contains()

```
bool QCPDataSelection::contains (
    const QCPDataSelection & other ) const
```

Returns true if the data selection *other* is contained entirely in this data selection, i.e. all data point indices that are in *other* are also in this data selection.

See also

[QCPDataRange::contains](#)

10.422.4.4 dataPointCount()

```
int QCPDataSelection::dataPointCount ( ) const
```

Returns the total number of data points contained in all data ranges that make up this data selection.

10.422.4.5 dataRange()

```
QCPDataRange QCPDataSelection::dataRange (
    int index = 0 ) const
```

Returns the data range with the specified *index*.

If the data selection is simplified (the usual state of the selection, see [simplify](#)), the ranges are sorted by ascending data point index.

See also

[dataRangeCount](#)

10.422.4.6 dataRangeCount()

```
int QCPDataSelection::dataRangeCount ( ) const [inline]
```

Returns the number of ranges that make up the data selection. The ranges can be accessed by [dataRange](#) via their index.

See also

[dataRange](#), [dataPointCount](#)

10.422.4.7 dataRanges()

```
QList< QCPDataRange > QCPDataSelection::dataRanges ( ) const [inline]
```

Returns all data ranges that make up the data selection. If the data selection is simplified (the usual state of the selection, see [simplify](#)), the ranges are sorted by ascending data point index.

See also

[dataRange](#)

10.422.4.8 enforceType()

```
void QCPDataSelection::enforceType (
    QCP::SelectionType type )
```

Makes sure this data selection conforms to the specified *type* selection type. Before the type is enforced, [simplify](#) is called.

Depending on *type*, enforcing means adding new data points that were previously not part of the selection, or removing data points from the selection. If the current selection already conforms to *type*, the data selection is not changed.

See also

[QCP::SelectionType](#)

10.422.4.9 intersection() [1/2]

```
QCPDataSelection QCPDataSelection::intersection (
    const QCPDataRange & other ) const
```

Returns a data selection containing the points which are both in this data selection and in the data range *other*.

A common use case is to limit an unknown data selection to the valid range of a data container, using [QCPDataContainer::dataRange](#) as *other*. One can then safely iterate over the returned data selection without exceeding the data container's bounds.

10.422.4.10 intersection() [2/2]

```
QCPDataSelection QCPDataSelection::intersection (
    const QCPDataSelection & other ) const
```

Returns a data selection containing the points which are both in this data selection and in the data selection *other*.

10.422.4.11 inverse()

```
QCPDataSelection QCPDataSelection::inverse (
    const QCPDataRange & outerRange ) const
```

Returns a data selection which is the exact inverse of this data selection, with *outerRange* defining the base range on which to invert. If *outerRange* is smaller than the [span](#) of this data selection, it is expanded accordingly.

For example, this method can be used to retrieve all unselected segments by setting *outerRange* to the full data range of the plottable, and calling this method on a data selection holding the selected segments.

10.422.4.12 isEmpty()

```
bool QCPDataSelection::isEmpty ( ) const [inline]
```

Returns true if there are no data ranges, and thus no data points, in this [QCPDataSelection](#) instance.

See also

[dataRangeCount](#)

10.422.4.13 operator+=() [1/2]

```
QCPDataSelection & QCPDataSelection::operator+= (
    const QCPDataRange & other )
```

Adds the data range *other* to this data selection, and then simplifies this data selection (see [simplify](#)).

10.422.4.14 operator+=() [2/2]

```
QCPDataSelection & QCPDataSelection::operator+= (
    const QCPDataSelection & other )
```

Adds the data selection of *other* to this data selection, and then simplifies this data selection (see [simplify](#)).

10.422.4.15 operator-=() [1/2]

```
QCPDataSelection & QCPDataSelection::operator-= (
    const QCPDataRange & other )
```

Removes all data point indices that are described by *other* from this data selection.

10.422.4.16 operator-=() [2/2]

```
QCPDataSelection & QCPDataSelection::operator-= (
    const QCPDataSelection & other )
```

Removes all data point indices that are described by *other* from this data selection.

10.422.4.17 operator==()

```
bool QCPDataSelection::operator==(
    const QCPDataSelection & other ) const
```

Returns true if this selection is identical (contains the same data ranges with the same begin and end indices) to *other*.

Note that both data selections must be in simplified state (the usual state of the selection, see [simplify](#)) for this operator to return correct results.

10.422.4.18 simplify()

```
void QCPDataSelection::simplify ( )
```

Sorts all data ranges by range begin index in ascending order, and then joins directly adjacent or overlapping ranges. This can reduce the number of individual data ranges in the selection, and prevents possible double-counting when iterating over the data points held by the data ranges.

This method is automatically called when using the addition/subtraction operators. The only case when [simplify](#) is left to the user, is when calling [addDataRange](#), with the parameter *simplify* explicitly set to false.

10.422.4.19 span()

```
QCPDataRange QCPDataSelection::span ( ) const
```

Returns a [QCPDataRange](#) which spans the entire data selection, including possible intermediate segments which are not part of the original data selection.

10.422.5 Friends And Related Symbol Documentation

10.422.5.1 `operator+` [1/4]

```
const QCPDataSelection operator+ (  
    const QCPDataRange & a,  
    const QCPDataRange & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points in *a* joined with the data points in *b*. The resulting data selection is already simplified (see [QCPDataSelection::simplify](#)).

10.422.5.2 `operator+` [2/4]

```
const QCPDataSelection operator+ (  
    const QCPDataRange & a,  
    const QCPDataSelection & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points in *a* joined with the data points in *b*. The resulting data selection is already simplified (see [QCPDataSelection::simplify](#)).

10.422.5.3 `operator+` [3/4]

```
const QCPDataSelection operator+ (  
    const QCPDataSelection & a,  
    const QCPDataRange & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points in *a* joined with the data points in *b*. The resulting data selection is already simplified (see [QCPDataSelection::simplify](#)).

10.422.5.4 `operator+` [4/4]

```
const QCPDataSelection operator+ (  
    const QCPDataSelection & a,  
    const QCPDataSelection & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points in *a* joined with the data points in *b*. The resulting data selection is already simplified (see [QCPDataSelection::simplify](#)).

10.422.5.5 `operator-` [1/4]

```
const QCPDataSelection operator- (  
    const QCPDataRange & a,  
    const QCPDataRange & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points which are in *a* but not in *b*.

10.422.5.6 operator- [2/4]

```
const QCPDataSelection operator- (
    const QCPDataRange & a,
    const QCPDataSelection & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points which are in *a* but not in *b*.

10.422.5.7 operator- [3/4]

```
const QCPDataSelection operator- (
    const QCPDataSelection & a,
    const QCPDataRange & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points which are in *a* but not in *b*.

10.422.5.8 operator- [4/4]

```
const QCPDataSelection operator- (
    const QCPDataSelection & a,
    const QCPDataSelection & b ) [friend]
```

Return a [QCPDataSelection](#) with the data points which are in *a* but not in *b*.

10.422.5.9 operator<<()

```
QDebug operator<< (
    QDebug d,
    const QCPDataSelection & selection ) [related]
```

Prints *selection* in a human readable format to the qDebug output.

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

Public Member Functions

- [QCPErrors](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QSharedPointer](#)< [QCPErrorsDataContainer](#) > [data](#) () const
- [QCPAbstractPlottable](#) * [dataPlottable](#) () const
- [ErrorType](#) [errorType](#) () const
- double [whiskerWidth](#) () const
- double [symbolGap](#) () const
- void [setData](#) ([QSharedPointer](#)< [QCPErrorsDataContainer](#) > [data](#))
- void [setData](#) (const [QVector](#)< double > &error)
- void [setData](#) (const [QVector](#)< double > &errorMinus, const [QVector](#)< double > &errorPlus)
- void [setDataPlottable](#) ([QCPAbstractPlottable](#) *plottable)
- void [setErrorType](#) ([ErrorType](#) type)
- void [setWhiskerWidth](#) (double pixels)
- void [setSymbolGap](#) (double pixels)
- void [addData](#) (const [QVector](#)< double > &error)
- void [addData](#) (const [QVector](#)< double > &errorMinus, const [QVector](#)< double > &errorPlus)
- void [addData](#) (double error)
- void [addData](#) (double errorMinus, double errorPlus)
- virtual int [dataCount](#) () const [Q_DECL_OVERRIDE](#)
- virtual double [dataMainKey](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual double [dataSortKey](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual double [dataMainValue](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) [dataValueRange](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual [QPointF](#) [dataPixelPosition](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual bool [sortKeysMainKey](#) () const [Q_DECL_OVERRIDE](#)
- virtual [QCPDataSelection](#) [selectTestRect](#) (const [QRectF](#) &rect, bool onlySelectable) const [Q_DECL_OVERRIDE](#)
- virtual int [findBegin](#) (double sortKey, bool expandedRange=true) const [Q_DECL_OVERRIDE](#)
- virtual int [findEnd](#) (double sortKey, bool expandedRange=true) const [Q_DECL_OVERRIDE](#)
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_DECL_OVERRIDE](#)
- virtual [QCPPlottableInterface1D](#) * [interface1D](#) () [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QString](#) [name](#) () const
- bool [antialiasedFill](#) () const
- bool [antialiasedScatters](#) () const
- [QPen](#) [pen](#) () const
- [QBrush](#) [brush](#) () const
- [QCPAxis](#) * [keyAxis](#) () const
- [QCPAxis](#) * [valueAxis](#) () const
- [QCP::SelectionType](#) [selectable](#) () const
- bool [selected](#) () const
- [QCPDataSelection](#) [selection](#) () const
- [QCPSelectionDecorator](#) * [selectionDecorator](#) () const
- void [setName](#) (const [QString](#) &name)
- void [setAntialiasedFill](#) (bool enabled)
- void [setAntialiasedScatters](#) (bool enabled)
- void [setPen](#) (const [QPen](#) &pen)
- void [setBrush](#) (const [QBrush](#) &brush)
- void [setKeyAxis](#) ([QCPAxis](#) *axis)

- void [setValueAxis](#) (QCPAxis *axis)
- Q_SLOT void [setSelectable](#) (QCP::SelectionType selectable)
- Q_SLOT void [setSelection](#) (QCPDataSelection selection)
- void [setSelectionDecorator](#) (QCPSelectionDecorator *decorator)
- void [coordsToPixels](#) (double key, double value, double &x, double &y) const
- const QPointF [coordsToPixels](#) (double key, double value) const
- void [pixelsToCoords](#) (double x, double y, double &key, double &value) const
- void [pixelsToCoords](#) (const QPointF &pixelPos, double &key, double &value) const
- void [rescaleAxes](#) (bool onlyEnlarge=false) const
- void [rescaleKeyAxis](#) (bool onlyEnlarge=false) const
- void [rescaleValueAxis](#) (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool [addToLegend](#) (QCPLegend *legend)
- bool [addToLegend](#) ()
- bool [removeFromLegend](#) (QCPLegend *legend) const
- bool [removeFromLegend](#) () const

Public Member Functions inherited from QCPLayerable

- QCPLayerable (QCustomPlot *plot, QString targetLayer=QString(), QCPLayerable *parentLayerable=nullptr)
- bool **visible** () const
- QCustomPlot * **parentPlot** () const
- QCPLayerable * **parentLayerable** () const
- QCPLayer * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) (QCPLayer *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [draw](#) (QCPPainter *painter) Q_DECL_OVERRIDE
- virtual void [drawLegendIcon](#) (QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE
- virtual QCPRange [getKeyRange](#) (bool &foundRange, QCP::SignDomain inSignDomain=QCP::sdBoth) const Q_DECL_OVERRIDE
- virtual QCPRange [getValueRange](#) (bool &foundRange, QCP::SignDomain inSignDomain=QCP::sdBoth, const QCPRange &inKeyRange=QCPRange()) const Q_DECL_OVERRIDE
- void **getErrorBarLines** (QCPErrorBarsDataContainer::const_iterator it, QVector< QLineF > &backbones, QVector< QLineF > &whiskers) const
- void **getVisibleDataBounds** (QCPErrorBarsDataContainer::const_iterator &begin, QCPErrorBarsDataContainer::const_iterator &end, const QCPDataRange &rangeRestriction) const
- double **pointDistance** (const QPointF &pixelPoint, QCPErrorBarsDataContainer::const_iterator &closestData) const
- void **getDataSegments** (QList< QCPDataRange > &selectedSegments, QList< QCPDataRange > &unselectedSegments) const
- bool **errorBarVisible** (int index) const
- bool **rectIntersectsLine** (const QRectF &pixelRect, const QLineF &line) const

Protected Member Functions inherited from [QCPAbstractPlottable](#)

- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- void [applyDefaultAntialiasingHint](#) (QCPPainter *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- void [applyFillAntialiasingHint](#) (QCPPainter *painter) const
- void [applyScattersAntialiasingHint](#) (QCPPainter *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) (QCustomPlot *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) (QCustomPlot *parentPlot)
- void [setParentLayerable](#) (QCPLayerable *parentLayerable)
- bool [moveToLayer](#) (QCPLayer *layer, bool prepend)
- void [applyAntialiasingHint](#) (QCPPainter *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QSharedPointer< QCPErrorsBarsDataContainer > **mDataContainer**
- QPointer< [QCPAbstractPlottable](#) > **mDataPlottable**
- [ErrorType](#) **mErrorType**
- double **mWhiskerWidth**
- double **mSymbolGap**

Protected Attributes inherited from [QCPAbstractPlottable](#)

- QString **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- QPen **mPen**
- QBrush **mBrush**
- QPointer< [QCPAxis](#) > **mKeyAxis**
- QPointer< [QCPAxis](#) > **mValueAxis**
- [QCP::SelectionType](#) **mSelectable**
- [QCPDataSelection](#) **mSelection**
- [QCPSelectionDecorator](#) * **mSelectionDecorator**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class **QCustomPlot**
- class **QCPLegend**

Additional Inherited Members**Signals inherited from [QCPAbstractPlottable](#)**

- void [selectionChanged](#) (bool [selected](#))
- void [selectionChanged](#) (const [QCPDataSelection](#) &[selection](#))
- void [selectableChanged](#) ([QCP::SelectionType](#) [selectable](#))

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *[newLayer](#))

10.423.1 Detailed Description

A plottable that adds a set of error bars to other plottables.

The [QCPErrorBars](#) plottable can be attached to other one-dimensional plottables (e.g. [QCPGraph](#), [QCPCurve](#), [QCPBars](#), etc.) and equips them with error bars.

Use [setDataPlottable](#) to define for which plottable the [QCPErrorBars](#) shall display the error bars. The orientation of the error bars can be controlled with [setErrorType](#).

By using [setData](#), you can supply the actual error data, either as symmetric error or plus/minus asymmetric errors. [QCPErrorBars](#) only stores the error data. The absolute key/value position of each error bar will be adopted from the configured data plottable. The error data of the [QCPErrorBars](#) are associated one-to-one via their index to the data points of the data plottable. You can directly access and manipulate the error bar data via [data](#).

[Set](#) either of the plus/minus errors to NaN ([qQNaN\(\)](#) or `std::numeric_limits<double>::quiet_NaN()`) to not show the respective error bar on the data point at that index.

10.423.2 Changing the appearance

The appearance of the error bars is defined by the pen ([setPen](#)), and the width of the whiskers ([setWhiskerWidth](#)). Further, the error bar backbones may leave a gap around the data point center to prevent that error bars are drawn too close to or even through scatter points. This gap size can be controlled via [setSymbolGap](#).

10.423.3 Member Enumeration Documentation**10.423.3.1 [ErrorType](#)**

```
enum QCPErrorBars::ErrorType
```

Defines in which orientation the error bars shall appear. If your data needs both error dimensions, create two [QCPErrorBars](#) with different [ErrorType](#).

See also

[setErrorType](#)

Enumerator

etKeyError	The errors are for the key dimension (bars appear parallel to the key axis)
etValueError	The errors are for the value dimension (bars appear parallel to the value axis)

10.423.4 Constructor & Destructor Documentation

10.423.4.1 QCPErrors()

```
QCPErrors::QCPErrors (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis ) [explicit]
```

Constructs an error bars plottable which uses *keyAxis* as its key axis ("x") and *valueAxis* as its value axis ("y"). *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#) instance and not have the same orientation. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

It is also important that the *keyAxis* and *valueAxis* are the same for the error bars plottable and the data plottable that the error bars shall be drawn on ([setDataPlottable](#)).

The created [QCPErrors](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPErrors](#), so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead.

10.423.5 Member Function Documentation

10.423.5.1 addData() [1/4]

```
void QCPErrors::addData (
    const QVector< double > & error )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds symmetrical error values as specified in *error*. The errors will be associated one-to-one by the data point index to the associated data plottable ([setDataPlottable](#)).

You can directly access and manipulate the error bar data via [data](#).

See also

[setData](#)

10.423.5.2 `addData()` [2/4]

```
void QCPErrors::addData (
    const QVector< double > & errorMinus,
    const QVector< double > & errorPlus )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds asymmetrical errors as specified in *errorMinus* and *errorPlus*. The errors will be associated one-to-one by the data point index to the associated data plottable ([setDataPlottable](#)).

You can directly access and manipulate the error bar data via [data](#).

See also

[setData](#)

10.423.5.3 `addData()` [3/4]

```
void QCPErrors::addData (
    double error )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds a single symmetrical error bar as specified in *error*. The errors will be associated one-to-one by the data point index to the associated data plottable ([setDataPlottable](#)).

You can directly access and manipulate the error bar data via [data](#).

See also

[setData](#)

10.423.5.4 `addData()` [4/4]

```
void QCPErrors::addData (
    double errorMinus,
    double errorPlus )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds a single asymmetrical error bar as specified in *errorMinus* and *errorPlus*. The errors will be associated one-to-one by the data point index to the associated data plottable ([setDataPlottable](#)).

You can directly access and manipulate the error bar data via [data](#).

See also

[setData](#)

10.423.5.5 data()

```
QSharedPointer< QCPErrorsDataContainer > QCPErrors::data ( ) const [inline]
```

Returns a shared pointer to the internal data storage of type `QCPErrorsDataContainer`. You may use it to directly manipulate the error values, which may be more convenient and faster than using the regular [setData](#) methods.

10.423.5.6 dataCount()

```
int QCPErrors::dataCount ( ) const [virtual]
```

Returns the number of data points of the plottable.

Implements [QCPPlottableInterface1D](#).

10.423.5.7 dataMainKey()

```
double QCPErrors::dataMainKey (
    int index ) const [virtual]
```

Returns the main key of the data point at the given *index*.

What the main key is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

10.423.5.8 dataMainValue()

```
double QCPErrors::dataMainValue (
    int index ) const [virtual]
```

Returns the main value of the data point at the given *index*.

What the main value is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

10.423.5.9 dataPixelPosition()

```
QPointF QCPErrors::dataPixelPosition (
    int index ) const [virtual]
```

Returns the pixel position on the widget surface at which the data point at the given *index* appears.

Usually this corresponds to the point of `dataMainKey/ dataMainValue`, in pixel coordinates. However, depending on the plottable, this might be a different apparent position than just a coord-to-pixel transform of those values. For example, [QCPBars](#) apparent data values can be shifted depending on their stacking, bar grouping or configured base value.

Implements [QCPPlottableInterface1D](#).

10.423.5.10 dataSortKey()

```
double QCPErrorBars::dataSortKey (
    int index ) const [virtual]
```

Returns the sort key of the data point at the given *index*.

What the sort key is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

10.423.5.11 dataValueRange()

```
QCPRange QCPErrorBars::dataValueRange (
    int index ) const [virtual]
```

Returns the value range of the data point at the given *index*.

What the value range is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implements [QCPPlottableInterface1D](#).

10.423.5.12 draw()

```
void QCPErrorBars::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

10.423.5.13 drawLegendIcon()

```
void QCPErrorBars::drawLegendIcon (
    QCPPainter * painter,
    const QRectF & rect ) const [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

10.423.5.14 findBegin()

```
int QCPErrorBars::findBegin (
    double sortKey,
    bool expandedRange = true ) const [virtual]
```

Returns the index of the data point with a (sort-)key that is equal to, just below, or just above *sortKey*. If *expandedRange* is true, the data point just below *sortKey* will be considered, otherwise the one just above.

This can be used in conjunction with [findEnd](#) to iterate over data points within a given key range, including or excluding the bounding data points that are just beyond the specified range.

If *expandedRange* is true but there are no data points below *sortKey*, 0 is returned.

If the container is empty, returns 0 (in that case, [findEnd](#) will also return 0, so a loop using these methods will not iterate over the index 0).

See also

[findEnd](#), [QCPDataContainer::findBegin](#)

Implements [QCPPlottableInterface1D](#).

10.423.5.15 findEnd()

```
int QCPErrors::findEnd (
    double sortKey,
    bool expandedRange = true ) const [virtual]
```

Returns the index one after the data point with a (sort-)key that is equal to, just above, or just below *sortKey*. If *expandedRange* is true, the data point just above *sortKey* will be considered, otherwise the one just below.

This can be used in conjunction with [findBegin](#) to iterate over data points within a given key range, including the bounding data points that are just below and above the specified range.

If *expandedRange* is true but there are no data points above *sortKey*, the index just above the highest data point is returned.

If the container is empty, returns 0.

See also

[findBegin](#), [QCPDataContainer::findEnd](#)

Implements [QCPPlottableInterface1D](#).

10.423.5.16 getKeyRange()

```
QCPRange QCPErrors::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [protected], [virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implements [QCPAbstractPlottable](#).

10.423.5.17 getValueRange()

```
QCPRange QCPErrorBars::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [protected], [virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implements [QCPAbstractPlottable](#).

10.423.5.18 interface1D()

```
virtual QCPPlottableInterface1D * QCPErrorBars::interface1D ( ) [inline], [virtual]
```

If this plottable is a one-dimensional plottable, i.e. it implements the [QCPPlottableInterface1D](#), returns the *this* pointer with that type. Otherwise (e.g. in the case of a [QCPColorMap](#)) returns zero.

You can use this method to gain read access to data coordinates while holding a pointer to the abstract base class only.

Reimplemented from [QCPAbstractPlottable](#).

10.423.5.19 selectTest()

```
double QCPErrorBars::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Implements a `selectTest` specific to this plottable's point geometry.

If *details* is not 0, it will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

\see [baseclassmethod QCPAbstractPlottable::selectTest](#)

Implements [QCPAbstractPlottable](#).

10.423.5.20 selectTestRect()

```
QCPDataSelection QCPErrors::selectTestRect (
    const QRectF & rect,
    bool onlySelectable ) const [virtual]
```

Returns a data selection containing all the data points of this plottable which are contained (or hit by) *rect*. This is used mainly in the selection rect interaction for data selection (data selection mechanism).

If *onlySelectable* is true, an empty [QCPDataSelection](#) is returned if this plottable is not selectable (i.e. if [QCPAbstractPlottable::setSelectable](#) is [QCP::stNone](#)).

Note

rect must be a normalized rect (positive or zero width and height). This is especially important when using the rect of [QCPSelectionRect::accepted](#), which is not necessarily normalized. Use `QRect::normalized()` when passing a rect which might not be normalized.

Implements [QCPPlottableInterface1D](#).

10.423.5.21 setData() [1/3]

```
void QCPErrors::setData (
    const QVector< double > & error )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets symmetrical error values as specified in *error*. The errors will be associated one-to-one by the data point index to the associated data plottable ([setDataPlottable](#)).

You can directly access and manipulate the error bar data via [data](#).

See also

[addData](#)

10.423.5.22 setData() [2/3]

```
void QCPErrors::setData (
    const QVector< double > & errorMinus,
    const QVector< double > & errorPlus )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets asymmetrical errors as specified in *errorMinus* and *errorPlus*. The errors will be associated one-to-one by the data point index to the associated data plottable ([setDataPlottable](#)).

You can directly access and manipulate the error bar data via [data](#).

See also

[addData](#)

10.423.5.23 setData() [3/3]

```
void QCPErrors::setData (
    QSharedPointer< QCPErrorsDataContainer > data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data container with the provided *data* container.

Since a QSharedPointer is used, multiple [QCPErrors](#) instances may share the same data container safely. Modifying the data in the container will then affect all [QCPErrors](#) instances that share the container. Sharing can be achieved by simply exchanging the data containers wrapped in shared pointers:

If you do not wish to share containers, but create a copy from an existing container, assign the data containers directly:

(This uses different notation compared with other plottables, because the [QCPErrors](#) uses a `QVector< QCPErrorsData >` as its data container, instead of a [QCPErrorsDataContainer](#).)

See also

[addData](#)

10.423.5.24 setDataPlottable()

```
void QCPErrors::setDataPlottable (
    QCPErrorsPlottable * plottable )
```

Sets the data plottable to which the error bars will be applied. The error values specified e.g. via [setData](#) will be associated one-to-one by the data point index to the data points of *plottable*. This means that the error bars will adopt the key/value coordinates of the data point with the same index.

The passed *plottable* must be a one-dimensional plottable, i.e. it must implement the [QCPErrorsPlottableInterface1D](#). Further, it must not be a [QCPErrors](#) instance itself. If either of these restrictions is violated, a corresponding qDebug output is generated, and the data plottable of this [QCPErrors](#) instance is set to zero.

For proper display, care must also be taken that the key and value axes of the *plottable* match those configured for this [QCPErrors](#) instance.

10.423.5.25 setErrorType()

```
void QCPErrors::setErrorType (
    ErrorType type )
```

Sets in which orientation the error bars shall appear on the data points. If your data needs both error dimensions, create two [QCPErrors](#) with different *type*.

10.423.5.26 setSymbolGap()

```
void QCPErrors::setSymbolGap (
    double pixels )
```

Sets the gap diameter around the data points that will be left out when drawing the error bar backbones. This gap prevents that error bars are drawn too close to or even through scatter points.

10.423.5.27 setWhiskerWidth()

```
void QCPErrorsData::setWhiskerWidth (
    double pixels )
```

Sets the width of the whiskers (the short bars at the end of the actual error bar backbones) to *pixels*.

10.423.5.28 sortKeysMainKey()

```
bool QCPErrorsData::sortKeyIsMainKey ( ) const [virtual]
```

Returns whether the sort key ([dataSortKey](#)) is identical to the main key ([dataMainKey](#)).

What the sort and main keys are, is defined by the plottable's data type. See the [QCPErrorsData Container DataType](#) documentation for details about this naming convention.

Implements [QCPErrorsDataInterface1D](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.424 QCPErrorsData Class Reference

Holds the data of one single error bar for [QCPErrorsData](#).

Public Member Functions

- [QCPErrorsData](#) ()
- [QCPErrorsData](#) (double error)
- [QCPErrorsData](#) (double errorMinus, double errorPlus)

Public Attributes

- double **errorMinus**
- double **errorPlus**

10.424.1 Detailed Description

Holds the data of one single error bar for [QCPErrorsData](#).

The stored data is:

- *errorMinus*: how much the error bar extends towards negative coordinates from the data point position
- *errorPlus*: how much the error bar extends towards positive coordinates from the data point position

The container for storing the error bar information is [QCPErrorsDataContainer](#). It is a typedef for `QVector<QCPErrorsData>`.

See also

[QCPErrorsDataContainer](#)

10.424.2 Constructor & Destructor Documentation

10.424.2.1 QCPErrorsData() [1/3]

```
QCPErrorsData::QCPErrorsData ( )
```

Constructs an error bar with errors set to zero.

10.424.2.2 QCPErrorsData() [2/3]

```
QCPErrorsData::QCPErrorsData (
    double error ) [explicit]
```

Constructs an error bar with equal *error* in both negative and positive direction.

10.424.2.3 QCPErrorsData() [3/3]

```
QCPErrorsData::QCPErrorsData (
    double errorMinus,
    double errorPlus )
```

Constructs an error bar with negative and positive errors set to *errorMinus* and *errorPlus*, respectively.

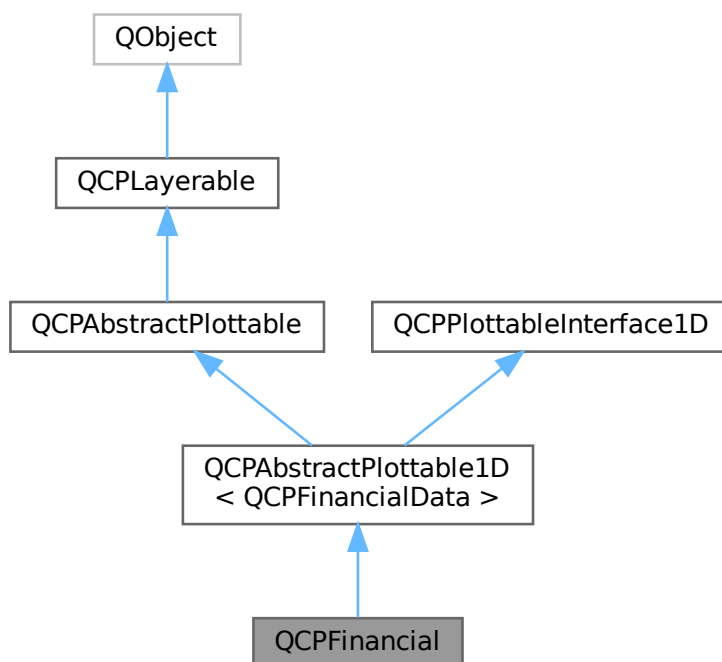
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

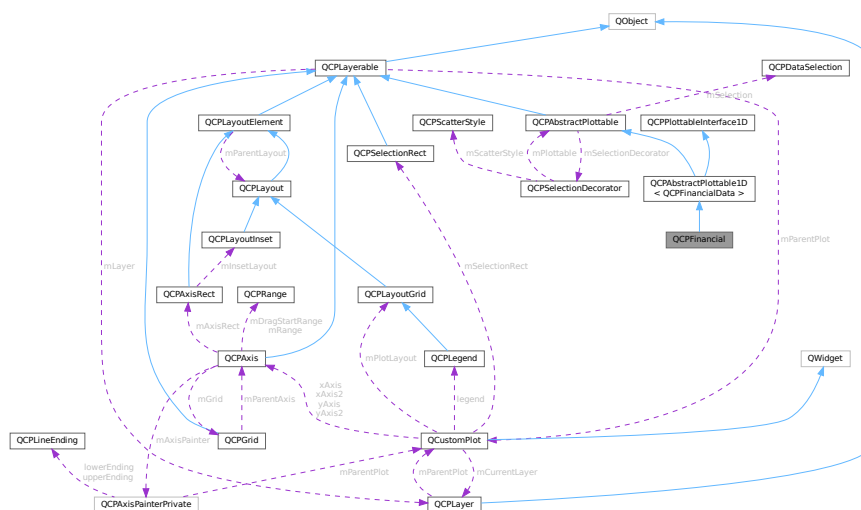
10.425 QCPFinancial Class Reference

A plottable representing a financial stock chart.

Inheritance diagram for QCPFinancial:



Collaboration diagram for QCPFinancial:



Public Types

- enum WidthType { wtAbsolute , wtAxisRectRatio , wtPlotCoords }
- enum ChartStyle { csOhlc , csCandlestick }

Public Member Functions

- [QCPFinancial](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QSharedPointer](#)< [QCPFinancialDataContainer](#) > data () const
- [ChartStyle](#) chartStyle () const
- double width () const
- [WidthType](#) widthType () const
- bool twoColored () const
- [QBrush](#) brushPositive () const
- [QBrush](#) brushNegative () const
- [QPen](#) penPositive () const
- [QPen](#) penNegative () const
- void setData ([QSharedPointer](#)< [QCPFinancialDataContainer](#) > data)
- void setData (const [QVector](#)< double > &keys, const [QVector](#)< double > &open, const [QVector](#)< double > &high, const [QVector](#)< double > &low, const [QVector](#)< double > &close, bool alreadySorted=false)
- void setChartStyle ([ChartStyle](#) style)
- void setWidth (double width)
- void setWidthType ([WidthType](#) widthType)
- void setTwoColored (bool twoColored)
- void setBrushPositive (const [QBrush](#) &brush)
- void setBrushNegative (const [QBrush](#) &brush)
- void setPenPositive (const [QPen](#) &pen)
- void setPenNegative (const [QPen](#) &pen)
- void addData (const [QVector](#)< double > &keys, const [QVector](#)< double > &open, const [QVector](#)< double > &high, const [QVector](#)< double > &low, const [QVector](#)< double > &close, bool alreadySorted=false)
- void addData (double key, double open, double high, double low, double close)
- virtual [QCPDataSelection](#) selectTestRect (const [QRectF](#) &rect, bool onlySelectable) const [Q_DECL_OVERRIDE](#)
- virtual double selectTest (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) getKeyRange (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) getValueRange (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from [QCPAbstractPlottable1D](#)< [QCPFinancialData](#) >

- [QCPAbstractPlottable1D](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- virtual int dataCount () const [Q_DECL_OVERRIDE](#)
- virtual double dataMainKey (int index) const [Q_DECL_OVERRIDE](#)
- virtual double dataSortKey (int index) const [Q_DECL_OVERRIDE](#)
- virtual double dataMainValue (int index) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) dataValueRange (int index) const [Q_DECL_OVERRIDE](#)
- virtual [QPointF](#) dataPixelPosition (int index) const [Q_DECL_OVERRIDE](#)
- virtual bool sortKeysMainKey () const [Q_DECL_OVERRIDE](#)
- virtual int findBegin (double sortKey, bool expandedRange=true) const [Q_DECL_OVERRIDE](#)
- virtual int findEnd (double sortKey, bool expandedRange=true) const [Q_DECL_OVERRIDE](#)
- virtual [QCPPlottableInterface1D](#) * interface1D () [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from QCPAbstractPlottable

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QString](#) **name** () const
- bool **antialiasedFill** () const
- bool **antialiasedScatters** () const
- [QPen](#) **pen** () const
- [QBrush](#) **brush** () const
- [QCPAxis](#) * **keyAxis** () const
- [QCPAxis](#) * **valueAxis** () const
- [QCP::SelectionType](#) **selectable** () const
- bool **selected** () const
- [QCPDataSelection](#) **selection** () const
- [QCPSelectionDecorator](#) * **selectionDecorator** () const
- void **setName** (const [QString](#) &name)
- void **setAntialiasedFill** (bool enabled)
- void **setAntialiasedScatters** (bool enabled)
- void **setPen** (const [QPen](#) &pen)
- void **setBrush** (const [QBrush](#) &brush)
- void **setKeyAxis** ([QCPAxis](#) *axis)
- void **setValueAxis** ([QCPAxis](#) *axis)
- [Q_SLOT](#) void **setSelectable** ([QCP::SelectionType](#) selectable)
- [Q_SLOT](#) void **setSelection** ([QCPDataSelection](#) selection)
- void **setSelectionDecorator** ([QCPSelectionDecorator](#) *decorator)
- void **coordsToPixels** (double key, double value, double &x, double &y) const
- const [QPointF](#) **coordsToPixels** (double key, double value) const
- void **pixelsToCoords** (double x, double y, double &key, double &value) const
- void **pixelsToCoords** (const [QPointF](#) &pixelPos, double &key, double &value) const
- void **rescaleAxes** (bool onlyEnlarge=false) const
- void **rescaleKeyAxis** (bool onlyEnlarge=false) const
- void **rescaleValueAxis** (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool **addToLegend** ([QCPLegend](#) *legend)
- bool **addToLegend** ()
- bool **removeFromLegend** ([QCPLegend](#) *legend) const
- bool **removeFromLegend** () const

Public Member Functions inherited from QCPLayerable

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- [Q_SLOT](#) bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const [QString](#) &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Static Public Member Functions

- static [QCPFinancialDataContainer](#) **timeSeriesToOhlc** (const [QVector](#)< double > &time, const [QVector](#)< double > &value, double timeBinSize, double timeBinOffset=0)

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void [drawLegendIcon](#) ([QCPPainter](#) *painter, const QRectF &rect) const Q_DECL_OVERRIDE
- void [drawOhlcPlot](#) ([QCPPainter](#) *painter, const QCPFinancialDataContainer::const_iterator &begin, const QCPFinancialDataContainer::const_iterator &end, bool isSelected)
- void [drawCandlestickPlot](#) ([QCPPainter](#) *painter, const QCPFinancialDataContainer::const_iterator &begin, const QCPFinancialDataContainer::const_iterator &end, bool isSelected)
- double [getPixelWidth](#) (double key, double keyPixel) const
- double [ohlcSelectTest](#) (const QPointF &pos, const QCPFinancialDataContainer::const_iterator &begin, const QCPFinancialDataContainer::const_iterator &end, QCPFinancialDataContainer::const_iterator &closestDataPoint) const
- double [candlestickSelectTest](#) (const QPointF &pos, const QCPFinancialDataContainer::const_iterator &begin, const QCPFinancialDataContainer::const_iterator &end, QCPFinancialDataContainer::const_iterator &closestDataPoint) const
- void [getVisibleDataBounds](#) (QCPFinancialDataContainer::const_iterator &begin, QCPFinancialDataContainer::const_iterator &end) const
- QRectF [selectionHitBox](#) (QCPFinancialDataContainer::const_iterator it) const

Protected Member Functions inherited from [QCPAbstractPlottable1D](#) < [QCPFinancialData](#) >

- void [getDataSegments](#) (QList< [QCPDataRange](#) > &selectedSegments, QList< [QCPDataRange](#) > &unselectedSegments) const
- void [drawPolyline](#) ([QCPPainter](#) *painter, const QVector< QPointF > &lineData) const

Protected Member Functions inherited from [QCPAbstractPlottable](#)

- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- void [applyFillAntialiasingHint](#) ([QCPPainter](#) *painter) const
- void [applyScattersAntialiasingHint](#) ([QCPPainter](#) *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [ChartStyle](#) **mChartStyle**
- double **mWidth**
- [WidthType](#) **mWidthType**
- bool **mTwoColored**
- QBrush **mBrushPositive**
- QBrush **mBrushNegative**
- QPen **mPenPositive**
- QPen **mPenNegative**

Protected Attributes inherited from [QCPAbstractPlottable1D](#)< [QCPFinancialData](#) >

- QSharedPointer< [QCPDataContainer](#)< [QCPFinancialData](#) > > **mDataContainer**

Protected Attributes inherited from [QCPAbstractPlottable](#)

- QString **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- QPen **mPen**
- QBrush **mBrush**
- QPointer< [QCPAxis](#) > **mKeyAxis**
- QPointer< [QCPAxis](#) > **mValueAxis**
- [QCP::SelectionType](#) **mSelectable**
- [QCPDataSelection](#) **mSelection**
- [QCPSelectionDecorator](#) * **mSelectionDecorator**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class **QCustomPlot**
- class **QCPLegend**

Additional Inherited Members**Signals inherited from [QCPAbstractPlottable](#)**

- void [selectionChanged](#) (bool [selected](#))
- void [selectionChanged](#) (const [QCPDataSelection](#) &[selection](#))
- void [selectableChanged](#) ([QCP::SelectionType](#) [selectable](#))

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.425.1 Detailed Description

A plottable representing a financial stock chart.

This plottable represents time series data binned to certain intervals, mainly used for stock charts. The two common representations OHLC (Open-High-Low-Close) bars and Candlesticks can be set via [setChartStyle](#).

The data is passed via [setData](#) as a set of open/high/low/close values at certain keys (typically times). This means the data must be already binned appropriately. If data is only available as a series of values (e.g. *price* against *time*), you can use the static convenience function [timeSeriesToOhlc](#) to generate binned OHLC-data which can then be passed to [setData](#).

The width of the OHLC bars/candlesticks can be controlled with [setWidth](#) and [setWidthType](#). A typical choice is to set the width type to [wtPlotCoords](#) (the default) and the width to (or slightly less than) one time bin interval width.

10.425.2 Changing the appearance

Charts can be either single- or two-colored ([setTwoColored](#)). If set to be single-colored, lines are drawn with the plottable's pen ([setPen](#)) and fills with the brush ([setBrush](#)).

If set to two-colored, positive changes of the value during an interval ($close \geq open$) are represented with a different pen and brush than negative changes ($close < open$). These can be configured with [setPenPositive](#), [setPenNegative](#), [setBrushPositive](#), and [setBrushNegative](#). In two-colored mode, the normal plottable pen/brush is ignored. Upon selection however, the normal selected pen/brush (provided by the [selectionDecorator](#)) is used, irrespective of whether the chart is single- or two-colored.

10.425.3 Usage

Like all data representing objects in [QCustomPlot](#), the [QCPFinancial](#) is a plottable ([QCPAbstractPlottable](#)). So the plottable-interface of [QCustomPlot](#) applies ([QCustomPlot::plottable](#), [QCustomPlot::removePlottable](#), etc.)

Usually, you first create an instance:

which registers it with the [QCustomPlot](#) instance of the passed axes. Note that this [QCustomPlot](#) instance takes ownership of the plottable, so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead. The newly created plottable can be modified, e.g.:

Here we have used the static helper method [timeSeriesToOhlc](#), to turn a time-price data series into a 24-hour binned open-high-low-close data series as [QCPFinancial](#) uses.

10.425.4 Member Enumeration Documentation

10.425.4.1 ChartStyle

```
enum QCPFinancial::ChartStyle
```

Defines the possible representations of OHLC data in the plot.

See also

[setChartStyle](#)

Enumerator

csOhlc	Open-High-Low-Close bar representation.
csCandlestick	Candlestick representation.

10.425.4.2 WidthType

```
enum QCPFinancial::WidthType
```

Defines the ways the width of the financial bar can be specified. Thus it defines what the number passed to [setWidth](#) actually means.

See also

[setWidthType](#), [setWidth](#)

Enumerator

wtAbsolute	width is in absolute pixels
wtAxisRectRatio	width is given by a fraction of the axis rect size
wtPlotCoords	width is in key coordinates and thus scales with the key axis range

10.425.5 Constructor & Destructor Documentation

10.425.5.1 QCPFinancial()

```
QCPFinancial::QCPFinancial (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis ) [explicit]
```

Constructs a financial chart which uses *keyAxis* as its key axis ("x") and *valueAxis* as its value axis ("y"). *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#) instance and not have the same orientation. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

The created [QCPFinancial](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPFinancial](#), so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead.

10.425.6 Member Function Documentation

10.425.6.1 addData() [1/2]

```
void QCPFinancial::addData (
    const QVector< double > & keys,
    const QVector< double > & open,
    const QVector< double > & high,
```

```
const QVector< double > & low,
const QVector< double > & close,
bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided points in *keys*, *open*, *high*, *low* and *close* to the current data. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

See also

[timeSeriesToOhlc](#)

10.425.6.2 addData() [2/2]

```
void QCPFinancial::addData (
    double key,
    double open,
    double high,
    double low,
    double close )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided data point as *key*, *open*, *high*, *low* and *close* to the current data.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

See also

[timeSeriesToOhlc](#)

10.425.6.3 data()

```
QCPFinancialDataContainer * QCPFinancial::data ( ) const [inline]
```

Returns a pointer to the internal data storage of type `QCPFinancialDataContainer`. You may use it to directly manipulate the data, which may be more convenient and faster than using the regular [setData](#) or [addData](#) methods, in certain situations.

10.425.6.4 draw()

```
void QCPFinancial::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

10.425.6.5 drawLegendIcon()

```
void QCPFinancial::drawLegendIcon (
    QCPPainter * painter,
    const QRectF & rect ) const [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

10.425.6.6 getKeyRange()

```
QCPRange QCPFinancial::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implements [QCPAbstractPlottable](#).

10.425.6.7 getValueRange()

```
QCPRange QCPFinancial::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implements [QCPAbstractPlottable](#).

10.425.6.8 selectTest()

```
double QCPFinancial::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Implements a selectTest specific to this plottable's point geometry.

If *details* is not 0, it will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

\seebaseclassmethod [QCPAbstractPlottable::selectTest](#)

Reimplemented from [QCPAbstractPlottable1D< QCPFinancialData >](#).

10.425.6.9 selectTestRect()

```
QCPDataSelection QCPFinancial::selectTestRect (
    const QRectF & rect,
    bool onlySelectable ) const [virtual]
```

Returns a data selection containing all the data points of this plottable which are contained (or hit by) *rect*. This is used mainly in the selection rect interaction for data selection (data selection mechanism).

If *onlySelectable* is true, an empty [QCPDataSelection](#) is returned if this plottable is not selectable (i.e. if [QCPAbstractPlottable::setSelectable](#) is [QCP::stNone](#)).

Note

rect must be a normalized rect (positive or zero width and height). This is especially important when using the rect of [QCPSelectionRect::accepted](#), which is not necessarily normalized. Use `QRect::normalized()` when passing a rect which might not be normalized.

Reimplemented from [QCPAbstractPlottable1D< QCPFinancialData >](#).

10.425.6.10 setBrushNegative()

```
void QCPFinancial::setBrushNegative (
    const QBrush & brush )
```

If [setTwoColored](#) is set to true, this function controls the brush that is used to draw fills of data points with a negative trend (i.e. bars/candlesticks with close < open).

If *twoColored* is false, the normal plottable's pen and brush are used ([setPen](#), [setBrush](#)).

See also

[setBrushPositive](#), [setPenNegative](#), [setPenPositive](#)

10.425.6.11 `setBrushPositive()`

```
void QCPFinancial::setBrushPositive (
    const QBrush & brush )
```

If [setTwoColored](#) is set to true, this function controls the brush that is used to draw fills of data points with a positive trend (i.e. bars/candlesticks with close >= open).

If *twoColored* is false, the normal plottable's pen and brush are used ([setPen](#), [setBrush](#)).

See also

[setBrushNegative](#), [setPenPositive](#), [setPenNegative](#)

10.425.6.12 `setChartStyle()`

```
void QCPFinancial::setChartStyle (
    QCPFinancial::ChartStyle style )
```

Sets which representation style shall be used to display the OHLC data.

10.425.6.13 `setData()` [1/2]

```
void QCPFinancial::setData (
    const QVector< double > & keys,
    const QVector< double > & open,
    const QVector< double > & high,
    const QVector< double > & low,
    const QVector< double > & close,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data with the provided points in *keys*, *open*, *high*, *low* and *close*. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

See also

[addData](#), [timeSeriesToOhlc](#)

10.425.6.14 setData() [2/2]

```
void QCPFinancial::setData (
    QSharedPointer< QCPFinancialDataContainer > data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data container with the provided *data* container.

Since a QSharedPointer is used, multiple QCPFinancials may share the same data container safely. Modifying the data in the container will then affect all financials that share the container. Sharing can be achieved by simply exchanging the data containers wrapped in shared pointers:

If you do not wish to share containers, but create a copy from an existing container, rather use the [QCPDataContainer<DataType>::set](#) method on the financial's data container directly:

See also

[addData](#), [timeSeriesToOhlc](#)

10.425.6.15 setPenNegative()

```
void QCPFinancial::setPenNegative (
    const QPen & pen )
```

If [setTwoColored](#) is set to true, this function controls the pen that is used to draw outlines of data points with a negative trend (i.e. bars/candlesticks with close < open).

If *twoColored* is false, the normal plottable's pen and brush are used ([setPen](#), [setBrush](#)).

See also

[setPenPositive](#), [setBrushNegative](#), [setBrushPositive](#)

10.425.6.16 setPenPositive()

```
void QCPFinancial::setPenPositive (
    const QPen & pen )
```

If [setTwoColored](#) is set to true, this function controls the pen that is used to draw outlines of data points with a positive trend (i.e. bars/candlesticks with close >= open).

If *twoColored* is false, the normal plottable's pen and brush are used ([setPen](#), [setBrush](#)).

See also

[setPenNegative](#), [setBrushPositive](#), [setBrushNegative](#)

10.425.6.17 setTwoColored()

```
void QCPFinancial::setTwoColored (
    bool twoColored )
```

Sets whether this chart shall contrast positive from negative trends per data point by using two separate colors to draw the respective bars/candlesticks.

If *twoColored* is false, the normal plottable's pen and brush are used ([setPen](#), [setBrush](#)).

See also

[setPenPositive](#), [setPenNegative](#), [setBrushPositive](#), [setBrushNegative](#)

10.425.6.18 setWidth()

```
void QCPFinancial::setWidth (
    double width )
```

Sets the width of the individual bars/candlesticks to *width* in plot key coordinates.

A typical choice is to set it to (or slightly less than) one bin interval width.

10.425.6.19 setWidthType()

```
void QCPFinancial::setWidthType (
    QCPFinancial::WidthType widthType )
```

Sets how the width of the financial bars is defined. See the documentation of [WidthType](#) for an explanation of the possible values for *widthType*.

The default value is [wtPlotCoords](#).

See also

[setWidth](#)

10.425.6.20 timeSeriesToOhlc()

```
QCPFinancialDataContainer QCPFinancial::timeSeriesToOhlc (
    const QVector< double > & time,
    const QVector< double > & value,
    double timeBinSize,
    double timeBinOffset = 0 ) [static]
```

A convenience function that converts time series data (*value* against *time*) to OHLC binned data points. The return value can then be passed on to `QCPFinancialDataContainer::set(const QCPFinancialDataContainer&)`.

The size of the bins can be controlled with *timeBinSize* in the same units as *time* is given. For example, if the unit of *time* is seconds and single OHLC/Candlesticks should span an hour each, set *timeBinSize* to 3600.

timeBinOffset allows to control precisely at what *time* coordinate a bin should start. The value passed as *timeBinOffset* doesn't need to be in the range encompassed by the *time* keys. It merely defines the mathematical offset/phase of the bins that will be used to process the data.

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.426 QCPFinancialData Class Reference

Holds the data of one single data point for [QCPFinancial](#).

Public Member Functions

- [QCPFinancialData](#) ()
- [QCPFinancialData](#) (double key, double open, double high, double low, double close)
- double [sortKey](#) () const
- double [mainKey](#) () const
- double [mainValue](#) () const
- [QCPRange](#) [valueRange](#) () const

Static Public Member Functions

- static [QCPFinancialData](#) [fromSortKey](#) (double [sortKey](#))
- static bool [sortKeysMainKey](#) ()

Public Attributes

- double **key**
- double **open**
- double **high**
- double **low**
- double **close**

10.426.1 Detailed Description

Holds the data of one single data point for [QCPFinancial](#).

The stored data is:

- *key*: coordinate on the key axis of this data point (this is the *mainKey* and the *sortKey*)
- *open*: The opening value at the data point (this is the *mainValue*)
- *high*: The high/maximum value at the data point
- *low*: The low/minimum value at the data point
- *close*: The closing value at the data point

The container for storing multiple data points is [QCPFinancialDataContainer](#). It is a typedef for [QCPDataContainer](#) with [QCPFinancialData](#) as the `DataType` template parameter. See the documentation there for an explanation regarding the data type's generic methods.

See also

[QCPFinancialDataContainer](#)

10.426.2 Constructor & Destructor Documentation

10.426.2.1 QCPFinancialData() [1/2]

```
QCPFinancialData::QCPFinancialData ( )
```

Constructs a data point with key and all values set to zero.

10.426.2.2 QCPFinancialData() [2/2]

```
QCPFinancialData::QCPFinancialData (
    double key,
    double open,
    double high,
    double low,
    double close )
```

Constructs a data point with the specified *key* and OHLC values.

10.426.3 Member Function Documentation

10.426.3.1 fromSortKey()

```
static QCPFinancialData QCPFinancialData::fromSortKey (
    double sortKey ) [inline], [static]
```

Returns a data point with the specified *sortKey*. All other members are set to zero.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.426.3.2 mainKey()

```
double QCPFinancialData::mainKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.426.3.3 mainValue()

```
double QCPFinancialData::mainValue ( ) const [inline]
```

Returns the *open* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.426.3.4 `sortKey()`

```
double QCPFinancialData::sortKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.426.3.5 `sortKeyIsMainKey()`

```
static static bool QCPFinancialData::sortKeyIsMainKey ( ) [inline], [static]
```

Since the member *key* is both the data point key coordinate and the data ordering parameter, this method returns true.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.426.3.6 `valueRange()`

```
QCPRange QCPFinancialData::valueRange ( ) const [inline]
```

Returns a [QCPRange](#) spanning from the *low* to the *high* value of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.427 QCPGraph Class Reference

A plottable representing a graph in a plot.

Public Member Functions

- [QCPGraph](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QSharedPointer](#)< [QCPGraphDataContainer](#) > [data](#) () const
- [LineStyle](#) [lineStyle](#) () const
- [QCPScatterStyle](#) [scatterStyle](#) () const
- int [scatterSkip](#) () const
- [QCPGraph](#) * [channelFillGraph](#) () const
- bool [adaptiveSampling](#) () const
- void [setData](#) ([QSharedPointer](#)< [QCPGraphDataContainer](#) > [data](#))
- void [setData](#) (const [QVector](#)< double > &keys, const [QVector](#)< double > &values, bool alreadySorted=false)
- void [setLineStyle](#) ([LineStyle](#) ls)
- void [setScatterStyle](#) (const [QCPScatterStyle](#) &style)
- void [setScatterSkip](#) (int skip)
- void [setChannelFillGraph](#) ([QCPGraph](#) *targetGraph)
- void [setAdaptiveSampling](#) (bool enabled)
- void [addData](#) (const [QVector](#)< double > &keys, const [QVector](#)< double > &values, bool alreadySorted=false)
- void [addData](#) (double key, double value)
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) [getKeyRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) [getValueRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from [QCPAbstractPlottable1D](#)< [QCPGraphData](#) >

- [QCPAbstractPlottable1D](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- virtual int [dataCount](#) () const [Q_DECL_OVERRIDE](#)
- virtual double [dataMainKey](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual double [dataSortKey](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual double [dataMainValue](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) [dataValueRange](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual [QPointF](#) [dataPixelPosition](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual bool [sortKeysIsMainKey](#) () const [Q_DECL_OVERRIDE](#)
- virtual [QCPDataSelection](#) [selectTestRect](#) (const [QRectF](#) &rect, bool onlySelectable) const [Q_DECL_OVERRIDE](#)
- virtual int [findBegin](#) (double sortKey, bool expandedRange=true) const [Q_DECL_OVERRIDE](#)
- virtual int [findEnd](#) (double sortKey, bool expandedRange=true) const [Q_DECL_OVERRIDE](#)
- virtual [QCPPlottableInterface1D](#) * [interface1D](#) () [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QString](#) [name](#) () const
- bool [antialiasedFill](#) () const
- bool [antialiasedScatters](#) () const
- [QPen](#) [pen](#) () const
- [QBrush](#) [brush](#) () const
- [QCPAxis](#) * [keyAxis](#) () const
- [QCPAxis](#) * [valueAxis](#) () const
- [QCP::SelectionType](#) [selectable](#) () const

- bool [selected](#) () const
- [QCPDataSelection](#) [selection](#) () const
- [QCPSelectionDecorator](#) * [selectionDecorator](#) () const
- void [setName](#) (const QString &name)
- void [setAntialiasedFill](#) (bool enabled)
- void [setAntialiasedScatters](#) (bool enabled)
- void [setPen](#) (const QPen &pen)
- void [setBrush](#) (const QBrush &brush)
- void [setKeyAxis](#) ([QCPAxis](#) *axis)
- void [setValueAxis](#) ([QCPAxis](#) *axis)
- Q_SLOT void [setSelectable](#) ([QCP::SelectionType](#) selectable)
- Q_SLOT void [setSelection](#) ([QCPDataSelection](#) selection)
- void [setSelectionDecorator](#) ([QCPSelectionDecorator](#) *decorator)
- void [coordsToPixels](#) (double key, double value, double &x, double &y) const
- const QPointF [coordsToPixels](#) (double key, double value) const
- void [pixelsToCoords](#) (double x, double y, double &key, double &value) const
- void [pixelsToCoords](#) (const QPointF &pixelPos, double &key, double &value) const
- void [rescaleAxes](#) (bool onlyEnlarge=false) const
- void [rescaleKeyAxis](#) (bool onlyEnlarge=false) const
- void [rescaleValueAxis](#) (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool [addToLegend](#) ([QCPLegend](#) *legend)
- bool [addToLegend](#) ()
- bool [removeFromLegend](#) ([QCPLegend](#) *legend) const
- bool [removeFromLegend](#) () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void [drawLegendIcon](#) ([QCPPainter](#) *painter, const QRectF &rect) const Q_DECL_OVERRIDE
- virtual void **drawFill** ([QCPPainter](#) *painter, QVector< QPointF > *lines) const
- virtual void **drawScatterPlot** ([QCPPainter](#) *painter, const QVector< QPointF > &scatters, const [QCPScatterStyle](#) &style) const
- virtual void **drawLinePlot** ([QCPPainter](#) *painter, const QVector< QPointF > &lines) const
- virtual void **drawImpulsePlot** ([QCPPainter](#) *painter, const QVector< QPointF > &lines) const
- virtual void **getOptimizedLineData** (QVector< [QCPGraphData](#) > *lineData, const [QCPGraphDataContainer::const_iterator](#) &begin, const [QCPGraphDataContainer::const_iterator](#) &end) const
- virtual void **getOptimizedScatterData** (QVector< [QCPGraphData](#) > *scatterData, [QCPGraphDataContainer::const_iterator](#) begin, [QCPGraphDataContainer::const_iterator](#) end) const

- void [setVisibleDataBounds](#) (QCPGraphDataContainer::const_iterator &begin, QCPGraphDataContainer::const_iterator &end, const [QCPDataRange](#) &rangeRestriction) const
- void **getLines** (QVector< QPointF > *lines, const [QCPDataRange](#) &dataRange) const
- void **getScatters** (QVector< QPointF > *scatters, const [QCPDataRange](#) &dataRange) const
- QVector< QPointF > **dataToLines** (const QVector< [QCPGraphData](#) > &data) const
- QVector< QPointF > **dataToStepLeftLines** (const QVector< [QCPGraphData](#) > &data) const
- QVector< QPointF > **dataToStepRightLines** (const QVector< [QCPGraphData](#) > &data) const
- QVector< QPointF > **dataToStepCenterLines** (const QVector< [QCPGraphData](#) > &data) const
- QVector< QPointF > **dataToImpulseLines** (const QVector< [QCPGraphData](#) > &data) const
- QVector< [QCPDataRange](#) > **getNonNaNSegments** (const QVector< QPointF > *lineData, Qt::Orientation keyOrientation) const
- QVector< QPair< [QCPDataRange](#), [QCPDataRange](#) > > **getOverlappingSegments** (QVector< [QCPDataRange](#) > thisSegments, const QVector< QPointF > *thisData, QVector< [QCPDataRange](#) > otherSegments, const QVector< QPointF > *otherData) const
- bool **segmentsIntersect** (double aLower, double aUpper, double bLower, double bUpper, int &bPrecedence) const
- QPointF **getFillBasePoint** (QPointF matchingDataPoint) const
- const QPolygonF **getFillPolygon** (const QVector< QPointF > *lineData, [QCPDataRange](#) segment) const
- const QPolygonF **getChannelFillPolygon** (const QVector< QPointF > *thisData, [QCPDataRange](#) thisSegment, const QVector< QPointF > *otherData, [QCPDataRange](#) otherSegment) const
- int **findIndexBelowX** (const QVector< QPointF > *data, double x) const
- int **findIndexAboveX** (const QVector< QPointF > *data, double x) const
- int **findIndexBelowY** (const QVector< QPointF > *data, double y) const
- int **findIndexAboveY** (const QVector< QPointF > *data, double y) const
- double **pointDistance** (const QPointF &pixelPoint, QCPGraphDataContainer::const_iterator &closestData) const

Protected Member Functions inherited from [QCPAbstractPlottable1D< QCPGraphData >](#)

- void [getDataSegments](#) (QList< [QCPDataRange](#) > &selectedSegments, QList< [QCPDataRange](#) > &unselectedSegments) const
- void [drawPolyline](#) (QCPPainter *painter, const QVector< QPointF > &lineData) const

Protected Member Functions inherited from [QCPAbstractPlottable](#)

- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- void [applyDefaultAntialiasingHint](#) (QCPPainter *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- void [applyFillAntialiasingHint](#) (QCPPainter *painter) const
- void [applyScattersAntialiasingHint](#) (QCPPainter *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** (QCPPainter *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [LineStyle](#) **mLineStyle**
- [QCPScatterStyle](#) **mScatterStyle**
- int **mScatterSkip**
- QPointer< [QCPGraph](#) > **mChannelFillGraph**
- bool **mAdaptiveSampling**

Protected Attributes inherited from [QCPAbstractPlottable1D](#)< [QCPGraphData](#) >

- QSharedPointer< [QCPDataContainer](#)< [QCPGraphData](#) > > **mDataContainer**

Protected Attributes inherited from [QCPAbstractPlottable](#)

- QString **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- QPen **mPen**
- QBrush **mBrush**
- QPointer< [QCPAxis](#) > **mKeyAxis**
- QPointer< [QCPAxis](#) > **mValueAxis**
- [QCP::SelectionType](#) **mSelectable**
- [QCPDataSelection](#) **mSelection**
- [QCPSelectionDecorator](#) * **mSelectionDecorator**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPLegend](#)

Additional Inherited Members

Signals inherited from [QCPAbstractPlottable](#)

- void [selectionChanged](#) (bool [selected](#))
- void [selectionChanged](#) (const [QCPDataSelection](#) &[selection](#))
- void [selectableChanged](#) ([QCP::SelectionType](#) [selectable](#))

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *[newLayer](#))

10.427.1 Detailed Description

A plottable representing a graph in a plot.

Usually you create new graphs by calling [QCustomPlot::addGraph](#). The resulting instance can be accessed via [QCustomPlot::graph](#).

To plot data, assign it with the [setData](#) or [addData](#) functions. Alternatively, you can also access and modify the data via the [data](#) method, which returns a pointer to the internal QCPGraphDataContainer.

Graphs are used to display single-valued data. Single-valued means that there should only be one data point per unique key coordinate. In other words, the graph can't have *loops*. If you do want to plot non-single-valued curves, rather use the [QCPCurve](#) plottable.

Gaps in the graph line can be created by adding data points with NaN as value (`qQNaN()` or `std::numeric_limits<double>::quiet_NaN()`) in between the two data points that shall be separated.

10.427.2 Changing the appearance

The appearance of the graph is mainly determined by the line style, scatter style, brush and pen of the graph ([setLineStyle](#), [setScatterStyle](#), [setBrush](#), [setPen](#)).

10.427.2.1 Filling under or between graphs

[QCPGraph](#) knows two types of fills: Normal graph fills towards the zero-value-line parallel to the key axis of the graph, and fills between two graphs, called channel fills. To enable a fill, just set a brush with [setBrush](#) which is neither Qt::NoBrush nor fully transparent.

By default, a normal fill towards the zero-value-line will be drawn. To set up a channel fill between this graph and another one, call [setChannelFillGraph](#) with the other graph as parameter.

See also

[QCustomPlot::addGraph](#), [QCustomPlot::graph](#)

10.427.3 Member Enumeration Documentation

10.427.3.1 LineStyle

```
enum QCPGraph::LineStyle
```

Defines how the graph's line is represented visually in the plot. The line is drawn with the current pen of the graph ([setPen](#)).

See also

[setLineStyle](#)

Enumerator

IsNone	with symbols according to the scatter style, see setScatterStyle) data points are not connected with any lines (e.g. data only represented)
IsLine	data points are connected by a straight line
IsStepLeft	line is drawn as steps where the step height is the value of the left data point
IsStepRight	line is drawn as steps where the step height is the value of the right data point
IsStepCenter	line is drawn as steps where the step is in between two data points
IsImpulse	each data point is represented by a line parallel to the value axis, which reaches from the data point to the zero-value-line

10.427.4 Constructor & Destructor Documentation

10.427.4.1 QCPGraph()

```
QCPGraph::QCPGraph (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis ) [explicit]
```

Constructs a graph which uses *keyAxis* as its key axis ("x") and *valueAxis* as its value axis ("y"). *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#) instance and not have the same orientation. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

The created [QCPGraph](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPGraph](#), so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead.

To directly create a graph inside a plot, you can also use the simpler [QCustomPlot::addGraph](#) function.

10.427.5 Member Function Documentation

10.427.5.1 addData() [1/2]

```
void QCPGraph::addData (
    const QVector< double > & keys,
    const QVector< double > & values,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided points in *keys* and *values* to the current data. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

10.427.5.2 addData() [2/2]

```
void QCPGraph::addData (
    double key,
    double value )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided data point as *key* and *value* to the current data.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

10.427.5.3 data()

```
QSharedPointer< QCPGraphDataContainer > QCPGraph::data ( ) const [inline]
```

Returns a shared pointer to the internal data storage of type `QCPGraphDataContainer`. You may use it to directly manipulate the data, which may be more convenient and faster than using the regular [setData](#) or [addData](#) methods.

10.427.5.4 draw()

```
void QCPGraph::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

10.427.5.5 drawLegendIcon()

```
void QCPGraph::drawLegendIcon (
    QCPPainter * painter,
    const QRectF & rect ) const [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

10.427.5.6 getKeyRange()

```
QCPRange QCPGraph::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implements [QCPAbstractPlottable](#).

10.427.5.7 getValueRange()

```
QCPRange QCPGraph::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implements [QCPAbstractPlottable](#).

10.427.5.8 getVisibleDataBounds()

```
void QCPGraph::getVisibleDataBounds (
    QCPGraphDataContainer::const_iterator & begin,
    QCPGraphDataContainer::const_iterator & end,
    const QCPDataRange & rangeRestriction ) const [protected]
```

This method outputs the currently visible data range via *begin* and *end*. The returned range will also never exceed *rangeRestriction*.

This method takes into account that the drawing of data lines at the axis rect border always requires the points just outside the visible axis range. So *begin* and *end* may actually indicate a range that contains one additional data point to the left and right of the visible axis range.

10.427.5.9 selectTest()

```
double QCPGraph::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Implements a *selectTest* specific to this plottable's point geometry.

If *details* is not 0, it will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

\see [baseclassmethod QCPAbstractPlottable::selectTest](#)

Reimplemented from [QCPAbstractPlottable1D< QCPGraphData >](#).

10.427.5.10 setAdaptiveSampling()

```
void QCPGraph::setAdaptiveSampling (
    bool enabled )
```

Sets whether adaptive sampling shall be used when plotting this graph. [QCustomPlot](#)'s adaptive sampling technique can drastically improve the replot performance for graphs with a larger number of points (e.g. above 10,000), without notably changing the appearance of the graph.

By default, adaptive sampling is enabled. Even if enabled, [QCustomPlot](#) decides whether adaptive sampling shall actually be used on a per-graph basis. So leaving adaptive sampling enabled has no disadvantage in almost all cases.

As can be seen, line plots experience no visual degradation from adaptive sampling. Outliers are reproduced reliably, as well as the overall shape of the data set. The replot time reduces dramatically though. This allows [QCustomPlot](#) to display large amounts of data in realtime.

Care must be taken when using high-density scatter plots in combination with adaptive sampling. The adaptive sampling algorithm treats scatter plots more carefully than line plots which still gives a significant reduction of replot times, but not quite as much as for line plots. This is because scatter plots inherently need more data points to be preserved in order to still resemble the original, non-adaptive-sampling plot. As shown above, the results still aren't quite identical, as banding occurs for the outer data points. This is in fact intentional, such that the boundaries of the data cloud stay visible to the viewer. How strong the banding appears, depends on the point density, i.e. the number of points in the plot.

For some situations with scatter plots it might thus be desirable to manually turn adaptive sampling off. For example, when saving the plot to disk. This can be achieved by setting *enabled* to false before issuing a command like [QCustomPlot::savePng](#), and setting *enabled* back to true afterwards.

10.427.5.11 setChannelFillGraph()

```
void QCPGraph::setChannelFillGraph (
    QCPGraph * targetGraph )
```

Sets the target graph for filling the area between this graph and *targetGraph* with the current brush ([setBrush](#)).

When *targetGraph* is set to 0, a normal graph fill to the zero-value-line will be shown. To disable any filling, set the brush to `Qt::NoBrush`.

See also

[setBrush](#)

10.427.5.12 setData() [1/2]

```
void QCPGraph::setData (
    const QVector< double > & keys,
    const QVector< double > & values,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data with the provided points in *keys* and *values*. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

See also

[addData](#)

10.427.5.13 setData() [2/2]

```
void QCPGraph::setData (
    QSharedPointer< QCPGraphDataContainer > data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data container with the provided *data* container.

Since a QSharedPointer is used, multiple QCPGraphs may share the same data container safely. Modifying the data in the container will then affect all graphs that share the container. Sharing can be achieved by simply exchanging the data containers wrapped in shared pointers:

If you do not wish to share containers, but create a copy from an existing container, rather use the [QCPDataContainer<DataType>::set](#) method on the graph's data container directly:

See also

[addData](#)

10.427.5.14 setLineStyle()

```
void QCPGraph::setLineStyle (
    LineStyle ls )
```

Sets how the single data points are connected in the plot. For scatter-only plots, set *ls* to [lsNone](#) and [setScatterStyle](#) to the desired scatter style.

See also

[setScatterStyle](#)

10.427.5.15 setScatterSkip()

```
void QCPGraph::setScatterSkip (
    int skip )
```

If scatters are displayed (scatter style not [QCPScatterStyle::ssNone](#)), *skip* number of scatter points are skipped/not drawn after every drawn scatter point.

This can be used to make the data appear sparser while for example still having a smooth line, and to improve performance for very high density plots.

If *skip* is set to 0 (default), all scatter points are drawn.

See also

[setScatterStyle](#)

10.427.5.16 setScatterStyle()

```
void QCPGraph::setScatterStyle (
    const QCPScatterStyle & style )
```

Sets the visual appearance of single data points in the plot. If set to [QCPScatterStyle::ssNone](#), no scatter points are drawn (e.g. for line-only-plots with appropriate line style).

See also

[QCPScatterStyle](#), [setLineStyle](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.428 QCPGraphData Class Reference

Holds the data of one single data point for [QCPGraph](#).

Public Member Functions

- [QCPGraphData](#) ()
- [QCPGraphData](#) (double key, double value)
- double [sortKey](#) () const
- double [mainKey](#) () const
- double [mainValue](#) () const
- [QCPRange](#) [valueRange](#) () const

Static Public Member Functions

- static [QCPGraphData](#) [fromSortKey](#) (double [sortKey](#))
- static bool [sortKeysIsMainKey](#) ()

Public Attributes

- double **key**
- double **value**

10.428.1 Detailed Description

Holds the data of one single data point for [QCPGraph](#).

The stored data is:

- *key*: coordinate on the key axis of this data point (this is the *mainKey* and the *sortKey*)
- *value*: coordinate on the value axis of this data point (this is the *mainValue*)

The container for storing multiple data points is [QCPGraphDataContainer](#). It is a typedef for [QCPDataContainer](#) with [QCPGraphData](#) as the `DataType` template parameter. See the documentation there for an explanation regarding the data type's generic methods.

See also

[QCPGraphDataContainer](#)

10.428.2 Constructor & Destructor Documentation

10.428.2.1 QCPGraphData() [1/2]

```
QCPGraphData::QCPGraphData ( )
```

Constructs a data point with key and value set to zero.

10.428.2.2 QCPGraphData() [2/2]

```
QCPGraphData::QCPGraphData (
    double key,
    double value )
```

Constructs a data point with the specified *key* and *value*.

10.428.3 Member Function Documentation

10.428.3.1 fromSortKey()

```
static QCPGraphData QCPGraphData::fromSortKey (
    double sortKey ) [inline], [static]
```

Returns a data point with the specified *sortKey*. All other members are set to zero.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.428.3.2 mainKey()

```
double QCPGraphData::mainKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.428.3.3 mainValue()

```
double QCPGraphData::mainValue ( ) const [inline]
```

Returns the *value* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.428.3.4 sortKey()

```
double QCPGraphData::sortKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.428.3.5 sortKeyIsMainKey()

```
static static bool QCPGraphData::sortKeyIsMainKey ( ) [inline], [static]
```

Since the member *key* is both the data point key coordinate and the data ordering parameter, this method returns true.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.428.3.6 valueRange()

```
QCPRange QCPGraphData::valueRange ( ) const [inline]
```

Returns a [QCPRange](#) with both lower and upper boundary set to *value* of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

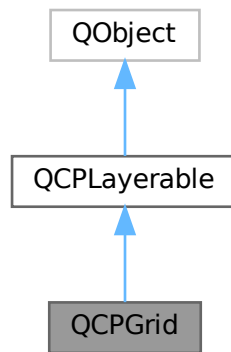
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

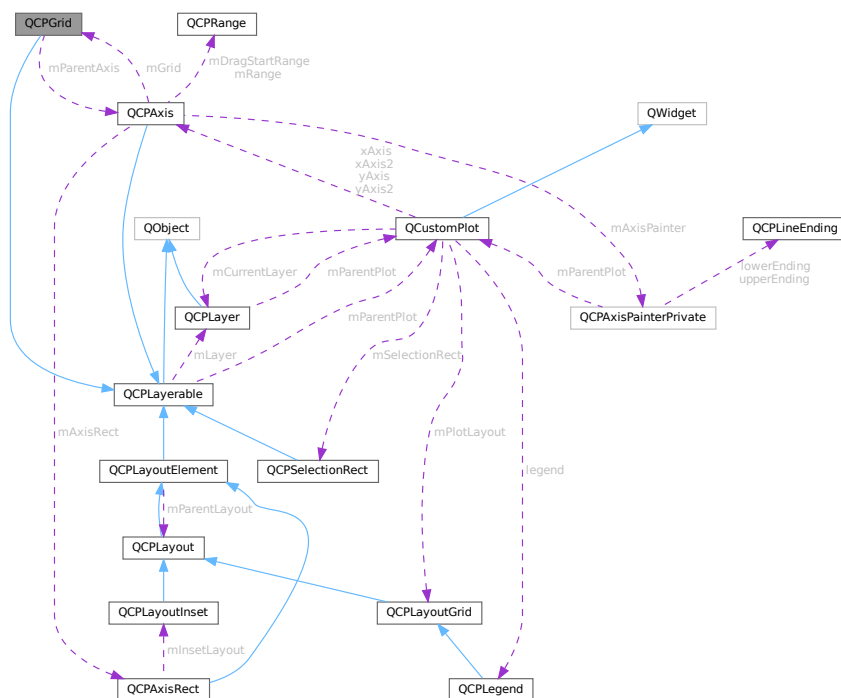
10.429 QCPGrid Class Reference

Responsible for drawing the grid of a [QCPAxis](#).

Inheritance diagram for QCPGrid:



Collaboration diagram for QCPGrid:



Public Member Functions

- [QCPGrid](#) ([QCPAxis](#) *parentAxis)
- bool **subGridVisible** () const
- bool **antialiasedSubGrid** () const
- bool **antialiasedZeroLine** () const
- [QPen](#) **pen** () const
- [QPen](#) **subGridPen** () const
- [QPen](#) **zeroLinePen** () const
- void [setSubGridVisible](#) (bool visible)
- void [setAntialiasedSubGrid](#) (bool enabled)
- void [setAntialiasedZeroLine](#) (bool enabled)
- void [setPen](#) (const [QPen](#) &pen)
- void [setSubGridPen](#) (const [QPen](#) &pen)
- void [setZeroLinePen](#) (const [QPen](#) &pen)

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const [QString](#) &layerName)
- void [setAntialiased](#) (bool enabled)
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- void **drawGridLines** ([QCPPainter](#) *painter) const
- void **drawSubGridLines** ([QCPPainter](#) *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual [QRect](#) **clipRect** () const
- virtual void **selectEvent** ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState↔ Changed)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void [mousePressEvent](#) ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void [mouseMoveEvent](#) ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void [mouseReleaseEvent](#) ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void [mouseDoubleClickEvent](#) ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void [wheelEvent](#) ([QWheelEvent](#) *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- bool **mSubGridVisible**
- bool **mAntialiasedSubGrid**
- bool **mAntialiasedZeroLine**
- QPen **mPen**
- QPen **mSubGridPen**
- QPen **mZeroLinePen**
- [QCPAxis](#) * **mParentAxis**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCPAxis](#)

Additional Inherited Members**Signals inherited from [QCPLayerable](#)**

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.429.1 Detailed Description

Responsible for drawing the grid of a [QCPAxis](#).

This class is tightly bound to [QCPAxis](#). Every axis owns a grid instance and uses it to draw the grid lines, sub grid lines and zero-line. You can interact with the grid of an axis via [QCPAxis::grid](#). Normally, you don't need to create an instance of [QCPGrid](#) yourself.

The axis and grid drawing was split into two classes to allow them to be placed on different layers (both [QCPAxis](#) and [QCPGrid](#) inherit from [QCPLayerable](#)). Thus it is possible to have the grid in the background and the axes in the foreground, and any plottables/items in between. This described situation is the default setup, see the [QCPLayer](#) documentation.

10.429.2 Constructor & Destructor Documentation**10.429.2.1 QCPGrid()**

```
QCPGrid::QCPGrid (
    QCPAxis * parentAxis ) [explicit]
```

Creates a [QCPGrid](#) instance and sets default values.

You shouldn't instantiate grids on their own, since every [QCPAxis](#) brings its own [QCPGrid](#).

10.429.3 Member Function Documentation

10.429.3.1 `applyDefaultAntialiasingHint()`

```
void QCPGrid::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

10.429.3.2 `draw()`

```
void QCPGrid::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPLayerable](#).

10.429.3.3 `setAntialiasedSubGrid()`

```
void QCPGrid::setAntialiasedSubGrid (
    bool enabled )
```

Sets whether sub grid lines are drawn antialiased.

10.429.3.4 `setAntialiasedZeroLine()`

```
void QCPGrid::setAntialiasedZeroLine (
    bool enabled )
```

Sets whether zero lines are drawn antialiased.

10.429.3.5 `setPen()`

```
void QCPGrid::setPen (
    const QPen & pen )
```

Sets the pen with which (major) grid lines are drawn.

10.429.3.6 `setSubGridPen()`

```
void QCPGrid::setSubGridPen (
    const QPen & pen )
```

Sets the pen with which sub grid lines are drawn.

10.429.3.7 setSubGridVisible()

```
void QCPGrid::setSubGridVisible (
    bool visible )
```

Sets whether grid lines at sub tick marks are drawn.

See also

[setSubGridPen](#)

10.429.3.8 setZeroLinePen()

```
void QCPGrid::setZeroLinePen (
    const QPen & pen )
```

Sets the pen with which zero lines are drawn.

Zero lines are lines at value coordinate 0 which may be drawn with a different pen than other grid lines. To disable zero lines and just draw normal grid lines at zero, set *pen* to Qt::NoPen.

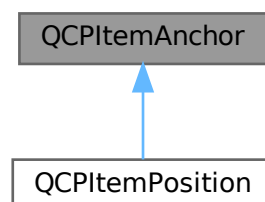
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

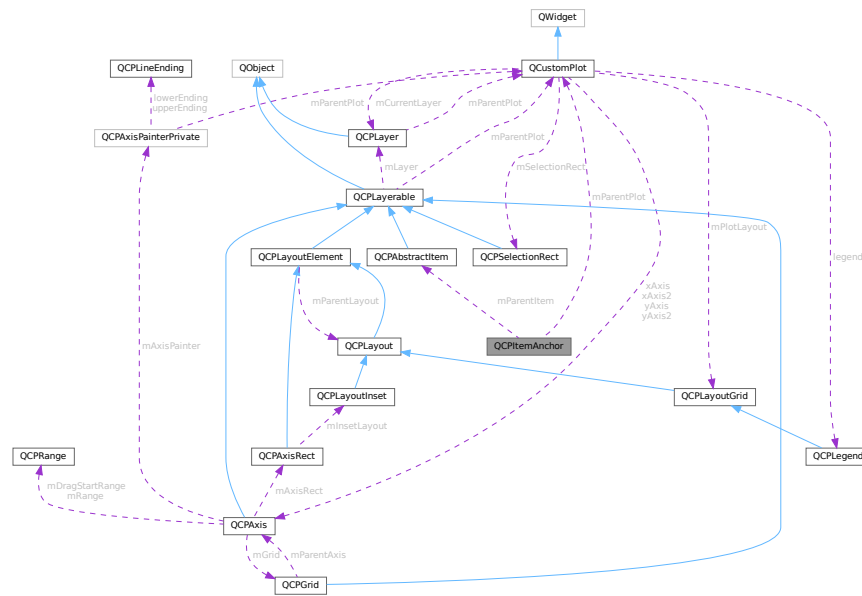
10.430 QCPLItemAnchor Class Reference

An anchor of an item to which positions can be attached to.

Inheritance diagram for QCPLItemAnchor:



Collaboration diagram for QCPItemAnchor:



Public Member Functions

- **QCPItemAnchor** (**QCustomPlot** *parentPlot, **QCPAbstractItem** *parentItem, const QString &name, int anchorId=-1)
- QString **name** () const
- virtual QPointF **pixelPosition** () const

Protected Member Functions

- virtual **QCPItemPosition** * **toQCPItemPosition** ()
- void **addChildX** (**QCPItemPosition** *pos)
- void **removeChildX** (**QCPItemPosition** *pos)
- void **addChildY** (**QCPItemPosition** *pos)
- void **removeChildY** (**QCPItemPosition** *pos)

Protected Attributes

- QString **mName**
- **QCustomPlot** * **mParentPlot**
- **QCPAbstractItem** * **mParentItem**
- int **mAnchorId**
- QSet< **QCPItemPosition** * > **mChildrenX**
- QSet< **QCPItemPosition** * > **mChildrenY**

Friends

- class **QCPItemPosition**

10.430.1 Detailed Description

An anchor of an item to which positions can be attached to.

An item ([QCPAbstractItem](#)) may have one or more anchors. Unlike [QCPItemPosition](#), an anchor doesn't control anything on its item, but provides a way to tie other items via their positions to the anchor.

For example, a [QCPItemRect](#) is defined by its positions *topLeft* and *bottomRight*. Additionally it has various anchors like *top*, *topRight* or *bottomLeft* etc. So you can attach the *start* (which is a [QCPItemPosition](#)) of a [QCPItemLine](#) to one of the anchors by calling [QCPItemPosition::setParentAnchor](#) on *start*, passing the wanted anchor of the [QCPItemRect](#). This way the start of the line will now always follow the respective anchor location on the rect item.

Note that [QCPItemPosition](#) derives from [QCPItemAnchor](#), so every position can also serve as an anchor to other positions.

To learn how to provide anchors in your own item subclasses, see the subclassing section of the [QCPAbstractItem](#) documentation.

10.430.2 Constructor & Destructor Documentation

10.430.2.1 QCPItemAnchor()

```
QCPItemAnchor::QCPItemAnchor (
    QCustomPlot * parentPlot,
    QCPAbstractItem * parentItem,
    const QString & name,
    int anchorId = -1 )
```

Creates a new [QCPItemAnchor](#). You shouldn't create [QCPItemAnchor](#) instances directly, even if you want to make a new item subclass. Use [QCPAbstractItem::createAnchor](#) instead, as explained in the subclassing section of the [QCPAbstractItem](#) documentation.

10.430.3 Member Function Documentation

10.430.3.1 pixelPosition()

```
QPointF QCPItemAnchor::pixelPosition ( ) const [virtual]
```

Returns the final absolute pixel position of the [QCPItemAnchor](#) on the [QCustomPlot](#) surface.

The pixel information is internally retrieved via [QCPAbstractItem::anchorPixelPosition](#) of the parent item, [QCPItemAnchor](#) is just an intermediary.

Reimplemented in [QCPItemPosition](#).

10.430.3.2 toQCItemPosition()

```
QCItemPosition * QCItemAnchor::toQCItemPosition ( ) [inline], [protected], [virtual]
```

Returns `nullptr` if this instance is merely a [QCItemAnchor](#), and a valid pointer of type `QCItemPosition*` if it actually is a [QCItemPosition](#) (which is a subclass of [QCItemAnchor](#)).

This safe downcast functionality could also be achieved with a `dynamic_cast`. However, [QCustomPlot](#) avoids `dynamic_cast` to work with projects that don't have RTTI support enabled (e.g. `-fno-rtti` flag with gcc compiler).

Reimplemented in [QCItemPosition](#).

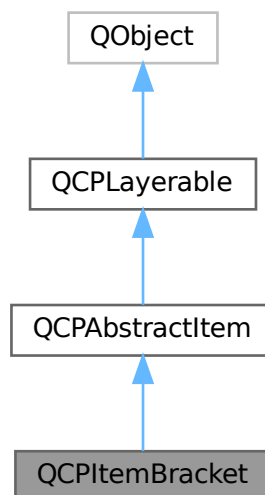
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

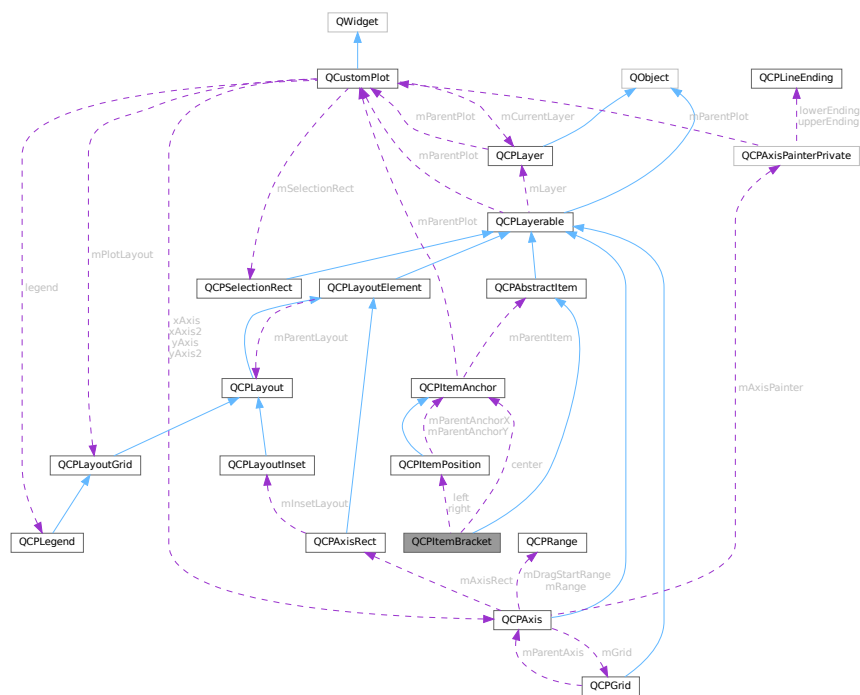
10.431 QCItemBracket Class Reference

A bracket for referencing/highlighting certain parts in the plot.

Inheritance diagram for `QCItemBracket`:



Collaboration diagram for QCPItemBracket:



Public Types

- enum BracketStyle { bsSquare , bsRound , bsCurly , bsCalligraphic }

Public Member Functions

- [QCPItemBracket](#) ([QCustomPlot](#) *parentPlot)
- QPen **pen** () const
- QPen **selectedPen** () const
- double **length** () const
- [BracketStyle](#) **style** () const
- void [setPen](#) (const QPen &pen)
- void [setSelectedPen](#) (const QPen &pen)
- void [setLength](#) (double length)
- void [setStyle](#) ([BracketStyle](#) style)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_
DECL OVERRIDE

Public Member Functions inherited from QCPAbstractItem

- **QCPAbstractItem** (**QCustomPlot** *parentPlot)
- bool **clipToAxisRect** () const
- **QCPAxisRect** * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void **setClipToAxisRect** (bool clip)

- void [setClipAxisRect](#) ([QCPAxisRect](#) *rect)
- Q_SLOT void [setSelectable](#) (bool selectable)
- Q_SLOT void [setSelected](#) (bool selected)
- QList< [QCItemPosition](#) * > [positions](#) () const
- QList< [QCItemAnchor](#) * > [anchors](#) () const
- [QCItemPosition](#) * [position](#) (const QString &name) const
- [QCItemAnchor](#) * [anchor](#) (const QString &name) const
- bool [hasAnchor](#) (const QString &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Public Attributes

- [QCItemPosition](#) *const **left**
- [QCItemPosition](#) *const **right**
- [QCItemAnchor](#) *const **center**

Protected Types

- enum **AnchorIndex** { **aiCenter** }

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual QPointF [anchorPixelPosition](#) (int anchorId) const Q_DECL_OVERRIDE
- QPen **mainPen** () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionState←Changed) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- double **rectDistance** (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCItemPosition](#) * **createPosition** (const QString &name)
- [QCItemAnchor](#) * **createAnchor** (const QString &name, int anchorId)

Protected Member Functions inherited from QCPLayerable

- virtual void **parentPlotInitialized** (QCustomPlot *parentPlot)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** (QCustomPlot *parentPlot)
- void **setParentLayerable** (QCPLayerable *parentLayerable)
- bool **moveToLayer** (QCPLayer *layer, bool prepend)
- void **applyAntialiasingHint** (QCPPainter *painter, bool localAntialiased, QCP::AntialiasedElement overrideElement) const

Protected Attributes

- QPen **mPen**
- QPen **mSelectedPen**
- double **mLength**
- BracketStyle **mStyle**

Protected Attributes inherited from QCPAbstractItem

- bool **mClipToAxisRect**
- QPointer< QCPAxisRect > **mClipAxisRect**
- QList< QCPLItemPosition * > **mPositions**
- QList< QCPLItemAnchor * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from QCPLayerable

- bool **mVisible**
- QCustomPlot * **mParentPlot**
- QPointer< QCPLayerable > **mParentLayerable**
- QCPLayer * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from QCPAbstractItem

- void **selectionChanged** (bool selected)
- void **selectableChanged** (bool selectable)

Signals inherited from QCPLayerable

- void **layerChanged** (QCPLayer *newLayer)

10.431.1 Detailed Description

A bracket for referencing/highlighting certain parts in the plot.

It has two positions, *left* and *right*, which define the span of the bracket. If *left* is actually farther to the left than *right*, the bracket is opened to the bottom, as shown in the example image.

The bracket supports multiple styles via [setStyle](#). The length, i.e. how far the bracket stretches away from the embraced span, can be controlled with [setLength](#).

Demonstrating the effect of different values for [setLength](#), for styles [bsCalligraphic](#) and [bsSquare](#). Anchors and positions are displayed for reference.

It provides an anchor *center*, to allow connection of other items, e.g. an arrow ([QCPItemLine](#) or [QCPItemCurve](#)) or a text label ([QCPItemText](#)), to the bracket.

10.431.2 Member Enumeration Documentation

10.431.2.1 BracketStyle

enum [QCPItemBracket::BracketStyle](#)

Defines the various visual shapes of the bracket item. The appearance can be further modified by [setLength](#) and [setPen](#).

See also

[setStyle](#)

Enumerator

bsSquare	A brace with angled edges.
bsRound	A brace with round edges.
bsCurly	A curly brace.
bsCalligraphic	A curly brace with varying stroke width giving a calligraphic impression.

10.431.3 Constructor & Destructor Documentation

10.431.3.1 QCPItemBracket()

```
QCPItemBracket::QCPItemBracket (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a bracket item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

10.431.4 Member Function Documentation

10.431.4.1 anchorPixelPosition()

```
QPointF QCPItemBracket::anchorPixelPosition (
    int anchorId ) const [protected], [virtual]
```

Reimplemented from [QCPAbstractItem](#).

10.431.4.2 draw()

```
void QCPItemBracket::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

10.431.4.3 selectTest()

```
double QCPItemBracket::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the mouseReleaseEvent occurs, and the finally selected object is notified via the selectEvent/ deselectEvent methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to selectEvent when the parent [QCustomPlot](#) decides on the basis of this selectTest call, that the object was successfully selected. The subsequent call to selectEvent will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent selectEvent, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

selectEvent, deselectEvent, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

10.431.4.4 `setLength()`

```
void QCPItemBracket::setLength (
    double length )
```

Sets the *length* in pixels how far the bracket extends in the direction towards the embraced span of the bracket (i.e. perpendicular to the *left-right*-direction)

Demonstrating the effect of different values for [setLength](#), for styles [bsCalligraphic](#) and [bsSquare](#). Anchors and positions are displayed for reference.

10.431.4.5 `setPen()`

```
void QCPItemBracket::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the bracket.

Note that when the style is [bsCalligraphic](#), only the color will be taken from the pen, the stroke and width are ignored. To change the apparent stroke width of a calligraphic bracket, use [setLength](#), which has a similar effect.

See also

[setSelectedPen](#)

10.431.4.6 `setSelectedPen()`

```
void QCPItemBracket::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the bracket when selected

See also

[setPen](#), [setSelected](#)

10.431.4.7 `setStyle()`

```
void QCPItemBracket::setStyle (
    QCPItemBracket::BracketStyle style )
```

Sets the style of the bracket, i.e. the shape/visual appearance.

See also

[setPen](#)

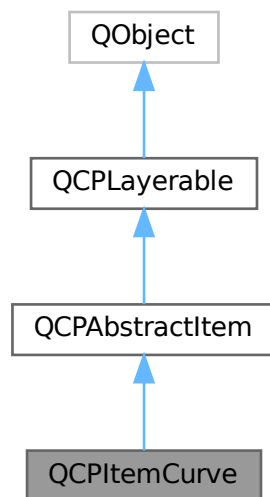
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

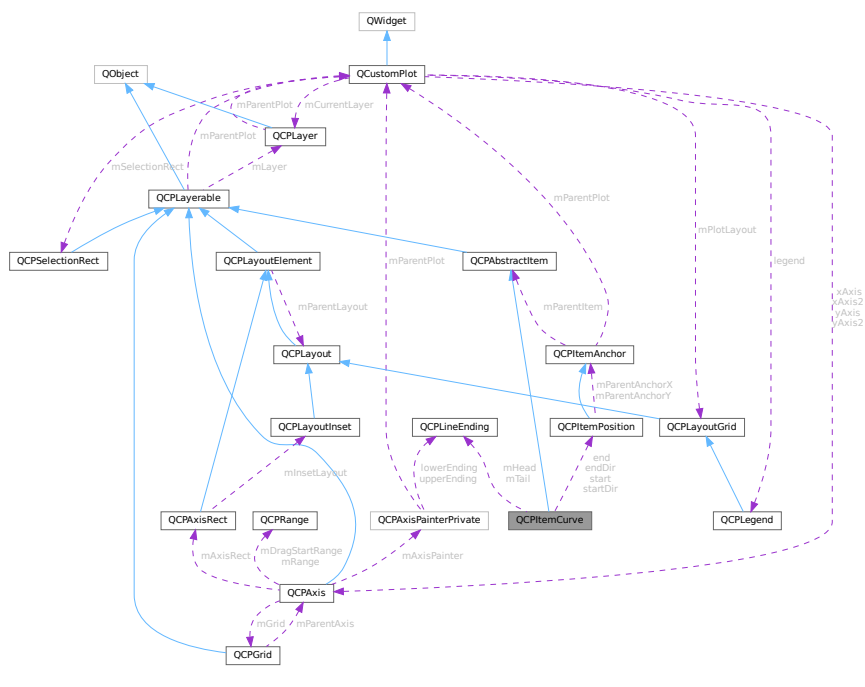
10.432 QCPLItemCurve Class Reference

A curved line from one point to another.

Inheritance diagram for QCPLItemCurve:



Collaboration diagram for QCPLItemCurve:



Public Member Functions

- [QCItemCurve](#) ([QCustomPlot](#) *parentPlot)
- [QPen](#) **pen** () const
- [QPen](#) **selectedPen** () const
- [QCPLineEnding](#) **head** () const
- [QCPLineEnding](#) **tail** () const
- void [setPen](#) (const [QPen](#) &pen)
- void [setSelectedPen](#) (const [QPen](#) &pen)
- void [setHead](#) (const [QCPLineEnding](#) &head)
- void [setTail](#) (const [QCPLineEnding](#) &tail)
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_↔](#)
DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool **clipToAxisRect** () const
- [QCPAxisRect](#) * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void [setClipToAxisRect](#) (bool clip)
- void [setClipAxisRect](#) ([QCPAxisRect](#) *rect)
- [Q_SLOT](#) void [setSelectable](#) (bool selectable)
- [Q_SLOT](#) void [setSelected](#) (bool selected)
- [QList](#)< [QCItemPosition](#) * > **positions** () const
- [QList](#)< [QCItemAnchor](#) * > **anchors** () const
- [QCItemPosition](#) * **position** (const [QString](#) &name) const
- [QCItemAnchor](#) * **anchor** (const [QString](#) &name) const
- bool **hasAnchor** (const [QString](#) &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- [Q_SLOT](#) bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const [QString](#) &layerName)
- void [setAntialiased](#) (bool enabled)
- bool **realVisibility** () const

Public Attributes

- [QCItemPosition](#) *const **start**
- [QCItemPosition](#) *const **startDir**
- [QCItemPosition](#) *const **endDir**
- [QCItemPosition](#) *const **end**

Protected Member Functions

- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- QPen **mainPen** () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- virtual QRect **clipRect** () const Q_DECL_OVERRIDE
- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual QPointF **anchorPixelPosition** (int anchorId) const
- double **rectDistance** (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCPItemPosition](#) * **createPosition** (const QString &name)
- [QCPItemAnchor](#) * **createAnchor** (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QPen **mPen**
- QPen **mSelectedPen**
- [QCPLineEnding](#) **mHead**
- [QCPLineEnding](#) **mTail**

Protected Attributes inherited from [QCPAbstractItem](#)

- bool **mClipToAxisRect**
- QPointer< [QCPAxisRect](#) > **mClipAxisRect**
- QList< [QCPItemPosition](#) * > **mPositions**
- QList< [QCPItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer](#)< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void [selectionChanged](#) (bool selected)
- void **selectableChanged** (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.432.1 Detailed Description

A curved line from one point to another.

It has four positions, *start* and *end*, which define the end points of the line, and two control points which define the direction the line exits from the start and the direction from which it approaches the end: *startDir* and *endDir*.

With [setHead](#) and [setTail](#) you may set different line ending styles, e.g. to create an arrow.

Often it is desirable for the control points to stay at fixed relative positions to the start/end point. This can be achieved by setting the parent anchor e.g. of *startDir* simply to *start*, and then specify the desired pixel offset with [QCPLItemPosition::setCoords](#) on *startDir*.

10.432.2 Constructor & Destructor Documentation

10.432.2.1 QCPLItemCurve()

```
QCPLItemCurve::QCPLItemCurve (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a curve item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

10.432.3 Member Function Documentation

10.432.3.1 draw()

```
void QCPLItemCurve::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

10.432.3.2 selectTest()

```
double QCPLItemCurve::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPLItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the `mouseReleaseEvent` occurs, and the finally selected object is notified via the `selectEvent/` `deselectEvent` methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to `selectEvent` when the parent [QCustomPlot](#) decides on the basis of this `selectTest` call, that the object was successfully selected. The subsequent call to `selectEvent` will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in `selectTest`. The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent `selectEvent`, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

`selectEvent`, `deselectEvent`, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

10.432.3.3 setHead()

```
void QCPLItemCurve::setHead (
    const QCPLLineEnding & head )
```

Sets the line ending style of the head. The head corresponds to the *end* position.

Note that due to the overloaded [QCPLLineEnding](#) constructor, you may directly specify a [QCPLLineEnding::EndingStyle](#) here, e.g.

```
setHead(QCPLLineEnding::esSpikeArrow)
```

See also

[setTail](#)

10.432.3.4 `setPen()`

```
void QCPItemCurve::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line

See also

[setSelectedPen](#)

10.432.3.5 `setSelectedPen()`

```
void QCPItemCurve::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line when selected

See also

[setPen](#), [setSelected](#)

10.432.3.6 `setTail()`

```
void QCPItemCurve::setTail (
    const QCPLLineEnding & tail )
```

Sets the line ending style of the tail. The tail corresponds to the *start* position.

Note that due to the overloaded [QCPLLineEnding](#) constructor, you may directly specify a [QCPLLineEnding::EndingStyle](#) here, e.g.

```
setTail(QCPLLineEnding::esSpikeArrow)
```

See also

[setHead](#)

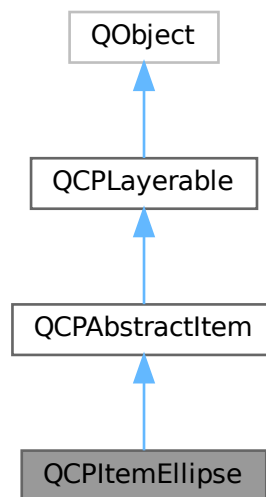
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

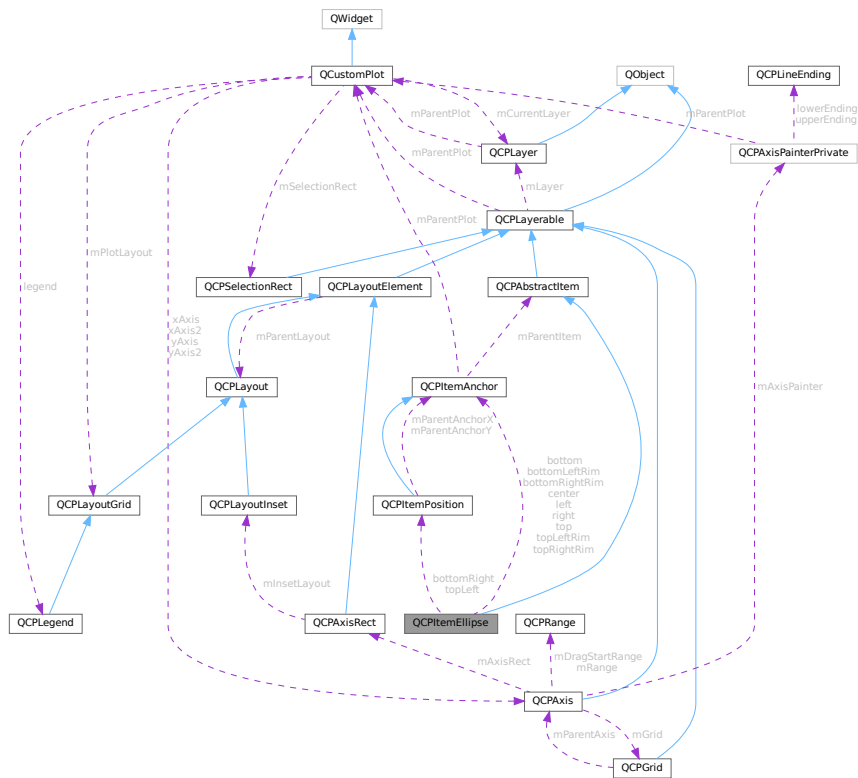
10.433 QCPItemEllipse Class Reference

An ellipse.

Inheritance diagram for QCPItemEllipse:



Collaboration diagram for QCPItemEllipse:



Public Member Functions

- [QCPItemEllipse](#) ([QCustomPlot](#) *parentPlot)
- QPen **pen** () const
- QPen **selectedPen** () const
- QBrush **brush** () const
- QBrush **selectedBrush** () const
- void [setPen](#) (const QPen &pen)
- void [setSelectedPen](#) (const QPen &pen)
- void [setBrush](#) (const QBrush &brush)
- void [setSelectedBrush](#) (const QBrush &brush)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_
DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool **clipToAxisRect** () const
- [QCPAxisRect](#) * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void [setClipToAxisRect](#) (bool clip)
- void [setClipAxisRect](#) ([QCPAxisRect](#) *rect)
- Q_SLOT void [setSelectable](#) (bool selectable)

- Q_SLOT void [setSelected](#) (bool selected)
- QList< [QCPLItemPosition](#) * > [positions](#) () const
- QList< [QCPLItemAnchor](#) * > [anchors](#) () const
- [QCPLItemPosition](#) * [position](#) (const QString &name) const
- [QCPLItemAnchor](#) * [anchor](#) (const QString &name) const
- bool [hasAnchor](#) (const QString &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Public Attributes

- [QCPLItemPosition](#) *const **topLeft**
- [QCPLItemPosition](#) *const **bottomRight**
- [QCPLItemAnchor](#) *const **topLeftRim**
- [QCPLItemAnchor](#) *const **top**
- [QCPLItemAnchor](#) *const **topRightRim**
- [QCPLItemAnchor](#) *const **right**
- [QCPLItemAnchor](#) *const **bottomRightRim**
- [QCPLItemAnchor](#) *const **bottom**
- [QCPLItemAnchor](#) *const **bottomLeftRim**
- [QCPLItemAnchor](#) *const **left**
- [QCPLItemAnchor](#) *const **center**

Protected Types

- enum **AnchorIndex** {
aiTopLeftRim , **aiTop** , **aiTopRightRim** , **aiRight** ,
aiBottomRightRim , **aiBottom** , **aiBottomLeftRim** , **aiLeft** ,
aiCenter }

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual QPointF [anchorPixelPosition](#) (int anchorId) const Q_DECL_OVERRIDE
- QPen **mainPen** () const
- QBrush **mainBrush** () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- double **rectDistance** (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCPItemPosition](#) * **createPosition** (const QString &name)
- [QCPItemAnchor](#) * **createAnchor** (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QPen **mPen**
- QPen **mSelectedPen**
- QBrush **mBrush**
- QBrush **mSelectedBrush**

Protected Attributes inherited from [QCPAbstractItem](#)

- bool **mClipToAxisRect**
- QPointer< [QCPAxisRect](#) > **mClipAxisRect**
- QList< [QCPItemPosition](#) * > **mPositions**
- QList< [QCPItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.433.1 Detailed Description

An ellipse.

It has two positions, *topLeft* and *bottomRight*, which define the rect the ellipse will be drawn in.

10.433.2 Constructor & Destructor Documentation

10.433.2.1 QCPIItemEllipse()

```
QCPIItemEllipse::QCPIItemEllipse (  
    QCustomPlot * parentPlot ) [explicit]
```

Creates an ellipse item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

10.433.3 Member Function Documentation

10.433.3.1 anchorPixelPosition()

```
QPointF QCPIItemEllipse::anchorPixelPosition (  
    int anchorId ) const [protected], [virtual]
```

Reimplemented from [QCPAbstractItem](#).

10.433.3.2 draw()

```
void QCPIItemEllipse::draw (  
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

10.433.3.3 selectTest()

```
double QCPItemEllipse::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the mouseReleaseEvent occurs, and the finally selected object is notified via the selectEvent/ deselectEvent methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to selectEvent when the parent [QCustomPlot](#) decides on the basis of this selectTest call, that the object was successfully selected. The subsequent call to selectEvent will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent selectEvent, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

[selectEvent](#), [deselectEvent](#), [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

10.433.3.4 setBrush()

```
void QCPItemEllipse::setBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill the ellipse. To disable filling, set *brush* to `Qt::NoBrush`.

See also

[setSelectedBrush](#), [setPen](#)

10.433.3.5 setPen()

```
void QCPItemEllipse::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line of the ellipse

See also

[setSelectedPen](#), [setBrush](#)

10.433.3.6 setSelectedBrush()

```
void QCPItemEllipse::setSelectedBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill the ellipse when selected. To disable filling, set *brush* to Qt::NoBrush.

See also

[setBrush](#)

10.433.3.7 setSelectedPen()

```
void QCPItemEllipse::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line of the ellipse when selected

See also

[setPen](#), [setSelected](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

Public Member Functions

- [QCPLItemLine](#) ([QCustomPlot](#) *parentPlot)
- [QPen](#) **pen** () const
- [QPen](#) **selectedPen** () const
- [QCPLLineEnding](#) **head** () const
- [QCPLLineEnding](#) **tail** () const
- void [setPen](#) (const [QPen](#) &pen)
- void [setSelectedPen](#) (const [QPen](#) &pen)
- void [setHead](#) (const [QCPLLineEnding](#) &head)
- void [setTail](#) (const [QCPLLineEnding](#) &tail)
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_↔](#)
DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool **clipToAxisRect** () const
- [QCPAxisRect](#) * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void [setClipToAxisRect](#) (bool clip)
- void [setClipAxisRect](#) ([QCPAxisRect](#) *rect)
- [Q_SLOT](#) void [setSelectable](#) (bool selectable)
- [Q_SLOT](#) void [setSelected](#) (bool selected)
- [QList](#)< [QCPLItemPosition](#) * > **positions** () const
- [QList](#)< [QCPLItemAnchor](#) * > **anchors** () const
- [QCPLItemPosition](#) * **position** (const [QString](#) &name) const
- [QCPLItemAnchor](#) * **anchor** (const [QString](#) &name) const
- bool **hasAnchor** (const [QString](#) &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- [Q_SLOT](#) bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const [QString](#) &layerName)
- void [setAntialiased](#) (bool enabled)
- bool **realVisibility** () const

Public Attributes

- [QCPLItemPosition](#) *const **start**
- [QCPLItemPosition](#) *const **end**

Protected Member Functions

- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- [QPointF](#) **getRectClippedLine** (const [QCPVector2D](#) &start, const [QCPVector2D](#) &end, const [QRect](#) &rect) const
- [QPen](#) **mainPen** () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- virtual [QRect](#) **clipRect** () const Q_DECL_OVERRIDE
- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void **selectEvent** ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState↔ Changed) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual [QPointF](#) **anchorPixelPosition** (int anchorId) const
- double **rectDistance** (const [QRectF](#) &rect, const [QPointF](#) &pos, bool filledRect) const
- [QCPItemPosition](#) * **createPosition** (const [QString](#) &name)
- [QCPItemAnchor](#) * **createAnchor** (const [QString](#) &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **mouseMoveEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseReleaseEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseDoubleClickEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **wheelEvent** ([QWheelEvent](#) *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QPen](#) **mPen**
- [QPen](#) **mSelectedPen**
- [QCPLineEnding](#) **mHead**
- [QCPLineEnding](#) **mTail**

Protected Attributes inherited from [QCPAbstractItem](#)

- bool **mClipToAxisRect**
- [QPointer](#)< [QCPAxisRect](#) > **mClipAxisRect**
- [QList](#)< [QCPItemPosition](#) * > **mPositions**
- [QList](#)< [QCPItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from QCPLayerable

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from QCPAbstractItem

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from QCPLayerable

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.434.1 Detailed Description

A line from one point to another.

It has two positions, *start* and *end*, which define the end points of the line.

With [setHead](#) and [setTail](#) you may set different line ending styles, e.g. to create an arrow.

10.434.2 Constructor & Destructor Documentation

10.434.2.1 QCPLItemLine()

```
QCPLItemLine::QCPLItemLine (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a line item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

10.434.3 Member Function Documentation

10.434.3.1 draw()

```
void QCPLItemLine::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

10.434.3.2 selectTest()

```
double QCPItemLine::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the `mouseReleaseEvent` occurs, and the finally selected object is notified via the `selectEvent/` `deselectEvent` methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to `selectEvent` when the parent [QCustomPlot](#) decides on the basis of this `selectTest` call, that the object was successfully selected. The subsequent call to `selectEvent` will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in `selectTest`. The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent `selectEvent`, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

`selectEvent`, `deselectEvent`, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

10.434.3.3 setHead()

```
void QCPItemLine::setHead (
    const QCPLLineEnding & head )
```

Sets the line ending style of the head. The head corresponds to the *end* position.

Note that due to the overloaded [QCPLLineEnding](#) constructor, you may directly specify a [QCPLLineEnding::EndingStyle](#) here, e.g.

```
setHead(QCPLLineEnding::esSpikeArrow)
```

See also

[setTail](#)

10.434.3.4 setPen()

```
void QCPLItemLine::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line

See also

[setSelectedPen](#)

10.434.3.5 setSelectedPen()

```
void QCPLItemLine::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line when selected

See also

[setPen](#), [setSelected](#)

10.434.3.6 setTail()

```
void QCPLItemLine::setTail (
    const QCPLLineEnding & tail )
```

Sets the line ending style of the tail. The tail corresponds to the *start* position.

Note that due to the overloaded [QCPLLineEnding](#) constructor, you may directly specify a [QCPLLineEnding::EndingStyle](#) here, e.g.

```
setTail(QCPLLineEnding::esSpikeArrow)
```

See also

[setHead](#)

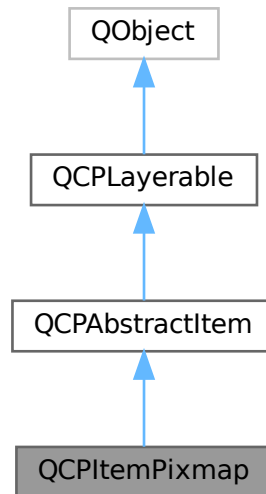
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

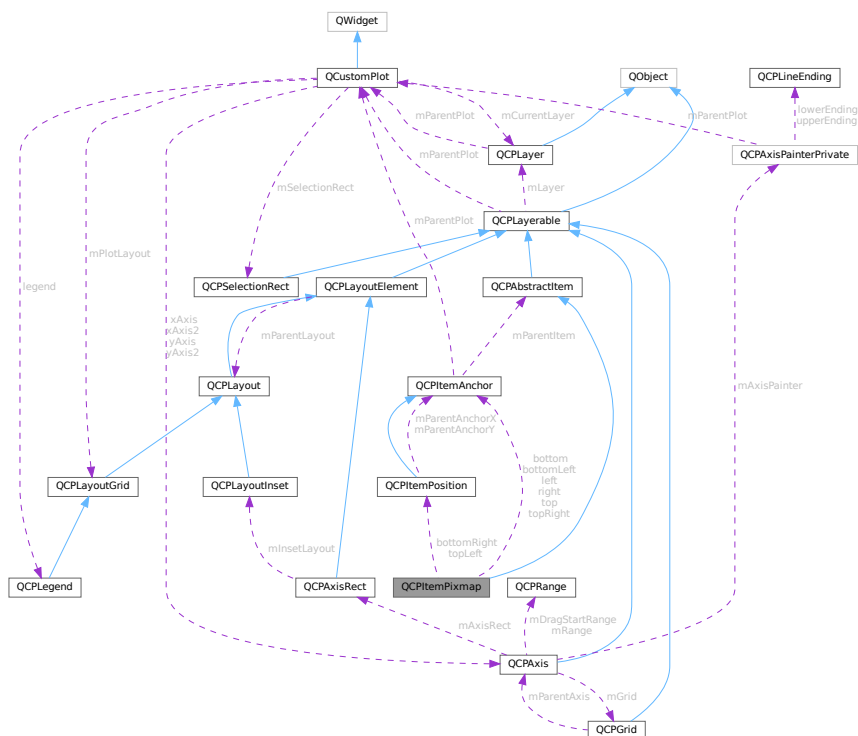
10.435 QCItemPixmap Class Reference

An arbitrary pixmap.

Inheritance diagram for QCItemPixmap:



Collaboration diagram for QCItemPixmap:



Public Member Functions

- [QCItemPixmap](#) ([QCustomPlot](#) *parentPlot)
- QPixmap **pixmap** () const
- bool **scaled** () const
- Qt::AspectRatioMode **aspectRatioMode** () const
- Qt::TransformationMode **transformationMode** () const
- QPen **pen** () const
- QPen **selectedPen** () const
- void [setPixmap](#) (const QPixmap &pixmap)
- void [setScaled](#) (bool scaled, Qt::AspectRatioMode aspectRatioMode=Qt::KeepAspectRatio, Qt::TransformationMode transformationMode=Qt::SmoothTransformation)
- void [setPen](#) (const QPen &pen)
- void [setSelectedPen](#) (const QPen &pen)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool **clipToAxisRect** () const
- [QCPAxisRect](#) * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void [setClipToAxisRect](#) (bool clip)
- void [setClipAxisRect](#) ([QCPAxisRect](#) *rect)
- Q_SLOT void [setSelectable](#) (bool selectable)
- Q_SLOT void [setSelected](#) (bool selected)
- QList< [QCItemPosition](#) * > **positions** () const
- QList< [QCItemAnchor](#) * > **anchors** () const
- [QCItemPosition](#) * **position** (const QString &name) const
- [QCItemAnchor](#) * **anchor** (const QString &name) const
- bool **hasAnchor** (const QString &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Public Attributes

- [QCItemPosition](#) *const **topLeft**
- [QCItemPosition](#) *const **bottomRight**
- [QCItemAnchor](#) *const **top**
- [QCItemAnchor](#) *const **topRight**
- [QCItemAnchor](#) *const **right**
- [QCItemAnchor](#) *const **bottom**
- [QCItemAnchor](#) *const **bottomLeft**
- [QCItemAnchor](#) *const **left**

Protected Types

- enum **AnchorIndex** {
aiTop , **aiTopRight** , **aiRight** , **aiBottom** ,
aiBottomLeft , **aiLeft** }

Protected Member Functions

- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual QPointF **anchorPixelPosition** (int anchorId) const Q_DECL_OVERRIDE
- void **updateScaledPixmap** (QRect finalRect=QRect(), bool flipHorz=false, bool flipVert=false)
- QRect **getFinalRect** (bool *flippedHorz=nullptr, bool *flippedVert=nullptr) const
- QPen **mainPen** () const

Protected Member Functions inherited from [QCAbstractItem](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- virtual QRect **clipRect** () const Q_DECL_OVERRIDE
- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionState↔ Changed) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE
- double **rectDistance** (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCItemPosition](#) * **createPosition** (const QString &name)
- [QCItemAnchor](#) * **createAnchor** (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QPixmap **mPixmap**
- QPixmap **mScaledPixmap**
- bool **mScaled**
- bool **mScaledPixmapInvalidated**
- Qt::AspectRatioMode **mAspectRatioMode**
- Qt::TransformationMode **mTransformationMode**
- QPen **mPen**
- QPen **mSelectedPen**

Protected Attributes inherited from [QCPAbstractItem](#)

- bool **mClipToAxisRect**
- QPointer< [QCPAxisRect](#) > **mClipAxisRect**
- QList< [QCPItemPosition](#) * > **mPositions**
- QList< [QCPItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.435.1 Detailed Description

An arbitrary pixmap.

It has two positions, *topLeft* and *bottomRight*, which define the rectangle the pixmap will be drawn in. Depending on the scale setting ([setScale](#)), the pixmap will be either scaled to fit the rectangle or be drawn aligned to the *topLeft* position.

If scaling is enabled and *topLeft* is further to the bottom/right than *bottomRight* (as shown on the right side of the example image), the pixmap will be flipped in the respective orientations.

10.435.2 Constructor & Destructor Documentation

10.435.2.1 QCItemPixmap()

```
QCItemPixmap::QCItemPixmap (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a rectangle item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

10.435.3 Member Function Documentation

10.435.3.1 anchorPixelPosition()

```
QPointF QCItemPixmap::anchorPixelPosition (
    int anchorId ) const [protected], [virtual]
```

Reimplemented from [QCPAbstractItem](#).

10.435.3.2 draw()

```
void QCItemPixmap::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

10.435.3.3 selectTest()

```
double QCItemPixmap::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the mouseReleaseEvent occurs, and the finally selected object is notified via the selectEvent/ deselectEvent methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to selectEvent when the parent [QCustomPlot](#) decides on the basis of this selectTest call, that the object was successfully selected. The subsequent call to selectEvent will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent selectEvent, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

[selectEvent](#), [deselectEvent](#), [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

10.435.3.4 setPen()

```
void QCPItemPixmap::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw a border around the pixmap.

See also

[setSelectedPen](#), [setBrush](#)

10.435.3.5 setPixmap()

```
void QCPItemPixmap::setPixmap (
    const QPixmap & pixmap )
```

Sets the pixmap that will be displayed.

10.435.3.6 setScaled()

```
void QCPItemPixmap::setScaled (
    bool scaled,
    Qt::AspectRatioMode aspectRatioMode = Qt::KeepAspectRatio,
    Qt::TransformationMode transformationMode = Qt::SmoothTransformation )
```

Sets whether the pixmap will be scaled to fit the rectangle defined by the *topLeft* and *bottomRight* positions.

10.435.3.7 setSelectedPen()

```
void QCPItemPixmap::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw a border around the pixmap when selected

See also

[setPen](#), [setSelected](#)

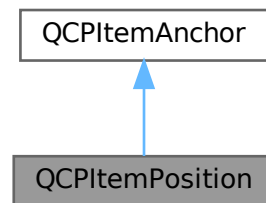
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

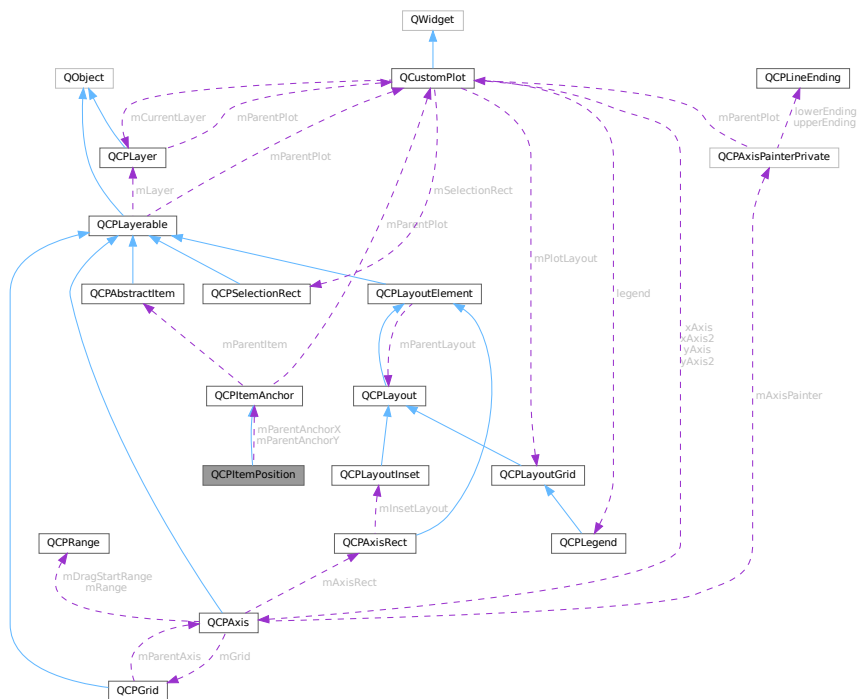
10.436 QCItemPosition Class Reference

Manages the position of an item.

Inheritance diagram for QCItemPosition:



Collaboration diagram for QCItemPosition:



Public Types

- enum **PositionType** { **ptAbsolute** , **ptViewportRatio** , **ptAxisRectRatio** , **ptPlotCoords** }

Public Member Functions

- [QCItemPosition](#) ([QCustomPlot](#) *parentPlot, [QCPAbstractItem](#) *parentItem, const QString &name)
- [PositionType](#) **type** () const
- [PositionType](#) **typeX** () const
- [PositionType](#) **typeY** () const
- [QCItemAnchor](#) * [parentAnchor](#) () const
- [QCItemAnchor](#) * **parentAnchorX** () const
- [QCItemAnchor](#) * **parentAnchorY** () const
- double **key** () const
- double **value** () const
- [QPointF](#) **coords** () const
- [QCPAxis](#) * **keyAxis** () const
- [QCPAxis](#) * **valueAxis** () const
- [QCPAxisRect](#) * **axisRect** () const
- virtual [QPointF](#) [pixelPosition](#) () const Q_DECL_OVERRIDE
- void [setType](#) ([PositionType](#) type)
- void [setTypeX](#) ([PositionType](#) type)
- void [setTypeY](#) ([PositionType](#) type)
- bool [setParentAnchor](#) ([QCItemAnchor](#) *parentAnchor, bool keepPixelPosition=false)
- bool [setParentAnchorX](#) ([QCItemAnchor](#) *parentAnchor, bool keepPixelPosition=false)
- bool [setParentAnchorY](#) ([QCItemAnchor](#) *parentAnchor, bool keepPixelPosition=false)
- void [setCoords](#) (double key, double value)
- void [setCoords](#) (const [QPointF](#) &pos)
- void [setAxes](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- void [setAxisRect](#) ([QCPAxisRect](#) *axisRect)
- void [setPixelPosition](#) (const [QPointF](#) &pixelPosition)

Public Member Functions inherited from [QCItemAnchor](#)

- [QCItemAnchor](#) ([QCustomPlot](#) *parentPlot, [QCPAbstractItem](#) *parentItem, const QString &name, int anchorId=-1)
- QString **name** () const

Protected Member Functions

- virtual [QCItemPosition](#) * [toQCItemPosition](#) () Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCItemAnchor](#)

- void **addChildX** ([QCItemPosition](#) *pos)
- void **removeChildX** ([QCItemPosition](#) *pos)
- void **addChildY** ([QCItemPosition](#) *pos)
- void **removeChildY** ([QCItemPosition](#) *pos)

Protected Attributes

- [PositionType](#) **mPositionTypeX**
- [PositionType](#) **mPositionTypeY**
- [QPointer](#)< [QCPAxis](#) > **mKeyAxis**
- [QPointer](#)< [QCPAxis](#) > **mValueAxis**
- [QPointer](#)< [QCPAxisRect](#) > **mAxisRect**
- double **mKey**
- double **mValue**
- [QCItemAnchor](#) * **mParentAnchorX**
- [QCItemAnchor](#) * **mParentAnchorY**

Protected Attributes inherited from [QCItemAnchor](#)

- `QString mName`
- `QCustomPlot * mParentPlot`
- `QCItemAbstractItem * mParentItem`
- `int mAnchorId`
- `QSet< QCItemPosition * > mChildrenX`
- `QSet< QCItemPosition * > mChildrenY`

10.436.1 Detailed Description

Manages the position of an item.

Every item has at least one public [QCItemPosition](#) member pointer which provides ways to position the item on the [QCustomPlot](#) surface. Some items have multiple positions, for example [QCItemRect](#) has two: *topLeft* and *bottomRight*.

[QCItemPosition](#) has a type ([PositionType](#)) that can be set with [setType](#). This type defines how coordinates passed to [setCoords](#) are to be interpreted, e.g. as absolute pixel coordinates, as plot coordinates of certain axes ([QCItemPosition::setAxes](#)), as fractions of the axis rect ([QCItemPosition::setAxisRect](#)), etc. For more advanced plots it is also possible to assign different types per X/Y coordinate of the position (see [setTypeX](#), [setTypeY](#)). This way an item could be positioned for example at a fixed pixel distance from the top in the Y direction, while following a plot coordinate in the X direction.

A [QCItemPosition](#) may have a parent [QCItemAnchor](#), see [setParentAnchor](#). This way you can tie multiple items together. If the [QCItemPosition](#) has a parent, its coordinates ([setCoords](#)) are considered to be absolute pixels in the reference frame of the parent anchor, where (0, 0) means directly ontop of the parent anchor. For example, You could attach the *start* position of a [QCItemLine](#) to the *bottom* anchor of a [QCItemText](#) to make the starting point of the line always be centered under the text label, no matter where the text is moved to. For more advanced plots, it is possible to assign different parent anchors per X/Y coordinate of the position, see [setParentAnchorX](#), [setParentAnchorY](#). This way an item could follow another item in the X direction but stay at a fixed position in the Y direction. Or even follow item A in X, and item B in Y.

Note that every [QCItemPosition](#) inherits from [QCItemAnchor](#) and thus can itself be used as parent anchor for other positions.

To set the apparent pixel position on the [QCustomPlot](#) surface directly, use [setPixelPosition](#). This works no matter what type this [QCItemPosition](#) is or what parent-child situation it is in, as [setPixelPosition](#) transforms the coordinates appropriately, to make the position appear at the specified pixel values.

10.436.2 Member Enumeration Documentation

10.436.2.1 PositionType

```
enum QCItemPosition::PositionType
```

Defines the ways an item position can be specified. Thus it defines what the numbers passed to [setCoords](#) actually mean.

See also

[setType](#)

Enumerator

ptAbsolute	Static positioning in pixels, starting from the top left corner of the viewport/widget.
ptViewportRatio	Static positioning given by a fraction of the viewport size. For example, if you call <code>setCoords(0, 0)</code> , the position will be at the top left corner of the viewport/widget. <code>setCoords(1, 1)</code> will be at the bottom right corner, <code>setCoords(0.5, 0)</code> will be horizontally centered and vertically at the top of the viewport/widget, etc.
ptAxisRectRatio	Static positioning given by a fraction of the axis rect size (see setAxisRect). For example, if you call <code>setCoords(0, 0)</code> , the position will be at the top left corner of the axis rect. <code>setCoords(1, 1)</code> will be at the bottom right corner, <code>setCoords(0.5, 0)</code> will be horizontally centered and vertically at the top of the axis rect, etc. You can also go beyond the axis rect by providing negative coordinates or coordinates larger than 1.
ptPlotCoords	Dynamic positioning at a plot coordinate defined by two axes (see setAxes).

10.436.3 Constructor & Destructor Documentation

10.436.3.1 QCPItemPosition()

```
QCPItemPosition::QCPItemPosition (
    QCustomPlot * parentPlot,
    QCPAbstractItem * parentItem,
    const QString & name )
```

Creates a new [QCPItemPosition](#). You shouldn't create [QCPItemPosition](#) instances directly, even if you want to make a new item subclass. Use [QCPAbstractItem::createPosition](#) instead, as explained in the subclassing section of the [QCPAbstractItem](#) documentation.

10.436.4 Member Function Documentation

10.436.4.1 parentAnchor()

```
QCPItemAnchor * QCPItemPosition::parentAnchor ( ) const [inline]
```

Returns the current parent anchor.

If different parent anchors were set for X and Y ([setParentAnchorX](#), [setParentAnchorY](#)), this method returns the parent anchor of the Y coordinate. In that case rather use [parentAnchorX\(\)](#) and [parentAnchorY\(\)](#).

See also

[setParentAnchor](#)

10.436.4.2 pixelPosition()

```
QPointF QCPItemPosition::pixelPosition ( ) const [virtual]
```

Returns the final absolute pixel position of the [QCPItemPosition](#) on the [QCustomPlot](#) surface. It includes all effects of type ([setType](#)) and possible parent anchors ([setParentAnchor](#)).

See also

[setPixelPosition](#)

Reimplemented from [QCPItemAnchor](#).

10.436.4.3 setAxes()

```
void QCPItemPosition::setAxes (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis )
```

When [setType](#) is [ptPlotCoords](#), this function may be used to specify the axes the coordinates set with [setCoords](#) relate to. By default they are set to the initial [xAxis](#) and [yAxis](#) of the [QCustomPlot](#).

10.436.4.4 setAxisRect()

```
void QCPItemPosition::setAxisRect (
    QCPAxisRect * axisRect )
```

When [setType](#) is [ptAxisRectRatio](#), this function may be used to specify the axis rect the coordinates set with [setCoords](#) relate to. By default this is set to the main axis rect of the [QCustomPlot](#).

10.436.4.5 setCoords() [1/2]

```
void QCPItemPosition::setCoords (
    const QPointF & pos )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the coordinates as a [QPointF](#) *pos* where *pos.x* has the meaning of *key* and *pos.y* the meaning of *value* of the [setCoords\(double key, double value\)](#) method.

10.436.4.6 setCoords() [2/2]

```
void QCPItemPosition::setCoords (
    double key,
    double value )
```

Sets the coordinates of this [QCPItemPosition](#). What the coordinates mean, is defined by the type ([setType](#), [setTypeX](#), [setTypeY](#)).

For example, if the type is [ptAbsolute](#), *key* and *value* mean the x and y pixel position on the [QCustomPlot](#) surface. In that case the origin (0, 0) is in the top left corner of the [QCustomPlot](#) viewport. If the type is [ptPlotCoords](#), *key* and *value* mean a point in the plot coordinate system defined by the axes set by [setAxes](#). By default those are the [QCustomPlot](#)'s [xAxis](#) and [yAxis](#). See the documentation of [setType](#) for other available coordinate types and their meaning.

If different types were configured for X and Y ([setTypeX](#), [setTypeY](#)), *key* and *value* must also be provided in the different coordinate systems. Here, the X type refers to *key*, and the Y type refers to *value*.

See also

[setPixelPosition](#)

10.436.4.7 setParentAnchor()

```
bool QCPItemPosition::setParentAnchor (
    QCPItemAnchor * parentAnchor,
    bool keepPixelPosition = false )
```

Sets the parent of this [QCPItemPosition](#) to *parentAnchor*. This means the position will now follow any position changes of the anchor. The local coordinate system of positions with a parent anchor always is absolute pixels, with (0, 0) being exactly on top of the parent anchor. (Hence the type shouldn't be set to [ptPlotCoords](#) for positions with parent anchors.)

if *keepPixelPosition* is true, the current pixel position of the [QCPItemPosition](#) is preserved during reparenting. If it's set to false, the coordinates are set to (0, 0), i.e. the position will be exactly on top of the parent anchor.

To remove this [QCPItemPosition](#) from any parent anchor, set *parentAnchor* to `nullptr`.

If the [QCPItemPosition](#) previously had no parent and the type is [ptPlotCoords](#), the type is set to [ptAbsolute](#), to keep the position in a valid state.

This method sets the parent anchor for both X and Y directions. It is also possible to set different parents for X and Y, see [setParentAnchorX](#), [setParentAnchorY](#).

10.436.4.8 setParentAnchorX()

```
bool QCPItemPosition::setParentAnchorX (
    QCPItemAnchor * parentAnchor,
    bool keepPixelPosition = false )
```

This method sets the parent anchor of the X coordinate to *parentAnchor*.

For a detailed description of what a parent anchor is, see the documentation of [setParentAnchor](#).

See also

[setParentAnchor](#), [setParentAnchorY](#)

10.436.4.9 setParentAnchorY()

```
bool QCPItemPosition::setParentAnchorY (
    QCPItemAnchor * parentAnchor,
    bool keepPixelPosition = false )
```

This method sets the parent anchor of the Y coordinate to *parentAnchor*.

For a detailed description of what a parent anchor is, see the documentation of [setParentAnchor](#).

See also

[setParentAnchor](#), [setParentAnchorX](#)

10.436.4.10 `setPixelPosition()`

```
void QCPItemPosition::setPixelPosition (
    const QPointF & pixelPosition )
```

Sets the apparent pixel position. This works no matter what type ([setType](#)) this [QCPItemPosition](#) is or what parent-child situation it is in, as coordinates are transformed appropriately, to make the position finally appear at the specified pixel values.

Only if the type is [ptAbsolute](#) and no parent anchor is set, this function's effect is identical to that of [setCoords](#).

See also

[pixelPosition](#), [setCoords](#)

10.436.4.11 `setType()`

```
void QCPItemPosition::setType (
    QCPItemPosition::PositionType type )
```

Sets the type of the position. The type defines how the coordinates passed to [setCoords](#) should be handled and how the [QCPItemPosition](#) should behave in the plot.

The possible values for *type* can be separated in two main categories:

- The position is regarded as a point in plot coordinates. This corresponds to [ptPlotCoords](#) and requires two axes that define the plot coordinate system. They can be specified with [setAxes](#). By default, the [QCustomPlot](#)'s x- and yAxis are used.
- The position is fixed on the [QCustomPlot](#) surface, i.e. independent of axis ranges. This corresponds to all other types, i.e. [ptAbsolute](#), [ptViewportRatio](#) and [ptAxisRectRatio](#). They differ only in the way the absolute position is described, see the documentation of [PositionType](#) for details. For [ptAxisRectRatio](#), note that you can specify the axis rect with [setAxisRect](#). By default this is set to the main axis rect.

Note that the position type [ptPlotCoords](#) is only available (and sensible) when the position has no parent anchor ([setParentAnchor](#)).

If the type is changed, the apparent pixel position on the plot is preserved. This means the coordinates as retrieved with [coords\(\)](#) and set with [setCoords](#) may change in the process.

This method sets the type for both X and Y directions. It is also possible to set different types for X and Y, see [setTypeX](#), [setTypeY](#).

10.436.4.12 `setTypeX()`

```
void QCPItemPosition::setTypeX (
    QCPItemPosition::PositionType type )
```

This method sets the position type of the X coordinate to *type*.

For a detailed description of what a position type is, see the documentation of [setType](#).

See also

[setType](#), [setTypeY](#)

10.436.4.13 setTypeY()

```
void QCPItemPosition::setTypeY (
    QCPItemPosition::PositionType type )
```

This method sets the position type of the Y coordinate to *type*.

For a detailed description of what a position type is, see the documentation of [setType](#).

See also

[setType](#), [setTypeX](#)

10.436.4.14 toQCPItemPosition()

```
virtual QCPItemPosition * QCPItemPosition::toQCPItemPosition ( ) [inline], [protected], [virtual]
```

Returns `nullptr` if this instance is merely a [QCPItemAnchor](#), and a valid pointer of type `QCPItemPosition*` if it actually is a [QCPItemPosition](#) (which is a subclass of [QCPItemAnchor](#)).

This safe downcast functionality could also be achieved with a `dynamic_cast`. However, [QCustomPlot](#) avoids `dynamic_cast` to work with projects that don't have RTTI support enabled (e.g. `-fno-rtti` flag with gcc compiler).

Reimplemented from [QCPItemAnchor](#).

10.436.4.15 type()

```
QCPItemPosition::PositionType * QCPItemPosition::type ( ) const [inline]
```

Returns the current position type.

If different types were set for X and Y ([setTypeX](#), [setTypeY](#)), this method returns the type of the X coordinate. In that case rather use *typeX()* and *typeY()*.

See also

[setType](#)

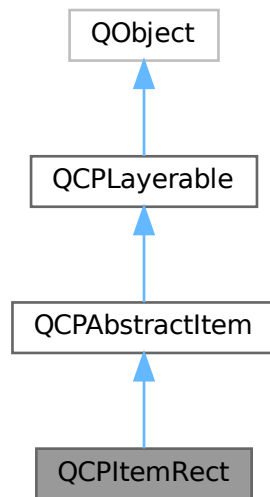
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

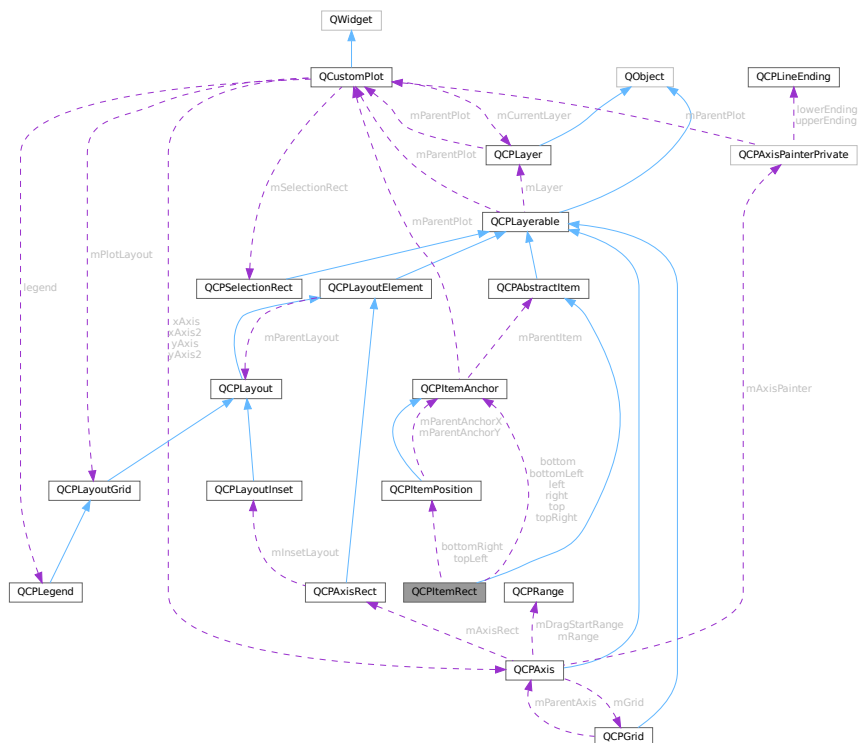
10.437 QCItemRect Class Reference

A rectangle.

Inheritance diagram for QCItemRect:



Collaboration diagram for QCItemRect:



Public Member Functions

- [QCPLItemRect](#) ([QCustomPlot](#) *parentPlot)
- [QPen](#) **pen** () const
- [QPen](#) **selectedPen** () const
- [QBrush](#) **brush** () const
- [QBrush](#) **selectedBrush** () const
- void [setPen](#) (const [QPen](#) &pen)
- void [setSelectedPen](#) (const [QPen](#) &pen)
- void [setBrush](#) (const [QBrush](#) &brush)
- void [setSelectedBrush](#) (const [QBrush](#) &brush)
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_↔](#)
DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool **clipToAxisRect** () const
- [QCPAxisRect](#) * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void [setClipToAxisRect](#) (bool clip)
- void [setClipAxisRect](#) ([QCPAxisRect](#) *rect)
- Q_SLOT void [setSelectable](#) (bool selectable)
- Q_SLOT void [setSelected](#) (bool selected)
- [QList](#)< [QCPLItemPosition](#) * > **positions** () const
- [QList](#)< [QCPLItemAnchor](#) * > **anchors** () const
- [QCPLItemPosition](#) * **position** (const [QString](#) &name) const
- [QCPLItemAnchor](#) * **anchor** (const [QString](#) &name) const
- bool **hasAnchor** (const [QString](#) &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const [QString](#) &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Public Attributes

- [QCPLItemPosition](#) *const **topLeft**
- [QCPLItemPosition](#) *const **bottomRight**
- [QCPLItemAnchor](#) *const **top**
- [QCPLItemAnchor](#) *const **topRight**
- [QCPLItemAnchor](#) *const **right**
- [QCPLItemAnchor](#) *const **bottom**
- [QCPLItemAnchor](#) *const **bottomLeft**
- [QCPLItemAnchor](#) *const **left**

Protected Types

- enum **AnchorIndex** {
[aiTop](#) , [aiTopRight](#) , [aiRight](#) , [aiBottom](#) ,
[aiBottomLeft](#) , [aiLeft](#) }

Protected Member Functions

- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual QPointF **anchorPixelPosition** (int anchorId) const Q_DECL_OVERRIDE
- QPen **mainPen** () const
- QBrush **mainBrush** () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- virtual QRect **clipRect** () const Q_DECL_OVERRIDE
- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionState↵ Changed) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE
- double **rectDistance** (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCPItemPosition](#) * **createPosition** (const QString &name)
- [QCPItemAnchor](#) * **createAnchor** (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QPen **mPen**
- QPen **mSelectedPen**
- QBrush **mBrush**
- QBrush **mSelectedBrush**

Protected Attributes inherited from [QCPAbstractItem](#)

- bool **mClipToAxisRect**
- QPointer< [QCPAxisRect](#) > **mClipAxisRect**
- QList< [QCPItemPosition](#) * > **mPositions**
- QList< [QCPItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.437.1 Detailed Description

A rectangle.

It has two positions, *topLeft* and *bottomRight*, which define the rectangle.

10.437.2 Constructor & Destructor Documentation

10.437.2.1 QCPLItemRect()

```
QCPLItemRect::QCPLItemRect (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a rectangle item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

10.437.3 Member Function Documentation

10.437.3.1 anchorPixelPosition()

```
QPointF QCPLItemRect::anchorPixelPosition (
    int anchorId ) const [protected], [virtual]
```

Reimplemented from [QCPAbstractItem](#).

10.437.3.2 draw()

```
void QCPItemRect::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

10.437.3.3 selectTest()

```
double QCPItemRect::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the `mouseReleaseEvent` occurs, and the finally selected object is notified via the `selectEvent/` `deselectEvent` methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to `selectEvent` when the parent [QCustomPlot](#) decides on the basis of this `selectTest` call, that the object was successfully selected. The subsequent call to `selectEvent` will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in `selectTest`. The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent `selectEvent`, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

`selectEvent`, `deselectEvent`, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

10.437.3.4 setBrush()

```
void QCPItemRect::setBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill the rectangle. To disable filling, set *brush* to Qt::NoBrush.

See also

[setSelectedBrush](#), [setPen](#)

10.437.3.5 setPen()

```
void QCPItemRect::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line of the rectangle

See also

[setSelectedPen](#), [setBrush](#)

10.437.3.6 setSelectedBrush()

```
void QCPItemRect::setSelectedBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill the rectangle when selected. To disable filling, set *brush* to Qt::NoBrush.

See also

[setBrush](#)

10.437.3.7 setSelectedPen()

```
void QCPItemRect::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line of the rectangle when selected

See also

[setPen](#), [setSelected](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

Public Member Functions

- [QCPLItemStraightLine](#) ([QCustomPlot](#) *parentPlot)
- [QPen](#) **pen** () const
- [QPen](#) **selectedPen** () const
- void [setPen](#) (const [QPen](#) &pen)
- void [setSelectedPen](#) (const [QPen](#) &pen)
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_↔](#)
DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool **clipToAxisRect** () const
- [QCPAxisRect](#) * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void [setClipToAxisRect](#) (bool clip)
- void [setClipAxisRect](#) ([QCPAxisRect](#) *rect)
- [Q_SLOT](#) void [setSelectable](#) (bool selectable)
- [Q_SLOT](#) void [setSelected](#) (bool selected)
- [QList](#)< [QCPLItemPosition](#) * > **positions** () const
- [QList](#)< [QCPLItemAnchor](#) * > **anchors** () const
- [QCPLItemPosition](#) * **position** (const [QString](#) &name) const
- [QCPLItemAnchor](#) * **anchor** (const [QString](#) &name) const
- bool **hasAnchor** (const [QString](#) &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- [Q_SLOT](#) bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const [QString](#) &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Public Attributes

- [QCPLItemPosition](#) *const **point1**
- [QCPLItemPosition](#) *const **point2**

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) [Q_DECL_OVERRIDE](#)
- [QLineF](#) [getRectClippedStraightLine](#) (const [QCPVector2D](#) &base, const [QCPVector2D](#) &vec, const [QRect](#) &rect) const
- [QPen](#) **mainPen** () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionState, bool Changed) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual QPointF [anchorPixelPosition](#) (int anchorId) const
- double [rectDistance](#) (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCPItemPosition](#) * [createPosition](#) (const QString &name)
- [QCPItemAnchor](#) * [createAnchor](#) (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QPen [mPen](#)
- QPen [mSelectedPen](#)

Protected Attributes inherited from [QCPAbstractItem](#)

- bool [mClipToAxisRect](#)
- QPointer< [QCPAxisRect](#) > [mClipAxisRect](#)
- QList< [QCPItemPosition](#) * > [mPositions](#)
- QList< [QCPItemAnchor](#) * > [mAnchors](#)
- bool [mSelectable](#)
- bool [mSelected](#)

Protected Attributes inherited from [QCPLayerable](#)

- bool [mVisible](#)
- [QCustomPlot](#) * [mParentPlot](#)
- QPointer< [QCPLayerable](#) > [mParentLayerable](#)
- [QCPLayer](#) * [mLayer](#)
- bool [mAntialiased](#)

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.438.1 Detailed Description

A straight line that spans infinitely in both directions.

It has two positions, *point1* and *point2*, which define the straight line.

10.438.2 Constructor & Destructor Documentation

10.438.2.1 QCPLItemStraightLine()

```
QCPLItemStraightLine::QCPLItemStraightLine (  
    QCustomPlot * parentPlot ) [explicit]
```

Creates a straight line item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

10.438.3 Member Function Documentation

10.438.3.1 draw()

```
void QCPLItemStraightLine::draw (  
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

10.438.3.2 selectTest()

```
double QCPItemStraightLine::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the `mouseReleaseEvent` occurs, and the finally selected object is notified via the `selectEvent/` `deselectEvent` methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to `selectEvent` when the parent [QCustomPlot](#) decides on the basis of this `selectTest` call, that the object was successfully selected. The subsequent call to `selectEvent` will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in `selectTest`. The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent `selectEvent`, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

`selectEvent`, `deselectEvent`, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

10.438.3.3 setPen()

```
void QCPItemStraightLine::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line

See also

[setSelectedPen](#)

10.438.3.4 setSelectedPen()

```
void QCPItemStraightLine::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line when selected

See also

setPen, setSelected

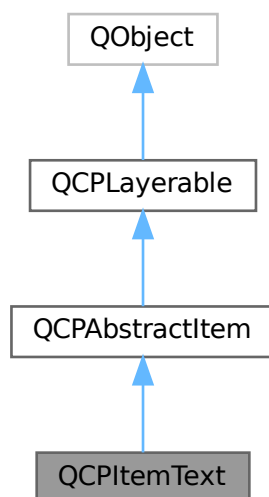
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.439 QCPItemText Class Reference

A text label.

Inheritance diagram for QCPIItemText:



- void [setPositionAlignment](#) (Qt::Alignment alignment)
- void [setTextAlignment](#) (Qt::Alignment alignment)
- void [setRotation](#) (double degrees)
- void [setPadding](#) (const QMargins &padding)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_
DECL_OVERRIDE

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool **clipToAxisRect** () const
- [QCPAxisRect](#) * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void [setClipToAxisRect](#) (bool clip)
- void [setClipAxisRect](#) ([QCPAxisRect](#) *rect)
- Q_SLOT void [setSelectable](#) (bool selectable)
- Q_SLOT void [setSelected](#) (bool selected)
- QList< [QCPItemPosition](#) * > **positions** () const
- QList< [QCPItemAnchor](#) * > **anchors** () const
- [QCPItemPosition](#) * **position** (const QString &name) const
- [QCPItemAnchor](#) * **anchor** (const QString &name) const
- bool **hasAnchor** (const QString &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Public Attributes

- [QCPItemPosition](#) *const **position**
- [QCPItemAnchor](#) *const **topLeft**
- [QCPItemAnchor](#) *const **top**
- [QCPItemAnchor](#) *const **topRight**
- [QCPItemAnchor](#) *const **right**
- [QCPItemAnchor](#) *const **bottomRight**
- [QCPItemAnchor](#) *const **bottom**
- [QCPItemAnchor](#) *const **bottomLeft**
- [QCPItemAnchor](#) *const **left**

Protected Types

- enum **AnchorIndex** {
aiTopLeft , **aiTop** , **aiTopRight** , **aiRight** ,
aiBottomRight , **aiBottom** , **aiBottomLeft** , **aiLeft** }

Protected Member Functions

- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual QPointF **anchorPixelPosition** (int anchorId) const Q_DECL_OVERRIDE
- QPointF **getTextDrawPoint** (const QPointF &pos, const QRectF &rect, Qt::Alignment positionAlignment) const
- QFont **mainFont** () const
- QColor **mainColor** () const
- QPen **mainPen** () const
- QBrush **mainBrush** () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- virtual QRect **clipRect** () const Q_DECL_OVERRIDE
- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE
- double **rectDistance** (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCPItemPosition](#) * **createPosition** (const QString &name)
- [QCPItemAnchor](#) * **createAnchor** (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QColor **mColor**
- QColor **mSelectedColor**
- QPen **mPen**
- QPen **mSelectedPen**
- QBrush **mBrush**
- QBrush **mSelectedBrush**
- QFont **mFont**
- QFont **mSelectedFont**
- QString **mText**
- Qt::Alignment **mPositionAlignment**
- Qt::Alignment **mTextAlignment**
- double **mRotation**
- QMargins **mPadding**

Protected Attributes inherited from [QCPAbstractItem](#)

- bool **mClipToAxisRect**
- QPointer< [QCPAxisRect](#) > **mClipAxisRect**
- QList< [QCPItemPosition](#) * > **mPositions**
- QList< [QCPItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.439.1 Detailed Description

A text label.

Its position is defined by the member *position* and the setting of [setPositionAlignment](#). The latter controls which part of the text rect shall be aligned with *position*.

The text alignment itself (i.e. left, center, right) can be controlled with [setTextAlignment](#).

The text may be rotated around the *position* point with [setRotation](#).

10.439.2 Constructor & Destructor Documentation

10.439.2.1 QCPLItemText()

```
QCPLItemText::QCPLItemText (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a text item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

10.439.3 Member Function Documentation

10.439.3.1 `anchorPixelPosition()`

```
QPointF QCPItemText::anchorPixelPosition (
    int anchorId ) const [protected], [virtual]
```

Reimplemented from [QCPAbstractItem](#).

10.439.3.2 `draw()`

```
void QCPItemText::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractItem](#).

10.439.3.3 `selectTest()`

```
double QCPItemText::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the `selectionTolerance` multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the `mouseReleaseEvent` occurs, and the finally selected object is notified via the `selectEvent/` `deselectEvent` methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to `selectEvent` when the parent [QCustomPlot](#) decides on the basis of this `selectTest` call, that the object was successfully selected. The subsequent call to `selectEvent` will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in `selectTest`. The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent `selectEvent`, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

`selectEvent`, `deselectEvent`, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

10.439.3.4 setBrush()

```
void QCPItemText::setBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill the background of the text. To disable the background, set *brush* to Qt::NoBrush.

See also

[setSelectedBrush](#), [setPen](#), [setPadding](#)

10.439.3.5 setColor()

```
void QCPItemText::setColor (
    const QColor & color )
```

Sets the color of the text.

10.439.3.6 setFont()

```
void QCPItemText::setFont (
    const QFont & font )
```

Sets the font of the text.

See also

[setSelectedFont](#), [setColor](#)

10.439.3.7 setPadding()

```
void QCPItemText::setPadding (
    const QMargins & padding )
```

Sets the distance between the border of the text rectangle and the text. The appearance (and visibility) of the text rectangle can be controlled with [setPen](#) and [setBrush](#).

10.439.3.8 setPen()

```
void QCPItemText::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw a rectangular border around the text. To disable the border, set *pen* to Qt::NoPen.

See also

[setSelectedPen](#), [setBrush](#), [setPadding](#)

10.439.3.9 setPositionAlignment()

```
void QCPIItemText::setPositionAlignment (
    Qt::Alignment alignment )
```

Sets which point of the text rect shall be aligned with *position*.

Examples:

- If *alignment* is `Qt::AlignHCenter` | `Qt::AlignTop`, the text will be positioned such that the top of the text rect will be horizontally centered on *position*.
- If *alignment* is `Qt::AlignLeft` | `Qt::AlignBottom`, *position* will indicate the bottom left corner of the text rect.

If you want to control the alignment of (multi-lined) text within the text rect, use [setTextAlignment](#).

10.439.3.10 setRotation()

```
void QCPIItemText::setRotation (
    double degrees )
```

Sets the angle in degrees by which the text (and the text rectangle, if visible) will be rotated around *position*.

10.439.3.11 setSelectedBrush()

```
void QCPIItemText::setSelectedBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill the background of the text, when the item is selected. To disable the background, set *brush* to `Qt::NoBrush`.

See also

[setBrush](#)

10.439.3.12 setSelectedColor()

```
void QCPIItemText::setSelectedColor (
    const QColor & color )
```

Sets the color of the text that will be used when the item is selected.

10.439.3.13 setSelectedFont()

```
void QCPIItemText::setSelectedFont (
    const QFont & font )
```

Sets the font of the text that will be used when the item is selected.

See also

[setFont](#)

10.439.3.14 setSelectedPen()

```
void QCPItemText::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw a rectangular border around the text, when the item is selected. To disable the border, set *pen* to `Qt::NoPen`.

See also

[setPen](#)

10.439.3.15 setText()

```
void QCPItemText::setText (
    const QString & text )
```

Sets the text that will be displayed. Multi-line texts are supported by inserting a line break character, e.g. `'\n'`.

See also

[setFont](#), [setColor](#), [setTextAlignment](#)

10.439.3.16 setTextAlignment()

```
void QCPItemText::setTextAlignment (
    Qt::Alignment alignment )
```

Controls how (multi-lined) text is aligned inside the text rect (typically `Qt::AlignLeft`, `Qt::AlignCenter` or `Qt::AlignRight`).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

Public Member Functions

- [QCPLItemTracer](#) ([QCustomPlot](#) *parentPlot)
- [QPen](#) **pen** () const
- [QPen](#) **selectedPen** () const
- [QBrush](#) **brush** () const
- [QBrush](#) **selectedBrush** () const
- double **size** () const
- [TracerStyle](#) **style** () const
- [QCPGraph](#) * **graph** () const
- double **graphKey** () const
- bool **interpolating** () const
- void **setPen** (const [QPen](#) &pen)
- void **setSelectedPen** (const [QPen](#) &pen)
- void **setBrush** (const [QBrush](#) &brush)
- void **setSelectedBrush** (const [QBrush](#) &brush)
- void **setSize** (double size)
- void **setStyle** ([TracerStyle](#) style)
- void **setGraph** ([QCPGraph](#) *graph)
- void **setGraphKey** (double key)
- void **setInterpolating** (bool enabled)
- virtual double **selectTest** (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_↔](#)
DECL_OVERRIDE
- void **updatePosition** ()

Public Member Functions inherited from [QCPAbstractItem](#)

- [QCPAbstractItem](#) ([QCustomPlot](#) *parentPlot)
- bool **clipToAxisRect** () const
- [QCPAxisRect](#) * **clipAxisRect** () const
- bool **selectable** () const
- bool **selected** () const
- void **setClipToAxisRect** (bool clip)
- void **setClipAxisRect** ([QCPAxisRect](#) *rect)
- Q_SLOT void **setSelectable** (bool selectable)
- Q_SLOT void **setSelected** (bool selected)
- [QList](#)< [QCPLItemPosition](#) * > **positions** () const
- [QList](#)< [QCPLItemAnchor](#) * > **anchors** () const
- [QCPLItemPosition](#) * **position** (const [QString](#) &name) const
- [QCPLItemAnchor](#) * **anchor** (const [QString](#) &name) const
- bool **hasAnchor** (const [QString](#) &name) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const [QString](#) &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Public Attributes

- [QCPItemPosition](#) *const **position**

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- QPen **mainPen** () const
- QBrush **mainBrush** () const

Protected Member Functions inherited from [QCPAbstractItem](#)

- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionState↔ Changed) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual QPointF [anchorPixelPosition](#) (int anchorId) const
- double **rectDistance** (const QRectF &rect, const QPointF &pos, bool filledRect) const
- [QCPItemPosition](#) * **createPosition** (const QString &name)
- [QCPItemAnchor](#) * **createAnchor** (const QString &name, int anchorId)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QPen **mPen**
- QPen **mSelectedPen**
- QBrush **mBrush**
- QBrush **mSelectedBrush**
- double **mSize**
- [TracerStyle](#) **mStyle**
- [QCPGraph](#) * **mGraph**
- double **mGraphKey**
- bool **mInterpolating**

Protected Attributes inherited from [QCPAbstractItem](#)

- bool **mClipToAxisRect**
- [QPointer](#)< [QCPAxisRect](#) > **mClipAxisRect**
- [QList](#)< [QCPItemPosition](#) * > **mPositions**
- [QList](#)< [QCPItemAnchor](#) * > **mAnchors**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer](#)< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from [QCPAbstractItem](#)

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.440.1 Detailed Description

Item that sticks to [QCPGraph](#) data points.

The tracer can be connected with a [QCPGraph](#) via [setGraph](#). Then it will automatically adopt the coordinate axes of the graph and update its *position* to be on the graph's data. This means the key stays controllable via [setGraphKey](#), but the value will follow the graph data. If a [QCPGraph](#) is connected, note that setting the coordinates of the tracer item directly via *position* will have no effect because they will be overridden in the next redraw (this is when the coordinate update happens).

If the specified key in [setGraphKey](#) is outside the key bounds of the graph, the tracer will stay at the corresponding end of the graph.

With [setInterpolating](#) you may specify whether the tracer may only stay exactly on data points or whether it interpolates data points linearly, if given a key that lies between two data points of the graph.

The tracer has different visual styles, see [setStyle](#). It is also possible to make the tracer have no own visual appearance (set the style to [tsNone](#)), and just connect other item positions to the tracer *position* (used as an anchor) via [QCPItemPosition::setParentAnchor](#).

Note

The tracer position is only automatically updated upon redraws. So when the data of the graph changes and immediately afterwards (without a redraw) the position coordinates of the tracer are retrieved, they will not reflect the updated data of the graph. In this case [updatePosition](#) must be called manually, prior to reading the tracer coordinates.

10.440.2 Member Enumeration Documentation

10.440.2.1 TracerStyle

enum [QCPItemTracer::TracerStyle](#)

The different visual appearances a tracer item can have. Some styles size may be controlled with [setSize](#).

See also

[setStyle](#)

Enumerator

tsNone	The tracer is not visible.
tsPlus	A plus shaped crosshair with limited size.
tsCrosshair	A plus shaped crosshair which spans the complete axis rect.
tsCircle	A circle.
tsSquare	A square.

10.440.3 Constructor & Destructor Documentation

10.440.3.1 QCPItemTracer()

```
QCPItemTracer::QCPItemTracer (  
    QCustomPlot * parentPlot )    [explicit]
```

Creates a tracer item and sets default values.

The created item is automatically registered with *parentPlot*. This [QCustomPlot](#) instance takes ownership of the item, so do not delete it manually but use [QCustomPlot::removeItem\(\)](#) instead.

10.440.4 Member Function Documentation

10.440.4.1 draw()

```
void QCPItemTracer::draw (  
    QCPPainter * painter )    [protected], [virtual]
```

Implements [QCPAbstractItem](#).

10.440.4.2 selectTest()

```
double QCPItemTracer::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the `mouseReleaseEvent` occurs, and the finally selected object is notified via the `selectEvent/` `deselectEvent` methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to `selectEvent` when the parent [QCustomPlot](#) decides on the basis of this `selectTest` call, that the object was successfully selected. The subsequent call to `selectEvent` will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in `selectTest`. The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent `selectEvent`, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

`selectEvent`, `deselectEvent`, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Implements [QCPAbstractItem](#).

10.440.4.3 setBrush()

```
void QCPItemTracer::setBrush (
    const QBrush & brush )
```

Sets the brush that will be used to draw any fills of the tracer

See also

[setSelectedBrush](#), [setPen](#)

10.440.4.4 setGraph()

```
void QCPItemTracer::setGraph (
    QCPGraph * graph )
```

Sets the [QCPGraph](#) this tracer sticks to. The tracer *position* will be set to type [QCPItemPosition::ptPlotCoords](#) and the axes will be set to the axes of *graph*.

To free the tracer from any graph, set *graph* to `nullptr`. The tracer *position* can then be placed freely like any other item position. This is the state the tracer will assume when its graph gets deleted while still attached to it.

See also

[setGraphKey](#)

10.440.4.5 setGraphKey()

```
void QCPItemTracer::setGraphKey (
    double key )
```

Sets the key of the graph's data point the tracer will be positioned at. This is the only free coordinate of a tracer when attached to a graph.

Depending on [setInterpolating](#), the tracer will be either positioned on the data point closest to *key*, or will stay exactly at *key* and interpolate the value linearly.

See also

[setGraph](#), [setInterpolating](#)

10.440.4.6 setInterpolating()

```
void QCPItemTracer::setInterpolating (
    bool enabled )
```

Sets whether the value of the graph's data points shall be interpolated, when positioning the tracer.

If *enabled* is set to false and a key is given with [setGraphKey](#), the tracer is placed on the data point of the graph which is closest to the key, but which is not necessarily exactly there. If *enabled* is true, the tracer will be positioned exactly at the specified key, and the appropriate value will be interpolated from the graph's data points linearly.

See also

[setGraph](#), [setGraphKey](#)

10.440.4.7 setPen()

```
void QCPItemTracer::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line of the tracer

See also

[setSelectedPen](#), [setBrush](#)

10.440.4.8 setSelectedBrush()

```
void QCPItemTracer::setSelectedBrush (
    const QBrush & brush )
```

Sets the brush that will be used to draw any fills of the tracer, when selected.

See also

[setBrush](#), [setSelected](#)

10.440.4.9 setSelectedPen()

```
void QCPItemTracer::setSelectedPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the line of the tracer when selected

See also

[setPen](#), [setSelected](#)

10.440.4.10 setSize()

```
void QCPItemTracer::setSize (
    double size )
```

Sets the size of the tracer in pixels, if the style supports setting a size (e.g. [tsSquare](#) does, [tsCrosshair](#) does not).

10.440.4.11 setStyle()

```
void QCPItemTracer::setStyle (
    QCPItemTracer::TracerStyle style )
```

Sets the style/visual appearance of the tracer.

If you only want to use the tracer *position* as an anchor for other items, set *style* to [tsNone](#).

10.440.4.12 updatePosition()

```
void QCPItemTracer::updatePosition ( )
```

If the tracer is connected with a graph ([setGraph](#)), this function updates the tracer's *position* to reside on the graph data, depending on the configured key ([setGraphKey](#)).

It is called automatically on every redraw and normally doesn't need to be called manually. One exception is when you want to read the tracer coordinates via *position* and are not sure that the graph's data (or the tracer key with [setGraphKey](#)) hasn't changed since the last redraw. In that situation, call this function before accessing *position*, to make sure you don't get out-of-date coordinates.

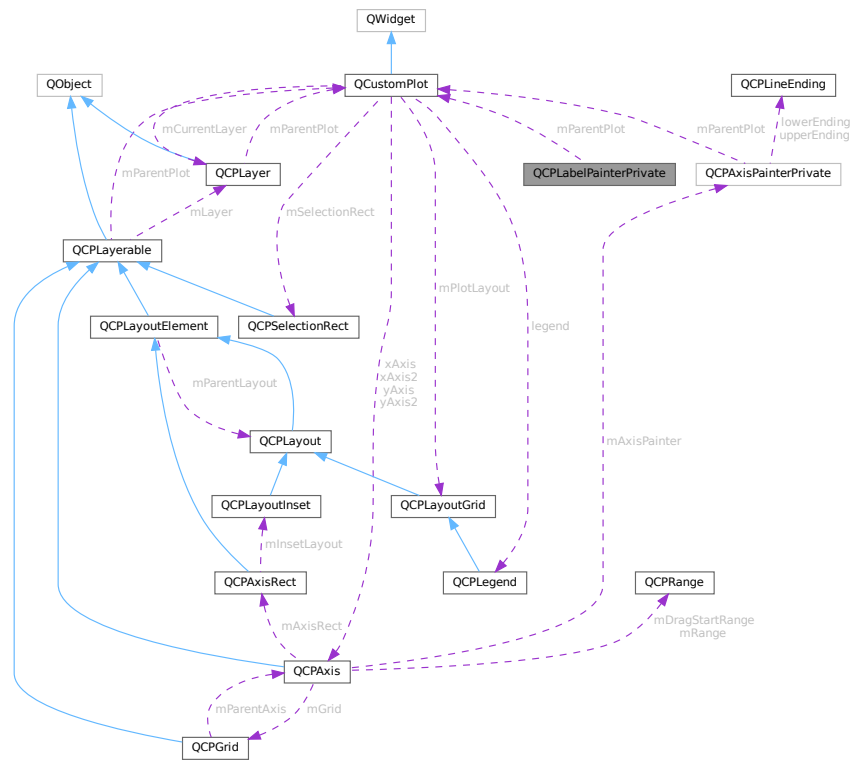
If there is no graph set on this tracer, this function does nothing.

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.441 QCPLabelPainterPrivate Class Reference

Collaboration diagram for QCPLabelPainterPrivate:



Classes

- struct [CachedLabel](#)
- struct [LabelData](#)

Public Types

- enum [AnchorMode](#) { **amRectangular** , **amSkewedUpright** , **amSkewedRotated** }
- enum [AnchorReferenceType](#) { **artNormal** , **artTangent** }
- enum [AnchorSide](#) {
asLeft , **asRight** , **asTop** , **asBottom** ,
asTopLeft , **asTopRight** , **asBottomRight** , **asBottomLeft** }

Public Member Functions

- [QCPLabelPainterPrivate](#) ([QCustomPlot](#) *parentPlot)
- void **setAnchorSide** ([AnchorSide](#) side)
- void **setAnchorMode** ([AnchorMode](#) mode)
- void **setAnchorReference** (const [QPointF](#) &pixelPoint)
- void **setAnchorReferenceType** ([AnchorReferenceType](#) type)
- void **setFont** (const [QFont](#) &font)

- void **setColor** (const QColor &color)
- void **setPadding** (int padding)
- void **setRotation** (double rotation)
- void **setSubstituteExponent** (bool enabled)
- void **setMultiplicationSymbol** (QChar symbol)
- void **setAbbreviateDecimalPowers** (bool enabled)
- void **setCacheSize** (int labelCount)
- [AnchorMode](#) **anchorMode** () const
- [AnchorSide](#) **anchorSide** () const
- QPointF **anchorReference** () const
- [AnchorReferenceType](#) **anchorReferenceType** () const
- QFont **font** () const
- QColor **color** () const
- int **padding** () const
- double **rotation** () const
- bool **substituteExponent** () const
- QChar **multiplicationSymbol** () const
- bool **abbreviateDecimalPowers** () const
- int **cacheSize** () const
- void **drawTickLabel** ([QCPPainter](#) *painter, const QPointF &tickPos, const QString &text)
- void **clearCache** ()

Static Public Attributes

- static const QChar **SymbolDot**
- static const QChar **SymbolCross**

Protected Member Functions

- virtual void **drawLabelMaybeCached** ([QCPPainter](#) *painter, const QFont &font, const QColor &color, const QPointF &pos, [AnchorSide](#) side, double rotation, const QString &text)
- virtual QByteArray **generateLabelParameterHash** () const
- QPointF **getAnchorPos** (const QPointF &tickPos)
- void **drawText** ([QCPPainter](#) *painter, const QPointF &pos, const [LabelData](#) &labelData) const
- [LabelData](#) **getTickLabelData** (const QFont &font, const QColor &color, double rotation, [AnchorSide](#) side, const QString &text) const
- void **applyAnchorTransform** ([LabelData](#) &labelData) const
- [CachedLabel](#) * **createCachedLabel** (const [LabelData](#) &labelData) const
- QByteArray **cacheKey** (const QString &text, const QColor &color, double rotation, [AnchorSide](#) side) const
- [AnchorSide](#) **skewedAnchorSide** (const QPointF &tickPos, double sideExpandHorz, double sideExpandVert) const
- [AnchorSide](#) **rotationCorrectedSide** ([AnchorSide](#) side, double rotation) const
- void **analyzeFontMetrics** ()

Protected Attributes

- [AnchorMode](#) **mAnchorMode**
- [AnchorSide](#) **mAnchorSide**
- [QPointF](#) **mAnchorReference**
- [AnchorReferenceType](#) **mAnchorReferenceType**
- [QFont](#) **mFont**
- [QColor](#) **mColor**
- [int](#) **mPadding**
- [double](#) **mRotation**
- [bool](#) **mSubstituteExponent**
- [QChar](#) **mMultiplicationSymbol**
- [bool](#) **mAbbreviateDecimalPowers**
- [QCustomPlot](#) * **mParentPlot**
- [QByteArray](#) **mLabelParameterHash**
- [QCache](#)< [QString](#), [CachedLabel](#) > **mLabelCache**
- [QRect](#) **mAxisSelectionBox**
- [QRect](#) **mTickLabelsSelectionBox**
- [QRect](#) **mLabelSelectionBox**
- [int](#) **mLetterCapHeight**
- [int](#) **mLetterDescent**

10.441.1 Member Enumeration Documentation

10.441.1.1 AnchorMode

```
enum QCPLabelPainterPrivate::AnchorMode
```

TODO

10.441.1.2 AnchorReferenceType

```
enum QCPLabelPainterPrivate::AnchorReferenceType
```

TODO

10.441.1.3 AnchorSide

```
enum QCPLabelPainterPrivate::AnchorSide
```

TODO

10.441.2 Constructor & Destructor Documentation

10.441.2.1 QCPLabelPainterPrivate()

```
QCPLabelPainterPrivate::QCPLabelPainterPrivate (
    QCustomPlot * parentPlot ) [explicit]
```

Constructs a [QCPLabelPainterPrivate](#) instance. Make sure to not create a new instance on every redraw, to utilize the caching mechanisms.

the *parentPlot* does not take ownership of the label painter. Make sure to delete it appropriately.

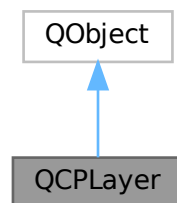
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.442 QCPLayer Class Reference

A layer that may contain objects, to control the rendering order.

Inheritance diagram for QCPLayer:



Protected Attributes

- [QCustomPlot](#) * **mParentPlot**
- [QString](#) **mName**
- [int](#) **mIndex**
- [QList](#)< [QCPLayerable](#) * > **mChildren**
- [bool](#) **mVisible**
- [LayerMode](#) **mMode**
- [QWeakPointer](#)< [QCPLayerable](#) > **mPaintBuffer**

Friends

- class [QCustomPlot](#)
- class [QCPLayerable](#)

10.442.1 Detailed Description

A layer that may contain objects, to control the rendering order.

The Layering system of [QCustomPlot](#) is the mechanism to control the rendering order of the elements inside the plot.

It is based on the two classes [QCPLayer](#) and [QCPLayerable](#). [QCustomPlot](#) holds an ordered list of one or more instances of [QCPLayer](#) (see [QCustomPlot::addLayer](#), [QCustomPlot::layer](#), [QCustomPlot::moveLayer](#), etc.). When replotting, [QCustomPlot](#) goes through the list of layers bottom to top and successively draws the layerables of the layers into the paint buffer(s).

A [QCPLayer](#) contains an ordered list of [QCPLayerable](#) instances. [QCPLayerable](#) is an abstract base class from which almost all visible objects derive, like axes, grids, graphs, items, etc.

10.442.2 Default layers

Initially, [QCustomPlot](#) has six layers: "background", "grid", "main", "axes", "legend" and "overlay" (in that order). On top is the "overlay" layer, which only contains the [QCustomPlot](#)'s selection rect ([QCustomPlot::selectionRect](#)). The next two layers "axes" and "legend" contain the default axes and legend, so they will be drawn above plottables. In the middle, there is the "main" layer. It is initially empty and set as the current layer (see [QCustomPlot::setCurrentLayer](#)). This means, all new plottables, items etc. are created on this layer by default. Then comes the "grid" layer which contains the [QCPGrid](#) instances (which belong tightly to [QCPAxis](#), see [QCPAxis::grid](#)). The Axis rect background shall be drawn behind everything else, thus the default [QCPAxisRect](#) instance is placed on the "background" layer. Of course, the layer affiliation of the individual objects can be changed as required ([QCPLayerable::setLayer](#)).

10.442.3 Controlling the rendering order via layers

Controlling the ordering of layerables in the plot is easy: [Create](#) a new layer in the position you want the layerable to be in, e.g. above "main", with [QCustomPlot::addLayer](#). Then set the current layer with [QCustomPlot::setCurrentLayer](#) to that new layer and finally create the objects normally. They will be placed on the new layer automatically, due to the current layer setting. Alternatively you could have also ignored the current layer setting and just moved the objects with [QCPLayerable::setLayer](#) to the desired layer after creating them.

It is also possible to move whole layers. For example, If you want the grid to be shown in front of all plottables/items on the "main" layer, just move it above "main" with [QCustomPlot::moveLayer](#).

The rendering order within one layer is simply by order of creation or insertion. The item created last (or added last to the layer), is drawn on top of all other objects on that layer.

When a layer is deleted, the objects on it are not deleted with it, but fall on the layer below the deleted layer, see [QCustomPlot::removeLayer](#).

10.442.4 Replotting only a specific layer

If the layer mode ([setMode](#)) is set to [ImBuffered](#), you can replot only this specific layer by calling [replot](#). In certain situations this can provide better replot performance, compared with a full replot of all layers. Upon creation of a new layer, the layer mode is initialized to [ImLogical](#). The only layer that is set to [ImBuffered](#) in a new [QCustomPlot](#) instance is the "overlay" layer, containing the selection rect.

10.442.5 Member Enumeration Documentation

10.442.5.1 LayerMode

```
enum QCPLayer::LayerMode
```

Defines the different rendering modes of a layer. Depending on the mode, certain layers can be replotted individually, without the need to replot (possibly complex) layerables on other layers.

See also

[setMode](#)

Enumerator

ImLogical	Layer is used only for rendering order, and shares paint buffer with all other adjacent logical layers.
ImBuffered	Layer has its own paint buffer and may be replotted individually (see replot).

10.442.6 Constructor & Destructor Documentation

10.442.6.1 QCPLayer()

```
QCPLayer::QCPLayer (
    QCustomPlot * parentPlot,
    const QString & layerName )
```

Creates a new [QCPLayer](#) instance.

Normally you shouldn't directly instantiate layers, use [QCustomPlot::addLayer](#) instead.

Warning

It is not checked that *layerName* is actually a unique layer name in *parentPlot*. This check is only performed by [QCustomPlot::addLayer](#).

10.442.7 Member Function Documentation

10.442.7.1 children()

```
QList< QCPLayerable * > QCPLayer::children ( ) const [inline]
```

Returns a list of all layerables on this layer. The order corresponds to the rendering order: layerables with higher indices are drawn above layerables with lower indices.

10.442.7.2 index()

```
int QCPLayer::index ( ) const [inline]
```

Returns the index this layer has in the [QCustomPlot](#). The index is the integer number by which this layer can be accessed via [QCustomPlot::layer](#).

Layers with higher indices will be drawn above layers with lower indices.

10.442.7.3 replot()

```
void QCPLayer::replot ( )
```

If the layer mode ([setMode](#)) is set to [lmBuffered](#), this method allows replotting only the layerables on this specific layer, without the need to replot all other layers (as a call to [QCustomPlot::replot](#) would do).

[QCustomPlot](#) also makes sure to replot all layers instead of only this one, if the layer ordering or any layerable-layer-association has changed since the last full replot and any other paint buffers were thus invalidated.

If the layer mode is [lmLogical](#) however, this method simply calls [QCustomPlot::replot](#) on the parent [QCustomPlot](#) instance.

See also

[draw](#)

10.442.7.4 setMode()

```
void QCPLayer::setMode (
    QCPLayer::LayerMode mode )
```

Sets the rendering mode of this layer.

If *mode* is set to [lmBuffered](#) for a layer, it will be given a dedicated paint buffer by the parent [QCustomPlot](#) instance. This means it may be replotted individually by calling [QCPLayer::replot](#), without needing to replot all other layers.

Layers which are set to [lmLogical](#) (the default) are used only to define the rendering order and can't be replotted individually.

Note that each layer which is set to [lmBuffered](#) requires additional paint buffers for the layers below, above and for the layer itself. This increases the memory consumption and (slightly) decreases the repainting speed because multiple paint buffers need to be joined. So you should carefully choose which layers benefit from having their own paint buffer. A typical example would be a layer which contains certain layerables (e.g. items) that need to be changed and thus replotted regularly, while all other layerables on other layers stay static. By default, only the topmost layer called "overlay" is in mode [lmBuffered](#), and contains the selection rect.

See also

[replot](#)

Protected Member Functions

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual [QRect](#) **clipRect** () const
- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const =0
- virtual void **draw** ([QCPPainter](#) *painter)=0
- virtual void **selectEvent** ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState↔ Changed)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mousePressEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **mouseMoveEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseReleaseEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseDoubleClickEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **wheelEvent** ([QWheelEvent](#) *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer](#)< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class **QCustomPlot**
- class **QCPLayer**
- class **QCPAxisRect**

10.443.1 Detailed Description

Base class for all drawable objects.

This is the abstract base class most visible objects derive from, e.g. plottables, axes, grid etc.

Every layerable is on a layer ([QCPLayer](#)) which allows controlling the rendering order by stacking the layers accordingly.

For details about the layering mechanism, see the [QCPLayer](#) documentation.

10.443.2 Constructor & Destructor Documentation

10.443.2.1 QCPLayerable()

```
QCPLayerable::QCPLayerable (
    QCustomPlot * plot,
    QString targetLayer = QString(),
    QCPLayerable * parentLayerable = nullptr )
```

Creates a new [QCPLayerable](#) instance.

Since [QCPLayerable](#) is an abstract base class, it can't be instantiated directly. Use one of the derived classes.

If *plot* is provided, it automatically places itself on the layer named *targetLayer*. If *targetLayer* is an empty string, it places itself on the current layer of the plot (see [QCustomPlot::setCurrentLayer](#)).

It is possible to provide `nullptr` as *plot*. In that case, you should assign a parent plot at a later time with `initializeParentPlot`.

The layerable's parent layerable is set to *parentLayerable*, if provided. Direct layerable parents are mainly used to control visibility in a hierarchy of layerables. This means a layerable is only drawn, if all its ancestor layerables are also visible. Note that *parentLayerable* does not become the QObject-parent (for memory management) of this layerable, *plot* does. It is not uncommon to set the QObject-parent to something else in the constructors of [QCPLayerable](#) subclasses, to guarantee a working destruction hierarchy.

10.443.3 Member Function Documentation

10.443.3.1 layerChanged

```
void QCPLayerable::layerChanged (
    QCPLayer * newLayer ) [signal]
```

This signal is emitted when the layer of this layerable changes, i.e. this layerable is moved to a different layer.

See also

[setLayer](#)

10.443.3.2 mouseDoubleClickEvent()

```
void QCPLayerable::mouseDoubleClickEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses the mouse button a second time in a double-click, while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The [mouseDoubleClickEvent](#) is called instead of the second [mousePressEvent](#). So in the case of a double-click, the event succession is *pressEvent* – *releaseEvent* – *doubleClickEvent* – *releaseEvent*.

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *details* contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the *details* parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

Similarly to [mousePressEvent](#), once a layerable has accepted the [mouseDoubleClickEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) and [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [wheelEvent](#)

Reimplemented in [QCPTextElement](#).

10.443.3.3 [mouseMoveEvent\(\)](#)

```
void QCPLayerable::mouseMoveEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user moves the mouse while holding a mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented in [QCPAxis](#), [QCPAxisRect](#), [QCPColorScaleAxisRectPrivate](#), [QCPColorScale](#), [QCPPolarAxisRadial](#), and [QCPPolarAxisAngular](#).

10.443.3.4 [mousePressEvent\(\)](#)

```
void QCPLayerable::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses a mouse button while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *details* contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the *details* parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

[QCustomPlot](#) uses an event propagation system that works the same as Qt's system. If your layerable doesn't reimplement the [mousePressEvent](#) or explicitly calls `event->ignore()` in its reimplementation, the event will be propagated to the next layerable in the stacking order.

Once a layerable has accepted the [mousePressEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) or [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented in [QCPAxis](#), [QCPAxisRect](#), [QCPTextElement](#), [QCPColorScaleAxisRectPrivate](#), [QCPColorScale](#), [QCPPolarAxisRadial](#), and [QCPPolarAxisAngular](#).

10.443.3.5 mouseReleaseEvent()

```
void QCPLayerable::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user releases the mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented in [QCPAxis](#), [QCPAxisRect](#), [QCPTextElement](#), [QCPColorScaleAxisRectPrivate](#), [QCPColorScale](#), [QCPPolarAxisRadial](#), and [QCPPolarAxisAngular](#).

10.443.3.6 parentLayerable()

```
QCPLayerable * QCPLayerable::parentLayerable ( ) const [inline]
```

Returns the parent layerable of this layerable. The parent layerable is used to provide visibility hierarchies in conjunction with the method [realVisibility](#). This way, layerables only get drawn if their parent layerables are visible, too.

Note that a parent layerable is not necessarily also the QObject parent for memory management. Further, a layerable doesn't always have a parent layerable, so this function may return `nullptr`.

A parent layerable is set implicitly when placed inside layout elements and doesn't need to be set manually by the user.

10.443.3.7 realVisibility()

```
bool QCPLayerable::realVisibility ( ) const
```

Returns whether this layerable is visible, taking the visibility of the layerable parent and the visibility of this layerable's layer into account. This is the method that is consulted to decide whether a layerable shall be drawn or not.

If this layerable has a direct layerable parent (usually set via hierarchies implemented in subclasses, like in the case of [QCPLayoutElement](#)), this function returns true only if this layerable has its visibility set to true and the parent layerable's [realVisibility](#) returns true.

10.443.3.8 selectTest()

```
double QCPLayerable::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPLItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the `mouseReleaseEvent` occurs, and the finally selected object is notified via the `selectEvent/` `deselectEvent` methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to `selectEvent` when the parent [QCustomPlot](#) decides on the basis of this `selectTest` call, that the object was successfully selected. The subsequent call to `selectEvent` will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in `selectTest`. The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent `selectEvent`, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

`selectEvent`, `deselectEvent`, `mousePressEvent`, `wheelEvent`, `QCustomPlot::setInteractions`, `QCPAbstractPlottable1D::selectTest`

Reimplemented in [QCPPolarGraph](#), [QCPPolarAxisRadial](#), [QCPPolarAxisAngular](#), [QCPLayoutElement](#), [QCPLayoutInset](#), [QCPAxis](#), [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPCurve >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatBox >](#), [QCPAbstractLegendItem](#), [QCPLegend](#), [QCPLegendItem](#), [QCPTextElement](#), [QCPGraph](#), [QCPCurve](#), [QCPBars](#), [QCPStatisticalBox](#), [QCPColorMap](#), [QCPFinancial](#), [QCPErrorBars](#), [QCPItemStraightLine](#), [QCPItemLine](#), [QCPItemCurve](#), [QCPItemRect](#), [QCPItemText](#), [QCPItemEllipse](#), [QCPItemPixmap](#), [QCPItemTracer](#), [QCPItemBracket](#), [QCPAbstractPlottable](#), and [QCPAbstractItem](#).

10.443.3.9 setAntialiased()

```
void QCPLayerable::setAntialiased (
    bool enabled )
```

Sets whether this object will be drawn antialiased or not.

Note that antialiasing settings may be overridden by `QCustomPlot::setAntialiasedElements` and `QCustomPlot::setNotAntialiasedElements`.

10.443.3.10 setLayer() [1/2]

```
bool QCPLayerable::setLayer (
    const QString & layerName )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts. Sets the layer of this layerable object by name

Returns true on success, i.e. if *layerName* is a valid layer name.

10.443.3.11 setLayer() [2/2]

```
bool QCPLayerable::setLayer (
    QCPLayer * layer )
```

Sets the *layer* of this layerable object. The object will be placed on top of the other objects already on *layer*.

If *layer* is 0, this layerable will not be on any layer and thus not appear in the plot (or interact/receive events).

Returns true if the layer of this layerable was successfully changed to *layer*.

10.443.3.12 setVisible()

```
void QCPLayerable::setVisible (
    bool on )
```

Sets the visibility of this layerable object. If an object is not visible, it will not be drawn on the [QCustomPlot](#) surface, and user interaction with it (e.g. click and selection) is not possible.

10.443.3.13 wheelEvent()

```
void QCPLayerable::wheelEvent (
    QWheelEvent * event ) [protected], [virtual]
```

This event gets called when the user turns the mouse scroll wheel while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`.

The `event->angleDelta()` indicates how far the mouse wheel was turned, which is usually +/- 120 for single rotation steps. However, if the mouse wheel is turned rapidly, multiple steps may accumulate to one event, making the delta larger. On the other hand, if the wheel has very smooth steps or none at all, the delta may be smaller.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#)

Reimplemented in [QCPAxis](#), [QCPAxisRect](#), [QCPCColorScaleAxisRectPrivate](#), [QCPCColorScale](#), [QCPPolarAxisRadial](#), and [QCPPolarAxisAngular](#).

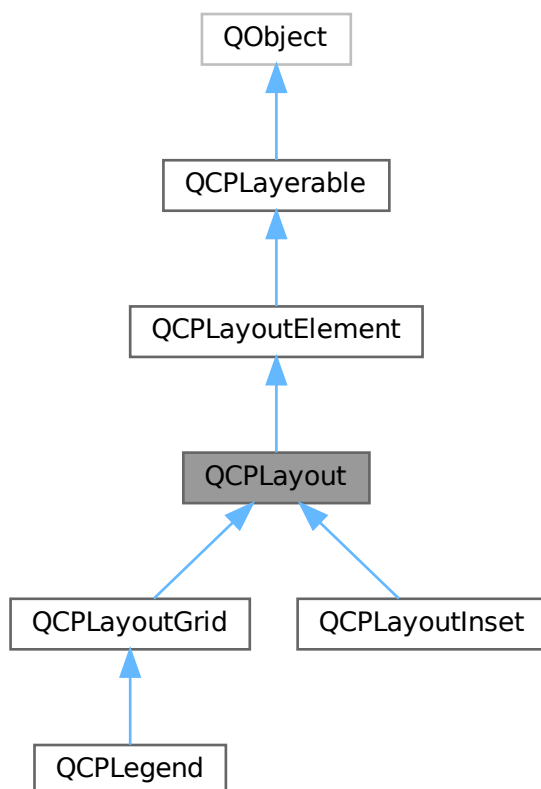
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.444 QCPLayout Class Reference

The abstract base class for layouts.

Inheritance diagram for QCPLayout:



- void [setMinimumMargins](#) (const QMargins &margins)
- void [setAutoMargins](#) (QCP::MarginSides sides)
- void [setMinimumSize](#) (const QSize &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const QSize &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) (SizeConstraintRect constraintRect)
- void [setMarginGroup](#) (QCP::MarginSides sides, [QCPMarginGroup](#) *group)
- virtual QSize [minimumOuterSizeHint](#) () const
- virtual QSize [maximumOuterSizeHint](#) () const
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void **updateLayout** ()
- void [sizeConstraintsChanged](#) () const
- void **adoptElement** ([QCPLayoutElement](#) *el)
- void **releaseElement** ([QCPLayoutElement](#) *el)
- QVector< int > **getSectionSizes** (QVector< int > maxSizes, QVector< int > minSizes, QVector< double > stretchFactors, int totalSize) const

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionState↔ Changed)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Static Protected Member Functions

- static QSize **getFinalMinimumOuterSize** (const [QCPLayoutElement](#) *el)
- static QSize **getFinalMaximumOuterSize** (const [QCPLayoutElement](#) *el)

Friends

- class [QCPLayoutElement](#)

Additional Inherited Members

Public Types inherited from [QCPLayoutElement](#)

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer](#)< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

10.444.1 Detailed Description

The abstract base class for layouts.

This is an abstract base class for layout elements whose main purpose is to define the position and size of other child layout elements. In most cases, layouts don't draw anything themselves (but there are exceptions to this, e.g. [QCPLegend](#)).

[QCPLayout](#) derives from [QCPLayoutElement](#), and thus can itself be nested in other layouts.

[QCPLayout](#) introduces a common interface for accessing and manipulating the child elements. Those functions are most notably [elementCount](#), [elementAt](#), [takeAt](#), [take](#), [simplify](#), [removeAt](#), [remove](#) and [clear](#). Individual subclasses may add more functions to this interface which are more specialized to the form of the layout. For example, [QCPLayoutGrid](#) adds functions that take row and column indices to access cells of the layout grid more conveniently.

Since this is an abstract base class, you can't instantiate it directly. Rather use one of its subclasses like [QCPLayoutGrid](#) or [QCPLayoutInset](#).

For a general introduction to the layout system, see the dedicated documentation page [The Layout System](#).

10.444.2 Constructor & Destructor Documentation

10.444.2.1 [QCPLayout](#)()

```
QCPLayout::QCPLayout ( ) [explicit]
```

Creates an instance of [QCPLayout](#) and sets default values. Note that since [QCPLayout](#) is an abstract base class, it can't be instantiated directly.

10.444.3 Member Function Documentation

10.444.3.1 [clear](#)()

```
void QCPLayout::clear ( )
```

Removes and deletes all layout elements in this layout. Finally calls [simplify](#) to make sure all empty cells are collapsed.

See also

[remove](#), [removeAt](#)

10.444.3.2 elementAt()

```
QCPLayoutElement * QCPLayout::elementAt (
    int index ) const [pure virtual]
```

Returns the element in the cell with the given *index*. If *index* is invalid, returns `nullptr`.

Note that even if *index* is valid, the respective cell may be empty in some layouts (e.g. [QCPLayoutGrid](#)), so this function may return `nullptr` in those cases. You may use this function to check whether a cell is empty or not.

See also

[elements](#), [elementCount](#), [takeAt](#)

Implemented in [QCPLayoutGrid](#), and [QCPLayoutInset](#).

10.444.3.3 elementCount()

```
int QCPLayout::elementCount ( ) const [pure virtual]
```

Returns the number of elements/cells in the layout.

See also

[elements](#), [elementAt](#)

Implemented in [QCPLayoutGrid](#), and [QCPLayoutInset](#).

10.444.3.4 elements()

```
QList< QCPLayoutElement * > QCPLayout::elements (
    bool recursive ) const [virtual]
```

Returns a list of all child elements in this layout element. If *recursive* is true, all sub-child elements are included in the list, too.

Warning

There may be `nullptr` entries in the returned list. For example, [QCPLayoutGrid](#) may have empty cells which yield `nullptr` at the respective index.

Reimplemented from [QCPLayoutElement](#).

Reimplemented in [QCPLayoutGrid](#).

10.444.3.5 `remove()`

```
bool QCPLayout::remove (
    QCPLayoutElement * element )
```

Removes and deletes the provided *element*. Returns true on success. If *element* is not in the layout, returns false.

This function internally uses [takeAt](#) to remove the element from the layout and then deletes the element. Note that some layouts don't remove the respective cell right away but leave an empty cell after successful removal of the layout element. To collapse empty cells, use [simplify](#).

See also

[removeAt](#), [take](#)

10.444.3.6 `removeAt()`

```
bool QCPLayout::removeAt (
    int index )
```

Removes and deletes the element at the provided *index*. Returns true on success. If *index* is invalid or points to an empty cell, returns false.

This function internally uses [takeAt](#) to remove the element from the layout and then deletes the returned element. Note that some layouts don't remove the respective cell right away but leave an empty cell after successful removal of the layout element. To collapse empty cells, use [simplify](#).

See also

[remove](#), [takeAt](#)

10.444.3.7 `simplify()`

```
void QCPLayout::simplify ( ) [virtual]
```

Simplifies the layout by collapsing empty cells. The exact behavior depends on subclasses, the default implementation does nothing.

Not all layouts need simplification. For example, [QCPLayoutInset](#) doesn't use explicit simplification while [QCPLayoutGrid](#) does.

Reimplemented in [QCPLayoutGrid](#), and [QCPLayoutInset](#).

10.444.3.8 `sizeConstraintsChanged()`

```
void QCPLayout::sizeConstraintsChanged ( ) const [protected]
```

Subclasses call this method to report changed (minimum/maximum) size constraints.

If the parent of this layout is again a [QCPLayout](#), forwards the call to the parent's [sizeConstraintsChanged](#). If the parent is a [QWidget](#) (i.e. is the [QCustomPlot::plotLayout](#) of [QCustomPlot](#)), calls [QWidget::updateGeometry](#), so if the [QCustomPlot](#) widget is inside a Qt [QLayout](#), it may update itself and resize cells accordingly.

10.444.3.9 take()

```
bool QCPLLayout::take (
    QCPLLayoutElement * element ) [pure virtual]
```

Removes the specified *element* from the layout and returns true on success.

If the *element* isn't in this layout, returns false.

Note that some layouts don't remove the respective cell right away but leave an empty cell after successful removal of the layout element. To collapse empty cells, use [simplify](#).

See also

[takeAt](#)

Implemented in [QCPLLayoutGrid](#), and [QCPLLayoutInset](#).

10.444.3.10 takeAt()

```
QCPLLayoutElement * QCPLLayout::takeAt (
    int index ) [pure virtual]
```

Removes the element with the given *index* from the layout and returns it.

If the *index* is invalid or the cell with that index is empty, returns `nullptr`.

Note that some layouts don't remove the respective cell right away but leave an empty cell after successful removal of the layout element. To collapse empty cells, use [simplify](#).

See also

[elementAt](#), [take](#)

Implemented in [QCPLLayoutGrid](#), and [QCPLLayoutInset](#).

10.444.3.11 update()

```
void QCPLLayout::update (
    UpdatePhase phase ) [virtual]
```

If *phase* is [upLayout](#), calls `updateLayout`, which subclasses may reimplement to reposition and resize their cells.

Finally, the call is propagated down to all child [QCPLLayoutElements](#).

For details about this method and the update phases, see the documentation of [QCPLLayoutElement::update](#).

Reimplemented from [QCPLLayoutElement](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

Public Member Functions

- [QCPLayOutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayOut](#) * [layout](#) () const
- [QRect](#) [rect](#) () const
- [QRect](#) [outerRect](#) () const
- [QMargins](#) [margins](#) () const
- [QMargins](#) [minimumMargins](#) () const
- [QCP::MarginSides](#) [autoMargins](#) () const
- [QSize](#) [minimumSize](#) () const
- [QSize](#) [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- [QHash](#)< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const [QRect](#) &[rect](#))
- void [setMargins](#) (const [QMargins](#) &margins)
- void [setMinimumMargins](#) (const [QMargins](#) &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)
- void [setMinimumSize](#) (const [QSize](#) &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const [QSize](#) &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)
- virtual void [update](#) ([UpdatePhase](#) phase)
- virtual [QSize](#) [minimumOuterSizeHint](#) () const
- virtual [QSize](#) [maximumOuterSizeHint](#) () const
- virtual [QList](#)< [QCPLayOutElement](#) * > [elements](#) (bool recursive) const
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_↔](#)
DECL_OVERRIDE

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool [visible](#) () const
- [QCustomPlot](#) * [parentPlot](#) () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * [layer](#) () const
- bool [antialiased](#) () const
- void [setVisible](#) (bool on)
- [Q_SLOT](#) bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const [QString](#) &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual int [calculateAutoMargin](#) ([QCP::MarginSide](#) side)
- virtual void [layoutChanged](#) ()
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const [Q_DECL_OVERRIDE](#)
- virtual void [draw](#) ([QCPPainter](#) *painter) [Q_DECL_OVERRIDE](#)
- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) [Q_DECL_OVERRIDE](#)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionState↔ Changed)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPLayout](#)
- class [QCPMarginGroup](#)

Additional Inherited Members

Signals inherited from [QCPLayerable](#)

- void **layerChanged** ([QCPLayer](#) *newLayer)

10.445.1 Detailed Description

The abstract base class for all objects that form the layout system.

This is an abstract base class. As such, it can't be instantiated directly, rather use one of its subclasses.

A Layout element is a rectangular object which can be placed in layouts. It has an outer rect ([QCPLayoutElement::outerRect](#)) and an inner rect ([QCPLayoutElement::rect](#)). The difference between outer and inner rect is called its margin. The margin can either be set to automatic or manual ([setAutoMargins](#)) on a per-side basis. If a side is set to manual, that margin can be set explicitly with [setMargins](#) and will stay fixed at that value. If it's set to automatic, the layout element subclass will control the value itself (via [calculateAutoMargin](#)).

Layout elements can be placed in layouts (base class [QCPLayout](#)) like [QCPLayoutGrid](#). The top level layout is reachable via [QCustomPlot::plotLayout](#), and is a [QCPLayoutGrid](#). Since [QCPLayout](#) itself derives from [QCPLayoutElement](#), layouts can be nested.

Thus in [QCustomPlot](#) one can divide layout elements into two categories: The ones that are invisible by themselves, because they don't draw anything. Their only purpose is to manage the position and size of other layout elements. This category of layout elements usually use [QCPLayout](#) as base class. Then there is the category of layout elements which actually draw something. For example, [QCPAxisRect](#), [QCPLegend](#) and [QCPTextElement](#) are of this category. This does not necessarily mean that the latter category can't have child layout elements. [QCPLegend](#) for instance, actually derives from [QCPLayoutGrid](#) and the individual legend items are child layout elements in the grid layout.

10.445.2 Member Enumeration Documentation

10.445.2.1 SizeConstraintRect

```
enum QCPLayoutElement::SizeConstraintRect
```

Defines to which rect of a layout element the size constraints that can be set via [setMinimumSize](#) and [setMaximumSize](#) apply. The outer rect ([outerRect](#)) includes the margins (e.g. in the case of a [QCPAxisRect](#) the axis labels), whereas the inner rect ([rect](#)) does not.

See also

[setSizeConstraintRect](#)

Enumerator

scrInnerRect	Minimum/Maximum size constraints apply to inner rect.
scrOuterRect	Minimum/Maximum size constraints apply to outer rect, thus include layout element margins.

10.445.2.2 UpdatePhase

```
enum QCPLayoutElement::UpdatePhase
```

Defines the phases of the update process, that happens just before a replot. At each phase, [update](#) is called with the according UpdatePhase value.

Enumerator

upPreparation	Phase used for any type of preparation that needs to be done before margin calculation and layout.
upMargins	Phase in which the margins are calculated and set.
upLayout	Final phase in which the layout system places the rects of the elements.

10.445.3 Constructor & Destructor Documentation

10.445.3.1 QCPLayoutElement()

```
QCPLayoutElement::QCPLayoutElement (
    QCustomPlot * parentPlot = nullptr ) [explicit]
```

Creates an instance of [QCPLayoutElement](#) and sets default values.

10.445.4 Member Function Documentation

10.445.4.1 applyDefaultAntialiasingHint()

```
virtual void QCPLayoutElement::applyDefaultAntialiasingHint (
    QCPainter * painter ) const [inline], [protected], [virtual]
```

Implements [QCPLayerable](#).

10.445.4.2 draw()

```
virtual void QCPLayoutElement::draw (
    QCPainter * painter ) [inline], [protected], [virtual]
```

Implements [QCPLayerable](#).

10.445.4.3 elements()

```
QList< QCPLayoutElement * > QCPLayoutElement::elements (
    bool recursive ) const [virtual]
```

Returns a list of all child elements in this layout element. If *recursive* is true, all sub-child elements are included in the list, too.

Warning

There may be `nullptr` entries in the returned list. For example, [QCPLayoutGrid](#) may have empty cells which yield `nullptr` at the respective index.

Reimplemented in [QCPLayout](#), [QCPLayoutGrid](#), [QCPAxisRect](#), and [QCPPolarAxisAngular](#).

10.445.4.4 layout()

```
QCPLayOut * QCPLayOutElement::layout ( ) const [inline]
```

Returns the parent layout of this layout element.

10.445.4.5 maximumOuterSizeHint()

```
QSize QCPLayOutElement::maximumOuterSizeHint ( ) const [virtual]
```

Returns the suggested maximum size this layout element (the [outerRect](#)) may be expanded to, if no manual maximum size is set.

if a maximum size ([setMaximumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through [QCPLayOut::getFinalMaximumOuterSize](#)) to determine the maximum allowed size of this layout element.

A manual maximum size is considered set if it is smaller than Qt's `QWIDGETSIZE_MAX`.

The default implementation simply returns `QWIDGETSIZE_MAX` for both width and height, implying no suggested maximum size. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented in [QCPLayOutGrid](#), and [QCPTextElement](#).

10.445.4.6 minimumOuterSizeHint()

```
QSize QCPLayOutElement::minimumOuterSizeHint ( ) const [virtual]
```

Returns the suggested minimum size this layout element (the [outerRect](#)) may be compressed to, if no manual minimum size is set.

if a minimum size ([setMinimumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through [QCPLayOut::getFinalMinimumOuterSize](#)) to determine the minimum allowed size of this layout element.

A manual minimum size is considered set if it is non-zero.

The default implementation simply returns the sum of the horizontal margins for the width and the sum of the vertical margins for the height. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented in [QCPLayOutGrid](#), [QCPPlottableLegendItem](#), [QCPTextElement](#), and [QCPPolarLegendItem](#).

10.445.4.7 outerRect()

```
QRect QCPLayOutElement::outerRect ( ) const [inline]
```

Returns the outer rect of this layout element. The outer rect is the inner rect expanded by the margins ([setMargins](#), [setAutoMargins](#)). The outer rect is used (and set via [setOuterRect](#)) by the parent [QCPLayOut](#) to control the size of this layout element.

See also

[rect](#)

10.445.4.8 parentPlotInitialized()

```
void QCPLayoutElement::parentPlotInitialized (
    QCustomPlot * parentPlot ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.445.4.9 rect()

```
QRect QCPLayoutElement::rect ( ) const [inline]
```

Returns the inner rect of this layout element. The inner rect is the outer rect ([outerRect](#), [setOuterRect](#)) shrunk by the margins ([setMargins](#), [setAutoMargins](#)).

In some cases, the area between outer and inner rect is left blank. In other cases the margin area is used to display peripheral graphics while the main content is in the inner rect. This is where automatic margin calculation becomes interesting because it allows the layout element to adapt the margins to the peripheral graphics it wants to draw. For example, [QCPAxisRect](#) draws the axis labels and tick labels in the margin area, thus needs to adjust the margins (if [setAutoMargins](#) is enabled) according to the space required by the labels of the axes.

See also

[outerRect](#)

10.445.4.10 selectTest()

```
double QCPLayoutElement::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Layout elements are sensitive to events inside their outer rect. If *pos* is within the outer rect, this method returns a value corresponding to 0.99 times the parent plot's selection tolerance. However, layout elements are not selectable by default. So if *onlySelectable* is true, -1.0 is returned.

See [QCPLayerable::selectTest](#) for a general explanation of this virtual method.

[QCPLayoutElement](#) subclasses may reimplement this method to provide more specific selection test behaviour.

Reimplemented from [QCPLayerable](#).

Reimplemented in [QCPPolarAxisAngular](#), [QCPLayoutInset](#), [QCPAbstractLegendItem](#), [QCPLegend](#), and [QCPTextElement](#).

10.445.4.11 setAutoMargins()

```
void QCPLayoutElement::setAutoMargins (
    QCP::MarginSides sides )
```

Sets on which sides the margin shall be calculated automatically. If a side is calculated automatically, a minimum margin value may be provided with [setMinimumMargins](#). If a side is set to be controlled manually, the value may be specified with [setMargins](#).

Margin sides that are under automatic control may participate in a [QCPMarginGroup](#) (see [setMarginGroup](#)), to synchronize (align) it with other layout elements in the plot.

See also

[setMinimumMargins](#), [setMargins](#), [QCP::MarginSide](#)

10.445.4.12 setMarginGroup()

```
void QCPLayoutElement::setMarginGroup (
    QCP::MarginSides sides,
    QCPMarginGroup * group )
```

Sets the margin *group* of the specified margin *sides*.

Margin groups allow synchronizing specified margins across layout elements, see the documentation of [QCPMarginGroup](#).

To unset the margin group of *sides*, set *group* to `nullptr`.

Note that margin groups only work for margin sides that are set to automatic ([setAutoMargins](#)).

See also

[QCP::MarginSide](#)

10.445.4.13 setMargins()

```
void QCPLayoutElement::setMargins (
    const QMargins & margins )
```

Sets the margins of this layout element. If [setAutoMargins](#) is disabled for some or all sides, this function is used to manually set the margin on those sides. Sides that are still set to be handled automatically are ignored and may have any value in *margins*.

The margin is the distance between the outer rect (controlled by the parent layout via [setOuterRect](#)) and the inner [rect](#) (which usually contains the main content of this layout element).

See also

[setAutoMargins](#)

10.445.4.14 setMaximumSize() [1/2]

```
void QCPLayoutElement::setMaximumSize (
    const QSize & size )
```

Sets the maximum size of this layout element. A parent layout tries to respect the *size* here by changing row/column sizes in the layout accordingly.

Whether this constraint applies to the inner or the outer rect can be specified with [setSizeConstraintRect](#) (see [rect](#) and [outerRect](#)).

10.445.4.15 `setMaximumSize()` [2/2]

```
void QCPLayoutElement::setMaximumSize (
    int width,
    int height )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the maximum size of this layout element.

Whether this constraint applies to the inner or the outer rect can be specified with [setSizeConstraintRect](#) (see [rect](#) and [outerRect](#)).

10.445.4.16 `setMinimumMargins()`

```
void QCPLayoutElement::setMinimumMargins (
    const QMargins & margins )
```

If [setAutoMargins](#) is enabled on some or all margins, this function is used to provide minimum values for those margins.

The minimum values are not enforced on margin sides that were set to be under manual control via [setAutoMargins](#).

See also

[setAutoMargins](#)

10.445.4.17 `setMinimumSize()` [1/2]

```
void QCPLayoutElement::setMinimumSize (
    const QSize & size )
```

Sets the minimum size of this layout element. A parent layout tries to respect the *size* here by changing row/column sizes in the layout accordingly.

If the parent layout size is not sufficient to satisfy all minimum size constraints of its child layout elements, the layout may set a size that is actually smaller than *size*. [QCustomPlot](#) propagates the layout's size constraints to the outside by setting its own minimum `QWidget` size accordingly, so violations of *size* should be exceptions.

Whether this constraint applies to the inner or the outer rect can be specified with [setSizeConstraintRect](#) (see [rect](#) and [outerRect](#)).

10.445.4.18 `setMinimumSize()` [2/2]

```
void QCPLayoutElement::setMinimumSize (
    int width,
    int height )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the minimum size of this layout element.

Whether this constraint applies to the inner or the outer rect can be specified with [setSizeConstraintRect](#) (see [rect](#) and [outerRect](#)).

10.445.4.19 setOuterRect()

```
void QCPLayoutElement::setOuterRect (
    const QRect & rect )
```

Sets the outer rect of this layout element. If the layout element is inside a layout, the layout sets the position and size of this layout element using this function.

Calling this function externally has no effect, since the layout will overwrite any changes to the outer rect upon the next replot.

The layout element will adapt its inner [rect](#) by applying the margins inward to the outer rect.

See also

[rect](#)

10.445.4.20 setSizeConstraintRect()

```
void QCPLayoutElement::setSizeConstraintRect (
    SizeConstraintRect constraintRect )
```

Sets to which rect of a layout element the size constraints apply. Size constraints can be set via [setMinimumSize](#) and [setMaximumSize](#).

The outer rect ([outerRect](#)) includes the margins (e.g. in the case of a [QCPAxisRect](#) the axis labels), whereas the inner rect ([rect](#)) does not.

See also

[setMinimumSize](#), [setMaximumSize](#)

10.445.4.21 update()

```
void QCPLayoutElement::update (
    UpdatePhase phase ) [virtual]
```

Updates the layout element and sub-elements. This function is automatically called before every replot by the parent layout element. It is called multiple times, once for every [UpdatePhase](#). The phases are run through in the order of the enum values. For details about what happens at the different phases, see the documentation of [UpdatePhase](#).

Layout elements that have child elements should call the [update](#) method of their child elements, and pass the current *phase* unchanged.

The default implementation executes the automatic margin mechanism in the [upMargins](#) phase. Subclasses should make sure to call the base class implementation.

Reimplemented in [QCPLayout](#), [QCPAxisRect](#), [QCPCoordinateAxisRectPrivate](#), [QCPCoordinateAxisRect](#), and [QCPPolarAxisAngular](#).

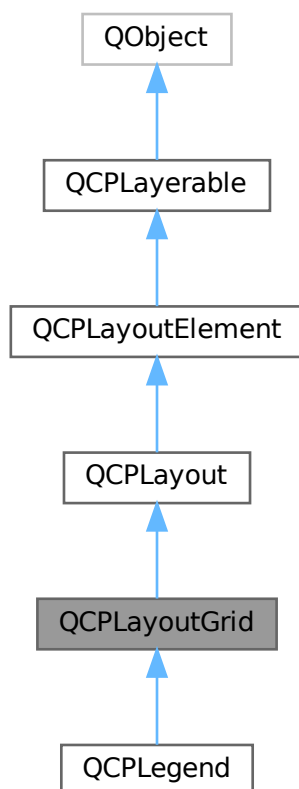
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

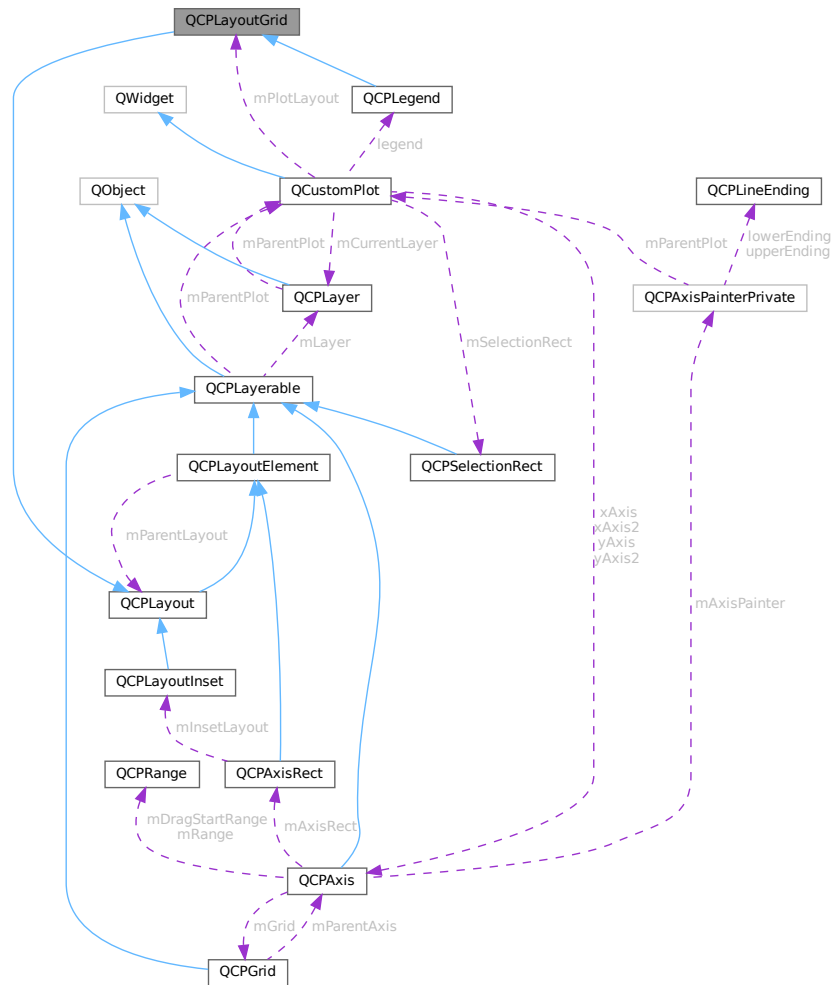
10.446 QCPLayoutGrid Class Reference

A layout that arranges child elements in a grid.

Inheritance diagram for QCPLayoutGrid:



Collaboration diagram for QCPLayouGrid:



Public Types

- enum `FillOrder` { `foRowsFirst` , `foColumnsFirst` }

Public Types inherited from QCPLayouElement

- enum `UpdatePhase` { `upPreparation` , `upMargins` , `upLayout` }
- enum `SizeConstraintRect` { `scrInnerRect` , `scrOuterRect` }

Public Member Functions

- `QCPLayouGrid` ()
- int `rowCount` () const
- int `columnCount` () const
- QList< double > `columnStretchFactors` () const
- QList< double > `rowStretchFactors` () const

- int **columnSpacing** () const
- int **rowSpacing** () const
- int **wrap** () const
- **FillOrder** **fillOrder** () const
- void **setColumnStretchFactor** (int column, double factor)
- void **setColumnStretchFactors** (const QList< double > &factors)
- void **setRowStretchFactor** (int row, double factor)
- void **setRowStretchFactors** (const QList< double > &factors)
- void **setColumnSpacing** (int pixels)
- void **setRowSpacing** (int pixels)
- void **setWrap** (int count)
- void **setFillOrder** (**FillOrder** order, bool rearrange=true)
- virtual void **updateLayout** () Q_DECL_OVERRIDE
- virtual int **elementCount** () const Q_DECL_OVERRIDE
- virtual **QCPLayoutElement** * **elementAt** (int index) const Q_DECL_OVERRIDE
- virtual **QCPLayoutElement** * **takeAt** (int index) Q_DECL_OVERRIDE
- virtual bool **take** (**QCPLayoutElement** *element) Q_DECL_OVERRIDE
- virtual QList< **QCPLayoutElement** * > **elements** (bool recursive) const Q_DECL_OVERRIDE
- virtual void **simplify** () Q_DECL_OVERRIDE
- virtual QSize **minimumOuterSizeHint** () const Q_DECL_OVERRIDE
- virtual QSize **maximumOuterSizeHint** () const Q_DECL_OVERRIDE
- **QCPLayoutElement** * **element** (int row, int column) const
- bool **addElement** (int row, int column, **QCPLayoutElement** *element)
- bool **addElement** (**QCPLayoutElement** *element)
- bool **hasElement** (int row, int column)
- void **expandTo** (int newRowCount, int newColumnCount)
- void **insertRow** (int newIndex)
- void **insertColumn** (int newIndex)
- int **rowColToIndex** (int row, int column) const
- void **indexToRowCol** (int index, int &row, int &column) const

Public Member Functions inherited from **QCPLayout**

- **QCPLayout** ()
- virtual void **update** (**UpdatePhase** phase) Q_DECL_OVERRIDE
- bool **removeAt** (int index)
- bool **remove** (**QCPLayoutElement** *element)
- void **clear** ()

Public Member Functions inherited from **QCPLayoutElement**

- **QCPLayoutElement** (**QCustomPlot** *parentPlot=nullptr)
- **QCPLayout** * **layout** () const
- **QRect** **rect** () const
- **QRect** **outerRect** () const
- **QMargins** **margins** () const
- **QMargins** **minimumMargins** () const
- **QCP::MarginSides** **autoMargins** () const
- **QSize** **minimumSize** () const
- **QSize** **maximumSize** () const
- **SizeConstraintRect** **sizeConstraintRect** () const
- **QCPMarginGroup** * **marginGroup** (**QCP::MarginSide** side) const
- **QHash**< **QCP::MarginSide**, **QCPMarginGroup** * > **marginGroups** () const

- void [setOuterRect](#) (const QRect &rect)
- void [setMargins](#) (const QMargins &margins)
- void [setMinimumMargins](#) (const QMargins &margins)
- void [setAutoMargins](#) (QCP::MarginSides sides)
- void [setMinimumSize](#) (const QSize &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const QSize &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) (QCP::MarginSides sides, [QCPMarginGroup](#) *group)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- void **getMinimumRowColSizes** (QVector< int > *minColWidths, QVector< int > *minRowHeights) const
- void **getMaximumRowColSizes** (QVector< int > *maxColWidths, QVector< int > *maxRowHeights) const

Protected Member Functions inherited from [QCPLayout](#)

- void [sizeConstraintsChanged](#) () const
- void **adoptElement** ([QCPLayoutElement](#) *el)
- void **releaseElement** ([QCPLayoutElement](#) *el)
- QVector< int > **getSectionSizes** (QVector< int > maxSizes, QVector< int > minSizes, QVector< double > stretchFactors, int totalSize) const

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionState← Changed)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** (QCustomPlot *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QList< QList< [QCPLayoutElement](#) * > > **mElements**
- QList< double > **mColumnStretchFactors**
- QList< double > **mRowStretchFactors**
- int **mColumnSpacing**
- int **mRowSpacing**
- int **mWrap**
- [FillOrder](#) **mFillOrder**

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

Static Protected Member Functions inherited from [QCPLayout](#)

- static QSize [getFinalMinimumOuterSize](#) (const [QCPLayoutElement](#) *el)
- static QSize [getFinalMaximumOuterSize](#) (const [QCPLayoutElement](#) *el)

10.446.1 Detailed Description

A layout that arranges child elements in a grid.

Elements are laid out in a grid with configurable stretch factors ([setColumnStretchFactor](#), [setRowStretchFactor](#)) and spacing ([setColumnSpacing](#), [setRowSpacing](#)).

Elements can be added to cells via [addElement](#). The grid is expanded if the specified row or column doesn't exist yet. Whether a cell contains a valid layout element can be checked with [hasElement](#), that element can be retrieved with [element](#). If rows and columns that only have empty cells shall be removed, call [simplify](#). Removal of elements is either done by just adding the element to a different layout or by using the [QCPLayout](#) interface [take](#) or [remove](#).

If you use [addElement\(QCPLayoutElement*\)](#) without explicit parameters for *row* and *column*, the grid layout will choose the position according to the current [setFillOrder](#) and the wrapping ([setWrap](#)).

Row and column insertion can be performed with [insertRow](#) and [insertColumn](#).

10.446.2 Member Enumeration Documentation

10.446.2.1 FillOrder

enum [QCPLayoutGrid::FillOrder](#)

Defines in which direction the grid is filled when using [addElement\(QCPLayoutElement*\)](#). The column/row at which wrapping into the next row/column occurs can be specified with [setWrap](#).

See also

[setFillOrder](#)

Enumerator

foRowsFirst	Rows are filled first, and a new element is wrapped to the next column if the row count would exceed setWrap .
foColumnsFirst	Columns are filled first, and a new element is wrapped to the next row if the column count would exceed setWrap .

10.446.3 Constructor & Destructor Documentation

10.446.3.1 QCPLayoutGrid()

```
QCPLayoutGrid::QCPLayoutGrid ( ) [explicit]
```

Creates an instance of [QCPLayoutGrid](#) and sets default values.

10.446.4 Member Function Documentation

10.446.4.1 addElement() [1/2]

```
bool QCPLayoutGrid::addElement (
    int row,
    int column,
    QCPLayoutElement * element )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the *element* to cell with *row* and *column*. If *element* is already in a layout, it is first removed from there. If *row* or *column* don't exist yet, the layout is expanded accordingly.

Returns true if the element was added successfully, i.e. if the cell at *row* and *column* didn't already have an element.

Use the overload of this method without explicit row/column index to place the element according to the configured fill order and wrapping settings.

See also

[element](#), [hasElement](#), [take](#), [remove](#)

10.446.4.2 addElement() [2/2]

```
bool QCPLayoutGrid::addElement (
    QCPLayoutElement * element )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the *element* to the next empty cell according to the current fill order ([setFillOrder](#)) and wrapping ([setWrap](#)). If *element* is already in a layout, it is first removed from there. If necessary, the layout is expanded to hold the new element.

Returns true if the element was added successfully.

See also

[setFillOrder](#), [setWrap](#), [element](#), [hasElement](#), [take](#), [remove](#)

10.446.4.3 columnCount()

```
int QCPLayoutGrid::columnCount ( ) const [inline]
```

Returns the number of columns in the layout.

See also

[rowCount](#)

10.446.4.4 element()

```
QCPLayoutElement * QCPLayoutGrid::element (
    int row,
    int column ) const
```

Returns the element in the cell in *row* and *column*.

Returns `nullptr` if either the row/column is invalid or if the cell is empty. In those cases, a `qDebug` message is printed. To check whether a cell exists and isn't empty, use [hasElement](#).

See also

[addElement](#), [hasElement](#)

10.446.4.5 elementAt()

```
QCPLayoutElement * QCPLayoutGrid::elementAt (
    int index ) const [virtual]
```

\see [baseclassmethod](#)

Note that the association of the linear *index* to the row/column based cells depends on the current setting of [setFillOrder](#).

See also

[rowColToIndex](#)

Implements [QCPLayout](#).

10.446.4.6 elementCount()

```
virtual int QCPLayoutGrid::elementCount ( ) const [inline], [virtual]
```

Returns the number of elements/cells in the layout.

See also

[elements](#), [elementAt](#)

Implements [QCPLayout](#).

10.446.4.7 elements()

```
QList< QCPLayoutElement * > QCPLayoutGrid::elements (
    bool recursive ) const [virtual]
```

Returns a list of all child elements in this layout element. If *recursive* is true, all sub-child elements are included in the list, too.

Warning

There may be `nullptr` entries in the returned list. For example, [QCPLayoutGrid](#) may have empty cells which yield `nullptr` at the respective index.

Reimplemented from [QCPLayout](#).

10.446.4.8 expandTo()

```
void QCPLayoutGrid::expandTo (
    int newRowCount,
    int newColumnCount )
```

Expands the layout to have *newRowCount* rows and *newColumnCount* columns. So the last valid row index will be *newRowCount-1*, the last valid column index will be *newColumnCount-1*.

If the current column/row count is already larger or equal to *newColumnCount/newRowCount*, this function does nothing in that dimension.

Newly created cells are empty, new rows and columns have the stretch factor 1.

Note that upon a call to [addElement](#), the layout is expanded automatically to contain the specified row and column, using this function.

See also

[simplify](#)

10.446.4.9 hasElement()

```
bool QCPLayoutGrid::hasElement (
    int row,
    int column )
```

Returns whether the cell at *row* and *column* exists and contains a valid element, i.e. isn't empty.

See also

[element](#)

10.446.4.10 indexToRowCol()

```
void QCPLayoutGrid::indexToRowCol (
    int index,
    int & row,
    int & column ) const
```

Converts the linear index to row and column indices and writes the result to *row* and *column*.

The way the cells are indexed depends on [setFillOrder](#). If it is [foRowsFirst](#), the indices increase left to right and then top to bottom. If it is [foColumnsFirst](#), the indices increase top to bottom and then left to right.

If there are no cells (i.e. column or row count is zero), sets *row* and *column* to -1.

For the retrieved *row* and *column* to be valid, the passed *index* must be valid itself, i.e. greater or equal to zero and smaller than the current [elementCount](#).

See also

[rowColToIndex](#)

10.446.4.11 insertColumn()

```
void QCPLayoutGrid::insertColumn (
    int newIndex )
```

Inserts a new column with empty cells at the column index *newIndex*. Valid values for *newIndex* range from 0 (inserts a column at the left) to *columnCount* (appends a column at the right).

See also

[insertRow](#)

10.446.4.12 insertRow()

```
void QCPLayoutGrid::insertRow (
    int newIndex )
```

Inserts a new row with empty cells at the row index *newIndex*. Valid values for *newIndex* range from 0 (inserts a row at the top) to *rowCount* (appends a row at the bottom).

See also

[insertColumn](#)

10.446.4.13 `maximumOuterSizeHint()`

```
QSize QCPLayoutGrid::maximumOuterSizeHint ( ) const [virtual]
```

Returns the suggested maximum size this layout element (the [outerRect](#)) may be expanded to, if no manual maximum size is set.

if a maximum size ([setMaximumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through `QCPLayout::getFinalMaximumOuterSize`) to determine the maximum allowed size of this layout element.

A manual maximum size is considered set if it is smaller than Qt's `QWIDGETSIZE_MAX`.

The default implementation simply returns `QWIDGETSIZE_MAX` for both width and height, implying no suggested maximum size. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented from [QCPLayoutElement](#).

10.446.4.14 `minimumOuterSizeHint()`

```
QSize QCPLayoutGrid::minimumOuterSizeHint ( ) const [virtual]
```

Returns the suggested minimum size this layout element (the [outerRect](#)) may be compressed to, if no manual minimum size is set.

if a minimum size ([setMinimumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through `QCPLayout::getFinalMinimumOuterSize`) to determine the minimum allowed size of this layout element.

A manual minimum size is considered set if it is non-zero.

The default implementation simply returns the sum of the horizontal margins for the width and the sum of the vertical margins for the height. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented from [QCPLayoutElement](#).

10.446.4.15 `rowColToIndex()`

```
int QCPLayoutGrid::rowColToIndex (
    int row,
    int column ) const
```

Converts the given *row* and *column* to the linear index used by some methods of [QCPLayoutGrid](#) and [QCPLayout](#).

The way the cells are indexed depends on [setFillOrder](#). If it is [foRowsFirst](#), the indices increase left to right and then top to bottom. If it is [foColumnsFirst](#), the indices increase top to bottom and then left to right.

For the returned index to be valid, *row* and *column* must be valid indices themselves, i.e. greater or equal to zero and smaller than the current `rowCount/ columnCount`.

See also

[indexToRowCol](#)

10.446.4.16 rowCount()

```
int QCPLayoutGrid::rowCount ( ) const [inline]
```

Returns the number of rows in the layout.

See also

[columnCount](#)

10.446.4.17 setColumnSpacing()

```
void QCPLayoutGrid::setColumnSpacing (
    int pixels )
```

Sets the gap that is left blank between columns to *pixels*.

See also

[setRowSpacing](#)

10.446.4.18 setColumnStretchFactor()

```
void QCPLayoutGrid::setColumnStretchFactor (
    int column,
    double factor )
```

Sets the stretch *factor* of *column*.

Stretch factors control the relative sizes of rows and columns. Cells will not be resized beyond their minimum and maximum widths/heights, regardless of the stretch factor. (see [QCPLayoutElement::setMinimumSize](#), [QCPLayoutElement::setMaximumSize](#), [QCPLayoutElement::setSizeConstraintRect](#).)

The default stretch factor of newly created rows/columns is 1.

See also

[setColumnStretchFactors](#), [setRowStretchFactor](#)

10.446.4.19 setColumnStretchFactors()

```
void QCPLayoutGrid::setColumnStretchFactors (
    const QList< double > & factors )
```

Sets the stretch *factors* of all columns. *factors* must have the size [columnCount](#).

Stretch factors control the relative sizes of rows and columns. Cells will not be resized beyond their minimum and maximum widths/heights, regardless of the stretch factor. (see [QCPLayoutElement::setMinimumSize](#), [QCPLayoutElement::setMaximumSize](#), [QCPLayoutElement::setSizeConstraintRect](#).)

The default stretch factor of newly created rows/columns is 1.

See also

[setColumnStretchFactor](#), [setRowStretchFactors](#)

10.446.4.20 `setFillOrder()`

```
void QCLayoutGrid::setFillOrder (
    FillOrder order,
    bool rearrange = true )
```

Sets the filling order and wrapping behaviour that is used when adding new elements with the method [addElement\(QCLayoutElement*\)](#).

The specified *order* defines whether rows or columns are filled first. Using [setWrap](#), you can control at which row/column count wrapping into the next column/row will occur. If you set it to zero, no wrapping will ever occur. Changing the fill order also changes the meaning of the linear index used e.g. in [elementAt](#) and [takeAt](#). The default fill order for `QCLayoutGrid` is [foColumnsFirst](#).

If you want to have all current elements arranged in the new order, set *rearrange* to true. The elements will be rearranged in a way that tries to preserve their linear index. However, empty cells are skipped during build-up of the new cell order, which shifts the succeeding element's index. The rearranging is performed even if the specified *order* is already the current fill order. Thus this method can be used to re-wrap the current elements.

If *rearrange* is false, the current element arrangement is not changed, which means the linear indexes change (because the linear index is dependent on the fill order).

Note that the method [addElement\(int row, int column, QCLayoutElement *element\)](#) with explicitly stated row and column is not subject to wrapping and can place elements even beyond the specified wrapping point.

See also

[setWrap](#), [addElement\(QCLayoutElement*\)](#)

10.446.4.21 `setRowSpacing()`

```
void QCLayoutGrid::setRowSpacing (
    int pixels )
```

Sets the gap that is left blank between rows to *pixels*.

See also

[setColumnSpacing](#)

10.446.4.22 `setRowStretchFactor()`

```
void QCLayoutGrid::setRowStretchFactor (
    int row,
    double factor )
```

Sets the stretch *factor* of *row*.

Stretch factors control the relative sizes of rows and columns. Cells will not be resized beyond their minimum and maximum widths/heights, regardless of the stretch factor. (see [QCLayoutElement::setMinimumSize](#), [QCLayoutElement::setMaximumSize](#), [QCLayoutElement::setSizeConstraintRect](#).)

The default stretch factor of newly created rows/columns is 1.

See also

[setColumnStretchFactors](#), [setRowStretchFactor](#)

10.446.4.23 setRowStretchFactors()

```
void QCPLayoutGrid::setRowStretchFactors (
    const QList< double > & factors )
```

Sets the stretch *factors* of all rows. *factors* must have the size [rowCount](#).

Stretch factors control the relative sizes of rows and columns. Cells will not be resized beyond their minimum and maximum widths/heights, regardless of the stretch factor. (see [QCPLayoutElement::setMinimumSize](#), [QCPLayoutElement::setMaximumSize](#), [QCPLayoutElement::setSizeConstraintRect](#).)

The default stretch factor of newly created rows/columns is 1.

See also

[setRowStretchFactor](#), [setColumnStretchFactors](#)

10.446.4.24 setWrap()

```
void QCPLayoutGrid::setWrap (
    int count )
```

Sets the maximum number of columns or rows that are used, before new elements added with [addElement\(QCPLayoutElement*\)](#) will start to fill the next row or column, respectively. It depends on [setFillOrder](#), whether rows or columns are wrapped.

If *count* is set to zero, no wrapping will ever occur.

If you wish to re-wrap the elements currently in the layout, call [setFillOrder](#) with *rearrange* set to true (the actual fill order doesn't need to be changed for the rearranging to be done).

Note that the method [addElement\(int row, int column, QCPLayoutElement *element\)](#) with explicitly stated row and column is not subject to wrapping and can place elements even beyond the specified wrapping point.

See also

[setFillOrder](#)

10.446.4.25 simplify()

```
void QCPLayoutGrid::simplify ( ) [virtual]
```

Simplifies the layout by collapsing rows and columns which only contain empty cells.

Reimplemented from [QCPLayout](#).

10.446.4.26 take()

```
bool QCPLayoutGrid::take (
    QCPLayoutElement * element ) [virtual]
```

Removes the specified *element* from the layout and returns true on success.

If the *element* isn't in this layout, returns false.

Note that some layouts don't remove the respective cell right away but leave an empty cell after successful removal of the layout element. To collapse empty cells, use [simplify](#).

See also

[takeAt](#)

Implements [QCPLayout](#).

10.446.4.27 takeAt()

```
QCPLayoutElement * QCPLayoutGrid::takeAt (
    int index ) [virtual]
```

\seebaseclassmethod

Note that the association of the linear *index* to the row/column based cells depends on the current setting of [setFillOrder](#).

See also

[rowColToIndex](#)

Implements [QCPLayout](#).

10.446.4.28 updateLayout()

```
void QCPLayoutGrid::updateLayout ( ) [virtual]
```

Reimplemented from [QCPLayout](#).

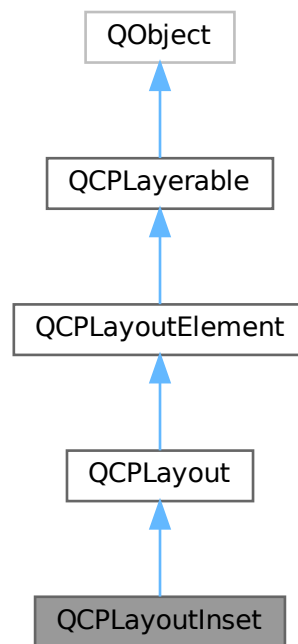
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.447 QCPLayoutInset Class Reference

A layout that places child elements aligned to the border or arbitrarily positioned.

Inheritance diagram for QCPLayoutInset:



Public Member Functions inherited from QCPLLayout

- [QCPLayout](#) ()
- virtual void [update](#) ([UpdatePhase](#) phase) Q_DECL_OVERRIDE
- virtual QList< [QCPLayoutElement](#) * > [elements](#) (bool recursive) const Q_DECL_OVERRIDE
- bool [removeAt](#) (int index)
- bool [remove](#) ([QCPLayoutElement](#) *element)
- void [clear](#) ()

Public Member Functions inherited from QCPLayoutElement

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- QRect [rect](#) () const
- QRect [outerRect](#) () const
- QMargins [margins](#) () const
- QMargins [minimumMargins](#) () const
- QCP::MarginSides [autoMargins](#) () const
- QSize [minimumSize](#) () const
- QSize [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const QRect &[rect](#))
- void [setMargins](#) (const QMargins &margins)
- void [setMinimumMargins](#) (const QMargins &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)
- void [setMinimumSize](#) (const QSize &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const QSize &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)
- virtual QSize [minimumOuterSizeHint](#) () const
- virtual QSize [maximumOuterSizeHint](#) () const

Public Member Functions inherited from QCPLayerable

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool [visible](#) () const
- [QCustomPlot](#) * [parentPlot](#) () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * [layer](#) () const
- bool [antialiased](#) () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Attributes

- `QList< QCPLayoutElement * > mElements`
- `QList< InsetPlacement > mInsetPlacement`
- `QList< Qt::Alignment > mInsetAlignment`
- `QList< QRectF > mInsetRect`

Protected Attributes inherited from [QCPLayoutElement](#)

- `QCPLayout * mParentLayout`
- `QSize mMinimumSize`
- `QSize mMaximumSize`
- `SizeConstraintRect mSizeConstraintRect`
- `QRect mRect`
- `QRect mOuterRect`
- `QMargins mMargins`
- `QMargins mMinimumMargins`
- `QCP::MarginSides mAutoMargins`
- `QHash< QCP::MarginSide, QCPMarginGroup * > mMarginGroups`

Protected Attributes inherited from [QCPLayerable](#)

- `bool mVisible`
- `QCustomPlot * mParentPlot`
- `QPointer< QCPLayerable > mParentLayerable`
- `QCPLayer * mLayer`
- `bool mAntialiased`

Additional Inherited Members**Signals inherited from [QCPLayerable](#)**

- `void layerChanged (QCPLayer *newLayer)`

Protected Member Functions inherited from [QCPLayout](#)

- `void sizeConstraintsChanged () const`
- `void adoptElement (QCPLayoutElement *el)`
- `void releaseElement (QCPLayoutElement *el)`
- `QVector< int > getSectionSizes (QVector< int > maxSizes, QVector< int > minSizes, QVector< double > stretchFactors, int totalSize) const`

Protected Member Functions inherited from [QCPLayoutElement](#)

- `virtual int calculateAutoMargin (QCP::MarginSide side)`
- `virtual void layoutChanged ()`
- `virtual void applyDefaultAntialiasingHint (QCPPainter *painter) const Q_DECL_OVERRIDE`
- `virtual void draw (QCPPainter *painter) Q_DECL_OVERRIDE`
- `virtual void parentPlotInitialized (QCustomPlot *parentPlot) Q_DECL_OVERRIDE`

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual [QRect](#) **clipRect** () const
- virtual void **selectEvent** ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState ← Changed)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mousePressEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **mouseMoveEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseReleaseEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseDoubleClickEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **wheelEvent** ([QWheelEvent](#) *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Static Protected Member Functions inherited from [QCPLayout](#)

- static [QSize](#) **getFinalMinimumOuterSize** (const [QCPLayoutElement](#) *el)
- static [QSize](#) **getFinalMaximumOuterSize** (const [QCPLayoutElement](#) *el)

10.447.1 Detailed Description

A layout that places child elements aligned to the border or arbitrarily positioned.

Elements are placed either aligned to the border or at arbitrary position in the area of the layout. Which placement applies is controlled with the [InsetPlacement](#) ([setInsetPlacement](#)).

Elements are added via [addElement\(QCPLayoutElement *element, Qt::Alignment alignment\)](#) or [addElement\(QCPLayoutElement *element, QRect rect\)](#). If the first method is used, the inset placement will default to [ipBorderAligned](#) and the element will be aligned according to the *alignment* parameter. The second method defaults to [ipFree](#) and allows placing elements at arbitrary position and size, defined by *rect*.

The alignment or rect can be set via [setInsetAlignment](#) or [setInsetRect](#), respectively.

This is the layout that every [QCPAxisRect](#) has as [QCPAxisRect::insetLayout](#).

10.447.2 Member Enumeration Documentation

10.447.2.1 InsetPlacement

enum [QCPLayoutInset::InsetPlacement](#)

Defines how the placement and sizing is handled for a certain element in a [QCPLayoutInset](#).

Enumerator

ipFree	The element may be positioned/sized arbitrarily, see setInsetRect .
ipBorderAligned	The element is aligned to one of the layout sides, see setInsetAlignment .

10.447.3 Constructor & Destructor Documentation

10.447.3.1 QCPLayoutInset()

```
QCPLayoutInset::QCPLayoutInset ( ) [explicit]
```

Creates an instance of [QCPLayoutInset](#) and sets default values.

10.447.4 Member Function Documentation

10.447.4.1 addElement() [1/2]

```
void QCPLayoutInset::addElement (
    QCPLayoutElement * element,
    const QRectF & rect )
```

Adds the specified *element* to the layout as an inset with free positioning/sizing ([setInsetAlignment](#) is initialized with [ipFree](#)). The position and size is set to *rect*.

rect is given in fractions of the whole inset layout rect. So an inset with rect (0, 0, 1, 1) will span the entire layout. An inset with rect (0.6, 0.1, 0.35, 0.35) will be in the top right corner of the layout, with 35% width and height of the parent layout.

See also

[addElement\(QCPLayoutElement *element, Qt::Alignment alignment\)](#)

10.447.4.2 addElement() [2/2]

```
void QCPLayoutInset::addElement (
    QCPLayoutElement * element,
    Qt::Alignment alignment )
```

Adds the specified *element* to the layout as an inset aligned at the border ([setInsetAlignment](#) is initialized with [ipBorderAligned](#)). The alignment is set to *alignment*.

alignment is an or combination of the following alignment flags: [Qt::AlignLeft](#), [Qt::AlignHCenter](#), [Qt::AlignRight](#), [Qt::AlignTop](#), [Qt::AlignVCenter](#), [Qt::AlignBottom](#). Any other alignment flags will be ignored.

See also

[addElement\(QCPLayoutElement *element, const QRectF &rect\)](#)

10.447.4.3 elementAt()

```
QCPLayoutElement * QCPLayoutInset::elementAt (
    int index ) const [virtual]
```

Returns the element in the cell with the given *index*. If *index* is invalid, returns `nullptr`.

Note that even if *index* is valid, the respective cell may be empty in some layouts (e.g. [QCPLayoutGrid](#)), so this function may return `nullptr` in those cases. You may use this function to check whether a cell is empty or not.

See also

[elements](#), [elementCount](#), [takeAt](#)

Implements [QCPLayout](#).

10.447.4.4 elementCount()

```
int QCPLayoutInset::elementCount ( ) const [virtual]
```

Returns the number of elements/cells in the layout.

See also

[elements](#), [elementAt](#)

Implements [QCPLayout](#).

10.447.4.5 insetAlignment()

```
Qt::Alignment QCPLayoutInset::insetAlignment (
    int index ) const
```

Returns the alignment of the element with the specified *index*. The alignment only has a meaning, if the inset placement ([setInsetPlacement](#)) is [ipBorderAligned](#).

10.447.4.6 insetPlacement()

```
QCPLayoutInset::InsetPlacement QCPLayoutInset::insetPlacement (
    int index ) const
```

Returns the placement type of the element with the specified *index*.

10.447.4.7 insetRect()

```
QRectF QCPLayoutInset::insetRect (
    int index ) const
```

Returns the rect of the element with the specified *index*. The rect only has a meaning, if the inset placement ([setInsetPlacement](#)) is [ipFree](#).

10.447.4.8 selectTest()

```
double QCPLayoutInset::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

The inset layout is sensitive to events only at areas where its (visible) child elements are sensitive. If the `selectTest` method of any of the child elements returns a positive number for *pos*, this method returns a value corresponding to 0.99 times the parent plot's selection tolerance. The inset layout is not selectable itself by default. So if *onlySelectable* is true, -1.0 is returned.

See [QCPLayerable::selectTest](#) for a general explanation of this virtual method.

Reimplemented from [QCPLayoutElement](#).

10.447.4.9 setInsetAlignment()

```
void QCPLayoutInset::setInsetAlignment (
    int index,
    Qt::Alignment alignment )
```

If the inset placement ([setInsetPlacement](#)) is [ipBorderAligned](#), this function is used to set the alignment of the element with the specified *index* to *alignment*.

alignment is an or combination of the following alignment flags: `Qt::AlignLeft`, `Qt::AlignHCenter`, `Qt::AlignRight`, `Qt::AlignTop`, `Qt::AlignVCenter`, `Qt::AlignBottom`. Any other alignment flags will be ignored.

10.447.4.10 setInsetPlacement()

```
void QCPLayoutInset::setInsetPlacement (
    int index,
    QCPLayoutInset::InsetPlacement placement )
```

Sets the inset placement type of the element with the specified *index* to *placement*.

See also

[InsetPlacement](#)

10.447.4.11 setInsetRect()

```
void QCPLayoutInset::setInsetRect (
    int index,
    const QRectF & rect )
```

If the inset placement ([setInsetPlacement](#)) is [ipFree](#), this function is used to set the position and size of the element with the specified *index* to *rect*.

rect is given in fractions of the whole inset layout rect. So an inset with rect (0, 0, 1, 1) will span the entire layout. An inset with rect (0.6, 0.1, 0.35, 0.35) will be in the top right corner of the layout, with 35% width and height of the parent layout.

Note that the minimum and maximum sizes of the embedded element ([QCPLayoutElement::setMinimumSize](#), [QCPLayoutElement::setMaximumSize](#)) are enforced.

10.447.4.12 `simplify()`

```
void QCPLayoutInset::simplify ( ) [inline], [virtual]
```

The QCPLInsetLayout does not need simplification since it can never have empty cells due to its linear index structure. This method does nothing.

Reimplemented from [QCPLayout](#).

10.447.4.13 `take()`

```
bool QCPLayoutInset::take (
    QCPLayoutElement * element ) [virtual]
```

Removes the specified *element* from the layout and returns true on success.

If the *element* isn't in this layout, returns false.

Note that some layouts don't remove the respective cell right away but leave an empty cell after successful removal of the layout element. To collapse empty cells, use [simplify](#).

See also

[takeAt](#)

Implements [QCPLayout](#).

10.447.4.14 `takeAt()`

```
QCPLayoutElement * QCPLayoutInset::takeAt (
    int index ) [virtual]
```

Removes the element with the given *index* from the layout and returns it.

If the *index* is invalid or the cell with that index is empty, returns `nullptr`.

Note that some layouts don't remove the respective cell right away but leave an empty cell after successful removal of the layout element. To collapse empty cells, use [simplify](#).

See also

[elementAt](#), [take](#)

Implements [QCPLayout](#).

10.447.4.15 `updateLayout()`

```
void QCPLayoutInset::updateLayout ( ) [virtual]
```

Reimplemented from [QCPLayout](#).

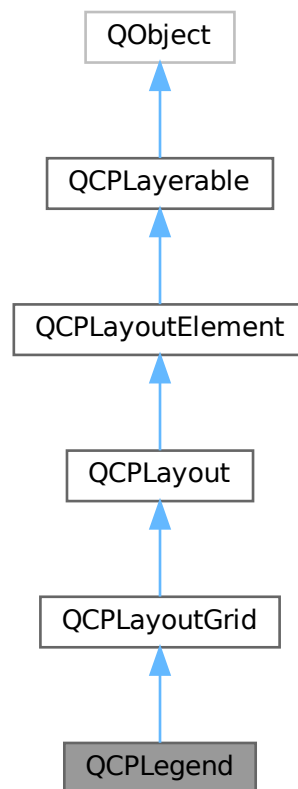
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.448 QCPLegend Class Reference

Manages a legend inside a [QCustomPlot](#).

Inheritance diagram for QCPLegend:



Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

Public Member Functions

- [QCPLegend](#) ()
- QPen **borderPen** () const
- QBrush **brush** () const
- QFont **font** () const
- QColor **textColor** () const
- QSize **iconSize** () const
- int **iconTextPadding** () const
- QPen **iconBorderPen** () const
- SelectableParts **selectableParts** () const
- SelectableParts **selectedParts** () const
- QPen **selectedBorderPen** () const
- QPen **selectedIconBorderPen** () const
- QBrush **selectedBrush** () const
- QFont **selectedFont** () const
- QColor **selectedTextColor** () const
- void [setBorderPen](#) (const QPen &pen)
- void [setBrush](#) (const QBrush &brush)
- void [setFont](#) (const QFont &font)
- void [setTextColor](#) (const QColor &color)
- void [setIconSize](#) (const QSize &size)
- void [setIconSize](#) (int width, int height)
- void [setIconTextPadding](#) (int padding)
- void [setIconBorderPen](#) (const QPen &pen)
- Q_SLOT void [setSelectableParts](#) (const SelectableParts &selectableParts)
- Q_SLOT void [setSelectedParts](#) (const SelectableParts &selectedParts)
- void [setSelectedBorderPen](#) (const QPen &pen)
- void [setSelectedIconBorderPen](#) (const QPen &pen)
- void [setSelectedBrush](#) (const QBrush &brush)
- void [setSelectedFont](#) (const QFont &font)
- void [setSelectedTextColor](#) (const QColor &color)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_
DECL_OVERRIDE
- [QCPAbstractLegendItem](#) * [item](#) (int index) const
- [QCPPlottableLegendItem](#) * [itemWithPlottable](#) (const [QCPAbstractPlottable](#) *plottable) const
- int [itemCount](#) () const
- bool [hasItem](#) ([QCPAbstractLegendItem](#) *item) const
- bool [hasItemWithPlottable](#) (const [QCPAbstractPlottable](#) *plottable) const
- bool [addItem](#) ([QCPAbstractLegendItem](#) *item)
- bool [removeItem](#) (int index)
- bool [removeItem](#) ([QCPAbstractLegendItem](#) *item)
- void [clearItems](#) ()
- QList< [QCPAbstractLegendItem](#) * > [selectedItems](#) () const

Public Member Functions inherited from [QCPLayoutGrid](#)

- [QCPLayoutGrid](#) ()
- int [rowCount](#) () const
- int [columnCount](#) () const
- QList< double > [columnStretchFactors](#) () const
- QList< double > [rowStretchFactors](#) () const
- int [columnSpacing](#) () const
- int [rowSpacing](#) () const
- int [wrap](#) () const
- [FillOrder](#) [fillOrder](#) () const
- void [setColumnStretchFactor](#) (int column, double factor)
- void [setColumnStretchFactors](#) (const QList< double > &factors)
- void [setRowStretchFactor](#) (int row, double factor)
- void [setRowStretchFactors](#) (const QList< double > &factors)
- void [setColumnSpacing](#) (int pixels)
- void [setRowSpacing](#) (int pixels)
- void [setWrap](#) (int count)
- void [setFillOrder](#) ([FillOrder](#) order, bool rearrange=true)
- virtual void [updateLayout](#) () Q_DECL_OVERRIDE
- virtual int [elementCount](#) () const Q_DECL_OVERRIDE
- virtual [QCPLayoutElement](#) * [elementAt](#) (int index) const Q_DECL_OVERRIDE
- virtual [QCPLayoutElement](#) * [takeAt](#) (int index) Q_DECL_OVERRIDE
- virtual bool [take](#) ([QCPLayoutElement](#) *element) Q_DECL_OVERRIDE
- virtual QList< [QCPLayoutElement](#) * > [elements](#) (bool recursive) const Q_DECL_OVERRIDE
- virtual void [simplify](#) () Q_DECL_OVERRIDE
- virtual QSize [minimumOuterSizeHint](#) () const Q_DECL_OVERRIDE
- virtual QSize [maximumOuterSizeHint](#) () const Q_DECL_OVERRIDE
- [QCPLayoutElement](#) * [element](#) (int row, int column) const
- bool [addElement](#) (int row, int column, [QCPLayoutElement](#) *element)
- bool [addElement](#) ([QCPLayoutElement](#) *element)
- bool [hasElement](#) (int row, int column)
- void [expandTo](#) (int newRowCount, int newColumnCount)
- void [insertRow](#) (int newIndex)
- void [insertColumn](#) (int newIndex)
- int [rowColToIndex](#) (int row, int column) const
- void [indexToRowCol](#) (int index, int &row, int &column) const

Public Member Functions inherited from [QCPLayout](#)

- [QCPLayout](#) ()
- virtual void [update](#) ([UpdatePhase](#) phase) Q_DECL_OVERRIDE
- bool [removeAt](#) (int index)
- bool [remove](#) ([QCPLayoutElement](#) *element)
- void [clear](#) ()

Public Member Functions inherited from [QCPLayoutElement](#)

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- [QRect](#) [rect](#) () const
- [QRect](#) [outerRect](#) () const
- [QMargins](#) [margins](#) () const
- [QMargins](#) [minimumMargins](#) () const
- [QCP::MarginSides](#) [autoMargins](#) () const
- [QSize](#) [minimumSize](#) () const
- [QSize](#) [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- [QHash](#)< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const [QRect](#) &[rect](#))
- void [setMargins](#) (const [QMargins](#) &margins)
- void [setMinimumMargins](#) (const [QMargins](#) &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)
- void [setMinimumSize](#) (const [QSize](#) &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const [QSize](#) &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool [visible](#) () const
- [QCustomPlot](#) * [parentPlot](#) () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * [layer](#) () const
- bool [antialiased](#) () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const [QString](#) &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void [selectEvent](#) ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState← Changed) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- [QPen](#) [getBorderPen](#) () const
- [QBrush](#) [getBrush](#) () const

Protected Member Functions inherited from [QCPLayoutGrid](#)

- void **getMinimumRowColSizes** (QVector< int > *minColWidths, QVector< int > *minRowHeights) const
- void **getMaximumRowColSizes** (QVector< int > *maxColWidths, QVector< int > *maxRowHeights) const

Protected Member Functions inherited from [QCPLayout](#)

- void [sizeConstraintsChanged](#) () const
- void **adoptElement** ([QCPLayoutElement](#) *el)
- void **releaseElement** ([QCPLayoutElement](#) *el)
- QVector< int > **getSectionSizes** (QVector< int > maxSizes, QVector< int > minSizes, QVector< double > stretchFactors, int totalSize) const

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()

Protected Member Functions inherited from [QCPLayerable](#)

- virtual QRect **clipRect** () const
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QPen **mBorderPen**
- QPen **mIconBorderPen**
- QBrush **mBrush**
- QFont **mFont**
- QColor **mTextColor**
- QSize **mIconSize**
- int **mIconTextPadding**
- SelectableParts **mSelectedParts**
- SelectableParts **mSelectableParts**
- QPen **mSelectedBorderPen**
- QPen **mSelectedIconBorderPen**
- QBrush **mSelectedBrush**
- QFont **mSelectedFont**
- QColor **mSelectedTextColor**

Protected Attributes inherited from [QCPLayoutGrid](#)

- `QList< QList< QCPLayoutElement * > > mElements`
- `QList< double > mColumnStretchFactors`
- `QList< double > mRowStretchFactors`
- `int mColumnSpacing`
- `int mRowSpacing`
- `int mWrap`
- `FillOrder mFillOrder`

Protected Attributes inherited from [QCPLayoutElement](#)

- `QCPLayout * mParentLayout`
- `QSize mMinimumSize`
- `QSize mMaximumSize`
- `SizeConstraintRect mSizeConstraintRect`
- `QRect mRect`
- `QRect mOuterRect`
- `QMargins mMargins`
- `QMargins mMinimumMargins`
- `QCP::MarginSides mAutoMargins`
- `QHash< QCP::MarginSide, QCPMarginGroup * > mMarginGroups`

Protected Attributes inherited from [QCPLayerable](#)

- `bool mVisible`
- `QCustomPlot * mParentPlot`
- `QPointer< QCPLayerable > mParentLayerable`
- `QCPLayer * mLayer`
- `bool mAntialiased`

Friends

- class `QCustomPlot`
- class `QCPAbstractLegendItem`

Additional Inherited Members

Static Protected Member Functions inherited from [QCPLayout](#)

- static `QSize getFinalMinimumOuterSize (const QCPLayoutElement *el)`
- static `QSize getFinalMaximumOuterSize (const QCPLayoutElement *el)`

10.448.1 Detailed Description

Manages a legend inside a [QCustomPlot](#).

A legend is a small box somewhere in the plot which lists plottables with their name and icon.

A legend is populated with legend items by calling [QCPAbstractPlottable::addToLegend](#) on the plottable, for which a legend item shall be created. In the case of the main legend ([QCustomPlot::legend](#)), simply adding plottables to the plot while [QCustomPlot::setAutoAddPlottableToLegend](#) is set to true (the default) creates corresponding legend items. The legend item associated with a certain plottable can be removed with [QCPAbstractPlottable::removeFromLegend](#). However, [QCPLegend](#) also offers an interface to add and manipulate legend items directly: [item](#), [itemWithPlottable](#), [itemCount](#), [addItem](#), [removeItem](#), etc.

Since [QCPLegend](#) derives from [QCPLayoutGrid](#), it can be placed in any position a [QCPLayoutElement](#) may be positioned. The legend items are themselves [QCPLayoutElements](#) which are placed in the grid layout of the legend. [QCPLegend](#) only adds an interface specialized for handling child elements of type [QCPAbstractLegendItem](#), as mentioned above. In principle, any other layout elements may also be added to a legend via the normal [QCPLayoutGrid](#) interface. See the special page about The Layout System for examples on how to add other elements to the legend and move it outside the axis rect.

Use the methods [setFillOrder](#) and [setWrap](#) inherited from [QCPLayoutGrid](#) to control in which order (column first or row first) the legend is filled up when calling [addItem](#), and at which column or row wrapping occurs. The default fill order for legends is [foRowsFirst](#).

By default, every [QCustomPlot](#) has one legend ([QCustomPlot::legend](#)) which is placed in the inset layout of the main axis rect ([QCPAxisRect::insetLayout](#)). To move the legend to another position inside the axis rect, use the methods of the [QCPLayoutInset](#). To move the legend outside of the axis rect, place it anywhere else with the [QCPLayout/QCPLayoutElement](#) interface.

10.448.2 Member Enumeration Documentation

10.448.2.1 SelectablePart

```
enum QCPLegend::SelectablePart
```

Defines the selectable parts of a legend

See also

[setSelectedParts](#), [setSelectableParts](#)

Enumerator

<code>spNone</code>	<code>0x000</code> None
<code>spLegendBox</code>	<code>0x001</code> The legend box (frame)
<code>splItems</code>	<code>0x002</code> Legend items individually (see selectedItems)

10.448.3 Constructor & Destructor Documentation

10.448.3.1 QCPLegend()

```
QCPLegend::QCPLegend ( ) [explicit]
```

Constructs a new [QCPLegend](#) instance with default values.

Note that by default, [QCustomPlot](#) already contains a legend ready to be used as [QCustomPlot::legend](#)

10.448.4 Member Function Documentation

10.448.4.1 addItem()

```
bool QCPLegend::addItem (
    QCPAbstractLegendItem * item )
```

Adds *item* to the legend, if it's not present already. The element is arranged according to the current fill order ([setFillOrder](#)) and wrapping ([setWrap](#)).

Returns true on success, i.e. if the item wasn't in the list already and has been successfully added.

The legend takes ownership of the item.

See also

[removeItem](#), [item](#), [hasItem](#)

10.448.4.2 applyDefaultAntialiasingHint()

```
void QCPLegend::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

10.448.4.3 clearItems()

```
void QCPLegend::clearItems ( )
```

Removes all items from the legend.

10.448.4.4 deselectEvent()

```
void QCPLegend::deselectEvent (
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.448.4.5 draw()

```
void QCPLegend::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

10.448.4.6 hasItem()

```
bool QCPLegend::hasItem (
    QCPAbstractLegendItem * item ) const
```

Returns whether the legend contains *item*.

See also

[hasItemWithPlottable](#)

10.448.4.7 hasItemWithPlottable()

```
bool QCPLegend::hasItemWithPlottable (
    const QCPAbstractPlottable * plottable ) const
```

Returns whether the legend contains a [QCPPlottableLegendItem](#) which is associated with *plottable* (e.g. a [QCPGraph*](#)). If such an item isn't in the legend, returns false.

See also

[itemWithPlottable](#)

10.448.4.8 item()

```
QCPAbstractLegendItem * QCPLegend::item (
    int index ) const
```

Returns the item with index *i*. If non-legend items were added to the legend, and the element at the specified cell index is not a [QCPAbstractLegendItem](#), returns `nullptr`.

Note that the linear index depends on the current fill order ([setFillOrder](#)).

See also

[itemCount](#), [addItem](#), [itemWithPlottable](#)

10.448.4.9 itemCount()

```
int QCPLegend::itemCount ( ) const
```

Returns the number of items currently in the legend. It is identical to the base class [QCPLayoutGrid::elementCount\(\)](#), and unlike the other "item" interface methods of [QCPLegend](#), doesn't only address elements which can be cast to [QCPLegendItem](#).

Note that if empty cells are in the legend (e.g. by calling methods of the [QCPLayoutGrid](#) base class which allows creating empty cells), they are included in the returned count.

See also

[item](#)

10.448.4.10 itemWithPlottable()

```
QCPLegendItem * QCPLegend::itemWithPlottable (
    const QCPLegendItem * plottable ) const
```

Returns the [QCPLegendItem](#) which is associated with *plottable* (e.g. a [QCPLegendItem](#)). If such an item isn't in the legend, returns `nullptr`.

See also

[hasItemWithPlottable](#)

10.448.4.11 parentPlotInitialized()

```
void QCPLegend::parentPlotInitialized (
    QCPLegend * parentPlot ) [protected], [virtual]
```

Reimplemented from [QCPLegendItem](#).

10.448.4.12 removeItem() [1/2]

```
bool QCPLegend::removeItem (
    int index )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes the item with the specified *index* from the legend and deletes it.

After successful removal, the legend is reordered according to the current fill order ([setFillOrder](#)) and wrapping ([setWrap](#)), so no empty cell remains where the removed *item* was. If you don't want this, rather use the raw element interface of [QCPLegendItem](#).

Returns true, if successful. Unlike [QCPLegendItem::removeAt](#), this method only removes elements derived from [QCPLegendItem](#).

See also

[itemCount](#), [clearItems](#)

10.448.4.13 removeItem() [2/2]

```
bool QCPLegend::removeItem (
    QCPAbstractLegendItem * item )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes *item* from the legend and deletes it.

After successful removal, the legend is reordered according to the current fill order ([setFillOrder](#)) and wrapping ([setWrap](#)), so no empty cell remains where the removed *item* was. If you don't want this, rather use the raw element interface of [QCPLayoutGrid](#).

Returns true, if successful.

See also

[clearItems](#)

10.448.4.14 selectedItems()

```
QList< QCPAbstractLegendItem * > QCPLegend::selectedItems ( ) const
```

Returns the legend items that are currently selected. If no items are selected, the list is empty.

See also

[QCPAbstractLegendItem::setSelected](#), [setSelectable](#)

10.448.4.15 selectEvent()

```
void QCPLegend::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.448.4.16 selectionCategory()

```
QCP::Interaction QCPLegend::selectionCategory ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.448.4.17 selectionChanged

```
void QCPLegend::selectionChanged (
    QCPLegend::SelectableParts selection ) [signal]
```

This signal is emitted when the selection state of this legend has changed.

See also

[setSelectedParts](#), [setSelectableParts](#)

10.448.4.18 selectTest()

```
double QCPLegend::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Layout elements are sensitive to events inside their outer rect. If *pos* is within the outer rect, this method returns a value corresponding to 0.99 times the parent plot's selection tolerance. However, layout elements are not selectable by default. So if *onlySelectable* is true, -1.0 is returned.

See [QCPLayerable::selectTest](#) for a general explanation of this virtual method.

[QCPLayoutElement](#) subclasses may reimplement this method to provide more specific selection test behaviour.

Reimplemented from [QCPLayoutElement](#).

10.448.4.19 setBorderPen()

```
void QCPLegend::setBorderPen (
    const QPen & pen )
```

Sets the pen, the border of the entire legend is drawn with.

10.448.4.20 setBrush()

```
void QCPLegend::setBrush (
    const QBrush & brush )
```

Sets the brush of the legend background.

10.448.4.21 setFont()

```
void QCPLegend::setFont (
    const QFont & font )
```

Sets the default font of legend text. Legend items that draw text (e.g. the name of a graph) will use this font by default. However, a different font can be specified on a per-item-basis by accessing the specific legend item.

This function will also set *font* on all already existing legend items.

See also

[QCPLegendItem::setFont](#)

10.448.4.22 setIconBorderPen()

```
void QCPLegend::setIconBorderPen (
    const QPen & pen )
```

Sets the pen used to draw a border around each legend icon. Legend items that draw an icon (e.g. a visual representation of the graph) will use this pen by default.

If no border is wanted, set this to *Qt::NoPen*.

10.448.4.23 setIconSize() [1/2]

```
void QCPLegend::setIconSize (
    const QSize & size )
```

Sets the size of legend icons. Legend items that draw an icon (e.g. a visual representation of the graph) will use this size by default.

10.448.4.24 setIconSize() [2/2]

```
void QCPLegend::setIconSize (
    int width,
    int height )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

10.448.4.25 setIconTextPadding()

```
void QCPLegend::setIconTextPadding (
    int padding )
```

Sets the horizontal space in pixels between the legend icon and the text next to it. Legend items that draw an icon (e.g. a visual representation of the graph) and text (e.g. the name of the graph) will use this space by default.

10.448.4.26 setSelectableParts()

```
void QCPLegend::setSelectableParts (
    const SelectableParts & selectable )
```

Sets whether the user can (de-)select the parts in *selectable* by clicking on the [QCustomPlot](#) surface. (When [QCustomPlot::setInteractions](#) contains [QCP::iSelectLegend](#).)

However, even when *selectable* is set to a value not allowing the selection of a specific part, it is still possible to set the selection of this part manually, by calling [setSelectedParts](#) directly.

See also

[SelectablePart](#), [setSelectedParts](#)

10.448.4.27 setSelectedBorderPen()

```
void QCPLegend::setSelectedBorderPen (
    const QPen & pen )
```

When the legend box is selected, this pen is used to draw the border instead of the normal pen set via [setBorderPen](#).

See also

[setSelectedParts](#), [setSelectableParts](#), [setSelectedBrush](#)

10.448.4.28 setSelectedBrush()

```
void QCPLegend::setSelectedBrush (
    const QBrush & brush )
```

When the legend box is selected, this brush is used to draw the legend background instead of the normal brush set via [setBrush](#).

See also

[setSelectedParts](#), [setSelectableParts](#), [setSelectedBorderPen](#)

10.448.4.29 setSelectedFont()

```
void QCPLegend::setSelectedFont (
    const QFont & font )
```

Sets the default font that is used by legend items when they are selected.

This function will also set *font* on all already existing legend items.

See also

[setFont](#), [QCPAbstractLegendItem::setSelectedFont](#)

10.448.4.30 setSelectedIconBorderPen()

```
void QCPLegend::setSelectedIconBorderPen (
    const QPen & pen )
```

Sets the pen legend items will use to draw their icon borders, when they are selected.

See also

[setSelectedParts](#), [setSelectableParts](#), [setSelectedFont](#)

10.448.4.31 setSelectedParts()

```
void QCPLegend::setSelectedParts (
    const SelectableParts & selected )
```

Sets the selected state of the respective legend parts described by [SelectablePart](#). When a part is selected, it uses a different pen/font and brush. If some legend items are selected and *selected* doesn't contain [splItems](#), those items become deselected.

The entire selection mechanism is handled automatically when [QCustomPlot::setInteractions](#) contains `iSelectLegend`. You only need to call this function when you wish to change the selection state manually.

This function can change the selection state of a part even when [setSelectableParts](#) was set to a value that actually excludes the part.

emits the [selectionChanged](#) signal when *selected* is different from the previous selection state.

Note that it doesn't make sense to set the selected state [splItems](#) here when it wasn't set before, because there's no way to specify which exact items to newly select. Do this by calling [QCPAbstractLegendItem::setSelected](#) directly on the legend item you wish to select.

See also

[SelectablePart](#), [setSelectableParts](#), [selectTest](#), [setSelectedBorderPen](#), [setSelectedIconBorderPen](#), [setSelectedBrush](#), [setSelectedFont](#)

10.448.4.32 setSelectedTextColor()

```
void QCPLegend::setSelectedTextColor (
    const QColor & color )
```

Sets the default text color that is used by legend items when they are selected.

This function will also set *color* on all already existing legend items.

See also

[setTextColor](#), [QCPAbstractLegendItem::setSelectedTextColor](#)

10.448.4.33 setTextColor()

```
void QCPLegend::setTextColor (
    const QColor & color )
```

Sets the default color of legend text. Legend items that draw text (e.g. the name of a graph) will use this color by default. However, a different colors can be specified on a per-item-basis by accessing the specific legend item.

This function will also set *color* on all already existing legend items.

See also

[QCPAbstractLegendItem::setTextColor](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.449 QCPLineEnding Class Reference

Handles the different ending decorations for line-like items.

Public Types

- enum [EndingStyle](#) {
 [esNone](#) , [esFlatArrow](#) , [esSpikeArrow](#) , [esLineArrow](#) ,
 [esDisc](#) , [esSquare](#) , [esDiamond](#) , [esBar](#) ,
 [esHalfBar](#) , [esSkewedBar](#) }

Public Member Functions

- [QCPLineEnding](#) ()
- [QCPLineEnding](#) ([EndingStyle](#) style, double width=8, double length=10, bool inverted=false)
- [EndingStyle](#) **style** () const
- double **width** () const
- double **length** () const
- bool **inverted** () const
- void [setStyle](#) ([EndingStyle](#) style)
- void [setWidth](#) (double width)
- void [setLength](#) (double length)
- void [setInverted](#) (bool inverted)
- double **boundingDistance** () const
- double [realLength](#) () const
- void **draw** ([QCPPainter](#) *painter, const [QCPVector2D](#) &pos, const [QCPVector2D](#) &dir) const
- void **draw** ([QCPPainter](#) *painter, const [QCPVector2D](#) &pos, double angle) const

Protected Attributes

- [EndingStyle](#) **mStyle**
- double **mWidth**
- double **mLength**
- bool **mInverted**

10.449.1 Detailed Description

Handles the different ending decorations for line-like items.

For every ending a line-like item has, an instance of this class exists. For example, [QCPLineEnding](#) has two endings which can be set with [QCPLineEnding::setHead](#) and [QCPLineEnding::setTail](#).

The styles themselves are defined via the enum [QCPLineEnding::EndingStyle](#). Most decorations can be modified regarding width and length, see [setWidth](#) and [setLength](#). The direction of the ending decoration (e.g. direction an arrow is pointing) is controlled by the line-like item. For example, when both endings of a [QCPLineEnding](#) are set to be arrows, they will point to opposite directions, e.g. "outward". This can be changed by [setInverted](#), which would make the respective arrow point inward.

Note that due to the overloaded [QCPLineEnding](#) constructor, you may directly specify a [QCPLineEnding::EndingStyle](#) where actually a [QCPLineEnding](#) is expected, e.g.

10.449.2 Member Enumeration Documentation

10.449.2.1 EndingStyle

enum [QCPLineEnding::EndingStyle](#)

Defines the type of ending decoration for line-like items, e.g. an arrow.

The width and length of these decorations can be controlled with the functions [setWidth](#) and [setLength](#). Some decorations like [esDisc](#), [esSquare](#), [esDiamond](#) and [esBar](#) only support a width, the length property is ignored.

See also

[QCPLItemLine::setHead](#), [QCPLItemLine::setTail](#), [QCPLItemCurve::setHead](#), [QCPLItemCurve::setTail](#), [QCPAxis::setLowerEnding](#), [QCPAxis::setUpperEnding](#)

Enumerator

esNone	No ending decoration.
esFlatArrow	A filled arrow head with a straight/flat back (a triangle)
esSpikeArrow	A filled arrow head with an indented back.
esLineArrow	A non-filled arrow head with open back.
esDisc	A filled circle.
esSquare	A filled square.
esDiamond	A filled diamond (45 degrees rotated square)
esBar	A bar perpendicular to the line.
esHalfBar	A bar perpendicular to the line, pointing out to only one side (to which side can be changed with setInverted)
esSkewedBar	A bar that is skewed (skew controllable via setLength)

10.449.3 Constructor & Destructor Documentation

10.449.3.1 QCPLineEnding() [1/2]

```
QCPLineEnding::QCPLineEnding ( )
```

Creates a [QCPLineEnding](#) instance with default values (style [esNone](#)).

10.449.3.2 QCPLineEnding() [2/2]

```
QCPLineEnding::QCPLineEnding (
    QCPLineEnding::EndingStyle style,
    double width = 8,
    double length = 10,
    bool inverted = false )
```

Creates a [QCPLineEnding](#) instance with the specified values.

10.449.4 Member Function Documentation

10.449.4.1 `realLength()`

```
double QCPLineEnding::realLength ( ) const
```

Starting from the origin of this line ending (which is style specific), returns the length covered by the line ending symbol, in backward direction.

For example, the [esSpikeArrow](#) has a shorter real length than a [esFlatArrow](#), even if both have the same [setLength](#) value, because the spike arrow has an inward curved back, which reduces the length along its center axis (the drawing origin for arrows is at the tip).

This function is used for precise, style specific placement of line endings, for example in QCPAxes.

10.449.4.2 `setInverted()`

```
void QCPLineEnding::setInverted (
    bool inverted )
```

Sets whether the ending decoration shall be inverted. For example, an arrow decoration will point inward when *inverted* is set to true.

Note that also the *width* direction is inverted. For symmetrical ending styles like arrows or discs, this doesn't make a difference. However, asymmetric styles like [esHalfBar](#) are affected by it, which can be used to control to which side the half bar points to.

10.449.4.3 `setLength()`

```
void QCPLineEnding::setLength (
    double length )
```

Sets the length of the ending decoration, if the style supports it. On arrows, for example, the length defines the size in pointing direction.

See also

[setWidth](#)

10.449.4.4 `setStyle()`

```
void QCPLineEnding::setStyle (
    QCPLineEnding::EndingStyle style )
```

Sets the style of the ending decoration.

10.449.4.5 setWidth()

```
void QCPLineEnding::setWidth (
    double width )
```

Sets the width of the ending decoration, if the style supports it. On arrows, for example, the width defines the size perpendicular to the arrow's pointing direction.

See also

[setLength](#)

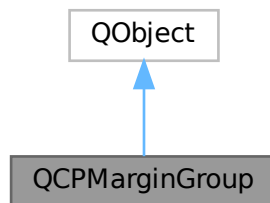
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.450 QCPMarginGroup Class Reference

A margin group allows synchronization of margin sides if working with multiple layout elements.

Inheritance diagram for QCPMarginGroup:



10.450.1 Detailed Description

A margin group allows synchronization of margin sides if working with multiple layout elements.

[QCPMarginGroup](#) allows you to tie a margin side of two or more layout elements together, such that they will all have the same size, based on the largest required margin in the group.

In certain situations it is desirable that margins at specific sides are synchronized across layout elements. For example, if one [QCPAxisRect](#) is below another one in a grid layout, it will provide a cleaner look to the user if the left and right margins of the two axis rects are of the same size. The left axis of the top axis rect will then be at the same horizontal position as the left axis of the lower axis rect, making them appear aligned. The same applies for the right axes. This is what [QCPMarginGroup](#) makes possible.

To add/remove a specific side of a layout element to/from a margin group, use the [QCPLayoutElement::setMarginGroup](#) method. To completely break apart the margin group, either call [clear](#), or just delete the margin group.

10.450.2 Example

First create a margin group:

Then set this group on the layout element sides:

Here, we've used the first two axis rects of the plot and synchronized their left margins with each other and their right margins with each other.

10.450.3 Constructor & Destructor Documentation

10.450.3.1 QCPMarginGroup()

```
QCPMarginGroup::QCPMarginGroup (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a new [QCPMarginGroup](#) instance in *parentPlot*.

10.450.4 Member Function Documentation

10.450.4.1 clear()

```
void QCPMarginGroup::clear ( )
```

Clears this margin group. The synchronization of the margin sides that use this margin group is lifted and they will use their individual margin sizes again.

10.450.4.2 elements()

```
QList< QCPLayoutElement * > QCPMarginGroup::elements (
    QCP::MarginSide side ) const [inline]
```

Returns a list of all layout elements that have their margin *side* associated with this margin group.

10.450.4.3 isEmpty()

```
bool QCPMarginGroup::isEmpty ( ) const
```

Returns whether this margin group is empty. If this function returns true, no layout elements use this margin group to synchronize margin sides.

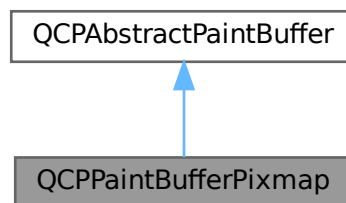
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

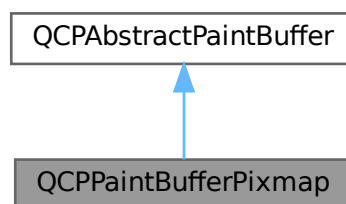
10.451 QCPPaintBufferPixmap Class Reference

A paint buffer based on QPixmap, using software raster rendering.

Inheritance diagram for QCPPaintBufferPixmap:



Collaboration diagram for QCPPaintBufferPixmap:



Public Member Functions

- [QCPPaintBufferPixmap](#) (const QSize &size, double devicePixelRatio)
- virtual [QCPPainter](#) * [startPainting](#) () Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- void [clear](#) (const QColor &color) Q_DECL_OVERRIDE

Public Member Functions inherited from QCPAbstractPaintBuffer

- [QCPAbstractPaintBuffer](#) (const QSize &size, double devicePixelRatio)
- QSize **size** () const
- bool **invalidated** () const
- double **devicePixelRatio** () const
- void [setSize](#) (const QSize &size)
- void [setInvalidated](#) (bool invalidated=true)
- void [setDevicePixelRatio](#) (double ratio)
- virtual void [donePainting](#) ()

Protected Member Functions

- virtual void [reallocateBuffer](#) () Q_DECL_OVERRIDE

Protected Attributes

- QPixmap **mBuffer**

Protected Attributes inherited from QCPAbstractPaintBuffer

- QSize **mSize**
- double **mDevicePixelRatio**
- bool **mInvalidated**

10.451.1 Detailed Description

A paint buffer based on QPixmap, using software raster rendering.

This paint buffer is the default and fall-back paint buffer which uses software rendering and QPixmap as internal buffer. It is used if [QCustomPlot::setOpenGL](#) is false.

10.451.2 Constructor & Destructor Documentation

10.451.2.1 QCPPaintBufferPixmap()

```
QCPPaintBufferPixmap::QCPPaintBufferPixmap (  
    const QSize & size,  
    double devicePixelRatio ) [explicit]
```

Creates a pixmap paint buffer instance with the specified *size* and *devicePixelRatio*, if applicable.

10.451.3 Member Function Documentation

10.451.3.1 clear()

```
void QCPPaintBufferPixmap::clear (
    const QColor & color ) [virtual]
```

Fills the entire buffer with the provided *color*. To have an empty transparent buffer, use the named color `Qt::transparent`.

This method must not be called if there is currently a painter (acquired with [startPainting](#)) active.

Implements [QCPAbstractPaintBuffer](#).

10.451.3.2 draw()

```
void QCPPaintBufferPixmap::draw (
    QCPPainter * painter ) const [virtual]
```

Draws the contents of this buffer with the provided *painter*. This is the method that is used to finally join all paint buffers and draw them onto the screen.

Implements [QCPAbstractPaintBuffer](#).

10.451.3.3 reallocateBuffer()

```
void QCPPaintBufferPixmap::reallocateBuffer ( ) [protected], [virtual]
```

Reallocates the internal buffer with the currently configured size ([setSize](#)) and device pixel ratio, if applicable ([setDevicePixelRatio](#)). It is called as soon as any of those properties are changed on this paint buffer.

Note

Subclasses of [QCPAbstractPaintBuffer](#) must call their reimplementation of this method in their constructor, to perform the first allocation (this can not be done by the base class because calling pure virtual methods in base class constructors is not possible).

Implements [QCPAbstractPaintBuffer](#).

10.451.3.4 startPainting()

```
QCPPainter * QCPPaintBufferPixmap::startPainting ( ) [virtual]
```

Returns a [QCPPainter](#) which is ready to draw to this buffer. The ownership and thus the responsibility to delete the painter after the painting operations are complete is given to the caller of this method.

Once you are done using the painter, delete the painter and call [donePainting](#).

While a painter generated with this method is active, you must not call [setSize](#), [setDevicePixelRatio](#) or [clear](#).

This method may return 0, if a painter couldn't be activated on the buffer. This usually indicates a problem with the respective painting backend.

Implements [QCPAbstractPaintBuffer](#).

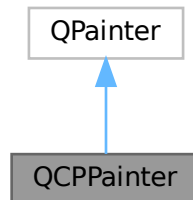
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

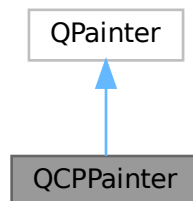
10.452 QCPPainter Class Reference

QPainter subclass used internally.

Inheritance diagram for QCPPainter:



Collaboration diagram for QCPPainter:



Public Types

- enum [PainterMode](#) { [pmDefault](#) = 0x00 , [pmVectorized](#) = 0x01 , [pmNoCaching](#) = 0x02 , [pmNonCosmetic](#) = 0x04 }

Public Member Functions

- [QCPPainter](#) ()
- [QCPPainter](#) (QPaintDevice *device)
- bool **antialiasing** () const
- PainterModes **modes** () const
- void [setAntialiasing](#) (bool enabled)
- void [setMode](#) ([PainterMode](#) mode, bool enabled=true)
- void [setModes](#) (PainterModes modes)
- bool [begin](#) (QPaintDevice *device)
- void [setPen](#) (const QPen &pen)

- void [setPen](#) (const QColor &color)
- void [setPen](#) (Qt::PenStyle penStyle)
- void [drawLine](#) (const QLineF &line)
- void [drawLine](#) (const QPointF &p1, const QPointF &p2)
- void [save](#) ()
- void [restore](#) ()
- void [makeNonCosmetic](#) ()

Protected Attributes

- PainterModes **mModes**
- bool **mlsAntialiasing**
- QStack< bool > **mAntialiasingStack**

10.452.1 Detailed Description

QPainter subclass used internally.

This QPainter subclass is used to provide some extended functionality e.g. for tweaking position consistency between antialiased and non-antialiased painting. Further it provides workarounds for QPainter quirks.

Warning

This class intentionally hides non-virtual functions of QPainter, e.g. [setPen](#), [save](#) and [restore](#). So while it is possible to pass a [QCPPainter](#) instance to a function that expects a QPainter pointer, some of the workarounds and tweaks will be unavailable to the function (because it will call the base class implementations of the functions actually hidden by [QCPPainter](#)).

10.452.2 Member Enumeration Documentation

10.452.2.1 PainterMode

enum [QCPPainter::PainterMode](#)

Defines special modes the painter can operate in. They disable or enable certain subsets of features/fixes/workarounds, depending on whether they are wanted on the respective output device.

Enumerator

<code>pmDefault</code>	<code>0x00</code> Default mode for painting on screen devices
<code>pmVectorized</code>	<code>0x01</code> Mode for vectorized painting (e.g. PDF export). For example, this prevents some antialiasing fixes.
<code>pmNoCaching</code>	<code>0x02</code> Mode for all sorts of exports (e.g. PNG, PDF,...). For example, this prevents using cached pixmap labels
<code>pmNonCosmetic</code>	<code>0x04</code> Turns pen widths 0 to 1, i.e. disables cosmetic pens. (A cosmetic pen is always drawn with width 1 pixel in the vector image/pdf viewer, independent of zoom.)

10.452.3 Constructor & Destructor Documentation

10.452.3.1 QCPPainter() [1/2]

```
QCPPainter::QCPPainter ( )
```

Creates a new [QCPPainter](#) instance and sets default values

10.452.3.2 QCPPainter() [2/2]

```
QCPPainter::QCPPainter (
    QPaintDevice * device ) [explicit]
```

Creates a new [QCPPainter](#) instance on the specified paint *device* and sets default values. Just like the analogous QPainter constructor, begins painting on *device* immediately.

Like [begin](#), this method sets QPainter::NonCosmeticDefaultPen in Qt versions before Qt5.

10.452.4 Member Function Documentation

10.452.4.1 begin()

```
bool QCPPainter::begin (
    QPaintDevice * device )
```

Sets the QPainter::NonCosmeticDefaultPen in Qt versions before Qt5 after beginning painting on *device*. This is necessary to get cosmetic pen consistency across Qt versions, because since Qt5, all pens are non-cosmetic by default, and in Qt4 this render hint must be set to get that behaviour.

The Constructor [QCPPainter\(QPaintDevice *device\)](#) which directly starts painting also sets the render hint as appropriate.

Note

this function hides the non-virtual base class implementation.

10.452.4.2 drawLine()

```
void QCPPainter::drawLine (
    const QLineF & line )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Works around a Qt bug introduced with Qt 4.8 which makes drawing QLineF unpredictable when antialiasing is disabled. Thus when antialiasing is disabled, it rounds the *line* to integer coordinates and then passes it to the original drawLine.

Note

this function hides the non-virtual base class implementation.

10.452.4.3 makeNonCosmetic()

```
void QCPPainter::makeNonCosmetic ( )
```

Changes the pen width to 1 if it currently is 0. This function is called in the [setPen](#) overrides when the [pmNonCosmetic](#) mode is set.

10.452.4.4 restore()

```
void QCPPainter::restore ( )
```

Restores the painter (see [QPainter::restore](#)). Since [QCPPainter](#) adds some new internal state to [QPainter](#), the save/restore functions are reimplemented to also save/restore those members.

Note

this function hides the non-virtual base class implementation.

See also

[save](#)

10.452.4.5 save()

```
void QCPPainter::save ( )
```

Saves the painter (see [QPainter::save](#)). Since [QCPPainter](#) adds some new internal state to [QPainter](#), the save/restore functions are reimplemented to also save/restore those members.

Note

this function hides the non-virtual base class implementation.

See also

[restore](#)

10.452.4.6 setAntialiasing()

```
void QCPPainter::setAntialiasing (
    bool enabled )
```

Sets whether painting uses antialiasing or not. Use this method instead of using [setRenderHint](#) with [QPainter::Antialiasing](#) directly, as it allows [QCPPainter](#) to regain pixel exactness between antialiased and non-antialiased painting (Since Qt < 5.0 uses slightly different coordinate systems for AA/Non-AA painting).

10.452.4.7 setMode()

```
void QCPPainter::setMode (
    QCPPainter::PainterMode mode,
    bool enabled = true )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the mode of the painter. This controls whether the painter shall adjust its fixes/workarounds optimized for certain output devices.

10.452.4.8 setModes()

```
void QCPPainter::setModes (
    PainterModes modes )
```

Sets the mode of the painter. This controls whether the painter shall adjust its fixes/workarounds optimized for certain output devices.

10.452.4.9 setPen() [1/3]

```
void QCPPainter::setPen (
    const QColor & color )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the pen (by color) of the painter and applies certain fixes to it, depending on the mode of this [QCPPainter](#).

Note

this function hides the non-virtual base class implementation.

10.452.4.10 setPen() [2/3]

```
void QCPPainter::setPen (
    const QPen & pen )
```

Sets the pen of the painter and applies certain fixes to it, depending on the mode of this [QCPPainter](#).

Note

this function hides the non-virtual base class implementation.

10.452.4.11 setPen() [3/3]

```
void QCPPainter::setPen (
    Qt::PenStyle penStyle )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the pen (by style) of the painter and applies certain fixes to it, depending on the mode of this [QCPPainter](#).

Note

this function hides the non-virtual base class implementation.

The documentation for this class was generated from the following files:

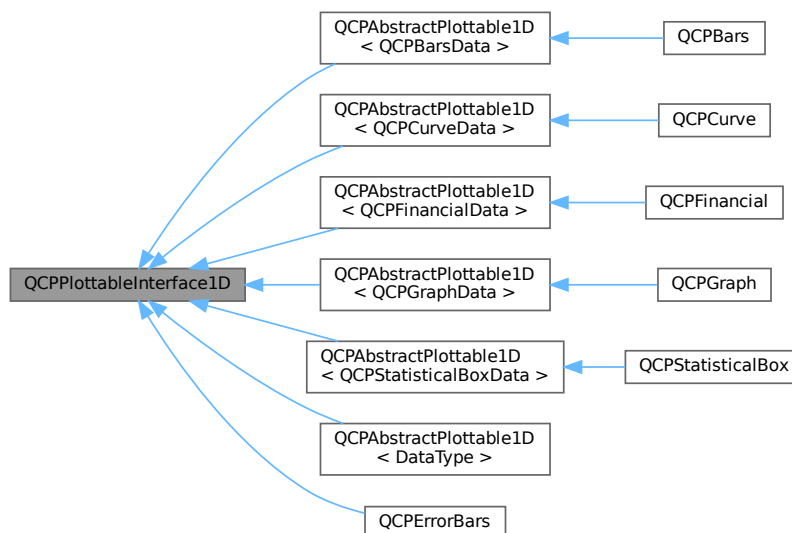
- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.453 QCPPlottableInterface1D Class Reference

Defines an abstract interface for one-dimensional plottables.

```
#include <applications/gui/genesys/qcustomplot.h>
```

Inheritance diagram for QCPPlottableInterface1D:



Public Member Functions

- virtual int [dataCount](#) () const =0
- virtual double [dataMainKey](#) (int index) const =0
- virtual double [dataSortKey](#) (int index) const =0
- virtual double [dataMainValue](#) (int index) const =0
- virtual [QCPRange](#) [dataValueRange](#) (int index) const =0
- virtual [QPointF](#) [dataPixelPosition](#) (int index) const =0
- virtual bool [sortKeysMainKey](#) () const =0
- virtual [QCPDataSelection](#) [selectTestRect](#) (const [QRectF](#) &rect, bool onlySelectable) const =0
- virtual int [findBegin](#) (double sortKey, bool expandedRange=true) const =0
- virtual int [findEnd](#) (double sortKey, bool expandedRange=true) const =0

10.453.1 Detailed Description

Defines an abstract interface for one-dimensional plottables.

This class contains only pure virtual methods which define a common interface to the data of one-dimensional plottables.

For example, it is implemented by the template class [QCPAbstractPlottable1D](#) (the preferred base class for one-dimensional plottables). So if you use that template class as base class of your one-dimensional plottable, you won't have to care about implementing the 1d interface yourself.

If your plottable doesn't derive from [QCPAbstractPlottable1D](#) but still wants to provide a 1d interface (e.g. like [QCPErrorBars](#) does), you should inherit from both [QCPAbstractPlottable](#) and [QCPPlottableInterface1D](#) and accordingly reimplement the pure virtual methods of the 1d interface, matching your data container. Also, reimplement [QCPAbstractPlottable::interface1D](#) to return the `this` pointer.

If you have a [QCPAbstractPlottable](#) pointer, you can check whether it implements this interface by calling [QCPAbstractPlottable::interface1D](#) and testing it for a non-zero return value. If it indeed implements this interface, you may use it to access the plottable's data without needing to know the exact type of the plottable or its data point type.

10.453.2 Member Function Documentation

10.453.2.1 dataCount()

```
int QCPPlottableInterface1D::dataCount ( ) const [pure virtual]
```

Returns the number of data points of the plottable.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#) and [QCPErrorBars](#).

10.453.2.2 dataMainKey()

```
double QCPPlottableInterface1D::dataMainKey (
    int index ) const [pure virtual]
```

Returns the main key of the data point at the given *index*.

What the main key is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#) and [QCPErrorBars](#).

10.453.2.3 dataMainValue()

```
double QCPPlottableInterface1D::dataMainValue (
    int index ) const [pure virtual]
```

Returns the main value of the data point at the given *index*.

What the main value is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#), and [QCPErrorBars](#).

10.453.2.4 dataPixelPosition()

```
QPointF QCPPlottableInterface1D::dataPixelPosition (
    int index ) const [pure virtual]
```

Returns the pixel position on the widget surface at which the data point at the given *index* appears.

Usually this corresponds to the point of dataMainKey/ dataMainValue, in pixel coordinates. However, depending on the plottable, this might be a different apparent position than just a coord-to-pixel transform of those values. For example, [QCPBars](#) apparent data values can be shifted depending on their stacking, bar grouping or configured base value.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#), [QCPBars](#), and [QCPErrorBars](#).

10.453.2.5 dataSortKey()

```
double QCPPlottableInterface1D::dataSortKey (
    int index ) const [pure virtual]
```

Returns the sort key of the data point at the given *index*.

What the sort key is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#), and [QCPErrorBars](#).

10.453.2.6 dataValueRange()

```
QCPRange QCPPlottableInterface1D::dataValueRange (
    int index ) const [pure virtual]
```

Returns the value range of the data point at the given *index*.

What the value range is, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#), and [QCPErrorBars](#).

10.453.2.7 findBegin()

```
int QCPPlottableInterface1D::findBegin (
    double sortKey,
    bool expandedRange = true ) const [pure virtual]
```

Returns the index of the data point with a (sort-)key that is equal to, just below, or just above *sortKey*. If *expandedRange* is true, the data point just below *sortKey* will be considered, otherwise the one just above.

This can be used in conjunction with [findEnd](#) to iterate over data points within a given key range, including or excluding the bounding data points that are just beyond the specified range.

If *expandedRange* is true but there are no data points below *sortKey*, 0 is returned.

If the container is empty, returns 0 (in that case, [findEnd](#) will also return 0, so a loop using these methods will not iterate over the index 0).

See also

[findEnd](#), [QCPDataContainer::findBegin](#)

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#) and [QCPErrorBars](#).

10.453.2.8 findEnd()

```
int QCPPlottableInterface1D::findEnd (
    double sortKey,
    bool expandedRange = true ) const [pure virtual]
```

Returns the index one after the data point with a (sort-)key that is equal to, just above, or just below *sortKey*. If *expandedRange* is true, the data point just above *sortKey* will be considered, otherwise the one just below.

This can be used in conjunction with [findBegin](#) to iterate over data points within a given key range, including the bounding data points that are just below and above the specified range.

If *expandedRange* is true but there are no data points above *sortKey*, the index just above the highest data point is returned.

If the container is empty, returns 0.

See also

[findBegin](#), [QCPDataContainer::findEnd](#)

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalData >](#) and [QCPErrorBars](#).

10.453.2.9 selectTestRect()

```
QCPDataSelection QCPPlottableInterface1D::selectTestRect (
    const QRectF & rect,
    bool onlySelectable ) const [pure virtual]
```

Returns a data selection containing all the data points of this plottable which are contained (or hit by) *rect*. This is used mainly in the selection rect interaction for data selection (data selection mechanism).

If *onlySelectable* is true, an empty [QCPDataSelection](#) is returned if this plottable is not selectable (i.e. if [QCPAbstractPlottable::setSelectable](#) is [QCP::stNone](#)).

Note

rect must be a normalized rect (positive or zero width and height). This is especially important when using the rect of [QCPSelectionRect::accepted](#), which is not necessarily normalized. Use `QRect::normalized()` when passing a rect which might not be normalized.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalBox >](#), [QCPBars](#), [QCPStatisticalBox](#), [QCPFinancial](#), and [QCPErrorBars](#).

10.453.2.10 sortKeysMainKey()

```
bool QCPPlottableInterface1D::sortKeyIsMainKey ( ) const [pure virtual]
```

Returns whether the sort key ([dataSortKey](#)) is identical to the main key ([dataMainKey](#)).

What the sort and main keys are, is defined by the plottable's data type. See the [QCPDataContainer DataType](#) documentation for details about this naming convention.

Implemented in [QCPAbstractPlottable1D< DataType >](#), [QCPAbstractPlottable1D< QCPBarsData >](#), [QCPAbstractPlottable1D< QCPFinancialData >](#), [QCPAbstractPlottable1D< QCPGraphData >](#), [QCPAbstractPlottable1D< QCPStatisticalBox >](#), [QCPBars](#), [QCPStatisticalBox](#), [QCPFinancial](#), and [QCPErrorBars](#).

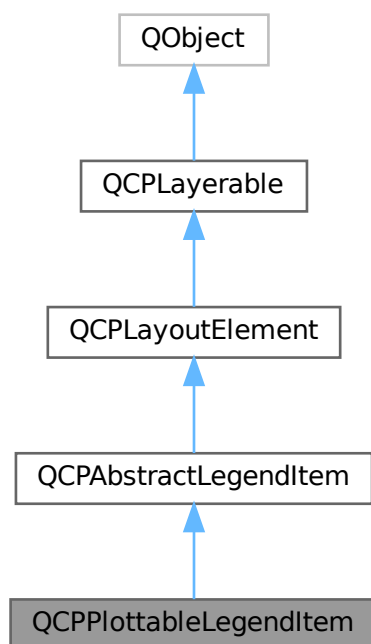
The documentation for this class was generated from the following file:

- applications/gui/genesys/qcustomplot.h

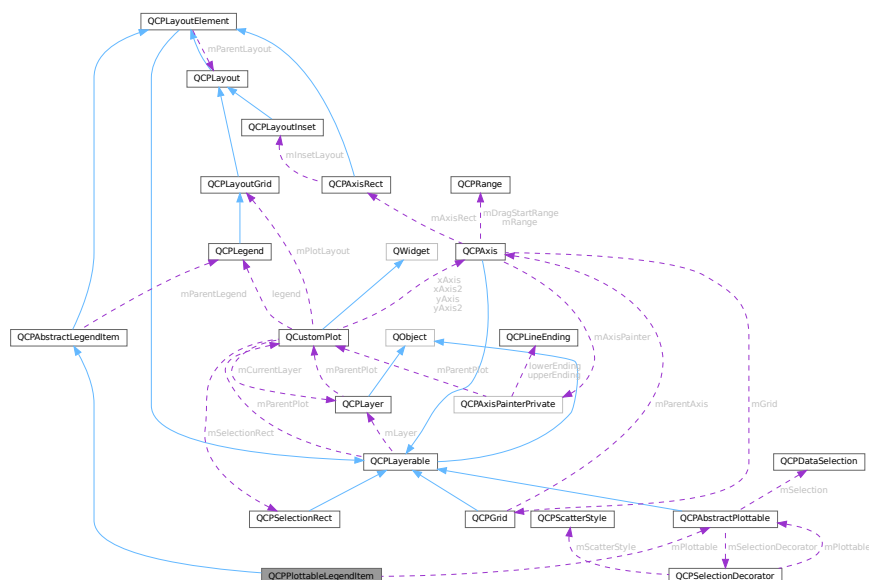
10.454 QCPPlottableLegendItem Class Reference

A legend item representing a plottable with an icon and the plottable name.

Inheritance diagram for QCPPlottableLegendItem:



Collaboration diagram for QCPPlottableLegendItem:



Public Member Functions

- `QCPlotLegendItem` (`QCPlotLegend *parent`, `QCAbstractPlottable *plottable`)
- `QCAbstractPlottable * plottable ()`

Public Member Functions inherited from [QCPAbstractLegendItem](#)

- [QCPAbstractLegendItem](#) ([QCPLegend](#) *parent)
- [QCPLegend](#) * **parentLegend** () const
- QFont **font** () const
- QColor **textColor** () const
- QFont **selectedFont** () const
- QColor **selectedTextColor** () const
- bool **selectable** () const
- bool **selected** () const
- void **setFont** (const QFont &font)
- void **setTextColor** (const QColor &color)
- void **setSelectedFont** (const QFont &font)
- void **setSelectedTextColor** (const QColor &color)
- Q_SLOT void **setSelectable** (bool selectable)
- Q_SLOT void **setSelected** (bool selected)
- virtual double **selectTest** (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_
DECL_OVERRIDE

Public Member Functions inherited from [QCPLayoutElement](#)

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * **layout** () const
- QRect **rect** () const
- QRect **outerRect** () const
- QMargins **margins** () const
- QMargins **minimumMargins** () const
- QCP::MarginSides **autoMargins** () const
- QSize **minimumSize** () const
- QSize **maximumSize** () const
- [SizeConstraintRect](#) **sizeConstraintRect** () const
- [QCPMarginGroup](#) * **marginGroup** ([QCP::MarginSide](#) side) const
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **marginGroups** () const
- void **setOuterRect** (const QRect &rect)
- void **setMargins** (const QMargins &margins)
- void **setMinimumMargins** (const QMargins &margins)
- void **setAutoMargins** ([QCP::MarginSides](#) sides)
- void **setMinimumSize** (const QSize &size)
- void **setMinimumSize** (int width, int height)
- void **setMaximumSize** (const QSize &size)
- void **setMaximumSize** (int width, int height)
- void **setSizeConstraintRect** ([SizeConstraintRect](#) constraintRect)
- void **setMarginGroup** ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)
- virtual void **update** ([UpdatePhase](#) phase)
- virtual QSize **maximumOuterSizeHint** () const
- virtual QList< [QCPLayoutElement](#) * > **elements** (bool recursive) const

Public Member Functions inherited from QCPLayerable

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const [QString](#) &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual [QSize](#) **minimumOuterSizeHint** () const Q_DECL_OVERRIDE
- [QPen](#) **getIconBorderPen** () const
- [QColor](#) **getTextColor** () const
- [QFont](#) **getFont** () const

Protected Member Functions inherited from QCPAbstractLegendItem

- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual [QRect](#) **clipRect** () const Q_DECL_OVERRIDE
- virtual void **selectEvent** ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState↔ Changed) Q_DECL_OVERRIDE
- virtual void **deselectEvent** (bool *selectionStateChanged) Q_DECL_OVERRIDE

Protected Member Functions inherited from QCPLayoutElement

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()
- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from QCPLayerable

- virtual void **mousePressEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **mouseMoveEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseReleaseEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseDoubleClickEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **wheelEvent** ([QWheelEvent](#) *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QCPAbstractPlottable](#) * **mPlottable**

Protected Attributes inherited from [QCPAbstractLegendItem](#)

- [QCPLegend](#) * **mParentLegend**
- QFont **mFont**
- QColor **mTextColor**
- QFont **mSelectedFont**
- QColor **mSelectedTextColor**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Public Types inherited from [QCPLayoutElement](#)

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

Signals inherited from [QCPAbstractLegendItem](#)

- void [selectionChanged](#) (bool selected)
- void **selectableChanged** (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.454.1 Detailed Description

A legend item representing a plottable with an icon and the plottable name.

This is the standard legend item for plottables. It displays an icon of the plottable next to the plottable name. The icon is drawn by the respective plottable itself ([QCPAbstractPlottable::drawLegendIcon](#)), and tries to give an intuitive symbol for the plottable. For example, the [QCPGraph](#) draws a centered horizontal line and/or a single scatter point in the middle.

Legend items of this type are always associated with one plottable (retrievable via the [plottable\(\)](#) function and settable with the constructor). You may change the font of the plottable name with [setFont](#). Icon padding and border pen is taken from the parent [QCPLegend](#), see [QCPLegend::setIconBorderPen](#) and [QCPLegend::setIconTextPadding](#).

The function [QCPAbstractPlottable::addToLegend](#)/ [QCPAbstractPlottable::removeFromLegend](#) creates/removes legend items of this type.

Since [QCPLegend](#) is based on [QCPLayoutGrid](#), a legend item itself is just a subclass of [QCPLayoutElement](#). While it could be added to a legend (or any other layout) via the normal layout interface, [QCPLegend](#) has specialized functions for handling legend items conveniently, see the documentation of [QCPLegend](#).

10.454.2 Constructor & Destructor Documentation

10.454.2.1 QCPPlottableLegendItem()

```
QCPPlottableLegendItem::QCPPlottableLegendItem (
    QCPLegend * parent,
    QCPAbstractPlottable * plottable )
```

Creates a new legend item associated with *plottable*.

Once it's created, it can be added to the legend via [QCPLegend::addItem](#).

A more convenient way of adding/removing a plottable to/from the legend is via the functions [QCPAbstractPlottable::addToLegend](#) and [QCPAbstractPlottable::removeFromLegend](#).

10.454.3 Member Function Documentation

10.454.3.1 draw()

```
void QCPPlottableLegendItem::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractLegendItem](#).

10.454.3.2 `minimumOuterSizeHint()`

```
QSize QCPPlottableLegendItem::minimumOuterSizeHint ( ) const [protected], [virtual]
```

Returns the suggested minimum size this layout element (the [outerRect](#)) may be compressed to, if no manual minimum size is set.

if a minimum size ([setMinimumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through `QCPLayout::getFinalMinimumOuterSize`) to determine the minimum allowed size of this layout element.

A manual minimum size is considered set if it is non-zero.

The default implementation simply returns the sum of the horizontal margins for the width and the sum of the vertical margins for the height. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented from [QCPLayoutElement](#).

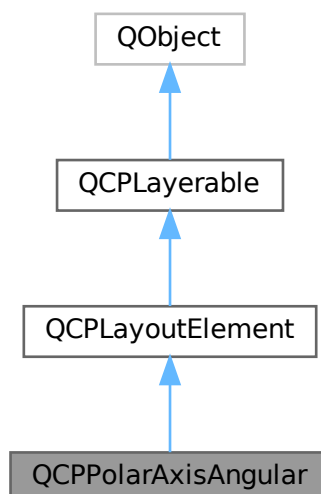
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

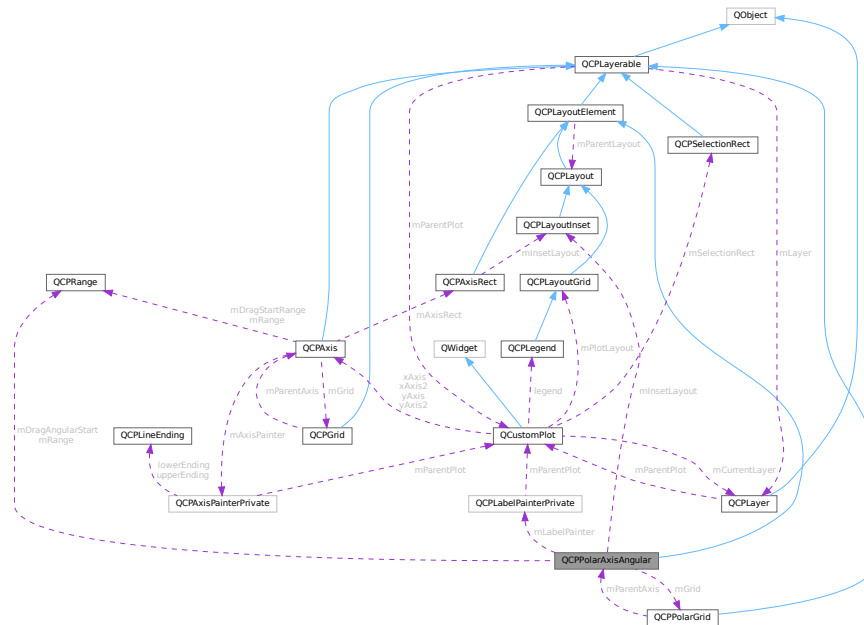
10.455 `QCPPolarAxisAngular` Class Reference

The main container for polar plots, representing the angular axis as a circle.

Inheritance diagram for `QCPPolarAxisAngular`:



Collaboration diagram for QCPolarAxisAngular:



Public Types

- enum `SelectablePart` { `spNone` = 0, `spAxis` = 0x001, `spTickLabels` = 0x002, `spAxisLabel` = 0x004 }
- enum `LabelMode` { `ImUpright`, `ImRotated` }

Public Types inherited from QCPLayoutElement

- enum `UpdatePhase` { `upPreparation` , `upMargins` , `upLayout` }
- enum `SizeConstraintRect` { `scrInnerRect` , `scrOuterRect` }

Signals

- void **rangeChanged** (const [QCPRange](#) &newRange)
- void **rangeChanged** (const [QCPRange](#) &newRange, const [QCPRange](#) &oldRange)
- void **selectionChanged** (const QCPPolarAxisAngular::SelectableParts &parts)
- void **selectableChanged** (const QCPPolarAxisAngular::SelectableParts &parts)

Signals inherited from QCPLayerable

- void layerChanged (QCPLayer *newLayer)

Public Member Functions

- [QCPPolarAxisAngular](#) ([QCustomPlot](#) *parentPlot)
- QPixmap **background** () const
- QBrush **backgroundBrush** () const
- bool **backgroundScaled** () const
- Qt::AspectRatioMode **backgroundScaledMode** () const
- bool **rangeDrag** () const
- bool **rangeZoom** () const
- double **rangeZoomFactor** () const
- const [QCPRange](#) **range** () const
- bool **rangeReversed** () const
- double **angle** () const
- QSharedPointer< [QCPAxisTicker](#) > **ticker** () const
- bool **ticks** () const
- bool **tickLabels** () const
- int **tickLabelPadding** () const
- QFont **tickLabelFont** () const
- QColor **tickLabelColor** () const
- double **tickLabelRotation** () const
- [LabelMode](#) **tickLabelMode** () const
- QString **numberFormat** () const
- int **numberPrecision** () const
- QVector< double > **tickVector** () const
- QVector< QString > **tickVectorLabels** () const
- int **tickLengthIn** () const
- int **tickLengthOut** () const
- bool **subTicks** () const
- int **subTickLengthIn** () const
- int **subTickLengthOut** () const
- QPen **basePen** () const
- QPen **tickPen** () const
- QPen **subTickPen** () const
- QFont **labelFont** () const
- QColor **labelColor** () const
- QString **label** () const
- int **labelPadding** () const
- SelectableParts **selectedParts** () const
- SelectableParts **selectableParts** () const
- QFont **selectedTickLabelFont** () const
- QFont **selectedLabelFont** () const
- QColor **selectedTickLabelColor** () const
- QColor **selectedLabelColor** () const
- QPen **selectedBasePen** () const
- QPen **selectedTickPen** () const
- QPen **selectedSubTickPen** () const
- [QCPPolarGrid](#) * **grid** () const
- void [setBackground](#) (const QPixmap &pm)
- void [setBackground](#) (const QPixmap &pm, bool scaled, Qt::AspectRatioMode mode=Qt::KeepAspectRatio↔ByExpanding)
- void [setBackground](#) (const QBrush &brush)
- void [setBackgroundScaled](#) (bool scaled)
- void [setBackgroundScaledMode](#) (Qt::AspectRatioMode mode)
- void **setRangeDrag** (bool enabled)
- void **setRangeZoom** (bool enabled)

- void **setRangeZoomFactor** (double factor)
- Q_SLOT void **setRange** (const [QCPRange](#) &range)
- void **setRange** (double lower, double upper)
- void **setRange** (double position, double [size](#), Qt::AlignmentFlag alignment)
- void **setRangeLower** (double lower)
- void **setRangeUpper** (double upper)
- void **setRangeReversed** (bool reversed)
- void **setAngle** (double degrees)
- void **setTicker** (QSharedPointer< [QCPAxisTicker](#) > ticker)
- void **setTicks** (bool show)
- void **setTickLabels** (bool show)
- void **setTickLabelPadding** (int padding)
- void **setTickLabelFont** (const QFont &font)
- void **setTickLabelColor** (const QColor &color)
- void **setTickLabelRotation** (double degrees)
- void **setTickLabelMode** ([LabelMode](#) mode)
- void **setNumberFormat** (const QString &formatCode)
- void **setNumberPrecision** (int precision)
- void **setTickLength** (int inside, int outside=0)
- void **setTickLengthIn** (int inside)
- void **setTickLengthOut** (int outside)
- void **setSubTicks** (bool show)
- void **setSubTickLength** (int inside, int outside=0)
- void **setSubTickLengthIn** (int inside)
- void **setSubTickLengthOut** (int outside)
- void **setBasePen** (const QPen &pen)
- void **setTickPen** (const QPen &pen)
- void **setSubTickPen** (const QPen &pen)
- void **setLabelFont** (const QFont &font)
- void **setLabelColor** (const QColor &color)
- void **setLabel** (const QString &str)
- void **setLabelPadding** (int padding)
- void **setLabelPosition** (Qt::AlignmentFlag position)
- void **setSelectedTickLabelFont** (const QFont &font)
- void **setSelectedLabelFont** (const QFont &font)
- void **setSelectedTickLabelColor** (const QColor &color)
- void **setSelectedLabelColor** (const QColor &color)
- void **setSelectedBasePen** (const QPen &pen)
- void **setSelectedTickPen** (const QPen &pen)
- void **setSelectedSubTickPen** (const QPen &pen)
- Q_SLOT void **setSelectableParts** (const QCPPolarAxisAngular::SelectableParts &selectableParts)
- Q_SLOT void **setSelectedParts** (const QCPPolarAxisAngular::SelectableParts &selectedParts)
- virtual double **selectTest** (const QPointF &pos, bool onlySelectable, QVariant *details=0) const Q_DECL_OVERRIDE
- virtual void **update** ([UpdatePhase](#) phase) Q_DECL_OVERRIDE
- virtual QList< [QCPLayoutElement](#) * > **elements** (bool recursive) const Q_DECL_OVERRIDE
- bool **removeGraph** ([QCPPolarGraph](#) *graph)
- int **radialAxisCount** () const
- [QCPPolarAxisRadial](#) * **radialAxis** (int index=0) const
- QList< [QCPPolarAxisRadial](#) * > **radialAxes** () const
- [QCPPolarAxisRadial](#) * **addRadialAxis** ([QCPPolarAxisRadial](#) *axis=0)
- bool **removeRadialAxis** ([QCPPolarAxisRadial](#) *axis)
- [QCPLayoutInset](#) * **insetLayout** () const
- QRegion **exactClipRegion** () const
- void **moveRange** (double diff)

- void [scaleRange](#) (double factor)
- void [scaleRange](#) (double factor, double [center](#))
- void [rescale](#) (bool onlyVisiblePlottables=false)
- double [coordToAngleRad](#) (double coord) const
- double [angleRadToCoord](#) (double angleRad) const
- void [pixelToCoord](#) (QPointF pixelPos, double &angleCoord, double &radiusCoord) const
- QPointF [coordToPixel](#) (double angleCoord, double radiusCoord) const
- [SelectablePart](#) [getPartAt](#) (const QPointF &pos) const
- int [left](#) () const
- int [right](#) () const
- int [top](#) () const
- int [bottom](#) () const
- int [width](#) () const
- int [height](#) () const
- QSize [size](#) () const
- QPoint [topLeft](#) () const
- QPoint [topRight](#) () const
- QPoint [bottomLeft](#) () const
- QPoint [bottomRight](#) () const
- QPointF [center](#) () const
- double [radius](#) () const

Public Member Functions inherited from [QCPLayoutElement](#)

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- QRect [rect](#) () const
- QRect [outerRect](#) () const
- QMargins [margins](#) () const
- QMargins [minimumMargins](#) () const
- QCP::MarginSides [autoMargins](#) () const
- QSize [minimumSize](#) () const
- QSize [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const QRect &rect)
- void [setMargins](#) (const QMargins &margins)
- void [setMinimumMargins](#) (const QMargins &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)
- void [setMinimumSize](#) (const QSize &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const QSize &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)
- virtual QSize [minimumOuterSizeHint](#) () const
- virtual QSize [maximumOuterSizeHint](#) () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const [QString](#) &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void **draw** ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) **selectionCategory** () const Q_DECL_OVERRIDE
- virtual void **mousePressEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details) Q_DECL_OVERRIDE
- virtual void **mouseMoveEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos) Q_DECL_OVERRIDE
- virtual void **mouseReleaseEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos) Q_DECL_OVERRIDE
- virtual void **wheelEvent** ([QWheelEvent](#) *event) Q_DECL_OVERRIDE
- bool **registerPolarGraph** ([QCPPolarGraph](#) *graph)
- void **drawBackground** ([QCPPainter](#) *painter, const [QPointF](#) ¢er, double radius)
- void **setupTickVectors** ()
- [QPen](#) **getBasePen** () const
- [QPen](#) **getTickPen** () const
- [QPen](#) **getSubTickPen** () const
- [QFont](#) **getTickLabelFont** () const
- [QFont](#) **getLabelFont** () const
- [QColor](#) **getTickLabelColor** () const
- [QColor](#) **getLabelColor** () const

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()
- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QRect](#) **clipRect** () const
- virtual void **selectEvent** ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState↔ Changed)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mouseDoubleClickEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QBrush **mBackgroundBrush**
- QPixmap **mBackgroundPixmap**
- QPixmap **mScaledBackgroundPixmap**
- bool **mBackgroundScaled**
- Qt::AspectRatioMode **mBackgroundScaledMode**
- [QCPLayoutInset](#) * **mInsetLayout**
- bool **mRangeDrag**
- bool **mRangeZoom**
- double **mRangeZoomFactor**
- double **mAngle**
- double **mAngleRad**
- SelectableParts **mSelectableParts**
- SelectableParts **mSelectedParts**
- QPen **mBasePen**
- QPen **mSelectedBasePen**
- int **mLabelPadding**
- QString **mLabel**
- QFont **mLabelFont**
- QFont **mSelectedLabelFont**
- QColor **mLabelColor**
- QColor **mSelectedLabelColor**
- bool **mTickLabels**
- QFont **mTickLabelFont**
- QFont **mSelectedTickLabelFont**
- QColor **mTickLabelColor**
- QColor **mSelectedTickLabelColor**
- int **mNumberPrecision**
- QLatin1Char **mNumberFormatChar**
- bool **mNumberBeautifulPowers**
- bool **mNumberMultiplyCross**
- bool **mTicks**
- bool **mSubTicks**
- int **mTickLengthIn**
- int **mTickLengthOut**
- int **mSubTickLengthIn**
- int **mSubTickLengthOut**
- QPen **mTickPen**
- QPen **mSelectedTickPen**
- QPen **mSubTickPen**
- QPen **mSelectedSubTickPen**
- [QCPRange](#) **mRange**
- bool **mRangeReversed**
- QPointF **mCenter**
- double **mRadius**
- QList< [QCPPolarAxisRadial](#) * > **mRadialAxes**
- [QCPPolarGrid](#) * **mGrid**
- QList< [QCPPolarGraph](#) * > **mGraphs**
- QSharedPointer< [QCPAxisTicker](#) > **mTicker**
- QVector< double > **mTickVector**
- QVector< QString > **mTickVectorLabels**
- QVector< QPointF > **mTickVectorCosSin**
- QVector< double > **mSubTickVector**
- QVector< QPointF > **mSubTickVectorCosSin**

- bool **mDragging**
- [QCPRange](#) **mDragAngularStart**
- QList< [QCPRange](#) > **mDragRadialStart**
- QCP::AntialiasedElements **mAADragBackup**
- QCP::AntialiasedElements **mNotAADragBackup**
- [QCPLabelPainterPrivate](#) **mLabelPainter**

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPPolarGrid](#)
- class [QCPPolarGraph](#)

10.455.1 Detailed Description

The main container for polar plots, representing the angular axis as a circle.

Warning

In this [QCustomPlot](#) version, polar plots are a tech preview. Expect documentation and functionality to be incomplete, as well as changing public interfaces in the future.

10.455.2 Member Enumeration Documentation

10.455.2.1 LabelMode

```
enum QCPPolarAxisAngular::LabelMode
```

TODO

10.455.2.2 SelectablePart

```
enum QCPPolarAxisAngular::SelectablePart
```

Defines the selectable parts of an axis.

See also

[setSelectableParts](#), [setSelectedParts](#)

Enumerator

spNone	None of the selectable parts.
spAxis	The axis backbone and tick marks.
spTickLabels	Tick labels (numbers) of this axis (as a whole, not individually)
spAxisLabel	The axis label.

10.455.3 Constructor & Destructor Documentation

10.455.3.1 QCPPolarAxisAngular()

```
QCPPolarAxisAngular::QCPPolarAxisAngular (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a QCPPolarAxis instance and sets default values. An axis is added for each of the four sides, the top and right axes are set invisible initially.

10.455.4 Member Function Documentation

10.455.4.1 addRadialAxis()

```
QCPPolarAxisRadial * QCPPolarAxisAngular::addRadialAxis (
    QCPPolarAxisRadial * axis = 0 )
```

Adds a new axis to the axis rect side specified with *type*, and returns it. If *axis* is 0, a new [QCPAxis](#) instance is created internally. [QCustomPlot](#) owns the returned axis, so if you want to remove an axis, use [removeAxis](#) instead of deleting it manually.

You may inject [QCPAxis](#) instances (or subclasses of [QCPAxis](#)) by setting *axis* to an axis that was previously created outside [QCustomPlot](#). It is important to note that [QCustomPlot](#) takes ownership of the axis, so you may not delete it afterwards. Further, the *axis* must have been created with this axis rect as parent and with the same axis type as specified in *type*. If this is not the case, a debug output is generated, the axis is not added, and the method returns 0.

This method can not be used to move *axis* between axis rects. The same *axis* instance must not be added multiple times to the same or different axis rects.

If an axis rect side already contains one or more axes, the lower and upper endings of the new axis ([QCPAxis::setLowerEnding](#), [QCPAxis::setUpperEnding](#)) are set to [QCPLineEnding::esHalfBar](#).

See also

[addAxes](#), [setupFullAxesBox](#)

10.455.4.2 applyDefaultAntialiasingHint()

```
void QCPPolarAxisAngular::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

10.455.4.3 bottom()

```
int QCPPolarAxisAngular::bottom ( ) const [inline]
```

Returns the pixel position of the bottom border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.455.4.4 bottomLeft()

```
QPoint QCPPolarAxisAngular::bottomLeft ( ) const [inline]
```

Returns the bottom left corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.455.4.5 bottomRight()

```
QPoint QCPPolarAxisAngular::bottomRight ( ) const [inline]
```

Returns the bottom right corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.455.4.6 center()

```
QPoint QCPPolarAxisAngular::center ( ) const [inline]
```

Returns the center of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.455.4.7 coordToPixel()

```
QPointF QCPPolarAxisAngular::coordToPixel (
    double angleCoord,
    double radiusCoord ) const
```

Transforms *value*, in coordinates of the axis, to pixel coordinates of the [QCustomPlot](#) widget.

10.455.4.8 draw()

```
void QCPPolarAxisAngular::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

10.455.4.9 elements()

```
QList< QCPLLayoutElement * > QCPPolarAxisAngular::elements (
    bool recursive ) const [virtual]
```

Returns a list of all child elements in this layout element. If *recursive* is true, all sub-child elements are included in the list, too.

Warning

There may be `nullptr` entries in the returned list. For example, [QCPLLayoutGrid](#) may have empty cells which yield `nullptr` at the respective index.

Reimplemented from [QCPLLayoutElement](#).

10.455.4.10 getPartAt()

```
QCPPolarAxisAngular::SelectablePart QCPPolarAxisAngular::getPartAt (
    const QPointF & pos ) const
```

Returns the part of the axis that is hit by *pos* (in pixels). The return value of this function is independent of the user-selectable parts defined with [setSelectableParts](#). Further, this function does not change the current selection state of the axis.

If the axis is not visible ([setVisible](#)), this function always returns [spNone](#).

See also

[setSelectedParts](#), [setSelectableParts](#), [QCustomPlot::setInteractions](#)

10.455.4.11 height()

```
int QCPPolarAxisAngular::height ( ) const [inline]
```

Returns the pixel height of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.455.4.12 insetLayout()

```
QCPLLayoutInset * QCPPolarAxisAngular::insetLayout ( ) const [inline]
```

Returns the inset layout of this axis rect. It can be used to place other layout elements (or even layouts with multiple other elements) inside/on top of an axis rect.

See also

[QCPLLayoutInset](#)

10.455.4.13 left()

```
int QCPPolarAxisAngular::left ( ) const [inline]
```

Returns the pixel position of the left border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.455.4.14 mouseMoveEvent()

```
void QCPPolarAxisAngular::mouseMoveEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user moves the mouse while holding a mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

10.455.4.15 mousePressEvent()

```
void QCPPolarAxisAngular::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses a mouse button while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *details* contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the *details* parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

[QCustomPlot](#) uses an event propagation system that works the same as Qt's system. If your layerable doesn't reimplement the [mousePressEvent](#) or explicitly calls `event->ignore()` in its reimplementation, the event will be propagated to the next layerable in the stacking order.

Once a layerable has accepted the [mousePressEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) or [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

10.455.4.16 mouseReleaseEvent()

```
void QCPPolarAxisAngular::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user releases the mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

10.455.4.17 moveRange()

```
void QCPPolarAxisAngular::moveRange (
    double diff )
```

If the scale type (`setScaleType()`) is `stLinear`, *diff* is added to the lower and upper bounds of the range. The range is simply moved by *diff*.

If the scale type is `stLogarithmic`, the range bounds are multiplied by *diff*. This corresponds to an apparent "linear" move in logarithmic scaling by a distance of $\log(\text{diff})$.

10.455.4.18 pixelToCoord()

```
void QCPPolarAxisAngular::pixelToCoord (
    QPointF pixelPos,
    double & angleCoord,
    double & radiusCoord ) const
```

Transforms *value*, in pixel coordinates of the [QCustomPlot](#) widget, to axis coordinates.

10.455.4.19 radialAxes()

```
QList< QCPPolarAxisRadial * > QCPPolarAxisAngular::radialAxes ( ) const
```

Returns all axes on the axis rect sides specified with *types*.

types may be a single [QCPAxis::AxisType](#) or an `or`-combination, to get the axes of multiple sides.

See also

[axis](#)

10.455.4.20 radialAxis()

```
QCPPolarAxisRadial * QCPPolarAxisAngular::radialAxis (
    int index = 0 ) const
```

Returns the axis with the given *index* on the axis rect side specified with *type*.

See also

axisCount, axes

10.455.4.21 radialAxisCount()

```
int QCPPolarAxisAngular::radialAxisCount ( ) const
```

Returns the number of axes on the axis rect side specified with *type*.

See also

axis

10.455.4.22 removeRadialAxis()

```
bool QCPPolarAxisAngular::removeRadialAxis (
    QCPPolarAxisRadial * radialAxis )
```

Removes the specified *axis* from the axis rect and deletes it.

Returns true on success, i.e. if *axis* was a valid axis in this axis rect.

See also

addAxis

10.455.4.23 rescale()

```
void QCPPolarAxisAngular::rescale (
    bool onlyVisiblePlottables = false )
```

Changes the axis range such that all plottables associated with this axis are fully visible in that dimension.

See also

[QCPAbstractPlottable::rescaleAxes](#), [QCustomPlot::rescaleAxes](#)

10.455.4.24 right()

```
int QCPPolarAxisAngular::right ( ) const [inline]
```

Returns the pixel position of the right border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.455.4.25 scaleRange() [1/2]

```
void QCPPolarAxisAngular::scaleRange (
    double factor )
```

Scales the range of this axis by *factor* around the center of the current axis range. For example, if *factor* is 2.0, then the axis range will double its size, and the point at the axis range center won't have changed its position in the [QCustomPlot](#) widget (i.e. coordinates around the center will have moved symmetrically closer).

If you wish to scale around a different coordinate than the current axis range center, use the overload [scaleRange\(double factor, double center\)](#).

10.455.4.26 scaleRange() [2/2]

```
void QCPPolarAxisAngular::scaleRange (
    double factor,
    double center )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Scales the range of this axis by *factor* around the coordinate *center*. For example, if *factor* is 2.0, *center* is 1.0, then the axis range will double its size, and the point at coordinate 1.0 won't have changed its position in the [QCustomPlot](#) widget (i.e. coordinates around 1.0 will have moved symmetrically closer to 1.0).

See also

[scaleRange\(double factor\)](#)

10.455.4.27 selectionCategory()

```
QCP::Interaction QCPPolarAxisAngular::selectionCategory ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.455.4.28 selectTest()

```
double QCPPolarAxisAngular::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = 0 ) const [virtual]
```

Layout elements are sensitive to events inside their outer rect. If *pos* is within the outer rect, this method returns a value corresponding to 0.99 times the parent plot's selection tolerance. However, layout elements are not selectable by default. So if *onlySelectable* is true, -1.0 is returned.

See [QCPLayerable::selectTest](#) for a general explanation of this virtual method.

[QCPLayoutElement](#) subclasses may reimplement this method to provide more specific selection test behaviour.

Reimplemented from [QCPLayoutElement](#).

10.455.4.29 setBackground() [1/3]

```
void QCPPolarAxisAngular::setBackground (
    const QBrush & brush )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets *brush* as the background brush. The axis rect background will be filled with this brush. Since axis rects place themselves on the "background" layer by default, the axis rect backgrounds are usually drawn below everything else.

The brush will be drawn before (under) any background pixmap, which may be specified with [setBackground\(const QPixmap &pm\)](#).

To disable drawing of a background brush, set *brush* to Qt::NoBrush.

See also

[setBackground\(const QPixmap &pm\)](#)

10.455.4.30 setBackground() [2/3]

```
void QCPPolarAxisAngular::setBackground (
    const QPixmap & pm )
```

Sets *pm* as the axis background pixmap. The axis background pixmap will be drawn inside the axis rect. Since axis rects place themselves on the "background" layer by default, the axis rect backgrounds are usually drawn below everything else.

For cases where the provided pixmap doesn't have the same size as the axis rect, scaling can be enabled with [setBackgroundScaled](#) and the scaling mode (i.e. whether and how the aspect ratio is preserved) can be set with [setBackgroundScaledMode](#). To set all these options in one call, consider using the overloaded version of this function.

Below the pixmap, the axis rect may be optionally filled with a brush, if specified with [setBackground\(const QBrush &brush\)](#).

See also

[setBackgroundScaled](#), [setBackgroundScaledMode](#), [setBackground\(const QBrush &brush\)](#)

10.455.4.31 setBackground() [3/3]

```
void QCPPolarAxisAngular::setBackground (
    const QPixmap & pm,
    bool scaled,
    Qt::AspectRatioMode mode = Qt::KeepAspectRatioByExpanding )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Allows setting the background pixmap of the axis rect, whether it shall be scaled and how it shall be scaled in one call.

See also

[setBackground\(const QPixmap &pm\)](#), [setBackgroundScaled](#), [setBackgroundScaledMode](#)

10.455.4.32 setBackgroundScaled()

```
void QCPPolarAxisAngular::setBackgroundScaled (
    bool scaled )
```

Sets whether the axis background pixmap shall be scaled to fit the axis rect or not. If *scaled* is set to true, you may control whether and how the aspect ratio of the original pixmap is preserved with [setBackgroundScaledMode](#).

Note that the scaled version of the original pixmap is buffered, so there is no performance penalty on replots. (Except when the axis rect dimensions are changed continuously.)

See also

[setBackground](#), [setBackgroundScaledMode](#)

10.455.4.33 setBackgroundScaledMode()

```
void QCPPolarAxisAngular::setBackgroundScaledMode (
    Qt::AspectRatioMode mode )
```

If scaling of the axis background pixmap is enabled ([setBackgroundScaled](#)), use this function to define whether and how the aspect ratio of the original pixmap passed to [setBackground](#) is preserved.

See also

[setBackground](#), [setBackgroundScaled](#)

10.455.4.34 setBasePen()

```
void QCPPolarAxisAngular::setBasePen (
    const QPen & pen )
```

Sets the pen, the axis base line is drawn with.

See also

[setTickPen](#), [setSubTickPen](#)

10.455.4.35 setLabel()

```
void QCPPolarAxisAngular::setLabel (
    const QString & str )
```

Sets the text of the axis label that will be shown below/above or next to the axis, depending on its orientation. To disable axis labels, pass an empty string as *str*.

10.455.4.36 `setLabelColor()`

```
void QCPPolarAxisAngular::setLabelColor (
    const QColor & color )
```

Sets the color of the axis label.

See also

[setLabelFont](#)

10.455.4.37 `setLabelFont()`

```
void QCPPolarAxisAngular::setLabelFont (
    const QFont & font )
```

Sets the font of the axis label.

See also

[setLabelColor](#)

10.455.4.38 `setLabelPadding()`

```
void QCPPolarAxisAngular::setLabelPadding (
    int padding )
```

Sets the distance between the tick labels and the axis label.

See also

[setTickLabelPadding](#), [setPadding](#)

10.455.4.39 `setNumberFormat()`

```
void QCPPolarAxisAngular::setNumberFormat (
    const QString & formatCode )
```

Sets the number format for the numbers in tick labels. This *formatCode* is an extended version of the format code used e.g. by `QString::number()` and `QLocale::toString()`. For reference about that, see the "Argument Formats" section in the detailed description of the `QString` class.

formatCode is a string of one, two or three characters. The first character is identical to the normal format code used by Qt. In short, this means: 'e'/'E' scientific format, 'f' fixed format, 'g'/'G' scientific or fixed, whichever is shorter.

The second and third characters are optional and specific to [QCustomPlot](#):

If the first char was 'e' or 'g', numbers are/might be displayed in the scientific format, e.g. "5.5e9", which might be visually unappealing in a plot. So when the second char of *formatCode* is set to 'b' (for "beautiful"), those exponential numbers are formatted in a more natural way, i.e. "5.5 [multiplication sign] 10 [superscript] 9". By default, the multiplication sign is a centered dot. If instead a cross should be shown (as is usual in the USA), the third char of *formatCode* can be set to 'c'. The inserted multiplication signs are the UTF-8 characters 215 (0xD7) for the cross and 183 (0xB7) for the dot.

Examples for *formatCode*:

- `g` normal format code behaviour. If number is small, fixed format is used, if number is large, normal scientific format is used
- `gb` If number is small, fixed format is used, if number is large, scientific format is used with beautifully typeset decimal powers and a dot as multiplication sign
- `ebc` All numbers are in scientific format with beautifully typeset decimal power and a cross as multiplication sign
- `fb` illegal format code, since fixed format doesn't support (or need) beautifully typeset decimal powers. Format code will be reduced to 'f'.
- `hello` illegal format code, since first char is not 'e', 'E', 'f', 'g' or 'G'. Current format code will not be changed.

10.455.4.40 `setNumberPrecision()`

```
void QCPPolarAxisAngular::setNumberPrecision (
    int precision )
```

Sets the precision of the tick label numbers. See `QLocale::toString(double i, char f, int prec)` for details. The effect of precisions are most notably for number Formats starting with 'e', see [setNumberFormat](#)

10.455.4.41 `setRange()` [1/3]

```
void QCPPolarAxisAngular::setRange (
    const QCPRange & range )
```

Sets the range of the axis.

This slot may be connected with the `rangeChanged` signal of another axis so this axis is always synchronized with the other axis range, when it changes.

To invert the direction of an axis, use [setRangeReversed](#).

10.455.4.42 `setRange()` [2/3]

```
void QCPPolarAxisAngular::setRange (
    double lower,
    double upper )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the lower and upper bound of the axis range.

To invert the direction of an axis, use [setRangeReversed](#).

There is also a slot to set a range, see [setRange\(const QCPRange &range\)](#).

10.455.4.43 setRange() [3/3]

```
void QCPPolarAxisAngular::setRange (
    double position,
    double size,
    Qt::AlignmentFlag alignment )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the range of the axis.

The *position* coordinate indicates together with the *alignment* parameter, where the new range will be positioned. *size* defines the size of the new axis range. *alignment* may be Qt::AlignLeft, Qt::AlignRight or Qt::AlignCenter. This will cause the left border, right border, or center of the range to be aligned with *position*. Any other values of *alignment* will default to Qt::AlignCenter.

10.455.4.44 setRangeLower()

```
void QCPPolarAxisAngular::setRangeLower (
    double lower )
```

Sets the lower bound of the axis range. The upper bound is not changed.

See also

[setRange](#)

10.455.4.45 setRangeReversed()

```
void QCPPolarAxisAngular::setRangeReversed (
    bool reversed )
```

Sets whether the axis range (direction) is displayed reversed. Normally, the values on horizontal axes increase left to right, on vertical axes bottom to top. When *reversed* is set to true, the direction of increasing values is inverted.

Note that the range and data interface stays the same for reversed axes, e.g. the *lower* part of the [setRange](#) interface will still reference the mathematically smaller number than the *upper* part.

10.455.4.46 setRangeUpper()

```
void QCPPolarAxisAngular::setRangeUpper (
    double upper )
```

Sets the upper bound of the axis range. The lower bound is not changed.

See also

[setRange](#)

10.455.4.47 `setSelectableParts()`

```
void QCPPolarAxisAngular::setSelectableParts (
    const QCPPolarAxisAngular::SelectableParts & selectableParts )
```

Sets whether the user can (de-)select the parts in *selectable* by clicking on the [QCustomPlot](#) surface. (When [QCustomPlot::setInteractions](#) contains `iSelectAxes`.)

However, even when *selectable* is set to a value not allowing the selection of a specific part, it is still possible to set the selection of this part manually, by calling [setSelectedParts](#) directly.

See also

[SelectablePart](#), [setSelectedParts](#)

10.455.4.48 `setSelectedBasePen()`

```
void QCPPolarAxisAngular::setSelectedBasePen (
    const QPen & pen )
```

Sets the pen that is used to draw the axis base line when selected.

See also

[setBasePen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.455.4.49 `setSelectedLabelColor()`

```
void QCPPolarAxisAngular::setSelectedLabelColor (
    const QColor & color )
```

Sets the color that is used for the axis label when it is selected.

See also

[setLabelColor](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.455.4.50 `setSelectedLabelFont()`

```
void QCPPolarAxisAngular::setSelectedLabelFont (
    const QFont & font )
```

Sets the font that is used for the axis label when it is selected.

See also

[setLabelFont](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.455.4.51 setSelectedParts()

```
void QCPPolarAxisAngular::setSelectedParts (
    const QCPPolarAxisAngular::SelectableParts & selectedParts )
```

Sets the selected state of the respective axis parts described by [SelectablePart](#). When a part is selected, it uses a different pen/font.

The entire selection mechanism for axes is handled automatically when [QCustomPlot::setInteractions](#) contains [iSelectAxes](#). You only need to call this function when you wish to change the selection state manually.

This function can change the selection state of a part, independent of the [setSelectableParts](#) setting.

emits the `selectionChanged` signal when *selected* is different from the previous selection state.

See also

[SelectablePart](#), [setSelectableParts](#), [selectTest](#), [setSelectedBasePen](#), [setSelectedTickPen](#), [setSelectedSubTickPen](#), [setSelectedTickLabelFont](#), [setSelectedLabelFont](#), [setSelectedTickLabelColor](#), [setSelectedLabelColor](#)

10.455.4.52 setSelectedSubTickPen()

```
void QCPPolarAxisAngular::setSelectedSubTickPen (
    const QPen & pen )
```

Sets the pen that is used to draw the subticks when selected.

See also

[setSubTickPen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.455.4.53 setSelectedTickLabelColor()

```
void QCPPolarAxisAngular::setSelectedTickLabelColor (
    const QColor & color )
```

Sets the color that is used for tick labels when they are selected.

See also

[setTickLabelColor](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.455.4.54 setSelectedTickLabelFont()

```
void QCPPolarAxisAngular::setSelectedTickLabelFont (
    const QFont & font )
```

Sets the font that is used for tick labels when they are selected.

See also

[setTickLabelFont](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.455.4.55 setSelectedTickPen()

```
void QCPPolarAxisAngular::setSelectedTickPen (
    const QPen & pen )
```

Sets the pen that is used to draw the (major) ticks when selected.

See also

[setTickPen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.455.4.56 setSubTickLength()

```
void QCPPolarAxisAngular::setSubTickLength (
    int inside,
    int outside = 0 )
```

Sets the length of the subticks in pixels. *inside* is the length the subticks will reach inside the plot and *outside* is the length they will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setTickLength](#), [setSubTickLengthIn](#), [setSubTickLengthOut](#)

10.455.4.57 setSubTickLengthIn()

```
void QCPPolarAxisAngular::setSubTickLengthIn (
    int inside )
```

Sets the length of the inward subticks in pixels. *inside* is the length the subticks will reach inside the plot.

See also

[setSubTickLengthOut](#), [setSubTickLength](#), [setTickLength](#)

10.455.4.58 setSubTickLengthOut()

```
void QCPPolarAxisAngular::setSubTickLengthOut (
    int outside )
```

Sets the length of the outward subticks in pixels. *outside* is the length the subticks will reach outside the plot. If *outside* is greater than zero, the tick labels will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setSubTickLengthIn](#), [setSubTickLength](#), [setTickLength](#)

10.455.4.59 setSubTickPen()

```
void QCPPolarAxisAngular::setSubTickPen (
    const QPen & pen )
```

Sets the pen, subtick marks will be drawn with.

See also

setSubTickCount, [setSubTickLength](#), [setBasePen](#)

10.455.4.60 setSubTicks()

```
void QCPPolarAxisAngular::setSubTicks (
    bool show )
```

Sets whether sub tick marks are displayed.

Sub ticks are only potentially visible if (major) ticks are also visible (see [setTicks](#))

See also

[setTicks](#)

10.455.4.61 setTicker()

```
void QCPPolarAxisAngular::setTicker (
    QSharedPointer< QCPAxisTicker > ticker )
```

The axis ticker is responsible for generating the tick positions and tick labels. See the documentation of [QCPAxisTicker](#) for details on how to work with axis tickers.

You can change the tick positioning/labeling behaviour of this axis by setting a different [QCPAxisTicker](#) subclass using this method. If you only wish to modify the currently installed axis ticker, access it via `ticker`.

Since the ticker is stored in the axis as a shared pointer, multiple axes may share the same axis ticker simply by passing the same shared pointer to multiple axes.

See also

`ticker`

10.455.4.62 setTickLabelColor()

```
void QCPPolarAxisAngular::setTickLabelColor (
    const QColor & color )
```

Sets the color of the tick labels.

See also

[setTickLabels](#), [setTickLabelFont](#)

10.455.4.63 setTickLabelFont()

```
void QCPPolarAxisAngular::setTickLabelFont (
    const QFont & font )
```

Sets the font of the tick labels.

See also

[setTickLabels](#), [setTickLabelColor](#)

10.455.4.64 setTickLabelPadding()

```
void QCPPolarAxisAngular::setTickLabelPadding (
    int padding )
```

Sets the distance between the axis base line (including any outward ticks) and the tick labels.

See also

[setLabelPadding](#), [setPadding](#)

10.455.4.65 setTickLabelRotation()

```
void QCPPolarAxisAngular::setTickLabelRotation (
    double degrees )
```

Sets the rotation of the tick labels. If *degrees* is zero, the labels are drawn normally. Else, the tick labels are drawn rotated by *degrees* clockwise. The specified angle is bound to values from -90 to 90 degrees.

If *degrees* is exactly -90, 0 or 90, the tick labels are centered on the tick coordinate. For other angles, the label is drawn with an offset such that it seems to point toward or away from the tick mark.

10.455.4.66 setTickLabels()

```
void QCPPolarAxisAngular::setTickLabels (
    bool show )
```

Sets whether tick labels are displayed. Tick labels are the numbers drawn next to tick marks.

10.455.4.67 setTickLength()

```
void QCPPolarAxisAngular::setTickLength (
    int inside,
    int outside = 0 )
```

Sets the length of the ticks in pixels. *inside* is the length the ticks will reach inside the plot and *outside* is the length they will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setSubTickLength](#), [setTickLengthIn](#), [setTickLengthOut](#)

10.455.4.68 setTickLengthIn()

```
void QCPPolarAxisAngular::setTickLengthIn (
    int inside )
```

Sets the length of the inward ticks in pixels. *inside* is the length the ticks will reach inside the plot.

See also

[setTickLengthOut](#), [setTickLength](#), [setSubTickLength](#)

10.455.4.69 setTickLengthOut()

```
void QCPPolarAxisAngular::setTickLengthOut (
    int outside )
```

Sets the length of the outward ticks in pixels. *outside* is the length the ticks will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setTickLengthIn](#), [setTickLength](#), [setSubTickLength](#)

10.455.4.70 setTickPen()

```
void QCPPolarAxisAngular::setTickPen (
    const QPen & pen )
```

Sets the pen, tick marks will be drawn with.

See also

[setTickLength](#), [setBasePen](#)

10.455.4.71 setTicks()

```
void QCPPolarAxisAngular::setTicks (
    bool show )
```

Sets whether tick marks are displayed.

Note that setting *show* to false does not imply that tick labels are invisible, too. To achieve that, see [setTickLabels](#).

See also

[setSubTicks](#)

10.455.4.72 size()

```
QSize QCPPolarAxisAngular::size ( ) const [inline]
```

Returns the pixel size of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.455.4.73 top()

```
int QCPPolarAxisAngular::top ( ) const [inline]
```

Returns the pixel position of the top border of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.455.4.74 topLeft()

```
QPoint QCPPolarAxisAngular::topLeft ( ) const [inline]
```

Returns the top left corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.455.4.75 topRight()

```
QPoint QCPPolarAxisAngular::topRight ( ) const [inline]
```

Returns the top right corner of this axis rect in pixels. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

10.455.4.76 update()

```
void QCPPolarAxisAngular::update (
    UpdatePhase phase ) [virtual]
```

This method is called automatically upon replot and doesn't need to be called by users of [QCPPolarAxisAngular](#).

Calls the base class implementation to update the margins (see [QCPLayoutElement::update](#)), and finally passes the [rect](#) to the inset layout ([insetLayout](#)) and calls its [QCPInsetLayout::update](#) function.

\see [baseclassmethod](#)

Reimplemented from [QCPLayoutElement](#).

10.455.4.77 wheelEvent()

```
void QCPPolarAxisAngular::wheelEvent (
    QWheelEvent * event ) [protected], [virtual]
```

This event gets called when the user turns the mouse scroll wheel while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`.

The `event->angleDelta()` indicates how far the mouse wheel was turned, which is usually +/- 120 for single rotation steps. However, if the mouse wheel is turned rapidly, multiple steps may accumulate to one event, making the delta larger. On the other hand, if the wheel has very smooth steps or none at all, the delta may be smaller.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#)

Reimplemented from [QCPLayerable](#).

10.455.4.78 width()

```
int QCPPolarAxisAngular::width ( ) const [inline]
```

Returns the pixel width of this axis rect. Margins are not taken into account here, so the returned value is with respect to the inner [rect](#).

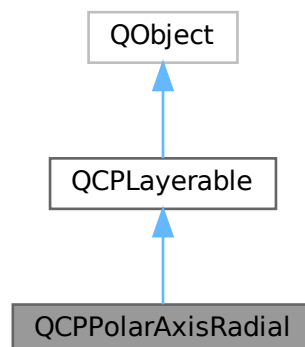
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

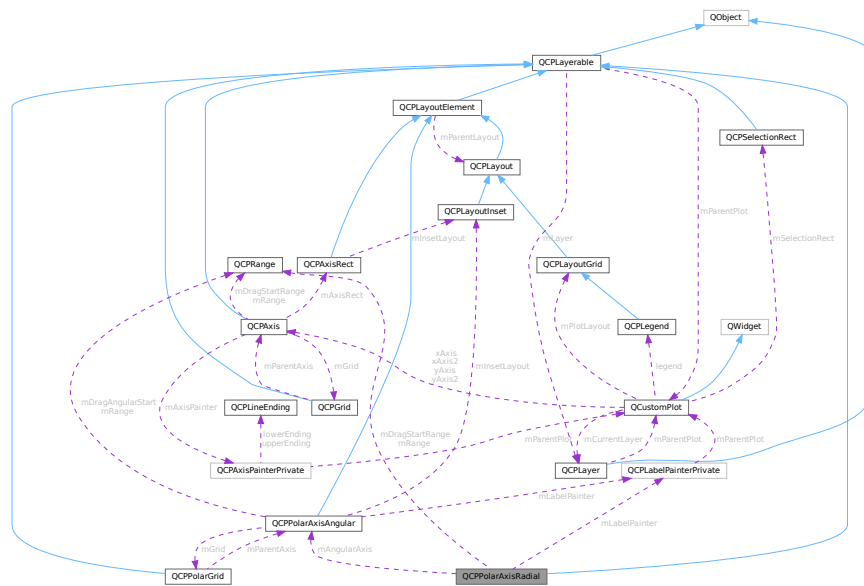
10.456 QCPPolarAxisRadial Class Reference

The radial axis inside a radial plot.

Inheritance diagram for QCPPolarAxisRadial:



Collaboration diagram for QCPPolarAxisRadial:



Public Types

- enum [AngleReference](#) { [arAbsolute](#) , [arAngularAxis](#) }
- enum [ScaleType](#) { [stLinear](#) , [stLogarithmic](#) }
- enum [SelectablePart](#) { [spNone](#) = 0 , [spAxis](#) = 0x001 , [spTickLabels](#) = 0x002 , [spAxisLabel](#) = 0x004 }
- enum [LabelMode](#) { [ImUpright](#) , [ImRotated](#) }

Signals

- void [rangeChanged](#) (const [QCPRange](#) &newRange)
- void [rangeChanged](#) (const [QCPRange](#) &newRange, const [QCPRange](#) &oldRange)
- void [scaleTypeChanged](#) ([QCPPolarAxisRadial::ScaleType](#) scaleType)
- void [selectionChanged](#) (const [QCPPolarAxisRadial::SelectableParts](#) &parts)
- void [selectableChanged](#) (const [QCPPolarAxisRadial::SelectableParts](#) &parts)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

Public Member Functions

- [QCPPolarAxisRadial](#) ([QCPPolarAxisAngular](#) *parent)
- bool [rangeDrag](#) () const
- bool [rangeZoom](#) () const
- double [rangeZoomFactor](#) () const
- [QCPPolarAxisAngular](#) * [angularAxis](#) () const
- [ScaleType](#) [scaleType](#) () const
- const [QCPRange](#) [range](#) () const

- bool **rangeReversed** () const
- double **angle** () const
- [AngleReference](#) **angleReference** () const
- QSharedPointer< [QCPAxisTicker](#) > **ticker** () const
- bool **ticks** () const
- bool **tickLabels** () const
- int **tickLabelPadding** () const
- QFont **tickLabelFont** () const
- QColor **tickLabelColor** () const
- double **tickLabelRotation** () const
- LabelMode **tickLabelMode** () const
- QString **numberFormat** () const
- int **numberPrecision** () const
- QVector< double > **tickVector** () const
- QVector< double > **subTickVector** () const
- QVector< QString > **tickVectorLabels** () const
- int **tickLengthIn** () const
- int **tickLengthOut** () const
- bool **subTicks** () const
- int **subTickLengthIn** () const
- int **subTickLengthOut** () const
- QPen **basePen** () const
- QPen **tickPen** () const
- QPen **subTickPen** () const
- QFont **labelFont** () const
- QColor **labelColor** () const
- QString **label** () const
- int **labelPadding** () const
- SelectableParts **selectedParts** () const
- SelectableParts **selectableParts** () const
- QFont **selectedTickLabelFont** () const
- QFont **selectedLabelFont** () const
- QColor **selectedTickLabelColor** () const
- QColor **selectedLabelColor** () const
- QPen **selectedBasePen** () const
- QPen **selectedTickPen** () const
- QPen **selectedSubTickPen** () const
- void **setRangeDrag** (bool enabled)
- void **setRangeZoom** (bool enabled)
- void **setRangeZoomFactor** (double factor)
- Q_SLOT void **setScaleType** ([QCPPolarAxisRadial::ScaleType](#) type)
- Q_SLOT void **setRange** (const [QCPRange](#) &range)
- void **setRange** (double lower, double upper)
- void **setRange** (double position, double size, Qt::AlignmentFlag alignment)
- void **setRangeLower** (double lower)
- void **setRangeUpper** (double upper)
- void **setRangeReversed** (bool reversed)
- void **setAngle** (double degrees)
- void **setAngleReference** ([AngleReference](#) reference)
- void **setTicker** (QSharedPointer< [QCPAxisTicker](#) > ticker)
- void **setTicks** (bool show)
- void **setTickLabels** (bool show)
- void **setTickLabelPadding** (int padding)
- void **setTickLabelFont** (const QFont &font)
- void **setTickLabelColor** (const QColor &color)

- void [setTickLabelRotation](#) (double degrees)
- void **setTickLabelMode** (LabelMode mode)
- void [setNumberFormat](#) (const QString &formatCode)
- void [setNumberPrecision](#) (int precision)
- void [setTickLength](#) (int inside, int outside=0)
- void [setTickLengthIn](#) (int inside)
- void [setTickLengthOut](#) (int outside)
- void [setSubTicks](#) (bool show)
- void [setSubTickLength](#) (int inside, int outside=0)
- void [setSubTickLengthIn](#) (int inside)
- void [setSubTickLengthOut](#) (int outside)
- void [setBasePen](#) (const QPen &pen)
- void [setTickPen](#) (const QPen &pen)
- void [setSubTickPen](#) (const QPen &pen)
- void [setLabelFont](#) (const QFont &font)
- void [setLabelColor](#) (const QColor &color)
- void [setLabel](#) (const QString &str)
- void [setLabelPadding](#) (int padding)
- void [setSelectedTickLabelFont](#) (const QFont &font)
- void [setSelectedLabelFont](#) (const QFont &font)
- void [setSelectedTickLabelColor](#) (const QColor &color)
- void [setSelectedLabelColor](#) (const QColor &color)
- void [setSelectedBasePen](#) (const QPen &pen)
- void [setSelectedTickPen](#) (const QPen &pen)
- void [setSelectedSubTickPen](#) (const QPen &pen)
- Q_SLOT void [setSelectableParts](#) (const QCPPolarAxisRadial::SelectableParts &selectableParts)
- Q_SLOT void [setSelectedParts](#) (const QCPPolarAxisRadial::SelectableParts &selectedParts)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=0) const Q_DECL_OVERRIDE
- void [moveRange](#) (double diff)
- void [scaleRange](#) (double factor)
- void [scaleRange](#) (double factor, double center)
- void [rescale](#) (bool onlyVisiblePlottables=false)
- void [pixelToCoord](#) (QPointF pixelPos, double &angleCoord, double &radiusCoord) const
- QPointF [coordToPixel](#) (double angleCoord, double radiusCoord) const
- double **coordToRadius** (double coord) const
- double **radiusToCoord** (double radius) const
- [SelectablePart](#) [getPartAt](#) (const QPointF &pos) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE
- virtual void [wheelEvent](#) (QWheelEvent *event) Q_DECL_OVERRIDE
- void [updateGeometry](#) (const QPointF ¢er, double radius)
- void [setupTickVectors](#) ()
- QPen [getBasePen](#) () const
- QPen [getTickPen](#) () const
- QPen [getSubTickPen](#) () const
- QFont [getTickLabelFont](#) () const
- QFont [getLabelFont](#) () const
- QColor [getTickLabelColor](#) () const
- QColor [getLabelColor](#) () const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot)
- virtual QRect [clipRect](#) () const
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- bool [mRangeDrag](#)
- bool [mRangeZoom](#)
- double [mRangeZoomFactor](#)
- [QCPPolarAxisAngular](#) * [mAngularAxis](#)
- double [mAngle](#)
- [AngleReference](#) [mAngleReference](#)
- SelectableParts [mSelectableParts](#)
- SelectableParts [mSelectedParts](#)
- QPen [mBasePen](#)
- QPen [mSelectedBasePen](#)
- int [mLabelPadding](#)
- QString [mLabel](#)
- QFont [mLabelFont](#)
- QFont [mSelectedLabelFont](#)
- QColor [mLabelColor](#)
- QColor [mSelectedLabelColor](#)
- bool [mTickLabels](#)
- QFont [mTickLabelFont](#)

- QFont **mSelectedTickLabelFont**
- QColor **mTickLabelColor**
- QColor **mSelectedTickLabelColor**
- int **mNumberPrecision**
- QLatin1Char **mNumberFormatChar**
- bool **mNumberBeautifulPowers**
- bool **mNumberMultiplyCross**
- bool **mTicks**
- bool **mSubTicks**
- int **mTickLengthIn**
- int **mTickLengthOut**
- int **mSubTickLengthIn**
- int **mSubTickLengthOut**
- QPen **mTickPen**
- QPen **mSelectedTickPen**
- QPen **mSubTickPen**
- QPen **mSelectedSubTickPen**
- [QCPRange](#) **mRange**
- bool **mRangeReversed**
- [ScaleType](#) **mScaleType**
- QPointF **mCenter**
- double **mRadius**
- QSharedPointer< [QCPAxisTicker](#) > **mTicker**
- QVector< double > **mTickVector**
- QVector< QString > **mTickVectorLabels**
- QVector< double > **mSubTickVector**
- bool **mDragging**
- [QCPRange](#) **mDragStartRange**
- QCP::AntialiasedElements **mAADragBackup**
- QCP::AntialiasedElements **mNotAADragBackup**
- [QCPLabelPainterPrivate](#) **mLabelPainter**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPPolarAxisAngular](#)

10.456.1 Detailed Description

The radial axis inside a radial plot.

Warning

In this [QCustomPlot](#) version, polar plots are a tech preview. Expect documentation and functionality to be incomplete, as well as changing public interfaces in the future.

Each axis holds an instance of [QCPAxisTicker](#) which is used to generate the tick coordinates and tick labels. You can access the currently installed [ticker](#) or set a new one (possibly one of the specialized subclasses, or your own subclass) via [setTicker](#). For details, see the documentation of [QCPAxisTicker](#).

10.456.2 Member Enumeration Documentation

10.456.2.1 AngleReference

enum [QCPPolarAxisRadial::AngleReference](#)

Defines the reference of the angle at which a radial axis is tilted ([setAngle](#)).

Enumerator

arAbsolute	The axis tilt is given in absolute degrees. The zero is to the right and positive angles are measured counter-clockwise.
arAngularAxis	The axis tilt is measured in the angular coordinate system given by the parent angular axis.

10.456.2.2 ScaleType

enum [QCPPolarAxisRadial::ScaleType](#)

Defines the scale of an axis.

See also

[setScaleType](#)

Enumerator

stLinear	Linear scaling.
stLogarithmic	Logarithmic scaling with correspondingly transformed axis coordinates (possibly also setTicker to a QCPAxisTickerLog instance).

10.456.2.3 SelectablePart

enum [QCPPolarAxisRadial::SelectablePart](#)

Defines the selectable parts of an axis.

See also

[setSelectableParts](#), [setSelectedParts](#)

Enumerator

spNone	None of the selectable parts.
spAxis	The axis backbone and tick marks.
spTickLabels	Tick labels (numbers) of this axis (as a whole, not individually)
spAxisLabel	The axis label.

10.456.3 Constructor & Destructor Documentation

10.456.3.1 QCPPolarAxisRadial()

```
QCPPolarAxisRadial::QCPPolarAxisRadial (
    QCPPolarAxisAngular * parent ) [explicit]
```

Constructs an Axis instance of Type *type* for the axis rect *parent*.

Usually it isn't necessary to instantiate axes directly, because you can let [QCustomPlot](#) create them for you with [QCPAxisRect::addAxis](#). If you want to use own QCPAxis-subclasses however, create them manually and then inject them also via [QCPAxisRect::addAxis](#).

10.456.4 Member Function Documentation

10.456.4.1 applyDefaultAntialiasingHint()

```
void QCPPolarAxisRadial::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

10.456.4.2 coordToPixel()

```
QPointF QCPPolarAxisRadial::coordToPixel (
    double angleCoord,
    double radiusCoord ) const
```

Transforms *value*, in coordinates of the axis, to pixel coordinates of the [QCustomPlot](#) widget.

10.456.4.3 deselectEvent()

```
void QCPPolarAxisRadial::deselectEvent (
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.456.4.4 draw()

```
void QCPPolarAxisRadial::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPLayerable](#).

10.456.4.5 getPartAt()

```
QCPPolarAxisRadial::SelectablePart QCPPolarAxisRadial::getPartAt (
    const QPointF & pos ) const
```

Returns the part of the axis that is hit by *pos* (in pixels). The return value of this function is independent of the user-selectable parts defined with [setSelectableParts](#). Further, this function does not change the current selection state of the axis.

If the axis is not visible ([setVisible](#)), this function always returns [spNone](#).

See also

[setSelectedParts](#), [setSelectableParts](#), [QCustomPlot::setInteractions](#)

10.456.4.6 mouseMoveEvent()

```
void QCPPolarAxisRadial::mouseMoveEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user moves the mouse while holding a mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *startPos* indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

10.456.4.7 mousePressEvent()

```
void QCPPolarAxisRadial::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [protected], [virtual]
```

This event gets called when the user presses a mouse button while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter *details* contains layerable-specific details about the hit, which were generated in the previous call to [selectTest](#). For example, One-dimensional plottables like [QCPGraph](#) or [QCPBars](#) convey the clicked data point in the *details* parameter, as [QCPDataSelection](#) packed as `QVariant`. Multi-part objects convey the specific `SelectablePart` that was hit (e.g. [QCPAxis::SelectablePart](#) in the case of axes).

[QCustomPlot](#) uses an event propagation system that works the same as Qt's system. If your layerable doesn't reimplement the [mousePressEvent](#) or explicitly calls `event->ignore()` in its reimplementation, the event will be propagated to the next layerable in the stacking order.

Once a layerable has accepted the [mousePressEvent](#), it is considered the mouse grabber and will receive all following calls to [mouseMoveEvent](#) or [mouseReleaseEvent](#) for this mouse interaction (a "mouse interaction" in this context ends with the release).

The default implementation does nothing except explicitly ignoring the event with `event->ignore()`.

See also

[mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

10.456.4.8 mouseReleaseEvent()

```
void QCPPolarAxisRadial::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [protected], [virtual]
```

This event gets called when the user releases the mouse button, after this layerable has become the mouse grabber by accepting the preceding [mousePressEvent](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`. The parameter `startPos` indicates the position where the initial [mousePressEvent](#) occurred, that started the mouse interaction.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseDoubleClickEvent](#), [wheelEvent](#)

Reimplemented from [QCPLayerable](#).

10.456.4.9 moveRange()

```
void QCPPolarAxisRadial::moveRange (
    double diff )
```

If the scale type ([setScaleType](#)) is [stLinear](#), `diff` is added to the lower and upper bounds of the range. The range is simply moved by `diff`.

If the scale type is [stLogarithmic](#), the range bounds are multiplied by `diff`. This corresponds to an apparent "linear" move in logarithmic scaling by a distance of $\log(\text{diff})$.

10.456.4.10 pixelToCoord()

```
void QCPPolarAxisRadial::pixelToCoord (
    QPointF pixelPos,
    double & angleCoord,
    double & radiusCoord ) const
```

Transforms `value`, in pixel coordinates of the [QCustomPlot](#) widget, to axis coordinates.

10.456.4.11 rangeChanged [1/2]

```
void QCPPolarAxisRadial::rangeChanged (
    const QCPRange & newRange ) [signal]
```

This signal is emitted when the range of this axis has changed. You can connect it to the [setRange](#) slot of another axis to communicate the new range to the other axis, in order for it to be synchronized.

You may also manipulate/correct the range with [setRange](#) in a slot connected to this signal. This is useful if for example a maximum range span shall not be exceeded, or if the lower/upper range shouldn't go beyond certain values (see [QCPRange::bounded](#)). For example, the following slot would limit the x axis to ranges between 0 and 10:

```
customPlot->xAxis->setRange(newRange.bounded(0, 10))
```


10.456.4.12 rangeChanged [2/2]

```
void QCPPolarAxisRadial::rangeChanged (
    const QCPRange & newRange,
    const QCPRange & oldRange ) [signal]
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Additionally to the new range, this signal also provides the previous range held by the axis as *oldRange*.

10.456.4.13 rescale()

```
void QCPPolarAxisRadial::rescale (
    bool onlyVisiblePlottables = false )
```

Changes the axis range such that all plottables associated with this axis are fully visible in that dimension.

See also

[QCPAbstractPlottable::rescaleAxes](#), [QCustomPlot::rescaleAxes](#)

10.456.4.14 scaleRange() [1/2]

```
void QCPPolarAxisRadial::scaleRange (
    double factor )
```

Scales the range of this axis by *factor* around the center of the current axis range. For example, if *factor* is 2.0, then the axis range will double its size, and the point at the axis range center won't have changed its position in the [QCustomPlot](#) widget (i.e. coordinates around the center will have moved symmetrically closer).

If you wish to scale around a different coordinate than the current axis range center, use the overload [scaleRange\(double factor, double center\)](#).

10.456.4.15 scaleRange() [2/2]

```
void QCPPolarAxisRadial::scaleRange (
    double factor,
    double center )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Scales the range of this axis by *factor* around the coordinate *center*. For example, if *factor* is 2.0, *center* is 1.0, then the axis range will double its size, and the point at coordinate 1.0 won't have changed its position in the [QCustomPlot](#) widget (i.e. coordinates around 1.0 will have moved symmetrically closer to 1.0).

See also

[scaleRange\(double factor\)](#)

10.456.4.16 scaleTypeChanged

```
void QCPPolarAxisRadial::scaleTypeChanged (
    QCPPolarAxisRadial::ScaleType scaleType ) [signal]
```

This signal is emitted when the scale type changes, by calls to [setScaleType](#)

10.456.4.17 selectableChanged

```
void QCPPolarAxisRadial::selectableChanged (
    const QCPPolarAxisRadial::SelectableParts & parts ) [signal]
```

This signal is emitted when the selectability changes, by calls to [setSelectableParts](#)

10.456.4.18 selectEvent()

```
void QCPPolarAxisRadial::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.456.4.19 selectionCategory()

```
QCP::Interaction QCPPolarAxisRadial::selectionCategory ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.456.4.20 selectionChanged

```
void QCPPolarAxisRadial::selectionChanged (
    const QCPPolarAxisRadial::SelectableParts & parts ) [signal]
```

This signal is emitted when the selection state of this axis has changed, either by user interaction or by a direct call to [setSelectedParts](#).

10.456.4.21 selectTest()

```
double QCPPolarAxisRadial::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = 0 ) const [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the selectionTolerance multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than 0.99*selectionTolerance).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the mouseReleaseEvent occurs, and the finally selected object is notified via the selectEvent/ deselectEvent methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to selectEvent when the parent [QCustomPlot](#) decides on the basis of this selectTest call, that the object was successfully selected. The subsequent call to selectEvent will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in [selectTest](#). The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent selectEvent, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

[selectEvent](#), [deselectEvent](#), [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Reimplemented from [QCPLayerable](#).

10.456.4.22 setBasePen()

```
void QCPPolarAxisRadial::setBasePen (
    const QPen & pen )
```

Sets the pen, the axis base line is drawn with.

See also

[setTickPen](#), [setSubTickPen](#)

10.456.4.23 setLabel()

```
void QCPPolarAxisRadial::setLabel (
    const QString & str )
```

Sets the text of the axis label that will be shown below/above or next to the axis, depending on its orientation. To disable axis labels, pass an empty string as *str*.

10.456.4.24 setLabelColor()

```
void QCPPolarAxisRadial::setLabelColor (
    const QColor & color )
```

Sets the color of the axis label.

See also

[setLabelFont](#)

10.456.4.25 setLabelFont()

```
void QCPPolarAxisRadial::setLabelFont (
    const QFont & font )
```

Sets the font of the axis label.

See also

[setLabelColor](#)

10.456.4.26 setLabelPadding()

```
void QCPPolarAxisRadial::setLabelPadding (
    int padding )
```

Sets the distance between the tick labels and the axis label.

See also

[setTickLabelPadding](#), [setPadding](#)

10.456.4.27 setNumberFormat()

```
void QCPPolarAxisRadial::setNumberFormat (
    const QString & formatCode )
```

Sets the number format for the numbers in tick labels. This *formatCode* is an extended version of the format code used e.g. by `QString::number()` and `QLocale::toString()`. For reference about that, see the "Argument Formats" section in the detailed description of the `QString` class.

formatCode is a string of one, two or three characters. The first character is identical to the normal format code used by Qt. In short, this means: 'e'/'E' scientific format, 'f' fixed format, 'g'/'G' scientific or fixed, whichever is shorter.

The second and third characters are optional and specific to [QCustomPlot](#):

If the first char was 'e' or 'g', numbers are/might be displayed in the scientific format, e.g. "5.5e9", which is ugly in a plot. So when the second char of *formatCode* is set to 'b' (for "beautiful"), those exponential numbers are formatted in a more natural way, i.e. "5.5 [multiplication sign] 10 [superscript] 9". By default, the multiplication sign is a centered dot. If instead a cross should be shown (as is usual in the USA), the third char of *formatCode* can be set to 'c'. The inserted multiplication signs are the UTF-8 characters 215 (0xD7) for the cross and 183 (0xB7) for the dot.

Examples for *formatCode*:

- `g` normal format code behaviour. If number is small, fixed format is used, if number is large, normal scientific format is used
- `gb` If number is small, fixed format is used, if number is large, scientific format is used with beautifully typeset decimal powers and a dot as multiplication sign
- `ebc` All numbers are in scientific format with beautifully typeset decimal power and a cross as multiplication sign
- `fb` illegal format code, since fixed format doesn't support (or need) beautifully typeset decimal powers. Format code will be reduced to 'f'.
- `hello` illegal format code, since first char is not 'e', 'E', 'f', 'g' or 'G'. Current format code will not be changed.

10.456.4.28 setNumberPrecision()

```
void QCPPolarAxisRadial::setNumberPrecision (
    int precision )
```

Sets the precision of the tick label numbers. See `QLocale::toString(double i, char f, int prec)` for details. The effect of precisions are most notably for number Formats starting with 'e', see [setNumberFormat](#)

10.456.4.29 setRange() [1/3]

```
void QCPPolarAxisRadial::setRange (
    const QCPRange & range )
```

Sets the range of the axis.

This slot may be connected with the [rangeChanged](#) signal of another axis so this axis is always synchronized with the other axis range, when it changes.

To invert the direction of an axis, use [setRangeReversed](#).

10.456.4.30 `setRange()` [2/3]

```
void QCPPolarAxisRadial::setRange (
    double lower,
    double upper )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the lower and upper bound of the axis range.

To invert the direction of an axis, use [setRangeReversed](#).

There is also a slot to set a range, see [setRange\(const QCPRange &range\)](#).

10.456.4.31 `setRange()` [3/3]

```
void QCPPolarAxisRadial::setRange (
    double position,
    double size,
    Qt::AlignmentFlag alignment )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the range of the axis.

The *position* coordinate indicates together with the *alignment* parameter, where the new range will be positioned. *size* defines the size of the new axis range. *alignment* may be `Qt::AlignLeft`, `Qt::AlignRight` or `Qt::AlignCenter`. This will cause the left border, right border, or center of the range to be aligned with *position*. Any other values of *alignment* will default to `Qt::AlignCenter`.

10.456.4.32 `setRangeLower()`

```
void QCPPolarAxisRadial::setRangeLower (
    double lower )
```

Sets the lower bound of the axis range. The upper bound is not changed.

See also

[setRange](#)

10.456.4.33 `setRangeReversed()`

```
void QCPPolarAxisRadial::setRangeReversed (
    bool reversed )
```

Sets whether the axis range (direction) is displayed reversed. Normally, the values on horizontal axes increase left to right, on vertical axes bottom to top. When *reversed* is set to true, the direction of increasing values is inverted.

Note that the range and data interface stays the same for reversed axes, e.g. the *lower* part of the [setRange](#) interface will still reference the mathematically smaller number than the *upper* part.

10.456.4.34 setRangeUpper()

```
void QCPPolarAxisRadial::setRangeUpper (
    double upper )
```

Sets the upper bound of the axis range. The lower bound is not changed.

See also

[setRange](#)

10.456.4.35 setScaleType()

```
void QCPPolarAxisRadial::setScaleType (
    QCPPolarAxisRadial::ScaleType type )
```

Sets whether the axis uses a linear scale or a logarithmic scale.

Note that this method controls the coordinate transformation. For logarithmic scales, you will likely also want to use a logarithmic tick spacing and labeling, which can be achieved by setting the axis ticker to an instance of [QCPAxisTickerLog](#) :

See the documentation of [QCPAxisTickerLog](#) about the details of logarithmic axis tick creation.

[setNumberPrecision](#)

10.456.4.36 setSelectableParts()

```
void QCPPolarAxisRadial::setSelectableParts (
    const QCPPolarAxisRadial::SelectableParts & selectableParts )
```

Sets whether the user can (de-)select the parts in *selectable* by clicking on the [QCustomPlot](#) surface. (When [QCustomPlot::setInteractions](#) contains `iSelectAxes`.)

However, even when *selectable* is set to a value not allowing the selection of a specific part, it is still possible to set the selection of this part manually, by calling [setSelectedParts](#) directly.

See also

[SelectablePart](#), [setSelectedParts](#)

10.456.4.37 setSelectedBasePen()

```
void QCPPolarAxisRadial::setSelectedBasePen (
    const QPen & pen )
```

Sets the pen that is used to draw the axis base line when selected.

See also

[setBasePen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.456.4.38 setSelectedLabelColor()

```
void QCPPolarAxisRadial::setSelectedLabelColor (
    const QColor & color )
```

Sets the color that is used for the axis label when it is selected.

See also

[setLabelColor](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.456.4.39 setSelectedLabelFont()

```
void QCPPolarAxisRadial::setSelectedLabelFont (
    const QFont & font )
```

Sets the font that is used for the axis label when it is selected.

See also

[setLabelFont](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.456.4.40 setSelectedParts()

```
void QCPPolarAxisRadial::setSelectedParts (
    const QCPPolarAxisRadial::SelectableParts & selectedParts )
```

Sets the selected state of the respective axis parts described by [SelectablePart](#). When a part is selected, it uses a different pen/font.

The entire selection mechanism for axes is handled automatically when [QCustomPlot::setInteractions](#) contains [iSelectAxes](#). You only need to call this function when you wish to change the selection state manually.

This function can change the selection state of a part, independent of the [setSelectableParts](#) setting.

emits the [selectionChanged](#) signal when *selected* is different from the previous selection state.

See also

[SelectablePart](#), [setSelectableParts](#), [selectTest](#), [setSelectedBasePen](#), [setSelectedTickPen](#), [setSelectedSubTickPen](#), [setSelectedTickLabelFont](#), [setSelectedLabelFont](#), [setSelectedTickLabelColor](#), [setSelectedLabelColor](#)

10.456.4.41 setSelectedSubTickPen()

```
void QCPPolarAxisRadial::setSelectedSubTickPen (
    const QPen & pen )
```

Sets the pen that is used to draw the subticks when selected.

See also

[setSubTickPen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.456.4.42 setSelectedTickLabelColor()

```
void QCPPolarAxisRadial::setSelectedTickLabelColor (
    const QColor & color )
```

Sets the color that is used for tick labels when they are selected.

See also

[setTickLabelColor](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.456.4.43 setSelectedTickLabelFont()

```
void QCPPolarAxisRadial::setSelectedTickLabelFont (
    const QFont & font )
```

Sets the font that is used for tick labels when they are selected.

See also

[setTickLabelFont](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.456.4.44 setSelectedTickPen()

```
void QCPPolarAxisRadial::setSelectedTickPen (
    const QPen & pen )
```

Sets the pen that is used to draw the (major) ticks when selected.

See also

[setTickPen](#), [setSelectableParts](#), [setSelectedParts](#), [QCustomPlot::setInteractions](#)

10.456.4.45 setSubTickLength()

```
void QCPPolarAxisRadial::setSubTickLength (
    int inside,
    int outside = 0 )
```

Sets the length of the subticks in pixels. *inside* is the length the subticks will reach inside the plot and *outside* is the length they will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setTickLength](#), [setSubTickLengthIn](#), [setSubTickLengthOut](#)

10.456.4.46 setSubTickLengthIn()

```
void QCPPolarAxisRadial::setSubTickLengthIn (
    int inside )
```

Sets the length of the inward subticks in pixels. *inside* is the length the subticks will reach inside the plot.

See also

[setSubTickLengthOut](#), [setSubTickLength](#), [setTickLength](#)

10.456.4.47 setSubTickLengthOut()

```
void QCPPolarAxisRadial::setSubTickLengthOut (
    int outside )
```

Sets the length of the outward subticks in pixels. *outside* is the length the subticks will reach outside the plot. If *outside* is greater than zero, the tick labels will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setSubTickLengthIn](#), [setSubTickLength](#), [setTickLength](#)

10.456.4.48 setSubTickPen()

```
void QCPPolarAxisRadial::setSubTickPen (
    const QPen & pen )
```

Sets the pen, subtick marks will be drawn with.

See also

[setSubTickCount](#), [setSubTickLength](#), [setBasePen](#)

10.456.4.49 setSubTicks()

```
void QCPPolarAxisRadial::setSubTicks (
    bool show )
```

Sets whether sub tick marks are displayed.

Sub ticks are only potentially visible if (major) ticks are also visible (see [setTicks](#))

See also

[setTicks](#)

10.456.4.50 `setTicker()`

```
void QCPPolarAxisRadial::setTicker (
    QSharedPointer< QCPAxisTicker > ticker )
```

The axis ticker is responsible for generating the tick positions and tick labels. See the documentation of [QCPAxisTicker](#) for details on how to work with axis tickers.

You can change the tick positioning/labeling behaviour of this axis by setting a different [QCPAxisTicker](#) subclass using this method. If you only wish to modify the currently installed axis ticker, access it via [ticker](#).

Since the ticker is stored in the axis as a shared pointer, multiple axes may share the same axis ticker simply by passing the same shared pointer to multiple axes.

See also

[ticker](#)

10.456.4.51 `setTickLabelColor()`

```
void QCPPolarAxisRadial::setTickLabelColor (
    const QColor & color )
```

Sets the color of the tick labels.

See also

[setTickLabels](#), [setTickLabelFont](#)

10.456.4.52 `setTickLabelFont()`

```
void QCPPolarAxisRadial::setTickLabelFont (
    const QFont & font )
```

Sets the font of the tick labels.

See also

[setTickLabels](#), [setTickLabelColor](#)

10.456.4.53 `setTickLabelPadding()`

```
void QCPPolarAxisRadial::setTickLabelPadding (
    int padding )
```

Sets the distance between the axis base line (including any outward ticks) and the tick labels.

See also

[setLabelPadding](#), [setPadding](#)

10.456.4.54 setTickLabelRotation()

```
void QCPPolarAxisRadial::setTickLabelRotation (
    double degrees )
```

Sets the rotation of the tick labels. If *degrees* is zero, the labels are drawn normally. Else, the tick labels are drawn rotated by *degrees* clockwise. The specified angle is bound to values from -90 to 90 degrees.

If *degrees* is exactly -90, 0 or 90, the tick labels are centered on the tick coordinate. For other angles, the label is drawn with an offset such that it seems to point toward or away from the tick mark.

10.456.4.55 setTickLabels()

```
void QCPPolarAxisRadial::setTickLabels (
    bool show )
```

Sets whether tick labels are displayed. Tick labels are the numbers drawn next to tick marks.

10.456.4.56 setTickLength()

```
void QCPPolarAxisRadial::setTickLength (
    int inside,
    int outside = 0 )
```

Sets the length of the ticks in pixels. *inside* is the length the ticks will reach inside the plot and *outside* is the length they will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setSubTickLength](#), [setTickLengthIn](#), [setTickLengthOut](#)

10.456.4.57 setTickLengthIn()

```
void QCPPolarAxisRadial::setTickLengthIn (
    int inside )
```

Sets the length of the inward ticks in pixels. *inside* is the length the ticks will reach inside the plot.

See also

[setTickLengthOut](#), [setTickLength](#), [setSubTickLength](#)

10.456.4.58 setTickLengthOut()

```
void QCPPolarAxisRadial::setTickLengthOut (
    int outside )
```

Sets the length of the outward ticks in pixels. *outside* is the length the ticks will reach outside the plot. If *outside* is greater than zero, the tick labels and axis label will increase their distance to the axis accordingly, so they won't collide with the ticks.

See also

[setTickLengthIn](#), [setTickLength](#), [setSubTickLength](#)

10.456.4.59 setTickPen()

```
void QCPPolarAxisRadial::setTickPen (
    const QPen & pen )
```

Sets the pen, tick marks will be drawn with.

See also

[setTickLength](#), [setBasePen](#)

10.456.4.60 setTicks()

```
void QCPPolarAxisRadial::setTicks (
    bool show )
```

Sets whether tick marks are displayed.

Note that setting *show* to false does not imply that tick labels are invisible, too. To achieve that, see [setTickLabels](#).

See also

[setSubTicks](#)

10.456.4.61 ticker()

```
QSharedPointer< QCPAxisTicker > QCPPolarAxisRadial::ticker ( ) const [inline]
```

Returns a modifiable shared pointer to the currently installed axis ticker. The axis ticker is responsible for generating the tick positions and tick labels of this axis. You can access the [QCPAxisTicker](#) with this method and modify basic properties such as the approximate tick count ([QCPAxisTicker::setTickCount](#)).

You can gain more control over the axis ticks by setting a different [QCPAxisTicker](#) subclass, see the documentation there. A new axis ticker can be set with [setTicker](#).

Since the ticker is stored in the axis as a shared pointer, multiple axes may share the same axis ticker simply by passing the same shared pointer to multiple axes.

See also

[setTicker](#)

10.456.4.62 wheelEvent()

```
void QCPPolarAxisRadial::wheelEvent (
    QWheelEvent * event ) [protected], [virtual]
```

This event gets called when the user turns the mouse scroll wheel while the cursor is over the layerable. Whether a cursor is over the layerable is decided by a preceding call to [selectTest](#).

The current pixel position of the cursor on the [QCustomPlot](#) widget is accessible via `event->pos()`.

The `event->angleDelta()` indicates how far the mouse wheel was turned, which is usually +/- 120 for single rotation steps. However, if the mouse wheel is turned rapidly, multiple steps may accumulate to one event, making the delta larger. On the other hand, if the wheel has very smooth steps or none at all, the delta may be smaller.

The default implementation does nothing.

See also

[mousePressEvent](#), [mouseMoveEvent](#), [mouseReleaseEvent](#), [mouseDoubleClickEvent](#)

Reimplemented from [QCPLayerable](#).

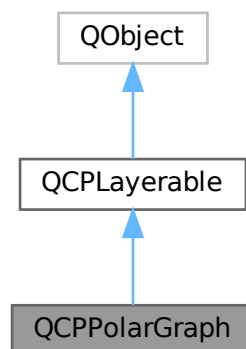
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.457 QCPPolarGraph Class Reference

A radial graph used to display data in polar plots.

Inheritance diagram for QCPPolarGraph:



- QSharedPointer< [QCPGraphDataContainer](#) > **data** () const
- [LineStyle](#) **lineStyle** () const
- [QCPScatterStyle](#) **scatterStyle** () const
- void **setName** (const QString &name)
- void **setAntialiasedFill** (bool enabled)
- void **setAntialiasedScatters** (bool enabled)
- void **setPen** (const QPen &pen)
- void **setBrush** (const QBrush &brush)
- void **setPeriodic** (bool enabled)
- void **setKeyAxis** ([QCPPolarAxisAngular](#) *axis)
- void **setValueAxis** ([QCPPolarAxisRadial](#) *axis)
- Q_SLOT void **setSelectable** ([QCP::SelectionType](#) selectable)
- Q_SLOT void **setSelection** ([QCPDataSelection](#) selection)
- void **setData** (QSharedPointer< [QCPGraphDataContainer](#) > data)
- void **setData** (const QVector< double > &keys, const QVector< double > &values, bool alreadySorted=false)
- void **setLineStyle** ([LineStyle](#) ls)
- void **setScatterStyle** (const [QCPScatterStyle](#) &style)
- void **addData** (const QVector< double > &keys, const QVector< double > &values, bool alreadySorted=false)
- void **addData** (double key, double value)
- void **coordsToPixels** (double key, double value, double &x, double &y) const
- const QPointF **coordsToPixels** (double key, double value) const
- void **pixelsToCoords** (double x, double y, double &key, double &value) const
- void **pixelsToCoords** (const QPointF &pixelPos, double &key, double &value) const
- void **rescaleAxes** (bool onlyEnlarge=false) const
- void **rescaleKeyAxis** (bool onlyEnlarge=false) const
- void **rescaleValueAxis** (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool **addToLegend** ([QCPLegend](#) *legend)
- bool **addToLegend** ()
- bool **removeFromLegend** ([QCPLegend](#) *legend) const
- bool **removeFromLegend** () const
- virtual double **selectTest** (const QPointF &pos, bool onlySelectable, QVariant *details=0) const
- virtual [QCPPlottableInterface1D](#) * **interface1D** ()
- virtual [QCPRange](#) **getKeyRange** (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const
- virtual [QCPRange](#) **getValueRange** (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const QString &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual QRect [clipRect](#) () const
- virtual void [draw](#) (QCPPainter *painter)
- virtual [QCP::Interaction](#) [selectionCategory](#) () const
- void [applyDefaultAntialiasingHint](#) (QCPPainter *painter) const
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionState↵ Changed)
- virtual void [deselectEvent](#) (bool *selectionStateChanged)
- virtual void [drawLinePlot](#) (QCPPainter *painter, const QVector< QPointF > &lines) const
- virtual void [drawFill](#) (QCPPainter *painter, QVector< QPointF > &lines) const
- virtual void [drawScatterPlot](#) (QCPPainter *painter, const QVector< QPointF > &scatters, const [QCPScatterStyle](#) &style) const
- virtual void [drawLegendIcon](#) (QCPPainter *painter, const QRectF &rect) const
- void [applyFillAntialiasingHint](#) (QCPPainter *painter) const
- void [applyScattersAntialiasingHint](#) (QCPPainter *painter) const
- double [pointDistance](#) (const QPointF &pixelPoint, QCPGraphDataContainer::const_iterator &closestData) const
- virtual int [dataCount](#) () const
- void [getDataSegments](#) (QList< [QCPDataRange](#) > &selectedSegments, QList< [QCPDataRange](#) > &unselectedSegments) const
- void [drawPolyline](#) (QCPPainter *painter, const QVector< QPointF > &lineData) const
- void [getVisibleDataBounds](#) (QCPGraphDataContainer::const_iterator &begin, QCPGraphDataContainer↵ ::const_iterator &end, const [QCPDataRange](#) &rangeRestriction) const
- void [getLines](#) (QVector< QPointF > *lines, const [QCPDataRange](#) &dataRange) const
- void [getScatters](#) (QVector< QPointF > *scatters, const [QCPDataRange](#) &dataRange) const
- void [getOptimizedLineData](#) (QVector< [QCPGraphData](#) > *lineData, const QCPGraphDataContainer↵ ::const_iterator &begin, const QCPGraphDataContainer::const_iterator &end) const
- void [getOptimizedScatterData](#) (QVector< [QCPGraphData](#) > *scatterData, QCPGraphDataContainer↵ ::const_iterator begin, QCPGraphDataContainer::const_iterator end) const
- QVector< QPointF > [dataToLines](#) (const QVector< [QCPGraphData](#) > &data) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) (QCustomPlot *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) (QCustomPlot *parentPlot)
- void [setParentLayerable](#) (QCPLayerable *parentLayerable)
- bool [moveToLayer](#) (QCPLayer *layer, bool prepend)
- void [applyAntialiasingHint](#) (QCPPainter *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QSharedPointer< [QCPGraphDataContainer](#) > **mDataContainer**
- [LineStyle](#) **mLineStyle**
- [QCPScatterStyle](#) **mScatterStyle**
- QString **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- QPen **mPen**
- QBrush **mBrush**
- bool **mPeriodic**
- QPointer< [QCPPolarAxisAngular](#) > **mKeyAxis**
- QPointer< [QCPPolarAxisRadial](#) > **mValueAxis**
- [QCP::SelectionType](#) **mSelectable**
- [QCPDataSelection](#) **mSelection**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCPPolarLegendItem](#)

10.457.1 Detailed Description

A radial graph used to display data in polar plots.

Warning

In this [QCustomPlot](#) version, polar plots are a tech preview. Expect documentation and functionality to be incomplete, as well as changing public interfaces in the future.

10.457.2 Member Enumeration Documentation

10.457.2.1 LineStyle

```
enum QCPPolarGraph::LineStyle
```

Defines how the graph's line is represented visually in the plot. The line is drawn with the current pen of the graph ([setPen](#)).

See also

[setLineStyle](#)

Enumerator

IsNone	with symbols according to the scatter style, see setScatterStyle) data points are not connected with any lines (e.g. data only represented)
IsLine	data points are connected by a straight line

10.457.3 Constructor & Destructor Documentation

10.457.3.1 QCPPolarGraph()

```
QCPPolarGraph::QCPPolarGraph (
    QCPPolarAxisAngular * keyAxis,
    QCPPolarAxisRadial * valueAxis )
```

Constructs a graph which uses *keyAxis* as its angular and *valueAxis* as its radial axis. *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#), and the radial axis must be associated with the angular axis. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

The created [QCPPolarGraph](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPPolarGraph](#), so do not delete it manually but use [QCPPolarAxisAngular::removeGraph\(\)](#) instead.

To directly create a [QCPPolarGraph](#) inside a plot, you should use the [QCPPolarAxisAngular::addGraph](#) method.

10.457.4 Member Function Documentation

10.457.4.1 applyDefaultAntialiasingHint()

```
void QCPPolarGraph::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

10.457.4.2 clipRect()

```
QRect QCPPolarGraph::clipRect ( ) const [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.457.4.3 coordsToPixels()

```
void QCPPolarGraph::coordsToPixels (
    double key,
    double value,
    double & x,
    double & y ) const
```

Use this method to set an own [QCPSelectionDecorator](#) (subclass) instance. This allows you to customize the visual representation of selected data ranges further than by using the default [QCPSelectionDecorator](#).

The plottable takes ownership of the *decorator*.

The currently set decorator can be accessed via `selectionDecorator`.

10.457.4.4 `deselectEvent()`

```
void QCPPolarGraph::deselectEvent (
    bool * selectionStateChanged )    [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.457.4.5 `draw()`

```
void QCPPolarGraph::draw (
    QCPPainter * painter )    [protected], [virtual]
```

Implements [QCPLayerable](#).

10.457.4.6 `selectEvent()`

```
void QCPPolarGraph::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged )    [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.457.4.7 `selectionCategory()`

```
QCP::Interaction QCPPolarGraph::selectionCategory ( ) const    [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.457.4.8 `selectTest()`

```
double QCPPolarGraph::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = 0 ) const    [virtual]
```

This function is used to decide whether a click hits a layerable object or not.

pos is a point in pixel coordinates on the [QCustomPlot](#) surface. This function returns the shortest pixel distance of this point to the object. If the object is either invisible or the distance couldn't be determined, -1.0 is returned. Further, if *onlySelectable* is true and the object is not selectable, -1.0 is returned, too.

If the object is represented not by single lines but by an area like a [QCPItemText](#) or the bars of a [QCPBars](#) plottable, a click inside the area should also be considered a hit. In these cases this function thus returns a constant value greater zero but still below the parent plot's selection tolerance. (typically the `selectionTolerance` multiplied by 0.99).

Providing a constant value for area objects allows selecting line objects even when they are obscured by such area objects, by clicking close to the lines (i.e. closer than $0.99 \cdot \text{selectionTolerance}$).

The actual setting of the selection state is not done by this function. This is handled by the parent [QCustomPlot](#) when the `mouseReleaseEvent` occurs, and the finally selected object is notified via the `selectEvent/` `deselectEvent` methods.

details is an optional output parameter. Every layerable subclass may place any information in *details*. This information will be passed to `selectEvent` when the parent [QCustomPlot](#) decides on the basis of this `selectTest` call, that the object was successfully selected. The subsequent call to `selectEvent` will carry the *details*. This is useful for multi-part objects (like [QCPAxis](#)). This way, a possibly complex calculation to decide which part was clicked is only done once in `selectTest`. The result (i.e. the actually clicked part) can then be placed in *details*. So in the subsequent `selectEvent`, the decision which part was selected doesn't have to be done a second time for a single selection operation.

In the case of 1D Plottables ([QCPAbstractPlottable1D](#), like [QCPGraph](#) or [QCPBars](#)) *details* will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

You may pass `nullptr` as *details* to indicate that you are not interested in those selection details.

See also

`selectEvent`, `deselectEvent`, [mousePressEvent](#), [wheelEvent](#), [QCustomPlot::setInteractions](#), [QCPAbstractPlottable1D::selectTest](#)

Reimplemented from [QCPLayerable](#).

10.457.4.9 setAntialiasedFill()

```
void QCPPolarGraph::setAntialiasedFill (
    bool enabled )
```

Sets whether fills of this plottable are drawn antialiased or not.

Note that this setting may be overridden by [QCustomPlot::setAntialiasedElements](#) and [QCustomPlot::setNotAntialiasedElements](#).

10.457.4.10 setAntialiasedScatters()

```
void QCPPolarGraph::setAntialiasedScatters (
    bool enabled )
```

Sets whether the scatter symbols of this plottable are drawn antialiased or not.

Note that this setting may be overridden by [QCustomPlot::setAntialiasedElements](#) and [QCustomPlot::setNotAntialiasedElements](#).

10.457.4.11 setBrush()

```
void QCPPolarGraph::setBrush (
    const QBrush & brush )
```

The brush is used to draw basic fills of the plottable representation in the plot. The Fill can be a color, gradient or texture, see the usage of `QBrush`.

For example, the [QCPGraph](#) subclass draws the fill under the graph with this brush, when it's not set to `Qt::NoBrush`.

See also

[setPen](#)

10.457.4.12 setData() [1/2]

```
void QCPPolarGraph::setData (
    const QVector< double > & keys,
    const QVector< double > & values,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data with the provided points in *keys* and *values*. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

See also

[addData](#)

10.457.4.13 setData() [2/2]

```
void QCPPolarGraph::setData (
    QSharedPointer< QCPGraphDataContainer > data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data container with the provided *data* container.

Since a QSharedPointer is used, multiple QCPPolarGraphs may share the same data container safely. Modifying the data in the container will then affect all graphs that share the container. Sharing can be achieved by simply exchanging the data containers wrapped in shared pointers:

If you do not wish to share containers, but create a copy from an existing container, rather use the [QCPDataContainer<DataType>::set](#) method on the graph's data container directly:

See also

[addData](#)

10.457.4.14 setKeyAxis()

```
void QCPPolarGraph::setKeyAxis (
    QCPPolarAxisAngular * axis )
```

The key axis of a plottable can be set to any axis of a [QCustomPlot](#), as long as it is orthogonal to the plottable's value axis. This function performs no checks to make sure this is the case. The typical mathematical choice is to use the x-axis ([QCustomPlot::xAxis](#)) as key axis and the y-axis ([QCustomPlot::yAxis](#)) as value axis.

Normally, the key and value axes are set in the constructor of the plottable (or [QCustomPlot::addGraph](#) when working with QCPGraphs through the dedicated graph interface).

See also

[setValueAxis](#)

10.457.4.15 `setLineStyle()`

```
void QCPPolarGraph::setLineStyle (
    LineStyle ls )
```

Sets how the single data points are connected in the plot. For scatter-only plots, set *ls* to [lsNone](#) and [setScatterStyle](#) to the desired scatter style.

See also

[setScatterStyle](#)

10.457.4.16 `setName()`

```
void QCPPolarGraph::setName (
    const QString & name )
```

The name is the textual representation of this plottable as it is displayed in the legend ([QCPLegend](#)). It may contain any UTF-8 characters, including newlines.

10.457.4.17 `setPen()`

```
void QCPPolarGraph::setPen (
    const QPen & pen )
```

The pen is used to draw basic lines that make up the plottable representation in the plot.

For example, the [QCPGraph](#) subclass draws its graph lines with this pen.

See also

[setBrush](#)

10.457.4.18 `setScatterStyle()`

```
void QCPPolarGraph::setScatterStyle (
    const QCPScatterStyle & style )
```

Sets the visual appearance of single data points in the plot. If set to [QCPScatterStyle::ssNone](#), no scatter points are drawn (e.g. for line-only-plots with appropriate line style).

See also

[QCPScatterStyle](#), [setLineStyle](#)

10.457.4.19 `setSelectable()`

```
void QCPPolarGraph::setSelectable (
    QCP::SelectionType selectable )
```

Sets whether and to which granularity this plottable can be selected.

A selection can happen by clicking on the [QCustomPlot](#) surface (When [QCustomPlot::setInteractions](#) contains [QCP::iSelectPlottables](#)), by dragging a selection rect (When [QCustomPlot::setSelectionRectMode](#) is [QCP::srmSelect](#)), or programmatically by calling [setSelection](#).

See also

[setSelection](#), [QCP::SelectionType](#)

10.457.4.20 `setSelection()`

```
void QCPPolarGraph::setSelection (
    QCPDataSelection selection )
```

Sets which data ranges of this plottable are selected. Selected data ranges are drawn differently (e.g. color) in the plot. This can be controlled via the selection decorator (see [selectionDecorator](#)).

The entire selection mechanism for plottables is handled automatically when [QCustomPlot::setInteractions](#) contains [iSelectPlottables](#). You only need to call this function when you wish to change the selection state programmatically.

Using [setSelectable](#) you can further specify for each plottable whether and to which granularity it is selectable. If *selection* is not compatible with the current [QCP::SelectionType](#) set via [setSelectable](#), the resulting selection will be adjusted accordingly (see [QCPDataSelection::enforceType](#)).

emits the `selectionChanged` signal when *selected* is different from the previous selection state.

See also

[setSelectable](#), [selectTest](#)

10.457.4.21 `setValueAxis()`

```
void QCPPolarGraph::setValueAxis (
    QCPPolarAxisRadial * axis )
```

The value axis of a plottable can be set to any axis of a [QCustomPlot](#), as long as it is orthogonal to the plottable's key axis. This function performs no checks to make sure this is the case. The typical mathematical choice is to use the x-axis ([QCustomPlot::xAxis](#)) as key axis and the y-axis ([QCustomPlot::yAxis](#)) as value axis.

Normally, the key and value axes are set in the constructor of the plottable (or [QCustomPlot::addGraph](#) when working with QCPGraphs through the dedicated graph interface).

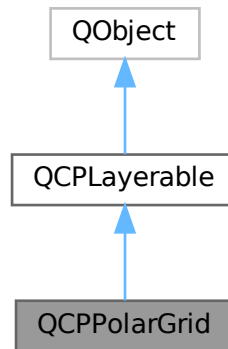
See also

[setKeyAxis](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

Inheritance diagram for QCPPolarGrid:



- enum **GridType** { **gtAngular** = 0x01 , **gtRadial** = 0x02 , **gtAll** = 0xFF , **gtNone** = 0x00 }

- **QCPPolarGrid** (**QCPPolarAxisAngular** *parentAxis)
- **QCPPolarAxisRadial** * **radialAxis** () const
- GridTypes **type** () const
- GridTypes **subGridType** () const
- bool **antialiasedSubGrid** () const
- bool **antialiasedZeroLine** () const
- QPen **angularPen** () const
- QPen **angularSubGridPen** () const
- QPen **radialPen** () const

- QPen **radialSubGridPen** () const
- QPen **radialZeroLinePen** () const
- void **setRadialAxis** (QCPPolarAxisRadial *axis)
- void **setType** (GridTypes type)
- void **setSubGridType** (GridTypes type)
- void **setAntialiasedSubGrid** (bool enabled)
- void **setAntialiasedZeroLine** (bool enabled)
- void **setAngularPen** (const QPen &pen)
- void **setAngularSubGridPen** (const QPen &pen)
- void **setRadialPen** (const QPen &pen)
- void **setRadialSubGridPen** (const QPen &pen)
- void **setRadialZeroLinePen** (const QPen &pen)

Public Member Functions inherited from QCPLayerable

- QCPLayerable (QCustomPlot *plot, QString targetLayer=QString(), QCPLayerable *parentLayerable=nullptr)
- bool **visible** () const
- QCustomPlot * **parentPlot** () const
- QCPLayerable * **parentLayerable** () const
- QCPLayer * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** (QCPLayer *layer)
- bool **setLayer** (const QString &layerName)
- void **setAntialiased** (bool enabled)
- virtual double **selectTest** (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const
- bool **realVisibility** () const

Protected Member Functions

- virtual void **applyDefaultAntialiasingHint** (QCPPainter *painter) const Q_DECL_OVERRIDE
- virtual void **draw** (QCPPainter *painter) Q_DECL_OVERRIDE
- void **drawRadialGrid** (QCPPainter *painter, const QPointF ¢er, const QVector< double > &coords, const QPen &pen, const QPen &zeroPen=Qt::NoPen)
- void **drawAngularGrid** (QCPPainter *painter, const QPointF ¢er, double radius, const QVector< QPointF > &ticksCosSin, const QPen &pen)

Protected Member Functions inherited from QCPLayerable

- virtual void **parentPlotInitialized** (QCustomPlot *parentPlot)
- virtual QCP::Interaction **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void **selectEvent** (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mousePressEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **mouseMoveEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseReleaseEvent** (QMouseEvent *event, const QPointF &startPos)
- virtual void **mouseDoubleClickEvent** (QMouseEvent *event, const QVariant &details)
- virtual void **wheelEvent** (QWheelEvent *event)
- void **initializeParentPlot** (QCustomPlot *parentPlot)
- void **setParentLayerable** (QCPLayerable *parentLayerable)
- bool **moveToLayer** (QCPLayer *layer, bool prepend)
- void **applyAntialiasingHint** (QCPPainter *painter, bool localAntialiased, QCP::AntialiasedElement overrideElement) const

Protected Attributes

- GridTypes **mType**
- GridTypes **mSubGridType**
- bool **mAntialiasedSubGrid**
- bool **mAntialiasedZeroLine**
- QPen **mAngularPen**
- QPen **mAngularSubGridPen**
- QPen **mRadialPen**
- QPen **mRadialSubGridPen**
- QPen **mRadialZeroLinePen**
- [QCPPolarAxisAngular](#) * **mParentAxis**
- QPointer< [QCPPolarAxisRadial](#) > **mRadialAxis**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.458.1 Detailed Description

The grid in both angular and radial dimensions for polar plots.

Warning

In this [QCustomPlot](#) version, polar plots are a tech preview. Expect documentation and functionality to be incomplete, as well as changing public interfaces in the future.

10.458.2 Member Enumeration Documentation

10.458.2.1 GridType

```
enum QCPPolarGrid::GridType
```

TODO

10.458.3 Constructor & Destructor Documentation

10.458.3.1 QCPPolarGrid()

```
QCPPolarGrid::QCPPolarGrid (
    QCPPolarAxisAngular * parentAxis ) [explicit]
```

Creates a [QCPPolarGrid](#) instance and sets default values.

You shouldn't instantiate grids on their own, since every axis brings its own grid.

10.458.4 Member Function Documentation

10.458.4.1 applyDefaultAntialiasingHint()

```
void QCPPolarGrid::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

10.458.4.2 draw()

```
void QCPPolarGrid::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPLayerable](#).

10.458.4.3 setAngularPen()

```
void QCPPolarGrid::setAngularPen (
    const QPen & pen )
```

Sets the pen with which (major) grid lines are drawn.

10.458.4.4 setAngularSubGridPen()

```
void QCPPolarGrid::setAngularSubGridPen (
    const QPen & pen )
```

Sets the pen with which sub grid lines are drawn.

10.458.4.5 setAntialiasedSubGrid()

```
void QCPPolarGrid::setAntialiasedSubGrid (
    bool enabled )
```

Sets whether sub grid lines are drawn antialiased.

10.458.4.6 setAntialiasedZeroLine()

```
void QCPPolarGrid::setAntialiasedZeroLine (
    bool enabled )
```

Sets whether zero lines are drawn antialiased.

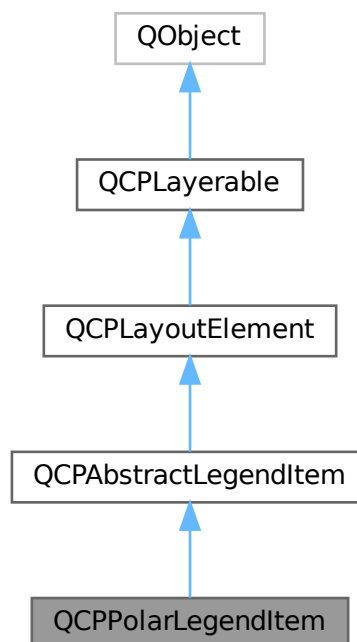
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.459 QCPPolarLegendItem Class Reference

A legend item for polar plots.

Inheritance diagram for QCPPolarLegendItem:



Public Member Functions inherited from QCPLayoutElement

- [QCPLayoutElement](#) ([QCustomPlot](#) *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- [QRect](#) [rect](#) () const
- [QRect](#) [outerRect](#) () const
- [QMargins](#) [margins](#) () const
- [QMargins](#) [minimumMargins](#) () const
- [QCP::MarginSides](#) [autoMargins](#) () const
- [QSize](#) [minimumSize](#) () const
- [QSize](#) [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) ([QCP::MarginSide](#) side) const
- [QHash](#)< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const [QRect](#) &[rect](#))
- void [setMargins](#) (const [QMargins](#) &margins)
- void [setMinimumMargins](#) (const [QMargins](#) &margins)
- void [setAutoMargins](#) ([QCP::MarginSides](#) sides)
- void [setMinimumSize](#) (const [QSize](#) &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const [QSize](#) &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) ([QCP::MarginSides](#) sides, [QCPMarginGroup](#) *group)
- virtual void [update](#) ([UpdatePhase](#) phase)
- virtual [QSize](#) [maximumOuterSizeHint](#) () const
- virtual [QList](#)< [QCPLayoutElement](#) * > [elements](#) (bool recursive) const

Public Member Functions inherited from QCPLayerable

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool [visible](#) () const
- [QCustomPlot](#) * [parentPlot](#) () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * [layer](#) () const
- bool [antialiased](#) () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const [QString](#) &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual [QSize](#) [minimumOuterSizeHint](#) () const Q_DECL_OVERRIDE
- [QPen](#) [getIconBorderPen](#) () const
- [QColor](#) [getTextColor](#) () const
- [QFont](#) [getFont](#) () const

Protected Member Functions inherited from [QCPAbstractLegendItem](#)

- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual int [calculateAutoMargin](#) ([QCP::MarginSide](#) side)
- virtual void [layoutChanged](#) ()
- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QCPPolarGraph](#) * [mPolarGraph](#)

Protected Attributes inherited from [QCPAbstractLegendItem](#)

- [QCPLegend](#) * [mParentLegend](#)
- QFont [mFont](#)
- QColor [mTextColor](#)
- QFont [mSelectedFont](#)
- QColor [mSelectedTextColor](#)
- bool [mSelectable](#)
- bool [mSelected](#)

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * [mParentLayout](#)
- QSize [mMinimumSize](#)
- QSize [mMaximumSize](#)
- [SizeConstraintRect](#) [mSizeConstraintRect](#)
- QRect [mRect](#)
- QRect [mOuterRect](#)
- QMargins [mMargins](#)
- QMargins [mMinimumMargins](#)
- QCP::MarginSides [mAutoMargins](#)
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > [mMarginGroups](#)

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer](#)< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Public Types inherited from [QCPLayoutElement](#)

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

Signals inherited from [QCPAbstractLegendItem](#)

- void [selectionChanged](#) (bool selected)
- void [selectableChanged](#) (bool selectable)

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *newLayer)

10.459.1 Detailed Description

A legend item for polar plots.

Warning

In this [QCustomPlot](#) version, polar plots are a tech preview. Expect documentation and functionality to be incomplete, as well as changing public interfaces in the future.

10.459.2 Member Function Documentation

10.459.2.1 draw()

```
void QCPPolarLegendItem::draw (  
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractLegendItem](#).

10.459.2.2 `minimumOuterSizeHint()`

```
QSize QCPolarLegendItem::minimumOuterSizeHint ( ) const [protected], [virtual]
```

Returns the suggested minimum size this layout element (the [outerRect](#)) may be compressed to, if no manual minimum size is set.

if a minimum size ([setMinimumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through `QCPLayout::getFinalMinimumOuterSize`) to determine the minimum allowed size of this layout element.

A manual minimum size is considered set if it is non-zero.

The default implementation simply returns the sum of the horizontal margins for the width and the sum of the vertical margins for the height. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented from [QCPLayoutElement](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.460 `QCPRange` Class Reference

Represents the range an axis is encompassing.

Public Member Functions

- [QCPRange](#) ()
- [QCPRange](#) (double lower, double upper)
- bool **operator==** (const [QCPRange](#) &other) const
- bool **operator!=** (const [QCPRange](#) &other) const
- [QCPRange](#) & **operator+=** (const double &value)
- [QCPRange](#) & **operator-=** (const double &value)
- [QCPRange](#) & **operator*=** (const double &value)
- [QCPRange](#) & **operator/=** (const double &value)
- double [size](#) () const
- double [center](#) () const
- void [normalize](#) ()
- void [expand](#) (const [QCPRange](#) &otherRange)
- void [expand](#) (double includeCoord)
- [QCPRange](#) [expanded](#) (const [QCPRange](#) &otherRange) const
- [QCPRange](#) [expanded](#) (double includeCoord) const
- [QCPRange](#) [bounded](#) (double lowerBound, double upperBound) const
- [QCPRange](#) [sanitizedForLogScale](#) () const
- [QCPRange](#) [sanitizedForLinScale](#) () const
- bool [contains](#) (double value) const

Static Public Member Functions

- static bool [validRange](#) (double lower, double upper)
- static bool [validRange](#) (const [QCPRange](#) &range)

Public Attributes

- double **lower**
- double **upper**

Static Public Attributes

- static const double [minRange](#) = 1e-280
- static const double [maxRange](#) = 1e250

Friends

- const [QCPRange operator+](#) (const [QCPRange](#) &, double)
- const [QCPRange operator+](#) (double, const [QCPRange](#) &)
- const [QCPRange operator-](#) (const [QCPRange](#) &range, double value)
- const [QCPRange operator*](#) (const [QCPRange](#) &range, double value)
- const [QCPRange operator*](#) (double value, const [QCPRange](#) &range)
- const [QCPRange operator/](#) (const [QCPRange](#) &range, double value)

Related Symbols

(Note that these are not member symbols.)

- QDebug [operator<<](#) (QDebug d, const [QCPRange](#) &range)

10.460.1 Detailed Description

Represents the range an axis is encompassing.

contains a *lower* and *upper* double value and provides convenience input, output and modification functions.

See also

[QCPAxis::setRange](#)

10.460.2 Constructor & Destructor Documentation

10.460.2.1 QCPRange() [1/2]

```
QCPRange::QCPRange ( )
```

Constructs a range with *lower* and *upper* set to zero.

10.460.2.2 QCPRange() [2/2]

```
QCPRange::QCPRange (
    double lower,
    double upper )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Constructs a range with the specified *lower* and *upper* values.

The resulting range will be normalized (see [normalize](#)), so if *lower* is not numerically smaller than *upper*, they will be swapped.

10.460.3 Member Function Documentation

10.460.3.1 bounded()

```
QCPRange QCPRange::bounded (
    double lowerBound,
    double upperBound ) const
```

Returns this range, possibly modified to not exceed the bounds provided as *lowerBound* and *upperBound*. If possible, the size of the current range is preserved in the process.

If the range shall only be bounded at the lower side, you can set *upperBound* to [QCPRange::maxRange](#). If it shall only be bounded at the upper side, set *lowerBound* to [-QCPRange::maxRange](#).

10.460.3.2 center()

```
double QCPRange::center ( ) const [inline]
```

Returns the center of the range, i.e. $(upper+lower)*0.5$

10.460.3.3 contains()

```
bool QCPRange::contains (
    double value ) const [inline]
```

Returns true when *value* lies within or exactly on the borders of the range.

10.460.3.4 expand() [1/2]

```
void QCPRange::expand (
    const QCPRange & otherRange )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Expands this range such that *otherRange* is contained in the new range. It is assumed that both this range and *otherRange* are normalized (see [normalize](#)).

If this range contains NaN as lower or upper bound, it will be replaced by the respective bound of *otherRange*.

If *otherRange* is already inside the current range, this function does nothing.

See also

[expanded](#)

10.460.3.5 `expand()` [2/2]

```
void QCPRange::expand (
    double includeCoord )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Expands this range such that *includeCoord* is contained in the new range. It is assumed that this range is normalized (see [normalize](#)).

If this range contains NaN as lower or upper bound, the respective bound will be set to *includeCoord*.

If *includeCoord* is already inside the current range, this function does nothing.

See also

[expand](#)

10.460.3.6 `expanded()` [1/2]

```
QCPRange QCPRange::expanded (
    const QCPRange & otherRange ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns an expanded range that contains this and *otherRange*. It is assumed that both this range and *otherRange* are normalized (see [normalize](#)).

If this range contains NaN as lower or upper bound, the returned range's bound will be taken from *otherRange*.

See also

[expand](#)

10.460.3.7 `expanded()` [2/2]

```
QCPRange QCPRange::expanded (
    double includeCoord ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns an expanded range that includes the specified *includeCoord*. It is assumed that this range is normalized (see [normalize](#)).

If this range contains NaN as lower or upper bound, the returned range's bound will be set to *includeCoord*.

See also

[expand](#)

10.460.3.8 normalize()

```
void QCPRange::normalize ( ) [inline]
```

Makes sure *lower* is numerically smaller than *upper*. If this is not the case, the values are swapped.

10.460.3.9 operator*=()

```
QCPRange & QCPRange::operator*= (
    const double & value ) [inline]
```

Multiplies both boundaries of the range by *value*.

10.460.3.10 operator+=()

```
QCPRange & QCPRange::operator+= (
    const double & value ) [inline]
```

Adds *value* to both boundaries of the range.

10.460.3.11 operator-=()

```
QCPRange & QCPRange::operator-= (
    const double & value ) [inline]
```

Subtracts *value* from both boundaries of the range.

10.460.3.12 operator/=()

```
QCPRange & QCPRange::operator/= (
    const double & value ) [inline]
```

Divides both boundaries of the range by *value*.

10.460.3.13 sanitizedForLinScale()

```
QCPRange QCPRange::sanitizedForLinScale ( ) const
```

Returns a sanitized version of the range. Sanitized means for linear scales, that *lower* will always be numerically smaller (or equal) to *upper*.

10.460.3.14 `sanitizedForLogScale()`

```
QCPRange QCPRange::sanitizedForLogScale ( ) const
```

Returns a sanitized version of the range. Sanitized means for logarithmic scales, that the range won't span the positive and negative sign domain, i.e. contain zero. Further *lower* will always be numerically smaller (or equal) to *upper*.

If the original range does span positive and negative sign domains or contains zero, the returned range will try to approximate the original range as good as possible. If the positive interval of the original range is wider than the negative interval, the returned range will only contain the positive interval, with lower bound set to *rangeFac* or *rangeFac * upper*, whichever is closer to zero. Same procedure is used if the negative interval is wider than the positive interval, this time by changing the *upper* bound.

10.460.3.15 `size()`

```
double QCPRange::size ( ) const [inline]
```

Returns the size of the range, i.e. *upper-lower*

10.460.3.16 `validRange()` [1/2]

```
bool QCPRange::validRange (
    const QCPRange & range ) [static]
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts. Checks, whether the specified range is within valid bounds, which are defined as [QCPRange::maxRange](#) and [QCPRange::minRange](#). A valid range means:

- range bounds within -maxRange and maxRange
- range size above minRange
- range size below maxRange

10.460.3.17 `validRange()` [2/2]

```
bool QCPRange::validRange (
    double lower,
    double upper ) [static]
```

Checks, whether the specified range is within valid bounds, which are defined as [QCPRange::maxRange](#) and [QCPRange::minRange](#). A valid range means:

- range bounds within -maxRange and maxRange
- range size above minRange
- range size below maxRange

10.460.4 Friends And Related Symbol Documentation

10.460.4.1 `operator*` [1/2]

```
const QCPRange operator* (  
    const QCPRange & range,  
    double value ) [friend]
```

Multiplies both boundaries of the range by *value*.

10.460.4.2 `operator*` [2/2]

```
const QCPRange operator* (  
    double value,  
    const QCPRange & range ) [friend]
```

Multiplies both boundaries of the range by *value*.

10.460.4.3 `operator+` [1/2]

```
const QCPRange operator+ (  
    const QCPRange & range,  
    double value ) [friend]
```

Adds *value* to both boundaries of the range.

10.460.4.4 `operator+` [2/2]

```
const QCPRange operator+ (  
    double value,  
    const QCPRange & range ) [friend]
```

Adds *value* to both boundaries of the range.

10.460.4.5 `operator-`

```
const QCPRange operator- (  
    const QCPRange & range,  
    double value ) [friend]
```

Subtracts *value* from both boundaries of the range.

10.460.4.6 `operator/`

```
const QCPRange operator/ (  
    const QCPRange & range,  
    double value ) [friend]
```

Divides both boundaries of the range by *value*.

10.460.4.7 operator<<()

```
QDebug operator<< (
    QDebug d,
    const QCPRange & range ) [related]
```

Prints *range* in a human readable format to the qDebug output.

10.460.5 Member Data Documentation

10.460.5.1 maxRange

```
const double QCPRange::maxRange = 1e250 [static]
```

Maximum values (negative and positive) the range will accept in range-changing functions. Larger absolute values would cause errors due to the 11-bit exponent of double precision numbers, corresponding to a maximum magnitude of roughly 1e308.

Warning

Do not use this constant to indicate "arbitrarily large" values in plotting logic (as values that will appear in the plot)! It is intended only as a bound to compare against, e.g. to prevent axis ranges from obtaining overflowing ranges.

See also

[validRange](#), [minRange](#)

10.460.5.2 minRange

```
const double QCPRange::minRange = 1e-280 [static]
```

Minimum range size (*upper - lower*) the range changing functions will accept. Smaller intervals would cause errors due to the 11-bit exponent of double precision numbers, corresponding to a minimum magnitude of roughly 1e-308.

Warning

Do not use this constant to indicate "arbitrarily small" values in plotting logic (as values that will appear in the plot)! It is intended only as a bound to compare against, e.g. to prevent axis ranges from obtaining underflowing ranges.

See also

[validRange](#), [maxRange](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.461 QCPScatterStyle Class Reference

Represents the visual appearance of scatter points.

Public Types

- enum [ScatterProperty](#) {
[spNone](#) = 0x00 , [spPen](#) = 0x01 , [spBrush](#) = 0x02 , [spSize](#) = 0x04 ,
[spShape](#) = 0x08 , [spAll](#) = 0xFF }
- enum [ScatterShape](#) {
[ssNone](#) , [ssDot](#) , [ssCross](#) , [ssPlus](#) ,
[ssCircle](#) , [ssDisc](#) , [ssSquare](#) , [ssDiamond](#) ,
[ssStar](#) , [ssTriangle](#) , [ssTriangleInverted](#) , [ssCrossSquare](#) ,
[ssPlusSquare](#) , [ssCrossCircle](#) , [ssPlusCircle](#) , [ssPeace](#) ,
[ssPixmap](#) , [ssCustom](#) }

Public Member Functions

- [QCPScatterStyle](#) ()
- [QCPScatterStyle](#) ([ScatterShape](#) shape, double size=6)
- [QCPScatterStyle](#) ([ScatterShape](#) shape, const QColor &color, double size)
- [QCPScatterStyle](#) ([ScatterShape](#) shape, const QColor &color, const QColor &fill, double size)
- [QCPScatterStyle](#) ([ScatterShape](#) shape, const QPen &pen, const QBrush &brush, double size)
- [QCPScatterStyle](#) (const QPixmap &pixmap)
- [QCPScatterStyle](#) (const QPainterPath &customPath, const QPen &pen, const QBrush &brush=Qt::NoBrush, double size=6)
- double **size** () const
- [ScatterShape](#) **shape** () const
- QPen **pen** () const
- QBrush **brush** () const
- QPixmap **pixmap** () const
- QPainterPath **customPath** () const
- void [setFromOther](#) (const [QCPScatterStyle](#) &other, ScatterProperties properties)
- void [setSize](#) (double size)
- void [setShape](#) ([ScatterShape](#) shape)
- void [setPen](#) (const QPen &pen)
- void [setBrush](#) (const QBrush &brush)
- void [setPixmap](#) (const QPixmap &pixmap)
- void [setCustomPath](#) (const QPainterPath &customPath)
- bool [isNone](#) () const
- bool [isPenDefined](#) () const
- void [undefinePen](#) ()
- void [applyTo](#) ([QCPPainter](#) *painter, const QPen &defaultPen) const
- void [drawShape](#) ([QCPPainter](#) *painter, const QPointF &pos) const
- void [drawShape](#) ([QCPPainter](#) *painter, double x, double y) const

Protected Attributes

- double **mSize**
- [ScatterShape](#) **mShape**
- QPen **mPen**
- QBrush **mBrush**
- QPixmap **mPixmap**
- QPainterPath **mCustomPath**
- bool **mPenDefined**

10.461.1 Detailed Description

Represents the visual appearance of scatter points.

This class holds information about shape, color and size of scatter points. In plottables like [QCPGraph](#) it is used to store how scatter points shall be drawn. For example, [QCPGraph::setScatterStyle](#) takes a [QCPScatterStyle](#) instance.

A scatter style consists of a shape ([setShape](#)), a line color ([setPen](#)) and possibly a fill ([setBrush](#)), if the shape provides a fillable area. Further, the size of the shape can be controlled with [setSize](#).

10.461.2 Specifying a scatter style

You can set all these configurations either by calling the respective functions on an instance:

Or you can use one of the various constructors that take different parameter combinations, making it easy to specify a scatter style in a single call, like so:

10.461.3 Leaving the color/pen up to the plottable

There are two constructors which leave the pen undefined: [QCPScatterStyle\(\)](#) and [QCPScatterStyle\(ScatterShape shape, double size\)](#). If those constructors are used, a call to [isPenDefined](#) will return false. It leads to scatter points that inherit the pen from the plottable that uses the scatter style. Thus, if such a scatter style is passed to [QCPGraph](#), the line color of the graph ([QCPGraph::setPen](#)) will be used by the scatter points. This makes it very convenient to set up typical scatter settings:

Notice that it wasn't even necessary to explicitly call a [QCPScatterStyle](#) constructor. This works because [QCPScatterStyle](#) provides a constructor that can transform a [ScatterShape](#) directly into a [QCPScatterStyle](#) instance (that's the [QCPScatterStyle\(ScatterShape shape, double size\)](#) constructor with a default for *size*). In those cases, C++ allows directly supplying a [ScatterShape](#), where actually a [QCPScatterStyle](#) is expected.

10.461.4 Custom shapes and pixmaps

[QCPScatterStyle](#) supports drawing custom shapes and arbitrary pixmaps as scatter points.

For custom shapes, you can provide a [QPainterPath](#) with the desired shape to the [setCustomPath](#) function or call the constructor that takes a painter path. The scatter shape will automatically be set to [ssCustom](#).

For pixmaps, you call [setPixmap](#) with the desired [QPixmap](#). Alternatively you can use the constructor that takes a [QPixmap](#). The scatter shape will automatically be set to [ssPixmap](#). Note that [setSize](#) does not influence the appearance of the pixmap.

10.461.5 Member Enumeration Documentation

10.461.5.1 ScatterProperty

```
enum QCPScatterStyle::ScatterProperty
```

Represents the various properties of a scatter style instance. For example, this enum is used to specify which properties of [QCPSelectionDecorator::setScatterStyle](#) will be used when highlighting selected data points.

Specific scatter properties can be transferred between [QCPScatterStyle](#) instances via [setFromOther](#).

Enumerator

spNone	0x00 None
spPen	0x01 The pen property, see setPen
spBrush	0x02 The brush property, see setBrush
spSize	0x04 The size property, see setSize
spShape	0x08 The shape property, see setShape
spAll	0xFF All properties

10.461.5.2 ScatterShape

```
enum QCPScatterStyle::ScatterShape
```

Defines the shape used for scatter points.

On plottables/items that draw scatters, the sizes of these visualizations (with exception of [ssDot](#) and [ssPixmap](#)) can be controlled with the [setSize](#) function. Scatters are drawn with the pen and brush specified with [setPen](#) and [setBrush](#).

Enumerator

ssNone	no scatter symbols are drawn (e.g. in QCPGraph , data only represented with lines)
ssDot	<code>\enumimage{ssDot.png}</code> a single pixel (use ssDisc or ssCircle if you want a round shape with a certain radius)
ssCross	<code>\enumimage{ssCross.png}</code> a cross
ssPlus	<code>\enumimage{ssPlus.png}</code> a plus
ssCircle	<code>\enumimage{ssCircle.png}</code> a circle
ssDisc	<code>\enumimage{ssDisc.png}</code> a circle which is filled with the pen's color (not the brush as with ssCircle)
ssSquare	<code>\enumimage{ssSquare.png}</code> a square
ssDiamond	<code>\enumimage{ssDiamond.png}</code> a diamond
ssStar	<code>\enumimage{ssStar.png}</code> a star with eight arms, i.e. a combination of cross and plus
ssTriangle	<code>\enumimage{ssTriangle.png}</code> an equilateral triangle, standing on baseline
ssTriangleInverted	<code>\enumimage{ssTriangleInverted.png}</code> an equilateral triangle, standing on corner
ssCrossSquare	<code>\enumimage{ssCrossSquare.png}</code> a square with a cross inside
ssPlusSquare	<code>\enumimage{ssPlusSquare.png}</code> a square with a plus inside
ssCrossCircle	<code>\enumimage{ssCrossCircle.png}</code> a circle with a cross inside
ssPlusCircle	<code>\enumimage{ssPlusCircle.png}</code> a circle with a plus inside
ssPeace	<code>\enumimage{ssPeace.png}</code> a circle, with one vertical and two downward diagonal lines
ssPixmap	a custom pixmap specified by setPixmap , centered on the data point coordinates
ssCustom	custom painter operations are performed per scatter (As QPainterPath , see setCustomPath)

10.461.6 Constructor & Destructor Documentation

10.461.6.1 QCPScatterStyle() [1/7]

```
QCPScatterStyle::QCPScatterStyle ( )
```

Creates a new [QCPScatterStyle](#) instance with size set to 6. No shape, pen or brush is defined.

Since the pen is undefined ([isPenDefined](#) returns false), the scatter color will be inherited from the plottable that uses this scatter style.

10.461.6.2 QCPScatterStyle() [2/7]

```
QCPScatterStyle::QCPScatterStyle (
    ScatterShape shape,
    double size = 6 )
```

Creates a new [QCPScatterStyle](#) instance with shape set to *shape* and size to *size*. No pen or brush is defined.

Since the pen is undefined ([isPenDefined](#) returns false), the scatter color will be inherited from the plottable that uses this scatter style.

10.461.6.3 QCPScatterStyle() [3/7]

```
QCPScatterStyle::QCPScatterStyle (
    ScatterShape shape,
    const QColor & color,
    double size )
```

Creates a new [QCPScatterStyle](#) instance with shape set to *shape*, the pen color set to *color*, and size to *size*. No brush is defined, i.e. the scatter point will not be filled.

10.461.6.4 QCPScatterStyle() [4/7]

```
QCPScatterStyle::QCPScatterStyle (
    ScatterShape shape,
    const QColor & color,
    const QColor & fill,
    double size )
```

Creates a new [QCPScatterStyle](#) instance with shape set to *shape*, the pen color set to *color*, the brush color to *fill* (with a solid pattern), and size to *size*.

10.461.6.5 QCPScatterStyle() [5/7]

```
QCPScatterStyle::QCPScatterStyle (
    ScatterShape shape,
    const QPen & pen,
    const QBrush & brush,
    double size )
```

Creates a new [QCPScatterStyle](#) instance with shape set to *shape*, the pen set to *pen*, the brush to *brush*, and size to *size*.

Warning

In some cases it might be tempting to directly use a pen style like `Qt::NoPen` as *pen* and a color like `Qt::blue` as *brush*. Notice however, that the corresponding call `QCPScatterStyle(QCPScatterShape::ssCircle, Qt::NoPen, Qt::blue, 5)` doesn't necessarily lead C++ to use this constructor in some cases, but might mistake `Qt::NoPen` for a `QColor` and use the `QCPScatterStyle(ScatterShape shape, const QColor &color, const QColor &fill, double size)` constructor instead (which will lead to an unexpected look of the scatter points). To prevent this, be more explicit with the parameter types. For example, use `QBrush(Qt::blue)` instead of just `Qt::blue`, to clearly point out to the compiler that this constructor is wanted.

10.461.6.6 QCPScatterStyle() [6/7]

```
QCPScatterStyle::QCPScatterStyle (
    const QPixmap & pixmap )
```

Creates a new [QCPScatterStyle](#) instance which will show the specified *pixmap*. The scatter shape is set to [ssPixmap](#).

10.461.6.7 QCPScatterStyle() [7/7]

```
QCPScatterStyle::QCPScatterStyle (
    const QPainterPath & customPath,
    const QPen & pen,
    const QBrush & brush = Qt::NoBrush,
    double size = 6 )
```

Creates a new [QCPScatterStyle](#) instance with a custom shape that is defined via *customPath*. The scatter shape is set to [ssCustom](#).

The custom shape line will be drawn with *pen* and filled with *brush*. The size has a slightly different meaning than for built-in scatter points: The custom path will be drawn scaled by a factor of *size/6.0*. Since the default *size* is 6, the custom path will appear in its original size by default. To for example double the size of the path, set *size* to 12.

10.461.7 Member Function Documentation

10.461.7.1 applyTo()

```
void QCPScatterStyle::applyTo (
    QCPPainter * painter,
    const QPen & defaultPen ) const
```

Applies the pen and the brush of this scatter style to *painter*. If this scatter style has an undefined pen ([isPenDefined](#)), sets the pen of *painter* to *defaultPen* instead.

This function is used by plottables (or any class that wants to draw scatters) just before a number of scatters with this style shall be drawn with the *painter*.

See also

[drawShape](#)

10.461.7.2 drawShape() [1/2]

```
void QCPScatterStyle::drawShape (
    QCPPainter * painter,
    const QPointF & pos ) const
```

Draws the scatter shape with *painter* at position *pos*.

This function does not modify the pen or the brush on the painter, as [applyTo](#) is meant to be called before scatter points are drawn with [drawShape](#).

See also

[applyTo](#)

10.461.7.3 drawShape() [2/2]

```
void QCPScatterStyle::drawShape (
    QCPPainter * painter,
    double x,
    double y ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts. Draws the scatter shape with *painter* at position *x* and *y*.

10.461.7.4 isNone()

```
bool QCPScatterStyle::isNone ( ) const [inline]
```

Returns whether the scatter shape is [ssNone](#).

See also

[setShape](#)

10.461.7.5 isPenDefined()

```
bool QCPScatterStyle::isPenDefined ( ) const [inline]
```

Returns whether a pen has been defined for this scatter style.

The pen is undefined if a constructor is called that does not carry *pen* as parameter. Those are [QCPScatterStyle\(\)](#) and [QCPScatterStyle\(ScatterShape shape, double size\)](#). If the pen is undefined, the pen of the respective plottable will be used for drawing scatters.

If a pen was defined for this scatter style instance, and you now wish to undefine the pen, call [undefinePen](#).

See also

[setPen](#)

10.461.7.6 setBrush()

```
void QCPScatterStyle::setBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill scatter points to *brush*. Note that not all scatter shapes have fillable areas. For example, [ssPlus](#) does not while [ssCircle](#) does.

See also

[setPen](#)

10.461.7.7 setCustomPath()

```
void QCPScatterStyle::setCustomPath (
    const QPainterPath & customPath )
```

Sets the custom shape that will be drawn as scatter point to *customPath*.

The scatter shape is automatically set to [ssCustom](#).

10.461.7.8 setFromOther()

```
void QCPScatterStyle::setFromOther (
    const QCPScatterStyle & other,
    ScatterProperties properties )
```

Copies the specified *properties* from the *other* scatter style to this scatter style.

10.461.7.9 setPen()

```
void QCPScatterStyle::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw scatter points to *pen*.

If the pen was previously undefined (see [isPenDefined](#)), the pen is considered defined after a call to this function, even if *pen* is `Qt::NoPen`. If you have defined a pen previously by calling this function and now wish to undefine the pen, call [undefinePen](#).

See also

[setBrush](#)

10.461.7.10 setPixmap()

```
void QCPScatterStyle::setPixmap (
    const QPixmap & pixmap )
```

Sets the pixmap that will be drawn as scatter point to *pixmap*.

Note that [setSize](#) does not influence the appearance of the pixmap.

The scatter shape is automatically set to [ssPixmap](#).

10.461.7.11 setShape()

```
void QCPScatterStyle::setShape (
    QCPScatterStyle::ScatterShape shape )
```

Sets the shape to *shape*.

Note that the calls [setPixmap](#) and [setCustomPath](#) automatically set the shape to [ssPixmap](#) and [ssCustom](#), respectively.

See also

[setSize](#)

10.461.7.12 setSize()

```
void QCPScatterStyle::setSize (
    double size )
```

Sets the size (pixel diameter) of the drawn scatter points to *size*.

See also

[setShape](#)

10.461.7.13 undefinePen()

```
void QCPScatterStyle::undefinePen ( )
```

Sets this scatter style to have an undefined pen (see [isPenDefined](#) for what an undefined pen implies).

A call to [setPen](#) will define a pen.

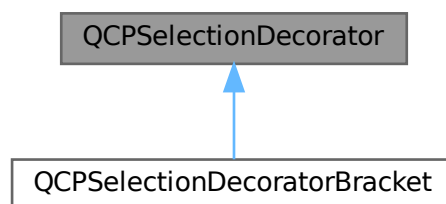
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.462 QCPSelectionDecorator Class Reference

Controls how a plottable's data selection is drawn.

Inheritance diagram for QCPSelectionDecorator:



Friends

- class [QCPAbstractPlottable](#)

10.462.1 Detailed Description

Controls how a plottable's data selection is drawn.

Each [QCPAbstractPlottable](#) instance has one [QCPSelectionDecorator](#) (accessible via [QCPAbstractPlottable::selectionDecorator](#)) and uses it when drawing selected segments of its data.

The selection decorator controls both pen ([setPen](#)) and brush ([setBrush](#)), as well as the scatter style ([setScatterStyle](#)) if the plottable draws scatters. Since a [QCPScatterStyle](#) is itself composed of different properties such as color shape and size, the decorator allows specifying exactly which of those properties shall be used for the selected data point, via [setUsedScatterProperties](#).

A [QCPSelectionDecorator](#) subclass instance can be passed to a plottable via [QCPAbstractPlottable::setSelectionDecorator](#), allowing greater customizability of the appearance of selected segments.

Use [copyFrom](#) to easily transfer the settings of one decorator to another one. This is especially useful since plottables take ownership of the passed selection decorator, and thus the same decorator instance can not be passed to multiple plottables.

Selection decorators can also themselves perform drawing operations by reimplementing [drawDecoration](#), which is called by the plottable's draw method. The base class [QCPSelectionDecorator](#) does not make use of this however. For example, [QCPSelectionDecoratorBracket](#) draws brackets around selected data segments.

10.462.2 Constructor & Destructor Documentation

10.462.2.1 QCPSelectionDecorator()

```
QCPSelectionDecorator::QCPSelectionDecorator ( )
```

Creates a new [QCPSelectionDecorator](#) instance with default values

10.462.3 Member Function Documentation

10.462.3.1 applyBrush()

```
void QCPSelectionDecorator::applyBrush (
    QCPPainter * painter ) const
```

Sets the brush of *painter* to the brush of this selection decorator.

See also

[applyPen](#), [getFinalScatterStyle](#)

10.462.3.2 applyPen()

```
void QCPSelectionDecorator::applyPen (
    QCPPainter * painter ) const
```

Sets the pen of *painter* to the pen of this selection decorator.

See also

[applyBrush](#), [getFinalScatterStyle](#)

10.462.3.3 copyFrom()

```
void QCPSelectionDecorator::copyFrom (
    const QCPSelectionDecorator * other ) [virtual]
```

Copies all properties (e.g. color, fill, scatter style) of the *other* selection decorator to this selection decorator.

10.462.3.4 drawDecoration()

```
void QCPSelectionDecorator::drawDecoration (
    QCPPainter * painter,
    QCPDataSelection selection ) [virtual]
```

This method is called by all plottables' draw methods to allow custom selection decorations to be drawn. Use the passed *painter* to perform the drawing operations. *selection* carries the data selection for which the decoration shall be drawn.

The default base class implementation of [QCPSelectionDecorator](#) has no special decoration, so this method does nothing.

Reimplemented in [QCPSelectionDecoratorBracket](#).

10.462.3.5 getFinalScatterStyle()

```
QCPScatterStyle QCPSelectionDecorator::getFinalScatterStyle (
    const QCPScatterStyle & unselectedStyle ) const
```

Returns the scatter style that the parent plottable shall use for selected scatter points. The plottable's original (unselected) scatter style must be passed as *unselectedStyle*. Depending on the setting of [setUsedScatterProperties](#), the returned scatter style is a mixture of this selection decorator's scatter style ([setScatterStyle](#)), and *unselectedStyle*.

See also

[applyPen](#), [applyBrush](#), [setScatterStyle](#)

10.462.3.6 setBrush()

```
void QCPSelectionDecorator::setBrush (
    const QBrush & brush )
```

Sets the brush that will be used by the parent plottable to draw selected data segments.

10.462.3.7 setPen()

```
void QCPSelectionDecorator::setPen (
    const QPen & pen )
```

Sets the pen that will be used by the parent plottable to draw selected data segments.

10.462.3.8 setScatterStyle()

```
void QCPSelectionDecorator::setScatterStyle (
    const QCPScatterStyle & scatterStyle,
    QCPScatterStyle::ScatterProperties usedProperties = QCPScatterStyle::spPen )
```

Sets the scatter style that will be used by the parent plottable to draw scatters in selected data segments.

usedProperties specifies which parts of the passed *scatterStyle* will be used by the plottable. The used properties can also be changed via [setUsedScatterProperties](#).

10.462.3.9 setUsedScatterProperties()

```
void QCPSelectionDecorator::setUsedScatterProperties (
    const QCPScatterStyle::ScatterProperties & properties )
```

Use this method to define which properties of the scatter style (set via [setScatterStyle](#)) will be used for selected data segments. All properties of the scatter style that are not specified in *properties* will remain as specified in the plottable's original scatter style.

See also

[QCPScatterStyle::ScatterProperty](#)

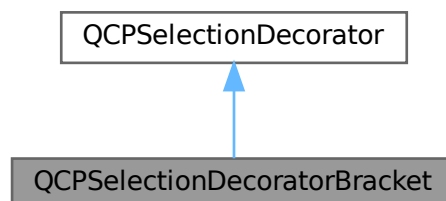
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.463 QCPSelectionDecoratorBracket Class Reference

A selection decorator which draws brackets around each selected data segment.

Inheritance diagram for QCPSelectionDecoratorBracket:



Public Member Functions inherited from QCPSelectionDecorator

- [QCPSelectionDecorator](#) ()
- QPen **pen** () const
- QBrush **brush** () const
- [QCPScatterStyle](#) **scatterStyle** () const
- QCPScatterStyle::ScatterProperties **usedScatterProperties** () const
- void **setPen** (const QPen &pen)
- void **setBrush** (const QBrush &brush)
- void **setScatterStyle** (const [QCPScatterStyle](#) &scatterStyle, QCPScatterStyle::ScatterProperties used←Properties=[QCPScatterStyle::spPen](#))
- void **setUsedScatterProperties** (const QCPScatterStyle::ScatterProperties &properties)
- void **applyPen** (QCPPainter *painter) const
- void **applyBrush** (QCPPainter *painter) const
- [QCPScatterStyle](#) **getFinalScatterStyle** (const [QCPScatterStyle](#) &unselectedStyle) const
- virtual void **copyFrom** (const [QCPSelectionDecorator](#) *other)

Protected Member Functions

- double **getTangentAngle** (const [QCPPlottableInterface1D](#) *interface1d, int dataIndex, int direction) const
- QPointF **getPixelCoordinates** (const [QCPPlottableInterface1D](#) *interface1d, int dataIndex) const

Protected Member Functions inherited from QCPSelectionDecorator

- virtual bool **registerWithPlottable** ([QCPAbstractPlottable](#) *plottable)

Protected Attributes

- QPen **mBracketPen**
- QBrush **mBracketBrush**
- int **mBracketWidth**
- int **mBracketHeight**
- [BracketStyle](#) **mBracketStyle**
- bool **mTangentToData**
- int **mTangentAverage**

Protected Attributes inherited from QCPSelectionDecorator

- QPen **mPen**
- QBrush **mBrush**
- [QCPScatterStyle](#) **mScatterStyle**
- QCPScatterStyle::ScatterProperties **mUsedScatterProperties**
- [QCPAbstractPlottable](#) * **mPlottable**

10.463.1 Detailed Description

A selection decorator which draws brackets around each selected data segment.

Additionally to the regular highlighting of selected segments via color, fill and scatter style, this [QCPSelectionDecorator](#) subclass draws markers at the begin and end of each selected data segment of the plottable.

The shape of the markers can be controlled with [setBracketStyle](#), [setBracketWidth](#) and [setBracketHeight](#). The color/fill can be controlled with [setBracketPen](#) and [setBracketBrush](#).

To introduce custom bracket styles, it is only necessary to subclass [QCPSelectionDecoratorBracket](#) and reimplement [drawBracket](#). The rest will be managed by the base class.

10.463.2 Member Enumeration Documentation

10.463.2.1 BracketStyle

enum [QCPSelectionDecoratorBracket::BracketStyle](#)

Defines which shape is drawn at the boundaries of selected data ranges.

Some of the bracket styles further allow specifying a height and/or width, see [setBracketHeight](#) and [setBracketWidth](#).

Enumerator

<code>bsSquareBracket</code>	A square bracket is drawn.
<code>bsHalfEllipse</code>	A half ellipse is drawn. The size of the ellipse is given by the bracket width/height properties.
<code>bsEllipse</code>	An ellipse is drawn. The size of the ellipse is given by the bracket width/height properties.
<code>bsPlus</code>	A plus is drawn.
<code>bsUserStyle</code>	Start custom bracket styles at this index when subclassing and reimplementing drawBracket .

10.463.3 Constructor & Destructor Documentation

10.463.3.1 QCPSelectionDecoratorBracket()

`QCPSelectionDecoratorBracket::QCPSelectionDecoratorBracket ( )`

Creates a new [QCPSelectionDecoratorBracket](#) instance with default values.

10.463.4 Member Function Documentation

10.463.4.1 drawBracket()

```
void QCPSelectionDecoratorBracket::drawBracket (
    QCPPainter * painter,
    int direction ) const [virtual]
```

Draws the bracket shape with *painter*. The parameter *direction* is either -1 or 1 and indicates whether the bracket shall point to the left or the right (i.e. is a closing or opening bracket, respectively).

The passed *painter* already contains all transformations that are necessary to position and rotate the bracket appropriately. Painting operations can be performed as if drawing upright brackets on flat data with horizontal key axis, with (0, 0) being the center of the bracket.

If you wish to subclass [QCPSelectionDecoratorBracket](#) in order to provide custom bracket shapes (see [QCPSelectionDecoratorBracket::bsUserStyle](#)), this is the method you should reimplement.

10.463.4.2 drawDecoration()

```
void QCPSelectionDecoratorBracket::drawDecoration (
    QCPPainter * painter,
    QCPDataSelection selection ) [virtual]
```

Draws the bracket decoration on the data points at the begin and end of each selected data segment given in *selection*.

It uses the method [drawBracket](#) to actually draw the shapes.

\see [baseclassmethod](#)

Reimplemented from [QCPSelectionDecorator](#).

10.463.4.3 setBracketBrush()

```
void QCPSelectionDecoratorBracket::setBracketBrush (
    const QBrush & brush )
```

Sets the brush that will be used to draw the brackets at the beginning and end of each selected data segment.

10.463.4.4 setBracketHeight()

```
void QCPSelectionDecoratorBracket::setBracketHeight (
    int height )
```

Sets the height of the drawn bracket. The height dimension is always perpendicular to the key axis of the data, or the tangent direction of the current data slope, if [setTangentToData](#) is enabled.

10.463.4.5 setBracketPen()

```
void QCPSelectionDecoratorBracket::setBracketPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the brackets at the beginning and end of each selected data segment.

10.463.4.6 setBracketStyle()

```
void QCPSelectionDecoratorBracket::setBracketStyle (
    QCPSelectionDecoratorBracket::BracketStyle style )
```

Sets the shape that the bracket/marker will have.

See also

[setBracketWidth](#), [setBracketHeight](#)

10.463.4.7 setBracketWidth()

```
void QCPSelectionDecoratorBracket::setBracketWidth (
    int width )
```

Sets the width of the drawn bracket. The width dimension is always parallel to the key axis of the data, or the tangent direction of the current data slope, if [setTangentToData](#) is enabled.

10.463.4.8 setTangentAverage()

```
void QCPSelectionDecoratorBracket::setTangentAverage (
    int pointCount )
```

Controls over how many data points the slope shall be averaged, when brackets shall be aligned with the data (if [setTangentToData](#) is true).

From the position of the bracket, *pointCount* points towards the selected data range will be taken into account. The smallest value of *pointCount* is 1, which is effectively equivalent to disabling [setTangentToData](#).

10.463.4.9 setTangentToData()

```
void QCPSelectionDecoratorBracket::setTangentToData (
    bool enabled )
```

Sets whether the brackets will be rotated such that they align with the slope of the data at the position that they appear in.

For noisy data, it might be more visually appealing to average the slope over multiple data points. This can be configured via [setTangentAverage](#).

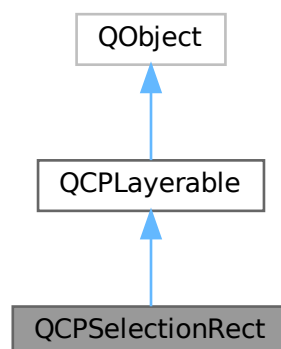
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

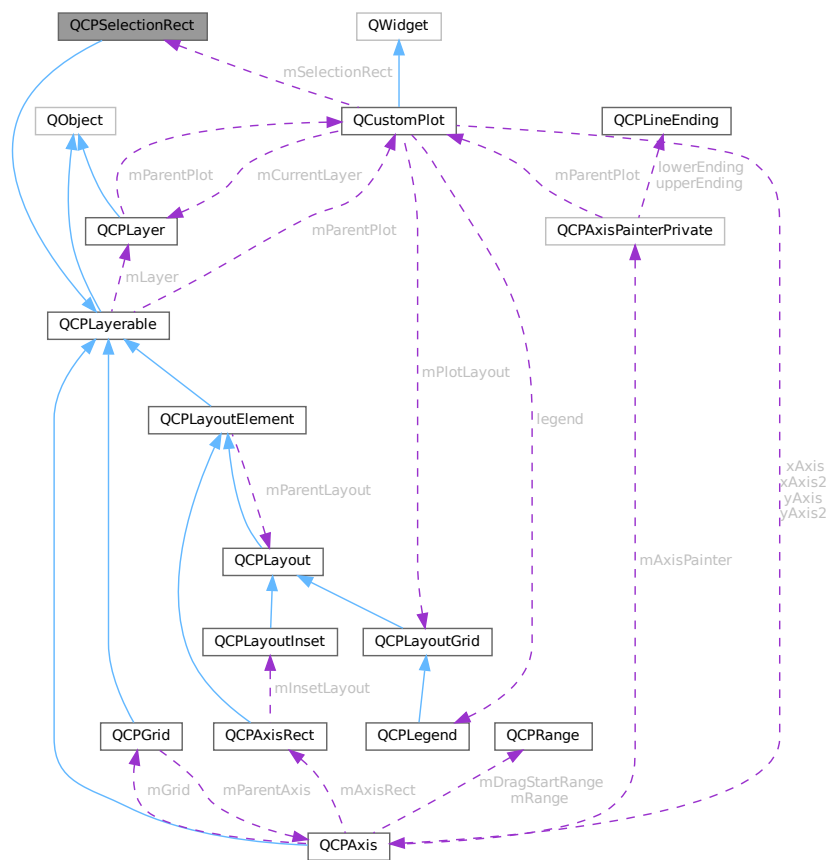
10.464 QCPSelectionRect Class Reference

Provides rect/rubber-band data selection and range zoom interaction.

Inheritance diagram for QCPSelectionRect:



Collaboration diagram for QCPSelectionRect:



Signals

- void **started** (QMouseEvent *event)
- void **changed** (const QRect &rect, QMouseEvent *event)
- void **canceled** (const QRect &rect, QInputEvent *event)
- void **accepted** (const QRect &rect, QMouseEvent *event)

Signals inherited from QCPLayerable

- void layerChanged (QCPLayer *newLayer)

Public Member Functions

- **QCPSelectionRect** (**QCustomPlot** *parentPlot)
- **QRect rect** () const
- **QCPSRange range** (const **QCPAxis** *axis) const
- **QPen pen** () const
- **QBrush brush** () const
- **bool isActive** () const
- **void setPen** (const **QPen** &pen)
- **void setBrush** (const **QBrush** &brush)
- **Q_SLOT void cancel** ()

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const [QString](#) &layerName)
- void **setAntialiased** (bool enabled)
- virtual double **selectTest** (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const
- bool **realVisibility** () const

Protected Member Functions

- virtual void **startSelection** ([QMouseEvent](#) *event)
- virtual void **moveSelection** ([QMouseEvent](#) *event)
- virtual void **endSelection** ([QMouseEvent](#) *event)
- virtual void **keyPressEvent** ([QKeyEvent](#) *event)
- virtual void **applyDefaultAntialiasingHint** ([QCPPainter](#) *painter) const [Q_DECL_OVERRIDE](#)
- virtual void **draw** ([QCPPainter](#) *painter) [Q_DECL_OVERRIDE](#)

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void **parentPlotInitialized** ([QCustomPlot](#) *parentPlot)
- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual [QRect](#) **clipRect** () const
- virtual void **selectEvent** ([QMouseEvent](#) *event, bool additive, const [QVariant](#) &details, bool *selectionState↔ Changed)
- virtual void **deselectEvent** (bool *selectionStateChanged)
- virtual void **mousePressEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **mouseMoveEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseReleaseEvent** ([QMouseEvent](#) *event, const [QPointF](#) &startPos)
- virtual void **mouseDoubleClickEvent** ([QMouseEvent](#) *event, const [QVariant](#) &details)
- virtual void **wheelEvent** ([QWheelEvent](#) *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- [QRect](#) **mRect**
- [QPen](#) **mPen**
- [QBrush](#) **mBrush**
- bool **mActive**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- [QPointer](#)< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)

10.464.1 Detailed Description

Provides rect/rubber-band data selection and range zoom interaction.

[QCPSelectionRect](#) is used by [QCustomPlot](#) when the [QCustomPlot::setSelectionRectMode](#) is not [QCP::srmNone](#). When the user drags the mouse across the plot, the current selection rect instance ([QCustomPlot::setSelectionRect](#)) is forwarded these events and makes sure an according rect shape is drawn. At the begin, during, and after completion of the interaction, it emits the corresponding signals [started](#), [changed](#), [canceled](#), and [accepted](#).

The [QCustomPlot](#) instance connects own slots to the current selection rect instance, in order to react to an accepted selection rect interaction accordingly.

[isActive](#) can be used to check whether the selection rect is currently active. An ongoing selection interaction can be cancelled programmatically via calling [cancel](#) at any time.

The appearance of the selection rect can be controlled via [setPen](#) and [setBrush](#).

If you wish to provide custom behaviour, e.g. a different visual representation of the selection rect ([QCPSelectionRect::draw](#)), you can subclass [QCPSelectionRect](#) and pass an instance of your subclass to [QCustomPlot::setSelectionRect](#).

10.464.2 Constructor & Destructor Documentation

10.464.2.1 QCPSelectionRect()

```
QCPSelectionRect::QCPSelectionRect (
    QCustomPlot * parentPlot ) [explicit]
```

Creates a new [QCPSelectionRect](#) instance. To make [QCustomPlot](#) use the selection rect instance, pass it to [QCustomPlot::setSelectionRect](#). *parentPlot* should be set to the same [QCustomPlot](#) widget.

10.464.3 Member Function Documentation

10.464.3.1 accepted

```
void QCPSelectionRect::accepted (
    const QRect & rect,
    QMouseEvent * event ) [signal]
```

This signal is emitted when the selection interaction was completed by the user releasing the mouse button.

Note that *rect* may have a negative width or height, if the selection is being dragged to the upper or left side of the selection rect origin.

10.464.3.2 `applyDefaultAntialiasingHint()`

```
void QCPSelectionRect::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Implements [QCPLayerable](#).

10.464.3.3 `cancel()`

```
void QCPSelectionRect::cancel ( )
```

If there is currently a selection interaction going on ([isActive](#)), the interaction is canceled. The selection rect will emit the [canceled](#) signal.

10.464.3.4 `canceled`

```
void QCPSelectionRect::canceled (
    const QRect & rect,
    QInputEvent * event ) [signal]
```

This signal is emitted when the selection interaction was cancelled. Note that *event* is `nullptr` if the selection interaction was cancelled programmatically, by a call to [cancel](#).

The user may cancel the selection interaction by pressing the escape key. In this case, *event* holds the respective input event.

Note that *rect* may have a negative width or height, if the selection is being dragged to the upper or left side of the selection rect origin.

10.464.3.5 `changed`

```
void QCPSelectionRect::changed (
    const QRect & rect,
    QMouseEvent * event ) [signal]
```

This signal is emitted while the selection rect interaction is ongoing and the *rect* has changed its size due to the user moving the mouse.

Note that *rect* may have a negative width or height, if the selection is being dragged to the upper or left side of the selection rect origin.

10.464.3.6 `draw()`

```
void QCPSelectionRect::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPLayerable](#).

10.464.3.7 isActive()

```
bool QCPSelectionRect::isActive ( ) const [inline]
```

Returns true if there is currently a selection going on, i.e. the user has started dragging a selection rect, but hasn't released the mouse button yet.

See also

[cancel](#)

10.464.3.8 range()

```
QCPRange QCPSelectionRect::range (
    const QCPAxis * axis ) const
```

A convenience function which returns the coordinate range of the provided *axis*, that this selection rect currently encompasses.

10.464.3.9 setBrush()

```
void QCPSelectionRect::setBrush (
    const QBrush & brush )
```

Sets the brush that will be used to fill the selection rect. By default the selection rect is not filled, i.e. *brush* is `Qt::NoBrush`.

See also

[setPen](#)

10.464.3.10 setPen()

```
void QCPSelectionRect::setPen (
    const QPen & pen )
```

Sets the pen that will be used to draw the selection rect outline.

See also

[setBrush](#)

10.464.3.11 started

```
void QCPSelectionRect::started (
    QMouseEvent * event ) [signal]
```

This signal is emitted when a selection rect interaction was initiated, i.e. the user just started dragging the selection rect with the mouse.

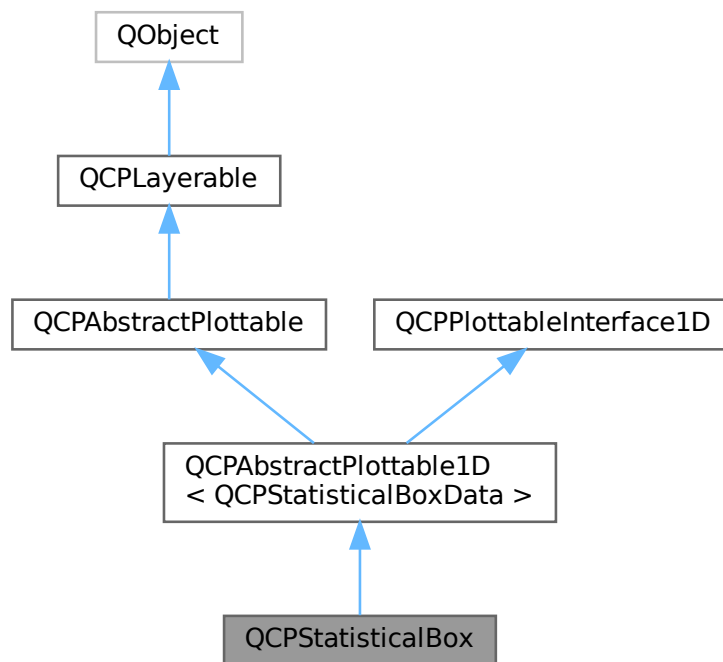
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

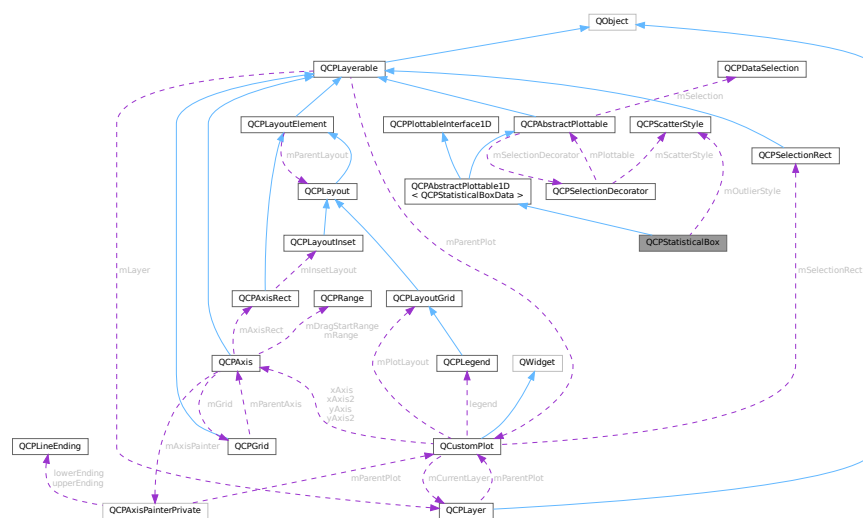
10.465 QCPStatisticalBox Class Reference

A plottable representing a single statistical box in a plot.

Inheritance diagram for QCPStatisticalBox:



Collaboration diagram for QCPStatisticalBox:



Public Member Functions

- [QCPStatisticalBox](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QSharedPointer](#)< [QCPStatisticalBoxDataContainer](#) > [data](#) () const
- double [width](#) () const
- double [whiskerWidth](#) () const
- [QPen](#) [whiskerPen](#) () const
- [QPen](#) [whiskerBarPen](#) () const
- bool [whiskerAntialiased](#) () const
- [QPen](#) [medianPen](#) () const
- [QCPScatterStyle](#) [outlierStyle](#) () const
- void [setData](#) ([QSharedPointer](#)< [QCPStatisticalBoxDataContainer](#) > data)
- void [setData](#) (const [QVector](#)< double > &keys, const [QVector](#)< double > &minimum, const [QVector](#)< double > &lowerQuartile, const [QVector](#)< double > &median, const [QVector](#)< double > &upperQuartile, const [QVector](#)< double > &maximum, bool alreadySorted=false)
- void [setWidth](#) (double width)
- void [setWhiskerWidth](#) (double width)
- void [setWhiskerPen](#) (const [QPen](#) &pen)
- void [setWhiskerBarPen](#) (const [QPen](#) &pen)
- void [setWhiskerAntialiased](#) (bool enabled)
- void [setMedianPen](#) (const [QPen](#) &pen)
- void [setOutlierStyle](#) (const [QCPScatterStyle](#) &style)
- void [addData](#) (const [QVector](#)< double > &keys, const [QVector](#)< double > &minimum, const [QVector](#)< double > &lowerQuartile, const [QVector](#)< double > &median, const [QVector](#)< double > &upperQuartile, const [QVector](#)< double > &maximum, bool alreadySorted=false)
- void [addData](#) (double key, double minimum, double lowerQuartile, double median, double upperQuartile, double maximum, const [QVector](#)< double > &outliers=[QVector](#)< double > ())
- virtual [QCPDataSelection](#) [selectTestRect](#) (const [QRectF](#) &rect, bool onlySelectable) const [Q_DECL_OVERRIDE](#)
- virtual double [selectTest](#) (const [QPointF](#) &pos, bool onlySelectable, [QVariant](#) *details=nullptr) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) [getKeyRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#)) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) [getValueRange](#) (bool &foundRange, [QCP::SignDomain](#) inSignDomain=[QCP::sdBoth](#), const [QCPRange](#) &inKeyRange=[QCPRange](#)()) const [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from [QCPAbstractPlottable1D](#)< [QCPStatisticalBoxData](#) >

- [QCPAbstractPlottable1D](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- virtual int [dataCount](#) () const [Q_DECL_OVERRIDE](#)
- virtual double [dataMainKey](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual double [dataSortKey](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual double [dataMainValue](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual [QCPRange](#) [dataValueRange](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual [QPointF](#) [dataPixelPosition](#) (int index) const [Q_DECL_OVERRIDE](#)
- virtual bool [sortKeysMainKey](#) () const [Q_DECL_OVERRIDE](#)
- virtual int [findBegin](#) (double sortKey, bool expandedRange=true) const [Q_DECL_OVERRIDE](#)
- virtual int [findEnd](#) (double sortKey, bool expandedRange=true) const [Q_DECL_OVERRIDE](#)
- virtual [QCPPlottableInterface1D](#) * [interface1D](#) () [Q_DECL_OVERRIDE](#)

Public Member Functions inherited from [QCPAbstractPlottable](#)

- [QCPAbstractPlottable](#) ([QCPAxis](#) *keyAxis, [QCPAxis](#) *valueAxis)
- [QString](#) **name** () const
- bool **antialiasedFill** () const
- bool **antialiasedScatters** () const
- [QPen](#) **pen** () const
- [QBrush](#) **brush** () const
- [QCPAxis](#) * **keyAxis** () const
- [QCPAxis](#) * **valueAxis** () const
- [QCP::SelectionType](#) **selectable** () const
- bool **selected** () const
- [QCPDataSelection](#) **selection** () const
- [QCPSelectionDecorator](#) * **selectionDecorator** () const
- void **setName** (const [QString](#) &name)
- void **setAntialiasedFill** (bool enabled)
- void **setAntialiasedScatters** (bool enabled)
- void **setPen** (const [QPen](#) &pen)
- void **setBrush** (const [QBrush](#) &brush)
- void **setKeyAxis** ([QCPAxis](#) *axis)
- void **setValueAxis** ([QCPAxis](#) *axis)
- Q_SLOT void **setSelectable** ([QCP::SelectionType](#) selectable)
- Q_SLOT void **setSelection** ([QCPDataSelection](#) selection)
- void **setSelectionDecorator** ([QCPSelectionDecorator](#) *decorator)
- void **coordsToPixels** (double key, double value, double &x, double &y) const
- const [QPointF](#) **coordsToPixels** (double key, double value) const
- void **pixelsToCoords** (double x, double y, double &key, double &value) const
- void **pixelsToCoords** (const [QPointF](#) &pixelPos, double &key, double &value) const
- void **rescaleAxes** (bool onlyEnlarge=false) const
- void **rescaleKeyAxis** (bool onlyEnlarge=false) const
- void **rescaleValueAxis** (bool onlyEnlarge=false, bool inKeyRange=false) const
- bool **addToLegend** ([QCPLegend](#) *legend)
- bool **addToLegend** ()
- bool **removeFromLegend** ([QCPLegend](#) *legend) const
- bool **removeFromLegend** () const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) ([QCustomPlot](#) *plot, [QString](#) targetLayer=[QString](#)(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool **visible** () const
- [QCustomPlot](#) * **parentPlot** () const
- [QCPLayerable](#) * **parentLayerable** () const
- [QCPLayer](#) * **layer** () const
- bool **antialiased** () const
- void **setVisible** (bool on)
- Q_SLOT bool **setLayer** ([QCPLayer](#) *layer)
- bool **setLayer** (const [QString](#) &layerName)
- void **setAntialiased** (bool enabled)
- bool **realVisibility** () const

Protected Member Functions

- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual void [drawLegendIcon](#) ([QCPPainter](#) *painter, const QRectF &rect) const Q_DECL_OVERRIDE
- virtual void [drawStatisticalBox](#) ([QCPPainter](#) *painter, [QCPStatisticalBoxDataContainer::const_iterator](#) it, const [QCPScatterStyle](#) &outlierStyle) const
- void [getVisibleDataBounds](#) ([QCPStatisticalBoxDataContainer::const_iterator](#) &begin, [QCPStatisticalBoxDataContainer::const_iterator](#) &end) const
- QRectF [getQuartileBox](#) ([QCPStatisticalBoxDataContainer::const_iterator](#) it) const
- QVector< QLineF > [getWhiskerBackboneLines](#) ([QCPStatisticalBoxDataContainer::const_iterator](#) it) const
- QVector< QLineF > [getWhiskerBarLines](#) ([QCPStatisticalBoxDataContainer::const_iterator](#) it) const

Protected Member Functions inherited from [QCPAbstractPlottable1D](#)< [QCPStatisticalBoxData](#) >

- void [getDataSegments](#) (QList< [QCPDataRange](#) > &selectedSegments, QList< [QCPDataRange](#) > &unselectedSegments) const
- void [drawPolyline](#) ([QCPPainter](#) *painter, const QVector< QPointF > &lineData) const

Protected Member Functions inherited from [QCPAbstractPlottable](#)

- virtual QRect [clipRect](#) () const Q_DECL_OVERRIDE
- virtual [QCP::Interaction](#) [selectionCategory](#) () const Q_DECL_OVERRIDE
- void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- void [applyFillAntialiasingHint](#) ([QCPPainter](#) *painter) const
- void [applyScattersAntialiasingHint](#) ([QCPPainter](#) *painter) const

Protected Member Functions inherited from [QCPLayerable](#)

- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot)
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void [initializeParentPlot](#) ([QCustomPlot](#) *parentPlot)
- void [setParentLayerable](#) ([QCPLayerable](#) *parentLayerable)
- bool [moveToLayer](#) ([QCPLayer](#) *layer, bool prepend)
- void [applyAntialiasingHint](#) ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- double [mWidth](#)
- double [mWhiskerWidth](#)
- QPen [mWhiskerPen](#)
- QPen [mWhiskerBarPen](#)
- bool [mWhiskerAntialiased](#)
- QPen [mMedianPen](#)
- [QCPScatterStyle](#) [mOutlierStyle](#)

Protected Attributes inherited from [QCPAbstractPlottable1D](#)< [QCPStatisticalBoxData](#) >

- QSharedPointer< [QCPDataContainer](#)< [QCPStatisticalBoxData](#) > > **mDataContainer**

Protected Attributes inherited from [QCPAbstractPlottable](#)

- QString **mName**
- bool **mAntialiasedFill**
- bool **mAntialiasedScatters**
- QPen **mPen**
- QBrush **mBrush**
- QPointer< [QCPAxis](#) > **mKeyAxis**
- QPointer< [QCPAxis](#) > **mValueAxis**
- [QCP::SelectionType](#) **mSelectable**
- [QCPDataSelection](#) **mSelection**
- [QCPSelectionDecorator](#) * **mSelectionDecorator**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Friends

- class [QCustomPlot](#)
- class [QCPLegend](#)

Additional Inherited Members

Signals inherited from [QCPAbstractPlottable](#)

- void [selectionChanged](#) (bool [selected](#))
- void [selectionChanged](#) (const [QCPDataSelection](#) &[selection](#))
- void [selectableChanged](#) ([QCP::SelectionType](#) [selectable](#))

Signals inherited from [QCPLayerable](#)

- void [layerChanged](#) ([QCPLayer](#) *[newLayer](#))

10.465.1 Detailed Description

A plottable representing a single statistical box in a plot.

To plot data, assign it with the [setData](#) or [addData](#) functions. Alternatively, you can also access and modify the data via the [data](#) method, which returns a pointer to the internal [QCPStatisticalBoxDataContainer](#).

Additionally each data point can itself have a list of outliers, drawn as scatter points at the key coordinate of the respective statistical box data point. They can either be set by using the respective [addData](#) method or accessing the individual data points through [data](#), and setting the `QVector<double> outliers` of the data points directly.

10.465.2 Changing the appearance

The appearance of each data point box, ranging from the lower to the upper quartile, is controlled via [setPen](#) and [setBrush](#). You may change the width of the boxes with [setWidth](#) in plot coordinates.

Each data point's visual representation also consists of two whiskers. Whiskers are the lines which reach from the upper quartile to the maximum, and from the lower quartile to the minimum. The appearance of the whiskers can be modified with: [setWhiskerPen](#), [setWhiskerBarPen](#), [setWhiskerWidth](#). The whisker width is the width of the bar perpendicular to the whisker at the top (for maximum) and bottom (for minimum). If the whisker pen is changed, make sure to set the `capStyle` to `Qt::FlatCap`. Otherwise the backbone line might exceed the whisker bars by a few pixels due to the pen cap being not perfectly flat.

The median indicator line inside the box has its own pen, [setMedianPen](#).

The outlier data points are drawn as normal scatter points. Their look can be controlled with [setOutlierStyle](#)

10.465.3 Usage

Like all data representing objects in [QCustomPlot](#), the [QCPStatisticalBox](#) is a plottable ([QCPAbstractPlottable](#)). So the plottable-interface of [QCustomPlot](#) applies ([QCustomPlot::plottable](#), [QCustomPlot::removePlottable](#), etc.)

Usually, you first create an instance:

which registers it with the [QCustomPlot](#) instance of the passed axes. Note that this [QCustomPlot](#) instance takes ownership of the plottable, so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead. The newly created plottable can be modified, e.g.:

10.465.4 Constructor & Destructor Documentation

10.465.4.1 QCPStatisticalBox()

```
QCPStatisticalBox::QCPStatisticalBox (
    QCPAxis * keyAxis,
    QCPAxis * valueAxis ) [explicit]
```

Constructs a statistical box which uses *keyAxis* as its key axis ("x") and *valueAxis* as its value axis ("y"). *keyAxis* and *valueAxis* must reside in the same [QCustomPlot](#) instance and not have the same orientation. If either of these restrictions is violated, a corresponding message is printed to the debug output (qDebug), the construction is not aborted, though.

The created [QCPStatisticalBox](#) is automatically registered with the [QCustomPlot](#) instance inferred from *keyAxis*. This [QCustomPlot](#) instance takes ownership of the [QCPStatisticalBox](#), so do not delete it manually but use [QCustomPlot::removePlottable\(\)](#) instead.

10.465.5 Member Function Documentation

10.465.5.1 `addData()` [1/2]

```
void QCPStatisticalBox::addData (
    const QVector< double > & keys,
    const QVector< double > & minimum,
    const QVector< double > & lowerQuartile,
    const QVector< double > & median,
    const QVector< double > & upperQuartile,
    const QVector< double > & maximum,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided points in *keys*, *minimum*, *lowerQuartile*, *median*, *upperQuartile* and *maximum* to the current data. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

10.465.5.2 `addData()` [2/2]

```
void QCPStatisticalBox::addData (
    double key,
    double minimum,
    double lowerQuartile,
    double median,
    double upperQuartile,
    double maximum,
    const QVector< double > & outliers = QVector<double>() )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Adds the provided data point as *key*, *minimum*, *lowerQuartile*, *median*, *upperQuartile* and *maximum* to the current data.

Alternatively, you can also access and modify the data directly via the [data](#) method, which returns a pointer to the internal data container.

10.465.5.3 `data()`

```
QSharedPointer< QCPStatisticalBoxDataContainer > QCPStatisticalBox::data ( ) const [inline]
```

Returns a shared pointer to the internal data storage of type `QCPStatisticalBoxDataContainer`. You may use it to directly manipulate the data, which may be more convenient and faster than using the regular [setData](#) or [addData](#) methods.

10.465.5.4 draw()

```
void QCPStatisticalBox::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

10.465.5.5 drawLegendIcon()

```
void QCPStatisticalBox::drawLegendIcon (
    QCPPainter * painter,
    const QRectF & rect ) const [protected], [virtual]
```

Implements [QCPAbstractPlottable](#).

10.465.5.6 drawStatisticalBox()

```
void QCPStatisticalBox::drawStatisticalBox (
    QCPPainter * painter,
    QCPStatisticalBoxDataContainer::const_iterator it,
    const QCPScatterStyle & outlierStyle ) const [protected], [virtual]
```

Draws the graphical representation of a single statistical box with the data given by the iterator *it* with the provided *painter*.

If the statistical box has a set of outlier data points, they are drawn with *outlierStyle*.

See also

[getQuartileBox](#), [getWhiskerBackboneLines](#), [getWhiskerBarLines](#)

10.465.5.7 getKeyRange()

```
QCPRange QCPStatisticalBox::getKeyRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth ) const [virtual]
```

Returns the coordinate range that all data in this plottable span in the key axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getValueRange](#)

Implements [QCPAbstractPlottable](#).

10.465.5.8 `getValueRange()`

```
QCPRange QCPStatisticalBox::getValueRange (
    bool & foundRange,
    QCP::SignDomain inSignDomain = QCP::sdBoth,
    const QCPRange & inKeyRange = QCPRange() ) const [virtual]
```

Returns the coordinate range that the data points in the specified key range (*inKeyRange*) span in the value axis dimension. For logarithmic plots, one can set *inSignDomain* to either [QCP::sdNegative](#) or [QCP::sdPositive](#) in order to restrict the returned range to that sign domain. E.g. when only negative range is wanted, set *inSignDomain* to [QCP::sdNegative](#) and all positive points will be ignored for range calculation. For no restriction, just set *inSignDomain* to [QCP::sdBoth](#) (default). *foundRange* is an output parameter that indicates whether a range could be found or not. If this is false, you shouldn't use the returned range (e.g. no points in data).

If *inKeyRange* has both lower and upper bound set to zero (is equal to [QCPRange\(\)](#)), all data points are considered, without any restriction on the keys.

Note that *foundRange* is not the same as [QCPRange::validRange](#), since the range returned by this function may have size zero (e.g. when there is only one data point). In this case *foundRange* would return true, but the returned range is not a valid range in terms of [QCPRange::validRange](#).

See also

[rescaleAxes](#), [getKeyRange](#)

Implements [QCPAbstractPlottable](#).

10.465.5.9 `selectTest()`

```
double QCPStatisticalBox::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Implements a `selectTest` specific to this plottable's point geometry.

If *details* is not 0, it will be set to a [QCPDataSelection](#), describing the closest data point to *pos*.

\see [baseclassmethod QCPAbstractPlottable::selectTest](#)

Reimplemented from [QCPAbstractPlottable1D< QCPStatisticalBoxData >](#).

10.465.5.10 `selectTestRect()`

```
QCPDataSelection QCPStatisticalBox::selectTestRect (
    const QRectF & rect,
    bool onlySelectable ) const [virtual]
```

Returns a data selection containing all the data points of this plottable which are contained (or hit by) *rect*. This is used mainly in the selection rect interaction for data selection (data selection mechanism).

If *onlySelectable* is true, an empty [QCPDataSelection](#) is returned if this plottable is not selectable (i.e. if [QCPAbstractPlottable::setSelectable](#) is [QCP::stNone](#)).

Note

rect must be a normalized rect (positive or zero width and height). This is especially important when using the rect of [QCPSelectionRect::accepted](#), which is not necessarily normalized. Use [QRect::normalized\(\)](#) when passing a rect which might not be normalized.

Reimplemented from [QCPAbstractPlottable1D< QCPStatisticalBoxData >](#).

10.465.5.11 setData() [1/2]

```
void QCPStatisticalBox::setData (
    const QVector< double > & keys,
    const QVector< double > & minimum,
    const QVector< double > & lowerQuartile,
    const QVector< double > & median,
    const QVector< double > & upperQuartile,
    const QVector< double > & maximum,
    bool alreadySorted = false )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data with the provided points in *keys*, *minimum*, *lowerQuartile*, *median*, *upperQuartile* and *maximum*. The provided vectors should have equal length. Else, the number of added points will be the size of the smallest vector.

If you can guarantee that the passed data points are sorted by *keys* in ascending order, you can set *alreadySorted* to true, to improve performance by saving a sorting run.

See also

[addData](#)

10.465.5.12 setData() [2/2]

```
void QCPStatisticalBox::setData (
    QSharedPointer< QCPStatisticalBoxDataContainer > data )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Replaces the current data container with the provided *data* container.

Since a QSharedPointer is used, multiple QCPStatisticalBoxes may share the same data container safely. Modifying the data in the container will then affect all statistical boxes that share the container. Sharing can be achieved by simply exchanging the data containers wrapped in shared pointers:

If you do not wish to share containers, but create a copy from an existing container, rather use the [QCPDataContainer<DataType>::set](#) method on the statistical box data container directly:

See also

[addData](#)

10.465.5.13 setMedianPen()

```
void QCPStatisticalBox::setMedianPen (
    const QPen & pen )
```

Sets the pen used for drawing the median indicator line inside the statistical boxes.

10.465.5.14 `setOutlierStyle()`

```
void QCPStatisticalBox::setOutlierStyle (
    const QCPScatterStyle & style )
```

Sets the appearance of the outlier data points.

Outliers can be specified with the method [addData\(double key, double minimum, double lowerQuartile, double median, double upperQ](#)

10.465.5.15 `setWhiskerAntialiased()`

```
void QCPStatisticalBox::setWhiskerAntialiased (
    bool enabled )
```

Sets whether the statistical boxes whiskers are drawn with antialiasing or not.

Note that antialiasing settings may be overridden by [QCustomPlot::setAntialiasedElements](#) and [QCustomPlot::setNotAntialiasedElement](#)

10.465.5.16 `setWhiskerBarPen()`

```
void QCPStatisticalBox::setWhiskerBarPen (
    const QPen & pen )
```

Sets the pen used for drawing the whisker bars. Those are the lines parallel to the key axis at each end of the whisker backbone.

Whiskers are the lines which reach from the upper quartile to the maximum, and from the lower quartile to the minimum.

See also

[setWhiskerPen](#)

10.465.5.17 `setWhiskerPen()`

```
void QCPStatisticalBox::setWhiskerPen (
    const QPen & pen )
```

Sets the pen used for drawing the whisker backbone.

Whiskers are the lines which reach from the upper quartile to the maximum, and from the lower quartile to the minimum.

Make sure to set the `capStyle` of the passed `pen` to `Qt::FlatCap`. Otherwise the backbone line might exceed the whisker bars by a few pixels due to the pen cap being not perfectly flat.

See also

[setWhiskerBarPen](#)

10.465.5.18 setWhiskerWidth()

```
void QCPStatisticalBox::setWhiskerWidth (
    double width )
```

Sets the width of the whiskers in key coordinates.

Whiskers are the lines which reach from the upper quartile to the maximum, and from the lower quartile to the minimum.

See also

[setWidth](#)

10.465.5.19 setWidth()

```
void QCPStatisticalBox::setWidth (
    double width )
```

Sets the width of the boxes in key coordinates.

See also

[setWhiskerWidth](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.466 QCPStatisticalBoxData Class Reference

Holds the data of one single data point for [QCPStatisticalBox](#).

Public Member Functions

- [QCPStatisticalBoxData](#) ()
- [QCPStatisticalBoxData](#) (double key, double minimum, double lowerQuartile, double median, double upperQuartile, double maximum, const QVector< double > &outliers=QVector< double >())
- double [sortKey](#) () const
- double [mainKey](#) () const
- double [mainValue](#) () const
- [QCPRange](#) [valueRange](#) () const

Static Public Member Functions

- static [QCPStatisticalBoxData](#) [fromSortKey](#) (double [sortKey](#))
- static bool [sortKeyIsMainKey](#) ()

Public Attributes

- double **key**
- double **minimum**
- double **lowerQuartile**
- double **median**
- double **upperQuartile**
- double **maximum**
- QVector< double > **outliers**

10.466.1 Detailed Description

Holds the data of one single data point for [QCPStatisticalBox](#).

The stored data is:

- *key*: coordinate on the key axis of this data point (this is the *mainKey* and the *sortKey*)
- *minimum*: the position of the lower whisker, typically the minimum measurement of the sample that's not considered an outlier.
- *lowerQuartile*: the lower end of the box. The lower and the upper quartiles are the two statistical quartiles around the median of the sample, they should contain 50% of the sample data.
- *median*: the value of the median mark inside the quartile box. The median separates the sample data in half (50% of the sample data is below/above the median). (This is the *mainValue*)
- *upperQuartile*: the upper end of the box. The lower and the upper quartiles are the two statistical quartiles around the median of the sample, they should contain 50% of the sample data.
- *maximum*: the position of the upper whisker, typically the maximum measurement of the sample that's not considered an outlier.
- *outliers*: a QVector of outlier values that will be drawn as scatter points at the *key* coordinate of this data point (see [QCPStatisticalBox::setOutlierStyle](#))

The container for storing multiple data points is [QCPStatisticalBoxDataContainer](#). It is a typedef for [QCPDataContainer](#) with [QCPStatisticalBoxData](#) as the `DataType` template parameter. See the documentation there for an explanation regarding the data type's generic methods.

See also

[QCPStatisticalBoxDataContainer](#)

10.466.2 Constructor & Destructor Documentation

10.466.2.1 QCPStatisticalBoxData() [1/2]

```
QCPStatisticalBoxData::QCPStatisticalBoxData ( )
```

Constructs a data point with *key* and all values set to zero.

10.466.2.2 QCPStatisticalBoxData() [2/2]

```
QCPStatisticalBoxData::QCPStatisticalBoxData (
    double key,
    double minimum,
    double lowerQuartile,
    double median,
    double upperQuartile,
    double maximum,
    const QVector< double > & outliers = QVector<double>() )
```

Constructs a data point with the specified *key*, *minimum*, *lowerQuartile*, *median*, *upperQuartile*, *maximum* and optionally a number of *outliers*.

10.466.3 Member Function Documentation

10.466.3.1 fromSortKey()

```
static QCPStatisticalBoxData QCPStatisticalBoxData::fromSortKey (
    double sortKey ) [inline], [static]
```

Returns a data point with the specified *sortKey*. All other members are set to zero.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.466.3.2 mainKey()

```
double QCPStatisticalBoxData::mainKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.466.3.3 mainValue()

```
double QCPStatisticalBoxData::mainValue ( ) const [inline]
```

Returns the *median* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.466.3.4 sortKey()

```
double QCPStatisticalBoxData::sortKey ( ) const [inline]
```

Returns the *key* member of this data point.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.466.3.5 sortKeyIsMainKey()

```
static static bool QCPStatisticalBoxData::sortKeyIsMainKey ( ) [inline], [static]
```

Since the member *key* is both the data point key coordinate and the data ordering parameter, this method returns true.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

10.466.3.6 valueRange()

```
QCPRange QCPStatisticalBoxData::valueRange ( ) const [inline]
```

Returns a [QCPRange](#) spanning from the *minimum* to the *maximum* member of this statistical box data point, possibly further expanded by outliers.

For a general explanation of what this method is good for in the context of the data container, see the documentation of [QCPDataContainer](#).

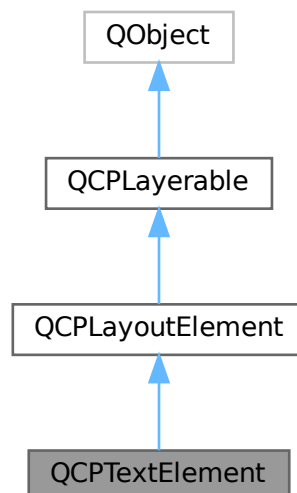
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

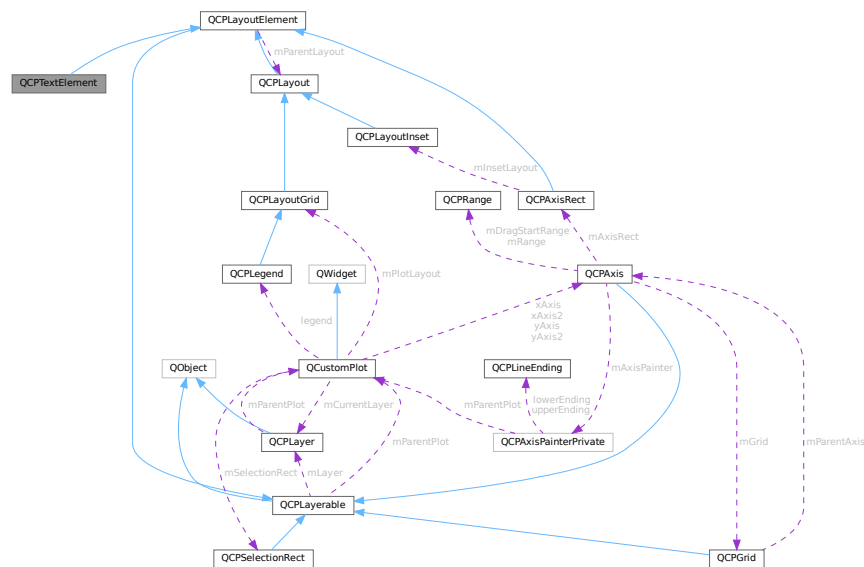
10.467 QCPTextElement Class Reference

A layout element displaying a text.

Inheritance diagram for QCPTextElement:



Collaboration diagram for QCPTextElement:



Signals

- void **selectionChanged** (bool selected)
- void **selectableChanged** (bool selectable)
- void **clicked** (QMouseEvent *event)
- void **doubleClicked** (QMouseEvent *event)

Signals inherited from QCPLayerable

- void layerChanged (QCPLayer *newLayer)

Public Member Functions

- [QCPTextElement](#) ([QCustomPlot](#) *parentPlot)
- [QCPTextElement](#) ([QCustomPlot](#) *parentPlot, const QString &text)
- [QCPTextElement](#) ([QCustomPlot](#) *parentPlot, const QString &text, double pointSize)
- [QCPTextElement](#) ([QCustomPlot](#) *parentPlot, const QString &text, const QString &fontFamily, double pointSize)
- [QCPTextElement](#) ([QCustomPlot](#) *parentPlot, const QString &text, const QFont &font)
- QString **text** () const
- int **textFlags** () const
- QFont **font** () const
- QColor **textColor** () const
- QFont **selectedFont** () const
- QColor **selectedTextColor** () const
- bool **selectable** () const
- bool **selected** () const
- void [setText](#) (const QString &text)
- void [setTextFlags](#) (int flags)

- void [setFont](#) (const QFont &font)
- void [setTextColor](#) (const QColor &color)
- void [setSelectedFont](#) (const QFont &font)
- void [setSelectedTextColor](#) (const QColor &color)
- Q_SLOT void [setSelectable](#) (bool selectable)
- Q_SLOT void [setSelected](#) (bool selected)
- virtual double [selectTest](#) (const QPointF &pos, bool onlySelectable, QVariant *details=nullptr) const Q_DECL_OVERRIDE
- virtual void [mousePressEvent](#) (QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE

Public Member Functions inherited from [QCPLayoutElement](#)

- [QCPLayoutElement](#) (QCustomPlot *parentPlot=nullptr)
- [QCPLayout](#) * [layout](#) () const
- QRect [rect](#) () const
- QRect [outerRect](#) () const
- QMargins [margins](#) () const
- QMargins [minimumMargins](#) () const
- QCP::MarginSides [autoMargins](#) () const
- QSize [minimumSize](#) () const
- QSize [maximumSize](#) () const
- [SizeConstraintRect](#) [sizeConstraintRect](#) () const
- [QCPMarginGroup](#) * [marginGroup](#) (QCP::MarginSide side) const
- QHash< QCP::MarginSide, [QCPMarginGroup](#) * > [marginGroups](#) () const
- void [setOuterRect](#) (const QRect &rect)
- void [setMargins](#) (const QMargins &margins)
- void [setMinimumMargins](#) (const QMargins &margins)
- void [setAutoMargins](#) (QCP::MarginSides sides)
- void [setMinimumSize](#) (const QSize &size)
- void [setMinimumSize](#) (int width, int height)
- void [setMaximumSize](#) (const QSize &size)
- void [setMaximumSize](#) (int width, int height)
- void [setSizeConstraintRect](#) ([SizeConstraintRect](#) constraintRect)
- void [setMarginGroup](#) (QCP::MarginSides sides, [QCPMarginGroup](#) *group)
- virtual void [update](#) ([UpdatePhase](#) phase)
- virtual QList< [QCPLayoutElement](#) * > [elements](#) (bool recursive) const

Public Member Functions inherited from [QCPLayerable](#)

- [QCPLayerable](#) (QCustomPlot *plot, QString targetLayer=QString(), [QCPLayerable](#) *parentLayerable=nullptr)
- bool [visible](#) () const
- QCustomPlot * [parentPlot](#) () const
- [QCPLayerable](#) * [parentLayerable](#) () const
- [QCPLayer](#) * [layer](#) () const
- bool [antialiased](#) () const
- void [setVisible](#) (bool on)
- Q_SLOT bool [setLayer](#) ([QCPLayer](#) *layer)
- bool [setLayer](#) (const QString &layerName)
- void [setAntialiased](#) (bool enabled)
- bool [realVisibility](#) () const

Protected Member Functions

- virtual void [applyDefaultAntialiasingHint](#) ([QCPPainter](#) *painter) const Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter) Q_DECL_OVERRIDE
- virtual QSize [minimumOuterSizeHint](#) () const Q_DECL_OVERRIDE
- virtual QSize [maximumOuterSizeHint](#) () const Q_DECL_OVERRIDE
- virtual void [selectEvent](#) (QMouseEvent *event, bool additive, const QVariant &details, bool *selectionState↵ Changed) Q_DECL_OVERRIDE
- virtual void [deselectEvent](#) (bool *selectionStateChanged) Q_DECL_OVERRIDE
- QFont **mainFont** () const
- QColor **mainTextColor** () const

Protected Member Functions inherited from [QCPLayoutElement](#)

- virtual int **calculateAutoMargin** ([QCP::MarginSide](#) side)
- virtual void **layoutChanged** ()
- virtual void [parentPlotInitialized](#) ([QCustomPlot](#) *parentPlot) Q_DECL_OVERRIDE

Protected Member Functions inherited from [QCPLayerable](#)

- virtual [QCP::Interaction](#) **selectionCategory** () const
- virtual QRect **clipRect** () const
- virtual void [mouseMoveEvent](#) (QMouseEvent *event, const QPointF &startPos)
- virtual void [wheelEvent](#) (QWheelEvent *event)
- void **initializeParentPlot** ([QCustomPlot](#) *parentPlot)
- void **setParentLayerable** ([QCPLayerable](#) *parentLayerable)
- bool **moveToLayer** ([QCPLayer](#) *layer, bool prepend)
- void **applyAntialiasingHint** ([QCPPainter](#) *painter, bool localAntialiased, [QCP::AntialiasedElement](#) overrideElement) const

Protected Attributes

- QString **mText**
- int **mTextFlags**
- QFont **mFont**
- QColor **mTextColor**
- QFont **mSelectedFont**
- QColor **mSelectedTextColor**
- QRect **mTextBoundingRect**
- bool **mSelectable**
- bool **mSelected**

Protected Attributes inherited from [QCPLayoutElement](#)

- [QCPLayout](#) * **mParentLayout**
- QSize **mMinimumSize**
- QSize **mMaximumSize**
- [SizeConstraintRect](#) **mSizeConstraintRect**
- QRect **mRect**
- QRect **mOuterRect**
- QMargins **mMargins**
- QMargins **mMinimumMargins**
- QCP::MarginSides **mAutoMargins**
- QHash< [QCP::MarginSide](#), [QCPMarginGroup](#) * > **mMarginGroups**

Protected Attributes inherited from [QCPLayerable](#)

- bool **mVisible**
- [QCustomPlot](#) * **mParentPlot**
- QPointer< [QCPLayerable](#) > **mParentLayerable**
- [QCPLayer](#) * **mLayer**
- bool **mAntialiased**

Additional Inherited Members

Public Types inherited from [QCPLayoutElement](#)

- enum [UpdatePhase](#) { [upPreparation](#) , [upMargins](#) , [upLayout](#) }
- enum [SizeConstraintRect](#) { [scrInnerRect](#) , [scrOuterRect](#) }

10.467.1 Detailed Description

A layout element displaying a text.

The text may be specified with [setText](#), the formatting can be controlled with [setFont](#), [setTextColor](#), and [setTextFlags](#).

A text element can be added as follows:

10.467.2 Constructor & Destructor Documentation

10.467.2.1 QCPTextElement() [1/5]

```
QCPTextElement::QCPTextElement (
    QCustomPlot * parentPlot ) [explicit]
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Creates a new [QCPTextElement](#) instance and sets default values. The initial text is empty ([setText](#)).

10.467.2.2 QCPTextElement() [2/5]

```
QCPTextElement::QCPTextElement (
    QCustomPlot * parentPlot,
    const QString & text )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Creates a new [QCPTextElement](#) instance and sets default values.

The initial text is set to *text*.

10.467.2.3 QCPTextElement() [3/5]

```
QCPTextElement::QCPTextElement (
    QCustomPlot * parentPlot,
    const QString & text,
    double pointSize )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Creates a new [QCPTextElement](#) instance and sets default values.

The initial text is set to *text* with *pointSize*.

10.467.2.4 QCPTextElement() [4/5]

```
QCPTextElement::QCPTextElement (
    QCustomPlot * parentPlot,
    const QString & text,
    const QString & fontFamily,
    double pointSize )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Creates a new [QCPTextElement](#) instance and sets default values.

The initial text is set to *text* with *pointSize* and the specified *fontFamily*.

10.467.2.5 QCPTextElement() [5/5]

```
QCPTextElement::QCPTextElement (
    QCustomPlot * parentPlot,
    const QString & text,
    const QFont & font )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Creates a new [QCPTextElement](#) instance and sets default values.

The initial text is set to *text* with the specified *font*.

10.467.3 Member Function Documentation

10.467.3.1 applyDefaultAntialiasingHint()

```
void QCPTextElement::applyDefaultAntialiasingHint (
    QCPPainter * painter ) const [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

10.467.3.2 clicked

```
void QCPTextElement::clicked (
    QMouseEvent * event ) [signal]
```

This signal is emitted when the text element is clicked.

See also

[doubleClicked](#), [selectTest](#)

10.467.3.3 deselectEvent()

```
void QCPTextElement::deselectEvent (
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.467.3.4 doubleClicked

```
void QCPTextElement::doubleClicked (
    QMouseEvent * event ) [signal]
```

This signal is emitted when the text element is double clicked.

See also

[clicked](#), [selectTest](#)

10.467.3.5 draw()

```
void QCPTextElement::draw (
    QCPPainter * painter ) [protected], [virtual]
```

Reimplemented from [QCPLayoutElement](#).

10.467.3.6 maximumOuterSizeHint()

```
QSize QCPTextElement::maximumOuterSizeHint ( ) const [protected], [virtual]
```

Returns the suggested maximum size this layout element (the [outerRect](#)) may be expanded to, if no manual maximum size is set.

if a maximum size ([setMaximumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through [QCPLayout::getFinalMaximumOuterSize](#)) to determine the maximum allowed size of this layout element.

A manual maximum size is considered set if it is smaller than Qt's `QWIDGETSIZE_MAX`.

The default implementation simply returns `QWIDGETSIZE_MAX` for both width and height, implying no suggested maximum size. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented from [QCPLayoutElement](#).

10.467.3.7 minimumOuterSizeHint()

```
QSize QCPTextElement::minimumOuterSizeHint ( ) const [protected], [virtual]
```

Returns the suggested minimum size this layout element (the [outerRect](#)) may be compressed to, if no manual minimum size is set.

if a minimum size ([setMinimumSize](#)) was not set manually, parent layouts use the returned size (usually indirectly through [QCPLayout::getFinalMinimumOuterSize](#)) to determine the minimum allowed size of this layout element.

A manual minimum size is considered set if it is non-zero.

The default implementation simply returns the sum of the horizontal margins for the width and the sum of the vertical margins for the height. Reimplementations may use their detailed knowledge about the layout element's content to provide size hints.

Reimplemented from [QCPLayoutElement](#).

10.467.3.8 mouseDoubleClickEvent()

```
void QCPTextElement::mouseDoubleClickEvent (
    QMouseEvent * event,
    const QVariant & details ) [virtual]
```

Emits the [doubleClicked](#) signal.

\see [baseclassmethod](#)

Reimplemented from [QCPLayerable](#).

10.467.3.9 mousePressEvent()

```
void QCPTextElement::mousePressEvent (
    QMouseEvent * event,
    const QVariant & details ) [virtual]
```

Accepts the mouse event in order to emit the according click signal in the [mouseReleaseEvent](#).

\see [baseclassmethod](#)

Reimplemented from [QCPLayerable](#).

10.467.3.10 mouseReleaseEvent()

```
void QCPTextElement::mouseReleaseEvent (
    QMouseEvent * event,
    const QPointF & startPos ) [virtual]
```

Emits the [clicked](#) signal if the cursor hasn't moved by more than a few pixels since the [mousePressEvent](#).

\see [baseclassmethod](#)

Reimplemented from [QCPLayerable](#).

10.467.3.11 selectEvent()

```
void QCPTextElement::selectEvent (
    QMouseEvent * event,
    bool additive,
    const QVariant & details,
    bool * selectionStateChanged ) [protected], [virtual]
```

Reimplemented from [QCPLayerable](#).

10.467.3.12 selectionChanged

```
void QCPTextElement::selectionChanged (
    bool selected ) [signal]
```

This signal is emitted when the selection state has changed to *selected*, either by user interaction or by a direct call to [setSelected](#).

See also

[setSelected](#), [setSelectable](#)

10.467.3.13 selectTest()

```
double QCPTextElement::selectTest (
    const QPointF & pos,
    bool onlySelectable,
    QVariant * details = nullptr ) const [virtual]
```

Returns $0.99 * \text{selectionTolerance}$ (see [QCustomPlot::setSelectionTolerance](#)) when *pos* is within the bounding box of the text element's text. Note that this bounding box is updated in the draw call.

If *pos* is outside the text's bounding box or if *onlySelectable* is true and this text element is not selectable ([setSelectable](#)), returns -1.

\seebaseclassmethod

Reimplemented from [QCPLayoutElement](#).

10.467.3.14 setFont()

```
void QCPTextElement::setFont (
    const QFont & font )
```

Sets the *font* of the text.

See also

[setTextColor](#), [setSelectedFont](#)

10.467.3.15 setSelectable()

```
void QCPTextElement::setSelectable (
    bool selectable )
```

Sets whether the user may select this text element.

Note that even when *selectable* is set to `false`, the selection state may be changed programmatically via [setSelected](#).

10.467.3.16 setSelected()

```
void QCPTextElement::setSelected (
    bool selected )
```

Sets the selection state of this text element to *selected*. If the selection has changed, [selectionChanged](#) is emitted.

Note that this function can change the selection state independently of the current [setSelectable](#) state.

10.467.3.17 setSelectedFont()

```
void QCPTextElement::setSelectedFont (
    const QFont & font )
```

Sets the *font* of the text that will be used if the text element is selected ([setSelected](#)).

See also

[setFont](#)

10.467.3.18 setSelectedTextColor()

```
void QCPTextElement::setSelectedTextColor (
    const QColor & color )
```

Sets the *color* of the text that will be used if the text element is selected ([setSelected](#)).

See also

[setTextColor](#)

10.467.3.19 setText()

```
void QCPTextElement::setText (
    const QString & text )
```

Sets the text that will be displayed to *text*. Multiple lines can be created by insertion of `"\n"`.

See also

[setFont](#), [setTextColor](#), [setTextFlags](#)

10.467.3.20 `setTextColor()`

```
void QCPTextElement::setTextColor (
    const QColor & color )
```

Sets the *color* of the text.

See also

[setFont](#), [setSelectedTextColor](#)

10.467.3.21 `setTextFlags()`

```
void QCPTextElement::setTextFlags (
    int flags )
```

Sets options for text alignment and wrapping behaviour. *flags* is a bitwise OR-combination of `Qt::AlignmentFlag` and `Qt::TextFlag` enums.

Possible enums are:

- `Qt::AlignLeft`
- `Qt::AlignRight`
- `Qt::AlignHCenter`
- `Qt::AlignJustify`
- `Qt::AlignTop`
- `Qt::AlignBottom`
- `Qt::AlignVCenter`
- `Qt::AlignCenter`
- `Qt::TextDontClip`
- `Qt::TextSingleLine`
- `Qt::TextExpandTabs`
- `Qt::TextShowMnemonic`
- `Qt::TextWordWrap`
- `Qt::TextIncludeTrailingSpaces`

The documentation for this class was generated from the following files:

- `applications/gui/genesys/qcustomplot.h`
- `applications/gui/genesys/qcustomplot.cpp`

10.468 QCPVector2D Class Reference

Represents two doubles as a mathematical 2D vector.

Public Member Functions

- [QCPVector2D](#) ()
- [QCPVector2D](#) (double x, double y)
- [QCPVector2D](#) (const QPoint &point)
- [QCPVector2D](#) (const QPointF &point)
- double **x** () const
- double **y** () const
- double & **rx** ()
- double & **ry** ()
- void [setX](#) (double x)
- void [setY](#) (double y)
- double [length](#) () const
- double [lengthSquared](#) () const
- double [angle](#) () const
- QPoint [toPoint](#) () const
- QPointF [toPointF](#) () const
- bool [isNull](#) () const
- void [normalize](#) ()
- [QCPVector2D](#) [normalized](#) () const
- [QCPVector2D](#) [perpendicular](#) () const
- double [dot](#) (const [QCPVector2D](#) &vec) const
- double [distanceSquaredToLine](#) (const [QCPVector2D](#) &start, const [QCPVector2D](#) &end) const
- double [distanceSquaredToLine](#) (const QLineF &line) const
- double [distanceToStraightLine](#) (const [QCPVector2D](#) &base, const [QCPVector2D](#) &direction) const
- [QCPVector2D](#) & [operator*=](#) (double factor)
- [QCPVector2D](#) & [operator/=](#) (double divisor)
- [QCPVector2D](#) & [operator+=](#) (const [QCPVector2D](#) &vector)
- [QCPVector2D](#) & [operator-=](#) (const [QCPVector2D](#) &vector)

Friends

- const [QCPVector2D](#) [operator*](#) (double factor, const [QCPVector2D](#) &vec)
- const [QCPVector2D](#) [operator*](#) (const [QCPVector2D](#) &vec, double factor)
- const [QCPVector2D](#) [operator/](#) (const [QCPVector2D](#) &vec, double divisor)
- const [QCPVector2D](#) [operator+](#) (const [QCPVector2D](#) &vec1, const [QCPVector2D](#) &vec2)
- const [QCPVector2D](#) [operator-](#) (const [QCPVector2D](#) &vec1, const [QCPVector2D](#) &vec2)
- const [QCPVector2D](#) [operator-](#) (const [QCPVector2D](#) &vec)

Related Symbols

(Note that these are not member symbols.)

- QDebug [operator<<](#) (QDebug d, const [QCPVector2D](#) &vec)

10.468.1 Detailed Description

Represents two doubles as a mathematical 2D vector.

This class acts as a replacement for QVector2D with the advantage of double precision instead of single, and some convenience methods tailored for the [QCustomPlot](#) library.

10.468.2 Constructor & Destructor Documentation

10.468.2.1 QCPVector2D() [1/4]

```
QCPVector2D::QCPVector2D ( )
```

Creates a [QCPVector2D](#) object and initializes the x and y coordinates to 0.

10.468.2.2 QCPVector2D() [2/4]

```
QCPVector2D::QCPVector2D (
    double x,
    double y )
```

Creates a [QCPVector2D](#) object and initializes the x and y coordinates with the specified values.

10.468.2.3 QCPVector2D() [3/4]

```
QCPVector2D::QCPVector2D (
    const QPoint & point )
```

Creates a [QCPVector2D](#) object and initializes the x and y coordinates respective coordinates of the specified *point*.

10.468.2.4 QCPVector2D() [4/4]

```
QCPVector2D::QCPVector2D (
    const QPointF & point )
```

Creates a [QCPVector2D](#) object and initializes the x and y coordinates respective coordinates of the specified *point*.

10.468.3 Member Function Documentation

10.468.3.1 angle()

```
double QCPVector2D::angle ( ) const [inline]
```

Returns the angle of the vector in radians. The angle is measured between the positive x line and the vector, counter-clockwise in a mathematical coordinate system (y axis upwards positive). In screen/widget coordinates where the y axis is inverted, the angle appears clockwise.

10.468.3.2 distanceSquaredToLine() [1/2]

```
double QCPVector2D::distanceSquaredToLine (
    const QCPVector2D & start,
    const QCPVector2D & end ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the squared shortest distance of this vector (interpreted as a point) to the finite line segment given by *start* and *end*.

See also

[distanceToStraightLine](#)

10.468.3.3 distanceSquaredToLine() [2/2]

```
double QCPVector2D::distanceSquaredToLine (
    const QLineF & line ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the squared shortest distance of this vector (interpreted as a point) to the finite line segment given by *line*.

See also

[distanceToStraightLine](#)

10.468.3.4 distanceToStraightLine()

```
double QCPVector2D::distanceToStraightLine (
    const QCPVector2D & base,
    const QCPVector2D & direction ) const
```

Returns the shortest distance of this vector (interpreted as a point) to the infinite straight line given by a *base* point and a *direction* vector.

See also

[distanceSquaredToLine](#)

10.468.3.5 dot()

```
double QCPVector2D::dot (
    const QCPVector2D & vec ) const [inline]
```

Returns the dot/scalar product of this vector with the specified vector *vec*.

10.468.3.6 isNull()

```
bool QCPVector2D::isNull ( ) const [inline]
```

Returns whether this vector is null. A vector is null if `qIsNull` returns true for both x and y coordinates, i.e. if both are binary equal to 0.

10.468.3.7 length()

```
double QCPVector2D::length ( ) const [inline]
```

Returns the length of this vector.

See also

[lengthSquared](#)

10.468.3.8 lengthSquared()

```
double QCPVector2D::lengthSquared ( ) const [inline]
```

Returns the squared length of this vector. In some situations, e.g. when just trying to find the shortest vector of a group, this is faster than calculating [length](#), because it avoids calculation of a square root.

See also

[length](#)

10.468.3.9 normalize()

```
void QCPVector2D::normalize ( )
```

Normalizes this vector. After this operation, the length of the vector is equal to 1.

If the vector has both entries set to zero, this method does nothing.

See also

[normalized](#), [length](#), [lengthSquared](#)

10.468.3.10 normalized()

```
QCPVector2D QCPVector2D::normalized ( ) const
```

Returns a normalized version of this vector. The length of the returned vector is equal to 1.

If the vector has both entries set to zero, this method returns the vector unmodified.

See also

[normalize](#), [length](#), [lengthSquared](#)

10.468.3.11 operator*=()

```
QCPVector2D & QCPVector2D::operator*= (
    double factor )
```

Scales this vector by the given *factor*, i.e. the x and y components are multiplied by *factor*.

10.468.3.12 operator+=()

```
QCPVector2D & QCPVector2D::operator+= (
    const QCPVector2D & vector )
```

Adds the given *vector* to this vector component-wise.

10.468.3.13 operator-=()

```
QCPVector2D & QCPVector2D::operator-= (
    const QCPVector2D & vector )
```

subtracts the given *vector* from this vector component-wise.

10.468.3.14 operator/=()

```
QCPVector2D & QCPVector2D::operator/= (
    double divisor )
```

Scales this vector by the given *divisor*, i.e. the x and y components are divided by *divisor*.

10.468.3.15 perpendicular()

```
QCPVector2D QCPVector2D::perpendicular ( ) const [inline]
```

Returns a vector perpendicular to this vector, with the same length.

10.468.3.16 setX()

```
void QCPVector2D::setX (
    double x ) [inline]
```

Sets the x coordinate of this vector to *x*.

See also

[setY](#)

10.468.3.17 setY()

```
void QCPVector2D::setY (
    double y ) [inline]
```

Sets the y coordinate of this vector to *y*.

See also

[setX](#)

10.468.3.18 toPoint()

```
QPoint QCPVector2D::toPoint ( ) const [inline]
```

Returns a QPoint which has the x and y coordinates of this vector, truncating any floating point information.

See also

[toPointF](#)

10.468.3.19 toPointF()

```
QPointF QCPVector2D::toPointF ( ) const [inline]
```

Returns a QPointF which has the x and y coordinates of this vector.

See also

[toPoint](#)

10.468.4 Friends And Related Symbol Documentation

10.468.4.1 operator<<()

```
QDebug operator<< (
    QDebug d,
    const QCPVector2D & vec ) [related]
```

Prints *vec* in a human readable format to the qDebug output.

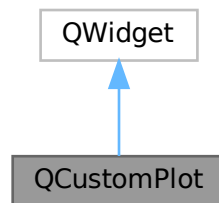
The documentation for this class was generated from the following files:

- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

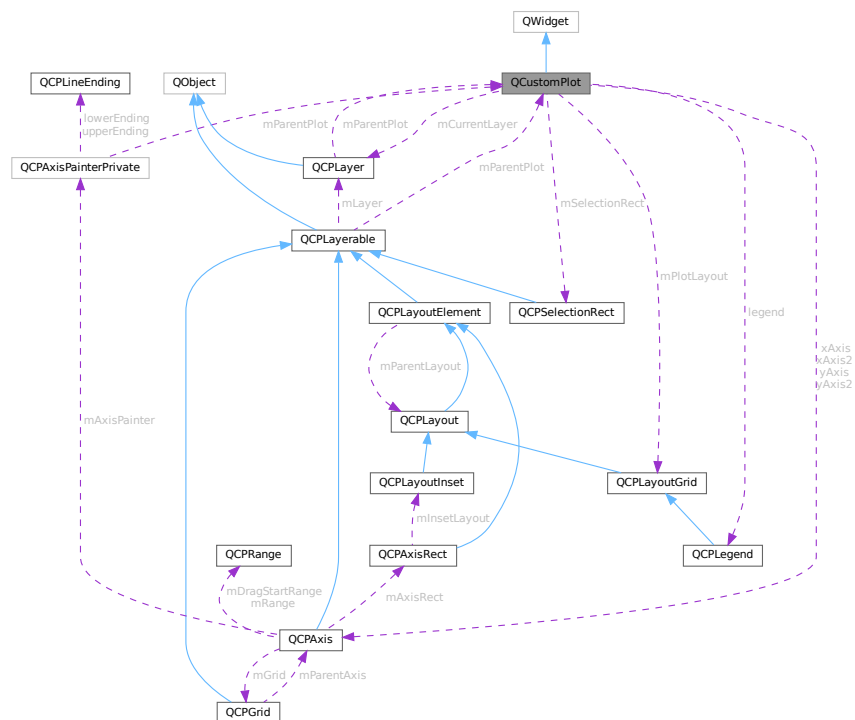
10.469 QCustomPlot Class Reference

The central class of the library. This is the QWidget which displays the plot and interacts with the user.

Inheritance diagram for QCustomPlot:



Collaboration diagram for QCustomPlot:



Public Types

- enum [LayerInsertMode](#) { `limBelow` , `limAbove` }
- enum [RefreshPriority](#) { `rplImmediateRefresh` , `rpQueuedRefresh` , `rpRefreshHint` , `rpQueuedReplot` }

Signals

- void [mouseDoubleClick](#) (QMouseEvent *event)
- void [mousePress](#) (QMouseEvent *event)
- void [mouseMove](#) (QMouseEvent *event)
- void [mouseRelease](#) (QMouseEvent *event)
- void [mouseWheel](#) (QWheelEvent *event)
- void [plottableClick](#) (QCPAbstractPlottable *[plottable](#), int dataIndex, QMouseEvent *event)
- void [plottableDoubleClick](#) (QCPAbstractPlottable *[plottable](#), int dataIndex, QMouseEvent *event)
- void [itemClick](#) (QCPAbstractItem *[item](#), QMouseEvent *event)
- void [itemDoubleClick](#) (QCPAbstractItem *[item](#), QMouseEvent *event)
- void [axisClick](#) (QCPAxis *axis, QCPAxis::SelectablePart part, QMouseEvent *event)
- void [axisDoubleClick](#) (QCPAxis *axis, QCPAxis::SelectablePart part, QMouseEvent *event)
- void [legendClick](#) (QCPLegend *[legend](#), QCPAbstractLegendItem *[item](#), QMouseEvent *event)
- void [legendDoubleClick](#) (QCPLegend *[legend](#), QCPAbstractLegendItem *[item](#), QMouseEvent *event)
- void [selectionChangedByUser](#) ()
- void [beforeReplot](#) ()
- void [afterLayout](#) ()
- void [afterReplot](#) ()

Public Member Functions

- [QCustomPlot](#) (QWidget *parent=nullptr)
- QRect **viewport** () const
- double **bufferDevicePixelRatio** () const
- QPixmap **background** () const
- bool **backgroundScaled** () const
- Qt::AspectRatioMode **backgroundScaledMode** () const
- QCPLayoutGrid * [plotLayout](#) () const
- QCP::AntialiasedElements **antialiasedElements** () const
- QCP::AntialiasedElements **notAntialiasedElements** () const
- bool **autoAddPlottableToLegend** () const
- const QCP::Interactions **interactions** () const
- int **selectionTolerance** () const
- bool **noAntialiasingOnDrag** () const
- QCP::PlottingHints **plottingHints** () const
- Qt::KeyboardModifier **multiSelectModifier** () const
- QCP::SelectionRectMode **selectionRectMode** () const
- QCPSelectionRect * [selectionRect](#) () const
- bool **openGL** () const
- void [setViewport](#) (const QRect &rect)
- void [setBufferDevicePixelRatio](#) (double ratio)
- void [setBackground](#) (const QPixmap &pm)
- void [setBackground](#) (const QPixmap &pm, bool scaled, Qt::AspectRatioMode mode=Qt::KeepAspectRatio$\leftrightarrow$ByExpanding)
- void [setBackground](#) (const QBrush &brush)
- void [setBackgroundScaled](#) (bool scaled)
- void [setBackgroundScaledMode](#) (Qt::AspectRatioMode mode)
- void [setAntialiasedElements](#) (const QCP::AntialiasedElements &antialiasedElements)
- void [setAntialiasedElement](#) (QCP::AntialiasedElement antialiasedElement, bool enabled=true)
- void [setNotAntialiasedElements](#) (const QCP::AntialiasedElements ¬AntialiasedElements)
- void [setNotAntialiasedElement](#) (QCP::AntialiasedElement notAntialiasedElement, bool enabled=true)
- void [setAutoAddPlottableToLegend](#) (bool on)
- void [setInteractions](#) (const QCP::Interactions &interactions)

- void [setInteraction](#) (const [QCP::Interaction](#) &interaction, bool enabled=true)
- void [setSelectionTolerance](#) (int pixels)
- void [setNoAntialiasingOnDrag](#) (bool enabled)
- void [setPlottingHints](#) (const [QCP::PlottingHints](#) &hints)
- void [setPlottingHint](#) ([QCP::PlottingHint](#) hint, bool enabled=true)
- void [setMultiSelectModifier](#) (Qt::KeyboardModifier modifier)
- void [setSelectionRectMode](#) ([QCP::SelectionRectMode](#) mode)
- void [setSelectionRect](#) ([QCPSelectionRect](#) *selectionRect)
- void [setOpenGL](#) (bool enabled, int multisampling=16)
- [QCPAbstractPlottable](#) * [plottable](#) (int index)
- [QCPAbstractPlottable](#) * [plottable](#) ()
- bool [removePlottable](#) ([QCPAbstractPlottable](#) *plottable)
- bool [removePlottable](#) (int index)
- int [clearPlottables](#) ()
- int [plottableCount](#) () const
- [QList](#)< [QCPAbstractPlottable](#) * > [selectedPlottables](#) () const
- template<class PlottableType >
PlottableType * [plottableAt](#) (const [QPointF](#) &pos, bool onlySelectable=false, int *dataIndex=nullptr) const
- [QCPAbstractPlottable](#) * [plottableAt](#) (const [QPointF](#) &pos, bool onlySelectable=false, int *dataIndex=nullptr) const
- bool [hasPlottable](#) ([QCPAbstractPlottable](#) *plottable) const
- [QCPGraph](#) * [graph](#) (int index) const
- [QCPGraph](#) * [graph](#) () const
- [QCPGraph](#) * [addGraph](#) ([QCPAxis](#) *keyAxis=nullptr, [QCPAxis](#) *valueAxis=nullptr)
- bool [removeGraph](#) ([QCPGraph](#) *graph)
- bool [removeGraph](#) (int index)
- int [clearGraphs](#) ()
- int [graphCount](#) () const
- [QList](#)< [QCPGraph](#) * > [selectedGraphs](#) () const
- [QCPAbstractItem](#) * [item](#) (int index) const
- [QCPAbstractItem](#) * [item](#) () const
- bool [removeItem](#) ([QCPAbstractItem](#) *item)
- bool [removeItem](#) (int index)
- int [clearItems](#) ()
- int [itemCount](#) () const
- [QList](#)< [QCPAbstractItem](#) * > [selectedItems](#) () const
- template<class ItemType >
ItemType * [itemAt](#) (const [QPointF](#) &pos, bool onlySelectable=false) const
- [QCPAbstractItem](#) * [itemAt](#) (const [QPointF](#) &pos, bool onlySelectable=false) const
- bool [hasItem](#) ([QCPAbstractItem](#) *item) const
- [QCPLayer](#) * [layer](#) (const [QString](#) &name) const
- [QCPLayer](#) * [layer](#) (int index) const
- [QCPLayer](#) * [currentLayer](#) () const
- bool [setCurrentLayer](#) (const [QString](#) &name)
- bool [setCurrentLayer](#) ([QCPLayer](#) *layer)
- int [layerCount](#) () const
- bool [addLayer](#) (const [QString](#) &name, [QCPLayer](#) *otherLayer=nullptr, [LayerInsertMode](#) insertMode=[limAbove](#))
- bool [removeLayer](#) ([QCPLayer](#) *layer)
- bool [moveLayer](#) ([QCPLayer](#) *layer, [QCPLayer](#) *otherLayer, [LayerInsertMode](#) insertMode=[limAbove](#))
- int [axisRectCount](#) () const
- [QCPAxisRect](#) * [axisRect](#) (int index=0) const
- [QList](#)< [QCPAxisRect](#) * > [axisRects](#) () const
- [QCPLayoutElement](#) * [layoutElementAt](#) (const [QPointF](#) &pos) const
- [QCPAxisRect](#) * [axisRectAt](#) (const [QPointF](#) &pos) const

- Q_SLOT void [rescaleAxes](#) (bool onlyVisiblePlottables=false)
- QList< [QCPAxis](#) * > [selectedAxes](#) () const
- QList< [QCPLegend](#) * > [selectedLegends](#) () const
- Q_SLOT void [deselectAll](#) ()
- bool [savePdf](#) (const QString &fileName, int width=0, int height=0, [QCP::ExportPen](#) exportPen=[QCP::epAllowCosmetic](#), const QString &pdfCreator=QString(), const QString &pdfTitle=QString())
- bool [savePng](#) (const QString &fileName, int width=0, int height=0, double scale=1.0, int quality=-1, int resolution=96, [QCP::ResolutionUnit](#) resolutionUnit=[QCP::ruDotsPerInch](#))
- bool [saveJpg](#) (const QString &fileName, int width=0, int height=0, double scale=1.0, int quality=-1, int resolution=96, [QCP::ResolutionUnit](#) resolutionUnit=[QCP::ruDotsPerInch](#))
- bool [saveBmp](#) (const QString &fileName, int width=0, int height=0, double scale=1.0, int resolution=96, [QCP::ResolutionUnit](#) resolutionUnit=[QCP::ruDotsPerInch](#))
- bool [saveRastered](#) (const QString &fileName, int width, int height, double scale, const char *format, int quality=-1, int resolution=96, [QCP::ResolutionUnit](#) resolutionUnit=[QCP::ruDotsPerInch](#))
- QPixmap [toPixmap](#) (int width=0, int height=0, double scale=1.0)
- void [toPainter](#) ([QCPPainter](#) *painter, int width=0, int height=0)
- Q_SLOT void [replot](#) ([QCustomPlot::RefreshPriority](#) refreshPriority=[QCustomPlot::rpRefreshHint](#))
- double [replotTime](#) (bool average=false) const

Public Attributes

- [QCPAxis](#) * [xAxis](#)
- [QCPAxis](#) * [yAxis](#)
- [QCPAxis](#) * [xAxis2](#)
- [QCPAxis](#) * [yAxis2](#)
- [QCPLegend](#) * [legend](#)

Protected Member Functions

- virtual QSize [minimumSizeHint](#) () const Q_DECL_OVERRIDE
- virtual QSize [sizeHint](#) () const Q_DECL_OVERRIDE
- virtual void [paintEvent](#) (QPaintEvent *event) Q_DECL_OVERRIDE
- virtual void [resizeEvent](#) (QResizeEvent *event) Q_DECL_OVERRIDE
- virtual void [mouseDoubleClickEvent](#) (QMouseEvent *event) Q_DECL_OVERRIDE
- virtual void [mousePressEvent](#) (QMouseEvent *event) Q_DECL_OVERRIDE
- virtual void [mouseMoveEvent](#) (QMouseEvent *event) Q_DECL_OVERRIDE
- virtual void [mouseReleaseEvent](#) (QMouseEvent *event) Q_DECL_OVERRIDE
- virtual void [wheelEvent](#) (QWheelEvent *event) Q_DECL_OVERRIDE
- virtual void [draw](#) ([QCPPainter](#) *painter)
- virtual void [updateLayout](#) ()
- virtual void [axisRemoved](#) ([QCPAxis](#) *axis)
- virtual void [legendRemoved](#) ([QCPLegend](#) *legend)
- virtual Q_SLOT void [processRectSelection](#) (QRect rect, QMouseEvent *event)
- virtual Q_SLOT void [processRectZoom](#) (QRect rect, QMouseEvent *event)
- virtual Q_SLOT void [processPointSelection](#) (QMouseEvent *event)
- bool [registerPlottable](#) ([QCPAbstractPlottable](#) *plottable)
- bool [registerGraph](#) ([QCPGraph](#) *graph)
- bool [registerItem](#) ([QCPAbstractItem](#) *item)
- void [updateLayerIndices](#) () const
- [QCPLayerable](#) * [layerableAt](#) (const QPointF &pos, bool onlySelectable, QVariant *selectionDetails=nullptr) const
- QList< [QCPLayerable](#) * > [layerableListAt](#) (const QPointF &pos, bool onlySelectable, QList< QVariant > *selectionDetails=nullptr) const

- void **drawBackground** ([QCPPainter](#) *painter)
- void **setupPaintBuffers** ()
- [QCPAbstractPaintBuffer](#) * **createPaintBuffer** ()
- bool **hasInvalidatedPaintBuffers** ()
- bool **setupOpenGL** ()
- void **freeOpenGL** ()

Protected Attributes

- [QRect](#) **mViewport**
- double **mBufferDevicePixelRatio**
- [QCPLayoutGrid](#) * **mPlotLayout**
- bool **mAutoAddPlottableToLegend**
- [QList](#)< [QCPAbstractPlottable](#) * > **mPlottables**
- [QList](#)< [QCPGraph](#) * > **mGraphs**
- [QList](#)< [QCPAbstractItem](#) * > **mItems**
- [QList](#)< [QCPLayer](#) * > **mLayers**
- [QCP::AntialiasedElements](#) **mAntialiasedElements**
- [QCP::AntialiasedElements](#) **mNotAntialiasedElements**
- [QCP::Interactions](#) **mInteractions**
- int **mSelectionTolerance**
- bool **mNoAntialiasingOnDrag**
- [QBrush](#) **mBackgroundBrush**
- [QPixmap](#) **mBackgroundPixmap**
- [QPixmap](#) **mScaledBackgroundPixmap**
- bool **mBackgroundScaled**
- [Qt::AspectRatioMode](#) **mBackgroundScaledMode**
- [QCPLayer](#) * **mCurrentLayer**
- [QCP::PlottingHints](#) **mPlottingHints**
- [Qt::KeyboardModifier](#) **mMultiSelectModifier**
- [QCP::SelectionRectMode](#) **mSelectionRectMode**
- [QCPSelectionRect](#) * **mSelectionRect**
- bool **mOpenGL**
- [QList](#)< [QSharedPointer](#)< [QCPAbstractPaintBuffer](#) > > **mPaintBuffers**
- [QPoint](#) **mMousePressPos**
- bool **mMouseHasMoved**
- [QPointer](#)< [QCPLayerable](#) > **mMouseEventLayerable**
- [QPointer](#)< [QCPLayerable](#) > **mMouseSignalLayerable**
- [QVariant](#) **mMouseEventLayerableDetails**
- [QVariant](#) **mMouseSignalLayerableDetails**
- bool **mReplotting**
- bool **mReplotQueued**
- double **mReplotTime**
- double **mReplotTimeAverage**
- int **mOpenGLMultisamples**
- [QCP::AntialiasedElements](#) **mOpenGLAntialiasedElementsBackup**
- bool **mOpenGLCacheLabelsBackup**

Friends

- class **QCPLegend**
- class **QCPAxis**
- class **QCPLayer**
- class **QCPAxisRect**
- class **QCPAbstractPlottable**
- class **QCPGraph**
- class **QCPAbstractItem**

10.469.1 Detailed Description

The central class of the library. This is the QWidget which displays the plot and interacts with the user.

For tutorials on how to use [QCustomPlot](#), see the website

<http://www.qcustomplot.com/>

10.469.2 Member Enumeration Documentation

10.469.2.1 LayerInsertMode

```
enum QCustomPlot::LayerInsertMode
```

Defines how a layer should be inserted relative to an other layer.

See also

[addLayer](#), [moveLayer](#)

Enumerator

limBelow	Layer is inserted below other layer.
limAbove	Layer is inserted above other layer.

10.469.2.2 RefreshPriority

```
enum QCustomPlot::RefreshPriority
```

Defines with what timing the [QCustomPlot](#) surface is refreshed after a replot.

See also

[replot](#)

Enumerator

rpImmediateRefresh	Replots immediately and repaints the widget immediately by calling QWidget::repaint() after the replot.
rpQueuedRefresh	Replots immediately, but queues the widget repaint, by calling QWidget::update() after the replot. This way multiple redundant widget repaints can be avoided.
rpRefreshHint	Whether to use immediate or queued refresh depends on whether the plotting hint QCP::phImmediateRefresh is set, see setPlottingHints .
rpQueuedReplot	Queues the entire replot for the next event loop iteration. This way multiple redundant replots can be avoided. The actual replot is then done with rpRefreshHint priority.

10.469.3 Constructor & Destructor Documentation

10.469.3.1 QCustomPlot()

```
QCustomPlot::QCustomPlot (
    QWidget * parent = nullptr ) [explicit]
```

Constructs a [QCustomPlot](#) and sets reasonable default values.

10.469.4 Member Function Documentation

10.469.4.1 addGraph()

```
QCPGraph * QCustomPlot::addGraph (
    QCPAxis * keyAxis = nullptr,
    QCPAxis * valueAxis = nullptr )
```

Creates a new graph inside the plot. If *keyAxis* and *valueAxis* are left unspecified (0), the bottom (xAxis) is used as key and the left (yAxis) is used as value axis. If specified, *keyAxis* and *valueAxis* must reside in this [QCustomPlot](#).

keyAxis will be used as key axis (typically "x") and *valueAxis* as value axis (typically "y") for the graph.

Returns a pointer to the newly created graph, or `nullptr` if adding the graph failed.

See also

[graph](#), [graphCount](#), [removeGraph](#), [clearGraphs](#)

10.469.4.2 addLayer()

```
bool QCustomPlot::addLayer (
    const QString & name,
    QCPLayer * otherLayer = nullptr,
    QCustomPlot::LayerInsertMode insertMode = limAbove )
```

Adds a new layer to this [QCustomPlot](#) instance. The new layer will have the name *name*, which must be unique. Depending on *insertMode*, it is positioned either below or above *otherLayer*.

Returns true on success, i.e. if there is no other layer named *name* and *otherLayer* is a valid layer inside this [QCustomPlot](#).

If *otherLayer* is 0, the highest layer in the [QCustomPlot](#) will be used.

For an explanation of what layers are in [QCustomPlot](#), see the documentation of [QCPLayer](#).

See also

[layer](#), [moveLayer](#), [removeLayer](#)

10.469.4.3 afterLayout

```
void QCustomPlot::afterLayout ( ) [signal]
```

This signal is emitted immediately after the layout step has been completed, which occurs right before drawing the plot. This is typically during a call to [replot](#), and in such cases this signal is emitted in between the signals [beforeReplot](#) and [afterReplot](#). Unlike those signals however, this signal is also emitted during off-screen painting, such as when calling [toPixmap](#) or [savePdf](#).

The layout step queries all layouts and layout elements in the plot for their proposed size and arranges the objects accordingly as preparation for the subsequent drawing step. Through this signal, you have the opportunity to update certain things in your plot that depend crucially on the exact dimensions/positioning of layout elements such as axes and axis rects.

Warning

However, changing any parameters of this [QCustomPlot](#) instance which would normally affect the layouting (e.g. axis range order of magnitudes, tick label sizes, etc.) will not issue a second run of the layout step. It will propagate directly to the draw step and may cause graphical inconsistencies such as overlapping objects, if sizes or positions have changed.

See also

[updateLayout](#), [beforeReplot](#), [afterReplot](#)

10.469.4.4 afterReplot

```
void QCustomPlot::afterReplot ( ) [signal]
```

This signal is emitted immediately after a replot has taken place (caused by a call to the slot [replot](#)).

It is safe to mutually connect the replot slot with this signal on two [QCustomPlots](#) to make them replot synchronously, it won't cause an infinite recursion.

See also

[replot](#), [beforeReplot](#), [afterLayout](#)

10.469.4.5 axisClick

```
void QCustomPlot::axisClick (
    QCPAxis * axis,
    QCPAxis::SelectablePart part,
    QMouseEvent * event ) [signal]
```

This signal is emitted when an axis is clicked.

event is the mouse event that caused the click, *axis* is the axis that received the click and *part* indicates the part of the axis that was clicked.

See also

[axisDoubleClick](#)

10.469.4.6 axisDoubleClick

```
void QCustomPlot::axisDoubleClick (
    QCPAxis * axis,
    QCPAxis::SelectablePart part,
    QMouseEvent * event ) [signal]
```

This signal is emitted when an axis is double clicked.

event is the mouse event that caused the click, *axis* is the axis that received the click and *part* indicates the part of the axis that was clicked.

See also

[axisClick](#)

10.469.4.7 axisRect()

```
QCPAxisRect * QCustomPlot::axisRect (
    int index = 0 ) const
```

Returns the axis rect with *index*.

Initially, only one axis rect (with index 0) exists in the plot. If multiple axis rects were added, all of them may be accessed with this function in a linear fashion (even when they are nested in a layout hierarchy or inside other axis rects via [QCPAxisRect::insetLayout](#)).

The order of the axis rects is given by the fill order of the [QCPLayout](#) that is holding them. For example, if the axis rects are in the top level grid layout (accessible via [QCustomPlot::plotLayout](#)), they are ordered from left to right, top to bottom, if the layout's default [setFillOrder](#) of [foColumnsFirst](#) wasn't changed.

If you want to access axis rects by their row and column index, use the layout interface. For example, use [QCPLayoutGrid::element](#) of the top level grid layout, and `qobject_cast` the returned layout element to [QCPAxisRect](#). (See also [thelayoutsystem](#).)

See also

[axisRectCount](#), [axisRects](#), [QCPLayoutGrid::setFillOrder](#)

10.469.4.8 axisRectAt()

```
QCPAxisRect * QCustomPlot::axisRectAt (
    const QPointF & pos ) const
```

Returns the layout element of type [QCPAxisRect](#) at pixel position *pos*. This method ignores other layout elements even if they are visually in front of the axis rect (e.g. a [QCPLegend](#)). If there is no axis rect at that position, returns `nullptr`.

Only visible axis rects are used. If [QCPLayoutElement::setVisible](#) on the axis rect itself or on any of its parent elements is set to false, it will not be considered.

See also

[layoutElementAt](#)

10.469.4.9 axisRectCount()

```
int QCustomPlot::axisRectCount ( ) const
```

Returns the number of axis rects in the plot.

All axis rects can be accessed via [QCustomPlot::axisRect\(\)](#).

Initially, only one axis rect exists in the plot.

See also

[axisRect](#), [axisRects](#)

10.469.4.10 axisRects()

```
QList< QCPAxisRect * > QCustomPlot::axisRects ( ) const
```

Returns all axis rects in the plot.

The order of the axis rects is given by the fill order of the [QCPLLayout](#) that is holding them. For example, if the axis rects are in the top level grid layout (accessible via [QCustomPlot::plotLayout](#)), they are ordered from left to right, top to bottom, if the layout's default [setFillOrder](#) of [foColumnsFirst](#) wasn't changed.

See also

[axisRectCount](#), [axisRect](#), [QCPLLayoutGrid::setFillOrder](#)

10.469.4.11 beforeReplot

```
void QCustomPlot::beforeReplot ( ) [signal]
```

This signal is emitted immediately before a replot takes place (caused by a call to the slot [replot](#)).

It is safe to mutually connect the replot slot with this signal on two QCustomPlots to make them replot synchronously, it won't cause an infinite recursion.

See also

[replot](#), [afterReplot](#), [afterLayout](#)

10.469.4.12 clearGraphs()

```
int QCustomPlot::clearGraphs ( )
```

Removes all graphs from the plot and deletes them. Corresponding legend items are also removed from the default legend ([QCustomPlot::legend](#)).

Returns the number of graphs removed.

See also

[removeGraph](#)

10.469.4.13 clearItems()

```
int QCustomPlot::clearItems ( )
```

Removes all items from the plot and deletes them.

Returns the number of items removed.

See also

[removeItem](#)

10.469.4.14 clearPlottables()

```
int QCustomPlot::clearPlottables ( )
```

Removes all plottables from the plot and deletes them. Corresponding legend items are also removed from the default legend ([QCustomPlot::legend](#)).

Returns the number of plottables removed.

See also

[removePlottable](#)

10.469.4.15 currentLayer()

```
QCPLayer * QCustomPlot::currentLayer ( ) const
```

Returns the layer that is set as current layer (see [setCurrentLayer](#)).

10.469.4.16 deselectAll()

```
void QCustomPlot::deselectAll ( )
```

Deselects all layerables (plottables, items, axes, legends,...) of the [QCustomPlot](#).

Since calling this function is not a user interaction, this does not emit the [selectionChangedByUser](#) signal. The individual selectionChanged signals are emitted though, if the objects were previously selected.

See also

[setInteractions](#), [selectedPlottables](#), [selectedItems](#), [selectedAxes](#), [selectedLegends](#)

10.469.4.17 graph() [1/2]

```
QCPGraph * QCustomPlot::graph ( ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the last graph, that was created with [addGraph](#). If there are no graphs in the plot, returns `nullptr`.

See also

[graphCount](#), [addGraph](#)

10.469.4.18 graph() [2/2]

```
QCPGraph * QCustomPlot::graph (
    int index ) const
```

Returns the graph with *index*. If the index is invalid, returns `nullptr`.

There is an overloaded version of this function with no parameter which returns the last created graph, see [QCustomPlot::graph\(\)](#)

See also

[graphCount](#), [addGraph](#)

10.469.4.19 graphCount()

```
int QCustomPlot::graphCount ( ) const
```

Returns the number of currently existing graphs in the plot

See also

[graph](#), [addGraph](#)

10.469.4.20 hasInvalidatedPaintBuffers()

```
bool QCustomPlot::hasInvalidatedPaintBuffers ( ) [protected]
```

This method returns whether any of the paint buffers held by this [QCustomPlot](#) instance are invalidated.

If any buffer is invalidated, a partial replot ([QCPLayer::replot](#)) is not allowed and always causes a full replot ([QCustomPlot::replot](#)) of all layers. This is the case when for example the layer order has changed, new layers were added or removed, layer modes were changed ([QCPLayer::setMode](#)), or layerables were added or removed.

See also

[QCPAbstractPaintBuffer::setInvalidated](#)

10.469.4.21 hasItem()

```
bool QCustomPlot::hasItem (
    QCPAbstractItem * item ) const
```

Returns whether this [QCustomPlot](#) contains the *item*.

See also

[item](#)

10.469.4.22 hasPlottable()

```
bool QCustomPlot::hasPlottable (
    QCPAbstractPlottable * plottable ) const
```

Returns whether this [QCustomPlot](#) instance contains the *plottable*.

10.469.4.23 item() [1/2]

```
QCPAbstractItem * QCustomPlot::item ( ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the last item that was added to this plot. If there are no items in the plot, returns `nullptr`.

See also

[itemCount](#)

10.469.4.24 item() [2/2]

```
QCPAbstractItem * QCustomPlot::item (
    int index ) const
```

Returns the item with *index*. If the index is invalid, returns `nullptr`.

There is an overloaded version of this function with no parameter which returns the last added item, see [QCustomPlot::item\(\)](#)

See also

[itemCount](#)

10.469.4.25 itemAt() [1/2]

```
template<class ItemType >
ItemType * QCustomPlot::itemAt (
    const QPointF & pos,
    bool onlySelectable = false ) const
```

Returns the item at the pixel position *pos*. The item type (a [QCPAbstractItem](#) subclass) that shall be taken into consideration can be specified via the template parameter. Items that only consist of single lines (e.g. [QCPItemLine](#) or [QCPItemCurve](#)) have a tolerance band around them, see [setSelectionTolerance](#). If multiple items come into consideration, the one closest to *pos* is returned.

If *onlySelectable* is true, only items that are selectable ([QCPAbstractItem::setSelectable](#)) are considered.

If there is no item at *pos*, returns `nullptr`.

See also

[plottableAt](#), [layoutElementAt](#)

10.469.4.26 itemAt() [2/2]

```
QCPAbstractItem * QCustomPlot::itemAt (
    const QPointF & pos,
    bool onlySelectable = false ) const
```

Returns the item at the pixel position *pos*. Since it can capture all items, the return type is the abstract base class of all items, [QCPAbstractItem](#).

For details, and if you wish to specify a certain item type (e.g. [QCPItemLine](#)), see the template method [itemAt<ItemType>\(\)](#)

See also

[itemAt<ItemType>\(\)](#), [plottableAt](#), [layoutElementAt](#)

10.469.4.27 itemClick

```
void QCustomPlot::itemClick (
    QCPAbstractItem * item,
    QMouseEvent * event ) [signal]
```

This signal is emitted when an item is clicked.

event is the mouse event that caused the click and *item* is the item that received the click.

See also

[itemDoubleClick](#)

10.469.4.28 itemCount()

```
int QCustomPlot::itemCount ( ) const
```

Returns the number of currently existing items in the plot

See also

[item](#)

10.469.4.29 itemDoubleClick

```
void QCustomPlot::itemDoubleClick (
    QCPAbstractItem * item,
    QMouseEvent * event ) [signal]
```

This signal is emitted when an item is double clicked.

event is the mouse event that caused the click and *item* is the item that received the click.

See also

[itemClick](#)

10.469.4.30 layer() [1/2]

```
QCPLayer * QCustomPlot::layer (
    const QString & name ) const
```

Returns the layer with the specified *name*. If there is no layer with the specified name, `nullptr` is returned.

Layer names are case-sensitive.

See also

[addLayer](#), [moveLayer](#), [removeLayer](#)

10.469.4.31 layer() [2/2]

```
QCPLayer * QCustomPlot::layer (
    int index ) const
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the layer by *index*. If the index is invalid, `nullptr` is returned.

See also

[addLayer](#), [moveLayer](#), [removeLayer](#)

10.469.4.32 layerCount()

```
int QCustomPlot::layerCount ( ) const
```

Returns the number of currently existing layers in the plot

See also

[layer](#), [addLayer](#)

10.469.4.33 layoutElementAt()

```
QCPLayoutElement * QCustomPlot::layoutElementAt (
    const QPointF & pos ) const
```

Returns the layout element at pixel position *pos*. If there is no element at that position, returns `nullptr`.

Only visible elements are used. If [QCPLayoutElement::setVisible](#) on the element itself or on any of its parent elements is set to false, it will not be considered.

See also

[itemAt](#), [plottableAt](#)

10.469.4.34 legendClick

```
void QCustomPlot::legendClick (
    QCPLegend * legend,
    QCPAbstractLegendItem * item,
    QMouseEvent * event ) [signal]
```

This signal is emitted when a legend (item) is clicked.

event is the mouse event that caused the click, *legend* is the legend that received the click and *item* is the legend item that received the click. If only the legend and no item is clicked, *item* is `nullptr`. This happens for a click inside the legend padding or the space between two items.

See also

[legendDoubleClick](#)

10.469.4.35 legendDoubleClick

```
void QCustomPlot::legendDoubleClick (
    QCPLegend * legend,
    QCPAbstractLegendItem * item,
    QMouseEvent * event ) [signal]
```

This signal is emitted when a legend (item) is double clicked.

event is the mouse event that caused the click, *legend* is the legend that received the click and *item* is the legend item that received the click. If only the legend and no item is clicked, *item* is `nullptr`. This happens for a click inside the legend padding or the space between two items.

See also

[legendClick](#)

10.469.4.36 mouseClicked

```
void QCustomPlot::mouseDoubleClick (
    QMouseEvent * event ) [signal]
```

This signal is emitted when the [QCustomPlot](#) receives a mouse double click event.

10.469.4.37 mouseMove

```
void QCustomPlot::mouseMove (
    QMouseEvent * event ) [signal]
```

This signal is emitted when the [QCustomPlot](#) receives a mouse move event.

It is emitted before [QCustomPlot](#) handles any other mechanism like range dragging. So a slot connected to this signal can still influence the behaviour e.g. with [QCPAxisRect::setRangeDrag](#) or [QCPAxisRect::setRangeDragAxes](#).

Warning

It is discouraged to change the drag-axes with [QCPAxisRect::setRangeDragAxes](#) here, because the dragging starting point was saved the moment the mouse was pressed. Thus it only has a meaning for the range drag axes that were set at that moment. If you want to change the drag axes, consider doing this in the [mousePress](#) signal instead.

10.469.4.38 mousePress

```
void QCustomPlot::mousePress (
    QMouseEvent * event ) [signal]
```

This signal is emitted when the [QCustomPlot](#) receives a mouse press event.

It is emitted before [QCustomPlot](#) handles any other mechanism like range dragging. So a slot connected to this signal can still influence the behaviour e.g. with [QCPAxisRect::setRangeDrag](#) or [QCPAxisRect::setRangeDragAxes](#).

10.469.4.39 mouseRelease

```
void QCustomPlot::mouseRelease (
    QMouseEvent * event ) [signal]
```

This signal is emitted when the [QCustomPlot](#) receives a mouse release event.

It is emitted before [QCustomPlot](#) handles any other mechanisms like object selection. So a slot connected to this signal can still influence the behaviour e.g. with [setInteractions](#) or [QCPAbstractPlottable::setSelectable](#).

10.469.4.40 mouseWheel

```
void QCustomPlot::mouseWheel (
    QWheelEvent * event ) [signal]
```

This signal is emitted when the [QCustomPlot](#) receives a mouse wheel event.

It is emitted before [QCustomPlot](#) handles any other mechanisms like range zooming. So a slot connected to this signal can still influence the behaviour e.g. with [QCPAxisRect::setRangeZoom](#), [QCPAxisRect::setRangeZoomAxes](#) or [QCPAxisRect::setRangeZoomFactor](#).

10.469.4.41 `moveLayer()`

```
bool QCustomPlot::moveLayer (
    QCPLayer * layer,
    QCPLayer * otherLayer,
    QCustomPlot::LayerInsertMode insertMode = limAbove )
```

Moves the specified *layer* either above or below *otherLayer*. Whether it's placed above or below is controlled with *insertMode*.

Returns true on success, i.e. when both *layer* and *otherLayer* are valid layers in the [QCustomPlot](#).

See also

[layer](#), [addLayer](#), [moveLayer](#)

10.469.4.42 `plotLayout()`

```
QCPLayoutGrid * QCustomPlot::plotLayout ( ) const [inline]
```

Returns the top level layout of this [QCustomPlot](#) instance. It is a [QCPLayoutGrid](#), initially containing just one cell with the main [QCPAxisRect](#) inside.

10.469.4.43 `plottable()` [1/2]

```
QCPAbstractPlottable * QCustomPlot::plottable ( )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the last plottable that was added to the plot. If there are no plottables in the plot, returns `nullptr`.

See also

[plottableCount](#)

10.469.4.44 `plottable()` [2/2]

```
QCPAbstractPlottable * QCustomPlot::plottable (
    int index )
```

Returns the plottable with *index*. If the index is invalid, returns `nullptr`.

There is an overloaded version of this function with no parameter which returns the last added plottable, see [QCustomPlot::plottable\(\)](#)

See also

[plottableCount](#)

10.469.4.45 `plottableAt()` [1/2]

```
template<class PlottableType >
PlottableType * QCustomPlot::plottableAt (
    const QPointF & pos,
    bool onlySelectable = false,
    int * dataIndex = nullptr ) const
```

Returns the plottable at the pixel position *pos*. The plottable type (a [QCPAbstractPlottable](#) subclass) that shall be taken into consideration can be specified via the template parameter.

Plottables that only consist of single lines (like graphs) have a tolerance band around them, see [setSelectionTolerance](#). If multiple plottables come into consideration, the one closest to *pos* is returned.

If *onlySelectable* is true, only plottables that are selectable ([QCPAbstractPlottable::setSelectable](#)) are considered.

if *dataIndex* is non-null, it is set to the index of the plottable's data point that is closest to *pos*.

If there is no plottable of the specified type at *pos*, returns `nullptr`.

See also

[itemAt](#), [layoutElementAt](#)

10.469.4.46 `plottableAt()` [2/2]

```
QCPAbstractPlottable * QCustomPlot::plottableAt (
    const QPointF & pos,
    bool onlySelectable = false,
    int * dataIndex = nullptr ) const
```

Returns any plottable at the pixel position *pos*. Since it can capture all plottables, the return type is the abstract base class of all plottables, [QCPAbstractPlottable](#).

For details, and if you wish to specify a certain plottable type (e.g. [QCPGraph](#)), see the template method [plottableAt<PlottableType>\(\)](#)

See also

[plottableAt<PlottableType>\(\)](#), [itemAt](#), [layoutElementAt](#)

10.469.4.47 `plottableClick`

```
void QCustomPlot::plottableClick (
    QCPAbstractPlottable * plottable,
    int dataIndex,
    QMouseEvent * event ) [signal]
```

This signal is emitted when a plottable is clicked.

event is the mouse event that caused the click and *plottable* is the plottable that received the click. The parameter *dataIndex* indicates the data point that was closest to the click position.

See also

[plottableDoubleClick](#)

10.469.4.48 `plottableCount()`

```
int QCustomPlot::plottableCount ( ) const
```

Returns the number of currently existing plottables in the plot

See also

[plottable](#)

10.469.4.49 `plottableDoubleClick`

```
void QCustomPlot::plottableDoubleClick (
    QCPAbstractPlottable * plottable,
    int dataIndex,
    QMouseEvent * event ) [signal]
```

This signal is emitted when a plottable is double clicked.

event is the mouse event that caused the click and *plottable* is the plottable that received the click. The parameter *dataIndex* indicates the data point that was closest to the click position.

See also

[plottableClick](#)

10.469.4.50 `removeGraph()` [1/2]

```
bool QCustomPlot::removeGraph (
    int index )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes and deletes the graph by its *index*.

10.469.4.51 `removeGraph()` [2/2]

```
bool QCustomPlot::removeGraph (
    QCPGraph * graph )
```

Removes the specified *graph* from the plot and deletes it. If necessary, the corresponding legend item is also removed from the default legend ([QCustomPlot::legend](#)). If any other graphs in the plot have a channel fill set towards the removed graph, the channel fill property of those graphs is reset to `nullptr` (no channel fill).

Returns true on success.

See also

[clearGraphs](#)

10.469.4.52 removeItem() [1/2]

```
bool QCustomPlot::removeItem (
    int index )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes and deletes the item by its *index*.

10.469.4.53 removeItem() [2/2]

```
bool QCustomPlot::removeItem (
    QCPAbstractItem * item )
```

Removes the specified item from the plot and deletes it.

Returns true on success.

See also

[clearItems](#)

10.469.4.54 removeLayer()

```
bool QCustomPlot::removeLayer (
    QCPLayer * layer )
```

Removes the specified *layer* and returns true on success.

All layerables (e.g. plottables and items) on the removed layer will be moved to the layer below *layer*. If *layer* is the bottom layer, the layerables are moved to the layer above. In both cases, the total rendering order of all layerables in the [QCustomPlot](#) is preserved.

If *layer* is the current layer ([setCurrentLayer](#)), the layer below (or above, if bottom layer) becomes the new current layer.

It is not possible to remove the last layer of the plot.

See also

[layer](#), [addLayer](#), [moveLayer](#)

10.469.4.55 removePlottable() [1/2]

```
bool QCustomPlot::removePlottable (
    int index )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Removes and deletes the plottable by its *index*.

10.469.4.56 `removePlottable()` [2/2]

```
bool QCustomPlot::removePlottable (
    QCPAbstractPlottable * plottable )
```

Removes the specified plottable from the plot and deletes it. If necessary, the corresponding legend item is also removed from the default legend ([QCustomPlot::legend](#)).

Returns true on success.

See also

[clearPlottables](#)

10.469.4.57 `replot()`

```
void QCustomPlot::replot (
    QCustomPlot::RefreshPriority refreshPriority = QCustomPlot::rpRefreshHint )
```

Causes a complete replot into the internal paint buffer(s). Finally, the widget surface is refreshed with the new buffer contents. This is the method that must be called to make changes to the plot, e.g. on the axis ranges or data points of graphs, visible.

The parameter *refreshPriority* can be used to fine-tune the timing of the replot. For example if your application calls [replot](#) very quickly in succession (e.g. multiple independent functions change some aspects of the plot and each wants to make sure the change gets replotted), it is advisable to set *refreshPriority* to [QCustomPlot::rpQueuedReplot](#). This way, the actual replotting is deferred to the next event loop iteration. Multiple successive calls of [replot](#) with this priority will only cause a single replot, avoiding redundant replots and improving performance.

Under a few circumstances, [QCustomPlot](#) causes a replot by itself. Those are resize events of the [QCustomPlot](#) widget and user interactions (object selection and range dragging/zooming).

Before the replot happens, the signal [beforeReplot](#) is emitted. After the replot, [afterReplot](#) is emitted. It is safe to mutually connect the replot slot with any of those two signals on two [QCustomPlots](#) to make them replot synchronously, it won't cause an infinite recursion.

If a layer is in mode [QCPLayer::lmBuffered](#) ([QCPLayer::setMode](#)), it is also possible to replot only that specific layer via [QCPLayer::replot](#). See the documentation there for details.

See also

[replotTime](#)

10.469.4.58 `replotTime()`

```
double QCustomPlot::replotTime (
    bool average = false ) const
```

Returns the time in milliseconds that the last replot took. If *average* is set to true, an exponential moving average over the last couple of replots is returned.

See also

[replot](#)

10.469.4.59 rescaleAxes()

```
void QCustomPlot::rescaleAxes (
    bool onlyVisiblePlottables = false )
```

Rescales the axes such that all plottables (like graphs) in the plot are fully visible.

if *onlyVisiblePlottables* is set to true, only the plottables that have their visibility set to true ([QCPLayerable::setVisible](#)), will be used to rescale the axes.

See also

[QCPAbstractPlottable::rescaleAxes](#), [QCPAxis::rescale](#)

10.469.4.60 saveBmp()

```
bool QCustomPlot::saveBmp (
    const QString & fileName,
    int width = 0,
    int height = 0,
    double scale = 1.0,
    int resolution = 96,
    QCP::ResolutionUnit resolutionUnit = QCP::ruDotsPerInch )
```

Saves a BMP image file to *fileName* on disc. The output plot will have the dimensions *width* and *height* in pixels, multiplied by *scale*. If either *width* or *height* is zero, the current width and height of the [QCustomPlot](#) widget is used instead. Line widths and texts etc. are not scaled up when larger widths/heights are used. If you want that effect, use the *scale* parameter.

For example, if you set both *width* and *height* to 100 and *scale* to 2, you will end up with an image file of size 200*200 in which all graphical elements are scaled up by factor 2 (line widths, texts, etc.). This scaling is not done by stretching a 100*100 image, the result will have full 200*200 pixel resolution.

If you use a high scaling factor, it is recommended to enable antialiasing for all elements by temporarily setting [QCustomPlot::setAntialiasedElements](#) to [QCP::aeAll](#) as this allows [QCustomPlot](#) to place objects with sub-pixel accuracy.

The *resolution* will be written to the image file header and has no direct consequence for the quality or the pixel size. However, if opening the image with a tool which respects the metadata, it will be able to scale the image to match either a given size in real units of length (inch, centimeters, etc.), or the target display DPI. You can specify in which units *resolution* is given, by setting *resolutionUnit*. The *resolution* is converted to the format's expected resolution unit internally.

Returns true on success. If this function fails, most likely the BMP format isn't supported by the system, see Qt docs about [QImageWriter::supportedImageFormats\(\)](#).

The objects of the plot will appear in the current selection state. If you don't want any selected objects to be painted in their selected look, deselect everything with [deselectAll](#) before calling this function.

Warning

If calling this function inside the constructor of the parent of the [QCustomPlot](#) widget (i.e. the [MainWindow](#) constructor, if [QCustomPlot](#) is inside the [MainWindow](#)), always provide explicit non-zero widths and heights. If you leave *width* or *height* as 0 (default), this function uses the current width and height of the [QCustomPlot](#) widget. However, in Qt, these aren't defined yet inside the constructor, so you would get an image that has strange widths/heights.

See also

[savePdf](#), [savePng](#), [saveJpg](#), [saveRastered](#)

10.469.4.61 `saveJpg()`

```
bool QCustomPlot::saveJpg (
    const QString & fileName,
    int width = 0,
    int height = 0,
    double scale = 1.0,
    int quality = -1,
    int resolution = 96,
    QCP::ResolutionUnit resolutionUnit = QCP::ruDotsPerInch )
```

Saves a JPEG image file to *fileName* on disc. The output plot will have the dimensions *width* and *height* in pixels, multiplied by *scale*. If either *width* or *height* is zero, the current width and height of the `QCustomPlot` widget is used instead. Line widths and texts etc. are not scaled up when larger widths/heights are used. If you want that effect, use the *scale* parameter.

For example, if you set both *width* and *height* to 100 and *scale* to 2, you will end up with an image file of size 200*200 in which all graphical elements are scaled up by factor 2 (line widths, texts, etc.). This scaling is not done by stretching a 100*100 image, the result will have full 200*200 pixel resolution.

If you use a high scaling factor, it is recommended to enable antialiasing for all elements by temporarily setting `QCustomPlot::setAntialiasedElements` to `QCP::aeAll` as this allows `QCustomPlot` to place objects with sub-pixel accuracy.

image compression can be controlled with the *quality* parameter which must be between 0 and 100 or -1 to use the default setting.

The *resolution* will be written to the image file header and has no direct consequence for the quality or the pixel size. However, if opening the image with a tool which respects the metadata, it will be able to scale the image to match either a given size in real units of length (inch, centimeters, etc.), or the target display DPI. You can specify in which units *resolution* is given, by setting *resolutionUnit*. The *resolution* is converted to the format's expected resolution unit internally.

Returns true on success. If this function fails, most likely the JPEG format isn't supported by the system, see Qt docs about `QImageWriter::supportedImageFormats()`.

The objects of the plot will appear in the current selection state. If you don't want any selected objects to be painted in their selected look, deselect everything with `deselectAll` before calling this function.

Warning

If calling this function inside the constructor of the parent of the `QCustomPlot` widget (i.e. the `MainWindow` constructor, if `QCustomPlot` is inside the `MainWindow`), always provide explicit non-zero widths and heights. If you leave *width* or *height* as 0 (default), this function uses the current width and height of the `QCustomPlot` widget. However, in Qt, these aren't defined yet inside the constructor, so you would get an image that has strange widths/heights.

See also

[savePdf](#), [savePng](#), [saveBmp](#), [saveRastered](#)

10.469.4.62 savePdf()

```
bool QCustomPlot::savePdf (
    const QString & fileName,
    int width = 0,
    int height = 0,
    QCP::ExportPen exportPen = QCP::epAllowCosmetic,
    const QString & pdfCreator = QString(),
    const QString & pdfTitle = QString() )
```

Saves a PDF with the vectorized plot to the file *fileName*. The axis ratio as well as the scale of texts and lines will be derived from the specified *width* and *height*. This means, the output will look like the normal on-screen output of a [QCustomPlot](#) widget with the corresponding pixel width and height. If either *width* or *height* is zero, the exported image will have the same dimensions as the [QCustomPlot](#) widget currently has.

Setting *exportPen* to [QCP::epNoCosmetic](#) allows to disable the use of cosmetic pens when drawing to the PDF file. Cosmetic pens are pens with numerical width 0, which are always drawn as a one pixel wide line, no matter what zoom factor is set in the PDF-Viewer. For more information about cosmetic pens, see the QPainter and QPen documentation.

The objects of the plot will appear in the current selection state. If you don't want any selected objects to be painted in their selected look, deselect everything with [deselectAll](#) before calling this function.

Returns true on success.

Warning

- If you plan on editing the exported PDF file with a vector graphics editor like Inkscape, it is advised to set *exportPen* to [QCP::epNoCosmetic](#) to avoid losing those cosmetic lines (which might be quite many, because cosmetic pens are the default for e.g. axes and tick marks).
- If calling this function inside the constructor of the parent of the [QCustomPlot](#) widget (i.e. the [MainWindow](#) constructor, if [QCustomPlot](#) is inside the [MainWindow](#)), always provide explicit non-zero widths and heights. If you leave *width* or *height* as 0 (default), this function uses the current width and height of the [QCustomPlot](#) widget. However, in Qt, these aren't defined yet inside the constructor, so you would get an image that has strange widths/heights.

pdfCreator and *pdfTitle* may be used to set the according metadata fields in the resulting PDF file.

Note

On Android systems, this method does nothing and issues an according qDebug warning message. This is also the case if for other reasons the define flag `QT_NO_PRINTER` is set.

See also

[savePng](#), [saveBmp](#), [saveJpg](#), [saveRastered](#)

10.469.4.63 savePng()

```
bool QCustomPlot::savePng (
    const QString & fileName,
    int width = 0,
    int height = 0,
    double scale = 1.0,
    int quality = -1,
    int resolution = 96,
    QCP::ResolutionUnit resolutionUnit = QCP::ruDotsPerInch )
```

Saves a PNG image file to *fileName* on disc. The output plot will have the dimensions *width* and *height* in pixels, multiplied by *scale*. If either *width* or *height* is zero, the current width and height of the [QCustomPlot](#) widget is used instead. Line widths and texts etc. are not scaled up when larger widths/heights are used. If you want that effect, use the *scale* parameter.

For example, if you set both *width* and *height* to 100 and *scale* to 2, you will end up with an image file of size 200*200 in which all graphical elements are scaled up by factor 2 (line widths, texts, etc.). This scaling is not done by stretching a 100*100 image, the result will have full 200*200 pixel resolution.

If you use a high scaling factor, it is recommended to enable antialiasing for all elements by temporarily setting [QCustomPlot::setAntialiasedElements](#) to [QCP::aeAll](#) as this allows [QCustomPlot](#) to place objects with sub-pixel accuracy.

image compression can be controlled with the *quality* parameter which must be between 0 and 100 or -1 to use the default setting.

The *resolution* will be written to the image file header and has no direct consequence for the quality or the pixel size. However, if opening the image with a tool which respects the metadata, it will be able to scale the image to match either a given size in real units of length (inch, centimeters, etc.), or the target display DPI. You can specify in which units *resolution* is given, by setting *resolutionUnit*. The *resolution* is converted to the format's expected resolution unit internally.

Returns true on success. If this function fails, most likely the PNG format isn't supported by the system, see Qt docs about [QImageWriter::supportedImageFormats\(\)](#).

The objects of the plot will appear in the current selection state. If you don't want any selected objects to be painted in their selected look, deselect everything with [deselectAll](#) before calling this function.

If you want the PNG to have a transparent background, call [setBackground\(const QBrush &brush\)](#) with no brush ([Qt::NoBrush](#)) or a transparent color ([Qt::transparent](#)), before saving.

Warning

If calling this function inside the constructor of the parent of the [QCustomPlot](#) widget (i.e. the [MainWindow](#) constructor, if [QCustomPlot](#) is inside the [MainWindow](#)), always provide explicit non-zero widths and heights. If you leave *width* or *height* as 0 (default), this function uses the current width and height of the [QCustomPlot](#) widget. However, in Qt, these aren't defined yet inside the constructor, so you would get an image that has strange widths/heights.

See also

[savePdf](#), [saveBmp](#), [saveJpg](#), [saveRastered](#)

10.469.4.64 saveRastered()

```
bool QCustomPlot::saveRastered (
    const QString & fileName,
    int width,
    int height,
    double scale,
    const char * format,
    int quality = -1,
    int resolution = 96,
    QCP::ResolutionUnit resolutionUnit = QCP::ruDotsPerInch )
```

Saves the plot to a rastered image file *fileName* in the image format *format*. The plot is sized to *width* and *height* in pixels and scaled with *scale*. (width 100 and scale 2.0 lead to a full resolution file with width 200.) If the *format* supports compression, *quality* may be between 0 and 100 to control it.

Returns true on success. If this function fails, most likely the given *format* isn't supported by the system, see Qt docs about `QImageWriter::supportedImageFormats()`.

The *resolution* will be written to the image file header (if the file format supports this) and has no direct consequence for the quality or the pixel size. However, if opening the image with a tool which respects the metadata, it will be able to scale the image to match either a given size in real units of length (inch, centimeters, etc.), or the target display DPI. You can specify in which units *resolution* is given, by setting *resolutionUnit*. The *resolution* is converted to the format's expected resolution unit internally.

See also

[saveBmp](#), [saveJpg](#), [savePng](#), [savePdf](#)

10.469.4.65 selectedAxes()

```
QList< QCPAxis * > QCustomPlot::selectedAxes ( ) const
```

Returns the axes that currently have selected parts, i.e. whose selection state is not `QCPAxis::spNone`.

See also

[selectedPlottables](#), [selectedLegends](#), [setInteractions](#), [QCPAxis::setSelectedParts](#), [QCPAxis::setSelectableParts](#)

10.469.4.66 selectedGraphs()

```
QList< QCPGraph * > QCustomPlot::selectedGraphs ( ) const
```

Returns a list of the selected graphs. If no graphs are currently selected, the list is empty.

If you are not only interested in selected graphs but other plottables like [QCPCurve](#), [QCPBars](#), etc., use [selectedPlottables](#).

See also

[setInteractions](#), [selectedPlottables](#), [QCPAbstractPlottable::setSelectable](#), [QCPAbstractPlottable::setSelection](#)

10.469.4.67 selectedItems()

```
QList< QCPAbstractItem * > QCustomPlot::selectedItems ( ) const
```

Returns a list of the selected items. If no items are currently selected, the list is empty.

See also

[setInteractions](#), [QCPAbstractItem::setSelectable](#), [QCPAbstractItem::setSelected](#)

10.469.4.68 selectedLegends()

```
QList< QCPLegend * > QCustomPlot::selectedLegends ( ) const
```

Returns the legends that currently have selected parts, i.e. whose selection state is not [QCPLegend::spNone](#).

See also

[selectedPlottables](#), [selectedAxes](#), [setInteractions](#), [QCPLegend::setSelectedParts](#), [QCPLegend::setSelectableParts](#), [QCPLegend::selectedItems](#)

10.469.4.69 selectedPlottables()

```
QList< QCPAbstractPlottable * > QCustomPlot::selectedPlottables ( ) const
```

Returns a list of the selected plottables. If no plottables are currently selected, the list is empty.

There is a convenience function if you're only interested in selected graphs, see [selectedGraphs](#).

See also

[setInteractions](#), [QCPAbstractPlottable::setSelectable](#), [QCPAbstractPlottable::setSelection](#)

10.469.4.70 selectionChangedByUser

```
void QCustomPlot::selectionChangedByUser ( ) [signal]
```

This signal is emitted after the user has changed the selection in the [QCustomPlot](#), e.g. by clicking. It is not emitted when the selection state of an object has changed programmatically by a direct call to `setSelected()/setSelection()` on an object or by calling [deselectAll](#).

In addition to this signal, selectable objects also provide individual signals, for example [QCPAxis::selectionChanged](#) or [QCPAbstractPlottable::selectionChanged](#). Note that those signals are emitted even if the selection state is changed programmatically.

See the documentation of [setInteractions](#) for details about the selection mechanism.

See also

[selectedPlottables](#), [selectedGraphs](#), [selectedItems](#), [selectedAxes](#), [selectedLegends](#)

10.469.4.71 selectionRect()

```
QCPSelectionRect * QCustomPlot::selectionRect ( ) const [inline]
```

Allows access to the currently used [QCPSelectionRect](#) instance (or subclass thereof), that is used to handle and draw selection rect interactions (see [setSelectionRectMode](#)).

See also

[setSelectionRect](#)

10.469.4.72 setAntialiasedElement()

```
void QCustomPlot::setAntialiasedElement (
    QCP::AntialiasedElement antialiasedElement,
    bool enabled = true )
```

Sets whether the specified *antialiasedElement* is forcibly drawn antialiased.

See [setAntialiasedElements](#) for details.

See also

[setNotAntialiasedElement](#)

10.469.4.73 setAntialiasedElements()

```
void QCustomPlot::setAntialiasedElements (
    const QCP::AntialiasedElements & antialiasedElements )
```

Sets which elements are forcibly drawn antialiased as an *or* combination of [QCP::AntialiasedElement](#).

This overrides the antialiasing settings for whole element groups, normally controlled with the *setAntialiasing* function on the individual elements. If an element is neither specified in [setAntialiasedElements](#) nor in [setNotAntialiasedElements](#), the antialiasing setting on each individual element instance is used.

For example, if *antialiasedElements* contains [QCP::aePlottables](#), all plottables will be drawn antialiased, no matter what the specific [QCPAbstractPlottable::setAntialiased](#) value was set to.

if an element in *antialiasedElements* is already set in [setNotAntialiasedElements](#), it is removed from there.

See also

[setNotAntialiasedElements](#)

10.469.4.74 `setAutoAddPlottableToLegend()`

```
void QCustomPlot::setAutoAddPlottableToLegend (
    bool on )
```

If set to true, adding a plottable (e.g. a graph) to the [QCustomPlot](#) automatically also adds the plottable to the legend ([QCustomPlot::legend](#)).

See also

[addGraph](#), [QCPLegend::addItem](#)

10.469.4.75 `setBackground()` [1/3]

```
void QCustomPlot::setBackground (
    const QBrush & brush )
```

Sets the background brush of the viewport (see [setViewport](#)).

Before drawing everything else, the background is filled with *brush*. If a background pixmap was set with [setBackground\(const QPixmap &pm\)](#), this brush will be used to fill the viewport before the background pixmap is drawn. This can be useful for background pixmaps with translucent areas.

Set *brush* to `Qt::NoBrush` or `Qt::Transparent` to leave background transparent. This can be useful for exporting to image formats which support transparency, e.g. [savePng](#).

See also

[setBackgroundScaled](#), [setBackgroundScaledMode](#)

10.469.4.76 `setBackground()` [2/3]

```
void QCustomPlot::setBackground (
    const QPixmap & pm )
```

Sets *pm* as the viewport background pixmap (see [setViewport](#)). The pixmap is always drawn below all other objects in the plot.

For cases where the provided pixmap doesn't have the same size as the viewport, scaling can be enabled with [setBackgroundScaled](#) and the scaling mode (whether and how the aspect ratio is preserved) can be set with [setBackgroundScaledMode](#). To set all these options in one call, consider using the overloaded version of this function.

If a background brush was set with [setBackground\(const QBrush &brush\)](#), the viewport will first be filled with that brush, before drawing the background pixmap. This can be useful for background pixmaps with translucent areas.

See also

[setBackgroundScaled](#), [setBackgroundScaledMode](#)

10.469.4.77 setBackground() [3/3]

```
void QCustomPlot::setBackground (
    const QPixmap & pm,
    bool scaled,
    Qt::AspectRatioMode mode = Qt::KeepAspectRatioByExpanding )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Allows setting the background pixmap of the viewport, whether it shall be scaled and how it shall be scaled in one call.

See also

[setBackground\(const QPixmap &pm\)](#), [setBackgroundScaled](#), [setBackgroundScaledMode](#)

10.469.4.78 setBackgroundScaled()

```
void QCustomPlot::setBackgroundScaled (
    bool scaled )
```

Sets whether the viewport background pixmap shall be scaled to fit the viewport. If *scaled* is set to true, control whether and how the aspect ratio of the original pixmap is preserved with [setBackgroundScaledMode](#).

Note that the scaled version of the original pixmap is buffered, so there is no performance penalty on replots. (Except when the viewport dimensions are changed continuously.)

See also

[setBackground](#), [setBackgroundScaledMode](#)

10.469.4.79 setBackgroundScaledMode()

```
void QCustomPlot::setBackgroundScaledMode (
    Qt::AspectRatioMode mode )
```

If scaling of the viewport background pixmap is enabled ([setBackgroundScaled](#)), use this function to define whether and how the aspect ratio of the original pixmap is preserved.

See also

[setBackground](#), [setBackgroundScaled](#)

10.469.4.80 setBufferDevicePixelRatio()

```
void QCustomPlot::setBufferDevicePixelRatio (
    double ratio )
```

Sets the device pixel ratio used by the paint buffers of this [QCustomPlot](#) instance.

Normally, this doesn't need to be set manually, because it is initialized with the regular *QWidget::devicePixelRatio* which is configured by Qt to fit the display device (e.g. 1 for normal displays, 2 for High-DPI displays).

Device pixel ratios are supported by Qt only for Qt versions since 5.4. If this method is called when [QCustomPlot](#) is being used with older Qt versions, outputs an according qDebug message and leaves the internal buffer device pixel ratio at 1.0.

10.469.4.81 setCurrentLayer() [1/2]

```
bool QCustomPlot::setCurrentLayer (
    const QString & name )
```

Sets the layer with the specified *name* to be the current layer. All layerables ([QCPLayerable](#)), e.g. plottables and items, are created on the current layer.

Returns true on success, i.e. if there is a layer with the specified *name* in the [QCustomPlot](#).

Layer names are case-sensitive.

See also

[addLayer](#), [moveLayer](#), [removeLayer](#), [QCPLayerable::setLayer](#)

10.469.4.82 setCurrentLayer() [2/2]

```
bool QCustomPlot::setCurrentLayer (
    QCPLayer * layer )
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Sets the provided *layer* to be the current layer.

Returns true on success, i.e. when *layer* is a valid layer in the [QCustomPlot](#).

See also

[addLayer](#), [moveLayer](#), [removeLayer](#)

10.469.4.83 setInteraction()

```
void QCustomPlot::setInteraction (
    const QCP::Interaction & interaction,
    bool enabled = true )
```

Sets the single *interaction* of this [QCustomPlot](#) to *enabled*.

For details about the interaction system, see [setInteractions](#).

See also

[setInteractions](#)

10.469.4.84 setInteractions()

```
void QCustomPlot::setInteractions (
    const QCP::Interactions & interactions )
```

Sets the possible interactions of this [QCustomPlot](#) as an or-combination of [QCP::Interaction](#) enums. There are the following types of interactions:

Axis range manipulation is controlled via [QCP::iRangeDrag](#) and [QCP::iRangeZoom](#). When the respective interaction is enabled, the user may drag axes ranges and zoom with the mouse wheel. For details how to control which axes the user may drag/zoom and in what orientations, see [QCPAxisRect::setRangeDrag](#), [QCPAxisRect::setRangeZoom](#), [QCPAxisRect::setRangeDragAxes](#), [QCPAxisRect::setRangeZoomAxes](#).

Plottable data selection is controlled by [QCP::iSelectPlottables](#). If [QCP::iSelectPlottables](#) is set, the user may select plottables (graphs, curves, bars,...) and their data by clicking on them or in their vicinity ([setSelectionTolerance](#)). Whether the user can actually select a plottable and its data can further be restricted with the [QCPAbstractPlottable::setSelectable](#) method on the specific plottable. For details, see the special page about the data selection mechanism. To retrieve a list of all currently selected plottables, call [selectedPlottables](#). If you're only interested in QCPGraphs, you may use the convenience function [selectedGraphs](#).

Item selection is controlled by [QCP::iSelectItems](#). If [QCP::iSelectItems](#) is set, the user may select items ([QCPItemLine](#), [QCPItemText](#),...) by clicking on them or in their vicinity. To find out whether a specific item is selected, call [QCPAbstractItem::selected\(\)](#). To retrieve a list of all currently selected items, call [selectedItems](#).

Axis selection is controlled with [QCP::iSelectAxes](#). If [QCP::iSelectAxes](#) is set, the user may select parts of the axes by clicking on them. What parts exactly (e.g. Axis base line, tick labels, axis label) are selectable can be controlled via [QCPAxis::setSelectableParts](#) for each axis. To retrieve a list of all axes that currently contain selected parts, call [selectedAxes](#). Which parts of an axis are selected, can be retrieved with [QCPAxis::selectedParts\(\)](#).

Legend selection is controlled with [QCP::iSelectLegend](#). If this is set, the user may select the legend itself or individual items by clicking on them. What parts exactly are selectable can be controlled via [QCPLegend::setSelectableParts](#). To find out whether the legend or any of its child items are selected, check the value of [QCPLegend::selectedParts](#). To find out which child items are selected, call [QCPLegend::selectedItems](#).

All other selectable elements The selection of all other selectable objects (e.g. [QCPTextElement](#), or your own layerable subclasses) is controlled with [QCP::iSelectOther](#). If set, the user may select those objects by clicking on them. To find out which are currently selected, you need to check their selected state explicitly.

If the selection state has changed by user interaction, the [selectionChangedByUser](#) signal is emitted. Each selectable object additionally emits an individual [selectionChanged](#) signal whenever their selection state has changed, i.e. not only by user interaction.

To allow multiple objects to be selected by holding the selection modifier ([setMultiSelectModifier](#)), set the flag [QCP::iMultiSelect](#).

Note

In addition to the selection mechanism presented here, [QCustomPlot](#) always emits corresponding signals, when an object is clicked or double clicked. see [plottableClick](#) and [plottableDoubleClick](#) for example.

See also

[setInteraction](#), [setSelectionTolerance](#)

10.469.4.85 `setMultiSelectModifier()`

```
void QCustomPlot::setMultiSelectModifier (
    Qt::KeyboardModifier modifier )
```

Sets the keyboard modifier that will be recognized as multi-select-modifier.

If `QCP::iMultiSelect` is specified in [setInteractions](#), the user may select multiple objects (or data points) by clicking on them one after the other while holding down *modifier*.

By default the multi-select-modifier is set to `Qt::ControlModifier`.

See also

[setInteractions](#)

10.469.4.86 `setNoAntialiasingOnDrag()`

```
void QCustomPlot::setNoAntialiasingOnDrag (
    bool enabled )
```

Sets whether antialiasing is disabled for this [QCustomPlot](#) while the user is dragging axes ranges. If many objects, especially plottables, are drawn antialiased, this greatly improves performance during dragging. Thus it creates a more responsive user experience. As soon as the user stops dragging, the last replot is done with normal antialiasing, to restore high image quality.

See also

[setAntialiasedElements](#), [setNotAntialiasedElements](#)

10.469.4.87 `setNotAntialiasedElement()`

```
void QCustomPlot::setNotAntialiasedElement (
    QCP::AntialiasedElement notAntialiasedElement,
    bool enabled = true )
```

Sets whether the specified *notAntialiasedElement* is forcibly drawn not antialiased.

See [setNotAntialiasedElements](#) for details.

See also

[setAntialiasedElement](#)

10.469.4.88 setNotAntialiasedElements()

```
void QCustomPlot::setNotAntialiasedElements (
    const QCP::AntialiasedElements & notAntialiasedElements )
```

Sets which elements are forcibly drawn not antialiased as an *or* combination of [QCP::AntialiasedElement](#).

This overrides the antialiasing settings for whole element groups, normally controlled with the *setAntialiasing* function on the individual elements. If an element is neither specified in [setAntialiasedElements](#) nor in [setNotAntialiasedElements](#), the antialiasing setting on each individual element instance is used.

For example, if *notAntialiasedElements* contains [QCP::aePlottables](#), no plottables will be drawn antialiased, no matter what the specific [QCPAbstractPlottable::setAntialiased](#) value was set to.

if an element in *notAntialiasedElements* is already set in [setAntialiasedElements](#), it is removed from there.

See also

[setAntialiasedElements](#)

10.469.4.89 setOpenGl()

```
void QCustomPlot::setOpenGl (
    bool enabled,
    int multisampling = 16 )
```

Warning

This is still an experimental feature and its performance depends on the system that it runs on. Having multiple [QCustomPlot](#) widgets in one application with enabled OpenGL rendering might cause context conflicts on some systems.

This method allows to enable OpenGL plot rendering, for increased plotting performance of graphically demanding plots (thick lines, translucent fills, etc.).

If *enabled* is set to true, [QCustomPlot](#) will try to initialize OpenGL and, if successful, continue plotting with hardware acceleration. The parameter *multisampling* controls how many samples will be used per pixel, it essentially controls the antialiasing quality. If *multisampling* is set too high for the current graphics hardware, the maximum allowed value will be used.

You can test whether switching to OpenGL rendering was successful by checking whether the according getter [QCustomPlot::openGl\(\)](#) returns true. If the OpenGL initialization fails, rendering continues with the regular software rasterizer, and an according qDebug output is generated.

If switching to OpenGL was successful, this method disables label caching ([setPlottingHint\(QCP::phCacheLabels, false\)](#)) and turns on [QCustomPlot](#)'s antialiasing override for all elements ([setAntialiasedElements\(QCP::aeAll\)](#)), leading to a higher quality output. The antialiasing override allows for pixel-grid aligned drawing in the OpenGL paint device. As stated before, in OpenGL rendering the actual antialiasing of the plot is controlled with *multisampling*. If *enabled* is set to false, the antialiasing/label caching settings are restored to what they were before OpenGL was enabled, if they weren't altered in the meantime.

Note

OpenGL support is only enabled if [QCustomPlot](#) is compiled with the macro `QCUSTOMPLOT_USE_OPENGL` defined. This define must be set before including the [QCustomPlot](#) header both during compilation of the [QCustomPlot](#) library as well as when compiling your application. It is best to just include the line `DEFINES += QCUSTOMPLOT_USE_OPENGL` in the respective qmake project files.

If you are using a Qt version before 5.0, you must also add the module "opengl" to your `QT` variable in the qmake project files. For Qt versions 5.0 and higher, [QCustomPlot](#) switches to a newer OpenGL interface which is already in the "gui" module.

10.469.4.90 setPlottingHint()

```
void QCustomPlot::setPlottingHint (
    QCP::PlottingHint hint,
    bool enabled = true )
```

Sets the specified plotting *hint* to *enabled*.

See also

[setPlottingHints](#)

10.469.4.91 setPlottingHints()

```
void QCustomPlot::setPlottingHints (
    const QCP::PlottingHints & hints )
```

Sets the plotting hints for this [QCustomPlot](#) instance as an *or* combination of [QCP::PlottingHint](#).

See also

[setPlottingHint](#)

10.469.4.92 setSelectionRect()

```
void QCustomPlot::setSelectionRect (
    QCPSelectionRect * selectionRect )
```

Sets the [QCPSelectionRect](#) instance that [QCustomPlot](#) will use if *mode* is not [QCP::srmNone](#) and the user performs a click-and-drag interaction. [QCustomPlot](#) takes ownership of the passed *selectionRect*. It can be accessed later via [selectionRect](#).

This method is useful if you wish to replace the default [QCPSelectionRect](#) instance with an instance of a [QCPSelectionRect](#) subclass, to introduce custom behaviour of the selection rect.

See also

[setSelectionRectMode](#)

10.469.4.93 setSelectionRectMode()

```
void QCustomPlot::setSelectionRectMode (
    QCP::SelectionMode mode )
```

Sets how [QCustomPlot](#) processes mouse click-and-drag interactions by the user.

If *mode* is [QCP::srmNone](#), the mouse drag is forwarded to the underlying objects. For example, [QCPAxisRect](#) may process a mouse drag by dragging axis ranges, see [QCPAxisRect::setRangeDrag](#). If *mode* is not [QCP::srmNone](#), the current selection rect ([selectionRect](#)) becomes activated and allows e.g. rect zooming and data point selection.

If you wish to provide your user both with axis range dragging and data selection/range zooming, use this method to switch between the modes just before the interaction is processed, e.g. in reaction to the [mousePress](#) or [mouseMove](#) signals. For example you could check whether the user is holding a certain keyboard modifier, and then decide which *mode* shall be set.

If a selection rect interaction is currently active, and *mode* is set to [QCP::srmNone](#), the interaction is canceled ([QCPSelectionRect::cancel](#)). Switching between any of the other modes will keep the selection rect active. Upon completion of the interaction, the behaviour is as defined by the currently set *mode*, not the mode that was set when the interaction started.

See also

[setInteractions](#), [setSelectionRect](#), [QCPSelectionRect](#)

10.469.4.94 setSelectionTolerance()

```
void QCustomPlot::setSelectionTolerance (
    int pixels )
```

Sets the tolerance that is used to decide whether a click selects an object (e.g. a plottable) or not.

If the user clicks in the vicinity of the line of e.g. a [QCPGraph](#), it's only regarded as a potential selection when the minimum distance between the click position and the graph line is smaller than *pixels*. Objects that are defined by an area (e.g. [QCPBars](#)) only react to clicks directly inside the area and ignore this selection tolerance. In other words, it only has meaning for parts of objects that are too thin to exactly hit with a click and thus need such a tolerance.

See also

[setInteractions](#), [QCPLayerable::selectTest](#)

10.469.4.95 setViewport()

```
void QCustomPlot::setViewport (
    const QRect & rect )
```

Sets the viewport of this [QCustomPlot](#). Usually users of [QCustomPlot](#) don't need to change the viewport manually.

The viewport is the area in which the plot is drawn. All mechanisms, e.g. margin calculation take the viewport to be the outer border of the plot. The viewport normally is the `rect()` of the [QCustomPlot](#) widget, i.e. a rect with top left (0, 0) and size of the [QCustomPlot](#) widget.

Don't confuse the viewport with the axis rect ([QCustomPlot::axisRect](#)). An axis rect is typically an area enclosed by four axes, where the graphs/plottables are drawn in. The viewport is larger and contains also the axes themselves, their tick numbers, their labels, or even additional axis rects, color scales and other layout elements.

This function is used to allow arbitrary size exports with [toPixmap](#), [savePng](#), [savePdf](#), etc. by temporarily changing the viewport size.

10.469.4.96 toPainter()

```
void QCustomPlot::toPainter (
    QCPPainter * painter,
    int width = 0,
    int height = 0 )
```

Renders the plot using the passed *painter*.

The plot is sized to *width* and *height* in pixels. If the *painter's* scale is not 1.0, the resulting plot will appear scaled accordingly.

Note

If you are restricted to using a `QPainter` (instead of [QCPPainter](#)), create a temporary `QPicture` and open a [QCPPainter](#) on it. Then call [toPainter](#) with this [QCPPainter](#). After ending the paint operation on the picture, draw it with the `QPainter`. This will reproduce the painter actions the [QCPPainter](#) took, with a `QPainter`.

See also

[toPixmap](#)

10.469.4.97 toPixmap()

```
QPixmap QCustomPlot::toPixmap (
    int width = 0,
    int height = 0,
    double scale = 1.0 )
```

Renders the plot to a pixmap and returns it.

The plot is sized to *width* and *height* in pixels and scaled with *scale*. (width 100 and scale 2.0 lead to a full resolution pixmap with width 200.)

See also

[toPainter](#), [saveRastered](#), [saveBmp](#), [savePng](#), [saveJpg](#), [savePdf](#)

10.469.5 Member Data Documentation

10.469.5.1 legend

```
QCPLegend * QCustomPlot::legend
```

A pointer to the default legend of the main axis rect. The legend is invisible by default. Use [QCPLegend::setVisible](#) to change this.

[QCustomPlot](#) offers convenient pointers to the axes ([xAxis](#), [yAxis](#), [xAxis2](#), [yAxis2](#)) and the [legend](#). They make it very easy working with plots that only have a single axis rect and at most one axis at each axis rect side. If you use the layout system to add multiple legends to the plot, use the layout system interface to access the new legend. For example, legends can be placed inside an axis rect's [inset layout](#), and must then also be accessed via the inset layout. If the default legend is removed due to manipulation of the layout system (e.g. by removing the main axis rect), the corresponding pointer becomes `nullptr`.

If an axis convenience pointer is currently `nullptr` and a new axis rect or a corresponding axis is added in the place of the main axis rect, [QCustomPlot](#) resets the convenience pointers to the according new axes. Similarly the [legend](#) convenience pointer will be reset if a legend is added after the main legend was removed before.

10.469.5.2 xAxis

```
QCPAxis * QCustomPlot::xAxis
```

A pointer to the primary x Axis (bottom) of the main axis rect of the plot.

[QCustomPlot](#) offers convenient pointers to the axes ([xAxis](#), [yAxis](#), [xAxis2](#), [yAxis2](#)) and the [legend](#). They make it very easy working with plots that only have a single axis rect and at most one axis at each axis rect side. If you use the layout system to add multiple axis rects or multiple axes to one side, use the [QCPAxisRect::axis](#) interface to access the new axes. If one of the four default axes or the default legend is removed due to manipulation of the layout system (e.g. by removing the main axis rect), the corresponding pointers become `nullptr`.

If an axis convenience pointer is currently `nullptr` and a new axis rect or a corresponding axis is added in the place of the main axis rect, [QCustomPlot](#) resets the convenience pointers to the according new axes. Similarly the [legend](#) convenience pointer will be reset if a legend is added after the main legend was removed before.

10.469.5.3 xAxis2

```
QCPAxis * QCustomPlot::xAxis2
```

A pointer to the secondary x Axis (top) of the main axis rect of the plot. Secondary axes are invisible by default. Use [QCPAxis::setVisible](#) to change this (or use [QCPAxisRect::setupFullAxesBox](#)).

[QCustomPlot](#) offers convenient pointers to the axes ([xAxis](#), [yAxis](#), [xAxis2](#), [yAxis2](#)) and the [legend](#). They make it very easy working with plots that only have a single axis rect and at most one axis at each axis rect side. If you use the layout system to add multiple axis rects or multiple axes to one side, use the [QCPAxisRect::axis](#) interface to access the new axes. If one of the four default axes or the default legend is removed due to manipulation of the layout system (e.g. by removing the main axis rect), the corresponding pointers become `nullptr`.

If an axis convenience pointer is currently `nullptr` and a new axis rect or a corresponding axis is added in the place of the main axis rect, [QCustomPlot](#) resets the convenience pointers to the according new axes. Similarly the [legend](#) convenience pointer will be reset if a legend is added after the main legend was removed before.

10.469.5.4 yAxis

```
QCPAxis * QCustomPlot::yAxis
```

A pointer to the primary y Axis (left) of the main axis rect of the plot.

[QCustomPlot](#) offers convenient pointers to the axes ([xAxis](#), [yAxis](#), [xAxis2](#), [yAxis2](#)) and the [legend](#). They make it very easy working with plots that only have a single axis rect and at most one axis at each axis rect side. If you use the layout system to add multiple axis rects or multiple axes to one side, use the [QCPAxisRect::axis](#) interface to access the new axes. If one of the four default axes or the default legend is removed due to manipulation of the layout system (e.g. by removing the main axis rect), the corresponding pointers become `nullptr`.

If an axis convenience pointer is currently `nullptr` and a new axis rect or a corresponding axis is added in the place of the main axis rect, [QCustomPlot](#) resets the convenience pointers to the according new axes. Similarly the [legend](#) convenience pointer will be reset if a legend is added after the main legend was removed before.

10.469.5.5 yAxis2

```
QCPAxis * QCustomPlot::yAxis2
```

A pointer to the secondary y Axis (right) of the main axis rect of the plot. Secondary axes are invisible by default. Use [QCPAxis::setVisible](#) to change this (or use [QCPAxisRect::setupFullAxesBox](#)).

[QCustomPlot](#) offers convenient pointers to the axes ([xAxis](#), [yAxis](#), [xAxis2](#), [yAxis2](#)) and the [legend](#). They make it very easy working with plots that only have a single axis rect and at most one axis at each axis rect side. If you use the layout system to add multiple axis rects or multiple axes to one side, use the [QCPAxisRect::axis](#) interface to access the new axes. If one of the four default axes or the default legend is removed due to manipulation of the layout system (e.g. by removing the main axis rect), the corresponding pointers become `nullptr`.

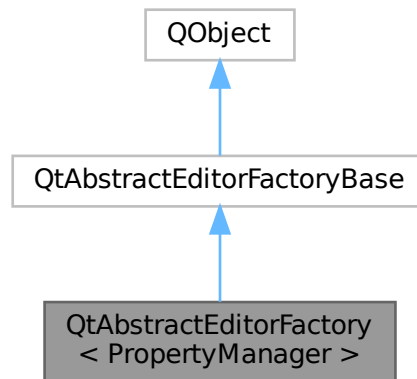
If an axis convenience pointer is currently `nullptr` and a new axis rect or a corresponding axis is added in the place of the main axis rect, [QCustomPlot](#) resets the convenience pointers to the according new axes. Similarly the [legend](#) convenience pointer will be reset if a legend is added after the main legend was removed before.

The documentation for this class was generated from the following files:

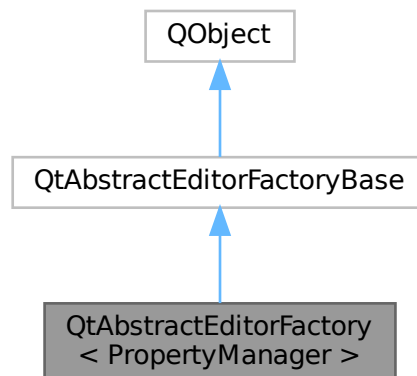
- applications/gui/genesys/qcustomplot.h
- applications/gui/genesys/qcustomplot.cpp

10.470 QtAbstractEditorFactory< PropertyManager > Class Template Reference

Inheritance diagram for QtAbstractEditorFactory< PropertyManager >:



Collaboration diagram for QtAbstractEditorFactory< PropertyManager >:



Public Member Functions

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (PropertyManager *manager)
- void [removePropertyManager](#) (PropertyManager *manager)
- QSet< PropertyManager * > [propertyManagers](#) () const
- PropertyManager * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- virtual void [connectPropertyManager](#) (PropertyManager *manager)=0
- virtual QWidget * [createEditor](#) (PropertyManager *manager, [QtProperty](#) *property, QWidget *parent)=0
- virtual void [disconnectPropertyManager](#) (PropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Friends

- class [QtAbstractPropertyEditor](#)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

10.470.1 Constructor & Destructor Documentation

10.470.1.1 QtAbstractEditorFactory()

```
template<class PropertyManager >
QtAbstractEditorFactory< PropertyManager >::QtAbstractEditorFactory (
    QObject * parent = 0 ) [inline], [explicit]
```

Creates an editor factory with the given *parent*.

See also

[addPropertyManager\(\)](#)

10.470.2 Member Function Documentation

10.470.2.1 addPropertyManager()

```
template<class PropertyManager >
void QtAbstractEditorFactory< PropertyManager >::addPropertyManager (
    PropertyManager * manager ) [inline]
```

Adds the given *manager* to this factory's set of managers, making this factory produce editing widgets for properties created by the given manager.

The PropertyManager type is a template argument class, and represents the chosen [QtAbstractPropertyManager](#) subclass.

See also

[propertyManagers\(\)](#), [removePropertyManager\(\)](#)

10.470.2.2 connectPropertyManager()

```
template<class PropertyManager >
void QtAbstractEditorFactory< PropertyManager >::connectPropertyManager (
    PropertyManager * manager ) [protected], [pure virtual]
```

Connects this factory to the given *manager's* signals. The PropertyManager type is a template argument class, and represents the chosen [QtAbstractPropertyManager](#) subclass.

This function is used internally by the [addPropertyManager\(\)](#) function, and makes it possible to update an editing widget when the associated property's data changes. This is typically done in custom slots responding to the signals emitted by the property's manager, e.g. [QtIntPropertyManager::valueChanged\(\)](#) and [QtIntPropertyManager::rangeChanged\(\)](#).

See also

[propertyManagers\(\)](#), [disconnectPropertyManager\(\)](#)

10.470.2.3 createEditor() [1/2]

```
template<class PropertyManager >
QWidget * QtAbstractEditorFactory< PropertyManager >::createEditor (
    PropertyManager * manager,
    QtProperty * property,
    QWidget * parent ) [protected], [pure virtual]
```

Creates an editing widget with the given *parent* for the specified *property* created by the given *manager*. The PropertyManager type is a template argument class, and represents the chosen [QtAbstractPropertyManager](#) subclass.

This function must be implemented in derived classes: It is recommended to store a pointer to the widget and map it to the given *property*, since the widget must be updated whenever the associated property's data changes. This is typically done in custom slots responding to the signals emitted by the property's manager, e.g. [QtIntPropertyManager::valueChanged\(\)](#) and [QtIntPropertyManager::rangeChanged\(\)](#).

See also

[connectPropertyManager\(\)](#)

10.470.2.4 createEditor() [2/2]

```
template<class PropertyManager >
QWidget * QtAbstractEditorFactory< PropertyManager >::createEditor (
    QtProperty * property,
    QWidget * parent ) [inline], [override], [virtual]
```

Creates an editing widget (with the given *parent*) for the given *property*.

Implements [QtAbstractEditorFactoryBase](#).

10.470.2.5 disconnectPropertyManager()

```
template<class PropertyManager >
void QtAbstractEditorFactory< PropertyManager >::disconnectPropertyManager (
    PropertyManager * manager ) [protected], [pure virtual]
```

Disconnects this factory from the given *manager's* signals. The PropertyManager type is a template argument class, and represents the chosen [QtAbstractPropertyManager](#) subclass.

This function is used internally by the [removePropertyManager\(\)](#) function.

See also

[propertyManagers\(\)](#), [connectPropertyManager\(\)](#)

10.470.2.6 managerDestroyed()

```
template<class PropertyManager >
void QtAbstractEditorFactory< PropertyManager >::managerDestroyed (
    QObject * manager ) [inline], [override], [protected], [virtual]
```

Implements [QtAbstractEditorFactoryBase](#).

10.470.2.7 propertyManager()

```
template<class PropertyManager >
PropertyManager * QtAbstractEditorFactory< PropertyManager >::propertyManager (
    QtProperty * property ) const [inline]
```

Returns the property manager for the given *property*, or 0 if the given *property* doesn't belong to any of this factory's registered managers.

The PropertyManager type is a template argument class, and represents the chosen [QtAbstractPropertyManager](#) subclass.

See also

[propertyManagers\(\)](#)

10.470.2.8 propertyManagers()

```
template<class PropertyManager >
QSet< PropertyManager * > QtAbstractEditorFactory< PropertyManager >::propertyManagers ( )
const [inline]
```

Returns the factory's set of associated managers. The PropertyManager type is a template argument class, and represents the chosen [QtAbstractPropertyManager](#) subclass.

See also

[addPropertyManager\(\)](#), [removePropertyManager\(\)](#)

10.470.2.9 removePropertyManager()

```
template<class PropertyManager >
void QtAbstractEditorFactory< PropertyManager >::removePropertyManager (
    PropertyManager * manager ) [inline]
```

Removes the given *manager* from this factory's set of managers. The PropertyManager type is a template argument class, and may be any [QtAbstractPropertyManager](#) subclass.

See also

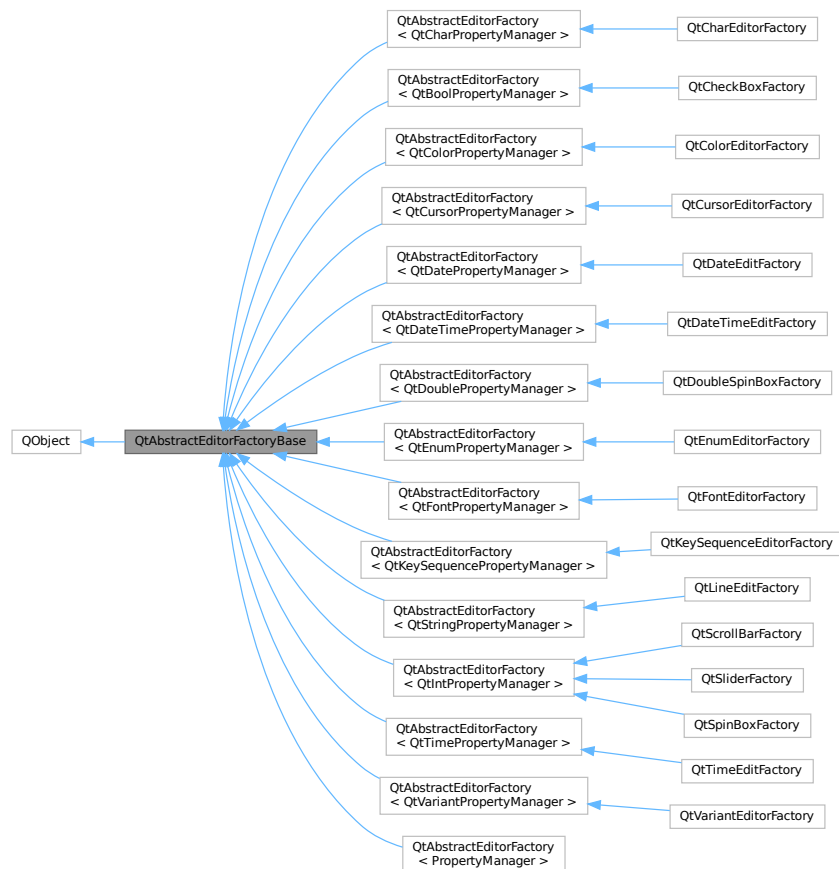
[propertyManagers\(\)](#), [addPropertyManager\(\)](#)

The documentation for this class was generated from the following files:

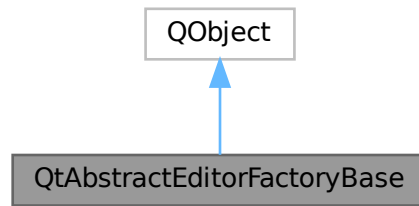
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

10.471 QtAbstractEditorFactoryBase Class Reference

Inheritance diagram for QtAbstractEditorFactoryBase:



Collaboration diagram for QtAbstractEditorFactoryBase:



Public Member Functions

- virtual QWidget * [createEditor](#) ([QtProperty](#) *property, QWidget *parent)=0

Protected Slots

- virtual void **managerDestroyed** (QObject *manager)=0

Protected Member Functions

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)
- virtual void **breakConnection** ([QtAbstractPropertyManager](#) *manager)=0

Friends

- class **QtAbstractPropertyBrowser**

10.471.1 Constructor & Destructor Documentation

10.471.1.1 QtAbstractEditorFactoryBase()

```
QtAbstractEditorFactoryBase::QtAbstractEditorFactoryBase (  
    QObject * parent = 0 ) [inline], [explicit], [protected]
```

Creates an abstract editor factory with the given *parent*.

10.471.2 Member Function Documentation

10.471.2.1 createEditor()

```
QWidget * QtAbstractEditorFactoryBase::createEditor (
    QtProperty * property,
    QWidget * parent ) [pure virtual]
```

Creates an editing widget (with the given *parent*) for the given *property*.

This function is reimplemented in [QtAbstractEditorFactory](#) template class which also provides a pure virtual convenience overload of this function enabling access to the property's manager.

See also

[QtAbstractEditorFactory::createEditor\(\)](#)

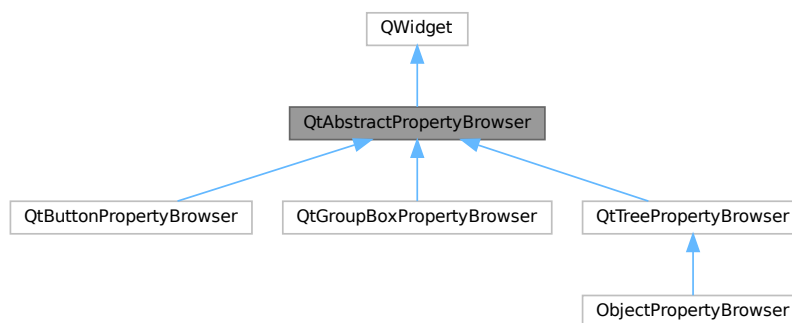
Implemented in [QtAbstractEditorFactory< PropertyManager >](#), [QtAbstractEditorFactory< QtBoolPropertyManager >](#), [QtAbstractEditorFactory< QtCharPropertyManager >](#), [QtAbstractEditorFactory< QColorPropertyManager >](#), [QtAbstractEditorFactory< QtCursorPropertyManager >](#), [QtAbstractEditorFactory< QtDatePropertyManager >](#), [QtAbstractEditorFactory< QDateTimePropertyManager >](#), [QtAbstractEditorFactory< QtDoublePropertyManager >](#), [QtAbstractEditorFactory< QtEnumPropertyManager >](#), [QtAbstractEditorFactory< QFontPropertyManager >](#), [QtAbstractEditorFactory< QtIntPropertyManager >](#), [QtAbstractEditorFactory< QtKeySequencePropertyManager >](#), [QtAbstractEditorFactory< QStringPropertyManager >](#), [QtAbstractEditorFactory< QtTimePropertyManager >](#), and [QtAbstractEditorFactory< QtVariantPropertyManager >](#).

The documentation for this class was generated from the following files:

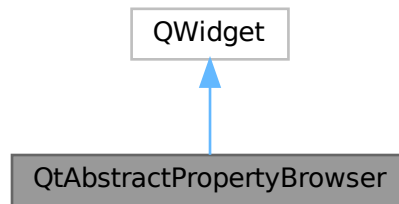
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

10.472 QtAbstractPropertyBrowser Class Reference

Inheritance diagram for QtAbstractPropertyBrowser:



Collaboration diagram for QtAbstractPropertyBrowser:



Public Slots

- [QtBrowserItem](#) * [addProperty](#) ([QtProperty](#) *property)
- [QtBrowserItem](#) * [insertProperty](#) ([QtProperty](#) *property, [QtProperty](#) *afterProperty)
- void [removeProperty](#) ([QtProperty](#) *property)

Signals

- void [currentItemChanged](#) ([QtBrowserItem](#) *)

Public Member Functions

- [QtAbstractPropertyBrowser](#) ([QWidget](#) *parent=0)
- [~QtAbstractPropertyBrowser](#) ()
- [QList](#)< [QtProperty](#) * > [properties](#) () const
- [QList](#)< [QtBrowserItem](#) * > [items](#) ([QtProperty](#) *property) const
- [QtBrowserItem](#) * [topLevelItem](#) ([QtProperty](#) *property) const
- [QList](#)< [QtBrowserItem](#) * > [topLevelItems](#) () const
- void [clear](#) ()
- template<class [PropertyManager](#) >
void [setFactoryForManager](#) ([PropertyManager](#) *manager, [QtAbstractEditorFactory](#)< [PropertyManager](#) > *factory)
- void [unsetFactoryForManager](#) ([QtAbstractPropertyManager](#) *manager)
- [QtBrowserItem](#) * [currentItem](#) () const
- void [setCurrentItem](#) ([QtBrowserItem](#) *)

Protected Member Functions

- virtual void [itemInserted](#) ([QtBrowserItem](#) *item, [QtBrowserItem](#) *afterItem)=0
- virtual void [itemRemoved](#) ([QtBrowserItem](#) *item)=0
- virtual void [itemChanged](#) ([QtBrowserItem](#) *item)=0
- virtual [QWidget](#) * [createEditor](#) ([QtProperty](#) *property, [QWidget](#) *parent)

10.472.1 Constructor & Destructor Documentation

10.472.1.1 QtAbstractPropertyBrowser()

```
QtAbstractPropertyBrowser::QtAbstractPropertyBrowser (
    QWidget * parent = 0 ) [explicit]
```

Creates an abstract property browser with the given *parent*.

10.472.1.2 ~QtAbstractPropertyBrowser()

```
QtAbstractPropertyBrowser::~QtAbstractPropertyBrowser ( )
```

Destroys the property browser, and destroys all the items that were created by this property browser.

Note that the properties that were displayed in the editor are not deleted since they still can be used in other editors. Neither does the destructor delete the property managers and editor factories that were used by this property browser widget unless this widget was their parent.

See also

[QtAbstractPropertyManager::~~QtAbstractPropertyManager\(\)](#)

10.472.2 Member Function Documentation

10.472.2.1 addProperty

```
QtBrowserItem * QtAbstractPropertyBrowser::addProperty (
    QtProperty * property ) [slot]
```

Appends the given *property* (and its subproperties) to the property browser's list of top level properties. Returns the item created by property browser which is associated with the *property*. In order to get all children items created by the property browser in this call, the returned item should be traversed.

If the specified *property* is already added, this function does nothing and returns 0.

See also

[insertProperty\(\)](#), [QtProperty::addSubProperty\(\)](#), [properties\(\)](#)

10.472.2.2 clear()

```
void QtAbstractPropertyBrowser::clear ( )
```

Removes all the properties from the editor, but does not delete them since they can still be used in other editors.

See also

[removeProperty\(\)](#), [QtAbstractPropertyManager::clear\(\)](#)

10.472.2.3 createEditor()

```
QWidget * QtAbstractPropertyBrowser::createEditor (
    QtProperty * property,
    QWidget * parent ) [protected], [virtual]
```

Creates an editing widget (with the given *parent*) for the given *property* according to the previously established associations between property managers and editor factories.

If the property is created by a property manager which was not associated with any of the existing factories in *this* property editor, the function returns 0.

To make a property editable in the property browser, the [createEditor\(\)](#) function must be called to provide the property with a suitable editing widget.

Reimplement this function to provide additional decoration for the editing widgets created by the installed factories.

See also

[setFactoryForManager\(\)](#)

10.472.2.4 currentItem()

```
QtBrowserItem * QtAbstractPropertyBrowser::currentItem ( ) const
```

Returns the current item in the property browser.

See also

[setCurrentItem\(\)](#)

10.472.2.5 currentItemChanged

```
void QtAbstractPropertyBrowser::currentItemChanged (
    QtBrowserItem * current ) [signal]
```

This signal is emitted when the current item changes. The current item is specified by *current*.

See also

[QtAbstractPropertyBrowser::setCurrentItem\(\)](#)

10.472.2.6 insertProperty

```
QtBrowserItem * QtAbstractPropertyBrowser::insertProperty (
    QtProperty * property,
    QtProperty * afterProperty ) [slot]
```

Inserts the given *property* (and its subproperties) after the specified *afterProperty* in the browser's list of top level properties. Returns item created by property browser which is associated with the *property*. In order to get all children items created by the property browser in this call returned item should be traversed.

If the specified *afterProperty* is 0, the given *property* is inserted at the beginning of the list. If *property* is already inserted, this function does nothing and returns 0.

See also

[addProperty\(\)](#), [QtProperty::insertSubProperty\(\)](#), [properties\(\)](#)

10.472.2.7 itemChanged()

```
void QtAbstractPropertyBrowser::itemChanged (
    QtBrowserItem * item ) [protected], [pure virtual]
```

This function is called whenever a property's data changes, passing a pointer to the *item* of property as parameter.

This function must be reimplemented in derived classes in order to update the property browser widget whenever a property's name, tool tip, status tip, "what's this" text, value text or value icon changes.

Note that if the property browser contains several occurrences of the same property, this method will be called once for each occurrence (with a different item each time).

See also

[QtProperty](#), [items\(\)](#)

Implemented in [QtButtonPropertyBrowser](#), [QtGroupBoxPropertyBrowser](#), and [QtTreePropertyBrowser](#).

10.472.2.8 itemInserted()

```
void QtAbstractPropertyBrowser::itemInserted (
    QtBrowserItem * insertedItem,
    QtBrowserItem * precedingItem ) [protected], [pure virtual]
```

This function is called to update the widget whenever a property is inserted or added to the property browser, passing pointers to the *insertedItem* of property and the specified *precedingItem* as parameters.

If *precedingItem* is 0, the *insertedItem* was put at the beginning of its parent item's list of subproperties. If the parent of *insertedItem* is 0, the *insertedItem* was added as a top level property of *this* property browser.

This function must be reimplemented in derived classes. Note that if the *insertedItem*'s property has subproperties, this method will be called for those properties as soon as the current call is finished.

See also

[insertProperty\(\)](#), [addProperty\(\)](#)

Implemented in [QtButtonPropertyBrowser](#), [QtGroupBoxPropertyBrowser](#), and [QtTreePropertyBrowser](#).

10.472.2.9 itemRemoved()

```
void QtAbstractPropertyBrowser::itemRemoved (
    QtBrowserItem * item ) [protected], [pure virtual]
```

This function is called to update the widget whenever a property is removed from the property browser, passing the pointer to the *item* of the property as parameters. The passed *item* is deleted just after this call is finished.

If the the parent of *item* is 0, the removed *item* was a top level property in this editor.

This function must be reimplemented in derived classes. Note that if the removed *item*'s property has subproperties, this method will be called for those properties just before the current call is started.

See also

[removeProperty\(\)](#)

Implemented in [QtButtonPropertyBrowser](#), [QtGroupBoxPropertyBrowser](#), and [QtTreePropertyBrowser](#).

10.472.2.10 items()

```
QList< QtBrowserItem * > QtAbstractPropertyBrowser::items (
    QtProperty * property ) const
```

Returns the property browser's list of all items associated with the given *property*.

There is one item per instance of the property in the browser.

See also

[topLevelItem\(\)](#)

10.472.2.11 properties()

```
QList< QtProperty * > QtAbstractPropertyBrowser::properties ( ) const
```

Returns the property browser's list of top level properties.

To traverse the subproperties, use the [QtProperty::subProperties\(\)](#) function.

See also

[addProperty\(\)](#), [insertProperty\(\)](#), [removeProperty\(\)](#)

10.472.2.12 removeProperty

```
void QtAbstractPropertyBrowser::removeProperty (
    QtProperty * property ) [slot]
```

Removes the specified *property* (and its subproperties) from the property browser's list of top level properties. All items that were associated with the given *property* and its children are deleted.

Note that the properties are *not* deleted since they can still be used in other editors.

See also

[clear\(\)](#), [QtProperty::removeSubProperty\(\)](#), [properties\(\)](#)

10.472.2.13 setCurrentItem()

```
void QtAbstractPropertyBrowser::setCurrentItem (
    QtBrowserItem * item )
```

Sets the current item in the property browser to *item*.

See also

[currentItem\(\)](#), [currentItemChanged\(\)](#)

10.472.2.14 setFactoryForManager()

```
template<class PropertyManager >
void QtAbstractPropertyBrowser::setFactoryForManager (
    PropertyManager * manager,
    QtAbstractEditorFactory< PropertyManager > * factory ) [inline]
```

Connects the given *manager* to the given *factory*, ensuring that properties of the *manager*'s type will be displayed with an editing widget suitable for their value.

For example:

In this example the `myInteger` property's value is displayed with a `QSpinBox` widget, while the `myDouble` property's value is displayed with a `QDoubleSpinBox` widget.

Note that a factory can be associated with many managers, but a manager can only be associated with one single factory. If the given *manager* already is associated with another factory, the old association is broken before the new one established.

This function ensures that the given *manager* and the given *factory* are compatible, and it automatically calls the `QtAbstractEditorFactory::addPropertyManager()` function if necessary.

See also

[unsetFactoryForManager\(\)](#)

10.472.2.15 topLevelItem()

```
QtBrowserItem * QtAbstractPropertyBrowser::topLevelItem (
    QtProperty * property ) const
```

Returns the top-level items associated with the given *property*.

Returns 0 if *property* wasn't inserted into this property browser or isn't a top-level one.

See also

[topLevelItems\(\)](#), [items\(\)](#)

10.472.2.16 topLevelItems()

```
QList< QtBrowserItem * > QtAbstractPropertyBrowser::topLevelItems ( ) const
```

Returns the list of top-level items.

See also

[topLevelItem\(\)](#)

10.472.2.17 unsetFactoryForManager()

```
void QtAbstractPropertyBrowser::unsetFactoryForManager (
    QtAbstractPropertyManager * manager )
```

Removes the association between the given *manager* and the factory bound to it, automatically calling the [QtAbstractEditorFactory::removePropertyManager\(\)](#) function if necessary.

See also

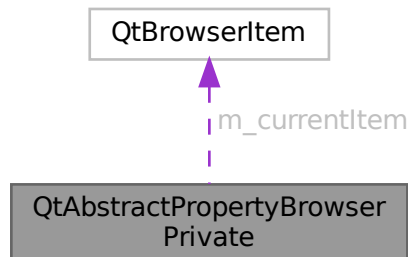
[setFactoryForManager\(\)](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

10.473 QtAbstractPropertyBrowserPrivate Class Reference

Collaboration diagram for QtAbstractPropertyBrowserPrivate:



Public Member Functions

- void **insertSubTree** ([QtProperty](#) *property, [QtProperty](#) *parentProperty)
- void **removeSubTree** ([QtProperty](#) *property, [QtProperty](#) *parentProperty)
- void **createBrowserIndexes** ([QtProperty](#) *property, [QtProperty](#) *parentProperty, [QtProperty](#) *afterProperty)
- void **removeBrowserIndexes** ([QtProperty](#) *property, [QtProperty](#) *parentProperty)
- [QtBrowserItem](#) * **createBrowserIndex** ([QtProperty](#) *property, [QtBrowserItem](#) *parentIndex, [QtBrowserItem](#) *afterIndex)
- void **removeBrowserIndex** ([QtBrowserItem](#) *index)
- void **clearIndex** ([QtBrowserItem](#) *index)
- void **slotPropertyInserted** ([QtProperty](#) *property, [QtProperty](#) *parentProperty, [QtProperty](#) *afterProperty)
- void **slotPropertyRemoved** ([QtProperty](#) *property, [QtProperty](#) *parentProperty)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)
- void **slotPropertyDataChanged** ([QtProperty](#) *property)

Public Attributes

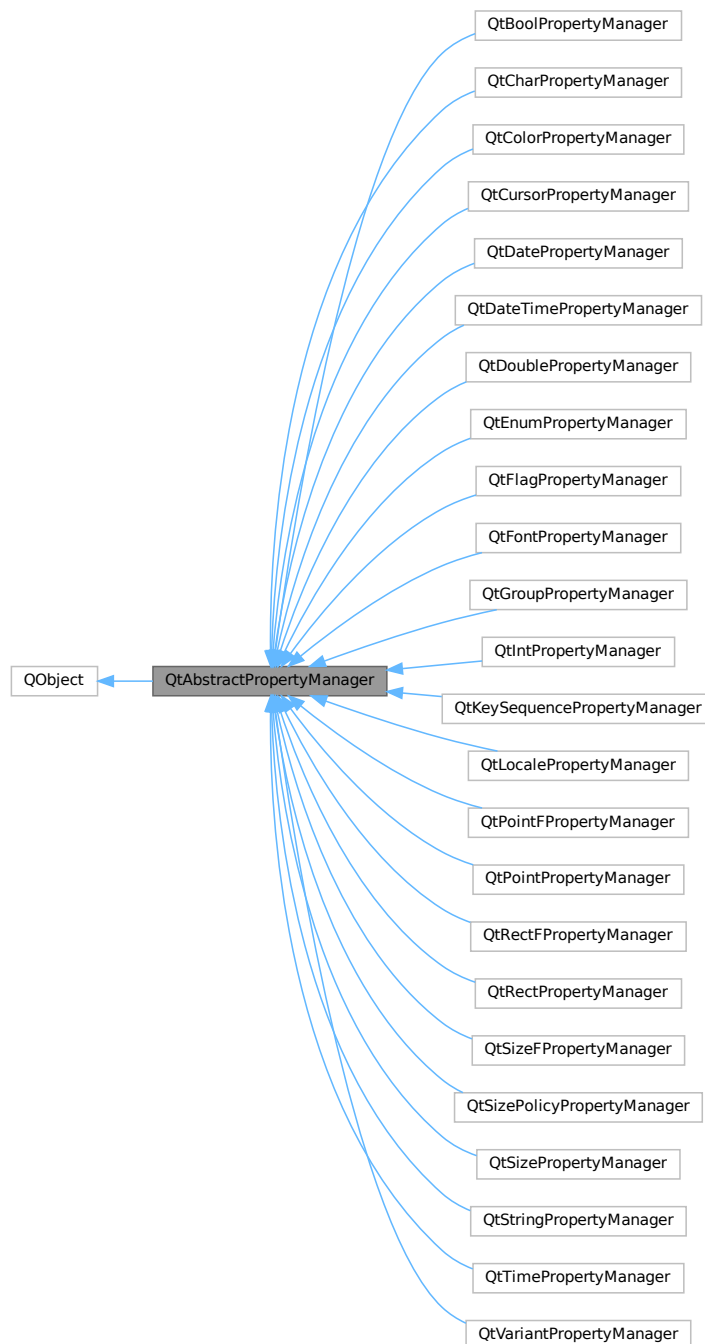
- `QList< QtProperty * > m_subItems`
- `QMap< QtAbstractPropertyManager *, QList< QtProperty * > > m_managerToProperties`
- `QMap< QtProperty *, QList< QtProperty * > > m_propertyToParents`
- `QMap< QtProperty *, QtBrowserItem * > m_topLevelPropertyToIndex`
- `QList< QtBrowserItem * > m_topLevelIndexes`
- `QMap< QtProperty *, QList< QtBrowserItem * > > m_propertyToIndexes`
- `QtBrowserItem * m_currentItem`

The documentation for this class was generated from the following file:

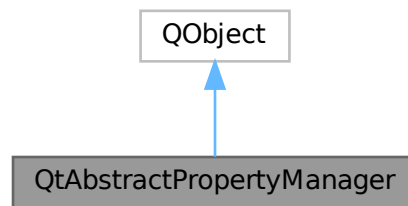
- `applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp`

10.474 QtAbstractPropertyManager Class Reference

Inheritance diagram for QtAbstractPropertyManager:



Collaboration diagram for QtAbstractPropertyManager:



Signals

- void `propertyInserted` (`QtProperty` *property, `QtProperty` *parent, `QtProperty` *after)
- void `propertyChanged` (`QtProperty` *property)
- void `propertyRemoved` (`QtProperty` *property, `QtProperty` *parent)
- void `propertyDestroyed` (`QtProperty` *property)

Public Member Functions

- `QtAbstractPropertyManager` (`QObject` *parent=0)
- `~QtAbstractPropertyManager` ()
- `QSet< QtProperty * > properties` () const
- void `clear` () const
- `QtProperty` * `addProperty` (const `QString` &name=`QString`())

Protected Member Functions

- virtual bool `hasValue` (const `QtProperty` *property) const
- virtual `QIcon` `valueIcon` (const `QtProperty` *property) const
- virtual `QString` `valueText` (const `QtProperty` *property) const
- virtual void `initializeProperty` (`QtProperty` *property)=0
- virtual void `uninitializeProperty` (`QtProperty` *property)
- virtual `QtProperty` * `createProperty` ()

Friends

- class `QtProperty`

10.474.1 Constructor & Destructor Documentation

10.474.1.1 QtAbstractPropertyManager()

```

QtAbstractPropertyManager::QtAbstractPropertyManager (
    QObject * parent = 0 ) [explicit]
  
```

Creates an abstract property manager with the given *parent*.

10.474.1.2 ~QtAbstractPropertyManager()

```
QtAbstractPropertyManager::~QtAbstractPropertyManager ( )
```

Destroys the manager. All properties created by the manager are destroyed.

10.474.2 Member Function Documentation

10.474.2.1 addProperty()

```
QtProperty * QtAbstractPropertyManager::addProperty (
    const QString & name = QString() )
```

Creates a property with the given *name* which then is owned by this manager.

Internally, this function calls the [createProperty\(\)](#) and [initializeProperty\(\)](#) functions.

See also

[initializeProperty\(\)](#), [properties\(\)](#)

10.474.2.2 clear()

```
void QtAbstractPropertyManager::clear ( ) const
```

Destroys all the properties that this manager has created.

See also

[propertyDestroyed\(\)](#), [uninitializeProperty\(\)](#)

10.474.2.3 createProperty()

```
QtProperty * QtAbstractPropertyManager::createProperty ( ) [protected], [virtual]
```

Creates a property.

The base implementation produce [QtProperty](#) instances; Reimplement this function to make this manager produce objects of a [QtProperty](#) subclass.

See also

[addProperty\(\)](#), [initializeProperty\(\)](#)

Reimplemented in [QtVariantPropertyManager](#).

10.474.2.4 hasValue()

```
bool QtAbstractPropertyManager::hasValue (
    const QtProperty * property ) const    [protected], [virtual]
```

Returns whether the given *property* has a value.

The default implementation of this function returns true.

See also

[QtProperty::hasValue\(\)](#)

Reimplemented in [QtGroupPropertyManager](#), and [QtVariantPropertyManager](#).

10.474.2.5 initializeProperty()

```
void QtAbstractPropertyManager::initializeProperty (
    QtProperty * property )    [protected], [pure virtual]
```

This function is called whenever a new valid property pointer has been created, passing the pointer as parameter.

The purpose is to let the manager know that the *property* has been created so that it can provide additional attributes for the new property, e.g. [QtIntPropertyManager](#) adds `\l {QtIntPropertyManager::value()}{value}`, `\l {QtIntPropertyManager::minimum()}{minimum}` and `\l {QtIntPropertyManager::maximum()}{maximum}` attributes. Since each manager subclass adds type specific attributes, this function is pure virtual and must be reimplemented when deriving from the [QtAbstractPropertyManager](#) class.

See also

[addProperty\(\)](#), [createProperty\(\)](#)

Implemented in [QtGroupPropertyManager](#), [QtIntPropertyManager](#), [QtBoolPropertyManager](#), [QtDoublePropertyManager](#), [QStringPropertyManager](#), [QtDatePropertyManager](#), [QtTimePropertyManager](#), [QtDateTimePropertyManager](#), [QtKeySequencePropertyManager](#), [QtCharPropertyManager](#), [QtLocalePropertyManager](#), [QtPointPropertyManager](#), [QtPointFPropertyManager](#), [QtSizePropertyManager](#), [QtSizeFPropertyManager](#), [QtRectPropertyManager](#), [QtRectFPropertyManager](#), [QtEnumPropertyManager](#), [QtFlagPropertyManager](#), [QtSizePolicyPropertyManager](#), [QtFontPropertyManager](#), [QtColorPropertyManager](#), [QtCursorPropertyManager](#), and [QtVariantPropertyManager](#).

10.474.2.6 properties()

```
QSet< QtProperty * > QtAbstractPropertyManager::properties ( ) const
```

Returns the set of properties created by this manager.

See also

[addProperty\(\)](#)

10.474.2.7 propertyChanged

```
void QtAbstractPropertyManager::propertyChanged (
    QtProperty * property ) [signal]
```

This signal is emitted whenever a property's data changes, passing a pointer to the *property* as parameter.

Note that signal is only emitted for properties that are created by this manager.

See also

[QtAbstractPropertyBrowser::itemChanged\(\)](#)

10.474.2.8 propertyDestroyed

```
void QtAbstractPropertyManager::propertyDestroyed (
    QtProperty * property ) [signal]
```

This signal is emitted when the specified *property* is about to be destroyed.

Note that signal is only emitted for properties that are created by this manager.

See also

[clear\(\)](#), [uninitializeProperty\(\)](#)

10.474.2.9 propertyInserted

```
void QtAbstractPropertyManager::propertyInserted (
    QtProperty * newProperty,
    QtProperty * parentProperty,
    QtProperty * precedingProperty ) [signal]
```

This signal is emitted when a new subproperty is inserted into an existing property, passing pointers to the *newProperty*, *parentProperty* and *precedingProperty* as parameters.

If *precedingProperty* is 0, the *newProperty* was inserted at the beginning of the *parentProperty*'s subproperties list.

Note that signal is emitted only if the *parentProperty* is created by this manager.

See also

[QtAbstractPropertyBrowser::itemInserted\(\)](#)

10.474.2.10 propertyRemoved

```
void QtAbstractPropertyManager::propertyRemoved (
    QtProperty * property,
    QtProperty * parent ) [signal]
```

This signal is emitted when a subproperty is removed, passing pointers to the removed *property* and the *parent* property as parameters.

Note that signal is emitted only when the *parent* property is created by this manager.

See also

[QtAbstractPropertyBrowser::itemRemoved\(\)](#)

10.474.2.11 uninitializedProperty()

```
void QtAbstractPropertyManager::uninitializeProperty (
    QtProperty * property ) [protected], [virtual]
```

This function is called just before the specified *property* is destroyed.

The purpose is to let the property manager know that the *property* is being destroyed so that it can remove the property's additional attributes.

See also

[clear\(\)](#), [propertyDestroyed\(\)](#)

Reimplemented in [QtGroupPropertyManager](#), [QtIntPropertyManager](#), [QtBoolPropertyManager](#), [QtDoublePropertyManager](#), [QStringPropertyManager](#), [QtDatePropertyManager](#), [QtTimePropertyManager](#), [QtDateTimePropertyManager](#), [QtKeySequencePropertyManager](#), [QtCharPropertyManager](#), [QtLocalePropertyManager](#), [QtPointPropertyManager](#), [QtPointFPropertyManager](#), [QtSizePropertyManager](#), [QtSizeFPropertyManager](#), [QtRectPropertyManager](#), [QtRectFPropertyManager](#), [QtEnumPropertyManager](#), [QtFlagPropertyManager](#), [QtSizePolicyPropertyManager](#), [QtFontPropertyManager](#), [QtColorPropertyManager](#), [QtCursorPropertyManager](#), and [QtVariantPropertyManager](#).

10.474.2.12 valueIcon()

```
QIcon QtAbstractPropertyManager::valueIcon (
    const QtProperty * property ) const [protected], [virtual]
```

Returns an icon representing the current state of the given *property*.

The default implementation of this function returns an invalid icon.

See also

[QtProperty::valueIcon\(\)](#)

Reimplemented in [QtBoolPropertyManager](#), [QtEnumPropertyManager](#), [QtFontPropertyManager](#), [QtColorPropertyManager](#), [QtCursorPropertyManager](#), and [QtVariantPropertyManager](#).

10.474.2.13 valueText()

```
QString QtAbstractPropertyManager::valueText (
    const QtProperty * property ) const    [protected], [virtual]
```

Returns a string representing the current state of the given *property*.

The default implementation of this function returns an empty string.

See also

[QtProperty::valueText\(\)](#)

Reimplemented in [QtIntPropertyManager](#), [QtBoolPropertyManager](#), [QtDoublePropertyManager](#), [QStringPropertyManager](#), [QtDatePropertyManager](#), [QtTimePropertyManager](#), [QtDateTimePropertyManager](#), [QtKeySequencePropertyManager](#), [QtCharPropertyManager](#), [QtLocalePropertyManager](#), [QtPointPropertyManager](#), [QtPointFPropertyManager](#), [QtSizePropertyManager](#), [QtSizeFPropertyManager](#), [QtRectPropertyManager](#), [QtRectFPropertyManager](#), [QtEnumPropertyManager](#), [QtFlagPropertyManager](#), [QtSizePolicyPropertyManager](#), [QtFontPropertyManager](#), [QtColorPropertyManager](#), [QtCursorPropertyManager](#), and [QtVariantPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

10.475 QtAbstractPropertyManagerPrivate Class Reference**Public Member Functions**

- void **propertyDestroyed** ([QtProperty](#) *property)
- void **propertyChanged** ([QtProperty](#) *property) const
- void **propertyRemoved** ([QtProperty](#) *property, [QtProperty](#) *parentProperty) const
- void **propertyInserted** ([QtProperty](#) *property, [QtProperty](#) *parentProperty, [QtProperty](#) *afterProperty) const

Public Attributes

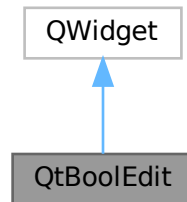
- QSet< [QtProperty](#) * > **m_properties**

The documentation for this class was generated from the following file:

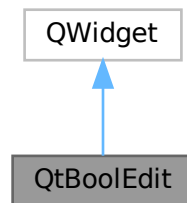
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

10.476 QtBoolEdit Class Reference

Inheritance diagram for QtBoolEdit:



Collaboration diagram for QtBoolEdit:



Signals

- void **toggled** (bool)

Public Member Functions

- **QtBoolEdit** (QWidget *parent=0)
- bool **textVisible** () const
- void **setTextVisible** (bool textVisible)
- Qt::CheckState **checkState** () const
- void **setCheckState** (Qt::CheckState state)
- bool **isChecked** () const
- void **setChecked** (bool c)
- void **setInitialChecked** (bool c)
- bool **blockCheckBoxSignals** (bool block)

Protected Member Functions

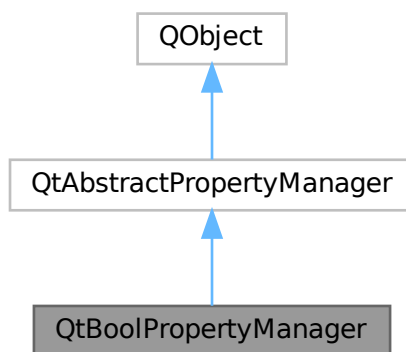
- void **mousePressEvent** (QMouseEvent *event) override

The documentation for this class was generated from the following files:

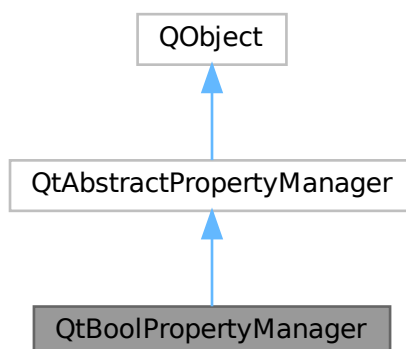
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

10.477 QtBoolPropertyManager Class Reference

Inheritance diagram for QtBoolPropertyManager:



Collaboration diagram for QtBoolPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, bool val)
- void [setValueOnly](#) ([QtProperty](#) *property, bool val)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, bool val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtBoolPropertyManager](#) (QObject *parent=0)
- [~QtBoolPropertyManager](#) ()
- bool [value](#) (const [QtProperty](#) *property) const
- bool [initialValue](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- QIcon [valueIcon](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.477.1 Constructor & Destructor Documentation**10.477.1.1 [QtBoolPropertyManager](#)()**

```
QtBoolPropertyManager::QtBoolPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.477.1.2 ~QtBoolPropertyManager()

```
QtBoolPropertyManager::~~QtBoolPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.477.2 Member Function Documentation

10.477.2.1 initializeProperty()

```
void QtBoolPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.477.2.2 setValue

```
void QtBoolPropertyManager::setValue (
    QtProperty * property,
    bool value ) [slot]
```

Sets the value of the given *property* to *value*.

See also

[value\(\)](#)

10.477.2.3 uninitializedProperty()

```
void QtBoolPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.477.2.4 value()

```
bool QtBoolPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by *this* manager, this function returns false.

See also

[setValue\(\)](#)

10.477.2.5 valueChanged

```
void QtBoolPropertyManager::valueChanged (
    QtProperty * property,
    bool value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

10.477.2.6 valueIcon()

```
QIcon QtBoolPropertyManager::valueIcon (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.477.2.7 valueText()

```
QString QtBoolPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.478 QtBoolPropertyManagerPrivate Class Reference

Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Attributes

- QMap< const [QtProperty](#) *, [Data](#) > **m_values**
- const QIcon **m_checkedIcon**
- const QIcon **m_uncheckedIcon**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.479 QtBrowserItem Class Reference

Public Member Functions

- [QtProperty](#) * [property](#) () const
- [QtBrowserItem](#) * [parent](#) () const
- [QList](#)< [QtBrowserItem](#) * > [children](#) () const
- [QtAbstractPropertyBrowser](#) * [browser](#) () const

Friends

- class [QtAbstractPropertyBrowserPrivate](#)

10.479.1 Member Function Documentation

10.479.1.1 browser()

```
QtAbstractPropertyBrowser * QtBrowserItem::browser ( ) const
```

Returns the property browser which owns *this* item.

10.479.1.2 children()

```
QList< QtBrowserItem * > QtBrowserItem::children ( ) const
```

Returns the children items of *this* item. The properties reproduced from children items are always the same as reproduced from associated property' children, for example:

The *childrenItems* list represents the same list as *childrenProperties*.

10.479.1.3 parent()

```
QtBrowserItem * QtBrowserItem::parent ( ) const
```

Returns the parent item of *this* item. Returns 0 if *this* item is associated with top-level property in item's property browser.

See also

[children\(\)](#)

10.479.1.4 property()

```
QtProperty * QtBrowserItem::property ( ) const
```

Returns the property which is associated with this item. Note that several items can be associated with the same property instance in the same property browser.

See also

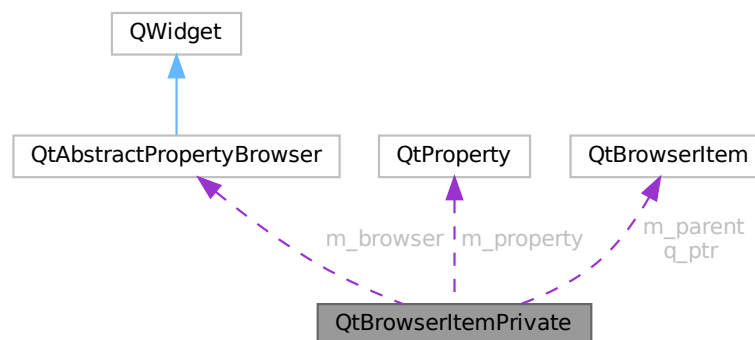
[QtAbstractPropertyBrowser::items\(\)](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

10.480 QtBrowserItemPrivate Class Reference

Collaboration diagram for QtBrowserItemPrivate:



Public Member Functions

- **QtBrowserItemPrivate** ([QtAbstractPropertyBrowser](#) *browser, [QtProperty](#) *property, [QtBrowserItem](#) *parent)
- void **addChild** ([QtBrowserItem](#) *index, [QtBrowserItem](#) *after)
- void **removeChild** ([QtBrowserItem](#) *index)

Public Attributes

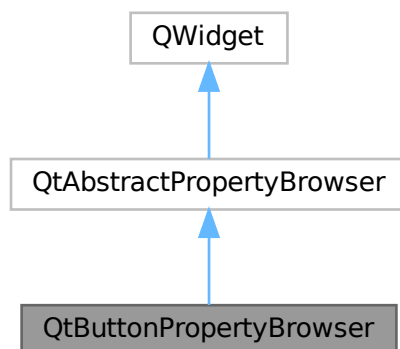
- [QtAbstractPropertyBrowser](#) *const **m_browser**
- [QtProperty](#) * **m_property**
- [QtBrowserItem](#) * **m_parent**
- [QtBrowserItem](#) * **q_ptr**
- [QList](#)< [QtBrowserItem](#) * > **m_children**

The documentation for this class was generated from the following file:

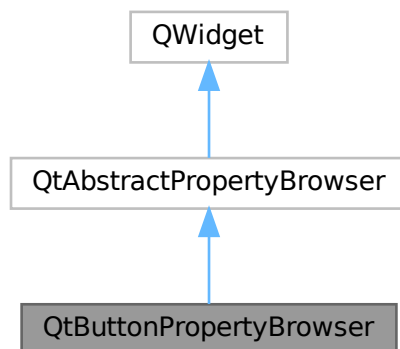
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

10.481 QPushButtonPropertyBrowser Class Reference

Inheritance diagram for QPushButtonPropertyBrowser:



Collaboration diagram for QPushButtonPropertyBrowser:



Signals

- void [collapsed](#) ([QtBrowserItem](#) *item)
- void [expanded](#) ([QtBrowserItem](#) *item)

Signals inherited from [QtAbstractPropertyBrowser](#)

- void [currentItemChanged](#) ([QtBrowserItem](#) *)

Public Member Functions

- [QtButtonPropertyBrowser](#) (QWidget *parent=0)
- [~QtButtonPropertyBrowser](#) ()
- void [setExpanded](#) (QtBrowserItem *item, bool [expanded](#))
- bool [isExpanded](#) (QtBrowserItem *item) const

Public Member Functions inherited from [QtAbstractPropertyBrowser](#)

- [QtAbstractPropertyBrowser](#) (QWidget *parent=0)
- [~QtAbstractPropertyBrowser](#) ()
- QList< [QtProperty](#) * > [properties](#) () const
- QList< [QtBrowserItem](#) * > [items](#) ([QtProperty](#) *property) const
- [QtBrowserItem](#) * [topLevelItem](#) ([QtProperty](#) *property) const
- QList< [QtBrowserItem](#) * > [topLevelItems](#) () const
- void [clear](#) ()
- template<class PropertyManager >
void [setFactoryForManager](#) (PropertyManager *manager, [QtAbstractEditorFactory](#)< PropertyManager > *factory)
- void [unsetFactoryForManager](#) ([QtAbstractPropertyManager](#) *manager)
- [QtBrowserItem](#) * [currentItem](#) () const
- void [setCurrentItem](#) ([QtBrowserItem](#) *)

Protected Member Functions

- void [itemInserted](#) ([QtBrowserItem](#) *item, [QtBrowserItem](#) *afterItem) override
- void [itemRemoved](#) ([QtBrowserItem](#) *item) override
- void [itemChanged](#) ([QtBrowserItem](#) *item) override

Protected Member Functions inherited from [QtAbstractPropertyBrowser](#)

- virtual QWidget * [createEditor](#) ([QtProperty](#) *property, QWidget *parent)

Additional Inherited Members

Public Slots inherited from [QtAbstractPropertyBrowser](#)

- [QtBrowserItem](#) * [addProperty](#) ([QtProperty](#) *property)
- [QtBrowserItem](#) * [insertProperty](#) ([QtProperty](#) *property, [QtProperty](#) *afterProperty)
- void [removeProperty](#) ([QtProperty](#) *property)

10.481.1 Constructor & Destructor Documentation

10.481.1.1 [QtButtonPropertyBrowser](#)()

```
QtButtonPropertyBrowser::QtButtonPropertyBrowser (
    QWidget * parent = 0 )
```

Creates a property browser with the given *parent*.

10.481.1.2 ~QPushButtonPropertyBrowser()

```
QPushButtonPropertyBrowser::~QPushButtonPropertyBrowser ( )
```

Destroys this property browser.

Note that the properties that were inserted into this browser are *not* destroyed since they may still be used in other browsers. The properties are owned by the manager that created them.

See also

[QtProperty](#), [QtAbstractPropertyManager](#)

10.481.2 Member Function Documentation

10.481.2.1 collapsed

```
void QPushButtonPropertyBrowser::collapsed (
    QtBrowserItem * item ) [signal]
```

This signal is emitted when the *item* is collapsed.

See also

[expanded\(\)](#), [setExpanded\(\)](#)

10.481.2.2 expanded

```
void QPushButtonPropertyBrowser::expanded (
    QtBrowserItem * item ) [signal]
```

This signal is emitted when the *item* is expanded.

See also

[collapsed\(\)](#), [setExpanded\(\)](#)

10.481.2.3 isExpanded()

```
bool QPushButtonPropertyBrowser::isExpanded (
    QtBrowserItem * item ) const
```

Returns true if the *item* is expanded; otherwise returns false.

See also

[setExpanded\(\)](#)

10.481.2.4 itemChanged()

```
void QtButtonPropertyBrowser::itemChanged (
    QtBrowserItem * item ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

10.481.2.5 itemInserted()

```
void QtButtonPropertyBrowser::itemInserted (
    QtBrowserItem * item,
    QtBrowserItem * afterItem ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

10.481.2.6 itemRemoved()

```
void QtButtonPropertyBrowser::itemRemoved (
    QtBrowserItem * item ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

10.481.2.7 setExpanded()

```
void QtButtonPropertyBrowser::setExpanded (
    QtBrowserItem * item,
    bool expanded )
```

Sets the *item* to either collapse or expanded, depending on the value of *expanded*.

See also

[isExpanded\(\)](#), [expanded\(\)](#), [collapsed\(\)](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtbuttonpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtbuttonpropertybrowser.cpp

10.482 QtButtonPropertyBrowserPrivate Class Reference

Classes

- struct [WidgetItem](#)

Public Member Functions

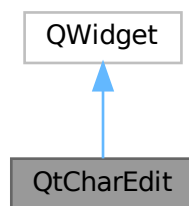
- void **init** (QWidget *parent)
- void **propertyInserted** (QtBrowserItem *index, QtBrowserItem *afterIndex)
- void **propertyRemoved** (QtBrowserItem *index)
- void **propertyChanged** (QtBrowserItem *index)
- QWidget * **createEditor** (QtProperty *property, QWidget *parent) const
- void **slotEditorDestroyed** ()
- void **slotUpdate** ()
- void **slotToggled** (bool checked)

The documentation for this class was generated from the following file:

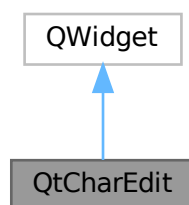
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtbuttonpropertybrowser.cpp

10.483 QtCharEdit Class Reference

Inheritance diagram for QtCharEdit:



Collaboration diagram for QtCharEdit:



Public Slots

- void **setValue** (const QChar &value)

Signals

- void **valueChanged** (const QChar &value)

Public Member Functions

- **QtCharEdit** (QWidget *parent=0)
- QChar **value** () const
- bool **eventFilter** (QObject *o, QEvent *e) override

Protected Member Functions

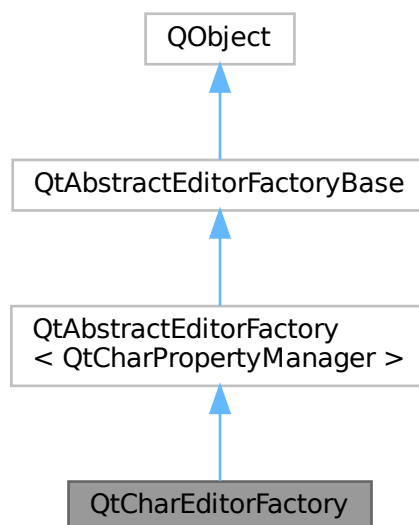
- void **focusInEvent** (QFocusEvent *e) override
- void **focusOutEvent** (QFocusEvent *e) override
- void **keyPressEvent** (QKeyEvent *e) override
- void **keyReleaseEvent** (QKeyEvent *e) override
- bool **event** (QEvent *e) override

The documentation for this class was generated from the following file:

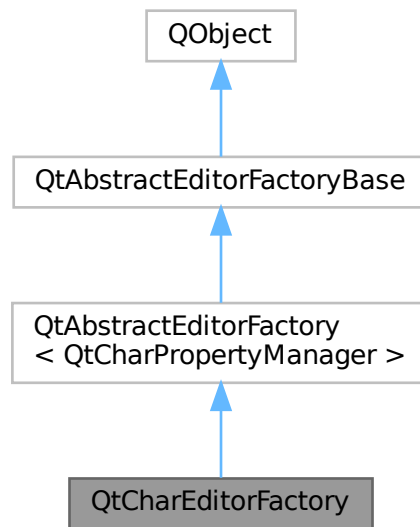
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.484 QtCharEditorFactory Class Reference

Inheritance diagram for QtCharEditorFactory:



Collaboration diagram for QtCharEditorFactory:



Public Member Functions

- [QtCharEditorFactory](#) (QObject *parent=0)
- [~QtCharEditorFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory < QtCharPropertyManager >](#)

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtCharPropertyManager *manager)
- void [removePropertyManager](#) (QtCharPropertyManager *manager)
- QSet< QtCharPropertyManager * > [propertyManagers](#) () const
- QtCharPropertyManager * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtCharPropertyManager *manager) override
- QWidget * [createEditor](#) (QtCharPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtCharPropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory < QtCharPropertyManager >](#)

- virtual QWidget * [createEditor](#) (QtCharPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtCharPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtCharPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

10.484.1 Constructor & Destructor Documentation

10.484.1.1 QtCharEditorFactory()

```
QtCharEditorFactory::QtCharEditorFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.484.1.2 ~QtCharEditorFactory()

```
QtCharEditorFactory::~~QtCharEditorFactory ( )
```

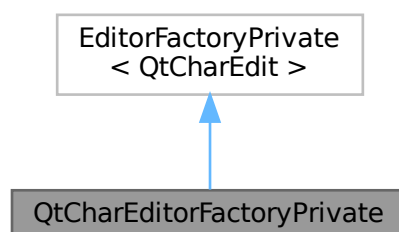
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

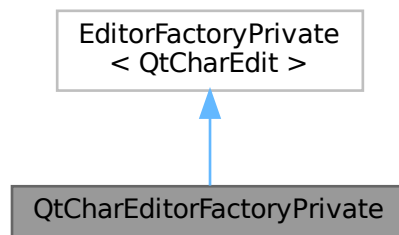
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.485 QtCharEditorFactoryPrivate Class Reference

Inheritance diagram for QtCharEditorFactoryPrivate:



Collaboration diagram for QtCharEditorFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, const QChar &value)
- void **slotSetValue** (const QChar &value)

Public Member Functions inherited from [EditorFactoryPrivate< QtCharEdit >](#)

- [QtCharEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QtCharEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QtCharEdit >](#)

- typedef QList< [QtCharEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QtCharEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QtCharEdit >](#)

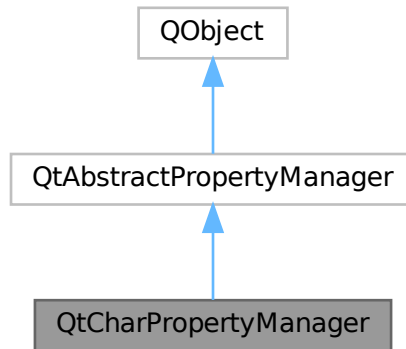
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

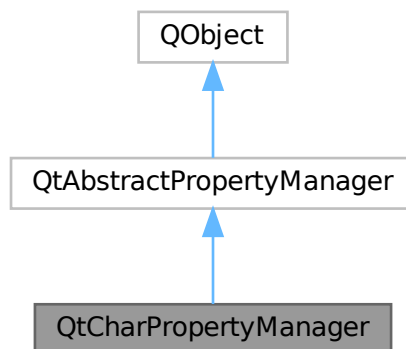
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.486 QtCharPropertyManager Class Reference

Inheritance diagram for QtCharPropertyManager:



Collaboration diagram for QtCharPropertyManager:



Public Slots

- void `setValue` (`QtProperty` *property, const QChar &val)

Signals

- void `valueChanged` (`QtProperty` *property, const QChar &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtCharPropertyManager](#) (QObject *parent=0)
- [~QtCharPropertyManager](#) ()
- QChar [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.486.1 Constructor & Destructor Documentation

10.486.1.1 [QtCharPropertyManager](#)()

```
QtCharPropertyManager::QtCharPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.486.1.2 [~QtCharPropertyManager](#)()

```
QtCharPropertyManager::~~QtCharPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.486.2 Member Function Documentation

10.486.2.1 initializeProperty()

```
void QtCharPropertyManager::initializeProperty (  
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.486.2.2 setValue

```
void QtCharPropertyManager::setValue (  
    QtProperty * property,  
    const QChar & value ) [slot]
```

Sets the value of the given *property* to *value*.

See also

[value\(\)](#), [valueChanged\(\)](#)

10.486.2.3 uninitializedProperty()

```
void QtCharPropertyManager::uninitializedProperty (  
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.486.2.4 value()

```
QChar QtCharPropertyManager::value (  
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an null QChar object.

See also

[setValue\(\)](#)

10.486.2.5 valueChanged

```
void QtCharPropertyManager::valueChanged (
    QtProperty * property,
    const QChar & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

10.486.2.6 valueText()

```
QString QtCharPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.487 QtCharPropertyManagerPrivate Class Reference

Public Types

- typedef QMap< const [QtProperty](#) *, QChar > **PropertyValueMap**

Public Attributes

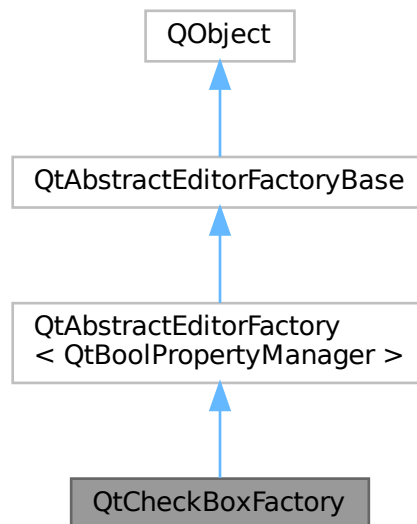
- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

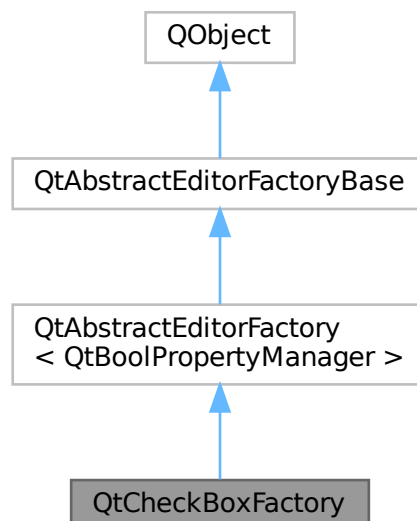
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.488 QtCheckBoxFactory Class Reference

Inheritance diagram for QtCheckBoxFactory:



Collaboration diagram for QtCheckBoxFactory:



Public Member Functions

- [QtCheckBoxFactory](#) (QObject *parent=0)
- [~QtCheckBoxFactory](#) ()

Public Member Functions inherited from**[QtAbstractEditorFactory](#) < [QtBoolPropertyManager](#) >**

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtBoolPropertyManager *manager)
- void [removePropertyManager](#) (QtBoolPropertyManager *manager)
- QSet< [QtBoolPropertyManager](#) * > [propertyManagers](#) () const
- [QtBoolPropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtBoolPropertyManager *manager) override
- QWidget * [createEditor](#) (QtBoolPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtBoolPropertyManager *manager) override

Protected Member Functions inherited from**[QtAbstractEditorFactory](#) < [QtBoolPropertyManager](#) >**

- virtual QWidget * [createEditor](#) (QtBoolPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtBoolPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtBoolPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members**Protected Slots inherited from [QtAbstractEditorFactoryBase](#)****10.488.1 Constructor & Destructor Documentation****10.488.1.1 QtCheckBoxFactory()**

```
QtCheckBoxFactory::QtCheckBoxFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.488.1.2 ~QtCheckBoxFactory()

```
QtCheckBoxFactory::~QtCheckBoxFactory ( )
```

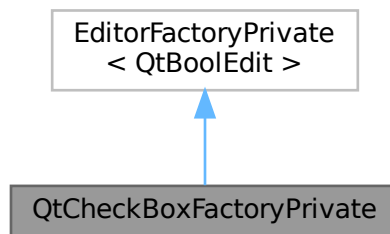
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

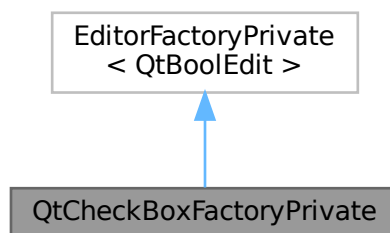
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.489 QtCheckBoxFactoryPrivate Class Reference

Inheritance diagram for QtCheckBoxFactoryPrivate:



Collaboration diagram for QtCheckBoxFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, bool value)
- void **slotSetValue** (bool value)

Public Member Functions inherited from [EditorFactoryPrivate< QtBoolEdit >](#)

- [QtBoolEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QtBoolEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members**Public Types inherited from [EditorFactoryPrivate< QtBoolEdit >](#)**

- typedef QList< [QtBoolEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QtBoolEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QtBoolEdit >](#)

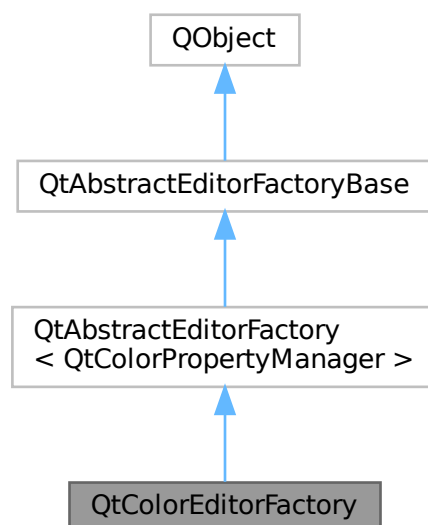
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

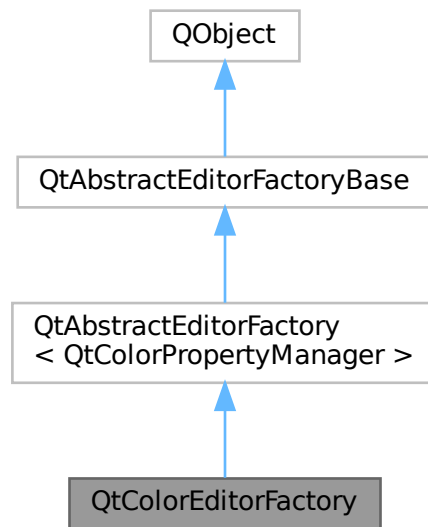
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.490 QColorEditorFactory Class Reference

Inheritance diagram for QColorEditorFactory:



Collaboration diagram for QColorEditorFactory:



Public Member Functions

- [QColorEditorFactory](#) (QObject *parent=0)
- [~QColorEditorFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory< QColorPropertyManager >](#)

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtColorPropertyManager *manager)
- void [removePropertyManager](#) (QtColorPropertyManager *manager)
- QSet< QtColorPropertyManager * > [propertyManagers](#) () const
- QtColorPropertyManager * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtColorPropertyManager *manager) override
- QWidget * [createEditor](#) (QtColorPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtColorPropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory< QColorPropertyManager >](#)

- virtual QWidget * [createEditor](#) (QtColorPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtColorPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtColorPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from QColorEditorFactoryBase

- QColorEditorFactoryBase (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from QColorEditorFactoryBase

10.490.1 Constructor & Destructor Documentation

10.490.1.1 QColorEditorFactory()

```
QColorEditorFactory::QColorEditorFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.490.1.2 ~QColorEditorFactory()

```
QColorEditorFactory::~QColorEditorFactory ( )
```

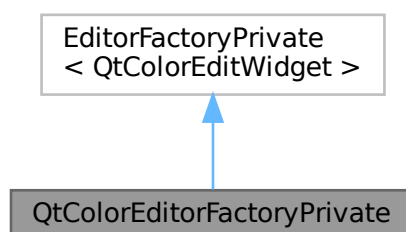
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

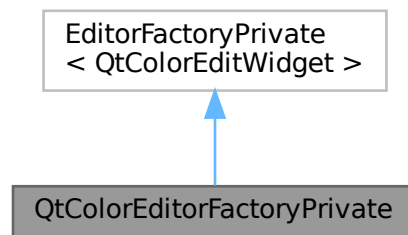
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.491 QColorEditorFactoryPrivate Class Reference

Inheritance diagram for QColorEditorFactoryPrivate:



Collaboration diagram for QColorEditorFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** (QtProperty *property, const QColor &value)
- void **slotSetValue** (const QColor &value)

Public Member Functions inherited from EditorFactoryPrivate< QColorEditWidget >

- QColorEditWidget * **createEditor** (QtProperty *property, QWidget *parent)
- void **initializeEditor** (QtProperty *property, QColorEditWidget *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from EditorFactoryPrivate< QColorEditWidget >

- typedef QList< QColorEditWidget * > **EditorList**
- typedef QMap< QtProperty *, EditorList > **PropertyToEditorListMap**
- typedef QMap< QColorEditWidget *, QtProperty * > **EditorToPropertyMap**

Public Attributes inherited from EditorFactoryPrivate< QColorEditWidget >

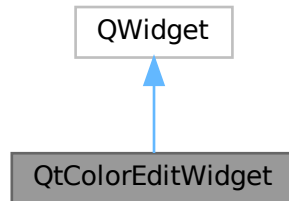
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

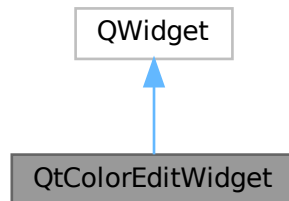
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.492 QColorEditWidget Class Reference

Inheritance diagram for QColorEditWidget:



Collaboration diagram for QColorEditWidget:



Public Slots

- void **setValue** (const QColor &value)

Signals

- void **valueChanged** (const QColor &value)

Public Member Functions

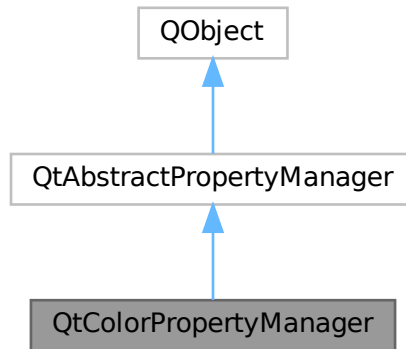
- **QColorEditWidget** (QWidget *parent)
- bool **eventFilter** (QObject *obj, QEvent *ev) override

The documentation for this class was generated from the following file:

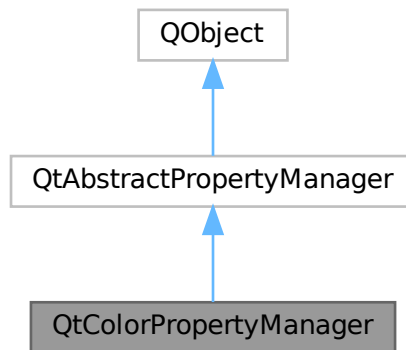
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.493 QColorPropertyManager Class Reference

Inheritance diagram for QColorPropertyManager:



Collaboration diagram for QColorPropertyManager:



Public Slots

- void `setValue` (`QtProperty` *property, const QColor &val)

Signals

- void `valueChanged` (`QtProperty` *property, const QColor &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QColorPropertyManager](#) (QObject *parent=0)
- [~QColorPropertyManager](#) ()
- [QtIntPropertyManager](#) * [subIntPropertyManager](#) () const
- QColor [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- QIcon [valueIcon](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.493.1 Constructor & Destructor Documentation

10.493.1.1 QColorPropertyManager()

```
QColorPropertyManager::QColorPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.493.1.2 ~QColorPropertyManager()

```
QColorPropertyManager::~~QColorPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.493.2 Member Function Documentation

10.493.2.1 initializeProperty()

```
void QColorPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.493.2.2 setValue

```
void QColorPropertyManager::setValue (
    QtProperty * property,
    const QColor & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

See also

[value\(\)](#), [valueChanged\(\)](#)

10.493.2.3 subIntPropertyManager()

```
QtIntPropertyManager * QColorPropertyManager::subIntPropertyManager ( ) const
```

Returns the manager that produces the nested *red*, *green* and *blue* subproperties.

In order to provide editing widgets for the subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

10.493.2.4 uninitializedProperty()

```
void QColorPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.493.2.5 value()

```
QColor QColorPropertyManager::value (
    const QColorProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by *this* manager, this function returns an invalid color.

See also

[setValue\(\)](#)

10.493.2.6 valueChanged

```
void QColorPropertyManager::valueChanged (
    QColorProperty * property,
    const QColor & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.493.2.7 valueIcon()

```
QIcon QColorPropertyManager::valueIcon (
    const QColorProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.493.2.8 valueText()

```
QString QColorPropertyManager::valueText (
    const QColorProperty * property ) const [override], [protected], [virtual]
```

\reimp

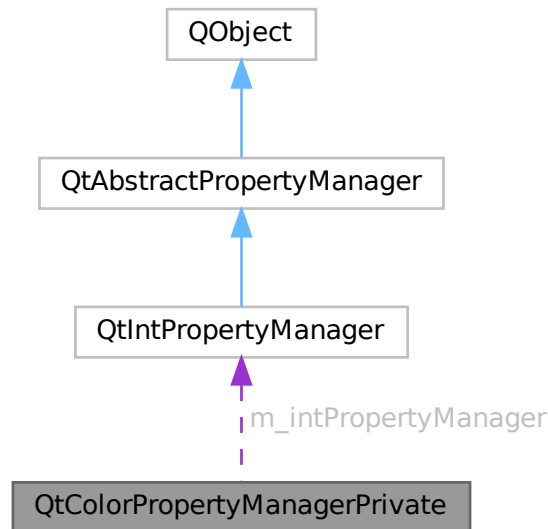
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.494 QColorPropertyManagerPrivate Class Reference

Collaboration diagram for QColorPropertyManagerPrivate:



Public Types

- typedef QMap< const QColorProperty *, QColor > **PropertyValueMap**

Public Member Functions

- void **slotIntChanged** (QColorProperty *property, int value)
- void **slotPropertyDestroyed** (QColorProperty *property)

Public Attributes

- PropertyValueMap **m_values**
- QColorPropertyManager * **m_intPropertyManager**
- QMap< const QColorProperty *, QColorProperty * > **m_propertyToR**
- QMap< const QColorProperty *, QColorProperty * > **m_propertyToG**
- QMap< const QColorProperty *, QColorProperty * > **m_propertyToB**
- QMap< const QColorProperty *, QColorProperty * > **m_propertyToA**
- QMap< const QColorProperty *, QColorProperty * > **m_rToProperty**
- QMap< const QColorProperty *, QColorProperty * > **m_gToProperty**
- QMap< const QColorProperty *, QColorProperty * > **m_bToProperty**
- QMap< const QColorProperty *, QColorProperty * > **m_aToProperty**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.495 QtCursorDatabase Class Reference

Public Member Functions

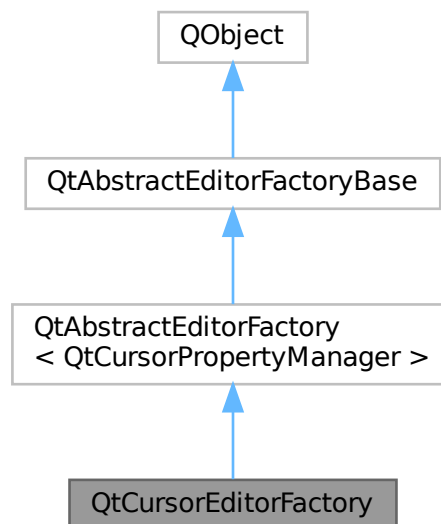
- void **clear** ()
- QStringList **cursorShapeNames** () const
- QMap< int, QIcon > **cursorShapelcons** () const
- QString **cursorToShapeName** (const QCursor &cursor) const
- QIcon **cursorToShapelcon** (const QCursor &cursor) const
- int **cursorToValue** (const QCursor &cursor) const
- QCursor **valueToCursor** (int value) const

The documentation for this class was generated from the following files:

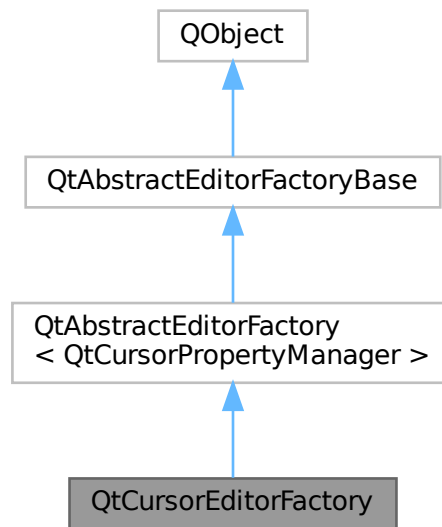
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

10.496 QtCursorEditorFactory Class Reference

Inheritance diagram for QtCursorEditorFactory:



Collaboration diagram for QtCursorEditorFactory:



Public Member Functions

- [QtCursorEditorFactory](#) (QObject *parent=0)
- [~QtCursorEditorFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory< QtCursorPropertyManager >](#)

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtCursorPropertyManager *manager)
- void [removePropertyManager](#) (QtCursorPropertyManager *manager)
- QSet< QtCursorPropertyManager * > [propertyManagers](#) () const
- QtCursorPropertyManager * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtCursorPropertyManager *manager) override
- QWidget * [createEditor](#) (QtCursorPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtCursorPropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory< QtCursorPropertyManager >](#)

- virtual QWidget * [createEditor](#) (QtCursorPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtCursorPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtCursorPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

10.496.1 Constructor & Destructor Documentation

10.496.1.1 QtCursorEditorFactory()

```
QtCursorEditorFactory::QtCursorEditorFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.496.1.2 ~QtCursorEditorFactory()

```
QtCursorEditorFactory::~~QtCursorEditorFactory ( )
```

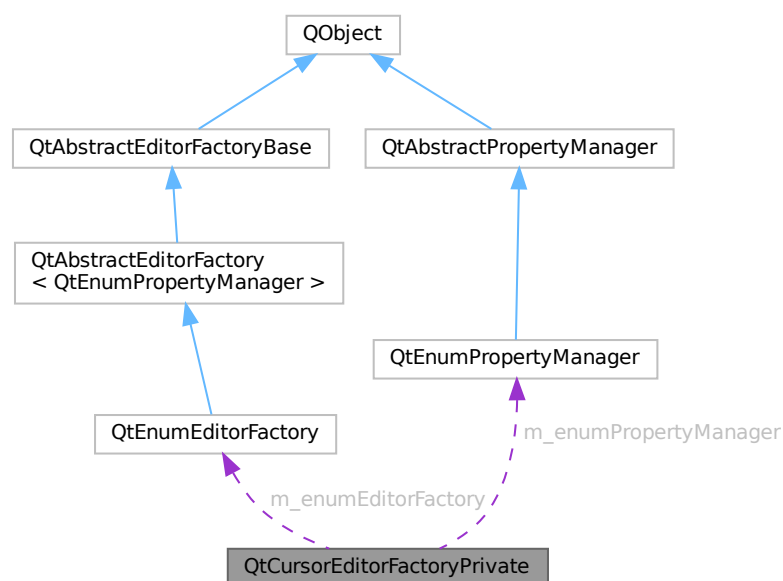
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.497 QtCursorEditorFactoryPrivate Class Reference

Collaboration diagram for QtCursorEditorFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, const [QCursor](#) &cursor)
- void **slotEnumChanged** ([QtProperty](#) *property, int value)
- void **slotEditorDestroyed** ([QObject](#) *object)

Public Attributes

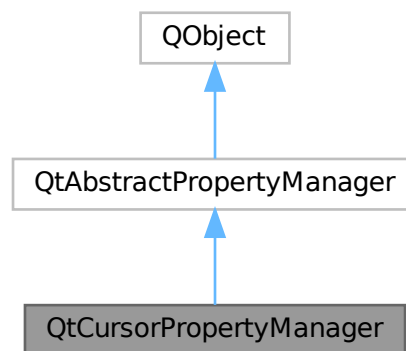
- [QtEnumEditorFactory](#) * **m_enumEditorFactory**
- [QtEnumPropertyManager](#) * **m_enumPropertyManager**
- [QMap](#)< [QtProperty](#) *, [QtProperty](#) * > **m_propertyToEnum**
- [QMap](#)< [QtProperty](#) *, [QtProperty](#) * > **m_enumToProperty**
- [QMap](#)< [QtProperty](#) *, [QWidgetList](#) > **m_enumToEditors**
- [QMap](#)< [QWidget](#) *, [QtProperty](#) * > **m_editorToEnum**
- bool **m_updatingEnum**

The documentation for this class was generated from the following file:

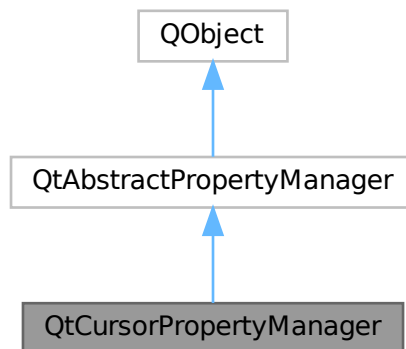
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.498 QtCursorPropertyManager Class Reference

Inheritance diagram for QtCursorPropertyManager:



Collaboration diagram for QtCursorPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const [QCursor](#) &val)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const [QCursor](#) &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtCursorPropertyManager](#) ([QObject](#) *parent=0)
- [~QtCursorPropertyManager](#) ()
- [QCursor](#) [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) ([QObject](#) *parent=0)
- [~QtAbstractPropertyManager](#) ()
- [QSet](#)< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const [QString](#) &name=[QString](#)())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- QIcon [valueIcon](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.498.1 Constructor & Destructor Documentation

10.498.1.1 [QtCursorPropertyManager\(\)](#)

```
QtCursorPropertyManager::QtCursorPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.498.1.2 [~QtCursorPropertyManager\(\)](#)

```
QtCursorPropertyManager::~~QtCursorPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.498.2 Member Function Documentation

10.498.2.1 [initializeProperty\(\)](#)

```
void QtCursorPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.498.2.2 [setValue](#)

```
void QtCursorPropertyManager::setValue (
    QtProperty * property,
    const QCursor & value ) [slot]
```

Sets the value of the given *property* to *value*.

See also

[value\(\)](#), [valueChanged\(\)](#)

10.498.2.3 uninitializedProperty()

```
void QtCursorPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.498.2.4 value()

```
QCursor QtCursorPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns a default QCursor object.

See also

[setValue\(\)](#)

10.498.2.5 valueChanged

```
void QtCursorPropertyManager::valueChanged (
    QtProperty * property,
    const QCursor & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.498.2.6 valueIcon()

```
QIcon QtCursorPropertyManager::valueIcon (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.498.2.7 valueText()

```
QString QtCursorPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.499 QtCursorPropertyManagerPrivate Class Reference

Public Types

- typedef QMap< const [QtProperty](#) *, QCursor > **PropertyValueMap**

Public Attributes

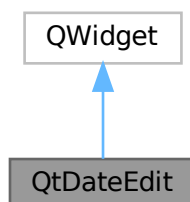
- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

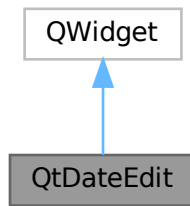
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.500 QtDateEdit Class Reference

Inheritance diagram for QtDateEdit:



Collaboration diagram for QtDateEdit:



Signals

- void **dateChanged** (QDate)

Public Member Functions

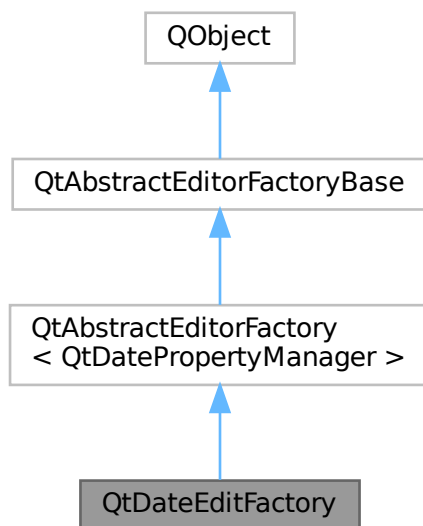
- **QtDateEdit** (QWidget *parent=Q_NULLPTR)
- QDate **date** ()
- void **setDate** (QDate)
- void **setInitialDate** (QDate)
- void **setDateRange** (QDate, QDate)
- void **setDisplayFormat** (QString)
- void **setCalendarPopup** (bool)

The documentation for this class was generated from the following files:

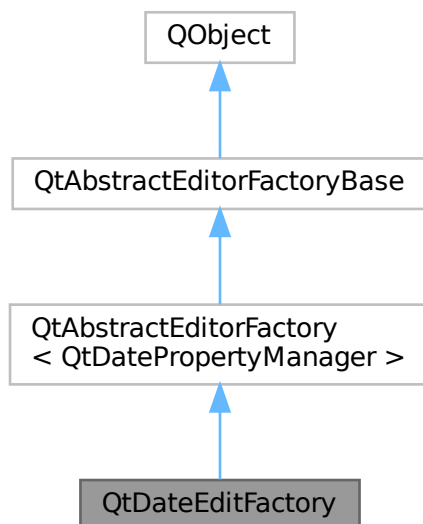
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

10.501 QtDateEditFactory Class Reference

Inheritance diagram for QtDateEditFactory:



Collaboration diagram for QtDateEditFactory:



Public Member Functions

- [QtDateEditFactory](#) (QObject *parent=0)
- [~QtDateEditFactory](#) ()

Public Member Functions inherited from**[QtAbstractEditorFactory](#) < [QtDatePropertyManager](#) >**

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) ([QtProperty](#) *property, QWidget *parent) override
- void [addPropertyManager](#) ([QtDatePropertyManager](#) *manager)
- void [removePropertyManager](#) ([QtDatePropertyManager](#) *manager)
- QSet< [QtDatePropertyManager](#) * > [propertyManagers](#) () const
- [QtDatePropertyManager](#) * [propertyManager](#) ([QtProperty](#) *property) const

Protected Member Functions

- void **[connectPropertyManager](#)** ([QtDatePropertyManager](#) *manager) override
- QWidget * **[createEditor](#)** ([QtDatePropertyManager](#) *manager, [QtProperty](#) *property, QWidget *parent) override
- void **[disconnectPropertyManager](#)** ([QtDatePropertyManager](#) *manager) override

Protected Member Functions inherited from**[QtAbstractEditorFactory](#) < [QtDatePropertyManager](#) >**

- virtual QWidget * [createEditor](#) ([QtDatePropertyManager](#) *manager, [QtProperty](#) *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) ([QtDatePropertyManager](#) *manager)=0
- virtual void [disconnectPropertyManager](#) ([QtDatePropertyManager](#) *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members**Protected Slots inherited from [QtAbstractEditorFactoryBase](#)****10.501.1 Constructor & Destructor Documentation****10.501.1.1 [QtDateEditFactory](#)()**

```
QtDateEditFactory::QtDateEditFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.501.1.2 ~QtDateEditFactory()

```
QtDateEditFactory::~QtDateEditFactory ( )
```

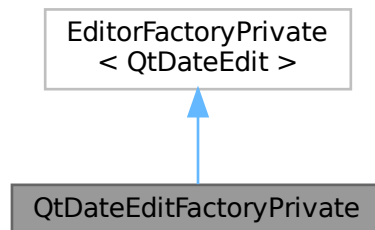
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

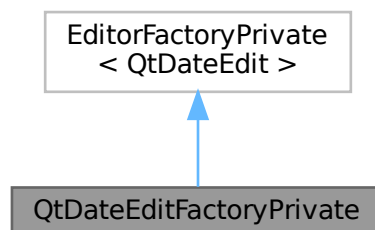
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.502 QtDateEditFactoryPrivate Class Reference

Inheritance diagram for QtDateEditFactoryPrivate:



Collaboration diagram for QtDateEditFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, QDate value)
- void **slotRangeChanged** ([QtProperty](#) *property, QDate min, QDate max)
- void **slotSetValue** (QDate value)

Public Member Functions inherited from [EditorFactoryPrivate< QtDateEdit >](#)

- [QtDateEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QtDateEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members**Public Types inherited from [EditorFactoryPrivate< QtDateEdit >](#)**

- typedef QList< [QtDateEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QtDateEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QtDateEdit >](#)

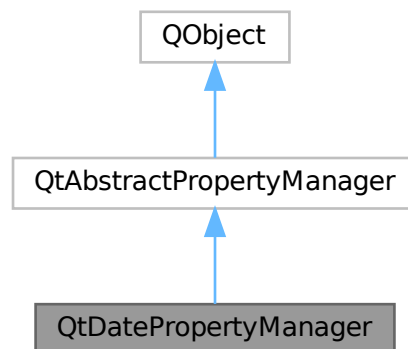
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

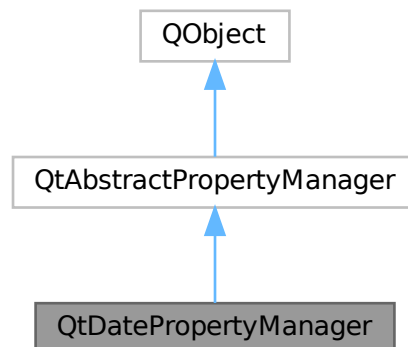
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.503 QtDatePropertyManager Class Reference

Inheritance diagram for QtDatePropertyManager:



Collaboration diagram for `QtDatePropertyManager`:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, `QDate` val)
- void **setValueOnly** ([QtProperty](#) *property, `QDate` val)
- void [setMinimum](#) ([QtProperty](#) *property, `QDate` minVal)
- void [setMaximum](#) ([QtProperty](#) *property, `QDate` maxVal)
- void [setRange](#) ([QtProperty](#) *property, `QDate` minVal, `QDate` maxVal)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, `QDate` val)
- void [rangeChanged](#) ([QtProperty](#) *property, `QDate` minVal, `QDate` maxVal)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtDatePropertyManager](#) (`QObject` *parent=0)
- [~QtDatePropertyManager](#) ()
- `QDate` [value](#) (const [QtProperty](#) *property) const
- `QDate` **initialValue** (const [QtProperty](#) *property) const
- `QDate` [minimum](#) (const [QtProperty](#) *property) const
- `QDate` [maximum](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.503.1 Constructor & Destructor Documentation

10.503.1.1 QtDatePropertyManager()

```
QtDatePropertyManager::QtDatePropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.503.1.2 ~QtDatePropertyManager()

```
QtDatePropertyManager::~~QtDatePropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.503.2 Member Function Documentation

10.503.2.1 initializeProperty()

```
void QtDatePropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.503.2.2 maximum()

```
QDate QtDatePropertyManager::maximum (
    const QtProperty * property ) const
```

Returns the given *property*'s maximum date.

See also

[minimum\(\)](#), [setRange\(\)](#)

10.503.2.3 minimum()

```
QDate QtDatePropertyManager::minimum (
    const QtProperty * property ) const
```

Returns the given *property*'s minimum date.

See also

[maximum\(\)](#), [setRange\(\)](#)

10.503.2.4 rangeChanged

```
void QtDatePropertyManager::rangeChanged (
    QtProperty * property,
    QDate minimum,
    QDate maximum ) [signal]
```

This signal is emitted whenever a property created by this manager changes its range of valid dates, passing a pointer to the *property* and the new *minimum* and *maximum* dates.

See also

[setRange\(\)](#)

10.503.2.5 setMaximum

```
void QtDatePropertyManager::setMaximum (
    QtProperty * property,
    QDate maxVal ) [slot]
```

Sets the maximum value for the given *property* to *maxVal*.

When setting the maximum value, the minimum and current values are adjusted if necessary (ensuring that the range remains valid and that the current value is within in the range).

See also

[maximum\(\)](#), [setRange\(\)](#)

10.503.2.6 setMinimum

```
void QtDatePropertyManager::setMinimum (
    QtProperty * property,
    QDate minVal ) [slot]
```

Sets the minimum value for the given *property* to *minVal*.

When setting the minimum value, the maximum and current values are adjusted if necessary (ensuring that the range remains valid and that the current value is within in the range).

See also

[minimum\(\)](#), [setRange\(\)](#)

10.503.2.7 setRange

```
void QtDatePropertyManager::setRange (
    QtProperty * property,
    QDate minimum,
    QDate maximum ) [slot]
```

Sets the range of valid dates.

This is a convenience function defining the range of valid dates in one go; setting the *minimum* and *maximum* values for the given *property* with a single function call.

When setting a new date range, the current value is adjusted if necessary (ensuring that the value remains in date range).

See also

[setMinimum\(\)](#), [setMaximum\(\)](#), [rangeChanged\(\)](#)

10.503.2.8 setValue

```
void QtDatePropertyManager::setValue (
    QtProperty * property,
    QDate value ) [slot]
```

Sets the value of the given *property* to *value*.

If the specified *value* is not a valid date according to the given *property's* range, the value is adjusted to the nearest valid value within the range.

See also

[value\(\)](#), [setRange\(\)](#), [valueChanged\(\)](#)

10.503.2.9 uninitializedProperty()

```
void QtDatePropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.503.2.10 value()

```
QDate QtDatePropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by *this* manager, this function returns an invalid date.

See also

[setValue\(\)](#)

10.503.2.11 valueChanged

```
void QtDatePropertyManager::valueChanged (
    QtProperty * property,
    QDate value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.503.2.12 valueText()

```
QString QtDatePropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.504 QtDatePropertyManagerPrivate Class Reference

Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- **QtDatePropertyManagerPrivate** ([QtDatePropertyManager](#) *q)

Public Attributes

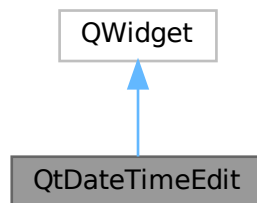
- QString **m_format**
- QMap< const [QtProperty](#) *, [Data](#) > **m_values**

The documentation for this class was generated from the following file:

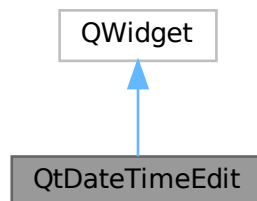
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.505 QtDateTimeEdit Class Reference

Inheritance diagram for QtDateTimeEdit:



Collaboration diagram for QtDateTimeEdit:



Signals

- void **dateTimeChanged** (const QDateTime &)

Public Member Functions

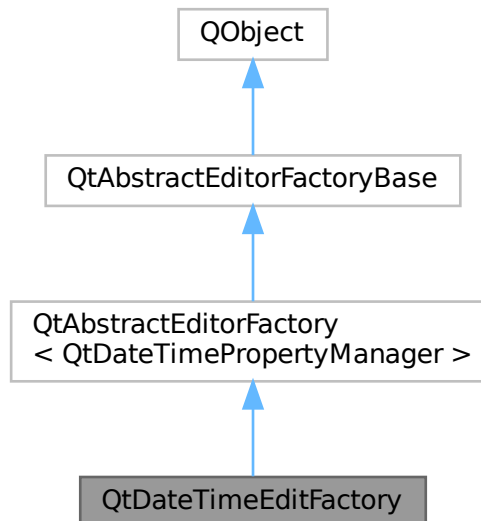
- **QtDateTimeEdit** (QWidget *parent=Q_NULLPTR)
- QDateTime **dateTime** ()
- void **setDateTime** (const QDateTime &)
- void **setInitialDateTime** (const QDateTime &)
- void **setDateTimeRange** (const QDateTime &, const QDateTime &)
- void **setDisplayFormat** (const QString &)
- void **setCalendarPopup** (bool)

The documentation for this class was generated from the following files:

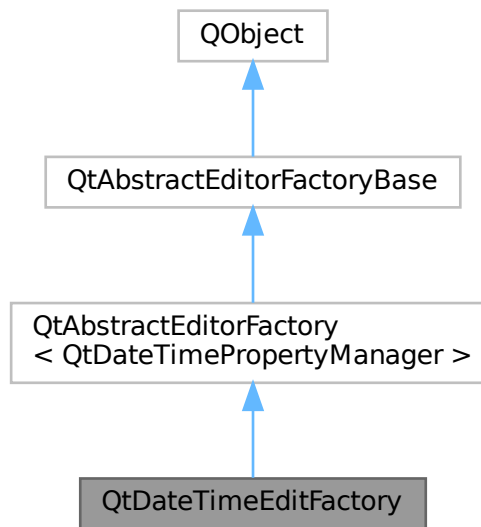
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

10.506 QtDateTimeEditFactory Class Reference

Inheritance diagram for QtDateTimeEditFactory:



Collaboration diagram for QDateTimeEditFactory:



Public Member Functions

- [QDateTimeEditFactory](#) (QObject *parent=0)
- [~QDateTimeEditFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory< QDateTimePropertyManager >](#)

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QDateTimePropertyManager *manager)
- void [removePropertyManager](#) (QDateTimePropertyManager *manager)
- QSet< QDateTimePropertyManager * > [propertyManagers](#) () const
- QDateTimePropertyManager * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QDateTimePropertyManager *manager) override
- QWidget * [createEditor](#) (QDateTimePropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QDateTimePropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory< QDateTimePropertyManager >](#)

- virtual QWidget * [createEditor](#) (QDateTimePropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QDateTimePropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QDateTimePropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

10.506.1 Constructor & Destructor Documentation

10.506.1.1 QDateTimeEditFactory()

```
QDateTimeEditFactory::QDateTimeEditFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.506.1.2 ~QDateTimeEditFactory()

```
QDateTimeEditFactory::~~QDateTimeEditFactory ( )
```

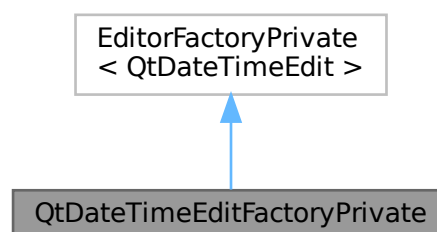
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

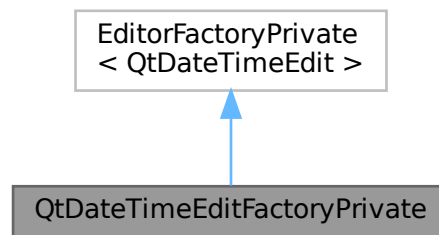
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.507 QDateTimeEditFactoryPrivate Class Reference

Inheritance diagram for QDateTimeEditFactoryPrivate:



Collaboration diagram for QDateTimeEditFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, const QDateTime &value)
- void **slotSetValue** (const QDateTime &value)

Public Member Functions inherited from [EditorFactoryPrivate< QDateTimeEdit >](#)

- [QDateTimeEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QDateTimeEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QDateTimeEdit >](#)

- typedef QList< [QDateTimeEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QDateTimeEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QDateTimeEdit >](#)

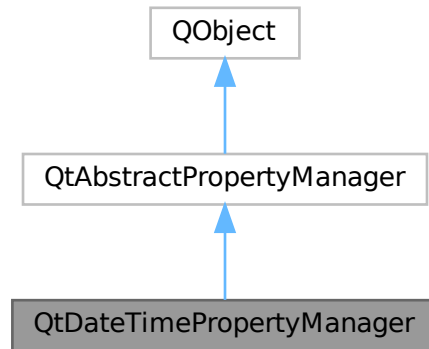
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

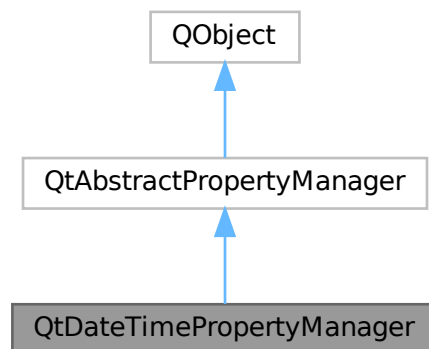
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.508 QDateTimePropertyManager Class Reference

Inheritance diagram for QDateTimePropertyManager:



Collaboration diagram for QDateTimePropertyManager:



Public Slots

- void `setValue` ([QtProperty](#) *property, const QDateTime &val)
- void `setValueOnly` ([QtProperty](#) *property, const QDateTime &val)

Signals

- void `valueChanged` ([QtProperty](#) *property, const QDateTime &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QDateTimePropertyManager](#) (QObject *parent=0)
- [~QDateTimePropertyManager](#) ()
- QDateTime [value](#) (const [QtProperty](#) *property) const
- QDateTime [initialValue](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.508.1 Constructor & Destructor Documentation

10.508.1.1 QDateTimePropertyManager()

```
QDateTimePropertyManager::QDateTimePropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.508.1.2 ~QDateTimePropertyManager()

```
QDateTimePropertyManager::~~QDateTimePropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.508.2 Member Function Documentation

10.508.2.1 initializeProperty()

```
void QDateTimePropertyManager::initializeProperty (  
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.508.2.2 setValue

```
void QDateTimePropertyManager::setValue (  
    QtProperty * property,  
    const QDateTime & value ) [slot]
```

Sets the value of the given *property* to *value*.

See also

[value\(\)](#), [valueChanged\(\)](#)

10.508.2.3 uninitializedProperty()

```
void QDateTimePropertyManager::uninitializedProperty (  
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.508.2.4 value()

```
QDateTime QDateTimePropertyManager::value (  
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an invalid QDateTime object.

See also

[setValue\(\)](#)

10.508.2.5 valueChanged

```
void QDateTimePropertyManager::valueChanged (
    QtProperty * property,
    const QDateTime & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

10.508.2.6 valueText()

```
QString QDateTimePropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.509 QDateTimePropertyManagerPrivate Class Reference

Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- **QDateTimePropertyManagerPrivate** ([QDateTimePropertyManager](#) *q)

Public Attributes

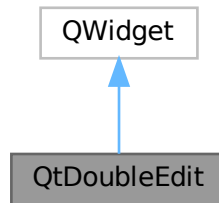
- const QString **m_format**
- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

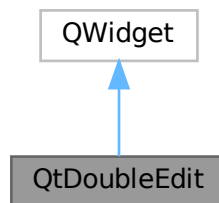
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.510 QtDoubleEdit Class Reference

Inheritance diagram for QtDoubleEdit:



Collaboration diagram for QtDoubleEdit:



Signals

- void **valueChanged** (double)

Public Member Functions

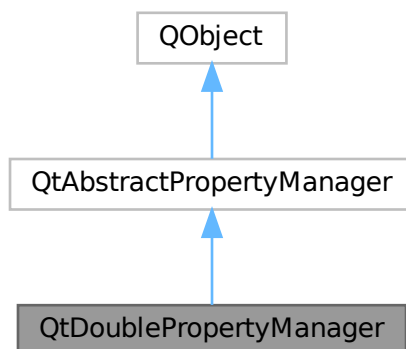
- **QtDoubleEdit** (QWidget *parent=Q_NULLPTR)
- int **value** ()
- void **setValue** (double)
- void **setInitialValue** (double)
- void **setDecimals** (double)
- void **setSingleStep** (double)
- void **setRange** (double, double)
- void **setKeyboardTracking** (bool)

The documentation for this class was generated from the following files:

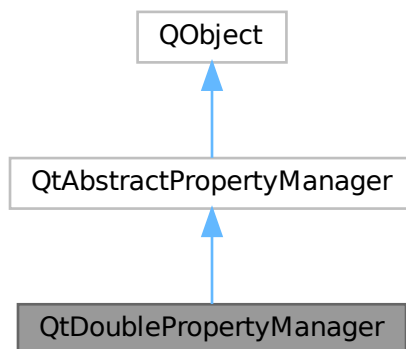
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

10.511 QtDoublePropertyManager Class Reference

Inheritance diagram for QtDoublePropertyManager:



Collaboration diagram for QtDoublePropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, double val)
- void **setValueOnly** ([QtProperty](#) *property, double val)
- void [setMinimum](#) ([QtProperty](#) *property, double minVal)
- void [setMaximum](#) ([QtProperty](#) *property, double maxVal)
- void [setRange](#) ([QtProperty](#) *property, double minVal, double maxVal)
- void [setSingleStep](#) ([QtProperty](#) *property, double step)
- void [setDecimals](#) ([QtProperty](#) *property, int prec)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, double val)
- void [rangeChanged](#) ([QtProperty](#) *property, double minVal, double maxVal)
- void [singleStepChanged](#) ([QtProperty](#) *property, double step)
- void [decimalsChanged](#) ([QtProperty](#) *property, int prec)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtDoublePropertyManager](#) (QObject *parent=0)
- [~QtDoublePropertyManager](#) ()
- double [value](#) (const [QtProperty](#) *property) const
- double [initialValue](#) (const [QtProperty](#) *property) const
- double [minimum](#) (const [QtProperty](#) *property) const
- double [maximum](#) (const [QtProperty](#) *property) const
- double [singleStep](#) (const [QtProperty](#) *property) const
- int [decimals](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.511.1 Constructor & Destructor Documentation

10.511.1.1 QtDoublePropertyManager()

```
QtDoublePropertyManager::QtDoublePropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.511.1.2 ~QtDoublePropertyManager()

```
QtDoublePropertyManager::~~QtDoublePropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.511.2 Member Function Documentation

10.511.2.1 decimals()

```
int QtDoublePropertyManager::decimals (
    const QtProperty * property ) const
```

Returns the given *property*'s precision, in decimals.

See also

[setDecimals\(\)](#)

10.511.2.2 decimalsChanged

```
void QtDoublePropertyManager::decimalsChanged (
    QtProperty * property,
    int prec ) [signal]
```

This signal is emitted whenever a property created by this manager changes its precision of value, passing a pointer to the *property* and the new *prec* value

See also

[setDecimals\(\)](#)

10.511.2.3 initializeProperty()

```
void QtDoublePropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.511.2.4 maximum()

```
double QtDoublePropertyManager::maximum (
    const QtProperty * property ) const
```

Returns the given *property*'s maximum value.

See also

[minimum\(\)](#), [setRange\(\)](#)

10.511.2.5 minimum()

```
double QtDoublePropertyManager::minimum (
    const QtProperty * property ) const
```

Returns the given *property*'s minimum value.

See also

[maximum\(\)](#), [setRange\(\)](#)

10.511.2.6 rangeChanged

```
void QtDoublePropertyManager::rangeChanged (
    QtProperty * property,
    double minimum,
    double maximum ) [signal]
```

This signal is emitted whenever a property created by this manager changes its range of valid values, passing a pointer to the *property* and the new *minimum* and *maximum* values

See also

[setRange\(\)](#)

10.511.2.7 setDecimals

```
void QtDoublePropertyManager::setDecimals (
    QtProperty * property,
    int prec ) [slot]
```

Sets the precision of the given *property* to *prec*.

The valid decimal range is 0-13. The default is 2.

See also

[decimals\(\)](#)

10.511.2.8 setMaximum

```
void QtDoublePropertyManager::setMaximum (
    QtProperty * property,
    double maxVal ) [slot]
```

Sets the maximum value for the given *property* to *maxVal*.

When setting the maximum value, the minimum and current values are adjusted if necessary (ensuring that the range remains valid and that the current value is within in the range).

See also

[maximum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

10.511.2.9 setMinimum

```
void QtDoublePropertyManager::setMinimum (
    QtProperty * property,
    double minVal ) [slot]
```

Sets the minimum value for the given *property* to *minVal*.

When setting the minimum value, the maximum and current values are adjusted if necessary (ensuring that the range remains valid and that the current value is within in the range).

See also

[minimum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

10.511.2.10 setRange

```
void QtDoublePropertyManager::setRange (
    QtProperty * property,
    double minimum,
    double maximum ) [slot]
```

Sets the range of valid values.

This is a convenience function defining the range of valid values in one go; setting the *minimum* and *maximum* values for the given *property* with a single function call.

When setting a new range, the current value is adjusted if necessary (ensuring that the value remains within range).

See also

[setMinimum\(\)](#), [setMaximum\(\)](#), [rangeChanged\(\)](#)

10.511.2.11 setSingleStep

```
void QtDoublePropertyManager::setSingleStep (
    QtProperty * property,
    double step ) [slot]
```

Sets the step value for the given *property* to *step*.

The step is typically used to increment or decrement a property value while pressing an arrow key.

See also

[singleStep\(\)](#)

10.511.2.12 setValue

```
void QtDoublePropertyManager::setValue (
    QtProperty * property,
    double value ) [slot]
```

Sets the value of the given *property* to *value*.

If the specified *value* is not valid according to the given *property*'s range, the *value* is adjusted to the nearest valid value within the range.

See also

[value\(\)](#), [setRange\(\)](#), [valueChanged\(\)](#)

10.511.2.13 singleStep()

```
double QtDoublePropertyManager::singleStep (
    const QtProperty * property ) const
```

Returns the given *property*'s step value.

The step is typically used to increment or decrement a property value while pressing an arrow key.

See also

[setSingleStep\(\)](#)

10.511.2.14 singleStepChanged

```
void QtDoublePropertyManager::singleStepChanged (
    QtProperty * property,
    double step ) [signal]
```

This signal is emitted whenever a property created by this manager changes its single step property, passing a pointer to the *property* and the new *step* value

See also

[setSingleStep\(\)](#)

10.511.2.15 uninitializedProperty()

```
void QtDoublePropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.511.2.16 value()

```
double QtDoublePropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given property is not managed by this manager, this function returns 0.

See also

[setValue\(\)](#)

10.511.2.17 valueChanged

```
void QtDoublePropertyManager::valueChanged (
    QtProperty * property,
    double value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.511.2.18 valueText()

```
QString QtDoublePropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.512 QtDoublePropertyManagerPrivate Class Reference

Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Attributes

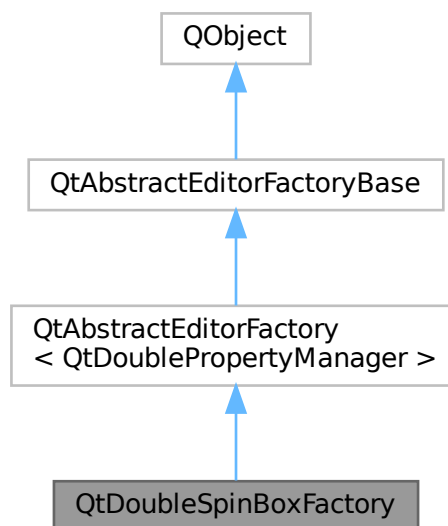
- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

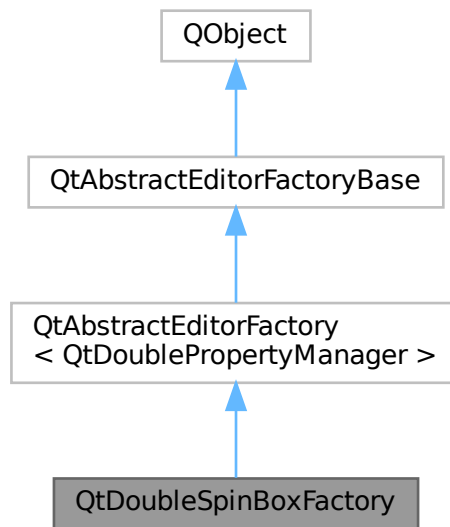
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.513 QtDoubleSpinBoxFactory Class Reference

Inheritance diagram for QtDoubleSpinBoxFactory:



Collaboration diagram for QtDoubleSpinBoxFactory:



Public Member Functions

- [QtDoubleSpinBoxFactory](#) (QObject *parent=0)
- [~QtDoubleSpinBoxFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtDoublePropertyManager](#) >

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtDoublePropertyManager *manager)
- void [removePropertyManager](#) (QtDoublePropertyManager *manager)
- QSet < [QtDoublePropertyManager](#) * > [propertyManagers](#) () const
- [QtDoublePropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtDoublePropertyManager *manager) override
- QWidget * [createEditor](#) (QtDoublePropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtDoublePropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtDoublePropertyManager](#) >

- virtual QWidget * [createEditor](#) (QtDoublePropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtDoublePropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtDoublePropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

10.513.1 Constructor & Destructor Documentation

10.513.1.1 QtDoubleSpinBoxFactory()

```
QtDoubleSpinBoxFactory::QtDoubleSpinBoxFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.513.1.2 ~QtDoubleSpinBoxFactory()

```
QtDoubleSpinBoxFactory::~~QtDoubleSpinBoxFactory ( )
```

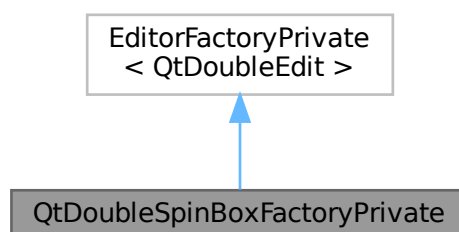
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

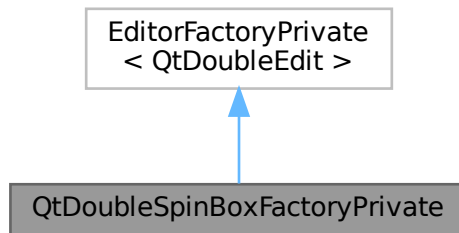
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.514 QtDoubleSpinBoxFactoryPrivate Class Reference

Inheritance diagram for QtDoubleSpinBoxFactoryPrivate:



Collaboration diagram for QtDoubleSpinBoxFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, double value)
- void **slotRangeChanged** ([QtProperty](#) *property, double min, double max)
- void **slotSingleStepChanged** ([QtProperty](#) *property, double step)
- void **slotDecimalsChanged** ([QtProperty](#) *property, int prec)
- void **slotSetValue** (double value)

Public Member Functions inherited from [EditorFactoryPrivate< QtDoubleEdit >](#)

- [QtDoubleEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QtDoubleEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QtDoubleEdit >](#)

- typedef QList< [QtDoubleEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QtDoubleEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QtDoubleEdit >](#)

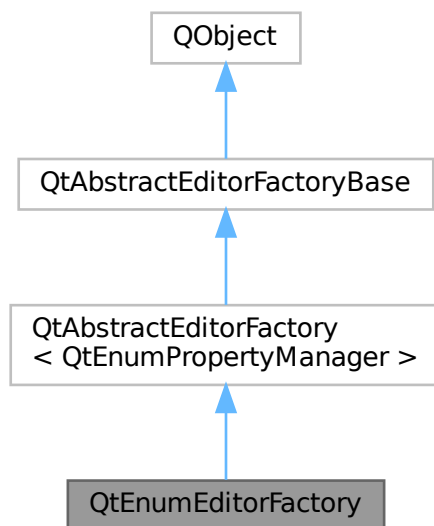
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

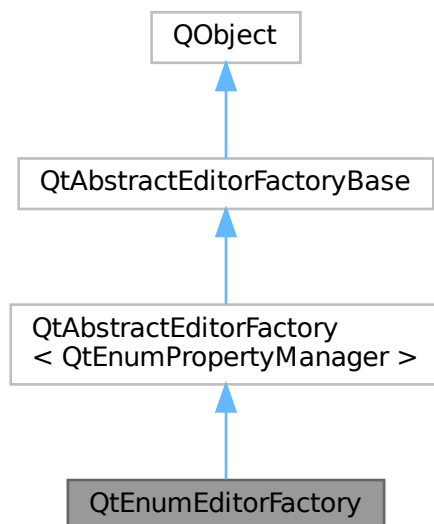
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.515 QtEnumEditorFactory Class Reference

Inheritance diagram for QtEnumEditorFactory:



Collaboration diagram for QtEnumEditorFactory:



Public Member Functions

- [QtEnumEditorFactory](#) (QObject *parent=0)
- [~QtEnumEditorFactory](#) ()

Public Member Functions inherited from**[QtAbstractEditorFactory](#) < [QtEnumPropertyManager](#) >**

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtEnumPropertyManager *manager)
- void [removePropertyManager](#) (QtEnumPropertyManager *manager)
- QSet< [QtEnumPropertyManager](#) * > [propertyManagers](#) () const
- [QtEnumPropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtEnumPropertyManager *manager) override
- QWidget * [createEditor](#) (QtEnumPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtEnumPropertyManager *manager) override

Protected Member Functions inherited from**[QtAbstractEditorFactory](#) < [QtEnumPropertyManager](#) >**

- virtual QWidget * [createEditor](#) (QtEnumPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtEnumPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtEnumPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members**Protected Slots inherited from [QtAbstractEditorFactoryBase](#)****10.515.1 Constructor & Destructor Documentation****10.515.1.1 QtEnumEditorFactory()**

```
QtEnumEditorFactory::QtEnumEditorFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.515.1.2 ~QtEnumEditorFactory()

`QtEnumEditorFactory::~QtEnumEditorFactory ( )`

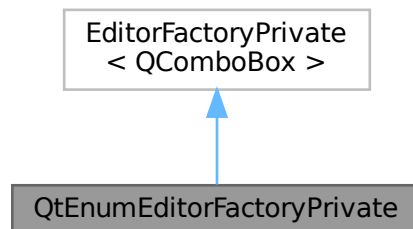
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

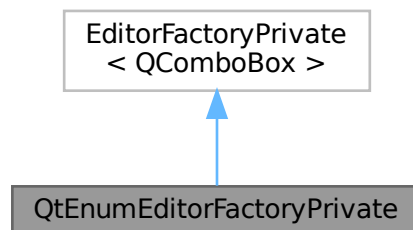
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.516 QtEnumEditorFactoryPrivate Class Reference

Inheritance diagram for QtEnumEditorFactoryPrivate:



Collaboration diagram for QtEnumEditorFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, int value)
- void **slotEnumNamesChanged** ([QtProperty](#) *property, const QStringList &)
- void **slotEnumIconsChanged** ([QtProperty](#) *property, const QMap< int, QIcon > &)
- void **slotSetValue** (int value)

Public Member Functions inherited from [EditorFactoryPrivate< QComboBox >](#)

- `QComboBox * createEditor (QtProperty *property, QWidget *parent)`
- `void initializeEditor (QtProperty *property, QComboBox *e)`
- `void slotEditorDestroyed (QObject *object)`

Additional Inherited Members**Public Types inherited from [EditorFactoryPrivate< QComboBox >](#)**

- `typedef QList< QComboBox * > EditorList`
- `typedef QMap< QtProperty *, EditorList > PropertyToEditorListMap`
- `typedef QMap< QComboBox *, QtProperty * > EditorToPropertyMap`

Public Attributes inherited from [EditorFactoryPrivate< QComboBox >](#)

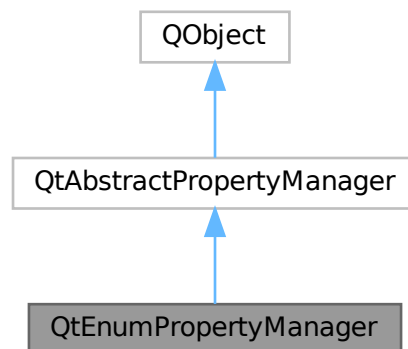
- `PropertyToEditorListMap m_createdEditors`
- `EditorToPropertyMap m_editorToProperty`

The documentation for this class was generated from the following file:

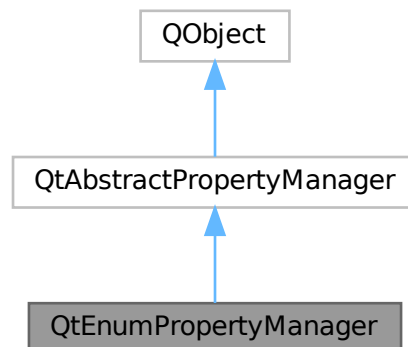
- `applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp`

10.517 QtEnumPropertyManager Class Reference

Inheritance diagram for QtEnumPropertyManager:



Collaboration diagram for QtEnumPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, int val)
- void [setEnumNames](#) ([QtProperty](#) *property, const QStringList &names)
- void [setEnumIcons](#) ([QtProperty](#) *property, const QMap< int, QIcon > &icons)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, int val)
- void [enumNamesChanged](#) ([QtProperty](#) *property, const QStringList &names)
- void [enumIconsChanged](#) ([QtProperty](#) *property, const QMap< int, QIcon > &icons)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtEnumPropertyManager](#) (QObject *parent=0)
- [~QtEnumPropertyManager](#) ()
- int [value](#) (const [QtProperty](#) *property) const
- QStringList [enumNames](#) (const [QtProperty](#) *property) const
- QMap< int, QIcon > [enumIcons](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- QIcon [valueIcon](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.517.1 Constructor & Destructor Documentation

10.517.1.1 QtEnumPropertyManager()

```
QtEnumPropertyManager::QtEnumPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.517.1.2 ~QtEnumPropertyManager()

```
QtEnumPropertyManager::~~QtEnumPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.517.2 Member Function Documentation

10.517.2.1 enumIcons()

```
QMap< int, QIcon > QtEnumPropertyManager::enumIcons (
    const QtProperty * property ) const
```

Returns the given *property*'s map of enum values to their icons.

See also

[value\(\)](#), [setEnumIcons\(\)](#)

10.517.2.2 enumIconsChanged

```
void QtEnumPropertyManager::enumIconsChanged (
    QtProperty * property,
    const QMap< int, QIcon > & icons ) [signal]
```

This signal is emitted whenever a property created by this manager changes its enum icons, passing a pointer to the *property* and the new mapping of values to *icons* as parameters.

See also

[setEnumIcons\(\)](#)

10.517.2.3 enumNames()

```
QStringList QtEnumPropertyManager::enumNames (
    const QtProperty * property ) const
```

Returns the given *property*'s list of enum names.

See also

[value\(\)](#), [setEnumNames\(\)](#)

10.517.2.4 enumNamesChanged

```
void QtEnumPropertyManager::enumNamesChanged (
    QtProperty * property,
    const QStringList & names ) [signal]
```

This signal is emitted whenever a property created by this manager changes its enum names, passing a pointer to the *property* and the new *names* as parameters.

See also

[setEnumNames\(\)](#)

10.517.2.5 initializeProperty()

```
void QtEnumPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.517.2.6 setEnumIcons

```
void QtEnumPropertyManager::setEnumIcons (
    QtProperty * property,
    const QMap< int, QIcon > & enumIcons ) [slot]
```

Sets the given *property*'s map of enum values to their icons to *enumIcons*.

Each enum value can have associated icon. This association is represented with passed *enumIcons* map.

See also

[enumNames\(\)](#), [enumNamesChanged\(\)](#)

10.517.2.7 setEnumNames

```
void QtEnumPropertyManager::setEnumNames (
    QtProperty * property,
    const QStringList & enumNames ) [slot]
```

Sets the given *property*'s list of enum names to *enumNames*. The *property*'s current value is reset to 0 indicating the first item of the list.

If the specified *enumNames* list is empty, the *property*'s current value is set to -1.

See also

[enumNames\(\)](#), [enumNamesChanged\(\)](#)

10.517.2.8 setValue

```
void QtEnumPropertyManager::setValue (
    QtProperty * property,
    int value ) [slot]
```

Sets the value of the given *property* to *value*.

The specified *value* must be less than the size of the given *property*'s [enumNames\(\)](#) list. Use -1 to leave the editor without a current selection.

See also

[value\(\)](#), [valueChanged\(\)](#)

10.517.2.9 uninitializedProperty()

```
void QtEnumPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.517.2.10 value()

```
int QtEnumPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value which is an index in the list returned by [enumNames\(\)](#)

If the given property is not managed by this manager, this function returns -1.

See also

[enumNames\(\)](#), [setValue\(\)](#)

10.517.2.11 valueChanged

```
void QtEnumPropertyManager::valueChanged (
    QtProperty * property,
    int value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.517.2.12 valueIcon()

```
QIcon QtEnumPropertyManager::valueIcon (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.517.2.13 valueText()

```
QString QtEnumPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.518 QtEnumPropertyManagerPrivate Class Reference

Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Attributes

- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

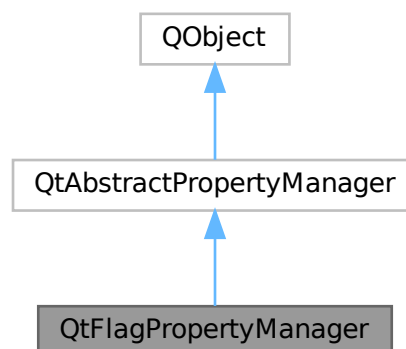
10.519 QtEnumPropertyType Class Reference

The documentation for this class was generated from the following file:

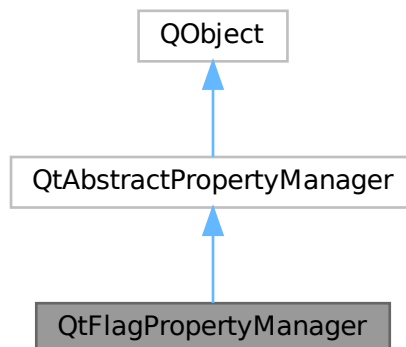
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

10.520 QtFlagPropertyManager Class Reference

Inheritance diagram for QtFlagPropertyManager:



Collaboration diagram for QtFlagPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, int val)
- void [setFlagNames](#) ([QtProperty](#) *property, const QStringList &names)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, int val)
- void [flagNamesChanged](#) ([QtProperty](#) *property, const QStringList &names)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtFlagPropertyManager](#) (QObject *parent=0)
- [~QtFlagPropertyManager](#) ()
- [QtBoolPropertyManager](#) * [subBoolPropertyManager](#) () const
- int [value](#) (const [QtProperty](#) *property) const
- QStringList [flagNames](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.520.1 Constructor & Destructor Documentation**10.520.1.1 QtFlagPropertyManager()**

```
QtFlagPropertyManager::QtFlagPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.520.1.2 ~QtFlagPropertyManager()

```
QtFlagPropertyManager::~~QtFlagPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.520.2 Member Function Documentation**10.520.2.1 flagNames()**

```
QStringList QtFlagPropertyManager::flagNames (
    const QtProperty * property ) const
```

Returns the given *property*'s list of flag names.

See also

[value\(\)](#), [setFlagNames\(\)](#)

10.520.2.2 flagNamesChanged

```
void QtFlagPropertyManager::flagNamesChanged (
    QtProperty * property,
    const QStringList & names ) [signal]
```

This signal is emitted whenever a property created by this manager changes its flag names, passing a pointer to the *property* and the new *names* as parameters.

See also

[setFlagNames\(\)](#)

10.520.2.3 initializeProperty()

```
void QtFlagPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.520.2.4 setFlagNames

```
void QtFlagPropertyManager::setFlagNames (
    QtProperty * property,
    const QStringList & flagNames ) [slot]
```

Sets the given *property*'s list of flag names to *flagNames*. The property's current value is reset to 0 indicating the first item of the list.

See also

[flagNames\(\)](#), [flagNamesChanged\(\)](#)

10.520.2.5 setValue

```
void QtFlagPropertyManager::setValue (
    QtProperty * property,
    int value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

The specified *value* must be less than the binary combination of the property's [flagNames\(\)](#) list size (i.e. less than $2^{\sup n}$, where n is the size of the list) and larger than (or equal to) 0.

See also

[value\(\)](#), [valueChanged\(\)](#)

10.520.2.6 subBoolPropertyManager()

```
QtBoolPropertyManager * QtFlagPropertyManager::subBoolPropertyManager ( ) const
```

Returns the manager that produces the nested boolean subproperties representing each flag.

In order to provide editing widgets for the subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

10.520.2.7 uninitializedProperty()

```
void QtFlagPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.520.2.8 value()

```
int QtFlagPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given property is not managed by this manager, this function returns 0.

See also

[flagNames\(\)](#), [setValue\(\)](#)

10.520.2.9 valueChanged

```
void QtFlagPropertyManager::valueChanged (
    QtProperty * property,
    int value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.520.2.10 valueText()

```
QString QtFlagPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

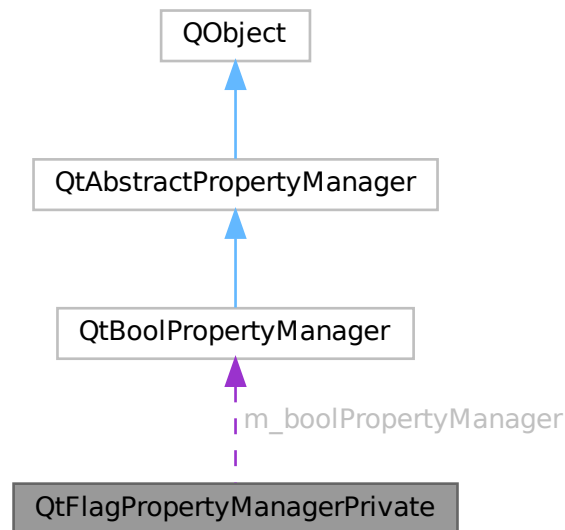
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.521 QtFlagPropertyManagerPrivate Class Reference

Collaboration diagram for QtFlagPropertyManagerPrivate:



Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- void **slotBoolChanged** ([QtProperty](#) *property, bool value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)

Public Attributes

- PropertyValueMap **m_values**
- [QtBoolPropertyManager](#) * **m_boolPropertyManager**
- QMap< const [QtProperty](#) *, QList< [QtProperty](#) * > > **m_propertyToFlags**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_flagToProperty**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

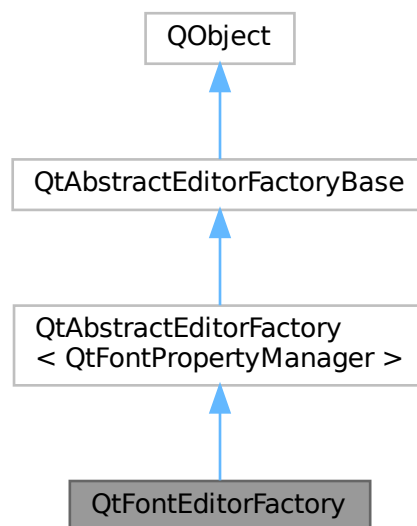
10.522 QtFlagPropertyType Class Reference

The documentation for this class was generated from the following file:

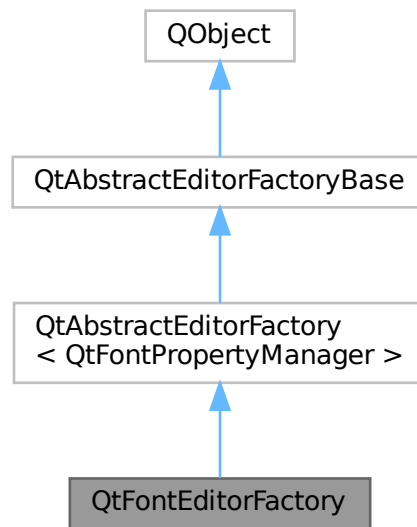
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

10.523 QFontEditorFactory Class Reference

Inheritance diagram for QFontEditorFactory:



Collaboration diagram for QFontEditorFactory:



Public Member Functions

- [QFontEditorFactory](#) (QObject *parent=0)
- [~QFontEditorFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory< QFontPropertyManager >](#)

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtFontPropertyManager *manager)
- void [removePropertyManager](#) (QtFontPropertyManager *manager)
- QSet< [QtFontPropertyManager](#) * > [propertyManagers](#) () const
- [QtFontPropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtFontPropertyManager *manager) override
- QWidget * [createEditor](#) (QtFontPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtFontPropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory< QFontPropertyManager >](#)

- virtual QWidget * [createEditor](#) (QtFontPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtFontPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtFontPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

10.523.1 Constructor & Destructor Documentation

10.523.1.1 QtFontEditorFactory()

```
QtFontEditorFactory::QtFontEditorFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.523.1.2 ~QtFontEditorFactory()

```
QtFontEditorFactory::~~QtFontEditorFactory ( )
```

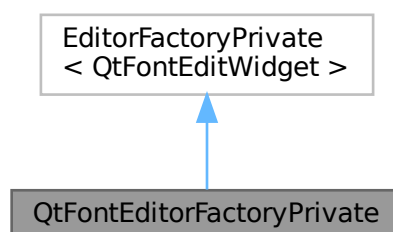
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

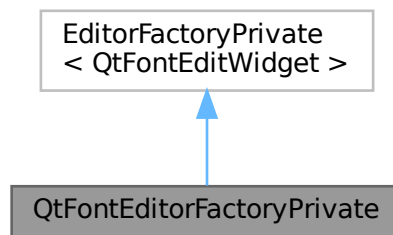
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.524 QtFontEditorFactoryPrivate Class Reference

Inheritance diagram for QtFontEditorFactoryPrivate:



Collaboration diagram for QFontEditorFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, const QFont &value)
- void **slotSetValue** (const QFont &value)

Public Member Functions inherited from [EditorFactoryPrivate< QFontEditWidget >](#)

- [QFontEditWidget](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QFontEditWidget](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QFontEditWidget >](#)

- typedef QList< [QFontEditWidget](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QFontEditWidget](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QFontEditWidget >](#)

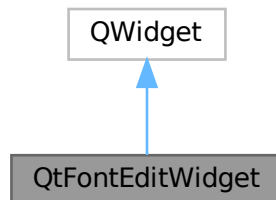
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

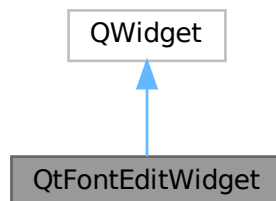
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.525 QFontEditWidget Class Reference

Inheritance diagram for QFontEditWidget:



Collaboration diagram for QFontEditWidget:



Public Slots

- void **setValue** (const QFont &value)

Signals

- void **valueChanged** (const QFont &value)

Public Member Functions

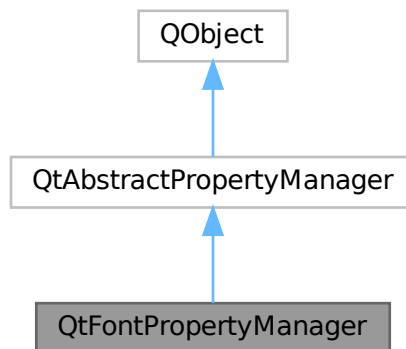
- **QFontEditWidget** (QWidget *parent)
- bool **eventFilter** (QObject *obj, QEvent *ev) override

The documentation for this class was generated from the following file:

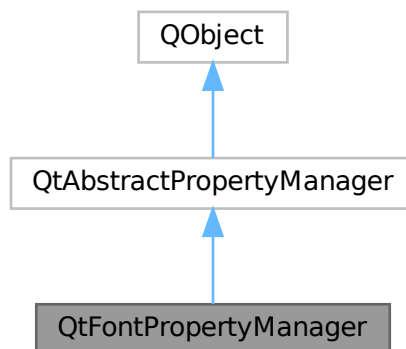
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.526 QFontPropertyManager Class Reference

Inheritance diagram for QFontPropertyManager:



Collaboration diagram for QFontPropertyManager:



Public Slots

- void `setValue` (`QtProperty` *property, const QFont &val)

Signals

- void `valueChanged` (`QtProperty` *property, const QFont &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QFontPropertyManager](#) ([QObject](#) *parent=0)
- [~QFontPropertyManager](#) ()
- [QtIntPropertyManager](#) * [subIntPropertyManager](#) () const
- [QtEnumPropertyManager](#) * [subEnumPropertyManager](#) () const
- [QtBoolPropertyManager](#) * [subBoolPropertyManager](#) () const
- [QFont](#) [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) ([QObject](#) *parent=0)
- [~QtAbstractPropertyManager](#) ()
- [QSet](#)< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const [QString](#) &name=[QString](#)())

Protected Member Functions

- [QString](#) [valueText](#) (const [QtProperty](#) *property) const override
- [QIcon](#) [valueIcon](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.526.1 Constructor & Destructor Documentation

10.526.1.1 [QFontPropertyManager](#)()

```
QFontPropertyManager::QFontPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.526.1.2 [~QFontPropertyManager](#)()

```
QFontPropertyManager::~~QFontPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.526.2 Member Function Documentation

10.526.2.1 initializeProperty()

```
void QFontPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.526.2.2 setValue

```
void QFontPropertyManager::setValue (
    QtProperty * property,
    const QFont & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

See also

[value\(\)](#), [valueChanged\(\)](#)

10.526.2.3 subBoolPropertyManager()

```
QtBoolPropertyManager * QFontPropertyManager::subBoolPropertyManager ( ) const
```

Returns the manager that creates the *bold*, *italic*, *underline*, *strikeOut* and *kerling* subproperties.

In order to provide editing widgets for the mentioned properties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

10.526.2.4 subEnumPropertyManager()

```
QtEnumPropertyManager * QFontPropertyManager::subEnumPropertyManager ( ) const
```

Returns the manager that create the *family* subproperty.

In order to provide editing widgets for the *family* property in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

10.526.2.5 subIntPropertyManager()

```
QtIntPropertyManager * QFontPropertyManager::subIntPropertyManager ( ) const
```

Returns the manager that creates the *pointSize* subproperty.

In order to provide editing widgets for the *pointSize* property in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

10.526.2.6 uninitializedProperty()

```
void QFontPropertyManager::uninitializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.526.2.7 value()

```
QFont QFontPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given property is not managed by this manager, this function returns a font object that uses the application's default font.

See also

[setValue\(\)](#)

10.526.2.8 valueChanged

```
void QFontPropertyManager::valueChanged (
    QtProperty * property,
    const QFont & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.526.2.9 valueIcon()

```
QIcon QtFontPropertyManager::valueIcon (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.526.2.10 valueText()

```
QString QtFontPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

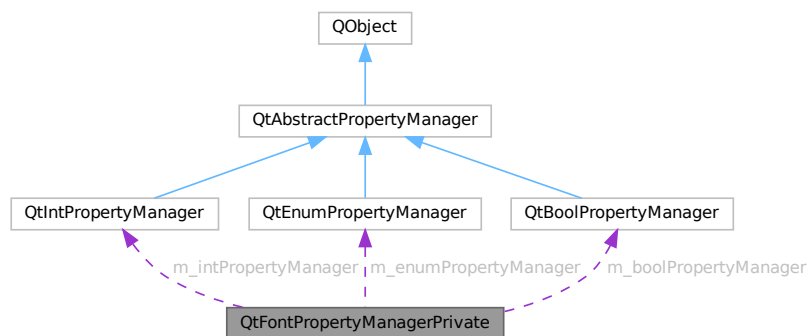
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.527 QtFontPropertyManagerPrivate Class Reference

Collaboration diagram for QtFontPropertyManagerPrivate:



Public Types

- typedef QMap< const [QtProperty](#) *, QFont > **PropertyValueMap**

Public Member Functions

- void **slotIntChanged** (QtProperty *property, int value)
- void **slotEnumChanged** (QtProperty *property, int value)
- void **slotBoolChanged** (QtProperty *property, bool value)
- void **slotPropertyDestroyed** (QtProperty *property)
- void **slotFontDatabaseChanged** ()
- void **slotFontDatabaseDelayedChange** ()

Public Attributes

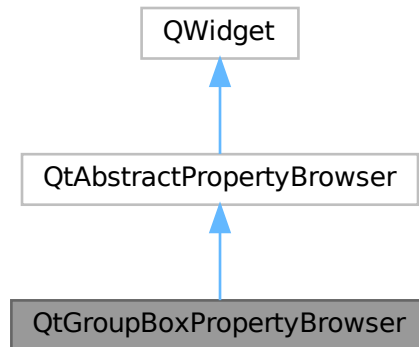
- QStringList **m_familyNames**
- PropertyValueMap **m_values**
- QtIntPropertyManager * **m_intPropertyManager**
- QtEnumPropertyManager * **m_enumPropertyManager**
- QtBoolPropertyManager * **m_boolPropertyManager**
- QMap< const QtProperty *, QtProperty * > **m_propertyToFamily**
- QMap< const QtProperty *, QtProperty * > **m_propertyToPointSize**
- QMap< const QtProperty *, QtProperty * > **m_propertyToBold**
- QMap< const QtProperty *, QtProperty * > **m_propertyToItalic**
- QMap< const QtProperty *, QtProperty * > **m_propertyToUnderline**
- QMap< const QtProperty *, QtProperty * > **m_propertyToStrikeOut**
- QMap< const QtProperty *, QtProperty * > **m_propertyToKerning**
- QMap< const QtProperty *, QtProperty * > **m_familyToProperty**
- QMap< const QtProperty *, QtProperty * > **m_pointSizeToProperty**
- QMap< const QtProperty *, QtProperty * > **m_boldToProperty**
- QMap< const QtProperty *, QtProperty * > **m_italicToProperty**
- QMap< const QtProperty *, QtProperty * > **m_underlineToProperty**
- QMap< const QtProperty *, QtProperty * > **m_strikeOutToProperty**
- QMap< const QtProperty *, QtProperty * > **m_kerningToProperty**
- bool **m_settingValue**
- QTimer * **m_fontDatabaseChangeTimer**

The documentation for this class was generated from the following file:

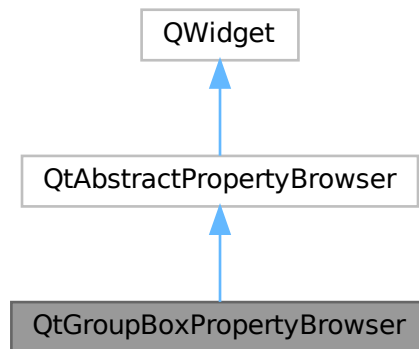
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.528 QtGroupBoxPropertyBrowser Class Reference

Inheritance diagram for QtGroupBoxPropertyBrowser:



Collaboration diagram for QtGroupBoxPropertyBrowser:



Public Member Functions

- [QtGroupBoxPropertyBrowser](#) (QWidget *parent=0)
- [~QtGroupBoxPropertyBrowser](#) ()

Public Member Functions inherited from [QtAbstractPropertyBrowser](#)

- [QtAbstractPropertyBrowser](#) (QWidget *parent=0)
- [~QtAbstractPropertyBrowser](#) ()

- `QList< QtProperty * > properties () const`
- `QList< QtBrowserItem * > items (QtProperty *property) const`
- `QtBrowserItem * topLevelItem (QtProperty *property) const`
- `QList< QtBrowserItem * > topLevelItems () const`
- `void clear ()`
- `template<class PropertyManager >`
`void setFactoryForManager (PropertyManager *manager, QtAbstractEditorFactory< PropertyManager > *factory)`
- `void unsetFactoryForManager (QtAbstractPropertyManager *manager)`
- `QtBrowserItem * currentItem () const`
- `void setCurrentItem (QtBrowserItem *)`

Protected Member Functions

- `void itemInserted (QtBrowserItem *item, QtBrowserItem *afterItem) override`
- `void itemRemoved (QtBrowserItem *item) override`
- `void itemChanged (QtBrowserItem *item) override`

Protected Member Functions inherited from QtAbstractPropertyBrowser

- `virtual QWidget * createEditor (QtProperty *property, QWidget *parent)`

Additional Inherited Members

Public Slots inherited from QtAbstractPropertyBrowser

- `QtBrowserItem * addProperty (QtProperty *property)`
- `QtBrowserItem * insertProperty (QtProperty *property, QtProperty *afterProperty)`
- `void removeProperty (QtProperty *property)`

Signals inherited from QtAbstractPropertyBrowser

- `void currentItemChanged (QtBrowserItem *)`

10.528.1 Constructor & Destructor Documentation

10.528.1.1 QtGroupBoxPropertyBrowser()

```
QtGroupBoxPropertyBrowser::QtGroupBoxPropertyBrowser (
    QWidget * parent = 0 )
```

Creates a property browser with the given *parent*.

10.528.1.2 ~QtGroupBoxPropertyBrowser()

```
QtGroupBoxPropertyBrowser::~QtGroupBoxPropertyBrowser ( )
```

Destroys this property browser.

Note that the properties that were inserted into this browser are *not* destroyed since they may still be used in other browsers. The properties are owned by the manager that created them.

See also

[QtProperty](#), [QtAbstractPropertyManager](#)

10.528.2 Member Function Documentation

10.528.2.1 itemChanged()

```
void QtGroupBoxPropertyBrowser::itemChanged (
    QtBrowserItem * item ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

10.528.2.2 itemInserted()

```
void QtGroupBoxPropertyBrowser::itemInserted (
    QtBrowserItem * item,
    QtBrowserItem * afterItem ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

10.528.2.3 itemRemoved()

```
void QtGroupBoxPropertyBrowser::itemRemoved (
    QtBrowserItem * item ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtgroupboxpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtgroupboxpropertybrowser.cpp

10.529 QtGroupBoxPropertyBrowserPrivate Class Reference

Classes

- struct [WidgetItem](#)

Public Member Functions

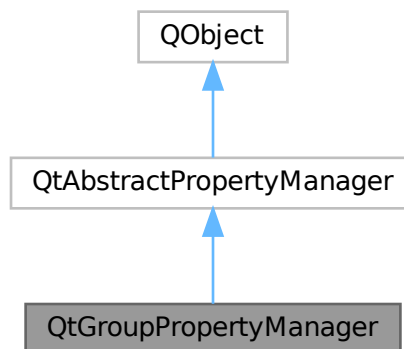
- void **init** (QWidget *parent)
- void **propertyInserted** ([QtBrowserItem](#) *index, [QtBrowserItem](#) *afterIndex)
- void **propertyRemoved** ([QtBrowserItem](#) *index)
- void **propertyChanged** ([QtBrowserItem](#) *index)
- QWidget * **createEditor** ([QtProperty](#) *property, QWidget *parent) const
- void **slotEditorDestroyed** ()
- void **slotUpdate** ()

The documentation for this class was generated from the following file:

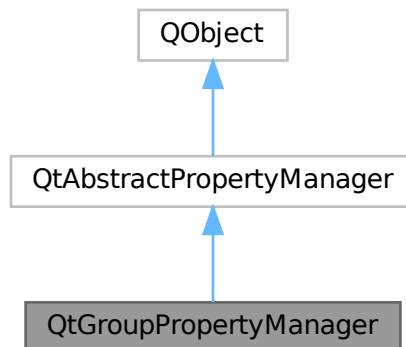
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtgroupboxpropertybrowser.cpp

10.530 QtGroupPropertyManager Class Reference

Inheritance diagram for QtGroupPropertyManager:



Collaboration diagram for QtGroupPropertyManager:



Public Member Functions

- [QtGroupPropertyManager](#) (QObject *parent=0)
- [~QtGroupPropertyManager](#) ()

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- [QSet< QtProperty * > properties](#) () const
- void [clear](#) () const
- [QtProperty * addProperty](#) (const QString &name=QString())

Protected Member Functions

- bool [hasValue](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual QString [valueText](#) (const [QtProperty](#) *property) const
- virtual [QtProperty * createProperty](#) ()

Additional Inherited Members

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

10.530.1 Constructor & Destructor Documentation

10.530.1.1 QtGroupPropertyManager()

```
QtGroupPropertyManager::QtGroupPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.530.1.2 ~QtGroupPropertyManager()

```
QtGroupPropertyManager::~QtGroupPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.530.2 Member Function Documentation

10.530.2.1 hasValue()

```
bool QtGroupPropertyManager::hasValue (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.530.2.2 initializeProperty()

```
void QtGroupPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.530.2.3 uninitializedProperty()

```
void QtGroupPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

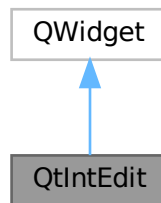
10.531 QtGroupPropertyType Class Reference

The documentation for this class was generated from the following file:

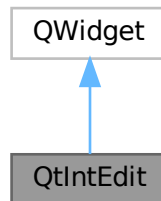
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

10.532 QtIntEdit Class Reference

Inheritance diagram for QtIntEdit:



Collaboration diagram for QtIntEdit:



Signals

- void **valueChanged** (int)

Public Member Functions

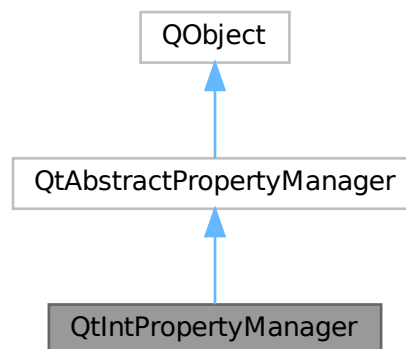
- **QtIntEdit** (QWidget *parent=Q_NULLPTR)
- int **value** ()
- void **setValue** (int)
- void **setInitialValue** (int)
- void **setSingleStep** (int)
- void **setRange** (int, int)
- void **setKeyboardTracking** (bool)

The documentation for this class was generated from the following files:

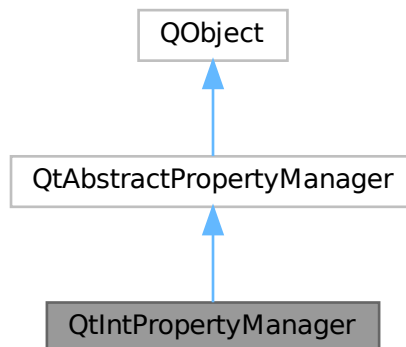
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

10.533 QtIntPropertyManager Class Reference

Inheritance diagram for QtIntPropertyManager:



Collaboration diagram for QtIntPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, int val)
- void **setValueOnly** ([QtProperty](#) *property, int val)
- void [setMinimum](#) ([QtProperty](#) *property, int minVal)
- void [setMaximum](#) ([QtProperty](#) *property, int maxVal)
- void [setRange](#) ([QtProperty](#) *property, int minVal, int maxVal)
- void [setSingleStep](#) ([QtProperty](#) *property, int step)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, int val)
- void [rangeChanged](#) ([QtProperty](#) *property, int minVal, int maxVal)
- void [singleStepChanged](#) ([QtProperty](#) *property, int step)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtIntPropertyManager](#) (QObject *parent=0)
- [~QtIntPropertyManager](#) ()
- int [value](#) (const [QtProperty](#) *property) const
- int **initialValue** (const [QtProperty](#) *property) const
- int [minimum](#) (const [QtProperty](#) *property) const
- int [maximum](#) (const [QtProperty](#) *property) const
- int [singleStep](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.533.1 Constructor & Destructor Documentation

10.533.1.1 QtIntPropertyManager()

```
QtIntPropertyManager::QtIntPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.533.1.2 ~QtIntPropertyManager()

```
QtIntPropertyManager::~~QtIntPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.533.2 Member Function Documentation

10.533.2.1 initializeProperty()

```
void QtIntPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.533.2.2 maximum()

```
int QtIntPropertyManager::maximum (
    const QtProperty * property ) const
```

Returns the given *property*'s maximum value.

See also

[setMaximum\(\)](#), [minimum\(\)](#), [setRange\(\)](#)

10.533.2.3 minimum()

```
int QtIntPropertyManager::minimum (
    const QtProperty * property ) const
```

Returns the given *property*'s minimum value.

See also

[setMinimum\(\)](#), [maximum\(\)](#), [setRange\(\)](#)

10.533.2.4 rangeChanged

```
void QtIntPropertyManager::rangeChanged (
    QtProperty * property,
    int minimum,
    int maximum ) [signal]
```

This signal is emitted whenever a property created by this manager changes its range of valid values, passing a pointer to the *property* and the new *minimum* and *maximum* values.

See also

[setRange\(\)](#)

10.533.2.5 setMaximum

```
void QtIntPropertyManager::setMaximum (
    QtProperty * property,
    int maxVal ) [slot]
```

Sets the maximum value for the given *property* to *maxVal*.

When setting maximum value, the minimum and current values are adjusted if necessary (ensuring that the range remains valid and that the current value is within the range).

See also

[maximum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

10.533.2.6 setMinimum

```
void QtIntPropertyManager::setMinimum (
    QtProperty * property,
    int minVal ) [slot]
```

Sets the minimum value for the given *property* to *minVal*.

When setting the minimum value, the maximum and current values are adjusted if necessary (ensuring that the range remains valid and that the current value is within the range).

See also

[minimum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

10.533.2.7 setRange

```
void QtIntPropertyManager::setRange (
    QtProperty * property,
    int minimum,
    int maximum ) [slot]
```

Sets the range of valid values.

This is a convenience function defining the range of valid values in one go; setting the *minimum* and *maximum* values for the given *property* with a single function call.

When setting a new range, the current value is adjusted if necessary (ensuring that the value remains within range).

See also

[setMinimum\(\)](#), [setMaximum\(\)](#), [rangeChanged\(\)](#)

10.533.2.8 setSingleStep

```
void QtIntPropertyManager::setSingleStep (
    QtProperty * property,
    int step ) [slot]
```

Sets the step value for the given *property* to *step*.

The step is typically used to increment or decrement a property value while pressing an arrow key.

See also

[singleStep\(\)](#)

10.533.2.9 setValue

```
void QtIntPropertyManager::setValue (
    QtProperty * property,
    int value ) [slot]
```

Sets the value of the given *property* to *value*.

If the specified *value* is not valid according to the given *property*'s range, the *value* is adjusted to the nearest valid value within the range.

See also

[value\(\)](#), [setRange\(\)](#), [valueChanged\(\)](#)

10.533.2.10 singleStep()

```
int QtIntPropertyManager::singleStep (
    const QtProperty * property ) const
```

Returns the given *property*'s step value.

The step is typically used to increment or decrement a property value while pressing an arrow key.

See also

[setSingleStep\(\)](#)

10.533.2.11 singleStepChanged

```
void QtIntPropertyManager::singleStepChanged (
    QtProperty * property,
    int step ) [signal]
```

This signal is emitted whenever a property created by this manager changes its single step property, passing a pointer to the *property* and the new *step* value

See also

[setSingleStep\(\)](#)

10.533.2.12 uninitializedProperty()

```
void QtIntPropertyManager::uninitializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.533.2.13 value()

```
int QtIntPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given property is not managed by this manager, this function returns 0.

See also

[setValue\(\)](#)

10.533.2.14 valueChanged

```
void QtIntPropertyManager::valueChanged (
    QtProperty * property,
    int value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.533.2.15 valueText()

```
QString QtIntPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.534 QtIntPropertyManagerPrivate Class Reference**Classes**

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Attributes

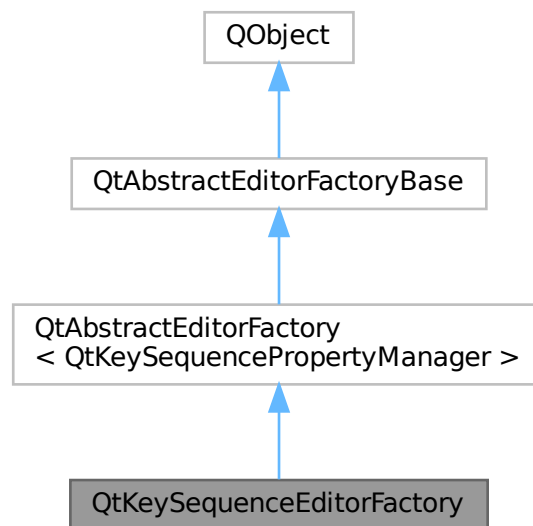
- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

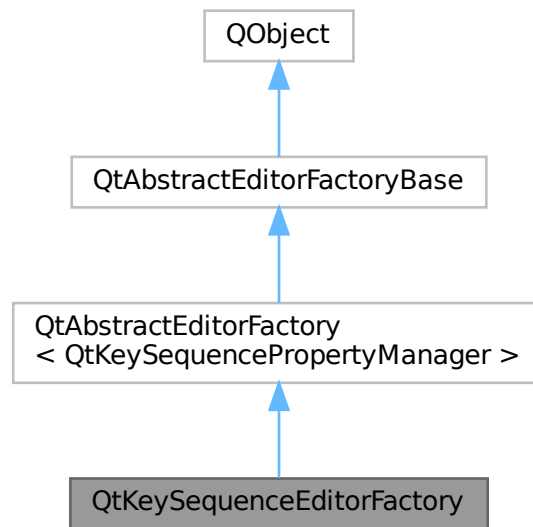
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.535 QtKeySequenceEditorFactory Class Reference

Inheritance diagram for QtKeySequenceEditorFactory:



Collaboration diagram for QtKeySequenceEditorFactory:



Public Member Functions

- [QtKeySequenceEditorFactory](#) (QObject *parent=0)
- [~QtKeySequenceEditorFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory < QtKeySequencePropertyManager >](#)

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtKeySequencePropertyManager *manager)
- void [removePropertyManager](#) (QtKeySequencePropertyManager *manager)
- QSet< [QtKeySequencePropertyManager](#) * > [propertyManagers](#) () const
- [QtKeySequencePropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtKeySequencePropertyManager *manager) override
- QWidget * [createEditor](#) (QtKeySequencePropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtKeySequencePropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory < QtKeySequencePropertyManager >](#)

- virtual QWidget * [createEditor](#) (QtKeySequencePropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtKeySequencePropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtKeySequencePropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

10.535.1 Constructor & Destructor Documentation

10.535.1.1 QtKeySequenceEditorFactory()

```
QtKeySequenceEditorFactory::QtKeySequenceEditorFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.535.1.2 ~QtKeySequenceEditorFactory()

```
QtKeySequenceEditorFactory::~~QtKeySequenceEditorFactory ( )
```

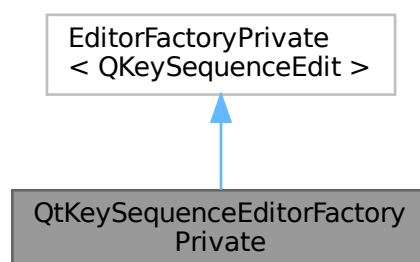
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

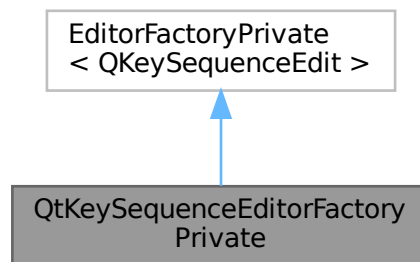
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.536 QtKeySequenceEditorFactoryPrivate Class Reference

Inheritance diagram for QtKeySequenceEditorFactoryPrivate:



Collaboration diagram for QtKeySequenceEditorFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, const QKeySequence &value)
- void **slotSetValue** (const QKeySequence &value)

Public Member Functions inherited from [EditorFactoryPrivate < QKeySequenceEdit >](#)

- QKeySequenceEdit * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, QKeySequenceEdit *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate < QKeySequenceEdit >](#)

- typedef QList< QKeySequenceEdit * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< QKeySequenceEdit *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate < QKeySequenceEdit >](#)

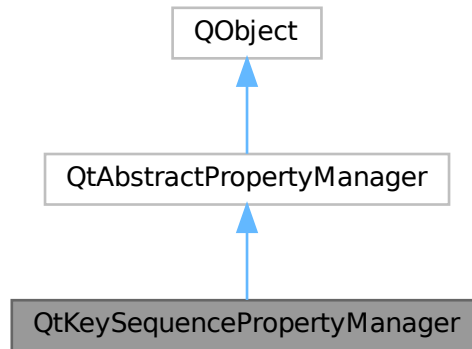
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

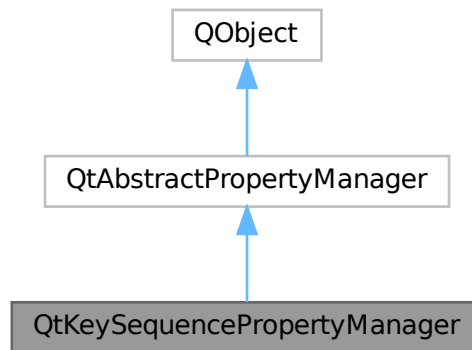
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.537 QtKeySequencePropertyManager Class Reference

Inheritance diagram for QtKeySequencePropertyManager:



Collaboration diagram for QtKeySequencePropertyManager:



Public Slots

- void `setValue` (`QtProperty` *property, const QKeySequence &val)

Signals

- void `valueChanged` (`QtProperty` *property, const QKeySequence &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtKeySequencePropertyManager](#) (QObject *parent=0)
- [~QtKeySequencePropertyManager](#) ()
- QKeySequence [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.537.1 Constructor & Destructor Documentation

10.537.1.1 [QtKeySequencePropertyManager](#)()

```
QtKeySequencePropertyManager::QtKeySequencePropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.537.1.2 [~QtKeySequencePropertyManager](#)()

```
QtKeySequencePropertyManager::~~QtKeySequencePropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.537.2 Member Function Documentation

10.537.2.1 initializeProperty()

```
void QtKeySequencePropertyManager::initializeProperty (  
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.537.2.2 setValue

```
void QtKeySequencePropertyManager::setValue (  
    QtProperty * property,  
    const QKeySequence & value ) [slot]
```

Sets the value of the given *property* to *value*.

See also

[value\(\)](#), [valueChanged\(\)](#)

10.537.2.3 uninitializedProperty()

```
void QtKeySequencePropertyManager::uninitializedProperty (  
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.537.2.4 value()

```
QKeySequence QtKeySequencePropertyManager::value (  
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an empty QKeySequence object.

See also

[setValue\(\)](#)

10.537.2.5 valueChanged

```
void QtKeySequencePropertyManager::valueChanged (
    QtProperty * property,
    const QKeySequence & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

10.537.2.6 valueText()

```
QString QtKeySequencePropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.538 QtKeySequencePropertyManagerPrivate Class Reference

Public Types

- typedef QMap< const [QtProperty](#) *, QKeySequence > **PropertyValueMap**

Public Attributes

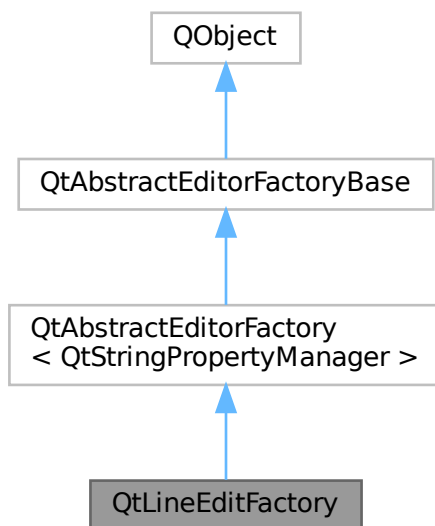
- QString **m_format**
- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

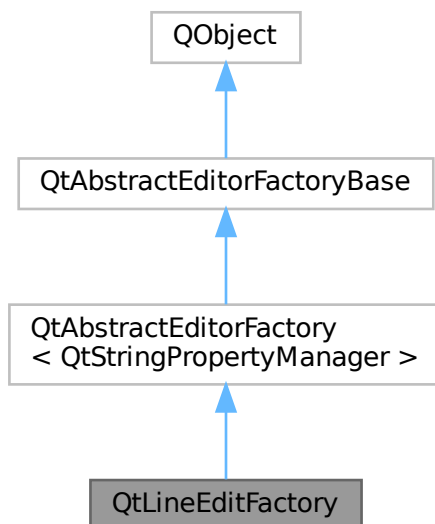
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.539 QtLineEditFactory Class Reference

Inheritance diagram for QtLineEditFactory:



Collaboration diagram for QtLineEditFactory:



Public Member Functions

- [QtLineEditFactory](#) (QObject *parent=0)
- [~QtLineEditFactory](#) ()

Public Member Functions inherited from**[QtAbstractEditorFactory](#) < [QtStringPropertyManager](#) >**

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtStringPropertyManager *manager)
- void [removePropertyManager](#) (QtStringPropertyManager *manager)
- QSet< [QtStringPropertyManager](#) * > [propertyManagers](#) () const
- [QtStringPropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtStringPropertyManager *manager) override
- QWidget * [createEditor](#) (QtStringPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtStringPropertyManager *manager) override

Protected Member Functions inherited from**[QtAbstractEditorFactory](#) < [QtStringPropertyManager](#) >**

- virtual QWidget * [createEditor](#) (QtStringPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtStringPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtStringPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members**Protected Slots inherited from [QtAbstractEditorFactoryBase](#)****10.539.1 Constructor & Destructor Documentation****10.539.1.1 QtLineEditFactory()**

```
QtLineEditFactory::QtLineEditFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.539.1.2 ~QtLineEditFactory()

```
QtLineEditFactory::~QtLineEditFactory ( )
```

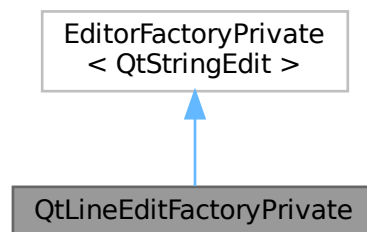
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

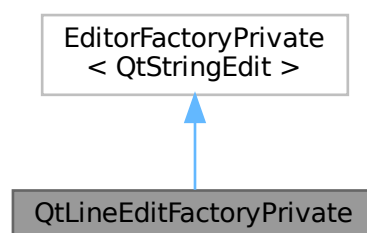
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.540 QtLineEditFactoryPrivate Class Reference

Inheritance diagram for QtLineEditFactoryPrivate:



Collaboration diagram for QtLineEditFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, const QString &value)
- void **slotRegExpChanged** ([QtProperty](#) *property, const QRegularExpression ®Exp)
- void **slotSetValue** (const QString &value)

Public Member Functions inherited from [EditorFactoryPrivate< QStringEdit >](#)

- [QStringEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QStringEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members**Public Types inherited from [EditorFactoryPrivate< QStringEdit >](#)**

- typedef QList< [QStringEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QStringEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QStringEdit >](#)

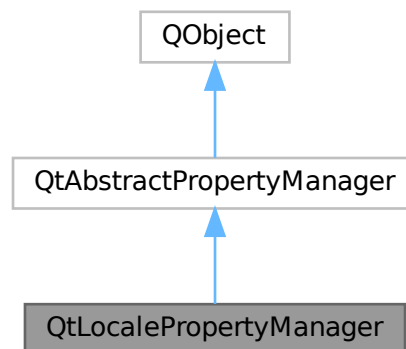
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

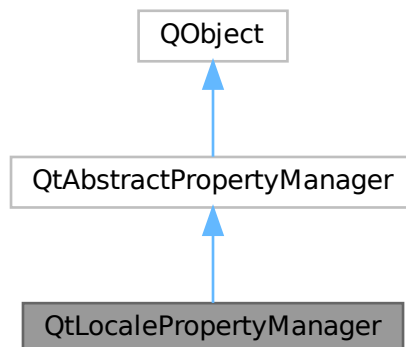
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.541 QtLocalePropertyManager Class Reference

Inheritance diagram for QtLocalePropertyManager:



Collaboration diagram for QtLocalePropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QLocale &val)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QLocale &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtLocalePropertyManager](#) (QObject *parent=0)
- [~QtLocalePropertyManager](#) ()
- [QtEnumPropertyManager](#) * [subEnumPropertyManager](#) () const
- QLocale [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.541.1 Constructor & Destructor Documentation**10.541.1.1 QtLocalePropertyManager()**

```
QtLocalePropertyManager::QtLocalePropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.541.1.2 ~QtLocalePropertyManager()

```
QtLocalePropertyManager::~~QtLocalePropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.541.2 Member Function Documentation**10.541.2.1 initializeProperty()**

```
void QtLocalePropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.541.2.2 setValue

```
void QtLocalePropertyManager::setValue (
    QtProperty * property,
    const QLocale & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

See also

[value\(\)](#), [valueChanged\(\)](#)

10.541.2.3 subEnumPropertyManager()

```
QtEnumPropertyManager * QtLocalePropertyManager::subEnumPropertyManager ( ) const
```

Returns the manager that creates the nested *language* and *country* subproperties.

In order to provide editing widgets for the mentioned subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

10.541.2.4 uninitializedProperty()

```
void QtLocalePropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.541.2.5 value()

```
QLocale QtLocalePropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given property is not managed by this manager, this function returns the default locale.

See also

[setValue\(\)](#)

10.541.2.6 valueChanged

```
void QtLocalePropertyManager::valueChanged (
    QtProperty * property,
    const QLocale & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.541.2.7 valueText()

```
QString QtLocalePropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

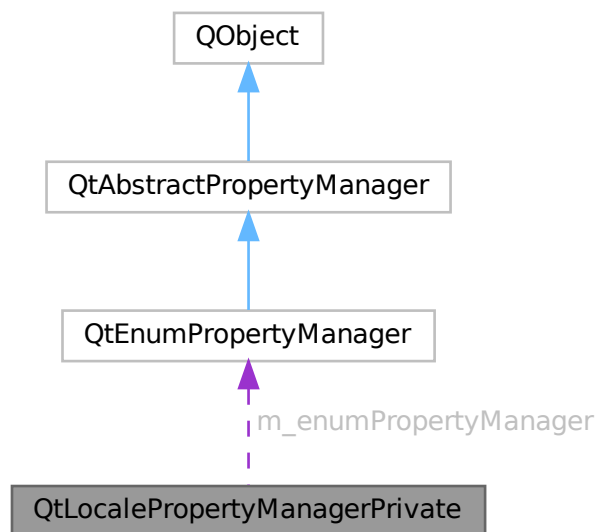
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.542 QtLocalePropertyManagerPrivate Class Reference

Collaboration diagram for QtLocalePropertyManagerPrivate:



Public Types

- typedef QMap< const [QtProperty](#) *, QLocale > **PropertyValueMap**

Public Member Functions

- void **slotEnumChanged** ([QtProperty](#) *property, int value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)

Public Attributes

- PropertyValueMap **m_values**
- [QtEnumPropertyManager](#) * **m_enumPropertyManager**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToLanguage**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToCountry**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_languageToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_countryToProperty**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.543 QtMetaEnumProvider Class Reference**Public Member Functions**

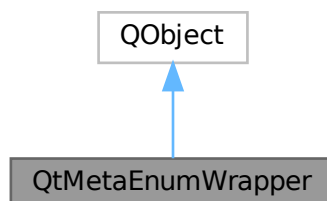
- QStringList **policyEnumNames** () const
- QStringList **languageEnumNames** () const
- QStringList **countryEnumNames** (QLocale::Language language) const
- QSizePolicy::Policy **indexToSizePolicy** (int index) const
- int **sizePolicyToIndex** (QSizePolicy::Policy policy) const
- void **indexToLocale** (int languageIndex, int countryIndex, QLocale::Language *language, QLocale::Country *country) const
- void **localeToIndex** (QLocale::Language language, QLocale::Country country, int *languageIndex, int *countryIndex) const

The documentation for this class was generated from the following file:

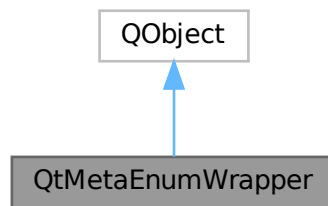
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.544 QtMetaEnumWrapper Class Reference

Inheritance diagram for QtMetaEnumWrapper:



Collaboration diagram for QtMetaEnumWrapper:



Public Member Functions

- QSizePolicy::Policy **policy** () const

Properties

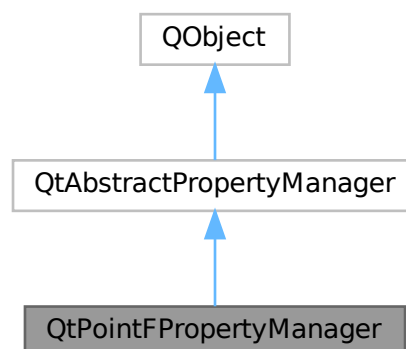
- QSizePolicy::Policy **policy**

The documentation for this class was generated from the following file:

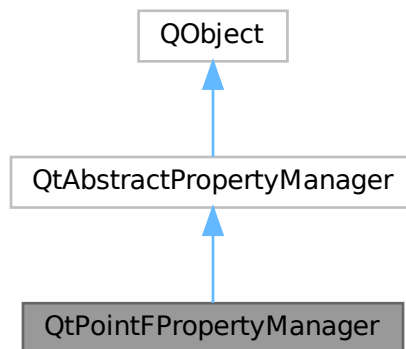
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.545 QtPointFPropertyManager Class Reference

Inheritance diagram for QtPointFPropertyManager:



Collaboration diagram for QtPointFPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QPointF &val)
- void [setDecimals](#) ([QtProperty](#) *property, int prec)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QPointF &val)
- void [decimalsChanged](#) ([QtProperty](#) *property, int prec)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtPointFPropertyManager](#) (QObject *parent=0)
- [~QtPointFPropertyManager](#) ()
- [QtDoublePropertyManager](#) * [subDoublePropertyManager](#) () const
- QPointF [value](#) (const [QtProperty](#) *property) const
- int [decimals](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.545.1 Constructor & Destructor Documentation

10.545.1.1 QtPointFPropertyManager()

```
QtPointFPropertyManager::QtPointFPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.545.1.2 ~QtPointFPropertyManager()

```
QtPointFPropertyManager::~~QtPointFPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.545.2 Member Function Documentation

10.545.2.1 decimals()

```
int QtPointFPropertyManager::decimals (
    const QtProperty * property ) const
```

Returns the given *property*'s precision, in decimals.

See also

[setDecimals\(\)](#)

10.545.2.2 decimalsChanged

```
void QtPointFPropertyManager::decimalsChanged (
    QtProperty * property,
    int prec ) [signal]
```

This signal is emitted whenever a property created by this manager changes its precision of value, passing a pointer to the *property* and the new *prec* value

See also

[setDecimals\(\)](#)

10.545.2.3 initializeProperty()

```
void QtPointFPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.545.2.4 setDecimals

```
void QtPointFPropertyManager::setDecimals (
    QtProperty * property,
    int prec ) [slot]
```

Sets the precision of the given *property* to *prec*.

The valid decimal range is 0-13. The default is 2.

See also

[decimals\(\)](#)

10.545.2.5 setValue

```
void QtPointFPropertyManager::setValue (
    QtProperty * property,
    const QPointF & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

See also

[value\(\)](#), [valueChanged\(\)](#)

10.545.2.6 subDoublePropertyManager()

```
QtDoublePropertyManager * QtPointFPropertyManager::subDoublePropertyManager ( ) const
```

Returns the manager that creates the nested *x* and *y* subproperties.

In order to provide editing widgets for the subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

10.545.2.7 uninitializedProperty()

```
void QtPointFPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.545.2.8 value()

```
QPointF QtPointFPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns a point with coordinates (0, 0).

See also

[setValue\(\)](#)

10.545.2.9 valueChanged

```
void QtPointFPropertyManager::valueChanged (
    QtProperty * property,
    const QPointF & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.545.2.10 valueText()

```
QString QtPointFPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

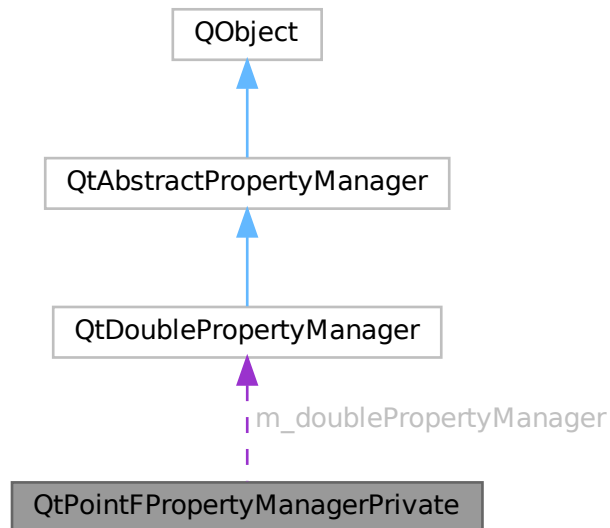
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.546 QtPointFPropertyManagerPrivate Class Reference

Collaboration diagram for QtPointFPropertyManagerPrivate:



Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- void **slotDoubleChanged** ([QtProperty](#) *property, double value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)

Public Attributes

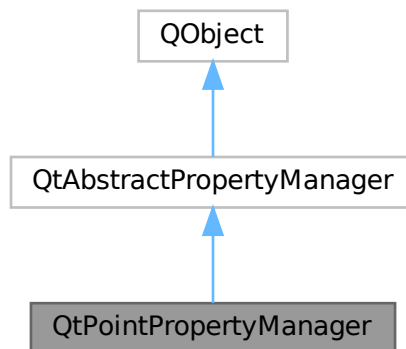
- PropertyValueMap **m_values**
- [QtDoublePropertyManager](#) * **m_doublePropertyManager**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToX**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToY**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_xToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_yToProperty**

The documentation for this class was generated from the following file:

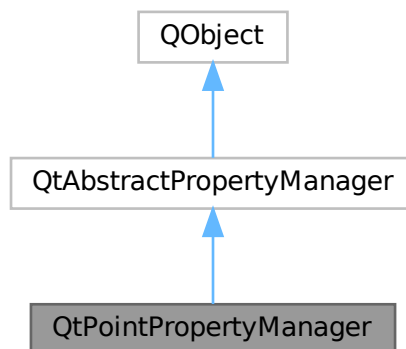
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.547 QtPointPropertyManager Class Reference

Inheritance diagram for QtPointPropertyManager:



Collaboration diagram for QtPointPropertyManager:



Public Slots

- void `setValue` (`QtProperty` *property, const QPoint &val)

Signals

- void `valueChanged` (`QtProperty` *property, const QPoint &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtPointPropertyManager](#) (QObject *parent=0)
- [~QtPointPropertyManager](#) ()
- [QtIntPropertyManager](#) * [subIntPropertyManager](#) () const
- QPoint [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.547.1 Constructor & Destructor Documentation

10.547.1.1 [QtPointPropertyManager](#)()

```
QtPointPropertyManager::QtPointPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.547.1.2 [~QtPointPropertyManager](#)()

```
QtPointPropertyManager::~~QtPointPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.547.2 Member Function Documentation

10.547.2.1 initializeProperty()

```
void QtPointPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.547.2.2 setValue

```
void QtPointPropertyManager::setValue (
    QtProperty * property,
    const QPoint & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

See also

[value\(\)](#), [valueChanged\(\)](#)

10.547.2.3 subIntPropertyManager()

```
QtIntPropertyManager * QtPointPropertyManager::subIntPropertyManager ( ) const
```

Returns the manager that creates the nested *x* and *y* subproperties.

In order to provide editing widgets for the subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

10.547.2.4 uninitializedProperty()

```
void QtPointPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.547.2.5 value()

```
QPoint QtPointPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns a point with coordinates (0, 0).

See also

[setValue\(\)](#)

10.547.2.6 valueChanged

```
void QtPointPropertyManager::valueChanged (
    QtProperty * property,
    const QPoint & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.547.2.7 valueText()

```
QString QtPointPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

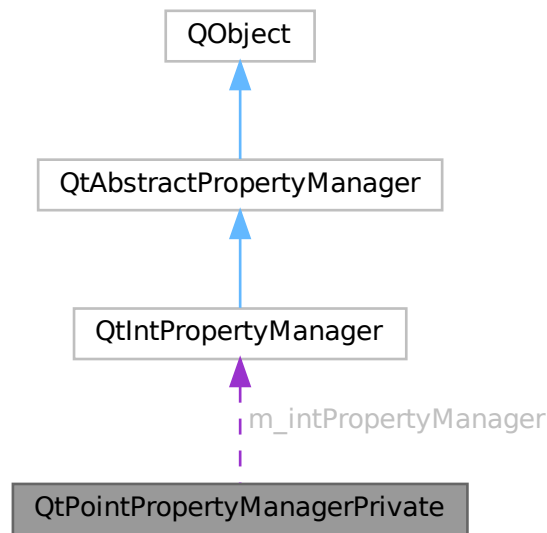
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.548 QtPointPropertyManagerPrivate Class Reference

Collaboration diagram for QtPointPropertyManagerPrivate:



Public Types

- typedef QMap< const [QtProperty](#) *, QPoint > **PropertyValueMap**

Public Member Functions

- void **slotIntChanged** ([QtProperty](#) *property, int value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)

Public Attributes

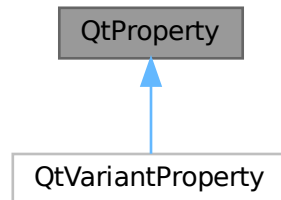
- PropertyValueMap **m_values**
- [QtIntPropertyManager](#) * **m_intPropertyManager**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToX**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToY**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_xToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_yToProperty**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.549 QtProperty Class Reference

Inheritance diagram for QtProperty:



Public Member Functions

- virtual [~QtProperty](#) ()
- [QList< QtProperty * > subProperties](#) () const
- [QtAbstractPropertyManager * propertyManager](#) () const
- [QString toolTip](#) () const
- [QString valueToolTip](#) () const
- [QString descriptionToolTip](#) () const
- [QString statusTip](#) () const
- [QString whatsThis](#) () const
- [QString propertyName](#) () const
- [bool isEnabled](#) () const
- [bool isModified](#) () const
- [bool hasValue](#) () const
- [QIcon valueIcon](#) () const
- [QString valueText](#) () const
- void [setToolTip](#) (const [QString &text](#))
- void [setValueToolTip](#) (const [QString &text](#))
- void [setDescriptionToolTip](#) (const [QString &text](#))
- void [setStatusTip](#) (const [QString &text](#))
- void [setWhatsThis](#) (const [QString &text](#))
- void [setPropertyname](#) (const [QString &text](#))
- void [setEnabled](#) (bool enable)
- void [setModified](#) (bool modified)
- void [addSubProperty](#) ([QtProperty *property](#))
- void [insertSubProperty](#) ([QtProperty *property](#), [QtProperty *afterProperty](#))
- void [removeSubProperty](#) ([QtProperty *property](#))

Protected Member Functions

- [QtProperty](#) ([QtAbstractPropertyManager *manager](#))
- void [propertyChanged](#) ()

Friends

- class **QtAbstractPropertyManager**

10.549.1 Constructor & Destructor Documentation

10.549.1.1 ~QtProperty()

```
QtProperty::~QtProperty ( ) [virtual]
```

Destroys this property.

Note that subproperties are detached but not destroyed, i.e. they can still be used in another context.

See also

[QtAbstractPropertyManager::clear\(\)](#)

10.549.1.2 QtProperty()

```
QtProperty::QtProperty (
    QtAbstractPropertyManager * manager ) [explicit], [protected]
```

Creates a property with the given *manager*.

This constructor is only useful when creating a custom [QtProperty](#) subclass (e.g. [QtVariantProperty](#)). To create a regular [QtProperty](#) object, use the [QtAbstractPropertyManager::addProperty\(\)](#) function instead.

See also

[QtAbstractPropertyManager::addProperty\(\)](#)

10.549.2 Member Function Documentation

10.549.2.1 addSubProperty()

```
void QtProperty::addSubProperty (
    QtProperty * property )
```

Appends the given *property* to this property's subproperties.

If the given *property* already is added, this function does nothing.

See also

[insertSubProperty\(\)](#), [removeSubProperty\(\)](#)

10.549.2.2 descriptionToolTip()

```
QString QtProperty::descriptionToolTip ( ) const
```

Returns the property description's tool tip.

This is suitable for tool tips over the description (label).

Since

5.6

See also

[setDescriptionToolTip\(\)](#)

10.549.2.3 hasValue()

```
bool QtProperty::hasValue ( ) const
```

Returns whether the property has a value.

See also

[QtAbstractPropertyManager::hasValue\(\)](#)

10.549.2.4 insertSubProperty()

```
void QtProperty::insertSubProperty (
    QtProperty * property,
    QtProperty * precedingProperty )
```

Inserts the given *property* after the specified *precedingProperty* into this property's list of subproperties. If *precedingProperty* is 0, the specified *property* is inserted at the beginning of the list.

If the given *property* already is inserted, this function does nothing.

See also

[addSubProperty\(\)](#), [removeSubProperty\(\)](#)

10.549.2.5 isEnabled()

```
bool QtProperty::isEnabled ( ) const
```

Returns whether the property is enabled.

See also

[setEnabled\(\)](#)

10.549.2.6 isModified()

```
bool QtProperty::isModified ( ) const
```

Returns whether the property is modified.

See also

[setModified\(\)](#)

10.549.2.7 propertyManager()

```
QtAbstractPropertyManager * QtProperty::propertyManager ( ) const
```

Returns a pointer to the manager that owns this property.

10.549.2.8 propertyName()

```
QString QtProperty::propertyName ( ) const
```

Returns the property's name.

See also

[setPropertyName\(\)](#)

10.549.2.9 removeSubProperty()

```
void QtProperty::removeSubProperty (
    QtProperty * property )
```

Removes the given *property* from the list of subproperties without deleting it.

See also

[addSubProperty\(\)](#), [insertSubProperty\(\)](#)

10.549.2.10 setDescriptionToolTip()

```
void QtProperty::setDescriptionToolTip (
    const QString & text )
```

Sets the property description's tool tip to the given *text*.

Since

5.6

See also

[descriptionToolTip\(\)](#)

10.549.2.11 setEnabled()

```
void QtProperty::setEnabled (
    bool enable )
```

Enables or disables the property according to the passed *enable* value.

See also

[isEnabled\(\)](#)

10.549.2.12 setModified()

```
void QtProperty::setModified (
    bool modified )
```

Sets the property's modified state according to the passed *modified* value.

See also

[isModified\(\)](#)

10.549.2.13 setPropertyName()

```
void QtProperty::setPropertyName (
    const QString & name )
```

Sets the property's name to the given *name*.

See also

[propertyName\(\)](#)

10.549.2.14 setStatusTip()

```
void QtProperty::setStatusTip (
    const QString & text )
```

Sets the property's status tip to the given *text*.

See also

[statusTip\(\)](#)

10.549.2.15 setValueToolTip()

```
void QtProperty::setValueToolTip (
    const QString & text )
```

Sets the property value's tool tip to the given *text*.

Since

5.6

See also

[valueToolTip\(\)](#)

10.549.2.16 setWhatsThis()

```
void QtProperty::setWhatsThis (
    const QString & text )
```

Sets the property's "What's This" help text to the given *text*.

See also

[whatsThis\(\)](#)

10.549.2.17 statusTip()

```
QString QtProperty::statusTip ( ) const
```

Returns the property's status tip.

See also

[setStatusTip\(\)](#)

10.549.2.18 subProperties()

```
QList< QtProperty * > QtProperty::subProperties ( ) const
```

Returns the set of subproperties.

Note that subproperties are not owned by *this* property, but by the manager that created them.

See also

[insertSubProperty\(\)](#), [removeSubProperty\(\)](#)

10.549.2.19 valueIcon()

```
QIcon QtProperty::valueIcon ( ) const
```

Returns an icon representing the current state of this property.

If the given property type can not generate such an icon, this function returns an invalid icon.

See also

[QtAbstractPropertyManager::valueIcon\(\)](#)

10.549.2.20 valueText()

```
QString QtProperty::valueText ( ) const
```

Returns a string representing the current state of this property.

If the given property type can not generate such a string, this function returns an empty string.

See also

[QtAbstractPropertyManager::valueText\(\)](#)

10.549.2.21 valueToolTip()

```
QString QtProperty::valueToolTip ( ) const
```

Returns the property value's tool tip.

This is suitable for tool tips over the value (item delegate).

Since

5.6

See also

[setValueToolTip\(\)](#)

10.549.2.22 whatsThis()

```
QString QtProperty::whatsThis ( ) const
```

Returns the property's "What's This" help text.

See also

[setWhatsThis\(\)](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

10.550 QtPropertyBrowserUtils Class Reference

Static Public Member Functions

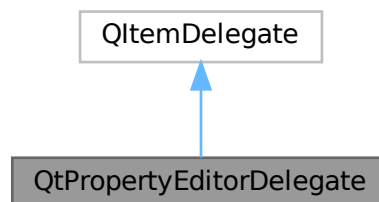
- static QPixmap **brushValuePixmap** (const QBrush &b)
- static QIcon **brushValueIcon** (const QBrush &b)
- static QString **colorValueText** (const QColor &c)
- static QPixmap **fontValuePixmap** (const QFont &f)
- static QIcon **fontValueIcon** (const QFont &f)
- static QString **fontValueText** (const QFont &f)
- static QString **dateFormat** ()
- static QString **timeFormat** ()
- static QString **dateTimeFormat** ()

The documentation for this class was generated from the following files:

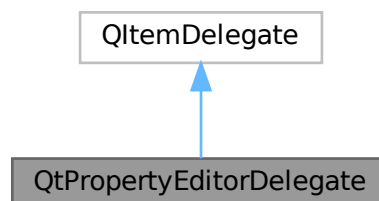
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

10.551 QtPropertyEditorDelegate Class Reference

Inheritance diagram for QtPropertyEditorDelegate:



Collaboration diagram for QtPropertyEditorDelegate:



Public Member Functions

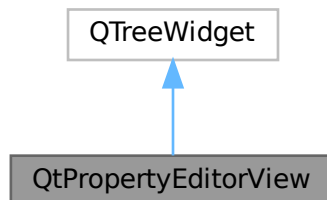
- **QtPropertyEditorDelegate** (QObject *parent=0)
- void **setEditorPrivate** (QtTreePropertyBrowserPrivate *editorPrivate)
- QWidget * **createEditor** (QWidget *parent, const QStyleOptionViewItem &option, const QModelIndex &index) const override
- void **updateEditorGeometry** (QWidget *editor, const QStyleOptionViewItem &option, const QModelIndex &index) const override
- void **paint** (QPainter *painter, const QStyleOptionViewItem &option, const QModelIndex &index) const override
- QSize **sizeHint** (const QStyleOptionViewItem &option, const QModelIndex &index) const override
- void **setModelData** (QWidget *, QAbstractItemModel *, const QModelIndex &) const override
- void **setEditorData** (QWidget *, const QModelIndex &) const override
- bool **eventFilter** (QObject *object, QEvent *event) override
- void **closeEditor** (QtProperty *property)
- QTreeWidgetItem * **editedItem** () const

The documentation for this class was generated from the following file:

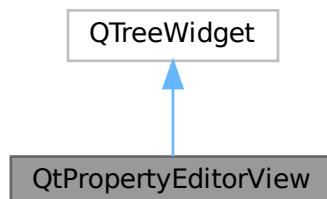
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qttreepropertybrowser.cpp

10.552 QtPropertyEditorView Class Reference

Inheritance diagram for QtPropertyEditorView:



Collaboration diagram for QtPropertyEditorView:



Public Member Functions

- **QtPropertyEditorView** (QWidget *parent=0)
- void **setEditorPrivate** ([QtTreePropertyBrowserPrivate](#) *editorPrivate)
- QTreeWidgetItem * **indexToItem** (const QModelIndex &index) const

Protected Member Functions

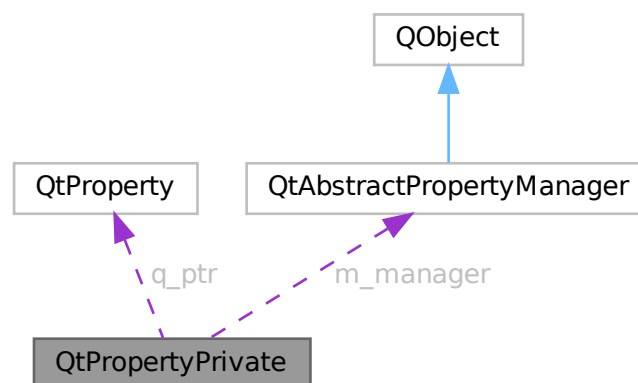
- void **keyPressEvent** (QKeyEvent *event) override
- void **mousePressEvent** (QMouseEvent *event) override
- void **drawRow** (QPainter *painter, const QStyleOptionViewItem &option, const QModelIndex &index) const override

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qttreepropertybrowser.cpp

10.553 QtPropertyPrivate Class Reference

Collaboration diagram for QtPropertyPrivate:



Public Member Functions

- **QtPropertyPrivate** ([QtAbstractPropertyManager](#) *manager)

Public Attributes

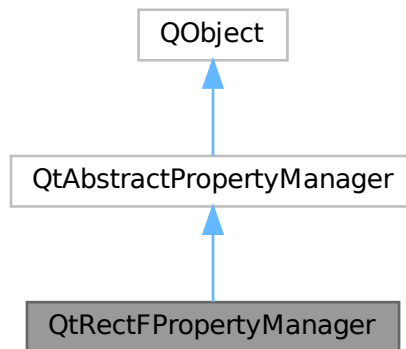
- [QtProperty](#) * **q_ptr**
- [QSet](#)< [QtProperty](#) * > **m_parentItems**
- [QList](#)< [QtProperty](#) * > **m_subItems**
- [QString](#) **m_valueToolTip**
- [QString](#) **m_descriptionToolTip**
- [QString](#) **m_statusTip**
- [QString](#) **m_whatsThis**
- [QString](#) **m_name**
- [bool](#) **m_enabled**
- [bool](#) **m_modified**
- [QtAbstractPropertyManager](#) *const **m_manager**

The documentation for this class was generated from the following file:

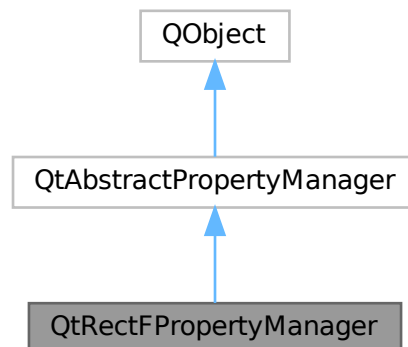
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowser.cpp

10.554 QtRectFPropertyManager Class Reference

Inheritance diagram for QtRectFPropertyManager:



Collaboration diagram for QtRectFPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QRectF &val)
- void [setConstraint](#) ([QtProperty](#) *property, const QRectF &[constraint](#))
- void [setDecimals](#) ([QtProperty](#) *property, int prec)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QRectF &val)
- void [constraintChanged](#) ([QtProperty](#) *property, const QRectF &[constraint](#))
- void [decimalsChanged](#) ([QtProperty](#) *property, int prec)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtRectFPropertyManager](#) (QObject *parent=0)
- [~QtRectFPropertyManager](#) ()
- [QtDoublePropertyManager](#) * [subDoublePropertyManager](#) () const
- QRectF [value](#) (const [QtProperty](#) *property) const
- QRectF [constraint](#) (const [QtProperty](#) *property) const
- int [decimals](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.554.1 Constructor & Destructor Documentation

10.554.1.1 [QtRectFPropertyManager](#)()

```
QtRectFPropertyManager::QtRectFPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.554.1.2 [~QtRectFPropertyManager](#)()

```
QtRectFPropertyManager::~~QtRectFPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.554.2 Member Function Documentation

10.554.2.1 [constraint](#)()

```
QRectF QtRectFPropertyManager::constraint (
    const QtProperty * property ) const
```

Returns the given *property*'s constraining rectangle. If returned value is null QRectF it means there is no constraint applied.

See also

[value\(\)](#), [setConstraint\(\)](#)

10.554.2.2 constraintChanged

```
void QtRectFPropertyManager::constraintChanged (
    QtProperty * property,
    const QRectF & constraint ) [signal]
```

This signal is emitted whenever property changes its constraint rectangle, passing a pointer to the *property* and the new *constraint* rectangle as parameters.

See also

[setConstraint\(\)](#)

10.554.2.3 decimals()

```
int QtRectFPropertyManager::decimals (
    const QtProperty * property ) const
```

Returns the given *property*'s precision, in decimals.

See also

[setDecimals\(\)](#)

10.554.2.4 decimalsChanged

```
void QtRectFPropertyManager::decimalsChanged (
    QtProperty * property,
    int prec ) [signal]
```

This signal is emitted whenever a property created by this manager changes its precision of value, passing a pointer to the *property* and the new *prec* value

See also

[setDecimals\(\)](#)

10.554.2.5 initializeProperty()

```
void QtRectFPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.554.2.6 setConstraint

```
void QtRectFPropertyManager::setConstraint (
    QtProperty * property,
    const QRectF & constraint ) [slot]
```

Sets the given *property*'s constraining rectangle to *constraint*.

When setting the constraint, the current value is adjusted if necessary (ensuring that the current rectangle value is inside the constraint). In order to reset the constraint pass a null QRectF value.

See also

[setValue\(\)](#), [constraint\(\)](#), [constraintChanged\(\)](#)

10.554.2.7 setDecimals

```
void QtRectFPropertyManager::setDecimals (
    QtProperty * property,
    int prec ) [slot]
```

Sets the precision of the given *property* to *prec*.

The valid decimal range is 0-13. The default is 2.

See also

[decimals\(\)](#)

10.554.2.8 setValue

```
void QtRectFPropertyManager::setValue (
    QtProperty * property,
    const QRectF & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

If the specified *value* is not inside the given *property*'s constraining rectangle, the value is adjusted accordingly to fit within the constraint.

See also

[value\(\)](#), [setConstraint\(\)](#), [valueChanged\(\)](#)

10.554.2.9 subDoublePropertyManager()

```
QtDoublePropertyManager * QtRectFPropertyManager::subDoublePropertyManager ( ) const
```

Returns the manager that creates the nested *x*, *y*, *width* and *height* subproperties.

In order to provide editing widgets for the mentioned subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

10.554.2.10 uninitializedProperty()

```
void QtRectFPropertyManager::uninitializedProperty (  
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.554.2.11 value()

```
QRectF QtRectFPropertyManager::value (  
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an invalid rectangle.

See also

[setValue\(\)](#), [constraint\(\)](#)

10.554.2.12 valueChanged

```
void QtRectFPropertyManager::valueChanged (  
    QtProperty * property,  
    const QRectF & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.554.2.13 valueText()

```
QString QtRectFPropertyManager::valueText (  
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

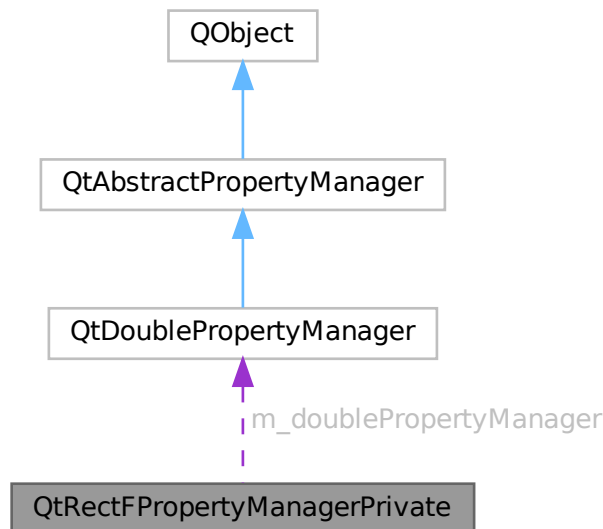
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.555 QtRectFPropertyManagerPrivate Class Reference

Collaboration diagram for QtRectFPropertyManagerPrivate:



Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- void **slotDoubleChanged** ([QtProperty](#) *property, double value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)
- void **setConstraint** ([QtProperty](#) *property, const QRectF &constraint, const QRectF &val)

Public Attributes

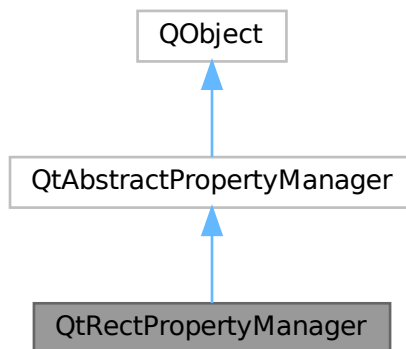
- PropertyValueMap **m_values**
- [QtDoublePropertyManager](#) * **m_doublePropertyManager**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToX**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToY**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToW**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToH**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_xToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_yToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_wToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_hToProperty**

The documentation for this class was generated from the following file:

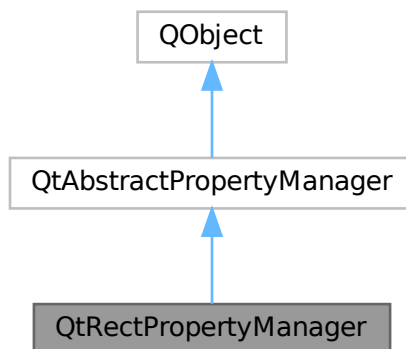
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.556 QtRectPropertyManager Class Reference

Inheritance diagram for QtRectPropertyManager:



Collaboration diagram for QtRectPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QRect &val)
- void [setConstraint](#) ([QtProperty](#) *property, const QRect &[constraint](#))

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QRect &val)
- void [constraintChanged](#) ([QtProperty](#) *property, const QRect &[constraint](#))

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtRectPropertyManager](#) (QObject *parent=0)
- [~QtRectPropertyManager](#) ()
- [QtIntPropertyManager](#) * [subIntPropertyManager](#) () const
- QRect [value](#) (const [QtProperty](#) *property) const
- QRect [constraint](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.556.1 Constructor & Destructor Documentation

10.556.1.1 [QtRectPropertyManager](#)()

```
QtRectPropertyManager::QtRectPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.556.1.2 [~QtRectPropertyManager](#)()

```
QtRectPropertyManager::~~QtRectPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.556.2 Member Function Documentation

10.556.2.1 `constraint()`

```
QRect QtRectPropertyManager::constraint (
    const QtProperty * property ) const
```

Returns the given *property*'s constraining rectangle. If returned value is null QRect it means there is no constraint applied.

See also

[value\(\)](#), [setConstraint\(\)](#)

10.556.2.2 `constraintChanged`

```
void QtRectPropertyManager::constraintChanged (
    QtProperty * property,
    const QRect & constraint ) [signal]
```

This signal is emitted whenever property changes its constraint rectangle, passing a pointer to the *property* and the new *constraint* rectangle as parameters.

See also

[setConstraint\(\)](#)

10.556.2.3 `initializeProperty()`

```
void QtRectPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.556.2.4 `setConstraint`

```
void QtRectPropertyManager::setConstraint (
    QtProperty * property,
    const QRect & constraint ) [slot]
```

Sets the given *property*'s constraining rectangle to *constraint*.

When setting the constraint, the current value is adjusted if necessary (ensuring that the current rectangle value is inside the constraint). In order to reset the constraint pass a null QRect value.

See also

[setValue\(\)](#), [constraint\(\)](#), [constraintChanged\(\)](#)

10.556.2.5 setValue

```
void QtRectPropertyManager::setValue (
    QtProperty * property,
    const QRect & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

If the specified *value* is not inside the given *property*'s constraining rectangle, the value is adjusted accordingly to fit within the constraint.

See also

[value\(\)](#), [setConstraint\(\)](#), [valueChanged\(\)](#)

10.556.2.6 subIntPropertyManager()

```
QtIntPropertyManager * QtRectPropertyManager::subIntPropertyManager ( ) const
```

Returns the manager that creates the nested *x*, *y*, *width* and *height* subproperties.

In order to provide editing widgets for the mentioned subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

10.556.2.7 uninitializedProperty()

```
void QtRectPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.556.2.8 value()

```
QRect QtRectPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an invalid rectangle.

See also

[setValue\(\)](#), [constraint\(\)](#)

10.556.2.9 valueChanged

```
void QtRectPropertyManager::valueChanged (
    QtProperty * property,
    const QRect & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.556.2.10 valueText()

```
QString QtRectPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

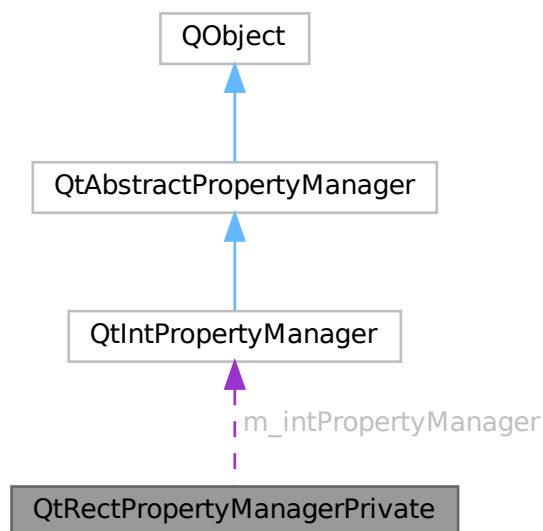
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.557 QtRectPropertyManagerPrivate Class Reference

Collaboration diagram for QtRectPropertyManagerPrivate:



Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- void **slotIntChanged** ([QtProperty](#) *property, int value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)
- void **setConstraint** ([QtProperty](#) *property, const QRect &constraint, const QRect &val)

Public Attributes

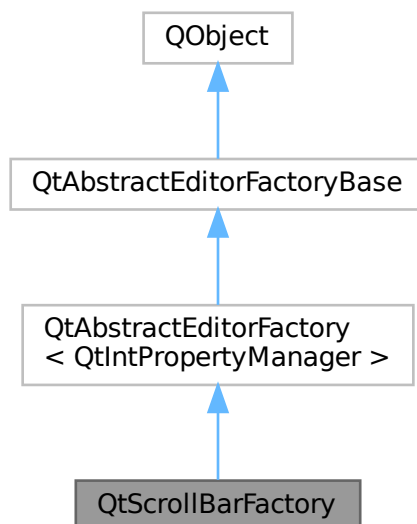
- PropertyValueMap **m_values**
- [QtIntPropertyManager](#) * **m_intPropertyManager**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToX**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToY**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToW**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToH**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_xToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_yToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_wToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_hToProperty**

The documentation for this class was generated from the following file:

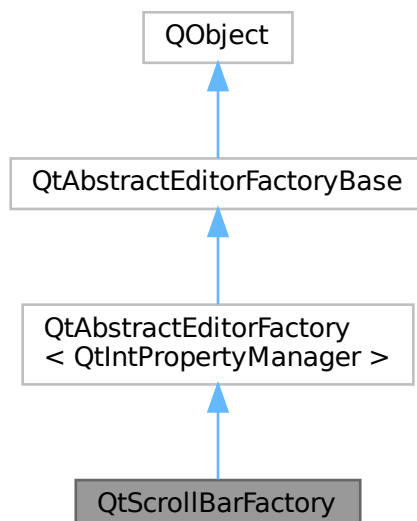
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.558 QtScrollBarFactory Class Reference

Inheritance diagram for QtScrollBarFactory:



Collaboration diagram for QtScrollBarFactory:



Public Member Functions

- [QtScrollBarFactory](#) (QObject *parent=0)
- [~QtScrollBarFactory](#) ()

Public Member Functions inherited from [QtAbstractEditorFactory](#) < [QtIntPropertyManager](#) >

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtIntPropertyManager *manager)
- void [removePropertyManager](#) (QtIntPropertyManager *manager)
- QSet< [QtIntPropertyManager](#) * > [propertyManagers](#) () const
- [QtIntPropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtIntPropertyManager *manager) override
- QWidget * [createEditor](#) (QtIntPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtIntPropertyManager *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactory](#) < [QtIntPropertyManager](#) >

- virtual QWidget * [createEditor](#) (QtIntPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtIntPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtIntPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

10.558.1 Constructor & Destructor Documentation

10.558.1.1 QtScrollBarFactory()

```
QtScrollBarFactory::QtScrollBarFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.558.1.2 ~QtScrollBarFactory()

```
QtScrollBarFactory::~QtScrollBarFactory ( )
```

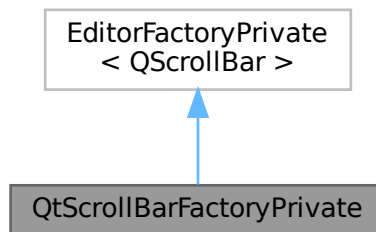
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

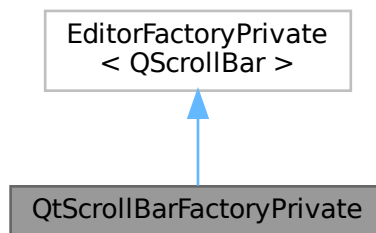
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.559 QtScrollBarFactoryPrivate Class Reference

Inheritance diagram for QtScrollBarFactoryPrivate:



Collaboration diagram for QtScrollBarFactoryPrivate:

**Public Member Functions**

- void **slotPropertyChanged** ([QtProperty](#) *property, int value)
- void **slotRangeChanged** ([QtProperty](#) *property, int min, int max)
- void **slotSingleStepChanged** ([QtProperty](#) *property, int step)
- void **slotSetValue** (int value)

Public Member Functions inherited from [EditorFactoryPrivate< QScrollBar >](#)

- `QScrollBar * createEditor (QtProperty *property, QWidget *parent)`
- `void initializeEditor (QtProperty *property, QScrollBar *e)`
- `void slotEditorDestroyed (QObject *object)`

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QScrollBar >](#)

- `typedef QList< QScrollBar * > EditorList`
- `typedef QMap< QtProperty *, EditorList > PropertyToEditorListMap`
- `typedef QMap< QScrollBar *, QtProperty * > EditorToPropertyMap`

Public Attributes inherited from [EditorFactoryPrivate< QScrollBar >](#)

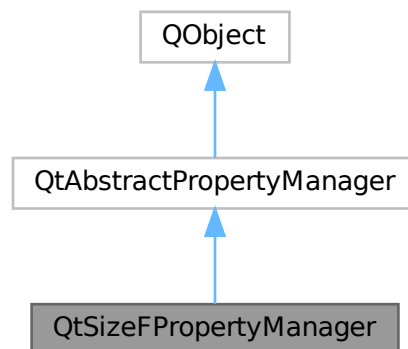
- `PropertyToEditorListMap m_createdEditors`
- `EditorToPropertyMap m_editorToProperty`

The documentation for this class was generated from the following file:

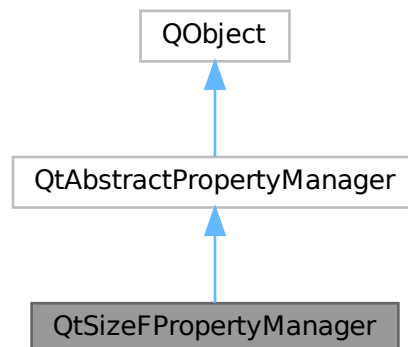
- `applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp`

10.560 QtSizeFPropertyManager Class Reference

Inheritance diagram for QtSizeFPropertyManager:



Collaboration diagram for QSizeFPropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QSizeF &val)
- void [setMinimum](#) ([QtProperty](#) *property, const QSizeF &minVal)
- void [setMaximum](#) ([QtProperty](#) *property, const QSizeF &maxVal)
- void [setRange](#) ([QtProperty](#) *property, const QSizeF &minVal, const QSizeF &maxVal)
- void [setDecimals](#) ([QtProperty](#) *property, int prec)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QSizeF &val)
- void [rangeChanged](#) ([QtProperty](#) *property, const QSizeF &minVal, const QSizeF &maxVal)
- void [decimalsChanged](#) ([QtProperty](#) *property, int prec)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QSizeFPropertyManager](#) (QObject *parent=0)
- [~QSizeFPropertyManager](#) ()
- [QtDoublePropertyManager](#) * [subDoublePropertyManager](#) () const
- QSizeF [value](#) (const [QtProperty](#) *property) const
- QSizeF [minimum](#) (const [QtProperty](#) *property) const
- QSizeF [maximum](#) (const [QtProperty](#) *property) const
- int [decimals](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.560.1 Constructor & Destructor Documentation

10.560.1.1 QtSizeFPropertyManager()

```
QtSizeFPropertyManager::QtSizeFPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.560.1.2 ~QtSizeFPropertyManager()

```
QtSizeFPropertyManager::~~QtSizeFPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.560.2 Member Function Documentation

10.560.2.1 decimals()

```
int QtSizeFPropertyManager::decimals (
    const QtProperty * property ) const
```

Returns the given *property*'s precision, in decimals.

See also

[setDecimals\(\)](#)

10.560.2.2 decimalsChanged

```
void QtSizeFPropertyManager::decimalsChanged (
    QtProperty * property,
    int prec ) [signal]
```

This signal is emitted whenever a property created by this manager changes its precision of value, passing a pointer to the *property* and the new *prec* value

See also

[setDecimals\(\)](#)

10.560.2.3 initializeProperty()

```
void QtSizeFPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.560.2.4 maximum()

```
QSizeF QtSizeFPropertyManager::maximum (
    const QtProperty * property ) const
```

Returns the given *property*'s maximum size value.

See also

[setMaximum\(\)](#), [minimum\(\)](#), [setRange\(\)](#)

10.560.2.5 minimum()

```
QSizeF QtSizeFPropertyManager::minimum (
    const QtProperty * property ) const
```

Returns the given *property*'s minimum size value.

See also

[setMinimum\(\)](#), [maximum\(\)](#), [setRange\(\)](#)

10.560.2.6 rangeChanged

```
void QSizeFPropertyManager::rangeChanged (
    QtProperty * property,
    const QSizeF & minimum,
    const QSizeF & maximum ) [signal]
```

This signal is emitted whenever a property created by this manager changes its range of valid sizes, passing a pointer to the *property* and the new *minimum* and *maximum* sizes.

See also

[setRange\(\)](#)

10.560.2.7 setDecimals

```
void QSizeFPropertyManager::setDecimals (
    QtProperty * property,
    int prec ) [slot]
```

Sets the precision of the given *property* to *prec*.

The valid decimal range is 0-13. The default is 2.

See also

[decimals\(\)](#)

10.560.2.8 setMaximum

```
void QSizeFPropertyManager::setMaximum (
    QtProperty * property,
    const QSizeF & maxVal ) [slot]
```

Sets the maximum size value for the given *property* to *maxVal*.

When setting the maximum size value, the minimum and current values are adjusted if necessary (ensuring that the size range remains valid and that the current value is within the range).

See also

[maximum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

10.560.2.9 setMinimum

```
void QSizeFPropertyManager::setMinimum (
    QtProperty * property,
    const QSizeF & minVal ) [slot]
```

Sets the minimum size value for the given *property* to *minVal*.

When setting the minimum size value, the maximum and current values are adjusted if necessary (ensuring that the size range remains valid and that the current value is within the range).

See also

[minimum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

10.560.2.10 setRange

```
void QSizeFPropertyManager::setRange (
    QtProperty * property,
    const QSizeF & minimum,
    const QSizeF & maximum ) [slot]
```

Sets the range of valid values.

This is a convenience function defining the range of valid values in one go; setting the *minimum* and *maximum* values for the given *property* with a single function call.

When setting a new range, the current value is adjusted if necessary (ensuring that the value remains within the range).

See also

[setMinimum\(\)](#), [setMaximum\(\)](#), [rangeChanged\(\)](#)

10.560.2.11 setValue

```
void QSizeFPropertyManager::setValue (
    QtProperty * property,
    const QSizeF & value ) [slot]
```

Sets the value of the given *property* to *value*.

If the specified *value* is not valid according to the given *property*'s size range, the *value* is adjusted to the nearest valid value within the size range.

See also

[value\(\)](#), [setRange\(\)](#), [valueChanged\(\)](#)

10.560.2.12 subDoublePropertyManager()

```
QtDoublePropertyManager * QSizeFPropertyManager::subDoublePropertyManager ( ) const
```

Returns the manager that creates the nested *width* and *height* subproperties.

In order to provide editing widgets for the *width* and *height* properties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

10.560.2.13 uninitializedProperty()

```
void QSizeFPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.560.2.14 value()

```
QSizeF QSizeFPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an invalid size

See also

[setValue\(\)](#)

10.560.2.15 valueChanged

```
void QSizeFPropertyManager::valueChanged (
    QtProperty * property,
    const QSizeF & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.560.2.16 valueText()

```
QString QSizeFPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

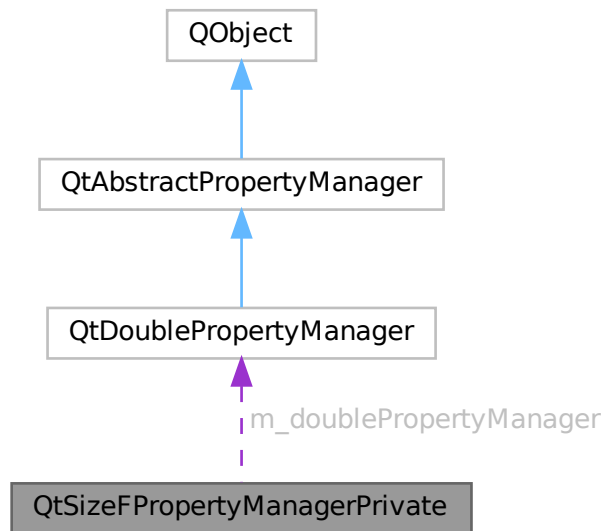
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.561 QtSizeFPropertyManagerPrivate Class Reference

Collaboration diagram for QtSizeFPropertyManagerPrivate:



Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- void **slotDoubleChanged** ([QtProperty](#) *property, double value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)
- void **setValue** ([QtProperty](#) *property, const QSizeF &val)
- void **setRange** ([QtProperty](#) *property, const QSizeF &minVal, const QSizeF &maxVal, const QSizeF &val)

Public Attributes

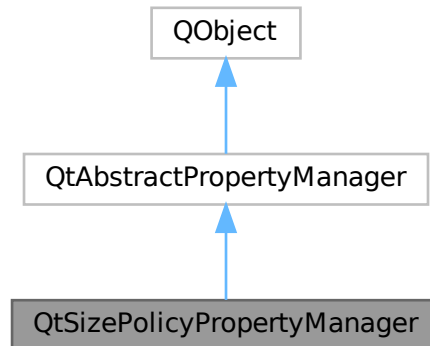
- PropertyValueMap **m_values**
- [QtDoublePropertyManager](#) * **m_doublePropertyManager**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToW**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToH**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_wToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_hToProperty**

The documentation for this class was generated from the following file:

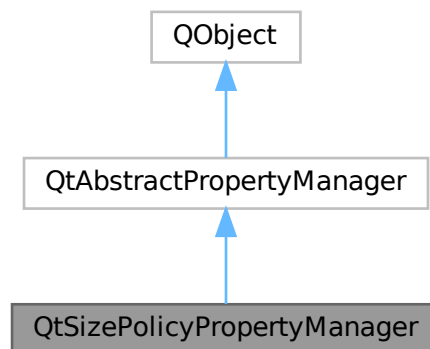
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.562 QSizePolicyPropertyManager Class Reference

Inheritance diagram for QSizePolicyPropertyManager:



Collaboration diagram for QSizePolicyPropertyManager:



Public Slots

- void `setValue` ([QtProperty](#) *property, const QSizePolicy &val)

Signals

- void `valueChanged` ([QtProperty](#) *property, const QSizePolicy &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtSizePolicyPropertyManager](#) (QObject *parent=0)
- [~QtSizePolicyPropertyManager](#) ()
- [QtIntPropertyManager](#) * [subIntPropertyManager](#) () const
- [QtEnumPropertyManager](#) * [subEnumPropertyManager](#) () const
- QSizePolicy [value](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.562.1 Constructor & Destructor Documentation

10.562.1.1 [QtSizePolicyPropertyManager](#)()

```
QtSizePolicyPropertyManager::QtSizePolicyPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.562.1.2 [~QtSizePolicyPropertyManager](#)()

```
QtSizePolicyPropertyManager::~~QtSizePolicyPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.562.2 Member Function Documentation

10.562.2.1 initializeProperty()

```
void QtSizePolicyPropertyManager::initializeProperty (  
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.562.2.2 setValue

```
void QtSizePolicyPropertyManager::setValue (  
    QtProperty * property,  
    const QSizePolicy & value ) [slot]
```

Sets the value of the given *property* to *value*. Nested properties are updated automatically.

See also

[value\(\)](#), [valueChanged\(\)](#)

10.562.2.3 subEnumPropertyManager()

```
QtEnumPropertyManager * QtSizePolicyPropertyManager::subEnumPropertyManager ( ) const
```

Returns the manager that creates the nested *horizontalPolicy* and *verticalPolicy* subproperties.

In order to provide editing widgets for the mentioned subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

10.562.2.4 subIntPropertyManager()

```
QtIntPropertyManager * QtSizePolicyPropertyManager::subIntPropertyManager ( ) const
```

Returns the manager that creates the nested *horizontalStretch* and *verticalStretch* subproperties.

In order to provide editing widgets for the mentioned subproperties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

10.562.2.5 uninitializedProperty()

```
void QtSizePolicyPropertyManager::uninitializedProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.562.2.6 value()

```
QSizePolicy QtSizePolicyPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given property is not managed by this manager, this function returns the default size policy.

See also

[setValue\(\)](#)

10.562.2.7 valueChanged

```
void QtSizePolicyPropertyManager::valueChanged (
    QtProperty * property,
    const QSizePolicy & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.562.2.8 valueText()

```
QString QtSizePolicyPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp Unknown size policy on reading invalid uic3 files

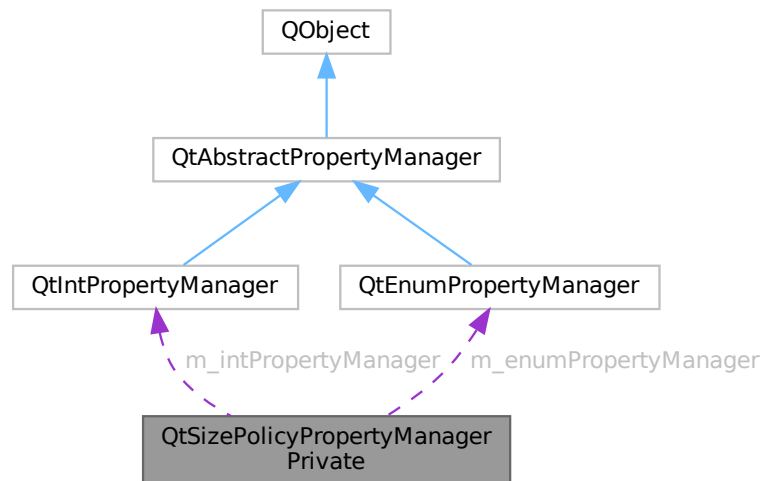
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.563 QtSizePolicyPropertyManagerPrivate Class Reference

Collaboration diagram for QtSizePolicyPropertyManagerPrivate:



Public Types

- typedef QMap< const [QtProperty](#) *, QSizePolicy > **PropertyValueMap**

Public Member Functions

- void **slotIntChanged** ([QtProperty](#) *property, int value)
- void **slotEnumChanged** ([QtProperty](#) *property, int value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)

Public Attributes

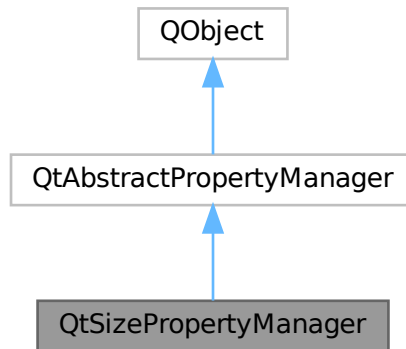
- PropertyValueMap **m_values**
- [QtIntPropertyManager](#) * **m_intPropertyManager**
- [QtEnumPropertyManager](#) * **m_enumPropertyManager**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToHPolicy**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToVPolicy**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToHStretch**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToVStretch**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_hPolicyToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_vPolicyToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_hStretchToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_vStretchToProperty**

The documentation for this class was generated from the following file:

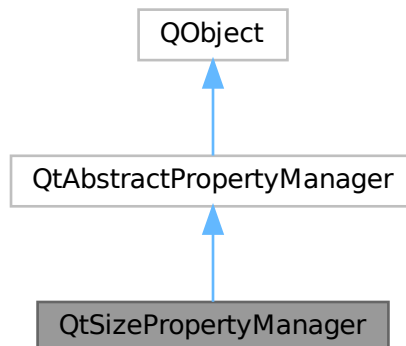
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.564 QSizePropertyManager Class Reference

Inheritance diagram for QSizePropertyManager:



Collaboration diagram for QSizePropertyManager:



Public Slots

- void [setValue](#) ([QtProperty](#) *property, const QSize &val)
- void [setMinimum](#) ([QtProperty](#) *property, const QSize &minVal)
- void [setMaximum](#) ([QtProperty](#) *property, const QSize &maxVal)
- void [setRange](#) ([QtProperty](#) *property, const QSize &minVal, const QSize &maxVal)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QSize &val)
- void [rangeChanged](#) ([QtProperty](#) *property, const QSize &minVal, const QSize &maxVal)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtSizePropertyManager](#) (QObject *parent=0)
- [~QtSizePropertyManager](#) ()
- [QtIntPropertyManager](#) * [subIntPropertyManager](#) () const
- QSize [value](#) (const [QtProperty](#) *property) const
- QSize [minimum](#) (const [QtProperty](#) *property) const
- QSize [maximum](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.564.1 Constructor & Destructor Documentation

10.564.1.1 [QtSizePropertyManager](#)()

```
QtSizePropertyManager::QtSizePropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.564.1.2 [~QtSizePropertyManager](#)()

```
QtSizePropertyManager::~~QtSizePropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.564.2 Member Function Documentation

10.564.2.1 initializeProperty()

```
void QSizePropertyManager::initializeProperty (  
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.564.2.2 maximum()

```
QSize QSizePropertyManager::maximum (  
    const QtProperty * property ) const
```

Returns the given *property*'s maximum size value.

See also

[setMaximum\(\)](#), [minimum\(\)](#), [setRange\(\)](#)

10.564.2.3 minimum()

```
QSize QSizePropertyManager::minimum (  
    const QtProperty * property ) const
```

Returns the given *property*'s minimum size value.

See also

[setMinimum\(\)](#), [maximum\(\)](#), [setRange\(\)](#)

10.564.2.4 rangeChanged

```
void QSizePropertyManager::rangeChanged (  
    QtProperty * property,  
    const QSize & minimum,  
    const QSize & maximum ) [signal]
```

This signal is emitted whenever a property created by this manager changes its range of valid sizes, passing a pointer to the *property* and the new *minimum* and *maximum* sizes.

See also

[setRange\(\)](#)

10.564.2.5 `setMaximum`

```
void QtSizePropertyManager::setMaximum (
    QtProperty * property,
    const QSize & maxVal ) [slot]
```

Sets the maximum size value for the given *property* to *maxVal*.

When setting the maximum size value, the minimum and current values are adjusted if necessary (ensuring that the size range remains valid and that the current value is within the range).

See also

[maximum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

10.564.2.6 `setMinimum`

```
void QtSizePropertyManager::setMinimum (
    QtProperty * property,
    const QSize & minVal ) [slot]
```

Sets the minimum size value for the given *property* to *minVal*.

When setting the minimum size value, the maximum and current values are adjusted if necessary (ensuring that the size range remains valid and that the current value is within the range).

See also

[minimum\(\)](#), [setRange\(\)](#), [rangeChanged\(\)](#)

10.564.2.7 `setRange`

```
void QtSizePropertyManager::setRange (
    QtProperty * property,
    const QSize & minimum,
    const QSize & maximum ) [slot]
```

Sets the range of valid values.

This is a convenience function defining the range of valid values in one go; setting the *minimum* and *maximum* values for the given *property* with a single function call.

When setting a new range, the current value is adjusted if necessary (ensuring that the value remains within the range).

See also

[setMinimum\(\)](#), [setMaximum\(\)](#), [rangeChanged\(\)](#)

10.564.2.8 setValue

```
void QSizePropertyManager::setValue (
    QtProperty * property,
    const QSize & value ) [slot]
```

Sets the value of the given *property* to *value*.

If the specified *value* is not valid according to the given *property*'s size range, the *value* is adjusted to the nearest valid value within the size range.

See also

[value\(\)](#), [setRange\(\)](#), [valueChanged\(\)](#)

10.564.2.9 subIntPropertyManager()

```
QtIntPropertyManager * QSizePropertyManager::subIntPropertyManager ( ) const
```

Returns the manager that creates the nested *width* and *height* subproperties.

In order to provide editing widgets for the *width* and *height* properties in a property browser widget, this manager must be associated with an editor factory.

See also

[QtAbstractPropertyBrowser::setFactoryForManager\(\)](#)

10.564.2.10 uninitializedProperty()

```
void QSizePropertyManager::uninitializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.564.2.11 value()

```
QSize QSizePropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an invalid size

See also

[setValue\(\)](#)

10.564.2.12 valueChanged

```
void QSizePropertyManager::valueChanged (
    QtProperty * property,
    const QSize & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.564.2.13 valueText()

```
QString QSizePropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

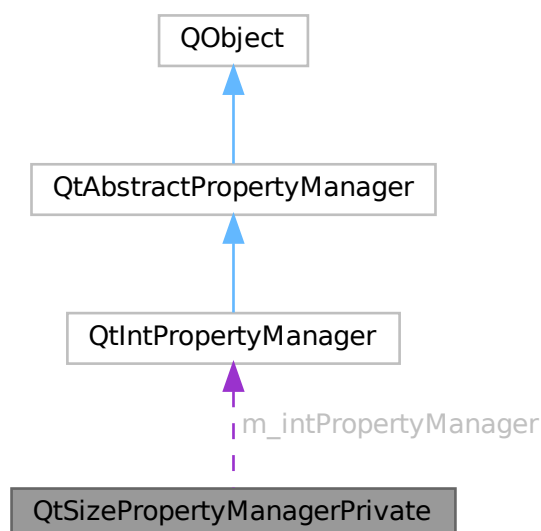
Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.565 QSizePropertyManagerPrivate Class Reference

Collaboration diagram for QSizePropertyManagerPrivate:



Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- void **slotIntChanged** ([QtProperty](#) *property, int value)
- void **slotPropertyDestroyed** ([QtProperty](#) *property)
- void **setValue** ([QtProperty](#) *property, const QSize &val)
- void **setRange** ([QtProperty](#) *property, const QSize &minVal, const QSize &maxVal, const QSize &val)

Public Attributes

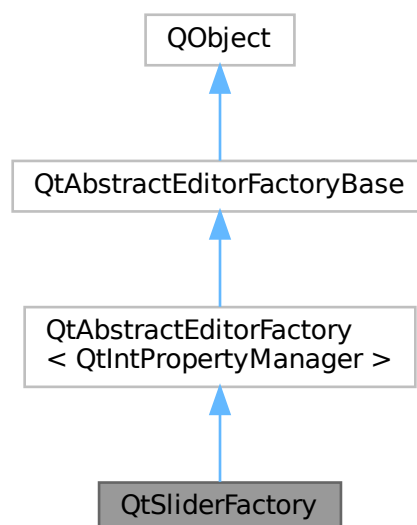
- PropertyValueMap **m_values**
- [QtIntPropertyManager](#) * **m_intPropertyManager**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToW**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_propertyToH**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_wToProperty**
- QMap< const [QtProperty](#) *, [QtProperty](#) * > **m_hToProperty**

The documentation for this class was generated from the following file:

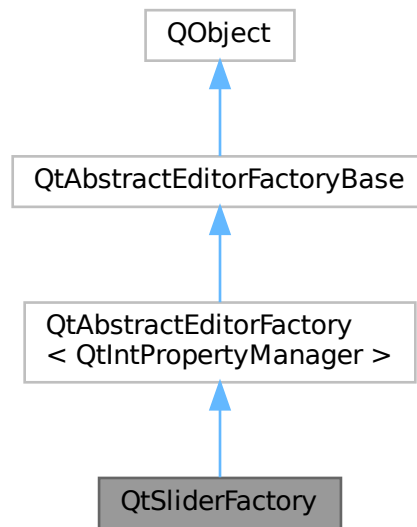
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.566 QtSliderFactory Class Reference

Inheritance diagram for QtSliderFactory:



Collaboration diagram for QtSliderFactory:



Public Member Functions

- [QtSliderFactory](#) (QObject *parent=0)
- [~QtSliderFactory](#) ()

Public Member Functions inherited from [QtAbstractEditorFactory < QtIntPropertyManager >](#)

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtIntPropertyManager *manager)
- void [removePropertyManager](#) (QtIntPropertyManager *manager)
- QSet < QtIntPropertyManager * > [propertyManagers](#) () const
- QtIntPropertyManager * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtIntPropertyManager *manager) override
- QWidget * [createEditor](#) (QtIntPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtIntPropertyManager *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactory < QtIntPropertyManager >](#)

- virtual QWidget * [createEditor](#) (QtIntPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtIntPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtIntPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

10.566.1 Constructor & Destructor Documentation

10.566.1.1 QtSliderFactory()

```
QtSliderFactory::QtSliderFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.566.1.2 ~QtSliderFactory()

```
QtSliderFactory::~QtSliderFactory ( )
```

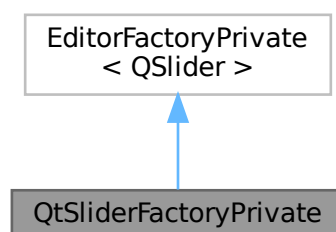
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

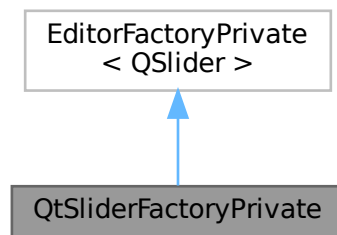
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.567 QtSliderFactoryPrivate Class Reference

Inheritance diagram for QtSliderFactoryPrivate:



Collaboration diagram for QtSliderFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, int value)
- void **slotRangeChanged** ([QtProperty](#) *property, int min, int max)
- void **slotSingleStepChanged** ([QtProperty](#) *property, int step)
- void **slotSetValue** (int value)

Public Member Functions inherited from [EditorFactoryPrivate< QSlider >](#)

- [QSlider](#) * **createEditor** ([QtProperty](#) *property, [QWidget](#) *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QSlider](#) *e)
- void **slotEditorDestroyed** ([QObject](#) *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QSlider >](#)

- typedef [QList](#)< [QSlider](#) * > **EditorList**
- typedef [QMap](#)< [QtProperty](#) *, [EditorList](#) > **PropertyToEditorListMap**
- typedef [QMap](#)< [QSlider](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QSlider >](#)

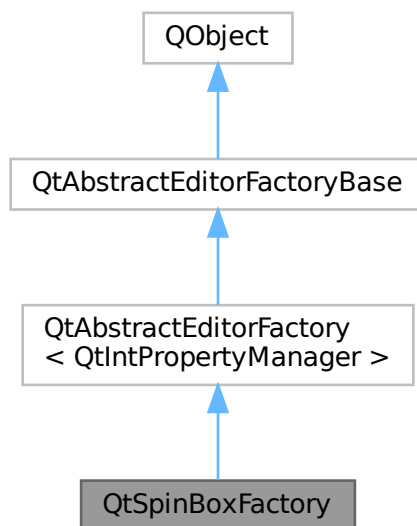
- [PropertyToEditorListMap](#) **m_createdEditors**
- [EditorToPropertyMap](#) **m_editorToProperty**

The documentation for this class was generated from the following file:

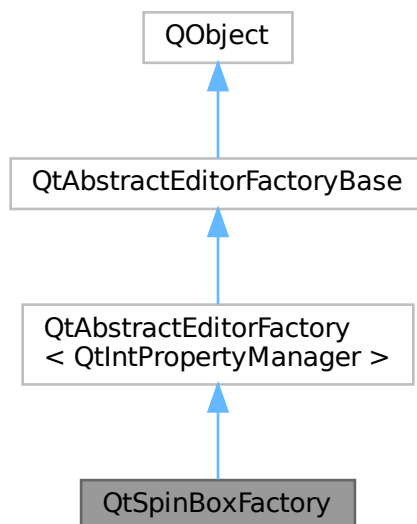
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.568 QtSpinBoxFactory Class Reference

Inheritance diagram for QtSpinBoxFactory:



Collaboration diagram for QtSpinBoxFactory:



Public Member Functions

- [QtSpinBoxFactory](#) (QObject *parent=0)
- [~QtSpinBoxFactory](#) ()

Public Member Functions inherited from [QtAbstractEditorFactory](#) < [QtIntPropertyManager](#) >

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtIntPropertyManager *manager)
- void [removePropertyManager](#) (QtIntPropertyManager *manager)
- QSet< [QtIntPropertyManager](#) * > [propertyManagers](#) () const
- [QtIntPropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtIntPropertyManager *manager) override
- QWidget * [createEditor](#) (QtIntPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtIntPropertyManager *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactory](#) < [QtIntPropertyManager](#) >

- virtual QWidget * [createEditor](#) (QtIntPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtIntPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtIntPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

10.568.1 Constructor & Destructor Documentation

10.568.1.1 QtSpinBoxFactory()

```
QtSpinBoxFactory::QtSpinBoxFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.568.1.2 ~QtSpinBoxFactory()

```
QtSpinBoxFactory::~QtSpinBoxFactory ( )
```

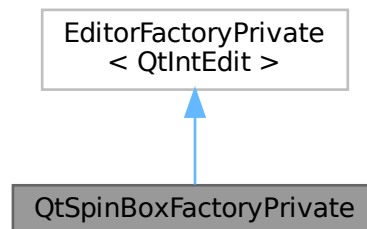
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

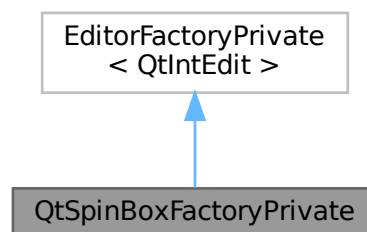
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.569 QtSpinBoxFactoryPrivate Class Reference

Inheritance diagram for QtSpinBoxFactoryPrivate:



Collaboration diagram for QtSpinBoxFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, int value)
- void **slotRangeChanged** ([QtProperty](#) *property, int min, int max)
- void **slotSingleStepChanged** ([QtProperty](#) *property, int step)
- void **slotSetValue** (int value)

Public Member Functions inherited from [EditorFactoryPrivate< QtIntEdit >](#)

- [QtIntEdit *](#) **createEditor** ([QtProperty *](#)property, QWidget *parent)
- void **initializeEditor** ([QtProperty *](#)property, [QtIntEdit *](#)e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QtIntEdit >](#)

- typedef QList< [QtIntEdit *](#) > **EditorList**
- typedef QMap< [QtProperty *](#), EditorList > **PropertyToEditorListMap**
- typedef QMap< [QtIntEdit *](#), [QtProperty *](#) > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QtIntEdit >](#)

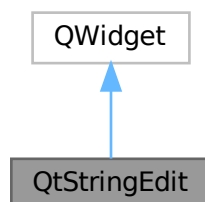
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

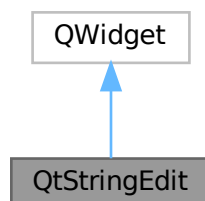
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.570 QtStringEdit Class Reference

Inheritance diagram for QtStringEdit:



Collaboration diagram for QtStringEdit:



Signals

- void **textChanged** (QString)
- void **textEdited** (QString)

Public Member Functions

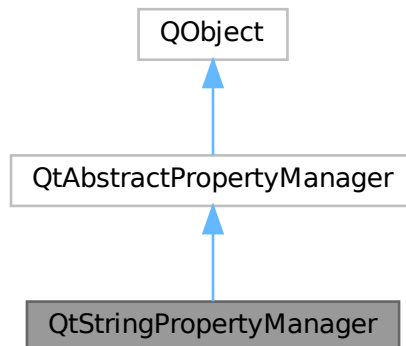
- **QtStringEdit** (QWidget *parent=Q_NULLPTR)
- QString **text** ()
- void **setText** (QString)
- void **setInitialText** (QString)
- const QValidator * **validator** ()
- void **setValidator** (QValidator *)

The documentation for this class was generated from the following files:

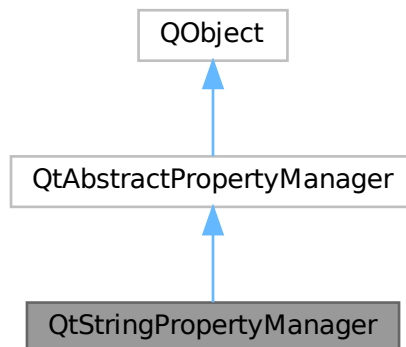
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

10.571 QtStringPropertyManager Class Reference

Inheritance diagram for QtStringPropertyManager:



Collaboration diagram for `QtStringPropertyManager`:



Public Slots

- void `setValue` (`QtProperty` *property, const `QString` &val)
- void `setValueOnly` (`QtProperty` *property, const `QString` &val)
- void `setRegExp` (`QtProperty` *property, const `QRegularExpression` ®Exp)

Signals

- void `valueChanged` (`QtProperty` *property, const `QString` &val)
- void `regExpChanged` (`QtProperty` *property, const `QRegularExpression` ®Exp)

Signals inherited from `QtAbstractPropertyManager`

- void `propertyInserted` (`QtProperty` *property, `QtProperty` *parent, `QtProperty` *after)
- void `propertyChanged` (`QtProperty` *property)
- void `propertyRemoved` (`QtProperty` *property, `QtProperty` *parent)
- void `propertyDestroyed` (`QtProperty` *property)

Public Member Functions

- `QtStringPropertyManager` (`QObject` *parent=0)
- `~QtStringPropertyManager` ()
- `QString value` (const `QtProperty` *property) const
- `QString initialValue` (const `QtProperty` *property) const
- `QRegularExpression regExp` (const `QtProperty` *property) const

Public Member Functions inherited from `QtAbstractPropertyManager`

- `QtAbstractPropertyManager` (`QObject` *parent=0)
- `~QtAbstractPropertyManager` ()
- `QSet< QtProperty * > properties` () const
- void `clear` () const
- `QtProperty` * `addProperty` (const `QString` &name=`QString`())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.571.1 Constructor & Destructor Documentation

10.571.1.1 QStringPropertyManager()

```
QStringPropertyManager::QStringPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.571.1.2 ~QStringPropertyManager()

```
QStringPropertyManager::~~QStringPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.571.2 Member Function Documentation

10.571.2.1 initializeProperty()

```
void QStringPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.571.2.2 regExp()

```
QRegularExpression QStringPropertyManager::regExp (
    const QtProperty * property ) const
```

Returns the given *property*'s currently set regular expression.

If the given *property* is not managed by this manager, this function returns an empty expression.

See also

[setRegExp\(\)](#)

10.571.2.3 regExpChanged

```
void QtStringPropertyManager::regExpChanged (
    QtProperty * property,
    const QRegularExpression & regExp ) [signal]
```

This signal is emitted whenever a property created by this manager changes its currently set regular expression, passing a pointer to the *property* and the new *regExp* as parameters.

See also

[setRegExp\(\)](#)

10.571.2.4 setRegExp

```
void QtStringPropertyManager::setRegExp (
    QtProperty * property,
    const QRegularExpression & regExp ) [slot]
```

Sets the regular expression of the given *property* to *regExp*.

See also

[regExp\(\)](#), [setValue\(\)](#), [regExpChanged\(\)](#)

10.571.2.5 setValue

```
void QtStringPropertyManager::setValue (
    QtProperty * property,
    const QString & value ) [slot]
```

Sets the value of the given *property* to *value*.

If the specified *value* doesn't match the given *property*'s regular expression, this function does nothing.

See also

[value\(\)](#), [setRegExp\(\)](#), [valueChanged\(\)](#)

10.571.2.6 uninitializedProperty()

```
void QtStringPropertyManager::uninitializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.571.2.7 value()

```
QString QStringPropertyManager::value (
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given property is not managed by this manager, this function returns an empty string.

See also

[setValue\(\)](#)

10.571.2.8 valueChanged

```
void QStringPropertyManager::valueChanged (
    QtProperty * property,
    const QString & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.571.2.9 valueText()

```
QString QStringPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.572 QStringPropertyManagerPrivate Class Reference

Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Attributes

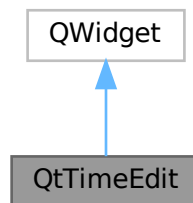
- QMap< const QtProperty *, Data > m_values

The documentation for this class was generated from the following file:

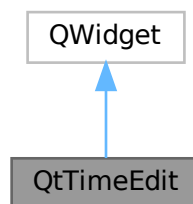
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.573 QtTimeEdit Class Reference

Inheritance diagram for QtTimeEdit:



Collaboration diagram for QtTimeEdit:



Signals

- void timeChanged (QTime)

Public Member Functions

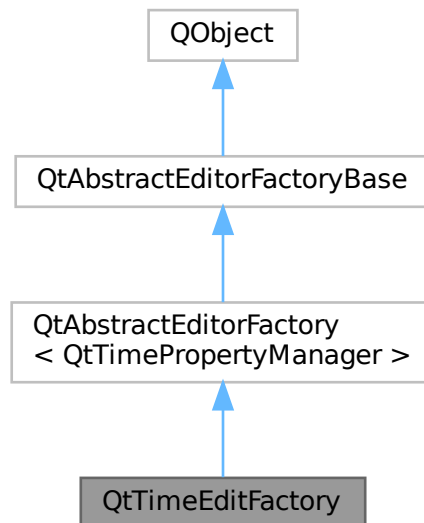
- **QtTimeEdit** (QWidget *parent=Q_NULLPTR)
- QTime **time** ()
- void **setTime** (QTime)
- void **setInitialTime** (QTime)
- void **setTimeRange** (QTime, QTime)
- void **setDisplayFormat** (QString)
- void **setCalendarPopup** (bool)

The documentation for this class was generated from the following files:

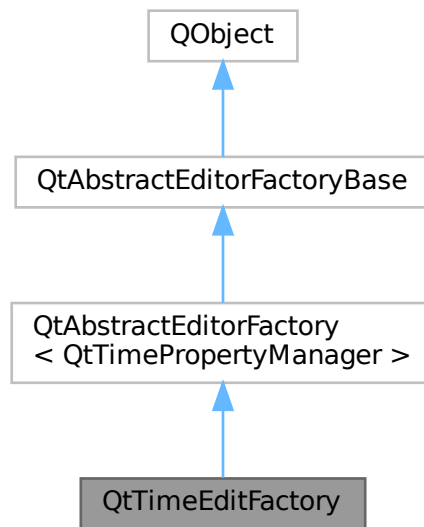
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils_p.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertybrowserutils.cpp

10.574 QtTimeEditFactory Class Reference

Inheritance diagram for QtTimeEditFactory:



Collaboration diagram for QtTimeEditFactory:



Public Member Functions

- [QtTimeEditFactory](#) (QObject *parent=0)
- [~QtTimeEditFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory < QtTimePropertyManager >](#)

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtTimePropertyManager *manager)
- void [removePropertyManager](#) (QtTimePropertyManager *manager)
- QSet< [QtTimePropertyManager](#) * > [propertyManagers](#) () const
- [QtTimePropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtTimePropertyManager *manager) override
- QWidget * [createEditor](#) (QtTimePropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtTimePropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory < QtTimePropertyManager >](#)

- virtual QWidget * [createEditor](#) (QtTimePropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtTimePropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtTimePropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

10.574.1 Constructor & Destructor Documentation

10.574.1.1 QtTimeEditFactory()

```
QtTimeEditFactory::QtTimeEditFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.574.1.2 ~QtTimeEditFactory()

```
QtTimeEditFactory::~QtTimeEditFactory ( )
```

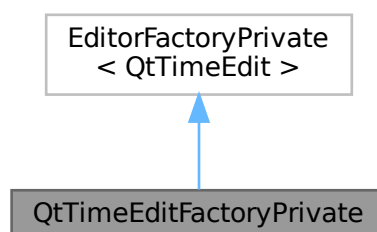
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

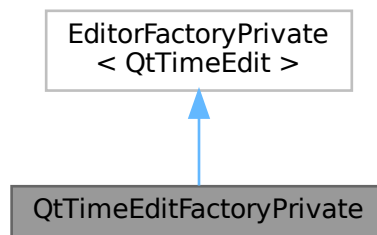
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.575 QtTimeEditFactoryPrivate Class Reference

Inheritance diagram for QtTimeEditFactoryPrivate:



Collaboration diagram for QtTimeEditFactoryPrivate:



Public Member Functions

- void **slotPropertyChanged** ([QtProperty](#) *property, QTime value)
- void **slotSetValue** (QTime value)

Public Member Functions inherited from [EditorFactoryPrivate< QtTimeEdit >](#)

- [QtTimeEdit](#) * **createEditor** ([QtProperty](#) *property, QWidget *parent)
- void **initializeEditor** ([QtProperty](#) *property, [QtTimeEdit](#) *e)
- void **slotEditorDestroyed** (QObject *object)

Additional Inherited Members

Public Types inherited from [EditorFactoryPrivate< QtTimeEdit >](#)

- typedef QList< [QtTimeEdit](#) * > **EditorList**
- typedef QMap< [QtProperty](#) *, EditorList > **PropertyToEditorListMap**
- typedef QMap< [QtTimeEdit](#) *, [QtProperty](#) * > **EditorToPropertyMap**

Public Attributes inherited from [EditorFactoryPrivate< QtTimeEdit >](#)

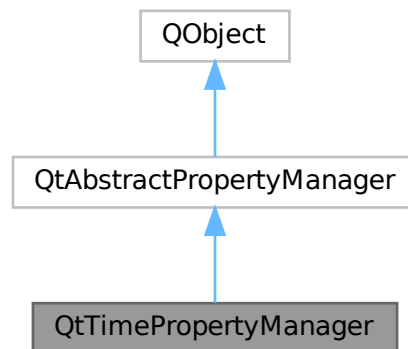
- PropertyToEditorListMap **m_createdEditors**
- EditorToPropertyMap **m_editorToProperty**

The documentation for this class was generated from the following file:

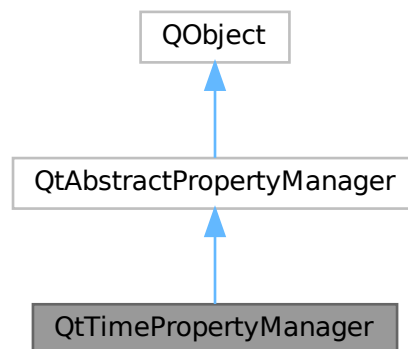
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qteditorfactory.cpp

10.576 QtTimePropertyManager Class Reference

Inheritance diagram for QtTimePropertyManager:



Collaboration diagram for QtTimePropertyManager:



Public Slots

- void `setValue` (`QtProperty` *property, QTime val)
- void `setValueOnly` (`QtProperty` *property, QTime val)

Signals

- void `valueChanged` (`QtProperty` *property, QTime val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtTimePropertyManager](#) (QObject *parent=0)
- [~QtTimePropertyManager](#) ()
- QTime [value](#) (const [QtProperty](#) *property) const
- QTime [initialValue](#) (const [QtProperty](#) *property) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Protected Member Functions

- QString [valueText](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override

Protected Member Functions inherited from [QtAbstractPropertyManager](#)

- virtual bool [hasValue](#) (const [QtProperty](#) *property) const
- virtual QIcon [valueIcon](#) (const [QtProperty](#) *property) const
- virtual [QtProperty](#) * [createProperty](#) ()

10.576.1 Constructor & Destructor Documentation

10.576.1.1 [QtTimePropertyManager](#)()

```
QtTimePropertyManager::QtTimePropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.576.1.2 [~QtTimePropertyManager](#)()

```
QtTimePropertyManager::~~QtTimePropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.576.2 Member Function Documentation

10.576.2.1 initializeProperty()

```
void QtTimePropertyManager::initializeProperty (  
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyManager](#).

10.576.2.2 setValue

```
void QtTimePropertyManager::setValue (  
    QtProperty * property,  
    QTime value ) [slot]
```

Sets the value of the given *property* to *value*.

See also

[value\(\)](#), [valueChanged\(\)](#)

10.576.2.3 uninitializedProperty()

```
void QtTimePropertyManager::uninitializedProperty (  
    QtProperty * property ) [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

10.576.2.4 value()

```
QTime QtTimePropertyManager::value (  
    const QtProperty * property ) const
```

Returns the given *property*'s value.

If the given property is not managed by this manager, this function returns an invalid time object.

See also

[setValue\(\)](#)

10.576.2.5 valueChanged

```
void QtTimePropertyManager::valueChanged (
    QtProperty * property,
    QTime value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.576.2.6 valueText()

```
QString QtTimePropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

\reimp

Reimplemented from [QtAbstractPropertyManager](#).

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.577 QtTimePropertyManagerPrivate Class Reference

Classes

- struct [Data](#)

Public Types

- typedef QMap< const [QtProperty](#) *, [Data](#) > **PropertyValueMap**

Public Member Functions

- **QtTimePropertyManagerPrivate** ([QtTimePropertyManager](#) *q)

Public Attributes

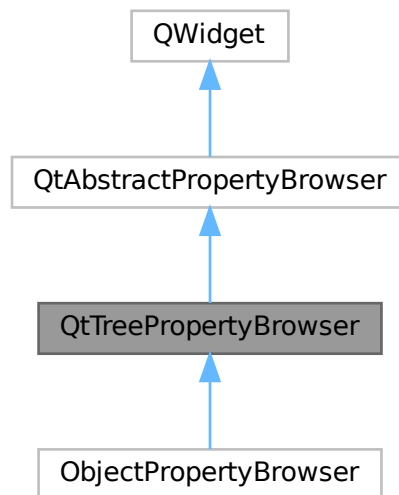
- const QString **m_format**
- PropertyValueMap **m_values**

The documentation for this class was generated from the following file:

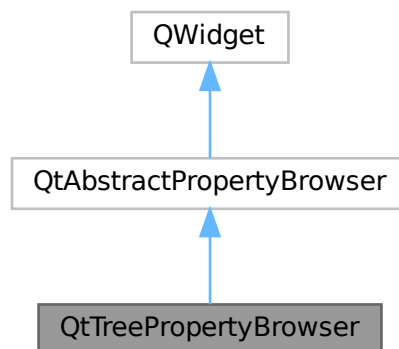
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtpropertymanager.cpp

10.578 QtTreePropertyBrowser Class Reference

Inheritance diagram for QtTreePropertyBrowser:



Collaboration diagram for QtTreePropertyBrowser:



Public Types

- enum [ResizeMode](#) { **Interactive** , **Stretch** , **Fixed** , **ResizeToContents** }

Signals

- void [collapsed](#) ([QtBrowserItem](#) *item)
- void [expanded](#) ([QtBrowserItem](#) *item)

Signals inherited from [QtAbstractPropertyBrowser](#)

- void [currentItemChanged](#) ([QtBrowserItem](#) *)

Public Member Functions

- [QtTreePropertyBrowser](#) (QWidget *parent=0)
- [~QtTreePropertyBrowser](#) ()
- int **indentation** () const
- void **setIndentation** (int i)
- bool **rootsDecorated** () const
- void **setRootsDecorated** (bool show)
- bool **alternatingRowColors** () const
- void **setAlternatingRowColors** (bool enable)
- bool **isHeaderVisible** () const
- void **setHeaderVisible** (bool visible)
- [ResizeMode](#) **resizeMode** () const
- void **setResizeMode** ([ResizeMode](#) mode)
- int **splitterPosition** () const
- void **setSplitterPosition** (int position)
- void [setExpanded](#) ([QtBrowserItem](#) *item, bool [expanded](#))
- bool [isExpanded](#) ([QtBrowserItem](#) *item) const
- bool [isItemVisible](#) ([QtBrowserItem](#) *item) const
- void [setItemVisible](#) ([QtBrowserItem](#) *item, bool visible)
- void [setBackground](#) ([QtBrowserItem](#) *item, const QColor &color)
- QColor [backgroundColor](#) ([QtBrowserItem](#) *item) const
- QColor [calculatedBackgroundColor](#) ([QtBrowserItem](#) *item) const
- void **setPropertyWithoutValueMarked** (bool mark)
- bool **propertiesWithoutValueMarked** () const
- void [editItem](#) ([QtBrowserItem](#) *item)

Public Member Functions inherited from [QtAbstractPropertyBrowser](#)

- [QtAbstractPropertyBrowser](#) (QWidget *parent=0)
- [~QtAbstractPropertyBrowser](#) ()
- QList< [QtProperty](#) * > **properties** () const
- QList< [QtBrowserItem](#) * > **items** ([QtProperty](#) *property) const
- [QtBrowserItem](#) * **topLevelItem** ([QtProperty](#) *property) const
- QList< [QtBrowserItem](#) * > **topLevelItems** () const
- void **clear** ()
- template<class PropertyManager >
void [setFactoryForManager](#) (PropertyManager *manager, [QtAbstractEditorFactory](#)< PropertyManager > *factory)
- void **unsetFactoryForManager** ([QtAbstractPropertyManager](#) *manager)
- [QtBrowserItem](#) * **currentItem** () const
- void **setCurrentItem** ([QtBrowserItem](#) *)

Protected Member Functions

- void [itemInserted](#) (QtBrowserItem *item, QtBrowserItem *afterItem) override
- void [itemRemoved](#) (QtBrowserItem *item) override
- void [itemChanged](#) (QtBrowserItem *item) override

Protected Member Functions inherited from [QtAbstractPropertyBrowser](#)

- virtual QWidget * [createEditor](#) (QtProperty *property, QWidget *parent)

Properties

- int **indentation**
indentation of the items in the tree view.
- bool **rootsDecorated**
whether to show controls for expanding and collapsing root items.
- bool **alternatingRowColors**
whether to draw the background using alternating colors. By default this property is set to true.
- bool **headerVisible**
whether to show the header.
- [ResizeMode](#) **resizeMode**
the resize mode of sections in the header.
- int **splitterPosition**
the position of the splitter between the columns.
- bool [propertiesWithoutValueMarked](#)
whether to enable or disable marking properties without value.

Additional Inherited Members**Public Slots inherited from [QtAbstractPropertyBrowser](#)**

- QtBrowserItem * [addProperty](#) (QtProperty *property)
- QtBrowserItem * [insertProperty](#) (QtProperty *property, QtProperty *afterProperty)
- void [removeProperty](#) (QtProperty *property)

10.578.1 Member Enumeration Documentation**10.578.1.1 [ResizeMode](#)**

enum [QtTreePropertyBrowser::ResizeMode](#)

The resize mode specifies the behavior of the header sections.

\value Interactive The user can resize the sections. The sections can also be resized programmatically using [setSplitterPosition\(\)](#).

\value Fixed The user cannot resize the section. The section can only be resized programmatically using [setSplitterPosition\(\)](#).

\value Stretch QHeaderView will automatically resize the section to fill the available space. The size cannot be changed by the user or programmatically.

\value ResizeToContents QHeaderView will automatically resize the section to its optimal size based on the contents of the entire column. The size cannot be changed by the user or programmatically.

See also

[setResizeMode\(\)](#)

10.578.2 Constructor & Destructor Documentation

10.578.2.1 QtTreePropertyBrowser()

```
QtTreePropertyBrowser::QtTreePropertyBrowser (
    QWidget * parent = 0 )
```

Creates a property browser with the given *parent*.

10.578.2.2 ~QtTreePropertyBrowser()

```
QtTreePropertyBrowser::~QtTreePropertyBrowser ( )
```

Destroys this property browser.

Note that the properties that were inserted into this browser are *not* destroyed since they may still be used in other browsers. The properties are owned by the manager that created them.

See also

[QtProperty](#), [QtAbstractPropertyManager](#)

10.578.3 Member Function Documentation

10.578.3.1 backgroundColor()

```
QColor QtTreePropertyBrowser::backgroundColor (
    QtBrowserItem * item ) const
```

Returns the *item*'s color. If there is no color set for item it returns invalid color.

See also

[calculatedBackgroundColor\(\)](#), [setBackgroundColor\(\)](#)

10.578.3.2 calculatedBackgroundColor()

```
QColor QtTreePropertyBrowser::calculatedBackgroundColor (
    QtBrowserItem * item ) const
```

Returns the *item*'s color. If there is no color set for item it returns parent *item*'s color (if there is no color set for parent it returns grandparent's color and so on). In case the color is not set for *item* and it's top level item it returns invalid color.

See also

[backgroundColor\(\)](#), [setBackgroundColor\(\)](#)

10.578.3.3 collapsed

```
void QtTreePropertyBrowser::collapsed (
    QtBrowserItem * item ) [signal]
```

This signal is emitted when the *item* is collapsed.

See also

[expanded\(\)](#), [setExpanded\(\)](#)

10.578.3.4 editItem()

```
void QtTreePropertyBrowser::editItem (
    QtBrowserItem * item )
```

Sets the current item to *item* and opens the relevant editor for it.

10.578.3.5 expanded

```
void QtTreePropertyBrowser::expanded (
    QtBrowserItem * item ) [signal]
```

This signal is emitted when the *item* is expanded.

See also

[collapsed\(\)](#), [setExpanded\(\)](#)

10.578.3.6 isExpanded()

```
bool QtTreePropertyBrowser::isExpanded (
    QtBrowserItem * item ) const
```

Returns true if the *item* is expanded; otherwise returns false.

See also

[setExpanded\(\)](#)

10.578.3.7 isItemVisible()

```
bool QtTreePropertyBrowser::isItemVisible (
    QtBrowserItem * item ) const
```

Returns true if the *item* is visible; otherwise returns false.

See also

[setItemVisible\(\)](#)

Since

4.5

10.578.3.8 itemChanged()

```
void QtTreePropertyBrowser::itemChanged (
    QtBrowserItem * item ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

10.578.3.9 itemInserted()

```
void QtTreePropertyBrowser::itemInserted (
    QtBrowserItem * item,
    QtBrowserItem * afterItem ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

10.578.3.10 itemRemoved()

```
void QtTreePropertyBrowser::itemRemoved (
    QtBrowserItem * item ) [override], [protected], [virtual]
```

\reimp

Implements [QtAbstractPropertyBrowser](#).

10.578.3.11 setBackgroundColor()

```
void QtTreePropertyBrowser::setBackgroundColor (
    QtBrowserItem * item,
    const QColor & color )
```

Sets the *item's* background color to *color*. Note that while item's background is rendered every second row is being drawn with alternate color (which is a bit lighter than items *color*)

See also

[backgroundColor\(\)](#), [calculatedBackgroundColor\(\)](#)

10.578.3.12 setExpanded()

```
void QtTreePropertyBrowser::setExpanded (
    QtBrowserItem * item,
    bool expanded )
```

Sets the *item* to either collapse or expanded, depending on the value of *expanded*.

See also

[isExpanded\(\)](#), [expanded\(\)](#), [collapsed\(\)](#)

10.578.3.13 setItemVisible()

```
void QtTreePropertyBrowser::setItemVisible (
    QtBrowserItem * item,
    bool visible )
```

Sets the *item* to be visible, depending on the value of *visible*.

See also

[isItemVisible\(\)](#)

Since

4.5

10.578.4 Property Documentation

10.578.4.1 propertiesWithoutValueMarked

```
QtTreePropertyBrowser::propertiesWithoutValueMarked [read], [write]
```

whether to enable or disable marking properties without value.

When marking is enabled the item's background is rendered in dark color and item's foreground is rendered with light color.

See also

[propertiesWithoutValueMarked\(\)](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qttreepropertybrowser.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qttreepropertybrowser.cpp

10.579 QtTreePropertyBrowserPrivate Class Reference

Public Member Functions

- void **init** (QWidget *parent)
- void **propertyInserted** (QtBrowserItem *index, QtBrowserItem *afterIndex)
- void **propertyRemoved** (QtBrowserItem *index)
- void **propertyChanged** (QtBrowserItem *index)
- QWidget * **createEditor** (QtProperty *property, QWidget *parent) const
- QtProperty * **indexToProperty** (const QModelIndex &index) const
- QTreeWidgetItem * **indexToItem** (const QModelIndex &index) const
- QtBrowserItem * **indexToBrowserItem** (const QModelIndex &index) const
- bool **lastColumn** (int column) const
- void **disableItem** (QTreeWidgetItem *item) const

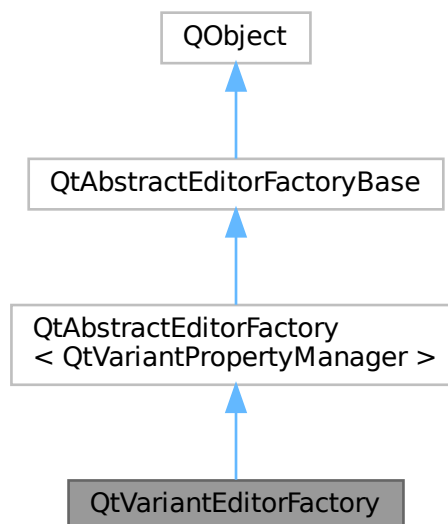
- void **enableItem** (QTreeWidgetItem *item) const
- bool **hasValue** (QTreeWidgetItem *item) const
- void **slotCollapsed** (const QModelIndex &index)
- void **slotExpanded** (const QModelIndex &index)
- QColor **calculatedBackgroundColor** (QtBrowserItem *item) const
- QtPropertyEditorView * **treeWidget** () const
- bool **markPropertiesWithoutValue** () const
- QtBrowserItem * **currentItem** () const
- void **setCurrentItem** (QtBrowserItem *browserItem, bool block)
- void **editItem** (QtBrowserItem *browserItem)
- void **slotCurrentBrowserItemChanged** (QtBrowserItem *item)
- void **slotCurrentTreeItemChanged** (QTreeWidgetItem *newItem, QTreeWidgetItem *)
- QTreeWidgetItem * **editedItem** () const

The documentation for this class was generated from the following file:

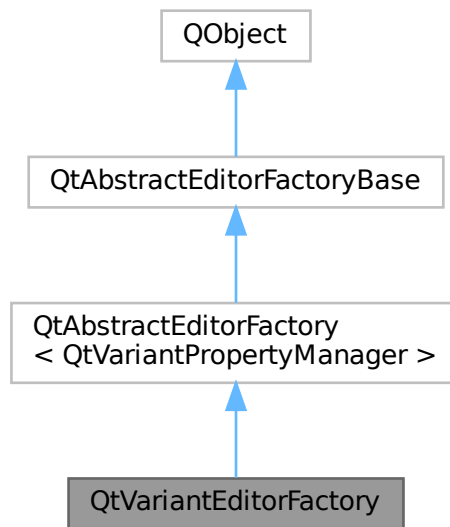
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qttreepropertybrowser.cpp

10.580 QtVariantEditorFactory Class Reference

Inheritance diagram for QtVariantEditorFactory:



Collaboration diagram for QtVariantEditorFactory:



Public Member Functions

- [QtVariantEditorFactory](#) (QObject *parent=0)
- [~QtVariantEditorFactory](#) ()

Public Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtVariantPropertyManager](#) >

- [QtAbstractEditorFactory](#) (QObject *parent)
- QWidget * [createEditor](#) (QtProperty *property, QWidget *parent) override
- void [addPropertyManager](#) (QtVariantPropertyManager *manager)
- void [removePropertyManager](#) (QtVariantPropertyManager *manager)
- QSet< [QtVariantPropertyManager](#) * > [propertyManagers](#) () const
- [QtVariantPropertyManager](#) * [propertyManager](#) (QtProperty *property) const

Protected Member Functions

- void [connectPropertyManager](#) (QtVariantPropertyManager *manager) override
- QWidget * [createEditor](#) (QtVariantPropertyManager *manager, QtProperty *property, QWidget *parent) override
- void [disconnectPropertyManager](#) (QtVariantPropertyManager *manager) override

Protected Member Functions inherited from

[QtAbstractEditorFactory](#) < [QtVariantPropertyManager](#) >

- virtual QWidget * [createEditor](#) (QtVariantPropertyManager *manager, QtProperty *property, QWidget *parent)=0
- virtual void [connectPropertyManager](#) (QtVariantPropertyManager *manager)=0
- virtual void [disconnectPropertyManager](#) (QtVariantPropertyManager *manager)=0
- void [managerDestroyed](#) (QObject *manager) override

Protected Member Functions inherited from [QtAbstractEditorFactoryBase](#)

- [QtAbstractEditorFactoryBase](#) (QObject *parent=0)

Additional Inherited Members

Protected Slots inherited from [QtAbstractEditorFactoryBase](#)

10.580.1 Constructor & Destructor Documentation

10.580.1.1 QtVariantEditorFactory()

```
QtVariantEditorFactory::QtVariantEditorFactory (
    QObject * parent = 0 )
```

Creates a factory with the given *parent*.

10.580.1.2 ~QtVariantEditorFactory()

```
QtVariantEditorFactory::~QtVariantEditorFactory ( )
```

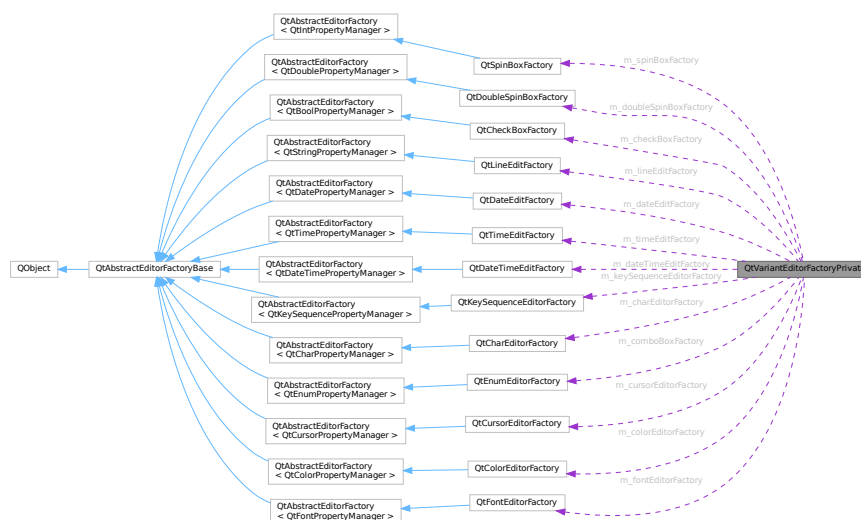
Destroys this factory, and all the widgets it has created.

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

10.581 QtVariantEditorFactoryPrivate Class Reference

Collaboration diagram for QtVariantEditorFactoryPrivate:



Public Attributes

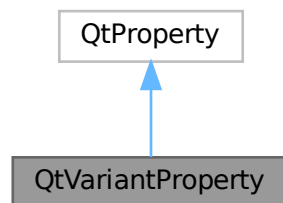
- [QtSpinBoxFactory](#) * **m_spinBoxFactory**
- [QtDoubleSpinBoxFactory](#) * **m_doubleSpinBoxFactory**
- [QtCheckBoxFactory](#) * **m_checkBoxFactory**
- [QtLineEditFactory](#) * **m_lineEditFactory**
- [QtDateEditFactory](#) * **m_dateEditFactory**
- [QtTimeEditFactory](#) * **m_timeEditFactory**
- [QtDateTimeEditFactory](#) * **m_dateTimeEditFactory**
- [QtKeySequenceEditorFactory](#) * **m_keySequenceEditorFactory**
- [QtCharEditorFactory](#) * **m_charEditorFactory**
- [QtEnumEditorFactory](#) * **m_comboBoxFactory**
- [QtCursorEditorFactory](#) * **m_cursorEditorFactory**
- [QtColorEditorFactory](#) * **m_colorEditorFactory**
- [QtFontEditorFactory](#) * **m_fontEditorFactory**
- QMap< [QtAbstractEditorFactoryBase](#) *, int > **m_factoryToType**
- QMap< int, [QtAbstractEditorFactoryBase](#) * > **m_typeToFactory**

The documentation for this class was generated from the following file:

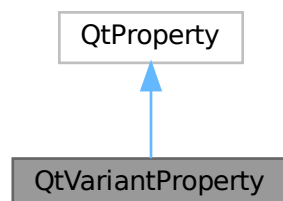
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

10.582 QtVariantProperty Class Reference

Inheritance diagram for QtVariantProperty:



Collaboration diagram for QtVariantProperty:



Public Member Functions

- [~QtVariantProperty](#) ()
- QVariant [value](#) () const
- QVariant [attributeValue](#) (const QString &attribute) const
- int [valueType](#) () const
- int [propertyType](#) () const
- void [setValue](#) (const QVariant &[value](#))
- void [setAttribute](#) (const QString &attribute, const QVariant &[value](#))

Public Member Functions inherited from [QtProperty](#)

- virtual [~QtProperty](#) ()
- QList< [QtProperty](#) * > [subProperties](#) () const
- [QtAbstractPropertyManager](#) * [propertyManager](#) () const
- QString [toolTip](#) () const
- QString [valueToolTip](#) () const
- QString [descriptionToolTip](#) () const
- QString [statusTip](#) () const
- QString [whatsThis](#) () const
- QString [propertyName](#) () const
- bool [isEnabled](#) () const
- bool [isModified](#) () const
- bool [hasValue](#) () const
- QIcon [valueIcon](#) () const
- QString [valueText](#) () const
- void [setToolTip](#) (const QString &text)
- void [setValueToolTip](#) (const QString &text)
- void [setDescriptionToolTip](#) (const QString &text)
- void [setStatusTip](#) (const QString &text)
- void [setWhatsThis](#) (const QString &text)
- void [setPropertyName](#) (const QString &text)
- void [setEnabled](#) (bool enable)
- void [setModified](#) (bool modified)
- void [addSubProperty](#) ([QtProperty](#) *property)
- void [insertSubProperty](#) ([QtProperty](#) *property, [QtProperty](#) *afterProperty)
- void [removeSubProperty](#) ([QtProperty](#) *property)

Protected Member Functions

- [QtVariantProperty](#) ([QtVariantPropertyManager](#) *manager)

Protected Member Functions inherited from [QtProperty](#)

- [QtProperty](#) ([QtAbstractPropertyManager](#) *manager)
- void [propertyChanged](#) ()

Friends

- class [QtVariantPropertyManager](#)

10.582.1 Constructor & Destructor Documentation

10.582.1.1 ~QtVariantProperty()

```
QtVariantProperty::~QtVariantProperty ( )
```

Destroys this property.

See also

[QtProperty::~~QtProperty\(\)](#)

10.582.1.2 QtVariantProperty()

```
QtVariantProperty::QtVariantProperty (
    QtVariantPropertyManager * manager ) [protected]
```

Creates a variant property using the given *manager*.

Do not use this constructor to create variant property instances; use the [QtVariantPropertyManager::addProperty\(\)](#) function instead. This constructor is used internally by the [QtVariantPropertyManager::createProperty\(\)](#) function.

See also

[QtVariantPropertyManager](#)

10.582.2 Member Function Documentation

10.582.2.1 attributeValue()

```
QVariant QtVariantProperty::attributeValue (
    const QString & attribute ) const
```

Returns this property's value for the specified *attribute*.

[QtVariantPropertyManager](#) provides a couple of related functions: [\{QtVariantPropertyManager::attributes\(\)\}](#)[{attributes\(\)}](#) and [\{QtVariantPropertyManager::attributeType\(\)\}](#)[{attributeType\(\)}](#).

See also

[setAttribute\(\)](#)

10.582.2.2 propertyType()

```
int QtVariantProperty::propertyType ( ) const
```

Returns this property's type.

[QtVariantPropertyManager](#) provides several related functions: [\{QtVariantPropertyManager::enumTypeId\(\)\}](#)[{enum↵TypeId\(\)↵}](#), [\{QtVariantPropertyManager::flagTypeId\(\)\}](#)[{flagTypeId\(\)↵}](#) and [\{QtVariantPropertyManager::groupTypeId\(\)\}](#)[{group↵TypeId\(\)↵}](#).

See also

[valueType\(\)](#)

10.582.2.3 `setAttribute()`

```
void QtVariantProperty::setAttribute (
    const QString & attribute,
    const QVariant & value )
```

Sets the *attribute* of property to *value*.

[QtVariantPropertyManager](#) provides the related `\\{QtVariantPropertyManager::setAttribute()}\\{setAttribute()}` function.

See also

[attributeValue\(\)](#)

10.582.2.4 `setValue()`

```
void QtVariantProperty::setValue (
    const QVariant & value )
```

Sets the value of this property to *value*.

The specified *value* must be of the type returned by `valueType()`, or of a type that can be converted to `valueType()` using the `QVariant::canConvert()` function; otherwise this function does nothing.

See also

[value\(\)](#)

10.582.2.5 `value()`

```
QVariant QtVariantProperty::value ( ) const
```

Returns the property's current value.

See also

`valueType()`, [setValue\(\)](#)

10.582.2.6 `valueType()`

```
int QtVariantProperty::valueType ( ) const
```

Returns the type of this property's value.

See also

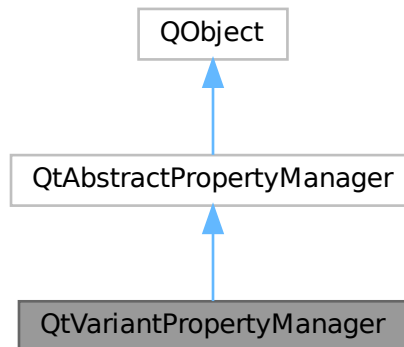
[propertyType\(\)](#)

The documentation for this class was generated from the following files:

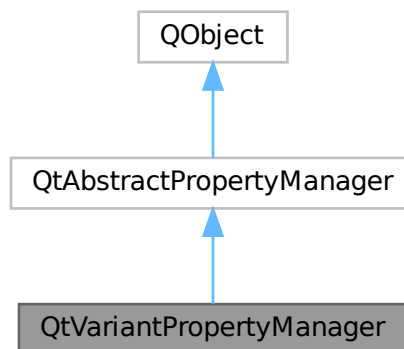
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

10.583 QtVariantPropertyManager Class Reference

Inheritance diagram for QtVariantPropertyManager:



Collaboration diagram for QtVariantPropertyManager:



Public Slots

- virtual void [setValue](#) ([QtProperty](#) *property, const QVariant &val)
- virtual void [setAttribute](#) ([QtProperty](#) *property, const QString &attribute, const QVariant &value)

Signals

- void [valueChanged](#) ([QtProperty](#) *property, const QVariant &val)
- void [attributeChanged](#) ([QtProperty](#) *property, const QString &attribute, const QVariant &val)

Signals inherited from [QtAbstractPropertyManager](#)

- void [propertyInserted](#) ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void [propertyChanged](#) ([QtProperty](#) *property)
- void [propertyRemoved](#) ([QtProperty](#) *property, [QtProperty](#) *parent)
- void [propertyDestroyed](#) ([QtProperty](#) *property)

Public Member Functions

- [QtVariantPropertyManager](#) (QObject *parent=0)
- [~QtVariantPropertyManager](#) ()
- virtual [QtVariantProperty](#) * [addProperty](#) (int [propertyType](#), const QString &name=QString())
- int [propertyType](#) (const [QtProperty](#) *property) const
- int [valueType](#) (const [QtProperty](#) *property) const
- [QtVariantProperty](#) * [variantProperty](#) (const [QtProperty](#) *property) const
- virtual bool [isPropertyTypeSupported](#) (int [propertyType](#)) const
- virtual int [valueType](#) (int [propertyType](#)) const
- virtual QStringList [attributes](#) (int [propertyType](#)) const
- virtual int [attributeType](#) (int [propertyType](#), const QString &attribute) const
- virtual QVariant [value](#) (const [QtProperty](#) *property) const
- virtual QVariant [attributeValue](#) (const [QtProperty](#) *property, const QString &attribute) const

Public Member Functions inherited from [QtAbstractPropertyManager](#)

- [QtAbstractPropertyManager](#) (QObject *parent=0)
- [~QtAbstractPropertyManager](#) ()
- QSet< [QtProperty](#) * > [properties](#) () const
- void [clear](#) () const
- [QtProperty](#) * [addProperty](#) (const QString &name=QString())

Static Public Member Functions

- static int [enumTypeId](#) ()
- static int [flagTypeId](#) ()
- static int [groupTypeId](#) ()
- static int [iconMapTypeId](#) ()

Protected Member Functions

- bool [hasValue](#) (const [QtProperty](#) *property) const override
- QString [valueText](#) (const [QtProperty](#) *property) const override
- QIcon [valueIcon](#) (const [QtProperty](#) *property) const override
- void [initializeProperty](#) ([QtProperty](#) *property) override
- void [uninitializeProperty](#) ([QtProperty](#) *property) override
- [QtProperty](#) * [createProperty](#) () override

10.583.1 Constructor & Destructor Documentation

10.583.1.1 QtVariantPropertyManager()

```
QtVariantPropertyManager::QtVariantPropertyManager (
    QObject * parent = 0 )
```

Creates a manager with the given *parent*.

10.583.1.2 ~QtVariantPropertyManager()

```
QtVariantPropertyManager::~~QtVariantPropertyManager ( )
```

Destroys this manager, and all the properties it has created.

10.583.2 Member Function Documentation

10.583.2.1 addProperty()

```
QtVariantProperty * QtVariantPropertyManager::addProperty (
    int propertyType,
    const QString & name = QString() ) [virtual]
```

Creates and returns a variant property of the given *propertyType* with the given *name*.

If the specified *propertyType* is not supported by this variant manager, this function returns 0.

Do not use the inherited [QtAbstractPropertyManager::addProperty\(\)](#) function to create a variant property (that function will always return 0 since it will not be clear what type the property should have).

See also

[isPropertyTypeSupported\(\)](#)

10.583.2.2 attributeChanged

```
void QtVariantPropertyManager::attributeChanged (
    QtProperty * property,
    const QString & attribute,
    const QVariant & value ) [signal]
```

This signal is emitted whenever an attribute of a property created by this manager changes its value, passing a pointer to the *property*, the *attribute* and the new *value* as parameters.

See also

[setAttribute\(\)](#)

10.583.2.3 attributes()

```
QStringList QtVariantPropertyManager::attributes (
    int propertyType ) const [virtual]
```

Returns a list of the given *propertyType*'s attributes.

See also

[attributeValue\(\)](#), [attributeType\(\)](#)

10.583.2.4 attributeType()

```
int QtVariantPropertyManager::attributeType (
    int propertyType,
    const QString & attribute ) const [virtual]
```

Returns the type of the specified *attribute* of the given *propertyType*.

If the given *propertyType* is not supported by *this* manager, or if the given *propertyType* does not possess the specified *attribute*, this function returns `QMetaType::UnknownType`.

See also

[attributes\(\)](#), [valueType\(\)](#)

10.583.2.5 attributeValue()

```
QVariant QtVariantPropertyManager::attributeValue (
    const QtProperty * property,
    const QString & attribute ) const [virtual]
```

Returns the given *property*'s value for the specified *attribute*

If the given *property* was not created by *this* manager, or if the specified *attribute* does not exist, this function returns an invalid variant.

See also

[attributes\(\)](#), [attributeType\(\)](#), [setAttribute\(\)](#)

10.583.2.6 createProperty()

```
QtProperty * QtVariantPropertyManager::createProperty ( ) [override], [protected], [virtual]
```

Creates a property.

The base implementation produce [QtProperty](#) instances; Reimplement this function to make this manager produce objects of a [QtProperty](#) subclass.

See also

[addProperty\(\)](#), [initializeProperty\(\)](#)

Reimplemented from [QtAbstractPropertyManager](#).

10.583.2.7 enumTypeId()

```
QT_END_NAMESPACE QT_BEGIN_NAMESPACE int QtVariantPropertyManager::enumTypeId ( ) [static]
```

Returns the type id for an enum property.

Note that the property's value type can be retrieved using the `valueType()` function (which is `QMetaType::Int` for the enum property type).

See also

[propertyType\(\)](#), [valueType\(\)](#)

10.583.2.8 flagTypeId()

```
int QtVariantPropertyManager::flagTypeId ( ) [static]
```

Returns the type id for a flag property.

Note that the property's value type can be retrieved using the `valueType()` function (which is `QMetaType::Int` for the flag property type).

See also

[propertyType\(\)](#), [valueType\(\)](#)

10.583.2.9 groupTypeId()

```
int QtVariantPropertyManager::groupTypeId ( ) [static]
```

Returns the type id for a group property.

Note that the property's value type can be retrieved using the `valueType()` function (which is `QMetaType::Unknown`↵
Type for the group property type, since it doesn't provide any value).

See also

[propertyType\(\)](#), [valueType\(\)](#)

10.583.2.10 hasValue()

```
bool QtVariantPropertyManager::hasValue (
    const QtProperty * property ) const [override], [protected], [virtual]
```

Returns whether the given *property* has a value.

The default implementation of this function returns true.

See also

[QtProperty::hasValue\(\)](#)

Reimplemented from [QtAbstractPropertyManager](#).

10.583.2.11 iconMapTypeId()

```
int QtVariantPropertyManager::iconMapTypeId ( ) [static]
```

Returns the type id for a icon map attribute.

Note that the property's attribute type can be retrieved using the [attributeType\(\)](#) function.

See also

[attributeType\(\)](#), [QtEnumPropertyManager::enumIcons\(\)](#)

10.583.2.12 initializeProperty()

```
void QtVariantPropertyManager::initializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

This function is called whenever a new valid property pointer has been created, passing the pointer as parameter.

The purpose is to let the manager know that the *property* has been created so that it can provide additional attributes for the new property, e.g. [QtIntPropertyManager](#) adds `\ {QtIntPropertyManager::value()}{value}`, `\ {QtIntPropertyManager::minimum()}{minimum}` and `\ {QtIntPropertyManager::maximum()}{maximum}` attributes. Since each manager subclass adds type specific attributes, this function is pure virtual and must be reimplemented when deriving from the [QtAbstractPropertyManager](#) class.

See also

[addProperty\(\)](#), [createProperty\(\)](#)

Implements [QtAbstractPropertyManager](#).

10.583.2.13 isPropertyTypeSupported()

```
bool QtVariantPropertyManager::isPropertyTypeSupported (
    int propertyType ) const [virtual]
```

Returns true if the given *propertyType* is supported by this variant manager; otherwise false.

See also

[propertyType\(\)](#)

10.583.2.14 propertyType()

```
int QtVariantPropertyManager::propertyType (
    const QtProperty * property ) const
```

Returns the given *property*'s type.

See also

[valueType\(\)](#)

10.583.2.15 setAttribute

```
void QtVariantPropertyManager::setAttribute (
    QtProperty * property,
    const QString & attribute,
    const QVariant & value ) [virtual], [slot]
```

Sets the value of the specified *attribute* of the given *property*, to *value*.

The new *value*'s type must be of the type returned by [attributeType\(\)](#), or of a type that can be converted to [attributeType\(\)](#) using the `QVariant::canConvert()` function, otherwise this function does nothing.

See also

[attributeValue\(\)](#), [QtVariantProperty::setAttribute\(\)](#), [attributeChanged\(\)](#)

10.583.2.16 setValue

```
void QtVariantPropertyManager::setValue (
    QtProperty * property,
    const QVariant & value ) [virtual], [slot]
```

Sets the value of the given *property* to *value*.

The specified *value* must be of a type returned by [valueType\(\)](#), or of type that can be converted to [valueType\(\)](#) using the `QVariant::canConvert()` function, otherwise this function does nothing.

See also

[value\(\)](#), [QtVariantProperty::setValue\(\)](#), [valueChanged\(\)](#)

10.583.2.17 uninitializedProperty()

```
void QtVariantPropertyManager::uninitializeProperty (
    QtProperty * property ) [override], [protected], [virtual]
```

This function is called just before the specified *property* is destroyed.

The purpose is to let the property manager know that the *property* is being destroyed so that it can remove the property's additional attributes.

See also

[clear\(\)](#), [propertyDestroyed\(\)](#)

Reimplemented from [QtAbstractPropertyManager](#).

10.583.2.18 value()

```
QVariant QtVariantPropertyManager::value (
    const QtProperty * property ) const [virtual]
```

Returns the given *property*'s value.

If the given *property* is not managed by this manager, this function returns an invalid variant.

See also

[setValue\(\)](#)

10.583.2.19 valueChanged

```
void QtVariantPropertyManager::valueChanged (
    QtProperty * property,
    const QVariant & value ) [signal]
```

This signal is emitted whenever a property created by this manager changes its value, passing a pointer to the *property* and the new *value* as parameters.

See also

[setValue\(\)](#)

10.583.2.20 valueIcon()

```
QIcon QtVariantPropertyManager::valueIcon (
    const QtProperty * property ) const [override], [protected], [virtual]
```

Returns an icon representing the current state of the given *property*.

The default implementation of this function returns an invalid icon.

See also

[QtProperty::valueIcon\(\)](#)

Reimplemented from [QtAbstractPropertyManager](#).

10.583.2.21 valueText()

```
QString QtVariantPropertyManager::valueText (
    const QtProperty * property ) const [override], [protected], [virtual]
```

Returns a string representing the current state of the given *property*.

The default implementation of this function returns an empty string.

See also

[QtProperty::valueText\(\)](#)

Reimplemented from [QtAbstractPropertyManager](#).

10.583.2.22 valueType() [1/2]

```
int QtVariantPropertyManager::valueType (
    const QtProperty * property ) const
```

Returns the given *property*'s value type.

See also

[propertyType\(\)](#)

10.583.2.23 valueType() [2/2]

```
int QtVariantPropertyManager::valueType (
    int propertyType ) const [virtual]
```

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

Returns the value type associated with the given *propertyType*.

10.583.2.24 variantProperty()

```
QtVariantProperty * QtVariantPropertyManager::variantProperty (
    const QtProperty * property ) const
```

Returns the given *property* converted into a [QtVariantProperty](#).

If the *property* was not created by this variant manager, the function returns 0.

See also

[createProperty\(\)](#)

The documentation for this class was generated from the following files:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.h
- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

10.584 QtVariantPropertyManagerPrivate Class Reference

Public Member Functions

- void **slotValueChanged** ([QtProperty](#) *property, int val)
- void **slotRangeChanged** ([QtProperty](#) *property, int min, int max)
- void **slotSingleStepChanged** ([QtProperty](#) *property, int step)
- void **slotValueChanged** ([QtProperty](#) *property, double val)
- void **slotRangeChanged** ([QtProperty](#) *property, double min, double max)
- void **slotSingleStepChanged** ([QtProperty](#) *property, double step)
- void **slotDecimalsChanged** ([QtProperty](#) *property, int prec)
- void **slotValueChanged** ([QtProperty](#) *property, bool val)
- void **slotValueChanged** ([QtProperty](#) *property, const QString &val)
- void **slotRegExpChanged** ([QtProperty](#) *property, const QRegularExpression ®Exp)
- void **slotValueChanged** ([QtProperty](#) *property, QDate val)
- void **slotRangeChanged** ([QtProperty](#) *property, QDate min, QDate max)
- void **slotValueChanged** ([QtProperty](#) *property, QTime val)
- void **slotValueChanged** ([QtProperty](#) *property, const QDateTime &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QKeySequence &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QChar &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QLocale &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QPoint &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QPointF &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QSize &val)
- void **slotRangeChanged** ([QtProperty](#) *property, const QSize &min, const QSize &max)
- void **slotValueChanged** ([QtProperty](#) *property, const QSizeF &val)
- void **slotRangeChanged** ([QtProperty](#) *property, const QSizeF &min, const QSizeF &max)
- void **slotValueChanged** ([QtProperty](#) *property, const QRect &val)
- void **slotConstraintChanged** ([QtProperty](#) *property, const QRect &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QRectF &val)
- void **slotConstraintChanged** ([QtProperty](#) *property, const QRectF &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QColor &val)
- void **slotEnumChanged** ([QtProperty](#) *property, int val)
- void **slotEnumNamesChanged** ([QtProperty](#) *property, const QStringList &enumNames)
- void **slotEnumIconsChanged** ([QtProperty](#) *property, const QMap< int, QIcon > &enumIcons)
- void **slotValueChanged** ([QtProperty](#) *property, const QSizePolicy &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QFont &val)
- void **slotValueChanged** ([QtProperty](#) *property, const QCursor &val)
- void **slotFlagChanged** ([QtProperty](#) *property, int val)
- void **slotFlagNamesChanged** ([QtProperty](#) *property, const QStringList &flagNames)
- void **slotPropertyInserted** ([QtProperty](#) *property, [QtProperty](#) *parent, [QtProperty](#) *after)
- void **slotPropertyRemoved** ([QtProperty](#) *property, [QtProperty](#) *parent)
- void **valueChanged** ([QtProperty](#) *property, const QVariant &val)
- int **internalPropertyToType** ([QtProperty](#) *property) const
- [QtVariantProperty](#) * **createSubProperty** ([QtVariantProperty](#) *parent, [QtVariantProperty](#) *after, [QtProperty](#) *internal)
- void **removeSubProperty** ([QtVariantProperty](#) *property)

Public Attributes

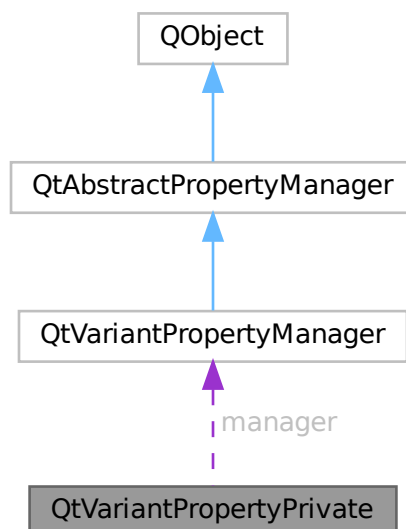
- bool **m_creatingProperty**
- bool **m_creatingSubProperties**
- bool **m_destroyingSubProperties**
- int **m_propertyType**
- QMap< int, [QtAbstractPropertyManager](#) * > **m_typeToPropertyManager**
- QMap< int, QMap< QString, int > > **m_typeToAttributeToAttributeType**
- QMap< const [QtProperty](#) *, QMap< [QtVariantProperty](#) *, int > > **m_propertyToType**
- QMap< int, int > **m_typeToValueType**
- QMap< [QtProperty](#) *, [QtVariantProperty](#) * > **m_internalToProperty**
- const QString **m_constraintAttribute**
- const QString **m_singleStepAttribute**
- const QString **m_decimalsAttribute**
- const QString **m_enumIconsAttribute**
- const QString **m_enumNamesAttribute**
- const QString **m_flagNamesAttribute**
- const QString **m_maximumAttribute**
- const QString **m_minimumAttribute**
- const QString **m_regExpAttribute**

The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

10.585 QtVariantPropertyPrivate Class Reference

Collaboration diagram for QtVariantPropertyPrivate:



Public Member Functions

- **QtVariantPropertyPrivate** ([QtVariantPropertyManager](#) *m)

Public Attributes

- [QtVariantPropertyManager](#) * **manager**

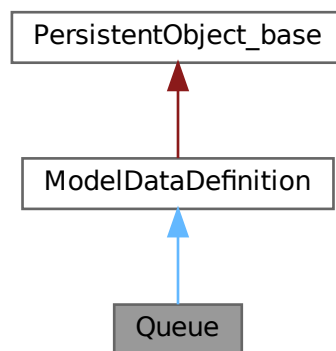
The documentation for this class was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtvariantproperty.cpp

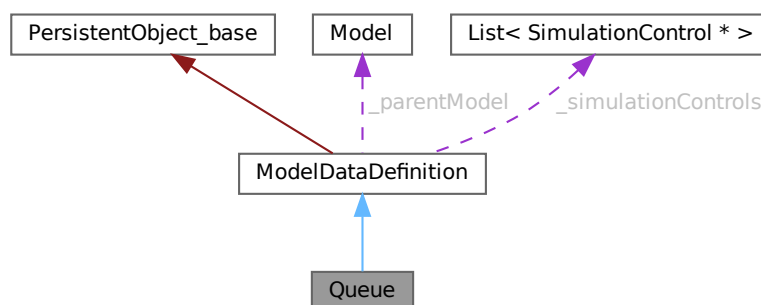
10.586 Queue Class Reference

```
#include <plugins/data/DiscreteProcessing/Queue.h>
```

Inheritance diagram for Queue:



Collaboration diagram for Queue:



Public Types

- enum class **OrderRule** : int {
FIFO = 0 , **LIFO** = 1 , **HIGHESTVALUE** = 2 , **SMALLESTVALUE** = 3 ,
num_elements = 4 }

Public Member Functions

- Queue** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
Returns a textual representation of this data definition.
- void **insertElement** ([Waiting](#) *modeldatum)
- void **removeElement** ([Waiting](#) *modeldatum)
- unsigned int **size** ()
- [Waiting](#) * **first** ()
- [Waiting](#) * **getAtRank** (unsigned int rank)
- void **setAttributeName** (std::string _attributeName)
- std::string **getAttributeName** () const
- void **setOrderRule** (OrderRule _orderRule)
- Queue::OrderRule **getOrderRule** () const
- void **setOrderRuleInt** (int orderRule)
- int **getOrderRuleInt** () const
- double **sumAttributesFromWaiting** (Util::identification attributeID)
- double **getAttributeFromWaitingRank** (unsigned int rank, Util::identification attributeID)

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↔IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const

- Looks up a named internal child data definition.*

 - `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.
- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.
- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)`

Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`

Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`

Enables or disables the specific trace level.

Static Public Member Functions

- `static std::string convertEnumToStr (OrderRule rule)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelDataDefinition](#)

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
- Loads a model data definition instance from serialized fields.*
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
- Creates a new instance of a data definition using the plugin registry.*
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
- Serializes a data definition instance into a persistence record.*
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
- Validates a data definition instance.*
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`
- Resets simulation-time state between replications.*
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`
- Creates internal and attached data required by this instance.*
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
- Creates related data and validates the instance in a single call.*

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_initBetweenReplications` () override
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` () override
- virtual void `_createInternalStatisticReporters` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.586.1 Detailed Description

[Queue](#) module DESCRIPTION This data module may be utilized to change the ranking rule for a specified queue. The default ranking rule for all queues is First In, First Out unless otherwise specified in this module. There is an additional field that allows the queue to be defined as shared. TYPICAL USES Stack of work waiting for a resource at a [Process](#) module Holding area for documents waiting to be collated at a [Batch](#) module Prompt Description Name The name of the queue whose characteristics are being defined. This name must be unique. Type Ranking rule for the queue, which can be based on an attribute. Types include First In, First Out; Last In, First Out; Lowest [Attribute](#) Value (first); and Highest [Attribute](#) Value (first). A low attribute value would be 0 or 1, while a high value may be 200 or 300. [Attribute](#) Name [Attribute](#) that will be evaluated for the Lowest [Attribute](#) Value or Highest [Attribute](#) Value types. Entities with lowest or highest values of the attribute will be ranked first in the queue, with ties being broken using the First In, First Out. Shared Check box that determines whether a specific queue is used in multiple places within the simulation model. Shared queues can only be used for seizing resources (for example, with the [Seize](#) module from the Advanced [Process](#) panel). Report Statistics Specifies whether or not statistics will be collected automatically and stored in the report database for this queue.

10.586.2 Member Function Documentation

10.586.2.1 **_check()**

```
bool Queue::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.586.2.2 `_createInternalStatisticReporters()`

```
void Queue::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.586.2.3 `_getParserChangesInformation()`

```
ParserChangesInformation * Queue::_getParserChangesInformation ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelData. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

10.586.2.4 `_initBetweenReplications()`

```
void Queue::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.586.2.5 `_loadInstance()`

```
bool Queue::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overriden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.586.2.6 `_saveInstance()`

```
void Queue::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.586.2.7 show()

```
std::string Queue::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- [plugins/data/DiscreteProcessing/Queue.h](#)
- [plugins/data/DiscreteProcessing/Queue.cpp](#)

10.587 QueueableItem Class Reference

Public Types

- enum class **QueueableType** : int { **QUEUE** = 0 , **SET** = 1 , **num_elements** = 2 }

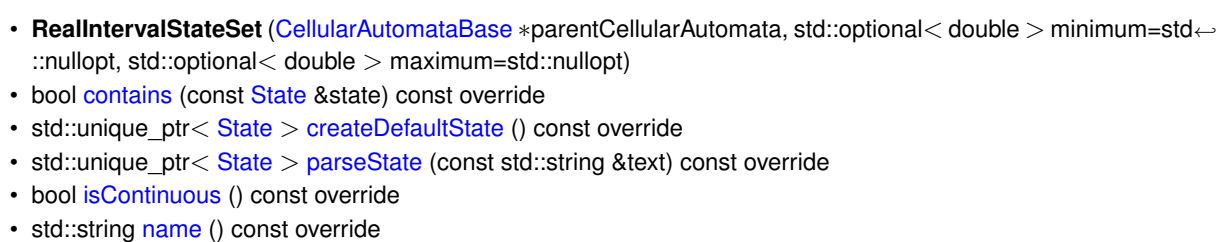
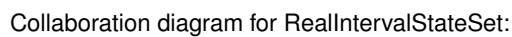
Public Member Functions

- **QueueableItem** ([ModelDataDefinition](#) *queueOrSet, QueueableItem::QueueableType queueable↔ Type=QueueableItem::QueueableType::QUEUE, std::string index="0")
- **QueueableItem** ([Model](#) *model, std::string queueName)
- bool **loadInstance** ([PersistenceRecord](#) *fields)
- void **saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues)
- std::string **show** ()
- void **setIndex** (std::string index)
- std::string **getIndex** () const
- std::string **getQueueableName** () const
- std::string **getName** () const
- void **setQueue** ([Queue](#) *resource)
- [Queue](#) * **getQueue** () const
- void **setSet** ([Set](#) *set)
- [Set](#) * **getSet** () const
- void **setQueueableType** (QueueableType queueableType)
- QueueableType **getQueueableType** () const
- [ModelDataDefinition](#) * **getQueueable** () const
- void **setElementManager** ([ModelDataManager](#) *_modeldataManager)
- [List](#)< [SimulationControl](#) * > * **getSimulationControls** () const
- void **_addSimulationControl** ([SimulationControl](#) *control)
- std::string **getTypeDC** ()

- static std::string **convertEnumToStr** (QueueableType type)

- `plugins/components/DiscreteProcessing/auxiliar/QueueableItem.h`
- `plugins/components/DiscreteProcessing/auxiliar/QueueableItem.cpp`

Inheritance diagram for RealIntervalStateSet:



Public Member Functions inherited from [StateSet](#)

- **StateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- **StateSet** (const [StateSet](#) &orig)
- virtual bool **isFinite** () const
- virtual bool **isDiscrete** () const
- virtual bool **isComposite** () const
- virtual std::vector< std::unique_ptr< [State](#) > > **enumerateStates** () const

Additional Inherited Members

Protected Attributes inherited from [StateSet](#)

- [CellularAutomataBase](#) * parentCellularAutomata

10.588.1 Member Function Documentation

10.588.1.1 contains()

```
bool RealIntervalStateSet::contains (
    const State & state ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.588.1.2 createDefaultState()

```
std::unique_ptr< State > RealIntervalStateSet::createDefaultState ( ) const [inline], [override],
[virtual]
```

Reimplemented from [StateSet](#).

10.588.1.3 isContinuous()

```
bool RealIntervalStateSet::isContinuous ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.588.1.4 name()

```
std::string RealIntervalStateSet::name ( ) const [inline], [override], [virtual]
```

Reimplemented from [StateSet](#).

10.588.1.5 parseState()

```
std::unique_ptr< State > RealIntervalStateSet::parseState (
    const std::string & text ) const [inline], [override], [virtual]
```

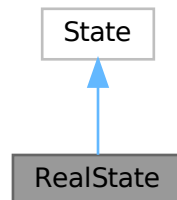
Reimplemented from [StateSet](#).

The documentation for this class was generated from the following file:

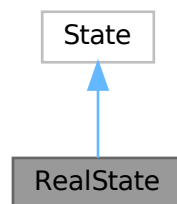
- plugins/components/ModalModel/CellularAutomata/StateSet.h

10.589 RealState Class Reference

Inheritance diagram for RealState:



Collaboration diagram for RealState:



Public Member Functions

- **RealState** (double value=0.0)
- std::unique_ptr< [State](#) > [clone](#) () const override
- bool [equals](#) (const [State](#) &other) const override
- std::string [toString](#) () const override
- std::string [typeName](#) () const override
- long [getValue](#) () const override
- void [setValue](#) (long value) override
- double **realValue** () const
- void **setRealValue** (double value)

Public Member Functions inherited from [State](#)

- **State** (long value)
- **State** (const [State](#) &orig)
- virtual std::string **show** () const
- template<typename T >
const T * **as** () const noexcept
- template<typename T >
T * **as** () noexcept

Additional Inherited Members

Protected Attributes inherited from [State](#)

- long **_value**

10.589.1 Member Function Documentation

10.589.1.1 clone()

```
std::unique_ptr< State > RealState::clone ( ) const [inline], [override], [virtual]
```

Reimplemented from [State](#).

10.589.1.2 equals()

```
bool RealState::equals (
    const State & other ) const [inline], [override], [virtual]
```

Reimplemented from [State](#).

10.589.1.3 getValue()

```
long RealState::getValue ( ) const [inline], [override], [virtual]
```

Reimplemented from [State](#).

10.589.1.4 setValue()

```
void RealState::setValue (
    long value ) [inline], [override], [virtual]
```

Reimplemented from [State](#).

10.589.1.5 toString()

```
std::string RealState::toString ( ) const [inline], [override], [virtual]
```

Reimplemented from [State](#).

10.589.1.6 typeName()

```
std::string RealState::typeName ( ) const [inline], [override], [virtual]
```

Reimplemented from [State](#).

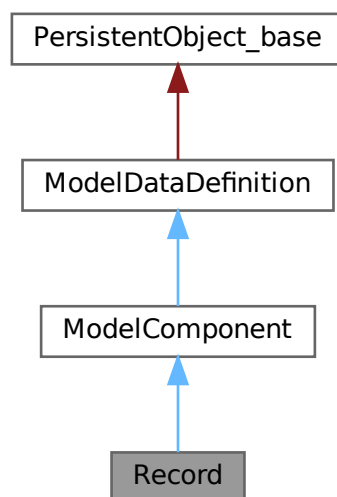
The documentation for this class was generated from the following file:

- plugins/components/ModalModel/CellularAutomata/State.h

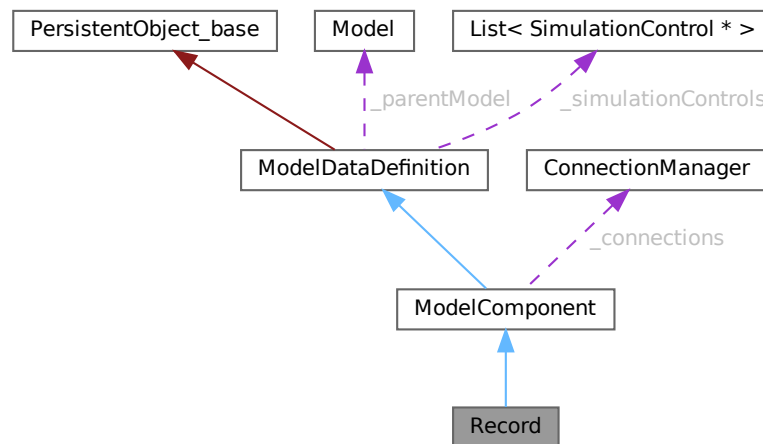
10.590 Record Class Reference

```
#include <plugins/components/InputOutput/Record.h>
```

Inheritance diagram for Record:



Collaboration diagram for Record:



Public Member Functions

- **Record** ([Model](#) *model, std::string name="")
- void **setFilename** (std::string filename)
- std::string **getFileName** () const
- void **setExpression** (const std::string expression)
- std::string **getExpression** () const
- void [setExpressionName](#) (std::string expressionName)
- std::string **getExpressionName** () const
- void [setDatasetName](#) (std::string datasetName)
- std::string **getDatasetName** () const
- void [setRandomVariableName](#) (std::string randomVariableName)
- std::string **getRandomVariableName** () const
- void [setVariableType](#) (std::string variableType)
- std::string **getVariableType** () const
- void [setDatasetDescription](#) (std::string description)
- std::string **getDatasetDescription** () const
- [StatisticsCollector](#) * **getCstatExpression** () const
- bool **getTimeDependent** () const
- void **setTimeDependent** (bool timeDependent)
- virtual std::string [show](#) () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const

- new public user methods for this component*
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↔IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

- Sets the model nesting level for this definition.*
 • `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation ()`
- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `bool Check (ModelComponent *component)`
Validates a component instance.
- static `void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- static `void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- static `void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- static `void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- static `bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- static `void InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- static `void CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- static `bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void `_onDispatchEvent` (`Entity` *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool `_loadInstance` (`PersistenceRecord` *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` (`PersistenceRecord` *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void `_initBetweenReplications` () override
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_createInternalStatisticReporters` () override

Protected Member Functions inherited from `ModelDataDefinition`

- bool `_getSaveDefaultsOption` ()
- virtual `ParserChangesInformation` * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` (`SimulationControl` *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, `ModelDataDefinition` *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, `ModelDataDefinition` *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()

- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.590.1 Detailed Description

[Record](#) module DESCRIPTION This module is used to collect statistics in the simulation model. Various types of observational statistics are available, including time between exits through the module, entity statistics (such as time or costing), general observations, and interval statistics (from some time stamp to the current simulation time). A count type of statistic is available as well. Tally and [Counter](#) sets can also be specified. TYPICAL USES Collect the number of jobs completed each hour Count how many orders have been late being fulfilled [Record](#) the time spent by priority customers in the main check-out line PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Type Type of observational (tally) or count statistic to be generated. Count will increase or decrease the value of the named statistic by the specified value. [Entity](#) Statistics will generate general entity statistics, such as time and costing/duration information. Time Interval will calculate and record the difference between a specified attribute's value and current simulation time. Time Between will track and record the time between entities entering the module. Expression will record the value of the specified expression. [Attribute](#) Name Name of the attribute whose value will be used for the interval statistics. Applies only when Type is Interval. Value Value that will be recorded to the observational statistic when Type is Expression or added to the counter when Type is Count. Tally Name This field defines the symbol name of the tally into which the observation is to be recorded. Applies only when Type is Time Interval, Time Between, or Expression. [Counter](#) This field defines the symbol name of the counter to Name increment/decrement. Applies only when Type is [Counter](#). [Record](#) into [Set](#) Check box to specify whether or not a tally or counter set will be used. Tally [Set](#) Name Name of the tally set that will be used to record the observational-type statistic. Applies only when Type is Time Interval, Time Between, or Expression. [Counter Set](#) Name Name of the counter set that will be used to record the count-type statistic. Applies only when Type is Count. [Set](#) Index Index into the tally or counter set.

10.590.2 Member Function Documentation

10.590.2.1 `_check()`

```
bool Record::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.590.2.2 `_createInternalStatisticReporters()`

```
void Record::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.590.2.3 `_initBetweenReplications()`

```
void Record::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.590.2.4 `_loadInstance()`

```
bool Record::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.590.2.5 `_onDispatchEvent()`

```
void Record::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.590.2.6 _saveInstance()

```
void Record::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.590.2.7 setDatasetDescription()

```
void Record::setDatasetDescription (
    std::string description )
```

Sets the free-text dataset description emitted in enriched [Record](#) file headers.

10.590.2.8 setDatasetName()

```
void Record::setDatasetName (
    std::string datasetName )
```

Sets the analytical dataset name emitted in enriched [Record](#) file headers.

10.590.2.9 setExpressionName()

```
void Record::setExpressionName (
    std::string expressionName )
```

Sets the legacy expression-name metadata kept for old [Record](#) datasets.

10.590.2.10 setRandomVariableName()

```
void Record::setRandomVariableName (
    std::string randomVariableName )
```

Sets the random variable name emitted in enriched [Record](#) file headers.

10.590.2.11 setVariableType()

```
void Record::setVariableType (
    std::string variableType )
```

Sets the canonical random variable type for enriched [Record](#) file headers.

10.590.2.12 show()

```
std::string Record::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

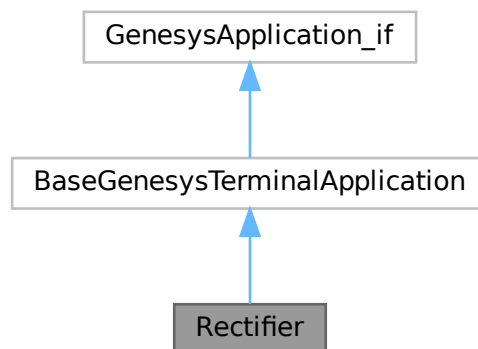
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

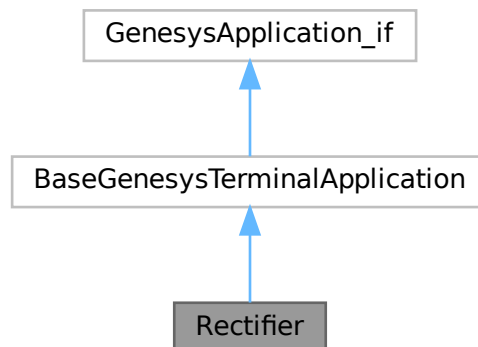
- plugins/components/InOutOutput/Record.h
- plugins/components/InOutOutput/Record.cpp

10.591 Rectifier Class Reference

Inheritance diagram for Rectifier:



Collaboration diagram for Rectifier:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.591.1 Member Function Documentation

10.591.1.1 main()

```
int Rectifier::main (
    int argc,
    char ** argv )  [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

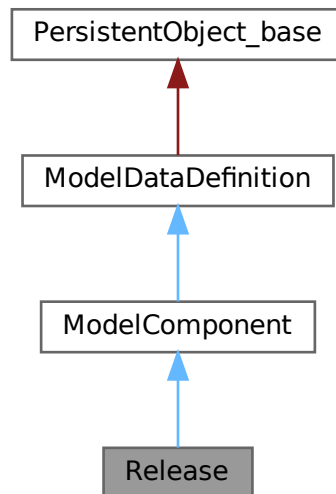
The documentation for this class was generated from the following files:

- applications/modelSpecific/teaching/Rectifier.h
- applications/modelSpecific/teaching/Rectifier.cpp

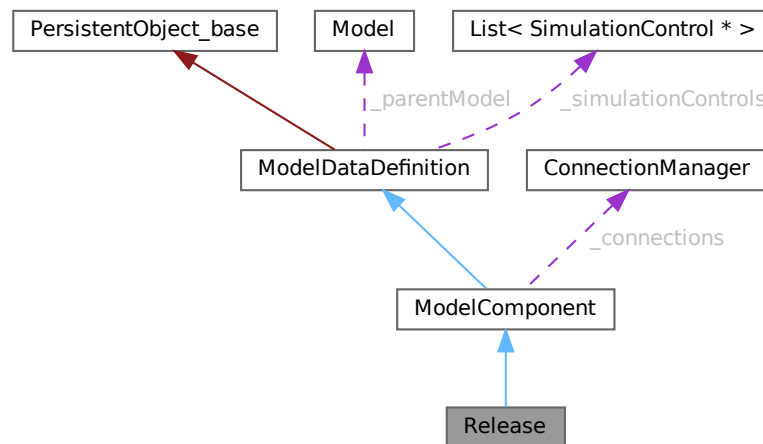
10.592 Release Class Reference

```
#include <plugins/components/DiscreteProcessing/Release.h>
```

Inheritance diagram for Release:



Collaboration diagram for Release:



Public Member Functions

- **Release** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
virtual public methods
- void **setPriority** (unsigned short _priority)
- unsigned short **priority** () const
- [List](#)< [SeizableItem](#) * > * **getReleaseRequests** () const
- void **addReleaseRequests** ([SeizableItem](#) *newRequest)
- void **removeReleaseRequests** ([SeizableItem](#) *request)

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_initBetweenReplications** () override
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_createEditableDataDefinitions** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.592.1 Detailed Description

[Release](#) module DESCRIPTION The [Release](#) module is used to release units of a resource that an entity previously has seized. This module may be used to release individual resources or may be used to release resources within a set. For each resource to be released, the name and quantity to release are specified. When the entity enters the [Release](#) module, it gives up control of the specified resource(s). Any entities waiting in queues for those resources will gain control of the resources immediately. TYPICAL USES Finishing a customer order (release the operator) Completing a tax return (release the accountant) Leaving the hospital (release the doctor, nurse, hospital room) PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Type Type of resource for releasing, either specifying a particular resource, or selecting from a pool of resources (that is, a resource set). The resource name may also be specified by an expression or attribute value. [Resource](#) Name Name of the resource that will be released. [Set](#) Name Name of the resource set from which a member will be released. [Attribute](#) Name Name of the attribute that specifies the resource name to be released. Expression Name of the expression that specifies the name of the resource to be released. Quantity Number of resources of a given name or from a given set that will be released. For sets, this value specifies only the number of a selected resource that will be released (based on the resource's capacity), not the number of members to be released within the set. [Release](#) Rule Method of determining which resource within a set to release. Last Member Seized and First Member Seized will release the last/first member from within the set that was seized. Specific member indicates that a member number or attribute (with a member number value) will be used to specify the member to release. [Set](#) Index Member index of the resource set that the entity will release.

10.592.2 Member Function Documentation

10.592.2.1 `_check()`

```
bool Release::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.592.2.2 `_createEditableDataDefinitions()`

```
void Release::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.592.2.3 `_initBetweenReplications()`

```
void Release::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.592.2.4 `_loadInstance()`

```
bool Release::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.592.2.5 `_onDispatchEvent()`

```
void Release::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.592.2.6 `_saveInstance()`

```
void Release::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.592.2.7 show()

```
std::string Release::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

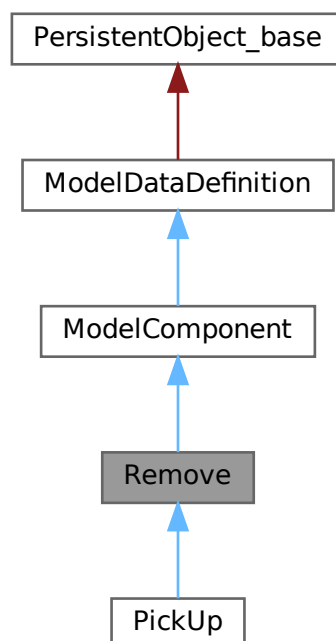
The documentation for this class was generated from the following files:

- plugins/components/DiscreteProcessing/Release.h
- plugins/components/DiscreteProcessing/Release.cpp

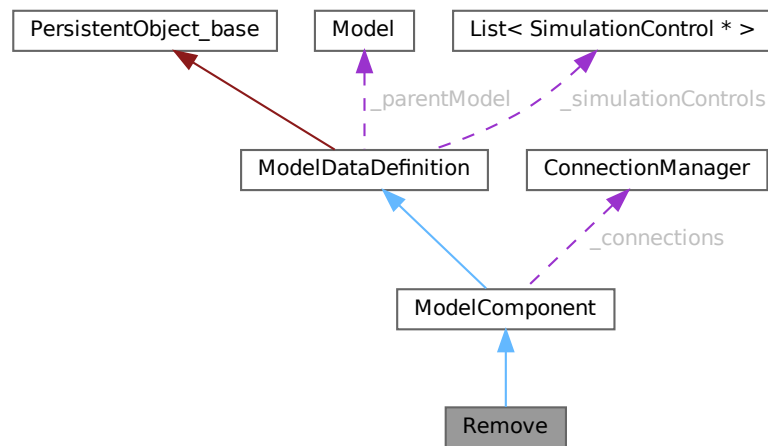
10.593 Remove Class Reference

```
#include <plugins/components/Decisions/Remove.h>
```

Inheritance diagram for Remove:



Collaboration diagram for Remove:



Public Types

- enum class **RemoveFromType** : int { **QUEUE** = 0 , **ENTITYGROUP** = 1 , **num_elements** = 2 }

Public Member Functions

- Remove** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- virtual public methods*
- void **setRemoveStartRank** (std::string _removeFromRank)
- std::string **getRemoveStartRank** () const
- void **setRemoveEndRank** (std::string _removeEndRank)
- std::string **getRemoveEndRank** () const
- void **setRemoveFromType** (Remove::RemoveFromType _removeFromType)
- Remove::RemoveFromType **getRemoveFromType** () const
- void **setRemoveFrom** ([ModelDataDefinition](#) *_removeFrom)
- [ModelDataDefinition](#) * **getRemoveFrom** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- constructors*
- virtual ~**ModelComponent** ()
- Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- Connects this component to another component.*
- bool **hasBreakpointAt** ()

- Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
 IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager](#)::Level [getTraceLevelSpecific](#) () const

- Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) (TraceManager::Level level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelspecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static std::string [convertEnumToStr](#) (RemoveFromType type)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void `_onDispatchEvent` ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_createEditableDataDefinitions` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()

- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.593.1 Detailed Description

[Remove](#) module DESCRIPTION The [Remove](#) module removes a single entity from a specified position in a queue and sends it to a designated module. When an entity arrives at a [Remove](#) module, it removes the entity from the specified queue and sends it to the connected module. The rank of the entity signifies the location of the entity within the queue. The entity that caused the removal proceeds to the next module specified and is processed before the removed entity. TYPICAL USES Removing an order from a queue that is due to be completed next Calling a patient from a waiting room for an examination Retrieving the next order to be processed from a pile of documents Prompt Description Name Unique module identifier displayed on the module shape. [Queue](#) Name Name of the queue from which the entity will be removed. Rank of [Entity](#) Rank of the entity to remove from within the queue.

10.593.2 Member Function Documentation

10.593.2.1 `_check()`

```
bool Remove::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.593.2.2 `_createEditableDataDefinitions()`

```
void Remove::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

10.593.2.3 `_loadInstance()`

```
bool Remove::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.593.2.4 `_onDispatchEvent()`

```
void Remove::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.593.2.5 `_saveInstance()`

```
void Remove::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.593.2.6 `show()`

```
std::string Remove::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

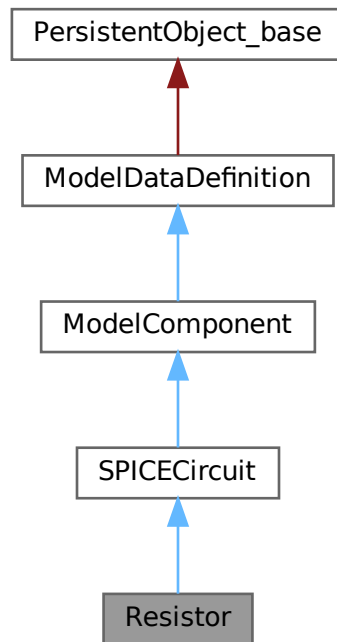
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

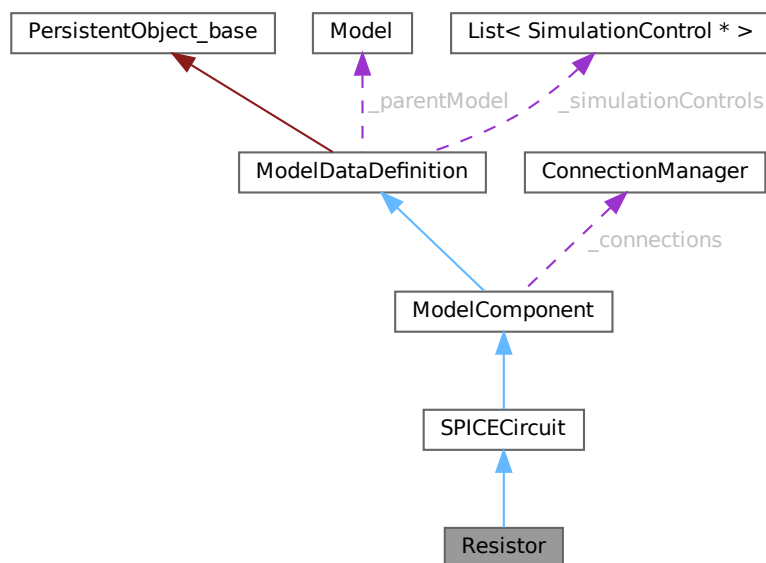
- `plugins/components/Decisions/Remove.h`
- `plugins/components/Decisions/Remove.cpp`

10.594 Resistor Class Reference

Inheritance diagram for Resistor:



Collaboration diagram for Resistor:



Public Member Functions

- **Resistor** ([Model](#) *model, std::string name="")
- void [Build](#) () override
- void **setResistance** (float resistance)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string [show](#) () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)

- Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const
- Returns the runtime class name used by the plugin system.*
- [Model](#) * [getParentModel](#) () const
- Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const
- Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)
- Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
- Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
- Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
- Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const
- Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))
- Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const
- Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int _modelLevel)
- Sets the model nesting level for this definition.*
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
- Returns the simulation controls owned by this data definition.*
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
- Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
- Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)
- Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const
- Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
- Enables or disables the specific trace level.*

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split_char)
- Externalize function GetPluginInformation to be accessible through dynamic linked library.*

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void **traceReport** (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.594.1 Member Function Documentation

10.594.1.1 `_loadInstance()`

```
bool Resistor::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.594.1.2 `_saveInstance()`

```
void Resistor::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.594.1.3 Build()

```
void Resistor::Build ( ) [override], [virtual]
```

Reimplemented from [SPICECircuit](#).

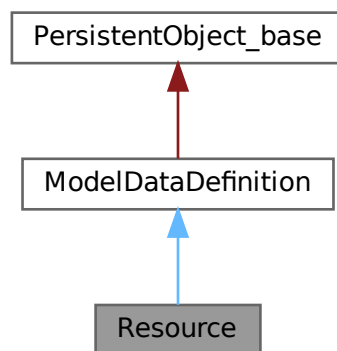
The documentation for this class was generated from the following files:

- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

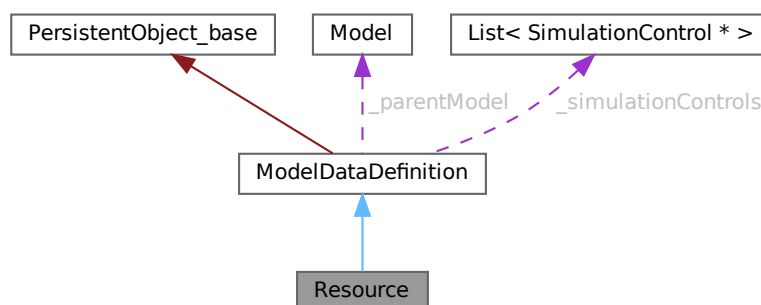
10.595 Resource Class Reference

```
#include <plugins/data/DiscreteProcessing/Resource.h>
```

Inheritance diagram for Resource:



Collaboration diagram for Resource:



Public Types

- enum class **ResourceState** : int {
IDLE = 0 , **BUSY** = 1 , **FAILED** = 2 , **INACTIVE** = 3 ,
OTHER = 4 , **num_elements** = 5 }
- typedef std::function< void([Resource](#) *) > **ResourceEventHandler**
- typedef std::pair< std::pair< ResourceEventHandler, [ModelComponent](#) * >, unsigned int > **SortedResourceEventHandler**

Public Member Functions

- [Resource](#) ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- bool **seize** (unsigned int quantity, double priority=0)
- void **release** (unsigned int quantity)
- void **insertFailure** ([Failure](#) *failure)
- void **removeFailure** ([Failure](#) *failure)
- double **getInstantCapacityUtilization** () const
- double **getCapacityUtilization** () const
- double **getSeizedUtilization** () const
- double **getLastTimeSeized** () const
- void **addReleaseResourceEventHandler** (ResourceEventHandler eventHandler, [ModelComponent](#) *component, unsigned int priority)
- void **setResourceState** (ResourceState _resourceState)
- Resource::ResourceState **getResourceState** () const
- void **setCapacity** (unsigned int capacity)
- unsigned int **getCapacity** () const
- void **setCostBusyTimeUnit** (double _costBusyTimeUnit)
- double **getCostBusyTimeUnit** () const
- void **setCostIdleTimeUnit** (double _costIdleTimeUnit)
- double **getCostIdleTimeUnit** () const
- void **setCostPerUse** (double _costPerUse)
- double **getCostPerUse** () const
- void **setCapacitySchedule** ([Schedule](#) *_capacitySchedule)
- [Schedule](#) * **getCapacitySchedule** () const
- unsigned int **getNumberBusy** () const

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insertIntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

- Updates the user-visible label of this instance.*

 - `const std::string & getLabel () const`

Returns the user-visible label of this instance.
- `const std::string & getClassname () const`

Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`

Returns the parent model that owns this definition.
- `bool isReportStatistics () const`

Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`

Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`

Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`

Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`

Returns the full map of attached data definitions.
- `bool hasChanged () const`

Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`

Updates the changed flag for this data definition.
- `unsigned int getLevel () const`

Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`

Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`

Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`

Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`

Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`

Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`

Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`

Enables or disables the specific trace level.

Static Public Member Functions

- `template<typename Class >`
`static ResourceEventHandler SetResourceEventHandler (void(Class::*function)(Resource *), Class *object)`
- `static std::string convertEnumToStr (ResourceState state)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Friends

- class **Failure**
- class **ResourceTestProbe**

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.595.1 Detailed Description

Resource module DESCRIPTION This data module defines the resources in the simulation system, including costing information and resource availability. Resources may have a fixed capacity that does not vary over the simulation run or may operate based on a schedule. **Resource** failures and states can also be specified in this module. TYPICAL USES Equipment (machinery, cash register, phone line) People (clerical, order processing, sales clerks, operators) PROMPTS Prompt Description Name The name of the resource whose characteristics are being defined. This name must be unique. Type Method for determining the capacity for a resource. Fixed Capacity will not change during the simulation run. Based on **Schedule** signifies that a **Schedule** module is used to specify the capacity and duration information for the resource. Capacity Number of resource units of a given name that are available to the system for processing. Applies only when Type is Fixed Capacity. **Schedule** Name Identifies the name of the schedule to be used by the resource. The schedule defines the capacity of a resource for a given period of time. Applies only when type is **Schedule**. **Schedule** Rule Dictates when the actual capacity change is to occur when a decrease in capacity is required for a busy resource unit. Applies only when Type is **Schedule**. Busy/TimeUnit Cost per timeUnit of a resource that is processing an entity. The resource becomes busy when it is originally allocated to an entity and becomes idle when it is released. During the time when it is busy, cost will accumulate based on the busy/timeUnit cost. The busy cost per timeUnit is automatically converted to the appropriate base time unit specified within the Replication Parameters page of the Run > Setup menu item. Idle/TimeUnit Cost per timeUnit of a resource that is idle. The resource is idle while it is not processing an entity. During the time when it is idle, cost will accumulate based on the idle/timeUnit cost. The idle cost per timeUnit is automatically converted to the appropriate base time unit specified within the Replication Parameters page of the Run > Setup menu item. Per Use Cost of a resource on a usage basis, regardless of the time for which it is used. Each time the resource is allocated to an entity, it will incur a per-use cost. **StateSet** Name Name of states that the resource may be assigned during the simulation run. Initial **State** Initial state of a resource. If specified, the name must be defined within the repeat group of state names. This field is shown only when a **StateSet** Name is defined. Failures Lists all failures that will be associated with the resource. **Failure** Name—Name of the failure associated with the resource. **Failure** Rule—Behavior that should occur when a failure is to occur for a busy resource unit. Report Statistics Specifies whether or not statistics will be collected automatically and stored in the report database for this resource.

10.595.2 Constructor & Destructor Documentation

10.595.2.1 Resource()

```
Resource::Resource (
    Model * model,
    std::string name = "" )
```

Handlers are sorted by priority

10.595.3 Member Function Documentation

10.595.3.1 _check()

```
bool Resource::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Validate resource basic semantic constraints.

Reimplemented from [ModelDataDefinition](#).

10.595.3.2 _createAttachedAttributes()

```
void Resource::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.595.3.3 _initBetweenReplications()

```
void Resource::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.595.3.4 _loadInstance()

```
bool Resource::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.595.3.5 _saveInstance()

```
void Resource::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.595.3.6 show()

```
std::string Resource::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

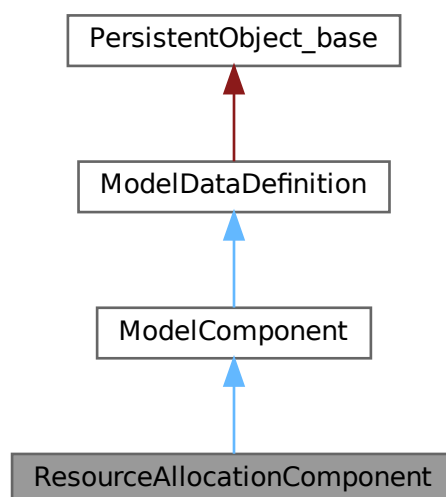
The documentation for this class was generated from the following files:

- `plugins/data/DiscreteProcessing/Resource.h`
- `plugins/data/DiscreteProcessing/Resource.cpp`

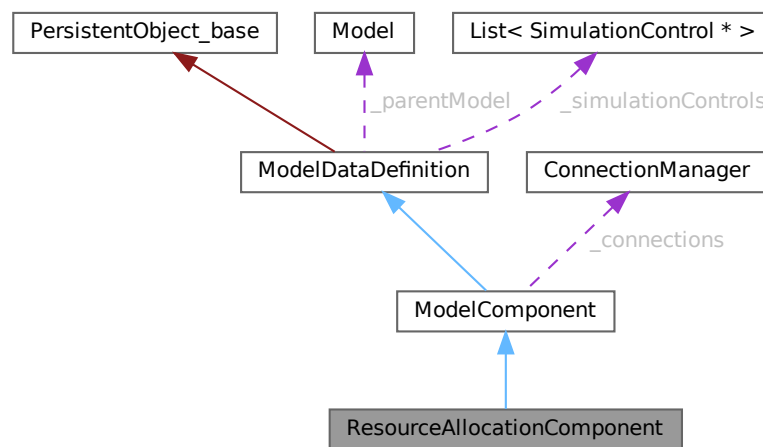
10.596 ResourceAllocationComponent Class Reference

```
#include <plugins/components/WholeCellModeling/ResourceAllocationComponent.h>
```

Inheritance diagram for ResourceAllocationComponent:



Collaboration diagram for ResourceAllocationComponent:



Public Member Functions

- **ResourceAllocationComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setRnapCountKey** (std::string key)
- std::string **getRnapCountKey** () const
- void **setRibosomeCountKey** (std::string key)
- std::string **getRibosomeCountKey** () const
- void **setMRNASpeciesPrefix** (std::string prefix)
- std::string **getMRNASpeciesPrefix** () const
- void **setProteinSpeciesPrefix** (std::string prefix)
- std::string **getProteinSpeciesPrefix** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()

Releases resources owned by this data definition.
- Util::identification [getId](#) () const

Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)

Updates the logical name of this instance.
- const std::string & [getName](#) () const

Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const

Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const

Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const

Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const

Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const

Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const

Returns the full map of attached data definitions.
- bool [hasChanged](#) () const

Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))

Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const

Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const

Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)

Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_initBetweenReplications** () override
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void **_createEditableDataDefinitions** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.596.1 Detailed Description

Fair allocation of molecular machines (RNAP, ribosomes) before each SSA step.

Implements the "ResourceAllocation" contract from Karr et al. (2012): available RNAP and ribosome counts are divided proportionally across active molecular species, then written to [WholeCellState::setResourceBudget\(\)](#) so [StochasticTranscription](#) and [StochasticTranslation](#) can honour the budget.

Allocation logic:

- Count `N_mRNA` = number of keys in [WholeCellState](#) molecule counts whose name starts with `mRNA_` (default "mRNA_").
- Count `N_protein` = number of keys starting with `proteinSpeciesPrefix` (default "protein_").
- RNAP budget per mRNA gene = `floor(RNAP_free / max(N_mRNA, 1))`
- Ribosome budget per protein = `floor(ribosome_free / max(N_mRNA, 1))` (per active mRNA transcript, not per protein species)
- [Write](#) per-species budget to [WholeCellState::setResourceBudget\(\)](#).

This component must fire BEFORE [StochasticTranscription](#) and [StochasticTranslation](#) in the simulation pipeline.

10.596.2 Member Function Documentation

10.596.2.1 `_check()`

```
bool ResourceAllocationComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.596.2.2 `_createEditableDataDefinitions()`

```
void ResourceAllocationComponent::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.596.2.3 `_initBetweenReplications()`

```
void ResourceAllocationComponent::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.596.2.4 `_loadInstance()`

```
bool ResourceAllocationComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.596.2.5 `_onDispatchEvent()`

```
void ResourceAllocationComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.596.2.6 `_saveInstance()`

```
void ResourceAllocationComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.596.2.7 show()

```
std::string ResourceAllocationComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

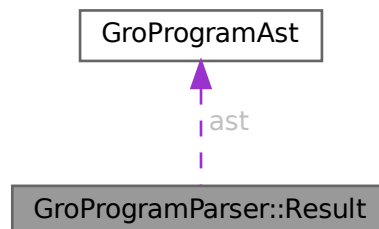
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/ResourceAllocationComponent.h
- plugins/components/WholeCellModeling/ResourceAllocationComponent.cpp

10.597 GroProgramParser::Result Struct Reference

Collaboration diagram for GroProgramParser::Result:



Public Attributes

- bool **accepted** = false
- std::string **errorMessage** = ""
- [GroProgramAst](#) **ast**

The documentation for this struct was generated from the following file:

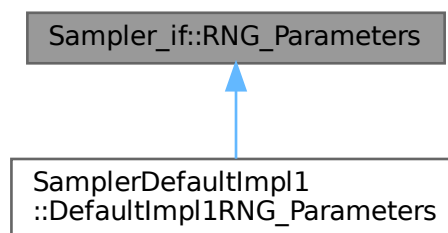
- plugins/data/BiochemicalSimulation/GroProgramParser.h

10.598 Sampler_if::RNG_Parameters Struct Reference

Encapsulates generator-specific configuration/state parameters.

```
#include <kernel/statistics/Sampler_if.h>
```

Inheritance diagram for Sampler_if::RNG_Parameters:



10.598.1 Detailed Description

Encapsulates generator-specific configuration/state parameters.

Implementations may derive this struct to expose seed, multiplier and other algorithm-specific controls required by a concrete RNG strategy.

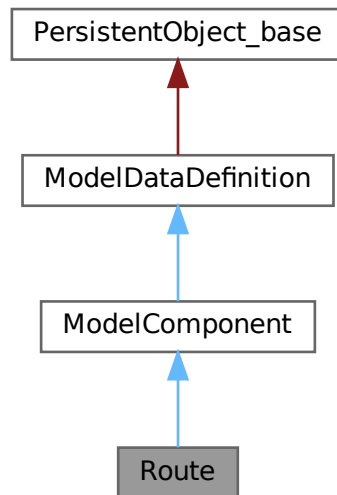
The documentation for this struct was generated from the following file:

- kernel/statistics/Sampler_if.h

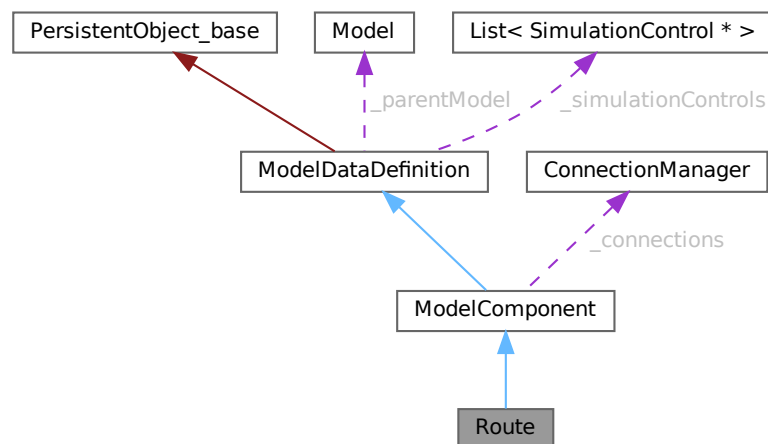
10.599 Route Class Reference

```
#include <plugins/components/MaterialHandling/Route.h>
```

Inheritance diagram for Route:



Collaboration diagram for Route:



Public Types

- enum class **DestinationType** : int { **Station** = 0 , **Sequence** = 1 , **Label** = 2 , **num_elements** = 3 }

Public Member Functions

- **Route** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- virtual public methods*
- void **setStationName** (std::string stationName)
- void **setStation** ([Station](#) *_station)
- [Station](#) * **getStation** () const
- void **setStationExpression** (std::string _stationExpression)
- std::string **getStationExpression** () const
- void **setRouteTimeExpression** (std::string _routeTimeExpression)
- void **setRouteTimeExpression** (std::string _routeTimeExpression, [Util::TimeUnit](#) _routeTimeTimeUnit)
- std::string **getRouteTimeExpression** () const
- void **setRouteTimeTimeUnit** ([Util::TimeUnit](#) _routeTimeTimeUnit)
- [Util::TimeUnit](#) **getRouteTimeTimeUnit** () const
- void **setRouteDestinationType** (DestinationType _routeDestinationType)
- Route::DestinationType **getRouteDestinationType** () const
- void **setLabel** ([Label](#) *_label)
- [Label](#) * **getLabel** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
- constructors*
- virtual ~**ModelComponent** ()
- Releases component-owned resources and unregisters it from the model.*
- [ConnectionManager](#) * **getConnectionManager** () const
- new public user methods for this component*
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
- Connects this component to another component.*
- bool **hasBreakpointAt** ()
- Indicates whether a simulation breakpoint is set on this component.*
- void **setDescription** (std::string _description)
- Updates the free-form description shown by the GUI.*
- std::string **getDescription** () const
- Returns the free-form description associated with this component.*
- void **remove** ([ModelComponent](#) *comp)
- Removes a connected component from this component.*

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- Releases resources owned by this data definition.*
- Util::identification **getId** () const
- Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & **getName** () const

- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static std::string [convertEnumToStr](#) (DestinationType type)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createInternalStatisticReporters](#) () override
- virtual void [_createEditableDataDefinitions](#) () override
- virtual void [_createAttachedAttributes](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- [bool](#) [_getSaveDefaultsOption](#) ()
- [virtual](#) [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- [virtual](#) [void](#) [_initBetweenReplications](#) ()
- [virtual](#) [void](#) [_addSimulationControl](#) ([SimulationControl](#) *control)
- [virtual](#) [void](#) [_createInternalAndAttachedData](#) ()
- [virtual](#) [void](#) [_createNonEditableDataDefinitions](#) ()
- [void](#) [_templateCreateInternalStatisticReporters](#) ()
- [void](#) [_templateCreateAttachedAttributes](#) ()
- [void](#) [_templateCreateNonEditableDataDefinitions](#) ()
- [void](#) [_templateCreateEditableDataDefinitions](#) ()
- [void](#) [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- [void](#) [_internalDataClear](#) ()
- [void](#) [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- [void](#) [_internalDataRemove](#) (const std::string &key)
- [void](#) [_attachedDataClear](#) ()
- [void](#) [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_attachedDataRemove](#) (const std::string &key)
- [void](#) [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- [void](#) [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_statisticReporterRemove](#) (const std::string &key)
- [void](#) [_statisticReportersClear](#) ()
- [void](#) [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- [void](#) [_mandatoryAttachedAttributesClear](#) ()
- [void](#) [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- [void](#) [_mandatoryEditableDataDefinitionsClear](#) ()
- [void](#) [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- [void](#) [_optionalEditableDataDefinitionsClear](#) ()
- [void](#) [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- [void](#) [_mandatoryNonEditableDataDefinitionsClear](#) ()
- [void](#) [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- [void](#) [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- [void](#) [_optionalNonEditableDataDefinitionsClear](#) ()
- [void](#) [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- [void](#) [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- [void](#) [traceError](#) (const std::string &text, const std::exception &e)
- [void](#) [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- [void](#) [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- [void](#) [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- [void](#) [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- [void](#) [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.599.1 Detailed Description

[Route](#) module DESCRIPTION The [Route](#) module transfers an entity to a specified station or the next station in the station visitation sequence defined for the entity. A delay time to transfer to the next station may be defined. When an entity enters the [Route](#) module, its [Station](#) attribute (Entity.Station) is set to the destination station. The entity is then sent to the destination station, using the route time specified. If the station destination is entered as By [Sequence](#), the next station is determined by the entity's sequence and step within the set (defined by special-purpose attributes Entity.Sequence and Entity.Jobstep, respectively). TYPICAL USES Send a part to its next processing station based on its routing slip Send an account balance call to an account agent Send restaurant customers to a specific table PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart. [Route](#) Time Travel time from the entity's current location to the destination station. Units Time units for route-time parameters. Destination Type Method for determining the entity destination location. Selection of By [Sequence](#) requires that the entity has been assigned a sequence name and that the sequence itself has been defined. [Station](#) Name Name of the individual destination station. [Attribute](#) Name Name of the attribute that stores the station name to which entities will route. Expression Expression that is evaluated to the station name where entities will route.

10.599.2 Member Function Documentation

10.599.2.1 `_check()`

```
bool Route::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.599.2.2 `_createAttachedAttributes()`

```
void Route::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.599.2.3 `_createEditableDataDefinitions()`

```
void Route::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.599.2.4 `_createInternalStatisticReporters()`

```
void Route::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.599.2.5 `_loadInstance()`

```
bool Route::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.599.2.6 `_onDispatchEvent()`

```
void Route::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.599.2.7 `_saveInstance()`

```
void Route::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.599.2.8 show()

```
std::string Route::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

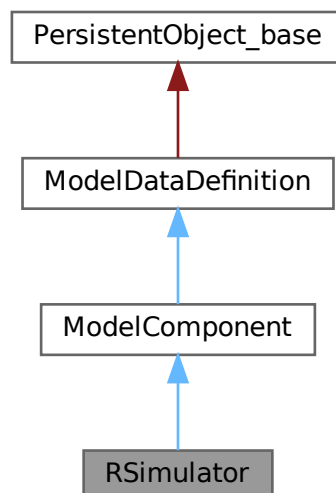
- plugins/components/MaterialHandling/Route.h
- plugins/components/MaterialHandling/Route.cpp

10.600 RSimulator Class Reference

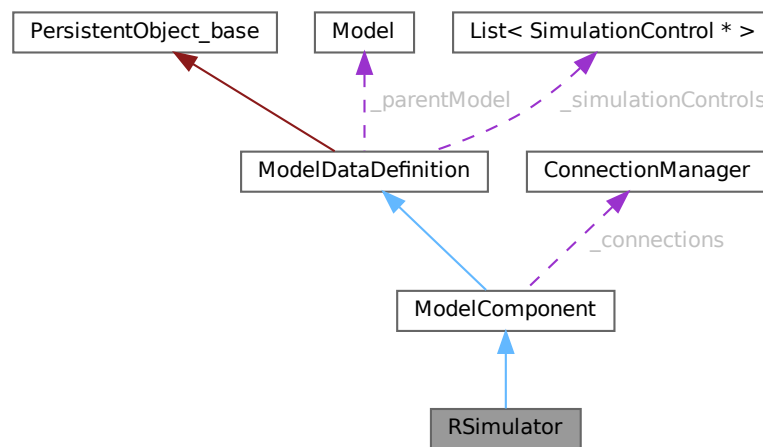
Component that sends configured R commands to an internal [RSimulatorRunner](#).

```
#include <plugins/components/ExternalIntegration/RSimulator.h>
```

Inheritance diagram for RSimulator:



Collaboration diagram for RSimulator:



Public Member Functions

- **RSimulator** ([Model](#) *model, std::string name="")
Creates an [RSimulator](#) component attached to a model.
- virtual ~**RSimulator** ()=default
Destroys the [RSimulator](#) component.
- void **setRExecutable** (std::string rExecutable)
Sets the Rscript executable name or path used by the internal runner.
- std::string **getRExecutable** () const
Returns the Rscript executable name or path used by the internal runner.
- void **setWorkingDirectory** (std::string workingDirectory)
Sets the working directory used by generated R scripts.
- std::string **getWorkingDirectory** () const
Returns the working directory used by generated R scripts.
- void **setPreludeScript** (std::string preludeScript)
Sets optional R code prepended to every command.
- std::string **getPreludeScript** () const
Returns optional R code prepended to every command.
- void **insertCommand** (std::string command)
Appends one R command to the execution list.
- void **setCommands** (std::list< std::string > commands)
Replaces the command list with the provided commands.
- [List](#)< std::string > * **getCommands** () const
Returns the command list used when entities arrive.
- void **setCommandList** (std::string commandList)
Replaces the command list using one command per non-empty line.
- std::string **getCommandList** () const
Returns commands serialized as one command per line for GUI editing.
- [RSimulatorRunner](#) * **getRunner** ()
Returns the internal runner, creating it first if necessary.
- virtual std::string **show** () override
Returns a compact human-readable description of the component.

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionType, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.

- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- List< `SimulationControl` * > * `getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelspecific`, bool `traceLevelSpecificEnabled`=true)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation` * `GetPluginInformation` ()
Returns plugin metadata for the `RSimulator` component.
- static `ModelComponent` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`)
Loads an `RSimulator` component from persistence.
- static `ModelDataDefinition` * `NewInstance` (`Model` *`model`, std::string `name`="")
Creates a new `RSimulator` component instance.

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent` *`component`)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent` *`component`)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord` *`fields`, `ModelComponent` *`component`)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event` *`event`)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Executes R commands for the arriving entity and forwards it.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
Loads persisted [RSimulator](#) configuration.
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves [RSimulator](#) configuration.
- virtual bool [_check](#) (std::string &errorMessage) override
Validates the executable, working directory and command list.
- virtual void [_createEditableDataDefinitions](#) () override
Creates or updates the internal [RSimulatorRunner](#).

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.600.1 Detailed Description

Component that sends configured R commands to an internal [RSimulatorRunner](#).

Each arriving entity triggers the configured command list. Commands are executed sequentially by the internal [RSimulatorRunner](#), and the first version exposes results through simulation trace messages before forwarding the entity.

10.600.2 Member Function Documentation

10.600.2.1 `_check()`

```
bool RSimulator::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

Validates the executable, working directory and command list.

Reimplemented from [ModelDataDefinition](#).

10.600.2.2 `_createEditableDataDefinitions()`

```
void RSimulator::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Creates or updates the internal [RSimulatorRunner](#).

Reimplemented from [ModelDataDefinition](#).

10.600.2.3 `_loadInstance()`

```
bool RSimulator::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

Loads persisted [RSimulator](#) configuration.

Reimplemented from [ModelComponent](#).

10.600.2.4 `_onDispatchEvent()`

```
void RSimulator::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Executes R commands for the arriving entity and forwards it.

Implements [ModelComponent](#).

10.600.2.5 `_saveInstance()`

```
void RSimulator::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves [RSimulator](#) configuration.

Reimplemented from [ModelComponent](#).

10.600.2.6 `show()`

```
std::string RSimulator::show ( ) [override], [virtual]
```

Returns a compact human-readable description of the component.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

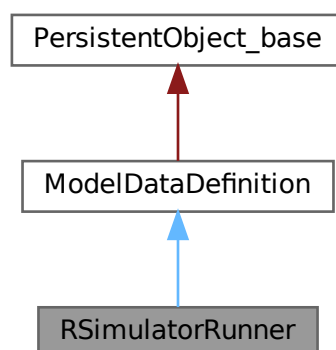
- `plugins/components/ExternalIntegration/RSimulator.h`
- `plugins/components/ExternalIntegration/RSimulator.cpp`

10.601 RSimulatorRunner Class Reference

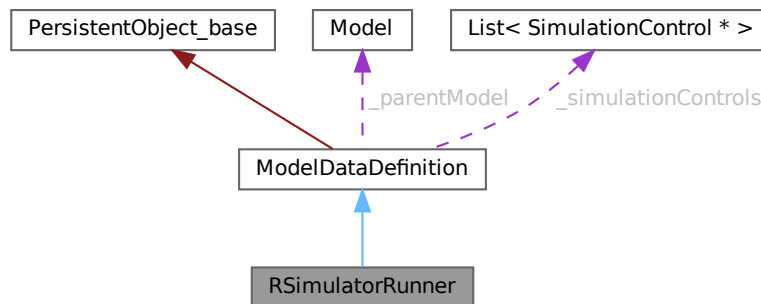
[Model](#) data definition that executes R code through Rscript.

```
#include <plugins/data/ExternalIntegration/RSimulatorRunner.h>
```

Inheritance diagram for RSimulatorRunner:



Collaboration diagram for RSimulatorRunner:



Public Member Functions

- **RSimulatorRunner** ([Model](#) *model, std::string name="")
Creates an R runner bound to a model.
- virtual ~**RSimulatorRunner** ()=default
Destroys the R runner.
- void **setRExecutable** (std::string rExecutable)
Sets the Rscript executable name or path.
- std::string **getRExecutable** () const
Returns the Rscript executable name or path.
- void **setWorkingDirectory** (std::string workingDirectory)
Sets the working directory used for generated scripts and command execution.
- std::string **getWorkingDirectory** () const
Returns the working directory used for generated scripts and command execution.
- void **setPreludeScript** (std::string preludeScript)
Sets optional R code prepended before the command block.
- std::string **getPreludeScript** () const
Returns optional R code prepended before the command block.
- void **setCommand** (std::string command)
Sets the R command/script body to execute.
- std::string **getCommand** () const
Returns the R command/script body to execute.
- void **setLastStatus** (std::string lastStatus)
Sets the last execution status.
- std::string **getLastStatus** () const
Returns the last execution status.
- void **setLastExitCode** (int lastExitCode)
Sets the last process exit code.
- int **getLastExitCode** () const
Returns the last process exit code.
- void **setLastStdout** (std::string lastStdout)
Sets stdout captured from the last R execution.
- std::string **getLastStdout** () const
Returns stdout captured from the last R execution.

- void **setLastStderr** (std::string lastStderr)
Sets stderr captured from the last R execution.
- std::string **getLastStderr** () const
Returns stderr captured from the last R execution.
- void **setLastResponsePayload** (std::string lastResponsePayload)
Sets the response payload exposed to other tools.
- std::string **getLastResponsePayload** () const
Returns the response payload exposed to other tools.
- void **setLastScriptFilename** (std::string lastScriptFilename)
Sets the generated R script filename from the last execution.
- std::string **getLastScriptFilename** () const
Returns the generated R script filename from the last execution.
- void **setLastResponseFilename** (std::string lastResponseFilename)
Sets the stdout file path from the last execution.
- std::string **getLastResponseFilename** () const
Returns the stdout file path from the last execution.
- bool **executeCommand** (std::string &errorMessage)
Executes the configured R code and updates last-execution fields atomically.
- virtual std::string **show** () override
Returns a compact human-readable description of the runner.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * **getInternalData** () const
Returns the full map of internal child data definitions.

- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
Loads an R runner from a persistence record.
- `static PluginInformation * GetPluginInformation ()`
Returns plugin metadata and system dependency declarations.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new R runner instance for plugin construction.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
Loads persisted R runner configuration and last-execution state.
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves R runner configuration and last-execution state.
- virtual bool `_check` (std::string &errorMessage) override
Validates the runner configuration without executing R.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.601.1 Detailed Description

[Model](#) data definition that executes R code through Rscript.

The runner stores a compact R execution configuration, writes a temporary script, executes it with Rscript, and keeps status, exit code, stdout, stderr and response payload inside the model data definition for later inspection.

10.601.2 Member Function Documentation

10.601.2.1 [_check\(\)](#)

```
bool RSimulatorRunner::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

Validates the runner configuration without executing R.

Reimplemented from [ModelDataDefinition](#).

10.601.2.2 `_loadInstance()`

```
bool RSimulatorRunner::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

Loads persisted R runner configuration and last-execution state.

Reimplemented from [ModelDataDefinition](#).

10.601.2.3 `_saveInstance()`

```
void RSimulatorRunner::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves R runner configuration and last-execution state.

Reimplemented from [ModelDataDefinition](#).

10.601.2.4 `show()`

```
std::string RSimulatorRunner::show ( ) [override], [virtual]
```

Returns a compact human-readable description of the runner.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

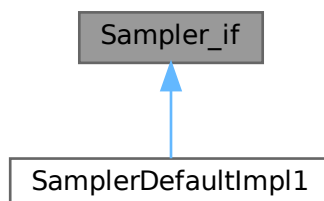
- `plugins/data/ExternalIntegration/RSimulatorRunner.h`
- `plugins/data/ExternalIntegration/RSimulatorRunner.cpp`

10.602 `Sampler_if` Class Reference

Abstraction for random-number generation and probability-distribution sampling.

```
#include <kernel/statistics/Sampler_if.h>
```

Inheritance diagram for `Sampler_if`:



Classes

- struct [RNG_Parameters](#)
Encapsulates generator-specific configuration/state parameters.

Public Member Functions

- virtual double [random](#) ()=0
Generates a pseudo-random value uniformly distributed in the range [0,1).
- virtual double [sampleBeta](#) (double alpha, double beta, double infLimit, double supLimit)=0
Samples a Beta-distributed value scaled to an arbitrary output interval.
- virtual double [sampleBeta](#) (double alpha, double beta)=0
Samples a standard Beta-distributed value in [0,1].
- virtual double [sampleErlang](#) (double mean, int M, double offset=0.0)=0
Samples from an Erlang distribution using the provided mean and number of phases.
- virtual double [sampleExponential](#) (double mean, double offset=0.0)=0
Samples from an Exponential distribution with the provided mean.
- virtual double [sampleGamma](#) (double alpha, double beta, double offset=0.0)=0
Samples from a Gamma distribution using the provided shape/scale parameters.
- virtual double [sampleGumbell](#) (double mode, double scale)=0
Samples from a Gumbel distribution using mode and scale.
- virtual double [sampleLogNormal](#) (double mean, double stddev, double offset=0.0)=0
Samples from a Lognormal distribution parameterized by mean and standard deviation.
- virtual double [sampleNormal](#) (double mean, double stddev)=0
Samples from a Normal distribution.
- virtual double [sampleTriangular](#) (double min, double mode, double max)=0
Samples from a Triangular distribution.
- virtual double [sampleUniform](#) (double min, double max)=0
Samples from a continuous Uniform distribution in [min, max].
- virtual double [sampleWeibull](#) (double alpha, double scale)=0
Samples from a Weibull distribution.
- virtual double [sampleBinomial](#) (int trials, double p)=0
*Samples from a Binomial distribution with *trials* and success probability *p*.*
- virtual double [sampleBernoulli](#) (double p)=0
Samples from a Bernoulli distribution (0/1).
- virtual double [sampleDiscrete](#) (double prob, double value,...)=0
Samples from a discrete distribution using a variadic list of probability/value pairs.
- virtual double [sampleDiscrete](#) (double *prob, double *value, int size)=0
Samples from a discrete distribution using probability/value arrays.
- virtual double [sampleGeometric](#) (double p)=0
*Samples from a Geometric distribution with success probability *p*.*
- virtual void [setRNGparameters](#) ([RNG_Parameters](#) *param)=0
Updates the pseudo-random generator internal parameters.
- virtual [RNG_Parameters](#) * [getRNGparameters](#) () const =0
Returns the current pseudo-random generator parameters.

10.602.1 Detailed Description

Abstraction for random-number generation and probability-distribution sampling.

Concrete implementations provide a base uniform pseudo-random generator and helper methods to sample supported continuous and discrete probability distributions used across the simulation kernel (e.g., parser functions and stochastic model logic).

10.602.2 Member Function Documentation

10.602.2.1 `getRNGparameters()`

```
virtual RNG_Parameters * Sampler_if::getRNGparameters ( ) const [pure virtual]
```

Returns the current pseudo-random generator parameters.

Implemented in [SamplerDefaultImpl1](#).

10.602.2.2 `random()`

```
virtual double Sampler_if::random ( ) [pure virtual]
```

Generates a pseudo-random value uniformly distributed in the range [0,1).

Returns

Base pseudo-random value used by the other distribution samplers.

Implemented in [SamplerDefaultImpl1](#).

10.602.2.3 `sampleBernoulli()`

```
virtual double Sampler_if::sampleBernoulli (
    double p ) [pure virtual]
```

Samples from a Bernoulli distribution (0/1).

Implemented in [SamplerDefaultImpl1](#).

10.602.2.4 `sampleBeta()` [1/2]

```
virtual double Sampler_if::sampleBeta (
    double alpha,
    double beta ) [pure virtual]
```

Samples a standard Beta-distributed value in [0,1].

Implemented in [SamplerDefaultImpl1](#).

10.602.2.5 `sampleBeta()` [2/2]

```
virtual double Sampler_if::sampleBeta (
    double alpha,
    double beta,
    double infLimit,
    double supLimit ) [pure virtual]
```

Samples a Beta-distributed value scaled to an arbitrary output interval.

Parameters

<i>alpha</i>	Alpha shape parameter (>0).
<i>beta</i>	Beta shape parameter (>0).
<i>infLimit</i>	Lower bound of the output interval.
<i>supLimit</i>	Upper bound of the output interval.

Returns

Sampled value within [infLimit, supLimit].

Implemented in [SamplerDefaultImpl1](#).

10.602.2.6 sampleBinomial()

```
virtual double Sampler_if::sampleBinomial (  
    int trials,  
    double p ) [pure virtual]
```

Samples from a Binomial distribution with `trials` and success probability `p`.

Implemented in [SamplerDefaultImpl1](#).

10.602.2.7 sampleDiscrete() [1/2]

```
virtual double Sampler_if::sampleDiscrete (  
    double * prob,  
    double * value,  
    int size ) [pure virtual]
```

Samples from a discrete distribution using probability/value arrays.

Implemented in [SamplerDefaultImpl1](#).

10.602.2.8 sampleDiscrete() [2/2]

```
virtual double Sampler_if::sampleDiscrete (  
    double prob,  
    double value,  
    ... ) [pure virtual]
```

Samples from a discrete distribution using a variadic list of probability/value pairs.

Implemented in [SamplerDefaultImpl1](#).

10.602.2.9 sampleErlang()

```
virtual double Sampler_if::sampleErlang (
    double mean,
    int M,
    double offset = 0.0 ) [pure virtual]
```

Samples from an Erlang distribution using the provided mean and number of phases.

Implemented in [SamplerDefaultImpl1](#).

10.602.2.10 sampleExponential()

```
virtual double Sampler_if::sampleExponential (
    double mean,
    double offset = 0.0 ) [pure virtual]
```

Samples from an Exponential distribution with the provided mean.

Implemented in [SamplerDefaultImpl1](#).

10.602.2.11 sampleGamma()

```
virtual double Sampler_if::sampleGamma (
    double alpha,
    double beta,
    double offset = 0.0 ) [pure virtual]
```

Samples from a Gamma distribution using the provided shape/scale parameters.

Implemented in [SamplerDefaultImpl1](#).

10.602.2.12 sampleGeometric()

```
virtual double Sampler_if::sampleGeometric (
    double p ) [pure virtual]
```

Samples from a Geometric distribution with success probability p.

Implemented in [SamplerDefaultImpl1](#).

10.602.2.13 sampleGumbell()

```
virtual double Sampler_if::sampleGumbell (
    double mode,
    double scale ) [pure virtual]
```

Samples from a Gumbel distribution using mode and scale.

Implemented in [SamplerDefaultImpl1](#).

10.602.2.14 sampleLogNormal()

```
virtual double Sampler_if::sampleLogNormal (
    double mean,
    double stddev,
    double offset = 0.0 ) [pure virtual]
```

Samples from a Lognormal distribution parameterized by mean and standard deviation.

Implemented in [SamplerDefaultImpl1](#).

10.602.2.15 sampleNormal()

```
virtual double Sampler_if::sampleNormal (
    double mean,
    double stddev ) [pure virtual]
```

Samples from a Normal distribution.

Implemented in [SamplerDefaultImpl1](#).

10.602.2.16 sampleTriangular()

```
virtual double Sampler_if::sampleTriangular (
    double min,
    double mode,
    double max ) [pure virtual]
```

Samples from a Triangular distribution.

Implemented in [SamplerDefaultImpl1](#).

10.602.2.17 sampleUniform()

```
virtual double Sampler_if::sampleUniform (
    double min,
    double max ) [pure virtual]
```

Samples from a continuous Uniform distribution in [min, max].

Implemented in [SamplerDefaultImpl1](#).

10.602.2.18 sampleWeibull()

```
virtual double Sampler_if::sampleWeibull (
    double alpha,
    double scale ) [pure virtual]
```

Samples from a Weibull distribution.

Implemented in [SamplerDefaultImpl1](#).

10.602.2.19 setRNGparameters()

```
virtual void Sampler_if::setRNGparameters (
    RNG_Parameters * param ) [pure virtual]
```

Updates the pseudo-random generator internal parameters.

Implemented in [SamplerDefaultImpl1](#).

The documentation for this class was generated from the following file:

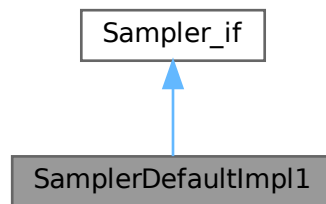
- kernel/statistics/Sampler_if.h

10.603 SamplerDefaultImpl1 Class Reference

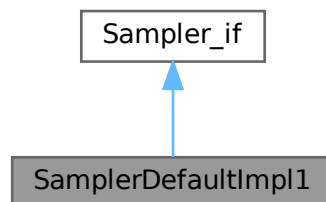
Default pseudo-random sampler implementation used by the kernel.

```
#include <kernel/statistics/SamplerDefaultImpl1.h>
```

Inheritance diagram for SamplerDefaultImpl1:



Collaboration diagram for SamplerDefaultImpl1:



Classes

- struct [DefaultImpl1RNG_Parameters](#)
Parameter bundle for the default RNG algorithm.

Public Member Functions

- **SamplerDefaultImpl1** ()
Creates the sampler with default RNG parameters.
- virtual $\sim$ **SamplerDefaultImpl1** ()
Destroys sampler and owned parameter objects when applicable.
- virtual double [random](#) () override
Generates one base uniform pseudo-random number in $[0, 1)$.
- virtual double [sampleBeta](#) (double alpha, double beta, double infLimit, double supLimit) override
Samples Beta in arbitrary interval $[infLimit, supLimit]$.
- virtual double [sampleBeta](#) (double alpha, double beta) override
Samples standard Beta in $[0, 1]$.
- virtual double [sampleErlang](#) (double mean, int M, double offset=0.0) override
Samples Erlang distribution.
- virtual double [sampleExponential](#) (double mean, double offset=0.0) override
Samples Exponential distribution.
- virtual double [sampleGamma](#) (double alpha, double beta, double offset=0.0) override
Samples Gamma distribution.
- virtual double [sampleGumbell](#) (double mode, double scale) override
Samples Gumbel distribution.
- virtual double [sampleLogNormal](#) (double mean, double stddev, double offset=0.0) override
Samples Lognormal distribution.
- virtual double [sampleNormal](#) (double mean, double stddev) override
Samples Normal distribution.
- virtual double [sampleTriangular](#) (double min, double mode, double max) override
Samples Triangular distribution.
- virtual double [sampleUniform](#) (double min, double max) override
Samples Uniform distribution in $[min, max]$.
- virtual double [sampleWeibull](#) (double alpha, double scale) override
Samples Weibull distribution.
- virtual double [sampleBinomial](#) (int trials, double p) override
Samples Binomial distribution.
- virtual double [sampleBernoulli](#) (double p) override
Samples Bernoulli distribution.
- virtual double [sampleDiscrete](#) (double prob, double value,...) override
Samples arbitrary discrete distribution (variadic interface).
- virtual double [sampleDiscrete](#) (double *prob, double *value, int size) override
Samples arbitrary discrete distribution (array interface).
- virtual double [sampleGeometric](#) (double p) override
Samples Geometric distribution.
- void **reset** ()
Reinitializes RNG state so the pseudo-random sequence restarts.
- virtual void [setRNGparameters](#) ([RNG_Parameters](#) *param) override
Updates the pseudo-random generator internal parameters.
- virtual [RNG_Parameters](#) * [getRNGparameters](#) () const override
Returns the current pseudo-random generator parameters.

10.603.1 Detailed Description

Default pseudo-random sampler implementation used by the kernel.

Implements a base RNG and multiple continuous/discrete distributions commonly used by simulation components and expression parser stochastic functions.

10.603.2 Member Function Documentation

10.603.2.1 `getRNGparameters()`

```
Sampler_if::RNG_Parameters * SamplerDefaultImpl1::getRNGparameters ( ) const [override], [virtual]
```

Returns the current pseudo-random generator parameters.

Implements [Sampler_if](#).

10.603.2.2 `random()`

```
double SamplerDefaultImpl1::random ( ) [override], [virtual]
```

Generates one base uniform pseudo-random number in [0,1).

Implements [Sampler_if](#).

10.603.2.3 `sampleBernoulli()`

```
double SamplerDefaultImpl1::sampleBernoulli (
    double p ) [override], [virtual]
```

Samples Bernoulli distribution.

Implements [Sampler_if](#).

10.603.2.4 `sampleBeta()` [1/2]

```
double SamplerDefaultImpl1::sampleBeta (
    double alpha,
    double beta ) [override], [virtual]
```

Samples standard Beta in [0,1].

Implements [Sampler_if](#).

10.603.2.5 sampleBeta() [2/2]

```
double SamplerDefaultImpl1::sampleBeta (
    double alpha,
    double beta,
    double infLimit,
    double supLimit ) [override], [virtual]
```

Samples Beta in arbitrary interval [infLimit, supLimit].

Implements [Sampler_if](#).

10.603.2.6 sampleBinomial()

```
double SamplerDefaultImpl1::sampleBinomial (
    int trials,
    double p ) [override], [virtual]
```

Samples Binomial distribution.

Implements [Sampler_if](#).

10.603.2.7 sampleDiscrete() [1/2]

```
double SamplerDefaultImpl1::sampleDiscrete (
    double * prob,
    double * value,
    int size ) [override], [virtual]
```

Samples arbitrary discrete distribution (array interface).

Implements [Sampler_if](#).

10.603.2.8 sampleDiscrete() [2/2]

```
double SamplerDefaultImpl1::sampleDiscrete (
    double prob,
    double value,
    ... ) [override], [virtual]
```

Samples arbitrary discrete distribution (variadic interface).

Implements [Sampler_if](#).

10.603.2.9 sampleErlang()

```
double SamplerDefaultImpl1::sampleErlang (
    double mean,
    int M,
    double offset = 0.0 ) [override], [virtual]
```

Samples Erlang distribution.

Implements [Sampler_if](#).

10.603.2.10 sampleExponential()

```
double SamplerDefaultImpl1::sampleExponential (
    double mean,
    double offset = 0.0 ) [override], [virtual]
```

Samples Exponential distribution.

Implements [Sampler_if](#).

10.603.2.11 sampleGamma()

```
double SamplerDefaultImpl1::sampleGamma (
    double alpha,
    double beta,
    double offset = 0.0 ) [override], [virtual]
```

Samples Gamma distribution.

Implements [Sampler_if](#).

10.603.2.12 sampleGeometric()

```
double SamplerDefaultImpl1::sampleGeometric (
    double p ) [override], [virtual]
```

Samples Geometric distribution.

Implements [Sampler_if](#).

10.603.2.13 sampleGumbell()

```
double SamplerDefaultImpl1::sampleGumbell (
    double mode,
    double scale ) [override], [virtual]
```

Samples Gumbel distribution.

Implements [Sampler_if](#).

10.603.2.14 sampleLogNormal()

```
double SamplerDefaultImpl1::sampleLogNormal (
    double mean,
    double stddev,
    double offset = 0.0 ) [override], [virtual]
```

Samples Lognormal distribution.

Implements [Sampler_if](#).

10.603.2.15 sampleNormal()

```
double SamplerDefaultImpl1::sampleNormal (
    double mean,
    double stddev ) [override], [virtual]
```

Samples Normal distribution.

Implements [Sampler_if](#).

10.603.2.16 sampleTriangular()

```
double SamplerDefaultImpl1::sampleTriangular (
    double min,
    double mode,
    double max ) [override], [virtual]
```

Samples Triangular distribution.

Implements [Sampler_if](#).

10.603.2.17 sampleUniform()

```
double SamplerDefaultImpl1::sampleUniform (
    double min,
    double max ) [override], [virtual]
```

Samples Uniform distribution in [min,max].

Implements [Sampler_if](#).

10.603.2.18 sampleWeibull()

```
double SamplerDefaultImpl1::sampleWeibull (
    double alpha,
    double scale ) [override], [virtual]
```

Samples Weibull distribution.

Implements [Sampler_if](#).

10.603.2.19 setRNGparameters()

```
void SamplerDefaultImpl1::setRNGparameters (
    RNG_Parameters * param ) [override], [virtual]
```

Updates the pseudo-random generator internal parameters.

Implements [Sampler_if](#).

The documentation for this class was generated from the following files:

- kernel/statistics/SamplerDefaultImpl1.h
- kernel/statistics/SamplerDefaultImpl1.cpp

10.604 ScenarioExperiment_if Class Reference

The documentation for this class was generated from the following file:

- kernel/simulator/ScenarioExperiment_if.h

10.605 SceneToolController Class Reference

Controller for scene visualization, drawing tools, alignment, and zoom orchestration.

```
#include <applications/gui/genesys/controllers/SceneToolController.h>
```

Public Member Functions

- [SceneToolController](#) ([ModelGraphicsView](#) *graphicsView, [Ui::MainWindow](#) *ui, std::function< [ModelGraphicsScene](#) *(> currentScene, std::function< bool(> createModelImage, std::function< void(> unselectDrawIcons, std::function< bool(> checkSelectedDrawIcons, std::function< void(double)> gentleZoom, std::function< void(> actualizeActions, std::function< void(> actualizeTabPanels, int &zoomValue, bool &firstClickShow↵ Connection)
- Builds the scene-tool delegated controller used by [MainWindow](#) compatibility slots.
- void [onActionShowGridTriggered](#) ()
- Toggles grid visibility and requests scene redraw updates.
- void [onActionShowRuleTriggered](#) ()
- Toggles ruler visibility through the delegated scene callback path.
- void [onActionShowGuidesTriggered](#) ()
- Toggles guide visibility and keeps tab/action synchronization.
- void [onActionShowSnapTriggered](#) ()
- Toggles snap behavior on the current scene without changing tool wiring.
- void [onActionZoomInTriggered](#) ()
- Applies incremental zoom-in and updates shared zoom state.
- void [onActionZoomOutTriggered](#) ()
- Applies incremental zoom-out and updates shared zoom state.
- void [onActionZoomAllTriggered](#) ()
- Fits the current graphical model in view and re-synchronizes zoom widgets.
- void [onActionZoomActualSizeTriggered](#) ()
- Restores the graphical view to its default 1:1 zoom.
- void [syncZoomWidgetsForCurrentScale](#) ()
- Synchronizes the zoom slider and label with the current view scale.
- void [onHorizontalSliderZoomGraphicalValueChanged](#) (int value)
- Synchronizes slider-originated zoom changes with scene/view scale.
- void [onActionDrawLineTriggered](#) ()
- Activates line drawing mode as a compatibility façade wrapper.
- void [onActionDrawRectangleTriggered](#) ()
- Activates rectangle drawing mode as a compatibility façade wrapper.
- void [onActionDrawEllipseTriggered](#) ()
- Activates ellipse drawing mode as a compatibility façade wrapper.
- void [onActionDrawTextTriggered](#) ()
- Activates text drawing mode as a compatibility façade wrapper.

- void **onActionDrawPoligonTriggered** ()
Activates polygon drawing mode as a compatibility façade wrapper.
- void **onActionAnimateSimulatedTimeTriggered** ()
Toggles simulated-time animation overlays in the current scene.
- void **onActionAnimateVariableTriggered** ()
Toggles variable animation overlays in the current scene.
- void **onActionAnimateCounterTriggered** ()
Toggles counter animation overlays in the current scene.
- void **onActionAnimateAttributeTriggered** ()
Activates attribute animation placeholder drawing.
- void **onActionAnimateEntityTriggered** ()
Activates entity animation placeholder drawing.
- void **onActionAnimateEventTriggered** ()
Activates event animation placeholder drawing.
- void **onActionAnimateExpressionTriggered** ()
Activates expression animation placeholder drawing.
- void **onActionAnimatePlotTriggered** ()
Activates plot animation placeholder drawing.
- void **onActionAnimateQueueTriggered** ()
Activates queue animation placeholder drawing.
- void **onActionAnimateResourceTriggered** ()
Activates resource animation placeholder drawing.
- void **onActionAnimateStationTriggered** ()
Activates station animation placeholder drawing.
- void **onActionAnimateStatisticsTriggered** ()
Activates statistics animation placeholder drawing.
- void **onActionConnectTriggered** ()
Activates connection-creation mode and resets first-click compatibility state.
- void **onActionArranjeLeftTriggered** ()
Aligns selected graphical items to the left boundary.
- void **onActionArranjeRightTriggered** ()
Aligns selected graphical items to the right boundary.
- void **onActionArranjeTopTriggered** ()
Aligns selected graphical items to the top boundary.
- void **onActionArranjeBototmTriggered** ()
Aligns selected graphical items to the bottom boundary.
- void **onActionArranjeCenterTriggered** ()
Aligns selected graphical items to horizontal center.
- void **onActionArranjeMiddleTriggered** ()
Aligns selected graphical items to vertical middle.
- void **onActionActivateGraphicalSimulationTriggered** ()
Toggles graphical simulation visualization mode and action state.
- void **onHorizontalSliderAnimationSpeedValueChanged** (int value)
Applies animation-speed slider values to scene animation pacing.
- void **onActionSelectAllTriggered** ()
Selects all graphical items in the active scene context.
- void **onActionShowInternalElementsTriggered** ()
Toggles rendering of statistics data definitions and triggers representation refresh.
- void **onActionShowEditableElementsTriggered** ()
Toggles rendering of editable data definitions and triggers representation refresh.
- void **onActionShowAttachedElementsTriggered** ()

- *Toggles rendering of shared data definitions and triggers representation refresh.*
- void **onCheckBoxShowElementsStateChanged** (int arg1)
Bridges checkbox state changes to the show-elements action wrapper.
- void **onCheckBoxShowInternalsStateChanged** (int arg1)
Bridges checkbox state changes to the show-internals action wrapper.
- void **onCheckBoxShowEditableElementsStateChanged** (int arg1)
Bridges checkbox state changes to the show-editable-elements action wrapper.
- void **onActionShowRecursiveElementsTriggered** ()
Toggles recursive data-definition expansion and triggers representation refresh.
- void **onCheckBoxShowRecursiveStateChanged** (int arg1)
Bridges recursive-render checkbox changes to model-image refresh workflow.
- void **onCheckBoxShowLevelsStateChanged** (int arg1)
Bridges level-annotation checkbox changes to model-image refresh workflow.
- void **onActionGModelShowConnectTriggered** ()
Toggles interactive connection-hint visibility in scene callback wiring.

10.605.1 Detailed Description

Controller for scene visualization, drawing tools, alignment, and zoom orchestration.

[MainWindow](#) delegates many menu/toolbar wrappers to this Phase-10 controller to keep the compatibility façade thin while concentrating scene-tool behavior in a dedicated component.

Responsibilities:

- handle grid/ruler/guides/snap toggles and zoom commands;
- orchestrate drawing/connection modes and alignment actions;
- coordinate animation toggles and scene-display filters through injected callbacks.

Boundaries:

- it does not own [MainWindow](#) or scene/view lifetime;
- it does not perform model persistence/export/simulation lifecycle work;
- it relies on injected references for shared UI state (zoom, first-click connection state).

10.605.2 Constructor & Destructor Documentation

10.605.2.1 SceneToolController()

```
SceneToolController::SceneToolController (
    ModelGraphicsView * graphicsView,
    Ui::MainWindow * ui,
    std::function< ModelGraphicsScene *()> currentScene,
    std::function< bool()> createModelImage,
    std::function< void()> unselectDrawIcons,
    std::function< bool()> checkSelectedDrawIcons,
    std::function< void(double)> gentleZoom,
    std::function< void()> actualizeActions,
    std::function< void()> actualizeTabPanels,
    int & zoomValue,
    bool & firstClickShowConnection )
```

Builds the scene-tool delegated controller used by [MainWindow](#) compatibility slots.

Stores only the narrow collaborators needed for scene-tool orchestration.

10.605.3 Member Function Documentation

10.605.3.1 onActionDrawLineTriggered()

```
void SceneToolController::onActionDrawLineTriggered ( )
```

Activates line drawing mode as a compatibility façade wrapper.

Activates the line drawing tool while preserving cursor semantics.

10.605.3.2 onActionShowGridTriggered()

```
void SceneToolController::onActionShowGridTriggered ( )
```

Toggles grid visibility and requests scene redraw updates.

Synchronizes the grid action with the current scene state.

10.605.3.3 onActionZoomActualSizeTriggered()

```
void SceneToolController::onActionZoomActualSizeTriggered ( )
```

Restores the graphical view to its default 1:1 zoom.

Restores the graphics view to the 1:1 zoom level and re-synchronizes the slider.

10.605.3.4 onActionZoomAllTriggered()

```
void SceneToolController::onActionZoomAllTriggered ( )
```

Fits the current graphical model in view and re-synchronizes zoom widgets.

Fits the scene to view and resets the zoom slider baseline.

10.605.3.5 onHorizontalSliderZoomGraphicalValueChanged()

```
void SceneToolController::onHorizontalSliderZoomGraphicalValueChanged (
    int value )
```

Synchronizes slider-originated zoom changes with scene/view scale.

Applies the zoom slider value to the graphics view transform.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/SceneToolController.h
- applications/gui/genesys/controllers/SceneToolController.cpp

10.606 SchedulableItem Class Reference

Public Types

- enum class **Rule** : int { **IGNORE** = 1 , **PREEMPT** = 2 , **WAIT** = 3 }

Public Member Functions

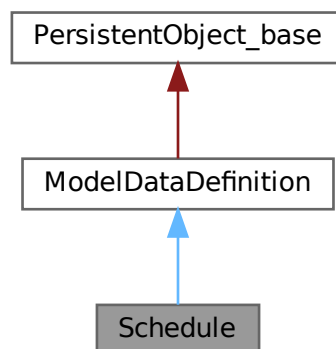
- **SchedulableItem** (std::string expression, double duration, SchedulableItem::Rule rule=SchedulableItem::Rule::IGNORE)
- void **setDuration** (double duration)
- double **getDuration** () const
- void **setExpression** (std::string expression)
- std::string **getExpression** () const
- void **setRule** (SchedulableItem::Rule rule)
- SchedulableItem::Rule **getRule** () const

The documentation for this class was generated from the following file:

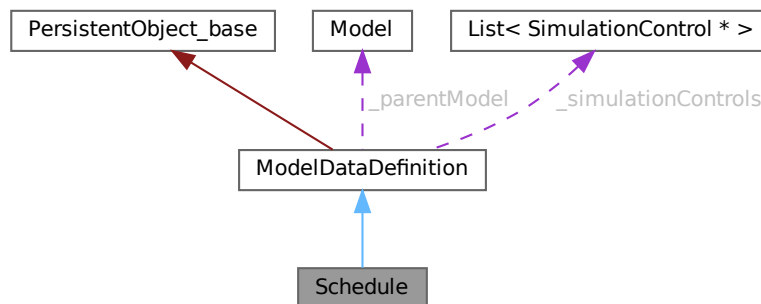
- plugins/data/DiscreteProcessing/Schedule.h

10.607 Schedule Class Reference

Inheritance diagram for Schedule:



Collaboration diagram for Schedule:



Public Member Functions

- **Schedule** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- std::string **getExpression** ()
- [List](#)< [SchedulableItem](#) * > * **getSchedulableItems** () const
- void **setRepeatAfterLast** (bool _repeatAfterLast)
- bool **isRepeatAfterLast** () const

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.

- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool hasChanged)
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` () override
- virtual void `_initBetweenReplications` () override
- virtual void `_addSimulationControl` ([SimulationControl](#) *property) override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.607.1 Member Function Documentation

10.607.1.1 [_addSimulationControl\(\)](#)

```
void Schedule::_addSimulationControl (
    SimulationControl * property ) [override], [protected], [virtual]
```

Keep local control registration aligned with the base model-data contract.

Reimplemented from [ModelDataDefinition](#).

10.607.1.2 [_check\(\)](#)

```
bool Schedule::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Validate that schedule has items and non-negative durations.

Reimplemented from [ModelDataDefinition](#).

10.607.1.3 `_getParserChangesInformation()`

```
ParserChangesInformation * Schedule::_getParserChangesInformation ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes Return parser changes required by [Schedule](#).

No parser customization is currently necessary.

Reimplemented from [ModelDataDefinition](#).

10.607.1.4 `_initBetweenReplications()`

```
void Schedule::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.607.1.5 `_loadInstance()`

```
bool Schedule::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overriden by derived classes

Load schedulable items and repeat behavior.

Reimplemented from [ModelDataDefinition](#).

10.607.1.6 `_saveInstance()`

```
void Schedule::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Persist schedule items and repeat behavior.

Reimplemented from [ModelDataDefinition](#).

10.607.1.7 `show()`

```
std::string Schedule::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal `ModelDataDefinitions` such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

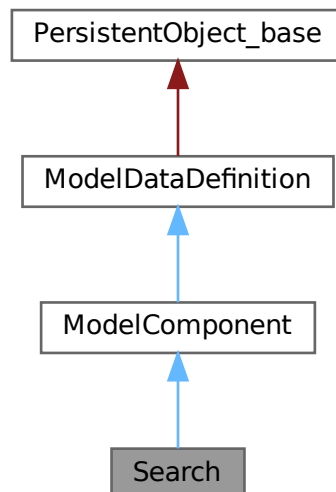
The documentation for this class was generated from the following files:

- `plugins/data/DiscreteProcessing/Schedule.h`
- `plugins/data/DiscreteProcessing/Schedule.cpp`

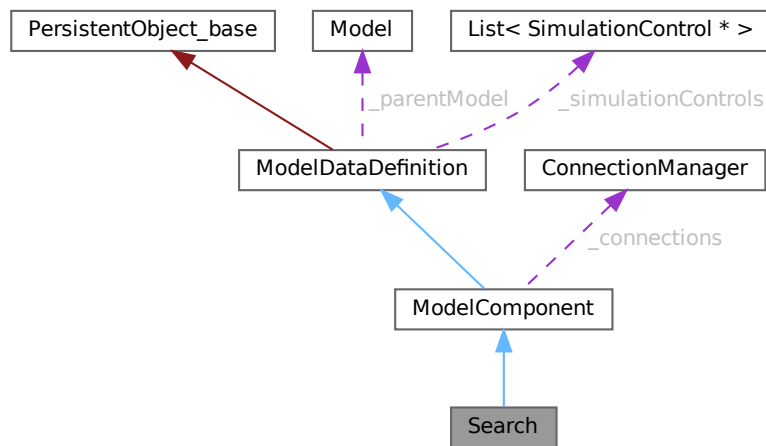
10.608 Search Class Reference

```
#include <plugins/components/Decisions/Search.h>
```

Inheritance diagram for Search:



Collaboration diagram for Search:



Public Types

- enum class **SearchInType** : int { **QUEUE** = 0 , **ENTITYGROUP** = 1 , **num_elements** = 2 }

Public Member Functions

- **Search** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
- *virtual public methods*
- void **setSaveFounRankAttribute** (std::string _saveFounRankAttribute)
- std::string **getSaveFounRankAttribute** () const
- void **setSearchCondition** (std::string _searchCondition)
- std::string **getSearchCondition** () const
- void **setEndRank** (std::string _endRank)
- std::string **getEndRank** () const
- void **setStartRank** (std::string _startRank)
- std::string **getStartRank** () const
- void **setSearchInName** (std::string searchInName)
- std::string **getSearchInName** () const
- void **setSearchIn** ([ModelDataDefinition](#) *_searchIn)
- [ModelDataDefinition](#) * **getSearchIn** () const
- void **setSearchInType** (Search::SearchInType _searchInType)
- Search::SearchInType **getSearchInType** () const

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.

- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static std::string [convertEnumToStr](#) ([SearchInType](#) type)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.608.1 Detailed Description

Search module DESCRIPTION The **Search** module searches a queue, a group (batch), or an expression to find the entity rank (for entities in a queue or group) or the value of the global variable J that satisfies the specified search condition. When searching a queue or group, the value of the global system variable J is set to the rank of the first entity that satisfies **Search** Condition, or to 0 if **Search** Condition is not satisfied. When searching an expression, the global system variable J is set to the value of the first index value that satisfies the search condition or to zero if no value of J in the specified range satisfies the search condition. When an entity arrives at a **Search** module, the index J is set to the starting index and the search condition is then checked. If the search condition is satisfied, the search ends and the current value of J is retained. Otherwise, the value of J is increased or decreased and the condition is rechecked. This process repeats until the search condition is satisfied or the ending value is reached. If the condition is not met or there are no entities in the queue or group, J is set equal to 0. TYPICAL USES Looking for a particular order number in a queue Searching a group for a certain part type Determining which process to enter based on availability of resources (search an expression) Prompt Description Name Unique module identifier displayed on the module shape. Type Determination of what will be searched. **Search** options include entities in a queue, entities within a group (batch) or some expression(s). **Queue** Name Name of the queue that will be searched. Applies only when the Type is **Search** a **Queue**. Starting Value Starting rank in the queue or group or starting value for J in an expression. Ending Value Ending rank in the queue or group or ending value for J in an expression. **Search** Condition Condition containing the index J for searching expressions or containing an attribute name(s) for searching queues or batches.

10.608.2 Member Function Documentation

10.608.2.1 _check()

```
bool Search::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.608.2.2 _createEditableDataDefinitions()

```
void Search::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dynamically linked to to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

10.608.2.3 _loadInstance()

```
bool Search::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.608.2.4 _onDispatchEvent()

```
void Search::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.608.2.5 _saveInstance()

```
void Search::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.608.2.6 show()

```
std::string Search::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- `plugins/components/Decisions/Search.h`
- `plugins/components/Decisions/Search.cpp`

10.609 SeizableItem Class Reference

Public Types

- enum class **SelectionRule** : int {
CYCLICAL = 0 , **RANDOM** = 1 , **SPECIFICMEMBER** = 2 , **LARGESTREMAININGCAPACITY** = 3 ,
SMALLESTNUMBERBUSY = 4 , **PREFEREDORDER** = 5 , **num_elements** = 6 }
- enum class **SeizableType** : int { **RESOURCE** = 0 , **SET** = 1 , **num_elements** = 2 }

Public Member Functions

- **SeizableItem** ([ModelDataDefinition](#) *resourceOrSet, std::string quantityExpression="1", SeizableItem↔::SelectionRule selectionRule=SeizableItem::SelectionRule::LARGESTREMAININGCAPACITY, std::string saveAttribute="", std::string index="0")
- **SeizableItem** ([Model](#) *model, std::string resourceName, std::string quantityExpression="1", SeizableItem↔::SelectionRule selectionRule=SeizableItem::SelectionRule::LARGESTREMAININGCAPACITY, std::string saveAttribute="", std::string index="0")
- **SeizableItem** ([SeizableItem](#) *original)
- bool **loadInstance** ([PersistenceRecord](#) *fields, unsigned int parentIndex)
- void **saveInstance** ([PersistenceRecord](#) *fields, unsigned int parentIndex, bool saveDefault)
- bool **loadInstance** ([PersistenceRecord](#) *fields)
- void **saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaults)
- std::string **show** ()
- void **setIndex** (std::string index)
- std::string **getIndex** () const
- void **setSaveAttribute** (std::string saveAttribute)
- std::string **getSaveAttribute** () const
- void **setSelectionRule** (SelectionRule selectionRule)
- SelectionRule **getSelectionRule** () const
- void **setQuantityExpression** (std::string quantityExpression)
- std::string **getQuantityExpression** () const
- std::string **getResourceName** () const
- std::string **getName** () const
- void **setResource** ([Resource](#) *resource)
- [Resource](#) * **getResource** () const
- void **setSet** ([Set](#) *set)
- [Set](#) * **getSet** () const
- SeizableType **getSeizableType** () const
- void **setLastMemberSeized** (unsigned int lastMemberSeized)
- unsigned int **getLastMemberSeized** () const
- [ModelDataDefinition](#) * **getSeizable** () const
- void **setElementManager** ([ModelDataManager](#) *modeldataManager)
- void **setLastPreferredOrder** (unsigned int lastPreferredOrder)
- unsigned int **getLastPreferredOrder** () const
- [List](#)< [SimulationControl](#) * > * **getSimulationControls** () const
- void **_addSimulationControl** ([SimulationControl](#) *control)
- std::string **getTypeDC** ()

Static Public Member Functions

- static std::string **convertEnumToStr** (SelectionRule rule)
- static std::string **convertEnumToStr** (SeizableType type)

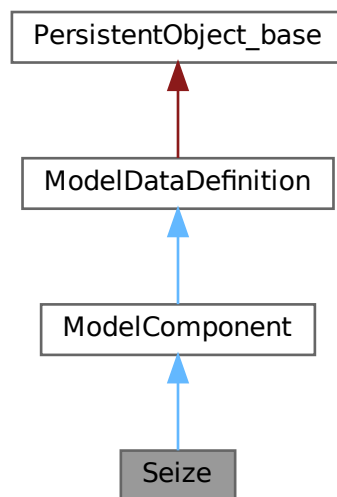
The documentation for this class was generated from the following files:

- plugins/components/DiscreteProcessing/auxiliar/SeizableItem.h
- plugins/components/DiscreteProcessing/auxiliar/SeizableItem.cpp

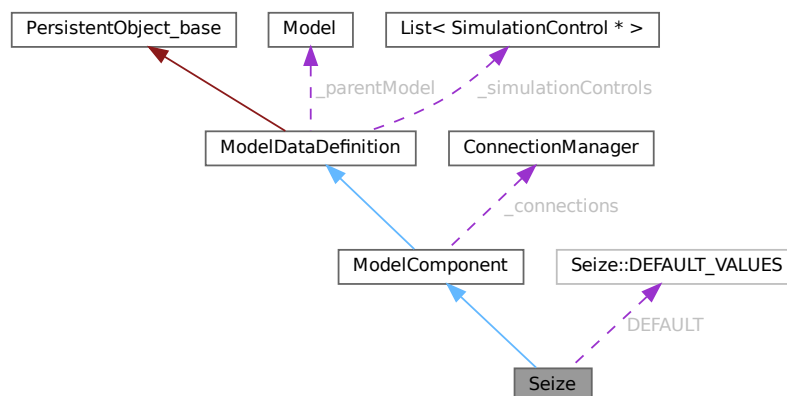
10.610 Seize Class Reference

```
#include <plugins/components/DiscreteProcessing/Seize.h>
```

Inheritance diagram for Seize:



Collaboration diagram for Seize:



Classes

- struct [DEFAULT_VALUES](#)

Public Member Functions

- **Seize** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
virtual public methods
- void **setPriority** (unsigned short _priority)
- unsigned short **getPriority** () const
- void **setAllocationType** (Util::AllocationType _allocationType)
- Util::AllocationType **getAllocationType** () const
- void **setQueue** ([Queue](#) *queue)
Deprecated.
- [List](#)< [SeizableItem](#) * > * **getSeizeRequests** () const
- void **addRequest** ([SeizableItem](#) *newRequest)
- void **removeRequest** ([SeizableItem](#) *request)
- void **setQueueableItem** ([QueueableItem](#) *_queueableItem)
- [QueueableItem](#) * **getQueueableItem** () const
- void **setPriorityExpression** (std::string _priorityExpression)
- std::string **getPriorityExpression** () const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↔IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const

- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Public Attributes

- const struct [Seize::DEFAULT_VALUES](#) **DEFAULT**

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_initBetweenReplications](#) () override
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool \_getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * \_getParserChangesInformation ()`
- `virtual void \_addSimulationControl (SimulationControl *control)`
- `virtual void \_createInternalAndAttachedData ()`
- `virtual void \_createInternalStatisticReporters ()`
- `virtual void \_createAttachedAttributes ()`
- `virtual void \_createNonEditableDataDefinitions ()`
- `void \_templateCreateInternalStatisticReporters ()`
- `void \_templateCreateAttachedAttributes ()`
- `void \_templateCreateNonEditableDataDefinitions ()`
- `void \_templateCreateEditableDataDefinitions ()`
- `void \_internalStatisticReportersClear ()`
- *legacy helper methods used by the granular create* hooks*
- `void \_internalDataClear ()`
- `void \_internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void \_internalDataRemove (const std::string &key)`
- `void \_attachedDataClear ()`
- `void \_attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_attachedDataRemove (const std::string &key)`
- `void \_attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void \_statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_statisticReporterRemove (const std::string &key)`
- `void \_statisticReportersClear ()`
- `void \_mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryAttachedAttributeRemove (const std::string &key)`
- `void \_mandatoryAttachedAttributesClear ()`
- `void \_mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void \_mandatoryEditableDataDefinitionsClear ()`
- `void \_optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_optionalEditableDataDefinitionRemove (const std::string &key)`
- `void \_optionalEditableDataDefinitionsClear ()`
- `void \_mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void \_mandatoryNonEditableDataDefinitionsClear ()`
- `void \_optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void \_optionalNonEditableDataDefinitionsClear ()`
- `void \_checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- *method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.610.1 Detailed Description

[Seize](#) module DESCRIPTION The [Seize](#) module allocates units of one or more resources to an entity. The [Seize](#) module may be used to seize units of a particular resource, a member of a resource set, or a resource as defined by an alternative method, such as an attribute or expression. When an entity enters this module, it waits in a queue (if specified) until all specified resources are available simultaneously. Allocation type for resource usage is also specified. TYPICAL USES Beginning a customer order (seize the operator) Starting a tax return (seize the accountant) Being admitted to hospital (seize the hospital room, nurse, doctor) PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Allocation Determines to which category the resource usage cost will be allocated for an entity going through the [Seize](#) module. Priority Priority value of the entity waiting at this module for the resource(s) specified if one or more entities from other modules are waiting for the same resource(s). Type Type of resource for seizing, either specifying a particular resource, or selecting from a pool of resources (that is, a resource set). The name of the resource may also be specified as an attribute value or within an expression. [Resource](#) Name Name of the resource that will be seized. [Set](#) Name Name of the resource set from which a member will be seized. [Attribute](#) Name Name of the attribute that stores the resource name to be seized. Expression Expression that evaluates to a resource name to be seized. Quantity Number of resources of a given name or from a given set that will be seized. For sets, this value specifies only the number of a selected resource that will be seized (based on the resource's capacity), not the number of members to be seized within the set. Selection Rule Method of selecting among available resources in a set. Cyclical will cycle through available members (for example, 1-2-3-1-2-3). Random will randomly select a member. Preferred Order will always select the first available member (for example, 1, if available; then 2, if available; then 3). Specific Member requires an input attribute value to specify which member of the set (previously saved in the Save [Attribute](#) field). Largest Remaining Capacity and Smallest Number Busy are used for resources with multiple capacity. Save [Attribute](#) [Attribute](#) name used to store the index number into the set of the member that is chosen. This attribute can later be referenced with the Specific Member selection rule. [Set](#) Index Index value into the set that identifies the number into the set of the member requested. If an attribute name is used, the entity must have a value for the attribute before utilizing this option. [Resource State](#) State of the resource that will be assigned after the resource is seized. The resource state must be defined with the [Resource](#) module. [Queue](#) Type Determines the type of queue used to hold the entities while waiting to seize the resource(s). If [Queue](#) is selected, the queue name is specified. If [Set](#) is selected, the queue set and member in the set are specified. If Internal is selected, an internal queue is used to hold all waiting entities. [Attribute](#) and Expression are additional methods for defining the queue to be used. [Queue](#) Name This field is visible only if [Queue](#) Type is [Queue](#), and it defines the symbol name of the queue. [Set](#) Name This field is visible only if [Queue](#) Type is [Set](#), and it defines the queue set that contains the queue being referenced. [Set](#) Index This field is visible only if [Queue](#) Type is [Set](#), and it defines the index into the queue set. Note that this is the index into the set and not the name of the queue in the set. For example, the only valid entries for a queue set containing three members is an expression that evaluates to 1, 2, or 3. [Attribute](#) This field is visible only if [Queue](#) Type is [Attribute](#). The attribute entered in this field will be evaluated to indicate which queue is to be used. Expression This field is visible only if [Queue](#) Type is Expression. The expression entered in this field will be evaluated to indicate which queue is to be used.

10.610.2 Member Function Documentation

10.610.2.1 `_check()`

```
bool Seize::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.610.2.2 `_createEditableDataDefinitions()`

```
void Seize::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.610.2.3 `_initBetweenReplications()`

```
void Seize::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.610.2.4 `_loadInstance()`

```
bool Seize::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.610.2.5 `_onDispatchEvent()`

```
void Seize::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for [ModelComponents](#), not for [ModelDataElements](#).

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.610.2.6 `_saveInstance()`

```
void Seize::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.610.2.7 `show()`

```
std::string Seize::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

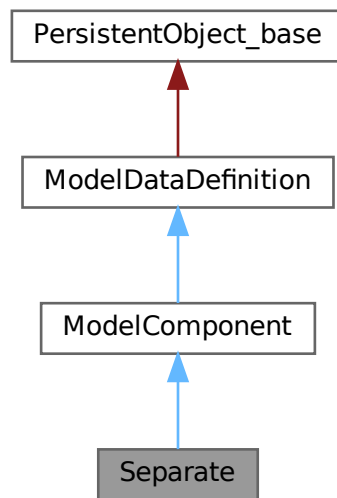
The documentation for this class was generated from the following files:

- `plugins/components/DiscreteProcessing/Seize.h`
- `plugins/components/DiscreteProcessing/Seize.cpp`

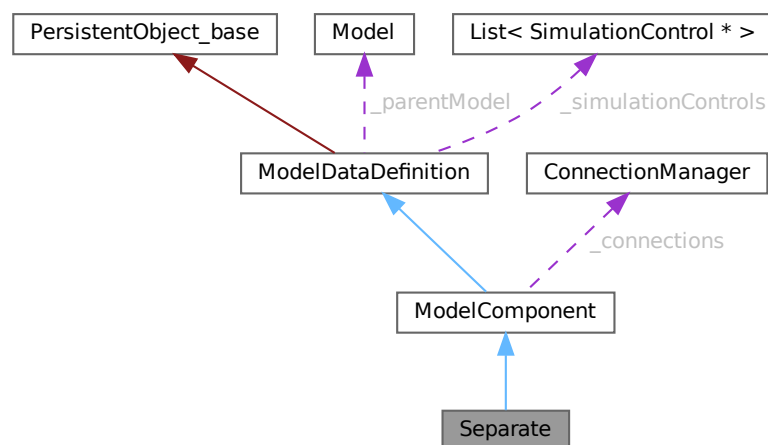
10.611 Separate Class Reference

```
#include <plugins/components/Grouping/Separate.h>
```


Inheritance diagram for Separate:



Collaboration diagram for Separate:



Public Member Functions

- **Separate** (`Model` *model, `std::string` name="")
- virtual `std::string` `show` () override
virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const
Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const
Returns the full map of attached data definitions.

- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelSpecific`, bool `traceLevelSpecificEnabled`)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation` ()
- static `ModelComponent * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
- static `ModelDataDefinition * NewInstance` (`Model *model`, `std::string name=""`)

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance` (`Model *model`, `PersistenceRecord *fields`)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent *component`)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent *component`)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord *fields`, `ModelComponent *component`)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event *event`)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.611.1 Detailed Description

[Separate](#) module DESCRIPTION This module can be used to either copy an incoming entity into multiple entities or to split a previously batched entity. Rules for allocating costs and times to the duplicate are also specified. Rules for attribute assignment to member entities are specified as well. When splitting existing batches, the temporary representative entity that was formed is disposed and the original entities that formed the group are recovered. The entities proceed sequentially from the module in the same order in which they originally were added to the batch. When duplicating entities, the specified number of copies is made and sent from the module. The original incoming entity also leaves the module. TYPICAL USES Send individual entities to represent boxes removed from a container Send an order both to fulfillment and billing for parallel processing [Separate](#) a previously batched set of documents PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Type Method of separating the incoming entity. Duplicate Original will simply take the original entity and make some number of identical duplicates. Split Existing [Batch](#) requires that the incoming entity be a temporarily batched entity using the [Batch](#) module. The original entities from the batch will be split. Percent Cost to Duplicates Allocation of costs and times of the incoming entity to the outgoing duplicates. This value is specified as a percentage of the original entity's costs and times (between 0-100). The percentage specified will be split evenly between the duplicates, while the original entity will retain any remaining cost/time percentage. Visible only when Type is Duplicate Original.

10.611.2 of Duplicates Number of outgoing entities that will leave the module, in

addition to the original incoming entity. Applies only when Type is Duplicate Original. Member Attributes Method of determining how to assign the representative entity attribute values to the original entities. These options relate to six of the special-purpose attributes (Entity.Type, Entity.Picture, Entity.Sequence, Entity.Station, Entity.Jobstep, and Entity.HoldCostRate) and all user-defined attributes. Applies only when Type is Split Existing [Batch](#). [Attribute](#) Name Name of representative entity attribute(s) that are assigned to original entities of the group. Applies only when Member Attributes is Take Specific Representative Values.

10.611.3 Member Function Documentation

10.611.3.1 _check()

```
bool Separate::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.611.3.2 _loadInstance()

```
bool Separate::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.611.3.3 _onDispatchEvent()

```
void Separate::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.611.3.4 _saveInstance()

```
void Separate::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.611.3.5 show()

```
std::string Separate::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

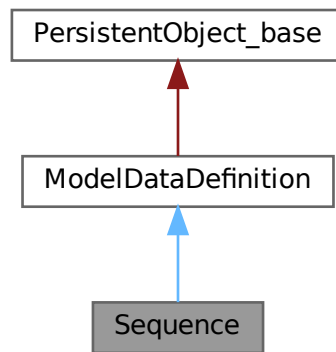
The documentation for this class was generated from the following files:

- plugins/components/Grouping/Separate.h
- plugins/components/Grouping/Separate.cpp

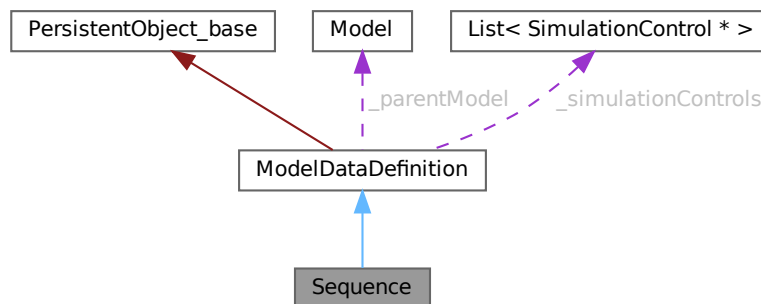
10.612 Sequence Class Reference

```
#include <plugins/data/MaterialHandling/Sequence.h>
```

Inheritance diagram for Sequence:



Collaboration diagram for Sequence:



Public Member Functions

- **Sequence** ([Model](#) *model, std::string name="")
- virtual std::string [show](#) () override
Returns a textual representation of this data definition.
- [List< SequenceStep * > * getSteps](#) () const

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()

Releases resources owned by this data definition.
- Util::identification [getId](#) () const

Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)

Updates the logical name of this instance.
- const std::string & [getName](#) () const

Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const

Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const

Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const

Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const

Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const

Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const

Returns the full map of attached data definitions.
- bool [hasChanged](#) () const

Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))

Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const

Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const

Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)

Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_createAttachedAttributes** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_initBetweenReplications** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createNonEditableDataDefinitions** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()

- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.612.1 Detailed Description

Sequence module DESCRIPTION The **Sequence** module is used to define a sequence for entity flow through the model. A sequence consists of an ordered list of stations that an entity will visit. For each station in the visitation sequence, attributes and variables may be assigned values. Each station in the visitation sequence is referred to as a step (or jobstep) in the sequence. Three special-purpose attributes are provided for all entities. The **Sequence** attribute (Entity.Sequence) defines the sequence that an entity is to follow; a value of 0 indicates that the entity is not following any sequence. In order for an entity to follow a sequence, its **Sequence** attribute must be assigned a value (for example, in the **Assign** module). The Jobstep attribute (Entity.Jobstep) stores the entity's current step number in the sequence. This value is updated automatically each time an entity is transferred. You typically do not need to assign explicitly a value to Jobstep in the model. The PlannedStation attribute (Entity.PlannedStation) stores the number of the station associated with the next jobstep in the sequence. This attribute is not user-assignable. It is automatically updated whenever Entity.Sequence or Entity.JobStep changes, or whenever the entity enters a station. Jobstep names must be globally unique. TYPICAL USES Define a routing path for part processing Define a sequence of steps patients must take upon arrival at an emergency room

10.612.2 Member Function Documentation

10.612.2.1 _check()

```
bool Sequence::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.612.2.2 _createAttachedAttributes()

```
void Sequence::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.612.2.3 _loadInstance()

```
bool Sequence::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.612.2.4 _saveInstance()

```
void Sequence::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.612.2.5 show()

```
std::string Sequence::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

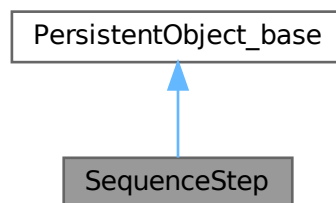
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

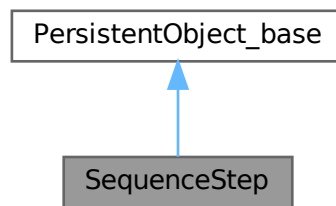
- plugins/data/MaterialHandling/Sequence.h
- plugins/data/MaterialHandling/Sequence.cpp

10.613 SequenceStep Class Reference

Inheritance diagram for SequenceStep:



Collaboration diagram for SequenceStep:



Public Member Functions

- **SequenceStep** ([Station](#) *station, std::list< [Assignment](#) * > *assignments=nullptr)
- **SequenceStep** ([Label](#) *label, std::list< [Assignment](#) * > *assignments=nullptr)
- **SequenceStep** ([Model](#) *model, std::string stationOrLabelName, bool isStation=true, std::list< [Assignment](#) * > *assignments=nullptr)
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields, unsigned int parentIndex)
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, unsigned int parentIndex, bool saveDefaultValues)
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
Loads object fields from a deserialization structure.
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- std::list< [Assignment](#) * > * **getAssignments** () const
- void **setStation** ([Station](#) *_station)
- [Station](#) * **getStation** () const
- void **setElementManager** ([ModelDataManager](#) *_modelDataManager)
- void **setLabel** ([Label](#) *_label)
- [Label](#) * **getLabel** () const

Additional Inherited Members

10.613.1 Member Function Documentation

10.613.1.1 _loadInstance()

```
bool SequenceStep::_loadInstance (
    PersistenceRecord * fields ) [override], [virtual]
```

Loads object fields from a deserialization structure.

Parameters

<i>fields</i>	a table-like structure which can be queried for parsed fields.
---------------	--

Returns

a boolean which is true iff the load operation was successful.

Implements [PersistentObject_base](#).

10.613.1.2 _saveInstance()

```
void SequenceStep::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Implements [PersistentObject_base](#).

The documentation for this class was generated from the following files:

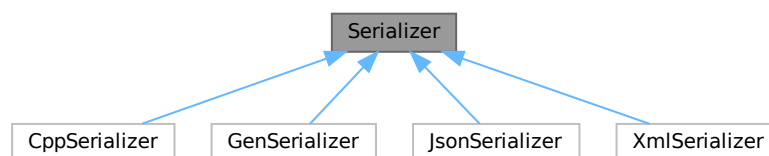
- plugins/data/MaterialHandling/Sequence.h
- plugins/data/MaterialHandling/Sequence.cpp

10.614 Serializer Class Reference

Serialization/Deserialization interface for simulation models.

```
#include <kernel/simulator/persistence/Serializer.h>
```

Inheritance diagram for Serializer:



Public Member Functions

- virtual [PersistenceRecord](#) * [newPersistenceRecord](#) ()=0
Creates an empty persistence record.
- virtual bool [dump](#) (std::ostream &output)=0
Writes formatted contents to a given output stream.
- virtual bool [load](#) (std::istream &input)=0
Attempts to load contents from a given input stream.
- virtual bool [get](#) (const std::string &name, [PersistenceRecord](#) *entry)=0
Tries to fetch an entry with the given label as a key.
- virtual bool [put](#) (const std::string name, const std::string type, const Util::identification id, [PersistenceRecord](#) *fields)=0
Upserts an entry into this structure.
- virtual int [for_each](#) (std::function< int(const std::string &) > delegate)=0
Iterates over this structure's entries, ordered by ID.

10.614.1 Detailed Description

Serialization/Deserialization interface for simulation models.

This can be seen as an associative array which knows how to ser/deserialize itself. The "contents" of a serializer are the set of entries it contains.

Its entries are identified by strings, which correspond to unique component/datadef names. Each entry has its own name, a type, an id, and an associated set of fields. All of these are contained within a [PersistenceRecord](#) instance.

Serializers also act as factories for [PersistenceRecords](#), which can then only be used with the serializer which originated them.

10.614.2 Member Function Documentation

10.614.2.1 [dump\(\)](#)

```
virtual bool Serializer::dump (
    std::ostream & output ) [pure virtual]
```

Writes formatted contents to a given output stream.

Parameters

<i>output</i>	output stream to write to.
---------------	----------------------------

Returns

whether the current state of this structure's contents was deemed valid for serialization.

Implemented in [CppSerializer](#), [GenSerializer](#), [JsonSerializer](#), and [XmlSerializer](#).

10.614.2.2 for_each()

```
virtual int Serializer::for_each (
    std::function< int(const std::string &) > delegate ) [pure virtual]
```

Iterates over this structure's entries, ordered by ID.

Parameters

<i>delegate</i>	delegate which is applied to every entry-identifying key, until it returns a non-zero value.
-----------------	--

Returns

either zero, or the first non-zero value returned by the delegate.

Implemented in [CppSerializer](#), [GenSerializer](#), [JsonSerializer](#), and [XmlSerializer](#).

10.614.2.3 get()

```
virtual bool Serializer::get (
    const std::string & name,
    PersistenceRecord * entry ) [pure virtual]
```

Tries to fetch an entry with the given label as a key.

Parameters

<i>name</i>	a label which uniquely identifies an entry.
<i>entry</i>	(out-param) a pointer to a caller-owned entry, which is filled on success.

Returns

whether the entry was found and the out-param was actually written to.

Implemented in [CppSerializer](#), [GenSerializer](#), [JsonSerializer](#), and [XmlSerializer](#).

10.614.2.4 load()

```
virtual bool Serializer::load (
    std::istream & input ) [pure virtual]
```

Attempts to load contents from a given input stream.

Returns

whether the input was well-formatted and loading was successful.

Implemented in [CppSerializer](#), [GenSerializer](#), [JsonSerializer](#), and [XmlSerializer](#).

10.614.2.5 newPersistenceRecord()

```
virtual PersistenceRecord * Serializer::newPersistenceRecord ( ) [pure virtual]
```

Creates an empty persistence record.

Although the returned record can be freed normally with `delete`, care must be taken to only use records which come from a implementation, when configuring that implementation's contents.

Implemented in [CppSerializer](#), [GenSerializer](#), [JsonSerializer](#), and [XmlSerializer](#).

10.614.2.6 put()

```
virtual bool Serializer::put (
    const std::string name,
    const std::string type,
    const Util::identification id,
    PersistenceRecord * fields ) [pure virtual]
```

Upserts an entry into this structure.

Parameters

<i>name</i>	a label which uniquely identifies the entry.
<i>type</i>	a type name.
<i>id</i>	a component/datadef ID number.
<i>fields</i>	a persistence record for the entry, whose contents will be copied if valid.

Returns

whether the entry was valid and was actually copied into this structure.

Implemented in [CppSerializer](#), [GenSerializer](#), [JsonSerializer](#), and [XmlSerializer](#).

The documentation for this class was generated from the following files:

- kernel/simulator/persistence/Serializer.h
- kernel/simulator/persistence/Serializer.cpp

10.615 SessionContext Struct Reference

Owns the state that belongs to one authenticated web session.

```
#include <applications/worker/session/SessionContext.h>
```

Public Attributes

- `std::string` **sessionId**
Stable session identifier returned to the client.
- `std::string` **accessToken**
Bearer token used to authenticate API calls.
- `std::filesystem::path` **workspacePath**
Filesystem workspace reserved for the session.
- `std::unique_ptr< Simulator >` **simulator**
[Simulator](#) instance scoped to this session.
- `std::mutex` **mutex**
Guards session-local mutable state.

10.615.1 Detailed Description

Owns the state that belongs to one authenticated web session.

Each session binds one access token to one workspace directory and one simulator instance. The mutex protects session-local state during API calls and background job execution.

The documentation for this struct was generated from the following file:

- `applications/worker/session/SessionContext.h`

10.616 SessionManager Class Reference

Creates, stores and resolves authenticated web sessions.

```
#include <applications/worker/session/SessionManager.h>
```

Public Types

- using **SimulatorFactory** = `std::function< std::unique_ptr< Simulator >()>`
Factory used to build one simulator per session.

Public Member Functions

- [SessionManager](#) ([TokenService](#) &tokenService, [SimulatorFactory](#) simulatorFactory)
Builds a manager with the token service and simulator factory to use.
- [SessionContext](#) * [createSession](#) ()
Creates a new session, workspace and simulator instance.
- [SessionContext](#) * [getSessionByToken](#) (const `std::string` &token)
Resolves the session associated with a bearer token.

10.616.1 Detailed Description

Creates, stores and resolves authenticated web sessions.

[SessionManager](#) owns the token-to-session map used by the web API. It also creates per-session simulator instances and assigns each session an isolated workspace directory.

10.616.2 Constructor & Destructor Documentation

10.616.2.1 SessionManager()

```
SessionManager::SessionManager (
    TokenService & tokenService,
    SimulatorFactory simulatorFactory )
```

Builds a manager with the token service and simulator factory to use.

Builds the session registry and session workspace root.

Parameters

<i>tokenService</i>	Service responsible for generating and validating tokens.
<i>simulatorFactory</i>	Factory used to create a fresh simulator per session.

10.616.3 Member Function Documentation

10.616.3.1 createSession()

```
SessionContext * SessionManager::createSession ( )
```

Creates a new session, workspace and simulator instance.

Creates a session, allocates token, simulator instance and workspace.

Returns

Pointer to the stored session context.

10.616.3.2 getSessionByToken()

```
SessionContext * SessionManager::getSessionByToken (
    const std::string & token )
```

Resolves the session associated with a bearer token.

Resolves a session by bearer token.

Parameters

<i>token</i>	Bearer token presented by the client.
--------------	---------------------------------------

Returns

Pointer to the matching session or nullptr when not found.

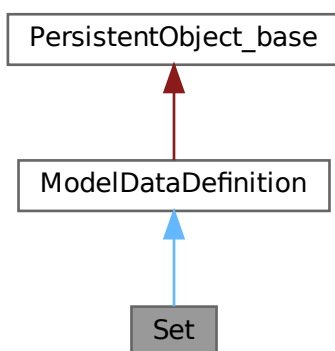
The documentation for this class was generated from the following files:

- applications/worker/session/SessionManager.h
- applications/worker/session/SessionManager.cpp

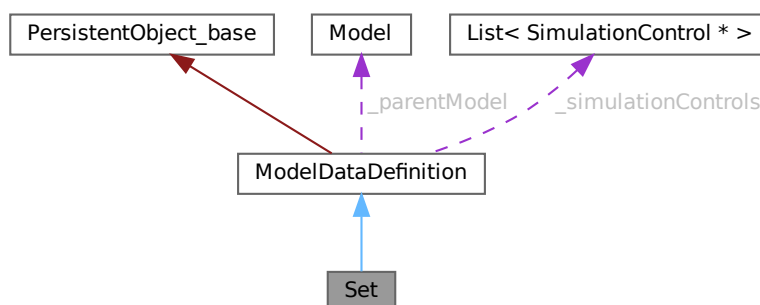
10.617 Set Class Reference

```
#include <plugins/data/Logic/Set.h>
```

Inheritance diagram for Set:



Collaboration diagram for Set:



Public Member Functions

- **Set** ([Model](#) *model, std::string name="", std::vector< std::string > allowedElementTypes={})
- virtual std::string [show](#) () override
Returns a textual representation of this data definition.
- std::string [getSetOfType](#) () const
- bool [setSetOfType](#) (const std::string &setOfType)
Sets the concrete member type for this [Set](#) while it is still structurally compatible.
- bool [canChangeSetOfType](#) () const
Returns true while the selected concrete member type can still be changed.
- void [setAllowedElementTypes](#) (const std::vector< std::string > &allowedElementTypes)
Restricts which [ModelDataDefinition](#) subclasses may be created through this [Set](#).
- void [addAllowedElementType](#) (const std::string &allowedElementType)
Adds one accepted [ModelDataDefinition](#) subclass name to this [Set](#) creation contract.
- std::vector< std::string > [getAllowedElementTypes](#) () const
Returns the currently accepted concrete member types for GUI and tooling contracts.
- [List](#)< [ModelDataDefinition](#) * > * [getElementSet](#) () const
- void [addElementSet](#) ([ModelDataDefinition](#) *newElement)
- void [removeElementSet](#) ([ModelDataDefinition](#) *element)
- [ModelDataDefinition](#) * [createElementSetOfType](#) (const std::string &typeName, const std::string &name="")
Creates a new model data definition of the selected [Set](#) member type.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.

- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelDataDefinition](#)

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` () override
- virtual void `_createAttachedAttributes` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.617.1 Detailed Description

Set module DESCRIPTION This data module defines various types of sets, including resource, counter, tally, entity type, and entity picture. [Resource](#) sets can be used in the [Process](#) modules (and [Seize](#), [Release](#), [Enter](#), and [Leave](#) of the Advanced [Process](#) and Advanced Transfer panels). [Counter](#) and Tally sets can be used in the [Record](#) module. [Queue](#) sets can be used with the [Seize](#), Hold, [Access](#), Request, [Leave](#), and Allocate modules of the Advanced [Process](#) and Advanced Transfer panels. TYPICAL USES Machines that can perform the same operations in a manufacturing facility Supervisors, check-out clerks in a store Shipping clerks, receptionists in an office **Set** of pictures corresponding to a set of entity types PROMPTS Prompt Description Name The unique name of the set being defined. Type Type of set being defined. Members Repeat group that specifies the resource members with the set. The order of listing the members within the repeat group is important when using selection rules such as Preferred Order and Cyclical. [Resource](#) Name Name of the resource to include in the resource set. Applies only when Type is [Resource](#). Tally Name Name of the tally within the tally set. Applies only when Type is Tally. [Counter](#) Name Name of the counter within the counter set. Applies only when Type is [Counter](#). [Entity](#) Type Name of the entity type within the entity type set. Applies only when Type is [Entity](#). Picture Name Name of the picture within the picture set. Applies only when Type is [Entity](#) Picture.

10.617.2 Member Function Documentation

10.617.2.1 **_check()**

```
bool Set::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.617.2.2 _createAttachedAttributes()

```
void Set::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.617.2.3 _getParserChangesInformation()

```
ParserChangesInformation * Set::_getParserChangesInformation ( ) [override], [protected], [virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

10.617.2.4 _loadInstance()

```
bool Set::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.617.2.5 _saveInstance()

```
void Set::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.617.2.6 createElementSetOfType()

```
ModelDataDefinition * Set::createElementSetOfType (
    const std::string & typeName,
    const std::string & name = "" )
```

Creates a new model data definition of the selected [Set](#) member type.

This method is the kernel-side factory used by generic property editors. The editor provides the intended type; [Set](#) validates the current type/allowed-type contract and delegates actual object construction to [PluginManager](#) so plugin registration rules stay centralized.

10.617.2.7 setAllowedElementTypes()

```
void Set::setAllowedElementTypes (
    const std::vector< std::string > & allowedElementTypes )
```

Restricts which [ModelDataDefinition](#) subclasses may be created through this [Set](#).

Empty means "use every creatable [ModelDataDefinition](#) plugin known by the simulator". Context owners such as [SeizableItem](#) and [QueueableItem](#) use this to expose Resource-only or Queue-only [Set](#) editing in the GUI without hard-coding those rules in the editor.

10.617.2.8 setSetOfType()

```
bool Set::setSetOfType (
    const std::string & setOfType )
```

Sets the concrete member type for this [Set](#) while it is still structurally compatible.

A [Set](#) is polymorphic only before its first member is inserted. Once members exist, all subsequent members must match this selected [ModelDataDefinition](#) subclass name.

10.617.2.9 show()

```
std::string Set::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal [ModelDataDefinitions](#) such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

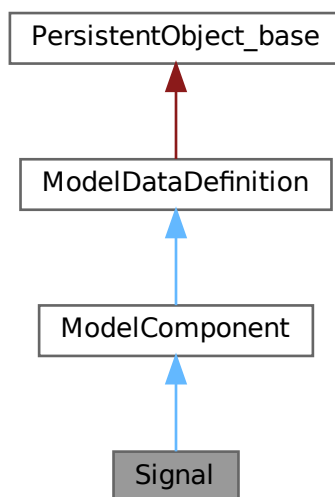
The documentation for this class was generated from the following files:

- plugins/data/Logic/Set.h
- plugins/data/Logic/Set.cpp

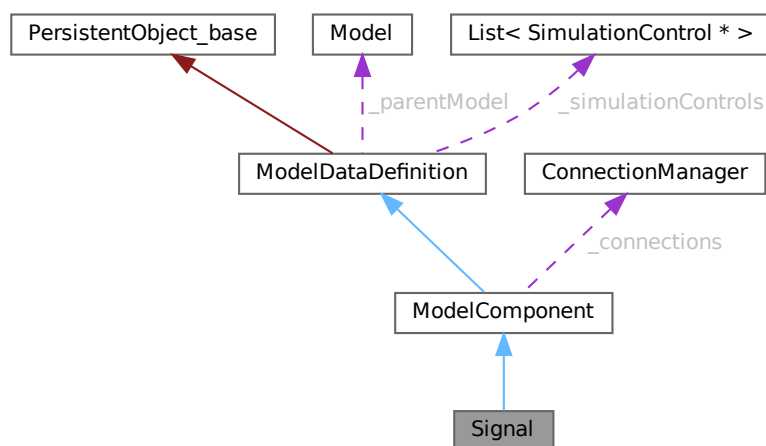
10.618 Signal Class Reference

```
#include <plugins/components/Synchronization/Signal.h>
```

Inheritance diagram for Signal:



Collaboration diagram for Signal:



Public Member Functions

- **Signal** ([Model](#) *model, std::string name="")

- virtual std::string **show** () override
virtual public methods
- void **setSignalData** (SignalData *signal)
- const std::string & **limitExpression** () const
- void **setLimitExpression** (const std::string &newLimitExpression)

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (Model *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (ModelComponent *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (ModelComponent *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (Model *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- Model * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

- Enables or disables report-statistics generation for this definition.*

 - `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
 - `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
 - `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
 - `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
 - `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
 - `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
 - `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
 - `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
 - `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
 - `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
 - `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
 - `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
 - `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation ()`
- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `bool Check (ModelComponent *component)`
Validates a component instance.
- static `void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- static `void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- static `void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.618.1 Detailed Description

Signal module DESCRIPTION The **Signal** module sends a signal value to each Hold module in the model set to **Wait** for **Signal** and releases the maximum specified number of entities. When an entity arrives at a **Signal** module, the signal is evaluated and the signal code is sent. At this time, entities at Hold modules that are waiting for the same signal are removed from their queues. The entity sending the signal continues processing until it encounters a delay, enters a queue, or is disposed. TYPICAL USES Analyzing traffic patterns at an intersection (signal when the light turns green) Signaling an operator to complete an order that was waiting for a component part PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. **Signal** Value Value of the signal to be sent to entities in Hold modules. Limit Maximum number of entities that are to be released from any Hold modules when the signal is received.

10.618.2 Member Function Documentation

10.618.2.1 _check()

```
bool Signal::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.618.2.2 _createEditableDataDefinitions()

```
void Signal::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.618.2.3 _initBetweenReplications()

```
void Signal::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.618.2.4 _loadInstance()

```
bool Signal::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.618.2.5 _onDispatchEvent()

```
void Signal::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.618.2.6 `_saveInstance()`

```
void Signal::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.618.2.7 `show()`

```
std::string Signal::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

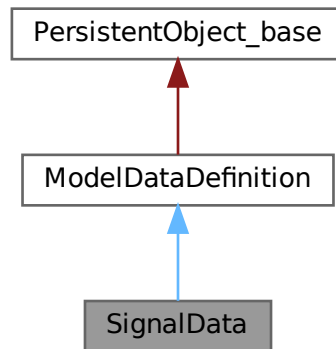
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

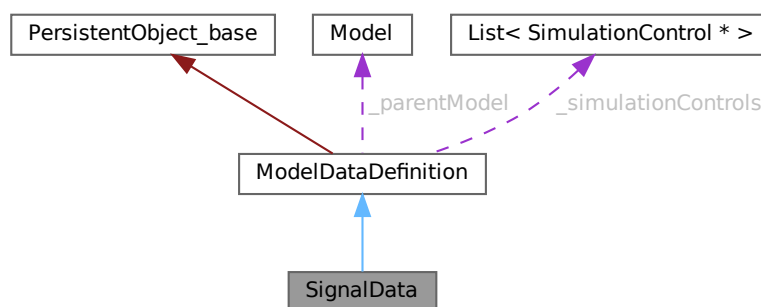
- plugins/components/Synchronization/Signal.h
- plugins/components/Synchronization/Signal.cpp

10.619 SignalData Class Reference

Inheritance diagram for SignalData:



Collaboration diagram for SignalData:



Public Types

- typedef std::function< unsigned int([SignalData](#) *) > **SignalDataEventHandler**
- typedef std::pair< SignalDataEventHandler, [ModelComponent](#) * > **PairSignalDataEventHandler**

Public Member Functions

- **SignalData** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- unsigned int **generateSignal** (double signalValue, unsigned int limit)
- void **addSignalDataEventHandler** (SignalDataEventHandler eventHandler, [ModelComponent](#) *component)
- void **removeSignalDataEventHandler** ([ModelComponent](#) *component)
- bool **hasSignalDataEventHandler** ([ModelComponent](#) *component) const
- unsigned int **remainsToLimit** () const
- void **decreaseRemainLimit** ()

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- `template<typename Class >`
`static SignalDataEventHandler SetSignalDataEventHandler (unsigned int(Class::*function)(SignalData *), Class *object)`
- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelDataDefinition](#)

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- `virtual bool _loadInstance (PersistenceRecord *fields) override`
must be overridden by derived classes
- `virtual void _saveInstance (PersistenceRecord *fields, bool saveDefaultValues) override`
Saves object fields into a serialization structure.
- `virtual bool _check (std::string &errorMessage) override`
could be overridden by derived classes
- `virtual void _initBetweenReplications () override`

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createInternalStatisticReporters ()`
- `virtual void _createAttachedAttributes ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `virtual void _createEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`

- void **_internaStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.619.1 Member Function Documentation

10.619.1.1 `_check()`

```
bool SignalData::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.619.1.2 `_initBetweenReplications()`

```
void SignalData::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.619.1.3 `_loadInstance()`

```
bool SignalData::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.619.1.4 `_saveInstance()`

```
void SignalData::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.619.1.5 `show()`

```
std::string SignalData::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- `plugins/data/Synchronization/SignalData.h`
- `plugins/data/Synchronization/SignalData.cpp`

10.620 **GroProgramRuntime::SignalMutation Struct Reference**

Public Attributes

- `SignalMutationType` **type** = `SignalMutationType::Emit`
- `double` **value** = `0.0`

The documentation for this struct was generated from the following file:

- `plugins/data/BiochemicalSimulation/GroProgramRuntime.h`

10.621 **SimpleHttpServer Class Reference**

Minimal blocking HTTP/1.1 server for single-request-per-connection handling.

```
#include <applications/worker/http/SimpleHttpServer.h>
```

Public Types

- using **RequestHandler** = `std::function< HttpResponse(const HttpRequest &)>`

Public Member Functions

- [SimpleHttpServer](#) (unsigned short port)
Construct a server bound to a TCP port.
- bool [serve](#) (const RequestHandler &handler, unsigned long maxRequests=0)
Serve HTTP requests until the optional request limit is reached.
- void **requestStop** ()
Requests the serving loop to stop as soon as it reaches a safe point.

10.621.1 Detailed Description

Minimal blocking HTTP/1.1 server for single-request-per-connection handling.

This server intentionally keeps the transport model simple (no keep-alive, no chunked transfer decoding) and delegates application routing to a request handler callback.

10.621.2 Constructor & Destructor Documentation

10.621.2.1 SimpleHttpServer()

```
SimpleHttpServer::SimpleHttpServer (
    unsigned short port ) [explicit]
```

Construct a server bound to a TCP port.

Parameters

<i>port</i>	TCP port that the server will bind/listen on.
-------------	---

10.621.3 Member Function Documentation

10.621.3.1 serve()

```
bool SimpleHttpServer::serve (
    const RequestHandler & handler,
    unsigned long maxRequests = 0 )
```

Serve HTTP requests until the optional request limit is reached.

Serves one request at a time over a blocking TCP listener.

Parameters

<i>handler</i>	Callback used to transform a parsed request into a response.
<i>maxRequests</i>	Maximum number of client requests to process (0 means unlimited).

Returns

true when the serving loop ends normally; false on socket setup/runtime failure.

The loop accepts a connection, reads a bounded HTTP/1.1 request, delegates to the provided handler, serializes the response and closes the connection.

The documentation for this class was generated from the following files:

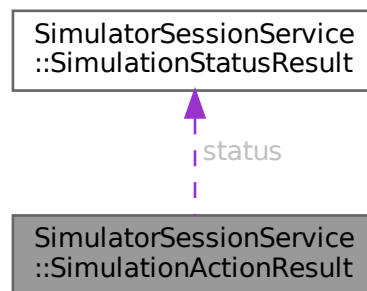
- applications/worker/http/SimpleHttpServer.h
- applications/worker/http/SimpleHttpServer.cpp

10.622 SimulatorSessionService::SimulationActionResult Struct Reference

Contains the result of a simulation action such as run or step.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::SimulationActionResult:



Public Attributes

- bool **success** = false
- bool **invalidToken** = false
- bool **missingCurrentModel** = false
- [SimulationStatusResult](#) **status**

10.622.1 Detailed Description

Contains the result of a simulation action such as run or step.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

10.623 SimulationCommandController Class Reference

Controller that orchestrates UI simulation commands over [SimulationController](#) guards.

```
#include <applications/gui/genesys/controllers/SimulationCommandController.h>
```

Public Member Functions

- **SimulationCommandController** ([SimulationController](#) *simulationController, const std::function< void(const std::string &)> &insertCommandInConsole, const std::function< void()> &actualizeActions, const std::function< bool()> &checkModel, const std::function< bool()> &setSimulationModelBasedOnText, const std::function< void()> &prepareNewSimulationOutputs)
Creates a command controller with narrow callbacks from [MainWindow](#).
- void **onActionSimulationStartTriggered** (bool modelChecked) const
Delegates the start simulation command flow.
- void **onActionSimulationStepTriggered** (bool modelChecked) const
Delegates the step simulation command flow.
- void **onActionSimulationPauseTriggered** () const
Delegates the pause simulation command flow.
- void **onActionSimulationResumeTriggered** (bool modelChecked) const
Delegates the resume simulation command flow.
- void **onActionSimulationStopTriggered** () const
Delegates the stop simulation command flow.

10.623.1 Detailed Description

Controller that orchestrates UI simulation commands over [SimulationController](#) guards.

This class sits between [MainWindow](#) compatibility slots and the low-level simulation readiness controller. It coordinates command intent (start/step/pause/resume/stop), command logging, and action refresh callbacks while preserving the existing UI trigger surface.

Responsibilities:

- execute simulation command flows delegated from [MainWindow](#);
- use [SimulationController](#) precondition checks before unsafe commands;
- invoke injected callbacks for console command logging and action-state refresh.

Boundaries:

- it does not own simulation state or kernel event wiring;
- it does not render trace/event widgets;
- it is a controller/orchestration layer, not a persistence or synchronization service.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/SimulationCommandController.h
- applications/gui/genesys/controllers/SimulationCommandController.cpp

10.624 SimulatorSessionService::SimulationConfigInput Struct Reference

Defines user-provided simulation parameters to apply to the current model.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Public Attributes

- unsigned int **numberOfReplications** = 0
- double **replicationLength** = 0.0
- double **warmUpPeriod** = 0.0
- bool **pauseOnEvent** = false
- bool **pauseOnReplication** = false
- bool **initializeStatistics** = false
- bool **initializeSystem** = false

10.624.1 Detailed Description

Defines user-provided simulation parameters to apply to the current model.

The documentation for this struct was generated from the following file:

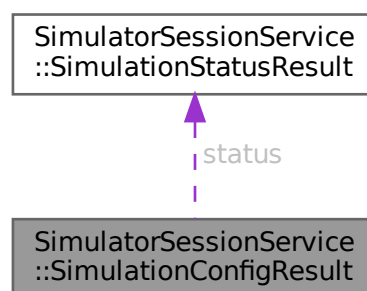
- applications/worker/service/SimulatorSessionService.h

10.625 SimulatorSessionService::SimulationConfigResult Struct Reference

Contains the result of applying simulation configuration.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::SimulationConfigResult:



Public Attributes

- bool **success** = false
- bool **invalidToken** = false
- bool **missingCurrentModel** = false
- [SimulationStatusResult](#) **status**

10.625.1 Detailed Description

Contains the result of applying simulation configuration.

The documentation for this struct was generated from the following file:

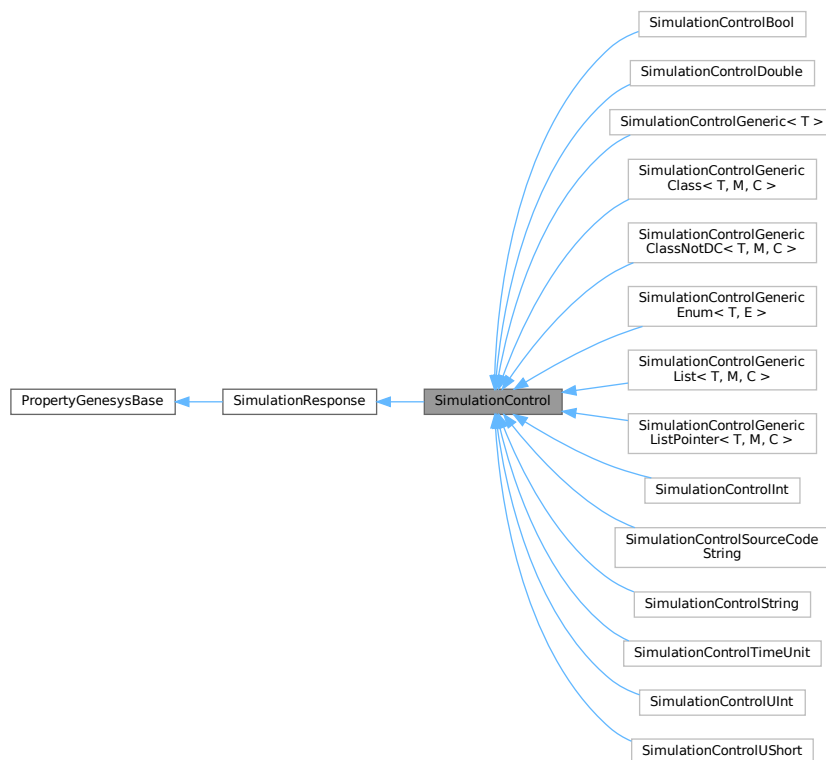
- applications/worker/service/SimulatorSessionService.h

10.626 SimulationControl Class Reference

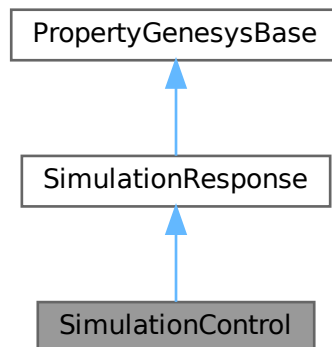
Read/write kernel-side simulation control abstraction.

```
#include <kernel/simulator/SimulationControlAndResponse.h>
```

Inheritance diagram for SimulationControl:



Collaboration diagram for SimulationControl:



Public Member Functions

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual void **setValue** (std::string value, bool remove=false)=0
- virtual List< SimulationControl * > * **getChildSimulationControls** (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual ModelDataDefinition * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual SimulationControlEditorHint **preferredEditorHint** () const
- void **setPreferredEditorHint** (SimulationControlEditorHint hint)
- virtual ModelDataDefinition * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual List< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual List< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual List< SimulationControl * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual std::string **getValue** () const =0
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Protected Member Functions

- void **_ensureWritable** (const char *operation) const

Protected Attributes

- bool **_readonly** = false
- SimulationControlEditorHint **_preferredEditorHint** = SimulationControlEditorHint::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.626.1 Detailed Description

Read/write kernel-side simulation control abstraction.

[SimulationControl](#) extends [SimulationResponse](#) with setter-based mutation and therefore represents writable kernel-side experiment controls.

10.626.2 Member Function Documentation

10.626.2.1 getChildSimulationControls()

```
virtual List< SimulationControl * > * SimulationControl::getChildSimulationControls (
    int index = 0 ) [inline], [virtual]
```

Returns nested SimulationControls exposed by this control.

This is the kernel-facing API for traversing child controls belonging to inline objects, references, or list elements represented by a control.

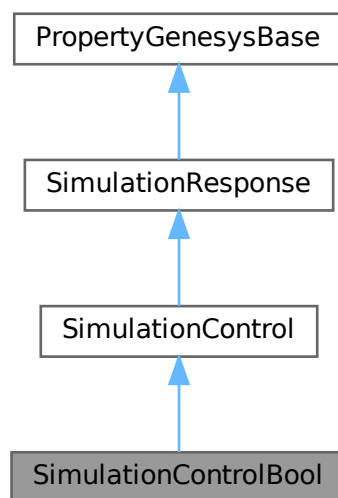
Reimplemented in [SimulationControlGenericClass< T, M, C >](#), [SimulationControlGenericClassNotDC< T, M, C >](#), and [SimulationControlGenericListPointer< T, M, C >](#).

The documentation for this class was generated from the following file:

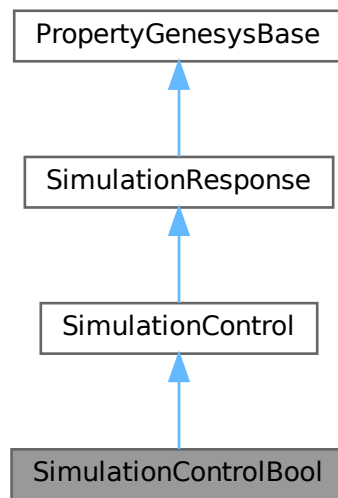
- kernel/simulator/SimulationControlAndResponse.h

10.627 SimulationControlBool Class Reference

Inheritance diagram for SimulationControlBool:



Collaboration diagram for SimulationControlBool:



Public Member Functions

- **SimulationControlBool** (GetterBool getter, SetterBool setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- virtual std::string **getValue** () const override
- virtual void **setValue** (std::string value, bool remove=false) override

Public Member Functions inherited from **SimulationControl**

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual **List**< **SimulationControl** * > * **getChildSimulationControls** (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual **ModelDataDefinition** * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual **SimulationControlEditorHint** **preferredEditorHint** () const
- void **setPreferredEditorHint** (**SimulationControlEditorHint** hint)
- virtual **ModelDataDefinition** * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const

- virtual bool **supportsNewListElementCreation** () const
- virtual [List](#)< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint::Default](#)

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.627.1 Member Function Documentation

10.627.1.1 `getValue()`

```
virtual std::string SimulationControlBool::getValue ( ) const [inline], [override], [virtual]
```

Implements [SimulationResponse](#).

10.627.1.2 `setValue()`

```
virtual void SimulationControlBool::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

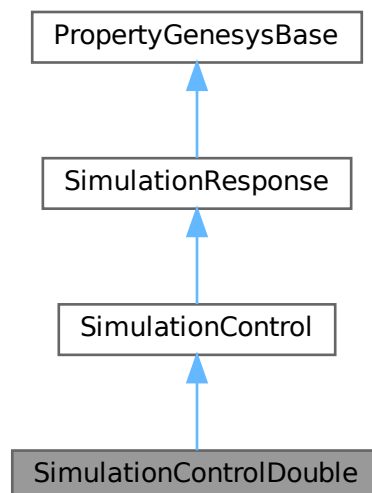
Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

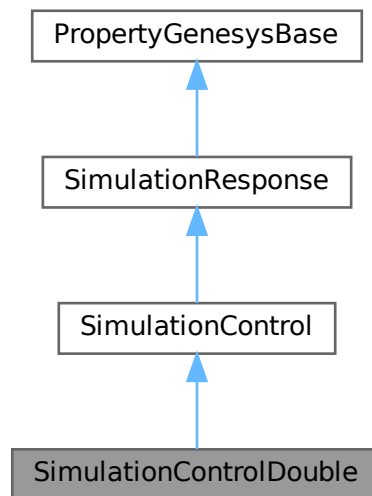
- kernel/simulator/SimulationControlAndResponse.h

10.628 SimulationControlDouble Class Reference

Inheritance diagram for SimulationControlDouble:



Collaboration diagram for SimulationControlDouble:



Public Member Functions

- **SimulationControlDouble** (GetterDouble getter, SetterDouble setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- virtual std::string [getValue](#) () const override
- virtual void [setValue](#) (std::string value, bool remove=false) override

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual [List](#)< [SimulationControl](#) * > * [getChildSimulationControls](#) (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual [ModelDataDefinition](#) * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual [SimulationControlEditorHint](#) **preferredEditorHint** () const
- void **setPreferredEditorHint** ([SimulationControlEditorHint](#) hint)
- virtual [ModelDataDefinition](#) * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const

- virtual bool **supportsNewListElementCreation** () const
- virtual [List](#)< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint::Default](#)

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.628.1 Member Function Documentation

10.628.1.1 `getValue()`

```
virtual std::string SimulationControlDouble::getValue ( ) const [inline], [override], [virtual]
```

Implements [SimulationResponse](#).

10.628.1.2 `setValue()`

```
virtual void SimulationControlDouble::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

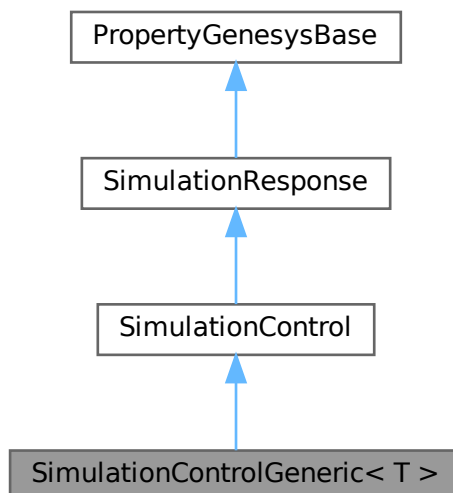
Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

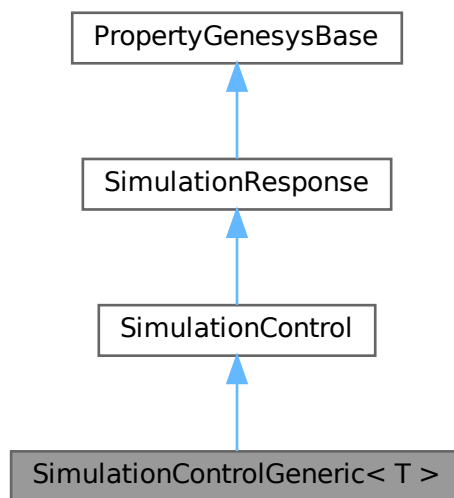
- kernel/simulator/SimulationControlAndResponse.h

10.629 `SimulationControlGeneric< T >` Class Template Reference

Inheritance diagram for `SimulationControlGeneric< T >`:



Collaboration diagram for SimulationControlGeneric< T >:



Public Member Functions

- **SimulationControlGeneric** (GetterGeneric< T > getter, SetterGeneric< T > setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- virtual std::string **getValue** () const override
- virtual void **setValue** (std::string value, bool remove=false) override

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual [List< SimulationControl * >](#) * **getChildSimulationControls** (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual [ModelDataDefinition](#) * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual SimulationControlEditorHint **preferredEditorHint** () const
- void **setPreferredEditorHint** (SimulationControlEditorHint hint)
- virtual [ModelDataDefinition](#) * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const

- virtual bool **supportsNewListElementCreation** () const
- virtual [List](#)< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint](#)::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.629.1 Member Function Documentation

10.629.1.1 [getValue\(\)](#)

```
template<typename T >
virtual std::string SimulationControlGeneric< T >::getValue ( ) const [inline], [override],
[virtual]
```

Implements [SimulationResponse](#).

10.629.1.2 [setValue\(\)](#)

```
template<typename T >
virtual void SimulationControlGeneric< T >::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

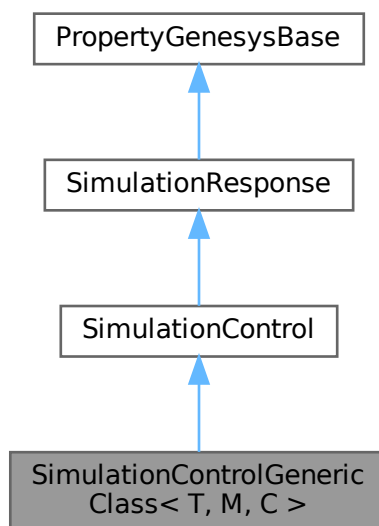
Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

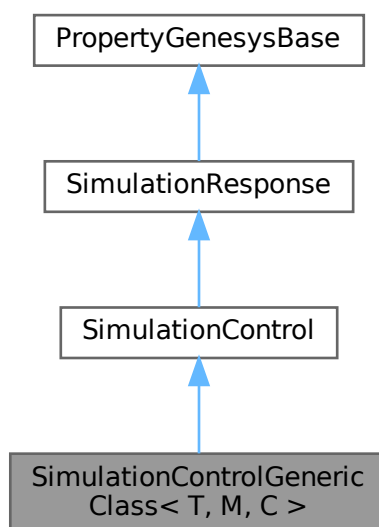
- kernel/simulator/SimulationControlAndResponse.h

10.630 SimulationControlGenericClass< T, M, C > Class Template Reference

Inheritance diagram for SimulationControlGenericClass< T, M, C >:



Collaboration diagram for SimulationControlGenericClass< T, M, C >:



Public Types

- using **Creator** = std::function< T(M, const std::string &)>

Public Member Functions

- **SimulationControlGenericClass** (M model, GetterGeneric< T > getter, SetterGeneric< T > setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=true, bool isEnum=false, Creator creator=nullptr)
- virtual std::string **getValue** () const override
- virtual void **setValue** (std::string value, bool remove=false) override
- virtual bool **isModelDataDefinitionReference** () const override
- virtual **ModelDataDefinition** * **getReferencedModelDataDefinition** () const override
- virtual bool **supportsInlineExpansion** () const override
- virtual bool **supportsExistingObjectSelection** () const override
- virtual bool **supportsObjectCreation** () const override
- virtual bool **isInlineObjectProperty** () const override
- virtual bool **hasObjectInstance** () const override
- virtual bool **ensureObjectInstance** () override
- virtual bool **createObjectInstance** (const std::string &value="") override
- virtual **List**< std::string > * **getStrValues** () override
- virtual **List**< **SimulationControl** * > * **getChildSimulationControls** (int index=0) override

Returns nested SimulationControls exposed by this control.

Public Member Functions inherited from **SimulationControl**

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual **SimulationControlEditorHint** **preferredEditorHint** () const
- void **setPreferredEditorHint** (**SimulationControlEditorHint** hint)
- virtual **ModelDataDefinition** * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual **List**< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual **List**< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual **List**< **SimulationControl** * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from **SimulationResponse**

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- SimulationControlEditorHint **_preferredEditorHint** = SimulationControlEditorHint::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.630.1 Member Function Documentation

10.630.1.1 createObjectInstance()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::createObjectInstance (
    const std::string & value = "" ) [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.630.1.2 ensureObjectInstance()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::ensureObjectInstance ( ) [inline],
[override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.630.1.3 getChildSimulationControls()

```
template<typename T , typename M , typename C >
virtual List< SimulationControl * > * SimulationControlGenericClass< T, M, C >::getChild←
SimulationControls (
    int index = 0 ) [inline], [override], [virtual]
```

Returns nested SimulationControls exposed by this control.

This is the kernel-facing API for traversing child controls belonging to inline objects, references, or list elements represented by a control.

Reimplemented from [SimulationControl](#).

10.630.1.4 getReferencedModelDataDefinition()

```
template<typename T , typename M , typename C >
virtual ModelDataDefinition * SimulationControlGenericClass< T, M, C >::getReferencedModel←
DataDefinition ( ) const [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.630.1.5 getStrValues()

```
template<typename T , typename M , typename C >
virtual List< std::string > * SimulationControlGenericClass< T, M, C >::getStrValues ( )
[inline], [override], [virtual]
```

Reimplemented from [SimulationResponse](#).

10.630.1.6 getValue()

```
template<typename T , typename M , typename C >
virtual std::string SimulationControlGenericClass< T, M, C >::getValue ( ) const [inline],
[override], [virtual]
```

Implements [SimulationResponse](#).

10.630.1.7 hasObjectInstance()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::hasObjectInstance ( ) const [inline],
[override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.630.1.8 isInlineObjectProperty()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::isInlineObjectProperty ( ) const [inline],
[override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.630.1.9 isModelDataDefinitionReference()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::isModelDataDefinitionReference ( )
const [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.630.1.10 setValue()

```
template<typename T , typename M , typename C >
virtual void SimulationControlGenericClass< T, M, C >::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

Implements [SimulationControl](#).

10.630.1.11 supportsExistingObjectSelection()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::supportsExistingObjectSelection ( )
const [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.630.1.12 supportsInlineExpansion()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClass< T, M, C >::supportsInlineExpansion ( ) const
[inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.630.1.13 supportsObjectCreation()

```
template<typename T , typename M , typename C >  
virtual bool SimulationControlGenericClass< T, M, C >::supportsObjectCreation ( ) const [inline],  
[override], [virtual]
```

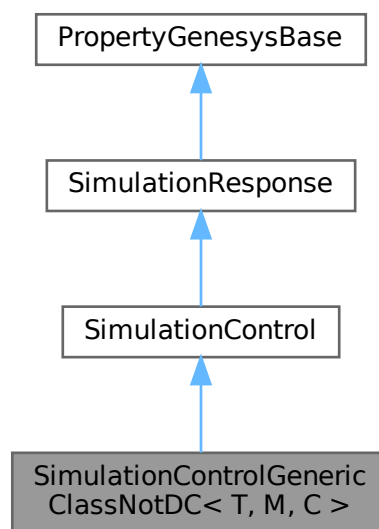
Reimplemented from [SimulationControl](#).

The documentation for this class was generated from the following file:

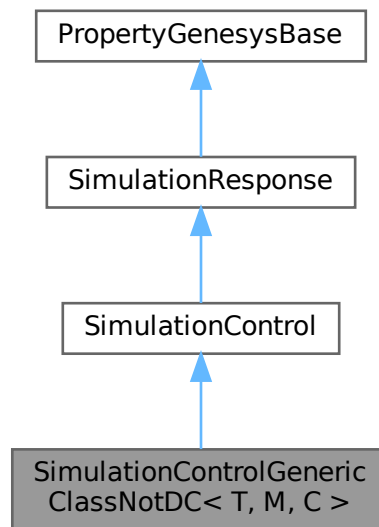
- kernel/simulator/SimulationControlAndResponse.h

10.631 SimulationControlGenericClassNotDC< T, M, C > Class Template Reference

Inheritance diagram for SimulationControlGenericClassNotDC< T, M, C >:



Collaboration diagram for `SimulationControlGenericClassNotDC< T, M, C >`:



Public Types

- using **Creator** = `std::function< T(M)>`
- using **NamedCreator** = `std::function< T(M, const std::string &)>`

Public Member Functions

- **SimulationControlGenericClassNotDC** (M model, GetterGeneric< T > getter, SetterGeneric< T > setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=true, bool isEnum=false, Creator creator=nullptr, NamedCreator namedCreator=nullptr)
- virtual std::string [getValue](#) () const override
- virtual void [setValue](#) (std::string value, bool remove=false) override
- virtual bool [hasObjectInstance](#) () const override
- virtual bool [supportsInlineExpansion](#) () const override
- virtual bool [supportsObjectCreation](#) () const override
- virtual bool [isInlineObjectProperty](#) () const override
- virtual bool [ensureObjectInstance](#) () override
- virtual bool [createObjectInstance](#) (const std::string &value="") override
- virtual [List< SimulationControl * > *](#) [getChildSimulationControls](#) (int index=0) override

Returns nested SimulationControls exposed by this control.

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual bool **isModelDataDefinitionReference** () const
- virtual [ModelDataDefinition](#) * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual SimulationControlEditorHint **preferredEditorHint** () const
- void **setPreferredEditorHint** (SimulationControlEditorHint hint)
- virtual [ModelDataDefinition](#) * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual [List](#)< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- SimulationControlEditorHint **_preferredEditorHint** = SimulationControlEditorHint::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.631.1 Member Function Documentation

10.631.1.1 createObjectInstance()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClassNotDC< T, M, C >::createObjectInstance (
    const std::string & value = "" ) [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.631.1.2 ensureObjectInstance()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClassNotDC< T, M, C >::ensureObjectInstance ( ) [inline],
[override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.631.1.3 getChildSimulationControls()

```
template<typename T , typename M , typename C >
virtual List< SimulationControl * > * SimulationControlGenericClassNotDC< T, M, C >::get↔
ChildSimulationControls (
    int index = 0 ) [inline], [override], [virtual]
```

Returns nested SimulationControls exposed by this control.

This is the kernel-facing API for traversing child controls belonging to inline objects, references, or list elements represented by a control.

Reimplemented from [SimulationControl](#).

10.631.1.4 getValue()

```
template<typename T , typename M , typename C >
virtual std::string SimulationControlGenericClassNotDC< T, M, C >::getValue ( ) const [inline],
[override], [virtual]
```

Implements [SimulationResponse](#).

10.631.1.5 hasObjectInstance()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClassNotDC< T, M, C >::hasObjectInstance ( ) const [inline],
[override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.631.1.6 isInlineObjectProperty()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClassNotDC< T, M, C >::isInlineObjectProperty ( ) const
[inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.631.1.7 setValue()

```
template<typename T , typename M , typename C >
virtual void SimulationControlGenericClassNotDC< T, M, C >::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

Implements [SimulationControl](#).

10.631.1.8 supportsInlineExpansion()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClassNotDC< T, M, C >::supportsInlineExpansion ( ) const
[inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.631.1.9 supportsObjectCreation()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericClassNotDC< T, M, C >::supportsObjectCreation ( ) const
[inline], [override], [virtual]
```

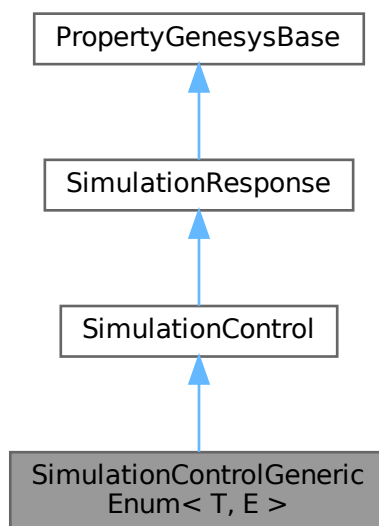
Reimplemented from [SimulationControl](#).

The documentation for this class was generated from the following file:

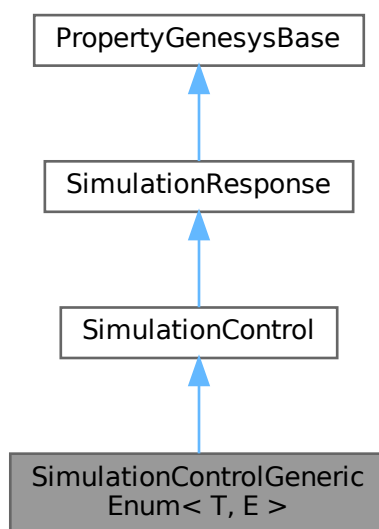
- kernel/simulator/SimulationControlAndResponse.h

10.632 SimulationControlGenericEnum< T, E > Class Template Reference

Inheritance diagram for SimulationControlGenericEnum< T, E >:



Collaboration diagram for SimulationControlGenericEnum< T, E >:



Public Member Functions

- **SimulationControlGenericEnum** (GetterGeneric< T > getter, SetterGeneric< T > setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=true)
- virtual std::string **getValue** () const override
- virtual void **setValue** (std::string value, bool remove=false) override
- virtual **List**< std::string > * **getStrValues** () override

Public Member Functions inherited from **SimulationControl**

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual **List**< **SimulationControl** * > * **getChildSimulationControls** (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual **ModelDataDefinition** * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual **SimulationControlEditorHint** **preferredEditorHint** () const
- void **setPreferredEditorHint** (**SimulationControlEditorHint** hint)
- virtual **ModelDataDefinition** * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual **List**< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual **List**< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual **List**< **SimulationControl** * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from **SimulationResponse**

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- SimulationControlEditorHint **_preferredEditorHint** = SimulationControlEditorHint::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.632.1 Member Function Documentation

10.632.1.1 getStrValues()

```
template<typename T , typename E >
virtual List< std::string > * SimulationControlGenericEnum< T, E >::getStrValues ( ) [inline],
[override], [virtual]
```

Reimplemented from [SimulationResponse](#).

10.632.1.2 `getValue()`

```
template<typename T , typename E >
virtual std::string SimulationControlGenericEnum< T, E >::getValue ( ) const [inline], [override],
[virtual]
```

Implements [SimulationResponse](#).

10.632.1.3 `setValue()`

```
template<typename T , typename E >
virtual void SimulationControlGenericEnum< T, E >::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

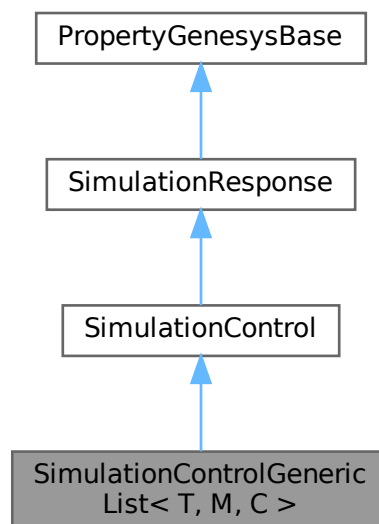
Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

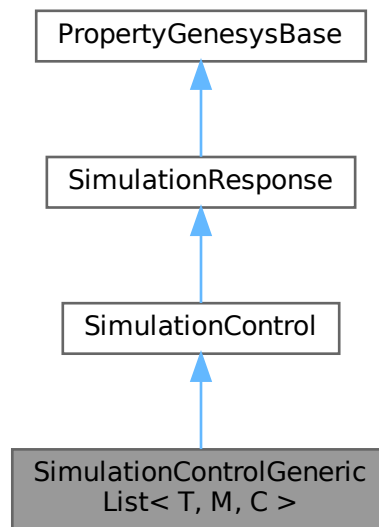
- kernel/simulator/SimulationControlAndResponse.h

10.633 SimulationControlGenericList< T, M, C > Class Template Reference

Inheritance diagram for SimulationControlGenericList< T, M, C >:



Collaboration diagram for `SimulationControlGenericList< T, M, C >`:



Public Member Functions

- **SimulationControlGenericList** (M model, GetterGeneric< [List](#)< T > * > getter, AdderGeneric< T > adder, RemoverGeneric< T > remover, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=true, bool isClass=false, bool isEnum=false)
- virtual std::string [getValue](#) () const override
- virtual void [setValue](#) (std::string value, bool remove=false) override
- virtual [List](#)< std::string > * [getStrValues](#) () override
- virtual bool [supportsListEditor](#) () const override

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [show](#) () const
- bool [isReadOnly](#) () const
- virtual [List](#)< [SimulationControl](#) * > * [getChildSimulationControls](#) (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool [hasObjectInstance](#) () const
- virtual bool [ensureObjectInstance](#) ()
- virtual bool [isModelDataDefinitionReference](#) () const
- virtual [ModelDataDefinition](#) * [getReferencedModelDataDefinition](#) () const
- virtual bool [validateProposedValue](#) (const std::string &value, std::string &errorMessage) const
- virtual [SimulationControlEditorHint](#) [preferredEditorHint](#) () const
- void [setPreferredEditorHint](#) ([SimulationControlEditorHint](#) hint)
- virtual [ModelDataDefinition](#) * [getListElementModelDataDefinition](#) (int index) const
- virtual bool [supportsInlineExpansion](#) () const

- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual [List](#)< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint](#)::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- `std::string _elementName`
- `std::string _className`
- `std::string _propertyName`
- `std::string _propertyType`
- `std::string _whatsThis`
- `bool _isList`
- `bool _isClass`
- `bool _isEnum`

10.633.1 Member Function Documentation

10.633.1.1 `getStrValues()`

```
template<typename T , typename M , typename C >
virtual List< std::string > * SimulationControlGenericList< T, M, C >::getStrValues ( ) [inline],
[override], [virtual]
```

Reimplemented from [SimulationResponse](#).

10.633.1.2 `getValue()`

```
template<typename T , typename M , typename C >
virtual std::string SimulationControlGenericList< T, M, C >::getValue ( ) const [inline],
[override], [virtual]
```

Implements [SimulationResponse](#).

10.633.1.3 `setValue()`

```
template<typename T , typename M , typename C >
virtual void SimulationControlGenericList< T, M, C >::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

Implements [SimulationControl](#).

10.633.1.4 `supportsListEditor()`

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericList< T, M, C >::supportsListEditor ( ) const [inline],
[override], [virtual]
```

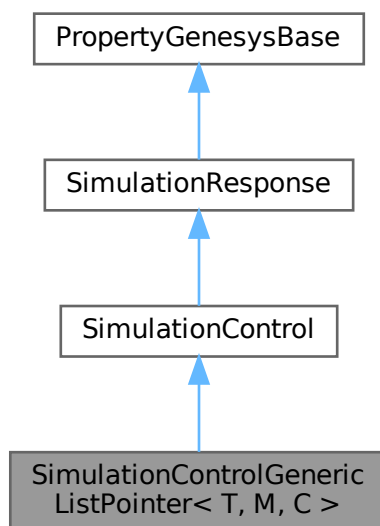
Reimplemented from [SimulationControl](#).

The documentation for this class was generated from the following file:

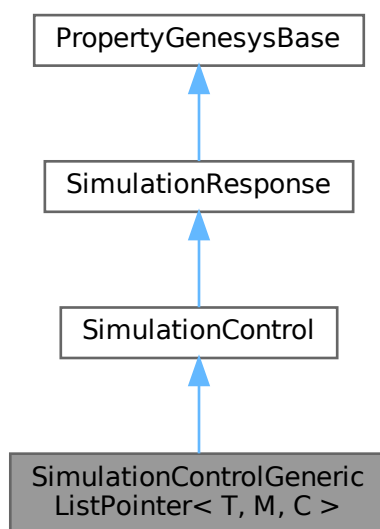
- `kernel/simulator/SimulationControlAndResponse.h`

10.634 SimulationControlGenericListPointer< T, M, C > Class Template Reference

Inheritance diagram for SimulationControlGenericListPointer< T, M, C >:



Collaboration diagram for SimulationControlGenericListPointer< T, M, C >:



Public Types

- using **Creator** = std::function< T(M, const std::string &)>
- using **TypedCreator** = std::function< T(M)>
- using **ElementTypeNamesGetter** = std::function< std::vector< std::string >()>
- using **CurrentElementTypeGetter** = std::function< std::string()>
- using **CurrentElementTypeSetter** = std::function< bool(const std::string &)>
- using **TypedNamedCreator** = std::function< T(M, const std::string &, const std::string &)>
- using **ListElementDataDefinitionGetter** = std::function< [ModelDataDefinition](#) *(int)>

Public Member Functions

- **SimulationControlGenericListPointer** (M model, GetterGeneric< [List](#)< T > * > getter, AdderGeneric< T > adder, RemoverGeneric< T > remover, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=true, bool isClass=true, bool isEnum=false, Creator creator=nullptr, TypedCreator typedCreator=nullptr, ElementTypeNamesGetter elementTypeNamesGetter=nullptr, CurrentElementTypeGetter currentElementTypeGetter=nullptr, CurrentElementTypeSetter currentElementTypeSetter=nullptr, TypedNamedCreator typedNamedCreator=nullptr, ListElementDataDefinitionGetter listElementDataDefinitionGetter=nullptr)
 - virtual std::string [getValue](#) () const override
 - virtual void [setValue](#) (std::string value, bool remove=false) override
 - virtual bool [supportsListEditor](#) () const override
 - virtual bool [supportsNewListElementCreation](#) () const override
 - virtual [List](#)< std::string > * [getCreatableListElementTypes](#) () override
 - virtual std::string [getCurrentListElementType](#) () const override
 - virtual bool [setCurrentListElementType](#) (const std::string &typeName) override
 - virtual bool [createNewListElement](#) (const std::string &value="") override
 - virtual bool [createNewListElementOfType](#) (const std::string &typeName, const std::string &value="") override
 - virtual [List](#)< [SimulationControl](#) * > * [getChildSimulationControls](#) (int index=0) override
- Returns nested SimulationControls exposed by this control.*
- virtual [ModelDataDefinition](#) * [getListElementModelDataDefinition](#) (int index) const override
 - virtual [List](#)< std::string > * [getStrValues](#) () override

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [show](#) () const
- bool [isReadOnly](#) () const
- virtual bool [hasObjectInstance](#) () const
- virtual bool [ensureObjectInstance](#) ()
- virtual bool [isModelDataDefinitionReference](#) () const
- virtual [ModelDataDefinition](#) * [getReferencedModelDataDefinition](#) () const
- virtual bool [validateProposedValue](#) (const std::string &value, std::string &errorMessage) const
- virtual [SimulationControlEditorHint](#) [preferredEditorHint](#) () const
- void [setPreferredEditorHint](#) ([SimulationControlEditorHint](#) hint)
- virtual bool [supportsInlineExpansion](#) () const
- virtual bool [supportsExistingObjectSelection](#) () const
- virtual bool [supportsObjectCreation](#) () const
- virtual bool [isInlineObjectProperty](#) () const
- virtual bool [createObjectInstance](#) (const std::string &value="")
- virtual [List](#)< std::string > * [getCreatableReferenceTypes](#) ()
- virtual std::string [getCurrentReferenceType](#) () const
- virtual bool [setCurrentReferenceType](#) (const std::string &typeName)
- virtual bool [createObjectInstanceOfType](#) (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * [getEditableChildSimulationControls](#) (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- SimulationControlEditorHint **_preferredEditorHint** = SimulationControlEditorHint::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.634.1 Member Function Documentation

10.634.1.1 createNewListElement()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericListPointer< T, M, C >::createNewListElement (
    const std::string & value = "" ) [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.634.1.2 createNewListElementOfType()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericListPointer< T, M, C >::createNewListElementOfType (
    const std::string & typeName,
    const std::string & value = "" ) [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.634.1.3 getChildSimulationControls()

```
template<typename T , typename M , typename C >
virtual List< SimulationControl * > * SimulationControlGenericListPointer< T, M, C >::get↔
ChildSimulationControls (
    int index = 0 ) [inline], [override], [virtual]
```

Returns nested SimulationControls exposed by this control.

This is the kernel-facing API for traversing child controls belonging to inline objects, references, or list elements represented by a control.

Reimplemented from [SimulationControl](#).

10.634.1.4 getCreatableListElementTypes()

```
template<typename T , typename M , typename C >
virtual List< std::string > * SimulationControlGenericListPointer< T, M, C >::getCreatable↔
ListElementTypes ( ) [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.634.1.5 getCurrentListElementType()

```
template<typename T , typename M , typename C >
virtual std::string SimulationControlGenericListPointer< T, M, C >::getCurrentListElementType
( ) const [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.634.1.6 getListElementModelDataDefinition()

```
template<typename T , typename M , typename C >
virtual ModelDataDefinition * SimulationControlGenericListPointer< T, M, C >::getListElement↔
ModelDataDefinition (
    int index ) const [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.634.1.7 getStrValues()

```
template<typename T , typename M , typename C >
virtual List< std::string > * SimulationControlGenericListPointer< T, M, C >::getStrValues (
) [inline], [override], [virtual]
```

Reimplemented from [SimulationResponse](#).

10.634.1.8 getValue()

```
template<typename T , typename M , typename C >
virtual std::string SimulationControlGenericListPointer< T, M, C >::getValue ( ) const [inline],
[override], [virtual]
```

Implements [SimulationResponse](#).

10.634.1.9 setCurrentListElementType()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericListPointer< T, M, C >::setCurrentListElementType (
    const std::string & typeName ) [inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.634.1.10 setValue()

```
template<typename T , typename M , typename C >
virtual void SimulationControlGenericListPointer< T, M, C >::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

Implements [SimulationControl](#).

10.634.1.11 supportsListEditor()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericListPointer< T, M, C >::supportsListEditor ( ) const
[inline], [override], [virtual]
```

Reimplemented from [SimulationControl](#).

10.634.1.12 supportsNewListElementCreation()

```
template<typename T , typename M , typename C >
virtual bool SimulationControlGenericListPointer< T, M, C >::supportsNewListElementCreation (
) const [inline], [override], [virtual]
```

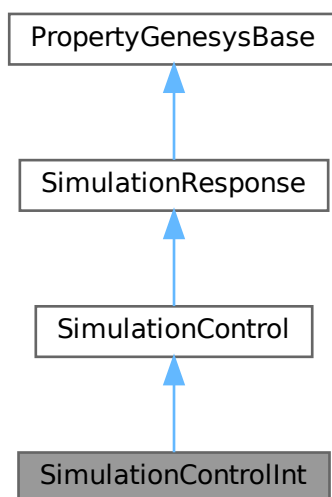
Reimplemented from [SimulationControl](#).

The documentation for this class was generated from the following file:

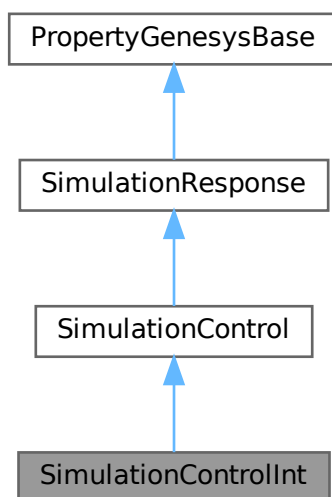
- kernel/simulator/SimulationControlAndResponse.h

10.635 SimulationControlInt Class Reference

Inheritance diagram for SimulationControlInt:



Collaboration diagram for SimulationControlInt:



Public Member Functions

- **SimulationControlInt** (GetterInt getter, SetterInt setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)

- virtual std::string [getValue](#) () const override
- virtual void [setValue](#) (std::string value, bool remove=false) override

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [show](#) () const
- bool [isReadOnly](#) () const
- virtual [List](#)< [SimulationControl](#) * > * [getChildSimulationControls](#) (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool [hasObjectInstance](#) () const
- virtual bool [ensureObjectInstance](#) ()
- virtual bool [isModelDataDefinitionReference](#) () const
- virtual [ModelDataDefinition](#) * [getReferencedModelDataDefinition](#) () const
- virtual bool [validateProposedValue](#) (const std::string &value, std::string &errorMessage) const
- virtual SimulationControlEditorHint [preferredEditorHint](#) () const
- void [setPreferredEditorHint](#) (SimulationControlEditorHint hint)
- virtual [ModelDataDefinition](#) * [getListElementModelDataDefinition](#) (int index) const
- virtual bool [supportsInlineExpansion](#) () const
- virtual bool [supportsListEditor](#) () const
- virtual bool [supportsExistingObjectSelection](#) () const
- virtual bool [supportsObjectCreation](#) () const
- virtual bool [supportsNewListElementCreation](#) () const
- virtual [List](#)< std::string > * [getCreatableListElementTypes](#) ()
- virtual std::string [getCurrentListElementType](#) () const
- virtual bool [setCurrentListElementType](#) (const std::string &typeName)
- virtual bool [isInlineObjectProperty](#) () const
- virtual bool [createObjectInstance](#) (const std::string &value="")
- virtual [List](#)< std::string > * [getCreatableReferenceTypes](#) ()
- virtual std::string [getCurrentReferenceType](#) () const
- virtual bool [setCurrentReferenceType](#) (const std::string &typeName)
- virtual bool [createObjectInstanceOfType](#) (const std::string &typeName, const std::string &value="")
- virtual bool [createNewListElement](#) (const std::string &value="")
- virtual bool [createNewListElementOfType](#) (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * [getEditableChildSimulationControls](#) (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [show](#) () const
- virtual [List](#)< std::string > * [getStrValues](#) ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [whatsThis](#) () const
- std::string [getClassname](#) () const
- std::string [getElementName](#) () const
- std::string [getName](#) () const
- std::string [propertyType](#) () const
- bool [getIsList](#) () const
- bool [getIsClass](#) () const
- bool [getIsEnum](#) () const
- void [setWhatsThis](#) (const std::string text)
- void [setName](#) (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- `void _ensureWritable (const char *operation) const`

Protected Attributes inherited from [SimulationControl](#)

- `bool _readonly = false`
- `SimulationControlEditorHint _preferredEditorHint = SimulationControlEditorHint::Default`

Protected Attributes inherited from [PropertyGenesysBase](#)

- `std::string _elementName`
- `std::string _className`
- `std::string _propertyName`
- `std::string _propertyType`
- `std::string _whatsThis`
- `bool _isList`
- `bool _isClass`
- `bool _isEnum`

10.635.1 Member Function Documentation

10.635.1.1 `getValue()`

```
virtual std::string SimulationControlInt::getValue ( ) const [inline], [override], [virtual]
```

Implements [SimulationResponse](#).

10.635.1.2 `setValue()`

```
virtual void SimulationControlInt::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

- `kernel/simulator/SimulationControlAndResponse.h`

10.636 SimulationController Class Reference

Controller that validates simulation command readiness for [MainWindow](#) wrappers.

```
#include <applications/gui/genesys/controllers/SimulationController.h>
```

Public Member Functions

- [SimulationController](#) (QWidget *ownerWidget, [Simulator](#) *simulator)
Creates a controller bound to GUI owner and simulator.
- bool [hasCurrentModelSimulation](#) () const
Checks whether there is a valid current model with simulation object.
- [ModelSimulation](#) * [currentSimulation](#) () const
Returns current simulation object when available.
- bool [ensureReady](#) (bool checkModel, bool modelChecked, const std::function< bool()> &runModelCheck, const std::function< bool()> &syncModelFromText) const
Validates whether GUI can execute a simulation command now.

10.636.1 Detailed Description

Controller that validates simulation command readiness for [MainWindow](#) wrappers.

This controller is part of the incremental extraction that keeps [MainWindow](#) as a compatibility façade while moving simulation-command orchestration behind smaller collaborators. It centralizes precondition checks shared by start/step/pause/resume/stop flows, including model availability and model-text synchronization guards.

Responsibilities:

- verify that the current model and its [ModelSimulation](#) are available;
- execute callback-based readiness checks delegated by [MainWindow](#);
- return a single command-ready decision used by higher-level command controllers.

Boundaries:

- it does not own simulation state or lifecycle (owned by [Simulator](#));
- it does not execute GUI commands directly (handled by [SimulationCommandController](#));
- it does not update widgets, tabs, or trace outputs.

Dependencies are injected narrowly (owner widget + simulator + callbacks) to preserve existing ownership and enable incremental compatibility without changing slot signatures.

10.636.2 Constructor & Destructor Documentation

10.636.2.1 SimulationController()

```
SimulationController::SimulationController (
    QWidget * ownerWidget,
    Simulator * simulator )
```

Creates a controller bound to GUI owner and simulator.

Parameters

<i>ownerWidget</i>	Widget used to show warning dialogs.
<i>simulator</i>	Simulator facade used by GUI.

10.636.3 Member Function Documentation**10.636.3.1 currentSimulation()**

```
ModelSimulation * SimulationController::currentSimulation ( ) const
```

Returns current simulation object when available.

Returns

Pointer to current [ModelSimulation](#) or nullptr when unavailable.

10.636.3.2 ensureReady()

```
bool SimulationController::ensureReady (
    bool checkModel,
    bool modelChecked,
    const std::function< bool()> & runModelCheck,
    const std::function< bool()> & syncModelFromText ) const
```

Validates whether GUI can execute a simulation command now.

Validation pipeline: 1) Ensure current model/simulation exists; 2) Optionally execute model check callback when model is not checked yet; 3) Execute model synchronization callback (text -> model) and ensure simulation still exists.

Parameters

<i>checkModel</i>	If true, model-check callback may be executed.
<i>modelChecked</i>	Indicates current model check state from MainWindow .
<i>runModelCheck</i>	Callback that executes model check when needed.
<i>syncModelFromText</i>	Callback that synchronizes textual model representation.

Returns

true if command can proceed safely.

Todo Replace callback-based contract by dedicated interfaces to improve testability.

10.636.3.3 hasCurrentModelSimulation()

```
bool SimulationController::hasCurrentModelSimulation ( ) const
```

Checks whether there is a valid current model with simulation object.

Returns

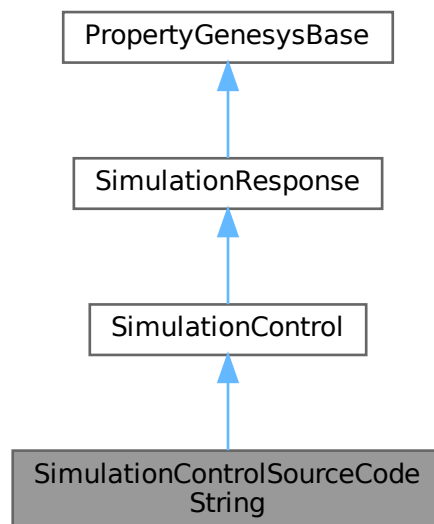
true when simulation commands can access `current()->getSimulation()` safely.

The documentation for this class was generated from the following files:

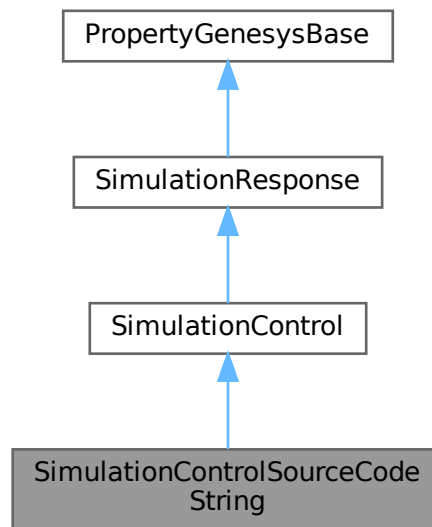
- applications/gui/genesys/controllers/SimulationController.h
- applications/gui/genesys/controllers/SimulationController.cpp

10.637 SimulationControlSourceCodeString Class Reference

Inheritance diagram for SimulationControlSourceCodeString:



Collaboration diagram for SimulationControlSourceCodeString:



Public Member Functions

- **SimulationControlSourceCodeString** (GetterSourceCodeString getter, SetterSourceCodeString setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false, ValidatorString validator=nullptr)
- virtual std::string **getValue** () const override
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const override
- virtual void **setValue** (std::string value, bool remove=false) override

Public Member Functions inherited from **SimulationControl**

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual **List**< **SimulationControl** * > * **getChildSimulationControls** (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual **ModelDataDefinition** * **getReferencedModelDataDefinition** () const
- virtual SimulationControlEditorHint **preferredEditorHint** () const
- void **setPreferredEditorHint** (SimulationControlEditorHint hint)
- virtual **ModelDataDefinition** * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const

- virtual bool **supportsObjectCreation** () const
- virtual bool **supportsNewListElementCreation** () const
- virtual [List](#)< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint](#)::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- `std::string _elementName`
- `std::string _className`
- `std::string _propertyName`
- `std::string _propertyType`
- `std::string _whatsThis`
- `bool _isList`
- `bool _isClass`
- `bool _isEnum`

10.637.1 Member Function Documentation

10.637.1.1 `getValue()`

```
virtual std::string SimulationControlSourceCodeString::getValue ( ) const [inline], [override], [virtual]
```

Implements [SimulationResponse](#).

10.637.1.2 `setValue()`

```
virtual void SimulationControlSourceCodeString::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

Implements [SimulationControl](#).

10.637.1.3 `validateProposedValue()`

```
virtual bool SimulationControlSourceCodeString::validateProposedValue (
    const std::string & value,
    std::string & errorMessage ) const [inline], [override], [virtual]
```

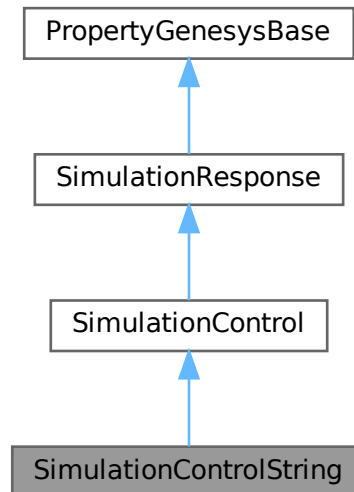
Reimplemented from [SimulationControl](#).

The documentation for this class was generated from the following file:

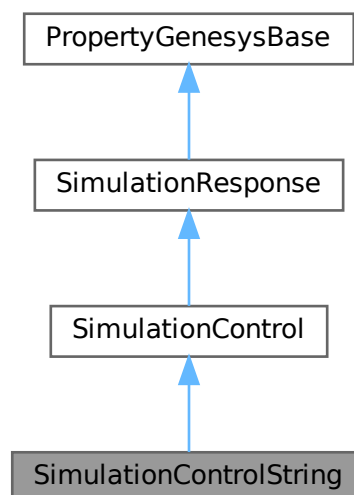
- `kernel/simulator/SimulationControlAndResponse.h`

10.638 SimulationControlString Class Reference

Inheritance diagram for SimulationControlString:



Collaboration diagram for SimulationControlString:



Public Member Functions

- **SimulationControlString** (GetString getter, SetterString setter, std::string className, std::string

- elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false, ValidatorString validator=nullptr)
- virtual std::string [getValue](#) () const override
- virtual bool [validateProposedValue](#) (const std::string &value, std::string &errorMessage) const override
- virtual void [setValue](#) (std::string value, bool remove=false) override

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [show](#) () const
- bool [isReadOnly](#) () const
- virtual [List](#)< [SimulationControl](#) * > * [getChildSimulationControls](#) (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool [hasObjectInstance](#) () const
- virtual bool [ensureObjectInstance](#) ()
- virtual bool [isModelDataDefinitionReference](#) () const
- virtual [ModelDataDefinition](#) * [getReferencedModelDataDefinition](#) () const
- virtual SimulationControlEditorHint [preferredEditorHint](#) () const
- void [setPreferredEditorHint](#) (SimulationControlEditorHint hint)
- virtual [ModelDataDefinition](#) * [getListElementModelDataDefinition](#) (int index) const
- virtual bool [supportsInlineExpansion](#) () const
- virtual bool [supportsListEditor](#) () const
- virtual bool [supportsExistingObjectSelection](#) () const
- virtual bool [supportsObjectCreation](#) () const
- virtual bool [supportsNewListElementCreation](#) () const
- virtual [List](#)< std::string > * [getCreatableListElementTypes](#) ()
- virtual std::string [getCurrentListElementType](#) () const
- virtual bool [setCurrentListElementType](#) (const std::string &typeName)
- virtual bool [isInlineObjectProperty](#) () const
- virtual bool [createObjectInstance](#) (const std::string &value="")
- virtual [List](#)< std::string > * [getCreatableReferenceTypes](#) ()
- virtual std::string [getCurrentReferenceType](#) () const
- virtual bool [setCurrentReferenceType](#) (const std::string &typeName)
- virtual bool [createObjectInstanceOfType](#) (const std::string &typeName, const std::string &value="")
- virtual bool [createNewListElement](#) (const std::string &value="")
- virtual bool [createNewListElementOfType](#) (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * [getEditableChildSimulationControls](#) (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string [show](#) () const
- virtual [List](#)< std::string > * [getStrValues](#) ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- SimulationControlEditorHint **_preferredEditorHint** = SimulationControlEditorHint::Default

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.638.1 Member Function Documentation

10.638.1.1 [getValue\(\)](#)

```
virtual std::string SimulationControlString::getValue ( ) const [inline], [override], [virtual]
```

Implements [SimulationResponse](#).

10.638.1.2 setValue()

```
virtual void SimulationControlString::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

Implements [SimulationControl](#).

10.638.1.3 validateProposedValue()

```
virtual bool SimulationControlString::validateProposedValue (
    const std::string & value,
    std::string & errorMessage ) const [inline], [override], [virtual]
```

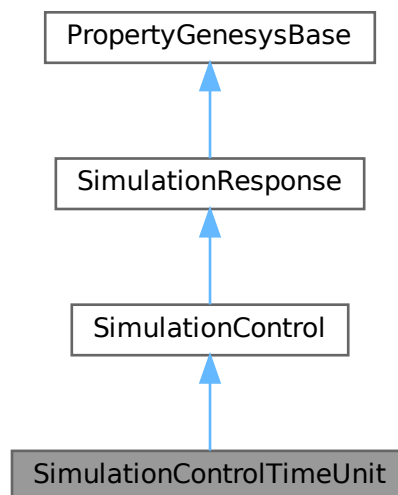
Reimplemented from [SimulationControl](#).

The documentation for this class was generated from the following file:

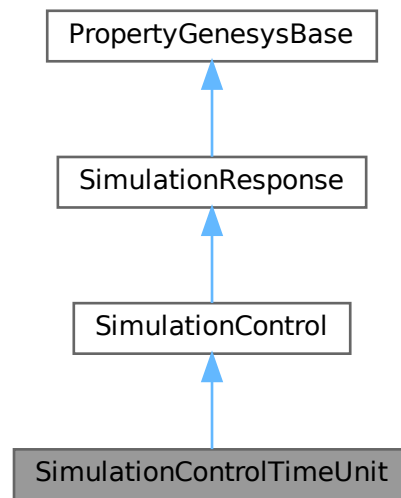
- kernel/simulator/SimulationControlAndResponse.h

10.639 SimulationControlTimeUnit Class Reference

Inheritance diagram for SimulationControlTimeUnit:



Collaboration diagram for SimulationControlTimeUnit:



Public Member Functions

- **SimulationControlTimeUnit** (GetterTimeUnit getter, SetterTimeUnit setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- virtual std::string [getValue](#) () const override
- virtual void [setValue](#) (std::string value, bool remove=false) override

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual [List](#)< [SimulationControl](#) * > * [getChildSimulationControls](#) (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual [ModelDataDefinition](#) * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual [SimulationControlEditorHint](#) **preferredEditorHint** () const
- void **setPreferredEditorHint** ([SimulationControlEditorHint](#) hint)
- virtual [ModelDataDefinition](#) * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const

- virtual bool **supportsNewListElementCreation** () const
- virtual [List](#)< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint::Default](#)

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.639.1 Member Function Documentation

10.639.1.1 `getValue()`

```
virtual std::string SimulationControlTimeUnit::getValue ( ) const [inline], [override], [virtual]
```

Implements [SimulationResponse](#).

10.639.1.2 `setValue()`

```
virtual void SimulationControlTimeUnit::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

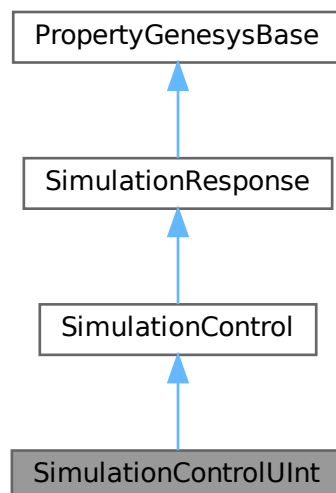
Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

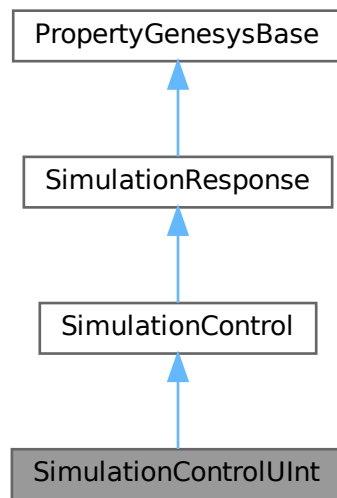
- kernel/simulator/SimulationControlAndResponse.h

10.640 SimulationControlUInt Class Reference

Inheritance diagram for SimulationControlUInt:



Collaboration diagram for SimulationControlUInt:



Public Member Functions

- **SimulationControlUInt** (GetterUInt getter, SetterUInt setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- virtual std::string [getValue](#) () const override
- virtual void [setValue](#) (std::string value, bool remove=false) override

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual [List](#)< [SimulationControl](#) * > * [getChildSimulationControls](#) (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual [ModelDataDefinition](#) * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual [SimulationControlEditorHint](#) **preferredEditorHint** () const
- void **setPreferredEditorHint** ([SimulationControlEditorHint](#) hint)
- virtual [ModelDataDefinition](#) * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const
- virtual bool **supportsNewListElementCreation** () const

- virtual [List](#)< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint::Default](#)

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.640.1 Member Function Documentation

10.640.1.1 `getValue()`

```
virtual std::string SimulationControlUInt::getValue ( ) const [inline], [override], [virtual]
```

Implements [SimulationResponse](#).

10.640.1.2 `setValue()`

```
virtual void SimulationControlUInt::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

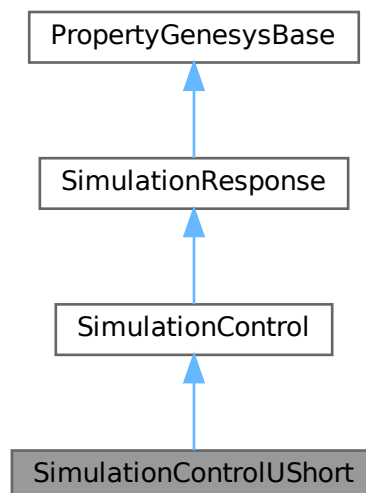
Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

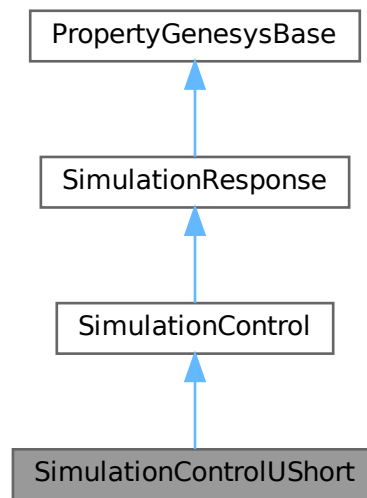
- kernel/simulator/SimulationControlAndResponse.h

10.641 SimulationControlUShort Class Reference

Inheritance diagram for SimulationControlUShort:



Collaboration diagram for SimulationControlUShort:



Public Member Functions

- **SimulationControlUShort** (GetterUShort getter, SetterUShort setter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- virtual std::string [getValue](#) () const override
- virtual void [setValue](#) (std::string value, bool remove=false) override

Public Member Functions inherited from [SimulationControl](#)

- **SimulationControl** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- bool **isReadOnly** () const
- virtual [List](#)< [SimulationControl](#) * > * [getChildSimulationControls](#) (int index=0)
Returns nested SimulationControls exposed by this control.
- virtual bool **hasObjectInstance** () const
- virtual bool **ensureObjectInstance** ()
- virtual bool **isModelDataDefinitionReference** () const
- virtual [ModelDataDefinition](#) * **getReferencedModelDataDefinition** () const
- virtual bool **validateProposedValue** (const std::string &value, std::string &errorMessage) const
- virtual [SimulationControlEditorHint](#) **preferredEditorHint** () const
- void **setPreferredEditorHint** ([SimulationControlEditorHint](#) hint)
- virtual [ModelDataDefinition](#) * **getListElementModelDataDefinition** (int index) const
- virtual bool **supportsInlineExpansion** () const
- virtual bool **supportsListEditor** () const
- virtual bool **supportsExistingObjectSelection** () const
- virtual bool **supportsObjectCreation** () const

- virtual bool **supportsNewListElementCreation** () const
- virtual [List](#)< std::string > * **getCreatableListElementTypes** ()
- virtual std::string **getCurrentListElementType** () const
- virtual bool **setCurrentListElementType** (const std::string &typeName)
- virtual bool **isInlineObjectProperty** () const
- virtual bool **createObjectInstance** (const std::string &value="")
- virtual [List](#)< std::string > * **getCreatableReferenceTypes** ()
- virtual std::string **getCurrentReferenceType** () const
- virtual bool **setCurrentReferenceType** (const std::string &typeName)
- virtual bool **createObjectInstanceOfType** (const std::string &typeName, const std::string &value="")
- virtual bool **createNewListElement** (const std::string &value="")
- virtual bool **createNewListElementOfType** (const std::string &typeName, const std::string &value="")
- virtual [List](#)< [SimulationControl](#) * > * **getEditableChildSimulationControls** (int index=0)

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Member Functions inherited from [SimulationControl](#)

- void **_ensureWritable** (const char *operation) const

Protected Attributes inherited from [SimulationControl](#)

- bool **_readonly** = false
- [SimulationControlEditorHint](#) **_preferredEditorHint** = [SimulationControlEditorHint::Default](#)

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.641.1 Member Function Documentation

10.641.1.1 `getValue()`

```
virtual std::string SimulationControlUShort::getValue ( ) const [inline], [override], [virtual]
```

Implements [SimulationResponse](#).

10.641.1.2 `setValue()`

```
virtual void SimulationControlUShort::setValue (
    std::string value,
    bool remove = false ) [inline], [override], [virtual]
```

Implements [SimulationControl](#).

The documentation for this class was generated from the following file:

- kernel/simulator/SimulationControlAndResponse.h

10.642 SimulationEvent Class Reference

```
#include <kernel/simulator/OnEventManager.h>
```

Public Member Functions

- void **setSimulatedTime** (double simulatedTime)
- double **getSimulatedTime** () const
- void **setStopRequested** (bool stopRequested)
- bool **isStopRequested** () const
- void **setPauseRequested** (bool pauseRequested)
- bool **isPauseRequested** () const
- void **setPaused** (bool Paused)
- bool **isPaused** () const
- void **setRunning** (bool Running)
- bool **isRunning** () const
- void **setCustomObject** (void *customObject)
- void * **getCustomObject** () const
- void **setCurrentReplicationNumber** (unsigned int currentReplicationNumber)
- unsigned int **getCurrentReplicationNumber** () const
- void **setCurrentEvent** ([Event](#) *currentEvent)
- [Event](#) * **getCurrentEvent** () const
- void **setEntityCreated** ([Entity](#) *entityCreated)
- [Entity](#) * **getEntityCreated** () const
- void **setDestinationComponent** ([ModelComponent](#) *destinationComponent)
- [ModelComponent](#) * **getDestinationComponent** () const
- void **setEntityMoveTimeDelay** (double entityMoveTimeDelay)
- double **getEntityMoveTimeDelay** () const

Friends

- class **ModelSimulation**

10.642.1 Detailed Description

Stores an event that happened on a specific replication

The documentation for this class was generated from the following file:

- kernel/simulator/OnEventManager.h

10.643 SimulationEventController Class Reference

Controller for simulation event callbacks, UI updates, and handler wiring.

```
#include <applications/gui/genesys/controllers/SimulationEventController.h>
```

Classes

- struct [Callbacks](#)

Public Member Functions

- **SimulationEventController** ([Simulator](#) *simulator, [ModelGraphicsScene](#) *scene, [ModelGraphicsView](#) *graphicsView, [QLabel](#) *replicationLabel, [QProgressBar](#) *simulationProgressBar, [QTableWidget](#) *simulationEventsTable, [QTableWidget](#) *entitiesTable, [QTableWidget](#) *variablesTable, [QTextEdit](#) *simulationText, [QTextEdit](#) *reportsText, [QTabWidget](#) *centralTabWidget, [QAction](#) *activateGraphicalSimulation, [QAction](#) *animationEnabledAction, bool *modelChecked, int tabCentralReportsIndex, [Callbacks](#) callbacks)
Creates the simulation-event bridge used by [MainWindow](#) compatibility handlers.
- void **onModelCheckSuccessHandler** ([ModelEvent](#) *re) const
Handles model-check success and updates delegated GUI state.
- void **onReplicationStartHandler** ([SimulationEvent](#) *re) const
Handles replication-start event and updates simulation/report panes.
- void **onSimulationStartHandler** ([SimulationEvent](#) *re) const
Handles simulation-start event and resets run-scoped UI state.
- void **onSimulationPausedHandler** ([SimulationEvent](#) *re) const
Handles simulation pause event and refreshes action availability.
- void **onSimulationResumeHandler** ([SimulationEvent](#) *re) const
Handles simulation resume event and resumes graphical simulation synchronization.
- void **onSimulationEndHandler** ([SimulationEvent](#) *re) const
Handles simulation end event and performs delegated cleanup/refresh.
- void **onReplicationEndHandler** ([SimulationEvent](#) *re) const
Handles replication end event and forces progress display completion.
- void **onProcessEventHandler** ([SimulationEvent](#) *re) const
Handles process-event updates for progress tables and scene refresh hooks.
- void **onEntityCreateHandler** ([SimulationEvent](#) *re) const
Handles entity-create events for compatibility callback wiring.
- void **onEntityRemoveHandler** ([SimulationEvent](#) *re) const
Handles entity-remove events for compatibility callback wiring.
- void **onMoveEntityEvent** ([SimulationEvent](#) *re) const
Handles entity-move event updates used by graphical simulation animation.
- void **onAfterProcessEvent** ([SimulationEvent](#) *re) const
Handles after-process event updates used by animation/frame synchronization.
- void **setOnEventHandlers** ([MainWindow](#) *owner) const
Registers kernel on-event callbacks through [MainWindow](#) compatibility façade wrappers.

10.643.1 Detailed Description

Controller for simulation event callbacks, UI updates, and handler wiring.

This controller concentrates event-reaction logic previously embedded in [MainWindow](#) and keeps compatibility by exposing methods invoked from [MainWindow](#) wrappers. It also provides the event-registration entry point used to wire kernel [OnEventManager](#) callbacks to those wrappers without changing the existing signal/slot surface.

Responsibilities:

- react to model-check, replication, simulation, process, and entity events;
- update progress/status widgets and debug tables through injected UI dependencies;
- trigger delegated refresh callbacks for actions, debug panes, and scene synchronization;
- register handlers through [MainWindow](#) compatibility façade ownership.

Boundaries:

- it does not own [MainWindow](#) or widget lifetime;
- it does not execute simulation commands (handled by command controllers);
- it does not perform persistence/export/model-language synchronization.

The documentation for this class was generated from the following files:

- applications/gui/genesys/controllers/SimulationEventController.h
- applications/gui/genesys/controllers/SimulationEventController.cpp

10.644 SimulationExperiment Class Reference

Represents a higher-level simulation experiment definition.

```
#include <kernel/simulator/SimulationExperiment.h>
```

10.644.1 Detailed Description

Represents a higher-level simulation experiment definition.

Historically, GenESyS started with experiment control concentrated in [ModelSimulation](#), mainly through replication count and replication length.

[SimulationExperiment](#) was introduced later as part of an unfinished effort to move richer experiment-design concepts into the simulation kernel, including experiment sets, scenarios and [Process](#) Analyzer-like workflows.

This class is still under development and should currently be treated as a placeholder for future experiment-layer behavior.

The documentation for this class was generated from the following files:

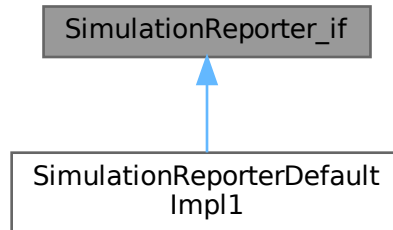
- kernel/simulator/SimulationExperiment.h
- kernel/simulator/SimulationExperiment.cpp

10.645 SimulationReporter_if Class Reference

Interface for reporting simulation results and experiment-facing outputs.

```
#include <kernel/simulator/SimulationReporter_if.h>
```

Inheritance diagram for SimulationReporter_if:



Public Member Functions

- virtual void [showReplicationStatistics](#) ()=0
showReplicationStatistics
- virtual void [showSimulationStatistics](#) ()=0
showSimulationStatistics
- virtual void [showSimulationResponses](#) ()=0
showSimulationResponses
- virtual void [showSimulationControls](#) ()=0
showSimulationControls

10.645.1 Detailed Description

Interface for reporting simulation results and experiment-facing outputs.

Reporter implementations define how replication/simulation summaries, responses and controls are presented to users or external consumers.

10.645.2 Member Function Documentation

10.645.2.1 showReplicationStatistics()

```
virtual void SimulationReporter_if::showReplicationStatistics ( ) [pure virtual]
```

[showReplicationStatistics](#)

Shows detailed statistics for the current/finished replication.

Implemented in [SimulationReporterDefaultImpl1](#).

10.645.2.2 showSimulationControls()

```
virtual void SimulationReporter_if::showSimulationControls ( ) [pure virtual]
```

showSimulationControls

Shows monitored/configurable simulation controls (inputs).

Implemented in [SimulationReporterDefaultImpl1](#).

10.645.2.3 showSimulationResponses()

```
virtual void SimulationReporter_if::showSimulationResponses ( ) [pure virtual]
```

showSimulationResponses

Shows configured simulation responses (outputs) for experimental analysis.

Implemented in [SimulationReporterDefaultImpl1](#).

10.645.2.4 showSimulationStatistics()

```
virtual void SimulationReporter_if::showSimulationStatistics ( ) [pure virtual]
```

showSimulationStatistics

Shows aggregated statistics for the full simulation.

Implemented in [SimulationReporterDefaultImpl1](#).

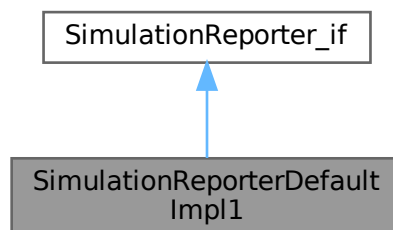
The documentation for this class was generated from the following file:

- kernel/simulator/SimulationReporter_if.h

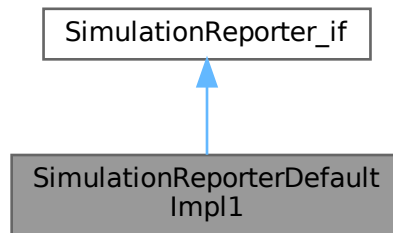
10.646 SimulationReporterDefaultImpl1 Class Reference

```
#include <kernel/simulator/SimulationReporterDefaultImpl1.h>
```

Inheritance diagram for SimulationReporterDefaultImpl1:



Collaboration diagram for SimulationReporterDefaultImpl1:



Public Member Functions

- **SimulationReporterDefaultImpl1** ([ModelSimulation](#) *simulation, [Model](#) *model, [List](#)< [ModelDataDefinition](#) * > *statsCountersSimulation)
- virtual void [showReplicationStatistics](#) () override
showReplicationStatistics
- virtual void [showSimulationStatistics](#) () override
showSimulationStatistics
- virtual void [showSimulationResponses](#) () override
showSimulationResponses
- virtual void [showSimulationControls](#) () override
showSimulationControls

10.646.1 Detailed Description

Class that implements [SimulationReporter_if](#) interface and is responsible for building and showing replication and simulation reports

10.646.2 Member Function Documentation

10.646.2.1 showReplicationStatistics()

```
void SimulationReporterDefaultImpl1::showReplicationStatistics ( ) [override], [virtual]
```

`showReplicationStatistics`

Shows detailed statistics for the current/finished replication.

Implements [SimulationReporter_if](#).

10.646.2.2 showSimulationControls()

```
void SimulationReporterDefaultImpl1::showSimulationControls ( ) [override], [virtual]
```

showSimulationControls

Shows monitored/configurable simulation controls (inputs).

Implements [SimulationReporter_if](#).

10.646.2.3 showSimulationResponses()

```
void SimulationReporterDefaultImpl1::showSimulationResponses ( ) [override], [virtual]
```

showSimulationResponses

Shows configured simulation responses (outputs) for experimental analysis.

Implements [SimulationReporter_if](#).

10.646.2.4 showSimulationStatistics()

```
void SimulationReporterDefaultImpl1::showSimulationStatistics ( ) [override], [virtual]
```

showSimulationStatistics

Shows aggregated statistics for the full simulation.

Implements [SimulationReporter_if](#).

The documentation for this class was generated from the following files:

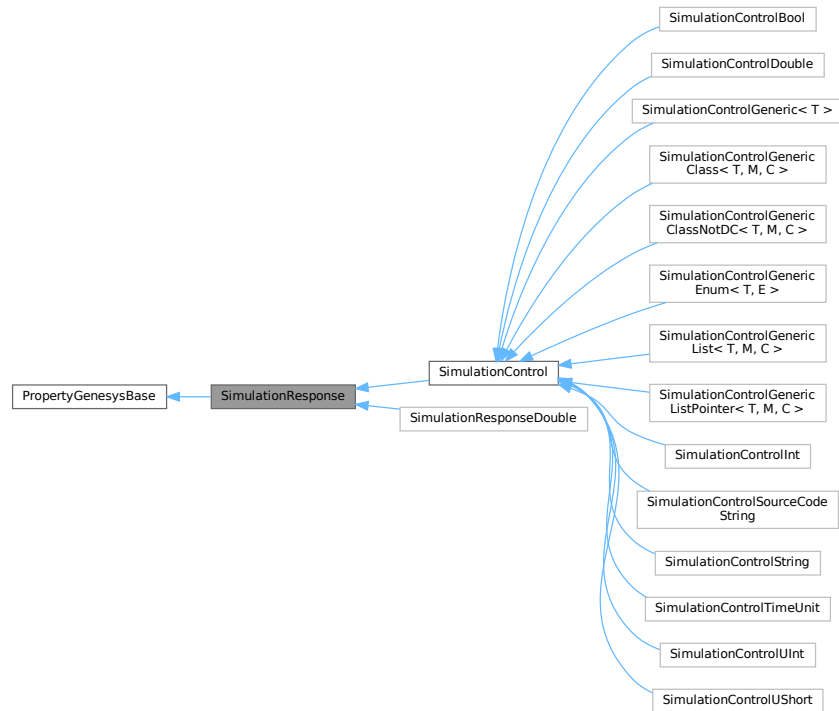
- kernel/simulator/SimulationReporterDefaultImpl1.h
- kernel/simulator/SimulationReporterDefaultImpl1.cpp

10.647 SimulationResponse Class Reference

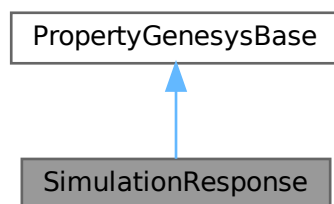
Read-only kernel-side simulation response abstraction.

```
#include <kernel/simulator/SimulationControlAndResponse.h>
```

Inheritance diagram for SimulationResponse:



Collaboration diagram for SimulationResponse:



Public Member Functions

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual std::string **getValue** () const =0
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.647.1 Detailed Description

Read-only kernel-side simulation response abstraction.

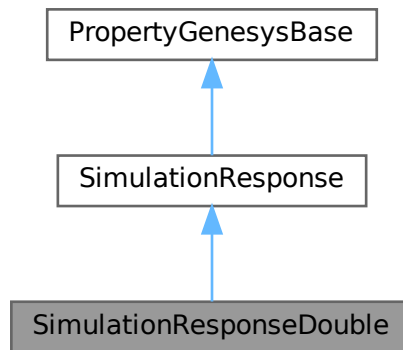
This is the intended kernel-level base for generic getter-based access to observable model attributes, statistics and experiment responses.

The documentation for this class was generated from the following file:

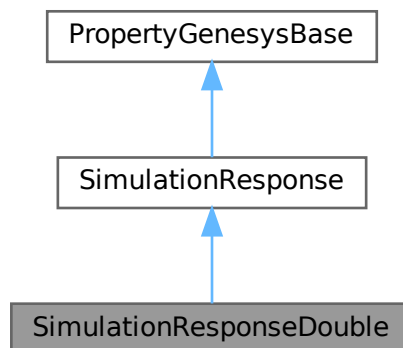
- kernel/simulator/SimulationControlAndResponse.h

10.648 SimulationResponseDouble Class Reference

Inheritance diagram for SimulationResponseDouble:



Collaboration diagram for SimulationResponseDouble:



Public Member Functions

- **SimulationResponseDouble** (GetterDouble getter, std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- virtual std::string [getValue](#) () const override

Public Member Functions inherited from [SimulationResponse](#)

- **SimulationResponse** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **show** () const
- virtual [List](#)< std::string > * **getStrValues** ()

Public Member Functions inherited from [PropertyGenesysBase](#)

- **PropertyGenesysBase** (std::string className, std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
- std::string **whatsThis** () const
- std::string **getClassname** () const
- std::string **getElementName** () const
- std::string **getName** () const
- std::string **propertyType** () const
- bool **getIsList** () const
- bool **getIsClass** () const
- bool **getIsEnum** () const
- void **setWhatsThis** (const std::string text)
- void **setName** (std::string name)

Additional Inherited Members

Protected Attributes inherited from [PropertyGenesysBase](#)

- std::string **_elementName**
- std::string **_className**
- std::string **_propertyName**
- std::string **_propertyType**
- std::string **_whatsThis**
- bool **_isList**
- bool **_isClass**
- bool **_isEnum**

10.648.1 Member Function Documentation

10.648.1.1 getValue()

```
virtual std::string SimulationResponseDouble::getValue ( ) const [inline], [override], [virtual]
```

Implements [SimulationResponse](#).

The documentation for this class was generated from the following file:

- kernel/simulator/SimulationControlAndResponse.h

10.649 SimulationScenario Class Reference

Represents a candidate simulation scenario within the unfinished experiment layer.

```
#include <kernel/simulator/SimulationScenario.h>
```

Public Member Functions

- bool **startSimulation** ([Simulator](#) *sim, std::string &errorMessage)
- std::list< std::pair< std::string, double > * > * [getResponseValues](#) () const
- std::list< std::pair< std::string, double > * > * **getControlValues** () const
- double **getResponseValue** (const std::string &responseName)
- void **setModelFilename** (std::string _modelFilename)
- std::string **getModelFilename** () const
- void **setScenarioName** (std::string _name)
- std::string **getScenarioName** () const
- void **setScenarioDescription** (std::string _scenarioDescription)
- std::string **getScenarioDescription** () const
- std::list< std::string > * **getSelectedControls** () const
- double **getControlValue** (const std::string &controlName)
- std::list< std::string > * **getSelectedResponses** () const
- void **setSelectedControls** (std::list< std::string > *selectedControls)
- void **setControl** (std::string name, double value) const

10.649.1 Detailed Description

Represents a candidate simulation scenario within the unfinished experiment layer.

Represents a scenario where a specific model (defined my ModelFilename) will be simulated. To each scenario will be associated a set of [SimulationControl](#) and [SimulationResponse](#), and their values are set to the scenario by the ProcessAnalyser.

The original GenESyS execution flow relied on [ModelSimulation](#) for the basic experimental controls already supported by the kernel.

[SimulationScenario](#) was introduced later to support richer scenario-based experimentation, potentially including alternative models, modified controls and future [Process](#) Analyzer-like workflows.

This class is still under development and should currently be treated as an evolving kernel-side abstraction for experiment scenarios.

10.649.2 Member Function Documentation

10.649.2.1 [getResponseValues\(\)](#)

```
std::list< std::pair< std::string, double > * > * SimulationScenario::getResponseValues ( )
const
```

The final result of the simulationScenario

The documentation for this class was generated from the following files:

- kernel/simulator/SimulationScenario.h
- kernel/simulator/SimulationScenario.cpp

10.650 SimulatorSessionService::SimulationStatusResult Struct Reference

Captures the current simulation execution state.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Public Attributes

- bool **success** = false
- bool **invalidToken** = false
- bool **hasCurrentModel** = false
- bool **isRunning** = false
- bool **isPaused** = false
- double **simulatedTime** = 0.0
- unsigned int **currentReplicationNumber** = 0
- unsigned int **numberOfReplications** = 0
- double **replicationLength** = 0.0
- double **warmUpPeriod** = 0.0
- bool **pauseOnEvent** = false
- bool **pauseOnReplication** = false
- bool **initializeStatistics** = false
- bool **initializeSystem** = false

10.650.1 Detailed Description

Captures the current simulation execution state.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

10.651 Simulator Class Reference

```
#include <kernel/simulator/Simulator.h>
```

Public Member Functions

- [Simulator](#) ()
- std::string [getVersion](#) () const
getVersion
- unsigned int [getVersionNumber](#) () const
getVersionNumber
- std::string [getName](#) () const
getName
- [LicenceManager](#) * [getLicenceManager](#) () const
getLicence
- [PluginManager](#) * [getPluginManager](#) () const
getPlugins
- [ModelManager](#) * [getModelManager](#) () const
getModelManager
- [TraceManager](#) * [getTraceManager](#) () const
getTraceManager
- [ParserManager](#) * [getParserManager](#) () const
getParserManager
- [ExperimentManager](#) * [getExperimentManager](#) () const
getExperimentManager

Friends

- class **PluginManager**

10.651.1 Detailed Description

The main class of the Genesys KERNEL simulation. It gives access to simulation models and tools. Simulation is the top level class and is supposed to be available to application as a dynamic linked library.

10.651.2 Constructor & Destructor Documentation

10.651.2.1 Simulator()

```
Simulator::Simulator ( )
```

[Simulator](#) constructor basically constructs inner classes (managers) and shows basic information on the console. This is the only method on the entire simulator that directly plots something to the output

10.651.3 Member Function Documentation

10.651.3.1 getExperimentManager()

```
ExperimentManager * Simulator::getExperimentManager ( ) const
```

[getExperimentManager](#)

Returns

10.651.3.2 getLicenceManager()

```
LicenceManager * Simulator::getLicenceManager ( ) const
```

getLicence

Returns

10.651.3.3 getModelManager()

```
ModelManager * Simulator::getModelManager ( ) const
```

getModelManager

Returns

10.651.3.4 getName()

```
std::string Simulator::getName ( ) const
```

getName

Returns

10.651.3.5 getParserManager()

```
ParserManager * Simulator::getParserManager ( ) const
```

getParserManager

Returns

10.651.3.6 getPluginManager()

```
PluginManager * Simulator::getPluginManager ( ) const
```

getPlugins

Returns

10.651.3.7 `getTraceManager()`

```
TraceManager * Simulator::getTraceManager ( ) const
```

`getTraceManager`

Returns

10.651.3.8 `getVersion()`

```
std::string Simulator::getVersion ( ) const
```

`getVersion`

Returns

10.651.3.9 `getVersionNumber()`

```
unsigned int Simulator::getVersionNumber ( ) const
```

`getVersionNumber`

Returns

The documentation for this class was generated from the following files:

- `kernel/simulator/Simulator.h`
- `kernel/simulator/Simulator.cpp`

10.652 SimulatorFacade Class Reference

High-level non-owning facade over [Simulator](#) and its main composed services.

```
#include <kernel/simulator/SimulatorFacade.h>
```

Public Member Functions

- **SimulatorFacade** ([Simulator](#) *simulator)
- std::string **getVersion** () const
- unsigned int **getVersionNumber** () const
- std::string **getName** () const
- const std::string **licenceShow** () const
- const std::string **licenceShowLimits** () const
- const std::string **licenceShowActivationCode** () const
- bool **licenceLookforActivationCode** ()
- bool **licenceInsertActivationCode** ()
- void **licenceRemoveActivationCode** ()
- unsigned int **licenceGetModelComponentsLimit** ()
- unsigned int **licenceGetModelDatasLimit** ()
- unsigned int **licenceGetEntityLimit** ()
- unsigned int **licenceGetHostsLimit** ()
- unsigned int **licenceGetThreadsLimit** ()
- std::string **showPlugins** ()
- [List](#)< [Plugin](#) * > * **completePluginsFieldsAndTemplates** ()
- bool **pluginCheck** (const std::string &dynamicLibraryFilename)
- [SystemDependencyCheckResult](#) **pluginCheckSystemDependencies** (const std::string &dynamicLibraryFilename)
- [List](#)< std::string > * **discoverPluginFileNames** () const
- [List](#)< [PluginLoadIssue](#) > * **getPluginLoadIssues** () const
- void **clearPluginLoadIssues** ()
- void **clearPluginLoadIssue** (const std::string &dynamicLibraryFilename)
- [Plugin](#) * **insertPlugin** (const std::string &dynamicLibraryFilename)
- [Plugin](#) * **insertPlugin** (const std::string &dynamicLibraryFilename, const [PluginInsertionOptions](#) &options)
- bool **removePlugin** (const std::string &dynamicLibraryFilename)
- bool **removePlugin** ([Plugin](#) *plugin)
- [Plugin](#) * **findPlugin** (const std::string &pluginTypeName)
- std::vector< std::string > **getDataDefinitionPluginTypenames** () const
- std::string **pluginSourceIncludePathFor** (const std::string &pluginTypeName)
- [List](#)< [Plugin](#) * > * **autoInsertPlugins** (const std::string &pluginsListFilename, const bool lookForPluginsIfFilenameNotFound=true)
- [List](#)< [Plugin](#) * > * **autoInsertPlugins** (const std::string &pluginsListFilename, const bool lookForPluginsIfFilenameNotFound, const [PluginInsertionOptions](#) &options)
- [List](#)< [Plugin](#) * > * **autoInsertPlugins** ()
- [List](#)< [Plugin](#) * > * **autoInsertPlugins** (const [PluginInsertionOptions](#) &options)
- [Plugin](#) * **firstPlugin** ()
- [Plugin](#) * **nextPlugin** ()
- [Plugin](#) * **lastPlugin** ()
- unsigned int **pluginCount** ()
- [Plugin](#) * **getPluginAtRank** (unsigned int rank)
- [ModelDataDefinition](#) * **newInstance** (const std::string &pluginTypename, [Model](#) *model, std::string name="")
- template<typename T >
T * **newInstance** ([Model](#) *model, std::string name="")
- [Model](#) * **newModel** ()
- void **insertModel** ([Model](#) *model)
- void **removeModel** ([Model](#) *model)
- bool **setCurrentModel** ([Model](#) *model)
- bool **saveModel** (std::string filename)
- [Model](#) * **loadModel** (std::string filename)
- [Model](#) * **createModelFromLanguage** (std::string modelSpecification)
- unsigned int **modelCount** ()

- `std::vector< Model * > models` () const
- `bool hasModel` ([Model](#) *model) const
- `int indexOfModel` ([Model](#) *model) const
- `Model * modelAt` (unsigned int index) const
- `int currentModelIndex` () const
- `Model * firstModel` ()
- `Model * lastModel` ()
- `Model * currentModel` ()
- `Model * nextModel` ()
- `Model * previousModel` ()
- `bool canGoNextModel` () const
- `bool canGoPreviousModel` () const
- `void addTraceHandler` (traceListener traceListener)
- `void addTraceReportHandler` (traceListener traceReportListener)
- `void addTraceSimulationHandler` (traceSimulationListener traceSimulationListener)
- `void addTraceErrorHandler` (traceErrorListener traceErrorListener)
- `template<typename Class >`
`void addTraceHandler` (Class *object, void(Class::*function)([TraceEvent](#)))
- `template<typename Class >`
`void addTraceErrorHandler` (Class *object, void(Class::*function)([TraceErrorEvent](#)))
- `template<typename Class >`
`void addTraceReportHandler` (Class *object, void(Class::*function)([TraceEvent](#)))
- `template<typename Class >`
`void addTraceSimulationHandler` (Class *object, void(Class::*function)([TraceSimulationEvent](#)))
- `void addTraceSimulationExceptionRuleModelData` (void *thisobject)
- `void beginTraceShutdown` ()
- `void trace` (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- `void traceError` (const std::string &text, const std::exception &e)
- `void traceError` (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- `void traceReport` (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- `void traceSimulation` (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed, bool showAnyway=false)
- `void traceSimulation` (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed, bool showAnyway=false)
- `List< std::string > * traceGetErrorMessages` () const
- `void setTraceLevel` (TraceManager::Level traceLevel)
- `TraceManager::Level getTraceLevel` () const
- `void setTraceSimulationRuleAllAllowed` (bool traceSimulationRuleAllAllowed)
- `bool isTraceSimulationRuleAllAllowed` () const
- `ParserManager::GenerateNewParserResult parserGenerateNewParser` ([ParserChangesInformation](#) *changes)
- `bool parserConnectNewParser` ([ParserManager::NewParser](#) newParser)
- `SimulationExperiment * newSimulationExperiment` ()
- `void insertSimulationExperiment` ([SimulationExperiment](#) *experiment)
- `void removeSimulationExperiment` ([SimulationExperiment](#) *experiment)
- `void setCurrentSimulationExperiment` ([SimulationExperiment](#) *experiment)
- `bool saveSimulationExperiment` (std::string filename)
- `bool loadSimulationExperiment` (std::string filename)
- `unsigned int simulationExperimentCount` ()
- `SimulationExperiment * firstSimulationExperiment` ()
- `SimulationExperiment * currentSimulationExperiment` ()
- `SimulationExperiment * nextSimulationExperiment` ()
- `List< SimulationExperiment * > * getSimulationExperiments` () const
- `bool modelSave` (std::string filename)
- `bool modelLoad` (std::string filename)

- bool **modelCheck** ()
- void **modelClear** ()
- std::string **modelShowLanguage** ()
- void **modelShow** ()
- bool **modelInsert** ([ModelDataDefinition](#) *elemOrComp)
- void **modelRemove** ([ModelDataDefinition](#) *elemOrComp)
- std::list< [ModelDataDefinition](#) * > **modelCollectDataDefinitionsRemovedWith** (const std::list< [ModelDataDefinition](#) * > &roots) const
- [Entity](#) * **modelCreateEntity** (std::string name, bool insertIntoModel=true)
- void **modelRemoveEntity** ([Entity](#) *entity)
- void **modelSendEntityToComponent** ([Entity](#) *entity, [Connection](#) *connection, double timeDelay=0.0)
- void **modelSendEntityToComponent** ([Entity](#) *entity, [ModelComponent](#) *component, double timeDelay=0.0, unsigned int componentinputPortNumber=0)
- double **modelParseExpression** (const std::string expression)
- double **modelParseExpression** (const std::string expression, bool &success, std::string &errorMessage)
- bool **modelCheckExpression** (const std::string expression, const std::string expressionName, std::string &errorMessage)
- void **modelCheckReferencesToDataDefinitions** (std::string expression, std::map< std::string, std::list< std::string > * > *referencedDataDefinitions)
- Util::identification **modelGetId** () const
- bool **modelHasChanged** () const
- void **modelSetHasChanged** (bool hasChanged)
- [List](#)< [Event](#) * > * **modelGetFutureEvents** () const
- [List](#)< [SimulationControl](#) * > * **modelGetControls** () const
- [List](#)< [SimulationResponse](#) * > * **modelGetResponses** () const
- [Persistence_if](#) * **modelGetPersistence** () const
- unsigned int **modelGetLevel** () const
- std::string **infoShow** ()
- void **infoSetName** (std::string name)
- std::string **infoGetName** () const
- void **infoSetAnalystName** (std::string analystName)
- std::string **infoGetAnalystName** () const
- void **infoSetDescription** (std::string description)
- std::string **infoGetDescription** () const
- void **infoSetProjectTitle** (std::string projectTitle)
- std::string **infoGetProjectTitle** () const
- void **infoSetVersion** (std::string version)
- std::string **infoGetVersion** () const
- void **infoLoadInstance** ([PersistenceRecord](#) *fields)
- void **infoSaveInstance** ([PersistenceRecord](#) *fields)
- bool **infoHasChanged** () const
- void **infoSetHasChanged** (bool hasChanged)
- std::string **simShow** ()
- void **simStart** ()
- void **simPause** ()
- void **simStep** ()
- void **simStop** ()
- void **simSetNumberOfReplications** (unsigned int numberOfReplications)
- unsigned int **simGetNumberOfReplications** () const
- void **simSetReplicationLength** (double replicationLength, [Util::TimeUnit](#) replicationLengthTimeUnit=[Util::TimeUnit::unknown](#))
- double **simGetReplicationLength** () const
- void **simSetReplicationLengthTimeUnit** ([Util::TimeUnit](#) replicationLengthTimeUnit)
- [Util::TimeUnit](#) **simGetReplicationLengthTimeUnit** () const
- void **simSetReplicationReportBaseTimeUnit** ([Util::TimeUnit](#) replicationReportBaseTimeUnit)

- [Util::TimeUnit](#) **simGetReplicationBaseTimeUnit** () const
- void **simSetWarmUpPeriod** (double warmUpPeriod, [Util::TimeUnit](#) warmUpPeriodTimeUnit=[Util::TimeUnit](#)←[::unknown](#))
- double **simGetWarmUpPeriod** () const
- void **simSetWarmUpPeriodTimeUnit** ([Util::TimeUnit](#) warmUpPeriodTimeUnit)
- [Util::TimeUnit](#) **simGetWarmUpPeriodTimeUnit** () const
- void **simSetTerminatingCondition** (std::string terminatingCondition)
- std::string **simGetTerminatingCondition** () const
- void **simSetPauseOnEvent** (bool pauseOnEvent)
- bool **simIsPauseOnEvent** () const
- void **simSetStepByStep** (bool stepByStep)
- bool **simIsStepByStep** () const
- void **simSetInitializeStatistics** (bool initializeStatistics)
- bool **simIsInitializeStatistics** () const
- void **simSetInitializeSystem** (bool initializeSystem)
- bool **simIsInitializeSystem** () const
- void **simSetPauseOnReplication** (bool pauseBetweenReplications)
- bool **simIsPauseOnReplication** () const
- void **simSetReporter** ([SimulationReporter_if](#) *simulationReporter)
- [SimulationReporter_if](#) * **simGetReporter** () const
- double **simGetSimulatedTime** () const
- bool **simIsRunning** () const
- bool **simIsPaused** () const
- unsigned int **simGetCurrentReplicationNumber** () const
- void **simSetShowReportsAfterReplication** (bool showReportsAfterReplication)
- bool **simIsShowReportsAfterReplication** () const
- void **simSetShowReportsAfterSimulation** (bool showReportsAfterSimulation)
- bool **simIsShowReportsAfterSimulation** () const
- [List](#)< double > * **simGetBreakpointsOnTime** () const
- [List](#)< [Entity](#) * > * **simGetBreakpointsOnEntity** () const
- [List](#)< [ModelComponent](#) * > * **simGetBreakpointsOnComponent** () const
- const [List](#)< [ModelDataDefinition](#) * > * **simGetSimulationStatisticsAggregates** () const
- void **simLoadInstance** ([PersistenceRecord](#) *fields)
- void **simSaveInstance** ([PersistenceRecord](#) *fields, bool saveDefaults)
- [Event](#) * **simGetCurrentEvent** () const
- void **simSetShowSimulationResponsesInReport** (bool showSimulationResponsesInReport)
- bool **simIsShowSimulationResponsesInReport** () const
- void **simSetShowSimulationControlsInReport** (bool showSimulationControlsInReport)
- bool **simIsShowSimulationControlsInReport** () const
- bool **dataInsert** ([ModelDataDefinition](#) *anElement)
- void **dataRemove** ([ModelDataDefinition](#) *anElement)
- bool **dataInsert** (std::string datadefinitionTypename, [ModelDataDefinition](#) *anElement)
- void **dataRemove** (std::string datadefinitionTypename, [ModelDataDefinition](#) *anElement)
- bool **dataCheck** (std::string datadefinitionTypename, [ModelDataDefinition](#) *anElement, std::string expressionName, std::string &errorMessage)
- bool **dataCheck** (std::string datadefinitionTypename, std::string elementName, std::string expressionName, bool mandatory, std::string &errorMessage)
- void **dataClear** ()
- [ModelDataDefinition](#) * **dataGetDataDefinition** (std::string datadefinitionTypename, [Util::identification](#) id)
- [ModelDataDefinition](#) * **dataGetDataDefinition** (std::string datadefinitionTypename, std::string name)
- unsigned int **dataGetNumberOfDataDefinitions** (std::string datadefinitionTypename)
- unsigned int **dataGetNumberOfDataDefinitions** ()
- int **dataGetRankOf** (std::string datadefinitionTypename, std::string name)
- std::list< std::string > **dataGetDataDefinitionClassnames** () const
- [List](#)< [ModelDataDefinition](#) * > * **dataGetDataDefinitionList** (std::string datadefinitionTypename) const

- void **dataShow** ()
- bool **dataHasChanged** () const
- void **dataSetHasChanged** (bool hasChanged)
- bool **componentInsert** ([ModelComponent](#) *comp)
- void **componentRemove** ([ModelComponent](#) *comp)
- [ModelComponent](#) * **componentFind** (std::string name)
- [ModelComponent](#) * **componentFind** (Util::identification id)
- void **componentClear** ()
- unsigned int **componentGetNumberOfComponents** ()
- std::list< [ModelComponent](#) * >::iterator **componentBegin** ()
- std::list< [ModelComponent](#) * >::iterator **componentEnd** ()
- [ModelComponent](#) * **componentFront** ()
- [ModelComponent](#) * **componentNext** ()
- bool **componentHasChanged** () const
- void **componentSetHasChanged** (bool hasChanged)
- std::list< [SourceModelComponent](#) * > * **componentGetSourceComponents** ()
- std::list< [ModelComponent](#) * > * **componentGetTransferInComponents** ()
- std::list< [ModelComponent](#) * > * **componentGetAllComponents** () const
- void **eventAddOnModelCheckSucessHandler** (modelEventHandler EventHandler)
- void **eventAddOnModelLoadHandler** (modelEventHandler EventHandler)
- void **eventAddOnModelSaveHandler** (modelEventHandler EventHandler)
- void **eventAddOnReplicationStartHandler** (simulationEventHandler EventHandler)
- void **eventAddOnReplicationStepHandler** (simulationEventHandler EventHandler)
- void **eventAddOnReplicationEndHandler** (simulationEventHandler EventHandler)
- void **eventAddOnProcessEventHandler** (simulationEventHandler EventHandler)
- void **eventAddOnAfterProcessEventHandler** (simulationEventHandler EventHandler)
- void **eventAddOnEntityCreateHandler** (simulationEventHandler EventHandler)
- void **eventAddOnEntityMoveHandler** (simulationEventHandler EventHandler)
- void **eventAddOnEntityRemoveHandler** (simulationEventHandler EventHandler)
- void **eventAddOnSimulationStartHandler** (simulationEventHandler EventHandler)
- void **eventAddOnSimulationPausedHandler** (simulationEventHandler EventHandler)
- void **eventAddOnSimulationResumeHandler** (simulationEventHandler EventHandler)
- void **eventAddOnSimulationEndHandler** (simulationEventHandler EventHandler)
- void **eventAddOnBreakpointHandler** (simulationEventHandler EventHandler)
- template<typename Class >
void **eventAddOnModelCheckSuccessHandler** (Class *object, void(Class::*function)([ModelEvent](#) *))
- template<typename Class >
void **eventAddOnModelLoadHandler** (Class *object, void(Class::*function)([ModelEvent](#) *))
- template<typename Class >
void **eventAddOnModelSaveHandler** (Class *object, void(Class::*function)([ModelEvent](#) *))
- template<typename Class >
void **eventAddOnReplicationStartHandler** (Class *object, void(Class::*function)([SimulationEvent](#) *))
- template<typename Class >
void **eventAddOnReplicationStepHandler** (Class *object, void(Class::*function)([SimulationEvent](#) *))
- template<typename Class >
void **eventAddOnReplicationEndHandler** (Class *object, void(Class::*function)([SimulationEvent](#) *))
- template<typename Class >
void **eventAddOnProcessEventHandler** (Class *object, void(Class::*function)([SimulationEvent](#) *))
- template<typename Class >
void **eventAddOnAfterProcessEventHandler** (Class *object, void(Class::*function)([SimulationEvent](#) *))
- template<typename Class >
void **eventAddOnEntityCreateHandler** (Class *object, void(Class::*function)([SimulationEvent](#) *))
- template<typename Class >
void **eventAddOnEntityMoveHandler** (Class *object, void(Class::*function)([SimulationEvent](#) *))

- `template<typename Class >`
`void eventAddOnEntityRemoveHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`
`void eventAddOnSimulationStartHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`
`void eventAddOnSimulationPausedHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`
`void eventAddOnSimulationResumeHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`
`void eventAddOnSimulationEndHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `template<typename Class >`
`void eventAddOnBreakpointHandler (Class *object, void(Class::*function)(SimulationEvent *))`
- `void eventNotifyModelCheckSuccessHandlers (ModelEvent *se)`
- `void eventNotifyModelLoadHandlers (ModelEvent *se)`
- `void eventNotifyModelSaveHandlers (ModelEvent *se)`
- `void eventNotifyReplicationStartHandlers (SimulationEvent *se)`
- `void eventNotifyReplicationStepHandlers (SimulationEvent *se)`
- `void eventNotifyReplicationEndHandlers (SimulationEvent *se)`
- `void eventNotifyProcessEventHandlers (SimulationEvent *se)`
- `void eventNotifyAfterProcessEventHandlers (SimulationEvent *se)`
- `void eventNotifyEntityCreateHandlers (SimulationEvent *se)`
- `void eventNotifyEntityMoveHandlers (SimulationEvent *se)`
- `void eventNotifyEntityRemoveHandlers (SimulationEvent *se)`
- `void eventNotifySimulationStartHandlers (SimulationEvent *se)`
- `void eventNotifySimulationPausedHandlers (SimulationEvent *se)`
- `void eventNotifySimulationResumeHandlers (SimulationEvent *se)`
- `void eventNotifySimulationEndHandlers (SimulationEvent *se)`
- `void eventNotifyBreakpointHandlers (SimulationEvent *se)`

Static Public Member Functions

- `static std::string traceConvertEnumToStr (TraceManager::Level level)`

10.652.1 Detailed Description

High-level non-owning facade over [Simulator](#) and its main composed services.

Construct with a non-owning pointer to an existing [Simulator](#) instance. All public methods delegate through the private helpers; none of those helpers are exposed publicly.

The documentation for this class was generated from the following file:

- `kernel/simulator/SimulatorFacade.h`

10.653 SimulatorSessionService::SimulatorInfoResult Struct Reference

Describes simulator identity metadata for a token-scoped session.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Public Attributes

- `std::string` **name**
- `std::string` **versionName**
- `unsigned int` **versionNumber**

10.653.1 Detailed Description

Describes simulator identity metadata for a token-scoped session.

The documentation for this struct was generated from the following file:

- `applications/worker/service/SimulatorSessionService.h`

10.654 SimulatorSessionService Class Reference

Encapsulates session-scoped simulator operations exposed by the Web API.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Classes

- struct [CreateSessionResult](#)
Contains session creation identifiers returned to API clients.
- struct [ModelImportResult](#)
Contains model import output and error state.
- struct [ModelInfoResult](#)
Describes the currently active model information for a session.
- struct [ModelPersistenceResult](#)
Contains save/load model operation output and error state.
- struct [SimulationActionResult](#)
Contains the result of a simulation action such as run or step.
- struct [SimulationConfigInput](#)
Defines user-provided simulation parameters to apply to the current model.
- struct [SimulationConfigResult](#)
Contains the result of applying simulation configuration.
- struct [SimulationStatusResult](#)
Captures the current simulation execution state.
- struct [SimulatorInfoResult](#)
Describes simulator identity metadata for a token-scoped session.
- struct [WorkerCapabilitiesResult](#)
Describes currently supported worker features.
- struct [WorkerInfoResult](#)
Describes worker identity metadata for distributed discovery.
- struct [WorkerJobCreationResult](#)
Contains worker job creation output and error state.
- struct [WorkerJobInfoResult](#)
Describes worker job metadata exposed by worker stage 3 endpoints.

- struct [WorkerJobQueryResult](#)
Contains worker job lookup output and error state.
- struct [WorkerJobResultInfo](#)
Describes terminal worker job result data exposed by worker stage 5 endpoint.
- struct [WorkerJobResultQueryResult](#)
Contains worker job terminal result lookup output and error state.
- struct [WorkerJobRunResult](#)
Contains worker job run output and error state.

Public Types

- enum class [PersistenceError](#) {
None , **InvalidToken** , **InvalidFilename** , **MissingCurrentModel** ,
FileNotFound , **OperationFailed** }
Enumerates persistence-specific errors for save/load model operations.
- enum class [ModelImportError](#) {
None , **InvalidToken** , **EmptySpecification** , **InvalidSpecification** ,
OperationFailed }
Enumerates model import errors for worker language-ingress operations.
- enum class [WorkerJobError](#) {
None , **InvalidToken** , **MissingCurrentModel** , **JobNotFound** ,
AccessDenied , **OperationFailed** , **ResultNotReady** }
Enumerates worker job operation errors.

Public Member Functions

- [SimulatorSessionService](#) ([SessionManager](#) &sessionManager, [WorkerJobManager](#) &workerJobManager)
Builds a service bound to the provided session manager.
- [CreateSessionResult](#) [createSession](#) ()
Creates a new authenticated session.
- bool [tryGetSimulatorInfo](#) (const std::string &accessToken, [SimulatorInfoResult](#) &outInfo)
Fetches simulator metadata using a session token.
- [WorkerInfoResult](#) [getWorkerInfo](#) () const
Fetches public worker identity metadata.
- [WorkerCapabilitiesResult](#) [getWorkerCapabilities](#) () const
Fetches worker capability flags for the current implementation state.
- bool [tryCreateModel](#) (const std::string &accessToken, [ModelInfoResult](#) &outInfo)
Creates a new model in the token-scoped simulator session.
- bool [tryGetCurrentModelInfo](#) (const std::string &accessToken, [ModelInfoResult](#) &outInfo)
Returns metadata for the current model in a session.
- [SimulationStatusResult](#) [getSimulationStatus](#) (const std::string &accessToken)
Returns simulation status for the session's current model.
- [SimulationConfigResult](#) [configureSimulation](#) (const std::string &accessToken, const [SimulationConfigInput](#) &input)
Applies simulation configuration to the current model.
- [SimulationActionResult](#) [runSimulation](#) (const std::string &accessToken)
Executes a synchronous simulation run.
- [SimulationActionResult](#) [stepSimulation](#) (const std::string &accessToken)
Executes one synchronous simulation step.
- [ModelPersistenceResult](#) [saveCurrentModel](#) (const std::string &accessToken, const std::string &filename)

Saves the current model into the session workspace.

- [ModelPersistenceResult loadModel](#) (const std::string &accessToken, const std::string &filename)

Loads a model from the session workspace by filename.

- [ModelImportResult importModelFromLanguage](#) (const std::string &accessToken, const std::string &model← Specification)

Imports a model from plain text model language into the token-scoped session.

- [WorkerJobCreationResult createWorkerJob](#) (const std::string &accessToken)

Creates a queued worker job for the token-scoped session model snapshot.

- [WorkerJobQueryResult getWorkerJob](#) (const std::string &accessToken, const std::string &jobId)

Returns worker job metadata for a token-scoped session.

- [WorkerJobRunResult runWorkerJob](#) (const std::string &accessToken, const std::string &jobId)

Executes a worker job synchronously in the token-scoped simulator session.

- [WorkerJobResultQueryResult getWorkerJobResult](#) (const std::string &accessToken, const std::string &jobId)

Returns persisted terminal result data for a token-scoped worker job.

10.654.1 Detailed Description

Encapsulates session-scoped simulator operations exposed by the Web API.

10.654.2 Constructor & Destructor Documentation

10.654.2.1 SimulatorSessionService()

```
SimulatorSessionService::SimulatorSessionService (
    SessionManager & sessionManager,
    WorkerJobManager & workerJobManager ) [explicit]
```

Builds a service bound to the provided session manager.

Wires session lookup with per-session worker job storage.

Parameters

<i>sessionManager</i>	Session manager used to resolve active sessions.
<i>workerJobManager</i>	In-memory worker job storage manager.

10.654.3 Member Function Documentation

10.654.3.1 configureSimulation()

```
SimulatorSessionService::SimulationConfigResult SimulatorSessionService::configureSimulation (
    const std::string & accessToken,
    const SimulationConfigInput & input )
```

Applies simulation configuration to the current model.

Applies user-provided simulation parameters to the current model.

Parameters

<i>accessToken</i>	Bearer token associated with a session.
<i>input</i>	Configuration values to apply.

Returns

Structured configuration result and resulting status.

10.654.3.2 createSession()

```
SimulatorSessionService::CreateSessionResult SimulatorSessionService::createSession ( )
```

Creates a new authenticated session.

Creates one authenticated session and returns its public identifiers.

Returns

Session identifiers and token for API usage.

10.654.3.3 createWorkerJob()

```
SimulatorSessionService::WorkerJobCreationResult SimulatorSessionService::createWorkerJob (
    const std::string & accessToken )
```

Creates a queued worker job for the token-scoped session model snapshot.

Creates a queued worker job by snapshotting the current model.

Parameters

<i>accessToken</i>	Bearer token associated with a session.
--------------------	---

Returns

Worker job creation output and error state.

10.654.3.4 getSimulationStatus()

```
SimulatorSessionService::SimulationStatusResult SimulatorSessionService::getSimulationStatus (
    const std::string & accessToken )
```

Returns simulation status for the session's current model.

Returns the current simulation status for a session token.

Parameters

<i>accessToken</i>	Bearer token associated with a session.
--------------------	---

Returns

Structured simulation status query result.

10.654.3.5 getWorkerCapabilities()

```
SimulatorSessionService::WorkerCapabilitiesResult SimulatorSessionService::getWorkerCapabilities  
( ) const
```

Fetches worker capability flags for the current implementation state.

Advertises the runtime capabilities supported by this worker flavor.

Returns

Worker capability flags.

10.654.3.6 getWorkerInfo()

```
SimulatorSessionService::WorkerInfoResult SimulatorSessionService::getWorkerInfo ( ) const
```

Fetches public worker identity metadata.

Returns the fixed worker identity metadata used for discovery.

Returns

Worker identity information for discovery endpoints.

10.654.3.7 getWorkerJob()

```
SimulatorSessionService::WorkerJobQueryResult SimulatorSessionService::getWorkerJob (  
    const std::string & accessToken,  
    const std::string & jobId )
```

Returns worker job metadata for a token-scoped session.

Retrieves worker-job metadata when the job belongs to the session.

Parameters

<i>accessToken</i>	Bearer token associated with a session.
<i>jobId</i>	Worker job identifier to query.

Returns

Worker job lookup output and error state.

10.654.3.8 getWorkerJobResult()

```
SimulatorSessionService::WorkerJobResultQueryResult SimulatorSessionService::getWorkerJobResult (
    const std::string & accessToken,
    const std::string & jobId )
```

Returns persisted terminal result data for a token-scoped worker job.

Retrieves the terminal result for a completed worker job.

Parameters

<i>accessToken</i>	Bearer token associated with a session.
<i>jobId</i>	Worker job identifier to query.

Returns

Worker job terminal result lookup output and error state.

10.654.3.9 importModelFromLanguage()

```
SimulatorSessionService::ModelImportResult SimulatorSessionService::importModelFromLanguage (
    const std::string & accessToken,
    const std::string & modelSpecification )
```

Imports a model from plain text model language into the token-scoped session.

Imports a model from textual Genesys language.

Parameters

<i>accessToken</i>	Bearer token associated with a session.
<i>modelSpecification</i>	Plain text model language content.

Returns

Import output and error state.

10.654.3.10 loadModel()

```
SimulatorSessionService::ModelPersistenceResult SimulatorSessionService::loadModel (
    const std::string & accessToken,
    const std::string & filename )
```


Loads a model from the session workspace by filename.

Loads a saved model file from the session workspace.

Parameters

<i>accessToken</i>	Bearer token associated with a session.
<i>filename</i>	Safe basename to load.

Returns

Persistence output and error state.

10.654.3.11 runSimulation()

```
SimulatorSessionService::SimulationActionResult SimulatorSessionService::runSimulation (
    const std::string & accessToken )
```

Executes a synchronous simulation run.

Starts the current simulation for the token-scoped session.

Parameters

<i>accessToken</i>	Bearer token associated with a session.
--------------------	---

Returns

Structured action result and updated simulation status.

10.654.3.12 runWorkerJob()

```
SimulatorSessionService::WorkerJobRunResult SimulatorSessionService::runWorkerJob (
    const std::string & accessToken,
    const std::string & jobId )
```

Executes a worker job synchronously in the token-scoped simulator session.

Executes a queued worker job synchronously from its stored snapshot.

Parameters

<i>accessToken</i>	Bearer token associated with a session.
<i>jobId</i>	Worker job identifier to execute.

Returns

Worker job run output and error state.

10.654.3.13 saveCurrentModel()

```
SimulatorSessionService::ModelPersistenceResult SimulatorSessionService::saveCurrentModel (
```

```
const std::string & accessToken,  
const std::string & filename )
```

Saves the current model into the session workspace.

Persists the current model into the session workspace.

Parameters

<i>accessToken</i>	Bearer token associated with a session.
<i>filename</i>	Safe basename to use for persistence.

Returns

Persistence output and error state.

10.654.3.14 stepSimulation()

```
SimulatorSessionService::SimulationActionResult SimulatorSessionService::stepSimulation (  
    const std::string & accessToken )
```

Executes one synchronous simulation step.

Advances the current simulation by one step.

Parameters

<i>accessToken</i>	Bearer token associated with a session.
--------------------	---

Returns

Structured action result and updated simulation status.

10.654.3.15 tryCreateModel()

```
bool SimulatorSessionService::tryCreateModel (  
    const std::string & accessToken,  
    ModelInfoResult & outInfo )
```

Creates a new model in the token-scoped simulator session.

Creates a new kernel model in the current session.

Parameters

<i>accessToken</i>	Bearer token associated with a session.
<i>outInfo</i>	Receives model information on success.

Returns

True when the model was created and output was populated.

10.654.3.16 tryGetCurrentModelInfo()

```
bool SimulatorSessionService::tryGetCurrentModelInfo (
    const std::string & accessToken,
    ModelInfoResult & outInfo )
```

Returns metadata for the current model in a session.

Returns metadata for the current model if one exists.

Parameters

<i>accessToken</i>	Bearer token associated with a session.
<i>outInfo</i>	Receives current model data.

Returns

True when token is valid and query completed.

10.654.3.17 tryGetSimulatorInfo()

```
bool SimulatorSessionService::tryGetSimulatorInfo (
    const std::string & accessToken,
    SimulatorInfoResult & outInfo )
```

Fetches simulator metadata using a session token.

Resolves the simulator metadata visible to a session token.

Parameters

<i>accessToken</i>	Bearer token associated with a session.
<i>outInfo</i>	Receives simulator identity information on success.

Returns

True when token is valid and metadata was populated.

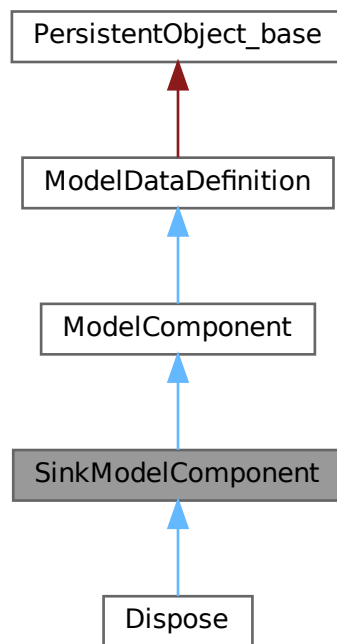
The documentation for this class was generated from the following files:

- applications/worker/service/SimulatorSessionService.h
- applications/worker/service/SimulatorSessionService.cpp

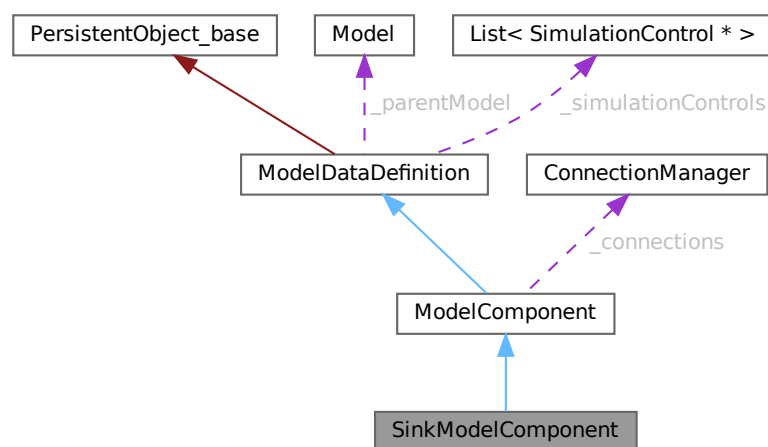
10.655 SinkModelComponent Class Reference

```
#include <kernel/simulator/SinkModelComponent.h>
```

Inheritance diagram for SinkModelComponent:



Collaboration diagram for SinkModelComponent:



Public Member Functions

- **SinkModelComponent** ([Model](#) *model, std::string componentTypename, std::string name="")

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.
- virtual std::string [show](#) () override
virtual public methods

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual **~ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.

- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelComponent](#)

- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber)=0
Handles a dispatched event for this component type.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * [_connections](#) = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification [_id](#)
- std::string [_typename](#)
- bool [_reportStatistics](#)
- bool [_hasChanged](#)
- unsigned int [_modelLevel](#) = 0
- [Model](#) * [_parentModel](#)
- [List](#)< [SimulationControl](#) * > * [_simulationControls](#) = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level [_traceLevelSpecific](#)
- bool [_traceLevelSpecificEnabled](#) = false

10.655.1 Detailed Description

This class is the basis for any component representing the end of a process flow, such as a [Dispose](#). It can remove entities from the system and collect statistics.

10.655.2 Member Function Documentation

10.655.2.1 `_check()`

```
bool SinkModelComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [Dispose](#).

10.655.2.2 `_loadInstance()`

```
bool SinkModelComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

Reimplemented in [Dispose](#).

10.655.2.3 `_saveInstance()`

```
void SinkModelComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

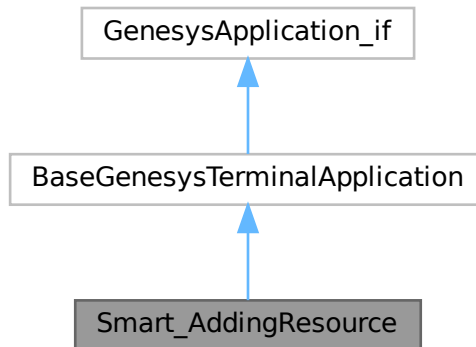
Reimplemented in [Dispose](#).

The documentation for this class was generated from the following files:

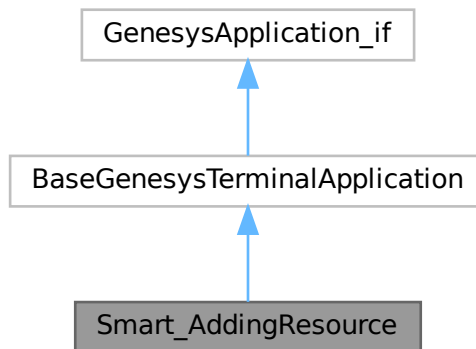
- kernel/simulator/SinkModelComponent.h
- kernel/simulator/SinkModelComponent.cpp

10.656 Smart_AddingResource Class Reference

Inheritance diagram for Smart_AddingResource:



Collaboration diagram for Smart_AddingResource:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.656.1 Member Function Documentation

10.656.1.1 main()

```
int Smart_AddingResource::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

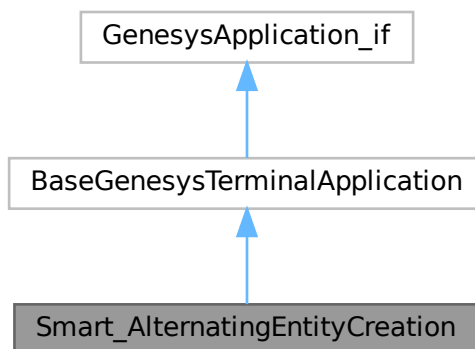
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

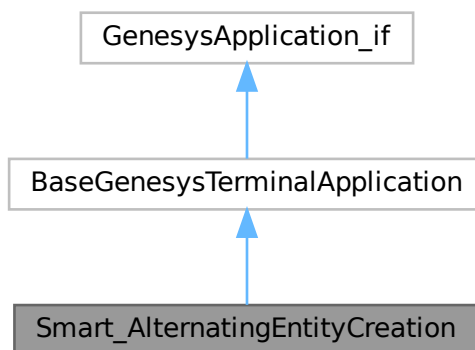
- applications/modelSpecific/arenaSmarts/Smart_AddingResource.h
- applications/modelSpecific/arenaSmarts/Smart_AddingResource.cpp

10.657 Smart_AlternatingEntityCreation Class Reference

Inheritance diagram for Smart_AlternatingEntityCreation:



Collaboration diagram for Smart_AlternatingEntityCreation:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.657.1 Member Function Documentation

10.657.1.1 main()

```
int Smart_AlternatingEntityCreation::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

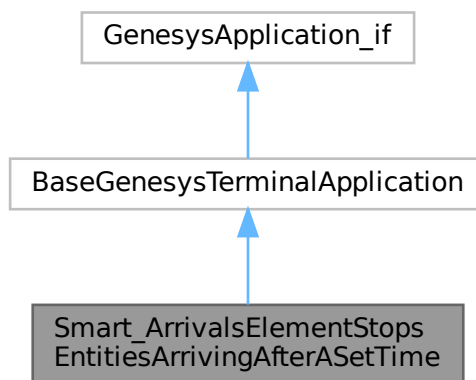
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

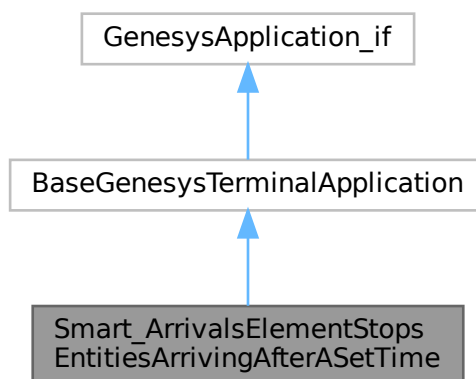
- applications/modelSpecific/arenaSmarts/Smart_AlternatingEntityCreation.h
- applications/modelSpecific/arenaSmarts/Smart_AlternatingEntityCreation.cpp

10.658 Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime Class Reference

Inheritance diagram for Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime:



Collaboration diagram for Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.658.1 Member Function Documentation

10.658.1.1 main()

```
int Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

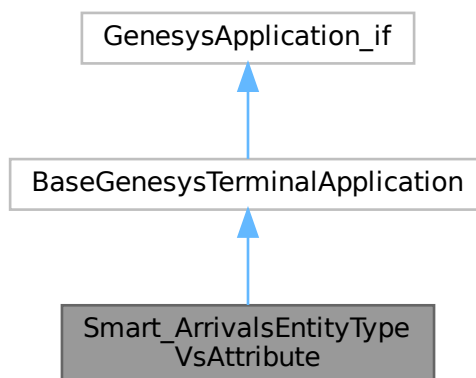
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

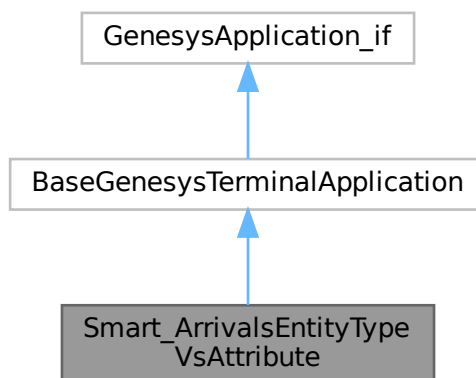
- applications/modelSpecific/arenaSmarts/Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime.h
- applications/modelSpecific/arenaSmarts/Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime.cpp

10.659 Smart_ArrivalsEntityTypeVsAttribute Class Reference

Inheritance diagram for Smart_ArrivalsEntityTypeVsAttribute:



Collaboration diagram for Smart_ArrivalsEntityTypeVsAttribute:



Public Member Functions

- virtual int `main` (int argc, char **argv) override

Public Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void `Simulate` (`Simulator` *genesys, `Model` *model, `PluginManager` *plugins)
- void `setDefaultTraceHandlers` (`TraceManager` *tm)
- void `setDefaultEventHandlers` (`OnEventManager` *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.659.1 Member Function Documentation

10.659.1.1 main()

```
int Smart_ArrivalsEntityTypeVsAttribute::main (
    int argc,
    char ** argv ) [override], [virtual]
```

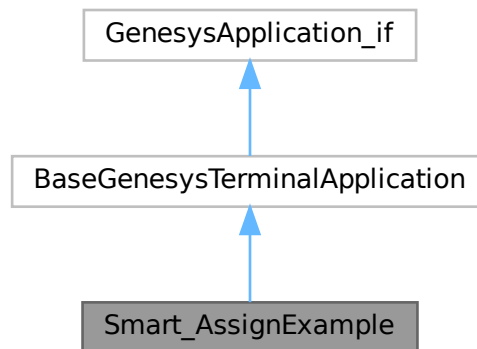
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

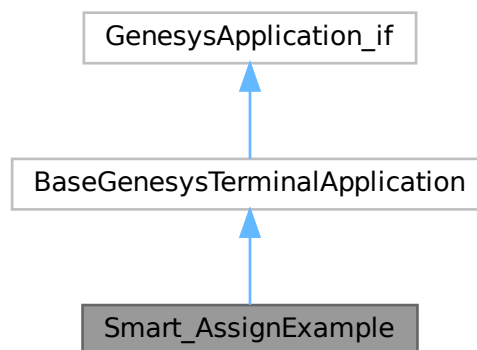
- applications/modelSpecific/arenaSmarts/Smart_ArrivalsEntityTypeVsAttribute.h
- applications/modelSpecific/arenaSmarts/Smart_ArrivalsEntityTypeVsAttribute.cpp

10.660 Smart_AssignExample Class Reference

Inheritance diagram for Smart_AssignExample:



Collaboration diagram for Smart_AssignExample:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.660.1 Member Function Documentation

10.660.1.1 main()

```
int Smart_AssignExample::main (
    int argc,
    char ** argv ) [override], [virtual]
```

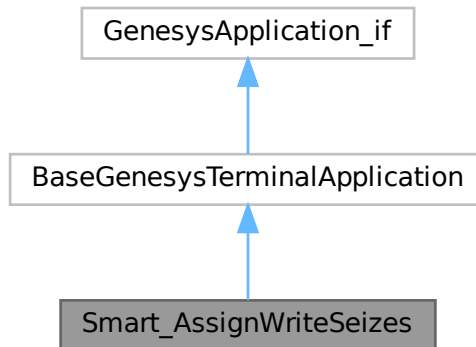
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

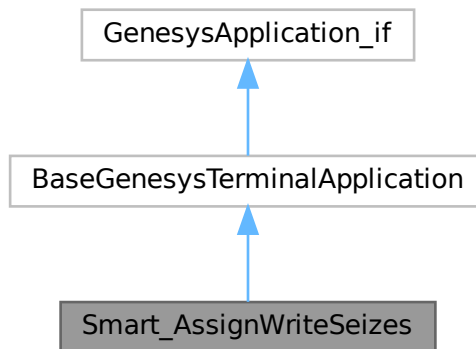
- applications/modelSpecific/arenaSmarts/Smart_AssignExample.h
- applications/modelSpecific/arenaSmarts/Smart_AssignExample.cpp

10.661 Smart_AssignWriteSeizes Class Reference

Inheritance diagram for Smart_AssignWriteSeizes:



Collaboration diagram for Smart_AssignWriteSeizes:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.661.1 Member Function Documentation

10.661.1.1 main()

```
int Smart_AssignWriteSeizes::main (
    int argc,
    char ** argv ) [override], [virtual]
```

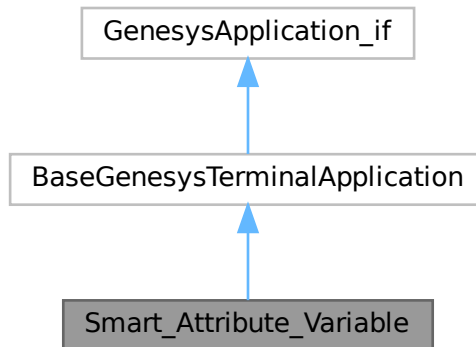
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

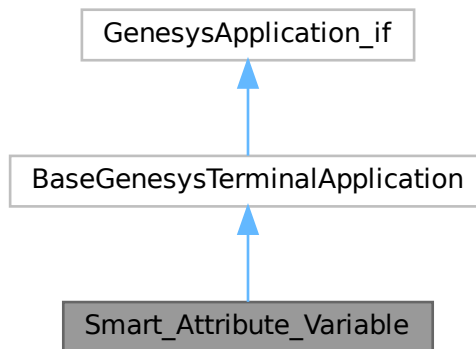
- applications/modelSpecific/smarts/Logic/Smart_AssignWriteSeizes.h
- applications/modelSpecific/smarts/Logic/Smart_AssignWriteSeizes.cpp

10.662 Smart_Attribute_Variable Class Reference

Inheritance diagram for Smart_Attribute_Variable:



Collaboration diagram for Smart_Attribute_Variable:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.662.1 Member Function Documentation

10.662.1.1 main()

```
int Smart_Attribute_Variable::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

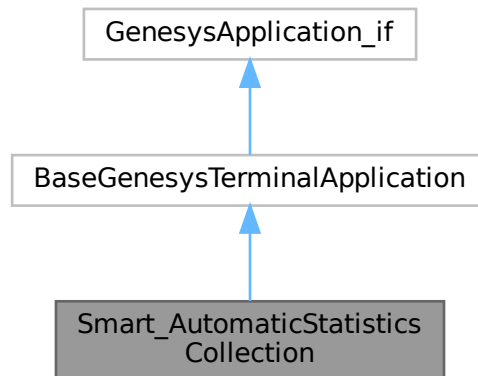
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

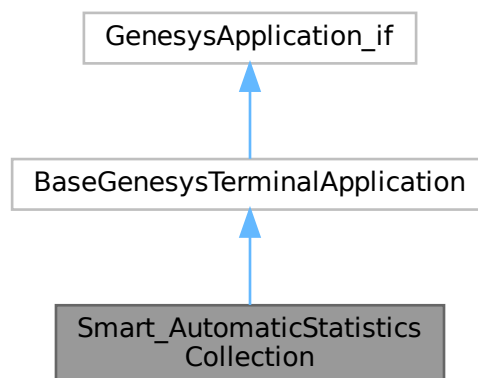
- applications/modelSpecific/smarts/Logic/Smart_Attribute_Variable.h
- applications/modelSpecific/smarts/Logic/Smart_Attribute_Variable.cpp

10.663 Smart_AutomaticStatisticsCollection Class Reference

Inheritance diagram for Smart_AutomaticStatisticsCollection:



Collaboration diagram for Smart_AutomaticStatisticsCollection:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.663.1 Member Function Documentation

10.663.1.1 main()

```
int Smart_AutomaticStatisticsCollection::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

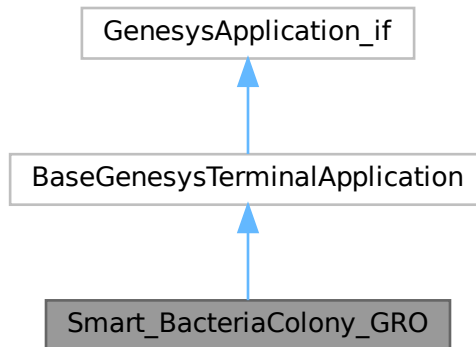
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

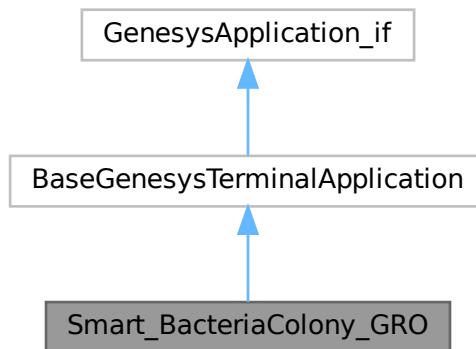
- applications/modelSpecific/arenaSmarts/Smart_AutomaticStatisticsCollection.h
- applications/modelSpecific/arenaSmarts/Smart_AutomaticStatisticsCollection.cpp

10.664 Smart_BacteriaColony_GRO Class Reference

Inheritance diagram for Smart_BacteriaColony_GRO:



Collaboration diagram for Smart_BacteriaColony_GRO:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.664.1 Member Function Documentation

10.664.1.1 main()

```
int Smart_BacteriaColony_GRO::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

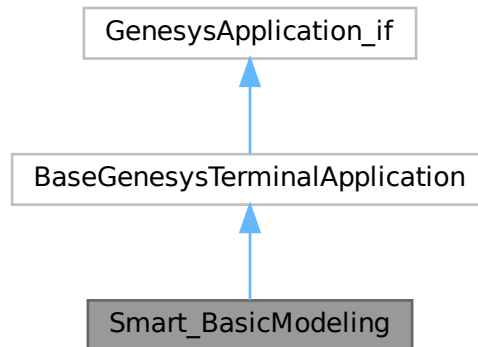
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

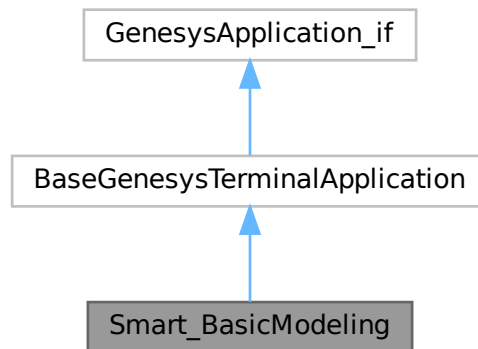
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BacteriaColony_GRO.h
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BacteriaColony_GRO.cpp

10.665 Smart_BasicModeling Class Reference

Inheritance diagram for Smart_BasicModeling:



Collaboration diagram for Smart_BasicModeling:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.665.1 Member Function Documentation

10.665.1.1 main()

```
int Smart_BasicModeling::main (
    int argc,
    char ** argv ) [override], [virtual]
```

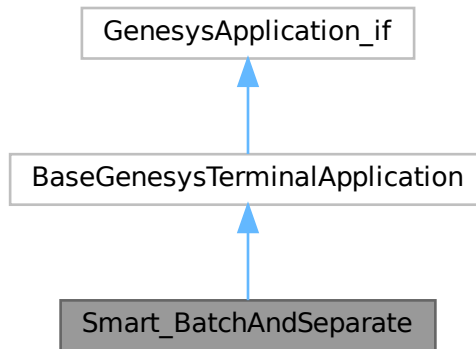
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

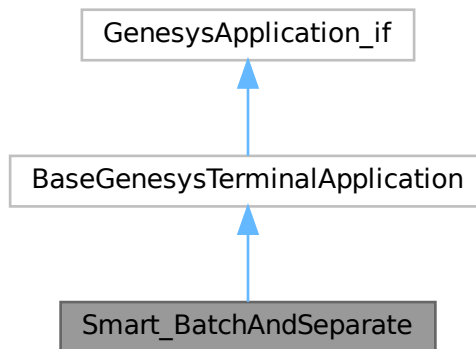
- applications/modelSpecific/arenaSmarts/Smart_BasicModeling.h
- applications/modelSpecific/arenaSmarts/Smart_BasicModeling.cpp

10.666 Smart_BatchAndSeparate Class Reference

Inheritance diagram for Smart_BatchAndSeparate:



Collaboration diagram for Smart_BatchAndSeparate:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.666.1 Member Function Documentation

10.666.1.1 main()

```
int Smart_BatchAndSeparate::main (
    int argc,
    char ** argv ) [override], [virtual]
```

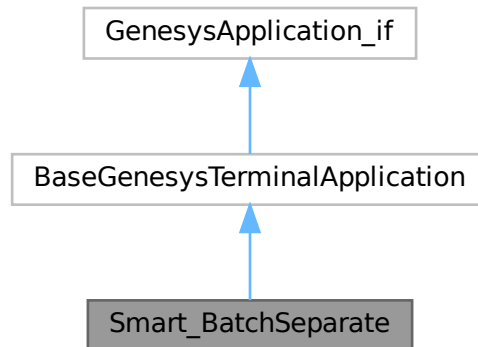
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

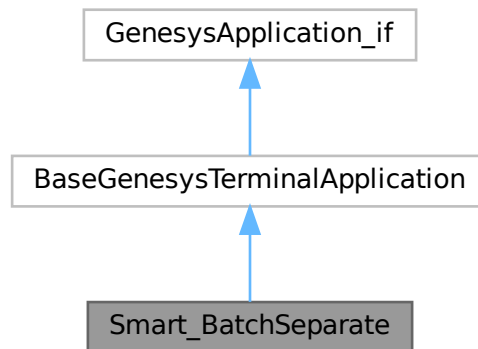
- applications/modelSpecific/arenaSmarts/Smart_BatchAndSeparate.h
- applications/modelSpecific/arenaSmarts/Smart_BatchAndSeparate.cpp

10.667 Smart_BatchSeparate Class Reference

Inheritance diagram for Smart_BatchSeparate:



Collaboration diagram for Smart_BatchSeparate:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.667.1 Member Function Documentation

10.667.1.1 main()

```
int Smart_BatchSeparate::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

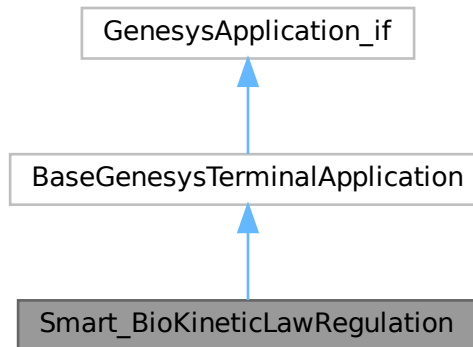
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

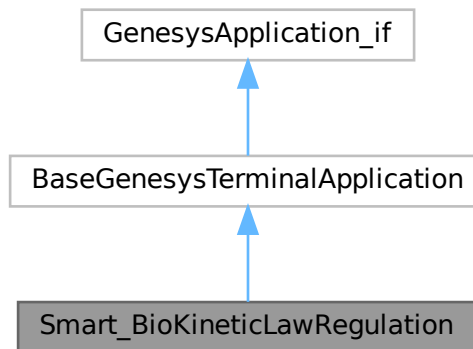
- applications/modelSpecific/smarts/Grouping/Smart_BatchSeparate.h
- applications/modelSpecific/smarts/Grouping/Smart_BatchSeparate.cpp

10.668 Smart_BioKineticLawRegulation Class Reference

Inheritance diagram for Smart_BioKineticLawRegulation:



Collaboration diagram for Smart_BioKineticLawRegulation:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.668.1 Member Function Documentation

10.668.1.1 main()

```
int Smart_BioKineticLawRegulation::main (
    int argc,
    char ** argv ) [override], [virtual]
```

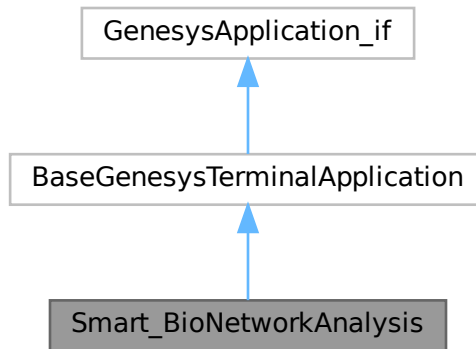
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

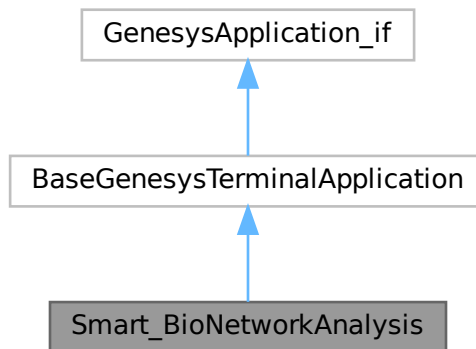
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioKineticLawRegulation.h
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioKineticLawRegulation.cpp

10.669 Smart_BioNetworkAnalysis Class Reference

Inheritance diagram for Smart_BioNetworkAnalysis:



Collaboration diagram for Smart_BioNetworkAnalysis:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.669.1 Member Function Documentation

10.669.1.1 main()

```
int Smart_BioNetworkAnalysis::main (
    int argc,
    char ** argv ) [override], [virtual]
```

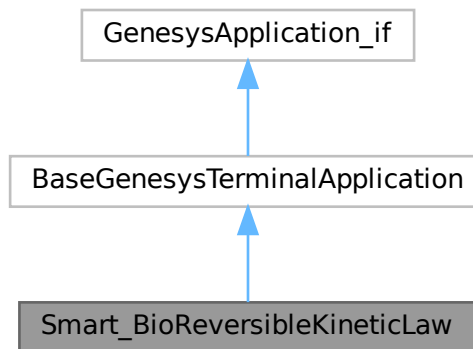
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

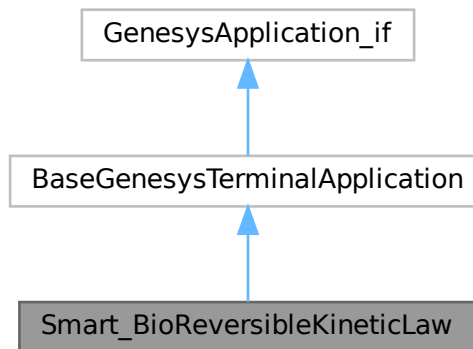
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioNetworkAnalysis.h
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioNetworkAnalysis.cpp

10.670 Smart_BioReversibleKineticLaw Class Reference

Inheritance diagram for Smart_BioReversibleKineticLaw:



Collaboration diagram for Smart_BioReversibleKineticLaw:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.670.1 Member Function Documentation

10.670.1.1 main()

```
int Smart_BioReversibleKineticLaw::main (
    int argc,
    char ** argv ) [override], [virtual]
```

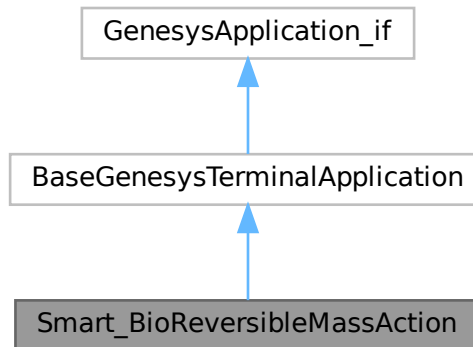
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

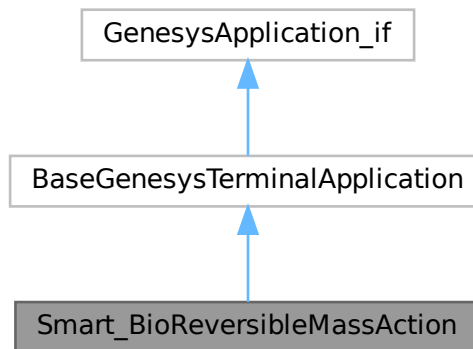
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioReversibleKineticLaw.h
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioReversibleKineticLaw.cpp

10.671 Smart_BioReversibleMassAction Class Reference

Inheritance diagram for Smart_BioReversibleMassAction:



Collaboration diagram for Smart_BioReversibleMassAction:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.671.1 Member Function Documentation

10.671.1.1 main()

```
int Smart_BioReversibleMassAction::main (
    int argc,
    char ** argv ) [override], [virtual]
```

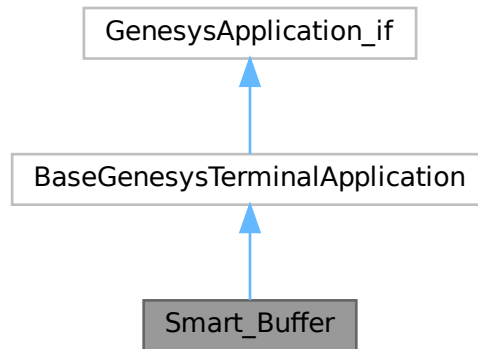
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

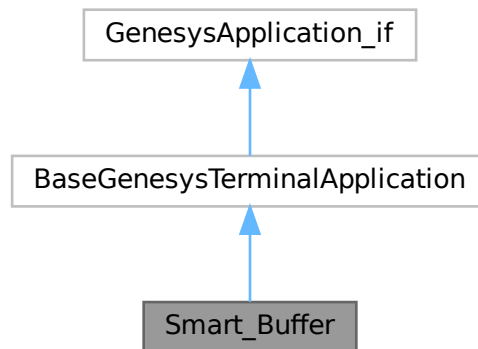
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioReversibleMassAction.h
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart_BioReversibleMassAction.cpp

10.672 Smart_Buffer Class Reference

Inheritance diagram for Smart_Buffer:



Collaboration diagram for Smart_Buffer:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.672.1 Member Function Documentation

10.672.1.1 main()

```
int Smart_Buffer::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

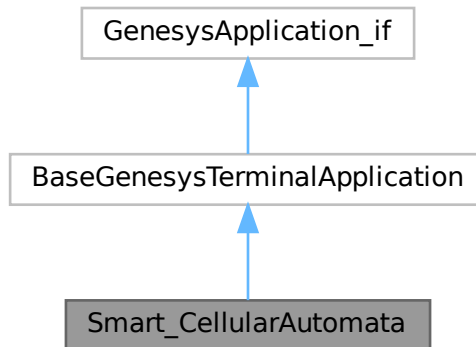
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

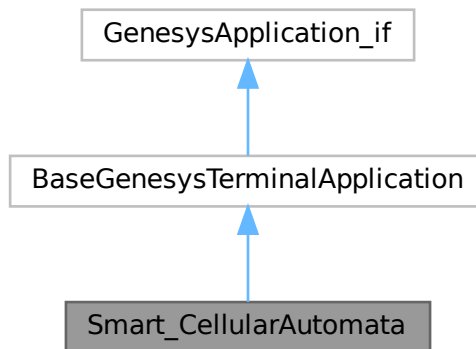
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_Buffer.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_Buffer.cpp

10.673 Smart_CellularAutomata Class Reference

Inheritance diagram for Smart_CellularAutomata:



Collaboration diagram for Smart_CellularAutomata:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.673.1 Member Function Documentation

10.673.1.1 main()

```
int Smart_CellularAutomata::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

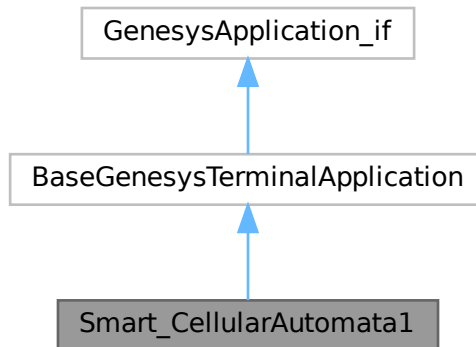
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

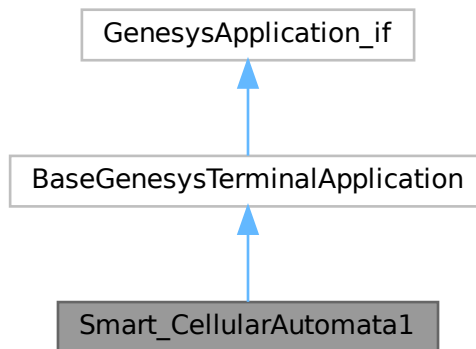
- applications/modelSpecific/smarts/ModalModel/Smart_CellularAutomata.h
- applications/modelSpecific/smarts/ModalModel/Smart_CellularAutomata.cpp

10.674 Smart_CellularAutomata1 Class Reference

Inheritance diagram for Smart_CellularAutomata1:



Collaboration diagram for Smart_CellularAutomata1:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.674.1 Member Function Documentation

10.674.1.1 main()

```
int Smart_CellularAutomata1::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

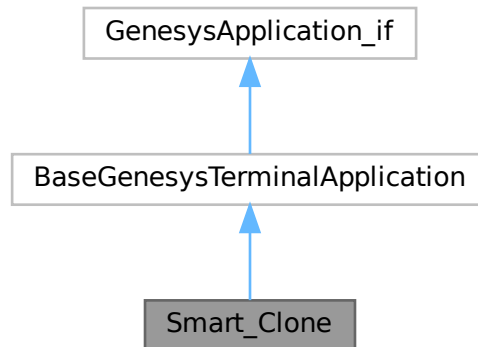
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

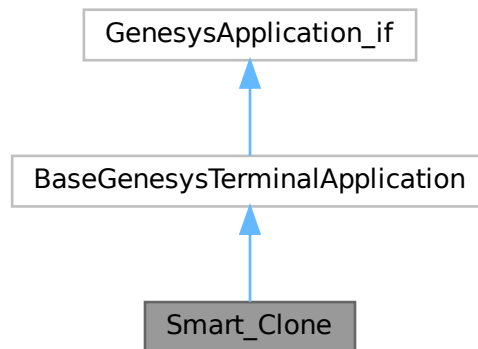
- applications/modelSpecific/smarts/ModalModel/Smart_CellularAutomata1.h
- applications/modelSpecific/smarts/ModalModel/Smart_CellularAutomata1.cpp

10.675 Smart_Clone Class Reference

Inheritance diagram for Smart_Clone:



Collaboration diagram for Smart_Clone:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.675.1 Member Function Documentation

10.675.1.1 main()

```
int Smart_Clone::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

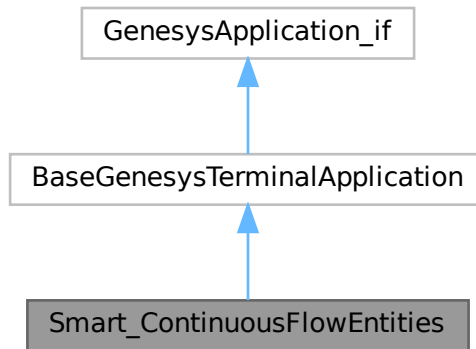
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

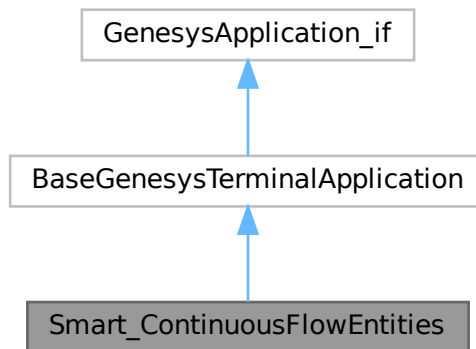
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_Clone.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_Clone.cpp

10.676 Smart_ContinuousFlowEntities Class Reference

Inheritance diagram for Smart_ContinuousFlowEntities:



Collaboration diagram for Smart_ContinuousFlowEntities:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.676.1 Member Function Documentation

10.676.1.1 main()

```
int Smart_ContinuousFlowEntities::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

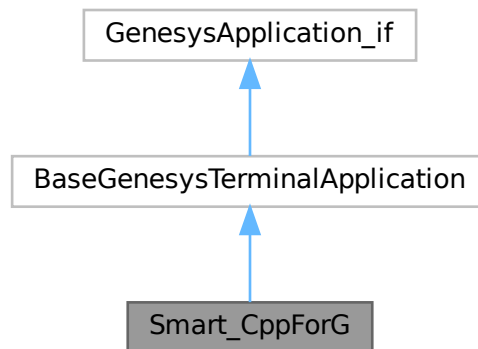
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

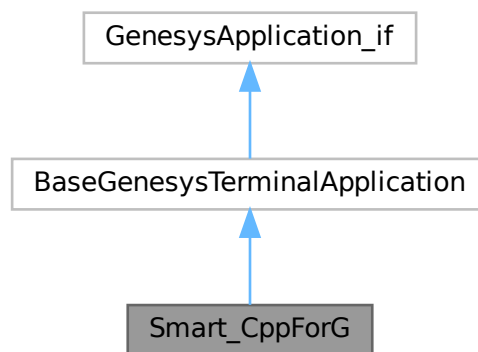
- applications/modelSpecific/arenaSmarts/Smart_ContinuousFlowEntities.h
- applications/modelSpecific/arenaSmarts/Smart_ContinuousFlowEntities.cpp

10.677 Smart_CppForG Class Reference

Inheritance diagram for Smart_CppForG:



Collaboration diagram for Smart_CppForG:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.677.1 Member Function Documentation

10.677.1.1 main()

```
int Smart_CppForG::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

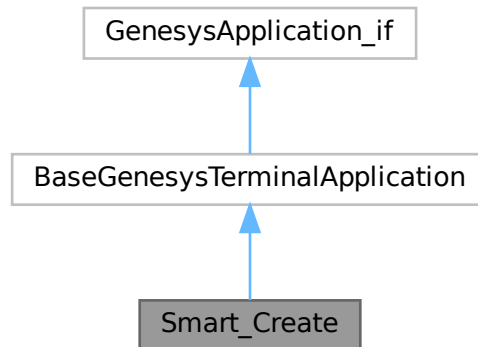
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

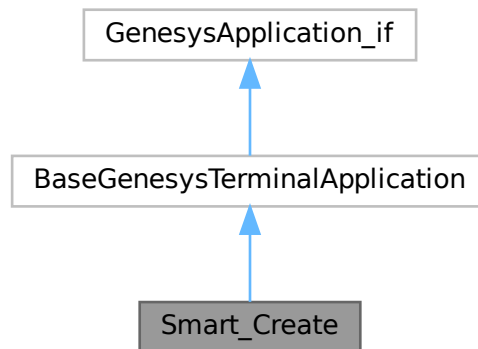
- applications/modelSpecific/smarts/ExternalIntegration/Smart_CppForG.h
- applications/modelSpecific/smarts/ExternalIntegration/Smart_CppForG.cpp

10.678 Smart_Create Class Reference

Inheritance diagram for Smart_Create:



Collaboration diagram for Smart_Create:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.678.1 Member Function Documentation

10.678.1.1 main()

```
int Smart_Create::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

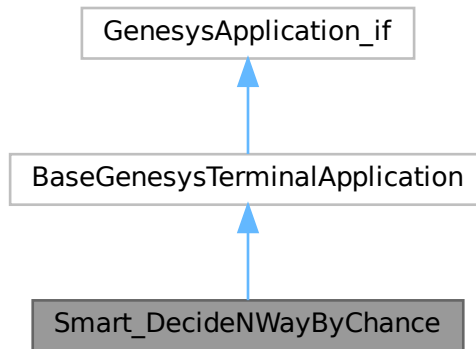
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

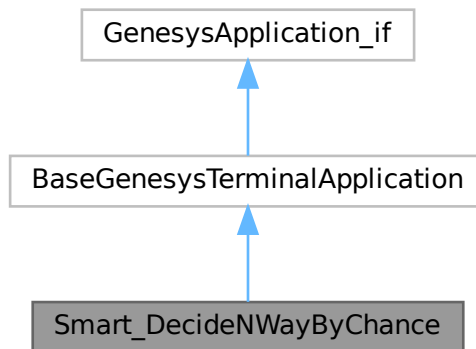
- applications/modelSpecific/arenaSmarts/Smart_Create.h
- applications/modelSpecific/arenaSmarts/Smart_Create.cpp

10.679 Smart_DecideNWayByChance Class Reference

Inheritance diagram for Smart_DecideNWayByChance:



Collaboration diagram for Smart_DecideNWayByChance:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.679.1 Member Function Documentation

10.679.1.1 main()

```
int Smart_DecideNWayByChance::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It create a new simulator instance, builds a simulation model and then simulate that model.

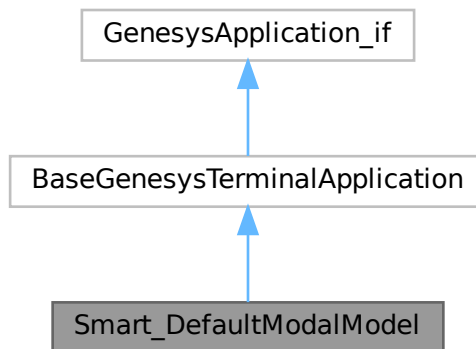
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

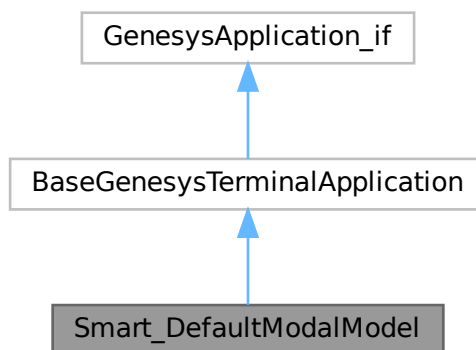
- applications/modelSpecific/arenaSmarts/Smart_DecideNWayByChance.h
- applications/modelSpecific/arenaSmarts/Smart_DecideNWayByChance.cpp

10.680 Smart_DefaultModalModel Class Reference

Inheritance diagram for Smart_DefaultModalModel:



Collaboration diagram for Smart_DefaultModalModel:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.680.1 Member Function Documentation

10.680.1.1 main()

```
int Smart_DefaultModalModel::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

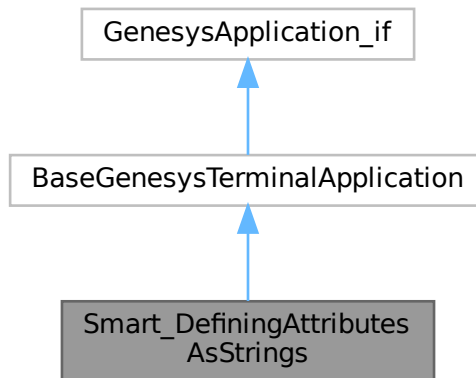
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

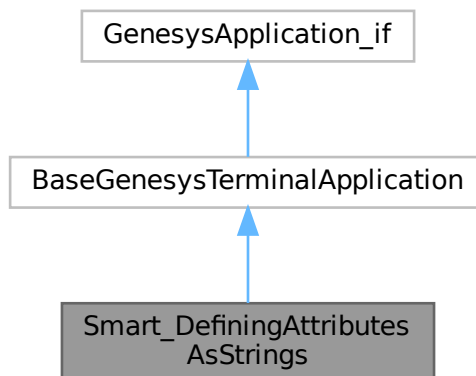
- applications/modelSpecific/smarts/ModalModel/Smart_DefaultModalModel.h
- applications/modelSpecific/smarts/ModalModel/Smaty_DefaultModalModel.cpp

10.681 Smart_DefiningAttributesAsStrings Class Reference

Inheritance diagram for Smart_DefiningAttributesAsStrings:



Collaboration diagram for Smart_DefiningAttributesAsStrings:



Public Member Functions

- virtual int `main` (int argc, char **argv) override

Public Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void `Simulate` (`Simulator` *genesys, `Model` *model, `PluginManager` *plugins)
- void `setDefaultTraceHandlers` (`TraceManager` *tm)
- void `setDefaultEventHandlers` (`OnEventManager` *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.681.1 Member Function Documentation

10.681.1.1 main()

```
int Smart_DefiningAttributesAsStrings::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

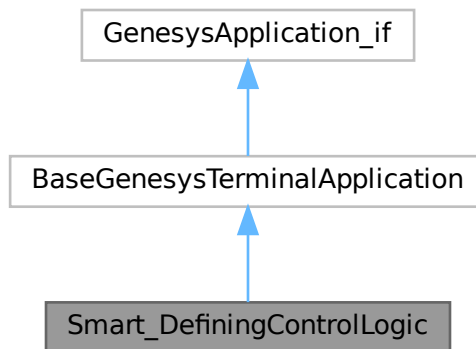
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

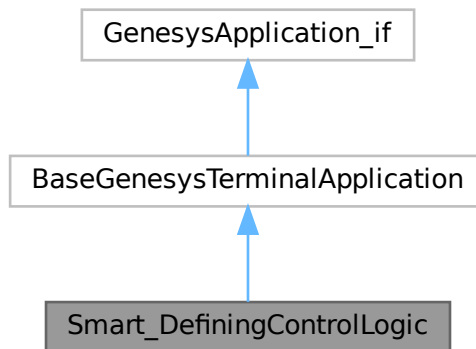
- applications/modelSpecific/arenaSmarts/Smart_DefiningAttributesAsStrings.h
- applications/modelSpecific/arenaSmarts/Smart_DefiningAttributesAsStrings.cpp

10.682 Smart_DefiningControlLogic Class Reference

Inheritance diagram for Smart_DefiningControlLogic:



Collaboration diagram for Smart_DefiningControlLogic:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.682.1 Member Function Documentation

10.682.1.1 main()

```
int Smart_DefiningControlLogic::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

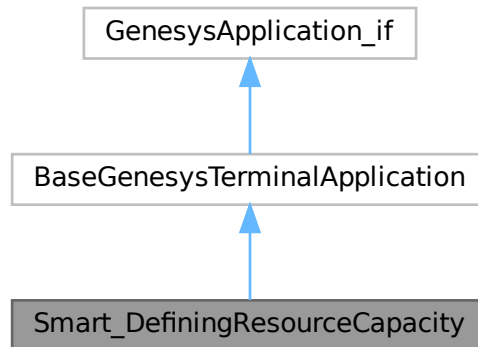
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

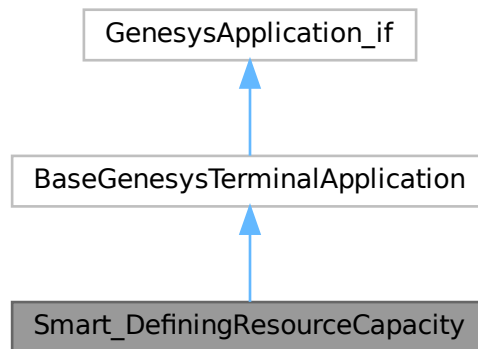
- applications/modelSpecific/arenaSmarts/Smart_DefiningControlLogic.h
- applications/modelSpecific/arenaSmarts/Smart_DefiningControlLogic.cpp

10.683 Smart_DefiningResourceCapacity Class Reference

Inheritance diagram for Smart_DefiningResourceCapacity:



Collaboration diagram for Smart_DefiningResourceCapacity:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.683.1 Member Function Documentation

10.683.1.1 main()

```
int Smart_DefiningResourceCapacity::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

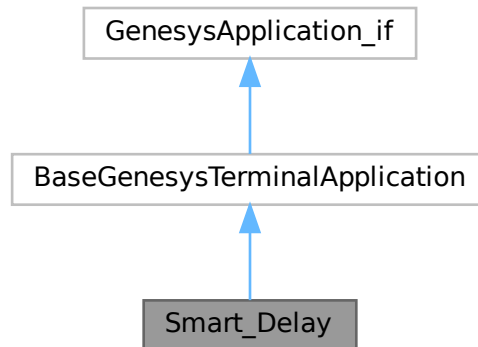
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

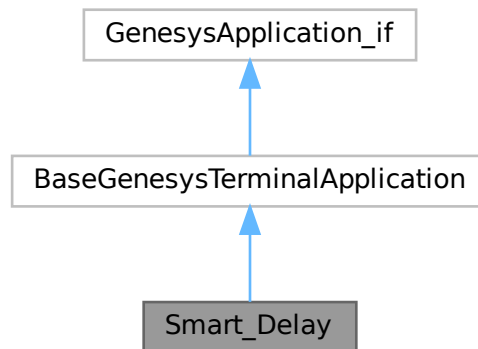
- applications/modelSpecific/arenaSmarts/Smart_DefiningResourceCapacity.h
- applications/modelSpecific/arenaSmarts/Smart_DefiningResourceCapacity.cpp

10.684 Smart_Delay Class Reference

Inheritance diagram for Smart_Delay:



Collaboration diagram for Smart_Delay:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.684.1 Member Function Documentation

10.684.1.1 main()

```
int Smart_Delay::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

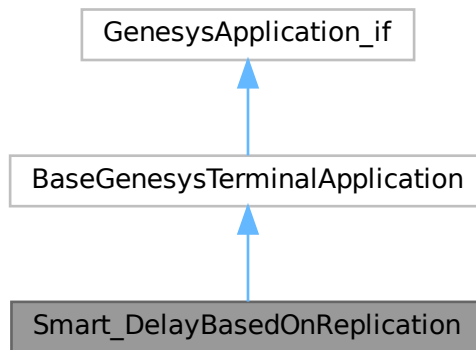
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

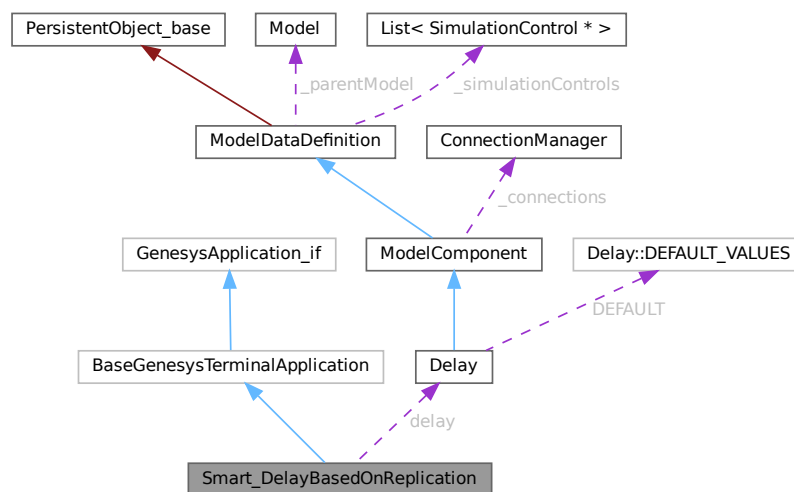
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_Delay.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_Delay.cpp

10.685 Smart_DelayBasedOnReplication Class Reference

Inheritance diagram for Smart_DelayBasedOnReplication:



Collaboration diagram for Smart_DelayBasedOnReplication:



Public Member Functions

- void `TestEventHandler` (`SimulationEvent` *re)
- virtual int `main` (int argc, char **argv) override

Public Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void `Simulate` (`Simulator` *genesys, `Model` *model, `PluginManager` *plugins)
- void `setDefaultTraceHandlers` (`TraceManager` *tm)
- void `setDefaultEventHandlers` (`OnEventManager` *oem)

Public Attributes

- [Delay](#) * **delay**
- int * **proctimes**

Additional Inherited Members**Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.685.1 Member Function Documentation**10.685.1.1 main()**

```
int Smart_DelayBasedOnReplication::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

10.685.1.2 TestEventHandler()

```
void Smart_DelayBasedOnReplication::TestEventHandler (
    SimulationEvent * re )
```

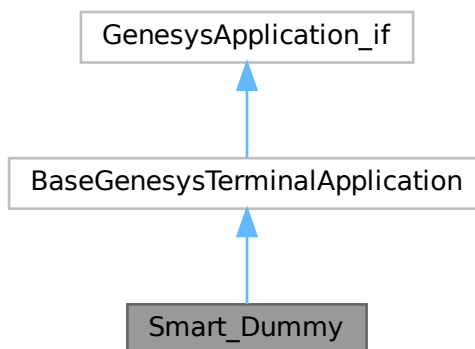
This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

The documentation for this class was generated from the following files:

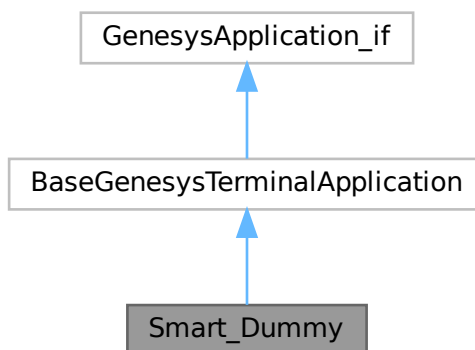
- applications/modelSpecific/arenaSmarts/Smart_DelayBasedOnReplication.h
- applications/modelSpecific/arenaSmarts/Smart_DelayBasedOnReplication.cpp

10.686 Smart_Dummy Class Reference

Inheritance diagram for Smart_Dummy:



Collaboration diagram for Smart_Dummy:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.686.1 Member Function Documentation

10.686.1.1 main()

```
int Smart_Dummy::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

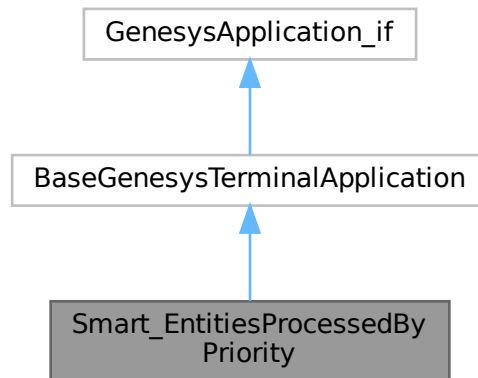
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

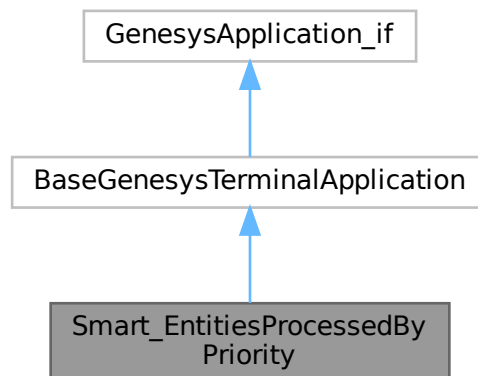
- applications/modelSpecific/smarts/ModalModel/Smart_Dummy.h
- applications/modelSpecific/smarts/ModalModel/Smart_Dummy.cpp

10.687 Smart_EntitiesProcessedByPriority Class Reference

Inheritance diagram for Smart_EntitiesProcessedByPriority:



Collaboration diagram for Smart_EntitiesProcessedByPriority:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.687.1 Member Function Documentation

10.687.1.1 main()

```
int Smart_EntitiesProcessedByPriority::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

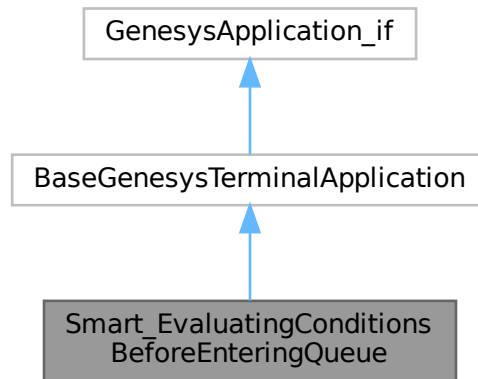
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

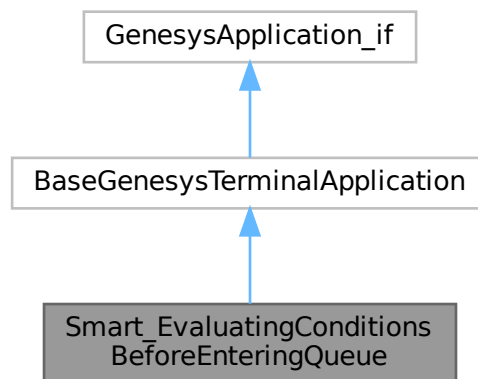
- applications/modelSpecific/arenaSmarts/Smart_EntitiesProcessedByPriority.h
- applications/modelSpecific/arenaSmarts/Smart_EntitiesProcessedByPriority.cpp

10.688 Smart_EvaluatingConditionsBeforeEnteringQueue Class Reference

Inheritance diagram for Smart_EvaluatingConditionsBeforeEnteringQueue:



Collaboration diagram for Smart_EvaluatingConditionsBeforeEnteringQueue:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.688.1 Member Function Documentation

10.688.1.1 main()

```
int Smart_EvaluatingConditionsBeforeEnteringQueue::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

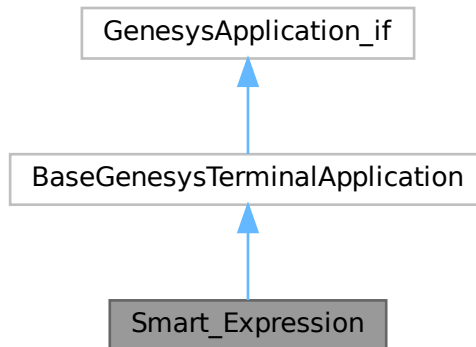
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

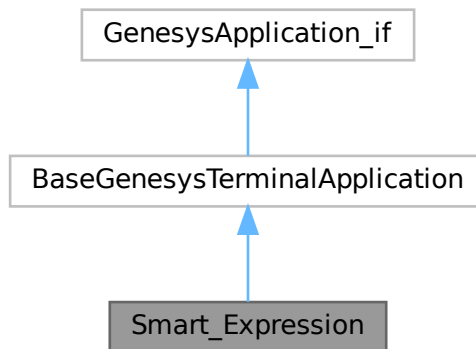
- applications/modelSpecific/arenaSmarts/Smart_EvaluatingConditionsBeforeEnteringQueue.h
- applications/modelSpecific/arenaSmarts/Smart_EvaluatingConditionsBeforeEnteringQueue.cpp

10.689 Smart_Expression Class Reference

Inheritance diagram for Smart_Expression:



Collaboration diagram for Smart_Expression:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.689.1 Member Function Documentation

10.689.1.1 main()

```
int Smart_Expression::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

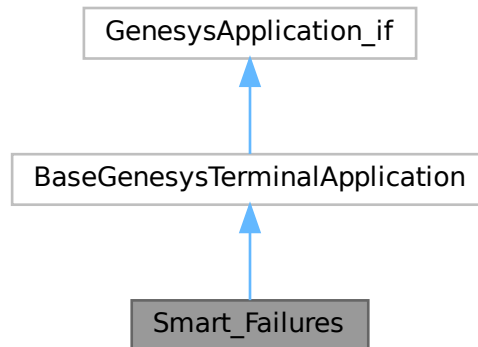
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

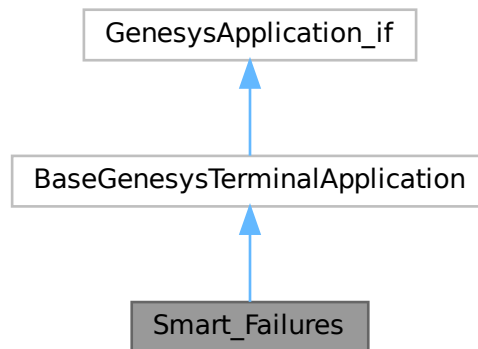
- applications/modelSpecific/arenaSmarts/Smart_Expression.h
- applications/modelSpecific/arenaSmarts/Smart_Expression.cpp

10.690 Smart_Failures Class Reference

Inheritance diagram for Smart_Failures:



Collaboration diagram for Smart_Failures:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.690.1 Member Function Documentation

10.690.1.1 main()

```
int Smart_Failures::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

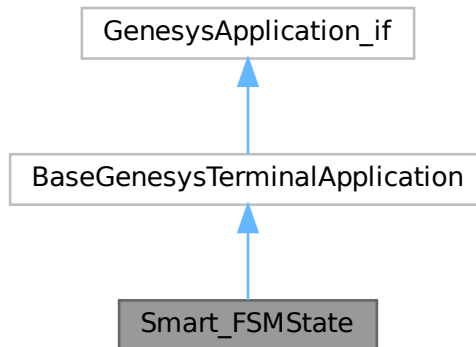
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

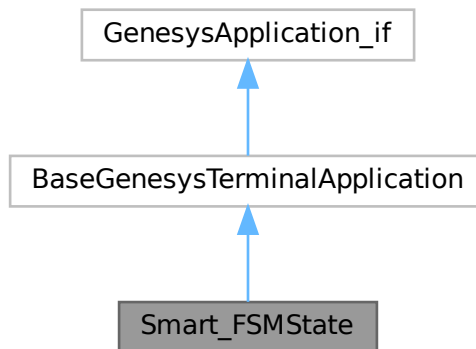
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_Failures.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_Failures.cpp

10.691 Smart_FSMState Class Reference

Inheritance diagram for Smart_FSMState:



Collaboration diagram for Smart_FSMState:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.691.1 Member Function Documentation

10.691.1.1 main()

```
int Smart_FSMState::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

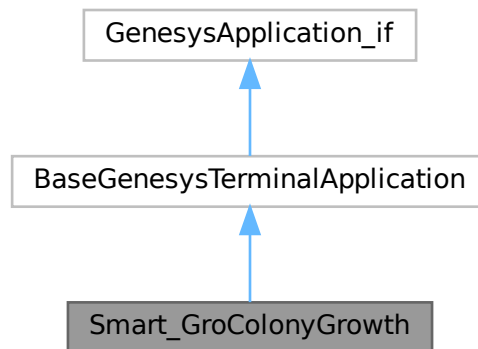
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

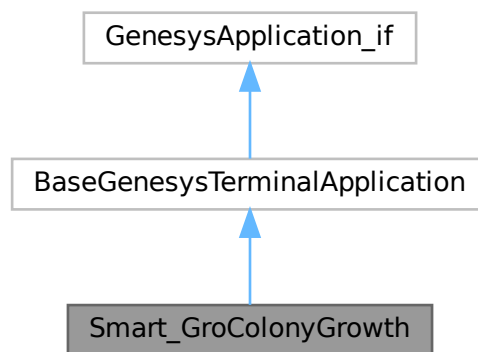
- applications/modelSpecific/smarts/ModalModel/Smart_FSMState.h
- applications/modelSpecific/smarts/ModalModel/Smart_FSMState.cpp

10.692 Smart_GroColonyGrowth Class Reference

Inheritance diagram for Smart_GroColonyGrowth:



Collaboration diagram for Smart_GroColonyGrowth:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.692.1 Member Function Documentation

10.692.1.1 main()

```
int Smart_GroColonyGrowth::main (
    int argc,
    char ** argv ) [override], [virtual]
```

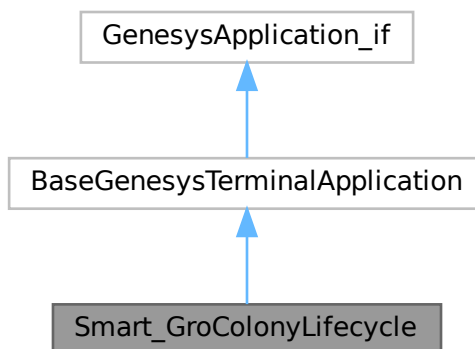
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

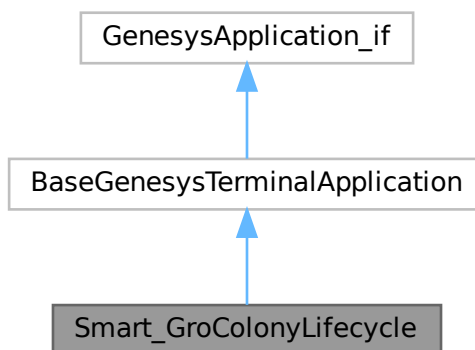
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart_GroColonyGrowth.h
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart_GroColonyGrowth.cpp

10.693 Smart_GroColonyLifecycle Class Reference

Inheritance diagram for Smart_GroColonyLifecycle:



Collaboration diagram for Smart_GroColonyLifecycle:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.693.1 Member Function Documentation

10.693.1.1 main()

```
int Smart_GroColonyLifecycle::main (
    int argc,
    char ** argv ) [override], [virtual]
```

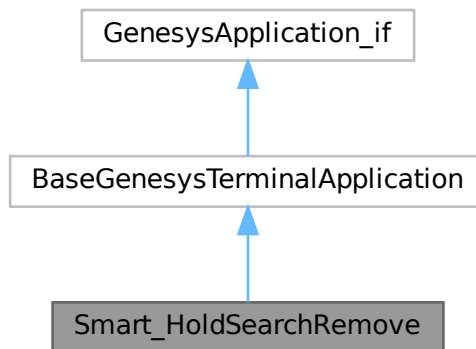
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

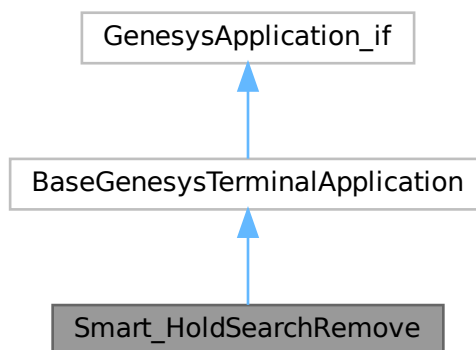
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart_GroColonyLifecycle.h
- applications/modelSpecific/smarts/BiochemicalSimulation/Smart_GroColonyLifecycle.cpp

10.694 Smart_HoldSearchRemove Class Reference

Inheritance diagram for Smart_HoldSearchRemove:



Collaboration diagram for Smart_HoldSearchRemove:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.694.1 Member Function Documentation

10.694.1.1 main()

```
int Smart_HoldSearchRemove::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

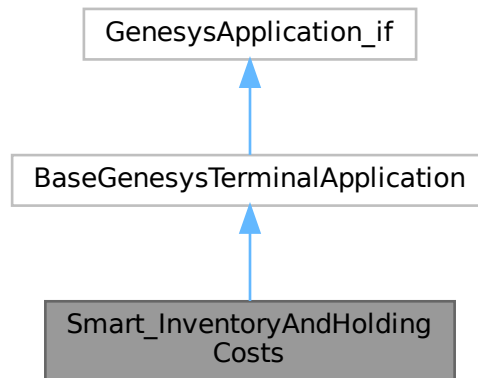
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

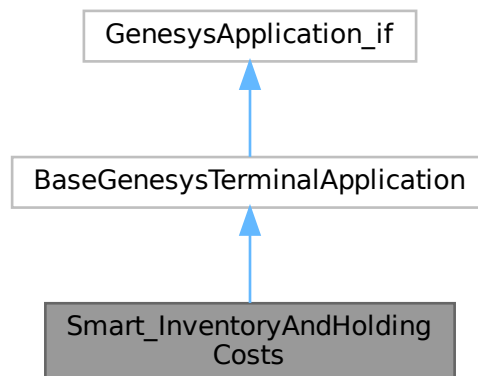
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_HoldSearchRemove.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_HoldSearchRemove.cpp

10.695 Smart_InventoryAndHoldingCosts Class Reference

Inheritance diagram for Smart_InventoryAndHoldingCosts:



Collaboration diagram for Smart_InventoryAndHoldingCosts:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.695.1 Member Function Documentation

10.695.1.1 main()

```
int Smart_InventoryAndHoldingCosts::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

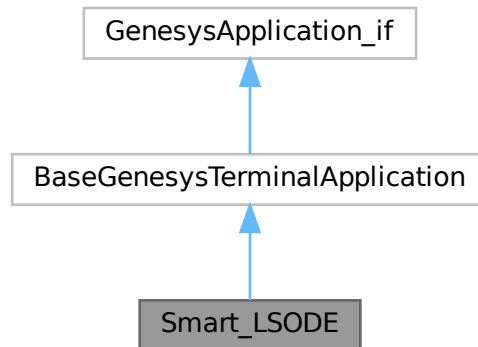
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

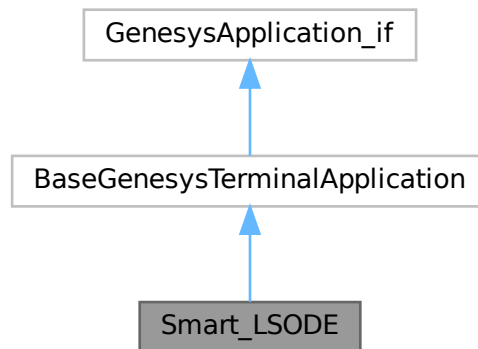
- applications/modelSpecific/arenaSmarts/Smart_InventoryAndHoldingCosts.h
- applications/modelSpecific/arenaSmarts/Smart_InventoryAndHoldingCosts.cpp

10.696 Smart_LCODE Class Reference

Inheritance diagram for Smart_LCODE:



Collaboration diagram for Smart_LCODE:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.696.1 Member Function Documentation

10.696.1.1 main()

```
int Smart_LCODE::main (
    int argc,
    char ** argv ) [override], [virtual]
```

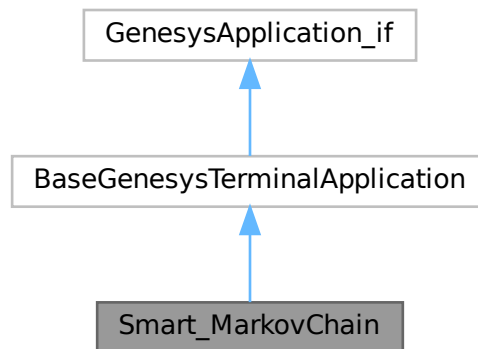
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

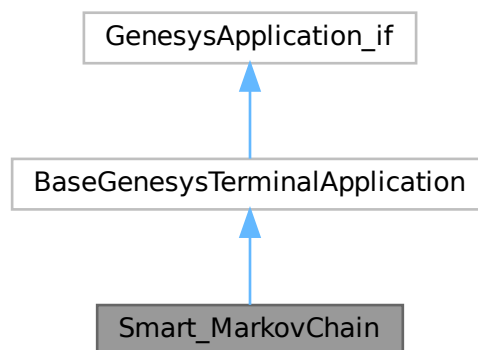
- applications/modelSpecific/smarts/Continuous/Smart_LCODE.h
- applications/modelSpecific/smarts/Continuous/Smart_LCODE.cpp

10.697 Smart_MarkovChain Class Reference

Inheritance diagram for Smart_MarkovChain:



Collaboration diagram for Smart_MarkovChain:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.697.1 Member Function Documentation

10.697.1.1 main()

```
int Smart_MarkovChain::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

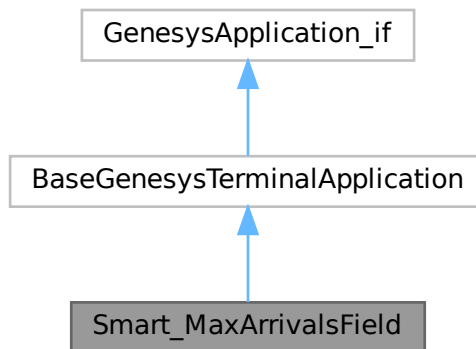
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

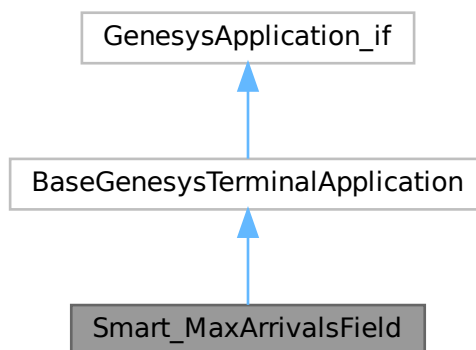
- applications/modelSpecific/smarts/AnalyticalModeling/Smart_MarkovChain.h
- applications/modelSpecific/smarts/AnalyticalModeling/Smart_MarkovChain.cpp

10.698 Smart_MaxArrivalsField Class Reference

Inheritance diagram for Smart_MaxArrivalsField:



Collaboration diagram for Smart_MaxArrivalsField:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.698.1 Member Function Documentation

10.698.1.1 main()

```
int Smart_MaxArrivalsField::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

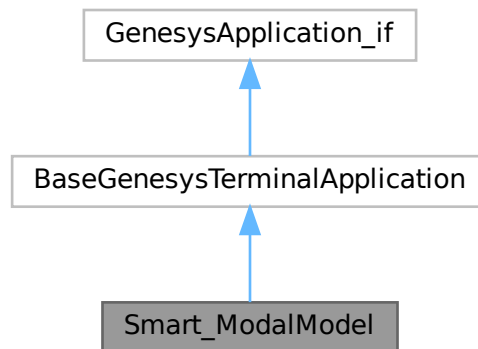
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

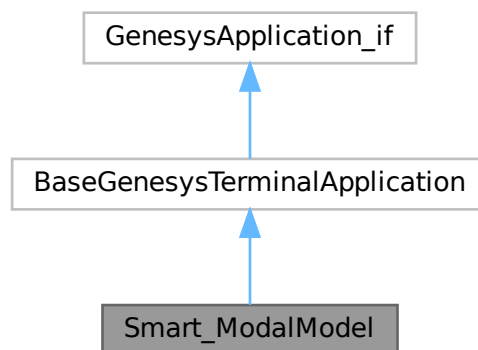
- applications/modelSpecific/arenaSmarts/Smart_MaxArrivalsField.h
- applications/modelSpecific/arenaSmarts/Smart_MaxArrivalsField.cpp

10.699 Smart_ModalModel Class Reference

Inheritance diagram for Smart_ModalModel:



Collaboration diagram for Smart_ModalModel:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.699.1 Member Function Documentation

10.699.1.1 main()

```
int Smart_ModalModel::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

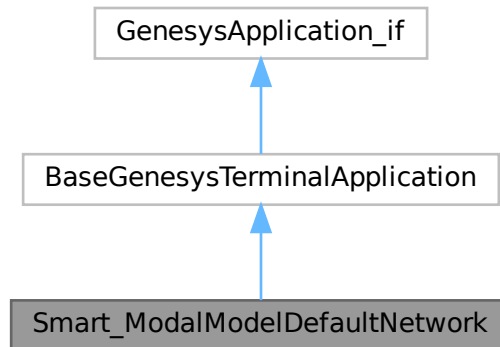
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

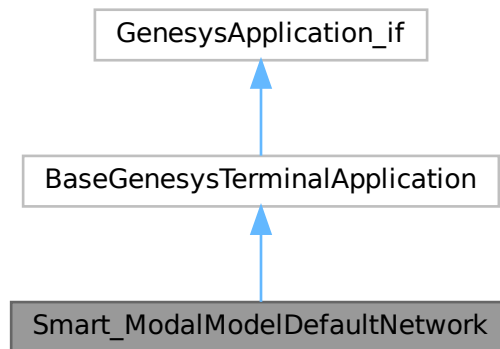
- applications/modelSpecific/smarts/ModalModel/Smart_Modalmodel.h
- applications/modelSpecific/smarts/ModalModel/Smart_Modalmodel.cpp

10.700 Smart_ModelModelDefaultNetwork Class Reference

Inheritance diagram for Smart_ModelModelDefaultNetwork:



Collaboration diagram for Smart_ModelModelDefaultNetwork:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.700.1 Member Function Documentation

10.700.1.1 main()

```
int Smart_ModalModelDefaultNetwork::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

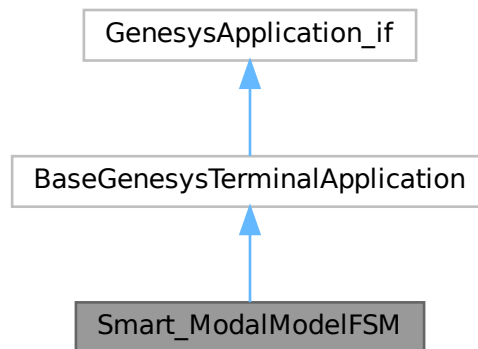
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

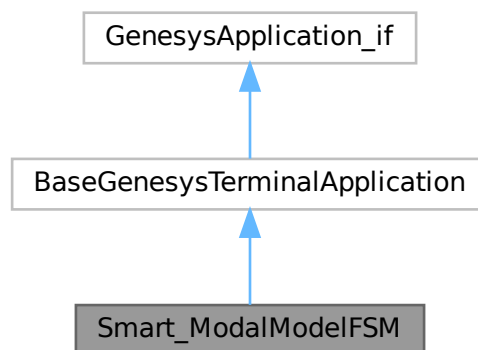
- applications/modelSpecific/smarts/ModalModel/Smart_ModalModelDefaultNetwork.h
- applications/modelSpecific/smarts/ModalModel/Smart_ModalModelDefaultNetwork.cpp

10.701 Smart_ModalModelFSM Class Reference

Inheritance diagram for Smart_ModalModelFSM:



Collaboration diagram for Smart_ModalModelFSM:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.701.1 Member Function Documentation

10.701.1.1 main()

```
int Smart_ModalModelFSM::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

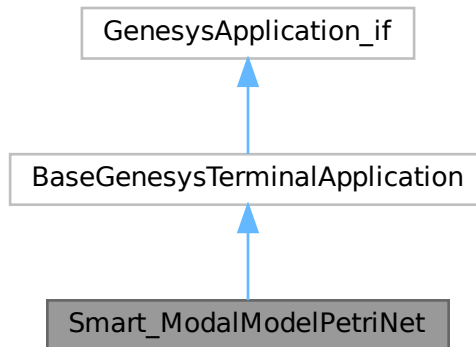
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

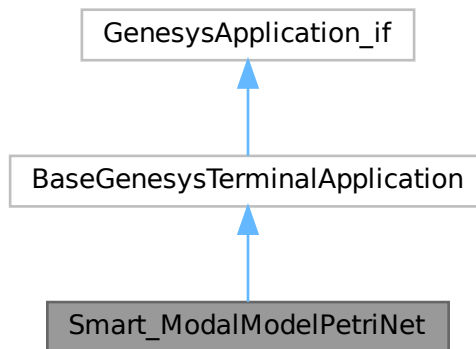
- applications/modelSpecific/smarts/ModalModel/Smart_ModalModelFSM.h
- applications/modelSpecific/smarts/ModalModel/Smart_ModalModelFSM.cpp

10.702 Smart_ModelModelPetriNet Class Reference

Inheritance diagram for Smart_ModelModelPetriNet:



Collaboration diagram for Smart_ModelModelPetriNet:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.702.1 Member Function Documentation

10.702.1.1 main()

```
int Smart_ModalModelPetriNet::main (
    int argc,
    char ** argv ) [override], [virtual]
```

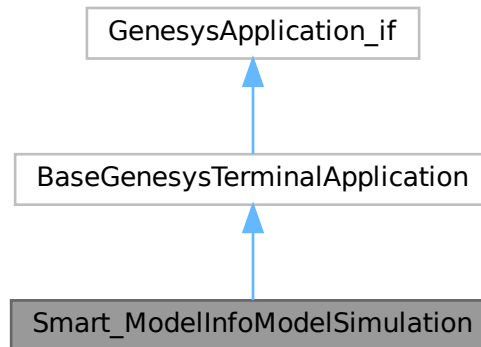
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

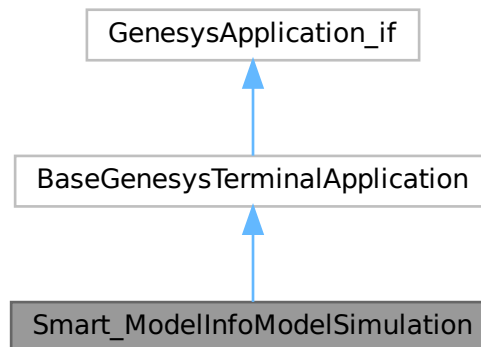
- applications/modelSpecific/smarts/ModalModel/Smart_ModalModelPetriNet.h
- applications/modelSpecific/smarts/ModalModel/Smart_ModalModelPetriNet.cpp

10.703 Smart_ModelInfoModelSimulation Class Reference

Inheritance diagram for Smart_ModelInfoModelSimulation:



Collaboration diagram for Smart_ModelInfoModelSimulation:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.703.1 Member Function Documentation

10.703.1.1 main()

```
int Smart_ModelInfoModelSimulation::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

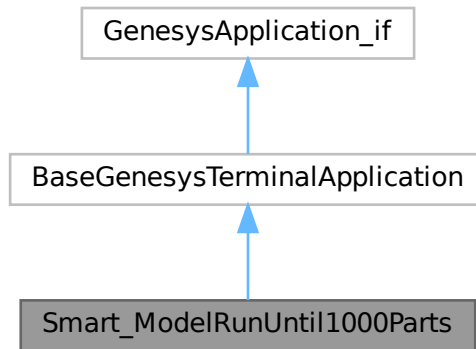
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

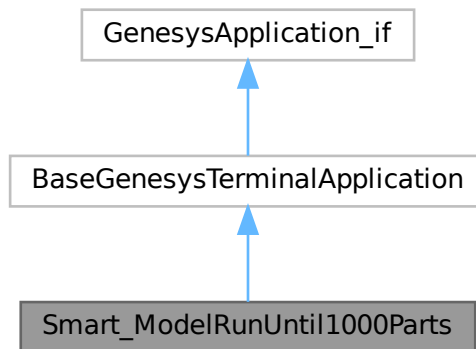
- applications/modelSpecific/smarts/Logic/Smart_ModelInfoModelSimulation.h
- applications/modelSpecific/smarts/Logic/Smart_ModelInfoModelSimulation.cpp

10.704 Smart_ModelRunUntil1000Parts Class Reference

Inheritance diagram for Smart_ModelRunUntil1000Parts:



Collaboration diagram for Smart_ModelRunUntil1000Parts:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.704.1 Member Function Documentation

10.704.1.1 main()

```
int Smart_ModelRunUntil1000Parts::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It create a new simulator instance, builds a simulation model and then simulate that model.

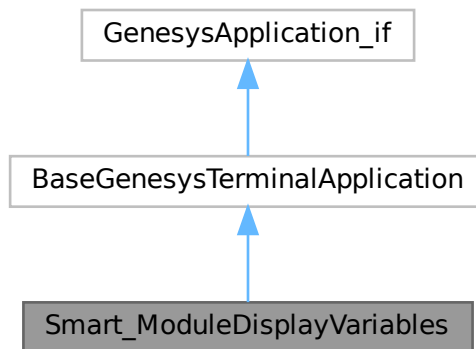
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

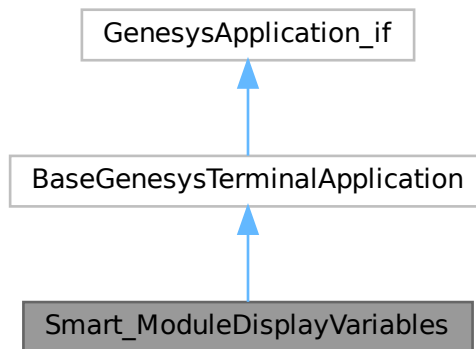
- applications/modelSpecific/arenaSmarts/Smart_ModelRunUntil1000Parts.h
- applications/modelSpecific/arenaSmarts/Smart_ModelRunUntil1000Parts.cpp

10.705 Smart_ModuleDisplayVariables Class Reference

Inheritance diagram for Smart_ModuleDisplayVariables:



Collaboration diagram for Smart_ModuleDisplayVariables:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.705.1 Member Function Documentation

10.705.1.1 main()

```
int Smart_ModuleDisplayVariables::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

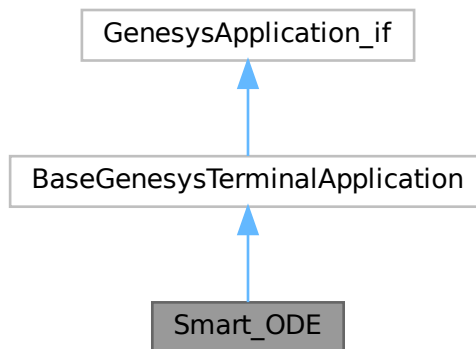
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

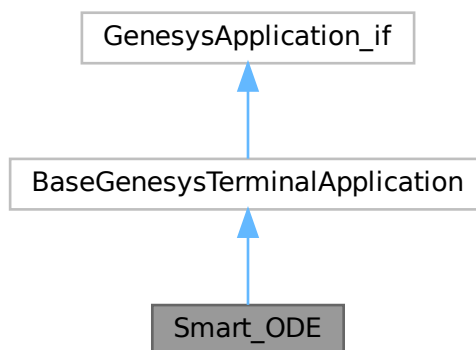
- applications/modelSpecific/arenaSmarts/Smart_ModuleDisplayVariables.h
- applications/modelSpecific/arenaSmarts/Smart_ModuleDisplayVariables.cpp

10.706 Smart_ODE Class Reference

Inheritance diagram for Smart_ODE:



Collaboration diagram for Smart_ODE:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.706.1 Member Function Documentation

10.706.1.1 main()

```
int Smart_ODE::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

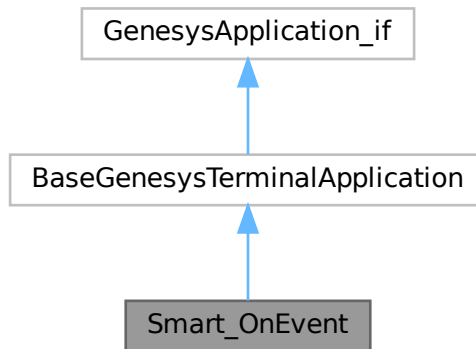
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

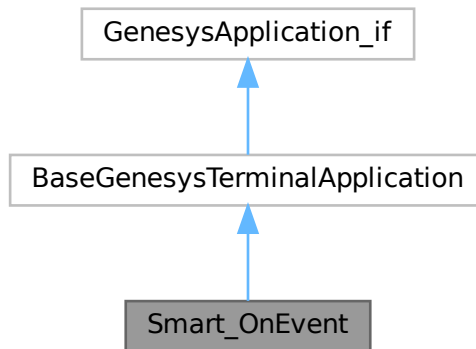
- applications/modelSpecific/smarts/Continuous/Smart_ODE.h
- applications/modelSpecific/smarts/Continuous/Smart_ODE.cpp

10.707 Smart_OnEvent Class Reference

Inheritance diagram for Smart_OnEvent:



Collaboration diagram for Smart_OnEvent:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override
- virtual void [onBreakpointHandler](#) ([SimulationEvent](#) *re) override
- virtual void [onEntityCreateHandler](#) ([SimulationEvent](#) *re) override
- virtual void [onEntityMoveHandler](#) ([SimulationEvent](#) *re) override
- virtual void [onSimulationStartHandler](#) ([SimulationEvent](#) *re) override
- virtual void [onReplicationStartHandler](#) ([SimulationEvent](#) *re) override
- virtual void [onReplicationStepHandler](#) ([SimulationEvent](#) *re) override
- virtual void [onProcessEventHandler](#) ([SimulationEvent](#) *re) override

- virtual void [onReplicationEndHandler](#) ([SimulationEvent](#) *re) override
- virtual void [onSimulationEndHandler](#) ([SimulationEvent](#) *re) override
- virtual void [onSimulationPausedHandler](#) ([SimulationEvent](#) *re) override
- virtual void [onSimulationResumeHandler](#) ([SimulationEvent](#) *re) override
- virtual void [onEntityRemoveHandler](#) ([SimulationEvent](#) *re) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)

10.707.1 Member Function Documentation

10.707.1.1 main()

```
int Smart_OnEvent::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

10.707.1.2 onBreakpointHandler()

```
void Smart_OnEvent::onBreakpointHandler (
    SimulationEvent * re ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

10.707.1.3 onEntityCreateHandler()

```
void Smart_OnEvent::onEntityCreateHandler (
    SimulationEvent * re ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

10.707.1.4 onEntityMoveHandler()

```
void Smart_OnEvent::onEntityMoveHandler (
    SimulationEvent * re ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

10.707.1.5 onEntityRemoveHandler()

```
void Smart_OnEvent::onEntityRemoveHandler (
    SimulationEvent * re ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

10.707.1.6 onProcessEventHandler()

```
void Smart_OnEvent::onProcessEventHandler (
    SimulationEvent * re ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

10.707.1.7 onReplicationEndHandler()

```
void Smart_OnEvent::onReplicationEndHandler (
    SimulationEvent * re ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

10.707.1.8 onReplicationStartHandler()

```
void Smart_OnEvent::onReplicationStartHandler (
    SimulationEvent * re ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

10.707.1.9 onReplicationStepHandler()

```
void Smart_OnEvent::onReplicationStepHandler (
    SimulationEvent * re ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

10.707.1.10 onSimulationEndHandler()

```
void Smart_OnEvent::onSimulationEndHandler (
    SimulationEvent * re ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

10.707.1.11 onSimulationPausedHandler()

```
void Smart_OnEvent::onSimulationPausedHandler (
    SimulationEvent * re ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

10.707.1.12 onSimulationResumeHandler()

```
void Smart_OnEvent::onSimulationResumeHandler (
    SimulationEvent * re ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

10.707.1.13 onSimulationStartHandler()

```
void Smart_OnEvent::onSimulationStartHandler (
    SimulationEvent * re ) [override], [virtual]
```

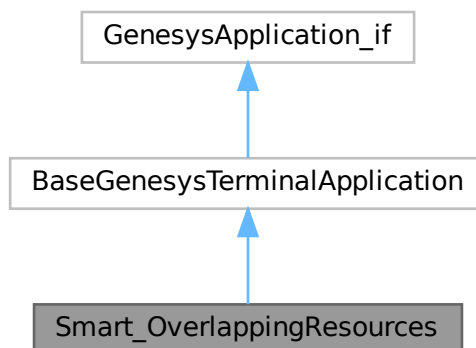
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

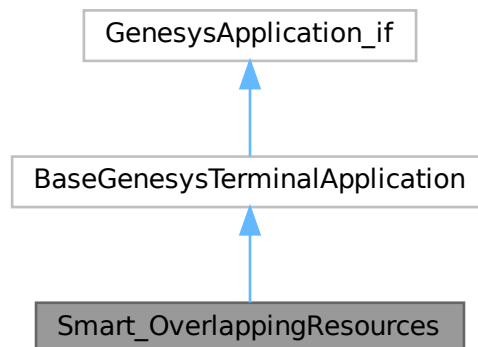
- applications/modelSpecific/smarts/Synchronization/Smart_OnEvent.h
- applications/modelSpecific/smarts/Synchronization/Smart_OnEvent.cpp

10.708 Smart_OverlappingResources Class Reference

Inheritance diagram for Smart_OverlappingResources:



Collaboration diagram for Smart_OverlappingResources:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.708.1 Member Function Documentation

10.708.1.1 main()

```
int Smart_OverlappingResources::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

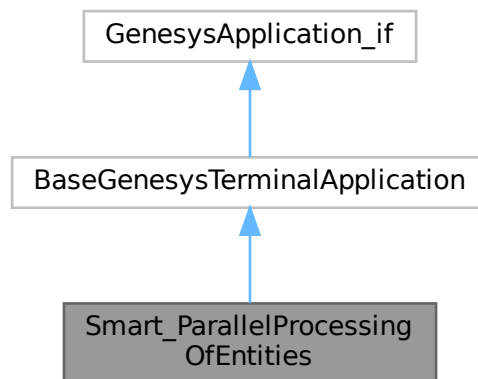
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

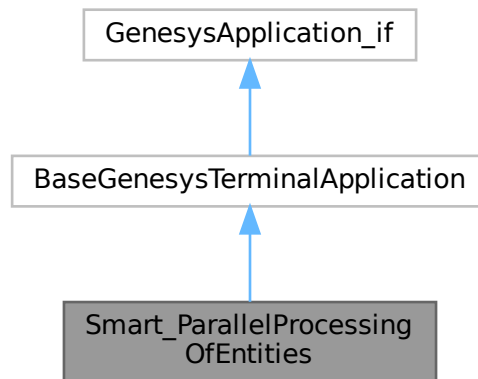
- applications/modelSpecific/arenaSmarts/Smart_OverlappingResources.h
- applications/modelSpecific/arenaSmarts/Smart_OverlappingResources.cpp

10.709 Smart_ParallelProcessingOfEntities Class Reference

Inheritance diagram for Smart_ParallelProcessingOfEntities:



Collaboration diagram for Smart_ParallelProcessingOfEntities:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.709.1 Member Function Documentation

10.709.1.1 main()

```
int Smart_ParallelProcessingOfEntities::main (
    int argc,
    char ** argv )  [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

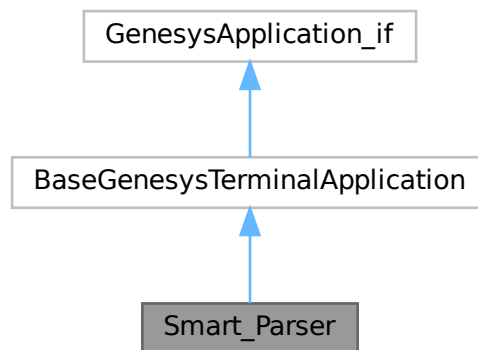
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

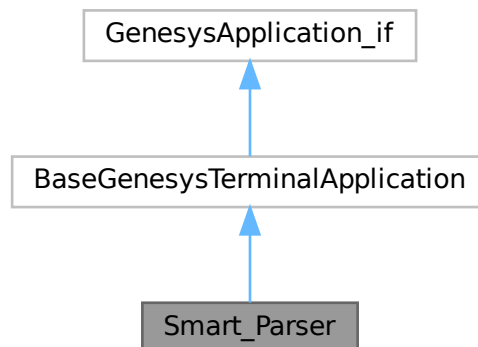
- applications/modelSpecific/arenaSmarts/Smart_ParallelProcessingOfEntities.h
- applications/modelSpecific/arenaSmarts/Smart_ParallelProcessingOfEntities.cpp

10.710 Smart_Parser Class Reference

Inheritance diagram for Smart_Parser:



Collaboration diagram for Smart_Parser:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.710.1 Member Function Documentation

10.710.1.1 main()

```
int Smart_Parser::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

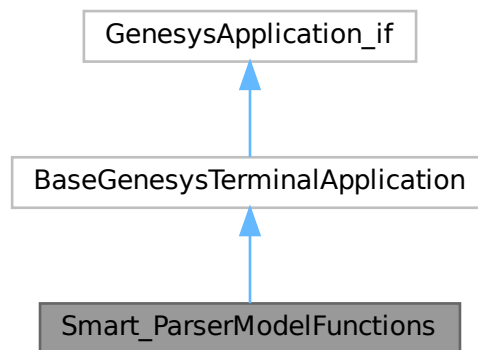
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

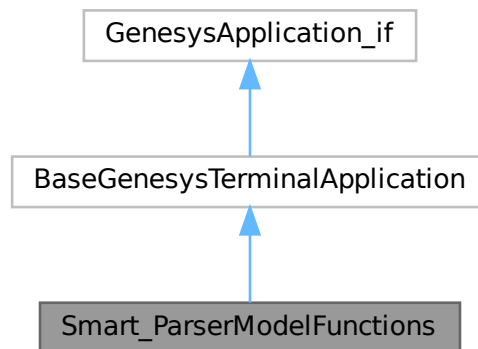
- applications/modelSpecific/smarts/Logic/Smart_Parser.h
- applications/modelSpecific/smarts/Logic/Smart_Parser.cpp

10.711 Smart_ParserModelFunctions Class Reference

Inheritance diagram for Smart_ParserModelFunctions:



Collaboration diagram for Smart_ParserModelFunctions:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.711.1 Member Function Documentation

10.711.1.1 main()

```
int Smart_ParserModelFunctions::main (
    int argc,
    char ** argv )  [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

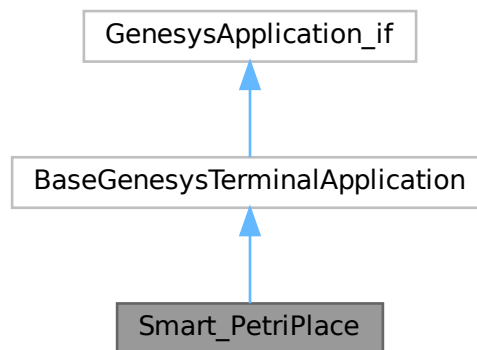
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

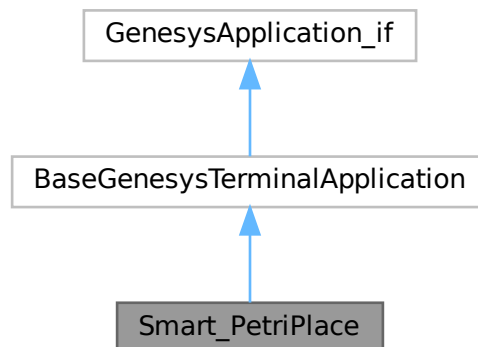
- applications/modelSpecific/smarts/Logic/Smart_ParserModelFunctions.h
- applications/modelSpecific/smarts/Logic/Smart_ParserModelFunctions.cpp

10.712 Smart_PetriPlace Class Reference

Inheritance diagram for Smart_PetriPlace:



Collaboration diagram for Smart_PetriPlace:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.712.1 Member Function Documentation

10.712.1.1 main()

```
int Smart_PetriPlace::main (
    int argc,
    char ** argv ) [override], [virtual]
```

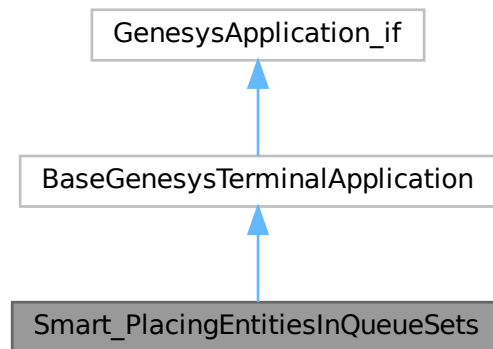
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

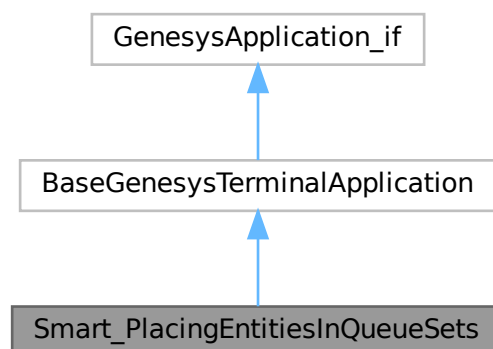
- applications/modelSpecific/smarts/ModalModel/Smart_PetriPlace.h
- applications/modelSpecific/smarts/ModalModel/Smart_PetriPlace.cpp

10.713 Smart_PlacingEntitiesInQueueSets Class Reference

Inheritance diagram for Smart_PlacingEntitiesInQueueSets:



Collaboration diagram for Smart_PlacingEntitiesInQueueSets:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.713.1 Member Function Documentation

10.713.1.1 main()

```
int Smart_PlacingEntitiesInQueueSets::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

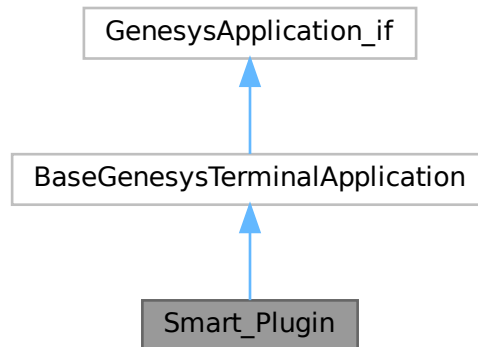
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

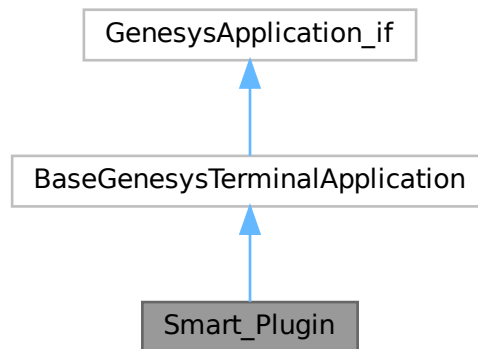
- applications/modelSpecific/arenaSmarts/Smart_PlacingEntitiesInQueueSets.h
- applications/modelSpecific/arenaSmarts/Smart_PlacingEntitiesInQueueSets.cpp

10.714 Smart_Plugin Class Reference

Inheritance diagram for Smart_Plugin:



Collaboration diagram for Smart_Plugin:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.714.1 Member Function Documentation

10.714.1.1 main()

```
int Smart_Plugin::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

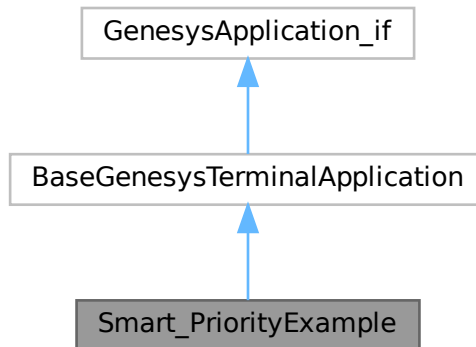
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

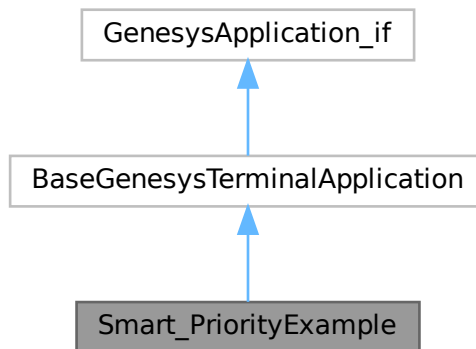
- applications/modelSpecific/smarts/Logic/Smart_Plugin.h
- applications/modelSpecific/smarts/Logic/Smart_Plugin.cpp

10.715 Smart_PriorityExample Class Reference

Inheritance diagram for Smart_PriorityExample:



Collaboration diagram for Smart_PriorityExample:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.715.1 Member Function Documentation

10.715.1.1 main()

```
int Smart_PriorityExample::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

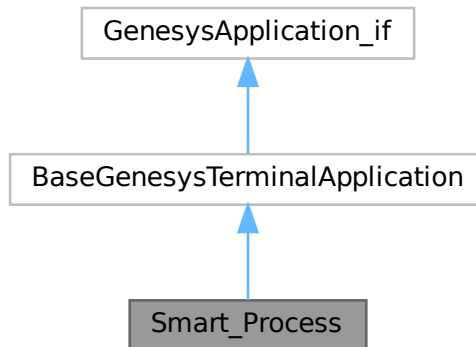
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

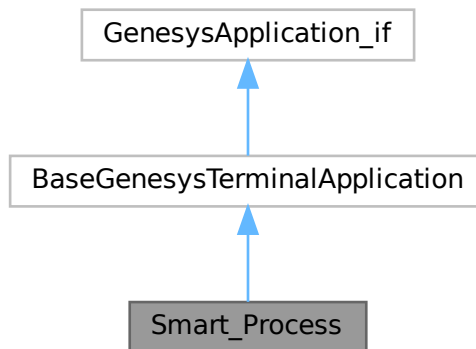
- applications/modelSpecific/arenaSmarts/Smart_PriorityExample.h
- applications/modelSpecific/arenaSmarts/Smart_PriorityExample.cpp

10.716 Smart_Process Class Reference

Inheritance diagram for Smart_Process:



Collaboration diagram for Smart_Process:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.716.1 Member Function Documentation

10.716.1.1 main()

```
int Smart_Process::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

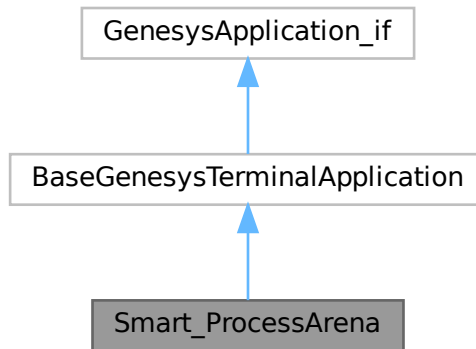
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

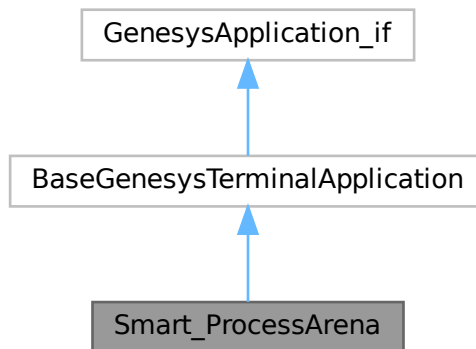
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_Process.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_Process.cpp

10.717 Smart_ProcessArena Class Reference

Inheritance diagram for Smart_ProcessArena:



Collaboration diagram for Smart_ProcessArena:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.717.1 Member Function Documentation

10.717.1.1 main()

```
int Smart_ProcessArena::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

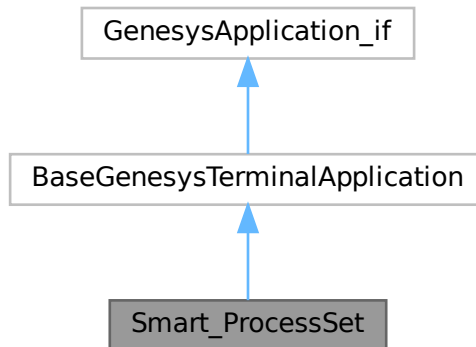
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

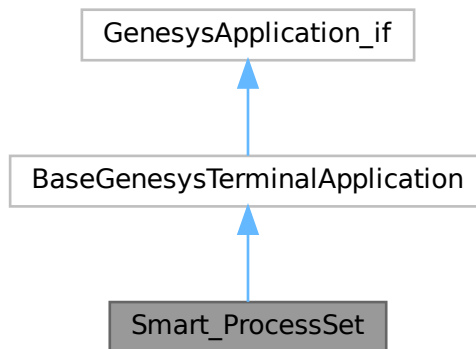
- applications/modelSpecific/arenaSmarts/Smart_ProcessArena.h
- applications/modelSpecific/arenaSmarts/Smart_ProcessArena.cpp

10.718 Smart_ProcessSet Class Reference

Inheritance diagram for Smart_ProcessSet:



Collaboration diagram for Smart_ProcessSet:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.718.1 Member Function Documentation

10.718.1.1 main()

```
int Smart_ProcessSet::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

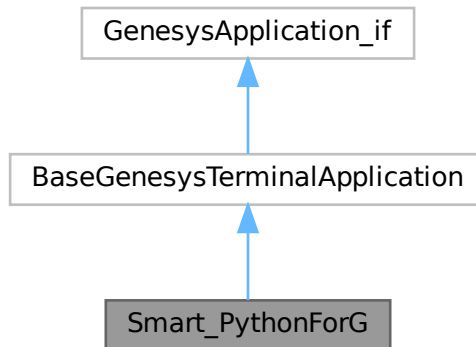
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

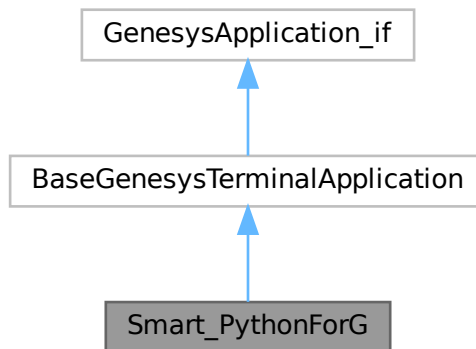
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_ProcessSet.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_ProcessSet.cpp

10.719 Smart_PythonForG Class Reference

Inheritance diagram for Smart_PythonForG:



Collaboration diagram for Smart_PythonForG:



Public Member Functions

- virtual int `main` (int argc, char **argv) override

Public Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void **Simulate** (`Simulator` *genesys, `Model` *model, `PluginManager` *plugins)
- void **setDefaultTraceHandlers** (`TraceManager` *tm)
- void **setDefaultEventHandlers** (`OnEventManager` *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.719.1 Member Function Documentation

10.719.1.1 main()

```
int Smart_PythonForG::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

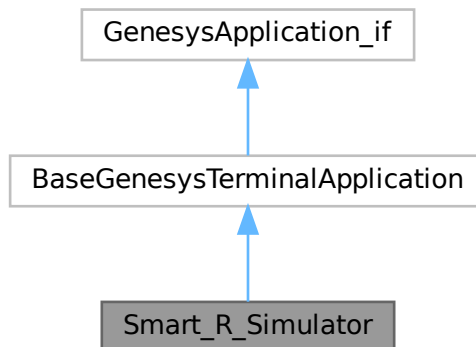
- applications/modelSpecific/smarts/ExternalIntegration/Smart_PythonForG.h
- applications/modelSpecific/smarts/ExternalIntegration/Smart_PythonForG.cpp

10.720 Smart_R_Simulator Class Reference

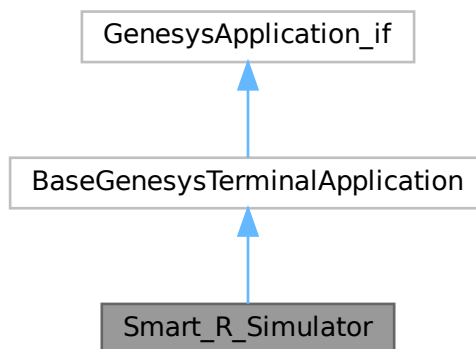
Builds a minimal [Create](#) -> [RSimulator](#) -> [Dispose](#) model.

```
#include <applications/modelSpecific/smarts/ExternalIntegration/Smart_R_↵
Simulator.h>
```

Inheritance diagram for Smart_R_Simulator:



Collaboration diagram for Smart_R_Simulator:



Public Member Functions

- **Smart_R_Simulator** ()
Creates the smart R simulator example.
- virtual int **main** (int argc, char **argv) override
Builds, saves and simulates the example model.

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.720.1 Detailed Description

Builds a minimal [Create](#) -> [RSimulator](#) -> [Dispose](#) model.

The example exercises the component-level R integration and is intended as a compact smoke model for [RSimulator](#) and its internal [RSimulatorRunner](#).

10.720.2 Member Function Documentation

10.720.2.1 main()

```
int Smart_R_Simulator::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Builds, saves and simulates the example model.

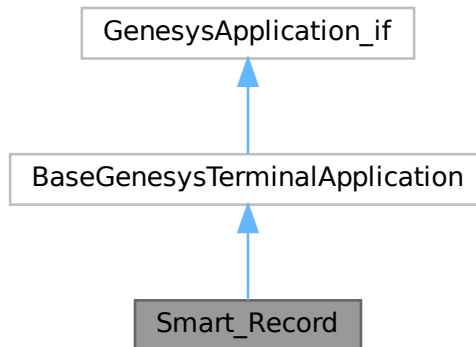
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

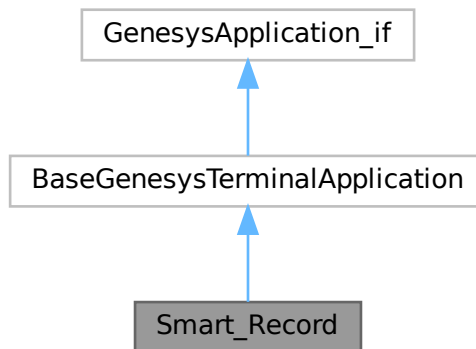
- applications/modelSpecific/smarts/ExternalIntegration/Smart_R_Simulator.h
- applications/modelSpecific/smarts/ExternalIntegration/Smart_R_Simulator.cpp

10.721 Smart_Record Class Reference

Inheritance diagram for Smart_Record:



Collaboration diagram for Smart_Record:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.721.1 Member Function Documentation

10.721.1.1 main()

```
int Smart_Record::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

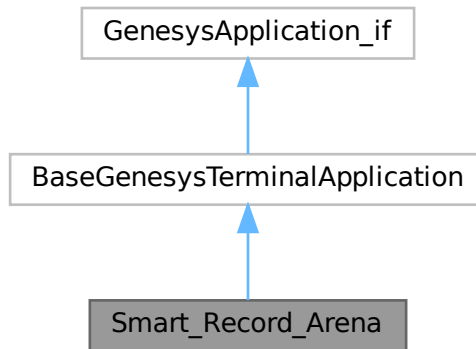
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

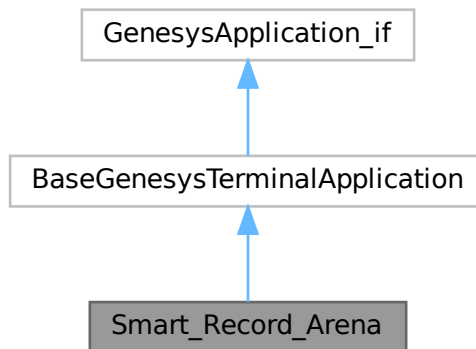
- applications/modelSpecific/smarts/InputOutput/Smart_Record.h
- applications/modelSpecific/smarts/InputOutput/Smart_Record.cpp

10.722 Smart_Record_Arena Class Reference

Inheritance diagram for Smart_Record_Arena:



Collaboration diagram for Smart_Record_Arena:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.722.1 Member Function Documentation

10.722.1.1 main()

```
int Smart_Record_Arena::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

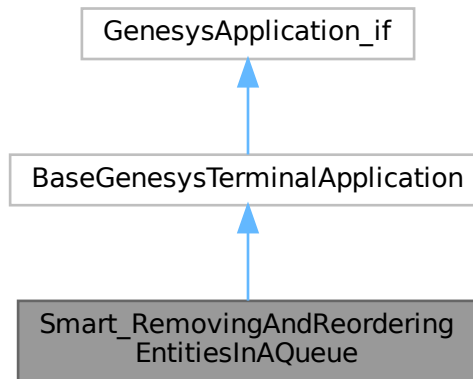
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

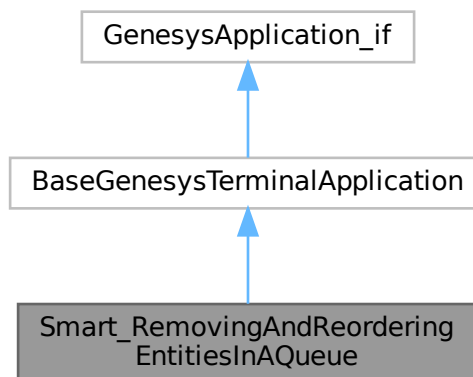
- applications/modelSpecific/arenaSmarts/Smart_Record_Arena.h
- applications/modelSpecific/arenaSmarts/Smart_Record_Arena.cpp

10.723 Smart_RemovingAndReorderingEntitiesInAQueue Class Reference

Inheritance diagram for Smart_RemovingAndReorderingEntitiesInAQueue:



Collaboration diagram for Smart_RemovingAndReorderingEntitiesInAQueue:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.723.1 Member Function Documentation

10.723.1.1 main()

```
int Smart_RemovingAndReorderingEntitiesInAQueue::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

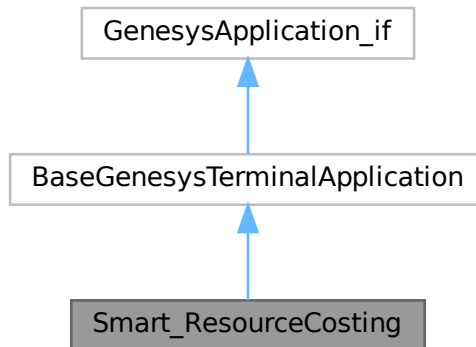
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

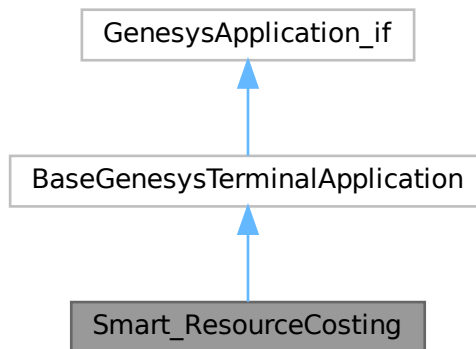
- applications/modelSpecific/arenaSmarts/Smart_RemovingAndReorderingEntitiesInAQueue.h
- applications/modelSpecific/arenaSmarts/Smart_RemovingAndReorderingEntitiesInAQueue.cpp

10.724 Smart_ResourceCosting Class Reference

Inheritance diagram for Smart_ResourceCosting:



Collaboration diagram for Smart_ResourceCosting:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.724.1 Member Function Documentation

10.724.1.1 main()

```
int Smart_ResourceCosting::main (
    int argc,
    char ** argv ) [override], [virtual]
```

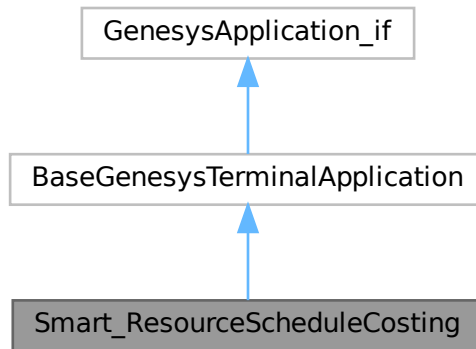
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

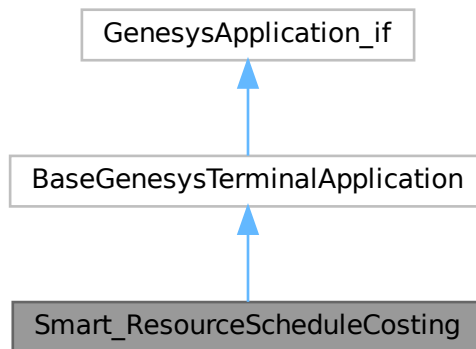
- applications/modelSpecific/arenaSmarts/Smart_ResourceCosting.h
- applications/modelSpecific/arenaSmarts/Smart_ResourceCosting.cpp

10.725 Smart_ResourceScheduleCosting Class Reference

Inheritance diagram for Smart_ResourceScheduleCosting:



Collaboration diagram for Smart_ResourceScheduleCosting:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.725.1 Member Function Documentation

10.725.1.1 main()

```
int Smart_ResourceScheduleCosting::main (
    int argc,
    char ** argv ) [override], [virtual]
```

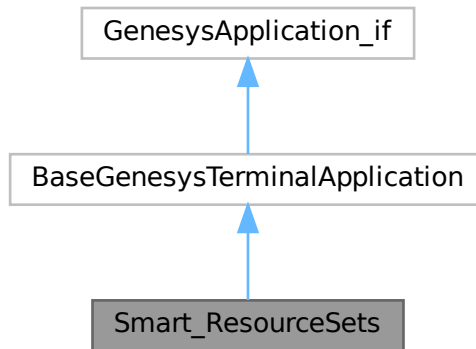
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

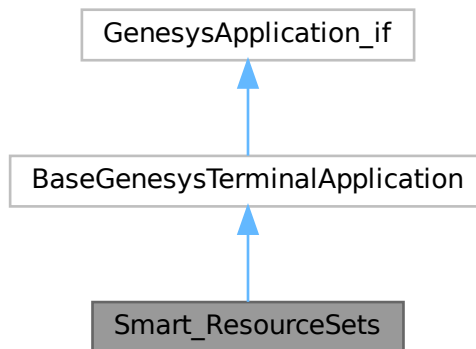
- applications/modelSpecific/arenaSmarts/Smart_ResourceScheduleCosting.h
- applications/modelSpecific/arenaSmarts/Smart_ResourceScheduleCosting.cpp

10.726 Smart_ResourceSets Class Reference

Inheritance diagram for Smart_ResourceSets:



Collaboration diagram for Smart_ResourceSets:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.726.1 Member Function Documentation

10.726.1.1 main()

```
int Smart_ResourceSets::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It create a new simulator instance, builds a simulation model and then simulate that model.

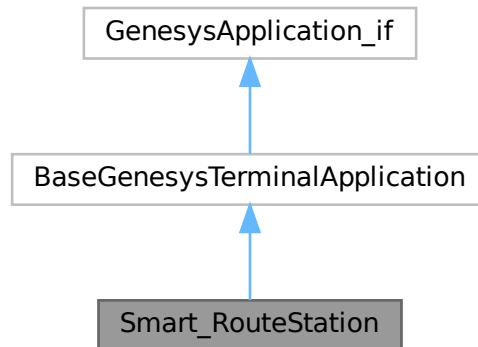
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

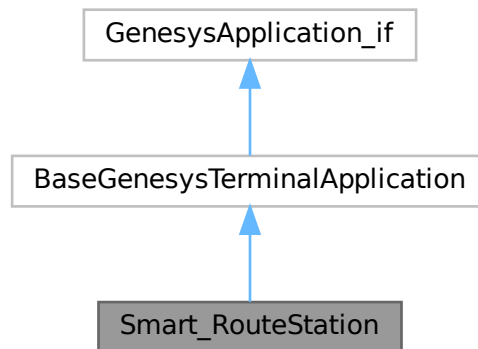
- applications/modelSpecific/arenaSmarts/Smart_ResourceSets.h
- applications/modelSpecific/arenaSmarts/Smart_ResourceSets.cpp

10.727 Smart_RouteStation Class Reference

Inheritance diagram for Smart_RouteStation:



Collaboration diagram for Smart_RouteStation:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.727.1 Member Function Documentation

10.727.1.1 main()

```
int Smart_RouteStation::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

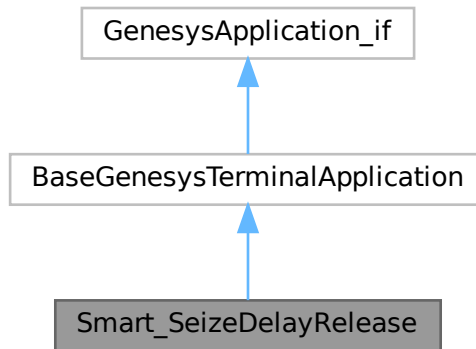
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

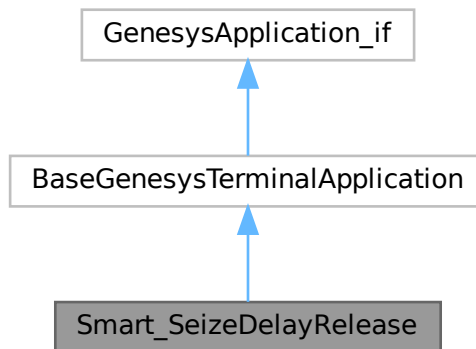
- applications/modelSpecific/smarts/MaterialHandling/Smart_RouteStation.h
- applications/modelSpecific/smarts/MaterialHandling/Smart_RouteStation.cpp

10.728 Smart_SeizeDelayRelease Class Reference

Inheritance diagram for Smart_SeizeDelayRelease:



Collaboration diagram for Smart_SeizeDelayRelease:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.728.1 Member Function Documentation

10.728.1.1 main()

```
int Smart_SeizeDelayRelease::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

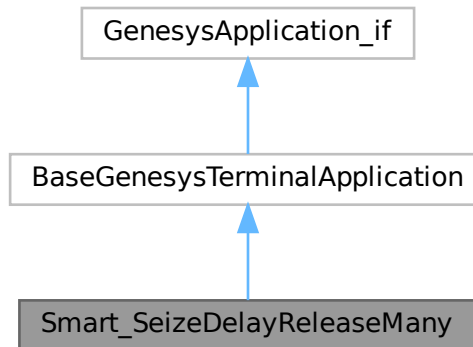
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

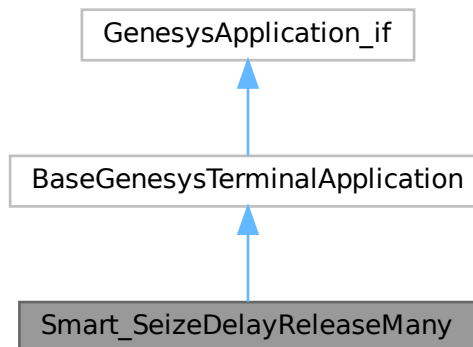
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_SeizeDelayRelease.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_SeizeDelayRelease.cpp

10.729 Smart_SeizeDelayReleaseMany Class Reference

Inheritance diagram for Smart_SeizeDelayReleaseMany:



Collaboration diagram for Smart_SeizeDelayReleaseMany:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.729.1 Member Function Documentation

10.729.1.1 main()

```
int Smart_SeizeDelayReleaseMany::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

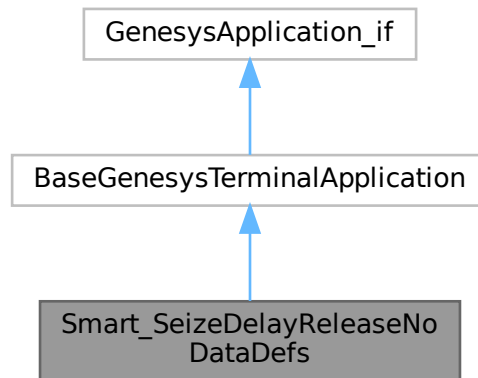
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

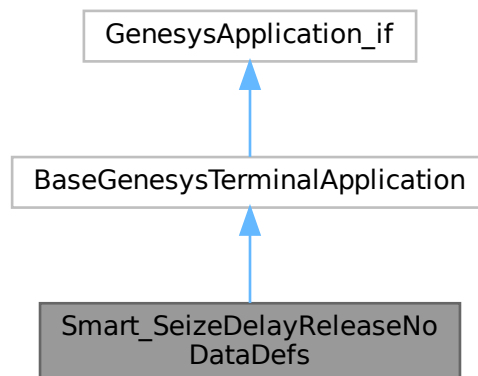
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_SeizeDelayReleaseMany.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_SeizeDelayReleaseMany.cpp

10.730 Smart_SeizeDelayReleaseNoDataDefs Class Reference

Inheritance diagram for Smart_SeizeDelayReleaseNoDataDefs:



Collaboration diagram for Smart_SeizeDelayReleaseNoDataDefs:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.730.1 Member Function Documentation

10.730.1.1 main()

```
int Smart_SeizeDelayReleaseNoDataDefs::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

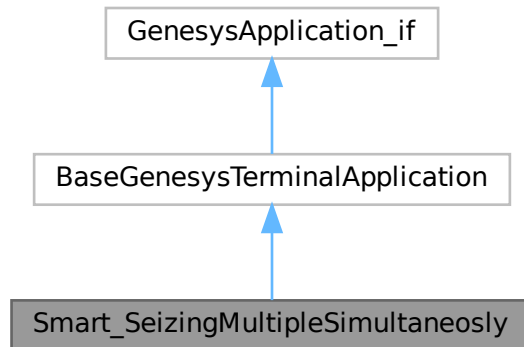
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

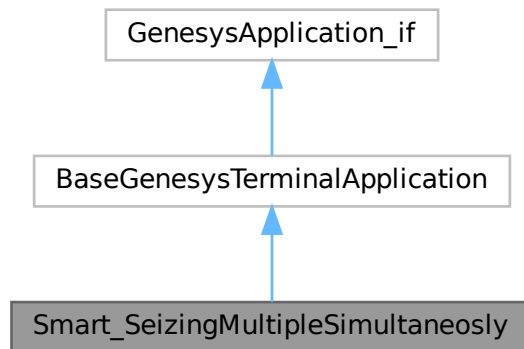
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_SeizeDelayReleaseNoDataDefs.h
- applications/modelSpecific/smarts/DiscreteProcessing/Smart_SeizeDelayReleaseNoDataDefs.cpp

10.731 Smart_SeizingMultipleSimultaneosly Class Reference

Inheritance diagram for Smart_SeizingMultipleSimultaneosly:



Collaboration diagram for Smart_SeizingMultipleSimultaneosly:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.731.1 Member Function Documentation

10.731.1.1 main()

```
int Smart_SeizingMultipleSimultaneosly::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

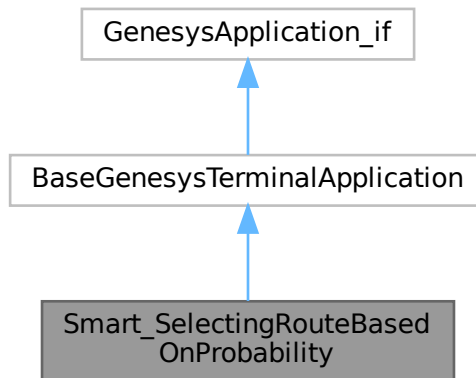
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

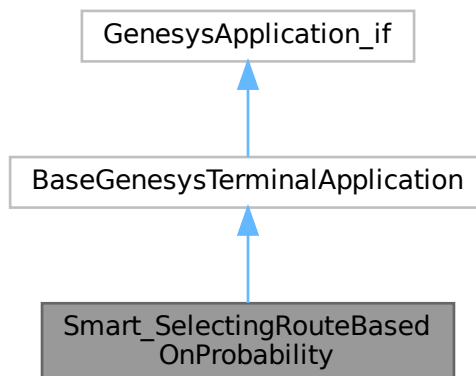
- applications/modelSpecific/arenaSmarts/Smart_SeizingMultipleSimultaneosly.h
- applications/modelSpecific/arenaSmarts/Smart_SeizingMultipleSimultaneosly.cpp

10.732 Smart_SelectingRouteBasedOnProbability Class Reference

Inheritance diagram for Smart_SelectingRouteBasedOnProbability:



Collaboration diagram for Smart_SelectingRouteBasedOnProbability:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.732.1 Member Function Documentation

10.732.1.1 main()

```
int Smart_SelectingRouteBasedOnProbability::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

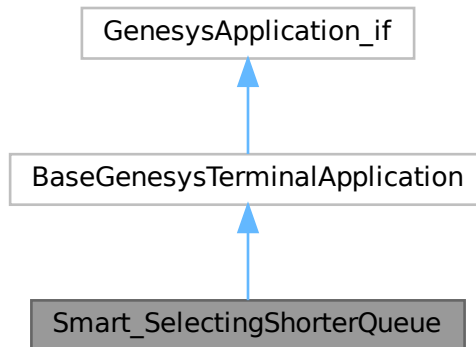
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

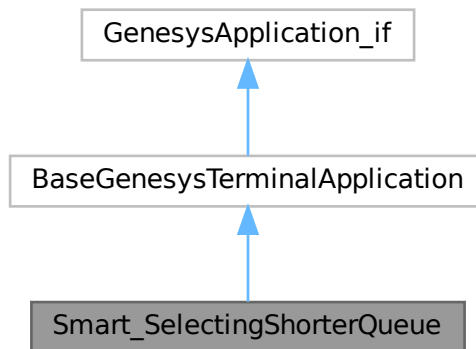
- applications/modelSpecific/arenaSmarts/Smart_SelectingRouteBasedOnProbability.h
- applications/modelSpecific/arenaSmarts/Smart_SelectingRouteBasedOnProbability.cpp

10.733 Smart_SelectingShorterQueue Class Reference

Inheritance diagram for Smart_SelectingShorterQueue:



Collaboration diagram for Smart_SelectingShorterQueue:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.733.1 Member Function Documentation

10.733.1.1 main()

```
int Smart_SelectingShorterQueue::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

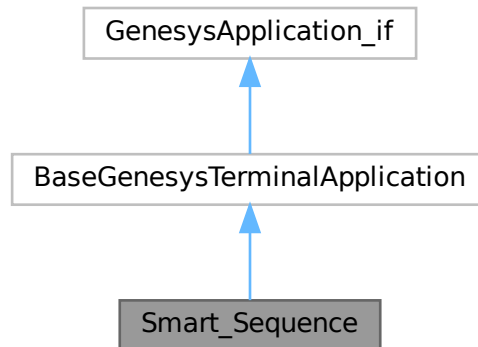
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

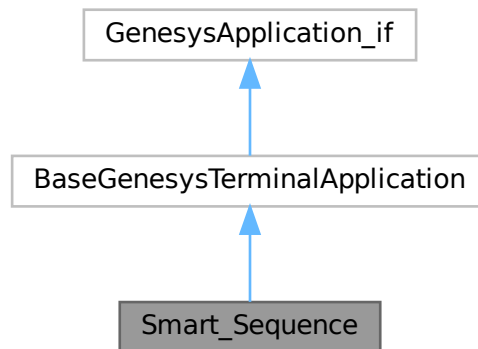
- applications/modelSpecific/arenaSmarts/Smart_SelectingShorterQueue.h
- applications/modelSpecific/arenaSmarts/Smart_SelectingShorterQueue.cpp

10.734 Smart_Sequence Class Reference

Inheritance diagram for Smart_Sequence:



Collaboration diagram for Smart_Sequence:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.734.1 Member Function Documentation

10.734.1.1 main()

```
int Smart_Sequence::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

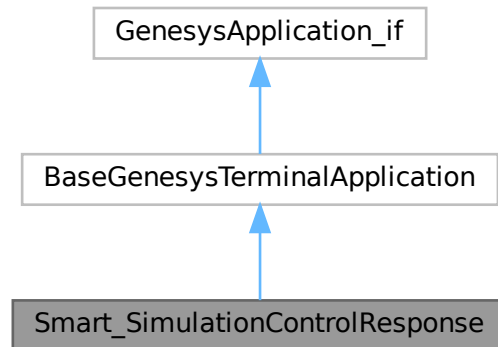
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

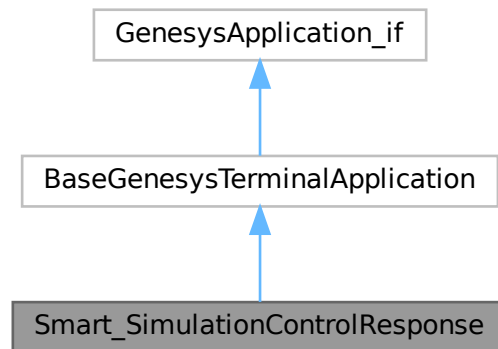
- applications/modelSpecific/smarts/MaterialHandling/Smart_Sequence.h
- applications/modelSpecific/smarts/MaterialHandling/Smart_Sequence.cpp

10.735 Smart_SimulationControlResponse Class Reference

Inheritance diagram for Smart_SimulationControlResponse:



Collaboration diagram for Smart_SimulationControlResponse:



Public Member Functions

- virtual int `main` (int argc, char **argv) override

Public Member Functions inherited from `BaseGenesysTerminalApplication`

- virtual void **Simulate** (`Simulator` *genesys, `Model` *model, `PluginManager` *plugins)
- void **setDefaultTraceHandlers** (`TraceManager` *tm)
- void **setDefaultEventHandlers** (`OnEventManager` *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.735.1 Member Function Documentation

10.735.1.1 main()

```
int Smart_SimulationControlResponse::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

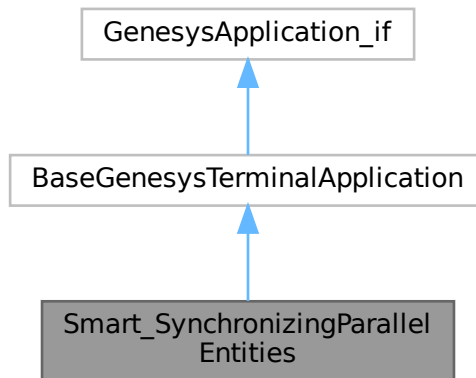
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

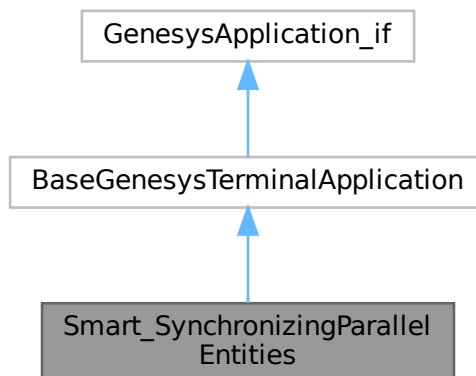
- applications/modelSpecific/smarts/Synchronization/Smart_SimulationControlResponse.h
- applications/modelSpecific/smarts/Synchronization/Smart_SimulationControlResponse.cpp

10.736 Smart_SynchronizingParallelEntities Class Reference

Inheritance diagram for Smart_SynchronizingParallelEntities:



Collaboration diagram for Smart_SynchronizingParallelEntities:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members**Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.736.1 Member Function Documentation**10.736.1.1 main()**

```
int Smart_SynchronizingParallelEntities::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

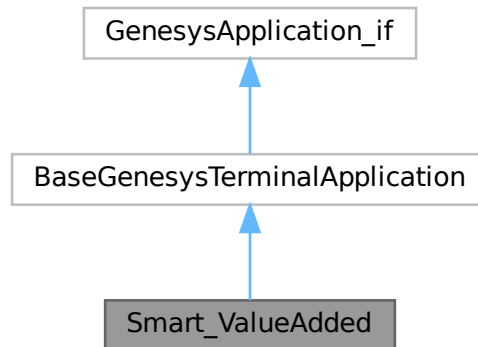
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

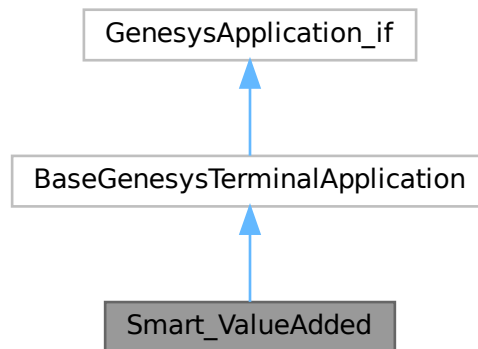
- applications/modelSpecific/arenaSmarts/Smart_SynchronizingParallelEntities.h
- applications/modelSpecific/arenaSmarts/Smart_SynchronizingParallelEntities.cpp

10.737 Smart_ValueAdded Class Reference

Inheritance diagram for Smart_ValueAdded:



Collaboration diagram for Smart_ValueAdded:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.737.1 Member Function Documentation

10.737.1.1 main()

```
int Smart_ValueAdded::main (
    int argc,
    char ** argv ) [override], [virtual]
```

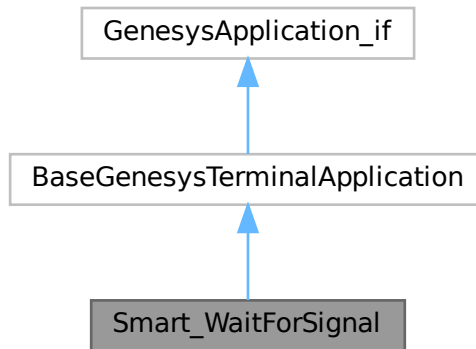
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

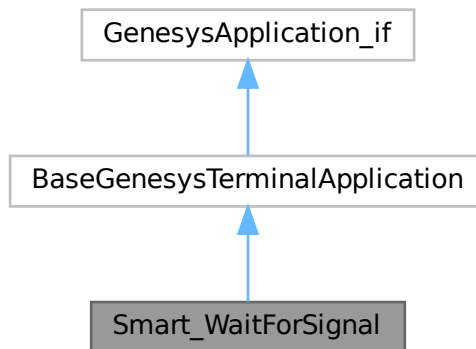
- applications/modelSpecific/arenaSmarts/Smart_ValueAdded.h
- applications/modelSpecific/arenaSmarts/Smart_ValueAdded.cpp

10.738 Smart_WaitForSignal Class Reference

Inheritance diagram for Smart_WaitForSignal:



Collaboration diagram for Smart_WaitForSignal:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.738.1 Member Function Documentation

10.738.1.1 main()

```
int Smart_WaitForSignal::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

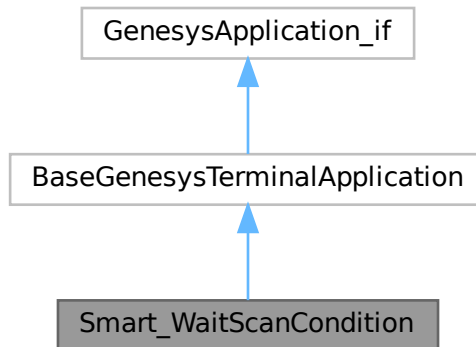
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

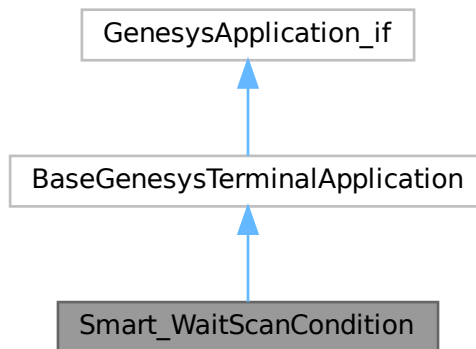
- applications/modelSpecific/arenaSmarts/Smart_WaitForSignal.h
- applications/modelSpecific/arenaSmarts/Smart_WaitForSignal.cpp

10.739 Smart_WaitScanCondition Class Reference

Inheritance diagram for Smart_WaitScanCondition:



Collaboration diagram for Smart_WaitScanCondition:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.739.1 Member Function Documentation

10.739.1.1 main()

```
int Smart_WaitScanCondition::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

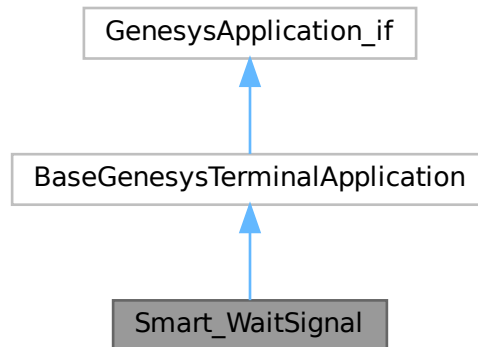
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

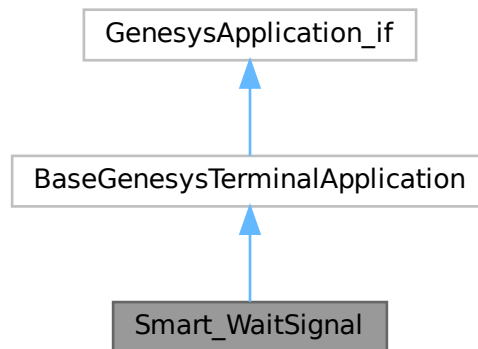
- applications/modelSpecific/smarts/Synchronization/Smart_WaitScanCondition.h
- applications/modelSpecific/smarts/Synchronization/Smart_WaitScanCondition.cpp

10.740 Smart_WaitSignal Class Reference

Inheritance diagram for Smart_WaitSignal:



Collaboration diagram for Smart_WaitSignal:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members**Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)**

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.740.1 Member Function Documentation**10.740.1.1 main()**

```
int Smart_WaitSignal::main (
    int argc,
    char ** argv ) [override], [virtual]
```

This is the main function of the application. It instantiates the simulator, builds a simulation model and then simulate that model.

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/smarts/Synchronization/Smart_WaitSignal.h
- applications/modelSpecific/smarts/Synchronization/Smart_WaitSignal.cpp

10.741 WebWorkerRuntime::Snapshot Struct Reference

Captures a thread-safe snapshot of the worker state.

```
#include <applications/worker/service/WebWorkerRuntime.h>
```

Public Attributes

- bool **isRunning** = false
- bool **isStarting** = false
- bool **isStopping** = false
- bool **hasFailed** = false
- unsigned short **port** = 8080
- unsigned long **maxRequests** = 0
- unsigned long **servedRequests** = 0
- unsigned long **activeRequests** = 0
- std::string **statusMessage**
- std::string **lastError**
- std::vector< std::string > **recentEvents**
- std::vector< std::string > **recentErrors**

10.741.1 Detailed Description

Captures a thread-safe snapshot of the worker state.

The documentation for this struct was generated from the following file:

- applications/worker/service/WebWorkerRuntime.h

10.742 SortFile Class Reference

Utility class that sorts numeric values persisted in a data file.

```
#include <kernel/statistics/SorttFile.h>
```

Public Member Functions

- **SortFile** ()
Creates an unbound sorter (filename must be configured before use).
- bool **sort** ()
Executes in-file sorting for the configured dataset.
- void **setDataFilename** (std::string filename)
Sets the target data filename to be sorted.

10.742.1 Detailed Description

Utility class that sorts numeric values persisted in a data file.

Used by file-based statistics implementations to support quantile/median computations that require ordered samples.

10.742.2 Member Function Documentation**10.742.2.1 setDataFilename()**

```
void SortFile::setDataFilename (
    std::string filename )
```

Sets the target data filename to be sorted.

Parameters

<i>filename</i>	Path to the backing data file.
-----------------	--------------------------------

10.742.2.2 sort()

```
bool SortFile::sort ( )
```

Executes in-file sorting for the configured dataset.

Returns

True if sorting operation completes successfully.

The documentation for this class was generated from the following files:

- kernel/statistics/SorttFile.h
- kernel/statistics/SorttFile.cpp

10.743 SourceCodeString Class Reference**Public Member Functions**

- **SourceCodeString** (const std::string &value)
- **SourceCodeString** (std::string &&value)
- const std::string & **str** () const
- **operator std::string** () const

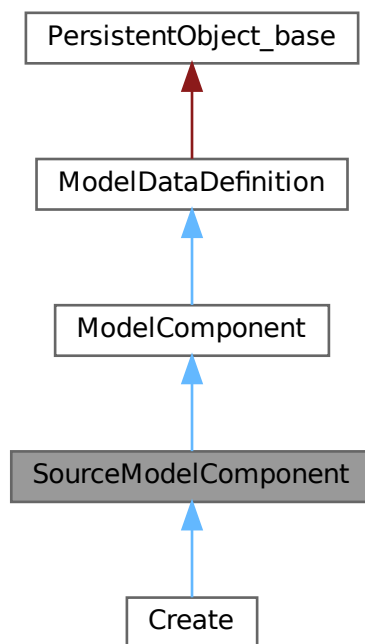
The documentation for this class was generated from the following file:

- kernel/simulator/SimulationControlAndResponse.h

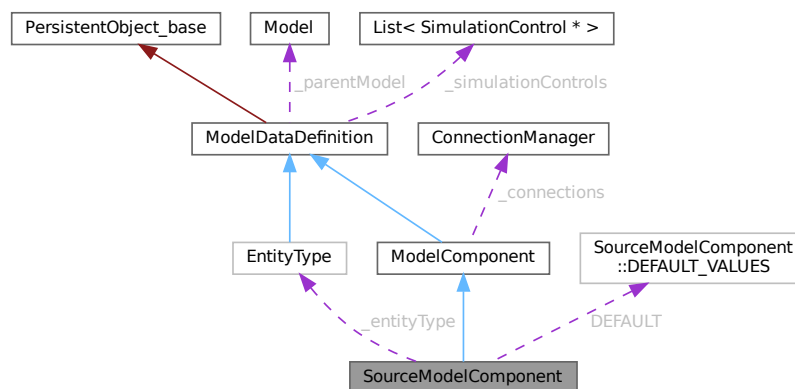
10.744 SourceModelComponent Class Reference

```
#include <kernel/simulator/SourceModelComponent.h>
```

Inheritance diagram for SourceModelComponent:



Collaboration diagram for SourceModelComponent:



Classes

- struct `DEFAULT_VALUES`

Public Member Functions

- **SourceModelComponent** ([Model](#) *model, std::string componentTypename, std::string name="")
setFirstCreation
- void [setFirstCreation](#) (double _firstCreation)
getFirstCreation
- double [getFirstCreation](#) () const
- void [setEntityType](#) ([EntityType](#) *_entityType)
setEntityType
- void [setEntityType](#) (std::string entityType) *setEntityType*
- [EntityType](#) * [getEntityType](#) () const
getEntityType
- void [setTimeUnit](#) ([Util::TimeUnit](#) _timeUnit)
setTimeUnit
- [Util::TimeUnit](#) [getTimeUnit](#) () const
getTimeUnit
- void [setTimeBetweenCreationsExpression](#) (std::string _timeBetweenCreations, [Util::TimeUnit](#) _timeUnit=[Util::TimeUnit::unknown](#))
setTimeBetweenCreationsExpression
- std::string [getTimeBetweenCreationsExpression](#) () const
getTimeBetweenCreationsExpression
- void [setMaxCreations](#) (unsigned long _maxCreations)
setMaxCreations
- void [setMaxCreations](#) (std::string _maxCreationsExpression)
setMaxCreations
- std::string [getMaxCreations](#) () const
getMaxCreations
- unsigned int [getEntitiesCreated](#) () const
getEntitiesCreated
- void [setEntitiesCreated](#) (unsigned int _entitiesCreated)
setEntitiesCreated
- void [setEntitiesPerCreation](#) (unsigned int _entitiesPerCreation)
setEntitiesPerCreation
- unsigned int [getEntitiesPerCreation](#) () const
getEntitiesPerCreation
- virtual std::string [show](#) () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()

- Indicates whether a simulation breakpoint is set on this component.*
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
 IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager](#)::Level [getTraceLevelSpecific](#) () const

- Returns the trace level that applies specifically to this definition.*
- void [setTraceLevelSpecific](#) (TraceManager::Level level)
- Updates the trace level used by this definition.*
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelspecific, bool traceLevelSpecificEnabled=true)
- Defines the specific trace level and optionally enables it immediately.*
- bool [isTraceLevelSpecificEnabled](#) () const
- Indicates whether the specific trace level is enabled.*
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
- Enables or disables the specific trace level.*

Protected Member Functions

- virtual bool [_loadInstance](#) (PersistenceRecord *fields) override
- virtual protected method that must be overridden*
- virtual void [_saveInstance](#) (PersistenceRecord *fields, bool saveDefaultValues) override
- Saves object fields into a serialization structure.*
- virtual void [_initBetweenReplications](#) () override
- virtual bool [_check](#) (std::string &errorMessage) override
- could be overridden by derived classes*
- virtual void [_createEditableDataDefinitions](#) ()
- virtual void [_createAttachedAttributes](#) ()

Protected Member Functions inherited from [ModelComponent](#)

- virtual void [_onDispatchEvent](#) (Entity *entity, unsigned int inputPortNumber)=0
- Handles a dispatched event for this component type.*

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) (SimulationControl *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Protected Attributes

- [EntityType](#) * **_entityType** = nullptr
- const struct [SourceModelComponent::DEFAULT_VALUES](#) **DEFAULT**
- double **_firstCreation** = DEFAULT.firstCreation
- unsigned int **_entitiesPerCreation** = DEFAULT.entitiesPerCreation
- std::string **_maxCreationsExpression** = DEFAULT.maxCreationsExpression
- std::string **_timeBetweenCreationsExpression** = DEFAULT.timeBetweenCreationsExpression
- [Util::TimeUnit](#) **_timeBetweenCreationsTimeUnit** = DEFAULT.timeBetweenCreationsTimeUnit
- unsigned int **_entitiesCreatedSoFar** = 0

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
 - virtual protected methods that could be overridden by derived classes, if needed*

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

Additional Inherited Members

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

10.744.1 Detailed Description

A source component implements the base for inserting entities into the model when its simulation is initialized. During the initialization, the new and empty future events list is populated by events of creating entities and sending them to the source components existing in the model

10.744.2 Member Function Documentation

10.744.2.1 `_check()`

```
bool SourceModelComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

Reimplemented in [Create](#).

10.744.2.2 `_createAttachedAttributes()`

```
void SourceModelComponent::_createAttachedAttributes ( ) [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.744.2.3 `_createEditableDataDefinitions()`

```
void SourceModelComponent::_createEditableDataDefinitions ( ) [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.744.2.4 `_initBetweenReplications()`

```
void SourceModelComponent::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.744.2.5 `_loadInstance()`

```
bool SourceModelComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

Reimplemented in [Create](#).

10.744.2.6 `_saveInstance()`

```
void SourceModelComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

Reimplemented in [Create](#).

10.744.2.7 getEntitiesCreated()

```
unsigned int SourceModelComponent::getEntitiesCreated ( ) const
```

getEntitiesCreated

Returns

10.744.2.8 getEntitiesPerCreation()

```
unsigned int SourceModelComponent::getEntitiesPerCreation ( ) const
```

getEntitiesPerCreation

Returns

10.744.2.9 getEntityType()

```
EntityType * SourceModelComponent::getEntityType ( ) const
```

getEntityType

Returns

10.744.2.10 getFirstCreation()

```
double SourceModelComponent::getFirstCreation ( ) const
```

getFirstCreation

Returns

10.744.2.11 getMaxCreations()

```
std::string SourceModelComponent::getMaxCreations ( ) const
```

getMaxCreations

Returns

10.744.2.12 getTimeBetweenCreationsExpression()

```
std::string SourceModelComponent::getTimeBetweenCreationsExpression ( ) const
```

getTimeBetweenCreationsExpression

Returns

10.744.2.13 getTimeUnit()

```
Util::TimeUnit SourceModelComponent::getTimeUnit ( ) const
```

getTimeUnit

Returns

10.744.2.14 setEntitiesCreated()

```
void SourceModelComponent::setEntitiesCreated (
    unsigned int _entitiesCreated )
```

setEntitiesCreated

Parameters

<i>_entitiesCreated</i>	
-------------------------	--

10.744.2.15 setEntitiesPerCreation()

```
void SourceModelComponent::setEntitiesPerCreation (
    unsigned int _entitiesPerCreation )
```

setEntitiesPerCreation

Parameters

<i>_entitiesPerCreation</i>	
-----------------------------	--

10.744.2.16 setEntityType()

```
void SourceModelComponent::setEntityType (
    EntityType * _entityType )
```

setEntityType

Parameters

<i>_entityType</i>	
--------------------	--

10.744.2.17 setEntityTypeName()

```
void SourceModelComponent::setEntityTypeName (
    std::string entityTypeName )
```

setEntityTypeName

Parameters

<i>entityTypeName</i>	
-----------------------	--

10.744.2.18 setFirstCreation()

```
void SourceModelComponent::setFirstCreation (
    double _firstCreation )
```

setFirstCreation

Parameters

<i>_firstCreation</i>	
-----------------------	--

10.744.2.19 setMaxCreations() [1/2]

```
void SourceModelComponent::setMaxCreations (
    std::string _maxCreationsExpression )
```

setMaxCreations

Parameters

<i>_maxCreationsExpression</i>	
--------------------------------	--

10.744.2.20 setMaxCreations() [2/2]

```
void SourceModelComponent::setMaxCreations (
    unsigned long _maxCreations )
```

setMaxCreations

Parameters

<i>_maxCreations</i>	
----------------------	--

10.744.2.21 setTimeBetweenCreationsExpression()

```
void SourceModelComponent::setTimeBetweenCreationsExpression (
    std::string _timeBetweenCreations,
    Util::TimeUnit _timeUnit = Util::TimeUnit::unknown )
```

setTimeBetweenCreationsExpression

Parameters

<i>_timeBetweenCreations</i>	
<i>_timeUnit</i>	

10.744.2.22 setTimeUnit()

```
void SourceModelComponent::setTimeUnit (
    Util::TimeUnit _timeUnit )
```

setTimeUnit

Parameters

<i>_timeUnit</i>	
------------------	--

10.744.2.23 show()

```
std::string SourceModelComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

Reimplemented in [Create](#).

The documentation for this class was generated from the following files:

- kernel/simulator/SourceModelComponent.h
- kernel/simulator/SourceModelComponent.cpp

10.745 SparseValueStore Class Reference

Sparse N-dimensional numeric store using GenESyS textual index keys.

```
#include <kernel/simulator/essentialPlugins/SparseValueStore.h>
```

Public Member Functions

- double **value** (const std::string &index="") const
Reads a sparse position, returning 0.0 when the position is absent.
- void **setValue** (double [value](#), const std::string &index="")
Writes a sparse position, creating it when needed.
- void **setValues** (const std::map< std::string, double > &[values](#))
Replaces all sparse values with a copy of the provided values.
- void **copyFrom** (const [SparseValueStore](#) &other)
Replaces this store with a deep copy of another store.
- void **insertDimensionSize** (unsigned int size)
Inserts one dimension size at the end of the dimension list.
- std::list< unsigned int > * **dimensionSizes** ()
Returns mutable dimension sizes for compatibility with legacy callers.
- const std::list< unsigned int > * **dimensionSizes** () const
Returns immutable dimension sizes.
- std::map< std::string, double > * **values** ()
Returns mutable sparse values for compatibility with legacy callers.
- const std::map< std::string, double > * **values** () const
Returns immutable sparse values.
- void **clear** ()
Clears dimensions and values.
- void **loadDimensions** ([PersistenceRecord](#) *fields)
Loads dimensions using the legacy fields dimensions/dimension[i].
- void **saveDimensions** ([PersistenceRecord](#) *fields, bool saveDefaultValues) const
Saves dimensions using the legacy fields dimensions/dimension[i].
- void **loadValues** ([PersistenceRecord](#) *fields, const std::string &countField, const std::string &positionPrefix, const std::string &valuePrefix)
Loads sparse values using the provided count/position/value field prefixes.
- void **saveValues** ([PersistenceRecord](#) *fields, const std::string &countField, const std::string &positionPrefix, const std::string &valuePrefix, bool saveDefaultValues) const
Saves sparse values using the provided count/position/value field prefixes.
- std::string **showValues** () const
Returns a compact textual representation used by show/debug paths.

Static Public Member Functions

- static std::string **makeIndexKey** (const std::vector< unsigned int > &indexes)
Builds the textual key used by legacy sparse matrices from index values.
- static std::string **makeIndexKeyFromDoubles** (const std::vector< double > &indexes)
Builds the textual key used by legacy sparse matrices from expression values.
- static std::string **appendIndexKey** (const std::string ¤tKey, unsigned int index)
Appends one index to an existing textual sparse key.
- static std::string **appendIndexKeyFromDouble** (const std::string ¤tKey, double index)
Appends one expression value to an existing textual sparse key.
- static std::string **normalizeIndexKey** (const std::string &index)
Returns a normalized copy of an already textual key.

10.745.1 Detailed Description

Sparse N-dimensional numeric store using GenESyS textual index keys.

The store preserves the existing [Variable](#) semantics: the scalar position is represented by an empty key, multidimensional indexes are represented as comma-separated unsigned integers such as "1,2,3", and any missing position reads back as 0.0. It is intentionally small so [Variable](#), [Attribute](#) and [Entity](#) can share the same behavior without changing the persistence format.

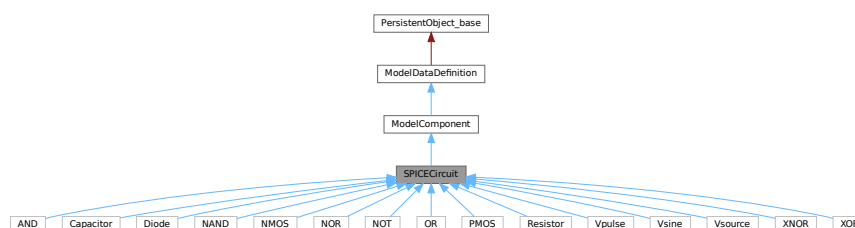
The documentation for this class was generated from the following file:

- kernel/simulator/essentialPlugins/SparseValueStore.h

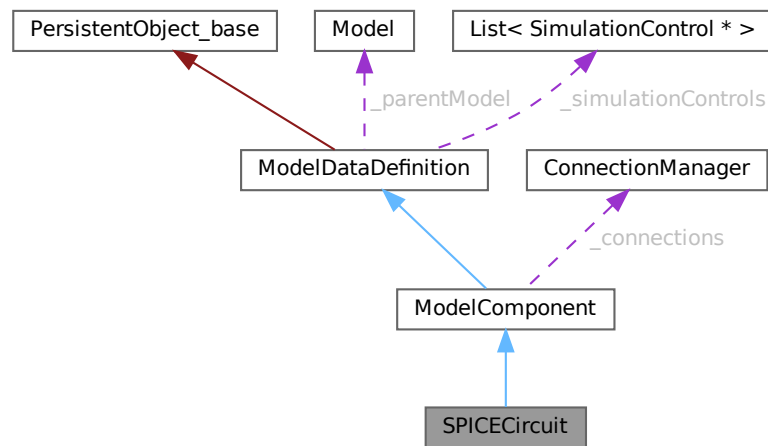
10.746 SPICECircuit Class Reference

```
#include <plugins/components/ElectronicsSimulation/SPICECircuit.h>
```

Inheritance diagram for SPICECircuit:



Collaboration diagram for SPICECircuit:



Public Member Functions

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
- static std::vector< std::string > **split** (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void **Build** ()
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void **_createNonEditableDataDefinitions** () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
Traces text to the simulation output.
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Protected Attributes

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.746.1 Detailed Description

This component ...

10.746.2 Member Function Documentation

10.746.2.1 [_createNonEditableDataDefinitions\(\)](#)

```
void SPICECircuit::_createNonEditableDataDefinitions ( ) [override], [protected], [virtual]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by [ModelChecker](#) during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model too run without errors in this component

This method returns all changes in the parser that are needed by plugins of this [ModelData](#)s. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

This method is called by [ModelSimulation](#) when initianting the replication. The model should set all value for a new replication (Ex: setting back to 0 any internal counter, clearing lists, etc.

Reimplemented from [ModelDataDefinition](#).

10.746.2.2 `_loadInstance()`

```
bool SPICECircuit::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

Reimplemented in [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), and [XNOR](#).

10.746.2.3 `_onDispatchEvent()`

```
void SPICECircuit::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

This method is only for ModelComponents, not ModelDataElements.

Implements [ModelComponent](#).

10.746.2.4 `_saveInstance()`

```
void SPICECircuit::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

Reimplemented in [Resistor](#), [Vsource](#), [Vpulse](#), [Vsine](#), [Capacitor](#), [Diode](#), [PMOS](#), [NMOS](#), [NOT](#), [NAND](#), [AND](#), [NOR](#), [OR](#), [XOR](#), and [XNOR](#).

10.746.2.5 `show()`

```
std::string SPICECircuit::show ( ) [override], [virtual]
```

virtual public methods

Reimplemented from [ModelComponent](#).

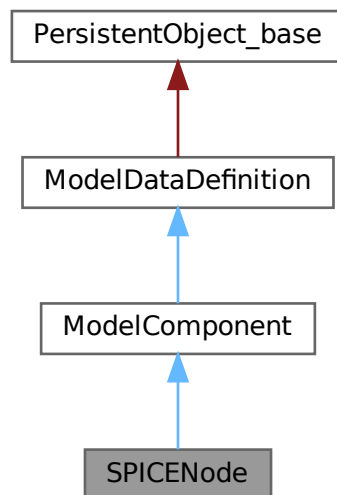
The documentation for this class was generated from the following files:

- `plugins/components/ElectronicsSimulation/SPICECircuit.h`
- `plugins/components/ElectronicsSimulation/SPICECircuit.cpp`

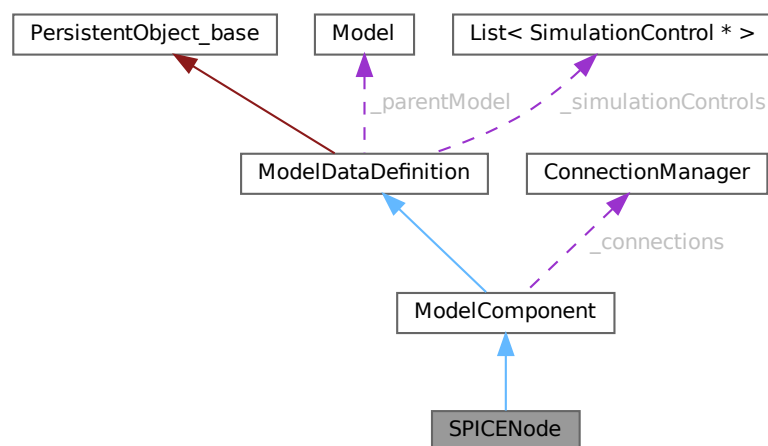
10.747 SPICENode Class Reference

```
#include <plugins/components/ElectronicsSimulation/SPICENode.h>
```

Inheritance diagram for SPICENode:



Collaboration diagram for SPICENode:



Public Member Functions

- **SPICENode** ([Model](#) *model, std::string name="")

constructors

- `std::string getNodeName ()`

new public user methods for this component

- virtual `std::string show ()` override

virtual public methods

- **operator** `std::string ()` const

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")

constructors

- virtual `~ModelComponent ()`

Releases component-owned resources and unregisters it from the model.

- [ConnectionManager](#) * [getConnectionManager \(\)](#) const

new public user methods for this component

- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)

Connects this component to another component.

- bool [hasBreakpointAt \(\)](#)

Indicates whether a simulation breakpoint is set on this component.

- void [setDescription](#) (std::string _description)

Updates the free-form description shown by the GUI.

- std::string [getDescription \(\)](#) const

Returns the free-form description associated with this component.

- void [remove](#) ([ModelComponent](#) *comp)

Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)

Creates a model data definition and optionally inserts it into the model.

- virtual `~ModelDataDefinition ()`

Releases resources owned by this data definition.

- Util::identification [getId \(\)](#) const

Returns the unique identifier of this instance.

- void [setName](#) (const std::string &name)

Updates the logical name of this instance.

- const std::string & [getName \(\)](#) const

Returns the logical name of this instance.

- void [setLabel](#) (const std::string &label)

Updates the user-visible label of this instance.

- const std::string & [getLabel \(\)](#) const

Returns the user-visible label of this instance.

- const std::string & [getClassname \(\)](#) const

Returns the runtime class name used by the plugin system.

- [Model](#) * [getParentModel \(\)](#) const

Returns the parent model that owns this definition.

- bool [isReportStatistics \(\)](#) const

Indicates whether this data definition contributes report statistics.

- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific↔ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)

- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.747.1 Detailed Description

This component ...

10.747.2 Member Function Documentation

10.747.2.1 `_loadInstance()`

```
bool SPICENode::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.747.2.2 `_onDispatchEvent()`

```
void SPICENode::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

This method is only for ModelComponents, not ModelDataElements.

Implements [ModelComponent](#).

10.747.2.3 `_saveInstance()`

```
void SPICENode::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.747.2.4 `show()`

```
std::string SPICENode::show ( ) [override], [virtual]
```

virtual public methods

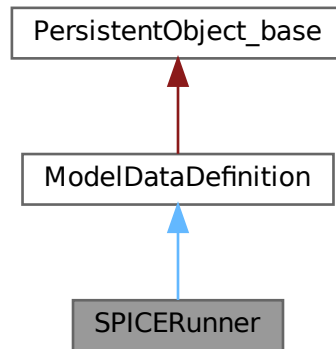
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

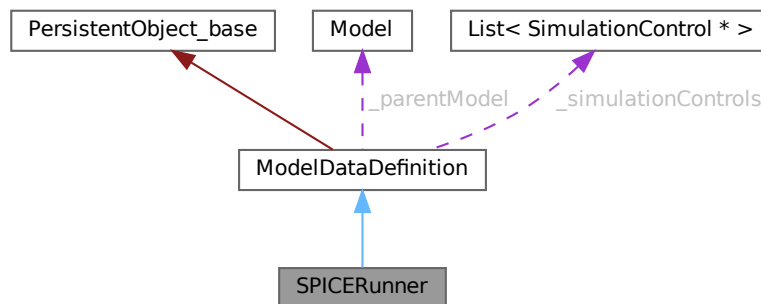
- `plugins/components/ElectronicsSimulation/SPICENode.h`
- `plugins/components/ElectronicsSimulation/SPICENode.cpp`

10.748 SPICERunner Class Reference

Inheritance diagram for SPICERunner:



Collaboration diagram for SPICERunner:



Public Member Functions

- **SPICERunner** ([Model](#) *model, std::string name="")
constructors
- std::string **CompileSpiceFile** ()
new public user methods for this component
- void **SendComponent** (std::string *instance, std::string subcircuit="", std::string model="")
- void **SendCallback** (std::function< void()> callback)
- void **ConfigSim** (double duration, double step)
- void **Run** ()
- void **PlotV** (std::string net)
- template<typename... Args>
 void **PlotV** (std::string net, Args... args)

- void **PlotVRelative** (std::string comparison_base, std::string net)
 - template<typename... Args>
void **PlotVRelative** (std::string comparison_base, std::string net, Args... args)
 - void **PlotI** (std::string net)
 - template<typename... Args>
void **PlotI** (std::string net, Args... args)
 - void **PlotIRelative** (std::string comparison_base, std::string net)
 - template<typename... Args>
void **PlotIRelative** (std::string comparison_base, std::string net, Args... args)
 - double * **MeasurePeak** (std::string label, std::string peak, std::string quantity, std::string node, float start, float finish)
 - double * **MeasureTrigTarg** (std::string label, std::string trig, float trig_value, std::string trig_inclin, std::string targ, float targ_value, std::string targ_inclin)
 - void **ParseOutput** ()
 - void **setRunnerCommand** (std::string runnerCommand)
 - std::string **getRunnerCommand** () const
 - void **setModelsPath** (std::string modelsPath)
 - std::string **getModelsPath** () const
 - void **setWorkingInputFilename** (std::string workingInputFilename)
 - std::string **getWorkingInputFilename** () const
 - void **setWorkingOutputFilename** (std::string workingOutputFilename)
 - std::string **getWorkingOutputFilename** () const
 - void **setWorkingDirectory** (std::string workingDirectory)
 - std::string **getWorkingDirectory** () const
 - std::string **getLastRunCommand** () const
 - virtual std::string **show** () override
- virtual public methods*

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

- Enables or disables report-statistics generation for this definition.*

 - `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
 - `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
 - `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
 - `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
 - `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
 - `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
 - `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
 - `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
 - `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
 - `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
 - `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
 - `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
 - `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static PluginInformation * GetPluginInformation ()`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
- `static std::vector< std::string > split (std::string word, char split_char)`

Static Public Member Functions inherited from ModelDataDefinition

- `static ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- `static void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- `static bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- `static void InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- `static void CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- `static bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_initBetweenReplications` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.748.1 Member Function Documentation

10.748.1.1 [_check\(\)](#)

```
bool SPICERunner::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

virtual protected methods that could be overridden by derived classes, if needed

This method is called by ModelChecker during model check. The component should check itself to verify if user parameters are ok (ex: correct syntax for the parser) and everything in its parameters allow the model to run without errors in this component

Reimplemented from [ModelDataDefinition](#).

10.748.1.2 [_loadInstance\(\)](#)

```
bool SPICERunner::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelDataDefinition](#).

10.748.1.3 [_saveInstance\(\)](#)

```
void SPICERunner::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.748.1.4 show()

```
std::string SPICERunner::show ( ) [override], [virtual]
```

virtual public methods

Reimplemented from [ModelDataDefinition](#).

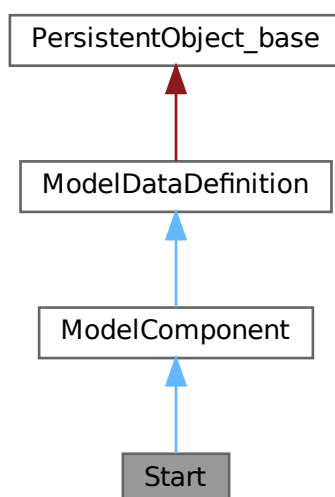
The documentation for this class was generated from the following files:

- plugins/data/ExternalIntegration/SPICERunner.h
- plugins/data/ExternalIntegration/SPICERunner.cpp

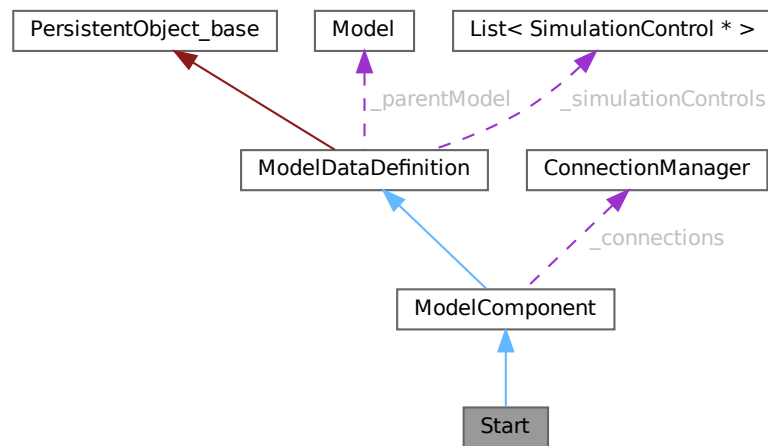
10.749 Start Class Reference

```
#include <plugins/components/MaterialHandling/Start.h>
```

Inheritance diagram for Start:



Collaboration diagram for Start:



Public Member Functions

- **Start** ([Model](#) *model, std::string name="")
 - virtual std::string [show](#) () override
- virtual public methods*

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.749.1 Detailed Description

[Start](#) module DESCRIPTION The [Start](#) module changes the status of a conveyor from inactive to active. The conveyor may have been deactivated from either the [Stop](#) module or by initially being set to inactive at the start of the simulation. The velocity of the conveyor may be changed permanently when the conveyor is started. TYPICAL USES [Start](#) a bottling conveyor after scheduled maintenance [Start](#) a baggage claim conveyor when bags have arrived PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart. Conveyor Name Name of the conveyor to start. Velocity Speed of the conveyor once it begins to operate. This value will change the speed of the conveyor permanently, until it is changed in another module. Units Velocity time units.

10.749.2 Member Function Documentation

10.749.2.1 `_check()`

```
bool Start::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.749.2.2 `_loadInstance()`

```
bool Start::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.749.2.3 `_onDispatchEvent()`

```
void Start::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.749.2.4 `_saveInstance()`

```
void Start::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.749.2.5 `show()`

```
std::string Start::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

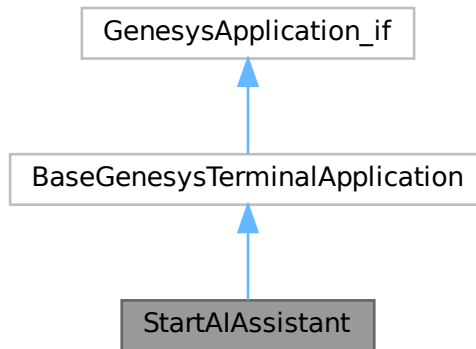
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

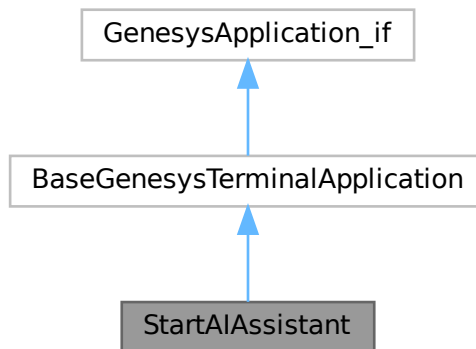
- plugins/components/MaterialHandling/Start.h
- plugins/components/MaterialHandling/Start.cpp

10.750 StartAIAssistant Class Reference

Inheritance diagram for StartAIAssistant:



Collaboration diagram for StartAIAssistant:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.750.1 Member Function Documentation

10.750.1.1 main()

```
int StartAIAssistant::main (
    int argc,
    char ** argv ) [override], [virtual]
```

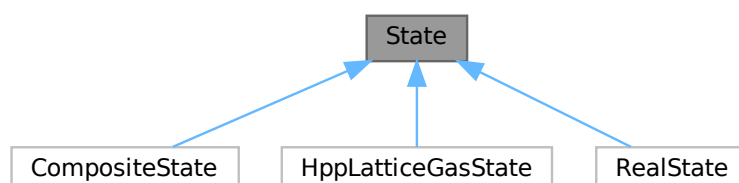
Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

- applications/modelSpecific/smarts/AI/StartAIAssistant.h
- applications/modelSpecific/smarts/AI/StartAIAssistant.cpp

10.751 State Class Reference

Inheritance diagram for State:



Public Member Functions

- **State** (long value)
- **State** (const [State](#) &orig)
- virtual std::unique_ptr< [State](#) > **clone** () const
- virtual bool **equals** (const [State](#) &other) const
- virtual std::string **toString** () const
- virtual std::string **show** () const
- virtual std::string **typeName** () const
- virtual long **getValue** () const
- virtual void **setValue** (long value)
- template<typename T >
const T * **as** () const noexcept
- template<typename T >
T * **as** () noexcept

Protected Attributes

- long **_value**

The documentation for this class was generated from the following file:

- plugins/components/ModalModel/CellularAutomata/State.h

10.752 GroProgramAst::Statement Struct Reference

Public Attributes

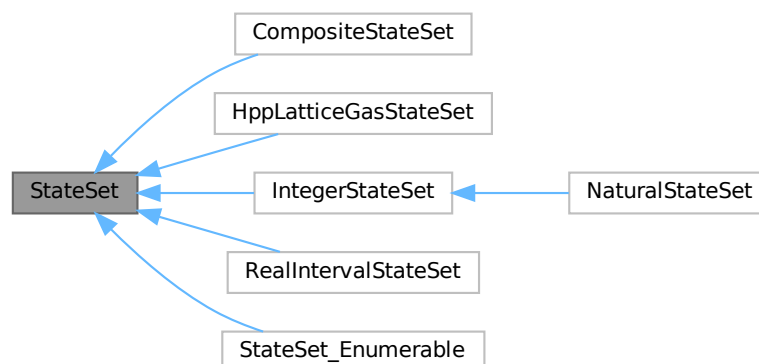
- std::string **sourceText** = ""

The documentation for this struct was generated from the following file:

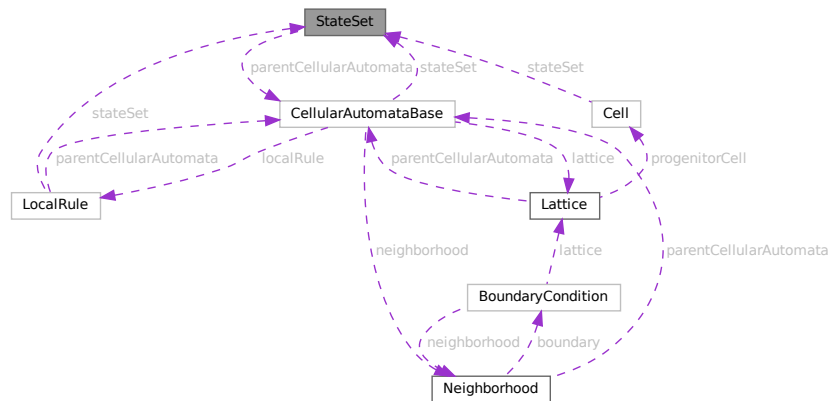
- plugins/data/BiochemicalSimulation/GroProgramAst.h

10.753 StateSet Class Reference

Inheritance diagram for StateSet:



Collaboration diagram for StateSet:



Public Member Functions

- **StateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- **StateSet** (const [StateSet](#) &orig)
- virtual bool **contains** (const [State](#) &state) const
- virtual std::unique_ptr< [State](#) > **createDefaultState** () const
- virtual std::unique_ptr< [State](#) > **parseState** (const std::string &text) const
- virtual bool **isFinite** () const
- virtual bool **isDiscrete** () const
- virtual bool **isContinuous** () const
- virtual bool **isComposite** () const
- virtual std::vector< std::unique_ptr< [State](#) > > **enumerateStates** () const
- virtual std::string **name** () const

Protected Attributes

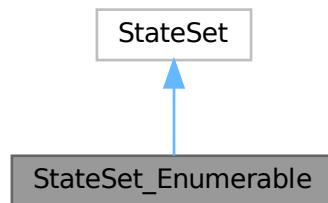
- [CellularAutomataBase](#) * **parentCellularAutomata**

The documentation for this class was generated from the following files:

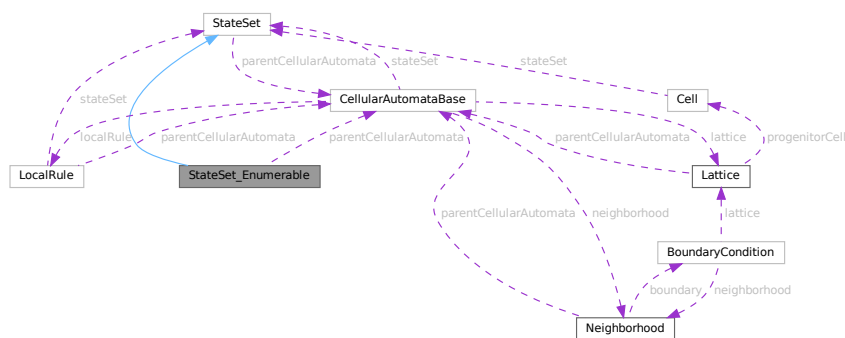
- plugins/components/ModalModel/CellularAutomata/StateSet.h
- plugins/components/ModalModel/CellularAutomata/StateSet.cpp

10.754 StateSet_Enumerable Class Reference

Inheritance diagram for StateSet_Enumerable:



Collaboration diagram for StateSet_Enumerable:



Public Member Functions

- **StateSet_Enumerable** (`CellularAutomataBase` *parentCellularAutomata, `std::vector`< `State` * > states={})
- **StateSet_Enumerable** (const `StateSet_Enumerable` &orig)
- virtual `std::string` **show** ()
- bool **contains** (const `State` &state) const override
- `std::unique_ptr`< `State` > **createDefaultState** () const override
- `std::unique_ptr`< `State` > **parseState** (const `std::string` &text) const override
- bool **isFinite** () const override
- bool **isDiscrete** () const override
- `std::vector`< `std::unique_ptr`< `State` > > **enumerateStates** () const override
- `std::string` **name** () const override
- unsigned int **size** ()
- void **addState** (`State` *state)
- void **addState** (const `State` &state)
- unsigned int **getStatesSize** ()
- `State` * **getState** (unsigned int rank)
- const `State` * **getState** (unsigned int rank) const
- `State` * **getState** (`std::string` name)
- void **setStates** (`std::vector`< `State` * > states)
- `std::vector`< `State` * > **getStates** () const

Public Member Functions inherited from [StateSet](#)

- **StateSet** ([CellularAutomataBase](#) *parentCellularAutomata)
- **StateSet** (const [StateSet](#) &orig)
- virtual bool **isContinuous** () const
- virtual bool **isComposite** () const

Protected Attributes

- [CellularAutomataBase](#) * **parentCellularAutomata**
- std::vector< std::unique_ptr< [State](#) > > **ownedStates**
- std::vector< [State](#) * > **states**

Protected Attributes inherited from [StateSet](#)

- [CellularAutomataBase](#) * **parentCellularAutomata**

10.754.1 Member Function Documentation

10.754.1.1 contains()

```
bool StateSet_Enumerable::contains (
    const State & state ) const [override], [virtual]
```

Reimplemented from [StateSet](#).

10.754.1.2 createDefaultState()

```
std::unique_ptr< State > StateSet_Enumerable::createDefaultState ( ) const [override], [virtual]
```

Reimplemented from [StateSet](#).

10.754.1.3 enumerateStates()

```
std::vector< std::unique_ptr< State > > StateSet_Enumerable::enumerateStates ( ) const [override],
[virtual]
```

Reimplemented from [StateSet](#).

10.754.1.4 isDiscrete()

```
bool StateSet_Enumerable::isDiscrete ( ) const [override], [virtual]
```

Reimplemented from [StateSet](#).

10.754.1.5 isFinite()

```
bool StateSet_Enumerable::isFinite ( ) const [override], [virtual]
```

Reimplemented from [StateSet](#).

10.754.1.6 name()

```
std::string StateSet_Enumerable::name ( ) const [override], [virtual]
```

Reimplemented from [StateSet](#).

10.754.1.7 parseState()

```
std::unique_ptr< State > StateSet_Enumerable::parseState (
    const std::string & text ) const [override], [virtual]
```

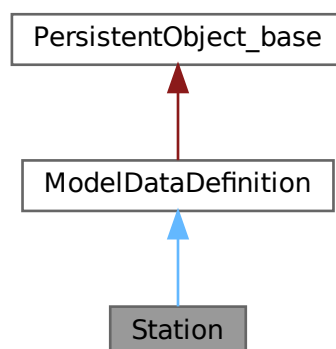
Reimplemented from [StateSet](#).

The documentation for this class was generated from the following files:

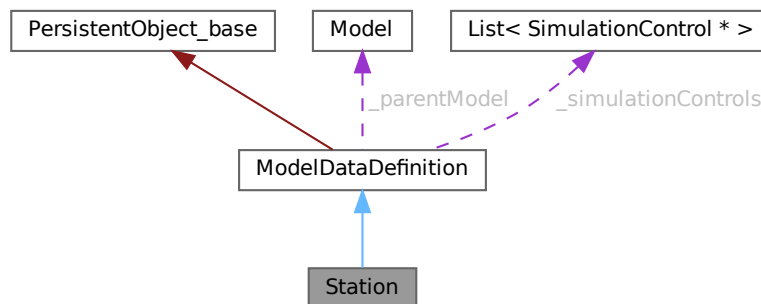
- plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h
- plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.cpp

10.755 Station Class Reference

Inheritance diagram for Station:



Collaboration diagram for Station:



Public Member Functions

- **Station** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **initBetweenReplications** ()
- void **enter** ([Entity](#) *entity)
- void **leave** ([Entity](#) *entity)
- void **setEnterIntoStationComponent** ([ModelComponent](#) * enterIntoStationComponent)
- [ModelComponent](#) * **getEnterIntoStationComponent** () const

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & **getLabel** () const
Returns the user-visible label of this instance.
- const std::string & **getClassname** () const
Returns the runtime class name used by the plugin system.
- [Model](#) * **getParentModel** () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

- Indicates whether this data definition contributes report statistics.*
 - void [setReportStatistics](#) (bool reportStatistics)
- Enables or disables report-statistics generation for this definition.*
 - [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
- Looks up a named internal child data definition.*
 - std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
- Returns the full map of internal child data definitions.*
 - std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
- Returns the full map of attached data definitions.*
 - bool [hasChanged](#) () const
- Indicates whether this instance has been modified since the last reset.*
 - void [setHasChanged](#) (bool [hasChanged](#))
- Updates the changed flag for this data definition.*
 - unsigned int [getLevel](#) () const
- Returns the level in the model hierarchy where this definition lives.*
 - void [setModelLevel](#) (unsigned int _modelLevel)
- Sets the model nesting level for this definition.*
 - [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
- Returns the simulation controls owned by this data definition.*
 - [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
- Returns the trace level that applies specifically to this definition.*
 - void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
- Updates the trace level used by this definition.*
 - void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
- Defines the specific trace level and optionally enables it immediately.*
 - bool [isTraceLevelSpecificEnabled](#) () const
- Indicates whether the specific trace level is enabled.*
 - void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
- Enables or disables the specific trace level.*

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
- Loads a model data definition instance from serialized fields.*
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
- Creates a new instance of a data definition using the plugin registry.*
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
- Serializes a data definition instance into a persistence record.*
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
- Validates a data definition instance.*
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
- Resets simulation-time state between replications.*
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
- Creates internal and attached data required by this instance.*
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
- Creates related data and validates the instance in a single call.*

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_initBetweenReplications` () override
- virtual void `_createInternalStatisticReporters` () override
- virtual void `_createAttachedAttributes` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Friends

- class **StationTestProbe**

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.755.1 Member Function Documentation

10.755.1.1 [_check\(\)](#)

```
bool Station::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.755.1.2 [_createAttachedAttributes\(\)](#)

```
void Station::_createAttachedAttributes ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.755.1.3 _createInternalStatisticReporters()

```
void Station::_createInternalStatisticReporters ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.755.1.4 _initBetweenReplications()

```
void Station::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.755.1.5 _loadInstance()

```
bool Station::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.755.1.6 _saveInstance()

```
void Station::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.755.1.7 show()

```
std::string Station::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

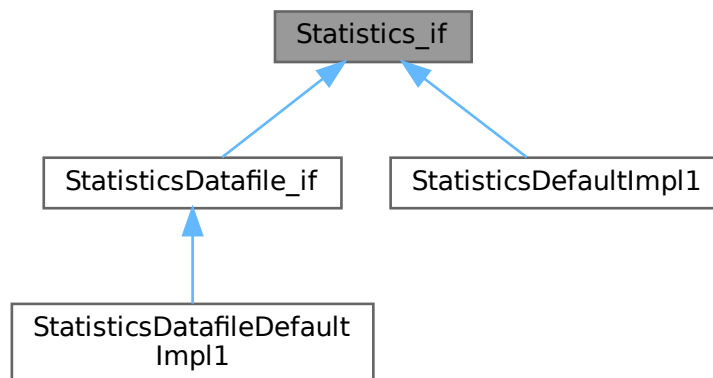
- plugins/data/MaterialHandling/Station.h
- plugins/data/MaterialHandling/Station.cpp

10.756 Statistics_if Class Reference

Interface for statistical synthesis over values provided by a [Collector_if](#).

```
#include <kernel/statistics/Statistics_if.h>
```

Inheritance diagram for Statistics_if:



Public Member Functions

- virtual [Collector_if](#) * [getCollector](#) () const =0
Returns the data collector associated with these statistics.
- virtual void [setCollector](#) ([Collector_if](#) *collector)=0
Sets the collector used as source for statistical calculations.
- virtual unsigned int [numElements](#) ()=0
numElements
- virtual double [min](#) ()=0
min
- virtual double [max](#) ()=0
max
- virtual double [average](#) ()=0

- average*
- virtual double [variance](#) ()=0
- variance*
- virtual double [stddeviation](#) ()=0
- stddeviation*
- virtual double [variationCoef](#) ()=0
- variationCoef*
- virtual double [halfWidthConfidenceInterval](#) ()=0
- halfWidthConfidenceInterval*
- virtual double [confidenceLevel](#) ()=0
- confidenceLevel*
- virtual unsigned int [newSampleSize](#) (double halfWidth)=0
- newSampleSize*
- virtual void [setConfidenceLevel](#) (double confidencelevel)=0
- setConfidenceLevel*

10.756.1 Detailed Description

Interface for statistical synthesis over values provided by a [Collector_if](#).

Implementations may compute metrics incrementally (online) or based on persisted datasets, but must expose a consistent API for descriptive and inferential measures.

10.756.2 Member Function Documentation

10.756.2.1 [average\(\)](#)

```
virtual double Statistics_if::average ( ) [pure virtual]
```

[average](#)

Returns

Returns the sample mean of observations.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

10.756.2.2 [confidenceLevel\(\)](#)

```
virtual double Statistics_if::confidenceLevel ( ) [pure virtual]
```

[confidenceLevel](#)

Returns

Returns the confidence level used for interval estimates.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

10.756.2.3 getCollector()

```
virtual Collector\_if * Statistics_if::getCollector ( ) const [pure virtual]
```

Returns the data collector associated with these statistics.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

10.756.2.4 halfWidthConfidenceInterval()

```
virtual double Statistics_if::halfWidthConfidenceInterval ( ) [pure virtual]
```

halfWidthConfidenceInterval

Returns

Returns the half-width of the current confidence interval.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

10.756.2.5 max()

```
virtual double Statistics_if::max ( ) [pure virtual]
```

max

Returns

Returns the maximum observed value.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

10.756.2.6 min()

```
virtual double Statistics_if::min ( ) [pure virtual]
```

min

Returns

Returns the minimum observed value.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

10.756.2.7 newSampleSize()

```
virtual unsigned int Statistics_if::newSampleSize (
    double halfWidth ) [pure virtual]
```

newSampleSize

Parameters

<i>halfWidth</i>	
------------------	--

Returns

Estimates the sample size needed to reach a target half-width.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

10.756.2.8 numElements()

```
virtual unsigned int Statistics_if::numElements ( ) [pure virtual]
```

numElements

Returns

Returns the number of observations considered in calculations.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

10.756.2.9 setCollector()

```
virtual void Statistics_if::setCollector (
    Collector_if * collector ) [pure virtual]
```

Sets the collector used as source for statistical calculations.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

10.756.2.10 setConfidenceLevel()

```
virtual void Statistics_if::setConfidenceLevel (
    double confidencelevel ) [pure virtual]
```

setConfidenceLevel

Parameters

<i>confidencelevel</i>	
------------------------	--

Sets the confidence level used in inferential calculations.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

10.756.2.11 stddeviation()

```
virtual double Statistics_if::stddeviation ( ) [pure virtual]
```

stddeviation

Returns

Returns the sample standard deviation.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

10.756.2.12 variance()

```
virtual double Statistics_if::variance ( ) [pure virtual]
```

variance

Returns

Returns the sample variance of observations.

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

10.756.2.13 variationCoef()

```
virtual double Statistics_if::variationCoef ( ) [pure virtual]
```

variationCoef

Returns

Returns the coefficient of variation (standard deviation / mean).

Implemented in [StatisticsDatafileDefaultImpl1](#), and [StatisticsDefaultImpl1](#).

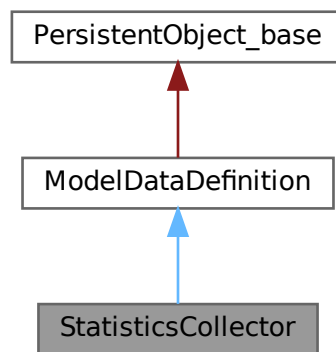
The documentation for this class was generated from the following file:

- kernel/statistics/Statistics_if.h

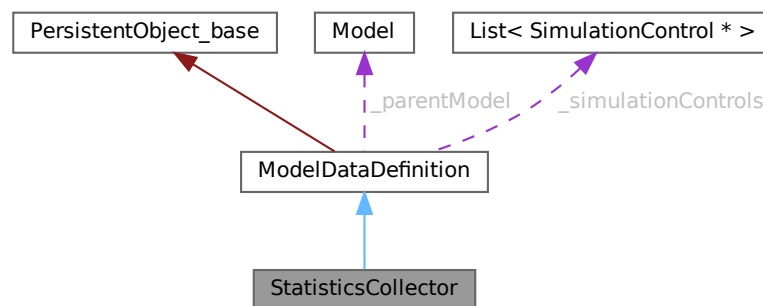
10.757 StatisticsCollector Class Reference

```
#include <kernel/simulator/essentialPlugins/StatisticsCollector.h>
```

Inheritance diagram for StatisticsCollector:



Collaboration diagram for StatisticsCollector:



Public Member Functions

- **StatisticsCollector** ([Model](#) *model, std::string name="", [ModelDataDefinition](#) *parent=nullptr, bool insert↔ IntoModel=true)
- virtual std::string [show](#) () override
Returns a textual representation of this data definition.
- [ModelDataDefinition](#) * [getParent](#) () const
getParent
- [Statistics_if](#) * [getStatistics](#) () const
getStatistics

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()

Releases resources owned by this data definition.
- Util::identification [getId](#) () const

Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)

Updates the logical name of this instance.
- const std::string & [getName](#) () const

Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const

Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const

Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const

Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const

Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const

Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const

Returns the full map of attached data definitions.
- bool [hasChanged](#) () const

Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))

Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const

Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const

Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)

Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_initBetweenReplications** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createNonEditableDataDefinitions** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()

- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.757.1 Detailed Description

The [StatisticsCollector](#) is the [ModelDataDefinition](#) responsible for collecting data from the model (using the Collector) and simultaneously keeping statistics updated (using the Statistics)

10.757.2 Member Function Documentation

10.757.2.1 `_check()`

```
bool StatisticsCollector::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.757.2.2 `_loadInstance()`

```
bool StatisticsCollector::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.757.2.3 `_saveInstance()`

```
void StatisticsCollector::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.757.2.4 `getParent()`

```
ModelDataDefinition * StatisticsCollector::getParent ( ) const
```

getParent

Returns

10.757.2.5 `getStatistics()`

```
Statistics_if * StatisticsCollector::getStatistics ( ) const
```

`getStatistics`

Returns

10.757.2.6 `show()`

```
std::string StatisticsCollector::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

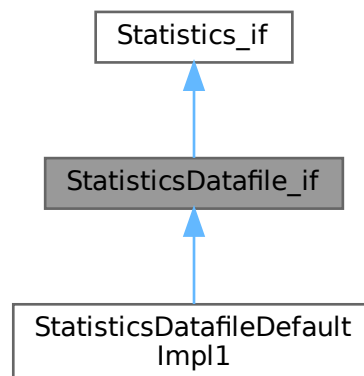
- kernel/simulator/essentialPlugins/StatisticsCollector.h
- kernel/simulator/essentialPlugins/StatisticsCollector.cpp

10.758 StatisticsDatafile_if Class Reference

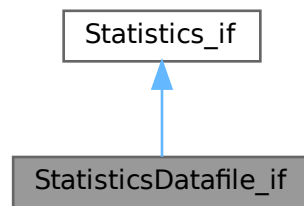
Extension of [Statistics_if](#) for metrics that require full data access.

```
#include <kernel/statistics/StatisticsDataFile_if.h>
```

Inheritance diagram for StatisticsDatafile_if:



Collaboration diagram for StatisticsDatafile_if:



Public Member Functions

- virtual double `mode` ()=0
mode
- virtual double `mediane` ()=0
mediane
- virtual double `quartil` (unsigned short num)=0
quartil
- virtual double `decil` (unsigned short num)=0
decil
- virtual double `centil` (unsigned short num)=0
centil
- virtual void `setHistogramNumClasses` (unsigned short num)=0
setHistogramNumClasses
- virtual unsigned short `histogramNumClasses` ()=0
histogramNumClasses
- virtual double `histogramClassLowerLimit` (unsigned short classNum)=0
histogramClassLowerLimit
- virtual unsigned int `histogramClassFrequency` (unsigned short classNum)=0
histogramClassFrequency

Public Member Functions inherited from `Statistics_if`

- virtual `Collector_if` * `getCollector` () const =0
Returns the data collector associated with these statistics.
- virtual void `setCollector` (`Collector_if` *collector)=0
Sets the collector used as source for statistical calculations.
- virtual unsigned int `numElements` ()=0
numElements
- virtual double `min` ()=0
min
- virtual double `max` ()=0
max
- virtual double `average` ()=0

- average*
- virtual double [variance](#) ()=0
- variance*
- virtual double [stddeviation](#) ()=0
- stddeviation*
- virtual double [variationCoef](#) ()=0
- variationCoef*
- virtual double [halfWidthConfidenceInterval](#) ()=0
- halfWidthConfidenceInterval*
- virtual double [confidenceLevel](#) ()=0
- confidenceLevel*
- virtual unsigned int [newSampleSize](#) (double halfWidth)=0
- newSampleSize*
- virtual void [setConfidenceLevel](#) (double confidencelevel)=0
- setConfidenceLevel*

10.758.1 Detailed Description

Extension of [Statistics_if](#) for metrics that require full data access.

This interface targets collector implementations backed by data files, enabling order-based measures (median, quantiles) and histogram-related operations.

10.758.2 Member Function Documentation

10.758.2.1 centil()

```
virtual double StatisticsDatafile_if::centil (
    unsigned short num ) [pure virtual]
```

centil

Parameters

<i>num</i>	
------------	--

Returns

Returns the requested percentile (1 to 99).

Implemented in [StatisticsDatafileDefaultImpl1](#).

10.758.2.2 decil()

```
virtual double StatisticsDatafile_if::decil (
    unsigned short num ) [pure virtual]
```

decil

Parameters

<i>num</i>	
------------	--

Returns

Returns the requested decile (1 to 9).

Implemented in [StatisticsDatafileDefaultImpl1](#).

10.758.2.3 histogramClassFrequency()

```
virtual unsigned int StatisticsDatafile_if::histogramClassFrequency (  
    unsigned short classNum ) [pure virtual]
```

histogramClassFrequency

Parameters

<i>classNum</i>	
-----------------	--

Returns

Returns the absolute frequency of the requested histogram class.

Implemented in [StatisticsDatafileDefaultImpl1](#).

10.758.2.4 histogramClassLowerLimit()

```
virtual double StatisticsDatafile_if::histogramClassLowerLimit (  
    unsigned short classNum ) [pure virtual]
```

histogramClassLowerLimit

Parameters

<i>classNum</i>	
-----------------	--

Returns

Returns the lower bound of the histogram class at index *classNum*.

Implemented in [StatisticsDatafileDefaultImpl1](#).

10.758.2.5 histogramNumClasses()

```
virtual unsigned short StatisticsDatafile_if::histogramNumClasses ( ) [pure virtual]
```

histogramNumClasses

Returns

Returns the configured number of histogram classes.

Implemented in [StatisticsDatafileDefaultImpl1](#).

10.758.2.6 mediane()

```
virtual double StatisticsDatafile_if::mediane ( ) [pure virtual]
```

mediane

Returns

Returns the sample median.

Implemented in [StatisticsDatafileDefaultImpl1](#).

10.758.2.7 mode()

```
virtual double StatisticsDatafile_if::mode ( ) [pure virtual]
```

mode

Returns

Returns the mode of the sample stored in file.

Implemented in [StatisticsDatafileDefaultImpl1](#).

10.758.2.8 quartil()

```
virtual double StatisticsDatafile_if::quartil (
    unsigned short num ) [pure virtual]
```

quartil

Parameters

<i>num</i>	
------------	--

Returns

Returns the requested quartile (1 to 3).

Implemented in [StatisticsDatafileDefaultImpl1](#).

10.758.2.9 setHistogramNumClasses()

```
virtual void StatisticsDatafile_if::setHistogramNumClasses (
    unsigned short num ) [pure virtual]
```

setHistogramNumClasses

Parameters

<i>num</i>	
------------	--

Sets the number of classes used in the histogram.

Implemented in [StatisticsDatafileDefaultImpl1](#).

The documentation for this class was generated from the following file:

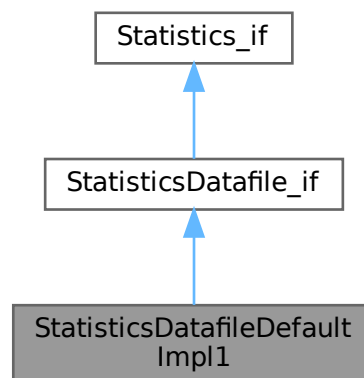
- kernel/statistics/StatisticsDataFile_if.h

10.759 StatisticsDatafileDefaultImpl1 Class Reference

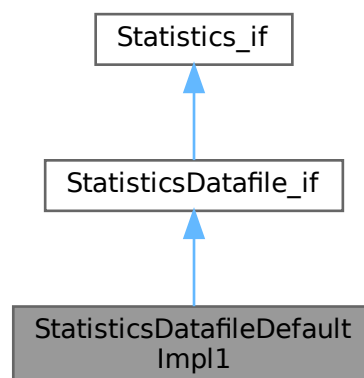
Default implementation of [StatisticsDatafile_if](#) for file-backed samples.

```
#include <kernel/statistics/StatisticsDataFileDefaultImpl.h>
```


Inheritance diagram for StatisticsDatafileDefaultImpl1:



Collaboration diagram for StatisticsDatafileDefaultImpl1:



Public Member Functions

- **StatisticsDatafileDefaultImpl1 ()**
Creates statistics object with default collector implementation.
- virtual **~StatisticsDatafileDefaultImpl1 ()**
Releases owned collector/sorter resources when configured as owner.
- virtual **Collector_if * getCollector ()** const override
Returns the collector currently used as data source.
- virtual void **setCollector (Collector_if *collector)** override
Sets a collector to be used as data source for calculations.

- virtual unsigned int `numElements` () override
Returns number of observations available in the collector.
- virtual double `min` () override
Returns minimum observed value.
- virtual double `max` () override
Returns maximum observed value.
- virtual double `average` () override
Returns arithmetic mean.
- virtual double `variance` () override
Returns sample variance.
- virtual double `stddeviation` () override
Returns sample standard deviation.
- virtual double `variationCoef` () override
Returns coefficient of variation.
- virtual double `halfWidthConfidenceInterval` () override
Returns current confidence interval half-width.
- virtual double `confidenceLevel` () override
Returns configured confidence level.
- virtual unsigned int `newSampleSize` (double halfWidth) override
Estimates sample size needed for a desired half-width.
- virtual void `setConfidenceLevel` (double confidencelevel) override
Sets confidence level used in inferential calculations.
- virtual double `mode` () override
Returns sample mode.
- virtual double `median` () override
Returns sample median.
- virtual double `quartil` (unsigned short num) override
Returns requested quartile.
- virtual double `decil` (unsigned short num) override
Returns requested decile.
- virtual double `centil` (unsigned short num) override
Returns requested percentile.
- virtual void `setHistogramNumClasses` (unsigned short num) override
Sets number of histogram classes.
- virtual unsigned short `histogramNumClasses` () override
Returns current number of histogram classes.
- virtual double `histogramClassLowerLimit` (unsigned short classNum) override
Returns lower limit of histogram class `classNum`.
- virtual unsigned int `histogramClassFrequency` (unsigned short classNum) override
Returns absolute frequency of histogram class `classNum`.

10.759.1 Detailed Description

Default implementation of `StatisticsDatafile_if` for file-backed samples.

Computes descriptive/inferential statistics from a `CollectorDatafile_if` and caches results to avoid recomputation until source data changes.

10.759.2 Member Function Documentation

10.759.2.1 average()

```
double StatisticsDatafileDefaultImpl1::average ( ) [override], [virtual]
```

Returns arithmetic mean.

Implements [Statistics_if](#).

10.759.2.2 centil()

```
double StatisticsDatafileDefaultImpl1::centil (
    unsigned short num ) [override], [virtual]
```

Returns requested percentile.

Implements [StatisticsDatafile_if](#).

10.759.2.3 confidenceLevel()

```
double StatisticsDatafileDefaultImpl1::confidenceLevel ( ) [override], [virtual]
```

Returns configured confidence level.

Implements [Statistics_if](#).

10.759.2.4 decil()

```
double StatisticsDatafileDefaultImpl1::decil (
    unsigned short num ) [override], [virtual]
```

Returns requested decile.

Implements [StatisticsDatafile_if](#).

10.759.2.5 getCollector()

```
Collector\_if * StatisticsDatafileDefaultImpl1::getCollector ( ) const [override], [virtual]
```

Returns the collector currently used as data source.

Implements [Statistics_if](#).

10.759.2.6 halfWidthConfidenceInterval()

```
double StatisticsDatafileDefaultImpl1::halfWidthConfidenceInterval ( ) [override], [virtual]
```

Returns current confidence interval half-width.

Implements [Statistics_if](#).

10.759.2.7 histogramClassFrequency()

```
unsigned int StatisticsDatafileDefaultImpl1::histogramClassFrequency (
    unsigned short classNum ) [override], [virtual]
```

Returns absolute frequency of histogram class `classNum`.

Implements [StatisticsDatafile_if](#).

10.759.2.8 histogramClassLowerLimit()

```
double StatisticsDatafileDefaultImpl1::histogramClassLowerLimit (
    unsigned short classNum ) [override], [virtual]
```

Returns lower limit of histogram class `classNum`.

Implements [StatisticsDatafile_if](#).

10.759.2.9 histogramNumClasses()

```
unsigned short StatisticsDatafileDefaultImpl1::histogramNumClasses ( ) [override], [virtual]
```

Returns current number of histogram classes.

Implements [StatisticsDatafile_if](#).

10.759.2.10 max()

```
double StatisticsDatafileDefaultImpl1::max ( ) [override], [virtual]
```

Returns maximum observed value.

Implements [Statistics_if](#).

10.759.2.11 mediane()

```
double StatisticsDatafileDefaultImpl1::mediane ( ) [override], [virtual]
```

Returns sample median.

Implements [StatisticsDatafile_if](#).

10.759.2.12 min()

```
double StatisticsDatafileDefaultImpl1::min ( ) [override], [virtual]
```

Returns minimum observed value.

Implements [Statistics_if](#).

10.759.2.13 mode()

```
double StatisticsDatafileDefaultImpl1::mode ( ) [override], [virtual]
```

Returns sample mode.

Implements [StatisticsDatafile_if](#).

10.759.2.14 newSampleSize()

```
unsigned int StatisticsDatafileDefaultImpl1::newSampleSize (
    double halfWidth ) [override], [virtual]
```

Estimates sample size needed for a desired half-width.

Implements [Statistics_if](#).

10.759.2.15 numElements()

```
unsigned int StatisticsDatafileDefaultImpl1::numElements ( ) [override], [virtual]
```

Returns number of observations available in the collector.

Implements [Statistics_if](#).

10.759.2.16 quartil()

```
double StatisticsDatafileDefaultImpl1::quartil (
    unsigned short num ) [override], [virtual]
```

Returns requested quartile.

Implements [StatisticsDatafile_if](#).

10.759.2.17 setCollector()

```
void StatisticsDatafileDefaultImpl1::setCollector (
    Collector_if * collector ) [override], [virtual]
```

Sets a collector to be used as data source for calculations.

Implements [Statistics_if](#).

10.759.2.18 setConfidenceLevel()

```
void StatisticsDatafileDefaultImpl1::setConfidenceLevel (
    double confidencelevel ) [override], [virtual]
```

Sets confidence level used in inferential calculations.

Implements [Statistics_if](#).

10.759.2.19 setHistogramNumClasses()

```
void StatisticsDatafileDefaultImpl1::setHistogramNumClasses (
    unsigned short num ) [override], [virtual]
```

Sets number of histogram classes.

Implements [StatisticsDatafile_if](#).

10.759.2.20 stddeviation()

```
double StatisticsDatafileDefaultImpl1::stddeviation ( ) [override], [virtual]
```

Returns sample standard deviation.

Implements [Statistics_if](#).

10.759.2.21 variance()

```
double StatisticsDatafileDefaultImpl1::variance ( ) [override], [virtual]
```

Returns sample variance.

Implements [Statistics_if](#).

10.759.2.22 variationCoef()

```
double StatisticsDatafileDefaultImpl1::variationCoef ( ) [override], [virtual]
```

Returns coefficient of variation.

Implements [Statistics_if](#).

The documentation for this class was generated from the following files:

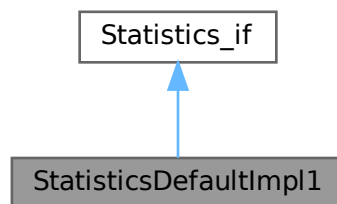
- kernel/statistics/StatisticsDataFileDefaultImpl.h
- kernel/statistics/StatisticsDataFileDefaultImpl.cpp

10.760 StatisticsDefaultImpl1 Class Reference

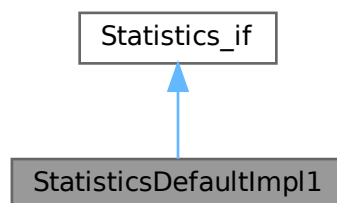
Default online statistics implementation over a [Collector_if](#) source.

```
#include <kernel/statistics/StatisticsDefaultImpl1.h>
```

Inheritance diagram for StatisticsDefaultImpl1:



Collaboration diagram for StatisticsDefaultImpl1:



Public Member Functions

- [StatisticsDefaultImpl1](#) ()
Creates statistics object using configured default collector implementation.
- **StatisticsDefaultImpl1** ([Collector_if](#) *collector)
Creates statistics object bound to an external collector.
- virtual [Collector_if](#) * [getCollector](#) () const override
Returns current collector data source.
- virtual void [setCollector](#) ([Collector_if](#) *collector) override
Sets collector source and reconnects internal callbacks.
- virtual unsigned int [numElements](#) () override
Returns number of observed samples.
- virtual double [min](#) () override
Returns minimum observed value.

- virtual double `max` () override
Returns maximum observed value.
- virtual double `average` () override
Returns arithmetic mean.
- virtual double `variance` () override
Returns sample variance.
- virtual double `stddeviation` () override
Returns sample standard deviation.
- virtual double `variationCoef` () override
Returns coefficient of variation.
- virtual double `halfWidthConfidenceInterval` () override
Returns confidence interval half-width.
- virtual double `confidenceLevel` () override
Returns configured confidence level.
- virtual unsigned int `newSampleSize` (double halfWidth) override
Estimates sample size required for desired half-width.
- virtual void `setConfidenceLevel` (double confidencelevel) override
Sets confidence level used in inferential formulas.

10.760.1 Detailed Description

Default online statistics implementation over a `Collector_if` source.

Updates core metrics incrementally through collector callbacks, avoiding complete rescans for every query.

10.760.2 Constructor & Destructor Documentation

10.760.2.1 StatisticsDefaultImpl1()

```
StatisticsDefaultImpl1::StatisticsDefaultImpl1 ( )
```

Creates statistics object using configured default collector implementation.

When constructor is invoked without a `Collector`, it is taken from `Traits<Statistics_if>::CollectorImplementation` configuration

10.760.3 Member Function Documentation

10.760.3.1 average()

```
double StatisticsDefaultImpl1::average ( ) [override], [virtual]
```

Returns arithmetic mean.

Implements `Statistics_if`.

10.760.3.2 confidenceLevel()

```
double StatisticsDefaultImpl1::confidenceLevel ( ) [override], [virtual]
```

Returns configured confidence level.

Implements [Statistics_if](#).

10.760.3.3 getCollector()

```
Collector_if * StatisticsDefaultImpl1::getCollector ( ) const [override], [virtual]
```

Returns current collector data source.

Implements [Statistics_if](#).

10.760.3.4 halfWidthConfidenceInterval()

```
double StatisticsDefaultImpl1::halfWidthConfidenceInterval ( ) [override], [virtual]
```

Returns confidence interval half-width.

Implements [Statistics_if](#).

10.760.3.5 max()

```
double StatisticsDefaultImpl1::max ( ) [override], [virtual]
```

Returns maximum observed value.

Implements [Statistics_if](#).

10.760.3.6 min()

```
double StatisticsDefaultImpl1::min ( ) [override], [virtual]
```

Returns minimum observed value.

Implements [Statistics_if](#).

10.760.3.7 newSampleSize()

```
unsigned int StatisticsDefaultImpl1::newSampleSize (
    double halfWidth ) [override], [virtual]
```

Estimates sample size required for desired half-width.

Implements [Statistics_if](#).

10.760.3.8 numElements()

```
unsigned int StatisticsDefaultImpl1::numElements ( ) [override], [virtual]
```

Returns number of observed samples.

Implements [Statistics_if](#).

10.760.3.9 setCollector()

```
void StatisticsDefaultImpl1::setCollector (
    Collector_if * collector ) [override], [virtual]
```

Sets collector source and reconnects internal callbacks.

Implements [Statistics_if](#).

10.760.3.10 setConfidenceLevel()

```
void StatisticsDefaultImpl1::setConfidenceLevel (
    double confidencelevel ) [override], [virtual]
```

Sets confidence level used in inferential formulas.

Implements [Statistics_if](#).

10.760.3.11 stddeviation()

```
double StatisticsDefaultImpl1::stddeviation ( ) [override], [virtual]
```

Returns sample standard deviation.

Implements [Statistics_if](#).

10.760.3.12 variance()

```
double StatisticsDefaultImpl1::variance ( ) [override], [virtual]
```

Returns sample variance.

Implements [Statistics_if](#).

10.760.3.13 variationCoef()

```
double StatisticsDefaultImpl1::variationCoef ( ) [override], [virtual]
```

Returns coefficient of variation.

Implements [Statistics_if](#).

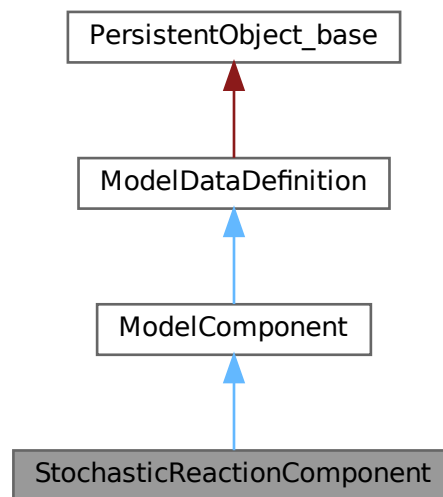
The documentation for this class was generated from the following files:

- kernel/statistics/StatisticsDefaultImpl1.h
- kernel/statistics/StatisticsDefaultImpl1.cpp

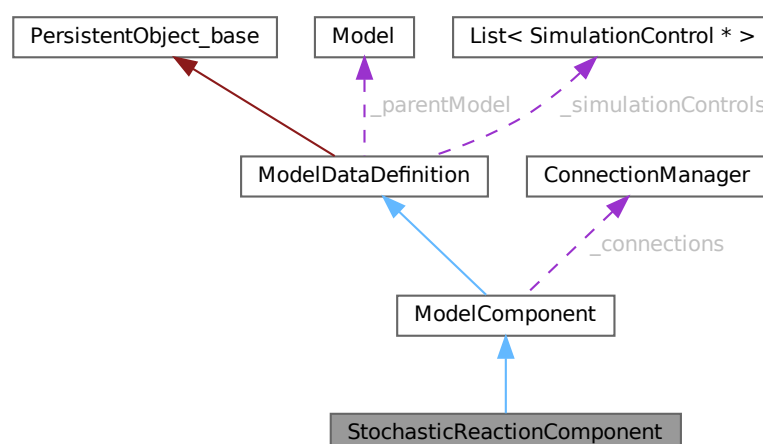
10.761 StochasticReactionComponent Class Reference

```
#include <plugins/components/WholeCellModeling/StochasticReactionComponent.h>
```

Inheritance diagram for StochasticReactionComponent:



Collaboration diagram for StochasticReactionComponent:



Public Member Functions

- **StochasticReactionComponent** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setTimeWindow** (double timeWindow)
- double **getTimeWindow** () const
- void **setAdvanceWholeCellClock** (bool advance)
- bool **getAdvanceWholeCellClock** () const
- void **setRandomSeed** (unsigned int seed)
- unsigned int **getRandomSeed** () const
- int **getLastReactionCount** () const
- double **getLastSimulatedTime** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const
Returns the logical name of this instance.
- void **setLabel** (const std::string &label)
Updates the user-visible label of this instance.

- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
 - Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.761.1 Detailed Description

Gillespie Direct Method SSA runner for whole-cell stochastic simulations.

On each entity arrival the component runs the Direct Method (Gillespie 1977) for a configurable time window [0, timeWindow] using all [StochasticReactionRule](#) definitions registered in the model. Species counts are read from and written back to the referenced [WholeCellState](#) (if set) or directly to [MolecularSpecies](#) objects in the model.

Contract (Q7b): Input: current molecule counts from [WholeCellState](#) (or [MolecularSpecies](#))

- resource budget from [WholeCellState](#) Output: updated counts written back to [WholeCellState](#); entity forwarded

[Resource](#) cap: reactions whose firing would deplete a species below its resource budget floor are skipped, matching the fair-allocation contract of the WholeCell M. genitalium model (Karr et al. 2012).

10.761.2 Member Function Documentation

10.761.2.1 `_check()`

```
bool StochasticReactionComponent::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.761.2.2 `_createEditableDataDefinitions()`

```
void StochasticReactionComponent::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.761.2.3 `_initBetweenReplications()`

```
void StochasticReactionComponent::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.761.2.4 `_loadInstance()`

```
bool StochasticReactionComponent::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.761.2.5 `_onDispatchEvent()`

```
void StochasticReactionComponent::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.761.2.6 `_saveInstance()`

```
void StochasticReactionComponent::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.761.2.7 show()

```
std::string StochasticReactionComponent::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

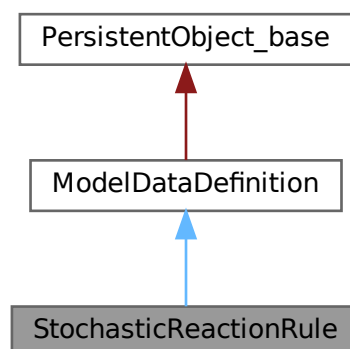
The documentation for this class was generated from the following files:

- plugins/components/WholeCellModeling/StochasticReactionComponent.h
- plugins/components/WholeCellModeling/StochasticReactionComponent.cpp

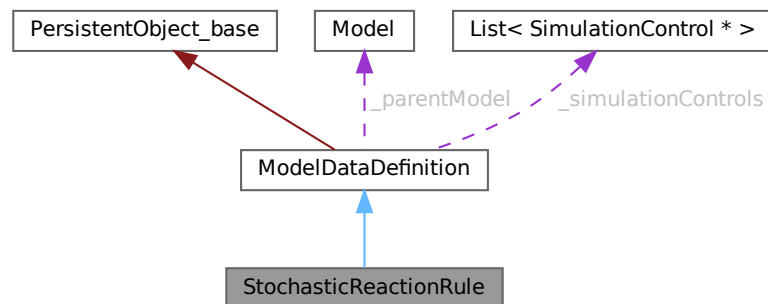
10.762 StochasticReactionRule Class Reference

```
#include <plugins/data/WholeCellModeling/StochasticReactionRule.h>
```

Inheritance diagram for StochasticReactionRule:



Collaboration diagram for StochasticReactionRule:



Classes

- struct [StoichiometricTerm](#)

Public Member Functions

- **StochasticReactionRule** ([Model](#) *model, std::string name="")
- void **addReactant** (std::string speciesName, int stoichiometry=1)
- void **addProduct** (std::string speciesName, int stoichiometry=1)
- void **clearReactants** ()
- void **clearProducts** ()
- const std::vector< [StoichiometricTerm](#) > & **getReactants** () const
- const std::vector< [StoichiometricTerm](#) > & **getProducts** () const
- void **setRateConstant** (double rateConstant)
- double **getRateConstant** () const
- void **setCompartment** (std::string compartment)
- std::string **getCompartment** () const
- double **computePropensity** (const std::map< std::string, int > &counts) const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)
- *Creates a model data definition and optionally inserts it into the model.*
- virtual ~**ModelDataDefinition** ()
- *Releases resources owned by this data definition.*
- Util::identification **getId** () const
- *Returns the unique identifier of this instance.*
- void **setName** (const std::string &name)
- *Updates the logical name of this instance.*
- const std::string & **getName** () const

- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)

- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.762.1 Detailed Description

Stochastic reaction rule for Gillespie SSA whole-cell simulations.

Reactants and products are named by [MolecularSpecies](#) with integer stoichiometry. The propensity is computed as $k * \text{combinatorial}(\text{counts})$, where the combinatorial term is the product of $C(x_i, n_i)$ for each reactant species i with stoichiometry n_i .

The rate constant k uses stochastic (microscopic) units: s^{-1} for unimolecular, $\text{molecules}^{-1} * s^{-1}$ for bimolecular reactions.

10.762.2 Member Function Documentation

10.762.2.1 _check()

```
bool StochasticReactionRule::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.762.2.2 _loadInstance()

```
bool StochasticReactionRule::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.762.2.3 _saveInstance()

```
void StochasticReactionRule::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.762.2.4 show()

```
std::string StochasticReactionRule::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

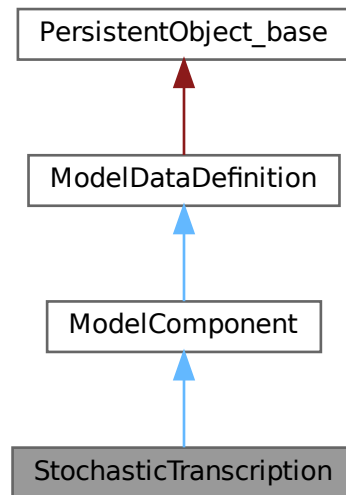
The documentation for this class was generated from the following files:

- plugins/data/WholeCellModeling/StochasticReactionRule.h
- plugins/data/WholeCellModeling/StochasticReactionRule.cpp

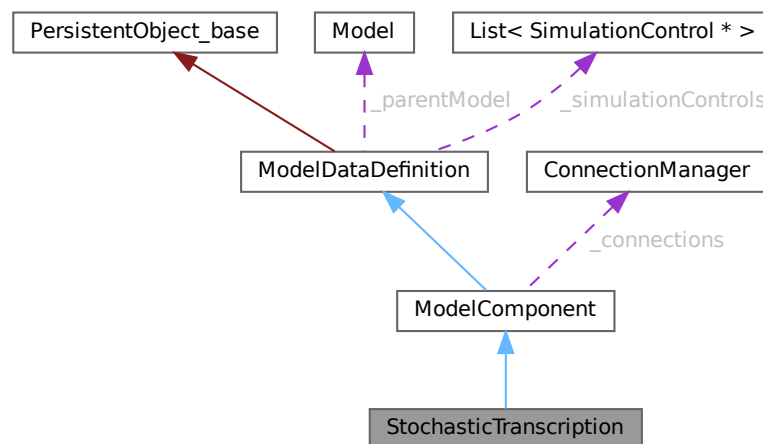
10.763 StochasticTranscription Class Reference

```
#include <plugins/components/WholeCellModeling/StochasticTranscription.h>
```

Inheritance diagram for StochasticTranscription:



Collaboration diagram for StochasticTranscription:



Public Member Functions

- **StochasticTranscription** ([Model](#) *model, std::string name="")

- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setElongationRate** (double rate)
- double **getElongationRate** () const
- void **setMeanGeneLength** (double length)
- double **getMeanGeneLength** () const
- void **setBindingProbability** (double prob)
- double **getBindingProbability** () const
- void **setTimeWindow** (double window)
- double **getTimeWindow** () const
- void **setMRNASpeciesPrefix** (std::string prefix)
- std::string **getMRNASpeciesPrefix** () const
- void **setRnapCountKey** (std::string key)
- std::string **getRnapCountKey** () const
- void **setRandomSeed** (unsigned int seed)
- unsigned int **getRandomSeed** () const
- int **getLastSynthesizedCount** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const

- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool \_getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * \_getParserChangesInformation ()`
- `virtual void \_addSimulationControl (SimulationControl *control)`
- `virtual void \_createInternalAndAttachedData ()`
- `virtual void \_createInternalStatisticReporters ()`
- `virtual void \_createAttachedAttributes ()`
- `virtual void \_createNonEditableDataDefinitions ()`
- `void \_templateCreateInternalStatisticReporters ()`
- `void \_templateCreateAttachedAttributes ()`
- `void \_templateCreateNonEditableDataDefinitions ()`
- `void \_templateCreateEditableDataDefinitions ()`
- `void \_internalStatisticReportersClear ()`
- *legacy helper methods used by the granular create* hooks*
- `void \_internalDataClear ()`
- `void \_internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void \_internalDataRemove (const std::string &key)`
- `void \_attachedDataClear ()`
- `void \_attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_attachedDataRemove (const std::string &key)`
- `void \_attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void \_statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_statisticReporterRemove (const std::string &key)`
- `void \_statisticReportersClear ()`
- `void \_mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryAttachedAttributeRemove (const std::string &key)`
- `void \_mandatoryAttachedAttributesClear ()`
- `void \_mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void \_mandatoryEditableDataDefinitionsClear ()`
- `void \_optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_optionalEditableDataDefinitionRemove (const std::string &key)`
- `void \_optionalEditableDataDefinitionsClear ()`
- `void \_mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void \_mandatoryNonEditableDataDefinitionsClear ()`
- `void \_optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void \_optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void \_optionalNonEditableDataDefinitionsClear ()`
- `void \_checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- *method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- *Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.763.1 Detailed Description

Stochastic mRNA synthesis via Poisson tau-leaping for whole-cell simulations.

Models the *M. genitalium* Transcription process (Karr et al. 2012): $\lambda_i = (\text{elongationRate} / \text{meanGeneLength}) \times \text{bindingProbability} \times \text{RNAP_free} \times \Delta t$ $N_{\text{synth}_i} \sim \text{Poisson}(\lambda_i)$

This is a tau-leaping approximation (not full Gillespie) appropriate for transcription because RNAP count (~200) acting over many genes (~473) with a 1–10 s window produces moderate propensities where the Poisson approximation is valid. Full Gillespie is implemented in [StochasticReactionComponent](#) for individual reactions with low copy numbers.

Parameters loaded from [WholeCellState](#) (set by `loadParameters()`): `param.rnaPolymeraseElongationRate` — 50 nt/s (from `parameters.json`)

mRNA species in [WholeCellState](#) are identified by the prefix set in `mRNASpeciesPrefix` (default "mRNA_"). RNAP free count is read from the [WholeCellState](#) molecule count with key `rnapCountKey` (default "RNAP_free").

10.763.2 Member Function Documentation

10.763.2.1 `_check()`

```
bool StochasticTranscription::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.763.2.2 _createEditableDataDefinitions()

```
void StochasticTranscription::_createEditableDataDefinitions ( ) [override], [protected],
[virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.763.2.3 _initBetweenReplications()

```
void StochasticTranscription::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.763.2.4 _loadInstance()

```
bool StochasticTranscription::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.763.2.5 _onDispatchEvent()

```
void StochasticTranscription::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.763.2.6 _saveInstance()

```
void StochasticTranscription::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.763.2.7 show()

```
std::string StochasticTranscription::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

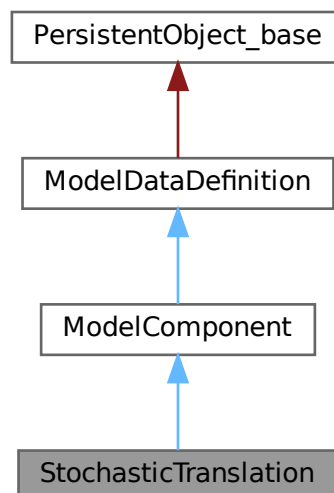
The documentation for this class was generated from the following files:

- `plugins/components/WholeCellModeling/StochasticTranscription.h`
- `plugins/components/WholeCellModeling/StochasticTranscription.cpp`

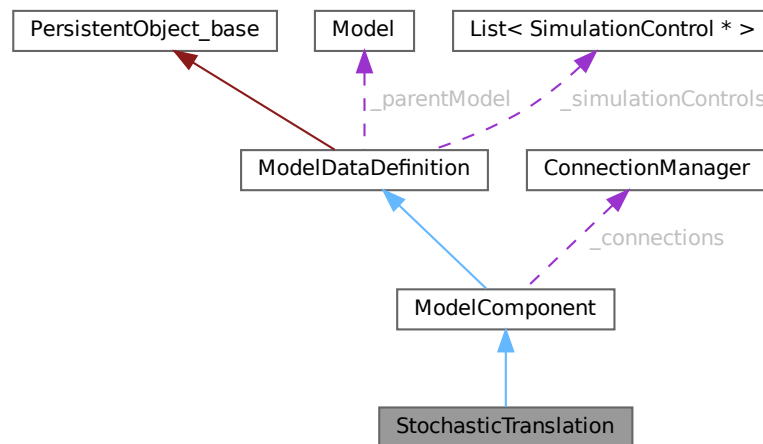
10.764 StochasticTranslation Class Reference

```
#include <plugins/components/WholeCellModeling/StochasticTranslation.h>
```

Inheritance diagram for StochasticTranslation:



Collaboration diagram for StochasticTranslation:



Public Member Functions

- **StochasticTranslation** ([Model](#) *model, std::string name="")
- void **setWholeCellState** ([WholeCellState](#) *state)
- [WholeCellState](#) * **getWholeCellState** () const
- void **setElongationRate** (double rate)
- double **getElongationRate** () const
- void **setMeanProteinLength** (double length)
- double **getMeanProteinLength** () const
- void **setTimeWindow** (double window)
- double **getTimeWindow** () const
- void **setMRNASpeciesPrefix** (std::string prefix)
- std::string **getMRNASpeciesPrefix** () const
- void **setProteinSpeciesPrefix** (std::string prefix)
- std::string **getProteinSpeciesPrefix** () const
- void **setRibosomeCountKey** (std::string key)
- std::string **getRibosomeCountKey** () const
- void **setRandomSeed** (unsigned int seed)
- unsigned int **getRandomSeed** () const
- int **getLastSynthesizedCount** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const

- new public user methods for this component*
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↔IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

- Sets the model nesting level for this definition.*
- `List< SimulationControl * > * getSimulationControls ()` const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific ()` const
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled ()` const
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation * GetPluginInformation ()`
- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `bool Check (ModelComponent *component)`
Validates a component instance.
- static `void CreateInternalData (ModelComponent *component)`
Creates internal data required by a component instance.
- static `void SaveInstance (PersistenceRecord *fields, ModelComponent *component)`
Serializes a component instance into a persistence record.
- static `void DispatchEvent (Event *event)`
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition * LoadInstance (Model *model, PersistenceRecord *fields, bool insertIntoModel)`
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition * NewInstance (Model *model, std::string name="")`
Creates a new instance of a data definition using the plugin registry.
- static `void SaveInstance (PersistenceRecord *fields, ModelDataDefinition *modeldatum)`
Serializes a data definition instance into a persistence record.
- static `bool Check (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Validates a data definition instance.
- static `void InitBetweenReplications (ModelDataDefinition *modeldatum)`
Resets simulation-time state between replications.
- static `void CreateInternalData (ModelDataDefinition *modeldatum)`
Creates internal and attached data required by this instance.
- static `bool CreateRelatedDataElements (ModelDataDefinition *modeldatum, std::string &errorMessage)`
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
legacy helper methods used by the granular create hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()

- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.764.1 Detailed Description

Stochastic protein synthesis via Poisson tau-leaping for whole-cell simulations.

Models the *M. genitalium* Translation process (Karr et al. 2012): $\lambda_i = (\text{ribosomeElongationRate} / \text{meanProteinLength}) \times \text{mRNA_count}_i \times \text{ribosome_free} / \max(1, \text{total_mRNA}) \times \Delta t$ $N_{\text{protein}_i} \sim \text{Poisson}(\lambda_i)$

Each mRNA template (prefix `proteinSpeciesPrefix`) is matched to its corresponding mRNA species (prefix `mRNASpeciesPrefix`) in [WholeCellState](#). Free ribosome count is read from [WholeCellState](#) with key `ribosomeCountKey`.

Parameters loaded from [WholeCellState](#) (set by `loadParameters()`): `param.ribosomeElongationRate` — 16 AA/s (from `parameters.json`)

mRNA degradation is not modeled here; use a separate [StochasticReactionRule](#) for mRNA decay (first-order, rate $\sim 1/\text{mRNA_halflife}$).

10.764.2 Member Function Documentation

10.764.2.1 _check()

```
bool StochasticTranslation::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.764.2.2 _createEditableDataDefinitions()

```
void StochasticTranslation::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.764.2.3 _initBetweenReplications()

```
void StochasticTranslation::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.764.2.4 _loadInstance()

```
bool StochasticTranslation::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.764.2.5 _onDispatchEvent()

```
void StochasticTranslation::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.764.2.6 `_saveInstance()`

```
void StochasticTranslation::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.764.2.7 `show()`

```
std::string StochasticTranslation::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- `plugins/components/WholeCellModeling/StochasticTranslation.h`
- `plugins/components/WholeCellModeling/StochasticTranslation.cpp`

10.765 `BioReaction::StoichiometricTerm` Struct Reference

Public Attributes

- `std::string` **speciesName**
- `double` **stoichiometry** = 1.0

The documentation for this struct was generated from the following file:

- `plugins/data/BiochemicalSimulation/BioReaction.h`

10.766 MetabolicReaction::StoichiometricTerm Struct Reference

Public Attributes

- std::string **speciesName**
- double **stoichiometry** = 1.0

The documentation for this struct was generated from the following file:

- plugins/data/BiochemicalSimulation/MetabolicReaction.h

10.767 StochasticReactionRule::StoichiometricTerm Struct Reference

Public Attributes

- std::string **speciesName**
- int **stoichiometry** = 1

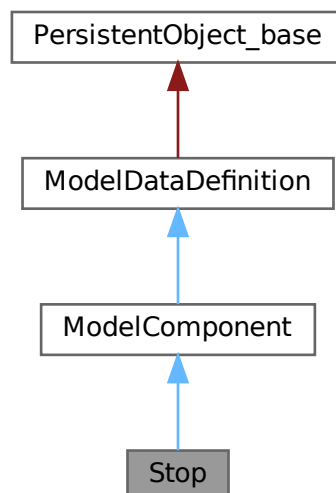
The documentation for this struct was generated from the following file:

- plugins/data/WholeCellModeling/StochasticReactionRule.h

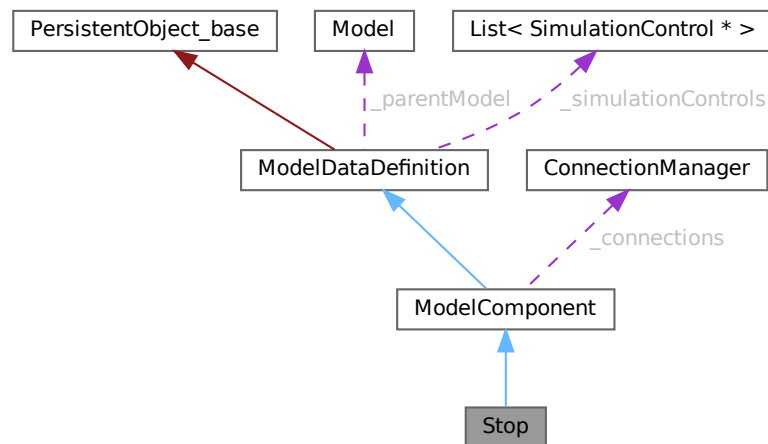
10.768 Stop Class Reference

```
#include <plugins/components/MaterialHandling/Stop.h>
```

Inheritance diagram for Stop:



Collaboration diagram for Stop:



Public Member Functions

- **Stop** ([Model](#) *model, std::string name="")
 - virtual std::string [show](#) () override
- virtual public methods*

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.768.1 Detailed Description

[Stop](#) module DESCRIPTION The [Stop](#) module sets the operational status of a conveyor to inactive. The conveyor may have been activated from either the [Start](#) module or by initially being set to active at the start of the simulation. When the entity enters the [Stop](#) module, the conveyor will stop immediately, regardless of the type of conveyor or the number of entities currently on the conveyor. TYPICAL USES [Stop](#) a baggage conveyor after a pre-determined amount of time [Stop](#) a conveyor for scheduled maintenance PROMPTS Prompt Description Name Unique name of the module that will be displayed in the flowchart. Conveyor Name Name of the conveyor to stop.

10.768.2 Member Function Documentation

10.768.2.1 `_check()`

```
bool Stop::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.768.2.2 `_loadInstance()`

```
bool Stop::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.768.2.3 `_onDispatchEvent()`

```
void Stop::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.768.2.4 _saveInstance()

```
void Stop::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.768.2.5 show()

```
std::string Stop::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

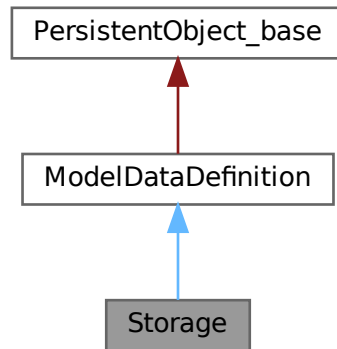
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

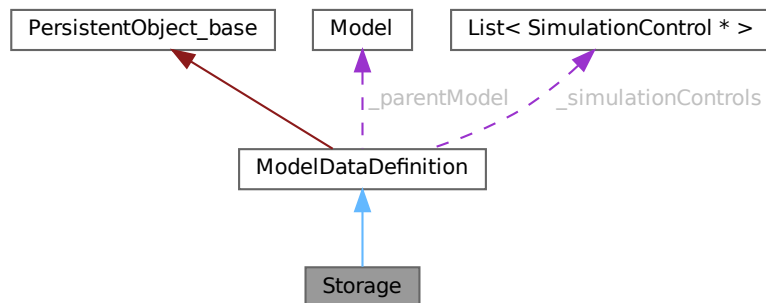
- plugins/components/MaterialHandling/Stop.h
- plugins/components/MaterialHandling/Stop.cpp

10.769 Storage Class Reference

Inheritance diagram for Storage:



Collaboration diagram for Storage:



Public Member Functions

- **Storage** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- void **setTotalArea** (double _totalArea)
- double **getTotalArea** () const
- void **setCapacity** (unsigned int _capacity)
- unsigned int **getCapacity** () const
- void **setUnitsPerArea** (double _unitsPerArea)
- double **getUnitsPerArea** () const

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual void **_initBetweenReplications** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createNonEditableDataDefinitions** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()

- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.769.1 Member Function Documentation

10.769.1.1 `_check()`

```
bool Storage::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.769.1.2 `_getParserChangesInformation()`

```
ParserChangesInformation * Storage::_getParserChangesInformation ( ) [override], [protected],
[virtual]
```

This method returns all changes in the parser that are needed by plugins of this ModelDatas. When connecting a new plugin, [ParserChangesInformation](#) are used to change parser source code, which is after compiled and dinamically linked to to simulator kernel to reflect the changes

Reimplemented from [ModelDataDefinition](#).

10.769.1.3 `_loadInstance()`

```
bool Storage::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.769.1.4 `_saveInstance()`

```
void Storage::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Persist [Storage](#) parameters using symmetric keys from `_loadInstance`.

Reimplemented from [ModelDataDefinition](#).

10.769.1.5 show()

```
std::string Storage::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

Reimplemented from [ModelDataDefinition](#).

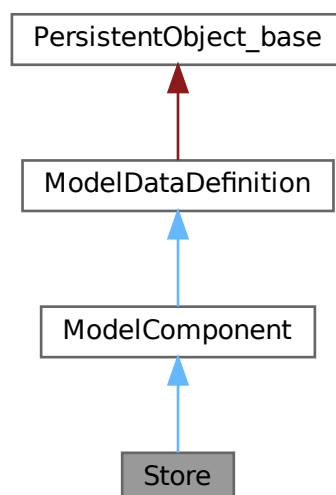
The documentation for this class was generated from the following files:

- plugins/data/MaterialHandling/Storage.h
- plugins/data/MaterialHandling/Storage.cpp

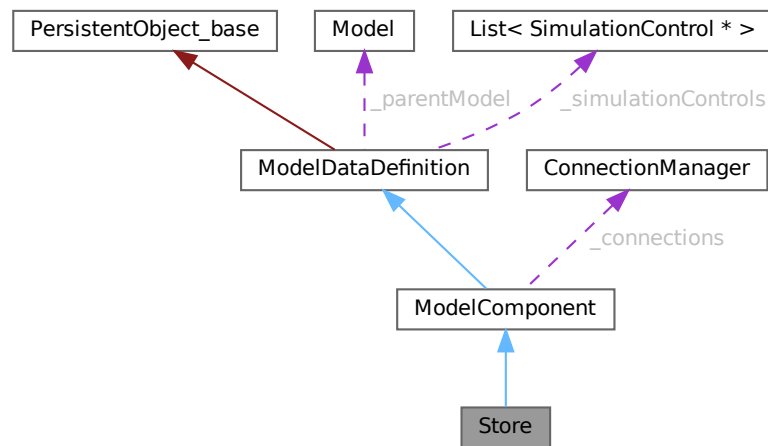
10.770 Store Class Reference

```
#include <plugins/components/MaterialHandling/Store.h>
```

Inheritance diagram for Store:



Collaboration diagram for Store:



Public Member Functions

- **Store** ([Model](#) *model, std::string name="")
 - virtual std::string [show](#) () override
- virtual public methods*

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.770.1 Detailed Description

[Store](#) module DESCRIPTION The [Store](#) module adds an entity to storage. The [Unstore](#) module may then be used to remove the entity from the storage. When an entity arrives at the [Store](#) module, the storage specified is incremented, and the entity immediately moves to the next module in the model. Storages are useful for displaying entity animation while an entity undergoes processing in other modules. Additionally, statistics may be kept on the number of entities in storage. TYPICAL USES Animating a part through a number of delay operations (load, setup, process, unload) Tracking the number of customers within a grocery store (place in storage upon entry) PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Type Method of specifying the storage name as a [Storage](#), [Set](#), [Attribute](#), or Expression. [Storage](#) Name Name of the storage to which the entity will be added. Applies only when the Type is [Storage](#). [Set](#) Name Name of the storage set from which the storage is to be selected. Applies only when the Type is [Set](#). [Set](#) Index Index into the defined storage set that contains the desired storage name. Applies only when the Type is [Set](#). [Attribute](#) Name of the attribute whose value contains the storage. Applies only when the Type is [Attribute](#). Expression Expression that is evaluated to the storage into which the entity is placed. Applies only when the Type is Expression.

10.770.2 Member Function Documentation

10.770.2.1 `_check()`

```
bool Store::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Validate minimal consistency for a pass-through [Store](#) component.

Reimplemented from [ModelDataDefinition](#).

10.770.2.2 `_loadInstance()`

```
bool Store::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Load Store-specific persistent fields.

[Store](#) currently has no extra persisted attributes beyond [ModelComponent](#). Keep the template commands below as a guide for future extensions.

Reimplemented from [ModelComponent](#).

10.770.2.3 `_onDispatchEvent()`

```
void Store::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.770.2.4 `_saveInstance()`

```
void Store::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Save Store-specific persistent fields.

Reimplemented from [ModelComponent](#).

10.770.2.5 `show()`

```
std::string Store::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

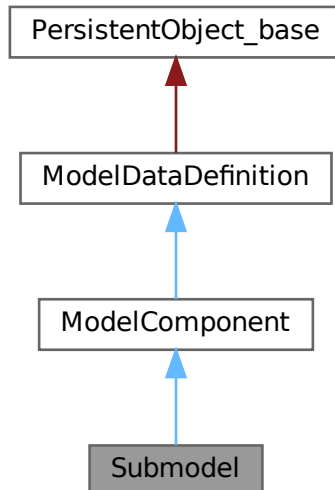
The documentation for this class was generated from the following files:

- `plugins/components/MaterialHandling/Store.h`
- `plugins/components/MaterialHandling/Store.cpp`

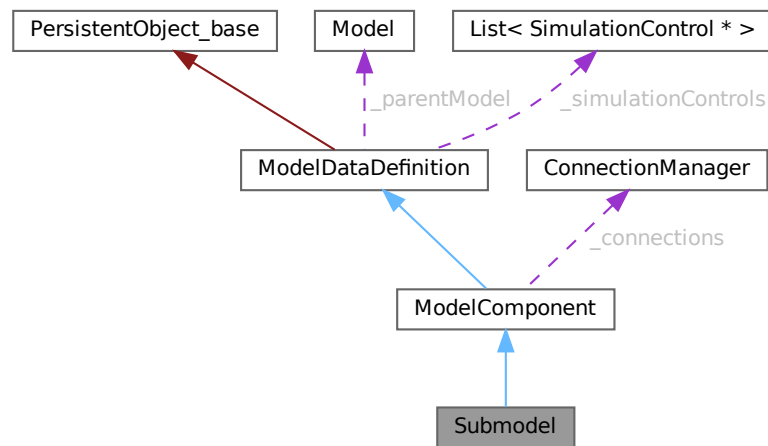
10.771 Submodel Class Reference

```
#include <plugins/components/ModalModel/Submodel.h>
```

Inheritance diagram for Submodel:



Collaboration diagram for Submodel:



Public Member Functions

- **Submodel** ([Model](#) *model, std::string name="")
 - virtual std::string [show](#) () override
- virtual public methods*

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual [~ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.771.1 Detailed Description

This component ...

10.771.2 Member Function Documentation

10.771.2.1 `_check()`

```
bool Submodel::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.771.2.2 `_loadInstance()`

```
bool Submodel::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.771.2.3 `_onDispatchEvent()`

```
void Submodel::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.771.2.4 _saveInstance()

```
void Submodel::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.771.2.5 show()

```
std::string Submodel::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

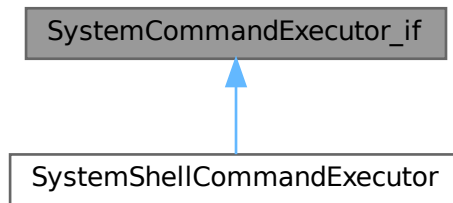
- plugins/components/ModalModel/Submodel.h
- plugins/components/ModalModel/Submodel.cpp

10.772 SystemCommandExecutor_if Class Reference

Minimal command execution interface used to make system dependency checks testable.

```
#include <kernel/simulator/SystemDependencyResolver.h>
```

Inheritance diagram for SystemCommandExecutor_if:



Public Member Functions

- virtual `~SystemCommandExecutor_if()`=default
Destroys the command executor abstraction.
- virtual `SystemCommandResult run` (const std::string &command)=0
Runs a shell command and returns its observable result.

10.772.1 Detailed Description

Minimal command execution interface used to make system dependency checks testable.

10.772.2 Member Function Documentation

10.772.2.1 run()

```
virtual SystemCommandResult SystemCommandExecutor_if::run (  
    const std::string & command ) [pure virtual]
```

Runs a shell command and returns its observable result.

Implemented in [SystemShellCommandExecutor](#).

The documentation for this class was generated from the following file:

- kernel/simulator/SystemDependencyResolver.h

10.773 SystemCommandResult Struct Reference

Captures the result of executing a system command used by dependency checks.

```
#include <kernel/simulator/SystemDependencyResolver.h>
```

Public Member Functions

- bool **succeeded** () const
Returns true when the command started and exited successfully.

Public Attributes

- bool **started** = false
True when the command process was started by the executor.
- int **exitCode** = -1
Process exit code, or -1 when the command could not be started.
- std::string **output** = ""
Combined standard output and diagnostic text produced by the command.
- std::string **errorMessage** = ""
Internal executor error message, when starting or collecting the command failed.

10.773.1 Detailed Description

Captures the result of executing a system command used by dependency checks.

The documentation for this struct was generated from the following files:

- kernel/simulator/SystemDependencyResolver.h
- kernel/simulator/SystemDependencyResolver.cpp

10.774 SystemDependency Class Reference

Public Types

- enum class **OS** {
 Any , **Linux** , **Windows** , **MacOS** ,
 Unknown }

Public Member Functions

- **SystemDependency** (OS os, std::string name, std::string installCommand="", std::string checkCommand="")
- OS **getOS** () const
- std::string **getName** () const
- std::string **getInstallCommand** () const
- std::string **getCheckCommand** () const
- std::string **show** () const

Static Public Member Functions

- static std::string **osToString** (OS os)

The documentation for this class was generated from the following files:

- kernel/simulator/PluginInformation.h
- kernel/simulator/PluginInformation.cpp

10.775 SystemDependencyCheckEntry Class Reference

One dependency entry produced by a system dependency preflight.

```
#include <kernel/simulator/SystemDependencyResolver.h>
```

Public Types

- enum class **Status** { **Satisfied** , **Missing** , **NotVerifiable** , **IgnoredDifferentOS** }
Classification assigned to one declared dependency after preflight.

Public Member Functions

- **SystemDependencyCheckEntry** (**SystemDependency** dependency, **Status** status, **SystemCommandResult** checkResult={}, std::string message="")
Builds a preflight entry for one declared dependency.
- const **SystemDependency** & **dependency** () const
Returns the dependency declaration evaluated by this entry.
- **Status** **status** () const
Returns the preflight status for this dependency.
- const **SystemCommandResult** & **checkResult** () const
Returns the result of the check command, when one was executed.
- const std::string & **message** () const
Returns human-readable diagnostic context for this entry.
- bool **blocksInsertion** () const
Returns true when this entry should prevent plugin insertion.
- bool **canAttemptInstall** () const
Returns true when this missing dependency has an installation command.
- std::string **diagnosticText** () const
Returns a multi-line diagnostic with check/install commands and command results.

Static Public Member Functions

- static std::string **statusToString** (**Status** status)
Converts a status value into a stable textual representation.

10.775.1 Detailed Description

One dependency entry produced by a system dependency preflight.

The documentation for this class was generated from the following files:

- kernel/simulator/SystemDependencyResolver.h
- kernel/simulator/SystemDependencyResolver.cpp

10.776 SystemDependencyCheckResult Class Reference

Full preflight result for the system dependencies declared by one plugin.

```
#include <kernel/simulator/SystemDependencyResolver.h>
```

Public Member Functions

- void **add** ([SystemDependencyCheckEntry](#) entry)
Adds one dependency evaluation entry.
- const std::list< [SystemDependencyCheckEntry](#) > & **entries** () const
Returns every evaluated dependency entry, including ignored entries.
- bool **canInsertPlugin** () const
Returns true when no applicable dependency blocks insertion.
- bool **hasBlockingEntries** () const
Returns true when at least one dependency blocks insertion.
- bool **canAttemptInstallForAllMissing** () const
Returns true when every blocking dependency has an installation command.
- std::string **summary** () const
Returns a compact diagnostic summary suitable for trace logs.
- std::string **diagnosticText** (bool includeSatisfied=false) const
Returns a multi-line diagnostic suitable for terminal, GUI and trace details.

10.776.1 Detailed Description

Full preflight result for the system dependencies declared by one plugin.

The documentation for this class was generated from the following files:

- kernel/simulator/SystemDependencyResolver.h
- kernel/simulator/SystemDependencyResolver.cpp

10.777 SystemDependencyInstallEntry Class Reference

One command result from an attempted system dependency installation.

```
#include <kernel/simulator/SystemDependencyResolver.h>
```

Public Member Functions

- **SystemDependencyInstallEntry** ([SystemDependency](#) dependency, [SystemCommandResult](#) installResult)
Builds an installation result for a single dependency.
- const [SystemDependency](#) & **dependency** () const
Returns the dependency whose install command was executed.
- const [SystemCommandResult](#) & **installResult** () const
Returns the result produced by the install command.
- std::string **diagnosticText** () const
Returns command, exit code and captured output for trace or GUI feedback.

10.777.1 Detailed Description

One command result from an attempted system dependency installation.

The documentation for this class was generated from the following files:

- kernel/simulator/SystemDependencyResolver.h
- kernel/simulator/SystemDependencyResolver.cpp

10.778 SystemDependencyInstallResult Class Reference

Aggregates commands executed while trying to satisfy missing system dependencies.

```
#include <kernel/simulator/SystemDependencyResolver.h>
```

Public Member Functions

- void **add** ([SystemDependencyInstallEntry](#) entry)
Adds one installation command result.
- const std::list< [SystemDependencyInstallEntry](#) > & **entries** () const
Returns all attempted installation command results.
- bool **succeeded** () const
Returns true when every attempted installation command succeeded.
- std::string **summary** () const
Returns a compact diagnostic summary suitable for trace logs.
- std::string **diagnosticText** () const
Returns detailed diagnostic text for every executed install command.

10.778.1 Detailed Description

Aggregates commands executed while trying to satisfy missing system dependencies.

The documentation for this class was generated from the following files:

- kernel/simulator/SystemDependencyResolver.h
- kernel/simulator/SystemDependencyResolver.cpp

10.779 SystemDependencyResolver Class Reference

Evaluates and optionally installs plugin system dependencies.

```
#include <kernel/simulator/SystemDependencyResolver.h>
```

Static Public Member Functions

- static SystemDependency::OS **currentOS** ()
Returns the operating system represented by the current executable.
- static bool **appliesToCurrentOS** (const SystemDependency &dependency)
Returns true when a dependency declaration applies to the current OS.
- static SystemDependencyCheckResult **evaluate** (const std::list< SystemDependency > *dependencies, SystemCommandExecutor_if &executor)
Evaluates all dependency declarations using check commands when available.
- static SystemDependencyInstallResult **installMissingDependencies** (const SystemDependencyCheckResult &checkResult, SystemCommandExecutor_if &executor)
Executes install commands for missing dependencies from a preflight result.

10.779.1 Detailed Description

Evaluates and optionally installs plugin system dependencies.

The resolver deliberately contains no user-interface code. It only decides whether dependencies apply to the current OS, runs declared check/install commands through an executor, and reports diagnostics to the caller.

The documentation for this class was generated from the following files:

- kernel/simulator/SystemDependencyResolver.h
- kernel/simulator/SystemDependencyResolver.cpp

10.780 SystemPreferences Class Reference

Persistent application preferences used by the Qt GUI.

```
#include <applications/gui/genesys/systempreferences.h>
```

Public Types

- enum class StartupModelMode : unsigned short { **NoModel** = 0 , **NewModel** = 1 , **OpenSpecificModel** = 2 , **OpenLastModel** = 3 }
Startup model selection remembered in the preferences file.
- enum class VisualTheme { **Light** , **Dark** }
Global visual style used by the GUI.
- enum class InterfaceStyle { **Classic** , **Modern** }
Interface density/style preset used by the Qt application.
- enum class AutomaticPositioningStrategy { **Legacy** = 0 , **Centered** = 1 }
Automatic positioning policy for graphical elements.

Static Public Member Functions

- static bool [load](#) ()
Loads preferences from the configuration file.
- static bool [save](#) ()
Saves the current preferences to disk.
- static void **resetToDefaults** ()
Restores every preference to its built-in default value.
- static QString **configFilePath** ()
Returns the full path to the active preferences file.
- static bool **startMaximized** ()
Returns whether the application should start maximized.
- static void **setStartMaximized** (bool newStartMaximized)
Updates the start-maximized preference.
- static bool **autoLoadPlugins** ()
Returns whether plug-ins are loaded automatically at startup.
- static void **setAutoLoadPlugins** (bool newAutoLoadPlugins)
Updates the automatic plug-in loading preference.
- static [StartupModelMode](#) **startupModelMode** ()
Returns the startup model mode.
- static void **setStartupModelMode** ([StartupModelMode](#) newStartupModelMode)
Updates the startup model mode.
- static unsigned short **modelAtStart** ()
Returns the configured model index for the startup flow.
- static void **setModelAtStart** (unsigned short newModelAtStart)
Updates the configured model index for the startup flow.
- static std::string **modelfilename** ()
Returns the filename requested for startup loading.
- static void **setModelfilename** (const std::string &newModelfilename)
Updates the filename requested for startup loading.
- static std::string **lastModelFilename** ()
Returns the last model filename successfully opened by the GUI.
- static void **setLastModelFilename** (const std::string &newLastModelFilename)
Updates the last model filename successfully opened by the GUI.
- static std::vector< std::string > **recentModelFiles** ()
Returns the list of recently opened model files.
- static void **setRecentModelFiles** (const std::vector< std::string > &[recentModelFiles](#))
Replaces the recent model file list.
- static void [pushRecentModelFile](#) (const std::string &modelFilename)
Pushes a file to the recent model list with limit handling.
- static unsigned int **recentModelFilesLimit** ()
Returns the maximum number of recent model entries to keep.
- static void **setRecentModelFilesLimit** (unsigned int [recentModelFilesLimit](#))
Updates the maximum number of recent model entries.
- static bool **checkSystemPackagesAtStart** ()
Returns whether system packages should be checked at startup.
- static void **setCheckSystemPackagesAtStart** (bool newCheckSystemPackagesAtStart)
Updates the system-package check preference.
- static TraceManager::Level **traceLevel** ()
Returns the current trace verbosity level.
- static void **setTraceLevel** (TraceManager::Level newTraceLevel)

- Updates the trace verbosity level.*
- static [VisualTheme](#) **visualTheme** ()
Returns the selected visual theme.
- static void **setVisualTheme** ([VisualTheme](#) newVisualTheme)
Updates the selected visual theme.
- static std::string **visualThemeName** ()
Returns the persisted name of the selected visual theme.
- static [InterfaceStyle](#) **interfaceStyle** ()
Returns the selected interface style preset.
- static void **setInterfaceStyle** ([InterfaceStyle](#) newInterfaceStyle)
Updates the selected interface style preset.
- static std::string **interfaceStyleName** ()
Returns the persisted name of the selected interface style.
- static int **applicationFontSize** ()
Returns the application font size used by the GUI.
- static void **setApplicationFontSize** (int newApplicationFontSize)
Updates the application font size used by the GUI.
- static bool **diagramUsesThemeColors** ()
Returns whether diagram widgets should derive colors from the theme.
- static void **setDiagramUsesThemeColors** (bool newDiagramUsesThemeColors)
Updates whether diagram widgets should derive colors from the theme.
- static [AutomaticPositioningStrategy](#) **automaticPositioningStrategy** ()
Returns the automatic placement strategy used by the diagram canvas.
- static void **setAutomaticPositioningStrategy** ([AutomaticPositioningStrategy](#) strategy)
Updates the automatic placement strategy used by the diagram canvas.
- static unsigned int **canvasBackgroundColor** ()
Returns the RGB color used for the main canvas background.
- static unsigned int **canvasDisabledBackgroundColor** ()
Returns the RGB color used for the disabled canvas background.
- static unsigned int **gridColor** ()
Returns the RGB color used for the diagram grid.

10.780.1 Detailed Description

Persistent application preferences used by the Qt GUI.

The class behaves like a singleton configuration store backed by a local preferences file. All access is static so the GUI can query and update the current runtime settings without manually passing a preferences object around.

10.780.2 Member Function Documentation

10.780.2.1 load()

```
bool SystemPreferences::load ( ) [static]
```

Loads preferences from the configuration file.

Loads GUI preferences from disk and normalizes invalid data.

10.780.2.2 pushRecentModelFile()

```
void SystemPreferences::pushRecentModelFile (
    const std::string & modelName ) [static]
```

Pushes a file to the recent model list with limit handling.

Pushes a model file to the top of the recent-file list.

10.780.2.3 save()

```
bool SystemPreferences::save ( ) [static]
```

Saves the current preferences to disk.

Serializes the current GUI preferences to the preferences file.

The documentation for this class was generated from the following files:

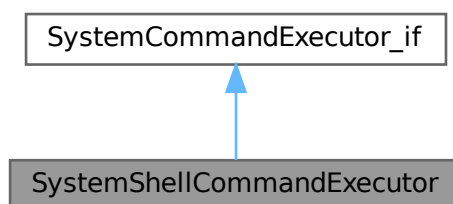
- applications/gui/genesys/systempreferences.h
- applications/gui/genesys/systempreferences.cpp

10.781 SystemShellCommandExecutor Class Reference

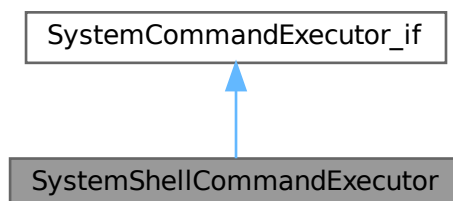
Shell-based executor used by production plugin dependency preflight.

```
#include <kernel/simulator/SystemDependencyResolver.h>
```

Inheritance diagram for SystemShellCommandExecutor:



Collaboration diagram for SystemShellCommandExecutor:



Public Member Functions

- [SystemCommandResult](#) **run** (const std::string &command) override
Executes a command through the host shell and captures combined output.

Public Member Functions inherited from [SystemCommandExecutor_if](#)

- virtual **~SystemCommandExecutor_if** ()=default
Destroys the command executor abstraction.

10.781.1 Detailed Description

Shell-based executor used by production plugin dependency preflight.

10.781.2 Member Function Documentation

10.781.2.1 run()

```
SystemCommandResult SystemShellCommandExecutor::run (
    const std::string & command ) [override], [virtual]
```

Executes a command through the host shell and captures combined output.

Implements [SystemCommandExecutor_if](#).

The documentation for this class was generated from the following files:

- kernel/simulator/SystemDependencyResolver.h
- kernel/simulator/SystemDependencyResolver.cpp

10.782 QCPAxisPainterPrivate::TickLabelData Struct Reference

Public Attributes

- QString **basePart**
- QString **expPart**
- QString **suffixPart**
- QRect **baseBounds**
- QRect **expBounds**
- QRect **suffixBounds**
- QRect **totalBounds**
- QRect **rotatedTotalBounds**
- QFont **baseFont**
- QFont **expFont**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/qcustomplot.h

10.783 Token Struct Reference

Public Types

- enum **Kind** {
eof , **null** , **f** , **t** ,
number , **string** , **lbracket** , **rbracket** ,
lbrace , **rbrace** , **comma** , **colon** }

Public Attributes

- enum Token::Kind **kind**
- std::string **str** {}

The documentation for this struct was generated from the following file:

- kernel/simulator/persistence/JsonSerializer.cpp

10.784 TokenService Class Reference

Public Member Functions

- std::string **generateSessionId** ()
- std::string **generateAccessToken** ()

The documentation for this class was generated from the following files:

- applications/worker/auth/TokenService.h
- applications/worker/auth/TokenService.cpp

10.785 TraceConsoleController Class Reference

Controller that routes simulator trace events to GUI text widgets.

```
#include <applications/gui/genesys/controllers/TraceConsoleController.h>
```

Public Member Functions

- TraceConsoleController** (QTextEdit *console, QTextEdit *simulationText, QTextEdit *reportsText, std::function< bool()> traceWidgetsEnabled={})
Creates the trace-output bridge used by [MainWindow](#) trace wrappers.
- void **simulatorTraceHandler** ([TraceEvent](#) e) const
Renders generic traces to the main console pane.
- void **simulatorTraceErrorHandler** ([TraceErrorEvent](#) e) const
Renders error traces to the main console pane.
- void **simulatorTraceSimulationHandler** ([TraceSimulationEvent](#) e) const
Renders simulation traces to the simulation output pane.
- void **simulatorTraceReportsHandler** ([TraceEvent](#) e) const
Renders reports traces to the reports output pane.

10.785.1 Detailed Description

Controller that routes simulator trace events to GUI text widgets.

This controller is the Phase-4 extraction for trace presentation. [MainWindow](#) keeps compatibility handlers registered in the kernel trace manager and delegates rendering to this class, reducing direct UI formatting logic in the façade.

Responsibilities:

- render regular/error traces to the main console pane;
- render simulation traces to the simulation output pane;
- render report traces to the reports pane.

Boundaries:

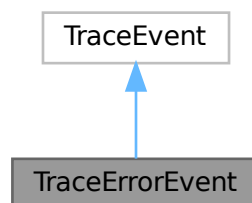
- it does not subscribe handlers by itself (registration stays in MainWindow/event layer);
- it does not change simulation flow, model state, or controller orchestration;
- it acts as a UI bridge, not as a domain service.

The documentation for this class was generated from the following files:

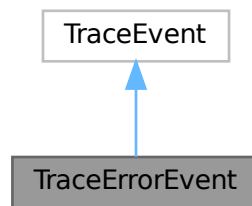
- applications/gui/genesys/controllers/TraceConsoleController.h
- applications/gui/genesys/controllers/TraceConsoleController.cpp

10.786 TraceErrorEvent Class Reference

Inheritance diagram for TraceErrorEvent:



Collaboration diagram for TraceErrorEvent:



Public Member Functions

- **TraceErrorEvent** (const std::string &text, const std::exception &e)
- **TraceErrorEvent** (const std::string &text, TraceManager::Level level)
- const std::string & **getExceptionMessage** () const

Public Member Functions inherited from [TraceEvent](#)

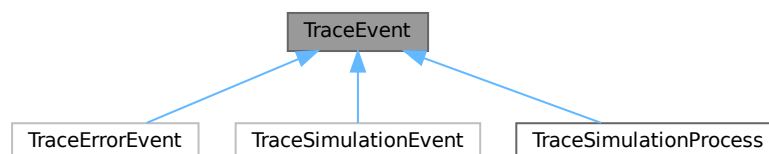
- **TraceEvent** (TraceManager::Level level, const std::string &text)
- **TraceEvent** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- TraceManager::Level **getTracelevel** () const
- const std::string & **getText** () const

The documentation for this class was generated from the following file:

- kernel/simulator/TraceManager.h

10.787 TraceEvent Class Reference

Inheritance diagram for TraceEvent:



Public Member Functions

- **TraceEvent** (TraceManager::Level level, const std::string &text)
- **TraceEvent** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- TraceManager::Level **getTracelevel** () const
- const std::string & **getText** () const

The documentation for this class was generated from the following file:

- kernel/simulator/TraceManager.h

10.788 TraceManager Class Reference

```
#include <kernel/simulator/TraceManager.h>
```

Public Types

- enum class **Level** : int {
L0_noTraces = 0 , **L1_errorFatal** = 1 , **L2_results** = 2 , **L3_errorRecover** = 3 ,
L4_warning = 4 , **L5_event** = 5 , **L6_arrival** = 6 , **L7_internal** = 7 ,
L8_detailed = 8 , **L9_mostDetailed** = 9 , **num_elements** = 10 }

Public Member Functions

- **TraceManager** ([Simulator](#) *simulator)
- void [addTraceHandler](#) (traceListener traceListener)
addTraceHandler
- void [addTraceReportHandler](#) (traceListener traceReportListener)
addTraceReportHandler
- void [addTraceSimulationHandler](#) (traceSimulationListener traceSimulationListener)
addTraceSimulationHandler
- void [addTraceErrorHandler](#) (traceErrorListener traceErrorListener)
addTraceErrorHandler
- template<typename Class >
void [addTraceHandler](#) (Class *object, void(Class::*function)([TraceEvent](#)))
addTraceHandler
- template<typename Class >
void [addTraceErrorHandler](#) (Class *object, void(Class::*function)([TraceErrorEvent](#)))
addTraceErrorHandler
- template<typename Class >
void [addTraceReportHandler](#) (Class *object, void(Class::*function)([TraceEvent](#)))
addTraceReportHandler
- template<typename Class >
void [addTraceSimulationHandler](#) (Class *object, void(Class::*function)([TraceSimulationEvent](#)))
addTraceSimulationHandler
- void **addTraceSimulationExceptionRuleModelData** (void *thisobject)
- void **trace** (TraceManager::Level level, const std::string &text)
Deprecated.

- void **traceSimulation** (void *thisobject, TraceManager::Level level, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text)
Deprecated.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)
Deprecated.
- void **beginShutdown** ()
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Trace to a general output, used when not simulating (eg: adding plugins, checking the model, etc)
- void **traceError** (const std::string &text, const std::exception &e)
Trace to the error output, used in every situation an error happens (simulating, report, general)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
Trace to the error output, used in every situation an error happens (simulating, report, general)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
Trace to the report output, used only when generating the simulation report.
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed, bool showAnyway=false)
Trace to the simulation output, used only when simulation is running (eg: components or dataElements inform something)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed, bool showAnyway=false)
Trace to the simulation output, used only when simulation is running (eg: components or dataElements inform something)
- [List](#)< std::string > * [errorMessagees](#) () const
errorMessagees
- void [setTraceLevel](#) (TraceManager::Level _traceLevel)
setTraceLevel
- TraceManager::Level [getTraceLevel](#) () const
getTraceLevel
- [Simulator](#) * [getParentSimulator](#) () const
getParentSimulator
- void [setTraceSimulationRuleAllAllowed](#) (bool _traceSimulationRuleAllAllowed)
setTraceSimulationRuleAllAllowed
- bool [isTraceSimulationRuleAllAllowed](#) () const
isTraceSimulationRuleAllAllowed

Static Public Member Functions

- static std::string **convertEnumToStr** (Level level)

10.788.1 Detailed Description

The [TraceManager](#) is used to trace back model simulation information and track/debug the simulation. It works as the model simulation output (cout) and allows external methods to hook up such output as listeners.

10.788.2 Member Function Documentation

10.788.2.1 addTraceErrorHandler() [1/2]

```
template<typename Class >
void TraceManager::addTraceErrorHandler (
    Class * object,
    void(Class::*)(TraceErrorEvent) function )
```

addTraceErrorHandler

Parameters

<i>object</i>	Add a method to by notified/invoked then a <code>traceError</code> is called
---------------	--

10.788.2.2 addTraceErrorHandler() [2/2]

```
void TraceManager::addTraceErrorHandler (
    traceErrorListener traceErrorListener )
```

`addTraceErrorHandler`

Parameters

<i>traceErrorListener</i>	Add a function to by notified/invoked then a <code>traceError</code> is called
---------------------------	--

10.788.2.3 addTraceHandler() [1/2]

```
template<typename Class >
void TraceManager::addTraceHandler (
    Class * object,
    void(Class::*)(TraceEvent) function )
```

`addTraceHandler`

Parameters

<i>object</i>	Add a method to by notified/invoked then a trace is called
---------------	--

10.788.2.4 addTraceHandler() [2/2]

```
void TraceManager::addTraceHandler (
    traceListener traceListener )
```

`addTraceHandler`

Parameters

<i>traceListener</i>	Add a function to by notified/invoked then a trace is called
----------------------	--

10.788.2.5 addTraceReportHandler() [1/2]

```
template<typename Class >
void TraceManager::addTraceReportHandler (
    Class * object,
    void(Class::*)(TraceEvent) function )
```

`addTraceReportHandler`

Parameters

<i>object</i>	Add a method to by notified/invoked then a traceReport is called
---------------	--

10.788.2.6 addTraceReportHandler() [2/2]

```
void TraceManager::addTraceReportHandler (
    traceListener traceReportListener )
```

addTraceReportHandler

Parameters

<i>traceReportListener</i>	Add a function to by notified/invoked then a traceReport is called
----------------------------	--

10.788.2.7 addTraceSimulationHandler() [1/2]

```
template<typename Class >
void TraceManager::addTraceSimulationHandler (
    Class * object,
    void(Class::*)(TraceSimulationEvent) function )
```

addTraceSimulationHandler

Parameters

<i>object</i>	Add a method to by notified/invoked then a traceSimulation is called
---------------	--

10.788.2.8 addTraceSimulationHandler() [2/2]

```
void TraceManager::addTraceSimulationHandler (
    traceSimulationListener traceSimulationListener )
```

addTraceSimulationHandler

Parameters

<i>traceSimulationListener</i>	Add a function to by notified/invoked then a traceSimulation is called
--------------------------------	--

10.788.2.9 errorMessages()

```
List< std::string > * TraceManager::errorMessages ( ) const
```

errorMessages

Returns

10.788.2.10 getParentSimulator()

```
Simulator * TraceManager::getParentSimulator ( ) const
```

getParentSimulator

Returns

10.788.2.11 getTraceLevel()

```
TraceManager::Level TraceManager::getTraceLevel ( ) const
```

getTraceLevel

Returns

10.788.2.12 isTraceSimulationRuleAllAllowed()

```
bool TraceManager::isTraceSimulationRuleAllAllowed ( ) const
```

isTraceSimulationRuleAllAllowed

Returns

10.788.2.13 setTraceLevel()

```
void TraceManager::setTraceLevel (
    TraceManager::Level _traceLevel )
```

setTraceLevel

Parameters

<i>_traceLevel</i>	
--------------------	--

10.788.2.14 setTraceSimulationRuleAllAllowed()

```
void TraceManager::setTraceSimulationRuleAllAllowed (
    bool _traceSimulationRuleAllAllowed )
```

setTraceSimulationRuleAllAllowed

Parameters

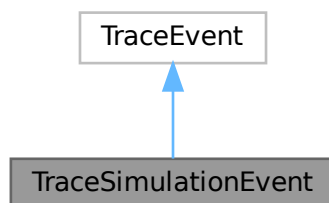
<code>_traceSimulationRuleAllAllowed</code>	
---	--

The documentation for this class was generated from the following files:

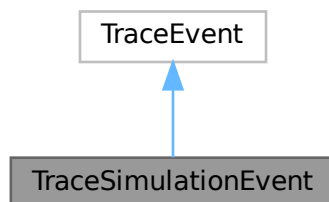
- kernel/simulator/TraceManager.h
- kernel/simulator/TraceManager.cpp

10.789 TraceSimulationEvent Class Reference

Inheritance diagram for TraceSimulationEvent:



Collaboration diagram for TraceSimulationEvent:



Public Member Functions

- **TraceSimulationEvent** (TraceManager::Level level, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text)
- [ModelComponent](#) * **getComponent** () const
- [Entity](#) * **getEntity** () const
- double **getTime** () const

Public Member Functions inherited from [TraceEvent](#)

- **TraceEvent** (TraceManager::Level level, const std::string &text)
- **TraceEvent** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- TraceManager::Level **getTracelevel** () const
- const std::string & **getText** () const

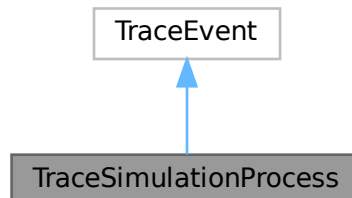
The documentation for this class was generated from the following file:

- kernel/simulator/TraceManager.h

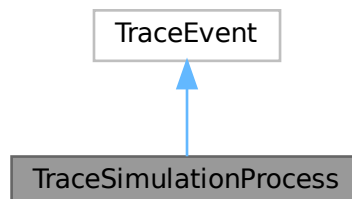
10.790 TraceSimulationProcess Class Reference

```
#include <kernel/simulator/TraceManager.h>
```

Inheritance diagram for TraceSimulationProcess:



Collaboration diagram for TraceSimulationProcess:



Public Member Functions

- **TraceSimulationProcess** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_↔ detailed)

Public Member Functions inherited from [TraceEvent](#)

- **TraceEvent** (TraceManager::Level level, const std::string &text)
- **TraceEvent** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- TraceManager::Level **getTracelevel** () const
- const std::string & **getText** () const

10.790.1 Detailed Description

Events related to simulation "process" (usually process analyser), associated to entire replication or simulation events (begin/end/pause of replication/simulation) @ToDo: (importante): CLASS **NOT** FULLY IMPLEMENTED (to be implemented for process analyser)

The documentation for this class was generated from the following file:

- kernel/simulator/TraceManager.h

10.791 TraitsGUI< T > Struct Template Reference

GUI-specific trait constants used to tune default rendering behavior.

```
#include <applications/gui/genesys/TraitsGUI.h>
```

10.791.1 Detailed Description

```
template<class T>
struct TraitsGUI< T >
```

GUI-specific trait constants used to tune default rendering behavior.

These specializations keep visual defaults close to the legacy Qt interface without scattering magic numbers across the graphics and editor code.

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

10.792 TraitsGUI< GAssociation > Struct Reference

Static Public Attributes

- static const uint64_t **foreColor** = 0x808080FF
- static constexpr const int **margin** = 2
- static constexpr const int **selectionWidth** = 8

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

10.793 TraitsGUI< GComponentPort > Struct Reference

Static Public Attributes

- static constexpr const int **width** = 20
- static constexpr const int **penWidth** = 1
- static constexpr const int **raise** = 3
- static constexpr const int **margin** = 2
- static const uint64_t **connectedBackgroundColor** = 0x2020FF80
- static const uint64_t **unconnectedBackgroundColor** = 0xFF202040
- static const uint64_t **borderColor** = 0x000000FF
- static const uint64_t **pathRaised** = 0x808080FF
- static const uint64_t **pathStunken** = 0xFFFFFFFF
- static const uint64_t **textColor** = 0xFFFFFFFF
- static const uint64_t **textShadowColor** = 0x000000FF
- static const uint64_t **selectionSquaresColor** = 0x000000FF

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

10.794 TraitsGUI< GConnection > Struct Reference

Static Public Attributes

- static const uint64_t **foreColor** = 0x000000FF
- static constexpr const int **margin** = 2
- static constexpr const int **selectionWidth** = 8

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

10.795 TraitsGUI< GMainWindow > Struct Reference

Static Public Attributes

- static const bool **startMaximized** = true
- static const int **zoomButtonChange** = 30
- static const uint64_t **consoleTextColor** = 0x008000FF

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

10.796 TraitsGUI< GModelComponent > Struct Reference

Static Public Attributes

- static constexpr const int **opacity** = 256.0 * 0.67
- static constexpr const int **colorIncrease** = 256.0 * 0.2
- static constexpr const int **lighter** = 133
- static constexpr const int **width** = 150
- static constexpr const int **penWidth** = 1
- static constexpr const int **raise** = 5
- static constexpr const int **margin** = 8
- static constexpr const int **selectionWidth** = 8
- static constexpr const float **stretchPos** = 0.5
- static constexpr const float **stretch** = 0.0
- static constexpr const float **heightProportion** = 0.67
- static const uint64_t **breakpointColor** = 0xFF0000FF
- static const uint64_t **borderColor** = 0x000000FF
- static const uint64_t **pathRaised** = 0xFFFFFFFF
- static const uint64_t **pathStunken** = 0x808080FF
- static const uint64_t **textColor** = 0xFFFFFFFF
- static const uint64_t **textShadowColor** = 0x000000FF
- static const uint64_t **selectionSquaresColor** = 0x000000FF

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

10.797 TraitsGUI< GModelDataDefinition > Struct Reference

Static Public Attributes

- static constexpr const int **opacity** = 256 * 0.33
- static constexpr const int **nonEditableOpacity** = 58
- static constexpr const int **selectedNonEditableOpacity** = 72
- static constexpr const int **width** = 100
- static constexpr const int **penWidth** = 1
- static constexpr const int **raise** = 3
- static constexpr const int **margin** = 5
- static constexpr const int **selectionWidth** = 8
- static constexpr const float **stretchPos** = 0.5
- static constexpr const float **stretch** = 0.0
- static constexpr const float **heightProportion** = 0.50
- static const uint64_t **breakpointColor** = 0xFF0000FF
- static const uint64_t **borderColor** = 0x000000FF
- static const uint64_t **pathRaised** = 0x808080FF
- static const uint64_t **pathStunken** = 0xFFFFFFFF
- static const uint64_t **textColor** = 0x0
- static const uint64_t **textShadowColor** = 0x000000FF
- static const uint64_t **selectionSquaresColor** = 0x000000FF

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

10.798 TraitsGUI< GModelGraphicPic > Struct Reference

Static Public Attributes

- static const int **nodeComponentFontSize** = 12
- static const uint64_t **nodeComponentFontColor** = 0x000000FF
- static const uint64_t **nodeComponentFillColor** = 0x80A040FF
- static const int **edgeComponentFontSize** = 7
- static const uint64_t **edgeComponentEdgeColor** = 0x000000FF
- static const uint64_t **edgeComponentFontColor** = 0x000000FF
- static const int **nodeDatadefInternalFontSize** = 8
- static const uint64_t **nodeDatadefInternalColor** = 0xA0A0A0FF
- static const uint64_t **nodeDatadefInternalFontColor** = 0xA0A0A0FF
- static const int **nodeDatadefAttachedFontSize** = 8
- static const uint64_t **nodeDatadefAttachedFontColor** = 0x808080FF

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

10.799 TraitsGUI< GScene > Struct Reference

Static Public Attributes

- static constexpr const int **gridInterval** = 20
- static constexpr const int **gridPenWidth** = 1
- static const uint64_t **gridColor** = 0xC0C0C0E0

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

10.800 TraitsGUI< GView > Struct Reference

Static Public Attributes

- static constexpr const int **sceneCenter** = 25000
- static constexpr const int **sceneDistanceCenter** = 25000
- static const uint64_t **backgroundEnabledColor** = 0xFFFFF8040
- static const uint64_t **backgroundDisabledColor** = 0xC0C0C020

The documentation for this struct was generated from the following file:

- applications/gui/genesys/TraitsGUI.h

10.801 TraitsKernel< T > Struct Template Reference

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

10.802 TraitsKernel< Collector_if > Struct Reference

Public Types

- typedef [CollectorDatafileDefaultImpl1](#) **Implementation**
- typedef double **DataType**

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

10.803 TraitsKernel< Model > Struct Reference

Public Types

- typedef [StatisticsDefaultImpl1](#) **StatisticsCollector_StatisticsImplementation**
- typedef [CollectorDefaultImpl1](#) **StatisticsCollector_CollectorImplementation**

Static Public Attributes

- static const TraceManager::Level **traceLevel** = TraceManager::Level::L5_event

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

10.804 TraitsKernel< ModelChecker_if > Struct Reference

Public Types

- typedef [ModelCheckerDefaultImpl1](#) **Implementation**

Static Public Attributes

- static const TraceManager::Level **traceLevel** = TraceManager::Level::L2_results

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

10.805 TraitsKernel< ModelComponent > Struct Reference

Static Public Attributes

- static constexpr bool **reportStatistics** = true
- static const TraceManager::Level **traceLevel** = TraceManager::Level::L2_results

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

10.806 TraitsKernel< ModelDataDefinition > Struct Reference

Static Public Attributes

- static constexpr bool **reportStatistics** = true
- static const TraceManager::Level **traceLevel** = TraceManager::Level::L2_results

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

10.807 TraitsKernel< Parser_if > Struct Reference

Public Types

- typedef [ParserDefaultImpl2](#) **Implementation**

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

10.808 TraitsKernel< Persistence_if > Struct Reference

Public Types

- typedef [PersistencePartialLoadImpl1](#) **Implementation**

Static Public Attributes

- static const TraceManager::Level **traceLevel** = TraceManager::Level::L2_results

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

10.809 TraitsKernel< PluginConnector_if > Struct Reference

Public Types

- typedef [PluginConnectorDummyImpl1](#) **Implementation**

Static Public Attributes

- static const `TraceManager::Level` **traceLevel** = `TraceManager::Level::L4_warning`

The documentation for this struct was generated from the following file:

- `kernel/TraitsKernel.h`

10.810 TraitsKernel< Sampler_if > Struct Reference

Public Types

- typedef [SamplerDefaultImpl1](#) **Implementation**
- typedef [SamplerDefaultImpl1::DefaultImpl1RNG_Parameters](#) **Parameters**

The documentation for this struct was generated from the following file:

- `kernel/TraitsKernel.h`

10.811 TraitsKernel< SimulationReporter_if > Struct Reference

Public Types

- typedef [SimulationReporterDefaultImpl1](#) **Implementation**
- typedef [Counter](#) **CounterImplementation**

Static Public Attributes

- static const `TraceManager::Level` **traceLevel** = `TraceManager::Level::L2_results`

The documentation for this struct was generated from the following file:

- `kernel/TraitsKernel.h`

10.812 TraitsKernel< Simulator > Struct Reference

Static Public Attributes

- static const `TraceManager::Level` **traceLevel** = `TraceManager::Level::L9_mostDetailed`

The documentation for this struct was generated from the following file:

- `kernel/TraitsKernel.h`

10.813 TraitsKernel< Statistics_if > Struct Reference

Public Types

- typedef double **DataType**

Static Public Attributes

- static constexpr double **SignificanceLevel** = 0.05

The documentation for this struct was generated from the following file:

- kernel/TraitsKernel.h

10.814 TraitsKernel< StatisticsDatafile_if > Struct Reference

Public Types

- typedef [StatisticsDatafileDefaultImpl1](#) **Implementation**
- typedef [CollectorDatafileDefaultImpl1](#) **CollectorImplementation**

Static Public Attributes

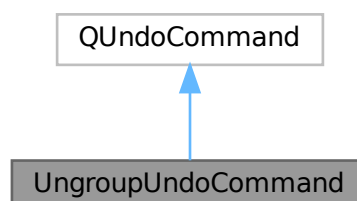
- static constexpr double **SignificanceLevel** = 0.05

The documentation for this struct was generated from the following file:

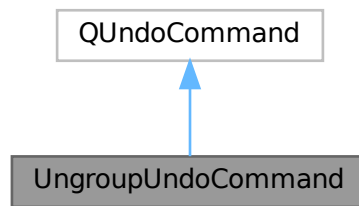
- kernel/TraitsKernel.h

10.815 UngroupUndoCommand Class Reference

Inheritance diagram for UngroupUndoCommand:



Collaboration diagram for UngroupUndoCommand:



Public Member Functions

- **UngroupUndoCommand** (QGraphicsItemGroup *group, [ModelGraphicsScene](#) *scene, QUndoCommand *parent=nullptr)
- void **undo** () override
- void **redo** () override

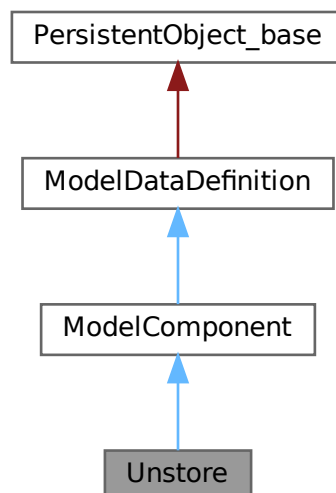
The documentation for this class was generated from the following files:

- applications/gui/genesys/actions/UngroupUndoCommand.h
- applications/gui/genesys/actions/UngroupUndoCommand.cpp

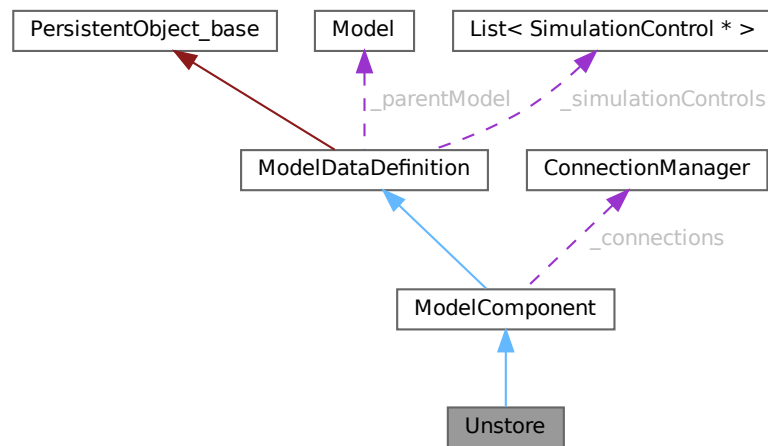
10.816 Unstore Class Reference

```
#include <plugins/components/MaterialHandling/Unstore.h>
```

Inheritance diagram for Unstore:



Collaboration diagram for Unstore:



Public Member Functions

- **Unstore** ([Model](#) *model, std::string name="")
 - virtual std::string [show](#) () override
- virtual public methods*

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~[ModelComponent](#) ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from **ModelDataDefinition**

- **ModelDataDefinition** (**Model** *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()

Releases resources owned by this data definition.
- Util::identification **getId** () const

Returns the unique identifier of this instance.
- void **setName** (const std::string &name)

Updates the logical name of this instance.
- const std::string & **getName** () const

Returns the logical name of this instance.
- void **setLabel** (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & **getLabel** () const

Returns the user-visible label of this instance.
- const std::string & **getClassname** () const

Returns the runtime class name used by the plugin system.
- **Model** * **getParentModel** () const

Returns the parent model that owns this definition.
- bool **isReportStatistics** () const

Indicates whether this data definition contributes report statistics.
- void **setReportStatistics** (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- **ModelDataDefinition** * **getInternalData** (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, **ModelDataDefinition** * > * **getInternalData** () const

Returns the full map of internal child data definitions.
- std::map< std::string, **ModelDataDefinition** * > * **getAttachedData** () const

Returns the full map of attached data definitions.
- bool **hasChanged** () const

Indicates whether this instance has been modified since the last reset.
- void **setHasChanged** (bool hasChanged)

Updates the changed flag for this data definition.
- unsigned int **getLevel** () const

Returns the level in the model hierarchy where this definition lives.
- void **setModelLevel** (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< **SimulationControl** * > * **getSimulationControls** () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level **getTraceLevelSpecific** () const

Returns the trace level that applies specifically to this definition.
- void **setTraceLevelSpecific** (TraceManager::Level level)

Updates the trace level used by this definition.
- void **defineTraceLevelSpecific** (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool **isTraceLevelSpecificEnabled** () const

Indicates whether the specific trace level is enabled.
- void **setTraceLevelSpecificEnabled** (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createNonEditableDataDefinitions](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()
- void [_templateCreateEditableDataDefinitions](#) ()
- void [_internalStatisticReportersClear](#) ()
- legacy helper methods used by the granular create* hooks*
- void [_internalDataClear](#) ()
- void [_internalDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *child)
- void [_internalDataRemove](#) (const std::string &key)
- void [_attachedDataClear](#) ()
- void [_attachedDataInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_attachedDataRemove](#) (const std::string &key)
- void [_attachedAttributesInsert](#) (const std::vector< std::string > &neededNames)
- void [_statisticReporterInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_statisticReporterRemove](#) (const std::string &key)
- void [_statisticReportersClear](#) ()
- void [_mandatoryAttachedAttributeInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryAttachedAttributeRemove](#) (const std::string &key)
- void [_mandatoryAttachedAttributesClear](#) ()
- void [_mandatoryEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryEditableDataDefinitionsClear](#) ()
- void [_optionalEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalEditableDataDefinitionsClear](#) ()
- void [_mandatoryNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_mandatoryNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_mandatoryNonEditableDataDefinitionsClear](#) ()
- void [_optionalNonEditableDataDefinitionInsert](#) (const std::string &key, [ModelDataDefinition](#) *data)
- void [_optionalNonEditableDataDefinitionRemove](#) (const std::string &key)
- void [_optionalNonEditableDataDefinitionsClear](#) ()
- void [_checkCreateAttachedReferencedDataDefinition](#) (const std::string &expression)
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void [trace](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void [traceError](#) (const std::string &text, const std::exception &e)
- void [traceError](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L1_errorFatal](#))
- void [traceReport](#) (const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L2_results](#))
- void [traceSimulation](#) (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- void [traceSimulation](#) (void *thisobject, const std::string &text, [TraceManager::Level](#) level=[TraceManager::Level::L8_detailed](#))
- Traces text to the simulation output.*
- void [traceSimulation](#) (void *thisobject, [TraceManager::Level](#) level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.816.1 Detailed Description

[Unstore](#) module DESCRIPTION The [Unstore](#) module removes an entity from storage. When an entity arrives at the [Unstore](#) module, the storage specified is decreased and the entity immediately moves to the next module in the model. TYPICAL USES Removing the entity from an animation location when processing is complete Tracking the number of customers within a grocery store (unstore upon exit) PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Type Method of specifying the storage name as a [Storage](#), [Set](#), [Attribute](#), or Expression. Default will remove an entity from the last storage that it entered. [Storage](#) Name Name of the storage to which the entity will be added. Applies only when the Type is [Storage](#). [Set](#) Name Name of the storage set from which the storage is to be selected. Applies only when the Type is [Set](#). [Set](#) Index Index into the defined storage set that contains the desired storage name. Applies only when the Type is [Set](#). [Attribute](#) Name of the attribute whose value contains the storage. Applies only when the Type is [Attribute](#). Expression Expression that is evaluated to the storage into which the entity is placed. Applies only when the Type is Expression.

10.816.2 Member Function Documentation

10.816.2.1 `_check()`

```
bool Unstore::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.816.2.2 `_loadInstance()`

```
bool Unstore::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.816.2.3 `_onDispatchEvent()`

```
void Unstore::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.816.2.4 `_saveInstance()`

```
void Unstore::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.816.2.5 `show()`

```
std::string Unstore::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/MaterialHandling/Unstore.h
- plugins/components/MaterialHandling/Unstore.cpp

10.817 Util Class Reference

Classes

- class [CommandResult](#)

Public Types

- enum class [TimeUnit](#) : int {
unknown = 0 , **picosecond** = 1 , **nanosecond** = 2 , **microsecond** = 3 ,
millisecond = 4 , **second** = 5 , **minute** = 6 , **hour** = 7 ,
day = 8 , **week** = 9 , **num_elements** = 10 }
The TimeUnit enum.
- enum class **AllocationType** : int {
ValueAdded = 0 , **NonValueAdded** = 1 , **Transfer** = 2 , **Wait** = 3 ,
Others = 4 , **num_elements** = 5 }
- enum class **TimeFormat** : unsigned int { **twelve** = 12 , **twentyFour** = 24 }
- typedef unsigned long **identification**
- typedef unsigned int **rank**

Static Public Member Functions

- static std::string [StrTimeUnitShort](#) (Util::TimeUnit timeUnit)
Returns the short textual symbol for a time unit.
- static std::string [StrTimeUnitLong](#) (Util::TimeUnit timeUnit)
Returns the long textual label for a time unit.
- static std::string [convertEnumToStr](#) (Util::TimeUnit timeUnit)
Converts a time unit enum value to its canonical string representation.
- static std::string [StrAllocation](#) (Util::AllocationType allocation)
Returns the string representation for an allocation category.
- static std::string [convertEnumToStr](#) (Util::AllocationType allocation)
Converts an allocation category enum value to a string.
- static void [SetIndent](#) (const unsigned short indent)
Sets the indentation level used by [Indent \(\)](#) .
- static void **IncIndent** ()
Increments the current indentation level.
- static void **DecIndent** ()
Decrements the current indentation level.
- static void [SepKeyVal](#) (std::string str, std::string &key, std::string &value)
Splits a key-value string around the first = character.
- static std::string [Indent](#) ()
Returns the current indentation prefix.
- static std::string [SetW](#) (std::string text, unsigned short width)
Pads or truncates text to a fixed width.
- static std::string [StrTruncfInt](#) (double value)
Formats a numeric value without losing integer semantics.
- static std::string [StrTruncfInt](#) (std::string strValue)
Removes a trailing .000000 suffix when present.
- static std::string [Trim](#) (std::string str)
Trims leading and trailing whitespace.

- static std::string [StrReplace](#) (std::string text, std::string searchFor, std::string replaceBy)
Replaces all occurrences of one substring with another.
- static std::string [StrReplaceSpecialChars](#) (std::string text)
Sanitizes a text for identifier-like usage.
- static std::string [StrIndex](#) (int index)
Formats an integer index in bracket notation.
- static void [Trimwithin](#) (std::string &str)
Removes all spaces from a string in place.
- static std::string [Map2str](#) (std::map< std::string, std::string > *mapss)
Serializes a string-to-string map to a compact text form.
- static std::string [Map2str](#) (std::map< std::string, double > *mapss)
Serializes a string-to-double map to a compact text form.
- static std::string [List2str](#) (std::list< unsigned int > *list)
Serializes a list of unsigned integers to a compact text form.
- static Util::identification [GenerateNewId](#) ()
Generates a globally unique identifier.
- static Util::identification [GenerateNewIdOfType](#) (std::string objtype)
Generates the next identifier for a given runtime type name.
- static Util::identification [GetLastIdOfType](#) (std::string objtype)
Returns the most recently generated identifier for a type.
- static void [ResetIdOfType](#) (std::string objtype)
Resets the identifier counter for a specific type.
- static void [ResetAllIds](#) ()
Resets all identifier counters maintained by the utility class.
- static double [TimeUnitConvert](#) (Util::TimeUnit timeUnit1, Util::TimeUnit timeUnit2)
Computes the multiplicative factor between two time units.
- static char [DirSeparator](#) ()
Returns the platform directory separator character.
- static std::string [FilenameFromFullFilename](#) (const std::string &s)
Extracts the file name from a full path.
- static void [FileDelete](#) (const std::string &filename)
Deletes a file if it exists.
- static std::string [PathFromFullFilename](#) (const std::string &s)
Extracts the directory path from a full filename.
- static std::string [RunningPath](#) ()
Returns the directory that contains the running binary.
- static std::vector< std::string > [ListFiles](#) (std::string dir, std::string fileFilter="", mode_t attribFilter=S_IFREG &S_IFDIR)
Lists files in a directory using an optional substring filter.
- static bool [FileExists](#) (const std::string &name)
Checks whether a file can be opened for reading.
- static Util::CommandResult [ExecuteCommand](#) (std::string command)
Executes a shell command and captures the result.
- template<typename T >
static std::string [TypeOf](#) ()
Returns the compiler type name used for T.
- template<class T >
static Util::identification [GenerateNewIdOfType](#) ()
Generates the next identifier in the namespace of T.

10.817.1 Member Function Documentation

10.817.1.1 convertEnumToStr() [1/2]

```
std::string Util::convertEnumToStr (
    Util::AllocationType allocation ) [static]
```

Converts an allocation category enum value to a string.

Parameters

<i>allocation</i>	Allocation category to convert.
-------------------	---------------------------------

Returns

Canonical string representation for the enum value.

10.817.1.2 convertEnumToStr() [2/2]

```
std::string Util::convertEnumToStr (
    Util::TimeUnit timeUnit ) [static]
```

Converts a time unit enum value to its canonical string representation.

Parameters

<i>timeUnit</i>	Time unit to convert.
-----------------	-----------------------

Returns

Canonical string representation for the enum value.

10.817.1.3 DirSeparator()

```
char Util::DirSeparator ( ) [static]
```

Returns the platform directory separator character.

Returns

Directory separator for the current platform.

10.817.1.4 ExecuteCommand()

```
Util::CommandResult Util::ExecuteCommand (
    std::string command ) [static]
```

Executes a shell command and captures the result.

Parameters

<i>command</i>	Command line to run.
----------------	----------------------

Returns

Command execution outcome and captured output buffers.

10.817.1.5 FileDelete()

```
void Util::FileDelete (
    const std::string & filename ) [static]
```

Deletes a file if it exists.

Parameters

<i>filename</i>	Path to the file to delete.
-----------------	-----------------------------

10.817.1.6 FileExists()

```
bool Util::FileExists (
    const std::string & name ) [static]
```

Checks whether a file can be opened for reading.

Parameters

<i>name</i>	File path to test.
-------------	------------------------------------

Returns

`true` if the file exists and can be opened.

10.817.1.7 FilenameFromFullFilename()

```
std::string Util::FilenameFromFullFilename (
    const std::string & s ) [static]
```

Extracts the file name from a full path.

Parameters

<i>s</i>	Full file path.
----------	-----------------

Returns

Trailing file name portion.

10.817.1.8 GenerateNewId()

```
Util::identification Util::GenerateNewId ( ) [static]
```

Generates a globally unique identifier.

Returns

Next process-wide identifier value.

10.817.1.9 GenerateNewIdOfType() [1/2]

```
template<class T >  
static Util::identification Util::GenerateNewIdOfType ( ) [inline], [static]
```

Generates the next identifier in the namespace of T.

Template Parameters

<i>T</i>	Type whose namespace should receive a new identifier.
----------	---

Returns

Next sequential identifier for the type.

10.817.1.10 GenerateNewIdOfType() [2/2]

```
Util::identification Util::GenerateNewIdOfType (  
    std::string objtype ) [static]
```

Generates the next identifier for a given runtime type name.

Parameters

<i>objtype</i>	Type name used as identifier namespace.
----------------	---

Returns

Next identifier within the requested type namespace.

10.817.1.11 GetLastIdOfType()

```
Util::identification Util::GetLastIdOfType (  
    std::string objtype ) [static]
```

Returns the most recently generated identifier for a type.

Parameters

<i>objtype</i>	Type name used as identifier namespace.
----------------	---

Returns

Last identifier value for the requested type.

10.817.1.12 Indent()

```
std::string Util::Indent ( ) [static]
```

Returns the current indentation prefix.

Returns

A string made of repeated indentation markers.

10.817.1.13 List2str()

```
std::string Util::List2str (
    std::list< unsigned int > * list ) [static]
```

Serializes a list of unsigned integers to a compact text form.

Parameters

<i>list</i>	List to serialize.
-------------	--------------------

Returns

One-line representation of the list contents.

10.817.1.14 ListFiles()

```
std::vector< std::string > Util::ListFiles (
    std::string dir,
    std::string fileFilter = "",
    mode_t attribFilter = S_IFREG & S_IFDIR ) [static]
```

Lists files in a directory using an optional substring filter.

Parameters

<i>dir</i>	Directory to scan.
<i>fileFilter</i>	Optional substring filter for file names.
<i>attribFilter</i>	Attribute mask applied to stat results.

Returns

Matching file names found in the directory.

10.817.1.15 Map2str() [1/2]

```
std::string Util::Map2str (  
    std::map< std::string, double > * mapss ) [static]
```

Serializes a string-to-double map to a compact text form.

Parameters

<i>mapss</i>	Map to serialize.
--------------	-------------------

Returns

One-line representation of the map contents.

10.817.1.16 Map2str() [2/2]

```
std::string Util::Map2str (  
    std::map< std::string, std::string > * mapss ) [static]
```

Serializes a string-to-string map to a compact text form.

Parameters

<i>mapss</i>	Map to serialize.
--------------	-------------------

Returns

One-line representation of the map contents.

10.817.1.17 PathFromFullFilename()

```
std::string Util::PathFromFullFilename (  
    const std::string & s ) [static]
```

Extracts the directory path from a full filename.

Parameters

<i>s</i>	Full file path.
----------	-----------------

Returns

Directory portion of the path.

10.817.1.18 ResetIdOfType()

```
void Util::ResetIdOfType (
    std::string objtype ) [static]
```

Resets the identifier counter for a specific type.

Parameters

<i>objtype</i>	Type name to reset.
----------------	---------------------

10.817.1.19 RunningPath()

```
std::string Util::RunningPath ( ) [static]
```

Returns the directory that contains the running binary.

Returns

Current executable directory.

10.817.1.20 SepKeyVal()

```
void Util::SepKeyVal (
    std::string str,
    std::string & key,
    std::string & value ) [static]
```

Splits a key-value string around the first = character.

Parameters

<i>str</i>	Source text in the form key=value.
<i>key</i>	Output key portion.
<i>value</i>	Output value portion.

10.817.1.21 SetIndent()

```
void Util::SetIndent (
    const unsigned short indent ) [static]
```

Sets the indentation level used by [Indent\(\)](#).

Parameters

<i>indent</i>	New indentation depth.
---------------	------------------------

10.817.1.22 SetW()

```
std::string Util::SetW (
    std::string text,
    unsigned short width ) [static]
```

Pads or truncates text to a fixed width.

Parameters

<i>text</i>	Text to adjust.
<i>width</i>	Target width.

Returns

Text resized to the requested width.

10.817.1.23 StrAllocation()

```
std::string Util::StrAllocation (
    Util::AllocationType allocation ) [static]
```

Returns the string representation for an allocation category.

Parameters

<i>allocation</i>	Allocation category to convert.
-------------------	---------------------------------

Returns

Allocation label used in reports and logs.

10.817.1.24 StrIndex()

```
std::string Util::StrIndex (
    int index ) [static]
```

Formats an integer index in bracket notation.

Parameters

<i>index</i>	Index to format.
--------------	------------------

Returns

Text such as [4].

10.817.1.25 StrReplace()

```
std::string Util::StrReplace (
    std::string text,
    std::string searchFor,
    std::string replaceBy ) [static]
```

Replaces all occurrences of one substring with another.

Parameters

<i>text</i>	Input text.
<i>searchFor</i>	Substring to replace.
<i>replaceBy</i>	Replacement text.

Returns

Updated text.

10.817.1.26 StrReplaceSpecialChars()

```
std::string Util::StrReplaceSpecialChars (
    std::string text ) [static]
```

Sanitizes a text for identifier-like usage.

Parameters

<i>text</i>	Input text.
-------------	-------------

Returns

Text with spaces and selected punctuation normalized.

10.817.1.27 StrTimeUnitLong()

```
std::string Util::StrTimeUnitLong (
    Util::TimeUnit timeUnit ) [static]
```

Returns the long textual label for a time unit.

Parameters

<i>timeUnit</i>	Time unit to convert.
-----------------	-----------------------

Returns

Long label such as `millisecond` or `hour`.

10.817.1.28 StrTimeUnitShort()

```
std::string Util::StrTimeUnitShort (
    Util::TimeUnit timeUnit ) [static]
```

Returns the short textual symbol for a time unit.

Parameters

<i>timeUnit</i>	Time unit to convert.
-----------------	-----------------------

Returns

Short symbol such as `ms` or `h`.

10.817.1.29 StrTruncfInt() [1/2]

```
std::string Util::StrTruncIfInt (
    double value ) [static]
```

Formats a numeric value without losing integer semantics.

Parameters

<i>value</i>	Value to format.
--------------	------------------

Returns

A string that preserves the decimal point for exact integers.

10.817.1.30 StrTruncfInt() [2/2]

```
std::string Util::StrTruncIfInt (
    std::string strValue ) [static]
```

Removes a trailing `.000000` suffix when present.

Parameters

<i>strValue</i>	String representation to normalize.
-----------------	-------------------------------------

Returns

Normalized string value.

10.817.1.31 TimeUnitConvert()

```
double Util::TimeUnitConvert (
    Util::TimeUnit timeUnit1,
    Util::TimeUnit timeUnit2 ) [static]
```

Computes the multiplicative factor between two time units.

Parameters

<i>timeUnit1</i>	Source unit.
<i>timeUnit2</i>	Target unit.

Returns

Conversion factor from `timeUnit1` to `timeUnit2`.

10.817.1.32 Trim()

```
std::string Util::Trim (
    std::string str ) [static]
```

Trims leading and trailing whitespace.

Parameters

<i>str</i>	Input string.
------------	---------------

Returns

Trimmed string.

10.817.1.33 Trimwithin()

```
void Util::Trimwithin (
    std::string & str ) [static]
```

Removes all spaces from a string in place.

Parameters

<i>str</i>	String to normalize.
------------	----------------------

10.817.1.34 TypeOf()

```
template<typename T >  
static std::string Util::TypeOf ( ) [inline], [static]
```

Returns the compiler type name used for T.

Template Parameters

<i>T</i>	Type to inspect.
----------	------------------

Returns

Compiler-provided type name, normalized when possible.

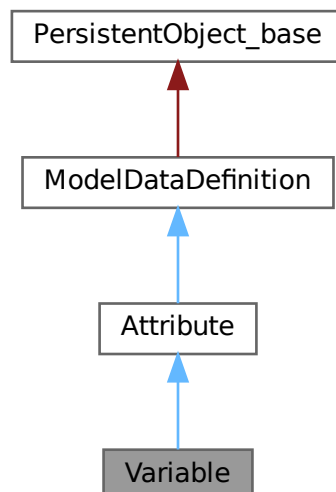
The documentation for this class was generated from the following files:

- kernel/util/Util.h
- kernel/util/Util.cpp

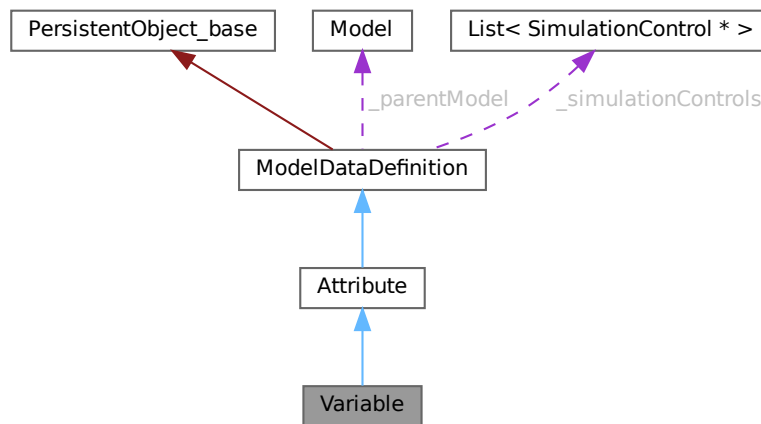
10.818 Variable Class Reference

```
#include <plugins/data/Logic/Variable.h>
```

Inheritance diagram for Variable:



Collaboration diagram for Variable:



Public Member Functions

- **Variable** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
Returns a textual representation of this data definition.
- double **getValue** (std::string index="")
Reads the current sparse value, returning 0.0 when the index is absent.
- void **setValue** (double value, std::string index="")
Writes the current sparse value at the scalar or indexed position.
- void **copyValuesFrom** (const [Variable](#) &source)
Copies the full runtime sparse store from another variable.
- std::string **getInitialValuesText** () const override
Returns the initial values serialized in bracket notation.
- void **setInitialValuesText** (std::string valuesText) override
Parses bracket notation and replaces both initial and runtime stores.
- void **insertDimentionSize** (unsigned int size)
Appends one dimension size to the variable definition.
- std::map< std::string, double > * **getValues** () const
Returns mutable current sparse values for compatibility with existing callers.
- [SparseValueStore](#) * **getValueStore** ()
Returns current value store used by this variable.
- [ModelDataDefinition](#) * **get_scope** ()
- void **set_scope** ([ModelDataDefinition](#) *const scope)

Public Member Functions inherited from [Attribute](#)

- **Attribute** ([Model](#) *model, std::string name="", std::string dataDefinitionTypename=[Util::TypeOf](#)< [Attribute](#) >())
Creates an entity attribute definition in the model.
- virtual ~**Attribute** () override

- Releases sparse initial values owned by this definition.*
- double **getInitialValue** (std::string index="")
Reads the initial sparse value, returning 0.0 when the index is absent.
- void **setInitialValue** (double value, std::string index="")
Writes the initial sparse value at the scalar or indexed position.
- void **setInitialValues** (const std::vector< std::pair< std::string, double > > values)
Replaces/extends initial sparse values from textual index/value pairs.
- void **copyInitialValuesFrom** (const [Attribute](#) &source)
Copies the full initial sparse store from another attribute.
- void **insertDimentionSize** (unsigned int size)
Appends one dimension size to the attribute definition.
- std::list< unsigned int > * **getDimensionSizes** () const
Returns dimension sizes for compatibility with existing callers.
- std::map< std::string, double > * **getInitialValues** () const
Returns mutable initial sparse values for compatibility with existing callers.
- [SparseValueStore](#) * **getInitialValueStore** ()
Returns the sparse initial value store used by this attribute.
- bool **isInitialValuesTextValid** () const
Returns whether the last textual initial value is syntactically valid.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool **isReportStatistics** () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const

- Returns the full map of attached data definitions.*
- bool `hasChanged` () const
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged` (bool `hasChanged`)
Updates the changed flag for this data definition.
- unsigned int `getLevel` () const
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel` (unsigned int `_modelLevel`)
Sets the model nesting level for this definition.
- List< `SimulationControl` * > * `getSimulationControls` () const
Returns the simulation controls owned by this data definition.
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (`TraceManager::Level` `level`)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (`TraceManager::Level` `traceLevelSpecific`, bool `traceLevelSpecificEnabled`=true)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool `traceLevelSpecificEnabled`)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation` * `GetPluginInformation` ()
- static `ModelDataDefinition` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`)
- static `ModelDataDefinition` * `NewInstance` (`Model` *`model`, std::string `name`="")

Static Public Member Functions inherited from `Attribute`

- static `PluginInformation` * `GetPluginInformation` ()
- static `ModelDataDefinition` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`)
- static `ModelDataDefinition` * `NewInstance` (`Model` *`model`, std::string `name`="")

Static Public Member Functions inherited from `ModelDataDefinition`

- static `ModelDataDefinition` * `LoadInstance` (`Model` *`model`, `PersistenceRecord` *`fields`, bool `insertIntoModel`)
Loads a model data definition instance from serialized fields.
- static `ModelDataDefinition` * `NewInstance` (`Model` *`model`, std::string `name`="")
Creates a new instance of a data definition using the plugin registry.
- static void `SaveInstance` (`PersistenceRecord` *`fields`, `ModelDataDefinition` *`modeldatum`)
Serializes a data definition instance into a persistence record.
- static bool `Check` (`ModelDataDefinition` *`modeldatum`, std::string &`errorMessage`)
Validates a data definition instance.
- static void `InitBetweenReplications` (`ModelDataDefinition` *`modeldatum`)
Resets simulation-time state between replications.
- static void `CreateInternalData` (`ModelDataDefinition` *`modeldatum`)
Creates internal and attached data required by this instance.
- static bool `CreateRelatedDataElements` (`ModelDataDefinition` *`modeldatum`, std::string &`errorMessage`)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool `_loadInstance` ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void `_saveInstance` ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool `_check` (std::string &errorMessage) override
could be overridden by derived classes
- virtual void `_initBetweenReplications` () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool `_getSaveDefaultsOption` ()
- virtual [ParserChangesInformation](#) * `_getParserChangesInformation` ()
- virtual void `_addSimulationControl` ([SimulationControl](#) *control)
- virtual void `_createInternalAndAttachedData` ()
- virtual void `_createInternalStatisticReporters` ()
- virtual void `_createAttachedAttributes` ()
- virtual void `_createNonEditableDataDefinitions` ()
- virtual void `_createEditableDataDefinitions` ()
- void `_templateCreateInternalStatisticReporters` ()
- void `_templateCreateAttachedAttributes` ()
- void `_templateCreateNonEditableDataDefinitions` ()
- void `_templateCreateEditableDataDefinitions` ()
- void `_internalStatisticReportersClear` ()
legacy helper methods used by the granular create hooks*
- void `_internalDataClear` ()
- void `_internalDataInsert` (const std::string &key, [ModelDataDefinition](#) *child)
- void `_internalDataRemove` (const std::string &key)
- void `_attachedDataClear` ()
- void `_attachedDataInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_attachedDataRemove` (const std::string &key)
- void `_attachedAttributesInsert` (const std::vector< std::string > &neededNames)
- void `_statisticReporterInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_statisticReporterRemove` (const std::string &key)
- void `_statisticReportersClear` ()
- void `_mandatoryAttachedAttributeInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryAttachedAttributeRemove` (const std::string &key)
- void `_mandatoryAttachedAttributesClear` ()
- void `_mandatoryEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryEditableDataDefinitionsClear` ()
- void `_optionalEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalEditableDataDefinitionsClear` ()
- void `_mandatoryNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_mandatoryNonEditableDataDefinitionRemove` (const std::string &key)
- void `_mandatoryNonEditableDataDefinitionsClear` ()
- void `_optionalNonEditableDataDefinitionInsert` (const std::string &key, [ModelDataDefinition](#) *data)
- void `_optionalNonEditableDataDefinitionRemove` (const std::string &key)
- void `_optionalNonEditableDataDefinitionsClear` ()
- void `_checkCreateAttachedReferencedDataDefinition` (const std::string &expression)

method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions

- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification _id
- std::string _typename
- bool _reportStatistics
- bool _hasChanged
- unsigned int _modelLevel = 0
- [Model](#) * _parentModel
- [List](#)< [SimulationControl](#) * > * _simulationControls = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level _traceLevelSpecific
- bool _traceLevelSpecificEnabled = false

10.818.1 Detailed Description

[Variable](#) module DESCRIPTION This data module is used to define a variable's dimension and values. You can reference variables in other modules (for example, the [Decide](#) module), reassign new values to variables with the [Assign](#) module, and use variables in any expression. You can use an external data file to specify variable values, and you can specify the variable's initial values in the [Variable](#) module. If you use both methods, the values are read at different times, depending on the options you specify, including the [File](#) Read Time, the Clear Option, and the replication parameters you specify in the Run Setup dialog box. For more information, see the online Help. There are three methods for manually editing the Initial Values of a [Variable](#) module: Using the standard spreadsheet interface. In the module spreadsheet, right-click on the Initial Values cell and select the Edit via spreadsheet menu item. The values for two-dimensional arrays should be entered one column at a time. Array elements not explicitly assigned are assumed to have the last entered value. Using the module dialog box. In the module spreadsheet, right-click on any cell and select the Edit via dialog menu item. The values for two-dimensional arrays should be entered one column at a time. Array elements not explicitly assigned are assumed to have the last entered value. Using the two-dimensional (2-D) spreadsheet interface. In the module spreadsheet, click on the Initial Values cell.

TYPICAL USES Number of documents processed per hour Serial number to assign to parts for unique identification Space available in a facility PROMPTS Prompt Description Name The unique name of the variable being defined. Rows Number of rows in a one- or two-dimensional variable. Columns Number of columns in a two-dimensional variable. Report Statistics Check box for determining whether or not statistics will be collected. This field is visible when the rows and columns are not specified (that is, for single variables). Data Type The data type of the values stored in the variable. Valid types are Real and String. The default type is Real. Clear Option Defines the time (if at all) when the value(s) of the variable is reset to the initial value(s) specified. Specifying Statistics resets this variable to its initial value(s) whenever statistics are cleared. Specifying System resets this variable to its initial value(s) whenever the system is cleared. Specifying None indicates that this variable is never reset to its initial value(s), except prior to the first replication. [File](#) Name Name of the file from which to read the variable's value or values. You

can use any file access type supported by Arena except sequential text files and Lotus spreadsheet (.wks) files. If the file name you specify has not been created yet, Arena will create it, but you must edit the file to specify the file access type, path, and recordset (if required). Recordset Name of the recordset in the specified file from which to read values. This field is available only if you specify a [File](#) Name for a file that has been set up with a file access type, path, and recordset. Arena uses the Rows and Columns properties to determine the amount of data to read from the recordset. A recordset is required for all file types except .xml. The recordset size must be equal to or greater than the number of rows and columns specified for the variable. [File](#) Read Time Specifies when to read the values from the file into the variable. If you select PreCheck, the values for the variable are read while the model is still in Edit mode (prior to the model being checked and compiled). If you select BeginSimulation, values are read when the model is compiled, prior to the first replication. If you select BeginReplication, values are read prior to each replication. Initial Values Lists the initial value or values of the variable. You can assign new values to the variable at different stages of the model by using the [Assign](#) module. Initial Value [Variable](#) value at the start of the simulation.

10.818.2 Member Function Documentation

10.818.2.1 `_check()`

```
bool Variable::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [Attribute](#).

10.818.2.2 `_initBetweenReplications()`

```
void Variable::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.818.2.3 `_loadInstance()`

```
bool Variable::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [Attribute](#).

10.818.2.4 `_saveInstance()`

```
void Variable::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [Attribute](#).

10.818.2.5 getInitialValuesText()

```
std::string Variable::getInitialValuesText ( ) const [override], [virtual]
```

Returns the initial values serialized in bracket notation.

Reimplemented from [Attribute](#).

10.818.2.6 setInitialValuesText()

```
void Variable::setInitialValuesText (
    std::string valuesText ) [override], [virtual]
```

Parses bracket notation and replaces both initial and runtime stores.

Reimplemented from [Attribute](#).

10.818.2.7 show()

```
std::string Variable::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

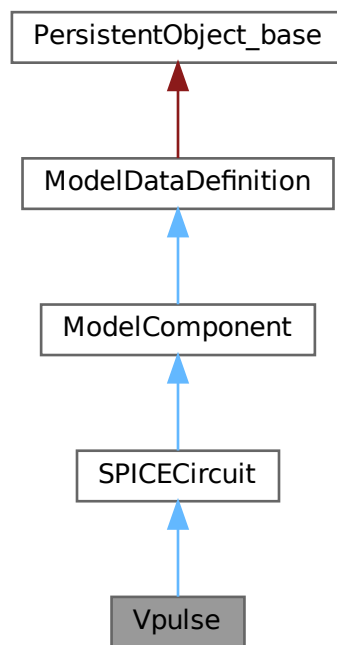
Reimplemented from [Attribute](#).

The documentation for this class was generated from the following files:

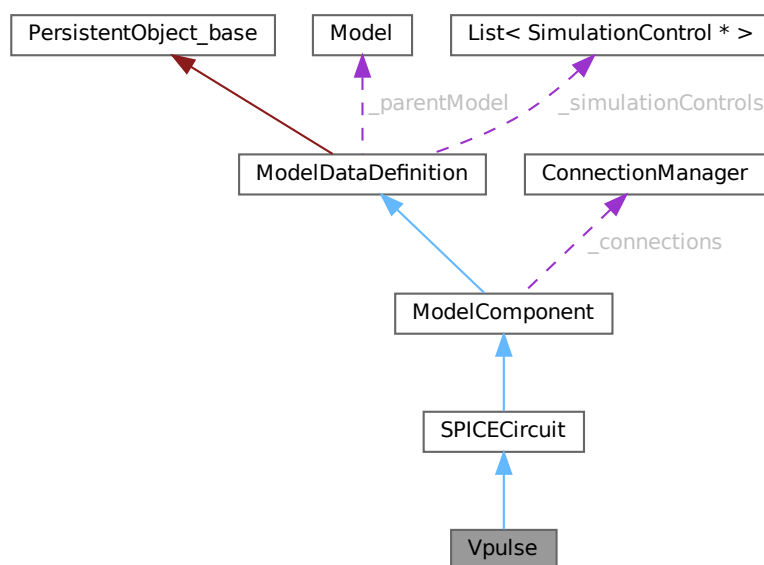
- plugins/data/Logic/Variable.h
- plugins/data/Logic/Variable.cpp

10.819 Vpulse Class Reference

Inheritance diagram for Vpulse:



Collaboration diagram for Vpulse:



Public Member Functions

- **Vpulse** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setVoltage** (float voltage)
- void **setFreq** (float freq)
- void **setSlope** (float slope)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- **ModelComponent** ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- **ModelDataDefinition** ([Model](#) *model, std::string datadefinitionType, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.
- const std::string & **getName** () const

- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level](#) [getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) * **GetPluginInformation** ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
- static std::vector< std::string > **split** (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void **_createNonEditableDataDefinitions** () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual bool **_check** (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_initBetweenReplications** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)

- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * **_connections** = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.819.1 Member Function Documentation

10.819.1.1 [_loadInstance\(\)](#)

```
bool Vpulse::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.819.1.2 _saveInstance()

```
void Vpulse::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.819.1.3 Build()

```
void Vpulse::Build ( ) [override], [virtual]
```

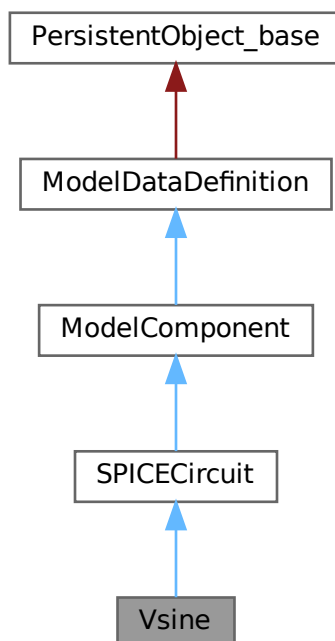
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

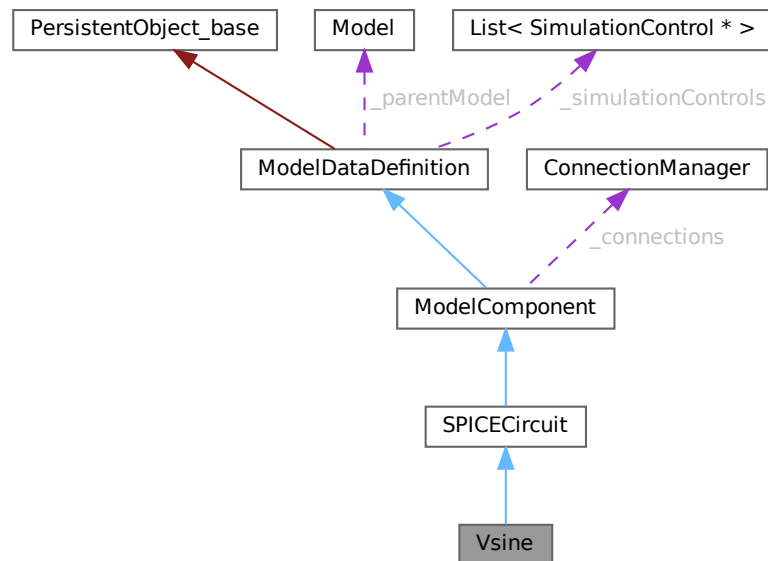
- plugins/components/ElectronicsSimulation/SPICECircuit.h
- plugins/components/ElectronicsSimulation/SPICECircuit.cpp

10.820 Vsine Class Reference

Inheritance diagram for Vsine:



Collaboration diagram for Vsine:



Public Member Functions

- **Vsine** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setVoltage** (float voltage)
- void **setFreq** (float freq)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentName, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)

- *Connects this component to another component.*
- bool `hasBreakpointAt ()`
Indicates whether a simulation breakpoint is set on this component.
- void `setDescription (std::string _description)`
Updates the free-form description shown by the GUI.
- std::string `getDescription () const`
Returns the free-form description associated with this component.
- void `remove (ModelComponent *comp)`
Removes a connected component from this component.

Public Member Functions inherited from `ModelDataDefinition`

- `ModelDataDefinition (Model *model, std::string datadefinitionTypename, std::string name="", bool insertIntoModel=true)`
Creates a model data definition and optionally inserts it into the model.
- virtual `~ModelDataDefinition ()`
Releases resources owned by this data definition.
- Util::identification `getId () const`
Returns the unique identifier of this instance.
- void `setName (const std::string &name)`
Updates the logical name of this instance.
- const std::string & `getName () const`
Returns the logical name of this instance.
- void `setLabel (const std::string &label)`
Updates the user-visible label of this instance.
- const std::string & `getLabel () const`
Returns the user-visible label of this instance.
- const std::string & `getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model *` `getParentModel () const`
Returns the parent model that owns this definition.
- bool `isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- void `setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition *` `getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- std::map< std::string, `ModelDataDefinition *` > `* getInternalData () const`
Returns the full map of internal child data definitions.
- std::map< std::string, `ModelDataDefinition *` > `* getAttachedData () const`
Returns the full map of attached data definitions.
- bool `hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- void `setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- unsigned int `getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- void `setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl *` > `* getSimulationControls () const`

- Returns the simulation controls owned by this data definition.*
- `TraceManager::Level` `getTraceLevelSpecific` () const
Returns the trace level that applies specifically to this definition.
- void `setTraceLevelSpecific` (TraceManager::Level level)
Updates the trace level used by this definition.
- void `defineTraceLevelSpecific` (TraceManager::Level traceLevelspecific, bool traceLevelSpecific←
Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool `isTraceLevelSpecificEnabled` () const
Indicates whether the specific trace level is enabled.
- void `setTraceLevelSpecificEnabled` (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static `PluginInformation` * `GetPluginInformation` ()
- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")

Static Public Member Functions inherited from `SPICECircuit`

- static `PluginInformation` * `GetPluginInformation` ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
- static `ModelDataDefinition` * `NewInstance` (`Model` *model, std::string name="")
- static std::vector< std::string > `split` (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from `ModelComponent`

- static `ModelComponent` * `LoadInstance` (`Model` *model, `PersistenceRecord` *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool `Check` (`ModelComponent` *component)
Validates a component instance.
- static void `CreateInternalData` (`ModelComponent` *component)
Creates internal data required by a component instance.
- static void `SaveInstance` (`PersistenceRecord` *fields, `ModelComponent` *component)
Serializes a component instance into a persistence record.
- static void `DispatchEvent` (`Event` *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()

- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
 - legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * [_connections](#) = new [ConnectionManager](#)()

virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification [_id](#)
- std::string [_typename](#)
- bool [_reportStatistics](#)
- bool [_hasChanged](#)
- unsigned int [_modelLevel](#) = 0
- [Model](#) * [_parentModel](#)
- [List](#)< [SimulationControl](#) * > * [_simulationControls](#) = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level [_traceLevelSpecific](#)
- bool [_traceLevelSpecificEnabled](#) = false

10.820.1 Member Function Documentation

10.820.1.1 [_loadInstance\(\)](#)

```
bool Vsine::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.820.1.2 [_saveInstance\(\)](#)

```
void Vsine::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.820.1.3 [Build\(\)](#)

```
void Vsine::Build ( ) [override], [virtual]
```

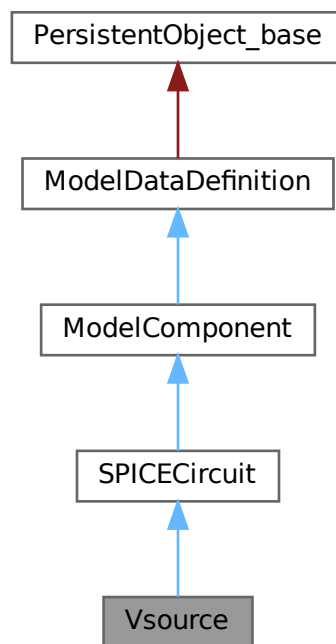
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

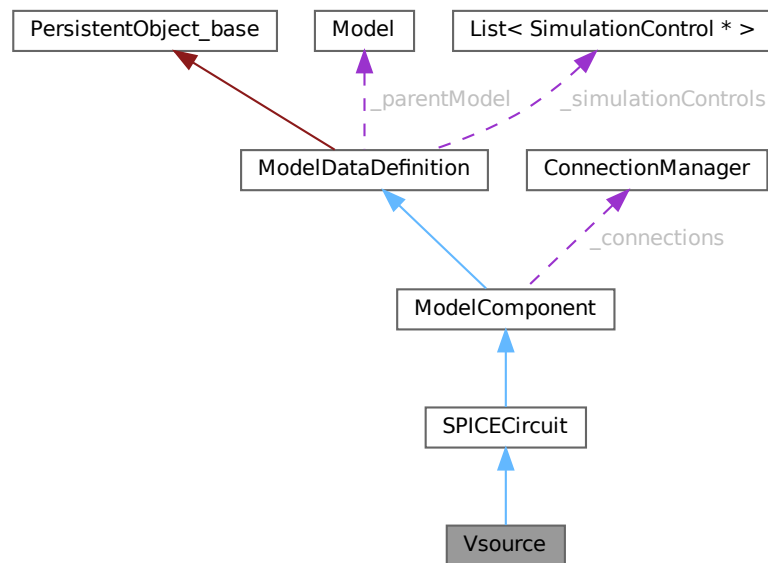
- plugins/components/ElectronicsSimulation/SPIECEircuit.h
- plugins/components/ElectronicsSimulation/SPIECEircuit.cpp

10.821 Vsource Class Reference

Inheritance diagram for Vsource:



Collaboration diagram for Vsource:



Public Member Functions

- **Vsource** (**Model** *model, std::string name="")
- void **Build** () override
- void **setVoltage** (float voltage)

Public Member Functions inherited from **SPICECircuit**

- **SPICECircuit** (**Model** *model, std::string name="")
constructors
- void **setRunner** (**SPICERunner** *runner)
new public user methods for this components
- **SPICERunner** * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)

- Connects this component to another component.*

 - bool [hasBreakpointAt](#) ()
 - Indicates whether a simulation breakpoint is set on this component.*
 - void [setDescription](#) (std::string _description)
 - Updates the free-form description shown by the GUI.*
 - std::string [getDescription](#) () const
 - Returns the free-form description associated with this component.*
 - void [remove](#) ([ModelComponent](#) *comp)
 - Removes a connected component from this component.*

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵IntoModel=true)
- Creates a model data definition and optionally inserts it into the model.*
- virtual ~[ModelDataDefinition](#) ()
- Releases resources owned by this data definition.*
- Util::identification [getId](#) () const
- Returns the unique identifier of this instance.*
- void [setName](#) (const std::string &name)
- Updates the logical name of this instance.*
- const std::string & [getName](#) () const
- Returns the logical name of this instance.*
- void [setLabel](#) (const std::string &label)
- Updates the user-visible label of this instance.*
- const std::string & [getLabel](#) () const
- Returns the user-visible label of this instance.*
- const std::string & [getClassname](#) () const
- Returns the runtime class name used by the plugin system.*
- [Model](#) * [getParentModel](#) () const
- Returns the parent model that owns this definition.*
- bool [isReportStatistics](#) () const
- Indicates whether this data definition contributes report statistics.*
- void [setReportStatistics](#) (bool reportStatistics)
- Enables or disables report-statistics generation for this definition.*
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
- Looks up a named internal child data definition.*
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
- Returns the full map of internal child data definitions.*
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
- Returns the full map of attached data definitions.*
- bool [hasChanged](#) () const
- Indicates whether this instance has been modified since the last reset.*
- void [setHasChanged](#) (bool [hasChanged](#))
- Updates the changed flag for this data definition.*
- unsigned int [getLevel](#) () const
- Returns the level in the model hierarchy where this definition lives.*
- void [setModelLevel](#) (unsigned int _modelLevel)
- Sets the model nesting level for this definition.*
- [List](#)< [SimulationControl](#) * > * [getSimulationControls](#) () const

- Returns the simulation controls owned by this data definition.*
 - TraceManager::Level [getTraceLevelSpecific](#) () const
 - Returns the trace level that applies specifically to this definition.*
 - void [setTraceLevelSpecific](#) (TraceManager::Level level)
 - Updates the trace level used by this definition.*
 - void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)
 - Defines the specific trace level and optionally enables it immediately.*
 - bool [isTraceLevelSpecificEnabled](#) () const
 - Indicates whether the specific trace level is enabled.*
 - void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
 - Enables or disables the specific trace level.*

Static Public Member Functions

- static [PluginInformation](#) * [GetPluginInformation](#) ()
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation](#) * [GetPluginInformation](#) ()
 - static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split_char)
 - Externalize function GetPluginInformation to be accessible through dynamic linked library.*

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
 - static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)*
- static bool [Check](#) ([ModelComponent](#) *component)
 - Validates a component instance.*
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
 - Creates internal data required by a component instance.*
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
 - Serializes a component instance into a persistence record.*
- static void [DispatchEvent](#) ([Event](#) *event)
 - Dispatches a simulation event to the target component.*

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()

- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
 - legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * [_connections](#) = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification [_id](#)
- std::string [_typename](#)
- bool [_reportStatistics](#)
- bool [_hasChanged](#)
- unsigned int [_modelLevel](#) = 0
- [Model](#) * [_parentModel](#)
- [List](#)< [SimulationControl](#) * > * [_simulationControls](#) = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level [_traceLevelSpecific](#)
- bool [_traceLevelSpecificEnabled](#) = false

10.821.1 Member Function Documentation

10.821.1.1 [_loadInstance\(\)](#)

```
bool Vsource::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.821.1.2 [_saveInstance\(\)](#)

```
void Vsource::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.821.1.3 [Build\(\)](#)

```
void Vsource::Build ( ) [override], [virtual]
```

Reimplemented from [SPICECircuit](#).

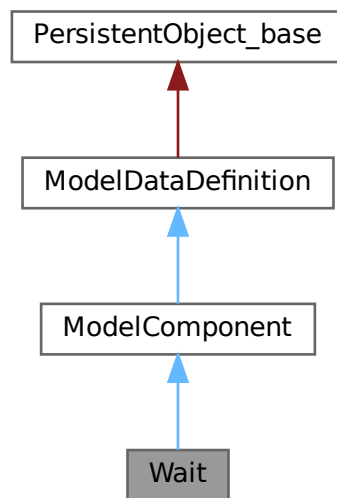
The documentation for this class was generated from the following files:

- plugins/components/ElectronicsSimulation/SPIECircuit.h
- plugins/components/ElectronicsSimulation/SPIECircuit.cpp

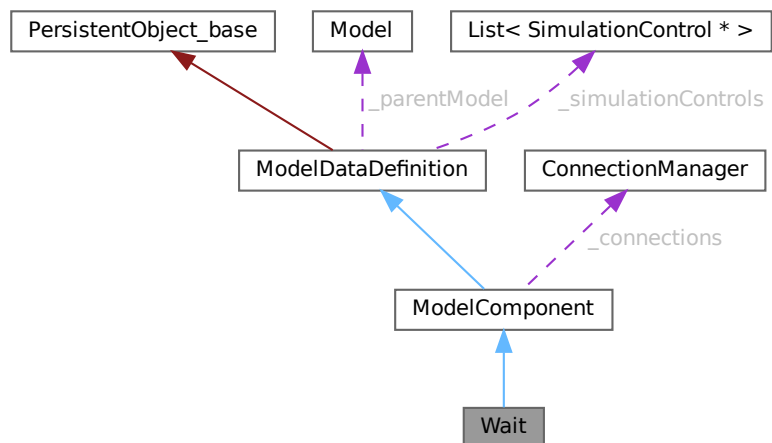
10.822 Wait Class Reference

```
#include <plugins/components/Synchronization/Wait.h>
```

Inheritance diagram for Wait:



Collaboration diagram for Wait:



Public Types

- enum class **WaitType** : int { **WaitForSignal** = 0 , **InfiniteHold** = 1 , **ScanForCondition** = 2 , **num_elements** = 3 }

Public Member Functions

- **Wait** ([Model](#) *model, std::string name="")
- virtual std::string **show** () override
virtual public methods
- void **setSignalData** ([SignalData](#) *signal)
- void **setWaitType** (WaitType _watType)
- Wait::WaitType **getWaitType** () const
- void **setCondition** (std::string _condition)
- std::string **getCondition** () const
- [Queue](#) * **getQueue** () const
- void **setQueue** ([Queue](#) *queue)
- std::string **getlimitExpression** () const
- void **setLimitExpression** (const std::string &newLimitExpression)

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- [ConnectionManager](#) * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~**ModelDataDefinition** ()
Releases resources owned by this data definition.
- Util::identification **getId** () const
Returns the unique identifier of this instance.
- void **setName** (const std::string &name)
Updates the logical name of this instance.

- `const std::string & getName () const`
Returns the logical name of this instance.
- `void setLabel (const std::string &label)`
Updates the user-visible label of this instance.
- `const std::string & getLabel () const`
Returns the user-visible label of this instance.
- `const std::string & getClassname () const`
Returns the runtime class name used by the plugin system.
- `Model * getParentModel () const`
Returns the parent model that owns this definition.
- `bool isReportStatistics () const`
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name) const`
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData () const`
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData () const`
Returns the full map of attached data definitions.
- `bool hasChanged () const`
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel () const`
Returns the level in the model hierarchy where this definition lives.
- `void setModelLevel (unsigned int _modelLevel)`
Sets the model nesting level for this definition.
- `List< SimulationControl * > * getSimulationControls () const`
Returns the simulation controls owned by this data definition.
- `TraceManager::Level getTraceLevelSpecific () const`
Returns the trace level that applies specifically to this definition.
- `void setTraceLevelSpecific (TraceManager::Level level)`
Updates the trace level used by this definition.
- `void defineTraceLevelSpecific (TraceManager::Level traceLevelSpecific, bool traceLevelSpecificEnabled=true)`
Defines the specific trace level and optionally enables it immediately.
- `bool isTraceLevelSpecificEnabled () const`
Indicates whether the specific trace level is enabled.
- `void setTraceLevelSpecificEnabled (bool traceLevelSpecificEnabled)`
Enables or disables the specific trace level.

Static Public Member Functions

- `static std::string convertEnumToStr (WaitType type)`
- `static PluginInformation * GetPluginInformation ()`
- `static ModelComponent * LoadInstance (Model *model, PersistenceRecord *fields)`
- `static ModelDataDefinition * NewInstance (Model *model, std::string name="")`

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool [_check](#) (std::string &errorMessage) override
could be overridden by derived classes
- virtual void [_initBetweenReplications](#) () override
- virtual void [_createNonEditableDataDefinitions](#) () override
- virtual void [_createEditableDataDefinitions](#) () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createInternalStatisticReporters ()`
- `virtual void _createAttachedAttributes ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
- legacy helper methods used by the granular create* hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
 - method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)`
 - Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.822.1 Detailed Description

[Wait](#) module DESCRIPTION This module will hold an entity in a queue to wait for a signal, wait for a specified condition to become true (scan), or be held infinitely (to be removed later with the [Remove](#) module). If the entity is holding for a signal, the [Signal](#) module is used elsewhere in the model to allow the entity to move on to the next module. If the entity is holding for a given condition to be true, the entity will remain at the module (either in a defined or internal queue) until the condition(s) becomes true. When the entity is in an infinite hold, the [Remove](#) module is used elsewhere in the model to allow the entity to continue processing. TYPICAL USES [Waiting](#) for a traffic light to turn green Holding a part for authorization Checking the status of a machine or operator to continue a process PROMPTS Prompt Description Name Unique module identifier displayed on the module shape. Type Indicates the reasoning for holding the entity within a specified or internal queue. [Wait](#) for [Signal](#) will hold the entity until a signal of the same value is received. Scan for Condition will hold the entity until the specified condition becomes true. Infinite [Wait](#) will hold the entity until it is removed from the queue by a [Remove](#) module. [Wait](#) for Value [Signal](#) code for the waiting entity. Applies only when Type is [Wait](#) for [Signal](#). Limit Maximum number of waiting entities that will be released upon receipt of a signal. Applies only when Type is [Wait](#) for [Signal](#). Condition Specifies the condition that will be evaluated to hold the entity at the module. If the condition is evaluated to true, the entity leaves the module immediately. If the condition is false, the entity will wait in the associated queue until the condition becomes true. Applies only when Type is Scan for Condition. [Queue](#) Type Determines the type of queue used to hold the entities. If [Queue](#) is selected, the queue name is specified. If [Set](#) is selected, the queue set and member in the set are specified. If Internal is selected, an internal queue is used to hold all waiting entities. [Attribute](#) and Expression are additional methods for defining the queue to be used. [Queue](#) Name This field is visible only if [Queue](#) Type is [Queue](#), and it defines the symbol name of the queue. [Set](#) Name This field is visible only if [Queue](#) Type is [Set](#), and it defines the queue set that contains the queue being referenced. [Set](#) Index This field is visible only if [Queue](#) Type is [Set](#), and it defines the index into the queue set. Note that this is the index into the set and not the name of the queue in the set. For example, the only valid entry for a queue set containing three members is an expression that evaluates to 1, 2, or 3. [Attribute](#) This field is visible only if [Queue](#) Type is [Attribute](#). The attribute entered in this field will be evaluated to indicate which queue is to be used. Expression This field is visible only if [Queue](#) Type is Expression. The expression entered in this field will be evaluated to indicate which queue is to be used.

10.822.2 Member Function Documentation

10.822.2.1 `_check()`

```
bool Wait::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.822.2.2 _createEditableDataDefinitions()

```
void Wait::_createEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.822.2.3 _createNonEditableDataDefinitions()

```
void Wait::_createNonEditableDataDefinitions ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.822.2.4 _initBetweenReplications()

```
void Wait::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.822.2.5 _loadInstance()

```
bool Wait::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.822.2.6 _onDispatchEvent()

```
void Wait::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.822.2.7 _saveInstance()

```
void Wait::_saveInstance (
```

```
PersistenceRecord * fields,
bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.822.2.8 show()

```
std::string Wait::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

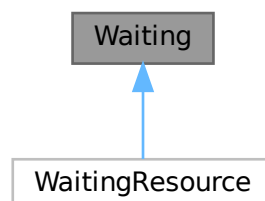
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

- plugins/components/Synchronization/Wait.h
- plugins/components/Synchronization/Wait.cpp

10.823 Waiting Class Reference

Inheritance diagram for Waiting:



Public Member Functions

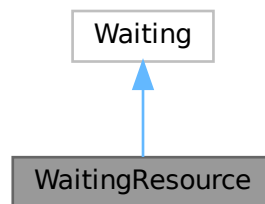
- **Waiting** ([Entity](#) *entity, double timeStartedWaiting, [ModelComponent](#) *thisComponent, unsigned int thisComponentOutputPort=0)
- virtual std::string **show** ()
- double **getTimeStartedWaiting** () const
- [ModelComponent](#) * **geComponent** () const
- [Entity](#) * **getEntity** () const
- unsigned int **geComponentOutputPort** () const
- unsigned long long **getArrivalOrder** () const
- void **setArrivalOrder** (unsigned long long arrivalOrder)

The documentation for this class was generated from the following file:

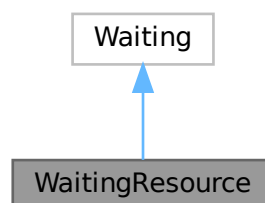
- plugins/data/DiscreteProcessing/Queue.h

10.824 WaitingResource Class Reference

Inheritance diagram for WaitingResource:



Collaboration diagram for WaitingResource:



Public Member Functions

- **WaitingResource** ([Entity](#) *entity, double timeStartedWaiting, unsigned int quantity, [ModelComponent](#) *thisComponent, unsigned int thisComponentOutputPort=0)
- **WaitingResource** (const [WaitingResource](#) &orig)
- virtual std::string **show** () override
- unsigned int **getQuantity** () const

Public Member Functions inherited from [Waiting](#)

- **Waiting** ([Entity](#) *entity, double timeStartedWaiting, [ModelComponent](#) *thisComponent, unsigned int thisComponentOutputPort=0)
- double **getTimeStartedWaiting** () const
- [ModelComponent](#) * **geComponent** () const
- [Entity](#) * **getEntity** () const
- unsigned int **geComponentOutputPort** () const
- unsigned long long **getArrivalOrder** () const
- void **setArrivalOrder** (unsigned long long arrivalOrder)

10.824.1 Member Function Documentation**10.824.1.1 show()**

```
virtual std::string WaitingResource::show ( ) [inline], [override], [virtual]
```

Reimplemented from [Waiting](#).

The documentation for this class was generated from the following file:

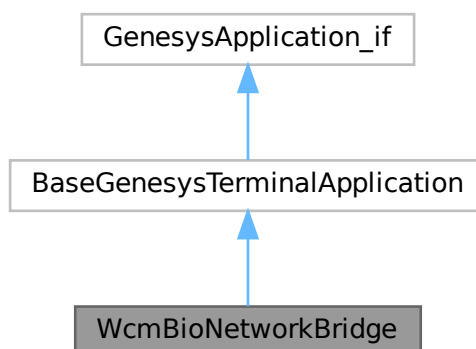
- plugins/components/DiscreteProcessing/Seize.h

10.825 WcmBioNetworkBridge Class Reference

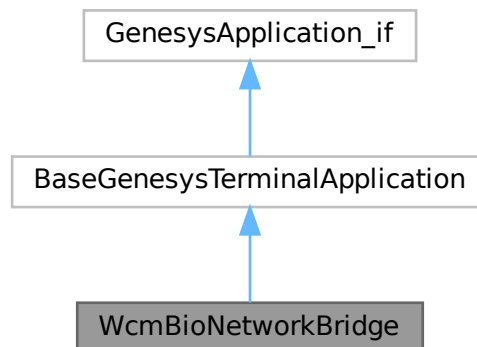
Demonstrates a minimal deterministic/stochastic whole-cell bridge.

```
#include <applications/modelSpecific/wcm/WcmBioNetworkBridge.h>
```

Inheritance diagram for WcmBioNetworkBridge:



Collaboration diagram for WcmBioNetworkBridge:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.825.1 Detailed Description

Demonstrates a minimal deterministic/stochastic whole-cell bridge.

The example builds a small deterministic [BioNetwork](#) for nutrient conversion, projects its final state into a [WholeCellState](#), and then runs stochastic transcription, translation, and degradation components. It is a didactic integration test for GenESyS abstractions, not a complete organism model.

10.825.2 Member Function Documentation

10.825.2.1 main()

```
int WcmBioNetworkBridge::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

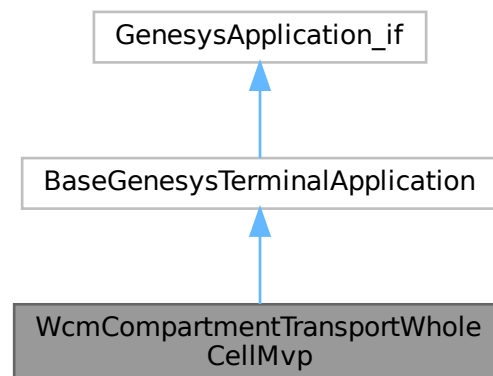
- applications/modelSpecific/wcm/WcmBioNetworkBridge.h
- applications/modelSpecific/wcm/WcmBioNetworkBridge.cpp

10.826 WcmCompartmentTransportWholeCellMvp Class Reference

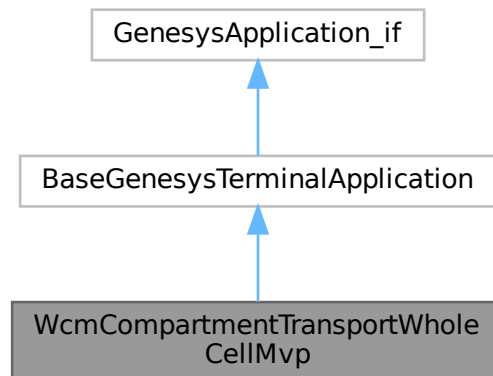
Whole-cell MVP with compartment-aware metabolic transport.

```
#include <applications/modelSpecific/wcm/WcmCompartmentTransportWholeCellMvp.h>
```

Inheritance diagram for WcmCompartmentTransportWholeCellMvp:



Collaboration diagram for WcmCompartmentTransportWholeCellMvp:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.826.1 Detailed Description

Whole-cell MVP with compartment-aware metabolic transport.

This example extends the flux-balance whole-cell path with a tiny extracellular-to-cytosol transport chain. It demonstrates how projected metabolic fluxes can feed:

- global energy proxies;
- compartment-scoped metabolite pools; and
- pathway activity checkpoints.

The biological scope is still didactic. The purpose is to validate the architecture for multi-compartment hybrid whole-cell simulations.

10.826.2 Member Function Documentation

10.826.2.1 main()

```
int WcmCompartmentTransportWholeCellMvp::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

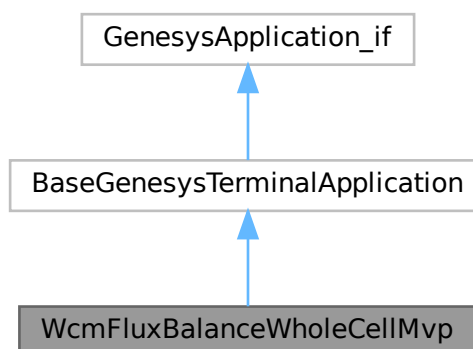
- applications/modelSpecific/wcm/WcmCompartmentTransportWholeCellMvp.h
- applications/modelSpecific/wcm/WcmCompartmentTransportWholeCellMvp.cpp

10.827 WcmFluxBalanceWholeCellMvp Class Reference

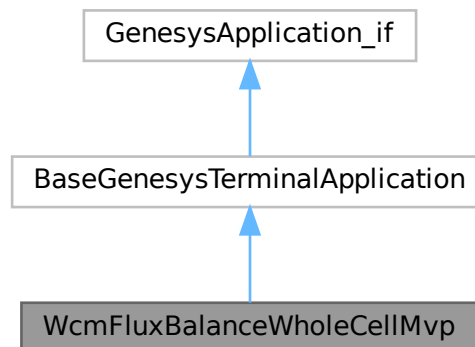
Whole-cell MVP driven by flux-balance metabolism.

```
#include <applications/modelSpecific/wcm/WcmFluxBalanceWholeCellMvp.h>
```

Inheritance diagram for WcmFluxBalanceWholeCellMvp:



Collaboration diagram for WcmFluxBalanceWholeCellMvp:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.827.1 Detailed Description

Whole-cell MVP driven by flux-balance metabolism.

This example demonstrates a second metabolic integration path for GenESyS whole-cell models. Instead of using the deterministic [BioNetwork](#) runner, it executes a small FBA problem through [MetabolicFluxBalance](#), projects selected fluxes into [WholeCellState](#), and then runs expression, growth, lifecycle checkpointing, fate routing, and division.

10.827.2 Member Function Documentation

10.827.2.1 main()

```
int WcmFluxBalanceWholeCellMvp::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

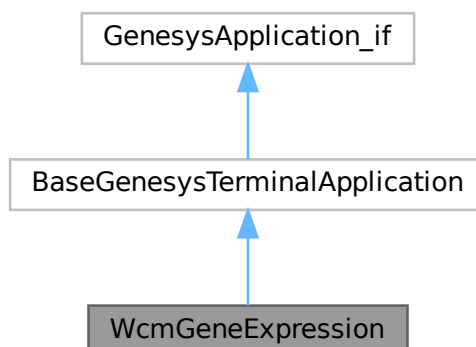
The documentation for this class was generated from the following files:

- applications/modelSpecific/wcm/WcmFluxBalanceWholeCellMvp.h
- applications/modelSpecific/wcm/WcmFluxBalanceWholeCellMvp.cpp

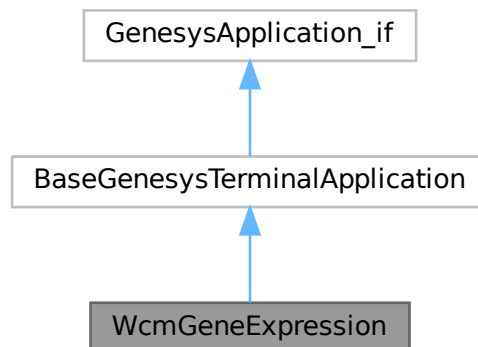
10.828 WcmGeneExpression Class Reference

```
#include <applications/modelSpecific/wcm/WcmGeneExpression.h>
```

Inheritance diagram for WcmGeneExpression:



Collaboration diagram for WcmGeneExpression:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.828.1 Detailed Description

Central dogma simulation: stochastic transcription + translation + SSA degradation.

Models 4 genes using two complementary approximations:

- Poisson tau-leaping for transcription ([StochasticTranscription](#)) and translation ([StochasticTranslation](#)) — valid for high-copy-number RNAP/ribosome
- Gillespie Direct Method SSA for low-copy-number mRNA and protein degradation

Biological parameters (M. genitalium ballpark): RNAP_free = 200 molecules ribosome_free = 400 molecules 4 genes: geneA (metabolic), geneB (regulatory), geneC (structural), geneD (transport) Transcription: elongRate=50 nt/s, meanGeneLen=900 nt, bindProb=0.25, $\Delta t=60$ s Translation: elongRate=16 AA/s, meanProtLen=300 AA, $\Delta t=60$ s mRNA decay: $k = 0.01/s$ (mean lifetime ≈ 100 s) Protein decay: $k = 0.001/s$ (mean lifetime ≈ 1000 s ≈ 16 min)

Simulation: 10 replications $\times$ 3600 s, 60 s time steps.

Run via GENESYS_TERMINAL_EXAMPLE: `cmake -DGENESYS_TERMINAL_EXAMPLE=wcm/WcmGeneExpression.cpp ...`

10.828.2 Member Function Documentation

10.828.2.1 main()

```
int WcmGeneExpression::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

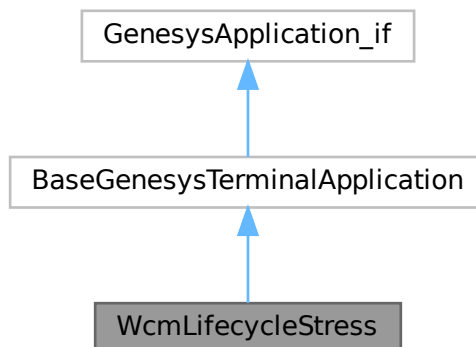
- applications/modelSpecific/wcm/WcmGeneExpression.h
- applications/modelSpecific/wcm/WcmGeneExpression.cpp

10.829 WcmLifecycleStress Class Reference

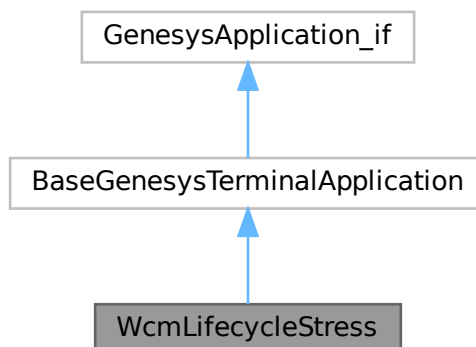
Demonstrates starvation and death routing in a whole-cell stress scenario.

```
#include <applications/modelSpecific/wcm/WcmLifecycleStress.h>
```

Inheritance diagram for WcmLifecycleStress:



Collaboration diagram for WcmLifecycleStress:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.829.1 Detailed Description

Demonstrates starvation and death routing in a whole-cell stress scenario.

This example focuses on lifecycle checkpointing rather than biosynthesis. It uses a low-energy whole-cell state plus the [CellCycleCheckpointComponent](#) and [CellFateDecisionComponent](#) to show the transition: newborn -> starved -> dead.

10.829.2 Member Function Documentation

10.829.2.1 main()

```
int WcmLifecycleStress::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

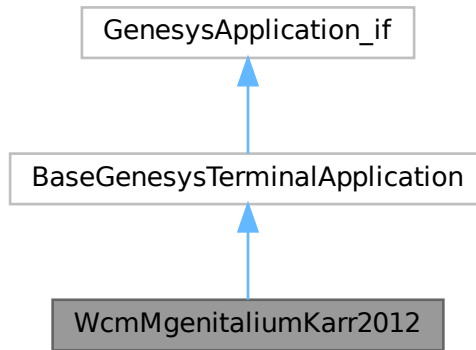
The documentation for this class was generated from the following files:

- applications/modelSpecific/wcm/WcmLifecycleStress.h
- applications/modelSpecific/wcm/WcmLifecycleStress.cpp

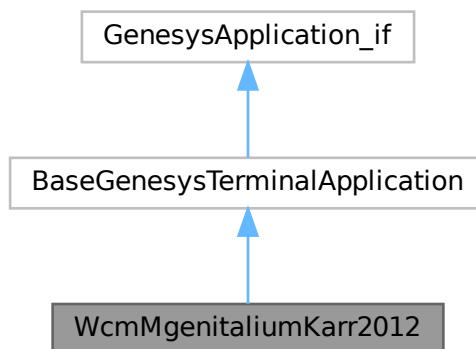
10.830 WcmMgenitaliumKarr2012 Class Reference

```
#include <applications/modelSpecific/wcm/WcmMgenitaliumKarr2012.h>
```

Inheritance diagram for WcmMgenitaliumKarr2012:



Collaboration diagram for WcmMgenitaliumKarr2012:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.830.1 Detailed Description

Whole-cell simulation analog of the *M. genitalium* model (Karr et al. 2012).

Reference: Karr JR, Sanghvi JC, Macklin DN, et al. "A Whole-Cell Computational Model Predicts Phenotype from Genotype." [Cell](#) 150(2):389–401, 2012. doi:10.1016/j.cell.2012.05.044

Architecture: This model uses all four GenESyS WholeCellModeling plugins in sequence:

SimClock → Transcription → Translation → MetabolicSSA → CellDivision |— port 0 (no division) → [Dispose](#) |— port 1 (division) → RecordDivision → [Dispose](#)

[StochasticTranscription](#): Poisson tau-leaping for 10 representative *M. genitalium* genes [StochasticTranslation](#) ↔ : Poisson tau-leaping mapping mRNA_ → prot_ species [StochasticReactionComponent](#): Gillespie SSA for ATP/↔ GTP metabolism + FtsZ ring polymerization [CellDivisionEvent](#): Triggers when FtsZ ring reaches 35% completion (350/1000 units) (mass-only trigger disabled; FtsZ-driven as in Karr et al.)

Biologically realistic parameters (from CovertLab/WholeCell parameters.json, MIT): RNAP_free = 200 molecules (Karr et al. 2012, Supplementary Table S2) ribosome_free = 400 molecules Transcription rate = 50 nt/s Translation rate = 16 AA/s Mean gene length = 900 nt Mean protein len = 300 AA [Cell](#) cycle ≈ 600 min (modeled as 600 × 60 s steps)

10 representative genes (selected from *M. genitalium* 473-gene model): MG_001 — metabolic (glycolytic enzyme) MG_006 — ATP synthase subunit (energy metabolism) MG_055 — DNA polymerase III subunit alpha (DNA replication) MG_139 — RNAP beta subunit (transcription machinery) MG_175 — 30S ribosomal protein S1 (translation machinery) MG_196 — FtsZ (cell division; drives the division trigger) MG_203 — RNAP sigma factor (transcription regulation) MG_339 — phosphoglycerate mutase (central metabolism) MG_398 — pyruvate kinase (central metabolism) MG_462 — ribosomal protein L7/L12 (translation machinery)

Simulation: 3 replications × 36000 s (10 h ≈ 1 *M. genitalium* cell cycle), 60 s steps.

Run via GENESYS_TERMINAL_EXAMPLE: `cmake -DGENESYS_TERMINAL_EXAMPLE=wcm/Wcm↔ MgenitaliumKarr2012.cpp ...`

10.830.2 Member Function Documentation

10.830.2.1 main()

```
int WcmMgenitaliumKarr2012::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

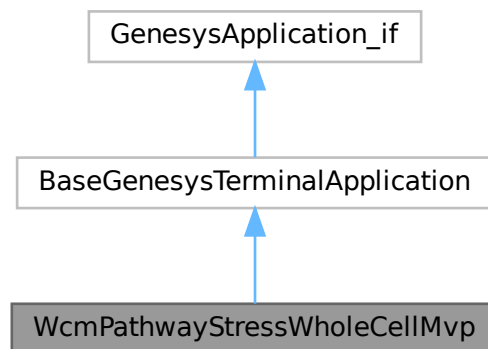
- applications/modelSpecific/wcm/WcmMgenitaliumKarr2012.h
- applications/modelSpecific/wcm/WcmMgenitaliumKarr2012.cpp

10.831 WcmPathwayStressWholeCellMvp Class Reference

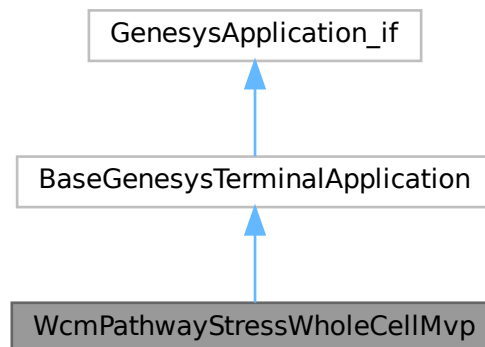
Whole-cell stress example driven by sustained low pathway activity.

```
#include <applications/modelSpecific/wcm/WcmPathwayStressWholeCellMvp.h>
```

Inheritance diagram for WcmPathwayStressWholeCellMvp:



Collaboration diagram for WcmPathwayStressWholeCellMvp:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.831.1 Detailed Description

Whole-cell stress example driven by sustained low pathway activity.

This example keeps metabolism active but configures a monitored pathway threshold above the achievable FBA objective. The result is an arrest-then- death sequence driven by [PathwayStressResponseComponent](#).

10.831.2 Member Function Documentation

10.831.2.1 main()

```
int WcmPathwayStressWholeCellMvp::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

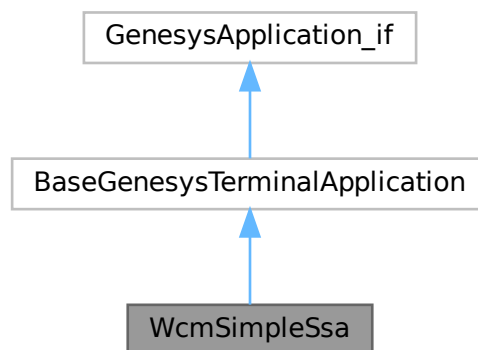
The documentation for this class was generated from the following files:

- applications/modelSpecific/wcm/WcmPathwayStressWholeCellMvp.h
- applications/modelSpecific/wcm/WcmPathwayStressWholeCellMvp.cpp

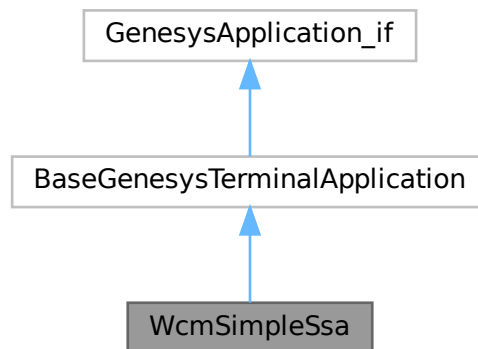
10.832 WcmSimpleSsa Class Reference

```
#include <applications/modelSpecific/wcm/WcmSimpleSsa.h>
```

Inheritance diagram for WcmSimpleSsa:



Collaboration diagram for WcmSimpleSsa:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.832.1 Detailed Description

Minimal whole-cell modeling example using the Gillespie Direct Method SSA.

Models stochastic gene expression of a single gene via 4 reactions: R1: transcription — $\text{DNA} + \text{RNAP} \rightarrow \text{DNA} + \text{RNAP} + \text{mRNA_geneA}$ ($k = 0.02/\text{s}$) R2: mRNA degradation — $\text{mRNA_geneA} \rightarrow \emptyset$ ($k = 0.01/\text{s}$) R3: translation — $\text{mRNA_geneA} \rightarrow \text{mRNA_geneA} + \text{prot_geneA}$ ($k = 0.10/\text{s}$) R4: protein degradation — $\text{prot_geneA} \rightarrow \emptyset$ ($k = 0.001/\text{s}$)

Steady-state analytic predictions: $\text{mRNA_geneA} \approx 2$ molecules ($k_{\text{tx}} / k_{\text{deg_mRNA}} = 0.02 / 0.01$) $\text{prot_geneA} \approx 200$ molecules ($k_{\text{tl}} * \text{mRNA_ss} / k_{\text{deg_prot}} = 0.1 * 2 / 0.001$)

Simulation runs 30 replications $\times$ 3600 s each (1 hour of biological time) in 60 s Gillespie windows triggered by periodic clock entities.

Run via GENESYS_TERMINAL_EXAMPLE: `cmake -DGENESYS_TERMINAL_EXAMPLE=wcm/WcmSimpleSsa.cpp ...`

10.832.2 Member Function Documentation

10.832.2.1 main()

```
int WcmSimpleSsa::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

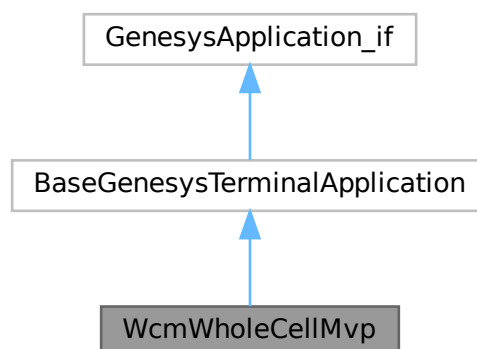
- applications/modelSpecific/wcm/WcmSimpleSsa.h
- applications/modelSpecific/wcm/WcmSimpleSsa.cpp

10.833 WcmWholeCellMvp Class Reference

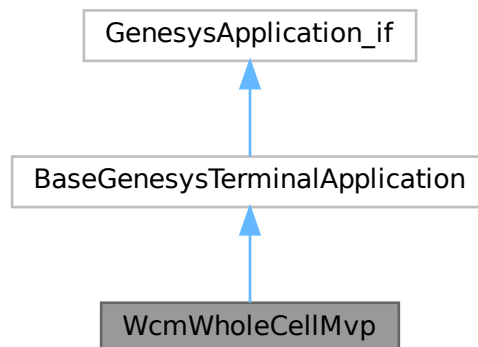
Minimal whole-cell MVP combining metabolism, expression, growth, and division.

```
#include <applications/modelSpecific/wcm/WcmWholeCellMvp.h>
```

Inheritance diagram for WcmWholeCellMvp:



Collaboration diagram for WcmWholeCellMvp:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.833.1 Detailed Description

Minimal whole-cell MVP combining metabolism, expression, growth, and division.

This example is intentionally small and didactic. It combines:

- deterministic biochemical metabolism through [BioNetwork](#);
- runtime projection into [WholeCellState](#);
- stochastic transcription and translation;
- energy-gated mass growth;
- mass-triggered cell division.

The result is not a calibrated organism model. It is a compact whole-cell analog intended to validate the GenESyS architecture for hybrid deterministic/stochastic cell simulations.

10.833.2 Member Function Documentation

10.833.2.1 main()

```
int WcmWholeCellMvp::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

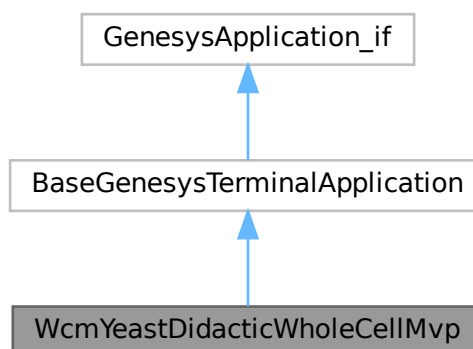
- applications/modelSpecific/wcm/WcmWholeCellMvp.h
- applications/modelSpecific/wcm/WcmWholeCellMvp.cpp

10.834 WcmYeastDidacticWholeCellMvp Class Reference

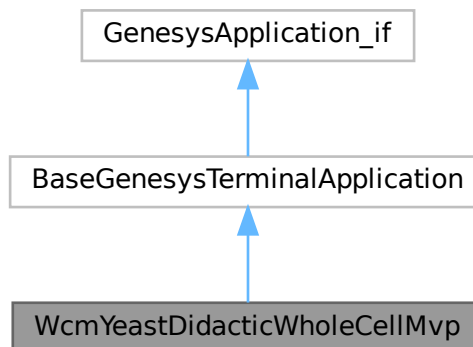
Didactic yeast-oriented whole-cell MVP built on GEM-inspired metabolism.

```
#include <applications/modelSpecific/wcm/WcmYeastDidacticWholeCellMvp.h>
```

Inheritance diagram for WcmYeastDidacticWholeCellMvp:



Collaboration diagram for WcmYeastDidacticWholeCellMvp:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.834.1 Detailed Description

Didactic yeast-oriented whole-cell MVP built on GEM-inspired metabolism.

This example is intentionally small and pedagogical. It combines a compact FBA network inspired by *Saccharomyces cerevisiae* carbon metabolism with whole-cell state projection, compartment exchange, stochastic expression, growth, lifecycle checkpointing, fate routing, and division.

It is not a canonical or validated yeast whole-cell model. It should be interpreted as a GenESyS architecture example informed by yeast-GEM style metabolic concepts, not as a public operational WCM for *S. cerevisiae*.

10.834.2 Member Function Documentation

10.834.2.1 main()

```
int WcmYeastDidacticWholeCellMvp::main (
    int argc,
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

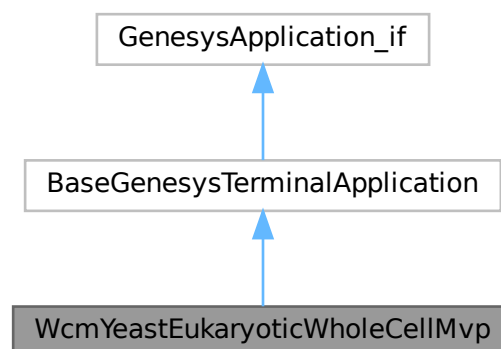
- applications/modelSpecific/wcm/WcmYeastDidacticWholeCellMvp.h
- applications/modelSpecific/wcm/WcmYeastDidacticWholeCellMvp.cpp

10.835 WcmYeastEukaryoticWholeCellMvp Class Reference

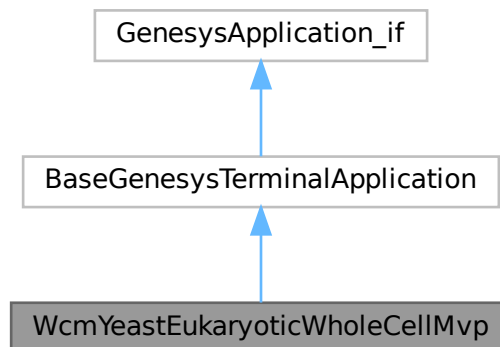
Didactic eukaryotic whole-cell MVP inspired by budding yeast.

```
#include <applications/modelSpecific/wcm/WcmYeastEukaryoticWholeCellMvp.h>
```

Inheritance diagram for WcmYeastEukaryoticWholeCellMvp:



Collaboration diagram for WcmYeastEukaryoticWholeCellMvp:



Public Member Functions

- virtual int [main](#) (int argc, char **argv) override

Public Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **Simulate** ([Simulator](#) *genesys, [Model](#) *model, [PluginManager](#) *plugins)
- void **setDefaultTraceHandlers** ([TraceManager](#) *tm)
- void **setDefaultEventHandlers** ([OnEventManager](#) *oem)

Additional Inherited Members

Protected Member Functions inherited from [BaseGenesysTerminalApplication](#)

- virtual void **traceHandler** ([TraceEvent](#) e)
- virtual void **traceErrorHandler** ([TraceErrorEvent](#) e)
- virtual void **traceReportHandler** ([TraceEvent](#) e)
- virtual void **traceSimulationHandler** ([TraceSimulationEvent](#) e)
- virtual void **onAfterProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onBreakpointHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityCreateHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityMoveHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStepHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationStartHandler** ([SimulationEvent](#) *re)
- virtual void **onProcessEventHandler** ([SimulationEvent](#) *re)
- virtual void **onReplicationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationEndHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationPausedHandler** ([SimulationEvent](#) *re)
- virtual void **onSimulationResumeHandler** ([SimulationEvent](#) *re)
- virtual void **onEntityRemoveHandler** ([SimulationEvent](#) *re)

10.835.1 Detailed Description

Didactic eukaryotic whole-cell MVP inspired by budding yeast.

This example combines:

- a compact GEM-inspired FBA core;
- explicit biological compartments;
- compartment exchange and organelle energy proxies;
- stochastic expression and growth; and
- a simplified eukaryotic cell-cycle coordinator.

It is a didactic whole-cell example for *Saccharomyces cerevisiae*-inspired architecture. It is not a canonical public yeast whole-cell model.

10.835.2 Member Function Documentation

10.835.2.1 main()

```
int WcmYeastEukaryoticWholeCellMvp::main (  
    int argc,  
    char ** argv ) [override], [virtual]
```

Reimplemented from [BaseGenesysTerminalApplication](#).

The documentation for this class was generated from the following files:

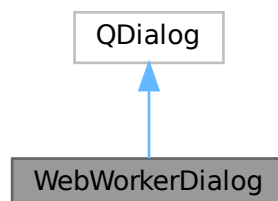
- applications/modelSpecific/wcm/WcmYeastEukaryoticWholeCellMvp.h
- applications/modelSpecific/wcm/WcmYeastEukaryoticWholeCellMvp.cpp

10.836 WebWorkerDialog Class Reference

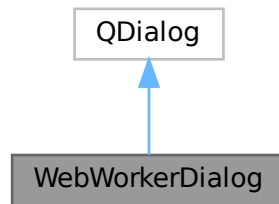
Lightweight control window for the embedded GenESyS web worker.

```
#include <applications/gui/httpworker/WebWorkerDialog.h>
```

Inheritance diagram for WebWorkerDialog:



Collaboration diagram for WebWorkerDialog:



Public Member Functions

- **WebWorkerDialog** (QWidget *parent=nullptr, bool allowClose=false)

Protected Member Functions

- void **closeEvent** (QCloseEvent *event) override
- void **showEvent** (QShowEvent *event) override

10.836.1 Detailed Description

Lightweight control window for the embedded GenESyS web worker.

The documentation for this class was generated from the following files:

- applications/gui/httpworker/WebWorkerDialog.h
- applications/gui/httpworker/WebWorkerDialog.cpp

10.837 WebWorkerRuntime Class Reference

Owns the embedded worker runtime used by the GUI and the standalone worker app.

```
#include <applications/worker/service/WebWorkerRuntime.h>
```

Classes

- struct [Snapshot](#)
Captures a thread-safe snapshot of the worker state.

Public Member Functions

- **WebWorkerRuntime** (unsigned short defaultPort=8080, unsigned long maxRequests=0)
Builds the reusable worker runtime with a default port and request cap.
- **~WebWorkerRuntime** ()
Stops the worker and releases background resources.
- bool **start** ()
Starts the worker if it is not already running.
- void **stop** ()
Requests a graceful shutdown and waits for the worker thread to finish.
- void **wait** ()
Waits for the worker thread to complete if it is running.
- bool **restart** ()
Restarts the worker using the currently configured settings.
- bool **isRunning** () const
Returns whether the worker thread is currently serving requests.
- **Snapshot snapshot** () const
Returns a snapshot of the current worker lifecycle and counters.
- void **setConfiguredPort** (unsigned short port)
Updates the configured listening port used by the next start.
- unsigned short **configuredPort** () const
Returns the listening port configured for the next start.
- void **setConfiguredMaxRequests** (unsigned long maxRequests)
Updates the configured request cap used by the next start.
- unsigned long **configuredMaxRequests** () const
Returns the configured request cap used by the next start.

10.837.1 Detailed Description

Owns the embedded worker runtime used by the GUI and the standalone worker app.

The runtime wires the HTTP server, routing stack, and simulator-backed session services into a reusable lifecycle object. GUI code can start, stop, and inspect the worker without embedding HTTP details in the main window.

10.837.2 Member Function Documentation

10.837.2.1 restart()

```
bool WebWorkerRuntime::restart ( )
```

Restarts the worker using the currently configured settings.

Returns

True when the worker was scheduled to start again.

10.837.2.2 start()

```
bool WebWorkerRuntime::start ( )
```

Starts the worker if it is not already running.

Returns

True when a new background worker thread was launched.

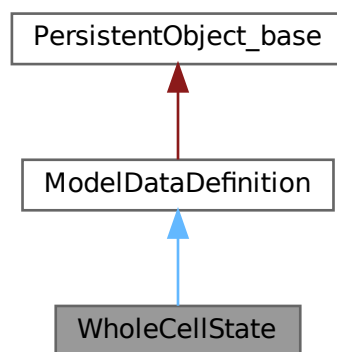
The documentation for this class was generated from the following files:

- applications/worker/service/WebWorkerRuntime.h
- applications/worker/service/WebWorkerRuntime.cpp

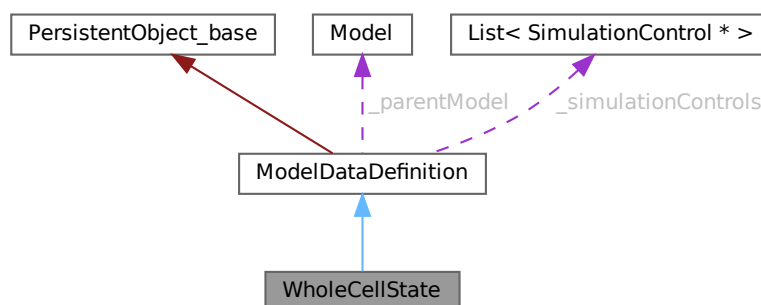
10.838 WholeCellState Class Reference

```
#include <plugins/data/WholeCellModeling/WholeCellState.h>
```

Inheritance diagram for WholeCellState:



Collaboration diagram for WholeCellState:



Public Member Functions

- **WholeCellState** ([Model](#) *model, std::string name="")
- void **setMoleculeCount** (const std::string &speciesName, int count)
- int **getMoleculeCount** (const std::string &speciesName) const
- bool **hasMoleculeCount** (const std::string &speciesName) const
- const std::map< std::string, int > & **getMoleculeCounts** () const
- void **setMetaboliteAmount** (const std::string &metaboliteName, double amount)
- double **getMetaboliteAmount** (const std::string &metaboliteName) const
- const std::map< std::string, double > & **getMetabolitePool** () const
- void **setCompartmentMetaboliteAmount** (const std::string &compartmentName, const std::string &metaboliteName, double amount)
- double **getCompartmentMetaboliteAmount** (const std::string &compartmentName, const std::string &metaboliteName) const
- bool **hasCompartmentMetaboliteAmount** (const std::string &compartmentName, const std::string &metaboliteName) const
- const std::map< std::string, double > & **getCompartmentMetabolitePool** () const
- void **setPathwayActivity** (const std::string &pathwayName, double activity)
- double **getPathwayActivity** (const std::string &pathwayName) const
- bool **hasPathwayActivity** (const std::string &pathwayName) const
- const std::map< std::string, double > & **getPathwayActivities** () const
- void **setResourceBudget** (const std::string &speciesName, int budget)
- int **getResourceBudget** (const std::string &speciesName) const
- void **clearResourceBudget** ()
- void **setCellVolume** (double volume)
- double **getCellVolume** () const
- void **setCellMass** (double mass)
- double **getCellMass** () const
- void **setCurrentTime** (double time)
- double **getCurrentTime** () const
- void **setStepCount** (int stepCount)
- int **getStepCount** () const
- void **incrementStep** ()
- void **setLifecyclePhase** (const std::string &phase)
- std::string **getLifecyclePhase** () const
- void **setGenerationCount** (int generationCount)
- int **getGenerationCount** () const
- void **setViable** (bool viable)
- bool **isViable** () const
- void **setLastDivisionTime** (double time)
- double **getLastDivisionTime** () const
- bool **loadFixedConstants** (const std::string &jsonPath)
- bool **loadParameters** (const std::string &jsonPath)
- void **setFixedConstantsPath** (std::string path)
- std::string **getFixedConstantsPath** () const
- void **setParametersPath** (std::string path)
- std::string **getParametersPath** () const
- virtual std::string **show** () override

Returns a textual representation of this data definition.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()

Releases resources owned by this data definition.
- Util::identification [getId](#) () const

Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)

Updates the logical name of this instance.
- const std::string & [getName](#) () const

Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const

Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const

Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const

Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const

Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const

Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const

Returns the full map of attached data definitions.
- bool [hasChanged](#) () const

Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))

Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const

Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const

Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)

Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelSpecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
must be overridden by derived classes
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_initBetweenReplications** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool **_getSaveDefaultsOption** ()
- virtual [ParserChangesInformation](#) * **_getParserChangesInformation** ()
- virtual void **_addSimulationControl** ([SimulationControl](#) *control)
- virtual void **_createInternalAndAttachedData** ()
- virtual void **_createInternalStatisticReporters** ()
- virtual void **_createAttachedAttributes** ()
- virtual void **_createNonEditableDataDefinitions** ()
- virtual void **_createEditableDataDefinitions** ()
- void **_templateCreateInternalStatisticReporters** ()
- void **_templateCreateAttachedAttributes** ()
- void **_templateCreateNonEditableDataDefinitions** ()
- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
legacy helper methods used by the granular create hooks*
- void **_internalDataClear** ()

- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
not just an easy access to trace manager, but wrappers to check if specific trace level applies
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void **traceSimulation** (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
Traces text to the simulation output.
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification **_id**
- std::string **_typename**
- bool **_reportStatistics**
- bool **_hasChanged**
- unsigned int **_modelLevel** = 0
- [Model](#) * **_parentModel**
- [List](#)< [SimulationControl](#) * > * **_simulationControls** = new [List](#)<[SimulationControl](#)*>()
- TraceManager::Level **_traceLevelSpecific**
- bool **_traceLevelSpecificEnabled** = false

10.838.1 Detailed Description

Shared state container for whole-cell model simulations.

[WholeCellState](#) is the GenESyS analog to the Vivarium [Store](#): a single [ModelDataDefinition](#) instance shared by all WCM component plugins in a simulation step. It holds:

- discrete molecule counts (mRNA, proteins) updated by SSA
- continuous metabolite pool (ATP, amino acids) updated by FBA
- resource budget allocated by fair-allocation before each step
- cell geometry (volume, mass) for propensity scaling

JSON loading (Q5a) reads `fixedConstants.json` and `parameters.json` from the CovertLab/WholeCell M. genitalium repository (MIT license).

10.838.2 Member Function Documentation

10.838.2.1 `_check()`

```
bool WholeCellState::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.838.2.2 `_initBetweenReplications()`

```
void WholeCellState::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.838.2.3 `_loadInstance()`

```
bool WholeCellState::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

must be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.838.2.4 `_saveInstance()`

```
void WholeCellState::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelDataDefinition](#).

10.838.2.5 show()

```
std::string WholeCellState::show ( ) [override], [virtual]
```

Returns a textual representation of this data definition.

Returns the keys of internal ModelDataDefinitions such as counters, statistics collectors and other children needed before model checking.

Returns

Human-readable summary used by traces and debug output.

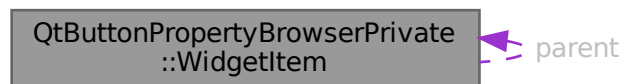
Reimplemented from [ModelDataDefinition](#).

The documentation for this class was generated from the following files:

- plugins/data/WholeCellModeling/WholeCellState.h
- plugins/data/WholeCellModeling/WholeCellState.cpp

10.839 QPushButtonPropertyBrowserPrivate::WidgetItem Struct Reference

Collaboration diagram for QPushButtonPropertyBrowserPrivate::WidgetItem:

**Public Attributes**

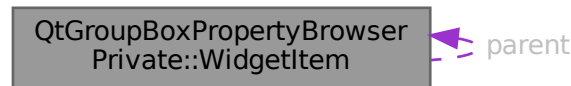
- QWidget * **widget** {nullptr}
- QLabel * **label** {nullptr}
- QLabel * **widgetLabel** {nullptr}
- QToolButton * **button** {nullptr}
- QWidget * **container** {nullptr}
- QGridLayout * **layout** {nullptr}
- [WidgetItem](#) * **parent** {nullptr}
- QList< [WidgetItem](#) * > **children**
- bool **expanded** {false}

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtbuttonpropertybrowser.cpp

10.840 QtGroupBoxPropertyBrowserPrivate::WidgetItem Struct Reference

Collaboration diagram for QtGroupBoxPropertyBrowserPrivate::WidgetItem:



Public Attributes

- QWidget * **widget** {nullptr}
- QLabel * **label** {nullptr}
- QLabel * **widgetLabel** {nullptr}
- QGroupBox * **groupBox** {nullptr}
- QGridLayout * **layout** {nullptr}
- QFrame * **line** {nullptr}
- [WidgetItem](#) * **parent** {nullptr}
- QList< [WidgetItem](#) * > **children**

The documentation for this struct was generated from the following file:

- applications/gui/genesys/propertyeditor/qtpropertybrowser/qtgroupboxpropertybrowser.cpp

10.841 SimulatorSessionService::WorkerCapabilitiesResult Struct Reference

Describes currently supported worker features.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Public Attributes

- bool **supportsSessionApi** = false
- bool **supportsSessionScopedSimulator** = false
- bool **supportsModelCreation** = false
- bool **supportsModelPersistence** = false
- bool **supportsSimulationStatus** = false
- bool **supportsSimulationConfig** = false
- bool **supportsSynchronousRun** = false
- bool **supportsSynchronousStep** = false
- bool **supportsDistributedJobs** = false
- bool **supportsJobPolling** = false
- bool **supportsBackgroundExecution** = false
- bool **supportsModelUpload** = false
- bool **supportsStreamingEvents** = false
- bool **supportsJobResultRetrieval** = false

10.841.1 Detailed Description

Describes currently supported worker features.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

10.842 SimulatorSessionService::WorkerInfoResult Struct Reference

Describes worker identity metadata for distributed discovery.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Public Attributes

- std::string **role**
- std::string **application**
- std::string **apiFamily**
- std::string **apiVersion**
- std::string **simulatorName**
- std::string **simulatorVersionName**
- unsigned int **simulatorVersionNumber** = 0

10.842.1 Detailed Description

Describes worker identity metadata for distributed discovery.

The documentation for this struct was generated from the following file:

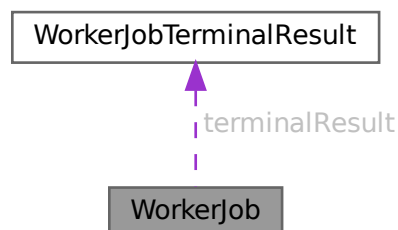
- applications/worker/service/SimulatorSessionService.h

10.843 WorkerJob Struct Reference

Stores metadata for a worker job snapshot owned by the web worker.

```
#include <applications/worker/worker/WorkerJob.h>
```

Collaboration diagram for WorkerJob:



Public Attributes

- `std::string jobId`
Stable job identifier returned to API clients.
- `std::string sessionId`
Session that owns the job snapshot and execution context.
- `WorkerJobState state = WorkerJobState::Queued`
Current lifecycle state of the job.
- `std::string snapshotFilename`
Snapshot filename stored in the session workspace.
- `std::string createdMarker`
Monotonic creation marker used to preserve job ordering.
- `std::string message`
Human-readable status or error text.
- `bool hasTerminalResult = false`
True when terminalResult contains valid completion data.
- [WorkerJobTerminalResult](#) `terminalResult`
Final simulation summary written when the job finishes.

10.843.1 Detailed Description

Stores metadata for a worker job snapshot owned by the web worker.

The documentation for this struct was generated from the following file:

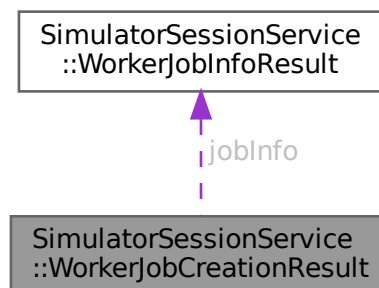
- `applications/worker/worker/WorkerJob.h`

10.844 SimulatorSessionService::WorkerJobCreationResult Struct Reference

Contains worker job creation output and error state.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::WorkerJobCreationResult:



Public Attributes

- bool **success** = false
- [WorkerJobError](#) **error** = WorkerJobError::None
- [WorkerJobInfoResult](#) **jobInfo**

10.844.1 Detailed Description

Contains worker job creation output and error state.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

10.845 SimulatorSessionService::WorkerJobInfoResult Struct Reference

Describes worker job metadata exposed by worker stage 3 endpoints.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Public Attributes

- std::string **jobId**
- std::string **sessionId**
- WorkerJobState **state** = WorkerJobState::Queued
- std::string **snapshotFilename**
- std::string **createdMarker**
- std::string **message**

10.845.1 Detailed Description

Describes worker job metadata exposed by worker stage 3 endpoints.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

10.846 WorkerJobManager Class Reference

Owns worker job records in memory and assigns unique identifiers.

```
#include <applications/worker/worker/WorkerJobManager.h>
```

Public Member Functions

- [WorkerJob](#) `createQueuedJob` (const std::string &sessionId)
Creates and stores a queued worker job with generated identifiers.
- bool `setSnapshotFilename` (const std::string &jobId, const std::string &snapshotFilename)
Replaces the snapshot filename for an existing job.
- bool `setState` (const std::string &jobId, WorkerJobState state)
Updates the lifecycle state for an existing job.
- bool `setMessage` (const std::string &jobId, const std::string &message)
Replaces the message text for an existing job.
- bool `setTerminalResult` (const std::string &jobId, const [WorkerJobTerminalResult](#) &terminalResult)
Persists the terminal simulation summary for an existing job.
- std::optional< [WorkerJob](#) > `getJob` (const std::string &jobId) const
Fetches a previously stored job record.

10.846.1 Detailed Description

Owns worker job records in memory and assigns unique identifiers.

The manager is intentionally simple: it keeps all job metadata in memory and provides synchronized read/write helpers for the web API and background jobs.

10.846.2 Member Function Documentation

10.846.2.1 `createQueuedJob()`

```
WorkerJob WorkerJobManager::createQueuedJob (
    const std::string & sessionId )
```

Creates and stores a queued worker job with generated identifiers.

Creates and stores a queued worker job with a monotonic identifier.

Parameters

<code>sessionId</code>	Identifier of the owning session.
------------------------	-----------------------------------

Returns

Newly stored worker job metadata.

10.846.2.2 `getJob()`

```
std::optional< WorkerJob > WorkerJobManager::getJob (
    const std::string & jobId ) const
```

Fetches a previously stored job record.

Retrieves a copy of a job record when it exists.

Parameters

<i>jobId</i>	Job identifier.
--------------	-----------------

Returns

Copy of the job when found.

10.846.2.3 setMessage()

```
bool WorkerJobManager::setMessage (
    const std::string & jobId,
    const std::string & message )
```

Replaces the message text for an existing job.

Replaces the human-readable message for a stored job.

Parameters

<i>jobId</i>	Job identifier to update.
<i>message</i>	Human-readable status or error message.

Returns

True when the job exists and was updated.

10.846.2.4 setSnapshotFilename()

```
bool WorkerJobManager::setSnapshotFilename (
    const std::string & jobId,
    const std::string & snapshotFilename )
```

Replaces the snapshot filename for an existing job.

Updates the snapshot filename for a stored job.

Parameters

<i>jobId</i>	Job identifier to update.
<i>snapshotFilename</i>	Snapshot filename persisted in session workspace.

Returns

True when the job exists and was updated.

10.846.2.5 setState()

```
bool WorkerJobManager::setState (
    const std::string & jobId,
    WorkerJobState state )
```

Updates the lifecycle state for an existing job.

Updates the lifecycle state for a stored job.

Parameters

<i>jobId</i>	Job identifier to update.
<i>state</i>	New worker job state.

Returns

True when the job exists and was updated.

10.846.2.6 setTerminalResult()

```
bool WorkerJobManager::setTerminalResult (
    const std::string & jobId,
    const WorkerJobTerminalResult & terminalResult )
```

Persists the terminal simulation summary for an existing job.

Stores the terminal simulation result and marks it available.

Parameters

<i>jobId</i>	Job identifier to update.
<i>terminalResult</i>	Terminal simulation summary values.

Returns

True when the job exists and was updated.

The documentation for this class was generated from the following files:

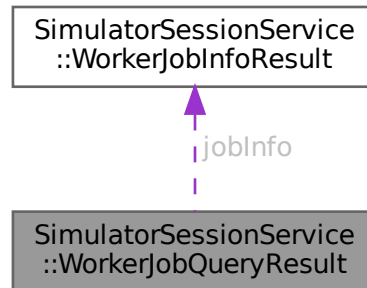
- applications/worker/worker/WorkerJobManager.h
- applications/worker/worker/WorkerJobManager.cpp

10.847 SimulatorSessionService::WorkerJobQueryResult Struct Reference

Contains worker job lookup output and error state.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::WorkerJobQueryResult:



Public Attributes

- bool **success** = false
- [WorkerJobError](#) **error** = WorkerJobError::None
- [WorkerJobInfoResult](#) **jobInfo**

10.847.1 Detailed Description

Contains worker job lookup output and error state.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

10.848 SimulatorSessionService::WorkerJobResultInfo Struct Reference

Describes terminal worker job result data exposed by worker stage 5 endpoint.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Public Attributes

- std::string **jobId**
- WorkerJobState **state** = WorkerJobState::Queued
- std::string **message**
- double **simulatedTime** = 0.0
- unsigned int **currentReplicationNumber** = 0
- unsigned int **numberOfReplications** = 0
- double **replicationLength** = 0.0
- double **warmUpPeriod** = 0.0
- bool **hasIsPaused** = false
- bool **isPaused** = false

10.848.1 Detailed Description

Describes terminal worker job result data exposed by worker stage 5 endpoint.

The documentation for this struct was generated from the following file:

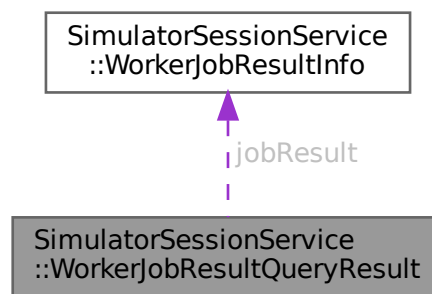
- applications/worker/service/SimulatorSessionService.h

10.849 SimulatorSessionService::WorkerJobResultQueryResult Struct Reference

Contains worker job terminal result lookup output and error state.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::WorkerJobResultQueryResult:



Public Attributes

- bool **success** = false
- [WorkerJobError](#) **error** = WorkerJobError::None
- [WorkerJobResultInfo](#) **jobResult**

10.849.1 Detailed Description

Contains worker job terminal result lookup output and error state.

The documentation for this struct was generated from the following file:

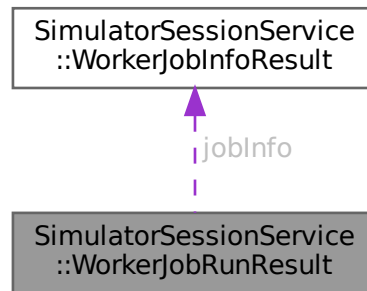
- applications/worker/service/SimulatorSessionService.h

10.850 SimulatorSessionService::WorkerJobRunResult Struct Reference

Contains worker job run output and error state.

```
#include <applications/worker/service/SimulatorSessionService.h>
```

Collaboration diagram for SimulatorSessionService::WorkerJobRunResult:



Public Attributes

- bool **success** = false
- [WorkerJobError](#) **error** = WorkerJobError::None
- [WorkerJobInfoResult](#) **jobInfo**

10.850.1 Detailed Description

Contains worker job run output and error state.

The documentation for this struct was generated from the following file:

- applications/worker/service/SimulatorSessionService.h

10.851 WorkerJobTerminalResult Struct Reference

Stores the terminal simulation summary persisted for a finished or failed worker job.

```
#include <applications/worker/worker/WorkerJob.h>
```

Public Attributes

- double **simulatedTime** = 0.0
Simulated time reached when the job completed.
- unsigned int **currentReplicationNumber** = 0
Current replication number at completion.
- unsigned int **numberOfReplications** = 0
Total number of replications configured for the run.
- double **replicationLength** = 0.0
Replication length used by the run.
- double **warmUpPeriod** = 0.0
Warm-up period used by the run.
- std::optional< bool > **isPaused**
Pause state when the simulator explicitly reported it.

10.851.1 Detailed Description

Stores the terminal simulation summary persisted for a finished or failed worker job.

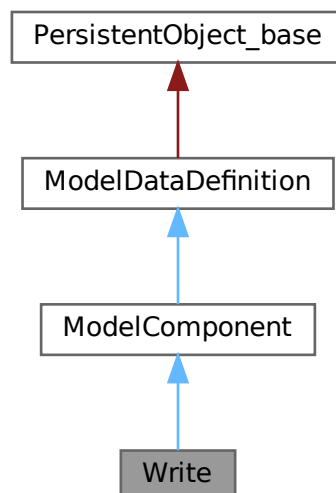
The documentation for this struct was generated from the following file:

- applications/worker/worker/WorkerJob.h

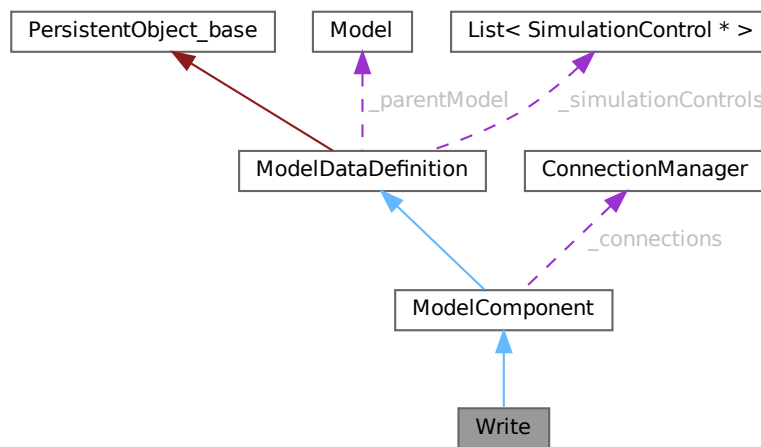
10.852 Write Class Reference

```
#include <plugins/components/InputOutput/Write.h>
```

Inheritance diagram for Write:



Collaboration diagram for Write:



Public Types

- enum class **WriteToType** : int { **SCREEN** = 0 , **FILE** = 1 , **num_elements** = 2 }

Public Member Functions

- **Write** (**Model** *model, std::string name="")
- void **insertText** (std::list< std::string > texts)
- void **setFilename** (std::string _filename)
- std::string **filename** () const
- void **setWriteToType** (WriteToType _writeToType)
- WriteToType **writeToType** () const
- virtual std::string **show** () override

virtual public methods

Public Member Functions inherited from **ModelComponent**

- **ModelComponent** (**Model** *model, std::string componentTypename, std::string name="")
constructors
- virtual ~**ModelComponent** ()
Releases component-owned resources and unregisters it from the model.
- **ConnectionManager** * **getConnectionManager** () const
new public user methods for this component
- void **connectTo** (**ModelComponent** *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool **hasBreakpointAt** ()
Indicates whether a simulation breakpoint is set on this component.
- void **setDescription** (std::string _description)
Updates the free-form description shown by the GUI.
- std::string **getDescription** () const
Returns the free-form description associated with this component.
- void **remove** (**ModelComponent** *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)

Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()

Releases resources owned by this data definition.
- Util::identification [getId](#) () const

Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)

Updates the logical name of this instance.
- const std::string & [getName](#) () const

Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)

Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const

Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const

Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const

Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const

Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)

Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const

Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const

Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const

Returns the full map of attached data definitions.
- bool [hasChanged](#) () const

Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))

Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const

Returns the level in the model hierarchy where this definition lives.
- void [setModelLevel](#) (unsigned int _modelLevel)

Sets the model nesting level for this definition.
- List< [SimulationControl](#) * > * [getSimulationControls](#) () const

Returns the simulation controls owned by this data definition.
- TraceManager::Level [getTraceLevelSpecific](#) () const

Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) (TraceManager::Level level)

Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) (TraceManager::Level traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)

Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const

Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)

Enables or disables the specific trace level.

Static Public Member Functions

- static std::string **convertEnumToStr** (WriteToType type)
- static [PluginInformation](#) * **GetPluginInformation** ()
- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool **Check** ([ModelComponent](#) *component)
Validates a component instance.
- static void **CreateInternalData** ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void **DispatchEvent** ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * **LoadInstance** ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * **NewInstance** ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void **SaveInstance** ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool **Check** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void **InitBetweenReplications** ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void **CreateInternalData** ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool **CreateRelatedDataElements** ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual void **_onDispatchEvent** ([Entity](#) *entity, unsigned int inputPortNumber) override
Handles a dispatched event for this component type.
- virtual bool **_loadInstance** ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void **_saveInstance** ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.
- virtual bool **_check** (std::string &errorMessage) override
could be overridden by derived classes
- virtual void **_initBetweenReplications** () override

Protected Member Functions inherited from [ModelDataDefinition](#)

- `bool _getSaveDefaultsOption ()`
- `virtual ParserChangesInformation * _getParserChangesInformation ()`
- `virtual void _addSimulationControl (SimulationControl *control)`
- `virtual void _createInternalAndAttachedData ()`
- `virtual void _createInternalStatisticReporters ()`
- `virtual void _createAttachedAttributes ()`
- `virtual void _createNonEditableDataDefinitions ()`
- `virtual void _createEditableDataDefinitions ()`
- `void _templateCreateInternalStatisticReporters ()`
- `void _templateCreateAttachedAttributes ()`
- `void _templateCreateNonEditableDataDefinitions ()`
- `void _templateCreateEditableDataDefinitions ()`
- `void _internalStatisticReportersClear ()`
- legacy helper methods used by the granular create* hooks*
- `void _internalDataClear ()`
- `void _internalDataInsert (const std::string &key, ModelDataDefinition *child)`
- `void _internalDataRemove (const std::string &key)`
- `void _attachedDataClear ()`
- `void _attachedDataInsert (const std::string &key, ModelDataDefinition *data)`
- `void _attachedDataRemove (const std::string &key)`
- `void _attachedAttributesInsert (const std::vector< std::string > &neededNames)`
- `void _statisticReporterInsert (const std::string &key, ModelDataDefinition *data)`
- `void _statisticReporterRemove (const std::string &key)`
- `void _statisticReportersClear ()`
- `void _mandatoryAttachedAttributeInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryAttachedAttributeRemove (const std::string &key)`
- `void _mandatoryAttachedAttributesClear ()`
- `void _mandatoryEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryEditableDataDefinitionsClear ()`
- `void _optionalEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalEditableDataDefinitionsClear ()`
- `void _mandatoryNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _mandatoryNonEditableDataDefinitionRemove (const std::string &key)`
- `void _mandatoryNonEditableDataDefinitionsClear ()`
- `void _optionalNonEditableDataDefinitionInsert (const std::string &key, ModelDataDefinition *data)`
- `void _optionalNonEditableDataDefinitionRemove (const std::string &key)`
- `void _optionalNonEditableDataDefinitionsClear ()`
- `void _checkCreateAttachedReferencedDataDefinition (const std::string &expression)`
- method to be called to insert attached data elements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- `void trace (const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- `void traceError (const std::string &text, const std::exception &e)`
- `void traceError (const std::string &text, TraceManager::Level level=TraceManager::Level::L1\_errorFatal)`
- `void traceReport (const std::string &text, TraceManager::Level level=TraceManager::Level::L2\_results)`
- `void traceSimulation (void *thisobject, double time, Entity *entity, ModelComponent *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- `void traceSimulation (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8\_detailed)`
- Traces text to the simulation output.*
- `void traceSimulation (void *thisobject, TraceManager::Level level, const std::string &text)`

Additional Inherited Members

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.852.1 Detailed Description

This component ...

10.852.2 Member Function Documentation

10.852.2.1 `_check()`

```
bool Write::_check (
    std::string & errorMessage ) [override], [protected], [virtual]
```

could be overridden by derived classes

Reimplemented from [ModelDataDefinition](#).

10.852.2.2 `_initBetweenReplications()`

```
void Write::_initBetweenReplications ( ) [override], [protected], [virtual]
```

Reimplemented from [ModelDataDefinition](#).

10.852.2.3 `_loadInstance()`

```
bool Write::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [ModelComponent](#).

10.852.2.4 `_onDispatchEvent()`

```
void Write::_onDispatchEvent (
    Entity * entity,
    unsigned int inputPortNumber ) [override], [protected], [virtual]
```

Handles a dispatched event for this component type.

This method is only for ModelComponents, not for ModelDataElements.

Parameters

<i>entity</i>	Entity being processed.
<i>inputPortNumber</i>	Input port where the entity arrived.

Implements [ModelComponent](#).

10.852.2.5 _saveInstance()

```
void Write::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [ModelComponent](#).

10.852.2.6 show()

```
std::string Write::show ( ) [override], [virtual]
```

virtual public methods

Returns a textual representation of this component.

Returns

Human-readable description used for tracing and debugging.

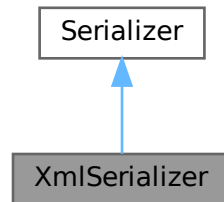
Reimplemented from [ModelComponent](#).

The documentation for this class was generated from the following files:

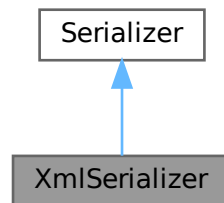
- plugins/components/InOutOutput/Write.h
- plugins/components/InOutOutput/Write.cpp

10.853 XmlSerializer Class Reference

Inheritance diagram for XmlSerializer:



Collaboration diagram for XmlSerializer:



Public Member Functions

- **XmlSerializer** ([Model](#) *model)
- [PersistenceRecord](#) * [newPersistenceRecord](#) () override
Creates an empty persistence record.
- bool [dump](#) (std::ostream &output) override
Writes formatted contents to a given output stream.
- bool [load](#) (std::istream &input) override
Attempts to load contents from a given input stream.
- bool [get](#) (const std::string &name, [PersistenceRecord](#) *entry) override
Tries to fetch an entry with the given label as a key.
- bool [put](#) (const std::string name, const std::string type, const Util::identification id, [PersistenceRecord](#) *fields) override
Upserts an entry into this structure.
- int [for_each](#) (std::function< int(const std::string &) > delegate) override
Iterates over this structure's entries, ordered by ID.

10.853.1 Member Function Documentation

10.853.1.1 dump()

```
bool XmlSerializer::dump (
    std::ostream & output ) [override], [virtual]
```

Writes formatted contents to a given output stream.

Parameters

<i>output</i>	output stream to write to.
---------------	----------------------------

Returns

whether the current state of this structure's contents was deemed valid for serialization.

Implements [Serializer](#).

10.853.1.2 for_each()

```
int XmlSerializer::for_each (
    std::function< int(const std::string &) > delegate ) [override], [virtual]
```

Iterates over this structure's entries, ordered by ID.

Parameters

<i>delegate</i>	delegate which is applied to every entry-identifying key, until it returns a non-zero value.
-----------------	--

Returns

either zero, or the first non-zero value returned by the delegate.

Implements [Serializer](#).

10.853.1.3 get()

```
bool XmlSerializer::get (
    const std::string & name,
    PersistenceRecord * entry ) [override], [virtual]
```

Tries to fetch an entry with the given label as a key.

Parameters

<i>name</i>	a label which uniquely identifies an entry.
<i>entry</i>	(out-param) a pointer to a caller-owned entry, which is filled on success.

Returns

whether the entry was found and the out-param was actually written to.

Implements [Serializer](#).

10.853.1.4 load()

```
bool XmlSerializer::load (
    std::istream & input ) [override], [virtual]
```

Attempts to load contents from a given input stream.

Returns

whether the input was well-formatted and loading was successful.

Implements [Serializer](#).

10.853.1.5 newPersistenceRecord()

```
PersistenceRecord * XmlSerializer::newPersistenceRecord ( ) [override], [virtual]
```

Creates an empty persistence record.

Although the returned record can be freed normally with `delete`, care must be taken to only use records which come from a implementation, when configuring that implementation's contents.

Implements [Serializer](#).

10.853.1.6 put()

```
bool XmlSerializer::put (
    const std::string name,
    const std::string type,
    const Util::identification id,
    PersistenceRecord * fields ) [override], [virtual]
```

Upserts an entry into this structure.

Parameters

<i>name</i>	a label which uniquely identifies the entry.
<i>type</i>	a type name.
<i>id</i>	a component/datadef ID number.
<i>fields</i>	a persistence record for the entry, whose contents will be copied if valid.

Returns

whether the entry was valid and was actually copied into this structure.

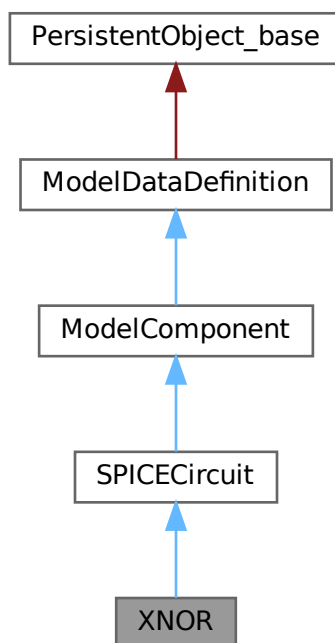
Implements [Serializer](#).

The documentation for this class was generated from the following files:

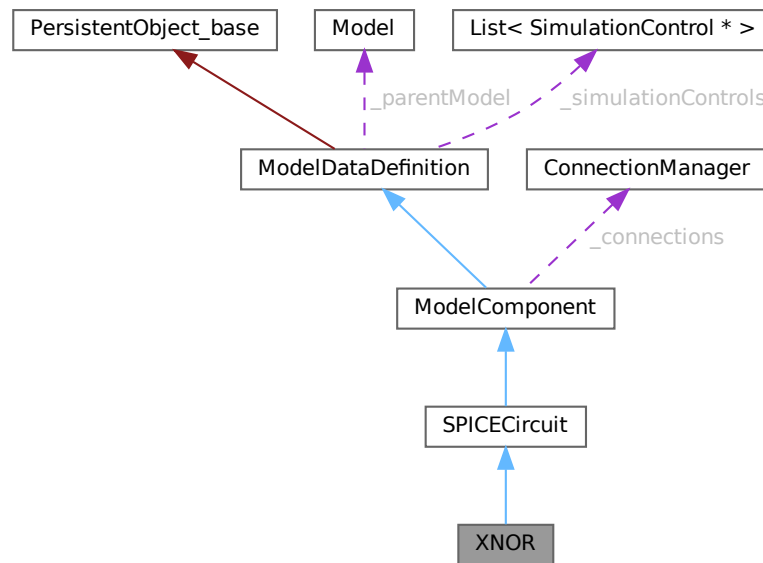
- kernel/simulator/persistence/XmlSerializer.h
- kernel/simulator/persistence/XmlSerializer.cpp

10.854 XNOR Class Reference

Inheritance diagram for XNOR:



Collaboration diagram for XNOR:



Public Member Functions

- **XNOR** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setPMOSWidth** (float PMOSWidth)
- void **setPMOSLength** (float length)
- void **setNMOSWidth** (float NMOSWidth)
- void **setNMOSLength** (float NMOSLength)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.

- [ConnectionManager](#) * [getConnectionManager](#) () const
new public user methods for this component
- void [connectTo](#) ([ModelComponent](#) *component, unsigned int inputPortNumber=0)
Connects this component to another component.
- bool [hasBreakpointAt](#) ()
Indicates whether a simulation breakpoint is set on this component.
- void [setDescription](#) (std::string _description)
Updates the free-form description shown by the GUI.
- std::string [getDescription](#) () const
Returns the free-form description associated with this component.
- void [remove](#) ([ModelComponent](#) *comp)
Removes a connected component from this component.

Public Member Functions inherited from [ModelDataDefinition](#)

- [ModelDataDefinition](#) ([Model](#) *model, std::string datadefinitionTypename, std::string name="", bool insert↵
IntoModel=true)
Creates a model data definition and optionally inserts it into the model.
- virtual ~[ModelDataDefinition](#) ()
Releases resources owned by this data definition.
- Util::identification [getId](#) () const
Returns the unique identifier of this instance.
- void [setName](#) (const std::string &name)
Updates the logical name of this instance.
- const std::string & [getName](#) () const
Returns the logical name of this instance.
- void [setLabel](#) (const std::string &label)
Updates the user-visible label of this instance.
- const std::string & [getLabel](#) () const
Returns the user-visible label of this instance.
- const std::string & [getClassname](#) () const
Returns the runtime class name used by the plugin system.
- [Model](#) * [getParentModel](#) () const
Returns the parent model that owns this definition.
- bool [isReportStatistics](#) () const
Indicates whether this data definition contributes report statistics.
- void [setReportStatistics](#) (bool reportStatistics)
Enables or disables report-statistics generation for this definition.
- [ModelDataDefinition](#) * [getInternalData](#) (const std::string &name) const
Looks up a named internal child data definition.
- std::map< std::string, [ModelDataDefinition](#) * > * [getInternalData](#) () const
Returns the full map of internal child data definitions.
- std::map< std::string, [ModelDataDefinition](#) * > * [getAttachedData](#) () const
Returns the full map of attached data definitions.
- bool [hasChanged](#) () const
Indicates whether this instance has been modified since the last reset.
- void [setHasChanged](#) (bool [hasChanged](#))
Updates the changed flag for this data definition.
- unsigned int [getLevel](#) () const
Returns the level in the model hierarchy where this definition lives.

- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List< SimulationControl * > * getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelspecific, bool traceLevelSpecific↵ Enabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation * GetPluginInformation](#) ()
- static [ModelComponent * LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition * NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation * GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent * LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition * NewInstance](#) ([Model](#) *model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent * LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()

- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
 - legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.854.1 Member Function Documentation

10.854.1.1 `_loadInstance()`

```
bool XNOR::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.854.1.2 `_saveInstance()`

```
void XNOR::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.854.1.3 `Build()`

```
void XNOR::Build ( ) [override], [virtual]
```

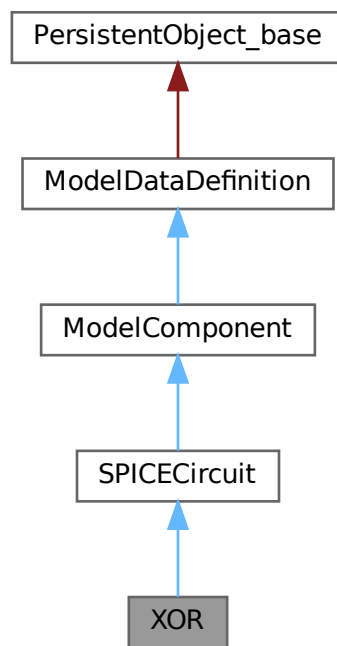
Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

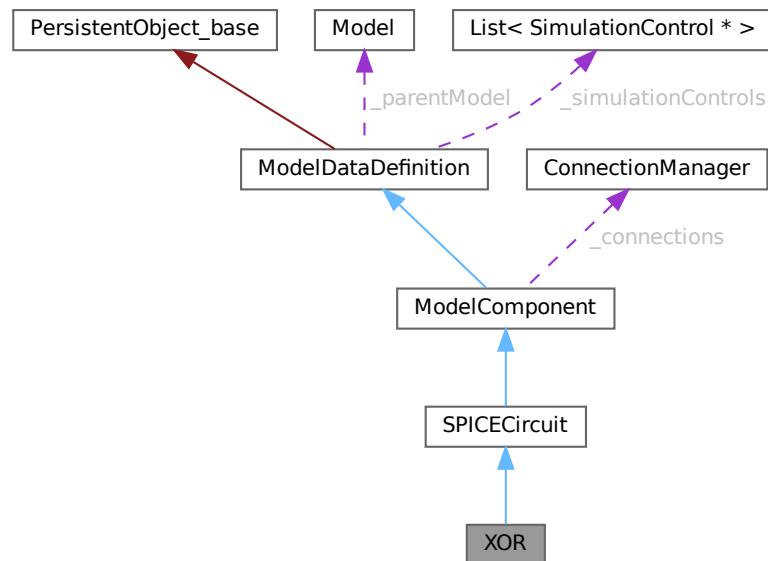
- `plugins/components/ElectronicsSimulation/SPIECEircuit.h`
- `plugins/components/ElectronicsSimulation/SPIECEircuit.cpp`

10.855 XOR Class Reference

Inheritance diagram for XOR:



Collaboration diagram for XOR:



Public Member Functions

- **XOR** ([Model](#) *model, std::string name="")
- void **Build** () override
- void **setPMOSWidth** (float PMOSWidth)
- void **setPMOSLength** (float length)
- void **setNMOSWidth** (float NMOSWidth)
- void **setNMOSLength** (float NMOSLength)
- void **setElectricalModel** (std::string electricalModel)

Public Member Functions inherited from [SPICECircuit](#)

- **SPICECircuit** ([Model](#) *model, std::string name="")
constructors
- void **setRunner** ([SPICERunner](#) *runner)
new public user methods for this components
- [SPICERunner](#) * **getRunner** ()
- virtual std::string **show** () override
virtual public methods

Public Member Functions inherited from [ModelComponent](#)

- [ModelComponent](#) ([Model](#) *model, std::string componentTypename, std::string name="")
constructors
- virtual **~ModelComponent** ()
Releases component-owned resources and unregisters it from the model.

- `ConnectionManager * getConnectionManager ()` const
new public user methods for this component
- `void connectTo (ModelComponent *component, unsigned int inputPortNumber=0)`
Connects this component to another component.
- `bool hasBreakpointAt ()`
Indicates whether a simulation breakpoint is set on this component.
- `void setDescription (std::string _description)`
Updates the free-form description shown by the GUI.
- `std::string getDescription ()` const
Returns the free-form description associated with this component.
- `void remove (ModelComponent *comp)`
Removes a connected component from this component.

Public Member Functions inherited from `ModelDataDefinition`

- `ModelDataDefinition (Model *model, std::string datadefinitionTypename, std::string name="", bool insert↵ IntoModel=true)`
Creates a model data definition and optionally inserts it into the model.
- `virtual ~ModelDataDefinition ()`
Releases resources owned by this data definition.
- `Util::identification getId ()` const
Returns the unique identifier of this instance.
- `void setName (const std::string &name)`
Updates the logical name of this instance.
- `const std::string & getName ()` const
Returns the logical name of this instance.
- `void setLabel (const std::string &label)`
Updates the user-visible label of this instance.
- `const std::string & getLabel ()` const
Returns the user-visible label of this instance.
- `const std::string & getClassname ()` const
Returns the runtime class name used by the plugin system.
- `Model * getParentModel ()` const
Returns the parent model that owns this definition.
- `bool isReportStatistics ()` const
Indicates whether this data definition contributes report statistics.
- `void setReportStatistics (bool reportStatistics)`
Enables or disables report-statistics generation for this definition.
- `ModelDataDefinition * getInternalData (const std::string &name)` const
Looks up a named internal child data definition.
- `std::map< std::string, ModelDataDefinition * > * getInternalData ()` const
Returns the full map of internal child data definitions.
- `std::map< std::string, ModelDataDefinition * > * getAttachedData ()` const
Returns the full map of attached data definitions.
- `bool hasChanged ()` const
Indicates whether this instance has been modified since the last reset.
- `void setHasChanged (bool hasChanged)`
Updates the changed flag for this data definition.
- `unsigned int getLevel ()` const
Returns the level in the model hierarchy where this definition lives.

- void [setModelLevel](#) (unsigned int _modelLevel)
Sets the model nesting level for this definition.
- [List< SimulationControl * > * getSimulationControls](#) () const
Returns the simulation controls owned by this data definition.
- [TraceManager::Level getTraceLevelSpecific](#) () const
Returns the trace level that applies specifically to this definition.
- void [setTraceLevelSpecific](#) ([TraceManager::Level](#) level)
Updates the trace level used by this definition.
- void [defineTraceLevelSpecific](#) ([TraceManager::Level](#) traceLevelSpecific, bool traceLevelSpecificEnabled=true)
Defines the specific trace level and optionally enables it immediately.
- bool [isTraceLevelSpecificEnabled](#) () const
Indicates whether the specific trace level is enabled.
- void [setTraceLevelSpecificEnabled](#) (bool traceLevelSpecificEnabled)
Enables or disables the specific trace level.

Static Public Member Functions

- static [PluginInformation * GetPluginInformation](#) ()
- static [ModelComponent * LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition * NewInstance](#) ([Model](#) *model, std::string name="")

Static Public Member Functions inherited from [SPICECircuit](#)

- static [PluginInformation * GetPluginInformation](#) ()
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static [ModelComponent * LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
- static [ModelDataDefinition * NewInstance](#) ([Model](#) *model, std::string name="")
- static std::vector< std::string > [split](#) (std::string word, char split_char)
Externalize function GetPluginInformation to be accessible through dynamic linked library.

Static Public Member Functions inherited from [ModelComponent](#)

- static [ModelComponent * LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields)
static public methods that must have implementations (Load and New just the same. GetInformation must provide specific infos for the new component)
- static bool [Check](#) ([ModelComponent](#) *component)
Validates a component instance.
- static void [CreateInternalData](#) ([ModelComponent](#) *component)
Creates internal data required by a component instance.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelComponent](#) *component)
Serializes a component instance into a persistence record.
- static void [DispatchEvent](#) ([Event](#) *event)
Dispatches a simulation event to the target component.

Static Public Member Functions inherited from [ModelDataDefinition](#)

- static [ModelDataDefinition](#) * [LoadInstance](#) ([Model](#) *model, [PersistenceRecord](#) *fields, bool insertIntoModel)
Loads a model data definition instance from serialized fields.
- static [ModelDataDefinition](#) * [NewInstance](#) ([Model](#) *model, std::string name="")
Creates a new instance of a data definition using the plugin registry.
- static void [SaveInstance](#) ([PersistenceRecord](#) *fields, [ModelDataDefinition](#) *modeldatum)
Serializes a data definition instance into a persistence record.
- static bool [Check](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Validates a data definition instance.
- static void [InitBetweenReplications](#) ([ModelDataDefinition](#) *modeldatum)
Resets simulation-time state between replications.
- static void [CreateInternalData](#) ([ModelDataDefinition](#) *modeldatum)
Creates internal and attached data required by this instance.
- static bool [CreateRelatedDataElements](#) ([ModelDataDefinition](#) *modeldatum, std::string &errorMessage)
Creates related data and validates the instance in a single call.

Protected Member Functions

- virtual bool [_loadInstance](#) ([PersistenceRecord](#) *fields) override
virtual protected method that must be overridden
- virtual void [_saveInstance](#) ([PersistenceRecord](#) *fields, bool saveDefaultValues) override
Saves object fields into a serialization structure.

Protected Member Functions inherited from [SPICECircuit](#)

- void **BuildCircuit** (std::string description, unsigned int id, std::vector< std::string > the_params={}, std::string the_model="", std::string the_model_file="")
virtual protected method that must be overridden
- void **UpdateConnections** ()
- virtual void [_onDispatchEvent](#) ([Entity](#) *entity, unsigned int inputPortNumber) override
This method is only for ModelComponents, not ModelDataElements.
- virtual void [_createNonEditableDataDefinitions](#) () override
virtual protected methods that could be overridden by derived classes, if needed

Protected Member Functions inherited from [ModelDataDefinition](#)

- bool [_getSaveDefaultsOption](#) ()
- virtual bool [_check](#) (std::string &errorMessage)
could be overridden by derived classes
- virtual [ParserChangesInformation](#) * [_getParserChangesInformation](#) ()
- virtual void [_initBetweenReplications](#) ()
- virtual void [_addSimulationControl](#) ([SimulationControl](#) *control)
- virtual void [_createInternalAndAttachedData](#) ()
- virtual void [_createInternalStatisticReporters](#) ()
- virtual void [_createAttachedAttributes](#) ()
- virtual void [_createEditableDataDefinitions](#) ()
- void [_templateCreateInternalStatisticReporters](#) ()
- void [_templateCreateAttachedAttributes](#) ()
- void [_templateCreateNonEditableDataDefinitions](#) ()

- void **_templateCreateEditableDataDefinitions** ()
- void **_internalStatisticReportersClear** ()
 - legacy helper methods used by the granular create* hooks*
- void **_internalDataClear** ()
- void **_internalDataInsert** (const std::string &key, [ModelDataDefinition](#) *child)
- void **_internalDataRemove** (const std::string &key)
- void **_attachedDataClear** ()
- void **_attachedDataInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_attachedDataRemove** (const std::string &key)
- void **_attachedAttributesInsert** (const std::vector< std::string > &neededNames)
- void **_statisticReporterInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_statisticReporterRemove** (const std::string &key)
- void **_statisticReportersClear** ()
- void **_mandatoryAttachedAttributeInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryAttachedAttributeRemove** (const std::string &key)
- void **_mandatoryAttachedAttributesClear** ()
- void **_mandatoryEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryEditableDataDefinitionsClear** ()
- void **_optionalEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalEditableDataDefinitionsClear** ()
- void **_mandatoryNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_mandatoryNonEditableDataDefinitionRemove** (const std::string &key)
- void **_mandatoryNonEditableDataDefinitionsClear** ()
- void **_optionalNonEditableDataDefinitionInsert** (const std::string &key, [ModelDataDefinition](#) *data)
- void **_optionalNonEditableDataDefinitionRemove** (const std::string &key)
- void **_optionalNonEditableDataDefinitionsClear** ()
- void **_checkCreateAttachedReferencedDataDefinition** (const std::string &expression)
 - method to be called to insert attached dataelements that are referenced by string expressions (detected by the parser), to avoid orphaned data definitions*
- void **trace** (const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - not just an easy access to trace manager, but wrappers to check if specific trace level applies*
- void **traceError** (const std::string &text, const std::exception &e)
- void **traceError** (const std::string &text, TraceManager::Level level=TraceManager::Level::L1_errorFatal)
- void **traceReport** (const std::string &text, TraceManager::Level level=TraceManager::Level::L2_results)
- void **traceSimulation** (void *thisobject, double time, [Entity](#) *entity, [ModelComponent](#) *component, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
- void [traceSimulation](#) (void *thisobject, const std::string &text, TraceManager::Level level=TraceManager::Level::L8_detailed)
 - Traces text to the simulation output.*
- void **traceSimulation** (void *thisobject, TraceManager::Level level, const std::string &text)

Additional Inherited Members

Protected Attributes inherited from [SPICECircuit](#)

- unsigned int **id**
- std::function< void()> **build**

Protected Attributes inherited from [ModelComponent](#)

- [ConnectionManager](#) * `_connections` = new [ConnectionManager](#)()
virtual protected methods that could be overridden by derived classes, if needed

Protected Attributes inherited from [ModelDataDefinition](#)

- Util::identification `_id`
- std::string `_typename`
- bool `_reportStatistics`
- bool `_hasChanged`
- unsigned int `_modelLevel` = 0
- [Model](#) * `_parentModel`
- [List](#)< [SimulationControl](#) * > * `_simulationControls` = new [List](#)<[SimulationControl](#)*>()
- [TraceManager](#)::Level `_traceLevelSpecific`
- bool `_traceLevelSpecificEnabled` = false

10.855.1 Member Function Documentation

10.855.1.1 `_loadInstance()`

```
bool XOR::_loadInstance (
    PersistenceRecord * fields ) [override], [protected], [virtual]
```

virtual protected method that must be overridden

Reimplemented from [SPICECircuit](#).

10.855.1.2 `_saveInstance()`

```
void XOR::_saveInstance (
    PersistenceRecord * fields,
    bool saveDefaultValues ) [override], [protected], [virtual]
```

Saves object fields into a serialization structure.

Parameters

<i>fields</i>	a table-like structure used for serialization.
<i>saveDefaultValues</i>	indicates whether or not to save fields which are set to their default value.

Reimplemented from [SPICECircuit](#).

10.855.1.3 `Build()`

```
void XOR::Build ( ) [override], [virtual]
```

Reimplemented from [SPICECircuit](#).

The documentation for this class was generated from the following files:

- `plugins/components/ElectronicsSimulation/SPICECircuit.h`
- `plugins/components/ElectronicsSimulation/SPICECircuit.cpp`

Chapter 11

File Documentation

11.1 BaseGenesysTerminalApplication.h

```
00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:      BaseGenesysTerminalApplication.h
00009  * Author:    rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 16:25
00012  */
00013
00014 #ifndef BaseGenesysTerminalApplication_H
00015 #define BaseGenesysTerminalApplication_H
00016
00017 #include "GenesysApplication_if.h"
00018 #include "kernel/simulator/TraceManager.h"
00019 #include "kernel/simulator/OnEventManager.h"
00020 #include "kernel/simulator/PluginManager.h"
00021 #include "../kernel/simulator/model/Model.h"
00022
00023 #include <thread>
00024
00025 class BaseGenesysTerminalApplication : public GenesysApplication_if {
00026 public:
00027     BaseGenesysTerminalApplication();
00028     virtual ~BaseGenesysTerminalApplication() = default;
00029 public:
00030     virtual int main(int argc, char** argv) override;
00031     virtual void Simulate(Simulator* genesys, Model* model, PluginManager *plugins);
00032 public:
00033     void setDefaultTraceHandlers(TraceManager* tm);
00034     void setDefaultEventHandlers(OnEventManager* oem);
00035 protected:
00036     // default Trace Handlers
00037     virtual void traceHandler(TraceEvent e);
00038     virtual void traceErrorHandler(TraceErrorEvent e);
00039     virtual void traceReportHandler(TraceEvent e);
00040     virtual void traceSimulationHandler(TraceSimulationEvent e);
00041     // default Event Handlers
00042     virtual void onAfterProcessEventHandler(SimulationEvent* re);
00043     virtual void onBreakpointHandler(SimulationEvent* re);
00044     virtual void onEntityCreateHandler(SimulationEvent* re);
00045     virtual void onEntityMoveHandler(SimulationEvent* re);
00046     virtual void onSimulationStartHandler(SimulationEvent* re);
00047     virtual void onReplicationStepHandler(SimulationEvent* re);
00048     virtual void onReplicationStartHandler(SimulationEvent* re);
00049     virtual void onProcessEventHandler(SimulationEvent* re);
00050     virtual void onReplicationEndHandler(SimulationEvent* re);
00051     virtual void onSimulationEndHandler(SimulationEvent* re);
00052     virtual void onSimulationPausedHandler(SimulationEvent* re);
00053     virtual void onSimulationResumeHandler(SimulationEvent* re);
00054     virtual void onEntityRemoveHandler(SimulationEvent* re);
00055 private:
00056
00057 };
00058
00059 #endif /* BaseGenesysTerminalApplication_H */
00060
```

11.2 GenesysApplication_if.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   GenesysApplication_if.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 2 de Outubro de 2018, 19:14
00012  */
00013
00014 #ifndef GENESYSAPPLICATION_IF_H
00015 #define GENESYSAPPLICATION_IF_H
00016
00017 class GenesysApplication_if {
00018 public:
00019     virtual ~GenesysApplication_if() = default;
00020     virtual int main(int argc, char** argv) = 0;
00021 };
00022
00023 #endif /* GENESYSAPPLICATION_IF_H */

```

11.3 AIAssistantWindow.h

```

00001 #ifndef AIASSISTANTWINDOW_H
00002 #define AIASSISTANTWINDOW_H
00003
00004 #include "../tools/AIAssistant/AIAssistantDefaultImpl.h"
00005 #include "../tools/AIAssistant/AISecretStore.h"
00006 #include "../kernel/simulator/SimulatorFacade.h"
00007
00008 #include <QMainWindow>
00009
00010 class QAction;
00011 class QCheckBox;
00012 class QComboBox;
00013 class QDoubleSpinBox;
00014 class QLabel;
00015 class QLineEdit;
00016 class QPlainTextEdit;
00017 class QPushButton;
00018 class QSpinBox;
00019 class QTabWidget;
00020 class QTableWidget;
00021 class QTextBrowser;
00022 class Simulator;
00023
00024 class AIAssistantWindow : public QMainWindow {
00025     Q_OBJECT
00026 public:
00027     explicit AIAssistantWindow(Simulator* simulator, QWidget* parent = nullptr);
00028     ~AIAssistantWindow() override;
00029
00030 private:
00031     void buildMenus();
00032     void buildWorkspace();
00033     void connectActions();
00034
00035     QWidget* buildConfigTab();
00036     QWidget* buildPromptTab();
00037     QWidget* buildExecutionTab();
00038     QWidget* buildAuditTab();
00039
00040     void applyConfigurationToBackend();
00041     void runAnalyzePrompt();
00042     void runCreateModelPlan();
00043     void runBuildModel();
00044     void runConfigureSimulation();
00045     void runSimulation();
00046     void runCollectResults();
00047     void runFullPipeline();
00048     void saveApiKeyToKeyring();
00049     void loadApiKeyFromKeyring();
00050     void refreshAuditTable();
00051     void exportAuditLog();
00052     void appendLog(const QString& text);
00053     void setStatus(const QString& text);
00054     void setRunning(bool running);
00055     AIAssistantRequest buildRequest() const;

```

```

00056     QString providerKeyringAccount() const;
00057
00058 private:
00059     Simulator*          _simulator = nullptr;
00060     SimulatorFacade     _facade;
00061     AIAssistantDefaultImpl _assistant;
00062
00063     // Configuration tab
00064     QComboBox*          _providerCombo      = nullptr;
00065     QLineEdit*          _modelNameEdit      = nullptr;
00066     QLineEdit*          _apiKeyEdit         = nullptr;
00067     QPushButton*        _apiKeyToggleBtn    = nullptr;
00068     QPushButton*        _saveKeyringBtn     = nullptr;
00069     QPushButton*        _loadKeyringBtn     = nullptr;
00070     QLineEdit*          _apiKeyEnvVarEdit   = nullptr;
00071     QLineEdit*          _baseUrlEdit        = nullptr;
00072     QSpinBox*           _timeoutSpin        = nullptr;
00073     QDoubleSpinBox*     _temperatureSpin    = nullptr;
00074     QCheckBox*          _allowNetworkChk     = nullptr;
00075     QCheckBox*          _allowMutationChk    = nullptr;
00076     QCheckBox*          _allowExecutionChk   = nullptr;
00077     QCheckBox*          _allowFilesystemChk  = nullptr;
00078     QCheckBox*          _sandboxChk         = nullptr;
00079     QCheckBox*          _dryRunChk          = nullptr;
00080
00081     // Prompt & Plan tab
00082     QPlainTextEdit*     _modelDescEdit      = nullptr;
00083     QPlainTextEdit*     _experimentDescEdit = nullptr;
00084     QPlainTextEdit*     _userPromptEdit     = nullptr;
00085     QPushButton*        _analyzeBtn         = nullptr;
00086     QPushButton*        _planBtn            = nullptr;
00087     QTextBrowser*       _planOutput         = nullptr;
00088
00089     // Execution tab
00090     QPushButton*        _buildModelBtn      = nullptr;
00091     QPushButton*        _configureSimBtn    = nullptr;
00092     QPushButton*        _runSimBtn          = nullptr;
00093     QPushButton*        _collectResultsBtn  = nullptr;
00094     QPushButton*        _fullPipelineBtn    = nullptr;
00095     QTextBrowser*       _executionLog       = nullptr;
00096     QLabel*             _statusLabel        = nullptr;
00097
00098     // Audit log tab
00099     QTableWidget*       _auditTable         = nullptr;
00100     QPushButton*        _refreshAuditBtn    = nullptr;
00101     QPushButton*        _exportAuditBtn     = nullptr;
00102     QLabel*             _auditLogPathLabel  = nullptr;
00103
00104     QTabWidget*         _tabs               = nullptr;
00105
00106     // Actions
00107     QAction*            _applyConfigAction  = nullptr;
00108     QAction*            _clearLogAction     = nullptr;
00109     QAction*            _refreshAuditAction = nullptr;
00110 };
00111
00112 #endif // AIASSISTANTWINDOW_H

```

11.4 AddUndoCommand.h

```

00001 #ifndef ADDUNDOCOMMAND_H
00002 #define ADDUNDOCOMMAND_H
00003
00004 #include <QUndoCommand>
00005 #include "graphicals/ModelGraphicsScene.h"
00006 #include "graphicals/GraphicalConnection.h"
00007 #include "graphicals/GraphicalModelComponent.h"
00008
00009 struct ComponentItem {
00010     GraphicalModelComponent *graphicalComponent;
00011     QPointF initialPosition;
00012     QList<GraphicalConnection *> inputConnections;
00013     QList<GraphicalConnection *> outputConnections;
00014 };
00015
00016 struct GroupItem {
00017     QGraphicsItemGroup *group;
00018     QList<ComponentItem> myComponentItems;
00019     QPointF initialPosition;
00020 };
00021
00022 struct DrawingItem {
00023     QGraphicsItem *myDrawingItem;

```

```

00024     QPointF initialPosition;
00025 };
00026
00027
00028 class AddUndoCommand: public QUndoCommand {
00029 public:
00030     AddUndoCommand(QGraphicsItem *item, ModelGraphicsScene *scene, QUndoCommand *parent = nullptr);
00031     ~AddUndoCommand();
00032
00033     void undo() override;
00034     void redo() override;
00035
00036 private:
00037     ComponentItem _myComponentItem;
00038     GraphicalConnection *_myConnectionItem;
00039     QGraphicsItem *_myDrawingItem;
00040     ModelGraphicsScene *_myGraphicsScene;
00041     bool _firstExecution;
00042 };
00043
00044 #endif // ADDUNDOCOMMAND_H

```

11.5 DeleteUndoCommand.h

```

00001 #ifndef DELETEUNDOCOMMAND_H
00002 #define DELETEUNDOCOMMAND_H
00003
00004 #include <QUndoCommand>
00005 #include <string>
00006 #include "AddUndoCommand.h"
00007 #include "graphicals/ModelGraphicsScene.h"
00008 #include "graphicals/GraphicalConnection.h"
00009 #include "graphicals/GraphicalModelDataDefinition.h"
00010
00011 struct DataDefinitionItem {
00012     GraphicalModelDataDefinition *graphicalDataDefinition;
00013     QPointF initialPosition;
00014     bool initiallySelected;
00015 };
00016
00017 struct DataDefinitionRelationItem {
00018     ModelDataDefinition *owner;
00019     ModelDataDefinition *dataDefinition;
00020     std::string key;
00021     bool internal;
00022 };
00023
00024 class DeleteUndoCommand : public QUndoCommand
00025 {
00026 public:
00027     explicit DeleteUndoCommand(QList<QGraphicsItem *> items, ModelGraphicsScene *scene, QUndoCommand
00028 *parent = nullptr);
00029     ~DeleteUndoCommand();
00030
00031     void undo() override;
00032     void redo() override;
00033
00034 private:
00035     void captureDataDefinitionsRemovedWithSelectedItems();
00036     void captureDataDefinitionRelations();
00037     void removeDataDefinitionRelations();
00038     void restoreDataDefinitionRelations();
00039     void removeDataDefinitionsFromModel();
00040     void restoreDataDefinitionsToModel();
00041     void detachDataDefinitionItems();
00042     void restoreDataDefinitionItems();
00043
00044     QList<ComponentItem> *_myComponentItems;
00045     QList<GraphicalConnection *> *_myConnectionItems;
00046     QList<DrawingItem> *_myDrawingItems;
00047     QList<GroupItem> *_myGroupItems;
00048     QList<DataDefinitionItem> *_myDataDefinitionItems;
00049     QList<DataDefinitionRelationItem> *_myDataDefinitionRelationItems;
00050     QList<ModelDataDefinition *> *_myDataDefinitions;
00051     ModelGraphicsScene *_myGraphicsScene;
00052 };
00053 #endif // DELETEUNDOCOMMAND_H

```

11.6 GroupUndoCommand.h

```

00001 #ifndef GROUPUNDOCOMMAND_H
00002 #define GROUPUNDOCOMMAND_H
00003
00004 #include <QUndoCommand>
00005 #include "graphicals/ModelGraphicsScene.h"
00006 #include "graphicals/GraphicalModelComponent.h"
00007
00008 class GroupUndoCommand: public QUndoCommand {
00009 public:
00010     GroupUndoCommand(QList<GraphicalModelComponent *> graphicalComponents, ModelGraphicsScene *scene,
00011                     QUndoCommand *parent = nullptr);
00012     ~GroupUndoCommand();
00013
00014     void undo() override;
00015     void redo() override;
00016 private:
00017     QList<GraphicalModelComponent *> _myGraphicalComponents;
00018     QGraphicsItemGroup *_group;
00019     ModelGraphicsScene *_myGraphicsScene;
00020     QPointF _initialPosition;
00021 };
00022
00023 #endif // GROUPUNDOCOMMAND_H

```

11.7 MoveUndoCommand.h

```

00001 #ifndef MOVEUNDOCOMMAND_H
00002 #define MOVEUNDOCOMMAND_H
00003
00004 #include <QUndoCommand>
00005 #include "graphicals/ModelGraphicsScene.h"
00006
00007 class MoveUndoCommand : public QUndoCommand
00008 {
00009 public:
00010     MoveUndoCommand(QList<QGraphicsItem *> item, ModelGraphicsScene *scene, QList<QPointF> &oldPos,
00011                    QList<QPointF> &newPos, QUndoCommand *parent = nullptr);
00012     ~MoveUndoCommand();
00013
00014     void undo() override;
00015     void redo() override;
00016
00017     //bool mergeWith(const QUndoCommand *command) override;
00018 private:
00019     QList<QGraphicsItem*> _myGraphicalItem;
00020     ModelGraphicsScene *_myGraphicsScene;
00021     QList<QPointF> _myOldPos;
00022     QList<QPointF> _myNewPos;
00023 };
00024
00025 #endif // MOVEUNDOCOMMAND_H

```

11.8 PasteUndoCommand.h

```

00001 #ifndef PASTEUNDOCOMMAND_H
00002 #define PASTEUNDOCOMMAND_H
00003
00004 #include <QUndoCommand>
00005 #include "AddUndoCommand.h"
00006 #include "graphicals/ModelGraphicsScene.h"
00007 #include "graphicals/GraphicalConnection.h"
00008
00009
00010 class PasteUndoCommand : public QUndoCommand
00011 {
00012 public:
00013     explicit PasteUndoCommand(QList<GraphicalModelComponent *> *graphicalComponents,
00014                               QList<GraphicalConnection *> *connections, QList<QGraphicsItemGroup *> *groups, QList<QGraphicsItem *>
00015                               *drawing, ModelGraphicsScene *scene, QUndoCommand *parent = nullptr);
00016     ~PasteUndoCommand();
00017
00018     void undo() override;
00019     void redo() override;
00020 private:

```

```

00020     QList<ComponentItem> *_myComponentItems;
00021     QList<GraphicalConnection *> *_myConnectionItems;
00022     QList<DrawingItem> *_myDrawingItems;
00023     QList<GroupItem> *_myGroupItems;
00024     ModelGraphicsScene *_myGraphicsScene;
00025 };
00026
00027 #endif // PASTEUNDOCOMMAND_H

```

11.9 UngroupUndoCommand.h

```

00001 #ifndef UNGROUPUNDOCOMMAND_H
00002 #define UNGROUPUNDOCOMMAND_H
00003
00004
00005 #include <QUndoCommand>
00006 #include "graphicals/ModelGraphicsScene.h"
00007
00008 class UngroupUndoCommand: public QUndoCommand {
00009 public:
00010     UngroupUndoCommand(QGraphicsItemGroup* group, ModelGraphicsScene *scene, QUndoCommand *parent =
00011         nullptr);
00012     ~UngroupUndoCommand();
00013     void undo() override;
00014     void redo() override;
00015 private:
00016     QGraphicsItemGroup *_group;
00017     QList<GraphicalModelComponent *> *_myComponentsGroup;
00018     ModelGraphicsScene *_myGraphicsScene;
00019     QPointF _initialPosition;
00020 };
00021
00022
00023 #endif // UNGROUPUNDOCOMMAND_H

```

11.10 AnimationCounter.h

```

00001 #ifndef ANIMATIONCOUNTER_H
00002 #define ANIMATIONCOUNTER_H
00003
00004 #include <QGraphicsRectItem>
00005 #include <QPainter>
00006 #include <QGraphicsSceneMouseEvent>
00007
00008 #include "../kernel/simulator/essentialPlugins/Counter.h"
00009 class AnimationCounter : public QGraphicsRectItem {
00010 public:
00011     // Construtor
00012     AnimationCounter();
00013
00014     // Sobrecarga do método paint
00015     void paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget = nullptr)
00016         override;
00017
00018     // Getters
00019     double getValue();
00020     Counter *getCounter();
00021
00022     // Setters
00023     void setValue(double value);
00024     void setCounter(Counter *myCounter);
00025     void setIdCounter(int id);
00026     void setNameCounter(std::string name);
00027
00028     // Outros
00029     void startDrawing(QGraphicsSceneMouseEvent *event); // Inicia o desenho da tela
00030     void continueDrawing(QGraphicsSceneMouseEvent *event); // Continua o desenho na tela
00031     void stopDrawing(QGraphicsSceneMouseEvent *event); // Finaliza o desenho na tela
00032     void adjustSizeAndPosition(QGraphicsSceneMouseEvent *event); // Ajusta posição e dimensão do
00033         retângulo no final
00034     bool isDrawingInialized(); // Diz se o contador começou a ser desenhada
00035     bool isDrawingFinalized(); // Diz se o contador terminou de ser desenhada
00036     void setWhenLoaded(QList<Counter *> *counters); // seta o contador quando o modelo é carregado
00037 private:
00038     double _value = 0.0;
00039     QPointF _startPoint = QPointF(0, 0);
00040     bool _isResizing = false;

```



```

00040     bool _isDrawingInicialized = false;
00041     bool _isDrawingFinalized = false;
00042     Counter *_counter = nullptr;
00043     int _idCounter = -1;
00044     std::string _counterName = "None";
00045 };
00046
00047 #endif // ANIMATIONCOUNTER_H

```

11.11 AnimationPlaceholder.h

```

00001 #ifndef ANIMATIONPLACEHOLDER_H
00002 #define ANIMATIONPLACEHOLDER_H
00003
00004 #include <QGraphicsRectItem>
00005 #include <QGraphicsSceneMouseEvent>
00006 #include <QPainter>
00007 #include <QVector>
00008 #include <QString>
00009 #include <limits>
00010
00011 class Model;
00012
00013 class AnimationPlaceholder : public QGraphicsRectItem {
00014 public:
00015     explicit AnimationPlaceholder(const QString& animationType);
00016
00017     void paint(QPainter* painter, const QStyleOptionGraphicsItem* option, QWidget* widget = nullptr)
00018         override;
00019
00019     QString getAnimationType() const;
00020     QString getTargetName() const;
00021     void setTargetName(const QString& targetName);
00022
00023     void startDrawing(QGraphicsSceneMouseEvent* event);
00024     void continueDrawing(QGraphicsSceneMouseEvent* event);
00025     void stopDrawing(QGraphicsSceneMouseEvent* event);
00026     void adjustSizeAndPosition(QGraphicsSceneMouseEvent* event);
00027     bool isDrawingInicialized() const;
00028     bool isDrawingFinalized() const;
00029
00030 private:
00031     QString _animationType;
00032     QString _targetName;
00033     QPointF _startPoint = QPointF(0, 0);
00034     bool _isResizing = false;
00035     bool _isDrawingInicialized = false;
00036     bool _isDrawingFinalized = false;
00037 };
00038
00039 class AnimationAttribute : public AnimationPlaceholder {
00040 public:
00041     AnimationAttribute();
00042 };
00043
00044 class AnimationEntity : public AnimationPlaceholder {
00045 public:
00046     AnimationEntity();
00047 };
00048
00049 class AnimationEvent : public AnimationPlaceholder {
00050 public:
00051     AnimationEvent();
00052 };
00053
00054 class AnimationExpression : public AnimationPlaceholder {
00055 public:
00056     AnimationExpression();
00057 };
00058
00059 class AnimationPlot : public AnimationPlaceholder {
00060 public:
00061     enum class PlotType {
00062         Line = 0,
00063         Bar = 1
00064     };
00065
00066     struct DataSeries {
00067         QString label;
00068         QString expression;
00069         QColor color = QColor(31, 119, 180);
00070         qreal lineWidth = 2.0;
00071         QVector<double> values;

```

```

00072     };
00073
00074     AnimationPlot();
00075
00076     QString getTitle() const;
00077     void setTitle(const QString& title);
00078     QString getYAxisTitle() const;
00079     void setYAxisTitle(const QString& title);
00080     QString getXAxisTitle() const;
00081     void setXAxisTitle(const QString& title);
00082     PlotType getPlotType() const;
00083     void setPlotType(PlotType type);
00084     QString getDatasetsText() const;
00085     void setDatasetsText(const QString& datasetsText);
00086     bool getShowGridLines() const;
00087     void setShowGridLines(bool show);
00088     bool getShowTicks() const;
00089     void setShowTicks(bool show);
00090     double getXAxisMin() const;
00091     void setXAxisMin(double value);
00092     double getXAxisMax() const;
00093     void setXAxisMax(double value);
00094     double getYAxisMin() const;
00095     void setYAxisMin(double value);
00096     double getYAxisMax() const;
00097     void setYAxisMax(double value);
00098     void clearValues();
00099     void appendSample(Model* model);
00100
00101     void paint(QPainter* painter, const QStyleOptionGraphicsItem* option, QWidget* widget = nullptr)
00102     override;
00103 private:
00104     static QVector<DataSeries> parseDatasetsText(const QString& datasetsText);
00105     static QString datasetsTextFromSeries(const QVector<DataSeries>& series);
00106
00107     QString _title = "Plot";
00108     QString _xAxisTitle = "Time";
00109     QString _yAxisTitle = "Value";
00110     PlotType _plotType = PlotType::Line;
00111     bool _showGridLines = true;
00112     bool _showTicks = true;
00113     double _xAxisMin = std::numeric_limits<double>::quiet_NaN();
00114     double _xAxisMax = std::numeric_limits<double>::quiet_NaN();
00115     double _yAxisMin = std::numeric_limits<double>::quiet_NaN();
00116     double _yAxisMax = std::numeric_limits<double>::quiet_NaN();
00117     QVector<DataSeries> _series;
00118 };
00119
00120 class AnimationQueueDisplay : public AnimationPlaceholder {
00121 public:
00122     AnimationQueueDisplay();
00123 };
00124
00125 class AnimationResource : public AnimationPlaceholder {
00126 public:
00127     AnimationResource();
00128 };
00129
00130 class AnimationStation : public AnimationPlaceholder {
00131 public:
00132     AnimationStation();
00133 };
00134
00135 class AnimationStatistics : public AnimationPlaceholder {
00136 public:
00137     AnimationStatistics();
00138 };
00139
00140 #endif // ANIMATIONPLACEHOLDER_H

```

11.12 AnimationQueue.h

```

00001 #ifndef ANIMATIONQUEUE_H
00002 #define ANIMATIONQUEUE_H
00003
00004 #include "graphicals/GraphicalModelComponent.h"
00005
00006 class ModelGraphicsScene;
00007
00008 class AnimationQueue
00009 {
00010 public:

```

```

00011 // Construtor
00012 AnimationQueue(ModelGraphicsScene* myScene, ModelComponent* component);
00013 // Destrutor
00014 ~AnimationQueue();
00015
00016 // Getters
00017 GraphicalModelComponent* getGraphicalComponent() const;
00018
00019 // quando uma entidade esta movendo em relacao a um componente que tem fila, adiciona a animação
00020 void addAnimationQueue(bool visivable);
00021 // quando uma entidade esta movendo de um componente que tem fila, remove a animação
00022 void removeAnimationQueue();
00023
00024 // faz a animação de acordo com o tamanho da fila, mas não ficou muito condizente com a
visualização da animação
00025 // verifica se precisa adicionar animação e adiciona se for o caso (observando valores da fila)
00026 void verifyAddAnimationQueue(bool visivable);
00027 // verifica se precisa remover animação e remove se for o caso (observando valores da fila)
00028 void verifyRemoveAnimationQueue();
00029
00030 // retorna a posição em que a animação da fila será desenhada
00031 QPointF calculatePositionImageQueue(unsigned int indexQueue, unsigned int sizeQueue, unsigned int
width, unsigned int height);
00032
00033 private:
00034 ModelGraphicsScene* _myScene;
00035 GraphicalModelComponent* _graphicalComponent;
00036 };
00037
00038 #endif // ANIMATIONQUEUE_H

```

11.13 AnimationTimer.h

```

00001 #ifndef ANIMATIONTIMER_H
00002 #define ANIMATIONTIMER_H
00003
00004 #include <QGraphicsRectItem>
00005 #include <QPainter>
00006 #include <QGraphicsSceneMouseEvent>
00007 #include "kernel/util/Util.h"
00008
00009 class ModelGraphicsScene;
00010
00011 class AnimationTimer : public QGraphicsRectItem {
00012 public:
00013 // Construtor
00014 AnimationTimer(ModelGraphicsScene* myScene);
00015
00016 // Sobrecarga do método paint
00017 void paint(QPainter* painter, const QStyleOptionGraphicsItem* option, QWidget* widget = nullptr)
override;
00018
00019 // Getters
00020 double getTime();
00021 unsigned int getInitialHours();
00022 unsigned int getInitialMinutes();
00023 unsigned int getInitialSeconds();
00024 unsigned int getHours();
00025 unsigned int getMinutes();
00026 unsigned int getSeconds();
00027 Util::TimeFormat getTimeFormat();
00028
00029 // Setters
00030 void setTime(double value);
00031 static void setConversionFactorToSeconds(double factor);
00032 void setInitialHours(unsigned int initialHours);
00033 void setInitialMinutes(unsigned int initialMinutes);
00034 void setInitialSeconds(unsigned int initialSeconds);
00035 void setHours(unsigned int hours);
00036 void setMinutes(unsigned int minutes);
00037 void setSeconds(unsigned int seconds);
00038 void setTimeFormat(Util::TimeFormat timeFormat);
00039
00040 // Outros
00041 void startDrawing(QGraphicsSceneMouseEvent* event); // Inicia o desenho da tela
00042 void continueDrawing(QGraphicsSceneMouseEvent* event); // Continua o desenho na tela
00043 void stopDrawing(QGraphicsSceneMouseEvent* event); // Finaliza o desenho na tela
00044 void adjustSizeAndPosition(QGraphicsSceneMouseEvent* event); // Ajusta posição e dimensão do
retângulo no final
00045 void convertSeconds(); // converte o tempo recebido em segundos para horas, dias e segundos
00046 bool isDrawingInialized(); // Diz se o relógio começou a ser desenhado
00047 bool isDrawingFinalized(); // Diz se o relógio terminou de ser desenhado
00048

```

```

00049 private:
00050     ModelGraphicsScene* _myScene;
00051     double _time = 0.0;
00052     static double _conversionFactorToSeconds;
00053     QPointF _startPoint = QPointF(0, 0);
00054     unsigned int _initialHours = 0;
00055     unsigned int _initialMinutes = 0;
00056     unsigned int _initialSeconds = 0;
00057     unsigned int _hours = 0;
00058     unsigned int _minutes = 0;
00059     unsigned int _seconds = 0;
00060     unsigned int _microseconds = 0;
00061     Util::TimeFormat _timeFormat = Util::TimeFormat(24);
00062     bool _showMicroseconds = true;
00063     bool _isResizing = false;
00064     bool _isDrawingInicialized = false;
00065     bool _isDrawingFinalized = false;
00066 };
00067
00068 #endif // ANIMATIONTIMER_H

```

11.14 AnimationTransition.h

```

00001 #ifndef ANIMATIONTRANSITION_H
00002 #define ANIMATIONTRANSITION_H
00003
00004 #include <QList>
00005 #include <QPainterPath>
00006 #include <QPointF>
00007 #include <QTimer>
00008 #include <QVariantAnimation>
00009 #include <QElapsedTimer>
00010
00011 #include "graphicals/GraphicalImageAnimation.h"
00012 #include "graphicals/GraphicalModelComponent.h"
00013 #include "graphicals/GraphicalConnection.h"
00014
00015 // Em segundos
00016 #define TEMPO_EXECUCAO_ANIMACAO 2.5
00017
00018 class ModelGraphicsScene;
00019
00020 class AnimationTransition : public QVariantAnimation
00021 {
00022 public:
00023     // Construtor
00024     AnimationTransition(ModelGraphicsScene* myScene, ModelComponent* graphicalStartComponent,
00025         ModelComponent* graphicalEndComponent, bool viewSimulation);
00026     // Destrutor
00027     ~AnimationTransition();
00028
00029     // Getters
00030     GraphicalModelComponent* getGraphicalStartComponent() const;
00031     GraphicalModelComponent* getGraphicalEndComponent() const;
00032     GraphicalConnection* getGraphicalConnection() const;
00033     static double getTimeExecution();
00034     QList<QPointF> getPointsForAnimation() const;
00035     GraphicalImageAnimation* getImageAnimation() const;
00036     unsigned int getPortNumber() const;
00037     // Expose whether transition has all runtime prerequisites to safely animate.
00038     bool isReadyToRun() const;
00039
00040     // Setters
00041     void setImageAnimation(GraphicalImageAnimation* imageAnimation);
00042     static void setTimeExecution(double timeExecution);
00043     static void setPause(bool pause);
00044     static void setRunning(bool running);
00045
00046     // Inicia a animação
00047     void startAnimation();
00048     // Finaliza a animação
00049     void stopAnimation();
00050     // Reinicia a animação
00051     void restartAnimation();
00052     // Configuração informações para a animação
00053     void configureAnimation();
00054     // Atualiza a duração da animação se ela for alterada em tempo de execução
00055     void updateDurationIfNeeded();
00056     // Conecta o sinal valueChanged da animação ao slot onAnimationValueChanged
00057     void connectValueChangedSignal();
00058     // Conecta o sinal finished da animação ao slot onAnimationFinished
00059     void connectFinishedSignal();

```

```

00060 public slots:
00061     // Slot para ser conectado ao sinal valueChanged
00062     void onAnimationValueChanged(const QVariant& value);
00063     // Slot que será conectado a um sinal
00064     void onAnimationFinished();
00065
00066 private:
00067     ModelGraphicsScene* _myScene;
00068     GraphicalModelComponent* _graphicalStartComponent;
00069     GraphicalModelComponent* _graphicalEndComponent;
00070     GraphicalConnection* _graphicalConnection;
00071     static double _timeExecution;
00072     static double _oldTimeExecution;
00073     static bool _pause;
00074     static bool _running;
00075     QList<QPointF> _pointsForAnimation;
00076     QPainterPath _pathForAnimation;
00077     GraphicalImageAnimation* _imageAnimation;
00078     unsigned int _portNumber;
00079     qreal _currentProgress;
00080     bool _viewSimulation;
00081     // Track terminal lifecycle paths to keep cleanup idempotent.
00082     bool _isStopping;
00083     // Track whether finished callback was already handled.
00084     bool _isFinishedHandled;
00085     bool _usesPathAnimation;
00086 };
00087
00088 #endif // ANIMATIONTRANSITION_H

```

11.15 AnimationVariable.h

```

00001 #ifndef ANIMATIONVARIABLE_H
00002 #define ANIMATIONVARIABLE_H
00003
00004 #include <QString>
00005 #include <QList>
00006 #include <QVector>
00007 #include <QGraphicsRectItem>
00008 #include <QGraphicsSceneMouseEvent>
00009 #include <QPainter>
00010 #include <string>
00011
00012 #include "../kernel/simulator/essentialPlugins/Attribute.h"
00013 #include "../kernel/simulator/essentialPlugins/Entity.h"
00014 #include "../kernel/simulator/model/Model.h"
00015 #include "../kernel/simulator/model/ModelDataDefinition.h"
00016 #include "../kernel/simulator/model/ModelDataManager.h"
00017 #include "../plugins/data/Logic/Variable.h"
00018
00019 class AnimationVariable : public QGraphicsRectItem {
00020 public:
00021     AnimationVariable();
00022
00023     void paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget = nullptr)
00024         override;
00025
00026     double getValue();
00027     ModelDataDefinition *getDataDefinition() const;
00028     Variable *getVariable();
00029     Attribute *getAttribute();
00030     QString getTargetName() const;
00031     int getTargetId() const;
00032     unsigned int getDimensionCount() const;
00033     unsigned int getDimensionSize(unsigned int dimensionIndex) const;
00034     unsigned int getSliceIndex(unsigned int dimensionIndex) const;
00035
00036     void setValue(double value);
00037     void setDataDefinition(ModelDataDefinition *definition);
00038     void setVariable(Variable *variable);
00039     void setAttribute(Attribute *attribute);
00040     void setModel(Model *model);
00041     void setTargetName(std::string name);
00042     void setIdVariable(int id);
00043     void setSliceIndex(unsigned int dimensionIndex, unsigned int value);
00044     void setWhenLoaded(QList<ModelDataDefinition*> *definitions);
00045     void refreshValue(Entity *entity = nullptr);
00046
00047     void startDrawing(QGraphicsSceneMouseEvent *event);
00048     void continueDrawing(QGraphicsSceneMouseEvent *event);
00049     void stopDrawing(QGraphicsSceneMouseEvent *event);
00050     void adjustSizeAndPosition(QGraphicsSceneMouseEvent *event);
00051     bool isDrawingInialized();

```

```

00051     bool isDrawingFinalized();
00052
00053 private:
00054     double _value = 0.0;
00055     Model *_model = nullptr;
00056     ModelDataDefinition *_dataDefinition = nullptr;
00057     QString _targetName = "None";
00058     int _targetId = -1;
00059     QVector<unsigned int> _sliceIndexes;
00060     QVector<double> _cachedValues;
00061     QVector<unsigned int> _cachedShape;
00062     QString _cachedTypeName;
00063     QString _cachedTitle;
00064     QPointF _startPoint = QPointF(0, 0);
00065     bool _isResizing = false;
00066     bool _isDrawingInicialized = false;
00067     bool _isDrawingFinalized = false;
00068 };
00069
00070 #endif // ANIMATIONVARIABLE_H

```

11.16 BaseMVController.h

```

00001 #pragma once
00002
00003 #include <QMainWindow>
00004 #include <QObject>
00005 #include <QWidget>
00006 #include "kernel/simulator/Simulator.h"
00007 #include "ui_mainwindow.h"
00008
00016 class BaseMVController
00017 {
00018 public:
00019     BaseMVController();
00020 protected: // View (UI) and Model (Simulator) main elements to join to the Controller
00022     Ui::MainWindow *ui;
00024     Simulator* simulator;
00025 };

```

11.17 CellularAutomataDemoBuilder.h

```

00001 #pragma once
00002
00003 /*
00004
00005 • Para que a GUI use um novo autômato celular com nova regra local,
00006 você mexe em 3 camadas:
00007
00008 1) Implementar a regra no plugin
00009
00010 - Crie a classe da regra em source/plugins/components/ModalModel/
00011 CellularAutomata/.
00012 - Normalmente isso significa algo como:
00013     - LocalRule_MyRule.h
00014     - opcionalmente LocalRule_MyRule.cpp
00015 - A regra precisa herdar de LocalRule e sobrescrever
00016 applyRule(Cell* cell).
00017 - Se a regra precisar de vizinhança específica, ela vai ler
00018 cell->getNeighbors().
00019
00020 2) Registrar essa regra no builder da GUI
00021
00022 - O ponto de montagem do demo fica em:
00023     - source/applications/gui/genesys/cellular_automata/
00024 CellularAutomataDemoBuilder.h
00025     - source/applications/gui/genesys/cellular_automata/
00026 CellularAutomataDemoBuilder.cpp
00027 - É aqui que a GUI escolhe qual regra concreta instanciar.
00028 - Você precisa:
00029     - incluir o header da nova regra;
00030     - adicionar um valor novo no enum CellularAutomataRulePreset;
00031     - estender cellularAutomataRulePresetText(...);
00032     - atualizar makeLocalRule(...) para retornar sua classe.
00033
00034 3) Expor a escolha na janela
00035
00036 - A UI de seleção fica em:
00037     - source/applications/gui/genesys/cellular_automata/

```

```

00038     CellularAutomataViewerWindow.cpp
00039     - source/applications/gui/genesys/cellular_automata/
00040     CellularAutomataViewerWindow.h
00041     - Você precisa adicionar o novo preset ao combo de regra em
00042     _syncControlsFromController().
00043     - Se quiser salvar/restaurar essa escolha no JSON, ajuste também:
00044     - source/applications/gui/genesys/cellular_automata/
00045     CellularAutomataViewerController.cpp
00046
00047     Se a nova regra exigir outro tipo de vizinhança ou estado
00048
00049     - O ajuste continua na mesma área do builder:
00050     - CellularAutomataDemoBuilder.h/.cpp
00051     - Para vizinhança, hoje a GUI já suporta:
00052     - Moore
00053     - VonNeumann
00054     - Para tipo de estado, hoje ela expõe:
00055     - Binary
00056     - Enumerated
00057     - Numeric
00058     - Se sua regra precisar de um estado diferente, você precisa
00059     ampliar o preset de estado e o mapeamento de cores/menu.
00060
00061     Fluxo mínimo para fazer funcionar
00062
00063     1. Criar LocalRule_MyRule.
00064     2. Incluir essa classe em CellularAutomataDemoBuilder.h.
00065     3. Adicionar o preset no enum CellularAutomataRulePreset.
00066     4. Implementar a criação da regra em makeLocalRule(...).
00067     5. Adicionar o item no combo da janela em
00068     CellularAutomataViewerWindow.cpp.
00069     6. Atualizar serialização em CellularAutomataViewerController.cpp
00070     se quiser persistência.
00071
00072     Arquivos que você provavelmente vai mexer
00073
00074     - source/plugins/components/ModalModel/CellularAutomata/
00075     LocalRule_MyRule.h
00076     - source/plugins/components/ModalModel/CellularAutomata/
00077     LocalRule_MyRule.cpp se necessário
00078     - source/applications/gui/genesys/cellular_automata/
00079     CellularAutomataDemoBuilder.h
00080     - source/applications/gui/genesys/cellular_automata/
00081     CellularAutomataDemoBuilder.cpp
00082     - source/applications/gui/genesys/cellular_automata/
00083     CellularAutomataViewerWindow.cpp
00084     - source/applications/gui/genesys/cellular_automata/
00085     CellularAutomataViewerController.cpp
00086
00087     */
00088
00089
00090
00091
00092     #include <memory>
00093     #include <vector>
00094
00095     #include <QSize>
00096
00097     #include "plugins/components/ModalModel/CellularAutomata/Boundary_Closed.h"
00098     #include "plugins/components/ModalModel/CellularAutomata/Boundary_Fixed.h"
00099     #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00100     #include "plugins/components/ModalModel/CellularAutomata/CellularAutomata_Classic.h"
00101     #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00102     #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00103     #include "plugins/components/ModalModel/CellularAutomata/LocalRule_GameOfLife.h"
00104     #include "plugins/components/ModalModel/CellularAutomata/LocalRule_FlorestalFire.h"
00105     #include "plugins/components/ModalModel/CellularAutomata/LocalRule_Growty.h"
00106     #include "plugins/components/ModalModel/CellularAutomata/LocalRule_HppLatticeGas.h"
00107     #include "plugins/components/ModalModel/CellularAutomata/Neighborhood_Moore.h"
00108     #include "plugins/components/ModalModel/CellularAutomata/Neighborhood_VonNeumann.h"
00109     #include "plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h"
00110
00111     enum class CellularAutomataRulePreset {
00112         GameOfLife = 0,
00113         Growty = 1,
00114         Identity = 2,
00115         ForestFire = 3,
00116         HppLatticeGas = 4,
00117         SandRock = 5
00118     };
00119
00120     enum class CellularAutomataNeighborhoodPreset {
00121         Moore = 0,
00122         VonNeumann = 1
00123     };
00124

```

```

00125 enum class CellularAutomataBoundaryPreset {
00126     Closed = 0,
00127     Fixed = 1
00128 };
00129
00130 enum class CellularAutomataStatePreset {
00131     Binary = 0,
00132     Enumerated = 1,
00133     Numeric = 2,
00134     HppLatticeGas = 3
00135 };
00136
00137 struct CellularAutomataDemoSettings {
00138     QSize latticeSize = QSize{100, 100};
00139     CellularAutomataRulePreset rulePreset = CellularAutomataRulePreset::SandRock;
00140     CellularAutomataNeighborhoodPreset neighborhoodPreset = CellularAutomataNeighborhoodPreset::Moore;
00141     CellularAutomataBoundaryPreset boundaryPreset = CellularAutomataBoundaryPreset::Fixed;
00142     CellularAutomataStatePreset statePreset = CellularAutomataStatePreset::Enumerated;
00143 };
00144
00145 QString cellularAutomataRulePresetText(CellularAutomataRulePreset preset);
00146 QString cellularAutomataNeighborhoodPresetText(CellularAutomataNeighborhoodPreset preset);
00147 QString cellularAutomataBoundaryPresetText(CellularAutomataBoundaryPreset preset);
00148 QString cellularAutomataStatePresetText(CellularAutomataStatePreset preset);
00149
00150 class LocalRule_CopyCurrentState final : public LocalRule {
00151 public:
00152     explicit LocalRule_CopyCurrentState(CellularAutomataBase* parentCellularAutomata)
00153         : LocalRule(parentCellularAutomata) {
00154     }
00155
00156     void applyRule(Cell* cell) override {
00157         if (cell != nullptr) {
00158             cell->setNextState(cell->getCurrentState());
00159         }
00160     }
00161 };
00162
00163 struct CellularAutomataDemoModel {
00164     explicit CellularAutomataDemoModel(CellularAutomataDemoSettings settings = {});
00165
00166     CellularAutomataDemoSettings settings;
00167     std::vector<State> ownedStates;
00168     std::vector<State*> statePointers;
00169     std::unique_ptr<CellularAutomata_Classic> automaton;
00170     std::unique_ptr<Lattice> lattice;
00171     std::unique_ptr<StateSet> stateSet;
00172     std::unique_ptr<BoundaryCondition> boundary;
00173     std::unique_ptr<Neighborhood> neighborhood;
00174     std::unique_ptr<LocalRule> localRule;
00175     std::unique_ptr<LocalRule_CopyCurrentState> identityLocalRule;
00176
00177     int columns() const;
00178     int rows() const;
00179     bool isValidPosition(int x, int y) const;
00180     Cell* cellAt(int x, int y) const;
00181     long stateAt(int x, int y) const;
00182     QString stateTextAt(int x, int y) const;
00183     bool setStateAt(int x, int y, long value);
00184     void fill(long value);
00185     void seedDefaultPattern();
00186     std::vector<long> availablePaintStates() const;
00187 };
00188
00189 class CellularAutomataDemoBuilder {
00190 public:
00191     static std::unique_ptr<CellularAutomataDemoModel> buildDemo(CellularAutomataDemoSettings settings
00192 = {});
00193 };

```

11.18 CellularAutomataLatticeWidget.h

```

00001 #pragma once
00002
00003 #include <QPoint>
00004 #include <QRectF>
00005 #include <QWidget>
00006
00007 class CellularAutomataViewerController;
00008
00009 class CellularAutomataLatticeWidget : public QWidget {
00010 public:
00011     explicit CellularAutomataLatticeWidget(QWidget* parent = nullptr);

```



```

00012
00013     void setController(CellularAutomataViewerController* controller);
00014
00015 protected:
00016     void paintEvent(QPaintEvent* event) override;
00017     void mousePressEvent(QMouseEvent* event) override;
00018     void mouseMoveEvent(QMouseEvent* event) override;
00019     void mouseReleaseEvent(QMouseEvent* event) override;
00020     void contextMenuEvent(QContextMenuEvent* event) override;
00021     QSize minimumSizeHint() const override;
00022
00023 private:
00024     bool _ensureSelectedPaintState();
00025     QPoint _cellAtPosition(const QPoint& widgetPosition) const;
00026     QRectF _gridRect() const;
00027     void _paintCellState(const QPoint& cellPosition);
00028     void _showStateContextMenu(const QPoint& globalPosition, const QPoint& cellPosition);
00029
00030 private:
00031     CellularAutomataViewerController* _controller = nullptr;
00032     bool _dragging = false;
00033     QPoint _lastPaintedCell = QPoint{-1, -1};
00034 };

```

11.19 CellularAutomataStateColorMap.h

```

00001 #pragma once
00002
00003 #include <QColor>
00004 #include <QString>
00005 #include <functional>
00006
00007 class CellularAutomataStateColorMap {
00008 public:
00009     static QColor colorForState(long state) {
00010         if (state <= 0) {
00011             return QColor(248, 248, 248);
00012         }
00013         if (state == 1) {
00014             return QColor(28, 30, 36);
00015         }
00016
00017         const int hue = static_cast<int>((state * 47) % 360);
00018         return QColor::fromHsv(hue, 180, 215);
00019     }
00020
00021     static QColor colorForStateText(const QString& stateText, long fallbackState) {
00022         if (stateText.isEmpty() || stateText == QString::number(fallbackState)) {
00023             return colorForState(fallbackState);
00024         }
00025         const size_t hash = std::hash<std::string>{}(stateText.toStdString());
00026         return QColor::fromHsv(static_cast<int>(hash % 360), 170, 220);
00027     }
00028
00029     static QColor backgroundColor() {
00030         return QColor(238, 239, 242);
00031     }
00032
00033     static QColor gridLineColor() {
00034         return QColor(190, 194, 201);
00035     }
00036
00037     static QColor emptyTextColor() {
00038         return QColor(110, 115, 124);
00039     }
00040 };

```

11.20 CellularAutomataViewerController.h

```

00001 #pragma once
00002
00003 #include <memory>
00004 #include <optional>
00005 #include <vector>
00006
00007 #include <QSize>
00008 #include <QString>
00009 #include <QStringList>
00010 #include <QTimer>

```

```

00011 #include <QObject>
00012
00013 #include "CellularAutomataDemoBuilder.h"
00014
00015 class CellularAutomataViewerController : public QObject {
00016     Q_OBJECT
00017 public:
00018     explicit CellularAutomataViewerController(QObject* parent = nullptr);
00019
00020     CellularAutomataDemoModel* model() const;
00021     CellularAutomataDemoSettings settings() const;
00022     QSize latticeSize() const;
00023     bool hasDemo() const;
00024     unsigned int simulatedTime() const;
00025     bool isRunning() const;
00026     QString configurationSummary() const;
00027     std::vector<long> availablePaintStates() const;
00028
00029     long cellState(int x, int y) const;
00030     bool setCellState(int x, int y, long value);
00031     void fill(long value);
00032
00033     bool hasSelectedPaintState() const;
00034     std::optional<long> selectedPaintState() const;
00035     QString selectedPaintStateText() const;
00036     void setSelectedPaintState(long state);
00037     void clearSelectedPaintState();
00038
00039     int autoStepIntervalMs() const;
00040     void setAutoStepIntervalMs(int intervalMs);
00041
00042     void rebuildDemo(CellularAutomataDemoSettings settings = {});
00043     void setSettings(CellularAutomataDemoSettings settings);
00044     void setRulePreset(CellularAutomataRulePreset preset);
00045     void setNeighborhoodPreset(CellularAutomataNeighborhoodPreset preset);
00046     void setBoundaryPreset(CellularAutomataBoundaryPreset preset);
00047     void setStatePreset(CellularAutomataStatePreset preset);
00048     void resetToInitialState();
00049     void step();
00050     void start();
00051     void pause();
00052     void stop();
00053
00054     bool saveConfiguration(const QString& filePath, QString* errorMessage = nullptr) const;
00055     bool loadConfiguration(const QString& filePath, QString* errorMessage = nullptr);
00056
00057 signals:
00058     void modelChanged();
00059     void settingsChanged();
00060     void timeChanged(unsigned int time);
00061     void runningChanged(bool running);
00062     void paintStateChanged();
00063     void statusMessage(const QString& message);
00064
00065 private slots:
00066     void _onTimerTimeout();
00067
00068 private:
00069     void _resetRuntimeState();
00070
00071 private:
00072     std::unique_ptr<CellularAutomataDemoModel> _demo;
00073     CellularAutomataDemoSettings _settings;
00074     QTimer _timer;
00075     unsigned int _simulatedTime = 0;
00076     bool _running = false;
00077     int _autoStepIntervalMs = 0;
00078     std::optional<long> _selectedPaintState;
00079 };

```

11.21 CellularAutomataViewerWindow.h

```

00001 #pragma once
00002
00003 #include <QMainWindow>
00004 #include <QString>
00005
00006 class QAction;
00007 class QComboBox;
00008 class QLabel;
00009 class CellularAutomataLatticeWidget;
00010 class CellularAutomataViewerController;
00011

```

```

00012 class CellularAutomataViewerWindow : public QMainWindow {
00013 public:
00014     explicit CellularAutomataViewerWindow(QWidget* parent = nullptr);
00015 private:
00016     void _updateTimeLabel(unsigned int time);
00017     void _updateRunningState(bool running);
00018     void _updatePaintStateLabel();
00019     void _updateConfigurationLabel();
00020     void _updateStatusMessage(const QString& message);
00021     void _refreshUi();
00022     void _syncControlsFromController();
00023     void _onRulePresetChanged(int index);
00024     void _onNeighborhoodPresetChanged(int index);
00025     void _onBoundaryPresetChanged(int index);
00026     void _onStatePresetChanged(int index);
00027     bool _promptSavePath(QString* path) const;
00028     bool _promptLoadPath(QString* path) const;
00030 private:
00031     CellularAutomataViewerController* _controller = nullptr;
00032     CellularAutomataLatticeWidget* _latticeWidget = nullptr;
00033     QComboBox* _ruleCombo = nullptr;
00034     QComboBox* _neighborhoodCombo = nullptr;
00035     QComboBox* _boundaryCombo = nullptr;
00036     QComboBox* _stateCombo = nullptr;
00037     QLabel* _timeLabel = nullptr;
00038     QLabel* _runningLabel = nullptr;
00039     QLabel* _paintStateLabel = nullptr;
00040     QLabel* _configurationLabel = nullptr;
00041     QLabel* _statusLabel = nullptr;
00042     QAction* _startAction = nullptr;
00043     QAction* _stepAction = nullptr;
00044     QAction* _continueAction = nullptr;
00045     QAction* _pauseAction = nullptr;
00046     QAction* _stopAction = nullptr;
00047     QAction* _restartAction = nullptr;
00048     QAction* _saveAction = nullptr;
00049     QAction* _loadAction = nullptr;
00050 };

```

11.22 CodeEditor.h

```

00001 #ifndef CODEEDITOR_H
00002 #define CODEEDITOR_H
00003
00004 #include <QPlainTextEdit>
00005
00006 class CodeEditor : public QPlainTextEdit {
00007     Q_OBJECT
00008 public:
00009     CodeEditor(QWidget *parent = nullptr);
00010
00011     void lineNumberAreaPaintEvent(QPaintEvent *event);
00012     int lineNumberAreaWidth();
00013
00014 protected:
00015     void resizeEvent(QResizeEvent *event) override;
00016
00017 private slots:
00018     void updateLineNumberAreaWidth(int newBlockCount);
00019     void highlightCurrentLine();
00020     void updateLineNumberArea(const QRect &rect, int dy);
00021
00022 private:
00023     QWidget *lineNumberArea;
00024 };
00025 #endif // CODEEDITOR_H

```

11.23 LineNumberArea.h

```

00001 #ifndef LINENUMBERAREA_H
00002 #define LINENUMBERAREA_H
00003
00004 #include <QWidget>
00005 #include "CodeEditor.h"
00006

```

```

00007 class LineNumberArea : public QWidget {
00008 public:
00009     LineNumberArea(CodeEditor *editor) : QWidget(editor), codeEditor(editor) {
00010     }
00011     QSize sizeHint() const override {
00012         return QSize(codeEditor->lineNumberAreaWidth(), 0);
00013     }
00014 protected:
00015     void paintEvent(QPaintEvent *event) override {
00016         codeEditor->lineNumberAreaPaintEvent(event);
00017     }
00018 private:
00019     CodeEditor *codeEditor;
00020 };
00021 #endif // LINENUMBERAREA_H

```

11.24 DialogUtilityController.h

```

00001 #ifndef DIALOGUTILITYCONTROLLER_H
00002 #define DIALOGUTILITYCONTROLLER_H
00003
00004 #include <functional>
00005
00006 class MainWindow;
00007 class Simulator;
00008 class ModelGraphicsView;
00009 class ModelGraphicsScene;
00010 class QString;
00011
00012 namespace Ui {
00013     class MainWindow;
00014 }
00015
00016 // Document the dialog and utility controller extracted from remaining MainWindow actions.
00017 class DialogUtilityController {
00018 public:
00019     DialogUtilityController(MainWindow* ownerWidget,
00020                             Simulator* simulator,
00021                             Ui::MainWindow* ui,
00022                             ModelGraphicsView* graphicsView,
00023                             std::function<void()> showMessageNotImplemented,
00024                             std::function<void(bool)> actualizeDebugBreakpoints,
00025                             std::function<bool()> createModelImage,
00026                             std::function<void()> actualizeActions,
00027                             std::function<void()> actualizeTabPanels,
00028                             std::function<void()> reloadPluginCatalog,
00029                             std::function<ModelGraphicsScene*> currentScene,
00030                             double& optimizerPrecision,
00031                             unsigned int& optimizerMaxSteps,
00032                             bool& parallelizationEnabled,
00033                             int& parallelizationThreads,
00034                             int& parallelizationBatchSize,
00035                             QString& lastDataAnalyzerPath);
00036
00037     void onActionAboutAboutTriggered();
00038     void onActionAboutLicenceTriggered();
00039     void onActionAboutGetInvolvedTriggered();
00040     void onActionAboutReportIssueTriggered();
00041     void onActionEditFindTriggered();
00042     void onActionEditReplaceTriggered();
00043     void onActionToolsParserGrammarCheckerTriggered();
00044     void onActionToolsOptimizerTriggered();
00045     void onActionToolsAIAssistantTriggered();
00046     void onActionToolsDataAnalyzerTriggered();
00047     void onActionViewConfigureTriggered();
00048     void onActionSimulatorPreferencesTriggered();
00049     void onActionSimulatorsPluginManagerTriggered(bool showProblemPlugins = false);
00050     void onPushButtonBreakpointInsertClicked();
00051     void onPushButtonBreakpointRemoveClicked();
00052     void onPushButtonExportClicked();
00053     void onActionParallelizationTriggered();
00054
00055 private:
00056     MainWindow* _ownerWidget;
00057     Simulator* _simulator;
00058     Ui::MainWindow* _ui;
00059     ModelGraphicsView* _graphicsView;

```

```

00095     std::function<void()> _showMessageNotImplemented;
00096     std::function<void(bool)> _actualizeDebugBreakpoints;
00097     std::function<bool()> _createModelImage;
00098     std::function<void()> _actualizeActions;
00099     std::function<void()> _actualizeTabPanes;
00100     std::function<void()> _reloadPluginCatalog;
00101     std::function<ModelGraphicsScene*>() _currentScene;
00102     double& _optimizerPrecision;
00103     unsigned int& _optimizerMaxSteps;
00104     bool& _parallelizationEnabled;
00105     int& _parallelizationThreads;
00106     int& _parallelizationBatchSize;
00107     QString& _lastDataAnalyzerPath;
00108 };
00109
00110 #endif // DIALOGUTILITYCONTROLLER_H

```

11.25 EditCommandController.h

```

00001 #ifndef EDITCOMMANDCONTROLLER_H
00002 #define EDITCOMMANDCONTROLLER_H
00003
00004 #include <QList>
00005 #include <QGraphicsItem>
00006 #include <functional>
00007
00008 class Simulator;
00009 class ModelGraphicsView;
00010 class GraphicalModelComponent;
00011 class GraphicalConnection;
00012 class QGraphicsItemGroup;
00013
00014 // Document edit command orchestration extracted from MainWindow wrappers.
00032 class EditCommandController {
00033 public:
00035     EditCommandController(Simulator* simulator,
00036                          ModelGraphicsView* graphicsView,
00037                          std::function<void()> actualizeActions,
00038                          bool* cut,
00039                          QList<GraphicalModelComponent*>* gmcCopies,
00040                          QList<GraphicalConnection*>* portsCopies,
00041                          QList<QGraphicsItem*>* drawCopy,
00042                          QList<QGraphicsItemGroup*>* groupCopy);
00043
00045     void onActionEditUndoTriggered() const;
00047     void onActionEditRedoTriggered() const;
00049     void onActionEditCutTriggered() const;
00051     void onActionEditCopyTriggered() const;
00053     void onActionEditPasteTriggered() const;
00055     void onActionEditDeleteTriggered() const;
00057     void onActionEditGroupTriggered() const;
00059     void onActionEditUngroupTriggered() const;
00061     void onActionViewGroupTriggered() const;
00063     void onActionViewUngroupTriggered() const;
00064
00066     void saveItemForCopy(QList<GraphicalModelComponent*>* gmcList, QList<GraphicalConnection*>*
connList) const;
00068     void helpCopy() const;
00069
00070 private:
00071     // Resolve current scene from the injected graphics view.
00072     class ModelGraphicsScene* scene() const;
00073
00074 private:
00075     Simulator* _simulator;
00076     ModelGraphicsView* _graphicsView;
00077     std::function<void()> _actualizeActions;
00078     bool* _cut;
00079     QList<GraphicalModelComponent*>* _gmcCopies;
00080     QList<GraphicalConnection*>* _portsCopies;
00081     QList<QGraphicsItem*>* _drawCopy;
00082     QList<QGraphicsItemGroup*>* _groupCopy;
00083 };
00084
00085 #endif // EDITCOMMANDCONTROLLER_H

```

11.26 GraphicalContextMenuController.h

```

00001 #ifndef GRAPHICALCONTEXTMENUCONTROLLER_H

```

```

00002 #define GRAPHICALCONTEXTMENUCONTROLLER_H
00003
00004 #include <functional>
00005
00006 class QContextMenuEvent;
00007 class QGraphicsItem;
00008 class QMenu;
00009 class QAction;
00010 class ModelGraphicsView;
00011 class ModelGraphicsScene;
00012 class ModelDataDefinition;
00013
00014 namespace Ui {
00015     class MainWindow;
00016 }
00017
00024 class GraphicalContextMenuController {
00025 public:
00029     GraphicalContextMenuController(ModelGraphicsView* graphicsView,
00030                                     Ui::MainWindow* ui,
00031                                     QAction* openSubmodelAction,
00032                                     std::function<ModelGraphicsScene*>() currentScene,
00033                                     std::function<void()> actualizeActions);
00034
00038     void handleGraphicsViewContextMenu(QContextMenuEvent* event);
00039 private:
00041     enum class ContextKind {
00042         Background,
00043         SingleComponent,
00044         SingleDataDefinition,
00045         SingleConnection,
00046         SingleDiagramConnection,
00047         SingleGroup,
00048         MultiSelection,
00049         GenericItem
00050     };
00051
00055     QGraphicsItem* semanticItemFor(QGraphicsItem* item) const;
00056
00060     void synchronizeSelectionForContext(QGraphicsItem* item) const;
00061
00065     ContextKind classifyContext(QGraphicsItem* item) const;
00066
00070     ModelDataDefinition* modelDataDefinitionFor(QGraphicsItem* item) const;
00071
00075     void populateBackgroundMenu(QMenu* menu) const;
00076
00080     void populateItemMenu(QMenu* menu, ContextKind contextKind, QGraphicsItem* clickedItem) const;
00081
00085     void addEditActions(QMenu* menu) const;
00086
00090     void addShowMenu(QMenu* menu) const;
00091
00095     void addZoomMenu(QMenu* menu) const;
00096
00100     void addTraceLevelMenu(QMenu* menu, QGraphicsItem* clickedItem) const;
00101
00105     void addBioReactionLinkMenu(QMenu* menu) const;
00106
00110     void addDrawMenu(QMenu* menu) const;
00111
00115     void addAnimateMenu(QMenu* menu) const;
00116
00120     void addAlignMenu(QMenu* menu) const;
00121
00122     ModelGraphicsView* _graphicsView;
00123     Ui::MainWindow* _ui;
00124     QAction* _openSubmodelAction;
00125     std::function<ModelGraphicsScene*>() _currentScene;
00126     std::function<void()> _actualizeActions;
00127 };
00128
00129 #endif // GRAPHICALCONTEXTMENUCONTROLLER_H

```

11.27 ModellInspectorController.h

```

00001 #ifndef MODELINSPECTORCONTROLLER_H
00002 #define MODELINSPECTORCONTROLLER_H
00003
00004 #include <QTreeWidget>
00005
00006 class Simulator;

```

```

00007 class ModelGraphicsView;
00008 class ModelDataDefinition;
00009 class GraphicalModelDataDefinition;
00010
00011 // Document the controller boundary for model-inspection and tree/scene synchronization.
00030 class ModelInspectorController {
00031 public:
00032     struct DataDefinitionSelection {
00033         bool handled = false;
00034         ModelDataDefinition* dataDefinition = nullptr;
00035         GraphicalModelDataDefinition* graphicalDataDefinition = nullptr;
00036     };
00037
00039     ModelInspectorController(Simulator* simulator,
00040                             QTreeWidget* componentsTree,
00041                             QTreeWidget* dataDefinitionsTree,
00042                             ModelGraphicsView* graphicsView);
00043
00045     void actualizeModelComponents(bool force) const;
00047     void actualizeModelDataDefinitions(bool force) const;
00049     void beginDataDefinitionNameEdit(QTreeWidgetItem* item, int column) const;
00051     void applyDataDefinitionNameChange(QTreeWidgetItem* item, int column) const;
00053     void syncSelectedComponentTreeItemToScene() const;
00055     DataDefinitionSelection syncSelectedDataDefinitionTreeItemToScene() const;
00056
00057 private:
00058     // Keep simulator access scoped to model-inspection responsibilities.
00059     Simulator* _simulator;
00060     // Keep direct access to the Components tree used by this controller.
00061     QTreeWidget* _componentsTree;
00062     // Keep direct access to the Data Definitions tree used by this controller.
00063     QTreeWidget* _dataDefinitionsTree;
00064     // Keep direct access to the graphics view for selection synchronization.
00065     ModelGraphicsView* _graphicsView;
00066 };
00067
00068 #endif // MODELINSPECTORCONTROLLER_H

```

11.28 ModelLifecycleController.h

```

00001 #ifndef MODELLIFECYCLECONTROLLER_H
00002 #define MODELLIFECYCLECONTROLLER_H
00003
00004 #include <functional>
00005 #include <string>
00006
00007 class QFile;
00008 class QString;
00009 class QWidget;
00010 class Simulator;
00011 class Model;
00012
00013 namespace Ui {
00014 class MainWindow;
00015 }
00016
00017 // Document model lifecycle orchestration delegated by MainWindow compatibility wrappers.
00036 class ModelLifecycleController {
00037 public:
00038     // Group callback dependencies to keep constructor arguments focused and explicit.
00039     struct Callbacks {
00040         std::function<void(const std::string&)> insertCommandInConsole;
00041         std::function<void(Model*)> initUiForNewModel;
00042         std::function<void()> actualizeActions;
00043         std::function<void()> actualizeTabPanels;
00044         std::function<void(bool)> actualizeModelTextHasChanged;
00045         std::function<bool()> checkModel;
00046         std::function<bool()> setSimulationModelBasedOnText;
00047         std::function<void()> clearModelEditors;
00048         std::function<bool(QString)> saveGraphicalModel;
00049         std::function<bool(QFile*, QString)> saveTextModel;
00050         std::function<Model*(std::string)> loadGraphicalModel;
00051         std::function<void()> connectSceneSignals;
00052         std::function<void(const char*)> disconnectSceneSignals;
00053     };
00054
00056     ModelLifecycleController(QWidget* ownerWidget,
00057                             Simulator* simulator,
00058                             Ui::MainWindow* ui,
00059                             QString* modelFilename,
00060                             bool* textModelHasChanged,
00061                             bool* graphicalModelHasChanged,
00062                             bool* closingApproved,

```

```

00063         bool* loaded,
00064         bool& parallelizationEnabled,
00065         int& parallelizationThreads,
00066         int& parallelizationBatchSize,
00067         Callbacks callbacks);
00068
00070     void onActionModelNewTriggered() const;
00072     void onActionModelOpenTriggered() const;
00074     bool openModelFile(const QString& fileName) const;
00076     void onActionModelSaveTriggered() const;
00078     void onActionModelCloseTriggered() const;
00080     void onActionModelInformationTriggered() const;
00082     void onActionModelCheckTriggered() const;
00084     void onActionSimulationConfigureTriggered() const;
00086     void onActionSimulatorExitTriggered() const;
00088     bool hasPendingModelChanges() const;
00090     bool confirmApplicationExit() const;
00091
00092 private:
00093     bool openModelFileInternal(const QString& fileName, bool showDialogs) const;
00094
00095     // Keep QObject ownership separate while allowing dialog parenting.
00096     QWidget* _ownerWidget;
00097     Simulator* _simulator;
00098     Ui::MainWindow* _ui;
00099     QString* _modelFilename;
00100     bool* _textModelHasChanged;
00101     bool* _graphicalModelHasChanged;
00102     bool* _closingApproved;
00103     bool* _loaded;
00104     bool& _parallelizationEnabled;
00105     int& _parallelizationThreads;
00106     int& _parallelizationBatchSize;
00107     Callbacks _callbacks;
00108 };
00109
00110 #endif // MODELLIFECYCLECONTROLLER_H

```

11.29 PluginCatalogController.h

```

00001 #ifndef PLUGINCATALOGCONTROLLER_H
00002 #define PLUGINCATALOGCONTROLLER_H
00003
00004 #include <map>
00005 #include <string>
00006 #include <vector>
00007
00008 #include <QColor>
00009
00010 class Plugin;
00011 class Simulator;
00012 class QPlainTextEdit;
00013 class QPoint;
00014 class QString;
00015 class QTreeWidgetItem;
00016 class QTreeWidgetItem;
00017
00018 // Document the plugin-catalog controller extracted from MainWindow UI code.
00036 class PluginCatalogController {
00037 public:
00039     PluginCatalogController(Simulator* simulator,
00040                             QTreeWidgetItem* pluginsTree,
00041                             QPlainTextEdit* modelTextEditor,
00042                             std::map<std::string, QColor>* pluginCategoryColor);
00043
00045     void insertPluginUI(Plugin* plugin) const;
00047     void reloadFromPluginManager() const;
00049     void applyCategoryExpansionPolicy() const;
00051     void showContextMenu(const QPoint& pos) const;
00053     void insertFakePlugins() const;
00055     void handlePluginItemDoubleClicked(QTreeWidgetItem* item, int column) const;
00057     void handlePluginItemClicked(QTreeWidgetItem* item, int column) const;
00059     void moveTopLevelCategory(QTreeWidgetItem* item, int delta) const;
00060
00061 private:
00062     static QString topLevelCategory(const QString& categoryPath);
00063     QTreeWidgetItem* ensureCategoryPath(const QString& categoryPath) const;
00064     QTreeWidgetItem* ensureCategoryChild(QTreeWidgetItem* parent, const QString& segment) const;
00065     QTreeWidgetItem* findDirectChild(QTreeWidgetItem* parent, const QString& text) const;
00066     QColor backgroundColorForCategory(const QString& categoryPath) const;
00067     void rememberTopLevelCategory(const QString& topLevelCategory) const;
00068     void applyTopLevelCategoryOrder() const;
00069     void applyExpansionPolicyRecursive(QTreeWidgetItem* item) const;

```



```

00070     void registerCategoryColorPrefixes(const QString& categoryPath, const QColor& categoryColor)
00071     const;
00072 private:
00073     Simulator* _simulator;
00074     QTreeWidget* _pluginsTree;
00075     QPlainTextEdit* _modelTextEditor;
00076     std::map<std::string, QColor>* _pluginCategoryColor;
00077     mutable std::vector<std::string> _topLevelCategoryOrder;
00078 };
00079
00080 #endif /* PLUGINCATALOGCONTROLLER_H */

```

11.30 PropertyEditorController.h

```

00001 #ifndef PROPERTYEDITORCONTROLLER_H
00002 #define PROPERTYEDITORCONTROLLER_H
00003
00004 #include <functional>
00005 #include <map>
00006
00007 class ObjectPropertyBrowser;
00008 class ModelGraphicsView;
00009 class PropertyEditorGenesys;
00010 class SimulationControl;
00011 class DataComponentProperty;
00012 class DataComponentEditor;
00013 class ComboBoxEnum;
00014 class ModelDataDefinition;
00015 class QObject;
00016
00017 // Document the property-editor controller and post-commit synchronization pipeline.
00035 class PropertyEditorController {
00036 public:
00037     PropertyEditorController(ObjectPropertyBrowser* propertyBrowser,
00038                             ModelGraphicsView* graphicsView,
00039                             PropertyEditorGenesys* propertyGenesys,
00040                             std::map<SimulationControl*, DataComponentProperty*>* propertyList,
00041                             std::map<SimulationControl*, DataComponentEditor*>* propertyEditorUI,
00042                             std::map<SimulationControl*, ComboBoxEnum*>* propertyBox,
00043                             std::function<void()> actualizeModelSimLanguage,
00044                             std::function<void(bool)> actualizeModelComponents,
00045                             std::function<void(bool)> actualizeModelDataDefinitions,
00046                             std::function<void()> actualizeModelCppCode,
00047                             std::function<bool()> createModelImage,
00048                             std::function<void()> actualizeTabPanels,
00049                             std::function<void()> actualizeActions);
00050
00051     void sceneSelectionChanged() const;
00052     void onPropertyEditorModelChanged() const;
00053     void clearPropertyEditorSelection() const;
00054     void bindDataDefinitionFromInspector(ModelDataDefinition* dataDefinition, QObject* graphicalObject
00055     = nullptr) const;
00056     bool isPostCommitPipelineActive() const;
00057 private:
00058     // Keep direct access to the property browser used by this controller.
00059     ObjectPropertyBrowser* _propertyBrowser;
00060     // Keep direct access to the graphical view used for selection inspection.
00061     ModelGraphicsView* _graphicsView;
00062     // Keep direct access to property-editor dependencies required for activation.
00063     PropertyEditorGenesys* _propertyGenesys;
00064     std::map<SimulationControl*, DataComponentProperty*>* _propertyList;
00065     std::map<SimulationControl*, DataComponentEditor*>* _propertyEditorUI;
00066     std::map<SimulationControl*, ComboBoxEnum*>* _propertyBox;
00067
00068     // Keep narrow callbacks for MainWindow-owned update methods.
00069     std::function<void()> _actualizeModelSimLanguage;
00070     std::function<void(bool)> _actualizeModelComponents;
00071     std::function<void(bool)> _actualizeModelDataDefinitions;
00072     std::function<void()> _actualizeModelCppCode;
00073     std::function<bool()> _createModelImage;
00074     std::function<void()> _actualizeTabPanels;
00075     std::function<void()> _actualizeActions;
00076
00077     // Coalesce and serialize heavy refresh requests after property commits.
00078     mutable bool _isGlobalRefreshQueued = false;
00079     mutable bool _isGlobalRefreshRunning = false;
00080     mutable bool _pendingGlobalRefresh = false;
00081     // Coalesce deferred selection synchronization while commit/refresh is active.
00082     mutable bool _isDeferredSelectionSyncScheduled = false;
00083
00084     void _runGlobalRefresh() const;
00085
00086
00087
00088
00089
00090

```

```

00091 };
00092
00093 #endif // PROPERTYEDITORCONTROLLER_H

```

11.31 SceneToolController.h

```

00001 #ifndef SCENETOOLCONTROLLER_H
00002 #define SCENETOOLCONTROLLER_H
00003
00004 #include <functional>
00005 #include <QRectF>
00006
00007 class ModelGraphicsView;
00008 class ModelGraphicsScene;
00009 class QAction;
00010
00011 namespace Ui {
00012 class MainWindow;
00013 }
00014
00015 // Document the scene tooling controller that receives delegated UI commands.
00032 class SceneToolController {
00033 public:
00037     SceneToolController(ModelGraphicsView* graphicsView,
00038                         Ui::MainWindow* ui,
00039                         std::function<ModelGraphicsScene*>()> currentScene,
00040                         std::function<bool()> createModelImage,
00041                         std::function<void()> unselectDrawIcons,
00042                         std::function<bool()> checkSelectedDrawIcons,
00043                         std::function<void(double)> gentleZoom,
00044                         std::function<void()> actualizeActions,
00045                         std::function<void()> actualizeTabPanels,
00046                         int& zoomValue,
00047                         bool& firstClickShowConnection);
00048
00050     void onActionShowGridTriggered();
00052     void onActionShowRuleTriggered();
00054     void onActionShowGuidesTriggered();
00056     void onActionShowSnapTriggered();
00058     void onActionZoomInTriggered();
00060     void onActionZoomOutTriggered();
00062     void onActionZoomAllTriggered();
00064     void onActionZoomActualSizeTriggered();
00066     void syncZoomWidgetsForCurrentScale();
00068     void onHorizontalSliderZoomGraphicalValueChanged(int value);
00070     void onActionDrawLineTriggered();
00072     void onActionDrawRectangleTriggered();
00074     void onActionDrawEllipseTriggered();
00076     void onActionDrawTextTriggered();
00078     void onActionDrawPolygonTriggered();
00080     void onActionAnimateSimulatedTimeTriggered();
00082     void onActionAnimateVariableTriggered();
00084     void onActionAnimateCounterTriggered();
00086     void onActionAnimateAttributeTriggered();
00088     void onActionAnimateEntityTriggered();
00090     void onActionAnimateEventTriggered();
00092     void onActionAnimateExpressionTriggered();
00094     void onActionAnimatePlotTriggered();
00096     void onActionAnimateQueueTriggered();
00098     void onActionAnimateResourceTriggered();
00100     void onActionAnimateStationTriggered();
00102     void onActionAnimateStatisticsTriggered();
00104     void onActionConnectTriggered();
00106     void onActionArrangeLeftTriggered();
00108     void onActionArrangeRightTriggered();
00110     void onActionArrangeTopTriggered();
00112     void onActionArrangeBottomTriggered();
00114     void onActionArrangeCenterTriggered();
00116     void onActionArrangeMiddleTriggered();
00118     void onActionActivateGraphicalSimulationTriggered();
00120     void onHorizontalSliderAnimationSpeedValueChanged(int value);
00122     void onActionSelectAllTriggered();
00124     void onActionShowInternalElementsTriggered();
00126     void onActionShowEditableElementsTriggered();
00128     void onActionShowAttachedElementsTriggered();
00130     void onCheckBoxShowElementsStateChanged(int arg1);
00132     void onCheckBoxShowInternalsStateChanged(int arg1);
00134     void onCheckBoxShowEditableElementsStateChanged(int arg1);
00136     void onActionShowRecursiveElementsTriggered();
00138     void onCheckBoxShowRecursiveStateChanged(int arg1);
00140     void onCheckBoxShowLevelsStateChanged(int arg1);
00142     void onActionGModelShowConnectTriggered();
00143

```

```

00144 private:
00145     QRectF _zoomableModelBounds() const;
00146     double _currentViewScale() const;
00147     double _minimumZoomScale() const;
00148     double _maximumZoomScale() const;
00149     double _zoomScaleFromSliderValue(int value) const;
00150     int _sliderValueForZoomScale(double scale) const;
00151     void _syncZoomWidgetsForCurrentScale();
00152
00153     void activateAnimationDrawingTool(QAction* action, void (ModelGraphicsScene::*drawingFunction)());
00154
00155     ModelGraphicsView* _graphicsView;
00156     Ui::MainWindow* _ui;
00157     std::function<ModelGraphicsScene*>()> _currentScene;
00158     std::function<bool()> _createModelImage;
00159     std::function<void()> _unselectDrawIcons;
00160     std::function<bool()> _checkSelectedDrawIcons;
00161     std::function<void(double)> _gentleZoom;
00162     std::function<void()> _actualizeActions;
00163     std::function<void()> _actualizeTabPanels;
00164     int& _zoomValue;
00165     bool& _firstClickShowConnection;
00166 };
00167
00168 #endif // SCENETOOLCONTROLLER_H

```

11.32 SimulationCommandController.h

```

00001 #ifndef SIMULATIONCOMMANDCONTROLLER_H
00002 #define SIMULATIONCOMMANDCONTROLLER_H
00003
00004 #include <functional>
00005 #include <string>
00006
00007 class SimulationController;
00008
00009 // Document simulation command orchestration delegated by MainWindow slots.
00027 class SimulationCommandController {
00028 public:
00032     SimulationCommandController(
00033         SimulationController* simulationController,
00034         const std::function<void(const std::string&)>& insertCommandInConsole,
00035         const std::function<void()>& actualizeActions,
00036         const std::function<bool()>& checkModel,
00037         const std::function<bool()>& setSimulationModelBasedOnText,
00038         const std::function<void()>& prepareNewSimulationOutputs);
00039
00041     void onActionSimulationStartTriggered(bool modelChecked) const;
00043     void onActionSimulationStepTriggered(bool modelChecked) const;
00045     void onActionSimulationPauseTriggered() const;
00047     void onActionSimulationResumeTriggered(bool modelChecked) const;
00049     void onActionSimulationStopTriggered() const;
00050
00051 private:
00052     SimulationController* _simulationController;
00053     std::function<void(const std::string&)> _insertCommandInConsole;
00054     std::function<void()> _actualizeActions;
00055     std::function<bool()> _checkModel;
00056     std::function<bool()> _setSimulationModelBasedOnText;
00057     std::function<void()> _prepareNewSimulationOutputs;
00058 };
00059
00060 #endif // SIMULATIONCOMMANDCONTROLLER_H

```

11.33 SimulationController.h

```

00001 #ifndef SIMULATIONCONTROLLER_H
00002 #define SIMULATIONCONTROLLER_H
00003
00004 #include <functional>
00005
00006 class QWidget;
00007 class Simulator;
00008 class ModelSimulation;
00009
00010 // Document the simulation precondition controller role in the refactored GUI architecture.
00032 class SimulationController {
00033 public:
00039     SimulationController(QWidget* ownerWidget, Simulator* simulator);

```

```

00040
00045     bool hasCurrentModelSimulation() const;
00046
00051     ModelSimulation* currentSimulation() const;
00052
00069     bool ensureReady(bool checkModel,
00070                     bool modelChecked,
00071                     const std::function<bool()>& runModelCheck,
00072                     const std::function<bool()>& syncModelFromText) const;
00073
00074 private:
00075     QWidget* _ownerWidget;
00076     Simulator* _simulator;
00077 };
00078
00079 #endif // SIMULATIONCONTROLLER_H

```

11.34 SimulationEventController.h

```

00001 #ifndef SIMULATIONEVENTCONTROLLER_H
00002 #define SIMULATIONEVENTCONTROLLER_H
00003
00004 #include <functional>
00005
00006 #include "../kernel/simulator/model/ModelSimulation.h"
00007 #include "kernel/simulator/OnEventManager.h"
00008 #include "kernel/simulator/Simulator.h"
00009
00010 class MainWindow;
00011 class ModelGraphicsScene;
00012 class ModelGraphicsView;
00013 class QLabel;
00014 class QProgressBar;
00015 class QTableWidgetItem;
00016 class QTextEdit;
00017 class QTabWidget;
00018 class QAction;
00019
00020 // Document the controller that bridges kernel simulation events to GUI updates.
00040 class SimulationEventController {
00041 public:
00042     // Group callback dependencies to keep constructor explicit and narrow.
00043     struct Callbacks {
00044         std::function<void()> actualizeActions;
00045         std::function<void(SimulationEvent*)> actualizeSimulationEvents;
00046         std::function<void(bool)> actualizeDebugEntities;
00047         std::function<void(bool)> actualizeDebugVariables;
00048         std::function<void(SimulationEvent*)> actualizeGraphicalModel;
00049     };
00050
00052     SimulationEventController(Simulator* simulator,
00053                             ModelGraphicsScene* scene,
00054                             ModelGraphicsView* graphicsView,
00055                             QLabel* replicationLabel,
00056                             QProgressBar* simulationProgressBar,
00057                             QTableWidgetItem* simulationEventsTable,
00058                             QTableWidgetItem* entitiesTable,
00059                             QTableWidgetItem* variablesTable,
00060                             QTextEdit* simulationText,
00061                             QTextEdit* reportsText,
00062                             QTabWidget* centralTabWidget,
00063                             QAction* activateGraphicalSimulation,
00064                             QAction* animationEnabledAction,
00065                             bool* modelChecked,
00066                             int tabCentralReportsIndex,
00067                             Callbacks callbacks);
00068
00070     void onModelCheckSuccessHandler(ModelEvent* re) const;
00072     void onReplicationStartHandler(SimulationEvent* re) const;
00074     void onSimulationStartHandler(SimulationEvent* re) const;
00076     void onSimulationPausedHandler(SimulationEvent* re) const;
00078     void onSimulationResumeHandler(SimulationEvent* re) const;
00080     void onSimulationEndHandler(SimulationEvent* re) const;
00082     void onReplicationEndHandler(SimulationEvent* re) const;
00084     void onProcessEventHandler(SimulationEvent* re) const;
00086     void onEntityCreateHandler(SimulationEvent* re) const;
00088     void onEntityRemoveHandler(SimulationEvent* re) const;
00090     void onMoveEntityEvent(SimulationEvent* re) const;
00092     void onAfterProcessEvent(SimulationEvent* re) const;
00094     void setOnEventHandlers(MainWindow* owner) const;
00095
00096 private:
00097     // Keep simulator access scoped to simulation-event responsibilities.

```

```

00098     Simulator* _simulator;
00099     // Keep scene dependency for animation-related handlers.
00100     ModelGraphicsScene* _scene;
00101     // Keep graphics-view dependency for legacy compatibility and potential scene access.
00102     ModelGraphicsView* _graphicsView;
00103     // Keep replication label dependency for replication-start updates.
00104     QLabel* _replicationLabel;
00105     // Keep simulation progress bar dependency for process-event updates.
00106     QProgressBar* _simulationProgressBar;
00107     // Keep simulation-events table dependency for replication-start updates.
00108     QTableWidgetItem* _simulationEventsTable;
00109     // Keep entities table dependency for simulation-start resets.
00110     QTableWidgetItem* _entitiesTable;
00111     // Keep variables table dependency for simulation-start resets.
00112     QTableWidgetItem* _variablesTable;
00113     // Keep simulation text dependency for simulation-start resets.
00114     QTextEdit* _simulationText;
00115     // Keep reports text dependency for simulation-start resets.
00116     QTextEdit* _reportsText;
00117     // Keep central-tab dependency for simulation-end tab switch.
00118     QTabWidget* _centralTabWidget;
00119     // Keep action dependency used by graphical simulation toggle.
00120     QAction* _activateGraphicalSimulation;
00121     // Keep action dependency used by the global Simulation/Animation runtime gate.
00122     QAction* _animationEnabledAction;
00123     // Keep mutable model-check flag dependency used at simulation end.
00124     bool* _modelChecked;
00125     // Keep tab index dependency used for report-tab activation.
00126     int _tabCentralReportsIndex;
00127     // Keep callback dependencies for delegated MainWindow operations.
00128     Callbacks _callbacks;
00129     // Throttle GUI event processing when animation/output updates are suppressed.
00130     mutable unsigned int _suppressedUiProgressEvents = 0;
00131
00132     bool animationsEnabled() const;
00133     void processSuppressedUiProgressEvents() const;
00134 };
00135
00136 #endif // SIMULATIONEVENTCONTROLLER_H

```

11.35 TraceConsoleController.h

```

00001 #ifndef TRACECONSOLECONTROLLER_H
00002 #define TRACECONSOLECONTROLLER_H
00003
00004 #include "kernel/simulator/TraceManager.h"
00005
00006 #include <functional>
00007
00008 class QTextEdit;
00009
00010 // Document the trace rendering bridge used by MainWindow simulator callbacks.
00028 class TraceConsoleController {
00029 public:
00031     TraceConsoleController(QTextEdit* console,
00032                           QTextEdit* simulationText,
00033                           QTextEdit* reportsText,
00034                           std::function<bool()> traceWidgetsEnabled = {});
00035
00037     void simulatorTraceHandler(TraceEvent e) const;
00039     void simulatorTraceErrorHandler(TraceErrorEvent e) const;
00041     void simulatorTraceSimulationHandler(TraceSimulationEvent e) const;
00043     void simulatorTraceReportsHandler(TraceEvent e) const;
00044
00045 private:
00046     // Keep the console trace sink used by normal/error traces.
00047     QTextEdit* _console;
00048     // Keep the simulation trace sink used by simulation traces.
00049     QTextEdit* _simulationText;
00050     // Keep the reports trace sink used by reports traces.
00051     QTextEdit* _reportsText;
00052     // Runtime gate used to avoid expensive trace-widget churn during non-animated simulations.
00053     std::function<bool()> _traceWidgetsEnabled;
00054
00055     bool shouldRenderTraceWidgets() const;
00056 };
00057
00058 #endif // TRACECONSOLECONTROLLER_H

```

11.36 dialogBreakpoint.h

```

00001 /*
00002  * The MIT License
00003  *
00004  * Copyright 2022 rlcancian.
00005  *
00006  * Permission is hereby granted, free of charge, to any person obtaining a copy
00007  * of this software and associated documentation files (the "Software"), to deal
00008  * in the Software without restriction, including without limitation the rights
00009  * to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00010  * copies of the Software, and to permit persons to whom the Software is
00011  * furnished to do so, subject to the following conditions:
00012  *
00013  * The above copyright notice and this permission notice shall be included in
00014  * all copies or substantial portions of the Software.
00015  *
00016  * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017  * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018  * FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00019  * AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00020  * LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00021  * OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
00022  * THE SOFTWARE.
00023  */
00024
00025 /*
00026  * File:    dialogBreakpoint.h
00027  * Author:  rlcancian
00028  *
00029  * Created on 6 de fevereiro de 2022, 14:24
00030  */
00031
00032 #ifndef _DIALOGBREAKPOINT_H
00033 #define _DIALOGBREAKPOINT_H
00034
00035 #include "ui_dialogBreakpoint.h"
00036 #include "kernel/simulator/Simulator.h"
00037
00038 class dialogBreakpoint : public QDialog {
00039
00040     Q_OBJECT
00041 public:
00042     class MVCResult {
00043     public:
00044
00045         MVCResult(std::string type, std::string on) {
00046             this->type = type;
00047             this->on = on;
00048         }
00049         std::string type;
00050         std::string on;
00051     };
00052 public:
00053     dialogBreakpoint();
00054     virtual ~dialogBreakpoint();
00055 public:
00056     void setMVCModel(Simulator* simulator);
00057     dialogBreakpoint::MVCResult* getMVCResult();
00058 private slots:
00059     void on_comboBox_Type_activated(const QString &arg1);
00060     void on_comboBox_On_activated(const QString &arg1);
00061     void on_buttonBox_accepted();
00062 private:
00063     Ui::dialogBreakpoint widget;
00064     Simulator* simulator;
00065 };
00066
00067 #endif /* _DIALOGBREAKPOINT_H */

```

11.37 DialogFind.h

```

00001 #ifndef DIALOGFIND_H
00002 #define DIALOGFIND_H
00003
00004 #include <QDialog>
00005 #include <QLabel>
00006 #include <QHBoxLayout>
00007 #include <QLineEdit>
00008 #include <QPushButton>
00009 #include <QMessageBox>
00010 #include "../graphics/GraphicalModelComponent.h"
00011 #include "graphics/ModelGraphicsScene.h"

```

```

00012 #include <QGraphicsItem>
00013 #include <QList>
00014
00015
00016 class QLineEdit;
00017 class QPushButton;
00018
00019 class FindLineEdit : public QLineEdit {
00020     Q_OBJECT
00021
00022 public:
00023     FindLineEdit(QWidget *parent = nullptr) : QLineEdit(parent) {
00024         connect(this, &QLineEdit::textChanged, this, &FindLineEdit::onTextChanged);
00025     }
00026
00027 private slots:
00028     void onTextChanged(const QString &text) {
00029         // qDebug() << "Texto atual: " << text;
00030
00031         if (text.length() < previousText.length()) {
00032             this->setStyleSheet("color: black");
00033         }
00034
00035         previousText = text;
00036     }
00037
00038 private:
00039     QString previousText;
00040 };
00041
00042
00043 class DialogFind : public QDialog
00044 {
00045     Q_OBJECT
00046
00047 public:
00048     DialogFind(QWidget *parent = nullptr, ModelGraphicsScene * scene = nullptr);
00049     QString getFindText();
00050
00051 private slots:
00052     void findClicked();
00053     void nextClicked();
00054     void previousClicked();
00055
00056 private:
00057     QPushButton *findButton;
00058     QPushButton *nextButton;
00059     QPushButton *previousButton;
00060     FindLineEdit *lineEdit;
00061     bool clicked;
00062     int counter;
00063     QList<QGraphicsItem*> * finded = new QList<QGraphicsItem*>();
00064     ModelGraphicsScene * scene;
00065
00066 };
00067
00068 #endif // DIALOGFIND_H
00069
00070

```

11.38 Dialogmodelinformation.h

```

00001 #ifndef DIALOGMODELINFORMATION_H
00002 #define DIALOGMODELINFORMATION_H
00003
00004 #include "../kernel/simulator/model/ModelInfo.h"
00005 #include <QDialog>
00006 #include <string>
00007
00008 namespace Ui {
00009     class DialogModelInformation;
00010 }
00011
00012 class DialogModelInformation : public QDialog
00013 {
00014     Q_OBJECT
00015
00016 public:
00017     explicit DialogModelInformation(QWidget *parent = nullptr);
00018     ~DialogModelInformation();
00019     void setModelInfo(ModelInfo* modelInfo);
00020
00021 private slots:

```

```

00022     void accept() override;
00023     void reject() override;
00024
00025 private:
00026     void _loadModelInfo();
00027     bool _hasPendingChanges() const;
00028     std::string _projectTitleFromUi() const;
00029     std::string _nameFromUi() const;
00030     std::string _versionFromUi() const;
00031     std::string _descriptionFromUi() const;
00032     std::string _analystNameFromUi() const;
00033
00034     Ui::DialogModelInformation *ui;
00035     ModelInfo* _modelInfo = nullptr;
00036     std::string _originalProjectTitle;
00037     std::string _originalName;
00038     std::string _originalVersion;
00039     std::string _originalDescription;
00040     std::string _originalAnalystName;
00041 };
00042
00043 #endif // DIALOGMODELINFORMATION_H

```

11.39 dialogpluginmanager.h

```

00001 #ifndef DIALOGPLUGINMANAGER_H
00002 #define DIALOGPLUGINMANAGER_H
00003
00004 #include <QDialog>
00005
00006 #include <functional>
00007 #include <string>
00008 #include <vector>
00009
00010 class Plugin;
00011 class PluginInformation;
00012 class PluginLoadIssue;
00013 class SystemDependencyCheckResult;
00014 class Simulator;
00015 class QTableWidgetItem;
00016 class QTableWidgetItem;
00017
00018 namespace Ui {
00019     class DialogPluginManager;
00020 }
00021
00022 class DialogPluginManager : public QDialog {
00023     Q_OBJECT
00024
00025 public:
00026     explicit DialogPluginManager(QWidget* parent = nullptr);
00027     ~DialogPluginManager();
00028     void setSimulator(Simulator* simulator);
00029     void setPluginCatalogRefreshCallback(std::function<void()> refreshCallback);
00030     void showProblemPluginsTab();
00031
00032 private slots:
00033     void on_pushButtonBrowseAutoload_clicked();
00034     void on_pushButtonAutoLoadNow_clicked();
00035     void on_pushButtonCheck_clicked();
00036     void on_pushButtonInsert_clicked();
00037     void on_pushButtonResolveSelected_clicked();
00038     void on_pushButtonRemove_clicked();
00039     void on_pushButtonRefresh_clicked();
00040
00041 private:
00042     void _refreshPluginTable();
00043     void _configureLoadedPluginTable();
00044     void _configurePluginIssuesTable();
00045     void _refreshPluginCatalog();
00046     void _appendPluginIssueRow(const PluginLoadIssue& issue);
00047     Plugin* _selectedPlugin() const;
00048     std::vector<PluginLoadIssue> _selectedPluginLoadIssues() const;
00049     bool _isKernelPlugin(const Plugin* plugin) const;
00050     bool _isPluginUsedByCurrentModel(const Plugin* plugin) const;
00051     bool _choosePluginFilename(std::string& filename) const;
00052     QString _formatDynamicDependencyTableText(const PluginInformation* info) const;
00053     QString _formatSystemDependencyTableText(const PluginInformation* info) const;
00054     QString _formatSystemDependencyPreflight(const SystemDependencyCheckResult& result) const;
00055     QString _formatFields(const PluginInformation* info) const;
00056     bool _confirmSystemDependencyInstallation(const SystemDependencyCheckResult& result) const;
00057     bool _runInstallCommandsForIssue(const PluginLoadIssue& issue, QString* feedback);
00058     bool _runInstallCommandInteractive(const QString& command, QString* feedback);

```



```

00078     bool _terminalCommandForScript(const QString& script, QString* program, QStringList* arguments)
00079     const;
00080     void _configureCommonTable(QTableWidget* table) const;
00081     void _showOperationResult(const QString& title, const QString& message) const;
00082
00083     Ui::DialogPluginManager* ui;
00084     Simulator* _simulator = nullptr;
00085     std::function<void()> _refreshPluginCatalogCallback;
00086 };
00087
00088 #endif // DIALOGPLUGINMANAGER_H

```

11.40 DialogReportIssue.h

```

00001 #ifndef DIALOGREPORTISSUE_H
00002 #define DIALOGREPORTISSUE_H
00003
00004 #include <QDialog>
00005
00006 #include "../services/IssueReportRelayService.h"
00007
00008 class QCheckBox;
00009 class QComboBox;
00010 class QLabel;
00011 class QLineEdit;
00012 class QPlainTextEdit;
00013 class QPushButton;
00014
00028 class DialogReportIssue : public QDialog {
00029 public:
00030     explicit DialogReportIssue(QWidget* parent = nullptr);
00031
00033     IssueReportDraft draft() const;
00035     void setRelayConfigurationStatus(const QString& endpoint, bool configured, const QString& detail);
00036
00037 private:
00039     void _updateSendButton();
00040
00041     QComboBox* _categoryCombo = nullptr;
00042     QLineEdit* _titleEdit = nullptr;
00043     QPlainTextEdit* _summaryEdit = nullptr;
00044     QPlainTextEdit* _stepsEdit = nullptr;
00045     QPlainTextEdit* _expectedEdit = nullptr;
00046     QPlainTextEdit* _observedEdit = nullptr;
00047     QPlainTextEdit* _additionalContextEdit = nullptr;
00048     QLineEdit* _contactEdit = nullptr;
00049     QCheckBox* _includeScreenshotCheck = nullptr;
00050     QCheckBox* _includeLogTailCheck = nullptr;
00051     QCheckBox* _includeModelTextCheck = nullptr;
00052     QLabel* _relayStatusLabel = nullptr;
00053     QPushButton* _sendButton = nullptr;
00054     bool _relayConfigured = true;
00055 };
00056
00057 #endif // DIALOGREPORTISSUE_H

```

11.41 DialogSelectCounter.h

```

00001 #ifndef DIALOGSELECTCOUNTER_H
00002 #define DIALOGSELECTCOUNTER_H
00003
00004 #include <QDialog>
00005 #include <QComboBox>
00006 #include <QPushButton>
00007
00008 #include "../kernel/simulator/essentialPlugins/Counter.h"
00009 class DialogSelectCounter : public QDialog {
00010     Q_OBJECT
00011
00012 public:
00013     explicit DialogSelectCounter(QWidget *parent = nullptr);
00014
00015     Counter* selectedIndex() const;
00016     void setCounterItems(QList<Counter *> *counters, Counter *counter);
00017
00018 private slots:
00019     void onOkClicked();
00020     void onCancelClicked();
00021

```

```

00022 private:
00023     QComboBox *comboBox;
00024     QPushButton *okButton;
00025     QPushButton *cancelButton;
00026     QMap<int, Counter*> indexToCounter;
00027 };
00028
00029 #endif // DIALOGSELECTCOUNTER_H

```

11.42 DialogSelectVariable.h

```

00001 #ifndef DIALOGSELECTVARIABLE_H
00002 #define DIALOGSELECTVARIABLE_H
00003
00004 #include <QDialog>
00005 #include <QComboBox>
00006 #include <QPushButton>
00007 #include <QList>
00008 #include <QMap>
00009
00010 class ModelDataDefinition;
00011
00012 class DialogSelectVariable : public QDialog {
00013     Q_OBJECT
00014
00015 public:
00016     explicit DialogSelectVariable(QWidget *parent = nullptr);
00017
00018     ModelDataDefinition* selectedIndex() const;
00019     void setVariableItems(QList<ModelDataDefinition *> *definitions, ModelDataDefinition *definition);
00020
00021 private slots:
00022     void onOkClicked();
00023     void onCancelClicked();
00024
00025 private:
00026     QComboBox *comboBox;
00027     QPushButton *okButton;
00028     QPushButton *cancelButton;
00029     QMap<int, ModelDataDefinition*> indexToVariable;
00030 };
00031
00032 #endif // DIALOGSELECTVARIABLE_H

```

11.43 dialogsimulationconfigure.h

```

00001 #ifndef DIALOGSIMULATIONCONFIGURE_H
00002 #define DIALOGSIMULATIONCONFIGURE_H
00003
00004 #include "../kernel/simulator/model/ModelSimulation.h"
00005 #include <QDialog>
00006 #include <string>
00007
00008 class QComboBox;
00009 class ExperimentManager;
00010 class SimulationReporter_if;
00011
00012 namespace Ui {
00013     class DialogSimulationConfigure;
00014 }
00015
00016 class DialogSimulationConfigure : public QDialog
00017 {
00018     Q_OBJECT
00019
00020 public:
00021     explicit DialogSimulationConfigure(QWidget *parent = nullptr);
00022     ~DialogSimulationConfigure();
00023     void setModelSimulation(ModelSimulation* modelSimulation);
00024     void setExperimentManager(ExperimentManager* experimentManager);
00025     void setParallelizationSettings(bool* enabled, int* localThreads, int* batchSize);
00026
00027 public slots:
00028     void accept() override;
00029     void reject() override;
00030
00031 private:
00032     struct SimulationConfiguration {
00033         unsigned int numberOfReplications = 1;

```

```

00034         Util::TimeUnit replicationBaseTimeUnit = Util::TimeUnit::unknown;
00035         double replicationLength = 0.0;
00036         Util::TimeUnit replicationLengthTimeUnit = Util::TimeUnit::unknown;
00037         double warmUpPeriod = 0.0;
00038         Util::TimeUnit warmUpPeriodTimeUnit = Util::TimeUnit::unknown;
00039         std::string terminatingCondition;
00040         bool initializeSystem = true;
00041         bool initializeStatistics = true;
00042         bool stepByStep = false;
00043         bool pauseOnEvent = false;
00044         bool pauseOnReplication = false;
00045         bool parallelizationEnabled = false;
00046         int parallelizationThreads = 1;
00047         int parallelizationBatchSize = 100;
00048         bool distributedParallelizationEnabled = false;
00049         std::string distributedCoordinatorUrl;
00050         std::string distributedToken;
00051     };
00052
00053     void _populateTimeUnitComboBoxes();
00054     void _loadModelSimulation();
00055     void _loadSimulationReporter();
00056     void _loadExperimentManager();
00057     void _loadParallelizationSettings();
00058     SimulationConfiguration _configurationFromUi() const;
00059     bool _hasPendingChanges() const;
00060     Util::TimeUnit _timeUnitFromComboBox(const QComboBox* comboBox) const;
00061     void _setTimeUnitComboBoxIndex(QComboBox* comboBox, Util::TimeUnit timeUnit);
00062     void _applyIfChanged(const SimulationConfiguration& edited);
00063
00064     Ui::DialogSimulationConfigure *ui;
00065     ModelSimulation* _modelSimulation = nullptr;
00066     SimulationReporter_if* _simulationReporter = nullptr;
00067     ExperimentManager* _experimentManager = nullptr;
00068     bool* _parallelizationEnabled = nullptr;
00069     int* _parallelizationThreads = nullptr;
00070     int* _parallelizationBatchSize = nullptr;
00071     SimulationConfiguration _originalConfiguration;
00072 };
00073
00074 #endif // DIALOGSIMULATIONCONFIGURE_H

```

11.44 dialogsystempreferences.h

```

00001 #ifndef DIALOGSYSTEMPREFERENCES_H
00002 #define DIALOGSYSTEMPREFERENCES_H
00003
00004 #include <QDialog>
00005
00006 class Simulator;
00007
00008 namespace Ui {
00009     class DialogSystemPreferences;
00010 }
00011
00012 class DialogSystemPreferences : public QDialog
00013 {
00014     Q_OBJECT
00015
00016 public:
00017     explicit DialogSystemPreferences(Simulator* simulator = nullptr, QWidget *parent = nullptr);
00018     ~DialogSystemPreferences();
00019
00020 private:
00021     void loadPreferencesIntoUi();
00022     void savePreferencesFromUi();
00023     void populateTraceLevels();
00024     void browseSpecificModelFile();
00025
00026     Ui::DialogSystemPreferences *ui;
00027     Simulator* _simulator = nullptr;
00028 };
00029
00030 #endif // DIALOGSYSTEMPREFERENCES_H

```

11.45 DialogTimerConfigure.h

```

00001 #ifndef DIALOGTIMERCONFIGURE_H
00002 #define DIALOGTIMERCONFIGURE_H

```

```

00003
00004 #include <QDialog>
00005 #include <QPushButton>
00006
00007 #include "../animations/AnimationTimer.h"
00008
00009 namespace Ui {
00010     class DialogTimerConfigure;
00011 }
00012
00013 class DialogTimerConfigure : public QDialog
00014 {
00015     Q_OBJECT
00016
00017 public:
00018     explicit DialogTimerConfigure(QWidget *parent = nullptr);
00019     ~DialogTimerConfigure();
00020
00021     // recebe o Timer
00022     void setAnimationTimer(AnimationTimer *animationTimer);
00023     // atualiza os valores no Timer
00024     void updateAnimationTimer();
00025
00026 private slots:
00027     // slot para tratar o clique no botão "OK"
00028     void on_buttonBox_accepted();
00029
00030     // slot para tratar mudança no formato de hora para 12 horas
00031     void on_radioButtonTwelve_clicked(bool checked);
00032
00033     // slot para tratar mudança no formato de hora para 24 horas
00034     void on_radioButtonTwentyFour_clicked(bool checked);
00035
00036 private:
00037     Ui::DialogTimerConfigure *ui;
00038     AnimationTimer *_animationTimer;
00039 };
00040
00041 #endif // DIALOGTIMERCONFIGURE_H

```

11.46 GuiExtensionConfigurationDialog.h

```

00001 #ifndef GUIEXTENSIONCONFIGURATIONDIALOG_H
00002 #define GUIEXTENSIONCONFIGURATIONDIALOG_H
00003
00004 #include "GuiExtensionContracts.h"
00005
00006 #include <QDialog>
00007 #include <QJsonObject>
00008
00009 #include <functional>
00010 #include <vector>
00011
00012 class QTableWidget;
00013 class QPushButton;
00014
00015 class GuiExtensionConfigurationDialog : public QDialog {
00016 public:
00017     GuiExtensionConfigurationDialog(
00018         QWidget* parent,
00019         std::vector<const GuiExtensionPlugin*> plugins,
00020         std::function<void()> onSave = {});
00021
00022 private:
00023     void reloadFromDisk();
00024     void saveToDisk();
00025     QString _configPath() const;
00026     QJsonObject _loadConfigRoot() const;
00027     bool _writeConfigRoot(const QJsonObject& root) const;
00028     void _populateFromConfig(const QJsonObject& root);
00029     void _configureTable();
00030     static std::string _normalizeId(const std::string& text);
00031
00032 private:
00033     std::vector<const GuiExtensionPlugin*> _plugins;
00034     std::function<void()> _onSaved;
00035     QTableWidget* _table = nullptr;
00036     QPushButton* _reloadButton = nullptr;
00037     QPushButton* _saveButton = nullptr;
00038     QPushButton* _closeButton = nullptr;
00039 };
00040
00041 #endif // GUIEXTENSIONCONFIGURATIONDIALOG_H

```

11.47 GuiExtensionContracts.h

```

00001 #ifndef GUIEXTENSIONCONTRACTS_H
00002 #define GUIEXTENSIONCONTRACTS_H
00003
00004 #include <functional>
00005 #include <string>
00006 #include <vector>
00007
00008 class Simulator;
00009 class QMainWindow;
00010 class QWidget;
00011 class ModelGraphicsView;
00012 class ModelGraphicsScene;
00013
00014 namespace Ui {
00015 class MainWindow;
00016 }
00017
00018 struct GuiExtensionRuntimeContext {
00019     Simulator* simulator = nullptr;
00020     QMainWindow* mainWindow = nullptr;
00021     Ui::MainWindow* ui = nullptr;
00022     ModelGraphicsView* graphicsView = nullptr;
00023     ModelGraphicsScene* graphicsScene = nullptr;
00024 };
00025
00026 struct GuiActionContribution {
00027     std::string id;
00028     std::string text;
00029     std::string menuPath;
00030     std::string toolBarId;
00031     std::string statusTip;
00032     std::string shortcut;
00033     std::function<bool(const GuiExtensionRuntimeContext&)> isVisible;
00034     std::function<bool(const GuiExtensionRuntimeContext&)> isEnabled;
00035     std::function<void(const GuiExtensionRuntimeContext&)> trigger;
00036 };
00037
00038 struct GuiDockContribution {
00039     std::string id;
00040     std::string title;
00041     std::string areaHint;
00042     std::function<bool(const GuiExtensionRuntimeContext&)> isVisible;
00043     std::function<QWidget*(const GuiExtensionRuntimeContext&)> createWidget;
00044 };
00045
00046 struct GuiWindowContribution {
00047     std::string id;
00048     std::string menuPath;
00049     std::string text;
00050     std::function<bool(const GuiExtensionRuntimeContext&)> isVisible;
00051     std::function<void(const GuiExtensionRuntimeContext&)> open;
00052 };
00053
00054 class GuiExtensionRegistry {
00055 public:
00056     void addAction(GuiActionContribution contribution) {
00057         _actions.push_back(std::move(contribution));
00058     }
00059     void addDock(GuiDockContribution contribution) {
00060         _docks.push_back(std::move(contribution));
00061     }
00062     void addWindow(GuiWindowContribution contribution) {
00063         _windows.push_back(std::move(contribution));
00064     }
00065     const std::vector<GuiActionContribution>& actions() const {
00066         return _actions;
00067     }
00068     const std::vector<GuiDockContribution>& docks() const {
00069         return _docks;
00070     }
00071     const std::vector<GuiWindowContribution>& windows() const {
00072         return _windows;
00073     }
00074     void clear() {
00075         _actions.clear();
00076         _docks.clear();
00077         _windows.clear();
00078     }
00079
00080 };
00081
00082
00083
00084
00085

```

```

00086 private:
00087     std::vector<GuiActionContribution> _actions;
00088     std::vector<GuiDockContribution> _docks;
00089     std::vector<GuiWindowContribution> _windows;
00090 };
00091
00092 class GuiExtensionPlugin {
00093 public:
00094     virtual ~GuiExtensionPlugin() = default;
00095
00096     virtual std::string extensionId() const = 0;
00097     virtual std::string displayName() const = 0;
00098     virtual std::string version() const = 0;
00099     virtual std::vector<std::string> requiredModelPlugins() const = 0;
00100     virtual void registerContributions(GuiExtensionRegistry* registry) const = 0;
00101 };
00102
00103 #endif /* GUIEXTENSIONCONTRACTS_H */

```

11.48 GuiExtensionManager.h

```

00001 #ifndef GUIEXTENSIONMANAGER_H
00002 #define GUIEXTENSIONMANAGER_H
00003
00004 #include "GuiExtensionContracts.h"
00005
00006 #include <string>
00007 #include <vector>
00008
00009 class QAction;
00010 class QDockWidget;
00011 class QMainWindow;
00012 class QMenu;
00013 class QToolBar;
00014
00015 class GuiExtensionManager {
00016 public:
00017     explicit GuiExtensionManager(QMainWindow* mainWindow);
00018     ~GuiExtensionManager();
00019
00020     void setPlugins(std::vector<const GuiExtensionPlugin*> plugins);
00021     void setLoadedModelPluginIds(std::vector<std::string> modelPluginIds);
00022     void rebuild(const GuiExtensionRuntimeContext& context);
00023     void clear();
00024
00025 private:
00026     QMenu* _resolveMenuPath(const std::string& menuPath);
00027     QToolBar* _resolveToolBar(const std::string& toolBarId) const;
00028     void _applyActionContribution(const GuiActionContribution& contribution, const
GuiExtensionRuntimeContext& context);
00029     void _applyWindowContribution(const GuiWindowContribution& contribution, const
GuiExtensionRuntimeContext& context);
00030     void _applyDockContribution(const GuiDockContribution& contribution, const
GuiExtensionRuntimeContext& context);
00031     bool _isPluginDependenciesSatisfied(const GuiExtensionPlugin* plugin) const;
00032
00033 private:
00034     QMainWindow* _mainWindow = nullptr;
00035     std::vector<const GuiExtensionPlugin*> _plugins;
00036     std::vector<std::string> _loadedModelPluginIds;
00037     std::vector<QAction*> _createdActions;
00038     std::vector<QAction*> _createdMenuActions;
00039     std::vector<QDockWidget*> _createdDocks;
00040 };
00041
00042 #endif /* GUIEXTENSIONMANAGER_H */

```

11.49 GuiExtensionPluginCatalog.h

```

00001 #ifndef GUIEXTENSIONPLUGINCATALOG_H
00002 #define GUIEXTENSIONPLUGINCATALOG_H
00003
00004 #include "GuiExtensionContracts.h"
00005
00006 #include <string>
00007 #include <vector>
00008
00009 struct GuiExtensionPresentationMetadata {
00010     std::string category;

```

```

00011     std::string group;
00012     int priority = 0;
00013 };
00014
00015 class GuiExtensionPluginCatalog {
00016 public:
00017     static const std::vector<const GuiExtensionPlugin*> plugins();
00018     static std::vector<const GuiExtensionPlugin*> resolvedPlugins();
00019     static GuiExtensionPresentationMetadata presentationMetadata(const std::string& extensionId);
00020     static void registerPlugin(const GuiExtensionPlugin* plugin);
00021 };
00022
00023 class GuiExtensionPluginAutoRegistrar {
00024 public:
00025     explicit GuiExtensionPluginAutoRegistrar(const GuiExtensionPlugin* plugin);
00026 };
00027
00028 #define REGISTER_GUI_EXTENSION_PLUGIN(PluginType)          \
00029     namespace {                                           \
00030         static const PluginType _instance_##PluginType; \
00031         static const GuiExtensionPluginAutoRegistrar     \
00032             _registrar_##PluginType(&_instance_##PluginType); \
00033     }
00034
00035 #endif /* GUIEXTENSIONPLUGINCATALOG_H */

```

11.50 GraphicalAnimateExpression.h

```

00001 #ifndef GRAPHICALANIMATEEXPRESSION_H
00002 #define GRAPHICALANIMATEEXPRESSION_H
00003
00004 #include <QGraphicsObject>
00005
00006 class GraphicalAnimateExpression : public QGraphicsObject {
00007 public:
00008     GraphicalAnimateExpression();
00009 };
00010
00011 #endif // GRAPHICALANIMATEEXPRESSION_H

```

11.51 GraphicalAssociation.h

```

00001 #ifndef GRAPHICALASSOCIATION_H
00002 #define GRAPHICALASSOCIATION_H
00003
00004
00005 class GraphicalAssociation
00006 {
00007 public:
00008     GraphicalAssociation();
00009 };
00010
00011 #endif // GRAPHICALASSOCIATION_H

```

11.52 GraphicalComponentPort.h

```

00001 #ifndef GRAPHICALCOMPONENTPORT_H
00002 #define GRAPHICALCOMPONENTPORT_H
00003
00004 #include <QGraphicsItem>
00005 #include <QGraphicsObject>
00006 #include <QPen>
00007 #include <QBrush>
00008 #include "../kernel/simulator/model/ModelComponent.h"
00009 #include "TraitsGUI.h"
00010 // #include "GraphicalConnection.h"
00011
00012 class GraphicalModelComponent;
00013 class GraphicalConnection;
00014
00015 class GraphicalComponentPort : public QGraphicsObject {
00016 public:
00024     GraphicalComponentPort(GraphicalModelComponent* componentGraph, bool isInputPort, unsigned int
        portNum = 0, QGraphicsItem *parent = nullptr);
00025

```

```

00026 protected: // virtual functions
00027     //virtual void mousePressEvent(QGraphicsSceneMouseEvent *mouseEvent);
00028     //virtual void mouseReleaseEvent(QGraphicsSceneMouseEvent *mouseEvent);
00029     //virtual void mouseMoveEvent(QGraphicsSceneMouseEvent *mouseEvent);
00031     virtual QVariant itemChange(QGraphicsItem::GraphicsItemChange change, const QVariant &value)
        override;
00032 public: //overrides
00034     QRectF boundingRect() const override;
00036     void paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override;
00038     qreal width() const;
00040     qreal height() const;
00041 public:
00043     void addGraphicalConnection(GraphicalConnection* connection);
00045     void removeGraphicalConnection(GraphicalConnection* connection);
00046
00047 public: // sets and gets
00049     unsigned int portNum() const;
00051     bool isInputPort() const;
00053     GraphicalModelComponent *graphicalComponent() const;
00055     QList<GraphicalConnection*>*getConnections() const;
00056 private:
00057     QColor myrgba(uint64_t color); // TODO: Should NOT be here, but in UtilGUI.h, but then it
        generates multiple definitions error
00058 private:
00059     qreal _width = TraitsGUI<GComponentPort>::width;//20;
00060     qreal _height = _width;
00061     qreal _margin = TraitsGUI<GComponentPort>::margin; //2
00062     int _raise = TraitsGUI<GComponentPort>::raise; //3;
00063     bool _isInputPort;
00064     unsigned int _portNum;
00065     GraphicalModelComponent* _componentGraph;
00066     QList<GraphicalConnection*>* _connections = new QList<GraphicalConnection*>();
00067 };
00068
00069 #endif // GRAPHICALCOMPONENTPORT_H

```

11.53 GraphicalConnection.h

```

00001 #ifndef GRAPHICALCONNECTION_H
00002 #define GRAPHICALCONNECTION_H
00003
00004 #include <QGraphicsObject>
00005
00006 #include <QGraphicsItem>
00007 #include <QGraphicsObject>
00008 #include <QPointer>
00009 #include <QPainterPath>
00010 #include <QPen>
00011 #include <QBrush>
00012 #include "GraphicalConnectionStyle.h"
00013 #include "GraphicalComponentPort.h"
00014 // #include "kernel/simulator/ModelComponent.h"
00015 // #include "kernel/simulator/Plugin.h"
00016 #include "kernel/simulator/ConnectionManager.h"
00017
00029 class GraphicalConnection : public QGraphicsObject {
00030 public:
00031     enum class ConnectionType : int {
00032         HORIZONTAL = 1, VERTICAL = 2, DIRECT = 3, USERDEFINED = 4
00033     };
00034
00035 public:
00037     GraphicalConnection(GraphicalComponentPort* sourceGraphicalPort, GraphicalComponentPort*
        destinationGraphicalPort, unsigned int portSourceConnection = 0, unsigned int
        portDestinationConnection = 0, QColor color = Qt::black, QGraphicsItem *parent = nullptr);
00039     GraphicalConnection(const GraphicalConnection& orig);
00041     virtual ~GraphicalConnection();
00042 public:
00044     QRectF boundingRect() const override;
00046     QPainterPath shape() const override;
00048     void paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override;
00050     Connection* getSource() const;
00052     Connection* getDestination() const;
00054     GraphicalComponentPort* getSourceGraphicalPort();
00056     GraphicalComponentPort* getDestinationGraphicalPort();
00058     GraphicalModelComponent* getSourceGraphicalComponent() const;
00060     GraphicalModelComponent* getDestinationGraphicalComponent() const;
00062     void updateDimensionsAndPosition();
00064     GraphicalConnection::ConnectionType connectionType() const;
00066     unsigned int getPortSourceConnection() const;
00068     unsigned int getPortDestinationConnection() const;
00070     void setConnectionType(GraphicalConnection::ConnectionType newConnectionType);
00072     QList<QPointF> getPoints() const;

```



```

00074     bool usesCurvedStyle() const;
00076     QPainterPath animationPathForImage(qreal imageWidth, qreal imageHeight) const;
00077
00078 protected: // virtual
00079     virtual bool sceneEvent(QEvent *event) override;
00080     //virtual void hoverEnterEvent(QGraphicsSceneHoverEvent * event)
00081     //virtual void hoverLeaveEvent(QGraphicsSceneHoverEvent * event)
00082     //virtual void hoverMoveEvent(QGraphicsSceneHoverEvent * event)
00083     //virtual void inputMethodEvent(QInputMethodEvent * event)
00084     //virtual QVariant inputMethodQuery(Qt::InputMethodQuery query) const
00085     //virtual QVariant itemChange(GraphicsItemChange change, const QVariant & value)
00086     //virtual void keyPressEvent(QKeyEvent * event)
00087     //virtual void keyReleaseEvent(QKeyEvent * event)
00088     //virtual void mouseDoubleClickEvent(QGraphicsSceneMouseEvent * event);
00089     //virtual void mouseMoveEvent(QGraphicsSceneMouseEvent * event);
00090     //virtual void mousePressEvent(QGraphicsSceneMouseEvent * event);
00091     //virtual void mouseReleaseEvent(QGraphicsSceneMouseEvent * event)
00092 private:
00093     bool canRefreshGeometry() const;
00094     QPainterPath connectionPath() const;
00095     GraphicalConnectionStyle::RouteType routeTypeForStyle() const;
00096     qreal _width = 0.0;
00097     qreal _height = 0.0;
00098     QPointF _sourcePointLocal;
00099     QPointF _destinationPointLocal;
00100     unsigned int _margin = TraitsGUI<GConnection>::margin;//2;
00101     unsigned int _selWidth = TraitsGUI<GConnection>::selectionWidth;//8;
00102     ConnectionType _connectionType = ConnectionType::HORIZONTAL;
00103     Connection* _sourceConnection;
00104     Connection* _destinationConnection;
00105     QPointer<GraphicalComponentPort> _sourceGraphicalPort;
00106     QPointer<GraphicalComponentPort> _destinationGraphicalPort;
00107     unsigned int _portSourceConnection;
00108     unsigned int _portDestinationConnection;
00109     QColor _color;
00110     QList<QPointF> _points;
00111 };
00112 #endif // GRAPHICALCONNECTIONSTYLE_H

```

11.54 GraphicalConnectionStyle.h

```

00001 #ifndef GRAPHICALCONNECTIONSTYLE_H
00002 #define GRAPHICALCONNECTIONSTYLE_H
00003
00004 #include <QPainterPath>
00005 #include <QPointF>
00006
00007 class GraphicalConnectionStyle {
00008 public:
00009     enum class RouteType {
00010         Horizontal,
00011         Vertical,
00012         Direct,
00013         UserDefined
00014     };
00015
00016     static bool usesModernCurves();
00017     static QPainterPath modelConnectionPath(const QPointF& source,
00018                                             const QPointF& destination,
00019                                             RouteType routeType,
00020                                             bool modernCurves);
00021     static QPainterPath diagramConnectionPath(const QPointF& source,
00022                                              const QPointF& destination,
00023                                              bool modernCurves);
00024     static QPointF pointAtProgress(const QPainterPath& path, qreal progress);
00025
00026 private:
00027     GraphicalConnectionStyle() = default;
00028 };
00029
00030 #endif // GRAPHICALCONNECTIONSTYLE_H

```

11.55 GraphicalDiagramConnection.h

```

00001 #ifndef GRAPHICALDIAGRAMCONNECTION_H
00002 #define GRAPHICALDIAGRAMCONNECTION_H
00003
00004 #include <QGraphicsItem>
00005 #include <QGraphicsObject>

```

```

00006 #include <QPen>
00007 #include <QBrush>
00008 #include <QPainter>
00009 #include <QPainterPath>
00010 #include <cmath>
00011 #include <QGraphicsLineItem>
00012 #include "GraphicalConnectionStyle.h"
00013 #include "../kernel/simulator/model/ModelComponent.h"
00014 #include "../kernel/simulator/model/ModelDataDefinition.h"
00015 #include "GraphicalModelComponent.h"
00016 #include "GraphicalModelDataDefinition.h"
00017 #include "TraitsGUI.h"
00018 #include "UtilGUI.h"
00019
00020 class GraphicalDiagramConnection : public QGraphicsLineItem {
00021 public:
00022     enum class ConnectionType{
00023         ATTACHED, INTERNAL
00024     };
00025
00026 public:
00027     GraphicalDiagramConnection(QGraphicsItem* dataDefinition, QGraphicsItem* linkedTo, ConnectionType
type);
00028     void refreshGeometry();
00029
00030     QRectF boundingRect() const override;
00031     QPainterPath shape() const override;
00032     void paint(QPainter* painter, const QStyleOptionGraphicsItem* option, QWidget* widget = nullptr)
override;
00033
00034     QGraphicsItem* getDataDefinition();
00035     QGraphicsItem* getLinkedDataDefinition();
00036     ConnectionType getConnectionType();
00037
00038 private:
00039     ConnectionType _type;
00040     QGraphicsItem* _item1;
00041     QGraphicsItem* _item2;
00042     QPainterPath connectionPath() const;
00043 };
00044
00045
00046 #endif // GRAPHICALDIAGRAMCONNECTION_H

```

11.56 GraphicalImageAnimation.h

```

00001 #ifndef GRAPHICALIMAGEANIMATION_H
00002 #define GRAPHICALIMAGEANIMATION_H
00003
00004 #include <QGraphicsPixmapItem>
00005 #include <QPixmap>
00006
00007 class GraphicalImageAnimation : public QGraphicsPixmapItem {
00008 public:
00009     // Contrutor
00010     GraphicalImageAnimation(const QPointF startPoint, unsigned int width, unsigned int height, const
QString imageName);
00011     // Destrutor
00012     ~GraphicalImageAnimation();
00013
00014     // Getters
00015     QString getImagePath() const;
00016     QString getImageName() const;
00017     QPointF getStartPoint() const;
00018     unsigned int getWidth() const;
00019     unsigned int getHeight() const;
00020
00021     // Setters
00022     void setImageName(const QString &imageName);
00023     void setStartPoint(const QPointF &startPoint);
00024     void setWidth(unsigned int imageWidth);
00025     void setHeight(unsigned int imageHeight);
00026
00027     // Atualiza informações da imagem
00028     void updateImage();
00029
00030 private:
00031     // Builds an in-memory marker image when the file-based image cannot be loaded.
00032     QPixmap buildFallbackPixmap() const;
00033
00034     // Use a Qt resource base path to avoid working-directory dependent file paths.
00035     QString _resourceBasePath = ":/animations/";
00036

```

```

00037     // Atributos
00038     QString _imagePath;
00039     QPointF _startPoint;
00040     unsigned int _width;
00041     unsigned int _height;
00042     QString _imageName;
00043 };
00044
00045 #endif // GRAPHICALIMAGEANIMATION_H

```

11.57 GraphicalModelComponent.h

```

00001 #ifndef GRAPHICALMODELCOMPONENT_H
00002 #define GRAPHICALMODELCOMPONENT_H
00003
00004 /*
00005  * File:    ModelComponentGraphicItem.h
00006  * Author:  rlcancian
00007  *
00008  * Created on 16 de fevereiro de 2022, 11:41
00009  */
00010
00011 #include <QGraphicsItem>
00012 #include <QGraphicsObject>
00013 #include <QPen>
00014 #include <QBrush>
00015 #include "../kernel/simulator/model/ModelComponent.h"
00016 #include "../kernel/simulator/model/ModelDataDefinition.h"
00017 #include "../kernel/simulator/essentialPlugins/Attribute.h"
00018 #include "../kernel/simulator/essentialPlugins/Counter.h"
00019 #include "../kernel/simulator/PropertyGenesys.h"
00020 #include "../kernel/simulator/Simulator.h"
00021 #include "../plugins/data/Logic/Variable.h"
00022 #include "kernel/simulator/Plugin.h"
00023 #include "plugins/data/DiscreteProcessing/Queue.h"
00024 #include "graphicals/GraphicalImageAnimation.h"
00025 #include "GraphicalComponentPort.h"
00026 #include "GraphicalModelDataDefinition.h"
00027 #include "TraitsGUI.h"
00028
00035 class GraphicalModelComponent : public GraphicalModelDataDefinition {
00036 public:
00038     GraphicalModelComponent(Plugin* plugin, ModelComponent* component, QPointF position, QColor color
00039 = Qt::blue, QGraphicsItem *parent = nullptr);
00040     GraphicalModelComponent(const GraphicalModelComponent& orig);
00042     virtual ~GraphicalModelComponent();
00043 public:
00045     QRectF boundingRect() const override;
00047     void paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override;
00049     ModelComponent* getComponent() const;
00051     qreal getHeight() const;
00053     QPointF getOldPosition() const;
00055     void setOldPosition(QPointF oldPosition);
00057     QList<GraphicalComponentPort *> getGraphicalInputPorts() const;
00059     QList<GraphicalComponentPort *> getGraphicalOutputPorts() const;
00061     QColor getColor() const;
00063     void setColor(const QColor& color);
00065     QList<ModelDataDefinition *> *getInternalData() const;
00067     QList<ModelDataDefinition *> *getAttachedData() const;
00069     EntityType* getEntityType() const;
00071     void setEntityType(EntityType *entityType);
00073     unsigned int getOccupiedInputPorts() const;
00075     unsigned int getOccupiedOutputPorts() const;
00077     void setOccupiedInputPorts(unsigned int value);
00079     void setOccupiedOutputPorts(unsigned int value);
00080 private:
00081     QColor myrgba(uint64_t color); // TODO: Should NOT be here, but in UtilGUI.h, but then it
00082     generates multiple definitions error
00083 protected: // virtual
00084     virtual bool sceneEvent(QEvent *event) override;
00085     QVariant itemChange(GraphicsItemChange change, const QVariant& value) override;
00086     GraphicalModelItemRenderContext renderContext() const override;
00087     //virtual void hoverEnterEvent(QGraphicsSceneHoverEvent * event)
00088     //virtual void hoverLeaveEvent(QGraphicsSceneHoverEvent * event)
00089     //virtual void hoverMoveEvent(QGraphicsSceneHoverEvent * event)
00090     //virtual void inputMethodEvent(QInputMethodEvent * event)
00091     //virtual QVariant itemChange(GraphicsItemChange change, const QVariant & value)
00092     //virtual void keyPressEvent(QKeyEvent * event)
00093     //virtual void keyReleaseEvent(QKeyEvent * event)
00094     // virtual void mouseDoubleClickEvent(QGraphicsSceneMouseEvent * event);
00095     // virtual void mouseMoveEvent(QGraphicsSceneMouseEvent * event);
00096     // virtual void mousePressEvent(QGraphicsSceneMouseEvent * event);

```

```

00097     //virtual void mouseReleaseEvent(QGraphicsSceneMouseEvent * event)
00098 protected:
00099     qreal _width = TraitsGUI<GModelComponent>::width; //150;
00100     qreal _height = _width * TraitsGUI<GModelComponent>::heightProportion; //0.67;
00101     unsigned int _margin = TraitsGUI<GModelComponent>::margin; //8;
00102     unsigned int _selWidth = TraitsGUI<GModelComponent>::selectionWidth; //8;
00103     ModelComponent* _component;
00104     QColor _color;
00105     QPointF _oldPosition;
00106     qreal _stretchPosTop = TraitsGUI<GModelComponent>::stretchPos; //0.5;
00107     qreal _stretchPosBottom = TraitsGUI<GModelComponent>::stretchPos; //0.5;
00108     qreal _stretchPosLeft = TraitsGUI<GModelComponent>::stretchPos; //0.5;
00109     qreal _stretchPosRigth = TraitsGUI<GModelComponent>::stretchPos; //0.5;
00110     qreal _stretchRigth = TraitsGUI<GModelComponent>::stretch; //0;
00111     qreal _stretchLeft = TraitsGUI<GModelComponent>::stretch; //0;
00112     qreal _stretchRigthMidle = TraitsGUI<GModelComponent>::stretch; //0;
00113     qreal _stretchLeftMidle = TraitsGUI<GModelComponent>::stretch; //0;
00114     qreal _stretchTop = TraitsGUI<GModelComponent>::stretch; //0;
00115     qreal _stretchBottom = TraitsGUI<GModelComponent>::stretch; //0;
00116     qreal _stretchTopMidle = TraitsGUI<GModelComponent>::stretch; //0;
00117     qreal _stretchBottomMidle = TraitsGUI<GModelComponent>::stretch; //0;
00118 private:
00119     QList<GraphicalComponentPort*> _graphicalInputPorts = QList<GraphicalComponentPort*>();
00120     QList<GraphicalComponentPort*> _graphicalOutputPorts = QList<GraphicalComponentPort*>();
00121     QList<ModelDataDefinition *> *_internalData = new QList<ModelDataDefinition*>();
00122     QList<ModelDataDefinition *> *_attachedData = new QList<ModelDataDefinition*>();
00123     EntityType* _entityType = nullptr;
00124     unsigned int _ocupiedInputPorts = 0;
00125     unsigned int _ocupiedOutputPorts = 0;
00126
00127 private:
00128     QString _animationImageName = "default.png";
00129     // Map que mapeia o ponteiro da fila do componente para seu índice (caso tiver mais de uma) e
    tamanho (usado para animação)
00130     QMap<Queue *, QPair<unsigned int, unsigned int>*> *_mapQueue = new QMap<Queue *, QPair<unsigned int,
    unsigned int>*>();
00131     // Irá conter referência para as imagens usadas para mostrar a fila do componente
00132     QList<QList<GraphicalImageAnimation *>*> *_imagesQueue = new QList<QList<GraphicalImageAnimation
    *>*>();
00133     bool _hasQueue = false;
00134
00135 public:
00137     QString getAnimationImageName();
00139     void setAnimationImageName(QString name);
00141     QMap<Queue *, QPair<unsigned int, unsigned int>*> getMapQueue();
00143     QList<QList<GraphicalImageAnimation *>*> getImagesQueue();
00144
00146     void verifyQueue();
00148     bool hasQueue();
00150     static bool compareQueuesById(const Queue* a, const Queue* b);
00152     unsigned int getIndexQueue(Queue *queue);
00154     unsigned int getSizeQueue(Queue *queue);
00156     void populateMapQueue(QList<Queue *> queues);
00158     bool insertImageQueue(Queue *queue, GraphicalImageAnimation *image);
00160     QList<GraphicalImageAnimation *> *removeImageQueue(Queue *queue, unsigned int quantityRemoved);
00162     bool insertImageQueue(GraphicalImageAnimation *image);
00164     GraphicalImageAnimation *removeImageQueue();
00166     void atualizeMapQueue(Queue *queue);
00168     void visivibleImageQueue(bool visivible);
00170     void clearQueues();
00171 };
00172
00173 #endif /* MODELCOMPONENTGRAPHICITEM_H */

```

11.58 GraphicalModelDataDefinition.h

```

00001 #ifndef GRAPHICALMODELDATADEFINITION_H
00002 #define GRAPHICALMODELDATADEFINITION_H
00003
00004 /*
00005  * File:   ModelDataDefinitionGraphicItem.h
00006  * Author: rlcancian
00007  *
00008  * Created on 16 de fevereiro de 2022, 11:41
00009  */
00010
00011 #include <QGraphicsItem>
00012 #include <QGraphicsObject>
00013 #include <QPainterPath>
00014 #include <QPen>
00015 #include <QBrush>
00016 #include "kernel/simulator/Plugin.h"
00017 #include "graphics/GraphicalModelItemRenderStrategy.h"

```

```

00018 #include "TraitsGUI.h"
00019
00026 class GraphicalModelDataDefinition : public QGraphicsObject {
00027 public:
00029     GraphicalModelDataDefinition(Plugin* plugin, ModelDataDefinition* element, QPointF position,
    QColor color = Qt::blue, QGraphicsItem *parent = nullptr);
00031     GraphicalModelDataDefinition(const GraphicalModelDataDefinition& orig);
00033     virtual ~GraphicalModelDataDefinition();
00034 public:
00036     QRectF boundingRect() const override;
00038     QPainterPath shape() const override;
00040     void paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget) override;
00042     ModelDataDefinition* getDataDefinition() const;
00044     bool isEditableInPropertyEditor() const;
00046     void setEditableInPropertyEditor(bool editable);
00048     QPointF getOldPosition() const;
00050     void setOldPosition(qreal x, qreal y);
00052     QColor getColor() const;
00054     void setColor(QColor newColor);
00056     qreal getHeight() const;
00057
00058 protected:
00059     QColor myrgba(uint64_t color); // TODO: Should NOT be here, but in UtilGUI.h, but then it
    generates multiple definitions error
00060     virtual GraphicalModelItemRenderContext renderContext() const;
00061 protected: // virtual
00062     virtual bool sceneEvent(QEvent *event) override;
00063     //virtual void hoverEnterEvent(QGraphicsSceneHoverEvent * event)
00064     //virtual void hoverLeaveEvent(QGraphicsSceneHoverEvent * event)
00065     //virtual void hoverMoveEvent(QGraphicsSceneHoverEvent * event)
00066     //virtual void inputMethodEvent(QInputMethodEvent * event)
00067     //virtual QVariant inputMethodQuery(Qt::InputMethodQuery query) const
00068     //virtual QVariant itemChange(GraphicsItemChange change, const QVariant & value)
00069     //virtual void keyPressEvent(QKeyEvent * event)
00070     //virtual void keyReleaseEvent(QKeyEvent * event)
00071     //virtual void mouseDoubleClickEvent(QGraphicsSceneMouseEvent * event);
00072     //virtual void mouseMoveEvent(QGraphicsSceneMouseEvent * event);
00073     //virtual void mousePressEvent(QGraphicsSceneMouseEvent * event);
00074     //virtual void mouseReleaseEvent(QGraphicsSceneMouseEvent * event)
00075 protected:
00076     qreal _width = TraitsGUI<GModelDataDefinition>::width; //150;
00077     qreal _height = _width * TraitsGUI<GModelDataDefinition>::heightProportion; //0.67;
00078     unsigned int _margin = TraitsGUI<GModelDataDefinition>::margin; //8;
00079     unsigned int _selWidth = TraitsGUI<GModelDataDefinition>::selectionWidth; //8;
00080     ModelDataDefinition* _element;
00081     bool _editableInPropertyEditor = false;
00082     QColor _color;
00083     QPointF _oldPosition;
00084     qreal _stretchPosTop = TraitsGUI<GModelDataDefinition>::stretchPos; //0.5;
00085     qreal _stretchPosBottom = TraitsGUI<GModelDataDefinition>::stretchPos; //0.5;
00086     qreal _stretchPosLeft = TraitsGUI<GModelDataDefinition>::stretchPos; //0.5;
00087     qreal _stretchPosRigth = TraitsGUI<GModelDataDefinition>::stretchPos; //0.5;
00088     qreal _stretchRigth = TraitsGUI<GModelDataDefinition>::stretch; //0;
00089     qreal _stretchLeft = TraitsGUI<GModelDataDefinition>::stretch; //0;
00090     qreal _stretchRigthMiddle = TraitsGUI<GModelDataDefinition>::stretch; //0;
00091     qreal _stretchLeftMiddle = TraitsGUI<GModelDataDefinition>::stretch; //0;
00092     qreal _stretchTop = TraitsGUI<GModelDataDefinition>::stretch; //0;
00093     qreal _stretchBottom = TraitsGUI<GModelDataDefinition>::stretch; //0;
00094     qreal _stretchTopMiddle = TraitsGUI<GModelDataDefinition>::stretch; //0;
00095     qreal _stretchBottomMiddle = TraitsGUI<GModelDataDefinition>::stretch; //0;
00096 private:
00097 };
00098
00099 #endif /* GRAPHICALMODELDATADEFINITION_H */

```

11.59 GraphicalModelItemRenderStrategy.h

```

00001 #ifndef GRAPHICALMODELITEMRENDERSTRATEGY_H
00002 #define GRAPHICALMODELITEMRENDERSTRATEGY_H
00003
00004 #include <stdint>
00005
00006 #include <QColor>
00007 #include <QPainterPath>
00008 #include <QRectF>
00009 #include <QString>
00010
00011 class QPainter;
00012
00013 struct GraphicalModelItemRenderContext {
00014     enum class ItemKind {
00015         DataDefinition,
00016         Component
    };

```

```

00017     };
00018
00019     ItemKind kind = ItemKind::DataDefinition;
00020     QRectF bounds;
00021     QColor fillColor;
00022     QString primaryText;
00023     QString secondaryText;
00024     QString tertiaryText;
00025     QString semanticClassName;
00026     QString semanticSubtype;
00027     bool selected = false;
00028     bool breakpoint = false;
00029
00030     qreal width = 0.0;
00031     qreal height = 0.0;
00032     int penWidth = 1;
00033     int raise = 0;
00034     int margin = 0;
00035     int selectionWidth = 0;
00036
00037     qreal stretchPosTop = 0.5;
00038     qreal stretchPosBottom = 0.5;
00039     qreal stretchPosLeft = 0.5;
00040     qreal stretchPosRight = 0.5;
00041     qreal stretchRight = 0.0;
00042     qreal stretchLeft = 0.0;
00043     qreal stretchRightMiddle = 0.0;
00044     qreal stretchLeftMiddle = 0.0;
00045     qreal stretchTop = 0.0;
00046     qreal stretchBottom = 0.0;
00047     qreal stretchTopMiddle = 0.0;
00048     qreal stretchBottomMiddle = 0.0;
00049
00050     uint64_t borderColor = 0;
00051     uint64_t raisedColor = 0;
00052     uint64_t sunkenColor = 0;
00053     uint64_t textColor = 0;
00054     uint64_t textShadowColor = 0;
00055     uint64_t selectionColor = 0;
00056     uint64_t breakpointColor = 0;
00057 };
00058
00059 class GraphicalModelItemRenderStrategy {
00060 public:
00061     virtual ~GraphicalModelItemRenderStrategy() = default;
00062
00063     virtual const char* name() const = 0;
00064     virtual void paint(QPainter* painter, const GraphicalModelItemRenderContext& context) const = 0;
00065     virtual QPainterPath shape(const GraphicalModelItemRenderContext& context) const = 0;
00066 };
00067
00068 class GraphicalModelItemRenderer {
00069 public:
00070     static void paint(QPainter* painter, const GraphicalModelItemRenderContext& context);
00071     static QPainterPath shape(const GraphicalModelItemRenderContext& context);
00072     static const GraphicalModelItemRenderStrategy& currentStrategy();
00073 };
00074
00075 #endif // GRAPHICALMODELITEMRENDERSTRATEGY_H

```

11.60 ModelGraphicsScene.h

```

00001 /*
00002  * The MIT License
00003  *
00004  * Copyright 2022 rlcancian.
00005  *
00006  * Permission is hereby granted, free of charge, to any person obtaining a copy
00007  * of this software and associated documentation files (the "Software"), to deal
00008  * in the Software without restriction, including without limitation the rights
00009  * to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00010  * copies of the Software, and to permit persons to whom the Software is
00011  * furnished to do so, subject to the following conditions:
00012  *
00013  * The above copyright notice and this permission notice shall be included in
00014  * all copies or substantial portions of the Software.
00015  *
00016  * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017  * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018  * FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00019  * AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00020  * LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00021  * OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN

```

```

00022  * THE SOFTWARE.
00023  */
00024
00025  /*
00026  * File:   ModelGraphicsScene.h
00027  * Author: rlcancian
00028  *
00029  * Created on 16 de fevereiro de 2022, 09:52
00030  */
00031
00032  #ifndef MODELGRAPHICSSCENE_H
00033  #define MODELGRAPHICSSCENE_H
00034
00035  #include <QGraphicsScene>
00036  #include <QGraphicsProxyWidget>
00037  #include <QGraphicsRectItem>
00038  #include <QTreeWidgetItem>
00039  #include <QUndoStack>
00040  #include <QAction>
00041  #include <QEventLoop>
00042  #include "graphicals/GraphicalModelComponent.h"
00043  #include "graphicals/GraphicalComponentPort.h"
00044  #include "graphicals/GraphicalDiagramConnection.h"
00045  #include "propertyeditor/DataComponentProperty.h"
00046  #include "propertyeditor/DataComponentEditor.h"
00047  #include "propertyeditor/ComboBoxEnum.h"
00048  #include "../kernel/simulator/model/ModelComponent.h"
00049  #include "kernel/simulator/Simulator.h"
00050  #include "kernel/simulator/Plugin.h"
00051  #include "animations/AnimationTransition.h"
00052  #include "animations/AnimationVariable.h"
00053  #include "animations/AnimationCounter.h"
00054  #include "animations/AnimationTimer.h"
00055  #include "animations/AnimationPlaceholder.h"
00056  #include "../kernel/simulator/essentialPlugins/Counter.h"
00057  #include "../kernel/simulator/essentialPlugins/Attribute.h"
00058  #include "kernel/simulator/PropertyGenesys.h"
00059  #include "../kernel/simulator/model/ModelDataDefinition.h"
00060  #include "../plugins/data/Logic/Variable.h"
00064  class GraphicalModelEvent {
00065  public:
00066
00067      enum class EventType : int {
00068          CREATE = 1,
00069          REMOVE = 2,
00070          EDIT = 3,
00071          CLONE = 4,
00072          OTHER = 9
00073      };
00074
00075      enum class EventObjectType : int {
00076          COMPONENT = 1,
00077          DATADEFINITION = 2,
00078          CONNECTION = 3,
00079          DRAWING = 4,
00080          ANIMATION = 5,
00081          PLOT = 6,
00082          OTHER = 9
00083      };
00084
00085  public:
00086
00088      GraphicalModelEvent(GraphicalModelEvent::EventType eventType, GraphicalModelEvent::EventObjectType
eventObjectType, QGraphicsItem* item) {
00089          this->eventType = eventType;
00090          this->eventObjectType = eventObjectType;
00091          this->item = item;
00092      }
00093      GraphicalModelEvent::EventType eventType;
00094      GraphicalModelEvent::EventObjectType eventObjectType;
00095      QGraphicsItem* item;
00096  };
00097
00105  class ModelGraphicsScene : public QGraphicsScene {
00106  public:
00107      ModelGraphicsScene(qreal x, qreal y, qreal width, qreal height, QObject *parent = nullptr);
00108      // Disable copy construction to keep scene ownership state unique.
00109      ModelGraphicsScene(const ModelGraphicsScene& orig) = delete;
00110      // Disable copy assignment to prevent shallow copies of GUI-owned resources.
00111      ModelGraphicsScene& operator=(const ModelGraphicsScene& other) = delete;
00112      virtual ~ModelGraphicsScene();
00113  public: // editing graphic model
00114      enum DrawingMode{
00115          NONE, LINE, TEXT, RECTANGLE, ELLIPSE, POLYGON, POLYGON_POINTS, POLYGON_FINISHED,
00116          COUNTER, VARIABLE, TIMER,
00117          ATTRIBUTE, ENTITY, EVENT, EXPRESSION, PLOT, QUEUE_PLACEHOLDER, RESOURCE, STATION, STATISTICS
00118      };

```



```

00120     GraphicalModelComponent* addGraphicalModelComponent(Plugin* plugin, ModelComponent* component,
    QPointF position, QColor color = Qt::blue, bool notify = false, GraphicalModelComponent*
    autoConnectSource = nullptr);
00122     GraphicalConnection* addGraphicalConnection(GraphicalComponentPort* sourcePort,
    GraphicalComponentPort* destinationPort, unsigned int portSourceConnection, unsigned int
    portDestinationConnection, bool notify = false);
00124     GraphicalModelDataDefinition* addGraphicalModelDataDefinition(Plugin* plugin, ModelDataDefinition*
    element, QPointF position, QColor color = Qt::blue);
00126     GraphicalDiagramConnection* addGraphicalDiagramConnection(QGraphicsItem* dataDefinition,
    QGraphicsItem* linkedTo, GraphicalDiagramConnection::ConnectionType type);
00128     void initializeAnimationDrawing(QGraphicsSceneMouseEvent *mouseEvent);
00130     void continueAnimationDrawing(QGraphicsSceneMouseEvent *mouseEvent);
00132     void finishAnimationDrawing(QGraphicsSceneMouseEvent *mouseEvent);
00134     void startTextEditing();
00136     void removeComponent(GraphicalModelComponent* gmc, bool notify = false);
00138     void clearConnectionsComponent(GraphicalModelComponent* gmc);
00140     void clearInputConnectionsComponent(GraphicalModelComponent* graphicalComponent);
00142     void clearOutputConnectionsComponent(GraphicalModelComponent* graphicalComponent);
00144     void clearPorts(GraphicalConnection* connection, GraphicalModelComponent *source,
    GraphicalModelComponent *destination);
00146     void connectComponents(GraphicalConnection* connection, GraphicalModelComponent *source = nullptr,
    GraphicalModelComponent *destination = nullptr, bool notify = false);
00148     bool connectSource(GraphicalConnection* connection, GraphicalModelComponent *source = nullptr);
00150     bool connectDestination(GraphicalConnection* connection, GraphicalModelComponent *destination =
    nullptr);
00152     void redoConnections(GraphicalModelComponent *graphicalComponent, QList<GraphicalConnection *>
    *inputConnections, QList<GraphicalConnection *> *outputConnections);
00154     void removeComponentInModel(GraphicalModelComponent* gmc);
00156     void insertComponent(GraphicalModelComponent* gmc, QList<GraphicalConnection *> *inputConnections,
    QList<GraphicalConnection *> *outputConnections, bool addGMC = true, bool addAllGMC = true, bool
    notify = false);
00158     void removeGraphicalConnection(GraphicalConnection* graphicalConnection, GraphicalModelComponent
    *source, GraphicalModelComponent *destination, bool notify = false);
00160     void removeConnectionInModel(GraphicalConnection* graphicalConnection, GraphicalModelComponent
    *source);
00162     void addGeometry(QPointF endPoint, bool moving);
00164     void addDrawing(QGraphicsItem * item, bool notify = false);
00166     bool addDrawingGeometry(QGraphicsItem * item);
00168     bool addDrawingAnimation(QGraphicsItem * item);
00170     void removeGraphicalModelDataDefinition(GraphicalModelDataDefinition* gmdd);
00172     void detachGraphicalModelDataDefinition(GraphicalModelDataDefinition* gmdd);
00174     void restoreGraphicalModelDataDefinition(GraphicalModelDataDefinition* gmdd);
00176     void removeGraphicalDiagramConnection(GraphicalDiagramConnection* connection);
00178     void clearGraphicalModelDataDefinitions();
00180     void clearGraphicalDiagramConnections();
00182     void sanitizeGraphicalDataDefinitionsBookkeeping();
00184     void setDiagramLayerState(bool diagramCreated, bool visible);
00185     // Return only items that can be directly manipulated by user edit commands.
00187     QList<QGraphicsItem*> userOperableItems(const QList<QGraphicsItem*>& items) const;
00188     // Filter out non-deletable items from user-triggered delete flows.
00190     QList<QGraphicsItem*> userDeletableItems(const QList<QGraphicsItem*>& items) const;
00191     // Keep internal data-definition initial grouping opt-in during model rebuild only.
00193     void ensureInitialInternalDataDefinitionGrouping(GraphicalModelDataDefinition* dataDefinition,
    GraphicalModelComponent* component);
00194     // Allow serializer to disable automatic initial grouping while persisted GUI state is being
    restored.
00196     void setPersistedGuiRestoreInProgress(bool restoring);
00198     bool isPersistedGuiRestoreInProgress() const;
00200     void removeDrawing(QGraphicsItem * item, bool notify = false);
00201     bool removeDrawingGeometry(QGraphicsItem * item);
00202     bool removeDrawingAnimation(QGraphicsItem * item);
00204     void removeGroup(QGraphicsItemGroup* group, bool notify = false);
00206     void clearGraphicalModelComponents();
00208     void clearGraphicalModelConnections();
00210     void groupComponents(bool notify = false); // tenta agrupar (verifica se sao
    ModelGraphicalComponents)
00212     void groupModelComponents(QList<GraphicalModelComponent *> *graphicalComponents,
    QGraphicsItemGroup *group); // agrupa componentes
00214     void ungroupComponents(bool notify = false);
00216     void ungroupModelComponents(QGraphicsItemGroup *group);
00218     void notifyGraphicalModelChange(GraphicalModelEvent::EventType eventType,
    GraphicalModelEvent::EventObjectType eventObjectType, QGraphicsItem *item);
00219     // Return model component items by value to avoid heap ownership transfer.
00221     QList<GraphicalModelComponent*> graphicalModelComponentItems();
00223     GraphicalModelComponent* findGraphicalModelComponent(Util::identification id);
00225     GraphicalModelDataDefinition* findGraphicalModelDataDefinition(ModelDataDefinition*
    dataDefinition);
00227     GraphicalModelComponent* resolveSourceComponent(GraphicalConnection* connection) const;
00229     GraphicalModelComponent* resolveDestinationComponent(GraphicalConnection* connection) const;
00230 public:
00231     struct GRID {
00232         unsigned int interval;
00233         QPen pen;
00234         std::list<QGraphicsLineItem *> *lines;
00235         bool visible;
00236         void clear();
00237     };

```



```

00239     GRID *grid();
00241     void showGrid();
00242     // Aplica o estado visual do grid de forma determinística sem alternância implícita.
00244     void setGridVisible(bool visible);
00245     // Informa o estado visual atual do grid para sincronização com QAction.
00247     bool isGridVisible() const;
00249     void snapItemsToGrid();
00251     void actualizeDiagramArrows();
00253     void showDiagrams();
00255     void hideDiagrams();
00257     bool existDiagram();
00259     bool visibleDiagram();
00261     void destroyDiagram();
00263     void createDiagrams();
00265     QUndoStack* getUndoStack();
00267     Simulator* getSimulator();
00269     void setUndoStack(QUndoStack* undo);
00271     void beginConnection();
00273     void setSimulator(Simulator *simulator);
00275     void setPropertyEditor(PropertyEditorGenesys *propEditor);
00277     void setPropertyList(std::map<SimulationControl*, DataComponentProperty*>* propList);
00279     void setPropertyEditorUI(std::map<SimulationControl*, DataComponentEditor*>* propEditorUI);
00281     void setComboBox(std::map<SimulationControl*, ComboBoxEnum*>* propCombo);
00283     void setObjectBeingDragged(QTreeWidgetItem* objectBeingDragged);
00285     void setRestoringPersistedGuiLayout(bool restoring);
00287     bool isRestoringPersistedGuiLayout() const;
00289     void setParentWidget(QWidget *parentWidget);
00291     unsigned short connectingStep() const;
00293     void setConnectingStep(unsigned short connectingStep);
00295     void setSnapToGrid(bool activated);
00297     bool getSnapToGrid();
00299     void arranjeModels(int direction);
00301     void setDrawingMode(DrawingMode drawingMode);
00303     void clearDrawingMode();
00305     void setGraphicalComponentPort(GraphicalComponentPort * in);
00307     DrawingMode getDrawingMode();
00309     void setAction(QAction* action);
00311     QList<GraphicalModelComponent*> *getAllComponents();
00313     QList<GraphicalConnection*> *getAllConnections();
00315     QList<GraphicalModelDataDefinition*> *getAllDataDefinitions();
00317     QList<GraphicalDiagramConnection*> *getAllGraphicalDiagramsConnections();
00319     QMap<QGraphicsItemGroup *, QList<GraphicalModelComponent *> *getListComponentGroup();
00321     void insertComponentGroup(QGraphicsItemGroup *group, QList<GraphicalModelComponent *>
componentsGroup);
00323     void insertOldDataPositionItem(QGraphicsItem *item, QPointF position);
00325     QPointF getOldDataPositionItem(QGraphicsItem *item) const;
00327     QList<AnimationTransition *> *getAnimationsTransition();
00329     QList<AnimationCounter *> *getAnimationsCounter();
00331     QList<AnimationVariable *> *getAnimationsVariable();
00333     QList<AnimationTimer *> *getAnimationsTimer();
00335     QList<AnimationPlaceholder *> *getAnimationsPlaceholder();
00337     QMap<Event *, QList<AnimationTransition *> *> *getAnimationPaused();
00339     bool checkIgnoreEvent();
00341     void clearAnimations();
00343     void clearAnimationsTransition();
00345     void clearAnimationsCounter();
00347     void clearAnimationsVariable();
00349     void clearAnimationsTimer();
00351     void clearAnimationsPlaceholder();
00353     void clearAnimationsQueue();
00355     void animatePlot();
00357     QList<QString> *getImageAnimation();
00359     void drawingCounter();
00361     void drawingVariable();
00363     void drawingTimer();
00365     void drawingAttribute();
00367     void drawingEntity();
00369     void drawingEvent();
00371     void drawingExpression();
00373     void drawingPlot();
00375     void drawingQueue();
00377     void drawingResource();
00379     void drawingStation();
00381     void drawingStatistics();
00383     void clearAnimationsValues();
00385     void setCounters();
00387     void setVariables();
00388     // TODO: Funções abaixo são usadas para reaver os dataDefinitions do componente nos
dataDefinitions do modelo dos componentes deletados, "checados" e reinseridos (Control Z de um delete,
por exemplo).
00389     // O kernel não trata este caso, ale acusa erro, pois não encontra os dataDefinitions do
componente nos dataDefinitions do modelo, pois ele os remove como "órfãos" e não os reinsere quando
voltados ao modelo.
00391     void insertRestoredDataDefinitions(bool loaded);
00393     void saveDataDefinitions();
00394     // -----
00396     void requestGraphicalDataDefinitionsSync();

```

```

00398 void scheduleGraphicalDataDefinitionsSync();
00400 bool isGraphicalDataDefinitionsSyncInProgress() const;
00402 void setShowStatisticsDataDefinitions(bool show);
00404 bool showStatisticsDataDefinitions() const;
00406 void setShowEditableDataDefinitions(bool show);
00408 bool showEditableDataDefinitions() const;
00410 void setShowSharedDataDefinitions(bool show);
00412 bool showSharedDataDefinitions() const;
00414 void setShowRecursiveDataDefinitions(bool show);
00416 bool showRecursiveDataDefinitions() const;
00424 void setModelLevelFilter(unsigned int modelLevel);
00426 void clearModelLevelFilter();
00428 bool hasModelLevelFilter() const;
00430 unsigned int modelLevelFilter() const;
00432 void setShowInternalDataDefinitions(bool show);
00434 bool showInternalDataDefinitions() const;
00436 void setShowAttachedDataDefinitions(bool show);
00438 bool showAttachedDataDefinitions() const;
00440 void setConnectionGeometryUpdatesBlocked(bool blocked);
00442 bool areConnectionGeometryUpdatesBlocked() const;
00443
00444 public:
00446 QList<QGraphicsItem*>*getGraphicalModelDataDefinitions() const;
00448 QList<QGraphicsItem*>*getGraphicalModelComponents() const;
00450 QList<QGraphicsItem*>*getGraphicalConnections() const;
00452 QList<QGraphicsItem*>*getGraphicalGeometries() const;
00454 QList<QGraphicsItem*>*getGraphicalAnimations() const;
00456 QList<QGraphicsItem*>*getGraphicalEntities() const;
00458 QList<QGraphicsItem*>*getGraphicalDiagramsConnections() const;
00460 QList<QGraphicsItemGroup*>*getGraphicalGroups() const;
00461
00462 public:
00464 void animateTransition(ModelComponent *source, ModelComponent *destination, bool viewSimulation,
Event *event);
00466 void runAnimateTransition(AnimationTransition *animationTransition, Event *event, bool restart =
false);
00468 void handleAnimationStateChanged(QAbstractAnimation::State newState, QEventLoop* loop, Event*
event, AnimationTransition* animationTransition);
00470 void animateQueueInsert(ModelComponent *component, bool visible);
00472 void animateQueueRemove(ModelComponent *component);
00474 void animateCounter();
00476 void animateVariable(Entity* entity = nullptr);
00478 void animateTimer(double time);
00479
00480 protected: // virtual functions
00481 virtual void contextMenuEvent(QGraphicsSceneContextMenuEvent *contextMenuEvent);
00482 virtual void dragEnterEvent(QGraphicsSceneDragDropEvent *event);
00483 virtual void dragLeaveEvent(QGraphicsSceneDragDropEvent *event);
00484 virtual void dragMoveEvent(QGraphicsSceneDragDropEvent *event);
00485 //virtual void drawBackground(QPainter *painter, const QRectF &rect);
00486 //virtual void drawForeground(QPainter *painter, const QRectF &rect);
00487 virtual void dropEvent(QGraphicsSceneDragDropEvent *event);
00488 virtual void focusInEvent(QFocusEvent *focusEvent);
00489 virtual void focusOutEvent(QFocusEvent *focusEvent);
00490 //virtual void helpEvent(QGraphicsSceneHelpEvent *helpEvent);
00491 //virtual void inputMethodEvent(QInputMethodEvent *event);
00492 virtual void keyPressEvent(QKeyEvent *keyEvent);
00493 virtual void keyReleaseEvent(QKeyEvent *keyEvent);
00494 virtual void mouseDoubleClickEvent(QGraphicsSceneMouseEvent *mouseEvent);
00495 virtual void mouseMoveEvent(QGraphicsSceneMouseEvent *mouseEvent);
00496 virtual void mousePressEvent(QGraphicsSceneMouseEvent *mouseEvent);
00497 virtual void mouseReleaseEvent(QGraphicsSceneMouseEvent *mouseEvent);
00498 virtual void wheelEvent(QGraphicsSceneWheelEvent *wheelEvent);
00499
00500 private:
00501 bool tryCreateConnection(GraphicalComponentPort* source, GraphicalComponentPort* destination, bool
notify = true);
00502 GraphicalComponentPort* firstAvailableOutputPort(GraphicalModelComponent* component) const;
00503 GraphicalComponentPort* firstInputPort(GraphicalModelComponent* component) const;
00504 void resetConnectingState();
00505 private:
00506 GRID _grid;
00507 Simulator* _simulator = nullptr;
00508 PropertyEditorGenesys* _propertyEditor = nullptr;
00509 std::map<SimulationControl*, DataComponentProperty*>* _propertyList = nullptr;
00510 std::map<SimulationControl*, DataComponentEditor*>* _propertyEditorUI = nullptr;
00511 std::map<SimulationControl*, ComboBoxEnum*>* _propertyBox = nullptr;
00512 QTreeWidgetItem* _objectBeingDragged = nullptr;
00513 // Initialize the parent widget pointer to a known null state.
00514 QWidget* _parentWidget = nullptr;
00515 QList<GraphicalModelComponent*> _allGraphicalModelComponents;
00516 QList<GraphicalConnection*> _allGraphicalConnections;
00517 QList<GraphicalModelDataDefinition*> _allGraphicalModelDataDefinitions;
00518 QList<GraphicalDiagramConnection*> _allGraphicalDiagramConnections;
00519 QUndoStack *_undoStack = nullptr;
00520 QMap<QGraphicsItemGroup *, QList<GraphicalModelComponent *> > _listComponentsGroup;
00521 QMap<QGraphicsItem *, QPointF> _oldPositionsItems;

```

```

00522     QList<Counter *> *_counters = new QList<Counter *>();
00523     QList<ModelDataDefinition *> *_variables = new QList<ModelDataDefinition *>();
00524 private:
00525     DrawingMode _drawingMode = NONE;
00526     QGraphicsRectItem* _currentRectangle = nullptr;
00527     QGraphicsLineItem* _currentLine = nullptr;
00528     QGraphicsPolygonItem* _currentPolygon = nullptr;
00529     QGraphicsEllipseItem* _currentEllipse = nullptr;
00530     QPolygonF _currentPolygonPoints;
00531     QPointF _drawingStartPoint;
00532     QAction* _currentAction = nullptr;
00533     bool _drawing = false;
00534     bool _diagram = false;
00535     bool _visibleDiagram = false;
00536     bool _persistedGuiRestoreInProgress = false;
00537     unsigned short _connectingStep = 0; //0:nothing, 1:waiting click on source or destination, 2:
    click on source, 3: click on destination
00538     bool _controlIsPressed = false;
00539     bool _shiftSelectionInProgress = false;
00540     QGraphicsItem* _shiftClickedSelectableItem = nullptr;
00541     QList<QGraphicsItem*> _shiftSelectionBeforeClick;
00542     bool _snapToGrid = false;
00543     // Initialize the source port pointer before connection drawing starts.
00544     GraphicalComponentPort* _sourceGraphicalComponentPort = nullptr;
00545     // Initialize the destination port pointer before connection drawing starts.
00546     GraphicalComponentPort* _destinationGraphicalComponentPort = nullptr;
00547     // Initialize the current counter drawing pointer to avoid indeterminate access.
00548     AnimationCounter *_currentCounter = nullptr;
00549     // Initialize the current variable drawing pointer to avoid indeterminate access.
00550     AnimationVariable *_currentVariable = nullptr;
00551     // Initialize the current timer drawing pointer to avoid indeterminate access.
00552     AnimationTimer *_currentTimer = nullptr;
00553     AnimationPlaceholder *_currentPlaceholderAnimation = nullptr;
00554     QMap<Event *, QList<AnimationTransition *> *> *_animationPaused = new QMap<Event *,
    QList<AnimationTransition *> *>();
00555
00556 private:
00557     QList<QString> *_imagesAnimation = new QList<QString>;
00558     QList<AnimationTransition *> *_animationsTransition = new QList<AnimationTransition*>();
00559     QList<AnimationCounter *> *_animationsCounter = new QList<AnimationCounter*>();
00560     QList<AnimationVariable *> *_animationsVariable = new QList<AnimationVariable*>();
00561     QList<AnimationTimer *> *_animationsTimer = new QList<AnimationTimer*>();
00562     QList<AnimationPlaceholder *> *_animationsPlaceholder = new QList<AnimationPlaceholder*>();
00563 private:
00564     // IMPORTANT. MUST BE CONSISTENT WITH SIMULATOR->MODEL
00565     QList<QGraphicsItem*> *_graphicalModelComponents = new QList<QGraphicsItem*>();
00566     QList<QGraphicsItem*> *_graphicalModelDataDefinitions = new QList<QGraphicsItem*>();
00567     QList<QGraphicsItem*> *_graphicalConnections = new QList<QGraphicsItem*>();
00568     QList<QGraphicsItem*> *_graphicalDiagramConnections = new QList<QGraphicsItem*>();
00569     QList<QGraphicsItem*> *_graphicalAssociations = new QList<QGraphicsItem*>();
00570     QList<QGraphicsItem*> *_graphicalGeometries = new QList<QGraphicsItem*>();
00571     QList<QGraphicsItem*> *_graphicalAnimations = new QList<QGraphicsItem*>();
00572     QList<QGraphicsItem*> *_graphicalEntities = new QList<QGraphicsItem*>();
00573     QList<QGraphicsItemGroup*> *_graphicalGroups = new QList<QGraphicsItemGroup*>();
00574     bool _restoringPersistedGuiLayout = false;
00575     bool _graphicalDataDefinitionsSyncPending = false;
00576     bool _graphicalDataDefinitionsSyncInProgress = false;
00577     bool _graphicalDataDefinitionsSyncDeferredDuringRestore = false;
00578     bool _connectionGeometryUpdatesBlocked = false;
00579     bool _showStatisticsDataDefinitions = true;
00580     bool _showEditableDataDefinitions = true;
00581     bool _showSharedDataDefinitions = true;
00582     bool _showRecursiveDataDefinitions = true;
00583     // Rendering scope used by root/submodel tabs. The filter is intentionally scene-local so
00584     // static graphical synchronization calls can keep the same scope as the active view.
00585     bool _hasModelLevelFilter = false;
00586     unsigned int _modelLevelFilter = 0;
00587 };
00588
00589 #endif /* MODELGRAPHICSSCENE_H */

```

11.61 ModelGraphicsView.h

```

00001 /*
00002  * The MIT License
00003  *
00004  * Copyright 2022 rlcancian.
00005  *
00006  * Permission is hereby granted, free of charge, to any person obtaining a copy
00007  * of this software and associated documentation files (the "Software"), to deal
00008  * in the Software without restriction, including without limitation the rights
00009  * to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00010  * copies of the Software, and to permit persons to whom the Software is

```

```

00011  * furnished to do so, subject to the following conditions:
00012  *
00013  * The above copyright notice and this permission notice shall be included in
00014  * all copies or substantial portions of the Software.
00015  *
00016  * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017  * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018  * FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00019  * AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00020  * LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00021  * OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
00022  * THE SOFTWARE.
00023  */
00024
00025 /*
00026  * File:    QModelGraphicView.h
00027  * Author:  rlcancian
00028  *
00029  * Created on 15 de fevereiro de 2022, 21:12
00030  */
00031
00032 #ifndef QMODELGRAPHICVIEW_H
00033 #define QMODELGRAPHICVIEW_H
00034
00035 #include <QGraphicsView>
00036 #include <Qt>
00037 #include <QColor>
00038 #include <QStyle>
00039 #include <QGraphicsSceneMouseEvent>
00040 #include <QContextMenuEvent>
00041 #include <QPainter>
00042 #include "graphics/ModelGraphicsScene.h"
00043 #include "propertyeditor/DataComponentProperty.h"
00044 #include "propertyeditor/DataComponentEditor.h"
00045 #include "propertyeditor/ComboBoxEnum.h"
00046 #include "../kernel/simulator/model/ModelComponent.h"
00047 #include "kernel/simulator/Simulator.h"
00048 #include "kernel/simulator/PropertyGenesys.h"
00049 // #include "kernel/simulator/Plugin.h"
00050
00051 class ModelGraphicsView : public QGraphicsView {
00052 public:
00060     ModelGraphicsView(QWidget *parent = nullptr);
00062     ModelGraphicsView(const ModelGraphicsView& orig);
00064     virtual ~ModelGraphicsView();
00065 public: // editing graphic model
00066     // TODO: AddGraphicalModelComponent should be only on scene
00067     //GraphicalModelComponent* addGraphicalModelComponent(Plugin* plugin, ModelComponent* component,
00068     QPointF position);
00069     //bool removeGraphicalModelComponent(GraphicalModelComponent* gmc);
00069     //bool addGraphicalConnection(GraphicalComponentPort* sourcePort, GraphicalComponentPort*
00070     destinationPort);
00070     //bool removeGraphicalConnection(GraphicalConnection* gc);
00071     //bool addDrawing();
00072     //bool removeDrawing();
00073     //bool addAnimation();
00074     //bool removeAnimation();
00076     ModelGraphicsScene* getScene();
00077 public:
00079     void showGrid();
00081     void clear();
00083     void beginConnection();
00085     void selectModelComponent(ModelComponent* component);
00087     void setSimulator(Simulator* simulator);
00089     void setPropertyEditor(PropertyEditorGenesys* propEditor);
00091     void setPropertyList(std::map<SimulationControl*, DataComponentProperty*>* propList);
00093     void setPropertyEditorUI(std::map<SimulationControl*, DataComponentEditor*>* propEditorUI);
00095     void setComboBox(std::map<SimulationControl*, ComboBoxEnum*>* propBox);
00097     void setEnabled(bool enabled);
00099     QList<QGraphicsItem*> selectedItems();
00100 public: // events and notifications
00101
00102     template<typename Class> void setSceneMouseEventHandler(Class * object, void
00103     (Class::*function)(QGraphicsSceneMouseEvent*)) {
00104         sceneMouseEventHandlerMethod handlerMethod = std::bind(function, object,
00105         std::placeholders::_1);
00106         this->_sceneMouseEventHandler = handlerMethod;
00107     }
00107     template<typename Class> void setGraphicalModelEventHandler(Class * object, void
00108     (Class::*function)(const GraphicalModelEvent&)) {
00109         sceneGraphicalModelEventHandlerMethod handlerMethod = std::bind(function, object,
00110         std::placeholders::_1);
00111         this->_sceneGraphicalModelEventHandler = handlerMethod;
00112     }
00113     template <typename Class> void setSceneWheelInEventHandler(Class *object, void

```

```

(Class::*function)() {
00113     sceneWheelEventHandlerMethod handlerMethod = std::bind(function, object);
00114     this->_sceneWheelInEventHandler = handlerMethod;
00115 }
00116
00117     template <typename Class> void setSceneWheelOutEventHandler(Class *object, void
(Class::*function)() {
00118     sceneWheelEventHandlerMethod handlerMethod = std::bind(function, object);
00119     this->_sceneWheelOutEventHandler = handlerMethod;
00120 }
00121
00122     template <typename Class> void setContextMenuEventHandler(Class* object, void
(Class::*function)(QContextMenuEvent*)) {
00123     // Keep context-menu construction outside the view so menus can reuse MainWindow actions.
00124     contextMenuEventHandlerMethod handlerMethod = std::bind(function, object,
std::placeholders::_1);
00125     this->_contextMenuEventHandler = handlerMethod;
00126 }
00127
00129     void notifySceneMouseEventHandler(QGraphicsSceneMouseEvent* mouseEvent);
00131     void notifySceneWheelInEventHandler();
00133     void notifySceneWheelOutEventHandler();
00135     void notifySceneGraphicalModelEventHandler(const GraphicalModelEvent& modelGraphicsEvent);
00137     void setCanNotifyGraphicalModelEventHandlers(bool can);
00139     void clearEventHandlers();
00141     void setParentWidget(QWidget *parentWidget);
00142     // Enables or disables ruler rendering over the graphics viewport.
00144     void setRuleVisible(bool visible);
00145     // Informs if ruler rendering is active for action synchronization.
00147     bool isRuleVisible() const;
00148     // Enables or disables guide line rendering over the graphics viewport.
00150     void setGuidesVisible(bool visible);
00151     // Informs if guide line rendering is active for action synchronization.
00153     bool isGuidesVisible() const;
00154 protected:// slots:
00155     void changed(const QList<QRectF> &region);
00156     void focusItemChanged(QGraphicsItem *newFocusItem, QGraphicsItem *oldFocusItem, Qt::FocusReason
reason);
00157     void sceneRectChanged(const QRectF &rect);
00158     void selectionChanged();
00159 protected: // virtual functions
00160     virtual void contextMenuEvent(QContextMenuEvent *event) override;
00161     virtual void dragEnterEvent(QDragEnterEvent *event) override;
00162     //virtual void dragLeaveEvent(QDragLeaveEvent *event) override;
00163     //virtual void dragMoveEvent(QDragMoveEvent *event) override;
00164     //virtual void dropEvent(QDropEvent *event) override;
00165     //virtual bool event(QEvent *event) override;
00166     //virtual void focusInEvent(QFocusEvent *event) override;
00167     //virtual bool focusNextPrevChild(bool next) override;
00168     //virtual void focusOutEvent(QFocusEvent *event) override;
00169     //virtual void inputMethodEvent(QInputMethodEvent *event) override;
00170     virtual void keyPressEvent(QKeyEvent *event) override;
00171     virtual void keyReleaseEvent(QKeyEvent *event) override;
00172     //virtual void mouseDoubleClickEvent(QMouseEvent *event) override;
00173     //virtual void mouseMoveEvent(QMouseEvent *event) override;
00174     //virtual void mousePressEvent(QMouseEvent *event) override;
00175     //virtual void mouseReleaseEvent(QMouseEvent *event) override;
00176     //virtual void paintEvent(QPaintEvent *event) override;
00177     //virtual void resizeEvent(QResizeEvent *event) override;
00178     //virtual void scrollContentsBy(int dx, int dy) override;
00179     //virtual void showEvent(QShowEvent *event) override;
00180     //virtual bool viewportEvent(QEvent *event) override;
00181     virtual void wheelEvent(QWheelEvent *event) override;
00182     // Draws optional rulers and guides using the current visible scene rectangle.
00183     virtual void drawForeground(QPainter *painter, const QRectF &rect) override;
00184 private:
00185     typedef std::function<void(QGraphicsSceneMouseEvent*) > sceneMouseEventHandlerMethod;
00186     typedef std::function<void()> sceneWheelEventHandlerMethod;
00187     typedef std::function<void(const GraphicalModelEvent&)> sceneGraphicalModelEventHandlerMethod;
00188     typedef std::function<void(QContextMenuEvent*)> contextMenuEventHandlerMethod;
00189     sceneMouseEventHandlerMethod _sceneMouseEventHandler;
00190     sceneWheelEventHandlerMethod _sceneWheelInEventHandler;
00191     sceneWheelEventHandlerMethod _sceneWheelOutEventHandler;
00192     sceneGraphicalModelEventHandlerMethod _sceneGraphicalModelEventHandler;
00193     contextMenuEventHandlerMethod _contextMenuEventHandler;
00194     Simulator* _simulator = nullptr;
00195     PropertyEditorGenesys* _propertyEditor = nullptr;
00196     std::map<SimulationControl*, DataComponentProperty*> _propertyList = nullptr;
00197     std::map<SimulationControl*, DataComponentEditor*> _propertyEditorUI = nullptr;
00198     std::map<SimulationControl*, ComboBoxEnum*> _propertyBox = nullptr;
00199     QWidget* _parentWidget;
00200     bool _notifyGraphicalModelEventHandlers = true;
00201     bool _ruleVisible = false;
00202     bool _guidesVisible = false;
00203 };
00204
00205 #endif /* QMODELGRAPHICVIEW_H */

```

11.62 GuiCrashDiagnostics.h

```

00001 #ifndef GUICRASHDIAGNOSTICS_H
00002 #define GUICRASHDIAGNOSTICS_H
00003
00011 namespace GuiCrashDiagnostics {
00012
00016 void installFatalSignalHandlers();
00017
00018 } // namespace GuiCrashDiagnostics
00019
00020 #endif // GUICRASHDIAGNOSTICS_H

```

11.63 GuiScopeTrace.h

```

00001 #ifndef GUISCOPETRACE_H
00002 #define GUISCOPETRACE_H
00003
00004 #include <chrono>
00005
00013 class GuiScopeTrace {
00014 public:
00020     GuiScopeTrace(const char* methodName, const void* thisPtr);
00024     ~GuiScopeTrace();
00025
00026 private:
00028     const char* _methodName;
00030     const void* _thisPtr;
00032     int _depth;
00034     std::chrono::steady_clock::time_point _start;
00035 };
00036
00037 #endif // GUISCOPETRACE_H

```

11.64 guithememanager.h

```

00001 #ifndef GUITHEMEMANAGER_H
00002 #define GUITHEMEMANAGER_H
00003
00004 class QApplication;
00005 class ModelGraphicsView;
00006
00013 class GuiThemeManager {
00014 public:
00019     static void applyApplicationTheme(QApplication* application);
00024     static void applyModelGraphicsTheme(ModelGraphicsView* graphicsView);
00025
00026 private:
00027     GuiThemeManager() = default;
00028 };
00029
00030 #endif // GUITHEMEMANAGER_H

```

11.65 applications/gui/genesys/mainwindow.cpp File Reference

Composition-root partition of [MainWindow](#) implementation.

```

#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "kernel/simulator/SinkModelComponent.h"
#include "../.../kernel/simulator/essentialPlugins/Attribute.h"
#include "../.../kernel/simulator/essentialPlugins/Counter.h"
#include "../.../kernel/simulator/essentialPlugins/StatisticsCollector.h"
#include "kernel/simulator/PluginManager.h"
#include "kernel/simulator/Plugin.h"
#include "../.../kernel/simulator/model/ModelComponent.h"
#include "../.../kernel/simulator/model/ModelComponentManager.h"

```

```
#include "../.../kernel/simulator/model/ModelDataManager.h"
#include "graphicals/ModelGraphicsScene.h"
#include "TraitsGUI.h"
#include "graphicals/GraphicalConnection.h"
#include "graphicals/GraphicalModelDataDefinition.h"
#include "controllers/SimulationController.h"
#include "controllers/ModelInspectorController.h"
#include "controllers/TraceConsoleController.h"
#include "controllers/SimulationEventController.h"
#include "controllers/PluginCatalogController.h"
#include "controllers/PropertyEditorController.h"
#include "controllers/ModelLifecycleController.h"
#include "controllers/SimulationCommandController.h"
#include "controllers/EditCommandController.h"
#include "controllers/SceneToolController.h"
#include "controllers/GraphicalContextMenuController.h"
#include "controllers/DialogUtilityController.h"
#include "extensions/GuiExtensionContracts.h"
#include "extensions/GuiExtensionManager.h"
#include "extensions/GuiExtensionPluginCatalog.h"
#include "services/ModelLanguageSynchronizer.h"
#include "services/GraphvizModelExporter.h"
#include "services/CppModelExporter.h"
#include "services/GraphicalModelSerializer.h"
#include "services/GraphicalModelBuilder.h"
#include "UtilGUI.h"
#include "guithememanager.h"
#include "propertyeditor/qtpropertybrowser/qttreepropertybrowser.h"
#include "animations/AnimationVariable.h"
#include "systempreferences.h"
#include "../.../plugins/data/Logic/Variable.h"
#include <string>
#include <fstream>
#include <sstream>
#include <cstdlib>
#include <algorithm>
#include <cmath>
#include <limits>
#include <QMessageBox>
#include <QTextStream>
#include <QFileDialog>
#include <QFileInfo>
#include <QDateTime>
#include <QEventLoop>
#include <QTemporaryFile>
#include <Qt>
#include <QGraphicsPixmapItem>
#include <QKeySequence>
#include <QPropertyAnimation>
#include <QtWidgets/qgraphicsitem.h>
#include <QScreen>
#include <QDebug>
#include <QRegularExpression>
#include <QRandomGenerator>
#include <QAction>
#include <QFrame>
#include <QLabel>
#include <QMap>
```

```
#include <QMenu>
#include <QPainter>
#include <QScrollArea>
#include <QSignalBlocker>
#include <QSizePolicy>
#include <QSplitter>
#include <QTabBar>
#include <QTabWidget>
#include <QTreeWidget>
#include <QTimer>
#include <QToolBar>
#include <QVBoxLayout>
```

11.65.1 Detailed Description

Composition-root partition of [MainWindow](#) implementation.

This file contains central [MainWindow](#) construction, initialization, and wiring logic, including creation of extracted controllers/services and baseline UI setup. It does not define a new class; it is a physical partition of the same [MainWindow](#) implementation used as a compatibility façade in the incremental refactoring.

11.66 mainwindow.h

```
00001 #ifndef MAINWINDOW_H
00002 #define MAINWINDOW_H
00003
00004 #include <QMainWindow>
00005 #include <QListWidget>
00006 #include <QTreeWidgetItem>
00007 #include <QGraphicsItem>
00008 #include <QUndoView>
00009 #include <QMetaObject>
00010 #include <memory>
00011 #include <map>
00012
00013 #include "propertyeditor/DataComponentProperty.h"
00014 #include "propertyeditor/DataComponentEditor.h"
00015 #include "propertyeditor/ComboBoxEnum.h"
00016
00017 #include "kernel/simulator/Simulator.h"
00018 #include "kernel/simulator/PropertyGenesys.h"
00019 #include "kernel/simulator/TraceManager.h"
00020 #include "graphicals/ModelGraphicsScene.h"
00021
00022 QT_BEGIN_NAMESPACE
00023     namespace Ui {
00024         class MainWindow;
00025     }
00026
00027 QT_END_NAMESPACE
00028
00029 class ModelLanguageSynchronizer;
00030 class GraphvizModelExporter;
00031 class CppModelExporter;
00032 class GraphicalModelSerializer;
00033 class GraphicalModelBuilder;
00034 class ModelInspectorController;
00035 class TraceConsoleController;
00036 class SimulationEventController;
00037 class PluginCatalogController;
00038 class PropertyEditorController;
00039 class ModelLifecycleController;
00040 class SimulationCommandController;
00041 class EditCommandController;
00042 class SceneToolController;
00043 class GraphicalContextMenuController;
00044 class DialogUtilityController;
00045 class GuiExtensionManager;
00046 class QAction;
```



```
00047 class QMenu;
00048 class QTabWidget;
00049 class ModelGraphicsView;
00050 class ModelComponent;
00051 class ModelDataDefinition;
00052
00053 // Document MainWindow as composition root and incremental compatibility façade.
00073 class MainWindow : public QMainWindow {
00074     Q_OBJECT
00075
00076     // Allow the Phase 4 event controller to register private compatibility handlers.
00077     friend class SimulationEventController;
00078
00079 public:
00081     MainWindow(QWidget *parent = nullptr);
00083     ~MainWindow();
00085     ModelGraphicsScene* myScene() const;
00086
00087 public: // to notify changes
00089     bool graphicalModelHasChanged() const;
00091     void setGraphicalModelHasChanged(bool graphicalModelHasChanged);
00093     void unselectDrawIcons();
00095     bool checkSelectedDrawIcons();
00097     void refreshGuiExtensions();
00099     void refreshPluginCatalog();
00100
00101 private slots:
00102     // actions
00103     void on_actionEditUndo_triggered();
00104     void on_actionEditRedo_triggered();
00105     void on_actionEditFind_triggered();
00106     // void on_actionReplace_triggered(); // old name (without menu namespace)
00107     void on_actionEditReplace_triggered();
00108     void on_actionEditCut_triggered();
00109     void on_actionEditCopy_triggered();
00110     void on_actionEditPaste_triggered();
00111     void on_actionEditDelete_triggered();
00112     void on_actionEditGroup_triggered();
00113     void on_actionEditUngroup_triggered();
00114     void on_actionViewGroup_triggered();
00115     void on_actionViewUngroup_triggered();
00116
00117     void on_actionShowGrid_triggered();
00118     void on_actionShowRule_triggered();
00119     void on_actionShowGuides_triggered();
00120     void on_actionViewConfigure_triggered();
00121
00122     void on_actionZoom_In_triggered();
00123     void on_actionZoom_Out_triggered();
00124     void on_actionZoom_All_triggered();
00125     void on_actionZoom_OneToOne_triggered();
00126
00127     void on_actionDrawLine_triggered();
00128     void on_actionDrawRectangle_triggered();
00129     void on_actionDrawEllipse_triggered();
00130     void on_actionDrawText_triggered();
00131     void on_actionDrawPolygon_triggered();
00132
00133     void on_actionAnimateSimulatedTime_triggered();
00134     void on_actionAnimateVariable_triggered();
00135     void on_actionAnimateExpression_triggered();
00136     void on_actionAnimateResource_triggered();
00137     void on_actionAnimateQueue_triggered();
00138     void on_actionAnimateStation_triggered();
00139     void on_actionAnimateCounter_triggered();
00140     void on_actionAnimateEntity_triggered();
00141     void on_actionAnimateEvent_triggered();
00142     void on_actionAnimateAttribute_triggered();
00143     void on_actionAnimateStatistics_triggered();
00144     void on_actionAnimatePlot_triggered();
00145
00146     void on_actionSimulationStop_triggered();
00148     void on_actionSimulationStart_triggered();
00150     void on_actionSimulationStep_triggered();
00152     void on_actionSimulationPause_triggered();
00154     void on_actionSimulationResume_triggered();
00156     void on_actionSimulationConfigure_triggered();
00157
00158     void on_actionAboutAbout_triggered();
00159     void on_actionAboutLicence_triggered();
00160     void on_actionAboutGetInvolved_triggered();
00161
00162     void on_actionAlignMiddle_triggered();
00163     void on_actionAlignTop_triggered();
00164     void on_actionAlignRight_triggered();
00165     void on_actionAlignCenter_triggered();
00166     void on_actionAlignLeft_triggered();
```

```

00167
00168 void on_actionToolsParserGrammarChecker_triggered();
00169 void on_actionToolsWebWorker_triggered();
00170 // Legacy slot: there is no matching QAction in mainwindow.ui at this moment.
00171 void on_actionToolsExperimentation_triggered();
00172 void on_actionToolsOptimizator_triggered();
00173 void on_actionToolsAIAssistant_triggered();
00174 void on_actionToolsDataAnalyzer_triggered();
00175
00176 void on_actionSimulatorsPluginManager_triggered();
00177 void on_actionSimulatorExit_triggered();
00178 void on_actionSimulatorPreferences_triggered();
00179
00181 void on_actionModelNew_triggered();
00183 void on_actionModelOpen_triggered();
00185 void on_actionModelSave_triggered();
00187 void on_actionModelClose_triggered();
00189 void on_actionAnimation_triggered();
00191 void on_actionModelInformation_triggered();
00193 void on_actionModelCheck_triggered();
00194
00195 void on_actionGModelComponentBreakpoint_triggered();
00196 void on_actionShowInternalElements_triggered();
00197 void on_actionShowEditableElements_triggered();
00198 void on_actionShowAttachedElements_triggered();
00199 void on_actionShowRecursiveElements_triggered();
00200
00201
00202 // widget events
00203 //void on_textCodeEdit_Model_textChanged();
00204 void on_tabWidget_Model_tabBarClicked(int index);
00205 void on_tabWidget_Debug_currentChanged(int index);
00206 void on_tabWidgetCentral_currentChanged(int index);
00207 void on_tabWidgetCentral_tabBarClicked(int index);
00208 void on_tabWidgetModel_currentChanged(int index);
00209 void on_tabWidgetSimulation_currentChanged(int index);
00210 void on_tabWidgetReports_currentChanged(int index);
00211 void on_tabWidgetModelLanguages_currentChanged(int index);
00212
00213 void on_treeWidget_Plugins_itemDoubleClicked(QTreeWidgetItem *item, int column);
00214 void on_treeWidgetComponents_itemSelectionChanged();
00215 void on_treeWidgetDataDefinitions_itemSelectionChanged();
00216 void on_treeWidget_Plugins_itemClicked(QTreeWidgetItem *item, int column);
00217
00218 void on_horizontalSlider_Zoom_valueChanged(int value);
00219 void on_horizontalSlider_ZoomGraphical_valueChanged(int value);
00220
00221 void on_checkBox_ShowElements_stateChanged(int arg1);
00222 void on_checkBox_ShowInternals_stateChanged(int arg1);
00223 void on_checkBox_ShowEditableElements_stateChanged(int arg1);
00224 void on_checkBox_ShowRecursive_stateChanged(int arg1);
00225 void on_checkBox_ShowLevels_stateChanged(int arg1);
00226
00227 void on_pushButton_Breakpoint_Insert_clicked();
00228 void on_pushButton_Breakpoint_Remove_clicked();
00229 void on_pushButton_Export_clicked();
00230
00232 void on_graphicsView_rubberBandChanged(const QRect &viewportRect, const QPointF &fromScenePoint,
const QPointF &toScenePoint);
00234 void on_TextCodeEditor_textChanged();
00235
00236 void on_treeWidgetDataDefinitions_itemDoubleClicked(QTreeWidgetItem *item, int column);
00237 void on_treeWidgetDataDefinitions_itemChanged(QTreeWidgetItem *item, int column);
00238 void on_actionArranjeLeft_triggered();
00239 void on_actionArranjeRight_triggered();
00240 void on_actionArranjeTop_triggered();
00241 void on_actionArranjeBototm_triggered();
00242 void on_actionArranjeCenter_triggered();
00243 void on_actionArranjeMiddle_triggered();
00244 void on_actionShowSnap_triggered();
00245 void on_actionGModelShowConnect_triggered(bool checked = false);
00246
00247 void on_actionActivateGraphicalSimulation_triggered();
00248 void on_horizontalSliderAnimationSpeed_valueChanged(int value);
00249 void on_actionSelectAll_triggered();
00250
00251 void on_actionParallelization_triggered();
00252
00253 void on_horizontalSlider_ZoomGraphical_actionTriggered(int action);
00255 void _onPropertyEditorModelChanged();
00256
00257 protected:
00258 void closeEvent(QCloseEvent *event) override;
00259
00260 private: // VIEW
00261
00262 private: // trace handlers

```

```

00264     void _simulatorTraceHandler(TraceEvent e);
00266     void _simulatorTraceErrorHandler(TraceErrorEvent e);
00268     void _simulatorTraceSimulationHandler(TraceSimulationEvent e);
00270     void _simulatorTraceReportsHandler(TraceEvent e);
00271 private: // simulator event handlers
00273     void _onModelCheckSuccessHandler(ModelEvent* re);
00275     void _onReplicationStartHandler(SimulationEvent* re);
00277     void _onReplicationEndHandler(SimulationEvent* re);
00279     void _onSimulationStartHandler(SimulationEvent* re);
00281     void _onSimulationPausedHandler(SimulationEvent* re);
00283     void _onSimulationResumeHandler(SimulationEvent* re);
00285     void _onSimulationEndHandler(SimulationEvent* re);
00287     void _onProcessEventHandler(SimulationEvent* re);
00289     void _onEntityCreateHandler(SimulationEvent* re);
00291     void _onEntityRemoveHandler(SimulationEvent* re);
00293     void _onMoveEntityEvent(SimulationEvent * re);
00295     void _onAfterProcessEvent(SimulationEvent * re);
00296 private: // model Graphics View handlers
00298     void _onSceneMouseEvent(QGraphicsSceneMouseEvent* mouseEvent);
00300     void _onSceneWheelInEvent();
00302     void _onSceneWheelOutEvent();
00304     void _onSceneGraphicalModelEvent(const GraphicalModelEvent& event);
00305 private: // QGraphicsScene Slots
00307     void sceneChanged(const QList<QRectF> &region);
00309     void sceneFocusItemChanged(QGraphicsItem *newFocusItem, QGraphicsItem *oldFocusItem,
Qt::FocusReason reason);
00310     //void sceneRectChanged(const QRectF &rect);
00312     void sceneSelectionChanged();
00313 private: // Similar to QGraphicsScene Slots
00315     void sceneGraphicalModelChanged();
00316 private: // simulator related
00318     void _setOnEventHandlers();
00320     bool _ensureSimulationReady(bool checkModel = true);
00322     bool _hasCurrentModelSimulation() const;
00324     void _insertPluginUI(Plugin* plugin);
00326     void _insertFakePlugins();
00328     bool _setSimulationModelBasedOnText();
00330     bool _createModelImage();
00332     std::string _adjustDotName(std::string name);
00334     void _insertTextInDot(std::string text, unsigned int compLevel, unsigned int compRank,
std::map<unsigned int, std::map<unsigned int, std::list<std::string>*>*> dotmap, bool isNode =
false);
00336     void _recursiveCreateModelGraphicPicture(ModelDataDefinition* componentOrData,
std::list<ModelDataDefinition*> visited, std::map<unsigned int, std::map<unsigned int,
std::list<std::string>*>*> dotmap);
00338     void _actualizeModelCppCode();
00340     std::string _addCppCodeLine(std::string line, unsigned int indent = 0);
00341 private: // view
00343     void _initModelGraphicsView();
00345     void _connectSceneSignals();
00347     void _disconnectSceneSignals(const char* context);
00349     void _initializeModelTabs();
00351     void _configureModelGraphicsView(ModelGraphicsView* graphicsView);
00353     ModelGraphicsView* _createModelGraphicsView(QWidget* parent);
00355     void _activateModelGraphicsView(ModelGraphicsView* graphicsView, bool rebuildControllers = true);
00357     ModelGraphicsView* _ensureModelTab(Model* model);
00359     void _removeModelTab(Model* model);
00361     void _updateModelTabs();
00363     void _activateModelTab(int index);
00365     void _activatePreviousModel();
00367     void _activateNextModel();
00369     void _rebuildViewDependentControllers();
00371     void _refreshGuiExtensions();
00373     void _initUiForNewModel(Model* m);
00375     void _actualizeActions();
00377     void _actualizeUndo();
00379     void _actualizeTabPanels();
00381     void _actualizeModelSimLanguage();
00383     void _actualizeModelTextHasChanged(bool hasChanged);
00385     void _actualizeSimulationEvents(SimulationEvent * re);
00387     void _actualizeDebugVariables(bool force);
00389     void _actualizeDebugEntities(bool force);
00391     void _actualizeDebugBreakpoints(bool force);
00393     void _prepareReportsResultsTable();
00395     void _clearReportsResultsTable();
00397     void _actualizeReportsResultsTable();
00399     void _prepareReportsPlots();
00401     void _clearReportsPlots();
00403     void _actualizeReportsPlots();
00405     void _showReplicationReportIfSuppressed();
00407     void _showSimulationReportIfSuppressed();
00409     void _actualizeModelComponents(bool force);
00411     void _actualizeModelDataDefinitions(bool force);
00413     void _actualizeGraphicalModel(SimulationEvent * re);
00415     void _insertCommandInConsole(std::string text);
00417     void _clearModelEditors();
00419     void _gentle_zoom(double factor);

```

```

00421     void _showMessageNotImplemented();
00423     void _recursivelyGenerateGraphicalModelFromModel(ModelComponent* component, List<ModelComponent*>*
visited, std::map<ModelComponent*, GraphicalModelComponent*>* map, int *x, int *y, int *ymax, int
sequenceInline);
00425     void _generateGraphicalModelFromModel();
00427     void saveItemForCopy(QList<GraphicalModelComponent*> * gmcList, QList<GraphicalConnection*> *
connList);
00428     //bool _checkStartSimulation();
00429 private: // opened-model document state
00431     enum class GraphicalModelViewContextKind {
00432         RootModel,
00433         Submodel
00434     };
00435
00444     struct GraphicalModelViewContext {
00445         GraphicalModelViewContextKind kind = GraphicalModelViewContextKind::RootModel;
00446         Model* rootModel = nullptr;
00447         unsigned int modelLevel = 0;
00448         ModelComponent* ownerComponent = nullptr;
00449         ModelDataDefinition* ownerDataDefinition = nullptr;
00450         ModelGraphicsView* graphicsView = nullptr;
00451         QString explicitTitle;
00452     };
00453
00455     GraphicalModelViewContext* _activeViewContext() const;
00457     GraphicalModelViewContext* _contextForView(ModelGraphicsView* graphicsView) const;
00459     GraphicalModelViewContext* _rootContextForModel(Model* model) const;
00461     GraphicalModelViewContext* _ensureRootModelViewContext(Model* model, ModelGraphicsView*
graphicsView);
00463     ModelGraphicsView* _ensureSubmodelViewContext(ModelDataDefinition* owner);
00465     void _closeSubmodelViewContext(ModelGraphicsView* graphicsView);
00467     void _removeSubmodelViewContextsForModel(Model* model);
00469     void _removeViewContext(ModelGraphicsView* graphicsView);
00471     QString _viewContextTitle(const GraphicalModelViewContext& context) const;
00473     bool _viewContextBelongsToOpenModel(const GraphicalModelViewContext& context) const;
00475     ModelDataDefinition* _selectedSubmodelOwner() const;
00477     bool _canOpenSubmodelFor(ModelDataDefinition* owner) const;
00479     void _openSelectedSubmodel();
00480     void _refreshRecentModelsMenu();
00481     bool _openRecentModelFile(const QString& fileName);
00483     void _syncCurrentModelDocumentState();
00485     void _restoreModelDocumentState(Model* model);
00487     QString _modelDisplayName(Model* model) const;
00489     QString _modelSaveBaseFilename(QString filename) const;
00491     bool _modelHasPendingChanges(Model* model) const;
00493     bool _saveCurrentModel(bool promptForFilename = true);
00495     bool _saveModelWithActivation(Model* model);
00497     bool _confirmCloseModel(Model* model);
00499     bool _closeModel(Model* model);
00501     void _clearModelGraphicsViewForClose(ModelGraphicsView* graphicsView);
00502 private: // graphical model persistence
00504     bool _saveGraphicalModel(QString filename);
00506     bool _saveTextModel(QFile *saveFile, QString data);
00508     Model* _loadGraphicalModel(std::string filename);
00509 private: //???
00511     void _helpCopy();
00512 private: // interface and model main elements to join
00513     Ui::MainWindow *ui;
00514     Simulator* simulator;
00515     // Hosts one graphical scene/view per opened kernel model.
00516     QWidget* _modelGraphicsTabs = nullptr;
00517     // Maps opened kernel models to their graphical view so tab switching preserves each scene.
00518     std::map<Model*, ModelGraphicsView*> _modelGraphicsViews;
00519     // Reverse lookup used by tab activation without relying on QVariant pointer casts.
00520     std::map<ModelGraphicsView*, Model*> _modelsByGraphicsView;
00521     // Owns graphical contexts by view; currently every context is root-model, future ones may be
submodels.
00522     std::map<ModelGraphicsView*, std::unique_ptr<GraphicalModelViewContext>
_viewContextsByGraphicsView;
00523     // Fast lookup for the root graphical context of each opened kernel model.
00524     std::map<Model*, GraphicalModelViewContext*> _rootViewContextsByModel;
00525     // Keeps each opened model's last graphical/text filename independent while switching tabs.
00526     std::map<Model*, QString> _modelFileNames;
00527     // Caches the editable Simulang text per opened model so tab switches do not discard unsaved text.
00528     std::map<Model*, QString> _modelTextContents;
00529     // Tracks text-editor dirty state per opened model while legacy flags mirror only the active
model.
00530     std::map<Model*, bool> _modelTextHasChanged;
00531     // Tracks graphical dirty state per opened model while legacy flags mirror only the active model.
00532     std::map<Model*, bool> _modelGraphicalHasChanged;
00533     // Tracks tabs being changed programmatically to avoid re-entrant model switching.
00534     bool _changingModelTabProgrammatically = false;
00535     // Runtime menu actions for navigating the opened-model list.
00536     QAction* _actionModelPrevious = nullptr;
00537     QAction* _actionModelNext = nullptr;
00538     // Runtime action that opens a level-scoped graphical tab from the selected owner item.
00539     QAction* _actionOpenSelectedSubmodel = nullptr;

```

```

00540     QMenu* _menuOpenRecent = nullptr;
00541     QList<QAction*> _recentModelActions;
00542     // Keep core simulation command gateway owned by MainWindow composition root.
00543     std::unique_ptr<class SimulationController> _simulationController;
00544     // Keep phase-ordered services to make composition dependencies explicit in Phase 12.
00545     // Synchronize textual model language with the kernel model manager.
00546     std::unique_ptr<ModelLanguageSynchronizer> _modelLanguageSynchronizer;
00547     // Generate Graphviz DOT and rendered model images for the GUI pane.
00548     std::unique_ptr<GraphvizModelExporter> _graphvizModelExporter;
00549     // Generate C++ model representation text for the code viewer pane.
00550     std::unique_ptr<CppModelExporter> _cppModelExporter;
00551     // Persist and restore graphical .gui state through the Phase 2 serializer service.
00552     std::unique_ptr<GraphicalModelSerializer> _graphicalModelSerializer;
00553     // Rebuild graphical components and links through the Phase 2 builder service.
00554     std::unique_ptr<GraphicalModelBuilder> _graphicalModelBuilder;
00555     // Keep phase-ordered controllers to preserve the final compatibility façade surface.
00556     // Add the Phase 3 model-inspector controller owned by MainWindow.
00557     std::unique_ptr<ModelInspectorController> _modelInspectorController;
00558     // Add the Phase 4 trace controller owned by MainWindow.
00559     std::unique_ptr<TraceConsoleController> _traceConsoleController;
00560     // Add the Phase 4 simulation-event controller owned by MainWindow.
00561     std::unique_ptr<SimulationEventController> _simulationEventController;
00562     // Add the Phase 5 plugin-catalog controller owned by MainWindow.
00563     std::unique_ptr<PluginCatalogController> _pluginCatalogController;
00564     // Add the Phase 6 property-editor controller owned by MainWindow.
00565     std::unique_ptr<PropertyEditorController> _propertyEditorController;
00566     // Add the Phase 7 model-lifecycle controller owned by MainWindow.
00567     std::unique_ptr<ModelLifecycleController> _modelLifecycleController;
00568     // Add the Phase 8 simulation-command controller owned by MainWindow.
00569     std::unique_ptr<SimulationCommandController> _simulationCommandController;
00570     // Add the Phase 9 edit-command controller owned by MainWindow.
00571     std::unique_ptr<EditCommandController> _editCommandController;
00572     // Add the Phase 10 scene-tool controller owned by MainWindow.
00573     std::unique_ptr<SceneToolController> _sceneToolController;
00574     // Add the graphical context-menu controller owned by MainWindow.
00575     std::unique_ptr<GraphicalContextMenuController> _graphicalContextMenuController;
00576     // Add the Phase 11 dialog-utility controller owned by MainWindow.
00577     std::unique_ptr<DialogUtilityController> _dialogUtilityController;
00578     // Keeps runtime GUI extension contributions isolated from core static menus/toolbars.
00579     std::unique_ptr<GuiExtensionManager> _guiExtensionManager;
00580     PropertyEditorGenesys* propertyGenesys;
00581     std::map<SimulationControl*, DataComponentProperty*>* propertyList;
00582     std::map<SimulationControl*, DataComponentEditor*>* propertyEditorUI;
00583     std::map<SimulationControl*, ComboBoxEnum*>* propertyBox;
00584 private: // attributes to be saved and loaded withing the graphical model
00585     int _zoomValue; // todo should be set for each open graphical model, such as view rect, etc
00586 private: // misc useful
00587     bool _check(bool success = true);
00588     bool _hasPendingModelChanges() const;
00589     bool _confirmApplicationExit();
00590     bool _textModelHasChanged = false;
00591     bool _graphicalModelHasChanged = false;
00592     bool _modelWasOpened = false;
00593     // Avoids duplicated prompts when exit is approved and Qt emits close events during shutdown.
00594     bool _closingApproved = false;
00595     QString _autoLoadPluginsFilename = "autoloadplugins.txt";
00596     bool _checkItemsScene();
00597     QString _modelFilename;
00598     std::map<std::string /*category*/, QColor*> _pluginCategoryColor = new
00599     std::map<std::string, QColor>();
00600     // TODO 1: Faz parte do mecanismo de restaurar dataDefinitions deletados do modelo e que são
00601     restaurados com um Control Z
00602     // Caso: Ao adicionar um Create no modelo e dar um check() o EntityType será criado,
00603     // mas ao deletar o Create, dar outro check() e em seguida dar um Control Z (voltando o Create no
00604     modelo) e checar novamente, o EntityType não é restaurado
00605     // na lista de dataDefinitions do modelo, apesar de ainda existir no Create.
00606     // Neste caso, ocorre um erro acusando que o EntityType do Create não está no modelo.
00607     // Isso se dá ao fato de que no Kernel ele verifica se _entityType = nullptr para criar um novo e
00608     reinserir no modelo, mas não trata o caso dele não ser nullptr
00609     // e não estar nos dataDefinitions do modelo, que é o que ocorre na situação que foi descrita.
00610     // Esta função foi feita para tratar esse caso, assim como é feito com
00611     insertRestoredDataDefinitions e saveDataDefinitions em ModelGraphicScene
00612     void setStatisticsCollector();
00613     bool _cut;
00614     QList<GraphicalModelComponent*> * _gmc_copies = new QList<GraphicalModelComponent*>();
00615     QList<GraphicalConnection*> * _ports_copies = new QList<GraphicalConnection*>();
00616     QList<QGraphicsItem *> * _draw_copy = new QList<QGraphicsItem *>();
00617     QList<QGraphicsItemGroup *> * _group_copy = new QList<QGraphicsItemGroup *>();
00618     struct COPY {
00619         GraphicalModelComponent * old;
00620         GraphicalModelComponent * copy;
00621     };
00622     bool _firstClickShowConnection = true;
00623
00624
00625

```

```

00626 private:
00627
00628     const struct TABINDEXES_STRUC {
00629         const int TabCentralModelIndex = 0;
00630         const int TabCentralSimulationIndex = 1;
00631         const int TabCentralReportsIndex = 2;
00632         //
00633         const int TabModelSimLangIndex = 0;
00634         const int TabModelCppCodeIndex = 1;
00635         const int TabModelDiagramIndex = 2;
00636         const int TabModelComponentsIndex = 3;
00637         //
00638         const int TabModelDataDefinitionsIndex = 4;
00639         const int TabSimulationBreakpointsIndex = 0;
00640         const int TabSimulationVariablesIndex = 1;
00641         const int TabSimulationEntitiesIndex = 2;
00642         const int TabSimulationTracesIndex = 3;
00643         const int TabSimulationEventsIndex = 4;
00644         //
00645         const int TabReportReportIndex = 0;
00646         const int TabReportResultIndex = 1;
00647         const int TabReportPlotIndex = 2;
00648     } CONST;
00649
00650     QUndoView *undoView = nullptr;
00651     bool _graphicalSimulation = false;
00652     bool _modelCheked = false;
00653     bool _loaded = false;
00654     bool _shuttingDown = false;
00655     // Persists lightweight tool settings introduced in Phase 5 dialogs.
00656     double _optimizerPrecision = 1e-6;
00657     unsigned int _optimizerMaxSteps = 1000;
00658     // Persists minimal parallel execution preferences configured in GUI.
00659     bool _parallelizationEnabled = false;
00660     int _parallelizationThreads = 1;
00661     int _parallelizationBatchSize = 100;
00662     // Remembers the last dataset location used by the data analyzer workflow.
00663     QString _lastDataAnalyzerPath;
00664     QMetaObject::Connection _sceneChangedConnection;
00665     QMetaObject::Connection _sceneFocusItemChangedConnection;
00666     QMetaObject::Connection _sceneSelectionChangedConnection;
00667     bool _isDeferredPropertyEditorModelChangedScheduled = false;
00668     //CodeEditor* textCodeEdit_Model;
00669 };
00670 #endif // MAINWINDOW_H

```

11.67 applications/gui/genesys/mainwindow_controller.cpp File Reference

Controller-delegation partition of [MainWindow](#) implementation.

```

#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "controllers/ModelInspectorController.h"
#include "controllers/PropertyEditorController.h"
#include "controllers/PluginCatalogController.h"
#include "controllers/ModelLifecycleController.h"
#include "controllers/SimulationCommandController.h"
#include "controllers/EditCommandController.h"
#include "controllers/SceneToolController.h"
#include "controllers/DialogUtilityController.h"
#include "dialogs/dialogBreakpoint.h"
#include "dialogs/Dialogmodelinformation.h"
#include "dialogs/dialogsimulationconfigure.h"
#include "dialogs/dialogpluginmanager.h"
#include "dialogs/dialogsystempreferences.h"
#include "dialogs/DialogFind.h"
#include "controllers/SimulationController.h"
#include "services/ToolLauncher.h"
#include <string>

```

```
#include <fstream>
#include <memory>
#include <algorithm>
#include <cmath>
#include <cstdlib>
#include <QCoreApplication>
#include <QMessageBox>
#include <QTextStream>
#include <QFileDialog>
#include <QGraphicsScene>
#include <QDateTime>
#include <QEventLoop>
#include <QTemporaryFile>
#include <Qt>
#include <QGraphicsPixmapItem>
#include <QPropertyAnimation>
#include <QtWidgets/qgraphicsitem.h>
#include <QScreen>
#include <QDebug>
#include <QRegularExpression>
#include <QRandomGenerator>
#include <QSignalBlocker>
#include <QDialog>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QFormLayout>
#include <QLineEdit>
#include <QPushButton>
#include <QDialogButtonBox>
#include <QLabel>
#include <QCheckBox>
#include <QSpinBox>
#include <QDoubleSpinBox>
#include <QPlainTextEdit>
#include <QTextEdit>
#include <QStatusBar>
#include <QImage>
#include <QPainter>
#include <QFileInfo>
#include "../kernel/simulator/model/ModelSimulation.h"
#include "../tools/Continuous/SolverDefaultImpl1.h"
```

11.67.1 Detailed Description

Controller-delegation partition of [MainWindow](#) implementation.

This file groups menu/toolbar/widget slot wrappers that delegate behavior to extracted controllers (for example lifecycle, simulation command, edit, scene tool, and dialog utility controllers). It does not define a new class; it is a thematic partition of [MainWindow](#).

11.68 applications/gui/genesys/mainwindow_modelrepresentations.cpp File Reference

[Model](#) representation/persistence wrapper partition of [MainWindow](#) implementation.


```

#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "controllers/ModelInspectorController.h"
#include "services/ModelLanguageSynchronizer.h"
#include "services/GraphvizModelExporter.h"
#include "services/CppModelExporter.h"
#include "services/GraphicalModelSerializer.h"
#include "services/GraphicalModelBuilder.h"
#include "kernel/simulator/SinkModelComponent.h"
#include <string>
#include <fstream>
#include <sstream>
#include <cstdlib>
#include <cctype>
#include <QMessageBox>
#include <QTextStream>
#include <QFileDialog>
#include <QDateTime>
#include <QEventLoop>
#include <QTemporaryFile>
#include <Qt>
#include <QGraphicsPixmapItem>
#include <QPropertyAnimation>
#include <QtWidgets/qgraphicsitem.h>
#include <QScreen>
#include <QDebug>
#include <QRegularExpression>
#include <QRandomGenerator>
#include <QScrollBar>
#include <QSignalBlocker>
#include <QTimer>
#include <QUrl>

```

Include dependency graph for `mainwindow_modelrepresentations.cpp`:



11.68.1 Detailed Description

[Model](#) representation/persistence wrapper partition of [MainWindow](#) implementation.

This file groups [MainWindow](#) wrappers related to textual synchronization, Graphviz and C++ exports, graphical persistence/serialization, and graphical model reconstruction through extracted services/controllers. It does not define a new class; it is a thematic partition of [MainWindow](#).

11.69 applications/gui/genesys/mainwindow_scene.cpp File Reference

Scene callback and scene synchronization partition of [MainWindow](#) implementation.

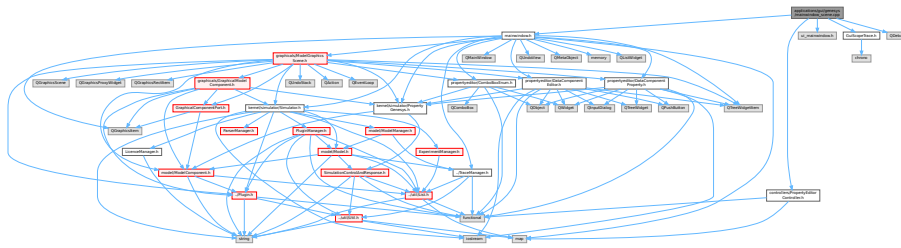
```

#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "controllers/PropertyEditorController.h"

```



```
#include "GuiScopeTrace.h"
#include <QDebug>
Include dependency graph for mainwindow_scene.cpp:
```



11.69.1 Detailed Description

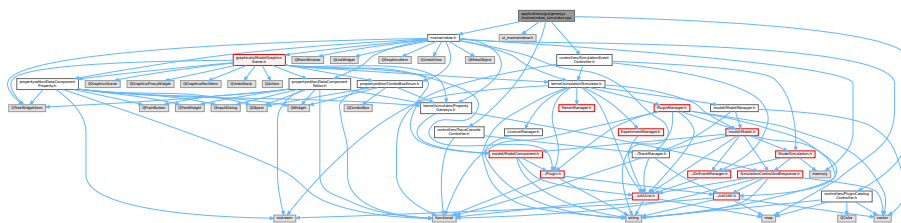
Scene callback and scene synchronization partition of [MainWindow](#) implementation.

This file concentrates scene-related callbacks, selection/focus/zoom bridging, and wrappers that synchronize scene state with UI elements such as actions and the property editor controller. It does not define a new class; it is a thematic partition of [MainWindow](#).

11.70 applications/gui/genesys/mainwindow_simulator.cpp File Reference

[Simulator](#) trace/event wrapper partition of [MainWindow](#) implementation.

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "controllers/TraceConsoleController.h"
#include "controllers/SimulationEventController.h"
#include "controllers/PluginCatalogController.h"
Include dependency graph for mainwindow_simulator.cpp:
```



11.70.1 Detailed Description

[Simulator](#) trace/event wrapper partition of [MainWindow](#) implementation.

This file centralizes [MainWindow](#) compatibility wrappers for simulator traces and simulation event callbacks, delegating rendering and reactions to [TraceConsoleController](#) and [SimulationEventController](#) (plus plugin-catalog related delegation where applicable). It does not define a new class; it is a thematic partition of [MainWindow](#).

11.71 MovedGuiCompatibilityIncludes.h

```

00001 #ifndef MOVED_GUI_COMPATIBILITY_INCLUDES_H
00002 #define MOVED_GUI_COMPATIBILITY_INCLUDES_H
00003
00004 // Temporary compatibility header for the physical GUI source-tree move.
00005 // Some legacy source files still contain multiple includes on one line after
00006 // the automated path adjustment used during the salvage move. The first include
00007 // on those lines is still seen by the preprocessor, but subsequent includes are
00008 // ignored as extra tokens. This header keeps the moved GUI buildable while those
00009 // source files are repaired incrementally.
00010
00011 #include "../kernel/simulator/Simulator.h"
00012 #include "../kernel/simulator/Plugin.h"
00013 #include "../kernel/simulator/PropertyGenesys.h"
00014 #include "../kernel/simulator/SimulationControlAndResponse.h"
00015 #include "../kernel/simulator/LicenceManager.h"
00016 #include "../kernel/simulator/SimulatorFacade.h"
00017
00018 #include "../kernel/simulator/model/Model.h"
00019 #include "../kernel/simulator/model/ModelManager.h"
00020 #include "../kernel/simulator/model/ModelSimulation.h"
00021 #include "../kernel/simulator/model/ModelComponent.h"
00022 #include "../kernel/simulator/model/ModelDataDefinition.h"
00023 #include "../kernel/simulator/model/ModelDataManager.h"
00024
00025 #include "../kernel/simulator/essentialPlugins/Attribute.h"
00026 #include "../kernel/simulator/essentialPlugins/Counter.h"
00027 #include "../kernel/simulator/essentialPlugins/Entity.h"
00028
00029 #include "../plugins/data/Logic/Variable.h"
00030 #include "../plugins/data/DiscreteProcessing/Queue.h"
00031
00032 #include "../tools/Statistics/FitterDefaultImpl.h"
00033 #include "../tools/Statistics/HypothesisTesterDefaultImpl.h"
00034 #include "../tools/Optimization/OptimizerDefaultImpl.h"
00035 #include "../tools/Continuous/SolverDefaultImpl.h"
00036
00037 #endif // MOVED_GUI_COMPATIBILITY_INCLUDES_H

```

11.72 ComboBoxEnum.h

```

00001 #ifndef COMBOBOXENUM_H
00002 #define COMBOBOXENUM_H
00003
00004 #include <functional>
00005
00006 #include <QObject>
00007 #include <QWidget>
00008 #include <QComboBox>
00009
00010 #include "kernel/simulator/PropertyGenesys.h"
00011
00012 class ComboBoxEnum : public QObject {
00013 public:
00014     using AfterChange = std::function<void()>;
00015
00016     ComboBoxEnum(
00017         PropertyEditorGenesys* editor,
00018         SimulationControl* property,
00019         AfterChange afterChange = {}
00020     );
00021
00022     ~ComboBoxEnum() override = default;
00023
00024 public:
00025     void open_box();
00026     void configure_options(SimulationControl* property);
00027
00028 private:
00029     Q_SLOTS:
00030     void changeValue();
00031
00032 private:
00033     QWidget* _window = nullptr;
00034     QComboBox* _comboBox = nullptr;
00035
00036     PropertyEditorGenesys* _editor = nullptr;
00037     SimulationControl* _property = nullptr;
00038     AfterChange _afterChange;
00039 };
00040 #endif // COMBOBOXENUM_H

```

11.73 DataComponentEditor.h

```

00001 #ifndef DATACOMPONENTEDITOR_H
00002 #define DATACOMPONENTEDITOR_H
00003
00004 #include <functional>
00005 #include <iostream>
00006
00007 #include <QObject>
00008 #include <QWidget>
00009 #include <QTreeWidget>
00010 #include <QTreeWidgetItem>
00011 #include <QPushButton>
00012 #include <QInputDialog>
00013
00014 #include "kernel/simulator/PropertyGenesys.h"
00015
00016 class DataComponentProperty;
00017 class ComboBoxEnum;
00018
00019 class DataComponentEditor : public QObject {
00020 public:
00021     using AfterChange = std::function<void()>;
00022
00023     DataComponentEditor(
00024         PropertyEditorGenesys* editor,
00025         SimulationControl* property,
00026         AfterChange afterChange = {}
00027     );
00028
00029     DataComponentEditor(
00030         PropertyEditorGenesys* editor,
00031         List<SimulationControl*>* properties,
00032         AfterChange afterChange = {}
00033     );
00034
00035     ~DataComponentEditor() override = default;
00036
00037 public:
00038     void open_window(SimulationControl* property);
00039     void open_window(List<SimulationControl*>* properties);
00040
00041     void configure_properties(SimulationControl* property);
00042     void configure_properties(List<SimulationControl*>* properties);
00043
00044 private:
00045     void _notifyChanged();
00046
00047     void editProperty(SimulationControl* property);
00048     void editProperty(List<SimulationControl*>* properties);
00049
00050 private:
00051     QWidget* _window = nullptr;
00052     QTreeWidget* _view = nullptr;
00053     QPushButton* _edit = nullptr;
00054     QInputDialog* _newValue = nullptr;
00055
00056     PropertyEditorGenesys* _editor = nullptr;
00057     AfterChange _afterChange;
00058 };
00059
00060 #endif // DATACOMPONENTEDITOR_H

```

11.74 DataComponentProperty.h

```

00001 #ifndef DATACOMPONENTPROPERTY_H
00002 #define DATACOMPONENTPROPERTY_H
00003
00004 #include <functional>
00005 #include <iostream>
00006
00007 #include <QObject>
00008 #include <QWidget>
00009 #include <QTreeWidget>
00010 #include <QTreeWidgetItem>
00011 #include <QPushButton>
00012 #include <QInputDialog>
00013
00014 #include "kernel/simulator/PropertyGenesys.h"
00015
00016 class DataComponentEditor;
00017 class ComboBoxEnum;
00018

```

```

00019 class DataComponentProperty : public QObject {
00020 public:
00021     using AfterChange = std::function<void()>;
00022
00023     DataComponentProperty(
00024         PropertyEditorGenesys* editor,
00025         SimulationControl* property,
00026         bool necessaryConfig,
00027         AfterChange afterChange = {}
00028     );
00029
00030     ~DataComponentProperty() override = default;
00031
00032 public:
00033     void open_window();
00034     void config_values();
00035     bool isInList(const std::string& value) const;
00036
00037 private:
00038     void _notifyChanged();
00039
00040 private Q_SLOTS:
00041     void addElement();
00042     void removeElement();
00043     void editProperty();
00044
00045 private:
00046     QWidget* _window = nullptr;
00047     QTreeWidget* _view = nullptr;
00048     QPushButton* _add = nullptr;
00049     QPushButton* _remove = nullptr;
00050     QPushButton* _edit = nullptr;
00051     QInputDialog* _confirmation = nullptr;
00052
00053     PropertyEditorGenesys* _editor = nullptr;
00054     SimulationControl* _property = nullptr;
00055     AfterChange _afterChange;
00056 };
00057
00058 #endif // DATACOMPONENTPROPERTY_H

```

11.75 ObjectPropertyBrowser.h

```

00001 #ifndef OBJECTPROPERTYBROWSER_H
00002 #define OBJECTPROPERTYBROWSER_H
00003
00004 #include <map>
00005 #include <vector>
00006 #include <set>
00007 #include <functional>
00008
00009 #include <QObject>
00010 #include <QBrush>
00011 #include <QPen>
00012 #include <QPointer>
00013 #include <QMap>
00014 #include <QSet>
00015 #include <QStringList>
00016 #include <QContextMenuEvent>
00017 #include <QGraphicsItem>
00018 #include <QKeyEvent>
00019 #include <QMouseEvent>
00020 #include <QVariant>
00021
00022 #include "qtpropertybrowser/qttreepropertybrowser.h"
00023 #include "qtpropertybrowser/qtvariantproperty.h"
00024 #include "qtpropertybrowser/qtpropertymanager.h"
00025 #include "qtpropertybrowser/qteditorfactory.h"
00026
00027 #include "DataComponentProperty.h"
00028 #include "DataComponentEditor.h"
00029 #include "ComboBoxEnum.h"
00030
00031 #include "../kernel/simulator/model/ModelDataDefinition.h"
00032 #include "kernel/simulator/PropertyGenesys.h"
00033 #include "kernel/simulator/GenesysPropertyIntrospection.h"
00034
00035 class ObjectPropertyBrowser : public QtTreePropertyBrowser {
00036     Q_OBJECT
00037
00038 public:
00039     explicit ObjectPropertyBrowser(QWidget* parent = nullptr);
00040     using ModelChangedCallback = std::function<void()>;

```

```

00041
00042     void setActiveObject(
00043         QObject *obj,
00044         ModelDataDefinition* mdd = nullptr,
00045         const QSet<QString>& graphicallyRepresentedModelObjects = {},
00046         const QSet<QString>& editableModelObjects = {},
00047         PropertyEditorGenesys* peg = nullptr,
00048         std::map<SimulationControl*, DataComponentProperty*>* pl = nullptr,
00049         std::map<SimulationControl*, DataComponentEditor*>* peUI = nullptr,
00050         std::map<SimulationControl*, ComboBoxEnum*>* pb = nullptr
00051     );
00052     void setActiveGraphicsItem(QGraphicsItem* item);
00053
00054     void clearCurrentlyConnectedObject();
00055     void setModelChangedCallback(ModelChangedCallback callback);
00056     bool isCommitPipelineBusy() const;
00057
00058 private:
00059     enum class ActiveMode {
00060         None,
00061         KernelObject,
00062         GraphicsItem
00063     };
00064
00065     enum class BindingKind {
00066         Kernel,
00067         GraphicsVariant,
00068         GraphicsEnum
00069     };
00070
00071     struct Binding {
00072         BindingKind kind = BindingKind::Kernel;
00073         ModelDataDefinition* owner = nullptr;
00074         SimulationControl* control = nullptr;
00075         GenesysPropertyDescriptor descriptor;
00076         bool isObjectSelector = false;
00077         bool isModelObjectAction = false;
00078         bool isObjectListAction = false;
00079         bool isMultilineTextAction = false;
00080         QGraphicsItem* graphicsItem = nullptr;
00081         bool graphicsRequiresCommit = false;
00082         std::function<void(const QVariant&)> graphicsVariantSetter;
00083         std::function<void(int)> graphicsEnumSetter;
00084     };
00085
00086     struct ModelObjectRelation {
00087         bool exists = false;
00088         bool internal = false;
00089         std::string key;
00090         ModelDataDefinition* dataDefinition = nullptr;
00091     };
00092
00093     struct ObjectListAction {
00094         enum class Kind {
00095             None,
00096             CreateNew,
00097             SetCurrentType
00098         };
00099
00100         Kind kind = Kind::None;
00101         std::string typeName;
00102     };
00103
00104     struct ModelObjectAction {
00105         enum class Kind {
00106             None,
00107             SetReference,
00108             CreateNew,
00109             SelectAndEdit,
00110             RemoveReference
00111         };
00112
00113         Kind kind = Kind::None;
00114         std::string objectValue;
00115         std::string typeName;
00116     };
00117
00118 private:
00119     // Ensure the property browser infrastructure is created exactly once.
00120     void _ensureBrowserInfrastructure();
00121     void _notifyModelChangeApplied();
00122     void _clearAll();
00123     void _rebuildProperties();
00124     void _populateKernelProperties(ModelDataDefinition* mdd);
00125     void _populateGraphicsProperties(QGraphicsItem* item);
00126     void _appendDescriptorRecursively(
00127         QtProperty* parent,

```

```

00128         SimulationControl* control,
00129         std::set<const SimulationControl*>& recursionPath,
00130         int depth = 0
00131     );
00132
00133     QtProperty* _createLeafProperty(const GenesysPropertyDescriptor& desc);
00134     QtProperty* _createObjectListProperty(
00135         SimulationControl* control,
00136         const GenesysPropertyDescriptor& desc,
00137         std::set<const SimulationControl*>& recursionPath,
00138         int depth
00139     );
00140     QtProperty* _createModelObjectActionProperty(const GenesysPropertyDescriptor& desc);
00141     bool _isObjectListProperty(const GenesysPropertyDescriptor& desc) const;
00142     bool _isKernelEditingEnabled(const GenesysPropertyDescriptor& desc) const;
00143     bool _hasGraphicalRepresentation(const GenesysPropertyDescriptor& desc) const;
00144     bool _isModelObjectActionProperty(const GenesysPropertyDescriptor& desc, SimulationControl*
control) const;
00145     ModelObjectRelation _relationForDataDefinition(ModelDataDefinition* dataDefinition) const;
00146     ModelDataDefinition* _referencedModelDataDefinition(const Binding& binding) const;
00147     QString _modelObjectTypeName(const GenesysPropertyDescriptor& desc) const;
00148     QString _defaultModelObjectName(const GenesysPropertyDescriptor& desc, const std::string&
concreteTypeName = "") const;
00149     bool _isRegisteredModelDataDefinition(ModelDataDefinition* dataDefinition) const;
00150     void _materializeAffectedModelDataDefinitions(ModelDataDefinition* referencedDataDefinition =
nullptr) const;
00151     bool _createNewListElementForProperty(QtProperty* property, const std::string& typeName = "");
00152     bool _setCurrentListElementTypeForProperty(QtProperty* property, const std::string& typeName);
00153     bool _createModelObjectForProperty(QtProperty* property, const std::string& typeName = "");
00154     bool _setModelObjectReferenceForProperty(QtProperty* property, const QString& objectName);
00155     bool _selectModelObjectForProperty(QtProperty* property);
00156     bool _removeModelObjectReferenceForProperty(QtProperty* property);
00157     bool _deleteModelObjectForProperty(QtProperty* property);
00158     bool _applyObjectListSelection(QtProperty* property, int value);
00159     bool _applyModelObjectReferenceSelection(QtProperty* property, int value);
00160     void _synchronizeGraphicalModelDataDefinitionsNow() const;
00161     QtVariantProperty* _createGraphicsVariantProperty(
00162         const QString& name,
00163         int variantType,
00164         const QVariant& value,
00165         std::function<void(const QVariant&)> setter,
00166         bool requiresCommit = false,
00167         bool enabled = true
00168     );
00169     QtProperty* _createGraphicsEnumProperty(
00170         const QString& name,
00171         const QStringList& choices,
00172         int value,
00173         std::function<void(int)> setter,
00174         bool enabled = true
00175     );
00176     void _appendCommonGraphicsProperties(QtProperty* group, QGraphicsItem* item);
00177     void _appendPenProperties(QtProperty* group, const QPen& pen, std::function<void(const QPen&)>
setter);
00178     void _appendBrushProperties(QtProperty* group, const QBrush& brush, std::function<void(const
QBrush&)> setter);
00179     void _notifyGraphicsChangeApplied(QGraphicsItem* item);
00180     QVariant _toVariant(const GenesysPropertyDescriptor& desc) const;
00181     std::string _fromVariant(const GenesysPropertyDescriptor& desc, const QVariant& value) const;
00182     int _enumIndexFor(const GenesysPropertyDescriptor& desc) const;
00183     QStringList _toQStringList(const std::vector<std::string>& values) const;
00184     bool _hasSpecializedEditor(const GenesysPropertyDescriptor& desc) const;
00185     QString _specializedEditorActionText(const GenesysPropertyDescriptor& desc) const;
00186
00187     bool _openSpecializedEditorForCurrentItem();
00188     bool _openSpecializedEditor(QtProperty* property);
00189     bool _openTextDialogEditor(const Binding& binding, bool compactEditor);
00190     bool _openMultilineTextDialogEditor(const Binding& binding);
00191     bool _createObjectForProperty(QtProperty* property);
00192
00193 protected:
00194     void keyPressEvent(QKeyEvent* event) override;
00195     void contextMenuEvent(QContextMenuEvent* event) override;
00196     void mouseDoubleClickEvent(QMouseEvent* event) override;
00197
00198 private:
00199     // Guard against reentrant callback chains during property rebuild and notification.
00200     bool _isRebuildingProperties = false;
00201     // Guard against nested model-change notifications triggered by editor callbacks.
00202     bool _isNotifyingModelChange = false;
00203     // Keep track of pending rebuild requests raised during guarded execution.
00204     bool _pendingRebuild = false;
00205     // Coalesce queued rebuild dispatches while one deferred rebuild request is already scheduled.
00206     bool _isDeferredRebuildScheduled = false;
00207     // Coalesce queued model-change callbacks while one deferred dispatch is already scheduled.
00208     bool _isDeferredModelChangedScheduled = false;
00209     // Prevent recursive handling when a multiline text action resets its combo-box selection.

```

```

00210     bool _isHandlingMultilineTextAction = false;
00211
00212 private:
00213     // Execute property rebuild with reentrancy suppression and deferred retry support.
00214     void _rebuildPropertiesGuarded();
00215     // Request a deferred rebuild after current signal/slot stack finishes.
00216     void _scheduleDeferredRebuild();
00217     // Request a deferred model-changed callback after current signal/slot stack finishes.
00218     void _scheduleDeferredModelChangedCallback();
00219     // Check whether active editor bindings are currently valid before mutating model state.
00220     bool _hasValidActiveBindingContext(QtProperty* property = nullptr) const;
00221
00222 private:
00223     // Track the selected graphical object safely in case Qt destroys it during callbacks.
00224     QPointer<QObject> _graphicalObject = nullptr;
00225     QGraphicsItem* _graphicalItem = nullptr;
00226     ModelDataDefinition* _modelObject = nullptr;
00227     ActiveMode _activeMode = ActiveMode::None;
00228     QSet<QString> _graphicallyRepresentedModelObjects;
00229     QSet<QString> _editableModelObjects;
00230     bool _activeKernelObjectReadOnly = false;
00231     PropertyEditorGenesys* _propertyEditor = nullptr;
00232
00233     std::map<SimulationControl*, DataComponentProperty*> _propertyList = nullptr;
00234     std::map<SimulationControl*, DataComponentEditor*> _propertyEditorUI = nullptr;
00235     std::map<SimulationControl*, ComboBoxEnum*> _propertyBox = nullptr;
00236
00237     QtVariantPropertyManager* _variantManager = nullptr;
00238     QtGroupPropertyManager* _groupManager = nullptr;
00239     QtEnumPropertyManager* _enumManager = nullptr;
00240
00241     QtVariantEditorFactory* _variantFactory = nullptr;
00242     QtEnumEditorFactory* _enumFactory = nullptr;
00243     // Track whether factories were already bound to managers in the browser.
00244     bool _browserInfrastructureBound = false;
00245
00246     QMap<QtProperty*, Binding> _bindings;
00247     QMap<QtProperty*, QStringList> _enumNames;
00248     QMap<QtProperty*, std::vector<ObjectListAction>> _objectListActions;
00249     QMap<QtProperty*, std::vector<ModelObjectAction>> _modelObjectActions;
00250     QSet<QtProperty*> _pendingCommittedProperties;
00251     QMap<QtProperty*, QVariant> _pendingCommittedValues;
00252     ModelChangedCallback _modelChangedCallback;
00253
00254 private:
00255     bool _requiresCommitConfirmation(const Binding& binding) const;
00256     bool _applyVariantChange(QtProperty* property, const QVariant& value, bool committed);
00257     bool _applyGraphicsVariantChange(QtProperty* property, const QVariant& value, bool committed);
00258     bool _applyGraphicsEnumChange(QtProperty* property, int value);
00259
00260 private slots:
00261     void valueChanged(QtProperty* property, const QVariant& value);
00262     void enumValueChanged(QtProperty* property, int value);
00263     void onVariantEditorCommitted(QtProperty* property, const QVariant& committedValue = QVariant());
00264
00265 public slots:
00266     void objectUpdated();
00267 };
00268
00269 #endif // OBJECTPROPERTYBROWSER_H

```

11.76 PropertyEditor.h

```

00001 #ifndef PROPERTYEDITOR_H
00002 #define PROPERTYEDITOR_H
00003
00004 #include <QTreeWidget>
00005 #include <QObject>
00006
00007 class ModelDataDefinition;
00008
00009 class PropertyEditor : public QTreeWidget {
00010 public:
00011     PropertyEditor(QWidget* parent = nullptr);
00012 public:
00013     void setModelBlock(ModelDataDefinition* modelblock);
00014 };
00015
00016 #endif // PROPERTYEDITOR_H

```

11.77 qtbuttonpropertybrowser.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public license version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 #ifndef QTBUTTONPROPERTYBROWSER_H
00041 #define QTBUTTONPROPERTYBROWSER_H
00042
00043 #include "qtpropertybrowser.h"
00044
00045 QT_BEGIN_NAMESPACE
00046
00047 class QtButtonPropertyBrowserPrivate;
00048
00049 class QT_QTPROPERTYBROWSER_EXPORT QtButtonPropertyBrowser : public QtAbstractPropertyBrowser
00050 {
00051     Q_OBJECT
00052 public:
00053     QtButtonPropertyBrowser(QWidget *parent = 0);
00054     ~QtButtonPropertyBrowser();
00055     void setExpanded(QtBrowserItem *item, bool expanded);
00056     bool isExpanded(QtBrowserItem *item) const;
00057
00058     Q_SIGNALS:
00059
00060     void collapsed(QtBrowserItem *item);
00061     void expanded(QtBrowserItem *item);
00062
00063 protected:
00064     void itemInserted(QtBrowserItem *item, QtBrowserItem *afterItem) override;
00065     void itemRemoved(QtBrowserItem *item) override;
00066     void itemChanged(QtBrowserItem *item) override;
00067
00068 private:
00069     QScopedPointer<QtButtonPropertyBrowserPrivate> d_ptr;
00070     Q_DECLARE_PRIVATE(QtButtonPropertyBrowser)
00071     Q_DISABLE_COPY_MOVE(QtButtonPropertyBrowser)
00072     Q_PRIVATE_SLOT(d_func(), void slotUpdate())
00073     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed())
00074     Q_PRIVATE_SLOT(d_func(), void slotToggled(bool))
00075 };
00076
00077 QT_END_NAMESPACE
00078
00079 #endif

```


11.78 qteditorfactory.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public license version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 #ifndef QTEDITORFACTORY_H
00041 #define QTEDITORFACTORY_H
00042
00043 #include "qtpropertymanager.h"
00044
00045 QT_BEGIN_NAMESPACE
00046
00047 class QRegularExpression;
00048
00049 class QtSpinBoxFactoryPrivate;
00050
00051 class QT_QTPROPERTYBROWSER_EXPORT QtSpinBoxFactory : public
QtAbstractEditorFactory<QtIntPropertyManager>
00052 {
00053     Q_OBJECT
00054 public:
00055     QtSpinBoxFactory(QObject *parent = 0);
00056     ~QtSpinBoxFactory();
00057 protected:
00058     void connectPropertyManager(QtIntPropertyManager *manager) override;
00059     QWidget *createEditor(QtIntPropertyManager *manager, QtProperty *property,
00060         QWidget *parent) override;
00061     void disconnectPropertyManager(QtIntPropertyManager *manager) override;
00062 private:
00063     QScopedPointer<QtSpinBoxFactoryPrivate> d_ptr;
00064     Q_DECLARE_PRIVATE(QtSpinBoxFactory)
00065     Q_DISABLE_COPY_MOVE(QtSpinBoxFactory)
00066     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, int))
00067     Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, int, int))
00068     Q_PRIVATE_SLOT(d_func(), void slotSingleStepChanged(QtProperty *, int))
00069     Q_PRIVATE_SLOT(d_func(), void slotSetValue(int))
00070     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00071 };
00072
00073 class QtSliderFactoryPrivate;
00074
00075 class QT_QTPROPERTYBROWSER_EXPORT QtSliderFactory : public
QtAbstractEditorFactory<QtIntPropertyManager>
00076 {
00077     Q_OBJECT
00078 public:
00079     QtSliderFactory(QObject *parent = 0);
00080     ~QtSliderFactory();
00081 protected:
00082     void connectPropertyManager(QtIntPropertyManager *manager) override;
00083     QWidget *createEditor(QtIntPropertyManager *manager, QtProperty *property,

```

```

00084         QWidget *parent) override;
00085     void disconnectPropertyManager(QtIntPropertyManager *manager) override;
00086 private:
00087     QScopedPointer<QtSliderFactoryPrivate> d_ptr;
00088     Q_DECLARE_PRIVATE(QtSliderFactory)
00089     Q_DISABLE_COPY_MOVE(QtSliderFactory)
00090     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, int))
00091     Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, int, int))
00092     Q_PRIVATE_SLOT(d_func(), void slotSingleStepChanged(QtProperty *, int))
00093     Q_PRIVATE_SLOT(d_func(), void slotSetValue(int))
00094     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00095 };
00096
00097 class QtScrollBarFactoryPrivate;
00098
00099 class QT_QTPROPERTYBROWSER_EXPORT QtScrollBarFactory : public
QtAbstractEditorFactory<QtIntPropertyManager>
00100 {
00101     Q_OBJECT
00102 public:
00103     QtScrollBarFactory(QObject *parent = 0);
00104     ~QtScrollBarFactory();
00105 protected:
00106     void connectPropertyManager(QtIntPropertyManager *manager) override;
00107     QWidget *createEditor(QtIntPropertyManager *manager, QtProperty *property,
00108         QWidget *parent) override;
00109     void disconnectPropertyManager(QtIntPropertyManager *manager) override;
00110 private:
00111     QScopedPointer<QtScrollBarFactoryPrivate> d_ptr;
00112     Q_DECLARE_PRIVATE(QtScrollBarFactory)
00113     Q_DISABLE_COPY_MOVE(QtScrollBarFactory)
00114     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, int))
00115     Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, int, int))
00116     Q_PRIVATE_SLOT(d_func(), void slotSingleStepChanged(QtProperty *, int))
00117     Q_PRIVATE_SLOT(d_func(), void slotSetValue(int))
00118     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00119 };
00120
00121 class QtCheckBoxFactoryPrivate;
00122
00123 class QT_QTPROPERTYBROWSER_EXPORT QtCheckBoxFactory : public
QtAbstractEditorFactory<QtBoolPropertyManager>
00124 {
00125     Q_OBJECT
00126 public:
00127     QtCheckBoxFactory(QObject *parent = 0);
00128     ~QtCheckBoxFactory();
00129 protected:
00130     void connectPropertyManager(QtBoolPropertyManager *manager) override;
00131     QWidget *createEditor(QtBoolPropertyManager *manager, QtProperty *property,
00132         QWidget *parent) override;
00133     void disconnectPropertyManager(QtBoolPropertyManager *manager) override;
00134 private:
00135     QScopedPointer<QtCheckBoxFactoryPrivate> d_ptr;
00136     Q_DECLARE_PRIVATE(QtCheckBoxFactory)
00137     Q_DISABLE_COPY_MOVE(QtCheckBoxFactory)
00138     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, bool))
00139     Q_PRIVATE_SLOT(d_func(), void slotSetValue(bool))
00140     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00141 };
00142
00143 class QtDoubleSpinBoxFactoryPrivate;
00144
00145 class QT_QTPROPERTYBROWSER_EXPORT QtDoubleSpinBoxFactory : public
QtAbstractEditorFactory<QtDoublePropertyManager>
00146 {
00147     Q_OBJECT
00148 public:
00149     QtDoubleSpinBoxFactory(QObject *parent = 0);
00150     ~QtDoubleSpinBoxFactory();
00151 protected:
00152     void connectPropertyManager(QtDoublePropertyManager *manager) override;
00153     QWidget *createEditor(QtDoublePropertyManager *manager, QtProperty *property,
00154         QWidget *parent) override;
00155     void disconnectPropertyManager(QtDoublePropertyManager *manager) override;
00156 private:
00157     QScopedPointer<QtDoubleSpinBoxFactoryPrivate> d_ptr;
00158     Q_DECLARE_PRIVATE(QtDoubleSpinBoxFactory)
00159     Q_DISABLE_COPY_MOVE(QtDoubleSpinBoxFactory)
00160     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, double))
00161     Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, double, double))
00162     Q_PRIVATE_SLOT(d_func(), void slotSingleStepChanged(QtProperty *, double))
00163     Q_PRIVATE_SLOT(d_func(), void slotDecimalsChanged(QtProperty *, int))
00164     Q_PRIVATE_SLOT(d_func(), void slotSetValue(double))
00165     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00166 };
00167

```

```

00168 class QtLineEditFactoryPrivate;
00169
00170 class QT_QTPROPERTYBROWSER_EXPORT QtLineEditFactory : public
    QtAbstractEditorFactory<QtStringPropertyManager>
00171 {
00172     Q_OBJECT
00173 public:
00174     QtLineEditFactory(QObject *parent = 0);
00175     ~QtLineEditFactory();
00176 protected:
00177     void connectPropertyManager(QtStringPropertyManager *manager) override;
00178     QWidget *createEditor(QtStringPropertyManager *manager, QtProperty *property,
00179         QWidget *parent) override;
00180     void disconnectPropertyManager(QtStringPropertyManager *manager) override;
00181 private:
00182     QScopedPointer<QtLineEditFactoryPrivate> d_ptr;
00183     Q_DECLARE_PRIVATE(QtLineEditFactory)
00184     Q_DISABLE_COPY_MOVE(QtLineEditFactory)
00185     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, const QString &))
00186     Q_PRIVATE_SLOT(d_func(), void slotRegExpChanged(QtProperty *, const QRegularExpression &))
00187     Q_PRIVATE_SLOT(d_func(), void slotSetValues(const QStringList &))
00188     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00189 };
00190
00191 class QtDateEditFactoryPrivate;
00192
00193 class QT_QTPROPERTYBROWSER_EXPORT QtDateEditFactory : public
    QtAbstractEditorFactory<QtDatePropertyManager>
00194 {
00195     Q_OBJECT
00196 public:
00197     QtDateEditFactory(QObject *parent = 0);
00198     ~QtDateEditFactory();
00199 protected:
00200     void connectPropertyManager(QtDatePropertyManager *manager) override;
00201     QWidget *createEditor(QtDatePropertyManager *manager, QtProperty *property,
00202         QWidget *parent) override;
00203     void disconnectPropertyManager(QtDatePropertyManager *manager) override;
00204 private:
00205     QScopedPointer<QtDateEditFactoryPrivate> d_ptr;
00206     Q_DECLARE_PRIVATE(QtDateEditFactory)
00207     Q_DISABLE_COPY_MOVE(QtDateEditFactory)
00208     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, QDate))
00209     Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, QDate, QDate))
00210     Q_PRIVATE_SLOT(d_func(), void slotSetValues(QDate))
00211     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00212 };
00213
00214 class QtTimeEditFactoryPrivate;
00215
00216 class QT_QTPROPERTYBROWSER_EXPORT QtTimeEditFactory : public
    QtAbstractEditorFactory<QtTimePropertyManager>
00217 {
00218     Q_OBJECT
00219 public:
00220     QtTimeEditFactory(QObject *parent = 0);
00221     ~QtTimeEditFactory();
00222 protected:
00223     void connectPropertyManager(QtTimePropertyManager *manager) override;
00224     QWidget *createEditor(QtTimePropertyManager *manager, QtProperty *property,
00225         QWidget *parent) override;
00226     void disconnectPropertyManager(QtTimePropertyManager *manager) override;
00227 private:
00228     QScopedPointer<QtTimeEditFactoryPrivate> d_ptr;
00229     Q_DECLARE_PRIVATE(QtTimeEditFactory)
00230     Q_DISABLE_COPY_MOVE(QtTimeEditFactory)
00231     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, QTime))
00232     Q_PRIVATE_SLOT(d_func(), void slotSetValues(QTime))
00233     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00234 };
00235
00236 class QtDateTimeEditFactoryPrivate;
00237
00238 class QT_QTPROPERTYBROWSER_EXPORT QtDateTimeEditFactory : public
    QtAbstractEditorFactory<QtDateTimePropertyManager>
00239 {
00240     Q_OBJECT
00241 public:
00242     QtDateTimeEditFactory(QObject *parent = 0);
00243     ~QtDateTimeEditFactory();
00244 protected:
00245     void connectPropertyManager(QtDateTimePropertyManager *manager) override;
00246     QWidget *createEditor(QtDateTimePropertyManager *manager, QtProperty *property,
00247         QWidget *parent) override;
00248     void disconnectPropertyManager(QtDateTimePropertyManager *manager) override;
00249 private:
00250     QScopedPointer<QtDateTimeEditFactoryPrivate> d_ptr;

```

```

00251     Q_DECLARE_PRIVATE(QtDateTimeEditFactory)
00252     Q_DISABLE_COPY_MOVE(QtDateTimeEditFactory)
00253     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, const QDateTime &))
00254     Q_PRIVATE_SLOT(d_func(), void slotSetValue(const QDateTime &))
00255     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00256 };
00257
00258 class QtKeySequenceEditorFactoryPrivate;
00259
00260 class QT_QTPROPERTYBROWSER_EXPORT QtKeySequenceEditorFactory : public
    QtAbstractEditorFactory<QtKeySequencePropertyManager>
00261 {
00262     Q_OBJECT
00263 public:
00264     QtKeySequenceEditorFactory(QObject *parent = 0);
00265     ~QtKeySequenceEditorFactory();
00266 protected:
00267     void connectPropertyManager(QtKeySequencePropertyManager *manager) override;
00268     QWidget *createEditor(QtKeySequencePropertyManager *manager, QtProperty *property,
00269         QWidget *parent) override;
00270     void disconnectPropertyManager(QtKeySequencePropertyManager *manager) override;
00271 private:
00272     QScopedPointer<QtKeySequenceEditorFactoryPrivate> d_ptr;
00273     Q_DECLARE_PRIVATE(QtKeySequenceEditorFactory)
00274     Q_DISABLE_COPY_MOVE(QtKeySequenceEditorFactory)
00275     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, const QKeySequence &))
00276     Q_PRIVATE_SLOT(d_func(), void slotSetValue(const QKeySequence &))
00277     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00278 };
00279
00280 class QtCharEditorFactoryPrivate;
00281
00282 class QT_QTPROPERTYBROWSER_EXPORT QtCharEditorFactory : public
    QtAbstractEditorFactory<QtCharPropertyManager>
00283 {
00284     Q_OBJECT
00285 public:
00286     QtCharEditorFactory(QObject *parent = 0);
00287     ~QtCharEditorFactory();
00288 protected:
00289     void connectPropertyManager(QtCharPropertyManager *manager) override;
00290     QWidget *createEditor(QtCharPropertyManager *manager, QtProperty *property,
00291         QWidget *parent) override;
00292     void disconnectPropertyManager(QtCharPropertyManager *manager) override;
00293 private:
00294     QScopedPointer<QtCharEditorFactoryPrivate> d_ptr;
00295     Q_DECLARE_PRIVATE(QtCharEditorFactory)
00296     Q_DISABLE_COPY_MOVE(QtCharEditorFactory)
00297     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, const QChar &))
00298     Q_PRIVATE_SLOT(d_func(), void slotSetValue(const QChar &))
00299     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00300 };
00301
00302 class QtEnumEditorFactoryPrivate;
00303
00304 class QT_QTPROPERTYBROWSER_EXPORT QtEnumEditorFactory : public
    QtAbstractEditorFactory<QtEnumPropertyManager>
00305 {
00306     Q_OBJECT
00307 public:
00308     QtEnumEditorFactory(QObject *parent = 0);
00309     ~QtEnumEditorFactory();
00310 protected:
00311     void connectPropertyManager(QtEnumPropertyManager *manager) override;
00312     QWidget *createEditor(QtEnumPropertyManager *manager, QtProperty *property,
00313         QWidget *parent) override;
00314     void disconnectPropertyManager(QtEnumPropertyManager *manager) override;
00315 private:
00316     QScopedPointer<QtEnumEditorFactoryPrivate> d_ptr;
00317     Q_DECLARE_PRIVATE(QtEnumEditorFactory)
00318     Q_DISABLE_COPY_MOVE(QtEnumEditorFactory)
00319     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, int))
00320     Q_PRIVATE_SLOT(d_func(), void slotEnumNamesChanged(QtProperty *,
00321         const QStringList &))
00322     Q_PRIVATE_SLOT(d_func(), void slotEnumIconsChanged(QtProperty *,
00323         const QMap<int, QIcon> &))
00324     Q_PRIVATE_SLOT(d_func(), void slotSetValue(int))
00325     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00326 };
00327
00328 class QtCursorEditorFactoryPrivate;
00329
00330 class QT_QTPROPERTYBROWSER_EXPORT QtCursorEditorFactory : public
    QtAbstractEditorFactory<QtCursorPropertyManager>
00331 {
00332     Q_OBJECT
00333 public:

```

```

00334     QtCursorEditorFactory(QObject *parent = 0);
00335     ~QtCursorEditorFactory();
00336 protected:
00337     void connectPropertyManager(QtCursorPropertyManager *manager) override;
00338     QWidget *createEditor(QtCursorPropertyManager *manager, QtProperty *property,
00339         QWidget *parent) override;
00340     void disconnectPropertyManager(QtCursorPropertyManager *manager) override;
00341 private:
00342     QScopedPointer<QtCursorEditorFactoryPrivate> d_ptr;
00343     Q_DECLARE_PRIVATE(QtCursorEditorFactory)
00344     Q_DISABLE_COPY_MOVE(QtCursorEditorFactory)
00345     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, const QCursor &))
00346     Q_PRIVATE_SLOT(d_func(), void slotEnumChanged(QtProperty *, int))
00347     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00348 };
00349
00350 class QtColorEditorFactoryPrivate;
00351
00352 class QT_QTPROPERTYBROWSER_EXPORT QtColorEditorFactory : public
    QtAbstractEditorFactory<QtColorPropertyManager>
00353 {
00354     Q_OBJECT
00355 public:
00356     QtColorEditorFactory(QObject *parent = 0);
00357     ~QtColorEditorFactory();
00358 protected:
00359     void connectPropertyManager(QtColorPropertyManager *manager) override;
00360     QWidget *createEditor(QtColorPropertyManager *manager, QtProperty *property,
00361         QWidget *parent) override;
00362     void disconnectPropertyManager(QtColorPropertyManager *manager) override;
00363 private:
00364     QScopedPointer<QtColorEditorFactoryPrivate> d_ptr;
00365     Q_DECLARE_PRIVATE(QtColorEditorFactory)
00366     Q_DISABLE_COPY_MOVE(QtColorEditorFactory)
00367     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, const QColor &))
00368     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00369     Q_PRIVATE_SLOT(d_func(), void slotSetValue(const QColor &))
00370 };
00371
00372 class QtFontEditorFactoryPrivate;
00373
00374 class QT_QTPROPERTYBROWSER_EXPORT QtFontEditorFactory : public
    QtAbstractEditorFactory<QtFontPropertyManager>
00375 {
00376     Q_OBJECT
00377 public:
00378     QtFontEditorFactory(QObject *parent = 0);
00379     ~QtFontEditorFactory();
00380 protected:
00381     void connectPropertyManager(QtFontPropertyManager *manager) override;
00382     QWidget *createEditor(QtFontPropertyManager *manager, QtProperty *property,
00383         QWidget *parent) override;
00384     void disconnectPropertyManager(QtFontPropertyManager *manager) override;
00385 private:
00386     QScopedPointer<QtFontEditorFactoryPrivate> d_ptr;
00387     Q_DECLARE_PRIVATE(QtFontEditorFactory)
00388     Q_DISABLE_COPY_MOVE(QtFontEditorFactory)
00389     Q_PRIVATE_SLOT(d_func(), void slotPropertyChanged(QtProperty *, const QFont &))
00390     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed(QObject *))
00391     Q_PRIVATE_SLOT(d_func(), void slotSetValue(const QFont &))
00392 };
00393
00394 QT_END_NAMESPACE
00395
00396 #endif

```

11.79 qtgroupboxpropertybrowser.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPLS
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.

```

```

00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public license version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 #ifndef QTGROUPBOXPROPERTYBROWSER_H
00041 #define QTGROUPBOXPROPERTYBROWSER_H
00042
00043 #include "qtpropertybrowser.h"
00044
00045 QT_BEGIN_NAMESPACE
00046
00047 class QtGroupBoxPropertyBrowserPrivate;
00048
00049 class QT_QTPROPERTYBROWSER_EXPORT QtGroupBoxPropertyBrowser : public QtAbstractPropertyBrowser
00050 {
00051     Q_OBJECT
00052 public:
00053
00054     QtGroupBoxPropertyBrowser(QWidget *parent = 0);
00055     ~QtGroupBoxPropertyBrowser();
00056
00057 protected:
00058     void itemInserted(QtBrowserItem *item, QtBrowserItem *afterItem) override;
00059     void itemRemoved(QtBrowserItem *item) override;
00060     void itemChanged(QtBrowserItem *item) override;
00061
00062 private:
00063
00064     QScopedPointer<QtGroupBoxPropertyBrowserPrivate> d_ptr;
00065     Q_DECLARE_PRIVATE(QtGroupBoxPropertyBrowser)
00066     Q_DISABLE_COPY_MOVE(QtGroupBoxPropertyBrowser)
00067     Q_PRIVATE_SLOT(d_func(), void slotUpdate())
00068     Q_PRIVATE_SLOT(d_func(), void slotEditorDestroyed())
00069
00070 };
00071
00072 QT_END_NAMESPACE
00073
00074 #endif

```

11.80 qtpropertybrowser.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to

```

```

00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public license version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 #ifndef QTPROPERTYBROWSER_H
00041 #define QTPROPERTYBROWSER_H
00042
00043 #include <QtWidgets/QWidget>
00044 #include <QtCore/QSet>
00045
00046 QT_BEGIN_NAMESPACE
00047
00048 #if defined(Q_OS_WIN)
00049 # if !defined(QT_QTPROPERTYBROWSER_EXPORT) && !defined(QT_QTPROPERTYBROWSER_IMPORT)
00050 #   define QT_QTPROPERTYBROWSER_EXPORT
00051 # elif defined(QT_QTPROPERTYBROWSER_IMPORT)
00052 #   if defined(QT_QTPROPERTYBROWSER_EXPORT)
00053 #     undef QT_QTPROPERTYBROWSER_EXPORT
00054 #   endif
00055 #   define QT_QTPROPERTYBROWSER_EXPORT __declspec(dllimport)
00056 # elif defined(QT_QTPROPERTYBROWSER_EXPORT)
00057 #   undef QT_QTPROPERTYBROWSER_EXPORT
00058 #   define QT_QTPROPERTYBROWSER_EXPORT __declspec(dllexport)
00059 # endif
00060 #else
00061 # define QT_QTPROPERTYBROWSER_EXPORT
00062 #endif
00063
00064 class QtAbstractPropertyManager;
00065 class QtPropertyPrivate;
00066
00067 class QT_QTPROPERTYBROWSER_EXPORT QtProperty
00068 {
00069 public:
00070     virtual ~QtProperty();
00071
00072     QList<QtProperty *> subProperties() const;
00073
00074     QtAbstractPropertyManager *propertyManager() const;
00075
00076     QString tooltip() const { return valueToolTip(); } // Compatibility
00077     QString valueToolTip() const;
00078     QString descriptionToolTip() const;
00079     QString statusTip() const;
00080     QString whatsThis() const;
00081     QString propertyName() const;
00082     bool isEnabled() const;
00083     bool isModified() const;
00084
00085     bool hasValue() const;
00086     QIcon valueIcon() const;
00087     QString valueText() const;
00088
00089     void setToolTip(const QString &text) { setValueToolTip(text); } // Compatibility
00090     void setValueToolTip(const QString &text);
00091     void setDescriptionToolTip(const QString &text);
00092     void setStatusTip(const QString &text);
00093     void setWhatsThis(const QString &text);
00094     void setPropertyName(const QString &text);
00095     void setEnabled(bool enable);
00096     void setModified(bool modified);
00097
00098     void addSubProperty(QtProperty *property);
00099     void insertSubProperty(QtProperty *property, QtProperty *afterProperty);
00100     void removeSubProperty(QtProperty *property);
00101 protected:
00102     explicit QtProperty(QtAbstractPropertyManager *manager);
00103     void propertyChanged();
00104 private:
00105     friend class QtAbstractPropertyManager;
00106     QScopedPointer<QtPropertyPrivate> d_ptr;
00107 };
00108

```



```

00109 class QtAbstractPropertyManagerPrivate;
00110
00111 class QT_QTPROPERTYBROWSER_EXPORT QtAbstractPropertyManager : public QObject
00112 {
00113     Q_OBJECT
00114 public:
00115     explicit QtAbstractPropertyManager(QObject *parent = 0);
00116     ~QtAbstractPropertyManager();
00117
00118     QSet<QtProperty *> properties() const;
00119     void clear() const;
00120
00121     QtProperty *addProperty(const QString &name = QString());
00122 Q_SIGNALS:
00123     void propertyInserted(QtProperty *property,
00124                           QtProperty *parent, QtProperty *after);
00125     void propertyChanged(QtProperty *property);
00126     void propertyRemoved(QtProperty *property, QtProperty *parent);
00127     void propertyDestroyed(QtProperty *property);
00128 protected:
00129     virtual bool hasValue(const QtProperty *property) const;
00130     virtual QIcon valueIcon(const QtProperty *property) const;
00131     virtual QString valueText(const QtProperty *property) const;
00132     virtual void initializeProperty(QtProperty *property) = 0;
00133     virtual void uninitializeProperty(QtProperty *property);
00134     virtual QtProperty *createProperty();
00135 private:
00136     friend class QtProperty;
00137     QScopedPointer<QtAbstractPropertyManagerPrivate> d_ptr;
00138     Q_DECLARE_PRIVATE(QtAbstractPropertyManager)
00139     Q_DISABLE_COPY_MOVE(QtAbstractPropertyManager)
00140 };
00141
00142 class QT_QTPROPERTYBROWSER_EXPORT QtAbstractEditorFactoryBase : public QObject
00143 {
00144     Q_OBJECT
00145 public:
00146     virtual QWidget *createEditor(QtProperty *property, QWidget *parent) = 0;
00147 protected:
00148     explicit QtAbstractEditorFactoryBase(QObject *parent = 0)
00149         : QObject(parent) {}
00150     virtual void breakConnection(QtAbstractPropertyManager *manager) = 0;
00151 protected Q_SLOTS:
00152     virtual void managerDestroyed(QObject *manager) = 0;
00153     friend class QtAbstractPropertyBrowser;
00154 };
00155
00156 template <class PropertyManager>
00157 class QtAbstractEditorFactory : public QtAbstractEditorFactoryBase
00158 {
00159 public:
00160     explicit QtAbstractEditorFactory(QObject *parent) : QtAbstractEditorFactoryBase(parent) {}
00161     QWidget *createEditor(QtProperty *property, QWidget *parent) override
00162     {
00163         for (PropertyManager *manager : qAsConst(m_managers)) {
00164             if (manager == property->propertyManager()) {
00165                 return createEditor(manager, property, parent);
00166             }
00167         }
00168         return 0;
00169     }
00170     void addPropertyManager(PropertyManager *manager)
00171     {
00172         if (m_managers.contains(manager))
00173             return;
00174         m_managers.insert(manager);
00175         connectPropertyManager(manager);
00176         connect(manager, SIGNAL(destroyed(QObject *)),
00177               this, SLOT(managerDestroyed(QObject *)));
00178     }
00179     void removePropertyManager(PropertyManager *manager)
00180     {
00181         if (!m_managers.contains(manager))
00182             return;
00183         disconnect(manager, SIGNAL(destroyed(QObject *)),
00184               this, SLOT(managerDestroyed(QObject *)));
00185         disconnectPropertyManager(manager);
00186         m_managers.remove(manager);
00187     }
00188     QSet<PropertyManager *> propertyManagers() const
00189     {
00190         return m_managers;
00191     }
00192 }

```



```

00196     PropertyManager *propertyManager(QtProperty *property) const
00197     {
00198         QtAbstractPropertyManager *manager = property->propertyManager();
00199         for (PropertyManager *m : qAsConst(m_managers)) {
00200             if (m == manager) {
00201                 return m;
00202             }
00203         }
00204         return 0;
00205     }
00206 protected:
00207     virtual void connectPropertyManager(PropertyManager *manager) = 0;
00208     virtual QWidget *createEditor(PropertyManager *manager, QtProperty *property,
00209                                   QWidget *parent) = 0;
00210     virtual void disconnectPropertyManager(PropertyManager *manager) = 0;
00211     void managerDestroyed(QObject *manager) override
00212     {
00213         for (PropertyManager *m : qAsConst(m_managers)) {
00214             if (m == manager) {
00215                 m_managers.remove(m);
00216                 return;
00217             }
00218         }
00219     }
00220 private:
00221     void breakConnection(QtAbstractPropertyManager *manager) override
00222     {
00223         for (PropertyManager *m : qAsConst(m_managers)) {
00224             if (m == manager) {
00225                 removePropertyManager(m);
00226                 return;
00227             }
00228         }
00229     }
00230 private:
00231     QSet<PropertyManager *> m_managers;
00232     friend class QtAbstractPropertyEditor;
00233 };
00234
00235 class QtAbstractPropertyBrowser;
00236 class QtBrowserItemPrivate;
00237
00238 class QT_QTPROPERTYBROWSER_EXPORT QtBrowserItem
00239 {
00240 public:
00241     QtProperty *property() const;
00242     QtBrowserItem *parent() const;
00243     QList<QtBrowserItem *> children() const;
00244     QtAbstractPropertyBrowser *browser() const;
00245 private:
00246     explicit QtBrowserItem(QtAbstractPropertyBrowser *browser, QtProperty *property, QtBrowserItem
00247                             *parent);
00247     ~QtBrowserItem();
00248     QScopedPointer<QtBrowserItemPrivate> d_ptr;
00249     friend class QtAbstractPropertyBrowserPrivate;
00250 };
00251
00252 class QtAbstractPropertyBrowserPrivate;
00253
00254 class QT_QTPROPERTYBROWSER_EXPORT QtAbstractPropertyBrowser : public QWidget
00255 {
00256     Q_OBJECT
00257 public:
00258
00259     explicit QtAbstractPropertyBrowser(QWidget *parent = 0);
00260     ~QtAbstractPropertyBrowser();
00261
00262     QList<QtProperty *> properties() const;
00263     QList<QtBrowserItem *> items(QtProperty *property) const;
00264     QtBrowserItem *topLevelItem(QtProperty *property) const;
00265     QList<QtBrowserItem *> topLevelItems() const;
00266     void clear();
00267
00268     template <class PropertyManager>
00269     void setFactoryForManager(PropertyManager *manager,
00270                               QtAbstractEditorFactory<PropertyManager> *factory) {
00271         QtAbstractPropertyManager *abstractManager = manager;
00272         QtAbstractEditorFactoryBase *abstractFactory = factory;
00273
00274         if (addFactory(abstractManager, abstractFactory))
00275             factory->addPropertyManager(manager);
00276     }
00277
00278     void unsetFactoryForManager(QtAbstractPropertyManager *manager);
00279
00280     QtBrowserItem *currentItem() const;
00281     void setCurrentItem(QtBrowserItem *);

```

```

00282
00283 Q_SIGNALS:
00284     void currentItemChanged(QtBrowserItem *);
00285
00286 public Q_SLOTS:
00287
00288     QtBrowserItem *addProperty(QtProperty *property);
00289     QtBrowserItem *insertProperty(QtProperty *property, QtProperty *afterProperty);
00290     void removeProperty(QtProperty *property);
00291
00292 protected:
00293
00294     virtual void itemInserted(QtBrowserItem *item, QtBrowserItem *afterItem) = 0;
00295     virtual void itemRemoved(QtBrowserItem *item) = 0;
00296     // can be tooltip, statustip, whatsthis, name, icon, text.
00297     virtual void itemChanged(QtBrowserItem *item) = 0;
00298
00299     virtual QWidget *createEditor(QtProperty *property, QWidget *parent);
00300 private:
00301
00302     bool addFactory(QtAbstractPropertyManager *abstractManager,
00303                   QtAbstractEditorFactoryBase *abstractFactory);
00304
00305     QScopedPointer<QtAbstractPropertyBrowserPrivate> d_ptr;
00306     Q_DECLARE_PRIVATE(QtAbstractPropertyBrowser)
00307     Q_DISABLE_COPY_MOVE(QtAbstractPropertyBrowser)
00308     Q_PRIVATE_SLOT(d_func(), void slotPropertyInserted(QtProperty *,
00309                                                         QtProperty *, QtProperty *))
00310     Q_PRIVATE_SLOT(d_func(), void slotPropertyRemoved(QtProperty *,
00311                                                         QtProperty *))
00312     Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty *))
00313     Q_PRIVATE_SLOT(d_func(), void slotPropertyDataChanged(QtProperty *))
00314
00315 };
00316
00317 QT_END_NAMESPACE
00318
00319 #endif // QTPROPERTYBROWSER_H

```

11.81 qtpropertybrowserutils_p.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public license version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 //
00041 // W A R N I N G
00042 // -----

```

```

00043 //
00044 // This file is not part of the Qt API. It exists for the convenience
00045 // of Qt Designer. This header
00046 // file may change from version to version without notice, or even be removed.
00047 //
00048 // We mean it.
00049 //
00050
00051 #ifndef QTPROPERTYBROWSERUTILS_H
00052 #define QTPROPERTYBROWSERUTILS_H
00053
00054 #include <QtCore/QMap>
00055 #include <QtGui/QIcon>
00056 #include <QtWidgets/QWidget>
00057 #include <QtWidgets/QPushButton>
00058 #include <QtWidgets/QDateEdit>
00059 #include <QtWidgets/QSpinBox>
00060 #include <QtCore/QDateTime>
00061 #include <QtCore/QStringList>
00062
00063 QT_BEGIN_NAMESPACE
00064
00065 class QMouseEvent;
00066 class QCheckBox;
00067 class QLineEdit;
00068
00069 class QtCursorDatabase
00070 {
00071 public:
00072     QtCursorDatabase();
00073     void clear();
00074
00075     QStringList cursorShapeNames() const;
00076     QMap<int, QIcon> cursorShapeIcons() const;
00077     QString cursorToShapeName(const QCursor& cursor) const;
00078     QIcon cursorToShapeIcon(const QCursor& cursor) const;
00079     int cursorToValue(const QCursor& cursor) const;
00080 #ifndef QT_NO_CURSOR
00081     QCursor valueToCursor(int value) const;
00082 #endif
00083 private:
00084     void appendCursor(Qt::CursorShape shape, const QString& name, const QIcon& icon);
00085     QStringList m_cursorNames;
00086     QMap<int, QIcon> m_cursorIcons;
00087     QMap<int, Qt::CursorShape> m_valueToCursorShape;
00088     QMap<Qt::CursorShape, int> m_cursorShapeToValue;
00089 };
00090
00091 class QtPropertyBrowserUtils
00092 {
00093 public:
00094     static QPixmap brushValuePixmap(const QBrush& b);
00095     static QIcon brushValueIcon(const QBrush& b);
00096     static QString colorValueText(const QColor& c);
00097     static QPixmap fontValuePixmap(const QFont& f);
00098     static QIcon fontValueIcon(const QFont& f);
00099     static QString fontValueText(const QFont& f);
00100     static QString dateFormat();
00101     static QString timeFormat();
00102     static QString dateTimeFormat();
00103 };
00104
00105 #pragma region QtBoolEdit
00106
00107 class QtBoolEdit : public QWidget {
00108     Q_OBJECT
00109 public:
00110     QtBoolEdit(QWidget* parent = 0);
00111
00112     bool textVisible() const { return m_textVisible; }
00113     void setTextVisible(bool textVisible);
00114
00115     Qt::CheckState checkState() const;
00116     void setCheckState(Qt::CheckState state);
00117
00118     bool isChecked() const;
00119     void setChecked(bool c);
00120     void setInitialChecked(bool c);
00121
00122     bool blockCheckBoxSignals(bool block);
00123
00124     Q_SIGNALS:
00125         void toggled(bool);
00126
00127 protected:
00128     void mousePressEvent(QMouseEvent* event) override;
00129

```

```
00130 private:
00131     QCheckBox* m_checkBox;
00132     QPushButton* m_button;
00133     bool m_textVisible;
00134     bool m_initialChecked;
00135 };
00136 #pragma endregion
00137
00138 #pragma region QtIntEdit
00139
00140 class QtIntEdit : public QWidget {
00141     Q_OBJECT
00142 public:
00143     QtIntEdit(QWidget* parent = Q_NULLPTR);
00144
00145     int value();
00146     void setValue(int);
00147     void setInitialValue(int);
00148     void setSingleStep(int);
00149     void setRange(int, int);
00150     void setKeyboardTracking(bool);
00151
00152 Q_SIGNALS:
00153     void valueChanged(int);
00154
00155 private:
00156     QSpinBox* m_spinbox;
00157     QPushButton* m_button;
00158     int m_initialvalue;
00159 };
00160 #pragma endregion
00161
00162 #pragma region QtDoubleEdit
00163
00164 class QtDoubleEdit : public QWidget {
00165     Q_OBJECT
00166 public:
00167     QtDoubleEdit(QWidget* parent = Q_NULLPTR);
00168
00169     int value();
00170     void setValue(double);
00171     void setInitialValue(double);
00172     void setDecimals(double);
00173     void setSingleStep(double);
00174     void setRange(double, double);
00175     void setKeyboardTracking(bool);
00176
00177 Q_SIGNALS:
00178     void valueChanged(double);
00179
00180 private:
00181     QDoubleSpinBox* m_doublespinbox;
00182     QPushButton* m_button;
00183     double m_initialvalue;
00184 };
00185 #pragma endregion
00186
00187 #pragma region QtStringEdit
00188
00189 class QtStringEdit : public QWidget {
00190     Q_OBJECT
00191 public:
00192     QtStringEdit(QWidget* parent = Q_NULLPTR);
00193
00194     QString text();
00195     void setText(QString);
00196     void setInitialText(QString);
00197     const QValidator* validator();
00198     void setValidator(QValidator*);
00199
00200 Q_SIGNALS:
00201     void textChanged(QString);
00202     void textEdited(QString);
00203
00204 private:
00205     QLineEdit* m_lineedit;
00206     QPushButton* m_button;
00207     QString m_initialvalue;
00208 };
00209 #pragma endregion
00210
00211 #pragma region QtDateEdit
00212
00213 class QtDateEdit : public QWidget {
```

```

00217     Q_OBJECT
00218 public:
00219     QtDateEdit(QWidget* parent = Q_NULLPTR);
00220
00221     QDate date();
00222     void setDate(QDate);
00223     void setInitialDate(QDate);
00224     void setDateRange(QDate, QDate);
00225     void setDisplayFormat(QString);
00226     void setCalendarPopup(bool);
00227
00228 Q_SIGNALS:
00229     void dateChanged(QDate);
00230
00231 private:
00232     QDateEdit* m_dateedit;
00233     QPushButton* m_button;
00234     QDate m_initialvalue;
00235 };
00236
00237 #pragma endregion
00238
00239 #pragma region QtTimeEdit
00240
00241 class QtTimeEdit : public QWidget {
00242     Q_OBJECT
00243 public:
00244     QtTimeEdit(QWidget* parent = Q_NULLPTR);
00245
00246     QTime time();
00247     void setTime(QTime);
00248     void setInitialTime(QTime);
00249     void setTimeRange(QTime, QTime);
00250     void setDisplayFormat(QString);
00251     void setCalendarPopup(bool);
00252
00253 Q_SIGNALS:
00254     void timeChanged(QTime);
00255
00256 private:
00257     QTimeEdit* m_timeedit;
00258     QPushButton* m_button;
00259     QTime m_initialvalue;
00260 };
00261
00262 #pragma endregion
00263
00264 #pragma region QDateTimeEdit
00265
00266 class QDateTimeEdit : public QWidget {
00267     Q_OBJECT
00268 public:
00269     QDateTimeEdit(QWidget* parent = Q_NULLPTR);
00270
00271     QDateTime dateTime();
00272     void setDateTime(const QDateTime&);
00273     void setInitialDateTime(const QDateTime&);
00274     void setDateTimeRange(const QDateTime&, const QDateTime&);
00275     void setDisplayFormat(const QString&);
00276     void setCalendarPopup(bool);
00277
00278 Q_SIGNALS:
00279     void dateTimeChanged(const QDateTime&);
00280
00281 private:
00282     QDateTimeEdit* m_timeedit;
00283     QPushButton* m_button;
00284     QDateTime m_initialvalue;
00285 };
00286
00287 #pragma endregion
00288
00289
00290 QT_END_NAMESPACE
00291
00292 #endif

```

11.82 qtpropertymanager.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/

```

```

00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public license version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 #ifndef QTPROPERTYMANAGER_H
00041 #define QTPROPERTYMANAGER_H
00042
00043 #include "qtpropertybrowser.h"
00044
00045 QT_BEGIN_NAMESPACE
00046
00047 class QDate;
00048 class QTime;
00049 class QDateTime;
00050 class QLocale;
00051 class QRegularExpression;
00052
00053 #pragma region QtGroupPropertyManager
00054
00055 class QT_QTPROPERTYBROWSER_EXPORT QtGroupPropertyManager : public QtAbstractPropertyManager
00056 {
00057     Q_OBJECT
00058 public:
00059     QtGroupPropertyManager(QObject* parent = 0);
00060     ~QtGroupPropertyManager();
00061
00062 protected:
00063     bool hasValue(const QtProperty* property) const override;
00064
00065     void initializeProperty(QtProperty* property) override;
00066     void uninitializeProperty(QtProperty* property) override;
00067 };
00068
00069 #pragma endregion
00070
00071 #pragma region QtIntPropertyManager
00072
00073 class QtIntPropertyManagerPrivate;
00074
00075 class QT_QTPROPERTYBROWSER_EXPORT QtIntPropertyManager : public QtAbstractPropertyManager
00076 {
00077     Q_OBJECT
00078 public:
00079     QtIntPropertyManager(QObject* parent = 0);
00080     ~QtIntPropertyManager();
00081
00082     int value(const QtProperty* property) const;
00083     int initialValue(const QtProperty* property) const;
00084     int minimum(const QtProperty* property) const;
00085     int maximum(const QtProperty* property) const;
00086     int singleStep(const QtProperty* property) const;
00087
00088 public Q_SLOTS:
00089     void setValue(QtProperty* property, int val);
00090     void setValueOnly(QtProperty* property, int val);
00091     void setMinimum(QtProperty* property, int minVal);

```

```

00092     void setMaximum(QtProperty* property, int maxVal);
00093     void setRange(QtProperty* property, int minVal, int maxVal);
00094     void setSingleStep(QtProperty* property, int step);
00095 Q_SIGNALS:
00096     void valueChanged(QtProperty* property, int val);
00097     void rangeChanged(QtProperty* property, int minVal, int maxVal);
00098     void singleStepChanged(QtProperty* property, int step);
00099 protected:
00100     QString valueText(const QtProperty* property) const override;
00101     void initializeProperty(QtProperty* property) override;
00102     void uninitializedProperty(QtProperty* property) override;
00103 private:
00104     QScopedPointer<QtIntPropertyManagerPrivate> d_ptr;
00105     Q_DECLARE_PRIVATE(QtIntPropertyManager)
00106     Q_DISABLE_COPY_MOVE(QtIntPropertyManager)
00107 };
00108 #pragma endregion
00109
00110 #pragma region QtBoolPropertyManager
00111
00112 class QtBoolPropertyManagerPrivate;
00113
00114 class QT_QTPROPERTYBROWSER_EXPORT QtBoolPropertyManager : public QtAbstractPropertyManager
00115 {
00116     Q_OBJECT
00117 public:
00118     QtBoolPropertyManager(QObject* parent = 0);
00119     ~QtBoolPropertyManager();
00120
00121     bool value(const QtProperty* property) const;
00122     bool initialValue(const QtProperty* property) const;
00123
00124 public Q_SLOTS:
00125     void setValue(QtProperty* property, bool val);
00126     void setValueOnly(QtProperty* property, bool val);
00127 Q_SIGNALS:
00128     void valueChanged(QtProperty* property, bool val);
00129 protected:
00130     QString valueText(const QtProperty* property) const override;
00131     QIcon valueIcon(const QtProperty* property) const override;
00132     void initializeProperty(QtProperty* property) override;
00133     void uninitializedProperty(QtProperty* property) override;
00134 private:
00135     QScopedPointer<QtBoolPropertyManagerPrivate> d_ptr;
00136     Q_DECLARE_PRIVATE(QtBoolPropertyManager)
00137     Q_DISABLE_COPY_MOVE(QtBoolPropertyManager)
00138 };
00139
00140 #pragma endregion
00141
00142 #pragma region QtDoublePropertyManager
00143
00144 class QtDoublePropertyManagerPrivate;
00145
00146 class QT_QTPROPERTYBROWSER_EXPORT QtDoublePropertyManager : public QtAbstractPropertyManager
00147 {
00148     Q_OBJECT
00149 public:
00150     QtDoublePropertyManager(QObject* parent = 0);
00151     ~QtDoublePropertyManager();
00152
00153     double value(const QtProperty* property) const;
00154     double initialValue(const QtProperty* property) const;
00155     double minimum(const QtProperty* property) const;
00156     double maximum(const QtProperty* property) const;
00157     double singleStep(const QtProperty* property) const;
00158     int decimals(const QtProperty* property) const;
00159
00160 public Q_SLOTS:
00161     void setValue(QtProperty* property, double val);
00162     void setValueOnly(QtProperty* property, double val);
00163     void setMinimum(QtProperty* property, double minVal);
00164     void setMaximum(QtProperty* property, double maxVal);
00165     void setRange(QtProperty* property, double minVal, double maxVal);
00166     void setSingleStep(QtProperty* property, double step);
00167     void setDecimals(QtProperty* property, int prec);
00168 Q_SIGNALS:
00169     void valueChanged(QtProperty* property, double val);
00170     void rangeChanged(QtProperty* property, double minVal, double maxVal);
00171     void singleStepChanged(QtProperty* property, double step);
00172     void decimalsChanged(QtProperty* property, int prec);
00173 protected:
00174     QString valueText(const QtProperty* property) const override;
00175     void initializeProperty(QtProperty* property) override;
00176     void uninitializedProperty(QtProperty* property) override;
00177 private:
00178     QScopedPointer<QtDoublePropertyManagerPrivate> d_ptr;

```

```

00179     Q_DECLARE_PRIVATE(QtDoublePropertyManager)
00180     Q_DISABLE_COPY_MOVE(QtDoublePropertyManager)
00181 };
00182
00183 #pragma endregion
00184
00185 #pragma region QtStringPropertyManager
00186
00187 class QtStringPropertyManagerPrivate;
00188
00189 class QT_QTPROPERTYBROWSER_EXPORT QtStringPropertyManager : public QtAbstractPropertyManager
00190 {
00191     Q_OBJECT
00192 public:
00193     QtStringPropertyManager(QObject* parent = 0);
00194     ~QtStringPropertyManager();
00195
00196     QString value(const QtProperty* property) const;
00197     QString initialValue(const QtProperty* property) const;
00198     QRegularExpression regExp(const QtProperty* property) const;
00199
00200 public Q_SLOTS:
00201     void setValue(QtProperty* property, const QString& val);
00202     void setValueOnly(QtProperty* property, const QString& val);
00203     void setRegExp(QtProperty* property, const QRegularExpression& regExp);
00204 Q_SIGNALS:
00205     void valueChanged(QtProperty* property, const QString& val);
00206     void regExpChanged(QtProperty* property, const QRegularExpression& regExp);
00207 protected:
00208     QString valueText(const QtProperty* property) const override;
00209     void initializeProperty(QtProperty* property) override;
00210     void uninitializeProperty(QtProperty* property) override;
00211 private:
00212     QScopedPointer<QtStringPropertyManagerPrivate> d_ptr;
00213     Q_DECLARE_PRIVATE(QtStringPropertyManager)
00214     Q_DISABLE_COPY_MOVE(QtStringPropertyManager)
00215 };
00216
00217 #pragma endregion
00218
00219 #pragma region QtDatePropertyManager
00220
00221 class QtDatePropertyManagerPrivate;
00222
00223 class QT_QTPROPERTYBROWSER_EXPORT QtDatePropertyManager : public QtAbstractPropertyManager
00224 {
00225     Q_OBJECT
00226 public:
00227     QtDatePropertyManager(QObject* parent = 0);
00228     ~QtDatePropertyManager();
00229
00230     QDate value(const QtProperty* property) const;
00231     QDate initialValue(const QtProperty* property) const;
00232     QDate minimum(const QtProperty* property) const;
00233     QDate maximum(const QtProperty* property) const;
00234
00235 public Q_SLOTS:
00236     void setValue(QtProperty* property, QDate val);
00237     void setValueOnly(QtProperty* property, QDate val);
00238     void setMinimum(QtProperty* property, QDate minVal);
00239     void setMaximum(QtProperty* property, QDate maxVal);
00240     void setRange(QtProperty* property, QDate minVal, QDate maxVal);
00241 Q_SIGNALS:
00242     void valueChanged(QtProperty* property, QDate val);
00243     void rangeChanged(QtProperty* property, QDate minVal, QDate maxVal);
00244 protected:
00245     QString valueText(const QtProperty* property) const override;
00246     void initializeProperty(QtProperty* property) override;
00247     void uninitializeProperty(QtProperty* property) override;
00248 private:
00249     QScopedPointer<QtDatePropertyManagerPrivate> d_ptr;
00250     Q_DECLARE_PRIVATE(QtDatePropertyManager)
00251     Q_DISABLE_COPY_MOVE(QtDatePropertyManager)
00252 };
00253
00254 #pragma endregion
00255
00256 #pragma region QtTimePropertyManager
00257
00258 class QtTimePropertyManagerPrivate;
00259
00260 class QT_QTPROPERTYBROWSER_EXPORT QtTimePropertyManager : public QtAbstractPropertyManager
00261 {
00262     Q_OBJECT
00263 public:
00264     QtTimePropertyManager(QObject* parent = 0);
00265     ~QtTimePropertyManager();

```



```

00266
00267     QTime value(const QtProperty* property) const;
00268     QTime initialValue(const QtProperty* property) const;
00269
00270 public Q_SLOTS:
00271     void setValue(QtProperty* property, QTime val);
00272     void setValueOnly(QtProperty* property, QTime val);
00273 Q_SIGNALS:
00274     void valueChanged(QtProperty* property, QTime val);
00275 protected:
00276     QString valueText(const QtProperty* property) const override;
00277     void initializeProperty(QtProperty* property) override;
00278     void uninitializedProperty(QtProperty* property) override;
00279 private:
00280     QScopedPointer<QtTimePropertyManagerPrivate> d_ptr;
00281     Q_DECLARE_PRIVATE(QtTimePropertyManager)
00282     Q_DISABLE_COPY_MOVE(QtTimePropertyManager)
00283 };
00284
00285 #pragma endregion
00286
00287 #pragma region QDateTimePropertyManager
00288
00289 class QDateTimePropertyManagerPrivate;
00290
00291 class QT_QTPROPERTYBROWSER_EXPORT QDateTimePropertyManager : public QtAbstractPropertyManager
00292 {
00293     Q_OBJECT
00294 public:
00295     QDateTimePropertyManager(QObject* parent = 0);
00296     ~QtDateTimePropertyManager();
00297
00298     QDateTime value(const QtProperty* property) const;
00299     QDateTime initialValue(const QtProperty* property) const;
00300
00301 public Q_SLOTS:
00302     void setValue(QtProperty* property, const QDateTime& val);
00303     void setValueOnly(QtProperty* property, const QDateTime& val);
00304 Q_SIGNALS:
00305     void valueChanged(QtProperty* property, const QDateTime& val);
00306 protected:
00307     QString valueText(const QtProperty* property) const override;
00308     void initializeProperty(QtProperty* property) override;
00309     void uninitializedProperty(QtProperty* property) override;
00310 private:
00311     QScopedPointer<QtDateTimePropertyManagerPrivate> d_ptr;
00312     Q_DECLARE_PRIVATE(QtDateTimePropertyManager)
00313     Q_DISABLE_COPY_MOVE(QtDateTimePropertyManager)
00314 };
00315
00316 #pragma endregion
00317
00318 #pragma region QtKeySequencePropertyManager
00319
00320 class QtKeySequencePropertyManagerPrivate;
00321
00322 class QT_QTPROPERTYBROWSER_EXPORT QtKeySequencePropertyManager : public QtAbstractPropertyManager
00323 {
00324     Q_OBJECT
00325 public:
00326     QtKeySequencePropertyManager(QObject* parent = 0);
00327     ~QtKeySequencePropertyManager();
00328
00329     QKeySequence value(const QtProperty* property) const;
00330
00331 public Q_SLOTS:
00332     void setValue(QtProperty* property, const QKeySequence& val);
00333 Q_SIGNALS:
00334     void valueChanged(QtProperty* property, const QKeySequence& val);
00335 protected:
00336     QString valueText(const QtProperty* property) const override;
00337     void initializeProperty(QtProperty* property) override;
00338     void uninitializedProperty(QtProperty* property) override;
00339 private:
00340     QScopedPointer<QtKeySequencePropertyManagerPrivate> d_ptr;
00341     Q_DECLARE_PRIVATE(QtKeySequencePropertyManager)
00342     Q_DISABLE_COPY_MOVE(QtKeySequencePropertyManager)
00343 };
00344
00345 #pragma endregion
00346
00347 #pragma region QtCharPropertyManager
00348
00349 class QtCharPropertyManagerPrivate;
00350
00351 class QT_QTPROPERTYBROWSER_EXPORT QtCharPropertyManager : public QtAbstractPropertyManager
00352 {

```

```

00353     Q_OBJECT
00354 public:
00355     QtCharPropertyManager(QObject* parent = 0);
00356     ~QtCharPropertyManager();
00357
00358     QChar value(const QtProperty* property) const;
00359
00360 public Q_SLOTS:
00361     void setValue(QtProperty* property, const QChar& val);
00362 Q_SIGNALS:
00363     void valueChanged(QtProperty* property, const QChar& val);
00364 protected:
00365     QString valueText(const QtProperty* property) const override;
00366     void initializeProperty(QtProperty* property) override;
00367     void uninitializeProperty(QtProperty* property) override;
00368 private:
00369     QScopedPointer<QtCharPropertyManagerPrivate> d_ptr;
00370     Q_DECLARE_PRIVATE(QtCharPropertyManager)
00371     Q_DISABLE_COPY_MOVE(QtCharPropertyManager)
00372 };
00373
00374 #pragma endregion
00375
00376 #pragma region QtLocalePropertyManager
00377
00378 class QtEnumPropertyManager;
00379 class QtLocalePropertyManagerPrivate;
00380
00381 class QT_QTPROPERTYBROWSER_EXPORT QtLocalePropertyManager : public QtAbstractPropertyManager
00382 {
00383     Q_OBJECT
00384 public:
00385     QtLocalePropertyManager(QObject* parent = 0);
00386     ~QtLocalePropertyManager();
00387
00388     QtEnumPropertyManager* subEnumPropertyManager() const;
00389
00390     QLocale value(const QtProperty* property) const;
00391
00392 public Q_SLOTS:
00393     void setValue(QtProperty* property, const QLocale& val);
00394 Q_SIGNALS:
00395     void valueChanged(QtProperty* property, const QLocale& val);
00396 protected:
00397     QString valueText(const QtProperty* property) const override;
00398     void initializeProperty(QtProperty* property) override;
00399     void uninitializeProperty(QtProperty* property) override;
00400 private:
00401     QScopedPointer<QtLocalePropertyManagerPrivate> d_ptr;
00402     Q_DECLARE_PRIVATE(QtLocalePropertyManager)
00403     Q_DISABLE_COPY_MOVE(QtLocalePropertyManager)
00404     Q_PRIVATE_SLOT(d_func(), void slotEnumChanged(QtProperty*, int))
00405     Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00406 };
00407
00408 #pragma endregion
00409
00410 #pragma region QtPointPropertyManager
00411
00412 class QtPointPropertyManagerPrivate;
00413
00414 class QT_QTPROPERTYBROWSER_EXPORT QtPointPropertyManager : public QtAbstractPropertyManager
00415 {
00416     Q_OBJECT
00417 public:
00418     QtPointPropertyManager(QObject* parent = 0);
00419     ~QtPointPropertyManager();
00420
00421     QtIntPropertyManager* subIntPropertyManager() const;
00422
00423     QPoint value(const QtProperty* property) const;
00424
00425 public Q_SLOTS:
00426     void setValue(QtProperty* property, const QPoint& val);
00427 Q_SIGNALS:
00428     void valueChanged(QtProperty* property, const QPoint& val);
00429 protected:
00430     QString valueText(const QtProperty* property) const override;
00431     void initializeProperty(QtProperty* property) override;
00432     void uninitializeProperty(QtProperty* property) override;
00433 private:
00434     QScopedPointer<QtPointPropertyManagerPrivate> d_ptr;
00435     Q_DECLARE_PRIVATE(QtPointPropertyManager)
00436     Q_DISABLE_COPY_MOVE(QtPointPropertyManager)
00437     Q_PRIVATE_SLOT(d_func(), void slotIntChanged(QtProperty*, int))
00438     Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00439 };

```

```

00440
00441 #pragma endregion
00442
00443 #pragma region QtPointFPropertyManager
00444
00445 class QtPointFPropertyManagerPrivate;
00446
00447 class QT_QTPROPERTYBROWSER_EXPORT QtPointFPropertyManager : public QtAbstractPropertyManager
00448 {
00449     Q_OBJECT
00450 public:
00451     QtPointFPropertyManager(QObject* parent = 0);
00452     ~QtPointFPropertyManager();
00453
00454     QtDoublePropertyManager* subDoublePropertyManager() const;
00455
00456     QPointF value(const QtProperty* property) const;
00457     int decimals(const QtProperty* property) const;
00458
00459 public Q_SLOTS:
00460     void setValue(QtProperty* property, const QPointF& val);
00461     void setDecimals(QtProperty* property, int prec);
00462 Q_SIGNALS:
00463     void valueChanged(QtProperty* property, const QPointF& val);
00464     void decimalsChanged(QtProperty* property, int prec);
00465 protected:
00466     QString valueText(const QtProperty* property) const override;
00467     void initializeProperty(QtProperty* property) override;
00468     void uninitializedProperty(QtProperty* property) override;
00469 private:
00470     QScopedPointer<QtPointFPropertyManagerPrivate> d_ptr;
00471     Q_DECLARE_PRIVATE(QtPointFPropertyManager)
00472     Q_DISABLE_COPY_MOVE(QtPointFPropertyManager)
00473     Q_PRIVATE_SLOT(d_func(), void slotDoubleChanged(QtProperty*, double))
00474     Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00475 };
00476
00477 #pragma endregion
00478
00479 #pragma region QSizePropertyManager
00480
00481 class QSizePropertyManagerPrivate;
00482
00483 class QT_QTPROPERTYBROWSER_EXPORT QSizePropertyManager : public QtAbstractPropertyManager
00484 {
00485     Q_OBJECT
00486 public:
00487     QSizePropertyManager(QObject* parent = 0);
00488     ~QSizePropertyManager();
00489
00490     QtIntPropertyManager* subIntPropertyManager() const;
00491
00492     QSize value(const QtProperty* property) const;
00493     QSize minimum(const QtProperty* property) const;
00494     QSize maximum(const QtProperty* property) const;
00495
00496 public Q_SLOTS:
00497     void setValue(QtProperty* property, const QSize& val);
00498     void setMinimum(QtProperty* property, const QSize& minVal);
00499     void setMaximum(QtProperty* property, const QSize& maxVal);
00500     void setRange(QtProperty* property, const QSize& minVal, const QSize& maxVal);
00501 Q_SIGNALS:
00502     void valueChanged(QtProperty* property, const QSize& val);
00503     void rangeChanged(QtProperty* property, const QSize& minVal, const QSize& maxVal);
00504 protected:
00505     QString valueText(const QtProperty* property) const override;
00506     void initializeProperty(QtProperty* property) override;
00507     void uninitializedProperty(QtProperty* property) override;
00508 private:
00509     QScopedPointer<QSizePropertyManagerPrivate> d_ptr;
00510     Q_DECLARE_PRIVATE(QSizePropertyManager)
00511     Q_DISABLE_COPY_MOVE(QSizePropertyManager)
00512     Q_PRIVATE_SLOT(d_func(), void slotIntChanged(QtProperty*, int))
00513     Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00514 };
00515
00516 #pragma endregion
00517
00518 #pragma region QSizeFPropertyManager
00519
00520 class QSizeFPropertyManagerPrivate;
00521
00522 class QT_QTPROPERTYBROWSER_EXPORT QSizeFPropertyManager : public QtAbstractPropertyManager
00523 {
00524     Q_OBJECT
00525 public:
00526     QSizeFPropertyManager(QObject* parent = 0);

```

```

00527     ~QtSizeFPropertyManager();
00528
00529     QtDoublePropertyManager* subDoublePropertyManager() const;
00530
00531     QSizeF value(const QtProperty* property) const;
00532     QSizeF minimum(const QtProperty* property) const;
00533     QSizeF maximum(const QtProperty* property) const;
00534     int decimals(const QtProperty* property) const;
00535
00536 public Q_SLOTS:
00537     void setValue(QtProperty* property, const QSizeF& val);
00538     void setMinimum(QtProperty* property, const QSizeF& minVal);
00539     void setMaximum(QtProperty* property, const QSizeF& maxVal);
00540     void setRange(QtProperty* property, const QSizeF& minVal, const QSizeF& maxVal);
00541     void setDecimals(QtProperty* property, int prec);
00542 Q_SIGNALS:
00543     void valueChanged(QtProperty* property, const QSizeF& val);
00544     void rangeChanged(QtProperty* property, const QSizeF& minVal, const QSizeF& maxVal);
00545     void decimalsChanged(QtProperty* property, int prec);
00546 protected:
00547     QString valueText(const QtProperty* property) const override;
00548     void initializeProperty(QtProperty* property) override;
00549     void uninitializeProperty(QtProperty* property) override;
00550 private:
00551     QScopedPointer<QtSizeFPropertyManagerPrivate> d_ptr;
00552     Q_DECLARE_PRIVATE(QtSizeFPropertyManager)
00553     Q_DISABLE_COPY_MOVE(QtSizeFPropertyManager)
00554     Q_PRIVATE_SLOT(d_func(), void slotDoubleChanged(QtProperty*, double))
00555     Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00556 };
00557
00558 #pragma endregion
00559
00560 #pragma region QtRectPropertyManager
00561
00562 class QtRectPropertyManagerPrivate;
00563
00564 class QT_QTPROPERTYBROWSER_EXPORT QtRectPropertyManager : public QtAbstractPropertyManager
00565 {
00566     Q_OBJECT
00567 public:
00568     QtRectPropertyManager(QObject* parent = 0);
00569     ~QtRectPropertyManager();
00570
00571     QtIntPropertyManager* subIntPropertyManager() const;
00572
00573     QRect value(const QtProperty* property) const;
00574     QRect constraint(const QtProperty* property) const;
00575
00576 public Q_SLOTS:
00577     void setValue(QtProperty* property, const QRect& val);
00578     void setConstraint(QtProperty* property, const QRect& constraint);
00579 Q_SIGNALS:
00580     void valueChanged(QtProperty* property, const QRect& val);
00581     void constraintChanged(QtProperty* property, const QRect& constraint);
00582 protected:
00583     QString valueText(const QtProperty* property) const override;
00584     void initializeProperty(QtProperty* property) override;
00585     void uninitializeProperty(QtProperty* property) override;
00586 private:
00587     QScopedPointer<QtRectPropertyManagerPrivate> d_ptr;
00588     Q_DECLARE_PRIVATE(QtRectPropertyManager)
00589     Q_DISABLE_COPY_MOVE(QtRectPropertyManager)
00590     Q_PRIVATE_SLOT(d_func(), void slotIntChanged(QtProperty*, int))
00591     Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00592 };
00593
00594 #pragma endregion
00595
00596 #pragma region QtRectFPropertyManager
00597
00598 class QtRectFPropertyManagerPrivate;
00599
00600 class QT_QTPROPERTYBROWSER_EXPORT QtRectFPropertyManager : public QtAbstractPropertyManager
00601 {
00602     Q_OBJECT
00603 public:
00604     QtRectFPropertyManager(QObject* parent = 0);
00605     ~QtRectFPropertyManager();
00606
00607     QtDoublePropertyManager* subDoublePropertyManager() const;
00608
00609     QRectF value(const QtProperty* property) const;
00610     QRectF constraint(const QtProperty* property) const;
00611     int decimals(const QtProperty* property) const;
00612
00613 public Q_SLOTS:

```

```

00614     void setValue(QtProperty* property, const QRectF& val);
00615     void setConstraint(QtProperty* property, const QRectF& constraint);
00616     void setDecimals(QtProperty* property, int prec);
00617 Q_SIGNALS:
00618     void valueChanged(QtProperty* property, const QRectF& val);
00619     void constraintChanged(QtProperty* property, const QRectF& constraint);
00620     void decimalsChanged(QtProperty* property, int prec);
00621 protected:
00622     QString valueText(const QtProperty* property) const override;
00623     void initializeProperty(QtProperty* property) override;
00624     void uninitializedProperty(QtProperty* property) override;
00625 private:
00626     QScopedPointer<QtRectFPropertyManagerPrivate> d_ptr;
00627     Q_DECLARE_PRIVATE(QtRectFPropertyManager)
00628     Q_DISABLE_COPY_MOVE(QtRectFPropertyManager)
00629     Q_PRIVATE_SLOT(d_func(), void slotDoubleChanged(QtProperty*, double))
00630     Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00631 };
00632
00633 #pragma endregion
00634
00635 #pragma region QtEnumPropertyManager
00636
00637 class QtEnumPropertyManagerPrivate;
00638
00639 class QT_QTPROPERTYBROWSER_EXPORT QtEnumPropertyManager : public QtAbstractPropertyManager
00640 {
00641     Q_OBJECT
00642 public:
00643     QtEnumPropertyManager(QObject* parent = 0);
00644     ~QtEnumPropertyManager();
00645
00646     int value(const QtProperty* property) const;
00647     QStringList enumNames(const QtProperty* property) const;
00648     QMap<int, QIcon> enumIcons(const QtProperty* property) const;
00649
00650 public Q_SLOTS:
00651     void setValue(QtProperty* property, int val);
00652     void setEnumNames(QtProperty* property, const QStringList& names);
00653     void setEnumIcons(QtProperty* property, const QMap<int, QIcon>& icons);
00654 Q_SIGNALS:
00655     void valueChanged(QtProperty* property, int val);
00656     void enumNamesChanged(QtProperty* property, const QStringList& names);
00657     void enumIconsChanged(QtProperty* property, const QMap<int, QIcon>& icons);
00658 protected:
00659     QString valueText(const QtProperty* property) const override;
00660     QIcon valueIcon(const QtProperty* property) const override;
00661     void initializeProperty(QtProperty* property) override;
00662     void uninitializedProperty(QtProperty* property) override;
00663 private:
00664     QScopedPointer<QtEnumPropertyManagerPrivate> d_ptr;
00665     Q_DECLARE_PRIVATE(QtEnumPropertyManager)
00666     Q_DISABLE_COPY_MOVE(QtEnumPropertyManager)
00667 };
00668
00669 #pragma endregion
00670
00671 #pragma region QtFlagPropertyManager
00672
00673 class QtFlagPropertyManagerPrivate;
00674
00675 class QT_QTPROPERTYBROWSER_EXPORT QtFlagPropertyManager : public QtAbstractPropertyManager
00676 {
00677     Q_OBJECT
00678 public:
00679     QtFlagPropertyManager(QObject* parent = 0);
00680     ~QtFlagPropertyManager();
00681
00682     QtBoolPropertyManager* subBoolPropertyManager() const;
00683
00684     int value(const QtProperty* property) const;
00685     QStringList flagNames(const QtProperty* property) const;
00686
00687 public Q_SLOTS:
00688     void setValue(QtProperty* property, int val);
00689     void setFlagNames(QtProperty* property, const QStringList& names);
00690 Q_SIGNALS:
00691     void valueChanged(QtProperty* property, int val);
00692     void flagNamesChanged(QtProperty* property, const QStringList& names);
00693 protected:
00694     QString valueText(const QtProperty* property) const override;
00695     void initializeProperty(QtProperty* property) override;
00696     void uninitializedProperty(QtProperty* property) override;
00697 private:
00698     QScopedPointer<QtFlagPropertyManagerPrivate> d_ptr;
00699     Q_DECLARE_PRIVATE(QtFlagPropertyManager)
00700     Q_DISABLE_COPY_MOVE(QtFlagPropertyManager)

```



```

00788     Q_OBJECT
00789 public:
00790     QColorPropertyManager(QObject* parent = 0);
00791     ~QtColorPropertyManager();
00792
00793     QtIntPropertyManager* subIntPropertyManager() const;
00794
00795     QColor value(const QtProperty* property) const;
00796
00797 public Q_SLOTS:
00798     void setValue(QtProperty* property, const QColor& val);
00799 Q_SIGNALS:
00800     void valueChanged(QtProperty* property, const QColor& val);
00801 protected:
00802     QString valueText(const QtProperty* property) const override;
00803     QIcon valueIcon(const QtProperty* property) const override;
00804     void initializeProperty(QtProperty* property) override;
00805     void uninitializedProperty(QtProperty* property) override;
00806 private:
00807     QScopedPointer<QtColorPropertyManagerPrivate> d_ptr;
00808     Q_DECLARE_PRIVATE(QtColorPropertyManager)
00809     Q_DISABLE_COPY_MOVE(QtColorPropertyManager)
00810     Q_PRIVATE_SLOT(d_func(), void slotIntChanged(QtProperty*, int))
00811     Q_PRIVATE_SLOT(d_func(), void slotPropertyDestroyed(QtProperty*))
00812 };
00813
00814 #pragma endregion
00815
00816 #pragma region QtCursorPropertyManager
00817
00818 class QtCursorPropertyManagerPrivate;
00819
00820 class QT_QTPROPERTYBROWSER_EXPORT QtCursorPropertyManager : public QtAbstractPropertyManager
00821 {
00822     Q_OBJECT
00823 public:
00824     QtCursorPropertyManager(QObject* parent = 0);
00825     ~QtCursorPropertyManager();
00826
00827 #ifndef QT_NO_CURSOR
00828     QCursor value(const QtProperty* property) const;
00829 #endif
00830
00831 public Q_SLOTS:
00832     void setValue(QtProperty* property, const QCursor& val);
00833 Q_SIGNALS:
00834     void valueChanged(QtProperty* property, const QCursor& val);
00835 protected:
00836     QString valueText(const QtProperty* property) const override;
00837     QIcon valueIcon(const QtProperty* property) const override;
00838     void initializeProperty(QtProperty* property) override;
00839     void uninitializedProperty(QtProperty* property) override;
00840 private:
00841     QScopedPointer<QtCursorPropertyManagerPrivate> d_ptr;
00842     Q_DECLARE_PRIVATE(QtCursorPropertyManager)
00843     Q_DISABLE_COPY_MOVE(QtCursorPropertyManager)
00844 };
00845
00846 #pragma endregion
00847
00848 QT_END_NAMESPACE
00849
00850 #endif

```

11.83 qttreepropertybrowser.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage

```



```

00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public license version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 #ifndef QTTREEPROPERTYBROWSER_H
00041 #define QTTREEPROPERTYBROWSER_H
00042
00043 #include "qtpropertybrowser.h"
00044
00045 QT_BEGIN_NAMESPACE
00046
00047 class QTreeWidgetItem;
00048 class QtTreePropertyBrowserPrivate;
00049
00050 class QT_QTPROPERTYBROWSER_EXPORT QtTreePropertyBrowser : public QtAbstractPropertyBrowser
00051 {
00052     Q_OBJECT
00053     Q_PROPERTY(int indentation READ indentation WRITE setIndentation)
00054     Q_PROPERTY(bool rootIsDecorated READ rootIsDecorated WRITE setRootIsDecorated)
00055     Q_PROPERTY(bool alternatingRowColors READ alternatingRowColors WRITE setAlternatingRowColors)
00056     Q_PROPERTY(bool headerVisible READ isHeaderVisible WRITE setHeaderVisible)
00057     Q_PROPERTY(ResizeMode resizeMode READ resizeMode WRITE setResizeMode)
00058     Q_PROPERTY(int splitterPosition READ splitterPosition WRITE setSplitterPosition)
00059     Q_PROPERTY(bool propertiesWithoutValueMarked READ propertiesWithoutValueMarked WRITE
setPropertiesWithoutValueMarked)
00060 public:
00061
00062     enum ResizeMode
00063     {
00064         Interactive,
00065         Stretch,
00066         Fixed,
00067         ResizeToContents
00068     };
00069     Q_ENUM(ResizeMode)
00070
00071     QtTreePropertyBrowser(QWidget *parent = 0);
00072     ~QtTreePropertyBrowser();
00073
00074     int indentation() const;
00075     void setIndentation(int i);
00076
00077     bool rootIsDecorated() const;
00078     void setRootIsDecorated(bool show);
00079
00080     bool alternatingRowColors() const;
00081     void setAlternatingRowColors(bool enable);
00082
00083     bool isHeaderVisible() const;
00084     void setHeaderVisible(bool visible);
00085
00086     ResizeMode resizeMode() const;
00087     void setResizeMode(ResizeMode mode);
00088
00089     int splitterPosition() const;
00090     void setSplitterPosition(int position);
00091
00092     void setExpanded(QTreeWidgetItem *item, bool expanded);
00093     bool isExpanded(QTreeWidgetItem *item) const;
00094
00095     bool isItemVisible(QTreeWidgetItem *item) const;
00096     void setItemVisible(QTreeWidgetItem *item, bool visible);
00097
00098     void setBackgroundColor(QTreeWidgetItem *item, const QColor &color);
00099     QColor backgroundColor(QTreeWidgetItem *item) const;
00100     QColor calculatedBackgroundColor(QTreeWidgetItem *item) const;
00101
00102     void setPropertiesWithoutValueMarked(bool mark);
00103     bool propertiesWithoutValueMarked() const;

```



```

00104
00105     void editItem(QtBrowserItem *item);
00106
00107 Q_SIGNALS:
00108
00109     void collapsed(QtBrowserItem *item);
00110     void expanded(QtBrowserItem *item);
00111
00112 protected:
00113     void itemInserted(QtBrowserItem *item, QtBrowserItem *afterItem) override;
00114     void itemRemoved(QtBrowserItem *item) override;
00115     void itemChanged(QtBrowserItem *item) override;
00116
00117 private:
00118
00119     QScopedPointer<QtTreePropertyBrowserPrivate> d_ptr;
00120     Q_DECLARE_PRIVATE(QtTreePropertyBrowser)
00121     Q_DISABLE_COPY_MOVE(QtTreePropertyBrowser)
00122
00123     Q_PRIVATE_SLOT(d_func(), void slotCollapsed(const QModelIndex &))
00124     Q_PRIVATE_SLOT(d_func(), void slotExpanded(const QModelIndex &))
00125     Q_PRIVATE_SLOT(d_func(), void slotCurrentBrowserItemChanged(QtBrowserItem *))
00126     Q_PRIVATE_SLOT(d_func(), void slotCurrentTreeItemChanged(QTreeWidgetItem *, QTreeWidgetItem *))
00127
00128 };
00129
00130 QT_END_NAMESPACE
00131
00132 #endif

```

11.84 qtvariantproperty.h

```

00001 /*****
00002 **
00003 ** Copyright (C) 2016 The Qt Company Ltd.
00004 ** Contact: https://www.qt.io/licensing/
00005 **
00006 ** This file is part of the tools applications of the Qt Toolkit.
00007 **
00008 ** $QT_BEGIN_LICENSE:LGPL$
00009 ** Commercial License Usage
00010 ** Licensees holding valid commercial Qt licenses may use this file in
00011 ** accordance with the commercial license agreement provided with the
00012 ** Software or, alternatively, in accordance with the terms contained in
00013 ** a written agreement between you and The Qt Company. For licensing terms
00014 ** and conditions see https://www.qt.io/terms-conditions. For further
00015 ** information use the contact form at https://www.qt.io/contact-us.
00016 **
00017 ** GNU Lesser General Public License Usage
00018 ** Alternatively, this file may be used under the terms of the GNU Lesser
00019 ** General Public License version 3 as published by the Free Software
00020 ** Foundation and appearing in the file LICENSE.LGPL3 included in the
00021 ** packaging of this file. Please review the following information to
00022 ** ensure the GNU Lesser General Public License version 3 requirements
00023 ** will be met: https://www.gnu.org/licenses/lgpl-3.0.html.
00024 **
00025 ** GNU General Public License Usage
00026 ** Alternatively, this file may be used under the terms of the GNU
00027 ** General Public License version 2.0 or (at your option) the GNU General
00028 ** Public License version 3 or any later version approved by the KDE Free
00029 ** Qt Foundation. The licenses are as published by the Free Software
00030 ** Foundation and appearing in the file LICENSE.GPL2 and LICENSE.GPL3
00031 ** included in the packaging of this file. Please review the following
00032 ** information to ensure the GNU General Public License requirements will
00033 ** be met: https://www.gnu.org/licenses/gpl-2.0.html and
00034 ** https://www.gnu.org/licenses/gpl-3.0.html.
00035 **
00036 ** $QT_END_LICENSE$
00037 **
00038 *****/
00039
00040 #ifndef QTVARIANTPROPERTY_H
00041 #define QTVARIANTPROPERTY_H
00042
00043 #include "qtpropertybrowser.h"
00044 #include <QtCore/QVariant>
00045 #include <QtGui/QIcon>
00046
00047 QT_BEGIN_NAMESPACE
00048
00049 class QRegularExpression;
00050
00051 typedef QMap<int, QIcon> QtIconMap;

```

```

00052
00053 class QtVariantPropertyManager;
00054
00055 class QT_QTPROPERTYBROWSER_EXPORT QtVariantProperty : public QtProperty
00056 {
00057 public:
00058     ~QtVariantProperty();
00059     QVariant value() const;
00060     QVariant attributeValue(const QString &attribute) const;
00061     int valueType() const;
00062     int propertyType() const;
00063
00064     void setValue(const QVariant &value);
00065     void setAttribute(const QString &attribute, const QVariant &value);
00066 protected:
00067     QtVariantProperty(QtVariantPropertyManager *manager);
00068 private:
00069     friend class QtVariantPropertyManager;
00070     QScopedPointer<class QtVariantPropertyPrivate> d_ptr;
00071 };
00072
00073 class QT_QTPROPERTYBROWSER_EXPORT QtVariantPropertyManager : public QtAbstractPropertyManager
00074 {
00075     Q_OBJECT
00076 public:
00077     QtVariantPropertyManager(QObject *parent = 0);
00078     ~QtVariantPropertyManager();
00079
00080     virtual QtVariantProperty *addProperty(int propertyType, const QString &name = QString());
00081
00082     int propertyType(const QtProperty *property) const;
00083     int valueType(const QtProperty *property) const;
00084     QtVariantProperty *variantProperty(const QtProperty *property) const;
00085
00086     virtual bool isPropertyTypeSupported(int propertyType) const;
00087     virtual int valueType(int propertyType) const;
00088     virtual QStringList attributes(int propertyType) const;
00089     virtual int attributeType(int propertyType, const QString &attribute) const;
00090
00091     virtual QVariant value(const QtProperty *property) const;
00092     virtual QVariant attributeValue(const QtProperty *property, const QString &attribute) const;
00093
00094     static int enumTypeId();
00095     static int flagTypeId();
00096     static int groupTypeId();
00097     static int iconMapTypeId();
00098 public Q_SLOTS:
00099     virtual void setValue(QtProperty *property, const QVariant &val);
00100     virtual void setAttribute(QtProperty *property,
00101                             const QString &attribute, const QVariant &value);
00102 Q_SIGNALS:
00103     void valueChanged(QtProperty *property, const QVariant &val);
00104     void attributeChanged(QtProperty *property,
00105                          const QString &attribute, const QVariant &val);
00106 protected:
00107     bool hasValue(const QtProperty *property) const override;
00108     QString valueText(const QtProperty *property) const override;
00109     QIcon valueIcon(const QtProperty *property) const override;
00110     void initializeProperty(QtProperty *property) override;
00111     void uninitializeProperty(QtProperty *property) override;
00112     QtProperty *createProperty() override;
00113 private:
00114     QScopedPointer<class QtVariantPropertyManagerPrivate> d_ptr;
00115     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, int))
00116     Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, int, int))
00117     Q_PRIVATE_SLOT(d_func(), void slotSingleStepChanged(QtProperty *, int))
00118     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, double))
00119     Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, double, double))
00120     Q_PRIVATE_SLOT(d_func(), void slotSingleStepChanged(QtProperty *, double))
00121     Q_PRIVATE_SLOT(d_func(), void slotDecimalsChanged(QtProperty *, int))
00122     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, bool))
00123     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QString &))
00124     Q_PRIVATE_SLOT(d_func(), void slotRegExpChanged(QtProperty *, const QRegularExpression &))
00125     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, QDate))
00126     Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, QDate, QDate))
00127     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, QTime))
00128     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QDateTime &))
00129     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QKeySequence &))
00130     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QChar &))
00131     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QLocale &))
00132     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QPoint &))
00133     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QPointF &))
00134     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QSize &))
00135     Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, const QSize &, const QSize &))
00136     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QSizeF &))
00137     Q_PRIVATE_SLOT(d_func(), void slotRangeChanged(QtProperty *, const QSizeF &, const QSizeF &))
00138     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QRect &))

```

```

00139     Q_PRIVATE_SLOT(d_func(), void slotConstraintChanged(QtProperty *, const QRect &))
00140     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QRectF &))
00141     Q_PRIVATE_SLOT(d_func(), void slotConstraintChanged(QtProperty *, const QRectF &))
00142     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QColor &))
00143     Q_PRIVATE_SLOT(d_func(), void slotEnumNamesChanged(QtProperty *, const QStringList &))
00144     Q_PRIVATE_SLOT(d_func(), void slotEnumIconsChanged(QtProperty *, const QMap<int, QIcon> &))
00145     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QSizePolicy &))
00146     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QFont &))
00147     Q_PRIVATE_SLOT(d_func(), void slotValueChanged(QtProperty *, const QCursor &))
00148     Q_PRIVATE_SLOT(d_func(), void slotFlagNamesChanged(QtProperty *, const QStringList &))
00149
00150     Q_PRIVATE_SLOT(d_func(), void slotPropertyInserted(QtProperty *, QtProperty *, QtProperty *))
00151     Q_PRIVATE_SLOT(d_func(), void slotPropertyRemoved(QtProperty *, QtProperty *))
00152     Q_DECLARE_PRIVATE(QtVariantPropertyManager)
00153     Q_DISABLE_COPY_MOVE(QtVariantPropertyManager)
00154 };
00155
00156 class QT_QTPROPERTYBROWSER_EXPORT QtVariantEditorFactory : public
    QtAbstractEditorFactory<QtVariantPropertyManager>
00157 {
00158     Q_OBJECT
00159 public:
00160     QtVariantEditorFactory(QObject *parent = 0);
00161     ~QtVariantEditorFactory();
00162 protected:
00163     void connectPropertyManager(QtVariantPropertyManager *manager) override;
00164     QWidget *createEditor(QtVariantPropertyManager *manager, QtProperty *property,
00165         QWidget *parent) override;
00166     void disconnectPropertyManager(QtVariantPropertyManager *manager) override;
00167 private:
00168     QScopedPointer<class QtVariantEditorFactoryPrivate> d_ptr;
00169     Q_DECLARE_PRIVATE(QtVariantEditorFactory)
00170     Q_DISABLE_COPY_MOVE(QtVariantEditorFactory)
00171 };
00172
00173 QT_END_NAMESPACE
00174
00175 Q_DECLARE_METATYPE(QIcon)
00176 Q_DECLARE_METATYPE(QIconMap)
00177 #endif

```

11.85 qcustomplot.h

```

00001 /*****
00002 **
00003 ** QCustomPlot, an easy to use, modern plotting widget for Qt
00004 ** Copyright (C) 2011-2021 Emanuel Eichhammer
00005 **
00006 ** This program is free software: you can redistribute it and/or modify
00007 ** it under the terms of the GNU General Public License as published by
00008 ** the Free Software Foundation, either version 3 of the License, or
00009 ** (at your option) any later version.
00010 **
00011 ** This program is distributed in the hope that it will be useful,
00012 ** but WITHOUT ANY WARRANTY; without even the implied warranty of
00013 ** MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00014 ** GNU General Public License for more details.
00015 **
00016 ** You should have received a copy of the GNU General Public License
00017 ** along with this program. If not, see http://www.gnu.org/licenses/.
00018 **
00019 *****/
00020 ** Author: Emanuel Eichhammer
00021 ** Website/Contact: http://www.qcustomplot.com/
00022 ** Date: 29.03.21
00023 ** Version: 2.1.0
00024 *****/
00025
00026 // https://www.qcustomplot.com/documentation/index.html
00027
00028 #ifndef QCUSTOMPLOT_H
00029 #define QCUSTOMPLOT_H
00030
00031 #include <QtCore/qglobal.h>
00032
00033 // some Qt version/configuration dependent macros to include or exclude certain code paths:
00034 #ifndef QCUSTOMPLOT_USE_OPENGL
00035 #if QT_VERSION < QT_VERSION_CHECK(5, 0, 0)
00036 #define QCP_OPENGL_PBUFFER
00037 #else
00038 #define QCP_OPENGL_FBO
00039 #endif
00040 #if QT_VERSION >= QT_VERSION_CHECK(5, 3, 0)

```

```
00041 #define QCP_OPENGL_OFFSCREENSURFACE
00042 #endif
00043 #endif
00044
00045 #if QT_VERSION >= QT_VERSION_CHECK(5, 4, 0)
00046 #define QCP_DEVICEPIXELRATIO_SUPPORTED
00047 #if QT_VERSION >= QT_VERSION_CHECK(5, 6, 0)
00048 #define QCP_DEVICEPIXELRATIO_FLOAT
00049 #endif
00050 #endif
00051
00052 #include <QtCore/QObject>
00053 #include <QtCore/QPointer>
00054 #include <QtCore/QSharedPointer>
00055 #include <QtCore/Qtimer>
00056 #include <QtGui/QPainter>
00057 #include <QtGui/QPainterPath>
00058 #include <QtGui/QPaintEvent>
00059 #include <QtGui/QMouseEvent>
00060 #include <QtGui/QWheelEvent>
00061 #include <QtGui/QPixmap>
00062 #include <QtCore/QVector>
00063 #include <QtCore/QString>
00064 #include <QtCore/QDateTime>
00065 #include <QtCore/QMultiMap>
00066 #include <QtCore/QFlags>
00067 #include <QtCore/QDebug>
00068 #include <QtCore/QStack>
00069 #include <QtCore/QCache>
00070 #include <QtCore/QMargins>
00071 #include <qmath.h>
00072 #include <limits>
00073 #include <algorithm>
00074 #ifdef QCP_OPENGL_FBO
00075 #include <QtGui/QOpenGLContext>
00076 #if QT_VERSION < QT_VERSION_CHECK(6, 0, 0)
00077 #include <QtGui/QOpenGLFramebufferObject>
00078 #else
00079 #include <QOpenGLFramebufferObject>
00080 #include <QOpenGLPaintDevice>
00081 #endif
00082 #ifdef QCP_OPENGL_OFFSCREENSURFACE
00083 #include <QtGui/QOffscreenSurface>
00084 #else
00085 #include <QtGui/QWindow>
00086 #endif
00087 #endif
00088 #ifdef QCP_OPENGL_PBUFFER
00089 #include <QtOpenGL/QGLPixelBuffer>
00090 #endif
00091 #if QT_VERSION < QT_VERSION_CHECK(5, 0, 0)
00092 #include <qnumeric.h>
00093 #include <QtGui/QWidget>
00094 #include <QtGui/QPrinter>
00095 #include <QtGui/QPrintEngine>
00096 #else
00097 #include <QtNumeric>
00098 #include <QtWidgets/QWidget>
00099 #include <QtPrintSupport/QtPrintSupport>
00100 #endif
00101 #if QT_VERSION >= QT_VERSION_CHECK(4, 8, 0)
00102 #include <QtCore/QElapsedTimer>
00103 #endif
00104 #if QT_VERSION >= QT_VERSION_CHECK(5, 2, 0)
00105 #include <QtCore/QTimeZone>
00106 #endif
00107
00108 class QCPPainter;
00109 class QCustomPlot;
00110 class QCPLayerable;
00111 class QCPLayoutElement;
00112 class QCPLayout;
00113 class QCPAxis;
00114 class QCPAxisRect;
00115 class QCPAxisPainterPrivate;
00116 class QCPAbstractPlottable;
00117 class QCPGraph;
00118 class QCPAbstractItem;
00119 class QCPPlottableInterface1D;
00120 class QCPLegend;
00121 class QCPItemPosition;
00122 class QCPLayer;
00123 class QCPAbstractLegendItem;
00124 class QCPSelectionRect;
00125 class QCPColorMap;
00126 class QCPColorScale;
00127 class QCPBars;
```

```

00128 class QCPPolarAxisRadial;
00129 class QCPPolarAxisAngular;
00130 class QCPPolarGrid;
00131 class QCPPolarGraph;
00132
00133 /* including file 'src/global.h' */
00134 /* modified 2021-03-29T02:30:44, size 16981 */
00135
00136 #define QCUSTOMPLOT_VERSION_STR "2.1.0"
00137 #define QCUSTOMPLOT_VERSION 0x020100
00138
00139 // decl definitions for shared library compilation/usage:
00140 #if defined(QT_STATIC_BUILD)
00141 #define QCP_LIB_DECL
00142 #elif defined(QCUSTOMPLOT_COMPILE_LIBRARY)
00143 #define QCP_LIB_DECL Q_DECL_EXPORT
00144 #elif defined(QCUSTOMPLOT_USE_LIBRARY)
00145 #define QCP_LIB_DECL Q_DECL_IMPORT
00146 #else
00147 #define QCP_LIB_DECL
00148 #endif
00149
00150 // define empty macro for Q_DECL_OVERRIDE if it doesn't exist (Qt < 5)
00151 #ifndef Q_DECL_OVERRIDE
00152 #define Q_DECL_OVERRIDE
00153 #endif
00154
00161 #ifndef Q_MOC_RUN
00162 namespace QCP {
00163 #else
00164
00165 class QCP { // when in moc-run, make it look like a class, so we get Q_GADGET, Q_ENUMS/Q_FLAGS
    features in namespace
00166
00167     Q_GADGET
00168     Q_ENUMS(ExportPen)
00169     Q_ENUMS(ResolutionUnit)
00170     Q_ENUMS(SignDomain)
00171     Q_ENUMS(MarginSide)
00172     Q_FLAGS(MarginSides)
00173     Q_ENUMS(AntialiasedElement)
00174     Q_FLAGS(AntialiasedElements)
00175     Q_ENUMS(PlottingHint)
00176     Q_FLAGS(PlottingHints)
00177     Q_ENUMS(Interaction)
00178     Q_FLAGS(Interactions)
00179     Q_ENUMS(SelectionRectMode)
00180     Q_ENUMS(SelectionType)
00181 public:
00182 #endif
00183
00190     enum ResolutionUnit {
00191         ruDotsPerMeter
00192         , ruDotsPerCentimeter
00193         , ruDotsPerInch
00194     };
00195
00201     enum ExportPen {
00202         epNoCosmetic
00203         , epAllowCosmetic
00204     };
00205
00213     enum SignDomain {
00214         sdNegative
00215         , sdBoth
00216         , sdPositive
00217     };
00218
00224     enum MarginSide {
00225         msLeft = 0x01
00226         , msRight = 0x02
00227         , msTop = 0x04
00228         , msBottom = 0x08
00229         , msAll = 0xFF
00230         , msNone = 0x00
00231     };
00232
00233     Q_DECLARE_FLAGS(MarginSides, MarginSide)
00234
00235
00244     enum AntialiasedElement {
00245         aeAxes = 0x0001
00246         , aeGrid = 0x0002
00247         , aeSubGrid = 0x0004
00248         , aeLegend = 0x0008
00249         , aeLegendItems = 0x0010
00250         , aePlottables = 0x0020

```

```

00251         , aeItems = 0x0040
00252         , aeScatters = 0x0080
00253         , aeFills = 0x0100
00254         , aeZeroLine = 0x0200
00255         , aeOther = 0x8000
00256         , aeAll = 0xFFFF
00257         , aeNone = 0x0000
00258     };
00259
00260     Q_DECLARE_FLAGS(AntialiasedElements, AntialiasedElement)
00261
00262
00263     enum PlottingHint {
00264         phNone = 0x000
00265         , phFastPolylines = 0x001
00266         , phImmediateRefresh = 0x002
00267         , phCacheLabels = 0x004
00268     };
00269
00270     Q_DECLARE_FLAGS(PlottingHints, PlottingHint)
00271
00272
00273     enum Interaction {
00274         iNone = 0x000
00275         , iRangeDrag = 0x001
00276         , iRangeZoom = 0x002
00277         , iMultiSelect = 0x004
00278         , iSelectPlottables = 0x008
00279         , iSelectAxes = 0x010
00280         , iSelectLegend = 0x020
00281         , iSelectItems = 0x040
00282         , iSelectOther = 0x080
00283         , iSelectPlottablesBeyondAxisRect = 0x100
00284     };
00285
00286     Q_DECLARE_FLAGS(Interactions, Interaction)
00287
00288
00289     enum SelectionRectMode {
00290         srmNone
00291         , srmZoom
00292         , srmSelect
00293         , srmCustom
00294     };
00295
00296
00297     enum SelectionType {
00298         stNone
00299         , stWhole
00300         , stSingleData
00301         , stDataRange
00302         , stMultipleDataRanges
00303     };
00304
00305     inline bool isValidData(double value) {
00306         return !qIsNaN(value) || !qIsInf(value);
00307     }
00308
00309     inline bool isValidData(double value1, double value2) {
00310         return isValidData(value1) || isValidData(value2);
00311     }
00312
00313     inline void setMarginValue(QMargins &margins, QCP::MarginSide side, int value) {
00314         switch (side) {
00315             case QCP::msLeft: margins.setLeft(value);
00316                             break;
00317             case QCP::msRight: margins.setRight(value);
00318                             break;
00319             case QCP::msTop: margins.setTop(value);
00320                             break;
00321             case QCP::msBottom: margins.setBottom(value);
00322                             break;
00323             case QCP::msAll: margins = QMargins(value, value, value, value);
00324                             break;
00325             default: break;
00326         }
00327     }
00328
00329     inline int getMarginValue(const QMargins &margins, QCP::MarginSide side) {
00330         switch (side) {
00331             case QCP::msLeft: return margins.left();
00332             case QCP::msRight: return margins.right();
00333             case QCP::msTop: return margins.top();
00334             case QCP::msBottom: return margins.bottom();
00335             default: break;
00336         }
00337         return 0;
00338     }

```

```

00396
00397
00398     extern const QMetaObject staticMetaObject; // in moc-run we create a static meta object for QCP
00399     "fake" object. This line is the link to it via QCP::staticMetaObject in normal operation as namespace
00400 } // end of namespace QCP
00401
00402 Q_DECLARE_OPERATORS_FOR_FLAGS(QCP::AntialiasedElements)
00403 Q_DECLARE_OPERATORS_FOR_FLAGS(QCP::PlottingHints)
00404 Q_DECLARE_OPERATORS_FOR_FLAGS(QCP::MarginSides)
00405 Q_DECLARE_OPERATORS_FOR_FLAGS(QCP::Interactions)
00406 Q_DECLARE_METATYPE(QCP::ExportPen)
00407 Q_DECLARE_METATYPE(QCP::ResolutionUnit)
00408 Q_DECLARE_METATYPE(QCP::SignDomain)
00409 Q_DECLARE_METATYPE(QCP::MarginSide)
00410 Q_DECLARE_METATYPE(QCP::AntialiasedElement)
00411 Q_DECLARE_METATYPE(QCP::PlottingHint)
00412 Q_DECLARE_METATYPE(QCP::Interaction)
00413 Q_DECLARE_METATYPE(QCP::SelectionRectMode)
00414 Q_DECLARE_METATYPE(QCP::SelectionType)
00415
00416 /* end of 'src/global.h' */
00417
00418
00419 /* including file 'src/vector2d.h' */
00420 /* modified 2021-03-29T02:30:44, size 4988 */
00421
00422 class QCP_LIB_DECL QCPVector2D {
00423 public:
00424     QCPVector2D();
00425     QCPVector2D(double x, double y);
00426     QCPVector2D(const QPoint &point);
00427     QCPVector2D(const QPointF &point);
00428
00429     // getters:
00430
00431     double x() const {
00432         return mX;
00433     }
00434
00435     double y() const {
00436         return mY;
00437     }
00438
00439     double &rx() {
00440         return mX;
00441     }
00442
00443     double &ry() {
00444         return mY;
00445     }
00446
00447     // setters:
00448
00449     void setX(double x) {
00450         mX = x;
00451     }
00452
00453     void setY(double y) {
00454         mY = y;
00455     }
00456
00457     // non-virtual methods:
00458
00459     double length() const {
00460         return qSqrt(mX * mX + mY * mY);
00461     }
00462
00463     double lengthSquared() const {
00464         return mX * mX + mY * mY;
00465     }
00466
00467     double angle() const {
00468         return qAtan2(mY, mX);
00469     }
00470
00471     QPoint toPoint() const {
00472         return QPoint(int(mX), int(mY));
00473     }
00474
00475     QPointF toPointF() const {
00476         return QPointF(mX, mY);
00477     }
00478
00479     bool isNull() const {
00480         return qIsNull(mX) && qIsNull(mY);
00481     }

```

```

00482     void normalize();
00483     QCPVector2D normalized() const;
00484
00485     QCPVector2D perpendicular() const {
00486         return QCPVector2D(-mY, mX);
00487     }
00488
00489     double dot(const QCPVector2D &vec) const {
00490         return mX * vec.mX + mY * vec.mY;
00491     }
00492     double distanceSquaredToLine(const QCPVector2D &start, const QCPVector2D &end) const;
00493     double distanceSquaredToLine(const QLineF &line) const;
00494     double distanceToStraightLine(const QCPVector2D &base, const QCPVector2D &direction) const;
00495
00496     QCPVector2D &operator*=(double factor);
00497     QCPVector2D &operator/=(double divisor);
00498     QCPVector2D &operator+=(const QCPVector2D &vector);
00499     QCPVector2D &operator-=(const QCPVector2D &vector);
00500
00501 private:
00502     // property members:
00503     double mX, mY;
00504
00505     friend inline const QCPVector2D operator*(double factor, const QCPVector2D &vec);
00506     friend inline const QCPVector2D operator*(const QCPVector2D &vec, double factor);
00507     friend inline const QCPVector2D operator/(const QCPVector2D &vec, double divisor);
00508     friend inline const QCPVector2D operator+(const QCPVector2D &vec1, const QCPVector2D &vec2);
00509     friend inline const QCPVector2D operator-(const QCPVector2D &vec1, const QCPVector2D &vec2);
00510     friend inline const QCPVector2D operator-(const QCPVector2D &vec);
00511 };
00512 Q_DECLARE_TYPEINFO(QCPVector2D, Q_MOVABLE_TYPE);
00513
00514 inline const QCPVector2D operator*(double factor, const QCPVector2D &vec) {
00515     return QCPVector2D(vec.mX*factor, vec.mY * factor);
00516 }
00517
00518 inline const QCPVector2D operator*(const QCPVector2D &vec, double factor) {
00519     return QCPVector2D(vec.mX*factor, vec.mY * factor);
00520 }
00521
00522 inline const QCPVector2D operator/(const QCPVector2D &vec, double divisor) {
00523     return QCPVector2D(vec.mX / divisor, vec.mY / divisor);
00524 }
00525
00526 inline const QCPVector2D operator+(const QCPVector2D &vec1, const QCPVector2D &vec2) {
00527     return QCPVector2D(vec1.mX + vec2.mX, vec1.mY + vec2.mY);
00528 }
00529
00530 inline const QCPVector2D operator-(const QCPVector2D &vec1, const QCPVector2D &vec2) {
00531     return QCPVector2D(vec1.mX - vec2.mX, vec1.mY - vec2.mY);
00532 }
00533
00534 inline const QCPVector2D operator-(const QCPVector2D &vec) {
00535     return QCPVector2D(-vec.mX, -vec.mY);
00536 }
00537
00542 inline QDebug operator<<(QDebug d, const QCPVector2D &vec) {
00543     d.nospace() << "QCPVector2D(" << vec.x() << ", " << vec.y() << ")";
00544     return d.space();
00545 }
00546
00547 /* end of 'src/vector2d.h' */
00548
00549
00550 /* including file 'src/painter.h' */
00551
00552 /* modified 2021-03-29T02:30:44, size 4035 */
00553
00554 class QCP_LIB_DECL QCPPainter : public QPainter {
00555     Q_GADGET
00556 public:
00562     enum PainterMode {
00563         pmDefault = 0x00
00564         , pmVectorized = 0x01
00565         , pmNoCaching = 0x02
00566         , pmNonCosmetic = 0x04
00567     };
00568     Q_ENUMS(PainterMode)
00569     Q_FLAGS(PainterModes)
00570     Q_DECLARE_FLAGS(PainterModes, PainterMode)
00571
00572     QCPPainter();
00573     explicit QCPPainter(QPaintDevice *device);
00574
00575     // getters:
00576

```



```

00577     bool antialiasing() const {
00578         return testRenderHint(QPainter::Antialiasing);
00579     }
00580
00581     PainterModes modes() const {
00582         return mModes;
00583     }
00584
00585     // setters:
00586     void setAntialiasing(bool enabled);
00587     void setMode(PainterMode mode, bool enabled = true);
00588     void setModes(PainterModes modes);
00589
00590     // methods hiding non-virtual base class functions (QPainter bug workarounds):
00591     bool begin(QPaintDevice *device);
00592     void setPen(const QPen &pen);
00593     void setPen(const QColor &color);
00594     void setPen(Qt::PenStyle penStyle);
00595     void drawLine(const QLineF &line);
00596
00597     void drawLine(const QPointF &p1, const QPointF &p2) {
00598         drawLine(QLineF(p1, p2));
00599     }
00600     void save();
00601     void restore();
00602
00603     // non-virtual methods:
00604     void makeNonCosmetic();
00605
00606 protected:
00607     // property members:
00608     PainterModes mModes;
00609     bool mIsAntialiasing;
00610
00611     // non-property members:
00612     QStack<bool> mAntialiasingStack;
00613 };
00614
00615 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPainter::PainterModes)
00616 Q_DECLARE_METATYPE(QCPainter::PainterMode)
00617
00618 /* end of 'src/painter.h' */
00619
00620
00621 /* including file 'src/paintbuffer.h' */
00622 /* modified 2021-03-29T02:30:44, size 5006 */
00623
00624 class QCP_LIB_DECL QCPAbstractPaintBuffer {
00625 public:
00626     explicit QCPAbstractPaintBuffer(const QSize &size, double devicePixelRatio);
00627     virtual ~QCPAbstractPaintBuffer();
00628
00629     // getters:
00630
00631     QSize size() const {
00632         return mSize;
00633     }
00634
00635     bool invalidated() const {
00636         return mInvalidated;
00637     }
00638
00639     double devicePixelRatio() const {
00640         return mDevicePixelRatio;
00641     }
00642
00643     // setters:
00644     void setSize(const QSize &size);
00645     void setInvalidated(bool invalidated = true);
00646     void setDevicePixelRatio(double ratio);
00647
00648     // introduced virtual methods:
00649     virtual QCPainter *startPainting() = 0;
00650
00651     virtual void donePainting() {
00652     }
00653     virtual void draw(QCPainter *painter) const = 0;
00654     virtual void clear(const QColor &color) = 0;
00655
00656 protected:
00657     // property members:
00658     QSize mSize;
00659     double mDevicePixelRatio;
00660
00661     // non-property members:
00662     bool mInvalidated;
00663

```

```

00664     // introduced virtual methods:
00665     virtual void reallocateBuffer() = 0;
00666 };
00667
00668 class QCP_LIB_DECL QCPPaintBufferPixmap : public QCPAbstractPaintBuffer {
00669 public:
00670     explicit QCPPaintBufferPixmap(const QSize &size, double devicePixelRatio);
00671     virtual ~QCPPaintBufferPixmap() Q_DECL_OVERRIDE;
00672
00673     // reimplemented virtual methods:
00674     virtual QCPPainter *startPainting() Q_DECL_OVERRIDE;
00675     virtual void draw(QCPPainter *painter) const Q_DECL_OVERRIDE;
00676     void clear(const QColor &color) Q_DECL_OVERRIDE;
00677
00678 protected:
00679     // non-property members:
00680     QPixmap mBuffer;
00681
00682     // reimplemented virtual methods:
00683     virtual void reallocateBuffer() Q_DECL_OVERRIDE;
00684 };
00685
00686 #ifndef QCP_OPENGL_PBUFFER
00687
00688 class QCP_LIB_DECL QCPPaintBufferGLPbuffer : public QCPAbstractPaintBuffer {
00689 public:
00690     explicit QCPPaintBufferGLPbuffer(const QSize &size, double devicePixelRatio, int multisamples);
00691     virtual ~QCPPaintBufferGLPbuffer() Q_DECL_OVERRIDE;
00692
00693     // reimplemented virtual methods:
00694     virtual QCPPainter *startPainting() Q_DECL_OVERRIDE;
00695     virtual void draw(QCPPainter *painter) const Q_DECL_OVERRIDE;
00696     void clear(const QColor &color) Q_DECL_OVERRIDE;
00697
00698 protected:
00699     // non-property members:
00700     QGLPixelBuffer *mGLPbuffer;
00701     int mMultisamples;
00702
00703     // reimplemented virtual methods:
00704     virtual void reallocateBuffer() Q_DECL_OVERRIDE;
00705 };
00706 #endif // QCP_OPENGL_PBUFFER
00707
00708 #ifndef QCP_OPENGL_FBO
00709
00710 class QCP_LIB_DECL QCPPaintBufferGLFbo : public QCPAbstractPaintBuffer {
00711 public:
00712     explicit QCPPaintBufferGLFbo(const QSize &size, double devicePixelRatio,
00713     QWeakPointer<QOpenGLContext> glContext, QWeakPointer<QOpenGLPaintDevice> glPaintDevice);
00714     virtual ~QCPPaintBufferGLFbo() Q_DECL_OVERRIDE;
00715
00716     // reimplemented virtual methods:
00717     virtual QCPPainter *startPainting() Q_DECL_OVERRIDE;
00718     virtual void donePainting() Q_DECL_OVERRIDE;
00719     virtual void draw(QCPPainter *painter) const Q_DECL_OVERRIDE;
00720     void clear(const QColor &color) Q_DECL_OVERRIDE;
00721
00722 protected:
00723     // non-property members:
00724     QWeakPointer<QOpenGLContext> mGlContext;
00725     QWeakPointer<QOpenGLPaintDevice> mGlPaintDevice;
00726     QOpenGLFramebufferObject *mGlFramebuffer;
00727
00728     // reimplemented virtual methods:
00729     virtual void reallocateBuffer() Q_DECL_OVERRIDE;
00730 };
00731 #endif // QCP_OPENGL_FBO
00732
00733 /* end of 'src/paintbuffer.h' */
00734
00735 /* including file 'src/layer.h' */
00736
00737 /* modified 2021-03-29T02:30:44, size 7038 */
00738
00739 class QCP_LIB_DECL QCPLayer : public QObject {
00740
00741     Q_OBJECT
00742     Q_PROPERTY(QCustomPlot* parentPlot READ parentPlot)
00743     Q_PROPERTY(QString name READ name)
00744     Q_PROPERTY(int index READ index)
00745     Q_PROPERTY(List<QCPLayerable*> children READ children)
00746     Q_PROPERTY(bool visible READ visible WRITE setVisible)
00747     Q_PROPERTY(LayerMode mode READ mode WRITE setMode)
00748
00749

```

```

00752 public:
00753
00761     enum LayerMode {
00762         lmLogical
00763         , lmBuffered
00764     };
00765     Q_ENUMS(LayerMode)
00766
00767     QCPLayer(QCustomPlot* parentPlot, const QString &layerName);
00768     virtual ~QCPLayer();
00769
00770     // getters:
00771
00772     QCustomPlot *parentPlot() const {
00773         return mParentPlot;
00774     }
00775
00776     QString name() const {
00777         return mName;
00778     }
00779
00780     int index() const {
00781         return mIndex;
00782     }
00783
00784     QList<QCPLayerable*> children() const {
00785         return mChildren;
00786     }
00787
00788     bool visible() const {
00789         return mVisible;
00790     }
00791
00792     LayerMode mode() const {
00793         return mMode;
00794     }
00795
00796     // setters:
00797     void setVisible(bool visible);
00798     void setMode(LayerMode mode);
00799
00800     // non-virtual methods:
00801     void replot();
00802
00803 protected:
00804     // property members:
00805     QCustomPlot *mParentPlot;
00806     QString mName;
00807     int mIndex;
00808     QList<QCPLayerable*> mChildren;
00809     bool mVisible;
00810     LayerMode mMode;
00811
00812     // non-property members:
00813     QWeakPointer<QCPAbstractPaintBuffer> mPaintBuffer;
00814
00815     // non-virtual methods:
00816     void draw(QCPPainter *painter);
00817     void drawToPaintBuffer();
00818     void addChild(QCPLayerable *layerable, bool prepend);
00819     void removeChild(QCPLayerable *layerable);
00820
00821 private:
00822     Q_DISABLE_COPY(QCPLayer)
00823
00824     friend class QCustomPlot;
00825     friend class QCPLayerable;
00826 };
00827
00828 Q_DECLARE_METATYPE(QCPLayer::LayerMode)
00829
00830 class QCP_LIB_DECL QCPLayerable : public QObject {
00831     Q_OBJECT
00832     Q_PROPERTY(bool visible READ visible WRITE setVisible)
00833     Q_PROPERTY(QCustomPlot* parentPlot READ parentPlot)
00834     Q_PROPERTY(QCPLayerable* parentLayerable READ parentLayerable)
00835     Q_PROPERTY(QCPLayer* layer READ layer WRITE setLayer NOTIFY layerChanged)
00836     Q_PROPERTY(bool antialiased READ antialiased WRITE setAntialiased)
00837 public:
00838     QCPLayerable(QCustomPlot *plot, QString targetLayer = QString(), QCPLayerable *parentLayerable =
00839         nullptr);
00840     virtual ~QCPLayerable();
00841
00842     // getters:
00843
00844     bool visible() const {
00845         return mVisible;

```

```

00847     }
00848
00849     QCustomPlot *parentPlot() const {
00850         return mParentPlot;
00851     }
00852
00853     QCPLayerable *parentLayerable() const {
00854         return mParentLayerable.data();
00855     }
00856
00857     QCPLayer *layer() const {
00858         return mLayer;
00859     }
00860
00861     bool antialiased() const {
00862         return mAntialiased;
00863     }
00864
00865     // setters:
00866     void setVisible(bool on);
00867     Q_SLOT bool setLayer(QCPLayer *layer);
00868     bool setLayer(const QString &layerName);
00869     void setAntialiased(bool enabled);
00870
00871     // introduced virtual methods:
00872     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
00873     const;
00874
00875     // non-property methods:
00876     bool realVisibility() const;
00877
00878 signals:
00879     void layerChanged(QCPLayer *newLayer);
00880
00881 protected:
00882     // property members:
00883     bool mVisible;
00884     QCustomPlot *mParentPlot;
00885     QPointer<QCPLayerable> mParentLayerable;
00886     QCPLayer *mLayer;
00887     bool mAntialiased;
00888
00889     // introduced virtual methods:
00890     virtual void parentPlotInitialized(QCustomPlot *parentPlot);
00891     virtual QCP::Interaction selectionCategory() const;
00892     virtual QRect clipRect() const;
00893     virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const = 0;
00894     virtual void draw(QCPPainter *painter) = 0;
00895     // selection events:
00896     virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
00897     *selectionStateChanged);
00898     virtual void deselectEvent(bool *selectionStateChanged);
00899     // low-level mouse events:
00900     virtual void mousePressEvent(QMouseEvent *event, const QVariant &details);
00901     virtual void mouseMoveEvent(QMouseEvent *event, const QPointF &startPos);
00902     virtual void mouseReleaseEvent(QMouseEvent *event, const QPointF &startPos);
00903     virtual void mouseDoubleClickEvent(QMouseEvent *event, const QVariant &details);
00904     virtual void wheelEvent(QWheelEvent *event);
00905
00906     // non-property methods:
00907     void initializeParentPlot(QCustomPlot *parentPlot);
00908     void setParentLayerable(QCPLayerable* parentLayerable);
00909     bool moveToLayer(QCPLayer *layer, bool prepend);
00910     void applyAntialiasingHint(QCPPainter *painter, bool localAntialiased, QCP::AntialiasedElement
00911     overrideElement) const;
00912
00913 private:
00914     Q_DISABLE_COPY(QCPLayerable)
00915
00916     friend class QCustomPlot;
00917     friend class QCPLayer;
00918     friend class QCPAxisRect;
00919 };
00920
00921 /* end of 'src/layer.h' */
00922
00923 /* including file 'src/axis/range.h' */
00924
00925 /* modified 2021-03-29T02:30:44, size 5280 */
00926
00927 class QCP_LIB_DECL QCPRange {
00928 public:
00929     double lower, upper;
00930
00931     QCPRange();
00932     QCPRange(double lower, double upper);

```

```

00931
00932     bool operator==(const QCPRange& other) const {
00933         return lower == other.lower && upper == other.upper;
00934     }
00935
00936     bool operator!=(const QCPRange& other) const {
00937         return !(*this == other);
00938     }
00939
00940     QCPRange &operator+=(const double& value) {
00941         lower += value;
00942         upper += value;
00943         return *this;
00944     }
00945
00946     QCPRange &operator-=(const double& value) {
00947         lower -= value;
00948         upper -= value;
00949         return *this;
00950     }
00951
00952     QCPRange &operator*=(const double& value) {
00953         lower *= value;
00954         upper *= value;
00955         return *this;
00956     }
00957
00958     QCPRange &operator/=(const double& value) {
00959         lower /= value;
00960         upper /= value;
00961         return *this;
00962     }
00963     friend inline const QCPRange operator+(const QCPRange&, double);
00964     friend inline const QCPRange operator+(double, const QCPRange&);
00965     friend inline const QCPRange operator-(const QCPRange& range, double value);
00966     friend inline const QCPRange operator*(const QCPRange& range, double value);
00967     friend inline const QCPRange operator*(double value, const QCPRange& range);
00968     friend inline const QCPRange operator/(const QCPRange& range, double value);
00969
00970     double size() const {
00971         return upper - lower;
00972     }
00973
00974     double center() const {
00975         return (upper + lower)*0.5;
00976     }
00977
00978     void normalize() {
00979         if (lower > upper) qSwap(lower, upper);
00980     }
00981     void expand(const QCPRange &otherRange);
00982     void expand(double includeCoord);
00983     QCPRange expanded(const QCPRange &otherRange) const;
00984     QCPRange expanded(double includeCoord) const;
00985     QCPRange bounded(double lowerBound, double upperBound) const;
00986     QCPRange sanitizedForLogScale() const;
00987     QCPRange sanitizedForLinScale() const;
00988
00989     bool contains(double value) const {
00990         return value >= lower && value <= upper;
00991     }
00992
00993     static bool validRange(double lower, double upper);
00994     static bool validRange(const QCPRange &range);
00995     static const double minRange;
00996     static const double maxRange;
00997
00998 };
00999 Q_DECLARE_TYPEINFO(QCPRange, Q_MOVABLE_TYPE);
01000
01005 inline QDebug operator<<(QDebug d, const QCPRange &range) {
01006     d.nospace() << "QCPRange(" << range.lower << ", " << range.upper << ")";
01007     return d.space();
01008 }
01009
01013 inline const QCPRange operator+(const QCPRange& range, double value) {
01014     QCPRange result(range);
01015     result += value;
01016     return result;
01017 }
01018
01022 inline const QCPRange operator+(double value, const QCPRange& range) {
01023     QCPRange result(range);
01024     result += value;
01025     return result;
01026 }
01027

```

```

01031 inline const QCPRange operator-(const QCPRange& range, double value) {
01032     QCPRange result(range);
01033     result -= value;
01034     return result;
01035 }
01036
01040 inline const QCPRange operator*(const QCPRange& range, double value) {
01041     QCPRange result(range);
01042     result *= value;
01043     return result;
01044 }
01045
01049 inline const QCPRange operator*(double value, const QCPRange& range) {
01050     QCPRange result(range);
01051     result *= value;
01052     return result;
01053 }
01054
01058 inline const QCPRange operator/(const QCPRange& range, double value) {
01059     QCPRange result(range);
01060     result /= value;
01061     return result;
01062 }
01063
01064 /* end of 'src/axis/range.h' */
01065
01066
01067 /* including file 'src/selection.h' */
01068
01069 /* modified 2021-03-29T02:30:44, size 8569 */
01070
01071 class QCP_LIB_DECL QCPDataRange {
01072 public:
01073     QCPDataRange();
01074     QCPDataRange(int begin, int end);
01075
01076     bool operator==(const QCPDataRange& other) const {
01077         return mBegin == other.mBegin && mEnd == other.mEnd;
01078     }
01079
01080     bool operator!=(const QCPDataRange& other) const {
01081         return !(*this == other);
01082     }
01083
01084     // getters:
01085
01086     int begin() const {
01087         return mBegin;
01088     }
01089
01090     int end() const {
01091         return mEnd;
01092     }
01093
01094     int size() const {
01095         return mEnd - mBegin;
01096     }
01097
01098     int length() const {
01099         return size();
01100     }
01101
01102     // setters:
01103
01104     void setBegin(int begin) {
01105         mBegin = begin;
01106     }
01107
01108     void setEnd(int end) {
01109         mEnd = end;
01110     }
01111
01112     // non-property methods:
01113
01114     bool isValid() const {
01115         return (mEnd >= mBegin) && (mBegin >= 0);
01116     }
01117
01118     bool isEmpty() const {
01119         return length() == 0;
01120     }
01121
01122     QCPDataRange bounded(const QCPDataRange &other) const;
01123     QCPDataRange expanded(const QCPDataRange &other) const;
01124     QCPDataRange intersection(const QCPDataRange &other) const;
01125
01126     QCPDataRange adjusted(int changeBegin, int changeEnd) const {
01127         return QCPDataRange(mBegin + changeBegin, mEnd + changeEnd);
01128     }

```

```

01127     }
01128     bool intersects(const QCPDataRange &other) const;
01129     bool contains(const QCPDataRange &other) const;
01130
01131 private:
01132     // property members:
01133     int mBegin, mEnd;
01134
01135 };
01136 Q_DECLARE_TYPEINFO(QCPDataRange, Q_MOVABLE_TYPE);
01137
01138 class QCP_LIB_DECL QCPDataSelection {
01139 public:
01140     explicit QCPDataSelection();
01141     explicit QCPDataSelection(const QCPDataRange &range);
01142
01143     bool operator==(const QCPDataSelection& other) const;
01144
01145     bool operator!=(const QCPDataSelection& other) const {
01146         return !(*this == other);
01147     }
01148     QCPDataSelection &operator+=(const QCPDataSelection& other);
01149     QCPDataSelection &operator+=(const QCPDataRange& other);
01150     QCPDataSelection &operator-=(const QCPDataSelection& other);
01151     QCPDataSelection &operator-=(const QCPDataRange& other);
01152     friend inline const QCPDataSelection operator+(const QCPDataSelection& a, const QCPDataSelection&
b);
01153     friend inline const QCPDataSelection operator+(const QCPDataRange& a, const QCPDataSelection& b);
01154     friend inline const QCPDataSelection operator+(const QCPDataSelection& a, const QCPDataRange& b);
01155     friend inline const QCPDataSelection operator+(const QCPDataRange& a, const QCPDataRange& b);
01156     friend inline const QCPDataSelection operator-(const QCPDataSelection& a, const QCPDataSelection&
b);
01157     friend inline const QCPDataSelection operator-(const QCPDataRange& a, const QCPDataSelection& b);
01158     friend inline const QCPDataSelection operator-(const QCPDataSelection& a, const QCPDataRange& b);
01159     friend inline const QCPDataSelection operator-(const QCPDataRange& a, const QCPDataRange& b);
01160
01161     // getters:
01162
01163     int dataRangeCount() const {
01164         return mDataRanges.size();
01165     }
01166     int dataPointCount() const;
01167     QCPDataRange dataRange(int index = 0) const;
01168
01169     QList<QCPDataRange> dataRanges() const {
01170         return mDataRanges;
01171     }
01172     QCPDataRange span() const;
01173
01174     // non-property methods:
01175     void addDataRange(const QCPDataRange &dataRange, bool simplify = true);
01176     void clear();
01177
01178     bool isEmpty() const {
01179         return mDataRanges.isEmpty();
01180     }
01181     void simplify();
01182     void enforceType(QCP::SelectionType type);
01183     bool contains(const QCPDataSelection &other) const;
01184     QCPDataSelection intersection(const QCPDataRange &other) const;
01185     QCPDataSelection intersection(const QCPDataSelection &other) const;
01186     QCPDataSelection inverse(const QCPDataRange &outerRange) const;
01187
01188 private:
01189     // property members:
01190     QList<QCPDataRange> mDataRanges;
01191
01192     inline static bool lessThanDataRangeBegin(const QCPDataRange &a, const QCPDataRange &b) {
01193         return a.begin() < b.begin();
01194     }
01195 };
01196
01197 Q_DECLARE_METATYPE(QCPDataSelection)
01198
01199
01200
01204 inline const QCPDataSelection operator+(const QCPDataSelection& a, const QCPDataSelection& b) {
01205     QCPDataSelection result(a);
01206     result += b;
01207     return result;
01208 }
01209
01214 inline const QCPDataSelection operator+(const QCPDataRange& a, const QCPDataSelection& b) {
01215     QCPDataSelection result(a);
01216     result += b;
01217     return result;
01218 }

```

```

01219
01224 inline const QCPDataSelection operator+(const QCPDataSelection& a, const QCPDataRange& b) {
01225     QCPDataSelection result(a);
01226     result += b;
01227     return result;
01228 }
01229
01234 inline const QCPDataSelection operator+(const QCPDataRange& a, const QCPDataRange& b) {
01235     QCPDataSelection result(a);
01236     result += b;
01237     return result;
01238 }
01239
01243 inline const QCPDataSelection operator-(const QCPDataSelection& a, const QCPDataSelection& b) {
01244     QCPDataSelection result(a);
01245     result -= b;
01246     return result;
01247 }
01248
01252 inline const QCPDataSelection operator-(const QCPDataRange& a, const QCPDataSelection& b) {
01253     QCPDataSelection result(a);
01254     result -= b;
01255     return result;
01256 }
01257
01261 inline const QCPDataSelection operator-(const QCPDataSelection& a, const QCPDataRange& b) {
01262     QCPDataSelection result(a);
01263     result -= b;
01264     return result;
01265 }
01266
01270 inline const QCPDataSelection operator-(const QCPDataRange& a, const QCPDataRange& b) {
01271     QCPDataSelection result(a);
01272     result -= b;
01273     return result;
01274 }
01275
01280 inline QDebug operator<<(QDebug d, const QCPDataRange &dataRange) {
01281     d.nospace() << "QCPDataRange(" << dataRange.begin() << ", " << dataRange.end() << ")";
01282     return d;
01283 }
01284
01289 inline QDebug operator<<(QDebug d, const QCPDataSelection &selection) {
01290     d.nospace() << "QCPDataSelection(";
01291     for (int i = 0; i < selection.dataRangeCount(); ++i) {
01292         if (i != 0)
01293             d << ", ";
01294         d << selection.dataRange(i);
01295     }
01296     d << ")";
01297     return d;
01298 }
01299
01300
01301
01302 /* end of 'src/selection.h' */
01303
01304
01305 /* including file 'src/selectionrect.h' */
01306
01307 /* modified 2021-03-29T02:30:44, size 3354 */
01308
01309 class QCP_LIB_DECL QCPSelectionRect : public QCPLayerable {
01310     Q_OBJECT
01311 public:
01312     explicit QCPSelectionRect(QCustomPlot *parentPlot);
01313     virtual ~QCPSelectionRect() Q_DECL_OVERRIDE;
01314
01315     // getters:
01316
01317     QRect rect() const {
01318         return mRect;
01319     }
01320     QCPRange range(const QCPAxis *axis) const;
01321
01322     QPen pen() const {
01323         return mPen;
01324     }
01325
01326     QBrush brush() const {
01327         return mBrush;
01328     }
01329
01330     bool isActive() const {
01331         return mActive;
01332     }
01333

```



```

01334     // setters:
01335     void setPen(const QPen &pen);
01336     void setBrush(const QBrush &brush);
01337
01338     // non-property methods:
01339     Q_SLOT void cancel();
01340
01341 signals:
01342     void started(QMouseEvent *event);
01343     void changed(const QRect &rect, QMouseEvent *event);
01344     void canceled(const QRect &rect, QInputEvent *event);
01345     void accepted(const QRect &rect, QMouseEvent *event);
01346
01347 protected:
01348     // property members:
01349     QRect mRect;
01350     QPen mPen;
01351     QBrush mBrush;
01352     // non-property members:
01353     bool mActive;
01354
01355     // introduced virtual methods:
01356     virtual void startSelection(QMouseEvent *event);
01357     virtual void moveSelection(QMouseEvent *event);
01358     virtual void endSelection(QMouseEvent *event);
01359     virtual void keyPressEvent(QKeyEvent *event);
01360
01361     // reimplemented virtual methods
01362     virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
01363     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
01364
01365     friend class QCustomPlot;
01366 };
01367
01368 /* end of 'src/selectionrect.h' */
01369
01370
01371 /* including file 'src/layout.h' */
01372
01373 /* modified 2021-03-29T02:30:44, size 14279 */
01374
01375 class QCP_LIB_DECL QCPMarginGroup : public QObject {
01376     Q_OBJECT
01377 public:
01378     explicit QCPMarginGroup(QCustomPlot *parentPlot);
01379     virtual ~QCPMarginGroup();
01380
01381     // non-virtual methods:
01382
01383     QList<QCPLayoutElement*> elements(QCP::MarginSide side) const {
01384         return mChildren.value(side);
01385     }
01386     bool isEmpty() const;
01387     void clear();
01388
01389 protected:
01390     // non-property members:
01391     QCustomPlot *mParentPlot;
01392     QHash<QCP::MarginSide, QList<QCPLayoutElement*> > mChildren;
01393
01394     // introduced virtual methods:
01395     virtual int commonMargin(QCP::MarginSide side) const;
01396
01397     // non-virtual methods:
01398     void addChild(QCP::MarginSide side, QCPLayoutElement *element);
01399     void removeChild(QCP::MarginSide side, QCPLayoutElement *element);
01400
01401 private:
01402     Q_DISABLE_COPY(QCPMarginGroup)
01403
01404     friend class QCPLayoutElement;
01405 };
01406
01407 class QCP_LIB_DECL QCPLayoutElement : public QCPLayerable {
01408     Q_OBJECT
01409     Q_PROPERTY(QCPLayout* layout READ layout)
01410     Q_PROPERTY(QRect rect READ rect)
01411     Q_PROPERTY(QRect outerRect READ outerRect WRITE setOuterRect)
01412     Q_PROPERTY(QMargins margins READ margins WRITE setMargins)
01413     Q_PROPERTY(QMargins minimumMargins READ minimumMargins WRITE setMinimumMargins)
01414     Q_PROPERTY(QSize minimumSize READ minimumSize WRITE setMinimumSize)
01415     Q_PROPERTY(QSize maximumSize READ maximumSize WRITE setMaximumSize)
01416     Q_PROPERTY(SizeConstraintRect sizeConstraintRect READ sizeConstraintRect WRITE
01417         setSizeConstraintRect)
01420 public:
01425     enum UpdatePhase {

```

```

01426         upPreparation
01427         , upMargins
01428         , upLayout
01429     };
01430
01431     Q_ENUMS(UpdatePhase)
01432
01433     enum SizeConstraintRect {
01441         scrInnerRect
01442         , scrOuterRect
01443     };
01444     Q_ENUMS(SizeConstraintRect)
01445
01446     explicit QCPLLayoutElement(QCustomPlot *parentPlot = nullptr);
01447     virtual ~QCPLLayoutElement() Q_DECL_OVERRIDE;
01448
01449     // getters:
01450
01451     QCPLLayout *layout() const {
01452         return mParentLayout;
01453     }
01454
01455     QRect rect() const {
01456         return mRect;
01457     }
01458
01459     QRect outerRect() const {
01460         return mOuterRect;
01461     }
01462
01463     QMargins margins() const {
01464         return mMargins;
01465     }
01466
01467     QMargins minimumMargins() const {
01468         return mMinimumMargins;
01469     }
01470
01471     QCP::MarginSides autoMargins() const {
01472         return mAutoMargins;
01473     }
01474
01475     QSize minimumSize() const {
01476         return mMinimumSize;
01477     }
01478
01479     QSize maximumSize() const {
01480         return mMaximumSize;
01481     }
01482
01483     SizeConstraintRect sizeConstraintRect() const {
01484         return mSizeConstraintRect;
01485     }
01486
01487     QCPMarginGroup *marginGroup(QCP::MarginSide side) const {
01488         return mMarginGroups.value(side, nullptr);
01489     }
01490
01491     QHash<QCP::MarginSide, QCPMarginGroup*> marginGroups() const {
01492         return mMarginGroups;
01493     }
01494
01495     // setters:
01496     void setOuterRect(const QRect &rect);
01497     void setMargins(const QMargins &margins);
01498     void setMinimumMargins(const QMargins &margins);
01499     void setAutoMargins(QCP::MarginSides sides);
01500     void setMinimumSize(const QSize &size);
01501     void setMinimumSize(int width, int height);
01502     void setMaximumSize(const QSize &size);
01503     void setMaximumSize(int width, int height);
01504     void setSizeConstraintRect(SizeConstraintRect constraintRect);
01505     void setMarginGroup(QCP::MarginSides sides, QCPMarginGroup *group);
01506
01507     // introduced virtual methods:
01508     virtual void update(UpdatePhase phase);
01509     virtual QSize minimumOuterSizeHint() const;
01510     virtual QSize maximumOuterSizeHint() const;
01511     virtual QList<QCPLLayoutElement*> elements(bool recursive) const;
01512
01513     // reimplemented virtual methods:
01514     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
01515         const Q_DECL_OVERRIDE;
01516
01517 protected:
01518     // property members:

```

```

01519     QCPLLayout *mParentLayout;
01520     QSize mMinimumSize, mMaximumSize;
01521     SizeConstraintRect mSizeConstraintRect;
01522     QRect mRect, mOuterRect;
01523     QMargins mMargins, mMinimumMargins;
01524     QCP::MarginSides mAutoMargins;
01525     QHash<QCP::MarginSide, QCPMarginGroup*> mMarginGroups;
01526
01527     // introduced virtual methods:
01528     virtual int calculateAutoMargin(QCP::MarginSide side);
01529     virtual void layoutChanged();
01530
01531     // reimplemented virtual methods:
01532
01533     virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE {
01534         Q_UNUSED(painter)
01535     }
01536
01537     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE {
01538         Q_UNUSED(painter)
01539     }
01540     virtual void parentPlotInitialized(QCustomPlot *parentPlot) Q_DECL_OVERRIDE;
01541
01542 private:
01543     Q_DISABLE_COPY(QCPLLayoutElement)
01544
01545     friend class QCustomPlot;
01546     friend class QCPLLayout;
01547     friend class QCPMarginGroup;
01548 };
01549
01550 Q_DECLARE_METATYPE(QCPLLayoutElement::UpdatePhase)
01551
01552
01553 class QCP_LIB_DECL QCPLLayout : public QCPLLayoutElement {
01554     Q_OBJECT
01555 public:
01556     explicit QCPLLayout();
01557
01558     // reimplemented virtual methods:
01559     virtual void update(UpdatePhase phase) Q_DECL_OVERRIDE;
01560     virtual QList<QCPLLayoutElement*> elements(bool recursive) const Q_DECL_OVERRIDE;
01561
01562     // introduced virtual methods:
01563     virtual int elementCount() const = 0;
01564     virtual QCPLLayoutElement* elementAt(int index) const = 0;
01565     virtual QCPLLayoutElement* takeAt(int index) = 0;
01566     virtual bool take(QCPLLayoutElement* element) = 0;
01567     virtual void simplify();
01568
01569     // non-virtual methods:
01570     bool removeAt(int index);
01571     bool remove(QCPLLayoutElement* element);
01572     void clear();
01573
01574 protected:
01575     // introduced virtual methods:
01576     virtual void updateLayout();
01577
01578     // non-virtual methods:
01579     void sizeConstraintsChanged() const;
01580     void adoptElement(QCPLLayoutElement *el);
01581     void releaseElement(QCPLLayoutElement *el);
01582     QVector<int> getSectionSizes(QVector<int> maxSizes, QVector<int> minSizes, QVector<double>
stretchFactors, int totalSize) const;
01583     static QSize getFinalMinimumOuterSize(const QCPLLayoutElement *el);
01584     static QSize getFinalMaximumOuterSize(const QCPLLayoutElement *el);
01585
01586 private:
01587     Q_DISABLE_COPY(QCPLLayout)
01588     friend class QCPLLayoutElement;
01589 };
01590
01591 class QCP_LIB_DECL QCPLLayoutGrid : public QCPLLayout {
01592
01593     Q_OBJECT
01594     Q_PROPERTY(int rowCount READ rowCount)
01595     Q_PROPERTY(int columnCount READ columnCount)
01596     Q_PROPERTY(QList<double> columnStretchFactors READ columnStretchFactors WRITE
setColumnStretchFactors)
01597     Q_PROPERTY(QList<double> rowStretchFactors READ rowStretchFactors WRITE setRowStretchFactors)
01598     Q_PROPERTY(int columnSpacing READ columnSpacing WRITE setColumnSpacing)
01599     Q_PROPERTY(int rowSpacing READ rowSpacing WRITE setRowSpacing)
01600     Q_PROPERTY(FillOrder fillOrder READ fillOrder WRITE setFillOrder)
01601     Q_PROPERTY(int wrap READ wrap WRITE setWrap)
01602 public:
01603
01604
01605

```

```

01613     enum FillOrder {
01614         foRowsFirst
01615         , foColumnsFirst
01616     };
01617     Q_ENUMS(FillOrder)
01618
01619     explicit QCPLLayoutGrid();
01620     virtual ~QCPLLayoutGrid() Q_DECL_OVERRIDE;
01621
01622     // getters:
01623
01624     int rowCount() const {
01625         return mElements.size();
01626     }
01627
01628     int columnCount() const {
01629         return mElements.size() > 0 ? mElements.first().size() : 0;
01630     }
01631
01632     QList<double> columnStretchFactors() const {
01633         return mColumnStretchFactors;
01634     }
01635
01636     QList<double> rowStretchFactors() const {
01637         return mRowStretchFactors;
01638     }
01639
01640     int columnSpacing() const {
01641         return mColumnSpacing;
01642     }
01643
01644     int rowSpacing() const {
01645         return mRowSpacing;
01646     }
01647
01648     int wrap() const {
01649         return mWrap;
01650     }
01651
01652     FillOrder fillOrder() const {
01653         return mFillOrder;
01654     }
01655
01656     // setters:
01657     void setColumnStretchFactor(int column, double factor);
01658     void setColumnStretchFactors(const QList<double> &factors);
01659     void setRowStretchFactor(int row, double factor);
01660     void setRowStretchFactors(const QList<double> &factors);
01661     void setColumnSpacing(int pixels);
01662     void setRowSpacing(int pixels);
01663     void setWrap(int count);
01664     void setFillOrder(FillOrder order, bool rearrange = true);
01665
01666     // reimplemented virtual methods:
01667     virtual void updateLayout() Q_DECL_OVERRIDE;
01668
01669     virtual int elementCount() const Q_DECL_OVERRIDE {
01670         return rowCount() * columnCount();
01671     }
01672     virtual QCPLLayoutElement* elementAt(int index) const Q_DECL_OVERRIDE;
01673     virtual QCPLLayoutElement* takeAt(int index) Q_DECL_OVERRIDE;
01674     virtual bool take(QCPLLayoutElement* element) Q_DECL_OVERRIDE;
01675     virtual QList<QCPLLayoutElement*> elements(bool recursive) const Q_DECL_OVERRIDE;
01676     virtual void simplify() Q_DECL_OVERRIDE;
01677     virtual QSize minimumOuterSizeHint() const Q_DECL_OVERRIDE;
01678     virtual QSize maximumOuterSizeHint() const Q_DECL_OVERRIDE;
01679
01680     // non-virtual methods:
01681     QCPLLayoutElement *element(int row, int column) const;
01682     bool addElement(int row, int column, QCPLLayoutElement *element);
01683     bool addElement(QCPLLayoutElement *element);
01684     bool hasElement(int row, int column);
01685     void expandTo(int newRowCount, int newColumnCount);
01686     void insertRow(int newIndex);
01687     void insertColumn(int newIndex);
01688     int rowColToIndex(int row, int column) const;
01689     void indexToRowCol(int index, int &row, int &column) const;
01690
01691 protected:
01692     // property members:
01693     QList<QList<QCPLLayoutElement*> > mElements;
01694     QList<double> mColumnStretchFactors;
01695     QList<double> mRowStretchFactors;
01696     int mColumnSpacing, mRowSpacing;
01697     int mWrap;
01698     FillOrder mFillOrder;
01699

```

```

01700     // non-virtual methods:
01701     void getMinimumRowColSizes(QVector<int> *minColWidths, QVector<int> *minRowHeights) const;
01702     void getMaximumRowColSizes(QVector<int> *maxColWidths, QVector<int> *maxRowHeights) const;
01703
01704 private:
01705     Q_DISABLE_COPY(QCPLLayoutGrid)
01706 };
01707
01708 Q_DECLARE_METATYPE(QCPLLayoutGrid::FillOrder)
01709
01710
01711 class QCP_LIB_DECL QCPLLayoutInset : public QCPLLayout {
01712     Q_OBJECT
01713 public:
01714     enum InsetPlacement {
01715         ipFree
01716         , ipBorderAligned
01717     };
01718     Q_ENUMS(InsetPlacement)
01719
01720     explicit QCPLLayoutInset();
01721     virtual ~QCPLLayoutInset() Q_DECL_OVERRIDE;
01722
01723     // getters:
01724     InsetPlacement insetPlacement(int index) const;
01725     Qt::Alignment insetAlignment(int index) const;
01726     QRectF insetRect(int index) const;
01727
01728     // setters:
01729     void setInsetPlacement(int index, InsetPlacement placement);
01730     void setInsetAlignment(int index, Qt::Alignment alignment);
01731     void setInsetRect(int index, const QRectF &rect);
01732
01733     // reimplemented virtual methods:
01734     virtual void updateLayout() Q_DECL_OVERRIDE;
01735     virtual int elementCount() const Q_DECL_OVERRIDE;
01736     virtual QCPLLayoutElement* elementAt(int index) const Q_DECL_OVERRIDE;
01737     virtual QCPLLayoutElement* takeAt(int index) Q_DECL_OVERRIDE;
01738     virtual bool take(QCPLLayoutElement* element) Q_DECL_OVERRIDE;
01739
01740     virtual void simplify() Q_DECL_OVERRIDE {
01741     }
01742     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
01743     const Q_DECL_OVERRIDE;
01744
01745     // non-virtual methods:
01746     void addElement(QCPLLayoutElement *element, Qt::Alignment alignment);
01747     void addElement(QCPLLayoutElement *element, const QRectF &rect);
01748
01749 protected:
01750     // property members:
01751     QList<QCPLLayoutElement*> mElements;
01752     QList<InsetPlacement> mInsetPlacement;
01753     QList<Qt::Alignment> mInsetAlignment;
01754     QList<QRectF> mInsetRect;
01755
01756 private:
01757     Q_DISABLE_COPY(QCPLLayoutInset)
01758 };
01759
01760 Q_DECLARE_METATYPE(QCPLLayoutInset::InsetPlacement)
01761
01762 /* end of 'src/layout.h' */
01763
01764
01765 /* including file 'src/lineending.h' */
01766 /* modified 2021-03-29T02:30:44, size 4426 */
01767
01768 class QCP_LIB_DECL QCPLLineEnding {
01769     Q_GADGET
01770 public:
01771     enum EndingStyle {
01772         esNone
01773         , esFlatArrow
01774         , esSpikeArrow
01775         , esLineArrow
01776         , esDisc
01777         , esSquare
01778         , esDiamond
01779         , esBar
01780         , esHalfBar
01781         , esSkewedBar
01782     };
01783     Q_ENUMS(EndingStyle)
01784
01785
01786

```

```

01800     QCPLLineEnding();
01801     QCPLLineEnding(EndingStyle style, double width = 8, double length = 10, bool inverted = false);
01802
01803     // getters:
01804
01805     EndingStyle style() const {
01806         return mStyle;
01807     }
01808
01809     double width() const {
01810         return mWidth;
01811     }
01812
01813     double length() const {
01814         return mLength;
01815     }
01816
01817     bool inverted() const {
01818         return mInverted;
01819     }
01820
01821     // setters:
01822     void setStyle(EndingStyle style);
01823     void setWidth(double width);
01824     void setLength(double length);
01825     void setInverted(bool inverted);
01826
01827     // non-property methods:
01828     double boundingDistance() const;
01829     double realLength() const;
01830     void draw(QCPLPainter *painter, const QCPLVector2D &pos, const QCPLVector2D &dir) const;
01831     void draw(QCPLPainter *painter, const QCPLVector2D &pos, double angle) const;
01832
01833 protected:
01834     // property members:
01835     EndingStyle mStyle;
01836     double mWidth, mLength;
01837     bool mInverted;
01838 };
01839 Q_DECLARE_TYPEINFO(QCPLLineEnding, Q_MOVABLE_TYPE);
01840
01841 Q_DECLARE_METATYPE(QCPLLineEnding::EndingStyle)
01842
01843 /* end of 'src/lineending.h' */
01844
01845
01846 /* including file 'src/axis/labelpainter.h' */
01847 /* modified 2021-03-29T02:30:44, size 7086 */
01848
01849 class QCPLLabelPainterPrivate {
01850
01851     Q_GADGET
01852 public:
01853     enum AnchorMode {
01854         amRectangular
01855         , amSkewedUpright
01856         , amSkewedRotated
01857     };
01858
01859     Q_ENUMS(AnchorMode)
01860
01861     enum AnchorReferenceType {
01862         artNormal
01863         , artTangent
01864     };
01865
01866     Q_ENUMS(AnchorReferenceType)
01867
01868     enum AnchorSide {
01869         asLeft
01870         , asRight
01871         , asTop
01872         , asBottom
01873         , asTopLeft
01874         , asTopRight
01875         , asBottomRight
01876         , asBottomLeft
01877     };
01878
01879     Q_ENUMS(AnchorSide)
01880
01881     explicit QCPLLabelPainterPrivate(QCustomPlot *parentPlot);
01882     virtual ~QCPLLabelPainterPrivate();
01883
01884     // setters:
01885     void setAnchorSide(AnchorSide side);

```

```

01894     void setAnchorMode(AnchorMode mode);
01895     void setAnchorReference(const QPointF &pixelPoint);
01896     void setAnchorReferenceType(AnchorReferenceType type);
01897     void setFont(const QFont &font);
01898     void setColor(const QColor &color);
01899     void setPadding(int padding);
01900     void setRotation(double rotation);
01901     void setSubstituteExponent(bool enabled);
01902     void setMultiplicationSymbol(QChar symbol);
01903     void setAbbreviateDecimalPowers(bool enabled);
01904     void setCacheSize(int labelCount);
01905
01906     // getters:
01907
01908     AnchorMode anchorMode() const {
01909         return mAnchorMode;
01910     }
01911
01912     AnchorSide anchorSide() const {
01913         return mAnchorSide;
01914     }
01915
01916     QPointF anchorReference() const {
01917         return mAnchorReference;
01918     }
01919
01920     AnchorReferenceType anchorReferenceType() const {
01921         return mAnchorReferenceType;
01922     }
01923
01924     QFont font() const {
01925         return mFont;
01926     }
01927
01928     QColor color() const {
01929         return mColor;
01930     }
01931
01932     int padding() const {
01933         return mPadding;
01934     }
01935
01936     double rotation() const {
01937         return mRotation;
01938     }
01939
01940     bool substituteExponent() const {
01941         return mSubstituteExponent;
01942     }
01943
01944     QChar multiplicationSymbol() const {
01945         return mMultiplicationSymbol;
01946     }
01947
01948     bool abbreviateDecimalPowers() const {
01949         return mAbbreviateDecimalPowers;
01950     }
01951     int cacheSize() const;
01952
01953     //virtual int size() const;
01954
01955     // non-property methods:
01956     void drawTickLabel(QCPPainter *painter, const QPointF &tickPos, const QString &text);
01957     void clearCache();
01958
01959     // constants that may be used with setMultiplicationSymbol:
01960     static const QChar SymbolDot;
01961     static const QChar SymbolCross;
01962
01963 protected:
01964
01965     struct CachedLabel {
01966         QPoint offset;
01967         QPixmap pixmap;
01968     };
01969
01970     struct LabelData {
01971         AnchorSide side;
01972         double rotation; // angle in degrees
01973         QTransform transform; // the transform about the label anchor which is at (0, 0). Does not
01974         contain final absolute x/y positioning on the plot/axis
01975         QString basePart, expPart, suffixPart;
01976         QRect baseBounds, expBounds, suffixBounds;
01977         QRect totalBounds; // is in a coordinate system where label top left is at (0, 0)
01978         QRect rotatedTotalBounds; // is in a coordinate system where the label anchor is at (0, 0)
01979         QFont baseFont, expFont;
01980         QColor color;

```

```

01980     };
01981
01982     // property members:
01983     AnchorMode mAnchorMode;
01984     AnchorSide mAnchorSide;
01985     QPointF mAnchorReference;
01986     AnchorReferenceType mAnchorReferenceType;
01987     QFont mFont;
01988     QColor mColor;
01989     int mPadding;
01990     double mRotation; // this is the rotation applied uniformly to all labels, not the heterogeneous
rotation in amCircularRotated mode
01991     bool mSubstituteExponent;
01992     QChar mMultiplicationSymbol;
01993     bool mAbbreviateDecimalPowers;
01994     // non-property members:
01995     QCustomPlot *mParentPlot;
01996     QByteArray mLabelParameterHash; // to determine whether mLabelCache needs to be cleared due to
changed parameters
01997     QCache<QString, CachedLabel> mLabelCache;
01998     QRect mAxisSelectionBox, mTickLabelsSelectionBox, mLabelSelectionBox;
01999     int mLetterCapHeight, mLetterDescent;
02000
02001     // introduced virtual methods:
02002     virtual void drawLabelMaybeCached(QCPPainter *painter, const QFont &font, const QColor &color,
const QPointF &pos, AnchorSide side, double rotation, const QString &text);
02003     virtual QByteArray generateLabelParameterHash() const; // TODO: get rid of this in favor of
invalidation flag upon setters?
02004
02005     // non-virtual methods:
02006     QPointF getAnchorPos(const QPointF &tickPos);
02007     void drawText(QCPPainter *painter, const QPointF &pos, const LabelData &labelData) const;
02008     LabelData getTickLabelData(const QFont &font, const QColor &color, double rotation, AnchorSide
side, const QString &text) const;
02009     void applyAnchorTransform(LabelData &labelData) const;
02010     //void getMaxTickLabelSize(const QFont &font, const QString &text, QSize *tickLabelsSize) const;
02011     CachedLabel *createCachedLabel(const LabelData &labelData) const;
02012     QByteArray cacheKey(const QString &text, const QColor &color, double rotation, AnchorSide side)
const;
02013     AnchorSide skewedAnchorSide(const QPointF &tickPos, double sideExpandHorz, double sideExpandVert)
const;
02014     AnchorSide rotationCorrectedSide(AnchorSide side, double rotation) const;
02015     void analyzeFontMetrics();
02016 };
02017
02018 Q_DECLARE_METATYPE(QCPLLabelPainterPrivate::AnchorMode)
02019 Q_DECLARE_METATYPE(QCPLLabelPainterPrivate::AnchorSide)
02020
02021 /* end of 'src/axis/labelpainter.h' */
02022
02023 /* including file 'src/axis/axisticker.h' */
02024 /* modified 2021-03-29T02:30:44, size 4230 */
02025
02026 class QCP_LIB_DECL QCPAxisTicker {
02027
02028     Q_GADGET
02029 public:
02030     enum TickStepStrategy {
02031         tssReadability
02032         , tssMeetTickCount
02033     };
02034     Q_ENUMS(TickStepStrategy)
02035
02036     QCPAxisTicker();
02037     virtual ~QCPAxisTicker();
02038
02039     // getters:
02040
02041     TickStepStrategy tickStepStrategy() const {
02042         return mTickStepStrategy;
02043     }
02044
02045     int tickCount() const {
02046         return mTickCount;
02047     }
02048
02049     double tickOrigin() const {
02050         return mTickOrigin;
02051     }
02052
02053     // setters:
02054     void setTickStepStrategy(TickStepStrategy strategy);
02055     void setTickCount(int count);
02056     void setTickOrigin(double origin);
02057
02058 };

```



```

02065     // introduced virtual methods:
02066     virtual void generate(const QCPRange &range, const QLocale &locale, QChar formatChar, int
precision, QVector<double> &ticks, QVector<double> *subTicks, QVector<QString> *tickLabels);
02067
02068 protected:
02069     // property members:
02070     TickStepStrategy mTickStepStrategy;
02071     int mTickCount;
02072     double mTickOrigin;
02073
02074     // introduced virtual methods:
02075     virtual double getTickStep(const QCPRange &range);
02076     virtual int getSubTickCount(double tickStep);
02077     virtual QString getTickLabel(double tick, const QLocale &locale, QChar formatChar, int precision);
02078     virtual QVector<double> createTickVector(double tickStep, const QCPRange &range);
02079     virtual QVector<double> createSubTickVector(int subTickCount, const QVector<double> &ticks);
02080     virtual QVector<QString> createLabelVector(const QVector<double> &ticks, const QLocale &locale,
QChar formatChar, int precision);
02081
02082     // non-virtual methods:
02083     void trimTicks(const QCPRange &range, QVector<double> &ticks, bool keepOneOutlier) const;
02084     double pickClosest(double target, const QVector<double> &candidates) const;
02085     double getMantissa(double input, double *magnitude = nullptr) const;
02086     double cleanMantissa(double input) const;
02087
02088 private:
02089     Q_DISABLE_COPY(QCPAxisTicker)
02090
02091 };
02092
02093 Q_DECLARE_METATYPE(QCPAxisTicker::TickStepStrategy)
02094 Q_DECLARE_METATYPE(QSharedPointer<QCPAxisTicker>)
02095
02096 /* end of 'src/axis/axisticker.h' */
02097
02098
02099 /* including file 'src/axis/axistickerdatetime.h' */
02100 /* modified 2021-03-29T02:30:44, size 3600 */
02101
02102 class QCP_LIB_DECL QCPAxisTickerDateTime : public QCPAxisTicker {
02103 public:
02104     QCPAxisTickerDateTime();
02105
02106     // getters:
02107
02108     QString dateTimeFormat() const {
02109         return mDateTimeFormat;
02110     }
02111
02112     Qt::TimeSpec dateTimeSpec() const {
02113         return mDateTimeSpec;
02114     }
02115     #if QT_VERSION >= QT_VERSION_CHECK(5, 2, 0)
02116     QTimeZone timeZone() const {
02117         return mTimeZone;
02118     }
02119     #endif
02120
02121     // setters:
02122     void setDateTimeFormat(const QString &format);
02123     void setDateTimeSpec(Qt::TimeSpec spec);
02124     #if QT_VERSION >= QT_VERSION_CHECK(5, 2, 0)
02125     void setTimeZone(const QTimeZone &zone);
02126     #endif
02127     void setTickOrigin(double origin); // hides base class method but calls baseclass implementation
    ("using" throws off IDEs and doxygen)
02129     void setTickOrigin(const QDateTime &origin);
02130
02131     // static methods:
02132     static QDateTime keyToDateTime(double key);
02133     static double dateTimeToKey(const QDateTime &dateTime);
02134     static double dateTimeToKey(const QDate &date, Qt::TimeSpec timeSpec = Qt::LocalTime);
02135
02136 protected:
02137     // property members:
02138     QString mDateTimeFormat;
02139     Qt::TimeSpec mDateTimeSpec;
02140     #if QT_VERSION >= QT_VERSION_CHECK(5, 2, 0)
02141     QTimeZone mTimeZone;
02142     #endif
02143
02144     // non-property members:
02145     enum DateStrategy {
02146         dsNone, dsUniformTimeInDay, dsUniformDayInMonth
02147     } mDateStrategy;
02148

```

```

02149     // reimplemented virtual methods:
02150     virtual double getTickStep(const QCPRange &range) Q_DECL_OVERRIDE;
02151     virtual int getSubTickCount(double tickStep) Q_DECL_OVERRIDE;
02152     virtual QString getTickLabel(double tick, const QLocale &locale, QChar formatChar, int precision)
02153     Q_DECL_OVERRIDE;
02153     virtual QVector<double> createTickVector(double tickStep, const QCPRange &range) Q_DECL_OVERRIDE;
02154 };
02155
02156 /* end of 'src/axis/axistickerdatetime.h' */
02157
02158
02159 /* including file 'src/axis/axistickertime.h' */
02160
02161 /* modified 2021-03-29T02:30:44, size 3542 */
02162
02163 class QCP_LIB_DECL QCPAxisTickerTime : public QCPAxisTicker {
02164
02165     Q_GADGET
02166 public:
02172     enum TimeUnit {
02173         tuMilliseconds
02174         , tuSeconds
02175         , tuMinutes
02176         , tuHours
02177         , tuDays
02178     };
02179     Q_ENUMS(TimeUnit)
02180
02181     QCPAxisTickerTime();
02182
02183     // getters:
02184     QString timeFormat() const {
02185         return mTimeFormat;
02186     }
02187
02188     int fieldWidth(TimeUnit unit) const {
02189         return mFieldWidth.value(unit);
02190     }
02191
02192     // setters:
02193     void setTimeFormat(const QString &format);
02194     void setFieldWidth(TimeUnit unit, int width);
02195
02196 protected:
02197     // property members:
02198     QString mTimeFormat;
02199     QHash<TimeUnit, int> mFieldWidth;
02200
02201     // non-property members:
02202     TimeUnit mSmallestUnit, mBiggestUnit;
02203     QHash<TimeUnit, QString> mFormatPattern;
02204
02205     // reimplemented virtual methods:
02206     virtual double getTickStep(const QCPRange &range) Q_DECL_OVERRIDE;
02207     virtual int getSubTickCount(double tickStep) Q_DECL_OVERRIDE;
02208     virtual QString getTickLabel(double tick, const QLocale &locale, QChar formatChar, int precision)
02209     Q_DECL_OVERRIDE;
02210
02211     // non-virtual methods:
02212     void replaceUnit(QString &text, TimeUnit unit, int value) const;
02213 };
02214
02215 Q_DECLARE_METATYPE(QCPAxisTickerTime::TimeUnit)
02216
02217 /* end of 'src/axis/axistickertime.h' */
02218
02219
02220 /* including file 'src/axis/axistickerfixed.h' */
02221 /* modified 2021-03-29T02:30:44, size 3308 */
02222
02223 class QCP_LIB_DECL QCPAxisTickerFixed : public QCPAxisTicker {
02224
02225     Q_GADGET
02226 public:
02233     enum ScaleStrategy {
02234         ssNone
02235         , ssMultiples
02236         , ssPowers
02237     };
02238     Q_ENUMS(ScaleStrategy)
02239
02240     QCPAxisTickerFixed();
02241
02242     // getters:
02243     double tickStep() const {

```

```

02245         return mTickStep;
02246     }
02247
02248     ScaleStrategy scaleStrategy() const {
02249         return mScaleStrategy;
02250     }
02251
02252     // setters:
02253     void setTickStep(double step);
02254     void setScaleStrategy(ScaleStrategy strategy);
02255
02256 protected:
02257     // property members:
02258     double mTickStep;
02259     ScaleStrategy mScaleStrategy;
02260
02261     // reimplemented virtual methods:
02262     virtual double getTickStep(const QCPRange &range) Q_DECL_OVERRIDE;
02263 };
02264
02265 Q_DECLARE_METATYPE(QCPAxisTickerFixed::ScaleStrategy)
02266
02267 /* end of 'src/axis/axistickerfixed.h' */
02268
02269
02270 /* including file 'src/axis/axistickertext.h' */
02271 /* modified 2021-03-29T02:30:44, size 3090 */
02272
02273 class QCP_LIB_DECL QCPAxisTickerText : public QCPAxisTicker {
02274 public:
02275     QCPAxisTickerText();
02276
02277     // getters:
02278
02279     QMap<double, QString> &ticks() {
02280         return mTicks;
02281     }
02282
02283     int subTickCount() const {
02284         return mSubTickCount;
02285     }
02286
02287     // setters:
02288     void setTicks(const QMap<double, QString> &ticks);
02289     void setTicks(const QVector<double> &positions, const QVector<QString> &labels);
02290     void setSubTickCount(int subTicks);
02291
02292     // non-virtual methods:
02293     void clear();
02294     void addTick(double position, const QString &label);
02295     void addTicks(const QMap<double, QString> &ticks);
02296     void addTicks(const QVector<double> &positions, const QVector<QString> &labels);
02297
02298 protected:
02299     // property members:
02300     QMap<double, QString> mTicks;
02301     int mSubTickCount;
02302
02303     // reimplemented virtual methods:
02304     virtual double getTickStep(const QCPRange &range) Q_DECL_OVERRIDE;
02305     virtual int getSubTickCount(double tickStep) Q_DECL_OVERRIDE;
02306     virtual QString getTickLabel(double tick, const QLocale &locale, QChar formatChar, int precision)
02307         Q_DECL_OVERRIDE;
02308     virtual QVector<double> createTickVector(double tickStep, const QCPRange &range) Q_DECL_OVERRIDE;
02309 };
02310
02311 /* end of 'src/axis/axistickertext.h' */
02312
02313 /* including file 'src/axis/axistickerpi.h' */
02314
02315 /* modified 2021-03-29T02:30:44, size 3911 */
02316
02317 class QCP_LIB_DECL QCPAxisTickerPi : public QCPAxisTicker {
02318 public:
02319     Q_GADGET
02320     public:
02321         enum FractionStyle {
02322             fsFloatingPoint
02323             , fsAsciiFractions
02324             , fsUnicodeFractions
02325         };
02326         Q_ENUMS(FractionStyle)
02327
02328         QCPAxisTickerPi();
02329
02330     // getters:

```

```

02336
02337     QString piSymbol() const {
02338         return mPiSymbol;
02339     }
02340
02341     double piValue() const {
02342         return mPiValue;
02343     }
02344
02345     bool periodicity() const {
02346         return mPeriodicity;
02347     }
02348
02349     FractionStyle fractionStyle() const {
02350         return mFractionStyle;
02351     }
02352
02353     // setters:
02354     void setPiSymbol(QString symbol);
02355     void setPiValue(double pi);
02356     void setPeriodicity(int multiplesOfPi);
02357     void setFractionStyle(FractionStyle style);
02358
02359 protected:
02360     // property members:
02361     QString mPiSymbol;
02362     double mPiValue;
02363     int mPeriodicity;
02364     FractionStyle mFractionStyle;
02365
02366     // non-property members:
02367     double mPiTickStep; // size of one tick step in units of mPiValue
02368
02369     // reimplemented virtual methods:
02370     virtual double getTickStep(const QCPRange &range) Q_DECL_OVERRIDE;
02371     virtual int getSubTickCount(double tickStep) Q_DECL_OVERRIDE;
02372     virtual QString getTickLabel(double tick, const QLocale &locale, QChar formatChar, int precision)
02373         Q_DECL_OVERRIDE;
02374
02375     // non-virtual methods:
02376     void simplifyFraction(int &numerator, int &denominator) const;
02377     QString fractionToString(int numerator, int denominator) const;
02378     QString unicodeFraction(int numerator, int denominator) const;
02379     QString unicodeSuperscript(int number) const;
02380     QString unicodeSubscript(int number) const;
02381 };
02382 Q_DECLARE_METATYPE(QCPAxisTickerPi::FractionStyle)
02383
02384 /* end of 'src/axis/axistickerpi.h' */
02385
02386
02387 /* including file 'src/axis/axistickerlog.h' */
02388 /* modified 2021-03-29T02:30:44, size 2594 */
02389
02390 class QCP_LIB_DECL QCPAxisTickerLog : public QCPAxisTicker {
02391 public:
02392     QCPAxisTickerLog();
02393
02394     // getters:
02395
02396     double logBase() const {
02397         return mLogBase;
02398     }
02399
02400     int subTickCount() const {
02401         return mSubTickCount;
02402     }
02403
02404     // setters:
02405     void setLogBase(double base);
02406     void setSubTickCount(int subTicks);
02407
02408 protected:
02409     // property members:
02410     double mLogBase;
02411     int mSubTickCount;
02412
02413     // non-property members:
02414     double mLogBaseLnInv;
02415
02416     // reimplemented virtual methods:
02417     virtual int getSubTickCount(double tickStep) Q_DECL_OVERRIDE;
02418     virtual QVector<double> createTickVector(double tickStep, const QCPRange &range) Q_DECL_OVERRIDE;
02419 };
02420
02421 /* end of 'src/axis/axistickerlog.h' */

```

```

02422
02423
02424 /* including file 'src/axis/axis.h' */
02425
02426 /* modified 2021-03-29T02:30:44, size 20913 */
02427
02428 class QCP_LIB_DECL QCPGrid : public QCPLayerable {
02429     Q_OBJECT
02430     Q_PROPERTY(bool subGridVisible READ subGridVisible WRITE setSubGridVisible)
02431     Q_PROPERTY(bool antialiasedSubGrid READ antialiasedSubGrid WRITE setAntialiasedSubGrid)
02432     Q_PROPERTY(bool antialiasedZeroLine READ antialiasedZeroLine WRITE setAntialiasedZeroLine)
02433     Q_PROPERTY(QPen pen READ pen WRITE setPen)
02434     Q_PROPERTY(QPen subGridPen READ subGridPen WRITE setSubGridPen)
02435     Q_PROPERTY(QPen zeroLinePen READ zeroLinePen WRITE setZeroLinePen)
02436 public:
02437     explicit QCPGrid(QCPAxis *parentAxis);
02438
02439     // getters:
02440
02441     bool subGridVisible() const {
02442         return mSubGridVisible;
02443     }
02444
02445     bool antialiasedSubGrid() const {
02446         return mAntialiasedSubGrid;
02447     }
02448
02449     bool antialiasedZeroLine() const {
02450         return mAntialiasedZeroLine;
02451     }
02452
02453     QPen pen() const {
02454         return mPen;
02455     }
02456
02457     QPen subGridPen() const {
02458         return mSubGridPen;
02459     }
02460
02461     QPen zeroLinePen() const {
02462         return mZeroLinePen;
02463     }
02464
02465     // setters:
02466     void setSubGridVisible(bool visible);
02467     void setAntialiasedSubGrid(bool enabled);
02468     void setAntialiasedZeroLine(bool enabled);
02469     void setPen(const QPen &pen);
02470     void setSubGridPen(const QPen &pen);
02471     void setZeroLinePen(const QPen &pen);
02472
02473 protected:
02474     // property members:
02475     bool mSubGridVisible;
02476     bool mAntialiasedSubGrid, mAntialiasedZeroLine;
02477     QPen mPen, mSubGridPen, mZeroLinePen;
02478
02479     // non-property members:
02480     QCPAxis *mParentAxis;
02481
02482     // reimplemented virtual methods:
02483     virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
02484     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
02485
02486     // non-virtual methods:
02487     void drawGridLines(QCPPainter *painter) const;
02488     void drawSubGridLines(QCPPainter *painter) const;
02489
02490     friend class QCPAxis;
02491 };
02492
02493 class QCP_LIB_DECL QCPAxis : public QCPLayerable {
02494     Q_OBJECT
02495     Q_PROPERTY(AxisType axisType READ axisType)
02496     Q_PROPERTY(QCPAxisRect* axisRect READ axisRect)
02497     Q_PROPERTY(ScaleType scaleType READ scaleType WRITE setScaleType NOTIFY scaleTypeChanged)
02498     Q_PROPERTY(QCPRange range READ range WRITE setRange NOTIFY rangeChanged)
02499     Q_PROPERTY(bool rangeReversed READ rangeReversed WRITE setRangeReversed)
02500     Q_PROPERTY(QSharedPointer<QCPAxisTicker> ticker READ ticker WRITE setTicker)
02501     Q_PROPERTY(bool ticks READ ticks WRITE setTicks)
02502     Q_PROPERTY(bool tickLabels READ tickLabels WRITE setTickLabels)
02503     Q_PROPERTY(int tickLabelPadding READ tickLabelPadding WRITE setTickLabelPadding)
02504     Q_PROPERTY(QFont tickLabelFont READ tickLabelFont WRITE setTickLabelFont)
02505     Q_PROPERTY(QColor tickLabelColor READ tickLabelColor WRITE setTickLabelColor)
02506     Q_PROPERTY(double tickLabelRotation READ tickLabelRotation WRITE setTickLabelRotation)
02507     Q_PROPERTY(LabelSide tickLabelSide READ tickLabelSide WRITE setTickLabelSide)

```

```

02512     Q_PROPERTY(QString numberFormat READ numberFormat WRITE setNumberFormat)
02513     Q_PROPERTY(int numberPrecision READ numberPrecision WRITE setNumberPrecision)
02514     Q_PROPERTY(QVector<double> tickVector READ tickVector)
02515     Q_PROPERTY(QVector<QString> tickVectorLabels READ tickVectorLabels)
02516     Q_PROPERTY(int tickLengthIn READ tickLengthIn WRITE setTickLengthIn)
02517     Q_PROPERTY(int tickLengthOut READ tickLengthOut WRITE setTickLengthOut)
02518     Q_PROPERTY(bool subTicks READ subTicks WRITE setSubTicks)
02519     Q_PROPERTY(int subTickLengthIn READ subTickLengthIn WRITE setSubTickLengthIn)
02520     Q_PROPERTY(int subTickLengthOut READ subTickLengthOut WRITE setSubTickLengthOut)
02521     Q_PROPERTY(QPen basePen READ basePen WRITE setBasePen)
02522     Q_PROPERTY(QPen tickPen READ tickPen WRITE setTickPen)
02523     Q_PROPERTY(QPen subTickPen READ subTickPen WRITE setSubTickPen)
02524     Q_PROPERTY(QFont labelFont READ labelFont WRITE setLabelFont)
02525     Q_PROPERTY(QColor labelColor READ labelColor WRITE setLabelColor)
02526     Q_PROPERTY(QString label READ label WRITE setLabel)
02527     Q_PROPERTY(int labelPadding READ labelPadding WRITE setLabelPadding)
02528     Q_PROPERTY(int padding READ padding WRITE setPadding)
02529     Q_PROPERTY(int offset READ offset WRITE setOffset)
02530     Q_PROPERTY(SelectableParts selectedParts READ selectedParts WRITE setSelectedParts NOTIFY
selectionChanged)
02531     Q_PROPERTY(SelectableParts selectableParts READ selectableParts WRITE setSelectedParts NOTIFY
selectableChanged)
02532     Q_PROPERTY(QFont selectedTickLabelFont READ selectedTickLabelFont WRITE setSelectedTickLabelFont)
02533     Q_PROPERTY(QFont selectedLabelFont READ selectedLabelFont WRITE setSelectedLabelFont)
02534     Q_PROPERTY(QColor selectedTickLabelColor READ selectedTickLabelColor WRITE
setSelectedTickLabelColor)
02535     Q_PROPERTY(QColor selectedLabelColor READ selectedLabelColor WRITE setSelectedLabelColor)
02536     Q_PROPERTY(QPen selectedBasePen READ selectedBasePen WRITE setSelectedBasePen)
02537     Q_PROPERTY(QPen selectedTickPen READ selectedTickPen WRITE setSelectedTickPen)
02538     Q_PROPERTY(QPen selectedSubTickPen READ selectedSubTickPen WRITE setSelectedSubTickPen)
02539     Q_PROPERTY(QCPLLineEnding lowerEnding READ lowerEnding WRITE setLowerEnding)
02540     Q_PROPERTY(QCPLLineEnding upperEnding READ upperEnding WRITE setUpperEnding)
02541     Q_PROPERTY(QCPGrid* grid READ grid)
02542 public:
02543     enum AxisType {
02544         atLeft = 0x01
02545         , atRight = 0x02
02546         , atTop = 0x04
02547         , atBottom = 0x08
02548     };
02549     Q_ENUMS(AxisType)
02550     Q_FLAGS(AxisTypes)
02551     Q_DECLARE_FLAGS(AxisTypes, AxisType)
02552     enum LabelSide {
02553         lsInside
02554         , lsOutside
02555     };
02556     Q_ENUMS(LabelSide)
02557     enum ScaleType {
02558         stLinear
02559         , stLogarithmic
02560     };
02561     Q_ENUMS(ScaleType)
02562     enum SelectablePart {
02563         spNone = 0
02564         , spAxis = 0x001
02565         , spTickLabels = 0x002
02566         , spAxisLabel = 0x004
02567     };
02568     Q_ENUMS(SelectablePart)
02569     Q_FLAGS(SelectableParts)
02570     Q_DECLARE_FLAGS(SelectableParts, SelectablePart)
02571
02572     explicit QCPAxis(QCPAxisRect *parent, AxisType type);
02573     virtual ~QCPAxis() Q_DECL_OVERRIDE;
02574
02575     // getters:
02576
02577     AxisType axisType() const {
02578         return mAxisType;
02579     }
02580
02581     QCPAxisRect *axisRect() const {
02582         return mAxisRect;
02583     }
02584
02585     ScaleType scaleType() const {
02586         return mScaleType;
02587     }
02588
02589     const QCPRange range() const {
02590         return mRange;
02591     }
02592
02593

```

```
02614     bool rangeReversed() const {
02615         return mRangeReversed;
02616     }
02617
02618     QSharedPointer<QCPAxisTicker> ticker() const {
02619         return mTicker;
02620     }
02621
02622     bool ticks() const {
02623         return mTicks;
02624     }
02625
02626     bool tickLabels() const {
02627         return mTickLabels;
02628     }
02629     int tickLabelPadding() const;
02630
02631     QFont tickLabelFont() const {
02632         return mTickLabelFont;
02633     }
02634
02635     QColor tickLabelColor() const {
02636         return mTickLabelColor;
02637     }
02638     double tickLabelRotation() const;
02639     LabelSide tickLabelSide() const;
02640     QString numberFormat() const;
02641
02642     int numberPrecision() const {
02643         return mNumberPrecision;
02644     }
02645
02646     QVector<double> tickVector() const {
02647         return mTickVector;
02648     }
02649
02650     QVector<QString> tickVectorLabels() const {
02651         return mTickVectorLabels;
02652     }
02653     int tickLengthIn() const;
02654     int tickLengthOut() const;
02655
02656     bool subTicks() const {
02657         return mSubTicks;
02658     }
02659     int subTickLengthIn() const;
02660     int subTickLengthOut() const;
02661
02662     QPen basePen() const {
02663         return mBasePen;
02664     }
02665
02666     QPen tickPen() const {
02667         return mTickPen;
02668     }
02669
02670     QPen subTickPen() const {
02671         return mSubTickPen;
02672     }
02673
02674     QFont labelFont() const {
02675         return mLabelFont;
02676     }
02677
02678     QColor labelColor() const {
02679         return mLabelColor;
02680     }
02681
02682     QString label() const {
02683         return mLabel;
02684     }
02685     int labelPadding() const;
02686
02687     int padding() const {
02688         return mPadding;
02689     }
02690     int offset() const;
02691
02692     SelectableParts selectedParts() const {
02693         return mSelectedParts;
02694     }
02695
02696     SelectableParts selectableParts() const {
02697         return mSelectableParts;
02698     }
02699
02700     QFont selectedTickLabelFont() const {
```

```

02701         return mSelectedTickLabelFont;
02702     }
02703
02704     QFont selectedLabelFont() const {
02705         return mSelectedLabelFont;
02706     }
02707
02708     QColor selectedTickLabelColor() const {
02709         return mSelectedTickLabelColor;
02710     }
02711
02712     QColor selectedLabelColor() const {
02713         return mSelectedLabelColor;
02714     }
02715
02716     QPen selectedBasePen() const {
02717         return mSelectedBasePen;
02718     }
02719
02720     QPen selectedTickPen() const {
02721         return mSelectedTickPen;
02722     }
02723
02724     QPen selectedSubTickPen() const {
02725         return mSelectedSubTickPen;
02726     }
02727     QCPLLineEnding lowerEnding() const;
02728     QCPLLineEnding upperEnding() const;
02729
02730     QCPGrid *grid() const {
02731         return mGrid;
02732     }
02733
02734     // setters:
02735     Q_SLOT void setScaleType(QCPAxis::ScaleType type);
02736     Q_SLOT void setRange(const QCPRange &range);
02737     void setRange(double lower, double upper);
02738     void setRange(double position, double size, Qt::AlignmentFlag alignment);
02739     void setRangeLower(double lower);
02740     void setRangeUpper(double upper);
02741     void setRangeReversed(bool reversed);
02742     void setTicker(QSharedPointer<QCPAxisTicker> ticker);
02743     void setTicks(bool show);
02744     void setTickLabels(bool show);
02745     void setTickLabelPadding(int padding);
02746     void setTickLabelFont(const QFont &font);
02747     void setTickLabelColor(const QColor &color);
02748     void setTickLabelRotation(double degrees);
02749     void setTickLabelSide(LabelSide side);
02750     void setNumberFormat(const QString &formatCode);
02751     void setNumberPrecision(int precision);
02752     void setTickLength(int inside, int outside = 0);
02753     void setTickLengthIn(int inside);
02754     void setTickLengthOut(int outside);
02755     void setSubTicks(bool show);
02756     void setSubTickLength(int inside, int outside = 0);
02757     void setSubTickLengthIn(int inside);
02758     void setSubTickLengthOut(int outside);
02759     void setBasePen(const QPen &pen);
02760     void setTickPen(const QPen &pen);
02761     void setSubTickPen(const QPen &pen);
02762     void setLabelFont(const QFont &font);
02763     void setLabelColor(const QColor &color);
02764     void setLabel(const QString &str);
02765     void setLabelPadding(int padding);
02766     void setPadding(int padding);
02767     void setOffset(int offset);
02768     void setSelectedTickLabelFont(const QFont &font);
02769     void setSelectedLabelFont(const QFont &font);
02770     void setSelectedTickLabelColor(const QColor &color);
02771     void setSelectedLabelColor(const QColor &color);
02772     void setSelectedBasePen(const QPen &pen);
02773     void setSelectedTickPen(const QPen &pen);
02774     void setSelectedSubTickPen(const QPen &pen);
02775     Q_SLOT void setSelectableParts(const QCPAxis::SelectableParts &selectableParts);
02776     Q_SLOT void setSelectedParts(const QCPAxis::SelectableParts &selectedParts);
02777     void setLowerEnding(const QCPLLineEnding &ending);
02778     void setUpperEnding(const QCPLLineEnding &ending);
02779
02780     // reimplemented virtual methods:
02781     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
02782
02783     // non-property methods:
02784
02785     Qt::Orientation orientation() const {
02786         return mOrientation;

```



```

02787     }
02788
02789     int pixelOrientation() const {
02790         return rangeReversed() != (orientation() == Qt::Vertical) ? -1 : 1;
02791     }
02792     void moveRange(double diff);
02793     void scaleRange(double factor);
02794     void scaleRange(double factor, double center);
02795     void setScaleRatio(const QCPAxis *otherAxis, double ratio = 1.0);
02796     void rescale(bool onlyVisiblePlottables = false);
02797     double pixelToCoord(double value) const;
02798     double coordToPixel(double value) const;
02799     SelectablePart getPartAt(const QPointF &pos) const;
02800     QList<QCPAbstractPlottable*> plottables() const;
02801     QList<QCPGraph*> graphs() const;
02802     QList<QCPAbstractItem*> items() const;
02803
02804     static AxisType marginSideToAxisType(QCP::MarginSide side);
02805
02806     static Qt::Orientation orientation(AxisType type) {
02807         return type == atBottom || type == atTop ? Qt::Horizontal : Qt::Vertical;
02808     }
02809     static AxisType opposite(AxisType type);
02810
02811     signals:
02812     void rangeChanged(const QCPRange &newRange);
02813     void rangeChanged(const QCPRange &newRange, const QCPRange &oldRange);
02814     void scaleTypeChanged(QCPAxis::ScaleType scaleType);
02815     void selectionChanged(const QCPAxis::SelectableParts &parts);
02816     void selectableChanged(const QCPAxis::SelectableParts &parts);
02817
02818     protected:
02819         // property members:
02820         // axis base:
02821         AxisType mAxisType;
02822         QCPAxisRect *mAxisRect;
02823         //int mOffset; // in QCPAxisPainter
02824         int mPadding;
02825         Qt::Orientation mOrientation;
02826         SelectableParts mSelectableParts, mSelectedParts;
02827         QPen mBasePen, mSelectedBasePen;
02828         //QCPLineEnding mLowerEnding, mUpperEnding; // in QCPAxisPainter
02829         // axis label:
02830         //int mLabelPadding; // in QCPAxisPainter
02831         QString mLabel;
02832         QFont mLabelFont, mSelectedLabelFont;
02833         QColor mLabelColor, mSelectedLabelColor;
02834         // tick labels:
02835         //int mTickLabelPadding; // in QCPAxisPainter
02836         bool mTickLabels;
02837         //double mTickLabelRotation; // in QCPAxisPainter
02838         QFont mTickLabelFont, mSelectedTickLabelFont;
02839         QColor mTickLabelColor, mSelectedTickLabelColor;
02840         int mNumberPrecision;
02841         QLatin1Char mNumberFormatChar;
02842         bool mNumberBeautifulPowers;
02843         //bool mNumberMultiplyCross; // QCPAxisPainter
02844         // ticks and subticks:
02845         bool mTicks;
02846         bool mSubTicks;
02847         //int mTickLengthIn, mTickLengthOut, mSubTickLengthIn, mSubTickLengthOut; // QCPAxisPainter
02848         QPen mTickPen, mSelectedTickPen;
02849         QPen mSubTickPen, mSelectedSubTickPen;
02850         // scale and range:
02851         QCPRange mRange;
02852         bool mRangeReversed;
02853         ScaleType mScaleType;
02854
02855         // non-property members:
02856         QCPGrid *mGrid;
02857         QCPAxisPainterPrivate *mAxisPainter;
02858         QSharedPointer<QCPAxisTicker> mTicker;
02859         QVector<double> mTickVector;
02860         QVector<QString> mTickVectorLabels;
02861         QVector<double> mSubTickVector;
02862         bool mCachedMarginValid;
02863         int mCachedMargin;
02864         bool mDragging;
02865         QCPRange mDragStartRange;
02866         QCP::AntialiasedElements mAADragBackup, mNotAADragBackup;
02867
02868         // introduced virtual methods:
02869         virtual int calculateMargin();
02870
02871         // reimplemented virtual methods:
02872         virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
02873         virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;

```

```

02874     virtual QCP::Interaction selectionCategory() const Q_DECL_OVERRIDE;
02875     // events:
02876     virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
*selectionStateChanged) Q_DECL_OVERRIDE;
02877     virtual void deselectEvent(bool *selectionStateChanged) Q_DECL_OVERRIDE;
02878     // mouse events:
02879     virtual void mousePressEvent(QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE;
02880     virtual void mouseMoveEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
02881     virtual void mouseReleaseEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
02882     virtual void wheelEvent(QWheelEvent *event) Q_DECL_OVERRIDE;
02883
02884     // non-virtual methods:
02885     void setupTickVectors();
02886     QPen getBasePen() const;
02887     QPen getTickPen() const;
02888     QPen getSubTickPen() const;
02889     QFont getTickLabelFont() const;
02890     QFont getLabelFont() const;
02891     QColor getTickLabelColor() const;
02892     QColor getLabelColor() const;
02893
02894 private:
02895     Q_DISABLE_COPY(QCPAxis)
02896
02897     friend class QCustomPlot;
02898     friend class QCPGrid;
02899     friend class QCPAxisRect;
02900 };
02901
02902 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPAxis::SelectableParts)
02903 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPAxis::AxisTypes)
02904 Q_DECLARE_METATYPE(QCPAxis::AxisType)
02905 Q_DECLARE_METATYPE(QCPAxis::LabelSide)
02906 Q_DECLARE_METATYPE(QCPAxis::ScaleType)
02907 Q_DECLARE_METATYPE(QCPAxis::SelectablePart)
02908
02909
02910 class QCPAxisPainterPrivate {
02911 public:
02912     explicit QCPAxisPainterPrivate(QCustomPlot *parentPlot);
02913     virtual ~QCPAxisPainterPrivate();
02914
02915     virtual void draw(QCPPainter *painter);
02916     virtual int size();
02917     void clearCache();
02918
02919     QRect axisSelectionBox() const {
02920         return mAxisSelectionBox;
02921     }
02922
02923     QRect tickLabelsSelectionBox() const {
02924         return mTickLabelsSelectionBox;
02925     }
02926
02927     QRect labelSelectionBox() const {
02928         return mLabelSelectionBox;
02929     }
02930
02931     // public property members:
02932     QCPAxis::AxisType type;
02933     QPen basePen;
02934     QCPLLineEnding lowerEnding, upperEnding; // directly accessed by QCPAxis setters/getters
02935     int labelPadding; // directly accessed by QCPAxis setters/getters
02936     QFont labelFont;
02937     QColor labelColor;
02938     QString label;
02939     int tickLabelPadding; // directly accessed by QCPAxis setters/getters
02940     double tickLabelRotation; // directly accessed by QCPAxis setters/getters
02941     QCPAxis::LabelSide tickLabelSide; // directly accessed by QCPAxis setters/getters
02942     bool substituteExponent;
02943     bool numberMultiplyCross; // directly accessed by QCPAxis setters/getters
02944     int tickLengthIn, tickLengthOut, subTickLengthIn, subTickLengthOut; // directly accessed by
QCPAxis setters/getters
02945     QPen tickPen, subTickPen;
02946     QFont tickLabelFont;
02947     QColor tickLabelColor;
02948     QRect axisRect, viewportRect;
02949     int offset; // directly accessed by QCPAxis setters/getters
02950     bool abbreviateDecimalPowers;
02951     bool reversedEndings;
02952
02953     QVector<double> subTickPositions;
02954     QVector<double> tickPositions;
02955     QVector<QString> tickLabels;
02956
02957 protected:
02958

```

```

02959     struct CachedLabel {
02960         QPointF offset;
02961         QPixmap pixmap;
02962     };
02963
02964     struct TickLabelData {
02965         QString basePart, expPart, suffixPart;
02966         QRect baseBounds, expBounds, suffixBounds, totalBounds, rotatedTotalBounds;
02967         QFont baseFont, expFont;
02968     };
02969     QCustomPlot *mParentPlot;
02970     QByteArray mLabelParameterHash; // to determine whether mLabelCache needs to be cleared due to
changed parameters
02971     QCache<QString, CachedLabel> mLabelCache;
02972     QRect mAxisSelectionBox, mTickLabelsSelectionBox, mLabelSelectionBox;
02973
02974     virtual QByteArray generateLabelParameterHash() const;
02975
02976     virtual void placeTickLabel(QCPPainter *painter, double position, int distanceToAxis, const
QString &text, QSize *tickLabelsSize);
02977     virtual void drawTickLabel(QCPPainter *painter, double x, double y, const TickLabelData
&labelData) const;
02978     virtual TickLabelData getTickLabelData(const QFont &font, const QString &text) const;
02979     virtual QPointF getTickLabelDrawOffset(const TickLabelData &labelData) const;
02980     virtual void getMaxTickLabelSize(const QFont &font, const QString &text, QSize *tickLabelsSize)
const;
02981 };
02982
02983 /* end of 'src/axis/axis.h' */
02984
02985
02986 /* including file 'src/scatterstyle.h' */
02987
02988 /* modified 2021-03-29T02:30:44, size 7275 */
02989
02990 class QCP_LIB_DECL QCPScatterStyle {
02991
02992     Q_GADGET
02993 public:
03002     enum ScatterProperty {
03003         spNone = 0x00
03004         , spPen = 0x01
03005         , spBrush = 0x02
03006         , spSize = 0x04
03007         , spShape = 0x08
03008         , spAll = 0xFF
03009     };
03010
03011     Q_ENUMS(ScatterProperty)
03012     Q_FLAGS(ScatterProperties)
03013     Q_DECLARE_FLAGS(ScatterProperties, ScatterProperty)
03014
03015
03022     enum ScatterShape {
03023         ssNone
03024         , ssDot
03025         , ssCross
03026         , ssPlus
03027         , ssCircle
03028         , ssDisc
03029         , ssSquare
03030         , ssDiamond
03031         , ssStar
03032         , ssTriangle
03033         , ssTriangleInverted
03034         , ssCrossSquare
03035         , ssPlusSquare
03036         , ssCrossCircle
03037         , ssPlusCircle
03038         , ssPeace
03039         , ssPixmap
03040         , ssCustom
03041     };
03042     Q_ENUMS(ScatterShape)
03043
03044     QCPScatterStyle();
03045     QCPScatterStyle(ScatterShape shape, double size = 6);
03046     QCPScatterStyle(ScatterShape shape, const QColor &color, double size);
03047     QCPScatterStyle(ScatterShape shape, const QColor &color, const QColor &fill, double size);
03048     QCPScatterStyle(ScatterShape shape, const QPen &pen, const QBrush &brush, double size);
03049     QCPScatterStyle(const QPixmap &pixmap);
03050     QCPScatterStyle(const QPainterPath &customPath, const QPen &pen, const QBrush &brush =
Qt::NoBrush, double size = 6);
03051
03052     // getters:
03053
03054     double size() const {

```

```

03055         return mSize;
03056     }
03057
03058     ScatterShape shape() const {
03059         return mShape;
03060     }
03061
03062     QPen pen() const {
03063         return mPen;
03064     }
03065
03066     QBrush brush() const {
03067         return mBrush;
03068     }
03069
03070     QPixmap pixmap() const {
03071         return mPixmap;
03072     }
03073
03074     QPainterPath customPath() const {
03075         return mCustomPath;
03076     }
03077
03078     // setters:
03079     void setFromOther(const QCPScatterStyle &other, ScatterProperties properties);
03080     void setSize(double size);
03081     void setShape(ScatterShape shape);
03082     void setPen(const QPen &pen);
03083     void setBrush(const QBrush &brush);
03084     void setPixmap(const QPixmap &pixmap);
03085     void setCustomPath(const QPainterPath &customPath);
03086
03087     // non-property methods:
03088
03089     bool isNone() const {
03090         return mShape == ssNone;
03091     }
03092
03093     bool isPenDefined() const {
03094         return mPenDefined;
03095     }
03096     void undefinePen();
03097     void applyTo(QCPPainter *painter, const QPen &defaultPen) const;
03098     void drawShape(QCPPainter *painter, const QPointF &pos) const;
03099     void drawShape(QCPPainter *painter, double x, double y) const;
03100
03101 protected:
03102     // property members:
03103     double mSize;
03104     ScatterShape mShape;
03105     QPen mPen;
03106     QBrush mBrush;
03107     QPixmap mPixmap;
03108     QPainterPath mCustomPath;
03109
03110     // non-property members:
03111     bool mPenDefined;
03112 };
03113 Q_DECLARE_TYPEINFO(QCPScatterStyle, Q_MOVABLE_TYPE);
03114
03115 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPScatterStyle::ScatterProperties)
03116 Q_DECLARE_METATYPE(QCPScatterStyle::ScatterProperty)
03117 Q_DECLARE_METATYPE(QCPScatterStyle::ScatterShape)
03118
03119 /* end of 'src/scatterstyle.h' */
03120
03121
03122 /* including file 'src/datacontainer.h' */
03123 /* modified 2021-03-29T02:30:44, size 34070 */
03124
03125
03130 template <class DataType>
03131 inline bool qcplLessThanSortKey(const DataType &a, const DataType &b) {
03132     return a.sortKey() < b.sortKey();
03133 }
03134
03135 template <class DataType>
03136 class QCPCDataContainer // no QCP_LIB_DECL, template class ends up in header (cpp included below)
03137 {
03138 public:
03139     typedef typename QVector<DataType>::const_iterator const_iterator;
03140     typedef typename QVector<DataType>::iterator iterator;
03141
03142     QCPCDataContainer();
03143
03144     // getters:
03145

```

```

03146     int size() const {
03147         return mData.size() - mPreallocSize;
03148     }
03149
03150     bool isEmpty() const {
03151         return size() == 0;
03152     }
03153
03154     bool autoSqueeze() const {
03155         return mAutoSqueeze;
03156     }
03157
03158     // setters:
03159     void setAutoSqueeze(bool enabled);
03160
03161     // non-virtual methods:
03162     void set(const QCPDataContainer<DataType> &data);
03163     void set(const QVector<DataType> &data, bool alreadySorted = false);
03164     void add(const QCPDataContainer<DataType> &data);
03165     void add(const QVector<DataType> &data, bool alreadySorted = false);
03166     void add(const DataType &data);
03167     void removeBefore(double sortKey);
03168     void removeAfter(double sortKey);
03169     void remove(double sortKeyFrom, double sortKeyTo);
03170     void remove(double sortKey);
03171     void clear();
03172     void sort();
03173     void squeeze(bool preAllocation = true, bool postAllocation = true);
03174
03175     const_iterator constBegin() const {
03176         return mData.constBegin() + mPreallocSize;
03177     }
03178
03179     const_iterator constEnd() const {
03180         return mData.constEnd();
03181     }
03182
03183     iterator begin() {
03184         return mData.begin() + mPreallocSize;
03185     }
03186
03187     iterator end() {
03188         return mData.end();
03189     }
03190     const_iterator findBegin(double sortKey, bool expandedRange = true) const;
03191     const_iterator findEnd(double sortKey, bool expandedRange = true) const;
03192
03193     const_iterator at(int index) const {
03194         return constBegin() + qBound(0, index, size());
03195     }
03196     QCPRange keyRange(bool &foundRange, QCP::SignDomain signDomain = QCP::sdBoth);
03197     QCPRange valueRange(bool &foundRange, QCP::SignDomain signDomain = QCP::sdBoth, const QCPRange
&inKeyRange = QCPRange());
03198
03199     QCPDataRange dataRange() const {
03200         return QCPDataRange(0, size());
03201     }
03202     void limitIteratorsToDataRange(const_iterator &begin, const_iterator &end, const QCPDataRange
&dataRange) const;
03203
03204 protected:
03205     // property members:
03206     bool mAutoSqueeze;
03207
03208     // non-property members:
03209     QVector<DataType> mData;
03210     int mPreallocSize;
03211     int mPreallocIteration;
03212
03213     // non-virtual methods:
03214     void preallocateGrow(int minimumPreallocSize);
03215     void performAutoSqueeze();
03216 };
03217
03218
03219
03220 // include implementation in header since it is a class template:
03224
03292 /* start documentation of inline functions */
03293
03348 /* end documentation of inline functions */
03349
03354 template <class DataType>
03355 QCPDataContainer<DataType>::QCPDataContainer() :
03356     mAutoSqueeze(true),
03357     mPreallocSize(0),
03358     mPreallocIteration(0) {

```

```

03359 }
03360
03369 template <class DataType>
03370 void QCPDataContainer<DataType>::setAutoSqueeze(bool enabled) {
03371     if (mAutoSqueeze != enabled) {
03372         mAutoSqueeze = enabled;
03373         if (mAutoSqueeze)
03374             performAutoSqueeze();
03375     }
03376 }
03377
03384 template <class DataType>
03385 void QCPDataContainer<DataType>::set(const QCPDataContainer<DataType> &data) {
03386     clear();
03387     add(data);
03388 }
03389
03399 template <class DataType>
03400 void QCPDataContainer<DataType>::set(const QVector<DataType> &data, bool alreadySorted) {
03401     mData = data;
03402     mPreallocSize = 0;
03403     mPreallocIteration = 0;
03404     if (!alreadySorted)
03405         sort();
03406 }
03407
03414 template <class DataType>
03415 void QCPDataContainer<DataType>::add(const QCPDataContainer<DataType> &data) {
03416     if (data.isEmpty())
03417         return;
03418
03419     const int n = data.size();
03420     const int oldSize = size();
03421
03422     if (oldSize > 0 && !qcpLessThanSortKey<DataType>(*constBegin(), *(data.constEnd() - 1))) //
    prepend if new data keys are all smaller than or equal to existing ones
03423     {
03424         if (mPreallocSize < n)
03425             preallocateGrow(n);
03426         mPreallocSize -= n;
03427         std::copy(data.constBegin(), data.constEnd(), begin());
03428     } else // don't need to prepend, so append and merge if necessary
03429     {
03430         mData.resize(mData.size() + n);
03431         std::copy(data.constBegin(), data.constEnd(), end() - n);
03432         if (oldSize > 0 && !qcpLessThanSortKey<DataType>*(constEnd() - n - 1), *(constEnd() - n))) //
    if appended range keys aren't all greater than existing ones, merge the two partitions
03433             std::inplace_merge(begin(), end() - n, end(), qcpLessThanSortKey<DataType>);
03434     }
03435 }
03436
03445 template <class DataType>
03446 void QCPDataContainer<DataType>::add(const QVector<DataType> &data, bool alreadySorted) {
03447     if (data.isEmpty())
03448         return;
03449     if (isEmpty()) {
03450         set(data, alreadySorted);
03451         return;
03452     }
03453
03454     const int n = data.size();
03455     const int oldSize = size();
03456
03457     if (alreadySorted && oldSize > 0 && !qcpLessThanSortKey<DataType>(*constBegin(), *(data.constEnd()
    - 1))) // prepend if new data is sorted and keys are all smaller than or equal to existing ones
03458     {
03459         if (mPreallocSize < n)
03460             preallocateGrow(n);
03461         mPreallocSize -= n;
03462         std::copy(data.constBegin(), data.constEnd(), begin());
03463     } else // don't need to prepend, so append and then sort and merge if necessary
03464     {
03465         mData.resize(mData.size() + n);
03466         std::copy(data.constBegin(), data.constEnd(), end() - n);
03467         if (!alreadySorted) // sort appended subrange if it wasn't already sorted
03468             std::sort(end() - n, end(), qcpLessThanSortKey<DataType>);
03469         if (oldSize > 0 && !qcpLessThanSortKey<DataType>*(constEnd() - n - 1), *(constEnd() - n))) //
    if appended range keys aren't all greater than existing ones, merge the two partitions
03470             std::inplace_merge(begin(), end() - n, end(), qcpLessThanSortKey<DataType>);
03471     }
03472 }
03473
03480 template <class DataType>
03481 void QCPDataContainer<DataType>::add(const DataType &data) {
03482     if (isEmpty() || !qcpLessThanSortKey<DataType>(data, *(constEnd() - 1))) // quickly handle appends
    if new data key is greater or equal to existing ones
03483     {

```

```

03484         mData.append(data);
03485     } else if (qcpLessThanSortKey<DataType>(data, *constBegin())) // quickly handle prepends using
    preallocated space
03486     {
03487         if (mPreallocSize < 1)
03488             preallocateGrow(1);
03489         --mPreallocSize;
03490         *begin() = data;
03491     } else // handle inserts, maintaining sorted keys
03492     {
03493         QCPDataContainer<DataType>::iterator insertionPoint = std::lower_bound(begin(), end(), data,
    qcpLessThanSortKey<DataType>);
03494         mData.insert(insertionPoint, data);
03495     }
03496 }
03497
03503 template <class DataType>
03504 void QCPDataContainer<DataType>::removeBefore(double sortKey) {
03505     QCPDataContainer<DataType>::iterator it = begin();
03506     QCPDataContainer<DataType>::iterator itEnd = std::lower_bound(begin(), end(),
    DataType::fromSortKey(sortKey), qcpLessThanSortKey<DataType>);
03507     mPreallocSize += int(itEnd - it); // don't actually delete, just add it to the preallocated block
    (if it gets too large, squeeze will take care of it)
03508     if (mAutoSqueeze)
03509         performAutoSqueeze();
03510 }
03511
03517 template <class DataType>
03518 void QCPDataContainer<DataType>::removeAfter(double sortKey) {
03519     QCPDataContainer<DataType>::iterator it = std::upper_bound(begin(), end(),
    DataType::fromSortKey(sortKey), qcpLessThanSortKey<DataType>);
03520     QCPDataContainer<DataType>::iterator itEnd = end();
03521     mData.erase(it, itEnd); // typically adds it to the postallocated block
03522     if (mAutoSqueeze)
03523         performAutoSqueeze();
03524 }
03525
03533 template <class DataType>
03534 void QCPDataContainer<DataType>::remove(double sortKeyFrom, double sortKeyTo) {
03535     if (sortKeyFrom >= sortKeyTo || isEmpty())
03536         return;
03537
03538     QCPDataContainer<DataType>::iterator it = std::lower_bound(begin(), end(),
    DataType::fromSortKey(sortKeyFrom), qcpLessThanSortKey<DataType>);
03539     QCPDataContainer<DataType>::iterator itEnd = std::upper_bound(it, end(),
    DataType::fromSortKey(sortKeyTo), qcpLessThanSortKey<DataType>);
03540     mData.erase(it, itEnd);
03541     if (mAutoSqueeze)
03542         performAutoSqueeze();
03543 }
03544
03554 template <class DataType>
03555 void QCPDataContainer<DataType>::remove(double sortKey) {
03556     QCPDataContainer<DataType>::iterator it = std::lower_bound(begin(), end(), DataType::fromSortKey(sortKey),
    qcpLessThanSortKey<DataType>);
03557     if (it != end() && it->sortKey() == sortKey) {
03558         if (it == begin())
03559             ++mPreallocSize; // don't actually delete, just add it to the preallocated block (if it
    gets too large, squeeze will take care of it)
03560         else
03561             mData.erase(it);
03562     }
03563     if (mAutoSqueeze)
03564         performAutoSqueeze();
03565 }
03566
03572 template <class DataType>
03573 void QCPDataContainer<DataType>::clear() {
03574     mData.clear();
03575     mPreallocIteration = 0;
03576     mPreallocSize = 0;
03577 }
03578
03590 template <class DataType>
03591 void QCPDataContainer<DataType>::sort() {
03592     std::sort(begin(), end(), qcpLessThanSortKey<DataType>);
03593 }
03594
03605 template <class DataType>
03606 void QCPDataContainer<DataType>::squeeze(bool preAllocation, bool postAllocation) {
03607     if (preAllocation) {
03608         if (mPreallocSize > 0) {
03609             std::copy(begin(), end(), mData.begin());
03610             mData.resize(size());
03611             mPreallocSize = 0;
03612         }
03613         mPreallocIteration = 0;

```

```

03614     }
03615     if (postAllocation)
03616         mData.squeeze();
03617 }
03618
03634 template <class DataType>
03635 typename QCPDataContainer<DataType>::const_iterator QCPDataContainer<DataType>::findBegin(double
    sortKey, bool expandedRange) const {
03636     if (isEmpty())
03637         return constEnd();
03638
03639     QCPDataContainer<DataType>::const_iterator it = std::lower_bound(constBegin(), constEnd(),
    DataType::fromSortKey(sortKey), qcpLessThanSortKey<DataType>);
03640     if (expandedRange && it != constBegin()) // also covers it == constEnd case, and we know
    --constEnd is valid because mData isn't empty
03641         --it;
03642     return it;
03643 }
03644
03660 template <class DataType>
03661 typename QCPDataContainer<DataType>::const_iterator QCPDataContainer<DataType>::findEnd(double
    sortKey, bool expandedRange) const {
03662     if (isEmpty())
03663         return constEnd();
03664
03665     QCPDataContainer<DataType>::const_iterator it = std::upper_bound(constBegin(), constEnd(),
    DataType::fromSortKey(sortKey), qcpLessThanSortKey<DataType>);
03666     if (expandedRange && it != constEnd())
03667         ++it;
03668     return it;
03669 }
03670
03686 template <class DataType>
03687 QCPRange QCPDataContainer<DataType>::keyRange(bool &foundRange, QCP::SignDomain signDomain) {
03688     if (isEmpty()) {
03689         foundRange = false;
03690         return QCPRange();
03691     }
03692     QCPRange range;
03693     bool haveLower = false;
03694     bool haveUpper = false;
03695     double current;
03696
03697     QCPDataContainer<DataType>::const_iterator it = constBegin();
03698     QCPDataContainer<DataType>::const_iterator itEnd = constEnd();
03699     if (signDomain == QCP::sdBoth) // range may be anywhere
03700     {
03701         if (DataType::sortKeyIsMainKey()) // if DataType is sorted by main key (e.g. QCPGraph, but not
    QCPCurve), use faster algorithm by finding just first and last key with non-NaN value
03702         {
03703             while (it != itEnd) // find first non-nan going up from left
03704             {
03705                 if (!qIsNaN(it->mainValue())) {
03706                     range.lower = it->mainKey();
03707                     haveLower = true;
03708                     break;
03709                 }
03710                 ++it;
03711             }
03712             it = itEnd;
03713             while (it != constBegin()) // find first non-nan going down from right
03714             {
03715                 --it;
03716                 if (!qIsNaN(it->mainValue())) {
03717                     range.upper = it->mainKey();
03718                     haveUpper = true;
03719                     break;
03720                 }
03721             }
03722         } else // DataType is not sorted by main key, go through all data points and accordingly
    expand range
03723         {
03724             while (it != itEnd) {
03725                 if (!qIsNaN(it->mainValue())) {
03726                     current = it->mainKey();
03727                     if (current < range.lower || !haveLower) {
03728                         range.lower = current;
03729                         haveLower = true;
03730                     }
03731                     if (current > range.upper || !haveUpper) {
03732                         range.upper = current;
03733                         haveUpper = true;
03734                     }
03735                 }
03736                 ++it;
03737             }
03738         }

```



```

03739     } else if (signDomain == QCP::sdNegative) // range may only be in the negative sign domain
03740     {
03741         while (it != itEnd) {
03742             if (!qIsNaN(it->mainValue())) {
03743                 current = it->mainKey();
03744                 if ((current < range.lower || !haveLower) && current < 0) {
03745                     range.lower = current;
03746                     haveLower = true;
03747                 }
03748                 if ((current > range.upper || !haveUpper) && current < 0) {
03749                     range.upper = current;
03750                     haveUpper = true;
03751                 }
03752             }
03753             ++it;
03754         }
03755     } else if (signDomain == QCP::sdPositive) // range may only be in the positive sign domain
03756     {
03757         while (it != itEnd) {
03758             if (!qIsNaN(it->mainValue())) {
03759                 current = it->mainKey();
03760                 if ((current < range.lower || !haveLower) && current > 0) {
03761                     range.lower = current;
03762                     haveLower = true;
03763                 }
03764                 if ((current > range.upper || !haveUpper) && current > 0) {
03765                     range.upper = current;
03766                     haveUpper = true;
03767                 }
03768             }
03769             ++it;
03770         }
03771     }
03772
03773     foundRange = haveLower && haveUpper;
03774     return range;
03775 }
03776
03793 template <class DataType>
03794 QCPRange QCPDataContainer<DataType>::valueRange(bool &foundRange, QCP::SignDomain signDomain, const
QCPRange &inKeyRange) {
03795     if (isEmpty()) {
03796         foundRange = false;
03797         return QCPRange();
03798     }
03799     QCPRange range;
03800     const bool restrictKeyRange = inKeyRange != QCPRange();
03801     bool haveLower = false;
03802     bool haveUpper = false;
03803     QCPRange current;
03804     QCPDataContainer<DataType>::const_iterator itBegin = constBegin();
03805     QCPDataContainer<DataType>::const_iterator itEnd = constEnd();
03806     if (DataType::sortKeyIsMainKey() && restrictKeyRange) {
03807         itBegin = findBegin(inKeyRange.lower, false);
03808         itEnd = findEnd(inKeyRange.upper, false);
03809     }
03810     if (signDomain == QCP::sdBoth) // range may be anywhere
03811     {
03812         for (QCPDataContainer<DataType>::const_iterator it = itBegin; it != itEnd; ++it) {
03813             if (restrictKeyRange && (it->mainKey() < inKeyRange.lower || it->mainKey() >
inKeyRange.upper))
03814                 continue;
03815             current = it->valueRange();
03816             if ((current.lower < range.lower || !haveLower) && !qIsNaN(current.lower)) {
03817                 range.lower = current.lower;
03818                 haveLower = true;
03819             }
03820             if ((current.upper > range.upper || !haveUpper) && !qIsNaN(current.upper)) {
03821                 range.upper = current.upper;
03822                 haveUpper = true;
03823             }
03824         }
03825     } else if (signDomain == QCP::sdNegative) // range may only be in the negative sign domain
03826     {
03827         for (QCPDataContainer<DataType>::const_iterator it = itBegin; it != itEnd; ++it) {
03828             if (restrictKeyRange && (it->mainKey() < inKeyRange.lower || it->mainKey() >
inKeyRange.upper))
03829                 continue;
03830             current = it->valueRange();
03831             if ((current.lower < range.lower || !haveLower) && current.lower < 0 &&
!qIsNaN(current.lower)) {
03832                 range.lower = current.lower;
03833                 haveLower = true;
03834             }
03835             if ((current.upper > range.upper || !haveUpper) && current.upper < 0 &&
!qIsNaN(current.upper)) {
03836                 range.upper = current.upper;

```

```

03837         haveUpper = true;
03838     }
03839 }
03840 } else if (signDomain == QCP::sdPositive) // range may only be in the positive sign domain
03841 {
03842     for (QCPDataContainer<DataType>::const_iterator it = itBegin; it != itEnd; ++it) {
03843         if (restrictKeyRange && (it->mainKey() < inKeyRange.lower || it->mainKey() >
inKeyRange.upper))
03844             continue;
03845         current = it->valueRange();
03846         if ((current.lower < range.lower || !haveLower) && current.lower > 0 &&
!qIsNaN(current.lower)) {
03847             range.lower = current.lower;
03848             haveLower = true;
03849         }
03850         if ((current.upper > range.upper || !haveUpper) && current.upper > 0 &&
!qIsNaN(current.upper)) {
03851             range.upper = current.upper;
03852             haveUpper = true;
03853         }
03854     }
03855 }
03856
03857 foundRange = haveLower && haveUpper;
03858 return range;
03859 }
03860
03861 template <class DataType>
03862 void QCPDataContainer<DataType>::limitIteratorsToDataRange(const_iterator &begin, const_iterator &end,
const QCPDataRange &dataRange) const {
03871     QCPDataRange iteratorRange(int(begin - constBegin()), int(end - constBegin()));
03872     iteratorRange = iteratorRange.bounded(dataRange.bounded(this->dataRange()));
03873     begin = constBegin() + iteratorRange.begin();
03874     end = constBegin() + iteratorRange.end();
03875 }
03876
03877 template <class DataType>
03878 void QCPDataContainer<DataType>::preallocateGrow(int minimumPreallocSize) {
03879     if (minimumPreallocSize <= mPreallocSize)
03880         return;
03881     int newPreallocSize = minimumPreallocSize;
03882     newPreallocSize += (1u << qBound(4, mPreallocIteration + 4, 15)) - 12; // do 4 up to 32768-12
preallocation, doubling in each intermediate iteration
03893     ++mPreallocIteration;
03894
03895     int sizeDifference = newPreallocSize - mPreallocSize;
03896     mData.resize(mData.size() + sizeDifference);
03897     std::copy_backward(mData.begin() + mPreallocSize, mData.end() - sizeDifference, mData.end());
03898     mPreallocSize = newPreallocSize;
03899 }
03900
03901 template <class DataType>
03902 void QCPDataContainer<DataType>::performAutoSqueeze() {
03903     const int totalAlloc = mData.capacity();
03904     const int postAllocSize = totalAlloc - mData.size();
03905     const int usedSize = size();
03906     bool shrinkPostAllocation = false;
03907     bool shrinkPreAllocation = false;
03908     if (totalAlloc > 650000) // if allocation is larger, shrink earlier with respect to total used
size
03923     {
03924         shrinkPostAllocation = postAllocSize > usedSize * 1.5; // QVector grow strategy is 2^n for
static data. Watch out not to oscillate!
03925         shrinkPreAllocation = mPreallocSize * 10 > usedSize;
03926     } else if (totalAlloc > 1000) // below 10 MiB raw data be generous with preallocated memory, below
1k points don't even bother
03927     {
03928         shrinkPostAllocation = postAllocSize > usedSize * 5;
03929         shrinkPreAllocation = mPreallocSize > usedSize * 1.5; // preallocation can grow into
postallocation, so can be smaller
03930     }
03931
03932     if (shrinkPreAllocation || shrinkPostAllocation)
03933         squeeze(shrinkPreAllocation, shrinkPostAllocation);
03934 }
03935
03936
03937 /* end of 'src/datacontainer.h' */
03938
03939
03940 /* including file 'src/plottable.h' */
03941
03942 /* modified 2021-03-29T02:30:44, size 8461 */
03943
03944 class QCP_LIB_DECL QCPSelectionDecorator {
03945     Q_GADGET

```

```

03946 public:
03947     QCPSelectionDecorator();
03948     virtual ~QCPSelectionDecorator();
03949
03950     // getters:
03951
03952     QPen pen() const {
03953         return mPen;
03954     }
03955
03956     QBrush brush() const {
03957         return mBrush;
03958     }
03959
03960     QCPScatterStyle scatterStyle() const {
03961         return mScatterStyle;
03962     }
03963
03964     QCPScatterStyle::ScatterProperties usedScatterProperties() const {
03965         return mUsedScatterProperties;
03966     }
03967
03968     // setters:
03969     void setPen(const QPen &pen);
03970     void setBrush(const QBrush &brush);
03971     void setScatterStyle(const QCPScatterStyle &scatterStyle, QCPScatterStyle::ScatterProperties
usedProperties = QCPScatterStyle::spPen);
03972     void setUsedScatterProperties(const QCPScatterStyle::ScatterProperties &properties);
03973
03974     // non-virtual methods:
03975     void applyPen(QCPPainter *painter) const;
03976     void applyBrush(QCPPainter *painter) const;
03977     QCPScatterStyle getFinalScatterStyle(const QCPScatterStyle &unselectedStyle) const;
03978
03979     // introduced virtual methods:
03980     virtual void copyFrom(const QCPSelectionDecorator *other);
03981     virtual void drawDecoration(QCPPainter *painter, QCPDataSelection selection);
03982
03983 protected:
03984     // property members:
03985     QPen mPen;
03986     QBrush mBrush;
03987     QCPScatterStyle mScatterStyle;
03988     QCPScatterStyle::ScatterProperties mUsedScatterProperties;
03989     // non-property members:
03990     QCPAbstractPlottable *mPlottable;
03991
03992     // introduced virtual methods:
03993     virtual bool registerWithPlottable(QCPAbstractPlottable *plottable);
03994
03995 private:
03996     Q_DISABLE_COPY(QCPSelectionDecorator)
03997     friend class QCPAbstractPlottable;
03998 };
03999
04000 Q_DECLARE_METATYPE(QCPSelectionDecorator*)
04001
04002
04003 class QCP_LIB_DECL QCPAbstractPlottable : public QCPLayerable {
04004     Q_OBJECT
04006     Q_PROPERTY(QString name READ name WRITE setName)
04007     Q_PROPERTY(bool antialiasedFill READ antialiasedFill WRITE setAntialiasedFill)
04008     Q_PROPERTY(bool antialiasedScatters READ antialiasedScatters WRITE setAntialiasedScatters)
04009     Q_PROPERTY(QPen pen READ pen WRITE setPen)
04010     Q_PROPERTY(QBrush brush READ brush WRITE setBrush)
04011     Q_PROPERTY(QCPAxis* keyAxis READ keyAxis WRITE setKeyAxis)
04012     Q_PROPERTY(QCPAxis* valueAxis READ valueAxis WRITE setValueAxis)
04013     Q_PROPERTY(QCP::SelectionType selectable READ selectable WRITE setSelectable NOTIFY
selectableChanged)
04014     Q_PROPERTY(QCPDataSelection selection READ selection WRITE setSelection NOTIFY selectionChanged)
04015     Q_PROPERTY(QCPSelectionDecorator* selectionDecorator READ selectionDecorator WRITE
setSelectionDecorator)
04017 public:
04018     QCPAbstractPlottable(QCPAxis *keyAxis, QCPAxis *valueAxis);
04019     virtual ~QCPAbstractPlottable() Q_DECL_OVERRIDE;
04020
04021     // getters:
04022
04023     QString name() const {
04024         return mName;
04025     }
04026
04027     bool antialiasedFill() const {
04028         return mAntialiasedFill;
04029     }
04030
04031     bool antialiasedScatters() const {

```

```

04032         return mAntialiasedScatters;
04033     }
04034
04035     QPen pen() const {
04036         return mPen;
04037     }
04038
04039     QBrush brush() const {
04040         return mBrush;
04041     }
04042
04043     QCPAxis *keyAxis() const {
04044         return mKeyAxis.data();
04045     }
04046
04047     QCPAxis *valueAxis() const {
04048         return mValueAxis.data();
04049     }
04050
04051     QCP::SelectionType selectable() const {
04052         return mSelectable;
04053     }
04054
04055     bool selected() const {
04056         return !mSelection.isEmpty();
04057     }
04058
04059     QCPDataSelection selection() const {
04060         return mSelection;
04061     }
04062
04063     QCPSelectionDecorator *selectionDecorator() const {
04064         return mSelectionDecorator;
04065     }
04066
04067     // setters:
04068     void setName(const QString &name);
04069     void setAntialiasedFill(bool enabled);
04070     void setAntialiasedScatters(bool enabled);
04071     void setPen(const QPen &pen);
04072     void setBrush(const QBrush &brush);
04073     void setKeyAxis(QCPAxis *axis);
04074     void setValueAxis(QCPAxis *axis);
04075     Q_SLOT void setSelectable(QCP::SelectionType selectable);
04076     Q_SLOT void setSelection(QCPDataSelection selection);
04077     void setSelectionDecorator(QCPSelectionDecorator *decorator);
04078
04079     // introduced virtual methods:
04080     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE = 0; // actually introduced in QCPLayerable as non-pure, but we want to force
reimplementation for plottables
04081
04082     virtual QCPPlottableInterfaceID *interfaceID() {
04083         return nullptr;
04084     }
04085     virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const =
0;
04086     virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
QCPRange &inKeyRange = QCPRange()) const = 0;
04087
04088     // non-property methods:
04089     void coordsToPixels(double key, double value, double &x, double &y) const;
04090     const QPointF coordsToPixels(double key, double value) const;
04091     void pixelsToCoords(double x, double y, double &key, double &value) const;
04092     void pixelsToCoords(const QPointF &pixelPos, double &key, double &value) const;
04093     void rescaleAxes(bool onlyEnlarge = false) const;
04094     void rescaleKeyAxis(bool onlyEnlarge = false) const;
04095     void rescaleValueAxis(bool onlyEnlarge = false, bool inKeyRange = false) const;
04096     bool addToLegend(QCPLegend *legend);
04097     bool addToLegend();
04098     bool removeFromLegend(QCPLegend *legend) const;
04099     bool removeFromLegend() const;
04100
04101 signals:
04102     void selectionChanged(bool selected);
04103     void selectionChanged(const QCPDataSelection &selection);
04104     void selectableChanged(QCP::SelectionType selectable);
04105
04106 protected:
04107     // property members:
04108     QString mName;
04109     bool mAntialiasedFill, mAntialiasedScatters;
04110     QPen mPen;
04111     QBrush mBrush;
04112     QPointer<QCPAxis> mKeyAxis, mValueAxis;
04113     QCP::SelectionType mSelectable;
04114     QCPDataSelection mSelection;

```

```

04115     QCPSelectionDecorator *mSelectionDecorator;
04116
04117     // reimplemented virtual methods:
04118     virtual QRect clipRect() const Q_DECL_OVERRIDE;
04119     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE = 0;
04120     virtual QCP::Interaction selectionCategory() const Q_DECL_OVERRIDE;
04121     void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
04122     // events:
04123     virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
*selectionStateChanged) Q_DECL_OVERRIDE;
04124     virtual void deselectEvent(bool *selectionStateChanged) Q_DECL_OVERRIDE;
04125
04126     // introduced virtual methods:
04127     virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const = 0;
04128
04129     // non-virtual methods:
04130     void applyFillAntialiasingHint(QCPPainter *painter) const;
04131     void applyScattersAntialiasingHint(QCPPainter *painter) const;
04132
04133 private:
04134     Q_DISABLE_COPY(QCPAbstractPlottable)
04135
04136     friend class QCustomPlot;
04137     friend class QCPAxis;
04138     friend class QCPPlottableLegendItem;
04139 };
04140
04141
04142 /* end of 'src/plottable.h' */
04143
04144
04145 /* including file 'src/item.h' */
04146
04147 /* modified 2021-03-29T02:30:44, size 9425 */
04148
04149 class QCP_LIB_DECL QCPItemAnchor {
04150     Q_GADGET
04151 public:
04152     QCPItemAnchor(QCustomPlot *parentPlot, QCPAbstractItem *parentItem, const QString &name, int
anchorId = -1);
04153     virtual ~QCPItemAnchor();
04154
04155     // getters:
04156
04157     QString name() const {
04158         return mName;
04159     }
04160     virtual QPointF pixelPosition() const;
04161
04162 protected:
04163     // property members:
04164     QString mName;
04165
04166     // non-property members:
04167     QCustomPlot *mParentPlot;
04168     QCPAbstractItem *mParentItem;
04169     int mAnchorId;
04170     QSet<QCPItemPosition*> mChildrenX, mChildrenY;
04171
04172     // introduced virtual methods:
04173
04174     virtual QCPItemPosition *toQCPItemPosition() {
04175         return nullptr;
04176     }
04177
04178     // non-virtual methods:
04179     void addChildX(QCPItemPosition* pos); // called from pos when this anchor is set as parent
04180     void removeChildX(QCPItemPosition *pos); // called from pos when its parent anchor is reset or pos
deleted
04181     void addChildY(QCPItemPosition* pos); // called from pos when this anchor is set as parent
04182     void removeChildY(QCPItemPosition *pos); // called from pos when its parent anchor is reset or pos
deleted
04183
04184 private:
04185     Q_DISABLE_COPY(QCPItemAnchor)
04186
04187     friend class QCPItemPosition;
04188 };
04189
04190 class QCP_LIB_DECL QCPItemPosition : public QCPItemAnchor {
04191     Q_GADGET
04192 public:
04193     enum PositionType {
04194         ptAbsolute
04195         , ptViewportRatio
04196         , ptAxisRectRatio
04197     };

```

```

04208         , ptPlotCoords
04209     };
04210     Q_ENUMS(PositionType)
04211
04212     QCPItemPosition(QCustomPlot *parentPlot, QCPAbstractItem *parentItem, const QString &name);
04213     virtual ~QCPItemPosition() Q_DECL_OVERRIDE;
04214
04215     // getters:
04216
04217     PositionType type() const {
04218         return typeX();
04219     }
04220
04221     PositionType typeX() const {
04222         return mPositionTypeX;
04223     }
04224
04225     PositionType typeY() const {
04226         return mPositionTypeY;
04227     }
04228
04229     QCPItemAnchor *parentAnchor() const {
04230         return parentAnchorX();
04231     }
04232
04233     QCPItemAnchor *parentAnchorX() const {
04234         return mParentAnchorX;
04235     }
04236
04237     QCPItemAnchor *parentAnchorY() const {
04238         return mParentAnchorY;
04239     }
04240
04241     double key() const {
04242         return mKey;
04243     }
04244
04245     double value() const {
04246         return mValue;
04247     }
04248
04249     QPointF coords() const {
04250         return QPointF(mKey, mValue);
04251     }
04252
04253     QCPAxis *keyAxis() const {
04254         return mKeyAxis.data();
04255     }
04256
04257     QCPAxis *valueAxis() const {
04258         return mValueAxis.data();
04259     }
04260     QCPAxisRect *axisRect() const;
04261     virtual QPointF pixelPosition() const Q_DECL_OVERRIDE;
04262
04263     // setters:
04264     void setType(PositionType type);
04265     void setTypeX(PositionType type);
04266     void setTypeY(PositionType type);
04267     bool setParentAnchor(QCPItemAnchor *parentAnchor, bool keepPixelPosition = false);
04268     bool setParentAnchorX(QCPItemAnchor *parentAnchor, bool keepPixelPosition = false);
04269     bool setParentAnchorY(QCPItemAnchor *parentAnchor, bool keepPixelPosition = false);
04270     void setCoords(double key, double value);
04271     void setCoords(const QPointF &pos);
04272     void setAxes(QCPAxis* keyAxis, QCPAxis* valueAxis);
04273     void setAxisRect(QCPAxisRect *axisRect);
04274     void setPixelPosition(const QPointF &pixelPosition);
04275
04276 protected:
04277     // property members:
04278     PositionType mPositionTypeX, mPositionTypeY;
04279     QPointer<QCPAxis> mKeyAxis, mValueAxis;
04280     QPointer<QCPAxisRect> mAxisRect;
04281     double mKey, mValue;
04282     QCPItemAnchor *mParentAnchorX, *mParentAnchorY;
04283
04284     // reimplemented virtual methods:
04285
04286     virtual QCPItemPosition *toQCPItemPosition() Q_DECL_OVERRIDE {
04287         return this;
04288     }
04289
04290 private:
04291     Q_DISABLE_COPY(QCPItemPosition)
04292
04293 };
04294

```

```

04295 Q_DECLARE_METATYPE(QCItemPosition::PositionType)
04296
04297
04298 class QCP_LIB_DECL QCPAbstractItem : public QCPLayerable {
04299     Q_OBJECT
04300     Q_PROPERTY(bool clipToAxisRect READ clipToAxisRect WRITE setClipToAxisRect)
04301     Q_PROPERTY(QCPAxisRect* clipAxisRect READ clipAxisRect WRITE setClipAxisRect)
04302     Q_PROPERTY(bool selectable READ selectable WRITE setSelectable NOTIFY selectableChanged)
04303     Q_PROPERTY(bool selected READ selected WRITE setSelected NOTIFY selectionChanged)
04304 public:
04305     explicit QCPAbstractItem(QCustomPlot *parentPlot);
04306     virtual ~QCPAbstractItem() Q_DECL_OVERRIDE;
04307
04308     // getters:
04309
04310     bool clipToAxisRect() const {
04311         return mClipToAxisRect;
04312     }
04313     QCPAxisRect *clipAxisRect() const;
04314
04315     bool selectable() const {
04316         return mSelectable;
04317     }
04318
04319     bool selected() const {
04320         return mSelected;
04321     }
04322
04323     // setters:
04324     void setClipToAxisRect(bool clip);
04325     void setClipAxisRect(QCPAxisRect *rect);
04326     Q_SLOT void setSelectable(bool selectable);
04327     Q_SLOT void setSelected(bool selected);
04328
04329     // reimplemented virtual methods:
04330     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr) const Q_DECL_OVERRIDE = 0;
04331
04332     // non-virtual methods:
04333     QList<QCItemPosition*> positions() const {
04334         return mPositions;
04335     }
04336     QList<QCItemAnchor*> anchors() const {
04337         return mAnchors;
04338     }
04339     QCItemPosition *position(const QString &name) const;
04340     QCItemAnchor *anchor(const QString &name) const;
04341     bool hasAnchor(const QString &name) const;
04342
04343 signals:
04344     void selectionChanged(bool selected);
04345     void selectableChanged(bool selectable);
04346
04347 protected:
04348     // property members:
04349     bool mClipToAxisRect;
04350     QPointer<QCPAxisRect> mClipAxisRect;
04351     QList<QCItemPosition*> mPositions;
04352     QList<QCItemAnchor*> mAnchors;
04353     bool mSelectable, mSelected;
04354
04355     // reimplemented virtual methods:
04356     virtual QCP::Interaction selectionCategory() const Q_DECL_OVERRIDE;
04357     virtual QRect clipRect() const Q_DECL_OVERRIDE;
04358     virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
04359     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE = 0;
04360
04361     // events:
04362     virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool *selectionStateChanged) Q_DECL_OVERRIDE;
04363     virtual void deselectEvent(bool *selectionStateChanged) Q_DECL_OVERRIDE;
04364
04365     // introduced virtual methods:
04366     virtual QPointF anchorPixelPosition(int anchorId) const;
04367
04368     // non-virtual methods:
04369     double rectDistance(const QRectF &rect, const QPointF &pos, bool filledRect) const;
04370     QCItemPosition *createPosition(const QString &name);
04371     QCItemAnchor *createAnchor(const QString &name, int anchorId);
04372
04373 private:
04374     Q_DISABLE_COPY(QCPAbstractItem)
04375
04376     friend class QCustomPlot;
04377     friend class QCItemAnchor;
04378 };

```

```

04382
04383 /* end of 'src/item.h' */
04384
04385
04386 /* including file 'src/core.h' */
04387
04388 /* modified 2021-03-29T02:30:44, size 19304 */
04389
04390 class QCP_LIB_DECL QCustomPlot : public QWidget {
04391
04392     Q_OBJECT
04393     Q_PROPERTY(QRect viewport READ viewport WRITE setViewport)
04394     Q_PROPERTY(QPixmap background READ background WRITE setBackground)
04395     Q_PROPERTY(bool backgroundScaled READ backgroundScaled WRITE setBackgroundScaled)
04396     Q_PROPERTY(Qt::AspectRatioMode backgroundScaledMode READ backgroundScaledMode WRITE
setBackgroundScaledMode)
04397     Q_PROPERTY(QCPLLayoutGrid* plotLayout READ plotLayout)
04398     Q_PROPERTY(bool autoAddPlottableToLegend READ autoAddPlottableToLegend WRITE
setAutoAddPlottableToLegend)
04399     Q_PROPERTY(int selectionTolerance READ selectionTolerance WRITE setSelectionTolerance)
04400     Q_PROPERTY(bool noAntialiasingOnDrag READ noAntialiasingOnDrag WRITE setNoAntialiasingOnDrag)
04401     Q_PROPERTY(Qt::KeyboardModifier multiSelectModifier READ multiSelectModifier WRITE
setMultiSelectModifier)
04402     Q_PROPERTY(bool openGl READ openGl WRITE setOpenGl)
04403 public:
04404     enum LayerInsertMode {
04405         limBelow
04406         , limAbove
04407     };
04408     Q_ENUMS(LayerInsertMode)
04409
04410     enum RefreshPriority {
04411         rpImmediateRefresh
04412         , rpQueuedRefresh
04413         , rpRefreshHint
04414         , rpQueuedReplot
04415     };
04416     Q_ENUMS(RefreshPriority)
04417
04418     explicit QCustomPlot(QWidget *parent = nullptr);
04419     virtual ~QCustomPlot() Q_DECL_OVERRIDE;
04420
04421     // getters:
04422
04423     QRect viewport() const {
04424         return mViewport;
04425     }
04426
04427     double bufferDevicePixelRatio() const {
04428         return mBufferDevicePixelRatio;
04429     }
04430
04431     QPixmap background() const {
04432         return mBackgroundPixmap;
04433     }
04434
04435     bool backgroundScaled() const {
04436         return mBackgroundScaled;
04437     }
04438
04439     Qt::AspectRatioMode backgroundScaledMode() const {
04440         return mBackgroundScaledMode;
04441     }
04442
04443     QCPLLayoutGrid* plotLayout() const {
04444         return mPlotLayout;
04445     }
04446
04447     QCP::AntialiasedElements antialiasedElements() const {
04448         return mAntialiasedElements;
04449     }
04450
04451     QCP::AntialiasedElements notAntialiasedElements() const {
04452         return mNotAntialiasedElements;
04453     }
04454
04455     bool autoAddPlottableToLegend() const {
04456         return mAutoAddPlottableToLegend;
04457     }
04458
04459     const QCP::Interactions interactions() const {
04460         return mInteractions;
04461     }
04462
04463     int selectionTolerance() const {

```



```

04477         return mSelectionTolerance;
04478     }
04479
04480     bool noAntialiasingOnDrag() const {
04481         return mNoAntialiasingOnDrag;
04482     }
04483
04484     QCP::PlottingHints plottingHints() const {
04485         return mPlottingHints;
04486     }
04487
04488     Qt::KeyboardModifier multiSelectModifier() const {
04489         return mMultiSelectModifier;
04490     }
04491
04492     QCP::SelectionRectMode selectionRectMode() const {
04493         return mSelectionRectMode;
04494     }
04495
04496     QCPSelectionRect *selectionRect() const {
04497         return mSelectionRect;
04498     }
04499
04500     bool openGl() const {
04501         return mOpenGl;
04502     }
04503
04504     // setters:
04505     void setViewport(const QRect &rect);
04506     void setBufferDevicePixelRatio(double ratio);
04507     void setBackground(const QPixmap &pm);
04508     void setBackground(const QPixmap &pm, bool scaled, Qt::AspectRatioMode mode =
04509         Qt::KeepAspectRatioByExpanding);
04510     void setBackground(const QBrush &brush);
04511     void setBackgroundScaled(bool scaled);
04512     void setBackgroundScaledMode(Qt::AspectRatioMode mode);
04513     void setAntialiasedElements(const QCP::AntialiasedElements &antialiasedElements);
04514     void setAntialiasedElement(QCP::AntialiasedElement antialiasedElement, bool enabled = true);
04515     void setNotAntialiasedElements(const QCP::AntialiasedElements &notAntialiasedElements);
04516     void setNotAntialiasedElement(QCP::AntialiasedElement notAntialiasedElement, bool enabled = true);
04517     void setAutoAddPlottableToLegend(bool on);
04518     void setInteractions(const QCP::Interactions &interactions);
04519     void setInteraction(const QCP::Interaction &interaction, bool enabled = true);
04520     void setSelectionTolerance(int pixels);
04521     void setNoAntialiasingOnDrag(bool enabled);
04522     void setPlottingHints(const QCP::PlottingHints &hints);
04523     void setPlottingHint(QCP::PlottingHint hint, bool enabled = true);
04524     void setMultiSelectModifier(Qt::KeyboardModifier modifier);
04525     void setSelectionRectMode(QCP::SelectionRectMode mode);
04526     void setSelectionRect(QCPSelectionRect *selectionRect);
04527     void setOpenGl(bool enabled, int multisampling = 16);
04528
04529     // non-property methods:
04530     // plottable interface:
04531     QCPAbstractPlottable *plottable(int index);
04532     QCPAbstractPlottable *plottable();
04533     bool removePlottable(QCPAbstractPlottable *plottable);
04534     bool removePlottable(int index);
04535     int clearPlottables();
04536     int plottableCount() const;
04537     QList<QCPAbstractPlottable*> selectedPlottables() const;
04538     template<class PlottableType>
04539     PlottableType *plottableAt(const QPointF &pos, bool onlySelectable = false, int *dataIndex =
04540         nullptr) const;
04541     QCPAbstractPlottable *plottableAt(const QPointF &pos, bool onlySelectable = false, int *dataIndex
04542         = nullptr) const;
04543     bool hasPlottable(QCPAbstractPlottable *plottable) const;
04544
04545     // specialized interface for QCPGraph:
04546     QCPGraph *graph(int index) const;
04547     QCPGraph *graph();
04548     QCPGraph *addGraph(QCPAxis *keyAxis = nullptr, QCPAxis *valueAxis = nullptr);
04549     bool removeGraph(QCPGraph *graph);
04550     bool removeGraph(int index);
04551     int clearGraphs();
04552     int graphCount() const;
04553     QList<QCPGraph*> selectedGraphs() const;
04554
04555     // item interface:
04556     QCPAbstractItem *item(int index) const;
04557     QCPAbstractItem *item();
04558     bool removeItem(QCPAbstractItem *item);
04559     bool removeItem(int index);
04560     int clearItems();
04561     int itemCount() const;
04562     QList<QCPAbstractItem*> selectedItems() const;
04563     template<class ItemType>

```

```

04561     ItemType *itemAt(const QPointF &pos, bool onlySelectable = false) const;
04562     QCPAbstractItem *itemAt(const QPointF &pos, bool onlySelectable = false) const;
04563     bool hasItem(QCPAbstractItem *item) const;
04564
04565     // layer interface:
04566     QCPLayer *layer(const QString &name) const;
04567     QCPLayer *layer(int index) const;
04568     QCPLayer *currentLayer() const;
04569     bool setCurrentLayer(const QString &name);
04570     bool setCurrentLayer(QCPLayer *layer);
04571     int layerCount() const;
04572     bool addLayer(const QString &name, QCPLayer *otherLayer = nullptr, LayerInsertMode insertMode =
limAbove);
04573     bool removeLayer(QCPLayer *layer);
04574     bool moveLayer(QCPLayer *layer, QCPLayer *otherLayer, LayerInsertMode insertMode = limAbove);
04575
04576     // axis rect/layout interface:
04577     int axisRectCount() const;
04578     QCPAxisRect* axisRect(int index = 0) const;
04579     QList<QCPAxisRect*> axisRects() const;
04580     QCPLayoutElement* layoutElementAt(const QPointF &pos) const;
04581     QCPAxisRect* axisRectAt(const QPointF &pos) const;
04582     Q_SLOT void rescaleAxes(bool onlyVisiblePlottables = false);
04583
04584     QList<QCPAxis*> selectedAxes() const;
04585     QList<QCPLegend*> selectedLegends() const;
04586     Q_SLOT void deselectAll();
04587
04588     bool savePdf(const QString &fileName, int width = 0, int height = 0, QCP::ExportPen exportPen =
QCP::epAllowCosmetic, const QString &pdfCreator = QString(), const QString &pdfTitle = QString());
04589     bool savePng(const QString &fileName, int width = 0, int height = 0, double scale = 1.0, int
quality = -1, int resolution = 96, QCP::ResolutionUnit resolutionUnit = QCP::ruDotsPerInch);
04590     bool saveJpg(const QString &fileName, int width = 0, int height = 0, double scale = 1.0, int
quality = -1, int resolution = 96, QCP::ResolutionUnit resolutionUnit = QCP::ruDotsPerInch);
04591     bool saveBmp(const QString &fileName, int width = 0, int height = 0, double scale = 1.0, int
resolution = 96, QCP::ResolutionUnit resolutionUnit = QCP::ruDotsPerInch);
04592     bool saveRastered(const QString &fileName, int width, int height, double scale, const char
*format, int quality = -1, int resolution = 96, QCP::ResolutionUnit resolutionUnit =
QCP::ruDotsPerInch);
04593     QPixmap toPixmap(int width = 0, int height = 0, double scale = 1.0);
04594     void toPainter(QCPPainter *painter, int width = 0, int height = 0);
04595     Q_SLOT void replot(QCustomPlot::RefreshPriority refreshPriority = QCustomPlot::rpRefreshHint);
04596     double replotTime(bool average = false) const;
04597
04598     QCPAxis *xAxis, *yAxis, *xAxis2, *yAxis2;
04599     QCPLegend *legend;
04600
04601 signals:
04602     void mouseDoubleClick(QMouseEvent *event);
04603     void mousePress(QMouseEvent *event);
04604     void mouseMove(QMouseEvent *event);
04605     void mouseRelease(QMouseEvent *event);
04606     void mouseWheel(QWheelEvent *event);
04607
04608     void plottableClick(QCPAbstractPlottable *plottable, int dataIndex, QMouseEvent *event);
04609     void plottableDoubleClick(QCPAbstractPlottable *plottable, int dataIndex, QMouseEvent *event);
04610     void itemClick(QCPAbstractItem *item, QMouseEvent *event);
04611     void itemDoubleClick(QCPAbstractItem *item, QMouseEvent *event);
04612     void axisClick(QCPAxis *axis, QCPAxis::SelectablePart part, QMouseEvent *event);
04613     void axisDoubleClick(QCPAxis *axis, QCPAxis::SelectablePart part, QMouseEvent *event);
04614     void legendClick(QCPLegend *legend, QCPAbstractLegendItem *item, QMouseEvent *event);
04615     void legendDoubleClick(QCPLegend *legend, QCPAbstractLegendItem *item, QMouseEvent *event);
04616
04617     void selectionChangedByUser();
04618     void beforeReplot();
04619     void afterLayout();
04620     void afterReplot();
04621
04622 protected:
04623     // property members:
04624     QRect mViewport;
04625     double mBufferDevicePixelRatio;
04626     QCPLayoutGrid *mPlotLayout;
04627     bool mAutoAddPlottableToLegend;
04628     QList<QCPAbstractPlottable*> mPlottables;
04629     QList<QCPGraph*> mGraphs; // extra list of plottables also in mPlottables that are of type
QCPGraph
04630     QList<QCPAbstractItem*> mItems;
04631     QList<QCPLayer*> mLayers;
04632     QCP::AntialiasedElements mAntialiasedElements, mNotAntialiasedElements;
04633     QCP::Interactions mInteractions;
04634     int mSelectionTolerance;
04635     bool mNoAntialiasingOnDrag;
04636     QBrush mBackgroundBrush;
04637     QPixmap mBackgroundPixmap;
04638     QPixmap mScaledBackgroundPixmap;
04639     bool mBackgroundScaled;

```

```

04640     Qt::AspectRatioMode mBackgroundScaledMode;
04641     QCPLayer *mCurrentLayer;
04642     QCP::PlottingHints mPlottingHints;
04643     Qt::KeyboardModifier mMultiSelectModifier;
04644     QCP::SelectionRectMode mSelectionRectMode;
04645     QCPSelectionRect *mSelectionRect;
04646     bool mOpenGL;
04647
04648     // non-property members:
04649     QList<QSharedPointer<QCPAbstractPaintBuffer> > mPaintBuffers;
04650     QPoint mMousePressPos;
04651     bool mMouseHasMoved;
04652     QPointer<QCPLayerable> mMouseEventLayerable;
04653     QPointer<QCPLayerable> mMouseSignalLayerable;
04654     QVariant mMouseEventLayerableDetails;
04655     QVariant mMouseSignalLayerableDetails;
04656     bool mReplotting;
04657     bool mReplotQueued;
04658     double mReplotTime, mReplotTimeAverage;
04659     int mOpenGLMultisamples;
04660     QCP::AntialiasedElements mOpenGLAntialiasedElementsBackup;
04661     bool mOpenGLCacheLabelsBackup;
04662 #ifdef QCP_OPENGL_FBO
04663     QSharedPointer<QOpenGLContext> mGlcContext;
04664     QSharedPointer<QSurface> mGlcSurface;
04665     QSharedPointer<QOpenGLPaintDevice> mGlcPaintDevice;
04666 #endif
04667
04668     // reimplemented virtual methods:
04669     virtual QSize minimumSizeHint() const Q_DECL_OVERRIDE;
04670     virtual QSize sizeHint() const Q_DECL_OVERRIDE;
04671     virtual void paintEvent(QPaintEvent *event) Q_DECL_OVERRIDE;
04672     virtual void resizeEvent(QResizeEvent *event) Q_DECL_OVERRIDE;
04673     virtual void mouseDoubleClickEvent(QMouseEvent *event) Q_DECL_OVERRIDE;
04674     virtual void mousePressEvent(QMouseEvent *event) Q_DECL_OVERRIDE;
04675     virtual void mouseMoveEvent(QMouseEvent *event) Q_DECL_OVERRIDE;
04676     virtual void mouseReleaseEvent(QMouseEvent *event) Q_DECL_OVERRIDE;
04677     virtual void wheelEvent(QWheelEvent *event) Q_DECL_OVERRIDE;
04678
04679     // introduced virtual methods:
04680     virtual void draw(QCPPainter *painter);
04681     virtual void updateLayout();
04682     virtual void axisRemoved(QCPAxis *axis);
04683     virtual void legendRemoved(QCPLegend *legend);
04684     Q_SLOT virtual void processRectSelection(QRect rect, QMouseEvent *event);
04685     Q_SLOT virtual void processRectZoom(QRect rect, QMouseEvent *event);
04686     Q_SLOT virtual void processPointSelection(QMouseEvent *event);
04687
04688     // non-virtual methods:
04689     bool registerPlottable(QCPAbstractPlottable *plottable);
04690     bool registerGraph(QCPGraph *graph);
04691     bool registerItem(QCPAbstractItem *item);
04692     void updateLayerIndices() const;
04693     QCPLayerable *layerableAt(const QPointF &pos, bool onlySelectable, QVariant *selectionDetails =
04694     nullptr) const;
04695     QList<QCPLayerable> layerableListAt(const QPointF &pos, bool onlySelectable, QList<QVariant>
04696     *selectionDetails = nullptr) const;
04697     void drawBackground(QCPPainter *painter);
04698     void setupPaintBuffers();
04699     QCPAbstractPaintBuffer *createPaintBuffer();
04700     bool hasInvalidatedPaintBuffers();
04701     bool setupOpenGL();
04702     void freeOpenGL();
04703
04704     friend class QCPLegend;
04705     friend class QCPAxis;
04706     friend class QCPLayer;
04707     friend class QCPAxisRect;
04708     friend class QCPAbstractPlottable;
04709     friend class QCPGraph;
04710     friend class QCPAbstractItem;
04711 };
04712
04713 Q_DECLARE_METATYPE(QCustomPlot::LayerInsertMode)
04714 Q_DECLARE_METATYPE(QCustomPlot::RefreshPriority)
04715
04716 // implementation of template functions:
04717
04718 template<class PlottableType>
04719 PlottableType *QCustomPlot::plottableAt(const QPointF &pos, bool onlySelectable, int *dataIndex) const
04720 {
04721     PlottableType *resultPlottable = 0;
04722     QVariant resultDetails;
04723     double resultDistance = mSelectionTolerance; // only regard clicks with distances smaller than
04724     mSelectionTolerance as selections, so initialize with that value

```

```

04741     foreach(QCPAbstractPlottable *plottable, mPlottables) {
04742         PlottableType *currentPlottable = qobject_cast<PlottableType*>(plottable);
04743         if (!currentPlottable || (onlySelectable && !currentPlottable->selectable())) // we could have
also passed onlySelectable to the selectTest function, but checking here is faster, because we have
access to QCPAbstractPlottable::selectable
04744             continue;
04745         if (currentPlottable->clipRect().contains(pos.toPoint())) // only consider clicks where the
plottable is actually visible
04746         {
04747             QVariant details;
04748             double currentDistance = currentPlottable->selectTest(pos, false, dataIndex ? &details :
nullptr);
04749             if (currentDistance >= 0 && currentDistance < resultDistance) {
04750                 resultPlottable = currentPlottable;
04751                 resultDetails = details;
04752                 resultDistance = currentDistance;
04753             }
04754         }
04755     }
04756     if (resultPlottable && dataIndex) {
04757         QCPDataSelection sel = resultDetails.value<QCPDataSelection>();
04758         if (!sel.isEmpty())
04759             *dataIndex = sel.dataRange(0).begin();
04760     }
04761     return resultPlottable;
04762 }
04763
04764 template<class ItemType>
04765 ItemType *QCustomPlot::itemAt(const QPointF &pos, bool onlySelectable) const {
04766     ItemType *resultItem = 0;
04767     double resultDistance = mSelectionTolerance; // only regard clicks with distances smaller than
mSelectionTolerance as selections, so initialize with that value
04768     foreach(QCPAbstractItem *item, mItems) {
04769         ItemType *currentItem = qobject_cast<ItemType*>(item);
04770         if (!currentItem || (onlySelectable && !currentItem->selectable())) // we could have also
passed onlySelectable to the selectTest function, but checking here is faster, because we have access
to QCPAbstractItem::selectable
04771             continue;
04772         if (!currentItem->clipToAxisRect() || currentItem->clipRect().contains(pos.toPoint())) // only
consider clicks inside axis cliprect of the item if actually clipped to it
04773         {
04774             double currentDistance = currentItem->selectTest(pos, false);
04775             if (currentDistance >= 0 && currentDistance < resultDistance) {
04776                 resultItem = currentItem;
04777                 resultDistance = currentDistance;
04778             }
04779         }
04780     }
04781     return resultItem;
04782 }
04783
04784 /* end of 'src/core.h' */
04785
04786 /* including file 'src/plottableId.h' */
04787
04788 /* modified 2021-03-29T02:30:44, size 25638 */
04789
04790 class QCPPlottableInterface1D {
04791 public:
04792     virtual ~QCPPlottableInterface1D() = default;
04793     // introduced pure virtual methods:
04794     virtual int dataCount() const = 0;
04795     virtual double dataMainKey(int index) const = 0;
04796     virtual double dataSortKey(int index) const = 0;
04797     virtual double dataMainValue(int index) const = 0;
04798     virtual QCPRange dataValueRange(int index) const = 0;
04799     virtual QPointF dataPixelPosition(int index) const = 0;
04800     virtual bool sortKeyIsMainKey() const = 0;
04801     virtual QCPDataSelection selectTestRect(const QRectF &rect, bool onlySelectable) const = 0;
04802     virtual int findBegin(double sortKey, bool expandedRange = true) const = 0;
04803     virtual int findEnd(double sortKey, bool expandedRange = true) const = 0;
04804 };
04805
04806 template <class DataType>
04807 class QCPAbstractPlottable1D : public QCPAbstractPlottable, public QCPPlottableInterface1D // no
QCP_LIB_DECL, template class ends up in header (cpp included below)
04808 {
04809     // No Q_OBJECT macro due to template class
04810 public:
04811     QCPAbstractPlottable1D(QCPAxis *keyAxis, QCPAxis *valueAxis);

```

```

04832     virtual ~QCPAbstractPlottableID() Q_DECL_OVERRIDE;
04833
04834     // virtual methods of 1d plottable interface:
04835     virtual int dataCount() const Q_DECL_OVERRIDE;
04836     virtual double dataMainKey(int index) const Q_DECL_OVERRIDE;
04837     virtual double dataSortKey(int index) const Q_DECL_OVERRIDE;
04838     virtual double dataMainValue(int index) const Q_DECL_OVERRIDE;
04839     virtual QCPRange dataValueRange(int index) const Q_DECL_OVERRIDE;
04840     virtual QPointF dataPixelPosition(int index) const Q_DECL_OVERRIDE;
04841     virtual bool sortKeyIsMainKey() const Q_DECL_OVERRIDE;
04842     virtual QCPDataSelection selectTestRect(const QRectF &rect, bool onlySelectable) const
Q_DECL_OVERRIDE;
04843     virtual int findBegin(double sortKey, bool expandedRange = true) const Q_DECL_OVERRIDE;
04844     virtual int findEnd(double sortKey, bool expandedRange = true) const Q_DECL_OVERRIDE;
04845
04846     // reimplemented virtual methods:
04847     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
04848
04849     virtual QCPPlottableInterfaceID *interfaceID() Q_DECL_OVERRIDE {
04850         return this;
04851     }
04852
04853 protected:
04854     // property members:
04855     QSharedPointer<QCPDataContainer<DataType> > mDataContainer;
04856
04857     // helpers for subclasses:
04858     void getDataSegments(QList<QCPDataRange> &selectedSegments, QList<QCPDataRange>
&unselectedSegments) const;
04859     void drawPolyline(QCPPainter *painter, const QVector<QPointF> &lineData) const;
04860
04861 private:
04862     Q_DISABLE_COPY(QCPAbstractPlottableID)
04863 };
04864
04865
04866
04867
04868 // include implementation in header since it is a class template:
04869
04896 /* start documentation of pure virtual functions */
04897
05007 /* end documentation of pure virtual functions */
05008
05009
05013
05038 /* start documentation of inline functions */
05039
05048 /* end documentation of inline functions */
05049
05054 template <class DataType>
05055 QCPAbstractPlottableID<DataType>::QCPAbstractPlottableID(QCPAxis *keyAxis, QCPAxis *valueAxis) :
05056 QCPAbstractPlottable(keyAxis, valueAxis),
05057 mDataContainer(new QCPDataContainer<DataType>) {
05058 }
05059
05060 template <class DataType>
05061 QCPAbstractPlottableID<DataType>::~QCPAbstractPlottableID() {
05062 }
05063
05067 template <class DataType>
05068 int QCPAbstractPlottableID<DataType>::dataCount() const {
05069     return mDataContainer->size();
05070 }
05071
05075 template <class DataType>
05076 double QCPAbstractPlottableID<DataType>::dataMainKey(int index) const {
05077     if (index >= 0 && index < mDataContainer->size()) {
05078         return (mDataContainer->constBegin() + index)->mainKey();
05079     } else {
05080         qDebug() << Q_FUNC_INFO << "Index out of bounds" << index;
05081         return 0;
05082     }
05083 }
05084
05088 template <class DataType>
05089 double QCPAbstractPlottableID<DataType>::dataSortKey(int index) const {
05090     if (index >= 0 && index < mDataContainer->size()) {
05091         return (mDataContainer->constBegin() + index)->sortKey();
05092     } else {
05093         qDebug() << Q_FUNC_INFO << "Index out of bounds" << index;
05094         return 0;
05095     }
05096 }
05097
05101 template <class DataType>

```

```

05102 double QCPAbstractPlottable1D<DataType>::dataMainValue(int index) const {
05103     if (index >= 0 && index < mDataContainer->size()) {
05104         return (mDataContainer->constBegin() + index)->mainValue();
05105     } else {
05106         qDebug() << Q_FUNC_INFO << "Index out of bounds" << index;
05107         return 0;
05108     }
05109 }
05110
05114 template <class DataType>
05115 QCPRange QCPAbstractPlottable1D<DataType>::dataValueRange(int index) const {
05116     if (index >= 0 && index < mDataContainer->size()) {
05117         return (mDataContainer->constBegin() + index)->valueRange();
05118     } else {
05119         qDebug() << Q_FUNC_INFO << "Index out of bounds" << index;
05120         return QCPRange(0, 0);
05121     }
05122 }
05123
05127 template <class DataType>
05128 QPointF QCPAbstractPlottable1D<DataType>::dataPixelPosition(int index) const {
05129     if (index >= 0 && index < mDataContainer->size()) {
05130         const typename QCPDataContainer<DataType>::const_iterator it = mDataContainer->constBegin() +
05131         index;
05132         return coordsToPixels(it->mainKey(), it->mainValue());
05133     } else {
05134         qDebug() << Q_FUNC_INFO << "Index out of bounds" << index;
05135         return QPointF();
05136     }
05137 }
05141 template <class DataType>
05142 bool QCPAbstractPlottable1D<DataType>::sortKeyIsMainKey() const {
05143     return DataType::sortKeyIsMainKey();
05144 }
05145
05153 template <class DataType>
05154 QCPDataSelection QCPAbstractPlottable1D<DataType>::selectTestRect(const QRectF &rect, bool
05155     onlySelectable) const {
05156     QCPDataSelection result;
05157     if ((onlySelectable && mSelectable == QCP::stNone) || mDataContainer->isEmpty())
05158         return result;
05159     if (!mKeyAxis || !mValueAxis)
05160         return result;
05161     // convert rect given in pixels to ranges given in plot coordinates:
05162     double key1, value1, key2, value2;
05163     pixelsToCoords(rect.topLeft(), key1, value1);
05164     pixelsToCoords(rect.bottomRight(), key2, value2);
05165     QCPRange keyRange(key1, key2); // QCPRange normalizes internally so we don't have to care about
05166     whether key1 < key2
05167     QCPRange valueRange(value1, value2);
05168     typename QCPDataContainer<DataType>::const_iterator begin = mDataContainer->constBegin();
05169     typename QCPDataContainer<DataType>::const_iterator end = mDataContainer->constEnd();
05170     if (DataType::sortKeyIsMainKey()) // we can assume that data is sorted by main key, so can reduce
05171     the searched key interval:
05172     {
05173         begin = mDataContainer->findBegin(keyRange.lower, false);
05174         end = mDataContainer->findEnd(keyRange.upper, false);
05175     }
05176     if (begin == end)
05177         return result;
05178     int currentSegmentBegin = -1; // -1 means we're currently not in a segment that's contained in
05179     rect
05180     for (typename QCPDataContainer<DataType>::const_iterator it = begin; it != end; ++it) {
05181         if (currentSegmentBegin == -1) {
05182             if (valueRange.contains(it->mainValue()) && keyRange.contains(it->mainKey())) // start
05183             segment
05184             {
05185                 currentSegmentBegin = int(it - mDataContainer->constBegin());
05186             } else if (!valueRange.contains(it->mainValue()) || !keyRange.contains(it->mainKey())) //
05187             segment just ended
05188             {
05189                 result.addDataRange(QCPDataRange(currentSegmentBegin, int(it -
05190                 mDataContainer->constBegin())), false);
05191                 currentSegmentBegin = -1;
05192             }
05193         }
05194         // process potential last segment:
05195         if (currentSegmentBegin != -1)
05196             result.addDataRange(QCPDataRange(currentSegmentBegin, int(end -
05197             mDataContainer->constBegin())), false);
05198     }
05199     result.simplify();
05200     return result;
05201 }

```

```

05199 template <class DataType>
05200 int QCPAbstractPlottable1D<DataType>::findBegin(double sortKey, bool expandedRange) const {
05201     return int(mDataContainer->findBegin(sortKey, expandedRange) - mDataContainer->constBegin());
05202 }
05203
05207 template <class DataType>
05208 int QCPAbstractPlottable1D<DataType>::findEnd(double sortKey, bool expandedRange) const {
05209     return int(mDataContainer->findEnd(sortKey, expandedRange) - mDataContainer->constBegin());
05210 }
05211
05222 template <class DataType>
05223 double QCPAbstractPlottable1D<DataType>::selectTest(const QPointF &pos, bool onlySelectable, QVariant
*details) const {
05224     if ((onlySelectable && mSelectable == QCP::stNone) || mDataContainer->isEmpty())
05225         return -1;
05226     if (!mKeyAxis || !mValueAxis)
05227         return -1;
05228
05229     QCPDataSelection selectionResult;
05230     double minDistSqr = (std::numeric_limits<double>::max)();
05231     int minDistIndex = mDataContainer->size();
05232
05233     typename QCPDataContainer<DataType>::const_iterator begin = mDataContainer->constBegin();
05234     typename QCPDataContainer<DataType>::const_iterator end = mDataContainer->constEnd();
05235     if (DataType::sortKeyIsMainKey()) // we can assume that data is sorted by main key, so can reduce
the searched key interval:
05236     {
05237         // determine which key range comes into question, taking selection tolerance around pos into
account:
05238         double posKeyMin, posKeyMax, dummy;
05239         pixelsToCoords(pos - QPointF(mParentPlot->selectionTolerance(),
mParentPlot->selectionTolerance()), posKeyMin, dummy);
05240         pixelsToCoords(pos + QPointF(mParentPlot->selectionTolerance(),
mParentPlot->selectionTolerance()), posKeyMax, dummy);
05241         if (posKeyMin > posKeyMax)
05242             qSwap(posKeyMin, posKeyMax);
05243         begin = mDataContainer->findBegin(posKeyMin, true);
05244         end = mDataContainer->findEnd(posKeyMax, true);
05245     }
05246     if (begin == end)
05247         return -1;
05248     QCPRange keyRange(mKeyAxis->range());
05249     QCPRange valueRange(mValueAxis->range());
05250     for (typename QCPDataContainer<DataType>::const_iterator it = begin; it != end; ++it) {
05251         const double mainKey = it->mainKey();
05252         const double mainValue = it->mainValue();
05253         if (keyRange.contains(mainKey) && valueRange.contains(mainValue)) // make sure data point is
inside visible range, for speedup in cases where sort key isn't main key and we iterate over all
points
05254         {
05255             const double currentDistSqr = QCPVector2D(coordsToPixels(mainKey, mainValue) -
pos).lengthSquared();
05256             if (currentDistSqr < minDistSqr) {
05257                 minDistSqr = currentDistSqr;
05258                 minDistIndex = int(it - mDataContainer->constBegin());
05259             }
05260         }
05261     }
05262     if (minDistIndex != mDataContainer->size())
05263         selectionResult.addDataRange(QCPDataRange(minDistIndex, minDistIndex + 1), false);
05264
05265     selectionResult.simplify();
05266     if (details)
05267         details->setValue(selectionResult);
05268     return qSqrt(minDistSqr);
05269 }
05270
05281 template <class DataType>
05282 void QCPAbstractPlottable1D<DataType>::getDataSegments(QList<QCPDataRange> &selectedSegments,
QList<QCPDataRange> &unselectedSegments) const {
05283     selectedSegments.clear();
05284     unselectedSegments.clear();
05285     if (mSelectable == QCP::stWhole) // stWhole selection type draws the entire plottable with
selected style if mSelection isn't empty
05286     {
05287         if (selected())
05288             selectedSegments << QCPDataRange(0, dataCount());
05289         else
05290             unselectedSegments << QCPDataRange(0, dataCount());
05291     } else {
05292         QCPDataSelection sel(selection());
05293         sel.simplify();
05294         selectedSegments = sel.dataRanges();
05295         unselectedSegments = sel.inverse(QCPDataRange(0, dataCount())).dataRanges();
05296     }
05297 }
05298

```



```

05399
05400
05401     enum GradientPreset {
05402         gpGrayscale
05403         , gpHot
05404         , gpCold
05405         , gpNight
05406         , gpCandy
05407         , gpGeography
05408         , gpIon
05409         , gpThermal
05410         , gpPolar
05411         , gpSpectrum
05412         , gpJet
05413         , gpHues
05414     };
05415     Q_ENUMS(GradientPreset)
05416
05417     QCPColorGradient();
05418     QCPColorGradient(GradientPreset preset);
05419     bool operator==(const QCPColorGradient &other) const;
05420
05421     bool operator!=(const QCPColorGradient &other) const {
05422         return !(*this == other);
05423     }
05424
05425     // getters:
05426
05427     int levelCount() const {
05428         return mLevelCount;
05429     }
05430
05431     QMap<double, QColor> colorStops() const {
05432         return mColorStops;
05433     }
05434
05435     ColorInterpolation colorInterpolation() const {
05436         return mColorInterpolation;
05437     }
05438
05439     NanHandling nanHandling() const {
05440         return mNanHandling;
05441     }
05442
05443     QColor nanColor() const {
05444         return mNanColor;
05445     }
05446
05447     bool periodic() const {
05448         return mPeriodic;
05449     }
05450
05451     // setters:
05452     void setLevelCount(int n);
05453     void setColorStops(const QMap<double, QColor> &colorStops);
05454     void setColorStopAt(double position, const QColor &color);
05455     void setColorInterpolation(ColorInterpolation interpolation);
05456     void setNanHandling(NanHandling handling);
05457     void setNanColor(const QColor &color);
05458     void setPeriodic(bool enabled);
05459
05460     // non-property methods:
05461     void colorize(const double *data, const QCPRange &range, QRgb *scanLine, int n, int
dataIndexFactor = 1, bool logarithmic = false);
05462     void colorize(const double *data, const unsigned char *alpha, const QCPRange &range, QRgb
*scanLine, int n, int dataIndexFactor = 1, bool logarithmic = false);
05463     QRgb color(double position, const QCPRange &range, bool logarithmic = false);
05464     void loadPreset(GradientPreset preset);
05465     void clearColorStops();
05466     QCPColorGradient inverted() const;
05467
05468 protected:
05469     // property members:
05470     int mLevelCount;
05471     QMap<double, QColor> mColorStops;
05472     ColorInterpolation mColorInterpolation;
05473     NanHandling mNanHandling;
05474     QColor mNanColor;
05475     bool mPeriodic;
05476
05477     // non-property members:
05478     QVector<QRgb> mColorBuffer; // have colors premultiplied with alpha (for usage with
 QImage::Format_ARGB32_Premultiplied)
05479     bool mColorBufferInvalidated;
05480
05481     // non-virtual methods:
05482     bool stopsUseAlpha() const;

```

```

05486     void updateColorBuffer();
05487 };
05488
05489 Q_DECLARE_METATYPE(QCPCColorGradient::ColorInterpolation)
05490 Q_DECLARE_METATYPE(QCPCColorGradient::NanHandling)
05491 Q_DECLARE_METATYPE(QCPCColorGradient::GradientPreset)
05492
05493 /* end of 'src/colorgradient.h' */
05494
05495
05496 /* including file 'src/selectiondecorator-bracket.h' */
05497 /* modified 2021-03-29T02:30:44, size 4458 */
05498
05499 class QCP_LIB_DECL QCPSelectionDecoratorBracket : public QCPSelectionDecorator {
05500
05501     Q_GADGET
05502 public:
05503
05504     enum BracketStyle {
05505         bsSquareBracket
05506         , bsHalfEllipse
05507         , bsEllipse
05508         , bsPlus
05509         , bsUserStyle
05510     };
05511     Q_ENUMS(BracketStyle)
05512
05513     QCPSelectionDecoratorBracket();
05514     virtual ~QCPSelectionDecoratorBracket() Q_DECL_OVERRIDE;
05515
05516     // getters:
05517
05518     QPen bracketPen() const {
05519         return mBracketPen;
05520     }
05521
05522     QBrush bracketBrush() const {
05523         return mBracketBrush;
05524     }
05525
05526     int bracketWidth() const {
05527         return mBracketWidth;
05528     }
05529
05530     int bracketHeight() const {
05531         return mBracketHeight;
05532     }
05533
05534     BracketStyle bracketStyle() const {
05535         return mBracketStyle;
05536     }
05537
05538     bool tangentToData() const {
05539         return mTangentToData;
05540     }
05541
05542     int tangentAverage() const {
05543         return mTangentAverage;
05544     }
05545
05546     // setters:
05547     void setBracketPen(const QPen &pen);
05548     void setBracketBrush(const QBrush &brush);
05549     void setBracketWidth(int width);
05550     void setBracketHeight(int height);
05551     void setBracketStyle(BracketStyle style);
05552     void setTangentToData(bool enabled);
05553     void setTangentAverage(int pointCount);
05554
05555     // introduced virtual methods:
05556     virtual void drawBracket(QCPPainter *painter, int direction) const;
05557
05558     // virtual methods:
05559     virtual void drawDecoration(QCPPainter *painter, QCPDataSelection selection) Q_DECL_OVERRIDE;
05560
05561 protected:
05562     // property members:
05563     QPen mBracketPen;
05564     QBrush mBracketBrush;
05565     int mBracketWidth;
05566     int mBracketHeight;
05567     BracketStyle mBracketStyle;
05568     bool mTangentToData;
05569     int mTangentAverage;
05570
05571     // non-virtual methods:
05572     double getTangentAngle(const QCPPlottableInterface1D *interfaceld, int dataIndex, int direction)

```

```

    const;
05579     QPointF getPixelCoordinates(const QCPPlottableInterfaceID *interfaceId, int dataIndex) const;
05580
05581 };
05582
05583 Q_DECLARE_METATYPE(QCPSelectionDecoratorBracket::BracketStyle)
05584
05585 /* end of 'src/selectiondecorator-bracket.h' */
05586
05587
05588 /* including file 'src/layoutelements/layoutelement-axisrect.h' */
05589 /* modified 2021-03-29T02:30:44, size 7529 */
05590
05591 class QCP_LIB_DECL QCPAxisRect : public QCPLayoutElement {
05592     Q_OBJECT
05593     Q_PROPERTY(QPixmap background READ background WRITE setBackground)
05594     Q_PROPERTY(bool backgroundScaled READ backgroundScaled WRITE setBackgroundScaled)
05595     Q_PROPERTY(Qt::AspectRatioMode backgroundScaledMode READ backgroundScaledMode WRITE
setBackgroundScaledMode)
05597     Q_PROPERTY(Qt::Orientations rangeDrag READ rangeDrag WRITE setRangeDrag)
05598     Q_PROPERTY(Qt::Orientations rangeZoom READ rangeZoom WRITE setRangeZoom)
05600 public:
05601     explicit QCPAxisRect(QCustomPlot *parentPlot, bool setupDefaultAxes = true);
05602     virtual ~QCPAxisRect() Q_DECL_OVERRIDE;
05603
05604     // getters:
05605
05606     QPixmap background() const {
05607         return mBackgroundPixmap;
05608     }
05609
05610     QBrush backgroundBrush() const {
05611         return mBackgroundBrush;
05612     }
05613
05614     bool backgroundScaled() const {
05615         return mBackgroundScaled;
05616     }
05617
05618     Qt::AspectRatioMode backgroundScaledMode() const {
05619         return mBackgroundScaledMode;
05620     }
05621
05622     Qt::Orientations rangeDrag() const {
05623         return mRangeDrag;
05624     }
05625
05626     Qt::Orientations rangeZoom() const {
05627         return mRangeZoom;
05628     }
05629     QCPAxis *rangeDragAxis(Qt::Orientation orientation);
05630     QCPAxis *rangeZoomAxis(Qt::Orientation orientation);
05631     QList<QCPAxis*> rangeDragAxes(Qt::Orientation orientation);
05632     QList<QCPAxis*> rangeZoomAxes(Qt::Orientation orientation);
05633     double rangeZoomFactor(Qt::Orientation orientation);
05634
05635     // setters:
05636     void setBackground(const QPixmap &pm);
05637     void setBackground(const QPixmap &pm, bool scaled, Qt::AspectRatioMode mode =
Qt::KeepAspectRatioByExpanding);
05638     void setBackground(const QBrush &brush);
05639     void setBackgroundScaled(bool scaled);
05640     void setBackgroundScaledMode(Qt::AspectRatioMode mode);
05641     void setRangeDrag(Qt::Orientations orientations);
05642     void setRangeZoom(Qt::Orientations orientations);
05643     void setRangeDragAxes(QCPAxis *horizontal, QCPAxis *vertical);
05644     void setRangeDragAxes(QList<QCPAxis*> axes);
05645     void setRangeDragAxes(QList<QCPAxis*> horizontal, QList<QCPAxis*> vertical);
05646     void setRangeZoomAxes(QCPAxis *horizontal, QCPAxis *vertical);
05647     void setRangeZoomAxes(QList<QCPAxis*> axes);
05648     void setRangeZoomAxes(QList<QCPAxis*> horizontal, QList<QCPAxis*> vertical);
05649     void setRangeZoomFactor(double horizontalFactor, double verticalFactor);
05650     void setRangeZoomFactor(double factor);
05651
05652     // non-property methods:
05653     int axisCount(QCPAxis::AxisType type) const;
05654     QCPAxis *axis(QCPAxis::AxisType type, int index = 0) const;
05655     QList<QCPAxis*> axes(QCPAxis::AxisTypes types) const;
05656     QList<QCPAxis*> axes() const;
05657     QCPAxis *addAxis(QCPAxis::AxisType type, QCPAxis *axis = nullptr);
05658     QList<QCPAxis*> addAxes(QCPAxis::AxisTypes types);
05659     bool removeAxis(QCPAxis *axis);
05660
05661     QCPLayoutInset *insetLayout() const {
05662         return mInsetLayout;
05663     }
05664

```

```

05665 void zoom(const QRectF &pixelRect);
05666 void zoom(const QRectF &pixelRect, const QList<QCPAxis*> &affectedAxes);
05667 void setupFullAxesBox(bool connectRanges = false);
05668 QList<QCPAbstractPlottable*> plottables() const;
05669 QList<QCPGraph*> graphs() const;
05670 QList<QCPAbstractItem*> items() const;
05671
05672 // read-only interface imitating a QRect:
05673
05674 int left() const {
05675     return mRect.left();
05676 }
05677
05678 int right() const {
05679     return mRect.right();
05680 }
05681
05682 int top() const {
05683     return mRect.top();
05684 }
05685
05686 int bottom() const {
05687     return mRect.bottom();
05688 }
05689
05690 int width() const {
05691     return mRect.width();
05692 }
05693
05694 int height() const {
05695     return mRect.height();
05696 }
05697
05698 QSize size() const {
05699     return mRect.size();
05700 }
05701
05702 QPoint topLeft() const {
05703     return mRect.topLeft();
05704 }
05705
05706 QPoint topRight() const {
05707     return mRect.topRight();
05708 }
05709
05710 QPoint bottomLeft() const {
05711     return mRect.bottomLeft();
05712 }
05713
05714 QPoint bottomRight() const {
05715     return mRect.bottomRight();
05716 }
05717
05718 QPoint center() const {
05719     return mRect.center();
05720 }
05721
05722 // reimplemented virtual methods:
05723 virtual void update(UpdatePhase phase) Q_DECL_OVERRIDE;
05724 virtual QList<QCPLayoutElement*> elements(bool recursive) const Q_DECL_OVERRIDE;
05725
05726 protected:
05727     // property members:
05728     QBrush mBackgroundBrush;
05729     QPixmap mBackgroundPixmap;
05730     QPixmap mScaledBackgroundPixmap;
05731     bool mBackgroundScaled;
05732     Qt::AspectRatioMode mBackgroundScaledMode;
05733     QCPLayoutInset *mInsetLayout;
05734     Qt::Orientations mRangeDrag, mRangeZoom;
05735     QList<QPointer<QCPAxis> > mRangeDragHorzAxis, mRangeDragVertAxis;
05736     QList<QPointer<QCPAxis> > mRangeZoomHorzAxis, mRangeZoomVertAxis;
05737     double mRangeZoomFactorHorz, mRangeZoomFactorVert;
05738
05739     // non-property members:
05740     QList<QCPRange> mDragStartHorzRange, mDragStartVertRange;
05741     QCP::AntialiasedElements mAADragBackup, mNotAADragBackup;
05742     bool mDragging;
05743     QHash<QCPAxis::AxisType, QList<QCPAxis*> > mAxes;
05744
05745     // reimplemented virtual methods:
05746     virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
05747     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
05748     virtual int calculateAutoMargin(QCP::MarginSide side) Q_DECL_OVERRIDE;
05749     virtual void layoutChanged() Q_DECL_OVERRIDE;
05750     // events:
05751     virtual void mousePressEvent(QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE;

```

```

05752     virtual void mouseMoveEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
05753     virtual void mouseReleaseEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
05754     virtual void wheelEvent(QWheelEvent *event) Q_DECL_OVERRIDE;
05755
05756     // non-property methods:
05757     void drawBackground(QCPPainter *painter);
05758     void updateAxesOffset(QCPAxis::AxisType type);
05759
05760 private:
05761     Q_DISABLE_COPY(QCPAxisRect)
05762
05763     friend class QCustomPlot;
05764 };
05765
05766
05767 /* end of 'src/layoutelements/layoutelement-axisrect.h' */
05768
05769
05770 /* including file 'src/layoutelements/layoutelement-legend.h' */
05771
05772 /* modified 2021-03-29T02:30:44, size 10425 */
05773
05774 class QCP_LIB_DECL QCPAbstractLegendItem : public QCPLayoutElement {
05775     Q_OBJECT
05776     Q_PROPERTY(QCPLegend* parentLegend READ parentLegend)
05777     Q_PROPERTY(QFont font READ font WRITE setFont)
05778     Q_PROPERTY(QColor textColor READ textColor WRITE setTextColor)
05779     Q_PROPERTY(QFont selectedFont READ selectedFont WRITE setSelectedFont)
05780     Q_PROPERTY(QColor selectedTextColor READ selectedTextColor WRITE setSelectedTextColor)
05781     Q_PROPERTY(bool selectable READ selectable WRITE setSelectable NOTIFY selectableChanged)
05782     Q_PROPERTY(bool selected READ selected WRITE setSelected NOTIFY selectableChanged)
05783 public:
05784     explicit QCPAbstractLegendItem(QCPLegend *parent);
05785
05786     // getters:
05787
05788     QCPLegend *parentLegend() const {
05789         return mParentLegend;
05790     }
05791
05792     QFont font() const {
05793         return mFont;
05794     }
05795
05796     QColor textColor() const {
05797         return mTextColor;
05798     }
05799
05800     QFont selectedFont() const {
05801         return mSelectedFont;
05802     }
05803
05804     QColor selectedTextColor() const {
05805         return mSelectedTextColor;
05806     }
05807
05808     bool selectable() const {
05809         return mSelectable;
05810     }
05811
05812     bool selected() const {
05813         return mSelected;
05814     }
05815
05816     // setters:
05817     void setFont(const QFont &font);
05818     void setTextColor(const QColor &color);
05819     void setSelectedFont(const QFont &font);
05820     void setSelectedTextColor(const QColor &color);
05821     Q_SLOT void setSelectable(bool selectable);
05822     Q_SLOT void setSelected(bool selected);
05823
05824     // reimplemented virtual methods:
05825     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr) const Q_DECL_OVERRIDE;
05826
05827 signals:
05828     void selectionChanged(bool selected);
05829     void selectableChanged(bool selectable);
05830
05831 protected:
05832     // property members:
05833     QCPLegend *mParentLegend;
05834     QFont mFont;
05835     QColor mTextColor;
05836     QFont mSelectedFont;
05837     QColor mSelectedTextColor;

```

```

05840     bool mSelectable, mSelected;
05841
05842     // reimplemented virtual methods:
05843     virtual QCP::Interaction selectionCategory() const Q_DECL_OVERRIDE;
05844     virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
05845     virtual QRect clipRect() const Q_DECL_OVERRIDE;
05846     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE = 0;
05847     // events:
05848     virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
*selectionStateChanged) Q_DECL_OVERRIDE;
05849     virtual void deselectEvent(bool *selectionStateChanged) Q_DECL_OVERRIDE;
05850
05851 private:
05852     Q_DISABLE_COPY(QCPAbstractLegendItem)
05853
05854     friend class QCPLegend;
05855 };
05856
05857 class QCP_LIB_DECL QCPPlottableLegendItem : public QCPAbstractLegendItem {
05858     Q_OBJECT
05859 public:
05860     QCPPlottableLegendItem(QCPLegend *parent, QCPAbstractPlottable *plottable);
05861
05862     // getters:
05863
05864     QCPAbstractPlottable *plottable() {
05865         return mPlottable;
05866     }
05867
05868 protected:
05869     // property members:
05870     QCPAbstractPlottable *mPlottable;
05871
05872     // reimplemented virtual methods:
05873     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
05874     virtual QSize minimumOuterSizeHint() const Q_DECL_OVERRIDE;
05875
05876     // non-virtual methods:
05877     QPen getIconBorderPen() const;
05878     QColor getTextColor() const;
05879     QFont getFont() const;
05880 };
05881
05882 class QCP_LIB_DECL QCPLegend : public QCPLayoutGrid {
05883     Q_OBJECT
05884     Q_PROPERTY(QPen borderPen READ borderPen WRITE setBorderPen)
05885     Q_PROPERTY(QBrush brush READ brush WRITE setBrush)
05886     Q_PROPERTY(QFont font READ font WRITE setFont)
05887     Q_PROPERTY(QColor textColor READ textColor WRITE setTextColor)
05888     Q_PROPERTY(QSize iconSize READ iconSize WRITE setIconSize)
05889     Q_PROPERTY(int iconTextPadding READ iconTextPadding WRITE setIconTextPadding)
05890     Q_PROPERTY(QPen iconBorderPen READ iconBorderPen WRITE setIconBorderPen)
05891     Q_PROPERTY(SelectableParts selectableParts READ selectableParts WRITE setSelectableParts NOTIFY
selectionChanged)
05892     Q_PROPERTY(SelectableParts selectedParts READ selectedParts WRITE setSelectedParts NOTIFY
selectableChanged)
05893     Q_PROPERTY(QPen selectedBorderPen READ selectedBorderPen WRITE setSelectedBorderPen)
05894     Q_PROPERTY(QBrush selectedBrush READ selectedBrush WRITE setSelectedBrush)
05895     Q_PROPERTY(QFont selectedFont READ selectedFont WRITE setSelectedFont)
05896     Q_PROPERTY(QColor selectedTextColor READ selectedTextColor WRITE setSelectedTextColor)
05901 public:
05902     enum SelectablePart {
05903         spNone = 0x000
05904         , spLegendBox = 0x001
05905         , spItems = 0x002
05906     };
05907     Q_ENUMS(SelectablePart)
05908     Q_FLAGS(SelectableParts)
05909     Q_DECLARE_FLAGS(SelectableParts, SelectablePart)
05910
05911     explicit QCPLegend();
05912     virtual ~QCPLegend() Q_DECL_OVERRIDE;
05913
05914     // getters:
05915
05916     QPen borderPen() const {
05917         return mBorderPen;
05918     }
05919
05920     QBrush brush() const {
05921         return mBrush;
05922     }
05923
05924     QFont font() const {
05925         return mFont;
05926     }

```

```

05931     }
05932
05933     QColor textColor() const {
05934         return mTextColor;
05935     }
05936
05937     QSize iconSize() const {
05938         return mIconSize;
05939     }
05940
05941     int iconTextPadding() const {
05942         return mIconTextPadding;
05943     }
05944
05945     QPen iconBorderPen() const {
05946         return mIconBorderPen;
05947     }
05948
05949     SelectableParts selectableParts() const {
05950         return mSelectableParts;
05951     }
05952     SelectableParts selectedParts() const;
05953
05954     QPen selectedBorderPen() const {
05955         return mSelectedBorderPen;
05956     }
05957
05958     QPen selectedIconBorderPen() const {
05959         return mSelectedIconBorderPen;
05960     }
05961
05962     QBrush selectedBrush() const {
05963         return mSelectedBrush;
05964     }
05965
05966     QFont selectedFont() const {
05967         return mSelectedFont;
05968     }
05969
05970     QColor selectedTextColor() const {
05971         return mSelectedTextColor;
05972     }
05973
05974     // setters:
05975     void setBorderPen(const QPen &pen);
05976     void setBrush(const QBrush &brush);
05977     void setFont(const QFont &font);
05978     void setTextColor(const QColor &color);
05979     void setIconSize(const QSize &size);
05980     void setIconSize(int width, int height);
05981     void setIconTextPadding(int padding);
05982     void setIconBorderPen(const QPen &pen);
05983     Q_SLOT void setSelectableParts(const SelectableParts &selectableParts);
05984     Q_SLOT void setSelectedParts(const SelectableParts &selectedParts);
05985     void setSelectedBorderPen(const QPen &pen);
05986     void setSelectedIconBorderPen(const QPen &pen);
05987     void setSelectedBrush(const QBrush &brush);
05988     void setSelectedFont(const QFont &font);
05989     void setSelectedTextColor(const QColor &color);
05990
05991     // reimplemented virtual methods:
05992     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
05993
05994     // non-virtual methods:
05995     QCPAbstractLegendItem *item(int index) const;
05996     QCPPlottableLegendItem *itemWithPlottable(const QCPAbstractPlottable *plottable) const;
05997     int itemCount() const;
05998     bool hasItem(QCPAbstractLegendItem *item) const;
05999     bool hasItemWithPlottable(const QCPAbstractPlottable *plottable) const;
06000     bool addItem(QCPAbstractLegendItem *item);
06001     bool removeItem(int index);
06002     bool removeItem(QCPAbstractLegendItem *item);
06003     void clearItems();
06004     QList<QCPAbstractLegendItem*> selectedItems() const;
06005
06006 signals:
06007     void selectionChanged(QCPLegend::SelectableParts parts);
06008     void selectableChanged(QCPLegend::SelectableParts parts);
06009
06010 protected:
06011     // property members:
06012     QPen mBorderPen, mIconBorderPen;
06013     QBrush mBrush;
06014     QFont mFont;
06015     QColor mTextColor;
06016     QSize mIconSize;

```

```

06017     int mIconTextPadding;
06018     SelectableParts mSelectedParts, mSelectableParts;
06019     QPen mSelectedBorderPen, mSelectedIconBorderPen;
06020     QBrush mSelectedBrush;
06021     QFont mSelectedFont;
06022     QColor mSelectedTextColor;
06023
06024     // reimplemented virtual methods:
06025     virtual void parentPlotInitialized(QCustomPlot *parentPlot) Q_DECL_OVERRIDE;
06026     virtual QCP::Interaction selectionCategory() const Q_DECL_OVERRIDE;
06027     virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
06028     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
06029     // events:
06030     virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
*selectionStateChanged) Q_DECL_OVERRIDE;
06031     virtual void deselectEvent(bool *selectionStateChanged) Q_DECL_OVERRIDE;
06032
06033     // non-virtual methods:
06034     QPen getBorderPen() const;
06035     QBrush getBrush() const;
06036
06037 private:
06038     Q_DISABLE_COPY(QCPLegend)
06039
06040     friend class QCustomPlot;
06041     friend class QCPAbstractLegendItem;
06042 };
06043
06044 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPLegend::SelectableParts)
06045 Q_DECLARE_METATYPE(QCPLegend::SelectablePart)
06046
06047 /* end of 'src/layoutelements/layoutelement-legend.h' */
06048
06049
06050 /* including file 'src/layoutelements/layoutelement-textelement.h' */
06051 /* modified 2021-03-29T02:30:44, size 5359 */
06052
06053 class QCP_LIB_DECL QCPTextElement : public QCPLayoutElement {
06054     Q_OBJECT
06055     Q_PROPERTY(QString text READ text WRITE setText)
06056     Q_PROPERTY(QFont font READ font WRITE setFont)
06057     Q_PROPERTY(QColor textColor READ textColor WRITE setTextColor)
06058     Q_PROPERTY(QFont selectedFont READ selectedFont WRITE setSelectedFont)
06059     Q_PROPERTY(QColor selectedTextColor READ selectedTextColor WRITE setSelectedTextColor)
06060     Q_PROPERTY(bool selectable READ selectable WRITE setSelectable NOTIFY selectableChanged)
06061     Q_PROPERTY(bool selected READ selected WRITE setSelected NOTIFY selectionChanged)
06062 public:
06063     explicit QCPTextElement(QCustomPlot *parentPlot);
06064     QCPTextElement(QCustomPlot *parentPlot, const QString &text);
06065     QCPTextElement(QCustomPlot *parentPlot, const QString &text, double pointSize);
06066     QCPTextElement(QCustomPlot *parentPlot, const QString &text, const QString &fontFamily, double
pointSize);
06067     QCPTextElement(QCustomPlot *parentPlot, const QString &text, const QFont &font);
06068
06069     // getters:
06070
06071     QString text() const {
06072         return mText;
06073     }
06074
06075     int textFlags() const {
06076         return mTextFlags;
06077     }
06078
06079     QFont font() const {
06080         return mFont;
06081     }
06082
06083     QColor textColor() const {
06084         return mTextColor;
06085     }
06086
06087     QFont selectedFont() const {
06088         return mSelectedFont;
06089     }
06090
06091     QColor selectedTextColor() const {
06092         return mSelectedTextColor;
06093     }
06094
06095     bool selectable() const {
06096         return mSelectable;
06097     }
06098
06099     bool selected() const {
06100         return mSelected;
06101     }
06102
06103

```



```

06104
06105     // setters:
06106     void setText(const QString &text);
06107     void setTextFlags(int flags);
06108     void setFont(const QFont &font);
06109     void setTextColor(const QColor &color);
06110     void setSelectedFont(const QFont &font);
06111     void setSelectedTextColor(const QColor &color);
06112     Q_SLOT void setSelectable(bool selectable);
06113     Q_SLOT void setSelected(bool selected);
06114
06115     // reimplemented virtual methods:
06116     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
06117     virtual void mousePressEvent(QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE;
06118     virtual void mouseReleaseEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
06119     virtual void mouseDoubleClickEvent(QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE;
06120
06121 signals:
06122     void selectionChanged(bool selected);
06123     void selectableChanged(bool selectable);
06124     void clicked(QMouseEvent *event);
06125     void doubleClicked(QMouseEvent *event);
06126
06127 protected:
06128     // property members:
06129     QString mText;
06130     int mTextFlags;
06131     QFont mFont;
06132     QColor mTextColor;
06133     QFont mSelectedFont;
06134     QColor mSelectedTextColor;
06135     QRect mTextBoundingRect;
06136     bool mSelectable, mSelected;
06137
06138     // reimplemented virtual methods:
06139     virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
06140     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
06141     virtual QSize minimumOuterSizeHint() const Q_DECL_OVERRIDE;
06142     virtual QSize maximumOuterSizeHint() const Q_DECL_OVERRIDE;
06143     // events:
06144     virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
*selectionStateChanged) Q_DECL_OVERRIDE;
06145     virtual void deselectEvent(bool *selectionStateChanged) Q_DECL_OVERRIDE;
06146
06147     // non-virtual methods:
06148     QFont mainFont() const;
06149     QColor mainTextColor() const;
06150
06151 private:
06152     Q_DISABLE_COPY(QCPCoordinateSystem)
06153 };
06154
06155
06156
06157 /* end of 'src/layoutelements/layoutelement-textelement.h' */
06158
06159
06160 /* including file 'src/layoutelements/layoutelement-colorscale.h' */
06161
06162 /* modified 2021-03-29T02:30:44, size 5939 */
06163
06164
06165 class QCPCoordinateSystemPrivate : public QCPAxisRect {
06166     Q_OBJECT
06167 public:
06168     explicit QCPCoordinateSystemPrivate(QCPCoordinateSystem *parentColorScale);
06169 protected:
06170     QCPCoordinateSystem *mParentColorScale;
06171     QImage mGradientImage;
06172     bool mGradientImageInvalidated;
06173     // re-using some methods of QCPAxisRect to make them available to friend class QCPCoordinateSystem
06174     using QCPAxisRect::calculateAutoMargin;
06175     using QCPAxisRect::mousePressEvent;
06176     using QCPAxisRect::mouseMoveEvent;
06177     using QCPAxisRect::mouseReleaseEvent;
06178     using QCPAxisRect::wheelEvent;
06179     using QCPAxisRect::update;
06180     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
06181     void updateGradientImage();
06182     Q_SLOT void axisSelectionChanged(QCPAxis::SelectableParts selectedParts);
06183     Q_SLOT void axisSelectableChanged(QCPAxis::SelectableParts selectableParts);
06184     friend class QCPCoordinateSystem;
06185 };
06186
06187 class QCP_LIB_DECL QCPCoordinateSystem : public QCPLayoutElement {
06188     Q_OBJECT

```

```

06190     Q_PROPERTY(QCPAxis::AxisType type READ type WRITE setType)
06191     Q_PROPERTY(QCPRange dataRange READ dataRange WRITE setDataRange NOTIFY dataRangeChanged)
06192     Q_PROPERTY(QCPAxis::ScaleType dataScaleType READ dataScaleType WRITE setDataScaleType NOTIFY
dataScaleTypeChanged)
06193     Q_PROPERTY(QCPCColorGradient gradient READ gradient WRITE setGradient NOTIFY gradientChanged)
06194     Q_PROPERTY(QString label READ label WRITE setLabel)
06195     Q_PROPERTY(int barWidth READ barWidth WRITE setBarWidth)
06196     Q_PROPERTY(bool rangeDrag READ rangeDrag WRITE setRangeDrag)
06197     Q_PROPERTY(bool rangeZoom READ rangeZoom WRITE setRangeZoom)
06199 public:
06200     explicit QCPCColorScale(QCustomPlot *parentPlot);
06201     virtual ~QCPCColorScale() Q_DECL_OVERRIDE;
06202
06203     // getters:
06204
06205     QCPAxis *axis() const {
06206         return mColorAxis.data();
06207     }
06208
06209     QCPAxis::AxisType type() const {
06210         return mType;
06211     }
06212
06213     QCPRange dataRange() const {
06214         return mDataRange;
06215     }
06216
06217     QCPAxis::ScaleType dataScaleType() const {
06218         return mDataScaleType;
06219     }
06220
06221     QCPCColorGradient gradient() const {
06222         return mGradient;
06223     }
06224     QString label() const;
06225
06226     int barWidth() const {
06227         return mBarWidth;
06228     }
06229     bool rangeDrag() const;
06230     bool rangeZoom() const;
06231
06232     // setters:
06233     void setType(QCPAxis::AxisType type);
06234     Q_SLOT void setDataRange(const QCPRange &dataRange);
06235     Q_SLOT void setDataScaleType(QCPAxis::ScaleType scaleType);
06236     Q_SLOT void setGradient(const QCPCColorGradient &gradient);
06237     void setLabel(const QString &str);
06238     void setBarWidth(int width);
06239     void setRangeDrag(bool enabled);
06240     void setRangeZoom(bool enabled);
06241
06242     // non-property methods:
06243     QList<QCPCColorMap*> colorMaps() const;
06244     void rescaleDataRange(bool onlyVisibleMaps);
06245
06246     // reimplemented virtual methods:
06247     virtual void update(UpdatePhase phase) Q_DECL_OVERRIDE;
06248
06249 signals:
06250     void dataRangeChanged(const QCPRange &newRange);
06251     void dataScaleTypeChanged(QCPAxis::ScaleType scaleType);
06252     void gradientChanged(const QCPCColorGradient &newGradient);
06253
06254 protected:
06255     // property members:
06256     QCPAxis::AxisType mType;
06257     QCPRange mDataRange;
06258     QCPAxis::ScaleType mDataScaleType;
06259     QCPCColorGradient mGradient;
06260     int mBarWidth;
06261
06262     // non-property members:
06263     QPointer<QCPCColorScaleAxisRectPrivate> mAxisRect;
06264     QPointer<QCPAxis> mColorAxis;
06265
06266     // reimplemented virtual methods:
06267     virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
06268     // events:
06269     virtual void mousePressEvent(QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE;
06270     virtual void mouseMoveEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
06271     virtual void mouseReleaseEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
06272     virtual void wheelEvent(QWheelEvent *event) Q_DECL_OVERRIDE;
06273
06274 private:
06275     Q_DISABLE_COPY(QCPCColorScale)
06276

```

```

06277     friend class QCPColorScaleAxisRectPrivate;
06278 };
06279
06280
06281 /* end of 'src/layoutelements/layoutelement-colorscale.h' */
06282
06283
06284 /* including file 'src/plottables/plottable-graph.h' */
06285
06286 /* modified 2021-03-29T02:30:44, size 9316 */
06287
06288 class QCP_LIB_DECL QCPGraphData {
06289 public:
06290     QCPGraphData();
06291     QCPGraphData(double key, double value);
06292
06293     inline double sortKey() const {
06294         return key;
06295     }
06296
06297     inline static QCPGraphData fromSortKey(double sortKey) {
06298         return QCPGraphData(sortKey, 0);
06299     }
06300
06301     inline static bool sortKeyIsMainKey() {
06302         return true;
06303     }
06304
06305     inline double mainKey() const {
06306         return key;
06307     }
06308
06309     inline double mainValue() const {
06310         return value;
06311     }
06312
06313     inline QCPRange valueRange() const {
06314         return QCPRange(value, value);
06315     }
06316
06317     double key, value;
06318 };
06319 Q_DECLARE_TYPEINFO(QCPGraphData, Q_PRIMITIVE_TYPE);
06320
06321
06322 typedef QCPDataContainer<QCPGraphData> QCPGraphDataContainer;
06323
06324
06325 class QCP_LIB_DECL QCPGraph : public QCPAbstractPlottable1D<QCPGraphData> {
06326 public:
06327     Q_OBJECT
06328     Q_PROPERTY(LineStyle lineStyle READ lineStyle WRITE setLineStyle)
06329     Q_PROPERTY(QCPScatterStyle scatterStyle READ scatterStyle WRITE setScatterStyle)
06330     Q_PROPERTY(int scatterSkip READ scatterSkip WRITE setScatterSkip)
06331     Q_PROPERTY(QCPGraph* channelFillGraph READ channelFillGraph WRITE setChannelFillGraph)
06332     Q_PROPERTY(bool adaptiveSampling READ adaptiveSampling WRITE setAdaptiveSampling)
06333
06334     public:
06335         enum LineStyle {
06336             lsNone
06337             , lsLine
06338             , lsStepLeft
06339             , lsStepRight
06340             , lsStepCenter
06341             , lsImpulse
06342         };
06343         Q_ENUMS(LineStyle)
06344
06345         explicit QCPGraph(QCPAxis *keyAxis, QCPAxis *valueAxis);
06346         virtual ~QCPGraph() Q_DECL_OVERRIDE;
06347
06348         // getters:
06349
06350         QSharedPointer<QCPGraphDataContainer> data() const {
06351             return mDataContainer;
06352         }
06353
06354         LineStyle lineStyle() const {
06355             return mLineStyle;
06356         }
06357
06358         QCPScatterStyle scatterStyle() const {
06359             return mScatterStyle;
06360         }
06361
06362         int scatterSkip() const {
06363             return mScatterSkip;
06364         }
06365
06366
06367
06368
06369
06370
06371
06372
06373
06374
06375
06376
06377
06378
06379
06380

```

```

06381     QCPGraph *channelFillGraph() const {
06382         return mChannelFillGraph.data();
06383     }
06384
06385     bool adaptiveSampling() const {
06386         return mAdaptiveSampling;
06387     }
06388
06389     // setters:
06390     void setData(QSharedPointer<QCPGraphDataContainer> data);
06391     void setData(const QVector<double> &keys, const QVector<double> &values, bool alreadySorted =
false);
06392     void setLineStyle(LineStyle ls);
06393     void setScatterStyle(const QCPScatterStyle &style);
06394     void setScatterSkip(int skip);
06395     void setChannelFillGraph(QCPGraph *targetGraph);
06396     void setAdaptiveSampling(bool enabled);
06397
06398     // non-property methods:
06399     void addData(const QVector<double> &keys, const QVector<double> &values, bool alreadySorted =
false);
06400     void addData(double key, double value);
06401
06402     // reimplemented virtual methods:
06403     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
06404     virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const
Q_DECL_OVERRIDE;
06405     virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
QCPRange &inKeyRange = QCPRange()) const Q_DECL_OVERRIDE;
06406
06407 protected:
06408     // property members:
06409     LineStyle mLineStyle;
06410     QCPScatterStyle mScatterStyle;
06411     int mScatterSkip;
06412     QPointer<QCPGraph> mChannelFillGraph;
06413     bool mAdaptiveSampling;
06414
06415     // reimplemented virtual methods:
06416     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
06417     virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE;
06418
06419     // introduced virtual methods:
06420     virtual void drawFill(QCPPainter *painter, QVector<QPointF> *lines) const;
06421     virtual void drawScatterPlot(QCPPainter *painter, const QVector<QPointF> &scatters, const
QCPScatterStyle &style) const;
06422     virtual void drawLinePlot(QCPPainter *painter, const QVector<QPointF> &lines) const;
06423     virtual void drawImpulsePlot(QCPPainter *painter, const QVector<QPointF> &lines) const;
06424
06425     virtual void getOptimizedLineData(QVector<QCPGraphData> *lineData, const
QCPGraphDataContainer::const_iterator &begin, const QCPGraphDataContainer::const_iterator &end) const;
06426     virtual void getOptimizedScatterData(QVector<QCPGraphData> *scatterData,
QCPGraphDataContainer::const_iterator begin, QCPGraphDataContainer::const_iterator end) const;
06427
06428     // non-virtual methods:
06429     void getVisibleDataBounds(QCPGraphDataContainer::const_iterator &begin,
QCPGraphDataContainer::const_iterator &end, const QCPDataRange &rangeRestriction) const;
06430     void getLines(QVector<QPointF> *lines, const QCPDataRange &dataRange) const;
06431     void getScatters(QVector<QPointF> *scatters, const QCPDataRange &dataRange) const;
06432     QVector<QPointF> dataToLines(const QVector<QCPGraphData> &data) const;
06433     QVector<QPointF> dataToStepLeftLines(const QVector<QCPGraphData> &data) const;
06434     QVector<QPointF> dataToStepRightLines(const QVector<QCPGraphData> &data) const;
06435     QVector<QPointF> dataToStepCenterLines(const QVector<QCPGraphData> &data) const;
06436     QVector<QPointF> dataToImpulseLines(const QVector<QCPGraphData> &data) const;
06437     QVector<QCPDataRange> getNonNanSegments(const QVector<QPointF> *lineData, Qt::Orientation
keyOrientation) const;
06438     QVector<QPair<QCPDataRange, QCPDataRange>> getOverlappingSegments(QVector<QCPDataRange>
thisSegments, const QVector<QPointF> *thisData, QVector<QCPDataRange> otherSegments, const
QVector<QPointF> *otherData) const;
06439     bool segmentsIntersect(double aLower, double aUpper, double bLower, double bUpper, int
&bPrecedence) const;
06440     QPointF getFillBasePoint(QPointF matchingDataPoint) const;
06441     const QPolygonF getFillPolygon(const QVector<QPointF> *lineData, QCPDataRange segment) const;
06442     const QPolygonF getChannelFillPolygon(const QVector<QPointF> *thisData, QCPDataRange thisSegment,
const QVector<QPointF> *otherData, QCPDataRange otherSegment) const;
06443     int findIndexBelowX(const QVector<QPointF> *data, double x) const;
06444     int findIndexAboveX(const QVector<QPointF> *data, double x) const;
06445     int findIndexBelowY(const QVector<QPointF> *data, double y) const;
06446     int findIndexAboveY(const QVector<QPointF> *data, double y) const;
06447     double pointDistance(const QPointF &pixelPoint, QCPGraphDataContainer::const_iterator
&closestData) const;
06448
06449     friend class QCustomPlot;
06450     friend class QCPLegend;
06451 };
06452

```

```

06453 Q_DECLARE_METATYPE(QCPGraph::LineStyle)
06454
06455 /* end of 'src/plottables/plottable-graph.h' */
06456
06457
06458 /* including file 'src/plottables/plottable-curve.h' */
06459 /* modified 2021-03-29T02:30:44, size 7434 */
06460
06461 class QCP_LIB_DECL QCPCurveData {
06462 public:
06463     QCPCurveData();
06464     QCPCurveData(double t, double key, double value);
06465
06466     inline double sortKey() const {
06467         return t;
06468     }
06469
06470     inline static QCPCurveData fromSortKey(double sortKey) {
06471         return QCPCurveData(sortKey, 0, 0);
06472     }
06473
06474     inline static bool sortKeyIsMainKey() {
06475         return false;
06476     }
06477
06478     inline double mainKey() const {
06479         return key;
06480     }
06481
06482     inline double mainValue() const {
06483         return value;
06484     }
06485
06486     inline QCPRange valueRange() const {
06487         return QCPRange(value, value);
06488     }
06489
06490     double t, key, value;
06491 };
06492 Q_DECLARE_TYPEINFO(QCPCurveData, Q_PRIMITIVE_TYPE);
06493
06494
06505 typedef QCPDataContainer<QCPCurveData> QCPCurveDataContainer;
06506
06507 class QCP_LIB_DECL QCPCurve : public QCPAbstractPlottable1D<QCPCurveData> {
06508
06509     Q_OBJECT
06510     Q_PROPERTY(QCPScatterStyle scatterStyle READ scatterStyle WRITE setScatterStyle)
06511     Q_PROPERTY(int scatterSkip READ scatterSkip WRITE setScatterSkip)
06512     Q_PROPERTY(Linestyle lineStyle READ lineStyle WRITE setLineStyle)
06513 public:
06514     enum LineStyle {
06515         lsNone
06516         , lsLine
06517     };
06518     Q_ENUMS(Linestyle)
06519
06520     explicit QCPCurve(QCPAxis *keyAxis, QCPAxis *valueAxis);
06521     virtual ~QCPCurve() Q_DECL_OVERRIDE;
06522
06523     // getters:
06524
06525     QSharedPointer<QCPCurveDataContainer> data() const {
06526         return mDataContainer;
06527     }
06528
06529     QCPScatterStyle scatterStyle() const {
06530         return mScatterStyle;
06531     }
06532
06533     int scatterSkip() const {
06534         return mScatterSkip;
06535     }
06536
06537     LineStyle lineStyle() const {
06538         return mLineStyle;
06539     }
06540
06541     // setters:
06542     void setData(QSharedPointer<QCPCurveDataContainer> data);
06543     void setData(const QVector<double> &t, const QVector<double> &keys, const QVector<double> &values,
06544         bool alreadySorted = false);
06545     void setData(const QVector<double> &keys, const QVector<double> &values);
06546     void setScatterStyle(const QCPScatterStyle &style);
06547     void setScatterSkip(int skip);
06548     void setLineStyle(Linestyle style);
06549
06550

```

```

06556     // non-property methods:
06557     void addData(const QVector<double> &t, const QVector<double> &keys, const QVector<double> &values,
06558 bool alreadySorted = false);
06558     void addData(const QVector<double> &keys, const QVector<double> &values);
06559     void addData(double t, double key, double value);
06560     void addData(double key, double value);
06561
06562     // reimplemented virtual methods:
06563     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
06564 const Q_DECL_OVERRIDE;
06564     virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const
06565 Q_DECL_OVERRIDE;
06565     virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
06566 QCPRange &inKeyRange = QCPRange()) const Q_DECL_OVERRIDE;
06566
06567 protected:
06568     // property members:
06569     QCPScatterStyle mScatterStyle;
06570     int mScatterSkip;
06571     LineStyle mLineStyle;
06572
06573     // reimplemented virtual methods:
06574     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
06575     virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE;
06576
06577     // introduced virtual methods:
06578     virtual void drawCurveLine(QCPPainter *painter, const QVector<QPointF> &lines) const;
06579     virtual void drawScatterPlot(QCPPainter *painter, const QVector<QPointF> &points, const
06580 QCPScatterStyle &style) const;
06580
06581     // non-virtual methods:
06582     void getCurveLines(QVector<QPointF> *lines, const QCPDataRange &dataRange, double penWidth) const;
06583     void getScatters(QVector<QPointF> *scatters, const QCPDataRange &dataRange, double scatterWidth)
06584 const;
06584     int getRegion(double key, double value, double keyMin, double valueMax, double keyMax, double
06585 valueMin) const;
06585     QPointF getOptimizedPoint(int otherRegion, double otherKey, double otherValue, double key, double
06586 value, double keyMin, double valueMax, double keyMax, double valueMin) const;
06586     QVector<QPointF> getOptimizedCornerPoints(int prevRegion, int currentRegion, double prevKey,
06587 double prevValue, double key, double value, double keyMin, double valueMax, double keyMax, double
06588 valueMin) const;
06587     bool mayTraverse(int prevRegion, int currentRegion) const;
06588     bool getTraverse(double prevKey, double prevValue, double key, double value, double keyMin, double
06589 valueMax, double keyMax, double valueMin, QPointF &crossA, QPointF &crossB) const;
06589     void getTraverseCornerPoints(int prevRegion, int currentRegion, double keyMin, double valueMax,
06590 double keyMax, double valueMin, QVector<QPointF> &beforeTraverse, QVector<QPointF> &afterTraverse)
06591 const;
06590     double pointDistance(const QPointF &pixelPoint, QCPCurveDataContainer::const_iterator
06591 &closestData) const;
06591
06592     friend class QCustomPlot;
06593     friend class QCPLegend;
06594 };
06595
06596 Q_DECLARE_METATYPE(QCPCurve::LineStyle)
06597
06598 /* end of 'src/plottables/plottable-curve.h' */
06599
06600
06601 /* including file 'src/plottables/plottable-bars.h' */
06602 /* modified 2021-03-29T02:30:44, size 8955 */
06603
06604 class QCP_LIB_DECL QCPBarsGroup : public QObject {
06605
06606     Q_OBJECT
06607     Q_PROPERTY(SpacingType spacingType READ spacingType WRITE setSpacingType)
06608     Q_PROPERTY(double spacing READ spacing WRITE setSpacing)
06609 public:
06610     enum SpacingType {
06611         stAbsolute
06612         , stAxisRectRatio
06613         , stPlotCoords
06614     };
06615     Q_ENUMS(SpacingType)
06616
06617     explicit QCPBarsGroup(QCustomPlot *parentPlot);
06618     virtual ~QCPBarsGroup();
06619
06620     // getters:
06621     SpacingType spacingType() const {
06622         return mSpacingType;
06623     }
06624     double spacing() const {
06625         return mSpacing;
06626     }
06627
06628

```

```

06637
06638 // setters:
06639 void setSpacingType(SpacingType spacingType);
06640 void setSpacing(double spacing);
06641
06642 // non-virtual methods:
06643
06644 QList<QCPBars*> bars() const {
06645     return mBars;
06646 }
06647 QCPBars* bars(int index) const;
06648
06649 int size() const {
06650     return mBars.size();
06651 }
06652
06653 bool isEmpty() const {
06654     return mBars.isEmpty();
06655 }
06656 void clear();
06657
06658 bool contains(QCPBars *bars) const {
06659     return mBars.contains(bars);
06660 }
06661 void append(QCPBars *bars);
06662 void insert(int i, QCPBars *bars);
06663 void remove(QCPBars *bars);
06664
06665 protected:
06666 // non-property members:
06667 QCustomPlot *mParentPlot;
06668 SpacingType mSpacingType;
06669 double mSpacing;
06670 QList<QCPBars*> mBars;
06671
06672 // non-virtual methods:
06673 void registerBars(QCPBars *bars);
06674 void unregisterBars(QCPBars *bars);
06675
06676 // virtual methods:
06677 double keyPixelOffset(const QCPBars *bars, double keyCoord);
06678 double getPixelSpacing(const QCPBars *bars, double keyCoord);
06679
06680 private:
06681     Q_DISABLE_COPY(QCPBarsGroup)
06682
06683     friend class QCPBars;
06684 };
06685
06686 Q_DECLARE_METATYPE(QCPBarsGroup::SpacingType)
06687
06688
06689 class QCP_LIB_DECL QCPBarsData {
06690 public:
06691     QCPBarsData();
06692     QCPBarsData(double key, double value);
06693
06694     inline double sortKey() const {
06695         return key;
06696     }
06697
06698     inline static QCPBarsData fromSortKey(double sortKey) {
06699         return QCPBarsData(sortKey, 0);
06700     }
06701
06702     inline static bool sortKeyIsMainKey() {
06703         return true;
06704     }
06705
06706     inline double mainKey() const {
06707         return key;
06708     }
06709
06710     inline double mainValue() const {
06711         return value;
06712     }
06713
06714     inline QCPRange valueRange() const {
06715         return QCPRange(value, value);
06716     } // note that bar base value isn't held in each QCPBarsData and thus can't/shouldn't be returned
06717     here
06718     double key, value;
06719 };
06720 Q_DECLARE_TYPEINFO(QCPBarsData, Q_PRIMITIVE_TYPE);
06721
06722

```

```

06732 typedef QCPDataContainer<QCPBarsData> QCPBarsDataContainer;
06733
06734 class QCP_LIB_DECL QCPBars : public QCPAbstractPlottable1D<QCPBarsData> {
06735
06736     Q_OBJECT
06737     Q_PROPERTY(double width READ width WRITE setWidth)
06738     Q_PROPERTY(WidthType widthType READ widthType WRITE setWidthType)
06739     Q_PROPERTY(QCPBarsGroup* barsGroup READ barsGroup WRITE setBarsGroup)
06740     Q_PROPERTY(double baseValue READ baseValue WRITE setBaseValue)
06741     Q_PROPERTY(double stackingGap READ stackingGap WRITE setStackingGap)
06742     Q_PROPERTY(QCPBars* barBelow READ barBelow)
06743     Q_PROPERTY(QCPBars* barAbove READ barAbove)
06744 public:
06745     enum WidthType {
06746         wtAbsolute
06747         , wtAxisRectRatio
06748         , wtPlotCoords
06749     };
06750     Q_ENUMS(WidthType)
06751
06752     explicit QCPBars(QCPAxis *keyAxis, QCPAxis *valueAxis);
06753     virtual ~QCPBars() Q_DECL_OVERRIDE;
06754
06755     // getters:
06756
06757     double width() const {
06758         return mWidth;
06759     }
06760
06761     WidthType widthType() const {
06762         return mWidthType;
06763     }
06764
06765     QCPBarsGroup *barsGroup() const {
06766         return mBarsGroup;
06767     }
06768
06769     double baseValue() const {
06770         return mBaseValue;
06771     }
06772
06773     double stackingGap() const {
06774         return mStackingGap;
06775     }
06776
06777     QCPBars *barBelow() const {
06778         return mBarBelow.data();
06779     }
06780
06781     QCPBars *barAbove() const {
06782         return mBarAbove.data();
06783     }
06784
06785     QSharedPointer<QCPBarsDataContainer> data() const {
06786         return mDataContainer;
06787     }
06788
06789     // setters:
06790     void setData(QSharedPointer<QCPBarsDataContainer> data);
06791     void setData(const QVector<double> &keys, const QVector<double> &values, bool alreadySorted =
06792 false);
06793     void setWidth(double width);
06794     void setWidthType(WidthType widthType);
06795     void setBarsGroup(QCPBarsGroup *barsGroup);
06796     void setBaseValue(double baseValue);
06797     void setStackingGap(double pixels);
06798
06799     // non-property methods:
06800     void addData(const QVector<double> &keys, const QVector<double> &values, bool alreadySorted =
06801 false);
06802     void addData(double key, double value);
06803     void moveBelow(QCPBars *bars);
06804     void moveAbove(QCPBars *bars);
06805
06806     // reimplemented virtual methods:
06807     virtual QCPDataSelection selectTestRect(const QRectF &rect, bool onlySelectable) const
06808 Q_DECL_OVERRIDE;
06809     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
06810 const Q_DECL_OVERRIDE;
06811     virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const
06812 Q_DECL_OVERRIDE;
06813     virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
06814 QCPRange &inKeyRange = QCPRange()) const Q_DECL_OVERRIDE;
06815     virtual QPointF dataPixelPosition(int index) const Q_DECL_OVERRIDE;
06816
06817 protected:
06818     // property members:
06819

```



```

06821     double mWidth;
06822     WidthType mWidthType;
06823     QCPBarsGroup *mBarsGroup;
06824     double mBaseValue;
06825     double mStackingGap;
06826     QPointer<QCPBars> mBarBelow, mBarAbove;
06827
06828     // reimplemented virtual methods:
06829     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
06830     virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE;
06831
06832     // non-virtual methods:
06833     void getVisibleDataBounds(QCPBarsDataContainer::const_iterator &begin,
06834                               QCPBarsDataContainer::const_iterator &end) const;
06835     QRectF getBarRect(double key, double value) const;
06836     void getPixelWidth(double key, double &lower, double &upper) const;
06837     double getStackedBaseValue(double key, bool positive) const;
06838     static void connectBars(QCPBars* lower, QCPBars* upper);
06839
06839     friend class QCustomPlot;
06840     friend class QCPLegend;
06841     friend class QCPBarsGroup;
06842 };
06843
06844 Q_DECLARE_METATYPE(QCPBars::WidthType)
06845
06846 /* end of 'src/plottables/plottable-bars.h' */
06847
06848 /* including file 'src/plottables/plottable-statisticalbox.h' */
06849 /* modified 2021-03-29T02:30:44, size 7522 */
06850
06851 class QCP_LIB_DECL QCPStatisticalBoxData {
06852 public:
06853     QCPStatisticalBoxData();
06854     QCPStatisticalBoxData(double key, double minimum, double lowerQuartile, double median, double
06855         upperQuartile, double maximum, const QVector<double>& outliers = QVector<double>());
06856
06857     inline double sortKey() const {
06858         return key;
06859     }
06860
06861     inline static QCPStatisticalBoxData fromSortKey(double sortKey) {
06862         return QCPStatisticalBoxData(sortKey, 0, 0, 0, 0, 0);
06863     }
06864
06865     inline static bool sortKeyIsMainKey() {
06866         return true;
06867     }
06868
06869     inline double mainKey() const {
06870         return key;
06871     }
06872
06873     inline double mainValue() const {
06874         return median;
06875     }
06876
06877     inline QCPRange valueRange() const {
06878         QCPRange result(minimum, maximum);
06879         for (QVector<double>::const_iterator it = outliers.constBegin(); it != outliers.constEnd();
06880 ++it)
06881             result.expand(*it);
06882         return result;
06883     }
06884
06885     double key, minimum, lowerQuartile, median, upperQuartile, maximum;
06886     QVector<double> outliers;
06887 };
06888 Q_DECLARE_TYPEINFO(QCPStatisticalBoxData, Q_MOVABLE_TYPE);
06889
06890 typedef QCPDataContainer<QCPStatisticalBoxData> QCPStatisticalBoxDataContainer;
06891
06892 class QCP_LIB_DECL QCPStatisticalBox : public QCPAbstractPlottable1D<QCPStatisticalBoxData> {
06893     Q_OBJECT
06894     Q_PROPERTY(double width READ width WRITE setWidth)
06895     Q_PROPERTY(double whiskerWidth READ whiskerWidth WRITE setWhiskerWidth)
06896     Q_PROPERTY(QPen whiskerPen READ whiskerPen WRITE setWhiskerPen)
06897     Q_PROPERTY(QPen whiskerBarPen READ whiskerBarPen WRITE setWhiskerBarPen)
06898     Q_PROPERTY(bool whiskerAntialiased READ whiskerAntialiased WRITE setWhiskerAntialiased)
06899     Q_PROPERTY(QPen medianPen READ medianPen WRITE setMedianPen)
06900     Q_PROPERTY(QCPScatterStyle outlierStyle READ outlierStyle WRITE setOutlierStyle)
06901 public:
06902     explicit QCPStatisticalBox(QCPAxis *keyAxis, QCPAxis *valueAxis);
06903
06904     // getters:

```

```

06917
06918     QSharedPointer<QCPStatisticalBoxDataContainer> data() const {
06919         return mDataContainer;
06920     }
06921
06922     double width() const {
06923         return mWidth;
06924     }
06925
06926     double whiskerWidth() const {
06927         return mWhiskerWidth;
06928     }
06929
06930     QPen whiskerPen() const {
06931         return mWhiskerPen;
06932     }
06933
06934     QPen whiskerBarPen() const {
06935         return mWhiskerBarPen;
06936     }
06937
06938     bool whiskerAntialiased() const {
06939         return mWhiskerAntialiased;
06940     }
06941
06942     QPen medianPen() const {
06943         return mMedianPen;
06944     }
06945
06946     QCPScatterStyle outlierStyle() const {
06947         return mOutlierStyle;
06948     }
06949
06950     // setters:
06951     void setData(QSharedPointer<QCPStatisticalBoxDataContainer> data);
06952     void setData(const QVector<double> &keys, const QVector<double> &minimum, const QVector<double>
&lowerQuartile, const QVector<double> &median, const QVector<double> &upperQuartile, const
V QVector<double> &maximum, bool alreadySorted = false);
06953     void setWidth(double width);
06954     void setWhiskerWidth(double width);
06955     void setWhiskerPen(const QPen &pen);
06956     void setWhiskerBarPen(const QPen &pen);
06957     void setWhiskerAntialiased(bool enabled);
06958     void setMedianPen(const QPen &pen);
06959     void setOutlierStyle(const QCPScatterStyle &style);
06960
06961     // non-property methods:
06962     void addData(const QVector<double> &keys, const QVector<double> &minimum, const QVector<double>
&lowerQuartile, const QVector<double> &median, const QVector<double> &upperQuartile, const
V QVector<double> &maximum, bool alreadySorted = false);
06963     void addData(double key, double minimum, double lowerQuartile, double median, double
upperQuartile, double maximum, const QVector<double> &outliers = QVector<double>());
06964
06965     // reimplemented virtual methods:
06966     virtual QCPDataSelection selectTestRect(const QRectF &rect, bool onlySelectable) const
Q_DECL_OVERRIDE;
06967     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
06968     virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const
Q_DECL_OVERRIDE;
06969     virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
QCPRange &inKeyRange = QCPRange()) const Q_DECL_OVERRIDE;
06970
06971 protected:
06972     // property members:
06973     double mWidth;
06974     double mWhiskerWidth;
06975     QPen mWhiskerPen, mWhiskerBarPen;
06976     bool mWhiskerAntialiased;
06977     QPen mMedianPen;
06978     QCPScatterStyle mOutlierStyle;
06979
06980     // reimplemented virtual methods:
06981     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
06982     virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE;
06983
06984     // introduced virtual methods:
06985     virtual void drawStatisticalBox(QCPPainter *painter,
QCPStatisticalBoxDataContainer::const_iterator it, const QCPScatterStyle &outlierStyle) const;
06986
06987     // non-virtual methods:
06988     void getVisibleDataBounds(QCPStatisticalBoxDataContainer::const_iterator &begin,
QCPStatisticalBoxDataContainer::const_iterator &end) const;
06989     QRectF getQuartileBox(QCPStatisticalBoxDataContainer::const_iterator it) const;
06990     QVector<QLineF> getWhiskerBackboneLines(QCPStatisticalBoxDataContainer::const_iterator it) const;
06991     QVector<QLineF> getWhiskerBarLines(QCPStatisticalBoxDataContainer::const_iterator it) const;
06992

```

```

06993     friend class QCustomPlot;
06994     friend class QCPLegend;
06995 };
06996
06997 /* end of 'src/plottables/plottable-statisticalbox.h' */
06998
06999
07000 /* including file 'src/plottables/plottable-colormap.h' */
07001
07002 /* modified 2021-03-29T02:30:44, size 7092 */
07003
07004 class QCP_LIB_DECL QCPColorMapData {
07005 public:
07006     QCPColorMapData(int keySize, int valueSize, const QCPRange &keyRange, const QCPRange &valueRange);
07007     ~QCPColorMapData();
07008     QCPColorMapData(const QCPColorMapData &other);
07009     QCPColorMapData &operator=(const QCPColorMapData &other);
07010
07011     // getters:
07012
07013     int keySize() const {
07014         return mKeySize;
07015     }
07016
07017     int valueSize() const {
07018         return mValueSize;
07019     }
07020
07021     QCPRange keyRange() const {
07022         return mKeyRange;
07023     }
07024
07025     QCPRange valueRange() const {
07026         return mValueRange;
07027     }
07028
07029     QCPRange dataBounds() const {
07030         return mDataBounds;
07031     }
07032     double data(double key, double value);
07033     double cell(int keyIndex, int valueIndex);
07034     unsigned char alpha(int keyIndex, int valueIndex);
07035
07036     // setters:
07037     void setSize(int keySize, int valueSize);
07038     void setKeySize(int keySize);
07039     void setValueSize(int valueSize);
07040     void setRange(const QCPRange &keyRange, const QCPRange &valueRange);
07041     void setKeyRange(const QCPRange &keyRange);
07042     void setValueRange(const QCPRange &valueRange);
07043     void setData(double key, double value, double z);
07044     void setCell(int keyIndex, int valueIndex, double z);
07045     void setAlpha(int keyIndex, int valueIndex, unsigned char alpha);
07046
07047     // non-property methods:
07048     void recalculateDataBounds();
07049     void clear();
07050     void clearAlpha();
07051     void fill(double z);
07052     void fillAlpha(unsigned char alpha);
07053
07054     bool isEmpty() const {
07055         return mIsEmpty;
07056     }
07057     void coordToCell(double key, double value, int *keyIndex, int *valueIndex) const;
07058     void cellToCoord(int keyIndex, int valueIndex, double *key, double *value) const;
07059
07060 protected:
07061     // property members:
07062     int mKeySize, mValueSize;
07063     QCPRange mKeyRange, mValueRange;
07064     bool mIsEmpty;
07065
07066     // non-property members:
07067     double *mData;
07068     unsigned char *mAlpha;
07069     QCPRange mDataBounds;
07070     bool mDataModified;
07071
07072     bool createAlpha(bool initializeOpaque = true);
07073
07074     friend class QCPColorMap;
07075 };
07076
07077 class QCP_LIB_DECL QCPColorMap : public QCPAbstractPlottable {
07078     Q_OBJECT
07079     Q_PROPERTY(QCPRange dataRange READ dataRange WRITE setDataRange NOTIFY dataRangeChanged)

```

```

07081     Q_PROPERTY(QCPAxis::ScaleType dataScaleType READ dataScaleType WRITE setDataScaleType NOTIFY
dataScaleTypeChanged)
07082     Q_PROPERTY(QCPColorGradient gradient READ gradient WRITE setGradient NOTIFY gradientChanged)
07083     Q_PROPERTY(bool interpolate READ interpolate WRITE setInterpolate)
07084     Q_PROPERTY(bool tightBoundary READ tightBoundary WRITE setTightBoundary)
07085     Q_PROPERTY(QCPColorScale* colorScale READ colorScale WRITE setColorScale)
07086 public:
07087     explicit QCPColorMap(QCPAxis *keyAxis, QCPAxis *valueAxis);
07088     virtual ~QCPColorMap() Q_DECL_OVERRIDE;
07089
07090     // getters:
07091
07092     QCPColorMapData *data() const {
07093         return mMapData;
07094     }
07095
07096     QCPRange dataRange() const {
07097         return mDataRange;
07098     }
07099
07100     QCPAxis::ScaleType dataScaleType() const {
07101         return mDataScaleType;
07102     }
07103
07104     bool interpolate() const {
07105         return mInterpolate;
07106     }
07107
07108     bool tightBoundary() const {
07109         return mTightBoundary;
07110     }
07111
07112     QCPColorGradient gradient() const {
07113         return mGradient;
07114     }
07115
07116     QCPColorScale *colorScale() const {
07117         return mColorScale.data();
07118     }
07119
07120     // setters:
07121     void setData(QCPColorMapData *data, bool copy = false);
07122     Q_SLOT void setDataRange(const QCPRange &dataRange);
07123     Q_SLOT void setDataScaleType(QCPAxis::ScaleType scaleType);
07124     Q_SLOT void setGradient(const QCPColorGradient &gradient);
07125     void setInterpolate(bool enabled);
07126     void setTightBoundary(bool enabled);
07127     void setColorScale(QCPColorScale *colorScale);
07128
07129     // non-property methods:
07130     void rescaleDataRange(bool recalculateDataBounds = false);
07131     Q_SLOT void updateLegendIcon(Qt::TransformationMode transformMode = Qt::SmoothTransformation,
const QSize &thumbSize = QSize(32, 18));
07132
07133     // reimplemented virtual methods:
07134     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
07135     virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const
Q_DECL_OVERRIDE;
07136     virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
QCPRange &inKeyRange = QCPRange()) const Q_DECL_OVERRIDE;
07137
07138 signals:
07139     void dataRangeChanged(const QCPRange &newRange);
07140     void dataScaleTypeChanged(QCPAxis::ScaleType scaleType);
07141     void gradientChanged(const QCPColorGradient &newGradient);
07142
07143 protected:
07144     // property members:
07145     QCPRange mDataRange;
07146     QCPAxis::ScaleType mDataScaleType;
07147     QCPColorMapData *mMapData;
07148     QCPColorGradient mGradient;
07149     bool mInterpolate;
07150     bool mTightBoundary;
07151     QPointer<QCPColorScale> mColorScale;
07152
07153     // non-property members:
07154     QImage mMapImage, mUndersampledMapImage;
07155     QPixmap mLegendIcon;
07156     bool mMapImageInvalidated;
07157
07158     // introduced virtual methods:
07159     virtual void updateMapImage();
07160
07161     // reimplemented virtual methods:
07162     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;

```

```

07164     virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE;
07165
07166     friend class QCustomPlot;
07167     friend class QCPLegend;
07168 };
07169
07170 /* end of 'src/plottables/plottable-colormap.h' */
07171
07172
07173 /* including file 'src/plottables/plottable-financial.h' */
07174
07175 /* modified 2021-03-29T02:30:44, size 8644 */
07176
07177 class QCP_LIB_DECL QCPFinancialData {
07178 public:
07179     QCPFinancialData();
07180     QCPFinancialData(double key, double open, double high, double low, double close);
07181
07182     inline double sortKey() const {
07183         return key;
07184     }
07185
07186     inline static QCPFinancialData fromSortKey(double sortKey) {
07187         return QCPFinancialData(sortKey, 0, 0, 0, 0);
07188     }
07189
07190     inline static bool sortKeyIsMainKey() {
07191         return true;
07192     }
07193
07194     inline double mainKey() const {
07195         return key;
07196     }
07197
07198     inline double mainValue() const {
07199         return open;
07200     }
07201
07202     inline QCPRange valueRange() const {
07203         return QCPRange(low, high);
07204     } // open and close must lie between low and high, so we don't need to check them
07205
07206     double key, open, high, low, close;
07207 };
07208 Q_DECLARE_TYPEINFO(QCPFinancialData, Q_PRIMITIVE_TYPE);
07209
07210 typedef QCPDataContainer<QCPFinancialData> QCPFinancialDataContainer;
07211
07212 class QCP_LIB_DECL QCPFinancial : public QCPAbstractPlottable1D<QCPFinancialData> {
07213 public:
07214     Q_OBJECT
07215     Q_PROPERTY(ChartStyle chartStyle READ chartStyle WRITE setChartStyle)
07216     Q_PROPERTY(double width READ width WRITE setWidth)
07217     Q_PROPERTY(WidthType widthType READ widthType WRITE setWidthType)
07218     Q_PROPERTY(bool twoColored READ twoColored WRITE setTwoColored)
07219     Q_PROPERTY(QBrush brushPositive READ brushPositive WRITE setBrushPositive)
07220     Q_PROPERTY(QBrush brushNegative READ brushNegative WRITE setBrushNegative)
07221     Q_PROPERTY(QPen penPositive READ penPositive WRITE setPenPositive)
07222     Q_PROPERTY(QPen penNegative READ penNegative WRITE setPenNegative)
07223
07224 public:
07225     enum WidthType {
07226         wtAbsolute
07227         , wtAxisRectRatio
07228         , wtPlotCoords
07229     };
07230
07231     Q_ENUMS(WidthType)
07232
07233     enum ChartStyle {
07234         csOhlc
07235         , csCandlestick
07236     };
07237     Q_ENUMS(ChartStyle)
07238
07239     explicit QCPFinancial(QCPAxis *keyAxis, QCPAxis *valueAxis);
07240     virtual ~QCPFinancial() Q_DECL_OVERRIDE;
07241
07242     // getters:
07243     QSharedPointer<QCPFinancialDataContainer> data() const {
07244         return mDataContainer;
07245     }
07246
07247     ChartStyle chartStyle() const {
07248         return mChartStyle;
07249     }

```

```

07272     }
07273
07274     double width() const {
07275         return mWidth;
07276     }
07277
07278     WidthType widthType() const {
07279         return mWidthType;
07280     }
07281
07282     bool twoColored() const {
07283         return mTwoColored;
07284     }
07285
07286     QBrush brushPositive() const {
07287         return mBrushPositive;
07288     }
07289
07290     QBrush brushNegative() const {
07291         return mBrushNegative;
07292     }
07293
07294     QPen penPositive() const {
07295         return mPenPositive;
07296     }
07297
07298     QPen penNegative() const {
07299         return mPenNegative;
07300     }
07301
07302     // setters:
07303     void setData(QSharedPointer<QCPFinancialDataContainer> data);
07304     void setData(const QVector<double> &keys, const QVector<double> &open, const QVector<double>
&high, const QVector<double> &low, const QVector<double> &close, bool alreadySorted = false);
07305     void setChartStyle(ChartStyle style);
07306     void setWidth(double width);
07307     void setWidthType(WidthType widthType);
07308     void setTwoColored(bool twoColored);
07309     void setBrushPositive(const QBrush &brush);
07310     void setBrushNegative(const QBrush &brush);
07311     void setPenPositive(const QPen &pen);
07312     void setPenNegative(const QPen &pen);
07313
07314     // non-property methods:
07315     void addData(const QVector<double> &keys, const QVector<double> &open, const QVector<double>
&high, const QVector<double> &low, const QVector<double> &close, bool alreadySorted = false);
07316     void addData(double key, double open, double high, double low, double close);
07317
07318     // reimplemented virtual methods:
07319     virtual QCPDataSelection selectTestRect(const QRectF &rect, bool onlySelectable) const
Q_DECL_OVERRIDE;
07320     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
07321     virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const
Q_DECL_OVERRIDE;
07322     virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
QCPRange &inKeyRange = QCPRange()) const Q_DECL_OVERRIDE;
07323
07324     // static methods:
07325     static QCPFinancialDataContainer timeSeriesToOhlc(const QVector<double> &time, const
QVector<double> &value, double timeBinSize, double timeBinOffset = 0);
07326
07327 protected:
07328     // property members:
07329     ChartStyle mChartStyle;
07330     double mWidth;
07331     WidthType mWidthType;
07332     bool mTwoColored;
07333     QBrush mBrushPositive, mBrushNegative;
07334     QPen mPenPositive, mPenNegative;
07335
07336     // reimplemented virtual methods:
07337     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07338     virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE;
07339
07340     // non-virtual methods:
07341     void drawOhlcPlot(QCPPainter *painter, const QCPFinancialDataContainer::const_iterator &begin,
const QCPFinancialDataContainer::const_iterator &end, bool isSelected);
07342     void drawCandlestickPlot(QCPPainter *painter, const QCPFinancialDataContainer::const_iterator
&begin, const QCPFinancialDataContainer::const_iterator &end, bool isSelected);
07343     double getPixelWidth(double key, double keyPixel) const;
07344     double ohlcSelectTest(const QPointF &pos, const QCPFinancialDataContainer::const_iterator &begin,
const QCPFinancialDataContainer::const_iterator &end, QCPFinancialDataContainer::const_iterator
&closestDataPoint) const;
07345     double candlestickSelectTest(const QPointF &pos, const QCPFinancialDataContainer::const_iterator
&begin, const QCPFinancialDataContainer::const_iterator &end,
QCPFinancialDataContainer::const_iterator &closestDataPoint) const;

```

```

07346     void getVisibleDataBounds(QCPFinancialDataContainer::const_iterator &begin,
QCPFinancialDataContainer::const_iterator &end) const;
07347     QRectF selectionHitBox(QCPFinancialDataContainer::const_iterator it) const;
07348
07349     friend class QCustomPlot;
07350     friend class QCPLegend;
07351 };
07352
07353 Q_DECLARE_METATYPE(QCPFinancial::ChartStyle)
07354
07355 /* end of 'src/plottables/plottable-financial.h' */
07356
07357
07358 /* including file 'src/plottables/plottable-errorbar.h' */
07359 /* modified 2021-03-29T02:30:44, size 7749 */
07360
07361 class QCP_LIB_DECL QCPErrorsData {
07362 public:
07363     QCPErrorsData();
07364     explicit QCPErrorsData(double error);
07365     QCPErrorsData(double errorMinus, double errorPlus);
07366
07367     double errorMinus, errorPlus;
07368 };
07369 Q_DECLARE_TYPEINFO(QCPErrorsData, Q_PRIMITIVE_TYPE);
07370
07371 typedef QVector<QCPErrorsData> QCPErrorsDataContainer;
07372
07373 class QCP_LIB_DECL QCPErrors : public QCPAbstractPlottable, public QCPPlottableInterface1D {
07374 public:
07375     Q_OBJECT
07376     Q_PROPERTY(QSharedPointer<QCPErrorsDataContainer> data READ data WRITE setData)
07377     Q_PROPERTY(QCPAbstractPlottable* dataPlottable READ dataPlottable WRITE setDataPlottable)
07378     Q_PROPERTY(ErrorType errorType READ errorType WRITE setErrorType)
07379     Q_PROPERTY(double whiskerWidth READ whiskerWidth WRITE setWhiskerWidth)
07380     Q_PROPERTY(double symbolGap READ symbolGap WRITE setSymbolGap)
07381 public:
07382
07383     enum ErrorType {
07384         etKeyError
07385         , etValueError
07386     };
07387     Q_ENUMS(ErrorType)
07388
07389     explicit QCPErrors(QCPAxis *keyAxis, QCPAxis *valueAxis);
07390     virtual ~QCPErrors() Q_DECL_OVERRIDE;
07391     // getters:
07392
07393     QSharedPointer<QCPErrorsDataContainer> data() const {
07394         return mDataContainer;
07395     }
07396
07397     QCPAbstractPlottable *dataPlottable() const {
07398         return mDataPlottable.data();
07399     }
07400
07401     ErrorType errorType() const {
07402         return mErrorType;
07403     }
07404
07405     double whiskerWidth() const {
07406         return mWhiskerWidth;
07407     }
07408
07409     double symbolGap() const {
07410         return mSymbolGap;
07411     }
07412
07413     // setters:
07414     void setData(QSharedPointer<QCPErrorsDataContainer> data);
07415     void setData(const QVector<double> &error);
07416     void setData(const QVector<double> &errorMinus, const QVector<double> &errorPlus);
07417     void setDataPlottable(QCPAbstractPlottable* plottable);
07418     void setErrorType(ErrorType type);
07419     void setWhiskerWidth(double pixels);
07420     void setSymbolGap(double pixels);
07421
07422     // non-property methods:
07423     void addData(const QVector<double> &error);
07424     void addData(const QVector<double> &errorMinus, const QVector<double> &errorPlus);
07425     void addData(double error);
07426     void addData(double errorMinus, double errorPlus);
07427
07428     // virtual methods of 1d plottable interface:
07429     virtual int dataCount() const Q_DECL_OVERRIDE;
07430     virtual double dataMainKey(int index) const Q_DECL_OVERRIDE;

```

```

07455     virtual double dataSortKey(int index) const Q_DECL_OVERRIDE;
07456     virtual double dataMainValue(int index) const Q_DECL_OVERRIDE;
07457     virtual QCPRange dataValueRange(int index) const Q_DECL_OVERRIDE;
07458     virtual QPointF dataPixelPosition(int index) const Q_DECL_OVERRIDE;
07459     virtual bool sortKeyIsMainKey() const Q_DECL_OVERRIDE;
07460     virtual QCPDataSelection selectTestRect(const QRectF &rect, bool onlySelectable) const
Q_DECL_OVERRIDE;
07461     virtual int findBegin(double sortKey, bool expandedRange = true) const Q_DECL_OVERRIDE;
07462     virtual int findEnd(double sortKey, bool expandedRange = true) const Q_DECL_OVERRIDE;
07463
07464     // reimplemented virtual methods:
07465     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
07466
07467     virtual QCPPlottableInterface1D *interface1D() Q_DECL_OVERRIDE {
07468         return this;
07469     }
07470
07471 protected:
07472     // property members:
07473     QSharedPointer<QCPErrorsDataContainer> mDataContainer;
07474     QPointer<QCPAbstractPlottable> mDataPlottable;
07475     ErrorType mErrorType;
07476     double mWhiskerWidth;
07477     double mSymbolGap;
07478
07479     // reimplemented virtual methods:
07480     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07481     virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const Q_DECL_OVERRIDE;
07482     virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const
Q_DECL_OVERRIDE;
07483     virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
QCPRange &inKeyRange = QCPRange()) const Q_DECL_OVERRIDE;
07484
07485     // non-virtual methods:
07486     void getErrorBarLines(QCPErrorsDataContainer::const_iterator it, QVector<QLineF> &backbones,
QVector<QLineF> &whiskers) const;
07487     void getVisibleDataBounds(QCPErrorsDataContainer::const_iterator &begin,
QCPErrorsDataContainer::const_iterator &end, const QCPDataRange &rangeRestriction) const;
07488     double pointDistance(const QPointF &pixelPoint, QCPErrorsDataContainer::const_iterator
&closestData) const;
07489     // helpers:
07490     void getDataSegments(QList<QCPDataRange> &selectedSegments, QList<QCPDataRange>
&unselectedSegments) const;
07491     bool errorBarVisible(int index) const;
07492     bool rectIntersectsLine(const QRectF &pixelRect, const QLineF &line) const;
07493
07494     friend class QCustomPlot;
07495     friend class QCPLegend;
07496 };
07497
07498 /* end of 'src/plottables/plottable-errorbar.h' */
07499
07500
07501 /* including file 'src/items/item-straightline.h' */
07502
07503 /* modified 2021-03-29T02:30:44, size 3137 */
07504
07505 class QCP_LIB_DECL QCPItemStraightLine : public QCPAbstractItem {
07506     Q_OBJECT
07507     Q_PROPERTY(QPen pen READ pen WRITE setPen)
07508     Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
07509 public:
07510     explicit QCPItemStraightLine(QCustomPlot *parentPlot);
07511     virtual ~QCPItemStraightLine() Q_DECL_OVERRIDE;
07512
07513     // getters:
07514
07515     QPen pen() const {
07516         return mPen;
07517     }
07518
07519     QPen selectedPen() const {
07520         return mSelectedPen;
07521     }
07522
07523     // setters:
07524     void setPen(const QPen &pen);
07525     void setSelectedPen(const QPen &pen);
07526
07527     // reimplemented virtual methods:
07528     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
07529
07530     QCPItemPosition * const point1;
07531     QCPItemPosition * const point2;
07532
07533
07534

```



```

07535 protected:
07536     // property members:
07537     QPen mPen, mSelectedPen;
07538
07539     // reimplemented virtual methods:
07540     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07541
07542     // non-virtual methods:
07543     QLineF getRectClippedStraightLine(const QCPVector2D &base, const QCPVector2D &vec, const QRect
&rect) const;
07544     QPen mainPen() const;
07545 };
07546
07547 /* end of 'src/items/item-straightline.h' */
07548
07549
07550 /* including file 'src/items/item-line.h' */
07551
07552 /* modified 2021-03-29T02:30:44, size 3429 */
07553
07554 class QCP_LIB_DECL QCPItemLine : public QCPAbstractItem {
07555     Q_OBJECT
07556     Q_PROPERTY(QPen pen READ pen WRITE setPen)
07557     Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
07558     Q_PROPERTY(QCPLineEnding head READ head WRITE setHead)
07559     Q_PROPERTY(QCPLineEnding tail READ tail WRITE setTail)
07560 public:
07561     explicit QCPItemLine(QCustomPlot *parentPlot);
07562     virtual ~QCPItemLine() Q_DECL_OVERRIDE;
07563
07564     // getters:
07565     QPen pen() const {
07566         return mPen;
07567     }
07568     QPen selectedPen() const {
07569         return mSelectedPen;
07570     }
07571     QCPLineEnding head() const {
07572         return mHead;
07573     }
07574     QCPLineEnding tail() const {
07575         return mTail;
07576     }
07577
07578     // setters:
07579     void setPen(const QPen &pen);
07580     void setSelectedPen(const QPen &pen);
07581     void setHead(const QCPLineEnding &head);
07582     void setTail(const QCPLineEnding &tail);
07583
07584     // reimplemented virtual methods:
07585     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
07586
07587     QCPItemPosition * const start;
07588     QCPItemPosition * const end;
07589
07590 protected:
07591     // property members:
07592     QPen mPen, mSelectedPen;
07593     QCPLineEnding mHead, mTail;
07594
07595     // reimplemented virtual methods:
07596     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07597
07598     // non-virtual methods:
07599     QLineF getRectClippedLine(const QCPVector2D &start, const QCPVector2D &end, const QRect &rect)
const;
07600     QPen mainPen() const;
07601 };
07602
07603 /* end of 'src/items/item-line.h' */
07604
07605
07606 /* including file 'src/items/item-curve.h' */
07607
07608 /* modified 2021-03-29T02:30:44, size 3401 */
07609
07610 class QCP_LIB_DECL QCPItemCurve : public QCPAbstractItem {
07611     Q_OBJECT
07612     Q_PROPERTY(QPen pen READ pen WRITE setPen)
07613     Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
07614     Q_PROPERTY(QCPLineEnding head READ head WRITE setHead)

```

```

07622     Q_PROPERTY(QCPLLineEnding tail READ tail WRITE setTail)
07624 public:
07625     explicit QCPIItemCurve(QCustomPlot *parentPlot);
07626     virtual ~QCPIItemCurve() Q_DECL_OVERRIDE;
07627
07628     // getters:
07629
07630     QPen pen() const {
07631         return mPen;
07632     }
07633
07634     QPen selectedPen() const {
07635         return mSelectedPen;
07636     }
07637
07638     QCPLLineEnding head() const {
07639         return mHead;
07640     }
07641
07642     QCPLLineEnding tail() const {
07643         return mTail;
07644     }
07645
07646     // setters;
07647     void setPen(const QPen &pen);
07648     void setSelectedPen(const QPen &pen);
07649     void setHead(const QCPLLineEnding &head);
07650     void setTail(const QCPLLineEnding &tail);
07651
07652     // reimplemented virtual methods:
07653     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
07654     const Q_DECL_OVERRIDE;
07655
07656     QCPIItemPosition * const start;
07657     QCPIItemPosition * const startDir;
07658     QCPIItemPosition * const endDir;
07659     QCPIItemPosition * const end;
07660 protected:
07661     // property members:
07662     QPen mPen, mSelectedPen;
07663     QCPLLineEnding mHead, mTail;
07664
07665     // reimplemented virtual methods:
07666     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07667
07668     // non-virtual methods:
07669     QPen mainPen() const;
07670 };
07671
07672 /* end of 'src/items/item-curve.h' */
07673
07674
07675 /* including file 'src/items/item-rect.h' */
07676
07677 /* modified 2021-03-29T02:30:44, size 3710 */
07678
07679 class QCP_LIB_DECL QCPIItemRect : public QCPAbstractItem {
07680     Q_OBJECT
07681     Q_PROPERTY(QPen pen READ pen WRITE setPen)
07682     Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
07683     Q_PROPERTY(QBrush brush READ brush WRITE setBrush)
07684     Q_PROPERTY(QBrush selectedBrush READ selectedBrush WRITE setSelectedBrush)
07685 public:
07686     explicit QCPIItemRect(QCustomPlot *parentPlot);
07687     virtual ~QCPIItemRect() Q_DECL_OVERRIDE;
07688
07689     // getters:
07690
07691     QPen pen() const {
07692         return mPen;
07693     }
07694
07695     QPen selectedPen() const {
07696         return mSelectedPen;
07697     }
07698
07699     QBrush brush() const {
07700         return mBrush;
07701     }
07702
07703     QBrush selectedBrush() const {
07704         return mSelectedBrush;
07705     }
07706
07707     // setters;
07708     void setPen(const QPen &pen);

```

```

07711     void setSelectedPen(const QPen &pen);
07712     void setBrush(const QBrush &brush);
07713     void setSelectedBrush(const QBrush &brush);
07714
07715     // reimplemented virtual methods:
07716     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
const Q_DECL_OVERRIDE;
07717
07718     QCPItemPosition * const topLeft;
07719     QCPItemPosition * const bottomRight;
07720     QCPItemAnchor * const top;
07721     QCPItemAnchor * const topRight;
07722     QCPItemAnchor * const right;
07723     QCPItemAnchor * const bottom;
07724     QCPItemAnchor * const bottomLeft;
07725     QCPItemAnchor * const left;
07726
07727 protected:
07728
07729     enum AnchorIndex {
07730         aiTop, aiTopRight, aiRight, aiBottom, aiBottomLeft, aiLeft
07731     };
07732
07733     // property members:
07734     QPen mPen, mSelectedPen;
07735     QBrush mBrush, mSelectedBrush;
07736
07737     // reimplemented virtual methods:
07738     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07739     virtual QPointF anchorPixelPosition(int anchorId) const Q_DECL_OVERRIDE;
07740
07741     // non-virtual methods:
07742     QPen mainPen() const;
07743     QBrush mainBrush() const;
07744 };
07745
07746 /* end of 'src/items/item-rect.h' */
07747
07748
07749 /* including file 'src/items/item-text.h' */
07750
07751 /* modified 2021-03-29T02:30:44, size 5576 */
07752
07753 class QCP_LIB_DECL QCPItemText : public QCPAbstractItem {
07754     Q_OBJECT
07755     Q_PROPERTY(QColor color READ color WRITE setColor)
07756     Q_PROPERTY(QColor selectedColor READ selectedColor WRITE setSelectedColor)
07757     Q_PROPERTY(QPen pen READ pen WRITE setPen)
07758     Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
07759     Q_PROPERTY(QBrush brush READ brush WRITE setBrush)
07760     Q_PROPERTY(QBrush selectedBrush READ selectedBrush WRITE setSelectedBrush)
07761     Q_PROPERTY(QFont font READ font WRITE setFont)
07762     Q_PROPERTY(QFont selectedFont READ selectedFont WRITE setSelectedFont)
07763     Q_PROPERTY(QString text READ text WRITE setText)
07764     Q_PROPERTY(Qt::Alignment positionAlignment READ positionAlignment WRITE setPositionAlignment)
07765     Q_PROPERTY(Qt::Alignment textAlignment READ textAlignment WRITE setTextAlignment)
07766     Q_PROPERTY(double rotation READ rotation WRITE setRotation)
07767     Q_PROPERTY(QMargins padding READ padding WRITE setPadding)
07768 public:
07769     explicit QCPItemText(QCustomPlot *parentPlot);
07770     virtual ~QCPItemText() Q_DECL_OVERRIDE;
07771
07772     // getters:
07773
07774     QColor color() const {
07775         return mColor;
07776     }
07777
07778     QColor selectedColor() const {
07779         return mSelectedColor;
07780     }
07781
07782     QPen pen() const {
07783         return mPen;
07784     }
07785
07786     QPen selectedPen() const {
07787         return mSelectedPen;
07788     }
07789
07790     QBrush brush() const {
07791         return mBrush;
07792     }
07793
07794     QBrush selectedBrush() const {
07795         return mSelectedBrush;
07796     }
07797
07798     QFont font() const {
07799         return mFont;
07800     }
07801
07802     QFont selectedFont() const {
07803         return mSelectedFont;
07804     }
07805
07806     QString text() const {
07807         return mText;
07808     }
07809
07810     Qt::Alignment positionAlignment() const {
07811         return mPositionAlignment;
07812     }
07813
07814     Qt::Alignment textAlignment() const {
07815         return mTextAlignment;
07816     }
07817
07818     double rotation() const {
07819         return mRotation;
07820     }
07821
07822     QMargins padding() const {
07823         return mPadding;
07824     }
07825
07826     void setColor(const QColor &color) {
07827         mColor = color;
07828     }
07829
07830     void setSelectedColor(const QColor &color) {
07831         mSelectedColor = color;
07832     }
07833
07834     void setPen(const QPen &pen) {
07835         mPen = pen;
07836     }
07837
07838     void setSelectedPen(const QPen &pen) {
07839         mSelectedPen = pen;
07840     }
07841
07842     void setBrush(const QBrush &brush) {
07843         mBrush = brush;
07844     }
07845
07846     void setSelectedBrush(const QBrush &brush) {
07847         mSelectedBrush = brush;
07848     }
07849
07850     void setFont(const QFont &font) {
07851         mFont = font;
07852     }
07853
07854     void setSelectedFont(const QFont &font) {
07855         mSelectedFont = font;
07856     }
07857
07858     void setText(const QString &text) {
07859         mText = text;
07860     }
07861
07862     void setPositionAlignment(Qt::Alignment positionAlignment) {
07863         mPositionAlignment = positionAlignment;
07864     }
07865
07866     void setTextAlignment(Qt::Alignment textAlignment) {
07867         mTextAlignment = textAlignment;
07868     }
07869
07870     void setRotation(double rotation) {
07871         mRotation = rotation;
07872     }
07873
07874     void setPadding(const QMargins &padding) {
07875         mPadding = padding;
07876     }
07877
07878     void setTextColor(const QColor &color) {
07879         mColor = color;
07880     }
07881
07882     void setSelectedTextColor(const QColor &color) {
07883         mSelectedColor = color;
07884     }
07885
07886     void setPenColor(const QColor &color) {
07887         mPen.setColor(color);
07888     }
07889
07890     void setSelectedPenColor(const QColor &color) {
07891         mSelectedPen.setColor(color);
07892     }
07893
07894     void setPenWidth(int width) {
07895         mPen.setWidth(width);
07896     }
07897
07898     void setSelectedPenWidth(int width) {
07899         mSelectedPen.setWidth(width);
07900     }
07901
07902     void setPenStyle(Qt::PenStyle style) {
07903         mPen.setStyle(style);
07904     }
07905
07906     void setSelectedPenStyle(Qt::PenStyle style) {
07907         mSelectedPen.setStyle(style);
07908     }
07909
07910     void setBrushColor(const QColor &color) {
07911         mBrush.setColor(color);
07912     }
07913
07914     void setSelectedBrushColor(const QColor &color) {
07915         mSelectedBrush.setColor(color);
07916     }
07917
07918     void setBrushStyle(Qt::BrushStyle style) {
07919         mBrush.setStyle(style);
07920     }
07921
07922     void setSelectedBrushStyle(Qt::BrushStyle style) {
07923         mSelectedBrush.setStyle(style);
07924     }
07925
07926     void setFontPointSize(int pointSize) {
07927         mFont.setPointSize(pointSize);
07928     }
07929
07930     void setSelectedFontPointSize(int pointSize) {
07931         mSelectedFont.setPointSize(pointSize);
07932     }
07933
07934     void setTextPosition(const QPointF &textPosition) {
07935         mTextPosition = textPosition;
07936     }
07937
07938     void setRotationAngle(double angle) {
07939         mRotation = angle;
07940     }
07941
07942     void setPaddingLeft(int left) {
07943         mPadding.setLeft(left);
07944     }
07945
07946     void setPaddingRight(int right) {
07947         mPadding.setRight(right);
07948     }
07949
07950     void setPaddingTop(int top) {
07951         mPadding.setTop(top);
07952     }
07953
07954     void setPaddingBottom(int bottom) {
07955         mPadding.setBottom(bottom);
07956     }
07957
07958     void setTextColorAlpha(float alpha) {
07959         mColor.setAlpha(alpha);
07960     }
07961
07962     void setSelectedTextColorAlpha(float alpha) {
07963         mSelectedColor.setAlpha(alpha);
07964     }
07965
07966     void setPenColorAlpha(float alpha) {
07967         mPen.setColorAlpha(alpha);
07968     }
07969
07970     void setSelectedPenColorAlpha(float alpha) {
07971         mSelectedPen.setColorAlpha(alpha);
07972     }
07973
07974     void setBrushColorAlpha(float alpha) {
07975         mBrush.setColorAlpha(alpha);
07976     }
07977
07978     void setSelectedBrushColorAlpha(float alpha) {
07979         mSelectedBrush.setColorAlpha(alpha);
07980     }
07981
07982     void setFontPointSizeF(float pointSizeF) {
07983         mFont.setPointSizeF(pointSizeF);
07984     }
07985
07986     void setSelectedFontPointSizeF(float pointSizeF) {
07987         mSelectedFont.setPointSizeF(pointSizeF);
07988     }
07989
07990     void setTextPositionF(const QPointF &textPositionF) {
07991         mTextPosition = textPositionF;
07992     }
07993
07994     void setRotationAngleF(float angleF) {
07995         mRotation = angleF;
07996     }
07997
07998     void setPaddingLeftF(float leftF) {
07999         mPadding.setLeft(leftF);
08000     }
08001
08002     void setPaddingRightF(float rightF) {
08003         mPadding.setRight(rightF);
08004     }
08005
08006     void setPaddingTopF(float topF) {
08007         mPadding.setTop(topF);
08008     }
08009
08010     void setPaddingBottomF(float bottomF) {
08011         mPadding.setBottom(bottomF);
08012     }
08013
08014     void setTextColorF(float r, float g, float b, float a) {
08015         mColor.setRgbF(r, g, b, a);
08016     }
08017
08018     void setSelectedTextColorF(float r, float g, float b, float a) {
08019         mSelectedColor.setRgbF(r, g, b, a);
08020     }
08021
08022     void setPenColorF(float r, float g, float b, float a) {
08023         mPen.setColorF(r, g, b, a);
08024     }
08025
08026     void setSelectedPenColorF(float r, float g, float b, float a) {
08027         mSelectedPen.setColorF(r, g, b, a);
08028     }
08029
08030     void setBrushColorF(float r, float g, float b, float a) {
08031         mBrush.setColorF(r, g, b, a);
08032     }
08033
08034     void setSelectedBrushColorF(float r, float g, float b, float a) {
08035         mSelectedBrush.setColorF(r, g, b, a);
08036     }
08037
08038     void setFontPointSizeF(float pointSizeF) {
08039         mFont.setPointSizeF(pointSizeF);
08040     }
08041
08042     void setSelectedFontPointSizeF(float pointSizeF) {
08043         mSelectedFont.setPointSizeF(pointSizeF);
08044     }
08045
08046     void setTextPositionF(const QPointF &textPositionF) {
08047         mTextPosition = textPositionF;
08048     }
08049
08050     void setRotationAngleF(float angleF) {
08051         mRotation = angleF;
08052     }
08053
08054     void setPaddingLeftF(float leftF) {
08055         mPadding.setLeft(leftF);
08056     }
08057
08058     void setPaddingRightF(float rightF) {
08059         mPadding.setRight(rightF);
08060     }
08061
08062     void setPaddingTopF(float topF) {
08063         mPadding.setTop(topF);
08064     }
08065
08066     void setPaddingBottomF(float bottomF) {
08067         mPadding.setBottom(bottomF);
08068     }
08069
08070     void setTextColorF(float r, float g, float b, float a) {
08071         mColor.setRgbF(r, g, b, a);
08072     }
08073
08074     void setSelectedTextColorF(float r, float g, float b, float a) {
08075         mSelectedColor.setRgbF(r, g, b, a);
08076     }
08077
08078     void setPenColorF(float r, float g, float b, float a) {
08079         mPen.setColorF(r, g, b, a);
08080     }
08081
08082     void setSelectedPenColorF(float r, float g, float b, float a) {
08083         mSelectedPen.setColorF(r, g, b, a);
08084     }
08085
08086     void setBrushColorF(float r, float g, float b, float a) {
08087         mBrush.setColorF(r, g, b, a);
08088     }
08089
08090     void setSelectedBrushColorF(float r, float g, float b, float a) {
08091         mSelectedBrush.setColorF(r, g, b, a);
08092     }
08093
08094     void setFontPointSizeF(float pointSizeF) {
08095         mFont.setPointSizeF(pointSizeF);
08096     }
08097
08098     void setSelectedFontPointSizeF(float pointSizeF) {
08099         mSelectedFont.setPointSizeF(pointSizeF);
08100     }
08101
08102     void setTextPositionF(const QPointF &textPositionF) {
08103         mTextPosition = textPositionF;
08104     }
08105
08106     void setRotationAngleF(float angleF) {
08107         mRotation = angleF;
08108     }
08109
08110     void setPaddingLeftF(float leftF) {
08111         mPadding.setLeft(leftF);
08112     }
08113
08114     void setPaddingRightF(float rightF) {
08115         mPadding.setRight(rightF);
08116     }
08117
08118     void setPaddingTopF(float topF) {
08119         mPadding.setTop(topF);
08120     }
08121
08122     void setPaddingBottomF(float bottomF) {
08123         mPadding.setBottom(bottomF);
08124     }
08125
08126     void setTextColorF(float r, float g, float b, float a) {
08127         mColor.setRgbF(r, g, b, a);
08128     }
08129
08130     void setSelectedTextColorF(float r, float g, float b, float a) {
08131         mSelectedColor.setRgbF(r, g, b, a);
08132     }
08133
08134     void setPenColorF(float r, float g, float b, float a) {
08135         mPen.setColorF(r, g, b, a);
08136     }
08137
08138     void setSelectedPenColorF(float r, float g, float b, float a) {
08139         mSelectedPen.setColorF(r, g, b, a);
08140     }
08141
08142     void setBrushColorF(float r, float g, float b, float a) {
08143         mBrush.setColorF(r, g, b, a);
08144     }
08145
08146     void setSelectedBrushColorF(float r, float g, float b, float a) {
08147         mSelectedBrush.setColorF(r, g, b, a);
08148     }
08149
08150     void setFontPointSizeF(float pointSizeF) {
08151         mFont.setPointSizeF(pointSizeF);
08152     }
08153
08154     void setSelectedFontPointSizeF(float pointSizeF) {
08155         mSelectedFont.setPointSizeF(pointSizeF);
08156     }
08157
08158     void setTextPositionF(const QPointF &textPositionF) {
08159         mTextPosition = textPositionF;
08160     }
08161
08162     void setRotationAngleF(float angleF) {
08163         mRotation = angleF;
08164     }
08165
08166     void setPaddingLeftF(float leftF) {
08167         mPadding.setLeft(leftF);
08168     }
08169
08170     void setPaddingRightF(float rightF) {
08171         mPadding.setRight(rightF);
08172     }
08173
08174     void setPaddingTopF(float topF) {
08175         mPadding.setTop(topF);
08176     }
08177
08178     void setPaddingBottomF(float bottomF) {
08179         mPadding.setBottom(bottomF);
08180     }
08181
08182     void setTextColorF(float r, float g, float b, float a) {
08183         mColor.setRgbF(r, g, b, a);
08184     }
08185
08186     void setSelectedTextColorF(float r, float g, float b, float a) {
08187         mSelectedColor.setRgbF(r, g, b, a);
08188     }
08189
08190     void setPenColorF(float r, float g, float b, float a) {
08191         mPen.setColorF(r, g, b, a);
08192     }
08193
08194     void setSelectedPenColorF(float r, float g, float b, float a) {
08195         mSelectedPen.setColorF(r, g, b, a);
08196     }
08197
08198     void setBrushColorF(float r, float g, float b, float a) {
08199         mBrush.setColorF(r, g, b, a);
08200     }
08201
08202     void setSelectedBrushColorF(float r, float g, float b, float a) {
08203         mSelectedBrush.setColorF(r, g, b, a);
08204     }
08205
08206     void setFontPointSizeF(float pointSizeF) {
08207         mFont.setPointSizeF(pointSizeF);
08208     }
08209
08210     void setSelectedFontPointSizeF(float pointSizeF) {
08211         mSelectedFont.setPointSizeF(pointSizeF);
08212     }
08213
08214     void setTextPositionF(const QPointF &textPositionF) {
08215         mTextPosition = textPositionF;
08216     }
08217
08218     void setRotationAngleF(float angleF) {
08219         mRotation = angleF;
08220     }
08221
08222     void setPaddingLeftF(float leftF) {
08223         mPadding.setLeft(leftF);
08224     }
08225
08226     void setPaddingRightF(float rightF) {
08227         mPadding.setRight(rightF);
08228     }
08229
08230     void setPaddingTopF(float topF) {
08231         mPadding.setTop(topF);
08232     }
08233
08234     void setPaddingBottomF(float bottomF) {
08235         mPadding.setBottom(bottomF);
08236     }
08237
08238     void setTextColorF(float r, float g, float b, float a) {
08239         mColor.setRgbF(r, g, b, a);
08240     }
08241
08242     void setSelectedTextColorF(float r, float g, float b, float a) {
08243         mSelectedColor.setRgbF(r, g, b, a);
08244     }
08245
08246     void setPenColorF(float r, float g, float b, float a) {
08247         mPen.setColorF(r, g, b, a);
08248     }
08249
08250     void setSelectedPenColorF(float r, float g, float b, float a) {
08251         mSelectedPen.setColorF(r, g, b, a);
08252     }
08253
08254     void setBrushColorF(float r, float g, float b, float a) {
08255         mBrush.setColorF(r, g, b, a);
08256     }
08257
08258     void setSelectedBrushColorF(float r, float g, float b, float a) {
08259         mSelectedBrush.setColorF(r, g, b, a);
08260     }
08261
08262     void setFontPointSizeF(float pointSizeF) {
08263         mFont.setPointSizeF(pointSizeF);
08264     }
08265
08266     void setSelectedFontPointSizeF(float pointSizeF) {
08267         mSelectedFont.setPointSizeF(pointSizeF);
08268     }
08269
08270     void setTextPositionF(const QPointF &textPositionF) {
08271         mTextPosition = textPositionF;
08272     }
08273
08274     void setRotationAngleF(float angleF) {
08275         mRotation = angleF;
08276     }
08277
08278     void setPaddingLeftF(float leftF) {
08279         mPadding.setLeft(leftF);
08280     }
08281
08282     void setPaddingRightF(float rightF) {
08283         mPadding.setRight(rightF);
08284     }
08285
08286     void setPaddingTopF(float topF) {
08287         mPadding.setTop(topF);
08288     }
08289
08290     void setPaddingBottomF(float bottomF) {
08291         mPadding.setBottom(bottomF);
08292     }
08293
08294     void setTextColorF(float r, float g, float b, float a) {
08295         mColor.setRgbF(r, g, b, a);
08296     }
08297
08298     void setSelectedTextColorF(float r, float g, float b, float a) {
08299         mSelectedColor.setRgbF(r, g, b, a);
08300     }
08301
08302     void setPenColorF(float r, float g, float b, float a) {
08303         mPen.setColorF(r, g, b, a);
08304     }
08305
08306     void setSelectedPenColorF(float r, float g, float b, float a) {
08307         mSelectedPen.setColorF(r, g, b, a);
08308     }
08309
08310     void setBrushColorF(float r, float g, float b, float a) {
08311         mBrush.setColorF(r, g, b, a);
08312     }
08313
08314     void setSelectedBrushColorF(float r, float g, float b, float a) {
08315         mSelectedBrush.setColorF(r, g, b, a);
08316     }
08317
08318     void setFontPointSizeF(float pointSizeF) {
08319         mFont.setPointSizeF(pointSizeF);
08320     }
08321
08322     void setSelectedFontPointSizeF(float pointSizeF) {
08323         mSelectedFont.setPointSizeF(pointSizeF);
08324     }
08325
08326     void setTextPositionF(const QPointF &textPositionF) {
08327         mTextPosition = textPositionF;
08328     }
08329
08330     void setRotationAngleF(float angleF) {
08331         mRotation = angleF;
08332     }
08333
08334     void setPaddingLeftF(float leftF) {
08335         mPadding.setLeft(leftF);
08336     }
08337
08338     void setPaddingRightF(float rightF) {
08339         mPadding.setRight(rightF);
08340     }
08341
08342     void setPaddingTopF(float topF) {
08343         mPadding.setTop(topF);
08344     }
08345
08346     void setPaddingBottomF(float bottomF) {
08347         mPadding.setBottom(bottomF);
08348     }
08349
08350     void setTextColorF(float r, float g, float b, float a) {
08351         mColor.setRgbF(r, g, b, a);
08352     }
08353
08354     void setSelectedTextColorF(float r, float g, float b, float a) {
08355         mSelectedColor.setRgbF(r, g, b, a);
08356     }
08357
08358     void setPenColorF(float r, float g, float b, float a) {
08359         mPen.setColorF(r, g, b, a);
08360     }
08361
08362     void setSelectedPenColorF(float r, float g, float b, float a) {
08363         mSelectedPen.setColorF(r, g, b, a);
08364     }
08365
08366     void setBrushColorF(float r, float g, float b, float a) {
08367         mBrush.setColorF(r, g, b, a);
08368     }
08369
08370     void setSelectedBrushColorF(float r, float g, float b, float a) {
08371         mSelectedBrush.setColorF(r, g, b, a);
08372     }
08373
08374     void setFontPointSizeF(float pointSizeF) {
08375         mFont.setPointSizeF(pointSizeF);
08376     }
08377
08378     void setSelectedFontPointSizeF(float pointSizeF) {
08379         mSelectedFont.setPointSizeF(pointSizeF);
08380     }
08381
08382     void setTextPositionF(const QPointF &textPositionF) {
08383         mTextPosition = textPositionF;
08384     }
08385
08386     void setRotationAngleF(float angleF) {
08387         mRotation = angleF;
08388     }
08389
08390     void setPaddingLeftF(float leftF) {
08391         mPadding.setLeft(leftF);
08392     }
08393
08394     void setPaddingRightF(float rightF) {
08395         mPadding.setRight(rightF);
08396     }
08397
08398     void setPaddingTopF(float topF) {
08399         mPadding.setTop(topF);
08400     }
08401
08402     void setPaddingBottomF(float bottomF) {
08403         mPadding.setBottom(bottomF);
08404     }
08405
08406     void setTextColorF(float r, float g, float b, float a) {
08407         mColor.setRgbF(r, g, b, a);
08408     }
08409
08410     void setSelectedTextColorF(float r, float g, float b, float a) {
08411         mSelectedColor.setRgbF(r, g, b, a);
08412     }
08413
08414     void setPenColorF(float r, float g, float b, float a) {
08415         mPen.setColorF(r, g, b, a);
08416     }
08417
08418     void setSelectedPenColorF(float r, float g, float b, float a) {
08419         mSelectedPen.setColorF(r, g, b, a);
08420     }
08421
08422     void setBrushColorF(float r, float g, float b, float a) {
08423         mBrush.setColorF(r, g, b, a);
08424     }
08425
08426     void setSelectedBrushColorF(float r, float g, float b, float a) {
08427         mSelectedBrush.setColorF(r, g, b, a);
08428     }
08429
08430     void setFontPointSizeF(float pointSizeF) {
08431         mFont.setPointSizeF(pointSizeF);
08432     }
08433
08434     void setSelectedFontPointSizeF(float pointSizeF) {
08435         mSelectedFont.setPointSizeF(pointSizeF);
08436     }
08437
08438     void setTextPositionF(const QPointF &textPositionF) {
08439         mTextPosition = textPositionF;
08440     }
08441
08442     void setRotationAngleF(float angleF) {
08443         mRotation = angleF;
08444     }
08445
08446     void setPaddingLeftF(float leftF) {
08447         mPadding.setLeft(leftF);
08448     }
08449
08450     void setPaddingRightF(float rightF) {
08451         mPadding.setRight(rightF);
08452     }
08453
08454     void setPaddingTopF(float topF) {
08455         mPadding.setTop(topF);
08456     }
08457
08458     void setPaddingBottomF(float bottomF) {
08459         mPadding.setBottom(bottomF);
08460     }
08461
08462     void setTextColorF(float r, float g, float b, float a) {
08463         mColor.setRgbF(r, g, b, a);
08464     }
08465
08466     void setSelectedTextColorF(float r, float g, float b, float a) {
08467         mSelectedColor.setRgbF(r, g, b, a);
08468     }
08469
08470     void setPenColorF(float r, float g, float b, float a) {
08471         mPen.setColorF(r, g, b, a);
08472     }
08473
08474     void setSelectedPenColorF(float r, float g, float b, float a) {
08475         mSelectedPen.setColorF(r, g, b, a);
08476     }
08477
08478     void setBrushColorF(float r, float g, float b, float a) {
08479         mBrush.setColorF(r, g, b, a);
08480     }
08481
08482     void setSelectedBrushColorF(float r, float g, float b, float a) {
08483         mSelectedBrush.setColorF(r, g, b, a);
08484     }
08485
08486     void setFontPointSizeF(float pointSizeF) {
08487         mFont.setPointSizeF(pointSizeF);
08488     }
08489
08490     void setSelectedFontPointSizeF(float pointSizeF) {
08491         mSelectedFont.setPointSizeF(pointSizeF);
08492     }
08493
08494     void setTextPositionF(const QPointF &textPositionF) {
08495         mTextPosition = textPositionF;
08496     }
08497
08498     void setRotationAngleF(float angleF) {
08499         mRotation = angleF;
08500     }
08501
08502     void setPaddingLeftF(float leftF) {
08503         mPadding.setLeft(leftF);
08504     }
08505
08506     void setPaddingRightF(float rightF) {
08507         mPadding.setRight(rightF);
08508     }
08509
08510     void setPaddingTopF(float topF) {
08511         mPadding.setTop(topF);
08512     }
08513
08514     void setPaddingBottomF(float bottomF) {
08515         mPadding.setBottom(bottomF);
08516     }
08517
08518     void setTextColorF(float r, float g, float b, float a) {
08519         mColor.setRgbF(r, g, b, a);
08520     }
08521
08522     void setSelectedTextColorF(float r, float g, float b, float a) {
08523         mSelectedColor.setRgbF(r, g, b, a);
08524     }
08525
08526     void setPenColorF(float r, float g, float b, float a) {
08527         mPen.setColorF(r, g, b, a);
08528     }
08529
08530     void setSelectedPenColorF(float r, float g, float b, float a) {
08531         mSelectedPen.setColorF(r, g, b, a);
08532     }
08533
08534     void setBrushColorF(float r, float g, float b, float a) {
08535         mBrush.setColorF(r, g, b, a);
08536     }
08537
08538     void setSelectedBrushColorF(float r, float g, float b, float a) {
08539         mSelectedBrush.setColorF(r, g, b, a);
08540     }
08541
08542     void setFontPointSizeF(float pointSizeF) {
08543         mFont.setPointSizeF(pointSizeF);
08544     }
08545
08546     void setSelectedFontPointSizeF(float pointSizeF) {
08547         mSelectedFont.setPointSizeF(pointSizeF);
08548     }
08549
08550     void setTextPositionF(const QPointF &textPositionF) {
08551         mTextPosition = textPositionF;
08552     }
08553
08554     void setRotationAngleF(float angleF) {
08555         mRotation = angleF;
08556     }
08557
08558     void setPaddingLeftF(float leftF) {
08559         mPadding.setLeft(leftF);
08560     }
08561
08562     void setPaddingRightF(float rightF) {
08563         mPadding.setRight(rightF);
08564     }
08565
08566     void setPaddingTopF(float topF) {
08567         mPadding.setTop(topF);
08568     }
08569
08570     void setPaddingBottomF(float bottomF) {
08571         mPadding.setBottom(bottomF);
08572     }
08573
08574     void setTextColorF(float r, float g, float b, float a) {
08575         mColor.setRgbF(r, g, b, a);
08576     }
08577
08578     void setSelectedTextColorF(float r, float g, float b, float a) {
08579         mSelectedColor.setRgbF(r, g, b, a);
08580     }
08581
08582     void setPenColorF(float r, float g, float b, float a) {
08583         mPen.setColorF(r, g, b, a);
08584     }
08585
08586     void setSelectedPenColorF(float r, float g, float b, float a) {
08587         mSelectedPen.setColorF(r, g, b, a);
08588     }
08589
08590     void setBrushColorF(float r, float g, float b, float a) {
08591         mBrush.setColorF(r, g, b, a);
08592     }
08593
08594     void setSelectedBrushColorF(float r, float g, float b, float a) {
08595         mSelectedBrush.setColorF(r, g, b, a);
08596     }
08597
08598     void setFontPointSizeF(float pointSizeF) {
08599         mFont.setPointSizeF(pointSizeF);
08600     }
08601
08602     void setSelectedFontPointSizeF(float pointSizeF) {
08603         mSelectedFont.setPointSizeF(pointSizeF);
08604     }
08605
08606     void setTextPositionF(const QPointF &textPositionF) {
08607         mTextPosition = textPositionF;
08608     }
08609
08610     void setRotationAngleF(float angleF) {
08611         mRotation = angleF;
08612     }
08613
08614     void setPaddingLeftF(float leftF) {
08615         mPadding.setLeft(leftF);
08616     }
08617
08618     void setPaddingRightF(float rightF) {
08619         mPadding.setRight(rightF);
08620     }
08621
08622     void setPaddingTopF(float topF) {
08623         mPadding.setTop(topF);
08624     }
08625
08626     void setPaddingBottomF(float bottomF) {
08627         mPadding.setBottom(bottomF);
08628     }
08629
08630     void setTextColorF(float r, float g, float b, float a) {
08631         mColor.setRgbF(r, g, b, a);
08632     }
08633
08634     void setSelectedTextColorF(float r, float g, float b, float a) {
08635         mSelectedColor.setRgbF(r, g, b, a);
08636     }
08637
08638     void setPenColorF(float r, float g, float b, float a) {
08639         mPen.setColorF(r, g, b, a);
08640     }
08641
08642     void setSelectedPenColorF(float r, float g, float b, float a) {
08643         mSelectedPen.setColorF(r, g, b, a);
08644     }
08645
08646     void setBrushColorF(float r, float g, float b, float a) {
08647         mBrush.setColorF(r, g, b, a);
08648     }
08649
08650     void setSelectedBrushColorF(float r, float g, float b, float a) {
08651         mSelectedBrush.setColorF(r, g, b, a);
08652     }
08653
08654     void setFontPointSizeF(float pointSizeF) {
08655         mFont.setPointSizeF(pointSizeF);
08656     }
08657
08658     void setSelectedFontPointSizeF(float pointSizeF) {
08659         mSelectedFont.setPointSizeF(pointSizeF);
08660     }
08661
08662     void setTextPositionF(const QPointF &textPositionF) {
08663         mTextPosition = textPositionF;
08664     }
08665
08666     void setRotationAngleF(float angleF) {
08667         mRotation = angleF;
08668     }
08669
08670     void setPaddingLeftF(float leftF) {
08671         mPadding.setLeft(leftF);
08672     }
08673
08674     void setPaddingRightF(float rightF) {
08675         mPadding.setRight(rightF);
08676     }
08677
08678     void setPaddingTopF(float topF) {
08679         mPadding.setTop(topF);
08680     }
08681
08682     void setPaddingBottomF(float bottomF) {
08683         mPadding.setBottom(bottomF);
08684     }
08685
08686     void setTextColorF(float r, float g, float b, float a) {
08687         mColor.setRgbF(r, g, b, a);
08688     }
08689
08690     void setSelectedTextColorF(float r, float g, float b, float a) {
08691         mSelectedColor.setRgbF(r, g, b, a);
08692     }
08693
08694     void setPenColorF(float r, float g, float b, float a) {
08695         m
```

```

07799
07800     QFont font() const {
07801         return mFont;
07802     }
07803
07804     QFont selectedFont() const {
07805         return mSelectedFont;
07806     }
07807
07808     QString text() const {
07809         return mText;
07810     }
07811
07812     Qt::Alignment positionAlignment() const {
07813         return mPositionAlignment;
07814     }
07815
07816     Qt::Alignment textAlignment() const {
07817         return mTextAlignment;
07818     }
07819
07820     double rotation() const {
07821         return mRotation;
07822     }
07823
07824     QMargins padding() const {
07825         return mPadding;
07826     }
07827
07828     // setters;
07829     void setColor(const QColor &color);
07830     void setSelectedColor(const QColor &color);
07831     void setPen(const QPen &pen);
07832     void setSelectedPen(const QPen &pen);
07833     void setBrush(const QBrush &brush);
07834     void setSelectedBrush(const QBrush &brush);
07835     void setFont(const QFont &font);
07836     void setSelectedFont(const QFont &font);
07837     void setText(const QString &text);
07838     void setPositionAlignment(Qt::Alignment alignment);
07839     void setTextAlignment(Qt::Alignment alignment);
07840     void setRotation(double degrees);
07841     void setPadding(const QMargins &padding);
07842
07843     // reimplemented virtual methods:
07844     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
07845         const Q_DECL_OVERRIDE;
07846
07847     QCPItemPosition * const position;
07848     QCPItemAnchor * const topLeft;
07849     QCPItemAnchor * const top;
07850     QCPItemAnchor * const topRight;
07851     QCPItemAnchor * const right;
07852     QCPItemAnchor * const bottomRight;
07853     QCPItemAnchor * const bottom;
07854     QCPItemAnchor * const bottomLeft;
07855     QCPItemAnchor * const left;
07856
07857 protected:
07858     enum AnchorIndex {
07859         aiTopLeft, aiTop, aiTopRight, aiRight, aiBottomRight, aiBottom, aiBottomLeft, aiLeft
07860     };
07861
07862     // property members:
07863     QColor mColor, mSelectedColor;
07864     QPen mPen, mSelectedPen;
07865     QBrush mBrush, mSelectedBrush;
07866     QFont mFont, mSelectedFont;
07867     QString mText;
07868     Qt::Alignment mPositionAlignment;
07869     Qt::Alignment mTextAlignment;
07870     double mRotation;
07871     QMargins mPadding;
07872
07873     // reimplemented virtual methods:
07874     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07875     virtual QPointF anchorPixelPosition(int anchorId) const Q_DECL_OVERRIDE;
07876
07877     // non-virtual methods:
07878     QPointF getTextDrawPoint(const QPointF &pos, const QRectF &rect, Qt::Alignment positionAlignment)
07879         const;
07880     QFont mainFont() const;
07881     QColor mainColor() const;
07882     QPen mainPen() const;
07883     QBrush mainBrush() const;
07884 };

```

```

07884
07885 /* end of 'src/items/item-text.h' */
07886
07887
07888 /* including file 'src/items/item-ellipse.h' */
07889
07890 /* modified 2021-03-29T02:30:44, size 3890 */
07891
07892 class QCP_LIB_DECL QCPItemEllipse : public QCPAbstractItem {
07893     Q_OBJECT
07894     Q_PROPERTY(QPen pen READ pen WRITE setPen)
07895     Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
07896     Q_PROPERTY(QBrush brush READ brush WRITE setBrush)
07897     Q_PROPERTY(QBrush selectedBrush READ selectedBrush WRITE setSelectedBrush)
07898 public:
07899     explicit QCPItemEllipse(QCustomPlot *parentPlot);
07900     virtual ~QCPItemEllipse() Q_DECL_OVERRIDE;
07901
07902     // getters:
07903
07904     QPen pen() const {
07905         return mPen;
07906     }
07907
07908     QPen selectedPen() const {
07909         return mSelectedPen;
07910     }
07911
07912     QBrush brush() const {
07913         return mBrush;
07914     }
07915
07916     QBrush selectedBrush() const {
07917         return mSelectedBrush;
07918     }
07919
07920     // setters:
07921     void setPen(const QPen &pen);
07922     void setSelectedPen(const QPen &pen);
07923     void setBrush(const QBrush &brush);
07924     void setSelectedBrush(const QBrush &brush);
07925
07926     // reimplemented virtual methods:
07927     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr) const Q_DECL_OVERRIDE;
07928
07929     QCPItemPosition * const topLeft;
07930     QCPItemPosition * const bottomRight;
07931     QCPItemAnchor * const topLeftRim;
07932     QCPItemAnchor * const top;
07933     QCPItemAnchor * const topRightRim;
07934     QCPItemAnchor * const right;
07935     QCPItemAnchor * const bottomRightRim;
07936     QCPItemAnchor * const bottom;
07937     QCPItemAnchor * const bottomLeftRim;
07938     QCPItemAnchor * const left;
07939     QCPItemAnchor * const center;
07940
07941 protected:
07942     enum AnchorIndex {
07943         aiTopLeftRim, aiTop, aiTopRightRim, aiRight, aiBottomRightRim, aiBottom, aiBottomLeftRim,
07944         aiLeft, aiCenter
07945     };
07946     QPen mPen, mSelectedPen;
07947     QBrush mBrush, mSelectedBrush;
07948
07949     // reimplemented virtual methods:
07950     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
07951     virtual QPointF anchorPixelPosition(int anchorId) const Q_DECL_OVERRIDE;
07952
07953     // non-virtual methods:
07954     QPen mainPen() const;
07955     QBrush mainBrush() const;
07956 };
07957
07958 /* end of 'src/items/item-ellipse.h' */
07959
07960 /* including file 'src/items/item-pixmap.h' */
07961
07962 /* modified 2021-03-29T02:30:44, size 4407 */
07963
07964 class QCP_LIB_DECL QCPItemPixmap : public QCPAbstractItem {
07965     Q_OBJECT

```

```

07972     Q_PROPERTY(QPixmap pixmap READ pixmap WRITE setPixmap)
07973     Q_PROPERTY(bool scaled READ scaled WRITE setScaled)
07974     Q_PROPERTY(Qt::AspectRatioMode aspectRatioMode READ aspectRatioMode)
07975     Q_PROPERTY(Qt::TransformationMode transformationMode READ transformationMode)
07976     Q_PROPERTY(QPen pen READ pen WRITE setPen)
07977     Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
07979 public:
07980     explicit QCPItemPixmap(QCustomPlot *parentPlot);
07981     virtual ~QCPItemPixmap() Q_DECL_OVERRIDE;
07982
07983     // getters:
07984
07985     QPixmap pixmap() const {
07986         return mPixmap;
07987     }
07988
07989     bool scaled() const {
07990         return mScaled;
07991     }
07992
07993     Qt::AspectRatioMode aspectRatioMode() const {
07994         return mAspectRatioMode;
07995     }
07996
07997     Qt::TransformationMode transformationMode() const {
07998         return mTransformationMode;
07999     }
08000
08001     QPen pen() const {
08002         return mPen;
08003     }
08004
08005     QPen selectedPen() const {
08006         return mSelectedPen;
08007     }
08008
08009     // setters;
08010     void setPixmap(const QPixmap &pixmap);
08011     void setScaled(bool scaled, Qt::AspectRatioMode aspectRatioMode = Qt::KeepAspectRatio,
08012 Qt::TransformationMode transformationMode = Qt::SmoothTransformation);
08013     void setPen(const QPen &pen);
08014     void setSelectedPen(const QPen &pen);
08015
08016     // reimplemented virtual methods:
08017     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr)
08018     const Q_DECL_OVERRIDE;
08019
08020     QCPItemPosition * const topLeft;
08021     QCPItemPosition * const bottomRight;
08022     QCPItemAnchor * const top;
08023     QCPItemAnchor * const topRight;
08024     QCPItemAnchor * const right;
08025     QCPItemAnchor * const bottom;
08026     QCPItemAnchor * const bottomLeft;
08027     QCPItemAnchor * const left;
08028
08029 protected:
08030
08031     enum AnchorIndex {
08032         aiTop, aiTopRight, aiRight, aiBottom, aiBottomLeft, aiLeft
08033     };
08034
08035     // property members:
08036     QPixmap mPixmap;
08037     QPixmap mScaledPixmap;
08038     bool mScaled;
08039     bool mScaledPixmapInvalidated;
08040     Qt::AspectRatioMode mAspectRatioMode;
08041     Qt::TransformationMode mTransformationMode;
08042     QPen mPen, mSelectedPen;
08043
08044     // reimplemented virtual methods:
08045     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
08046     virtual QPointF anchorPixelPosition(int anchorId) const Q_DECL_OVERRIDE;
08047
08048     // non-virtual methods:
08049     void updateScaledPixmap(QRect finalRect = QRect(), bool flipHorz = false, bool flipVert = false);
08050     QRect getFinalRect(bool *flippedHorz = nullptr, bool *flippedVert = nullptr) const;
08051     QPen mainPen() const;
08052 };
08053
08054 /* end of 'src/items/item-pixmap.h' */
08055
08056 /* including file 'src/items/item-tracer.h' */
08057
08058 /* modified 2021-03-29T02:30:44, size 4811 */

```

```

08058
08059 class QCP_LIB_DECL QCPItemTracer : public QCPAbstractItem {
08060
08061     Q_OBJECT
08062     Q_PROPERTY(QPen pen READ pen WRITE setPen)
08063     Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
08064     Q_PROPERTY(QBrush brush READ brush WRITE setBrush)
08065     Q_PROPERTY(QBrush selectedBrush READ selectedBrush WRITE setSelectedBrush)
08066     Q_PROPERTY(double size READ size WRITE setSize)
08067     Q_PROPERTY(TracerStyle style READ style WRITE setStyle)
08068     Q_PROPERTY(QCPGraph* graph READ graph WRITE setGraph)
08069     Q_PROPERTY(double graphKey READ graphKey WRITE setGraphKey)
08070     Q_PROPERTY(bool interpolating READ interpolating WRITE setInterpolating)
08071 public:
08072     enum TracerStyle {
08073         tsNone
08074         , tsPlus
08075         , tsCrosshair
08076         , tsCircle
08077         , tsSquare
08078     };
08079     Q_ENUMS(TracerStyle)
08080
08081     explicit QCPItemTracer(QCustomPlot *parentPlot);
08082     virtual ~QCPItemTracer() Q_DECL_OVERRIDE;
08083
08084     // getters:
08085
08086     QPen pen() const {
08087         return mPen;
08088     }
08089
08090     QPen selectedPen() const {
08091         return mSelectedPen;
08092     }
08093
08094     QBrush brush() const {
08095         return mBrush;
08096     }
08097
08098     QBrush selectedBrush() const {
08099         return mSelectedBrush;
08100     }
08101
08102     double size() const {
08103         return mSize;
08104     }
08105
08106     TracerStyle style() const {
08107         return mStyle;
08108     }
08109
08110     QCPGraph *graph() const {
08111         return mGraph;
08112     }
08113
08114     double graphKey() const {
08115         return mGraphKey;
08116     }
08117
08118     bool interpolating() const {
08119         return mInterpolating;
08120     }
08121
08122     // setters:
08123     void setPen(const QPen &pen);
08124     void setSelectedPen(const QPen &pen);
08125     void setBrush(const QBrush &brush);
08126     void setSelectedBrush(const QBrush &brush);
08127     void setSize(double size);
08128     void setStyle(TracerStyle style);
08129     void setGraph(QCPGraph *graph);
08130     void setGraphKey(double key);
08131     void setInterpolating(bool enabled);
08132
08133     // reimplemented virtual methods:
08134     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr) const Q_DECL_OVERRIDE;
08135
08136     // non-virtual methods:
08137     void updatePosition();
08138
08139     QCPItemPosition * const position;
08140 protected:
08141     // property members:
08142     QPen mPen, mSelectedPen;

```

```

08151     QBrush mBrush, mSelectedBrush;
08152     double mSize;
08153     TracerStyle mStyle;
08154     QCPGraph *mGraph;
08155     double mGraphKey;
08156     bool mInterpolating;
08157
08158     // reimplemented virtual methods:
08159     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
08160
08161     // non-virtual methods:
08162     QPen mainPen() const;
08163     QBrush mainBrush() const;
08164 };
08165
08166 Q_DECLARE_METATYPE(QCPItemTracer::TracerStyle)
08167
08168 /* end of 'src/items/item-tracer.h' */
08169
08170
08171 /* including file 'src/items/item-bracket.h' */
08172 /* modified 2021-03-29T02:30:44, size 3991 */
08173
08174 class QCP_LIB_DECL QCPItemBracket : public QCPAbstractItem {
08175
08176     Q_OBJECT
08177     Q_PROPERTY(QPen pen READ pen WRITE setPen)
08178     Q_PROPERTY(QPen selectedPen READ selectedPen WRITE setSelectedPen)
08179     Q_PROPERTY(double length READ length WRITE setLength)
08180     Q_PROPERTY(BracketStyle style READ style WRITE setStyle)
08181 public:
08182     enum BracketStyle {
08183         bsSquare
08184         , bsRound
08185         , bsCurly
08186         , bsCalligraphic
08187     };
08188     Q_ENUMS(BracketStyle)
08189
08190     explicit QCPItemBracket(QCustomPlot *parentPlot);
08191     virtual ~QCPItemBracket() Q_DECL_OVERRIDE;
08192
08193     // getters:
08194     QPen pen() const {
08195         return mPen;
08196     }
08197     QPen selectedPen() const {
08198         return mSelectedPen;
08199     }
08200     double length() const {
08201         return mLength;
08202     }
08203     BracketStyle style() const {
08204         return mStyle;
08205     }
08206     // setters:
08207     void setPen(const QPen &pen);
08208     void setSelectedPen(const QPen &pen);
08209     void setLength(double length);
08210     void setStyle(BracketStyle style);
08211
08212     // reimplemented virtual methods:
08213     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = nullptr) const Q_DECL_OVERRIDE;
08214
08215     QCPItemPosition * const left;
08216     QCPItemPosition * const right;
08217     QCPItemAnchor * const center;
08218
08219 protected:
08220     // property members:
08221     enum AnchorIndex {
08222         aiCenter
08223     };
08224     QPen mPen, mSelectedPen;
08225     double mLength;
08226     BracketStyle mStyle;
08227
08228     // reimplemented virtual methods:
08229     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
08230     virtual QPointF anchorPixelPosition(int anchorId) const Q_DECL_OVERRIDE;

```



```

08245
08246     // non-virtual methods:
08247     QPen mainPen() const;
08248 };
08249
08250 Q_DECLARE_METATYPE(QCItemBracket::BracketStyle)
08251
08252 /* end of 'src/items/item-bracket.h' */
08253
08254
08255 /* including file 'src/polar/radialaxis.h' */
08256 /* modified 2021-03-29T02:30:44, size 12227 */
08257
08258
08259 class QCP_LIB_DECL QCPPolarAxisRadial : public QCPLayerable {
08260
08261     Q_OBJECT
08262
08263 public:
08264     enum AngleReference {
08265         arAbsolute
08266         , arAngularAxis
08267     };
08268
08269     Q_ENUMS(AngleReference)
08270     enum ScaleType {
08271         stLinear
08272         , stLogarithmic
08273     };
08274
08275     Q_ENUMS(ScaleType)
08276     enum SelectablePart {
08277         spNone = 0
08278         , spAxis = 0x001
08279         , spTickLabels = 0x002
08280         , spAxisLabel = 0x004
08281     };
08282
08283     Q_ENUMS(SelectablePart)
08284     Q_FLAGS(SelectableParts)
08285     Q_DECLARE_FLAGS(SelectableParts, SelectablePart)
08286
08287     enum LabelMode {
08288         lmUpright
08289         , lmRotated
08290     };
08291
08292     Q_ENUMS(LabelMode)
08293
08294     explicit QCPPolarAxisRadial(QCPPolarAxisAngular *parent);
08295     virtual ~QCPPolarAxisRadial();
08296
08297     // getters:
08298
08299     bool rangeDrag() const {
08300         return mRangeDrag;
08301     }
08302
08303     bool rangeZoom() const {
08304         return mRangeZoom;
08305     }
08306
08307     double rangeZoomFactor() const {
08308         return mRangeZoomFactor;
08309     }
08310
08311     QCPPolarAxisAngular *angularAxis() const {
08312         return mAngularAxis;
08313     }
08314
08315     ScaleType scaleType() const {
08316         return mScaleType;
08317     }
08318
08319     const QCPRange range() const {
08320         return mRange;
08321     }
08322
08323     bool rangeReversed() const {
08324         return mRangeReversed;
08325     }
08326
08327     double angle() const {
08328         return mAngle;
08329     }
08330
08331     AngleReference angleReference() const {
08332         return mAngleReference;
08333     }

```

```
08345     }
08346
08347     QSharedPointer<QCPAxisTicker> ticker() const {
08348         return mTicker;
08349     }
08350
08351     bool ticks() const {
08352         return mTicks;
08353     }
08354
08355     bool tickLabels() const {
08356         return mTickLabels;
08357     }
08358
08359     int tickLabelPadding() const {
08360         return mLabelPainter.padding();
08361     }
08362
08363     QFont tickLabelFont() const {
08364         return mTickLabelFont;
08365     }
08366
08367     QColor tickLabelColor() const {
08368         return mTickLabelColor;
08369     }
08370
08371     double tickLabelRotation() const {
08372         return mLabelPainter.rotation();
08373     }
08374     LabelMode tickLabelMode() const;
08375     QString numberFormat() const;
08376
08377     int numberPrecision() const {
08378         return mNumberPrecision;
08379     }
08380
08381     QVector<double> tickVector() const {
08382         return mTickVector;
08383     }
08384
08385     QVector<double> subTickVector() const {
08386         return mSubTickVector;
08387     }
08388
08389     QVector<QString> tickVectorLabels() const {
08390         return mTickVectorLabels;
08391     }
08392     int tickLengthIn() const;
08393     int tickLengthOut() const;
08394
08395     bool subTicks() const {
08396         return mSubTicks;
08397     }
08398     int subTickLengthIn() const;
08399     int subTickLengthOut() const;
08400
08401     QPen basePen() const {
08402         return mBasePen;
08403     }
08404
08405     QPen tickPen() const {
08406         return mTickPen;
08407     }
08408
08409     QPen subTickPen() const {
08410         return mSubTickPen;
08411     }
08412
08413     QFont labelFont() const {
08414         return mLabelFont;
08415     }
08416
08417     QColor labelColor() const {
08418         return mLabelColor;
08419     }
08420
08421     QString label() const {
08422         return mLabel;
08423     }
08424     int labelPadding() const;
08425
08426     SelectableParts selectedParts() const {
08427         return mSelectedParts;
08428     }
08429
08430     SelectableParts selectableParts() const {
08431         return mSelectableParts;
```

```

08432     }
08433
08434     QFont selectedTickLabelFont() const {
08435         return mSelectedTickLabelFont;
08436     }
08437
08438     QFont selectedLabelFont() const {
08439         return mSelectedLabelFont;
08440     }
08441
08442     QColor selectedTickLabelColor() const {
08443         return mSelectedTickLabelColor;
08444     }
08445
08446     QColor selectedLabelColor() const {
08447         return mSelectedLabelColor;
08448     }
08449
08450     QPen selectedBasePen() const {
08451         return mSelectedBasePen;
08452     }
08453
08454     QPen selectedTickPen() const {
08455         return mSelectedTickPen;
08456     }
08457
08458     QPen selectedSubTickPen() const {
08459         return mSelectedSubTickPen;
08460     }
08461
08462     // setters:
08463     void setRangeDrag(bool enabled);
08464     void setRangeZoom(bool enabled);
08465     void setRangeZoomFactor(double factor);
08466
08467     Q_SLOT void setScaleType(QCPPolarAxisRadial::ScaleType type);
08468     Q_SLOT void setRange(const QCPRange &range);
08469     void setRange(double lower, double upper);
08470     void setRange(double position, double size, Qt::AlignmentFlag alignment);
08471     void setRangeLower(double lower);
08472     void setRangeUpper(double upper);
08473     void setRangeReversed(bool reversed);
08474     void setAngle(double degrees);
08475     void setAngleReference(AngleReference reference);
08476     void setTicker(QSharedPointer<QCPAxisTicker> ticker);
08477     void setTicks(bool show);
08478     void setTickLabels(bool show);
08479     void setTickLabelPadding(int padding);
08480     void setTickLabelFont(const QFont &font);
08481     void setTickLabelColor(const QColor &color);
08482     void setTickLabelRotation(double degrees);
08483     void setTickLabelMode(LabelMode mode);
08484     void setNumberFormat(const QString &formatCode);
08485     void setNumberPrecision(int precision);
08486     void setTickLength(int inside, int outside = 0);
08487     void setTickLengthIn(int inside);
08488     void setTickLengthOut(int outside);
08489     void setSubTicks(bool show);
08490     void setSubTickLength(int inside, int outside = 0);
08491     void setSubTickLengthIn(int inside);
08492     void setSubTickLengthOut(int outside);
08493     void setBasePen(const QPen &pen);
08494     void setTickPen(const QPen &pen);
08495     void setSubTickPen(const QPen &pen);
08496     void setLabelFont(const QFont &font);
08497     void setLabelColor(const QColor &color);
08498     void setLabel(const QString &str);
08499     void setLabelPadding(int padding);
08500     void setSelectedTickLabelFont(const QFont &font);
08501     void setSelectedLabelFont(const QFont &font);
08502     void setSelectedTickLabelColor(const QColor &color);
08503     void setSelectedLabelColor(const QColor &color);
08504     void setSelectedBasePen(const QPen &pen);
08505     void setSelectedTickPen(const QPen &pen);
08506     void setSelectedSubTickPen(const QPen &pen);
08507     Q_SLOT void setSelectableParts(const QCPPolarAxisRadial::SelectableParts &selectableParts);
08508     Q_SLOT void setSelectedParts(const QCPPolarAxisRadial::SelectableParts &selectedParts);
08509
08510     // reimplemented virtual methods:
08511     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = 0) const
08512         Q_DECL_OVERRIDE;
08513
08514     // non-property methods:
08515     void moveRange(double diff);
08516     void scaleRange(double factor);
08517     void scaleRange(double factor, double center);
08518     void rescale(bool onlyVisiblePlottables = false);

```

```

08518     void pixelToCoord(QPointF pixelPos, double &angleCoord, double &radiusCoord) const;
08519     QPointF coordToPixel(double angleCoord, double radiusCoord) const;
08520     double coordToRadius(double coord) const;
08521     double radiusToCoord(double radius) const;
08522     SelectablePart getPartAt(const QPointF &pos) const;
08523
08524 signals:
08525     void rangeChanged(const QCPRange &newRange);
08526     void rangeChanged(const QCPRange &newRange, const QCPRange &oldRange);
08527     void scaleTypeChanged(QCPPolarAxisRadial::ScaleType scaleType);
08528     void selectionChanged(const QCPPolarAxisRadial::SelectableParts &parts);
08529     void selectableChanged(const QCPPolarAxisRadial::SelectableParts &parts);
08530
08531 protected:
08532     // property members:
08533     bool mRangeDrag;
08534     bool mRangeZoom;
08535     double mRangeZoomFactor;
08536
08537     // axis base:
08538     QCPPolarAxisAngular *mAngularAxis;
08539     double mAngle;
08540     AngleReference mAngleReference;
08541     SelectableParts mSelectableParts, mSelectedParts;
08542     QPen mBasePen, mSelectedBasePen;
08543     // axis label:
08544     int mLabelPadding;
08545     QString mLabel;
08546     QFont mLabelFont, mSelectedLabelFont;
08547     QColor mLabelColor, mSelectedLabelColor;
08548     // tick labels:
08549     //int mTickLabelPadding; in label painter
08550     bool mTickLabels;
08551     //double mTickLabelRotation; in label painter
08552     QFont mTickLabelFont, mSelectedTickLabelFont;
08553     QColor mTickLabelColor, mSelectedTickLabelColor;
08554     int mNumberPrecision;
08555     QLatin1Char mNumberFormatChar;
08556     bool mNumberBeautifulPowers;
08557     bool mNumberMultiplyCross;
08558     // ticks and subticks:
08559     bool mTicks;
08560     bool mSubTicks;
08561     int mTickLengthIn, mTickLengthOut, mSubTickLengthIn, mSubTickLengthOut;
08562     QPen mTickPen, mSelectedTickPen;
08563     QPen mSubTickPen, mSelectedSubTickPen;
08564     // scale and range:
08565     QCPRange mRange;
08566     bool mRangeReversed;
08567     ScaleType mScaleType;
08568
08569     // non-property members:
08570     QPointF mCenter;
08571     double mRadius;
08572     QSharedPointer<QCPAxisTicker> mTicker;
08573     QVector<double> mTickVector;
08574     QVector<QString> mTickVectorLabels;
08575     QVector<double> mSubTickVector;
08576     bool mDragging;
08577     QCPRange mDragStartRange;
08578     QCP::AntialiasedElements mAADragBackup, mNotAADragBackup;
08579     QCPLabelPainterPrivate mLabelPainter;
08580
08581     // reimplemented virtual methods:
08582     virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
08583     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
08584     virtual QCP::Interaction selectionCategory() const Q_DECL_OVERRIDE;
08585     // events:
08586     virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
*selectionStateChanged) Q_DECL_OVERRIDE;
08587     virtual void deselectEvent(bool *selectionStateChanged) Q_DECL_OVERRIDE;
08588     // mouse events:
08589     virtual void mousePressEvent(QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE;
08590     virtual void mouseMoveEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
08591     virtual void mouseReleaseEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
08592     virtual void wheelEvent(QWheelEvent *event) Q_DECL_OVERRIDE;
08593
08594     // non-virtual methods:
08595     void updateGeometry(const QPointF &center, double radius);
08596     void setupTickVectors();
08597     QPen getBasePen() const;
08598     QPen getTickPen() const;
08599     QPen getSubTickPen() const;
08600     QFont getTickLabelFont() const;
08601     QFont getLabelFont() const;
08602     QColor getTickLabelColor() const;
08603     QColor getLabelColor() const;

```

```

08604
08605 private:
08606     Q_DISABLE_COPY(QCPPolarAxisRadial)
08607
08608     friend class QCustomPlot;
08609     friend class QCPPolarAxisAngular;
08610 };
08611
08612 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPPolarAxisRadial::SelectableParts)
08613 Q_DECLARE_METATYPE(QCPPolarAxisRadial::AngleReference)
08614 Q_DECLARE_METATYPE(QCPPolarAxisRadial::ScaleType)
08615 Q_DECLARE_METATYPE(QCPPolarAxisRadial::SelectablePart)
08616
08617
08618
08619 /* end of 'src/polar/radialaxis.h' */
08620
08621
08622 /* including file 'src/polar/layoutelement-angularaxis.h' */
08623 /* modified 2021-03-29T02:30:44, size 13461 */
08624
08625 class QCP_LIB_DECL QCPPolarAxisAngular : public QCPLayoutElement {
08626
08627     Q_OBJECT
08629
08631 public:
08636     enum SelectablePart {
08637         spNone = 0
08638         , spAxis = 0x001
08639         , spTickLabels = 0x002
08640         , spAxisLabel = 0x004
08641     };
08642
08643     Q_ENUMS(SelectablePart)
08644     Q_FLAGS(SelectableParts)
08645     Q_DECLARE_FLAGS(SelectableParts, SelectablePart)
08646
08647
08650     enum LabelMode {
08651         lmUpright
08652         , lmRotated
08653     };
08654     Q_ENUMS(LabelMode)
08655
08656     explicit QCPPolarAxisAngular(QCustomPlot *parentPlot);
08657     virtual ~QCPPolarAxisAngular();
08658
08659     // getters:
08660
08661     QPixmap background() const {
08662         return mBackgroundPixmap;
08663     }
08664
08665     QBrush backgroundBrush() const {
08666         return mBackgroundBrush;
08667     }
08668
08669     bool backgroundScaled() const {
08670         return mBackgroundScaled;
08671     }
08672
08673     Qt::AspectRatioMode backgroundScaledMode() const {
08674         return mBackgroundScaledMode;
08675     }
08676
08677     bool rangeDrag() const {
08678         return mRangeDrag;
08679     }
08680
08681     bool rangeZoom() const {
08682         return mRangeZoom;
08683     }
08684
08685     double rangeZoomFactor() const {
08686         return mRangeZoomFactor;
08687     }
08688
08689     const QCPRange range() const {
08690         return mRange;
08691     }
08692
08693     bool rangeReversed() const {
08694         return mRangeReversed;
08695     }
08696
08697     double angle() const {
08698         return mAngle;

```

```
08699     }
08700
08701     QSharedPointer<QCPAxisTicker> ticker() const {
08702         return mTicker;
08703     }
08704
08705     bool ticks() const {
08706         return mTicks;
08707     }
08708
08709     bool tickLabels() const {
08710         return mTickLabels;
08711     }
08712
08713     int tickLabelPadding() const {
08714         return mLabelPainter.padding();
08715     }
08716
08717     QFont tickLabelFont() const {
08718         return mTickLabelFont;
08719     }
08720
08721     QColor tickLabelColor() const {
08722         return mTickLabelColor;
08723     }
08724
08725     double tickLabelRotation() const {
08726         return mLabelPainter.rotation();
08727     }
08728     LabelMode tickLabelMode() const;
08729     QString numberFormat() const;
08730
08731     int numberPrecision() const {
08732         return mNumberPrecision;
08733     }
08734
08735     QVector<double> tickVector() const {
08736         return mTickVector;
08737     }
08738
08739     QVector<QString> tickVectorLabels() const {
08740         return mTickVectorLabels;
08741     }
08742
08743     int tickLengthIn() const {
08744         return mTickLengthIn;
08745     }
08746
08747     int tickLengthOut() const {
08748         return mTickLengthOut;
08749     }
08750
08751     bool subTicks() const {
08752         return mSubTicks;
08753     }
08754
08755     int subTickLengthIn() const {
08756         return mSubTickLengthIn;
08757     }
08758
08759     int subTickLengthOut() const {
08760         return mSubTickLengthOut;
08761     }
08762
08763     QPen basePen() const {
08764         return mBasePen;
08765     }
08766
08767     QPen tickPen() const {
08768         return mTickPen;
08769     }
08770
08771     QPen subTickPen() const {
08772         return mSubTickPen;
08773     }
08774
08775     QFont labelFont() const {
08776         return mLabelFont;
08777     }
08778
08779     QColor labelColor() const {
08780         return mLabelColor;
08781     }
08782
08783     QString label() const {
08784         return mLabel;
08785     }
```

```

08786
08787     int labelPadding() const {
08788         return mLabelPadding;
08789     }
08790
08791     SelectableParts selectedParts() const {
08792         return mSelectedParts;
08793     }
08794
08795     SelectableParts selectableParts() const {
08796         return mSelectableParts;
08797     }
08798
08799     QFont selectedTickLabelFont() const {
08800         return mSelectedTickLabelFont;
08801     }
08802
08803     QFont selectedLabelFont() const {
08804         return mSelectedLabelFont;
08805     }
08806
08807     QColor selectedTickLabelColor() const {
08808         return mSelectedTickLabelColor;
08809     }
08810
08811     QColor selectedLabelColor() const {
08812         return mSelectedLabelColor;
08813     }
08814
08815     QPen selectedBasePen() const {
08816         return mSelectedBasePen;
08817     }
08818
08819     QPen selectedTickPen() const {
08820         return mSelectedTickPen;
08821     }
08822
08823     QPen selectedSubTickPen() const {
08824         return mSelectedSubTickPen;
08825     }
08826
08827     QCPPolarGrid *grid() const {
08828         return mGrid;
08829     }
08830
08831     // setters:
08832     void setBackground(const QPixmap &pm);
08833     void setBackground(const QPixmap &pm, bool scaled, Qt::AspectRatioMode mode =
Qt::KeepAspectRatioByExpanding);
08834     void setBackground(const QBrush &brush);
08835     void setBackgroundScaled(bool scaled);
08836     void setBackgroundScaledMode(Qt::AspectRatioMode mode);
08837     void setRangeDrag(bool enabled);
08838     void setRangeZoom(bool enabled);
08839     void setRangeZoomFactor(double factor);
08840
08841     Q_SLOT void setRange(const QCPRange &range);
08842     void setRange(double lower, double upper);
08843     void setRange(double position, double size, Qt::AlignmentFlag alignment);
08844     void setRangeLower(double lower);
08845     void setRangeUpper(double upper);
08846     void setRangeReversed(bool reversed);
08847     void setAngle(double degrees);
08848     void setTicker(QSharedPointer<QCPAxisTicker> ticker);
08849     void setTicks(bool show);
08850     void setTickLabels(bool show);
08851     void setTickLabelPadding(int padding);
08852     void setTickLabelFont(const QFont &font);
08853     void setTickLabelColor(const QColor &color);
08854     void setTickLabelRotation(double degrees);
08855     void setTickLabelMode(LabelMode mode);
08856     void setNumberFormat(const QString &formatCode);
08857     void setNumberPrecision(int precision);
08858     void setTickLength(int inside, int outside = 0);
08859     void setTickLengthIn(int inside);
08860     void setTickLengthOut(int outside);
08861     void setSubTicks(bool show);
08862     void setSubTickLength(int inside, int outside = 0);
08863     void setSubTickLengthIn(int inside);
08864     void setSubTickLengthOut(int outside);
08865     void setBasePen(const QPen &pen);
08866     void setTickPen(const QPen &pen);
08867     void setSubTickPen(const QPen &pen);
08868     void setLabelFont(const QFont &font);
08869     void setLabelColor(const QColor &color);
08870     void setLabel(const QString &str);
08871     void setLabelPadding(int padding);

```

```

08872     void setLabelPosition(Qt::AlignmentFlag position);
08873     void setSelectedTickLabelFont(const QFont &font);
08874     void setSelectedLabelFont(const QFont &font);
08875     void setSelectedTickLabelColor(const QColor &color);
08876     void setSelectedLabelColor(const QColor &color);
08877     void setSelectedBasePen(const QPen &pen);
08878     void setSelectedTickPen(const QPen &pen);
08879     void setSelectedSubTickPen(const QPen &pen);
08880     Q_SLOT void setSelectableParts(const QCPPolarAxisAngular::SelectableParts &selectableParts);
08881     Q_SLOT void setSelectedParts(const QCPPolarAxisAngular::SelectableParts &selectedParts);
08882
08883     // reimplemented virtual methods:
08884     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = 0) const
Q_DECL_OVERRIDE;
08885     virtual void update(UpdatePhase phase) Q_DECL_OVERRIDE;
08886     virtual QList<QCPLLayoutElement*> elements(bool recursive) const Q_DECL_OVERRIDE;
08887
08888     // non-property methods:
08889     bool removeGraph(QCPPolarGraph *graph);
08890     int radialAxisCount() const;
08891     QCPPolarAxisRadial *radialAxis(int index = 0) const;
08892     QList<QCPPolarAxisRadial*> radialAxes() const;
08893     QCPPolarAxisRadial *addRadialAxis(QCPPolarAxisRadial *axis = 0);
08894     bool removeRadialAxis(QCPPolarAxisRadial *axis);
08895
08896     QCPLLayoutInset *insetLayout() const {
08897         return mInsetLayout;
08898     }
08899
08900     QRegion exactClipRegion() const;
08901
08902     void moveRange(double diff);
08903     void scaleRange(double factor);
08904     void scaleRange(double factor, double center);
08905     void rescale(bool onlyVisiblePlottables = false);
08906
08907     double coordToAngleRad(double coord) const {
08908         return mAngleRad + (coord - mRange.lower) / mRange.size() * (mRangeReversed ? -2.0 * M_PI : 2.0
* M_PI);
08909     } // mention in doc that return doesn't wrap
08910
08911     double angleRadToCoord(double angleRad) const {
08912         return mRange.lower + (angleRad - mAngleRad) / (mRangeReversed ? -2.0 * M_PI : 2.0 * M_PI) *
mRange.size();
08913     }
08914     void pixelToCoord(QPointF pixelPos, double &angleCoord, double &radiusCoord) const;
08915     QPointF coordToPixel(double angleCoord, double radiusCoord) const;
08916     SelectablePart getPartAt(const QPointF &pos) const;
08917
08918     // read-only interface imitating a QRect:
08919     int left() const {
08920         return mRect.left();
08921     }
08922
08923     int right() const {
08924         return mRect.right();
08925     }
08926
08927     int top() const {
08928         return mRect.top();
08929     }
08930
08931     int bottom() const {
08932         return mRect.bottom();
08933     }
08934
08935     int width() const {
08936         return mRect.width();
08937     }
08938
08939     int height() const {
08940         return mRect.height();
08941     }
08942
08943     QSize size() const {
08944         return mRect.size();
08945     }
08946
08947     QPoint topLeft() const {
08948         return mRect.topLeft();
08949     }
08950
08951     QPoint topRight() const {
08952         return mRect.topRight();
08953     }
08954
08955     QPoint bottomLeft() const {

```



```

08956         return mRect.bottomLeft();
08957     }
08958
08959     QPoint bottomRight() const {
08960         return mRect.bottomRight();
08961     }
08962
08963     QPointF center() const {
08964         return mCenter;
08965     }
08966
08967     double radius() const {
08968         return mRadius;
08969     }
08970
08971 signals:
08972     void rangeChanged(const QCPRange &newRange);
08973     void rangeChanged(const QCPRange &newRange, const QCPRange &oldRange);
08974     void selectionChanged(const QCPPolarAxisAngular::SelectableParts &parts);
08975     void selectableChanged(const QCPPolarAxisAngular::SelectableParts &parts);
08976
08977 protected:
08978     // property members:
08979     QBrush mBackgroundBrush;
08980     QPixmap mBackgroundPixmap;
08981     QPixmap mScaledBackgroundPixmap;
08982     bool mBackgroundScaled;
08983     Qt::AspectRatioMode mBackgroundScaledMode;
08984     QCPLayoutInset *mInsetLayout;
08985     bool mRangeDrag;
08986     bool mRangeZoom;
08987     double mRangeZoomFactor;
08988
08989     // axis base:
08990     double mAngle, mAngleRad;
08991     SelectableParts mSelectableParts, mSelectedParts;
08992     QPen mBasePen, mSelectedBasePen;
08993     // axis label:
08994     int mLabelPadding;
08995     QString mLabel;
08996     QFont mLabelFont, mSelectedLabelFont;
08997     QColor mLabelColor, mSelectedLabelColor;
08998     // tick labels:
08999     //int mTickLabelPadding; in label painter
09000     bool mTickLabels;
09001     //double mTickLabelRotation; in label painter
09002     QFont mTickLabelFont, mSelectedTickLabelFont;
09003     QColor mTickLabelColor, mSelectedTickLabelColor;
09004     int mNumberPrecision;
09005     QLatin1Char mNumberFormatChar;
09006     bool mNumberBeautifulPowers;
09007     bool mNumberMultiplyCross;
09008     // ticks and subticks:
09009     bool mTicks;
09010     bool mSubTicks;
09011     int mTickLengthIn, mTickLengthOut, mSubTickLengthIn, mSubTickLengthOut;
09012     QPen mTickPen, mSelectedTickPen;
09013     QPen mSubTickPen, mSelectedSubTickPen;
09014     // scale and range:
09015     QCPRange mRange;
09016     bool mRangeReversed;
09017
09018     // non-property members:
09019     QPointF mCenter;
09020     double mRadius;
09021     QList<QCPPolarAxisRadial*> mRadialAxes;
09022     QCPPolarGrid *mGrid;
09023     QList<QCPPolarGraph*> mGraphs;
09024     QSharedPointer<QCPAxisTicker> mTicker;
09025     QVector<double> mTickVector;
09026     QVector<QString> mTickVectorLabels;
09027     QVector<QPointF> mTickVectorCosSin;
09028     QVector<double> mSubTickVector;
09029     QVector<QPointF> mSubTickVectorCosSin;
09030     bool mDragging;
09031     QCPRange mDragAngularStart;
09032     QList<QCPRange> mDragRadialStart;
09033     QCP::AntialiasedElements mADragBackup, mNotAADragBackup;
09034     QCPLabelPainterPrivate mLabelPainter;
09035
09036     // reimplemented virtual methods:
09037     virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
09038     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
09039     virtual QCP::Interaction selectionCategory() const Q_DECL_OVERRIDE;
09040     // events:
09041     virtual void mousePressEvent(QMouseEvent *event, const QVariant &details) Q_DECL_OVERRIDE;
09042     virtual void mouseMoveEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;

```

```

09043     virtual void mouseReleaseEvent(QMouseEvent *event, const QPointF &startPos) Q_DECL_OVERRIDE;
09044     virtual void wheelEvent(QWheelEvent *event) Q_DECL_OVERRIDE;
09045
09046     // non-virtual methods:
09047     bool registerPolarGraph(QCPPolarGraph *graph);
09048     void drawBackground(QCPPainter *painter, const QPointF &center, double radius);
09049     void setupTickVectors();
09050     QPen getBasePen() const;
09051     QPen getTickPen() const;
09052     QPen getSubTickPen() const;
09053     QFont getTickLabelFont() const;
09054     QFont getLabelFont() const;
09055     QColor getTickLabelColor() const;
09056     QColor getLabelColor() const;
09057
09058 private:
09059     Q_DISABLE_COPY(QCPPolarAxisAngular)
09060
09061     friend class QCustomPlot;
09062     friend class QCPPolarGrid;
09063     friend class QCPPolarGraph;
09064 };
09065
09066 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPPolarAxisAngular::SelectableParts)
09067 Q_DECLARE_METATYPE(QCPPolarAxisAngular::SelectablePart)
09068
09069 /* end of 'src/polar/layoutelement-angularaxis.h' */
09070
09071
09072 /* including file 'src/polar/polargrid.h' */
09073 /* modified 2021-03-29T02:30:44, size 4506 */
09074
09075 class QCP_LIB_DECL QCPPolarGrid : public QCPLayerable {
09076
09077     Q_OBJECT
09079
09081 public:
09085     enum GridType {
09086         gtAngular = 0x01
09087         , gtRadial = 0x02
09088         , gtAll = 0xFF
09089         , gtNone = 0x00
09090     };
09091     Q_ENUMS(GridType)
09092     Q_FLAGS(GridTypes)
09093     Q_DECLARE_FLAGS(GridTypes, GridType)
09094
09095     explicit QCPPolarGrid(QCPPolarAxisAngular *parentAxis);
09096
09097     // getters:
09098
09099     QCPPolarAxisRadial *radialAxis() const {
09100         return mRadialAxis.data();
09101     }
09102
09103     GridTypes type() const {
09104         return mType;
09105     }
09106
09107     GridTypes subGridType() const {
09108         return mSubGridType;
09109     }
09110
09111     bool antialiasedSubGrid() const {
09112         return mAntialiasedSubGrid;
09113     }
09114
09115     bool antialiasedZeroLine() const {
09116         return mAntialiasedZeroLine;
09117     }
09118
09119     QPen angularPen() const {
09120         return mAngularPen;
09121     }
09122
09123     QPen angularSubGridPen() const {
09124         return mAngularSubGridPen;
09125     }
09126
09127     QPen radialPen() const {
09128         return mRadialPen;
09129     }
09130
09131     QPen radialSubGridPen() const {
09132         return mRadialSubGridPen;
09133     }
09134

```

```

09135     QPen radialZeroLinePen() const {
09136         return mRadialZeroLinePen;
09137     }
09138
09139     // setters:
09140     void setRadialAxis(QCPPolarAxisRadial *axis);
09141     void setType(GridTypes type);
09142     void setSubGridType(GridTypes type);
09143     void setAntialiasedSubGrid(bool enabled);
09144     void setAntialiasedZeroLine(bool enabled);
09145     void setAngularPen(const QPen &pen);
09146     void setAngularSubGridPen(const QPen &pen);
09147     void setRadialPen(const QPen &pen);
09148     void setRadialSubGridPen(const QPen &pen);
09149     void setRadialZeroLinePen(const QPen &pen);
09150
09151 protected:
09152     // property members:
09153     GridTypes mType;
09154     GridTypes mSubGridType;
09155     bool mAntialiasedSubGrid, mAntialiasedZeroLine;
09156     QPen mAngularPen, mAngularSubGridPen;
09157     QPen mRadialPen, mRadialSubGridPen, mRadialZeroLinePen;
09158
09159     // non-property members:
09160     QCPPolarAxisAngular *mParentAxis;
09161     QPointer<QCPPolarAxisRadial> mRadialAxis;
09162
09163     // reimplemented virtual methods:
09164     virtual void applyDefaultAntialiasingHint(QCPPainter *painter) const Q_DECL_OVERRIDE;
09165     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
09166
09167     // non-virtual methods:
09168     void drawRadialGrid(QCPPainter *painter, const QPointF &center, const QVector<double> &coords,
09169         const QPen &pen, const QPen &zeroPen = Qt::NoPen);
09169     void drawAngularGrid(QCPPainter *painter, const QPointF &center, double radius, const
09170         QVector<QPointF> &ticksCosSin, const QPen &pen);
09171 private:
09172     Q_DISABLE_COPY(QCPPolarGrid)
09173 };
09174
09175
09176 Q_DECLARE_OPERATORS_FOR_FLAGS(QCPPolarGrid::GridTypes)
09177 Q_DECLARE_METATYPE(QCPPolarGrid::GridType)
09178
09179 /* end of 'src/polar/polargrid.h' */
09180
09181
09182 /* including file 'src/polar/polargraph.h' */
09183 /* modified 2021-03-29T02:30:44, size 9606 */
09184
09185
09186 class QCP_LIB_DECL QCPPolarLegendItem : public QCPAbstractLegendItem {
09187     Q_OBJECT
09188 public:
09189     QCPPolarLegendItem(QCPLegend *parent, QCPPolarGraph *graph);
09190
09191     // getters:
09192     QCPPolarGraph *polarGraph() {
09193         return mPolarGraph;
09194     }
09195
09196 protected:
09197     // property members:
09198     QCPPolarGraph *mPolarGraph;
09199
09200     // reimplemented virtual methods:
09201     virtual void draw(QCPPainter *painter) Q_DECL_OVERRIDE;
09202     virtual QSize minimumOuterSizeHint() const Q_DECL_OVERRIDE;
09203
09204     // non-virtual methods:
09205     QPen getIconBorderPen() const;
09206     QColor getTextColor() const;
09207     QFont getFont() const;
09208 };
09209
09210 class QCP_LIB_DECL QCPPolarGraph : public QCPLayerable {
09211     Q_OBJECT
09212 public:
09213     enum LineStyle {
09214         lsNone
09215         , lsLine

```

```

09228     };
09229     Q_ENUMS(LineStyle)
09230
09231     QCPPolarGraph(QCPPolarAxisAngular *keyAxis, QCPPolarAxisRadial *valueAxis);
09232     virtual ~QCPPolarGraph();
09233
09234     // getters:
09235
09236     QString name() const {
09237         return mName;
09238     }
09239
09240     bool antialiasedFill() const {
09241         return mAntialiasedFill;
09242     }
09243
09244     bool antialiasedScatters() const {
09245         return mAntialiasedScatters;
09246     }
09247
09248     QPen pen() const {
09249         return mPen;
09250     }
09251
09252     QBrush brush() const {
09253         return mBrush;
09254     }
09255
09256     bool periodic() const {
09257         return mPeriodic;
09258     }
09259
09260     QCPPolarAxisAngular *keyAxis() const {
09261         return mKeyAxis.data();
09262     }
09263
09264     QCPPolarAxisRadial *valueAxis() const {
09265         return mValueAxis.data();
09266     }
09267
09268     QCP::SelectionType selectable() const {
09269         return mSelectable;
09270     }
09271
09272     bool selected() const {
09273         return !mSelection.isEmpty();
09274     }
09275
09276     QCPDataSelection selection() const {
09277         return mSelection;
09278     }
09279     //QCPSelectionDecorator *selectionDecorator() const { return mSelectionDecorator; }
09280
09281     QSharedPointer<QCPGraphDataContainer> data() const {
09282         return mDataContainer;
09283     }
09284
09285     LineStyle lineStyle() const {
09286         return mLineStyle;
09287     }
09288
09289     QCPScatterStyle scatterStyle() const {
09290         return mScatterStyle;
09291     }
09292
09293     // setters:
09294     void setName(const QString &name);
09295     void setAntialiasedFill(bool enabled);
09296     void setAntialiasedScatters(bool enabled);
09297     void setPen(const QPen &pen);
09298     void setBrush(const QBrush &brush);
09299     void setPeriodic(bool enabled);
09300     void setKeyAxis(QCPPolarAxisAngular *axis);
09301     void setValueAxis(QCPPolarAxisRadial *axis);
09302     Q_SLOT void setSelectable(QCP::SelectionType selectable);
09303     Q_SLOT void setSelection(QCPDataSelection selection);
09304     //void setSelectionDecorator(QCPSelectionDecorator *decorator);
09305     void setData(QSharedPointer<QCPGraphDataContainer> data);
09306     void setData(const QVector<double> &keys, const QVector<double> &values, bool alreadySorted =
false);
09307     void setLineStyle(LineStyle ls);
09308     void setScatterStyle(const QCPScatterStyle &style);
09309
09310     // non-property methods:
09311     void addData(const QVector<double> &keys, const QVector<double> &values, bool alreadySorted =
false);
09312     void addData(double key, double value);

```

```

09313     void coordsToPixels(double key, double value, double &x, double &y) const;
09314     const QPointF coordsToPixels(double key, double value) const;
09315     void pixelsToCoords(double x, double y, double &key, double &value) const;
09316     void pixelsToCoords(const QPointF &pixelPos, double &key, double &value) const;
09317     void rescaleAxes(bool onlyEnlarge = false) const;
09318     void rescaleKeyAxis(bool onlyEnlarge = false) const;
09319     void rescaleValueAxis(bool onlyEnlarge = false, bool inKeyRange = false) const;
09320     bool addToLegend(QCPLegend *legend);
09321     bool addToLegend();
09322     bool removeFromLegend(QCPLegend *legend) const;
09323     bool removeFromLegend() const;
09324
09325     // introduced virtual methods:
09326     virtual double selectTest(const QPointF &pos, bool onlySelectable, QVariant *details = 0) const;
    // actually introduced in QCPLayerable as non-pure, but we want to force reimplementation for
    plottables
09327
09328     virtual QCPPlottableInterfaceD *interfaceD() {
09329         return 0;
09330     } // TODO: return this later, when QCPAbstractPolarPlottable is created
09331     virtual QCPRange getKeyRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth) const;
09332     virtual QCPRange getValueRange(bool &foundRange, QCP::SignDomain inSignDomain = QCP::sdBoth, const
    QCPRange &inKeyRange = QCPRange()) const;
09333
09334 signals:
09335     void selectionChanged(bool selected);
09336     void selectionChanged(const QCPDataSelection &selection);
09337     void selectableChanged(QCP::SelectionType selectable);
09338
09339 protected:
09340     // property members:
09341     QSharedPointer<QCPGraphDataContainer> mDataContainer;
09342     LineStyle mLineStyle;
09343     QCPScatterStyle mScatterStyle;
09344     QString mName;
09345     bool mAntialiasedFill, mAntialiasedScatters;
09346     QPen mPen;
09347     QBrush mBrush;
09348     bool mPeriodic;
09349     QPointer<QCPPolarAxisAngular> mKeyAxis;
09350     QPointer<QCPPolarAxisRadial> mValueAxis;
09351     QCP::SelectionType mSelectable;
09352     QCPDataSelection mSelection;
09353     //QCPSelectionDecorator *mSelectionDecorator;
09354
09355     // introduced virtual methods (later reimplemented TODO from QCPAbstractPolarPlottable):
09356     virtual QRect clipRect() const;
09357     virtual void draw(QCPPainter *painter);
09358     virtual QCP::Interaction selectionCategory() const;
09359     void applyDefaultAntialiasingHint(QCPPainter *painter) const;
09360     // events:
09361     virtual void selectEvent(QMouseEvent *event, bool additive, const QVariant &details, bool
    *selectionStateChanged);
09362     virtual void deselectEvent(bool *selectionStateChanged);
09363     // virtual drawing helpers:
09364     virtual void drawLinePlot(QCPPainter *painter, const QVector<QPointF> &lines) const;
09365     virtual void drawFill(QCPPainter *painter, QVector<QPointF> *lines) const;
09366     virtual void drawScatterPlot(QCPPainter *painter, const QVector<QPointF> &scatters, const
    QCPScatterStyle &style) const;
09367
09368     // introduced virtual methods:
09369     virtual void drawLegendIcon(QCPPainter *painter, const QRectF &rect) const;
09370
09371     // non-virtual methods:
09372     void applyFillAntialiasingHint(QCPPainter *painter) const;
09373     void applyScattersAntialiasingHint(QCPPainter *painter) const;
09374     double pointDistance(const QPointF &pixelPoint, QCPGraphDataContainer::const_iterator
    &closestData) const;
09375     // drawing helpers:
09376     virtual int dataCount() const;
09377     void getDataSegments(QList<QCPDataRange> &selectedSegments, QList<QCPDataRange>
    &unselectedSegments) const;
09378     void drawPolyline(QCPPainter *painter, const QVector<QPointF> &lineData) const;
09379     void setVisibleDataBounds(QCPGraphDataContainer::const_iterator &begin,
    QCPGraphDataContainer::const_iterator &end, const QCPDataRange &rangeRestriction) const;
09380     void getLines(QVector<QPointF> *lines, const QCPDataRange &dataRange) const;
09381     void getScatters(QVector<QPointF> *scatters, const QCPDataRange &dataRange) const;
09382     void getOptimizedLineData(QVector<QCPGraphData> *lineData, const
    QCPGraphDataContainer::const_iterator &begin, const QCPGraphDataContainer::const_iterator &end) const;
09383     void getOptimizedScatterData(QVector<QCPGraphData> *scatterData,
    QCPGraphDataContainer::const_iterator begin, QCPGraphDataContainer::const_iterator end) const;
09384     QVector<QPointF> dataToLines(const QVector<QCPGraphData> &data) const;
09385
09386 private:
09387     Q_DISABLE_COPY(QCPPolarGraph)
09388
09389     friend class QCPPolarLegendItem;

```

```

09390 };
09391
09392 /* end of 'src/polar/polargraph.h' */
09393
09394
09395 #endif // QCUSTOMPLOT_H
09396

```

11.86 CppModelExporter.h

```

00001 #ifndef CPPMODELEXPORTER_H
00002 #define CPPMODELEXPORTER_H
00003
00004 #include <string>
00005
00006 class QPlainTextEdit;
00007 class Simulator;
00008
00009 // Document the service that generates C++ model representation text.
00026 class CppModelExporter {
00027 public:
00029     CppModelExporter(Simulator* simulator, QPlainTextEdit* cppCodeEditor);
00030
00035     std::string addCppCodeLine(const std::string& line, unsigned int indent = 0) const;
00037     void actualizeModelCppCode() const;
00038
00039 private:
00040     Simulator* _simulator;
00041     QPlainTextEdit* _cppCodeEditor;
00042 };
00043
00044 #endif // CPPMODELEXPORTER_H

```

11.87 GraphicalAutomaticPositioningStrategy.h

```

00001 #ifndef GRAPHICALAUTOMATICPOSITIONINGSTRATEGY_H
00002 #define GRAPHICALAUTOMATICPOSITIONINGSTRATEGY_H
00003
00004 #include <QPointF>
00005 #include <QRectF>
00006 #include <QSizeF>
00007
00008 #include <memory>
00009
00010 class GraphicalAutomaticPositioningStrategy {
00011 public:
00019     virtual ~GraphicalAutomaticPositioningStrategy() = default;
00020
00022     virtual QPointF initialComponentLayoutOrigin() const = 0;
00023
00035     virtual QPointF dataDefinitionPosition(const QRectF& anchorBounds,
00036                                           const QSizeF& childSize,
00037                                           int index,
00038                                           int count,
00039                                           bool upperArc,
00040                                           int radialLayer) const = 0;
00041 };
00042
00046 class GraphicalAutomaticPositioningStrategyFactory {
00047 public:
00049     static std::unique_ptr<GraphicalAutomaticPositioningStrategy> createFromPreferences();
00050 };
00051
00052 #endif // GRAPHICALAUTOMATICPOSITIONINGSTRATEGY_H

```

11.88 GraphicalDataDefinitionLayout.h

```

00001 #ifndef GRAPHICALDATADEFINITIONLAYOUT_H
00002 #define GRAPHICALDATADEFINITIONLAYOUT_H
00003
00004 #include <QPointF>
00005 #include <QRectF>
00006 #include <QSizeF>
00007
00014 class GraphicalDataDefinitionLayout {

```

```

00015 public:
00027     static QPointF arcPosition(const QRectF& anchorBounds,
00028                               const QSizeF& childSize,
00029                               int index,
00030                               int count,
00031                               bool upperArc,
00032                               int radialLayer = 0);
00033
00034 private:
00035     GraphicalDataDefinitionLayout () = default;
00036 };
00037
00038 #endif // GRAPHICALDATADEFINITIONLAYOUT_H

```

11.89 GraphicalModelBuilder.h

```

00001 #ifndef GRAPHICALMODELBUILER_H
00002 #define GRAPHICALMODELBUILER_H
00003
00004 #include <map>
00005 #include <string>
00006
00007 #include <QColor>
00008
00009 class Simulator;
00010 class QTextEdit;
00011 class ModelGraphicsView;
00012 class ModelGraphicsScene;
00013 class ModelComponent;
00014 class GraphicalModelComponent;
00015 class ModelDataDefinition;
00016 class GraphicalModelDataDefinition;
00017 template<typename T> class List;
00018
00019 // Document the service that rebuilds graphical scene artifacts from kernel models.
00037 class GraphicalModelBuilder {
00038 public:
00040     GraphicalModelBuilder(Simulator* simulator,
00041                          ModelGraphicsView* graphicsView,
00042                          ModelGraphicsScene* scene,
00043                          std::map<std::string, QColor>* pluginCategoryColor,
00044                          QTextEdit* console);
00045
00052     void setModelLevelFilter(unsigned int modelLevel);
00054     void clearModelLevelFilter();
00055
00059     void recursivlyGenerateGraphicalModelFromModel(ModelComponent* component,
00060                                                    List<ModelComponent*>* visited,
00061                                                    std::map<ModelComponent*,
00062                                                    GraphicalModelComponent*>* map,
00063                                                    int* x,
00064                                                    int* y,
00065                                                    int* ymax,
00066                                                    int sequenceInLine);
00068     void generateGraphicalModelFromModel();
00070     static void synchronizeGraphicalDataDefinitionsLayer(Simulator* simulator, ModelGraphicsScene*
00071                                                         scene);
00072 private:
00073     void rebuildGraphicalDataDefinitionsLayer(std::map<ModelComponent*, GraphicalModelComponent*>*
00074                                               componentMap);
00075     bool componentBelongsToActiveModelLevel(ModelComponent* component) const;
00076     Simulator* _simulator;
00077     ModelGraphicsView* _graphicsView;
00078     ModelGraphicsScene* _scene;
00079     std::map<std::string, QColor>* _pluginCategoryColor;
00080     QTextEdit* _console;
00081 };
00082
00083 #endif // GRAPHICALMODELBUILER_H

```

11.90 GraphicalModelSerializer.h

```

00001 #ifndef GRAPHICALMODELSERIALIZER_H
00002 #define GRAPHICALMODELSERIALIZER_H
00003
00004 #include <functional>

```

```

00005 #include <string>
00006
00007 class Simulator;
00008 class Model;
00009 class QWidget;
00010 class QFile;
00011 class QString;
00012 class QPlainTextEdit;
00013 class QTextEdit;
00014 class QSlider;
00015 class QAction;
00016 class ModelGraphicsView;
00017
00018 // Document persistence responsibilities for textual and graphical GUI model data.
00036 class GraphicalModelSerializer {
00037 public:
00039     GraphicalModelSerializer(Simulator* simulator,
00040                             QWidget* ownerWidget,
00041                             QPlainTextEdit* modelTextEditor,
00042                             ModelGraphicsView* graphicsView,
00043                             QSlider* zoomSlider,
00044                             QAction* actionShowGrid,
00045                             QAction* actionShowRule,
00046                             QAction* actionShowSnap,
00047                             QAction* actionShowGuides,
00048                             QAction* actionShowInternalElements,
00049                             QAction* actionShowEditableElements,
00050                             QAction* actionShowAttachedElements,
00051                             QAction* actionShowRecursiveElements,
00052                             QTextEdit* console,
00053                             QString* modelFilename,
00054                             std::function<void()> clearModelEditors,
00055                             std::function<void()> rebuildGraphicalModelFromModel,
00056                             std::function<void()> applyShowInternalElements,
00057                             std::function<void()> applyShowEditableElements,
00058                             std::function<void()> applyShowAttachedElements);
00059
00064     bool saveTextModel(QFile* saveFile, const QString& data) const;
00069     bool saveGraphicalModel(const QString& filename) const;
00074     Model* loadGraphicalModel(const std::string& filename) const;
00075
00076 private:
00077     // Encode free text fields into a tab-safe persisted value.
00078     static QString encodeGuiText(const QString& text);
00079     // Decode tab-safe persisted text into original UTF-8 value.
00080     static QString decodeGuiText(const QString& text);
00081
00082 private:
00083     Simulator* _simulator;
00084     QWidget* _ownerWidget;
00085     QPlainTextEdit* _modelTextEditor;
00086     ModelGraphicsView* _graphicsView;
00087     QSlider* _zoomSlider;
00088     QAction* _actionShowGrid;
00089     QAction* _actionShowRule;
00090     QAction* _actionShowSnap;
00091     QAction* _actionShowGuides;
00092     QAction* _actionShowInternalElements;
00093     QAction* _actionShowEditableElements;
00094     QAction* _actionShowAttachedElements;
00095     QAction* _actionShowRecursiveElements;
00096     QTextEdit* _console;
00097     QString* _modelFilename;
00098     std::function<void()> _clearModelEditors;
00099     std::function<void()> _rebuildGraphicalModelFromModel;
00100     std::function<void()> _applyShowInternalElements;
00101     std::function<void()> _applyShowEditableElements;
00102     std::function<void()> _applyShowAttachedElements;
00103 };
00104
00105 #endif // GRAPHICALMODELSerializer_H

```

11.91 GraphvizModelExporter.h

```

00001 #ifndef GRAPHVIZMODELEXPORTER_H
00002 #define GRAPHVIZMODELEXPORTER_H
00003
00004 #include <functional>
00005 #include <list>
00006 #include <map>
00007 #include <string>
00008
00009 class QLabel;

```



```

00010 class QCheckBox;
00011 class Simulator;
00012 class ModelDataDefinition;
00013
00014 // Document the service used to export graphical model representations via Graphviz.
00032 class GraphvizModelExporter {
00033 public:
00037     GraphvizModelExporter(Simulator* simulator,
00038                           QLabel* modelGraphicLabel,
00039                           QCheckBox* showStatistics,
00040                           QCheckBox* showEditable,
00041                           QCheckBox* showShared,
00042                           QCheckBox* showRecursive,
00043                           QCheckBox* showLevels,
00044                           std::function<bool()> ensureModelSynchronized);
00045
00051     std::string adjustDotName(std::string name) const;
00052
00056     void insertTextInDot(std::string text,
00057                         unsigned int compLevel,
00058                         unsigned int compRank,
00059                         std::map<unsigned int, std::map<unsigned int, std::list<std::string>*>*>
00060                         dotmap,
00061                         bool isNode = false) const;
00064     void recursiveCreateModelGraphicPicture(ModelDataDefinition* componentOrData,
00065                                             std::list<ModelDataDefinition*>* visited,
00066                                             std::map<unsigned int, std::map<unsigned int,
00067                                             std::list<std::string>*>*> dotmap) const;
00072     bool createModelImage() const;
00073
00074 private:
00075     Simulator* _simulator;
00076     QLabel* _modelGraphicLabel;
00077     QCheckBox* _showStatistics;
00078     QCheckBox* _showEditable;
00079     QCheckBox* _showShared;
00080     QCheckBox* _showRecursive;
00081     QCheckBox* _showLevels;
00082     // Keep synchronization dependency narrow and explicit.
00083     std::function<bool()> _ensureModelSynchronized;
00084 };
00085
00086 #endif // GRAPHVIZMODELEXPORTER_H

```

11.92 IssueReportRelayService.h

```

00001 #ifndef ISSUEREPORTRELAYSERVICE_H
00002 #define ISSUEREPORTRELAYSERVICE_H
00003
00004 #include <QByteArray>
00005 #include <QString>
00006
00010 struct IssueReportDraft {
00011     QString categoryKey;
00012     QString categoryLabel;
00013     QString title;
00014     QString summary;
00015     QString reproductionSteps;
00016     QString expectedBehavior;
00017     QString observedBehavior;
00018     QString additionalContext;
00019     QString contactEmail;
00020     bool includeScreenshot = false;
00021     bool includeLogTail = true;
00022     bool includeModelText = false;
00023 };
00024
00028 struct IssueReportRuntimeContext {
00029     QString applicationName;
00030     QString applicationVersion;
00031     QString qtVersion;
00032     QString operatingSystem;
00033     QString currentModelName;
00034     QString currentModelSimulanguage;
00035     QString consoleTail;
00036     QString simulationTail;
00037     QString reportsTail;
00038     QString processLogTail;
00039     QByteArray screenshotPngBase64;
00040 };
00041
00045 struct IssueReportSubmissionResult {

```

```

00046     bool success = false;
00047     int httpStatus = 0;
00048     int issueNumber = 0;
00049     QString issueUrl;
00050     QString errorMessage;
00051     QString responseBody;
00052 };
00053
00067 class IssueReportRelayService {
00068 public:
00069     static QString configuredEndpoint();
00070     static QString optionalClientKey();
00071     static int requestTimeoutMs();
00072     static bool isEndpointConfigured(QString* reason = nullptr);
00073     static IssueReportSubmissionResult submit(const IssueReportDraft& draft,
00074                                              const IssueReportRuntimeContext& runtimeContext);
00075 };
00076
00077 #endif // ISSUEREPORTRELAYSERVICE_H

```

11.93 ModellLanguageSynchronizer.h

```

00001 #ifndef MODELLLANGUAGESYNCHRONIZER_H
00002 #define MODELLLANGUAGESYNCHRONIZER_H
00003
00004 #include <functional>
00005
00006 class QWidget;
00007 class QPlainTextEdit;
00008 class Simulator;
00009
00010 // Document the service boundary for textual model representation synchronization.
00028 class ModellLanguageSynchronizer {
00029 public:
00033     ModellLanguageSynchronizer(Simulator* simulator,
00034                               QPlainTextEdit* modelTextEditor,
00035                               bool* textModelHasChangedFlag,
00036                               QWidget* ownerWidget,
00037                               std::function<void()> onModelCreatedOrLoaded);
00038
00044     void actualizeModelSimLanguage() const;
00045
00050     bool setSimulationModelBasedOnText() const;
00051
00052 private:
00053     Simulator* _simulator;
00054     QPlainTextEdit* _modelTextEditor;
00055     bool* _textModelHasChangedFlag;
00056     QWidget* _ownerWidget;
00057     std::function<void()> _onModelCreatedOrLoaded;
00058 };
00059
00060 #endif // MODELLLANGUAGESYNCHRONIZER_H

```

11.94 ToolLauncher.h

```

00001 #pragma once
00002
00003 #include <QString>
00004 #include <QStringList>
00005
00006 class QWidget;
00007
00008 namespace ToolLauncher {
00009
00010 bool launchDetached(QWidget* parent,
00011                    const QString& executableBaseName,
00012                    const QString& displayName,
00013                    const QStringList& arguments = {},
00014                    const QStringList& relativeSearchDirs = {});
00015
00016 } // namespace ToolLauncher

```

11.95 systempreferences.h

```

00001 #ifndef SYSTEMPREFERENCES_H

```

```

00002 #define SYSTEMPREFERENCES_H
00003
00004 #include "kernel/simulator/TraceManager.h"
00005
00006 #include <QString>
00007 #include <string>
00008 #include <vector>
00009
00017 class SystemPreferences {
00018 public:
00022     enum class StartupModelMode : unsigned short {
00023         NoModel = 0,
00024         NewModel = 1,
00025         OpenSpecificModel = 2,
00026         OpenLastModel = 3
00027     };
00028
00032     enum class VisualTheme {
00033         Light,
00034         Dark
00035     };
00036
00040     enum class InterfaceStyle {
00041         Classic,
00042         Modern
00043     };
00044
00048     enum class AutomaticPositioningStrategy {
00049         Legacy = 0,
00050         Centered = 1
00051     };
00052
00054     static bool load();
00056     static bool save();
00058     static void resetToDefaults();
00060     static QString configFilePath();
00061
00063     static bool startMaximized();
00065     static void setStartMaximized(bool newStartMaximized);
00066
00068     static bool autoLoadPlugins();
00070     static void setAutoLoadPlugins(bool newAutoLoadPlugins);
00071
00073     static StartupModelMode startupModelMode();
00075     static void setStartupModelMode(StartupModelMode newStartupModelMode);
00077     static unsigned short modelAtStart();
00079     static void setModelAtStart(unsigned short newModelAtStart);
00080
00082     static std::string modelfilename();
00084     static void setModelfilename(const std::string &newModelfilename);
00086     static std::string lastModelFilename();
00088     static void setLastModelFilename(const std::string& newLastModelFilename);
00090     static std::vector<std::string> recentModelFiles();
00092     static void setRecentModelFiles(const std::vector<std::string>& recentModelFiles);
00094     static void pushRecentModelFile(const std::string& modelFilename);
00096     static unsigned int recentModelFilesLimit();
00098     static void setRecentModelFilesLimit(unsigned int recentModelFilesLimit);
00099
00101     static bool checkSystemPackagesAtStart();
00103     static void setCheckSystemPackagesAtStart(bool newCheckSystemPackagesAtStart);
00104
00106     static TraceManager::Level traceLevel();
00108     static void setTraceLevel(TraceManager::Level newTraceLevel);
00109
00111     static VisualTheme visualTheme();
00113     static void setVisualTheme(VisualTheme newVisualTheme);
00115     static std::string visualThemeName();
00116
00118     static InterfaceStyle interfaceStyle();
00120     static void setInterfaceStyle(InterfaceStyle newInterfaceStyle);
00122     static std::string interfaceStyleName();
00123
00125     static int applicationFontSize();
00127     static void setApplicationFontSize(int newApplicationFontSize);
00128
00130     static bool diagramUsesThemeColors();
00132     static void setDiagramUsesThemeColors(bool newDiagramUsesThemeColors);
00134     static AutomaticPositioningStrategy automaticPositioningStrategy();
00136     static void setAutomaticPositioningStrategy(AutomaticPositioningStrategy strategy);
00137
00139     static unsigned int canvasBackgroundColor();
00141     static unsigned int canvasDisabledBackgroundColor();
00143     static unsigned int gridColor();
00144
00145 private:
00146     SystemPreferences(){};
00147

```

```

00148     static QString defaultConfigDirectory();
00149     static StartupModelMode startupModelModeFromString(const QString& value);
00150     static QString startupModelModeToString(StartupModelMode value);
00151     static VisualTheme visualThemeFromString(const QString& value);
00152     static QString visualThemeToString(VisualTheme value);
00153     static InterfaceStyle interfaceStyleFromString(const QString& value);
00154     static QString interfaceStyleToString(InterfaceStyle value);
00155     static AutomaticPositioningStrategy automaticPositioningStrategyFromString(const QString& value);
00156     static QString automaticPositioningStrategyToString(AutomaticPositioningStrategy value);
00157     static TraceManager::Level traceLevelFromInt(int value);
00158
00159     static bool _startMaximized;
00160     static bool _autoLoadPlugins;
00161     static bool _checkSystemPackagesAtStart;
00162     static StartupModelMode _startupModelMode;
00163     static std::string _modelfilenameToOpen;
00164     static std::string _lastModelFilename;
00165     static std::vector<std::string> _recentModelFiles;
00166     static unsigned int _recentModelFilesLimit;
00167     static TraceManager::Level _traceLevel;
00168     static VisualTheme _visualTheme;
00169     static InterfaceStyle _interfaceStyle;
00170     static int _applicationFontPointSize;
00171     static bool _diagramUsesThemeColors;
00172     static AutomaticPositioningStrategy _automaticPositioningStrategy;
00173 };
00174
00175 #endif // SYSTEMPREFERENCES_H

```

11.96 DataAnalyzerWindow.h

```

00001 #ifndef DATAANALYZERWINDOW_H
00002 #define DATAANALYZERWINDOW_H
00003
00004 #include <QList>
00005 #include <QMainWindow>
00006 #include <QString>
00007 #include <QStringList>
00008
00009 class QGroupBox;
00010 class QCheckBox;
00011 class QComboBox;
00012 class QDoubleSpinBox;
00013 class QLabel;
00014 class QListWidget;
00015 class QPlainTextEdit;
00016 class QSpinBox;
00017 class QSplitter;
00018 class QTableWidget;
00019 class QTabWidget;
00020 class QTextEdit;
00021 class QTextBrowser;
00022 class Simulator;
00023
00024 class DataAnalyzerWindow : public QMainWindow {
00025 public:
00026     explicit DataAnalyzerWindow(Simulator* simulator, QString& lastDataAnalyzerPath, QWidget* parent =
00027         nullptr);
00028 private:
00029     struct DataSummary {
00030         int count = 0;
00031         double min = 0.0;
00032         double max = 0.0;
00033         double mean = 0.0;
00034         double median = 0.0;
00035         QString mode;
00036         double firstQuartile = 0.0;
00037         double thirdQuartile = 0.0;
00038         double variance = 0.0;
00039         double stddev = 0.0;
00040         double cv = 0.0;
00041         double kurtosis = 0.0;
00042     };
00043
00044     struct DatasetObservation {
00045         int replication = 1;
00046         double time = 0.0;
00047         bool hasTime = false;
00048         double value = 0.0;
00049     };
00050
00051     struct DatasetDescriptor {

```

```

00052     QString datasetName;
00053     QString randomVariableName;
00054     QString description;
00055     QString variableType;
00056     QString source;
00057     QStringList previewLines;
00058     QList<double> values;
00059     bool raw = true;
00060     QList<DatasetObservation> observations;
00061     bool recordFile = false;
00062     bool timeDependent = false;
00063     QString formatKind;
00064     QString expression;
00065     QString expressionName;
00066     QString formatVersion;
00067 };
00068
00069 class HistogramPreview;
00070 class DoePreviewPlot;
00071
00072 void buildMenus();
00073 void buildWorkspace();
00074 void connectActions();
00075 void refreshModelResponses();
00076 void openDataset();
00077 void importSimulationResponsesSnapshot();
00078 void loadDatasetFromFile(const QString& fileName);
00079 void addDataset(const DatasetDescriptor& dataset);
00080 void filterDataset();
00081 void refreshDatasetList();
00082 void refreshScopeSelector();
00083 void showAnalysisSetupDialog(const QString& analysisName, int targetTabIndex);
00084 void refreshAnalysisViews();
00085 QList<double> scopedValues() const;
00086 QList<DatasetObservation> scopedObservations() const;
00087 QString scopedDatasetLabel() const;
00088 QString scopedDatasetDescription() const;
00089 QStringList scopedPreviewLines() const;
00090 void refreshStudySummaryView();
00091 void refreshDescriptiveView(const DataSummary& summary);
00092 void refreshDistributionFitView(const DataSummary& summary);
00093 void refreshInferenceView(const DataSummary& summary);
00094 void refreshReportView(const DataSummary& summary, const QString& fitConclusion);
00095 void refreshDoeDemoViews();
00096 void refreshDoePlanSummary();
00097 void exportDataset();
00098 void saveReport();
00099 void showSkeletonMessage(const QString& featureName);
00100
00101 static DataSummary summarizeValues(const QList<double>& values);
00102 static QList<double> sortedValues(const QList<double>& values);
00103 static double percentile(const QList<double>& sorted, double probability);
00104 static QString exactMode(const QList<double>& values);
00105 static QString formatNumber(double value);
00106
00107 private:
00108     Simulator* _simulator = nullptr;
00109     QString& _lastDataAnalyzerPath;
00110     QString _studyName;
00111     QList<DatasetDescriptor> _datasets;
00112     QString _lastFitConclusion;
00113
00114     QAction* _openDatasetAction = nullptr;
00115     QAction* _saveReportAction = nullptr;
00116     QAction* _importSimulationResponsesAction = nullptr;
00117     QAction* _refreshModelResponsesAction = nullptr;
00118     QAction* _exportDatasetAction = nullptr;
00119
00120     QListWidget* _datasetsList = nullptr;
00121     QListWidget* _analysisNavigator = nullptr;
00122     QTableWidget* _modelResponsesTable = nullptr;
00123     QTabWidget* _workspaceTabs = nullptr;
00124     QComboBox* _scopeCombo = nullptr;
00125     QLabel* _datasetSourceLabel = nullptr;
00126     QPlainTextEdit* _datasetPreview = nullptr;
00127     QTableWidget* _studySummaryTable = nullptr;
00128     QTableWidget* _centralTendencyTable = nullptr;
00129     QTableWidget* _dispersionTable = nullptr;
00130     QTableWidget* _rawDataTable = nullptr;
00131     QTableWidget* _movingAverageTable = nullptr;
00132     HistogramPreview* _histogramPreview = nullptr;
00133     QTableWidget* _fitTable = nullptr;
00134     QLabel* _fitConclusionLabel = nullptr;
00135     QTableWidget* _inferenceTable = nullptr;
00136     QTableWidget* _doeFactorsTable = nullptr;
00137     QTableWidget* _doeResponsesTable = nullptr;
00138     QTableWidget* _doeDesignTable = nullptr;

```

```

00139     QTableWidgetItem* _doeAnovaTable = nullptr;
00140     QTableWidgetItem* _doeOptimizationTable = nullptr;
00141     QTableWidgetItem* _doeAdviceTable = nullptr;
00142     QTableWidgetItem* _doeDesignQualityTable = nullptr;
00143     QTableWidgetItem* _doeDesirabilityTable = nullptr;
00144     QComboBox* _doeDesignFamilyCombo = nullptr;
00145     QComboBox* _doeModelOrderCombo = nullptr;
00146     QComboBox* _doePlotSelector = nullptr;
00147     QSpinBox* _doeReplicationsSpin = nullptr;
00148     QDoubleSpinBox* _doeAlphaSpin = nullptr;
00149     QCheckBox* _doeRandomizeCheck = nullptr;
00150     QLabel* _doeAdvisorLabel = nullptr;
00151     QLabel* _doePlanSummaryLabel = nullptr;
00152     DoePreviewPlot* _doePreviewPlot = nullptr;
00153     QTextBrowser* _startPage = nullptr;
00154     QTextEdit* _reportText = nullptr;
00155 };
00156
00157 #endif // DATAANALYZERWINDOW_H

```

11.97 ExpressionBuilder.h

```

00001 #ifndef EXPRESSIONBUILDER_H
00002 #define EXPRESSIONBUILDER_H
00003
00004 #include <QDialog>
00005 #include <QTreeWidget>
00006 #include <QListWidget>
00007 #include <QFormLayout>
00008 #include <QLineEdit>
00009 #include <QPushButton>
00010 #include <QClipboard>
00011 #include <QApplication>
00012 #include <QVBoxLayout>
00013 #include <QHBoxLayout>
00014 #include <QLabel>
00015 #include <QString>
00016 #include <QMap>
00017 #include <QList>
00018
00019 namespace Ui {
00020 class ExpressionBuilder;
00021 }
00022
00023 struct ExpressionParameter {
00024     QString name;
00025     QString defaultValue;
00026     QString description;
00027 };
00028
00029 struct ExpressionTemplate {
00030     QString name;
00031     QString pattern;
00032     QString description;
00033     QList<ExpressionParameter> parameters;
00034 };
00035
00036 struct ExpressionCategory {
00037     QString name;
00038     QList<ExpressionTemplate> expressions;
00039 };
00040
00041 class ExpressionBuilder : public QDialog
00042 {
00043     Q_OBJECT
00044
00045 public:
00046     explicit ExpressionBuilder(QWidget *parent = nullptr);
00047     ~ExpressionBuilder();
00048
00049 private slots:
00050     void on_categorySelected(QTreeWidgetItem *item, int column);
00051     void on_expressionSelected(QListWidgetItem *item);
00052     void on_copyToClipboard_clicked();
00053     void on_close_clicked();
00054
00055 private:
00056     Ui::ExpressionBuilder *ui;
00057
00058     // Expression data
00059     QList<ExpressionCategory> categories;
00060
00061     // UI Components

```

```

00062     QTreeWidget *treeCategories;
00063     QListWidget *listExpressions;
00064     QWidget *parameterWidget;
00065     QFormLayout *parameterLayout;
00066     QLabel *expressionPreview;
00067     QPushButton *btnCopyToClipboard;
00068     QPushButton *btnClose;
00069
00070     // Parameter inputs
00071     QMap<QString, QLineEdit*> parameterInputs;
00072
00073     // Current selection
00074     ExpressionTemplate *currentExpression;
00075
00076     // Initialize expression categories
00077     void initializeCategories();
00078     void updateParameterForm();
00079     void updateExpressionPreview();
00080     QString buildExpression();
00081
00082     // Extensibility: Register new categories and expressions
00083     void registerCategory(const ExpressionCategory &category);
00084     void registerExpression(const QString &categoryName, const ExpressionTemplate &expression);
00085 };
00086
00087 #endif // EXPRESSIONBUILDER_H

```

11.98 OptimizerWindow.h

```

00001 #ifndef OPTIMIZERWINDOW_H
00002 #define OPTIMIZERWINDOW_H
00003
00004 #include "tools/Optimization/OptimizerDefaultImpl.h"
00005
00006 #include <QList>
00007 #include <QMainWindow>
00008 #include <QString>
00009 #include <QVector>
00010
00011 #include <vector>
00012
00013 class QAction;
00014 class QCheckBox;
00015 class QComboBox;
00016 class QDoubleSpinBox;
00017 class QLabel;
00018 class QPlainTextEdit;
00019 class QPushButton;
00020 class QSpinBox;
00021 class QTableWidget;
00022 class QTableWidgetItem;
00023 class QTabWidget;
00024 class QTextBrowser;
00025 class Simulator;
00026 class SimulationControl;
00027 class SimulationResponse;
00028
00029 class OptimizerWindow : public QMainWindow {
00030 public:
00031     explicit OptimizerWindow(Simulator* simulator, QWidget* parent = nullptr);
00032
00033 private:
00034     class ProgressPlot;
00035
00036     void buildMenus();
00037     void buildWorkspace();
00038     void connectActions();
00039     void refreshFromCurrentModel();
00040     void refreshControlsTable();
00041     void refreshResponsesTable();
00042     void refreshReport();
00043     void applyConfigurationToBackend();
00044     void refreshRunState();
00045     void addObjective();
00046     void removeSelectedObjective();
00047     void addConstraint();
00048     void removeSelectedConstraint();
00049     void configureTechnique();
00050     void checkReadiness();
00051     void startOptimization();
00052     void stepOptimization();
00053     void pauseOptimization();
00054     void resumeOptimization();

```

```

00055     void stopOptimization();
00056     void showSkeletonMessage(const QString& featureName);
00057
00058     static QString formatNumber(double value);
00059     static QString stateText(Optimizer_if::ExecutionState state);
00060     static QString techniqueDescription(const QString& technique);
00061     static bool stringIsNumeric(const QString& value);
00062     static QTableWidgetItem* readOnlyItem(const QString& text);
00063     static QTableWidgetItem* editableItem(const QString& text);
00064     static QTableWidgetItem* checkableItem(bool checked, bool enabled = true);
00065
00066 private:
00067     Simulator* _simulator = nullptr;
00068     OptimizerDefaultImpl1 _optimizer;
00069     std::vector<SimulationControl*> _controls;
00070     std::vector<SimulationResponse*> _responses;
00071     QVector<double> _progressValues;
00072
00073     QAction* _refreshModelAction = nullptr;
00074     QAction* _configureTechniqueAction = nullptr;
00075     QAction* _checkAction = nullptr;
00076     QAction* _startAction = nullptr;
00077     QAction* _stepAction = nullptr;
00078     QAction* _pauseAction = nullptr;
00079     QAction* _resumeAction = nullptr;
00080     QAction* _stopAction = nullptr;
00081
00082     QTabWidget* _tabs = nullptr;
00083     QLabel* _modelStatusLabel = nullptr;
00084     QLabel* _runStatusLabel = nullptr;
00085     QTableWidgetItem* _controlsTable = nullptr;
00086     QTableWidgetItem* _responsesTable = nullptr;
00087     QTableWidgetItem* _objectivesTable = nullptr;
00088     QTableWidgetItem* _constraintsTable = nullptr;
00089     QTableWidgetItem* _bestSolutionsTable = nullptr;
00090     QComboBox* _techniqueCombo = nullptr;
00091     QSpinBox* _maxIterationsSpin = nullptr;
00092     QSpinBox* _maxSimulationsSpin = nullptr;
00093     QSpinBox* _replicationsSpin = nullptr;
00094     QSpinBox* _bestSolutionsSpin = nullptr;
00095     QSpinBox* _randomSeedSpin = nullptr;
00096     QSpinBox* _populationSizeSpin = nullptr;
00097     QDoubleSpinBox* _mutationRateSpin = nullptr;
00098     QDoubleSpinBox* _crossoverRateSpin = nullptr;
00099     QDoubleSpinBox* _improvementToleranceSpin = nullptr;
00100     QDoubleSpinBox* _timeLimitSpin = nullptr;
00101     QPlainTextEdit* _techniqueNotes = nullptr;
00102     QTextBrowser* _overviewText = nullptr;
00103     QTextBrowser* _reportText = nullptr;
00104     ProgressPlot* _progressPlot = nullptr;
00105 };
00106
00107 #endif // OPTIMIZERWINDOW_H

```

11.99 TraitsGUI.h

```

00001 #ifndef TRAITSGUI_H
00002 #define TRAITSGUI_H
00003
00004 #include <stdint>
00005 #include <string>
00006
00013 template <class T>
00014 struct TraitsGUI {
00015 };
00016
00017 // include only features for this GUI, such as theme, default widgets, ...
00018
00019 class GMainWindow;
00020
00021 template <>
00022 struct TraitsGUI<GMainWindow> {
00023     static const bool startMaximized = true;
00024     static const int zoomButtonChange = 30;
00025     static const uint64_t consoleTextColor = 0x008000FF;
00026 };
00027
00028 class GModelGraphicPic;
00029
00030 template <>
00031 struct TraitsGUI<GModelGraphicPic> {
00032     static const int nodeComponentFontSize = 12;
00033     static const uint64_t nodeComponentFontColor = 0x000000FF;

```



```

00034     static const uint64_t nodeComponentFillColor = 0x80A040FF;
00035     static const int edgeComponentFontSize = 7;
00036     static const uint64_t edgeComponentEdgeColor = 0x000000FF;
00037     static const uint64_t edgeComponentFontColor = 0x000000FF;
00038     static const int nodeDatadefInternalFontSize = 8;
00039     static const uint64_t nodeDatadefInternalColor = 0xA0A0A0FF;
00040     static const uint64_t nodeDatadefInternalFontColor = 0xA0A0A0FF;
00041     static const int nodeDatadefAttachedFontSize = 8;
00042     static const uint64_t nodeDatadefAttachedFontColor = 0x808080FF;
00043 };
00044
00045 class GAssociation;
00046
00047 template <>
00048 struct TraitsGUI<GAssociation> {
00049     static const uint64_t foreColor = 0x808080FF;
00050     constexpr static const int margin = 2;
00051     constexpr static const int selectionWidth = 8;
00052 };
00053
00054 class GConnection;
00055
00056 template <>
00057 struct TraitsGUI<GConnection> {
00058     static const uint64_t foreColor = 0x000000FF;
00059     constexpr static const int margin = 2;
00060     constexpr static const int selectionWidth = 8;
00061 };
00062
00063 class GView;
00064
00065 template <>
00066 struct TraitsGUI<GView> {
00067     constexpr static const int sceneCenter = 25000;
00068     constexpr static const int sceneDistanceCenter = 25000;
00069     static const uint64_t backgroundEnabledColor = 0xFFFF8040;
00070     static const uint64_t backgroundDisabledColor = 0xC0C0C020;
00071 };
00072
00073 class GScene;
00074
00075 template <>
00076 struct TraitsGUI<GScene> {
00077     constexpr static const int gridInterval = 20;
00078     constexpr static const int gridPenWidth = 1;
00079     static const uint64_t gridColor = 0xC0C0C0E0;
00080 };
00081
00082 class GComponentPort;
00083
00084 template <>
00085 struct TraitsGUI<GComponentPort> {
00086     constexpr static const int width = 20;
00087     constexpr static const int penWidth = 1;
00088     constexpr static const int raise = 3;
00089     constexpr static const int margin = 2;
00090     static const uint64_t connectedBackgroundColor = 0x2020FF80;
00091     static const uint64_t unconnectedBackgroundColor = 0xFF202040;
00092     static const uint64_t borderColor = 0x000000FF;
00093     static const uint64_t pathRaised = 0x808080FF;
00094     static const uint64_t pathStunken = 0xFFFFFFFF;
00095     static const uint64_t textColor = 0xFFFFFFFF;
00096     static const uint64_t textShadowColor = 0x000000FF;
00097     static const uint64_t selectionSquaresColor = 0x000000FF;
00098 };
00099
00100 class GModelDataDefinition;
00101
00102 template <>
00103 struct TraitsGUI<GModelDataDefinition> {
00104     //<GraphicalModelComponent> {
00105     constexpr static const int opacity = 256 * 0.33;
00106     constexpr static const int nonEditableOpacity = 58;
00107     constexpr static const int selectedNonEditableOpacity = 72;
00108     constexpr static const int width = 100;
00109     constexpr static const int penWidth = 1;
00110     constexpr static const int raise = 3;
00111     constexpr static const int margin = 5;
00112     constexpr static const int selectionWidth = 8;
00113     constexpr static const float stretchPos = 0.5;
00114     constexpr static const float stretch = 0.0;
00115     constexpr static const float heightProportion = 0.50;
00116     static const uint64_t breakpointColor = 0xFF0000FF;
00117     static const uint64_t borderColor = 0x000000FF;
00118     static const uint64_t pathRaised = 0x808080FF;
00119     static const uint64_t pathStunken = 0xFFFFFFFF;
00120     static const uint64_t textColor = 0x0; // 0xFFFFFFFF;

```

```

00121     static const uint64_t textShadowColor = 0x000000FF;
00122     static const uint64_t selectionSquaresColor = 0x000000FF;
00123 };
00124
00125 class GModelComponent;
00126
00127 template <>
00128 struct TraitsGUI<GModelComponent> {
00129     //<GraphicalModelComponent> {
00130     constexpr static const int opacity = 256.0 * 0.67;
00131     constexpr static const int colorIncrease = 256.0 * 0.2;
00132     // 20% more some color component for sources, sinks, transfers
00133     constexpr static const int lighter = 133; // 33% lighter for some components
00134     constexpr static const int width = 150;
00135     constexpr static const int penWidth = 1;
00136     constexpr static const int raise = 5;
00137     constexpr static const int margin = 8;
00138     constexpr static const int selectionWidth = 8;
00139     constexpr static const float stretchPos = 0.5;
00140     constexpr static const float stretch = 0.0;
00141     constexpr static const float heightProportion = 0.67;
00142     static const uint64_t breakpointColor = 0xFF0000FF;
00143     static const uint64_t borderColor = 0x000000FF;
00144     static const uint64_t pathRaised = 0xFFFFFFFF;
00145     static const uint64_t pathStunken = 0x808080FF;
00146     static const uint64_t textColor = 0xFFFFFFFF;
00147     static const uint64_t textShadowColor = 0x000000FF;
00148     static const uint64_t selectionSquaresColor = 0x000000FF;
00149 };
00150
00151 #endif // TRAITSGUI_H

```

11.100 UtilGUI.h

```

00001 #ifndef UTILGUI_H
00002 #define UTILGUI_H
00003 #include <QColor>
00004 #include <cstdint>
00005 #include <sstream>
00006 #include <string>
00007
00008 namespace UtilGUI {
00009
00017 inline QColor rgbaFromPacked(uint64_t color) {
00018     uint8_t r = (color & 0xFF000000) >> 24;
00019     uint8_t g = (color & 0x00FF0000) >> 16;
00020     uint8_t b = (color & 0x0000FF00) >> 8;
00021     uint8_t a = (color & 0x000000FF);
00022     return QColor(r, g, b, a);
00023 }
00024
00032 inline std::string dotColorFromPacked(uint64_t color) {
00033     std::stringstream stream;
00034     stream << std::hex << "#" << color;
00035     return stream.str();
00036 }
00037
00038 }
00039
00040 #endif // UTILGUI_H

```

11.101 WebWorkerDialog.h

```

00001 #pragma once
00002
00003 #include <QDialog>
00004 #include <memory>
00005
00006 class QCloseEvent;
00007 class QShowEvent;
00008 class QPlainTextEdit;
00009 class QSpinBox;
00010 class QLabel;
00011 class WebWorkerRuntime;
00012
00016 class WebWorkerDialog : public QDialog {
00017     Q_OBJECT
00018
00019 public:

```

```

00020     explicit WebWorkerDialog(QWidget* parent = nullptr, bool allowClose = false);
00021     ~WebWorkerDialog() override;
00022
00023 protected:
00024     void closeEvent(QCloseEvent* event) override;
00025     void showEvent(QShowEvent* event) override;
00026
00027 private:
00028     void _refresh();
00029
00030     bool _allowClose = false;
00031     std::unique_ptr<WebWorkerRuntime> _runtime;
00032     QSpinBox* _portSpin = nullptr;
00033     QSpinBox* _maxRequestsSpin = nullptr;
00034     QLabel* _statusValue = nullptr;
00035     QLabel* _phaseValue = nullptr;
00036     QLabel* _urlValue = nullptr;
00037     QLabel* _servedValue = nullptr;
00038     QLabel* _activeValue = nullptr;
00039     QLabel* _stateValue = nullptr;
00040     QPlainTextEdit* _historyText = nullptr;
00041     QPlainTextEdit* _errorText = nullptr;
00042 };

```

11.102 Airport_Extended1.h

```

00001 #ifndef AIRPORT_EXTENDED1_H
00002 #define AIRPORT_EXTENDED1_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00006 class Airport_Extended1 : public BaseGenesysTerminalApplication {
00007 public:
00008     Airport_Extended1();
00009 public:
00010     virtual int main(int argc, char** argv) override;
00011 };
00012
00013 #endif // SMART_FAILURES_H

```

11.103 AirportSecurityExample.h

```

00001 #pragma once
00002
00003 #include "../BaseGenesysTerminalApplication.h"
00004
00005 class AirportSecurityExample : public BaseGenesysTerminalApplication {
00006 public:
00007     AirportSecurityExample();
00008     virtual int main(int argc, char** argv) override;
00009 };

```

11.104 AirportSecurityExampleExtended.h

```

00001 #pragma once
00002
00003 #include "../BaseGenesysTerminalApplication.h"
00004
00005 class AirportSecurityExampleExtended : public BaseGenesysTerminalApplication {
00006 public:
00007     AirportSecurityExampleExtended();
00008     virtual int main(int argc, char** argv) override;
00009 };

```

11.105 Assembly_Line.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */

```

```

00006
00007 /*
00008  * File:    Smart_Delay.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef ASSEMBLY_LINE_H
00015 #define ASSEMBLY_LINE_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Assembly_Line : public BaseGenesysTerminalApplication {
00020 public:
00021     Assembly_Line();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_DELAY_H */
00027

```

11.106 Banking_Transactions.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Smart_Dummy.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef BANKING_TRANSACTIONS_H
00015 #define BANKING_TRANSACTIONS_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Banking_Transactions : public BaseGenesysTerminalApplication {
00020 public:
00021     Banking_Transactions();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* BANKING_TRANSACTIONS_H */
00027

```

11.107 Example_BasicOrderShipping.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Example_PublicTransport_SARA.h
00009  * Author:  Bruno & Henrique
00010  *
00011  * Created 2022-2 DS3
00012  */
00013
00014 #ifndef EXAMPLE_BASIC_ORDER_SHIPPING_H
00015 #define EXAMPLE_BASIC_ORDER_SHIPPING_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Example_BasicOrderShipping : public BaseGenesysTerminalApplication {
00020 public:
00021     Example_BasicOrderShipping();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024

```

```

00025
00026 };
00027
00028 #endif /* EXAMPLE_BASIC_ORDER_SHIPPING_H */
00029

```

11.108 Example_PortModel.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007
00008 #ifndef EXAMPLE_PORTMODEL_H
00009 #define EXAMPLE_PORTMODEL_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Example_PortModel : public BaseGenesysTerminalApplication {
00014 public:
00015     Example_PortModel();
00016 public:
00017     virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* EXAMPLE_PORTMODEL_H */
00021

```

11.109 Example_PublicTransport.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Example_PublicTransport.h
00009  * Author:  Bruno & Henrique
00010  *
00011  * Created
00012  * */
00013
00014 #ifndef EXAMPLE_PUBLIC_TRANSPORT_H
00015 #define EXAMPLE_PUBLIC_TRANSPORT_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Example_PublicTransport : public BaseGenesysTerminalApplication {
00020 public:
00021     Example_PublicTransport();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024
00025
00026 };
00027
00028 #endif /* EXAMPLE_PUBLIC_TRANSPORT_H */
00029

```

11.110 Smart_AddingResource.h

```

00001 /*
00002  * File:    AddingResource.h
00003  * Author:  genesys
00004  *
00005  * Created on 10 de dezembro de 2022, 17:57
00006  */
00007
00008 #ifndef ADDINGRESOURCE_H
00009 #define ADDINGRESOURCE_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"

```

```

00012
00013 class Smart_AddingResource : public BaseGenesysTerminalApplication {
00014 public:
00015     Smart_AddingResource();
00016 public:
00017     virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* ADDINGRESOURCE_H */
00021

```

11.111 Smart_AlternatingEntityCreation.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_AlternatingEntityCreation.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_ALTERNATINGENTITYCREATION_H
00015 #define SMART_ALTERNATINGENTITYCREATION_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_AlternatingEntityCreation : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_AlternatingEntityCreation();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_ALTERNATINGENTITYCREATION_H */

```

11.112 Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime_H
00015 #define Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime : public BaseGenesysTerminalApplication
00020 {
00021 public:
00022     Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime();
00023 public:
00024     virtual int main(int argc, char** argv) override;
00025 };
00026 #endif /* Smart_ArrivalsElementStopsEntitiesArrivingAfterASetTime_H */

```

11.113 Smart_ArrivalsEntityTypeVsAttribute.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/file.h to edit this template

```

```

00004  */
00005
00006  /*
00007   * File:    Smart_ArrivalsEntityTypeVsAttribute.h
00008   * Author:  genesys
00009   *
00010   * Created on 16 de dezembro de 2022, 15:22
00011   */
00012
00013  #ifndef SMART_ARRIVALSENTITYTYPEVSATTRIBUTE_H
00014  #define SMART_ARRIVALSENTITYTYPEVSATTRIBUTE_H
00015
00016  #include "../BaseGenesysTerminalApplication.h"
00017
00018  class Smart_ArrivalsEntityTypeVsAttribute : public BaseGenesysTerminalApplication {
00019      public:
00020          Smart_ArrivalsEntityTypeVsAttribute();
00021          virtual int main(int argc, char** argv) override;
00022  };
00023
00024  #endif /* SMART_ARRIVALSENTITYTYPEVSATTRIBUTE_H */
00025

```

11.114 Smart_AssignExample.h

```

00001  /*
00002   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003   * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/file.h to edit this template
00004   */
00005
00006  /*
00007   * File:    Smart_AssignExample.h
00008   * Author:  genesys
00009   *
00010   * Created on 15 de dezembro de 2022, 23:46
00011   */
00012
00013  #ifndef SMART_ASSIGNEXAMPLE_H
00014  #define SMART_ASSIGNEXAMPLE_H
00015
00016  #include "../BaseGenesysTerminalApplication.h"
00017
00018  class Smart_AssignExample : public BaseGenesysTerminalApplication {
00019      public:
00020          Smart_AssignExample();
00021          virtual int main(int argc, char** argv) override;
00022  };
00023
00024
00025  #endif /* SMART_ASSIGNEXAMPLE_H */
00026

```

11.115 Smart_AutomaticStatisticsCollection.h

```

00001  /*
00002   * To change this license header, choose License Headers in Project Properties.
00003   * To change this template file, choose Tools | Templates
00004   * and open the template in the editor.
00005   */
00006
00007  /*
00008   * File:    Smart_AutomaticStatisticsCollection.h
00009   * Author:  rlcancian
00010   *
00011   * Created on 3 de Setembro de 2019, 18:34
00012   */
00013
00014  #ifndef Smart_AutomaticStatisticsCollection_H
00015  #define Smart_AutomaticStatisticsCollection_H
00016
00017  #include "../BaseGenesysTerminalApplication.h"
00018
00019  class Smart_AutomaticStatisticsCollection : public BaseGenesysTerminalApplication {
00020      public:
00021          Smart_AutomaticStatisticsCollection();
00022      public:
00023          virtual int main(int argc, char** argv) override;
00024  };
00025
00026  #endif /* Smart_AutomaticStatisticsCollection_H */

```

11.116 Smart_BasicModeling.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File:      BasicExample.h
00008  * Author:    genesys
00009  *
00010  * Created on 12 de dezembro de 2022, 12:38
00011  */
00012
00013 #ifndef SMART_BASICMODELING_H
00014 #define SMART_BASICMODELING_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_BasicModeling : public BaseGenesysTerminalApplication {
00019     public:
00020         Smart_BasicModeling();
00021         virtual int main(int argc, char** argv) override;
00022 };
00023
00024
00025 #endif /* SMART_BASICMODELING_H */
00026

```

11.117 Smart_BatchAndSeparate.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/file.h to edit this template
00004  */
00005
00006 /*
00007  * File:      Smart_BatchAndSeparate.h
00008  * Author:    genesys
00009  *
00010  * Created on 16 de dezembro de 2022, 00:26
00011  */
00012
00013 #ifndef SMART_BATCHHANDSEPARATE_H
00014 #define SMART_BATCHHANDSEPARATE_H
00015
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_BatchAndSeparate : public BaseGenesysTerminalApplication {
00020     public:
00021         Smart_BatchAndSeparate();
00022         virtual int main(int argc, char** argv) override;
00023 };
00024
00025 #endif /* SMART_BATCHHANDSEPARATE_H */
00026

```

11.118 Smart_ContinuousFlowEntities.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:      Smart_ModellingLearningCurve.h
00009  * Author:    rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_CONTINUOUSFLOWENTITIES_H
00015 #define SMART_CONTINUOUSFLOWENTITIES_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ContinuousFlowEntities : public BaseGenesysTerminalApplication {

```



```

00020 public:
00021     Smart_ContinuousFlowEntities();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_DUMMY_H */
00027

```

11.119 Smart_Create.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:     Smart_Create.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_CREATE_H
00015 #define SMART_CREATE_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Create : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_Create();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_CREATE_H */
00027

```

11.120 Smart_DecideNWayByChance.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:     Smart_DecideNWayByChance.h
00009  * Author:  guifrrs and LuizValdiero
00010  *
00011  * Created on December 3, 2022, 8:04 PM
00012  */
00013
00014 #ifndef SMART_DECIDENWAYBYCHANCE_H
00015 #define SMART_DECIDENWAYBYCHANCE_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_DecideNWayByChance : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_DecideNWayByChance();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_DECIDENWAYBYCHANCE_H */
00027

```

11.121 Smart_DefiningAttributesAsStrings.h

```

00001 /*
00002  * File:     Smart_MaxArrivalsField.h
00003  * Author:  genesys
00004  *

```

```

00005  * Created on 10 de dezembro de 2022, 15:34
00006  */
00007
00008 #ifndef SMART_DEFININGATTRIBUTESASSTRINGS_H
00009 #define SMART_DEFININGATTRIBUTESASSTRINGS_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_DefiningAttributesAsStrings: public BaseGenesysTerminalApplication {
00014 public:
00015     Smart_DefiningAttributesAsStrings();
00016 public:
00017     virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* SMART_DEFININGATTRIBUTESASSTRINGS_H */
00021

```

11.122 Smart_DefiningControlLogic.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_DefiningControlLogic.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef Smart_DEFININGCONTROLLOGIC_H
00015 #define Smart_DEFININGCONTROLLOGIC_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018 #include "plugins/components/DiscreteProcessing/Delay.h"
00019
00020
00021 class Smart_DefiningControlLogic : public BaseGenesysTerminalApplication {
00022 public:
00023     Smart_DefiningControlLogic();
00024 public:
00025     virtual int main(int argc, char** argv) override;
00026 };
00027
00028 #endif /* Smart_DEFININGCONTROLLOGIC_H */

```

11.123 Smart_DefiningResourceCapacity.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_DefiningResourceCapacity.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef Smart_DefiningResourceCapacity_H
00015 #define Smart_DefiningResourceCapacity_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_DefiningResourceCapacity : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_DefiningResourceCapacity();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* Smart_DefiningResourceCapacity_H */

```

11.124 Smart_DelayBasedOnReplication.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_DelayBasedOnReplication.h
00009  * Author: João Janini
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef Smart_DELAYBASEDONREPLICATION_H
00015 #define Smart_DELAYBASEDONREPLICATION_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018 #include "plugins/components/DiscreteProcessing/Delay.h"
00019
00020
00021 class Smart_DelayBasedOnReplication : public BaseGenesysTerminalApplication {
00022 public:
00023     Smart_DelayBasedOnReplication();
00024     void TestEventHandler(SimulationEvent *re);
00025 public:
00026     virtual int main(int argc, char** argv) override;
00027     Delay* delay;
00028     int *proctimes;
00029 };
00030
00031 #endif /* Smart_DelayBasedOnReplication_H */

```

11.125 Smart_EntitiesProcessedByPriority.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_EntitiesProcessedByPriority.h
00009  * Author: bepasar
00010  *
00011  * Created on 09/12/22
00012  */
00013
00014 #ifndef SMART_ENTITIESPROCESSEDBYPRIORITY_H
00015 #define SMART_ENTITIESPROCESSEDBYPRIORITY_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_EntitiesProcessedByPriority : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_EntitiesProcessedByPriority();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_ENTITIESPROCESSEDBYPRIORITY_H */
00027

```

11.126 Smart_EvaluatingConditionsBeforeEnteringQueue.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_EvaluatingConditionsBeforeEnteringQueue.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013

```

```

00014 #ifndef Smart_EvaluatingConditionsBeforeEnteringQueue_H
00015 #define Smart_EvaluatingConditionsBeforeEnteringQueue_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_EvaluatingConditionsBeforeEnteringQueue : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_EvaluatingConditionsBeforeEnteringQueue();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* Smart_EvaluatingConditionsBeforeEnteringQueue_H */

```

11.127 Smart_Expression.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_Expression.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_EXPRESSION_H
00015 #define SMART_EXPRESSION_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Expression : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_Expression();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_EXPRESSION_H */
00027

```

11.128 Smart_InventoryAndHoldingCosts.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_Dummy.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_INVENTORYANDHOLDINGCOSTS_H
00015 #define SMART_INVENTORYANDHOLDINGCOSTS_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_InventoryAndHoldingCosts : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_InventoryAndHoldingCosts();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_INVENTORYANDHOLDINGCOSTS_H */
00027

```

11.129 Smart_MaxArrivalsField.h

```

00001 /*

```

```

00002  * File:    Smart_MaxArrivalsField.h
00003  * Author:  genesys
00004  *
00005  * Created on 10 de dezembro de 2022, 15:34
00006  */
00007
00008 #ifndef SMART_MAXARRIVALSFIELD_H
00009 #define SMART_MAXARRIVALSFIELD_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_MaxArrivalsField : public BaseGenesysTerminalApplication {
00014 public:
00015     Smart_MaxArrivalsField();
00016 public:
00017     virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* SMART_MAXARRIVALSFIELD_H */
00021

```

11.130 Smart_ModelRunUntil1000Parts.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:    Smart_ModelRunUntil1000Parts.h
00009  * Author:  guifrrs and LuizValdiero
00010  *
00011  * Created on December 3, 2022, 8:04 PM
00012  */
00013
00014 #ifndef SMART_MODELRUNUNTIL1000PARTS_H
00015 #define SMART_MODELRUNUNTIL1000PARTS_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ModelRunUntil1000Parts : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_ModelRunUntil1000Parts();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_MODELRUNUNTIL1000PARTS_H */

```

11.131 Smart_ModuleDisplayVariables.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:    Smart_ModuleDisplayVariables.h
00009  * Author:  bepasar
00010  *
00011  * Created on 06/12/22, 21:27
00012  */
00013
00014 #ifndef SMART_MODULEDISPLAYVARIABLES_H
00015 #define SMART_MODULEDISPLAYVARIABLES_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ModuleDisplayVariables : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_ModuleDisplayVariables();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_MODULEDISPLAYVARIABLES_H */
00027

```

11.132 Smart_OverlappingResources.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007
00008 #ifndef SMART_OVERLAPPINGRESOURCES_H
00009 #define SMART_OVERLAPPINGRESOURCES_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_OverlappingResources : public BaseGenesysTerminalApplication {
00014 public:
00015     Smart_OverlappingResources();
00016 public:
00017     virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* SMART_OVERLAPPINGRESOURCES_H */
00021

```

11.133 Smart_ParallelProcessingOfEntities.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_ParallelProcessingOfEntities.h
00009  * Author: Luiz e Guilherme
00010  *
00011  * Created on Dezembro de 2022
00012  */
00013
00014 #ifndef SMART_PARALLEL_PROCESSING_OF_ENTITIES_H
00015 #define SMART_PARALLEL_PROCESSING_OF_ENTITIES_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ParallelProcessingOfEntities : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_ParallelProcessingOfEntities();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_PARALLEL_PROCESSING_OF_ENTITIES_H */

```

11.134 Smart_PlacingEntitiesInQueueSets.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_PlacingEntitiesInQueueSets.h
00009  * Author: Joçao Janini
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_PLACINGENTITIESINQUEUESETS_H
00015 #define SMART_PLACINGENTITIESINQUEUESETS_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_PlacingEntitiesInQueueSets : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_PlacingEntitiesInQueueSets();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_PLACINGENTITIESINQUEUESETS_H */

```

11.135 Smart_PriorityExample.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_Dummy.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_PRIORITYEXAMPLE_H
00015 #define SMART_PRIORITYEXAMPLE_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_PriorityExample : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_PriorityExample();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_DUMMY_H */
00027

```

11.136 Smart_ProcessArena.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_ProcessArena.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_PROCESSARENA_H
00015 #define SMART_PROCESSARENA_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ProcessArena : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_ProcessArena();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_PROCESSARENA_H */
00027

```

11.137 Smart_Record_Arena.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_Record_Arena.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef Smart_Record_Arena_ARENA_H
00015 #define Smart_Record_Arena_ARENA_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"

```

```

00018
00019 class Smart_Record_Arena : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_Record_Arena();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* Smart_Record_Arena_ARENA_H */
00027

```

11.138 Smart_RemovingAndReorderingEntitiesInAQueue.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007
00008 #ifndef SMART_REMOVINGANDREORDERINGENTITIESINAQUEUE_H
00009 #define SMART_REMOVINGANDREORDERINGENTITIESINAQUEUE_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_RemovingAndReorderingEntitiesInAQueue : public BaseGenesysTerminalApplication {
00014 public:
00015     Smart_RemovingAndReorderingEntitiesInAQueue();
00016 public:
00017     virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* SMART_REMOVINGANDREORDERINGENTITIESINAQUEUE_H */
00021

```

11.139 Smart_ResourceCosting.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_Dummy.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_RESOURCECOSTING_H
00015 #define SMART_RESOURCECOSTING_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ResourceCosting : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_ResourceCosting();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_RESOURCECOSTING_H */
00027

```

11.140 Smart_ResourceScheduleCosting.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_Dummy.h

```



```

00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_RESOURCESCHEDECOSTING_H
00015 #define SMART_RESOURCESCHEDECOSTING_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ResourceScheduleCosting : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_ResourceScheduleCosting();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_RESOURCESCHEDECOSTING_H */
00027

```

11.141 Smart_ResourceSets.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_ResourceSets.h
00009  * Author: guifrrs and LuizValdiero
00010  *
00011  * Created on December 3, 2022, 8:04 PM
00012  */
00013
00014 #ifndef SMART_RESOURCESETS_H
00015 #define SMART_RESOURCESETS_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ResourceSets : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_ResourceSets();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_RESOURCESETS_H */
00027

```

11.142 Smart_SeizingMultipleSimultaneosly.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_SeizingMultipleSimultaneosly.h
00009  * Author: douglaspereira04
00010  *
00011  */
00012
00013 #ifndef SMART_SEIZINGMULTIPLESIMULTANEOSLY_H
00014 #define SMART_SEIZINGMULTIPLESIMULTANEOSLY_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_SeizingMultipleSimultaneosly : public BaseGenesysTerminalApplication {
00019 public:
00020     Smart_SeizingMultipleSimultaneosly();
00021 public:
00022     virtual int main(int argc, char** argv) override;
00023 };
00024 #endif /* SMART_SEIZINGMULTIPLESIMULTANEOSLY_H */

```

11.143 Smart_SelectingRouteBasedOnProbability.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_WaitSignal.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_SELECTINGROUTEBASEDONPROBABILITY_H
00015 #define SMART_SELECTINGROUTEBASEDONPROBABILITY_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_SelectingRouteBasedOnProbability : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_SelectingRouteBasedOnProbability();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_SELECTINGROUTEBASEDONPROBABILITY_H */
00027

```

11.144 Smart_SelectingShorterQueue.h

```

00001 /*
00002  * File: Smart_SelectingShorterQueue.h
00003  * Author: genesys
00004  *
00005  * Created on 10 de dezembro de 2022, 17:59
00006  */
00007
00008 #ifndef SMART_SELECTINGSHORTERQUEUE_H
00009 #define SMART_SELECTINGSHORTERQUEUE_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_SelectingShorterQueue : public BaseGenesysTerminalApplication {
00014 public:
00015     Smart_SelectingShorterQueue();
00016 public:
00017     virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* SMART_SELECTINGSHORTERQUEUE_H */
00021

```

11.145 Smart_SynchronizingParallelEntities.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_SynchronizingParallelEntities.h
00009  * Author:
00010  *
00011  */
00012
00013 #ifndef SMART_SYNCHRONIZINGPARALLELENTITIES_H
00014 #define SMART_SYNCHRONIZINGPARALLELENTITIES_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_SynchronizingParallelEntities : public BaseGenesysTerminalApplication {
00019 public:
00020     Smart_SynchronizingParallelEntities();
00021 public:
00022     virtual int main(int argc, char** argv) override;
00023 };
00024 #endif /* SMART_SYNCHRONIZINGPARALLELENTITIES_H */
00025

```

11.146 Smart_ValueAdded.h

```
00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_Dummy.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_VALUEADDED_H
00015 #define SMART_VALUEADDED_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ValueAdded : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_ValueAdded();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_VALUEADDED_H */
00027
```

11.147 Smart_WaitForSignal.h

```
00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_WaitForSignal.h
00009  * Author: douglaspereira04
00010  *
00011  */
00012
00013 #ifndef SMART_WAITFORSIGNAL_H
00014 #define SMART_WAITFORSIGNAL_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_WaitForSignal : public BaseGenesysTerminalApplication {
00019 public:
00020     Smart_WaitForSignal();
00021 public:
00022     virtual int main(int argc, char** argv) override;
00023 };
00024 #endif /* SMART_WAITFORSIGNAL_H */
```

11.148 Book_Cap02_Example01.h

```
00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File: Book_Cap02_Example01.h
00008  * Author: rlcancian
00009  *
00010  * Created on 19 de outubro de 2021, 16:23
00011  */
00012
00013 #ifndef BOOK_CAP02_EXAMPLE01_H
00014 #define BOOK_CAP02_EXAMPLE01_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Book_Cap02_Example01 : public BaseGenesysTerminalApplication {
00019 public:
00020     Book_Cap02_Example01();
```

```
00021     virtual int main(int argc, char** argv) override;
00022 };
00023
00024 #endif /* BOOK_CAP02_EXAMPLE01_H */
00025
```

11.149 StartAIAssistant.h

```
00001 #pragma once
00002
00003 #include "../BaseGenesysTerminalApplication.h"
00004
00005 class StartAIAssistant : public BaseGenesysTerminalApplication {
00006 public:
00007     StartAIAssistant();
00008 public:
00009     virtual int main(int argc, char** argv) override;
00010 };
```

11.150 Smart_MarkovChain.h

```
00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_MarkovChain.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_MARKOVCHAIN_H
00015 #define SMART_MARKOVCHAIN_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_MarkovChain : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_MarkovChain();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_MARKOVCHAIN_H */
00027
```

11.151 Smart_BacteriaColony_GRO.h

```
00001 /*
00002  * File: Smart_BacteriaColony_GRO.h
00003  * Author: GRO
00004  *
00005  * Created on 1 de Maio de 2026
00006  */
00007
00008 #ifndef SMART_BACTERIACOLONY_GRO_H
00009 #define SMART_BACTERIACOLONY_GRO_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_BacteriaColony_GRO : public BaseGenesysTerminalApplication {
00014 public:
00015     Smart_BacteriaColony_GRO();
00016
00017 public:
00018     virtual int main(int argc, char** argv) override;
00019 };
00020
00021 #endif /* SMART_BACTERIACOLONY_GRO_H */
```

11.152 Smart_BioKineticLawRegulation.h

```

00001  /*
00002  * File:    Smart_BioKineticLawRegulation.h
00003  */
00004
00005  #ifndef SMART_BIOKINETICLAWREGULATION_H
00006  #define SMART_BIOKINETICLAWREGULATION_H
00007
00008  #include "../BaseGenesysTerminalApplication.h"
00009
00010  class Smart_BioKineticLawRegulation : public BaseGenesysTerminalApplication {
00011  public:
00012      Smart_BioKineticLawRegulation();
00013  public:
00014      virtual int main(int argc, char** argv) override;
00015  };
00016
00017  #endif /* SMART_BIOKINETICLAWREGULATION_H */

```

11.153 Smart_BioNetworkAnalysis.h

```

00001  /*
00002  * File:    Smart_BioNetworkAnalysis.h
00003  */
00004
00005  #ifndef SMART_BIONETWORKANALYSIS_H
00006  #define SMART_BIONETWORKANALYSIS_H
00007
00008  #include "../BaseGenesysTerminalApplication.h"
00009
00010  class Smart_BioNetworkAnalysis : public BaseGenesysTerminalApplication {
00011  public:
00012      Smart_BioNetworkAnalysis();
00013  public:
00014      virtual int main(int argc, char** argv) override;
00015  };
00016
00017  #endif /* SMART_BIONETWORKANALYSIS_H */

```

11.154 Smart_BioReversibleKineticLaw.h

```

00001  /*
00002  * File:    Smart_BioReversibleKineticLaw.h
00003  */
00004
00005  #ifndef SMART_BIOREVERSIBLEKINETICLAW_H
00006  #define SMART_BIOREVERSIBLEKINETICLAW_H
00007
00008  #include "../BaseGenesysTerminalApplication.h"
00009
00010  class Smart_BioReversibleKineticLaw : public BaseGenesysTerminalApplication {
00011  public:
00012      Smart_BioReversibleKineticLaw();
00013  public:
00014      virtual int main(int argc, char** argv) override;
00015  };
00016
00017  #endif /* SMART_BIOREVERSIBLEKINETICLAW_H */

```

11.155 Smart_BioReversibleMassAction.h

```

00001  /*
00002  * File:    Smart_BioReversibleMassAction.h
00003  */
00004
00005  #ifndef SMART_BIOREVERSIBLEMASSACTION_H
00006  #define SMART_BIOREVERSIBLEMASSACTION_H
00007
00008  #include "../BaseGenesysTerminalApplication.h"
00009
00010  class Smart_BioReversibleMassAction : public BaseGenesysTerminalApplication {
00011  public:
00012      Smart_BioReversibleMassAction();
00013  public:

```

```

00014     virtual int main(int argc, char** argv) override;
00015 };
00016
00017 #endif /* SMART_BIOREVERSIBLEMASSACTION_H */

```

11.156 Smart_GroColonyGrowth.h

```

00001 /*
00002  * File:    Smart_GroColonyGrowth.h
00003  * Author:  GRO
00004  *
00005  * Created on 17 de Abril de 2026
00006  */
00007
00008 #ifndef SMART_GROCOLONYGROWTH_H
00009 #define SMART_GROCOLONYGROWTH_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_GroColonyGrowth : public BaseGenesysTerminalApplication {
00014 public:
00015     Smart_GroColonyGrowth();
00016
00017 public:
00018     virtual int main(int argc, char** argv) override;
00019 };
00020
00021 #endif /* SMART_GROCOLONYGROWTH_H */

```

11.157 Smart_GroColonyLifecycle.h

```

00001 /*
00002  * File:    Smart_GroColonyLifecycle.h
00003  * Author:  GRO
00004  *
00005  * Created on 17 de Abril de 2026
00006  */
00007
00008 #ifndef SMART_GROCOLONYLIFECYCLE_H
00009 #define SMART_GROCOLONYLIFECYCLE_H
00010
00011 #include "../BaseGenesysTerminalApplication.h"
00012
00013 class Smart_GroColonyLifecycle : public BaseGenesysTerminalApplication {
00014 public:
00015     Smart_GroColonyLifecycle();
00016
00017 public:
00018     virtual int main(int argc, char** argv) override;
00019 };
00020
00021 #endif /* SMART_GROCOLONYLIFECYCLE_H */

```

11.158 Smart_LSODE.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File:    Smart_ODE.h
00008  * Author:  rlcancian
00009  *
00010  * Created on 11 de janeiro de 2022, 19:29
00011  */
00012
00013 #ifndef SMART_LSODE_H
00014 #define SMART_LSODE_H
00015
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_LSODE : public BaseGenesysTerminalApplication {
00020 public:

```

```

00021     Smart_LCODE();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_LCODE_H */
00027

```

11.159 Smart_ODE.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File:     Smart_ODE.h
00008  * Author:  rlcancian
00009  *
00010  * Created on 11 de janeiro de 2022, 19:29
00011  */
00012
00013 #ifndef SMART_ODE_H
00014 #define SMART_ODE_H
00015
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ODE : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_ODE();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_ODE_H */
00027

```

11.160 Smart_Buffer.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:     Smart_Buffer.h
00009  * Author:  rlcancian
00010  *
00011  * Created on
00012  */
00013
00014 #ifndef SMART_BUFFER_H
00015 #define SMART_BUFFER_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Buffer : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_Buffer();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_BUFFER_H */
00027

```

11.161 Smart_Clone.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */

```

```

00006
00007 /*
00008  * File:    Smart_Clone.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 30 de Novembro de 2022, 19:25
00012  */
00013
00014 #ifndef SMART_CLONE_H
00015 #define SMART_CLONE_H
00016
00017 #include "../.../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Clone : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_Clone();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_CLONE_H */
00027

```

11.162 Smart_Delay.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Smart_Delay.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_DELAY_H
00015 #define SMART_DELAY_H
00016
00017 #include "../.../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Delay : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_Delay();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_DELAY_H */
00027

```

11.163 Smart_Failures.h

```

00001 #ifndef SMART_FAILURES_H
00002 #define SMART_FAILURES_H
00003
00004 #include "../.../BaseGenesysTerminalApplication.h"
00005
00006 class Smart_Failures : public BaseGenesysTerminalApplication {
00007 public:
00008     Smart_Failures();
00009 public:
00010     virtual int main(int argc, char** argv) override;
00011 };
00012
00013 #endif // SMART_FAILURES_H

```

11.164 Smart_HoldSearchRemove.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.

```



```

00005  */
00006
00007  /*
00008  * File:    Smart_HoldSearchRemove.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 01 de Dezembro de 2022, 09:31
00012  */
00013
00014 #ifndef SMART_HOLDSEARCHREMOVE_H
00015 #define SMART_HOLDSEARCHREMOVE_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_HoldSearchRemove : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_HoldSearchRemove();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_HOLDSEARCHREMOVE_H */
00027

```

11.165 Smart_Process.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:    Smart_Process.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_PROCESS_H
00015 #define SMART_PROCESS_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Process : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_Process();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_PROCESS_H */
00027

```

11.166 Smart_ProcessSet.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:    Smart_ProcessSet.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_PROCESSSET_H
00015 #define SMART_PROCESSSET_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ProcessSet : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_ProcessSet();
00022 public:
00023     virtual int main(int argc, char** argv) override;

```

```

00024 };
00025
00026 #endif /* SMART_PROCESSET_H */
00027

```

11.167 Smart_SeizeDelayRelease.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_SeizeDelayRelease.h
00009  * Author: rlcancian
00010  *
00011  * Created on 24 de Setembro de 2019, 16:43
00012  */
00013
00014 #ifndef SMART_SEIZEDELAYRELEASE_H
00015 #define SMART_SEIZEDELAYRELEASE_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_SeizeDelayRelease : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_SeizeDelayRelease();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025 #endif /* SMART_SEIZEDELAYRELEASE_H */
00026

```

11.168 Smart_SeizeDelayReleaseMany.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_SeizeDelayReleaseMany.h
00009  * Author: rlcancian
00010  *
00011  * Created on 9 de abril de 2021, 18:09
00012  */
00013
00014 #ifndef MODEL_SEIZEDELAYRELEASEMANY_H
00015 #define MODEL_SEIZEDELAYRELEASEMANY_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_SeizeDelayReleaseMany : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_SeizeDelayReleaseMany();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* MODEL_SEIZEDELAYRELEASEMANY_H */
00027

```

11.169 Smart_SeizeDelayReleaseNoDataDefs.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File: Smart_SeizeDelayReleaseNoDataDefs.h
00008  * Author: rlcancian
00009  *

```

```

00010  * Created on 23 de maio de 2022, 12:39
00011  */
00012
00013 #ifndef SMART_SEIZEDELAYRELEASENODATADEFS_H
00014 #define SMART_SEIZEDELAYRELEASENODATADEFS_H
00015
00016 #include ".././././BaseGenesysTerminalApplication.h"
00017
00018 class Smart_SeizeDelayReleaseNoDataDefs : public BaseGenesysTerminalApplication {
00019 public:
00020     Smart_SeizeDelayReleaseNoDataDefs();
00021 public:
00022     virtual int main(int argc, char** argv) override;
00023 };
00024
00025 #endif /* SMART_SEIZEDELAYRELEASENODATADEFS_H */
00026

```

11.170 Smart_CppForG.h

```

00001  /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006  /*
00007  * File: Smart_CppForG.h
00008  * Author: rlcancian
00009  *
00010  * Created on 11 de janeiro de 2022, 22:39
00011  */
00012
00013 #pragma once
00014
00015 #include ".././././BaseGenesysTerminalApplication.h"
00016
00017 class Smart_CppForG : public BaseGenesysTerminalApplication {
00018 public:
00019     Smart_CppForG();
00020 public:
00021     virtual int main(int argc, char** argv) override;
00022 };
00023

```

11.171 Smart_PythonForG.h

```

00001 #ifndef SMART_PYTHONFORG_H
00002 #define SMART_PYTHONFORG_H
00003
00004 #include ".././././BaseGenesysTerminalApplication.h"
00005
00006 class Smart_PythonForG : public BaseGenesysTerminalApplication {
00007 public:
00008     Smart_PythonForG();
00009     virtual int main(int argc, char** argv) override;
00010 };
00011
00012 #endif /* SMART_PYTHONFORG_H */

```

11.172 Smart_R_Simulator.h

```

00001  /*
00002  * File: Smart_R_Simulator.h
00003  * Author: GenESyS
00004  *
00005  * Minimal smart model for the RSimulator component.
00006  */
00007
00008 #ifndef SMART_R_SIMULATOR_H
00009 #define SMART_R_SIMULATOR_H
00010
00011 #include ".././././BaseGenesysTerminalApplication.h"
00012
00013 class Smart_R_Simulator : public BaseGenesysTerminalApplication {
00014 public:

```

```

00022     Smart_R_Simulator();
00023
00024 public:
00026     virtual int main(int argc, char** argv) override;
00027 };
00028
00029 #endif /* SMART_R_SIMULATOR_H */

```

11.173 Smart_BatchSeparate.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File:     Smart_BatchSeparate.h
00008  * Author:    rlcancian
00009  *
00010  * Created on 15 de janeiro de 2022, 09:51
00011  */
00012
00013 #ifndef SMART_BATCHSEPARATE_H
00014 #define SMART_BATCHSEPARATE_H
00015
00016 #include "../.../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_BatchSeparate : public BaseGenesysTerminalApplication {
00019 public:
00020     Smart_BatchSeparate();
00021 public:
00022     virtual int main(int argc, char** argv) override;
00023 };
00024
00025 #endif /* SMART_BATCHSEPARATE_H */
00026

```

11.174 Smart_Record.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:     Smart_Record.h
00009  * Author:    rlcancian
00010  *
00011  * Created on
00012  */
00013
00014 #pragma once
00015
00016 #include "../.../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_Record : public BaseGenesysTerminalApplication {
00019 public:
00020     Smart_Record();
00021 public:
00022     virtual int main(int argc, char** argv) override;
00023 };
00024
00025

```

11.175 Smart_AssignWriteSeizes.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:     Smart_AssignWriteSeizes.h

```

```

00009  * Author: rlcancian
00010  *
00011  * Created on 24 de Setembro de 2019, 20:56
00012  */
00013
00014 #ifndef SMART_ASSIGNWRITE3SEIZES_H
00015 #define SMART_ASSIGNWRITE3SEIZES_H
00016
00017 #include "../.../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_AssignWriteSeizes : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_AssignWriteSeizes();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_ASSIGNWRITE3SEIZES_H */
00027

```

11.176 Smart_Attribute_Variable.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_Attribute_Variable.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_ATTRIBUTE_VARIABLE_H
00015 #define SMART_ATTRIBUTE_VARIABLE_H
00016
00017 #include "../.../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Attribute_Variable : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_Attribute_Variable();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_ATTRIBUTE_VARIABLE_H */

```

11.177 Smart_ModelInfoModelSimulation.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_ModelInfoModelSimulation.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:59
00012  */
00013
00014 #ifndef SMART_MODELINFOMODELSIMULATION_H
00015 #define SMART_MODELINFOMODELSIMULATION_H
00016
00017 #include "../.../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ModelInfoModelSimulation : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_ModelInfoModelSimulation();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_MODELINFOMODELSIMULATION_H */
00027

```

11.178 Smart_Parser.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_Parser.h
00009  * Author: rlcancian
00010  *
00011  * Created on 5 de Janeiro de 2022, 07:21
00012  */
00013
00014 #ifndef SMART_PARSER_H
00015 #define SMART_PARSER_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Parser : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_Parser();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_PARSER_H */
00027

```

11.179 Smart_ParserModelFunctions.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_ParserModelFunctions.h
00009  * Author: rlcancian
00010  *
00011  * Created on 5 de Janeiro de 2022, 07:21
00012  */
00013
00014 #ifndef SMART_PARSERMODELFUNCTIONS_H
00015 #define SMART_PARSERMODELFUNCTIONS_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ParserModelFunctions : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_ParserModelFunctions();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_PARSERMODELFUNCTIONS_H */
00027

```

11.180 Smart_Plugin.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File: Smart_Plugin.h
00008  * Author: rlcancian
00009  *
00010  * Created on 13 de janeiro de 2022, 17:47
00011  */
00012
00013 #ifndef SMART_PLUGIN_H
00014 #define SMART_PLUGIN_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017

```

```

00018 class Smart_Plugin : public BaseGenesysTerminalApplication {
00019 public:
00020     Smart_Plugin();
00021 public:
00022     virtual int main(int argc, char** argv) override;
00023 };
00024
00025 #endif /* SMART_PLUGIN_H */
00026

```

11.181 Smart_RouteStation.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:     Smart_RouteStation.h
00009  * Author:   rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_ROUTESTATION_H
00015 #define SMART_ROUTESTATION_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_RouteStation : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_RouteStation();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_ROUTESTATION_H */
00027

```

11.182 Smart_Sequence.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:     Smart_Sequence.h
00009  * Author:   rlcancian
00010  *
00011  * Created on 12 de março de 2021, 15:39
00012  */
00013
00014 #ifndef SMART_SEQUENCE_H
00015 #define SMART_SEQUENCE_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Sequence : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_Sequence();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_SEQUENCE_H */
00027

```

11.183 Smart_CellularAutomata.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates

```

```

00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:    Smart_CellularAutomata.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014  #ifndef SMART_CELLULARAUTOMATA_H
00015  #define SMART_CELLULARAUTOMATA_H
00016
00017  #include "../BaseGenesysTerminalApplication.h"
00018
00019  class Smart_CellularAutomata : public BaseGenesysTerminalApplication {
00020  public:
00021      Smart_CellularAutomata();
00022  public:
00023      virtual int main(int argc, char** argv) override;
00024  };
00025
00026  #endif /* SMART_CELLULARAUTOMATA_H */
00027

```

11.184 Smart_CellularAutomata1.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:    Smart_CellularAutomata.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 3 de Setembro de 2025, 18:34
00012  */
00013
00014  #ifndef SMART_CELLULARAUTOMATA1_H
00015  #define SMART_CELLULARAUTOMATA1_H
00016
00017  #include "../BaseGenesysTerminalApplication.h"
00018
00019  class Smart_CellularAutomata1 : public BaseGenesysTerminalApplication {
00020  public:
00021      Smart_CellularAutomata1();
00022  public:
00023      virtual int main(int argc, char** argv) override;
00024  };
00025
00026  #endif /* SMART_CELLULARAUTOMATA_H */

```

11.185 Smart_DefaultModalModel.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007
00008  #pragma once
00009
00010  #include "../BaseGenesysTerminalApplication.h"
00011
00012  class Smart_DefaultModalModel : public BaseGenesysTerminalApplication {
00013  public:
00014      Smart_DefaultModalModel();
00015  public:
00016      virtual int main(int argc, char** argv) override;
00017  };
00018
00019

```


11.186 Smart_Dummy.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_Dummy.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_DUMMY_H
00015 #define SMART_DUMMY_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_Dummy : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_Dummy();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_DUMMY_H */
00027

```

11.187 Smart_FSMState.h

```

00001 #pragma once
00002
00003 #include "../BaseGenesysTerminalApplication.h"
00004
00005 class Smart_FSMState : public BaseGenesysTerminalApplication {
00006 public:
00007     Smart_FSMState();
00008 public:
00009     virtual int main(int argc, char** argv) override;
00010 };

```

11.188 Smart_ModelModel.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_ModelModel.h
00009  * Author: rlcancian
00010  *
00011  * Created on 30 de Setembro de 2025, 17:46
00012  */
00013
00014 #ifndef SMART_MODELMODEL_H
00015 #define SMART_MODELMODEL_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_ModelModel : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_ModelModel();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_MODELMODEL_H */
00027
00028

```

11.189 Smart_ModelModelDefaultNetwork.h

```

00001 #pragma once

```

```

00002
00003 #include "../.../BaseGenesysTerminalApplication.h"
00004
00005 class Smart_ModelModelDefaultNetwork : public BaseGenesysTerminalApplication {
00006 public:
00007     Smart_ModelModelDefaultNetwork();
00008 public:
00009     virtual int main(int argc, char** argv) override;
00010 };

```

11.190 Smart_ModelModelFSM.h

```

00001 #pragma once
00002
00003 #include "../.../BaseGenesysTerminalApplication.h"
00004
00005 class Smart_ModelModelFSM : public BaseGenesysTerminalApplication {
00006 public:
00007     Smart_ModelModelFSM();
00008 public:
00009     virtual int main(int argc, char** argv) override;
00010 };

```

11.191 Smart_ModelModelPetriNet.h

```

00001 #pragma once
00002
00003 #include "../.../BaseGenesysTerminalApplication.h"
00004
00005 class Smart_ModelModelPetriNet : public BaseGenesysTerminalApplication {
00006 public:
00007     Smart_ModelModelPetriNet();
00008 public:
00009     virtual int main(int argc, char** argv) override;
00010 };

```

11.192 Smart_PetriPlace.h

```

00001 #pragma once
00002
00003 #include "../.../BaseGenesysTerminalApplication.h"
00004
00005 class Smart_PetriPlace : public BaseGenesysTerminalApplication {
00006 public:
00007     Smart_PetriPlace();
00008 public:
00009     virtual int main(int argc, char** argv) override;
00010 };

```

11.193 Smart_OnEvent.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:     Smart_OnEvent.h
00009  * Author:   rlcancian
00010  *
00011  * Created on 5 de Janeiro de 2021, 18:00
00012  */
00013
00014 #ifndef SMART_ONEVENT_H
00015 #define SMART_ONEVENT_H
00016
00017 #include "../.../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_OnEvent : public BaseGenesysTerminalApplication {
00020 public:

```

```

00021     Smart_OnEvent();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 public:
00025     virtual void onBreakpointHandler(SimulationEvent* re) override;
00026     virtual void onEntityCreateHandler(SimulationEvent* re) override;
00027     virtual void onEntityMoveHandler(SimulationEvent* re) override;
00028     virtual void onSimulationStartHandler(SimulationEvent* re) override;
00029     virtual void onReplicationStartHandler(SimulationEvent* re) override;
00030     virtual void onReplicationStepHandler(SimulationEvent* re) override;
00031     virtual void onProcessEventHandler(SimulationEvent* re) override;
00032     virtual void onReplicationEndHandler(SimulationEvent* re) override;
00033     virtual void onSimulationEndHandler(SimulationEvent* re) override;
00034     virtual void onSimulationPausedHandler(SimulationEvent* re) override;
00035     virtual void onSimulationResumeHandler(SimulationEvent* re) override;
00036     virtual void onEntityRemoveHandler(SimulationEvent* re) override;
00037 };
00038
00039 #endif /* SMART_ONEVENT_H */

```

11.194 Smart_SimulationControlResponse.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File:     Smart_SimulationControlResponse.h
00008  * Author:   rlcancian
00009  *
00010  * Created on 29 de novembro de 2022, 21:07
00011  */
00012
00013 #ifndef SMART_SIMULATIONCONTROLRESPONSE_H
00014 #define SMART_SIMULATIONCONTROLRESPONSE_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_SimulationControlResponse : public BaseGenesysTerminalApplication {
00019 public:
00020     Smart_SimulationControlResponse();
00021 public:
00022     virtual int main(int argc, char** argv) override;
00023 };
00024
00025 #endif /* SMART_SIMULATIONCONTROLRESPONSE_H */
00026

```

11.195 Smart_WaitScanCondition.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File:     Smart_WaitScanCondition.h
00008  * Author:   rlcancian
00009  *
00010  * Created on 14 de junho de 2022, 13:00
00011  */
00012
00013 #ifndef SMART_WAITSCANCONDITION_H
00014 #define SMART_WAITSCANCONDITION_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class Smart_WaitScanCondition : public BaseGenesysTerminalApplication {
00019 public:
00020     Smart_WaitScanCondition();
00021 public:
00022     virtual int main(int argc, char** argv) override;
00023 };
00024
00025 #endif /* SMART_WAITSCANCONDITION_H */
00026

```

11.196 Smart_WaitSignal.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_WaitSignal.h
00009  * Author: rlcancian
00010  *
00011  * Created on 3 de Setembro de 2019, 18:34
00012  */
00013
00014 #ifndef SMART_WAIT SIGNAL_H
00015 #define SMART_WAIT SIGNAL_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class Smart_WaitSignal : public BaseGenesysTerminalApplication {
00020 public:
00021     Smart_WaitSignal();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* SMART_WAIT SIGNAL_H */
00027

```

11.197 AnElectronicAssemblyAndTestSystem.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File: Smart_AnElectronicAssemblyAndTestSystem.h
00008  * Author: rlcancian
00009  *
00010  * Created on 6 de dezembro de 2021, 11:09
00011  */
00012
00013 #ifndef ANELECTRONICASSEMBLYANDTESTSYSTEM_H
00014 #define ANELECTRONICASSEMBLYANDTESTSYSTEM_H
00015
00016 #include "../BaseGenesysTerminalApplication.h"
00017
00018 class AnElectronicAssemblyAndTestSystem : public BaseGenesysTerminalApplication {
00019 public:
00020     AnElectronicAssemblyAndTestSystem();
00021 public:
00022     virtual int main(int argc, char** argv) override;
00023
00024 };
00025
00026 #endif /* ANELECTRONICASSEMBLYANDTESTSYSTEM_H */
00027

```

11.198 BufferFIFO.h

```

00001 #pragma once
00002
00003 /*
00004  * File: BufferFIFO.h
00005  * Author: rlcancian
00006  *
00007  * Created on 13 de setembro de 2023, 18:03
00008  */
00009
00010 #include "../BaseGenesysTerminalApplication.h"
00011
00012 #include "kernel/simulator/Simulator.h"
00013 #include "plugins/components/Logic/Create.h"
00014 #include "plugins/components/DiscreteProcessing/Clone.h"
00015 #include "plugins/components/Synchronization/Wait.h"
00016 #include "plugins/components/Synchronization/Signal.h"
00017 #include "plugins/components/Logic/Dispose.h"

```

```

00018 #include "plugins/components/Decisions/Decide.h"
00019 #include "plugins/data/Synchronization/SignalData.h"
00020 #include "plugins/data/Logic/Variable.h"
00021
00022 class BufferFIFO : public BaseGenesysTerminalApplication {
00023 public:
00024     BufferFIFO() {}
00025 public:
00026     virtual int main(int argc, char** argv) {
00027         Simulator* genesys = new Simulator();
00028         this->setDefaultTraceHandlers(genesys->getTraceManager());
00029         genesys->getPluginManager()->autoInsertPlugins("autoloadplugins.txt");
00030         Model* model = genesys->getModelManager()->newModel();
00031         PluginManager* plugins = genesys->getPluginManager();
00032
00033         Create* createl = plugins->newInstance<Create>(model);
00034         createl->setTimeBetweenCreationsExpression("1", Util::TimeUnit::hour);
00035
00036         Clone* clone1 = plugins->newInstance<Clone>(model);
00037         clone1->setNumClonesExpression("1");
00038
00039         createl->getConnectionManager()->insert(clone1);
00040
00041         SignalData* sigdata1 = plugins->newInstance<SignalData>(model);
00042         Queue* queue1 = plugins->newInstance<Queue>(model, "waitingQueue");
00043         Wait* wait1 = plugins->newInstance<Wait>(model);
00044         wait1->setWaitType(Wait::WaitType::WaitForSignal);
00045         wait1->setSignalData(sigdata1);
00046         wait1->setQueue(queue1);
00047
00048         clone1->getConnectionManager()->insert(wait1);
00049
00050         Dispose* dispose1 = plugins->newInstance<Dispose>(model);
00051
00052         wait1->getConnectionManager()->insert(dispose1);
00053
00054         Decide* decide1 = plugins->newInstance<Decide>(model);
00055         decide1->getConditions()->insert("nq(waitingQueue)<tamBuffer");
00056         Variable* var1 = plugins->newInstance<Variable>(model, "tamBuffer");
00057         var1->setInitialValue(5);
00058
00059         clone1->getConnectionManager()->insert(decide1);
00060
00061         Dispose* dispose2 = plugins->newInstance<Dispose>(model);
00062         dispose2->setReportStatistics(false);
00063
00064         decide1->getConnectionManager()->insert(dispose2);
00065
00066         Signal* signal1 = plugins->newInstance<Signal>(model);
00067         signal1->setSignalData(sigdata1);
00068
00069         decide1->getConnectionManager()->insert(signal1);
00070         signal1->getConnectionManager()->insert(dispose2);
00071
00072         ModelSimulation* sim = model->getSimulation();
00073         sim->setReplicationLength(50, Util::TimeUnit::hour);
00074         sim->setReplicationReportBaseTimeUnit(Util::TimeUnit::hour);
00075
00076         model->save("./model/teachingBufferFIFO.gen");
00077         model->getTracer()->setTraceLevel(TraceManager::Level::L9_mostDetailed);
00078
00079         do {
00080             sim->step();
00081             std::cout << "Press ENTER to continue...";
00082             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00083         } while (sim->isPaused());
00084         delete genesys;
00085         return 0;
00086     }
00087 };
00088

```

11.199 ContinuousModel.h

```

00001 #pragma once
00002
00003 #include "../BaseGenesysTerminalApplication.h"
00004
00005 class ContinuousModel: public BaseGenesysTerminalApplication {
00006 public:
00007     ContinuousModel();
00008 public: // virtual
00009     virtual void Simulate(Simulator* genesys, Model* model, PluginManager *plugins) override;

```

```
00010 };
00011
```

11.200 FullSimulationOfComplexModel.h

```
00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:      buildSimpleModel1.h
00009  * Author:    rafael.luiz.cancian
00010  *
00011  * Created on 2 de Outubro de 2018, 19:18
00012  */
00013
00014 #ifndef COMPLEXMODEL01_H
00015 #define COMPLEXMODEL01_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class FullSimulationOfComplexModel : public BaseGenesysTerminalApplication {
00020 public:
00021     FullSimulationOfComplexModel();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* COMPLEXMODEL01_H */
00027
```

11.201 Half_Adder.h

```
00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 #ifndef HALF_ADDER_H
00008 #define HALF_ADDER_H
00009
00010 #include "../BaseGenesysTerminalApplication.h"
00011
00012 class Half_Adder : public BaseGenesysTerminalApplication {
00013 public:
00014     Half_Adder();
00015 public:
00016     virtual int main(int argc, char** argv) override;
00017 };
00018
00019 #endif /* HALF_ADDER_H */
00020
```

11.202 Loja01.h

```
00001 #pragma once
00002
00003
00004 /*
00005  * File:      Loja01.h
00006  * Author:    rlcancian
00007  *
00008  * Created on 13 de Novembro de 2024, 10:25
00009  */
00010 #include "../BaseGenesysTerminalApplication.h"
00011
00012 class Loja01 : public BaseGenesysTerminalApplication {
00013 public:
00014     Loja01();
00015 public:
00016     virtual int main(int argc, char** argv) override;
00017 };
00018
00019
00020
```

11.203 OperatingSystem02.h

```
00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_SistemasOperacionais02.h
00009  * Author: rlcancian
00010  *
00011  * Created on 12 de abril de 2021, 21:54
00012  */
00013
00014 #ifndef OPERATINGSYSTEMSS02_H
00015 #define OPERATINGSYSTEMSS02_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class OperatingSystem02 : public BaseGenesysTerminalApplication {
00020 public:
00021     OperatingSystem02();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* OPERATINGSYSTEMSS02_H */
00027
```

11.204 OperatingSystem03.h

```
00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Smart_SistemaOperacional03.h
00009  * Author: rlcancian
00010  *
00011  * Created on 23 de abril de 2021, 14:03
00012  */
00013
00014 #ifndef OPERATINGSYSTEMS03_H
00015 #define OPERATINGSYSTEMS03_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018
00019 class OperatingSystem03 : public BaseGenesysTerminalApplication {
00020 public:
00021     OperatingSystem03();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 private:
00025
00026 };
00027
00028 #endif /* OPERATINGSYSTEMS03_H */
00029
```

11.205 Rectifier.h

```
00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 #ifndef RECTIFIER_H
00008 #define RECTIFIER_H
00009
00010 #include "../BaseGenesysTerminalApplication.h"
00011
00012 class Rectifier : public BaseGenesysTerminalApplication {
00013 public:
00014     Rectifier();
00015 public:
```

```
00016     virtual int main(int argc, char** argv) override;
00017 };
00018
00019 #endif /* HALF_ADDER_H */
00020
```

11.206 WcmBioNetworkBridge.h

```
00001 #ifndef WCMBIONETWORKBRIDGE_H
00002 #define WCMBIONETWORKBRIDGE_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00014 class WcmBioNetworkBridge : public BaseGenesysTerminalApplication {
00015 public:
00016     WcmBioNetworkBridge();
00017     virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* WCMBIONETWORKBRIDGE_H */
```

11.207 WcmCompartmentTransportWholeCellMvp.h

```
00001 #ifndef WCMCOMPARTMENTTRANSPORTWHOLECELLMVP_H
00002 #define WCMCOMPARTMENTTRANSPORTWHOLECELLMVP_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00019 class WcmCompartmentTransportWholeCellMvp : public BaseGenesysTerminalApplication {
00020 public:
00021     WcmCompartmentTransportWholeCellMvp();
00022     virtual int main(int argc, char** argv) override;
00023 };
00024
00025 #endif /* WCMCOMPARTMENTTRANSPORTWHOLECELLMVP_H */
```

11.208 WcmFluxBalanceWholeCellMvp.h

```
00001 #ifndef WCMFLUXBALANCEWHOLECELLMVP_H
00002 #define WCMFLUXBALANCEWHOLECELLMVP_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00015 class WcmFluxBalanceWholeCellMvp : public BaseGenesysTerminalApplication {
00016 public:
00017     WcmFluxBalanceWholeCellMvp();
00018     virtual int main(int argc, char** argv) override;
00019 };
00020
00021 #endif /* WCMFLUXBALANCEWHOLECELLMVP_H */
```

11.209 WcmGeneExpression.h

```
00001 #ifndef WCMGENEEXPRESSION_H
00002 #define WCMGENEEXPRESSION_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00028 class WcmGeneExpression : public BaseGenesysTerminalApplication {
00029 public:
00030     WcmGeneExpression();
00031     virtual int main(int argc, char** argv) override;
00032 };
00033
00034 #endif /* WCMGENEEXPRESSION_H */
```


11.210 WcmLifecycleStress.h

```
00001 #ifndef WCMLIFECYCLESTRESS_H
00002 #define WCMLIFECYCLESTRESS_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00014 class WcmLifecycleStress : public BaseGenesysTerminalApplication {
00015 public:
00016     WcmLifecycleStress();
00017     virtual int main(int argc, char** argv) override;
00018 };
00019
00020 #endif /* WCMLIFECYCLESTRESS_H */
```

11.211 WcmMgenitaliumKarr2012.h

```
00001 #ifndef WCMMGENITALIUMKARR2012_H
00002 #define WCMMGENITALIUMKARR2012_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00053 class WcmMgenitaliumKarr2012 : public BaseGenesysTerminalApplication {
00054 public:
00055     WcmMgenitaliumKarr2012();
00056     virtual int main(int argc, char** argv) override;
00057 };
00058
00059 #endif /* WCMMGENITALIUMKARR2012_H */
```

11.212 WcmPathwayStressWholeCellMvp.h

```
00001 #ifndef WCMPATHWAYSTRESSWHOLECELLMVP_H
00002 #define WCMPATHWAYSTRESSWHOLECELLMVP_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00013 class WcmPathwayStressWholeCellMvp : public BaseGenesysTerminalApplication {
00014 public:
00015     WcmPathwayStressWholeCellMvp();
00016     virtual int main(int argc, char** argv) override;
00017 };
00018
00019 #endif /* WCMPATHWAYSTRESSWHOLECELLMVP_H */
```

11.213 WcmSimpleSsa.h

```
00001 #ifndef WCMSIMPLESSA_H
00002 #define WCMSIMPLESSA_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00025 class WcmSimpleSsa : public BaseGenesysTerminalApplication {
00026 public:
00027     WcmSimpleSsa();
00028     virtual int main(int argc, char** argv) override;
00029 };
00030
00031 #endif /* WCMSIMPLESSA_H */
```

11.214 WcmWholeCellMvp.h

```
00001 #ifndef WCMWHOLECELLMVP_H
00002 #define WCMWHOLECELLMVP_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00020 class WcmWholeCellMvp : public BaseGenesysTerminalApplication {
00021 public:
00022     WcmWholeCellMvp();
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* WCMWHOLECELLMVP_H */
```

11.215 WcmYeastDidacticWholeCellMvp.h

```

00001 #ifndef WCMYEASTDIDACTICWHOLECELLMVP_H
00002 #define WCMYEASTDIDACTICWHOLECELLMVP_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00018 class WcmYeastDidacticWholeCellMvp : public BaseGenesysTerminalApplication {
00019 public:
00020     WcmYeastDidacticWholeCellMvp();
00021     virtual int main(int argc, char** argv) override;
00022 };
00023
00024 #endif /* WCMYEASTDIDACTICWHOLECELLMVP_H */

```

11.216 WcmYeastEukaryoticWholeCellMvp.h

```

00001 #ifndef WCMYEASTEUKARYOTICWHOLECELLMVP_H
00002 #define WCMYEASTEUKARYOTICWHOLECELLMVP_H
00003
00004 #include "../BaseGenesysTerminalApplication.h"
00005
00019 class WcmYeastEukaryoticWholeCellMvp : public BaseGenesysTerminalApplication {
00020 public:
00021     WcmYeastEukaryoticWholeCellMvp();
00022 public:
00023     virtual int main(int argc, char** argv) override;
00024 };
00025
00026 #endif /* WCMYEASTEUKARYOTICWHOLECELLMVP_H */

```

11.217 GenesysShell.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: GenesysTerminalAppApp.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 23 de Maio de 2019, 13:02
00012  */
00013
00014 #ifndef GENESYSHELL_H
00015 #define GENESYSHELL_H
00016
00017 #include "../BaseGenesysTerminalApplication.h"
00018 #include "kernel/simulator/SimulatorFacade.h"
00019 #include "kernel/simulator/Simulator.h"
00020 #include "kernel/util/List.h"
00021 #include <memory>
00022 #include <regex>
00023 #include <fstream>
00024 #include <assert.h>
00025
00026 class GenesysShell : public BaseGenesysTerminalApplication {
00027 private:
00028     typedef std::function<void() > ExecuterMember;
00029
00030     template<typename Class>
00031     ExecuterMember DefineExecuterMember(Class * object, void (Class::*function)()) {
00032         return std::bind(function, object); //, std::placeholders::_1
00033     }
00034
00035     class ShellCommand {
00036     public:
00037
00038         ShellCommand(std::string shortname, std::string longname, std::string parameters, std::string
description, ExecuterMember executer) {
00039             this->description = description;
00040             this->longname = longname;
00041             this->parameters = parameters;
00042             this->shortname = shortname;
00043             this->executer = executer;
00044         }
00045         std::string shortname;

```

```

00046         std::string longname;
00047         std::string parameters;
00048         std::string description;
00049         ExecuterMember executer;
00050     };
00051 public:
00052     GenesysShell();
00053     GenesysShell(const GenesysShell& orig);
00054     virtual ~GenesysShell() = default;
00055 public:
00056     virtual int main(int argc, char** argv) override;
00057 public: // commands
00058     void cmdScript();
00059     void cmdExecuteScript();
00060     void cmdHelp();
00061     void cmdQuit();
00062     void cmdBash();
00063     void cmdFacade();
00064     void cmdLicence();
00065     void cmdInfo();
00066     void cmdSim();
00067     void cmdData();
00068     void cmdComponent();
00069     void cmdExperiment();
00070     void cmdParser();
00071     void cmdTraceLevel();
00072     void cmdPlugin();
00073     //void cmdPluginInfo();
00074     //void cmdPluginAdd();
00075     //void cmdPluginRemove();
00076     void cmdSimulation();
00077     //void cmdSimulationStep();
00078     //void cmdSimulationStop();
00079     //void cmdSimulationInfo();
00080     void cmdReplication();
00081     //void cmdShowReport();
00082     void cmdModel();
00083     //void cmdModelClose();
00084     void cmdModelLoad();
00085     //void cmdModelCheck();
00086     void cmdModelSave();
00087     //void cmdModelShow();
00088     //void cmdModelSetInfos();
00089 private:
00090     void defineCommands();
00091     void run(List<std::string>* arguments);
00092     void Trace(std::string message);
00093     //void tryExecuteCommand(std::string inputText, const char* shortPrefix, const char* longPrefix,
std::string separator);
00094     void tryExecuteCommand(std::string inputText);
00095     std::vector<std::string> split(std::string text, std::string separatorRegex);
00096     std::string joinArguments(std::size_t firstArgumentIndex = 1) const;
00097     std::string lowercase(std::string text) const;
00098     bool requireCurrentModel(const std::string& actionDescription);
00099     bool parseUnsigned(const std::string& text, unsigned int& value) const;
00100     bool parseInt(const std::string& text, int& value) const;
00101     bool parseDouble(const std::string& text, double& value) const;
00102     bool parseBool(const std::string& text, bool& value) const;
00103     bool parseTraceLevel(const std::string& text, TraceManager::Level& level) const;
00104     bool parseTimeUnit(const std::string& text, Util::TimeUnit& timeUnit) const;
00105 private:
00106     std::unique_ptr<Simulator> simulator;
00107     SimulatorFacade facade;
00108     Model* model = nullptr;
00109     std::vector<std::string> *_typedWords = new std::vector<std::string>();
00110     std::vector<std::string> *_history = new std::vector<std::string>();
00111     List<ShellCommand*> *_commands = new List<ShellCommand*>();
00112     std::string _prompt = "$genesys> ";
00113     bool _exitRequested = false;
00114 };
00115 #endif /* GENESYSHELL_H */

```

11.218 GenesysShell_if.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   GenesysShell_if.h
00009  * Author: rlcancian

```

```

00010  *
00011  * Created on 24 de Maio de 2019, 11:02
00012  */
00013
00014 #ifndef GENESYSSHELL_IF_H
00015 #define GENESYSSHELL_IF_H
00016
00017 #include "GenesysApplication_if.h"
00018 #include "Util.h"
00019
00020 class GenesysShell_if : public GenesysApplication_if {
00021 public:
00022     virtual ~GenesysShell_if() = default;
00023     virtual void openModel(std::string filename) = 0;
00024     virtual void saveModelAs(std::string filename) = 0;
00025     virtual void saveModel() = 0;
00026     virtual void listElements() = 0;
00027     virtual void listComponents() = 0;
00028     virtual void listHosts() = 0;
00029     virtual void listPlugins() = 0;
00030     virtual void deleteTraceFiles() = 0;
00031     virtual void traceLevel(TraceManager::TraceLevel tracelevel) = 0;
00032     virtual void addPlugin(std::string filename) = 0;
00033     virtual void addFromFile(std::string filename) = 0;
00034     virtual void readCommandsFromFile(std::string filename) = 0;
00035     virtual void redirectTrace(std::string trace, std::string dest, std::string filename) = 0;
00036     virtual void closeModel() = 0;
00037     virtual void createModel() = 0;
00038     virtual void execLinuxCommand(std::string command) = 0;
00039     virtual void verboseMode(bool on) = 0;
00040     virtual void check() = 0;
00041     virtual void getGenesysInfo() = 0;
00042     virtual void getCommandLine() = 0;
00043     virtual void sendFile(std::string filename, std::string hostname, std::string portname) = 0;
00044     virtual void setActivationCode(std::string code) = 0;
00045     virtual void receiveFile(std::string filename) = 0;
00046     virtual void startSimulation() = 0;
00047     virtual void stepSimulation() = 0;
00048     virtual void stopSimulation() = 0;
00049     virtual void showInit() = 0;
00050     virtual void showHelp() = 0;
00051     virtual void showHostName() = 0;
00052     //virtual void execute() = 0;
00053     //property TraceHandler : TProcTrace write aTraceHandler;
00054 };
00055
00056 #endif /* GENESYSSHELL_IF_H */

```

11.219 ApiRouter.h

```

00001 #pragma once
00002
00003 #include "../http/HttpRequest.h"
00004 #include "../http/HttpResponse.h"
00005 #include "../service/SimulatorSessionService.h"
00006
00007 #include <optional>
00008
00012 class ApiRouter {
00013 public:
00018     explicit ApiRouter(SimulatorSessionService& simulatorService);
00019
00025     HttpResponse handle(const HttpRequest& request) const;
00026
00027 private:
00028     SimulatorSessionService& _simulatorService;
00029
00037     static HttpResponse _jsonError(int status, const char* code, const char* message);
00043     static std::string _extractBearerToken(const HttpRequest& request);
00050     static bool _tryExtractFilenameFromBody(const std::string& body, std::string& outFilename);
00058     static bool _tryExtractUnsignedIntField(const std::string& body, const std::string& fieldName,
00066     unsigned int& outValue);
00074     static bool _tryExtractDoubleField(const std::string& body, const std::string& fieldName, double&
00080     outValue);
00086     static bool _tryExtractBooleanField(const std::string& body, const std::string& fieldName, bool&
00092     outValue);
00098     static std::optional<SimulatorSessionService::SimulationConfigInput>
00104     _parseSimulationConfigBody(const std::string& body);
00110     static std::string _simulationStatusDataJson(const
00116     SimulatorSessionService::SimulationStatusResult& status);
00122     static HttpResponse _mapPersistenceError(const SimulatorSessionService::ModelPersistenceResult&
00128     result);
00134     static HttpResponse _mapModelImportError(const SimulatorSessionService::ModelImportResult&
00140     result);

```

```

00104     static std::string _escapeJson(const std::string& value);
00110     static std::string _modelInfoDataJson(const SimulatorSessionService::ModelInfoResult& info);
00116     static std::string _workerInfoDataJson(const SimulatorSessionService::WorkerInfoResult& info);
00122     static std::string _workerCapabilitiesDataJson(const
SimulatorSessionService::WorkerCapabilitiesResult& capabilities);
00129     static bool _tryExtractWorkerJobIdFromPath(const std::string& path, std::string& outJobId);
00136     static bool _tryExtractWorkerJobRunIdFromPath(const std::string& path, std::string& outJobId);
00143     static bool _tryExtractWorkerJobResultIdFromPath(const std::string& path, std::string& outJobId);
00149     static const char* _workerJobStateToString(WorkerJobState state);
00155     static std::string _workerJobDataJson(const SimulatorSessionService::WorkerJobInfoResult& job);
00161     static std::string _workerJobResultDataJson(const SimulatorSessionService::WorkerJobResultInfo&
result);
00168     static HttpResponse _mapWorkerJobError(SimulatorSessionService::WorkerJobError error, bool
includeMissingModelMessage);
00169 };

```

11.220 TokenService.h

```

00001 #pragma once
00002
00003 #include <cstdint>
00004 #include <string>
00005
00006 class TokenService {
00007 public:
00008     TokenService();
00009
00010     [[nodiscard]] std::string generateSessionId();
00011     [[nodiscard]] std::string generateAccessToken();
00012
00013 private:
00014     [[nodiscard]] std::string _generateHexToken(std::size_t bytes);
00015 };

```

11.221 BaseGenesysWebApplication.h

```

00001 #pragma once
00002
00003 #include "../GenesysApplication_if.h"
00004
00012 class BaseGenesysWebApplication : public GenesysApplication_if {
00013 public:
00020     int main(int argc, char** argv) override;
00021
00022 private:
00029     static unsigned short _readPort(int argc, char** argv);
00036     static unsigned long _readMaxRequests(int argc, char** argv);
00037 };

```

11.222 HttpRequest.h

```

00001 #pragma once
00002
00003 #include <algorithm>
00004 #include <cctype>
00005 #include <string>
00006 #include <unordered_map>
00007
00015 struct HttpRequest {
00017     std::string method;
00019     std::string path;
00021     std::string body;
00023     std::unordered_map<std::string, std::string> headers;
00024
00030     [[nodiscard]] std::string header(const std::string& name) const {
00031         std::string normalized = name;
00032         std::transform(normalized.begin(), normalized.end(), normalized.begin(), [](unsigned char c) {
00033             return static_cast<char>(std::tolower(c));
00034         });
00035         auto it = headers.find(normalized);
00036         if (it == headers.end()) {
00037             return {};
00038         }
00039         return it->second;
00040     }
00041 };

```

11.223 HttpResponse.h

```

00001 #pragma once
00002
00003 #include <string>
00004 #include <unordered_map>
00005
00012 struct HttpResponse {
00014     int status = 200;
00016     std::string contentType = "application/json";
00018     std::string body;
00020     std::unordered_map<std::string, std::string> headers;
00021 };

```

11.224 SimpleHttpServer.h

```

00001 #pragma once
00002
00003 #include "HttpRequest.h"
00004 #include "HttpResponse.h"
00005
00006 #include <atomic>
00007 #include <functional>
00008
00015 class SimpleHttpServer {
00016 public:
00017     using RequestHandler = std::function<HttpResponse(const HttpRequest&)>;
00018
00023     explicit SimpleHttpServer(unsigned short port);
00024
00031     bool serve(const RequestHandler& handler, unsigned long maxRequests = 0);
00032
00036     void requestStop();
00037
00038 private:
00039     unsigned short _port;
00040     std::atomic<bool> _stopRequested{false};
00041 };

```

11.225 SimulatorSessionService.h

```

00001 #pragma once
00002
00003 #include "../session/SessionManager.h"
00004 #include "../worker/WorkerJob.h"
00005 #include "../worker/WorkerJobManager.h"
00006
00007 #include <string>
00008
00012 class SimulatorSessionService {
00013 public:
00017     enum class PersistenceError {
00018         None,
00019         InvalidToken,
00020         InvalidFilename,
00021         MissingCurrentModel,
00022         FileNotFound,
00023         OperationFailed
00024     };
00025
00029     enum class ModelImportError {
00030         None,
00031         InvalidToken,
00032         EmptySpecification,
00033         InvalidSpecification,
00034         OperationFailed
00035     };
00036
00040     enum class WorkerJobError {
00041         None,
00042         InvalidToken,
00043         MissingCurrentModel,
00044         JobNotFound,
00045         AccessDenied,
00046         OperationFailed,
00047         ResultNotReady
00048     };
00049
00053     struct CreateSessionResult {

```

```

00054         std::string sessionId;
00055         std::string accessToken;
00056     };
00057
00061     struct SimulatorInfoResult {
00062         std::string name;
00063         std::string versionName;
00064         unsigned int versionNumber;
00065     };
00066
00070     struct WorkerInfoResult {
00071         std::string role;
00072         std::string application;
00073         std::string apiFamily;
00074         std::string apiVersion;
00075         std::string simulatorName;
00076         std::string simulatorVersionName;
00077         unsigned int simulatorVersionNumber = 0;
00078     };
00079
00083     struct WorkerCapabilitiesResult {
00084         bool supportsSessionApi = false;
00085         bool supportsSessionScopedSimulator = false;
00086         bool supportsModelCreation = false;
00087         bool supportsModelPersistence = false;
00088         bool supportsSimulationStatus = false;
00089         bool supportsSimulationConfig = false;
00090         bool supportsSynchronousRun = false;
00091         bool supportsSynchronousStep = false;
00092         bool supportsDistributedJobs = false;
00093         bool supportsJobPolling = false;
00094         bool supportsBackgroundExecution = false;
00095         bool supportsModelUpload = false;
00096         bool supportsStreamingEvents = false;
00097         bool supportsJobResultRetrieval = false;
00098     };
00099
00103     struct ModelInfoResult {
00104         bool exists = false;
00105         unsigned int modelId = 0;
00106         bool hasChanged = false;
00107         unsigned int level = 0;
00108         std::string name;
00109         std::string analystName;
00110         std::string projectTitle;
00111         std::string version;
00112         std::string description;
00113         unsigned int componentCount = 0;
00114     };
00115
00119     struct ModelPersistenceResult {
00120         bool success = false;
00121         PersistenceError error = PersistenceError::None;
00122         std::string filename;
00123         ModelInfoResult modelInfo;
00124     };
00125
00129     struct ModelImportResult {
00130         bool success = false;
00131         ModelImportError error = ModelImportError::None;
00132         ModelInfoResult modelInfo;
00133     };
00134
00138     struct WorkerJobInfoResult {
00139         std::string jobId;
00140         std::string sessionId;
00141         WorkerJobState state = WorkerJobState::Queued;
00142         std::string snapshotFilename;
00143         std::string createdMarker;
00144         std::string message;
00145     };
00146
00150     struct WorkerJobCreationResult {
00151         bool success = false;
00152         WorkerJobError error = WorkerJobError::None;
00153         WorkerJobInfoResult jobInfo;
00154     };
00155
00159     struct WorkerJobQueryResult {
00160         bool success = false;
00161         WorkerJobError error = WorkerJobError::None;
00162         WorkerJobInfoResult jobInfo;
00163     };
00164
00168     struct WorkerJobRunResult {
00169         bool success = false;
00170         WorkerJobError error = WorkerJobError::None;

```

```

00171         WorkerJobInfoResult jobInfo;
00172     };
00173
00174     struct WorkerJobResultInfo {
00175         std::string jobId;
00176         WorkerJobState state = WorkerJobState::Queued;
00177         std::string message;
00178         double simulatedTime = 0.0;
00179         unsigned int currentReplicationNumber = 0;
00180         unsigned int numberOfReplications = 0;
00181         double replicationLength = 0.0;
00182         double warmUpPeriod = 0.0;
00183         bool hasIsPaused = false;
00184         bool isPaused = false;
00185     };
00186
00187     struct WorkerJobResultQueryResult {
00188         bool success = false;
00189         WorkerJobError error = WorkerJobError::None;
00190         WorkerJobResultInfo jobResult;
00191     };
00192
00193     struct SimulationStatusResult {
00194         bool success = false;
00195         bool invalidToken = false;
00196         bool hasCurrentModel = false;
00197         bool isRunning = false;
00198         bool isPaused = false;
00199         double simulatedTime = 0.0;
00200         unsigned int currentReplicationNumber = 0;
00201         unsigned int numberOfReplications = 0;
00202         double replicationLength = 0.0;
00203         double warmUpPeriod = 0.0;
00204         bool pauseOnEvent = false;
00205         bool pauseOnReplication = false;
00206         bool initializeStatistics = false;
00207         bool initializeSystem = false;
00208     };
00209
00210     struct SimulationConfigInput {
00211         unsigned int numberOfReplications = 0;
00212         double replicationLength = 0.0;
00213         double warmUpPeriod = 0.0;
00214         bool pauseOnEvent = false;
00215         bool pauseOnReplication = false;
00216         bool initializeStatistics = false;
00217         bool initializeSystem = false;
00218     };
00219
00220     struct SimulationConfigResult {
00221         bool success = false;
00222         bool invalidToken = false;
00223         bool missingCurrentModel = false;
00224         SimulationStatusResult status;
00225     };
00226
00227     struct SimulationActionResult {
00228         bool success = false;
00229         bool invalidToken = false;
00230         bool missingCurrentModel = false;
00231         SimulationStatusResult status;
00232     };
00233
00234     explicit SimulatorSessionService(SessionManager& sessionManager, WorkerJobManager&
workerJobManager);
00235
00236     CreateSessionResult createSession();
00237     bool tryGetSimulatorInfo(const std::string& accessToken, SimulatorInfoResult& outInfo);
00238     WorkerInfoResult getWorkerInfo() const;
00239     WorkerCapabilitiesResult getWorkerCapabilities() const;
00240     bool tryCreateModel(const std::string& accessToken, ModelInfoResult& outInfo);
00241     bool tryGetCurrentModelInfo(const std::string& accessToken, ModelInfoResult& outInfo);
00242     SimulationStatusResult getSimulationStatus(const std::string& accessToken);
00243     SimulationConfigResult configureSimulation(const std::string& accessToken, const
SimulationConfigInput& input);
00244     SimulationActionResult runSimulation(const std::string& accessToken);
00245     SimulationActionResult stepSimulation(const std::string& accessToken);
00246     ModelPersistenceResult saveCurrentModel(const std::string& accessToken, const std::string&
filename);
00247     ModelPersistenceResult loadModel(const std::string& accessToken, const std::string& filename);
00248     ModelImportResult importModelFromLanguage(const std::string& accessToken, const std::string&
modelSpecification);
00249     WorkerJobCreationResult createWorkerJob(const std::string& accessToken);
00250     WorkerJobQueryResult getWorkerJob(const std::string& accessToken, const std::string& jobId);
00251     WorkerJobRunResult runWorkerJob(const std::string& accessToken, const std::string& jobId);
00252     WorkerJobResultQueryResult getWorkerJobResult(const std::string& accessToken, const std::string&
jobId);

```



```

00368
00369 private:
00375     static bool _isSafeFilename(const std::string& filename);
00376
00382     static WorkerJobInfoResult _toWorkerJobInfoResult(const WorkerJob& job);
00383
00389     static WorkerJobResultInfo _toWorkerJobResultInfo(const WorkerJob& job);
00390
00391     SessionManager& _sessionManager;
00392     WorkerJobManager& _workerJobManager;
00393 };

```

11.226 WebWorkerRuntime.h

```

00001 #pragma once
00002
00003 #include "api/ApiRouter.h"
00004 #include "auth/TokenService.h"
00005 #include "http/SimpleHttpServer.h"
00006 #include "kernel/simulator/Simulator.h"
00007 #include "service/SimulatorSessionService.h"
00008 #include "session/SessionManager.h"
00009 #include "worker/WorkerJobManager.h"
00010
00011 #include <atomic>
00012 #include <deque>
00013 #include <memory>
00014 #include <mutex>
00015 #include <string>
00016 #include <thread>
00017 #include <vector>
00018
00026 class WebWorkerRuntime {
00027 public:
00031     struct Snapshot {
00032         bool isRunning = false;
00033         bool isStarting = false;
00034         bool isStopping = false;
00035         bool hasFailed = false;
00036         unsigned short port = 8080;
00037         unsigned long maxRequests = 0;
00038         unsigned long servedRequests = 0;
00039         unsigned long activeRequests = 0;
00040         std::string statusMessage;
00041         std::string lastError;
00042         std::vector<std::string> recentEvents;
00043         std::vector<std::string> recentErrors;
00044     };
00045
00049     explicit WebWorkerRuntime(unsigned short defaultPort = 8080, unsigned long maxRequests = 0);
00053     ~WebWorkerRuntime();
00054
00059     bool start();
00063     void stop();
00067     void wait();
00072     bool restart();
00073
00077     bool isRunning() const;
00081     Snapshot snapshot() const;
00082
00086     void setConfiguredPort(unsigned short port);
00090     unsigned short configuredPort() const;
00091
00095     void setConfiguredMaxRequests(unsigned long maxRequests);
00099     unsigned long configuredMaxRequests() const;
00100
00101 private:
00105     void _run();
00109     void _fail(const std::string& errorMessage);
00113     void _recordEventLocked(const std::string& message);
00117     void _recordErrorLocked(const std::string& message);
00121     std::string _statusMessageLocked() const;
00125     static std::unique_ptr<Simulator> _createSimulator();
00126
00127     mutable std::mutex _mutex;
00128     std::atomic<bool> _stopRequested{false};
00129     std::atomic<bool> _threadRunning{false};
00130     std::atomic<bool> _starting{false};
00131     std::atomic<bool> _stopping{false};
00132     std::atomic<unsigned long> _servedRequests{0};
00133     std::atomic<unsigned long> _activeRequests{0};
00134     unsigned short _configuredPort = 8080;
00135     unsigned long _configuredMaxRequests = 0;

```

```

00136     std::string _lastError;
00137     std::string _statusMessage;
00138     std::deque<std::string> _recentEvents;
00139     std::deque<std::string> _recentErrors;
00140     std::thread _thread;
00141
00142     std::unique_ptr<TokenService> _tokenService;
00143     std::unique_ptr<SessionManager> _sessionManager;
00144     std::unique_ptr<WorkerJobManager> _workerJobManager;
00145     std::unique_ptr<SimulatorSessionService> _simulatorSessionService;
00146     std::unique_ptr<ApiRouter> _router;
00147     std::unique_ptr<SimpleHttpServer> _server;
00148 };

```

11.227 SessionContext.h

```

00001 #pragma once
00002
00003 #include "kernel/simulator/Simulator.h"
00004
00005 #include <filesystem>
00006 #include <memory>
00007 #include <mutex>
00008 #include <string>
00009
00017 struct SessionContext {
00019     std::string sessionId;
00021     std::string accessToken;
00023     std::filesystem::path workspacePath;
00025     std::unique_ptr<Simulator> simulator;
00027     mutable std::mutex mutex;
00028 };

```

11.228 SessionManager.h

```

00001 #pragma once
00002
00003 #include "SessionContext.h"
00004 #include "../auth/TokenService.h"
00005
00006 #include <functional>
00007 #include <filesystem>
00008 #include <memory>
00009 #include <mutex>
00010 #include <string>
00011 #include <unordered_map>
00012
00020 class SessionManager {
00021 public:
00023     using SimulatorFactory = std::function<std::unique_ptr<Simulator>()>;
00024
00030     SessionManager(TokenService& tokenService, SimulatorFactory simulatorFactory);
00031
00036     SessionContext* createSession();
00042     SessionContext* getSessionByToken(const std::string& token);
00043
00044 private:
00049     static std::filesystem::path _buildWorkspaceRoot();
00050
00051     TokenService& _tokenService;
00052     SimulatorFactory _simulatorFactory;
00053     std::filesystem::path _workspaceRoot;
00054     mutable std::mutex _sessionsMutex;
00055     std::unordered_map<std::string, std::unique_ptr<SessionContext> _sessionsByToken;
00056 };

```

11.229 WorkerJob.h

```

00001 #pragma once
00002
00003 #include <optional>
00004 #include <string>
00005
00009 enum class WorkerJobState {
00010     Queued,

```

```

00011     Running,
00012     Finished,
00013     Failed
00014 };
00015
00019 struct WorkerJobTerminalResult {
00021     double simulatedTime = 0.0;
00023     unsigned int currentReplicationNumber = 0;
00025     unsigned int numberOfReplications = 0;
00027     double replicationLength = 0.0;
00029     double warmUpPeriod = 0.0;
00031     std::optional<bool> isPaused;
00032 };
00033
00037 struct WorkerJob {
00039     std::string jobId;
00041     std::string sessionId;
00043     WorkerJobState state = WorkerJobState::Queued;
00045     std::string snapshotFilename;
00047     std::string createdMarker;
00049     std::string message;
00051     bool hasTerminalResult = false;
00053     WorkerJobTerminalResult terminalResult;
00054 };

```

11.230 WorkerJobManager.h

```

00001 #pragma once
00002
00003 #include "WorkerJob.h"
00004
00005 #include <mutex>
00006 #include <optional>
00007 #include <string>
00008 #include <unordered_map>
00009
00016 class WorkerJobManager {
00017 public:
00023     WorkerJob createQueuedJob(const std::string& sessionId);
00024
00031     bool setSnapshotFilename(const std::string& jobId, const std::string& snapshotFilename);
00038     bool setState(const std::string& jobId, WorkerJobState state);
00039
00046     bool setMessage(const std::string& jobId, const std::string& message);
00047
00054     bool setTerminalResult(const std::string& jobId, const WorkerJobTerminalResult& terminalResult);
00055
00061     std::optional<WorkerJob> getJob(const std::string& jobId) const;
00062
00063 private:
00069     static std::string _makeCreationMarker(unsigned long long numericId);
00070
00071     mutable std::mutex _mutex;
00072     std::unordered_map<std::string, WorkerJob> _jobs;
00073     unsigned long long _nextJobNumber = 1;
00074 };

```

11.231 ConnectionManager.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: ConnectionManager.h
00009  * Author: rlcancian
00010  *
00011  * Created on 1 de Julho de 2019, 18:39
00012  */
00013
00014 #ifndef CONNECTIONMANAGER_H
00015 #define CONNECTIONMANAGER_H
00016
00017 #include <utility>
00018 #include <map>
00019 #include "../util/List.h"
00020

```

```

00021 //namespace GenesysKernel {
00022
00023 class ModelComponent;
00024
00025 //typedef unsigned int ConnectionPort;
00029 struct ConnectionChannel { /*< actually a port is only a uint, but it could be more complex, including
type, presence or absence of data, etc */
00030     unsigned int portNumber = 0;
00031     std::string portDescription = "";
00032 };
00033
00034
00040 struct Connection {
00041     ModelComponent* component;
00042     ConnectionChannel channel;
00043 };
00044
00051 class ConnectionManager {
00052 public:
00054     ConnectionManager();
00055     virtual ~ConnectionManager();
00056 public:
00061     unsigned int size();
00066     Connection* getFrontConnection();
00072     Connection* getConnectionAtPort(unsigned int rank);
00081     void insert(ModelComponent* component, unsigned int inputPortNumber = 0);
00087     void insert(Connection* connection);
00094     void insertAtPort(unsigned int port, Connection* connection);
00100     void remove(Connection* connection);
00106     void removeAtPort(unsigned int port);
00111     std::map<unsigned int, Connection*>* connections() const;
00112     //void setCurrentOutputConnections(unsigned int _currentOutputConnections);
00117     unsigned int getCurrentOutputConnectionsSize() const;
00122     void setMaxOutputConnections(unsigned int _maxOutputConnections);
00127     unsigned int getMaxOutputConnections() const;
00132     void setMinOutputConnections(unsigned int _minOutputConnections);
00137     unsigned int getMinOutputConnections() const;
00138     //void setCurrentInputConnections(unsigned int _currentInputConnections);
00143     unsigned int getCurrentInputConnectionsSize() const;
00148     void setMaxInputConnections(unsigned int _maxInputConnections);
00153     unsigned int getMaxInputConnections() const;
00158     void setMinInputConnections(unsigned int _minInputConnections);
00163     unsigned int getMinInputConnections() const;
00164 private:
00165     std::map<unsigned int, Connection*>* _nextConnections = new std::map<unsigned int, Connection*>();
00166     unsigned int _minInputConnections = 0;
00167     unsigned int _maxInputConnections = 0;
00168     unsigned int _currentInputConnections = 0;
00169     unsigned int _minOutputConnections = 0;
00170     unsigned int _maxOutputConnections = 0;
00171 };
00172 //namespace\\}
00173 #endif /* CONNECTIONMANAGER_H */

```

11.232 DefineGetterSetter.h

```

00001 #ifndef DEFINEGETTERSETTER_H
00002 #define DEFINEGETTERSETTER_H
00003
00004 #endif

```

11.233 Attribute.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   Attribute.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 25 de Setembro de 2018, 16:37
00012  */
00013
00014 #ifndef ATTRIBUTE_H
00015 #define ATTRIBUTE_H
00016

```

```

00017 #include <string>
00018 #include <list>
00019 #include <map>
00020 #include <utility>
00021 #include <vector>
00022 #include "../model/ModelDataDefinition.h"
00023 #include "../model/ModelDataManager.h"
00024 #include "../PluginInformation.h"
00025 #include "SparseValueStore.h"
00026
00027 //namespace GenesysKernel {
00028
00067 class Attribute : public ModelDataDefinition {
00068 public:
00070     Attribute(Model* model, std::string name = "", std::string dataDefinitionTypeName =
        Util::TypeOf<Attribute>());
00072     virtual ~Attribute() override;
00073 public:
00074     virtual std::string show() override;
00075 public: // public static methods
00076     static PluginInformation* GetPluginInformation();
00077     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00078     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00079 public:
00081     virtual std::string getInitialValuesText() const;
00083     virtual void setInitialValuesText(std::string valuesText);
00085     double getInitialValue(std::string index = "");
00087     void setInitialValue(double value, std::string index = "");
00089     void setInitialValues(const std::vector<std::pair<std::string, double> values);
00091     void copyInitialValuesFrom(const Attribute& source);
00093     void insertDimentionSize(unsigned int size);
00095     std::list<unsigned int>* getDimensionSizes() const;
00097     std::map<std::string, double>* getInitialValues() const;
00099     SparseValueStore* getInitialValueStore();
00101     bool isInitialValuesTextValid() const;
00102 protected:
00103     virtual bool _loadInstance(PersistenceRecord *fields) override;
00104     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00105 protected:
00106     virtual bool _check(std::string& errorMessage) override;
00107 private:
00108     void _syncInitialValuesTextFromStore();
00109     SparseValueStore* _initialValues = new SparseValueStore();
00110     std::string _initialValuesText = "0";
00111     bool _initialValuesTextValid = true;
00112     std::string _initialValuesTextErrorMessage;
00113 };
00114 //namespace}
00115 #endif /* ATTRIBUTE_H */

```

11.234 Counter.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: CounterDefaultImpl.h
00009  * Author: rafael.lui.cancian
00010  *
00011  * Created on 29 de Maio de 2019, 11:24
00012  */
00013
00014 #ifndef COUNTERDEFAULTIMPL_H
00015 #define COUNTERDEFAULTIMPL_H
00016
00017 #include "../model/ModelDataDefinition.h"
00018 #include "../model/ModelDataManager.h"
00019 #include "../Plugin.h"
00020 //namespace GenesysKernel {
00021
00028 class Counter : public ModelDataDefinition {
00029 public:
00031     Counter(Model* model, std::string name = "", ModelDataDefinition* parent = nullptr);
00032     virtual ~Counter() override;
00033 public:
00034     virtual std::string show() override;
00035 public: // public static methods
00036     static PluginInformation* GetPluginInformation();
00037     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00038     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");

```

```

00039 public:
00040     void clear();
00041     void incCountValue(/*int*/double value = 1.0);
00042     double /*unsigned long*/ getCountValue() const;
00043     ModelDataDefinition* getParent() const;
00044 protected:
00045     virtual bool _loadInstance(PersistenceRecord *fields) override;
00046     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00047 protected:
00048     virtual bool _check(std::string& errorMessage) override;
00049     virtual void _initBetweenReplications() override;
00050 private:
00051     ModelDataDefinition* _parent;
00052     double /*unsigned long*/ _count = 0;
00053     SimulationResponse* _countValueResponse = nullptr;
00054 };
00055 //namespace\}
00056 #endif /* COUNTERDEFAULTIMPL1_H */

```

11.235 Entity.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Entity.h
00009  * Author: rafael.lui.cancian
00010  *
00011  * Created on 21 de Junho de 2018, 16:30
00012  */
00013
00014 #ifndef ENTITY_H
00015 #define ENTITY_H
00016
00017 #include <string>
00018 #include <map>
00019
00020 #include "../util/Util.h"
00021 #include "../util/List.h"
00022 #include "../model/ModelDataDefinition.h"
00023 #include "EntityType.h"
00024 #include "SparseValueStore.h"
00025 //namespace GenesysKernel {
00026
00027 class Entity : public ModelDataDefinition {
00028 private: // no one can create or destroy entities directly. This can be done one through friend class
00029     Model
00030     Entity(Model* model, std::string name = "", bool insertIntoModel = true);
00031     virtual ~Entity();
00032     // friend Entity* Model::createEntity(std::string name, bool insertIntoModel); // It would be
00033     // better, but Model is not known at this point of compilation
00034     friend class Model;
00035 public:
00036     virtual std::string show() override;
00037
00038 public: // g & s
00039     void setEntityType(std::string entityType); /*!< indirect access to EntityType
00040     std::string getEntityType() const;
00041     void setEntityType(EntityType* entityType); /*!< direct access to EntityType
00042     EntityType* getEntityType() const;
00043 public:
00044     double getAttributeValue(std::string attributeName, std::string index="");
00045     double getAttributeValue(Util::identification attributeID, std::string index="");
00046     void setAttributeValue(std::string attributeName, double value, std::string index="", bool
00047     createIfNotFound=false);
00048     void setAttributeValue(Util::identification attributeID, double value, std::string index="");
00049     void copyAttributeValues(std::string destinationAttributeName, std::string sourceAttributeName);
00050     Util::identification entityNumber() const;
00051 protected:
00052     virtual bool _loadInstance(PersistenceRecord *fields) override;
00053     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00054     virtual bool _check(std::string& errorMessage) override;
00055 private:
00056     SparseValueStore* _ensureAttributeStore(unsigned int rank);
00057
00058     Util::identification _entityNumber;
00059     EntityType* _entityType = nullptr;
00060     List<SparseValueStore*>* _attributeValues = new List<SparseValueStore*>();
00061 };
00062 //namespace\}
00063 #endif /* ENTITY_H */

```

11.236 EntityType.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:   EntityType.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 14 de Agosto de 2018, 19:24
00012  */
00013
00014  #ifndef ENTITTYTYPE_H
00015  #define ENTITTYTYPE_H
00016
00017  #include <string>
00018  #include "../model/ModelDataDefinition.h"
00019  #include "StatisticsCollector.h"
00020  #include "../model/ModelDataManager.h"
00021  #include "../Plugin.h"
00022
00023  // #include "Model.h"
00024  // namespace GenesysKernel {
00025
00026  class EntityType : public ModelDataDefinition {
00027  public:
00028      EntityType(Model* model, std::string name = "");
00029      virtual ~EntityType();
00030  public:
00031      virtual std::string show() override;
00032  public: //static
00033      static PluginInformation* GetPluginInformation();
00034      static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00035      static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00036  public: //get & set
00037      // @ToDo: (importante): Change categories Waiting, Others, Transfer, etc to customizable ones
00038      void setInitialWaitingCost(double _initialWaitingCost);
00039      double initialWaitingCost() const;
00040      void setInitialOtherCost(double _initialOtherCost);
00041      double initialOtherCost() const;
00042      void setInitialNVACost(double _initialNVACost);
00043      double initialNVACost() const;
00044      void setInitialVACost(double _initialVACost);
00045      double initialVACost() const;
00046      void setInitialPicture(std::string _initialPicture);
00047      std::string initialPicture() const;
00048  public: //get
00049      StatisticsCollector* addGetStatisticsCollector(std::string name);
00050
00051  protected: // must be overridden
00052      virtual bool _loadInstance(PersistenceRecord *fields) override;
00053      virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00054  protected: // could be overriden
00055      //virtual bool _check(std::string& errorMessage) override;
00056      virtual void _initBetweenReplications() override;
00057      virtual void _createInternalStatisticReporters() override;
00058  private:
00059      void _initCostsAndStatistics();
00060  private:
00061
00062      const struct DEFAULT_VALUES {
00063          const std::string initialPicture = "report";
00064          const double initialCost = 0.0;
00065      } DEFAULT;
00066      std::string _initialPicture = DEFAULT.initialPicture;
00067      double _initialVACost = DEFAULT.initialCost;
00068      double _initialNVACost = DEFAULT.initialCost;
00069      double _initialOtherCost = DEFAULT.initialCost;
00070      double _initialWaitingCost = DEFAULT.initialCost;
00071  private: //l:n
00072      List<StatisticsCollector*>* _statisticsCollectors = new List<StatisticsCollector*>();
00073  };
00074  // namespace\\
00075  #endif /* ENTITTYTYPE_H */

```

11.237 SparseValueStore.h

```

00001  /*
00002  * File:   SparseValueStore.h
00003  * Author: rafael.luiz.cancian

```

```

00004  *
00005  * Created on 16 de Abril de 2026
00006  */
00007
00008 #ifndef SPARSEVALUESTORE_H
00009 #define SPARSEVALUESTORE_H
00010
00011 #include "../Persistence.h"
00012 #include "../util/Util.h"
00013
00014 #include <list>
00015 #include <map>
00016 #include <algorithm>
00017 #include <cctype>
00018 #include <sstream>
00019 #include <string>
00020 #include <utility>
00021 #include <vector>
00022
00023 class SparseValueStore {
00024 public:
00025     static std::string makeIndexKey(const std::vector<unsigned int>& indexes);
00026
00027     static std::string makeIndexKeyFromDoubles(const std::vector<double>& indexes);
00028
00029     static std::string appendIndexKey(const std::string& currentKey, unsigned int index);
00030
00031     static std::string appendIndexKeyFromDouble(const std::string& currentKey, double index);
00032
00033     static std::string normalizeIndexKey(const std::string& index);
00034
00035     double value(const std::string& index = "") const;
00036
00037     void setValue(double value, const std::string& index = "");
00038
00039     void setValues(const std::map<std::string, double>& values);
00040
00041     void copyFrom(const SparseValueStore& other);
00042
00043     void insertDimensionSize(unsigned int size);
00044
00045     std::list<unsigned int>* dimensionSizes();
00046
00047     const std::list<unsigned int>* dimensionSizes() const;
00048
00049     std::map<std::string, double>* values();
00050
00051     const std::map<std::string, double>* values() const;
00052
00053     void clear();
00054
00055     void loadDimensions(PersistenceRecord* fields);
00056
00057     void saveDimensions(PersistenceRecord* fields, bool saveDefaultValues) const;
00058
00059     void loadValues(PersistenceRecord* fields,
00060                     const std::string& countField,
00061                     const std::string& positionPrefix,
00062                     const std::string& valuePrefix);
00063
00064     void saveValues(PersistenceRecord* fields,
00065                     const std::string& countField,
00066                     const std::string& positionPrefix,
00067                     const std::string& valuePrefix,
00068                     bool saveDefaultValues) const;
00069
00070     std::string showValues() const;
00071
00072 private:
00073     static std::string trimCopy(const std::string& text) {
00074         std::size_t first = 0;
00075         while (first < text.size() && std::isspace(static_cast<unsigned char>(text[first]))) {
00076             ++first;
00077         }
00078         std::size_t last = text.size();
00079         while (last > first && std::isspace(static_cast<unsigned char>(text[last - 1]))) {
00080             --last;
00081         }
00082         return text.substr(first, last - first);
00083     }
00084
00085     // Returns true when every comma-separated token in index consists of digits only
00086     // (or when index is empty, representing the scalar position).
00087     static bool isNumericKey(const std::string& index) {
00088         if (index.empty()) {
00089             return true;
00090         }
00091     }

```



```

00120         for (char c : index) {
00121             if (!std::isdigit(static_cast<unsigned char>(c)) && c != ',' && c != ' ' && c != '\\t') {
00122                 return false;
00123             }
00124         }
00125         return true;
00126     }
00127
00128     static std::vector<unsigned int> parseIndex(const std::string& index) {
00129         std::vector<unsigned int> parsed;
00130         if (index.empty()) {
00131             return parsed;
00132         }
00133         std::stringstream input(index);
00134         std::string token;
00135         while (std::getline(input, token, ',')) {
00136             token = trimCopy(token);
00137             if (token.empty()) {
00138                 continue;
00139             }
00140             parsed.push_back(static_cast<unsigned int>(std::stoul(token)));
00141         }
00142         return parsed;
00143     }
00144
00145     static std::size_t productOfDimensions(const std::list<unsigned int>& dimensions) {
00146         std::size_t product = 1u;
00147         for (unsigned int dimension : dimensions) {
00148             product *= static_cast<std::size_t>(dimension);
00149         }
00150         return product;
00151     }
00152
00153     static std::vector<std::size_t> stridesFor(const std::list<unsigned int>& dimensions) {
00154         std::vector<std::size_t> strides;
00155         strides.reserve(dimensions.size());
00156         std::size_t stride = 1u;
00157         for (auto it = dimensions.rbegin(); it != dimensions.rend(); ++it) {
00158             strides.push_back(stride);
00159             stride *= static_cast<std::size_t>(*it);
00160         }
00161         std::reverse(strides.begin(), strides.end());
00162         return strides;
00163     }
00164
00165     std::size_t linearIndex(const std::vector<unsigned int>& indexes) const {
00166         if (_dimensionSizes.empty()) {
00167             return indexes.empty() ? 0u : static_cast<std::size_t>(-1);
00168         }
00169         if (indexes.size() != _dimensionSizes.size()) {
00170             return static_cast<std::size_t>(-1);
00171         }
00172         const std::vector<std::size_t> strides = stridesFor(_dimensionSizes);
00173         std::size_t linear = 0u;
00174         auto dimIt = _dimensionSizes.begin();
00175         for (std::size_t i = 0; i < indexes.size(); ++i, ++dimIt) {
00176             if (indexes[i] >= *dimIt) {
00177                 return static_cast<std::size_t>(-1);
00178             }
00179             linear += static_cast<std::size_t>(indexes[i]) * strides[i];
00180         }
00181         return linear;
00182     }
00183
00184     void rebuildDenseFromCache() {
00185         const std::size_t size = _dimensionSizes.empty() ? 1u : productOfDimensions(_dimensionSizes);
00186         _denseValues.assign(size, 0.0);
00187         for (const auto& entry : _values) {
00188             if (entry.first.empty()) {
00189                 continue;
00190             }
00191             // Skip non-numeric string keys -- they live only in the sparse map.
00192             if (!isNumericKey(entry.first)) {
00193                 continue;
00194             }
00195             const std::vector<unsigned int> indexes = parseIndex(entry.first);
00196             const std::size_t linear = linearIndex(indexes);
00197             if (linear != static_cast<std::size_t>(-1) && linear < _denseValues.size()) {
00198                 _denseValues[linear] = entry.second;
00199             }
00200         }
00201         _denseActive = true;
00202     }
00203
00204     void syncCacheFromDense() {
00205         // Preserve non-numeric string keys across the rebuild.
00206         std::map<std::string, double> stringKeyed;

```

```

00207         for (const auto& entry : _values) {
00208             if (!entry.first.empty() && !isNumericKey(entry.first)) {
00209                 stringKeyed.insert(entry);
00210             }
00211         }
00212         _values.clear();
00213         if (_hasScalarValue) {
00214             _values[""] = _scalarValue;
00215         }
00216         if (!_denseActive) {
00217             _values.insert(stringKeyed.begin(), stringKeyed.end());
00218             return;
00219         }
00220         if (_dimensionSizes.empty()) {
00221             if (!_denseValues.empty() && !_hasScalarValue) {
00222                 _values[""] = _denseValues.front();
00223             }
00224             _values.insert(stringKeyed.begin(), stringKeyed.end());
00225             return;
00226         }
00227         std::vector<unsigned int> indexes;
00228         indexes.reserve(_dimensionSizes.size());
00229         const std::vector<std::size_t> strides = stridesFor(_dimensionSizes);
00230         for (std::size_t linear = 0; linear < _denseValues.size(); ++linear) {
00231             if (_denseValues[linear] == 0.0) {
00232                 continue;
00233             }
00234             std::size_t remainder = linear;
00235             indexes.clear();
00236             for (std::size_t i = 0; i < strides.size(); ++i) {
00237                 const unsigned int index = static_cast<unsigned int>(remainder / strides[i]);
00238                 indexes.push_back(index);
00239                 remainder %= strides[i];
00240             }
00241             _values[makeIndexKey(indexes)] = _denseValues[linear];
00242         }
00243         _values.insert(stringKeyed.begin(), stringKeyed.end());
00244     }
00245
00246     void ensureDenseForIndex(const std::vector<unsigned int>& indexes) {
00247         if (indexes.empty() && _dimensionSizes.empty()) {
00248             if (!_denseActive) {
00249                 _denseActive = true;
00250                 _denseValues.assign(1u, 0.0);
00251             }
00252             return;
00253         }
00254
00255         if (_dimensionSizes.empty()) {
00256             _dimensionSizes.clear();
00257             for (unsigned int index : indexes) {
00258                 _dimensionSizes.push_back(index + 1u);
00259             }
00260             rebuildDenseFromCache();
00261             return;
00262         }
00263
00264         if (indexes.size() != _dimensionSizes.size()) {
00265             return;
00266         }
00267
00268         bool needsResize = false;
00269         auto dimIt = _dimensionSizes.begin();
00270         for (std::size_t i = 0; i < indexes.size(); ++i, ++dimIt) {
00271             if (indexes[i] >= *dimIt) {
00272                 *dimIt = indexes[i] + 1u;
00273                 needsResize = true;
00274             }
00275         }
00276         if (needsResize || !_denseActive) {
00277             rebuildDenseFromCache();
00278         }
00279     }
00280
00281     bool _denseActive = false;
00282     bool _hasScalarValue = false;
00283     double _scalarValue = 0.0;
00284     std::vector<double> _denseValues;
00285     std::list<unsigned int> _dimensionSizes;
00286     std::map<std::string, double> _values;
00287 };
00288
00289 inline std::string SparseValueStore::makeIndexKey(const std::vector<unsigned int>& indexes) {
00290     std::ostringstream key;
00291     bool first = true;
00292     for (const unsigned int index : indexes) {
00293         if (!first) {

```

```

00294         key << ", ";
00295     }
00296     first = false;
00297     key << index;
00298 }
00299 return key.str();
00300 }
00301
00302 inline std::string SparseValueStore::makeIndexKeyFromDoubles(const std::vector<double>& indexes) {
00303     std::vector<unsigned int> unsignedIndexes;
00304     unsignedIndexes.reserve(indexes.size());
00305     for (const double index : indexes) {
00306         unsignedIndexes.push_back(static_cast<unsigned int>(index));
00307     }
00308     return makeIndexKey(unsignedIndexes);
00309 }
00310
00311 inline std::string SparseValueStore::appendIndexKey(const std::string& currentKey, unsigned int index)
00312 {
00313     const std::string nextPart = std::to_string(index);
00314     if (currentKey.empty()) {
00315         return nextPart;
00316     }
00317     return currentKey + "," + nextPart;
00318 }
00319 inline std::string SparseValueStore::appendIndexKeyFromDouble(const std::string& currentKey, double
00320 index) {
00321     return appendIndexKey(currentKey, static_cast<unsigned int>(index));
00322 }
00323 inline std::string SparseValueStore::normalizeIndexKey(const std::string& index) {
00324     return index;
00325 }
00326
00327 inline double SparseValueStore::value(const std::string& index) const {
00328     if (index.empty() && _hasScalarValue) {
00329         return _scalarValue;
00330     }
00331     // Non-numeric string keys (e.g. "slot", "pi") bypass dense storage.
00332     if (!index.empty() && !isNumericKey(index)) {
00333         const auto it = _values.find(index);
00334         return (it != _values.end()) ? it->second : 0.0;
00335     }
00336     if (_denseActive) {
00337         const std::vector<unsigned int> indexes = parseIndex(index);
00338         const std::size_t linear = linearIndex(indexes);
00339         if (linear == static_cast<std::size_t>(-1) || linear >= _denseValues.size()) {
00340             // Dimension count mismatch or out of bounds -- fall back to sparse map.
00341             const auto it = _values.find(normalizeIndexKey(index));
00342             return (it != _values.end()) ? it->second : 0.0;
00343         }
00344         return _denseValues[linear];
00345     }
00346     const auto it = _values.find(normalizeIndexKey(index));
00347     if (it == _values.end()) {
00348         return 0.0;
00349     }
00350     return it->second;
00351 }
00352
00353 inline void SparseValueStore::setValue(double value, const std::string& index) {
00354     if (index.empty()) {
00355         _scalarValue = value;
00356         _hasScalarValue = true;
00357         _values[""] = value;
00358         if (_denseActive && _dimensionSizes.empty()) {
00359             if (_denseValues.empty()) {
00360                 _denseValues.assign(1u, 0.0);
00361             }
00362             _denseValues.front() = value;
00363         }
00364         return;
00365     }
00366     // Non-numeric string keys (e.g. "slot", "pi") are stored directly in the
00367     // sparse map without touching the dense storage or dimension metadata.
00368     if (!isNumericKey(index)) {
00369         _values[index] = value;
00370         return;
00371     }
00372     const std::vector<unsigned int> indexes = parseIndex(index);
00373     ensureDenseForIndex(indexes);
00374     if (_denseActive) {
00375         const std::size_t linear = linearIndex(indexes);
00376         if (linear != static_cast<std::size_t>(-1)) {
00377             if (linear >= _denseValues.size()) {
00378                 _denseValues.resize(linear + 1u, 0.0);

```

```

00379         }
00380         _denseValues[linear] = value;
00381         syncCacheFromDense();
00382         return;
00383     }
00384 }
00385 _values[normalizeIndexKey(index)] = value;
00386 }
00387
00388 inline void SparseValueStore::setValues(const std::map<std::string, double>& values) {
00389     _values = values;
00390     const auto scalarIt = _values.find("");
00391     _hasScalarValue = scalarIt != _values.end();
00392     if (_hasScalarValue) {
00393         _scalarValue = scalarIt->second;
00394     }
00395     if (_dimensionSizes.empty()) {
00396         if (_values.size() == 1u && _hasScalarValue) {
00397             _denseActive = true;
00398             _denseValues.assign(1u, _scalarValue);
00399         } else {
00400             _denseActive = false;
00401             _denseValues.clear();
00402         }
00403         return;
00404     }
00405     rebuildDenseFromCache();
00406 }
00407
00408 inline void SparseValueStore::copyFrom(const SparseValueStore& other) {
00409     *this = other;
00410 }
00411
00412 inline void SparseValueStore::insertDimensionSize(unsigned int size) {
00413     _dimensionSizes.insert(_dimensionSizes.end(), size);
00414     rebuildDenseFromCache();
00415 }
00416
00417 inline std::list<unsigned int>* SparseValueStore::dimensionSizes() {
00418     return &_dimensionSizes;
00419 }
00420
00421 inline const std::list<unsigned int>* SparseValueStore::dimensionSizes() const {
00422     return &_dimensionSizes;
00423 }
00424
00425 inline std::map<std::string, double>* SparseValueStore::values() {
00426     return &_values;
00427 }
00428
00429 inline const std::map<std::string, double>* SparseValueStore::values() const {
00430     return &_values;
00431 }
00432
00433 inline void SparseValueStore::clear() {
00434     _dimensionSizes.clear();
00435     _values.clear();
00436     _hasScalarValue = false;
00437     _scalarValue = 0.0;
00438     _denseValues.clear();
00439     _denseActive = false;
00440 }
00441
00442 inline void SparseValueStore::loadDimensions(PersistenceRecord* fields) {
00443     _dimensionSizes.clear();
00444     if (fields == nullptr) {
00445         return;
00446     }
00447     const unsigned int count = fields->loadField("dimensions", 0u);
00448     for (unsigned int i = 0; i < count; i++) {
00449         insertDimensionSize(fields->loadField("dimension" + Util::StrIndex(i), 0u));
00450     }
00451     if (!_dimensionSizes.empty()) {
00452         rebuildDenseFromCache();
00453     }
00454 }
00455
00456 inline void SparseValueStore::saveDimensions(PersistenceRecord* fields, bool saveDefaultValues) const
00457 {
00458     if (fields == nullptr) {
00459         return;
00460     }
00461     unsigned int i = 0;
00462     fields->saveField("dimensions", static_cast<unsigned int>(_dimensionSizes.size()), 0u,
00463         saveDefaultValues);
00464     for (const unsigned int dimension : _dimensionSizes) {
00465         fields->saveField("dimension" + Util::StrIndex(i), dimension, 1u, saveDefaultValues);

```

```

00464         i++;
00465     }
00466 }
00467
00468 inline void SparseValueStore::loadValues(PersistenceRecord* fields,
00469                                         const std::string& countField,
00470                                         const std::string& positionPrefix,
00471                                         const std::string& valuePrefix) {
00472     _values.clear();
00473     if (fields == nullptr) {
00474         return;
00475     }
00476     const unsigned int count = fields->loadField(countField, 0u);
00477     for (unsigned int i = 0; i < count; i++) {
00478         const std::string position = fields->loadField(positionPrefix + Util::StrIndex(i), "");
00479         const double loadedValue = fields->loadField(valuePrefix + Util::StrIndex(i), 0.0);
00480         setValue(loadedValue, position);
00481     }
00482 }
00483
00484 inline void SparseValueStore::saveValues(PersistenceRecord* fields,
00485                                         const std::string& countField,
00486                                         const std::string& positionPrefix,
00487                                         const std::string& valuePrefix,
00488                                         bool saveDefaultValues) const {
00489     if (fields == nullptr) {
00490         return;
00491     }
00492     unsigned int i = 0;
00493     fields->saveField(countField, static_cast<unsigned int>(_values.size()), 0u, saveDefaultValues);
00494     for (const auto& value : _values) {
00495         fields->saveField(positionPrefix + Util::StrIndex(i), value.first, "0", saveDefaultValues);
00496         fields->saveField(valuePrefix + Util::StrIndex(i), value.second, 0.0, saveDefaultValues);
00497         i++;
00498     }
00499 }
00500
00501 inline std::string SparseValueStore::showValues() const {
00502     if (_denseActive && _values.empty() && !_denseValues.empty() && _dimensionSizes.empty()) {
00503         return "[" + std::to_string(_denseValues.front()) + "]";
00504     }
00505     std::string text = "{";
00506     for (const auto& value : _values) {
00507         text += value.first + "=" + Util::StrTruncIfInt(std::to_string(value.second)) + ", ";
00508     }
00509     if (!_values.empty()) {
00510         text = text.substr(0, text.length() - 2);
00511     }
00512     text += "}";
00513     return text;
00514 }
00515
00516 #endif /* SPARSEVALUESTORE_H */

```

11.238 StatisticsCollector.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: StatisticsCollector.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 30 de Agosto de 2018, 17:24
00012  */
00013
00014 #ifndef STATISTICSCOLLECTOR_H
00015 #define STATISTICSCOLLECTOR_H
00016
00017 #include "../model/ModelDataDefinition.h"
00018 #include "../../statistics/Statistics_if.h"
00019 #include "../model/ModelDataManager.h"
00020 #include "../Plugin.h"
00021
00022 #include <vector>
00023
00024 //namespace GenesysKernel {
00025
00026 class StatisticsCollector : public ModelDataDefinition {
00027 public:

```

```

00031     StatisticsCollector(Model* model, std::string name = "", ModelDataDefinition* parent = nullptr,
    bool insertIntoModel = true);
00032     virtual ~StatisticsCollector() override;
00033 public:
00034     virtual std::string show() override;
00035 public:
00040     ModelDataDefinition* getParent() const;
00045     Statistics_if* getStatistics() const;
00046
00047 public:
00048     static PluginInformation* GetPluginInformation();
00049     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00050     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00051
00052 protected:
00053     virtual bool _loadInstance(PersistenceRecord *fields) override;
00054     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00055 protected:
00056     virtual bool _check(std::string& errorMessage) override;
00057
00058 private:
00059     void _initStaticsAndCollector();
00060     void _addOwnedResponse(SimulationResponse* response);
00061 private:
00062     ModelDataDefinition* _parent;
00063     Statistics_if* _statistics = nullptr;
00064     std::vector<SimulationResponse*> _ownedResponses;
00065 };
00066 //namespace\}
00067 #endif /* STATISTICSCOLLECTOR_H */

```

11.239 Event.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:     Event.h
00009  * Author:   raphael.l Luiz.Cancian
00010  *
00011  * Created on 21 de Junho de 2018, 19:41
00012  */
00013
00014 #ifndef EVENT_H
00015 #define EVENT_H
00016
00017 #include <string>
00018 #include <functional>
00019
00020 #include "model/ModelDataDefinition.h"
00021 #include "essentialPlugins/Entity.h"
00022 #include "model/ModelComponent.h"
00023
00024 //namespace GenesysKernel {
00025
00030 class Event {//: public ModelDataDefinition {
00031 public:
00032     Event(double time, Entity* entity, ModelComponent* component, unsigned int
    componentinputPortNumber = 0);
00033     Event(double time, Entity* entity, Connection* connection);
00034     virtual ~Event() = default;
00035 public:
00040     double getTime() const;
00045     ModelComponent* getComponent() const;
00050     Entity* getEntity() const;
00055     unsigned int getComponentinputPortNumber() const;
00060     void setEntity(Entity* _entity);
00061 public: // virtual
00062     virtual std::string show();
00063 protected:
00064     double _time;
00065 private:
00066     Entity* _entity;
00067     ModelComponent* _component;
00068     unsigned int _componentinputPortNumber;
00069 };
00070
00071 //***** 2022/07/05
00072
00073 typedef std::function<void(void*)> InternalEventHandler;

```

```

00074
00075 class InternalEvent: public Event { // @ToDo: (importante): BAD SW ENGINEERING Fix it
00076 public:
00077     InternalEvent(double time, std::string description): Event(time, nullptr, nullptr, 0){
00078         _description = description;
00079         _time = time;
00080     }
00081     virtual ~InternalEvent() = default;
00082
00083     template<typename Class>
00084     void setEventHandler(Class* object, void (Class::*function)(void*), void* parameter) {
00085         _object = object;
00086         _handler= std::bind(function, object, std::placeholders::_1);
00087         _parameter = parameter;
00088     }
00089
00090     virtual std::string show() override {
00091         std::string dataDefinitionName = static_cast<ModelDataDefinition*>(_object)->getName();
00092         std::string message = "time=" + std::to_string(_time) + //Util::StrTimeUnit(???) +
00093             ",description=\"\" + _description+\"\"+
00094             ",block=\"\" + dataDefinitionName+\"\";
00095         return message;
00096     }
00097
00098     const InternalEventHandler& eventHandler() const {
00099         return _handler;
00100     }
00101
00102     const std::string& description() const {
00103         return _description;
00104     }
00105
00106     void dispatchEvent() const {
00107         _handler(_parameter);
00108     }
00109
00110     void* object() const {
00111         return _object;
00112     }
00113
00114     void* parameter() const {
00115         return _parameter;
00116     }
00117
00118 private:
00119     InternalEventHandler _handler;
00120     std::string _description;
00121     void* _parameter;
00122     void* _object;
00123 };
00124
00125
00126 //namespace\\
00127 #endif /* EVENT_H */

```

11.240 ExperimentManager.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: ExperimentManager.h
00009  * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010  *
00011  * Created on 4 de maio de 2021, 11:43
00012  */
00013
00014 #ifndef EXPERIMENTMANAGER_H
00015 #define EXPERIMENTMANAGER_H
00016
00017 // #include "Simulator.h"
00018 #include "Persistence.h"
00019 // #include "SimulationControl.h"
00020 // #include "SimulationResponse.h"
00021 #include "SimulationExperiment.h"
00022 #include "../util/List.h"
00023
00024 class Simulator;
00025
00041 class ExperimentManager { // : PersistentObject_base {

```

```

00042 public:
00043
00044     /* class ModelSimulationInsOuts {
00045     public:
00046         ModelSimulationInsOuts() = default;
00047         struct ControlResponseInfos {
00048             std::string type;
00049             std::string parent;
00050             std::string name;
00051             double defaultValue;
00052         };
00053
00054         List<ControlResponseInfos*>* getResponsesInfos() const {
00055             return _responsesInfos;
00056         }
00057
00058         std::string getModelfilename() const {
00059             return _experimentfilename;
00060         }
00061
00062         List<ControlResponseInfos*>* getControlInfos() const {
00063             return _controlInfos;
00064         }
00065     private:
00066         std::string _modelfilename;
00067         List<ControlResponseInfos*>* _responsesInfos = new List<ControlResponseInfos*>();
00068         List<ControlResponseInfos*>* _controlInfos = new List<ControlResponseInfos*>();
00069     }; */
00070
00071 public:
00072     ExperimentManager(Simulator* simulator);
00073     // Release owned experiment instances and container storage during simulator shutdown.
00074     virtual ~ExperimentManager();
00075
00076 public:
00077     SimulationExperiment* newSimulationExperiment();
00078     void insert(SimulationExperiment* experiment);
00079     void remove(SimulationExperiment* experiment);
00080     void setCurrent(SimulationExperiment* experiment);
00081     // @ToDo: (importante): Implement persistence once SimulationExperiment serialization is defined.
00082     bool saveSimulationExperiment(std::string filename);
00083     // @ToDo: (importante): Implement loading once SimulationExperiment deserialization and scenario
00084     // execution are defined.
00085     bool loadSimulationExperiment(std::string filename);
00086     unsigned int size();
00087 public:
00088     SimulationExperiment* front();
00089     SimulationExperiment* current();
00090     SimulationExperiment* next();
00091     //SimulationExperiment* end();
00092 public:
00093     List<SimulationExperiment*>* getExperiments() const;
00094 private:
00095     Simulator* _simulator;
00096     SimulationExperiment* _currentSimulationExperiment;
00097     List<SimulationExperiment*>* _experiments = new List<SimulationExperiment*>();
00098 };
00099
00100 #endif /* EXPERIMENTMANAGER_H */

```

11.241 ExperimentManagerDefaultImpl1.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   ProcessAnalyserDefaultImpl1.h
00009  * Author: rlcancian
00010  *
00011  * Created on 20 de Maio de 2019, 20:45
00012  */
00013
00014 #ifndef PROCESSANALYSERDEFAULTIMPL1_H
00015 #define PROCESSANALYSERDEFAULTIMPL1_H
00016
00017 #include "ExperimetManager_if.h"
00018 #include "SimulationScenario.h"
00019
00020 class ExperimentManagerDefaultImpl1 : public ExperimentManager_if {

```



```

00021 public:
00022     ExperimentManagerDefaultImpl();
00023     virtual ~ExperimentManagerDefaultImpl() = default;
00024 public:
00025     virtual List<SimulationScenario*>* getScenarios() const override;
00026     //virtual List<SimulationControl*>* getControls() const;
00027     //virtual List<SimulationResponse*>* getResponses() const;
00028     //virtual List<SimulationControl*>* extractControlsFromModel(std::string modelName) const;
00029     //virtual List<SimulationResponse*>* extractResponsesFromModel(std::string modelName) const;
00030     virtual void startSimulationOfScenario(SimulationScenario* scenario) override;
00031     virtual void startExperiment() override;
00032     virtual void stopExperiment() override;
00033     virtual void addTraceSimulationHandler(traceSimulationProcessListener
        traceSimulationProcessListener) override;
00034 private:
00035     //List<SimulationControl*>* _controls = new List<SimulationControl*>();
00036 };
00037
00038 #endif /* PROCESSANALYSERDEFAULTIMPL_H */

```

11.242 ExperimentManager_if.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: ExperimentManager_if (old ProcessAnalyser_if.h)
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 10 de Outubro de 2018, 14:26
00012  */
00013
00014 #ifndef EXPERIMENTMANAGER_IF_H
00015 #define EXPERIMENTMANAGER_IF_H
00016
00017 #include "../util/List.h"
00018 #include "SimulationScenario.h"
00019 #include "PropertyGenesys.h"
00020 #include "TraceManager.h"
00021
00022 class ExperimentManager_if {
00023 public:
00024     virtual ~ExperimentManager_if() = default;
00025     virtual List<SimulationScenario*>* getScenarios() const = 0;
00026     //virtual List<SimulationControl*>* getControls() const = 0;
00027     //virtual List<SimulationResponse*>* getResponses() const = 0;
00028     //virtual List<SimulationControl*>* extractControlsFromModel(std::string modelName) const = 0;
00029     //virtual List<SimulationResponse*>* extractResponsesFromModel(std::string modelName) const =
00030     0;
00031     virtual void startSimulationOfScenario(SimulationScenario* scenario) = 0;
00032     virtual void startExperiment() = 0;
00033     virtual void stopExperiment() = 0;
00034     virtual void addTraceSimulationHandler(traceSimulationProcessListener
        traceSimulationProcessListener) = 0;
00035 };
00036
00037 #endif /* EXPERIMENTMANAGER_IF_H */

```

11.243 GenesysPropertyIntrospection.h

```

00001 #pragma once
00002
00003 #include <string>
00004 #include <vector>
00005
00006 #include "SimulationControlAndResponse.h"
00007 #include "../util/Util.h"
00008
00009 enum class GenesysPropertyKind {
00010     Unknown,
00011     Boolean,
00012     Integer,
00013     UnsignedInteger,
00014     UnsignedShort,
00015     Double,
00016     String,

```

```

00017     TimeUnit,
00018     Enum,
00019     Object,
00020     List
00021 };
00022
00023 struct GenesysPropertyDescriptor {
00024     SimulationControl* control = nullptr;
00025
00026     std::string ownerClassName;
00027     std::string ownerElementName;
00028     std::string displayName;
00029     std::string technicalTypeName;
00030
00031     GenesysPropertyKind kind = GenesysPropertyKind::Unknown;
00032
00033     bool readOnly = true;
00034     bool isList = false;
00035     bool isClass = false;
00036     bool isEnum = false;
00037     bool isInlineObject = false;
00038     bool isModelDataDefinitionReference = false;
00039     bool supportsInlineExpansion = false;
00040     bool supportsListEditor = false;
00041     bool supportsExistingObjectSelection = false;
00042     bool supportsObjectCreation = false;
00043     bool supportsNewListElementCreation = false;
00044     SimulationControlEditorHint editorHint = SimulationControlEditorHint::Default;
00045
00046     std::string currentValue;
00047     std::vector<std::string> choices;
00048     std::string currentReferenceType;
00049     std::vector<std::string> creatableReferenceTypes;
00050     // For polymorphic list controls, this is the currently selected concrete element type.
00051     //
00052     // Example: a Set physically stores ModelDataDefinition* but may currently be a Set of Resource.
00053     // Empty means the list has not been typed yet.
00054     std::string currentListElementType;
00055     // For polymorphic list controls, these are the concrete element types that the kernel allows
00056     // the GUI to select/create. Empty means the list should use its legacy single-type behavior.
00057     std::vector<std::string> creatableListElementTypes;
00058 };
00059
00060 class GenesysPropertyIntrospection {
00061 public:
00062     static GenesysPropertyDescriptor describe(SimulationControl* control);
00063     static std::vector<GenesysPropertyDescriptor> describe(List<SimulationControl*>* controls);
00064
00065     static GenesysPropertyKind deduceKind(const SimulationControl* control);
00066
00067     static bool setValue(
00068         SimulationControl* control,
00069         const std::string& value,
00070         bool remove = false,
00071         std::string* errorMessage = nullptr
00072     );
00073
00074     static std::vector<SimulationControl*> children(
00075         SimulationControl* control,
00076         int index = 0
00077     );
00078 };

```

11.244 LicenceManager.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   LicenceManager.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 29 de Maio de 2019, 11:45
00012  */
00013
00014 #ifndef LICENCEMANAGER_H
00015 #define LICENCEMANAGER_H
00016
00017 #include <string>
00018

```

```

00019 //namespace GenesysKernel {
00020 class Simulator;
00021
00025 class LicenceManager {
00026 public:
00027     LicenceManager(Simulator* simulator);
00028     virtual ~LicenceManager() = default;
00029     const std::string showLicence() const;
00030     const std::string showLimits() const;
00031     const std::string showActivationCode() const;
00032     bool lookforActivationCode();
00033     bool insertActivationCode();
00034     void removeActivationCode();
00035     unsigned int getModelComponentsLimit();
00036     unsigned int getModelDatasLimit();
00037     unsigned int getEntityLimit();
00038     unsigned int getHostsLimit();
00039     unsigned int getThreadsLimit();
00040 private:
00041     void setDefaultLicenceAndLimits();
00042 private:
00043     Simulator* _simulator;
00044     std::string _licence;
00045     std::string _activationCode;
00046     unsigned int _components, _datadeclarations, _entities, _hosts, _threads;
00047 };
00048 //namespace\}
00049 #endif /* LICENCEMANAGER_H */
00050

```

11.245 Model.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:      SimulationModel.h
00009  * Author:    Prof. Rafael Luiz Cancian, Dr. Eng.
00010  *
00011  * Created on 21 de Junho de 2018, 15:01
00012  */
00013
00014 #ifndef SIMULATIONMODEL_H
00015 #define SIMULATIONMODEL_H
00016
00017 #include <list>
00018 #include <string>
00019
00020 #include "../util/List.h"
00021 #include "ModelComponent.h"
00022 #include "../Event.h"
00023 #include "ModelChecker_if.h"
00024 #include "../Parser_if.h"
00025 #include "../persistence/Persistence_if.h"
00026 #include "ModelDataManager.h"
00027 #include "ModelComponentManager.h"
00028 #include "../TraceManager.h"
00029 #include "../OnEventManager.h"
00030 #include "ModelInfo.h"
00031 #include "ModelSimulation.h"
00032 #include "../SimulationControlAndResponse.h"
00033
00034 //namespace GenesysKernel {
00035 class Simulator;
00036
00043 class Model {
00044 public:
00050     Model(Simulator* simulator, unsigned int level = 0);
00054     virtual ~Model();
00055 public: // model control
00056     //void showReports();
00062     bool save(std::string filename);
00068     bool load(std::string filename);
00076     bool check();
00080     void clear();
00085     std::string showLanguage();
00089     void show();
00097     bool insert(ModelDataDefinition* elemOrComp);
00104     void remove(ModelDataDefinition* elemOrComp);
00110     std::list<ModelDataDefinition*> collectDataDefinitionsRemovedWith(const
std::list<ModelDataDefinition*>& roots) const;

```

```

00117     Entity* createEntity(std::string name, bool insertIntoModel = true);
00122     void removeEntity(Entity* entity); //, bool collectStatistics);
00131     void sendEntityToComponent(Entity* entity, Connection* connection, double timeDelay = 0.0);
00141     void sendEntityToComponent(Entity* entity, ModelComponent* component, double timeDelay = 0.0,
    unsigned int componentInputPortNumber = 0);
00149     double parseExpression(const std::string expression);
00160     double parseExpression(const std::string expression, bool& success, std::string& errorMessage);
00170     bool checkExpression(const std::string expression, const std::string expressionName, std::string&
    errorMessage);
00176     void checkReferencesToDataDefinitions(std::string expression, std::map<std::string,
    std::list<std::string>*> referencedDataDefinitions);
00177
00178 public: // only gets
00183     Util::identification getId() const;
00188     bool hasChanged() const;
00193     void setHasChanged(bool hasChanged);
00194     // 1:1
00202     OnEventManager* getOnEventManager() const;
00210     ModelDataManager* getDataManager() const;
00215     ModelComponentManager* getComponentManager() const;
00220     ModelInfo* getInfos() const;
00225     Simulator* getParentSimulator() const;
00231     ModelSimulation* getSimulation() const;
00232     // 1:n
00241     List<Event*>* getFutureEvents() const;
00249     List<SimulationControl*>* getControls() const;
00257     List<SimulationResponse*>* getResponses() const;
00258
00259 public: // gets and sets
00264     void setTracer(TraceManager* _tracerManager);
00269     TraceManager* getTracer() const;
00274     Persistence_if* getPersistence() const;
00281     unsigned int getLevel() const;
00282
00283 private:
00284     void _showConnections() const;
00285     void _showComponents() const;
00286     void _showElements() const;
00287     void _showSimulationControls() const;
00288     void _showSimulationResponses() const;
00289     void _destroyFutureEvents();
00290     void _destroyTransientEntities();
00291     void _destroyComponents();
00292     void _destroyModelDataDefinitions();
00293     void _clearOrphanedDataDefinitions();
00294     void _createInternalDataDefinitions();
00295 private:
00296     bool _hasChanged = false;
00297 private: // read only public access (gets)
00298     Util::identification _id;
00299     unsigned int _level = 0;
00300     Simulator* _parentSimulator;
00301     // 1:1 (associated classes)
00302     TraceManager* _tracerManager;
00303     OnEventManager* _eventManager;
00304     ModelDataManager* _modelDataManager;
00305     ModelComponentManager* _componentManager;
00306     ModelInfo* _modelInfo;
00307     ModelSimulation* _simulation;
00308     Persistence_if* _modelPersistence;
00309
00310     // 1:n
00311     //List<ModelComponent*> _components;
00312     List<Event*> _futureEvents;
00313     // for process analyser
00314     List<SimulationResponse*> _responses;
00315     List<SimulationControl*> _controls;
00316
00317 private: // no public access (no gets / sets)
00318     ModelChecker_if* _modelChecker;
00319     Parser_if* _parser;
00320 };
00321 //namespace\\}
00322 #endif /* SIMULATIONMODEL_H */

```

11.246 ModelChecker_if.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006

```

```

00007  /*
00008  * File:    ModelChecker_if.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 23 de Agosto de 2018, 15:48
00012  */
00013
00014  #ifndef MODELCHECKER_IF_H
00015  #define MODELCHECKER_IF_H
00016
00017  #include <typeinfo>
00018  #include <string>
00019
00020  // #include "Model.h"
00021  // class Model;
00022
00026  class ModelChecker_if {
00027  public:
00028      virtual ~ModelChecker_if() = default;
00033      virtual bool checkAll() = 0;
00038      virtual bool checkConnected() = 0;
00043      virtual bool checkSymbols() = 0;
00048      virtual bool checkActivationCode() = 0;
00053      virtual bool checkLimits() = 0;
00054  };
00055
00056  #endif /* MODELCHECKER_IF_H */

```

11.247 ModelCheckerDefaultImpl1.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:    ModelCheckerDefaultImpl1.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 8 de Agosto de 2018, 18:44
00012  */
00013
00014  #ifndef MODELCHECKERDEFAULTIMPL1_H
00015  #define MODELCHECKERDEFAULTIMPL1_H
00016
00017  #include "ModelChecker_if.h"
00018  #include "Model.h"
00019  #include "../PluginManager.h"
00020
00021  // namespace GenesysKernel {
00022
00023  class ModelCheckerDefaultImpl1 : public ModelChecker_if {
00024  public:
00025      ModelCheckerDefaultImpl1(Model* model);
00026      virtual ~ModelCheckerDefaultImpl1() = default;
00027  public:
00028      virtual bool checkAll() override;
00029      virtual bool checkConnected() override;
00030      virtual bool checkSymbols() override;
00031      virtual bool checkActivationCode() override;
00032      virtual bool checkLimits() override;
00033  private:
00034      void _recursiveConnectedTo(PluginManager* pluginManager, ModelComponent* comp,
00035                                List<ModelComponent*>* visited, List<ModelComponent*>* unconnected, bool* drenofound);
00036      void _showResult(bool result, std::string checking);
00037  private:
00038      Model* _model;
00039  };
00040  // namespace\\}
00041  #endif /* MODELCHECKERDEFAULTIMPL1_H */

```

11.248 ModelComponent.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.

```

```

00005  */
00006
00007  /*
00008   * File:    Element.h
00009   * Author:  rafael.luiz.cancian
00010   *
00011   * Created on 21 de Junho de 2018, 15:56
00012   */
00013
00014  #ifndef MODELCOMPONENT_H
00015  #define MODELCOMPONENT_H
00016
00017  #include <string>
00018  #include <list>
00019
00020  #include "../Plugin.h"
00021  #include "../util/List.h"
00022  #include "../essentialPlugins/Entity.h"
00023  #include "ModelDataDefinition.h"
00024  #include "../ConnectionManager.h"
00025  //namespace GenesysKernel {
00026
00027  class Model;
00028  class Event;
00029
00034  class ModelComponent : public ModelDataDefinition {
00035  public:
00042      ModelComponent(Model* model, std::string componentTypeName, std::string name = "");
00046      virtual ~ModelComponent();
00047
00048  public:
00059      ConnectionManager* getConnectionManager() const;
00065      void connectTo(ModelComponent* component, unsigned int inputPortNumber = 0);
00070      bool hasBreakpointAt();
00075      void setDescription(std::string _description);
00080      std::string getDescription() const;
00081
00086      void remove(ModelComponent * comp);
00087      // ...
00088
00089  public:
00094      virtual std::string show() override;
00095
00096  public:
00103      static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00104      // new static methods for all ModelComponents
00110      static bool Check(ModelComponent* component);
00115      static void CreateInternalData(ModelComponent* component);
00121      static void SaveInstance(PersistenceRecord *fields, ModelComponent* component);
00129      static void DispatchEvent(Event* event);
00130
00131  protected:
00132      virtual bool _loadInstance(PersistenceRecord *fields) override;
00133      virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00134      // new virtual methods for all ModelComponents
00142      virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) = 0;
00143
00144  protected:
00146      // virtual bool _check(std::string& errorMessage);
00148      // virtual ParserChangesInformation* _getParserChangesInformation();
00150      // virtual void _initBetweenReplications();
00152      // virtual void _createInternalAndAttachedData(); /* legacy combined lifecycle hook */
00154      // virtual void _addSimulationControl(SimulationControl* property);
00155
00156  protected: // new protected attributes for all ModelComponents
00157      ConnectionManager* _connections = new ConnectionManager();
00158
00159  private:
00160      // ...
00161
00162  private:
00163
00164      // Default values for the attributes. Used on initing, loading and saving
00165      const struct DEFAULT_VALUES {
00166          const unsigned int nextSize = 1; // No need for attribute. Taken from
00167          _connections
00167          const unsigned int nextinputPortNumber = 0;
00168          const std::string description = "";
00169      } DEFAULT;
00170      std::string _description = DEFAULT.description;
00171
00172  private:
00173      //...
00174
00175  private:
00176      // ...
00177

```

```

00178 };
00179 //namespace\\}
00180 #endif /* MODELCOMPONENT_H */

```

11.249 ModelComponentManager.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   ModelComponentManager.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 28 de Maio de 2019, 10:41
00012  */
00013
00014 #ifndef COMPONENTMANAGER_H
00015 #define COMPONENTMANAGER_H
00016
00017 #include "ModelComponent.h"
00018 #include "../SourceModelComponent.h"
00019
00020 //namespace GenesysKernel {
00021 //class Model;
00022
00023 class ModelComponentManager {
00024 public:
00025     ModelComponentManager(Model* model);
00026     virtual ~ModelComponentManager();
00027 public:
00028     bool insert(ModelComponent* comp);
00029     void remove(ModelComponent* comp);
00030     ModelComponent* find(std::string name);
00031     ModelComponent* find(Util::identification id);
00032     void clear();
00033 public:
00034     unsigned int getNumberOfComponents();
00035     std::list<ModelComponent*>::iterator begin();
00036     std::list<ModelComponent*>::iterator end();
00037     ModelComponent* front();
00038     ModelComponent* next();
00039     bool hasChanged() const;
00040     void setHasChanged(bool _hasChanged);
00041 public:
00042     std::list<SourceModelComponent*>* getSourceComponents();
00043     std::list<ModelComponent*>* getTransferInComponents();
00044     std::list<ModelComponent*>* getAllComponents() const;
00045 private:
00046     List<ModelComponent*>* _components;
00047     Model* _parentModel;
00048     bool _hasChanged = false;
00049     std::list<ModelComponent*>::iterator _componentIterator;
00050 };
00051 //namespace\\}
00052 #endif /* COMPONENTMANAGER_H */

```

11.250 ModelDataDefinition.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   ModelDataDefinition.h
00009  * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010  *
00011  * Created on 21 de Junho de 2018, 19:40
00012  */
00013
00014 #ifndef MODELELEMENT_H
00015 #define MODELELEMENT_H
00016
00017 #include <string>
00018 #include <list>

```

```

00019 #include <vector>
00020 #include <map>
00021
00022 #include "../util/List.h"
00023 #include "../ParserChangesInformation.h"
00024 #include "../Persistence.h"
00025 // #include "PropertyGenesys.h"
00026 #include "ModelDataDefinitionAssociations.h"
00027 #include "../SimulationControlAndResponse.h"
00028 #include "../TraceManager.h"
00029
00030 // namespace GenesysKernel {
00031 class Model;
00032
00033
00034 class ModelDataDefinition : PersistentObject_base {
00035 public:
00036     ModelDataDefinition(Model* model, std::string datadefinitionTypename, std::string name = "",
00037         bool insertIntoModel = true);
00038     // ModelDataDefinition(Model* model, std::string datadefinitionTypename, std::string name = "",
00039     // bool insertIntoModel = true);
00040     // ModelDataDefinition(const ModelDataDefinition &orig);
00041     virtual ~ModelDataDefinition();
00042
00043 public: // get & set
00044     Util::identification getId() const;
00045     void setName(const std::string& name);
00046     const std::string& getName() const;
00047     void setLabel(const std::string& label);
00048     const std::string& getLabel() const;
00049     const std::string& getClassname() const;
00050     Model* getParentModel() const;
00051     bool isReportStatistics() const;
00052     void setReportStatistics(bool reportStatistics);
00053
00054 public:
00055     ModelDataDefinition* getInternalData(const std::string& name) const;
00056     std::map<std::string, ModelDataDefinition*>* getInternalData() const;
00057     std::map<std::string, ModelDataDefinition*>* getAttachedData() const;
00058     // ModelDataDefinition* getInternalData(std::string key) const;
00059     bool hasChanged() const;
00060     void setHasChanged(bool hasChanged);
00061     unsigned int getLevel() const;
00062     void setModelLevel(unsigned int _modelLevel);
00063     List<SimulationControl*>* getSimulationControls() const;
00064     TraceManager::Level getTraceLevelSpecific() const;
00065     void setTraceLevelSpecific(TraceManager::Level level);
00066     void defineTraceLevelSpecific(TraceManager::Level traceLevelspecific, bool
00067         traceLevelSpecificEnabled = true);
00068     bool isTraceLevelSpecificEnabled() const;
00069     void setTraceLevelSpecificEnabled(bool traceLevelSpecificEnabled);
00070
00071 public: // public static methods
00072     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields, bool
00073         insertIntoModel);
00074     // @ToDo: (pequena alteração): return ModelComponent* ?
00075     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00076     static void SaveInstance(PersistenceRecord* fields, ModelDataDefinition* modeldatum);
00077     static bool Check(ModelDataDefinition* modeldatum, std::string& errorMessage);
00078     static void InitBetweenReplications(ModelDataDefinition* modeldatum);
00079     // static PluginInformation* GetPluginInformation();
00080
00081 public:
00082     static void CreateInternalData(ModelDataDefinition* modeldatum);
00083     static bool CreateRelatedDataElements(ModelDataDefinition* modeldatum, std::string& errorMessage);
00084     /* This class method is responsible for invoking the protected method _initBetweenReplication(),
00085     which clears all statistics, attributes, counters and other stuff before starting a new replication */
00086
00087 public: // public virtual methods
00088     virtual std::string show();
00089 protected:
00090     bool _getSaveDefaultsOption();
00091
00092 protected:
00093     virtual bool _loadInstance(PersistenceRecord* fields) override;
00094     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00095
00096 protected:
00097     virtual bool _check(std::string& errorMessage);
00098     virtual ParserChangesInformation* _getParserChangesInformation();
00099     virtual void _initBetweenReplications();
00100     /*< A ModelDataDefinition or ModelComponent that includes (internal) ou refers to (attach) other
00101     ModelDataDefinition must register them inside this method. */
00102     virtual void _addSimulationControl(SimulationControl* control);
00103     // virtual void _addSimulationResponse(SimulationControl* response);
00104
00105 /*@ToDo: Fix this mess about diffent kind of internal and attached data related to report statistics,

```



```

00271  * mandatory and optional editable internal or attached dataelements that are properties,
00272  * and, I don't know, maybe other kind of internal and attached nedded stuff?
00273  */
00274 protected:
00276     virtual void _createInternalAndAttachedData(); // final; //@Todo: will be final
00277 protected:
00278     virtual void _createInternalStatisticReporters();
00279     virtual void _createAttachedAttributes();
00280     virtual void _createNonEditableDataDefinitions();
00281     virtual void _createEditableDataDefinitions();
00282 protected:
00283     void _templateCreateInternalStatisticReporters();
00284     void _templateCreateAttachedAttributes();
00285     void _templateCreateNonEditableDataDefinitions();
00286     void _templateCreateEditableDataDefinitions();
00287 protected:
00288     void _internalStatisticReportersClear();
00289     void _internalDataClear();
00290     void _internalDataInsert(const std::string& key, ModelDataDefinition* child);
00291     void _internalDataRemove(const std::string& key);
00292     void _attachedDataClear();
00293     void _attachedDataInsert(const std::string& key, ModelDataDefinition* data);
00294     void _attachedDataRemove(const std::string& key);
00295     void _attachedAttributesInsert(const std::vector<std::string>& neededNames);
00296     void _statisticReporterInsert(const std::string& key, ModelDataDefinition* data);
00297     void _statisticReporterRemove(const std::string& key);
00298     void _statisticReportersClear();
00299     void _mandatoryAttachedAttributeInsert(const std::string& key, ModelDataDefinition* data);
00300     void _mandatoryAttachedAttributeRemove(const std::string& key);
00301     void _mandatoryAttachedAttributesClear();
00302     void _mandatoryEditableDataDefinitionInsert(const std::string& key, ModelDataDefinition* data);
00303     void _mandatoryEditableDataDefinitionRemove(const std::string& key);
00304     void _mandatoryEditableDataDefinitionsClear();
00305     void _optionalEditableDataDefinitionInsert(const std::string& key, ModelDataDefinition* data);
00306     void _optionalEditableDataDefinitionRemove(const std::string& key);
00307     void _optionalEditableDataDefinitionsClear();
00308     void _mandatoryNonEditableDataDefinitionInsert(const std::string& key, ModelDataDefinition* data);
00309     void _mandatoryNonEditableDataDefinitionRemove(const std::string& key);
00310     void _mandatoryNonEditableDataDefinitionsClear();
00311     void _optionalNonEditableDataDefinitionInsert(const std::string& key, ModelDataDefinition* data);
00312     void _optionalNonEditableDataDefinitionRemove(const std::string& key);
00313     void _optionalNonEditableDataDefinitionsClear();
00314 protected:
00316     void _checkCreateAttachedReferencedDataDefinition(const std::string& expression);
00317     //(std::map<std::string, std::list<std::string>*>* referencedDataDefinitions);
00318
00319
00320 private:
00321     // name is now private. So changes in name must be throught setName, wich gives oportunity to
    rename internalElements, SimulationControls and SimulationResponses
00322     std::string _name;
00323     std::string _label;
00324
00325 private:
00326     friend class ModelDataDefinitionAssociations;
00327     ModelDataDefinitionAssociations _dataAssociations;
00328
00329 protected:
00330     Util::identification _id;
00331     std::string _typename;
00332     bool _reportStatistics;
00333     bool _hasChanged;
00334     unsigned int _modelLevel = 0; // the ID of parent component (submodel or process, for now) in the
    "superlevel"
00335     Model* _parentModel;
00336
00337 private:
00338     bool _checkSpecificTraceLevel(TraceManager::Level level);
00339
00340 protected:
00341     void trace(const std::string& text, TraceManager::Level level = TraceManager::Level::L8_detailed);
00342     void traceError(const std::string& text, const std::exception& e);
00343     void traceError(const std::string& text, TraceManager::Level level =
    TraceManager::Level::L1_errorFatal);
00344     void traceReport(const std::string& text, TraceManager::Level level =
    TraceManager::Level::L2_results);
00345     void traceSimulation(void* thisobject, double time, Entity* entity, ModelComponent* component,
    const std::string& text,
00346         TraceManager::Level level = TraceManager::Level::L8_detailed);
00355     void traceSimulation(void* thisobject, const std::string& text,
00356         TraceManager::Level level = TraceManager::Level::L8_detailed);
00357     void traceSimulation(void* thisobject, TraceManager::Level level, const std::string& text);
00358
00359 protected:
00360     //List<SimulationControl*>* _simulationResponses = new List<SimulationControl*>();
00361     List<SimulationControl*>* _simulationControls = new List<SimulationControl*>();
00362     TraceManager::Level _traceLevelSpecific;

```

```

00363     bool _traceLevelSpecificEnabled = false;
00364     //PropertyListG* _propertiesG = new PropertyListG();
00365 };
00366
00367 //namespace\}
00368
00369 #endif /* MODELELEMENT_H */

```

11.251 ModelDataDefinitionAssociations.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   ModelDataDefinitionAssociations.h
00009  * Author: Codex
00010  *
00011  * Created on 6 de Maio de 2026
00012  */
00013
00014 #ifndef MODELDATEDEFINITIONASSOCIATIONS_H
00015 #define MODELDATEDEFINITIONASSOCIATIONS_H
00016
00017 #include <map>
00018 #include <string>
00019 #include <vector>
00020
00021 class Attribute;
00022 class Model;
00023 class ModelDataDefinition;
00024
00032 class ModelDataDefinitionAssociations {
00033 public:
00034     explicit ModelDataDefinitionAssociations(ModelDataDefinition* owner);
00035     ~ModelDataDefinitionAssociations();
00036
00037 public:
00038     std::map<std::string, ModelDataDefinition*>* getInternalData() const;
00039     std::map<std::string, ModelDataDefinition*>* getAttachedData() const;
00040     ModelDataDefinition* getInternalData(const std::string& key) const;
00041
00042     void clearAll();
00043     void clearInternalData();
00044     void clearAttachedData();
00045
00046     void internalDataInsert(const std::string& key, ModelDataDefinition* data);
00047     void internalDataRemove(const std::string& key);
00048     void attachedDataInsert(const std::string& key, ModelDataDefinition* data);
00049     void attachedDataRemove(const std::string& key);
00050     void attachedAttributesInsert(const std::vector<std::string>& neededNames);
00051     void checkCreateAttachedReferencedDataDefinition(const std::string& expression);
00052
00053     void createInternalAndAttachedData();
00054     void createInternalStatisticReporters();
00055     void createAttachedAttributes();
00056     void createNonEditableDataDefinitions();
00057     void createEditableDataDefinitions();
00058
00059     void insertStatisticReporter(const std::string& key, ModelDataDefinition* data);
00060     void removeStatisticReporter(const std::string& key);
00061     void clearStatisticReporters();
00062
00063     void insertMandatoryAttachedAttribute(const std::string& key, ModelDataDefinition* data);
00064     void removeMandatoryAttachedAttribute(const std::string& key);
00065     void clearMandatoryAttachedAttributes();
00066
00067     void insertMandatoryEditableDataDefinition(const std::string& key, ModelDataDefinition* data);
00068     void removeMandatoryEditableDataDefinition(const std::string& key);
00069     void clearMandatoryEditableDataDefinitions();
00070
00071     void insertOptionalEditableDataDefinition(const std::string& key, ModelDataDefinition* data);
00072     void removeOptionalEditableDataDefinition(const std::string& key);
00073     void clearOptionalEditableDataDefinitions();
00074
00075     void insertMandatoryNonEditableDataDefinition(const std::string& key, ModelDataDefinition* data);
00076     void removeMandatoryNonEditableDataDefinition(const std::string& key);
00077     void clearMandatoryNonEditableDataDefinitions();
00078
00079     void insertOptionalNonEditableDataDefinition(const std::string& key, ModelDataDefinition* data);
00080     void removeOptionalNonEditableDataDefinition(const std::string& key);

```

```

00081     void clearOptionalNonEditableDataDefinitions();
00082
00083 private:
00084     void _deleteOwnedInternalData(ModelDataDefinition* data);
00085     void _rebuildLegacyViews();
00086
00087 private:
00088     ModelDataDefinition* _owner = nullptr;
00089     std::map<std::string, ModelDataDefinition*> _internalData;
00090     std::map<std::string, ModelDataDefinition*> _attachedData;
00091     std::map<std::string, ModelDataDefinition*> _statisticReporters;
00092     std::map<std::string, ModelDataDefinition*> _mandatoryAttachedAttributes;
00093     std::map<std::string, ModelDataDefinition*> _mandatoryEditableDataDefinitions;
00094     std::map<std::string, ModelDataDefinition*> _optionalEditableDataDefinitions;
00095     std::map<std::string, ModelDataDefinition*> _mandatoryNonEditableDataDefinitions;
00096     std::map<std::string, ModelDataDefinition*> _optionalNonEditableDataDefinitions;
00097 };
00098
00099 #endif /* MODELDATADEFINITIONASSOCIATIONS_H */

```

11.252 ModelDataManager.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:      ElementManager.h
00009  * Author:    rafael.luiiz.cancian
00010  *
00011  * Created on 7 de Novembro de 2018, 12:48
00012  */
00013
00014 #ifndef ELEMENTMANAGER_H
00015 #define ELEMENTMANAGER_H
00016
00017 #include <list>
00018 #include <map>
00019 #include "../util/List.h"
00020 #include "ModelDataDefinition.h"
00021
00022 namespace GenesysKernel {
00023 class Model;
00024
00031 class ModelDataManager {
00032 public:
00034     ModelDataManager(Model* model);
00035     virtual ~ModelDataManager();
00036 public:
00042     bool insert(ModelDataDefinition* anElement);
00047     void remove(ModelDataDefinition* anElement);
00054     bool insert(std::string datadefinitionTypename, ModelDataDefinition* anElement);
00060     void remove(std::string datadefinitionTypename, ModelDataDefinition* anElement);
00069     bool check(std::string datadefinitionTypename, ModelDataDefinition* anElement, std::string
expressionName, std::string& errorMessage);
00079     bool check(std::string datadefinitionTypename, std::string elementName, std::string
expressionName, bool mandatory, std::string& errorMessage);
00083     void clear();
00084 public:
00091     ModelDataDefinition* getDataDefinition(std::string datadefinitionTypename, Util::identification
id);
00098     ModelDataDefinition* getDataDefinition(std::string datadefinitionTypename, std::string name);
00104     unsigned int getNumberOfDataDefinitions(std::string datadefinitionTypename);
00109     unsigned int getNumberOfDataDefinitions();
00116     int getRankOf(std::string datadefinitionTypename, std::string name);
00121     std::list<std::string> getDataDefinitionClassnames() const;
00122
00123     //private:
00124 public:
00125     // @ToDo: (importante): Should be removed and replaced by GetElement(elementType, rank)
00126     List<ModelDataDefinition*> getDataDefinitionList(std::string datadefinitionTypename) const;
00127 public:
00129     void show();
00131     Model* getParentModel() const;
00133     bool hasChanged() const;
00135     void setHasChanged(bool _hasChanged);
00136 private:
00137     std::map<std::string, List<ModelDataDefinition*>*> _datadefinitions;
00138     Model* _parentModel;
00139     bool _hasChanged = false;
00140 };

```

```

00141 //namespace\\}
00142 #endif /* ELEMENTMANAGER_H */

```

11.253 ModelInfo.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   ModelInfo.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 7 de Novembro de 2018, 18:13
00012  */
00013
00014 #ifndef MODELINFO_H
00015 #define MODELINFO_H
00016
00017 #include <string>
00018 #include "../util/Util.h"
00019 #include "../Persistence.h"
00020
00021 //namespace GenesysKernel {
00022
00026 class ModelInfo {
00027 public:
00028     ModelInfo();
00029     virtual ~ModelInfo() = default;
00030 public:
00031     std::string show();
00032 public: // gets and sets
00037     void setName(std::string _name);
00042     std::string getName() const;
00047     void setAnalystName(std::string _analystName);
00052     std::string getAnalystName() const;
00057     void setDescription(std::string _description);
00062     std::string getDescription() const;
00067     void setProjectTitle(std::string _projectTitle);
00072     std::string getProjectTitle() const;
00077     void setVersion(std::string _version);
00082     std::string getVersion() const;
00083 public:
00088     void loadInstance(PersistenceRecord *fields);
00093     void saveInstance(PersistenceRecord *fields);
00098     bool hasChanged() const;
00100     void setHasChanged(bool hasChanged);
00101 private: // with public access (get & set)
00102     // model general information
00103     std::string _name;
00104     std::string _analystName = "";
00105     std::string _description = "";
00106     std::string _projectTitle = "";
00107     std::string _version = "1.0";
00108     bool _hasChanged = false;
00109 };
00110 //namespace\\}
00111 #endif /* MODELINFO_H */

```

11.254 ModelManager.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   ModelManager.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 31 de Maio de 2019, 08:37
00012  */
00013
00014 #ifndef MODELMANAGER_H
00015 #define MODELMANAGER_H
00016

```

```

00017 #include "Model.h"
00018 #include "../TraceManager.h"
00019 #include <vector>
00020
00021 //namespace GenesysKernel {
00022
00026 class ModelManager {
00027 public:
00032     ModelManager(Simulator* simulator);
00036     virtual ~ModelManager();
00037 public:
00042     Model* newModel();
00047     void insert(Model* model);
00052     void remove(Model* model);
00058     bool setCurrent(Model* model);
00064     bool saveModel(std::string filename);
00070     Model* loadModel(std::string filename);
00076     Model* createFromLanguage(std::string modelSpecification);
00081     unsigned int size();
00082 public:
00087     std::vector<Model*> models() const;
00093     bool hasModel(Model* model) const;
00099     int indexOf(Model* model) const;
00105     Model* modelAt(unsigned int index) const;
00110     int currentIndex() const;
00115     Model* front();
00120     Model* last();
00125     Model* current();
00130     Model* next();
00135     Model* previous();
00140     bool canGoNext() const;
00145     bool canGoPrevious() const;
00146     //Model* end();
00147 private:
00148     List<Model*>* _models = new List<Model*>();
00149     Model* _currentModel = nullptr;
00150 private:
00151     Simulator* _simulator;
00152 };
00153 //namespace\\}
00154 #endif /* MODELMANAGER_H */

```

11.255 ModelSimulation.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:     ModelSimulation.h
00009  * Author:   Prof. Rafael Luiz Cancian, Dr. Eng.
00010  *
00011  * Created on 7 de Novembro de 2018, 18:04
00012  */
00013
00014 #ifndef MODELSIMULATION_H
00015 #define MODELSIMULATION_H
00016
00017 #include <chrono>
00018 #include <memory>
00019 #include "../SimulationControlAndResponse.h"
00020 #include "../Event.h"
00021 #include "../essentialPlugins/Entity.h"
00022 #include "ModelInfo.h"
00023 #include "../SimulationReporter_if.h"
00024 #include "../OnEventManager.h"
00025 // #include "Counter.h"
00026 //namespace GenesysKernel {
00027
00028 // #include "Model.h" // for friend functions
00029 class Model;
00030
00047 class ModelSimulation { // 202104 to be subjected to SimulationScenario
00048 public:
00049     ModelSimulation(Model* model);
00050     virtual ~ModelSimulation();
00051 public:
00052     std::string show();
00053 // @ToDo: (importante): Keep ModelSimulation as the stable
00054 // execution core while the higher-level experiment abstractions mature.
00055 // Any migration of responsibilities to SimulationExperiment/SimulationScenario

```

```

00056 // should preserve the already functional replication-based workflow here.
00057 public: // simulation control
00058     // @ToDo: (importante): Revisit how this class should interact with the unfinished higher-level
    experiment layer.
00059     void start();
00060     void pause();
00061     void step();
00062     void stop();
00063 public: // old modelInfos
00064     void setNumberOfReplications(unsigned int _numberOfReplications);
00065     unsigned int getNumberOfReplications() const;
00066     //void setReplicationLength(double _replicationLength);
00067     void setReplicationLength(double _replicationLength, Util::TimeUnit _replicationLengthTimeUnit =
    Util::TimeUnit::unknown);
00068     double getReplicationLength() const;
00069     void setReplicationLengthTimeUnit(Util::TimeUnit _replicationLengthTimeUnit);
00070     Util::TimeUnit getReplicationLengthTimeUnit() const;
00071     void setReplicationReportBaseTimeUnit(Util::TimeUnit _replicationReportBaseTimeUnit);
00072     Util::TimeUnit getReplicationBaseTimeUnit() const;
00073     //void setWarmUpPeriod(double warmUpPeriod);
00074     void setWarmUpPeriod(double warmUpPeriod, Util::TimeUnit warmUpPeriodTimeUnit =
    Util::TimeUnit::unknown);
00075     double getWarmUpPeriod() const;
00076     void setWarmUpPeriodTimeUnit(Util::TimeUnit _warmUpPeriodTimeUnit);
00077     Util::TimeUnit getWarmUpPeriodTimeUnit() const;
00078     void setTerminatingCondition(std::string _terminatingCondition);
00079     std::string getTerminatingCondition() const;
00080 public: // gets and sets
00081     void setPauseOnEvent(bool _pauseOnEvent);
00082     bool isPauseOnEvent() const;
00083     void setStepByStep(bool _stepByStep);
00084     bool isStepByStep() const;
00085     void setInitializeStatistics(bool _initializeStatistics);
00086     bool isInitializeStatistics() const;
00087     void setInitializeSystem(bool _initializeSystem);
00088     bool isInitializeSystem() const;
00089     void setPauseOnReplication(bool _pauseBetweenReplications);
00090     bool isPauseOnReplication() const;
00091 public: // gets and sets ModelSubParts
00092     void setReporter(SimulationReporter_if* _simulationReporter);
00093     SimulationReporter_if* getReporter() const;
00094 public: // only gets
00095     double getSimulatedTime() const;
00096     bool isRunning() const;
00097     bool isPaused() const;
00098     unsigned int getCurrentReplicationNumber() const;
00099     void setShowReportsAfterReplication(bool showReportsAfterReplication);
00100     bool isShowReportsAfterReplication() const;
00101     void setShowReportsAfterSimulation(bool showReportsAfterSimulation);
00102     bool isShowReportsAfterSimulation() const;
00103     List<double>* getBreakpointsOnTime() const;
00104     List<Entity*>* getBreakpointsOnEntity() const;
00105     List<ModelComponent*>* getBreakpointsOnComponent() const;
00106     const List<ModelDataDefinition*>* getSimulationStatisticsAggregates() const;
00107 public:
00108     void loadInstance(PersistenceRecord *fields);
00109     void saveInstance(PersistenceRecord *fields, bool saveDefaults);
00110     Event* getCurrentEvent() const;
00111     void setShowSimulationResponsesInReport(bool _showSimulationResponsesInReport);
00112     bool isShowSimulationResponsesInReport() const;
00113     void setShowSimulationControlsInReport(bool _showSimulationControlsInReport);
00114     bool isShowSimulationControlsInReport() const;
00115     /*
00116     * PRIVATE
00117     */
00118 private: // simulation control
00119     void _initSimulation();
00120     void _initReplication();
00121     void _clearStatistics();
00122     void _checkWarmUpTime(Event* nextEvent);
00123     void _stepSimulation();
00124     void _replicationEnded();
00125     void _simulationEnded();
00126 private:
00127     void _dispatchEvent(Event* event);
00128     bool _checkBreakpointAt(Event* event);
00129     bool _isReplicationEndCondition();
00130     void _actualizeSimulationStatistics();
00131     void _showSimulationHeader();
00132     void _traceReplicationEnded();
00133 private:
00134     std::unique_ptr<SimulationEvent> _createSimulationEvent(void* thiscustomObject = nullptr);
00135     //friend void Model::removeEntity(Entity* entity);
00136     //friend void Model::sendEntityToComponent(Entity* entity, ModelComponent* component, double
    timeDelay, unsigned int componentInputPortNumber);
00137     friend class Model;
00138 private:

```

```

00139     double _simulatedTime = 0.0;
00140     double _justTriggeredBreakpointsOnTime = 0.0;
00141     // @ToDo: (pequena alteração): list of double _breakOnTimes;
00142     // @ToDo: (pequena alteração): list of modules _breakOnModules;
00143     bool _stepByStep = false;
00144     bool _pauseOnReplication = false;
00145     bool _pauseOnEvent = false;
00146     bool _initializeStatisticsBetweenReplications = true;
00147     bool _initializeSystem = true;
00148     bool _isRunning = false;
00149     bool _isPaused = false;
00150     bool _pauseRequested = false;
00151     bool _stopRequested = false;
00152     bool _simulationIsInitiated = false;
00153     bool _replicationIsInitiated = false;
00154     bool _hasChanged = false;
00155 private:
00156
00157     const struct DEFAULT_VALUES {
00158         const unsigned int numberOfReplications = 1;
00159         const double replicationLength = 60.0; // by default, 60s = 1min
00160         const Util::TimeUnit replicationLengthTimeUnit = Util::TimeUnit::second;
00161         const Util::TimeUnit replicationBaseTimeUnit = Util::TimeUnit::second;
00162         const Util::TimeUnit replicationReportBaseTimeUnit = Util::TimeUnit::second;
00163         const double warmUpPeriod = 0.0;
00164         const Util::TimeUnit warmUpPeriodTimeUnit = Util::TimeUnit::second;
00165         const std::string terminatingCondition = "";
00166         const bool initializeStatisticsBetweenReplications = true;
00167         const bool initializeSystem = true;
00168         const bool showReportsAfterSimulation = true;
00169         const bool showReportsAfterReplication = true;
00170         const bool showSimulationControlsInReport = false;
00171         const bool showSimulationResponsesInReport = false;
00172     } DEFAULT;
00173     unsigned int _numberOfReplications = DEFAULT.numberOfReplications;
00174     double _replicationLength = DEFAULT.replicationLength;
00175     Util::TimeUnit _replicationLengthTimeUnit = DEFAULT.replicationLengthTimeUnit;
00176     Util::TimeUnit _replicationBaseTimeUnit = DEFAULT.replicationReportBaseTimeUnit;
00177     double _warmUpPeriod = DEFAULT.warmUpPeriod;
00178     Util::TimeUnit _warmUpPeriodTimeUnit = DEFAULT.warmUpPeriodTimeUnit;
00179     std::string _terminatingCondition = DEFAULT.terminatingCondition;
00180     bool _showReportsAfterSimulation = DEFAULT.showReportsAfterSimulation;
00181     bool _showReportsAfterReplication = DEFAULT.showReportsAfterReplication;
00182     bool _showSimulationControlsInReport = DEFAULT.showSimulationControlsInReport;
00183     bool _showSimulationResponsesInReport = DEFAULT.showSimulationResponsesInReport;
00184     //
00185     double _replicationTimeScaleFactorToBase; // a scale that converts ReplicationLengthTimeUnit to
    ReplicationBaseTimeUnit. Future events are in "times" of unit ReplicationBaseTimeUnit
00186     std::chrono::system_clock::time_point _startRealSimulationTimeSimulation;
00187     std::chrono::system_clock::time_point _startRealSimulationTimeReplication;
00188 private:
00189     Event* _currentEvent = nullptr;
00190     unsigned int _currentReplicationNumber;
00191 private:
00192     const std::string _cte_stCountSimulNamePrefix = ""; //Simul.";
00193 private:
00194     Model* _model;
00195     ModelInfo* _info;
00196     SimulationReporter_if* _simulationReporter;
00197     bool _ownsSimulationReporter = true;
00198     List<SimulationControl*>* _ownedControls = new List<SimulationControl*>();
00199     // @ToDo: (importante): Change List below to a MAP, associating every CstatOuCounter in the
    replication to the equivalent in the simulation
00200     List<ModelDataDefinition*>* _cstatsAndCountersSimulation = new List<ModelDataDefinition*>();
00201     std::map<ModelDataDefinition*, ModelDataDefinition*>* _cstatsAndCountersMapSimulation = new
    std::map<ModelDataDefinition*, ModelDataDefinition*>();
00202     List<double*>* _breakpointsOnTime = new List<double*>();
00203     List<ModelComponent*>* _breakpointsOnComponent = new List<ModelComponent*>();
00204     List<Entity*>* _breakpointsOnEntity = new List<Entity*>();
00205     ModelComponent* _justTriggeredBreakpointsOnComponent = nullptr;
00206     Entity* _justTriggeredBreakpointsOnEntity = nullptr;
00207 };
00208 //namespace\\
00209 #endif /* MODELSIMULATION_H */

```

11.256 OnEventManager.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006

```

```

00007  /*
00008  * File:    OnEventManager.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 7 de Novembro de 2018, 12:28
00012  */
00013
00014  #ifndef ONEVENTMANAGER_H
00015  #define ONEVENTMANAGER_H
00016
00017  #include "../util/List.h"
00018  #include "Event.h"
00019
00020  //namespace GenesysKernel {
00021  /*
00022  * used to get and set values no matter the class (for process analyser)
00023  * should be a wait to invoke a getter or setter no matter the class (a pointer to a member function
00024  * without specifying the class
00025  */
00026  //typedef double (*memberFunctionGetDoubleVarHandler)(); //template<> ... typedef double
00027  (T::*getDoubleVarHandler)() or something like that
00028  //typedef void (*memberFunctionSetDoubleVarHandler)(double);
00029
00030
00031  class ModelEvent {
00032  public:
00033      ModelEvent(Model* model){
00034          _model = model;
00035      };
00036  public:
00037      const Model* getModel() {
00038          return _model;
00039      }
00040  private:
00041      Model* _model;
00042  };
00043  typedef void (*modelEventHandler)(ModelEvent*);
00044  // for handlers that are class members (methods)
00045  typedef std::function<void(ModelEvent*) > modelEventHandlerMethod;
00046
00047
00048  class SimulationEvent {
00049  public:
00050
00051      void setSimulatedTime(double simulatedTime) {
00052          this->simulatedTime = simulatedTime;
00053      }
00054
00055      double getSimulatedTime() const {
00056          return simulatedTime;
00057      }
00058
00059      void setStopRequested(bool stopRequested) {
00060          this->stopRequested = stopRequested;
00061      }
00062
00063      bool isStopRequested() const {
00064          return stopRequested;
00065      }
00066
00067      void setPauseRequested(bool pauseRequested) {
00068          this->pauseRequested = pauseRequested;
00069      }
00070
00071      bool isPauseRequested() const {
00072          return pauseRequested;
00073      }
00074
00075      void setPaused(bool Paused) {
00076          _isPaused = Paused;
00077      }
00078
00079      bool isPaused() const {
00080          return _isPaused;
00081      }
00082
00083      void setRunning(bool Running) {
00084          _isRunning = Running;
00085      }
00086
00087      bool isRunning() const {
00088          return _isRunning;
00089      }
00090
00091      void setCustomObject(void* customObject) {
00092          this->customObject = customObject;
00093      }
00094
00095

```



```

00096
00097     void* getCustomObject() const {
00098         return customObject;
00099     }
00100
00101     void setCurrentReplicationNumber(unsigned int currentReplicationNumber) {
00102         this->currentReplicationNumber = currentReplicationNumber;
00103     }
00104
00105     unsigned int getCurrentReplicationNumber() const {
00106         return currentReplicationNumber;
00107     }
00108
00109     void setCurrentEvent(Event* currentEvent) {
00110         this->currentEvent = currentEvent;
00111     }
00112
00113     Event* getCurrentEvent() const {
00114         return currentEvent;
00115     }
00116     //void setCurrentInputPortNumber(unsigned int currentInputPortNumber) {
00117     //    this->currentInputPortNumber = currentInputPortNumber;
00118     //}
00119     //unsigned int getCurrentInputPortNumber() const {
00120     //    return currentInputPortNumber;
00121     //}
00122     //void setCurrentComponent(ModelComponent* currentComponent) {
00123     //    this->currentComponent = currentComponent;
00124     //}
00125     //ModelComponent* getCurrentComponent() const {
00126     //    return currentComponent;
00127     //}
00128     //void setCurrentEntity(Entity* currentEntity) {
00129     //    this->currentEntity = currentEntity;
00130     //}
00131     //Entity* getCurrentEntity() const {
00132     //    return currentEntity;
00133     //}
00134
00135     void setEntityCreated(Entity* entityCreated) {
00136         this->entityCreated = entityCreated;
00137     }
00138
00139     Entity* getEntityCreated() const {
00140         return entityCreated;
00141     }
00142
00143     void setDestinationComponent(ModelComponent* destinationComponent) {
00144         this->destinationComponent = destinationComponent;
00145     }
00146
00147     ModelComponent* getDestinationComponent() const {
00148         return destinationComponent;
00149     }
00150
00151     void setEntityMoveTimeDelay(double entityMoveTimeDelay) {
00152         this->entityMoveTimeDelay = entityMoveTimeDelay;
00153     }
00154
00155     double getEntityMoveTimeDelay() const {
00156         return entityMoveTimeDelay;
00157     }
00158 private:
00159     SimulationEvent() {
00160     }
00161     friend class ModelSimulation;
00162 private:
00163     Entity* entityCreated = nullptr;
00164     Event* currentEvent = nullptr;
00165     //Entity* currentEntity = nullptr;
00166     //unsigned int currentInputPortNumber = 0;
00167     //ModelComponent* currentComponent = nullptr;
00168     ModelComponent* destinationComponent = nullptr;
00169     void* customObject = nullptr;
00170     unsigned int currentReplicationNumber = 0;
00171     double simulatedTime = 0.0;
00172     double entityMoveTimeDelay = 0.0;
00173     bool _isRunning = false;
00174     bool _isPaused = false;
00175     bool pauseRequested = false;
00176     bool stopRequested = false;
00177 };
00178
00179 typedef void (*simulationEventHandler)(SimulationEvent*);
00180 // for handlers that are class members (methods)
00181 typedef std::function<void(SimulationEvent*)> simulationEventHandlerMethod;
00182

```

```

00187 class OnEventManager {
00188 public:
00189     OnEventManager();
00190     virtual ~OnEventManager();
00191 public: // event listeners (handlers)
00192     void addOnModelCheckSuccessHandler(modelEventHandler EventHandler);
00193     void addOnModelLoadHandler(modelEventHandler EventHandler);
00194     void addOnModelSaveHandler(modelEventHandler EventHandler);
00195     void addOnReplicationStartHandler(simulationEventHandler EventHandler);
00196     void addOnReplicationStepHandler(simulationEventHandler EventHandler);
00197     void addOnReplicationEndHandler(simulationEventHandler EventHandler);
00198     void addOnProcessEventHandler(simulationEventHandler EventHandler);
00199     void addOnAfterProcessEventHandler(simulationEventHandler EventHandler);
00200     void addOnEntityCreateHandler(simulationEventHandler EventHandler);
00201     void addOnEntityMoveHandler(simulationEventHandler EventHandler);
00202     void addOnEntityRemoveHandler(simulationEventHandler EventHandler);
00203     void addOnSimulationStartHandler(simulationEventHandler EventHandler);
00204     void addOnSimulationPausedHandler(simulationEventHandler EventHandler);
00205     void addOnSimulationResumeHandler(simulationEventHandler EventHandler);
00206     void addOnSimulationEndHandler(simulationEventHandler EventHandler);
00207     void addOnBreakpointHandler(simulationEventHandler EventHandler);
00208 public: // event listeners (method handlers)
00209     // for handlers that are class members (methods)
00210     template<typename Class> void addOnModelCheckSuccessHandler(Class * object, void
00211 (Class::*function) (ModelEvent*));
00211     template<typename Class> void addOnModelLoadHandler(Class * object, void
00212 (Class::*function) (ModelEvent*));
00212     template<typename Class> void addOnModelSaveHandler(Class * object, void
00213 (Class::*function) (ModelEvent*));
00213     template<typename Class> void addOnReplicationStartHandler(Class * object, void
00214 (Class::*function) (SimulationEvent*));
00214     template<typename Class> void addOnReplicationStepHandler(Class * object, void
00215 (Class::*function) (SimulationEvent*));
00215     template<typename Class> void addOnReplicationEndHandler(Class * object, void
00216 (Class::*function) (SimulationEvent*));
00216     template<typename Class> void addOnProcessEventHandler(Class * object, void
00217 (Class::*function) (SimulationEvent*));
00217     template<typename Class> void addOnAfterProcessEventHandler(Class * object, void
00218 (Class::*function) (SimulationEvent*));
00218     template<typename Class> void addOnEntityCreateHandler(Class * object, void
00219 (Class::*function) (SimulationEvent*));
00219     template<typename Class> void addOnEntityMoveHandler(Class * object, void
00220 (Class::*function) (SimulationEvent*));
00220     template<typename Class> void addOnEntityRemoveHandler(Class * object, void
00221 (Class::*function) (SimulationEvent*));
00221     template<typename Class> void addOnSimulationStartHandler(Class * object, void
00222 (Class::*function) (SimulationEvent*));
00222     template<typename Class> void addOnSimulationPausedHandler(Class * object, void
00223 (Class::*function) (SimulationEvent*));
00223     template<typename Class> void addOnSimulationResumeHandler(Class * object, void
00224 (Class::*function) (SimulationEvent*));
00224     template<typename Class> void addOnSimulationEndHandler(Class * object, void
00225 (Class::*function) (SimulationEvent*));
00225     template<typename Class> void addOnBreakpointHandler(Class * object, void
00226 (Class::*function) (SimulationEvent*));
00226 public:
00227     void NotifyModelCheckSuccessHandlers(ModelEvent* se);
00228     void NotifyModelLoadHandlers(ModelEvent* se);
00229     void NotifyModelSaveHandlers(ModelEvent* se);
00230     void NotifyReplicationStartHandlers(SimulationEvent* se);
00231     void NotifyReplicationStepHandlers(SimulationEvent* se);
00232     void NotifyReplicationEndHandlers(SimulationEvent* se);
00233     void NotifyProcessEventHandlers(SimulationEvent* se);
00234     void NotifyAfterProcessEventHandlers(SimulationEvent* se);
00235     void NotifyEntityCreateHandlers(SimulationEvent* se);
00236     void NotifyEntityMoveHandlers(SimulationEvent* se);
00237     void NotifyEntityRemoveHandlers(SimulationEvent* se);
00238     void NotifySimulationStartHandlers(SimulationEvent* se);
00239     void NotifySimulationPausedHandlers(SimulationEvent* se);
00240     void NotifySimulationResumeHandlers(SimulationEvent* se);
00241     void NotifySimulationEndHandlers(SimulationEvent* se);
00242     void NotifyBreakpointHandlers(SimulationEvent* se);
00243 private:
00244     void _NotifyHandlers(List<modelEventHandler>* list, ModelEvent* se);
00245     void _NotifyHandlers(List<simulationEventHandler>* list, SimulationEvent* se);
00246     void _NotifyHandlerMethods(List<modelEventHandlerMethod>* list, ModelEvent* se);
00247     void _NotifyHandlerMethods(List<simulationEventHandlerMethod>* list, SimulationEvent* se);
00248     void _addOnHandler(List<modelEventHandler>* list, modelEventHandler EventHandler);
00249     void _addOnHandler(List<simulationEventHandler>* list, simulationEventHandler EventHandler);
00250 private: // events listener
00251     List<modelEventHandler>* _onModelCheckSuccessHandlers = new List<modelEventHandler>();
00252     List<modelEventHandler>* _onModelLoadHandlers = new List<modelEventHandler>();
00253     List<modelEventHandler>* _onModelSaveHandlers = new List<modelEventHandler>();
00254     List<simulationEventHandler>* _onReplicationStartHandlers = new List<simulationEventHandler>();
00255     List<simulationEventHandler>* _onReplicationStepHandlers = new List<simulationEventHandler>();

```

```

00258     List<simulationEventHandler>* _onReplicationEndHandlers = new List<simulationEventHandler>();
00259     List<simulationEventHandler>* _onProcessEventHandlers = new List<simulationEventHandler>();
00260     List<simulationEventHandler>* _onAfterProcessEventHandlers = new List<simulationEventHandler>();
00261     List<simulationEventHandler>* _onEntityCreateHandlers = new List<simulationEventHandler>();
00262     List<simulationEventHandler>* _onEntityMoveHandlers = new List<simulationEventHandler>();
00263     List<simulationEventHandler>* _onEntityRemoveHandlers = new List<simulationEventHandler>();
00264     List<simulationEventHandler>* _onSimulationStartHandlers = new List<simulationEventHandler>();
00265     List<simulationEventHandler>* _onSimulationPausedHandlers = new List<simulationEventHandler>();
00266     List<simulationEventHandler>* _onSimulationResumeHandlers = new List<simulationEventHandler>();
00267     List<simulationEventHandler>* _onSimulationEndHandlers = new List<simulationEventHandler>();
00268     List<simulationEventHandler>* _onBreakpointHandlers = new List<simulationEventHandler>();
00269 private: // events listener for handlers that are class members (methods)
00270     List<modelEventHandlerMethod>* _onModelCheckSuccessHandlerMethods = new
List<modelEventHandlerMethod>();
00271     List<modelEventHandlerMethod>* _onModelLoadHandlerMethods = new List<modelEventHandlerMethod>();
00272     List<modelEventHandlerMethod>* _onModelSaveHandlerMethods = new List<modelEventHandlerMethod>();
00273     List<simulationEventHandlerMethod>* _onReplicationStartHandlerMethods = new
List<simulationEventHandlerMethod>();
00274     List<simulationEventHandlerMethod>* _onReplicationStepHandlerMethods = new
List<simulationEventHandlerMethod>();
00275     List<simulationEventHandlerMethod>* _onReplicationEndHandlerMethods = new
List<simulationEventHandlerMethod>();
00276     List<simulationEventHandlerMethod>* _onProcessEventHandlerMethods = new
List<simulationEventHandlerMethod>();
00277     List<simulationEventHandlerMethod>* _onAfterProcessEventHandlerMethods = new
List<simulationEventHandlerMethod>();
00278     List<simulationEventHandlerMethod>* _onEntityCreateHandlerMethods = new
List<simulationEventHandlerMethod>();
00279     List<simulationEventHandlerMethod>* _onEntityMoveHandlerMethods = new
List<simulationEventHandlerMethod>();
00280     List<simulationEventHandlerMethod>* _onEntityRemoveHandlerMethods = new
List<simulationEventHandlerMethod>();
00281     List<simulationEventHandlerMethod>* _onSimulationStartHandlerMethods = new
List<simulationEventHandlerMethod>();
00282     List<simulationEventHandlerMethod>* _onSimulationPausedHandlerMethods = new
List<simulationEventHandlerMethod>();
00283     List<simulationEventHandlerMethod>* _onSimulationResumeHandlerMethods = new
List<simulationEventHandlerMethod>();
00284     List<simulationEventHandlerMethod>* _onSimulationEndHandlerMethods = new
List<simulationEventHandlerMethod>();
00285     List<simulationEventHandlerMethod>* _onBreakpointHandlerMethods = new
List<simulationEventHandlerMethod>();
00286 };
00287
00288 //
00289 // implementation for template methods
00290 //
00291 // // @ToDo: (importante): (!!) Complicated: if handlerMethod already insert, should not insert it
again. Problem to solve <...> for function
00292 //if (_onProcessEventHandlerMethods->find(handlerMethod) ==
_onProcessEventHandlerMethods->list()->end())
00293 // trying unique to solve the issue
00294 //this->_onProcessEventHandlerMethods->list()->unique(); // does not work
00295 // @ToDo: (importante): probably to override == operator for type simulationEventHandlerMethod
00296 // ...
00297
00298 template<typename Class> void OnEventManager::addOnModelCheckSuccessHandler(Class * object, void
(Class::*function)(ModelEvent*)) {
00299     modelEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00300     this->_onModelCheckSuccessHandlerMethods->insert(handlerMethod);
00301 }
00302
00303 template<typename Class> void OnEventManager::addOnModelLoadHandler(Class * object, void
(Class::*function)(ModelEvent*)) {
00304     modelEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00305     this->_onModelLoadHandlerMethods->insert(handlerMethod);
00306 }
00307
00308 template<typename Class> void OnEventManager::addOnModelSaveHandler(Class * object, void
(Class::*function)(ModelEvent*)) {
00309     modelEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00310     this->_onModelSaveHandlerMethods->insert(handlerMethod);
00311 }
00312 template<typename Class> void OnEventManager::addOnReplicationStartHandler(Class * object, void
(Class::*function)(SimulationEvent*)) {
00313     simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00314     this->_onReplicationStartHandlerMethods->insert(handlerMethod);
00315 }
00316
00317 template<typename Class> void OnEventManager::addOnReplicationStepHandler(Class * object, void
(Class::*function)(SimulationEvent*)) {
00318     simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00319     this->_onReplicationStepHandlerMethods->insert(handlerMethod);
00320 }
00321
00322 template<typename Class> void OnEventManager::addOnReplicationEndHandler(Class * object, void
(Class::*function)(SimulationEvent*)) {

```

```

00323     simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00324     this->_onReplicationEndHandlerMethods->insert(handlerMethod);
00325 }
00326
00327 template<typename Class> void OnEventManager::addOnProcessEventHandler(Class * object, void
(Class::*function)(SimulationEvent*)) {
00328     simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00329     this->_onProcessEventHandlerMethods->insert(handlerMethod);
00330 }
00331
00332 template<typename Class> void OnEventManager::addOnAfterProcessEventHandler(Class * object, void
(Class::*function)(SimulationEvent*)) {
00333     simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00334     this->_onAfterProcessEventHandlerMethods->insert(handlerMethod);
00335 }
00336
00337 template<typename Class> void OnEventManager::addOnEntityCreateHandler(Class * object, void
(Class::*function)(SimulationEvent*)) {
00338     simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00339     this->_onEntityCreateHandlerMethods->insert(handlerMethod);
00340 }
00341
00342 template<typename Class> void OnEventManager::addOnEntityMoveHandler(Class * object, void
(Class::*function)(SimulationEvent*)) {
00343     simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00344     this->_onEntityMoveHandlerMethods->insert(handlerMethod);
00345 }
00346
00347 template<typename Class> void OnEventManager::addOnEntityRemoveHandler(Class * object, void
(Class::*function)(SimulationEvent*)) {
00348     simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00349     this->_onEntityRemoveHandlerMethods->insert(handlerMethod);
00350 }
00351
00352 template<typename Class> void OnEventManager::addOnSimulationStartHandler(Class * object, void
(Class::*function)(SimulationEvent*)) {
00353     simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00354     this->_onSimulationStartHandlerMethods->insert(handlerMethod);
00355 }
00356
00357 template<typename Class> void OnEventManager::addOnSimulationPausedHandler(Class * object, void
(Class::*function)(SimulationEvent*)) {
00358     simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00359     this->_onSimulationPausedHandlerMethods->insert(handlerMethod);
00360 }
00361
00362 template<typename Class> void OnEventManager::addOnSimulationResumeHandler(Class * object, void
(Class::*function)(SimulationEvent*)) {
00363     simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00364     this->_onSimulationResumeHandlerMethods->insert(handlerMethod);
00365 }
00366
00367 template<typename Class> void OnEventManager::addOnSimulationEndHandler(Class * object, void
(Class::*function)(SimulationEvent*)) {
00368     simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00369     this->_onSimulationEndHandlerMethods->insert(handlerMethod);
00370 }
00371
00372 template<typename Class> void OnEventManager::addOnBreakpointHandler(Class * object, void
(Class::*function)(SimulationEvent*)) {
00373     simulationEventHandlerMethod handlerMethod = std::bind(function, object, std::placeholders::_1);
00374     this->_onBreakpointHandlerMethods->insert(handlerMethod);
00375 }
00376
00377
00378 /*
00379 Private Sub ModelLogic_DocumentOpen()
00380 Private Sub ModelLogic_DocumentSave()
00381 Private Sub ModelLogic_OnClearStatistics()
00382 Private Sub ModelLogic_OnFileClose(ByVal filesElementNumber As Long, ByVal fileName As String)
00383 Private Sub ModelLogic_OnFileRead(ByVal filesElementNumber As Long, ByVal fileName As String, ByVal
numberOfItems As Long, readBuffer() As Double)
00384 Private Sub ModelLogic_OnFileReadAsVariant(ByVal filesElementNumber As Long, ByVal fileName As String,
ByVal numberOfItems As Long, readBuffer() As Variant)
00385 Private Sub ModelLogic_OnFileWrite(ByVal filesElementNumber As Long, ByVal fileName As String, ByVal
numberOfItems As Long, writeBuffer() As Double)
00386 Private Sub ModelLogic_OnFileWriteAsVariant(ByVal filesElementNumber As Long, ByVal fileName As
String, ByVal numberOfItems As Long, writeBuffer() As Variant)
00387 Private Sub ModelLogic_OnKeystroke(ByVal keyCode As Long)
00388 Private Function ModelLogic_RealTimeGetMaxTimeAdvance() As Long
00389 Private Function ModelLogic_RealTimeInitialize(ByVal processName As String, ByVal remoteProcessName As
String) As Long
00390 Private Sub ModelLogic_RealTimeInitializeMaxTimeAdvance()
00391 Private Function ModelLogic_RealTimeReceive(message As String) As Long
00392 Private Function ModelLogic_RealTimeSend(ByVal message As String) As Long
00393 Private Function ModelLogic_RealTimeTerminate() As Long
00394 Private Sub ModelLogic_RunBegin()

```

```

00395 Private Sub ModelLogic_RunBeginReplication()
00396 Private Sub ModelLogic_RunBeginSimulation()
00397 Private Sub ModelLogic_RunBreak()
00398 Private Sub ModelLogic_RunEnd()
00399 Private Sub ModelLogic_RunEndReplication()
00400 Private Sub ModelLogic_RunEndSimulation()
00401 Private Sub ModelLogic_RunFastForward()
00402 Private Sub ModelLogic_RunPause()
00403 Private Sub ModelLogic_RunRestart()
00404 Private Sub ModelLogic_RunResume()
00405 Private Sub ModelLogic_RunStep()
00406 Private Sub ModelLogic_SimanError(ByVal routineName As String, msgType As Long, msgNum As Long)
00407 Private Sub ModelLogic_UserContinuousEquations()
00408 Private Function ModelLogic_UserFunction(ByVal entityID As Long, ByVal functionID As Long) As Double
00409 Private Function ModelLogic_UserRule(ByVal entityID As Long, ByVal ruleID As Long) As Double
00410 Private Sub VBA_Block_1_Fire()
00411 */
00412
00413 //namespace\\}
00414 #endif /* ONEVENTMANAGER_H */

```

11.257 Parser_if.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005 */
00006
00007 /*
00008  * File:   Parser_if.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 23 de Agosto de 2018, 15:57
00012  */
00013
00014 #ifndef PARSE_IF_H
00015 #define PARSE_IF_H
00016
00017 #include <string>
00018 #include "../statistics/Sampler_if.h"
00019
00020 class genesyspp_driver;
00021
00022 class Parser_if {
00023 public:
00024     virtual ~Parser_if() = default;
00025     virtual double parse(const std::string expression) = 0; // may throw exception
00026     virtual double parse(const std::string expression, bool& success, std::string& errorMessage) = 0;
00027     // does not throw exceptions
00028     virtual std::string getErrorMessage() = 0; // to get error message in the case of thrown exception
00029     virtual void setSampler(Sampler_if* sampler) = 0;
00030     virtual Sampler_if* getSampler() const = 0;
00031     // @ToDo: (pequena alteração): Implement a method to get the TOKENS parsed. Example: If parsing
00032     // "nq(queue1) < var1" return a list containing something like "fNQ tLPAR eQUEUE tRPAR tLESS eVARIABLE"
00033     virtual genesyspp_driver getParser() const = 0;
00034 };
00035
00036 #endif /* PARSE_IF_H */

```

11.258 ParserChangesInformation.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005 */
00006
00007 /*
00008  * File:   ParserChangesInformation.h
00009  * Author: rlcancian
00010  *
00011  * Created on 11 de Setembro de 2019, 20:42
00012  */
00013
00014 #ifndef PARSECHANGESINFORMATION_H
00015 #define PARSECHANGESINFORMATION_H
00016
00017 #include <string>

```

```

00018 #include <list>
00019
00020 class ParserChangesInformation {
00021 public:
00022     ParserChangesInformation();
00023     virtual ~ParserChangesInformation() = default;
00024
00025 public: // gets and sets
00026     // @ToDo: (importante): Review the consistency of this class public API names.
00027     // There are mixed naming patterns such as getIncludes() and getFunctionProductions().
00028     // Any rename may break existing callers and should be planned carefully.
00029     std::string getIncludes() const;
00030     void setIncludes(const std::string &newIncludes);
00031
00032     std::string getTokens() const;
00033     void setTokens(const std::string &newTokens);
00034
00035     std::string getTypeObjs() const;
00036     void setTypeObjs(const std::string &newTypeObjs);
00037
00038     std::string getExpressions() const;
00039     void setExpressions(const std::string &newExpressions);
00040
00041     std::string getExpressionProductions() const;
00042     void setExpressionProductions(const std::string &newExpressionProductions);
00043
00044     std::string getAssignments() const;
00045     void setAssignments(const std::string &newAssignments);
00046
00047     // @ToDo: (importante): Confirm whether "Productions" should actually be "Productions".
00048     // Do not rename it directly without mapping impacts on existing callers.
00049     std::string getFunctionProductions() const;
00050     void setFunctionProductions(const std::string &newFunctionProductions);
00051
00052 private:
00053     std::string _includes = "";
00054     std::string _tokens = "";
00055     std::string _typeObjs = "";
00056     std::string _expressions = "";
00057     std::string _expressionProductions = "";
00058     std::string _assignments = "";
00059     std::string _functionProductions = "";
00060
00061 };
00062
00063 #endif /* PARSECHANGESINFORMATION_H */

```

11.259 ParserDefaultImpl1.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:      ParserDefaultImpl1.h
00009  * Author:    rafael.luiz.cancian
00010  *
00011  * Created on 2 de Agosto de 2018, 01:25
00012  */
00013
00014 #ifndef PARSEDEFAULTIMPL1_H
00015 #define PARSEDEFAULTIMPL1_H
00016
00017 #include "Parser_if.h"
00018 //namespace GenesysKernel {
00019 class Model;
00020
00021 class ParserDefaultImpl1 : public Parser_if {
00022 public:
00023     ParserDefaultImpl1(Model* model);
00024     virtual ~ParserDefaultImpl1() = default;
00025 public:
00026     virtual double parse(const std::string expression) override; // may throw exception
00027     virtual double parse(const std::string expression, bool& success, std::string& errorMessage)
00028         override;
00029     virtual std::string getErrorMessage() override;
00029 private:
00030     Model* _model;
00031 };
00032 //namespace\}
00033 #endif /* PARSEDEFAULTIMPL1_H */
00034

```

11.260 ParserDefaultImpl2.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   ParserDefaultImpl2.h
00009  * Author: Joao Ortigara (20180801-20181207) rafael.luiz.cancian (20181208-...(
00010  *
00011  *
00012  */
00013
00014 #ifndef PARSERDEFAULTIMPL2_H
00015 #define PARSERDEFAULTIMPL2_H
00016
00017 #include <string>
00018 #include "Parser_if.h"
00019 #include "model/Model.h"
00020 #include "../../parser/Genesys++-driver.h"
00021
00022
00023 //namespace GenesysKernel {
00024
00025 class ParserDefaultImpl2 : public Parser_if {
00026 public:
00027     ParserDefaultImpl2(Model* model, Sampler_if* sampler, bool throws = false);
00028     virtual ~ParserDefaultImpl2();
00029 public:
00030     virtual double parse(const std::string expression) override; // may throw exception
00031     virtual double parse(const std::string expression, bool& success, std::string& errorMessage)
00032         override;
00033     virtual std::string getErrorMessage() override;
00034     virtual void setSampler(Sampler_if* _sampler) override;
00035     virtual Sampler_if* getSampler() const override;
00036     virtual genesyspp_driver getParser() const override;
00037 private:
00038     void _setSamplerInternal(Sampler_if* sampler, bool ownsSampler);
00039 private:
00040     Model* _model;
00041     genesyspp_driver _wrapper;
00042     bool _ownsSampler = false;
00043 };
00044 //namespace\}
00045 #endif /* PARSERDEFAULTIMPL2_H */

```

11.261 ParserManager.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   ParserManager.h
00009  * Author: rlcancian
00010  *
00011  * Created on 11 de Setembro de 2019, 20:34
00012  */
00013
00014 #ifndef PARSERMANAGER_H
00015 #define PARSERMANAGER_H
00016
00017 #include "ParserChangesInformation.h"
00018
00019 //namespace GenesysKernel {
00020
00021 class ParserManager {
00022 public:
00023
00024     struct NewParser {
00025         std::string bisonFilename;
00026         std::string flexFilename;
00027         std::string compiledParserFilename;
00028     };
00029
00030     struct GenerateNewParserResult {
00031         bool result = false;
00032         std::string bisonMessages;
00033         std::string lexMessages;
00034     };
00035
00036 };
00037 //namespace\}

```



```

00040         std::string compilationMessages;
00041         NewParser newParser;
00042     };
00043 public:
00044     ParserManager();
00045     virtual ~ParserManager() = default;
00046 public:
00052     // @ToDo: (importante): Implement parser generation workflow and define the expected
    filesystem/toolchain contract.
00053     ParserManager::GenerateNewParserResult generateNewParser(ParserChangesInformation* changes);
00059     // @ToDo: (importante): Implement parser loading/activation and define ownership/lifetime rules
    for connected parsers.
00060     bool connectNewParser(ParserManager::NewParser newParser);
00061 private:
00062 };
00063 //namespace\\
00064 #endif /* PARSEMANAGER_H */

```

11.262 Persistence.h

```

00001 #ifndef PERSISTENCE_H
00002 #define PERSISTENCE_H
00003
00004 #include <string>
00005 #include <utility>
00006 #include <unordered_map>
00007 #include <memory>
00008
00009 #include "../util/Util.h"
00010 #include "persistence/Persistence_if.h"
00011
00012
00022 class PersistenceRecord {
00023 public:
00024     using Payload = std::string;
00026     struct Entry {
00027         enum Kind { text, numeric };
00028         std::string first;
00029         Payload second;
00030         Kind kind{Kind::text};
00031     };
00032     using Iterator = std::unordered_map<std::string, Entry>::iterator;
00033
00034     // this is the actual abstract interface
00035
00036     virtual ~PersistenceRecord();
00037
00038     virtual PersistenceRecord* newInstance();
00039
00040     virtual std::string loadField(const std::string& key, const std::string& defaultValue = "");
00041     virtual double loadField(const std::string& key, double defaultValue);
00042     virtual unsigned int loadField(const std::string& key, unsigned int defaultValue);
00043     virtual int loadField(const std::string& key, int defaultValue);
00044     virtual Util::TimeUnit loadField(const std::string& key, Util::TimeUnit defaultValue);
00045
00046     virtual void saveField(const std::string& key, const std::string& value, const std::string&
    defaultValue, const bool saveIfDefault = false);
00047     virtual void saveField(const std::string& key, const std::string& value);
00048     virtual void saveField(const std::string& key, double value, const double defaultValue, const bool
    saveIfDefault = false);
00049     virtual void saveField(const std::string& key, unsigned int value, const unsigned int
    defaultValue, const bool saveIfDefault = false);
00050     virtual void saveField(const std::string& key, unsigned int value);
00051     virtual void saveField(const std::string& key, int value, const int defaultValue, const bool
    saveIfDefault = false);
00052     virtual void saveField(const std::string& key, Util::TimeUnit value, const Util::TimeUnit
    defaultValue, const bool saveIfDefault = false);
00053
00054     // these are only needed because existing code does not abstract away the STL container API
00055     // still, subclasses implementing this interface may use them to insert post-processed entries
00056
00057     explicit PersistenceRecord(Persistence_if& config);
00058
00060     std::size_t size() const;
00061
00063     void insert(Entry entry);
00064
00066     void insert(Iterator it, Iterator end);
00067
00069     void erase(const std::string& key);
00070
00072     void clear();
00073

```



```

00075     Iterator find(const std::string& key);
00076
00077     Iterator begin();
00078
00079     Iterator end();
00080
00081 private:
00082     std::unordered_map<std::string, Entry> _fields;
00083     Persistence_if& _config;
00084 };
00085
00086 class PersistentObject_base {
00087 public:
00088     virtual ~PersistentObject_base() {};
00089
00090 protected:
00091     virtual bool _loadInstance(PersistenceRecord *fields) = 0;
00092
00093     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues = false) = 0;
00094 };
00095
00096 #endif /* PERSISTENCE_H */

```

11.263 CppSerializer.h

```

00001 #ifndef CPPSERIALIZER_H
00002 #define CPPSERIALIZER_H
00003
00004 #include <unordered_map>
00005 #include<memory>
00006
00007 #include "Serializer.h"
00008 #include "../model/Model.h"
00009
00010 class CppSerializer : public Serializer {
00011 public:
00012     explicit CppSerializer(Model *model);
00013
00014 public: // Serializer interface
00015     PersistenceRecord* newPersistenceRecord() override;
00016     bool dump(std::ostream& output) override;
00017     bool load(std::istream& input) override;
00018     bool get(const std::string& name, PersistenceRecord *entry) override;
00019     bool put(const std::string name, const std::string type, const Util::identification id,
00020             PersistenceRecord *fields) override;
00021     int for_each(std::function<int(const std::string&) > delegate) override;
00022 private:
00023     Model *_model{};
00024     std::unordered_map<std::string, std::unique_ptr<PersistenceRecord> _metaobjects
00025     {
00026     };
00027     std::unordered_map<std::string, std::unique_ptr<PersistenceRecord> _components
00028     {
00029     };
00030 };
00031
00032 #endif // CPPSERIALIZER_H

```

11.264 GenSerializer.h

```

00001 #ifndef GENSERIALIZER_H
00002 #define GENSERIALIZER_H
00003
00004 #include <unordered_map>
00005 #include<memory>
00006
00007 #include "Serializer.h"
00008 #include "../model/Model.h"
00009
00010 class GenSerializer : public Serializer {
00011 public:
00012     explicit GenSerializer(Model *model);
00013
00014 public: // Serializer interface
00015     PersistenceRecord* newPersistenceRecord() override;
00016     bool dump(std::ostream& output) override;
00017     bool load(std::istream& input) override;

```

```

00018     bool get(const std::string& name, PersistenceRecord *entry) override;
00019     bool put(const std::string name, const std::string type, const Util::identification id,
PersistenceRecord *fields) override;
00020     int for_each(std::function<int(const std::string&) > delegate) override;
00021
00022 private:
00023     static std::string linearize(PersistenceRecord *fields);
00024     friend class PersistenceDefaultImpl2;
00025
00026 private:
00027     Model *_model{};
00028     std::unordered_map<std::string, std::unique_ptr<PersistenceRecord> _components
00029     {
00030     };
00031 };
00032
00033 #endif // GENSERIALIZER_H

```

11.265 JsonSerializer.h

```

00001 #ifndef JSONSERIALIZER_H
00002 #define JSONSERIALIZER_H
00003
00004 #include <unordered_map>
00005 #include <memory>
00006
00007 #include "Serializer.h"
00008 #include "../model/Model.h"
00009
00010 class JsonSerializer : public Serializer {
00011 public:
00012     explicit JsonSerializer(Model *model);
00013
00014 public: // Serializer interface
00015     PersistenceRecord* newPersistenceRecord() override;
00016     bool dump(std::ostream& output) override;
00017     bool load(std::istream& input) override;
00018     bool get(const std::string& name, PersistenceRecord *entry) override;
00019     bool put(const std::string name, const std::string type, const Util::identification id,
PersistenceRecord *fields) override;
00020     int for_each(std::function<int(const std::string&) > delegate) override;
00021
00022 private:
00023     Model *_model{};
00024     std::unordered_map<std::string, std::unique_ptr<PersistenceRecord> _metaobjects
00025     {
00026     };
00027     std::unordered_map<std::string, std::unique_ptr<PersistenceRecord> _components
00028     {
00029     };
00030 };
00031
00032 #endif // JSONSERIALIZER_H

```

11.266 Persistence_if.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Persistence_if.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 24 de Agosto de 2018, 19:22
00012  */
00013
00014 #ifndef MODELPERISTENCE_IF_H
00015 #define MODELPERISTENCE_IF_H
00016
00017 #include <string>
00018 #include <map>
00019
00020
00021 // forward decl
00022 class PersistenceRecord;
00023

```

```

00031 class Persistence_if {
00032 public:
00033     virtual ~Persistence_if() = default;
00034
00035     enum class Options : int {
00036         SAVEDEFAULTS = 1, HIDEIDKEY = 2, HIDETYPEKEY = 4, HIDENAMEKEY = 8, SORTALPHLY = 16
00037     };
00038
00039 public:
00041     virtual bool save(std::string filename) = 0;
00043     virtual bool load(std::string filename) = 0;
00044
00045 public:
00047     virtual bool hasChanged() = 0;
00049     virtual void setHasChanged(bool hasChanged) = 0;
00051     virtual bool getOption(Persistence_if::Options option) = 0;
00053     virtual void setOption(Persistence_if::Options option, bool value) = 0;
00054 public:
00056     virtual std::string getFormatedField(PersistenceRecord *fields) = 0;
00057 };
00058
00059 #endif /* MODELPERISTENCE_IF_H */

```

11.267 PersistenceDefaultImpl2.h

```

00001 #ifndef MODELPERISTENCEDEFAULTIMPL_H
00002 #define MODELPERISTENCEDEFAULTIMPL_H
00003
00004 #include "Persistence_if.h"
00005 #include "../model/Model.h"
00006
00007 class PersistenceDefaultImpl2 : public Persistence_if {
00008 public:
00009     PersistenceDefaultImpl2(Model* model);
00010
00011 public: // Persistence_if interface
00012     bool save(std::string filename) override;
00013     bool load(std::string filename) override;
00014     bool hasChanged() override;
00015     void setHasChanged(bool hasChanged) override;
00016     bool getOption(Persistence_if::Options option) override;
00017     void setOption(Persistence_if::Options option, bool value) override;
00018     std::string getFormatedField(PersistenceRecord *fields) override;
00019
00020 private:
00021     Model* _model;
00022     unsigned short _options{0};
00023     bool _dirty{false};
00024 };
00025
00026 #endif /* MODELPERISTENCEDEFAULTIMPL_H */

```

11.268 PersistencePartialLoadImpl1.h

```

00001 #ifndef MODELPERISTENCEPARTIALLOADIMPL1_H
00002 #define MODELPERISTENCEPARTIALLOADIMPL1_H
00003
00004 #include "PersistenceDefaultImpl2.h"
00005 #include "../model/Model.h"
00006
00007 class PersistencePartialLoadImpl1 : public PersistenceDefaultImpl2 {
00008 public:
00009     explicit PersistencePartialLoadImpl1(Model* model);
00010
00011 public:
00012     bool load(std::string filename) override;
00013
00014 private:
00015     Model* _model;
00016 };
00017
00018 #endif /* MODELPERISTENCEPARTIALLOADIMPL1_H */

```

11.269 Serializer.h

```

00001 #ifndef MODELSERIALIZER_H

```

```

00002 #define MODELSERIALIZER_H
00003
00004 #include <iostream>
00005 #include <functional>
00006
00007 #include "../Persistence.h"
00008 #include "../util/Util.h"
00009
00023 class Serializer {
00024 public:
00025     virtual ~Serializer();
00026
00033     virtual PersistenceRecord* newPersistenceRecord() = 0;
00034
00042     virtual bool dump(std::ostream& output) = 0;
00043
00049     virtual bool load(std::istream& input) = 0;
00050
00059     virtual bool get(const std::string& name, PersistenceRecord *entry) = 0;
00060
00071     virtual bool put(const std::string name, const std::string type, const Util::identification id,
PersistenceRecord *fields) = 0;
00072
00080     virtual int for_each(std::function<int(const std::string&) > delegate) = 0;
00081 };
00082
00083 #endif // MODELSERIALIZER_H

```

11.270 XmlSerializer.h

```

00001 #ifndef XMLSERIALIZER_H
00002 #define XMLSERIALIZER_H
00003
00004 #include <unordered_map>
00005 #include <memory>
00006
00007 #include "Serializer.h"
00008 #include "../model/Model.h"
00009
00010 class XmlSerializer : public Serializer {
00011 public:
00012     explicit XmlSerializer(Model *model);
00013
00014 public: // Serializer interface
00015     PersistenceRecord* newPersistenceRecord() override;
00016     bool dump(std::ostream& output) override;
00017     bool load(std::istream& input) override;
00018     bool get(const std::string& name, PersistenceRecord *entry) override;
00019     bool put(const std::string name, const std::string type, const Util::identification id,
PersistenceRecord *fields) override;
00020     int for_each(std::function<int(const std::string&) > delegate) override;
00021
00022 private:
00023     Model *_model{};
00024     std::unordered_map<std::string, std::unique_ptr<PersistenceRecord> _metaobjects
00025     {
00026     };
00027     std::unordered_map<std::string, std::unique_ptr<PersistenceRecord> _components
00028     {
00029     };
00030 };
00031
00032 #endif // XMLSERIALIZER_H

```

11.271 Plugin.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   Plugin.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 21 de Junho de 2018, 12:58
00012  */
00013

```

```

00014 #ifndef PLUGIN_H
00015 #define PLUGIN_H
00016
00017 #include <string>
00018 #include <functional>
00019 #include <list>
00020
00021 #include "../util/Util.h"
00022 #include "Persistence.h"
00023 #include "PluginInformation.h"
00024
00025 //namespace GenesysKernel {
00026
00027 class Plugin {
00028 public:
00029     Plugin(std::string filename_so_dll);
00030     Plugin(StaticGetPluginInformation getInformation); // @ToDo: (importante): temporary. Just while
00031     compiled together
00032     virtual ~Plugin();
00033 public:
00034     std::string show();
00035 public:
00036     bool isValidPlugin() const;
00037     PluginInformation* getPluginInfo() const;
00038     ModelDataDefinition* loadNew(Model* model, PersistenceRecord *fields);
00039     bool loadAndInsertNew(Model* model, PersistenceRecord *fields);
00040     ModelDataDefinition* newInstance(Model* model, std::string name = "");
00041 private:
00042     ModelComponent* _loadNewComponent(Model* model, PersistenceRecord *fields);
00043     ModelDataDefinition* _loadNewElement(Model* model, PersistenceRecord *fields);
00044 private: // read only
00045     // Default invalid state avoids reading indeterminate validity after construction failures.
00046     bool _isValidPlugin = false;
00047     // Store plugin metadata pointer with a safe default for invalid plugin construction paths.
00048     PluginInformation* _pluginInfo = nullptr;
00049 private:
00050     StaticGetPluginInformation _StaticMethodGetInformation;
00051 };
00052 //namespace\\}
00053 #endif /* PLUGIN_H */

```

11.272 PluginConnector_if.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   PluginConnector_if.h
00009  * Author: rlcancian
00010  *
00011  * Created on 9 de Setembro de 2019, 19:17
00012  */
00013
00014 #ifndef PLUGINCONNECTOR_IF_H
00015 #define PLUGINCONNECTOR_IF_H
00016
00017 #include<string>
00018 #include "Plugin.h"
00019 #include "../util/List.h"
00020
00021 //namespace GenesysKernel {
00022
00023 class PluginConnector_if {
00024 public:
00025     virtual ~PluginConnector_if() = default;
00026     virtual Plugin* check(const std::string dynamicLibraryFilename) = 0;
00027     virtual Plugin* connect(const std::string dynamicLibraryFilename) = 0;
00028     virtual List<std::string>* find() = 0;
00029
00030     virtual bool disconnect(const std::string dynamicLibraryFilename) = 0;
00031     virtual bool disconnect(Plugin* plugin) = 0;
00032 };
00033 //namespace\\}
00034 #endif /* PLUGINCONNECTOR_IF_H */

```

11.273 PluginInformation.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   PluginInformation.h
00009  * Author: rlcancian
00010  *
00011  * Created on 9 de Setembro de 2019, 20:02
00012  */
00013
00014 #ifndef PLUGININFORMATION_H
00015 #define PLUGININFORMATION_H
00016
00017 #include <map>
00018 #include <list>
00019 #include <string>
00020
00021 //namespace GenesysKernel {
00022 class ModelDataDefinition;
00023 class ModelComponent;
00024 class Model;
00025 class ModelDataManager;
00026 class PersistenceRecord;
00027
00028 typedef ModelComponent* (*StaticLoaderComponentInstance)(Model*, PersistenceRecord*);
00029 typedef ModelDataDefinition* (*StaticLoaderDataDefinitionInstance)(Model*, PersistenceRecord*);
00030 typedef ModelDataDefinition* (*StaticConstructorDataDefinitionInstance)(Model*, std::string);
00031 class PluginInformation;
00032 typedef PluginInformation* (*StaticGetPluginInformation)();
00033
00034 class SystemDependency {
00035 public:
00036     enum class OS {
00037         Any,
00038         Linux,
00039         Windows,
00040         MacOS,
00041         Unknown
00042     };
00043
00044     SystemDependency() = default;
00045     SystemDependency(OS os, std::string name, std::string installCommand = "", std::string
checkCommand = "");
00046
00047     OS getOS() const;
00048     std::string getName() const;
00049     std::string getInstallCommand() const;
00050     std::string getCheckCommand() const;
00051     std::string show() const;
00052
00053     static std::string osToString(OS os);
00054
00055 private:
00056     OS _os = OS::Any;
00057     std::string _name = "";
00058     std::string _installCommand = "";
00059     std::string _checkCommand = "";
00060 };
00061
00062 class PluginInformation {
00063 public:
00064     PluginInformation(std::string pluginTypeName, StaticLoaderComponentInstance componentloader,
StaticConstructorDataDefinitionInstance elementConstructor);
00065     PluginInformation(std::string pluginTypeName, StaticLoaderDataDefinitionInstance elementloader,
StaticConstructorDataDefinitionInstance elementConstructor);
00066     // Release owned metadata containers allocated alongside plugin-information objects.
00067     virtual ~PluginInformation();
00068 public:
00069     // gets
00070     StaticLoaderDataDefinitionInstance getDataDefinitionLoader() const;
00071     StaticLoaderComponentInstance GetComponentLoader() const;
00072     StaticConstructorDataDefinitionInstance getDataDefinitionConstructor() const;
00073     bool isGenerateReport() const;
00074     bool isComponent() const;
00075     bool isSendTransfer() const;
00076     bool isReceiveTransfer() const;
00077     bool isSink() const;
00078     bool isSource() const;
00079     std::string getObservation() const;
00080     std::string getVersion() const;
00081     std::string getDate() const;
00082     std::string getAuthor() const;

```

```

00083     std::string getPluginTypeName() const;
00084     // sets
00085     void insertDynamicLibFileDependence(std::string filename);
00086     // @ToDo: (importante): Clarify ownership for the incoming list pointer.
00087     // This setter currently replaces the internal raw pointer without documenting transfer semantics.
00088     void setDynamicLibFilenameDependencies(std::list<std::string>* dynamicLibFilenameDependencies);
00089     std::list<std::string>* getDynamicLibFilenameDependencies() const;
00090     void insertSystemDependency(const SystemDependency& dependency);
00091     const std::list<SystemDependency>* getSystemDependencies() const;
00092     bool hasSystemDependencies() const;
00093     void setGenerateReport(bool generateReport);
00094     void setSendTransfer(bool sendTransfer);
00095     void setReceiveTransfer(bool receiveTransfer);
00096     void setSink(bool Sink);
00097     void setSource(bool Source);
00098     void setObservation(std::string observation);
00099     void setVersion(std::string version);
00100     void setDate(std::string date);
00101     void setAuthor(std::string author);
00102     void setMaximumOutputs(unsigned short _maximumOutputs);
00103     unsigned short getMaximumOutputs() const;
00104     void setMinimumOutputs(unsigned short _minimumOutputs);
00105     unsigned short getMinimumOutputs() const;
00106     void setMaximumInputs(unsigned short _maximumInputs);
00107     unsigned short getMaximumInputs() const;
00108     void setMinimumInputs(unsigned short _minimumInputs);
00109     unsigned short getMinimumInputs() const;
00110     void setDescriptionHelp(std::string _descriptionHelp);
00111     std::string getDescriptionHelp() const;
00112     // @ToDo: (importante): Clarify ownership for the incoming fields map.
00113     // This setter currently replaces the internal raw pointer without documenting lifetime
    expectations.
00114     void setFields(std::map<std::string, std::string>* _fiewlds);
00115     std::map<std::string, std::string>* getFields() const;
00116     void setLanguageTemplate(std::string _languageTemplate);
00117     std::string getLanguageTemplate() const;
00118     void setCategory(std::string _category);
00119     std::string getCategory() const;
00120     static std::string categoryFolderName(const std::string& category);
00121 public:
00122 private:
00123     std::string _author = "prof. Dr. Ing. Rafael Luiz Cancian";
00124     std::string _date = "01/08/2018";
00125     std::string _version = "0.9.1";
00126     std::string _category = "DiscreteProcessing";
00127     std::string _observation = "First implementation not fully completed nor tested. Use with
    caution.";
00128     std::string _descriptionHelp = "";
00129     std::string _languageTemplate = "";
00130     bool _isSource = false;
00131     bool _isSink = false;
00132     bool _receiveTransfer = false;
00133     bool _sendTransfer = false;
00134     bool _generateReport = false;
00135     std::list<std::string>* _dynamicLibFilenameDependencies = new std::list<std::string>();
00136     std::list<SystemDependency>* _systemDependencies = new std::list<SystemDependency>();
00137     std::map<std::string, std::string>* _fields = new std::map<std::string, std::string>();
00138     // set from constructor
00139     std::string _pluginTypeName;
00140     bool _isComponent = false;
00141     unsigned short _minimumInputs = 1;
00142     unsigned short _maximumInputs = 1;
00143     unsigned short _minimumOutputs = 1;
00144     unsigned short _maximumOutputs = 1;
00145     StaticLoaderComponentInstance _componentloader;
00146     StaticLoaderDataDefinitionInstance _elementloader;
00147     StaticConstructorDataDefinitionInstance _elementConstructor;
00148 };
00149 //namespace\}
00150
00151 #endif /* PLUGININFORMATION_H */

```

11.274 PluginManager.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   PluginManager.h
00009  * Author: rafael.luiiz.cancian

```

```

00010  *
00011  * Created on 30 de Maio de 2019, 17:49
00012  */
00013
00014 #ifndef PLUGINMANAGER_H
00015 #define PLUGINMANAGER_H
00016
00017 #include <functional>
00018 #include <string>
00019 #include <vector>
00020
00021 #include "model/Model.h"
00022 #include "../util/List.h"
00023 #include "../util/Util.h"
00024 #include "Plugin.h"
00025 #include "PluginConnector_if.h"
00026 #include "SystemDependencyResolver.h"
00027
00028 class Simulator;
00029 class Model;
00030
00031 struct PluginInsertionOptions {
00032     std::function<bool(const SystemDependencyCheckResult&)> confirmSystemDependencyInstallation;
00033 };
00034
00035 class PluginLoadIssue {
00036 public:
00037     enum class Reason {
00038         InvalidPlugin,
00039         MissingSystemDependency,
00040         DynamicDependencyFailure,
00041         ConnectionFailure,
00042         InsertionFailure,
00043         Exception
00044     };
00045
00046     PluginLoadIssue(); // fix // = default;
00047     PluginLoadIssue(std::string filename,
00048                     std::string pluginTypeName,
00049                     Reason reason,
00050                     std::string message,
00051                     SystemDependencyCheckResult systemDependencyResult = {}) noexcept;
00052
00053     const std::string& getFilename() const;
00054     const std::string& getPluginTypeName() const;
00055     Reason getReason() const;
00056     const std::string& getMessage() const;
00057     const SystemDependencyCheckResult& getSystemDependencyResult() const;
00058     bool hasSystemDependencyResult() const;
00059     static std::string reasonToString(Reason reason);
00060     std::string diagnosticText() const;
00061
00062 private:
00063     std::string _filename;
00064     std::string _pluginTypeName;
00065     Reason _reason;
00066     std::string _message;
00067     SystemDependencyCheckResult _systemDependencyResult;
00068 };
00069
00070 //namespace GenesysKernel {
00071
00072 class PluginManager {
00073 public:
00074     PluginManager(Simulator* simulator);
00075     PluginManager(Simulator* simulator, PluginConnector_if* pluginConnector, SystemCommandExecutor_if*
systemCommandExecutor);
00076     virtual ~PluginManager();
00077
00078 public:
00079     std::string show();
00080
00081 public:
00082     List<Plugin*>* completePluginsFieldsAndTemplates();
00083
00084 public:
00085     bool check(const std::string& dynamicLibraryFilename);
00086     SystemDependencyCheckResult checkSystemDependencies(const std::string& dynamicLibraryFilename);
00087     List<std::string*>* discoverPluginFileNames() const;
00088     List<PluginLoadIssue*>* getPluginLoadIssues() const;
00089     void clearPluginLoadIssues();
00090     void clearPluginLoadIssue(const std::string& dynamicLibraryFilename);
00091     Plugin* insert(const std::string& dynamicLibraryFilename);
00092     Plugin* insert(const std::string& dynamicLibraryFilename, const PluginInsertionOptions& options);
00093     bool remove(const std::string& dynamicLibraryFilename);
00094     bool remove(Plugin* plugin);
00095     Plugin* find(const std::string& pluginTypeName);
00096
00097
00098
00099
00100
00101
00102
00103
00104
00105
00106
00107
00108
00109
00110
00111
00112
00113
00114
00115
00116
00117
00118
00119
00120
00121
00122
00123
00124
00125
00126
00127
00128
00129
00130
00131
00132
00133
00134
00135
00136
00137
00138
00139
00140
00141
00142
00143
00144
00145
00146
00147
00148
00149
00150

```



```

00156     std::vector<std::string> getDataDefinitionPluginTypeNames() const;
00158     std::string sourceIncludePathFor(const std::string& pluginTypeName);
00160     List<Plugin*>* autoInsertPlugins(const std::string& pluginsListFilename,
00161                                     const bool lookForPluginsIfFilenameNotFound = true);
00163     List<Plugin*>* autoInsertPlugins(const std::string& pluginsListFilename,
00164                                     const bool lookForPluginsIfFilenameNotFound,
00165                                     const PluginInsertionOptions& options);
00167     List<Plugin*>* autoInsertPlugins();
00169     List<Plugin*>* autoInsertPlugins(const PluginInsertionOptions& options);
00170
00171 public:
00173     Plugin* front();
00175     Plugin* next();
00177     Plugin* last();
00179     unsigned int size();
00181     Plugin* getAtRank(unsigned int rank);
00182
00183 public:
00185     ModelDataDefinition* newInstance(const std::string& pluginTypeName, Model* model, std::string name
    = "");
00186
00187     template <typename T>
00188     T* newInstance(Model* model, std::string name = "") {
00189         name = Util::StrReplace(name, " ", "_");
00190         Plugin* plugin;
00191         std::string pluginTypeName = Util::TypeOf<T>();
00192         // @ToDo: (pequena alteração): Use Find method??
00193         for (unsigned short i = 0; i < _plugins->size(); i++) {
00194             plugin = _plugins->getAtRank(i);
00195             if (plugin->getPluginInfo()->getPluginTypeName() == pluginTypeName) {
00196                 T* instance;
00197                 StaticConstructorDataDefinitionInstance constructor = plugin->getPluginInfo()->
00198                     getDataDefinitionConstructor();
00199                 instance = static_cast<T*>(constructor(model, name));
00200                 if (model != nullptr) {
00201                     ModelDataDefinition::CreateInternalData(instance);
00202                 }
00203                 return instance;
00204             }
00205         }
00206         // Keep this header free of Simulator implementation details to avoid include cycles.
00207         return nullptr;
00208     }
00209
00210 private:
00212     bool _insert(Plugin* plugin, const PluginInsertionOptions& options, const std::string&
    dynamicLibraryFilename);
00214     bool _preflightAndMaybeInstallSystemDependencies(PluginInformation* plugInfo,
00215                                                         const PluginInsertionOptions& options,
00216                                                         SystemDependencyCheckResult* blockingResult =
    nullptr,
00217                                                         std::string* failureMessage = nullptr);
00219     void _recordLoadIssue(const PluginLoadIssue& issue);
00221     void _removeLoadIssue(const std::string& dynamicLibraryFilename, const std::string&
    pluginTypeName);
00223     void _insertDefaultKernelElements();
00225     List<Plugin*>* _autoFindPlugins(const PluginInsertionOptions& options);
00226
00227 private:
00228     List<Plugin*>* _plugins;
00229     List<PluginLoadIssue*>* _pluginLoadIssues;
00230     Simulator* _simulator;
00231     PluginConnector_if* _pluginConnector;
00232     SystemCommandExecutor_if* _systemCommandExecutor;
00233 };
00234
00235 //namespace\\
00236 #endif /* PLUGINMANAGER_H */

```

11.275 PropertyGenesys.h

```

00001 #ifndef PROPERTY_H
00002 #define PROPERTY_H
00003
00004 #include "SimulationControlAndResponse.h"
00005 #include "model/ModelComponent.h"
00006
00007 class PropertyEditorGenesys {
00008 public:
00009     PropertyEditorGenesys() = default;
00010     virtual ~PropertyEditorGenesys() = default;
00011

```

```

00012 public:
00013     std::list<ModelComponent*> getElements() {
00014         return _elements;
00015     }
00016
00017 public:
00018     void changeSimulationControl(SimulationControl* control, const std::string& value, bool remove =
00019         false) {
00019         if (control != nullptr) {
00020             control->setValue(value, remove);
00021         }
00022     }
00023
00024     SimulationControl* findSimulationControl(const std::string& id, const std::string& attribute) {
00025         for (auto element : _elements) {
00026             if (std::to_string(element->getId()) == id) {
00027                 for (auto control : *element->getSimulationControls()->list()) {
00028                     if (control->getName() == attribute) {
00029                         return control;
00030                     }
00031                 }
00032             }
00033         }
00034         return nullptr;
00035     }
00036
00037     // Legacy compatibility wrapper. Prefer changeSimulationControl.
00038     //void changeProperty(SimulationControl* control, const std::string& value, bool remove = false) {
00039     //    changeSimulationControl(control, value, remove);
00040     //}
00041
00042     // Legacy compatibility wrapper. Prefer findSimulationControl.
00043     //SimulationControl* findProperty(const std::string& id, const std::string& attribute) {
00044     //    return findSimulationControl(id, attribute);
00045     //}
00046
00047     void addElement(ModelComponent* component) {
00048         _elements.push_back(component);
00049     }
00050
00051 private:
00052     std::list<ModelComponent*> _elements;
00053 };
00054
00055 #endif

```

11.276 ScenarioExperiment_if.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: ScenarioExperiment_if.h
00009  * Author: rafael.lui.cancian
00010  *
00011  * Created on 10 de Outubro de 2018, 14:52
00012  */
00013
00014 #ifndef SCENARIOEXPERIMENT_IF_H
00015 #define SCENARIOEXPERIMENT_IF_H
00016
00017 class ScenarioExperiment_if {
00018 public:
00019     virtual ~ScenarioExperiment_if() = default;
00020 };
00021
00022 #endif /* SCENARIOEXPERIMENT_IF_H */

```

11.277 SimulationControlAndResponse.h

```

00001 #pragma once
00002
00003 #include <string>
00004 #include <sstream>
00005 #include <functional>
00006 #include <list>

```

```

00007 #include <vector>
00008 #include <stdexcept>
00009 #include <type_traits>
00010 #include "../util/Util.h"
00011 #include "../util/List.h"
00012
00013 //namespace GenesysKernel {
00014
00015 class ModelDataDefinition;
00016
00017 class SourceCodeString {
00018 public:
00019     SourceCodeString() = default;
00020     SourceCodeString(const std::string& value) : _value(value) {}
00021     SourceCodeString(std::string&& value) : _value(std::move(value)) {}
00022
00023     const std::string& str() const { return _value; }
00024     operator std::string() const { return _value; }
00025
00026 private:
00027     std::string _value;
00028 };
00029
00030 inline std::ostream& operator<<(std::ostream& os, const SourceCodeString& value) {
00031     os << value.str();
00032     return os;
00033 }
00034
00035 class PropertyGenesysBase {
00036 public:
00037     PropertyGenesysBase(std::string className, std::string elementName, std::string propertyName,
00038         std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false) {
00039         _className=className;
00040         _elementName=elementName;
00041         _propertyName=propertyName;
00042         _whatsThis=whatsThis;
00043         _isList=isList;
00044         _isClass=isClass;
00045         _isEnum=isEnum;
00046     }
00047     virtual ~PropertyGenesysBase() = default;
00048 public:
00049     std::string whatsThis() const { return _whatsThis; }
00050     std::string getClassName() const { return _className; }
00051     std::string getElementName() const { return _elementName; }
00052     std::string getName() const { return _propertyName; }
00053     std::string propertyType() const { return _propertyType; }
00054     bool getIsList() const { return _isList; }
00055     bool getIsClass() const { return _isClass; }
00056     bool getIsEnum() const { return _isEnum; }
00057     void setWhatsThis(const std::string text);
00058     void setName(std::string name) { _propertyName = name; }
00059 protected:
00060     std::string _elementName;
00061     std::string _className;
00062     std::string _propertyName;
00063     std::string _propertyType;
00064     std::string _whatsThis;
00065     bool _isList;
00066     bool _isClass;
00067     bool _isEnum;
00068 };
00069
00070 class PropertyGenesys: public PropertyGenesysBase {
00071 public:
00072     PropertyGenesys(std::string className, std::string elementName, std::string propertyName,
00073         std::string whatsThis="", std::string propertyId="", std::string toolTip="", std::string statusTip="")
00074         : PropertyGenesysBase(className, elementName, propertyName, whatsThis){
00075         _propertyId=propertyId;
00076         _toolTip=toolTip;
00077         _statusTip=statusTip;
00078     }
00079     virtual ~PropertyGenesys() = default;
00080 public:
00081     std::list<PropertyGenesys*> subProperties() const { return _subProperties; };
00082     std::string toolTip() const { return _toolTip; }
00083     std::string statusTip() const { return _statusTip; }
00084     std::string propertyName() const { return _propertyName; }
00085     std::string propertyId() const { return _propertyId; }
00086     bool isEnabled() const { return _isEnabled; }
00087     bool isModified() const { return _isModified; }
00088     bool isSubProperty()const { return _isSubProperty; }
00089     bool hasValue() const { return _hasValue; }
00090     std::string valueText() const { return "?"; } // @ToDo: (importante): implement value text
00091     void setToolTip(const std::string text);
00092     void setStatusTip(const std::string text);

```

```

00103     void setWhatsThis(const std::string text);
00104     void setPropertyName(const std::string text);
00105     void setPropertyId(const std::string text);
00106     void setEnabled(bool enable);
00107     void setModified(bool modified);
00108 public:
00109     virtual bool compare(PropertyGenesys* otherProperty) const { return true; } // @ToDo: (importante):
    implement comparison
00110 public:
00111     void addSubProperty(PropertyGenesys *property) {}
00112     void insertSubProperty(PropertyGenesys *property, PropertyGenesys *afterProperty) {}
00113     void removeSubProperty(PropertyGenesys *property) {}
00114 protected:
00115     void propertyChanged() {}
00116 protected:
00117     //friend class QtAbstractPropertyManager;
00118     std::string _className;
00119     std::string _propertyName;
00120     std::string _whatsThis;
00121     std::string _propertyId;
00122     std::string _toolTip;
00123     std::string _statusTip;
00124     std::list<PropertyGenesys*> _subProperties;
00125     bool _isEnabled;
00126     bool _isModified;
00127     bool _hasValue;
00128     bool _isSubProperty;
00129 };
00130
00137 class SimulationResponse: public PropertyGenesysBase {
00138 public:
00139     SimulationResponse(std::string className, std::string elementName, std::string propertyName,
    std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
00140         : PropertyGenesysBase(className, elementName, propertyName, whatsThis, isList, isClass,
    isEnum) {
00141     }
00142     virtual ~SimulationResponse() = default;
00143 public:
00144     std::string show() const {
00145         std::string msg = "classname="+_className+ " , elementName="+_elementName+", name=\"\" +
    _propertyName + "\"";
00146         msg += " , value="+getValue();
00147         return msg;
00148     }
00149     virtual std::string getValue() const = 0;
00150     virtual List<std::string>* getStrValues() { return nullptr; };
00151 };
00152
00153 enum class SimulationControlEditorHint {
00154     Default,
00155     CodeEditor,
00156     MultiLineText
00157 };
00158
00165 class SimulationControl: public SimulationResponse {
00166 public:
00167     SimulationControl(std::string className, std::string elementName, std::string propertyName,
    std::string whatsThis="", bool isList=false, bool isClass=false, bool isEnum=false)
00168         : SimulationResponse(className, elementName, propertyName, whatsThis, isList, isClass,
    isEnum) {
00169     }
00170     std::string show() const {
00171         std::string msg = "classname="+_className+ " , elementName="+_elementName+",
    name=\"\"+_propertyName+\"";
00172         if (_readonly) {
00173             msg += " (readonly)";
00174         }
00175         //msg += " , type="+_propertyType;
00176         msg += " , value="+getValue();
00177         return msg;
00178     }
00179     bool isReadOnly() const { return _readonly; }
00180 public:
00181     virtual void setValue(std::string value, bool remove=false) = 0;
00182     virtual List<SimulationControl*>* getChildSimulationControls(int index=0) { return nullptr; };
00183     virtual bool hasObjectInstance() const { return true; }
00184     virtual bool ensureObjectInstance() { return hasObjectInstance(); }
00185     virtual bool isModelDataDefinitionReference() const { return false; }
00186     virtual ModelDataDefinition* getReferencedModelDataDefinition() const { return nullptr; }
00187     // This method validates a proposed string value before GUI editors commit it.
00188     virtual bool validateProposedValue(const std::string& value, std::string& errorMessage) const {
00189         (void)value;
00190         errorMessage.clear();
00191         return true;
00192     }
00193     // This method exposes the preferred editor affordance for GUI-side property editing.
00194     virtual SimulationControlEditorHint preferredEditorHint() const { return _preferredEditorHint; }
00195
00196
00197
00198
00199
00200

```

```

00201     void setPreferredEditorHint(SimulationControlEditorHint hint) { _preferredEditorHint = hint; }
00202     // This method exposes the ModelDataDefinition represented by a class-list row, when any.
00203     //
00204     // It lets generic tooling classify list members themselves as editable references without
00205     // knowing the concrete owner class. Non ModelDataDefinition lists keep the default nullptr.
00206     virtual ModelDataDefinition* getListElementModelDataDefinition(int index) const {
00207         (void)index;
00208         return nullptr;
00209     }
00210     // This method exposes whether the property should be rendered as an inline expandable object
tree.
00211     virtual bool supportsInlineExpansion() const { return getIsClass() && !getIsList(); }
00212     // This method exposes whether the property should be edited by the dedicated list editor.
00213     virtual bool supportsListEditor() const { return getIsList(); }
00214     // This method exposes whether the property supports choosing an existing object instance.
00215     virtual bool supportsExistingObjectSelection() const { return false; }
00216     // This method exposes whether the property supports creating a new object instance.
00217     virtual bool supportsObjectCreation() const { return false; }
00218     // This method exposes whether the property supports typed creation of a new list element.
00219     virtual bool supportsNewListElementCreation() const { return false; }
00220     // This method exposes which concrete element types can be created for a polymorphic list.
00221     //
00222     // A nullptr return means the list has no typed-creation contract and should use the legacy
00223     // single "create new element" action. Callers own the returned List when it is not nullptr.
00224     virtual List<std::string>* getCreatableListElementTypes() { return nullptr; }
00225     // This method exposes the concrete type currently accepted by a polymorphic list.
00226     //
00227     // Empty string means no concrete type has been selected yet. Implementations may lock the
00228     // current type once the list has members, while still allowing it to be configured while empty.
00229     virtual std::string getCurrentListElementType() const { return ""; }
00230     // This method configures the concrete type currently accepted by a polymorphic list.
00231     //
00232     // The default implementation rejects the operation. Implementations that support it should
00233     // validate the requested type against getCreatableListElementTypes() and their own invariants.
00234     virtual bool setCurrentListElementType(const std::string& typeName) {
00235         (void)typeName;
00236         return false;
00237     }
00238     // This method exposes whether the property is an inline object (and not a ModelDataDefinition
reference).
00239     virtual bool isInlineObjectProperty() const {
00240         return supportsInlineExpansion() && !isModelDataDefinitionReference();
00241     }
00242     // This method provides an explicit object-creation operation for class-like properties.
00243     virtual bool createObjectInstance(const std::string& value = "") {
00244         (void)value;
00245         return false;
00246     }
00247     virtual List<std::string>* getCreatableReferenceTypes() { return nullptr; }
00248     virtual std::string getCurrentReferenceType() const { return ""; }
00249     virtual bool setCurrentReferenceType(const std::string& typeName) {
00250         (void)typeName;
00251         return false;
00252     }
00253     virtual bool createObjectInstanceOfType(const std::string& typeName, const std::string& value =
"" ) {
00254         if (typeName.empty()) {
00255             return createObjectInstance(value);
00256         }
00257         (void)value;
00258         return false;
00259     }
00260     // This method provides an explicit typed list-element creation operation for list-like
properties.
00261     virtual bool createNewListElement(const std::string& value = "") {
00262         (void)value;
00263         return false;
00264     }
00265     // This method creates a list element of a requested concrete type.
00266     //
00267     // The default keeps old controls working: when no concrete type is provided, it delegates to
00268     // createNewListElement(); otherwise the operation is unsupported.
00269     virtual bool createNewListElementOfType(const std::string& typeName, const std::string& value =
"" ) {
00270         if (typeName.empty()) {
00271             return createNewListElement(value);
00272         }
00273         (void)value;
00274         return false;
00275     }
00276     virtual List<SimulationControl*>* getEditableChildSimulationControls(int index=0) {
00277         if (supportsInlineExpansion() && !hasObjectInstance()) {
00278             if (!supportsObjectCreation()) {
00279                 return nullptr;
00280             }
00281             if (!ensureObjectInstance()) {
00282                 return nullptr;

```

```

00283         }
00284     }
00285     return getChildSimulationControls(index);
00286 }
00287 protected:
00288     void _ensureWritable(const char* operation) const {
00289         if (_readonly) {
00290             throw std::logic_error(std::string("Cannot ") + operation + " readonly SimulationControl
00291 \'" + _propertyName + "\'");
00292         }
00293         bool _readonly = false;
00294         SimulationControlEditorHint _preferredEditorHint = SimulationControlEditorHint::Default;
00295     };
00296
00297 // -----
00298
00299 template<typename Class, typename T>
00300 std::function<T()> DefineSimulationGetter(Class* object, T(Class::*function)() const) {
00301     return std::bind(function, object);
00302 }
00303
00304 template<typename Class, typename T>
00305 std::function<T()> DefineSimulationGetter(Class* object, T(Class::*function)()) {
00306     return std::bind(function, object);
00307 }
00308
00309 template<typename Class, typename T>
00310 std::function<void(T)> DefineSimulationSetter(Class* object, void (Class::*function)(T)) {
00311     return std::bind(function, object, std::placeholders::_1);
00312 }
00313
00314 typedef std::function<std::string()> GetterString;
00315 typedef std::function<void(std::string)> SetterString;
00316 typedef std::function<bool(const std::string&, std::string&)> ValidatorString;
00317 class SimulationControlString: public SimulationControl {
00318 public:
00319     // SimulationControlString(GetterString getter, std::string className, std::string elementName,
00320     // std::string propertyName, std::string whatsThis="") : SimulationControl(className, elementName,
00321     // propertyName, whatsThis) {
00322     //     SimulationControlString(getter, nullptr, className, propertyName, whatsThis);
00323     // }
00324     SimulationControlString(GetterString getter, SetterString setter, std::string className,
00325     std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
00326     isClass=false, bool isEnum=false, ValidatorString validator=nullptr) : SimulationControl(className,
00327     elementName, propertyName, whatsThis, isList, isClass, isEnum) {
00328         _getter= getter;
00329         _setter = setter;
00330         _validator = validator;
00331         _readonly = setter == nullptr;
00332         _propertyType = Util::TypeOf<std::string>();
00333     }
00334 public:
00335     virtual std::string getValue() const override { return _getter(); }
00336     virtual bool validateProposedValue(const std::string& value, std::string& errorMessage) const
00337     override {
00338         if (!_validator) {
00339             errorMessage.clear();
00340             return true;
00341         }
00342         return _validator(value, errorMessage);
00343     }
00344     virtual void setValue(std::string value, bool remove=false) override {
00345         _ensureWritable("set value of");
00346         if (!_setter) {
00347             throw std::logic_error("SimulationControlString setter is not defined");
00348         }
00349         _setter(value);
00350     };
00351 private:
00352     GetterString _getter;
00353     SetterString _setter;
00354     ValidatorString _validator;
00355 };
00356
00357 typedef std::function<SourceCodeString()> GetterSourceCodeString;
00358 typedef std::function<void(SourceCodeString)> SetterSourceCodeString;
00359 class SimulationControlSourceCodeString: public SimulationControl {
00360 public:
00361     SimulationControlSourceCodeString(GetterSourceCodeString getter, SetterSourceCodeString setter,
00362     std::string className, std::string elementName, std::string propertyName, std::string whatsThis="",
00363     bool isList=false, bool isClass=false, bool isEnum=false, ValidatorString validator=nullptr) :
00364     SimulationControl(className, elementName, propertyName, whatsThis, isList, isClass, isEnum) {
00365         _getter = getter;
00366         _setter = setter;
00367         _validator = validator;
00368         _readonly = setter == nullptr;

```

```

00360         _propertyType = Util::TypeOf<SourceCodeString>();
00361         _preferredEditorHint = SimulationControlEditorHint::CodeEditor;
00362     }
00363 public:
00364     virtual std::string getValue() const override { return _getter().str(); }
00365     virtual bool validateProposedValue(const std::string& value, std::string& errorMessage) const
00366     override {
00367         if (!_validator) {
00368             errorMessage.clear();
00369             return true;
00370         }
00371         return _validator(value, errorMessage);
00372     }
00373     virtual void setValue(std::string value, bool remove=false) override {
00374         (void)remove;
00375         _ensureWritable("set value of");
00376         if (!_setter) {
00377             throw std::logic_error("SimulationControlSourceCodeString setter is not defined");
00378         }
00379         _setter(SourceCodeString(std::move(value)));
00380     };
00381 private:
00382     GetterSourceCodeString _getter;
00383     SetterSourceCodeString _setter;
00384     ValidatorString _validator;
00385 };
00386
00387 // -----
00388
00389 typedef std::function<double()> GetterDouble;
00390 typedef std::function<void(double)> SetterDouble;
00391
00392 class SimulationResponseDouble: public SimulationResponse {
00393 public:
00394     SimulationResponseDouble(GetterDouble getter, std::string className, std::string elementName,
00395                             std::string propertyName, std::string whatsThis="", bool isList=false, bool isClass=false, bool
00396                             isEnum=false)
00397     : SimulationResponse(className, elementName, propertyName, whatsThis, isList, isClass, isEnum)
00398     {
00399         _getter = getter;
00400         _propertyType = Util::TypeOf<double>();
00401     }
00402 public:
00403     virtual std::string getValue() const override { return std::to_string(_getter()); }
00404 private:
00405     GetterDouble _getter;
00406 };
00407
00408 class SimulationControlDouble: public SimulationControl {
00409 public:
00410     // SimulationControlDouble(GetterDouble getter, std::string className, std::string elementName,
00411     // std::string propertyName, std::string whatsThis="") : SimulationControl(className, elementName,
00412     // propertyName, whatsThis) {
00413     //     SimulationControlDouble(getter, nullptr, className, propertyName, whatsThis);
00414     // }
00415     SimulationControlDouble(GetterDouble getter, SetterDouble setter, std::string className,
00416                             std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
00417                             isClass=false, bool isEnum=false) : SimulationControl(className, elementName, propertyName, whatsThis,
00418                             isList, isClass, isEnum){
00419         _getter= getter;
00420         _setter = setter;
00421         _readonly = setter == nullptr;
00422         _propertyType = Util::TypeOf<double>();
00423     }
00424 public:
00425     virtual std::string getValue() const override { return std::to_string(_getter()); }
00426     virtual void setValue(std::string value, bool remove=false) override {
00427         _ensureWritable("set value of");
00428         if (!_setter) {
00429             throw std::logic_error("SimulationControlDouble setter is not defined");
00430         }
00431         _setter(std::stod(value));
00432     };
00433 private:
00434     GetterDouble _getter;
00435     SetterDouble _setter;
00436 };
00437
00438 // -----
00439
00440 typedef std::function<bool()> GetterBool;
00441 typedef std::function<void(bool)> SetterBool;
00442
00443 class SimulationControlBool: public SimulationControl {
00444 public:
00445     // SimulationControlBool(GetterBool getter, std::string className, std::string elementName,
00446     // std::string propertyName, std::string whatsThis="") : SimulationControl(className, elementName,

```

```

        propertyName, whatsThis) {
00437 //      SimulationControlBool(getter, nullptr, className, propertyName, whatsThis);
00438 //  }
00439      SimulationControlBool(GetterBool getter, SetterBool setter, std::string className, std::string
        elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
        isClass=false, bool isEnum=false) : SimulationControl(className, elementName, propertyName, whatsThis,
        isList, isClass, isEnum){
00440          _getter= getter;
00441          _setter = setter;
00442          _readonly = setter == nullptr;
00443          _propertyType = Util::TypeOf<bool>();
00444      }
00445  public:
00446      virtual std::string getValue() const override { return std::to_string(_getter()); }
00447      virtual void setValue(std::string value, bool remove=false) override {
00448          _ensureWritable("set value of");
00449          if (!_setter) {
00450              throw std::logic_error("SimulationControlBool setter is not defined");
00451          }
00452          std::stringstream auxStr(value);
00453          auxStr >> std::boolalpha;
00454          bool boolVal = false;
00455          if (!(auxStr >> boolVal)) {
00456              boolVal = std::stoi(value) != 0;
00457          }
00458          _setter(boolVal);
00459      };
00460  private:
00461      GetterBool _getter;
00462      SetterBool _setter;
00463  };
00464
00465
00466 // -----
00467
00468 typedef std::function<unsigned int()> GetterUInt;
00469 typedef std::function<void(unsigned int)> SetterUInt;
00470 class SimulationControlUInt: public SimulationControl {
00471  public:
00472      // SimulationControlUInt(GetterUInt getter, std::string className, std::string elementName,
        std::string propertyName, std::string whatsThis="") : SimulationControl(className, elementName,
        propertyName, whatsThis) {
00473          //      SimulationControlUInt(getter, nullptr, className, propertyName, whatsThis);
00474          //  }
00475      SimulationControlUInt(GetterUInt getter, SetterUInt setter, std::string className, std::string
        elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
        isClass=false, bool isEnum=false) : SimulationControl(className, elementName, propertyName, whatsThis,
        isList, isClass, isEnum){
00476          _getter= getter;
00477          _setter = setter;
00478          _readonly = setter == nullptr;
00479          _propertyType = Util::TypeOf<unsigned int>();
00480      }
00481  public:
00482      virtual std::string getValue() const override { return std::to_string(_getter()); }
00483      virtual void setValue(std::string value, bool remove=false) override {
00484          _ensureWritable("set value of");
00485          if (!_setter) {
00486              throw std::logic_error("SimulationControlUInt setter is not defined");
00487          }
00488          _setter(std::stoul(value));
00489      };
00490  private:
00491      GetterUInt _getter;
00492      SetterUInt _setter;
00493  };
00494
00495 // -----
00496
00497 typedef std::function<unsigned short()> GetterUShort;
00498 typedef std::function<void(unsigned short)> SetterUShort;
00499 class SimulationControlUShort: public SimulationControl {
00500  public:
00501      // SimulationControlUShort(GetterUShort getter, std::string className, std::string elementName,
        std::string propertyName, std::string whatsThis="") : SimulationControl(className, elementName,
        propertyName, whatsThis) {
00502          //      SimulationControlUShort(getter, nullptr, className, propertyName, whatsThis);
00503          //  }
00504      SimulationControlUShort(GetterUShort getter, SetterUShort setter, std::string className,
        std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
        isClass=false, bool isEnum=false) : SimulationControl(className, elementName, propertyName, whatsThis,
        isList, isClass, isEnum){
00505          _getter= getter;
00506          _setter = setter;
00507          _readonly = setter == nullptr;
00508          _propertyType = Util::TypeOf<unsigned short>();
00509      }

```



```

00510 public:
00511     virtual std::string getValue() const override { return std::to_string(_getter()); }
00512     virtual void setValue(std::string value, bool remove=false) override {
00513         _ensureWritable("set value of");
00514         if (!_setter) {
00515             throw std::logic_error("SimulationControlUShort setter is not defined");
00516         }
00517         _setter(std::stoul(value));
00518     };
00519 private:
00520     GetterUShort _getter;
00521     SetterUShort _setter;
00522 };
00523
00524
00525 // -----
00526
00527 typedef std::function<int()> GetterInt;
00528 typedef std::function<void(int)> SetterInt;
00529 class SimulationControlInt: public SimulationControl {
00530 public:
00531     // SimulationControlInt(GetterInt getter, std::string className, std::string elementName, std::string
    propertyName, std::string whatsThis="") : SimulationControl(className, elementName, propertyName,
    whatsThis) {
00532     //     SimulationControlInt(getter, nullptr, className, propertyName, whatsThis);
00533     // }
00534     SimulationControlInt(GetterInt getter, SetterInt setter, std::string className, std::string
    elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
    isClass=false, bool isEnum=false) : SimulationControl(className, elementName, propertyName, whatsThis,
    isList, isClass, isEnum){
00535         _getter= getter;
00536         _setter = setter;
00537         _readonly = setter == nullptr;
00538         _propertyType = Util::TypeOf<int>();
00539     }
00540 public:
00541     virtual std::string getValue() const override { return std::to_string(_getter()); }
00542     virtual void setValue(std::string value, bool remove=false) override {
00543         _ensureWritable("set value of");
00544         if (!_setter) {
00545             throw std::logic_error("SimulationControlInt setter is not defined");
00546         }
00547         _setter(std::stoi(value));
00548     };
00549 private:
00550     GetterInt _getter;
00551     SetterInt _setter;
00552 };
00553
00554 // -----
00555
00556 typedef std::function<Util::TimeUnit()> GetterTimeUnit;
00557 typedef std::function<void(Util::TimeUnit)> SetterTimeUnit;
00558 class SimulationControlTimeUnit: public SimulationControl {
00559 public:
00560     // SimulationControlTimeUnit(GetterTimeUnit getter, std::string className, std::string elementName,
    std::string propertyName, std::string whatsThis="") : SimulationControl(className, elementName,
    propertyName, whatsThis) {
00561     //     SimulationControlTimeUnit(getter, nullptr, className, propertyName, whatsThis);
00562     // }
00563     SimulationControlTimeUnit(GetterTimeUnit getter, SetterTimeUnit setter, std::string className,
    std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
    isClass=false, bool isEnum=false) : SimulationControl(className, elementName, propertyName, whatsThis,
    isList, isClass, isEnum){
00564         _getter= getter;
00565         _setter = setter;
00566         _readonly = setter == nullptr;
00567         _propertyType = Util::TypeOf<Util::TimeUnit>();
00568     }
00569 public:
00570     virtual std::string getValue() const override {
00571         int intVal = static_cast<int>(_getter());
00572         return std::to_string(intVal);
00573     }
00574     virtual void setValue(std::string value, bool remove=false) override {
00575         _ensureWritable("set value of");
00576         if (!_setter) {
00577             throw std::logic_error("SimulationControlTimeUnit setter is not defined");
00578         }
00579         int intVal = std::stoul(value);
00580         _setter(static_cast<Util::TimeUnit>(intVal));
00581     };
00582 private:
00583     GetterTimeUnit _getter;
00584     SetterTimeUnit _setter;
00585 };
00586

```

```

00587
00588 template <typename T>
00589 using GetterGeneric = std::function<T()>;
00590 template <typename T>
00591 using SetterGeneric = std::function<void(T)>;
00592 template <typename T>
00593 using AdderGeneric = std::function<void(T)>;
00594 template <typename T>
00595 using RemoverGeneric = std::function<void(T)>;
00596
00597 template <typename T>
00598 class SimulationControlGeneric: public SimulationControl {
00599 public:
00600     SimulationControlGeneric(GetterGeneric<T> getter, SetterGeneric<T> setter, std::string className,
std::string elementName, std::string propertyName, std::string whatsThis="", bool isList=false, bool
isClass=false, bool isEnum=false) : SimulationControl(className, elementName, propertyName, whatsThis,
isList, isClass, isEnum){
00601         _getter= getter;
00602         _setter = setter;
00603         _readonly = setter == nullptr;
00604         _propertyType = Util::TypeOf<T>();
00605     }
00606 public:
00607     virtual std::string getValue() const override {
00608         T tVal = static_cast<T>(_getter());
00609
00610         std::string strVal;
00611         std::ostringstream auxStr;
00612         auxStr << tVal;
00613         strVal = auxStr.str();
00614
00615         return strVal;
00616     }
00617
00618     virtual void setValue(std::string value, bool remove=false) override {
00619         _ensureWritable("set value of");
00620         if (!_setter) {
00621             throw std::logic_error("SimulationControlGeneric setter is not defined");
00622         }
00623         if constexpr (std::is_same_v<T, std::string>) {
00624             _setter(value);
00625         } else {
00626             std::istringstream auxStr(value);
00627             T tVal{};
00628             auxStr >> tVal;
00629             _setter(static_cast<T>(tVal));
00630         }
00631     };
00632
00633 private:
00634     GetterGeneric<T> _getter;
00635     SetterGeneric<T> _setter;
00636 };
00637
00638 // @ToDo: (pequena alteração): remove typename E
00639 template <typename T, typename E>
00640 class SimulationControlGenericEnum: public SimulationControl {
00641 public:
00642     SimulationControlGenericEnum(GetterGeneric<T> getter, SetterGeneric<T> setter, std::string
className, std::string elementName, std::string propertyName, std::string whatsThis="", bool
isList=false, bool isClass=false, bool isEnum=true) : SimulationControl(className, elementName,
propertyName, whatsThis, isList, isClass, isEnum){
00643         static_assert(std::is_enum<T>::value, "SimulationControlGenericEnum requires T to be an
enum");
00644         _getter= getter;
00645         _setter = setter;
00646         _readonly = setter == nullptr;
00647         _propertyType = Util::TypeOf<T>();
00648         _isEnum=true;
00649     }
00650 public:
00651     virtual std::string getValue() const override {
00652         int intVal = static_cast<int>(_getter());
00653         int max_i = static_cast<int>(T::num_elements);
00654         if (intVal < 0 || intVal >= max_i) {
00655             return "";
00656         }
00657         return E::convertEnumToStr(static_cast<T>(intVal));
00658     }
00659
00660     virtual void setValue(std::string value, bool remove=false) override {
00661         _ensureWritable("set value of");
00662         if (!_setter) {
00663             throw std::logic_error("SimulationControlGenericEnum setter is not defined");
00664         }
00665         int intVal = std::stoul(value);
00666         _setter(static_cast<T>(intVal));

```

```

00667     };
00668
00669     virtual List<std::string>* getStrValues() override {
00670         List<std::string>* strOptions = new List<std::string>();
00671         int max_i = static_cast<int>(T::num_elements);
00672         for (int i=0; i<max_i; i++) {
00673             std::string value = E::convertEnumToStr((T)i);
00674             strOptions->insert(value);
00675         }
00676         return strOptions;
00677     }
00678
00679 private:
00680     GetterGeneric<T> _getter;
00681     SetterGeneric<T> _setter;
00682 };
00683
00684 // @ToDo: (pequena alteração): remove typename C
00685 template <typename T, typename M, typename C>
00686 class SimulationControlGenericClass: public SimulationControl {
00687 public:
00688     using Creator = std::function<T(M, const std::string&)>;
00689     SimulationControlGenericClass(M model, GetterGeneric<T> getter, SetterGeneric<T> setter,
std::string className, std::string elementName, std::string propertyName, std::string whatsThis="",
bool isList=false, bool isClass=true, bool isEnum=false, Creator creator=nullptr) :
SimulationControl(className, elementName, propertyName, whatsThis, isList, isClass, isEnum){
00690         static_assert(std::is_pointer<T>::value, "SimulationControlGenericClass requires pointer type
T");
00691         _model = model;
00692         _getter= getter;
00693         _setter = setter;
00694         _creator = creator;
00695         _readonly = setter == nullptr;
00696         _propertyType = Util::TypeOf<C>();
00697     }
00698 public:
00699     virtual std::string getValue() const override {
00700         T tVal = static_cast<T>(_getter());
00701         std::string strVal;
00702
00703         if (tVal != nullptr) {
00704             strVal = tVal->getName();
00705         } else {
00706             strVal = "";
00707         };
00708
00709         return strVal;
00710     }
00711
00712     virtual void setValue(std::string value, bool remove=false) override {
00713         _ensureWritable("set value of");
00714         if (!_setter) {
00715             throw std::logic_error("SimulationControlGenericClass setter is not defined");
00716         }
00717         if (remove) {
00718             _setter(nullptr);
00719             return;
00720         }
00721         bool exists = false;
00722         // value.pop_back();
00723         T newVal;
00724
00725         for (auto modeldata : *_model->getDataManager()->getDataDefinitionList(_propertyType)->list())
00726         {
00727             if (modeldata->getName() == value) {
00728                 exists = true;
00729                 newVal = static_cast<T>(modeldata);
00730                 break;
00731             };
00732         };
00733
00734         if (!exists) {
00735             if (_creator != nullptr) {
00736                 newVal = _creator(_model, value);
00737             } else {
00738                 newVal = new C(_model, value);
00739             }
00740             if (newVal == nullptr) {
00741                 throw std::logic_error("SimulationControlGenericClass creator returned null");
00742             }
00743             _model->getDataManager()->insert(newVal);
00744         }
00745
00746         _setter(newVal);
00747     };
00748     virtual bool isModelDataDefinitionReference() const override { return true; }

```

```

00749     virtual ModelDataDefinition* getReferencedModelDataDefinition() const override {
00750         return static_cast<T>(_getter());
00751     }
00752     // This method marks class references as inline-expandable in the universal contract.
00753     virtual bool supportsInlineExpansion() const override { return true; }
00754     // This method marks class references as selectable from existing ModelDataDefinition instances.
00755     virtual bool supportsExistingObjectSelection() const override { return true; }
00756     // This method marks class references as creatable when writable.
00757     virtual bool supportsObjectCreation() const override { return !_readonly; }
00758     // This method marks class references as non-inline object payloads.
00759     virtual bool isInlineObjectProperty() const override { return false; }
00760     // Property Editor advanced editing experiment, step 1:
00761     // ModelDataDefinition references must report whether the referenced object
00762     // actually exists. Previously the base implementation always returned true,
00763     // so the GUI could not distinguish "empty reference" from "loaded object".
00764     // Revert this override to restore the old always-available reference behavior.
00765     virtual bool hasObjectInstance() const override {
00766         return static_cast<T>(_getter()) != nullptr;
00767     }
00768     // Property Editor advanced editing experiment, step 1:
00769     // Do not auto-create ModelDataDefinition references while expanding the tree.
00770     // Unlike inline helper objects, Queue/Resource/Set references need an explicit
00771     // name or an existing target chosen by the user before creation is safe.
00772     // Revert this override if implicit expansion should create references again.
00773     virtual bool ensureObjectInstance() override {
00774         return hasObjectInstance();
00775     }
00776     // This method performs explicit object creation for class references.
00777     virtual bool createObjectInstance(const std::string& value = "") override {
00778         _ensureWritable("create instance for");
00779         std::string name = value;
00780         if (name.empty()) {
00781             name = getValue();
00782         }
00783         if (name.empty()) {
00784             return false;
00785         }
00786         setValue(name, false);
00787         return hasObjectInstance();
00788     }
00789
00790     virtual List<std::string>* getStrValues() override {
00791         List<std::string>* strOptions = new List<std::string>();
00792         for (auto modeldata : *_model->getDataManager()->getDataDefinitionList(_propertyType)->list())
00793         {
00794             if (modeldata != nullptr) {
00795                 strOptions->insert(modeldata->getName());
00796             }
00797         }
00798         return strOptions;
00799     }
00800     virtual List<SimulationControl*>* getChildSimulationControls(int index=0) override {
00801         T tVal = static_cast<T>(_getter());
00802
00803         if (tVal) {
00804             return tVal->getSimulationControls();
00805         } else {
00806             return nullptr;
00807         }
00808     }
00809
00810 private:
00811     M _model;
00812     GetterGeneric<T> _getter;
00813     SetterGeneric<T> _setter;
00814     Creator _creator;
00815 };
00816
00817 // @ToDo: (pequena alteração): remove typename C
00818 template <typename T, typename M, typename C>
00819 class SimulationControlGenericClassNotDC: public SimulationControl {
00820 public:
00821     using Creator = std::function<T(M)>;
00822     using NamedCreator = std::function<T(M, const std::string&)>;
00823     SimulationControlGenericClassNotDC(M model, GetterGeneric<T> getter, SetterGeneric<T> setter,
std::string className, std::string elementName, std::string propertyName, std::string whatsThis="",
bool isList=false, bool isClass=true, bool isEnum=false, Creator creator=nullptr, NamedCreator
namedCreator=nullptr) : SimulationControl(className, elementName, propertyName, whatsThis, isList,
isClass, isEnum){
00824         static_assert(std::is_pointer<T>::value, "SimulationControlGenericClassNotDC requires pointer
type T");
00825         _model = model;
00826         _getter= getter;
00827         _setter = setter;
00828         _creator = creator;
00829         _namedCreator = namedCreator;

```

```

00830         _readonly = setter == nullptr;
00831         _propertyType = Util::TypeOf<C>();
00832     }
00833 public:
00834     virtual std::string getValue() const override {
00835         T tVal = static_cast<T>(_getter());
00836         std::string strVal;
00837
00838         if (tVal != nullptr) {
00839             strVal = tVal->getName();
00840         } else {
00841             strVal = "";
00842         };
00843
00844         return strVal;
00845     }
00846
00847     virtual void setValue(std::string value, bool remove=false) override {
00848         _ensureWritable("set value of");
00849         if (!_setter) {
00850             throw std::logic_error("SimulationControlGenericClassNotDC setter is not defined");
00851         }
00852         T newVal = _createNewInstance(value);
00853
00854         _setter(newVal);
00855     };
00856
00857     virtual bool hasObjectInstance() const override {
00858         return static_cast<T>(_getter()) != nullptr;
00859     }
00860     // This method marks inline class payloads as expandable in the universal contract.
00861     virtual bool supportsInlineExpansion() const override { return true; }
00862     // This method marks inline class payloads as creatable when writable.
00863     virtual bool supportsObjectCreation() const override { return !_readonly; }
00864     // This method marks inline class payloads as non-reference object payloads.
00865     virtual bool isInlineObjectProperty() const override { return true; }
00866
00867     virtual bool ensureObjectInstance() override {
00868         _ensureWritable("ensure instance of");
00869         if (!_setter) {
00870             throw std::logic_error("SimulationControlGenericClassNotDC setter is not defined");
00871         }
00872
00873         T current = static_cast<T>(_getter());
00874         if (current != nullptr) {
00875             return true;
00876         }
00877
00878         T newVal;
00879         newVal = _createNewInstance("");
00880         _setter(newVal);
00881         return static_cast<T>(_getter()) != nullptr;
00882     }
00883     // This method performs explicit object creation for non-DataManager classes.
00884     virtual bool createObjectInstance(const std::string& value = "") override {
00885         if (value.empty()) {
00886             return ensureObjectInstance();
00887         }
00888         setValue(value, false);
00889         return hasObjectInstance();
00890     }
00891
00892     virtual List<SimulationControl*>* getChildSimulationControls(int index=0) override {
00893         T tVal = static_cast<T>(_getter());
00894
00895         if (tVal) {
00896             return tVal->getSimulationControls();
00897         } else {
00898             return nullptr;
00899         }
00900     }
00901
00902 private:
00903     T _createNewInstance(const std::string& value) const {
00904         if (_namedCreator != nullptr) {
00905             return _namedCreator(_model, value);
00906         }
00907         if (_creator != nullptr) {
00908             return _creator(_model);
00909         }
00910         return new C(_model, value);
00911     }
00912
00913 private:
00914     M _model;
00915     GetterGeneric<T> _getter;
00916     SetterGeneric<T> _setter;

```

```

00917     Creator _creator;
00918     NamedCreator _namedCreator;
00919 };
00920
00921 template <typename T, typename M, typename C>
00922 class SimulationControlGenericList: public SimulationControl {
00923 public:
00924     SimulationControlGenericList(M model, GetterGeneric<List<T>*> getter, AdderGeneric<T> adder,
00925     RemoverGeneric<T> remover, std::string className, std::string elementName, std::string propertyName,
00926     std::string whatsThis="", bool isList=true, bool isClass=false, bool isEnum=false) :
00927     SimulationControl(className, elementName, propertyName, whatsThis, isList, isClass, isEnum){
00928         _model = model;
00929         _getter= getter;
00930         _adder = adder;
00931         _remover = remover;
00932         _readonly = adder == nullptr;
00933         _propertyType = Util::TypeOf<C>();
00934     }
00935 public:
00936     virtual std::string getValue() const override {
00937         List<T>* tVal = static_cast<List<T>*>(_getter());
00938         std::string strVal = std::to_string(tVal->size()) + " Items";
00939
00940         return strVal;
00941     }
00942
00943     virtual void setValue(std::string value, bool remove=false) override {
00944         _ensureWritable(remove ? "remove value from" : "add value to");
00945         T newVal{};
00946         if constexpr (std::is_same_v<T, std::string>) {
00947             newVal = value;
00948         } else {
00949             std::istringstream auxStr(value);
00950             auxStr >> newVal;
00951             newVal = static_cast<T>(newVal);
00952         }
00953
00954         if (remove) {
00955             if (!_remover) {
00956                 throw std::logic_error("SimulationControlGenericList remover is not defined");
00957             }
00958             _remover(newVal);
00959         } else {
00960             if (!_adder) {
00961                 throw std::logic_error("SimulationControlGenericList adder is not defined");
00962             }
00963             bool exists = false;
00964             List<T>* tVal = static_cast<List<T>*>(_getter());
00965             for (auto element : *tVal->list()) {
00966                 std::ostringstream auxElement;
00967                 auxElement << element;
00968                 if (value == auxElement.str()) {
00969                     exists = true;
00970                     break;
00971                 }
00972             }
00973
00974             if (!exists) {
00975                 _adder(newVal);
00976             }
00977         }
00978     };
00979
00980     virtual List<std::string>* getStrValues() override {
00981         List<T>* tVal = static_cast<List<T>*>(_getter());
00982         List<std::string>* strOptions = new List<std::string>();
00983
00984         for (auto element : *tVal->list()) {
00985             std::string strVal;
00986             std::ostringstream auxStr;
00987             auxStr << element;
00988             strVal = auxStr.str();
00989
00990             strOptions->insert(strVal);
00991         }
00992         return strOptions;
00993     }
00994
00995     // This method marks scalar lists as editable through the list editor in the universal contract.
00996     virtual bool supportsListEditor() const override { return true; }
00997
00998 private:
00999     M _model;
01000     GetterGeneric<List<T>*> _getter;
01001     AdderGeneric<T> _adder;
01002     RemoverGeneric<T> _remover;
01003 };
01004
01005

```

```

01001 template <typename T, typename M, typename C>
01002 class SimulationControlGenericListPointer: public SimulationControl {
01003 public:
01004     using Creator = std::function<T(M, const std::string&)>;
01005     using TypedCreator = std::function<T(M)>;
01006     using ElementTypeNamesGetter = std::function<std::vector<std::string>()>;
01007     using CurrentElementTypeGetter = std::function<std::string()>;
01008     using CurrentElementTypeSetter = std::function<bool(const std::string&)>;
01009     using TypedNamedCreator = std::function<T(M, const std::string&, const std::string&)>;
01010     using ListElementDataDefinitionGetter = std::function<ModelDataDefinition*(int)>;
01011     SimulationControlGenericListPointer(M model, GetterGeneric<List<T>*> getter, AdderGeneric<T>
        adder, RemoverGeneric<T> remover, std::string className, std::string elementName, std::string
        propertyName, std::string whatsThis="", bool isList=true, bool isClass=true, bool isEnum=false,
        Creator creator=nullptr, TypedCreator typedCreator=nullptr, ElementTypeNamesGetter
        elementTypeNamesGetter=nullptr, CurrentElementTypeGetter currentElementTypeGetter=nullptr,
        CurrentElementTypeSetter currentElementTypeSetter=nullptr, TypedNamedCreator
        typedNamedCreator=nullptr, ListElementDataDefinitionGetter listElementDataDefinitionGetter=nullptr) :
        SimulationControl(className, elementName, propertyName, whatsThis, isList, isClass, isEnum){
01012         static_assert(std::is_pointer<T>::value, "SimulationControlGenericListPointer requires pointer
        type T");
01013         _model = model;
01014         _getter= getter;
01015         _adder = adder;
01016         _remover = remover;
01017         _creator = creator;
01018         _typedCreator = typedCreator;
01019         _elementTypeNamesGetter = elementTypeNamesGetter;
01020         _currentElementTypeGetter = currentElementTypeGetter;
01021         _currentElementTypeSetter = currentElementTypeSetter;
01022         _typedNamedCreator = typedNamedCreator;
01023         _listElementDataDefinitionGetter = listElementDataDefinitionGetter;
01024         _readonly = adder == nullptr;
01025         _propertyType = Util::TypeOf<C>();
01026     }
01027 public:
01028     virtual std::string getValue() const override {
01029         List<T>* tVal = static_cast<List<T>*>(_getter());
01030         std::string strVal = std::to_string(tVal->size()) + " Items";
01031
01032         return strVal;
01033     }
01034
01035     virtual void setValue(std::string value, bool remove=false) override {
01036         _ensureWritable(remove ? "remove value from" : "add value to");
01037         List<T>* tVal = static_cast<List<T>*>(_getter());
01038         T existingVal = nullptr;
01039         for (auto element : *tVal->list()) {
01040             if (element != nullptr && element->getName() == value) {
01041                 existingVal = element;
01042                 break;
01043             }
01044         }
01045
01046         if (remove) {
01047             if (!_remover) {
01048                 throw std::logic_error("SimulationControlGenericListPointer remover is not
        defined");
01049             }
01050             if (existingVal != nullptr) {
01051                 _remover(existingVal);
01052             }
01053         } else {
01054             if (!_adder) {
01055                 throw std::logic_error("SimulationControlGenericListPointer adder is not
        defined");
01056             }
01057             if (existingVal == nullptr) {
01058                 if (!createNewListElement(value)) {
01059                     throw std::logic_error("SimulationControlGenericListPointer could not create a
        new list element");
01060                 }
01061             }
01062         }
01063     };
01064     // This method marks pointer lists as editable through the list editor in the universal contract.
01065     virtual bool supportsListEditor() const override { return true; }
01066     // This method marks pointer lists as supporting explicit typed element creation when writable.
01067     virtual bool supportsNewListElementCreation() const override { return !_readonly; }
01068     // This method exposes concrete types for polymorphic pointer lists such as Set members.
01069     //
01070     // When no provider is configured, the list keeps the legacy single-type creation behavior.
01071     virtual List<std::string>* getCreatableListElementTypes() override {
01072         if (_elementTypeNamesGetter == nullptr) {
01073             return nullptr;
01074         }
01075         List<std::string>* types = new List<std::string>();
01076         for (const std::string& typeName : _elementTypeNamesGetter()) {

```

```

01077         if (!typeName.empty()) {
01078             types->insert(typeName);
01079         }
01080     }
01081     return types;
01082 }
01083 // This method reports the selected concrete type for typed polymorphic lists.
01084 virtual std::string getCurrentListElementType() const override {
01085     if (_currentElementTypeGetter != nullptr) {
01086         return _currentElementTypeGetter();
01087     }
01088     return "";
01089 }
01090 // This method lets the GUI configure the selected concrete type before creating members.
01091 virtual bool setCurrentListElementType(const std::string& typeName) override {
01092     if (_currentElementTypeSetter == nullptr) {
01093         return false;
01094     }
01095     return _currentElementTypeSetter(typeName);
01096 }
01097 // This method creates and inserts a typed list element using the configured creator or the
// default constructor path.
01098 virtual bool createNewListElement(const std::string& value = "") override {
01099     if (_typedNamedCreator != nullptr) {
01100         return createNewListElementOfType(getCurrentListElementType(), value);
01101     }
01102     _ensureWritable("create list element for");
01103     if (!_adder) {
01104         throw std::logic_error("SimulationControlGenericListPointer adder is not defined");
01105     }
01106     T newVal = _createElement(value);
01107     if (newVal == nullptr) {
01108         return false;
01109     }
01110     _adder(newVal);
01111     return true;
01112 }
01113 // This method creates and inserts a member of a caller-selected concrete type.
01114 //
01115 // It is intentionally part of the control contract instead of GUI code so each kernel owner can
01116 // validate type compatibility, naming and model-registration rules in one place.
01117 virtual bool createNewListElementOfType(const std::string& typeName, const std::string& value =
"" ) override {
01118     _ensureWritable("create typed list element for");
01119     if (!_adder) {
01120         throw std::logic_error("SimulationControlGenericListPointer adder is not defined");
01121     }
01122     if (_typedNamedCreator == nullptr) {
01123         return SimulationControl::createNewListElementOfType(typeName, value);
01124     }
01125     std::string effectiveTypeName = typeName;
01126     if (effectiveTypeName.empty()) {
01127         effectiveTypeName = getCurrentListElementType();
01128     }
01129     if (effectiveTypeName.empty()) {
01130         return false;
01131     }
01132     if (_currentElementTypeSetter != nullptr && !_currentElementTypeSetter(effectiveTypeName)) {
01133         return false;
01134     }
01135     T newVal = _typedNamedCreator(_model, effectiveTypeName, value);
01136     if (newVal == nullptr) {
01137         return false;
01138     }
01139     _adder(newVal);
01140     return true;
01141 }
01142
01143 virtual List<SimulationControl*>* getChildSimulationControls(int index=0) override {
01144     List<T*> tVal = static_cast<List<T*>*>(_getter());
01145     T selectedElement = nullptr;
01146
01147     int current_index = 0;
01148     for (auto element : *tVal->list()) {
01149         if (current_index == index) {
01150             selectedElement = element;
01151         }
01152         current_index++;
01153     }
01154
01155     if (selectedElement) {
01156         if constexpr (requires(T element, M model) { element->ensureSimulationControls(model); })
{
01157             selectedElement->ensureSimulationControls(_model);
01158         }
01159         return selectedElement->getSimulationControls();
01160     } else {

```



```

01161         return nullptr;
01162     }
01163 }
01164
01165 virtual ModelDataDefinition* getListElementModelDataDefinition(int index) const override {
01166     if (_listElementDataDefinitionGetter == nullptr) {
01167         return nullptr;
01168     }
01169     return _listElementDataDefinitionGetter(index);
01170 }
01171
01172 virtual List<std::string>* getStrValues() override {
01173     List<T>* tVal = static_cast<List<T>*>(_getter());
01174     List<std::string>* strOptions = new List<std::string>();
01175
01176     for (auto element : *tVal->list()) {
01177         std::string strVal;
01178         if (element != nullptr) {
01179             strVal = element->getName();
01180         } else {
01181             strVal = "";
01182         };
01183
01184         strOptions->insert(strVal);
01185     }
01186     return strOptions;
01187 }
01188
01189 private:
01190     T _createElement(const std::string& value) const {
01191         if (_typedCreator != nullptr) {
01192             return _typedCreator(_model);
01193         }
01194         if (_creator != nullptr) {
01195             return _creator(_model, value);
01196         }
01197         return new C(_model, value);
01198     }
01199
01200 private:
01201     M _model;
01202     GetterGeneric<List<T>*> _getter;
01203     AdderGeneric<T> _adder;
01204     RemoverGeneric<T> _remover;
01205     Creator _creator;
01206     TypedCreator _typedCreator;
01207     ElementTypeNamesGetter _elementTypeNamesGetter;
01208     CurrentElementTypeGetter _currentElementTypeGetter;
01209     CurrentElementTypeSetter _currentElementTypeSetter;
01210     TypedNamedCreator _typedNamedCreator;
01211     ListElementDataDefinitionGetter _listElementDataDefinitionGetter;
01212 };
01213
01214 //namespace\}

```

11.278 SimulationExperiment.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: SimulationExperiment.h
00009  * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010  *
00011  * Created on 4 de maio de 2021, 15:10
00012  */
00013
00014 #ifndef SIMULATIONEXPERIMENT_H
00015 #define SIMULATIONEXPERIMENT_H
00016
00030 class SimulationExperiment {
00031 public:
00032     SimulationExperiment();
00033     virtual ~SimulationExperiment() = default;
00034 private:
00035
00036 };
00037
00038 #endif /* SIMULATIONEXPERIMENT_H */
00039

```

11.279 SimulationReporter_if.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    SimulationReporter_if.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 8 de Agosto de 2018, 10:56
00012  */
00013
00014 #ifndef SIMULATIONREPORTER_IF_H
00015 #define SIMULATIONREPORTER_IF_H
00016
00017 #include "../util/List.h"
00018 // #include "StatisticsCollector.h"
00019
00026 class SimulationReporter_if {
00027 public:
00028     virtual ~SimulationReporter_if() = default;
00033     virtual void showReplicationStatistics() = 0;
00038     virtual void showSimulationStatistics() = 0;
00043     virtual void showSimulationResponses() = 0;
00048     virtual void showSimulationControls() = 0;
00049     // virtual void setFormat(std::string format)=0;
00050 };
00051
00052 #endif /* SIMULATIONREPORTER_IF_H */

```

11.280 SimulationReporterDefaultImpl1.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    SimulationReporterDefaultImpl1.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 8 de Agosto de 2018, 10:59
00012  */
00013
00014 #ifndef SIMULATIONREPORTERDEFAULTIMPL1_H
00015 #define SIMULATIONREPORTERDEFAULTIMPL1_H
00016
00017 #include "SimulationReporter_if.h"
00018 #include "model/ModelSimulation.h"
00019 #include "model/Model.h"
00020
00021 // namespace GenesysKernel {
00022
00026 class SimulationReporterDefaultImpl1 : public SimulationReporter_if {
00027 public:
00028     SimulationReporterDefaultImpl1(ModelSimulation* simulation, Model* model,
00029     List<ModelDataDefinition*>* statsCountersSimulation);
00029     virtual ~SimulationReporterDefaultImpl1() = default;
00030 public:
00031     virtual void showReplicationStatistics() override;
00032     virtual void showSimulationStatistics() override;
00033     virtual void showSimulationResponses() override;
00034     virtual void showSimulationControls() override;
00035 private:
00036     ModelSimulation* _simulation;
00037     Model* _model;
00038 private:
00039     List<ModelDataDefinition*>* _statsCountersSimulation;
00040 private:
00041     const unsigned short _w = 9;
00042     const unsigned short _nameW = 36;
00043 };
00044 // namespace\\}
00045 #endif /* SIMULATIONREPORTERDEFAULTIMPL1_H */
00046

```

11.281 SimulationScenario.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    SimulationScenario.h
00009  * Author:  Prof. Rafael Luiz Cancian, Dr. Eng.
00010  *
00011  * Created on 10 de Outubro de 2018, 18:21
00012  */
00013
00014 #ifndef SIMULATIONSCENARIO_H
00015 #define SIMULATIONSCENARIO_H
00016
00017 #include <string>
00018 #include <list>
00019
00020 class Simulator;
00021
00022 class SimulationScenario {
00023 public:
00024     SimulationScenario();
00025     virtual ~SimulationScenario();
00026 public: // results
00027     bool startSimulation(Simulator *sim, std::string& errorMessage);
00028     std::list<std::pair<std::string, double>*>* getResponseValues() const;
00029     std::list<std::pair<std::string, double>*>* getControlValues() const;
00030     double getResponseValue(const std::string& responseName);
00031 public: // gets and sets
00032     void setModelFilename(std::string _modelFilename);
00033     std::string getModelFilename() const;
00034     void setScenarioName(std::string _name);
00035     std::string getScenarioName() const;
00036     void setScenarioDescription(std::string _scenarioDescription);
00037     std::string getScenarioDescription() const;
00038     std::list<std::string*>* getSelectedControls() const; // access to the list to insert or remove
00039     controls
00040     double getControlValue(const std::string& controlName);
00041     std::list<std::string*>* getSelectedResponses() const; // access to the list to insert or remove
00042     responses
00043     // @ToDo: (importante): Clarify ownership and replacement semantics for selected controls.
00044     // This setter currently allocates a new list and replaces the stored pointer.
00045     void setSelectedControls(std::list<std::string*>* selectedControls);
00046     void setControl(std::string name, double value) const;
00047 private:
00048     std::string _scenarioName;
00049     std::string _scenarioDescription;
00050     std::string _modelFilename;
00051     std::list<std::string*>* _selectedControls = new std::list<std::string*>();
00052     std::list<std::string*>* _selectedResponses = new std::list<std::string*>();
00053     std::list<std::pair<std::string, double>*>* _controlValues = new std::list<std::pair<std::string,
00054     double>*>();
00055     // @ToDo: (importante): Define initialization and ownership rules for response values.
00056     std::list<std::pair<std::string, double>*>* _responseValues = new std::list<std::pair<std::string,
00057     double>*>();
00058 };
00059
00060 #endif /* SIMULATIONSCENARIO_H */

```

11.282 Simulator.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Genesys.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 21 de Junho de 2018, 12:48
00012  */
00013
00014 #ifndef GENESYS_H
00015 #define GENESYS_H
00016
00017 #include <string>
00018 #include <iostream>

```

```

00019 #include "model/Model.h"
00020 #include "Plugin.h"
00021 #include "../util/List.h"
00022 #include "LicenceManager.h"
00023 #include "PluginManager.h"
00024 #include "model/ModelManager.h"
00025 // #include "ToolManager.h"
00026 #include "ParserManager.h"
00027 #include "ExperimentManager.h"
00028
00029 // namespace GenesysKernel {
00030
00031 class PluginManager;
00032
00033 /* https://www.doxygen.nl/manual/commands.html
00034  * Brief description \class \implements
00035  *
00036  * Detailed description
00037  * \param \return
00038  */
00039
00040 class Simulator {
00041     typedef void (*eventHandler)();
00042 public:
00043     Simulator();
00044     virtual ~Simulator();
00045 public: // only get
00046     std::string getVersion() const;
00047     unsigned int getVersionNumber() const;
00048     std::string getName() const;
00049     LicenceManager* getLicenceManager() const;
00050     PluginManager* getPluginManager() const;
00051     ModelManager* getModelManager() const;
00052     TraceManager* getTraceManager() const;
00053     ParserManager* getParserManager() const;
00054     ExperimentManager* getExperimentManager() const;
00055 private:
00056     List<Plugin*> _completePluginsFieldsAndTemplate();
00057     friend class PluginManager; // @ToDo: (pequena alteração): should be only member function
00058     PluginManager::completePluginsFieldsAndTemplate()
00059 private: // attributes 1:1 objects
00060     LicenceManager* _licenceManager;
00061     PluginManager* _pluginManager;
00062     ModelManager* _modelManager;
00063     TraceManager* _traceManager;
00064     ParserManager* _parserManager;
00065     ExperimentManager* _experimentManager;
00066 private: // attributes 1:1 native
00067     const std::string _name = "GenESyS - GENeric and Expansible System Simulator";
00068     const std::string _versionName = "anaiera";
00069     const unsigned int _versionNumber = 260705;
00070 };
00071 // namespace\}
00072
00073 // passing a "C" class factory function which instantiates the class.
00074 typedef /*GenesysKernel::**/ Simulator* GenesysSimulator;
00075 #endif /* GENESYS_H */

```

11.283 SimulatorFacade.h

```

00001 /*
00002  * SimulatorFacade.h
00003  *
00004  * High-level non-owning facade over Simulator and its main composed services.
00005  *
00006  * This header delegates calls to Simulator, its managers, the current Model and
00007  * the current Model sub-services without forcing callers to navigate manually
00008  * through getPluginManager(), getModelManager(), getTraceManager(), etc.
00009  *
00010  * All methods tolerate a null _simulator or missing current model:
00011  * - pointer returns -> nullptr
00012  * - bool returns -> false
00013  * - numeric returns -> 0
00014  * - string returns -> ""
00015  * - void methods -> no-op
00016  *
00017  * Method prefixes resolve name conflicts between delegates:
00018  * (none) Simulator direct methods
00019  * licence* LicenceManager
00020  * plugin* PluginManager

```

```

00021 *   model*           ModelManager / Model
00022 *   info*            ModelInfo
00023 *   sim*             ModelSimulation
00024 *   data*            ModelDataManager
00025 *   component*       ModelComponentManager
00026 *   event*           OnEventManager
00027 *   trace*, setTrace* TraceManager
00028 *   parser*          ParserManager
00029 *   simulationExp*   ExperimentManager
00030 */
00031
00032 #ifndef SIMULATORFACADE_H
00033 #define SIMULATORFACADE_H
00034
00035 #include "Simulator.h"
00036 #include "LicenceManager.h"
00037 #include "PluginManager.h"
00038 #include "TraceManager.h"
00039 #include "ParserManager.h"
00040 #include "ParserChangesInformation.h"
00041 #include "ExperimentManager.h"
00042 #include "SimulationExperiment.h"
00043 #include "OnEventManager.h"
00044 #include "model/ModelManager.h"
00045 #include "model/Model.h"
00046 #include "model/ModelInfo.h"
00047 #include "model/ModelSimulation.h"
00048 #include "model/ModelDataManager.h"
00049 #include "model/ModelComponentManager.h"
00050 #include "model/ModelComponent.h"
00051 #include "model/ModelDataDefinition.h"
00052
00060 class SimulatorFacade {
00061 public:
00062     explicit SimulatorFacade(Simulator* simulator) : _simulator(simulator) {}
00063     virtual ~SimulatorFacade() = default;
00064
00065     // -----
00066     // Simulator -- direct methods
00067     // -----
00068
00069     std::string getVersion() const {
00070         return _simulator != nullptr ? _simulator->getVersion() : "";
00071     }
00072
00073     unsigned int getVersionNumber() const {
00074         return _simulator != nullptr ? _simulator->getVersionNumber() : 0u;
00075     }
00076
00077     std::string getName() const {
00078         return _simulator != nullptr ? _simulator->getName() : "";
00079     }
00080
00081     // -----
00082     // LicenceManager
00083     // -----
00084
00085     const std::string licenceShow() const {
00086         LicenceManager* m = licenceManager_();
00087         return m != nullptr ? m->showLicence() : "";
00088     }
00089
00090     const std::string licenceShowLimits() const {
00091         LicenceManager* m = licenceManager_();
00092         return m != nullptr ? m->showLimits() : "";
00093     }
00094
00095     const std::string licenceShowActivationCode() const {
00096         LicenceManager* m = licenceManager_();
00097         return m != nullptr ? m->showActivationCode() : "";
00098     }
00099
00100     bool licenceLookforActivationCode() {
00101         LicenceManager* m = licenceManager_();
00102         return m != nullptr ? m->lookforActivationCode() : false;
00103     }
00104
00105     bool licenceInsertActivationCode() {
00106         LicenceManager* m = licenceManager_();
00107         return m != nullptr ? m->insertActivationCode() : false;
00108     }
00109
00110     void licenceRemoveActivationCode() {
00111         LicenceManager* m = licenceManager_();
00112         if (m != nullptr) m->removeActivationCode();
00113     }
00114

```

```

00115     unsigned int licenceGetModelComponentsLimit() {
00116         LicenceManager* m = licenceManager_();
00117         return m != nullptr ? m->getModelComponentsLimit() : 0u;
00118     }
00119
00120     unsigned int licenceGetModelDatasLimit() {
00121         LicenceManager* m = licenceManager_();
00122         return m != nullptr ? m->getModelDatasLimit() : 0u;
00123     }
00124
00125     unsigned int licenceGetEntityLimit() {
00126         LicenceManager* m = licenceManager_();
00127         return m != nullptr ? m->getEntityLimit() : 0u;
00128     }
00129
00130     unsigned int licenceGetHostsLimit() {
00131         LicenceManager* m = licenceManager_();
00132         return m != nullptr ? m->getHostsLimit() : 0u;
00133     }
00134
00135     unsigned int licenceGetThreadsLimit() {
00136         LicenceManager* m = licenceManager_();
00137         return m != nullptr ? m->getThreadsLimit() : 0u;
00138     }
00139
00140     // -----
00141     // PluginManager
00142     // -----
00143
00144     std::string showPlugins() {
00145         PluginManager* m = pluginManager_();
00146         return m != nullptr ? m->show() : "";
00147     }
00148
00149     List<Plugin*>* completePluginsFieldsAndTemplates() {
00150         PluginManager* m = pluginManager_();
00151         return m != nullptr ? m->completePluginsFieldsAndTemplates() : nullptr;
00152     }
00153
00154     bool pluginCheck(const std::string& dynamicLibraryFilename) {
00155         PluginManager* m = pluginManager_();
00156         return m != nullptr ? m->check(dynamicLibraryFilename) : false;
00157     }
00158
00159     SystemDependencyCheckResult pluginCheckSystemDependencies(const std::string&
dynamicLibraryFilename) {
00160         PluginManager* m = pluginManager_();
00161         return m != nullptr ? m->checkSystemDependencies(dynamicLibraryFilename) :
SystemDependencyCheckResult{};
00162     }
00163
00164     List<std::string*> discoverPluginFileNames() const {
00165         PluginManager* m = pluginManager_();
00166         return m != nullptr ? m->discoverPluginFileNames() : nullptr;
00167     }
00168
00169     List<PluginLoadIssue*> getPluginLoadIssues() const {
00170         PluginManager* m = pluginManager_();
00171         return m != nullptr ? m->getPluginLoadIssues() : nullptr;
00172     }
00173
00174     void clearPluginLoadIssues() {
00175         PluginManager* m = pluginManager_();
00176         if (m != nullptr) m->clearPluginLoadIssues();
00177     }
00178
00179     void clearPluginLoadIssue(const std::string& dynamicLibraryFilename) {
00180         PluginManager* m = pluginManager_();
00181         if (m != nullptr) m->clearPluginLoadIssue(dynamicLibraryFilename);
00182     }
00183
00184     Plugin* insertPlugin(const std::string& dynamicLibraryFilename) {
00185         PluginManager* m = pluginManager_();
00186         return m != nullptr ? m->insert(dynamicLibraryFilename) : nullptr;
00187     }
00188
00189     Plugin* insertPlugin(const std::string& dynamicLibraryFilename, const PluginInsertionOptions&
options) {
00190         PluginManager* m = pluginManager_();
00191         return m != nullptr ? m->insert(dynamicLibraryFilename, options) : nullptr;
00192     }
00193
00194     bool removePlugin(const std::string& dynamicLibraryFilename) {
00195         PluginManager* m = pluginManager_();
00196         return m != nullptr ? m->remove(dynamicLibraryFilename) : false;
00197     }
00198

```

```

00199     bool removePlugin(Plugin* plugin) {
00200         PluginManager* m = pluginManager_();
00201         return m != nullptr ? m->remove(plugin) : false;
00202     }
00203
00204     Plugin* findPlugin(const std::string& pluginTypeName) {
00205         PluginManager* m = pluginManager_();
00206         return m != nullptr ? m->find(pluginTypeName) : nullptr;
00207     }
00208
00209     std::vector<std::string> getDataDefinitionPluginTypeNames() const {
00210         PluginManager* m = pluginManager_();
00211         return m != nullptr ? m->getDataDefinitionPluginTypeNames() : std::vector<std::string>{};
00212     }
00213
00214     std::string pluginSourceIncludePathFor(const std::string& pluginTypeName) {
00215         PluginManager* m = pluginManager_();
00216         return m != nullptr ? m->sourceIncludePathFor(pluginTypeName) : "";
00217     }
00218
00219     List<Plugin*>* autoInsertPlugins(const std::string& pluginsListFilename, const bool
lookForPluginsIfFilenameNotFound = true) {
00220         PluginManager* m = pluginManager_();
00221         return m != nullptr ? m->autoInsertPlugins(pluginsListFilename,
lookForPluginsIfFilenameNotFound) : nullptr;
00222     }
00223
00224     List<Plugin*>* autoInsertPlugins(const std::string& pluginsListFilename, const bool
lookForPluginsIfFilenameNotFound, const PluginInsertionOptions& options) {
00225         PluginManager* m = pluginManager_();
00226         return m != nullptr ? m->autoInsertPlugins(pluginsListFilename,
lookForPluginsIfFilenameNotFound, options) : nullptr;
00227     }
00228
00229     List<Plugin*>* autoInsertPlugins() {
00230         PluginManager* m = pluginManager_();
00231         return m != nullptr ? m->autoInsertPlugins() : nullptr;
00232     }
00233
00234     List<Plugin*>* autoInsertPlugins(const PluginInsertionOptions& options) {
00235         PluginManager* m = pluginManager_();
00236         return m != nullptr ? m->autoInsertPlugins(options) : nullptr;
00237     }
00238
00239     Plugin* firstPlugin() {
00240         PluginManager* m = pluginManager_();
00241         return m != nullptr ? m->front() : nullptr;
00242     }
00243
00244     Plugin* nextPlugin() {
00245         PluginManager* m = pluginManager_();
00246         return m != nullptr ? m->next() : nullptr;
00247     }
00248
00249     Plugin* lastPlugin() {
00250         PluginManager* m = pluginManager_();
00251         return m != nullptr ? m->last() : nullptr;
00252     }
00253
00254     unsigned int pluginCount() {
00255         PluginManager* m = pluginManager_();
00256         return m != nullptr ? m->size() : 0u;
00257     }
00258
00259     Plugin* getPluginAtRank(unsigned int rank) {
00260         PluginManager* m = pluginManager_();
00261         return m != nullptr ? m->getAtRank(rank) : nullptr;
00262     }
00263
00264     ModelDataDefinition* newInstance(const std::string& pluginTypeName, Model* model, std::string name
= "") {
00265         PluginManager* m = pluginManager_();
00266         return m != nullptr ? m->newInstance(pluginTypeName, model, name) : nullptr;
00267     }
00268
00269     template <typename T>
00270     T* newInstance(Model* model, std::string name = "") {
00271         PluginManager* m = pluginManager_();
00272         return m != nullptr ? m->newInstance<T>(model, name) : nullptr;
00273     }
00274
00275     // -----
00276     // ModelManager
00277     // -----
00278
00279     Model* newModel() {
00280         ModelManager* m = modelManager_();

```

```

00281         return m != nullptr ? m->newModel() : nullptr;
00282     }
00283
00284     void insertModel(Model* model) {
00285         ModelManager* m = modelManager_();
00286         if (m != nullptr) m->insert(model);
00287     }
00288
00289     void removeModel(Model* model) {
00290         ModelManager* m = modelManager_();
00291         if (m != nullptr) m->remove(model);
00292     }
00293
00294     bool setCurrentModel(Model* model) {
00295         ModelManager* m = modelManager_();
00296         return m != nullptr ? m->setCurrent(model) : false;
00297     }
00298
00299     bool saveModel(std::string filename) {
00300         ModelManager* m = modelManager_();
00301         return m != nullptr ? m->saveModel(filename) : false;
00302     }
00303
00304     Model* loadModel(std::string filename) {
00305         ModelManager* m = modelManager_();
00306         return m != nullptr ? m->loadModel(filename) : nullptr;
00307     }
00308
00309     Model* createModelFromLanguage(std::string modelSpecification) {
00310         ModelManager* m = modelManager_();
00311         return m != nullptr ? m->createFromLanguage(modelSpecification) : nullptr;
00312     }
00313
00314     unsigned int modelCount() {
00315         ModelManager* m = modelManager_();
00316         return m != nullptr ? m->size() : 0u;
00317     }
00318
00319     std::vector<Model*> models() const {
00320         ModelManager* m = modelManager_();
00321         return m != nullptr ? m->models() : std::vector<Model*>{};
00322     }
00323
00324     bool hasModel(Model* model) const {
00325         ModelManager* m = modelManager_();
00326         return m != nullptr ? m->hasModel(model) : false;
00327     }
00328
00329     int indexOfModel(Model* model) const {
00330         ModelManager* m = modelManager_();
00331         return m != nullptr ? m->indexOf(model) : -1;
00332     }
00333
00334     Model* modelAt(unsigned int index) const {
00335         ModelManager* m = modelManager_();
00336         return m != nullptr ? m->modelAt(index) : nullptr;
00337     }
00338
00339     int currentModelIndex() const {
00340         ModelManager* m = modelManager_();
00341         return m != nullptr ? m->currentIndex() : -1;
00342     }
00343
00344     Model* firstModel() {
00345         ModelManager* m = modelManager_();
00346         return m != nullptr ? m->front() : nullptr;
00347     }
00348
00349     Model* lastModel() {
00350         ModelManager* m = modelManager_();
00351         return m != nullptr ? m->last() : nullptr;
00352     }
00353
00354     Model* currentModel() {
00355         return currentModel_();
00356     }
00357
00358     Model* nextModel() {
00359         ModelManager* m = modelManager_();
00360         return m != nullptr ? m->next() : nullptr;
00361     }
00362
00363     Model* previousModel() {
00364         ModelManager* m = modelManager_();
00365         return m != nullptr ? m->previous() : nullptr;
00366     }
00367

```



```

00368     bool canGoNextModel() const {
00369         ModelManager* m = modelManager_();
00370         return m != nullptr ? m->canGoNext() : false;
00371     }
00372
00373     bool canGoPreviousModel() const {
00374         ModelManager* m = modelManager_();
00375         return m != nullptr ? m->canGoPrevious() : false;
00376     }
00377
00378     // -----
00379     // TraceManager
00380     // -----
00381
00382     static std::string traceConvertEnumToStr(TraceManager::Level level) {
00383         return TraceManager::convertEnumToStr(level);
00384     }
00385
00386     void addTraceHandler(traceListener traceListener) {
00387         TraceManager* m = traceManager_();
00388         if (m != nullptr) m->addTraceHandler(traceListener);
00389     }
00390
00391     void addTraceReportHandler(traceListener traceReportListener) {
00392         TraceManager* m = traceManager_();
00393         if (m != nullptr) m->addTraceReportHandler(traceReportListener);
00394     }
00395
00396     void addTraceSimulationHandler(traceSimulationListener traceSimulationListener) {
00397         TraceManager* m = traceManager_();
00398         if (m != nullptr) m->addTraceSimulationHandler(traceSimulationListener);
00399     }
00400
00401     void addTraceErrorHandler(traceErrorListener traceErrorListener) {
00402         TraceManager* m = traceManager_();
00403         if (m != nullptr) m->addTraceErrorHandler(traceErrorListener);
00404     }
00405
00406     template<typename Class>
00407     void addTraceHandler(Class* object, void (Class::*function)(TraceEvent)) {
00408         TraceManager* m = traceManager_();
00409         if (m != nullptr) m->addTraceHandler(object, function);
00410     }
00411
00412     template<typename Class>
00413     void addTraceErrorHandler(Class* object, void (Class::*function)(TraceErrorEvent)) {
00414         TraceManager* m = traceManager_();
00415         if (m != nullptr) m->addTraceErrorHandler(object, function);
00416     }
00417
00418     template<typename Class>
00419     void addTraceReportHandler(Class* object, void (Class::*function)(TraceEvent)) {
00420         TraceManager* m = traceManager_();
00421         if (m != nullptr) m->addTraceReportHandler(object, function);
00422     }
00423
00424     template<typename Class>
00425     void addTraceSimulationHandler(Class* object, void (Class::*function)(TraceSimulationEvent)) {
00426         TraceManager* m = traceManager_();
00427         if (m != nullptr) m->addTraceSimulationHandler(object, function);
00428     }
00429
00430     void addTraceSimulationExceptionRuleModelData(void* thisobject) {
00431         TraceManager* m = traceManager_();
00432         if (m != nullptr) m->addTraceSimulationExceptionRuleModelData(thisobject);
00433     }
00434
00435     void beginTraceShutdown() {
00436         TraceManager* m = traceManager_();
00437         if (m != nullptr) m->beginShutdown();
00438     }
00439
00440     void trace(const std::string& text, TraceManager::Level level = TraceManager::Level::L8_detailed)
00441     {
00442         TraceManager* m = traceManager_();
00443         if (m != nullptr) m->trace(text, level);
00444     }
00445
00446     void traceError(const std::string& text, const std::exception& e) {
00447         TraceManager* m = traceManager_();
00448         if (m != nullptr) m->traceError(text, e);
00449     }
00450
00451     void traceError(const std::string& text, TraceManager::Level level =
TraceManager::Level::L1_errorFatal) {
00452         TraceManager* m = traceManager_();
00453         if (m != nullptr) m->traceError(text, level);

```

```

00453     }
00454
00455     void traceReport(const std::string& text, TraceManager::Level level =
TraceManager::Level::L2_results) {
00456         TraceManager* m = traceManager_();
00457         if (m != nullptr) m->traceReport(text, level);
00458     }
00459
00460     void traceSimulation(void* thisobject, double time, Entity* entity, ModelComponent* component,
const std::string& text, TraceManager::Level level = TraceManager::Level::L8_detailed, bool showAnyway
= false) {
00461         TraceManager* m = traceManager_();
00462         if (m != nullptr) m->traceSimulation(thisobject, time, entity, component, text, level,
showAnyway);
00463     }
00464
00465     void traceSimulation(void* thisobject, const std::string& text, TraceManager::Level level =
TraceManager::Level::L8_detailed, bool showAnyway = false) {
00466         TraceManager* m = traceManager_();
00467         if (m != nullptr) m->traceSimulation(thisobject, text, level, showAnyway);
00468     }
00469
00470     List<std::string>* traceGetErrorMessages() const {
00471         TraceManager* m = traceManager_();
00472         return m != nullptr ? m->errorMessages() : nullptr;
00473     }
00474
00475     void setTraceLevel(TraceManager::Level traceLevel) {
00476         TraceManager* m = traceManager_();
00477         if (m != nullptr) m->setTraceLevel(traceLevel);
00478     }
00479
00480     TraceManager::Level getTraceLevel() const {
00481         TraceManager* m = traceManager_();
00482         return m != nullptr ? m->getTraceLevel() : TraceManager::Level::L0_noTraces;
00483     }
00484
00485     void setTraceSimulationRuleAllAllowed(bool traceSimulationRuleAllAllowed) {
00486         TraceManager* m = traceManager_();
00487         if (m != nullptr) m->setTraceSimulationRuleAllAllowed(traceSimulationRuleAllAllowed);
00488     }
00489
00490     bool isTraceSimulationRuleAllAllowed() const {
00491         TraceManager* m = traceManager_();
00492         return m != nullptr ? m->isTraceSimulationRuleAllAllowed() : false;
00493     }
00494
00495     // -----
00496     // ParserManager
00497     // -----
00498
00499     ParserManager::GenerateNewParserResult parserGenerateNewParser(ParserChangesInformation* changes)
{
00500         ParserManager* m = parserManager_();
00501         return m != nullptr ? m->generateNewParser(changes) :
ParserManager::GenerateNewParserResult{};
00502     }
00503
00504     bool parserConnectNewParser(ParserManager::NewParser newParser) {
00505         ParserManager* m = parserManager_();
00506         return m != nullptr ? m->connectNewParser(newParser) : false;
00507     }
00508
00509     // -----
00510     // ExperimentManager
00511     // -----
00512
00513     SimulationExperiment* newSimulationExperiment() {
00514         ExperimentManager* m = experimentManager_();
00515         return m != nullptr ? m->newSimulationExperiment() : nullptr;
00516     }
00517
00518     void insertSimulationExperiment(SimulationExperiment* experiment) {
00519         ExperimentManager* m = experimentManager_();
00520         if (m != nullptr) m->insert(experiment);
00521     }
00522
00523     void removeSimulationExperiment(SimulationExperiment* experiment) {
00524         ExperimentManager* m = experimentManager_();
00525         if (m != nullptr) m->remove(experiment);
00526     }
00527
00528     void setCurrentSimulationExperiment(SimulationExperiment* experiment) {
00529         ExperimentManager* m = experimentManager_();
00530         if (m != nullptr) m->setCurrent(experiment);
00531     }
00532

```

```

00533     bool saveSimulationExperiment(std::string filename) {
00534         ExperimentManager* m = experimentManager_();
00535         return m != nullptr ? m->saveSimulationExperiment(filename) : false;
00536     }
00537
00538     bool loadSimulationExperiment(std::string filename) {
00539         ExperimentManager* m = experimentManager_();
00540         return m != nullptr ? m->loadSimulationExperiment(filename) : false;
00541     }
00542
00543     unsigned int simulationExperimentCount() {
00544         ExperimentManager* m = experimentManager_();
00545         return m != nullptr ? m->size() : 0u;
00546     }
00547
00548     SimulationExperiment* firstSimulationExperiment() {
00549         ExperimentManager* m = experimentManager_();
00550         return m != nullptr ? m->front() : nullptr;
00551     }
00552
00553     SimulationExperiment* currentSimulationExperiment() {
00554         ExperimentManager* m = experimentManager_();
00555         return m != nullptr ? m->current() : nullptr;
00556     }
00557
00558     SimulationExperiment* nextSimulationExperiment() {
00559         ExperimentManager* m = experimentManager_();
00560         return m != nullptr ? m->next() : nullptr;
00561     }
00562
00563     List<SimulationExperiment*>* getSimulationExperiments() const {
00564         ExperimentManager* m = experimentManager_();
00565         return m != nullptr ? m->getExperiments() : nullptr;
00566     }
00567
00568     // -----
00569     // Model -- operations on current model
00570     // -----
00571
00572     bool modelSave(std::string filename) {
00573         Model* model = currentModel_();
00574         return model != nullptr ? model->save(filename) : false;
00575     }
00576
00577     bool modelLoad(std::string filename) {
00578         Model* model = currentModel_();
00579         return model != nullptr ? model->load(filename) : false;
00580     }
00581
00582     bool modelCheck() {
00583         Model* model = currentModel_();
00584         return model != nullptr ? model->check() : false;
00585     }
00586
00587     void modelClear() {
00588         Model* model = currentModel_();
00589         if (model != nullptr) model->clear();
00590     }
00591
00592     std::string modelShowLanguage() {
00593         Model* model = currentModel_();
00594         return model != nullptr ? model->showLanguage() : "";
00595     }
00596
00597     void modelShow() {
00598         Model* model = currentModel_();
00599         if (model != nullptr) model->show();
00600     }
00601
00602     bool modelInsert(ModelDataDefinition* elemOrComp) {
00603         Model* model = currentModel_();
00604         return model != nullptr ? model->insert(elemOrComp) : false;
00605     }
00606
00607     void modelRemove(ModelDataDefinition* elemOrComp) {
00608         Model* model = currentModel_();
00609         if (model != nullptr) model->remove(elemOrComp);
00610     }
00611
00612     std::list<ModelDataDefinition*> modelCollectDataDefinitionsRemovedWith(const
std::list<ModelDataDefinition*>& roots) const {
00613         Model* model = currentModel_();
00614         return model != nullptr ? model->collectDataDefinitionsRemovedWith(roots) :
std::list<ModelDataDefinition*>{};
00615     }
00616
00617     Entity* modelCreateEntity(std::string name, bool insertIntoModel = true) {

```

```

00618         Model* model = currentModel_();
00619         return model != nullptr ? model->createEntity(name, insertIntoModel) : nullptr;
00620     }
00621
00622     void modelRemoveEntity(Entity* entity) {
00623         Model* model = currentModel_();
00624         if (model != nullptr) model->removeEntity(entity);
00625     }
00626
00627     void modelSendEntityToComponent(Entity* entity, Connection* connection, double timeDelay = 0.0) {
00628         Model* model = currentModel_();
00629         if (model != nullptr) model->sendEntityToComponent(entity, connection, timeDelay);
00630     }
00631
00632     void modelSendEntityToComponent(Entity* entity, ModelComponent* component, double timeDelay = 0.0,
unsigned int componentInputPortNumber = 0) {
00633         Model* model = currentModel_();
00634         if (model != nullptr) model->sendEntityToComponent(entity, component, timeDelay,
componentInputPortNumber);
00635     }
00636
00637     double modelParseExpression(const std::string expression) {
00638         Model* model = currentModel_();
00639         return model != nullptr ? model->parseExpression(expression) : 0.0;
00640     }
00641
00642     double modelParseExpression(const std::string expression, bool& success, std::string&
errorMessage) {
00643         Model* model = currentModel_();
00644         if (model != nullptr) return model->parseExpression(expression, success, errorMessage);
00645         success = false; errorMessage = ""; return 0.0;
00646     }
00647
00648     bool modelCheckExpression(const std::string expression, const std::string expressionName,
std::string& errorMessage) {
00649         Model* model = currentModel_();
00650         return model != nullptr ? model->checkExpression(expression, expressionName, errorMessage) :
false;
00651     }
00652
00653     void modelCheckReferencesToDataDefinitions(std::string expression, std::map<std::string,
std::list<std::string>>* referencedDataDefinitions) {
00654         Model* model = currentModel_();
00655         if (model != nullptr) model->checkReferencesToDataDefinitions(expression,
referencedDataDefinitions);
00656     }
00657
00658     Util::identification modelGetId() const {
00659         Model* model = currentModel_();
00660         return model != nullptr ? model->getId() : static_cast<Util::identification>(0);
00661     }
00662
00663     bool modelHasChanged() const {
00664         Model* model = currentModel_();
00665         return model != nullptr ? model->hasChanged() : false;
00666     }
00667
00668     void modelSetHasChanged(bool hasChanged) {
00669         Model* model = currentModel_();
00670         if (model != nullptr) model->setHasChanged(hasChanged);
00671     }
00672
00673     List<Event*>* modelGetFutureEvents() const {
00674         Model* model = currentModel_();
00675         return model != nullptr ? model->getFutureEvents() : nullptr;
00676     }
00677
00678     List<SimulationControl*>* modelGetControls() const {
00679         Model* model = currentModel_();
00680         return model != nullptr ? model->getControls() : nullptr;
00681     }
00682
00683     List<SimulationResponse*>* modelGetResponses() const {
00684         Model* model = currentModel_();
00685         return model != nullptr ? model->getResponses() : nullptr;
00686     }
00687
00688     Persistence_if* modelGetPersistence() const {
00689         Model* model = currentModel_();
00690         return model != nullptr ? model->getPersistence() : nullptr;
00691     }
00692
00693     unsigned int modelGetLevel() const {
00694         Model* model = currentModel_();
00695         return model != nullptr ? model->getLevel() : 0u;
00696     }
00697

```

```

00698 // -----
00699 // ModelInfo -- metadata of current model
00700 // -----
00701
00702 std::string infoShow() {
00703     ModelInfo* info = modelInfo_();
00704     return info != nullptr ? info->show() : "";
00705 }
00706
00707 void infoSetName(std::string name) {
00708     ModelInfo* info = modelInfo_();
00709     if (info != nullptr) info->setName(name);
00710 }
00711
00712 std::string infoGetName() const {
00713     ModelInfo* info = modelInfo_();
00714     return info != nullptr ? info->getName() : "";
00715 }
00716
00717 void infoSetAnalystName(std::string analystName) {
00718     ModelInfo* info = modelInfo_();
00719     if (info != nullptr) info->setAnalystName(analystName);
00720 }
00721
00722 std::string infoGetAnalystName() const {
00723     ModelInfo* info = modelInfo_();
00724     return info != nullptr ? info->getAnalystName() : "";
00725 }
00726
00727 void infoSetDescription(std::string description) {
00728     ModelInfo* info = modelInfo_();
00729     if (info != nullptr) info->setDescription(description);
00730 }
00731
00732 std::string infoGetDescription() const {
00733     ModelInfo* info = modelInfo_();
00734     return info != nullptr ? info->getDescription() : "";
00735 }
00736
00737 void infoSetProjectTitle(std::string projectTitle) {
00738     ModelInfo* info = modelInfo_();
00739     if (info != nullptr) info->setProjectTitle(projectTitle);
00740 }
00741
00742 std::string infoGetProjectTitle() const {
00743     ModelInfo* info = modelInfo_();
00744     return info != nullptr ? info->getProjectTitle() : "";
00745 }
00746
00747 void infoSetVersion(std::string version) {
00748     ModelInfo* info = modelInfo_();
00749     if (info != nullptr) info->setVersion(version);
00750 }
00751
00752 std::string infoGetVersion() const {
00753     ModelInfo* info = modelInfo_();
00754     return info != nullptr ? info->getVersion() : "";
00755 }
00756
00757 void infoLoadInstance(PersistenceRecord* fields) {
00758     ModelInfo* info = modelInfo_();
00759     if (info != nullptr) info->loadInstance(fields);
00760 }
00761
00762 void infoSaveInstance(PersistenceRecord* fields) {
00763     ModelInfo* info = modelInfo_();
00764     if (info != nullptr) info->saveInstance(fields);
00765 }
00766
00767 bool infoHasChanged() const {
00768     ModelInfo* info = modelInfo_();
00769     return info != nullptr ? info->hasChanged() : false;
00770 }
00771
00772 void infoSetHasChanged(bool hasChanged) {
00773     ModelInfo* info = modelInfo_();
00774     if (info != nullptr) info->setHasChanged(hasChanged);
00775 }
00776
00777 // -----
00778 // ModelSimulation -- simulation control on current model
00779 // -----
00780
00781 std::string simShow() {
00782     ModelSimulation* s = modelSimulation_();
00783     return s != nullptr ? s->show() : "";
00784 }

```

```

00785
00786 void simStart() {
00787     ModelSimulation* s = modelSimulation_();
00788     if (s != nullptr) s->start();
00789 }
00790
00791 void simPause() {
00792     ModelSimulation* s = modelSimulation_();
00793     if (s != nullptr) s->pause();
00794 }
00795
00796 void simStep() {
00797     ModelSimulation* s = modelSimulation_();
00798     if (s != nullptr) s->step();
00799 }
00800
00801 void simStop() {
00802     ModelSimulation* s = modelSimulation_();
00803     if (s != nullptr) s->stop();
00804 }
00805
00806 void simSetNumberOfReplications(unsigned int numberOfReplications) {
00807     ModelSimulation* s = modelSimulation_();
00808     if (s != nullptr) s->setNumberOfReplications(numberOfReplications);
00809 }
00810
00811 unsigned int simGetNumberOfReplications() const {
00812     ModelSimulation* s = modelSimulation_();
00813     return s != nullptr ? s->getNumberOfReplications() : 0u;
00814 }
00815
00816 void simSetReplicationLength(double replicationLength, Util::TimeUnit replicationLengthTimeUnit =
Util::TimeUnit::unknown) {
00817     ModelSimulation* s = modelSimulation_();
00818     if (s != nullptr) s->setReplicationLength(replicationLength, replicationLengthTimeUnit);
00819 }
00820
00821 double simGetReplicationLength() const {
00822     ModelSimulation* s = modelSimulation_();
00823     return s != nullptr ? s->getReplicationLength() : 0.0;
00824 }
00825
00826 void simSetReplicationLengthTimeUnit(Util::TimeUnit replicationLengthTimeUnit) {
00827     ModelSimulation* s = modelSimulation_();
00828     if (s != nullptr) s->setReplicationLengthTimeUnit(replicationLengthTimeUnit);
00829 }
00830
00831 Util::TimeUnit simGetReplicationLengthTimeUnit() const {
00832     ModelSimulation* s = modelSimulation_();
00833     return s != nullptr ? s->getReplicationLengthTimeUnit() : Util::TimeUnit::unknown;
00834 }
00835
00836 void simSetReplicationReportBaseTimeUnit(Util::TimeUnit replicationReportBaseTimeUnit) {
00837     ModelSimulation* s = modelSimulation_();
00838     if (s != nullptr) s->setReplicationReportBaseTimeUnit(replicationReportBaseTimeUnit);
00839 }
00840
00841 Util::TimeUnit simGetReplicationBaseTimeUnit() const {
00842     ModelSimulation* s = modelSimulation_();
00843     return s != nullptr ? s->getReplicationBaseTimeUnit() : Util::TimeUnit::unknown;
00844 }
00845
00846 void simSetWarmUpPeriod(double warmUpPeriod, Util::TimeUnit warmUpPeriodTimeUnit =
Util::TimeUnit::unknown) {
00847     ModelSimulation* s = modelSimulation_();
00848     if (s != nullptr) s->setWarmUpPeriod(warmUpPeriod, warmUpPeriodTimeUnit);
00849 }
00850
00851 double simGetWarmUpPeriod() const {
00852     ModelSimulation* s = modelSimulation_();
00853     return s != nullptr ? s->getWarmUpPeriod() : 0.0;
00854 }
00855
00856 void simSetWarmUpPeriodTimeUnit(Util::TimeUnit warmUpPeriodTimeUnit) {
00857     ModelSimulation* s = modelSimulation_();
00858     if (s != nullptr) s->setWarmUpPeriodTimeUnit(warmUpPeriodTimeUnit);
00859 }
00860
00861 Util::TimeUnit simGetWarmUpPeriodTimeUnit() const {
00862     ModelSimulation* s = modelSimulation_();
00863     return s != nullptr ? s->getWarmUpPeriodTimeUnit() : Util::TimeUnit::unknown;
00864 }
00865
00866 void simSetTerminatingCondition(std::string terminatingCondition) {
00867     ModelSimulation* s = modelSimulation_();
00868     if (s != nullptr) s->setTerminatingCondition(terminatingCondition);
00869 }

```

```

00870
00871     std::string simGetTerminatingCondition() const {
00872         ModelSimulation* s = modelSimulation_();
00873         return s != nullptr ? s->getTerminatingCondition() : "";
00874     }
00875
00876     void simSetPauseOnEvent(bool pauseOnEvent) {
00877         ModelSimulation* s = modelSimulation_();
00878         if (s != nullptr) s->setPauseOnEvent(pauseOnEvent);
00879     }
00880
00881     bool simIsPauseOnEvent() const {
00882         ModelSimulation* s = modelSimulation_();
00883         return s != nullptr ? s->isPauseOnEvent() : false;
00884     }
00885
00886     void simSetStepByStep(bool stepByStep) {
00887         ModelSimulation* s = modelSimulation_();
00888         if (s != nullptr) s->setStepByStep(stepByStep);
00889     }
00890
00891     bool simIsStepByStep() const {
00892         ModelSimulation* s = modelSimulation_();
00893         return s != nullptr ? s->isStepByStep() : false;
00894     }
00895
00896     void simSetInitializeStatistics(bool initializeStatistics) {
00897         ModelSimulation* s = modelSimulation_();
00898         if (s != nullptr) s->setInitializeStatistics(initializeStatistics);
00899     }
00900
00901     bool simIsInitializeStatistics() const {
00902         ModelSimulation* s = modelSimulation_();
00903         return s != nullptr ? s->isInitializeStatistics() : false;
00904     }
00905
00906     void simSetInitializeSystem(bool initializeSystem) {
00907         ModelSimulation* s = modelSimulation_();
00908         if (s != nullptr) s->setInitializeSystem(initializeSystem);
00909     }
00910
00911     bool simIsInitializeSystem() const {
00912         ModelSimulation* s = modelSimulation_();
00913         return s != nullptr ? s->isInitializeSystem() : false;
00914     }
00915
00916     void simSetPauseOnReplication(bool pauseBetweenReplications) {
00917         ModelSimulation* s = modelSimulation_();
00918         if (s != nullptr) s->setPauseOnReplication(pauseBetweenReplications);
00919     }
00920
00921     bool simIsPauseOnReplication() const {
00922         ModelSimulation* s = modelSimulation_();
00923         return s != nullptr ? s->isPauseOnReplication() : false;
00924     }
00925
00926     void simSetReporter(SimulationReporter_if* simulationReporter) {
00927         ModelSimulation* s = modelSimulation_();
00928         if (s != nullptr) s->setReporter(simulationReporter);
00929     }
00930
00931     SimulationReporter_if* simGetReporter() const {
00932         ModelSimulation* s = modelSimulation_();
00933         return s != nullptr ? s->getReporter() : nullptr;
00934     }
00935
00936     double simGetSimulatedTime() const {
00937         ModelSimulation* s = modelSimulation_();
00938         return s != nullptr ? s->getSimulatedTime() : 0.0;
00939     }
00940
00941     bool simIsRunning() const {
00942         ModelSimulation* s = modelSimulation_();
00943         return s != nullptr ? s->isRunning() : false;
00944     }
00945
00946     bool simIsPaused() const {
00947         ModelSimulation* s = modelSimulation_();
00948         return s != nullptr ? s->isPaused() : false;
00949     }
00950
00951     unsigned int simGetCurrentReplicationNumber() const {
00952         ModelSimulation* s = modelSimulation_();
00953         return s != nullptr ? s->getCurrentReplicationNumber() : 0u;
00954     }
00955
00956     void simSetShowReportsAfterReplication(bool showReportsAfterReplication) {

```

```

00957         ModelSimulation* s = modelSimulation_();
00958         if (s != nullptr) s->setShowReportsAfterReplication(showReportsAfterReplication);
00959     }
00960
00961     bool simIsShowReportsAfterReplication() const {
00962         ModelSimulation* s = modelSimulation_();
00963         return s != nullptr ? s->isShowReportsAfterReplication() : false;
00964     }
00965
00966     void simSetShowReportsAfterSimulation(bool showReportsAfterSimulation) {
00967         ModelSimulation* s = modelSimulation_();
00968         if (s != nullptr) s->setShowReportsAfterSimulation(showReportsAfterSimulation);
00969     }
00970
00971     bool simIsShowReportsAfterSimulation() const {
00972         ModelSimulation* s = modelSimulation_();
00973         return s != nullptr ? s->isShowReportsAfterSimulation() : false;
00974     }
00975
00976     List<double>* simGetBreakpointsOnTime() const {
00977         ModelSimulation* s = modelSimulation_();
00978         return s != nullptr ? s->getBreakpointsOnTime() : nullptr;
00979     }
00980
00981     List<Entity>* simGetBreakpointsOnEntity() const {
00982         ModelSimulation* s = modelSimulation_();
00983         return s != nullptr ? s->getBreakpointsOnEntity() : nullptr;
00984     }
00985
00986     List<ModelComponent>* simGetBreakpointsOnComponent() const {
00987         ModelSimulation* s = modelSimulation_();
00988         return s != nullptr ? s->getBreakpointsOnComponent() : nullptr;
00989     }
00990
00991     const List<ModelDataDefinition>* simGetSimulationStatisticsAggregates() const {
00992         ModelSimulation* s = modelSimulation_();
00993         return s != nullptr ? s->getSimulationStatisticsAggregates() : nullptr;
00994     }
00995
00996     void simLoadInstance(PersistenceRecord* fields) {
00997         ModelSimulation* s = modelSimulation_();
00998         if (s != nullptr) s->loadInstance(fields);
00999     }
01000
01001     void simSaveInstance(PersistenceRecord* fields, bool saveDefaults) {
01002         ModelSimulation* s = modelSimulation_();
01003         if (s != nullptr) s->saveInstance(fields, saveDefaults);
01004     }
01005
01006     Event* simGetCurrentEvent() const {
01007         ModelSimulation* s = modelSimulation_();
01008         return s != nullptr ? s->getCurrentEvent() : nullptr;
01009     }
01010
01011     void simSetShowSimulationResponsesInReport(bool showSimulationResponsesInReport) {
01012         ModelSimulation* s = modelSimulation_();
01013         if (s != nullptr) s->setShowSimulationResponsesInReport(showSimulationResponsesInReport);
01014     }
01015
01016     bool simIsShowSimulationResponsesInReport() const {
01017         ModelSimulation* s = modelSimulation_();
01018         return s != nullptr ? s->isShowSimulationResponsesInReport() : false;
01019     }
01020
01021     void simSetShowSimulationControlsInReport(bool showSimulationControlsInReport) {
01022         ModelSimulation* s = modelSimulation_();
01023         if (s != nullptr) s->setShowSimulationControlsInReport(showSimulationControlsInReport);
01024     }
01025
01026     bool simIsShowSimulationControlsInReport() const {
01027         ModelSimulation* s = modelSimulation_();
01028         return s != nullptr ? s->isShowSimulationControlsInReport() : false;
01029     }
01030
01031     // -----
01032     // ModelDataManager -- data definitions in current model
01033     // -----
01034
01035     bool dataInsert(ModelDataDefinition* anElement) {
01036         ModelDataManager* m = modelDataManager_();
01037         return m != nullptr ? m->insert(anElement) : false;
01038     }
01039
01040     void dataRemove(ModelDataDefinition* anElement) {
01041         ModelDataManager* m = modelDataManager_();
01042         if (m != nullptr) m->remove(anElement);
01043     }

```



```

01044
01045     bool dataInsert(std::string datadefinitionTypename, ModelDataDefinition* anElement) {
01046         ModelDataManager* m = modelDataManager_();
01047         return m != nullptr ? m->insert(datadefinitionTypename, anElement) : false;
01048     }
01049
01050     void dataRemove(std::string datadefinitionTypename, ModelDataDefinition* anElement) {
01051         ModelDataManager* m = modelDataManager_();
01052         if (m != nullptr) m->remove(datadefinitionTypename, anElement);
01053     }
01054
01055     bool dataCheck(std::string datadefinitionTypename, ModelDataDefinition* anElement, std::string
expressionName, std::string& errorMessage) {
01056         ModelDataManager* m = modelDataManager_();
01057         return m != nullptr ? m->check(datadefinitionTypename, anElement, expressionName,
errorMessage) : false;
01058     }
01059
01060     bool dataCheck(std::string datadefinitionTypename, std::string elementName, std::string
expressionName, bool mandatory, std::string& errorMessage) {
01061         ModelDataManager* m = modelDataManager_();
01062         return m != nullptr ? m->check(datadefinitionTypename, elementName, expressionName, mandatory,
errorMessage) : false;
01063     }
01064
01065     void dataClear() {
01066         ModelDataManager* m = modelDataManager_();
01067         if (m != nullptr) m->clear();
01068     }
01069
01070     ModelDataDefinition* dataGetDataDefinition(std::string datadefinitionTypename,
Util::identification id) {
01071         ModelDataManager* m = modelDataManager_();
01072         return m != nullptr ? m->getDataDefinition(datadefinitionTypename, id) : nullptr;
01073     }
01074
01075     ModelDataDefinition* dataGetDataDefinition(std::string datadefinitionTypename, std::string name) {
01076         ModelDataManager* m = modelDataManager_();
01077         return m != nullptr ? m->getDataDefinition(datadefinitionTypename, name) : nullptr;
01078     }
01079
01080     unsigned int dataGetNumberOfDataDefinitions(std::string datadefinitionTypename) {
01081         ModelDataManager* m = modelDataManager_();
01082         return m != nullptr ? m->getNumberOfDataDefinitions(datadefinitionTypename) : 0u;
01083     }
01084
01085     unsigned int dataGetNumberOfDataDefinitions() {
01086         ModelDataManager* m = modelDataManager_();
01087         return m != nullptr ? m->getNumberOfDataDefinitions() : 0u;
01088     }
01089
01090     int dataGetRankOf(std::string datadefinitionTypename, std::string name) {
01091         ModelDataManager* m = modelDataManager_();
01092         return m != nullptr ? m->getRankOf(datadefinitionTypename, name) : -1;
01093     }
01094
01095     std::list<std::string> dataGetDataDefinitionClassnames() const {
01096         ModelDataManager* m = modelDataManager_();
01097         return m != nullptr ? m->getDataDefinitionClassnames() : std::list<std::string>{};
01098     }
01099
01100     List<ModelDataDefinition*> dataGetDataDefinitionList(std::string datadefinitionTypename) const {
01101         ModelDataManager* m = modelDataManager_();
01102         return m != nullptr ? m->getDataDefinitionList(datadefinitionTypename) : nullptr;
01103     }
01104
01105     void dataShow() {
01106         ModelDataManager* m = modelDataManager_();
01107         if (m != nullptr) m->show();
01108     }
01109
01110     bool dataHasChanged() const {
01111         ModelDataManager* m = modelDataManager_();
01112         return m != nullptr ? m->hasChanged() : false;
01113     }
01114
01115     void dataSetHasChanged(bool hasChanged) {
01116         ModelDataManager* m = modelDataManager_();
01117         if (m != nullptr) m->setHasChanged(hasChanged);
01118     }
01119
01120     // -----
01121     // ModelComponentManager -- components in current model
01122     // -----
01123
01124     bool componentInsert(ModelComponent* comp) {
01125         ModelComponentManager* m = modelComponentManager_();

```

```

01126         return m != nullptr ? m->insert(comp) : false;
01127     }
01128
01129     void componentRemove(ModelComponent* comp) {
01130         ModelComponentManager* m = modelComponentManager_();
01131         if (m != nullptr) m->remove(comp);
01132     }
01133
01134     ModelComponent* componentFind(std::string name) {
01135         ModelComponentManager* m = modelComponentManager_();
01136         return m != nullptr ? m->find(name) : nullptr;
01137     }
01138
01139     ModelComponent* componentFind(Util::identification id) {
01140         ModelComponentManager* m = modelComponentManager_();
01141         return m != nullptr ? m->find(id) : nullptr;
01142     }
01143
01144     void componentClear() {
01145         ModelComponentManager* m = modelComponentManager_();
01146         if (m != nullptr) m->clear();
01147     }
01148
01149     unsigned int componentGetNumberOfComponents() {
01150         ModelComponentManager* m = modelComponentManager_();
01151         return m != nullptr ? m->getNumberOfComponents() : 0u;
01152     }
01153
01154     std::list<ModelComponent*>::iterator componentBegin() {
01155         ModelComponentManager* m = modelComponentManager_();
01156         static std::list<ModelComponent*> _empty;
01157         return m != nullptr ? m->begin() : _empty.begin();
01158     }
01159
01160     std::list<ModelComponent*>::iterator componentEnd() {
01161         ModelComponentManager* m = modelComponentManager_();
01162         static std::list<ModelComponent*> _empty;
01163         return m != nullptr ? m->end() : _empty.end();
01164     }
01165
01166     ModelComponent* componentFront() {
01167         ModelComponentManager* m = modelComponentManager_();
01168         return m != nullptr ? m->front() : nullptr;
01169     }
01170
01171     ModelComponent* componentNext() {
01172         ModelComponentManager* m = modelComponentManager_();
01173         return m != nullptr ? m->next() : nullptr;
01174     }
01175
01176     bool componentHasChanged() const {
01177         ModelComponentManager* m = modelComponentManager_();
01178         return m != nullptr ? m->hasChanged() : false;
01179     }
01180
01181     void componentSetHasChanged(bool hasChanged) {
01182         ModelComponentManager* m = modelComponentManager_();
01183         if (m != nullptr) m->setHasChanged(hasChanged);
01184     }
01185
01186     std::list<SourceModelComponent*>* componentGetSourceComponents() {
01187         ModelComponentManager* m = modelComponentManager_();
01188         return m != nullptr ? m->getSourceComponents() : nullptr;
01189     }
01190
01191     std::list<ModelComponent*>* componentGetTransferInComponents() {
01192         ModelComponentManager* m = modelComponentManager_();
01193         return m != nullptr ? m->getTransferInComponents() : nullptr;
01194     }
01195
01196     std::list<ModelComponent*>* componentGetAllComponents() const {
01197         ModelComponentManager* m = modelComponentManager_();
01198         return m != nullptr ? m->getAllComponents() : nullptr;
01199     }
01200
01201     // -----
01202     // OnEventManager -- event handlers on current model
01203     // -----
01204
01205     void eventAddOnModelCheckSucessHandler(modelEventHandler EventHandler) {
01206         OnEventManager* m = onEventManager_();
01207         if (m != nullptr) m->addOnModelCheckSucessHandler(EventHandler);
01208     }
01209
01210     void eventAddOnModelLoadHandler(modelEventHandler EventHandler) {
01211         OnEventManager* m = onEventManager_();
01212         if (m != nullptr) m->addOnModelLoadHandler(EventHandler);

```

```

01213     }
01214
01215     void eventAddOnModelSaveHandler(modelEventHandler EventHandler) {
01216         OnEventManager* m = onEventManager_();
01217         if (m != nullptr) m->addOnModelSaveHandler(EventHandler);
01218     }
01219
01220     void eventAddOnReplicationStartHandler(simulationEventHandler EventHandler) {
01221         OnEventManager* m = onEventManager_();
01222         if (m != nullptr) m->addOnReplicationStartHandler(EventHandler);
01223     }
01224
01225     void eventAddOnReplicationStepHandler(simulationEventHandler EventHandler) {
01226         OnEventManager* m = onEventManager_();
01227         if (m != nullptr) m->addOnReplicationStepHandler(EventHandler);
01228     }
01229
01230     void eventAddOnReplicationEndHandler(simulationEventHandler EventHandler) {
01231         OnEventManager* m = onEventManager_();
01232         if (m != nullptr) m->addOnReplicationEndHandler(EventHandler);
01233     }
01234
01235     void eventAddOnProcessEventHandler(simulationEventHandler EventHandler) {
01236         OnEventManager* m = onEventManager_();
01237         if (m != nullptr) m->addOnProcessEventHandler(EventHandler);
01238     }
01239
01240     void eventAddOnAfterProcessEventHandler(simulationEventHandler EventHandler) {
01241         OnEventManager* m = onEventManager_();
01242         if (m != nullptr) m->addOnAfterProcessEventHandler(EventHandler);
01243     }
01244
01245     void eventAddOnEntityCreateHandler(simulationEventHandler EventHandler) {
01246         OnEventManager* m = onEventManager_();
01247         if (m != nullptr) m->addOnEntityCreateHandler(EventHandler);
01248     }
01249
01250     void eventAddOnEntityMoveHandler(simulationEventHandler EventHandler) {
01251         OnEventManager* m = onEventManager_();
01252         if (m != nullptr) m->addOnEntityMoveHandler(EventHandler);
01253     }
01254
01255     void eventAddOnEntityRemoveHandler(simulationEventHandler EventHandler) {
01256         OnEventManager* m = onEventManager_();
01257         if (m != nullptr) m->addOnEntityRemoveHandler(EventHandler);
01258     }
01259
01260     void eventAddOnSimulationStartHandler(simulationEventHandler EventHandler) {
01261         OnEventManager* m = onEventManager_();
01262         if (m != nullptr) m->addOnSimulationStartHandler(EventHandler);
01263     }
01264
01265     void eventAddOnSimulationPausedHandler(simulationEventHandler EventHandler) {
01266         OnEventManager* m = onEventManager_();
01267         if (m != nullptr) m->addOnSimulationPausedHandler(EventHandler);
01268     }
01269
01270     void eventAddOnSimulationResumeHandler(simulationEventHandler EventHandler) {
01271         OnEventManager* m = onEventManager_();
01272         if (m != nullptr) m->addOnSimulationResumeHandler(EventHandler);
01273     }
01274
01275     void eventAddOnSimulationEndHandler(simulationEventHandler EventHandler) {
01276         OnEventManager* m = onEventManager_();
01277         if (m != nullptr) m->addOnSimulationEndHandler(EventHandler);
01278     }
01279
01280     void eventAddOnBreakpointHandler(simulationEventHandler EventHandler) {
01281         OnEventManager* m = onEventManager_();
01282         if (m != nullptr) m->addOnBreakpointHandler(EventHandler);
01283     }
01284
01285     template<typename Class>
01286     void eventAddOnModelCheckSuccessHandler(Class* object, void (Class::*function)(ModelEvent*)) {
01287         OnEventManager* m = onEventManager_();
01288         if (m != nullptr) m->addOnModelCheckSuccessHandler(object, function);
01289     }
01290
01291     template<typename Class>
01292     void eventAddOnModelLoadHandler(Class* object, void (Class::*function)(ModelEvent*)) {
01293         OnEventManager* m = onEventManager_();
01294         if (m != nullptr) m->addOnModelLoadHandler(object, function);
01295     }
01296
01297     template<typename Class>
01298     void eventAddOnModelSaveHandler(Class* object, void (Class::*function)(ModelEvent*)) {
01299         OnEventManager* m = onEventManager_();

```

```

01300         if (m != nullptr) m->addOnModelSaveHandler(object, function);
01301     }
01302
01303     template<typename Class>
01304     void eventAddOnReplicationStartHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01305         OnEventManager* m = onEventManager_();
01306         if (m != nullptr) m->addOnReplicationStartHandler(object, function);
01307     }
01308
01309     template<typename Class>
01310     void eventAddOnReplicationStepHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01311         OnEventManager* m = onEventManager_();
01312         if (m != nullptr) m->addOnReplicationStepHandler(object, function);
01313     }
01314
01315     template<typename Class>
01316     void eventAddOnReplicationEndHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01317         OnEventManager* m = onEventManager_();
01318         if (m != nullptr) m->addOnReplicationEndHandler(object, function);
01319     }
01320
01321     template<typename Class>
01322     void eventAddOnProcessEventHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01323         OnEventManager* m = onEventManager_();
01324         if (m != nullptr) m->addOnProcessEventHandler(object, function);
01325     }
01326
01327     template<typename Class>
01328     void eventAddOnAfterProcessEventHandler(Class* object, void (Class::*function)(SimulationEvent*))
01329     {
01330         OnEventManager* m = onEventManager_();
01331         if (m != nullptr) m->addOnAfterProcessEventHandler(object, function);
01332     }
01333
01334     template<typename Class>
01335     void eventAddOnEntityCreateHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01336         OnEventManager* m = onEventManager_();
01337         if (m != nullptr) m->addOnEntityCreateHandler(object, function);
01338     }
01339
01340     template<typename Class>
01341     void eventAddOnEntityMoveHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01342         OnEventManager* m = onEventManager_();
01343         if (m != nullptr) m->addOnEntityMoveHandler(object, function);
01344     }
01345
01346     template<typename Class>
01347     void eventAddOnEntityRemoveHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01348         OnEventManager* m = onEventManager_();
01349         if (m != nullptr) m->addOnEntityRemoveHandler(object, function);
01350     }
01351
01352     template<typename Class>
01353     void eventAddOnSimulationStartHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01354         OnEventManager* m = onEventManager_();
01355         if (m != nullptr) m->addOnSimulationStartHandler(object, function);
01356     }
01357
01358     template<typename Class>
01359     void eventAddOnSimulationPausedHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01360         OnEventManager* m = onEventManager_();
01361         if (m != nullptr) m->addOnSimulationPausedHandler(object, function);
01362     }
01363
01364     template<typename Class>
01365     void eventAddOnSimulationResumeHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01366         OnEventManager* m = onEventManager_();
01367         if (m != nullptr) m->addOnSimulationResumeHandler(object, function);
01368     }
01369
01370     template<typename Class>
01371     void eventAddOnSimulationEndHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01372         OnEventManager* m = onEventManager_();
01373         if (m != nullptr) m->addOnSimulationEndHandler(object, function);
01374     }
01375
01376     template<typename Class>
01377     void eventAddOnBreakpointHandler(Class* object, void (Class::*function)(SimulationEvent*)) {
01378         OnEventManager* m = onEventManager_();
01379         if (m != nullptr) m->addOnBreakpointHandler(object, function);
01380     }
01381
01382     void eventNotifyModelCheckSuccessHandlers(ModelEvent* se) {
01383         OnEventManager* m = onEventManager_();
01384         if (m != nullptr) m->NotifyModelCheckSuccessHandlers(se);
01385     }

```

```

01386 void eventNotifyModelLoadHandlers(ModelEvent* se) {
01387     OnEventManager* m = onEventManager_();
01388     if (m != nullptr) m->NotifyModelLoadHandlers(se);
01389 }
01390
01391 void eventNotifyModelSaveHandlers(ModelEvent* se) {
01392     OnEventManager* m = onEventManager_();
01393     if (m != nullptr) m->NotifyModelSaveHandlers(se);
01394 }
01395
01396 void eventNotifyReplicationStartHandlers(SimulationEvent* se) {
01397     OnEventManager* m = onEventManager_();
01398     if (m != nullptr) m->NotifyReplicationStartHandlers(se);
01399 }
01400
01401 void eventNotifyReplicationStepHandlers(SimulationEvent* se) {
01402     OnEventManager* m = onEventManager_();
01403     if (m != nullptr) m->NotifyReplicationStepHandlers(se);
01404 }
01405
01406 void eventNotifyReplicationEndHandlers(SimulationEvent* se) {
01407     OnEventManager* m = onEventManager_();
01408     if (m != nullptr) m->NotifyReplicationEndHandlers(se);
01409 }
01410
01411 void eventNotifyProcessEventHandlers(SimulationEvent* se) {
01412     OnEventManager* m = onEventManager_();
01413     if (m != nullptr) m->NotifyProcessEventHandlers(se);
01414 }
01415
01416 void eventNotifyAfterProcessEventHandlers(SimulationEvent* se) {
01417     OnEventManager* m = onEventManager_();
01418     if (m != nullptr) m->NotifyAfterProcessEventHandlers(se);
01419 }
01420
01421 void eventNotifyEntityCreateHandlers(SimulationEvent* se) {
01422     OnEventManager* m = onEventManager_();
01423     if (m != nullptr) m->NotifyEntityCreateHandlers(se);
01424 }
01425
01426 void eventNotifyEntityMoveHandlers(SimulationEvent* se) {
01427     OnEventManager* m = onEventManager_();
01428     if (m != nullptr) m->NotifyEntityMoveHandlers(se);
01429 }
01430
01431 void eventNotifyEntityRemoveHandlers(SimulationEvent* se) {
01432     OnEventManager* m = onEventManager_();
01433     if (m != nullptr) m->NotifyEntityRemoveHandlers(se);
01434 }
01435
01436 void eventNotifySimulationStartHandlers(SimulationEvent* se) {
01437     OnEventManager* m = onEventManager_();
01438     if (m != nullptr) m->NotifySimulationStartHandlers(se);
01439 }
01440
01441 void eventNotifySimulationPausedHandlers(SimulationEvent* se) {
01442     OnEventManager* m = onEventManager_();
01443     if (m != nullptr) m->NotifySimulationPausedHandlers(se);
01444 }
01445
01446 void eventNotifySimulationResumeHandlers(SimulationEvent* se) {
01447     OnEventManager* m = onEventManager_();
01448     if (m != nullptr) m->NotifySimulationResumeHandlers(se);
01449 }
01450
01451 void eventNotifySimulationEndHandlers(SimulationEvent* se) {
01452     OnEventManager* m = onEventManager_();
01453     if (m != nullptr) m->NotifySimulationEndHandlers(se);
01454 }
01455
01456 void eventNotifyBreakpointHandlers(SimulationEvent* se) {
01457     OnEventManager* m = onEventManager_();
01458     if (m != nullptr) m->NotifyBreakpointHandlers(se);
01459 }
01460
01461 private:
01462     Simulator* _simulator;
01463
01464     LicenceManager* licenceManager_() const {
01465         return _simulator != nullptr ? _simulator->getLicenceManager() : nullptr;
01466     }
01467
01468     PluginManager* pluginManager_() const {
01469         return _simulator != nullptr ? _simulator->getPluginManager() : nullptr;
01470     }
01471
01472     ModelManager* modelManager_() const {

```

```

01473         return _simulator != nullptr ? _simulator->getModelManager() : nullptr;
01474     }
01475
01476     TraceManager* traceManager_() const {
01477         return _simulator != nullptr ? _simulator->getTraceManager() : nullptr;
01478     }
01479
01480     ParserManager* parserManager_() const {
01481         return _simulator != nullptr ? _simulator->getParserManager() : nullptr;
01482     }
01483
01484     ExperimentManager* experimentManager_() const {
01485         return _simulator != nullptr ? _simulator->getExperimentManager() : nullptr;
01486     }
01487
01488     Model* currentModel_() const {
01489         ModelManager* m = modelManager_();
01490         return m != nullptr ? m->current() : nullptr;
01491     }
01492
01493     ModelInfo* modelInfo_() const {
01494         Model* model = currentModel_();
01495         return model != nullptr ? model->getInfos() : nullptr;
01496     }
01497
01498     ModelSimulation* modelSimulation_() const {
01499         Model* model = currentModel_();
01500         return model != nullptr ? model->getSimulation() : nullptr;
01501     }
01502
01503     ModelDataManager* modelDataManager_() const {
01504         Model* model = currentModel_();
01505         return model != nullptr ? model->getDataManager() : nullptr;
01506     }
01507
01508     ModelComponentManager* modelComponentManager_() const {
01509         Model* model = currentModel_();
01510         return model != nullptr ? model->getComponentManager() : nullptr;
01511     }
01512
01513     OnEventManager* onEventManager_() const {
01514         Model* model = currentModel_();
01515         return model != nullptr ? model->getOnEventManager() : nullptr;
01516     }
01517 };
01518
01519 #endif /* SIMULATORFACADE_H */

```

11.284 SinkModelComponent.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: SinkModelComponent.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 14 de Agosto de 2018, 14:29
00012  */
00013
00014 #ifndef SINKMODELCOMPONENT_H
00015 #define SINKMODELCOMPONENT_H
00016
00017 #include "model/ModelComponent.h"
00018
00019 //namespace GenesysKernel {
00020
00021 class SinkModelComponent : public ModelComponent {
00022 public:
00023     SinkModelComponent(Model* model, std::string componentTypename, std::string name = "");
00024     virtual ~SinkModelComponent() = default;
00025 public:
00026 protected:
00027     virtual bool _loadInstance(PersistenceRecord *fields) override;
00028     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00029 protected:
00030     //virtual void _initBetweenReplications();
00031     virtual bool _check(std::string& errorMessage) override;
00032 private:
00033 };
00034
00035 };

```

```

00038 //namespace\\}
00039 #endif /* SINKMODELCOMPONENT_H */
00040

```

11.285 SourceModelComponent.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:      SourceModelComponent.h
00009  * Author:    rafael.luiz.cancian
00010  *
00011  * Created on 21 de Junho de 2018, 19:50
00012  */
00013
00014 #ifndef SOURCEMODELCOMPONENT_H
00015 #define SOURCEMODELCOMPONENT_H
00016
00017 #include "model/ModelComponent.h"
00018 #include <string>
00019 #include <limits>
00020 //namespace GenesysKernel {
00021
00022 class SourceModelComponent : public ModelComponent {
00023 public:
00024     SourceModelComponent(Model* model, std::string componentTypename, std::string name = "");
00025     virtual ~SourceModelComponent() = default;
00026 public: // get & set
00027     void setFirstCreation(double _firstCreation);
00028     double getFirstCreation() const;
00029     void setEntityType(EntityType* _entityType);
00030     void setEntityType(std::string entityTypeName);
00031     EntityType* getEntityType() const;
00032     void setTimeUnit(Util::TimeUnit _timeUnit);
00033     Util::TimeUnit getTimeUnit() const;
00034     void setTimeBetweenCreationsExpression(std::string _timeBetweenCreations, Util::TimeUnit
00035     _timeUnit=Util::TimeUnit::unknown);
00036     std::string getTimeBetweenCreationsExpression() const;
00037     void setMaxCreations(unsigned long _maxCreations);
00038     void setMaxCreations(std::string _maxCreationsExpression);
00039     std::string getMaxCreations() const;
00040     unsigned int getEntitiesCreated() const;
00041     void setEntitiesCreated(unsigned int _entitiesCreated);
00042     void setEntitiesPerCreation(unsigned int _entitiesPerCreation);
00043     unsigned int getEntitiesPerCreation() const;
00044 public:
00045     virtual std::string show() override;
00046
00047 protected:
00048     virtual bool _loadInstance(PersistenceRecord *fields) override;
00049     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00050 protected:
00051     virtual void _initBetweenReplications() override;
00052     virtual bool _check(std::string& errorMessage) override;
00053 protected:
00054     //virtual void _createInternalStatisticReporters();
00055     virtual void _createEditableDataDefinitions();
00056     virtual void _createAttachedAttributes();
00057 protected: // get & set
00058     EntityType* _entityType = nullptr;
00059
00060     const struct DEFAULT_VALUES {
00061     const double firstCreation = 0.0;
00062     const unsigned int entitiesPerCreation = 1;
00063     const std::string maxCreationsExpression = std::to_string(std::numeric_limits<unsigned
00064     int>::max());
00065     const std::string timeBetweenCreationsExpression = "EXPO(1.0)";
00066     const Util::TimeUnit timeBetweenCreationsTimeUnit = Util::TimeUnit::second;
00067     //
00068     const std::string entityTypeName = "entitytype";
00069     } DEFAULT;
00070     double _firstCreation = DEFAULT.firstCreation;
00071     unsigned int _entitiesPerCreation = DEFAULT.entitiesPerCreation;
00072     std::string _maxCreationsExpression = DEFAULT.maxCreationsExpression;
00073     std::string _timeBetweenCreationsExpression = DEFAULT.timeBetweenCreationsExpression;
00074     Util::TimeUnit _timeBetweenCreationsTimeUnit = DEFAULT.timeBetweenCreationsTimeUnit;
00075     unsigned int _entitiesCreatedSoFar = 0;
00076 };
00077 //namespace\\}
00078 #endif /* SOURCEMODELCOMPONENT_H */

```

11.286 SystemDependencyResolver.h

```

00001  /*
00002  * File:    SystemDependencyResolver.h
00003  * Author:  rafael.luiz.cancian
00004  *
00005  * Created on 16 de Abril de 2026
00006  */
00007
00008  #ifndef SYSTEMDEPENDENCYRESOLVER_H
00009  #define SYSTEMDEPENDENCYRESOLVER_H
00010
00011  #include "PluginInformation.h"
00012
00013  #include <list>
00014  #include <string>
00015
00016  struct SystemCommandResult {
00017      bool started = false;
00018      int exitCode = -1;
00019      std::string output = "";
00020      std::string errorMessage = "";
00021
00022      bool succeeded() const;
00023 };
00024
00025 class SystemCommandExecutor_if {
00026 public:
00027     virtual ~SystemCommandExecutor_if() = default;
00028     virtual SystemCommandResult run(const std::string& command) = 0;
00029 };
00030
00031 class SystemShellCommandExecutor : public SystemCommandExecutor_if {
00032 public:
00033     SystemCommandResult run(const std::string& command) override;
00034 };
00035
00036 class SystemDependencyCheckEntry {
00037 public:
00038     enum class Status {
00039         Satisfied,
00040         Missing,
00041         NotVerifiable,
00042         IgnoredDifferentOS
00043     };
00044
00045     SystemDependencyCheckEntry(SystemDependency dependency, Status status, SystemCommandResult
checkResult = {}, std::string message = "");
00046
00047     const SystemDependency& dependency() const;
00048     Status status() const;
00049     const SystemCommandResult& checkResult() const;
00050     const std::string& message() const;
00051     bool blocksInsertion() const;
00052     bool canAttemptInstall() const;
00053     std::string diagnosticText() const;
00054     static std::string statusToString(Status status);
00055 private:
00056     SystemDependency _dependency;
00057     Status _status = Status::NotVerifiable;
00058     SystemCommandResult _checkResult;
00059     std::string _message = "";
00060 };
00061
00062 class SystemDependencyCheckResult {
00063 public:
00064     void add(SystemDependencyCheckEntry entry);
00065     const std::list<SystemDependencyCheckEntry>& entries() const;
00066     bool canInsertPlugin() const;
00067     bool hasBlockingEntries() const;
00068     bool canAttemptInstallForAllMissing() const;
00069     std::string summary() const;
00070     std::string diagnosticText(bool includeSatisfied = false) const;
00071 private:
00072     std::list<SystemDependencyCheckEntry> _entries;
00073 };
00074
00075 class SystemDependencyInstallEntry {
00076 public:
00077     SystemDependencyInstallEntry(SystemDependency dependency, SystemCommandResult installResult);
00078     const SystemDependency& dependency() const;
00079     const SystemCommandResult& installResult() const;
00080     std::string diagnosticText() const;
00081 private:

```



```

00132     SystemDependency _dependency;
00133     SystemCommandResult _installResult;
00134 };
00135
00139 class SystemDependencyInstallResult {
00140 public:
00142     void add(SystemDependencyInstallEntry entry);
00144     const std::list<SystemDependencyInstallEntry>& entries() const;
00146     bool succeeded() const;
00148     std::string summary() const;
00150     std::string diagnosticText() const;
00151
00152 private:
00153     std::list<SystemDependencyInstallEntry> _entries;
00154 };
00155
00163 class SystemDependencyResolver {
00164 public:
00166     static SystemDependency::OS currentOS();
00168     static bool appliesToCurrentOS(const SystemDependency& dependency);
00170     static SystemDependencyCheckResult evaluate(const std::list<SystemDependency>* dependencies,
00171                                               SystemCommandExecutor_if& executor);
00173     static SystemDependencyInstallResult installMissingDependencies(const SystemDependencyCheckResult&
00174                           checkResult,
00175                           SystemCommandExecutor_if&
00176                           executor);
00177 };
00178
00179 #endif /* SYSTEMDEPENDENCYRESOLVER_H */

```

11.287 TraceManager.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:      TraceManager.h
00009  * Author:    rafael.luiz.cancian
00010  *
00011  * Created on 7 de Novembro de 2018, 11:59
00012  */
00013
00014 #ifndef TRACEMANAGER_H
00015 #define TRACEMANAGER_H
00016
00017 #include "../util/List.h"
00018 #include "../util/Util.h"
00019 #include <functional>
00020 //namespace GenesysKernel {
00021 class Model;
00022 class Entity;
00023 class ModelComponent;
00024 class Simulator;
00025
00026 class TraceEvent;
00027 class TraceErrorEvent;
00028 class TraceSimulationEvent;
00029 class TraceSimulationProcess;
00030
00031 // for handlers that are simple functions
00032 typedef void (*traceListener)(TraceEvent);
00033 typedef void (*traceErrorListener)(TraceErrorEvent);
00034 typedef void (*traceSimulationListener)(TraceSimulationEvent);
00035 typedef void (*traceSimulationProcessListener)(TraceSimulationProcess);
00036 // for handlers that are class members (methods)
00037 typedef std::function<void(TraceEvent)> traceListenerMethod;
00038 typedef std::function<void(TraceErrorEvent)> traceErrorListenerMethod;
00039 typedef std::function<void(TraceSimulationEvent)> traceSimulationListenerMethod;
00040 typedef std::function<void(TraceSimulationProcess)> traceSimulationProcessListenerMethod;
00041
00042 class TraceManager {
00043 public:
00044
00045     enum class Level : int {
00046         L0_noTraces = 0,
00047         L1_errorFatal = 1,
00048         L2_results = 2,
00049         L3_errorRecover = 3, // or progress
00050         L4_warning = 4,
00051         L5_event = 5,

```

```

00056         L6_arrival = 6,
00057         L7_internal = 7,
00058         L8_detailed = 8,
00059         L9_mostDetailed = 9,
00060         num_elements = 10
00061     };
00062 public:
00063     static std::string convertEnumToStr(Level level);
00064     TraceManager(Simulator* simulator); //(Model* model);
00065     virtual ~TraceManager();
00066 public: // add trace handlers
00067     // for handlers that are simple functions
00072     void addTraceHandler(traceListener traceListener);
00077     void addTraceReportHandler(traceListener traceReportListener);
00082     void addTraceSimulationHandler(traceSimulationListener traceSimulationListener);
00087     void addTraceErrorHandler(traceErrorListener traceErrorListener);
00088     // for handlers that are class members (methods)
00093     template<typename Class> void addTraceHandler(Class* object, void (Class::*function)(TraceEvent));
00098     template<typename Class> void addTraceErrorHandler(Class* object, void
00099 (Class::*function)(TraceErrorEvent));
00103     template<typename Class> void addTraceReportHandler(Class* object, void
00104 (Class::*function)(TraceEvent));
00108     template<typename Class> void addTraceSimulationHandler(Class* object, void
00109 (Class::*function)(TraceSimulationEvent));
00110 public: // add ModelDataDefinition (or subclasses) allowed (or restricted) to traceSimulation
00111     void addTraceSimulationExceptionRuleModelData(void* thisobject);
00112 public: // traces (invoke trace handlers)
00113     void trace(TraceManager::Level level, const std::string& text);
00114     void traceSimulation(void* thisobject, TraceManager::Level level, double time, Entity* entity,
00115 ModelComponent* component, const std::string& text);
00116     void traceSimulation(void* thisobject, TraceManager::Level level, const std::string& text);
00117 public: // traces
00118     // Disable callbacks and clear handler lists before simulator teardown destroys GUI objects.
00119     void beginShutdown();
00119     void trace(const std::string& text, TraceManager::Level level = TraceManager::Level::L8_detailed);
00120     void traceError(const std::string& text, const std::exception& e);
00121     void traceError(const std::string& text, TraceManager::Level level =
00122 TraceManager::Level::L1_errorFatal);
00122     void traceReport(const std::string& text, TraceManager::Level level =
00123 TraceManager::Level::L2_results);
00123     void traceSimulation(void* thisobject, double time, Entity* entity, ModelComponent* component,
00124 const std::string& text, TraceManager::Level level = TraceManager::Level::L8_detailed, bool showAnyway
00125 = false);
00124     void traceSimulation(void* thisobject, const std::string& text, TraceManager::Level level =
00125 TraceManager::Level::L8_detailed, bool showAnyway = false);
00126 public:
00131     List<std::string>* errorMessages() const;
00136     void setTraceLevel(TraceManager::Level _traceLevel);
00141     TraceManager::Level getTraceLevel() const;
00146     Simulator* getParentSimulator() const;
00151     void setTraceSimulationRuleAllAllowed(bool _traceSimulationRuleAllAllowed);
00156     bool isTraceSimulationRuleAllAllowed() const;
00157 private:
00158     //void _addHandler(List<traceListener>* list, )
00160     bool _traceConditionPassed(TraceManager::Level level);
00161     bool _traceSimulationConditionPassed(TraceManager::Level level, void* thisobject, bool showAnyway
00162 = false);
00162 private: // trace listener
00163     // for handlers that are simple functions
00164     List<traceListener>* _traceHandlers;
00165     List<traceErrorListener>* _traceErrorHandlers;
00166     List<traceListener>* _traceReportHandlers;
00167     List<traceSimulationListener>* _traceSimulationHandlers;
00168     // for handlers that are class members (methods)
00169     List<traceListenerMethod>* _traceHandlersMethod;
00170     List<traceErrorListenerMethod>* _traceErrorHandlersMethod;
00171     List<traceListenerMethod>* _traceReportHandlersMethod;
00172     List<traceSimulationListenerMethod>* _traceSimulationHandlersMethod;
00173 private:
00174     List<void*>* _traceSimulationExceptionRule;
00175 private:
00176     Simulator* _simulator;
00177 private:
00178     TraceManager::Level _traceLevel; // = TraceManager::Level::L9_mostDetailed;
00179     bool _shuttingDown;
00180     double _lastTimeTraceSimulation;
00181     Util::identification _lastEntityTraceSimulation;
00182     Util::identification _lastModuleTraceSimulation;
00183     bool _traceSimulationRuleAllAllowed;
00184     List<std::string>* _errorMessages;
00185 };
00186 };
00187 // implementation for template methods
00188

```

```

00189
00190 template<typename Class> void TraceManager::addTraceHandler(Class* object, void
    (Class::*function)(TraceEvent)) {
00191     this->_traceHandlersMethod->insert(std::bind(function, object, std::placeholders::_1));
00192 }
00193
00194 template<typename Class> void TraceManager::addTraceErrorHandler(Class* object, void
    (Class::*function)(TraceErrorEvent)) {
00195     this->_traceErrorHandlersMethod->insert(std::bind(function, object, std::placeholders::_1));
00196 }
00197
00198 template<typename Class> void TraceManager::addTraceReportHandler(Class* object, void
    (Class::*function)(TraceEvent)) {
00199     this->_traceReportHandlersMethod->insert(std::bind(function, object, std::placeholders::_1));
00200 }
00201
00202 template<typename Class> void TraceManager::addTraceSimulationHandler(Class* object, void
    (Class::*function)(TraceSimulationEvent)) {
00203     this->_traceSimulationHandlersMethod->insert(std::bind(function, object, std::placeholders::_1));
00204 }
00205
00206 class TraceEvent {
00207 public:
00208
00209     TraceEvent(TraceManager::Level level, const std::string& text) {
00210         _tracelevel = level;
00211         _text = text;
00212     }
00213
00214     TraceEvent(const std::string& text, TraceManager::Level level = TraceManager::Level::L8_detailed)
    {
00215         _tracelevel = level;
00216         _text = text;
00217     }
00218
00219     TraceManager::Level getTracelevel() const {
00220         return _tracelevel;
00221     }
00222
00223     const std::string& getText() const {
00224         return _text;
00225     }
00226 private:
00227     TraceManager::Level _tracelevel;
00228     std::string _text;
00229 };
00230
00231 class TraceErrorEvent : public TraceEvent {
00232 public:
00233
00234     TraceErrorEvent(const std::string& text, const std::exception& e) : TraceEvent(text,
    TraceManager::Level::L3_errorRecover), _exceptionMessage(e.what()) {
00235     }
00236
00237     TraceErrorEvent(const std::string& text, TraceManager::Level level) : TraceEvent(text, level),
    _exceptionMessage("") {
00238     }
00239
00240     const std::string& getExceptionMessage() const {
00241         return _exceptionMessage;
00242     }
00243 private:
00244     std::string _exceptionMessage;
00245 };
00246
00247 class TraceSimulationEvent : public TraceEvent {
00248 public:
00249
00250     TraceSimulationEvent(TraceManager::Level level, double time, Entity* entity, ModelComponent*
    component, const std::string& text) : TraceEvent(text, level) {
00251         _time = time;
00252         _entity = entity;
00253         _component = component;
00254     }
00255
00256     ModelComponent* getComponent() const {
00257         return _component;
00258     }
00259
00260     Entity* getEntity() const {
00261         return _entity;
00262     }
00263
00264     double getTime() const {
00265         return _time;
00266     }
00267

```

```

00268 private:
00269     double _time;
00270     Entity* _entity;
00271     ModelComponent* _component;
00272 };
00273
00274 class TraceSimulationProcess : public TraceEvent {
00275 public:
00276     TraceSimulationProcess(const std::string& text, TraceManager::Level level =
00277         TraceManager::Level::L8_detailed) : TraceEvent(text, level) {
00278     }
00279 };
00280
00281 //namespace\\}
00282 #endif /* TRACEMANAGER_H */

```

11.288 Collector_if.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Collector_if.h
00009  * Author: rafael.luiiz.cancian
00010  *
00011  * Created on 14 de Agosto de 2018, 14:16
00012  */
00013
00014 #ifndef COLLECTOR_IF_H
00015 #define COLLECTOR_IF_H
00016
00017 #include <string>
00018 #include <functional>
00019
00020 typedef std::function<void(double, double) > CollectorAddValueHandler;
00021
00022 template<typename Class>
00023 CollectorAddValueHandler setCollectorAddValueHandler(void (Class::*function)(double, double), Class *
00024 object) {
00025     return std::bind(function, object, std::placeholders::_1, std::placeholders::_2);
00026 }
00027
00028 typedef std::function<void() > CollectorClearHandler;
00029
00030 template<typename Class>
00031 CollectorClearHandler setCollectorClearHandler(void (Class::*function)(), Class * object) {
00032     return std::bind(function, object);
00033 }
00034
00035 class Collector_if {
00036 public:
00037     virtual ~Collector_if() = default;
00038     virtual void clear() = 0;
00039     virtual void addValue(double value, double weight=1) = 0;
00040     virtual double getLastValue() = 0;
00041     virtual unsigned long numElements() = 0;
00042 public:
00043     virtual void setAddValueHandler(CollectorAddValueHandler addValueHandler) = 0;
00044     virtual void setClearHandler(CollectorClearHandler clearHandler) = 0;
00045 };
00046
00047 #endif /* COLLECTOR_IF_H */

```

11.289 CollectorDatafile_if.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: CollectorDatafile_if.h
00009  * Author: rafael.luiiz.cancian

```

```

00010  *
00011  * Created on 30 de Agosto de 2018, 16:45
00012  */
00013
00014 #ifndef COLLECTORDATAFILE_IF_H
00015 #define COLLECTORDATAFILE_IF_H
00016
00017 #include "Collector_if.h"
00018
00025 class CollectorDatafile_if : public Collector_if {
00026 public:
00027     virtual ~CollectorDatafile_if() = default;
00033     virtual double getValue(unsigned int rank) = 0;
00037     virtual void seekFirstValue() = 0;
00042     virtual double getNextValue() = 0;
00048     virtual std::string getDataFilename() = 0;
00054     virtual void setDataFilename(std::string filename) = 0;
00055 };
00056
00057 #endif /* COLLECTORDATAFILE_IF_H */

```

11.290 CollectorDatafileDefaultImpl1.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File: CollectorDatafileDefaultImpl1.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 1 de Agosto de 2018, 20:58
00012  */
00013
00014 #ifndef COLLECTORDATAFILEDEFAULTIMPL1_H
00015 #define COLLECTORDATAFILEDEFAULTIMPL1_H
00016
00017 #include <limits>
00018 #include <string>
00019
00020 #include "CollectorDatafile_if.h"
00021
00028 class CollectorDatafileDefaultImpl1 : public CollectorDatafile_if {
00029 public:
00031     CollectorDatafileDefaultImpl1();
00032     virtual ~CollectorDatafileDefaultImpl1() = default;
00033 public: // inherited from Collector_if
00035     virtual void clear() override;
00037     virtual void addValue(double value, double weight=1) override;
00039     virtual double getLastValue() override;
00041     virtual unsigned long numElements() override;
00042 public:
00044     virtual double getValue(unsigned int num) override;
00046     virtual double getNextValue() override;
00048     virtual void seekFirstValue() override;
00050     virtual std::string getDataFilename() override;
00052     virtual void setDataFilename(std::string filename) override;
00053 public:
00055     virtual void setAddValueHandler(CollectorAddValueHandler addValueHandler) override;
00057     virtual void setClearHandler(CollectorClearHandler clearHandler) override;
00058 private:
00059     // Cache file metadata and sequential read position to avoid stale state.
00060     std::string _filename;
00061     double _lastValue = std::numeric_limits<double>::quiet_NaN();
00062     unsigned int _numElements = 0;
00063     unsigned int _nextReadIndex = 0;
00064 };
00065
00066 #endif /* COLLECTORDATAFILEDEFAULTIMPL1_H */

```

11.291 CollectorDefaultImpl1.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */

```

```

00006
00007 /*
00008  * File:    CollectorDefaultImpl1.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 1 de Agosto de 2018, 20:43
00012  */
00013
00014 #ifndef COLLECTORDEFAULTIMPL1_H
00015 #define COLLECTORDEFAULTIMPL1_H
00016
00017 #include "Collector_if.h"
00018 //namespace GenesysKernel {
00019
00026 class CollectorDefaultImpl1 : public Collector_if {
00027 public:
00029     CollectorDefaultImpl1();
00030     virtual ~CollectorDefaultImpl1() = default;
00031 public:
00033     virtual void clear() override;
00035     virtual void addValue(double value, double weight=1) override;
00037     virtual double getLastValue() override;
00039     virtual unsigned long numElements() override;
00040 public:
00042     virtual void setAddValueHandler(CollectorAddValueHandler addValueHandler) override;
00044     virtual void setClearHandler(CollectorClearHandler clearHandler) override;
00045 private:
00046     double _lastValue;
00047     unsigned long _numElements = 0;
00048     CollectorAddValueHandler _addValueHandler = nullptr;
00049     CollectorClearHandler _clearHandler = nullptr;
00050 };
00051 //namespace\\}
00052 #endif /* COLLECTORDEFAULTIMPL1_H */

```

11.292 Sampler_if.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Sampler_if.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 14 de Agosto de 2018, 13:20
00012  */
00013
00014 #ifndef Sampler_IF_H
00015 #define Sampler_IF_H
00016
00024 class Sampler_if {
00025 public:
00026     virtual ~Sampler_if() = default;
00027
00034     struct RNG_Parameters {
00035         virtual ~RNG_Parameters() = default;
00036     };
00037 public: // RNG
00042     virtual double random() = 0;
00043 public: // continuous probability distributions
00052     virtual double sampleBeta(double alpha, double beta, double infLimit, double supLimit) = 0;
00054     virtual double sampleBeta(double alpha, double beta) = 0;
00056     virtual double sampleErlang(double mean, int M, double offset = 0.0) = 0;
00058     virtual double sampleExponential(double mean, double offset = 0.0) = 0;
00059     //virtual double sampleGamma(double mean, double alpha, double offset=0.0) = 0;
00061     virtual double sampleGamma(double alpha, double beta, double offset = 0.0) = 0;
00063     virtual double sampleGumbell(double mode, double scale) = 0;
00065     virtual double sampleLogNormal(double mean, double stddev, double offset = 0.0) = 0;
00067     virtual double sampleNormal(double mean, double stddev) = 0;
00069     virtual double sampleTriangular(double min, double mode, double max) = 0;
00071     virtual double sampleUniform(double min, double max) = 0;
00073     virtual double sampleWeibull(double alpha, double scale) = 0;
00074 public: // discrete probability distributions
00075     // @ToDo: (pequena alteração): Poisson, si vous plait!!!!
00077     virtual double sampleBinomial(int trials, double p) = 0;
00079     virtual double sampleBernoulli(double p) = 0;
00081     virtual double sampleDiscrete(double prob, double value, ...) = 0;
00083     virtual double sampleDiscrete(double *prob, double *value, int size) = 0;
00085     virtual double sampleGeometric(double p) = 0;
00086 public:

```

```

00088     virtual void setRNGparameters(RNG_Parameters* param) = 0;
00090     virtual RNG_Parameters* getRNGparameters() const = 0;
00091 };
00092
00093 #endif /* Sampler_IF_H */

```

11.293 SamplerDefaultImpl1.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    SamplerDefaultImpl1.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 2 de Agosto de 2018, 01:10
00012  */
00013
00014 #ifndef SAMPLERDEFAULTIMPL1_H
00015 #define SAMPLERDEFAULTIMPL1_H
00016
00017 #include "Sampler_if.h"
00018 #include <stdint.h>
00019
00020 //namespace GenesysKernel {
00021
00022 class SamplerDefaultImpl1 : public Sampler_if {
00023 public:
00024
00025     struct DefaultImpl1RNG_Parameters : public RNG_Parameters {
00026     uint32_t seed = 16021974;
00027     uint32_t a = 279470273u; // multiplier
00028     uint32_t m = 0xffffffffb; // module
00029     uint32_t c = 0; // increment
00030     ~DefaultImpl1RNG_Parameters() = default;
00031     };
00032 public:
00033     SamplerDefaultImpl1();
00034     virtual ~SamplerDefaultImpl1();
00035 public: // RNG
00036     virtual double random() override;
00037 public: // continuous probability distributions
00038     virtual double sampleBeta(double alpha, double beta, double infLimit, double supLimit) override;
00039     virtual double sampleBeta(double alpha, double beta) override;
00040     virtual double sampleErlang(double mean, int M, double offset = 0.0) override;
00041     virtual double sampleExponential(double mean, double offset = 0.0) override;
00042     //virtual double sampleGamma(double mean, double alpha, double offset=0.0);
00043     virtual double sampleGamma(double alpha, double beta, double offset = 0.0) override;
00044     virtual double sampleGumbell(double mode, double scale) override;
00045     virtual double sampleLogNormal(double mean, double stddev, double offset = 0.0) override;
00046     virtual double sampleNormal(double mean, double stddev) override;
00047     virtual double sampleTriangular(double min, double mode, double max) override;
00048     virtual double sampleUniform(double min, double max) override;
00049     virtual double sampleWeibull(double alpha, double scale) override;
00050 public: // discrete probability distributions
00051     // @ToDo: (pequena alteração): Poisson, si vous plait!!!!
00052     virtual double sampleBinomial(int trials, double p) override;
00053     virtual double sampleBernoulli(double p) override;
00054     virtual double sampleDiscrete(double prob, double value, ...) override;
00055     virtual double sampleDiscrete(double *prob, double *value, int size) override;
00056     virtual double sampleGeometric(double p) override;
00057 public:
00058     void reset();
00059 public:
00060     virtual void setRNGparameters(RNG_Parameters* param) override;
00061     virtual RNG_Parameters* getRNGparameters() const override;
00062 private:
00063     RNG_Parameters* _param = new DefaultImpl1RNG_Parameters();
00064     bool _ownsParam = true;
00065     uint32_t _xi;
00066     bool _normalflag;
00067     double _lastnormal;
00068 };
00069
00070 /*
00071  (*
00072
00073                                     mrand.ras
00074
00075                                     Random generator for Borland Delphi 3.0
00076

```

```
00107
00108 Copyright(c) 1998 Private enterprise - firm SOFTLAND, Ukraine
00109
00110
00111 1. What is it?
00112
00113     TMultiRand is a component intended for generating random variates of
00114     different distributions:
00115
00116             Uniform
00117             Triangular
00118             Exponential
00119             Erlang
00120             Normal
00121             Gamma
00122             Beta
00123             Weibull
00124             Lognormal
00125
00126 2. Licensing.
00127
00128     a) This software is under Copyright(c) 1998 SOFTLAND,Ukraine
00129     b) This software is for free usage and distribution.
00130     c) The conditions of the distribution are
00131         - remaining the source code, comments and manual without changes
00132         - supplying the manual
00133     e) The user assumes all risks associated with this software.
00134         Softland is not responsible for any loss or damage incurred
00135         during the use of this software.
00136
00137
00138 3. Manual
00139
00140     The manual for this component is in mrand.pdf
00141
00142 4. Where is it used?
00143
00144     This component is used in the discrete-event simulation system Delsi 1.0,
00145     which is implemented as a set of components for Borland Delphi.
00146
00147     Find more info at http://www.softland.rovno.ua/delsi.htm
00148
00149 5. What is Softland ?
00150
00151     Softland is Ukrainian software firm.
00152
00153     Our skills:
00154
00155         - Delhpi, C++
00156         - SQL
00157         - Computer simulation of queuing systems
00158         - Web-design
00159
00160     We are looking for the remote development job in areas of
00161     general client/server applications and computer simulation.
00162     The rate of our service is extremely unexpensive.
00163
00164     Find more info at http://www.softland.rovno.ua
00165
00166 *)
00167
00168 unit Mrand;
00169
00170 interface
00171
00172 uses WinTypes, WinProcs, Classes, Dialogs;
00173
00174 const
00175
00176     MODULUS = 2147483647;
00177
00178 type
00179
00180 TMRand = class(TComponent)
00181 private
00182     FSeed: comp;
00183     FSeedInit: comp;
00184     FMultiplier: comp;
00185     NormalFlag: boolean;
00186     NormalResult: real;
00187     function Uni01: real;
00188     procedure SetSeed(Value: comp);
00189     function GetSeed: comp;
00190     procedure SetMultiplier(Value: comp);
00191     function GetMultiplier: comp;
00192     function GammaJonk(Alpha: real): real;
00193 public
```



```
00194     constructor Create(Owner: TComponent); override;
00195     procedure Reset;
00196     function Uniform(LowerBound: real; UpperBound: real): real;
00197     function Triangular(Min: real; Mode: real; Max: real): real;
00198     function Exponential(Mean: real): real;
00199     function Erlang(Mean: real; M: integer): real;
00200     function Normal(Mean: real; Deviation: real): real;
00201     function Gamma(Mean: real; Alpha: real): real;
00202     function Beta(ShapeAlpha: real; ShapeBeta: real;
00203         LowerBound: real; UpperBound: real): real;
00204     function Weibull(Alpha: real; Scale: real): real;
00205     function Lognormal(Mean: real; Deviation: real): real;
00206 published
00207     property Seed: comp read GetSeed write SetSeed;
00208     property Multiplier: comp read GetMultiplier write SetMultiplier;
00209 end;
00210
00211
00212 procedure Register;
00213
00214 implementation
00215
00216
00217 constructor TMRand.Create(Owner: TComponent);
00218 begin
00219     inherited Create(Owner);
00220     FSeed:=1000000000;
00221     FMultiplier:=950706376;
00222     NormalFlag:=True;
00223 end;
00224
00225 procedure TMRand.Reset;
00226 begin
00227     NormalFlag:=True;
00228     FSeed:=FSeedInit;
00229 end;
00230
00231
00232 procedure TMRand.SetSeed(Value: comp);
00233 begin
00234     if (Value<1) or (Value>2147483646) then
00235     begin
00236         MessageDlg(Name+'Seed must be in the range 2...2147483646',mtError,[mbOK],0);
00237         FSeed:=1000000000;
00238     end
00239     else
00240         FSeed:=Value;
00241     FSeedInit:=FSeed;
00242 end;
00243
00244 function TMRand.GetSeed: comp;
00245 begin
00246     if (FSeed<1) or (FSeed>2147483646) then
00247     begin
00248         FSeed:=1000000000;
00249         FSeedInit:=FSeed;
00250     end;
00251     Result:=FSeed;
00252 end;
00253
00254
00255 procedure TMRand.SetMultiplier(Value: comp);
00256 begin
00257     if (Value<2) or (Value>2147483646) then
00258     begin
00259         MessageDlg(Name+'Multuplier must be in the range 2...2147483646',mtError,[mbOK],0);
00260         FMultiplier:=950706376;
00261     end
00262     else
00263         FMultiplier:=Value;
00264 end;
00265
00266 function TMRand.GetMultiplier: comp;
00267 begin
00268     if (FMultiplier<2) or (FMultiplier>2147483646) then
00269         FMultiplier:=950706376;
00270     Result:=FMultiplier;
00271 end;
00272
00273 function TMRand.Uni01: real;
00274 begin
00275     FSeed:=FSeed*FMultiplier;
00276     FSeed:=FSeed-Int(FSeed/MODULUS)*MODULUS;
00277     Result:=FSeed/MODULUS;
00278 end;
00279
00280
```

```

00281 { Uniform distribution with the range LowerBound...UpperBound }
00282
00283 function TMRand.Uniform(LowerBound: real; UpperBound: real): real;
00284 begin
00285     if (LowerBound<0) or (UpperBound<0) or (UpperBound<LowerBound) then
00286     begin
00287         MessageDlg('Incorrect parameters of '+Name+'.Uniform',mtError,[mbOK],0);
00288         Result:=0.0;
00289         Exit;
00290     end;
00291     Result:=LowerBound+(UpperBound-LowerBound)*Uni01
00292 end;
00293
00294
00295 { Exponential distribution with mean "Mean" }
00296
00297 function TMRand.Exponential(Mean: real): real;
00298 begin
00299     if Mean>=0.0 then
00300         Result:=Mean*(-ln(Uni01))
00301     else
00302     begin
00303         MessageDlg('Incorrect parameter of '+Name+'.Exponential',mtError,[mbOK],0);
00304         Result:=0;
00305     end;
00306 end;
00307
00308
00309 { M-Erlang distribution with mean "Mean" }
00310
00311 function TMRand.Erlang(Mean: real; M: integer): real;
00312 var
00313     i: integer;
00314     P: real;
00315 begin
00316     if (Mean<0.0) or (M<=0) then
00317     begin
00318         MessageDlg('Incorrect parameters of '+Name+'.Erlang',mtError,[mbOK],0);
00319         Result:=0.0;
00320         Exit;
00321     end;
00322
00323     P:=1;
00324     for i:=1 to M do
00325         P:=P*Uni01;
00326     Result:=(Mean/M)*(-ln(P));
00327 end;
00328
00329
00330 { Normal distribution with mean "Mean" and deviation "Deviation" }
00331
00332 function TMRand.Normal(Mean: real; Deviation: real): real;
00333 var U1,U2: real;
00334     W,Y: real;
00335 begin
00336     if (Mean<=0.0) or (Deviation<=0.0) then
00337     begin
00338         MessageDlg('Incorrect parameter Deviation of '+Name+'.Normal',mtError,[mbOK],0);
00339         Result:=0.0;
00340         Exit;
00341     end;
00342
00343     if NormalFlag then
00344     begin
00345         repeat
00346             U1:=2*Uni01-1.0;
00347             U2:=2*Uni01-1.0;
00348             W:=U1*U1+U2*U2;
00349             until not (W>=1.0);
00350             Y:= sqrt((-2* ln (W)) / W);
00351             Result:=Mean+U1*Y*Deviation;
00352             NormalResult:=Mean+U2*Y*Deviation;
00353             NormalFlag:=False;
00354         end
00355     else
00356     begin
00357         Result:=NormalResult;
00358         NormalFlag:=True;
00359     end;
00360 end;
00361
00362
00363 { Gamma distribution for Alpha<1 and beta=1 }
00364
00365 function TMRand.GammaJonk(Alpha: real ): real;
00366 var R,R1,R2,X,Y:real;
00367 begin

```

```

00368     repeat
00369         repeat
00370             R1:=Uni01;
00371             R2:=Uni01;
00372             until ((R1>1e-30) and (R2>1e-30));
00373
00374             if (ln(R2)/alpha<-1e3) then X:=0 else X:= exp(ln(R2)/alpha);
00375             if (ln(R1)/(1-alpha)<-1e3) then Y:=0 else Y:=exp(ln(R1)/(1-alpha));
00376
00377             until (X+Y<=1);
00378
00379         repeat
00380             R:=Uni01;
00381             until (R>1e-20);
00382
00383         Result:=-ln(R)*X/(Y+X);
00384     end;
00385
00386
00387 { Gamma distribution with the mean "Mean" and shape-parameter "Alpha" }
00388
00389 function TMRand.Gamma(Mean: real; Alpha: real): real;
00390 var
00391     i: integer;
00392     P: real;
00393     IntAlpha: integer;
00394     OstAlpha: real;
00395 begin
00396
00397     if (Mean<=0.0) or (Alpha<=0.0) then
00398     begin
00399         MessageDlg('Incorrect parameters of '+Name+'.Gamma',mtError,[mbOK],0);
00400         Result:=0.0;
00401         Exit;
00402     end;
00403
00404     if Alpha<1.0 then
00405         Result:=(Mean/Alpha)*GammaJonk(Alpha)
00406     else
00407         if Alpha=1.0 then
00408             Result:=Mean*(-ln(Uni01))
00409         else
00410             begin
00411                 IntAlpha:=Round(Int(Alpha));
00412                 OstAlpha:=Alpha-IntAlpha;
00413                 repeat
00414                     P:=1;
00415                     for i:=1 to IntAlpha do
00416                         P:=P*Uni01;
00417                     until P>0;
00418                     if OstAlpha>0 then
00419                         Result:=(Mean/Alpha)*((-ln(P))+GammaJonk(OstAlpha))
00420                     else
00421                         Result:=(Mean/Alpha)*(-ln(P));
00422                 end;
00423             end;
00424         end;
00425
00426
00427 { Beta distribution with alpha-parameter "ShapeAlpha,
00428     beta-perameter "ShapeBeta,
00429     lower bound "LowerBound and
00430     upper bound "UpperBound
00431 }
00432
00433 function TMRand.Beta(ShapeAlpha: real; ShapeBeta: real;
00434     LowerBound: real; UpperBound: real): real;
00435 var
00436     X,Y1,Y2: real;
00437 begin
00438
00439     if (ShapeAlpha<=0.0) or (ShapeBeta<=0.0) or (LowerBound>UpperBound) or
00440         (LowerBound<0) or (UpperBound<0)
00441     then
00442     begin
00443         MessageDlg('Incorrect parameters of '+Name+'.Beta',mtError,[mbOK],0);
00444         Result:=0.0;
00445         Exit;
00446     end;
00447
00448     repeat
00449         Y1:=Gamma(ShapeAlpha,ShapeAlpha);
00450         Y2:=Gamma(ShapeBeta,ShapeBeta);
00451         X:=Y1/(Y1+Y2);
00452         until (X>=0) and (X<=1.0);
00453         Result:= LowerBound + (UpperBound - LowerBound ) * X;
00454     end;

```

```

00455
00456
00457 { Weibull distribution with shape-parameter "Alpha" and scale "Scale" }
00458
00459 function TMRand.Weibull(Alpha: real; Scale: real): real;
00460 begin
00461   if (Alpha<=0.0) or (Scale<=0.0) then
00462   begin
00463     MessageDlg('Incorrect parameters of '+Name+'.Weibull',mtError,[mbOK],0);
00464     Result:=0.0;
00465     Exit;
00466   end;
00467   Result:= exp(ln(Scale*(-ln(Uni01)))/Alpha);
00468 end;
00469
00470
00471 { Lognormal distribution with mean "Mean" and deviation "Deviation" }
00472
00473 function TMRand.Lognormal(Mean: real; Deviation: real): real;
00474 var
00475   MeanNorm,DispNorm: real;
00476 begin
00477
00478   if (Mean<=0.0) or (Deviation<=0.0) then
00479   begin
00480     MessageDlg('Incorrect parameters of '+Name+'.Lognormal',mtError,[mbOK],0);
00481     Result:=0.0;
00482     Exit;
00483   end;
00484
00485   DispNorm:=ln((Deviation*Deviation)/(Mean*Mean)+1.0);
00486   MeanNorm:=ln(Mean)-0.5*DispNorm;
00487   Result:=Exp(Normal(MeanNorm,SQRT(DispNorm)));
00488 end;
00489
00490
00491 { Triangular distribution with minimal value "Min", maximal - "Max" and mode "Mode" }
00492
00493 function TMRand.Triangular(Min: real; Mode: real; Max: real): real;
00494 var
00495   Part1,Part2,Full,R: real;
00496 begin
00497
00498   if (Min>Mode) or (Max<Mode) or (Min>Max) or
00499   (Min<0) or (Max<0) or (Mode<0) then
00500   begin
00501     MessageDlg('Incorrect parameters of '+Name+'.Triangular',mtError,[mbOK],0);
00502     Result:=0.0;
00503     Exit;
00504   end;
00505
00506   Part1:=Mode-Min;
00507   Part2:=Max-Mode;
00508   Full:=Max-Min;
00509   R:=Uni01;
00510   if R<=Part1/Full then
00511     Result:=Min+sqrt(Part1*Full*R)
00512   else
00513     Result:=Max-sqrt(Part2*Full*(1.0-R));
00514 end;
00515
00516
00517 procedure Register;
00518 begin
00519   RegisterComponents('MRand', [TMRand]);
00520 end;
00521
00522
00523 end.
00524
00525 */
00526
00527
00528
00529 //namespace\\}
00530 #endif /* SAMPLERDEFAULTIMPL1_H */

```

11.294 SorttFile.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */

```

```

00005
00006 /*
00007  * File:    SorttFile.h
00008  * Author:  rlcancian
00009  *
00010  * Created on 20 de novembro de 2019, 22:27
00011  */
00012
00013 #ifndef SORTTFILE_H
00014 #define SORTTFILE_H
00015
00016 #include <string>
00017
00024 class SortFile {
00025 public:
00027     SortFile();
00028     ~SortFile() = default;
00029 public:
00034     bool sort();
00039     void setDataFilename(std::string filename);
00040 private:
00042     void run(int start, int end);
00044     void swap(int position1, int position2);
00046     int partition(int start, int end);
00048     void addValue(double value, int position);
00050     double getValue(unsigned long position);
00051 private:
00052     std::string _filename;
00053     double _numElements = 0;
00054 };
00055
00056 #endif /* SORTTFILE_H */

```

11.295 Statistics_if.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Statistics_if.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 14 de Agosto de 2018, 13:47
00012  */
00013
00014 #ifndef STATISTICS_IF_H
00015 #define STATISTICS_IF_H
00016
00017 #include <string>
00018 #include "Collector_if.h"
00019
00026 class Statistics_if {
00027 public:
00028     virtual ~Statistics_if() = default;
00030     virtual Collector_if* getCollector() const = 0;
00032     virtual void setCollector(Collector_if* collector) = 0;
00033 public:
00039     virtual unsigned int numElements() = 0;
00045     virtual double min() = 0;
00051     virtual double max() = 0;
00057     virtual double average() = 0;
00063     virtual double variance() = 0;
00069     virtual double stddeviation() = 0;
00075     virtual double variationCoef() = 0;
00081     virtual double halfWidthConfidenceInterval() = 0;
00087     virtual double confidenceLevel() = 0;
00094     virtual unsigned int newSampleSize(double halfWidth) = 0;
00100     virtual void setConfidenceLevel(double confidencelevel) = 0;
00101 };
00102
00103 #endif /* STATISTICS_IF_H */

```

11.296 StatisticsDataFile_if.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.

```

```

00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:    StatisticsDataFile.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 22 de Novembro de 2018, 01:16
00012  */
00013
00014  #ifndef STATISTICSDATAFILE_IF_H
00015  #define STATISTICSDATAFILE_IF_H
00016
00017  #include "CollectorDatafile_if.h"
00018  #include "Statistics_if.h"
00019
00026  class StatisticsDatafile_if : public Statistics_if {
00027      //public:
00028      //    virtual CollectorDatafile_if* getCollector() = 0;
00029      //    virtual void setCollector(Collector_if* collector) = 0;
00030  public:
00031      virtual ~StatisticsDatafile_if() = default;
00037      virtual double mode() = 0;
00043      virtual double mediane() = 0;
00050      virtual double quartil(unsigned short num) = 0;
00057      virtual double decil(unsigned short num) = 0;
00064      virtual double centil(unsigned short num) = 0;
00070      virtual void setHistogramNumClasses(unsigned short num) = 0;
00076      virtual unsigned short histogramNumClasses() = 0;
00083      virtual double histogramClassLowerLimit(unsigned short classNum) = 0;
00090      virtual unsigned int histogramClassFrequency(unsigned short classNum) = 0;
00091
00092  };
00093
00094  #endif /* STATISTICSDATAFILE_IF_H */

```

11.297 StatisticsDataFileDefaultImpl.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:    StatisticsDataFileDefaultImpl.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 22 de Novembro de 2018, 01:24
00012  */
00013
00014  #ifndef STATISTICSDATAFILEDEFAULTIMPL_H
00015  #define STATISTICSDATAFILEDEFAULTIMPL_H
00016
00017  #include "StatisticsDataFile_if.h"
00018  #include "SorttFile.h"
00019  #include <limits.h>
00020  #include <map>
00021  #include <string>
00022
00023  typedef double valueType;
00024
00031  class StatisticsDatafileDefaultImpl1 : public StatisticsDatafile_if {
00032  public:
00034      StatisticsDatafileDefaultImpl1();
00036      virtual ~StatisticsDatafileDefaultImpl1();
00037  public:
00039      virtual Collector_if* getCollector() const override;
00041      virtual void setCollector(Collector_if* collector) override;
00042  public:
00044      virtual unsigned int numElements() override;
00046      virtual double min() override;
00048      virtual double max() override;
00050      virtual double average() override;
00052      virtual double variance() override;
00054      virtual double stddeviation() override;
00056      virtual double variationCoef() override;
00058      virtual double halfWidthConfidenceInterval() override;
00060      virtual double confidenceLevel() override;
00062      virtual unsigned int newSampleSize(double halfWidth) override;
00064      virtual void setConfidenceLevel(double confidencelevel) override;
00065  public:

```

```

00067     virtual double mode() override;
00069     virtual double mediane() override;
00071     virtual double quartil(unsigned short num) override;
00073     virtual double decil(unsigned short num) override;
00075     virtual double centil(unsigned short num) override;
00077     virtual void setHistogramNumClasses(unsigned short num) override;
00079     virtual unsigned short histogramNumClasses() override;
00081     virtual double histogramClassLowerLimit(unsigned short classNum) override;
00083     virtual unsigned int histogramClassFrequency(unsigned short classNum) override;
00084 private:
00086     void _invalidateCachedResults();
00088     void _resetSortedFileState();
00090     void _sortFile();
00092     bool _hasNewValue();
00094     double _getNormalProbability(double confidenceLevel);
00095 private:
00096     CollectorDatafile_if* _collector;
00097     bool _ownsCollector = true;
00098     CollectorDatafile_if* _collectorSorted;
00099     bool _ownsCollectorSorted = true;
00100     //CollectorDatafile_if* _collector;
00101     SortFile * sort = new SortFile();
00102     bool _ownsSort = true;
00103     double _confidenceLevel = 0.95;
00104     unsigned long _numElements = 0;
00105     std::map<double, double> _z;
00106     bool _fileSorted = false;
00107     bool _fileSortedCreated = false;
00108     std::string _sortedSourceFilename;
00109     //unsigned long _numElements = 0;
00110     bool _numElementsCalculated = false;
00111     double _max = INT_MIN;
00112     bool _maxCalculated = false;
00113     double _min = INT_MAX;
00114     bool _minCalculated = false;
00115     bool _modeCalculated = false;
00116     double _mode = 0;
00117     bool _medianeCalculated = false;
00118     double _mediane = 0;
00119     double _average = 0;
00120     bool _averageCalculated = false;
00121     double _variance = 0;
00122     bool _varianceCalculated = false;
00123     double _sumSquareElements = 0;
00124     double _sumElements = 0;
00125     double _stddeviation = 0;
00126     bool _stddeviationCalculated = false;
00127     double _variationCoef = 0;
00128     bool _variationCoefCalculated = false;
00129     bool _halfWidthConfidenceIntervalCalculated = false;
00130     double _lastConfidenceLevel = 0;
00131     double _halfWidthConfidenceInterval = 0;
00132     bool _newSampleSizeCalculated = false;
00133     double _lastNewSampleSizeConfidenceLevel = 0;
00134     double _lastNewSampleSizeHalfWidth = 0;
00135     double _newSampleSize = 0;
00136     bool _quartilCalculated = false;
00137     unsigned short _lastQuartilNum = 0;
00138     double _quartil = 0;
00139     bool _decilCalculated = false;
00140     unsigned short _lastDecilNum = 0;
00141     double _decil = 0;
00142     bool _centilCalculated = false;
00143     unsigned short _lastCentilNum = 0;
00144     double _centil = 0;
00145     bool _histogramNumClassesCalculated = false;
00146     unsigned short _histogramNumClasses = 1;
00147     bool _histogramClassLowerLimitCalculated = false;
00148     unsigned short _lastClassNumHistogramClassLowerLimit = 0;
00149     double _histogramClassLowerLimit = 0;
00150     bool _histogramClassFrequencyCalculated = false;
00151     unsigned short _lastClassNumHistogramClassFrequency = 0;
00152     unsigned int _histogramClassFrequency = 0;
00153     bool _proportionCalculated = false;
00154     double _proportion = 0;
00155 };
00156
00157 #endif /* STATISTICSDEFAULTIMPL1_H */

```

11.298 StatisticsDefaultImpl1.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.

```

```

00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:    StatisticsDefaultImpl1.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 1 de Agosto de 2018, 21:03
00012  */
00013
00014  #ifndef STATISTICSDEFAULTIMPL1_H
00015  #define STATISTICSDEFAULTIMPL1_H
00016
00017  #include "Statistics_if.h"
00018  #include "Collector_if.h"
00019  #include <map>
00020  //namespace GenesysKernel {
00021
00022  class StatisticsDefaultImpl1 : public Statistics_if {
00023  public:
00024      StatisticsDefaultImpl1();
00025      StatisticsDefaultImpl1(Collector_if* collector);
00026      virtual ~StatisticsDefaultImpl1() override;
00027  public:
00028      virtual Collector_if* getCollector() const override;
00029      virtual void setCollector(Collector_if* collector) override;
00030  public:
00031      virtual unsigned int numElements() override;
00032      virtual double min() override;
00033      virtual double max() override;
00034      virtual double average() override;
00035      virtual double variance() override;
00036      virtual double stddeviation() override;
00037      virtual double variationCoef() override;
00038      virtual double halfWidthConfidenceInterval() override;
00039      virtual double confidenceLevel() override;
00040      virtual unsigned int newSampleSize(double halfWidth) override;
00041      virtual void setConfidenceLevel(double confidencelevel) override;
00042  private:
00043      void collectorAddHandler(double newValue, double newWeight);
00044      void collectorClearHandler();
00045      void _bindCollectorHandlers(Collector_if* collector);
00046      void _resetStateForCurrentCollector();
00047      void initStatistics();
00048  private:
00049      Collector_if* _collector;
00050      bool _ownsCollector = false;
00051      bool _onlineStateValid = true;
00052      unsigned long _elems;
00053      double _sumData;
00054      double _sumDataSquare;
00055      double _sumWeight;
00056      double _sumWeightSquare;
00057      double _min;
00058      double _max;
00059      double _average;
00060      double _variance;
00061      double _unweightedvariance;
00062      double _unbiasedVariance;
00063      double _stddeviation;
00064      double _variationCoef;
00065      double _confidenceLevel = 0.95;
00066      double _criticalTn_1 = 1.96;
00067      double _halfWidth;
00068      const std::map<double, double> _supportedConfidenceLevels{
00069          {0.50, 0.0},
00070          {0.80, 1.282},
00071          {0.90, 1.645},
00072          {0.95, 1.96},
00073          {0.975, 2.06},
00074          {0.98, 2.326},
00075          {0.99, 2.576},
00076          {0.995, 2.807}
00077      };
00078  };
00079  };
00080  //namespace\\}
00081  #endif /* STATISTICSDEFAULTIMPL1_H */

```

11.299 TraitsKernel.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.

```



```

00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File: Traits.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 14 de Agosto de 2018, 19:36
00012  */
00013
00014
00015 #ifndef TRAITSKERNEL_H
00016 #define TRAITSKERNEL_H
00017
00018
00019 #include "simulator/persistence/PersistenceDefaultImpl2.h"
00020 #include "simulator/persistence/PersistencePartialLoadImpl1.h"
00021 #include "simulator/SimulationReporterDefaultImpl1.h"
00022 #include "simulator/essentialPlugins/Counter.h"
00023 #include "simulator/model/ModelCheckerDefaultImpl1.h"
00024 #include "simulator/ParserDefaultImpl2.h"
00025 #include "simulator/SimulationReporter_if.h"
00026
00027 #include "statistics/CollectorDatafileDefaultImpl1.h"
00028 #include "statistics/CollectorDefaultImpl1.h"
00029 #include "statistics/StatisticsDefaultImpl1.h"
00030 #include "statistics/StatisticsDataFileDefaultImpl.h"
00031 #include "statistics/SamplerDefaultImpl1.h"
00032
00033 #include "../plugins/PluginConnectorStaticImpl1.h"
00034 #include "../plugins/PluginConnectorDummyImpl1.h"
00035
00036
00037 //namespace GenesysKernel {
00038
00039 template <typename T>
00040 struct TraitsKernel {
00041 };
00042
00043 /*
00044  * Simulation and Simulation Parts
00045  */
00046
00047 template <>
00048 struct TraitsKernel<Simulator> {
00049     static const TraceManager::Level traceLevel = TraceManager::Level::L9_mostDetailed;
00050 };
00051
00052 template <>
00053 struct TraitsKernel<SimulationReporter_if> {
00054     typedef SimulationReporterDefaultImpl1 Implementation;
00055     typedef Counter CounterImplementation;
00056     static const TraceManager::Level traceLevel = TraceManager::Level::L2_results;
00057 };
00058
00059 template <>
00060 struct TraitsKernel<PluginConnector_if> {
00061     //typedef PluginConnectorStaticImpl1 Implementation;
00062     typedef PluginConnectorDummyImpl1 Implementation;
00063     //typedef PluginConnectorDynamicLibraryLoader Implementation;
00064     static const TraceManager::Level traceLevel = TraceManager::Level::L4_warning;
00065 };
00066
00067 template <>
00068 struct TraitsKernel<Parser_if> {
00069     typedef ParserDefaultImpl2 Implementation;
00070 };
00071
00072 /*
00073  * Model and Model Parts
00074  */
00075
00076 template <>
00077 struct TraitsKernel<Model> {
00078     typedef StatisticsDefaultImpl1 StatisticsCollector_StatisticsImplementation;
00079     typedef CollectorDefaultImpl1 StatisticsCollector_CollectorImplementation;
00080     static const TraceManager::Level traceLevel = TraceManager::Level::L5_event;
00081 };
00082
00083 template <>
00084 struct TraitsKernel<ModelComponent> {
00085     static constexpr bool reportStatistics = true;
00086     static const TraceManager::Level traceLevel = TraceManager::Level::L2_results;
00087 };
00088
00089 template <>

```

```

00090 struct TraitsKernel<ModelDataDefinition> {
00091     static constexpr bool reportStatistics = true;
00092     static const TraceManager::Level traceLevel = TraceManager::Level::L2_results;
00093 };
00094
00095 template <>
00096 struct TraitsKernel<ModelChecker_if> {
00097     typedef ModelCheckerDefaultImpl1 Implementation;
00098     static const TraceManager::Level traceLevel = TraceManager::Level::L2_results;
00099 };
00100
00101 template <>
00102 struct TraitsKernel<Persistence_if> {
00103     typedef PersistencePartialLoadImpl1 Implementation;
00104     static const TraceManager::Level traceLevel = TraceManager::Level::L2_results;
00105 };
00106
00107 /*
00108  * Statistics
00109  */
00110 template <>
00111 struct TraitsKernel<Statistics_if> {
00112     typedef double DataType;
00113     // @ToDo: (importante): not used yet. Change all classes that collect statistics to this type (so
00114     classes that deal with erros and bit limits can be assigned to it
00115     static constexpr double SignificanceLevel = 0.05;
00116 };
00117
00118 template <>
00119 struct TraitsKernel<StatisticsDatafile_if> {
00120     typedef StatisticsDatafileDefaultImpl1 Implementation;
00121     typedef CollectorDatafileDefaultImpl1 CollectorImplementation;
00122     static constexpr double SignificanceLevel = 0.05;
00123 };
00124
00125 template <>
00126 struct TraitsKernel<Sampler_if> {
00127     typedef SamplerDefaultImpl1 Implementation;
00128     typedef SamplerDefaultImpl1::DefaultImpl1RNG_Parameters Parameters;
00129 };
00130
00131 template <>
00132 struct TraitsKernel<Collector_if> {
00133     typedef CollectorDatafileDefaultImpl1 Implementation;
00134     typedef double DataType;
00135     // @ToDo: (importante): not used yet. Change all classes that collect statistics to this type (so
00136     classes that deal with erros and bit limits can be assigned to it
00137 };
00138
00139 //namespace\\
00140 #endif /* TRAITSKERNEL_H */

```

11.300 Exact.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Exact.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 12 de agosto de 2021, 18:55
00012  */
00013
00014 #ifndef Exact_H
00015 #define Exact_H
00016
00017 #include <iostream>
00018 #include <iomanip>
00019
00020 class Exact {
00021     //friend ostream &operator<<(ostream&, const Exact &);
00022     //friend istream &operator>>(istream&, Exact &);
00023 public:
00024     Exact() {
00025     };
00026
00027     Exact(int num, int den = 1) {

```

```

00029     _num = num;
00030     _den = den;
00031 };
00032
00033 const Exact &operator+(const Exact &right) {
00034     Exact e;
00035     e._num = this->_num * right._den + right._num * this->_den;
00036     e._den = this->_den * right._den;
00037     return e;
00038 }
00039
00040 const Exact &operator+=(const Exact &right) {
00041     this->_num = this->_num * right._den + right._num * this->_den;
00042     this->_den *= right._den;
00043 }
00044
00045 const Exact &operator-=(const Exact &right) {
00046     this->_num = this->_num * right._den - right._num * this->_den;
00047     this->_den *= right._den;
00048 }
00049
00050 const Exact &operator*=(const Exact &right) {
00051     this->_num *= right._num;
00052     this->_den *= right._den;
00053 }
00054
00055 const Exact &operator/=(const Exact &right) {
00056     this->_num *= right._num;
00057     this->_den *= right._den;
00058 }
00059
00060 const Exact &operator=(const Exact &right) {
00061     this->_num = right._num;
00062     this->_den = right._den;
00063 }
00064
00065 const Exact &operator=(const int &right) {
00066     this->_num = right;
00067     this->_den = 1;
00068 }
00069
00070 bool operator==(const Exact &right) {
00071     // verify proportionality is missing
00072     return (this->_num == right._num) && (this->_den == right._den);
00073 }
00074
00075 bool operator==(const double &right) {
00076     // verify proportionality is missing
00077     return (this->_num / this->_den) == right;
00078 }
00079
00080 bool operator<(const Exact &right) {
00081     // verify proportionality is missing
00082     return (this->_num / this->_den) < (right._num / right._den);
00083 }
00084
00085 bool operator<(const double &right) {
00086     // verify proportionality is missing
00087     return (this->_num / this->_den) < right;
00088 }
00089
00090 bool operator<=(const Exact &right) {
00091     // verify proportionality is missing
00092     return (this->_num / this->_den) <= (right._num / right._den);
00093 }
00094
00095 bool operator<=(const double &right) {
00096     // verify proportionality is missing
00097     return (this->_num / this->_den) <= right;
00098 }
00099
00100 bool operator>(const Exact &right) {
00101     // verify proportionality is missing
00102     return (this->_num / this->_den) > (right._num / right._den);
00103 }
00104
00105 bool operator>(const double &right) {
00106     // verify proportionality is missing
00107     return (this->_num / this->_den) > right;
00108 }
00109
00110 bool operator>=(const Exact &right) {
00111     // verify proportionality is missing
00112     return (this->_num / this->_den) >= (right._num / right._den);
00113 }
00114
00115 bool operator>=(const double &right) {

```

```

00116         // verify proportionality is missing
00117         return (this->_num / this->_den) >= right;
00118     }
00119
00120     void Simplify() {
00121     }
00122
00123     double toFloat() {
00124         return _num / _den;
00125     };
00126 protected:
00127 private:
00128     double _num, _den;
00129 };
00130
00131
00132 /*
00133 ostream &operator<<(ostream &output, const Exact &num) {
00134     output << num._num << "/" << num._den;
00135     return output;
00136 }
00137
00138 /*
00139 istream &operator>>(istream &input, Exact &num) {
00140     input >> num._num;
00141     input >> num._den;
00142     return input;
00143 }
00144 */
00145
00146
00147 #endif /* Exact_H */
00148

```

11.301 List.h

```

00001 #ifndef GENLIST_H
00002 #define GENLIST_H
00003 /*
00004  * To change this license header, choose License Headers in Project Properties.
00005  * To change this template file, choose Tools | Templates
00006  * and open the template in the editor.
00007  */
00008
00009 /*
00010  * File: List.h
00011  * Author: rafael.luiz.cancian
00012  *
00013  * Created on 21 de Junho de 2018, 12:55
00014  */
00015
00016
00017
00018 #include <string>
00019 #include <list>
00020 #include <map>
00021 #include <iterator>
00022 #include <functional>
00023 #include <algorithm>
00024
00025
00032 template <typename T>
00033 class List {
00034 public:
00035     using CompFuncnt = std::function<bool(const T, const T) >;
00036 public:
00037     List();
00038     List(const List<T>& origin);
00039     List<T>& operator=(const List<T>& origin);
00040     virtual ~List();
00041 public: // direct access to list
00042     unsigned int size();
00043     bool empty();
00044     void clear();
00045     void pop_front();
00046     template<class Compare>
00047     void sort(Compare comp);
00048     std::list<T>* list() const;
00049 public: // new methods
00050     T create();
00051     template<typename U>
00052     T create(U arg);
00053     std::string show();

```

```

00066     typename std::list<T>::iterator find(T element);
00067     //int rankOf(T modeldatum); ///< returns the position (1st position=0) of the modeldatum if found,
    or negative value if not found
00068 public: // improved (easier) methods
00070     void insert(T element);
00072     void remove(T element);
00074     void setAtRank(unsigned int rank, T element);
00076     T getAtRank(unsigned int rank);
00078     T next();
00080     T front();
00082     T last();
00084     T previous();
00086     T current(); // get current modeldatum on the list (the last used)
00088     void setSortFunc(CompFunc _sortFunc);
00089 protected:
00090     //std::map<Util::identification, T>* _map;
00091     std::list<T>* _list;
00092     CompFunc _sortFunc{[](const T, const T) {
00093         return false;
00094     }};
00095     typename std::list<T>::iterator _it;
00096 };
00097
00098 template <typename T>
00099 List<T>::List() {
00100     //_map = new std::map<Util::identification, T>();
00101     _list = new std::list<T>();
00102     _it = _list->begin();
00103 }
00104
00105 template <typename T>
00106 List<T>::List(const List<T>& origin) {
00107     _list = new std::list<T>(*origin._list);
00108     _sortFunc = origin._sortFunc;
00109     _it = _list->begin();
00110 }
00111
00112 template <typename T>
00113 List<T>& List<T>::operator=(const List<T>& origin) {
00114     if (this != &origin) {
00115         *_list = *origin._list;
00116         _sortFunc = origin._sortFunc;
00117         _it = _list->begin();
00118     }
00119     return *this;
00120 }
00121
00122 template <typename T>
00123 List<T>::~~List() {
00124     delete _list;
00125 }
00126
00127 template <typename T>
00128 std::list<T>* List<T>::list() const {
00129     return _list;
00130 }
00131
00132 template <typename T>
00133 unsigned int List<T>::size() {
00134     return _list->size();
00135 }
00136
00137 //template <typename T>
00138 //List<T>::List(const List& orig) {
00139 //}
00140
00141 //template <typename T>
00142 //List<T>::~~List() {
00143 //}
00144
00145 template <typename T>
00146 std::string List<T>::show() {
00147     int i = 0;
00148     std::string text = "{";
00149     for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++, i++) {
00150         text += "[" + std::to_string(i) + "]=(" + (*it)->show() + ")," ;
00151     }
00152     text += "}";
00153     return text;
00154 }
00155
00156 template <typename T>
00157 void List<T>::insert(T element) {
00158     const bool wasEmpty = _list->empty();
00159     _list->insert(std::upper_bound(_list->begin(), _list->end(), element, _sortFunc), element);
00160     if (wasEmpty) {
00161         _it = _list->begin();
    }

```

```

00162     }
00163 }
00164
00165 template <typename T>
00166 bool List<T>::empty() {
00167     return _list->empty();
00168 }
00169
00170 template <typename T>
00171 void List<T>::pop_front() {
00172     if (_list->empty()) {
00173         return;
00174     }
00175     const bool cursorAtFront = _it == _list->begin();
00176     _list->pop_front();
00177     if (cursorAtFront) {
00178         _it = _list->begin(); // if it points to the removed modeldatum, then changes to begin
00179     }
00180 }
00181
00182 template <typename T>
00183 void List<T>::remove(T element) {
00184     // Reset the internal cursor after erasure to avoid dereferencing a stale iterator during
    recursive removals.
00185     _list->remove(element);
00186     _it = _list->begin();
00187 }
00188
00189 template <typename T>
00190 T List<T>::create() {
00191     return new T();
00192 }
00193
00194 template <typename T>
00195 void List<T>::clear() {
00196     _list->clear();
00197     _it = _list->begin();
00198 }
00199
00200 template <typename T>
00201 T List<T>::getAtRank(unsigned int rank) {
00202     unsigned int thisRank = 0;
00203     for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00204         if (rank == thisRank) {
00205             return (*it);
00206         } else {
00207             thisRank++;
00208         }
00209     }
00210     return T{};
00211 }
00212
00213 template <typename T>
00214 void List<T>::setAtRank(unsigned int rank, T element) {
00215     if (rank == _list->size()) {
00216         _list->insert(_list->end(), element);
00217     } else {
00218         unsigned int thisRank = 0;
00219         for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00220             if (rank == thisRank) {
00221                 *it = element;
00222                 return;
00223             } else {
00224                 thisRank++;
00225             }
00226         }
00227     }
00228 }
00229
00230 template <typename T>
00231 T List<T>::next() {
00232     if (_list->empty() || _it == _list->end()) {
00233         return T{};
00234     }
00235     _it++;
00236     if (_it != _list->end())
00237         return (*_it);
00238     else
00239         return T{};
00240 }
00241
00242
00243 template <typename T>
00244 typename std::list<T>::iterator List<T>::find(const T element) {
00245     for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00246         if ((*it) == element) {
00247             return it;

```

```

00248     }
00249 }
00250 return _list->end();
00251 }
00252
00253 /*
00254 template <typename T>
00255 int List<T>::rankOf(T modeldatum) {
00256     int rank = 0;
00257     for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00258         if ((*it) == modeldatum) {
00259             return rank;
00260         } else
00261             rank++;
00262     }
00263     return -1; // not found -> negative rank
00264 }
00265 */
00266
00267 template <typename T>
00268 T List<T>::front() {
00269     if (_list->empty()) {
00270         return T{};
00271     }
00272     _it = _list->begin();
00273     //if (_it != _list->end())
00274     return (*_it);
00275     //else
00276     //return dynamic_cast<T>(nullptr);
00277 }
00278
00279 template <typename T>
00280 T List<T>::last() {
00281     if (_list->empty()) {
00282         return T{};
00283     }
00284     _it = _list->end();
00285     _it--;
00286     return (*_it);
00287     //else return nullptr;
00288 }
00289
00290 template <typename T>
00291 T List<T>::previous() {
00292     if (_list->empty() || _it == _list->begin()) {
00293         return T{};
00294     }
00295     _it--;
00296     return (*_it);
00297 }
00298
00299 template <typename T>
00300 T List<T>::current() {
00301     if (_list->empty() || _it == _list->end()) {
00302         return T{};
00303     }
00304     return (*_it);
00305 }
00306
00307 template <typename T>
00308 void List<T>::setSortFunc(CompFunct _sortFunc) {
00309     this->_sortFunc = _sortFunc;
00310 }
00311
00312 template <typename T>
00313 template<typename U>
00314 T List<T>::create(U arg) {
00315     return T(arg);
00316 }
00317
00318 template <typename T>
00319 template<class Compare>
00320 void List<T>::sort(Compare comp) {
00321     _list->sort(comp);
00322 }
00323
00324 #endif

```

11.302 ListObservable.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates

```

```

00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:   TManager.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 21 de Junho de 2018, 12:55
00012  */
00013
00014  #ifndef LISTOBSERVABLE_H
00015  #define LISTOBSERVABLE_H
00016
00017  #include <string>
00018  #include <list>
00019  #include <map>
00020  #include <iterator>
00021  #include <functional>
00022  #include <algorithm>
00023  #include "Util.h"
00024  #include "../simulator/ModelElement.h"
00025
00026  #include "List.h"
00027
00034  template <typename T>
00035  class ListObservable : public List {
00036  public:
00037      using CompFunc = std::function<bool(const T, const T)>;
00038  public:
00040      ListObservable();
00041      virtual ~ListObservable() = default;
00042  public: // direct access to list
00044      void clear();
00046      void pop_front();
00047  public: // improved (easier) methods
00049      void insert(T element);
00051      void remove(T element);
00053      void setAtRank(unsigned int rank, T element);
00054  };
00055
00056  template <typename T>
00057  ListObservable<T>::ListObservable() {
00058      //_map = new std::map<Util::identification, T>();
00059      _list = new std::list<T>();
00060      _it = _list->begin();
00061  }
00062
00063  template <typename T>
00064  std::list<T>* ListObservable<T>::list() const {
00065      return _list;
00066  }
00067
00068  template <typename T>
00069  unsigned int ListObservable<T>::size() {
00070      return _list->size();
00071  }
00072
00073  //template <typename T>
00074  //ListObservable<T>::ListObservable(const ListObservable& orig) {
00075  //}
00076
00077  //template <typename T>
00078  //ListObservable<T>::~ListObservable() {
00079  //}
00080
00081  template <typename T>
00082  std::string ListObservable<T>::show() {
00083      int i = 0;
00084      std::string text = "{";
00085      for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++, i++) {
00086          text += "[" + std::to_string(i) + "]=(" + (*it)->show() + ")," ;
00087      }
00088      text += "}";
00089      return text;
00090  }
00091
00092  template <typename T>
00093  void ListObservable<T>::insert(T element) {
00094      _list->insert(std::upper_bound(_list->begin(), _list->end(), element, _sortFunc), element);
00095  }
00096
00097  template <typename T>
00098  bool ListObservable<T>::empty() {
00099      return _list->empty();
00100  }
00101
00102  template <typename T>

```



```

00103 void ListObservable<T>::pop_front() {
00104     typename std::list<T>::iterator itTemp = _list->begin();
00105     _list->pop_front();
00106     if (_it == itTemp) { /* @ToDo: (importante): check this */
00107         _it = _list->begin(); // if it points to the removed element, then changes to begin
00108     }
00109 }
00110
00111 template <typename T>
00112 void ListObservable<T>::remove(T element) {
00113     _list->remove(element);
00114     if ((*_it) == element) { /* @ToDo: (importante): check this */
00115         _it = _list->begin(); // if it points to the removed element, then changes to begin
00116     }
00117 }
00118
00119 template <typename T>
00120 T ListObservable<T>::create() {
00121     return new T();
00122 }
00123
00124 template <typename T>
00125 void ListObservable<T>::clear() {
00126     _list->clear();
00127 }
00128
00129 template <typename T>
00130 T ListObservable<T>::getAtRank(unsigned int rank) {
00131     unsigned int thisRank = 0;
00132     for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00133         if (rank == thisRank) {
00134             return (*it);
00135         } else {
00136             thisRank++;
00137         }
00138     }
00139     return 0; /* @ToDo: (importante): Invalid return depends on T. If T is pointer, nullptr works
    fine. If T is double, it does not. I just let (*it), buut it is not nice */
00140 }
00141
00142 template <typename T>
00143 void ListObservable<T>::setAtRank(unsigned int rank, T element) {
00144     if (rank == _list->size()) {
00145         _list->insert(_list->end(), element);
00146     } else {
00147         unsigned int thisRank = 0;
00148         for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00149             if (rank == thisRank) {
00150                 *it = element;
00151                 return;
00152             } else {
00153                 thisRank++;
00154             }
00155         }
00156     }
00157 }
00158
00159 template <typename T>
00160 T ListObservable<T>::next() {
00161     _it++;
00162     if (_it != _list->end())
00163         return (*_it);
00164     else
00165         return nullptr;
00166 }
00167
00168 template <typename T>
00169 typename std::list<T>::iterator ListObservable<T>::find(T element) {
00170     for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00171         if ((*it) == element) {
00172             return it;
00173         }
00174     }
00175     return _list->end(); /* @ToDo: (importante): check nullptr or invalid iterator when not found */
00176     //return nullptr;
00177 }
00178
00179 /*
00180 template <typename T>
00181 int ListObservable<T>::rankOf(T element) {
00182     int rank = 0;
00183     for (typename std::list<T>::iterator it = _list->begin(); it != _list->end(); it++) {
00184         if ((*it) == element) {
00185             return rank;
00186         } else
00187             rank++;
00188     }

```

```

00189     }
00190     return -1; // not found -> negative rank
00191 }
00192 */
00193
00194 template <typename T>
00195 T ListObservable<T>::front() {
00196     _it = _list->begin();
00197     //if (_it != _list->end())
00198     return (*_it);
00199     //else
00200     //return dynamic_cast<T>(nullptr);
00201 }
00202
00203 template <typename T>
00204 T ListObservable<T>::last() {
00205     _it = _list->end();
00206     _it--;
00207     //if (_it != _list->end()) // @ToDo: (importante): CHECK!!!
00208     return (*_it);
00209     //else return nullptr;
00210 }
00211
00212 template <typename T>
00213 T ListObservable<T>::previous() {
00214     _it--; // @ToDo: (importante): CHECK!!!
00215     return (*_it);
00216 }
00217
00218 template <typename T>
00219 T ListObservable<T>::current() {
00220     /* @ToDo: (importante): To implement (i thing it's just to check). Must actualize _it on other
00221     methods when other elements are accessed */
00222     return (*_it);
00223 }
00224
00225 template <typename T>
00226 void ListObservable<T>::setSortFunc(CompFunc _sortFunc) {
00227     this->_sortFunc = _sortFunc;
00228 }
00229
00230 template <typename T>
00231 template <typename U>
00232 T ListObservable<T>::create(U arg) {
00233     return T(arg);
00234 }
00235
00236 template <typename T>
00237 template <class Compare>
00238 void ListObservable<T>::sort(Compare comp) {
00239     _list->sort(comp);
00240 }
00241 #endif /* LISTOBSERVABLE_H */

```

11.303 Util.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Util.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 21 de Junho de 2018, 13:02
00012  */
00013
00014
00015 //
00016 // VINCULADOR
00017 // SAIDA
00018 // ${CND_DISTDIR}/${CND_CONF}/${CND_PLATFORM}/genesysteminalapplication
00019 //
00020
00021
00022 #ifndef UTIL_H
00023 #define UTIL_H
00024
00025 #include <map>
00026 #include <typeinfo>

```

```

00027 #include <string>
00028 #include <list>
00029 #include <cctype>
00030 #include <algorithm>
00031 #include <cctype>
00032 #include <locale>
00033 #include <stdio.h>
00034 // dir
00035 #include <limits.h>
00036 #include <unistd.h>
00037 #include <vector>
00038 #include <dirent.h>
00039 #include <errno.h>
00040 #include <iostream>
00041 #include <sys/stat.h>
00042 #include <cstring>
00043
00044 #include <iostream>
00045 #include <string>
00046 //namespace GenesysKernel {
00047
00048 class Util {
00049 public:
00050     typedef unsigned long identification;
00051     typedef unsigned int rank;
00052
00053     // @ToDo: (pequena alteração): Should be insde ModelSimulation, where time goes on
00054
00055     enum class TimeUnit : int {
00056         unknown = 0,
00057         picosecond = 1,
00058         nanosecond = 2,
00059         microsecond = 3,
00060         milisecond = 4,
00061         second = 5,
00062         minute = 6,
00063         hour = 7,
00064         day = 8,
00065         week = 9,
00066         num_elements = 10
00067     };
00068
00069     static std::string StrTimeUnitShort(Util::TimeUnit timeUnit);
00070     static std::string StrTimeUnitLong(Util::TimeUnit timeUnit);
00071     static std::string convertEnumToStr(Util::TimeUnit timeUnit);
00072
00073     // @ToDo: (pequena alteração): check: here? Shouldn't it be on SimulationReport interface?
00074     enum class AllocationType : int {
00075         ValueAdded = 0, NonValueAdded = 1, Transfer = 2, Wait = 3, Others = 4, num_elements = 5
00076     };
00077
00078     static std::string StrAllocation(Util::AllocationType allocation);
00079     static std::string convertEnumToStr(Util::AllocationType allocation);
00080
00081     enum class TimeFormat : unsigned int {
00082         twelve = 12,
00083         twentyFour = 24,
00084     };
00085
00086 private:
00087     static unsigned int _S_indentation;
00088     static Util::identification _S_lastId;
00089     static std::map<std::string, Util::identification> _S_lastIdOfTpe;
00090     static std::map<std::string, std::string> _S_TypeOf;
00091
00092 public: // indentation and string
00093     static void SetIndent(const unsigned short indent);
00094     static void IncIndent();
00095     static void DecIndent();
00096     static void SepKeyVal(std::string str, std::string& key, std::string& value);
00097     static std::string Indent();
00098     static std::string SetW(std::string text, unsigned short width);
00099     static std::string StrTruncIfInt(double value);
00100     static std::string StrTruncIfInt(std::string strValue);
00101     static std::string Trim(std::string str);
00102     static std::string StrReplace(std::string text, std::string searchFor, std::string replaceBy);
00103     static std::string StrReplaceSpecialChars(std::string text);
00104     static std::string StrIndex(int index);
00105     //static char* Str2CharPtr(std::string str);
00106     static void Trimwithin(std::string& str);
00107
00108 public: // show strucutres
00109     static std::string Map2str(std::map<std::string, std::string>* mapss);
00110     static std::string Map2str(std::map<std::string, double>* mapss);
00111     static std::string List2str(std::list<unsigned int>* list);
00112
00113 public:

```

```

00219 // identification // @ToDo: (importante): CHECK ALL, since some should be private and available
    to FRIEND classes in the kernel
00224 static Util::identification GenerateNewId();
00230 static Util::identification GenerateNewIdOfType(std::string objtype);
00236 static Util::identification GetLastIdOfType(std::string objtype);
00241 static void ResetIdOfType(std::string objtype);
00245 static void ResetAllIds();
00246
00247 public: // simulation support
00254 static double TimeUnitConvert(Util::TimeUnit timeUnit1, Util::TimeUnit timeUnit2);
00255
00256 public: // files
00261 static char DirSeparator();
00267 static std::string FilenameFromFullFilename(const std::string& s);
00272 static void FileDelete(const std::string& filename);
00278 static std::string PathFromFullFilename(const std::string& s);
00283 static std::string RunningPath();
00291 static std::vector<std::string> ListFiles(std::string dir, std::string fileFilter = "",
00292                                         mode_t attribFilter = S_IFREG & S_IFDIR);
00298 static bool FileExists(const std::string& name);
00299
00300 //public: // operating system specifics
00301 class CommandResult {
00302 public:
00303     CommandResult() {
00304     }
00305
00306     bool success = false;
00307     std::string commandStdOutput = "";
00308     std::string commandErrOutput = "";
00309     std::string workingDirectory = "./";
00310     std::string destinationPath = "./";
00311 };
00312
00318 static Util::CommandResult ExecuteCommand(std::string command);
00319
00320 public: // template implementations
00321
00327 template <typename T>
00328 static std::string TypeOf() {
00329     std::string name = typeid(T).name();
00330     std::map<std::string, std::string>::iterator it = _S_TypeOf.find(name);
00331     if (it != _S_TypeOf.end()) {
00332         return (*it).second;
00333     }
00334     else {
00335         std::string newname(name);
00336         while (std::isdigit(newname[0])) {
00337             newname.erase(0, 1);
00338         }
00339         _S_TypeOf.insert(std::pair<std::string, std::string>(name, newname));
00340         return newname;
00341     }
00342 }
00343
00349 template <class T>
00350 static Util::identification GenerateNewIdOfType() {
00351     std::string objtype = Util::TypeOf<T>();
00352     std::map<std::string, Util::identification>::iterator it =
Util::_S_lastIdOfType.find(objtype);
00353     if (it == Util::_S_lastIdOfType.end()) {
00354         // a new one. create the pair
00355         Util::_S_lastIdOfType.insert(std::pair<std::string, Util::identification>(objtype, 0));
00356         it = Util::_S_lastIdOfType.find(objtype);
00357     }
00358     Util::identification next = (*it).second;
00359     next++;
00360     (*it).second = next;
00361     return (*it).second;
00362 }
00363
00364 private:
00365     Util();
00366     virtual ~Util() = default;
00367 };
00368
00369 //namespace\\}
00370 #endif /* UTIL_H */

```

11.304 AIAssistant.h

```

00001 #ifndef AIASSISTANTCOMPONENT_H
00002 #define AIASSISTANTCOMPONENT_H

```

```

00003
00004 #include "kernel/simulator/model/ModelComponent.h"
00005 #include "plugins/data/AI/AISupport.h"
00006
00007 #include <map>
00008
00009 class AIAssistant : public ModelComponent {
00010 public:
00011     AIAssistant(Model* model, std::string name = "");
00012     virtual ~AIAssistant() override = default;
00013
00014     static PluginInformation* GetPluginInformation();
00015     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00016     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00017
00018     void setAISupport(AISupport* support);
00019     AISupport* getAISupport() const;
00020     void setAISupportName(std::string value);
00021     std::string getAISupportName() const;
00022     void setPromptTemplate(std::string value);
00023     std::string getPromptTemplate() const;
00024     void setConversationScope(std::string value);
00025     std::string getConversationScope() const;
00026     void setConversationKeyExpression(std::string value);
00027     std::string getConversationKeyExpression() const;
00028     void setActionMappings(std::string value);
00029     std::string getActionMappings() const;
00030     void setDelayExpression(std::string value);
00031     std::string getDelayExpression() const;
00032     void setDefaultOutputPort(unsigned int value);
00033     unsigned int getDefaultOutputPort() const;
00034     void setErrorOutputPort(unsigned int value);
00035     unsigned int getErrorOutputPort() const;
00036     void setAllowDirectOutputPort(bool value);
00037     bool isAllowDirectOutputPort() const;
00038     void setMinimumConfidence(double value);
00039     double getMinimumConfidence() const;
00040     void setSaveConfidenceAttribute(std::string value);
00041     std::string getSaveConfidenceAttribute() const;
00042     void setSaveSuccessAttribute(std::string value);
00043     std::string getSaveSuccessAttribute() const;
00044     std::string getLastPrompt() const;
00045     std::string getLastResponse() const;
00046     std::string getLastError() const;
00047     bool renderPrompt(std::string& result, std::string& error);
00048
00049     virtual std::string show() override;
00050
00051 protected:
00052     virtual bool _loadInstance(PersistenceRecord* fields) override;
00053     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00054     virtual bool _check(std::string& errorMessage) override;
00055     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00056
00057 private:
00058     struct ParsedResponse {
00059         bool valid = false;
00060         std::string action;
00061         int outputPort = -1;
00062         double confidence = 1.0;
00063         std::string explanation;
00064         std::map<std::string, double> values;
00065         std::string error;
00066     };
00067
00068     AISupport* _resolveSupport();
00069     bool _renderPrompt(std::string& result, std::string& error);
00070     std::string _conversationKey(Entity* entity, std::string& error);
00071     ParsedResponse _parseResponse(const std::string& content) const;
00072     std::map<std::string, unsigned int> _parseActionMappings() const;
00073     unsigned int _selectPort(const ParsedResponse& response, bool& success) const;
00074     void _forward(Entity* entity, unsigned int port, double delay);
00075     void _fail(Entity* entity, const std::string& message);
00076
00077     const struct DEFAULT_VALUES {
00078         std::string aiSupportName = "";
00079         std::string promptTemplate = "Decide the route for this entity. Return only JSON with action,
confidence, explanation, and numeric values fields.";
00080         std::string conversationScope = "PerEntity";
00081         std::string conversationKeyExpression = "Entity.Id";
00082         std::string actionMappings = "route:0";
00083         std::string delayExpression = "0";
00084         unsigned int defaultOutputPort = 0;
00085         unsigned int errorOutputPort = 0;
00086         bool allowDirectOutputPort = false;
00087         double minimumConfidence = 0.0;
00088         std::string saveConfidenceAttribute = "";

```

```

00089         std::string saveSuccessAttribute = "";
00090     } DEFAULT;
00091
00092     AISupport* _aiSupport = nullptr;
00093     std::string _aiSupportName = DEFAULT.aiSupportName;
00094     std::string _promptTemplate = DEFAULT.promptTemplate;
00095     std::string _conversationScope = DEFAULT.conversationScope;
00096     std::string _conversationKeyExpression = DEFAULT.conversationKeyExpression;
00097     std::string _actionMappings = DEFAULT.actionMappings;
00098     std::string _delayExpression = DEFAULT.delayExpression;
00099     unsigned int _defaultOutputPort = DEFAULT.defaultOutputPort;
00100     unsigned int _errorOutputPort = DEFAULT.errorOutputPort;
00101     bool _allowDirectOutputPort = DEFAULT.allowDirectOutputPort;
00102     double _minimumConfidence = DEFAULT.minimumConfidence;
00103     std::string _saveConfidenceAttribute = DEFAULT.saveConfidenceAttribute;
00104     std::string _saveSuccessAttribute = DEFAULT.saveSuccessAttribute;
00105     std::string _lastPrompt;
00106     std::string _lastResponse;
00107     std::string _lastError;
00108 };
00109
00110 #endif

```

11.305 MarkovChain.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    MarkovChain.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 24 de Outubro de 2019, 18:26
00012  */
00013
00014 #ifndef MARKOVCHAIN_H
00015 #define MARKOVCHAIN_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "kernel/statistics/Sampler_if.h"
00019
00020 class MarkovChain : public ModelComponent {
00021 public: // constructors
00022     MarkovChain(Model* model, std::string name = "");
00023     virtual ~MarkovChain() override;
00024 public: // virtual
00025     virtual std::string show() override;
00026 public: // static
00027     static PluginInformation* GetPluginInformation();
00028     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00029     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00030 public: // get and set
00031     void setTransitionProbabilityMatrix(ModelDataDefinition* transitionMatrix);
00032     ModelDataDefinition* getTransitionMatrix() const;
00033     void setInitialDistribution(ModelDataDefinition* initialDistribution);
00034     ModelDataDefinition* getInitialDistribution() const;
00035     void setCurrentState(ModelDataDefinition* currentState);
00036     ModelDataDefinition* getCurrentState() const;
00037 protected: // virtual
00038     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00039     virtual bool _loadInstance(PersistenceRecord *fields) override;
00040     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00041 protected: // virtual
00042     virtual void _initBetweenReplications() override;
00043     virtual bool _check(std::string& errorMessage) override;
00044     // virtual void _createInternalAndAttachedData() override;
00045 protected:
00046     // virtual void _createInternalStatisticReporters() override;
00047     // virtual void _createNonEditableDataDefinitions() override;
00048     virtual void _createEditableDataDefinitions() override;
00049     // virtual void _createAttachedAttributes() override;
00050
00051 private: // methods
00052     unsigned int _drawNextState(Entity* entity, unsigned int currentState);
00053     double _readStateValue(Entity* entity) const;
00054     void _writeStateValue(Entity* entity, double value);
00055     double _readTransitionProbability(Entity* entity, unsigned int fromState, unsigned int toState)
00056         const;
00057     unsigned int _stateCount() const;
00058 private: // attributes 1:1

```

```

00058     ModelDataDefinition* _transitionProbMatrix = nullptr;
00059     ModelDataDefinition* _initialDistribution = nullptr;
00060     ModelDataDefinition* _currentState = nullptr;
00061 private: // attributes 1:n
00062     Sampler_if* _sampler = nullptr;
00063 };
00064
00065 #endif /* MARKOVCHAIN_H */

```

11.306 BacteriaColony.h

```

00001 /*
00002  * File:   BacteriaColony.h
00003  * Author: rlcancian
00004  *
00005  * Created on 17 de Abril de 2022
00006  */
00007
00008 #ifndef BACTERIACOLONY_H
00009 #define BACTERIACOLONY_H
00010
00011 #include "../kernel/simulator/model/ModelComponent.h"
00012 #include "kernel/simulator/Plugin.h"
00013 #include "kernel/util/Util.h"
00014 #include "plugins/data/BiochemicalSimulation/BioNetwork.h"
00015 #include "plugins/data/BiochemicalSimulation/GroProgram.h"
00016 #include "plugins/data/BiochemicalSimulation/GroProgramRuntime.h"
00017 #include "plugins/data/BiochemicalSimulation/BacteriaSignalGrid.h"
00018
00019 #include <cstdint>
00020 #include <map>
00021 #include <string>
00022 #include <vector>
00023
00032 class BacteriaColony : public ModelComponent {
00033 public:
00034     struct BarrierSegment {
00035         double x1 = 0.0;
00036         double y1 = 0.0;
00037         double x2 = 0.0;
00038         double y2 = 0.0;
00039     };
00040
00041     struct BacteriumState {
00042         unsigned int id = 0;
00043         unsigned int parentId = 0;
00044         unsigned int generation = 0;
00045         std::string programName = "";
00046         std::vector<double> programArguments;
00047         unsigned int divisionCount = 0;
00048         double birthTime = 0.0;
00049         double lastUpdateTime = 0.0;
00050         double lastDivisionTime = 0.0;
00051         double positionX = 0.0;
00052         double positionY = 0.0;
00053         double directionRadians = 0.0;
00054         double volume = 1.0;
00055         double size = 1.0;
00056         double gfp = 0.0;
00057         double rfp = 0.0;
00058         double yfp = 0.0;
00059         double cfp = 0.0;
00060         double speed = 0.1;
00061         unsigned int tickCount = 0;
00062         unsigned int gridX = 0;
00063         unsigned int gridY = 0;
00064         bool hasExplicitGridPosition = false;
00065         bool justDivided = false;
00066         bool daughter = false;
00067         bool alive = true;
00068         std::map<std::string, double> runtimeVariables;
00069     };
00070
00071 public:
00072     BacteriaColony(Model* model, std::string name = "");
00073     virtual ~BacteriaColony() override = default;
00074
00075 public:
00076     virtual std::string show() override;
00077
00078 public: // static plugin interface
00079     static PluginInformation* GetPluginInformation();
00080     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);

```

```

00081     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00082
00083 public:
00084     void setGroProgram(GroProgram* groProgram);
00085     GroProgram* getGroProgram() const;
00086     void setSimulationStep(double simulationStep);
00087     double getSimulationStep() const;
00088     void setNumSteps(unsigned int numSteps);
00089     unsigned int getNumSteps() const;
00090     void setColonyTimeUnit(Util::TimeUnit colonyTimeUnit);
00091     Util::TimeUnit getColonyTimeUnit() const;
00092     double getColonyTime() const;
00093     void setInitialPopulation(unsigned int initialPopulation);
00094     unsigned int getInitialPopulation() const;
00095     unsigned int getPopulationSize() const;
00096     void setBioNetwork(BioNetwork* bioNetwork);
00097     BioNetwork* getBioNetwork() const;
00098     void setSignalGrid(BacteriaSignalGrid* signalGrid);
00099     BacteriaSignalGrid* getSignalGrid() const;
00100     std::size_t getInternalBacteriaCount() const;
00101     const BacteriumState& getBacteriumState(std::size_t index) const;
00102     double getBacteriumAge(std::size_t index) const;
00103     double getBacteriumPositionX(std::size_t index) const;
00104     double getBacteriumPositionY(std::size_t index) const;
00105     double getBacteriumDirectionRadians(std::size_t index) const;
00106     double getBacteriumVolume(std::size_t index) const;
00107     double getBacteriumSize(std::size_t index) const;
00108     double getBacteriumGfp(std::size_t index) const;
00109     double getBacteriumRfp(std::size_t index) const;
00110     double getBacteriumRuntimeVariableValue(std::size_t index, const std::string& variableName) const;
00111     bool hasBacteriumRuntimeVariable(std::size_t index, const std::string& variableName) const;
00112     double getRuntimeVariableValue(const std::string& variableName) const;
00113     bool hasRuntimeVariable(const std::string& variableName) const;
00114     double getSignalValueAt(unsigned int x, unsigned int y) const;
00115     std::vector<std::vector<double>> getSignalMatrix() const;
00116     std::string getSignalMatrixDump(std::size_t maxRows = 0, std::size_t maxColumns = 0) const;
00117     double getBacteriumLocalSignal(std::size_t index) const;
00118     void setChemostatMode(bool enabled);
00119     bool getChemostatMode() const;
00120     void addBarrier(double x1, double y1, double x2, double y2);
00121     const std::vector<BarrierSegment>& getBarriers() const;
00122     const std::vector<double>& getMappedCellValues() const;
00123     const std::string& getMappedCellExpression() const;
00124     void setGridWidth(unsigned int gridWidth);
00125     unsigned int getGridWidth() const;
00126     void setGridHeight(unsigned int gridHeight);
00127     unsigned int getGridHeight() const;
00128     GroProgramRuntime::ExecutionResult executeGroProgram();
00129
00130 protected:
00131     virtual bool _loadInstance(PersistenceRecord* fields) override;
00132     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00133     virtual bool _check(std::string& errorMessage) override;
00134     virtual void _initBetweenReplications() override;
00135     // virtual void _createInternalAndAttachedData() override;
00136     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00137
00138 protected:
00139     // virtual void _createInternalStatisticReporters() override;
00140     // virtual void _createNonEditableDataDefinitions() override;
00141     // virtual void _createEditableDataDefinitions() override;
00142     virtual void _createAttachedAttributes() override;
00143
00144 private:
00145     const struct DEFAULT_VALUES {
00146     const std::string groProgramName = "";
00147     const std::string bioNetworkName = "";
00148     const std::string signalGridName = "";
00149     const double simulationStep = 0.01;
00150     const unsigned int numSteps = 100;
00151     const Util::TimeUnit colonyTimeUnit = Util::TimeUnit::second;
00152     const unsigned int initialPopulation = 1;
00153     const unsigned int gridWidth = 1;
00154     const unsigned int gridHeight = 1;
00155     } DEFAULT;
00156
00157     GroProgram* _groProgram = nullptr;
00158     BioNetwork* _bioNetwork = nullptr;
00159     BacteriaSignalGrid* _signalGrid = nullptr;
00160     double _simulationStep = DEFAULT.simulationStep;
00161     unsigned int _currentStep = 0;
00162     unsigned int _numSteps = DEFAULT.numSteps;
00163     Util::TimeUnit _colonyTimeUnit = DEFAULT.colonyTimeUnit;
00164     unsigned int _initialPopulation = DEFAULT.initialPopulation;
00165     unsigned int _populationSize = DEFAULT.initialPopulation;
00166     unsigned int _gridWidth = DEFAULT.gridWidth;

```



```

00213     unsigned int _gridHeight = DEFAULT.gridHeight;
00214     unsigned int _nextBacteriumId = 1;
00215     unsigned int _colonyTickCount = 0;
00216     bool _chemostatMode = false;
00217     std::vector<BacteriumState> _bacteria;
00218     std::map<std::string, double> _runtimeVariables;
00219     std::vector<BarrierSegment> _barriers;
00220     std::vector<double> _mappedCellValues;
00221     std::string _mappedCellExpression;
00222     std::vector<double> _signalField;
00223     struct GroSeedDefinition {
00224         unsigned int gridX = 0;
00225         unsigned int gridY = 0;
00226         std::string programName = "";
00227         std::vector<double> programArguments;
00228     };
00229     std::vector<GroSeedDefinition> _groSeedDefinitions;
00230     std::string _groSeedSignature = "";
00231     bool _groSeedsApplied = false;
00232
00233 private:
00234     // SignalGrid remains the authoritative source for spatial configuration
00235     // when attached; colony timing is always read from ModelSimulation.
00236     bool _usesSignalGridDimensions() const;
00237     void _synchronizeGridDimensionsFromSignalGrid();
00238     bool _resetRuntimeSignalField(std::string& errorMessage);
00239     bool _collectBioNetworkSpecies(std::vector<BioSpecies*>& species, std::string& errorMessage)
00240     const;
00241     bool _collectBioNetworkSpeciesByIdentifier(std::map<std::string, BioSpecies*>&
00242     speciesByIdentifier,
00243         std::string& errorMessage) const;
00244     void _appendBioNetworkContextVariables(GroProgramRuntimeState& runtimeState, std::string&
00245     errorMessage) const;
00246     bool _applyBioNetworkAssignments(const std::map<std::string, double>& assignedVariables,
00247     std::string& errorMessage);
00248     void _removeBioNetworkAssignmentVariables(std::map<std::string, double>& variables) const;
00249     bool _refreshRuntimeConfigurationFromGroProgram(std::string& errorMessage);
00250     void _ensureGroSeededBacteriaInitialized();
00251     void _rebuildInternalBacteriaFromGroSeeds();
00252     std::size_t _signalIndex(unsigned int x, unsigned int y) const;
00253     double _signalValueAt(unsigned int x, unsigned int y) const;
00254     void _setSignalValueAt(unsigned int x, unsigned int y, double value);
00255     void _addSignalAt(unsigned int x, unsigned int y, double value);
00256     std::vector<std::vector<double>> _buildSignalMatrix() const;
00257     std::string _buildSignalFieldDump(std::size_t maxRows = 0, std::size_t maxColumns = 0) const;
00258     void _captureSignalFieldSnapshot(GroProgramRuntime::ExecutionResult& result,
00259     std::size_t maxRows = 0,
00260     std::size_t maxColumns = 0) const;
00261     double _computeNeighborSignalSum(unsigned int x, unsigned int y) const;
00262     unsigned int _computeLocalBacteriaCount(unsigned int x, unsigned int y) const;
00263     void _applySignalFieldStep();
00264     void _applyBacteriumSignalMutations(const BacteriumState& bacterium,
00265     const std::vector<GroProgramRuntime::SignalMutation>&
00266     mutations);
00267     void _rebuildInternalBacteria(unsigned int populationSize);
00268     void _resizeInternalBacteria(unsigned int populationSize);
00269     void _applyRuntimePopulationMutations(const std::vector<GroProgramRuntime::PopulationMutation>&
00270     mutations,
00271     unsigned int finalPopulationSize);
00272     bool _applyColonyMutations(const std::vector<GroProgramRuntime::ColonyMutation>& mutations,
00273     GroProgramRuntime::ExecutionResult& result,
00274     bool allowStructureMutations,
00275     std::string& errorMessage);
00276     bool _executeSeededNamedGroPrograms(const GroProgramIr& ir, GroProgramRuntime::ExecutionResult&
00277     result);
00278     bool _executeBacteriumScopedGroProgram(const GroProgramIr& ir, GroProgramRuntime::ExecutionResult&
00279     result);
00280     bool _containsBacteriumScopedOnlyUnsupportedCommand(const std::vector<GroProgramIr::Command>&
00281     commands,
00282     std::string& unsupportedCommand) const;
00283     std::size_t _findBacteriumIndexById(unsigned int bacteriumId) const;
00284     GroProgramRuntimeState _createBacteriumRuntimeState(const BacteriumState& bacterium,
00285     std::size_t bacteriumIndex) const;
00286     void _applyBacteriumScopedPopulationMutations(unsigned int bacteriumId,
00287     unsigned int parentGeneration,
00288     const
00289     std::vector<GroProgramRuntime::PopulationMutation>& mutations,
00290     GroProgramRuntime::ExecutionResult& result);
00291     void _appendBacterium(unsigned int parentId = 0, unsigned int generation = 0,
00292     const std::string& programName = "");
00293     void _appendBacterium(const GroSeedDefinition& seedDefinition);
00294     void _removeBacteria(unsigned int amount);
00295     bool _removeBacteriumById(unsigned int bacteriumId);
00296     void _refreshBacteriaUpdateTime();
00297     void _applyBacteriumGrowth(BacteriumState& bacterium) const;
00298     void _rebuildBacteriaGridPositions();
00299     void _assignBacteriumGridPosition(BacteriumState& bacterium, std::size_t index) const;

```

```

00290     void _initializeBacteriumPhenotype(BacteriumState& bacterium, std::size_t index) const;
00291     void _syncBacteriumSpatialState(BacteriumState& bacterium, const GroProgramRuntimeState&
runtimeState) const;
00292     void _updateBacteriumSpatialMotion(BacteriumState& bacterium) const;
00293     void _setBacteriumPosition(BacteriumState& bacterium, double x, double y) const;
00294     double _clampPositionX(double value) const;
00295     double _clampPositionY(double value) const;
00296     void _bindProgramArguments(GroProgramRuntimeState& runtimeState,
                                const std::vector<std::string>& parameterNames,
00297                                const std::vector<double>& arguments) const;
00298 };
00299 };
00300
00301 #endif /* BACTERIACOLONY_H */

```

11.307 BioRunnerCommand.h

```

00001 /*
00002  * File:    BioRunnerCommand.h
00003  * Author:  rlcancian
00004  *
00005  * Component that executes BioSimulatorRunner commands when entities arrive.
00006  */
00007
00008 #ifndef BIORUNNERCOMMAND_H
00009 #define BIORUNNERCOMMAND_H
00010
00011 #include <string>
00012
00013 #include "../kernel/simulator/model/ModelComponent.h"
00014 #include "kernel/simulator/Plugin.h"
00015 #include "plugins/data/BiochemicalSimulation/BioSimulatorRunner.h"
00016
00024 class BioRunnerCommand : public ModelComponent {
00025 public:
00026     BioRunnerCommand(Model* model, std::string name = "");
00027     virtual ~BioRunnerCommand() override = default;
00028
00029 public:
00030     static PluginInformation* GetPluginInformation();
00031     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00032     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00033
00034 public:
00035     void setRunner(BioSimulatorRunner* runner);
00036     BioSimulatorRunner* getRunner() const;
00037     void setCommand(std::string command);
00038     std::string getCommand() const;
00039     void setLastSucceeded(bool lastSucceeded);
00040     bool getLastSucceeded() const;
00041     void setLastStatus(std::string lastStatus);
00042     std::string getLastStatus() const;
00043     void setLastMessage(std::string lastMessage);
00044     std::string getLastMessage() const;
00045
00046     virtual std::string show() override;
00047
00048 protected:
00049     virtual bool _loadInstance(PersistenceRecord* fields) override;
00050     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00051     virtual bool _check(std::string& errorMessage) override;
00052     // virtual void _createInternalAndAttachedData() override;
00053     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00054
00055
00056 protected:
00057     // virtual void _createInternalStatisticReporters() override;
00058     // virtual void _createNonEditableDataDefinitions() override;
00059     virtual void _createEditableDataDefinitions() override;
00060     // virtual void _createAttachedAttributes() override;
00061
00062 private:
00063     void _forwardEntity(Entity* entity);
00064
00065 private:
00066     const struct DEFAULT_VALUES {
00067         std::string runnerName = "";
00068         std::string command = "validateModel()";
00069         bool lastSucceeded = false;
00070         std::string lastStatus = "";
00071         std::string lastMessage = "";
00072     } DEFAULT;
00073
00074     BioSimulatorRunner* _runner = nullptr;

```

```

00075     std::string _command = DEFAULT.command;
00076     bool _lastSucceeded = DEFAULT.lastSucceeded;
00077     std::string _lastStatus = DEFAULT.lastStatus;
00078     std::string _lastMessage = DEFAULT.lastMessage;
00079 };
00080
00081 #endif /* BIORUNNERCOMMAND_H */

```

11.308 BioSimulate.h

```

00001 /*
00002  * File:    BioSimulate.h
00003  * Author:  rlcancian
00004  *
00005  * Component that executes BioNetwork simulation when entities arrive.
00006  */
00007
00008 #ifndef BIOSIMULATE_H
00009 #define BIOSIMULATE_H
00010
00011 #include <string>
00012
00013 #include "../kernel/simulator/model/ModelComponent.h"
00014 #include "kernel/simulator/Plugin.h"
00015 #include "plugins/data/BiochemicalSimulation/BioNetwork.h"
00016
00024 class BioSimulate : public ModelComponent {
00025 public:
00026     BioSimulate(Model* model, std::string name = "");
00027     virtual ~BioSimulate() override = default;
00028
00029 public:
00030     static PluginInformation* GetPluginInformation();
00031     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00032     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00033
00034 public:
00035     void setBioNetwork(BioNetwork* bioNetwork);
00036     BioNetwork* getBioNetwork() const;
00037     void setUseNetworkTimeWindow(bool useNetworkTimeWindow);
00038     bool getUseNetworkTimeWindow() const;
00039     void setStartTime(double startTime);
00040     double getStartTime() const;
00041     void setStopTime(double stopTime);
00042     double getStopTime() const;
00043     void setStepSize(double stepSize);
00044     double getStepSize() const;
00045     void setLastSucceeded(bool lastSucceeded);
00046     bool getLastSucceeded() const;
00047     void setLastMessage(std::string lastMessage);
00048     std::string getLastMessage() const;
00049
00050     virtual std::string show() override;
00051
00052 protected:
00053     virtual bool _loadInstance(PersistenceRecord* fields) override;
00054     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00055     virtual bool _check(std::string& errorMessage) override;
00056     // virtual void _createInternalAndAttachedData() override;
00057     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00058
00059
00060 protected:
00061     // virtual void _createInternalStatisticReporters() override;
00062     // virtual void _createNonEditableDataDefinitions() override;
00063     virtual void _createEditableDataDefinitions() override;
00064     // virtual void _createAttachedAttributes() override;
00065
00066 private:
00067     void _forwardEntity(Entity* entity);
00068
00069 private:
00070     const struct DEFAULT_VALUES {
00071         std::string bioNetworkName = "";
00072         bool useNetworkTimeWindow = true;
00073         double startTime = 0.0;
00074         double stopTime = 1.0;
00075         double stepSize = 0.1;
00076         bool lastSucceeded = false;
00077         std::string lastMessage = "";
00078     } DEFAULT;
00079
00080     BioNetwork* _bioNetwork = nullptr;

```

```

00081     bool _useNetworkTimeWindow = DEFAULT.useNetworkTimeWindow;
00082     double _startTime = DEFAULT.startTime;
00083     double _stopTime = DEFAULT.stopTime;
00084     double _stepSize = DEFAULT.stepSize;
00085     bool _lastSucceeded = DEFAULT.lastSucceeded;
00086     std::string _lastMessage = DEFAULT.lastMessage;
00087 };
00088
00089 #endif /* BIOSIMULATE_H */

```

11.309 BioSteadyState.h

```

00001 /*
00002  * File:    BioSteadyState.h
00003  * Author:  rlcancian
00004  *
00005  * Component that evaluates BioNetwork steady-state status.
00006  */
00007
00008 #ifndef BIOSTEADYSTATE_H
00009 #define BIOSTEADYSTATE_H
00010
00011 #include <string>
00012
00013 #include "../kernel/simulator/model/ModelComponent.h"
00014 #include "kernel/simulator/Plugin.h"
00015 #include "plugins/data/BiochemicalSimulation/BioNetwork.h"
00016
00023 class BioSteadyState : public ModelComponent {
00024 public:
00025     BioSteadyState(Model* model, std::string name = "");
00026     virtual ~BioSteadyState() override = default;
00027
00028 public:
00029     static PluginInformation* GetPluginInformation();
00030     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00031     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00032
00033 public:
00034     void setBioNetwork(BioNetwork* bioNetwork);
00035     BioNetwork* getBioNetwork() const;
00036     void setTolerance(double tolerance);
00037     double getTolerance() const;
00038     void setRunSimulationBeforeCheck(bool runSimulationBeforeCheck);
00039     bool getRunSimulationBeforeCheck() const;
00040     void setLastSteady(bool lastSteady);
00041     bool getLastSteady() const;
00042     void setLastMaxAbsoluteDerivative(double lastMaxAbsoluteDerivative);
00043     double getLastMaxAbsoluteDerivative() const;
00044     void setLastMessage(std::string lastMessage);
00045     std::string getLastMessage() const;
00046
00047     virtual std::string show() override;
00048
00049 protected:
00050     virtual bool _loadInstance(PersistenceRecord* fields) override;
00051     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00052     virtual bool _check(std::string& errorMessage) override;
00053     // virtual void _createInternalAndAttachedData() override;
00054     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00055
00056
00057 protected:
00058     // virtual void _createInternalStatisticReporters() override;
00059     // virtual void _createNonEditableDataDefinitions() override;
00060     // virtual void _createEditableDataDefinitions() override;
00061     virtual void _createAttachedAttributes() override;
00062
00063 private:
00064     void _forwardEntity(Entity* entity);
00065
00066 private:
00067     const struct DEFAULT_VALUES {
00068         std::string bioNetworkName = "";
00069         double tolerance = 1e-6;
00070         bool runSimulationBeforeCheck = false;
00071         bool lastSteady = false;
00072         double lastMaxAbsoluteDerivative = 0.0;
00073         std::string lastMessage = "";
00074     } DEFAULT;
00075
00076     BioNetwork* _bioNetwork = nullptr;
00077     double _tolerance = DEFAULT.tolerance;

```

```

00078     bool _runSimulationBeforeCheck = DEFAULT.runSimulationBeforeCheck;
00079     bool _lastSteady = DEFAULT.lastSteady;
00080     double _lastMaxAbsoluteDerivative = DEFAULT.lastMaxAbsoluteDerivative;
00081     std::string _lastMessage = DEFAULT.lastMessage;
00082 };
00083
00084 #endif /* BIOSTEADYSTATE_H */

```

11.310 GeneticCircuitSimulate.h

```

00001 #ifndef GENETICCIRCUITSIMULATE_H
00002 #define GENETICCIRCUITSIMULATE_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/BiochemicalSimulation/GeneticCircuit.h"
00009
00010 class GeneticCircuitPart;
00011
00012 struct GeneticCircuitSimulationSummary {
00013     bool succeeded = false;
00014     unsigned int sampleCount = 0;
00015     double totalExpression = 0.0;
00016     std::string message = "";
00017 };
00018
00022 class GeneticCircuitSimulate : public ModelComponent {
00023 public:
00024     GeneticCircuitSimulate(Model* model, std::string name = "");
00025     virtual ~GeneticCircuitSimulate() override = default;
00026
00027 public:
00028     static PluginInformation* GetPluginInformation();
00029     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00030     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00031
00032 public:
00033     void setGeneticCircuit(GeneticCircuit* geneticCircuit);
00034     GeneticCircuit* getGeneticCircuit() const;
00035     void setStartTime(double startTime);
00036     double getStartTime() const;
00037     void setStopTime(double stopTime);
00038     double getStopTime() const;
00039     void setStepSize(double stepSize);
00040     double getStepSize() const;
00041     void setApplyRegulation(bool applyRegulation);
00042     bool getApplyRegulation() const;
00043     void setLastSucceeded(bool lastSucceeded);
00044     bool getLastSucceeded() const;
00045     void setLastSampleCount(unsigned int lastSampleCount);
00046     unsigned int getLastSampleCount() const;
00047     void setLastTotalExpression(double lastTotalExpression);
00048     double getLastTotalExpression() const;
00049     void setLastMessage(std::string lastMessage);
00050     std::string getLastMessage() const;
00051     static bool simulateCircuit(Model* model, GeneticCircuit* geneticCircuit, double startTime, double
stopTime,
00052                                double stepSize, bool applyRegulation,
GeneticCircuitSimulationSummary* summary,
00053                                std::string& errorMessage);
00054     bool simulate(std::string& errorMessage);
00055
00056     virtual std::string show() override;
00057
00058 protected:
00059     virtual bool _loadInstance(PersistenceRecord* fields) override;
00060     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00061     virtual bool _check(std::string& errorMessage) override;
00062     // virtual void _createInternalAndAttachedData() override;
00063     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00064
00065 protected:
00066     // virtual void _createInternalStatisticReporters() override;
00067     // virtual void _createNonEditableDataDefinitions() override;
00068     virtual void _createEditableDataDefinitions() override;
00069     // virtual void _createAttachedAttributes() override;
00070
00071 private:
00072     double _computeRegulationMultiplier(const GeneticCircuitPart* part) const;
00073     void _executeStep(double timeStep);
00074     void _forwardEntity(Entity* entity);

```

```

00075
00076 private:
00077     const struct DEFAULT_VALUES {
00078         std::string geneticCircuitName = "";
00079         double startTime = 0.0;
00080         double stopTime = 10.0;
00081         double stepSize = 1.0;
00082         bool applyRegulation = true;
00083         bool lastSucceeded = false;
00084         unsigned int lastSampleCount = 0;
00085         double lastTotalExpression = 0.0;
00086         std::string lastMessage = "";
00087     } DEFAULT;
00088
00089     GeneticCircuit* _geneticCircuit = nullptr;
00090     double _startTime = DEFAULT.startTime;
00091     double _stopTime = DEFAULT.stopTime;
00092     double _stepSize = DEFAULT.stepSize;
00093     bool _applyRegulation = DEFAULT.applyRegulation;
00094     bool _lastSucceeded = DEFAULT.lastSucceeded;
00095     unsigned int _lastSampleCount = DEFAULT.lastSampleCount;
00096     double _lastTotalExpression = DEFAULT.lastTotalExpression;
00097     std::string _lastMessage = DEFAULT.lastMessage;
00098 };
00099
00100 #endif /* GENETICCIRCUITSIMULATE_H */

```

11.311 GeneticExpressionStep.h

```

00001 #ifndef GENETICEXPRESSIONSTEP_H
00002 #define GENETICEXPRESSIONSTEP_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/BiochemicalSimulation/GeneticCircuit.h"
00009
00010 class GeneticCircuitPart;
00011
00012 class GeneticExpressionStep : public ModelComponent {
00013 public:
00014     GeneticExpressionStep(Model* model, std::string name = "");
00015     virtual ~GeneticExpressionStep() override = default;
00016
00017 public:
00018     static PluginInformation* GetPluginInformation();
00019     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00020     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00021
00022 public:
00023     void setGeneticCircuit(GeneticCircuit* geneticCircuit);
00024     GeneticCircuit* getGeneticCircuit() const;
00025     void setTimeStep(double timeStep);
00026     double getTimeStep() const;
00027     void setApplyRegulation(bool applyRegulation);
00028     bool getApplyRegulation() const;
00029     void setLastSucceeded(bool lastSucceeded);
00030     bool getLastSucceeded() const;
00031     void setLastTotalExpression(double lastTotalExpression);
00032     double getLastTotalExpression() const;
00033     void setLastMessage(std::string lastMessage);
00034     std::string getLastMessage() const;
00035
00036     virtual std::string show() override;
00037
00038 protected:
00039     virtual bool _loadInstance(PersistenceRecord* fields) override;
00040     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00041     virtual bool _check(std::string& errorMessage) override;
00042     // virtual void _createInternalAndAttachedData() override;
00043     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00044 protected:
00045     // virtual void _createInternalStatisticReporters() override;
00046     // virtual void _createNonEditableDataDefinitions() override;
00047     virtual void _createEditableDataDefinitions() override;
00048     // virtual void _createAttachedAttributes() override;
00049
00050 private:
00051     double _computeRegulationMultiplier(const GeneticCircuitPart* part) const;
00052     void _forwardEntity(Entity* entity);
00053 private:

```

```

00058     const struct DEFAULT_VALUES {
00059         std::string geneticCircuitName = "";
00060         double timeStep = 1.0;
00061         bool applyRegulation = true;
00062         bool lastSucceeded = false;
00063         double lastTotalExpression = 0.0;
00064         std::string lastMessage = "";
00065     } DEFAULT;
00066
00067     GeneticCircuit* _geneticCircuit = nullptr;
00068     double _timeStep = DEFAULT.timeStep;
00069     bool _applyRegulation = DEFAULT.applyRegulation;
00070     bool _lastSucceeded = DEFAULT.lastSucceeded;
00071     double _lastTotalExpression = DEFAULT.lastTotalExpression;
00072     std::string _lastMessage = DEFAULT.lastMessage;
00073 };
00074
00075 #endif /* GENETICEXPRESSIONSTEP_H */

```

11.312 MetabolicFluxBalance.h

```

00001 #ifndef METABOLICFLUXBALANCE_H
00002 #define METABOLICFLUXBALANCE_H
00003
00004 #include <map>
00005 #include <string>
00006
00007 #include "../kernel/simulator/model/ModelComponent.h"
00008 #include "kernel/simulator/Plugin.h"
00009 #include "plugins/data/BiochemicalSimulation/MetabolicNetwork.h"
00010
00038 class MetabolicFluxBalance : public ModelComponent {
00039 public:
00040     MetabolicFluxBalance(Model* model, std::string name = "");
00041     virtual ~MetabolicFluxBalance() override = default;
00042
00043 public:
00044     static PluginInformation* GetPluginInformation();
00045     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00046     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00047
00048 public:
00049     void setMetabolicNetwork(MetabolicNetwork* metabolicNetwork);
00050     MetabolicNetwork* getMetabolicNetwork() const;
00051     void setObjectiveReactionName(std::string objectiveReactionName);
00052     std::string getObjectiveReactionName() const;
00053     void setLastSucceeded(bool lastSucceeded);
00054     bool getLastSucceeded() const;
00055     void setLastObjectiveValue(double lastObjectiveValue);
00056     double getLastObjectiveValue() const;
00057     void setLastMessage(std::string lastMessage);
00058     std::string getLastMessage() const;
00059     const std::map<std::string, double>& getLastFluxes() const;
00060
00061     virtual std::string show() override;
00062
00063 protected:
00064     virtual bool _loadInstance(PersistenceRecord* fields) override;
00065     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00066     virtual bool _check(std::string& errorMessage) override;
00067     // virtual void _createInternalAndAttachedData() override;
00068     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00069
00070 protected:
00071     // virtual void _createInternalStatisticReporters() override;
00072     // virtual void _createNonEditableDataDefinitions() override;
00073     virtual void _createEditableDataDefinitions() override;
00074     // virtual void _createAttachedAttributes() override;
00075
00076 private:
00077     void _forwardEntity(Entity* entity);
00078
00079 private:
00080     const struct DEFAULT_VALUES {
00081         std::string metabolicNetworkName = "";
00082         std::string objectiveReactionName = "";
00083         bool lastSucceeded = false;
00084         double lastObjectiveValue = 0.0;
00085         std::string lastMessage = "";
00086     } DEFAULT;
00087
00088     MetabolicNetwork* _metabolicNetwork = nullptr;
00089     std::string _objectiveReactionName = DEFAULT.objectiveReactionName;

```

```

00090     bool _lastSucceeded = DEFAULT.lastSucceeded;
00091     double _lastObjectiveValue = DEFAULT.lastObjectiveValue;
00092     std::string _lastMessage = DEFAULT.lastMessage;
00093     std::map<std::string, double> _lastFluxes;
00094 };
00095
00096 #endif /* METABOLICFLUXBALANCE_H */

```

11.313 ContinuousSystemComponent.h

```

00001 /*
00002  * File:    ContinuousSystemComponent.h
00003  * Author:  GenESyS
00004  *
00005  * ModelComponent that drives continuous-time simulation by delegating
00006  * numerical integration to an ODESolver data definition.
00007  */
00008
00009 #ifndef CONTINUOUSSYSTEMCOMPONENT_H
00010 #define CONTINUOUSSYSTEMCOMPONENT_H
00011
00012 #include <string>
00013
00014 #include "kernel/simulator/model/ModelComponent.h"
00015 #include "plugins/data/Continuous/ODESolver.h"
00016
00026 class ContinuousSystemComponent : public ModelComponent {
00027 public: // constructors
00028     ContinuousSystemComponent(Model* model, std::string name = "");
00029     virtual ~ContinuousSystemComponent() override = default;
00030
00031 public: // virtual
00032     virtual std::string show() override;
00033
00034 public: // static -- required by the plugin framework
00035     static PluginInformation* GetPluginInformation();
00036     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00037     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00038     static void SaveInstance(PersistenceRecord* fields, ModelComponent* component);
00039     static bool Check(ModelComponent* component);
00040     static void CreateInternalData(ModelComponent* component);
00041     static void DispatchEvent(Event* event);
00042
00043 public: // getters & setters
00044     void setOdeSolver(ODESolver* solver);
00045     ODESolver* getOdeSolver() const;
00046     void setOdeSolverName(std::string name);
00047     std::string getOdeSolverName() const;
00048
00049 protected: // virtual -- must be overridden
00050     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00051     virtual bool _loadInstance(PersistenceRecord* fields) override;
00052     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00053     virtual bool _check(std::string& errorMessage) override;
00054     virtual void _createEditableDataDefinitions() override;
00055
00056 private:
00057     const struct DEFAULT_VALUES {
00058         const std::string odeSolverName = "";
00059     } DEFAULT;
00060
00061     ODESolver* _odeSolver = nullptr;
00062     std::string _odeSolverName = DEFAULT.odeSolverName;
00063 };
00064
00065 #endif /* CONTINUOUSSYSTEMCOMPONENT_H */

```

11.314 DiffEquations.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    DiffEquations.h
00009  * Author:  Prof. Rafael Luiz Cancian, Dr. Eng.
00010  *

```



```

00011  * Created on
00012  */
00013
00014 #pragma once
00015
00016 #include "../kernel/simulator/model/ModelComponent.h"
00017 #include "../data/ExternalIntegration/CppCompiler.h"
00018
00019 //extern "C" typedef void (*initBetweenReplications_t)(Model* model);
00020 extern "C" typedef void (*diffEquationsOnDispatchEvent_t)(double* stateVars, int n, double t0, double
tTarget, double step);
00021
00022 class DiffEquations : public ModelComponent {
00023 public:
00024     DiffEquations(Model* model, std::string name = "");
00025     virtual ~DiffEquations() override;
00026
00027 public:
00028     List<std::string> *getEquations() const;
00029     List<std::string> *getStateVariableNames() const;
00030     double getprecision() const;
00031     void setPrecision(double newPrecision);
00032     double getMaxSteps() const;
00033     void setMaxSteps(double newMaxSteps);
00034     double getfinalTime() const;
00035     void setFinalTime(double newFinalTime);
00036     List<double> *getvaluesOnFinalTime() const;
00037
00038 public:
00039     virtual std::string show() override;
00040
00041 public:
00042     static PluginInformation* GetPluginInformation();
00043     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00044     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00045
00046 protected:
00047     virtual bool _loadInstance(PersistenceRecord *fields) override;
00048     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00049     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00050
00051 protected:
00052     virtual bool _check(std::string& errorMessage) override;
00053     // virtual ParserChangesInformation* _getParserChangesInformation();
00054     // virtual void _initBetweenReplications();
00055     // virtual void _addSimulationControl(SimulationControl* property);
00056
00057 protected:
00058     // virtual void _createInternalStatisticReporters() override;
00059     // virtual void _createNonEditableDataDefinitions() override;
00060     virtual void _createEditableDataDefinitions() override;
00061     // virtual void _createAttachedAttributes() override;
00062
00063 protected:
00064
00065 private:
00066     List<std::string> *_equations = new List<std::string>();
00067     List<std::string> *_stateVariableNames = new List<std::string>();
00068     List<double> *_valuesOnFinalTime = new List<double>();
00069     // pointers to functions on a shared library
00070     diffEquationsOnDispatchEvent_t dispatchEvent_SharedLibHandler;
00071     //initBetweenReplications_t initBetweenReplications_SharedLibHandler;
00072     // ...
00073
00074 private:
00075     // Default values for the attributes. Used on initing, loading and saving
00076     const struct DEFAULT_VALUES {
00077         const double precision = 1e-6;
00078         const double maxSteps = 1e3;
00079         const double finalTime = 1.0;
00080     } DEFAULT;
00081     double _precision = DEFAULT.precision;
00082     double _maxSteps = DEFAULT.maxSteps;
00083     double _finalTime = DEFAULT.finalTime;
00084
00085 private:
00086     CppCompiler *_cppCompiler = nullptr;
00087
00088 private:
00089 };

```

11.315 DiffusionSimulate.h

```

00001  /*
00002  * File:    DiffusionSimulate.h
00003  * Author:  GenESyS DCS (Tema 8.2)
00004  *
00005  * Component that advances a DiffusionField (continuous N-D diffusion) when
00006  * entities arrive, mirroring the BioSimulate -> BioNetwork pattern.
00007  */
00008
00009 #ifndef DIFFUSIONSIMULATE_H
00010 #define DIFFUSIONSIMULATE_H
00011
00012 #include <string>
00013
00014 #include "../kernel/simulator/model/ModelComponent.h"
00015 #include "kernel/simulator/Plugin.h"
00016 #include "plugins/data/Continuous/DiffusionField.h"
00017
00018 class DiffusionSimulate : public ModelComponent {
00019 public:
00020     DiffusionSimulate(Model* model, std::string name = "");
00021     virtual ~DiffusionSimulate() override = default;
00022
00023 public:
00024     static PluginInformation* GetPluginInformation();
00025     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00026     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00027
00028 public:
00029     void setDiffusionField(DiffusionField* diffusionField);
00030     DiffusionField* getDiffusionField() const;
00031     void setUseFieldTimeWindow(bool useFieldTimeWindow);
00032     bool getUseFieldTimeWindow() const;
00033     void setStartTime(double startTime);
00034     double getStartTime() const;
00035     void setStopTime(double stopTime);
00036     double getStopTime() const;
00037     void setStepSize(double stepSize);
00038     double getStepSize() const;
00039     void setLastSucceeded(bool lastSucceeded);
00040     bool getLastSucceeded() const;
00041     void setLastMessage(std::string lastMessage);
00042     std::string getLastMessage() const;
00043
00044     virtual std::string show() override;
00045
00046 protected:
00047     virtual bool _loadInstance(PersistenceRecord* fields) override;
00048     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00049     virtual bool _check(std::string& errorMessage) override;
00050     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00051
00052 protected:
00053     virtual void _createEditableDataDefinitions() override;
00054
00055 private:
00056     void _forwardEntity(Entity* entity);
00057
00058 private:
00059     const struct DEFAULT_VALUES {
00060         std::string diffusionFieldName = "";
00061         bool useFieldTimeWindow = true;
00062         double startTime = 0.0;
00063         double stopTime = 1.0;
00064         double stepSize = 0.1;
00065         bool lastSucceeded = false;
00066         std::string lastMessage = "";
00067     } DEFAULT;
00068
00069     DiffusionField* _diffusionField = nullptr;
00070     bool _useFieldTimeWindow = DEFAULT.useFieldTimeWindow;
00071     double _startTime = DEFAULT.startTime;
00072     double _stopTime = DEFAULT.stopTime;
00073     double _stepSize = DEFAULT.stepSize;
00074     bool _lastSucceeded = DEFAULT.lastSucceeded;
00075     std::string _lastMessage = DEFAULT.lastMessage;
00076 };
00077
00078 #endif /* DIFFUSIONSIMULATE_H */

```

11.316 LSODE.h

```

00001  /*

```

```

00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:   LSODE.h
00009  * Author: rlcancian
00010  *
00011  * Created on 22 de Outubro de 2019, 22:28
00012  */
00013
00014 #ifndef LSODE_H
00015 #define LSODE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../data/Logic/Formula.h"
00019 #include "../data/Logic/Variable.h"
00020
00024 class LSODE : public ModelComponent {
00025 public: // constructors
00026     LSODE(Model* model, std::string name = "");
00027     virtual ~LSODE() override;
00028 public: // virtual
00029     virtual std::string show() override;
00030 public: // static
00031     static PluginInformation* GetPluginInformation();
00032     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00033     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00034 public: // g&s
00035     void setTimeVariable(Variable* _timeVariable);
00036     Variable* getTimeVariable() const;
00037     void setStep(double _step);
00038     double getStep() const;
00039     void setVariable(Variable* _variables);
00040     Variable* getVariable() const;
00041     List<std::string>* getDiffEquations() const;
00042     void addDiffEquation(std::string newDiffEquation);
00043     void removeDiffEquation(std::string diffEquation);
00044     void setFilename(std::string filename);
00045     std::string getFileName() const;
00046 protected: // virtual
00047     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00048     virtual bool _loadInstance(PersistenceRecord *fields) override;
00049     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00050 protected: // virtual
00051     //virtual void _initBetweenReplications();
00052     // virtual void _createInternalStatisticReporters() override;
00053     // virtual void _createNonEditableDataDefinitions() override;
00054     // virtual void _createEditableDataDefinitions() override;
00055     // virtual void _createAttachedAttributes() override;
00056     virtual bool _check(std::string& errorMessage) override;
00057     //virtual void _createInternalAndAttachedData();
00058
00059 protected:
00060
00061 private: // methods
00062     bool _doStep();
00063 private: // attributes 1:1
00064
00065     const struct DEFAULT_VALUES {
00066         const std::string filename = "";
00067         const double step = 0.0;
00068     } DEFAULT;
00069     List<std::string>* _diffEquations = new List<std::string>();
00070     Variable* _variable = nullptr;
00071     Variable* _timeVariable = nullptr;
00072     std::string _filename = DEFAULT.filename;
00073     double _step = DEFAULT.step;
00074 private: // attributes 1:n
00075 };
00076
00077 #endif /* LSODE_H */

```

11.317 OLD_ODElement.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*

```

```

00008  * File:    ODE.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 13 de Junho de 2019, 19:12
00012  */
00013
00014 #ifndef OLD_ODEELEMENT_H
00015 #define OLD_ODEELEMENT_H
00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "kernel/simulator/model/ModelDataManager.h"
00019 #include "kernel/simulator/Plugin.h"
00020
00021 class ODEfunction {
00022 public:
00023     ODEfunction(std::string expression, double initialPoint, double initialValue) {
00024         this->expression = expression;
00025         this->initialPoint = initialPoint;
00026         this->initialValue = initialValue;
00027     }
00028     std::string expression;
00029     double initialPoint;
00030     double initialValue;
00031 };
00032
00033 class OLD_ODEElement : public ModelDataDefinition {
00034 public:
00035     OLD_ODEElement(Model* model, std::string name = "");
00036     virtual ~OLD_ODEElement() = default;
00037 public:
00038     virtual std::string show() override;
00039 public:
00040     static PluginInformation* GetPluginInformation();
00041     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00042     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00043 public:
00044     double solve();
00045     void setStepH(double _h);
00046     double getStepH() const;
00047     void setEndTime(double _endTime);
00048     double getEndTime() const;
00049     List<ODEfunction*>* getODEfunctions() const;
00050     void addODEfunction(ODEfunction* newFunction);
00051     void removeODEfunction(ODEfunction* function);
00052 protected: // must be overridden
00053     virtual bool _loadInstance(PersistenceRecord *fields) override;
00054     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00055     virtual bool _check(std::string& errorMessage) override;
00056
00057 protected:
00058     // virtual void _createInternalStatisticReporters() override;
00059     // virtual void _createNonEditableDataDefinitions() override;
00060     // virtual void _createEditableDataDefinitions() override;
00061     // virtual void _createAttachedAttributes() override;
00062
00063 private:
00064
00065 private:
00066     List<ODEfunction*>* _ODEfunctions = new List<ODEfunction*>();
00067     double _stepH = 0.1;
00068     double _endTime;
00069 };
00070
00071 #endif /* OLD_ODEELEMENT_H */

```

11.318 Decide.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:    Decide.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 9 de Agosto de 2018, 20:39
00012  */
00013
00014 #ifndef DECIDE_H
00015 #define DECIDE_H
00016

```

```

00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../kernel/simulator/essentialPlugins/Counter.h"
00019
00074 class Decide : public ModelComponent {
00075 public:
00076     Decide(Model* model, std::string name = "");
00077     virtual ~Decide() override;
00078 public:
00079     List<std::string>* getConditions() const;
00080     void addConditions(std::string newCondition);
00081     void removeConditions(std::string condition);
00082 public:
00083     virtual std::string show() override;
00084 public:
00085     static PluginInformation* GetPluginInformation();
00086     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00087     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00088 protected:
00089     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00090     virtual bool _loadInstance(PersistenceRecord *fields) override;
00091     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00092 protected:
00093     //virtual void _initBetweenReplications();
00094     virtual bool _check(std::string& errorMessage) override;
00095 protected:
00096     virtual void _createInternalStatisticReporters() override;
00097     // virtual void _createNonEditableDataDefinitions() override;
00098     // virtual void _createEditableDataDefinitions() override;
00099     // virtual void _createAttachedAttributes() override;
00100
00101 protected:
00102
00103 private:
00104     List<std::string>* _conditions = new List<std::string>();
00105     List<Counter*>* _numberOuts = nullptr;
00106 private:
00107
00108 };
00109
00110 #endif /* DECIDE_H */

```

11.319 DropOff.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: DropOff.h
00009  * Author: rlcancian
00010  *
00011  * Created on 03 de Junho de 2019, 15:15
00012  */
00013
00014 #ifndef DROPOFF_H
00015 #define DROPOFF_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00041 class DropOff : public ModelComponent {
00042 public: // constructors
00043     DropOff(Model* model, std::string name = "");
00044     virtual ~DropOff() = default;
00045 public: // virtual
00046     virtual std::string show() override;
00047 public: // static
00048     static PluginInformation* GetPluginInformation();
00049     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00050     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00051 protected: // virtual
00052     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00053     virtual bool _loadInstance(PersistenceRecord *fields) override;
00054 protected: // virtual
00055     //virtual void _initBetweenReplications();
00056     // virtual void _createInternalStatisticReporters() override;
00057     // virtual void _createNonEditableDataDefinitions() override;
00058     // virtual void _createEditableDataDefinitions() override;
00059     // virtual void _createAttachedAttributes() override;
00060     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00061     virtual bool _check(std::string& errorMessage) override;
00062

```

```

00063 protected:
00064
00065 private: // methods
00066 private: // attributes 1:1
00067 private: // attributes 1:n
00068 };
00069
00070
00071 #endif /* DROPOFF_H */

```

11.320 Pickup.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Pickup.h
00009  * Author: rlcancian
00010  *
00011  * Created on 03 de Junho de 2019, 15:15
00012  */
00013
00014 #ifndef PICKUP_H
00015 #define PICKUP_H
00016
00017 #include "plugins/components/Decisions/Remove.h"
00018
00019 class Pickup : public Remove {
00020 public: // constructors
00021     Pickup(Model* model, std::string name = "");
00022     virtual ~Pickup() = default;
00023 public: // static
00024     static PluginInformation* GetPluginInformation();
00025     //static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00026     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00027 };
00028
00029 #endif /* PICKUP_H */
00030

```

11.321 Remove.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Remove.h
00009  * Author: rlcancian
00010  *
00011  * Created on 03 de Junho de 2019, 15:20
00012  */
00013
00014 #ifndef REMOVE_H
00015 #define REMOVE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00019 class Remove : public ModelComponent {
00020 public:
00021     enum class RemoveFromType : int {
00022         QUEUE = 0, ENTITYGROUP = 1, num_elements = 2
00023     };
00024 public:
00025     static std::string convertEnumToStr(RemoveFromType type);
00026 public: // constructors
00027     Remove(Model* model, std::string name = "");
00028     virtual ~Remove() = default;
00029 public: // virtual
00030     virtual std::string show() override;
00031 public:
00032     void setRemoveStartRank(std::string _removeFromRank);

```

```

00052     std::string getRemoveStartRank() const;
00053     void setRemoveEndRank(std::string _removeEndRank);
00054     std::string getRemoveEndRank() const;
00055     void setRemoveFromType(Remove::RemoveFromType _removeFromType);
00056     Remove::RemoveFromType getRemoveFromType() const;
00057     void setRemoveFrom(ModelDataDefinition* _removeFrom);
00058     ModelDataDefinition* getRemoveFrom() const;
00059
00060 public: // static
00061     static PluginInformation* GetPluginInformation();
00062     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00063     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00064 protected: // must be overridden
00065     virtual bool _loadInstance(PersistenceRecord *fields) override;
00066     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00067     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00068 protected: // could be overridden by derived classes
00069     virtual bool _check(std::string& errorMessage) override;
00071     //virtual ParserChangesInformation* _getParserChangesInformation();
00072     //virtual void _initBetweenReplications();
00073 protected:
00074     // virtual void _createInternalStatisticReporters() override;
00075     // virtual void _createNonEditableDataDefinitions() override;
00076     virtual void _createEditableDataDefinitions() override;
00077     // virtual void _createAttachedAttributes() override;
00079     // virtual void _createInternalAndAttachedData() override;
00080     //virtual void _addSimulationControl(SimulationControl* property);
00081
00082 protected:
00083
00084 private: // methods
00085 private: // attributes 1:1
00086
00087     const struct DEFAULT_VALUES {
00088         const Remove::RemoveFromType removeFromType = Remove::RemoveFromType::QUEUE;
00089         const std::string removeStartRank = "";
00090         const std::string removeEndRank = "";
00091     } DEFAULT;
00092     ModelDataDefinition* _removeFrom = nullptr;
00093     Remove::RemoveFromType _removeFromType = DEFAULT.removeFromType;
00094     std::string _removeStartRank = DEFAULT.removeStartRank;
00095     std::string _removeEndRank = DEFAULT.removeStartRank;
00096     //std::string _removeFromRank = DEFAULT.removeFromRank;
00097
00098 private: // attributes 1:n
00099 };
00100
00101
00102 #endif /* REMOVE_H */

```

11.322 Search.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Search.h
00009  * Author: rlcancian
00010  *
00011  * Created on 03 de Junho de 2019, 15:20
00012  */
00013
00014 #ifndef SEARCH_H
00015 #define SEARCH_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00055 class Search : public ModelComponent {
00056 public:
00057     enum class SearchInType : int {
00058         QUEUE = 0, ENTITYGROUP = 1, num_elements = 2
00059     };
00060 public:
00061     static std::string convertEnumToStr(SearchInType type);
00062 public: // constructors
00063     Search(Model* model, std::string name = "");
00064     virtual ~Search() = default;
00065 public: // virtual
00066     virtual std::string show() override;
00067 public:

```

```

00068     void setSaveFounRankAttribute(std::string _saveFounRankAttribute);
00069     std::string getSaveFounRankAttribute() const;
00070     void setSearchCondition(std::string _searchCondition);
00071     std::string getSearchCondition() const;
00072     void setEndRank(std::string _endRank);
00073     std::string getEndRank() const;
00074     void setStartRank(std::string _startRank);
00075     std::string getStartRank() const;
00076     void setSearchInName(std::string searchInName);
00077     std::string getSearchInName() const;
00078     void setSearchIn(ModelDataDefinition* _searchIn);
00079     ModelDataDefinition* getSearchIn() const;
00080     void setSearchInType(Search::SearchInType _searchInType);
00081     Search::SearchInType getSearchInType() const;
00082 public: // static
00083     static PluginInformation* GetPluginInformation();
00084     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00085     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00086
00087 protected: // must be overridden
00088     virtual bool _loadInstance(PersistenceRecord *fields) override;
00089     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00090     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00091 protected: // could be overridden by derived classes
00092     virtual bool _check(std::string& errorMessage) override;
00093     //virtual ParserChangesInformation* _getParserChangesInformation();
00094     //virtual void _initBetweenReplications();
00095
00096 protected:
00097     // virtual void _createInternalStatisticReporters() override;
00098     // virtual void _createNonEditableDataDefinitions() override;
00099     virtual void _createEditableDataDefinitions() override;
00100     // virtual void _createAttachedAttributes() override;
00101     // virtual void _createInternalAndAttachedData() override;
00102     //virtual void _addSimulationControl(SimulationControl* property);
00103
00104
00105 protected:
00106
00107 private: // methods
00108 private: // attributes 1:1
00109
00110     const struct DEFAULT_VALUES {
00111         const SearchInType searchInType = SearchInType::QUEUE;
00112         const std::string startRank = "";
00113         const std::string endRank = "";
00114         const std::string searchCondition = "";
00115         const std::string saveFounRankAttribute = "";
00116     } DEFAULT;
00117     ModelDataDefinition* _searchIn = nullptr;
00118     Search::SearchInType _searchInType = DEFAULT.searchInType;
00119     std::string _startRank = DEFAULT.startRank;
00120     std::string _endRank = DEFAULT.endRank;
00121     std::string _searchCondition = DEFAULT.searchCondition;
00122     std::string _saveFounRankAttribute = DEFAULT.saveFounRankAttribute;
00123
00124 private: // attributes 1:n
00125 };
00126
00127
00128 #endif /* SEARCH_H */

```

11.323 QueueableItem.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: QueueableItem.h
00009  * Author: rlcancian
00010  *
00011  * Created on 23 de abril de 2021, 15:09
00012  */
00013
00014 #ifndef QUEUEABLEITEM_H
00015 #define QUEUEABLEITEM_H
00016
00017 #include "plugins/data/DiscreteProcessing/Queue.h"
00018 #include "../data/Logic/Set.h"
00019 #include "../kernel/simulator/model/ModelDataManager.h"
00020
00021 // @TODO should inherit from a common base to SeizeableItem

```



```

00022
00023 class QueueableItem {
00024 public:
00025
00026     enum class QueueableType : int {
00027         QUEUE = 0, SET = 1, num_elements = 2 //, HOLD = 2, num_elements = 3
00028     };
00029 public:
00030     static std::string convertEnumToStr(QueueableType type);
00031 public:
00032     QueueableItem(ModelDataDefinition* queueOrSet, QueueableItem::QueueableType queueableType =
QueueableItem::QueueableType::QUEUE, std::string index = "0");
00033     QueueableItem(Model* model, std::string queueName);
00034 public:
00035     bool loadInstance(PersistenceRecord *fields);
00036     void saveInstance(PersistenceRecord *fields, bool saveDefaultValues);
00037 public:
00038     std::string show();
00039     void setIndex(std::string index);
00040     std::string getIndex() const;
00041     std::string getQueueableName() const;
00042     std::string getName() const;
00043     void setQueue(Queue* resource);
00044     Queue* getQueue() const;
00045     void setSet(Set* set);
00046     Set* getSet() const;
00047     //
00048     void setQueueableType(QueueableType queueableType);
00049     QueueableType getQueueableType() const;
00050     ModelDataDefinition* getQueueable() const;
00051     void setElementManager(ModelDataManager* _modelDataManager);
00052     //void setComponentManager(ModelComponentManager* _componentManager);
00053     List<SimulationControl*>* getSimulationControls() const;
00054     void _addSimulationControl(SimulationControl* control);
00055
00056     std::string getTypeDC() {return _typeDC;};
00057 private:
00058     void _ensureSimulationControls(Model* model);
00059
00060 private:
00061
00062     const struct DEFAULT_VALUES {
00063         const std::string quantityExpression = "1";
00064         const QueueableItem::QueueableType queueableType = QueueableItem::QueueableType::QUEUE;
00065         const std::string saveAttribute = "";
00066         const std::string index = "0";
00067     } DEFAULT;
00068
00069     QueueableType _queueableType = DEFAULT.queueableType;
00070     ModelDataDefinition* _queueOrSet = nullptr;
00071     std::string _queueableName;
00072     std::string _index = DEFAULT.index;
00073     std::string _typeDC = Util::TypeOf<Queue>();
00074 private:
00075     //ModelComponentManager* _componentManager;
00076     ModelDataManager* _modelDataManager = nullptr;
00077     List<SimulationControl*>* _simulationControls = new List<SimulationControl*>();
00078 };
00079
00080
00081 #endif /* QUEUEABLEITEM_H */

```

11.324 SeizableItem.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:      SeizableItem.h
00009  * Author:    rlcancian
00010  *
00011  * Created on 10 de abril de 2021, 08:45
00012  */
00013
00014 #ifndef SEIZABLEITEM_H
00015 #define SEIZABLEITEM_H
00016
00017 #include "plugins/data/DiscreteProcessing/Resource.h"
00018 #include "../data/Logic/Set.h"
00019 #include "../kernel/simulator/model/ModelDataManager.h"

```

```

00020
00021 // @TODO should inhene from a common base to QueueableItem
00022
00023 class SeizableItem {
00024 public:
00025     enum class SelectionRule : int {
00026         CYCLICAL = 0, RANDOM = 1, SPECIFICMEMBER = 2, LARGESTREMAININGCAPACITY = 3, SMALLESTNUMBERBUSY
00027         = 4,
00028         PREFEREDORDER = 5, num_elements = 6
00029     };
00030
00031     enum class SeizableType : int {
00032         RESOURCE = 0, SET = 1, num_elements = 2
00033     };
00034
00035     static std::string convertEnumToStr(SelectionRule rule);
00036     static std::string convertEnumToStr(SeizableType type);
00037 public:
00038     SeizableItem(ModelDataDefinition* resourceOrSet, std::string quantityExpression = "1",
00039                 SeizableItem::SelectionRule selectionRule =
00040                 SeizableItem::SelectionRule::LARGESTREMAININGCAPACITY,
00041                 std::string saveAttribute = "", std::string index = "0");
00042     SeizableItem(Model* model, std::string resourceName, std::string quantityExpression = "1",
00043                 SeizableItem::SelectionRule selectionRule =
00044                 SeizableItem::SelectionRule::LARGESTREMAININGCAPACITY,
00045                 std::string saveAttribute = "", std::string index = "0");
00046     SeizableItem(SeizableItem* original);
00047 public:
00048     bool loadInstance(PersistenceRecord* fields, unsigned int parentIndex);
00049     void saveInstance(PersistenceRecord* fields, unsigned int parentIndex, bool saveDefault);
00050     bool loadInstance(PersistenceRecord* fields);
00051     void saveInstance(PersistenceRecord* fields, bool saveDefaults);
00052 public:
00053     std::string show();
00054     void setIndex(std::string index);
00055     std::string getIndex() const;
00056     void setSaveAttribute(std::string saveAttribute);
00057     std::string getSaveAttribute() const;
00058     void setSelectionRule(SelectionRule selectionRule);
00059     SelectionRule getSelectionRule() const;
00060     void setQuantityExpression(std::string quantityExpression);
00061     std::string getQuantityExpression() const;
00062     std::string getResourceName() const;
00063     std::string getName() const;
00064     void setResource(Resource* resource);
00065     Resource* getResource() const;
00066     void setSet(Set* set);
00067     Set* getSet() const;
00068     //void setSeizableType(SeizableType resourceType);
00069     SeizableType getSeizableType() const;
00070     void setLastMemberSeized(unsigned int lastMemberSeized);
00071     unsigned int getLastMemberSeized() const;
00072     ModelDataDefinition* getSeizable() const;
00073     void setElementManager(ModelDataManager* modeldataManager);
00074     void setLastPreferedOrder(unsigned int lastPreferedOrder);
00075     unsigned int getLastPreferedOrder() const;
00076     //void setComponentManager(ModelComponentManager* _componentManager);
00077     List<SimulationControl>* getSimulationControls() const;
00078     void _addSimulationControl(SimulationControl* control);
00079
00080     std::string getTypeDC() { return _typeDC; };
00081
00082 private:
00083     void _ensureSimulationControls(Model* model);
00084
00085 private:
00086     const struct DEFAULT_VALUES {
00087         const std::string quantityExpression = "1";
00088         const SeizableItem::SeizableType seizableType = SeizableItem::SeizableType::RESOURCE;
00089         const SeizableItem::SelectionRule selectionRule =
00090         SeizableItem::SelectionRule::LARGESTREMAININGCAPACITY;
00091         const std::string saveAttribute = "";
00092         const std::string index = "0";
00093     } DEFAULT;
00094
00095     SeizableType _seizableType = DEFAULT.seizableType;
00096     SelectionRule _selectionRule = DEFAULT.selectionRule;
00097     std::string _saveAttribute = DEFAULT.saveAttribute;
00098     std::string _index = DEFAULT.index;
00099     ModelDataDefinition* _resourceOrSet = nullptr;
00100     std::string _seizableName;
00101     std::string _quantityExpression;
00102     unsigned int _lastMemberSeized = 0;
00103     unsigned int _lastPreferedOrder = 0;

```

```

00103     std::string _typeDC = Util::TypeOf<Resource>();
00104
00105 private:
00106     ModelDataManager* _modeldataManager;
00107     List<SimulationControl*>* _simulationControls = new List<SimulationControl*>();
00108 };
00109
00110 #endif /* SEIZABLEITEM_H */

```

11.325 Buffer.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Buffer.h
00009  * Author:  Prof. Rafael Luiz Cancian, Dr. Eng.
00010  *
00011  * Created on
00012  */
00013
00014 #ifndef BUFFER_H
00015 #define BUFFER_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../data/Synchronization/SignalData.h"
00019
00023 class Buffer : public ModelComponent {
00024 public:
00025     enum class AdvanceOn : int {
00026         NewArrivals = 0, Signal = 1, num_elements = 2
00027     };
00028     enum class ArrivalOnFullBufferRule : int {
00029         Dispose = 0, SendToBulkPort = 1, ReplaceLastPosition = 2, num_elements = 3
00030     };
00031     static std::string convertEnumToStr(AdvanceOn advance);
00032     static std::string convertEnumToStr(ArrivalOnFullBufferRule arrival);
00033 public: // constructors
00034     Buffer(Model* model, std::string name = "");
00035     virtual ~Buffer() override;
00036 public: // virtual
00037     virtual std::string show() override;
00038 public: // static
00039     static PluginInformation* GetPluginInformation();
00040     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00041     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00042 public:
00043     Buffer::ArrivalOnFullBufferRule getArrivalOnFullBufferRule() const;
00044     void setArrivalOnFullBufferRule(Buffer::ArrivalOnFullBufferRule newArrivalOnFullBufferRule);
00045     Buffer::AdvanceOn getAdvanceOn() const;
00046     void setAdvanceOn(Buffer::AdvanceOn newAdvanceOn);
00047     unsigned int getCapacity() const;
00048     void setCapacity(unsigned int newCapacity);
00049     SignalData* getSignal() const;
00050     void setSignal(SignalData* newSignal);
00051
00052 protected: // must be overridden
00053     virtual bool _loadInstance(PersistenceRecord* fields) override;
00054     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00055     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00056 protected: // could be overridden by derived classes
00057     virtual bool _check(std::string& errorMessage) override;
00058     virtual ParserChangesInformation* _getParserChangesInformation() override;
00059     virtual void _initBetweenReplications() override;
00060     // virtual void _createInternalAndAttachedData() override;
00061     virtual void _addSimulationControl(SimulationControl* property) override;
00062
00063 protected:
00064     // virtual void _createInternalStatisticReporters() override;
00065     // virtual void _createNonEditableDataDefinitions() override;
00066     virtual void _createEditableDataDefinitions() override;
00067     // virtual void _createAttachedAttributes() override;
00068
00069 private: // methods
00070     unsigned int _handlerForSignalDataEvent(SignalData* signalData);
00071     Entity* _advance(Entity* enteringEntity);
00072 private: // attributes 1:1
00073
00074     const struct DEFAULT_VALUES {
00075         const AdvanceOn advanceOn = AdvanceOn::NewArrivals;
00076     };
00077

```

```

00078         const ArrivalOnFullBufferRule arrivalOnFullBufferRule = ArrivalOnFullBufferRule::Dispose;
00079         const unsigned int capacity = 1;
00080     } DEFAULT;
00081     ArrivalOnFullBufferRule _arrivalOnFullBufferRule = DEFAULT.arrivalOnFullBufferRule;
00082     AdvanceOn _advanceOn = DEFAULT.advanceOn;
00083     unsigned int _capacity = DEFAULT.capacity;
00084 private:
00085     std::vector<Entity*>* _buffer = new std::vector<Entity*>;
00086 private: // attached
00087     SignalData* _attachedSignal = nullptr;
00088     SignalData* _signalWithRegisteredHandler = nullptr;
00089 };
00090
00091 #endif /* BUFFER_H */

```

11.326 Clone.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Clone.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 30 de Novembro de 2021, 18:50
00012  */
00013
00014 #ifndef CLONE_H
00015 #define CLONE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../kernel/simulator/essentialPlugins/Counter.h"
00019
00023 class Clone : public ModelComponent {
00024 public: // constructors
00025     Clone(Model* model, std::string name = "");
00026     virtual ~Clone() = default;
00027 public: // virtual
00028     virtual std::string show() override;
00029 public: //
00030     void setNumClonesExpression(std::string numClones);
00031     std::string getNumClonesExpression() const;
00032 public: // static
00033     static PluginInformation* GetPluginInformation();
00034     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00035     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00036 protected: // must be overridden
00037     virtual bool _loadInstance(PersistenceRecord *fields) override;
00038     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00039     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00040 protected: // could be overridden
00041     virtual bool _check(std::string& errorMessage) override;
00042     virtual void _initBetweenReplications() override;
00043     // virtual void _createInternalAndAttachedData() override;
00044     //virtual ParserChangesInformation* _getParserChangesInformation();
00045
00046 protected:
00047     virtual void _createInternalStatisticReporters() override;
00048     // virtual void _createNonEditableDataDefinitions() override;
00049     // virtual void _createEditableDataDefinitions() override;
00050     // virtual void _createAttachedAttributes() override;
00051
00052 private: // methods
00053 private: // attributes 1:1
00054
00055     const struct DEFAULT_VALUES {
00056         const std::string numClonesExpression = "1";
00057     } DEFAULT;
00058     std::string _numClonesExpression = DEFAULT.numClonesExpression;
00059     Counter* _counter = nullptr;
00060 private: // attributes 1:n
00061 };
00062
00063 #endif /* CLONE_H */

```

11.327 Delay.h

```

00001 /*

```

```

00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:    Delay.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 21 de Junho de 2018, 19:49
00012  */
00013
00014  #ifndef DELAY_H
00015  #define DELAY_H
00016
00017  #include <string>
00018  #include <vector>
00019  #include "../kernel/simulator/model/ModelComponent.h"
00020  #include "kernel/simulator/Plugin.h"
00021
00022  class Delay : public ModelComponent {
00023  public:
00024      Delay(Model* model, std::string name = "");
00025      virtual ~Delay() = default;
00026  public:
00027      //void setDelayExpression(std::string _delayExpression);
00028      void setDelayExpression(std::string _delayExpression, Util::TimeUnit
00029      _delayTimeUnit=Util::TimeUnit::unknown);
00030      std::string delayExpression() const;
00031      void setDelay(double delay);
00032      double delay() const;
00033      void setDelayTimeUnit(Util::TimeUnit _delayTimeUnit);
00034      Util::TimeUnit delayTimeUnit() const;
00035      void setAllocation(Util::AllocationType allocation);
00036      Util::AllocationType getAllocation() const;
00037  public:
00038      virtual std::string show() override;
00039  public:
00040      static PluginInformation* GetPluginInformation();
00041      static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00042      static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00043  //public: // properties
00044      //using property_owner_t = Delay;
00045      //decl_property(DelayExpression, decl_get(std::string){return delayExpression();} void
00046      decl_set(std::string val){setDelayExpression(val)});
00047
00048  protected:
00049      virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00050      virtual bool _loadInstance(PersistenceRecord *fields) override;
00051      virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00052  protected:
00053      //virtual void _initBetweenReplications();
00054      virtual bool _check(std::string& errorMessage) override;
00055  protected:
00056      virtual void _createInternalStatisticReporters() override;
00057      // virtual void _createNonEditableDataDefinitions() override;
00058      // virtual void _createEditableDataDefinitions() override;
00059      virtual void _createAttachedAttributes() override;
00060  public:
00061      const struct DEFAULT_VALUES {
00062          const std::string delayExpression = "1.0";
00063          const Util::TimeUnit delayTimeUnit = Util::TimeUnit::second;
00064          const Util::AllocationType allocation = Util::AllocationType::Wait;
00065      } DEFAULT;
00066
00067  protected:
00068
00069  private:
00070      std::string _delayExpression = DEFAULT.delayExpression;
00071      Util::TimeUnit _delayTimeUnit = DEFAULT.delayTimeUnit;
00072      Util::AllocationType _allocation = DEFAULT.allocation;
00073  private:
00074      std::vector<std::string> _allAllocationAttachedAttributeName() const;
00075      std::string _allocationAttachedAttributeName(Util::AllocationType allocation) const;
00076      void _reconcileAllocationAttachedAttributes();
00077      void _initCStats();
00078  private: // inner internal elements
00079      friend class DelayProbe;
00080      StatisticsCollector* _cstatWaitTime = nullptr;
00081  };
00082  //enable_this_owner(Delay, DelayExpression);
00083
00084  #endif /* DELAY_H */

```

11.328 Process.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Process.h
00009  * Author: rafael.lui.cancian
00010  *
00011  * Created on 22 de Maio de 2019, 18:41
00012  */
00013
00014 #ifndef PROCESS_H
00015 #define PROCESS_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "plugins/components/DiscreteProcessing/Seize.h"
00019 #include "plugins/components/DiscreteProcessing/Delay.h"
00020 #include "plugins/components/DiscreteProcessing/Release.h"
00021
00022 class Process : public ModelComponent {
00023 public: // constructors
00024     Process(Model* model, std::string name = "");
00025     virtual ~Process() = default;
00026 public: // virtual
00027     virtual std::string show() override;
00028 public: // static
00029     static PluginInformation* GetPluginInformation();
00030     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00031     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00032 public:
00033     void setPriority(unsigned short _priority);
00034     unsigned short getPriority() const;
00035     void setPriorityExpression(std::string priority);
00036     std::string getPriorityExpression() const;
00037     void setAllocationType(Util::AllocationType _allocationType);
00038     Util::AllocationType getAllocationType() const;
00039     List<SeizableItem*> getSeizeRequests() const;
00040     void addSeizeRequest(SeizableItem* newRequest);
00041     void removeSeizeRequest(SeizableItem* request);
00042     void setQueueableItem(QueueableItem* _queueableItem);
00043     QueueableItem* getQueueableItem() const;
00044     void setDelayExpression(std::string _delayExpression);
00045     void setDelayExpression(std::string _delayExpression, Util::TimeUnit _delayTimeUnit);
00046     std::string delayExpression() const;
00047     void setDelayTimeUnit(Util::TimeUnit _delayTimeUnit);
00048     Util::TimeUnit delayTimeUnit() const;
00049 protected: // virtual
00050     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00051     // virtual void _initBetweenReplications();
00052     virtual bool _loadInstance(PersistenceRecord* fields) override;
00053     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00054 protected: // virtual
00055     virtual bool _check(std::string& errorMessage) override;
00056 protected: // virtual
00057     // virtual void _createInternalStatisticReporters() override;
00058     virtual void _createNonEditableDataDefinitions() override;
00059     virtual void _createEditableDataDefinitions() override;
00060     // virtual void _createAttachedAttributes() override;
00061     // virtual void _createInternalAndAttachedData() override;
00062
00063 protected:
00064
00065 private: // methods
00066     void _adjustConnections();
00067     void _ensureInternalComponents();
00068     void _reconcileInternalComponents();
00069     bool _hasConsistentInternalChain(std::string& errorMessage) const;
00070 private: // attributes 1:1
00071     Seize* _seize = nullptr;
00072     Delay* _delay = nullptr;
00073     Release* _release = nullptr;
00074     bool _flagConstructing;
00075     //ModelComponent* _nextComponent = nullptr;
00076     //unsigned int _nextInput;
00077 private: // attributes 1:n
00078 };
00079
00080 #endif /* PROCESS_H */

```

11.329 Release.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Release.h
00009  * Author:  raphael.luiz.cancian
00010  *
00011  * Created on 21 de Agosto de 2018, 16:17
00012  */
00013
00014 #ifndef RELEASE_H
00015 #define RELEASE_H
00016
00017 #include <string>
00018
00019 #include "../kernel/simulator/model/ModelComponent.h"
00020 #include "kernel/simulator/Plugin.h"
00021 #include "auxiliar/SeizableItem.h"
00022 #include "plugins/data/DiscreteProcessing/Resource.h"
00023
00024 class Release : public ModelComponent {
00025 public:
00026     Release(Model* model, std::string name = "");
00027     virtual ~Release() override;
00028 public:
00029     virtual std::string show() override;
00030 public: //static
00031     static PluginInformation* GetPluginInformation();
00032     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00033     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00034 public: // get & set
00035     void setPriority(unsigned short _priority);
00036     unsigned short priority() const;
00037 public: // gets
00038     List<SeizableItem*>* getReleaseRequests() const;
00039     void addReleaseRequests(SeizableItem* newRequest);
00040     void removeReleaseRequests(SeizableItem* request);
00041
00042 protected:
00043     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00044     virtual bool _loadInstance(PersistenceRecord *fields) override;
00045     virtual void _initBetweenReplications() override;
00046     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00047     virtual bool _check(std::string& errorMessage) override;
00048     // virtual void _createInternalAndAttachedData() override;
00049
00050 protected:
00051     // virtual void _createInternalStatisticReporters() override;
00052     // virtual void _createNonEditableDataDefinitions() override;
00053     virtual void _createEditableDataDefinitions() override;
00054     // virtual void _createAttachedAttributes() override;
00055
00056 private:
00057     Resource* _getResourceFromSeizableItem(SeizableItem* seizable, Entity* entity);
00058 private:
00059
00060     const struct DEFAULT_VALUES {
00061         const unsigned short priority = 0;
00062         const unsigned int releaseRequestSize = 1;
00063     } DEFAULT;
00064     unsigned short _priority = DEFAULT.priority;
00065     List<SeizableItem*>* _releaseRequests = new List<SeizableItem*>();
00066 };
00067
00068 #endif /* RELEASE_H */

```

11.330 Seize.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Seize.h
00009  * Author:  raphael.luiz.cancian
00010  *

```

```

00011  * Created on 21 de Agosto de 2018, 16:17
00012  */
00013
00014 #ifndef SEIZE_H
00015 #define SEIZE_H
00016
00017 #include <string>
00018 #include "../kernel/simulator/model/ModelComponent.h"
00019 #include "../kernel/simulator/model/Model.h"
00020 #include "plugins/data/DiscreteProcessing/Resource.h"
00021 #include "plugins/data/DiscreteProcessing/Queue.h"
00022 #include "kernel/simulator/Plugin.h"
00023 #include "auxiliar/SeizableItem.h"
00024 #include "auxiliar/QueueableItem.h"
00025
00026 class WaitingResource : public Waiting {
00027 public:
00028
00029     WaitingResource(Entity* entity, double timeStartedWaiting, unsigned int quantity, ModelComponent*
        thisComponent, unsigned int thisComponentOutputPort = 0) : Waiting(entity, timeStartedWaiting,
        thisComponent, thisComponentOutputPort) {
00030         _quantity = quantity;
00031     }
00032
00033     WaitingResource(const WaitingResource& orig) : Waiting(orig) {
00034     }
00035
00036     virtual ~WaitingResource() = default;
00037 public:
00038
00039     virtual std::string show() override {
00040         return Waiting::show() +
00041             ",quantity=" + std::to_string(this->_quantity);
00042     }
00043 public:
00044
00045     unsigned int getQuantity() const {
00046         return _quantity;
00047     }
00048
00049 protected:
00050 private:
00051     unsigned int _quantity;
00052 };
00053
00054 class Seize : public ModelComponent {
00055 public:
00056     Seize(Model* model, std::string name = "");
00057     virtual ~Seize() override;
00058 public:
00059     virtual std::string show() override;
00060 public:
00061     static PluginInformation* GetPluginInformation();
00062     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00063     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00064 public: // get & set
00065     void setPriority(unsigned short _priority);
00066     unsigned short getPriority() const;
00067     void setAllocationType(Util::AllocationType _allocationType);
00068     Util::AllocationType getAllocationType() const;
00069     // indirect access to Queue* and Resource*
00070     //void setResourceName(std::string _resourceName) throw ();
00071     //std::string getResourceName() const;
00072     //void setQueueName(std::string queueName) throw ();
00073     //std::string getQueueName() const;
00074     void setQueue(Queue* queue);
00075     //Queue* getQueue() const;
00076     List<SeizableItem*> getSeizeRequests() const;
00077     void addRequest(SeizableItem* newRequest);
00078     void removeRequest(SeizableItem* request);
00079     void setQueueableItem(QueueableItem* _queueableItem);
00080     QueueableItem* getQueueableItem() const;
00081     void setPriorityExpression(std::string _priorityExpression);
00082     std::string getPriorityExpression() const;
00083 protected:
00084     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00085     virtual bool _loadInstance(PersistenceRecord *fields) override;
00086     virtual void _initBetweenReplications() override;
00087     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00088     virtual bool _check(std::string& errorMessage) override;
00089 protected:
00090     // virtual void _createInternalStatisticReporters() override;
00091     // virtual void _createNonEditableDataDefinitions() override;
00092     virtual void _createEditableDataDefinitions() override;
00093     // virtual void _createAttachedAttributes() override;
00094     // virtual void _createInternalAndAttachedData() override;
00095 private:

```



```

00170     void _handlerForResourceEvent(Resource* resource);
00171     Resource* _getResourceFromSeizableItem(SeizableItem* seizable, Entity* entity, unsigned
int*indexPtr);
00172     Queue* _getQueue() const;
00173 public:
00174
00175     const struct DEFAULT_VALUES {
00176         const Util::AllocationType allocationType = Util::AllocationType::Others;
00177         const unsigned short priority = 0;
00178         const std::string priorityExpression = "";
00179         const unsigned int seizeRequestSize = 1;
00180     } DEFAULT;
00181 private:
00182     Util::AllocationType _allocationType = DEFAULT.allocationType; // uint ? enum?
00183     unsigned short _priority = DEFAULT.priority;
00184     std::string _priorityExpression = DEFAULT.priorityExpression;
00185     QueueableItem* _queueableItem = nullptr; // usually has a queue, but not always (it could be a
hold or a set)
00186     List<SeizableItem*> _seizeRequests = new List<SeizableItem*>();
00187 };
00188
00189 #endif /* SEIZE_H */

```

11.331 SPICECircuit.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:     SPICECircuit.h
00009  * Author:   bnmfw & joaomai
00010  *
00011  * Created on 10 de outubro de 2023, 23:00
00012  */
00013
00014 #pragma once
00015
00016 #include "../kernel/simulator/model/ModelComponent.h"
00017 #include "../data/ExternalIntegration/SPICERunner.h"
00018 #include "plugins/components/ElectronicsSimulation/SPICENode.h"
00019
00023 class SPICECircuit : public ModelComponent {
00024 public:
00025     SPICECircuit(Model* model, std::string name = "");
00026     virtual ~SPICECircuit() = default;
00027
00028 public:
00029     void setRunner(SPICERunner* runner);
00030     SPICERunner* getRunner();
00031
00032 public:
00033     virtual std::string show() override;
00034
00035 public:
00036     static PluginInformation* GetPluginInformation();
00037     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00038     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00039     // Added
00040     static std::vector<std::string> split(std::string word, char split_char);
00041
00042 protected:
00043     void BuildCircuit(std::string description, unsigned int id, std::vector<std::string> the_params =
{""}, std::string the_model = "", std::string the_model_file = "");
00044     void UpdateConnections();
00045     virtual void Build() { id = counter++; build = [this]() {this->Build();}};
00046     virtual bool _loadInstance(PersistenceRecord *fields) override;
00047     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00048     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00049     // virtual void _createInternalAndAttachedData() override;
00050
00051
00052 protected:
00053     // virtual bool _check(std::string& errorMessage);
00054     // virtual ParserChangesInformation* _getParserChangesInformation();
00055     // virtual void _initBetweenReplications();
00056     // virtual void _createInternalStatisticReporters() override;
00057     // virtual void _createEditableDataDefinitions() override;
00058     virtual void _createNonEditableDataDefinitions() override;
00059     // virtual void _createAttachedAttributes() override;
00060     // virtual void _addSimulationControl(SimulationControl* property);

```

```

00065 protected:
00066
00067 private:
00068     // Spice subcircuit declaration, if one exists
00069     std::string spice_template;
00070     // Name of component
00071     std::string spice_name;
00072     // Params of the component, such as resistance or voltage
00073     std::vector<std::string> params;
00074     // Pin map of the component
00075     std::vector<std::string> pins;
00076     // Instance of the component in spice
00077     std::string *spice_instance;
00078     // Model of the component, if one exists (the model name should also be the name of the .cir file)
00079     std::string model;
00080     std::string model_file;
00081     // Spice compiler
00082     SPICERunner *compiler = nullptr;
00083
00084     static unsigned int counter;
00085     bool plain_circuit;
00086 protected:
00087     unsigned int id;
00088     std::function<void()> build;
00089
00090 private:
00091
00092     const struct DEFAULT_VALUES {
00093         const std::string someString = "Test";
00094         const unsigned int someUInt = 1;
00095     } DEFAULT;
00096     std::string _someString = DEFAULT.someString;
00097     unsigned int _someUInt = DEFAULT.someUInt;
00098
00099 private:
00100
00101 private:
00102     // ...
00103 };
00104
00105 class Resistor : public SPICECircuit {
00106 public:
00107     Resistor(Model* model, std::string name = ""): SPICECircuit(model, name) {
00108         id = counter++;
00109         build = [this]() {this->Build();};
00110     }
00111     void Build() override;
00112     void setResistance(float resistance);
00113
00114     static PluginInformation* GetPluginInformation();
00115     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00116     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00117
00118 protected:
00119     virtual bool _loadInstance(PersistenceRecord *fields) override;
00120     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00121
00122 private:
00123     const struct DEFAULT_VALUES {
00124         const std::string someString = "Test";
00125         const unsigned int someUInt = 1;
00126         const float someResistance = 1000;
00127     } DEFAULT;
00128     static unsigned int counter;
00129     float resistance = DEFAULT.someResistance;
00130 };
00131
00132 class Vsource : public SPICECircuit {
00133 public:
00134     Vsource(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00135 [this]() {this->Build();};}
00136     void Build() override;
00137     void setVoltage(float voltage);
00138
00139     static PluginInformation* GetPluginInformation();
00140     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00141     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00142
00143 protected:
00144     virtual bool _loadInstance(PersistenceRecord *fields) override;
00145     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00146
00147 private:
00148     const struct DEFAULT_VALUES {
00149         const std::string someString = "Test";
00150         const unsigned int someUInt = 1;

```

```

00152         const float someVoltage = 5;
00153     } DEFAULT;
00154
00155     float voltage = DEFAULT.someVoltage;
00156     static unsigned int counter;
00157 };
00158
00159 class Vpulse : public SPICECircuit {
00160 public:
00161     Vpulse(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00162 [this]() {this->Build();}};
00163     void Build() override;
00164     void setVoltage(float voltage);
00165     void setFreq(float freq);
00166     void setSlope(float slope);
00167
00168     static PluginInformation* GetPluginInformation();
00169     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00170     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00171 protected:
00172     virtual bool _loadInstance(PersistenceRecord *fields) override;
00173     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00174
00175 private:
00176     const struct DEFAULT_VALUES {
00177         const std::string someString = "Test";
00178         const unsigned int someUint = 1;
00179         const float someVoltage = 5;
00180         const float someFreq = 60;
00181         const float someSlope = 1e-11;
00182     } DEFAULT;
00183
00184     static unsigned int counter;
00185     float voltage = DEFAULT.someVoltage;
00186     float freq = DEFAULT.someFreq;
00187     float slope = DEFAULT.someSlope;
00188 };
00189
00190 class Vsine : public SPICECircuit {
00191 public:
00192     Vsine(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00193 [this]() {this->Build();}};
00194     void Build() override;
00195     void setVoltage(float voltage);
00196     void setFreq(float freq);
00197
00198     static PluginInformation* GetPluginInformation();
00199     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00200     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00201 protected:
00202     virtual bool _loadInstance(PersistenceRecord *fields) override;
00203     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00204
00205 private:
00206     const struct DEFAULT_VALUES {
00207         const std::string someString = "Test";
00208         const unsigned int someUint = 1;
00209         const float someVoltage = 5;
00210         const float someFreq = 60;
00211     } DEFAULT;
00212
00213     static unsigned int counter;
00214     float voltage = DEFAULT.someVoltage;
00215     float freq = DEFAULT.someFreq;
00216 };
00217
00218 class Capacitor : public SPICECircuit {
00219 public:
00220     Capacitor(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build
00221 = [this]() {this->Build();}};
00222     void setCapacitance(float capacitance);
00223     void Build() override;
00224
00225     static PluginInformation* GetPluginInformation();
00226     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00227     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00228 protected:
00229     virtual bool _loadInstance(PersistenceRecord *fields) override;
00230     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00231
00232 private:
00233     const struct DEFAULT_VALUES {
00234         const std::string someString = "Test";
00235         const unsigned int someUint = 1;

```

```

00236         const float someCapacitance = 0.01;
00237     } DEFAULT;
00238
00239     static unsigned int counter;
00240     float capacitance = DEFAULT.someCapacitance;
00241 };
00242
00243 class Diode : public SPICECircuit {
00244 public:
00245     Diode(Model* model, std::string name = ""): SPICECircuit(model, name) {
00246         id = counter++; build = [this]() {this->Build();};
00247     }
00248     void Build() override;
00249     void setElectricalModel(std::string electricalModel);
00250
00251     static PluginInformation* GetPluginInformation();
00252     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00253     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00254
00255 protected:
00256     virtual bool _loadInstance(PersistenceRecord *fields) override;
00257     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00258
00259 private:
00260     const struct DEFAULT_VALUES {
00261         const std::string someString = "Test";
00262         const unsigned int someUInt = 1;
00263         const std::string someElectricalModel = "diode";
00264     } DEFAULT;
00265
00266     static unsigned int counter;
00267     std::string electricalModel = DEFAULT.someElectricalModel;
00268 };
00269
00270 class PMOS: public SPICECircuit {
00271 public:
00272     PMOS(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00273 [this]() {this->Build();};}
00274     void Build() override;
00275     void setWidth(float width);
00276     void setLength(float length);
00277     void setElectricalModel(std::string electricalModel);
00278
00279     static PluginInformation* GetPluginInformation();
00280     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00281     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00282
00283 protected:
00284     virtual bool _loadInstance(PersistenceRecord *fields) override;
00285     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00286
00287 private:
00288     const struct DEFAULT_VALUES {
00289         const std::string someString = "Test";
00290         const unsigned int someUInt = 1;
00291         const float someWidth = 140e-9;
00292         const float someLength = 32e-9;
00293         const std::string someElectricalModel = "32nmCMOS";
00294     } DEFAULT;
00295
00296     static unsigned int counter;
00297     float width = DEFAULT.someWidth;
00298     float length = DEFAULT.someLength;
00299     std::string electricalModel = DEFAULT.someElectricalModel;
00300 };
00301
00302 class NMOS: public SPICECircuit {
00303 public:
00304     NMOS(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00305 [this]() {this->Build();};}
00306     void Build() override;
00307     void setWidth(float width);
00308     void setLength(float length);
00309     void setElectricalModel(std::string electricalModel);
00310
00311     static PluginInformation* GetPluginInformation();
00312     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00313     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00314
00315 protected:
00316     virtual bool _loadInstance(PersistenceRecord *fields) override;
00317     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00318
00319 private:
00320     const struct DEFAULT_VALUES {
00321         const std::string someString = "Test";
00322         const unsigned int someUInt = 1;

```

```

00321         const float someWidth = 70e-9;
00322         const float someLength = 32e-9;
00323         const std::string someElectricalModel = "32nmCMOS";
00324     } DEFAULT;
00325
00326     static unsigned int counter;
00327     float width = DEFAULT.someWidth;
00328     float length = DEFAULT.someLength;
00329     std::string electricalModel = DEFAULT.someElectricalModel;
00330 };
00331
00332 class NOT : public SPICECircuit {
00333 public:
00334     NOT(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00335 [this]() {this->Build();}};
00336     void Build() override;
00337     void setPMOSWidth(float PMOSWidth);
00338     void setPMOSLength(float length);
00339     void setNMOSWidth(float NMOSWidth);
00340     void setNMOSLength(float NMOSLength);
00341     void setElectricalModel(std::string electricalModel);
00342
00343     static PluginInformation* GetPluginInformation();
00344     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00345     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00346 protected:
00347     virtual bool _loadInstance(PersistenceRecord *fields) override;
00348     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00349 private:
00350     const struct DEFAULT_VALUES {
00351         const std::string someString = "Test";
00352         const unsigned int someUInt = 1;
00353         const float somePMOSWidth = 140e-9;
00354         const float somePMOSLength = 32e-9;
00355         const float someNMOSWidth = 70e-9;
00356         const float someNMOSLength = 32e-9;
00357         const std::string someElectricalModel = "32nmCMOS";
00358     } DEFAULT;
00359
00360     static unsigned int counter;
00361     float PMOSWidth = DEFAULT.somePMOSWidth;
00362     float PMOSLength = DEFAULT.somePMOSLength;
00363     float NMOSWidth = DEFAULT.someNMOSWidth;
00364     float NMOSLength = DEFAULT.someNMOSLength;
00365     std::string electricalModel = DEFAULT.someElectricalModel;
00366 };
00367
00368 class NAND : public SPICECircuit {
00369 public:
00370     NAND(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00371 [this]() {this->Build();}};
00372     void Build() override;
00373     void setPMOSWidth(float PMOSWidth);
00374     void setPMOSLength(float length);
00375     void setNMOSWidth(float NMOSWidth);
00376     void setNMOSLength(float NMOSLength);
00377     void setElectricalModel(std::string electricalModel);
00378
00379     static PluginInformation* GetPluginInformation();
00380     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00381     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00382 protected:
00383     virtual bool _loadInstance(PersistenceRecord *fields) override;
00384     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00385 private:
00386     const struct DEFAULT_VALUES {
00387         const std::string someString = "Test";
00388         const unsigned int someUInt = 1;
00389         const float somePMOSWidth = 140e-9;
00390         const float somePMOSLength = 32e-9;
00391         const float someNMOSWidth = 70e-9;
00392         const float someNMOSLength = 32e-9;
00393         const std::string someElectricalModel = "32nmCMOS";
00394     } DEFAULT;
00395
00396     static unsigned int counter;
00397     float PMOSWidth = DEFAULT.somePMOSWidth;
00398     float PMOSLength = DEFAULT.somePMOSLength;
00399     float NMOSWidth = DEFAULT.someNMOSWidth;
00400     float NMOSLength = DEFAULT.someNMOSLength;
00401     std::string electricalModel = DEFAULT.someElectricalModel;
00402 };
00403
00404
00405

```

```

00406 class AND : public SPICECircuit {
00407 public:
00408     AND(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
[this]() {this->Build();}};
00409     void Build() override;
00410     void setPMOSWidth(float PMOSWidth);
00411     void setPMOSLength(float length);
00412     void setNMOSWidth(float NMOSWidth);
00413     void setNMOSLength(float NMOSLength);
00414     void setElectricalModel(std::string electricalModel);
00415
00416     static PluginInformation* GetPluginInformation();
00417     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00418     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00419
00420 protected:
00421     virtual bool _loadInstance(PersistenceRecord *fields) override;
00422     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00423
00424 private:
00425     const struct DEFAULT_VALUES {
00426         const std::string someString = "Test";
00427         const unsigned int someUInt = 1;
00428         const float somePMOSWidth = 140e-9;
00429         const float somePMOSLength = 32e-9;
00430         const float someNMOSWidth = 70e-9;
00431         const float someNMOSLength = 32e-9;
00432         const std::string someElectricalModel = "32nmCMOS";
00433     } DEFAULT;
00434
00435     static unsigned int counter;
00436     float PMOSWidth = DEFAULT.somePMOSWidth;
00437     float PMOSLength = DEFAULT.somePMOSLength;
00438     float NMOSWidth = DEFAULT.someNMOSWidth;
00439     float NMOSLength = DEFAULT.someNMOSLength;
00440     std::string electricalModel = DEFAULT.someElectricalModel;
00441 };
00442
00443 class NOR : public SPICECircuit {
00444 public:
00445     NOR(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
[this]() {this->Build();}};
00446     void Build() override;
00447     void setPMOSWidth(float PMOSWidth);
00448     void setPMOSLength(float length);
00449     void setNMOSWidth(float NMOSWidth);
00450     void setNMOSLength(float NMOSLength);
00451     void setElectricalModel(std::string electricalModel);
00452
00453     static PluginInformation* GetPluginInformation();
00454     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00455     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00456
00457 protected:
00458     virtual bool _loadInstance(PersistenceRecord *fields) override;
00459     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00460
00461 private:
00462     const struct DEFAULT_VALUES {
00463         const std::string someString = "Test";
00464         const unsigned int someUInt = 1;
00465         const float somePMOSWidth = 140e-9;
00466         const float somePMOSLength = 32e-9;
00467         const float someNMOSWidth = 70e-9;
00468         const float someNMOSLength = 32e-9;
00469         const std::string someElectricalModel = "32nmCMOS";
00470     } DEFAULT;
00471
00472     static unsigned int counter;
00473     float PMOSWidth = DEFAULT.somePMOSWidth;
00474     float PMOSLength = DEFAULT.somePMOSLength;
00475     float NMOSWidth = DEFAULT.someNMOSWidth;
00476     float NMOSLength = DEFAULT.someNMOSLength;
00477     std::string electricalModel = DEFAULT.someElectricalModel;
00478 };
00479
00480 class OR : public SPICECircuit {
00481 public:
00482     OR(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
[this]() {this->Build();}};
00483     void Build() override;
00484     void setPMOSWidth(float PMOSWidth);
00485     void setPMOSLength(float length);
00486     void setNMOSWidth(float NMOSWidth);
00487     void setNMOSLength(float NMOSLength);
00488     void setElectricalModel(std::string electricalModel);
00489

```

```

00490     static PluginInformation* GetPluginInformation();
00491     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00492     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00493
00494 protected:
00495     virtual bool _loadInstance(PersistenceRecord *fields) override;
00496     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00497
00498 private:
00499     const struct DEFAULT_VALUES {
00500         const std::string someString = "Test";
00501         const unsigned int someUInt = 1;
00502         const float somePMOSWidth = 140e-9;
00503         const float somePMOSLength = 32e-9;
00504         const float someNMOSWidth = 70e-9;
00505         const float someNMOSLength = 32e-9;
00506         const std::string someElectricalModel = "32nmCMOS";
00507     } DEFAULT;
00508
00509     static unsigned int counter;
00510     float PMOSWidth = DEFAULT.somePMOSWidth;
00511     float PMOSLength = DEFAULT.somePMOSLength;
00512     float NMOSWidth = DEFAULT.someNMOSWidth;
00513     float NMOSLength = DEFAULT.someNMOSLength;
00514     std::string electricalModel = DEFAULT.someElectricalModel;
00515 };
00516
00517 class XOR : public SPICECircuit {
00518 public:
00519     XOR(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00520 [this]() {this->Build();}};
00521     void Build() override;
00522     void setPMOSWidth(float PMOSWidth);
00523     void setPMOSLength(float length);
00524     void setNMOSWidth(float NMOSWidth);
00525     void setNMOSLength(float NMOSLength);
00526     void setElectricalModel(std::string electricalModel);
00527
00528     static PluginInformation* GetPluginInformation();
00529     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00530     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00531
00532 protected:
00533     virtual bool _loadInstance(PersistenceRecord *fields) override;
00534     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00535
00536 private:
00537     const struct DEFAULT_VALUES {
00538         const std::string someString = "Test";
00539         const unsigned int someUInt = 1;
00540         const float somePMOSWidth = 140e-9;
00541         const float somePMOSLength = 32e-9;
00542         const float someNMOSWidth = 70e-9;
00543         const float someNMOSLength = 32e-9;
00544         const std::string someElectricalModel = "32nmCMOS";
00545     } DEFAULT;
00546
00547     static unsigned int counter;
00548     float PMOSWidth = DEFAULT.somePMOSWidth;
00549     float PMOSLength = DEFAULT.somePMOSLength;
00550     float NMOSWidth = DEFAULT.someNMOSWidth;
00551     float NMOSLength = DEFAULT.someNMOSLength;
00552     std::string electricalModel = DEFAULT.someElectricalModel;
00553 };
00554
00555 class XNOR : public SPICECircuit {
00556 public:
00557     XNOR(Model* model, std::string name = ""): SPICECircuit(model, name) { id = counter++; build =
00558 [this]() {this->Build();}};
00559     void Build() override;
00560     void setPMOSWidth(float PMOSWidth);
00561     void setPMOSLength(float length);
00562     void setNMOSWidth(float NMOSWidth);
00563     void setNMOSLength(float NMOSLength);
00564     void setElectricalModel(std::string electricalModel);
00565
00566     static PluginInformation* GetPluginInformation();
00567     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00568     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00569
00570 protected:
00571     virtual bool _loadInstance(PersistenceRecord *fields) override;
00572     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00573
00574 private:
00575     const struct DEFAULT_VALUES {
00576         const std::string someString = "Test";

```

```

00575         const unsigned int someUint = 1;
00576         const float somePMOSWidth = 140e-9;
00577         const float somePMOSLength = 32e-9;
00578         const float someNMOSWidth = 70e-9;
00579         const float someNMOSLength = 32e-9;
00580         const std::string someElectricalModel = "32nmCMOS";
00581     } DEFAULT;
00582
00583     static unsigned int counter;
00584     float PMOSWidth = DEFAULT.somePMOSWidth;
00585     float PMOSLength = DEFAULT.somePMOSLength;
00586     float NMOSWidth = DEFAULT.someNMOSWidth;
00587     float NMOSLength = DEFAULT.someNMOSLength;
00588     std::string electricalModel = DEFAULT.someElectricalModel;
00589 };

```

11.332 SPICENode.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:     SPICENode.h
00009  * Author:   bnmfw & joaomai
00010  *
00011  * Created on 10 de outubro de 2023, 23:00
00012  */
00013
00014 #pragma once
00015
00016 #include "../kernel/simulator/model/ModelComponent.h"
00017 #include "../data/ExternalIntegration/SPICERunner.h"
00018 #include "plugins/components/ElectronicsSimulation/SPICENode.h"
00019
00020 class SPICENode : public ModelComponent {
00021 public:
00022     SPICENode(Model* model, std::string name = "");
00023     virtual ~SPICENode() = default;
00024
00025 public:
00026     std::string getNodeName();
00027
00028 public:
00029     virtual std::string show() override;
00030
00031 public:
00032     static PluginInformation* GetPluginInformation();
00033     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00034     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00035     operator std::string() const;
00036
00037 protected:
00038     virtual bool _loadInstance(PersistenceRecord *fields) override;
00039     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00040     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00041
00042 protected:
00043     // virtual bool _check(std::string& errorMessage);
00044     // virtual ParserChangesInformation* _getParserChangesInformation();
00045     // virtual void _initBetweenReplications();
00046     // virtual void _createInternalStatisticReporters() override;
00047     // virtual void _createNonEditableDataDefinitions() override;
00048     // virtual void _createEditableDataDefinitions() override;
00049     // virtual void _createAttachedAttributes() override;
00050     // virtual void _createInternalAndAttachedData(); /*< A ModelDataDefinition or ModelComponent that
00051     includes (internal) ou refers to (attach) other ModelDataDefinition must register them inside this
00052     method. */
00053     // virtual void _addSimulationControl(SimulationControl* property);
00054
00055 protected:
00056
00057 private:
00058     std::string node_name;
00059     static int std_node_count;
00060     static std::set<std::string> used_names;
00061
00062 private:

```



```

00072
00073     const struct DEFAULT_VALUES {
00074         const std::string someString = "Test";
00075         const unsigned int someUInt = 1;
00076     } DEFAULT;
00077     std::string _someString = DEFAULT.someString;
00078     unsigned int _someUInt = DEFAULT.someUInt;
00079
00080 private:
00081
00082 private:
00083     // ...
00084 };
00085

```

11.333 CppForG.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File: CppForG.h
00008  * Author: rlcancian
00009  *
00010  * Created on 11 de janeiro de 2022, 22:37
00011  */
00012
00013 #pragma once
00014
00015 #include "../kernel/simulator/model/ModelComponent.h"
00016 #include "kernel/simulator/Simulator.h"
00017 #include "../data/ExternalIntegration/CppCompiler.h"
00018
00019 extern "C" typedef void (*initBetweenReplications_t)(Model* model);
00020 extern "C" typedef void (*onDispatchEvent_t)(Simulator* simulator, Model* model, Entity* entity);
00021
00022 class CppForG : public ModelComponent {
00023 public: // constructors
00024     CppForG(Model* model, std::string name = "");
00025     virtual ~CppForG() = default;
00026 public: // virtual
00027     virtual std::string show() override;
00028 public: // static
00029     static PluginInformation* GetPluginInformation();
00030     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00031     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00032 public:
00033     void setCppCompiler(CppCompiler* _cppCompiler);
00034     CppCompiler* getCppCompiler() const;
00035     void setInitBetweenReplicationCode(std::string _initBetweenReplicationCode);
00036     std::string getInitBetweenReplicationCode() const;
00037     void setOnDispatchEventCode(std::string _onDispatchEventCode);
00038     std::string getOnDispatchEventCode() const;
00039     void setIncludesCode(std::string _includesCode);
00040     std::string getIncludesCode() const;
00041 protected: // must be overridden
00042     virtual bool _loadInstance(PersistenceRecord* fields) override;
00043     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00044     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00045 protected: // could be overridden by derived classes
00046     virtual bool _check(std::string& errorMessage) override;
00047     //virtual ParserChangesInformation* _getParserChangesInformation();
00048     virtual void _initBetweenReplications() override;
00049     // virtual void _createInternalAndAttachedData() override;
00050     //virtual void _addSimulationControl(SimulationControl* property);
00051 protected:
00052     // virtual void _createInternalStatisticReporters() override;
00053     virtual void _createNonEditableDataDefinitions() override;
00054     // virtual void _createAttachedAttributes() override;
00055
00056 private: // methods
00057
00058 private: // attributes 1:1
00059
00060     const struct DEFAULT_VALUES {
00061         std::string onDispatchEventCode = "";
00062         std::string initBetweenReplicationCode = "";
00063         std::string includesCode = "";
00064     } DEFAULT;
00065     std::string _onDispatchEventCode = DEFAULT.onDispatchEventCode;
00066     std::string _initBetweenReplicationCode = DEFAULT.initBetweenReplicationCode;
00067     std::string _includesCode = DEFAULT.includesCode;
00068

```

```

00073     //
00074     std::string _sourceFilename;
00075     std::string _outputFilename;
00076     // pointers to functions on a shared library
00077     onDispatchEvent_t dispatchEvent_SharedLibHandler;
00078     initBetweenReplications_t initBetweenReplications_SharedLibHandler;
00079 private: // internal (TODO: should be Attached??)
00080     CppCompiler* _cppCompiler = nullptr;
00081 private: // attributes 1:n
00082 };

```

11.334 PythonForG.h

```

00001 #ifndef PYTHONFORG_H
00002 #define PYTHONFORG_H
00003
00004 #include "../kernel/simulator/model/ModelComponent.h"
00005 #include "../kernel/simulator/SimulationControlAndResponse.h"
00006 #include "plugins/data/ExternalIntegration/PythonRuntime.h"
00007
00008 class PythonForG : public ModelComponent {
00009 public:
00010     PythonForG(Model* model, std::string name = "");
00011     virtual ~PythonForG() override = default;
00012
00013 public:
00014     static PluginInformation* GetPluginInformation();
00015     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00016     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00017
00018 public:
00019     void setInitBetweenReplicationCode(std::string initBetweenReplicationCode);
00020     std::string getInitBetweenReplicationCode() const;
00021     void setInitBetweenReplicationCodeProperty(SourceCodeString initBetweenReplicationCode);
00022     SourceCodeString getInitBetweenReplicationCodeProperty() const;
00023     void setOnDispatchEventCode(std::string onDispatchEventCode);
00024     std::string getOnDispatchEventCode() const;
00025     void setOnDispatchEventCodeProperty(SourceCodeString onDispatchEventCode);
00026     SourceCodeString getOnDispatchEventCodeProperty() const;
00027     void setForwardEntityOnError(bool forwardEntityOnError);
00028     bool isForwardEntityOnError() const;
00029     PythonRuntime* getPythonRuntime() const;
00030
00031     virtual std::string show() override;
00032
00033 protected:
00034     virtual bool _loadInstance(PersistenceRecord* fields) override;
00035     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00036     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00037     virtual bool _check(std::string& errorMessage) override;
00038     virtual void _initBetweenReplications() override;
00039     virtual void _createNonEditableDataDefinitions() override;
00040
00041 private:
00042     bool _validateInitCode(const std::string& candidate, std::string& errorMessage) const;
00043     bool _validateDispatchCode(const std::string& candidate, std::string& errorMessage) const;
00044
00045 private:
00046     const struct DEFAULT_VALUES {
00047         std::string initBetweenReplicationCode = "";
00048         std::string onDispatchEventCode = "";
00049         bool forwardEntityOnError = true;
00050     } DEFAULT;
00051
00052     std::string _initBetweenReplicationCode = DEFAULT.initBetweenReplicationCode;
00053     std::string _onDispatchEventCode = DEFAULT.onDispatchEventCode;
00054     bool _forwardEntityOnError = DEFAULT.forwardEntityOnError;
00055     PythonRuntime* _pythonRuntime = nullptr;
00056 };
00057
00058 #endif /* PYTHONFORG_H */

```

11.335 RSimulator.h

```

00001 /*
00002  * File:   RSimulator.h
00003  * Author: GenESys
00004  *
00005  * Component that executes R commands when entities arrive.

```

```

00006  */
00007
00008 #ifndef RSIMULATOR_H
00009 #define RSIMULATOR_H
00010
00011 #include <list>
00012 #include <string>
00013
00014 #include "../kernel/simulator/model/ModelComponent.h"
00015 #include "kernel/simulator/Plugin.h"
00016 #include "plugins/data/ExternalIntegration/RSimulatorRunner.h"
00017
00025 class RSimulator : public ModelComponent {
00026 public:
00028     RSimulator(Model* model, std::string name = "");
00030     virtual ~RSimulator() = default;
00031
00032 public:
00034     void setRExecutable(std::string rExecutable);
00036     std::string getRExecutable() const;
00038     void setWorkingDirectory(std::string workingDirectory);
00040     std::string getWorkingDirectory() const;
00042     void setPreludeScript(std::string preludeScript);
00044     std::string getPreludeScript() const;
00046     void insertCommand(std::string command);
00048     void setCommands(std::list<std::string> commands);
00050     List<std::string>* getCommands() const;
00052     void setCommandList(std::string commandList);
00054     std::string getCommandList() const;
00056     RSimulatorRunner* getRunner();
00058     virtual std::string show() override;
00059
00060 public:
00062     static PluginInformation* GetPluginInformation();
00064     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00066     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00067
00068 protected:
00070     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00072     virtual bool _loadInstance(PersistenceRecord *fields) override;
00074     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00076     virtual bool _check(std::string& errorMessage) override;
00078     // virtual void _createInternalAndAttachedData() override;
00079
00080
00081 protected:
00082     // virtual void _createInternalStatisticReporters() override;
00083     // virtual void _createNonEditableDataDefinitions() override;
00084     virtual void _createEditableDataDefinitions() override;
00085     // virtual void _createAttachedAttributes() override;
00086
00087 private:
00089     void _ensureRunner();
00091     void _syncRunnerConfiguration();
00093     bool _isNonEmptyCommand(const std::string& command) const;
00094
00095 private:
00096     const struct DEFAULT_VALUES {
00097         std::string rExecutable = "Rscript";
00098         std::string workingDirectory = "";
00099         std::string preludeScript = "";
00100     } DEFAULT;
00101
00102     std::string _rExecutable = DEFAULT.rExecutable;
00103     std::string _workingDirectory = DEFAULT.workingDirectory;
00104     std::string _preludeScript = DEFAULT.preludeScript;
00105     List<std::string>* _commands = new List<std::string>();
00106     RSimulatorRunner* _runner = nullptr;
00107 };
00108
00109 #endif /* RSIMULATOR_H */

```

11.336 Batch.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   Batch.h
00009  * Author: rlcancian

```

```

00010  *
00011  * Created on 03 de Junho de 2019, 15:14
00012  */
00013
00014 #ifndef BATCH_H
00015 #define BATCH_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "plugins/data/DiscreteProcessing/Queue.h"
00019 #include "../data/Grouping/EntityGroup.h"
00053 class Batch : public ModelComponent {
00054 public:
00055
00056     enum class BatchType : int {
00057         Temporary = 0, Permanent = 1, num_elements = 2
00058     };
00059
00060     enum class Rule : int {
00061         Any = 0, ByAttribute = 1, num_elements = 2
00062     };
00063
00064     enum class GroupedAttribs : int {
00065         FirstEntity = 0, LastEntity = 1, SumAttributes = 2, num_elements = 3
00066     };
00067 public:
00068     static std::string convertEnumToStr(BatchType type);
00069     static std::string convertEnumToStr(Rule rule);
00070     static std::string convertEnumToStr(GroupedAttribs attribs);
00071 public: // constructors
00072     Batch(Model* model, std::string name = "");
00073     virtual ~Batch() = default;
00074 public: // virtual
00075     virtual std::string show() override;
00076 public: // static
00077     static PluginInformation* GetPluginInformation();
00078     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00079     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00080 public:
00081     void setGroupedEntityType(EntityType* groupedEntityType);
00082     void setGroupedEntityType(std::string groupedEntityType);
00083     EntityType* getGroupedEntityType() const;
00084     void setBatchType(Batch::BatchType batchType);
00085     Batch::BatchType getBatchType() const;
00086     void setAttributeName(std::string attributeName);
00087     std::string getAttributeName() const;
00088     void setBatchSize(std::string batchSize);
00089     std::string getBatchSize() const;
00090     void setRule(Batch::Rule _rule);
00091     Batch::Rule getRule() const;
00092     void setGroupedAttributes(Batch::GroupedAttribs _groupedAttributes);
00093     Batch::GroupedAttribs getGroupedAttributes() const;
00094 protected: // virtual should
00095     //virtual void _initBetweenReplications();
00096     // virtual void _createInternalStatisticReporters() override;
00097     virtual void _createNonEditableDataDefinitions() override;
00098     virtual void _createEditableDataDefinitions() override;
00099     virtual void _createAttachedAttributes() override;
00100     // virtual void _createInternalAndAttachedData() override;
00101     virtual bool _check(std::string& errorMessage) override;
00102 protected: // virtual must
00103     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00104     virtual bool _loadInstance(PersistenceRecord *fields) override;
00105     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00106
00107 protected:
00108
00109 private: // methods
00110 private: // attributes 1:1
00111
00112     const struct DEFAULT_VALUES {
00113         const Batch::BatchType batchType = Batch::BatchType::Temporary;
00114         const Batch::Rule rule = Batch::Rule::Any;
00115         const Batch::GroupedAttribs groupedAttributes = Batch::GroupedAttribs::FirstEntity;
00116         const std::string batchSize = "2";
00117         const std::string attributeName = "";
00118     } DEFAULT;
00119     Batch::BatchType _batchType = DEFAULT.batchType;
00120     Batch::Rule _rule = DEFAULT.rule;
00121     Batch::GroupedAttribs _groupedAttributes = DEFAULT.groupedAttributes;
00122     std::string _batchSize = DEFAULT.batchSize;
00123     std::string _attributeName = DEFAULT.attributeName;
00124 private: // attributes 1:1
00125     EntityType* _groupedEntityType = nullptr;
00126     EntityGroup* _entityGroup = nullptr;
00127     Queue* _queue = nullptr;
00128 private: // attributes 1:n
00129     // count number of batches?

```

```

00130 };
00131
00132
00133 #endif /* BATCH_H */

```

11.337 Separate.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Separate.h
00009  * Author: rlcancian
00010  *
00011  * Created on 03 de Junho de 2019, 15:14
00012  */
00013
00014 #ifndef SEPARATE_H
00015 #define SEPARATE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00065 class Separate : public ModelComponent {
00066 public: // constructors
00067     Separate(Model* model, std::string name = "");
00068     virtual ~Separate() = default;
00069 public: // virtual
00070     virtual std::string show() override;
00071 public: // static
00072     static PluginInformation* GetPluginInformation();
00073     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00074     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00075 protected: // virtual
00076     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00077     virtual bool _loadInstance(PersistenceRecord *fields) override;
00078     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00079 protected: // virtual
00080     //virtual void _initBetweenReplications();
00081     // virtual void _createInternalStatisticReporters() override;
00082     // virtual void _createNonEditableDataDefinitions() override;
00083     // virtual void _createEditableDataDefinitions() override;
00084     // virtual void _createAttachedAttributes() override;
00085     virtual bool _check(std::string& errorMessage) override;
00086
00087 protected:
00088
00089 private: // methods
00090 private: // attributes 1:l
00091 private: // attributes 1:n
00092 };
00093
00094
00095 #endif /* SEPARATE_H */

```

11.338 Record.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Record.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 9 de Agosto de 2018, 13:52
00012  */
00013
00014 #ifndef RECORD_H
00015 #define RECORD_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include <string>
00019
00062 class Record : public ModelComponent {

```

```

00063 public:
00064     Record(Model* model, std::string name = "");
00065     virtual ~Record();
00066
00067 public:
00068     void setFilename(std::string filename);
00069     std::string getFileName() const;
00070     void setExpression(const std::string expression);
00071     std::string getExpression() const;
00075     void setExpressionName(std::string expressionName);
00076     std::string getExpressionName() const;
00080     void setDatasetName(std::string datasetName);
00081     std::string getDatasetName() const;
00085     void setRandomVariableName(std::string randomVariableName);
00086     std::string getRandomVariableName() const;
00090     void setVariableType(std::string variableType);
00091     std::string getVariableType() const;
00095     void setDatasetDescription(std::string description);
00096     std::string getDatasetDescription() const;
00097     StatisticsCollector* getCstatExpression() const;
00098     bool getTimeDependent() const;
00099     void setTimeDependent(bool timeDependent);
00100
00101 public:
00102     virtual std::string show() override;
00103
00104 public:
00105     static PluginInformation* GetPluginInformation();
00106     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00107     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00108
00109 protected:
00110     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00111     virtual bool _loadInstance(PersistenceRecord *fields) override;
00112     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00113
00114 protected:
00115     virtual void _initBetweenReplications() override;
00116     virtual bool _check(std::string& errorMessage) override;
00117     // virtual void _createInternalAndAttachedData() override;
00118
00119
00120 protected:
00121     virtual void _createInternalStatisticReporters() override;
00122     // virtual void _createNonEditableDataDefinitions() override;
00123     // virtual void _createEditableDataDefinitions() override;
00124     // virtual void _createAttachedAttributes() override;
00125
00126 private:
00127     const struct DEFAULT_VALUES {
00128         const bool timeDependent = false;
00129         const std::string expression = "";
00130         const std::string expressionName = "";
00131         const std::string datasetName = "";
00132         const std::string randomVariableName = "";
00133         const std::string variableType = "Continuous numeric";
00134         const std::string description = "";
00135         const std::string filename = "";
00136     } DEFAULT;
00137     bool _timeDependent = DEFAULT.timeDependent;
00138     std::string _expression = DEFAULT.expression;
00139     std::string _expressionName = DEFAULT.expressionName;
00140     std::string _datasetName = DEFAULT.datasetName;
00141     std::string _randomVariableName = DEFAULT.randomVariableName;
00142     std::string _variableType = DEFAULT.variableType;
00143     std::string _description = DEFAULT.description;
00144     std::string _filename = DEFAULT.filename;
00145 private:
00146     const std::string _separator = " ";
00147     StatisticsCollector* _cstatExpression = nullptr;
00148     /* @TODO: Create an internal class to aggregate ExpressionStatisticsCollector, and change Record to
        got a list of it, so Record can record a set of expressions into a set of files */
00149 };
00150
00151 #endif /* RECORD_H */

```

11.339 Write.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */

```

```

00006
00007 /*
00008  * File:    Write.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 11 de Setembro de 2019, 13:06
00012  */
00013
00014 #ifndef WRITE_H
00015 #define WRITE_H
00016
00017 #include <list>
00018 #include <fstream>
00019 #include "../kernel/simulator/model/ModelComponent.h"
00020
00024 class Write : public ModelComponent {
00025 public:
00026     enum class WriteToType : int {
00027         SCREEN = 0, FILE = 1, num_elements = 2
00028     };
00029 public:
00030     static std::string convertEnumToStr(WriteToType type);
00031 public: // constructors
00032     Write(Model* model, std::string name = "");
00033     virtual ~Write() override;
00034 public:
00035     void insertText(std::list<std::string> texts);
00036     void setFilename(std::string _filename);
00037     std::string filename() const;
00038     void setWriteToType(WriteToType _writeToType);
00039     Write::WriteToType writeToType() const;
00040 public: // virtual
00041     virtual std::string show() override;
00042 public: // static
00043     static PluginInformation* GetPluginInformation();
00044     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00045     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00046 protected: // virtual
00047     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00048     virtual bool _loadInstance(PersistenceRecord *fields) override;
00049     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00050 protected: // virtual
00051     virtual bool _check(std::string& errorMessage) override;
00052     virtual void _initBetweenReplications() override;
00053 protected:
00054     // virtual void _createInternalStatisticReporters() override;
00055     // virtual void _createNonEditableDataDefinitions() override;
00056     // virtual void _createEditableDataDefinitions() override;
00057     // virtual void _createAttachedAttributes() override;
00058 private: // methods
00059 private: // attributes 1:1
00060     const struct DEFAULT_VALUES {
00061         const WriteToType writeToType = Write::WriteToType::SCREEN;
00062         const std::string filename = "";
00063     } DEFAULT;
00064     WriteToType _writeToType = DEFAULT.writeToType;
00065     std::string _filename = DEFAULT.filename;
00066     std::ofstream _savefile;
00067 private: // attributes 1:n
00068     List<std::string>* _writeElements = new List<std::string>();
00069 };
00070
00071 #endif /* WRITE_H */

```

11.340 Assign.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*

```

```

00008  * File:    Assign.h
00009  * Author:  rafael.lui.cancian
00010  *
00011  * Created on 31 de Agosto de 2018, 10:10
00012  */
00013
00014 #ifndef ASSIGN_H
00015 #define ASSIGN_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../kernel/simulator/model/Model.h"
00019 #include "kernel/simulator/Plugin.h"
00020 #include "../kernel/simulator/essentialPlugins/Attribute.h"
00021 #include "../data/Logic/AssignmentItem.h"
00022 #include "../data/Logic/Variable.h"
00023
00061 class Assign : public ModelComponent {
00062 public:
00063     Assign(Model* model, std::string name = "");
00064     virtual ~Assign() override;
00065 public:
00066     virtual std::string show() override;
00067 public:
00068     static PluginInformation* GetPluginInformation();
00069     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00070     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00071 public:
00072     List<Assignment*> getAssignments() const;
00073     void addAssignment(Assignment* newAssignment);
00074     void removeAssignment(Assignment* assignment);
00075 protected: // must be overridden
00076     virtual bool _loadInstance(PersistenceRecord *fields) override;
00077     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00078     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00079 protected: // could be overridden by derived classes
00080     virtual bool _check(std::string& errorMessage) override;
00082     //virtual ParserChangesInformation* _getParserChangesInformation();
00083     //virtual void _initBetweenReplications();
00084 protected:
00085     // virtual void _createInternalStatisticReporters() override;
00086     // virtual void _createNonEditableDataDefinitions() override;
00087     virtual void _createEditableDataDefinitions() override;
00088     //virtual void _createAttachedAttributes() override;
00089     // virtual void _createInternalAndAttachedData() override;
00091     //virtual void _addSimulationControl(SimulationControl* property);
00092
00093 protected:
00094
00095 private:
00096     static std::string _destinationBaseName(const std::string& destination);
00097     std::string _destinationIndex(const std::string& destination);
00098     void _prepareAssignment(Assignment* assignment);
00099 private:
00100
00101     const struct DEFAULT_VALUES {
00102         const unsigned int assignmentsSize = 1;
00103     } DEFAULT;
00104     List<Assignment*> _assignments = new List<Assignment*>();
00105 };
00106
00107 #endif /* ASSIGN_H */

```

11.341 Create.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Create.h
00009  * Author:  rafael.lui.cancian
00010  *
00011  * Created on 21 de Junho de 2018, 20:12
00012  */
00013
00014 #ifndef CREATE_H
00015 #define CREATE_H
00016
00017 #include <string>
00018 #include <limits>
00019 #include "kernel/simulator/SourceModelComponent.h"

```



```

00020 #include "../kernel/simulator/essentialPlugins/EntityType.h"
00021 #include "../kernel/simulator/essentialPlugins/Counter.h"
00022 #include "kernel/simulator/Plugin.h"
00023
00024 #include "../data/Logic/Formula.h"
00025 #include "plugins/data/DiscreteProcessing/Schedule.h"
00026
00070 class Create : public SourceModelComponent {
00071 public:
00072     Create(Model* model, std::string name = "");
00073     virtual ~Create() = default;
00074 public: // virtual
00075     virtual std::string show() override;
00076 public:
00077     void setTimeBetweenCreationsFormula(Formula* _timeBetweenCreationsFormula);
00078     Formula* getTimeBetweenCreationsFormula() const;
00079     void setTimeBetweenCreationsSchedule(Schedule* _timeBetweenCreationsSchedule);
00080     Schedule* getTimeBetweenCreationsSchedule() const;
00081 public: // static
00082     static PluginInformation* GetPluginInformation();
00083     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00084     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00085
00086     double testePropertyCreateDouble() const;
00087     void setTestePropertyCreateDouble(double newTestePropertyCreateDouble);
00088
00089 protected:
00090     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00091     virtual bool _loadInstance(PersistenceRecord *fields) override;
00092     virtual void _initBetweenReplications() override;
00093     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00094     virtual bool _check(std::string& errorMessage) override;
00095     virtual void _createInternalStatisticReporters() override;
00096     virtual void _createEditableDataDefinitions() override;
00097 private:
00098     double _lastArrival = -1.0;
00099 private: // internal elements
00100     Counter* _numberOut = nullptr; // internal modeldatum
00101 private: // attached elements
00102     Schedule* _timeBetweenCreationsSchedule = nullptr;
00103     Formula* _timeBetweenCreationsFormula = nullptr;
00104 };
00105
00106 #endif /* CREATE_H */

```

11.342 Dispose.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Dispose.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 21 de Junho de 2018, 20:13
00012  */
00013
00014 #ifndef DISPOSE_H
00015 #define DISPOSE_H
00016
00017 #include "kernel/simulator/SinkModelComponent.h"
00018 #include "../kernel/simulator/essentialPlugins/Counter.h"
00019 #include "kernel/simulator/Plugin.h"
00020
00038 class Dispose : public SinkModelComponent {
00039 public:
00040     Dispose(Model* model, std::string name = "");
00041     virtual ~Dispose() = default;
00042 public:
00043     virtual std::string show() override;
00044 public:
00045     static PluginInformation* GetPluginInformation();
00046     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00047     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00048 protected:
00049     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00050     virtual bool _loadInstance(PersistenceRecord *fields) override;
00051     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00052 protected:
00053     virtual void _initBetweenReplications() override;

```

```

00054     virtual bool _check(std::string& errorMessage) override;
00055 protected:
00056     virtual void _createInternalStatisticReporters() override;
00057     // virtual void _createNonEditableDataDefinitions() override;
00058     // virtual void _createEditableDataDefinitions() override;
00059     virtual void _createAttachedAttributes() override;
00060 protected:
00061 private: // internal elements
00062     Counter* _numberOut = nullptr;
00063 };
00064
00065 #endif /* DISPOSE_H */

```

11.343 Access.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:     Access.h
00009  * Author:    rlcancian
00010  *
00011  * Created on 11 de Setembro de 2019, 13:14
00012  */
00013
00014 #ifndef ACCESS_H
00015 #define ACCESS_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00053 class Access : public ModelComponent {
00054 public: // constructors
00055     Access(Model* model, std::string name = "");
00056     virtual ~Access() = default;
00057 public: // virtual
00058     virtual std::string show() override;
00059 public: // static
00060     static PluginInformation* GetPluginInformation();
00061     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00062     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00063 protected: // virtual
00064     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00065     virtual bool _loadInstance(PersistenceRecord *fields) override;
00066     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00067 protected: // virtual
00068     //virtual void _initBetweenReplications();
00069     // virtual void _createInternalStatisticReporters() override;
00070     // virtual void _createNonEditableDataDefinitions() override;
00071     // virtual void _createEditableDataDefinitions() override;
00072     // virtual void _createAttachedAttributes() override;
00073     virtual bool _check(std::string& errorMessage) override;
00074
00075 protected:
00076
00077 private: // methods
00078 private: // attributes 1:1
00079 private: // attributes 1:n
00080 };
00081
00082
00083 #endif /* ACCESS_H */

```

11.344 Enter.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:     Enter.h
00009  * Author:    rlcancian
00010  *
00011  * Created on 03 de Junho de 2019, 15:14
00012  */

```

```

00013
00014 #ifndef ENTER_H
00015 #define ENTER_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../data/MaterialHandling/Station.h"
00019 #include "../kernel/simulator/essentialPlugins/Counter.h"
00020
00099 class Enter : public ModelComponent {
00100 public:
00101     Enter(Model* model, std::string name = "");
00102     virtual ~Enter() = default;
00103 public:
00104     virtual std::string show() override;
00105 public:
00106     static PluginInformation* GetPluginInformation();
00107     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00108     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00109 public:
00110     void setStation(Station* _station);
00111     void setStationName(std::string stationName);
00112     Station* getStation() const;
00113 protected:
00114     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00115     virtual bool _loadInstance(PersistenceRecord *fields) override;
00116     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00117 protected:
00118     //virtual void _initBetweenReplications();
00119     virtual bool _check(std::string& errorMessage) override;
00120
00121     virtual void _createInternalStatisticReporters() override;
00122     // virtual void _createNonEditableDataDefinitions() override;
00123     virtual void _createEditableDataDefinitions() override;
00124     // virtual void _createAttachedAttributes() override;
00125     // virtual void _createInternalAndAttachedData() override;
00126
00127 protected:
00128
00129 private: // association
00130     Station* _station = nullptr;
00131 private: // internal elements
00132     Counter* _numberIn = nullptr;
00133 };
00134
00135 #endif /* ENTER_H */

```

11.345 Exit.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Exit.h
00009  * Author: rlcancian
00010  *
00011  * Created on 11 de Setembro de 2019, 13:15
00012  */
00013
00014 #ifndef EXIT_H
00015 #define EXIT_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00037 class Exit : public ModelComponent {
00038 public: // constructors
00039     Exit(Model* model, std::string name = "");
00040     virtual ~Exit() = default;
00041 public: // virtual
00042     virtual std::string show() override;
00043 public: // static
00044     static PluginInformation* GetPluginInformation();
00045     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00046     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00047 protected: // virtual
00048     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00049     virtual bool _loadInstance(PersistenceRecord *fields) override;
00050     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00051 protected: // virtual
00052     //virtual void _initBetweenReplications();
00053     // virtual void _createInternalStatisticReporters() override;

```

```

00054     // virtual void _createNonEditableDataDefinitions() override;
00055     // virtual void _createEditableDataDefinitions() override;
00056     // virtual void _createAttachedAttributes() override;
00057     virtual bool _check(std::string& errorMessage) override;
00058
00059 protected:
00060
00061 private: // methods
00062 private: // attributes 1:1
00063 private: // attributes 1:n
00064 };
00065
00066
00067 #endif /* EXIT_H */

```

11.346 Leave.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Leave.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 03 de Junho de 2019, 15:14
00012  */
00013
00014 #ifndef LEAVE_H
00015 #define LEAVE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../data/MaterialHandling/Station.h"
00019 #include "../kernel/simulator/essentialPlugins/Counter.h"
00020
00021 class Leave : public ModelComponent {
00022 public:
00023     Leave(Model* model, std::string name = "");
00024     virtual ~Leave() = default;
00025 public:
00026     virtual std::string show() override;
00027 public:
00028     static PluginInformation* GetPluginInformation();
00029     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00030     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00031 public:
00032     void setStation(Station* _station);
00033     void setStationName(std::string stationName);
00034     Station* getStation() const;
00035 protected:
00036     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00037     virtual bool _loadInstance(PersistenceRecord *fields) override;
00038     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00039 protected:
00040     //virtual void _initBetweenReplications();
00041 protected:
00042     virtual void _createInternalStatisticReporters() override;
00043     // virtual void _createNonEditableDataDefinitions() override;
00044     virtual void _createEditableDataDefinitions() override;
00045     // virtual void _createAttachedAttributes() override;
00046     virtual bool _check(std::string& errorMessage) override;
00047     // virtual void _createInternalAndAttachedData() override;
00048
00049 protected:
00050
00051 private: // association
00052     Station* _station = nullptr;
00053 private: // internal elements
00054     Counter* _numberIn = nullptr;
00055 };
00056
00057 #endif /* LEAVE_H */

```

11.347 PickableStationItem.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license

```

```

00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006  /*
00007  * File:    PickableStationItem.h
00008  * Author:  Prof. Rafael Luiz Cancian, Dr. Eng.
00009  *
00010  * Created on 1 de dezembro de 2022, 14:00
00011  */
00012
00013  #ifndef PICKABLESTATIONITEM_H
00014  #define PICKABLESTATIONITEM_H
00015
00016  #include "../kernel/simulator/model/Model.h"
00017  #include "plugins/data/DiscreteProcessing/Queue.h"
00018  #include "plugins/data/DiscreteProcessing/Resource.h"
00019  #include "../data/MaterialHandling/Station.h"
00020
00021  class PickableStationItem {
00022  public:
00023      PickableStationItem(Model* model, std::string stationName, std::string queueName = "", std::string
resourceName = "");
00024      PickableStationItem(Model* model, std::string stationName, std::string expression);
00025      PickableStationItem(Station* station, Queue* queue);
00026      PickableStationItem(Station* station, Resource* resource);
00027      PickableStationItem(Station* station, std::string expression);
00028      PickableStationItem(Station* station);
00029      virtual ~PickableStationItem() = default;
00030  public:
00031      std::string getExpression() const;
00032      Queue* getQueue() const;
00033      Resource* getResource() const;
00034      Station* getStation() const;
00035      void setExpression(std::string _expression);
00036      void setQueue(Queue* _queue);
00037      void setResource(Resource* _resource);
00038      void setStation(Station* _station);
00039
00040      std::string getName();
00041      void _addSimulationControl(SimulationControl* control);
00042      List<SimulationControl*> getSimulationControls() const;
00043
00044      std::string getTypeDC() {return _typeDC;};
00045  private:
00046  private:
00047      Station* _station = nullptr;
00048      Resource* _resource = nullptr;
00049      Queue* _queue = nullptr;
00050      std::string _expression = "";
00051      std::string _stationName = "";
00052
00053      std::string _typeDC = Util::TypeOf<Station>();
00054      List<SimulationControl*> _simulationControls = new List<SimulationControl*>();
00055  };
00056
00057  #endif /* PICKABLESTATIONITEM_H */

```

11.348 PickStation.h

```

00001  /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:    PickStation.h
00009  * Author:  Prof. Rafael Luiz Cancian, Dr. Eng.
00010  *
00011  * Created on 11 de Setembro de 2019, 13:08
00012  */
00013
00014  #ifndef PICKSTATION_H
00015  #define PICKSTATION_H
00016
00017  #include "../kernel/simulator/model/ModelComponent.h"
00018  #include "../kernel/simulator/model/ModelDataDefinition.h"
00019  #include "kernel/util/List.h"
00020  #include "plugins/data/DiscreteProcessing/Queue.h"
00021  #include "plugins/data/DiscreteProcessing/Resource.h"
00022
00023  #include "PickableStationItem.h"
00024

```

```

00068 class PickStation : public ModelComponent {
00069 public:
00070
00071     enum class TestCondition : int {
00072         MINIMUM = 0, MAXIMUM = 1, num_elements = 2
00073     };
00074
00075     //enum class PickCondition : int {
00076     //    NUMBER_BUSY_RESOURCE = 1, NUMBER_IN_QUEUE = 2, EXPRESSION = 3
00077     //};
00078 public:
00079     static std::string convertEnumToStr(TestCondition condition);
00080 public: // constructors
00081     PickStation(Model* model, std::string name = "");
00082     virtual ~PickStation() override;
00083 public: // virtual
00084     virtual std::string show() override;
00085 public:
00086     //void setPickCondition(PickStation::PickCondition _pickCondition);
00087     //PickStation::PickCondition getPickCondition() const;
00088     void setTestCondition(PickStation::TestCondition _testCondition);
00089     PickStation::TestCondition getTestCondition() const;
00090     void setSaveAttribute(std::string _saveAttribute);
00091     std::string getSaveAttribute() const;
00092     List<PickableStationItem*>* getPickableStationItems() const;
00093     void addPickableStationItem(PickableStationItem* newItem);
00094     void removePickableStationItem(PickableStationItem* item);
00095     void setPickConditionExpression(bool _pickConditionExpression);
00096     bool isPickConditionExpression() const;
00097     void setPickConditionNumberInQueue(bool _pickConditionNumberInQueue);
00098     bool isPickConditionNumberInQueue() const;
00099     void setPickConditionNumberBusyResource(bool _pickConditionNumberBusyResource);
00100     bool isPickConditionNumberBusyResource() const;
00101 public: // static
00102     static PluginInformation* GetPluginInformation();
00103     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00104     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00105
00106
00107 protected: // must be overridden
00108     virtual bool _loadInstance(PersistenceRecord *fields) override;
00109     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00110     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00111 protected: // could be overridden by derived classes
00112     virtual bool _check(std::string& errorMessage) override;
00113     virtual ParserChangesInformation* _getParserChangesInformation() override;
00114     virtual void _initBetweenReplications() override;
00115     // virtual void _createInternalAndAttachedData() override;
00116     virtual void _addSimulationControl(SimulationControl* property) override;
00117
00118
00119
00120
00121 protected:
00122     // virtual void _createInternalStatisticReporters() override;
00123     // virtual void _createNonEditableDataDefinitions() override;
00124     virtual void _createEditableDataDefinitions() override;
00125     // virtual void _createAttachedAttributes() override;
00126
00127 private: // methods
00128 private: // attributes 1:1
00129
00130     const struct DEFAULT_VALUES {
00131         const std::string saveAttribute = "";
00132         const PickStation::TestCondition testCondition = TestCondition::MINIMUM;
00133         bool pickConditionNumberBusyResource = false;
00134         bool pickConditionNumberInQueue = false;
00135         bool pickConditionExpression = true;
00136         //const PickStation::PickCondition pickCondition = PickCondition::NUMBER_BUSY_RESOURCE;
00137     } DEFAULT;
00138     std::string _saveAttribute = DEFAULT.saveAttribute;
00139     PickStation::TestCondition _testCondition = DEFAULT.testCondition;
00140     //PickStation::PickCondition _pickCondition = DEFAULT.pickCondition;
00141     bool _pickConditionNumberBusyResource = DEFAULT.pickConditionNumberBusyResource;
00142     bool _pickConditionNumberInQueue = DEFAULT.pickConditionNumberInQueue;
00143     bool _pickConditionExpression = DEFAULT.pickConditionExpression;
00144
00145     //DummyElement* _internalDataDefinition = nullptr;
00146 private: // attributes 1:n
00147     List<PickableStationItem*>* _pickableStationItems = new List<PickableStationItem*>();
00148 };
00149
00150
00151 #endif /* PICKSTATION_H */

```

11.349 Route.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Route.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 03 de Junho de 2019, 15:15
00012  */
00013
00014 #ifndef ROUTE_H
00015 #define ROUTE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../kernel/simulator/essentialPlugins/Counter.h"
00019 #include "../data/MaterialHandling/Station.h"
00020 #include "../data/MaterialHandling/Sequence.h"
00021 #include "../data/Logic/Label.h"
00022
00055 class Route : public ModelComponent {
00056 public:
00057
00058     enum class DestinationType : int {
00059         Station = 0, Sequence = 1, Label = 2, num_elements = 3
00060     };
00061 public:
00062     static std::string convertEnumToStr(DestinationType type);
00063 public:
00064     Route(Model* model, std::string name = "");
00065     virtual ~Route() = default;
00066 public:
00067     virtual std::string show() override;
00068 public:
00069     static PluginInformation* GetPluginInformation();
00070     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00071     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00072 public:
00073     void setStationName(std::string stationName);
00074     void setStation(Station* _station);
00075     Station* getStation() const;
00076     void setStationExpression(std::string _stationExpression);
00077     std::string getStationExpression() const;
00078     void setRouteTimeExpression(std::string _routeTimeExpression);
00079     void setRouteTimeExpression(std::string _routeTimeExpression, Util::TimeUnit _routeTimeUnit);
00080     std::string getRouteTimeExpression() const;
00081     void setRouteTimeUnit(Util::TimeUnit _routeTimeUnit);
00082     Util::TimeUnit getRouteTimeUnit() const;
00083     void setRouteDestinationType(DestinationType _routeDestinationType);
00084     Route::DestinationType getRouteDestinationType() const;
00085     void setLabel(Label* _label);
00086     Label* getLabel() const;
00087 protected:
00088     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00089     virtual bool _loadInstance(PersistenceRecord *fields) override;
00090     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00091 protected:
00092     //virtual void _initBetweenReplications();
00093     virtual bool _check(std::string& errorMessage) override;
00094
00095 protected:
00096     virtual void _createInternalStatisticReporters() override;
00097     // virtual void _createNonEditableDataDefinitions() override;
00098     virtual void _createEditableDataDefinitions() override;
00099     virtual void _createAttachedAttributes() override;
00100     // virtual void _createInternalAndAttachedData() override;
00101
00102 protected:
00103
00104 private:
00105
00106     const struct DEFAULT_VALUES {
00107         const std::string routeTimeExpression = "0.0";
00108         const std::string stationExpression = "";
00109         const Util::TimeUnit routeTimeUnit = Util::TimeUnit::second;
00110         const Route::DestinationType routeDestinationType = DestinationType::Station;
00111     } DEFAULT;
00112     std::string _routeTimeExpression = DEFAULT.routeTimeExpression;
00113     std::string _stationExpression = DEFAULT.stationExpression;
00114     Util::TimeUnit _routeTimeUnit = DEFAULT.routeTimeUnit;
00115     Route::DestinationType _routeDestinationType = DEFAULT.routeDestinationType;
00116 private: // association
00117     Station* _station = nullptr;

```

```

00118     Label* _label = nullptr;
00119 private: // internal elements
00120     Counter* _numberIn = nullptr;
00121 };
00122
00123 #endif /* ROUTE_H */

```

11.350 Start.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:      Start.h
00009  * Author:    rlcancian
00010  *
00011  * Created on 11 de Setembro de 2019, 13:15
00012  */
00013
00014 #ifndef START_H
00015 #define START_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00039 class Start : public ModelComponent {
00040 public: // constructors
00041     Start(Model* model, std::string name = "");
00042     virtual ~Start() = default;
00043 public: // virtual
00044     virtual std::string show() override;
00045 public: // static
00046     static PluginInformation* GetPluginInformation();
00047     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00048     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00049 protected: // virtual
00050     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00051     virtual bool _loadInstance(PersistenceRecord *fields) override;
00052     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00053 protected: // virtual
00054     //virtual void _initBetweenReplications();
00055     // virtual void _createInternalStatisticReporters() override;
00056     // virtual void _createNonEditableDataDefinitions() override;
00057     // virtual void _createEditableDataDefinitions() override;
00058     // virtual void _createAttachedAttributes() override;
00059     virtual bool _check(std::string& errorMessage) override;
00060
00061 protected:
00062
00063 private: // methods
00064 private: // attributes 1:1
00065 private: // attributes 1:n
00066 };
00067
00068
00069 #endif /* START_H */

```

11.351 Stop.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:      Stop.h
00009  * Author:    rlcancian
00010  *
00011  * Created on 11 de Setembro de 2019, 13:15
00012  */
00013
00014 #ifndef STOP_H
00015 #define STOP_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018

```



```

00036 class Stop : public ModelComponent {
00037 public: // constructors
00038     Stop(Model* model, std::string name = "");
00039     virtual ~Stop() = default;
00040 public: // virtual
00041     virtual std::string show() override;
00042 public: // static
00043     static PluginInformation* GetPluginInformation();
00044     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00045     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00046 protected: // virtual
00047     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00048     virtual bool _loadInstance(PersistenceRecord *fields) override;
00049     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00050 protected: // virtual
00051     //virtual void _initBetweenReplications();
00052     // virtual void _createInternalStatisticReporters() override;
00053     // virtual void _createNonEditableDataDefinitions() override;
00054     // virtual void _createEditableDataDefinitions() override;
00055     // virtual void _createAttachedAttributes() override;
00056     virtual bool _check(std::string& errorMessage) override;
00057
00058 protected:
00059
00060 private: // methods
00061 private: // attributes 1:1
00062 private: // attributes 1:n
00063 };
00064
00065
00066 #endif /* STOP_H */

```

11.352 Store.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Store.h
00009  * Author: rlcancian
00010  *
00011  * Created on 11 de Setembro de 2019, 13:07
00012  */
00013
00014 #ifndef STORE_H
00015 #define STORE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00050 class Store : public ModelComponent {
00051 public: // constructors
00052     Store(Model* model, std::string name = "");
00053     virtual ~Store() = default;
00054 public: // virtual
00055     virtual std::string show() override;
00056 public: // static
00057     static PluginInformation* GetPluginInformation();
00058     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00059     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00060 protected: // virtual
00061     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00062     virtual bool _loadInstance(PersistenceRecord *fields) override;
00063     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00064 protected: // virtual
00065     //virtual void _initBetweenReplications();
00066     // virtual void _createInternalStatisticReporters() override;
00067     // virtual void _createNonEditableDataDefinitions() override;
00068     // virtual void _createEditableDataDefinitions() override;
00069     // virtual void _createAttachedAttributes() override;
00070     virtual bool _check(std::string& errorMessage) override;
00071
00072 protected:
00073
00074 private: // methods
00075 private: // attributes 1:1
00076 private: // attributes 1:n
00077 };
00078
00079
00080 #endif /* STORE_H */

```

11.353 Unstore.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   Unstore.h
00009  * Author: rlcancian
00010  *
00011  * Created on 11 de Setembro de 2019, 13:08
00012  */
00013
00014 #ifndef UNSTORE_H
00015 #define UNSTORE_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00045 class Unstore : public ModelComponent {
00046 public: // constructors
00047     Unstore(Model* model, std::string name = "");
00048     virtual ~Unstore() = default;
00049 public: // virtual
00050     virtual std::string show() override;
00051 public: // static
00052     static PluginInformation* GetPluginInformation();
00053     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00054     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00055 protected: // virtual
00056     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00057     virtual bool _loadInstance(PersistenceRecord *fields) override;
00058     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00059 protected: // virtual
00060     //virtual void _initBetweenReplications();
00061     // virtual void _createInternalStatisticReporters() override;
00062     // virtual void _createNonEditableDataDefinitions() override;
00063     // virtual void _createEditableDataDefinitions() override;
00064     // virtual void _createAttachedAttributes() override;
00065     virtual bool _check(std::string& errorMessage) override;
00066
00067 protected:
00068
00069 private: // methods
00070 private: // attributes 1:1
00071 private: // attributes 1:n
00072 };
00073
00074
00075 #endif /* UNSTORE_H */

```

11.354 Boundary_Closed.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File:   Boundary_Closed.h
00008  * Author: cancian
00009  *
00010  * Created on 10 de abril de 2023, 16:22
00011  */
00012 #pragma once
00013
00014 #include "plugins/components/ModalModel/CellularAutomata/BoundaryCondition.h"
00015 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00016 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00017 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00018
00019 class Boundary_Closed : public BoundaryCondition {
00020 public:
00021
00022     Boundary_Closed(Lattice* lattice = nullptr, Neighborhood* neighborhood = nullptr)
00023     : BoundaryCondition(lattice, neighborhood) {
00024         fixedCell = new Cell();
00025     }
00026
00027     Boundary_Closed(const Boundary_Closed& orig)
00028     : BoundaryCondition(orig) {
00029
00030     }
00031
00032
00033
00034
00035
00036
00037
00038
00039
00040
00041
00042
00043
00044
00045
00046
00047
00048
00049
00050
00051
00052
00053
00054
00055
00056
00057
00058
00059
00060
00061
00062
00063
00064
00065
00066
00067
00068
00069
00070
00071
00072
00073
00074
00075
00076
00077
00078
00079
00080
00081
00082
00083
00084
00085
00086
00087
00088
00089
00090
00091
00092
00093
00094
00095
00096
00097
00098
00099
00100
00101
00102
00103
00104
00105
00106
00107
00108
00109
00110
00111
00112
00113
00114
00115
00116
00117
00118
00119
00120
00121
00122
00123
00124
00125
00126
00127
00128
00129
00130
00131
00132
00133
00134
00135
00136
00137
00138
00139
00140
00141
00142
00143
00144
00145
00146
00147
00148
00149
00150
00151
00152
00153
00154
00155
00156
00157
00158
00159
00160
00161
00162
00163
00164
00165
00166
00167
00168
00169
00170
00171
00172
00173
00174
00175
00176
00177
00178
00179
00180
00181
00182
00183
00184
00185
00186
00187
00188
00189
00190
00191
00192
00193
00194
00195
00196
00197
00198
00199
00200
00201
00202
00203
00204
00205
00206
00207
00208
00209
00210
00211
00212
00213
00214
00215
00216
00217
00218
00219
00220
00221
00222
00223
00224
00225
00226
00227
00228
00229
00230
00231
00232
00233
00234
00235
00236
00237
00238
00239
00240
00241
00242
00243
00244
00245
00246
00247
00248
00249
00250
00251
00252
00253
00254
00255
00256
00257
00258
00259
00260
00261
00262
00263
00264
00265
00266
00267
00268
00269
00270
00271
00272
00273
00274
00275
00276
00277
00278
00279
00280
00281
00282
00283
00284
00285
00286
00287
00288
00289
00290
00291
00292
00293
00294
00295
00296
00297
00298
00299
00300
00301
00302
00303
00304
00305
00306
00307
00308
00309
00310
00311
00312
00313
00314
00315
00316
00317
00318
00319
00320
00321
00322
00323
00324
00325
00326
00327
00328
00329
00330
00331
00332
00333
00334
00335
00336
00337
00338
00339
00340
00341
00342
00343
00344
00345
00346
00347
00348
00349
00350
00351
00352
00353
00354
00355
00356
00357
00358
00359
00360
00361
00362
00363
00364
00365
00366
00367
00368
00369
00370
00371
00372
00373
00374
00375
00376
00377
00378
00379
00380
00381
00382
00383
00384
00385
00386
00387
00388
00389
00390
00391
00392
00393
00394
00395
00396
00397
00398
00399
00400
00401
00402
00403
00404
00405
00406
00407
00408
00409
00410
00411
00412
00413
00414
00415
00416
00417
00418
00419
00420
00421
00422
00423
00424
00425
00426
00427
00428
00429
00430
00431
00432
00433
00434
00435
00436
00437
00438
00439
00440
00441
00442
00443
00444
00445
00446
00447
00448
00449
00450
00451
00452
00453
00454
00455
00456
00457
00458
00459
00460
00461
00462
00463
00464
00465
00466
00467
00468
00469
00470
00471
00472
00473
00474
00475
00476
00477
00478
00479
00480
00481
00482
00483
00484
00485
00486
00487
00488
00489
00490
00491
00492
00493
00494
00495
00496
00497
00498
00499
00500
00501
00502
00503
00504
00505
00506
00507
00508
00509
00510
00511
00512
00513
00514
00515
00516
00517
00518
00519
00520
00521
00522
00523
00524
00525
00526
00527
00528
00529
00530
00531
00532
00533
00534
00535
00536
00537
00538
00539
00540
00541
00542
00543
00544
00545
00546
00547
00548
00549
00550
00551
00552
00553
00554
00555
00556
00557
00558
00559
00560
00561
00562
00563
00564
00565
00566
00567
00568
00569
00570
00571
00572
00573
00574
00575
00576
00577
00578
00579
00580
00581
00582
00583
00584
00585
00586
00587
00588
00589
00590
00591
00592
00593
00594
00595
00596
00597
00598
00599
00600
00601
00602
00603
00604
00605
00606
00607
00608
00609
00610
00611
00612
00613
00614
00615
00616
00617
00618
00619
00620
00621
00622
00623
00624
00625
00626
00627
00628
00629
00630
00631
00632
00633
00634
00635
00636
00637
00638
00639
00640
00641
00642
00643
00644
00645
00646
00647
00648
00649
00650
00651
00652
00653
00654
00655
00656
00657
00658
00659
00660
00661
00662
00663
00664
00665
00666
00667
00668
00669
00670
00671
00672
00673
00674
00675
00676
00677
00678
00679
00680
00681
00682
00683
00684
00685
00686
00687
00688
00689
00690
00691
00692
00693
00694
00695
00696
00697
00698
00699
00700
00701
00702
00703
00704
00705
00706
00707
00708
00709
00710
00711
00712
00713
00714
00715
00716
00717
00718
00719
00720
00721
00722
00723
00724
00725
00726
00727
00728
00729
00730
00731
00732
00733
00734
00735
00736
00737
00738
00739
00740
00741
00742
00743
00744
00745
00746
00747
00748
00749
00750
00751
00752
00753
00754
00755
00756
00757
00758
00759
00760
00761
00762
00763
00764
00765
00766
00767
00768
00769
00770
00771
00772
00773
00774
00775
00776
00777
00778
00779
00780
00781
00782
00783
00784
00785
00786
00787
00788
00789
00790
00791
00792
00793
00794
00795
00796
00797
00798
00799
00800
00801
00802
00803
00804
00805
00806
00807
00808
00809
00810
00811
00812
00813
00814
00815
00816
00817
00818
00819
00820
00821
00822
00823
00824
00825
00826
00827
00828
00829
00830
00831
00832
00833
00834
00835
00836
00837
00838
00839
00840
00841
00842
00843
00844
00845
00846
00847
00848
00849
00850
00851
00852
00853
00854
00855
00856
00857
00858
00859
00860
00861
00862
00863
00864
00865
00866
00867
00868
00869
00870
00871
00872
00873
00874
00875
00876
00877
00878
00879
00880
00881
00882
00883
00884
00885
00886
00887
00888
00889
00890
00891
00892
00893
00894
00895
00896
00897
00898
00899
00900
00901
00902
00903
00904
00905
00906
00907
00908
00909
00910
00911
00912
00913
00914
00915
00916
00917
00918
00919
00920
00921
00922
00923
00924
00925
00926
00927
00928
00929
00930
00931
00932
00933
00934
00935
00936
00937
00938
00939
00940
00941
00942
00943
00944
00945
00946
00947
00948
00949
00950
00951
00952
00953
00954
00955
00956
00957
00958
00959
00960
00961
00962
00963
00964
00965
00966
00967
00968
00969
00970
00971
00972
00973
00974
00975
00976
00977
00978
00979
00980
00981
00982
00983
00984
00985
00986
00987
00988
00989
00990
00991
00992
00993
00994
00995
00996
00997
00998
00999

```

```

00030     virtual ~Boundary_Closed() = default;
00031 public:
00032     virtual Cell* getNeighborCell(std::vector<int> cellNDimPosition, std::vector<int>
neighborNDimPosition) override {
00033         long neighborCellNumber = lattice->cellNDimPosition2Number(neighborNDimPosition);
00034         if (neighborCellNumber < 0) { // out of bounds
00035             int pos, maxPos;
00036             for (unsigned short dim = 0; dim < neighborNDimPosition.size(); dim++) {
00037                 pos = neighborNDimPosition.at(dim);
00038                 maxPos = (int) (lattice->getDimension(dim));
00039                 if (pos < 0 || pos >= maxPos) { // out of bound
00040                     // Keep periodic wrapping valid for any radius, including large negative offsets.
00041                     pos = ((pos % maxPos) + maxPos) % maxPos;
00042                     // position corrected
00043                     neighborNDimPosition.at(dim) = pos;
00044                 }
00045             }
00046             //std::cout << " bond fixed to " <<Util::showNDimPosition(&neighborNDimPosition);
00047             neighborCellNumber = lattice->cellNDimPosition2Number(neighborNDimPosition);
00048         }
00049         return lattice->getCell(neighborCellNumber);
00050     }
00051 public:
00052
00053     void setFixedState(const State& fixedState) {
00054         fixedCell->setCurrentState(fixedState);
00055     }
00056
00057     const State& getFixedState() const {
00058         return fixedCell->getCurrentState();
00059     }
00060 private:
00061     Cell* fixedCell;
00062 };

```

11.355 Boundary_Fixed.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File:    Boundary_Fixed.h
00008  * Author:  cancian
00009  *
00010  * Created on 6 de abril de 2023, 13:48
00011  */
00012 #pragma once
00013
00014 #include "plugins/components/ModalModel/CellularAutomata/BoundaryCondition.h"
00015 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00016 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00017 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00018
00019 class Boundary_Fixed : public BoundaryCondition {
00020 public:
00021
00022     Boundary_Fixed(Lattice* lattice = nullptr, Neighborhood* neighborhood = nullptr)
00023     : BoundaryCondition(lattice, neighborhood) {
00024         fixedCell = new Cell();
00025         fixedCell->setCellNumber(-99);
00026     }
00027
00028     Boundary_Fixed(const Boundary_Fixed& orig)
00029     : BoundaryCondition(orig) {
00030     }
00031     virtual ~Boundary_Fixed() = default;
00032 public:
00033
00034     virtual Cell* getNeighborCell(std::vector<int> cellNDimPosition, std::vector<int>
neighborNDimPosition) override {
00035         long neighborCellNumber = lattice->cellNDimPosition2Number(neighborNDimPosition);
00036         if (neighborCellNumber < 0) { // out of bounds
00037             //std::cout << " bond fixed to fixedCell";
00038             return fixedCell;
00039         }
00040         return lattice->getCell(neighborCellNumber);
00041     }
00042 public:
00043
00044     void setFixedState(const State& fixedState) {
00045         fixedCell->setCurrentState(fixedState);

```

```

00046     }
00047
00048     const State& getFixedState() const {
00049         return fixedCell->getCurrentState();
00050     }
00051 private:
00052     Cell* fixedCell;
00053 };

```

11.356 BoundaryCondition.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00004 // #include "plugins/components/ModalModel/CellularAutomata/Neighborhood.h"
00005 #include <string>
00006 #include <vector>
00007
00008
00009 class Neighborhood;
00010
00011 class BoundaryCondition {
00012 public:
00013     BoundaryCondition(Lattice* lattice = nullptr, Neighborhood* neighborhood=nullptr){
00014         this->lattice = lattice;
00015         this->neighborhood = neighborhood;
00016     }
00017     BoundaryCondition(const BoundaryCondition& orig){}
00018     virtual ~BoundaryCondition()=default;
00019 public:
00020     virtual std::string show() {
00021         return "todo Boundary.show()";
00022     }
00023     virtual Cell* getNeighborCell(std::vector<int> cellPosition, std::vector<int>
neighborCellPosition) = 0;
00024 public:
00025     void setNeighborhood(Neighborhood* neighborhood){
00026         this->neighborhood = neighborhood;
00027     }
00028     Neighborhood* getNeighborhood() const{
00029         return neighborhood;
00030     }
00031     void setLattice(Lattice* lattice){
00032         this->lattice = lattice;
00033     }
00034     Lattice* getLattice() const{
00035         return lattice;
00036     }
00037 protected:
00038     Lattice* lattice;
00039     Neighborhood* neighborhood;
00040 private:
00041 };

```

11.357 Cell.h

```

00001 #pragma once
00002
00003
00004
00005 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00006 #include "plugins/components/ModalModel/CellularAutomata/StateSet.h"
00007 #include <memory>
00008 #include <vector>
00009 #include <string>
00010
00011 class State;
00012
00013 class Cell {
00014 public: // constructors
00015     Cell(StateSet* stateSet = nullptr, long cellNum=0, std::vector<int> position={});
00016     Cell(long cellNum, std::vector<int> position);
00017     Cell(const Cell& orig);
00018     Cell& operator=(const Cell& orig);
00019     virtual ~Cell()=default;
00020 public: // methods
00021     virtual std::string show();
00022     bool updateState();
00023     bool isUpdatePending();

```

```

00024     virtual void draw();
00025 public: // getters
00026     void setCellNumber(long cellNum);
00027     long getCellNumber() const;
00028     void setPosition(std::vector<int> position);
00029     std::vector<int> getPosition() const;
00030     void setStateSet(StateSet* stateSet);
00031     StateSet* getStateSet() const;
00032     const State& getPreviousState() const;
00033     bool setCurrentState(const State& currentState);
00034     const State& getCurrentState() const;
00035     bool setNextState(const State& nextState);
00036     const State& getNextState() const;
00037     void setNeighbors(std::vector<Cell*> neighbors);
00038     std::vector<Cell*> getNeighbors() const;
00039 protected:
00040     bool acceptsState(const State& state) const;
00041     std::unique_ptr<State> previousState;
00042     std::unique_ptr<State> currentState;
00043     std::unique_ptr<State> nextState;
00044     StateSet* stateSet = nullptr;
00045     bool updatePending;
00046     std::vector<int> position;
00047     long cellNum;
00048     std::vector<Cell*> neighbors;
00049 protected: // for other CA types
00050 private:
00051 };

```

11.358 CellularAutomata.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/StateSet.h"
00005 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood.h"
00006 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00007 #include <string>
00008 #include <functional>
00009 #include <list>
00010 #include <thread>
00011
00012 class CellularAutomata;
00013
00014 class CellularAutomataEvent { // event for observers
00015 public:
00016     CellularAutomataEvent(CellularAutomata* ca) {
00017         this->ca = ca;
00018     }
00019     CellularAutomata* ca;
00020 };
00021
00022 typedef std::function<void(CellularAutomataEvent)> CellularAutomataEventObserver;
00023
00024 class CellularAutomata {
00025 public:
00026     CellularAutomata(std::string name = "", Lattice* lattice = nullptr, StateSet* stateSet = nullptr,
00027                     Neighborhood* neighborhood = nullptr, LocalRule* localRule = nullptr);
00028     CellularAutomata(const CellularAutomata& orig);
00029     virtual ~CellularAutomata() = default;
00030 public:
00031     //virtual std::string show();
00032     virtual bool init();
00033     virtual void step();
00034     virtual void start();
00035     virtual void pause();
00036     template<typename Class> void addObserver(Class* object, void
00037         (Class::*function)(CellularAutomataEvent));
00038 public:
00039     std::string getName() const;
00040     void setName(const std::string name);
00041     void setLocalRule(LocalRule* localRule);
00042     LocalRule* getLocalRule() const;
00043     void setNeighborhood(Neighborhood* neighborhood);
00044     Neighborhood* getNeighborhood() const;
00045     void setStateSet(StateSet* stateSet);
00046     StateSet* getStateSet() const;
00047     void setLattice(Lattice* lattice);
00048     Lattice* getLattice() const;
00049     void setSimulationLength(unsigned int simulationLength);
00050     unsigned int getSimulationLength() const;
00051     unsigned int getSimulatedTime() const;
00052     bool isRunning() const;

```

```

00051 protected:
00052     void notifyObservers(CellularAutomataEvent event);
00053     virtual void applyLocalRule() = 0;
00054 protected:
00055     Lattice* lattice;
00056     StateSet* stateSet;
00057     Neighborhood* neighborhood;
00058     LocalRule* localRule;
00059     std::string name;
00060     unsigned int simulatedTime;
00061     unsigned int simulationLength;
00062     bool running;
00063 private:
00064     std::list<CellularAutomataEventObserver>* observers = new
std::list<CellularAutomataEventObserver>();
00065 };
00066
00067 template<typename Class> void CellularAutomata::addObserver(Class * object, void
(Class::*function)(CellularAutomataEvent)) {
00068     CellularAutomataEventObserver observer = std::bind(function, object, std::placeholders::_1);
00069     observers->emplace_back(observer);
00070 }

```

11.359 CellularAutomata_1DTimed.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h"
00005 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00006 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00008
00009 class CellularAutomata_1DTimed: public CellularAutomataBase {
00010 public:
00011     CellularAutomata_1DTimed(Lattice* lattice = nullptr, StateSet* stateSet = nullptr, Neighborhood*
neighborhood = nullptr, LocalRule* localRule = nullptr)
00012 : CellularAutomataBase(lattice, stateSet, neighborhood, localRule) { }
00013     CellularAutomata_1DTimed(const CellularAutomata_1DTimed& orig)
00014 : CellularAutomataBase(orig) { }
00015     virtual ~CellularAutomata_1DTimed() = default;
00016 protected:
00017     virtual void applyLocalRule() override {
00018         Cell* cell;
00019         int nextRow;
00020         int maxRows = std::min<int>(lattice->getDimension(1)-2, lattice->getDimension(1)-2); //
simulatedTime);
00021         for (int row=maxRows; row>=0; row--) {
00022             nextRow = row + 1;
00023             // move cell states to the next row
00024             for (int col=0; col<lattice->getDimension(0); col++) { // move to row below
00025                 cell = lattice->getCell({col,row});
00026                 lattice->setCellState({col,nextRow}, const_cast<State*>(&cell->getCurrentState()));
00027             }
00028             if (row==0) {
00029                 // apply the same local rule for all unidimensional cells at the same time
00030                 for (int col=0; col<lattice->getDimension(0); col++) {
00031                     cell = lattice->getCell({col,row});
00032                     localRule->applyRule(cell);
00033                 }
00034             }
00035         }
00036         // alter changing the nextState of all unidimensional cells, update them (currentState =
nextState)
00037         for (int col=0; col<lattice->getDimension(0); col++) {
00038             cell = lattice->getCell({col,0});
00039             cell->updateState();
00040         }
00041     }
00042 protected:
00043 };

```

11.360 CellularAutomata_Classic.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00005

```

```

00007 class CellularAutomata_Classic: public CellularAutomataBase {
00008 public:
00009     CellularAutomata_Classic(Lattice* lattice = nullptr, StateSet* stateSet = nullptr, Neighborhood*
neighborhood = nullptr, LocalRule* localRule = nullptr)
00010     :CellularAutomataBase(lattice, stateSet, neighborhood, localRule) { }
00011     CellularAutomata_Classic(const CellularAutomata_Classic& orig): CellularAutomataBase(orig){ }
00012     virtual ~CellularAutomata_Classic() = default;
00013 protected:
00014     virtual void applyLocalRule() override {
00015         Cell* cell;
00016         // apply the same local rule to all cells at the same time
00017         for (long cellNumber = 0; cellNumber < lattice->getCellsSize(); cellNumber++) {
00018             cell = lattice->getCell(cellNumber);
00019             localRule->applyRule(cell);
00020         }
00021         // alter changing the nextState of all cells, update them (currentState = nextState)
00022         for (long cellNumber = 0; cellNumber < lattice->getCellsSize(); cellNumber++) {
00023             cell = lattice->getCell(cellNumber);
00024             cell->updateState();
00025         }
00026     }
00027 };

```

11.361 CellularAutomataAllHeaders.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/BoundaryCondition.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/Boundary_Closed.h"
00005 #include "plugins/components/ModalModel/CellularAutomata/Boundary_Fixed.h"
00006 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomata.h"
00008 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h"
00009 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomata_1DTimed.h"
00010 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomata_Classic.h"
00011 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00012 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00013 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_Elementary.h"
00014 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_FlorestalFire.h"
00015 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_GameOfLife.h"
00016 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_Growty.h"
00017 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_HppLatticeGas.h"
00018 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_SandRock_Student.h"
00019 #include "plugins/components/ModalModel/CellularAutomata/LocalRule_ShowNeighborhood.h"
00020 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood.h"
00021 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood_Center.h"
00022 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood_Moore.h"
00023 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood_VonNeumann.h"
00024 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00025 #include "plugins/components/ModalModel/CellularAutomata/StateSet.h"
00026 #include "plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h"
00027 #include "plugins/components/ModalModel/CellularAutomata/State_Numerical.h"

```

11.362 CellularAutomataBase.h

```

00001 #pragma once
00002
00003
00004 #include <string>
00005 #include <functional>
00006 #include <list>
00007 #include <thread>
00008
00009 class Lattice;
00010 class StateSet;
00011 class Neighborhood;
00012 class LocalRule;
00013
00014
00015 class CellularAutomataBase {
00016 public:
00017     CellularAutomataBase(Lattice* lattice = nullptr, StateSet* stateSet = nullptr, Neighborhood*
neighborhood = nullptr, LocalRule* localRule = nullptr);
00018     CellularAutomataBase(const CellularAutomataBase& orig);
00019     virtual ~CellularAutomataBase() = default;
00020 public:
00021     //virtual std::string show();
00022     virtual bool init();
00023     virtual void step();

```

```

00024 public:
00025     void setLocalRule(LocalRule* localRule);
00026     LocalRule* getLocalRule() const;
00027     void setNeighborhood(Neighborhood* neighborhood);
00028     Neighborhood* getNeighborhood() const;
00029     void setStateSet(StateSet* stateSet);
00030     StateSet* getStateSet() const;
00031     void setLattice(Lattice* lattice);
00032     Lattice* getLattice() const;
00033 protected:
00034     virtual void applyLocalRule() = 0;
00035 protected:
00036     Lattice* lattice;
00037     StateSet* stateSet;
00038     Neighborhood* neighborhood;
00039     LocalRule* localRule;
00040 };

```

11.363 Lattice.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00004 #include <vector>
00005 #include <string>
00006
00007 class CellularAutomataBase;
00008
00009 enum class LatticeType : int {
00010     RETICULAR = 1, TRIANGULAR = 2, HEXAGONAL = 3, NETWORK = 4, USERDEFINED = 5
00011 };
00012
00014 class Lattice {
00015 public:
00017     Lattice(CellularAutomataBase* parentCellularAutomata, Cell* progenitorCell = nullptr,
std::vector<unsigned short> dimensions={}, LatticeType latticeType = LatticeType::RETICULAR);
00018     Lattice(const Lattice& orig);
00019     virtual ~Lattice() = default;
00020 public:
00021     virtual std::string show();
00022     bool init();
00023     virtual long cellNDimPosition2Number(const std::vector<int> position);
00024     virtual std::vector<int> cellNumber2NDimPosition(const long cellNumber);
00025     Cell* getCell(const long cellNumber);
00026     Cell* getCell(const std::vector<int> position);
00027     void setCell(const std::vector<int> position, Cell* cell);
00028     bool setCellState(std::vector<int> position, State* state = nullptr, Cell* cell = nullptr);
00029     bool setCellState(long cellNumber, State* state = nullptr, Cell* cell = nullptr);
00030 public:
00031     void setProgenitorCell(Cell* progenitorCell);
00032     Cell* getProgenitorCell() const;
00033     bool setNumCells(unsigned short dimension, unsigned int numCells, bool createDimensions = true);
00034     unsigned short getNumDimensions();
00035     unsigned int getNumCell(unsigned short dimension);
00036     LatticeType getLatticeType() const;
00037     unsigned long getCellsSize() const;
00038     unsigned long getTotalCells() const;
00039     unsigned long calculateTotalCells();
00040     bool isInit() const;
00041     void setDimensions(std::vector<unsigned short> dimensions);
00042     std::vector<unsigned short> getDimensions() const;
00043     unsigned short getDimension(unsigned short dimensionNumber) const;
00044     std::vector<Cell*> getCells() const;
00045 protected:
00046     CellularAutomataBase* parentCellularAutomata;
00047     std::vector<Cell*> cells;
00048     std::vector<unsigned short> dimensions;
00049     Cell* progenitorCell;
00050     LatticeType latticeType;
00051     bool hasBeenInit;
00052     unsigned long totalCells;
00053 };

```

11.364 LocalRule.h

```

00001 #pragma once
00002 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00003 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h"

```



```

00004
00005
00006 class LocalRule {
00007 public:
00008     LocalRule(CellularAutomataBase* parentCellularAutomata, StateSet* stateSet=nullptr){
00009         this->parentCellularAutomata = parentCellularAutomata;
00010         parentCellularAutomata->setLocalRule(this);
00011         if (stateSet != nullptr)
00012             this->stateSet = stateSet;
00013         else
00014             this->stateSet = parentCellularAutomata->getStateSet();
00015     }
00016     LocalRule(const LocalRule& orig){}
00017     virtual ~LocalRule()=default;
00018 public:
00019     virtual void applyRule(Cell* cell) = 0;
00020 public:
00021     void setStateSet(StateSet* stateSet){
00022         this->stateSet = stateSet;
00023     }
00024 protected:
00025     CellularAutomataBase* parentCellularAutomata;
00026     StateSet* stateSet;
00027 };

```

11.365 LocalRule_Elementary.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00005 #include "plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h"
00006 #include <cmath>
00007
00008 class LocalRule_Elementary: public LocalRule {
00009 public:
00010     LocalRule_Elementary(CellularAutomataBase* parentCellularAutomata, uint8_t ruleNumber=30,
00011         StateSet* stateSet=nullptr)
00012         :LocalRule(parentCellularAutomata, stateSet) {
00013             this->ruleNumber = ruleNumber;
00014         }
00015     LocalRule_Elementary(const LocalRule_Elementary& orig): LocalRule(orig) { }
00016     virtual ~LocalRule_Elementary()=default;
00017 public:
00018     virtual void applyRule(Cell* cell) override {
00019         long number = 0;
00020         int bit, power = 2;
00021         for (Cell* neigh : cell->getNeighbors()) {
00022             number += neigh->getCurrentState().getValue() * std::pow(2, power--);
00023             if (power==1)
00024                 number += std::pow(2, power--)*cell->getCurrentState().getValue();
00025         }
00026         bit = (ruleNumber & (uint8_t)std::pow(2,number)) » number;
00027         cell->setNextState(State(bit));
00028     }
00029 public:
00030     uint8_t getRuleNumber() const { return ruleNumber; }
00031     void setRuleNumber(const uint8_t ruleNumber) {this->ruleNumber = ruleNumber;}
00032 private:
00033     uint8_t ruleNumber;
00034 };

```

11.366 LocalRule_FlorestalFire.h

```

00001 #pragma once
00002
00003 #include <random>
00004 #include <vector>
00005
00006 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00008 #include "plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h"
00009
00010 class LocalRule_FlorestalFire : public LocalRule {
00011 public:
00012
00013     LocalRule_FlorestalFire(CellularAutomataBase* parentCellularAutomata)
00014     : LocalRule(parentCellularAutomata) {
00015     }

```

```

00016
00017     LocalRule_FlorestalFire(const LocalRule_FlorestalFire& orig)
00018     : LocalRule(orig) {
00019     }
00020     virtual ~LocalRule_FlorestalFire() = default;
00021 public:
00022
00023     virtual void applyRule(Cell* cell) override {
00024         if (cell == nullptr || stateSet == nullptr) {
00025             return;
00026         }
00027
00028         auto* enumerableStateSet = dynamic_cast<StateSet_Enumerable*>(stateSet);
00029         if (enumerableStateSet == nullptr || enumerableStateSet->getStatesSize() < 4) {
00030             cell->setNextState(cell->getCurrentState());
00031             return;
00032         }
00033
00034         std::random_device randomDevice;
00035         std::default_random_engine engine(randomDevice());
00036         std::uniform_real_distribution<double> uniform(0.0, 1.0);
00037         std::vector<unsigned int> sumNeighStates(enumerableStateSet->getStatesSize(), 0);
00038
00039         for (Cell* neigh : cell->getNeighbors()) {
00040             const long stateValue = neigh->getCurrentState().getValue();
00041             if (stateValue >= 0 && static_cast<size_t>(stateValue) < sumNeighStates.size()) {
00042                 sumNeighStates.at(static_cast<size_t>(stateValue))++;
00043             }
00044         }
00045
00046         const long currentState = cell->getCurrentState().getValue();
00047         cell->setNextState(cell->getCurrentState());
00048
00049         switch (currentState) {
00050             case 0: // soil
00051                 if (sumNeighStates.size() > 1 && uniform(engine) < pSoil2Grass + sumNeighStates.at(1)
00052 * pSoil2Grass) {
00053                     cell->setNextState(*enumerableStateSet->getState(1));
00054                 }
00055                 break;
00056             case 1: // grass
00057                 if (sumNeighStates.size() > 2 && uniform(engine) < pGrass2Tree + sumNeighStates.at(2)
00058 * pGrass2Tree) {
00059                     cell->setNextState(*enumerableStateSet->getState(2));
00060                 } else if (sumNeighStates.size() > 3 &&
00061 uniform(engine) < pGrass2Fire + (sumNeighStates.at(3) > 0 ?
00062 pGrassPropagateFire : 0.0)) {
00063                     cell->setNextState(*enumerableStateSet->getState(3));
00064                 }
00065                 break;
00066             case 2: // tree
00067                 if (sumNeighStates.size() > 3 &&
00068 uniform(engine) < pTree2Fire + (sumNeighStates.at(3) > 0 ? pTreePropagateFire :
00069 0.0)) {
00070                     cell->setNextState(*enumerableStateSet->getState(3));
00071                 } else if (sumNeighStates.size() > 0 && uniform(engine) < pTree2Soil +
00072 sumNeighStates.at(0) * pTree2Soil) {
00073                     cell->setNextState(*enumerableStateSet->getState(0));
00074                 }
00075                 break;
00076             case 3: // fire
00077                 cell->setNextState(*enumerableStateSet->getState(0));
00078                 break;
00079             default:
00080                 break;
00081         }
00082     }
00083 protected:
00084 private:
00085     double pSoil2Grass = 0.01;
00086     double pGrass2Tree = 0.001;
00087     double pGrass2Fire = 0.00001;
00088     double pTree2Fire = 0.00001;
00089     double pGrassPropagateFire = 0.05;
00090     double pTreePropagateFire = 0.5;
00091     double pTree2Soil = 0.0005;
00092 };

```

11.367 LocalRule_GameOfLife.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"

```

```

00004 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00005 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00006
00007 class LocalRule_GameOfLife: public LocalRule {
00008 public:
00009     LocalRule_GameOfLife(CellularAutomataBase* parentCellularAutomata)
00010         : LocalRule(parentCellularAutomata) { }
00011     LocalRule_GameOfLife(const LocalRule_GameOfLife& orig): LocalRule(orig) { }
00012     virtual ~LocalRule_GameOfLife()=default;
00013 public:
00014     virtual void applyRule(Cell* cell) override {
00015         unsigned int living = 0;
00016         for (Cell* neigh : cell->getNeighbors()) {
00017             living += neigh->getCurrentState().getValue();
00018         }
00019         if (cell->getCurrentState().getValue() == 1) {
00020             if (living < 2 || living > 3)
00021                 cell->setNextState(State(0));
00022             else
00023                 cell->setNextState(cell->getCurrentState());
00024         } else if (living == 3)
00025             cell->setNextState(State(1));
00026         else
00027             cell->setNextState(cell->getCurrentState());
00028     }
00029 };

```

11.368 LocalRule_Growty.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00005
00006 class LocalRule_Growty : public LocalRule {
00007 public:
00008     LocalRule_Growty(CellularAutomataBase* parentCellularAutomata)
00009         : LocalRule(parentCellularAutomata) { }
00010     LocalRule_Growty(const LocalRule_Growty& orig): LocalRule(orig) {}
00011     virtual ~LocalRule_Growty() = default;
00012 public:
00013     virtual void applyRule(Cell* cell) override {
00014         unsigned int sum = 0;
00015         for (Cell* neigh : cell->getNeighbors()) {
00016             sum += neigh->getCurrentState().getValue();
00017         }
00018         if (sum <= 3 || sum == 5)
00019             cell->setNextState(State(0));
00020         else
00021             cell->setNextState(State(1));
00022     }
00023 };

```

11.369 LocalRule_HppLatticeGas.h

```

00001 #pragma once
00002
00003 #include <iostream>
00004 #include <vector>
00005
00006 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00008 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00009 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00010 #include "plugins/components/ModalModel/CellularAutomata/StateSet.h"
00011
00012 class LocalRule_HppLatticeGas : public LocalRule {
00013 public:
00014     // Simplified HPP lattice gas: particles stream on the four von Neumann
00015     // directions and head-on two-particle collisions rotate by 90 degrees.
00016     // Conceptual reference: local PDF "2018_Book_Adamatzky A. Cellular Automata...Systems Science 2ed
00017     // 2018.pdf",
00018     // physical systems / lattice-gas cellular automata discussion.
00019     explicit LocalRule_HppLatticeGas(CellularAutomataBase* parentCellularAutomata)
00020         : LocalRule(parentCellularAutomata) {
00021     }
00022     void applyRule(Cell* cell) override {

```

```

00023         if (cell == nullptr || parentCellularAutomata == nullptr ||
parentCellularAutomata->getLattice() == nullptr) {
00024             return;
00025         }
00026         if (stateSet != nullptr && dynamic_cast<HppLatticeGasStateSet*>(stateSet) == nullptr) {
00027             std::cerr << "LocalRule_HppLatticeGas requires HppLatticeGasStateSet." << std::endl;
00028             cell->setNextState(cell->getCurrentState());
00029             return;
00030         }
00031
00032         Lattice* lattice = parentCellularAutomata->getLattice();
00033         const std::vector<int> position = cell->getPosition();
00034         if (position.size() < 2 || lattice->getNumDimensions() < 2) {
00035             cell->setNextState(cell->getCurrentState());
00036             return;
00037         }
00038         // sem tempo para acessar por Von Neuman
00039         Cell* northCell = lattice->getCell(wrappedPosition({position[0], position[1] - 1}, lattice));
00040         Cell* eastCell = lattice->getCell(wrappedPosition({position[0] + 1, position[1]}, lattice));
00041         Cell* southCell = lattice->getCell(wrappedPosition({position[0], position[1] + 1}, lattice));
00042         Cell* westCell = lattice->getCell(wrappedPosition({position[0] - 1, position[1]}, lattice));
00043
00044         HppLatticeGasState incoming;
00045         incoming.set(HppLatticeGasState::North, hasDirection(southCell, HppLatticeGasState::North));
00046         incoming.set(HppLatticeGasState::East, hasDirection(westCell, HppLatticeGasState::East));
00047         incoming.set(HppLatticeGasState::South, hasDirection(northCell, HppLatticeGasState::South));
00048         incoming.set(HppLatticeGasState::West, hasDirection(eastCell, HppLatticeGasState::West));
00049
00050         const bool verticalCollision = incoming.has(HppLatticeGasState::North) &&
00051             incoming.has(HppLatticeGasState::South) &&
00052             !incoming.has(HppLatticeGasState::East) &&
00053             !incoming.has(HppLatticeGasState::West);
00054         const bool horizontalCollision = incoming.has(HppLatticeGasState::East) &&
00055             incoming.has(HppLatticeGasState::West) &&
00056             !incoming.has(HppLatticeGasState::North) &&
00057             !incoming.has(HppLatticeGasState::South);
00058
00059         if (verticalCollision) {
00060             cell->setNextState(HppLatticeGasState({{false, true, false, true}}));
00061         } else if (horizontalCollision) {
00062             cell->setNextState(HppLatticeGasState({{true, false, true, false}}));
00063         } else {
00064             cell->setNextState(incoming);
00065         }
00066     }
00067
00068 private:
00069     static std::vector<int> wrappedPosition(std::vector<int> position, Lattice* lattice) {
00070         for (unsigned short dimension = 0; dimension < position.size(); ++dimension) {
00071             const int dimensionSize = static_cast<int>(lattice->getDimension(dimension));
00072             position[dimension] = (position[dimension] % dimensionSize) + dimensionSize %
dimensionSize;
00073         }
00074         return position;
00075     }
00076
00077     static bool hasDirection(Cell* cell, HppLatticeGasState::Direction direction) {
00078         if (cell == nullptr) {
00079             return false;
00080         }
00081         const auto* hpp = cell->getCurrentState().as<HppLatticeGasState>();
00082         if (hpp != nullptr) {
00083             return hpp->has(direction);
00084         }
00085         return (cell->getCurrentState().getValue() & (1L << direction)) != 0;
00086     }
00087 };

```

11.370 LocalRule_SandRock.h

```

00001 // Aluno: Matheus Antonio de Souza - 21203363
00002 #pragma once
00003
00004 #include <vector>
00005
00006 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00008 #include "plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h"
00009
00010 class LocalRule_SandRock_Student : public LocalRule {
00011 public:
00012     LocalRule_SandRock_Student(CellularAutomataBase* parentCellularAutomata)
00013         : LocalRule(parentCellularAutomata) {

```

```

00014     }
00015
00016     LocalRule_SandRock_Student(const LocalRule_SandRock_Student& orig)
00017         : LocalRule(orig) {
00018     }
00019     virtual ~LocalRule_SandRock_Student() = default;
00020     static constexpr long EMPTY = 0;
00021     static constexpr long SAND = 1;
00022     static constexpr long ROCK = 2;
00023 public:
00024
00025     virtual void applyRule(Cell* cell) override {
00026         if (cell == nullptr || stateSet == nullptr) {
00027             return;
00028         }
00029
00030         auto* enumerableStateSet = dynamic_cast<StateSet_Enumerable*>(stateSet);
00031         if (enumerableStateSet == nullptr || enumerableStateSet->getStatesSize() < 3) {
00032             cell->setNextState(cell->getCurrentState());
00033             return;
00034         }
00035
00036         auto pos = cell->getPosition();
00037         // Cada célula possui uma preferência entre cair para esquerda ou para direita
00038         // Neste caso esta sendo utilizado um padrão xadrez que é visualmente agradável
00039         bool preferLeft = ((pos[0] + pos[1]) & 1) != 0;
00040
00041         const auto& nbrs = cell->getNeighbors();
00042         // 0   5   10  14  19
00043         // 1   6   11  15  20
00044         // 2   7       16  21
00045         // 3   8   12  17  22
00046         // 4   9   13  18  23
00047
00048         // nw2 nnw n2   nne ne2
00049         // wnw nw   n   ne  ene
00050         // w2  w    e   e2
00051         // wsw sw   s   se  ese
00052         // sw2 ssw s2   sse se2
00053
00054         long nw = nbrs[6]->getCurrentState().getValue();
00055         long w = nbrs[7]->getCurrentState().getValue();
00056         long sw = nbrs[8]->getCurrentState().getValue();
00057
00058         long n = nbrs[11]->getCurrentState().getValue();
00059         long s = nbrs[12]->getCurrentState().getValue();
00060
00061         long ne = nbrs[15]->getCurrentState().getValue();
00062         long e = nbrs[16]->getCurrentState().getValue();
00063         long se = nbrs[17]->getCurrentState().getValue();
00064
00065         const long wnw = nbrs[1]->getCurrentState().getValue();
00066         const long w2 = nbrs[2]->getCurrentState().getValue();
00067         const long wsw = nbrs[3]->getCurrentState().getValue();
00068
00069         const long ene = nbrs[20]->getCurrentState().getValue();
00070         const long e2 = nbrs[21]->getCurrentState().getValue();
00071         const long ese = nbrs[22]->getCurrentState().getValue();
00072
00073         const long currentState = cell->getCurrentState().getValue();
00074
00075         // A estratégia funciona da seguinte maneira para uma célula areia:
00076         // 1) Se a célula de baixo (s) estiver livre, a areia cai verticalmente (vira vazio)
00077         // 2) Se (s) está bloqueado, e preferLeft
00078         //     - tenta ir para (sw).
00079         //     - se falhar, tenta (se).
00080         // 3) Analogamente, em caso de !preferLeft
00081         //     - tenta ir para (se).
00082         //     - se falhar, tenta (sw).
00083         // 4) Não há movimentos válidos, permanece onde está
00084         //
00085         // Do ponto de vista de uma célula vazia, ela segue a mesma lógica observando os grãos de
00086         areia de cima
00087         // que tentariam escorregar para sua posição, e se o grão cederia ou não a preferencia.
00088         // Isto garante a conservação de matéria. Pelo menos nos casos que testei.
00089         // Um comportamento que não consegui reproduzir é o de preencher completamente um recipiente
00090         dependendo de
00091         // seu formato.
00092         switch (currentState) {
00093             case SAND: {
00094                 // 1) Queda vertical tem prioridade máxima.
00095                 if (s == EMPTY) {
00096                     cell->setNextState(*enumerableStateSet->getState(EMPTY));
00097                     return;
00098                 }
00099                 // 2) Tentativa de escorregamento diagonal.

```

```

00099         bool canSW = (sw == EMPTY) && (w != SAND && w != ROCK);
00100         bool canSE = (se == EMPTY) && (e != SAND && e != ROCK);
00101
00102         if (preferLeft) {
00103             if (canSW) {
00104                 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00105                 return;
00106             }
00107             // Fallback para SE: só escorrega para direita (se) se não ha uma areia a direita(e2)
00108             // preferencia para cair em SE. Por ser preferLeft ser definido em xadrez, sei que ela
00109             // também é preferLeft e cederá passagem para ela.
00110             if (canSE && !(e2 == SAND && ese != EMPTY)) {
00111                 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00112                 return;
00113             }
00114         } else {
00115             if (canSE) {
00116                 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00117                 return;
00118             }
00119             // Fallback para SW: análogo, cedo se w2 quer meu sw.
00120             if (canSW && !(w2 == SAND && wsw != EMPTY)) {
00121                 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00122                 return;
00123             }
00124         }
00125
00126         // Não consigo me mover: permaneço SAND.
00127         cell->setNextState(*enumerableStateSet->getState(SAND));
00128         break;
00129     }
00130     case EMPTY: {
00131         // 1) não cai verticalmente.
00132         if (n == SAND) {
00133             cell->setNextState(*enumerableStateSet->getState(SAND));
00134             return;
00135         }
00136
00137         // 2) se há rock em cima não tem como areia escorregar pra aqui
00138         if (n == ROCK) {
00139             cell->setNextState(*enumerableStateSet->getState(EMPTY));
00140             return;
00141         }
00142
00143         // 3) escorregamento diagonal.
00144         // (Sabemos n != SAND, então diagonais não competem com queda vertical)
00145         bool nw_fills = false;
00146         if (nw == SAND && w != EMPTY) {
00147             if (preferLeft) {
00148                 // por xadrez, nw prefere seu sw(w2)
00149                 const bool canSW_of_nw = (w2 == EMPTY) && (wnw == EMPTY);
00150                 //
00151                 const bool fallback_safe = !(ne == SAND && e != EMPTY);
00152
00153                 if (!canSW_of_nw && fallback_safe) {
00154                     nw_fills = true;
00155                 }
00156             } else {
00157                 // nw prefere seu se, direção preferida.
00158                 nw_fills = true;
00159             }
00160         }
00161
00162         // simétrico
00163         bool ne_fills = false;
00164         if (ne == SAND && e != EMPTY) {
00165             if (preferLeft) {
00166                 ne_fills = true;
00167             } else {
00168                 const bool canSE_of_ne = (e2 == EMPTY) && (ene == EMPTY);
00169                 const bool fallback_safe = !(nw == SAND && w != EMPTY);
00170                 if (!canSE_of_ne && fallback_safe) {
00171                     ne_fills = true;
00172                 }
00173             }
00174         }
00175
00176         if (nw_fills || ne_fills) {
00177             cell->setNextState(*enumerableStateSet->getState(SAND));
00178         } else {
00179             cell->setNextState(*enumerableStateSet->getState(EMPTY));
00180         }
00181         break;
00182     }
00183     case ROCK: {
00184         // objeto imóvel, sempre é pedra

```

```

00185         cell->setNextState(*enumerableStateSet->getState(ROCK));
00186         break;
00187     }
00188     default:
00189         cell->setNextState(cell->getCurrentState());
00190         break;
00191     }
00192 }
00193 protected:
00194 private:
00195 };

```

11.371 LocalRule_SandRock_Student.h

```

00001 // Aluno: Matheus Antonio de Souza - 21203363
00002 #pragma once
00003
00004 #include <vector>
00005
00006 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00008 #include "plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h"
00009
00010 class LocalRule_SandRock_Student : public LocalRule {
00011 public:
00012     LocalRule_SandRock_Student(CellularAutomataBase* parentCellularAutomata)
00013         : LocalRule(parentCellularAutomata) {
00014     }
00015
00016     LocalRule_SandRock_Student(const LocalRule_SandRock_Student& orig)
00017         : LocalRule(orig) {
00018     }
00019     virtual ~LocalRule_SandRock_Student() = default;
00020     static constexpr long EMPTY = 0;
00021     static constexpr long SAND = 1;
00022     static constexpr long ROCK = 2;
00023 public:
00024
00025     virtual void applyRule(Cell* cell) override {
00026         if (cell == nullptr || stateSet == nullptr) {
00027             return;
00028         }
00029
00030         auto* enumerableStateSet = dynamic_cast<StateSet_Enumerable*>(stateSet);
00031         if (enumerableStateSet == nullptr || enumerableStateSet->getStatesSize() < 3) {
00032             cell->setNextState(cell->getCurrentState());
00033             return;
00034         }
00035
00036         auto pos = cell->getPosition();
00037         // Cada célula possui uma preferência entre cair para esquerda ou para direita
00038         // Neste caso esta sendo utilizado um padrão xadrez que é visualmente agradável
00039         bool preferLeft = ((pos[0] + pos[1]) & 1) != 0;
00040
00041         const auto& nbrs = cell->getNeighbors();
00042         // 0   5   10  14  19
00043         // 1   6   11  15  20
00044         // 2   7   16  21
00045         // 3   8   12  17  22
00046         // 4   9   13  18  23
00047
00048         // nw2 nnw n2  nne ne2
00049         // wnw nw   n  ne  ene
00050         // w2  w   e   e2
00051         // wsw sw   s  se  ese
00052         // sw2 ssw s2  sse se2
00053
00054         long nw = nbrs[6]->getCurrentState().getValue();
00055         long w = nbrs[7]->getCurrentState().getValue();
00056         long sw = nbrs[8]->getCurrentState().getValue();
00057
00058         long n = nbrs[11]->getCurrentState().getValue();
00059         long s = nbrs[12]->getCurrentState().getValue();
00060
00061         long ne = nbrs[15]->getCurrentState().getValue();
00062         long e = nbrs[16]->getCurrentState().getValue();
00063         long se = nbrs[17]->getCurrentState().getValue();
00064
00065         const long wnw = nbrs[1]->getCurrentState().getValue();
00066         const long w2 = nbrs[2]->getCurrentState().getValue();
00067         const long wsw = nbrs[3]->getCurrentState().getValue();
00068
00069         const long ene = nbrs[20]->getCurrentState().getValue();

```

```

00070     const long e2 = nbrs[21]->getCurrentState().getValue();
00071     const long ese = nbrs[22]->getCurrentState().getValue();
00072
00073     const long currentState = cell->getCurrentState().getValue();
00074
00075     // A estratégia funciona da seguinte maneira para uma célula areia:
00076     // 1) Se a célula de baixo (s) estiver livre, a areia cai verticalmente (vira vazio)
00077     // 2) Se (s) está bloqueado, e preferLeft
00078     //     - tenta ir para (sw).
00079     //     - se falhar, tenta (se).
00080     // 3) Analogamente, em caso de !preferLeft
00081     //     - tenta ir para (se).
00082     //     - se falhar, tenta (sw).
00083     // 4) Não há movimentos válidos, permanece onde está
00084     //
00085     // Do ponto de vista de uma célula vazia, ela segue a mesma lógica observando os grãos de
areia de cima
00086     // que tentariam escorregar para sua posição, e se o grão cederia ou não a preferência.
00087     // Isto garante a conservação de matéria. Pelo menos nos casos que testei.
00088     // Um comportamento que não consegui reproduzir é o de preencher completamente um recipiente
dependendo de
00089     // seu formato.
00090     switch (currentState) {
00091     case SAND: {
00092         // 1) Queda vertical tem prioridade máxima.
00093         if (s == EMPTY) {
00094             cell->setNextState(*enumerableStateSet->getState(EMPTY));
00095             return;
00096         }
00097
00098         // 2) Tentativa de escorregamento diagonal.
00099         bool canSW = (sw == EMPTY) && (w != SAND && w != ROCK);
00100         bool canSE = (se == EMPTY) && (e != SAND && e != ROCK);
00101
00102         if (preferLeft) {
00103             if (canSW) {
00104                 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00105                 return;
00106             }
00107             // Fallback para SE: só escorrega para direita (se) se não ha uma areia a direita(e2)
com
00108             // preferência para cair em SE. Por ser preferLeft ser definido em xadrez, sei que ela
00109             // também é preferLeft e cederá passagem para ela.
00110             if (canSE && !(e2 == SAND && ese != EMPTY)) {
00111                 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00112                 return;
00113             }
00114         } else {
00115             if (canSE) {
00116                 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00117                 return;
00118             }
00119             // Fallback para SW: análogo, cedo se w2 quer meu sw.
00120             if (canSW && !(w2 == SAND && wsw != EMPTY)) {
00121                 cell->setNextState(*enumerableStateSet->getState(EMPTY));
00122                 return;
00123             }
00124         }
00125
00126         // Não consigo me mover: permaneço SAND.
00127         cell->setNextState(*enumerableStateSet->getState(SAND));
00128         break;
00129     }
00130     case EMPTY: {
00131         // 1) não cai verticalmente.
00132         if (n == SAND) {
00133             cell->setNextState(*enumerableStateSet->getState(SAND));
00134             return;
00135         }
00136
00137         // 2) se há rock em cima não tem como areia escorregar pra aqui
00138         if (n == ROCK) {
00139             cell->setNextState(*enumerableStateSet->getState(EMPTY));
00140             return;
00141         }
00142
00143         // 3) escorregamento diagonal.
00144         // (Sabemos n != SAND, então diagonais não competem com queda vertical)
00145         bool nw_fills = false;
00146         if (nw == SAND && w != EMPTY) {
00147             if (preferLeft) {
00148                 // por xadrez, nw prefere seu sw(w2)
00149                 const bool canSW_of_nw = (w2 == EMPTY) && (wnw == EMPTY);
00150                 //
00151                 const bool fallback_safe = !(ne == SAND && e != EMPTY);
00152
00153                 if (!canSW_of_nw && fallback_safe) {

```



```

00154         nw_fills = true;
00155     }
00156     } else {
00157         // nw prefere seu se, direção preferida.
00158         nw_fills = true;
00159     }
00160 }
00161
00162 // simétrico
00163 bool ne_fills = false;
00164 if (ne == SAND && e != EMPTY) {
00165     if (preferLeft) {
00166         ne_fills = true;
00167     } else {
00168         const bool canSE_of_ne = (e2 == EMPTY) && (ene == EMPTY);
00169         const bool fallback_safe = !(nw == SAND && w != EMPTY);
00170         if (!canSE_of_ne && fallback_safe) {
00171             ne_fills = true;
00172         }
00173     }
00174 }
00175
00176 if (nw_fills || ne_fills) {
00177     cell->setNextState(*enumerableStateSet->getState(SAND));
00178 } else {
00179     cell->setNextState(*enumerableStateSet->getState(EMPTY));
00180 }
00181 break;
00182 }
00183 case ROCK: {
00184     // objeto imóvel, sempre é pedra
00185     cell->setNextState(*enumerableStateSet->getState(ROCK));
00186     break;
00187 }
00188 default:
00189     cell->setNextState(cell->getCurrentState());
00190     break;
00191 }
00192 }
00193 protected:
00194 private:
00195 };

```

11.372 LocalRule_ShowNeighborhood.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/LocalRule.h"
00004 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00005 #include "plugins/components/ModalModel/CellularAutomata/StateSet_Enumerable.h"
00006
00007 class LocalRule_ShowNeighborhood : public LocalRule {
00008 public:
00009     LocalRule_ShowNeighborhood(CellularAutomataBase* parentCellularAutomata)
00010     : LocalRule(parentCellularAutomata) { }
00011     LocalRule_ShowNeighborhood(const LocalRule_ShowNeighborhood& orig): LocalRule(orig) {}
00012     virtual ~LocalRule_ShowNeighborhood() = default;
00013 public:
00014     virtual void applyRule(Cell* cell) {
00015         auto* enumerableStateSet = dynamic_cast<StateSet_Enumerable*>(stateSet);
00016         if (cell == nullptr || enumerableStateSet == nullptr || enumerableStateSet->getStatesSize() <
3)
00017             return;
00018         if (cell->getCurrentState() == *enumerableStateSet->getState(1))
00019             cell->setNextState(cell->getCurrentState());
00020         else {
00021             for (Cell* n: cell->getNeighbors()) {
00022                 if (n != nullptr && n->getCurrentState() == *enumerableStateSet->getState(1)) {
00023                     cell->setNextState(*enumerableStateSet->getState(2));
00024                     return;
00025                 }
00026             }
00027             cell->setNextState(*enumerableStateSet->getState(0));
00028         }
00029     }
00030 };

```

11.373 Neighborhood.h

```

00001 #pragma once

```

```

00002
00003 #include <string>
00004 #include <utility>
00005 #include <vector>
00006
00007
00008 class Cell;
00009 #include "plugins/components/ModalModel/CellularAutomata/BoundaryCondition.h"
00010 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h"
00011
00012 class Neighborhood {
00013 public:
00014     Neighborhood(CellularAutomataBase* parentCellularAutomata, unsigned short radius=1,
00015     BoundaryCondition* boundary= nullptr, bool includeCellItself=false){
00016         this->parentCellularAutomata = parentCellularAutomata;
00017         parentCellularAutomata->setNeighborhood(this);
00018         this->radius = radius;
00019         this->boundary = boundary;
00020         this->includeCellItself = includeCellItself;
00021     }
00022     Neighborhood(const Neighborhood& orig){}
00023     virtual ~Neighborhood()=default;
00024 public:
00025     virtual std::string show(){
00026         return "-";
00027     }
00028     virtual std::vector<Cell*> getNeighbors(Cell* cell) = 0;
00029 public:
00030     void setRadius(unsigned short radius) {
00031         this->radius = radius;
00032     }
00033     unsigned short getRadius() const{
00034         return radius;
00035     }
00036     void setBoundary(BoundaryCondition* boundary) {
00037         this->boundary = boundary;
00038         boundary->setNeighborhood(this);
00039     }
00040     BoundaryCondition* getBoundary() const{
00041         return boundary;
00042     }
00043     void setIncludeCellItself(bool includeCellItself){
00044         this->includeCellItself = includeCellItself;
00045     }
00046     bool getIncludeCellItself() const{
00047         return includeCellItself;
00048     }
00049 public:
00050 protected:
00051     Cell* getNeighborCell(std::vector<int> cellPosition, std::vector<std::pair<unsigned short, int>
00052     dimensionChanges){
00053         std::vector<int> neighborPosition;
00054         neighborPosition = cellPosition;
00055         for (std::pair<unsigned short, int> change : dimensionChanges) {
00056             neighborPosition.at(change.first) += change.second;
00057         }
00058         //std::cout << "Neig pos from " << Util::showNDimPosition(cellPosition) << " is "
00059         << Util::showNDimPosition(&neighborPosition);
00060         Cell* neighCell = boundary->getNeighborCell(cellPosition, neighborPosition);
00061         //std::cout << std::endl;
00062         return neighCell;
00063     }
00064 protected:
00065     std::string name;
00066     CellularAutomataBase* parentCellularAutomata;
00067     BoundaryCondition* boundary;
00068     unsigned short radius;
00069     bool includeCellItself;
00070 private:
00071 };

```

11.374 Neighborhood_Center.h

```

00001 #pragma once
00002
00003 #include <string>
00004 #include <vector>
00005
00006 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00008 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h"
00009
00010 class Neighborhood_Center: public Neighborhood {

```

```

00011 public:
00012     Neighborhood_Center(CellularAutomataBase* parentCellularAutomata, unsigned short radius=1,
00013                          BoundaryCondition* boundary= nullptr, bool includeCellItself=false)
00014     : Neighborhood(parentCellularAutomata, radius, boundary, includeCellItself) {
00015         this->name = "Center";
00016     }
00017     Neighborhood_Center(const Neighborhood_Center& orig)
00018     : Neighborhood(orig) { }
00019     virtual ~Neighborhood_Center()=default;
00020 public:
00021     virtual std::vector<Cell*> getNeighbors(Cell* cell) override {
00022         std::vector<Cell*> neighbors;
00023         //std::string m = "Center::getNeighbors(CellPos=" +
00024         Util::showNDimPosition(cell->getPosition())+"=");
00025         //unsigned short numDimensions = parentCellularAutomata->getLattice()->getNumDimensions();
00026         //std::vector<int>* cellPosition = cell->getPosition();
00027         for (int r = 1; r <= radius; r++) {
00028             int d = 0;
00029             //for (int d = 0; r < numDimensions; d++) {
00030                 neighbors.emplace_back(getNeighborCell(cell->getPosition(), {{d, -r}}));
00031                 neighbors.emplace_back(getNeighborCell(cell->getPosition(), {{d, r}}));
00032             }
00033             if (includeCellItself)
00034                 neighbors.emplace_back(cell);
00035         return neighbors;
00036     }
00037 protected:
00038 private:
00039 };

```

11.375 Neighborhood_Moore.h

```

00001 /*
00002  * File: Neighborhood_Moore.h
00003  * Author: cancian
00004  *
00005  * Created on 6 de abril de 2023, 14:27
00006  */
00007
00008 #pragma once
00009
00010 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00011 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood.h"
00012 #include "plugins/components/ModalModel/CellularAutomata/Cell.h"
00013
00014 #include <vector>
00015
00016 class Neighborhood_Moore : public Neighborhood {
00017 public:
00018     Neighborhood_Moore(CellularAutomataBase* parentCellularAutomata, unsigned short radius = 1,
00019                        BoundaryCondition* boundary = nullptr)
00020     : Neighborhood(parentCellularAutomata, radius, boundary) { this->name = "Moore"; }
00021     Neighborhood_Moore(const Neighborhood_Moore& orig):Neighborhood(orig) { }
00022     virtual ~Neighborhood_Moore() = default;
00023 public:
00024     virtual std::string show() override {
00025         return "todo";
00026     }
00027     virtual std::vector<Cell*> getNeighbors(Cell* cell) override {
00028         std::vector<Cell*> neighbors;
00029         unsigned short numDimensions = parentCellularAutomata->getLattice()->getNumDimensions();
00030         std::vector<int> cellPosition = cell->getPosition();
00031
00032         std::vector<int> offset(numDimensions, 0);
00033         collectNeighbors(cellPosition, offset, 0, neighbors);
00034
00035         if (includeCellItself)
00036             neighbors.emplace_back(cell);
00037         return neighbors;
00038     }
00039 private:
00040     void collectNeighbors(const std::vector<int>& cellPosition, std::vector<int>& offset, unsigned
00041                          short dimension, std::vector<Cell*>& neighbors) {
00042         if (dimension == offset.size()) {
00043             bool isZeroOffset = true;
00044             std::vector<std::pair<unsigned short, int>> dimensionChanges;
00045             for (unsigned short dim = 0; dim < offset.size(); ++dim) {
00046                 if (offset.at(dim) != 0) {
00047                     isZeroOffset = false;
00048                     dimensionChanges.emplace_back(dim, offset.at(dim));
00049                 }
00050             }
00051         }
00052     }
00053 }

```

```

00048         }
00049         if (!isZeroOffset) {
00050             // Offsets are visited from dimension 0 to n-1.
00051             neighbors.emplace_back(getNeighborCell(cellPosition, dimensionChanges));
00052         }
00053         return;
00054     }
00055
00056     for (int delta = -static_cast<int>(radius); delta <= static_cast<int>(radius); ++delta) {
00057         offset.at(dimension) = delta;
00058         collectNeighbors(cellPosition, offset, dimension + 1, neighbors);
00059     }
00060 }
00061 };

```

11.376 Neighborhood_VonNeumann.h

```

00001 /*
00002  * File: Neighborhood_VonNeumann.h
00003  * Author: cancian
00004  *
00005  * Created on 6 de abril de 2023, 14:27
00006  */
00007
00008 #pragma once
00009
00010 #include <string>
00011 #include <vector>
00012 #include <cstdlib>
00013
00014 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood.h"
00015 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00016
00017 class Neighborhood_VonNeumann : public Neighborhood {
00018 public:
00019
00020     Neighborhood_VonNeumann(CellularAutomataBase* parentCellularAutomata, unsigned short radius = 1,
00021                             BoundaryCondition* boundary = nullptr)
00022     : Neighborhood(parentCellularAutomata, radius, boundary) {
00023         this->name = "Von Neumann";
00024     }
00025
00026     Neighborhood_VonNeumann(const Neighborhood_VonNeumann& orig)
00027     : Neighborhood(orig) {
00028     }
00029     virtual ~Neighborhood_VonNeumann() = default;
00030 public:
00031     virtual std::string show() override {
00032         return "-";
00033     }
00034
00035     virtual std::vector<Cell*> getNeighbors(Cell* cell) override {
00036         std::vector<Cell*> neighbors;
00037         unsigned short numDimensions = parentCellularAutomata->getLattice()->getNumDimensions();
00038         std::vector<int> cellPosition = cell->getPosition();
00039
00040         std::vector<int> offset(numDimensions, 0);
00041         collectNeighbors(cellPosition, offset, 0, neighbors);
00042
00043         if (includeCellItself)
00044             neighbors.emplace_back(cell);
00045         return neighbors;
00046     }
00047 private:
00048     void collectNeighbors(const std::vector<int>& cellPosition, std::vector<int>& offset, unsigned
00049                          short dimension, std::vector<Cell*>& neighbors) {
00050         if (dimension == offset.size()) {
00051             bool isZeroOffset = true;
00052             unsigned int manhattanDistance = 0;
00053             std::vector<std::pair<unsigned short, int>> dimensionChanges;
00054             for (unsigned short dim = 0; dim < offset.size(); ++dim) {
00055                 int delta = offset.at(dim);
00056                 manhattanDistance += static_cast<unsigned int>(std::abs(delta));
00057                 if (delta != 0) {
00058                     isZeroOffset = false;
00059                     dimensionChanges.emplace_back(dim, delta);
00060                 }
00061             }
00062             if (!isZeroOffset && manhattanDistance <= radius) {
00063                 // Offsets are visited from dimension 0 to n-1.
00064                 neighbors.emplace_back(getNeighborCell(cellPosition, dimensionChanges));
00065             }
00066         }
00067     }

```

```

00065         return;
00066     }
00067
00068     for (int delta = -static_cast<int>(radius); delta <= static_cast<int>(radius); ++delta) {
00069         offset.at(dimension) = delta;
00070         collectNeighbors(cellPosition, offset, dimension + 1, neighbors);
00071     }
00072 }
00073 };

```

11.377 State.h

```

00001 #pragma once
00002
00003 #include <array>
00004 #include <memory>
00005 #include <sstream>
00006 #include <string>
00007 #include <typeinfo>
00008 #include <utility>
00009 #include <vector>
00010
00011 /*
00012 template<class T>
00013 class State {
00014 public:
00015     State<T>() {}
00016     State<T>(T value) {_value=value;}
00017     State<T>(const State& orig){}
00018     virtual ~State<T>() = default;
00019 public:
00020     T getValue(){return _value;}
00021     void setValue(T value) {_value=value;}
00022 protected:
00023     T _value;
00024 private:
00025 };
00026 */
00027
00028 class State {
00029 public:
00030     State() : _value(0) {}
00031     State(long value) : _value(value) {}
00032     State(const State& orig) : _value(orig._value) {}
00033     virtual ~State() = default;
00034 public:
00035     virtual std::unique_ptr<State> clone() const { return std::make_unique<State>(*this); }
00036     virtual bool equals(const State& other) const { return typeName() == other.typeName() &&
00037         getValue() == other.getValue(); }
00037     virtual std::string toString() const { return std::to_string(_value); }
00038     virtual std::string show() const { return toString(); }
00039     virtual std::string typeName() const { return "State"; }
00040     virtual long getValue() const { return _value; }
00041     virtual void setValue(long value) { _value=value; }
00042
00043     template <typename T>
00044     const T* as() const noexcept { return dynamic_cast<const T*>(this); }
00045
00046     template <typename T>
00047     T* as() noexcept { return dynamic_cast<T*>(this); }
00048 protected:
00049     long _value;
00050 private:
00051 };
00052
00053 inline bool operator==(const State& left, const State& right) {
00054     return left.equals(right);
00055 }
00056
00057 inline bool operator!=(const State& left, const State& right) {
00058     return !(left == right);
00059 }
00060
00061 class RealState : public State {
00062 public:
00063     explicit RealState(double value = 0.0)
00064         : State(static_cast<long>(value)), _realValue(value) {
00065     }
00066
00067     std::unique_ptr<State> clone() const override { return std::make_unique<RealState>(*this); }
00068     bool equals(const State& other) const override {
00069         const auto* real = other.as<RealState>();
00070         return real != nullptr && _realValue == real->_realValue;

```

```

00071     }
00072     std::string toString() const override {
00073         std::ostringstream stream;
00074         stream << _realValue;
00075         return stream.str();
00076     }
00077     std::string typeName() const override { return "RealState"; }
00078     long getValue() const override { return static_cast<long>(_realValue); }
00079     void setValue(long value) override { _realValue = static_cast<double>(value); _value = value; }
00080     double realValue() const { return _realValue; }
00081     void setRealValue(double value) { _realValue = value; _value = static_cast<long>(value); }
00082 private:
00083     double _realValue = 0.0;
00084 };
00085
00086 class CompositeState : public State {
00087 public:
00088     using Field = std::pair<std::string, std::unique_ptr<State>;
00089
00090     CompositeState() = default;
00091     CompositeState(const CompositeState& orig) : State(orig) {
00092         for (const auto& field : orig._fields) {
00093             _fields.emplace_back(field.first, field.second != nullptr ? field.second->clone() :
00094                 nullptr);
00095         }
00096     }
00097     CompositeState& operator=(const CompositeState& orig) {
00098         if (this == &orig) {
00099             return *this;
00100         }
00101         _value = orig._value;
00102         _fields.clear();
00103         for (const auto& field : orig._fields) {
00104             _fields.emplace_back(field.first, field.second != nullptr ? field.second->clone() :
00105                 nullptr);
00106         }
00107         return *this;
00108     }
00109     std::unique_ptr<State> clone() const override { return std::make_unique<CompositeState>(*this); }
00110     bool equals(const State& other) const override {
00111         const auto* composite = other.as<CompositeState>();
00112         if (composite == nullptr || _fields.size() != composite->_fields.size()) {
00113             return false;
00114         }
00115         for (size_t index = 0; index < _fields.size(); ++index) {
00116             const auto& left = _fields.at(index);
00117             const auto& right = composite->_fields.at(index);
00118             if (left.first != right.first || left.second == nullptr || right.second == nullptr ||
00119                 *left.second != *right.second) {
00120                 return false;
00121             }
00122         }
00123         return true;
00124     }
00125     std::string toString() const override {
00126         std::string text = "{";
00127         for (size_t index = 0; index < _fields.size(); ++index) {
00128             if (index > 0) {
00129                 text += ",";
00130             }
00131             text += _fields.at(index).first + ":" + (_fields.at(index).second != nullptr ?
00132                 _fields.at(index).second->toString() : "null");
00133         }
00134         return text + "}";
00135     }
00136     std::string typeName() const override { return "CompositeState"; }
00137     bool addField(std::string name, const State& state) {
00138         _fields.emplace_back(std::move(name), state.clone());
00139         return true;
00140     }
00141     const std::vector<Field>& fields() const { return _fields; }
00142 private:
00143     std::vector<Field> _fields;
00144 };
00145
00146 class HppLatticeGasState : public State {
00147 public:
00148     enum Direction : size_t { North = 0, East = 1, South = 2, West = 3 };
00149
00150     HppLatticeGasState() = default;
00151     explicit HppLatticeGasState(std::array<bool, 4> occupations)
00152         : State(maskFromOccupations(occupations)), _occupations(occupations) {}
00153
00154     std::unique_ptr<State> clone() const override { return
00155         std::make_unique<HppLatticeGasState>(*this); }

```

```

00153     bool equals(const State& other) const override {
00154         const auto* hpp = other.as<HppLatticeGasState>();
00155         return hpp != nullptr && _occupations == hpp->_occupations;
00156     }
00157     std::string toString() const override {
00158         return std::string("{N:" + (_occupations[North] ? "1" : "0") +
00159             ",E:" + (_occupations[East] ? "1" : "0") +
00160             ",S:" + (_occupations[South] ? "1" : "0") +
00161             ",W:" + (_occupations[West] ? "1" : "0") + "}");
00162     }
00163     std::string typeName() const override { return "HppLatticeGasState"; }
00164     long getValue() const override { return maskFromOccupations(_occupations); }
00165     void setValue(long value) override {
00166         _occupations = {
00167             (value & (1L << North)) != 0,
00168             (value & (1L << East)) != 0,
00169             (value & (1L << South)) != 0,
00170             (value & (1L << West)) != 0,
00171         };
00172         _value = getValue();
00173     }
00174     bool has(Direction direction) const { return _occupations[direction]; }
00175     void set(Direction direction, bool occupied) { _occupations[direction] = occupied; _value =
00176         getValue(); }
00176     const std::array<bool, 4>& occupations() const { return _occupations; }
00177     static long maskFromOccupations(const std::array<bool, 4>& occupations) {
00178         long mask = 0;
00179         for (size_t index = 0; index < occupations.size(); ++index) {
00180             if (occupations[index]) {
00181                 mask |= (1L << index);
00182             }
00183         }
00184         return mask;
00185     }
00186 private:
00187     std::array<bool, 4> _occupations{{false, false, false, false}};
00188 };

```

11.378 State_Numerical.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00004
00005 /*
00006 class State_Numerical: public State {
00007 public:
00008     State_Numerical(long value)
00009         : State(ColorCA(0,0,0), ' ', value, ""){
00010         this->longValue = value;
00011     }
00012     State_Numerical(const State_Numerical& orig)
00013         :State(orig) {
00014     }
00015     }
00016     virtual ~State_Numerical() = default;
00017 public:
00018     virtual ColorCA getColor() const override {
00019         int llongValue = 0;
00020         if (longValue)
00021             llongValue = 1<<9;
00022         uint8_t south = longValue&((1<<3)|(1<<5)|(1<<7))?0:255;
00023         uint8_t r = longValue&((1<<0)|(1<<4)|(1<<5))?0:south; // west : south
00024         uint8_t g = longValue&((1<<1)|(1<<6)|(1<<7))?0:south; // east : south
00025         uint8_t b = longValue&((1<<2)|(1<<4)|(1<<6))?0:south; // north : south
00026         //std::cout << longValue << ": " << (int)r << ", " << (int)g << ", " << (int)b << std::endl;
00027         return ColorCA(r,g,b);
00028     }
00029
00030 protected:
00031 private:
00032 };
00033 */

```

11.379 StateSet.h

```

00001 #pragma once
00002
00003 #include <limits>

```

```

00004 #include <memory>
00005 #include <optional>
00006 #include <sstream>
00007 #include <string>
00008 #include <vector>
00009 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00010
00011 class CellularAutomataBase;
00012
00013 class StateSet {
00014 public:
00015     StateSet(CellularAutomataBase* parentCellularAutomata);
00016     StateSet(const StateSet& orig);
00017     virtual ~StateSet()=default;
00018 public:
00019     virtual bool contains(const State& state) const;
00020     virtual std::unique_ptr<State> createDefaultState() const;
00021     virtual std::unique_ptr<State> parseState(const std::string& text) const;
00022     virtual bool isFinite() const;
00023     virtual bool isDiscrete() const;
00024     virtual bool isContinuous() const;
00025     virtual bool isComposite() const;
00026     virtual std::vector<std::unique_ptr<State>> enumerateStates() const;
00027     virtual std::string name() const;
00028 protected:
00029     CellularAutomataBase* parentCellularAutomata;
00030 private:
00031 };
00032
00033 class IntegerStateSet : public StateSet {
00034 public:
00035     IntegerStateSet(CellularAutomataBase* parentCellularAutomata,
00036                     std::optional<long> minimum = std::nullopt,
00037                     std::optional<long> maximum = std::nullopt)
00038         : StateSet(parentCellularAutomata), _minimum(minimum), _maximum(maximum) {
00039     }
00040
00041     bool contains(const State& state) const override {
00042         if (state.as<RealState>() != nullptr || state.as<CompositeState>() != nullptr ||
00043             state.as<HppLatticeGasState>() != nullptr) {
00044             return false;
00045         }
00046         const long value = state.getValue();
00047         return (!_minimum.has_value() || value >= *_minimum) && (!_maximum.has_value() || value <=
*_maximum);
00048     }
00049     std::unique_ptr<State> createDefaultState() const override {
00050         return std::make_unique<State>(_minimum.value_or(0));
00051     }
00052     std::unique_ptr<State> parseState(const std::string& text) const override {
00053         try {
00054             auto state = std::make_unique<State>(std::stol(text));
00055             return contains(*state) ? std::move(state) : nullptr;
00056         } catch (...) {
00057             return nullptr;
00058         }
00059     }
00060     bool isFinite() const override { return _minimum.has_value() && _maximum.has_value(); }
00061     bool isDiscrete() const override { return true; }
00062     std::vector<std::unique_ptr<State>> enumerateStates() const override {
00063         std::vector<std::unique_ptr<State>> states;
00064         if (!isFinite()) {
00065             return states;
00066         }
00067         for (long value = *_minimum; value <= *_maximum; ++value) {
00068             states.emplace_back(std::make_unique<State>(value));
00069             if (value == std::numeric_limits<long>::max()) {
00070                 break;
00071             }
00072         }
00073         return states;
00074     }
00075     std::string name() const override { return "IntegerStateSet"; }
00076 private:
00077     std::optional<long> _minimum;
00078     std::optional<long> _maximum;
00079 };
00080
00081 class NaturalStateSet : public IntegerStateSet {
00082 public:
00083     explicit NaturalStateSet(CellularAutomataBase* parentCellularAutomata, std::optional<long> maximum
= std::nullopt)
00084         : IntegerStateSet(parentCellularAutomata, 0, maximum) {
00085     }
00086     std::string name() const override { return "NaturalStateSet"; }
00087 };

```



```

00088 class RealIntervalStateSet : public StateSet {
00089 public:
00090     RealIntervalStateSet(CellularAutomataBase* parentCellularAutomata,
00091         std::optional<double> minimum = std::nullopt,
00092         std::optional<double> maximum = std::nullopt)
00093         : StateSet(parentCellularAutomata, _minimum(minimum), _maximum(maximum)) {
00094     }
00095
00096     bool contains(const State& state) const override {
00097         const auto* real = state.as<RealState>();
00098         const double value = real != nullptr ? real->realValue() :
00099             static_cast<double>(state.getValue());
00100         return (!_minimum.has_value() || value >= *_minimum) && (!_maximum.has_value() || value <=
00101             *_maximum);
00102     }
00103     std::unique_ptr<State> createDefaultState() const override {
00104         return std::make_unique<RealState>(_minimum.value_or(0.0));
00105     }
00106     std::unique_ptr<State> parseState(const std::string& text) const override {
00107         try {
00108             auto state = std::make_unique<RealState>(std::stod(text));
00109             return contains(*state) ? std::move(state) : nullptr;
00110         } catch (...) {
00111             return nullptr;
00112         }
00113     }
00114     bool isContinuous() const override { return true; }
00115     std::string name() const override { return "RealIntervalStateSet"; }
00116 private:
00117     std::optional<double> _minimum;
00118     std::optional<double> _maximum;
00119 };
00120
00121 class CompositeStateSet : public StateSet {
00122 public:
00123     explicit CompositeStateSet(CellularAutomataBase* parentCellularAutomata)
00124         : StateSet(parentCellularAutomata) {
00125     }
00126
00127     bool contains(const State& state) const override { return state.as<CompositeState>() != nullptr; }
00128     std::unique_ptr<State> createDefaultState() const override { return
00129         std::make_unique<CompositeState>(); }
00130     bool isComposite() const override { return true; }
00131     std::string name() const override { return "CompositeStateSet"; }
00132 };
00133
00134 class HppLatticeGasStateSet : public StateSet {
00135 public:
00136     explicit HppLatticeGasStateSet(CellularAutomataBase* parentCellularAutomata)
00137         : StateSet(parentCellularAutomata) {
00138     }
00139
00140     bool contains(const State& state) const override { return state.as<HppLatticeGasState>() !=
00141         nullptr; }
00142     std::unique_ptr<State> createDefaultState() const override { return
00143         std::make_unique<HppLatticeGasState>(); }
00144     std::unique_ptr<State> parseState(const std::string& text) const override {
00145         try {
00146             auto state = std::make_unique<HppLatticeGasState>();
00147             state->setValue(std::stol(text));
00148             return state;
00149         } catch (...) {
00150             return nullptr;
00151         }
00152     }
00153     bool isFinite() const override { return true; }
00154     bool isDiscrete() const override { return true; }
00155     bool isComposite() const override { return true; }
00156     std::vector<std::unique_ptr<State>> enumerateStates() const override {
00157         std::vector<std::unique_ptr<State>> states;
00158         for (long mask = 0; mask < 16; ++mask) {
00159             auto state = std::make_unique<HppLatticeGasState>();
00160             state->setValue(mask);
00161             states.emplace_back(std::move(state));
00162         }
00163         return states;
00164     }
00165     std::string name() const override { return "HppLatticeGasStateSet"; }
00166 };

```

11.380 StateSet_Enumerable.h

```
00001 #pragma once
```

```

00002
00003 #include <memory>
00004 #include <string>
00005 #include <vector>
00006 #include "plugins/components/ModalModel/CellularAutomata/State.h"
00007 #include "plugins/components/ModalModel/CellularAutomata/StateSet.h"
00008
00009 class CellularAutomataBase;
00010
00011 class StateSet_Enumerable: public StateSet{
00012 public:
00013     StateSet_Enumerable(CellularAutomataBase* parentCellularAutomata, std::vector<State*> states =
00014 {});
00015     StateSet_Enumerable(const StateSet_Enumerable& orig);
00016     virtual ~StateSet_Enumerable()=default;
00017 public:
00018     virtual std::string show();
00019     bool contains(const State& state) const override;
00020     std::unique_ptr<State> createDefaultState() const override;
00021     std::unique_ptr<State> parseState(const std::string& text) const override;
00022     bool isFinite() const override;
00023     bool isDiscrete() const override;
00024     std::vector<std::unique_ptr<State>> enumerateStates() const override;
00025     std::string name() const override;
00026     unsigned int size();
00027     void addState(State* state);
00028     void addState(const State& state);
00029     unsigned int getStatesSize();
00030     State* getState(unsigned int rank);
00031     const State* getState(unsigned int rank) const;
00032     State* getState(std::string name);
00033     void setStates(std::vector<State*> states);
00034     std::vector<State*> getStates() const;
00035 protected:
00036     CellularAutomataBase* parentCellularAutomata;
00037     std::vector<std::unique_ptr<State>> ownedStates;
00038     std::vector<State*> states;
00039 private:
00040 };

```

11.381 CellularAutomataComp.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Dummy.h
00009  * Author:  rafael.luiz.cancian
00010  *
00011  * Created on 22 de Maio de 2019, 18:41
00012  */
00013
00014 #pragma once
00015
00016 #include "../kernel/simulator/model/ModelComponent.h"
00017 #include "plugins/components/ModalModel/CellularAutomata/CellularAutomataBase.h"
00018 #include "plugins/components/ModalModel/CellularAutomata/Lattice.h"
00019 #include "plugins/components/ModalModel/CellularAutomata/Neighborhood.h"
00020 #include "plugins/components/ModalModel/CellularAutomata/StateSet.h"
00021
00022 class BoundaryCondition;
00023
00024 class CellularAutomataComp : public ModelComponent {
00025 public:
00026     enum class CellularAutomataType : int {
00027         CLASSIC = 1, TIMED_1D = 2, ASYNCHRONOUS = 3, NONUNIFORMRULE = 4, NONUNIFORMNEIGHBOOR = 5,
00028         USERDEFINED = 6
00029     };
00030
00031     enum class LatticeType : int {
00032         RETICULAR = 1, TRIANGULAR = 2, HEXAGONAL = 3, NETWORK = 4, USERDEFINED = 5
00033     };
00034
00035     enum class NeighborhoodType : int {
00036         CENTERED = 1, BACKWARD = 2, FORWARD = 3, VONNEUMANN = 4, MOORE = 5, USERDEFINED = 6
00037     };
00038
00039     enum class BoundaryType : int {
00040         FIXED = 1, CLOSED = 2, REFLEXIVE = 3, ADIABATIC = 4, USERDEFINED = 5
00041     };
00042 };

```

```

00044
00045     enum class StateSetType : int {
00046         ENUMERATED = 1, INTEGERBASED = 2, BITBASED = 3, DOUBLEBASED = 4, USERDEFINED = 5
00047     };
00048
00049     enum class LocalRuleType : int {
00050         ELEMENTAR_CA = 1, GAME_OF_LIFE = 2, BIASED_COMPETITION = 3, HPP = 4, SAND_ROCK = 5,
00051         USERDEFINED = 99
00052     };
00053
00054     enum class AutomataType : int {
00055         Temporary = 0, Permanent = 1
00056     };
00057 public:
00058     CellularAutomataComp(Model* model, std::string name = "");
00059     virtual ~CellularAutomataComp() = default;
00060
00061 public:
00062     CellularAutomataComp::CellularAutomataType getCellularAutomataType() const;
00063     void setCellularAutomataType(CellularAutomataComp::CellularAutomataType newCellularAutomataType);
00064
00065     CellularAutomataComp::LatticeType getLatticeType() const;
00066     void setLatticeType(CellularAutomataComp::LatticeType newLatticeStructure);
00067
00068     CellularAutomataComp::NeighborhoodType getNeighborhoodType() const;
00069     void setNeighborhoodType(CellularAutomataComp::NeighborhoodType newNeighborhood);
00070
00071     CellularAutomataComp::BoundaryType getBoundaryType() const;
00072     void setBoundaryType(CellularAutomataComp::BoundaryType newBoundary);
00073
00074     CellularAutomataComp::StateSetType getStateSetType() const;
00075     void setStateSetType(CellularAutomataComp::StateSetType newStateSetType);
00076
00077     //CellularAutomataBase *getcellularAutomata() const;
00078     Lattice *getlattice() const;
00079     Neighborhood *getNeighborhood() const;
00080     //BoundaryCondition *getBoundary() const;
00081     StateSet *getStateSet() const;
00082
00083 public:
00084     virtual std::string show() override;
00085
00086 public:
00087     static PluginInformation* GetPluginInformation();
00088     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00089     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00090
00091     CellularAutomataComp::LocalRuleType getlocalRuleType() const;
00092     void setLocalRuleType(CellularAutomataComp::LocalRuleType newLocalRuleType);
00093
00094     LocalRule *getlocalRule() const;
00095
00096 protected:
00097     virtual bool _loadInstance(PersistenceRecord *fields) override;
00098     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00099     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00100
00101
00102 protected:
00103     virtual bool _check(std::string* errorMessage);
00104     // virtual ParserChangesInformation* _getParserChangesInformation();
00105     virtual void _initBetweenReplications() override;
00106     // virtual void _createInternalAndAttachedData(); /*< A ModelDataDefinition or ModelComponent that
00107     includes (internal) ou refers to (attach) other ModelDataDefinition must register them inside this
00108     method. */
00109     // virtual void _addSimulationControl(SimulationControl* property);
00110
00111
00112
00113
00114
00115 protected:
00116     // virtual void _createInternalStatisticReporters() override;
00117     // virtual void _createNonEditableDataDefinitions() override;
00118     // virtual void _createEditableDataDefinitions() override;
00119     // virtual void _createAttachedAttributes() override;
00120
00121 private:
00122     // ...
00123
00124 private:
00125
00126     // Default values for the attributes. Used on initing, loading and saving
00127     const struct DEFAULT_VALUES {
00128         const CellularAutomataType cellularAutomataType = CellularAutomataType::CLASSIC;
00129         const LatticeType latticeType = LatticeType::RETICULAR;
00130         const NeighborhoodType neighborhoodType = NeighborhoodType::VONNEUMANN;
00131         const BoundaryType boundaryType = BoundaryType::FIXED;
00132         const StateSetType stateSetType = StateSetType::ENUMERATED;

```

```

00133         const LocalRuleType localRuleType = LocalRuleType::GAME_OF_LIFE;
00134     } DEFAULT;
00135     CellularAutomataComp::CellularAutomataType _cellularAutomataType = DEFAULT.cellularAutomataType;
00136     CellularAutomataComp::LatticeType _latticeType = DEFAULT.latticeType;
00137     CellularAutomataComp::NeighborhoodType _neighborhoodType = DEFAULT.neighborhoodType;
00138     CellularAutomataComp::BoundaryType _boundaryType = DEFAULT.boundaryType;
00139     CellularAutomataComp::StateSetType _stateSetType = DEFAULT.stateSetType;
00140     CellularAutomataComp::LocalRuleType _localRuleType = DEFAULT.localRuleType;
00141
00142 private:
00143     CellularAutomataBase* _cellularAutomata = nullptr;
00144     Lattice* _lattice = nullptr;
00145     Neighborhood* _neighborhood = nullptr;
00146     BoundaryCondition* _boundary = nullptr;
00147     StateSet* _stateSet = nullptr;
00148     LocalRule* _localRule = nullptr;
00149
00150 private:
00151
00152 private:
00153     // ...
00154 };

```

11.382 DefaultNode.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:DefaultNode.h
00009  * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00010  *
00011  * Created on 01 de Julho de 2025, 14:26
00012  */
00013
00014 #pragma once
00015
00016 #include "../kernel/simulator/model/ModelComponent.h"
00017 #include <functional>
00018 #include <string>
00019 // #include "plugins/data/Template/DummyElement.h"
00020
00021 class DefaultNode;
00022 class Model;
00023 class Entity;
00024
00025 class DefaultNodeTransition {
00026 public:
00027     enum class TransitionKind : int { DETERMINISTIC = 0, PROBABILISTIC = 1 };
00028
00029 public:
00030     DefaultNodeTransition(DefaultNode* source, DefaultNode* destination, std::string name = "");
00031     virtual ~DefaultNodeTransition() = default;
00032
00033 public:
00034     void setSource(DefaultNode* source);
00035     DefaultNode* getSource() const;
00036     void setDestination(DefaultNode* destination);
00037     DefaultNode* getDestination() const;
00038     void setName(std::string name);
00039     std::string getName() const;
00040     void setGuardExpression(std::string guardExpression);
00041     std::string getGuardExpression() const;
00042     void setOutputExpression(std::string outputExpression);
00043     std::string getOutputExpression() const;
00044     void setInputEvent(std::string inputEvent);
00045     std::string getInputEvent() const;
00046     void setPriority(unsigned int priority);
00047     unsigned int getPriority() const;
00048     void setProbability(double probability);
00049     double getProbability() const;
00050     void setTransitionKind(TransitionKind transitionKind);
00051     TransitionKind getTransitionKind() const;
00052
00053 public:
00054     virtual bool canFire(Model* model, Entity* entity) const;
00055     virtual void execute(Model* model, Entity* entity) const;
00056
00057 protected:

```

```

00059
00060 private:
00061     DefaultNode* _source = nullptr;
00062     DefaultNode* _destination = nullptr;
00063     std::string _name = "";
00064     std::string _guardExpression = "";
00065     std::string _outputExpression = "";
00066     std::string _inputEvent = "";
00067     unsigned int _priority = 0;
00068     double _probability = 1.0;
00069     TransitionKind _transitionKind = TransitionKind::DETERMINISTIC;
00070 };
00071
00072 class DefaultNode : public ModelComponent {
00073 public:
00074     DefaultNode(Model* model, std::string componentName, std::string name);
00075     DefaultNode(Model* model, std::string name = "");
00076     virtual ~DefaultNode() = default;
00077
00078 public:
00079     void addTransition(DefaultNodeTransition* transition);
00080     void removeTransition(DefaultNodeTransition* transition);
00081     List<DefaultNodeTransition*>* getTransitions() const;
00082     void setInitialNode(bool initialNode);
00083     bool isInitialNode() const;
00084     void setFinalNode(bool finalNode);
00085     bool isFinalNode() const;
00086
00087 public:
00088     virtual std::string show() override;
00089
00090 public:
00091     static PluginInformation* GetPluginInformation();
00092     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00093     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00094
00095 protected:
00096     virtual bool _loadInstance(PersistenceRecord* fields) override;
00097     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00098     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00099
00100 protected:
00101     // virtual bool _check(std::string& errorMessage);
00102     // virtual ParserChangesInformation* _getParserChangesInformation();
00103     // virtual void _initBetweenReplications();
00104     // virtual void _createInternalStatisticReporters() override;
00105     // virtual void _createNonEditableDataDefinitions() override;
00106     // virtual void _createEditableDataDefinitions() override;
00107     // virtual void _createAttachedAttributes() override;
00108     // virtual void _createInternalAndAttachedData(); /*< A ModelDataDefinition or ModelComponent that
00109     includes (internal) ou refers to (attach) other ModelDataDefinition must register them inside this
00110     method. */
00111     // virtual void _addSimulationControl(SimulationControl* property);
00112
00113 private:
00114     // ...
00115
00116 private:
00117     const struct DEFAULT_VALUES {
00118         const bool initialNode = false;
00119         const bool finalNode = false;
00120     } DEFAULT;
00121     bool _initialNode = DEFAULT.initialNode;
00122     bool _finalNode = DEFAULT.finalNode;
00123     List<DefaultNodeTransition*>* _transitions = new List<DefaultNodeTransition*>();
00124
00125 private:
00126     //DummyElement* _internalDataDefinition = nullptr;
00127
00128 private:
00129     // ...
00130 };

```

11.383 DefaultTransitionExtensions.h

```

00001 #pragma once
00002
00003 #include "DefaultNode.h"
00004 #include "plugins/components/ModalModel/PetriPlace.h"
00005 #include <map>
00006

```

```

00007 class EFSMTransition : public DefaultNodeTransition {
00008 public:
00009     EFSMTransition(DefaultNode* source, DefaultNode* destination, std::string name = "");
00010     virtual ~EFSMTransition() = default;
00011
00012 public:
00013     void setTriggerEvent(std::string triggerEvent);
00014     std::string getTriggerEvent() const;
00015     void setProbabilityExpression(std::string probabilityExpression);
00016     std::string getProbabilityExpression() const;
00017
00018 public:
00019     virtual bool canFire(Model* model, Entity* entity) const override;
00020     virtual void execute(Model* model, Entity* entity) const override;
00021
00022 private:
00023     std::string _triggerEvent = "";
00024     std::string _probabilityExpression = "";
00025 };
00026
00027 class PetriTransition : public DefaultNodeTransition {
00028 public:
00029     PetriTransition(DefaultNode* source, DefaultNode* destination, std::string name = "");
00030     virtual ~PetriTransition() = default;
00031
00032 public:
00033     void setInputArcWeight(std::string color, unsigned int weight);
00034     void setOutputArcWeight(std::string color, unsigned int weight);
00035
00036 public:
00037     virtual bool canFire(Model* model, Entity* entity) const override;
00038     virtual void execute(Model* model, Entity* entity) const override;
00039
00040 private:
00041     std::map<std::string, unsigned int> _inputArcWeights;
00042     std::map<std::string, unsigned int> _outputArcWeights;
00043 };

```

11.384 FSMState.h

```

00001 #pragma once
00002
00003 #include "DefaultNode.h"
00004
00005 class FSMState : public DefaultNode {
00006 public:
00007     FSMState(Model* model, std::string name = "");
00008     virtual ~FSMState() = default;
00009
00010 public:
00011     void setEntryActionExpression(std::string expression);
00012     std::string getEntryActionExpression() const;
00013     void setExitActionExpression(std::string expression);
00014     std::string getExitActionExpression() const;
00015
00016 public: // static
00017     static PluginInformation* GetPluginInformation();
00018     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00019     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00020
00021 protected:
00022     virtual bool _loadInstance(PersistenceRecord *fields) override;
00023     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00024
00025 private:
00026     const struct DEFAULT_VALUES {
00027         const std::string entryActionExpression = "";
00028         const std::string exitActionExpression = "";
00029     } DEFAULT;
00030     std::string _entryActionExpression = DEFAULT.entryActionExpression;
00031     std::string _exitActionExpression = DEFAULT.exitActionExpression;
00032 };

```

11.385 ModalModelDefault.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.

```

```

00005  */
00006
00007  /*
00008  * File:      ModalModelDefault.h
00009  * Author:    Prof. Rafael Luiz Cancian, Dr. Eng.
00010  *
00011  * Created on 01 de Julho de 2025, 14:26
00012  */
00013
00014  #pragma once
00015
00016  #include "../kernel/simulator/model/ModelComponent.h"
00017  #include "DefaultNode.h"
00021  class ModalModelDefault : public ModelComponent {
00022  public:
00023      ModalModelDefault(Model* model, std::string name = "");
00024      virtual ~ModalModelDefault() = default;
00025
00026  public: // getters and setters
00027      virtual void setMaxTransitionsPerDispatch(unsigned int maxTransitionsPerDispatch);
00028      virtual unsigned int getMaxTransitionsPerDispatch() const;
00029      virtual List<DefaultNode*>* getNodes() const;
00030      virtual List<DefaultNodeTransition*>* getTransitions() const;
00031      DefaultNode* getEntryNode();
00032      void setEntryNode(DefaultNode* const entry_node);
00033      DefaultNode* getCurrentNode();
00034      std::string getTimeDelayExpressionPerDispatch();
00035      void setTimeDelayExpressionPerDispatch(const std::string time_delay_expression_per_dispatch);
00036      Util::TimeUnit getTimeDelayPerDispatchTimeUnit();
00037      void setTimeDelayPerDispatchTimeUnit(const Util::TimeUnit time_delay_per_dispatch_time_unit);
00038
00039  public:
00040      virtual void addNode(DefaultNode* node);
00041      virtual void removeNode(DefaultNode* node);
00042      virtual void addTransition(DefaultNodeTransition* transition);
00043      virtual void removeTransition(DefaultNodeTransition* transition);
00044      virtual void addOutputExpressionReference(ModelDataDefinition* expressionReference);
00045      virtual void removeOutputExpressionReference(DefaultNodeTransition* expressionReference);
00046
00047  public:
00048      virtual std::string show() override;
00049
00050  public:
00051      static PluginInformation* GetPluginInformation();
00052      static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00053      static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00054
00055  protected:
00056      virtual bool _loadInstance(PersistenceRecord *fields) override;
00057      virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00058      virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00059
00060  protected:
00062      virtual bool _check(std::string& errorMessage) override;
00064      // virtual ParserChangesInformation* _getParserChangesInformation();
00066      virtual void _initBetweenReplications() override;
00068      // virtual void _createInternalAndAttachedData() override;
00070      // virtual void _addSimulationControl(SimulationControl* property);
00071
00072  // sections bellow are *protected* since ModalModelDefault is intended to be extended
00073  protected:
00074
00076      const struct DEFAULT_VALUES {
00077          const std::string timeDelayExpressionPerDispatch = "0";
00078          const Util::TimeUnit timeDelayPerDispatchTimeUnit = Util::TimeUnit::second;
00079          const unsigned int maxTransitionsPerDispatch = 1;
00080      } DEFAULT;
00081      //std::string _entryNodeName = DEFAULT.entryNodeName;
00082      unsigned int _maxTransitionsPerDispatch = DEFAULT.maxTransitionsPerDispatch;
00083      std::string _timeDelayExpressionPerDispatch = DEFAULT.timeDelayExpressionPerDispatch;
00084      Util::TimeUnit _timeDelayPerDispatchTimeUnit = DEFAULT.timeDelayPerDispatchTimeUnit;
00085
00086  protected:
00087      DefaultNode* _currentNode = nullptr;
00088      DefaultNode* _entryNode = nullptr;
00089
00090  protected:
00091      List<DefaultNode*>* _nodes = new List<DefaultNode*>();
00092      List<DefaultNodeTransition*>* _transitions = new List<DefaultNodeTransition*>();
00093      //List<ModelDataDefinition*>* _expressionDataReferences = new List<ModelDataDefinition*>();
00094
00095  protected:
00096      // virtual void _createInternalStatisticReporters() override;
00097      // virtual void _createNonEditableDataDefinitions() override;
00098      // virtual void _createEditableDataDefinitions() override;
00099      virtual void _createAttachedAttributes() override;
00100

```

```

00101 private:
00102     // ...
00103 private:
00104     // ...
00105
00106 private:
00107     // ...
00108
00109
00110 };

```

11.386 ModalModelFSM.h

```

00001 #pragma once
00002
00003 #include "FSMState.h"
00004 #include "plugins/components/ModalModel/ModalModelDefault.h"
00005
00006 class ModalModelFSM : public ModalModelDefault {
00007 public:
00008     ModalModelFSM(Model* model, std::string name = "");
00009     virtual ~ModalModelFSM() = default;
00010 public:
00011
00012 public: // static
00013     static PluginInformation* GetPluginInformation();
00014     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00015     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00016
00017 protected:
00018     virtual bool _check(std::string& errorMessage) override;
00019     // virtual ParserChangesInformation* _getParserChangesInformation();
00020     virtual void _initBetweenReplications();
00021     // virtual void _addSimulationControl(SimulationControl* property);
00022
00023 private:
00024     FSMState* _fsmInitialState;
00025
00026
00027 };

```

11.387 ModalModelPetriNet.h

```

00001 #pragma once
00002
00003 #include "plugins/components/ModalModel/ModalModelDefault.h"
00004
00005 class ModalModelPetriNet : public ModalModelDefault {
00006 public:
00007     ModalModelPetriNet(Model* model, std::string name = "");
00008     virtual ~ModalModelPetriNet() = default;
00009
00010 public: // static
00011     static PluginInformation* GetPluginInformation();
00012     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00013     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00014
00015 protected:
00016     virtual bool _check(std::string& errorMessage) override;
00017 };

```

11.388 PetriPlace.h

```

00001 #pragma once
00002
00003 #include "DefaultNode.h"
00004 #include <map>
00005
00006 class PetriPlace : public DefaultNode {
00007 public:
00008     PetriPlace(Model* model, std::string name = "");
00009     virtual ~PetriPlace() = default;
00010
00011 public:
00012     unsigned int getTokens(std::string color = "default") const;
00013     void addTokens(unsigned int quantity, std::string color = "default");

```



```

00014     bool removeTokens(unsigned int quantity, std::string color = "default");
00015     std::map<std::string, unsigned int>* getAllTokens();
00016
00017 public: // static
00018     static PluginInformation* GetPluginInformation();
00019     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00020     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00021
00022 protected:
00023     virtual bool _loadInstance(PersistenceRecord *fields) override;
00024     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00025
00026 private:
00027     std::map<std::string, unsigned int>* _tokensByColor = new std::map<std::string, unsigned int>();
00028 };

```

11.389 Submodel.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Submodel.h
00009  * Author: rlcancian
00010  *
00011  * Created on 11 de Setembro de 2019, 13:04
00012  */
00013
00014 #ifndef SUBMODEL_H
00015 #define SUBMODEL_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018
00022 class Submodel : public ModelComponent {
00023 public: // constructors
00024     Submodel(Model* model, std::string name = "");
00025     virtual ~Submodel() = default;
00026 public: // virtual
00027     virtual std::string show() override;
00028 public: // static
00029     static PluginInformation* GetPluginInformation();
00030     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00031     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00032 protected: // virtual
00033     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00034     virtual bool _loadInstance(PersistenceRecord *fields) override;
00035     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00036 protected: // virtual
00037     //virtual void _initBetweenReplications();
00038     // virtual void _createInternalStatisticReporters() override;
00039     // virtual void _createNonEditableDataDefinitions() override;
00040     // virtual void _createEditableDataDefinitions() override;
00041     // virtual void _createAttachedAttributes() override;
00042     virtual bool _check(std::string& errorMessage) override;
00043
00044 protected:
00045
00046 private: // methods
00047 private: // attributes 1:1
00048 private: // attributes 1:n
00049 };
00050
00051
00052 #endif /* SUBMODEL_H */

```

11.390 Match.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Match.h
00009  * Author: rlcancian

```

```

00010  *
00011  * Created on 03 de Junho de 2019, 15:14
00012  */
00013
00014 #ifndef MATCH_H
00015 #define MATCH_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "plugins/data/DiscreteProcessing/Queue.h"
00019
00048 class Match : public ModelComponent {
00049 public:
00050
00051     enum class Rule : int {
00052         Any = 0, ByAttribute = 1, num_elements = 2
00053     };
00054 public:
00055     static std::string convertEnumToStr(Rule rule);
00056 public: // constructors
00057     Match(Model* model, std::string name = "");
00058     virtual ~Match() override;
00059 public: // virtual
00060     virtual std::string show() override;
00061 public: // static
00062     static PluginInformation* GetPluginInformation();
00063     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00064     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00065 public: // virtual
00066     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00067     virtual bool _loadInstance(PersistenceRecord *fields) override;
00068     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00069 public:
00070     void setRule(Match::Rule _rule);
00071     Match::Rule getRule() const;
00072     void setAttributeName(std::string _attributeName);
00073     std::string getAttributeName() const;
00074     void setMatchSize(std::string _matchSize);
00075     std::string getMatchSize() const;
00076     void setNumberOfQueues(unsigned int _numberOfQueues);
00077     unsigned int getNumberOfQueues() const;
00078 protected: // virtual
00079     //virtual void _initBetweenReplications();
00080     // virtual void _createInternalStatisticReporters() override;
00081     // virtual void _createNonEditableDataDefinitions() override;
00082     virtual void _createEditableDataDefinitions() override;
00083     // virtual void _createAttachedAttributes() override;
00084     virtual bool _check(std::string& errorMessage) override;
00085     // virtual void _createInternalAndAttachedData() override;
00086
00087 protected:
00088
00089 private: // methods
00090 private: // attributes 1:1
00091
00092     const struct DEFAULT_VALUES {
00093         const Match::Rule rule = Match::Rule::Any;
00094         const unsigned int numberOfQueues = 2;
00095         const std::string matchSize = "1";
00096         const std::string attributeName = "";
00097     } DEFAULT;
00098     Match::Rule _rule = DEFAULT.rule;
00099     unsigned int _numberOfQueues = DEFAULT.numberOfQueues;
00100     std::string _matchSize = DEFAULT.matchSize;
00101     std::string _attributeName = DEFAULT.attributeName;
00102 private: // attributes 1:n
00103     std::map<Queue*, std::map<double, unsigned int>*>* _entitiesByAttrib = new std::map<Queue*,
00104     std::map<double, unsigned int>*>();
00105     List<Queue*>* _queues = new List<Queue*>();
00106 };
00107
00108
00109 #endif /* MATCH_H */

```

11.391 Signal.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*

```

```

00008  * File:    Signal.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 03 de Junho de 2019, 15:20
00012  */
00013
00014 #ifndef SIGNAL_H
00015 #define SIGNAL_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "../data/Synchronization/SignalData.h"
00019
00039 class Signal : public ModelComponent {
00040 public: // constructors
00041     Signal(Model* model, std::string name = "");
00042     virtual ~Signal() = default;
00043 public: // virtual
00044     virtual std::string show() override;
00045 public:
00046     void setSignalData(SignalData* signal);
00047     const std::string&limitExpression() const;
00048     void setLimitExpression(const std::string&newLimitExpression);
00049 public: // static
00050     static PluginInformation* GetPluginInformation();
00051     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00052     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00053
00054 protected: // must be overridden
00055     virtual bool _loadInstance(PersistenceRecord *fields) override;
00056     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00057     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00058 protected: // could be overridden
00059     virtual bool _check(std::string& errorMessage) override;
00060     virtual void _initBetweenReplications() override;
00061     // virtual void _createInternalAndAttachedData() override;
00062     //virtual ParserChangesInformation* _getParserChangesInformation();
00063
00064 protected:
00065     // virtual void _createInternalStatisticReporters() override;
00066     // virtual void _createNonEditableDataDefinitions() override;
00067     virtual void _createEditableDataDefinitions() override;
00068     // virtual void _createAttachedAttributes() override;
00069
00070 private: // methods
00071 private: // attributes 1:1
00072     const struct DEFAULT_VALUES {
00073         const std::string limitExpression = "1";
00074     } DEFAULT;
00075     std::string _limitExpression = DEFAULT.limitExpression;
00076     unsigned int _signalsTriggered = 0;
00077 private: // attributes 1:n
00078 private: // attached
00079     SignalData* _signalData = nullptr;
00080
00081 };
00082
00083
00084 #endif /* SIGNAL_H */

```

11.392 Wait.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Wait.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 03 de Junho de 2019, 15:20
00012  */
00013
00014 #ifndef WAIT_H
00015 #define WAIT_H
00016
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "plugins/data/DiscreteProcessing/Queue.h"
00019 #include "../data/Synchronization/SignalData.h"
00020 #include "kernel/simulator/OnEventManager.h"
00021
00078 class Wait : public ModelComponent {

```

```

00079 public:
00080
00081     enum class WaitType : int {
00082         WaitForSignal = 0, InfiniteHold = 1, ScanForCondition = 2, num_elements = 3
00083     };
00084 public:
00085     static std::string convertEnumToStr(WaitType type);
00086 public: // constructors
00087     Wait(Model* model, std::string name = "");
00088     virtual ~Wait() = default;
00089 public: // virtual
00090     virtual std::string show() override;
00091 public: //
00092     void setSignalData(SignalData* signal);
00093     void setWaitType(WaitType _watType);
00094     Wait::WaitType getWaitType() const;
00095     void setCondition(std::string _condition);
00096     std::string getCondition() const;
00097     Queue* getQueue() const;
00098     void setQueue(Queue* queue);
00099 public: // static
00100     static PluginInformation* GetPluginInformation();
00101     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00102     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00103     std::string getLimitExpression() const;
00104     void setLimitExpression(const std::string &newLimitExpression);
00105
00106 protected: // must be overridden
00107     virtual bool _loadInstance(PersistenceRecord *fields) override;
00108     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00109     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00110 protected: // could be overridden
00111     virtual bool _check(std::string& errorMessage) override;
00112     virtual void _initBetweenReplications() override;
00113     // virtual void _createInternalAndAttachedData() override;
00114     //virtual ParserChangesInformation* _getParserChangesInformation();
00115
00116 protected:
00117     //virtual void _createInternalStatisticReporters() override;
00118     virtual void _createNonEditableDataDefinitions() override;
00119     virtual void _createEditableDataDefinitions() override;
00120     //virtual void _createAttachedAttributes() override;
00121
00122 private: // methods
00123     unsigned int _handlerForSignalDataEvent(SignalData* signalData);
00124     void _handlerForAfterProcessEventEvent(SimulationEvent* event);
00125 private: // attributes 1:1
00126
00127     const struct DEFAULT_VALUES {
00128         const WaitType waitType = Wait::WaitType::WaitForSignal;
00129         const std::string condition = "";
00130         const std::string limitExpression = "0";
00131     } DEFAULT;
00132     WaitType _waitType = DEFAULT.waitType;
00133     std::string _condition = DEFAULT.condition;
00134     std::string limitExpression = DEFAULT.limitExpression;
00135 private: // internal
00136     Queue *_queue = nullptr; // @TODO: It should be a QueueableItem, (Queue or Set)
00137 private: // attached
00138     SignalData* _signalData = nullptr;
00139     bool _isScanConditionHandlerRegistered = false; // local guard to avoid duplicate registration in
repeated checks
00140 private: // attributes 1:n
00141 };
00142
00143
00144 #endif /* WAIT_H */

```

11.393 DummyComponent.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Dummy.h
00009  * Author:  Prof. Rafael Luiz Cancian, Dr. Eng.
00010  *
00011  * Created on 22 de Maio de 2019, 18:41
00012  */
00013

```

```

00014 #pragma once
00015
00016 #include "../kernel/simulator/model/ModelComponent.h"
00017 #include "plugins/data/Template/DummyElement.h"
00018
00022 class DummyComponent : public ModelComponent {
00023 public:
00024     DummyComponent(Model* model, std::string name = "");
00025     virtual ~DummyComponent() = default;
00026
00027 public:
00028     // ...
00029
00030 public:
00031     virtual std::string show() override;
00032
00033 public:
00034     static PluginInformation* GetPluginInformation();
00035     static ModelComponent* LoadInstance(Model* model, PersistenceRecord *fields);
00036     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00037
00038 protected:
00039     virtual bool _loadInstance(PersistenceRecord *fields) override;
00040     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00041     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00042
00043
00044 protected:
00045     // virtual bool _check(std::string& errorMessage);
00046     // virtual ParserChangesInformation* _getParserChangesInformation();
00047     // virtual void _initBetweenReplications();
00048     // virtual void _createInternalStatisticReporters() override;
00049     // virtual void _createNonEditableDataDefinitions() override;
00050     // virtual void _createEditableDataDefinitions() override;
00051     // virtual void _createAttachedAttributes() override;
00052     // virtual void _createInternalAndAttachedData(); /*< A ModelDataDefinition or ModelComponent that
includes (internal) ou refers to (attach) other ModelDataDefinition must register them inside this
method. */
00058     // virtual void _addSimulationControl(SimulationControl* property);
00059
00060
00061 protected:
00062
00063 private:
00064     // ...
00065
00066 private:
00067
00069     const struct DEFAULT_VALUES {
00070         const std::string someString = "Test";
00071         const unsigned int someUInt = 1;
00072     } DEFAULT;
00073     std::string _someString = DEFAULT.someString;
00074     unsigned int _someUInt = DEFAULT.someUInt;
00075
00076 private:
00077     DummyElement* _internalDataDefinition = nullptr;
00078
00079 private:
00080     // ...
00081 };

```

11.394 BioStateProjectionComponent.h

```

00001 #ifndef BIONETWORKPROJECTIONCOMPONENT_H
00002 #define BIONETWORKPROJECTIONCOMPONENT_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelComponent.h"
00008 #include "kernel/simulator/Plugin.h"
00009 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00010
00011 class BioSpecies;
00012
00022 class BioStateProjectionComponent : public ModelComponent {
00023 public:
00024     enum class ProjectionKind {
00025         MoleculeCount,
00026         MetaboliteAmount
00027     };
00028

```

```

00029     struct Projection {
00030         std::string sourceSpeciesName;
00031         std::string targetKey;
00032         ProjectionKind kind = ProjectionKind::MoleculeCount;
00033         double scale = 1.0;
00034         double offset = 0.0;
00035     };
00036
00037 public:
00038     BioStateProjectionComponent(Model* model, std::string name = "");
00039     virtual ~BioStateProjectionComponent() override = default;
00040
00041 public:
00042     static PluginInformation* GetPluginInformation();
00043     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00044     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00045
00046 public:
00047     void setWholeCellState(WholeCellState* state);
00048     WholeCellState* getWholeCellState() const;
00049     void clearProjections();
00050     void addMoleculeProjection(const std::string& sourceSpeciesName, const std::string& targetKey =
00051 "",
00052                                double scale = 1.0, double offset = 0.0);
00053     void addMetaboliteProjection(const std::string& sourceSpeciesName, const std::string& targetKey =
00054 "",
00055                                double scale = 1.0, double offset = 0.0);
00056     unsigned int getProjectionCount() const;
00057     void setLastSucceeded(bool lastSucceeded);
00058     bool getLastSucceeded() const;
00059     void setLastMessage(std::string lastMessage);
00060     std::string getLastMessage() const;
00061     virtual std::string show() override;
00062
00063 protected:
00064     virtual bool _loadInstance(PersistenceRecord* fields) override;
00065     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00066     virtual bool _check(std::string& errorMessage) override;
00067     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00068     virtual void _createEditableDataDefinitions() override;
00069
00070 private:
00071     void _forwardEntity(Entity* entity);
00072     void _syncEditableDataDefinitions();
00073
00074 private:
00075     const struct DEFAULT_VALUES {
00076         std::string wholeCellStateName = "";
00077         bool lastSucceeded = false;
00078         std::string lastMessage = "";
00079     } DEFAULT;
00080
00081     WholeCellState* _wholeCellState = nullptr;
00082     std::vector<Projection> _projections;
00083     bool _lastSucceeded = DEFAULT.lastSucceeded;
00084     std::string _lastMessage = DEFAULT.lastMessage;
00085 };
00086 #endif /* BIONETWORKPROJECTIONCOMPONENT_H */

```

11.395 CellCycleCheckpointComponent.h

```

00001 #ifndef CELLCYCLECHECKPOINTCOMPONENT_H
00002 #define CELLCYCLECHECKPOINTCOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00009
00027 class CellCycleCheckpointComponent : public ModelComponent {
00028 public:
00029     CellCycleCheckpointComponent(Model* model, std::string name = "");
00030     virtual ~CellCycleCheckpointComponent() override = default;
00031
00032 public:
00033     static PluginInformation* GetPluginInformation();
00034     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00035     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00036
00037 public:

```

```

00038     void setWholeCellState(WholeCellState* state);
00039     WholeCellState* getWholeCellState() const;
00040     void setDeltaT(double deltaT);
00041     double getDeltaT() const;
00042     void setAdvanceWholeCellClock(bool advance);
00043     bool getAdvanceWholeCellClock() const;
00044     void setEnergyMetaboliteKey(std::string key);
00045     std::string getEnergyMetaboliteKey() const;
00046     void setCompartmentEnergyRegion(std::string region);
00047     std::string getCompartmentEnergyRegion() const;
00048     void setCompartmentEnergyMetaboliteKey(std::string key);
00049     std::string getCompartmentEnergyMetaboliteKey() const;
00050     void setStarvationAtpThreshold(double threshold);
00051     double getStarvationAtpThreshold() const;
00052     void setCompartmentStarvationThreshold(double threshold);
00053     double getCompartmentStarvationThreshold() const;
00054     void setLethalStarvationSteps(unsigned int steps);
00055     unsigned int getLethalStarvationSteps() const;
00056     void setDivisionMassThreshold(double threshold);
00057     double getDivisionMassThreshold() const;
00058     void setFtsZRingKey(std::string key);
00059     std::string getFtsZRingKey() const;
00060     void setFtsZThreshold(double threshold);
00061     double getFtsZThreshold() const;
00062     void setCriticalPathwayActivityKey(std::string key);
00063     std::string getCriticalPathwayActivityKey() const;
00064     void setCriticalPathwayActivityThreshold(double threshold);
00065     double getCriticalPathwayActivityThreshold() const;
00066     unsigned int getStarvationStreak() const;
00067
00068     virtual std::string show() override;
00069
00070 protected:
00071     virtual bool _loadInstance(PersistenceRecord* fields) override;
00072     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00073     virtual bool _check(std::string& errorMessage) override;
00074     virtual void _initBetweenReplications() override;
00075     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00076
00077 protected:
00078     virtual void _createEditableDataDefinitions() override;
00079
00080 private:
00081     bool _isDivisionReady() const;
00082     void _updateLifecyclePhase();
00083     void _forwardEntity(Entity* entity);
00084
00085 private:
00086     const struct DEFAULT_VALUES {
00087         std::string wholeCellStateName = "";
00088         double deltaT = 60.0;
00089         bool advanceWholeCellClock = true;
00090         std::string energyMetaboliteKey = "ATP";
00091         std::string compartmentEnergyRegion = "";
00092         std::string compartmentEnergyMetaboliteKey = "";
00093         double starvationAtpThreshold = 0.25;
00094         double compartmentStarvationThreshold = 0.0;
00095         unsigned int lethalStarvationSteps = 0u;
00096         double divisionMassThreshold = 0.0;
00097         std::string ftsZRingKey = "FtsZ_ring_completion";
00098         double ftsZThreshold = 0.0;
00099         std::string criticalPathwayActivityKey = "";
00100         double criticalPathwayActivityThreshold = 0.0;
00101     } DEFAULT;
00102
00103     WholeCellState* _wholeCellState = nullptr;
00104     double _deltaT = DEFAULT.deltaT;
00105     bool _advanceWholeCellClock = DEFAULT.advanceWholeCellClock;
00106     std::string _energyMetaboliteKey = DEFAULT.energyMetaboliteKey;
00107     std::string _compartmentEnergyRegion = DEFAULT.compartmentEnergyRegion;
00108     std::string _compartmentEnergyMetaboliteKey = DEFAULT.compartmentEnergyMetaboliteKey;
00109     double _starvationAtpThreshold = DEFAULT.starvationAtpThreshold;
00110     double _compartmentStarvationThreshold = DEFAULT.compartmentStarvationThreshold;
00111     unsigned int _lethalStarvationSteps = DEFAULT.lethalStarvationSteps;
00112     double _divisionMassThreshold = DEFAULT.divisionMassThreshold;
00113     std::string _ftsZRingKey = DEFAULT.ftsZRingKey;
00114     double _ftsZThreshold = DEFAULT.ftsZThreshold;
00115     std::string _criticalPathwayActivityKey = DEFAULT.criticalPathwayActivityKey;
00116     double _criticalPathwayActivityThreshold = DEFAULT.criticalPathwayActivityThreshold;
00117     unsigned int _starvationStreak = 0u;
00118 };
00119
00120 #endif /* CELLCYCLECHECKPOINTCOMPONENT_H */

```

11.396 CellDivisionEvent.h

```

00001 #ifndef CELLDIVISIONEVENT_H
00002 #define CELLDIVISIONEVENT_H
00003
00004 #include <random>
00005 #include <string>
00006
00007 #include "../kernel/simulator/model/ModelComponent.h"
00008 #include "kernel/simulator/Plugin.h"
00009 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00010
00039 class CellDivisionEvent : public ModelComponent {
00040 public:
00041     CellDivisionEvent(Model* model, std::string name = "");
00042     virtual ~CellDivisionEvent() override = default;
00043
00044 public:
00045     static PluginInformation* GetPluginInformation();
00046     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00047     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00048
00049 public:
00050     void setWholeCellState(WholeCellState* state);
00051     WholeCellState* getWholeCellState() const;
00052     void setDivisionMassThreshold(double threshold);
00053     double getDivisionMassThreshold() const;
00054     void setFtsZThreshold(double threshold);
00055     double getFtsZThreshold() const;
00056     void setFtsZRingKey(std::string key);
00057     std::string getFtsZRingKey() const;
00058     void setRandomSeed(unsigned int seed);
00059     unsigned int getRandomSeed() const;
00060     bool getLastDivided() const;
00061     int getDivisionCount() const;
00062
00063     virtual std::string show() override;
00064
00065 protected:
00066     virtual bool _loadInstance(PersistenceRecord* fields) override;
00067     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00068     virtual bool _check(std::string& errorMessage) override;
00069     virtual void _initBetweenReplications() override;
00070     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00071
00072 protected:
00073     virtual void _createEditableDataDefinitions() override;
00074
00075 private:
00076     bool _checkDivisionCondition() const;
00077     void _applyDivision();
00078     void _forwardEntity(Entity* entity, unsigned int port);
00079
00080 private:
00081     const struct DEFAULT_VALUES {
00082         std::string wholeCellStateName = "";
00083         double divisionMassThreshold = 7.86e-15; // 2 x 3.93e-15 g (M. genitalium,
parameters.json)
00084         double ftsZThreshold = 0.0; // 0 = mass-only trigger; >0 requires FtsZ ring
00085         std::string ftsZRingKey = "FtsZ_ring_completion";
00086         unsigned int randomSeed = 44u;
00087     } DEFAULT;
00088
00089     WholeCellState* _wholeCellState = nullptr;
00090     double _divisionMassThreshold = DEFAULT.divisionMassThreshold;
00091     double _ftsZThreshold = DEFAULT.ftsZThreshold;
00092     std::string _ftsZRingKey = DEFAULT.ftsZRingKey;
00093     unsigned int _randomSeed = DEFAULT.randomSeed;
00094     bool _lastDivided = false;
00095     int _divisionCount = 0;
00096     std::mt19937 _rng{DEFAULT.randomSeed};
00097 };
00098
00099 #endif /* CELLDIVISIONEVENT_H */

```

11.397 CellFateDecisionComponent.h

```

00001 #ifndef CELLFATEDECISIONCOMPONENT_H
00002 #define CELLFATEDECISIONCOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"

```



```

00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00009
00024 class CellFateDecisionComponent : public ModelComponent {
00025 public:
00026     CellFateDecisionComponent(Model* model, std::string name = "");
00027     virtual ~CellFateDecisionComponent() override = default;
00028
00029 public:
00030     static PluginInformation* GetPluginInformation();
00031     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00032     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00033
00034 public:
00035     void setWholeCellState(WholeCellState* state);
00036     WholeCellState* getWholeCellState() const;
00037     void setDivisionReadyPhase(std::string phase);
00038     std::string getDivisionReadyPhase() const;
00039     void setStarvedPhase(std::string phase);
00040     std::string getStarvedPhase() const;
00041     void setArrestedPhase(std::string phase);
00042     std::string getArrestedPhase() const;
00043     void setDeadPhase(std::string phase);
00044     std::string getDeadPhase() const;
00045     unsigned int getLastRoutedPort() const;
00046
00047     virtual std::string show() override;
00048
00049 protected:
00050     virtual bool _loadInstance(PersistenceRecord* fields) override;
00051     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00052     virtual bool _check(std::string& errorMessage) override;
00053     virtual void _initBetweenReplications() override;
00054     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00055
00056 protected:
00057     virtual void _createEditableDataDefinitions() override;
00058
00059 private:
00060     unsigned int _selectOutputPort() const;
00061     void _forwardEntity(Entity* entity, unsigned int port);
00062
00063 private:
00064     const struct DEFAULT_VALUES {
00065         std::string wholeCellStateName = "";
00066         std::string divisionReadyPhase = "division_ready";
00067         std::string starvedPhase = "starved";
00068         std::string arrestedPhase = "arrested";
00069         std::string deadPhase = "dead";
00070         unsigned int lastRoutedPort = 0u;
00071     } DEFAULT;
00072
00073     WholeCellState* _wholeCellState = nullptr;
00074     std::string _divisionReadyPhase = DEFAULT.divisionReadyPhase;
00075     std::string _starvedPhase = DEFAULT.starvedPhase;
00076     std::string _arrestedPhase = DEFAULT.arrestedPhase;
00077     std::string _deadPhase = DEFAULT.deadPhase;
00078     unsigned int _lastRoutedPort = DEFAULT.lastRoutedPort;
00079 };
00080
00081 #endif /* CELLFATEDECISIONCOMPONENT_H */

```

11.398 CellGrowthComponent.h

```

00001 #ifndef CELLGROWTHCOMPONENT_H
00002 #define CELLGROWTHCOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00009
00022 class CellGrowthComponent : public ModelComponent {
00023 public:
00024     CellGrowthComponent(Model* model, std::string name = "");
00025     virtual ~CellGrowthComponent() override = default;
00026
00027 public:
00028     static PluginInformation* GetPluginInformation();
00029     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00030     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00031

```

```

00032 public:
00033     void setWholeCellState(WholeCellState* state);
00034     WholeCellState* getWholeCellState() const;
00035     void setGrowthRate(double rate);
00036     double getGrowthRate() const;
00037     void setEnergyMetaboliteKey(std::string key);
00038     std::string getEnergyMetaboliteKey() const;
00039     void setEnergyHalfSaturation(double value);
00040     double getEnergyHalfSaturation() const;
00041     void setDensity(double density);
00042     double getDensity() const;
00043     void setDeltaT(double dt);
00044     double getDeltaT() const;
00045
00046     virtual std::string show() override;
00047
00048 protected:
00049     virtual bool _loadInstance(PersistenceRecord* fields) override;
00050     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00051     virtual bool _check(std::string& errorMessage) override;
00052     virtual void _initBetweenReplications() override;
00053     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00054
00055 protected:
00056     virtual void _createEditableDataDefinitions() override;
00057
00058 private:
00059     void _forwardEntity(Entity* entity);
00060
00061 private:
00062     const struct DEFAULT_VALUES {
00063         std::string wholeCellStateName = "";
00064         double growthRate = 2.1393e-05; // /s -- M. genitalium (parameters.json)
00065         std::string energyMetaboliteKey = "";
00066         double energyHalfSaturation = 0.0; // 0 disables energy gating
00067         double density = 1100.0; // kg/m3 -- states.Geometry.density
00068         double deltaT = 60.0; // s -- must match clock step
00069     } DEFAULT;
00070
00071     WholeCellState* _wholeCellState = nullptr;
00072     double _growthRate = DEFAULT.growthRate;
00073     std::string _energyMetaboliteKey = DEFAULT.energyMetaboliteKey;
00074     double _energyHalfSaturation = DEFAULT.energyHalfSaturation;
00075     double _density = DEFAULT.density;
00076     double _deltaT = DEFAULT.deltaT;
00077 };
00078
00079 #endif /* CELLGROWTHCOMPONENT_H */

```

11.399 CompartmentExchangeComponent.h

```

00001 #ifndef COMPARTMENTEXCHANGECOMPONENT_H
00002 #define COMPARTMENTEXCHANGECOMPONENT_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelComponent.h"
00008 #include "kernel/simulator/Plugin.h"
00009 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00010
00024 class CompartmentExchangeComponent : public ModelComponent {
00025 public:
00026     struct ExchangeRule {
00027         std::string label;
00028         std::string sourceRegion;
00029         std::string sourceMetaboliteKey;
00030         std::string targetRegion;
00031         std::string targetMetaboliteKey;
00032         double exchangeFraction = 0.0;
00033         double baselineAmount = 0.0;
00034         std::string driverPathwayKey;
00035         double driverScale = 1.0;
00036         double maxTransferAmount = 0.0;
00037         bool conserveMass = true;
00038     };
00039
00040 public:
00041     CompartmentExchangeComponent(Model* model, std::string name = "");
00042     virtual ~CompartmentExchangeComponent() override = default;
00043
00044 public:
00045     static PluginInformation* GetPluginInformation();

```

```

00046     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00047     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00048
00049 public:
00050     void setWholeCellState(WholeCellState* state);
00051     WholeCellState* getWholeCellState() const;
00052     void setSourceRegion(std::string region);
00053     std::string getSourceRegion() const;
00054     void setSourceMetaboliteKey(std::string key);
00055     std::string getSourceMetaboliteKey() const;
00056     void setTargetRegion(std::string region);
00057     std::string getTargetRegion() const;
00058     void setTargetMetaboliteKey(std::string key);
00059     std::string getTargetMetaboliteKey() const;
00060     void setExchangeFraction(double fraction);
00061     double getExchangeFraction() const;
00062     void setBaselineAmount(double amount);
00063     double getBaselineAmount() const;
00064     void setDriverPathwayKey(std::string key);
00065     std::string getDriverPathwayKey() const;
00066     void setDriverScale(double scale);
00067     double getDriverScale() const;
00068     void setMaxTransferAmount(double amount);
00069     double getMaxTransferAmount() const;
00070     void setConserveMass(bool conserve);
00071     bool getConserveMass() const;
00072     void clearExchangeRules();
00073     void addExchangeRule(const std::string& label,
00074                          const std::string& sourceRegion,
00075                          const std::string& sourceMetaboliteKey,
00076                          const std::string& targetRegion,
00077                          const std::string& targetMetaboliteKey,
00078                          double exchangeFraction,
00079                          double baselineAmount = 0.0,
00080                          const std::string& driverPathwayKey = "",
00081                          double driverScale = 1.0,
00082                          double maxTransferAmount = 0.0,
00083                          bool conserveMass = true);
00084     unsigned int getExchangeRuleCount() const;
00085     const std::vector<ExchangeRule>& getExchangeRules() const;
00086     double getLastTransferAmount() const;
00087
00088     virtual std::string show() override;
00089
00090 protected:
00091     virtual bool _loadInstance(PersistenceRecord* fields) override;
00092     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00093     virtual bool _check(std::string& errorMessage) override;
00094     virtual void _initBetweenReplications() override;
00095     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00096     virtual void _createEditableDataDefinitions() override;
00097
00098 private:
00099     ExchangeRule _buildLegacyRule() const;
00100     std::vector<ExchangeRule> _getEffectiveRules() const;
00101     bool _checkRule(const ExchangeRule& rule, std::string& errorMessage, unsigned int index) const;
00102     double _computeTransferAmount(const ExchangeRule& rule) const;
00103     void _applyRule(const ExchangeRule& rule);
00104     double _getPoolAmount(const std::string& region, const std::string& key) const;
00105     void _setPoolAmount(const std::string& region, const std::string& key, double amount);
00106     void _forwardEntity(Entity* entity);
00107
00108 private:
00109     const struct DEFAULT_VALUES {
00110         std::string wholeCellStateName = "";
00111         std::string sourceRegion = "";
00112         std::string sourceMetaboliteKey = "";
00113         std::string targetRegion = "";
00114         std::string targetMetaboliteKey = "";
00115         double exchangeFraction = 0.0;
00116         double baselineAmount = 0.0;
00117         std::string driverPathwayKey = "";
00118         double driverScale = 1.0;
00119         double maxTransferAmount = 0.0;
00120         bool conserveMass = true;
00121         double lastTransferAmount = 0.0;
00122     } DEFAULT;
00123
00124     WholeCellState* _wholeCellState = nullptr;
00125     std::string _sourceRegion = DEFAULT.sourceRegion;
00126     std::string _sourceMetaboliteKey = DEFAULT.sourceMetaboliteKey;
00127     std::string _targetRegion = DEFAULT.targetRegion;
00128     std::string _targetMetaboliteKey = DEFAULT.targetMetaboliteKey;
00129     double _exchangeFraction = DEFAULT.exchangeFraction;
00130     double _baselineAmount = DEFAULT.baselineAmount;
00131     std::string _driverPathwayKey = DEFAULT.driverPathwayKey;
00132     double _driverScale = DEFAULT.driverScale;

```

```

00133     double _maxTransferAmount = DEFAULT.maxTransferAmount;
00134     bool _conserveMass = DEFAULT.conserveMass;
00135     std::vector<ExchangeRule> _exchangeRules;
00136     double _lastTransferAmount = DEFAULT.lastTransferAmount;
00137 };
00138
00139 #endif /* COMPARTMENTEXCHANGECOMPONENT_H */

```

11.400 EukaryoticCellCycleComponent.h

```

00001 #ifndef EUKARYOTICCELLCYCLECOMPONENT_H
00002 #define EUKARYOTICCELLCYCLECOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/BioCompartment.h"
00009 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00010
00039 class EukaryoticCellCycleComponent : public ModelComponent {
00040 public:
00041     EukaryoticCellCycleComponent(Model* model, std::string name = "");
00042     virtual ~EukaryoticCellCycleComponent() override = default;
00043
00044 public:
00045     static PluginInformation* GetPluginInformation();
00046     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00047     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00048
00049 public:
00050     void setWholeCellState(WholeCellState* state);
00051     WholeCellState* getWholeCellState() const;
00052     void setDeltaT(double deltaT);
00053     double getDeltaT() const;
00054     void setAdvanceWholeCellClock(bool advance);
00055     bool getAdvanceWholeCellClock() const;
00056
00057     void setCytosolCompartmentName(std::string name);
00058     std::string getCytosolCompartmentName() const;
00059     void setNucleusCompartmentName(std::string name);
00060     std::string getNucleusCompartmentName() const;
00061     void setMitochondrionCompartmentName(std::string name);
00062     std::string getMitochondrionCompartmentName() const;
00063     void setBudCompartmentName(std::string name);
00064     std::string getBudCompartmentName() const;
00065
00066     void setEnergyMetaboliteKey(std::string key);
00067     std::string getEnergyMetaboliteKey() const;
00068     void setCytosolicEnergyMetaboliteKey(std::string key);
00069     std::string getCytosolicEnergyMetaboliteKey() const;
00070     void setMitochondrialEnergyMetaboliteKey(std::string key);
00071     std::string getMitochondrialEnergyMetaboliteKey() const;
00072     void setBudEnergyMetaboliteKey(std::string key);
00073     std::string getBudEnergyMetaboliteKey() const;
00074
00075     void setGrowthFluxKey(std::string key);
00076     std::string getGrowthFluxKey() const;
00077     void setRespirationFluxKey(std::string key);
00078     std::string getRespirationFluxKey() const;
00079
00080     void setBudProgressKey(std::string key);
00081     std::string getBudProgressKey() const;
00082     void setDnaReplicationProgressKey(std::string key);
00083     std::string getDnaReplicationProgressKey() const;
00084     void setSpindleAssemblyProgressKey(std::string key);
00085     std::string getSpindleAssemblyProgressKey() const;
00086     void setMitoticExitProgressKey(std::string key);
00087     std::string getMitoticExitProgressKey() const;
00088
00089     void setGlobalAtpThreshold(double threshold);
00090     double getGlobalAtpThreshold() const;
00091     void setCytosolicAtpThreshold(double threshold);
00092     double getCytosolicAtpThreshold() const;
00093     void setMitochondrialAtpThreshold(double threshold);
00094     double getMitochondrialAtpThreshold() const;
00095     void setBudAtpThreshold(double threshold);
00096     double getBudAtpThreshold() const;
00097     void setGrowthFluxThreshold(double threshold);
00098     double getGrowthFluxThreshold() const;
00099     void setRespirationFluxThreshold(double threshold);
00100     double getRespirationFluxThreshold() const;
00101

```

```

00102     void setBudProgressRate(double rate);
00103     double getBudProgressRate() const;
00104     void setDnaReplicationRate(double rate);
00105     double getDnaReplicationRate() const;
00106     void setSpindleAssemblyRate(double rate);
00107     double getSpindleAssemblyRate() const;
00108     void setMitoticExitRate(double rate);
00109     double getMitoticExitRate() const;
00110
00111     void setBudProgressThreshold(double threshold);
00112     double getBudProgressThreshold() const;
00113     void setDnaReplicationThreshold(double threshold);
00114     double getDnaReplicationThreshold() const;
00115     void setSpindleAssemblyThreshold(double threshold);
00116     double getSpindleAssemblyThreshold() const;
00117     void setMitoticExitThreshold(double threshold);
00118     double getMitoticExitThreshold() const;
00119
00120     virtual std::string show() override;
00121
00122 protected:
00123     virtual bool _loadInstance(PersistenceRecord* fields) override;
00124     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00125     virtual bool _check(std::string& errorMessage) override;
00126     virtual void _initBetweenReplications() override;
00127     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00128     virtual void _createEditableDataDefinitions() override;
00129
00130 private:
00131     double _normalizedSignal(double value, double threshold) const;
00132     double _averageSignal(double a, double b) const;
00133     double _advanceProgress(const std::string& key, double increment) const;
00134     void _updateLifecyclePhase();
00135     void _forwardEntity(Entity* entity);
00136     bool _validateReferencedCompartment(const std::string& name, std::string* errorMessage) const;
00137     BioCompartment* _findCompartment(const std::string& name) const;
00138
00139 private:
00140     const struct DEFAULT_VALUES {
00141         std::string wholeCellStateName = "";
00142         double deltaT = 60.0;
00143         bool advanceWholeCellClock = true;
00144         std::string cytosolCompartmentName = "cytosol";
00145         std::string nucleusCompartmentName = "nucleus";
00146         std::string mitochondrionCompartmentName = "mitochondria";
00147         std::string budCompartmentName = "bud";
00148         std::string energyMetaboliteKey = "ATP";
00149         std::string cytosolicEnergyMetaboliteKey = "ATP_c";
00150         std::string mitochondrialEnergyMetaboliteKey = "ATP_m";
00151         std::string budEnergyMetaboliteKey = "ATP_bud";
00152         std::string growthFluxKey = "biomass_flux";
00153         std::string respirationFluxKey = "respiration_flux";
00154         std::string budProgressKey = "bud_growth_progress";
00155         std::string dnaReplicationProgressKey = "dna_replication_progress";
00156         std::string spindleAssemblyProgressKey = "spindle_assembly_progress";
00157         std::string mitoticExitProgressKey = "mitotic_exit_progress";
00158         double globalAtpThreshold = 0.50;
00159         double cytosolicAtpThreshold = 0.40;
00160         double mitochondrialAtpThreshold = 0.35;
00161         double budAtpThreshold = 0.20;
00162         double growthFluxThreshold = 4.0;
00163         double respirationFluxThreshold = 1.0;
00164         double budProgressRate = 0.30;
00165         double dnaReplicationRate = 0.24;
00166         double spindleAssemblyRate = 0.22;
00167         double mitoticExitRate = 0.20;
00168         double budProgressThreshold = 1.0;
00169         double dnaReplicationThreshold = 1.0;
00170         double spindleAssemblyThreshold = 1.0;
00171         double mitoticExitThreshold = 1.0;
00172     } DEFAULT;
00173
00174     WholeCellState* _wholeCellState = nullptr;
00175     double _deltaT = DEFAULT.deltaT;
00176     bool _advanceWholeCellClock = DEFAULT.advanceWholeCellClock;
00177     std::string _cytosolCompartmentName = DEFAULT.cytosolCompartmentName;
00178     std::string _nucleusCompartmentName = DEFAULT.nucleusCompartmentName;
00179     std::string _mitochondrionCompartmentName = DEFAULT.mitochondrionCompartmentName;
00180     std::string _budCompartmentName = DEFAULT.budCompartmentName;
00181     std::string _energyMetaboliteKey = DEFAULT.energyMetaboliteKey;
00182     std::string _cytosolicEnergyMetaboliteKey = DEFAULT.cytosolicEnergyMetaboliteKey;
00183     std::string _mitochondrialEnergyMetaboliteKey = DEFAULT.mitochondrialEnergyMetaboliteKey;
00184     std::string _budEnergyMetaboliteKey = DEFAULT.budEnergyMetaboliteKey;
00185     std::string _growthFluxKey = DEFAULT.growthFluxKey;
00186     std::string _respirationFluxKey = DEFAULT.respirationFluxKey;
00187     std::string _budProgressKey = DEFAULT.budProgressKey;
00188     std::string _dnaReplicationProgressKey = DEFAULT.dnaReplicationProgressKey;

```

```

00189     std::string _spindleAssemblyProgressKey = DEFAULT.spindleAssemblyProgressKey;
00190     std::string _mitoticExitProgressKey = DEFAULT.mitoticExitProgressKey;
00191     double _globalAtpThreshold = DEFAULT.globalAtpThreshold;
00192     double _cytosolicAtpThreshold = DEFAULT.cytosolicAtpThreshold;
00193     double _mitochondrialAtpThreshold = DEFAULT.mitochondrialAtpThreshold;
00194     double _budAtpThreshold = DEFAULT.budAtpThreshold;
00195     double _growthFluxThreshold = DEFAULT.growthFluxThreshold;
00196     double _respirationFluxThreshold = DEFAULT.respirationFluxThreshold;
00197     double _budProgressRate = DEFAULT.budProgressRate;
00198     double _dnaReplicationRate = DEFAULT.dnaReplicationRate;
00199     double _spindleAssemblyRate = DEFAULT.spindleAssemblyRate;
00200     double _mitoticExitRate = DEFAULT.mitoticExitRate;
00201     double _budProgressThreshold = DEFAULT.budProgressThreshold;
00202     double _dnaReplicationThreshold = DEFAULT.dnaReplicationThreshold;
00203     double _spindleAssemblyThreshold = DEFAULT.spindleAssemblyThreshold;
00204     double _mitoticExitThreshold = DEFAULT.mitoticExitThreshold;
00205 };
00206
00207 #endif /* EUKARYOTICCELLCYCLECOMPONENT_H */

```

11.401 FtsZPolymerizationComponent.h

```

00001 #ifndef FTSZPOLYMERIZATIONCOMPONENT_H
00002 #define FTSZPOLYMERIZATIONCOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00009
00035 class FtsZPolymerizationComponent : public ModelComponent {
00036 public:
00037     FtsZPolymerizationComponent(Model* model, std::string name = "");
00038     virtual ~FtsZPolymerizationComponent() override = default;
00039
00040 public:
00041     static PluginInformation* GetPluginInformation();
00042     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00043     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00044
00045 public:
00046     void setWholeCellState(WholeCellState* state);
00047     WholeCellState* getWholeCellState() const;
00048     void setActivationFwd(double rate);
00049     double getActivationFwd() const;
00050     void setActivationRev(double rate);
00051     double getActivationRev() const;
00052     void setVolumeNorm(double norm);
00053     double getVolumeNorm() const;
00054     void setDeltaT(double dt);
00055     double getDeltaT() const;
00056     void setFtsZRingKey(std::string key);
00057     std::string getFtsZRingKey() const;
00058     void setFtsZMonomerKey(std::string key);
00059     std::string getFtsZMonomerKey() const;
00060
00061     virtual std::string show() override;
00062
00063 protected:
00064     virtual bool _loadInstance(PersistenceRecord* fields) override;
00065     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00066     virtual bool _check(std::string& errorMessage) override;
00067     virtual void _initBetweenReplications() override;
00068     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00069
00070 protected:
00071     virtual void _createEditableDataDefinitions() override;
00072
00073 private:
00074     void _forwardEntity(Entity* entity);
00075
00076 private:
00077     const struct DEFAULT_VALUES {
00078         std::string wholeCellStateName = "";
00079         double activationFwd = 1.1; // /s -- processes.FtsZPolymerization.activationFwd
00080         double activationRev = 0.01; // /s -- derived from Karr 2012 ratio
00081         double volumeNorm = 3.94e-5; // dimensionless -- calibrated for 35% at ~12000 s
00082         double deltaT = 60.0; // s -- must match clock step
00083         std::string ftsZRingKey = "FtsZ_ring_completion";
00084         std::string ftsZMonomerKey = "FtsZ_monomer";
00085     } DEFAULT;
00086

```

```

00087     WholeCellState* _wholeCellState = nullptr;
00088     double _activationFwd = DEFAULT.activationFwd;
00089     double _activationRev = DEFAULT.activationRev;
00090     double _volumeNorm = DEFAULT.volumeNorm;
00091     double _deltaT = DEFAULT.deltaT;
00092     std::string _ftsZRingKey = DEFAULT.ftsZRingKey;
00093     std::string _ftsZMonomerKey = DEFAULT.ftsZMonomerKey;
00094     int _nRef = 0; // reference monomer count captured on first step
00095 };
00096
00097 #endif /* FTSZPOLYMERIZATIONCOMPONENT_H */

```

11.402 MetabolicStateProjectionComponent.h

```

00001 #ifndef METABOLICSTATEPROJECTIONCOMPONENT_H
00002 #define METABOLICSTATEPROJECTIONCOMPONENT_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelComponent.h"
00008 #include "kernel/simulator/Plugin.h"
00009 #include "plugins/components/BiochemicalSimulation/MetabolicFluxBalance.h"
00010 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00011
00027 class MetabolicStateProjectionComponent : public ModelComponent {
00028 public:
00029     enum class ProjectionKind {
00030         MetaboliteAmount,
00031         CompartmentMetaboliteAmount,
00032         PathwayActivity
00033     };
00034
00035     enum class ProjectionUpdateMode {
00036         Overwrite,
00037         Accumulate,
00038         Turnover
00039     };
00040
00041     struct Projection {
00042         std::string reactionName;
00043         std::string targetKey;
00044         ProjectionKind kind = ProjectionKind::MetaboliteAmount;
00045         ProjectionUpdateMode updateMode = ProjectionUpdateMode::Overwrite;
00046         double scale = 1.0;
00047         double offset = 0.0;
00048         double turnoverFraction = 0.0;
00049         std::string compartmentName;
00050     };
00051
00052 public:
00053     MetabolicStateProjectionComponent(Model* model, std::string name = "");
00054     virtual ~MetabolicStateProjectionComponent() override = default;
00055
00056 public:
00057     static PluginInformation* GetPluginInformation();
00058     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00059     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00060
00061 public:
00062     void setWholeCellState(WholeCellState* state);
00063     WholeCellState* getWholeCellState() const;
00064     void setFluxBalanceComponent(MetabolicFluxBalance* component);
00065     MetabolicFluxBalance* getFluxBalanceComponent() const;
00066     void setFluxBalanceComponentName(std::string componentName);
00067     std::string getFluxBalanceComponentName() const;
00068
00069     void clearObjectiveProjection();
00070     void setObjectiveAsMetabolite(const std::string& targetKey, double scale = 1.0, double offset =
00071 0.0,
00072                                     ProjectionUpdateMode updateMode = ProjectionUpdateMode::Overwrite,
00073                                     double turnoverFraction = 0.0);
00074     void setObjectiveAsCompartmentMetabolite(const std::string& compartmentName, const std::string&
00075 targetKey,
00076                                     double scale = 1.0, double offset = 0.0,
00077                                     ProjectionUpdateMode updateMode =
00078 ProjectionUpdateMode::Overwrite,
00079                                     double turnoverFraction = 0.0);
00080     void setObjectiveAsPathwayActivity(const std::string& targetKey, double scale = 1.0, double offset
00081 = 0.0,
00082                                     ProjectionUpdateMode updateMode =
00083 ProjectionUpdateMode::Overwrite,
00084                                     double turnoverFraction = 0.0);

```



```

00080     bool hasObjectiveProjection() const;
00081
00082     void clearFluxProjections();
00083     void addMetaboliteProjection(const std::string& reactionName, const std::string& targetKey,
00084                                double scale = 1.0, double offset = 0.0,
00085                                ProjectionUpdateMode updateMode = ProjectionUpdateMode::Overwrite,
00086                                double turnoverFraction = 0.0);
00087     void addCompartmentMetaboliteProjection(const std::string& reactionName, const std::string&
00088 compartmentName,
00089                                           const std::string& targetKey, double scale = 1.0, double
00090 offset = 0.0,
00091                                           ProjectionUpdateMode updateMode =
00092 ProjectionUpdateMode::Overwrite,
00093                                           double turnoverFraction = 0.0);
00094     void addPathwayProjection(const std::string& reactionName, const std::string& targetKey,
00095                              double scale = 1.0, double offset = 0.0,
00096                              ProjectionUpdateMode updateMode = ProjectionUpdateMode::Overwrite,
00097                              double turnoverFraction = 0.0);
00098     unsigned int getProjectionCount() const;
00099
00100     void setLastSucceeded(bool lastSucceeded);
00101     bool getLastSucceeded() const;
00102     void setLastMessage(std::string lastMessage);
00103     std::string getLastMessage() const;
00104
00105     virtual std::string show() override;
00106
00107 protected:
00108     virtual bool _loadInstance(PersistenceRecord* fields) override;
00109     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00110     virtual bool _check(std::string& errorMessage) override;
00111     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00112     virtual void _createEditableDataDefinitions() override;
00113
00114 private:
00115     void _forwardEntity(Entity* entity);
00116     void _syncEditableDataDefinitions();
00117     void _resolveFluxBalanceComponent();
00118     void _configureObjectiveProjection(ProjectionKind kind, const std::string& targetKey, const
00119 std::string& compartmentName,
00120                                     double scale, double offset, ProjectionUpdateMode updateMode,
00121                                     double turnoverFraction);
00122     void _appendProjection(const Projection& projection);
00123     void _applyProjection(const Projection& projection, double value);
00124     std::string _resolveCompartmentName(const Projection& projection) const;
00125     double _resolveCurrentValue(const Projection& projection) const;
00126
00127 private:
00128     const struct DEFAULT_VALUES {
00129         std::string wholeCellStateName = "";
00130         std::string fluxBalanceComponentName = "";
00131         bool objectiveProjectionEnabled = false;
00132         std::string objectiveTargetKey = "";
00133         std::string objectiveKind = "MetaboliteAmount";
00134         std::string objectiveUpdateMode = "Overwrite";
00135         double objectiveScale = 1.0;
00136         double objectiveOffset = 0.0;
00137         double objectiveTurnoverFraction = 0.0;
00138         std::string objectiveCompartmentName = "";
00139         bool lastSucceeded = false;
00140         std::string lastMessage = "";
00141     } DEFAULT;
00142
00143     WholeCellState* _wholeCellState = nullptr;
00144     MetabolicFluxBalance* _fluxBalanceComponent = nullptr;
00145     std::string _fluxBalanceComponentName = DEFAULT.fluxBalanceComponentName;
00146     bool _objectiveProjectionEnabled = DEFAULT.objectiveProjectionEnabled;
00147     Projection _objectiveProjection;
00148     std::vector<Projection> _projections;
00149     bool _lastSucceeded = DEFAULT.lastSucceeded;
00150     std::string _lastMessage = DEFAULT.lastMessage;
00151 };
00152
00153 #endif /* METABOLICSTATEPROJECTIONCOMPONENT_H */

```

11.403 MetabolicSubmodelComponent.h

```

00001 #ifndef METABOLICSUBMODELCOMPONENT_H
00002 #define METABOLICSUBMODELCOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"

```



```

00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00009
00029 class MetabolicSubmodelComponent : public ModelComponent {
00030 public:
00031     MetabolicSubmodelComponent(Model* model, std::string name = "");
00032     virtual ~MetabolicSubmodelComponent() override = default;
00033
00034 public:
00035     static PluginInformation* GetPluginInformation();
00036     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00037     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00038
00039 public:
00040     void setWholeCellState(WholeCellState* state);
00041     WholeCellState* getWholeCellState() const;
00042     void setAtpYieldRate(double rate);
00043     double getAtpYieldRate() const;
00044     void setConsumptionFraction(double frac);
00045     double getConsumptionFraction() const;
00046     void setAtpPoolMax(double maxPool);
00047     double getAtpPoolMax() const;
00048     void setDeltaT(double dt);
00049     double getDeltaT() const;
00050
00051     virtual std::string show() override;
00052
00053 protected:
00054     virtual bool _loadInstance(PersistenceRecord* fields) override;
00055     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00056     virtual bool _check(std::string& errorMessage) override;
00057     virtual void _initBetweenReplications() override;
00058     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00059
00060 protected:
00061     virtual void _createEditableDataDefinitions() override;
00062
00063 private:
00064     void _forwardEntity(Entity* entity);
00065
00066 private:
00067     const struct DEFAULT_VALUES {
00068         std::string wholeCellStateName = "";
00069         // mmol ATP per fL per second -- calibrated for M. genitalium basal metabolism
00070         double atpYieldRate = 1.0e-3;
00071         double consumptionFraction = 0.1; // fraction of ATP pool consumed per step
00072         double atpPoolMax = 10.0; // mmol -- approximate saturation
00073         double deltaT = 60.0; // s
00074     } DEFAULT;
00075
00076     WholeCellState* _wholeCellState = nullptr;
00077     double _atpYieldRate = DEFAULT.atpYieldRate;
00078     double _consumptionFraction = DEFAULT.consumptionFraction;
00079     double _atpPoolMax = DEFAULT.atpPoolMax;
00080     double _deltaT = DEFAULT.deltaT;
00081 };
00082
00083 #endif /* METABOLICSUBMODELCOMPONENT_H */

```

11.404 PathwayStressResponseComponent.h

```

00001 #ifndef PATHWAYSTRESSRESPONSECOMPONENT_H
00002 #define PATHWAYSTRESSRESPONSECOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00009
00026 class PathwayStressResponseComponent : public ModelComponent {
00027 public:
00028     PathwayStressResponseComponent(Model* model, std::string name = "");
00029     virtual ~PathwayStressResponseComponent() override = default;
00030
00031 public:
00032     static PluginInformation* GetPluginInformation();
00033     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00034     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00035
00036 public:
00037     void setWholeCellState(WholeCellState* state);
00038     WholeCellState* getWholeCellState() const;

```

```

00039     void setMonitoredPathwayKey(std::string key);
00040     std::string getMonitoredPathwayKey() const;
00041     void setStressThreshold(double threshold);
00042     double getStressThreshold() const;
00043     void setArrestAfterSteps(unsigned int steps);
00044     unsigned int getArrestAfterSteps() const;
00045     void setDeathAfterSteps(unsigned int steps);
00046     unsigned int getDeathAfterSteps() const;
00047     void setArrestPhase(std::string phase);
00048     std::string getArrestPhase() const;
00049     void setDeadPhase(std::string phase);
00050     std::string getDeadPhase() const;
00051     void setRecoveryPhase(std::string phase);
00052     std::string getRecoveryPhase() const;
00053     void setResetStreakOnRecovery(bool reset);
00054     bool getResetStreakOnRecovery() const;
00055     unsigned int getStressStreak() const;
00056     double getLastObservedActivity() const;
00057
00058     virtual std::string show() override;
00059
00060 protected:
00061     virtual bool _loadInstance(PersistenceRecord* fields) override;
00062     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00063     virtual bool _check(std::string& errorMessage) override;
00064     virtual void _initBetweenReplications() override;
00065     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00066     virtual void _createEditableDataDefinitions() override;
00067
00068 private:
00069     void _forwardEntity(Entity* entity);
00070
00071 private:
00072     const struct DEFAULT_VALUES {
00073         std::string wholeCellStateName = "";
00074         std::string monitoredPathwayKey = "biomass_objective";
00075         double stressThreshold = 0.0;
00076         unsigned int arrestAfterSteps = 0u;
00077         unsigned int deathAfterSteps = 0u;
00078         std::string arrestPhase = "arrested";
00079         std::string deadPhase = "dead";
00080         std::string recoveryPhase = "growth";
00081         bool resetStreakOnRecovery = true;
00082         unsigned int stressStreak = 0u;
00083         double lastObservedActivity = 0.0;
00084     } DEFAULT;
00085
00086     WholeCellState* _wholeCellState = nullptr;
00087     std::string _monitoredPathwayKey = DEFAULT.monitoredPathwayKey;
00088     double _stressThreshold = DEFAULT.stressThreshold;
00089     unsigned int _arrestAfterSteps = DEFAULT.arrestAfterSteps;
00090     unsigned int _deathAfterSteps = DEFAULT.deathAfterSteps;
00091     std::string _arrestPhase = DEFAULT.arrestPhase;
00092     std::string _deadPhase = DEFAULT.deadPhase;
00093     std::string _recoveryPhase = DEFAULT.recoveryPhase;
00094     bool _resetStreakOnRecovery = DEFAULT.resetStreakOnRecovery;
00095     unsigned int _stressStreak = DEFAULT.stressStreak;
00096     double _lastObservedActivity = DEFAULT.lastObservedActivity;
00097 };
00098
00099 #endif /* PATHWAYSTRESSRESPONSECOMPONENT_H */

```

11.405 ResourceAllocationComponent.h

```

00001 #ifndef RESOURCEALLOCATIONCOMPONENT_H
00002 #define RESOURCEALLOCATIONCOMPONENT_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelComponent.h"
00007 #include "kernel/simulator/Plugin.h"
00008 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00009
00031 class ResourceAllocationComponent : public ModelComponent {
00032 public:
00033     ResourceAllocationComponent(Model* model, std::string name = "");
00034     virtual ~ResourceAllocationComponent() override = default;
00035
00036 public:
00037     static PluginInformation* GetPluginInformation();
00038     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00039     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00040

```

```

00041 public:
00042     void setWholeCellState(WholeCellState* state);
00043     WholeCellState* getWholeCellState() const;
00044     void setRnapCountKey(std::string key);
00045     std::string getRnapCountKey() const;
00046     void setRibosomeCountKey(std::string key);
00047     std::string getRibosomeCountKey() const;
00048     void setMRNASpeciesPrefix(std::string prefix);
00049     std::string getMRNASpeciesPrefix() const;
00050     void setProteinSpeciesPrefix(std::string prefix);
00051     std::string getProteinSpeciesPrefix() const;
00052
00053     virtual std::string show() override;
00054
00055 protected:
00056     virtual bool _loadInstance(PersistenceRecord* fields) override;
00057     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00058     virtual bool _check(std::string& errorMessage) override;
00059     virtual void _initBetweenReplications() override;
00060     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00061
00062 protected:
00063     virtual void _createEditableDataDefinitions() override;
00064
00065 private:
00066     void _forwardEntity(Entity* entity);
00067
00068 private:
00069     const struct DEFAULT_VALUES {
00070         std::string wholeCellStateName = "";
00071         std::string rnapCountKey       = "RNAP_free";
00072         std::string ribosomeCountKey   = "ribosome_free";
00073         std::string mrNASpeciesPrefix  = "mRNA_";
00074         std::string proteinSpeciesPrefix = "protein_";
00075     } DEFAULT;
00076
00077     WholeCellState* _wholeCellState = nullptr;
00078     std::string _rnapCountKey       = DEFAULT.rnapCountKey;
00079     std::string _ribosomeCountKey   = DEFAULT.ribosomeCountKey;
00080     std::string _mrNASpeciesPrefix  = DEFAULT.mrNASpeciesPrefix;
00081     std::string _proteinSpeciesPrefix = DEFAULT.proteinSpeciesPrefix;
00082 };
00083
00084 #endif /* RESOURCEALLOCATIONCOMPONENT_H */

```

11.406 StochasticReactionComponent.h

```

00001 #ifndef STOCHASTICREACTIONCOMPONENT_H
00002 #define STOCHASTICREACTIONCOMPONENT_H
00003
00004 #include <map>
00005 #include <random>
00006 #include <string>
00007 #include <vector>
00008
00009 #include "../kernel/simulator/model/ModelComponent.h"
00010 #include "kernel/simulator/Plugin.h"
00011 #include "plugins/data/WholeCellModeling/StochasticReactionRule.h"
00012 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00013
00032 class StochasticReactionComponent : public ModelComponent {
00033 public:
00034     StochasticReactionComponent(Model* model, std::string name = "");
00035     virtual ~StochasticReactionComponent() override = default;
00036
00037 public:
00038     static PluginInformation* GetPluginInformation();
00039     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00040     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00041
00042 public:
00043     void setWholeCellState(WholeCellState* state);
00044     WholeCellState* getWholeCellState() const;
00045     void setTimeWindow(double timeWindow);
00046     double getTimeWindow() const;
00047     void setAdvanceWholeCellClock(bool advance);
00048     bool getAdvanceWholeCellClock() const;
00049     void setRandomSeed(unsigned int seed);
00050     unsigned int getRandomSeed() const;
00051     int getLastReactionCount() const;
00052     double getLastSimulatedTime() const;
00053
00054     virtual std::string show() override;

```

```

00055
00056 protected:
00057     virtual bool _loadInstance(PersistenceRecord* fields) override;
00058     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00059     virtual bool _check(std::string& errorMessage) override;
00060     virtual void _initBetweenReplications() override;
00061     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00062
00063 protected:
00064     virtual void _createEditableDataDefinitions() override;
00065
00066 private:
00067     struct GillespieState {
00068         std::map<std::string, int> counts;
00069         double simulatedTime = 0.0;
00070         int reactionCount = 0;
00071     };
00072
00073     GillespieState _runGillespie(
00074         const std::map<std::string, int>& initialCounts,
00075         const std::vector<StochasticReactionRule*>& rules,
00076         const std::map<std::string, int>& resourceBudget,
00077         double timeWindow);
00078
00079     void _collectCountsFromState(std::map<std::string, int>& counts) const;
00080     void _applyCountsToState(const std::map<std::string, int>& counts);
00081     std::vector<StochasticReactionRule*> _collectRules() const;
00082
00083     void _forwardEntity(Entity* entity);
00084
00085 private:
00086     const struct DEFAULT_VALUES {
00087         std::string wholeCellStateName = "";
00088         double timeWindow = 1.0;
00089         bool advanceWholeCellClock = true;
00090         unsigned int randomSeed = 42u;
00091     } DEFAULT;
00092
00093     WholeCellState* _wholeCellState = nullptr;
00094     double _timeWindow = DEFAULT.timeWindow;
00095     bool _advanceWholeCellClock = DEFAULT.advanceWholeCellClock;
00096     unsigned int _randomSeed = DEFAULT.randomSeed;
00097     int _lastReactionCount = 0;
00098     double _lastSimulatedTime = 0.0;
00099
00100     std::mt19937 _rng{DEFAULT.randomSeed};
00101 };
00102
00103 #endif /* STOCHASTICREACTIONCOMPONENT_H */

```

11.407 StochasticTranscription.h

```

00001 #ifndef STOCHASTICTRANSCRIPTION_H
00002 #define STOCHASTICTRANSCRIPTION_H
00003
00004 #include <random>
00005 #include <string>
00006
00007 #include "../kernel/simulator/model/ModelComponent.h"
00008 #include "kernel/simulator/Plugin.h"
00009 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00010
00031 class StochasticTranscription : public ModelComponent {
00032 public:
00033     StochasticTranscription(Model* model, std::string name = "");
00034     virtual ~StochasticTranscription() override = default;
00035
00036 public:
00037     static PluginInformation* GetPluginInformation();
00038     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00039     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00040
00041 public:
00042     void setWholeCellState(WholeCellState* state);
00043     WholeCellState* getWholeCellState() const;
00044     void setElongationRate(double rate);
00045     double getElongationRate() const;
00046     void setMeanGeneLength(double length);
00047     double getMeanGeneLength() const;
00048     void setBindingProbability(double prob);
00049     double getBindingProbability() const;
00050     void setTimeWindow(double window);
00051     double getTimeWindow() const;

```

```

00052     void setMRNASpeciesPrefix(std::string prefix);
00053     std::string getMRNASpeciesPrefix() const;
00054     void setRnapCountKey(std::string key);
00055     std::string getRnapCountKey() const;
00056     void setRandomSeed(unsigned int seed);
00057     unsigned int getRandomSeed() const;
00058     int getLastSynthesizedCount() const;
00059
00060     virtual std::string show() override;
00061
00062 protected:
00063     virtual bool _loadInstance(PersistenceRecord* fields) override;
00064     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00065     virtual bool _check(std::string& errorMessage) override;
00066     virtual void _initBetweenReplications() override;
00067     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00068
00069 protected:
00070     virtual void _createEditableDataDefinitions() override;
00071
00072 private:
00073     void _forwardEntity(Entity* entity);
00074
00075 private:
00076     const struct DEFAULT_VALUES {
00077         std::string wholeCellStateName = "";
00078         double elongationRate = 50.0; // nt/s -- M. genitalium RNA pol (parameters.json)
00079         double meanGeneLength = 900.0; // nt -- approximate M. genitalium average
00080         double bindingProbability = 1.0; // dimensionless; scale to calibrate output
00081         double timeWindow = 1.0; // s
00082         std::string mRNASpeciesPrefix = "mRNA_";
00083         std::string rnapCountKey = "RNAP_free";
00084         unsigned int randomSeed = 42u;
00085     } DEFAULT;
00086
00087     WholeCellState* _wholeCellState = nullptr;
00088     double _elongationRate = DEFAULT.elongationRate;
00089     double _meanGeneLength = DEFAULT.meanGeneLength;
00090     double _bindingProbability = DEFAULT.bindingProbability;
00091     double _timeWindow = DEFAULT.timeWindow;
00092     std::string _mRNASpeciesPrefix = DEFAULT.mRNASpeciesPrefix;
00093     std::string _rnapCountKey = DEFAULT.rnapCountKey;
00094     unsigned int _randomSeed = DEFAULT.randomSeed;
00095     int _lastSynthesizedCount = 0;
00096     std::mt19937 _rng(DEFAULT.randomSeed);
00097 };
00098
00099 #endif /* STOCHASTICTRANSCRIPTION_H */

```

11.408 StochasticTranslation.h

```

00001 #ifndef STOCHASTICTRANSLATION_H
00002 #define STOCHASTICTRANSLATION_H
00003
00004 #include <random>
00005 #include <string>
00006
00007 #include "../kernel/simulator/model/ModelComponent.h"
00008 #include "kernel/simulator/Plugin.h"
00009 #include "plugins/data/WholeCellModeling/WholeCellState.h"
00010
00011 class StochasticTranslation : public ModelComponent {
00012 public:
00013     StochasticTranslation(Model* model, std::string name = "");
00014     virtual ~StochasticTranslation() override = default;
00015
00016 public:
00017     static PluginInformation* GetPluginInformation();
00018     static ModelComponent* LoadInstance(Model* model, PersistenceRecord* fields);
00019     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00020
00021 public:
00022     void setWholeCellState(WholeCellState* state);
00023     WholeCellState* getWholeCellState() const;
00024     void setElongationRate(double rate);
00025     double getElongationRate() const;
00026     void setMeanProteinLength(double length);
00027     double getMeanProteinLength() const;
00028     void setTimeWindow(double window);
00029     double getTimeWindow() const;
00030     void setMRNASpeciesPrefix(std::string prefix);
00031     std::string getMRNASpeciesPrefix() const;
00032     void setProteinSpeciesPrefix(std::string prefix);

```

```

00051     std::string getProteinSpeciesPrefix() const;
00052     void setRibosomeCountKey(std::string key);
00053     std::string getRibosomeCountKey() const;
00054     void setRandomSeed(unsigned int seed);
00055     unsigned int getRandomSeed() const;
00056     int getLastSynthesizedCount() const;
00057
00058     virtual std::string show() override;
00059
00060 protected:
00061     virtual bool _loadInstance(PersistenceRecord* fields) override;
00062     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00063     virtual bool _check(std::string& errorMessage) override;
00064     virtual void _initBetweenReplications() override;
00065     virtual void _onDispatchEvent(Entity* entity, unsigned int inputPortNumber) override;
00066
00067 protected:
00068     virtual void _createEditableDataDefinitions() override;
00069
00070 private:
00071     void _forwardEntity(Entity* entity);
00072
00073 private:
00074     const struct DEFAULT_VALUES {
00075         std::string wholeCellStateName = "";
00076         double elongationRate = 16.0; // AA/s -- M. genitalium ribosome (parameters.json)
00077         double meanProteinLength = 300.0; // AA -- approximate M. genitalium average
00078         double timeWindow = 1.0; // s
00079         std::string mRNASpeciesPrefix = "mRNA_";
00080         std::string proteinSpeciesPrefix = "prot_";
00081         std::string ribosomeCountKey = "ribosome_free";
00082         unsigned int randomSeed = 43u;
00083     } DEFAULT;
00084
00085     WholeCellState* _wholeCellState = nullptr;
00086     double _elongationRate = DEFAULT.elongationRate;
00087     double _meanProteinLength = DEFAULT.meanProteinLength;
00088     double _timeWindow = DEFAULT.timeWindow;
00089     std::string _mRNASpeciesPrefix = DEFAULT.mRNASpeciesPrefix;
00090     std::string _proteinSpeciesPrefix = DEFAULT.proteinSpeciesPrefix;
00091     std::string _ribosomeCountKey = DEFAULT.ribosomeCountKey;
00092     unsigned int _randomSeed = DEFAULT.randomSeed;
00093     int _lastSynthesizedCount = 0;
00094     std::mt19937 _rng(DEFAULT.randomSeed);
00095 };
00096
00097 #endif /* STOCHASTICTRANSLATION_H */

```

11.409 AISupport.h

```

00001 #ifndef AISUPPORT_H
00002 #define AISUPPORT_H
00003
00004 #include "kernel/simulator/model/ModelDataDefinition.h"
00005 #include "kernel/simulator/PluginInformation.h"
00006 #include "tools/AIAssistant/AIConversationService.h"
00007
00008 class AISupport : public ModelDataDefinition {
00009 public:
00010     AISupport(Model* model, std::string name = "");
00011     virtual ~AISupport() override = default;
00012
00013     static PluginInformation* GetPluginInformation();
00014     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00015     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00016
00017     AIConversationResponse converse(const std::string& conversationKey, const std::string& prompt);
00018     void clearConversation(const std::string& conversationKey);
00019     void clearAllConversations();
00020
00021     void setProvider(std::string value);
00022     std::string getProvider() const;
00023     void setModel(std::string value);
00024     std::string getModel() const;
00025     void setBaseUrl(std::string value);
00026     std::string getBaseUrl() const;
00027     void setApiKeyEnvironmentVariable(std::string value);
00028     std::string getApiKeyEnvironmentVariable() const;
00029     void setSystemInstructions(std::string value);
00030     std::string getSystemInstructions() const;
00031     void setTemperature(double value);
00032     double getTemperature() const;
00033     void setMaxOutputTokens(unsigned int value);

```

```

00034     unsigned int getMaxOutputTokens() const;
00035     void setTimeoutSeconds(unsigned int value);
00036     unsigned int getTimeoutSeconds() const;
00037     void setHighReasoningMode(bool value);
00038     bool isHighReasoningMode() const;
00039     void setConversationEnabled(bool value);
00040     bool isConversationEnabled() const;
00041     void setMaxHistoryMessages(unsigned int value);
00042     unsigned int getMaxHistoryMessages() const;
00043     void setClearHistoryBetweenReplications(bool value);
00044     bool isClearHistoryBetweenReplications() const;
00045     unsigned long getCallCount() const;
00046     unsigned long getFailureCount() const;
00047
00048     virtual std::string show() override;
00049
00050 protected:
00051     virtual bool _loadInstance(PersistenceRecord* fields) override;
00052     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00053     virtual bool _check(std::string& errorMessage) override;
00054     virtual void _initBetweenReplications() override;
00055
00056 private:
00057     void _configurationChanged();
00058     AIConversationConfiguration _makeConfiguration() const;
00059
00060     const struct DEFAULT_VALUES {
00061         std::string provider = "Local";
00062         std::string model = "llama3.2";
00063         std::string baseUrl = "";
00064         std::string apiKeyEnvironmentVariable = "";
00065         std::string systemInstructions = "You are a decision service inside a GenESyS simulation.
00066         Return only valid JSON.";
00067         double temperature = 0.2;
00068         unsigned int maxOutputTokens = 512;
00069         unsigned int timeoutSeconds = 120;
00070         bool highReasoningMode = false;
00071         bool conversationEnabled = true;
00072         unsigned int maxHistoryMessages = 20;
00073         bool clearHistoryBetweenReplications = true;
00074     } DEFAULT;
00075
00076     std::string _provider = DEFAULT.provider;
00077     std::string _model = DEFAULT.model;
00078     std::string _baseUrl = DEFAULT.baseUrl;
00079     std::string _apiKeyEnvironmentVariable = DEFAULT.apiKeyEnvironmentVariable;
00080     std::string _systemInstructions = DEFAULT.systemInstructions;
00081     double _temperature = DEFAULT.temperature;
00082     unsigned int _maxOutputTokens = DEFAULT.maxOutputTokens;
00083     unsigned int _timeoutSeconds = DEFAULT.timeoutSeconds;
00084     bool _highReasoningMode = DEFAULT.highReasoningMode;
00085     bool _conversationEnabled = DEFAULT.conversationEnabled;
00086     unsigned int _maxHistoryMessages = DEFAULT.maxHistoryMessages;
00087     bool _clearHistoryBetweenReplications = DEFAULT.clearHistoryBetweenReplications;
00088     unsigned long _callCount = 0;
00089     unsigned long _failureCount = 0;
00090     AIConversationService _service;
00091 };
00092 #endif

```

11.410 BacteriaSignalGrid.h

```

00001 /*
00002  * File:    BacteriaSignalGrid.h
00003  * Author:  rlcancian
00004  *
00005  * Created on 28 de Abril de 2022
00006  */
00007
00008 #ifndef BACTERIASIGNALGRID_H
00009 #define BACTERIASIGNALGRID_H
00010
00011 #include "../kernel/simulator/model/ModelDataDefinition.h"
00012 #include "kernel/simulator/Plugin.h"
00013
00014 #include <string>
00015 #include <vector>
00016
00024 class BacteriaSignalGrid : public ModelDataDefinition {
00025 public:
00026     BacteriaSignalGrid(Model* model, std::string name = "");
00027     virtual ~BacteriaSignalGrid() override = default;

```

```

00028
00029 public:
00030     virtual std::string show() override;
00031
00032 public: // static plugin interface
00033     static PluginInformation* GetPluginInformation();
00034     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00035     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00036
00037 public:
00038     void setWidth(unsigned int width);
00039     unsigned int getWidth() const;
00040     void setHeight(unsigned int height);
00041     unsigned int getHeight() const;
00042     void setInitialSignal(double initialSignal);
00043     double getInitialSignal() const;
00044     void setDiffusionRate(double diffusionRate);
00045     double getDiffusionRate() const;
00046     void setDecayRate(double decayRate);
00047     double getDecayRate() const;
00048     void setInitialValues(std::string initialValues);
00049     std::string getInitialValues() const;
00050     unsigned int getCellCount() const;
00051     bool buildInitialField(std::vector<double>& values, std::string& errorMessage) const;
00052
00053 protected:
00054     virtual bool _loadInstance(PersistenceRecord* fields) override;
00055     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00056     virtual bool _check(std::string& errorMessage) override;
00057
00058 protected:
00059     // virtual void _createInternalStatisticReporters() override;
00060     // virtual void _createNonEditableDataDefinitions() override;
00061     // virtual void _createEditableDataDefinitions() override;
00062     // virtual void _createAttachedAttributes() override;
00063
00064 private:
00065     const struct DEFAULT_VALUES {
00066         const unsigned int width = 32;
00067         const unsigned int height = 32;
00068         const double initialSignal = 0.0;
00069         const double diffusionRate = 0.18;
00070         const double decayRate = 0.02;
00071         const std::string initialValues = "";
00072     } DEFAULT;
00073
00074     unsigned int _width = DEFAULT.width;
00075     unsigned int _height = DEFAULT.height;
00076     double _initialSignal = DEFAULT.initialSignal;
00077     double _diffusionRate = DEFAULT.diffusionRate;
00078     double _decayRate = DEFAULT.decayRate;
00079     std::string _initialValues = DEFAULT.initialValues;
00080 };
00081
00082 #endif /* BACTERIASIGNALGRID_H */

```

11.411 BioNetwork.h

```

00001 #ifndef BIONETWORK_H
00002 #define BIONETWORK_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelDataDefinition.h"
00008 #include "kernel/simulator/PluginInformation.h"
00009 #include "tools/Biochemical/BioSimulationAnalysis.h"
00010 #include "tools/Biochemical/BioSimulationResult.h"
00011
00012 class BioReaction;
00013 class BioSpecies;
00014 class MassActionOdeSystem;
00015
00025 class BioNetwork : public ModelDataDefinition {
00026 public:
00027     BioNetwork(Model* model, std::string name = "");
00028     virtual ~BioNetwork() override = default;
00029
00030 public:
00031     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00032     static PluginInformation* GetPluginInformation();
00033     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00034

```



```

00035 public:
00036     void setStartTime(double startTime);
00037     double getStartTime() const;
00038     void setStopTime(double stopTime);
00039     double getStopTime() const;
00040     void setStepSize(double stepSize);
00041     double getStepSize() const;
00042     void setCurrentTime(double currentTime);
00043     double getCurrentTime() const;
00044     void setAutoSchedule(bool autoSchedule);
00045     bool getAutoSchedule() const;
00046     void setLastStatus(std::string lastStatus);
00047     std::string getLastStatus() const;
00048     void setLastErrorMessage(std::string lastErrorMessage);
00049     std::string getLastErrorMessage() const;
00050     void setLastResponsePayload(std::string lastResponsePayload);
00051     std::string getLastResponsePayload() const;
00052     void addSpecies(std::string speciesName);
00053     void addReaction(std::string reactionName);
00054     void clearSpecies();
00055     void clearReactions();
00056     const std::vector<std::string>& getSpeciesNames() const;
00057     const std::vector<std::string>& getReactionNames() const;
00058     const BioSimulationResult& getLastSimulationResult() const;
00059     bool getSpeciesTimeCourseDataset(const std::string& speciesName, SimulationResultsDataset*
dataset,
00060                                     std::string* errorMessage = nullptr) const;
00061     bool buildOdeSystemForAnalysis(MassActionOdeSystem* system, std::string& errorMessage) const;
00062     bool getStoichiometryMatrix(BioStoichiometryMatrix* matrix, std::string* errorMessage = nullptr)
const;
00063     bool getReactionRateTimeCourse(BioReactionRateTimeCourse* timeCourse, std::string* errorMessage =
nullptr) const;
00064     bool checkLastSampleSteadyState(double tolerance, BioSteadyStateCheck* check, std::string*
errorMessage = nullptr) const;
00065     bool scanLocalParameterSensitivity(double relativeStep, double absoluteStep, BioSensitivityScan*
scan,
00066                                     std::string* errorMessage = nullptr) const;
00067
00068     bool simulate(std::string& errorMessage);
00069     bool simulate(double startTime, double stopTime, double stepSize, std::string& errorMessage);
00070     bool advanceOneStep(std::string& errorMessage);
00071
00072     virtual std::string show() override;
00073
00074 protected:
00075     virtual bool _loadInstance(PersistenceRecord *fields) override;
00076     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00077     virtual bool _check(std::string& errorMessage) override;
00078     virtual void _initBetweenReplications() override;
00079
00080
00081 protected:
00082     // virtual void _createInternalStatisticReporters() override;
00083     // virtual void _createNonEditableDataDefinitions() override;
00091     virtual void _createEditableDataDefinitions() override;
00092     // virtual void _createAttachedAttributes() override;
00093
00094 private:
00095     void handleInternalEvent(void* parameter);
00096     void scheduleNextInternalEvent();
00097     bool collectSpecies(std::vector<BioSpecies*>& species, std::string& errorMessage) const;
00098     bool collectReactions(std::vector<BioReaction*>& reactions, std::string& errorMessage) const;
00099     bool buildSystem(const std::vector<BioSpecies*>& species, const std::vector<BioReaction*>&
reactions, MassActionOdeSystem* system, std::string& errorMessage) const;
00100     void updatePayload(const std::vector<BioSpecies*>& species);
00101     void configureSimulationResult(const std::vector<BioSpecies*>& species);
00102     void appendSimulationResultSample(const std::vector<BioSpecies*>& species);
00103
00104 private:
00105     const struct DEFAULT_VALUES {
00106         double startTime = 0.0;
00107         double stopTime = 1.0;
00108         double stepSize = 0.1;
00109         double currentTime = 0.0;
00110         bool autoSchedule = false;
00111         std::string lastStatus = "Idle";
00112         std::string lastErrorMessage = "";
00113         std::string lastResponsePayload = "";
00114     } DEFAULT;
00115
00116     double _startTime = DEFAULT.startTime;
00117     double _stopTime = DEFAULT.stopTime;
00118     double _stepSize = DEFAULT.stepSize;
00119     double _currentTime = DEFAULT.currentTime;
00120     bool _autoSchedule = DEFAULT.autoSchedule;
00121     std::string _lastStatus = DEFAULT.lastStatus;
00122     std::string _lastErrorMessage = DEFAULT.lastErrorMessage;

```

```

00123     std::string _lastResponsePayload = DEFAULT.lastResponsePayload;
00124     std::vector<std::string> _speciesNames;
00125     std::vector<std::string> _reactionNames;
00126     BioSimulationResult _lastSimulationResult;
00127 };
00128
00129 #endif /* BIONETWORK_H */

```

11.412 BioParameter.h

```

00001 #ifndef BIOPARAMETER_H
00002 #define BIOPARAMETER_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelDataDefinition.h"
00007 #include "kernel/simulator/PluginInformation.h"
00008
00016 class BioParameter : public ModelDataDefinition {
00017 public:
00018     BioParameter(Model* model, std::string name = "");
00019     virtual ~BioParameter() override = default;
00020
00021 public:
00022     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00023     static PluginInformation* GetPluginInformation();
00024     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00025
00026 public:
00027     void setValue(double value);
00028     double getValue() const;
00029     void setUnit(std::string unit);
00030     std::string getUnit() const;
00031
00032     virtual std::string show() override;
00033
00034 protected:
00035     virtual bool _loadInstance(PersistenceRecord *fields) override;
00036     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00037     virtual bool _check(std::string& errorMessage) override;
00038
00039 protected:
00040     // virtual void _createInternalStatisticReporters() override;
00041     // virtual void _createNonEditableDataDefinitions() override;
00042     // virtual void _createEditableDataDefinitions() override;
00043     // virtual void _createAttachedAttributes() override;
00044
00045 private:
00046     const struct DEFAULT_VALUES {
00047         double value = 0.0;
00048         std::string unit = "";
00049     } DEFAULT;
00050
00051     double _value = DEFAULT.value;
00052     std::string _unit = DEFAULT.unit;
00053 };
00054
00055 #endif /* BIOPARAMETER_H */

```

11.413 BioReaction.h

```

00001 #ifndef BIOREACTION_H
00002 #define BIOREACTION_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelDataDefinition.h"
00008 #include "kernel/simulator/PluginInformation.h"
00009
00023 class BioReaction : public ModelDataDefinition {
00024 public:
00025     struct StoichiometricTerm {
00026         std::string speciesName;
00027         double stoichiometry = 1.0;
00028     };
00029
00030 public:

```

```

00036     BioReaction(Model* model, std::string name = "");
00040     virtual ~BioReaction() override = default;
00041
00042 public:
00049     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00054     static PluginInformation* GetPluginInformation();
00061     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00062
00063 public:
00069     void addReactant(std::string speciesName, double stoichiometry = 1.0);
00075     void addProduct(std::string speciesName, double stoichiometry = 1.0);
00080     void addModifier(std::string speciesName);
00084     void clearReactants();
00088     void clearProducts();
00092     void clearModifiers();
00097     const std::vector<StoichiometricTerm>& getReactants() const;
00102     const std::vector<StoichiometricTerm>& getProducts() const;
00107     const std::vector<std::string>& getModifiers() const;
00112     void setRateConstant(double rateConstant);
00117     double getRateConstant() const;
00122     void setRateConstantParameterName(std::string rateConstantParameterName);
00127     std::string getRateConstantParameterName() const;
00132     double resolveRateConstant() const;
00137     void setReverseRateConstant(double reverseRateConstant);
00142     double getReverseRateConstant() const;
00147     void setReverseRateConstantParameterName(std::string reverseRateConstantParameterName);
00152     std::string getReverseRateConstantParameterName() const;
00157     double resolveReverseRateConstant() const;
00162     void setKineticLawExpression(std::string kineticLawExpression);
00167     std::string getKineticLawExpression() const;
00172     void setReverseKineticLawExpression(std::string reverseKineticLawExpression);
00177     std::string getReverseKineticLawExpression() const;
00182     void setReversible(bool reversible);
00187     bool isReversible() const;
00188
00193     virtual std::string show() override;
00194
00195 protected:
00196     virtual bool _loadInstance(PersistenceRecord *fields) override;
00197     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00198     virtual bool _check(std::string& errorMessage) override;
00199     // virtual void _createInternalAndAttachedData() override;
00200
00201
00202 protected:
00203     // virtual void _createInternalStatisticReporters() override;
00204     // virtual void _createNonEditableDataDefinitions() override;
00205     // virtual void _createEditableDataDefinitions() override;
00206     virtual void _createAttachedAttributes() override;
00207
00208 private:
00209     bool checkTerms(const std::vector<StoichiometricTerm>& terms, const std::string& side,
00210 std::string& errorMessage) const;
00210     bool checkModifiers(std::string& errorMessage) const;
00211     bool validateKineticLawExpression(std::string& errorMessage) const;
00212     bool validateKineticLawExpression(const std::string& expression, const std::string& label,
00213 std::string& errorMessage) const;
00213     bool hasParticipantSpecies(const std::string& speciesName) const;
00214     bool resolveKineticLawSymbol(const std::string& symbolName, double& value) const;
00215     void syncParticipantAttachedSpecies();
00216
00217 private:
00218     const struct DEFAULT_VALUES {
00219         double rateConstant = 0.0;
00220         std::string rateConstantParameterName = "";
00221         double reverseRateConstant = 0.0;
00222         std::string reverseRateConstantParameterName = "";
00223         std::string kineticLawExpression = "";
00224         std::string reverseKineticLawExpression = "";
00225         bool reversible = false;
00226     } DEFAULT;
00227
00228     std::vector<StoichiometricTerm> _reactants;
00229     std::vector<StoichiometricTerm> _products;
00230     std::vector<std::string> _modifiers;
00231     double _rateConstant = DEFAULT.rateConstant;
00232     std::string _rateConstantParameterName = DEFAULT.rateConstantParameterName;
00233     double _reverseRateConstant = DEFAULT.reverseRateConstant;
00234     std::string _reverseRateConstantParameterName = DEFAULT.reverseRateConstantParameterName;
00235     std::string _kineticLawExpression = DEFAULT.kineticLawExpression;
00236     std::string _reverseKineticLawExpression = DEFAULT.reverseKineticLawExpression;
00237     bool _reversible = DEFAULT.reversible;
00238 };
00239
00240 #endif /* BIOREACTION_H */

```

11.414 BioSBMLBridge.h

```

00001 #ifndef BIOSBMLBRIDGE_H
00002 #define BIOSBMLBRIDGE_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 class Model;
00008
00009 struct BioSBMLImportResult {
00010     std::string networkName;
00011     unsigned int speciesImported = 0;
00012     unsigned int parametersImported = 0;
00013     unsigned int reactionsImported = 0;
00014     std::vector<std::string> warnings;
00015 };
00016
00017 struct BioSBMLExportResult {
00018     std::string networkName;
00019     unsigned int speciesExported = 0;
00020     unsigned int parametersExported = 0;
00021     unsigned int reactionsExported = 0;
00022     std::vector<std::string> warnings;
00023 };
00024
00025 class BioSBMLBridge {
00026 public:
00027     static bool importFromString(Model* model,
00028                                 const std::string& sbmlText,
00029                                 const std::string& preferredNetworkName,
00030                                 BioSBMLImportResult* result,
00031                                 std::string& errorMessage);
00032
00033     static bool exportToString(Model* model,
00034                                const std::string& requestedNetworkName,
00035                                std::string* sbmlText,
00036                                BioSBMLExportResult* result,
00037                                std::string& errorMessage);
00038 };
00039
00040 #endif /* BIOSBMLBRIDGE_H */

```

11.415 BioSimulatorRunner.h

```

00001 /*
00002  * File: BioSimulatorRunner.h
00003  * Author: rlcancian
00004  *
00005  * Structural biochemical simulator runner data definition.
00006  */
00007
00008 #ifndef BIOSIMULATORRUNNER_H
00009 #define BIOSIMULATORRUNNER_H
00010
00011 #include <string>
00012
00013 #include "../kernel/simulator/model/ModelDataDefinition.h"
00014 #include "kernel/simulator/PluginInformation.h"
00015
00016 class BioSimulatorRunner : public ModelDataDefinition {
00017 public:
00018     BioSimulatorRunner(Model* model, std::string name = "");
00019     virtual ~BioSimulatorRunner() = default;
00020
00021 public:
00022     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00023     static PluginInformation* GetPluginInformation();
00024     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00025
00026 public:
00027     void setBackend(std::string backend);
00028     std::string getBackend() const;
00029     void setModelSourceType(std::string modelSourceType);
00030     std::string getModelSourceType() const;
00031     void setModelSource(std::string modelSource);
00032     std::string getModelSource() const;
00033     void setCommand(std::string command);
00034     std::string getCommand() const;
00035     void setLastStatus(std::string lastStatus);
00036     std::string getLastStatus() const;
00037     void setLastErrorMessage(std::string lastErrorMessage);
00038     std::string getLastErrorMessage() const;

```

```

00039     void setLastResponsePayload(std::string lastResponsePayload);
00040     std::string getLastResponsePayload() const;
00041     void setLastResponseFilename(std::string lastResponseFilename);
00042     std::string getLastResponseFilename() const;
00043     void setTargetBioNetworkName(std::string targetBioNetworkName);
00044     std::string getTargetBioNetworkName() const;
00045     void setWorkingDirectory(std::string workingDirectory);
00046     std::string getWorkingDirectory() const;
00047     void setWorkingInputFilename(std::string workingInputFilename);
00048     std::string getWorkingInputFilename() const;
00049     void setWorkingOutputFilename(std::string workingOutputFilename);
00050     std::string getWorkingOutputFilename() const;
00051     void setEndpointOrLibrary(std::string endpointOrLibrary);
00052     std::string getEndpointOrLibrary() const;
00053     void setTimeoutSeconds(unsigned int timeoutSeconds);
00054     unsigned int getTimeoutSeconds() const;
00055     void setAutoValidateModel(bool autoValidateModel);
00056     bool getAutoValidateModel() const;
00057
00058     bool executeCommand(std::string& errorMessage);
00059
00060     virtual std::string show() override;
00061
00062 protected:
00063     virtual bool _loadInstance(PersistenceRecord *fields) override;
00064     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00065     virtual bool _check(std::string& errorMessage) override;
00066     // virtual void _createInternalAndAttachedData() override;
00067
00068
00069 protected:
00070     // virtual void _createInternalStatisticReporters() override;
00071     // virtual void _createNonEditableDataDefinitions() override;
00072     // virtual void _createEditableDataDefinitions() override;
00073     // virtual void _createAttachedAttributes() override;
00074
00075 private:
00076     const struct DEFAULT_VALUES {
00077         std::string backend = "RoadRunnerEmbedded";
00078         std::string modelSourceType = "SBMLString";
00079         std::string modelSource = "";
00080         std::string command = "";
00081         std::string lastStatus = "Idle";
00082         std::string lastErrorMessage = "";
00083         std::string lastResponsePayload = "";
00084         std::string lastResponseFilename = "";
00085         std::string targetBioNetworkName = "";
00086         std::string workingDirectory = "";
00087         std::string workingInputFilename = "biosim_input.xml";
00088         std::string workingOutputFilename = "biosim_output.json";
00089         std::string endpointOrLibrary = "";
00090         unsigned int timeoutSeconds = 30;
00091         bool autoValidateModel = true;
00092     } DEFAULT;
00093
00094     std::string _backend = DEFAULT.backend;
00095     std::string _modelSourceType = DEFAULT.modelSourceType;
00096     std::string _modelSource = DEFAULT.modelSource;
00097     std::string _command = DEFAULT.command;
00098     std::string _lastStatus = DEFAULT.lastStatus;
00099     std::string _lastErrorMessage = DEFAULT.lastErrorMessage;
00100     std::string _lastResponsePayload = DEFAULT.lastResponsePayload;
00101     std::string _lastResponseFilename = DEFAULT.lastResponseFilename;
00102     std::string _targetBioNetworkName = DEFAULT.targetBioNetworkName;
00103     std::string _workingDirectory = DEFAULT.workingDirectory;
00104     std::string _workingInputFilename = DEFAULT.workingInputFilename;
00105     std::string _workingOutputFilename = DEFAULT.workingOutputFilename;
00106     std::string _endpointOrLibrary = DEFAULT.endpointOrLibrary;
00107     unsigned int _timeoutSeconds = DEFAULT.timeoutSeconds;
00108     bool _autoValidateModel = DEFAULT.autoValidateModel;
00109 };
00110
00111 #endif /* BIOSIMULATORRUNNER_H */

```

11.416 BioSpecies.h

```

00001 #ifndef BIOSPECIES_H
00002 #define BIOSPECIES_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelDataDefinition.h"
00007 #include "kernel/simulator/PluginInformation.h"

```

```

00008
00017 class BioSpecies : public ModelDataDefinition {
00018 public:
00019     BioSpecies(Model* model, std::string name = "");
00020     virtual ~BioSpecies() override = default;
00021
00022 public:
00023     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00024     static PluginInformation* GetPluginInformation();
00025     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00026
00027 public:
00028     void setInitialAmount(double initialAmount);
00029     double getInitialAmount() const;
00030     void setAmount(double amount);
00031     double getAmount() const;
00032     void setConstant(bool constant);
00033     bool isConstant() const;
00034     void setBoundaryCondition(bool boundaryCondition);
00035     bool isBoundaryCondition() const;
00036     void setUnit(std::string unit);
00037     std::string getUnit() const;
00038
00039     virtual std::string show() override;
00040
00041 protected:
00042     virtual bool _loadInstance(PersistenceRecord *fields) override;
00043     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00044     virtual bool _check(std::string& errorMessage) override;
00045     virtual void _initBetweenReplications() override;
00046
00047 protected:
00048     // virtual void _createInternalStatisticReporters() override;
00049     // virtual void _createNonEditableDataDefinitions() override;
00050     // virtual void _createEditableDataDefinitions() override;
00051     // virtual void _createAttachedAttributes() override;
00052
00053 private:
00054     const struct DEFAULT_VALUES {
00055         double initialAmount = 0.0;
00056         double amount = 0.0;
00057         bool constant = false;
00058         bool boundaryCondition = false;
00059         std::string unit = "";
00060     } DEFAULT;
00061
00062     double _initialAmount = DEFAULT.initialAmount;
00063     double _amount = DEFAULT.amount;
00064     bool _constant = DEFAULT.constant;
00065     bool _boundaryCondition = DEFAULT.boundaryCondition;
00066     std::string _unit = DEFAULT.unit;
00067 };
00068
00069
00070 #endif /* BIOSPECIES_H */

```

11.417 GeneticCircuit.h

```

00001 #ifndef GENETICCIRCUIT_H
00002 #define GENETICCIRCUIT_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelDataDefinition.h"
00008 #include "kernel/simulator/PluginInformation.h"
00009
00017 class GeneticCircuit : public ModelDataDefinition {
00018 public:
00019     GeneticCircuit(Model* model, std::string name = "");
00020     virtual ~GeneticCircuit() override = default;
00021
00022 public:
00023     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00024     static PluginInformation* GetPluginInformation();
00025     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00026
00027 public:
00028     void addPart(std::string partName);
00029     void addRegulation(std::string regulationName);
00030     void clearParts();
00031     void clearRegulations();
00032     const std::vector<std::string>& getPartNames() const;

```

```

00033     const std::vector<std::string>& getRegulationNames() const;
00034     void setHostOrganism(std::string hostOrganism);
00035     std::string getHostOrganism() const;
00036     void setCompartment(std::string compartment);
00037     std::string getCompartment() const;
00038     void setEnabled(bool enabled);
00039     bool isEnabled() const;
00040
00041     virtual std::string show() override;
00042
00043 protected:
00044     virtual bool _loadInstance(PersistenceRecord* fields) override;
00045     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00046     virtual bool _check(std::string& errorMessage) override;
00047
00048 protected:
00049     // virtual void _createInternalStatisticReporters() override;
00050     // virtual void _createNonEditableDataDefinitions() override;
00051     // virtual void _createEditableDataDefinitions() override;
00052     // virtual void _createAttachedAttributes() override;
00053
00054 private:
00055     bool _checkPartNames(std::string& errorMessage) const;
00056     bool _checkRegulationNames(std::string& errorMessage) const;
00057
00058 private:
00059     const struct DEFAULT_VALUES {
00060         std::string hostOrganism = "";
00061         std::string compartment = "";
00062         bool enabled = true;
00063     } DEFAULT;
00064
00065     std::vector<std::string> _partNames;
00066     std::vector<std::string> _regulationNames;
00067     std::string _hostOrganism = DEFAULT.hostOrganism;
00068     std::string _compartment = DEFAULT.compartment;
00069     bool _enabled = DEFAULT.enabled;
00070 };
00071
00072 #endif /* GENETICCIRCUIT_H */

```

11.418 GeneticCircuitPart.h

```

00001 #ifndef GENETICCIRCUITPART_H
00002 #define GENETICCIRCUITPART_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelDataDefinition.h"
00007 #include "kernel/simulator/PluginInformation.h"
00008
00016 class GeneticCircuitPart : public ModelDataDefinition {
00017 public:
00018     GeneticCircuitPart(Model* model, std::string name = "");
00019     virtual ~GeneticCircuitPart() override = default;
00020
00021 public:
00022     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00023     static PluginInformation* GetPluginInformation();
00024     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00025
00026 public:
00027     void setPartType(std::string partType);
00028     std::string getPartType() const;
00029     void setSequence(std::string sequence);
00030     std::string getSequence() const;
00031     void setProductSpeciesName(std::string productSpeciesName);
00032     std::string getProductSpeciesName() const;
00033     void setCopyNumber(double copyNumber);
00034     double getCopyNumber() const;
00035     void setBasalExpressionRate(double basalExpressionRate);
00036     double getBasalExpressionRate() const;
00037     void setDegradationRate(double degradationRate);
00038     double getDegradationRate() const;
00039     void setEnabled(bool enabled);
00040     bool isEnabled() const;
00041
00042     virtual std::string show() override;
00043
00044 protected:
00045     virtual bool _loadInstance(PersistenceRecord* fields) override;
00046     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00047     virtual bool _check(std::string& errorMessage) override;

```

```

00048
00049 protected:
00050     // virtual void _createInternalStatisticReporters() override;
00051     // virtual void _createNonEditableDataDefinitions() override;
00052     // virtual void _createEditableDataDefinitions() override;
00053     // virtual void _createAttachedAttributes() override;
00054
00055 private:
00056     const struct DEFAULT_VALUES {
00057         std::string partType = "Promoter";
00058         std::string sequence = "";
00059         std::string productSpeciesName = "";
00060         double copyNumber = 1.0;
00061         double basalExpressionRate = 0.0;
00062         double degradationRate = 0.0;
00063         bool enabled = true;
00064     } DEFAULT;
00065
00066     std::string _partType = DEFAULT.partType;
00067     std::string _sequence = DEFAULT.sequence;
00068     std::string _productSpeciesName = DEFAULT.productSpeciesName;
00069     double _copyNumber = DEFAULT.copyNumber;
00070     double _basalExpressionRate = DEFAULT.basalExpressionRate;
00071     double _degradationRate = DEFAULT.degradationRate;
00072     bool _enabled = DEFAULT.enabled;
00073 };
00074
00075 #endif /* GENETICCIRCUITPART_H */

```

11.419 GeneticRegulation.h

```

00001 #ifndef GENETICREGULATION_H
00002 #define GENETICREGULATION_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelDataDefinition.h"
00007 #include "kernel/simulator/PluginInformation.h"
00008
00012 class GeneticRegulation : public ModelDataDefinition {
00013 public:
00014     GeneticRegulation(Model* model, std::string name = "");
00015     virtual ~GeneticRegulation() override = default;
00016
00017 public:
00018     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00019     static PluginInformation* GetPluginInformation();
00020     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00021
00022 public:
00023     void setRegulatorSpeciesName(std::string regulatorSpeciesName);
00024     std::string getRegulatorSpeciesName() const;
00025     void setTargetPartName(std::string targetPartName);
00026     std::string getTargetPartName() const;
00027     void setRegulationType(std::string regulationType);
00028     std::string getRegulationType() const;
00029     void setHillCoefficient(double hillCoefficient);
00030     double getHillCoefficient() const;
00031     void setDissociationConstant(double dissociationConstant);
00032     double getDissociationConstant() const;
00033     void setMaxFoldChange(double maxFoldChange);
00034     double getMaxFoldChange() const;
00035     void setLeakiness(double leakiness);
00036     double getLeakiness() const;
00037     void setEnabled(bool enabled);
00038     bool isEnabled() const;
00039
00040     virtual std::string show() override;
00041
00042 protected:
00043     virtual bool _loadInstance(PersistenceRecord* fields) override;
00044     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00045     virtual bool _check(std::string& errorMessage) override;
00046
00047 protected:
00048     // virtual void _createInternalStatisticReporters() override;
00049     // virtual void _createNonEditableDataDefinitions() override;
00050     // virtual void _createEditableDataDefinitions() override;
00051     // virtual void _createAttachedAttributes() override;
00052
00053 private:
00054     const struct DEFAULT_VALUES {
00055         std::string regulatorSpeciesName = "";

```



```

00056         std::string targetPartName = "";
00057         std::string regulationType = "Activation";
00058         double hillCoefficient = 1.0;
00059         double dissociationConstant = 1.0;
00060         double maxFoldChange = 1.0;
00061         double leakiness = 0.0;
00062         bool enabled = true;
00063     } DEFAULT;
00064
00065     std::string _regulatorSpeciesName = DEFAULT.regulatorSpeciesName;
00066     std::string _targetPartName = DEFAULT.targetPartName;
00067     std::string _regulationType = DEFAULT.regulationType;
00068     double _hillCoefficient = DEFAULT.hillCoefficient;
00069     double _dissociationConstant = DEFAULT.dissociationConstant;
00070     double _maxFoldChange = DEFAULT.maxFoldChange;
00071     double _leakiness = DEFAULT.leakiness;
00072     bool _enabled = DEFAULT.enabled;
00073 };
00074
00075 #endif /* GENETICREGULATION_H */

```

11.420 GroProgram.h

```

00001 /*
00002  * File: GroProgram.h
00003  * Author: rlcancian
00004  *
00005  * Created on 17 de Abril de 2022
00006  */
00007
00008 #ifndef GROPROGRAM_H
00009 #define GROPROGRAM_H
00010
00011 #include "../kernel/simulator/model/ModelDataDefinition.h"
00012 #include "kernel/simulator/Plugin.h"
00013 #include <string>
00014
00023 class GroProgram : public ModelDataDefinition {
00024 public:
00025     GroProgram(Model* model, std::string name = "");
00026     virtual ~GroProgram() override = default;
00027
00028 public:
00029     virtual std::string show() override;
00030
00031 public: // static plugin interface
00032     static PluginInformation* GetPluginInformation();
00033     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00034     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00035
00036 public:
00038     void setSourceCode(std::string sourceCode);
00040     void setSourceCodeProperty(SourceCodeString sourceCode);
00042     std::string getSourceCode() const;
00044     SourceCodeString getSourceCodeProperty() const;
00046     bool createDefaultGroProgram(const std::string& filename = "");
00048     bool validateSyntax(std::string& errorMessage) const;
00049
00050 protected:
00051     virtual bool _loadInstance(PersistenceRecord* fields) override;
00052     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00053     virtual bool _check(std::string& errorMessage) override;
00054
00055 protected:
00057     // virtual void _createInternalStatisticReporters() override;
00058     // virtual void _createNonEditableDataDefinitions() override;
00059     // virtual void _createEditableDataDefinitions() override;
00060     // virtual void _createAttachedAttributes() override;
00061
00062 private:
00063     bool _validateSourceCodeSyntax(const std::string& sourceCode, std::string& errorMessage) const;
00064
00065     private:
00066         const struct DEFAULT_VALUES {
00067             const std::string sourceCode = R"(
00068 include gro
00069 set ( "dt", 0.1 );
00070 set ( "population_max", 2000 );
00071 program p() := {
00072     skip();
00073 };
00074 program q() := {

```

```

00075 set ( "ecoli_growth_rate", 0.1 );
00076 set ( "ecoli_division_size_mean", 2.0 );
00077 set ( "ecoli_division_size_variance", 0.2 );
00078 };
00079 program r() := {
00080   selected : { message ( 1, tostring ( volume ) ) }
00081 };
00082 program s() := {
00083   set ( "ecoli_division_size_mean", 1000 ); // essentially turns off gro's division machinery
00084   rate(1) & volume > 3.14 : {
00085     divide()
00086   }
00087 };
00088 ecoli ( [ x := 0, y := 0 ], program q() ); // Try p(), q(), r() or s() here. They are all a bit
different.
00089 //-----
00090 set ( "dt", 0.1 );
00091 set ( "ecoli_growth_rate", 0.1 );
00092
00093 s0 := signal ( 1, 0.2 ); // create two new signals
00094 s1 := signal ( 1, 0.2 );
00095
00096 // These functions provide various growth rates in different states. a and b
00097 // are the intensities of signaling molecules.
00098 fun on a b . 0.1;
00099 fun off a b . 0.0;
00100 fun and a b . if a > 0.1 & b > 0.1 then 0.1 else 0.0 end;
00101
00102 //
00103 // The state program. Parameters are
00104 //
00105 //   this: the state index
00106 //   m_next, d_next: states to go to after division if mother or daughter cell
00107 //   t_next: state to go to after timer is up
00108 //   tf: timer length (use -1 for infinity)
00109 //   gr: growth rate function
00110 //   sigs: rates to emit signals
00111 //
00112 //
00113 program state ( this, m_next, d_next, t_next, tf, gr, sigs ) := {
00114   needs q, t;
00115   active := false;
00116   needs event;
00117
00118   true : { active := ( q = this ); }
00119
00120   !event & active & just_divided & !daughter & q != m_next : { q := m_next, t := 0, event := true }
00121   !event & active & just_divided & daughter & q != d_next : { q := d_next, t := 0, event := true }
00122   !event & active & tf > 0 & t > tf & q != t_next : { q := t_next, t := 0, event := true }
00123
00124   active : {
00125     set ( "ecoli_growth_rate", gr ( get_signal ( 0 ) ) ( get_signal ( 1 ) ) ),
00126     emit_signal ( 0, sigs[0] ),
00127     emit_signal ( 1, sigs[1] )
00128   }
00129 };
00130
00131 //
00132 // State machine program: coordinates states.
00133 //
00134 program sm() := {
00135   q := 0;
00136   t := 0;
00137   event := false;
00138
00139   true : { t := t + dt, event := false }
00140
00141 };
00142
00143 // The morphogenesis state machine example.
00144 //
00145 program p() := sm() + state ( 0, 1, 2, 0, -1, on, { 0, 0 } ) sharing q, t, event
00146 + state ( 1, 1, 3, 5, 60, on, { 50, 0 } ) sharing q, t, event
00147 + state ( 2, 2, 3, 6, 60, on, { 0, 50 } ) sharing q, t, event
00148 + state ( 3, 3, 3, 3, -1, and, { 0, 0 } ) sharing q, t, event
00149 + state ( 5, 5, 5, 7, 40, on, { 0, 0 } ) sharing q, t, event
00150 + state ( 6, 6, 6, 8, 40, on, { 0, 0 } ) sharing q, t, event
00151 + state ( 7, 7, 7, 7, 0, off, { 0, 0 } ) sharing q, t, event
00152 + state ( 8, 8, 8, 8, 0, off, { 0, 0 } ) sharing q, t, event;
00153
00154 // A helper function.
00155 fun test x y . if x = y then 1 else 0 end;
00156
00157
00158
00159
00160

```

```

00161 //
00162 // Makes cells in different states report different colors.
00163 //
00164 program report() := {
00165
00166     gfp := 0; rfp := 0; yfp := 0; cfp := 0;
00167     needs q;
00168
00169     true : {
00170         rfp := 100 * test ( q ) 3;
00171         yfp := 100 * ( test ( q ) 1 + test ( q ) 2 ),
00172         cfp := 100 * ( test ( q ) 5 + test ( q ) 7 );
00173         gfp := 100 * ( test ( q ) 6 + test ( q ) 8 );
00174     }
00175
00176     selected : { message ( 2, toString ( { id, q } ) ) }
00177 };
00178
00179
00180 //
00181 // Hides the state variable in all() -- see below -- so that it is not cut in half when the
00182 // cell divides.
00183 //
00184 program hide() := {
00185
00186     needs gfp;
00187     needs rfp;
00188     needs yfp;
00189     needs cfp;
00190
00191     skip();
00192 };
00193
00194
00195 //
00196 // All the programs stuck together
00197 //
00198 program all() := p() + report() sharing q, gfp, rfp, yfp, cfp + hide() sharing gfp, rfp, yfp, cfp;
00199
00200 //
00201 // The main program makes the simulation loop
00202 //
00203 program main() := {
00204
00205     t := 1;
00206
00207     true : { t := t + dt }
00208
00209     t > 120 : {
00210         reset(),
00211         ecoli ( [], program all() ),
00212         t := 0
00213     }
00214 };
00215
00216
00217 // associate the program with a cell
00218 ecoli ( [], program all() );
00219
00220
00221
00222 );
00223
00224     } DEFAULT;
00225
00226     std::string _sourceCode = DEFAULT.sourceCode;
00227 };
00228
00229 #endif /* GROPROGRAM_H */
00230
00231
00232 /*
00233
00234
00235
00236 //
00237 // gro is protected by the UW OPEN SOURCE LICENSE, which is summarized here.
00238 // Please see the file LICENSE.txt for the complete license.
00239 //
00240 // THE SOFTWARE (AS DEFINED BELOW) AND HARDWARE DESIGNS (AS DEFINED BELOW) IS PROVIDED
00241 // UNDER THE TERMS OF THIS OPEN SOURCE LICENSE ("LICENSE"). THE SOFTWARE IS PROTECTED
00242 // BY COPYRIGHT AND/OR OTHER APPLICABLE LAW. ANY USE OF THIS SOFTWARE OTHER THAN AS
00243 // AUTHORIZED UNDER THIS LICENSE OR COPYRIGHT LAW IS PROHIBITED.
00244 //
00245 // BY EXERCISING ANY RIGHTS TO THE SOFTWARE AND/OR HARDWARE PROVIDED HERE, YOU ACCEPT AND
00246 // AGREE TO BE BOUND BY THE TERMS OF THIS LICENSE. TO THE EXTENT THIS LICENSE MAY BE
00247 // CONSIDERED A CONTRACT, THE UNIVERSITY OF WASHINGTON ("UW") GRANTS YOU THE RIGHTS

```

```

00248 // CONTAINED HERE IN CONSIDERATION OF YOUR ACCEPTANCE OF SUCH TERMS AND CONDITIONS.
00249 //
00250 // TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION
00251 //
00252
00253 include gro
00254
00255 set ( "dt", 0.1 );
00256 set ( "ecoli_growth_rate", 0.1 );
00257
00258 s0 := signal ( 1, 0.2 ); // create two new signals
00259 s1 := signal ( 1, 0.2 );
00260
00261 // These functions provide various growth rates in different states. a and b
00262 // are the intensities of signaling molecules.
00263 fun on a b . 0.1;
00264 fun off a b . 0.0;
00265 fun and a b . if a > 0.1 & b > 0.1 then 0.1 else 0.0 end;
00266
00267 //
00268 // The state program. Parameters are
00269 //
00270 //   this: the state index
00271 //   m_next, d_next: states to go to after division if mother or daughter cell
00272 //   t_next: state to go to after timer is up
00273 //   tf: timer length (use -1 for infinity)
00274 //   gr: growth rate function
00275 //   sigs: rates to emit signals
00276 //
00277 program state ( this, m_next, d_next, t_next, tf, gr, sigs ) := {
00278
00279   needs q, t;
00280   active := false;
00281   needs event;
00282
00283   true : { active := ( q = this ); }
00284
00285   !event & active & just_divided & !daughter & q != m_next : { q := m_next, t := 0, event := true }
00286   !event & active & just_divided & daughter & q != d_next : { q := d_next, t := 0, event := true }
00287   !event & active & tf > 0 & t > tf & q != t_next : { q := t_next, t := 0, event := true }
00288
00289   active : {
00290     set ( "ecoli_growth_rate", gr ( get_signal ( 0 ) ) ( get_signal ( 1 ) ) ),
00291     emit_signal ( 0, sigs[0] ),
00292     emit_signal ( 1, sigs[1] )
00293   }
00294 };
00295
00296
00297 //
00298 // State machine program: coordinates states.
00299 //
00300 program sm() := {
00301
00302   q := 0;
00303   t := 0;
00304   event := false;
00305
00306   true : { t := t + dt, event := false }
00307 };
00308
00309
00310 //
00311 // The morphogenesis state machine example.
00312 //
00313 program p() := sm() + state ( 0, 1, 2, 0, -1, on, { 0, 0 } ) sharing q, t, event
00314 + state ( 1, 1, 3, 5, 60, on, { 50, 0 } ) sharing q, t, event
00315 + state ( 2, 2, 3, 6, 60, on, { 0, 50 } ) sharing q, t, event
00316 + state ( 3, 3, 3, 3, -1, and, { 0, 0 } ) sharing q, t, event
00317 + state ( 5, 5, 5, 7, 40, on, { 0, 0 } ) sharing q, t, event
00318 + state ( 6, 6, 6, 8, 40, on, { 0, 0 } ) sharing q, t, event
00319 + state ( 7, 7, 7, 7, 0, off, { 0, 0 } ) sharing q, t, event
00320 + state ( 8, 8, 8, 8, 0, off, { 0, 0 } ) sharing q, t, event;
00321
00322 // A helper function.
00323 fun test x y . if x = y then 1 else 0 end;
00324
00325 //
00326 // Makes cells in different states report different colors.
00327 //
00328 program report() := {
00329
00330   gfp := 0; rfp := 0; yfp := 0; cfp := 0;
00331   needs q;
00332
00333   true : {
00334     rfp := 100 * test ( q ) 3;

```

```

00335     yfp := 100 * ( test ( q ) 1 + test ( q ) 2 ),
00336     cfp := 100 * ( test ( q ) 5 + test ( q ) 7 );
00337     gfp := 100 * ( test ( q ) 6 + test ( q ) 8 );
00338 }
00339
00340 selected : { message ( 2, toString ( { id, q } ) ) }
00341 };
00342
00343
00344 //
00345 // Hides the state variable in all() -- see below -- so that it is not cut in half when the
00346 // cell divides.
00347 //
00348 program hide() := {
00349
00350     needs gfp;
00351     needs rfp;
00352     needs yfp;
00353     needs cfp;
00354
00355     skip();
00356
00357 };
00358
00359 //
00360 // All the programs stuck together
00361 //
00362 program all() := p() + report() sharing q, gfp, rfp, yfp, cfp + hide() sharing gfp, rfp, yfp, cfp;
00363
00364 //
00365 // The main program makes the simulation loop
00366 //
00367 program main() := {
00368
00369     t := 1;
00370
00371     true : { t := t + dt }
00372
00373     t > 120 : {
00374         reset(),
00375         ecoli ( [], program all() ),
00376         t := 0
00377     }
00378
00379 };
00380
00381 // associate the program with a cell
00382 ecoli ( [], program all() );
00383 */
00384
00385 /*
00386 include gro
00387 set ("dt",0.1);
00388 s := signal (1,1.05);
00389 k0 := 2.5; // base rate of oscillation
00390 kb := 5; // cell-cell feedback strength
00391 tr := 10; // refractory period length
00392 se := 650; // signal emit magnitude
00393 GO := 0;
00394 WAIT := 1;
00395 program oscillator(g0) := {
00396     gfp := 0.5*volume*g0;
00397     p := [mode := GO, t := 0, x := g0];
00398     true : {gfp := 0.5*volume*(g0 + p.x/150)}
00399     // advance the oscillation phase
00400     p.mode = GO & rate (k0) : {p.x := p.x + 0.01*(150-p.x)}
00401     p.mode = GO & rate(kb*get_signal(s)) : {p.x := p.x + 0.01*(150-p.x)}
00402     // phase reset
00403     p.x > 100 : {
00404         p.x := 0,
00405         p.mode := WAIT,
00406         emit_signal(s, se)
00407     }
00408     // refractory period timer
00409     p.mode = WAIT : {p.t := p.t+dt}
00410     p.mode = WAIT & p.t > tr : {
00411         p.mode := GO,
00412         p.t := 0
00413     }
00414 };
00415 ecoli ( [x:=0, y:=0], program oscillator(1));
00416 program main() := {
00417     //n := 0;
00418     //true : {snapshot("movie/oscillator/" <> toString(n) <> ".tif"), n:=n+1}
00419     skip();
00420 };
00421 */

```

11.421 GroProgramAst.h

```

00001 /*
00002  * File:    GroProgramAst.h
00003  * Author:  rlcancian
00004  *
00005  * Created on 17 de Abril de 2022
00006  */
00007
00008 #ifndef GROPROGRAMAST_H
00009 #define GROPROGRAMAST_H
00010
00011 #include <string>
00012 #include <vector>
00013
00021 struct GroProgramAst {
00022     enum class SourceForm {
00023         RawStatements,
00024         ProgramBlock
00025     };
00026
00027     struct Statement {
00028         std::string sourceText = "";
00029     };
00030
00031     struct NamedProgram {
00032         std::string name = "";
00033         std::vector<std::string> parameters;
00034         std::string bodySource = "";
00035         std::vector<Statement> statements;
00036     };
00037
00038     SourceForm sourceForm = SourceForm::RawStatements;
00039     std::string programName = "";
00040     std::vector<std::string> programParameters;
00041     std::string bodySource = "";
00042     std::vector<Statement> statements;
00043     std::vector<NamedProgram> namedPrograms;
00044
00045     bool isProgramBlock() const {
00046         return sourceForm == SourceForm::ProgramBlock;
00047     }
00048
00049     bool hasNamedPrograms() const {
00050         return !namedPrograms.empty();
00051     }
00052 };
00053
00054 #endif /* GROPROGRAMAST_H */

```

11.422 GroProgramCompiler.h

```

00001 /*
00002  * File:    GroProgramCompiler.h
00003  * Author:  rlcancian
00004  *
00005  * Created on 17 de Abril de 2022
00006  */
00007
00008 #ifndef GROPROGRAMCOMPILER_H
00009 #define GROPROGRAMCOMPILER_H
00010
00011 #include "plugins/data/BiochemicalSimulation/GroProgramAst.h"
00012 #include "plugins/data/BiochemicalSimulation/GroProgramIr.h"
00013
00017 class GroProgramCompiler {
00018 public:
00020     GroProgramIr compile(const GroProgramAst& ast) const;
00021 };
00022
00023 #endif /* GROPROGRAMCOMPILER_H */

```

11.423 GroProgramIr.h

```

00001 /*
00002  * File:    GroProgramIr.h
00003  * Author:  rlcancian
00004  *
00005  * Created on 17 de Abril de 2022

```

```

00006  */
00007
00008 #ifndef GROPROGRAMIR_H
00009 #define GROPROGRAMIR_H
00010
00011 #include "plugins/data/BiochemicalSimulation/GroProgramAst.h"
00012
00013 #include <map>
00014 #include <string>
00015 #include <vector>
00016
00024 struct GroProgramIr {
00025     struct Command {
00026         enum class Kind {
00027             RawStatement,
00028             FunctionCall,
00029             Assignment,
00030             IfStatement
00031         };
00032
00033         Kind kind = Kind::RawStatement;
00034         std::string sourceText = "";
00035         std::string functionName = "";
00036         std::vector<std::string> arguments;
00037         std::string assignmentTarget = "";
00038         std::string expressionText = "";
00039         bool assignmentOnlyIfUnset = false;
00040         std::vector<Command> thenCommands;
00041         std::vector<Command> elseCommands;
00042
00043         bool isFunctionCall() const {
00044             return kind == Kind::FunctionCall;
00045         }
00046
00047         bool isAssignment() const {
00048             return kind == Kind::Assignment;
00049         }
00050
00051         bool isIfStatement() const {
00052             return kind == Kind::IfStatement;
00053         }
00054     };
00055
00056     struct NamedProgramDefinition {
00057         std::vector<std::string> parameters;
00058         std::string sourceText = "";
00059         std::vector<Command> commands;
00060     };
00061
00062     GroProgramAst::SourceForm sourceForm = GroProgramAst::SourceForm::RawStatements;
00063     std::string programName = "";
00064     std::vector<Command> commands;
00065     std::map<std::string, NamedProgramDefinition> namedPrograms;
00066
00067     bool isProgramBlock() const {
00068         return sourceForm == GroProgramAst::SourceForm::ProgramBlock;
00069     }
00070
00071     bool hasNamedProgram(const std::string& name) const {
00072         return namedPrograms.find(name) != namedPrograms.end();
00073     }
00074 };
00075
00076 #endif /* GROPROGRAMIR_H */

```

11.424 GroProgramParser.h

```

00001 /*
00002  * File:   GroProgramParser.h
00003  * Author: rlcancian
00004  *
00005  * Created on 17 de Abril de 2022
00006  */
00007
00008 #ifndef GROPROGRAMPARSER_H
00009 #define GROPROGRAMPARSER_H
00010
00011 #include "plugins/data/BiochemicalSimulation/GroProgramAst.h"
00012
00013 #include <string>
00014
00022 class GroProgramParser {
00023 public:

```

```

00024     struct Result {
00025         bool accepted = false;
00026         std::string errorMessage = "";
00027         GroProgramAst ast;
00028     };
00029
00030 public:
00032     Result parse(const std::string& sourceCode) const;
00033 };
00034
00035 #endif /* GROPROGRAMPARSER_H */

```

11.425 GroProgramRuntime.h

```

00001 /*
00002  * File:    GroProgramRuntime.h
00003  * Author:  rlcancian
00004  *
00005  * Created on 17 de Abril de 2022
00006  */
00007
00008 #ifndef GROPROGRAMRUNTIME_H
00009 #define GROPROGRAMRUNTIME_H
00010
00011 #include "plugins/data/BiochemicalSimulation/GroProgramIr.h"
00012
00013 #include <map>
00014 #include <string>
00015 #include <vector>
00016
00020 struct GroProgramRuntimeState {
00021     double colonyTime = 0.0;
00022     double simulationStep = 1.0;
00023     unsigned int populationSize = 1;
00024     unsigned int tickCount = 0;
00025     std::map<std::string, double> contextVariables;
00026     std::map<std::string, double> variables;
00027 };
00028
00037 class GroProgramRuntime {
00038 public:
00039     enum class PopulationMutationType {
00040         Grow,
00041         Divide,
00042         Die,
00043         SetPopulation
00044     };
00045
00046     enum class SignalMutationType {
00047         Emit,
00048         Consume,
00049         Set
00050     };
00051
00052     enum class MotionMutationType {
00053         Run,
00054         Tumble
00055     };
00056
00057     struct PopulationMutation {
00058         PopulationMutationType type = PopulationMutationType::Grow;
00059         unsigned int value = 0;
00060         unsigned int previousPopulationSize = 0;
00061         unsigned int resultingPopulationSize = 0;
00062     };
00063
00064     struct SignalMutation {
00065         SignalMutationType type = SignalMutationType::Emit;
00066         double value = 0.0;
00067     };
00068
00069     struct MotionMutation {
00070         MotionMutationType type = MotionMutationType::Run;
00071         double value = 0.0;
00072         double previousDirection = 0.0;
00073         double resultingDirection = 0.0;
00074         double previousSpeed = 0.0;
00075         double resultingSpeed = 0.0;
00076     };
00077
00078     struct ColonyMutation {
00079         enum class Type {
00080             Reset,

```



```

00081         SpawnSeed,
00082         SetChemostatMode,
00083         AddBarrier,
00084         MapToCells,
00085         SetSignalAt,
00086         SetSignalRect,
00087         SetSignalGridWidth,
00088         SetSignalGridHeight,
00089         GetSignalMatrix,
00090         DumpSignalField
00091     };
00092
00093     Type type = Type::Reset;
00094     std::vector<std::string> arguments;
00095     std::vector<double> numericArguments;
00096     std::string expressionText;
00097     std::size_t previewRows = 0;
00098     std::size_t previewColumns = 0;
00099 };
00100
00101 struct ExecutionResult {
00102     bool succeeded = true;
00103     std::string errorMessage = "";
00104     unsigned int executedCommands = 0;
00105     std::vector<PopulationMutation> populationMutations;
00106     std::vector<SignalMutation> signalMutations;
00107     std::vector<MotionMutation> motionMutations;
00108     std::vector<ColonyMutation> colonyMutations;
00109     std::vector<double> mappedCellValues;
00110     std::string mappedCellExpression;
00111     std::vector<double> signalMatrixValues;
00112     unsigned int signalMatrixWidth = 0;
00113     unsigned int signalMatrixHeight = 0;
00114     std::string signalFieldDump;
00115     std::vector<std::string> messages;
00116     std::map<std::string, double> assignedVariables;
00117     std::vector<std::string> unsupportedCommands;
00118     std::vector<std::string> skippedRawStatements;
00119 };
00120
00121 public:
00122     ExecutionResult execute(const GroProgramIr& ir, GroProgramRuntimeState& state) const;
00123     static bool evaluateExpression(const std::string& expressionText,
00124                                   const GroProgramRuntimeState& state,
00125                                   double& value,
00126                                   std::string& errorMessage);
00127 };
00128
00129 };
00130
00131 #endif /* GROPROGRAMRUNTIME_H */

```

11.426 MetabolicNetwork.h

```

00001 #ifndef METABOLICNETWORK_H
00002 #define METABOLICNETWORK_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelDataDefinition.h"
00008 #include "kernel/simulator/PluginInformation.h"
00009
00016 class MetabolicNetwork : public ModelDataDefinition {
00017 public:
00018     MetabolicNetwork(Model* model, std::string name = "");
00019     virtual ~MetabolicNetwork() override = default;
00020
00021 public:
00022     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00023     static PluginInformation* GetPluginInformation();
00024     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00025
00026 public:
00027     void addReaction(std::string reactionName);
00028     void addExchangeSpecies(std::string speciesName);
00029     void clearReactions();
00030     void clearExchangeSpecies();
00031     const std::vector<std::string>& getReactionNames() const;
00032     const std::vector<std::string>& getExchangeSpeciesNames() const;
00033     void setObjectiveReactionName(std::string objectiveReactionName);
00034     std::string getObjectiveReactionName() const;
00035     void setObjectiveSense(std::string objectiveSense);
00036     std::string getObjectiveSense() const;
00037     void setCompartment(std::string compartment);

```

```

00038     std::string getCompartment() const;
00039     void setEnabled(bool enabled);
00040     bool isEnabled() const;
00041
00042     virtual std::string show() override;
00043
00044 protected:
00045     virtual bool _loadInstance(PersistenceRecord* fields) override;
00046     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00047     virtual bool _check(std::string& errorMessage) override;
00048     virtual void _createEditableDataDefinitions() override;
00049
00050 protected:
00051     // virtual void _createInternalStatisticReporters() override;
00052     // virtual void _createNonEditableDataDefinitions() override;
00053     // virtual void _createAttachedAttributes() override;
00054
00055 private:
00056     bool _checkReactionNames(std::string& errorMessage) const;
00057     bool _checkExchangeSpeciesNames(std::string& errorMessage) const;
00058
00059 private:
00060     const struct DEFAULT_VALUES {
00061         std::string objectiveReactionName = "";
00062         std::string objectiveSense = "Maximize";
00063         std::string compartment = "";
00064         bool enabled = true;
00065     } DEFAULT;
00066
00067     std::vector<std::string> _reactionNames;
00068     std::vector<std::string> _exchangeSpeciesNames;
00069     std::string _objectiveReactionName = DEFAULT.objectiveReactionName;
00070     std::string _objectiveSense = DEFAULT.objectiveSense;
00071     std::string _compartment = DEFAULT.compartment;
00072     bool _enabled = DEFAULT.enabled;
00073 };
00074
00075 #endif /* METABOLICNETWORK_H */

```

11.427 MetabolicReaction.h

```

00001 #ifndef METABOLICREACTION_H
00002 #define METABOLICREACTION_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelDataDefinition.h"
00008 #include "kernel/simulator/PluginInformation.h"
00009
00013 class MetabolicReaction : public ModelDataDefinition {
00014 public:
00015     struct StoichiometricTerm {
00016         std::string speciesName;
00017         double stoichiometry = 1.0;
00018     };
00019
00020 public:
00021     MetabolicReaction(Model* model, std::string name = "");
00022     virtual ~MetabolicReaction() override = default;
00023
00024 public:
00025     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00026     static PluginInformation* GetPluginInformation();
00027     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00028
00029 public:
00030     void addReactant(std::string speciesName, double stoichiometry = 1.0);
00031     void addProduct(std::string speciesName, double stoichiometry = 1.0);
00032     void clearReactants();
00033     void clearProducts();
00034     const std::vector<StoichiometricTerm>& getReactants() const;
00035     const std::vector<StoichiometricTerm>& getProducts() const;
00036     void setLowerBound(double lowerBound);
00037     double getLowerBound() const;
00038     void setUpperBound(double upperBound);
00039     double getUpperBound() const;
00040     void setObjectiveCoefficient(double objectiveCoefficient);
00041     double getObjectiveCoefficient() const;
00042     void setReversible(bool reversible);
00043     bool isReversible() const;
00044     void setGeneRule(std::string geneRule);
00045     std::string getGeneRule() const;

```

```

00046
00047     virtual std::string show() override;
00048
00049 protected:
00050     virtual bool _loadInstance(PersistenceRecord* fields) override;
00051     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00052     virtual bool _check(std::string& errorMessage) override;
00053     virtual void _createEditableDataDefinitions() override;
00054
00055 protected:
00056     // virtual void _createInternalStatisticReporters() override;
00057     // virtual void _createNonEditableDataDefinitions() override;
00058     // virtual void _createAttachedAttributes() override;
00059
00060 private:
00061     bool _checkTerms(const std::vector<StoichiometricTerm>& terms, const std::string& side,
00062                     std::string& errorMessage) const;
00063 private:
00064     const struct DEFAULT_VALUES {
00065         double lowerBound = 0.0;
00066         double upperBound = 1000.0;
00067         double objectiveCoefficient = 0.0;
00068         bool reversible = false;
00069         std::string geneRule = "";
00070     } DEFAULT;
00071
00072     std::vector<StoichiometricTerm> _reactants;
00073     std::vector<StoichiometricTerm> _products;
00074     double _lowerBound = DEFAULT.lowerBound;
00075     double _upperBound = DEFAULT.upperBound;
00076     double _objectiveCoefficient = DEFAULT.objectiveCoefficient;
00077     bool _reversible = DEFAULT.reversible;
00078     std::string _geneRule = DEFAULT.geneRule;
00079 };
00080
00081 #endif /* METABOLICREACTION_H */

```

11.428 DiffusionField.h

```

00001 #ifndef DIFFUSIONFIELD_H
00002 #define DIFFUSIONFIELD_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../kernel/simulator/model/ModelDataDefinition.h"
00008 #include "kernel/simulator/PluginInformation.h"
00009
00010 class DiffusionMethodOfLinesSystem;
00011
00012 // Plugin that solves the diffusion (heat) equation  $du/dt = D \cdot \text{Laplacian}(u)$  in N
00013 // dimensions.
00014 //
00015 // It works in two steps. First the Method of Lines turns the PDE into a system
00016 // of ODEs (central differences in space - see DiffusionMethodOfLinesSystem).
00017 // Then it integrates that system in time with a solver from OdeSolverFactory, so
00018 // the user can pick RK4 or Dormand-Prince 5(4), exactly like in BioNetwork.
00019 //
00020 // N is set by the Dimensions control; the grid has PointsPerDimension points and
00021 // length DomainLength on each axis. The boundary is Dirichlet (fixed value) or
00022 // Neumann (no flux, conserves mass), and the starting field is one of a few
00023 // named shapes (sine mode, Gaussian bump, constant).
00024 //
00025 // The class follows the same layout as BioNetwork (controls, save/load, _check,
00026 // simulate / advanceOneStep) to stay consistent with the rest of the code. The
00027 // grid doesn't change during a run, so each step just rebuilds the small system
00028 // object and reuses the stored field. The solver is never hard-coded - it comes
00029 // from the OdeSolver name through the factory.
00030 //
00031 // Equation representation (design choice): the diffusion PDE is encoded as a
00032 // dedicated C++ class (DiffusionMethodOfLinesSystem), not as a parser expression
00033 // nor as dynamically compiled C++. This is a deliberate scope decision - a
00034 // specific, well-tested class for diffusion - leaving a general user-defined
00035 // equation mechanism (parser or CppCompiler) as a future extension.
00036 //
00037 // Time semantics: with AutoSchedule enabled, the field steps once per kernel
00038 // InternalEvent (see scheduleNextInternalEvent), so the continuous solve advances
00039 // in sync with the discrete-event clock; otherwise simulate() runs the whole time
00040 // window synchronously when driven by the DiffusionSimulate component.
00041 class DiffusionField : public ModelDataDefinition {
00042 public:
00043     DiffusionField(Model* model, std::string name = "");

```

```

00044     virtual ~DiffusionField() override = default;
00045
00046 public:
00047     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00048     static PluginInformation* GetPluginInformation();
00049     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00050
00051 public:
00052     // --- configuration ---
00053     void setDimensions(unsigned int dimensions);
00054     unsigned int getDimensions() const;
00055     void setPointsPerDimension(unsigned int pointsPerDimension);
00056     unsigned int getPointsPerDimension() const;
00057     void setDomainLength(double domainLength);
00058     double getDomainLength() const;
00059     void setDiffusionCoefficient(double diffusionCoefficient);
00060     double getDiffusionCoefficient() const;
00061     void setBoundaryCondition(std::string boundaryCondition);
00062     std::string getBoundaryCondition() const;
00063     void setInitialCondition(std::string initialCondition);
00064     std::string getInitialCondition() const;
00065     void setInitialParameter(double initialParameter);
00066     double getInitialParameter() const;
00067     void setStartTime(double startTime);
00068     double getStartTime() const;
00069     void setStopTime(double stopTime);
00070     double getStopTime() const;
00071     void setStepSize(double stepSize);
00072     double getStepSize() const;
00073     void setCurrentTime(double currentTime);
00074     double getCurrentTime() const;
00075     // When true, the field schedules itself on the kernel's event queue and
00076     // advances one step per InternalEvent, staying in sync with simulated time.
00077     void setAutoSchedule(bool autoSchedule);
00078     bool getAutoSchedule() const;
00079     void setOdeSolver(std::string odeSolver);
00080     std::string getOdeSolver() const;
00081     void setLastStatus(std::string lastStatus);
00082     std::string getLastStatus() const;
00083     void setLastErrorMessage(std::string lastErrorMessage);
00084     std::string getLastErrorMessage() const;
00085
00086     // --- diagnostics (read-only outputs) ---
00087     double getTotalMass() const;
00088     double getMaxValue() const;
00089     double getL2Norm() const;
00090     const std::vector<double>& getField() const;
00091     double getFieldValue(const std::vector<unsigned int>& multiIndex) const;
00092
00093     // --- execution ---
00094     bool simulate(std::string& errorMessage);
00095     bool simulate(double startTime, double stopTime, double stepSize, std::string& errorMessage);
00096     bool advanceOneStep(std::string& errorMessage);
00097
00098     // --- display ---
00099     std::string showFieldAsAscii() const;
00100
00101     virtual std::string show() override;
00102
00103 protected:
00104     virtual bool _loadInstance(PersistenceRecord* fields) override;
00105     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00106     virtual bool _check(std::string& errorMessage) override;
00107     virtual void _initBetweenReplications() override;
00108
00109 private:
00110     bool buildSystem(DiffusionMethodOfLinesSystem* system, std::string& errorMessage) const;
00111     void buildInitialField(const DiffusionMethodOfLinesSystem& system, std::vector<double>& field) const;
00112     void refreshDiagnostics(const DiffusionMethodOfLinesSystem& system);
00113     // Event-driven advance (mirrors BioNetwork): each InternalEvent advances one
00114     // step and reschedules the next, coupling the continuous solve to the kernel clock.
00115     void handleInternalEvent(void* parameter);
00116     void scheduleNextInternalEvent();
00117
00118 private:
00119     const struct DEFAULT_VALUES {
00120         unsigned int dimensions = 2;
00121         unsigned int pointsPerDimension = 21;
00122         double domainLength = 1.0;
00123         double diffusionCoefficient = 0.1;
00124         std::string boundaryCondition = "Dirichlet";
00125         std::string initialCondition = "SineMode";
00126         double initialParameter = 1.0;
00127         double startTime = 0.0;
00128         double stopTime = 0.1;
00129         double stepSize = 0.01;
00130     };

```

```

00131         double currentTime = 0.0;
00132         bool autoSchedule = false;
00133         std::string odeSolver = "RungeKutta4";
00134         std::string lastStatus = "Idle";
00135         std::string lastErrorMessage = "";
00136     } DEFAULT;
00137
00138     unsigned int _dimensions = DEFAULT.dimensions;
00139     unsigned int _pointsPerDimension = DEFAULT.pointsPerDimension;
00140     double _domainLength = DEFAULT.domainLength;
00141     double _diffusionCoefficient = DEFAULT.diffusionCoefficient;
00142     std::string _boundaryCondition = DEFAULT.boundaryCondition;
00143     std::string _initialCondition = DEFAULT.initialCondition;
00144     double _initialParameter = DEFAULT.initialParameter;
00145     double _startTime = DEFAULT.startTime;
00146     double _stopTime = DEFAULT.stopTime;
00147     double _stepSize = DEFAULT.stepSize;
00148     double _currentTime = DEFAULT.currentTime;
00149     bool _autoSchedule = DEFAULT.autoSchedule;
00150     std::string _odeSolver = DEFAULT.odeSolver;
00151     std::string _lastStatus = DEFAULT.lastStatus;
00152     std::string _lastErrorMessage = DEFAULT.lastErrorMessage;
00153
00154     std::vector<double> _field;           // current flattened grid state
00155     double _lastTotalMass = 0.0;
00156     double _lastMaxValue = 0.0;
00157     double _lastL2Norm = 0.0;
00158 };
00159
00160 #endif /* DIFFUSIONFIELD_H */

```

11.429 ODESolver.h

```

00001 /*
00002  * File:   ODESolver.h
00003  * Author: GenESys
00004  *
00005  * Created on 13 de maio de 2026
00006  */
00007
00008 #ifndef ODESOLVER_H
00009 #define ODESOLVER_H
00010
00011 #include <string>
00012 #include <vector>
00013
00014 #include "kernel/simulator/model/ModelDataDefinition.h"
00015 #include "kernel/simulator/PluginInformation.h"
00016 #include "tools/Continuous/OdeSolver_if.h"
00017 #include "tools/Continuous/OdeSystem_if.h"
00018 #include "tools/Continuous/RungeKutta4OdeSolver.h"
00019
00020 class ODESolver : public ModelDataDefinition {
00021 public:
00022     ODESolver(Model* model, std::string name = "");
00023     virtual ~ODESolver() = default;
00024 public: // static
00025     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00026     static PluginInformation* GetPluginInformation();
00027     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00028     static void SaveInstance(PersistenceRecord *fields, ModelDataDefinition* dataDef);
00029     static bool Check(ModelDataDefinition* dataDef, std::string& errorMessage);
00030     static void InitBetweenReplications(ModelDataDefinition* dataDef);
00031     static void CreateInternalData(ModelDataDefinition* dataDef);
00032 public:
00033     virtual std::string show() override;
00034 public: // gets & sets
00035     void setTimeVariableName(std::string timeVariableName);
00036     std::string getTimeVariableName() const;
00037     void setStateVariableNames(std::vector<std::string> stateVariableNames);
00038     std::vector<std::string> getStateVariableNames() const;
00039     void setEquationExpressions(std::vector<std::string> equationExpressions);
00040     std::vector<std::string> getEquationExpressions() const;
00041     void setStep(double step);
00042     double getStep() const;
00043     void setPrecision(double precision);
00044     double getPrecision() const;
00045     void setMaxSteps(int maxSteps);
00046     int getMaxSteps() const;
00047     void setCurrentTime(double currentTime);
00048     double getCurrentTime() const;
00049     void setStateValues(std::vector<double> stateValues);
00050     std::vector<double> getStateValues() const;

```

```

00051     void setInitialStateValues(std::vector<double> initialStateValues);
00052     std::vector<double> getInitialStateValues() const;
00053     void setAutoAdvance(bool autoAdvance);
00054     bool isAutoAdvance() const;
00055     void setOutputFile(std::string outputFile);
00056     std::string getOutputFile() const;
00057 public: // new methods
00058     void integrate(double targetTime);
00059     void resetState();
00060     double getStateValue(unsigned int index) const;
00061     void setStateValue(unsigned int index, double value);
00062     void synchronizeStateFromVariables();
00063     void writeStateToVariables() const;
00064
00065 protected: // must be overridden
00066     virtual bool _loadInstance(PersistenceRecord *fields) override;
00067     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00068 protected: // could be overridden
00069     virtual bool _check(std::string& errorMessage) override;
00070     virtual void _initBetweenReplications() override;
00071     virtual void _createEditableDataDefinitions() override;
00072
00073 private:
00074     const struct DEFAULT_VALUES {
00075         const std::string timeVariableName = "t";
00076         const double step = 0.01;
00077         const double precision = 1e-6;
00078         const int maxSteps = 100000;
00079         const double currentTime = 0.0;
00080         const bool autoAdvance = true;
00081         const std::string outputFile = "";
00082     } DEFAULT;
00083     std::string _timeVariableName = DEFAULT.timeVariableName;
00084     std::vector<std::string> _stateVariableNames;
00085     std::vector<std::string> _equationExpressions;
00086     double _step = DEFAULT.step;
00087     double _precision = DEFAULT.precision;
00088     int _maxSteps = DEFAULT.maxSteps;
00089     double _currentTime = DEFAULT.currentTime;
00090     bool _autoAdvance = DEFAULT.autoAdvance;
00091     std::string _outputFile = DEFAULT.outputFile;
00092     std::vector<double> _stateValues;
00093     std::vector<double> _initialStateValues;
00094     RungeKutta4OdeSolver _solver;
00095
00096     std::vector<double> _loadStateVector(PersistenceRecord* fields, const std::string& prefix) const;
00097     void _saveStateVector(PersistenceRecord* fields,
00098                          const std::string& prefix,
00099                          const std::vector<double>& values,
00100                          bool saveDefaultValues) const;
00101     double _readVariableValue(const std::string& variableName) const;
00102     // Self-scheduling that couples the continuous integration to the discrete event calendar:
00103     // each internal event advances the ODE by one step and schedules the next, so the continuous
00104     // state evolves with the simulation clock instead of only when a discrete entity arrives.
00105     void _handleInternalEvent(void* parameter);
00106     void _scheduleNextInternalEvent();
00107     // Optional CSV dump of the trajectory: when _outputFile is set, one row per integration
00108     // step is written (t and each state variable), so any model can export its trajectory.
00109     void _writeOutputHeader() const;
00110     void _appendOutputRow() const;
00111 };
00112
00113 #endif /* ODESOLVER_H */

```

11.430 Failure.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Failure.h
00009  * Author: rlcancian
00010  *
00011  * Created on 20 de Failureembro de 2019, 20:07
00012  */
00013
00014 #ifndef FAILURE_H
00015 #define FAILURE_H
00016
00017

```

```

00018 #include "../kernel/simulator/model/ModelDataDefinition.h"
00019 #include "../kernel/simulator/model/ModelDataManager.h"
00020 #include "kernel/simulator/PluginInformation.h"
00021
00022 class Resource;
00023
00068 class Failure : public ModelDataDefinition {
00069 public:
00070
00071     enum class FailureType : int {
00072         COUNT = 0, TIME = 1, num_elements = 2
00073     };
00074
00075     enum class FailureRule : int {
00076         IGNORE = 0, PREEMPT = 1, WAIT = 2, num_elements = 3
00077     };
00078 public:
00079     static std::string convertEnumToStr(FailureType type);
00080     static std::string convertEnumToStr(FailureRule rule);
00081 public:
00082     Failure(Model* model, std::string name = "");
00083     virtual ~Failure() override;
00084 public: // static
00085     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00086     static PluginInformation* GetPluginInformation();
00087     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00088 public:
00089     virtual std::string show() override;
00090     void checkIsGoingToFailByCount(Resource* resource);
00091 public: // gets & sets
00092     void setFailureType(FailureType _failureType);
00093     FailureType getFailureType() const;
00094     void setCountExpression(std::string countExpression);
00095     std::string getCountExpression() const;
00096     void setDownTimeTimeUnit(Util::TimeUnit downTimeTimeUnit);
00097     Util::TimeUnit getDownTimeTimeUnit() const;
00098     void setDownTimeExpression(std::string downTimeExpression);
00099     std::string getDownTimeExpression() const;
00100     void setUpTimeTimeUnit(Util::TimeUnit upTimeTimeUnit);
00101     Util::TimeUnit getUpTimeTimeUnit() const;
00102     void setUpTimeExpression(std::string upTimeExpression);
00103     std::string getUpTimeExpression() const;
00104     void setFailureRule(FailureRule _failureRule);
00105     FailureRule getFailureRule() const;
00106
00107     List<Resource*>*failingResources() const;
00108     void addResource(Resource* newResource);
00109     void removeResource(Resource* resource);
00110
00111 protected: // must be overridden
00112     virtual bool _loadInstance(PersistenceRecord *fields) override;
00113     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00114 protected: // could be overridden
00115     virtual bool _check(std::string& errorMessage) override;
00116     virtual void _initBetweenReplications() override;
00117     //virtual ParserChangesInformation* _getParserChangesInformation();
00118 protected:
00119     // virtual void _createInternalStatisticReporters() override;
00120     // virtual void _createNonEditableDataDefinitions() override;
00121     // virtual void _createEditableDataDefinitions() override;
00122     // virtual void _createAttachedAttributes() override;
00123
00124 private: // simulation internal event handlers
00125     void _onFailureActiveEventHandler(void* resourcePtr);
00126     void _onFailureFailEventHandler(void* resourcePtr);
00127 private:
00128     void _scheduleActivation(Resource* resource);
00129 private:
00130
00131     const struct DEFAULT_VALUES {
00132         const FailureType failureType = FailureType::TIME;
00133         const FailureRule failureRule = FailureRule::IGNORE;
00134         const std::string countExpression = "";
00135         const std::string upTimeExpression = "";
00136         const Util::TimeUnit upTimeTimeUnit = Util::TimeUnit::second;
00137         const std::string downTimeExpression = "";
00138         const Util::TimeUnit downTimeTimeUnit = Util::TimeUnit::second;
00139     } DEFAULT;
00140     FailureType _failureType = DEFAULT.failureType;
00141     FailureRule _failureRule = DEFAULT.failureRule;
00142     std::string _countExpression = DEFAULT.countExpression;
00143     std::string _upTimeExpression = DEFAULT.upTimeExpression;
00144     Util::TimeUnit _upTimeTimeUnit = DEFAULT.upTimeTimeUnit;
00145     std::string _downTimeExpression = DEFAULT.downTimeExpression;
00146     Util::TimeUnit _downTimeTimeUnit = DEFAULT.downTimeTimeUnit;
00147 private:
00148     std::map<Resource*, unsigned int>* _releaseCounts = new std::map<Resource*, unsigned int>();

```

```

00149     List<Resource*>* _falingResources = new List<Resource*>();
00150 };
00151
00152 #endif /* FAILURE_H */

```

11.431 Queue.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   Queue.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 21 de Agosto de 2018, 17:12
00012  */
00013
00014 #ifndef QUEUE_H
00015 #define QUEUE_H
00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "kernel/util/List.h"
00019 #include "../kernel/simulator/essentialPlugins/Entity.h"
00020 #include "../kernel/simulator/model/ModelDataManager.h"
00021 #include "../kernel/simulator/essentialPlugins/StatisticsCollector.h"
00022 #include "kernel/simulator/Plugin.h"
00023 #include "../kernel/simulator/model/ModelComponent.h"
00024
00025 class Waiting {
00026 public:
00027     Waiting(Entity* entity, double timeStartedWaiting, ModelComponent* thisComponent, unsigned int
thisComponentOutputPort = 0) {
00028         _entity = entity;
00029         _thisComponent = thisComponent;
00030         _timeStartedWaiting = timeStartedWaiting;
00031         _thisComponentOutputPort = thisComponentOutputPort;
00032     }
00033
00034     virtual ~Waiting() = default;
00035 public:
00036     virtual std::string show() {
00037         return //ModelDataDefinition::show()+
00038             ",entity=" + std::to_string(_entity->getId()) +
00039             ",component=\"" + _thisComponent->getName() + "\"" +
00040             ",inputPort=\"" + std::to_string(_thisComponentOutputPort) + "\"" +
00041             ",timeStartedWaiting=" + std::to_string(_timeStartedWaiting);
00042     }
00043 public:
00044     double getTimeStartedWaiting() const {
00045         return _timeStartedWaiting;
00046     }
00047     ModelComponent* geComponent() const {
00048         return _thisComponent;
00049     }
00050     Entity* getEntity() const {
00051         return _entity;
00052     }
00053     unsigned int geComponentOutputPort() const {
00054         return _thisComponentOutputPort;
00055     }
00056     unsigned long long getArrivalOrder() const {
00057         return _arrivalOrder;
00058     }
00059     void setArrivalOrder(unsigned long long arrivalOrder) {
00060         _arrivalOrder = arrivalOrder;
00061     }
00062
00063 protected:
00064 private:
00065     Entity* _entity;
00066     ModelComponent* _thisComponent;
00067     double _timeStartedWaiting;
00068     unsigned int _thisComponentOutputPort;
00069     unsigned long long _arrivalOrder = 0;
00070 };
00071
00101 class Queue : public ModelDataDefinition {
00102 public:
00103
00104     enum class OrderRule : int {

```



```

00105         FIFO = 0, LIFO = 1, HIGHESTVALUE = 2, SMALLESTVALUE = 3, num_elements = 4
00106     };
00107 public:
00108     static std::string convertEnumToStr(OrderRule rule);
00109 public:
00110     Queue(Model* model, std::string name = "");
00111     virtual ~Queue();
00112 public:
00113     virtual std::string show() override;
00114 public: // static
00115     static PluginInformation* GetPluginInformation();
00116     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00117     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00118 public:
00119     void insertElement(Waiting* modeldatum);
00120     void removeElement(Waiting* modeldatum);
00121     unsigned int size();
00122     Waiting* first();
00123     Waiting* getAtRank(unsigned int rank);
00124     void setAttributeName(std::string _attributeName);
00125     std::string getAttributeName() const;
00126     void setOrderRule(OrderRule _orderRule);
00127     Queue::OrderRule getOrderRule() const;
00128     void setOrderRuleInt(int orderRule);
00129     int getOrderRuleInt() const;
00130 public: // to implement SIMAN functions
00131     double sumAttributesFromWaiting(Util::identification attributeID); // use to implement SIMAN SAQUE
    function
00132     double getAttributeFromWaitingRank(unsigned int rank, Util::identification attributeID);
00133     //public:
00134     // void initBetweenReplications();
00135 protected: // must be overridden
00136     virtual bool _loadInstance(PersistenceRecord *fields) override;
00137     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00138 protected: // could be overridden
00139     virtual bool _check(std::string& errorMessage) override;
00140     virtual void _initBetweenReplications() override;
00141     virtual ParserChangesInformation* _getParserChangesInformation() override;
00142 protected:
00143     virtual void _createInternalStatisticReporters() override;
00144     // virtual void _createNonEditableDataDefinitions() override;
00145     // virtual void _createEditableDataDefinitions() override;
00146     // virtual void _createAttachedAttributes() override;
00147
00148 private:
00149     void _initCStats();
00150     void _configureListComparator();
00151 private:
00152     List<Waiting*>* _list = new List<Waiting*>();
00153     double _lastTimeNumberInQueueChanged = 0.0;
00154     unsigned long long _nextArrivalOrder = 0;
00155 private: //1::1
00156
00157     const struct DEFAULT_VALUES {
00158         const OrderRule orderRule = OrderRule::FIFO;
00159         const std::string attributeName = "";
00160     } DEFAULT;
00161     OrderRule _orderRule = DEFAULT.orderRule;
00162     std::string _attributeName = DEFAULT.attributeName;
00163 private: // inner internal elements
00164     StatisticsCollector* _cstatNumberInQueue = nullptr;
00165     StatisticsCollector* _cstatTimeInQueue = nullptr;
00166 };
00167
00168 #endif /* QUEUE_H */

```

11.432 Resource.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:      Resource.h
00009  * Author:    rafael.luiz.cancian
00010  *
00011  * Created on 21 de Agosto de 2018, 16:52
00012  */
00013
00014 #ifndef RESOURCE_H
00015 #define RESOURCE_H

```

```

00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "../kernel/simulator/essentialPlugins/StatisticsCollector.h"
00019 #include "../kernel/simulator/model/ModelDataManager.h"
00020 #include "../kernel/simulator/essentialPlugins/Counter.h"
00021 #include "kernel/simulator/Plugin.h"
00022 #include "kernel/simulator/OnEventManager.h"
00023
00024 #include "plugins/data/DiscreteProcessing/Failure.h"
00025 #include "plugins/data/DiscreteProcessing/Schedule.h"
00026
00027 #include <functional>
00028
00029
00030
00031 class SeizableItem;
00032
00033
00034
00035
00089 class Resource : public ModelDataDefinition {
00090 public:
00091     typedef std::function<void(Resource*) > ResourceEventHandler;
00092     typedef std::pair<std::pair<ResourceEventHandler, ModelComponent*>, unsigned int>
SortedResourceEventHandler;
00093
00094     template<typename Class>
00095     static ResourceEventHandler SetResourceEventHandler(void (Class::*function)(Resource*), Class *
object) {
00096         return std::bind(function, object, std::placeholders::_1);
00097     }
00098
00099     enum class ResourceState : int {
00100         IDLE = 0, BUSY = 1, FAILED = 2, INACTIVE = 3, OTHER = 4, num_elements = 5
00101     };
00102 public:
00103     static std::string convertEnumToStr(ResourceState state);
00104 public:
00105     //Resource(Model* model);
00106     Resource(Model* model, std::string name = "");
00107     virtual ~Resource() override;
00108 public:
00109     virtual std::string show() override;
00110 public: // static
00111     static PluginInformation* GetPluginInformation();
00112     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00113     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00114 public:
00115     bool seize(unsigned int quantity, double priority = 0);
00116     void release(unsigned int quantity);
00117     void insertFailure(Failure* failure);
00118     void removeFailure(Failure* failure);
00119     double getInstantCapacityUtilization() const;
00120     double getCapacityUtilization() const;
00121     double getSeizedUtilization() const;
00122     double getLastTimeSeized() const; // used only by "Release" component
00123     void addReleaseResourceEventHandler(ResourceEventHandler eventHandler, ModelComponent* component,
unsigned int priority);
00124 public: // g&s
00125     void setResourceState(ResourceState _resourceState);
00126     Resource::ResourceState getResourceState() const;
00127     void setCapacity(unsigned int capacity);
00128     unsigned int getCapacity() const;
00129     void setCostBusyTimeUnit(double _costBusyTimeUnit);
00130     double getCostBusyTimeUnit() const;
00131     void setCostIdleTimeUnit(double _costIdleTimeUnit);
00132     double getCostIdleTimeUnit() const;
00133     void setCostPerUse(double _costPerUse);
00134     double getCostPerUse() const;
00135     void setCapacitySchedule(Schedule* _capacitySchedule);
00136     Schedule* getCapacitySchedule() const;
00137     unsigned int getNumberBusy() const;
00138
00139 protected: // protected must override
00140     virtual bool _loadInstance(PersistenceRecord *fields) override;
00141     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00142 protected: // protected could override
00143     virtual bool _check(std::string& errorMessage) override;
00144     // virtual void _createInternalAndAttachedData() override;
00145     virtual void _initBetweenReplications() override;
00146
00147
00148 protected:
00149     // virtual void _createInternalStatisticReporters() override;
00150     // virtual void _createNonEditableDataDefinitions() override;
00151     // virtual void _createEditableDataDefinitions() override;
00152     virtual void _createAttachedAttributes() override;
00153
00154 private: //methods
00155     void _notifyReleaseEventHandlers();

```

```

00156     void _onReplicationEnd(SimulationEvent* se);
00157     void _fail();
00158     void _active();
00159     void _checkFailByCount();
00160     void _initStatisticsAndAccounting();
00161     friend class Failure;
00162     friend class ResourceTestProbe;
00163
00164 private:
00165
00166     const struct DEFAULT_VALUES {
00167         const unsigned int capacity = 1;
00168         const double cost = 1.0;
00169         const ResourceState resourceState = ResourceState::IDLE;
00170     } DEFAULT;
00171     unsigned int _capacity = DEFAULT.capacity;
00172     double _costBusyTimeUnit = DEFAULT.cost;
00173     double _costIdleTimeUnit = DEFAULT.cost;
00174     double _costPerUse = DEFAULT.cost;
00175     ResourceState _resourceState = DEFAULT.resourceState;
00176 private: // only gets
00177     unsigned int _numberBusy = 0;
00178     double _lastTimeSeized = 0.0; // @TODO: It won't work for resources with capacity>1, when not all
    capacity is seized and them some more are seized. Seized time of first units will be lost. I don't
    have a solution so far
00179     double _lastTimeReleased = 0.0;
00180     double _lastTimeFailed = 0.0;
00181     double _lastTimeCapacityEvaluated = 0.0;
00182     double _lastTimeAnythingNumberBusy = 0.0;
00183     double _lastTimeIdle = 0.0;
00184     double _lastTimeBusy = 0.0;
00185     double _sumNumberBusyOverTime = 0.0;
00186     double _sumCapacityOverTime = 0.0;
00187     bool _isActive = true;
00188     bool _replicationEndHandlerRegistered = false;
00189 private: // not gets nor sets
00190     unsigned int _originalCapacity; // used for failing purposes, when _capacity changes to 0
00191 private: //!::n
00192     List<SortedResourceEventHandler*>* _resourceEventHandlers = new
    List<SortedResourceEventHandler*>();
00193     List<Failure*>* _failures = new List<Failure*>();
00194 private: // attached elements
00195     Schedule* _capacitySchedule = nullptr;
00196 private: // internal elements
00197     StatisticsCollector* _cstatTimeSeized = nullptr;
00198     StatisticsCollector* _cstatTimeFailed = nullptr;
00199     StatisticsCollector* _cstatProportionSeized = nullptr;
00200     StatisticsCollector* _cstatCapacityUtilization = nullptr;
00201     Counter* _counterTotalTimeSeized = nullptr;
00202     Counter* _counterTotalTimeFailed = nullptr;
00203     Counter* _counterNumSeizes = nullptr;
00204     Counter* _counterNumReleases = nullptr;
00205     Counter* _counterTotalCostPerUse = nullptr;
00206     Counter* _counterTotalCostBusy = nullptr;
00207     Counter* _counterTotalCostIdle = nullptr;
00208 };
00209
00210 #endif /* RESOURCE_H */

```

11.433 Schedule.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File:    Schedule.h
00008  * Author:  Prof. Rafael Luiz Cancian, Dr. Eng.
00009  *
00010  * Created on 14 de dezembro de 2022, 04:24
00011  */
00012
00013 #ifndef SCHEDULE_H
00014 #define SCHEDULE_H
00015
00016 #include "../kernel/simulator/model/ModelDataDefinition.h"
00017 #include "kernel/simulator/PluginInformation.h"
00018 #include "kernel/util/List.h"
00019
00020 class SchedulableItem {
00021 public:
00022

```

```

00023     enum class Rule : int {
00024         IGNORE = 1, PREEMPT = 2, WAIT = 3
00025     };
00026
00027 public:
00028
00029     SchedulableItem(std::string expression, double duration, SchedulableItem::Rule rule =
SchedulableItem::Rule::IGNORE) {
00030         this->expression = expression;
00031         this->duration = duration;
00032         this->rule = rule;
00033     }
00034 public:
00035
00036     void setDuration(double duration) {
00037         this->duration = duration;
00038     }
00039
00040     double getDuration() const {
00041         return duration;
00042     }
00043
00044     void setExpression(std::string expression) {
00045         this->expression = expression;
00046     }
00047
00048     std::string getExpression() const {
00049         return expression;
00050     }
00051
00052     void setRule(SchedulableItem::Rule rule) {
00053         this->rule = rule;
00054     }
00055
00056     SchedulableItem::Rule getRule() const {
00057         return rule;
00058     }
00059
00060 protected:
00061 private:
00062     std::string expression;
00063     double duration;
00064     SchedulableItem::Rule rule;
00065 };
00066
00067 //class SchedulableResourceItem : public SchedulableItem {
00068 //};
00069
00070 class Schedule : public ModelDataDefinition {
00071 public:
00072     Schedule(Model* model, std::string name = "");
00073     virtual ~Schedule() override;
00074 public: // static
00075     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00076     static PluginInformation* GetPluginInformation();
00077     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00078 public: // virtual
00079     virtual std::string show() override;
00080 public:
00081     std::string getExpression();
00082     List<SchedulableItem*>* getSchedulableItems() const;
00083     void setRepeatAfterLast(bool _repeatAfterLast);
00084     bool isRepeatAfterLast() const;
00085 protected: // must be overridden
00086     virtual bool _loadInstance(PersistenceRecord *fields) override;
00087     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00088 protected: // could be overridden by derived classes
00089     virtual bool _check(std::string& errorMessage) override;
00090     virtual ParserChangesInformation* _getParserChangesInformation() override;
00091     virtual void _initBetweenReplications() override;
00092     virtual void _addSimulationControl(SimulationControl* property) override;
00093 protected:
00094     // virtual void _createInternalStatisticReporters() override;
00095     // virtual void _createNonEditableDataDefinitions() override;
00096     // virtual void _createEditableDataDefinitions() override;
00097     // virtual void _createAttachedAttributes() override;
00098
00099
00100 private:
00101
00102     const struct DEFAULT_VALUES {
00103         //const std::string someString = "Test";
00104         //const unsigned int someUInt = 1;
00105         bool repeatAfterLast = true;
00106     } DEFAULT;
00107     bool _repeatAfterLast = DEFAULT.repeatAfterLast;
00108     List<SchedulableItem*>* _schedulableItems = new List<SchedulableItem*>();
00109     //std::string _someString = DEFAULT.someString;

```

```

00110      //unsigned int _someUInt = DEFAULT.someUInt;
00111  };
00112
00113  #endif /* SCHEDULE_H */

```

11.434 CppCompiler.h

```

00001  /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006  /*
00007  * File:      DynamicLinkedCode.h
00008  * Author:    rlcancian
00009  *
00010  * Created on 11 de janeiro de 2022, 22:24
00011  */
00012
00013  #ifndef CPPCOMPILER_H
00014  #define CPPCOMPILER_H
00015
00016  #ifndef OPEN_MAX
00017  #define OPEN_MAX 256
00018  #endif
00019
00020  #include <string>
00021
00022  #include <sys/types.h>
00023  #include <limits.h>
00024  #include <sys/stat.h>
00025  // #include <stdio.h>
00026  #include <unistd.h>
00027  // #include <errno.h>
00028  #include <paths.h>
00029
00030  #include "../kernel/simulator/model/ModelDataDefinition.h"
00031  #include "kernel/simulator/PluginInformation.h"
00032
00033
00034  class CompilerThread {
00035  public:
00036      void operator()();
00037  };
00038
00039  class CppCompiler : public ModelDataDefinition {
00040  public:
00041
00042      class CompilationResult {
00043      public:
00044          CompilationResult() { }
00045          bool success = false;
00046          std::string compilationStdOutput = "";
00047          std::string compilationErrOutput = "";
00048          std::string destinationPath = "";
00049          unsigned int lineNumber = 0;
00050      };
00051
00052  public:
00053      CppCompiler(Model* model, std::string name = "");
00054      virtual ~CppCompiler() = default;
00055      public: // static
00056          static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00057          static PluginInformation* GetPluginInformation();
00058          static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00059      public:
00060          virtual std::string show() override;
00061      public: // gets & sets
00062          void setSourceFilename(std::string sourceFilename);
00063          std::string getSourceFilename() const;
00064          bool IsLibraryLoaded() const;
00065          void setFlagsExecutable(std::string _flagsExecutable);
00066          std::string getFlagsExecutable() const;
00067          void setLibraryLoaded(bool libraryLoaded);
00068          void setFlagsStaticLibrary(std::string _flagsStaticLibrary);
00069          std::string getFlagsStaticLibrary() const;
00070          void setFlagsDynamicLibrary(std::string _flagsDynamicLibrary);
00071          std::string getFlagsDynamicLibrary() const;
00072          void setFlagsGeneral(std::string _flagsGeneral);
00073          std::string getFlagsGeneral() const;
00074          void setCompilerCommand(std::string _compilerCommand);
00075          std::string getCompilerCommand() const;
00076          void setOutputDir(std::string _outputDir);

```

```

00077     std::string getOutputDir() const;
00078     void setTempDir(std::string _tempDir);
00079     std::string getTempDir() const;
00080     void setOutputFilename(std::string _outputFilename);
00081     std::string getOutputFilename() const;
00082     void setObjectFiles(std::string _objectFiles);
00083     std::string getObjectFiles() const;
00084 public: // new methods
00085     CompilationResult compileToExecutable();
00086     CompilationResult compileToDynamicLibrary();
00087     CompilationResult compileToStaticLibrary();
00088     bool loadLibrary(std::string& errorMessage);
00089     bool unloadLibrary();
00090     void* getDynamicLibraryHandler() const;
00091
00092 protected: // must be overridden
00093     virtual bool _loadInstance(PersistenceRecord *fields) override;
00094     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00095 protected: // could be overridden
00096     virtual bool _check(std::string& errorMessage) override;
00097     // virtual ParserChangesInformation* _getParserChangesInformation();
00098     virtual void _initBetweenReplications() override;
00099     // virtual void _createInternalAndAttachedData() override;
00100 protected:
00101     CompilationResult _invokeCompiler(std::string command); //system(command.c_str());
00102     std::string _read(std::string filename);
00103
00104 protected:
00105     // virtual void _createInternalStatisticReporters() override;
00106     // virtual void _createNonEditableDataDefinitions() override;
00107     // virtual void _createEditableDataDefinitions() override;
00108     // virtual void _createAttachedAttributes() override;
00109
00110 private:
00111
00112     const struct DEFAULT_VALUES {
00113         const std::string sourceFilename = "";
00114         const std::string tempDir = ".temp/";
00115         const std::string outputDir = ".temp/";
00116         const std::string outputFilename = "";
00117         const std::string compiler = "g++";
00118         const std::string flagsGeneral = "-v -w -std=c++14";
00119         const std::string flagsDynamicLibrary = "-shared -fPIC";
00120         const std::string flagsStaticLibrary = "-fPIC";
00121         const std::string flagsExecutable = "";
00122         const std::string objectFiles = "";
00123     } DEFAULT;
00124     std::string _sourceFilename = DEFAULT.sourceFilename;
00125     std::string _outputFilename = DEFAULT.outputFilename;
00126     std::string _tempDir = DEFAULT.tempDir;
00127     std::string _outputDir = DEFAULT.outputDir;
00128     std::string _compilerCommand = DEFAULT.compiler;
00129     std::string _flagsGeneral = DEFAULT.flagsGeneral;
00130     std::string _flagsDynamicLibrary = DEFAULT.flagsDynamicLibrary;
00131     std::string _flagsStaticLibrary = DEFAULT.flagsStaticLibrary;
00132     std::string _flagsExecutable = DEFAULT.flagsExecutable;
00133     std::string _objectFiles = DEFAULT.objectFiles;
00134     //
00135     bool _libraryLoaded = false;
00136     bool _compiledToDynamicLibrary = false;
00137     void* _dynamicLibraryHandle = nullptr;
00138 };
00139
00140 #endif /* CPPCOMPILER_H */

```

11.435 OctaveRunner.h

```

00001 /*
00002  * File:   OctaveRunner.h
00003  * Author: rlcancian
00004  *
00005  * Octave integration data definition.
00006  */
00007
00008 #ifndef OCTAVERUNNER_H
00009 #define OCTAVERUNNER_H
00010
00011 #include <string>
00012
00013 #include "../kernel/simulator/model/ModelDataDefinition.h"
00014 #include "kernel/simulator/PluginInformation.h"
00015
00023 class OctaveRunner : public ModelDataDefinition {

```

```

00024 public:
00026     OctaveRunner(Model* model, std::string name = "");
00028     virtual ~OctaveRunner() = default;
00029
00030 public:
00032     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00034     static PluginInformation* GetPluginInformation();
00036     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00037
00038 public:
00040     void setOctaveExecutable(std::string octaveExecutable);
00042     std::string getOctaveExecutable() const;
00044     void setWorkingDirectory(std::string workingDirectory);
00046     std::string getWorkingDirectory() const;
00048     void setPreludeScript(std::string preludeScript);
00050     std::string getPreludeScript() const;
00052     void setCommand(std::string command);
00054     std::string getCommand() const;
00056     void setLastStatus(std::string lastStatus);
00058     std::string getLastStatus() const;
00060     void setLastExitCode(int lastExitCode);
00062     int getLastExitCode() const;
00064     void setLastStdout(std::string lastStdout);
00066     std::string getLastStdout() const;
00068     void setLastStderr(std::string lastStderr);
00070     std::string getLastStderr() const;
00072     void setLastResponsePayload(std::string lastResponsePayload);
00074     std::string getLastResponsePayload() const;
00076     void setLastScriptFilename(std::string lastScriptFilename);
00078     std::string getLastScriptFilename() const;
00080     void setLastResponseFilename(std::string lastResponseFilename);
00082     std::string getLastResponseFilename() const;
00083
00085     bool executeCommand(std::string& errorMessage);
00086
00088     virtual std::string show() override;
00089
00090 protected:
00092     virtual bool _loadInstance(PersistenceRecord *fields) override;
00094     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00096     virtual bool _check(std::string& errorMessage) override;
00098     // virtual void _createInternalAndAttachedData() override;
00099
00100
00101 protected:
00102     // virtual void _createInternalStatisticReporters() override;
00103     // virtual void _createNonEditableDataDefinitions() override;
00104     // virtual void _createEditableDataDefinitions() override;
00105     // virtual void _createAttachedAttributes() override;
00106
00107 private:
00108     const struct DEFAULT_VALUES {
00109         std::string octaveExecutable = "Ocavescript";
00110         std::string workingDirectory = "";
00111         std::string preludeScript = "";
00112         std::string command = "";
00113         std::string lastStatus = "Idle";
00114         int lastExitCode = -1;
00115         std::string lastStdout = "";
00116         std::string lastStderr = "";
00117         std::string lastResponsePayload = "";
00118         std::string lastScriptFilename = "";
00119         std::string lastResponseFilename = "";
00120     } DEFAULT;
00121
00122     std::string _octaveExecutable = DEFAULT.octaveExecutable;
00123     std::string _workingDirectory = DEFAULT.workingDirectory;
00124     std::string _preludeScript = DEFAULT.preludeScript;
00125     std::string _command = DEFAULT.command;
00126     std::string _lastStatus = DEFAULT.lastStatus;
00127     int _lastExitCode = DEFAULT.lastExitCode;
00128     std::string _lastStdout = DEFAULT.lastStdout;
00129     std::string _lastStderr = DEFAULT.lastStderr;
00130     std::string _lastResponsePayload = DEFAULT.lastResponsePayload;
00131     std::string _lastScriptFilename = DEFAULT.lastScriptFilename;
00132     std::string _lastResponseFilename = DEFAULT.lastResponseFilename;
00133 };
00134
00135 #endif /* OCTAVERUNNER_H */

```

11.436 PythonRuntime.h

```
00001 #ifndef PYTHONRUNTIME_H
```

```

00002 #define PYTHONRUNTIME_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelDataDefinition.h"
00007 #include "kernel/simulator/PluginInformation.h"
00008
00009 class Entity;
00010 class ModelComponent;
00011
00012 class PythonRuntime : public ModelDataDefinition {
00013 public:
00014     PythonRuntime(Model* model, std::string name = "");
00015     virtual ~PythonRuntime() override;
00016
00017 public:
00018     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00019     static PluginInformation* GetPluginInformation();
00020     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00021
00022 public:
00023     void setPythonExecutable(std::string pythonExecutable);
00024     std::string getPythonExecutable() const;
00025     void setFacadeObjectName(std::string facadeObjectName);
00026     std::string getFacadeObjectName() const;
00027     void setCaptureOutput(bool captureOutput);
00028     bool isCaptureOutput() const;
00029     void setLastStatus(std::string lastStatus);
00030     std::string getLastStatus() const;
00031     void setLastErrorMessage(std::string lastErrorMessage);
00032     std::string getLastErrorMessage() const;
00033     void setLastStdout(std::string lastStdout);
00034     std::string getLastStdout() const;
00035     void setLastStderr(std::string lastStderr);
00036     std::string getLastStderr() const;
00037
00038     bool validateHooks(const std::string& initCode,
00039                       const std::string& onDispatchCode,
00040                       std::string& errorMessage) const;
00041     bool executeInitHook(ModelComponent* component,
00042                          const std::string& initCode,
00043                          const std::string& onDispatchCode,
00044                          std::string& errorMessage);
00045     bool executeOnDispatchHook(ModelComponent* component,
00046                                Entity* entity,
00047                                const std::string& initCode,
00048                                const std::string& onDispatchCode,
00049                                std::string& errorMessage);
00050     void resetExecutionState();
00051
00052     virtual std::string show() override;
00053
00054 protected:
00055     virtual bool _loadInstance(PersistenceRecord* fields) override;
00056     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00057     virtual bool _check(std::string& errorMessage) override;
00058
00059 private:
00060     bool _validateIdentifier(std::string& errorMessage) const;
00061     void _invalidatePreparedHooks();
00062     bool _prepareHooks(ModelComponent* component,
00063                        const std::string& initCode,
00064                        const std::string& onDispatchCode,
00065                        std::string& errorMessage);
00066
00067 private:
00068     const struct DEFAULT_VALUES {
00069         std::string pythonExecutable = "python3";
00070         std::string facadeObjectName = "simulator";
00071         bool captureOutput = true;
00072         std::string lastStatus = "Idle";
00073         std::string lastErrorMessage = "";
00074         std::string lastStdout = "";
00075         std::string lastStderr = "";
00076     } DEFAULT;
00077
00078     std::string _pythonExecutable = DEFAULT.pythonExecutable;
00079     std::string _facadeObjectName = DEFAULT.facadeObjectName;
00080     bool _captureOutput = DEFAULT.captureOutput;
00081     std::string _lastStatus = DEFAULT.lastStatus;
00082     std::string _lastErrorMessage = DEFAULT.lastErrorMessage;
00083     std::string _lastStdout = DEFAULT.lastStdout;
00084     std::string _lastStderr = DEFAULT.lastStderr;
00085     std::string _preparedSignature = "";
00086
00087 #if GENESYS_HAS_PYTHON_INTEGRATION
00088     void* _preparedGlobals = nullptr;

```



```

00089     void* _preparedInitFunction = nullptr;
00090     void* _preparedDispatchFunction = nullptr;
00091     void* _preparedFacadeObject = nullptr;
00092 #endif
00093 };
00094
00095 #endif /* PYTHONRUNTIME_H */

```

11.437 RSimulatorRunner.h

```

00001 /*
00002  * File:    RSimulatorRunner.h
00003  * Author:  GenESyS
00004  *
00005  * R integration data definition.
00006  */
00007
00008 #ifndef RSIMULATORRUNNER_H
00009 #define RSIMULATORRUNNER_H
00010
00011 #include <string>
00012
00013 #include "../kernel/simulator/model/ModelDataDefinition.h"
00014 #include "kernel/simulator/PluginInformation.h"
00015
00023 class RSimulatorRunner : public ModelDataDefinition {
00024 public:
00026     RSimulatorRunner(Model* model, std::string name = "");
00028     virtual ~RSimulatorRunner() = default;
00029
00030 public:
00032     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00034     static PluginInformation* GetPluginInformation();
00036     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00037
00038 public:
00040     void setRExecutable(std::string rExecutable);
00042     std::string getRExecutable() const;
00044     void setWorkingDirectory(std::string workingDirectory);
00046     std::string getWorkingDirectory() const;
00048     void setPreludeScript(std::string preludeScript);
00050     std::string getPreludeScript() const;
00052     void setCommand(std::string command);
00054     std::string getCommand() const;
00056     void setLastStatus(std::string lastStatus);
00058     std::string getLastStatus() const;
00060     void setLastExitCode(int lastExitCode);
00062     int getLastExitCode() const;
00064     void setLastStdout(std::string lastStdout);
00066     std::string getLastStdout() const;
00068     void setLastStderr(std::string lastStderr);
00070     std::string getLastStderr() const;
00072     void setLastResponsePayload(std::string lastResponsePayload);
00074     std::string getLastResponsePayload() const;
00076     void setLastScriptFilename(std::string lastScriptFilename);
00078     std::string getLastScriptFilename() const;
00080     void setLastResponseFilename(std::string lastResponseFilename);
00082     std::string getLastResponseFilename() const;
00083
00085     bool executeCommand(std::string& errorMessage);
00086
00088     virtual std::string show() override;
00089
00090 protected:
00092     virtual bool _loadInstance(PersistenceRecord *fields) override;
00094     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00096     virtual bool _check(std::string& errorMessage) override;
00098     // virtual void _createInternalAndAttachedData() override;
00099
00100 protected:
00102     // virtual void _createInternalStatisticReporters() override;
00103     // virtual void _createNonEditableDataDefinitions() override;
00104     // virtual void _createEditableDataDefinitions() override;
00105     // virtual void _createAttachedAttributes() override;
00106
00107 private:
00108     const struct DEFAULT_VALUES {
00109         std::string rExecutable = "Rscript";
00110         std::string workingDirectory = "";
00111         std::string preludeScript = "";
00112         std::string command = "";
00113         std::string lastStatus = "Idle";

```

```

00114         int lastExitCode = -1;
00115         std::string lastStdout = "";
00116         std::string lastStderr = "";
00117         std::string lastResponsePayload = "";
00118         std::string lastScriptFilename = "";
00119         std::string lastResponseFilename = "";
00120     } DEFAULT;
00121
00122     std::string _rExecutable = DEFAULT.rExecutable;
00123     std::string _workingDirectory = DEFAULT.workingDirectory;
00124     std::string _preludeScript = DEFAULT.preludeScript;
00125     std::string _command = DEFAULT.command;
00126     std::string _lastStatus = DEFAULT.lastStatus;
00127     int _lastExitCode = DEFAULT.lastExitCode;
00128     std::string _lastStdout = DEFAULT.lastStdout;
00129     std::string _lastStderr = DEFAULT.lastStderr;
00130     std::string _lastResponsePayload = DEFAULT.lastResponsePayload;
00131     std::string _lastScriptFilename = DEFAULT.lastScriptFilename;
00132     std::string _lastResponseFilename = DEFAULT.lastResponseFilename;
00133 };
00134
00135 #endif /* RSIMULATORRUNNER_H */

```

11.438 SPICERunner.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File:    SPICERunner.h
00008  * Author:  bnmfw & joaomai
00009  *
00010  * Created on 10 de outubro de 2023, 23:00
00011  */
00012
00013 #ifndef SPICERUNNER_H
00014 #define SPICERUNNER_H
00015
00016 #include <iostream>
00017 #include <fstream>
00018 #include <sstream>
00019 #include <string>
00020 #include <vector>
00021 #include <map>
00022 #include <set>
00023 #include <algorithm>
00024 #include <functional>
00025
00026 #include "../kernel/simulator/model/ModelDataDefinition.h"
00027 #include "kernel/simulator/PluginInformation.h"
00028
00029 const std::vector<std::pair<std::string, float>>
    spice_unit_map{{"T",1e12},{ "G",1e9},{ "Meg",1e6},{ "K",1e3},{ "",1e0},{ "m",1e-3},{ "u",1e-6},{ "n",1e-9},{ "p",1e-12},{ "f",1e-15}};
00030 inline std::string uc(double value){
00031     float precision = 0.9999;
00032     for (auto element: spice_unit_map){
00033         if (value/element.second < precision) continue;
00034         return std::to_string(value/element.second) + element.first + " ";
00035     }
00036     return std::to_string(value/1e-18) + "a ";
00037 };
00038
00039 class SPICERunner : public ModelDataDefinition {
00040 public:
00041     SPICERunner(Model* model, std::string name = "");
00042     virtual ~SPICERunner();
00043
00044 public:
00045     std::string CompileSpiceFile();
00046     void SendComponent(std::string *instance, std::string subcircuit = "", std::string model = "");
00047     void SendCallback(std::function<void()> callback);
00048     void ConfigSim(double duration, double step);
00049     void Run();
00050     void PlotV(std::string net);
00051     template<typename... Args> void PlotV(std::string net, Args... args);
00052     void PlotVRelative(std::string comparison_base, std::string net);
00053     template<typename... Args> void PlotVRelative(std::string comparison_base, std::string net,
        Args... args);
00054     void PlotI(std::string net);
00055     template<typename... Args> void PlotI(std::string net, Args... args);
00056     void PlotIRelative(std::string comparison_base, std::string net);

```

```

00057     template<typename... Args> void PlotIRelative(std::string comparison_base, std::string net,
Args... args);
00058     double* MeasurePeak(std::string label, std::string peak, std::string quantity, std::string node,
float start, float finish);
00059     double* MeasureTrigTarg(std::string label, std::string trig, float trig_value, std::string
trig_inclin, std::string targ, float targ_value, std::string targ_inclin);
00060     void ParseOutput();
00061     void setRunnerCommand(std::string runnerCommand);
00062     std::string getRunnerCommand() const;
00063     void setModelsPath(std::string modelsPath);
00064     std::string getModelsPath() const;
00065     void setWorkingInputFilename(std::string workingInputFilename);
00066     std::string getWorkingInputFilename() const;
00067     void setWorkingOutputFilename(std::string workingOutputFilename);
00068     std::string getWorkingOutputFilename() const;
00069     void setWorkingDirectory(std::string workingDirectory);
00070     std::string getWorkingDirectory() const;
00071     std::string getLastRunCommand() const;
00072
00073 public:
00074     virtual std::string show() override;
00075
00076 public:
00077     static PluginInformation* GetPluginInformation();
00078     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00079     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00080     // Added
00081     static std::vector<std::string> split(std::string word, char split_char);
00082
00083 protected:
00084     virtual bool _loadInstance(PersistenceRecord *fields) override;
00085     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00086
00087 protected:
00088     virtual bool _check(std::string& errorMessage) override;
00089     // virtual ParserChangesInformation* _getParserChangesInformation();
00090     // virtual void _initBetweenReplications();
00091     // virtual void _createInternalStatisticReporters() override;
00092     // virtual void _createNonEditableDataDefinitions() override;
00093     // virtual void _createEditableDataDefinitions() override;
00094     // virtual void _createAttachedAttributes() override;
00095     // virtual void _createInternalAndAttachedData(); /*< A ModelDataDefinition or ModelComponent that
includes (internal) ou refers to (attach) other ModelDataDefinition must register them inside this
method. */
00101     // virtual void _addSimulationControl(SimulationControl* property);
00102
00103
00104 protected:
00105
00106 private:
00107     // ...
00108
00109 private:
00110     const struct DEFAULT_VALUES {
00111         const std::string runnerCommand = "ngspice";
00112         const std::string modelsPath = "./";
00113         const std::string workingInputFilename = "input.cir";
00114         const std::string workingOutputFilename = "output";
00115         const std::string workingDirectory = "";
00116     } DEFAULT;
00117     std::string _runnerCommand = DEFAULT.runnerCommand;
00118     std::string _modelsPath = DEFAULT.modelsPath;
00119     std::string _workingInputFilename = DEFAULT.workingInputFilename;
00120     std::string _workingOutputFilename = DEFAULT.workingOutputFilename;
00121     std::string _workingDirectory = DEFAULT.workingDirectory;
00122     std::string _lastRunCommand = "";
00123
00124     std::set<std::string> subcircuits;
00125     std::set<std::string> models;
00126
00127     std::vector<std::function<void()>> subscribers;
00128     std::vector<std::string*> instances;
00129
00130     // Plot
00131     std::vector<std::string> vplots;
00132     std::vector<std::string> iplots;
00133
00134     // Measure
00135     std::vector<std::string> measures;
00136     std::map<std::string, double*> promises;
00137
00138     std::string config;
00139
00140 private:
00141     // DummyElement* _internalDataDefinition = nullptr;
00142
00143 private:

```

```

00144     // ...
00145 };
00146
00147 #endif /* SPICERUNNER_H */

```

11.439 EntityGroup.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   Group.h
00009  * Author: rlcancian
00010  *
00011  * Created on 12 de Junho de 2019, 19:00
00012  */
00013
00014 #ifndef ENTITYGROUP_H
00015 #define ENTITYGROUP_H
00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "../kernel/simulator/essentialPlugins/Entity.h"
00019 #include "kernel/util/List.h"
00020 #include <list>
00021 #include <map>
00022
00023 /*
00024  The EntityGroup represents a map of entities grouped together somehow.
00025  * The entities in this group are kept into a internal queue.
00026  * Usually one entity representing all grouped entities is generated by ModelComponents and the
00027  EntityGroup it represented is accessed through an attribute.
00028  * The Key of this map is the ID of the representative entity
00029  */
00029 class EntityGroup : public ModelDataDefinition {
00030 public:
00031     EntityGroup(Model* model, std::string name = "");
00032     virtual ~EntityGroup();
00033 public:
00034     virtual std::string show() override;
00035 public:
00036     static PluginInformation* GetPluginInformation();
00037     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00038     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00039 public:
00040     void insertElement(unsigned int idKey, Entity* modeldatum);
00041     void removeElement(unsigned int idKey, Entity* modeldatum);
00042     List<Entity*>* getGroup(unsigned int idKey);
00043 protected:
00044     virtual bool _loadInstance(PersistenceRecord *fields) override;
00045     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00046     virtual bool _check(std::string& errorMessage) override;
00047     virtual void _initBetweenReplications() override;
00048     // virtual void _createInternalAndAttachedData() override;
00049
00050 protected:
00051     virtual void _createInternalStatisticReporters() override;
00052     // virtual void _createNonEditableDataDefinitions() override;
00053     // virtual void _createEditableDataDefinitions() override;
00054     virtual void _createAttachedAttributes() override;
00055
00056 private:
00057     void _clearGroups();
00058     void _initCStats();
00059 private: //!::n
00060     std::map<unsigned int, List<Entity*>*>* _groupMap = new std::map<unsigned int, List<Entity*>*>();
00061     std::list<List<Entity*>*>* _detachedEmptyGroups;
00062 private: // child inner modeldatum
00063     StatisticsCollector* _cstatNumberInGroup = nullptr;
00064 };
00065
00066 #endif /* ENTITYGROUP_H */

```

11.440 File.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.

```

```

00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007  /*
00008  * File:   File.h
00009  * Author: rlcancian
00010  *
00011  * Created on 20 de Fileembro de 2019, 20:07
00012  */
00013
00014  #ifndef FILE_H
00015  #define FILE_H
00016
00017
00018  #include "../kernel/simulator/model/ModelDataDefinition.h"
00019  #include "../kernel/simulator/model/ModelDataManager.h"
00020  // #include "ParserChangesInformation.h"
00021  #include "kernel/simulator/PluginInformation.h"
00022
00064  class File : public ModelDataDefinition {
00065  public:
00066      enum class AccessMode : unsigned int {
00067          Read = 0,
00068          Write = 1,
00069          Append = 2,
00070          ReadWrite = 3
00071      };
00072
00073      File(Model* model, std::string name = "");
00074      virtual ~File() = default;
00075  public: // static
00076      static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00077      static PluginInformation* GetPluginInformation();
00078      static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00079  public:
00080      void setSystemFilename(std::string systemFilename);
00081      std::string getSystemFilename() const;
00082      std::string getFilenameOnly() const;
00083      std::string getPathOnly() const;
00084
00085      void setAccessMode(AccessMode accessMode);
00086      AccessMode getAccessMode() const;
00087      void setAccessModeAsString(std::string accessMode);
00088      std::string getAccessModeAsString() const;
00089
00090      virtual std::string show() override;
00091
00092  protected: // must be overridden
00093      virtual bool _loadInstance(PersistenceRecord* fields) override;
00094      virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00095  protected: // could be overridden
00096      virtual bool _check(std::string& errorMessage) override;
00097      virtual ParserChangesInformation* _getParserChangesInformation() override;
00098
00099
00100  protected:
00101      // virtual void _createInternalStatisticReporters() override;
00102      // virtual void _createNonEditableDataDefinitions() override;
00103      // virtual void _createEditableDataDefinitions() override;
00104      // virtual void _createAttachedAttributes() override;
00105
00106  private:
00107      static std::string _accessModeToString(AccessMode accessMode);
00108      static bool _stringToAccessMode(const std::string& accessMode, AccessMode* parsedAccessMode);
00109      static std::string _normalizePathSeparators(const std::string& filename);
00110
00111  private:
00112      const struct DEFAULT_VALUES {
00113          std::string systemFilename = "";
00114          AccessMode accessMode = AccessMode::Read;
00115      } DEFAULT;
00116      std::string _systemFilename = DEFAULT.systemFilename;
00117      AccessMode _accessMode = DEFAULT.accessMode;
00118      bool _accessModeWasInvalid = false;
00119  };
00120
00121  #endif /* FILE_H */

```

11.441 AssignmentItem.h

```

00001  /*
00002  * The MIT License

```

```

00003  *
00004  * Copyright 2022 rlcancian.
00005  *
00006  * Permission is hereby granted, free of charge, to any person obtaining a copy
00007  * of this software and associated documentation files (the "Software"), to deal
00008  * in the Software without restriction, including without limitation the rights
00009  * to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00010  * copies of the Software, and to permit persons to whom the Software is
00011  * furnished to do so, subject to the following conditions:
00012  *
00013  * The above copyright notice and this permission notice shall be included in
00014  * all copies or substantial portions of the Software.
00015  *
00016  * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017  * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018  * FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00019  * AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00020  * LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00021  * OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
00022  * THE SOFTWARE.
00023  */
00024
00025 /*
00026  * File: AssignmentItem.h
00027  * Author: Prof. Rafael Luiz Cancian, Dr. Eng.
00028  *
00029  * Created on 2 de fevereiro de 2022, 14:12
00030  */
00031
00032 #ifndef ASSIGNMENTITEM_H
00033 #define ASSIGNMENTITEM_H
00034
00035 #include <functional>
00036 #include "../kernel/simulator/model/Model.h"
00037
00041 class Assignment {
00042 public:
00043     Assignment(Model* model, std::string destination, std::string expression="1", bool
isAttributeNotVariable = true);
00044     Assignment(std::string destination, std::string expression, bool isAttributeNotVariable = true);
00045     virtual ~Assignment();
00046     void setDestination(std::string _destination);
00047     std::string getDestination() const;
00048     void setExpression(std::string _expression);
00049     std::string getExpression() const;
00050     void setAttributeNotVariable(bool isAttributeNotVariable);
00051     bool isAttributeNotVariable() const;
00052     void setChangeCallback(std::function<void(Assignment*)> callback);
00053
00054     // TODO: don't have name
00055     std::string getName() const;
00056
00057     List<SimulationControl*>* getSimulationControls() const;
00058     void ensureSimulationControls(Model* model);
00059     void _addSimulationControl(SimulationControl* control);
00060
00061     std::string getTypeDC() {return _typeDC;};
00062 public:
00063     bool loadInstance(PersistenceRecord *fields, unsigned int parentIndex);
00064     void saveInstance(PersistenceRecord *fields, unsigned int parentIndex, bool saveDefault);
00065 private:
00066     void _updateTypeDC();
00067     std::string _destination = "";
00068     std::string _expression = "";
00069     bool _isAttributeNotVariable = true;
00070     std::string _typeDC;
00071     std::function<void(Assignment*)> _changeCallback = nullptr;
00072
00073     List<SimulationControl*>* _simulationControls = new List<SimulationControl*>();
00074 };
00075
00076
00077 #endif /* ASSIGNMENTITEM_H */

```

11.442 Formula.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*

```

```

00008  * File:    Formula.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 20 de Junho de 2019, 00:56
00012  */
00013
00014 #ifndef FORMULA_H
00015 #define FORMULA_H
00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "../kernel/simulator/model/ModelDataManager.h"
00019 #include "kernel/simulator/Plugin.h"
00020 // #include "kernel/simulator/Parser_if.h"
00021
00022 class Formula : public ModelDataDefinition {
00023 public:
00024     Formula(Model* model, std::string name = "");
00025     virtual ~Formula() override;
00026 public: // virtual
00027     virtual std::string show() override;
00028 public:
00029     unsigned int size();
00030     void setExpression(std::string formulaExpression, std::string index = "");
00031     // void setExpression(std::string formulaExpression);
00032     std::string getExpression(std::string index = "");
00033     // std::string getExpression();
00034     // double getValue();
00035     double getValue(std::string index);
00036 public: // statics
00037     static PluginInformation* GetPluginInformation();
00038     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00039     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00040 protected: // must be overridden by derived classes
00041     virtual bool _loadInstance(PersistenceRecord *fields) override;
00042     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00043 protected: // could be overridden by derived classes
00044     virtual bool _check(std::string& errorMessage) override;
00045     // virtual void _addSimulationControl(SimulationControl* property);
00046     // virtual ParserChangesInformation* _getParserChangesInformation();
00047     // virtual void _initBetweenReplications();
00048     // virtual void _createInternalStatisticReporters() override;
00049     // virtual void _createNonEditableDataDefinitions() override;
00050     // virtual void _createEditableDataDefinitions() override;
00051     virtual void _createAttachedAttributes() override;
00052
00053 private:
00054     std::map<std::string, std::string*> _formulaExpressions = new std::map<std::string,
00055     std::string*>(); // map<index, formula>
00056 private:
00057 };
00058
00059 #endif /* FORMULA_H */

```

11.443 Label.h

```

00001  /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006  /*
00007  * File:    Label.h
00008  * Author:  rlcancian
00009  *
00010  * Created on 15 de janeiro de 2022, 10:13
00011  */
00012
00013 #ifndef LABEL_H
00014 #define LABEL_H
00015
00016 #include "../kernel/simulator/model/ModelDataDefinition.h"
00017 #include "../kernel/simulator/essentialPlugins/Entity.h"
00018
00019 class Label : public ModelDataDefinition {
00020 public:
00021     Label(Model* model, std::string name = "");
00022     virtual ~Label() = default;
00023 public: // static
00024     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00025     static PluginInformation* GetPluginInformation();
00026     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00027 public:
00028     virtual std::string show() override;
00029     void setLabel(std::string _label);

```

```

00029     std::string getLabel() const;
00030     void setEnterIntoLabelComponent(ModelComponent* enteringLabelComponent);
00031     ModelComponent* getEnterIntoLabelComponent() const;
00032     void sendEntityToLabelComponent(Entity* entity, double timeDelay);
00033 protected: // must be overridden
00034     virtual bool _loadInstance(PersistenceRecord *fields) override;
00035     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00036 protected: // could be overridden
00037     virtual bool _check(std::string& errorMessage) override;
00038     //virtual ParserChangesInformation* _getParserChangesInformation();
00039     //virtual void _initBetweenReplications();
00040     // virtual void _createInternalStatisticReporters() override;
00041     // virtual void _createNonEditableDataDefinitions() override;
00042     // virtual void _createEditableDataDefinitions() override;
00043     // virtual void _createAttachedAttributes() override;
00044     //virtual void _createInternalAndAttachedData();
00045
00046 protected:
00047
00048 private:
00049     std::string _label;
00050     ModelComponent* _enteringLabelComponent = nullptr;
00051 };
00052
00053 #endif /* LABEL_H */

```

11.444 Set.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Set.h
00009  * Author: rlcancian
00010  *
00011  * Created on 03 de Junho de 2019, 15:11
00012  */
00013
00014 #ifndef SET_H
00015 #define SET_H
00016
00017 #include <vector>
00018
00019 #include "../kernel/simulator/model/ModelDataDefinition.h"
00020 #include "../kernel/simulator/model/ModelDataManager.h"
00021 #include "kernel/simulator/ParserChangesInformation.h"
00022 #include "kernel/simulator/PluginInformation.h"
00023 #include "../kernel/simulator/essentialPlugins/EntityType.h"
00024
00058 class Set : public ModelDataDefinition {
00059 public:
00060     Set(Model* model, std::string name = "", std::vector<std::string> allowedElementTypes = {});
00061     virtual ~Set() override;
00062 public: // static
00063     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00064     static PluginInformation* GetPluginInformation();
00065     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00066 public:
00067     virtual std::string show() override;
00068 public:
00069     //void setSetOfType(std::string _setOfType);
00070     std::string getSetOfType() const;
00076     bool setSetOfType(const std::string& setOfType);
00078     bool canChangeSetOfType() const;
00085     void setAllowedElementTypes(const std::vector<std::string>& allowedElementTypes);
00087     void addAllowedElementType(const std::string& allowedElementType);
00089     std::vector<std::string> getAllowedElementTypes() const;
00090     List<ModelDataDefinition*> getElementSet() const;
00091
00092     void addElementSet(ModelDataDefinition* newElement);
00093     void removeElementSet(ModelDataDefinition* element);
00100     ModelDataDefinition* createElementSetOfType(const std::string& typeName, const std::string& name =
00101     "");
00102 protected: // must be overridden
00103     virtual bool _loadInstance(PersistenceRecord *fields) override;
00104     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00105 protected: // could be overridden
00106     virtual bool _check(std::string& errorMessage) override;
00107     virtual ParserChangesInformation* _getParserChangesInformation() override;

```



```

00108 protected:
00109     // virtual void _createInternalStatisticReporters() override;
00110     // virtual void _createNonEditableDataDefinitions() override;
00111     // virtual void _createEditableDataDefinitions() override;
00112     virtual void _createAttachedAttributes() override;
00113
00114 private:
00115     //ElementManager* _elems;
00116
00117     const struct DEFAULT_VALUES {
00118         const unsigned int membersSize = 0;
00119         const std::string setOfType = "";
00120     } DEFAULT;
00121     List<ModelDataDefinition*>* _elementSet = new List<ModelDataDefinition*>();
00122     std::string _setOfType = DEFAULT.setOfType;
00123     std::vector<std::string> _allowedElementTypes;
00124 };
00125
00126 #endif /* SET_H */

```

11.445 Variable.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Variable.h
00009  * Author: rafael.luiz.cancian
00010  *
00011  * Created on 4 de Setembro de 2018, 18:28
00012  */
00013
00014 #ifndef VARIABLE_H
00015 #define VARIABLE_H
00016
00017 #include "../kernel/simulator/essentialPlugins/Attribute.h"
00018 #include "../kernel/simulator/model/ModelDataManager.h"
00019 #include "kernel/simulator/Plugin.h"
00020 #include "../kernel/simulator/essentialPlugins/SparseValueStore.h"
00021
00091 class Variable : public Attribute {
00092 public:
00093     Variable(Model* model, std::string name = "");
00094     virtual ~Variable() override;
00095 public:
00096     virtual std::string show() override;
00097 public: //static
00098     static PluginInformation* GetPluginInformation();
00099     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00100     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00101 public:
00102     double getValue(std::string index="");
00103     void setValue(double value, std::string index="");
00104     void copyValuesFrom(const Variable& source);
00105     std::string getInitialValuesText() const override;
00106     void setInitialValuesText(std::string valuesText) override;
00107     void insertDimentionSize(unsigned int size);
00108     std::map<std::string, double> *getValues() const;
00109     SparseValueStore* getValueStore();
00110     ModelDataDefinition* get_scope();
00111     void set_scope(ModelDataDefinition* const scope);
00112
00121 protected:
00122     virtual bool _loadInstance(PersistenceRecord *fields) override;
00123     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00124     virtual bool _check(std::string& errorMessage) override;
00125     virtual void _initBetweenReplications() override;
00126
00127
00128 protected:
00129     // virtual void _createInternalStatisticReporters() override;
00130     // virtual void _createNonEditableDataDefinitions() override;
00131     // virtual void _createEditableDataDefinitions() override;
00132     // virtual void _createAttachedAttributes() override;
00133
00134 private:
00135     //const struct DEFAULT_VALUES { } DEFAULT;
00136     SparseValueStore* _values = new SparseValueStore();
00137     ModelDataDefinition* scope = nullptr; // not used so far
00138 };

```

```

00139
00140 #endif /* VARIABLE_H */

```

11.446 Sequence.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:    Sequence.h
00009  * Author:  rlcancian
00010  *
00011  * Created on 03 de Junho de 2019, 15:12
00012  */
00013
00014 #ifndef SEQUENCE_H
00015 #define SEQUENCE_H
00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "../kernel/simulator/model/ModelDataManager.h"
00019 #include "kernel/simulator/PluginInformation.h"
00020 #include "Station.h"
00021 #include "../Logic/Label.h"
00022 #include "../Logic/AssignmentItem.h"
00023
00024 class SequenceStep : public PersistentObject_base {
00025 public:
00026     SequenceStep(Station* station, std::list<Assignment*>* assignments = nullptr);
00027     SequenceStep(Label* label, std::list<Assignment*>* assignments = nullptr);
00028     SequenceStep(Model* model, std::string stationOrLabelName, bool isStation = true,
00029                 std::list<Assignment*>* assignments = nullptr);
00029     virtual ~SequenceStep() override;
00030 public: // virtual
00031
00032     virtual bool _loadInstance(PersistenceRecord *fields, unsigned int parentIndex);
00033     virtual void _saveInstance(PersistenceRecord *fields, unsigned int parentIndex, bool
00034                               saveDefaultValues);
00034     virtual bool _loadInstance(PersistenceRecord *fields) override;
00035     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00036
00037 public:
00038
00039     std::list<Assignment*>* getAssignments() const;
00040     void setStation(Station* _station);
00041     Station* getStation() const;
00042     void setElementManager(ModelDataManager* _modeldataManager);
00043     void setLabel(Label* _label);
00044     Label* getLabel() const;
00045
00046 protected:
00047 private:
00048
00049     const struct DEFAULT_VALUES {
00050         const unsigned int assignmentsSize = 0;
00051     } DEFAULT;
00052     Station* _station = nullptr;
00053     Label* _label = nullptr;
00054     std::list<Assignment*>* _assignments = new std::list<Assignment*>();
00055 private:
00056     ModelDataManager* _modeldataManager = nullptr;
00057 };
00058
00059 class Sequence : public ModelDataDefinition {
00060 public:
00061
00062     Sequence(Model* model, std::string name = "");
00063     virtual ~Sequence() override;
00064 public:
00065     virtual std::string show() override;
00066 public: // static
00067     static PluginInformation* GetPluginInformation();
00068     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00069     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00070 public:
00071     List<SequenceStep*>* getSteps() const;
00072 protected:
00073     virtual bool _loadInstance(PersistenceRecord *fields) override;
00074     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;

```

```

00102     virtual bool _check(std::string& errorMessage) override;
00103 protected:
00104     // virtual void _createInternalStatisticReporters() override;
00105     // virtual void _createNonEditableDataDefinitions() override;
00106     // virtual void _createEditableDataDefinitions() override;
00107     virtual void _createAttachedAttributes() override;
00108 private:
00109     List<SequenceStep*>* _steps = new List<SequenceStep*>();
00110 };
00111
00112 #endif /* SEQUENCE_H */

```

11.447 Station.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:      Station.h
00009  * Author:    rlcancian
00010  *
00011  * Created on 03 de Junho de 2019, 15:12
00012  */
00013
00014 #ifndef STATION_H
00015 #define STATION_H
00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "../kernel/simulator/model/ModelDataManager.h"
00019 #include "kernel/simulator/Plugin.h"
00020 #include "../kernel/simulator/essentialPlugins/Entity.h"
00021
00022 /*
00023  Station module
00024  DESCRIPTION
00025  The Station module defines a station (or a set of stations) corresponding to a physical
00026  or logical location where processing occurs. If the Station module defines a station
00027  set, it is effectively defining multiple processing locations.
00028  The station (or each station within the defined set) has a matching Activity Area that
00029  is used to report all times and costs accrued by the entities in this station. This
00030  Activity Area's name is the same as the station. If a parent Activity Area is defined,
00031  then it also accrues any times and costs by the entities in this station.
00032  TYPICAL USES
00033  * Defining a lathe area
00034  * Defining a set of toll booths
00035  * Defining a food preparation area
00036  PROMPTS
00037  Prompt Description
00038  Name Unique name of the module that will be displayed in the
00039  flowchart.
00040  Station Type Type of station being defined, either as an individual Station or a
00041  station Set.
00042  Station Name Name of the individual station.
00043  Set Name Name of the station set.
00044  Parent Activity Area Name of the Activity Area's parent.
00045  Associated Intersection Name of the intersection associated with this station in a guided
00046  transporter network.
00047  Report Statistics Specifies whether or not statistics will automatically be collected
00048  and stored in the report database for this station and its
00049  corresponding activity area.
00050  Save Attribute Attribute name used to store the index number into the station set
00051  of the member that is selected.
00052  Station Set Members Names of the stations that are members of this station set.
00053  Station Name A given station can only exist once within a model. Therefore, an
00054  individual station can only be the member of one station set, and
00055  that individual station may not be the name of a station in
00056  another module.
00057  Parent Activity Area Name of the Activity Area's parent for the station set member.
00058  Associated Intersection Name of the intersection associated with this station set in a
00059  guided transporter network.
00060  Report Statistics Specifies whether or not statistics will automatically be collected
00061  and stored in the report database for this station set member and
00062  its corresponding activity area.
00063  */
00064 class Station : public ModelDataDefinition {
00065 public:
00066     Station(Model* model, std::string name = "");
00067     virtual ~Station();
00068 public:
00069     virtual std::string show() override;

```

```

00070 public: // static
00071     static PluginInformation* GetPluginInformation();
00072     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00073     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00074 public:
00075     void initBetweenReplications();
00076     void enter(Entity* entity);
00077     void leave(Entity* entity);
00078     void setEnterIntoStationComponent(ModelComponent* _enterIntoStationComponent);
00079     ModelComponent* getEnterIntoStationComponent() const;
00080 protected:
00081     virtual bool _loadInstance(PersistenceRecord *fields) override;
00082     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00083     virtual bool _check(std::string& errorMessage) override;
00084     // virtual void _createInternalAndAttachedData() override;
00085     virtual void _initBetweenReplications() override;
00086
00087 protected:
00088     virtual void _createInternalStatisticReporters() override;
00089     // virtual void _createNonEditableDataDefinitions() override;
00090     // virtual void _createEditableDataDefinitions() override;
00091     virtual void _createAttachedAttributes() override;
00092
00093 private:
00094     void _initCStats();
00095     unsigned int _numberInStation = 0;
00096     ModelComponent* _enterIntoStationComponent = nullptr;
00097 private: // inner elements
00098     StatisticsCollector* _cstatNumberInStation = nullptr;
00099     StatisticsCollector* _cstatTimeInStation = nullptr;
00100     friend class StationTestProbe;
00101 };
00102
00103 #endif /* STATION_H */

```

11.448 Storage.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File: Storage.h
00009  * Author: rlcancian
00010  *
00011  * Created on 20 de Storageembro de 2019, 20:06
00012  */
00013
00014 #ifndef STORAGE_H
00015 #define STORAGE_H
00016
00017 #include "../kernel/simulator/model/ModelDataDefinition.h"
00018 #include "../kernel/simulator/model/ModelDataManager.h"
00019 // #include "kernel/simulator/ParserChangesInformation.h"
00020 #include "kernel/simulator/PluginInformation.h"
00021
00022
00023 /*
00024 Storage module
00025 DESCRIPTION
00026 The Storage module defines the name of a storage. Storages are automatically created
00027 by any module that references the storage so that this module is seldom needed. The
00028 only time this module is needed is when a storage is defined as a member of a storage
00029 set or specified using an attribute or expression.
00030 TYPICAL USES
00031 * Defining an animate storage for a set of storages
00032 PROMPTS
00033 Prompt Description
00034 Name The name of the storage set being defined. This name must be
00035 unique.
00036 */
00037 class Storage : public ModelDataDefinition {
00038 public:
00039     Storage(Model* model, std::string name = "");
00040     virtual ~Storage() = default;
00041 public: // static
00042     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00043     static PluginInformation* GetPluginInformation();
00044     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00045 public:
00046     virtual std::string show() override;

```

```

00047     void setTotalArea(double _totalArea);
00048     double getTotalArea() const;
00049     void setCapacity(unsigned int _capacity);
00050     unsigned int getCapacity() const;
00051     void setUnitsPerArea(double _unitsPerArea);
00052     double getUnitsPerArea() const;
00053
00054 protected: // must be overridden
00055     virtual bool _loadInstance(PersistenceRecord *fields) override;
00056     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00057 protected: // could be overridden
00058     virtual bool _check(std::string& errorMessage) override;
00059     virtual ParserChangesInformation* _getParserChangesInformation() override;
00060
00061 protected:
00062     // virtual void _createInternalStatisticReporters() override;
00063     // virtual void _createNonEditableDataDefinitions() override;
00064     // virtual void _createEditableDataDefinitions() override;
00065     // virtual void _createAttachedAttributes() override;
00066
00067 private:
00068
00069     const struct DEFAULT_VALUES {
00070         const double totalArea = 1;
00071         const unsigned int capacity = 10;
00072         const double unitsPerArea = 1;
00073     } DEFAULT;
00074     double _totalArea = DEFAULT.totalArea;
00075     unsigned int _capacity = DEFAULT.capacity;
00076     double _unitsPerArea = DEFAULT.unitsPerArea;
00077
00078 private:
00079     // @TODO: Add statisticCollector for ProportionOfStorageUsage
00080 };
00081 #endif /* STORAGE_H */

```

11.449 SignalData.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File:     SignalData.h
00008  * Author:   rlcancian
00009  *
00010  * Created on 25 de maio de 2022, 14:37
00011  */
00012
00013 #ifndef SIGNALDATA_H
00014 #define SIGNALDATA_H
00015
00016 #include "../kernel/simulator/model/ModelDataDefinition.h"
00017 #include "../kernel/simulator/model/ModelComponent.h"
00018 #include "kernel/util/List.h"
00019
00020 class SignalData : public ModelDataDefinition {
00021 public:
00022     typedef std::function<unsigned int(SignalData*)> SignalDataEventHandler; //< Returns the total
    number of freed entities.
00023     typedef std::pair<SignalDataEventHandler, ModelComponent*> PairSignalDataEventHandler;
00024     template<typename Class>
00025     static SignalDataEventHandler SetSignalDataEventHandler(unsigned int
    (Class::*function)(SignalData*), Class * object) {
00026         return std::bind(function, object, std::placeholders::_1);
00027     }
00028 public:
00029     SignalData(Model* model, std::string name = "");
00030     virtual ~SignalData() override;
00031 public: // static
00032     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00033     static PluginInformation* GetPluginInformation();
00034     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00035 public: //virtual
00036     virtual std::string show() override;
00037 public:
00038     unsigned int generateSignal(double signalValue, unsigned int limit);
00039     void addSignalDataEventHandler(SignalDataEventHandler eventHandler, ModelComponent* component);
00040     void removeSignalDataEventHandler(ModelComponent* component);
00041     bool hasSignalDataEventHandler(ModelComponent* component) const;
00042     unsigned int remainsToLimit() const;
00043     void decreaseRemainLimit();

```

```

00044
00045 protected: // must be overridden
00046     virtual bool _loadInstance(PersistenceRecord *fields) override;
00047     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00048 protected: // could be overridden .
00049     virtual bool _check(std::string& errorMessage) override;
00050     virtual void _initBetweenReplications() override;
00051     //virtual void _createInternalAndAttachedData();
00052     //virtual ParserChangesInformation* _getParserChangesInformation();
00053
00054 protected:
00055     // virtual void _createInternalStatisticReporters() override;
00056     // virtual void _createNonEditableDataDefinitions() override;
00057     // virtual void _createEditableDataDefinitions() override;
00058     // virtual void _createAttachedAttributes() override;
00059
00060 private: // methods
00061     unsigned int _notifySignalDataEventHandlers();
00062 private: //1:1
00063     unsigned int _remainsToLimit = 0;
00064 private: //1:n
00065     List<PairSignalDataEventHandler*>* _signalDataEventHandlers = new
     List<PairSignalDataEventHandler*>();
00066 };
00067
00068 #endif /* SIGNALDATA_H */

```

11.450 DummyElement.h

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cppFiles/class.h to edit this template
00004  */
00005
00006 /*
00007  * File:    DummyElement.h
00008  * Author:  Prof. Rafael Luiz Cancian, Dr. Eng.
00009  *
00010  * Created on 11 de janeiro de 2022, 22:26
00011  */
00012
00013 #ifndef DUMMYELEMENT_H
00014 #define DUMMYELEMENT_H
00015
00016 #include "../kernel/simulator/model/ModelDataDefinition.h"
00017 #include "kernel/simulator/PluginInformation.h"
00018
00019 class DummyElement : public ModelDataDefinition {
00020 public:
00021     DummyElement(Model* model, std::string name = "");
00022     virtual ~DummyElement() = default;
00023
00024 public:
00025     void setSomeString(const std::string& someString);
00026     std::string getSomeString() const;
00027     void setSomeUint(unsigned int someUint);
00028     unsigned int getSomeUint() const;
00029
00030 public:
00031     virtual std::string show() override;
00032
00033 public:
00034     static PluginInformation* GetPluginInformation();
00035     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord *fields);
00036     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00037
00038 protected:
00039     virtual bool _loadInstance(PersistenceRecord *fields) override;
00040     virtual void _saveInstance(PersistenceRecord *fields, bool saveDefaultValues) override;
00041     virtual bool _check(std::string& errorMessage) override;
00042
00043 protected:
00044     // virtual bool _check(std::string& errorMessage);
00047     // virtual ParserChangesInformation* _getParserChangesInformation();
00049     // virtual void _initBetweenReplications();
00050     // virtual void _createInternalStatisticReporters() override;
00051     // virtual void _createNonEditableDataDefinitions() override;
00052     // virtual void _createEditableDataDefinitions() override;
00053     // virtual void _createAttachedAttributes() override;
00055     // virtual void _createInternalAndAttachedData(); /*< A ModelDataDefinition or ModelComponent that
     includes (internal) ou refers to (attach) other ModelDataDefinition must register them inside this
     method. */
00057     // virtual void _addSimulationControl(SimulationControl* property);

```

```

00058
00059
00060 protected:
00061
00062 private:
00063     // ...
00064
00065 private:
00066     const struct DEFAULT_VALUES {
00067         const std::string someString = "Test";
00068         const unsigned int someUInt = 1;
00069     } DEFAULT;
00070     std::string _someString = DEFAULT.someString;
00071     unsigned int _someUInt = DEFAULT.someUInt;
00072
00073 private:
00074     // DummyElement* _internalDataDefinition = nullptr;
00075
00076 private:
00077     // ...
00078 };
00079
00080 #endif /* DUMMYELEMENT_H */

```

11.451 BioCompartment.h

```

00001 #ifndef BIOCCompartment_H
00002 #define BIOCCompartment_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelDataDefinition.h"
00007 #include "kernel/simulator/PluginInformation.h"
00008
00025 class BioCompartment : public ModelDataDefinition {
00026 public:
00027     BioCompartment(Model* model, std::string name = "");
00028     virtual ~BioCompartment() override = default;
00029
00030 public:
00031     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00032     static PluginInformation* GetPluginInformation();
00033     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00034
00035 public:
00036     void setCompartmentType(std::string type);
00037     std::string getCompartmentType() const;
00038     void setParentCompartmentName(std::string parentName);
00039     std::string getParentCompartmentName() const;
00040     void setMembraneBounded(bool membraneBounded);
00041     bool isMembraneBounded() const;
00042     void setVolumeFraction(double volumeFraction);
00043     double getVolumeFraction() const;
00044     void setCopyNumber(unsigned int copyNumber);
00045     unsigned int getCopyNumber() const;
00046     void setRole(std::string role);
00047     std::string getRole() const;
00048
00049     virtual std::string show() override;
00050
00051 protected:
00052     virtual bool _loadInstance(PersistenceRecord* fields) override;
00053     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00054     virtual bool _check(std::string& errorMessage) override;
00055     virtual void _createEditableDataDefinitions() override;
00056
00057 private:
00058     const struct DEFAULT_VALUES {
00059         std::string compartmentType = "cytosol";
00060         std::string parentCompartmentName = "";
00061         bool membraneBounded = false;
00062         double volumeFraction = 1.0;
00063         unsigned int copyNumber = 1;
00064         std::string role = "generic";
00065     } DEFAULT;
00066
00067     std::string _compartmentType = DEFAULT.compartmentType;
00068     std::string _parentCompartmentName = DEFAULT.parentCompartmentName;
00069     bool _membraneBounded = DEFAULT.membraneBounded;
00070     double _volumeFraction = DEFAULT.volumeFraction;
00071     unsigned int _copyNumber = DEFAULT.copyNumber;
00072     std::string _role = DEFAULT.role;
00073 };

```

```

00074
00075 #endif /* BIOCOMPARTMENT_H */

```

11.452 MolecularSpecies.h

```

00001 #ifndef MOLECULARSPECIES_H
00002 #define MOLECULARSPECIES_H
00003
00004 #include <string>
00005
00006 #include "../kernel/simulator/model/ModelDataDefinition.h"
00007 #include "kernel/simulator/PluginInformation.h"
00008
00016 class MolecularSpecies : public ModelDataDefinition {
00017 public:
00018     MolecularSpecies(Model* model, std::string name = "");
00019     virtual ~MolecularSpecies() override = default;
00020
00021 public:
00022     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00023     static PluginInformation* GetPluginInformation();
00024     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00025
00026 public:
00027     void setInitialCount(int initialCount);
00028     int getInitialCount() const;
00029     void setCount(int count);
00030     int getCount() const;
00031     void setCompartment(std::string compartment);
00032     std::string getCompartment() const;
00033     void setUnit(std::string unit);
00034     std::string getUnit() const;
00035
00036     virtual std::string show() override;
00037
00038 protected:
00039     virtual bool _loadInstance(PersistenceRecord* fields) override;
00040     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00041     virtual bool _check(std::string& errorMessage) override;
00042     virtual void _initBetweenReplications() override;
00043
00044 private:
00045     const struct DEFAULT_VALUES {
00046         int initialCount = 0;
00047         int count = 0;
00048         std::string compartment = "cytoplasm";
00049         std::string unit = "molecules";
00050     } DEFAULT;
00051
00052     int _initialCount = DEFAULT.initialCount;
00053     int _count = DEFAULT.count;
00054     std::string _compartment = DEFAULT.compartment;
00055     std::string _unit = DEFAULT.unit;
00056 };
00057
00058 #endif /* MOLECULARSPECIES_H */

```

11.453 StochasticReactionRule.h

```

00001 #ifndef STOCHASTICREACTIONRULE_H
00002 #define STOCHASTICREACTIONRULE_H
00003
00004 #include <string>
00005 #include <vector>
00006 #include <map>
00007
00008 #include "../kernel/simulator/model/ModelDataDefinition.h"
00009 #include "kernel/simulator/PluginInformation.h"
00010
00022 class StochasticReactionRule : public ModelDataDefinition {
00023 public:
00024     struct StoichiometricTerm {
00025         std::string speciesName;
00026         int stoichiometry = 1;
00027     };
00028
00029 public:
00030     StochasticReactionRule(Model* model, std::string name = "");
00031     virtual ~StochasticReactionRule() override = default;

```



```

00032
00033 public:
00034     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00035     static PluginInformation* GetPluginInformation();
00036     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00037
00038 public:
00039     void addReactant(std::string speciesName, int stoichiometry = 1);
00040     void addProduct(std::string speciesName, int stoichiometry = 1);
00041     void clearReactants();
00042     void clearProducts();
00043     const std::vector<StoichiometricTerm>& getReactants() const;
00044     const std::vector<StoichiometricTerm>& getProducts() const;
00045
00046     void setRateConstant(double rateConstant);
00047     double getRateConstant() const;
00048     void setCompartment(std::string compartment);
00049     std::string getCompartment() const;
00050
00051     double computePropensity(const std::map<std::string, int>& counts) const;
00052
00053     virtual std::string show() override;
00054
00055 protected:
00056     virtual bool _loadInstance(PersistenceRecord* fields) override;
00057     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00058     virtual bool _check(std::string& errorMessage) override;
00059
00060 private:
00061     static double _combinatorial(int n, int k);
00062
00063 private:
00064     const struct DEFAULT_VALUES {
00065         double rateConstant = 0.0;
00066         std::string compartment = "cytoplasm";
00067     } DEFAULT;
00068
00069     std::vector<StoichiometricTerm> _reactants;
00070     std::vector<StoichiometricTerm> _products;
00071     double _rateConstant = DEFAULT.rateConstant;
00072     std::string _compartment = DEFAULT.compartment;
00073 };
00074
00075 #endif /* STOCHASTICREACTIONRULE_H */

```

11.454 WholeCellState.h

```

00001 #ifndef WHOLECELLSTATE_H
00002 #define WHOLECELLSTATE_H
00003
00004 #include <map>
00005 #include <string>
00006 #include <vector>
00007
00008 #include "../kernel/simulator/model/ModelDataDefinition.h"
00009 #include "kernel/simulator/PluginInformation.h"
00010
00025 class WholeCellState : public ModelDataDefinition {
00026 public:
00027     WholeCellState(Model* model, std::string name = "");
00028     virtual ~WholeCellState() override = default;
00029
00030 public:
00031     static ModelDataDefinition* LoadInstance(Model* model, PersistenceRecord* fields);
00032     static PluginInformation* GetPluginInformation();
00033     static ModelDataDefinition* NewInstance(Model* model, std::string name = "");
00034
00035 public:
00036     // Molecule count access (integer, for SSA)
00037     void setMoleculeCount(const std::string& speciesName, int count);
00038     int getMoleculeCount(const std::string& speciesName) const;
00039     bool hasMoleculeCount(const std::string& speciesName) const;
00040     const std::map<std::string, int>& getMoleculeCounts() const;
00041
00042     // Metabolite pool access (continuous, for FBA output)
00043     void setMetaboliteAmount(const std::string& metaboliteName, double amount);
00044     double getMetaboliteAmount(const std::string& metaboliteName) const;
00045     const std::map<std::string, double>& getMetabolitePool() const;
00046
00047     // Compartment-specific metabolite pool access (continuous, for compartment-aware metabolism)
00048     void setCompartmentMetaboliteAmount(const std::string& compartmentName, const std::string&
metaboliteName, double amount);
00049     double getCompartmentMetaboliteAmount(const std::string& compartmentName, const std::string&
metaboliteName) const;

```

```

00050     bool hasCompartmentMetaboliteAmount(const std::string& compartmentName, const std::string&
metaboliteName) const;
00051     const std::map<std::string, double>& getCompartmentMetabolitePool() const;
00052
00053     // Pathway activity access (continuous, for FBA objective/flux summaries)
00054     void setPathwayActivity(const std::string& pathwayName, double activity);
00055     double getPathwayActivity(const std::string& pathwayName) const;
00056     bool hasPathwayActivity(const std::string& pathwayName) const;
00057     const std::map<std::string, double>& getPathwayActivities() const;
00058
00059     // Resource budget access (integer, set by fair allocation before SSA)
00060     void setResourceBudget(const std::string& speciesName, int budget);
00061     int getResourceBudget(const std::string& speciesName) const;
00062     void clearResourceBudget();
00063
00064     // Cell geometry
00065     void setCellVolume(double volume);
00066     double getCellVolume() const;
00067     void setCellMass(double mass);
00068     double getCellMass() const;
00069
00070     // Simulation progress
00071     void setCurrentTime(double time);
00072     double getCurrentTime() const;
00073     void setStepCount(int stepCount);
00074     int getStepCount() const;
00075     void incrementStep();
00076
00077     // Lifecycle metadata
00078     void setLifecyclePhase(const std::string& phase);
00079     std::string getLifecyclePhase() const;
00080     void setGenerationCount(int generationCount);
00081     int getGenerationCount() const;
00082     void setViable(bool viable);
00083     bool isViable() const;
00084     void setLastDivisionTime(double time);
00085     double getLastDivisionTime() const;
00086
00087     // JSON loading from CovertLab/WholeCell data files (MIT)
00088     bool loadFixedConstants(const std::string& jsonPath);
00089     bool loadParameters(const std::string& jsonPath);
00090
00091     // JSON file paths (persisted so the model file remembers them)
00092     void setFixedConstantsPath(std::string path);
00093     std::string getFixedConstantsPath() const;
00094     void setParametersPath(std::string path);
00095     std::string getParametersPath() const;
00096
00097     virtual std::string show() override;
00098
00099 protected:
00100     virtual bool _loadInstance(PersistenceRecord* fields) override;
00101     virtual void _saveInstance(PersistenceRecord* fields, bool saveDefaultValues) override;
00102     virtual bool _check(std::string& errorMessage) override;
00103     virtual void _initBetweenReplications() override;
00104
00105 private:
00106     const struct DEFAULT_VALUES {
00107         double cellVolume = 1.0e-15; // 1 fL, typical M. genitalium volume
00108         double cellMass = 1.0e-13; // ~100 fg, typical M. genitalium mass
00109         double currentTime = 0.0;
00110         int stepCount = 0;
00111         std::string lifecyclePhase = "newborn";
00112         int generationCount = 0;
00113         bool viable = true;
00114         double lastDivisionTime = -1.0;
00115         std::string fixedConstantsPath = "";
00116         std::string parametersPath = "";
00117     } DEFAULT;
00118
00119     std::map<std::string, int> _moleculeCounts;
00120     std::map<std::string, int> _initialMoleculeCounts;
00121     std::map<std::string, double> _metabolitePool;
00122     std::map<std::string, double> _initialMetabolitePool;
00123     std::map<std::string, double> _compartmentMetabolitePool;
00124     std::map<std::string, double> _initialCompartmentMetabolitePool;
00125     std::map<std::string, double> _pathwayActivities;
00126     std::map<std::string, double> _initialPathwayActivities;
00127     std::map<std::string, int> _resourceBudget;
00128
00129     double _cellVolume = DEFAULT.cellVolume;
00130     double _cellMass = DEFAULT.cellMass;
00131     double _initialCellVolume = DEFAULT.cellVolume;
00132     double _initialCellMass = DEFAULT.cellMass;
00133     double _currentTime = DEFAULT.currentTime;
00134     int _stepCount = DEFAULT.stepCount;
00135     std::string _lifecyclePhase = DEFAULT.lifecyclePhase;

```

```

00136         std::string _initialLifecyclePhase = DEFAULT.lifecyclePhase;
00137         int _generationCount = DEFAULT.generationCount;
00138         int _initialGenerationCount = DEFAULT.generationCount;
00139         bool _viable = DEFAULT.viable;
00140         bool _initialViable = DEFAULT.viable;
00141         double _lastDivisionTime = DEFAULT.lastDivisionTime;
00142         double _initialLastDivisionTime = DEFAULT.lastDivisionTime;
00143         std::string _fixedConstantsPath = DEFAULT.fixedConstantsPath;
00144         std::string _parametersPath = DEFAULT.parametersPath;
00145     };
00146
00147 #endif /* WHOLECELLSTATE_H */

```

11.455 PluginConnectorDummyImpl1.h

```

00001 /*
00002  * To change this license header, choose License Headers in Project Properties.
00003  * To change this template file, choose Tools | Templates
00004  * and open the template in the editor.
00005  */
00006
00007 /*
00008  * File:   PluginConnectorDummyImpl1.h
00009  * Author: rlcancian
00010  *
00011  * Created on 9 de Setembro de 2019, 19:24
00012  */
00013
00014 #ifndef PLUGINCONNECTORDUMMYIMPL1_H
00015 #define PLUGINCONNECTORDUMMYIMPL1_H
00016
00017 #include "kernel/simulator/PluginConnector_if.h"
00018 #include "kernel/util/List.h"
00019 //namespace GenesysKernel {
00020
00027 class PluginConnectorDummyImpl1 : public PluginConnector_if {
00028 public:
00029     PluginConnectorDummyImpl1();
00030     virtual ~PluginConnectorDummyImpl1() = default;
00031 public:
00037     Plugin* check(const std::string dynamicLibraryFilename) override;
00043     Plugin* connect(const std::string dynamicLibraryFilename) override;
00048     List<std::string>* find() override;
00054     bool disconnect(const std::string dynamicLibraryFilename) override;
00060     bool disconnect(Plugin* plugin) override;
00061 private:
00062     void _connect(List<Plugin*>* plugins, Plugin* plugin);
00063 };
00064 //namespace\\}
00065 #endif /* PLUGINCONNECTORDUMMYIMPL1_H */

```

11.456 PluginConnectorStaticImpl1.h

```

00001 #ifndef PLUGINCONNECTORSTATICIMPL1_H
00002 #define PLUGINCONNECTORSTATICIMPL1_H
00003
00004 #include "kernel/simulator/PluginConnector_if.h"
00005
00012 class PluginConnectorStaticImpl1 : public PluginConnector_if {
00013 public:
00014     PluginConnectorStaticImpl1() = default;
00015     virtual ~PluginConnectorStaticImpl1() = default;
00016
00022     Plugin* check(const std::string dynamicLibraryFilename) override;
00028     Plugin* connect(const std::string dynamicLibraryFilename) override;
00033     List<std::string>* find() override;
00039     bool disconnect(const std::string dynamicLibraryFilename) override;
00045     bool disconnect(Plugin* plugin) override;
00046 };
00047
00048 #endif

```

